

ADSP-21560/21561/21564/21568 SHARC+ Processor

Hardware Reference

Revision 0.3, May 2024

Part Number
82-100154-01



Notices

Copyright Information

© 2024 Analog Devices, Inc., ALL RIGHTS RESERVED. This document may not be reproduced in any form without prior, express written consent from Analog Devices, Inc.

Printed in the USA.

Disclaimer

Analog Devices, Inc. reserves the right to change this product without prior notice. Information furnished by Analog Devices is believed to be accurate and reliable. However, no responsibility is assumed by Analog Devices for its use; nor for any infringement of patents or other rights of third parties which may result from its use. No license is granted by implication or otherwise under the patent rights of Analog Devices, Inc.

Trademark and Service Mark Notice

The Analog Devices logo, Blackfin, Blackfin+, CrossCore, EngineerZone, EZ-Board, EZ-KIT Lite, EZ-KIT Mini, EZ-Extender, SHARC, SHARC+, Blackfin+, A²B, E²B, EdgeStudio, SigmaStudio, SigmaStudio+, and VisualDSP+ are registered trademarks of Analog Devices, Inc.

All other brand and product names are trademarks or service marks of their respective owners.

Contents

Introduction

Power and Clock Management (DPM/RCU/CGU/CDU)	1-1
System Interrupts and Triggers (SEC/TRU)	1-1
System Memory (L2CTL/OTPC/SMPU)	1-2
Direct Memory Access (DMA/EMDMA/CRC)	1-3
Peripherals	1-4
General-Purpose I/O (GPIO) Peripherals	1-4
DAI/SRU Peripherals	1-5
System Accelerators ()	1-5
Security and Protection (SPU/PKTE/PKIC/PKA/TRNG)	1-6
System Debug (SCB/DBG/TAPC)	1-7

Clock Generation Unit (CGU)

CGU Features	2-1
CGU Functional Description.....	2-1
ADSP-21560_21561_21564_21568 CGU Register List	2-2
ADSP-21560_21561_21564_21568 CGU Interrupt List	2-2
ADSP-21560_21561_21564_21568 CGU Trigger List	2-3
CGU Definitions.....	2-3
CGU PLL Block Diagram	2-4
CLKOUT Selections	2-4
CGU Operating Modes	2-5
CGU Power-up Sequence	2-5
CGU Event Control	2-6
CGU Programming Model	2-6
Configuring CGU Modes	2-7
Changing the Clock Frequencies	2-7

Changing the PLL Clock Frequency	2-7
Changing the CCLKn or SYSCLK Frequency Without Modifying the PLLCLK Frequency	2-8
Changing the OCLK Frequency	2-9
Selecting SCLK1_0 Source	2-9
Changing SCLK1_0 Frequencies	2-9
Aligning All Clocks	2-10
Shutting Off CCLKn from Another Controller	2-10
Valid Clock Multiplier Settings	2-11
PLL Bypass and PLL Disable	2-11
Processor Specific Information	2-12
ADSP-21560_21561_21564_21568 CGU Register Descriptions	2-12
Core Clock Buffer Disable Register	2-13
Core Clock Buffer Status Register	2-14
CLKOUT Select Register	2-15
Control Register	2-17
Clocks Divisor Register	2-19
DIV Register Extension	2-22
PLL Control Register	2-23
Revision ID Register	2-25
System Clock Buffer Disable Register	2-26
System Clock Buffer Status Register	2-28
Status Register	2-30
Clock Distribution Unit (CDU)	
CDU Features	3-1
CDU Functional Description	3-1
CDU Block Diagram	3-1
CDU Definitions	3-2
ADSP-21560_21561_21564_21568 CDU Register List	3-2
CDU Clock Configuration Options	3-3
CDU Programming Model	3-3

Changing the PLL and Clock Frequency	3-3
Changing the Clock Frequency	3-4
ADSP-21560_21561_21564_21568 CDU Register Descriptions	3-5
CDU Configuration	3-6
CLKIN Select	3-7
CDU Revision ID	3-8
CDU Status	3-9
 Dynamic Power Management (DPM)	
DPM Features.....	4-1
DPM Functional Description	4-1
ADSP-21560_21561_21564_21568 DPM Register List.....	4-1
DPM Definitions	4-2
DPM Operating Modes.....	4-2
Reset State	4-2
Full-on Mode.....	4-2
DPM Event Control	4-3
DPM Programming Model.....	4-3
ADSP-21560_21561_21564_21568 DPM Register Descriptions	4-4
Control Register	4-6
Peripherals Disable Register 0	4-7
Peripherals Disable Register 1	4-9
Revision ID	4-11
Status Register	4-12
 Reset Control Unit (RCU)	
RCU Features	5-1
RCU Functional Description	5-1
ADSP-21560_21561_21564_21568 RCU Register List.....	5-2
ADSP-21560_21561_21564_21568 RCU Trigger List.....	5-2
RCU Definitions	5-3

RCU Architectural Concepts	5-3
RCU Status and Error Signals.....	5-4
ADSP-21560_21561_21564_21568 RCU Register Descriptions	5-4
Boot Code Register	5-6
Core Reset Outputs Control Register	5-9
Core Reset Outputs Status Register	5-10
Control Register	5-11
Message Register	5-13
Message Clear Bits Register	5-16
Message Set Bits Register	5-17
System Reset Request Status Register	5-18
Status Register	5-19
Software Vector Register 0	5-21
SVECT Lock Register	5-22
 System Event Controller (SEC)	
SEC Features.....	6-1
SEC Functional Description	6-2
ADSP-21560_21561_21564_21568 SEC Register List.....	6-2
ADSP-21560_21561_21564_21568 SEC Interrupt List	6-3
ADSP-21560_21561_21564_21568 SEC Trigger List.....	6-3
Combined Interrupts.....	0
SEC Definitions	6-3
SEC Block Diagram.....	6-4
SEC Fault Interface (SFI)	6-5
SEC Core Interface (SCI)	6-6
SEC Source Interface (SSI)	6-7
SEC Architectural Concepts	6-7
System Interrupt Acknowledge.....	6-7
System Interrupt Groups.....	6-8
System Interrupt Flow.....	6-8

System Interrupt Priorities	6–9
SEC Error.....	6–9
SEC Programming Model.....	6–9
Programming Concepts	6–10
Programming Examples.....	6–10
Fault Management Interface Programming Model	6–10
Configuring a System Source to Interrupt a Core.....	6–11
Core/SEC Handshake Requirements to Ensure Proper Interrupt Handling	6–12
Configuring a System Source as a Fault	6–13
SEC Programming Restrictions	6–13
ADSP-21560_21561_21564_21568 SEC Register Descriptions	6–13
SCI Active Register n	6–15
SCI Control Register n	6–16
SCI Group Mask Register n	6–18
SCI Priority Level Register n	6–20
SCI Priority Mask Register n	6–22
Core Pending Register n	6–23
SCI Source ID Register n	6–24
SCI Status Register n	6–25
Global End Register	6–27
Fault COP Period Register	6–28
Fault COP Period Current Register	6–29
Fault Control Register	6–30
Fault Delay Register	6–33
Fault Delay Current Register	6–34
Fault End Register	6–35
Fault Source ID Register	6–36
Fault System Reset Delay Register	6–37
Fault System Reset Delay Current Register	6–38
Fault Status Register	6–39

Global Control Register	6-41
Global Status Register	6-42
Global Raise Register	6-44
Source Control Register n	6-45
Source Status Register n	6-48

Trigger Routing Unit (TRU)

TRU Features	7-1
TRU Functional Description	7-1
ADSP-21560_21561_21564_21568 TRU Register List.....	7-2
ADSP-21560_21561_21564_21568 TRU Interrupt List	7-2
ADSP-21560_21561_21564_21568 Trigger List.....	7-2
TRU Definitions	7-8
TRU Block Diagram	7-8
TRU Architectural Concepts	7-8
TRU Programming Model.....	7-9
Programming Concepts.....	7-9
Programming Examples.....	7-9
Configuring a Simple Trigger Sequence.....	7-10
TRU Event Control	7-10
TRU Status and Error Signals.....	7-10
ADSP-21560_21561_21564_21568 TRU Register Descriptions	7-10
Error Address Register	7-11
Global Control Register	7-12
Generator Trigger Register	7-14
Receiver Select Register	7-15
Status Information Register	7-16

L2 System Memory

L2 System Memory Features	8-1
L2 System Memory Functional Description.....	8-1

ADSP-21560_21561_21564_21568 L2CTL Register List.....	8-1
ADSP-21560_21561_21564_21568 L2CTL Interrupt List	8-2
ADSP-21560_21561_21564_21568 L2CTL Trigger List	8-3
L2 System Memory Block Diagram.....	8-3
L2 System Memory Architectural Concepts.....	8-3
Access Characteristics	8-4
Read/Write Latency and Throughput	8-4
L2 Memory Controller Block Diagram (Instance)	8-4
Arbitration and Priority.....	8-5
Data Integrity.....	8-6
Parity	8-6
Parity Hardware Control.....	8-6
Parity Error Management.....	8-6
Power Modes.....	8-7
L2 System Memory Event Control.....	8-7
Parity Error Interrupt	8-7
ADSP-21560_21561_21564_21568 L2CTL Register Descriptions	8-8
Control Register	8-9
Error Type 0 Address Register	8-12
Error Type 1 Address Register	8-13
Error Type 0 Register	8-14
Error Type 1 Register	8-16
Initialization Register	8-17
Initialization Status Register	8-19
Power Control Register	8-21
Parity Error Address 0 Register	8-23
Parity Error Address 1 Register	8-24
Parity Error Address 2 Register	8-25
Parity Error Address 3 Register	8-26
Parity Error Address 4 Register	8-27

Parity Error Address 5 Register	8–28
Parity Error Address 6 Register	8–29
Parity Error Address 7 Register	8–30
Parity Error Address 8 Register	8–31
Revision ID Register	8–32
Read Priority Count Register	8–33
Status Register	8–34
Write Priority Count Register	8–37

One-Time Programmable Memory Controller (OTPC)

OTPC Features.....	9–1
OTPC Functional Description.....	9–1
ADSP-21560_21561_21564_21568 OTPC Register List	9–1
ADSP-21560_21561_21564_21568 OTPC Interrupt List	9–2
Error Correction	9–2
OTP Layout.....	9–2
OTPC Event Control.....	9–7
OTPC Interrupt Signals	9–7
OTPC Status and Error Signals	9–7
OTP API Overview	9–7
OTP Programming.....	9–7
OTP Reading	9–7
OTP Get Field	9–8
ADSP-21560_21561_21564_21568 OTPC Register Descriptions	9–8
OTP Security State Register	9–9
OTP Status Register	9–10

System Memory Protection Unit (SMPU)

SMPU Features.....	10–1
SMPU Functional Description	10–2
ADSP-21560_21561_21564_21568 SMPU Register List.....	10–3

SMPU Interrupts	10-3
Memory Writes.....	10-4
Memory Reads	10-4
ID Comparison	10-4
Memory Region.....	10-7
SMPU Definitions	10-8
SMPU Block Diagram.....	10-8
SMPU Architectural Concepts	10-9
SMPU Operating Modes	10-9
SMPU Interrupt Signals	10-10
SMPU Status and Error Signals	10-10
ADSP-21560_21561_21564_21568 SMPU Register Descriptions	10-11
Bus Error Address Register	10-12
Bus Error Details Register	10-13
SMPU Control Register	10-14
Interrupt Address Register	10-16
Interrupt Details Register	10-17
Region n Address Register	10-18
Region n Control Register	10-19
SMPU Revision ID Register	10-22
Region n ID A Register	10-23
Region n ID B Register	10-24
Region n ID Mask A Register	10-25
Region n ID Mask B Register	10-26
SMPU Control Secure Accesses Register	10-27
Region n Control Secure Accesses Register	10-29
SMPU Status Register	10-30

General-Purpose Ports (PORT)

PORT Features	11-2
---------------------	------

PORT Functional Description.....	11-2
ADSP-21560_21561_21564_21568 PORT Register List	11-3
ADSP-21560_21561_21564_21568 PORT Trigger List.....	11-3
ADSP-21560_21561_21564_21568 PINT Register List	11-4
ADSP-21560_21561_21564_21568 PINT Interrupt List	11-4
ADSP-21560_21561_21564_21568 PINT Trigger List.....	11-5
ADSP-21560_21561_21564_21568 PADS Register List	11-5
PORT Architectural Concepts.....	11-5
Internal Interfaces	11-6
External Interfaces.....	11-6
GPIO Pin Function.....	11-6
Input Mode.....	11-6
Output Mode	11-6
Trigger Toggle Mode.....	11-7
Open-Drain Mode	11-7
Port Multiplexing Control.....	11-7
PORT Event Control.....	11-7
PORT Interrupt Signals	11-8
PORT Programming Model	11-9
Programmable Pull-up Resistors for PORT and DAI	11-13
ADSP-21560_21561_21564_21568 PORT Register Descriptions	11-14
Port x GPIO Data Register	11-16
Port x GPIO Data Clear Register	11-20
Port x GPIO Data Set Register	11-24
Port x GPIO Output Toggle Register	11-27
Port x GPIO Direction Register	11-30
Port x GPIO Direction Clear Register	11-34
Port x GPIO Direction Set Register	11-38
Port x Function Enable Register	11-41
Port x Function Enable Clear Register	11-44

Port x Function Enable Set Register	11-47
Port x GPIO Input Enable Register	11-50
Port x GPIO Input Enable Clear Register	11-53
Port x GPIO Input Enable Set Register	11-56
Port x GPIO Lock Register	11-59
Port x Multiplexer Control Register	11-61
Port x GPIO Polarity Invert Register	11-63
Port x GPIO Polarity Invert Clear Register	11-67
Port x GPIO Polarity Invert Set Register	11-70
Port x GPIO Trigger Toggle Register	11-73
ADSP-21560_21561_21564_21568 PINT Register Descriptions	11-74
PINT Assign Register	11-76
PINT Edge Clear Register	11-78
PINT Edge Set Register	11-81
PINT Invert Clear Register	11-84
PINT Invert Set Register	11-87
PINT Latch Register	11-90
PINT Mask Clear Register	11-94
PINT Mask Set Register	11-97
PINT Pin State Register	11-100
PINT Request Register	11-104
ADSP-21560_21561_21564_21568 PADS Register Descriptions	11-108
DAI0 1 to 10 pins DS Control Register	11-109
DAI0 11 to 20 pins DS Control Register	11-112
DAI0 Port Input Enable Control Register	11-113
DAI1 1 to 10 pins DS Control Register	11-114
DAI1 11 to 20 pins DS Control Register	11-117
DAI1 Port Input Enable Control Register	11-118
DAIx Pull-Down Enable Register	11-119
DAIx Pull-Up Enable Register	11-120

Non-GPIO Drive Strength Register	11-121
Peripheral PAD Configuration 0 Register	11-123
PORTA 0 to 7 pins DS Control Register	11-125
PORTA 8 - 15 pins DS Control Register	11-128
PORTB 0 to 7 pins DS Control Register	11-131
PORTx Pull-Down Enable Register	11-133
PORTx Pull-Up Enable Register	11-134

Watchdog Timer (WDOG)

WDOG Features.....	12-1
WDOG Functional Description	12-2
ADSP-21560_21561_21564_21568 WDOG Register List.....	12-2
ADSP-21560_21561_21564_21568 WDOG Interrupt List	12-3
ADSP-21560_21561_21564_21568 WDOG Trigger List	12-3
WDOG Block Diagram.....	12-3
Internal Interface	12-4
External Interface	12-4
ADSP-21560_21561_21564_21568 WDOG Register Descriptions	12-4
Count Register	12-5
Control Register	12-6
Watchdog Timer Status Register	12-7
Watchdog Timer Window Register	12-9

Serial Peripheral Interface (SPI)

SPI Features.....	13-1
SPI Functional Description.....	13-2
ADSP-21560_21561_21564_21568 SPI Register List.....	13-2
ADSP-21560_21561_21564_21568 SPI Interrupt List	13-3
ADSP-21560_21561_21564_21568 SPI Trigger List	13-4
ADSP-21560_21561_21564_21568 SPI DMA Channel List	13-5
SPI Block Diagram.....	13-5

Transfer Protocol	13–6
Clock Considerations	13–8
Controlling Delay Between Frames	13–8
Flow Control	13–10
Target Select Operation	13–11
Beginning and Ending a Non-DMA SPI Transfer	13–12
Transmit Operation in Non-DMA Mode	13–13
Receive Operation in Non-DMA Mode	13–13
Dual I/O Mode	13–13
Quad I/O Mode (SPI1/SPI2)	13–14
Fast Mode	13–15
Memory-Mapped Mode (SPI1/SPI2)	13–16
Memory-Mapped Description of Operation.....	13–17
Memory-Mapped Architectural Concepts.....	13–18
Memory-Mapped Read Accesses.....	13–20
Memory-Mapped High-Performance Features.....	13–23
Merged Read Accesses	13–23
Wrap Around Accesses	13–23
Execute-In-Place (XIP, SPI2 only)	13–24
Memory-Mapped Mode Error Status Bits	13–25
Memory-Mapped Programming Guidelines	13–26
SPI Interrupt Signals	13–28
Data Interrupts.....	13–28
Status Interrupts.....	13–29
Error Conditions	13–29
SPI Programming Concepts.....	13–30
Controller Operation in Non-DMA Modes.....	13–31
Target Operation in Non-DMA Modes	13–31
Configuring DMA Controller Mode	13–32
Configuring DMA Target Mode Operation.....	13–33

ADSP-21560_21561_21564_21568 SPI Register Descriptions	13–34
Clock Rate Register	13–36
Control Register	13–37
Delay Register	13–43
Masked Interrupt Condition Register	13–44
Masked Interrupt Clear Register	13–46
Interrupt Mask Register	13–49
Interrupt Mask Clear Register	13–51
Interrupt Mask Set Register	13–54
Memory Mapped Read Header	13–57
SPI Memory Top Address	13–61
Receive FIFO Data Register	13–62
Received Word Count Register	13–63
Received Word Count Reload Register	13–64
Receive Control Register	13–65
Slave Select Register	13–68
Status Register	13–71
Transmit FIFO Data Register	13–76
Transmitted Word Count Register	13–77
Transmitted Word Count Reload Register	13–78
Transmit Control Register	13–79
 Extended Serial Peripheral Interface (xSPI)	
xSPI Features	14–1
Supported Memory Devices.....	14–2
xSPI Functional Description.....	14–2
ADSP-21560_21561_21564_21568 XSPI Register List	14–2
xSPI Block Diagram	14–5
Architectural Concepts	14–6
Operating Modes	14–7

Direct Mode.....	14-7
STIG Mode.....	14-8
ACMD Mode.....	14-9
Integrated Soft PHY	14-10
XIP Mode	14-10
Configuring the Controller.....	14-11
Operating in Direct Mode	14-12
Write Limitations	14-13
AXI Wrapping Bursts	14-13
Entering and Exiting XIP Mode	14-14
Operating in ACMD Mode	14-17
Using the Auto Command in CDMA Mode	14-18
Using the Auto Command in PIO Mode.....	14-24
Status Checking.....	14-27
Selecting the DMA Interface	14-29
Rules and Limitations	14-30
Operating in STIG Mode.....	14-30
Instruction Variants.....	14-31
Glued Data Instruction	14-40
Instruction Type Variants	14-44
Switching Work Modes	14-47
Programming STIG Mode	14-47
STIG Mode Status	14-48
STIG Limitations.....	14-49
Sequencing STIG Operation	14-50
Event Handling	14-51
Device Discovery	14-53
Discovery Modes	14-53
Discovery Flow.....	14-54
SFDP Command Structure	14-55
SPI NAND Workflow.....	14-62

Discovery Status	14-63
Discovery Restrictions	14-65
Discovery Limitations	14-66
Legacy Devices	14-66
Device Specific Configuration	14-66
Controller Reconfiguration	14-67
Common Sequence Register Updates	14-68
Reset Sequence Parameters for Profile 1 Devices	14-70
Erase Sequence Register Updates.....	14-71
Write Enable Sequence Register Updates.....	14-71
Program Sequence Register Updates	14-71
Read Sequence Register Updates	14-73
Status Sequence Register Updates	14-77
Boot Engine.....	14-82
Configuring the Boot Engine	14-83
xSPI PHY Controller.....	14-85
PHY Initialization	14-85
Initial PHY Registers to Configure.....	14-86
XSPI_PHY_DLL_CTL Register	14-86
XSPI_PHY_DQ_TR Register.....	14-87
XSPI_PHY_DQS_TR Register.....	14-87
XSPI_PHY_GATE_LPBK_CTL Register	14-87
XSPI_PHY_DLL_MSTR_CTL Register	14-88
XSPI_PHY_DLL_SLAVE_CTL Register	14-88
PHY Re-initialization	14-91
Resynchronization of Target Delay Lines	14-91
PHY Read Sampling Method Selection	14-92
DQS Mode for PHY Read Sampling.....	14-92
Non-DQS Mode (Loopback) for PHY Read Sampling.....	14-92
Memory Device Data Integrity	14-93
ECC Data Integrity	14-93
CRC Parity Check.....	14-93

CRC Using Direct or ACMD Mode.....	14-93
CRC Using STIG Mode.....	14-94
Miscellaneous Low-level Configurations.....	14-94
Write Protection.....	14-94
Device Reset.....	14-94
SPI Clock CPOL and CPHA Configuration	14-94
JEDEC Reset Delay Settings	14-94
Device Delay Settings.....	14-94
Reset Recovery Delay Settings.....	14-95
t _{CMS} Timing Pause Refresh Settings	14-95
Legacy HyperFlash or Profile 2 Address Settings	14-95
Multiple Device Connections to Controller.....	14-95
SPI NAND Device Operation Settings	14-95
ADSP-21560_21561_21564_21568 XSPI Register Descriptions	14-96
Boot Status Register	14-99
Command Register 0	14-100
Command Register 1	14-101
Command Register 2	14-102
Command Register 3	14-103
Command Register 4	14-104
Command Register 5	14-105
Command Status Register	14-106
Command Status Pointer Register	14-107
DAC Configuration Register	14-108
Address Remapping Register 0	14-110
Address Remapping Register 1	14-111
Device Discovery Control Register	14-112
DMA Interface Control Register	14-116
DMA Error Address High Register	14-118
DMA Error Address Low Register	14-119

Erase Sequence Configuration Register 0	14–120
Erase Sequence Configuration Register 1	14–122
Erase Sequence Configuration Register 2	14–123
Controller Features Register	14–124
General Controller Status Register	14–126
Interrupt Enable Register	14–128
Interrupt Status Register	14–131
Long Polling Count Register	14–134
Device Active Maximum Clock Cycle Register	14–135
Clock Mode Control Register	14–136
Device Delay Register	14–137
HyperFlash Offset Register	14–138
JEDEC Reset Delay Register	14–139
Hardware Reset Control Register	14–140
Reset Recovery Delay Register	14–143
Write Protect Register	14–144
PHY DLL Observable Points 0 Register	14–145
PHY DLL Observable Points 1 Register	14–147
PHY DLL Control Register	14–148
Master DLL Control Register	14–150
Slave DLL Control Register	14–152
PHY DLL Resynchronization Register	14–153
PHY DQS Timing Register	14–154
PHY DQ Timing Register	14–156
PHY Features Register	14–157
PHY Gate Loopback Control Register	14–160
PHY Global Control Register	14–162
PHY GPIO Control Register 0	14–163
PHY GPIO Control Register 1	14–164
PHY GPIO Status Register 0	14–165

PHY GPIO Status Register 1	14–166
PHY Global Termination Control Register	14–167
PHY DQS Input Enable Timing Register	14–168
PHY Observable Points Register	14–169
PHY Revision ID Register	14–171
PHY Static Aging Register	14–172
PHY Deskew Write Register	14–173
Program Sequence Configuration Register 0	14–175
Program Sequence Configuration Register 1	14–177
Program Sequence Configuration Register 2	14–178
Read Sequence Configuration Register 0	14–180
Read Sequence Configuration Register 1	14–182
Read Sequence Configuration Register 2	14–184
Revision ID Register	14–186
Reset Sequence Configuration Register 0	14–187
Reset Sequence Configuration Register 1	14–189
Host DMA Buffer Address Register 0	14–190
Host DMA Buffer Address Register 1	14–191
Host DMA Block Size Register	14–192
Host DMA Thread Status Register	14–193
Sequence Configuration Register 0	14–194
Sequence Configuration Register 1	14–198
Short Polling Count Register	14–200
Status Checking Sequence Configuration Register 0	14–201
Status Checking Sequence Configuration Register 1	14–203
Status Checking Sequence Configuration Register 10	14–205
Status Checking Sequence Configuration Register 2	14–207
Status Checking Sequence Configuration Register 3	14–208
Status Checking Sequence Configuration Register 4	14–209
Status Checking Sequence Configuration Register 5	14–210

Status Checking Sequence Configuration Register 7	14–212
Status Checking Sequence Configuration Register 8	14–213
Status Checking Sequence Configuration Register 9	14–214
Auto Command Engine Interrupt Status Thread Register	14–215
Thread Error Interrupt Enable Register	14–216
Thread Error Interrupt Status Register	14–217
Auto Command Engine Thread Status Register	14–219
WEL Sequence Configuration Register	14–220
Device Control Register	14–222
XIP Configuration Register	14–223

Universal Asynchronous Receiver/Transmitter (UART)

UART Features	15–1
UART Functional Description	15–2
ADSP-21560_21561_21564_21568 UART Register List	15–3
ADSP-21560_21561_21564_21568 UART Interrupt List	15–3
ADSP-21560_21561_21564_21568 UART Trigger List.....	15–4
ADSP-21560_21561_21564_21568 UART DMA Channel List.....	15–4
UART Block Diagram	15–4
UART Architectural Concepts	15–5
Internal Interface.....	15–5
External Interface	15–5
Hardware Flow Control.....	15–6
Bit Rate Generation	15–6
Autobaud Detection	15–7
UART Debug Features	15–8
UART Operating Modes.....	15–9
UART Mode.....	15–9
IrDA SIR Mode.....	15–10
Multi-Drop Bus Mode.....	15–10
UART Data Transfer Modes.....	15–11

UART Mode Transmit Operation (Core)	15-11
UART Mode LIN Break Command	15-12
UART Mode Receive Operation (Core).....	15-13
IrDA Transmit Operation	15-14
IrDA Receive Operation	15-14
MDB Transmit Operation.....	15-15
MDB Receive Operation	15-15
DMA Mode.....	15-16
Mixing DMA and Core Modes.....	15-17
Setting Up Hardware Flow Control.....	15-17
UART Event Control.....	15-17
Interrupt Masks.....	15-18
Interrupt Servicing	15-18
Transmit Interrupts	15-18
Receive Interrupts.....	15-20
Status Interrupts.....	15-21
Multi-Drop Bus Events.....	15-22
UART Programming Model	15-22
Detecting Autobaud	15-22
Using Common Initialization Steps.....	15-23
Using Core Transfers	15-23
Using DMA Transfers.....	15-23
Using Interrupts	15-23
Setting Up Hardware Flow Control.....	15-23
ADSP-21560_21561_21564_21568 UART Register Descriptions	15-23
Clock Rate Register	15-25
Control Register	15-26
Interrupt Mask Register	15-32
Interrupt Mask Clear Register	15-36
Interrupt Mask Set Register	15-38

Receive Buffer Register	15–40
Receive Shift Register	15–41
Receive Counter Register	15–42
Scratch Register	15–43
Status Register	15–44
Transmit Address/Insert Pulse Register	15–49
Transmit Hold Register	15–50
Transmit Shift Register	15–51
Transmit Counter Register	15–52

General-Purpose Timer (TIMER)

GP Timer Features.....	16–1
ADSP-21560_21561_21564_21568 TIMER Register List	16–2
ADSP-21560_21561_21564_21568 TIMER Interrupt List	16–2
ADSP-21560_21561_21564_21568 TIMER Trigger List.....	16–3
Timer Block Diagram.....	16–4
Internal Interface	16–5
Internal Timer Connections	16–5
External Interface	16–6
GP Timer Operating Modes	16–6
General Operation.....	16–6
Period, Width and Delay Register Interaction	16–7
Single-Pulse PWMOUT Mode	16–8
Continuous PWMOUT Mode.....	16–8
Width Capture (WIDCAP) Mode.....	16–10
Width Capture Mode Overflow.....	16–13
Windowed Watchdog (WATCHDOG) Modes	16–15
Windowed Watchdog Width Mode	16–15
Windowed Watchdog Period Mode.....	16–17
Pin Interrupt (PININT) Mode.....	16–19
External Clock (EXTCLK) Mode	16–19

GP Timer Programming Concepts	16–20
Setting Up Constantly Changing Timer Conditions	16–20
Configuring, Enabling, and Disabling One or More Timers.....	16–21
Configuring Timer Data and Status Interrupts	16–21
Configuring the Timer as a Trigger Receiver	16–21
Ordered Trigger Toggle Mode	16–22
Using the Timer Broadcast Feature.....	16–22
Timer Illegal States	16–23
Continuous PWMOUT Mode	16–23
Single Pulse PWMOUT Mode.....	16–25
WIDCAP Mode	16–25
EXTCLK Mode	16–25
WATCHDOG Events.....	16–26
ADSP-21560_21561_21564_21568 TIMER Register Descriptions	16–27
Broadcast Delay Register	16–29
Broadcast Period Register	16–30
Broadcast Width Register	16–31
Data Interrupt Latch Register	16–32
Data Interrupt Mask Register	16–33
Error Type Status Register	16–34
Run Register	16–37
Run Clear Register	16–38
Run Set Register	16–39
Status Interrupt Latch Register	16–40
Status Interrupt Mask Register	16–41
Stop Configuration Register	16–42
Stop Configuration Clear Register	16–43
Stop Configuration Set Register	16–44
Timer n Configuration Register	16–45
Timer n Counter Register	16–50

Timer n Delay Register	16-51
Timer n Period Register	16-52
Timer n Width Register	16-53
Trigger Receiver Enable Register	16-54
Trigger Generator Mask Register	16-55

General-Purpose Counter (CNT)

GP Counter Features	17-1
GP Counter Functional Description	17-1
ADSP-21560_21561_21564_21568 CNT Register List	17-2
ADSP-21560_21561_21564_21568 CNT Interrupt List	17-2
ADSP-21560_21561_21564_21568 CNT Trigger List.....	17-3
GP Counter Operating Modes.....	17-3
Quadrature Encoder Mode	17-3
Binary Encoder Mode.....	17-4
Up/Down Counter Mode	17-4
Direction Counter Mode	17-5
Timed Direction Mode.....	17-5
GP Counter Programming Model	17-5
GP Counter General Programming Flow	17-5
GP Counter Mode Configuration.....	17-5
Configuring GP Counter Push-Button Operation	17-6
Configuring Zero-Marker-Zeros-Counter Mode	17-6
Configuring Zero-Marker-Error Mode	17-6
Configuring Zero-Once Mode.....	17-6
Configuring Boundary Auto-Extend Mode	17-7
Configuring Boundary Capture Mode.....	17-7
Configuring Boundary Compare and Boundary Zero Modes	17-7
Configuring GP Counter Push-Button Operation	17-8
GP Counter Programming Concepts.....	17-8
CNT Input Noise Filtering	17-8

Capturing Counter Interval and CNT_CNTR Read Timing	17-9
Capturing Time Interval Between Successive Counter Events	17-10
GP Counter Event Control	17-11
Illegal Gray and Binary Code Events	17-11
Up/Down Count Events	17-11
Zero-Count Events	17-12
Overflow Events	17-12
Boundary Match Events	17-12
Zero Marker Events	17-12
ADSP-21560_21561_21564_21568 CNT Register Descriptions	17-12
Configuration Register	17-14
Command Register	17-17
Counter Register	17-20
Debounce Register	17-21
Interrupt Mask Register	17-23
Maximum Count Register	17-26
Minimum Count Register	17-27
Status Register	17-28
 Media Local Bus (MLB)	
Features.....	18-1
MLB Definitions	18-2
Clocking	18-3
Functional Description	18-3
ADSP-21560_21561_21564_21568 MLB Register List	18-3
ADSP-21560_21561_21564_21568 MLB Interrupt List	18-4
MediaLB Protocol	18-5
MLB Architectural Concepts	18-6
MediaLB Block Diagram	18-6
MediaLB Interface	18-6

Routing Fabric.....	18-7
Data Buffer RAM.....	18-7
Channel Table RAM	18-7
Address Mapping	18-7
Channel Allocation Table.....	18-8
Channel Set Up.....	18-10
Channel Descriptor Tables.....	18-10
AHB Descriptor Table (ADT).....	18-15
Interrupt Interface Block.....	18-19
Operating Modes.....	18-19
Isochronous Data Exchange.....	18-19
Asynchronous and Control Data Exchange.....	18-21
Synchronous Data Exchange.....	18-21
Data Transfer.....	18-22
DMA.....	18-22
Programming Model.....	18-22
Channel Initialization.....	18-22
Configure the Hardware.....	18-22
Program the CAT and the CDT.....	18-22
Program the ADT	18-24
Service	18-25
Servicing the DMA Channel Interrupts.....	18-25
Servicing the MediaLB Status Interrupts.....	18-25
Polling for MediaLB System Commands.....	18-26
ADSP-21560_21561_21564_21568 MLB Register Descriptions	18-26
Peripheral Channel Mask 0 Register	18-28
Peripheral Channel Mask 1 Register	18-29
Peripheral Channel Status 0 Register	18-30
Peripheral Channel Status 1 Register	18-31
Bus Control Register	18-32
MediaLB Control 0 Register	18-34

Control 1 Register	18-36
MLB Global Control Register	18-37
HBi Channel Busy 0 Register	18-38
HBi Channel Busy 1 Register	18-39
HBi Channel Error 0 Register	18-40
HBi Channel Error 1 Register	18-41
HBi Channel Mask 0 Register	18-42
HBi Channel Mask 1 Register	18-43
HBi Control Register	18-44
Memory Interface Address Register	18-45
Memory Interface Control Register	18-46
Memory Interface Control Data 0 Register	18-47
Memory Interface Control Data 1 Register	18-48
Memory Interface Control Data 2 Register	18-49
Memory Interface Control Data 3 Register	18-50
Memory Interface Control Data Write Enable 0 Register	18-51
Memory Interface Control Data Write Enable 1 Register	18-52
Memory Interface Control Data Write Enable 2 Register	18-53
Memory Interface Control Data Write Enable 3 Register	18-54
Interrupt Enable Register	18-55
Channel Status 0 Register	18-58
Channel Status 1 Register	18-59
System Data Register	18-60
System Status Register	18-61

Two-Wire Interface (TWI)

TWI Features.....	19-1
TWI Functional Description	19-2
ADSP-21560_21561_21564_21568 TWI Register List.....	19-2
ADSP-21560_21561_21564_21568 TWI Interrupt List	19-3

TWI Block Diagram.....	19-3
External Interface	19-3
Serial Clock Signal (SCL)	19-4
Serial Data Signal (SDA).....	19-4
Internal Interface.....	19-5
TWI Architectural Concepts	19-5
TWI Protocol.....	19-5
Clock Generation and Synchronization	19-6
Bus Arbitration	19-6
Start and Stop Conditions	19-7
General Call Support.....	19-7
Fast Mode	19-8
TWI Operating Modes	19-8
Repeated Start	19-8
Transmit Receive Repeated Start.....	19-8
Receive Transmit Repeated Start.....	19-9
Clock Stretching	19-10
Clock Stretching During FIFO Underflow	19-10
Clock Stretching During FIFO Overflow	19-11
Clock Stretching During Repeated Start.....	19-11
TWI Programming Model.....	19-12
General Setup	19-12
Completer Mode	19-13
Requester Mode Program Flow.....	19-14
Requester Mode Clock Setup.....	19-15
Requester Mode Transmit.....	19-15
Requester Mode Receive	19-16
ADSP-21560_21561_21564_21568 TWI Register Descriptions	19-17
SCL Clock Divider Register	19-18
Control Register	19-19
FIFO Control Register	19-21

FIFO Status Register	19–23
Interrupt Mask Register	19–24
Interrupt Status Register	19–26
Controller Mode Address Register	19–29
Controller Mode Registers	19–30
Controller Mode Status Register	19–33
Rx Data Double-Byte Register	19–36
Rx Data Single-Byte Register	19–37
Target Mode Address Register	19–38
Target Mode Control Register	19–39
Target Mode Status Register	19–41
Tx Data Double-Byte Register	19–42
Tx Data Single-Byte Register	19–43

Digital Audio Interface (DAI)

SRU Features	20–1
Functional Description	20–2
ADSP-21560_21561_21564_21568 DAI Register List.....	20–2
ADSP-21560_21561_21564_21568 DAI Interrupt List	20–4
ADSP-21560_21561_21564_21568 DAI Trigger List	20–4
DAI Block Diagram.....	20–5
DAI Signal Naming Conventions	20–6
I/O Pin Buffers.....	20–6
Pin Buffer Signals.....	20–7
Pin Buffer Input Signal	20–7
Pin Buffer Enable Signal	20–7
Pin Buffer Functions	20–7
Pin Buffers as Signal Input.....	20–7
Pin Buffers As Signal Output.....	20–8
DAI Pin Buffer Status	20–8
DAIn Peripherals.....	20–9

Output Signals With Pin Buffer Enable Control	20–9
Output Signals Without Pin Buffer Enable Control.....	20–9
Signal Routing Units (SRUs)	20–9
Cross Mode Connections	20–9
Signal Routing Matrix by Groups.....	20–11
DAI Group Routing.....	20–12
Rules for SRU Connections.....	20–13
Miscellaneous Buffers and Functions.....	20–14
DAI Routing Capabilities	20–14
DAI Default Routing	20–16
Unused DAI Connections.....	20–18
DAI Operating Modes	20–18
DAI Pin Buffer Polarity.....	20–18
DAI Miscellaneous Buffer Polarity	20–19
DAI System Interrupt Controller (SIC)	20–19
Signal Routing Unit Effect Latency.....	20–21
DAI Programming Model.....	20–21
Debug Features	20–22
DAI Sources Overview.....	20–22
DAI0 Group A – Clock Routing Source Signals	20–23
DAI0 Group B – Serial Data Source Signals	20–25
DAI0 Group C – Frame Sync Source Signals	20–26
DAI0 Group D – Pin Signal Assignment Source Signals	20–27
DAI0 Group E – Miscellaneous Source Signals	20–30
DAI0 Group F – Pin Output Enable Source Signals	20–32
DAI1 Group A – Clock Routing Source Signals	20–33
DAI1 Group B – Serial Data Source Signals	20–34
DAI1 Group C – Frame Sync Source Signals.....	20–36
DAI1 Group D – Pin Signal Assignment Source Signals	20–37
DAI1 Group E – Miscellaneous Source Signals	20–40

DAI1 Group F – Pin Output Enable Source Signals	20–41
DAI Destination Registers Overview	20–43
SPORT Grouping with DAI.....	20–49
ADSP-21560_21561_21564_21568 DAI Register Descriptions	20–49
Clock Routing Control Register 0	20–52
Clock Routing Control Register 1	20–54
Clock Routing Control Register 2	20–56
Clock Routing Control Register 3	20–57
Clock Routing Control Register 4	20–58
Clock Routing Control Register 5	20–59
Serial Data Routing Control Register 0	20–60
Serial Data Routing Control Register 1	20–61
Serial Data Routing Control Register 2	20–62
Serial Data Routing Control Register 3	20–63
Serial Data Routing Control Register 4	20–64
Serial Data Routing Control Register 5	20–65
Serial Data Routing Control Register 6	20–66
Frame Sync Routing Control Register 0	20–67
Frame Sync Routing Control Register 1	20–69
Frame Sync Routing Control Register 2	20–71
Frame Sync Routing Control Register 4	20–72
Global SPORT Interrupt Grouping Register	20–73
Global SPORT Enable Register	20–76
Falling-Edge Interrupt Mask Register	20–79
Core Interrupt Priority Assignment Register	20–82
Rising-Edge Interrupt Mask Register	20–85
High Priority Interrupt Latch Register	20–88
Shadow High Priority Interrupt Latch Register	20–91
Low Priority Interrupt Latch Register	20–95
Shadow Low Priority Interrupt Latch Register	20–98

Miscellaneous Control Register 0	20-102
Miscellaneous Control Register 1	20-104
Pin Buffer Enable Register 0	20-106
Pin Buffer Enable Register 1	20-107
Pin Buffer Enable Register 2	20-108
Pin Buffer Enable Register 3	20-109
Pin Buffer Assignment Register 0	20-110
Pin Buffer Assignment Register 1	20-111
Pin Buffer Assignment Register 2	20-112
Pin Buffer Assignment Register 4	20-113
Pin Status Register	20-114

Serial Port (SPORT)

Features.....	21-1
Signal Descriptions	21-3
SRU Programming	21-6
Functional Description	21-6
ADSP-21560_21561_21564_21568 SPORT Register List	21-6
ADSP-21560_21561_21564_21568 SPORT Interrupt List	21-7
ADSP-21560_21561_21564_21568 SPORT Trigger List.....	21-9
ADSP-21560_21561_21564_21568 SPORT DMA Channel List	21-10
Block Diagram.....	21-11
Architectural Concepts	21-12
Multiplexer Logic	21-13
Data Types and Companding	21-16
Companding as a Function.....	21-17
Transmit Path.....	21-17
Receive Path	21-19
Operating Modes and Options	21-19
Serial Word Length.....	21-21

Clock Sample and Drive Edges	21–21
Frame Sync Options	21–23
Data-Dependent versus Data-Independent Frame Syncs	21–23
Support for Edge-Detected and Level-Sensitive Frame Syncs.....	21–23
Early versus Late Frame Syncs	21–24
Framed versus Unframed Frame Syncs	21–25
Frame Sync Polarity.....	21–26
Premature Frame Sync Error Detection	21–26
Mode Selection.....	21–27
Standard DSP Serial Mode.....	21–27
Stereo Modes.....	21–28
I ² S Mode.....	21–29
Left-Justified Mode	21–30
Right-Justified Mode.....	21–31
Multichannel (TDM) Mode.....	21–33
Packed I ² S Mode.....	21–36
Gated Clock Mode	21–37
Data Transfers and Interrupts	21–38
Data Buffers	21–39
Data Buffer Status	21–40
Single-Word (Core) Transfers	21–41
DMA Transfers.....	21–41
Data Transfer Interrupt	21–42
Error Detection (Status) Interrupt.....	21–43
Grouping of SPORTs.....	21–45
SPORT Programming Model	21–46
Initializing Core-Driven (Non-MCM) Transfers.....	21–47
Initializing Multichannel Transfers	21–48
Using DMA for SPORT Transfers.....	21–49
Using Companding as a Function.....	21–49

Programming Global SPORT Groups	21-50
Disabling Global SPORT Groups.....	21-51
ADSP-21560_21561_21564_21568 SPORT Register Descriptions	21-51
Half SPORT 'A' Multichannel 0-31 Select Register	21-53
Half SPORT 'B' Multichannel 0-31 Select Register	21-54
Half SPORT 'A' Multichannel 32-63 Select Register	21-55
Half SPORT 'B' Multichannel 32-63 Select Register	21-56
Half SPORT 'A' Multichannel 64-95 Select Register	21-57
Half SPORT 'B' Multichannel 64-95 Select Register	21-58
Half SPORT 'A' Multichannel 96-127 Select Register	21-59
Half SPORT 'B' Multichannel 96-127 Select Register	21-60
Half SPORT 'A' Control 2 Register	21-61
Half SPORT 'B' Control 2 Register	21-62
Half SPORT 'A' Control Register	21-63
Half SPORT 'B' Control Register	21-71
Half SPORT 'A' Divisor Register	21-80
Half SPORT 'B' Divisor Register	21-81
Half SPORT 'A' Error Register	21-82
Half SPORT 'B' Error Register	21-84
Half SPORT 'A' Multichannel Control Register	21-86
Half SPORT 'B' Multichannel Control Register	21-88
Half SPORT 'A' Multichannel Status Register	21-90
Half SPORT 'B' Multichannel Status Register	21-91
Half SPORT 'A' Rx Buffer (Primary) Register	21-92
Half SPORT 'B' Rx Buffer (Primary) Register	21-93
Half SPORT 'A' Rx Buffer (Secondary) Register	21-94
Half SPORT 'B' Rx Buffer (Secondary) Register	21-95
Half SPORT 'A' Tx Buffer (Primary) Register	21-96
Half SPORT 'B' Tx Buffer (Primary) Register	21-97
Half SPORT 'A' Tx Buffer (Secondary) Register	21-98

Half SPORT 'B' Tx Buffer (Secondary) Register	21-99
Precision Clock Generators (PCG)	
Features.....	22-1
Functional Description	22-1
ADSP-21560_21561_21564_21568 PCG Register List.....	22-2
Internal Interface	22-2
Serial Clock	22-3
Frame Sync	22-3
Frame Sync Output	22-3
Divider Mode Selection	22-3
Phase Shift	22-4
Pulse Width	22-5
Default Pulse Width.....	22-5
Input Clock Source Considerations	22-5
Timing Example for I ² S Mode	22-6
Cross Mode Connections	22-6
Operating Modes	22-6
Normal Mode	22-6
Bypass Mode	22-7
One-Shot Mode.....	22-7
Audio System Example	22-8
Clock Configuration Examples	22-9
PCG Event Control	22-9
External Event Trigger	22-10
External Event Trigger Delay.....	22-10
Programming Model.....	22-10
Frame Sync Phase Setting	22-11
External Event Trigger	22-11
Debug Features	22-11
ADSP-21560_21561_21564_21568 PCG Register Descriptions	22-11

Precision Clock A Control 0 Register	22–13
Precision Clock A Control 1 Register	22–14
Precision Clock B Control 0 Register	22–15
Precision Clock B Control 1 Register	22–16
Precision Clock C Control 0 Register	22–17
Precision Clock C Control 1 Register	22–18
Precision Clock D Control 0 Register	22–19
Precision Clock D Control 1 Register	22–20
Precision Clock Pulse Width Control 1 Register	22–21
Precision Clock Pulse Width Control 2 Register	22–23
Precision Clock Frame Sync Synchronization 1 Register	22–25
Precision Clock Frame Sync Synchronization 2 Register	22–28

Asynchronous Sample Rate Converter (ASRC)

Features.....	23–1
Functional Description	23–2
ADSP-21560_21561_21564_21568 ASRC Register List.....	23–2
ASRC Interrupt List	23–2
ASRC Block Diagram.....	23–2
SRU Programming	23–3
Clocking.....	23–3
I/O Ports	23–4
De-Emphasis Filter	23–4
Mute Control	23–4
SRC Core	23–4
RAM FIFO	23–4
Digital Servo Loop	23–5
FIR Filter	23–5
Sample Rate Sensing	23–5
Digital Filter Group Delay	23–5
Data Format	23–6

Operating Modes.....	23–6
TDM Input Mode.....	23–6
TDM Output Mode.....	23–7
Matched-Phase Mode.....	23–7
Bypass Mode	23–8
De-Emphasis Mode	23–9
Dithering Mode.....	23–9
Muting Modes.....	23–9
Soft Mute	23–9
Hard Mute	23–10
Auto Mute.....	23–10
Interrupts.....	23–10
Sources	23–10
SRC Mute Out.....	23–10
Masking.....	23–10
Service	23–10
Programming Model.....	23–10
Debug Features.....	23–11
ADSP-21560_21561_21564_21568 ASRC Register Descriptions	23–11
Control Register for ASRC 0 and 1	23–12
Control Register for ASRC 2 and 3	23–17
Mute Register	23–22
Ratio Register for ASRC 0 and 1	23–23
Ratio Register for ASRC 2 and 3	23–25

Sony/Philips Digital Interface (S/PDIF)

Features.....	24–1
ADSP-21560_21561_21564_21568 SPDIF Register List.....	24–1
SRU Programming	24–3
S/PDIF Interrupt List.....	24–3
Clocking.....	24–3

S/PDIF Transmitter.....	24-3
Functional Description.....	24-3
Input Data Formats.....	24-5
Operating Modes.....	24-6
Full Serial Mode.....	24-6
Standalone Mode.....	24-6
Data Output Mode.....	24-7
S/PDIF Receiver.....	24-7
Functional Description.....	24-8
Clock Recovery.....	24-8
TDM Clock Output.....	24-9
Output Data Format.....	24-9
Channel Status.....	24-9
Operating Modes.....	24-9
Compressed or Non-linear Audio Data.....	24-10
Emphasized Audio Data.....	24-10
Single-Channel Double-Frequency Mode.....	24-10
Clock Recovery Modes.....	24-10
Interrupts.....	24-11
Sources.....	24-11
Transmit Block Start.....	24-11
Receiver Status.....	24-11
Receiver Error.....	24-11
Masking.....	24-11
Service.....	24-12
Programming Model.....	24-12
Programming the Transmitter.....	24-12
Programming the Receiver.....	24-12
Interrupted Data Streams on the Receiver.....	24-13
Debug Features.....	24-13
Loopback Routing.....	24-13

ADSP-21560_21561_21564_21568 SPDIF Register Descriptions	24-13
Receive Control	24-15
Receive Status Register	24-17
Receive Status A0 Register	24-20
Receive Status B0 Register	24-21
Receive Status A1 Register	24-22
Receive Status B1 Register	24-23
Transmit Control Register	24-24
Transmit Status A0 Register	24-27
Transmit Status A1 Register	24-28
Transmit Status A2 Register	24-29
Transmit Status A3 Register	24-30
Transmit Status A4 Register	24-31
Transmit Status A5 Register	24-32
Transmit Status B0 Register	24-33
Transmit Status B1 Register	24-34
Transmit Status B2 Register	24-35
Transmit Status B3 Register	24-36
Transmit Status B4 Register	24-37
Transmit Status B5 Register	24-38
Transmit User Buffer A0 Register	24-39
Transmit User Buffer A1 Register	24-40
Transmit User Buffer A2 Register	24-41
Transmit User Buffer A3 Register	24-42
Transmit User Buffer A4 Register	24-43
Transmit User Buffer A5 Register	24-44
Transmit User Buffer B0 Register	24-45
Transmit User Buffer B1 Register	24-46
Transmit User Buffer B2 Register	24-47
Transmit User Buffer B3 Register	24-48

Transmit User Buffer B4 Register	24-49
Transmit User Buffer B5 Register	24-50
User Bit Update Register	24-51

Direct Memory Access (DMA)

DMA Channel Features	25-1
DMA Channel Functional Description.....	25-3
ADSP-21560_21561_21564_21568 DMA Register List	25-3
ADSP-21560_21561_21564_21568 DMA Channel List.....	25-4
DMA Definitions	25-5
Block Diagram.....	25-7
Architectural Concepts	25-8
DMA Channel SCB Interface.....	25-8
SCB Interface Signals	25-8
SCB Burst Transfers	25-9
Data Address Alignment	25-9
Descriptor Set Address Alignment	25-10
Peripheral Control Commands.....	25-10
Idle Command.....	25-11
Request-Data Command.....	25-11
Request-Data Urgent Command.....	25-11
Peripheral-Control Command Restrictions	25-11
DMA Channel Peripheral DMA Bus.....	25-11
Memory DMA and Triggering	25-12
Medium Band Width DMA Channel MMR Access Bus	25-14
DMA Channel Operation Flow.....	25-14
Startup Flow	25-14
Refresh Flow	25-16
Work Unit Transition Flow	25-17
Transfer Termination and Shutdown Flow	25-19
DMA Channel Errors.....	25-21
Status and Debug Errors	25-21

DMA Configuration Register Errors	25–22
Illegal Register Write During Run.....	25–22
Address Alignment Error.....	25–23
Memory Access Error	25–23
Trigger Overrun Error	25–23
Bandwidth-Monitor Error.....	25–23
Control Interface Error	25–23
DMA Operating Modes.....	25–23
Register-Based Flow Modes	25–24
Stop Mode.....	25–24
Autobuffer Mode.....	25–24
Descriptor-Based Flow Modes	25–24
Descriptor-Array Mode	25–25
Descriptor-List Mode	25–26
Descriptor-On-Demand Modes.....	25–26
Data Transfer Modes	25–27
Two-Dimensional DMA.....	25–27
DMA Channel Event Control.....	25–28
Event Signals	25–29
Work Unit State Events	25–30
Peripheral Interrupt Request Events	25–30
Peripheral Data Request Events	25–30
DMA Channel Triggers	25–31
Issuing Triggers	25–31
Waiting For Triggers	25–32
DMA Channel Programming Model	25–33
Mode Configuration.....	25–33
Register-Based Linear-Buffer Stop Flow Mode	25–33
Register-Based Autobuffer Flow Mode	25–34
Descriptor-Array Flow Mode.....	25–35
Descriptor-List Flow Mode	25–36

Register-Based Memory-to-Memory Transfer in Stop Flow Mode.....	25–37
Programming Concepts.....	25–39
Synchronization of Software and DMA.....	25–39
Interrupt and Trigger Event-Based Synchronization.....	25–39
Register Polling Based Synchronization.....	25–39
Descriptor Queues	25–40
Queues Using Event Generation for Every Descriptor Set.....	25–40
Queues Using Minimal Events	25–41
ADSP-21560_21561_21564_21568 DMA Register Descriptions	25–42
Start Address of Current Buffer Register	25–44
Current Address Register	25–45
Bandwidth Limit Count Register	25–46
Bandwidth Limit Count Current Register	25–47
Bandwidth Monitor Count Register	25–48
Bandwidth Monitor Count Current Register	25–49
Configuration Register	25–50
Current Descriptor Pointer Register	25–58
Pointer to Next Initial Descriptor Register	25–59
Previous Initial Descriptor Pointer Register	25–60
Status Register	25–61
Inner Loop Count Start Value Register	25–64
Current Count (1D) or Intra-row XCNT (2D) Register	25–65
Inner Loop Address Increment Register	25–66
Outer Loop Count Start Value (2D only) Register	25–67
Current Row Count (2D only) Register	25–68
Outer Loop Address Increment (2D only) Register	25–69
 Extended Memory DMA (EMDMA)	
EMDMA Features	26–1
EMDMA Functional Description	26–1
ADSP-21560_21561_21564_21568 EMDMA Register List.....	26–2

ADSP-21560_21561_21564_21568 EMDMA Interrupt List	26-2
ADSP-21560_21561_21564_21568 EMDMA Trigger List	26-3
DMA Addressing	26-3
DMA Burst Transfers	26-3
Transfer Control Block (TCB) Memory Storage	26-3
Chain Assignment	26-4
Starting Chain Loading	26-4
Buffered Chain Loading Register	26-4
TCB Storage	26-4
EMDMA Operating Modes	26-7
Standard DMA	26-8
Circular Buffered DMA	26-8
Chained DMA Mode	26-9
Data Direction On-the-Fly	26-10
Write Back Circular Index Pointer	26-10
Scatter/Gather DMA	26-10
Pre Modified Read/Write Index	26-13
Delay Line DMA	26-13
ADSP-21560_21561_21564_21568 EMDMA Register Descriptions	26-16
External Base Address Register	26-17
Circular Buffer Length Register	26-18
Chain Pointer Register	26-19
Internal Count Register	26-21
External Count Register	26-22
External Memory DMA Control Register	26-23
Internal Index Register	26-27
External Index Register	26-28
Internal Modifier Register	26-29
External Modifier Register	26-30
Delay Line Tap Count Register	26-31

Tap List Pointer Register	26–32
---------------------------------	-------

Cyclic Redundancy Check (CRC)

CRC Features.....	27–1
CRC Functional Description	27–2
ADSP-21560_21561_21564_21568 CRC Register List.....	27–2
ADSP-21560_21561_21564_21568 CRC Interrupt List	27–3
CRC Definitions	27–3
CRC Block Diagram.....	27–4
Peripheral DMA Bus	27–5
MMR Access Bus.....	27–5
Mirror Block.....	27–6
Data FIFO	27–6
DMA Request Generator.....	27–6
CRC Engine	27–6
Compare Logic	27–6
CRC Architectural Concepts.....	27–6
Look-up Table	27–7
Data Mirroring.....	27–7
FIFO Status and Data Requests.....	27–8
CRC Operating Modes	27–9
Data Transfer Modes	27–9
Memory Scan Compute-and-Compare Mode.....	27–10
Memory Scan Data Verify	27–10
Memory Transfer Compute-and-Compare Mode	27–10
Memory Transfer Data Fill Mode.....	27–11
CRC Event Control	27–11
Interrupt Signals.....	27–11
CRC Programming Model.....	27–12
CRC Mode Configuration	27–12
Look-up Table Generation	27–12

Core Driven Memory Scan Compute-and-Compare Mode	27-13
DMA Driven Memory Scan Compute-and-Compare Mode	27-14
Core Driven Memory Scan Data Verify Mode	27-16
DMA Driven Memory Scan Data Verify Mode	27-17
Core Driven Memory Transfer Compute-and-Compare Mode	27-19
DMA Driven Memory Transfer Compute-and-Compare Mode	27-20
DMA Driven Memory Transfer Data Fill Mode	27-22
ADSP-21560_21561_21564_21568 CRC Register Descriptions	27-23
Data Compare Register	27-25
Control Register	27-26
Data Word Count Register	27-29
Data Count Capture Register	27-30
Data Word Count Reload Register	27-31
Data FIFO Register	27-32
Fill Value Register	27-33
Interrupt Enable Register	27-34
Interrupt Enable Clear Register	27-35
Interrupt Enable Set Register	27-36
Polynomial Register	27-37
CRC Current Result Register	27-38
CRC Final Result Register	27-39
Status Register	27-40
 System Security	
Security Features	28-1
Security Functional Description	28-2
Security Mode Configuration	28-4
Status and Error Signals	28-4
 System Protection Unit (SPU)	
SPU Features	29-1

SPU Functional Description	29–1
ADSP-21560_21561_21564_21568 SPU Register List	29–1
ADSP-21560_21561_21564_21568 SPU Interrupt List	29–2
Peripheral Register Write Protection.....	29–2
SPU Block Diagram.....	29–5
SPU Architectural Concepts	29–5
SPU Event Control	29–5
SPU Programming Model	29–6
SPU Mode Configuration.....	29–7
Locking Write-Protect Registers	29–7
Protecting a Peripheral	29–7
Configuring Security Privileges of a Peripheral	29–8
ADSP-21560_21561_21564_21568 SPU Register Descriptions	29–8
Control Register	29–9
Secure Check Register	29–10
Secure Control Register	29–11
Secure Core Registers	29–13
Secure Peripheral Register	29–14
Status Register	29–15
Write Protect Register n	29–16
Write-Protect, Secure Peripheral, and Secure Core Registers	29–16
Processor-Specific Information	29–20
 Security Packet Engine (PKTE)	
PKTE Features.....	30–1
PKTE Functional Description	30–1
ADSP-21560_21561_21564_21568 PKTE Register List.....	30–1
ADSP-21560_21561_21564_21568 PKTE Interrupt List	30–3
PKTE Definitions	30–4
Cipher Module	30–5

Hash Module.....	30-5
Pseudo-Random Number Generator	30-6
Packet Engine Processing Details.....	30-8
Crypto Padding.....	30-8
Pad Generation and Insertion	30-9
Pad Types.....	30-9
Pad Length.....	30-10
Pad Verification and Consumption	30-12
Crypto and Hash Algorithms	30-14
IV Processing.....	30-17
ARC4 Processing.....	30-18
Hash State Loading	30-19
Sequence Number Processing.....	30-19
Sequence Number Processing in Extended SSL/TLS.....	30-19
Sequence Number Processing in DTLS.....	30-20
PKTE Block Diagram.....	30-23
PKTE Architectural Concepts	30-24
Packet Engine.....	30-24
Input/Output FIFO Buffers	30-24
Parallel Operations	30-24
DMA Controller	30-24
Interrupt Controller	30-25
Clock Controller	30-25
PKTE Operating Modes	30-25
Autonomous Ring Mode (ARM)	30-26
Target Command Mode (TCM)	30-27
Direct Host Mode (DHM)	30-27
PKTE Event Control	30-27
PKTE Interrupt Signals.....	30-27
PKTE Programming Model.....	30-30
PKTE Mode Configuration.....	30-33

PKTE Programming Concepts	30–33
Packet Engine Descriptor	30–33
Descriptor Processing.....	30–35
Descriptor Ownership.....	30–36
SA Record and State Record Structure.....	30–37
SA Record Structure	30–37
SA State Structure	30–40
ARC4 State Structure	30–40
Configuring Operations in the PKTE	30–41
Basic Operations and Decoding	30–41
Error Code Description	30–42
Extended Error Codes	30–43
Number Format	30–45
PKTE Programming Examples	30–46
Calculating SHA in Direct Host Mode.....	30–46
Performing AES Decryption in Direct Host Mode	30–47
ADSP-21560_21561_21564_21568 PKTE Register Descriptions	30–48
Packet Engine ARC4 State Record Address	30–51
Starting Entry of 256-byte ARC4 State Buffer	30–52
Packet Engine Buffer Pointer Register	30–53
Packet Engine Buffer Threshold Register	30–54
Packet Engine Command Descriptor Ring Base Address	30–56
Packet Engine Command Descriptor Count Register	30–57
Packet Engine Command Descriptor Count Increment Register	30–58
Packet Engine Configuration Register	30–59
PE Clock Control Register	30–62
PKTE Continue Register	30–64
Packet Engine Control Register	30–65
Starting Entry of 256-byte Data Input/Output Buffer	30–70
Packet Engine Destination Address	30–71
Packet Engine DMA Configuration Register	30–72

Packet Engine Endian Configuration Register	30-74
Packet Engine Halt Control Register	30-76
Packet Engine Halt Status Register	30-78
Interrupt Mask Disable Register	30-81
Interrupt Mask Enable Register	30-83
Interrupt Masked Status Register	30-85
Packet Engine Input Buffer Count Register	30-87
Packet Engine Input Buffer Count Increment Register	30-88
Interrupt Configuration Register	30-89
Interrupt Clear Register	30-90
Interrupt Enable Register	30-92
Interrupt Unmasked Status Register	30-94
Packet Engine Length Register	30-96
Packet Engine Output Buffer Count Register	30-98
Packet Engine Output Buffer Count Decrement Register	30-99
Packet Engine Result Descriptor Ring Base Address	30-100
Packet Engine Result Descriptor Count Registers	30-101
Packet Engine Result Descriptor Count Decrement Registers	30-102
Packet Engine Ring Configuration	30-103
Packet Engine Ring Pointer Status	30-104
Packet Engine Ring Status	30-105
Packet Engine Ring Threshold Registers	30-106
Packet Engine SA Address	30-108
ARC4 i and j Pointer Register	30-109
SA Command 0	30-110
SA Command 1	30-115
SA Inner Hash Digest Registers	30-119
SA Key Registers	30-120
SA Initialization Vector Register	30-121
SA Outer Hash Digest Registers	30-122

SA Ready Indicator	30-123
SA Sequence Number Register	30-124
SA Sequence Number Mask Registers	30-125
SA SPI Register	30-126
Packet Engine Source Address	30-127
Packet Engine Status Register	30-128
Packet Engine State Record Address	30-132
State Hash Byte Count Registers	30-133
State Inner Digest Registers	30-134
State Initialization Vector Registers	30-135
Packet Engine User ID	30-136

Public Key Accelerator (PKA)

PKA Features	31-1
PKA Functional Description.....	31-1
ADSP-21560_21561_21564_21568 PKA Register List	31-2
PKA Definitions	31-2
PKA Architectural Concepts.....	31-3
PKA Block Diagram	31-3
PKCP Vector Operations	31-4
Modular Exponentiation Operations	31-6
Modular Inversion	31-13
Modular Inversion with an Even Modulus.....	31-14
Modular Inversion with a Prime Modulus.....	31-14
ECC Operations	31-14
ADSP-21560_21561_21564_21568 PKA Register Descriptions	31-20
PKA Vector_A Length	31-21
PKA Vector_A Address	31-22
PKA Vector_B Length	31-23
PKA Vector_B Address	31-24

PKA Compare Result	31–25
PKA Vector_C Address	31–26
PKA Most-Significant-Word of Divide Remainder	31–27
PKA Vector_D Address	31–28
PKA Function	31–29
Start of PKA RAM space	31–32
PKA Most-Significant-Word of Result Vector	31–33
PKA Bit Shift Value	31–34

Public Key Interrupt Controller (PKIC)

PKIC Functional Description	32–1
ADSP-21560_21561_21564_21568 PKIC Register List.....	32–1
ADSP-21560_21561_21564_21568 PKIC Interrupt List	32–2
PKIC Programming Model.....	32–2
PKIC Programming Concepts	32–2
ADSP-21560_21561_21564_21568 PKIC Register Descriptions	32–3
Acknowledge Register	32–4
Enable Clear Register	32–5
Enable Control Register	32–6
Enable Set Register	32–7
Enabled Status Register	32–8
Polarity Control Register	32–9
Raw Status Register	32–10
Type Control Register	32–11

True Random Number Generator (TRNG)

TRNG Features	33–1
TRNG Functional Description	33–1
ADSP-21560_21561_21564_21568 TRNG Register List	33–1
Random Number Generation	33–2
Locking Detection and Prevention.....	33–3

Run Testing.....	33–4
Monobit Testing.....	33–4
Poker Testing.....	33–5
Data for Tests	33–6
X9.31 Postprocessing.....	33–6
TRNG Block Diagram	33–6
TRNG Architectural Concepts	33–7
TRNG Operating Modes.....	33–8
TRNG Data Transfer Modes	33–9
TRNG Event Control.....	33–9
TRNG Interrupt Signals.....	33–9
ADSP-21560_21561_21564_21568 TRNG Register Descriptions	33–10
TRNG Alarm Counter Register	33–12
TRNG Alarm Mask Register	33–14
TRNG Alarm Stop Register	33–15
TRNG Block Count Register	33–16
TRNG Configuration Register	33–17
Counter Register	33–19
TRNG Control Register	33–20
TRNG FRO De-tune Register	33–23
TRNG FRO Enable Register	33–24
TRNG Input Registers	33–25
TRNG Interrupt Acknowledge Register	33–26
Post-Process Key Registers	33–28
TRNG LFSR Access Register	33–29
TRNG LFSR Access Register	33–30
TRNG LFSR Access Register	33–31
TRNG Monobit Test Result Register	33–32
TRNG Output Registers	33–33
TRNG Poker Test Result Registers	33–34

TRNG Run Count Registers	33–35
TRNG Run Test State and Result Registers	33–36
TRNG Status Register	33–38
TRNG Test Register	33–40
TRNG Post-Process "V" Value Registers	33–43

FIR Accelerator (FIR)

Features.....	34–1
Clocking	34–1
Functional Description	34–2
ADSP-21560_21561_21564_21568 FIR Register List.....	34–3
ADSP-21560_21561_21564_21568 FIR Interrupt List	34–3
ADSP-21560_21561_21564_21568 FIR Trigger List	34–4
Compute Block	34–4
Partial Sum Register	34–5
Delay Line Memory.....	34–5
Coefficient Memory	34–5
Prefetch Data Buffer.....	34–5
Processing Output	34–6
System Memory Storage	34–7
Coefficients and Input Buffer Storage	34–7
Single Rate Input Filtering	34–7
Decimation	34–7
Interpolation	34–8
Operating Modes.....	34–8
Single Rate Processing	34–8
Single Iteration.....	34–8
Floating-point Multi-Iteration.....	34–8
Window Processing	34–9
Multi-Rate Processing	34–9
Decimation.....	34–9

Interpolation	34-9
Floating-Point Data Format.....	34-10
Fixed-Point Data Format	34-10
Auto Configuration Mode (ACM)	34-10
Data Transfer.....	34-11
Chain Assignment	34-11
DMA Access	34-13
Burst Access Support	34-13
Accelerator TCB.....	34-13
Chain Pointer DMA.....	34-13
Programming Model.....	34-14
Legacy Mode	34-15
Auto Configuration Mode (ACM)	34-16
Debug Mode	34-17
Write to Local Memory	34-17
Read from Local Memory.....	34-17
Single-Step Mode	34-17
Computing FIR Output, Tap Length Greater than 4096.....	34-18
Debug Features	34-19
Local Memory Access	34-19
Single-Step Mode	34-19
Emulation Considerations	34-19
Interrupts.....	34-19
Sources	34-20
Window Complete	34-20
All Channels Complete	34-20
MAC Status	34-21
Service	34-21
ADSP-21560_21561_21564_21568 FIR Register Descriptions	34-21
FIR Chain Pointer Register	34-23

FIR Coefficient Count Register	34–24
FIR Coefficient Index Register	34–25
FIR Coefficient Modifier Register	34–26
FIR Global Control Register	34–27
FIR Channel Control Register	34–31
Debug Address Register	34–33
FIR Debug Control Register	34–34
FIR Debug Data Read Register	34–35
FIR Debug Data Write Register	34–36
FIR DMA Status Register	34–37
FIR Input Data Base Register	34–39
FIR Input Data Count Register	34–40
FIR Input Data Index Register	34–41
FIR Input Data Modifier Register	34–42
FIR MAC Status Register	34–43
FIR Output Data Base Register	34–46
FIR Output Data Count Register	34–47
FIR Output Data Index Register	34–48
FIR Output Data Modifier Register	34–49
Software Control Register 1	34–50
Software Control Register 2	34–51
Secondary Global Control Register	34–52

IIR Accelerator (IIR)

Features.....	35–1
Clocking	35–1
Functional Description	35–2
ADSP-21560_21561_21564_21568 IIR Register List	35–3
ADSP-21560_21561_21564_21568 IIR Interrupt List	35–4
ADSP-21560_21561_21564_21568 IIR Trigger List.....	35–4

Multiply and Accumulate (MAC) Unit	35-4
Input Data and Biquad State	35-5
Coefficient Memory	35-5
Internal Memory Storage.....	35-5
Coefficient Memory Storage.....	35-5
Operating Modes.....	35-6
Window Processing Mode	35-6
40-Bit Floating-Point Mode	35-6
Save Biquad State Mode	35-7
Data Transfers.....	35-7
IIR Accelerator TCB.....	35-7
DMA Access.....	35-9
Burst Mode Access	35-9
Chain Pointer DMA	35-9
Interrupts.....	35-11
Sources	35-11
Window Complete.....	35-11
All Channels Complete	35-11
MAC Status.....	35-12
ADSP-21560_21561_21564_21568 IIR Register Descriptions	35-12
Chain Pointer Register	35-14
Coefficient Buffer Index Register	35-15
Coefficient Buffer Length Register	35-16
Coefficient Index Modifier Register	35-17
Global Control Register	35-18
Channel Control Register	35-22
IIR Debug Address Register	35-24
IIR Debug Control Register	35-25
IIR Debug Read Data High Register	35-26
IIR Debug Read Data Low Register	35-27

IIR Debug Write Data High Register	35–28
IIR Debug Write Data Low Register	35–29
DMA Status Register	35–30
Input Buffer Base Register	35–32
Input Data Index Register	35–33
Input Data Buffer Length Register	35–34
Input Data Index Modifier Register	35–35
MAC Status Register	35–36
Output Buffer Base Register	35–37
Output Data Buffer Index Register	35–38
IIR Output Data Buffer Length Register	35–39
IIR Output Data Index Modifier Register	35–40
Software Control Register1	35–41
Software Control Register2	35–42
Secondary Global Control Register	35–43

Boot ROM and Booting the Processor

SRAM Requirements	36–1
Preboot Operations.....	36–2
Start-up Sequence.....	36–3
Core Reset Sequencing.....	36–3
Core Start-up	36–3
PLL Bypass	36–4
Boot Benchmark	36–4
Fault Configuration.....	36–5
L2 Controller Configuration	36–5
L2 Memory Initialization	36–6
Idle On Entry.....	36–6
SPU Configuration.....	36–6
SMPU Configuration	36–7
Secure Debug Key Processing	36–7

CGU Configuration	36-8
L1 Memory Initialization	36-10
Default Entry Point	36-10
NO-BOOT Processing	36-10
<u>SYS_RESOUT</u> Signal	36-11
Bypassing the Boot Process	36-11
Boot Mode Disable	36-11
Boot Command Customization	36-11
Boot Mode Specific SPU Configuration	36-12
Executing the Boot Mode	36-12
Benchmarking Boot Time	36-13
Watchdog Timeout	36-15
Boot Modes	36-15
No-Boot Mode	36-16
SPI Controller Boot Mode	36-16
SPI Target Boot Mode	36-22
UART Target Boot Mode	36-25
xSPI Controller Boot Mode	36-28
Boot Mode Swipe Through Command Customization	36-44
Alternate Controller/Target Boot Support	36-44
Boot Loader Stream	36-46
Block Types	36-50
Normal Block	36-50
First Block	36-50
Final Block	36-51
Indirect Block	36-51
Ignore Block	36-52
Fill Block	36-52
Init Block	36-53
Callback Block	36-54

Save Block	36-56
Single-Block Boot Streams	36-56
Direct Code Execution	36-56
Multi-Application Boot Streams	36-57
CRC32 Protection	36-59
Secure Boot.....	36-59
Terminology	36-61
Signing for Secure Boot Images	36-62
Secure Boot Image Types.....	36-62
Secure Boot Image Format.....	36-63
Secure Boot Image Attributes	36-66
Secure Boot Streams	36-67
Secure Debug Access.....	36-68
Boot ROM Errors and Failures	36-70
Boot ROM Programming Model	36-73
Boot Mode Driver API	36-73
Error Handler	36-75
Page Mode	36-76
Boot Hook Function	36-76
enum ROM_HOOK_CALL_CAUSE	36-77
Boot Return Feature	36-77
Boot Termination and Application Execution	36-77
Boot ROM OTP Customizations	36-78
API Reference	36-78
adi_rom_Boot()	36-78
adi_rom_BootKernel().....	36-81
adi_rom_Crc32Init()	36-82
adi_rom_Crc32Poly()	36-83
adi_rom_GetAddress().....	36-83
adi_rom_MemCompare()	36-83

adi_rom_MemCopy()	36-84
adi_rom_MemCrc()	36-85
adi_rom_MemDma()	36-85
adi_rom_MemFill()	36-88
adi_rom_PeriphDma()	36-88
callback()	36-89
initcode()	36-91
adi_rom_idle_loop()	36-92
adi_rom_CguInit()	36-92
adi_rom_ShaInit()	36-92
adi_rom_Sha()	36-93
Data Structures	36-96
struct ADI_ROM_BOOT_BUFFER	36-96
struct ADI_ROM_BOOT_CONFIG	36-96
struct ADI_ROM_BOOT_HEADER	36-106
struct ADI_ROM_BOOT_INTER_BUFFER	36-107
struct ADI_ROM_BOOT_INTER_BUFFERS	36-108
struct ADI_ROM_BOOT_MODES	36-108
struct ADI_ROM_BOOT_REGISTRY	36-109
struct ADI_ROM_BOOT_SPI	36-110
struct ADI_ROM_BOOT_XSPI	36-112
struct ADI_ROM_BOOT_UART	36-113
struct ADI_ROM_OTP_BOOT_CFG	36-114
struct ADI_ROM_OTP_BOOT_CGU_INFO	36-117
struct ADI_ROM_OTP_BOOT_CMD_INFO	36-119
struct ADI_ROM_OTP_BOOT_INFO	36-120
struct ROM_BOOT_DMA_INSTANCE	36-121
struct ROM_BOOT_MDMA	36-121
struct ROM_BOOT_MDMA_REGS	36-122
struct ROM_DMA_MDMA_CONFIG	36-122

struct ROM_DMA_PDMA_CONFIG	36–124
struct otp_data	36–125
Enumerations	36–128
enum ADI_ROM_BOOT_KEY_TYPE	36–128
enum ADI_ROM_BOOT_TYPE	36–128
enum ROM_BOOT_RESULT	36–129
enum ROM_BOOT_MDMA_CRC_SUPPORT	36–133
enum ROM_DMA_DONE_DETECT_METHOD	36–134
enum ROM_DMA_MDMA_ID	36–134
enum ROM_DMA_MDMA_OPERATION	36–135
enum ROM_DMA_PDMA_OPERATION	36–136
enum ROM_GETADDR_VALUE	36–136
enum ROM_HOOK_CALL_CAUSE	36–137
enum ROM_SB_IMAGE_TYPE	36–138
enum ROM_SPI_PROTOCOL	36–139

System Crossbars (SCB)

SCB Features	37–1
SCB Functional Description	37–1
ADSP-21560_21561_21564_21568 SCB0 Register List	37–2
ADSP-21560_21561_21564_21568 SCB3 Register List	37–5
ADSP-21560_21561_21564_21568 SCB4 Register List	37–5
ADSP-21560_21561_21564_21568 SCB5 Register List	37–6
SCB Architectural Concepts	37–7
SCB Block Diagram	37–7
SCB Block Diagram	37–8
System Crossbars	37–14
SCB Bus Requester IDs	37–15
SCB Programming Model	37–17
SCB Programming Concepts	37–18
QoS Programming	37–21

ADSP-21560_21561_21564_21568 SCB0 Register Descriptions	37-22
Accelerator Channel 0 Read Quality of Service Register	37-26
Accelerator Channel 0 Write Quality of Service Register	37-27
Accelerator Channel 1 Read Quality of Service Register	37-28
Accelerator Channel 1 Write Quality of Service Register	37-29
CRC0 Channel 0 Read Quality of Service Register	37-30
CRC0 Channel 0 Write Quality of Service Register	37-31
CRC0 Channel 1 Read Quality of Service Register	37-32
CRC0 Channel 1 Write Quality of Service Register	37-33
CRC1 Channel 0 Read Quality of Service Register	37-34
CRC1 Channel 0 Write Quality of Service Register	37-35
CRC1 Channel 1 Read Quality of Service Register	37-36
CRC1 Channel 1 Write Quality of Service Register	37-37
Crypto Read Quality of Service Register	37-38
CRYPTO Write Quality of Service Register	37-39
DBG Read Quality of Service Register	37-40
DBG Write Quality of Service Register	37-41
DLDMA0 Channel 0 Read Quality of Service Register	37-42
DLDMA0 Channel 0 Write Quality of Service Register	37-43
DLDMA0 Channel 1 Read Quality of Service Register	37-44
DLDMA0 Channel 1 Write Quality of Service Register	37-45
DLDMA1 Channel 0 Read Quality of Service Register	37-46
DLDMA1 Channel 0 Write Quality of Service Register	37-47
DLDMA1 Channel 1 Read Quality of Service Register	37-48
DLDMA1 Channel 1 Write Quality of Service Register	37-49
ETR Read Quality of Service Register	37-50
ETR Write Quality of Service Register	37-51
HSMDMA Channel 0 Read Quality of Service Register	37-52
HSMDMA Channel 0 Write Quality of Service Register	37-53
HSMDMA Channel 1 Read Quality of Service Register	37-54

HSMDMA Channel 1 Write Quality of Service Register	37-55
LP0 Read Quality of Service Register	37-56
LP0 Write Quality of Service Register	37-57
LP1 Read Quality of Service Register	37-58
LP1 Write Quality of Service Register	37-59
MLB Read Quality of Service Register	37-60
MLB Write Quality of Service Register	37-61
MSMDMA Channel 0 Read Quality of Service Register	37-62
MSMDMA Channel 0 Write Quality of Service Register	37-63
MSMDMA Channel 1 Read Quality of Service Register	37-64
MSMDMA Channel 1 Write Quality of Service Register	37-65
SH0 DPORT Read Quality of Service Register	37-66
SH0 DPORT Write Quality of Service Register	37-67
SH0 IPORT Read Quality of Service Register	37-68
SH0 IPORT Write Quality of Service Register	37-69
SH0 MMR Read Quality of Service Register	37-70
SH0 MMR Write Quality of Service Register	37-71
SP0A Read Quality of Service Register	37-72
SP0A Write Quality of Service Register	37-73
SP0B Read Quality of Service Register	37-74
SP0B Write Quality of Service Register	37-75
SP1A Read Quality of Service Register	37-76
SP1A Write Quality of Service Register	37-77
SP1B Read Quality of Service Register	37-78
SP1B Write Quality of Service Register	37-79
SP2A Read Quality of Service Register	37-80
SP2A Write Quality of Service Register	37-81
SP2B Read Quality of Service Register	37-82
SP2B Write Quality of Service Register	37-83
SP3A Read Quality of Service Register	37-84

SP3A Write Quality of Service Register	37–85
SP3B Read Quality of Service Register	37–86
SP3B Write Quality of Service Register	37–87
SP4A Read Quality of Service Register	37–88
SP4A Write Quality of Service Register	37–89
SP4B Read Quality of Service Register	37–90
SP4B Write Quality of Service Register	37–91
SP5A Read Quality of Service Register	37–92
SP5A Write Quality of Service Register	37–93
SP5B Read Quality of Service Register	37–94
SP5B Write Quality of Service Register	37–95
SP6A Read Quality of Service Register	37–96
SP6A Write Quality of Service Register	37–97
SP6B Read Quality of Service Register	37–98
SP6B Write Quality of Service Register	37–99
SP7A Read Quality of Service Register	37–100
SP7A Write Quality of Service Register	37–101
SP7B Read Quality of Service Register	37–102
SP7B Write Quality of Service Register	37–103
SPI0 RX Read Quality of Service Register	37–104
SPI0 RX Write Quality of Service Register	37–105
SPI0 TX Read Quality of Service Register	37–106
SPI0 TX Write Quality of Service Register	37–107
SPI1 RX Read Quality of Service Register	37–108
SPI1 RX Write Quality of Service Register	37–109
SPI1 TX Read Quality of Service Register	37–110
SPI1 TX Write Quality of Service Register	37–111
SPI2 RX Read Quality of Service Register	37–112
SPI2 RX Write Quality of Service Register	37–113
SPI2 TX Read Quality of Service Register	37–114

SPI2 TX Write Quality of Service Register	37-115
UART0 RX Read Quality of Service Register	37-116
UART0 RX Write Quality of Service Register	37-117
UART0 TX Read Quality of Service Register	37-118
UART0 TX Write Quality of Service Register	37-119
UART1 TX Read Quality of Service Register	37-120
UART1 RX Write Quality of Service Register	37-121
UART1 TX Read Quality of Service Register	37-122
UART1 TX Write Quality of Service Register	37-123
UART2 RX Read Quality of Service Register	37-124
UART2 RX Write Quality of Service Register	37-125
UART2 TX Read Quality of Service Register	37-126
UART2 TX Write Quality of Service Register	37-127
ADSP-21560_21561_21564_21568 SCB3 Register Descriptions	37-127
APB DCLK 0 Synchronization Mode Register	37-128
ADSP-21560_21561_21564_21568 SCB4 Register Descriptions	37-128
Comp Id 0	37-130
Comp Id 1	37-131
Comp Id 2	37-132
Comp Id 3	37-133
Fabric S2port Ib.fn Mod	37-134
Fabric S2port Ib.read Qos	37-135
Fabric S2port Ib.write Qos	37-136
Fir Ch0 Ib.fn Mod	37-137
Fir Ch0 Ib.fn Mod2	37-138
Fir Ch0 Ib.read Qos	37-139
Fir Ch0 Ib.write Qos	37-140
Fir Ch1 Ib.fn Mod	37-141
Fir Ch1 Ib.fn Mod2	37-142
Fir Ch1 Ib.read Qos	37-143

Fir Ch1 Ib.write Qos	37-144
Iir Ch0 Ib.fn Mod	37-145
Iir Ch0 Ib.fn Mod2	37-146
Iir Ch0 Ib.read Qos	37-147
Iir Ch0 Ib.write Qos	37-148
Iir Ch1 Ib.fn Mod	37-149
Iir Ch1 Ib.fn Mod2	37-150
Iir Ch1 Ib.read Qos	37-151
Iir Ch1 Ib.write Qos	37-152
Periph Id 0	37-153
Periph Id 1	37-154
Periph Id 2	37-155
Periph Id 3	37-156
Periph Id 4	37-157
Periph Id 5	37-158
Periph Id 6	37-159
Periph Id 7	37-160
Sharc Dport.fn Mod	37-161
Sharc Dport.read Qos	37-162
Sharc Dport.write Qos	37-163
ADSP-21560_21561_21564_21568 SCB5 Register Descriptions	37-163
Comp Id 0	37-165
Comp Id 1	37-166
Comp Id 2	37-167
Comp Id 3	37-168
Periph Id 0	37-169
Periph Id 1	37-170
Periph Id 2	37-171
Periph Id 3	37-172
Periph Id 4	37-173

Periph Id 5	37-174
Periph Id 6	37-175
Periph Id 7	37-176
Remap	37-177
Spi2 S Ib.fn Mod	37-178
Spi2 S Ib.fn Mod Iss Bm	37-179
Spif M.fn Mod	37-180

Memory Error Protection Unit (MEPU)

Memory Error Controller (MEC)	38-1
MEC Features	38-1
Parity Controller (PCTL)	38-2
PCTL Features.....	38-2
MEC Functional Description.....	38-2
ADSP-21560_21561_21564_21568 MEC Register List	38-2
ADSP-21560_21561_21564_21568 MEC Trigger List	38-3
MEPU Block Diagram	38-3
MEC Block Diagram.....	38-4
PCTL Block Diagram.....	38-4
MEC Architectural Concepts.....	38-4
PCTL Integration	38-4
ADSP-21560_21561_21564_21568 MEC Register Descriptions	38-4
Component ID0 Register	38-6
Component ID1 Register	38-7
Component ID2 Register	38-8
Component ID3 Register	38-9
Clear Register	38-10
L2 Parity Error Control Register	38-11
L2 Parity Error Interrupt Mask Register	38-12
L2 Parity Error Status Register	38-13

L2 Parity Error Interrupt Request Global Control Register	38–14
L2 Parity Error Interrupt Request Global Status Register	38–15
Parity Error Interrupt Request Global Control Register	38–16
Parity Error Interrupt Request Global Status Register	38–17
Parity Error Control Register	38–18
Parity Error Interrupt Mask Register	38–20
Parity Error Status Register	38–22
Peripheral ID0 Register	38–24
Peripheral ID1 Register	38–25
Peripheral ID2 Register	38–26
Peripheral ID3 Register	38–27
Peripheral ID4 Register	38–28

System Debug

DBG Features	39–1
Debug Functional Description.....	39–1
DBG Block Diagram	39–1
DBG Definitions.....	39–2
Test Access Port Controller (TAPC)	39–3
Debug Access Ports.....	39–3
Embedded Cross Trigger (ECT)	39–3
ADSP-21560_21561_21564_21568 CTI Register Descriptions	39–4
External Multiplexor Control Register	39–7
Authentication Status	39–8
Claim Tag Clear Register	39–9
Claim Tag Set Register	39–10
Component ID0	39–11
Component ID1	39–12
Component ID2	39–13
Component ID3	39–14

CTI Application Trigger Clear Register	39-15
CTI Application Pulse Register	39-16
CTI Application Trigger Set Register	39-17
CTI Channel In Status Register	39-18
CTI Channel Out Status Register	39-19
CTI Control Register	39-20
Enable CTI Channel Gate Register	39-21
CTI Trigger 0 to Channel Enable Register	39-22
CTI Trigger 1 to Channel Enable Register	39-23
CTI Trigger 2 to Channel Enable Register	39-24
CTI Trigger 3 to Channel Enable Register	39-25
CTI Trigger 4 to Channel Enable Register	39-26
CTI Trigger 5 to Channel Enable Register	39-27
CTI Trigger 6 to Channel Enable Register	39-28
CTI Trigger 7 to Channel Enable Register	39-29
CTI Interrupt Acknowledge Register	39-30
CTI Channel to Trigger 0 Enable Register	39-31
CTI Channel to Trigger 1 Enable Register	39-32
CTI Channel to Trigger 2 Enable Register	39-33
CTI Channel to Trigger 3 Enable Register	39-34
CTI Channel to Trigger 4 Enable Register	39-35
CTI Channel to Trigger 5 Enable Register	39-36
CTI Channel to Trigger 6 Enable Register	39-37
CTI Channel to Trigger 7 Enable Register	39-38
CTI Trigger In Status Register	39-39
CTI Trigger Out Status Register	39-40
Device ID	39-41
Device Type	39-42
Integration Test Channel Input	39-43
ITCHINACK	39-44

Integration Test Channel Output	39–45
ITCHOUTACK	39–46
Integration Mode Control Register	39–47
Integration Test Trigger Input	39–48
ITTRIGINACK	39–49
Integration Test Trigger Output	39–50
ITTRIGOUTACK	39–51
Lock Access Register	39–52
Lock Status Register	39–53
Peripheral ID0	39–54
Peripheral ID1	39–55
Peripheral ID2	39–56
Peripheral ID3	39–57
Peripheral ID4	39–58
Peripheral ID5	39–59
Peripheral ID6	39–60
Peripheral ID7	39–61
ADSP-21560_21561_21564_21568 TAPC Register Descriptions	39–61
Debug Control Register	39–62
IDCODE Register	39–64
Secure Debug Key 0 Register	39–65
Secure Debug Key 1 Register	39–66
Secure Debug Key 2 Register	39–67
Secure Debug Key 3 Register	39–68
Secure Debug Key 4 Register	39–69
Secure Debug Key 5 Register	39–70
Secure Debug Key 6 Register	39–71
Secure Debug Key 7 Register	39–72
Secure Debug Key Control Register	39–73
Secure Debug Key Status Register	39–74

USERCODE Register39–75

ADSP-21560_21561_21564_21568 Register List

Preface

Thank you for purchasing and developing systems using an ADSP-2156x SHARC+[®] processor from Analog Devices, Inc.

Purpose of This Manual

The *ADSP-21560/21561/21564/21568 SHARC+ Processor Hardware Reference* provides architectural information about the ADSP-2156x processors. This hardware reference provides the main architectural information about these processors. The architectural descriptions cover functional blocks, buses, and ports, including all features and processes that they support.

For timing, electrical, and package specifications, refer to the *SHARC+ Single Core High Performance DSP ADSP-21560/21561/21564/21568 Data Sheet*.

NOTE: In this document, all references to the ADSP-2156x refer to the ADSP-21560/21561/21564/21568 processors.

Intended Audience

The primary audience for this manual is a programmer who is familiar with Analog Devices products. The manual assumes the audience has a working knowledge of the appropriate product architecture and instruction set. Programmers who are unfamiliar with Analog Devices processors can use this manual, but should supplement it with other texts, such as programming reference books and data sheets, that describe their target architecture.

What's New in This Manual

This manual is the second preliminary revision (0.3) of the ADSP-21560/21561/21564/21568 SHARC+ Processor Hardware Reference.

NOTE: Analog Devices is in the process of updating documentation to provide terminology and language that is culturally appropriate. This is a process with a wide scope and will be phased in as quickly as possible. Thank you for your patience.

Technical or Customer Support

You can reach customer and technical support for processors and peripherals from Analog Devices using the following resources:

- Post your questions in the processors and DSP support community at *EngineerZone*[®]:

<http://ez.analog.com/community/dsp>

- Submit your questions to technical support at *Connect with ADI Specialists*:

<http://www.analog.com/support>

- Contact your Analog Devices sales office or authorized distributor. Locate one at:

<http://www.analog.com/adi-sales>

Product Information

Product information can be obtained from the Analog Devices Internet site

Analog Devices Web Site

The Analog Devices Web site, <http://www.analog.com>, provides information about a broad range of products—analog integrated circuits, amplifiers, converters, and digital signal processors.

To access a complete technical library for each processor family, go to: http://www.analog.com/processors/technical_library The manuals selection opens a list of current manuals related to the product as well as a link to the previous revisions of the manuals. When locating your manual title, note a possible errata check mark next to the title that leads to the current correction report against the manual.

Also note, [MyAnalog.com](http://www.analog.com/MyAnalog.com) is a free feature of the Analog Devices Web site that allows customization of a Web page to display only the latest information about products you are interested in. You can choose to receive weekly e-mail notifications containing updates to the Web pages that meet your interests, including documentation errata against all manuals. [MyAnalog.com](http://www.analog.com/MyAnalog.com) provides access to books, application notes, data sheets, code examples, and more.

Visit [MyAnalog.com](http://www.analog.com/MyAnalog.com) to sign up. If you are a registered user, just log on. Your user name is your e-mail address.

EngineerZone

[EngineerZone](http://www.analog.com/EngineerZone) is a technical support forum from Analog Devices. It allows you direct access to ADI technical support engineers. You can search FAQs and technical information to get quick answers to your design questions.

Use EngineerZone to connect with other developers who face similar design challenges. You can also use this open forum to share knowledge and collaborate with the ADI support team and your peers. Visit <http://ez.analog.com> to sign up.

1 Introduction

Power and Clock Management (DPM/RCU/CGU/CDU)

The processor contains four modules that control power and clocking.

Dynamic Power Management (DPM)

The [Dynamic Power Management \(DPM\)](#) unit of the processor controls transitions between different power-saving modes.

Reset Control Unit (RCU)

The [Reset Control Unit \(RCU\)](#) controls how all the functional units enter and exit reset. Differences in functional requirements and clocking constraints define how reset signals are generated. The RCU supports separate reset control for various chip sub-systems. For deterministic operation programmers should ensure there is no activity between separate chip sub-systems during reset activity. More global reset options are also supported. Programs must guarantee that none of the reset functions puts the system into an undefined state or causes resources to stall.

Clock Generation Unit (CGU)

The [Clock Generation Unit \(CGU\)](#) includes the phase locked loop (PLL) and the PLL control unit (PCU). The PLL generates a clock that runs at a frequency that is a multiple of the SYS_CLKINx input clock frequency. It also generates all on-chip clocks and synchronization signals. The PCU allows the application software to control the PLL module operation. All clocks derived from the CGU are then forwarded to the CDU for distribution.

Clock Distribution Unit (CDU)

The [Clock Distribution Unit \(CDU\)](#) consists of an array of software-configurable multiplexors that select clocks originated from up to four different clock sources that are generated from the CGUs. Unused input clocks are grounded internally and never selected. The output clock signal for each multiplexor is assigned to one or more destinations within the processor.

System Interrupts and Triggers (SEC/TRU)

The SEC and TRU modules provide interrupt and trigger management functions for the processor.

System Event Controller (SEC)

The SEC manages the configuration of all system event sources and the propagation of system events to and the system fault interface. The SEC arbitrates among all pending system interrupt requests and presents the highest priority interrupt to the for processing.

Trigger Routing Unit (TRU)

The [Trigger Routing Unit \(TRU\)](#) provides system-level sequence control without core intervention. The TRU maps trigger generators to trigger receivers. Receiver endpoints can be configured to respond to triggers in various ways. Multiple TRUs may be provided in a multiprocessor system to create a trigger network. Common applications enabled by the TRU include:

- Automatically triggering the start of a DMA sequence after a sequence from another DMA channel completes
- Software triggering
- Synchronization of concurrent activities

System Memory (L2CTL/OTPC/SMPU)

The following sections describe the memory architecture of the processor. For more information, refer to the processor data sheet.

L2 Memory Controller (L2CTL)

The [L2 System Memory](#) has significant bandwidth for core accesses. However, L2 responds slower to core accesses than L1 memories. Due to sophisticated data integrity protection and write protection, L2 SRAM is also ideal for data and instructions critical for safe operation of the application.

One-Time Programmable Memory Controller (OTPC)

The [One-Time Programmable Memory Controller \(OTPC\)](#) module is a complete system integrating an OTP memory core with a programming controller, charge pump, and voltage regulator. A built-in Hamming Code Error Correction (ECC), and a fully implemented double-redundant program or read scheme protect the OTP data.

System Memory Protection Unit (SMPU)

The [System Memory Protection Unit \(SMPU\)](#) provides a flexible way of protecting memory regions against read or write access from any or all masters in the system. In addition, it can guard against memory access depending on security privileges of the system master.

Direct Memory Access (DMA/EMDMA/CRC)

DMA Controller (DMA)

The processors use [Direct Memory Access \(DMA\)](#) to transfer data within memory spaces or between a memory space and a peripheral. The processors can specify data transfer operations and return to normal processing while the fully integrated DMA controller carries out the data transfers independent of processor activity.

DMA transfers can occur between memory and a peripheral or between one memory and another memory. Each Memory-to-memory DMA stream uses two channels, where one channel is the source channel, and the second is the destination channel. Most peripherals have at least one dedicated DMA channel associated with them.

All DMAs can transport data to and from all on-chip and off-chip memories. Programs can use two types of DMA transfers, descriptor-based or register-based. Register-based DMA allows the processors to directly program DMA control registers to initiate a DMA transfer. On completion, the control registers may be automatically updated with their original setup values for continuous transfer. Descriptor-based DMA transfers require a set of parameters stored within memory to initiate a DMA sequence. Descriptor-based DMA transfers allow multiple DMA sequences to be chained together and a DMA channel can be programmed to automatically set up and start another DMA transfer after the current sequence completes.

Extended Memory DMA (EMDMA)

The [Extended Memory DMA \(EMDMA\)](#) engine can be used in applications that copy data in a non-sequential manner. This includes delay lines, scatter and gather, and circular access types.

Unlike previous SHARC processors which contained external port DMA, the current EMDMA module can access all memory locations (L1/L2) for source and destination DMA operations.

Cyclic Redundancy Check (CRC)

The [Cyclic Redundancy Check \(CRC\)](#) peripheral performs the cyclic redundancy check (CRC) of the block of data that is presented to the peripheral. The peripheral provides a means to verify periodically the integrity of the system memory, the contents of memory-mapped registers (MMRs), or communication message objects. It is based on a CRC32 engine that computes the signature of 32-bit data presented to the peripheral.

The dedicated hardware compares the calculated signature of the operation to a pre-loaded expected signature. If the two signatures fail to match, the peripheral generates an error. The source channel of the memory-to-memory DMA channels can provide data. The CRC optionally forwards data to memory through the destination DMA channel. Alternatively, the peripheral supports data presented by core write transactions.

The CRC peripheral implements a reduced table-look-up algorithm to compute the signature of the data. The CRC uses a programmable 32-bit CRC polynomial to generate the look-up table (LUT) contents automatically.

More CRC peripheral modes allow for initializing large memory sections with a constant value, or for verifying that sections of memory are equal to a constant value.

Peripherals

The SHARC+ processor contains a rich set of industry leading system peripherals. The processor is the platform of choice for applications that require RISC-like programmability, multimedia support, and leading edge signal processing in one integrated package. These applications span a wide array of markets, including automotive, pro audio, and industrial-based applications that require high floating-point performance. These peripherals are described in the following sections.

- [General-Purpose I/O \(GPIO\) Peripherals](#)
- [DAI/SRU Peripherals](#)

General-Purpose I/O (GPIO) Peripherals

The SHARC+ processors feature general-purpose I/O pins mapped across ports (PORT A). Each pin can be configured individually to serve as a GPIO pin or as a peripheral-specific pin.

GPIO Ports (PORT)

When configured in the default GPIO mode, the PORT pins allow for the processor to interface to system components to provide handshaking functionality as either inputs or outputs. When in output mode, open-drain output is supported. A single MMR access can be used to sense or set individual pins or a complete port of 16 pins.

Additionally, each GPIO pin can optionally be configured to raise a system interrupt on the processor via a dedicated pin interrupt (PINT) block, and all peripheral functions are controlled via a set of port multiplexing registers, with specific settings defined in the processor data sheet.

Serial Peripheral Interface Ports (SPI)

The [Serial Peripheral Interface \(SPI\)](#) is an industry-standard synchronous serial link that supports communication with multiple SPI-compatible devices. The baseline SPI peripheral is a synchronous, four-wire interface consisting of two data pins, one device select pin, and a gated clock pin. The two data pins allow full-duplex operation to other SPI-compatible devices. Two extra (optional) data pins are provided on specific SPIs to support quad SPI operation. Enhanced modes of operation such as flow control, fast mode, and dual-I/O mode (DIOM) are also supported. In addition, a direct memory access (DMA) mode allows for transferring several words with minimal CPU interaction.

Universal Asynchronous Receiver/Transmitter (UART)

The [Universal Asynchronous Receiver/Transmitter \(UART\)](#) module is a full-duplex peripheral compatible with PC-style industry-standard UARTs. The UART converts data between serial and parallel formats. The serial communication follows an asynchronous protocol that supports various word lengths, stop bits, bit rates, and parity-generation options. The UART includes interrupt-handling hardware. Multiple events can generate interrupts.

The UART is logically compliant to EIA-232E, EIA-422, EIA-485 and LIN standards, but usually requires external transceiver devices to meet electrical requirements. In IrDA (Infrared Data Association) mode, the UART meets the half-duplex IrDA SIR (9.6/115.2 Kbps rate) protocol. In multi-drop bus mode, the UART meets the full-duplex MDB/ICP v2.0 protocol.

General-Purpose Counter (CNT)

The [General-Purpose Counter \(CNT\)](#) converts pulses from incremental position encoders into data that is representative of the actual position of the pulse. This conversion is done by integrating (counting) pulses on one or two inputs. Since integration provides relative position, some devices also feature a zero-position input (zero marker). The GP counter can use the zero position input feature to establish a reference point for verifying that the acquired position does not drift over time. In addition, the GP counter can use the incremental position information to determine speed, if the time intervals are measured.

Media Local Bus (MLB)

The [Media Local Bus \(MLB\)](#)[®] is an on-PCB or inter-chip communication bus, which allows an application to access the MOST network data. Media Local Bus supports all the MOST network data transport methods including synchronous stream data, asynchronous packet data, control message data and isochronous data. The MLB topology supports communication among the MLB controller and MLB devices, where the MLB controller is the interface between the MLB devices and the MOST network.

DAI/SRU Peripherals

The [Digital Audio Interface \(DAI\)](#) (DAIn) are comprised of groups of identical peripherals and their respective Signal Routing Units (SRUn). The SRU connects inputs and outputs of the DAI peripherals with each other and to the external pins. This configuration allows peripherals to be interconnected to accommodate a wide variety of systems without making external pin connections.

The [DAI Routing Capabilities](#) section provides an overview of the different routing capabilities for the DAI unit. An example for the eight SPORTs is shown in the following figure.

The following additional peripherals are connected using the DAI/SRU.

- Eight [Asynchronous Sample Rate Converter \(ASRC\)](#) blocks
- Two [Sony/Philips Digital Interface \(S/PDIF\)](#) transmit/receive blocks
- Four [Precision Clock Generators \(PCG\)](#)

System Accelerators ()

The following sections provide information about the high-performance acceleration engines on the processor.

FIR Accelerator (FIR)

[FIR Accelerator \(FIR\)](#) filters are frequently used in DSP applications. The FIR accelerator is a dedicated hardware interface used to perform filter processing to reduce the instruction processing load on the core. FIR filters are used in a wide array of applications including multi-rate processing with an interpolator or decimator.

IIR Accelerator (IIR)

The processor includes an [IIR Accelerator \(IIR\)](#) implemented in hardware that reduces the processing load on the core, freeing it up for other tasks.

Security and Protection (SPU/PKTE/PKIC/PKA/TRNG)

The following modules provide system safety and security.

System Protection Unit (SPU)

In a system with multiple system MMR masters, configurations of peripherals can be changed unintentionally leading to bad data or even system malfunctions. The peripherals are shared resources in the system. The [System Protection Unit \(SPU\)](#) restricts access to certain MMRs, similar to the functionality of a semaphore.

The SPU also protects peripherals based on security settings. It is part of the overall security infrastructure of the processor.

Security Packet Engine (PKTE)

The [Security Packet Engine \(PKTE\)](#) is a security packet engine designed to off-load the host processor to improve the speed of applications requiring cryptographic processing. The packet engine contains a set of modules for encryption and decryption, hashing, and pseudo-random number generation.

Public Key Accelerator (PKA)

The [Public Key Accelerator \(PKA\)](#) helps offload computationally-intensive operations commonly found in public key cryptography algorithms. The PKA also contains hardware logic to automatically zero out the PKA RAM buffer to clear out any information that is considered sensitive or secure.

Public Key Interrupt Controller (PKIC)

The [Public Key Interrupt Controller \(PKIC\)](#) is a common interrupt controller shared with the True Random Number Generator. The host processor configures the PKIC to generate interrupts when certain operations are complete or interrupts are caused by errors.

True Random Number Generator (TRNG)

The [True Random Number Generator \(TRNG\)](#) engine provides a true, non-deterministic, noise source for generating keys, Initialization Vectors (IVs), and other random number requirements. Other non-cryptographic purposes include statistical sampling, retry timers for communications protocols and noise generation.

System Debug (SCB/DBG/TAPC)

The System Debug and Trace Unit (DBG) is based on Arm CoreSight technology. CoreSight™ is a set of architecture specifications defining debug and trace architecture. The processor uses CoreSight infrastructure to provide industry standard debug and trace capabilities through the following modules.

- [Test Access Port Controller \(TAPC\)](#)

Additional debug resources are provided by the following modules.

- The [System Crossbars \(SCB\)](#) are the fundamental building blocks of a switch-fabric style for (on-chip) system bus interconnection. The SCBs connect system bus masters to system bus slaves, providing concurrent data transfer between multiple bus masters and multiple bus slaves. The SCB provides sustainable throughput for simultaneous transactions in the system with configurable Quality of Service for each type of transaction (traffic) as required. A hierarchical model, built from multiple SCBs, provides a power and area efficient system interconnect, which satisfies the performance and flexibility requirements of a specific system.

2 Clock Generation Unit (CGU)

The Clock Generation Unit (CGU) includes the phase locked loop (PLL) and the PLL control unit (PCU). The PLL generates a requester clock that runs at a frequency that is a multiple of the CLKIN input clock frequency. The PCU divides down the requester clock to generate various system clocks and synchronization signals.

CGU Features

The CGU module supports the following features:

- Provides smooth transitions from the current clock condition to a new condition with PLL logic and executes the changes to clocks due to register programming.
- Provides PLL and clock domain status reporting for event management.
- Supports the capability to bypass the PLL for power savings.
- Software controlled dynamic power management allows control of the core clock frequency (f_{CCLK}).
- Controls clock gating of the core and system clocks.

NOTE: For more information about processor-specific CGU features, see the processor data sheet.

CGU Functional Description

The CGU generates all on-chip clocks and synchronization signals based on the programmed PLL multiplication factor and dividers. The CGU provides the following functionality.

Change the PLL Clock Frequency

The CGU allows programs to change the PLL clock frequency by writing new values to bits in the control register. Any time the PLL relocks, the CGU aligns all core and system clocks.

Change Other Clock Frequencies

The CGU allows programs to change the CCLK_n, SYSCLK, SCLK_n, XCLK, and OCLK frequencies by writing values to the [CGU_DIV](#) register. Any time the clock frequency is changed, the OCLK, CCLK_n, SYSCLK, XCLK, and SCLK_n clocks exit the frequency change sequence aligned.

Perform Clock Alignment

The CGU can align all clocks by writing to the [CGU_DIV](#) register. This function aligns all PLL-based clocks.

For more information on these functions, see the [CGU Programming Model](#) section.

ADSP-21560_21561_21564_21568 CGU Register List

The clock generation unit (CGU) includes the phase locked loop (PLL) and the PLL control unit (PCU). The PLL generates a clock, running at a frequency that is a multiple of the CLKIN input clock's frequency. The CGU also generates all on-chip clocks and synchronization signals. The PCU permits application software control of the PLL's operation. A set of registers govern CGU operations. For more information on CGU functionality, see the CGU register descriptions.

Table 2-1: ADSP-21560_21561_21564_21568 CGU Register List

Name	Description
CGU_CCBF_DIS	Core Clock Buffer Disable Register
CGU_CCBF_STAT	Core Clock Buffer Status Register
CGU_CLKOUTSEL	CLKOUT Select Register
CGU_CTL	Control Register
CGU_DIV	Clocks Divisor Register
CGU_DIVEX	DIV Register Extension
CGU_PLLCTL	PLL Control Register
CGU_REVID	Revision ID Register
CGU_SCBF_DIS	System Clock Buffer Disable Register
CGU_SCBF_STAT	System Clock Buffer Status Register
CGU_STAT	Status Register

ADSP-21560_21561_21564_21568 CGU Interrupt List

Table 2-2: ADSP-21560_21561_21564_21568 CGU Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
1	CGU0_EVT	CGU0 Event	Edge	

ADSP-21560_21561_21564_21568 CGU Trigger List

Table 2-3: ADSP-21560_21561_21564_21568 CGU Trigger List Generators

Trigger ID	Name	Description	Sensitivity
1	CGU0_EVT	CGU0 Event	Edge

Table 2-4: ADSP-21560_21561_21564_21568 CGU Trigger List Receivers

Trigger ID	Name	Description	Sensitivity
None			

CGU Definitions

DPM

The Dynamic Power Management module (DPM) works with the CGU to provide flexible power dissipation modes for the processor.

PCU

The PLL control unit (PCU) in the CGU controls PLL operations. The MMR registers of the CGU are implemented in the PCU.

PLL

The phase-locked loop (PLL) operates within the CGU.

RCU

The reset control unit (RCU) provides input to the CGU to manage clocks during processor reset.

CDU

The clock distribution unit distributes the clocks from the CGU to different clock domains

CGU

The clock generation unit (CGU) is comprised of the PLL and PCU. The CGU generates the clocks listed in the *Clock Descriptions* table.

Table 2-5: Clock Descriptions

Clock	Description
CCLK0_0	CCLK0 derived from CGU0

Table 2-5: Clock Descriptions (Continued)

Clock	Description
SYSCLK_0	SYSCLK derived from CGU0
SCLK0_0	SCLK0 derived from CGU0
SCLK1_0	SCLK1 derived from CGU0
XCLK_0	XCLK derived from CGU0
OCLK_0	OCLK derived from CGU0

CGU PLL Block Diagram

The *CGU PLL Block Diagram* provides a top-level block diagram of the phase locked loop (PLL). The main blocks of the PLL are the phase/frequency detector (PFD), the charge pump, the loop filter and the voltage controlled oscillator (VCO). The VCO multiplies the SYS_CLKIN0 input to a higher frequency.

Additional configuration options are configured using the Clock Distribution Unit.

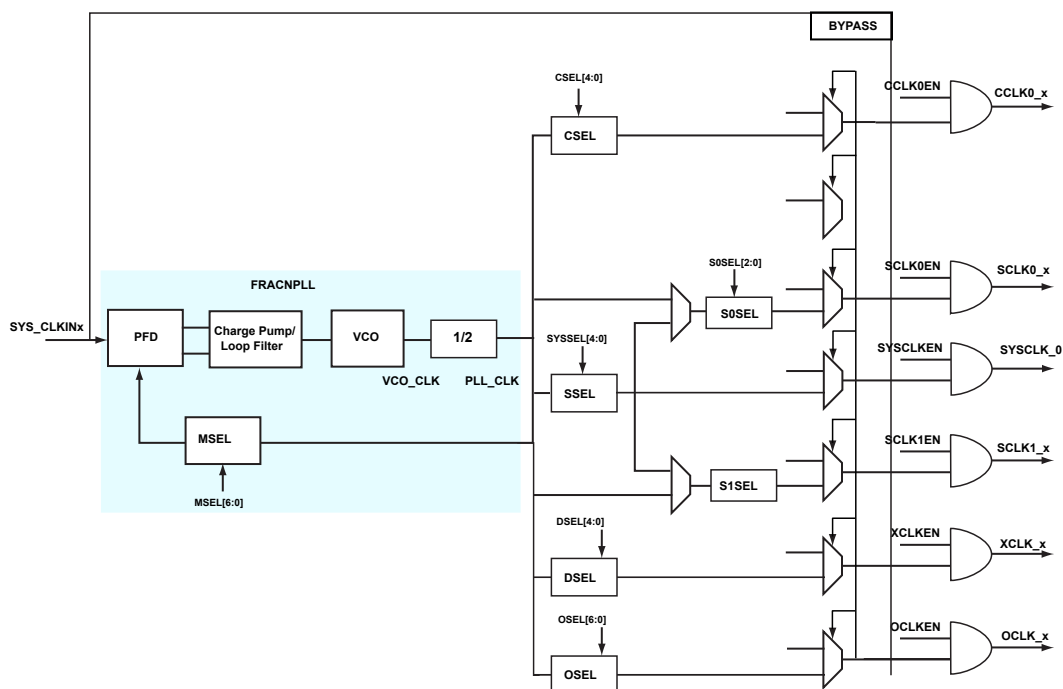


Figure 2-1: CGU PLL Block Diagram

CLKOUT Selections

The *SYS_CLKOUT Generation* figure is a conceptual representation of the CLKOUT module. Different clocks that originate from the CGU blocks are available on the SYS_CLKOUT output pin. The selection of the clock output on the SYS_CLKOUT pin is controlled by the CGU_CLKOUTSEL.CLKOUTSEL bit field. Most of the clock

frequencies are divided with a fixed divider value, for example CGU0_SYSCLK0 has a fixed divider of 8 for the CLKOUT pin.

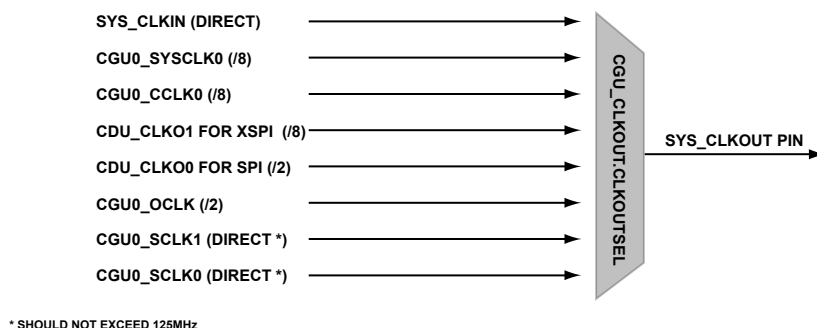


Figure 2-2: SYS_CLKOUT Generation

The BMODE pin affects the CLKOUT pins after a hardware reset is deasserted.

BMODE= 0 — When a hardware reset is deasserted, CGU0_OCLK is selected to come out of the CLKOUT pin. While programming all values of the CGU_CLKOUTSEL.CLKOUTSEL bit field is possible, changing it to 0 (SYS_CLKIN) is not possible until and unless a non-zero value is first programmed to the the CGU_CLKOUTSEL.CLKOUTSEL bit field.

BMODE = (non zero) — When a hardware reset is deasserted, SYS_CLKIN is selected by default.

CGU Operating Modes

The CGU does not have configurable operating modes, but CGU operations affect the operating modes of other modules. Some CGU operation issues that affect operation of other modules include the following:

- The PLL of the CGU operates in either normal mode (CGU clock divisors applied) or bypass mode (CGU PLL is bypassed and clock divisors ignored).
- The SCB uses the CGU for clock synchronization across clock domains. For more information, see [System Crossbars \(SCB\)](#).
- The DPM uses the CGU for clock management as power state transitions occur. For more information, see the [Dynamic Power Management \(DPM\)](#) chapter.
- The CGU uses clock gating control to obtain flexible low-power modes.

CGU Power-up Sequence

See the product data sheet for exact power-up requirements. The processor is configured to come up in clock bypass mode. The programs is required to configure full speed clocks and safety monitors. SYS_CLKIN0 or SYS_CLKIN1, and all supplies should be stable before the SYS_HWRST signal is deasserted.

CGU Event Control

The CGU generates an event or error for several different reasons. Events and errors are described in the following sections.

Events

After a frequency change, a CGU event indicates that the PLL has locked, and clocks are synchronized. If a core was idled while changing frequencies, the CGU can use an event interrupt to break the core idle. While in active mode, a CGU event indicates that the PLL has locked.

CGU Errors

A CGU error occurs under following conditions:

- A write access to the `CGU_DIV/CGU_DIVEX` registers trigger an alignment sequence while the PLL is locked and is still aligning the clocks.

The `CGU_STAT.WDIVERR` bit state indicates this error. If this error occurs, clear the `CGU_STAT.WDIVERR` bit and rewrite the desired values to the `CGU_DIV/CGU_DIVEX` registers.

- A change to the `CGU_DIV/CGU_DIVEX` registers occur while the PLL is locked and is still aligning the clocks.

The `CGU_STAT.WDIVERR` bit state indicates this error. If this error occurs, clear the `CGU_STAT.WDIVERR` bit and rewrite the desired values to the `CGU_DIV/CGU_DIVEX` registers.

- A write access to the bit field occurs or a write access to the `CGU_CTL.MSEL` bit field occurs while the PLL is locking.

The `CGU_STAT.WDFMSERR` bit state indicates this error. If this error occurs, wait until the PLL has finished locking, cleared the error, and rewritten the desired value change.

The CGU monitors changes to the following fields:

- SYCLK Divisor – `CGU_DIV.SYSSEL`

CGU Generated Bus Errors

The CGU generates a bus error when a read or write transaction to an unused address within the CGU address range is attempted. It also generates a bus error when a misaligned access is made to a CGU register. In addition to the bus error, the `CGU_STAT.ADDRERR` bit is set. If a write to a write-protected CGU register is attempted, the CGU generates a bus error. In addition, the `CGU_STAT.LWERR` bit is set.

CGU Programming Model

The programming model for the CGU involves the various mode configuration techniques.

Configuring CGU Modes

Use the following procedures to configure the clocks and PLL.

NOTE: The program needs to clear the `CGU_STAT.CLKSALGN` bit before changing clocks. The following sequence is executed once, inside the application, after coming out of reset.

```
*pREG_CGU0_PLLCTL |= BITM_CGU_PLLCTL_PLLBPCL; // come out of bypass and enter
Full ON
while( (pADI_CGU0 ->STAT & 0xF) != 0x5 ) { } // poll
// now clocks are running with hardware default divisors.
// now program can change frequencies If desired the program can put the PLL
again into bypass.
```

Changing the Clock Frequencies

Applications change clock frequencies in two ways. The first way is modifying the PLL multiplication value by writing to the `CGU_CTL` register and the second is modifying the clock dividers by writing to the `CGU_DIV` register. Both actions have different implications even if the frequencies of the final clock are the same. Write accesses to change the `CGU_CTL.MSEL` bit field while the PLL is locking set the `CGU_STAT.WDFMSERR` error bit. The `CGU_STAT.WDIVERR` error bit is set when one of following accesses is attempted while the PLL is locked, but still aligning the clocks:

- A write access to the `CGU_DIV/CGU_DIVEX` to trigger an alignment sequence.
- A write access to the `CGU_DIV/CGU_DIVEX` to change the `CGU_DIV.CSEL`/, `CGU_DIV.SYSSEL`, `CGU_DIV.S0SEL`, `CGU_DIV.S1SEL`, or `CGU_DIV.DSEL/CGU_DIVEX.S1SELEX` bits.

Read-after-write accesses to these registers return the new value, even if the frequency of the clock change is still in-progress.

Modifying the PLL multiplier requires the PLL to relock. Once the PLL locks, the CGU synchronizes the clocks. Changes to the `CGU_CTL.MSEL` bit field result in bypassing the PLL. By setting the `CGU_CTL.WFI` bit, programs force the PLL to wait for the core to return to the idle or reset state before the frequency changes. If necessary, clear the `CGU_DIV.UPDT` bit to avoid multiple clock alignment sequences. If the `CGU_DIV` register is not updated, the CGU uses the current values to determine the frequencies of the clock. It is the programs responsibility to guarantee that the new and `CGU_DIV/CGU_DIVEX` combinations are legal.

Changing the PLL Clock Frequency

To change the phase-locked loop clock (*PLLCLK*) frequency, write new values to the `CGU_CTL.MSEL` field. Any time the PLL relocks, all core and system clocks are aligned.

1. Read `CGU_STAT` register and verify that:
 - a. The `CGU_STAT.PLEN` bit =1 (PLL enabled)
 - b. The `CGU_STAT.PLOCK` bit =1 (PLL is not locking)

- c. The `CGU_STAT.CLKSALGN` bit =0 (clocks aligned)
2. Write the desired values to the clock divisor select fields of the `CGU_DIV/CGU_DIVEX` register with the `CGU_DIV.UPDT` bit =0.
3. Write the desired values to the `CGU_CTL.MSEL` field.
 - a. To change the PLL frequency while the core is idle, write to the `CGU_CTL` register with the `CGU_CTL.WFI` bit =1.
 - b. To change the PLL frequency while the core is active, write to the `CGU_CTL` register with the `CGU_CTL.WFI` bit =0.

This sequence performs these actions:

1. Updates the corresponding CGU registers.
2. Bypasses the PLL.
3. Makes the PLL lock to the new values in the `CGU_CTL.MSEL` field.
4. Changes the clock frequencies.
5. Exits the PLL bypass with all clocks aligned.

When exiting the PLL bypass state, a CGU event occurs.

The `CGU_STAT` register exits this sequence with the `CGU_STAT.PLEN` bit =1, the `CGU_STAT.PLOCK` bit =1, the `CGU_STAT.PLLBP` bit =0, and the `CGU_STAT.CLKSALGN` bit =0. Poll the `CGU_STAT.PLOCK` bit, `CGU_STAT.PLLBP` bit, and `CGU_STAT.CLKSALGN` bit to discover when the PLL is locked and the clocks are aligned.

Changing the frequency of the PLL is allowed while the PLL is bypassed. In this case the new PLLCLK frequency is not used until the PLL is no longer bypassed.

Changing the CCLKn or SYSCLK Frequency Without Modifying the PLLCLK Frequency

To change the clock frequencies, write new values to `CGU_DIV.CSEL` or `CGU_DIV.SYSSEL` bits. The frequency change occurs only when the PLL is not bypassed. Any time the CCLKn or SYSCLK clock frequencies are changed, they exit the frequency change sequence aligned.

1. Read the `CGU_STAT` register to verify that the `CGU_STAT.CLKSALGN` bit =0 (clocks aligned).
2. Write the desired `CGU_DIV.CSEL`, `CGU_DIV.SYSSEL`, and `CGU_DIV.OSEL` bitfield values with the `CGU_DIV.UPDT` bit = 1 and/or, write to the `CGU_DIVEX.S1SELEX` field. This write updates the `CGU_DIV` register, changes the SCLKn and SYSCLK frequencies, and aligns the clocks. When the clocks are aligned, a CGU event occurs.

The `CGU_STAT` register exits this sequence with the `CGU_STAT.CLKSALGN` bit =0. Poll the `CGU_STAT.CLKSALGN` bit to discover when the clocks are aligned. Any write attempt to change the

CGU_DIV.S0SEL or CGU_DIV.S1SEL bit fields while CGU_STAT.CLKSALGN bit =1 (clocks alignment in progress) triggers an MMR access bus error and the CGU_DIV register is not modified.

Also, any write attempt to change the CGU_DIVEX.S1SELEX field while the CGU_STAT.CLKSALGN bit =1 (clocks alignment in progress) triggers an MMR access bus error and the CGU_DIVEX register is not modified.

Programming the SYSCLK frequency to a higher value than CCLKn also triggers an MMR access bus error and the CGU_DIV/CGU_DIVEX register is not modified.

Writing to the CGU_DIV/CGU_DIVEX register is allowed while the processor is in active (PLL bypassed) mode. In this case the effect of the write is visible only after the transition to full-on (PLL not bypassed) mode.

Accessing the DDR memory while changing the SYSCLK frequency is not supported and can have unpredictable results.

Changing the OCLK Frequency

To change the OCLK clock frequency, write a new CGU_DIV.OSEL bit value. Any time the OCLK clock frequency is changed, the OCLK, CCLKn, SYSCLK, and SCLKn clocks exit the frequency change sequence aligned.

1. Read the CGU_STAT register to verify that the CGU_STAT.CLKSALGN bit =0 (clocks aligned).
2. Write the desired CGU_DIV.OSEL value with the CGU_DIV.UPDT bit =1. This write updates the CGU_DIV register, changes the OCLK frequency, and aligns all clocks except OCLK.

The CGU_STAT register exits this sequence with the CGU_STAT.CLKSALGN bit =0. Poll the CGU_STAT.CLKSALGN bit to discover when the clocks are aligned. Any write attempt to change the CGU_DIV.DSEL field while the CGU_STAT.CLKSALGN bit =1 (clock alignment in progress) triggers an MMR access bus error and the CGU_DIV/CGU_DIVEX is not modified. When the clocks are aligned, a CGU event occurs.

Writing to the CGU_DIV.OSEL bit field is allowed while the processor is in active (PLL bypassed) mode. In this case the effect of the write is visible only after the transition to full-on (PLL not bypassed) mode.

Selecting SCLK1_0 Source

SCLK1_0 sources the SPDIF receiver clock. This can be sourced from PLL clock or SYSCLK0_0. Programming the CGU_DIVEX enables PLL clock as source (bypassing SYSCLK0_0 that continues to source other peripherals). When CGU_DIVEX is enabled, the PLL clock is divided from CGU_DIVEX register value.

Changing SCLK1_0 Frequencies

Complete the following steps to change the SCLK0 or SCLK1 frequency.

1. Read the CGU_STAT register. Verify that CGU_STAT.CLKSALGN = 0 (Clocks aligned).

2. Read the [CGU_CTL](#) register. Verify that the `CGU_CTL.S1SELEXEN` bit is set as needed. (These bits can only be modified when the PLL is bypassed).
3. Read the [CGU_STAT](#) register. Verify that the PLL is not bypassed. (If it is bypassed, see [PLL Bypass and PLL Disable](#) topic.)
4. Read the [CGU_DIV](#) register. Verify that the `CGU_DIV.UPDT` bit is set.
5. Write the desired `CGU_DIVEX.S1SELEX` value to the [CGU_DIVEX](#) registers .

Aligning All Clocks

To align the clocks, write 1 to the `CGU_DIV.ALGN` bit. The frequency can also be changed, if necessary. The clocks aligned include:

- CCLK_n
- SYSCLK
- SCLK_n
- OCLK
- XCLK

1. Read the [CGU_STAT](#) register to verify that `CGU_STAT.CLKSALGN` bit =0 (clocks aligned).
2. Write 1 to the `CGU_DIV.ALGN` bit. All other fields can change.

ADDITIONAL INFORMATION: This write does not alter the [CGU_DIV/CGU_DIVEX](#) register unless one of the clock-select fields is modified. When the clocks are aligned, a CGU event occurs.

The [CGU_STAT](#) register exits this sequence with the `CGU_STAT.CLKSALGN` bit =0. Poll the `CGU_STAT.CLKSALGN` bit to discover when the clocks are aligned. Any write to the [CGU_DIV/CGU_DIVEX](#) register intended to align clocks or to change a clock select field while the `CGU_STAT.CLKSALGN` bit =1 (clocks alignment in progress) triggers an MMR access bus error. In this case, the [CGU_DIV/CGU_DIVEX](#) register is not modified.

Writing 1 to the `CGU_DIV.ALGN` bit has no effect while the processor is in active (PLL bypassed) mode.

Shutting Off CCLK_n from Another Controller

CCLK_n can be shut off to save power when it is not in use. In the ADSP-2156x processor, this controller is a memory DMA.

1. Disable interrupts to core n.
2. Set the bit to disable the interfaces of core n to:
 - a. Stop DMA accesses to its L1

- b. Stop memory accesses to core 0
 - c. Stop accesses to MMRs
- 3. Test the bit to detect when accesses to core n have been disabled and all the pending transactions have completed.
- 4. Set the `CGU_CCBF_DIS.CCBF0` bit to disable the CCLKn buffer.
- 5. Check the `CGU_CCBF_STAT.CCBF0` bit.

If the `CGU_CCBF_STAT.CCBF0` bit is set, continue.

Reenable CCLKn From Another Controller

- 1. Clear the `CGU_CCBF_DIS.CCBF0` to enable CCLKn.
- 2. Check the `CGU_CCBF_STAT.CCBF0` bit.
 - a. If the `CGU_CCBF_DIS.CCBF0` bit is cleared, continue
- 3. Clear the bit. The core deasserts its acknowledge signal in response to the `RCU_SYSRST0` signal. This operation clears the bit.

Valid Clock Multiplier Settings

Processor operations depend on valid settings in the `CGU_CTL` and `CGU_DIV` registers. These registers control the clock multiplier and divisor values. Set these registers such that the minimum and maximum clocks specified in the data sheet are not violated. All other clock specifications in the data sheet must also be adhered to for correct operation of the processor.

PLL Bypass and PLL Disable

Writing 1 to the `CGU_PLLCTL.PLLBPST` bit tells the PLL to apply `OSC_CLKIN` clock to CCLK, SYCLK, SCLK0, SCLK1, XCLK (PLL Bypass), and OCLK outputs. Writing 1 to the `CGU_PLLCTL.PLLBPCL` bit tells the PLL to exit its PLL Bypass state and make all output clocks align and transition to their programmed frequencies.

The PLL can be disabled by clearing the `CGU_PLLCTL.PLLEN` bit while in the bypass state. If necessary, clock buffers can be disabled.

CCLK0 clock can be disabled or enabled by writing 1 or 0 to the corresponding bit in the `CGU_CCBF_DIS` register.

To determine which core clock buffers were disabled since the last read, software reads the `CGU_CCBF_STAT` register. The SCLK0, SCLK1, XCLK and OCLK clocks can be disabled or enabled by writing 1 or 0 to the corresponding bit in the `CGU_SCBF_DIS` register. Software cannot disable SYCLK.

Processor Specific Information

The processor has one system crystal oscillator and two system CGU units to provide the clocks to the system. Both of the CGUs come up in bypass mode out of reset.

CGU0 is the main CGU which provides most of the clocks including CCLK0, SYSCLK (SYSCLK_0), SCLK0 (SCLK0_0), and SCLK1 (SCLK1_0) to the system buses, infrastructure, and most of the peripherals. The XCLK source can be selected between CGU0 and CGU1, through CDU0.

For details, refer to the [Clock Distribution Unit \(CDU\)](#).

NOTE: The frequency ratio of the core clock to SYSCLK is 2:1.

The frequency ratios of SYSCLK to SCLK0 are 6:1, 4:1 or 2:1.

The processor does not support any other frequency ratio.

ADSP-21560_21561_21564_21568 CGU Register Descriptions

Clock Generation Unit (CGU) contains the following registers.

Table 2-6: ADSP-21560_21561_21564_21568 CGU Register List

Name	Description
CGU_CCBF_DIS	Core Clock Buffer Disable Register
CGU_CCBF_STAT	Core Clock Buffer Status Register
CGU_CLKOUTSEL	CLKOUT Select Register
CGU_CTL	Control Register
CGU_DIV	Clocks Divisor Register
CGU_DIVEX	DIV Register Extension
CGU_PLLCTL	PLL Control Register
CGU_REVID	Revision ID Register
CGU_SCBF_DIS	System Clock Buffer Disable Register
CGU_SCBF_STAT	System Clock Buffer Status Register
CGU_STAT	Status Register

Core Clock Buffer Disable Register

The `CGU_CCBF_DIS` register controls each core's clock buffer to determine if the CCLK is enabled.

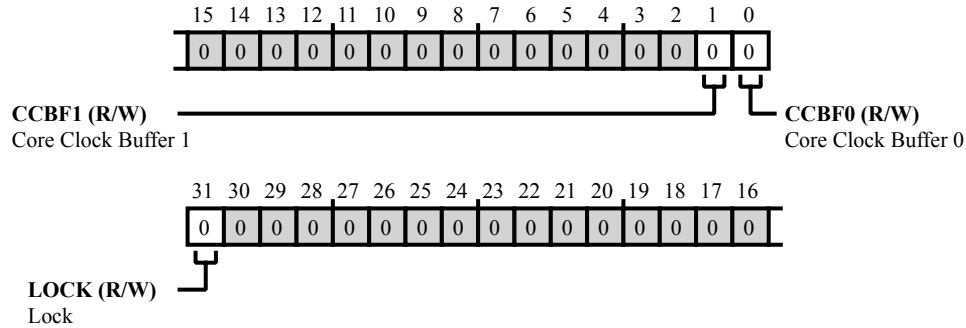


Figure 2-3: `CGU_CCBF_DIS` Register Diagram

Table 2-7: `CGU_CCBF_DIS` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock.
		If set (=1) the <code>CGU_CCBF_DIS</code> .LOCK bit locks the <code>CGU_CCBF_DIS</code> register.
		0 Unlock register
		1 Lock register
1 (R/W)	CCBF1	Core Clock Buffer 1. The <code>CGU_CCBF_DIS</code> .CCBF1 bit enables (=0) or disables (=1) CCLK1s buffer.
0 (R/W)	CCBF0	Core Clock Buffer 0.
		The <code>CGU_CCBF_DIS</code> .CCBF0 bit enables (=0) or disables (=1) CCLK0s buffer.
		0 Enable buffer
		1 Disable buffer

Core Clock Buffer Status Register

The `CGU_CCBF_STAT` register shows which core clock buffer(s) are disabled. For example clearing the `CGU_CCBF_DIS.CCBF0` bit clears the `CGU_CCBF_STAT.CCBF0` bit after a number of cycles. To guarantee that the correct value is read, this register should be read twice and the second result used.

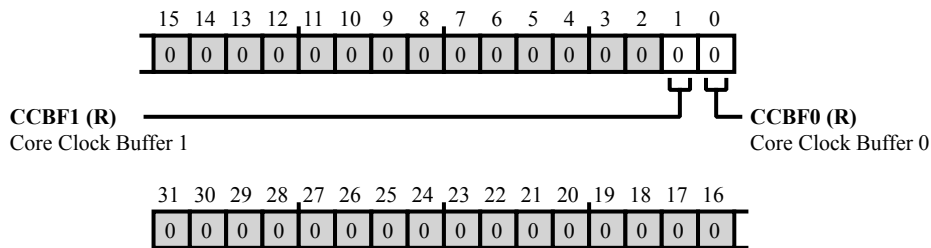


Figure 2-4: CGU_CCBF_STAT Register Diagram

Table 2-8: CGU_CCBF_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/NW)	CCBF1	Core Clock Buffer 1. The <code>CGU_CCBF_STAT.CCBF1</code> bit reports the status of the <code>CGU_CCBF_DIS.CCBF1</code> bit where 0 = enabled and 1 = disabled.
		0 Enabled
		1 Disabled
0 (R/NW)	CCBF0	Core Clock Buffer 0. The <code>CGU_CCBF_STAT.CCBF0</code> bit reports the status of the <code>CGU_CCBF_DIS.CCBF0</code> bit where 0 = enabled and 1 = disabled.
		0 Enabled
		1 Disabled

CLKOUT Select Register

The `CGU_CLKOUTSEL` selects the signal that the CGU drives through the CLKOUT multiplexer. Also, this register selects the divisor for the USBCLK output.

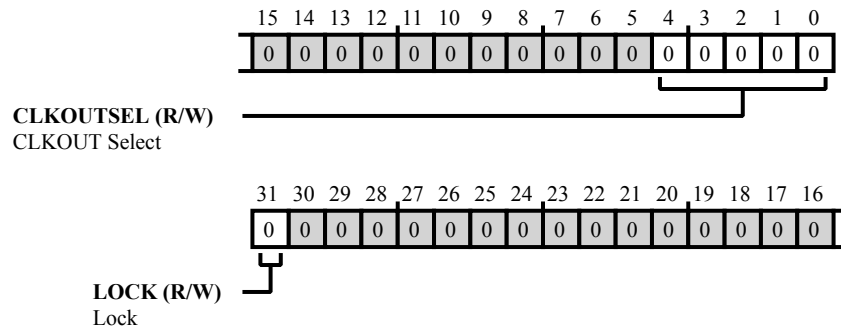


Figure 2-5: CGU_CLKOUTSEL Register Diagram

Table 2-9: CGU_CLKOUTSEL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock. If the global lock bit is set (<code>SPU_CTL.GLCK</code> bit =1) and the <code>CGU_CLKOUTSEL.LOCK</code> bit is set, the <code>CGU_CLKOUTSEL</code> register is read only (locked).
		0 Unlock
		1 Lock
4:0 (R/W)	CLKOUTSEL	CLKOUT Select. The <code>CGU_CLKOUTSEL.CLKOUTSEL</code> selects the signal that the CGU drives through the CLKOUT pin multiplexer.
		0 CLKIN0
		1 GND
		2 CGU0.SYSCLKO
		3 CGU0.CCLK0
		4 CDU.CLKO1 for XSPI
		5 CDU.CLKO0 for SPI
		6 Reserved
		7 Reserved
		8 Reserved
		9 CGU0.OCLK

Table 2-9: CGU_CLKOUTSEL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
		10	GND
		11	Reserved
		12	GND
		13	Reserved
		14	CGU0.SCLK1
		15	GND
		16	CGU0.SCLK0
		17	Reserved
		18	Reserved

Control Register

The `CGU_CTL` controls the clock generation divisors for `SYS_CLKIN` and the PLL. Read after write accesses to the `CGU_CTL` register returns the new value even if the clock's frequency change is still in progress.

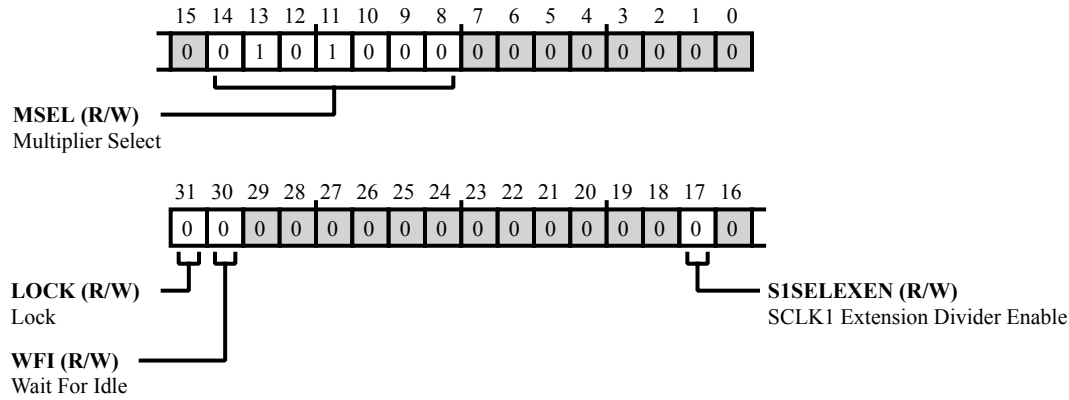


Figure 2-6: `CGU_CTL` Register Diagram

Table 2-10: `CGU_CTL` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock.
		If the global lock bit is set (<code>SPU_CTL.GLCK</code> bit =1) and the <code>CGU_CTL.LOCK</code> bit is set, the <code>CGU_CTL</code> register is read only (locked).
		0 Unlock
30 (R/W)	WFI	Wait For Idle.
		Modifying the PLL multiplier requires the PLL to re-lock and once the PLL locks, clocks have to be synchronized. Changes to the <code>CGU_CTL.MSEL</code> bit values results in bypassing the PLL.
		The <code>CGU_CTL.WFI</code> bit forces the PLL to wait for all processor cores to be in an idle or reset state before changing frequencies as a result of changes to the <code>CGU_CTL.MSEL</code> bits. Write accesses to the <code>CGU_CTL</code> to change the <code>CGU_CTL.MSEL</code> bit values while the PLL is locking sets the <code>CGU_STAT.WDFMSERR</code> bit.
		0 Update Immediately
		1 Wait for Idle

Table 2-10: CGU_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
17 (R/W)	S1SELEXEN	SCLK1 Extension Divider Enable. The CGU_CTL.S1SELEXEN directs the CGU to select S1SELEX instead of S1SEL. This bit can only be updated when the PLL is bypassed.
		0 Selects SYSCLK to SCLK1 Divider
		1 Selects PLLCLK to SCLK1 Divider
14:8 (R/W)	MSEL	Multiplier Select. The CGU_CTL.MSEL bit field selects the multiplier in the PLLCLK equation: $\text{PLLCLK frequency} = (\text{SYS_CLKIN frequency} * \text{MSEL})/2$ Where the value of MSEL is between 1 and 127. When the processor comes out of reset, the default multiplier value is 40.
		0 MSEL = 128
		1-127 MSEL = 1 to 127

Clocks Divisor Register

The `CGU_DIV` register controls clock divisors for core clocks, system clocks, external (off core) memory clocks, and output clock. Read after write accesses to the `CGU_DIV` register returns the new value even if the clock's frequency change is still in progress.

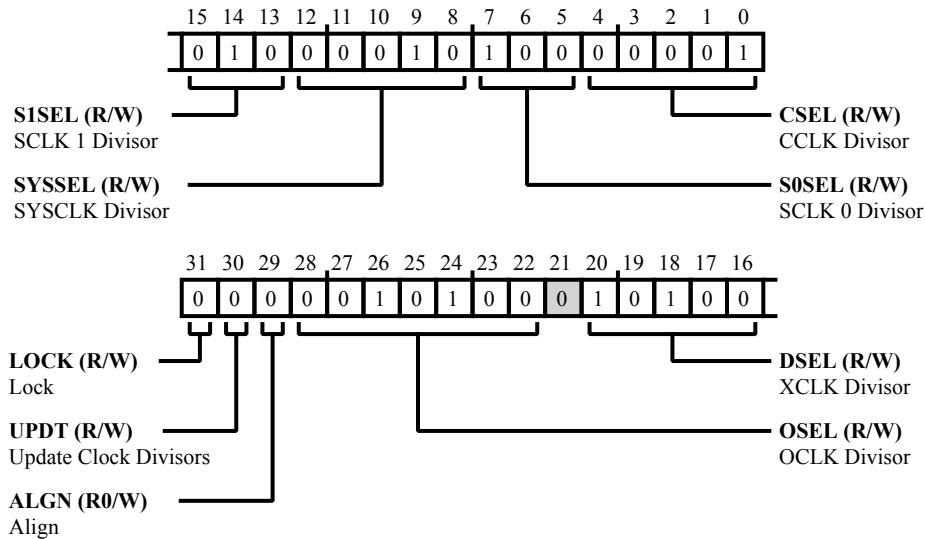


Figure 2-7: CGU_DIV Register Diagram

Table 2-11: CGU_DIV Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock.
		If the global lock bit is set (<code>SPU_CTL.GLCK</code> bit =1) and the <code>CGU_DIV.LOCK</code> bit is set, the <code>CGU0_DIV</code> and <code>CGU0_DIVEX</code> registers are read only (locked).
		0 Unlock
		1 Lock
30 (R/W)	UPDT	Update Clock Divisors.
		The <code>CGU_DIV.UPDT</code> controls whether the CGU drives new <code>CGU_DIV.CSEL</code> , <code>CGU_DIV.SYSSEL</code> , <code>CGU_DIV.S0SEL</code> , <code>CGU_DIV.S1SEL</code> , <code>CGU_DIV.DSEL</code> , and <code>CGU_DIV.OSEL</code> values to PLL after <code>CGU_DIV</code> register update.
		0 No PLL Update
		1 Drive Updated SEL Values to PLL

Table 2-11: CGU_DIV Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
29 (R0/W)	ALGN	Align. The CGU_DIV.ALGN directs the CGU to align the PLL-based clocks. The divisor selections (CGU_DIV.CSEL, CGU_DIV.SYSSEL, CGU_DIV.S0SEL, CGU_DIV.S1SEL, CGU_DIV.DSEL, and/or CGU_DIV.OSEL) do not have to change.
		0 No Action
		1 Align PLL Clocks
28:22 (R/W)	OSEL	OCLK Divisor. The CGU_DIV.OSEL selects the divisor in the OCLK equation: $\text{OCLK frequency} = (\text{SYS_CLKIN frequency} * \text{MSEL}) / (2 * \text{CGU_DIV.OSEL})$ Where the value of CGU_DIV.OSEL is between 1 and 127.
		0 OSEL = 128
		1-127 OSEL = 1 to 127
20:16 (R/W)	DSEL	XCLK Divisor. The CGU_DIV.DSEL selects the divisor in the XCLK equation: $\text{XCLK frequency} = (\text{SYS_CLKIN frequency} * \text{MSEL}) / (2 * \text{CGU_DIV.DSEL})$ Where the value of CGU_DIV.DSEL is between 1 and 31.
		0 DSEL = 32
		1-31 DSEL = 1 to 31
15:13 (R/W)	S1SEL	SCLK 1 Divisor. The CGU_DIV.S1SEL selects the divisor in the SCLK1 equation: $\text{SCLK1 frequency} = (\text{SYSCLK frequency}) / \text{CGU_DIV.S1SEL}$ Where the value of CGU_DIV.S1SEL is between 1 and 7.
		0 S1SEL = 8
		1-7 S1SEL = 1 to 7
12:8 (R/W)	SYSSEL	SYSCLK Divisor. The CGU_DIV.SYSSEL selects the divisor in the SYSCLK equation: $\text{SYSCLK frequency} = (\text{SYS_CLKIN frequency} * \text{MSEL}) / (2 * \text{CGU_DIV.SYSSEL})$ Where the value of CGU_DIV.SYSSEL is between 1 and 31.
		0 SYSSEL = 32
		1-31 SYSSEL = 1 to 31

Table 2-11: CGU_DIV Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
7:5 (R/W)	S0SEL	SCLK 0 Divisor. The <code>CGU_DIV.S0SEL</code> selects the divisor in the SCLK0 equation: $\text{SCLK0 frequency} = (\text{SYSCLK frequency}) / \text{CGU_DIV.S0SEL}$ Where the value of <code>CGU_DIV.S0SEL</code> is between 1 and 7.
		0 S0SEL = 8
		1-7 S0SEL = 1 to 7
4:0 (R/W)	CSEL	CCLK Divisor. The <code>CGU_DIV.CSEL</code> bit field selects the divisor in the CCLK equation: $\text{CCLK frequency} = (\text{SYS_CLKIN frequency} * \text{MSEL}) / (2 * \text{CGU_DIV.CSEL})$ Where the value of <code>CGU_DIV.CSEL</code> is between 1 and 31. When the processor comes out of reset, the default <code>CGU_DIV.CSEL</code> bit field value is 2.
		0 CSEL = 32
		1-31 CSEL= 1 to 31

DIV Register Extension

The `CGU_DIVEX` register contains bitfields for programming the divisor values of the system clock [SCLK0 and SCLK1] in extended divisor mode.

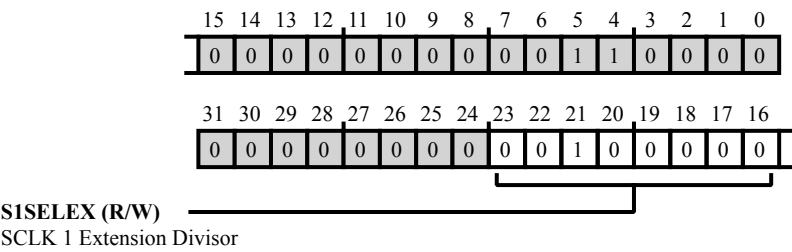


Figure 2-8: CGU_DIVEX Register Diagram

Table 2-12: CGU_DIVEX Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
23:16 (R/W)	S1SELEX	SCLK 1 Extension Divisor. The <code>CGU_DIVEX.S1SELEX</code> selects the divisor in the SCLK1EX equation: $\text{SCLK1EX frequency} = (\text{PLLCLK frequency}) / \text{CGU_DIVEX.S1SELEX}$ Where the value of <code>CGU_DIVEX.S1SELEX</code> is between 1 and 255.
		0 S1SELEX = 256
		1-255 S1SELEX = 1 to 255

PLL Control Register

The `CGU_PLLCTL` register contains bits that enable and disable the PLL as well as control its function.

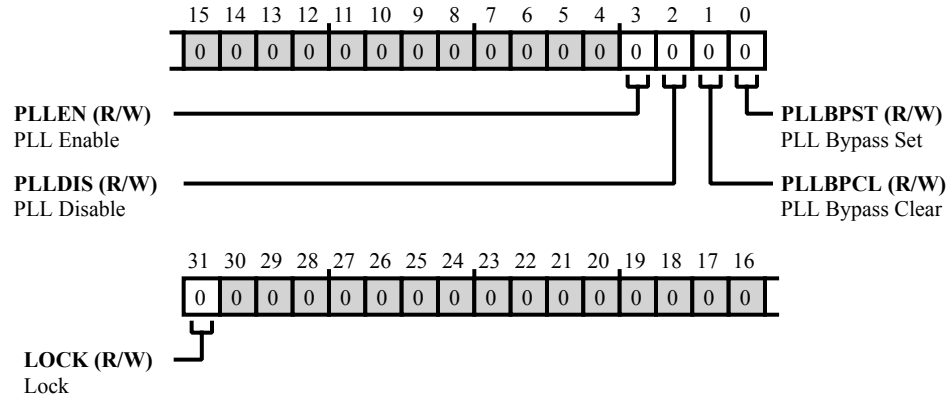


Figure 2-9: `CGU_PLLCTL` Register Diagram

Table 2-13: `CGU_PLLCTL` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock. Setting (=1) the <code>CGU_PLLCTL . LOCK</code> bit locks access to the <code>CGU_PLLCTL</code> register.
		0 Unlock register
		1 Lock register
3 (R/W)	PLLEN	PLL Enable. Setting (=1) the <code>CGU_PLLCTL . PLLEN</code> bit enables the PLL. Check the <code>CGU_STAT . PLLEN</code> bit to verify that the action is complete.
		0 No action
		1 Enable PLL
2 (R/W)	PLLDIS	PLL Disable. Setting (=1) the <code>CGU_PLLCTL . PLLDIS</code> bit disables the PLL. Check the <code>CGU_STAT . PLLEN</code> bit to verify that the action is complete.
		0 No action
		1 Disable PLL
1 (R/W)	PLLBPCL	PLL Bypass Clear. Setting (=1) the <code>CGU_PLLCTL . PLLBPCL</code> bit takes the PLL out of bypass mode. Check the <code>CGU_STAT . PLLBP</code> bit to verify that the action is complete.
		0 No action
		1 Exit bypass mode

Table 2-13: CGU_PLLCTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	PLLBPST	PLL Bypass Set. Setting (=1) the CGU_PLLCTL.PLLBPST bit bypasses the PLL and all the clocks run on CLKIN. Check the CGU_STAT.PLLBP bit to verify that the action is complete.
		0 Use PLL
		1 Bypass PLL

Revision ID Register

The CGU_REVID register reports the version of the CGU.

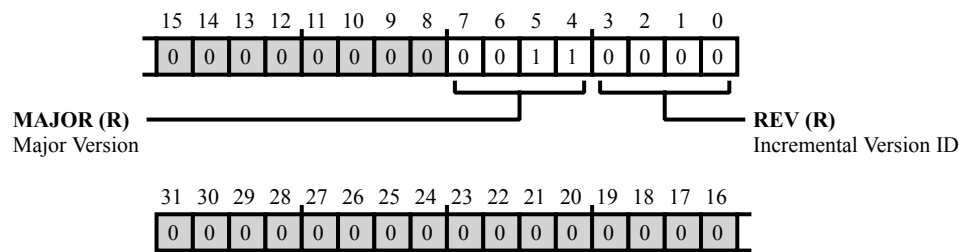


Figure 2-10: CGU_REVID Register Diagram

Table 2-14: CGU_REVID Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:4 (R/NW)	MAJOR	Major Version.
3:0 (R/NW)	REV	Incremental Version ID.

System Clock Buffer Disable Register

The `CGU_SCBF_DIS` register controls each system's clock buffer to determine if the SCLKn buffer is enabled.

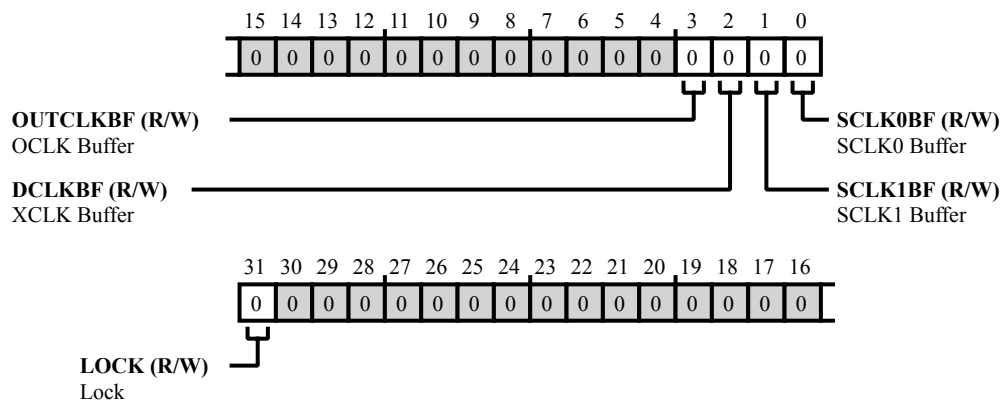


Figure 2-11: `CGU_SCBF_DIS` Register Diagram

Table 2-15: `CGU_SCBF_DIS` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock. The <code>CGU_SCBF_DIS</code> . <code>LOCK</code> bit allows writes to the <code>CGU_SCBF_DIS</code> register when cleared (=0) or blocks writes if set (=1) and the <code>SPU_CTL</code> . <code>GLCK</code> bit is set.
		0 Unlock register
		1 Lock register
3 (R/W)	OUTCLKBF	OCLK Buffer. The <code>CGU_SCBF_DIS</code> . <code>OUTCLKBF</code> bit enables (=0, default) or disables (=1) OCLKs buffer.
		0 Enable buffer
		1 Disable buffer
2 (R/W)	DCLKBF	XCLK Buffer. The <code>CGU_SCBF_DIS</code> . <code>DCLKBF</code> bit enables (=0, default) or disables (=1) XCLKs buffer.
		0 Enable buffer
		1 Disable buffer

Table 2-15: CGU_SCBF_DIS Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/W)	SCLK1BF	SCLK1 Buffer. The CGU_SCBF_DIS.SCLK1BF bit enables (=0, default) or disables (=1) SCLK1s buffer.
		0 Enable buffer
		1 Disable buffer
0 (R/W)	SCLK0BF	SCLK0 Buffer. The CGU_SCBF_DIS.SCLK0BF bit enables (=0, default) or disables (=1) SCLK0s buffer.
		0 Enable buffer
		1 Disable buffer

System Clock Buffer Status Register

The `CGU_SCBF_STAT` register shows which system clock buffer(s) are disabled. For example clearing the `CGU_CCBF_DIS.CCBF0` bit clears the `CGU_SCBF_STAT.SCLK0BF` bit after a number of cycles. To guarantee that the correct value is read, this register should be read twice and the second result used.

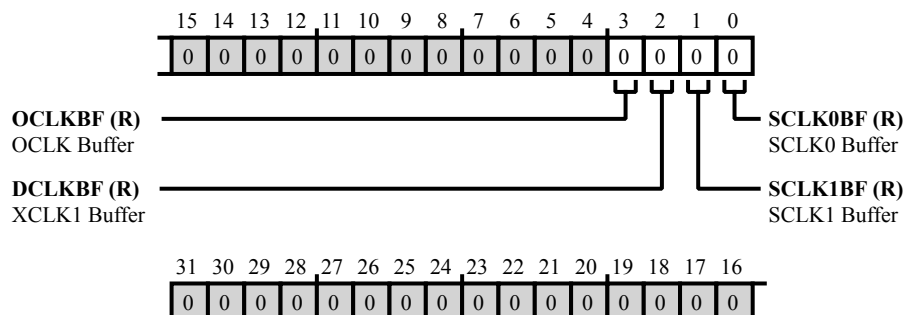


Figure 2-12: `CGU_SCBF_STAT` Register Diagram

Table 2-16: `CGU_SCBF_STAT` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3 (R/NW)	OCLKBF	OCLK Buffer. The <code>CGU_SCBF_STAT.OCLKBF</code> bit reports the status of the <code>CGU_SCBF_DIS.OUTCLKBF</code> bit where 0 = enabled and 1 = disabled.
		0 Enabled
		1 Disabled
2 (R/NW)	DCLKBF	XCLK1 Buffer. The <code>CGU_SCBF_STAT.DCLKBF</code> bit reports the status of the <code>CGU_SCBF_DIS.DCLKBF</code> bit where 0 = enabled and 1 = disabled.
		0 Enabled
		1 Disabled
1 (R/NW)	SCLK1BF	SCLK1 Buffer. The <code>CGU_SCBF_STAT.SCLK1BF</code> bit reports the status of the <code>CGU_SCBF_DIS.SCLK1BF</code> bit where 0 = enabled and 1 = disabled.
		0 Enabled
		1 Disabled

Table 2-16: CGU_SCBF_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/NW)	SCLK0BF	SCLK0 Buffer. The <code>CGU_SCBF_STAT.SCLK0BF</code> bit reports the status of the <code>CGU_SCBF_DIS.SCLK0BF</code> bit where 0 = enabled and 1 = disabled.
		0 Enabled
		1 Disabled

Status Register

The `CGU_STAT` register reflects the PLL status and errors detected during the PLL configuration. This register may be cleared asynchronously by a reset signal from the RCU module. All bits---except those defined as W1C (write-1-to-clear)---are read only.

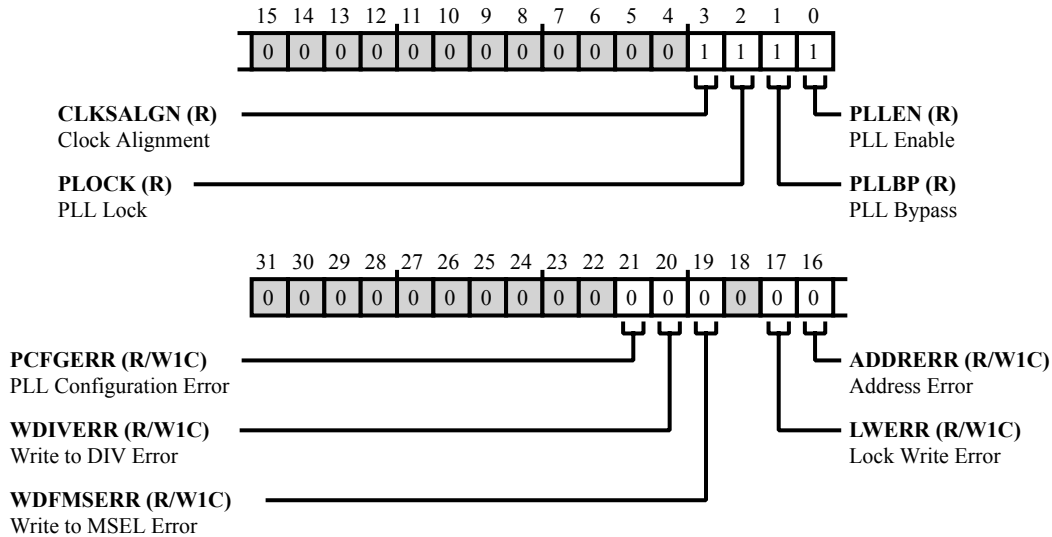


Figure 2-13: `CGU_STAT` Register Diagram

Table 2-17: `CGU_STAT` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
21 (R/W1C)	PCFGERR	PLL Configuration Error. If the <code>CGU_PLLCTL.PLLBPST</code> and the <code>CGU_PLLCTL.PLLBPCL</code> bits are set (=1) simultaneously or the <code>CGU_PLLCTL.PLLDIS</code> bit was set (=1) in full-on mode or while trying to enter full-on mode (<code>CGU_PLLCTL.PLLBPCL</code> =1), the <code>CGU_STAT.PCFGERR</code> bit triggers the bus error.
		0 No Error
		1 Configuration Error
20 (R/W1C)	WDIVERR	Write to DIV Error. The <code>CGU_STAT.WDIVERR</code> bit indicates a write access to the <code>CGU_DIV</code> register (to trigger an alignment sequence or to change the <code>CGU_DIV.CSEL</code> , <code>CGU_DIV.SYSSEL</code> , <code>CGU_DIV.S0SEL</code> , <code>CGU_DIV.S1SEL</code> , or <code>CGU_DIV.DSEL</code> bit values) while the PLL is locked, but still aligning the clocks. Read after write accesses to the <code>CGU_STAT</code> and <code>CGU_DIV</code> registers return the new value even if the clock frequency change is still in progress.
		0 No Error
		1 Write DIV Error

Table 2-17: CGU_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
19 (R/W1C)	WDFMSERR	Write to MSEL Error. The CGU_STAT.WDFMSERR bit indicates a write access to the CGU_CTL register to change the CGU_CTL.MSEL bit values while the PLL is locking.
		0 No Error
		1 Write MSEL Error
17 (R/W1C)	LWERR	Lock Write Error. The CGU_STAT.LWERR bit indicates an attempt to write to write-protected (locked) CGU registers. The CGU issues a bus error for this condition.
		0 No Error
		1 Lock Write Error
16 (R/W1C)	ADDRERR	Address Error. The CGU_STAT.ADDRERR bit indicates an attempt to make a read or write access to unimplemented addresses or accesses are non-aligned. The CGU issues a bus error for this condition.
		0 No Error
		1 Address Error
3 (R/NW)	CLKSALGN	Clock Alignment. The CGU_STAT.CLKSALGN bit indicates whether a clock alignment sequence is in progress. This bit is set when clocks alignment is required by changes to CGU_DIV.CSEL, CGU_DIV.S0SEL, CGU_DIV.S1SEL, CGU_DIV.DSEL, or CGU_DIV.OSEL. The CGU_STAT.CLKSALGN bit is cleared when clocks are aligned. Note that (after a PLL frequency change in active state) the CGU_STAT.CLKSALGN bit may indicate that clocks are not aligned even though the clocks are aligned (all clocks are aligned and running at CLKIN frequency).
		0 Clocks are Aligned
		1 Clocks not Aligned (alignment in progress)
2 (R/NW)	PLOCK	PLL Lock. The CGU_STAT.PLOCK bit indicates whether the PLL is locked. This bit is set when the PLL locks (PLL lock counter end-of-count). The CGU_STAT.PLOCK bit is cleared when requested PLL frequency change (for PLL reset, PLL disable-to-enable transition, or a change to the CGU_CTL.MSEL values) is in progress.
		0 PLL not Locked (PLL frequency change in progress)
		1 PLL Locked

Table 2-17: CGU_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/NW)	PLLBP	PLL Bypass. The CGU_STAT.PLLBP bit indicates whether the PLL is bypassed. The default value for the CGU_STAT.PLLBP bit is determined by the bypass strap pin.
		0 PLL not Bypassed
		1 PLL Bypassed
0 (R/NW)	PLLEN	PLL Enable. The CGU_STAT.PLLEN bit indicates whether the PLL is enabled.
		0 Disabled
		1 Enabled

3 Clock Distribution Unit (CDU)

The Clock Distribution Unit (CDU) consists of an array of multiplexors that select clocks originated from up to four different clock sources. These sources are different clocks that are generated from the Clock Generation Units (CGUs). The multiplexors are configured by software. All clocks multiplexors have up to four different sources. Unused input clocks are grounded internally and never selected. The output clock signal for each multiplexor is assigned to one or more destinations within the processor.

CDU Features

The CDU modules supports the following features:

- Generation of up to 3 output clocks
- Output clock buffers that can be disabled by software
- Each multiplexor has two input clocks
- Multiplexors that are configured by writing to configuration registers (`CDU_CFG[n]`)
- Clocks originated in CGU0 that are selected by default
- A CDU status register (`CDU_STAT`) that indicates a configuration change is in-progress

CDU Functional Description

The CDU functions as a set of software-configurable multiplexors that select clocks from different sources.

CDU Block Diagram

The *CGU/CDU Block Diagram* figure shows the functional blocks within the CDU. As shown in the figure, the CDU takes different clocks generated by the CGU blocks and provides flexibility to route any clock from possible options to the output CDU clocks. The output clocks from the CDU are connected to specific targets such as the xSPI module. All of the targets are clocked through the CDU output clocks rather than being directly clocked by CGU clocks.

This configuration provides the flexibility to meet the specific clock requirements of the different modules in the system (such as the core, xSPI, or SPI module) without compromising the clocking of other modules.

- CGU has its own set of clock multipliers and dividers, providing a greater number of orthogonal clocks.
- CGU is clocked from a single CLKIN source.

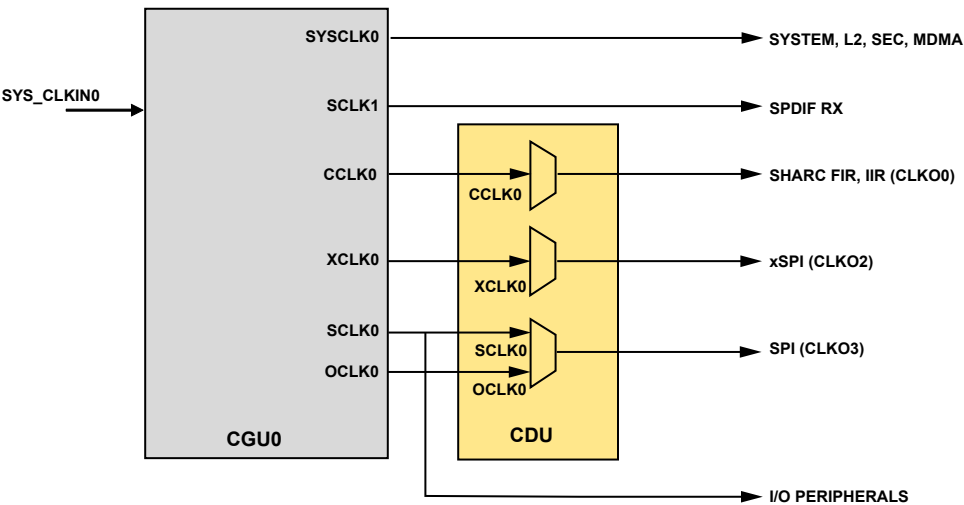


Figure 3-1: CGU/CDU Block Diagram

The xSPI is clocked from the XCLK from CGU0. This configuration provides the flexibility to program the DDR with frequencies orthogonal to the core clock frequencies. (Orthogonal frequencies have independent signal states. For example, frequency and amplitude modulation signals on the same carrier can be detected without interfering with each other).

CDU Definitions

The *Clock Descriptions* table provides a brief description of the clocks supported by the processor.

Table 3-1: Clock Descriptions

Clock	Description
CCLK0_0	CCLK0 derived from CGU0
SYSClk_0	SYSClk derived from CGU0
SCLK0_0	SCLK0 derived from CGU0
SCLK1_0	SCLK1 derived from CGU0
XCLK_0	XCLK derived from CGU0
OCLK_0	OCLK derived from CGU0
CDU_CLKOn	Clocks that come out of the CDU module and go to different blocks

ADSP-21560_21561_21564_21568 CDU Register List

Clock Distribution Unit

Table 3-2: ADSP-21560_21561_21564_21568 CDU Register List

Name	Description
CDU_CFG[n]	CDU Configuration
CDU_CLKINSEL	CLKIN Select
CDU_REVID	CDU Revision ID
CDU_STAT	CDU Status

CDU Clock Configuration Options

The *CDU Targets* table provides information about clock source and destination options.

Table 3-3: CDU Targets

CDU0 Input		CDU0 Output	Target
IN0_CLKOn	IN1_CLKOn		
CCLK_0	N/A	CLKO0	DSP0
XCLK_0	N/A	CLKO2	XSPI
SCLK_0	OCLK_0	CLKO3	SPI

CDU Programming Model

The [CDU_CFG\[n\]](#) registers are a system configuration resource. These registers are accessed by a system configuration routine that also handles the configuration of other system modules. Writes to a [CDU_CFG\[n\]](#) register must occur when there is not a CDU_CLKOn configuration change in progress. If a CGUn inputs are selected, that CGUn configuration must be completed first.

Changing the PLL and Clock Frequency

- Read the [CGU_STAT](#) register. Verify that:
 - `CGU_STAT.PLLEN = 1` (enabled)
 - `CGU_STAT.PLOCK = 1` (PLL is not locking)
 - `CGU_STAT.CLKSALGN = 0` (clocks aligned)
- Write the desired values into the `CGU_DIV.CSEL`, `CGU_DIV.S0SEL`, `CGU_DIV.SYSSEL`, `CGU_DIV.S1SEL`, `CGU_DIV.DSEL` and `CGU_DIV.OSEL` bits with the `CGU_DIV.UPDT` bit cleared (= 0).
- Write the desired values to the and `CGU_CTL.MSEL` bits.
 - To change the PLL frequency while all cores are idle, set the `CGU_CTL.WFI` bit (=1).
 - To change the PLL frequency while the cores are active, clear the `CGU_CTL.WFI` bit (=0).

4. Read the `CGU_STAT` register. Verify that:
 - `CGU_STAT.PLEN` = 1 (enabled)
 - `CGU_STAT.PLOCK` = 1 (not locking)
 - `CGU_STAT.CLKSALGN` = 0 (clocks aligned)
5. If clocks switch from CGUm clocks to CGUn input clocks, read the `CGU_SCBF_STAT` and `CGU_CCBF_STAT` registers. The `CGU_CCBF_STAT[1:0]` bit field corresponds to the CCLK1 and CCLK0 clock buffers, respectively. The `CGU_SCBF_STAT[3:0]` bit field corresponds to the OUTCLK, XCLK, SCLK1, and SCLK0 clock buffers, respectively.
6. Read the `CDU_STAT` register. Verify that the `CDU_STAT.CLK00` = 0 (no CLKOn configuration change in progress).
7. Write to the `CDU_CLKINSEL` register to select the CGU's CLKIN input clock.
8. Write to the `CDU_CFG[n].SEL` bit to select the clock source (the `CDU_CFG[n].EN` bit should =1.)
9. Read the `CDU_CFG[n]` register. Verify that the `CDU_CLKINSEL` has the programmed value.

CLKOn is reconfigured.

Changing the Clock Frequency

Use the following procedure to change a clock frequency.

1. Read the `CGU_STAT` register to verify that the `CGU_STAT.CLKSALGN` bit =0 (clocks aligned).
2. Write the desired values into the `CGU_DIV.CSEL`, `CGU_DIV.S0SEL`, `CGU_DIV.SYSSEL`, `CGU_DIV.S1SEL`, `CGU_DIV.DSEL` and `CGU_DIV.OSEL` bits with the `CGU_DIV.UPDT` bit = 1.
3. Read the `CGU_DIV` registers to verify that the `CGU_DIV.CSEL`, `CGU_DIV.S0SEL`, `CGU_DIV.SYSSEL`, `CGU_DIV.S1SEL`, `CGU_DIV.DSEL` and `CGU_DIV.OSEL` bit values are correct.
4. Read the `CGU_STAT` register to verify that the `CGU_STAT.CLKSALGN` = 0 (clocks aligned).
5. If clocks switch from CGUm clocks to CGUn input clocks, read the `CGU_SCBF_STAT` and `CGU_CCBF_STAT` registers. The `CGU_CCBF_STAT[1:0]` bit field corresponds to the CCLK1 and CCLK0 clock buffers, respectively. The `CGU_SCBF_STAT[3:0]` bit field corresponds to the OUTCLK, DCLK, SCLK1, and SCLK0 clock buffers, respectively.
6. Read the `CDU_STAT` register.
7. Write to the `CDU_CFG[n].SEL` bit to select the clock source (`CDU_CFG[n].EN` = 1).
8. Read the `CDU_CFG[n]` register. Verify that the `CDU_CLKINSEL` has the programmed value.
9. Verify that the `CDU_STAT.CLK00` = 0.

ADSP-21560_21561_21564_21568 CDU Register Descriptions

Clock Distribution Unit (CDU) contains the following registers.

Table 3-4: ADSP-21560_21561_21564_21568 CDU Register List

Name	Description
CDU_CFG[n]	CDU Configuration
CDU_CLKINSEL	CLKIN Select
CDU_REVID	CDU Revision ID
CDU_STAT	CDU Status

CDU Configuration

The `CDU_CFG[n]` registers control the configuration of the clock multiplexors. `CDU0_CFG[n]` corresponds to output clock `CDU_CLKO[n]`.

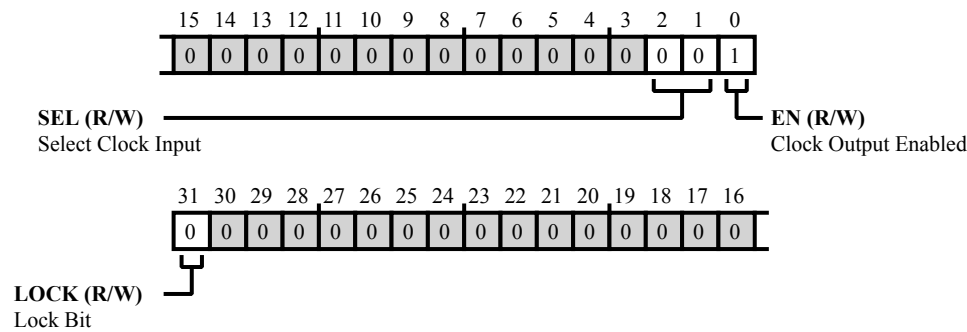


Figure 3-2: `CDU_CFG[n]` Register Diagram

Table 3-5: `CDU_CFG[n]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock Bit.
2:1 (R/W)	SEL	Select Clock Input.
		0 IN0_CLKOn Selected
		1 IN1_CLKOn Selected
		2 IN2_CLKOn Selected
		3 IN3_CLKOn Selected
0 (R/W)	EN	Clock Output Enabled. The <code>CDU_CFG[n]</code> .EN bit enables clock output.

CLKIN Select

The `CDU_CLKINSEL` register controls the configuration of the CLKIN multiplexors. One bit is assigned to each CGU in the system. This bit selects either CLKIN0 or CLKINn CGUn inputs.

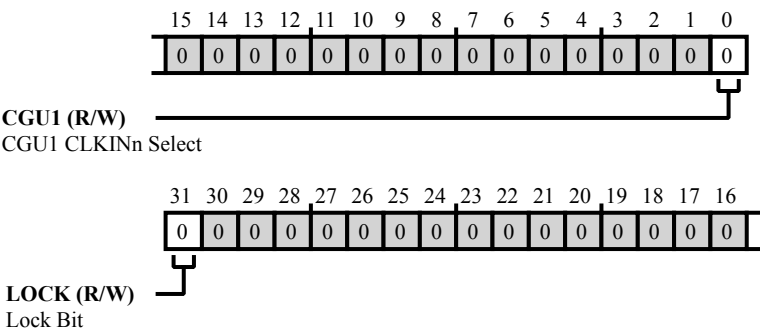


Figure 3-3: CDU_CLKINSEL Register Diagram

Table 3-6: CDU_CLKINSEL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock Bit.
0 (R/W)	CGU1	CGU1 CLKINn Select. The <code>CDU_CLKINSEL.CGU1</code> bit drives <code>CDU_CLKIN_SEL[0]</code> to CGU1.
		0 Selects CLKIN0
		1 Selects CLKIN1

CDU Revision ID

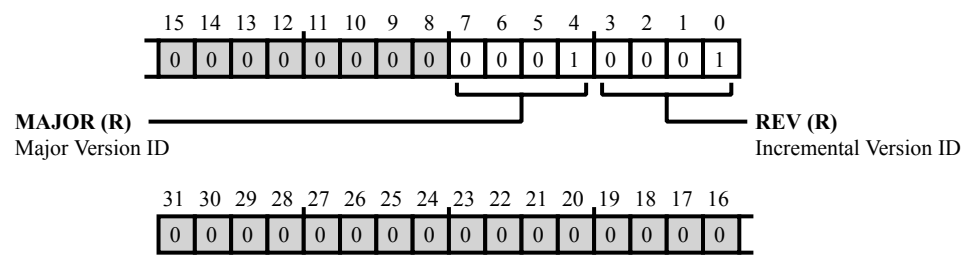


Figure 3-4: CDU_REVID Register Diagram

Table 3-7: CDU_REVID Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:4 (R/NW)	MAJOR	Major Version ID.
3:0 (R/NW)	REV	Incremental Version ID.

CDU Status

The `CDU_STAT` register reflects the status a change in the configuration of the clock muxes.

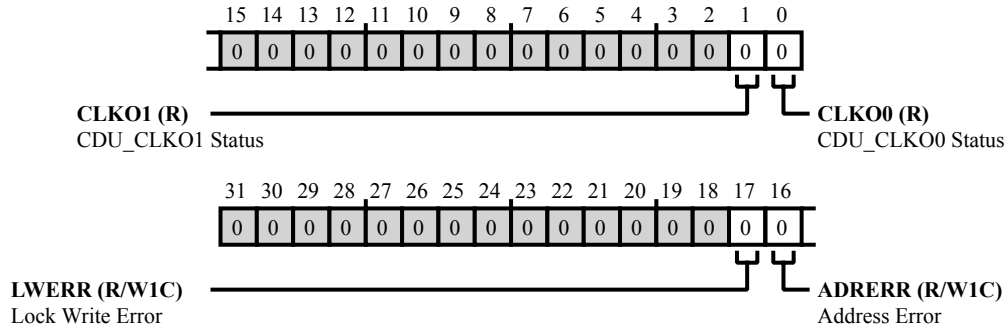


Figure 3-5: `CDU_STAT` Register Diagram

Table 3-8: `CDU_STAT` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
17 (R/W1C)	LWERR	Lock Write Error. The <code>CDU_STAT.LWERR</code> bit indicates a lock error where write transactions try to access write protected registers.
		0 No Lock Write Error
		1 Lock Write Error
16 (R/W1C)	ADRERR	Address Error. The <code>CDU_STAT.ADRERR</code> bit indicates an address error where read or write transactions try to access unimplemented addresses or accesses are non-aligned.
		0 No Address Error
		1 Address Error
1 (R/NW)	CLKO1	CDU_CLKO1 Status.
		0 No Configuration Change in Progress
		1 Configuration Change in Progress
0 (R/NW)	CLKO0	CDU_CLKO0 Status.
		0 No Configuration Change in Progress
		1 Configuration Change in Progress

4 Dynamic Power Management (DPM)

The dynamic power management (DPM) unit of the processor controls transitions between different power-saving modes.

DPM provides facility for shutting down clocks for various peripherals. Shutting down peripheral's clocks is considered a power saving feature. To support peripheral clock gating functionality, [DPM_PER_DIS0](#) and [DPM_PER_DIS1](#) are available in the DPM module. The bits in these registers need to be programmed to shut off the clocks to peripherals that are not used in an application to save clock switching power.

DPM Features

The DPM allows programs to control the power mode of the processor as follows.

- Permits operation of multiple, external wake-up sources

DPM Functional Description

The DPM can be programmed to transition between power modes.

ADSP-21560_21561_21564_21568 DPM Register List

A set of registers govern DPM operations. For more information on DPM functionality, see the DPM register descriptions.

Table 4-1: ADSP-21560_21561_21564_21568 DPM Register List

Name	Description
DPM_CTL	Control Register
DPM_PER_DIS0	Peripherals Disable Register 0
DPM_PER_DIS1	Peripherals Disable Register 1
DPM_REVID	Revision ID
DPM_STAT	Status Register

DPM Definitions

To make the best use of the DPM, it is useful to understand the following terms.

CGU

Acronym for the clock generation unit (CGU), which is comprised of the PLL and PCU

DPM

Acronym for the dynamic power management (DPM) controller.

Full-on mode

The normal operating mode in which all clock domains are derived from the PLL.

PCU

Acronym for the PLL control unit (PCU).

PLL

Acronym for the phase-locked loop (PLL).

RCU

Acronym for the reset control unit (RCU).

DPM Operating Modes

The DPM includes several operating modes. The modes are:

- Reset
- Full-on

Reset State

Reset is the initial state of the processor and is the result of a hardware or software triggered event. The DPM itself does not trigger entering reset. The external `SYS_HWRST` pin or the RCU triggers entering reset. The DPM responds to reset by transitioning to its default state.

From Reset, the DPM always transitions to PLL Bypassed state.

Full-on Mode

Full-on mode is the default state of the DPM after Reset.

In full-on mode, the processor can reach its maximum clock rate and power dissipation can be at its highest.

DPM Event Control

The DPM event is triggered when an enabled wake-up is asserted. The DPM generates bus errors when a misaligned access to a register occurs. It also generates errors when an attempt is made to access unused DPM address space or a write-protected register.

DPM Events

The DPM event interrupt is triggered when any bit in the [DPM_STAT](#) register is set, indicating that an enabled wake-up was asserted. The DPM event interrupt stays active until the user clears any bits that are set in the [DPM_STAT](#) register.

DPM Errors

The DPM generates a bus error when a read or write transaction is attempted to an unused address within the DPM address range. It also generates a bus error when a misaligned access is made to a DPM register. In addition to the bus error, the DPM sets the [DPM_STAT.ADDRERR](#) bit.

If a write to a write-protected DPM register is attempted, the DPM generates a bus error. In addition, the DPM sets the [DPM_STAT.LWERR](#) bit.

DPM Programming Model

The *DPM_PER_DIS0 Register Mapping* and *DPM_PER_DIS1 Register Mapping* tables show the module clocks and the corresponding peripheral. The [DPM_PER_DIS0](#) and [DPM_PER_DIS1](#) registers are used to shut off the clock to each peripheral if it is not required by the application.

Table 4-2: DPM0_PER_DIS0 Register Mapping

Peripheral Name	Gated Module Clocks	Type of sync on PER-DIS bit	Effect latency (In cycles of module clocks)	DPM0_PER_DISn bit
FIR0/FFT	cclk	Not Required	1	0
IIR0	cclk	Not Required	1	1
DAI0	sclk0	Single Flop	2	5
	sclk1	Single Flop	2	
	sysclk	Not Required	1	
DAI1	sclk0	Single Flop	2	6
	sclk1	Single Flop	2	
	sysclk	Not Required	1	

Table 4-2: DPM0_PER_DIS0 Register Mapping (Continued)

Peripheral Name	Gated Module Clocks	Type of sync on PER-DIS bit	Effect latency (In cycles of module clocks)	DPM0_PER_DISn bit
MLB0	sysclk	Not Required	1	9
EMDMA0	sysclk	Not Required	1	13
EMDMA1	sysclk	Not Required	1	14
CRYPTO ACCELERATOR-0 (EIP-150/PKP)	sysclk	Not Required	1	16
CRYPTO ACCELERATOR-1 (EIP-93/SPE)	sysclk	Not Required	1	17
SMPU-SPIF	sysclk	Not Required	1	19
SMPU-L2CTL-CL2-0	sysclk	Not Required	1	20
SMPU-L2CTL-DL2-0	sysclk	Not Required	1	21
SMPU-L2CTL1	sysclk	Not Required	1	27

Table 4-3: DPM0_PER_DIS1 Register Mapping

Peripheral Name	Gated Module Clocks	Type of sync on PER-DIS bit	Effect latency (In cycles of module clocks)	DPM0_PER_DISn bit
TWI0	sclk0	Single Flop	2	0
TWI1	sclk0	Single Flop	2	1
TWI2	sclk0	Single Flop	2	2
TWI3	sclk0	Single Flop	2	3
xSPI	sysclk	Not Required	1	6

ADSP-21560_21561_21564_21568 DPM Register Descriptions

Dynamic Power Management (DPM) contains the following registers.

Table 4-4: ADSP-21560_21561_21564_21568 DPM Register List

Name	Description
DPM_CTL	Control Register
DPM_PER_DIS0	Peripherals Disable Register 0
DPM_PER_DIS1	Peripherals Disable Register 1
DPM_REVID	Revision ID

Table 4-4: ADSP-21560_21561_21564_21568 DPM Register List (Continued)

Name	Description
DPM_STAT	Status Register

Control Register

The `DPM_CTL` register controls sleep modes selections and PLL operations of the DPM. A write protect feature permits locking out changes to this register.

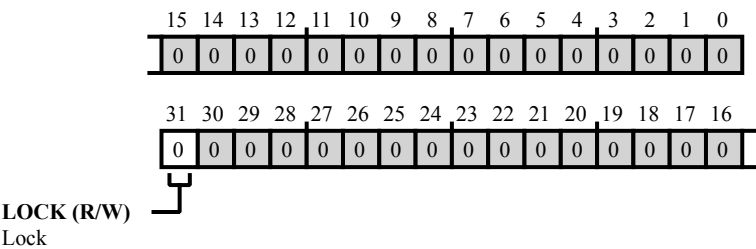


Figure 4-1: DPM_CTL Register Diagram

Table 4-5: DPM_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock. If the global lock bit is set (<code>SPU_CTL.GLCK</code> bit =1) and the <code>DPM_CTL.LOCK</code> bit is set, the <code>DPM_CTL</code> register is read only (locked).
		0 Unlock
		1 Lock

Peripherals Disable Register 0

The `DPM_PER_DIS0` register is used to shut off the clocks to peripherals that are not used in an application in order to save clock switching power.

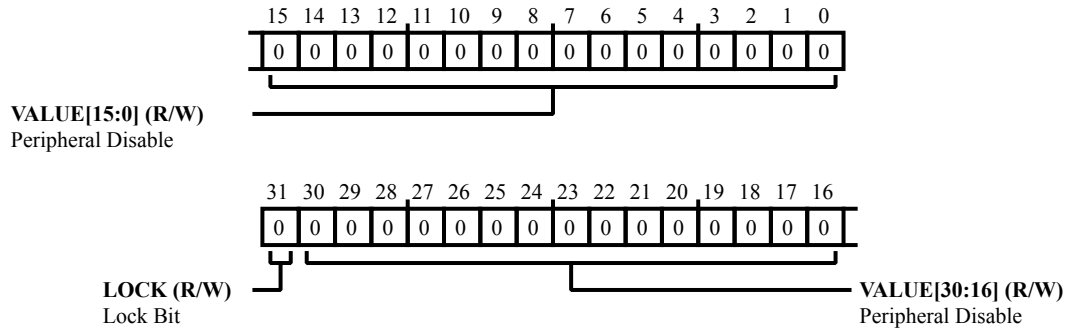


Figure 4-2: DPM_PER_DIS0 Register Diagram

Table 4-6: DPM_PER_DIS0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock Bit.
30:0 (R/W)	VALUE	Peripheral Disable. The <code>DPM_PER_DIS0.VALUE</code> bits are used to shut of clocks in individual peripherals.
		0 FIR0
		1 IIR0
		2 Reserved
		3 Reserved
		4 Reserved
		5 DAI0
		6 DAI1
		7 Reserved
		8 Reserved
		9 MLB0
		10 Reserved
		11 Reserved
		12 Reserved
		13 EMDMA0

Table 4-6: DPM_PER_DIS0 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
		14	EMDMA1
		15	Reserved
		16	CRYPTO ACCELERATOR- 0 (EIP-150/PKP)
		17	CRYPTO ACCELERATOR- 1 (EIP-93/SPE)
		18	Reserved
		19	SMPU-SPIF(SPI2)
		20	SMPU-L2CTL-CL2-0
		21	SMPU-L2CTL-DL2-0
		22	Reserved
		23	Reserved
		24	Reserved
		25	Reserved
		26	Reserved
		27	SMPU-L2CTL1
		28	Reserved
		29	Reserved
		30	Reserved

Peripherals Disable Register 1

The `DPM_PER_DIS1` register is used to shut off the clocks to peripherals that are not used in an application to save clock switching power.

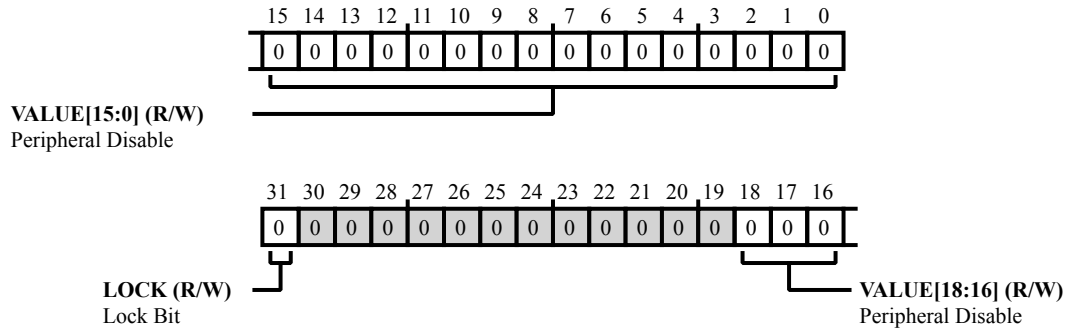


Figure 4-3: DPM_PER_DIS1 Register Diagram

Table 4-7: DPM_PER_DIS1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock Bit.
18:0 (R/W)	VALUE	Peripheral Disable. The <code>DPM_PER_DIS1.VALUE</code> bits are used to shut of clocks in individual peripherals.
		0 TWI0
		1 TWI1
		2 TWI2
		3 TWI3
		4 Reserved
		5 Reserved
		6 xSPI_SYSCLK
		7 Reserved
		8 Reserved
		9 Reserved
		10 Reserved
		11 Reserved
		12 Reserved
		13 Reserved

Table 4-7: DPM_PER_DIS1 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
		14	Reserved
		15	Reserved
		16	Reserved
		17	Reserved
		18	LPDDR4

Revision ID

The `DPM_REVID` register provides the revision of the DPM module.

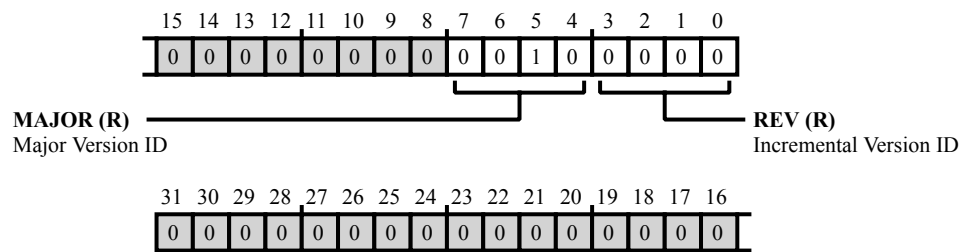


Figure 4-4: DPM_REVID Register Diagram

Table 4-8: DPM_REVID Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:4 (R/NW)	MAJOR	Major Version ID.
3:0 (R/NW)	REV	Incremental Version ID.

Status Register

The `DPM_STAT` register contains bits that report the state of the module and various errors.

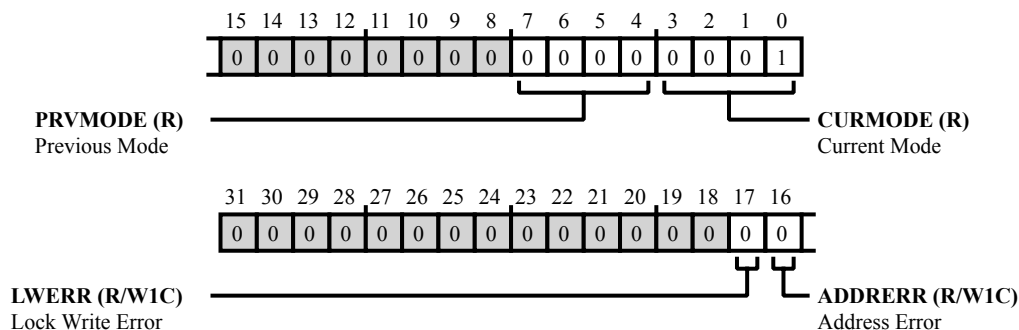


Figure 4-5: DPM_STAT Register Diagram

Table 4-9: DPM_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
17 (R/W1C)	LWERR	Lock Write Error. The <code>DPM_STAT.LWERR</code> bit indicates that a write transaction attempted an access to a write protected register. Triggers the <code>DPMLV_PSLVERR</code> interrupt.
		0 Inactive
		1 Active
16 (R/W1C)	ADDRERR	Address Error. The <code>DPM_STAT.ADDRERR</code> bit indicates that a read or write transaction attempted an access to an unimplemented address or a write transaction attempted an access to a read only register or accesses are non aligned. Triggers the <code>DPMLV_PSLVERR</code> interrupt.
		0 Inactive
		1 Active
7:4 (R/NW)	PRVMODE	Previous Mode. The <code>DPM_STAT.PRVMODE</code> bit field indicates the previous mode of the module.
		0 Reset
		1 Full-On
		2 Reserved
		3 Reserved
		4 Reserved
		5-15 Reserved

Table 4-9: DPM_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/NW)	CURMODE	Current Mode. The DPM_STAT.CURMODE bit field indicates the current mode of the module.
		0 Reserved
		1 Full-On
		2 Reserved
		3 Reserved
		4-15 Reserved

5 Reset Control Unit (RCU)

The Reset Control Unit (RCU) controls how all the functional units enter and exit reset. Differences in functional requirements and clocking constraints define how reset signals are generated. Programs must guarantee that none of the reset functions puts the system into an undefined state or causes resources to stall. While core resets and software system resets are controlled directly in the RCU, hardware resets can come from the TRU or SEC.

RCU Features

The RCU module supports the following features:

- Hardware reset through the `SYS_HWRST` pin
- Software system reset through the RCU control (`RCU_CTL`) register
- Hardware system reset through:
 - TRU module
 - SEC module (System Fault Unit)
- Core reset through RCU Core Reset Output (`RCU_CRCTL`) register

RCU Functional Description

This section provides information on the function of RCU module.

NOTE: In 2156x series of processors, there is only one core. The terminology *n* when describing number of cores (for example, core[n]) must be considered always as 0 (core 0).

Hardware reset using `SYS_HWRST` pin

Asserting the `SYS_HWRST` pin resets all functional units.

Hardware reset through RCU

The RCU can perform a full system reset which can be initiated through hardware blocks like the SEC or the TRU.

Software reset using RCU registers

Setting the `RCU_STAT.SWRST` bit issues a software reset for all system units except the `RCU_BCODE`, `RCU_CRCTL` and `RCU_STAT` registers.

Core reset RCU registers

A core can be individually reset by software, or by setting the `RCU_CRCTL.CR[n]` bit.

ADSP-21560_21561_21564_21568 RCU Register List

The Reset Control Unit (RCU) controls how all the functional units in the processor enter and exit Reset. Differences in functional requirements and clocking constraints exist (units in different clock domains have to enter reset asynchronously, but units exit reset in a deterministic way), and these differences define how reset signals are generated. Reset signals propagate through all functional units asynchronously. For more information on RCU functionality, see the RCU register descriptions.

Table 5-1: ADSP-21560_21561_21564_21568 RCU Register List

Name	Description
<code>RCU_BCODE</code>	Boot Code Register
<code>RCU_CRCTL</code>	Core Reset Outputs Control Register
<code>RCU_CRSTAT</code>	Core Reset Outputs Status Register
<code>RCU_CTL</code>	Control Register
<code>RCU_MSG</code>	Message Register
<code>RCU_MSG_CLR</code>	Message Clear Bits Register
<code>RCU_MSG_SET</code>	Message Set Bits Register
<code>RCU_SRRQSTAT</code>	System Reset Request Status Register
<code>RCU_STAT</code>	Status Register
<code>RCU_SVECT0</code>	Software Vector Register 0
<code>RCU_SVECT_LCK</code>	SVECT Lock Register

ADSP-21560_21561_21564_21568 RCU Trigger List

Table 5-2: ADSP-21560_21561_21564_21568 RCU Trigger List Generators

Trigger ID	Name	Description	Sensitivity

Table 5-2: ADSP-21560_21561_21564_21568 RCU Trigger List Generators (Continued)

Trigger ID	Name	Description	Sensitivity
None			

Table 5-3: ADSP-21560_21561_21564_21568 RCU Trigger List Receivers

Trigger ID	Name	Description	Sensitivity
44	RCU0_SYSRST0	RCU0 System Reset 0	Pulse
45	RCU0_SYSRST1	RCU0 System Reset 1	Pulse

RCU Definitions

To make the best use of the RCU, it is useful to understand the terms in this section.

The target or source defines the following are types of resets.

Hardware Reset (by target)

All functional units except a small subsection of debug interfaces are set to their default states. State is lost in all non-volatile storage.

System Reset (by target)

All functional units except the RCU, flash interface, and debug are set to their default states.

Core n Only Reset (by target)

Affects Core n only. The system software must guarantee that a bus requester cannot access the core in reset state.

Hardware Reset (by source)

The `SYS_HWRST` input signal is asserted active (pulled low).

System Reset (by source)

Software can trigger the reset by writing to the `RCU_CTL` register or by another functional unit such as the TRU or any of the generic reset inputs.

RCU Architectural Concepts

To understand the architecture of the RCU, it is important to consider the reset sources and how differing resets affect the functional units of the processor.

The RCU provides the hardware that controls how all the functional units enter and exit reset. Differences in functional requirements and clocking constraints define how reset signals are generated. For example, units in different clock domains must enter reset asynchronously but exit reset in a deterministic way.

The program must guarantee that none of the reset functions put the system in an undefined state or cause resources to stall. The program must guarantee that there is no pending system activity involving Core *n* before it is reset.

The *RCU Reset Sources* table defines how reset sources affect the different functional units.

Table 5-4: RCU Reset Sources

Reset Source	Reset Type	Affected Functional Units
<code>SYS_HWRST</code> pin assertion	Hardware Reset	All functional units, except RTC (if present)
<code>SYS_HWRST</code> pin assertion	Hardware Reset	All functional units
SYSCLK clock domain system reset by the Fault Unit (FMU) or the Trigger Routing Unit (TRU)	System Reset	All functional units, except: • RTC (if present) • RCU_STAT • RCU_BCODE • the units on the VDDEXT power domain
RCU_CTL.SYSRST bit set (software triggered reset)	System Reset	All functional units, except: • RTC (if present) • RCU_STAT • RCU_BCODE • the units on the VDDEXT power domain
RCU_CRCTL.CR[n] bit set (software triggered reset)	Core Only Reset	Core <i>n</i> only

RCU Status and Error Signals

The `RCU_STAT` register reflects status and error information. There are three kinds of errors that can occur in the RCU. The reset out error is triggered when `RSTOUT` is both asserted and deasserted at the same time. The lock write error occurs if an attempt is made to write a lock RCU register. The address error occurs if a read-only register is written to or if an attempt is made to a reserved address within the RCU MMR address range.

ADSP-21560_21561_21564_21568 RCU Register Descriptions

Reset Control Unit (RCU) contains the following registers.

Table 5-5: ADSP-21560_21561_21564_21568 RCU Register List

Name	Description
<code>RCU_BCODE</code>	Boot Code Register

Table 5-5: ADSP-21560_21561_21564_21568 RCU Register List (Continued)

Name	Description
RCU_CRCTL	Core Reset Outputs Control Register
RCU_CRSTAT	Core Reset Outputs Status Register
RCU_CTL	Control Register
RCU_MSG	Message Register
RCU_MSG_CLR	Message Clear Bits Register
RCU_MSG_SET	Message Set Bits Register
RCU_SRRQSTAT	System Reset Request Status Register
RCU_STAT	Status Register
RCU_SVECT0	Software Vector Register 0
RCU_SVECT_LCK	SVECT Lock Register

Boot Code Register

The `RCU_BCODE` register can be used to determine if and how core boots. This register is set to its default values by RESET.

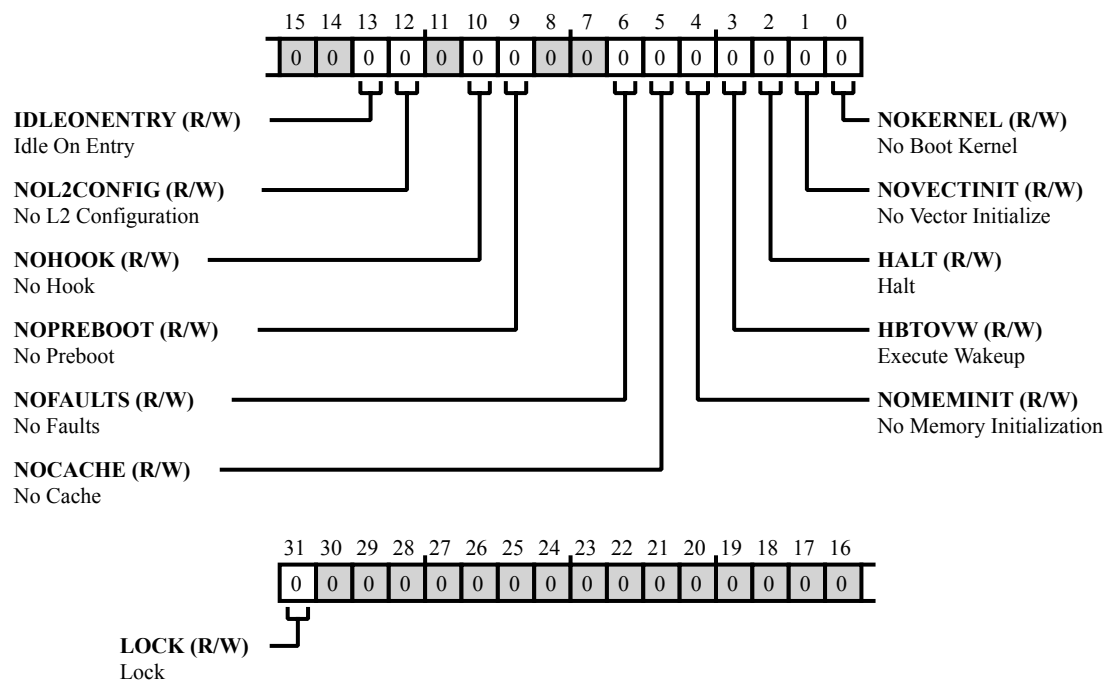


Figure 5-1: RCU_BCODE Register Diagram

Table 5-6: RCU_BCODE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock. If the global lock bit is set (<code>SPU_CTL.GLCK</code> bit =1) and the <code>RCU_BCODE.LOCK</code> bit is set, the <code>RCU_BCODE</code> register is read only (locked).
		0 Unlock
		1 Lock
13 (R/W)	IDLEONENTRY	Idle On Entry. The <code>RCU_BCODE.IDLEONENTRY</code> bit configures the RCU to enter the idle state at startup.
		0 Do not enter idle state
		1 Enter idle state

Table 5-6: RCU_BCODE Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
12 (R/W)	NOL2CONFIG	No L2 Configuration. The RCU_BCODE.NOL2CONFIG bit configures the RCU to not perform the L2 memory configuration.
		0 Configure L2 memory
		1 Do not configure L2 memory
10 (R/W)	NOHOOK	No Hook. The RCU_BCODE.NOHOOK bit configures the RCU to not perform the hook routine.
		0 Perform hook routine
		1 Do not perform hook routine
9 (R/W)	NOPREBOOT	No Preboot. The RCU_BCODE.NOPREBOOT bit configures the RCU to not perform the customer preboot routine.
		0 Perform preboot
		1 Do not perform preboot
6 (R/W)	NOFAULTS	No Faults. The RCU_BCODE.NOFAULTS bit configures the RCU to not perform fault initialization.
		0 Perform fault initialization
		1 Do not perform fault initialization
5 (R/W)	NOCACHE	No Cache. The RCU_BCODE.NOCACHE bit configures the RCU to not perform a cache initialization and to not enable the cache.
		0 Enable and initialize cache
		1 Do not initialize or enable cache
4 (R/W)	NOMEMINIT	No Memory Initialization. The RCU_BCODE.NOMEMINIT bit configures the RCU to not perform a memory initialization.
		0 Perform memory initialization
		1 Do not perform memory initialization

Table 5-6: RCU_BCODE Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
3 (R/W)	HBTOWV	Execute Wakeup. The RCU_BCODE.HBTOWV bit configures the RCU to execute a wakeup.
		0 Do not wakeup
		1 Execute wakeup
2 (R/W)	HALT	Halt. The RCU_BCODE.HALT bit configures the RCU to execute the no boot routine.
		0 Do not execute routine
		1 Execute routine
1 (R/W)	NOVECTINIT	No Vector Initialize. The RCU_BCODE.NOVECTINIT bit configures the RCU to not vector to the application.
		0 Vector
		1 Do not vector
0 (R/W)	NOKERNEL	No Boot Kernel. The RCU_BCODE.NOKERNEL bit configures the RCU to not execute the boot kernel.
		0 Execute boot kernel
		1 Do not execute boot kernel

Core Reset Outputs Control Register

The RCU core reset control n registers ([RCU_CRCTL](#)) include a lock bit ([RCU_CRCTL.LOCK](#)) and a core reset bit ([RCU_CRCTL.CR\[n\]](#)) for each core reset signal on the product.

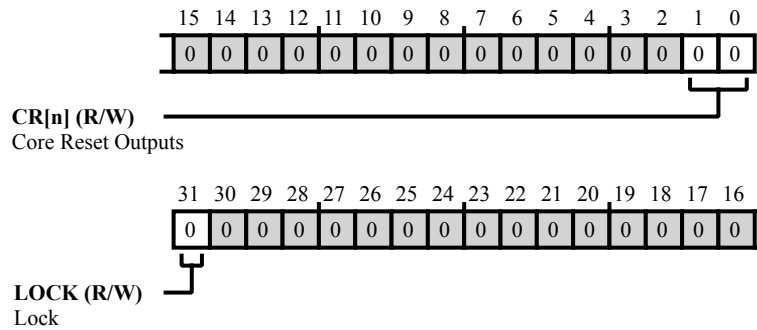


Figure 5-2: RCU_CRCTL Register Diagram

Table 5-7: RCU_CRCTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock. If the global lock bit is set (SPU_CTL.GLCK bit =1) and the RCU_CRCTL.LOCK bit is set, the RCU_CRCTL register is read only (locked).
		0 Unlock
		1 Lock
1:0 (R/W)	CR[n]	Core Reset Outputs. The RCU_CRCTL.CR[n] bits control CRES[1:0] core reset signals. The RCU_CRES[n] signals can be individually controlled. They are reset to their default value by a hard reset or a system reset. For each RCU_CRES[n] , the selected RCU0_CRMSKi[n] bit is cleared.
		0 RCU_CRES[1:0] Deasserted
		3 RCU_CRES[1:0] Asserted

Core Reset Outputs Status Register

The RCU core reset status register ([RCU_CRSTAT](#)) contains status bits, indicating which core reset signals have been asserted.

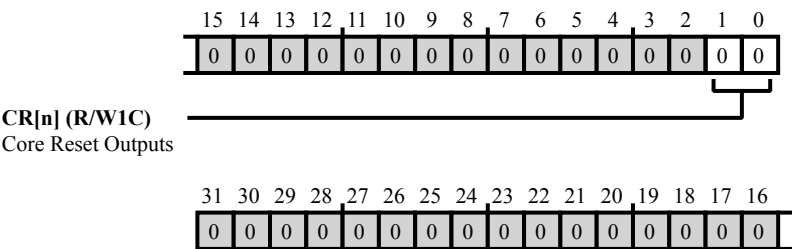


Figure 5-3: RCU_CRSTAT Register Diagram

Table 5-8: RCU_CRSTAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
1:0 (R/W1C)	CR[n]	Core Reset Outputs. The <code>RCU_CRSTAT.CR[n]</code> bits indicate which cores have been reset since the last time the bit was cleared. Bits masked by <code>CORE_DISABLE_MASK[15:0]</code> are permanently disabled and the corresponding CR bits set. CR bits are sticky, they need to be cleared by software.
		0 RCU_CRES[1:0] deasserted. CR[n] corresponds to RCU_CRES[n].
		3 RCU_CRES[1:0] were asserted since the last time bits were cleared. CR[n] corresponds to RCU_CRES[n].

Control Register

The RCU control register (`RCU_CTL`) provides a register lock, enables for the core and system reset requests inputs and control for the Reset Output pin.

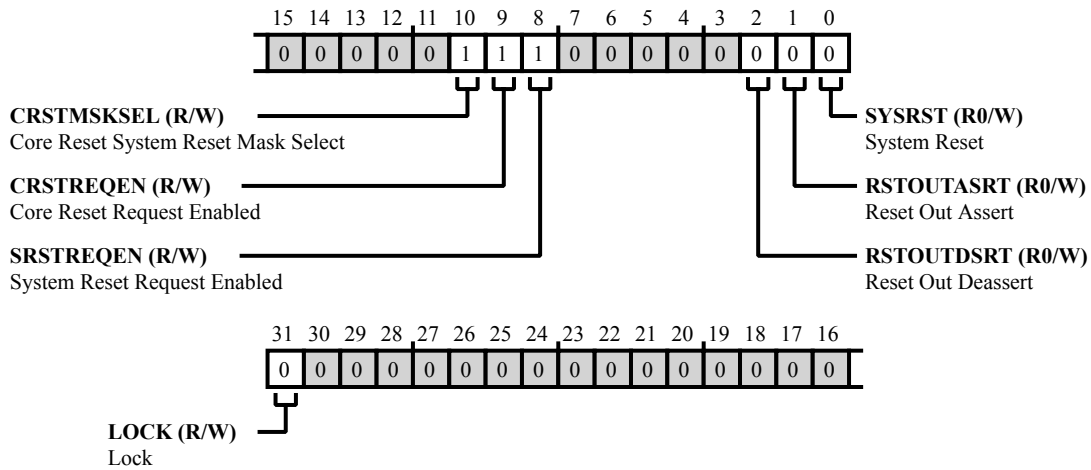


Figure 5-4: RCU_CTL Register Diagram

Table 5-9: RCU_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock.
		If the global lock bit is set (<code>SPU_CTL.GLCK</code> bit =1) and the <code>RCU_CTL.LOCK</code> bit is set, the <code>RCU_CTL</code> register is read only (locked). This bit is cleared by a hard reset or any system reset event.
		0 Unlock 1 Lock
10 (R/W)	CRSTMSKSEL	Core Reset System Reset Mask Select. The <code>RCU_CTL.CRSTMSKSEL</code> bit selects the core reset system reset mask. This bit is cleared by a hard reset.
9 (R/W)	CRSTREQEN	Core Reset Request Enabled.
		The <code>RCU_CTL.CRSTREQEN</code> bit controls whether the <code>SYSCLK</code> domain source(s) of reset is/are enabled to reset the core(s) when asserted. This bit is cleared by hard reset or any system reset event.
		0 Disabled 1 Enabled

Table 5-9: RCU_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
8 (R/W)	SRSTREQEN	System Reset Request Enabled. The RCU_CTL.SRSTREQEN bit controls whether the SYSCLK domain sources of reset are enabled to do a system reset when asserted. This bit is cleared by a hard reset.
		0 Disabled
		1 Enabled
2 (R0/W)	RSTOUTDSRT	Reset Out Deassert. The RCU_CTL.RSTOUTDSRT bit controls the deassertion of the system reset pin.
		0 No Action
		1 Deassert RSTOUT
1 (R0/W)	RSTOUTASRT	Reset Out Assert. The RCU_CTL.RSTOUTASRT bit controls assertion of the system reset pin.
		0 No Action
		1 Assert RSTOUT
0 (R0/W)	SYSRST	System Reset. The RCU_CTL.SYSRST bit provides reset for all system units.
		0 No Action
		1 System Reset

Message Register

The `RCU_MSG` is a general-purpose register. It is intended to provide flexibility for Boot ROM code and to pass predefined variables to the debugger. Please see the Booting chapter for product-specific details.

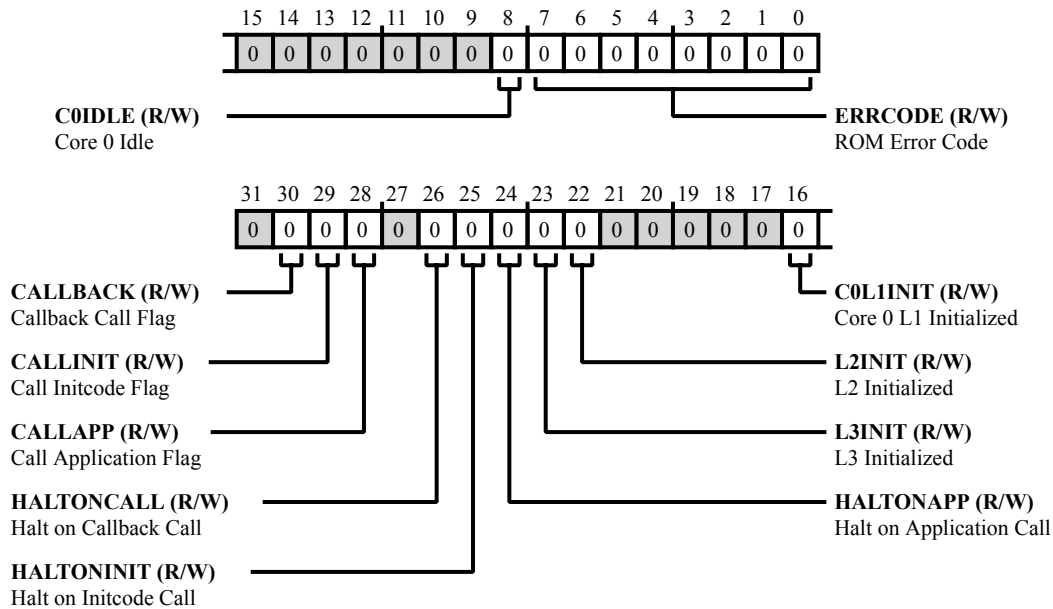


Figure 5-5: RCU_MSG Register Diagram

Table 5-10: RCU_MSG Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
30 (R/W)	CALLBACK	Callback Call Flag. The <code>RCU_MSG.CALLBACK</code> bit indicates that a flag has been set by the boot code prior to a callback call.
		0 Flag not set
		1 Flag set
29 (R/W)	CALLINIT	Call Initcode Flag. The <code>RCU_MSG.CALLINIT</code> bit indicates that a flag has been set by the boot code prior to an initcode call.
		0 Flag not set
		1 Flag set

Table 5-10: RCU_MSG Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
28 (R/W)	CALLAPP	Call Application Flag. The RCU_MSG.CALLAPP bit indicates that a flag has been set by the boot code prior to an application call.
		0 Flag not set
		1 Flag set
26 (R/W)	HALTONCALL	Halt on Callback Call. The RCU_MSG.HALTONCALL bit generates an emulation exception prior to a call-back call.
		0 Do not generate exception
		1 Generate exception
25 (R/W)	HALTONINIT	Halt on Initcode Call. The RCU_MSG.HALTONINIT bit generates an emulation exception prior to an init-code call.
		0 Do not generate exception
		1 Generate exception
24 (R/W)	HALTONAPP	Halt on Application Call. The RCU_MSG.HALTONAPP bit generates an emulation exception prior to an application call.
		0 Do not generate exception
		1 Generate exception
23 (R/W)	L3INIT	L3 Initialized. The RCU_MSG.L3INIT bit indicates that the L3 resource is initialized.
		0 Resource not initialized
		1 Resource initialized
22 (R/W)	L2INIT	L2 Initialized. The RCU_MSG.L2INIT bit indicates that the L2 resource is initialized.
		0 Resource not initialized
		1 Resource initialized
16 (R/W)	COL1INIT	Core 0 L1 Initialized. The RCU_MSG.COL1INIT bit indicates that the core 0 L1 resource is initialized.
		0 Resource not initialized
		1 Resource initialized

Table 5-10: RCU_MSG Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
8 (R/W)	C0IDLE	Core 0 Idle. The <code>RCU_MSG.C0IDLE</code> bit indicates that core 0 is in a safe idle state in ROM.
7:0 (R/W)	ERRCODE	ROM Error Code. The <code>RCU_MSG.ERRCODE</code> bit indicates the error code of the ROM. It is valid only when in the error handler.

Message Clear Bits Register

The `RCU_MSG_CLR` register is used to clear bits in `RCU_MSG` register. Reading this register returns 0x00000000.

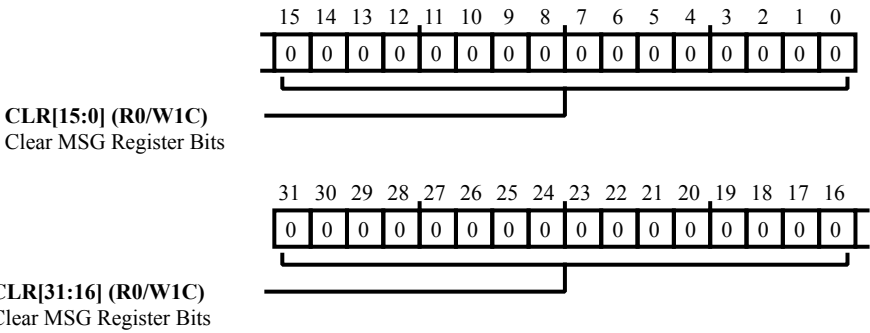


Figure 5-6: `RCU_MSG_CLR` Register Diagram

Table 5-11: `RCU_MSG_CLR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R0/W1C)	CLR	Clear MSG Register Bits. The <code>RCU_MSG_CLR.CLAR</code> bit resets MSG bit n.

Message Set Bits Register

The `RCU_MSG_SET` register is used to set bits in `RCU_MSG` register. Reading this register returns 0x00000000.

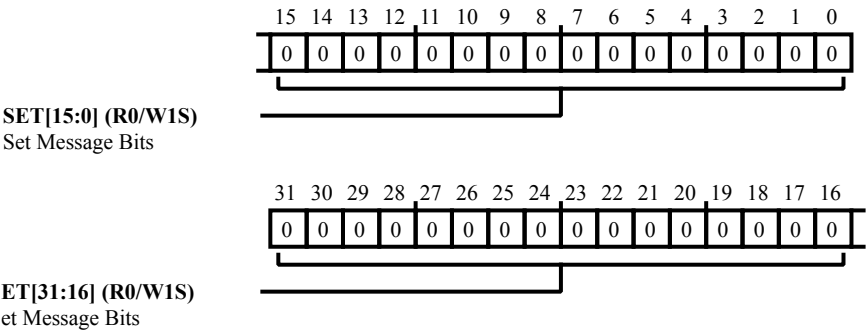


Figure 5-7: RCU_MSG_SET Register Diagram

Table 5-12: RCU_MSG_SET Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R0/W1S)	SET	Set Message Bits. The <code>RCU_MSG_SET.SET</code> bit sets MSG bit n.

System Reset Request Status Register

The RCU system reset request status register ([RCU_SRRQSTAT](#)) contains status bits, indicating which system reset request input triggered a system reset. This register is set to its default values by a hard reset.

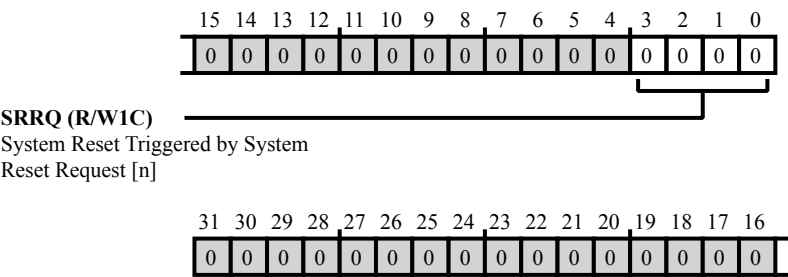


Figure 5-8: RCU_SRRQSTAT Register Diagram

Table 5-13: RCU_SRRQSTAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W1C)	SRRQ	System Reset Triggered by System Reset Request [n]. The <code>RCU_SRRQSTAT.SRRQ</code> bits are set by the assertion of the corresponding system reset request input and deasserted by writing "1" to the bit. The RCU_SRRQSTAT register is cleared by a hard reset.

Status Register

The RCU status register (`RCU_STAT`) contains status bits for all RCU reset sources, reset status, and boot mode inputs. Status bits for reset sources are sticky and can be cleared by software. Error status bits are cleared by any reset event.

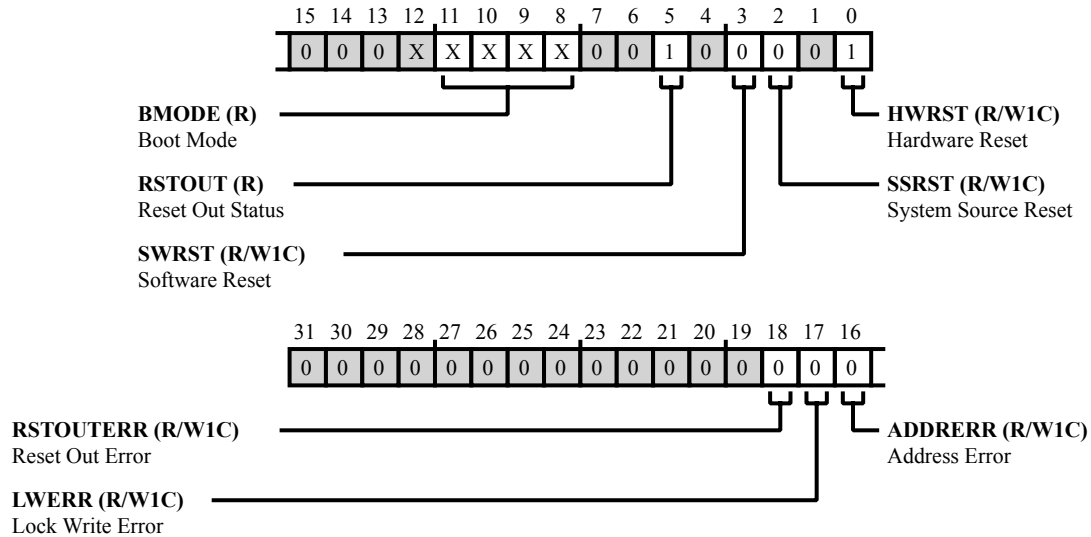


Figure 5-9: RCU_STAT Register Diagram

Table 5-14: RCU_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
18 (R/W1C)	RSTOUTERR	Reset Out Error. The <code>RCU_STAT.RSTOUTERR</code> bit indicates (if set) that a write attempted to set the <code>RCU_CTL.RSTOUTASRT</code> and <code>RCU_CTL.RSTOUTDSRT</code> simultaneously. This condition triggers a bus error.
		0 No Error
		1 Error Occurred
17 (R/W1C)	LWERR	Lock Write Error. The <code>RCU_STAT.LWERR</code> bit indicates (when set) there was an attempted write to an RCU register while the <code>RCU_CTL.LOCK</code> bit was set and the global lock bit is enabled (<code>SPU_CTL.GLCK</code> bit =1). This status bit is sticky; write-1-to-clear
		0 No Error
		1 Error Occurred

Table 5-14: RCU_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
16 (R/W1C)	ADDRERR	Address Error. The RCU_STAT.ADDRERR bit indicates that the RCU generated an address error. This status bit is sticky; write-1-to-clear it.
		0 No Error
		1 Error Occurred
11:8 (R/NW)	BMODE	Boot Mode. The RCU_STAT.BMODE bits indicate the input on the boot mode pins.
5 (R/NW)	RSTOUT	Reset Out Status. The RCU_STAT.RSTOUT bit indicates the assertion status of the system reset pin.
		0 RSTOUT Deasserted
		1 RSTOUT Asserted
3 (R/W1C)	SWRST	Software Reset. The RCU_STAT.SWRST bit indicates that a system reset (which was triggered by software) has occurred since the last time a hardware reset occurred or since the RCU_STAT.SWRST bit was cleared by software.
		0 Inactive
		1 Reset Occurred
2 (R/W1C)	SSRST	System Source Reset. The RCU_STAT.SSRST bit indicates that a system reset triggered by hardware in the system clock domain, clock A domain, or clock B domain has occurred since the last time a hardware reset occurred or since the RCU_STAT.SSRST bit was cleared by software.
		0 Inactive
		1 Reset Occurred
0 (R/W1C)	HWRST	Hardware Reset. The RCU_STAT.HWRST bit indicates that a hardware reset has occurred.
		0 Inactive
		1 Reset Occurred

Software Vector Register 0

The `RCU_SVECT0` register contains the default location of the first instruction to execute after a reset.

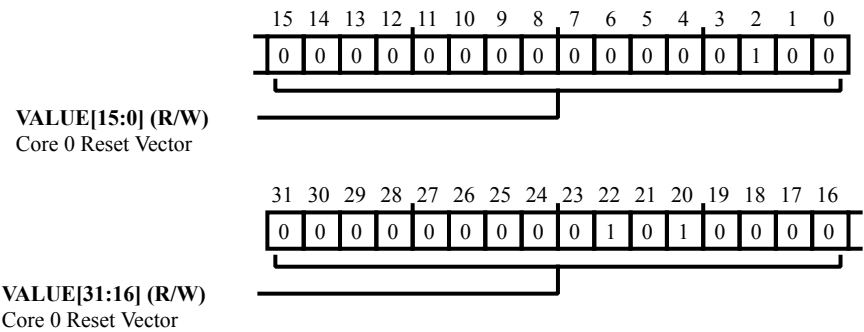


Figure 5-10: RCU_SVECT0 Register Diagram

Table 5-15: RCU_SVECT0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Core 0 Reset Vector. Core0 jumps to the address in the RCU_SVECT0 . VALUE bit field after reset.

SVECT Lock Register

The RCU software vector lock register ([RCU_SVECT_LCK](#)) provides a register lock and software vector n enable bits for each processor core on the product. This register is set to its default values by a hard reset or any system reset event.

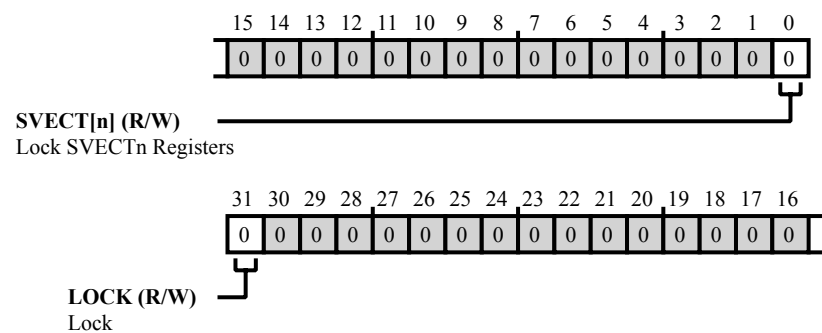


Figure 5-11: RCU_SVECT_LCK Register Diagram

Table 5-16: RCU_SVECT_LCK Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock. If the global lock bit is set (SPU_CTL . GLCK bit =1) and the RCU_SVECT_LCK . LOCK bit is set, the RCU_SVECT_LCK register is read only (locked).
		0 Unlock
		1 Lock
0 (R/W)	SVECT[n]	Lock SVECTn Registers. If the global lock bit is set (SPU_CTL . GLCK bit =1) and the RCU_SVECT_LCK . SVECT [n] bit is set, the SVECT registers are read only (locked).

6 System Event Controller (SEC)

System event management is the responsibility of the system event controller (SEC). The SEC manages the configuration of all system event sources. The SEC also manages the propagation of system events to all connected cores and the system fault interface.

All the peripheral interrupts are routed using a single SEC interrupt to the desired core. The SEC allows programmability of the peripheral interrupt's priority, supporting up to 256 priority levels that are arbitrated within the SEC itself. The SEC also allows these interrupts to be grouped and masked by priority level and provides the flexibility to choose which core(s) the interrupt is routed to.

The SEC also supports self-nesting of interrupts, which is required when sharing a single interrupt request to an individual core, as this allows for a higher-priority peripheral interrupt to be passed to the core while it is currently servicing a lower-priority peripheral interrupt. For more information, refer to “Self-Nesting Mode for System Event Controller Interrupt (SECI)” in the *SHARC+ Core Programming Reference*.

SEC Features

The following list describes the system event controller features.

- Comprehensive system event source management including interrupt enable, fault enable, priority, core mapping, and source grouping.
- Fault management including fault action configuration, timeout, external indication, and system reset.
- Determinism where all system events have the same propagation delay and provide unique identification of a specific system event source.
- Distributed programming model where each system event source control and all status fields are independent of all others.
- Completer control port which provides access to all SEC registers for configuration, status, and interrupt or fault service model.
- Global locking supports a register level protection model to prevent writes to “locked” registers.

SEC Functional Description

The following sections provide a functional description of the SEC.

The *SEC/GIC Interrupt Signal Flow* figure shows an overview of the interrupt systems.

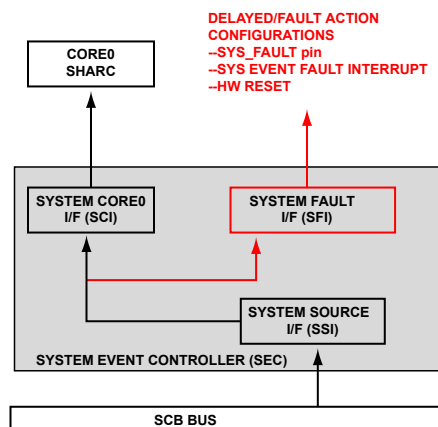


Figure 6-1: SEC Interrupt Signal Flow

ADSP-21560_21561_21564_21568 SEC Register List

The System Event Controller (SEC) manages the system fault sources, including control features such as enable/disable and active/pending source status. For more information on SEC functionality, see the SEC register descriptions.

Table 6-1: ADSP-21560_21561_21564_21568 SEC Register List

Name	Description
SEC_CACT[n]	SCI Active Register n
SEC_CCTL[n]	SCI Control Register n
SEC_CGMSK[n]	SCI Group Mask Register n
SEC_CPLVL[n]	SCI Priority Level Register n
SEC_CPMSK[n]	SCI Priority Mask Register n
SEC_CPND[n]	Core Pending Register n
SEC_CSID[n]	SCI Source ID Register n
SEC_CSTAT[n]	SCI Status Register n
SEC_END	Global End Register
SEC_FCOPP	Fault COP Period Register
SEC_FCOPP_CUR	Fault COP Period Current Register
SEC_FCTL	Fault Control Register
SEC_FDLY	Fault Delay Register

Table 6-1: ADSP-21560_21561_21564_21568 SEC Register List (Continued)

Name	Description
SEC_FDLY_CUR	Fault Delay Current Register
SEC_FEND	Fault End Register
SEC_FSID	Fault Source ID Register
SEC_FSRDLY	Fault System Reset Delay Register
SEC_FSRDLY_CUR	Fault System Reset Delay Current Register
SEC_FSTAT	Fault Status Register
SEC_GCTL	Global Control Register
SEC_GSTAT	Global Status Register
SEC_RAISE	Global Raise Register
SEC_SCTL[n]	Source Control Register n
SEC_SSTAT[n]	Source Status Register n

ADSP-21560_21561_21564_21568 SEC Interrupt List

Table 6-2: ADSP-21560_21561_21564_21568 SEC Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
0	SEC0_ERR	SEC0 Error	Level	

ADSP-21560_21561_21564_21568 SEC Trigger List

Table 6-3: ADSP-21560_21561_21564_21568 SEC Trigger List Generators

Trigger ID	Name	Description	Sensitivity
67	SEC0_FAULT	SEC0 Fault	Edge

Table 6-4: ADSP-21560_21561_21564_21568 SEC Trigger List Receivers

Trigger ID	Name	Description	Sensitivity
None			

SEC Definitions

The event controller uses the following definitions.

SCI

SEC core interface, core interface subblock of the SEC

SID (Identification, unique)

Source numeric identifier for each system source connected to the SEC.

SFI

SEC Fault Interface, fault management subblock of the SEC.

SPR

SEC prioritizer determines the highest priority pending interrupt and the highest priority active interrupt. The SPR provides these interrupts in the appropriate registers of the SCI for the priority and nesting model of the SCI.

SSI

SEC source interface, system event source control, and status subblock of the SEC.

System Events

System source indications including interrupts and faults.

System Source

Point of origin of system event.

SEC Block Diagram

The *SEC Block Diagram* shows the event management architecture.

System sources connect to the SEC through the SSI. Each core has a dedicated SCI. The SFI provides fault action connections to the rest of the system.

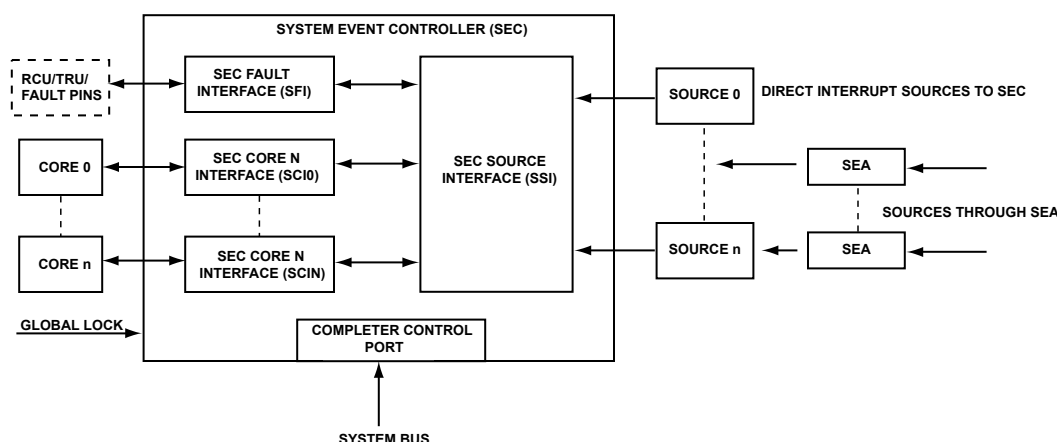


Figure 6-2: SEC Block Diagram

As shown in the figure, the SEC has three blocks for event management. The SFI monitors and manages any fault event triggered from various fault input sources. System interrupt sources are routed to the SFI through SEC source interface (SSI).

SEC Fault Interface (SFI)

The SFI manages fault events and associated actions. The fault management support provided in the SEC helps satisfy the safety requirements of various applications. The SSI provides the highest priority pending source that is enabled as a fault. The SFI captures this value and enables a countdown, and once the countdown expires, takes the prescribed action.

Fault actions which can be configured, as shown in *SFI Block Diagram*, include

- Trigger Output
- System Reset
- Fault Output
 - Computer Operating Properly (COP) mode
 - Fault Mode

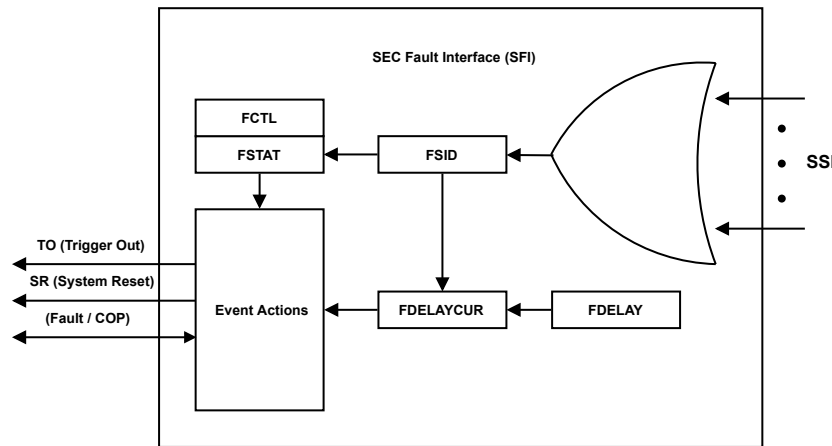


Figure 6-3: SFI Block Diagram

Fault Management

System sources can be enabled as fault sources in the `SEC_SCTL[n]` register. When a source enabled as a fault moves to pending, it is forwarded to the SFI as a fault indication. The pending bit (`SEC_FSTAT.PND`) indicates a source has signaled a fault assertion but it has not yet triggered the event actions (if delay is enabled). The SEC fault interface sets the `SEC_FSTAT.PND` bit when the fault source ID register (`SEC_FSID`) is updated on assertion of a fault source input. The system source pending triggers a fault pending and after a programmable delay the fault moves to active. Event actions then execute if appropriate action is not taken by the core. The `SEC_FSTAT.ACT` bit indicates that the SEC has received a fault source input, the delay has expired, and the fault actions are enabled.

The `SEC_FSTAT.NPND` bit indicates if one or more sources have signaled a fault assertion, but the input has not yet triggered the fault pending detection in the SEC fault interface. The SEC sets the `SEC_FSTAT.NPND` bit when the fault interface detects assertion of any enabled fault source input, while either the `SEC_FSTAT.PND` or `SEC_FSTAT.ACT` bits are set. The SEC clears the `SEC_FSTAT.NPND` bit when there are no fault sources waiting.

A fault indication from an external device can also be detected on sampling the fault signals. When a fault is detected the `SEC_FSTAT.ACT` and `SEC_FSID.FEXT` bits are set. The assertion of either signal results in a fault input detection.

The `SEC_FEND` register receives a fault end indication from the core. The core writes the SID of the fault to the `SEC_FEND` register. If the SID matches the value in the `SEC_FSID` register, the `SEC_FSTAT.PND` and `SEC_FSTAT.ACT` bits are cleared.

SEC Core Interface (SCI)

The SCI manages communication between the corresponding core and the SEC. The SEC prioritizer (SPR) of the SCI receives pending, active, and priority information from the SSI for each system event source assigned to this SCI. The SPR determines the highest priority pending system event and the SCI determines whether it propagates to the core. The SCI maintains the coherency for the system event service model implemented on the connected core.

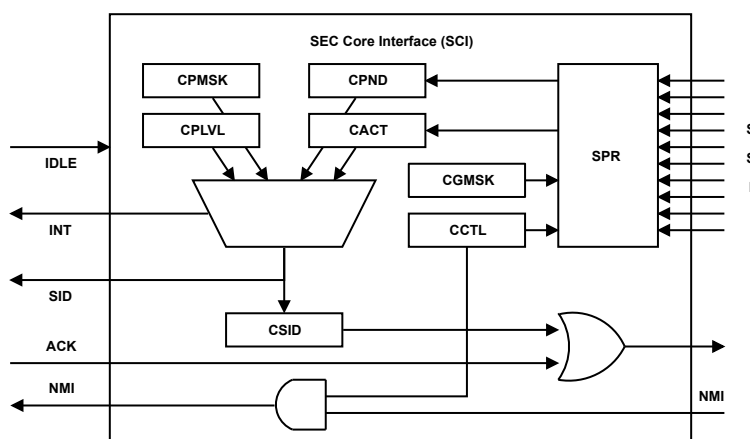


Figure 6-4: SCI Overview Block Diagram

SEC Source Interface (SSI)

The SSI manages all of the system event sources. It maintains the status of each source in the corresponding `SEC_SSTAT[n]` register. The corresponding `SEC_SCTL[n]` register manages the control of each source. A pending and enabled event passes its indication and priority to the SCI to which it is assigned for further processing.

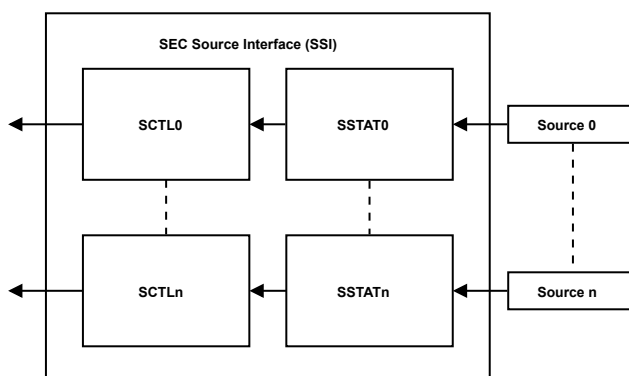


Figure 6-5: SSI Overview Block Diagram

SEC Architectural Concepts

The following sections describe SEC architectural features.

System Interrupt Acknowledge

A system interrupt acknowledge occurs when the core provides an indication that it has acquired the SID of the interrupt last issued by the SEC. The SEC core interface option allows generation by:

- A completer port write to the `SEC_CSID[n]` register.
- The assertion of an input acknowledge signal (the connected core generates the signal).

System Interrupt Groups

System sources can be assigned to groups using the `SEC_SCTL[n].GRP` bit field. Source groups allow fast context switching for system interrupts at each SCI. The `SEC_CGMSK[n]` register allows quick masking of interrupt groups of unlimited size with a single write operation.

System Interrupt Flow

An enabled and asserted system interrupt source is latched at the SSI and routed to the appropriate SCI based on the core target select (`SEC_SCTL[n].CTG`) bit field setting. The SEC priority ordering determines the highest priority pending system interrupt and the SCI updates the `SEC_CPND[n].SID` and `SEC_CACT[n].PRIO` bit field values. The SCI compares the `SEC_CPND[n]` register value against the highest priority active source in the `SEC_CACT[n]` register).

The priority level register (`SEC_CPLVL[n]`) determines how many of the MSBs the SEC uses in the comparison. The priority mask register (`SEC_CPMSK[n]`) and the group mask register (`SEC_CGMSK[n]`) determines which pending interrupt sources participate. If the `SEC_CPND[n]` register value is a higher priority (lower value) than the priority of the `SEC_CACT[n]` register from the comparison based on the `SEC_CPLVL[n]` register, the system interrupt output is asserted. The source ID register (`SEC_CSID[n]`) is updated with the `SEC_CPND[n].SID` bit field value and forwarded to the connected core.

After the core provides an interrupt acknowledgment, the interrupt source is active, until the SEC completes interrupt service with a write to the `SEC_END.SID` bit field with the same value. Note the following:

- Interrupt acknowledgement occurs with an MMR write of the `SEC_CSID[n]` register or the core version of the `SEC_CSID[n]` register.
- Interrupt active status indication is `SEC_SSTAT[n].ACT==1`.

The following sequence shows the example flow for a single interrupt.

1. The SEC compares the `SEC_CPND[n]` register value to the `SEC_CACT[n]` register value. If the interrupt in the `SEC_CPND[n]` register is higher priority, continue.
2. The SEC copies the `SEC_CPND[n]` register value to the `SEC_CSID[n]` register and asserts the interrupt signal.
3. The core reads the `SEC_CSID[n]` register (or core version).
4. The core writes to the `SEC_CSID[n]` register (or core version, asserts the acknowledge signal).
5. The SEC deasserts the interrupt signal and clears the `SEC_SSTAT[n].PND` bit and sets the `SEC_SSTAT[n].ACT` bit of the source going active.
6. The core writes the `SEC_CSID[n]` of the active interrupt to the `SEC_END` register.
7. The SEC clears the `SEC_SSTAT[n].ACT` bit of the source being ended.

The following sequence shows the example flow for interrupt nesting where interrupt A is a lower priority and occurs earlier than interrupt B.

1. The SEC compares the `SEC_CPND[n]` (A) register value to the `SEC_CACT[n]` register and if the interrupt in the `SEC_CPND[n]` register is a higher priority, continue.
2. The SEC copies `SEC_CPND[n]` (A) register to the `SEC_CSID[n]` register and asserts the interrupt signal.
3. The core reads the `SEC_CSID[n]` (A) register (or core version).
4. The core writes to the `SEC_CSID[n]` register (or core version, asserts the acknowledge signal).
5. The SEC deasserts the INT signal and clears the `SEC_SSTAT[n].PND` bit and sets the `SEC_SSTAT[n].ACT` bit of the source (A) going active.
6. The SEC compares the `SEC_CPND[n]` (B) register value to the `SEC_CACT[n]` (A) register value. If the `SEC_CACT[n]` (B) register value is a higher priority, continue.
7. The SEC copies the `SEC_CPND[n]` (B) register value to `SEC_CSID[n]` register and asserts the interrupt signal.
8. The core reads the `SEC_CSID[n]` (B) register (or core version).
9. The core writes to the `SEC_CSID[n]` register (or core version, asserts the acknowledge signal).
10. The SEC deasserts the INT signal and clears the `SEC_SSTAT[n].PND` bit and sets the `SEC_SSTAT[n].ACT` bit of the source (B) going active.
11. The core writes the `SEC_CSID[n]` of the active interrupt (B) to the `SEC_END` register.
12. The SEC clears the `SEC_SSTAT[n].ACT` bit of the source (B) being ended.
13. The core writes the `SEC_CSID[n]` of the active interrupt (A) to the `SEC_END` register.
14. The SEC clears the `SEC_SSTAT[n].ACT` bit of the source (A) being ended.

System Interrupt Priorities

Each system interrupt source has its own programmable priority level which is configured using the `SEC_SCTL[n].PRIO` bit field. The SCI evaluates the priority of all pending sources to determine the source of the highest priority pending system interrupt for forwarding to the attached core. If more than one source of the pending system interrupt has the same priority setting, the SCI chooses the one with the lowest SID. For example, if SID 0, SID 1, and SID 2 are all pending and have the same priority setting, the SCI chooses SID 0 as the highest-priority source.

SEC Error

The processor includes an SEC error (`SEC_GSTAT.ERR`) as a source input to the SEC to allow for handling the error as an interrupt or fault.

SEC Programming Model

Implementing a system interrupt service model using the SEC requires, at a minimum:

- Proper configuration of a system interrupt source (for example a peripheral or DMA)
- A core interrupt or event service model

The core must be configured for response to system interrupts from the SEC. The SEC must be configured to enable and map the system interrupt source to the correct SCI and to forward interrupts to the connected core.

The system interrupt source must be configured to generate interrupt assertions. Alternatively, the processor can use software triggering for interrupt assertion. Software driven interrupts are generated by writing the source ID of the interrupt to be triggered to the [SEC_RAISE](#) register.

Programming Concepts

The following list provides the basic programming concepts necessary for configuring the SEC.

- Configuring an SSI as a system interrupt for a specific core.
- Configuring an SCI to provide system interrupts to the connected core (See [Configuring a System Source to Interrupt a Core](#)).
- Configuring an SSI as a system fault (See [Configuring a System Source as a Fault](#)).
- Configuring the SFI to manage system faults.

Programming Examples

This section provides example programming tasks that are typical for SEC usage.

Fault Management Interface Programming Model

The SFI interface can be programmed to manage fault events from system sources and associated actions such as issuing a system reset when watchdog expiration event occurs.

1. Set the `SEC_GCTL.EN` bit to enable the SEC.
 2. Write to the `SEC_FCTL` register to configure specific fault actions.
 - **Trigger Output.** Set the `SEC_FCTL.TOEN` bit for the SEC to produce trigger outputs when a fault becomes active. The `SEC_FCTL.TES` bit can be programmed to select the event that directs the SEC to assert trigger output when a fault is pending or active. Configure receivers for SEC fault trigger generator output.
- NOTE:** If the `SEC_FCTL.TOEN` and or the `SEC_FCTL.TES` bits =1 (Trigger Output Enabled and Trigger on Fault Pending), an external fault (if enabled by the `SEC_FCTL.FIEN` bit) will not issue a trigger since Fault Pending is bypassed for external faults.
- **System Reset.** The Reset Control Unit (RCU) controls how the functional units enter and exit reset. Configure the `RCU_CTL.SRSTREQEN` bit. This bit controls whether the sources of reset are enabled to perform a system reset. To issue a system reset request when a fault becomes active, set the `SEC_FCTL.SREN` bit. The SEC fault system reset delay register ([SEC_FSRDLY](#)) can be programmed for the delay, if required, from a fault becoming active to system reset request assertion.

- **Fault Output.** This configuration allows the SEC to indicate the fault status based on the `SEC_FCTL.CMS` bit configuration.
 - **Computer Operating Normally (COP) mode.** To configure fault output for COP mode, set the `SEC_FCTL.FOEN` bit to enable fault output. Set the `SEC_FCTL.CMS` bit to select COP mode to toggle the fault pin when no fault is active. Program the `SEC_FCOPP` period register with a desired width value for the COP toggled output pin.
 - **Fault mode.** Set the `SEC_FCTL.FOEN` bit to enable fault output. The `SEC_FCTL.CMS` bit should be set to Fault mode to toggle the fault pin when a fault is active.
- 3. If required, program the Fault Input to sample fault inputs from external devices on fault pins. Configure the `SEC_FCTL.FIEN` bit to enable the SEC to sample a fault input from an external device.

ADDITIONAL INFORMATION: The `SEC_FCTL.FIEN` bit should be set only while the `SEC_FCTL.EN` bit is low. If the `SEC_FCTL.EN` bit is already high and the `SEC_FCTL.FIEN` bit needs to be set, the `SEC_FCTL.EN` bit should be cleared first. Fault input can only be enabled when Fault mode is selected by the `SEC_FCTL.CMS` bit.
- 4. Program the required fault delay to the `SEC_FDLY.COUNT` bit field if a delay between fault source assertion and the fault response is required.
- 5. Configure the `SEC_FCTL` register to enable the SEC.

ADDITIONAL INFORMATION: The `SEC_FCTL.EN` bit should be set only while the `SEC_FSTAT.ACT` bit is low.
- 6. Write to the control register of a specific source register using the `SEC_SCTL[n]` register to enable the source as a fault.

Configuring a System Source to Interrupt a Core

To configure a system source to interrupt a core, the SEC itself must be enabled with the source interface (SSI) and core interface (SCI) properly initialized. Specifically, the SCI must be set up to accept interrupt signaling from the SEC and pass them to the specified core, and the SSI must properly enable each of the peripheral interrupt sources to generate interrupt signals and optionally define a priority scheme that overrides the default priority settings. In summary:

1. Write to the `SEC_GCTL` register to enable the SEC.
2. Write to the appropriate SCI `SEC_CCTL[n]` register to enable SEC interrupts to be sent to that core.
3. Write to the appropriate SSI `SEC_SCTL[n]` register to enable that peripheral as an interrupt source and to set the core target field to map the source to the desired SCI.
4. (Optional) By default, all the SEC interrupts are grouped as a single priority level, so passing of peripheral interrupt requests from the SEC is based solely on the default enumerated source ID. By programming the `SEC_CPLVL[n].PLVL` register, interrupt sources can be grouped into priority levels within the SEC such that arbitration is first performed by source ID within a grouped priority level before proceeding to the next

priority level, thus providing the flexibility to have lower-priority interrupt sources considered before higher-priority sources.

ADDITIONAL INFORMATION: The `SEC_CPMSK[n]` and `SEC_CGMSK[n]` registers must also can be programmed to mask the interrupts based on the customized levels and grouping.

Core/SEC Handshake Requirements to Ensure Proper Interrupt Handling

A specific handshake with the SEC is required to handle interrupts associated with an individual core. The handshake ensures that nested interrupts are properly tracked and that new peripheral interrupts being raised within the SEC are either passed immediately to the core or held off and queued within the SEC for later servicing.

Use the following procedure to write a custom dispatcher inside the Interrupt Service Routine. Note that the core needs to read and acknowledge the `SEC_CSID[n]` register by writing the same value. The core must also write to the `SEC_END` register after the ISR execution completes.

1. Read the `SEC_CSID[n]` register to obtain the source ID of the peripheral interrupt request.
2. Write the read value back to the `SEC_CSID[n]` register to send the acknowledge signal to the SEC that the core has accepted and begun processing the interrupt request.
3. Execute the actual ISR (typically a call to a specific handler function from a look-up table based on the peripheral source ID). Write to the `SEC_GCTL` register to enable the SEC.
4. Write the value of the `SEC_CSID[n]` register of the active interrupt (read in step 1 above) to the `SEC_END` register to signal to the SEC that the interrupt has now been serviced.
5. Return from interrupt.

This procedure allows a higher-priority interrupt raised by the SEC to be serviced by the core after step 2. The SEC knows what it passed to the core because of the write to the `SEC_CSID[n]` register. After the core acknowledges that write, the SEC knows whether newly raised peripheral interrupts are a higher priority than the highest-priority interrupt currently being processed by the core.

- When a higher priority, the SEC pushes the current `SEC_CSID[n]` value to an internal stack, writes the new `SEC_CSID[n]` value, and asserts a new SEC interrupt request.
- When a lower priority, the SEC queues the interrupt until the core writes to the `SEC_END` register with the source ID of the higher-priority interrupt, confirming that it was fully processed.

At this point the `SEC_CSID[n]` value is popped from the internal stack and any pending peripheral interrupt requests are arbitrated before the SEC writes the new `SEC_CSID[n]` value and asserts a new interrupt request. At the same time the core self-nests the latched SEC interrupt requests as needed. When a higher-priority interrupt is presented to the core the write to the `SEC_END` register in the SEC handler epilog code guarantees that each nested level has the required handshake to signal to the SEC block that each individual source ID interrupt request is fully serviced. See the *SHARC+ Core Programming Reference* for more details about SEC handler code.

Configuring a System Source as a Fault

Use the following procedure configure a system source as a fault.

1. Write to the `SEC_GCTL` register to enable the SEC.
2. Write to the `SEC_FCTL` register to configure specific fault actions.
3. Write to the `SEC_FDLY` bit field to specify fault delay.
4. Write to the control register of a specific source to enable the source as a fault.

SEC Programming Restrictions

Setting the `SEC_FCTL.EN` bit while the `SEC_FSTAT.ACT` bit is high can result in unpredictable behavior. To avoid this issue, set the `SEC_FCTL.EN` bit while the `SEC_FSTAT.ACT` bit is low. The `SEC_FSTAT.ACT` bit is only set when the `SEC_FCTL.EN` bit is high. Therefore, the problem can only occur if the `SEC_FCTL.EN` bit transitions from 1 to 0 and then to 1 again.

Writing to `SEC_FEND` to end a fault with both the `SEC_FCTL.FOEN` bit and the `SEC_FCTL.FIEN` bit set can result in erroneous external fault detection. If this operation (ending a fault) and configuration (fault input and fault output enabled) are required by the application, clear the `SEC_FCTL.FOEN` bit prior to writing to `SEC_FEND`. The recommended sequence for ending a fault with the `SEC_FCTL.FIEN` or `SEC_FCTL.FOEN==1` is as follows:

1. Clear the `SEC_FCTL.FOEN` bit.
2. Write to the `SEC_FEND` register.
3. Set the `SEC_FCTL.FOEN` bit.

ADSP-21560_21561_21564_21568 SEC Register Descriptions

System Event Controller (SEC) contains the following registers.

Table 6-5: ADSP-21560_21561_21564_21568 SEC Register List

Name	Description
<code>SEC_CACT[n]</code>	SCI Active Register n
<code>SEC_CCTL[n]</code>	SCI Control Register n
<code>SEC_CGMSK[n]</code>	SCI Group Mask Register n
<code>SEC_CPLVL[n]</code>	SCI Priority Level Register n
<code>SEC_CPMSK[n]</code>	SCI Priority Mask Register n
<code>SEC_CPND[n]</code>	Core Pending Register n
<code>SEC_CSID[n]</code>	SCI Source ID Register n
<code>SEC_CSTAT[n]</code>	SCI Status Register n

Table 6-5: ADSP-21560_21561_21564_21568 SEC Register List (Continued)

Name	Description
SEC_END	Global End Register
SEC_FCOPP	Fault COP Period Register
SEC_FCOPP_CUR	Fault COP Period Current Register
SEC_FCTL	Fault Control Register
SEC_FDLY	Fault Delay Register
SEC_FDLY_CUR	Fault Delay Current Register
SEC_FEND	Fault End Register
SEC_FSID	Fault Source ID Register
SEC_FSRDLY	Fault System Reset Delay Register
SEC_FSRDLY_CUR	Fault System Reset Delay Current Register
SEC_FSTAT	Fault Status Register
SEC_GCTL	Global Control Register
SEC_GSTAT	Global Status Register
SEC_RAISE	Global Raise Register
SEC_SCTL[n]	Source Control Register n
SEC_SSTAT[n]	Source Status Register n

SCI Active Register n

The SEC SCI active interrupt register (`SEC_CACT[n]`) contains the source ID and priority of the highest priority active interrupt detected by the SEC prioritizer.

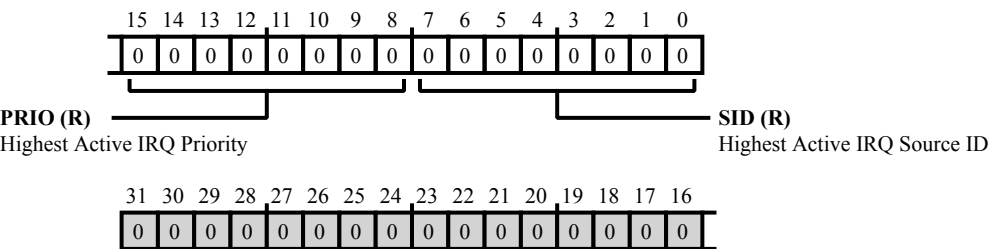


Figure 6-6: SEC_CACT[n] Register Diagram

Table 6-6: SEC_CACT[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:8 (R/NW)	PRIO	Highest Active IRQ Priority. The SEC_CACT[n] . PRIO indicates the priority value of the highest priority active interrupt for core n.
7:0 (R/NW)	SID	Highest Active IRQ Source ID. The SEC_CACT[n] . SID identifies the source ID value of the highest priority active interrupt for core n.

SCI Control Register n

The SEC control register (`SEC_CCTL[n]`) contains SCI control bits for all system sources.

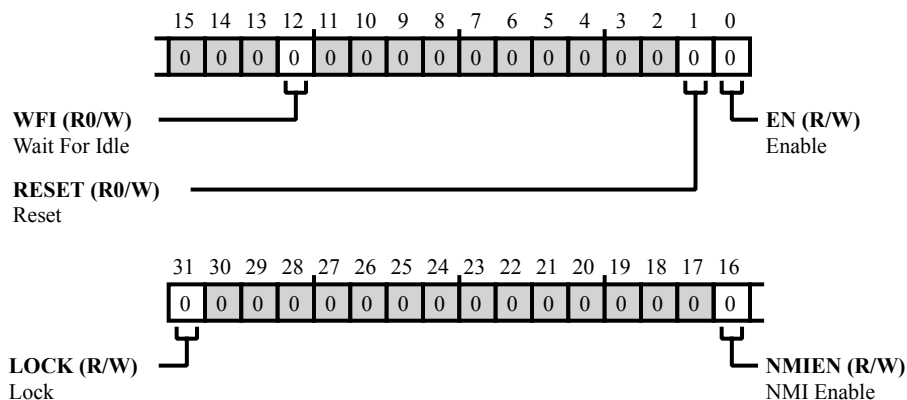


Figure 6-7: SEC_CCTL[n] Register Diagram

Table 6-7: SEC_CCTL[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock.
		If the global lock is enabled (<code>SPU_CTL . GLCK</code> bit =1) and the <code>SEC_CCTL[n] . LOCK</code> bit is enabled, the <code>SEC_CCTL[n]</code> register is read only.
		0 Unlock
		1 Lock
16 (R/W)	NMIEN	NMI Enable.
		The <code>SEC_CCTL[n] . NMIEN</code> bit controls NMI propagation to the core. When the <code>SEC_CCTL[n] . NMIEN</code> bit is enabled, the SCI allows NMIs to propagate to the core for servicing.
		0 Disable
		1 Enable
12 (R0/W)	WFI	Wait For Idle.
		When set, the <code>SEC_CCTL[n] . WFI</code> bit forces the SCI to wait for indication of core idle before the SCI resumes activity.
		0 No Action
		1 Wait for Idle

Table 6-7: SEC_CCTL[n] Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R0/W)	RESET	Reset. When set, the SEC_CCTL[n].RESET bit resets all SCI registers to their default values.
		0 No Action
		1 Reset
0 (R/W)	EN	Enable. The SEC_CCTL[n].EN bit controls operation of the SCI. Clearing the SEC_CCTL[n].EN bit halts the execution of the SCI without resetting status registers. (The INT signal to a core is not affected.) Setting the SEC_CCTL[n].EN bit enables the SCI to begin or resume operation with the current configuration and status.
		0 Disable
		1 Enable

SCI Group Mask Register n

The SEC SCI group mask register ([SEC_CGMSK\[n\]](#)) contains selections for a group mask, an ungroup mask, and a register lock. This register contains the system interrupt group masks for the connected core. The core uses the [SEC_CGMSK\[n\].UGRP](#) and [SEC_CGMSK\[n\].GRP](#) fields to mask (disable) interrupts from the specified groups.

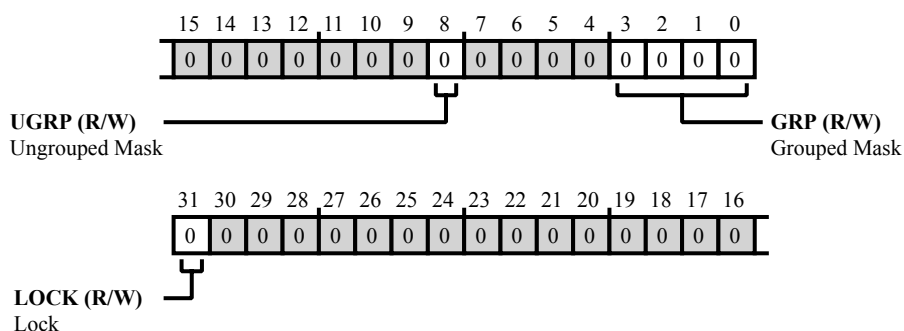


Figure 6-8: SEC_CGMSK[n] Register Diagram

Table 6-8: SEC_CGMSK[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock. If the global lock is enabled (SPU_CTL . GLCK bit =1) and the SEC_CGMSK[n].LOCK bit is enabled, the SEC_CGMSK[n] register is read only.
		0 Unlock
		1 Lock
8 (R/W)	UGRP	Ungrouped Mask. The SEC_CGMSK[n].UGRP bit masks interrupts (if set) for the ungrouped interrupt sources for core n.
		0 Unmask Ungrouped Sources
		1 Mask Ungrouped Sources
3:0 (R/W)	GRP	Grouped Mask. The SEC_CGMSK[n].GRP field selects a group of interrupt sources to mask for core n. (For more information about interrupt source groups, see the SEC_SCTL[n] register description.)
		0 No groups masked
		1 Mask group 0
		2 Mask group 1
		3 Mask groups 0, 1

Table 6-8: SEC_CGMSK[n] Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
		4 Mask group 2
		5 Mask groups 0, 2
		6 Mask groups 1, 2
		7 Mask groups 0, 1, 2
		8 Mask group 3
		9 Mask groups 0, 3
		10 Mask groups 1, 3
		11 Mask groups 0, 1, 3
		12 Mask groups 2, 3
		13 Mask groups 0, 2, 3
		14 Mask groups 1, 2, 3
		15 Mask groups 0, 1, 2, 3

SCI Priority Level Register n

The SEC SCI priority level register ([SEC_CPLVL\[n\]](#)) contains selections for priority levels and a register lock. This register is used to divide the total number of priority levels into sub-levels. The sub-level priority resolution provides the tie breaker for simultaneously pending interrupts assigned to the same level.

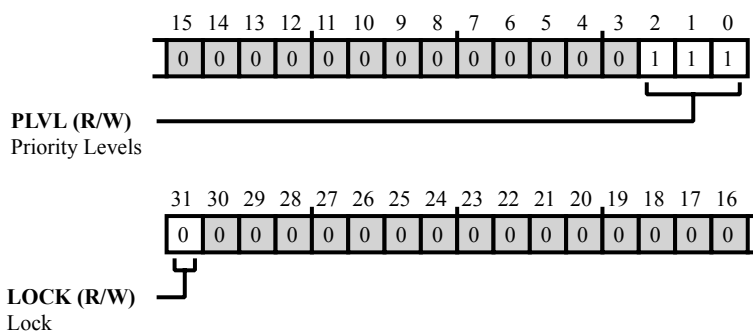


Figure 6-9: SEC_CPLVL[n] Register Diagram

Table 6-9: SEC_CPLVL[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock. If the global lock is enabled (SPU_CTL . GLCK bit =1) and the SEC_CPLVL[n] . LOCK bit is enabled, the SEC_CPLVL[n] register is read only.
		0 Unlock
		1 Lock
2:0 (R/W)	PLVL	Priority Levels. The SEC_CPLVL[n] . PLVL field serves to divide the total number of interrupt priority levels into sub-levels. The sub-level priority resolution provides the tie breaker for simultaneously pending interrupts assigned to the same interrupt level. The sub-level priority value specifies the number of MSBs (minus 1) designated to interrupt levels, while the remaining LSBs are designated for sub-level specification. For example, if the SEC_CPLVL[n] . PLVL field is set to two, the result is four priority levels are specified, because only the two MSBs are used for preemption evaluation. The remaining bits of the priority setting are used for sub-level prioritization.
		0 1 MSBs (2 priority levels)
		1 2 MSBs (4 priority levels)
		2 3 MSBs (8 priority levels)
		3 4 MSBs (16 priority levels)
		4 5 MSBs (32 priority levels)
		5 6 MSBs (64 priority levels)

Table 6-9: SEC_CPLVL[n] Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
		6	7 MSBs (128 priority levels)
		7	8 MSBs (256 priority levels)

SCI Priority Mask Register n

The SEC SCI priority mask register (`SEC_CPMSK[n]`) contains the SCI priority mask for core n and includes a register lock.

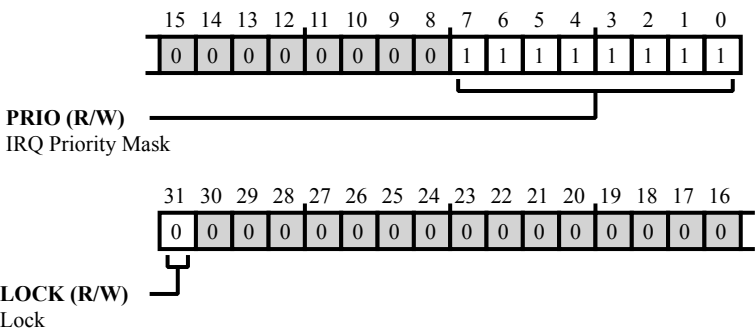


Figure 6-10: SEC_CPMSK[n] Register Diagram

Table 6-10: SEC_CPMSK[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock. If the global lock is enabled (<code>SPU_CTL.GLCK</code> bit =1) and the <code>SEC_CPMSK[n].LOCK</code> bit is enabled, the <code>SEC_CPMSK[n]</code> register is read only.
		0 Unlock
		1 Lock
7:0 (R/W)	PRIO	IRQ Priority Mask. The <code>SEC_CPMSK[n].PRIO</code> contains the system interrupt priority mask for core n. The core uses the <code>SEC_CPMSK[n].PRIO</code> field to mask (block) interrupts below the specified level.
		0 Priority level 0 (highest)
		1-254
		255 Priority level 255 (lowest)

Core Pending Register n

The SCI pending interrupt register (`SEC_CPND[n]`) contains the source ID and priority of the highest priority pending interrupt detected by the SEC prioritizer.

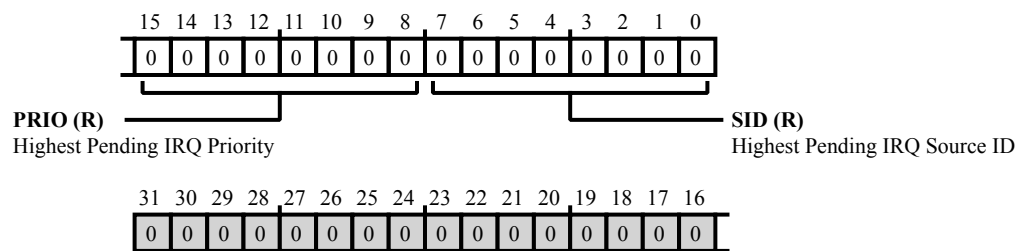


Figure 6-11: SEC_CPND[n] Register Diagram

Table 6-11: SEC_CPND[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:8 (R/NW)	PRIO	Highest Pending IRQ Priority. The SEC_CPND[n] . PRIO indicates the priority value of the highest priority pending interrupt for core n.
7:0 (R/NW)	SID	Highest Pending IRQ Source ID. The SEC_CPND[n] . SID identifies the source ID value of the highest priority pending interrupt for core n.

SCI Source ID Register n

The SCI source ID register (`SEC_CSID[n]`) contains the source ID of the interrupt last issued to core n. The `SEC_CSID[n]` register value is loaded by the SCI when a system interrupt indication is sent to core n. The SCI does not change the `SEC_CSID[n]` until after the interface receives an interrupt acknowledge from core n. Writing to the `SEC_CSID[n]` register generates an interrupt acknowledge, but does not update the value in the register.

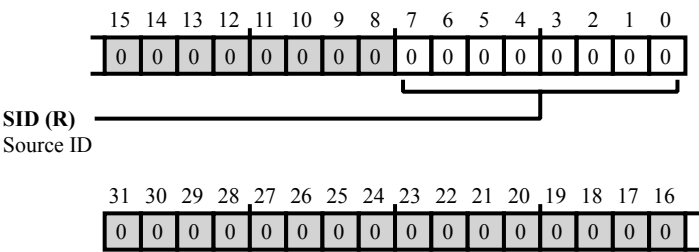


Figure 6-12: SEC_CSID[n] Register Diagram

Table 6-12: SEC_CSID[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	SID	Source ID. The <code>SEC_CSID[n].SID</code> bit is the source ID of the interrupt last issued to core n.

SCI Status Register n

The SCI status register ([SEC_CSTAT\[n\]](#)) contains status bits, indicating the operational status of the SCI.

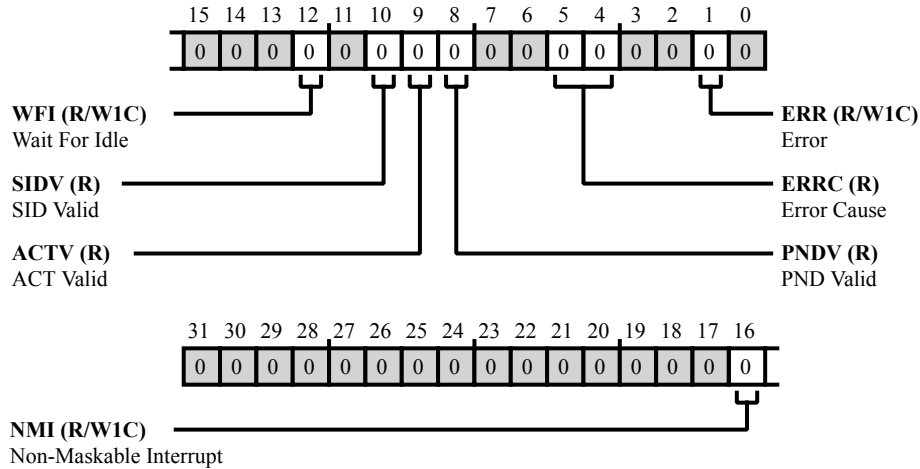


Figure 6-13: SEC_CSTAT[n] Register Diagram

Table 6-13: SEC_CSTAT[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
16 (R/W1C)	NMI	Non-Maskable Interrupt. The SEC_CSTAT[n].NMI bit indicates whether an NMI has occurred since the bit was last cleared.
		0 No NMI Occurred
		1 NMI Occurred
12 (R/W1C)	WFI	Wait For Idle. The SEC_CSTAT[n].WFI bit indicates (if set) that the SCI is temporarily disabled, pending a core idle indication. This bit is set when SEC_CCTL[n].WFI is set.
		0 Not Waiting
		1 Waiting
10 (R/NW)	SIDV	SID Valid. The SEC_CSTAT[n].SIDV bit indicates (if set) that the current value in the SEC_CSID[n] register is valid. The SCI sets the SEC_CSTAT[n].SIDV bit when the updating the SEC_CSID[n] register with a new value. The SEC_CSTAT[n].SIDV bit is cleared when the SEC_CSID[n] register is written. This status indication may be used to extract all pending interrupts in a single interrupt service routine.
		0 Invalid
		1 Valid

Table 6-13: SEC_CSTAT[n] Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
9 (R/NW)	ACTV	ACT Valid. The SEC_CSTAT[n].ACTV bit indicates (if set) that the current value in the SEC_CACT[n] register is valid. The SCI sets the SEC_CSTAT[n].ACTV bit when updating the SEC_CACT[n] registers with a new value. The SEC_CSTAT[n].ACTV bit is cleared when the SEC_CSID[n] register is written.
		0 Invalid
		1 Valid
8 (R/NW)	PNDV	PND Valid. The SEC_CSTAT[n].PNDV bit indicates (if set) that the current value in the SEC_CPND[n] register is valid. The SCI sets the SEC_CSTAT[n].PNDV bit when updating the SEC_CPND[n] register with a new value. The SEC_CSTAT[n].PNDV bit is cleared when the SEC_CSID[n] register is written.
		0 Invalid
		1 Valid
5:4 (R/NW)	ERRC	Error Cause. The SEC_CSTAT[n].ERRC bits are updated on assertion of the SEC_CSTAT[n].ERR bit to indicate the SCI error type. SEC_CSTAT[n].ERRC is only updated on the assertion of SEC_CSTAT[n].ERR. Subsequent errors while SEC_CSTAT[n].ERR is asserted do not update SEC_CSTAT[n].ERRC.
		0 Reserved
		1 Acknowledge Error. SCI has received an acknowledge without a pending, unacknowledged interrupt present.
		2 Reserved
		3 Reserved
1 (R/W1C)	ERR	Error. The SEC_CSTAT[n].ERR bit indicates that an error has occurred in the SCI. When SEC_CSTAT[n].ERR is set, the SCI updates the SEC_CSTAT[n].ERRC field to the value of the corresponding error cause.
		0 No Error
		1 Error Occurred

Global End Register

The SEC global end register (`SEC_END`) contains a source ID interrupt service end field (`SEC_END.SID`). When a core has finished servicing an interrupt, the core writes the `SEC_END.SID` field in the `SEC_END` register. This write causes the SEC to clear the `SEC_SSTAT[n].ACT` bit in the `SEC_SSTAT[n]` register of the corresponding interrupt.

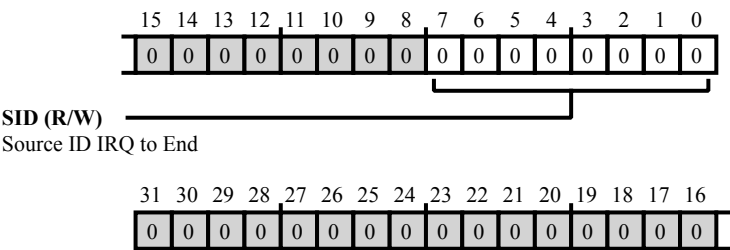


Figure 6-14: SEC_END Register Diagram

Table 6-14: SEC_END Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/W)	SID	Source ID IRQ to End. The SEC_END.SID bit field contains the source ID interrupt service end value.

Fault COP Period Register

The SEC fault COP period register ([SEC_FCOPP](#)) contains the width value (count in (SEC) clock cycles) for the high and low phase of the computer operating properly (COP) toggled output on the COP pin. Note that the actual high/low phase value is the `SEC_FCOPP.COUNT` programmed value plus 1.

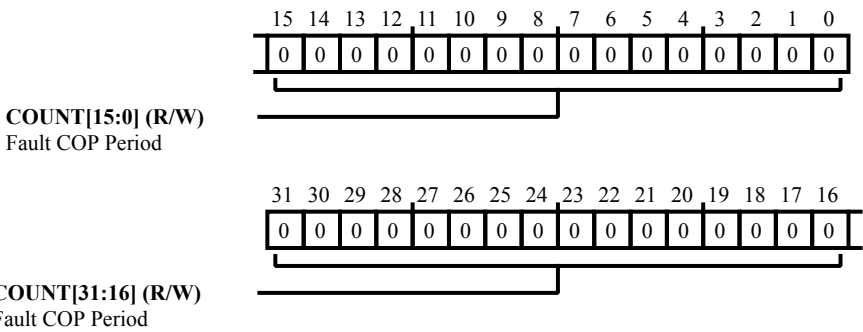


Figure 6-15: SEC_FCOPP Register Diagram

Table 6-15: SEC_FCOPP Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	COUNT	Fault COP Period. The <code>SEC_FCOPP.COUNT</code> bit field is the width value for the high and low phase of the computer operating properly (COP) toggled output on the COP pin.

Fault COP Period Current Register

The SEC fault COP period current register ([SEC_FCOPP_CUR](#)) contains the active count (in (SEC) clock periods) for the current phase (high or low) of the computer operating properly (COP) toggled output on the COP pin. The SEC loads the [SEC_FCOPP_CUR](#) register from the [SEC_FCOPP](#) register when the SEC_FCOPP_CUR.COUNT field is cleared and the SEC is in COP mode (SEC_FCTL.CMS bit =1). The SEC decrements the [SEC_FCOPP_CUR](#) count each (SEC) clock cycle while SEC_FCTL.CMS is set and the SEC_FSTAT.ACT bit is not set.

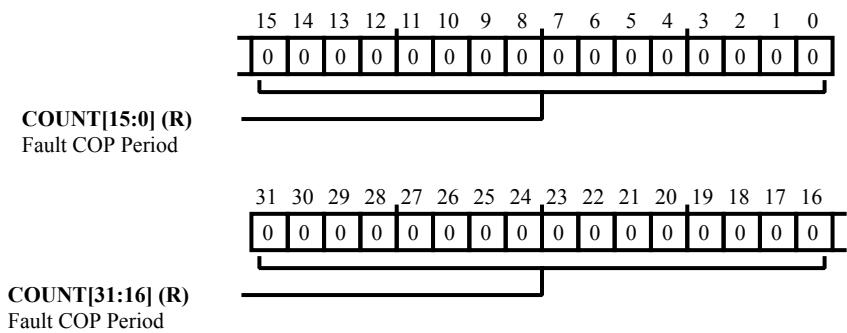


Figure 6-16: SEC_FCOPP_CUR Register Diagram

Table 6-16: SEC_FCOPP_CUR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	COUNT	Fault COP Period. The SEC_FCOPP_CUR.COUNT bit field is the active count for the current phase (high or low) of the computer operating properly (COP) toggled output on the COP pin.

Fault Control Register

The SEC fault control register ([SEC_FCTL](#)) contains fault control bits for all SEC channels. This register controls the operation of the System Fault Management Interface (SFI).

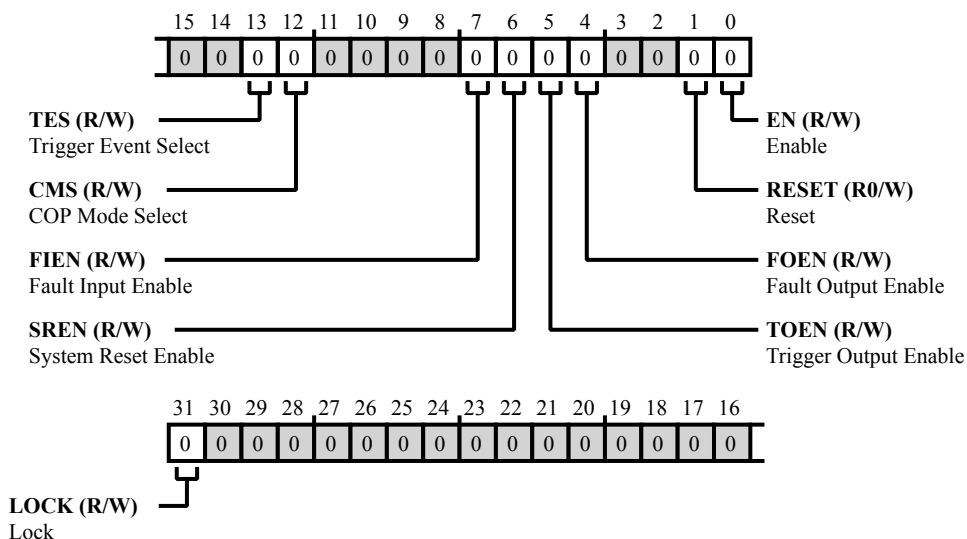


Figure 6-17: SEC_FCTL Register Diagram

Table 6-17: SEC_FCTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock. If the global lock is enabled (SPU_CTL.GLCK bit =1) and the SEC_FCTL.LOCK bit is enabled, the SEC_FCTL register is read only.
		0 UnLock
		1 Lock
13 (R/W)	TES	Trigger Event Select. The SEC_FCTL.TES bit selects the event that directs the SEC to assert trigger output. In fault pending mode, the SEC asserts trigger output when a fault is pending. In fault active mode, the SEC asserts trigger output when a fault is active.
		0 Fault Active Mode
		1 Fault Pending Mode

Table 6-17: SEC_FCTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
12 (R/W)	CMS	COP Mode Select. The SEC_FCTL.CMS selects the SEC mode for handling fault input. In COP mode, the SEC toggles the COP pin to indicate that no fault is active and ceases toggling the pin to indicate that a fault is active. In fault mode, the SEC deasserts the fault pin (=0) and fault_b pin (=1) when no fault is active and asserts the fault pin (=1) and fault_b pin (=0) when a fault is active. Not all processors feature both the fault and fault_b pins. Refer to the product data sheet for details.
		0 Fault Mode
		1 COP Mode
7 (R/W)	FIEN	Fault Input Enable. The SEC_FCTL.FIEN bit enables the SEC to sample fault input. If SEC_FCTL.FIEN is set (=1), a fault indication from an external device sets the SEC_FSTAT.ACT bit and SEC_FSID.FEXT bit.
		0 Disable
		1 Enable
6 (R/W)	SREN	System Reset Enable. The SEC_FCTL.SREN bit enables the SEC to issue a system reset request when a fault becomes active.
		0 Disable
		1 Enable
5 (R/W)	TOEN	Trigger Output Enable. The SEC_FCTL.TOEN bit enables the SEC to produce trigger output when a fault becomes active.
		0 Disable
		1 Enable
4 (R/W)	FOEN	Fault Output Enable. The SEC_FCTL.FOEN bit enables the SEC to indicate fault status, according to the SEC_FCTL.CMS bit configuration.
		0 Disable
		1 Enable
1 (R0/W)	RESET	Reset. Setting the SEC_FCTL.RESET bit resets ALL SEC registers to their default values.
		0 No Action
		1 Reset

Table 6-17: SEC_FCTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	EN	Enable. The SEC_FCTL.EN bit controls the operational state of the SEC. Clearing the SEC_FCTL.EN bit halts the execution of the SEC without resetting status registers. Setting the SEC_FCTL.EN bit enables the SEC to begin or resume operation with the current configuration and status.
		0 Disable
		1 Enable

Fault Delay Register

The SEC fault delay register ([SEC_FDLY](#)) contains the number ([SEC_FDLY . COUNT](#) field) of (SEC) clock periods to delay from fault pending to fault active, when actions are enabled.

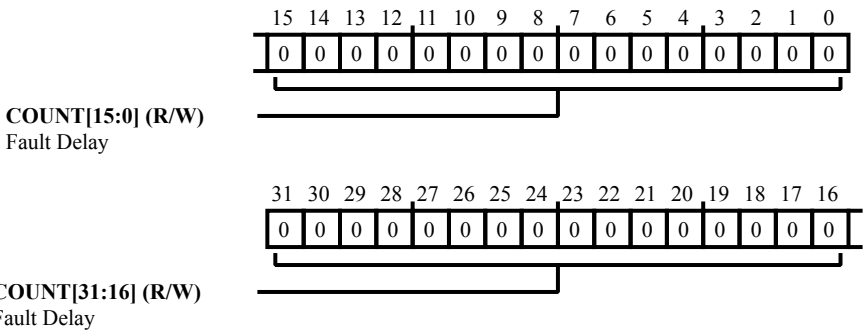


Figure 6-18: SEC_FDLY Register Diagram

Table 6-18: SEC_FDLY Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	COUNT	Fault Delay. The SEC_FDLY . COUNT bit field is the number of (SEC) clock periods to delay from fault pending to fault active, when actions are enabled.

Fault Delay Current Register

The SEC fault delay current register (`SEC_FDLY_CUR`) contains the active count (`SEC_FDLY_CUR.COUNT` field) in (SEC) clock periods for the delay from fault pending to fault active, when actions are enabled. The count is loaded from the `SEC_FDLY` register when a fault becomes pending (`SEC_FSTAT.PND` bit is set). The SEC decrements the value in `SEC_FDLY_CUR` each (SEC) clock cycle while the `SEC_FSTAT.PND` bit is set.

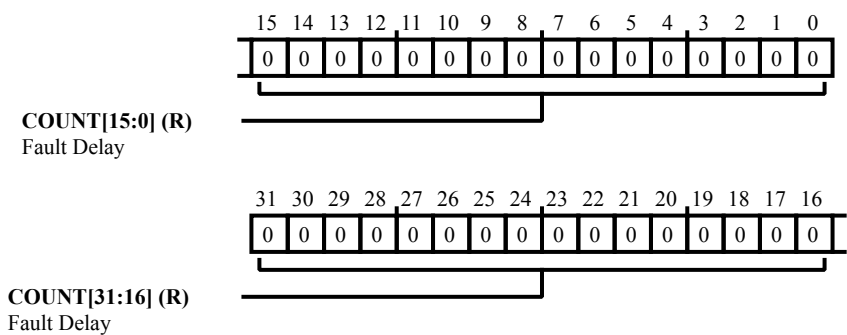


Figure 6-19: SEC_FDLY_CUR Register Diagram

Table 6-19: SEC_FDLY_CUR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	COUNT	Fault Delay. The <code>SEC_FDLY_CUR.COUNT</code> bit field is the active count in (SEC) clock periods for the delay from fault pending to fault active, when actions are enabled.

Fault End Register

The SEC fault end register (`SEC_FEND`) contains fault source ID and internal/external fields. This register receives fault end indication from a core.

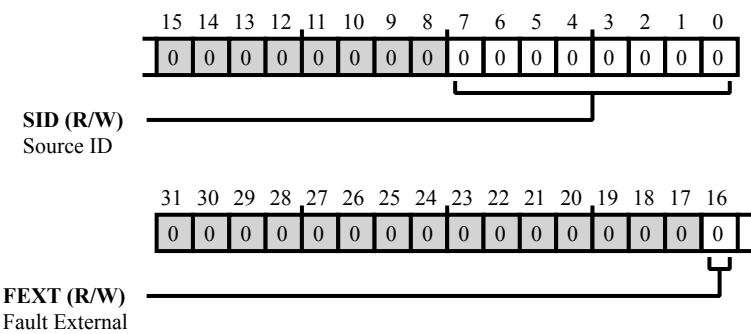


Figure 6-20: SEC_FEND Register Diagram

Table 6-20: SEC_FEND Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
16 (R/W)	FEXT	Fault External. Setting the SEC_FEND.FEXT bit, when the SEC_FEND.SID field is cleared, clears an active fault from an external source.
		0 Fault Internal
		1 Fault External
7:0 (R/W)	SID	Source ID. The SEC_FEND.SID identifies a fault to be ended as indicated to the SEC by the core. The core loads the SEC_FEND.SID field value. If the SEC_FEND.SID value matches the SEC_FSID.SID value, the SEC_FSTAT.PND bit and SEC_FSTAT.ACT bit are cleared.

Fault Source ID Register

The SEC fault source ID register ([SEC_FSID](#)) contains a fault source ID and internal/external fields.

NOTE:These bits are not reset by system reset so that a fault that automatically triggers a system reset to avoid a fault may be analyzed after the reset occurs.

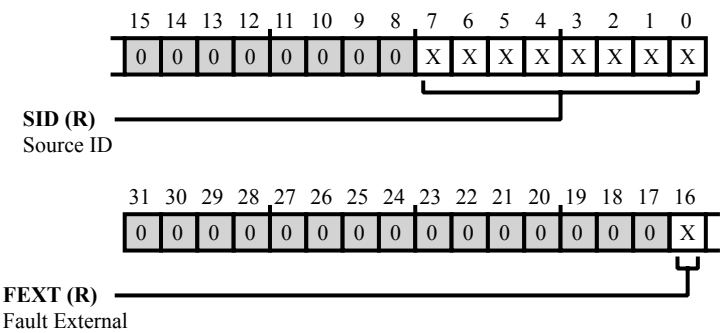


Figure 6-21: SEC_FSID Register Diagram

Table 6-21: SEC_FSID Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
16 (R/NW)	FEXT	Fault External. The SEC_FSID.FEXT bit indicates that the last active fault was asserted by an external device. The SEC sets the SEC_FSID.FEXT bit when the SEC_FSTAT.ACT bit is set by the fault input pins. The SEC_FSID.FEXT bit is cleared when the SEC_FSTAT.ACT bit is set by an internal fault or when the external fault is ended. When the SEC_FSID.FEXT bit is set, the SEC_FSID.SID is cleared.
		0 Fault Internal
		1 Fault External
7:0 (R/NW)	SID	Source ID. The SEC_FSID.SID identifies the fault assertion detected by the SEC fault interface. The SEC loads the SEC_FSID.SID field value when a system fault indication is asserted. The SEC fault interface does not change the SEC_FSID.SID value until the fault is no longer pending or active, as indicated by the SEC_FSTAT.PND bit and SEC_FSTAT.ACT bit being cleared in the SEC_FSTAT register.

Fault System Reset Delay Register

The SEC fault system reset delay register ([SEC_FSRDLY](#)) contains the number (`SEC_FSRDLY.COUNT` field) of (SEC) clock periods for the delay from a fault becoming active to system reset request assertion, if enabled.

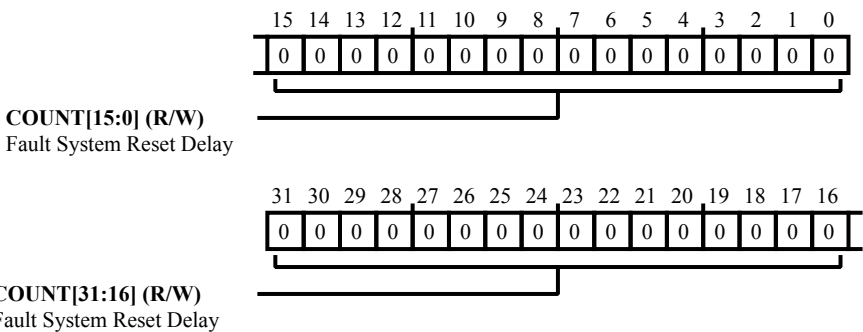


Figure 6-22: SEC_FSRDLY Register Diagram

Table 6-22: SEC_FSRDLY Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	COUNT	Fault System Reset Delay. The <code>SEC_FSRDLY.COUNT</code> bit field is the number of (SEC) clock periods for the delay from a fault becoming active to system reset request assertion.

Fault System Reset Delay Current Register

The SEC fault system reset delay current register (`SEC_FSRDLY_CUR`) contains the active count (`SEC_FSRDLY_CUR.COUNT` field) in (SEC) clock periods for the delay from fault active to system reset assertion, if enabled. The count is loaded from the `SEC_FSRDLY` register when a fault becomes active (`SEC_FSTAT.ACT` bit is set). The SEC decrements the value in `SEC_FSRDLY_CUR` each (SEC) clock cycle while the `SEC_FSTAT.ACT` bit is set.

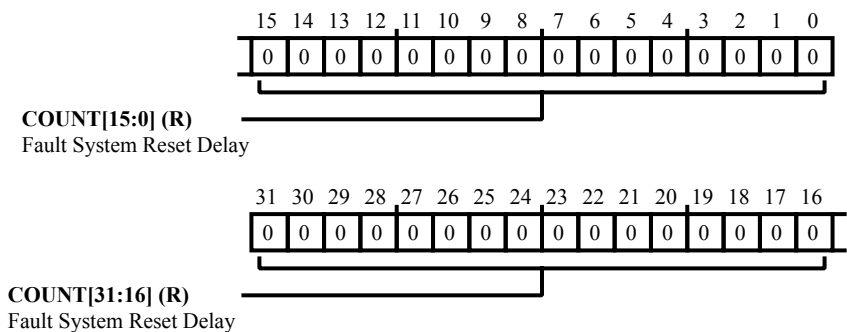


Figure 6-23: SEC_FSRDLY_CUR Register Diagram

Table 6-23: SEC_FSRDLY_CUR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	COUNT	Fault System Reset Delay. The <code>SEC_FSRDLY_CUR.COUNT</code> bit field is the active count in (SEC) clock periods for the delay from fault active to system reset assertion.

Fault Status Register

The SEC fault status register ([SEC_FSTAT](#)) indicates the operational status of the SFI.

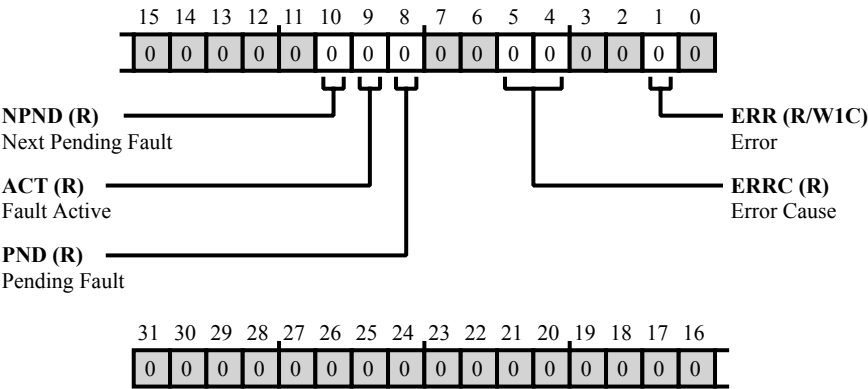


Figure 6-24: SEC_FSTAT Register Diagram

Table 6-24: SEC_FSTAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
10 (R/NW)	NPND	Next Pending Fault. The SEC_FSTAT.NPND bit indicates that one or more sources have signaled fault assertion, but the input has not yet triggered the fault pending detection in the SEC fault interface. The SEC sets the SEC_FSTAT.NPND bit when the fault interface detects assertion of any enabled fault source input, while either the SEC_FSTAT.PND or SEC_FSTAT.ACT bits are set. The SEC clears the SEC_FSTAT.NPND bit when there are no fault sources waiting.
		0 Not Pending
		1 Pending
9 (R/NW)	ACT	Fault Active. The SEC_FSTAT.ACT bit indicates that the SEC has received a fault source input, the current fault delay count (in the SEC_FDLY_CUR register) has expired, and the fault actions are enabled. The SEC also sets the SEC_FSTAT.ACT bit on fault input detection if the SEC_FCTL.FIEN bit is set. The SEC_FSTAT.ACT bit is cleared by writing the ID value of the asserted fault from SEC_FSID register to the SEC_FEND register.
		0 No Fault
		1 Active Fault

Table 6-24: SEC_FSTAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
8 (R/NW)	PND	Pending Fault. The SEC_FSTAT.PND bit indicates a fault source has signaled a fault assertion to the SEC, but the SEC has not yet triggered the event actions due to the delay selected with the SEC_FDLY register. The SEC fault interface sets the SEC_FSTAT.PND bit when the SEC_FSID is updated on assertion of a fault source input. The SEC_FSTAT.PND bit is only set when the SEC_FSTAT.ACT bit is cleared. The SEC updates the SEC_FSID register with the SID value when the SEC_FSTAT.PND bit is set. The SEC_FSTAT.PND bit is cleared <i>either</i> by the SEC fault interface when the current delay count in the SEC_FDLY_CUR register expires <i>or</i> by writing the SEC_FSID.SID field value (which indicates the ID of the asserted fault) to the SEC_FEND register.
		0 Not Pending
		1 Pending
5:4 (R/NW)	ERRC	Error Cause. When the SEC_FSTAT.ERR bit is asserted, the SEC updates SEC_FSTAT.ERRC field to convey the interrupt source error type. When the error type is source overflow, the status indicates that a source signal assertion occurred or an SEC raise operation was attempted while pending was already set. The source overflow is detected when the source is set for edge only. When the error type is end error, the status indicates that an end was received for a source while the SEC_FSTAT.ACT bit was not set.
		0 Source Overflow Error
		1 Reserved
		2 End Error
		3 Reserved
1 (R/W1C)	ERR	Error. The SEC_FSTAT.ERR bit indicates an SEC fault interface error. When SEC_FSTAT.ERR is set, the SEC updates the SEC_FSTAT.ERRC field to indicate the corresponding error cause. When multiple errors occur, the SEC_FSTAT register captures the status for the first error and does not capture subsequent errors until the status is cleared.
		0 No Error
		1 Error Occurred

Global Control Register

The SEC global control register ([SEC_GCTL](#)) provides register locking, reset, and enable for the SEC module.

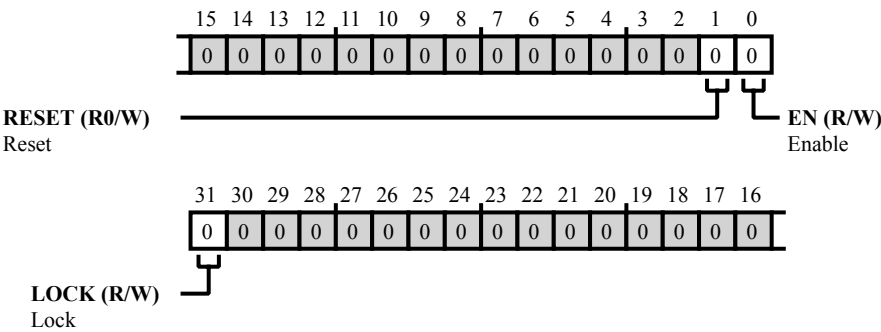


Figure 6-25: SEC_GCTL Register Diagram

Table 6-25: SEC_GCTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock. If the global lock is enabled (SPU_CTL . GLCK bit =1) and the SEC_GCTL . LOCK bit is enabled, the SEC_GCTL register is read only.
		0 Unlock
		1 Lock
1 (R0/W)	RESET	Reset. The SEC_GCTL . RESET bit is write-1-action and triggers a soft reset to all SEC registers.
		0 No Action
		1 Reset
0 (R/W)	EN	Enable. The SEC_GCTL . EN bit is read/write and must be set for the SEC to begin/resume SEC operation with the current configuration and status. Clearing the SEC_GCTL . EN bit halts the execution of the SFI and all SCIs. All SSIs remain active, along with all error detection, without resetting status registers.
		0 Disable
		1 Enable

Global Status Register

The SEC global status register ([SEC_GSTAT](#)) contains global status bits for the SEC.

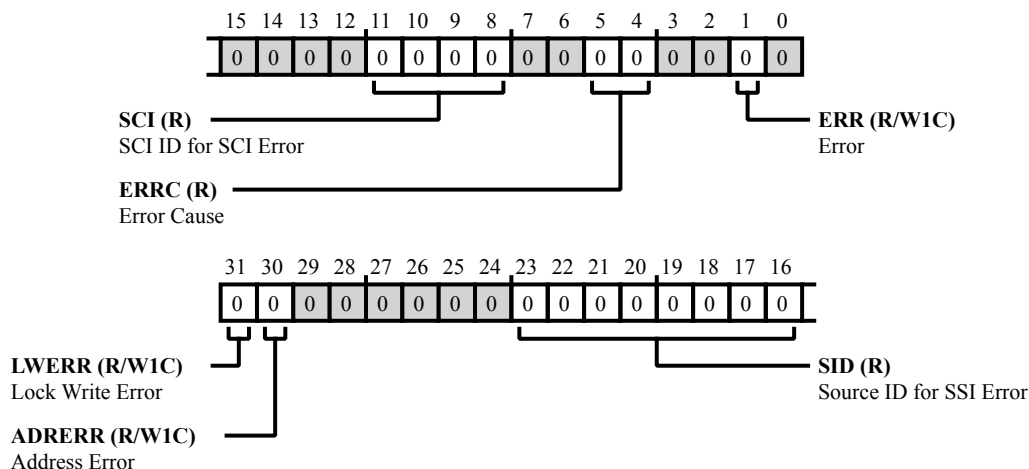


Figure 6-26: SEC_GSTAT Register Diagram

Table 6-26: SEC_GSTAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W1C)	LWERR	Lock Write Error. The SEC_GSTAT.LWERR bit indicates (when set) there was an attempted write to an SEC register while the SEC_GCTL.LOCK bit was set and while the global lock bit was enabled (SPU_CTL.GLCK bit =1). This status bit is sticky; write-1-to-clear it.
		0 No Error
		1 Error Occurred
30 (R/W1C)	ADRERR	Address Error. The SEC_GSTAT.ADRERR bit indicates that the SEC generated an address error. This status bit is sticky; write-1-to-clear it.
		0 No Error
		1 Error Occurred
23:16 (R/NW)	SID	Source ID for SSI Error. The SEC_GSTAT.SID bits indicate the source ID that generated the last SSI error conveyed in the SEC_GSTAT.ERRC field.
11:8 (R/NW)	SCI	SCI ID for SCI Error. The SEC_GSTAT.SCI bits indicate the number for the specific SCI that generated the last SCI error conveyed in the SEC_GSTAT.ERRC field.

Table 6-26: SEC_GSTAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
5:4 (R/NW)	ERRC	Error Cause. When the SEC updates the SEC_GSTAT.ERR bit, the SEC updates the SEC_GSTAT.ERRC bits to indicate the error type. Note that for SCI errors, the error status represents an OR of all the errors from each SCI. Note that for SSI errors, the error status indicates an error is active for any SSI input. This error is an OR of all the interrupt source errors.
		0 SFI Error
		1 SCI Error
		2 SSI Error
		3 Reserved
1 (R/W1C)	ERR	Error. The SEC_GSTAT.ERR bit indicates an error has occurred in the SEC. When the SEC asserts this bit (=1), the SEC updates the SEC_GSTAT.ERRC field to indicate the corresponding error cause. Even if multiple errors occur, only the first error is captured on assertion of this bit. This status bit is sticky; write-1-to-clear it.
		0 No Error
		1 Error Occurred

Global Raise Register

The SEC global raise register ([SEC_RAISE](#)) contains a source ID event set-to-pending field (`SEC_RAISE.SID`). When a source ID value is written to this field, the SEC raises the source's event status to pending.

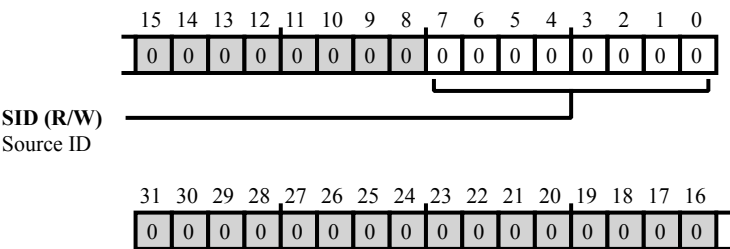


Figure 6-27: SEC_RAISE Register Diagram

Table 6-27: SEC_RAISE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/W)	SID	Source ID. The <code>SEC_RAISE.SID</code> bit field is the source ID of event that is set to pending status.

Source Control Register n

The SEC source control register (`SEC_SCTL[n]`) contains control bits to configure the SEC event sources. This register controls the configuration of the corresponding SEC event source.

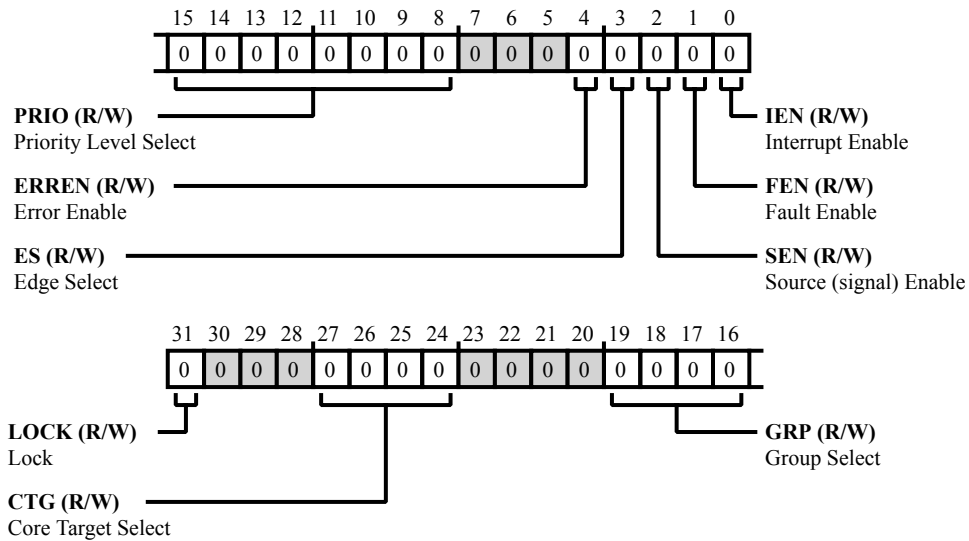


Figure 6-28: SEC_SCTL[n] Register Diagram

Table 6-28: SEC_SCTL[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock. If the global lock is enabled (<code>SPU_CTL.GLCK</code> bit =1) and the <code>SEC_SCTL[n].LOCK</code> bit is enabled, the <code>SEC_SCTL[n]</code> register is read only.
		0 Unlock
		1 Lock
27:24 (R/W)	CTG	Core Target Select. The <code>SEC_SCTL[n].CTG</code> bits selects the specific SEC core interface to which the interrupt is mapped. Each system interrupt is mapped uniquely to one specific SEC core interface and (as a result) to a specific core.
		1 CORE1 SHARC0
		2 Reserved

Table 6-28: SEC_SCTL[n] Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration				
19:16 (R/W)	GRP	Group Select. The SEC_SCTL[n] .GRP bits each select a specific group for the interrupt. Each system interrupt can be assigned to any combination of groups supported by the SEC_SCTL[n] .GRP field. For example, consider the situation where SEC_SCTL[n] .GRP0 represents interrupt group 0, SEC_SCTL[n] .GRP1 represents interrupt group 1, and so on. One group might be used for all enabled interrupts (for example, group 0) and an additional group might be used for all wakeup interrupts (for example, group 1). This approach supports a model of all interrupts and just the wakeup subset. Before going to idle or sleep, all non-wakeup interrupts can be masked off to allow only wakeup interrupts to be enabled for service. Selecting no group (all SEC_SCTL[n] .GRP bits = 0) places the interrupt source in the category of "un-grouped".				
15:8 (R/W)	PRI0	Priority Level Select. The SEC_SCTL[n] .PRI0 bits sets the relative priority for an interrupt request. A pending interrupt request forwards its SEC_SCTL[n] .PRI0 value to the SEC core interface.				
4 (R/W)	ERREN	Error Enable. The SEC_SCTL[n] .ERREN bit permits the SEC_SSTAT[n] .ERR status bit to be set on error detection. If SEC_SCTL[n] .ERREN is cleared, no errors are detected. <table><tr><td>0</td><td>Disable</td></tr><tr><td>1</td><td>Enable</td></tr></table>	0	Disable	1	Enable
0	Disable					
1	Enable					
3 (R/W)	ES	Edge Select. The SEC_SCTL[n] .ES bit selects the operational and sensitivity mode of the SEC source interface input. <table><tr><td>0</td><td>Level Sensitive</td></tr><tr><td>1</td><td>Edge Sensitive</td></tr></table>	0	Level Sensitive	1	Edge Sensitive
0	Level Sensitive					
1	Edge Sensitive					
2 (R/W)	SEN	Source (signal) Enable. The SEC_SCTL[n] .SEN bit controls whether the system event source input signal may affect the pending status of the source. Clearing the SEC_SCTL[n] .SEN bit disables the source input signal from affecting the pending status. Setting SEC_SCTL[n] .SEN enables the source input signal to affect the pending status. <table><tr><td>0</td><td>Disable</td></tr><tr><td>1</td><td>Enable</td></tr></table>	0	Disable	1	Enable
0	Disable					
1	Enable					

Table 6-28: SEC_SCTL[n] Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/W)	FEN	Fault Enable. The SEC_SCTL[n] . FEN bit controls whether the SEC may forward an interrupt request to the SEC fault interface as a fault source. This bit does not affect the ability of an interrupt source to set an interrupt as pending. The SEC_SCTL[n] . FEN bit only affects whether the pending request may be forwarded to the SEC fault interface.
		0 Disable
		1 Enable
0 (R/W)	IEN	Interrupt Enable. The SEC_SCTL[n] . IEN bit controls whether the SEC may forward an interrupt request to a core for servicing. This bit does not affect the ability of an interrupt source to set an interrupt as pending.
		0 Disable
		1 Enable

Source Status Register n

The SEC event source status register (`SEC_SSTAT[n]`) contains bits indicating the status of the corresponding event source n. An event source may be: pending, active, active and pending, or neither pending nor active.

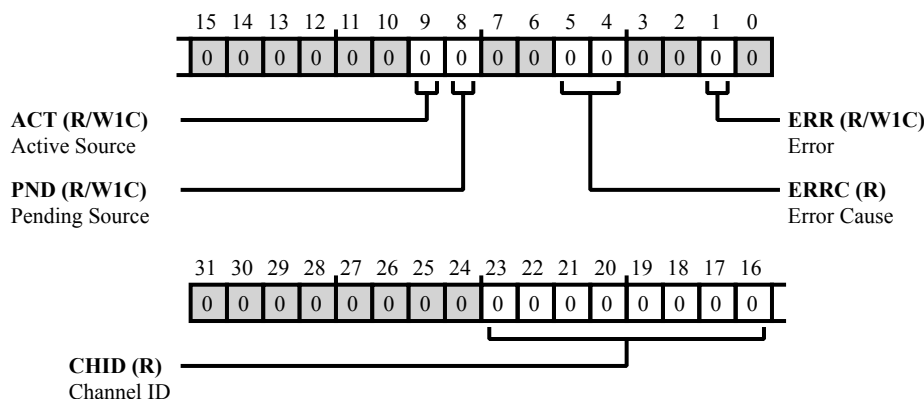


Figure 6-29: SEC_SSTAT[n] Register Diagram

Table 6-29: SEC_SSTAT[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
23:16 (R/NW)	CHID	Channel ID. The <code>SEC_SSTAT[n].CHID</code> bits indicate the ID of the specific source (from a set of sources sharing one SEC source interface input) that asserted the SEC source interface input. An SEC source interface input may support multiple system sources, in which case the assertion must be qualified by an identifier to determine the channel that generated the assertion. The <code>SEC_SSTAT[n].CHID</code> field provides this value in the form of a numeric reference that is mapped to a specific interrupt source. The prioritization for simultaneously asserted sources is according to ID, with 0 being the highest priority. The <code>SEC_SSTAT[n].CHID</code> is captured when the SEC source interface input is acknowledged.
9 (R/W1C)	ACT	Active Source. The <code>SEC_SSTAT[n].ACT</code> bit indicates the source has been accepted by a core for servicing, but the service is not yet complete. An <code>SEC_SSTAT[n].ACT</code> bit is set by the SEC when the specific system interrupt is acknowledged by the core through the SEC core interface. An <code>SEC_SSTAT[n].ACT</code> bit is cleared by the SEC when the core provides interrupt service end indication for the specific system interrupt through the SEC core interface.
		0 Not Active
		1 Active

Table 6-29: SEC_SSTAT[n] Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
8 (R/W1C)	PND	Pending Source. The SEC_SSTAT[n].PND bit indicates the source has signaled an event request, but the event request has not been (or is not currently being) serviced. A SEC_SSTAT[n].PND bit is set by the SEC on detection of an assertion of the corresponding system source input. A SEC_SSTAT[n].PND bit is cleared by the SEC when the specific system event is acknowledged by the core through the SEC core interface or by a W1C operation.
		0 Not Pending
		1 Pending
5:4 (R/NW)	ERRC	Error Cause. When the SEC_SSTAT[n].ERR bit is asserted, the SEC updates SEC_SSTAT[n].ERRC field to convey the interrupt source error type. When the error type is source overflow, the status indicates that a source signal assertion occurred or an SEC raise operation was attempted while pending was already set. The source overflow is detected when the source is set for edge only. When the error type is end error, the status indicates that an end was received for a source while the SEC_SSTAT[n].ACT bit was not set.
		0 Source Overflow Error
		1 Reserved
		2 End Error
		3 Reserved
1 (R/W1C)	ERR	Error. The SEC_SSTAT[n].ERR bit indicates an error for a specific system interrupt source. When the SEC_SSTAT[n].ERR bit is set, the SEC updates the SEC_SSTAT[n].ERRC field to the value of the corresponding error cause. Even if multiple errors occur, only the first error is captured on assertion of the SEC_SSTAT[n].ERR bit.
		0 No Error
		1 Error Occurred

7 Trigger Routing Unit (TRU)

The TRU provides system-level sequence control without core intervention. The TRU maps trigger generators to trigger receivers. Receiver endpoints can be configured to respond to triggers in various ways. Multiple TRUs may be provided in a multiprocessor system to create a trigger network. Common applications enabled by the TRU include:

- Automatically triggering the start of a DMA sequence after a sequence from another DMA channel completes
- Software triggering
- Synchronization of concurrent activities

TRU Features

The TRU supports the following features:

- Automatically triggering the start of a DMA sequence after a sequence from another DMA channel completes. Once a DMA channel completes data transfer, it can act as a trigger generator and signal an internal trigger pulse to the programmed trigger receiver which can also be another DMA channel. The receiver trigger connected to the DMA channel kicks off the DMA transfer automatically. None of this requires core intervention once the initialization is done.
- Software triggers. The best use of triggers is to minimize core intervention. It is also possible to initiate a trigger pulse to a trigger receiver, in the software.
- Synchronization of concurrent activities. A single trigger generator can initiate a trigger pulse to multiple trigger receiver so that several system level activities can be synchronized on an internally or externally generated event.
- Configuration protection through register-level lock bits and global lock indication

TRU Functional Description

The following sections provide a description of the TRU.

ADSP-21560_21561_21564_21568 TRU Register List

The Trigger Routing Unit (TRU) provides simple sequence control of distributed modules without the penalties associated with core intervention (for example, interrupt overhead). The TRU receives trigger inputs from all trigger generator inputs (MTI) and the TRU trigger receiver register ([TRU_MTR](#)). Based on these inputs, the TRU logic generates trigger outputs that initiate receiver operations in the processor core and peripherals. A set of registers governs TRU operations. For more information on TRU functionality, see the TRU register descriptions.

Table 7-1: ADSP-21560_21561_21564_21568 TRU Register List

Name	Description
TRU_ERRADDR	Error Address Register
TRU_GCTL	Global Control Register
TRU_MTR	Generator Trigger Register
TRU_SSR[n]	Receiver Select Register
TRU_STAT	Status Information Register

ADSP-21560_21561_21564_21568 TRU Interrupt List

Table 7-2: ADSP-21560_21561_21564_21568 TRU Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
112	TRU0_SLV0	TRU0 Interrupt 4 -- Core 0	Edge	
113	TRU0_SLV1	TRU0 Interrupt 5 -- Core 0	Edge	
114	TRU0_SLV2	TRU0 Interrupt 6 -- Core 0	Edge	
115	TRU0_SLV3	TRU0 Interrupt 7 -- Core 0	Edge	

ADSP-21560_21561_21564_21568 Trigger List

Table 7-3: ADSP-21560_21561_21564_21568 Trigger List Generators

Trigger ID	Name	Description	Sensitivity
0		Reserved	
1	CGU0_EVT	CGU0 Event	Edge
3	TIMER0_TMR00_MST	TIMER0 Timer 0	Edge
4	TIMER0_TMR01_MST	TIMER0 Timer 1	Edge
5	TIMER0_TMR02_MST	TIMER0 Timer 2	Edge
6	TIMER0_TMR03_MST	TIMER0 Timer 3	Edge
7	TIMER0_TMR04_MST	TIMER0 Timer 4	Edge

Table 7-3: ADSP-21560_21561_21564_21568 Trigger List Generators (Continued)

Trigger ID	Name	Description	Sensitivity
8	TIMER0_TMR05_MST	TIMER0 Timer 5	Edge
9	TIMER0_TMR06_MST	TIMER0 Timer 6	Edge
10	TIMER0_TMR07_MST	TIMER0 Timer 7	Edge
11	TIMER0_TMR08_MST	TIMER0 Timer 8	Edge
12	TIMER0_TMR09_MST	TIMER0 Timer 9	Edge
13	PINT0_BLOCK	PINT0 Pin Interrupt Block	Level
14	PINT1_BLOCK	PINT1 Pin Interrupt Block	Level
16	CNT0_STAT	CNT0 Status	Level
17	SPORT0_A_DMA	SPORT0 Channel A DMA	Edge
18	SPORT0_B_DMA	SPORT0 Channel B DMA	Edge
19	SPORT1_A_DMA	SPORT1 Channel A DMA	Edge
20	SPORT1_B_DMA	SPORT1 Channel B DMA	Edge
21	SPORT2_A_DMA	SPORT2 Channel A DMA	Edge
22	SPORT2_B_DMA	SPORT2 Channel B DMA	Edge
23	SPORT3_A_DMA	SPORT3 Channel A DMA	Edge
24	SPORT3_B_DMA	SPORT3 Channel B DMA	Edge
25	SPORT4_A_DMA	SPORT4 Channel A DMA	Edge
26	SPORT4_B_DMA	SPORT4 Channel B DMA	Edge
27	SPORT5_A_DMA	SPORT5 Channel A DMA	Edge
28	SPORT5_B_DMA	SPORT5 Channel B DMA	Edge
29	SPORT6_A_DMA	SPORT6 Channel A DMA	Edge
30	SPORT6_B_DMA	SPORT6 Channel B DMA	Edge
31	SPORT7_A_DMA	SPORT7 Channel A DMA	Edge
32	SPORT7_B_DMA	SPORT7 Channel B DMA	Edge
33	SPI0_TXDMA	SPI0 TX DMA Channel	Edge
34	SPI0_RXDMA	SPI0 RX DMA Channel	Edge
35	SPI1_TXDMA	SPI1 TX DMA Channel	Edge
36	SPI1_RXDMA	SPI1 RX DMA Channel	Edge
37	SPI2_TXDMA	SPI2 TX DMA Channel	Edge
38	SPI2_RXDMA	SPI2 RX DMA Channel	Edge
39	FIR0_DMA	FIR0 DMA	Edge

Table 7-3: ADSP-21560_21561_21564_21568 Trigger List Generators (Continued)

Trigger ID	Name	Description	Sensitivity
40	IIR0_DMA	IIR0 DMA	Edge
43	UART0_TXDMA	UART0 Transmit DMA	Edge
44	UART0_RXDMA	UART0 Receive DMA	Edge
45	UART1_TXDMA	UART1 Transmit DMA	Edge
46	UART1_RXDMA	UART1 Receive DMA	Edge
49	MDMA0_SRC	Standard BW DMA Channel 0 Source (CRC IN)	
50	MDMA0_DST	Standard BW DMA Channel 0 Source (CRC OUT)	
51	MDMA1_SRC	Standard BW DMA Channel 1 Source (CRC IN)	
52	MDMA1_DST	Standard BW DMA Channel 1 Source (CRC OUT)	
53	MDMA2_SRC	Enh BW DMA Channel 0	
54	MDMA2_DST	Enh BW DMA Channel 1	
55	MDMA3_SRC	Max BW DMA Channel 0	
56	MDMA3_DST	Max BW DMA Channel 1	
57	EMDMA0_DONE	EMDMA0 EMDMA0 DMA Done	Edge
58	EMDMA1_DONE	EMDMA1 EMDMA1 DMA Done	Edge
67	SEC0_FAULT	SEC0 Fault	Edge
68	SOFT0_MST	Software-driven Trigger 0	
69	SOFT1_MST	Software-driven Trigger 1	
70	SOFT2_MST	Software-driven Trigger 2	
71	SOFT3_MST	Software-driven Trigger 3	
72	SOFT4_MST	Software-driven Trigger 4	
73	SOFT5_MST	Software-driven Trigger 5	
75	L2CTL1_EVT	L2CTL1 Scrub/Initialization Done	Level
93	C0_SID_ACK	Core 0 System Interface Disable Acknowledge	
94	L2CTL0_EVT	L2CTL0 L2CTL0 Initialization Done	Level
95	MEC0_L2PIRQ0	MEC0 L2 Parity Interrupt Request	Level
96	MEC0_PEIRQ0	MEC0 Parity Error Interrupt Request	Level
97	MEC0_PEIRQ1	MEC0 Parity Error Interrupt Request	Level

Table 7-3: ADSP-21560_21561_21564_21568 Trigger List Generators (Continued)

Trigger ID	Name	Description	Sensitivity
98	CNT0_UD	CNT0 CNT0 Count Up and Direction	Level
99	CNT0_DG	CNT0 CNT0 Count Down and Gate	Level
100	CNT0_TO	CNT0 CNT0 Output to Timer Block	Level
101	WDOG0_EXP	WDOG0 Expiration	Level
102	WDOG1_EXP	WDOG1 Expiration	Level
103	FIR0_TRGO	FIR0 FIR Trigger Output	Edge
104	IIR0_TRGO	IIR0 IIR Trigger output	Edge
105	DAI0_GBL_SPORT_TRG_O0	DAI0 SPORT GROUP0 Trigger Output	None
106	DAI0_GBL_SPORT_TRG_O1	DAI0 SPORT GROUP1 Trigger Output	None
107	DAI1_GBL_SPORT_TRG_O0	DAI1 SPORT GROUP2 Trigger Output	None
108	DAI1_GBL_SPORT_TRG_O1	DAI1 SPORT Group3 Trigger Output	None

Table 7-4: ADSP-21560_21561_21564_21568 Trigger List Receivers

Trigger ID	Name	Description	Sensitivity
0	TIMER0_TMR00_SLV0	TIMER0 Timer 0	Pulse
1	TIMER0_TMR00_SLV1	TIMER0 Timer 0	Pulse
2	TIMER0_TMR01_SLV0	TIMER0 Timer 1	Pulse
3	TIMER0_TMR01_SLV1	TIMER0 Timer 1	Pulse
4	TIMER0_TMR02_SLV0	TIMER0 Timer 2	Pulse
5	TIMER0_TMR02_SLV1	TIMER0 Timer 3	Pulse
6	TIMER0_TMR03_SLV0	TIMER0 Timer 3	Pulse
7	TIMER0_TMR03_SLV1	TIMER0 Timer 3	Pulse
8	TIMER0_TMR04_SLV0	TIMER0 Timer 4	Pulse
9	TIMER0_TMR04_SLV1	TIMER0 Timer 4	Pulse
10	TIMER0_TMR05_SLV0	TIMER0 Timer 5	Pulse
11	TIMER0_TMR05_SLV1	TIMER0 Timer 5	Pulse
12	TIMER0_TMR06_SLV0	TIMER0 Timer 6	Pulse
13	TIMER0_TMR06_SLV1	TIMER0 Timer 6	Pulse

Table 7-4: ADSP-21560_21561_21564_21568 Trigger List Receivers (Continued)

Trigger ID	Name	Description	Sensitivity
14	TIMER0_TMR07_SLV0	TIMER0 Timer 7	Pulse
15	TIMER0_TMR07_SLV1	TIMER0 Timer 7	Pulse
16	TIMER0_TMR08_SLV0	TIMER0 Timer 8	Pulse
17	TIMER0_TMR08_SLV1	TIMER0 Timer 8	Pulse
18	TIMER0_TMR09_SLV0	TIMER0 Timer 9	Pulse
19	TIMER0_TMR09_SLV1	TIMER0 Timer 9	Pulse
20	SPORT0_A_DMA	SPORT0 Channel A DMA	Pulse
21	SPORT0_B_DMA	SPORT0 Channel B DMA	Pulse
22	SPORT1_A_DMA	SPORT1 Channel A DMA	Pulse
23	SPORT1_B_DMA	SPORT1 Channel B DMA	Pulse
24	SPORT2_A_DMA	SPORT2 Channel A DMA	Pulse
25	SPORT2_B_DMA	SPORT2 Channel B DMA	Pulse
26	SPORT3_A_DMA	SPORT3 Channel A DMA	Pulse
27	SPORT3_B_DMA	SPORT3 Channel B DMA	Pulse
28	SPORT4_A_DMA	SPORT4 Channel A DMA	Pulse
29	SPORT4_B_DMA	SPORT4 Channel B DMA	Pulse
30	SPORT5_A_DMA	SPORT5 Channel A DMA	Pulse
31	SPORT5_B_DMA	SPORT5 Channel B DMA	Pulse
32	SPORT6_A_DMA	SPORT6 Channel A DMA	Pulse
33	SPORT6_B_DMA	SPORT6 Channel B DMA	Pulse
34	SPORT7_A_DMA	SPORT7 Channel A DMA	Pulse
35	SPORT7_B_DMA	SPORT7 Channel B DMA	Pulse
36	SPI0_TXDMA	SPI0 TX DMA Channel	Pulse
37	SPI0_RXDMA	SPI0 RX DMA Channel	Pulse
38	SPI1_TXDMA	SPI1 TX DMA Channel	Pulse
39	SPI1_RXDMA	SPI1 RX DMA Channel	Pulse
40	SPI2_TXDMA	SPI2 TX DMA Channel	Pulse
41	SPI2_RXDMA	SPI2 RX DMA Channel	Pulse
44	RCU0_SYSRST0	RCU0 System Reset 0	Pulse
45	RCU0_SYSRST1	RCU0 System Reset 1	Pulse
78	TRU0_SLV0	TRU0 Interrupt Request 0	Pulse

Table 7-4: ADSP-21560_21561_21564_21568 Trigger List Receivers (Continued)

Trigger ID	Name	Description	Sensitivity
79	TRU0_SLV1	TRU0 Interrupt Request 1	Pulse
80	TRU0_SLV2	TRU0 Interrupt Request 2	Pulse
81	TRU0_SLV3	TRU0 Interrupt Request 3	Pulse
82	UART0_TXDMA	UART0 Transmit DMA	Pulse
83	UART0_RXDMA	UART0 Receive DMA	Pulse
84	UART1_TXDMA	UART1 Transmit DMA	Pulse
85	UART1_RXDMA	UART1 Receive DMA	Pulse
88	MDMA0_SRC	Standard BW DMA Channel 0 Source (CRC IN)	Pulse
89	MDMA0_DST	Standard BW DMA Channel 0 Source (CRC OUT)	Pulse
90	MDMA1_SRC	Standard BW DMA Channel 1 Source (CRC IN)	Pulse
91	MDMA1_DST	Standard BW DMA Channel 1 Source (CRC OUT)	Pulse
92	MDMA2_SRC	Enh BW DMA Channel 0	Pulse
93	MDMA2_DST	Enh BW DMA Channel 1	Pulse
94	MDMA3_SRC	Max BW DMA Channel 0	Pulse
95	MDMA3_DST	Max BW DMA Channel 1	Pulse
96	CTI1_SLV0	CTI1 CTI	Pulse
97	CTI1_SLV1	CTI1 CTI	Pulse
98	CTI1_SLV2	CTI1 CTI	Pulse
99	CTI1_SLV3	CTI1 CTI	Pulse
100	CTI1_SLV4	CTI1 CTI	Pulse
101	CTI1_SLV5	CTI1 CTI	Pulse
102	CTI1_SLV6	CTI1 CTI	Pulse
111	PORTA_TOGGLE	Port Toggle Trigger	Pulse
112	PORTB_TOGGLE	Port Toggle Trigger	Pulse
114	CNT0_UD	CNT0 CNT0 Count Up and Direction Input	Pulse
115	CNT0_DG	CNT0 CNT0 Count Down and Gate Input	Pulse
116	FIR0_TRGI	FIR0 FIR Trigger input	Pulse
117	IIR0_TRGI	IIR0 IIR Trigger input	Pulse

TRU Definitions

The following definitions are helpful when using the TRU module.

Trigger Generator

A trigger generator is any system module that provides trigger event indication to the TRU. Trigger generator modules define trigger events and conditions for assertion.

Trigger Generator ID

Trigger generators are assigned a unique numeric ID according to their physical connection to the TRU. Trigger receiver ID 0 is reserved and defined as null.

Trigger Receiver

A trigger receiver is any system module that receives a trigger event indication from the TRU. Trigger receiver modules define a trigger event response.

NOTE: For peripherals configured as a receiver: when the corresponding peripheral DMA is kept waiting for trigger from a generator, it can receive stale or garbage data. The peripheral receiver begins shifting in data as soon as it is enabled, and it does not wait for the trigger generator. Upon receiving the trigger, this data is moved into the DMA FIFO. Therefore, this sequence can result in the receiver getting unusable data.

TRU Block Diagram

The trigger generator and the Generator Trigger register (MTR) generate trigger assertions. Each trigger receiver has a dedicated Receiver Select register (SSR) that specifies the unique trigger generator from which it receives the trigger indication.

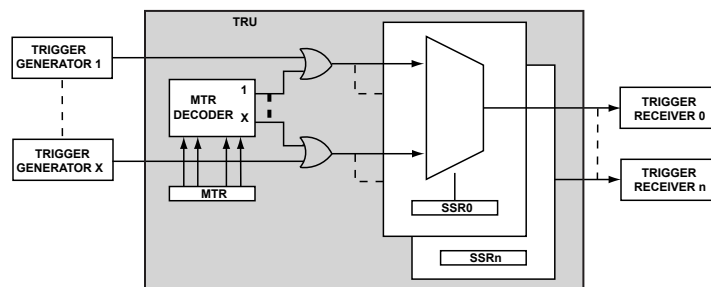


Figure 7-1: TRU Block Diagram

TRU Architectural Concepts

The TRU supports a simple trigger-in/trigger-out model for modules that comply with the triggering functional model. The TRU is the controller of the trigger system. Trigger outputs from trigger generators are mapped to trigger inputs of trigger receivers through a set of programmable registers ([TRU_SSR\[n\]](#)).

System modules are trigger generator only, trigger receiver only, or trigger generator and trigger receiver.

All of the trigger input and output signals are connected to a trigger routing unit (TRU) which manages the connections of triggers between modules.

In multi-processor systems, multiple TRU units are provided. These TRUs are networked together. Generic Trigger Ports (GTPs) are provided to forward trigger events from one TRU unit to another, forming a pathway from trigger generator to trigger receivers wherever they might lie in the system.

TRU Programming Model

Implementing sequence control using the TRU requires, at a minimum, proper configuration of a trigger receiver, a trigger generator, and the TRU module itself. The only requirement for the configuration procedure is that the trigger generator is configured and enabled as the last step.

Complete the following other steps:

- Configure the trigger receiver for response to triggers
- Configure the TRU to map the trigger generator to the trigger receiver through the `TRU_SSR[n]` registers
- Configure the trigger generator to generate trigger assertions
- Alternatively, use software triggering for trigger assertion. Writing the trigger generator ID to the MTR register generates software triggers.

Programming Concepts

The following concepts aid in programming the TRU.

- **Trigger Sequence Configuration.** A simple sequence consists of one trigger generator and one trigger receiver. More complex trigger sequences consist of several trigger receivers functioning as trigger receiver and trigger generator. Additionally, trigger sequences can loopback to the original generator forming a perpetual sequence.
- **Software Triggering.** Writing a trigger generator ID to the MTR generates a trigger within the TRU from the trigger generator ID specified.
- **Synchronization.** The TRU can be used to coarsely synchronize events by mapping multiple trigger receivers to the same trigger generator or by generating multiple trigger generator assertions simultaneously through the MTR.
- **Configuration Protection.** The `TRU_SSR[n].LOCK` bit and the `TRU_GCTL.LOCK` bit enable register level write-protection when the global lock is asserted in the SPU.

Programming Examples

The following examples shows the steps to create a single trigger.

Configuring a Simple Trigger Sequence

The following example shows the steps to create a simple trigger.

- 1. Enable the Global Lock feature bit on the SPU (set the `SPU_CTL.GLCK` bit).
- 2. Set the `TRU_GCTL.LOCK` bit to 1 so that the `TRU_GCTL` register becomes read-only.
- 3. Set the `TRU_GCTL.MTRL` bit to 1 so that the `TRU_MTR` register becomes read-only.

TRU Event Control

The TRU is a major part of event control solutions. It is the center of the trigger functional model and can extend to support the interrupt and fault management models as well.

TRU Status and Error Signals

The TRU does not have dedicated status and error output signals other than the MMR interface. Receiver errors are reported to the generator over the standard peripheral bus protocol.

ADSP-21560_21561_21564_21568 TRU Register Descriptions

Trigger Routing Unit (TRU) contains the following registers.

Table 7-5: ADSP-21560_21561_21564_21568 TRU Register List

Name	Description
<code>TRU_ERRADDR</code>	Error Address Register
<code>TRU_GCTL</code>	Global Control Register
<code>TRU_MTR</code>	Generator Trigger Register
<code>TRU_SSR[n]</code>	Receiver Select Register
<code>TRU_STAT</code>	Status Information Register

Error Address Register

The TRU error address register ([TRU_ERRADDR](#)) holds the address from the memory-mapped register access generating an access error of TRU registers.

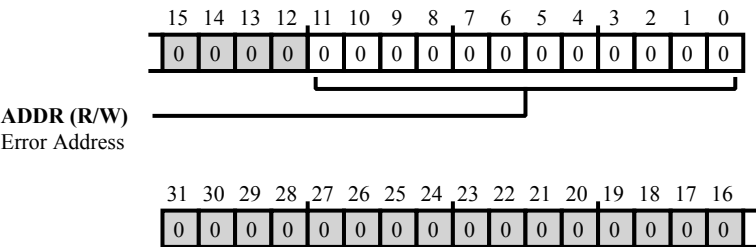


Figure 7-2: TRU_ERRADDR Register Diagram

Table 7-6: TRU_ERRADDR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
11:0 (R/W)	ADDR	Error Address. The <code>TRU_ERRADDR.ADDR</code> holds the address from the memory-mapped register access generating an access error of TRU registers. These errors occur on access to the TRU_SSR[n] or TRU_MTR registers when these registers are locked or on access to an invalid address. See the TRU_SSR[n] and TRU_MTR register descriptions for more information about locking. The TRU_ERRADDR register holds the address of the first error to occur. In the event of multiple errors occurring, the TRU_ERRADDR register contains the address of the first error. To re-enable the TRU_ERRADDR register for update, both status bits (<code>TRU_STAT.LWERR</code> and <code>TRU_STAT.ADDRERR</code>) in the TRU_STAT register must be cleared.

Global Control Register

The TRU global control register ([TRU_GCTL](#)) provides register locking, TRU reset, and TRU enable.

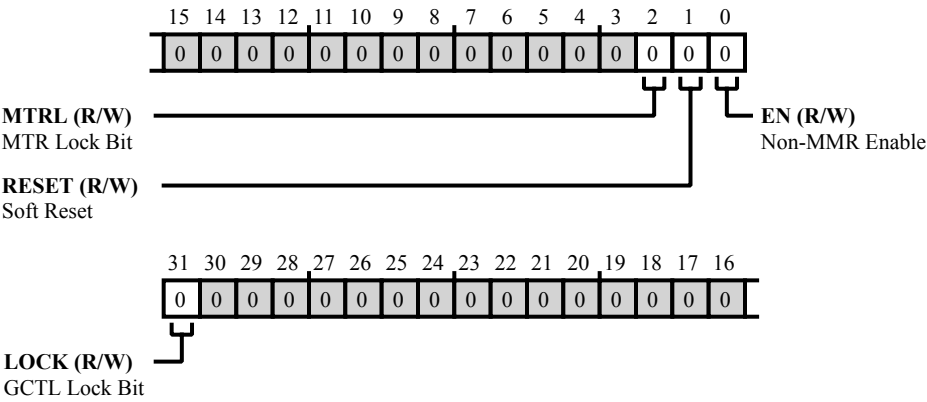


Figure 7-3: TRU_GCTL Register Diagram

Table 7-7: TRU_GCTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	GCTL Lock Bit. If the global lock is enabled (SPU_CTL . GLCK bit =1) and the TRU_GCTL . LOCK bit is enabled, the TRU_GCTL register is read only.
		0 Read write
		1 Read only
2 (R/W)	MTRL	MTR Lock Bit. If the global lock is enabled (SPU_CTL . GLCK bit =1) and the TRU_GCTL . MTRL bit is enabled, the TRU_MTR register is read only.
		0 Read write
		1 Read only
1 (R/W)	RESET	Soft Reset. The TRU_GCTL . RESET bit is write-1-action and triggers a soft reset to all TRU registers.
		0 No action
		1 Soft reset

Table 7-7: TRU_GCTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	EN	Non-MMR Enable. The TRU_GCTL.EN bit is read/write and must be set for the TRU to propagate trigger events. All TRU register read/write operations continue to operate independent of the TRU_GCTL.EN bit.
		0 No trigger events
		1 Propagate trigger events

Generator Trigger Register

The TRU generator trigger register ([TRU_MTR](#)) permits trigger generation through software by writing a trigger requester ID value to one of the four fields in the [TRU_MTR](#) register. If the global lock is enabled ([SPU_CTL](#).GLCK bit =1) and the [TRU_GCTL](#).LOCK bit is set, the [TRU_MTR](#) register is read only. Note this register is primarily used for debug to trigger a TRU output

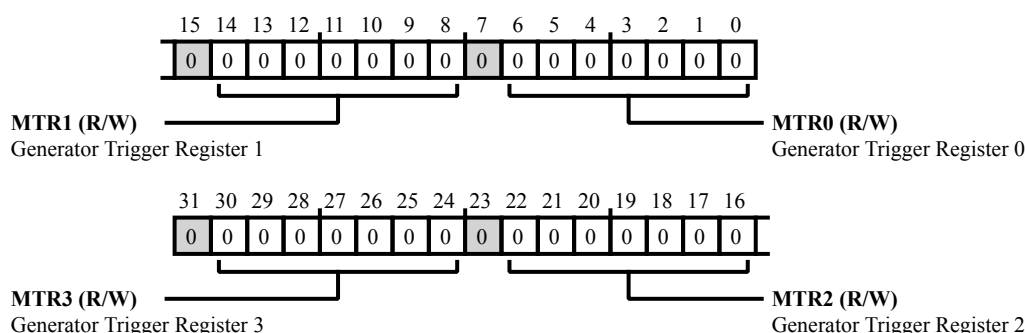


Figure 7-4: TRU_MTR Register Diagram

Table 7-8: TRU_MTR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
30:24 (R/W)	MTR3	Generator Trigger Register 3. The <code>TRU_MTR.MTR3</code> bit field is the trigger generator ID value for generator 3.
		0 No generator specified
		1-105 Range of valid generators
22:16 (R/W)	MTR2	Generator Trigger Register 2. The <code>TRU_MTR.MTR2</code> bit field is the trigger requester ID value for requester 2.
		0 No generator specified
		1-105 Range of valid generators
14:8 (R/W)	MTR1	Generator Trigger Register 1. The <code>TRU_MTR.MTR1</code> bit field is the trigger generator ID value for generator 1.
		0 No generator specified
		1-105 Range of valid generators
6:0 (R/W)	MTR0	Generator Trigger Register 0. The <code>TRU_MTR.MTR0</code> bit field is the trigger generator ID value for generator 0.
		0 No generator specified
		1-105 Range of valid generators

Receiver Select Register

The TRU receiver select registers (`TRU_SSR[n]`) each provide receiver selection and register locking.

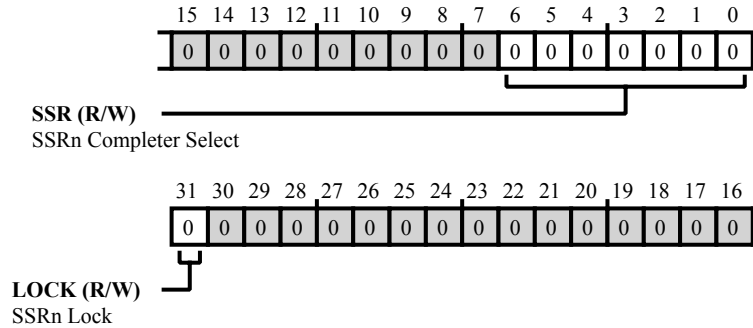


Figure 7-5: `TRU_SSR[n]` Register Diagram

Table 7-9: `TRU_SSR[n]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	SSRn Lock. If the global lock is enabled (<code>SPU_CTL.GLCK</code> bit =1) and the <code>TRU_SSR[n].LOCK</code> bit is enabled, the <code>TRU_SSR[n]</code> register is read only.
		0 Unlock register
		1 Lock register
6:0 (R/W)	SSR	SSRn Completer Select. The <code>TRU_SSR[n]</code> register selects the trigger generator ID to which the trigger receiver responds. For example, when a <code>TRU_SSR[n]</code> register is set to respond to trigger generator ID n, a trigger that is generated by trigger generator ID n results in a trigger out to the receiver.
		0 No generator specified
		1-105 Range of valid generators

Status Information Register

The TRU status register (`TRU_STAT`) contains the status of `TRU_MTR` and `TRU_SSR[n]` register writes and status of bus read/write errors.

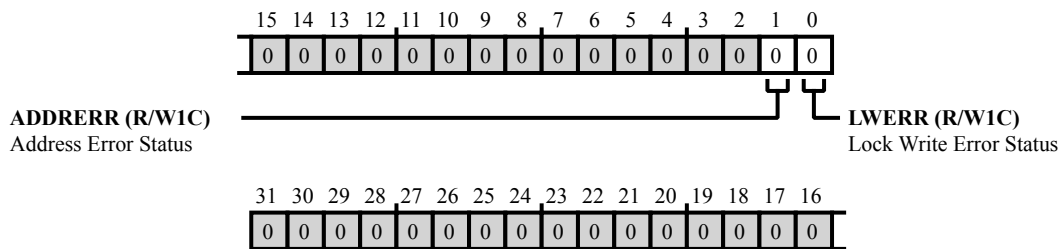


Figure 7-6: TRU_STAT Register Diagram

Table 7-10: TRU_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/W1C)	ADDRERR	Address Error Status. The <code>TRU_STAT.ADDRERR</code> bit is set when an invalid address is provided for an MMR access while the TRU is selected. Writing a one to this bit clears the error indication. The <code>TRU_ERRADDR</code> register also is updated when an address error occurs during an MMR access while the TRU is selected.
		0 No error
		1 Error occurred
0 (R/W1C)	LWERR	Lock Write Error Status. If <code>TRU_STAT.LWERR</code> is set, a lock write error has occurred. Writing a one to this bit clears the error indication.
		0 No error
		1 Error occurred

8 L2 System Memory

L2 system memories have significant bandwidth for core accesses, but it is important to note that L2 responds slower to core accesses than L1 memories. L2 SRAM is the ideal storage for multiple processor cores to share data and instruction resources, such as semaphores, shared buffers, and code libraries. Due to sophisticated data integrity protection and write protection, L2 SRAM is also ideal for data and instructions critical for safe operation of the application.

L2 System Memory Features

The L2 system memory features include:

- Operation at SYSCLK frequency
- Parity protection of SRAM area

There are two instances of L2 system memory: L2CTL0 and L2CTL1.

- L2CTL0 contains 1M byte of RAM grouped into eight banks, 128K bytes each and 96K bytes of boot ROM.
- L2CTL1 contains 1M byte of RAM grouped into eight banks, 128K bytes each.

L2 System Memory Functional Description

The L2 system memory manages all of the L2 SRAM and ROM memory banks. The system memory interface arbitrates competing accesses, write protection, and ensures SRAM data integrity. The L2 system memory domain is a unified instruction and data memory. It can hold any mixture of code and data required by the system design.

The following sections provide a functional description of the L2 system memory.

ADSP-21560_21561_21564_21568 L2CTL Register List

The L2 memory controller (L2CTL) includes the controls to manage each L2 memory bank independently. A set of registers governs L2CTL operations. For more information on L2CTL functionality, see the L2CTL register descriptions.

Table 8-1: ADSP-21560_21561_21564_21568 L2CTL Register List

Name	Description
L2CTL_CTL	Control Register
L2CTL_EADDR0	Error Type 0 Address Register
L2CTL_EADDR1	Error Type 1 Address Register
L2CTL_ET0	Error Type 0 Register
L2CTL_ET1	Error Type 1 Register
L2CTL_INIT	Initialization Register
L2CTL_ISTAT	Initialization Status Register
L2CTL_PCTL	Power Control Register
L2CTL_PERRADDR0	Parity Error Address 0 Register
L2CTL_PERRADDR1	Parity Error Address 1 Register
L2CTL_PERRADDR2	Parity Error Address 2 Register
L2CTL_PERRADDR3	Parity Error Address 3 Register
L2CTL_PERRADDR4	Parity Error Address 4 Register
L2CTL_PERRADDR5	Parity Error Address 5 Register
L2CTL_PERRADDR6	Parity Error Address 6 Register
L2CTL_PERRADDR7	Parity Error Address 7 Register
L2CTL_PERRADDR8	Parity Error Address 8 Register
L2CTL_REVID	Revision ID Register
L2CTL_RPCR	Read Priority Count Register
L2CTL_STAT	Status Register
L2CTL_WPCR	Write Priority Count Register

ADSP-21560_21561_21564_21568 L2CTL Interrupt List

Table 8-2: ADSP-21560_21561_21564_21568 L2CTL Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
9	L2CTL0_L2_PERR	L2CTL0 L2CTL Parity Error	Level	
11	L2CTL0_EVT	L2CTL0 Scrub/Initialization Done	Level	
179	L2CTL1_L2_PERR	L2CTL1 L2CTL Parity Error	Level	
180	L2CTL1_EVT	L2CTL1 Scrub/Initialization Done	Level	

ADSP-21560_21561_21564_21568 L2CTL Trigger List

Table 8-3: ADSP-21560_21561_21564_21568 L2CTL Trigger List Generators

Trigger ID	Name	Description	Sensitivity
75	L2CTL1_EVT	L2CTL1 Scrub/Initialization Done	Level
94	L2CTL0_EVT	L2CTL0 L2CTL0 Initialization Done	Level

Table 8-4: ADSP-21560_21561_21564_21568 L2CTL Trigger List Receivers

Trigger ID	Name	Description	Sensitivity
None			

L2 System Memory Block Diagram

The *L2 System Block Diagram* shows the complete L2 system memory, including the memory block instance L2CTL0. The L2CTL0 block contains boot ROM code and three banks of L2 ROM containing 96 Kbytes each.

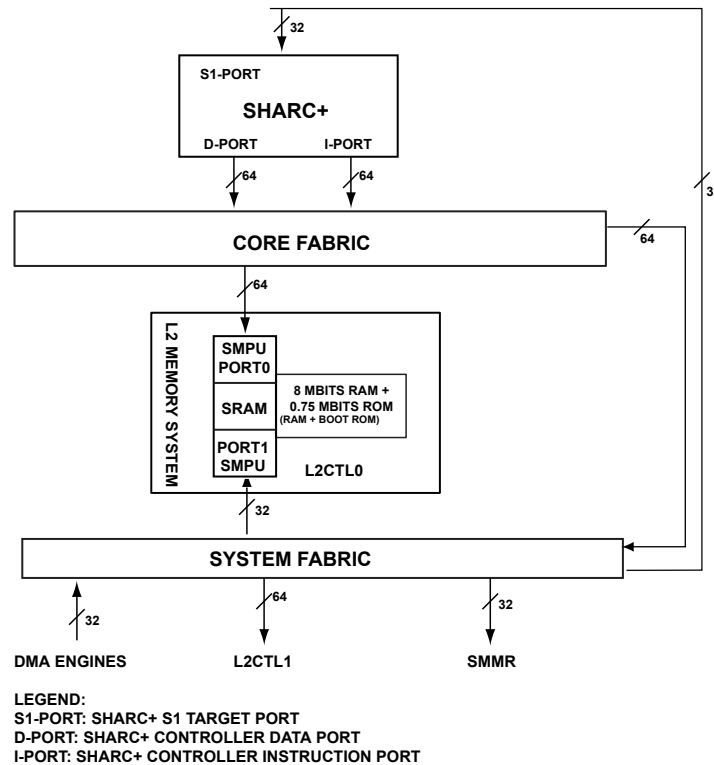


Figure 8-1: L2 System Block Diagram

L2 System Memory Architectural Concepts

The following sections describe the architecture features of the L2 system memory.

Access Characteristics

The L2 system memory interface converts all 8-bit, 16-bit, and 32-bit accesses to 64-bit accesses. Additionally, it converts 8-bit, 16-bit, and 32-bit bursts to an equivalent internal 64-bit access. For example, the L2 system memory interface converts a 64-bit address-aligned burst of 8-bit accesses of burst length 8 to a single 64-bit access.

Read/Write Latency and Throughput

The L2 memory design is optimized for burst accesses at the crossbar interface. The L2 system memory buffers and converts write data of 8/16/32-bit to an equivalent 64-bit access. This conversion creates modulo-32-bit writes if the starting addresses are 32-bit aligned. A single 8-bit or 16-bit access, or a non-32-bit address-aligned 8-bit or 16-bit burst access to a bank creates an extra latency of two SYSCLK cycles.

NOTE: Continuous 8/16-bit core access to a L2 bank is not recommended from a throughput perspective.

L2 Memory Controller Block Diagram (Instance)

As shown in the *L2 System Memory Block Diagram*, the L2 controller has two ports that interface to system cross-bars. Port 0 is a 64-bit interface that is dedicated to core and MDMA3 traffic, and port 1 is a 32-bit interface that connects through the rest of the DMA access. For L2 SRAM both ports (0/1) have a read channel and a write channel, for L2 ROM both ports (0/1) have read channels only. The SRAM is organized in multiple banks, and each bank has 128K Bytes. The 96KB ROM is divided into three banks of 32KB.

Within each bank, data is organized into 16384 words, with each word comprising 64 bits of data and two bits of parity. When the L2 controller accesses RAM and ROM cells, it always reads and writes whole 64-bit words. Despite this, the L2 controller supports 8-, 16-, and 32-bit reads and writes from cores and system by applying respective data masks.

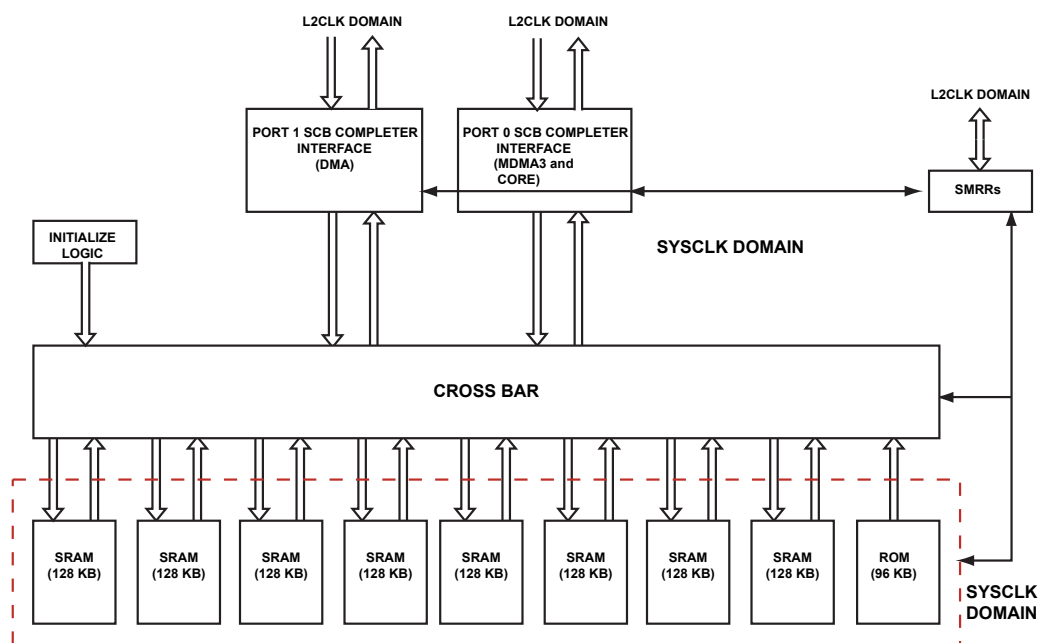


Figure 8-2: L2 System Memory Block Diagram

Arbitration and Priority

Each bank of L2 RAM or ROM has an arbiter which receives requests from the two crossbar ports.

Each arbiter follows a fixed priority scheme for giving grants when more than one channel requests the same bank. The arbiter also supports priority elevation through urgent priority requests.

NOTE: Attempting a write access to both L2 ROM spaces returns an error.

The *Fixed Priority* table shows the priority for fixed priority mode (with urgent priority disabled) for each SCB channel.

If two cores (or the 64-bit Max BW DMA) simultaneously try to access L2 for the same instance (both read or both write), even to different banks, software allows only one controller access at a time. One access port can support one read and write at the same time. However, if one core issues a write and the other issues a read, then access can proceed simultaneously. There is no extra latency inside L2, when the accesses are to different banks (assuming pending DMA traffic is also to a non-conflicting bank).

When a core and DMA both access the same bank via the same port (both read or both writes), the best access rate that DMA can achieve is one 64-bit access in every three SCLK cycles during the conflict period. This access rate is achieved by programming the read priority count register (`L2CTL_RPCR.RPC0`) bit and the write priority count register (`L2CTL_WPCR.WPC0`) bit to 0, while programming the `L2CTL_RPCR.RPC1` and the `L2CTL_WPCR.WPC1` bits to 1.

The arbiters also support priority elevation for a particular channel that has been starved of grants for many SCLK cycles. If a channel does not get a grant for N cycles after its request, then that channel can elevate the priority of its request by issuing an urgent priority request. This request causes that particular channel to become the highest priority controller for the next grant cycle (pipelined arbitration for urgent priority). The number of cycles N , after which the priority is elevated, can be programmed for each channel separately using the `L2CTL_RPCR` and `L2CTL_WPCR` registers.

Programming the bits in the `L2CTL_RPCR` and `L2CTL_WPCR` registers appropriately achieves the best grant rate for DMA. This grant rate of one in three SCLK cycles during the conflict period is achievable under the following conflict conditions:

- An access conflict between the core and DMA to the same memory bank in the fixed priority arbitration scheme with core activity always prioritized over DMA activity
- An access conflict within the pipelined implementation of urgent priority

To disable urgent priority requests, set the `L2CTL_CTL.DISURP` bit. This bit disables the urgent priority requests for all port channels. Each channel can also be prevented from raising the urgent priority request through the priority count register for the specific channel. However, there is no support for disabling urgent priority for a specific memory bank arbiter.

The *Fixed Priority With Priority Elevation* table provides the various priority levels for the L2 system memory.

Table 8-5: Fixed Priority With Priority Elevation

Channel	Priority Level
L2 Refresh Request	9 (highest)
Port 0 Read Channel Urgent Request	8
Port 0 Write Channel Urgent Request	7
Port 1 Read Channel Urgent Request	6
Port 1 Write Channel Urgent Request	5
Port 0 Read Channel Normal Request	4
Port 0 Write Channel Normal Request	3
Port 1 Read Channel Normal Request	2
Port 1 Write Channel Normal Request	1 (lowest)

Data Integrity

The following sections provide information on how the L2 system memory ensures data integrity.

Parity

Even parity is used to detect the odd number of erroneous bits in the data retrieved from memory. In this L2CTL architecture, there is one parity bit for 32-bit data. (It has word parity). If the number of bits including the parity bit has an odd number of ones, a parity error is generated.

By default, parity is enabled. However, parity can be disabled on a per bank basis. The parity implementation is in terms of 32 bits. Therefore, for any writes less than 32-bit wide (byte and half word) to the parity enabled RAM bank, the operation is implemented as a read-followed by write. It requires two extra read cycles. However, all writes to parity-disabled banks and all writes that are 32/64-bit (with addresses aligned to 32-bit boundaries) do not have a read cycle in between.

Parity Hardware Control

After reset, parity protection is enabled. The boot code initializes all L2 SRAM data and checksum cells. Parity protection adds some cycle penalty when 8-bit and 16-bit values write L2 memory. Disable parity protection for individual SRAM banks by setting the `L2CTL_CTL.BK0PDIS-L2CTL_CTL.BK8PDIS` disable bits. Due to caching mechanisms of the processor cores and data bursting of the DMA channels, 8-bit and 16-bit write accesses are rather uncommon. Typically, only two-dimensional DMA operations or uncached 8-bit and 16-bit store instruction can trigger these writes.

Parity Error Management

The L2 system memory flags parity errors to the system by:

- Raising the parity error interrupt
- Reporting a read error to the system bus
- Setting the sticky `L2CTL_STAT.PERR0 - L2CTL_STAT.PERR8` status flag

- Latching the address of the failing operation into the respective `L2CTL_PERRADDR0-L2CTL_PERRADDR8` register.

There is one error status bit and one error address register per L2 SRAM bank.

Typically, parity error events are declared as system faults in the system event controller (SEC). Whether these faults are reported, the interrupt service routine can consult the `L2CTL_STAT` register and the `L2CTL_PERRADDR0-L2CTL_PERRADDR8` registers to determine whether:

- The data at the failing L2 address was critical enough to require an immediate reboot of the system
- The data at the failing L2 address was less critical or can be restored

The `L2CTL_STAT.PERR0 - L2CTL_STAT.PERR8` flags are cleared with a W1C operation.

Power Modes

Each L2 memory bank supports low-power modes.

If the bank is not in use, put each L2 system memory bank into the following low-power modes to save power.

Deep Sleep Mode

Set the `L2CTL_PCTL.BK7DS` through `L2CTL_PCTL.BK0DS` control bits to enter this mode. This mode preserves the memory contents.

Shut Down Mode

Set the `L2CTL_PCTL.BK7SD` through `L2CTL_PCTL.BK0SD` control bits to enter this mode. The memory banks do not retain the data (existing data is lost).

NOTE: It may take up to 30 SYSCLK cycles to deactivate the power mode after writing to the `L2CTL_PCTL` register. Therefore, L2 access must be issued after this delay.

Access to a memory bank in shut down or deep sleep mode may result in an unpredictable behavior. Ensure that initialization/refresh and memory access to such banks are not issued.

L2 System Memory Event Control

The following sections describe event control features of the L2 system memory, such as error response.

Parity Error Interrupt

A bus error is signaled under any of the following conditions.

- A write access to ROM address space
- A read/write access to reserved address space
- An even parity error in an parity-enabled bank. A non-modulo, 32-bit write to an parity-enabled bank can also potentially create a bus error response due to an even parity error. This response is because the L2 system

memory implements a 32-bit parity error, and therefore a non-modulo, 32-bit write results in a read. This read can create parity errors even if the memory was initialized.

Bus error notifications are stored in the `L2CTL_STAT` register, and the addresses that generated the error on a given port are stored in the `L2CTL_EADDR0/L2CTL_EADDR1` register of that port. The details of the error are stored in the `L2CTL_ET0/L2CTL_ET1` register of the port.

ADSP-21560_21561_21564_21568 L2CTL Register Descriptions

L2 Memory Controller (L2CTL) contains the following registers.

Table 8-6: ADSP-21560_21561_21564_21568 L2CTL Register List

Name	Description
<code>L2CTL_CTL</code>	Control Register
<code>L2CTL_EADDR0</code>	Error Type 0 Address Register
<code>L2CTL_EADDR1</code>	Error Type 1 Address Register
<code>L2CTL_ET0</code>	Error Type 0 Register
<code>L2CTL_ET1</code>	Error Type 1 Register
<code>L2CTL_INIT</code>	Initialization Register
<code>L2CTL_ISTAT</code>	Initialization Status Register
<code>L2CTL_PCTL</code>	Power Control Register
<code>L2CTL_PERRADDR0</code>	Parity Error Address 0 Register
<code>L2CTL_PERRADDR1</code>	Parity Error Address 1 Register
<code>L2CTL_PERRADDR2</code>	Parity Error Address 2 Register
<code>L2CTL_PERRADDR3</code>	Parity Error Address 3 Register
<code>L2CTL_PERRADDR4</code>	Parity Error Address 4 Register
<code>L2CTL_PERRADDR5</code>	Parity Error Address 5 Register
<code>L2CTL_PERRADDR6</code>	Parity Error Address 6 Register
<code>L2CTL_PERRADDR7</code>	Parity Error Address 7 Register
<code>L2CTL_PERRADDR8</code>	Parity Error Address 8 Register
<code>L2CTL_REVID</code>	Revision ID Register
<code>L2CTL_RPCR</code>	Read Priority Count Register
<code>L2CTL_STAT</code>	Status Register
<code>L2CTL_WPCR</code>	Write Priority Count Register

Control Register

The `L2CTL_CTL` register includes a write protection bit, enables L2 banks, and selects mapping of banks (as ECC RAM or data RAM).

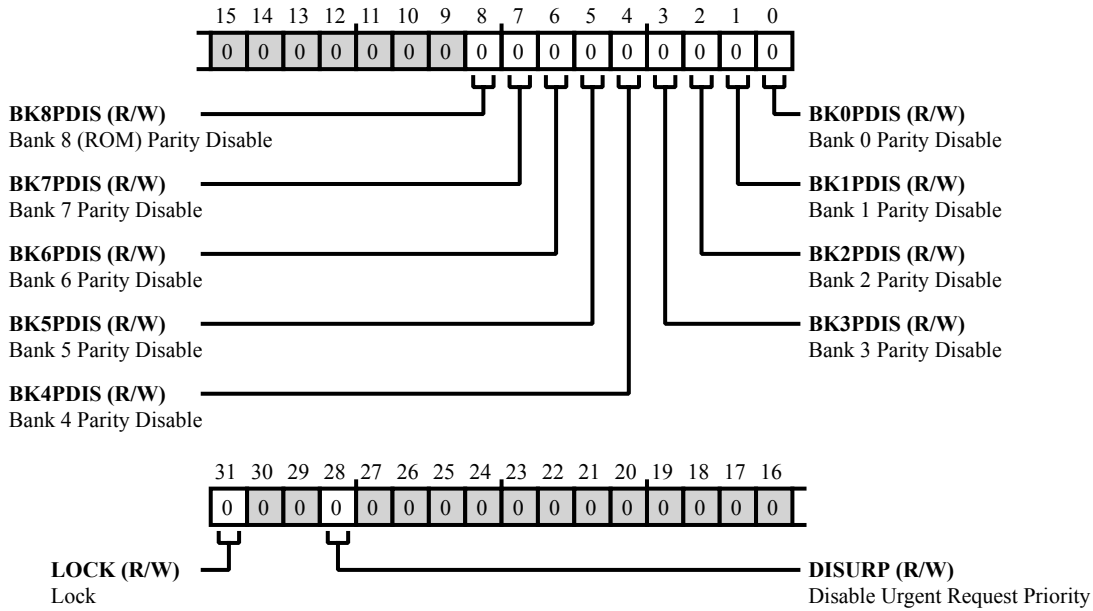


Figure 8-3: L2CTL_CTL Register Diagram

Table 8-7: L2CTL_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock. If the global lock bit is set (<code>SPU_CTL.GLCK</code> bit =1) and the <code>L2CTL_CTL.LOCK</code> bit is set, the <code>L2CTL_CTL</code> register is read only (locked).
		0 Unlock
		1 Lock
28 (R/W)	DISURP	Disable Urgent Request Priority. The <code>L2CTL_CTL.DISURP</code> disables urgent request priority mode for all L2 banks.
		0 Enable URP
		1 Disable URP
8 (R/W)	BK8PDIS	Bank 8 (ROM) Parity Disable. The <code>L2CTL_CTL.BK8PDIS</code> bit disables L2 bank 8 (ROM) Parity operation. Parity operation should be enabled or disabled before the start of L2 bank access.
		0 Enable ECC
		1 Disable ECC

Table 8-7: L2CTL_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
7 (R/W)	BK7PDIS	Bank 7 Parity Disable. The L2CTL_CTL.BK7PDIS bit disables L2 bank 7 Parity operation.
		0 Enable ECC
		1 Disable ECC
6 (R/W)	BK6PDIS	Bank 6 Parity Disable. The L2CTL_CTL.BK6PDIS bit disables L2 bank 6 Parity operation.
		0 Enable ECC
		1 Disable ECC
5 (R/W)	BK5PDIS	Bank 5 Parity Disable. The L2CTL_CTL.BK5PDIS bit disables L2 bank 5 Parity operation.
		0 Enable ECC
		1 Disable ECC
4 (R/W)	BK4PDIS	Bank 4 Parity Disable. The L2CTL_CTL.BK4PDIS bit disables L2 bank 4 Parity operation.
		0 Enable ECC
		1 Disable ECC
3 (R/W)	BK3PDIS	Bank 3 Parity Disable. The L2CTL_CTL.BK3PDIS bit disables L2 bank 3 Parity operation.
		0 Enable ECC
		1 Disable ECC
2 (R/W)	BK2PDIS	Bank 2 Parity Disable. The L2CTL_CTL.BK2PDIS bit disables L2 bank 2 Parity operation.
		0 Enable ECC
		1 Disable ECC
1 (R/W)	BK1PDIS	Bank 1 Parity Disable. The L2CTL_CTL.BK1PDIS bit disables L2 bank 1 Parity operation.
		0 Enable ECC
		1 Disable ECC

Table 8-7: L2CTL_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	BK0PDIS	Bank 0 Parity Disable. The L2CTL_CTL.BK0PDIS bit disables L2 bank 0 Parity operation.
		0 Enable Parity
		1 Disable Parity

Error Type 0 Address Register

The `L2CTL_EADDR0` register holds the address that created an access error on the L2 port 0 bus interface. This register is be updated only if the corresponding error status bit (`L2CTL_STAT.ERR0`) is cleared. After the status bit is set for an error, further errors do not update the `L2CTL_EADDR0` register until a W1C clears the corresponding status bit. If read and write access errors occur simultaneously, the register captures the write access error address.

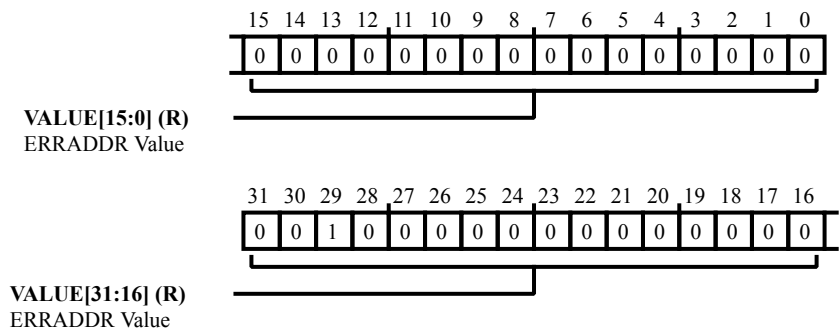


Figure 8-4: L2CTL_EADDR0 Register Diagram

Table 8-8: L2CTL_EADDR0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	ERRADDR Value. The <code>L2CTL_EADDR0.VALUE</code> bits hold the address causing the bus error.

Error Type 1 Address Register

The `L2CTL_EADDR1` register holds the address that created an access error on the L2 port 1 bus interface. This register is be updated only if the corresponding error status bit (`L2CTL_STAT.ERR1`) is cleared. After the status bit is set for an error, further errors do not update the `L2CTL_EADDR1` register until a W1C clears the corresponding status bit. If read and write access errors occur simultaneously, the register captures the write access error address.

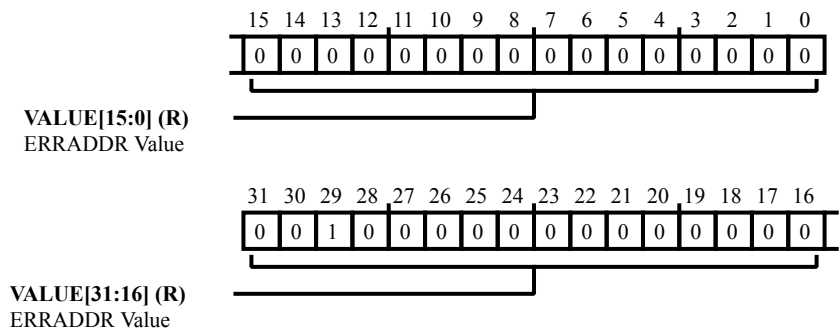


Figure 8-5: L2CTL_EADDR1 Register Diagram

Table 8-9: L2CTL_EADDR1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	ERRADDR Value. The <code>L2CTL_EADDR1.VALUE</code> bits hold the address causing the bus error.

Error Type 0 Register

The `L2CTL_ET0` register holds information about the error transaction that has occurred on the bus for the corresponding L2 bus port 0. This register is updated only if the corresponding error status bit `L2CTL_STAT.ERR0` is cleared. After the status bit is set for an error, further errors do not update the `L2CTL_ET0` register until a `W1C` clears the corresponding status bit. If read and write access errors occur simultaneously, the `L2CTL_ET0` captures the write access error, keeping in sync with the error address register (`L2CTL_EADDR0`).

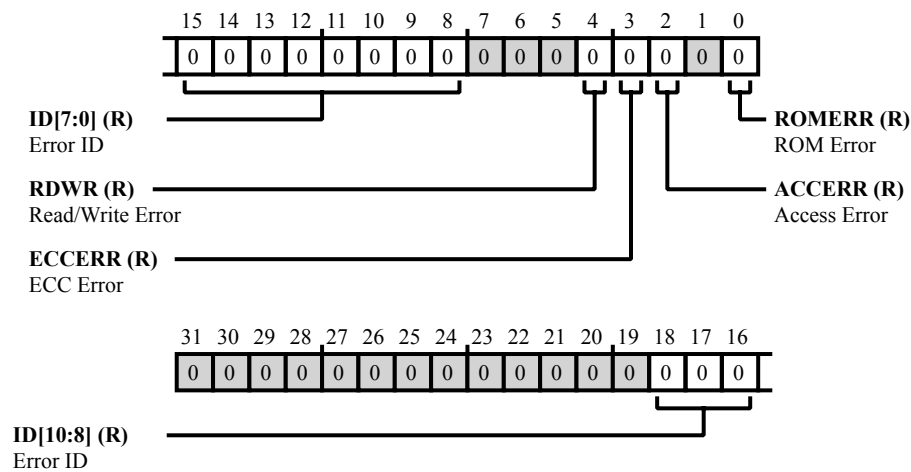


Figure 8-6: L2CTL_ET0 Register Diagram

Table 8-10: L2CTL_ET0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration	
18:8 (R/NW)	ID	Error ID. The L2CTL_ET0 . ID bits hold the bus requester ID of the access that caused an error.	
4 (R/NW)	RDWR	Read/Write Error. The L2CTL_ET0 . RDWR bit indicates whether a read or write access caused an error.	
		0	Read Access created Error
		1	Write Access created Error
3 (R/NW)	ECCERR	ECC Error. The L2CTL_ET0 . ECCERR bit indicates whether the access had an ECC double-bit error.	
2 (R/NW)	ACCERR	Access Error. The L2CTL_ET0 . ACCERR bit indicates whether the access went to a restricted bank.	

Table 8-10: L2CTL_ET0 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/NW)	ROMERR	ROM Error. The L2CTL_ET0.ROMERR bit indicates whether a write access went to a ROM area.

Error Type 1 Register

The `L2CTL_ET1` register holds information about the error transaction that has occurred on the bus for the corresponding L2 bus port 1. This register is updated only if the corresponding error status bit `L2CTL_STAT.ERR1` is cleared. After the status bit is set for an error, further errors do not update the `L2CTL_ET1` register until a W1C clears the corresponding status bit. If read and write access errors occur simultaneously, the `L2CTL_ET1` captures the write access error, keeping in sync with the error address register (`L2CTL_EADDR1`).

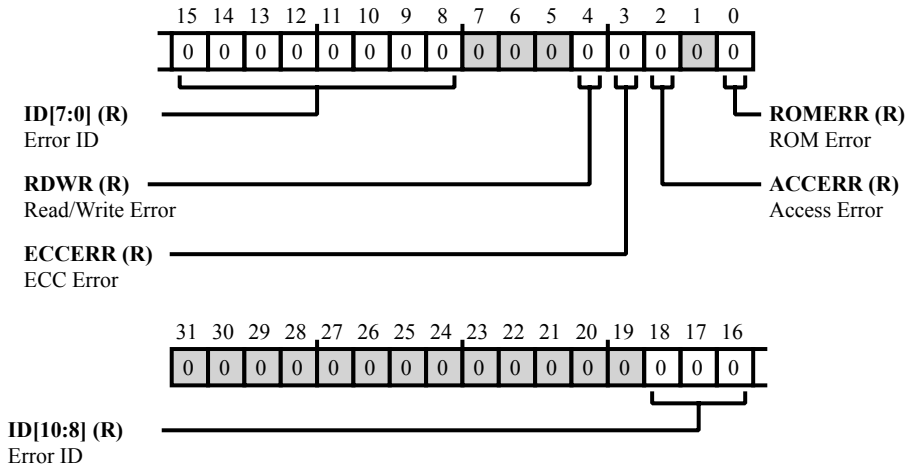


Figure 8-7: L2CTL_ET1 Register Diagram

Table 8-11: L2CTL_ET1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
18:8 (R/NW)	ID	Error ID. The <code>L2CTL_ET1.ID</code> bits hold the bus requester ID of the access that caused an error.
4 (R/NW)	RDWR	Read/Write Error. The <code>L2CTL_ET1.RDWR</code> bit indicates whether a read or write access caused an error.
		0 Read access created error
		1 Write access created error
3 (R/NW)	ECCERR	ECC Error. If the <code>L2CTL_ET1.ECCERR</code> bit =1, the access had an ECC double-bit error.
2 (R/NW)	ACCERR	Access Error. If the <code>L2CTL_ET1.ACCERR</code> bit =1, the access went to a restricted bank.
0 (R/NW)	ROMERR	ROM Error. If the <code>L2CTL_ET1.ROMERR</code> bit =1, a write access went to a ROM area.

Initialization Register

The `L2CTL_INIT` register initializes memory banks with 64'b0 and ECC bits corresponding to 64'b0. Any writes to the bits in this register while initialization is occurring to any of the banks is ignored. All bits are W1A (Write 1 for Action).

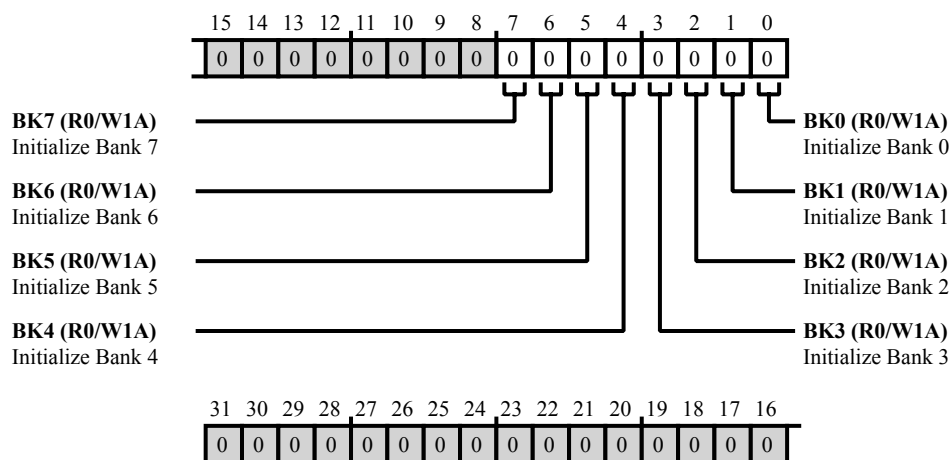


Figure 8-8: L2CTL_INIT Register Diagram

Table 8-12: L2CTL_INIT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7 (R0/W1A)	BK7	Initialize Bank 7. The write-1 to the <code>L2CTL_INIT.BK7</code> bit initializes bank 7 with 64b0 and ECC bits corresponding to 64b0. Any write to this bit during the initialization of the banks is ignored.
6 (R0/W1A)	BK6	Initialize Bank 6. The write-1 to the <code>L2CTL_INIT.BK6</code> bit initializes bank 6 with 64b0 and ECC bits corresponding to 64b0. Any write to this bit during the initialization of the banks is ignored.
5 (R0/W1A)	BK5	Initialize Bank 5. The write -1 to the <code>L2CTL_INIT.BK5</code> bit initializes bank 5 with 64b0 and ECC bits corresponding to 64b0. Any write to this bit during the initialization of the banks is ignored.
4 (R0/W1A)	BK4	Initialize Bank 4. The write-1 to the <code>L2CTL_INIT.BK4</code> bit initializes bank 4 with 64b0 and ECC bits corresponding to 64b0. Any write to this bit during the initialization of the banks is ignored.

Table 8-12: L2CTL_INIT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
3 (R0/W1A)	BK3	Initialize Bank 3. The write-1 to the L2CTL_INIT.BK3 bit initializes bank 3 with 64b0 and ECC bits corresponding to 64b0. Any write to this bit during the initialization of the banks is ignored.
2 (R0/W1A)	BK2	Initialize Bank 2. The write-1 to the L2CTL_INIT.BK2 bit initializes bank 2 with 64b0 and ECC bits corresponding to 64b0. Any write to this bit during the initialization of the banks is ignored.
1 (R0/W1A)	BK1	Initialize Bank 1. The write-1 to the L2CTL_INIT.BK1 bit initializes bank 1 with 64b0 and ECC bits corresponding to 64b0. Any write to this bit during the initialization of the banks is ignored.
0 (R0/W1A)	BK0	Initialize Bank 0. The write-1 to the L2CTL_INIT.BK0 bit initializes bank 0 with 64b0 and ECC bits corresponding to 64b0. Any write to this bit during the initialization of the banks is ignored.

Initialization Status Register

The `L2CTL_ISTAT` register holds the status of the RAM bank initialization. If set, the corresponding bank is initialized.

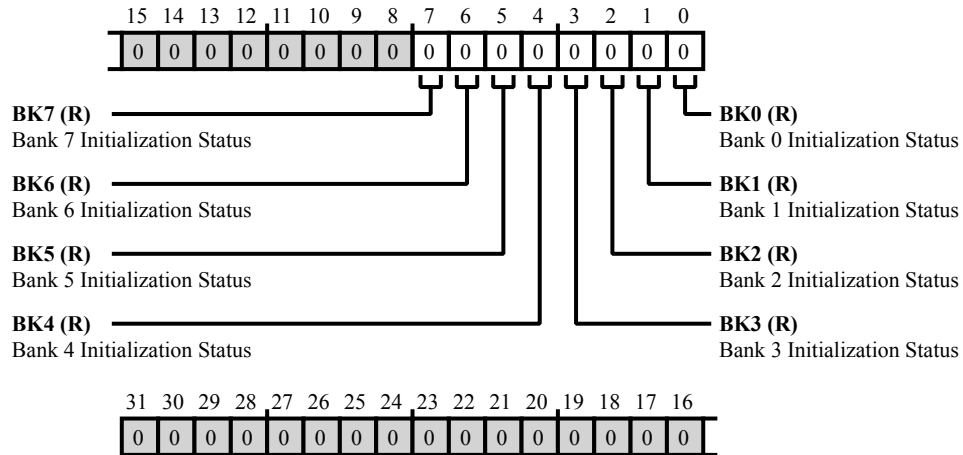


Figure 8-9: L2CTL_ISTAT Register Diagram

Table 8-13: L2CTL_ISTAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7 (R/NW)	BK7	Bank 7 Initialization Status. The <code>L2CTL_ISTAT.BK7</code> bits hold the bank 7 initialization status. A W1A on a <code>BKxINIT</code> bit clears this bit and the bit is set when initialization completes.
6 (R/NW)	BK6	Bank 6 Initialization Status. The <code>L2CTL_ISTAT.BK6</code> bits hold the bank 6 initialization status. A W1A on a <code>BKxINIT</code> bit clears this bit and the bit is set when initialization completes.
5 (R/NW)	BK5	Bank 5 Initialization Status. The <code>L2CTL_ISTAT.BK5</code> bits hold the bank 5 initialization status. A W1A on a <code>BKxINIT</code> bit clears this bit and the bit is set when initialization completes.
4 (R/NW)	BK4	Bank 4 Initialization Status. The <code>L2CTL_ISTAT.BK4</code> bits hold the bank 4 initialization status. A W1A on a <code>BKxINIT</code> bit clears this bit and the bit is set when initialization completes.
3 (R/NW)	BK3	Bank 3 Initialization Status. The <code>L2CTL_ISTAT.BK3</code> bits hold the bank 3 initialization status. A W1A on a <code>BKxINIT</code> bit clears this bit and the bit is set when initialization completes.
2 (R/NW)	BK2	Bank 2 Initialization Status. The <code>L2CTL_ISTAT.BK2</code> bits hold the bank 2 initialization status. A W1A on a <code>BKxINIT</code> bit clears this bit and the bit is set when initialization completes.

Table 8-13: L2CTL_ISTAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/NW)	BK1	Bank 1 Initialization Status. The L2CTL_ISTAT.BK1 bits hold the bank 1 initialization status. A W1A on a BKxINIT bit clears this bit and the bit is set when initialization completes.
0 (R/NW)	BK0	Bank 0 Initialization Status. The L2CTL_ISTAT.BK0 bits hold the bank 0 initialization status. A W1A on a BKxINIT bit clears this bit and the bit is set when initialization completes.

Power Control Register

The `L2CTL_PCTL` register has the various control settings for selectively enabling the deep sleep and shut down power saving features.

NOTE: The corresponding L2 bank should not be accessed if the power control feature is enabled for that bank. An access to a bank in deep sleep/shut down may create unpredictable behavior.

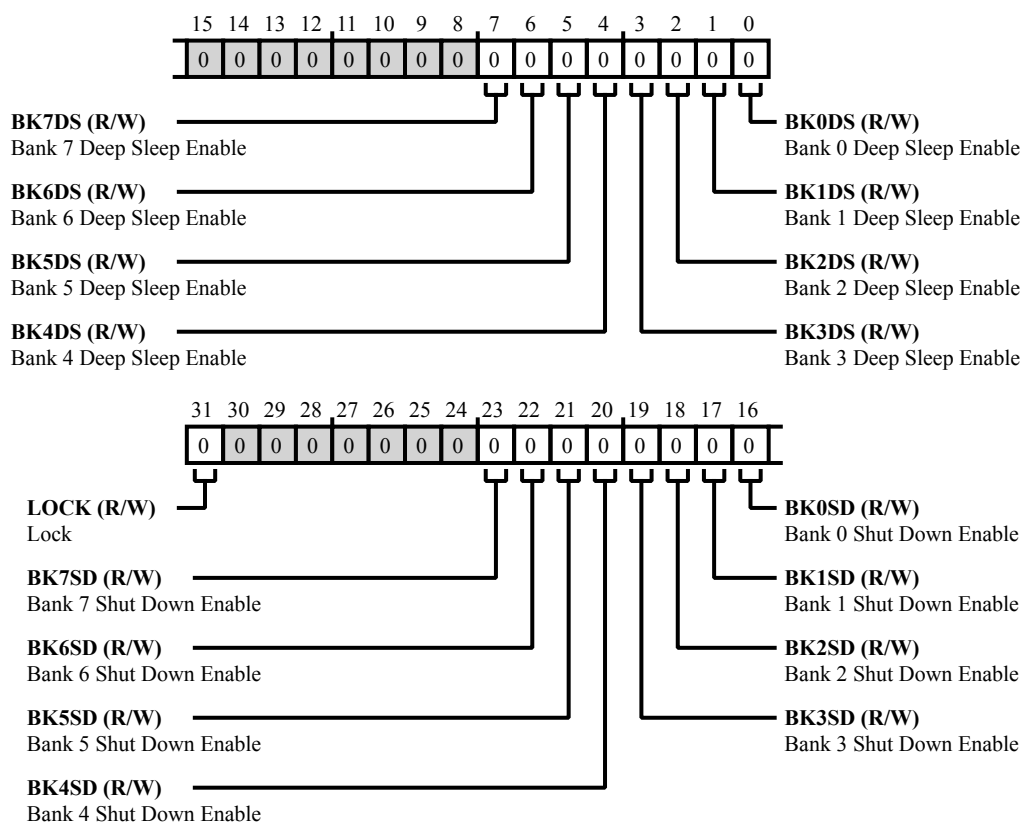


Figure 8-10: L2CTL_PCTL Register Diagram

Table 8-14: L2CTL_PCTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock. If the global lock bit is set (<code>SPU_CTL.GLCK</code> bit =1) and the <code>L2CTL_PCTL.LOCK</code> bit is set, the <code>L2CTL_PCTL</code> register is read only (locked).
		0 Unlock
		1 Lock
23 (R/W)	BK7SD	Bank 7 Shut Down Enable. The <code>L2CTL_PCTL.BK7SD</code> bits enables bank 7 shut down.

Table 8-14: L2CTL_PCTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
22 (R/W)	BK6SD	Bank 6 Shut Down Enable. The L2CTL_PCTL.BK6SD bits enables bank 6 shut down.
21 (R/W)	BK5SD	Bank 5 Shut Down Enable. The L2CTL_PCTL.BK5SD bits enables bank 5 shut down.
20 (R/W)	BK4SD	Bank 4 Shut Down Enable. The L2CTL_PCTL.BK4SD bits enables bank 4 shut down.
19 (R/W)	BK3SD	Bank 3 Shut Down Enable. The L2CTL_PCTL.BK3SD bits enables bank 3 shut down.
18 (R/W)	BK2SD	Bank 2 Shut Down Enable. The L2CTL_PCTL.BK2SD bits enables bank 2 shut down.
17 (R/W)	BK1SD	Bank 1 Shut Down Enable. The L2CTL_PCTL.BK1SD bits enables bank 1 shut down.
16 (R/W)	BK0SD	Bank 0 Shut Down Enable. The L2CTL_PCTL.BK0SD bits enables bank 0 shut down.
7 (R/W)	BK7DS	Bank 7 Deep Sleep Enable. The L2CTL_PCTL.BK7DS bits enables bank 7 deep sleep.
6 (R/W)	BK6DS	Bank 6 Deep Sleep Enable. The L2CTL_PCTL.BK6DS bits enables bank 6 deep sleep.
5 (R/W)	BK5DS	Bank 5 Deep Sleep Enable. The L2CTL_PCTL.BK5DS bits enables bank 5 deep sleep.
4 (R/W)	BK4DS	Bank 4 Deep Sleep Enable. The L2CTL_PCTL.BK4DS bits enables bank 4 deep sleep.
3 (R/W)	BK3DS	Bank 3 Deep Sleep Enable. The L2CTL_PCTL.BK3DS bits enables bank 3 deep sleep.
2 (R/W)	BK2DS	Bank 2 Deep Sleep Enable. The L2CTL_PCTL.BK2DS bits enables bank 2 deep sleep.
1 (R/W)	BK1DS	Bank 1 Deep Sleep Enable. The L2CTL_PCTL.BK1DS bits enables bank 1 deep sleep.
0 (R/W)	BK0DS	Bank 0 Deep Sleep Enable. The L2CTL_PCTL.BK0DS bits enables bank 0 deep sleep.

Parity Error Address 0 Register

The `L2CTL_PERRADDR0` register holds the address containing a Parity error for the corresponding bank. The L2CTL updates this register only if the bank’s status bit (`L2CTL_STAT.PERR0`) is cleared. After the bank’s status bit is set for an error, further errors in the same bank are not detected until a W1C clears the status bit.

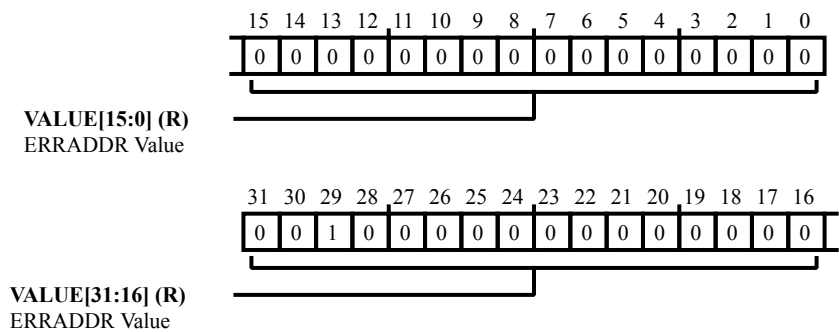


Figure 8-11: L2CTL_PERRADDR0 Register Diagram

Table 8-15: L2CTL_PERRADDR0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	ERRADDR Value. The <code>L2CTL_PERRADDR0.VALUE</code> bits hold the address containing the ECC double-bit error.

Parity Error Address 1 Register

The `L2CTL_PERRADDR1` register holds the address containing a Parity error for the corresponding bank. The L2CTL updates this register only if the bank’s status bit (`L2CTL_STAT.PERR1`) is cleared. After the bank’s status bit is set for an error, further errors in the same bank are not detected until a W1C clears the status bit.

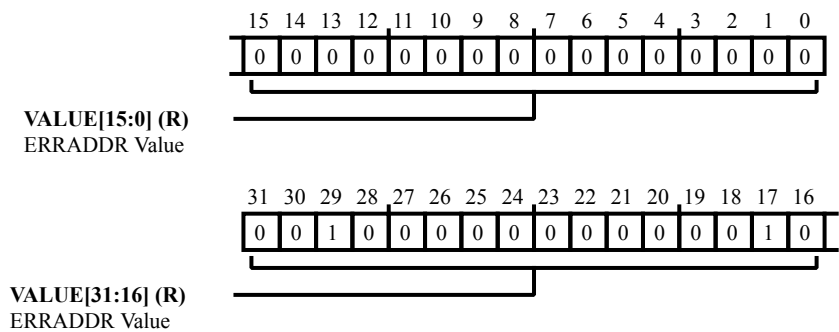


Figure 8-12: L2CTL_PERRADDR1 Register Diagram

Table 8-16: L2CTL_PERRADDR1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	ERRADDR Value. The <code>L2CTL_PERRADDR1.VALUE</code> bits hold the address containing the ECC double-bit error.

Parity Error Address 2 Register

The `L2CTL_PERRADDR2` register holds the address containing a parity error for the corresponding bank. The L2CTL updates this register only if the bank’s status bit (`L2CTL_STAT.PERR2`) is cleared. After the bank’s status bit is set for an error, further errors in the same bank are not detected until a W1C clears the status bit.

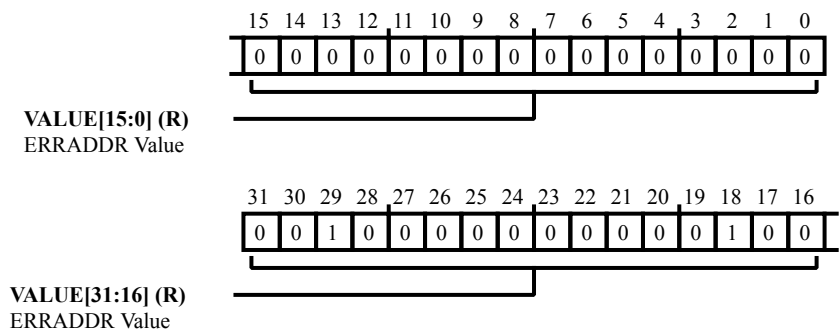


Figure 8-13: L2CTL_PERRADDR2 Register Diagram

Table 8-17: L2CTL_PERRADDR2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	ERRADDR Value. The <code>L2CTL_PERRADDR2.VALUE</code> bits hold the address containing the ECC double-bit error.

Parity Error Address 3 Register

The `L2CTL_PERRADDR3` register holds the address containing an ECC multi-bit error for the corresponding bank. The L2CTL updates this register only if the bank’s status bit (`L2CTL_STAT.PERR3`) is cleared. After the bank’s status bit is set for an error, further errors in the same bank are not detected until a W1C clears the status bit.

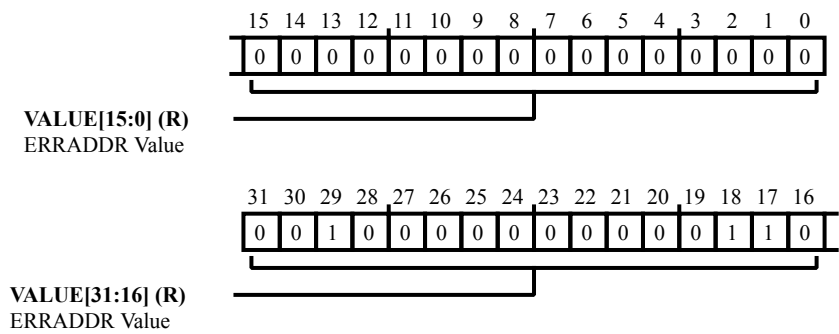


Figure 8-14: L2CTL_PERRADDR3 Register Diagram

Table 8-18: L2CTL_PERRADDR3 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	ERRADDR Value. The <code>L2CTL_PERRADDR3.VALUE</code> bits hold the address containing the ECC double-bit error.

Parity Error Address 4 Register

The `L2CTL_PERRADDR4` register holds the address containing a Parity error for the corresponding bank. The L2CTL updates this register only if the bank’s status bit (`L2CTL_STAT.PERR4`) is cleared. After the bank’s status bit is set for an error, further errors in the same bank are not detected until a W1C clears the status bit.

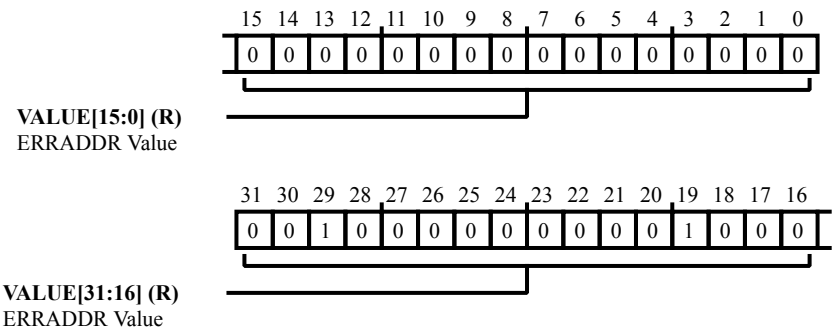


Figure 8-15: L2CTL_PERRADDR4 Register Diagram

Table 8-19: L2CTL_PERRADDR4 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	ERRADDR Value. The <code>L2CTL_PERRADDR4.VALUE</code> bits hold the address containing the ECC double-bit error.

Parity Error Address 5 Register

The `L2CTL_PERRADDR5` register holds the address containing a Parity error for the corresponding bank. The L2CTL updates this register only if the bank’s status bit (`L2CTL_STAT.PERR5`) is cleared. After the bank’s status bit is set for an error, further errors in the same bank are not detected until a W1C clears the status bit.

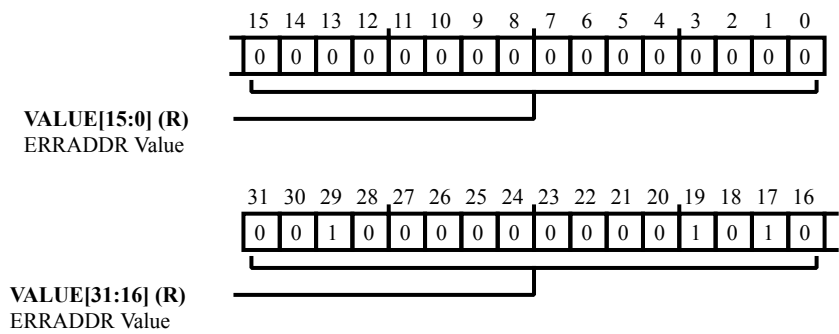


Figure 8-16: L2CTL_PERRADDR5 Register Diagram

Table 8-20: L2CTL_PERRADDR5 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	ERRADDR Value. The <code>L2CTL_PERRADDR5.VALUE</code> bits hold the address containing the ECC double-bit error.

Parity Error Address 6 Register

The `L2CTL_PERRADDR6` register holds the address containing a Parity error for the corresponding bank. The L2CTL updates this register only if the bank’s status bit (`L2CTL_STAT.PERR6`) is cleared. After the bank’s status bit is set for an error, further errors in the same bank are not detected until a W1C clears the status bit.

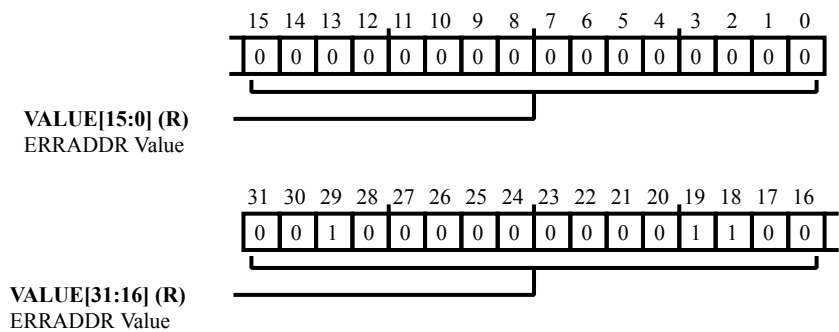


Figure 8-17: L2CTL_PERRADDR6 Register Diagram

Table 8-21: L2CTL_PERRADDR6 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	ERRADDR Value. The <code>L2CTL_PERRADDR6.VALUE</code> bits hold the address containing the ECC double-bit error.

Parity Error Address 7 Register

The `L2CTL_PERRADDR7` register holds the address containing a Parity error for the corresponding bank. The L2CTL updates this register only if the bank’s status bit (`L2CTL_STAT.PERR7`) is cleared. After the bank’s status bit is set for an error, further errors in the same bank are not detected until a W1C clears the status bit.

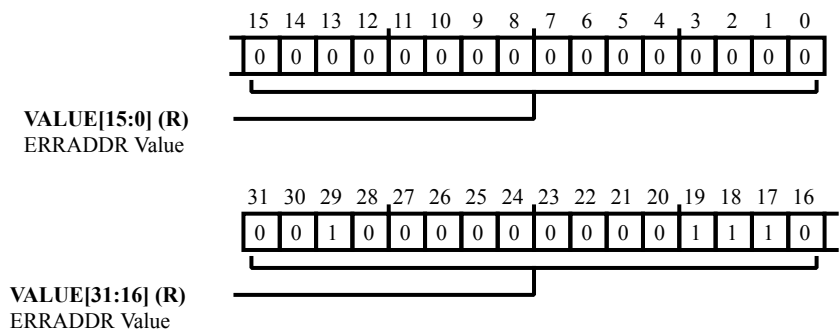


Figure 8-18: L2CTL_PERRADDR7 Register Diagram

Table 8-22: L2CTL_PERRADDR7 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	ERRADDR Value. The <code>L2CTL_PERRADDR7.VALUE</code> bits hold the address containing the ECC double-bit error.

Parity Error Address 8 Register

The `L2CTL_PERRADDR8` register holds the address containing an ECC multi-bit error for the corresponding bank. The L2CTL updates this register only if the bank’s status bit (`L2CTL_STAT.PERR8`) is cleared. After the bank’s status bit is set for an error, further errors in the same bank are not detected until a W1C clears the status bit.

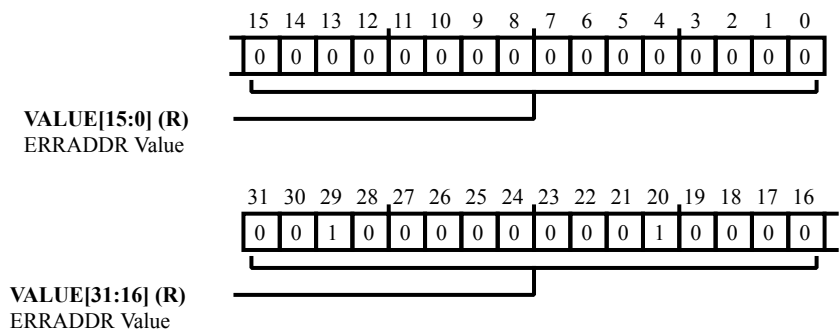


Figure 8-19: L2CTL_PERRADDR8 Register Diagram

Table 8-23: L2CTL_PERRADDR8 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	ERRADDR Value. The <code>L2CTL_PERRADDR8.VALUE</code> bits hold the address containing the ECC double-bit error.

Revision ID Register

The `L2CTL_REVID` register provides the L2 Revision ID.

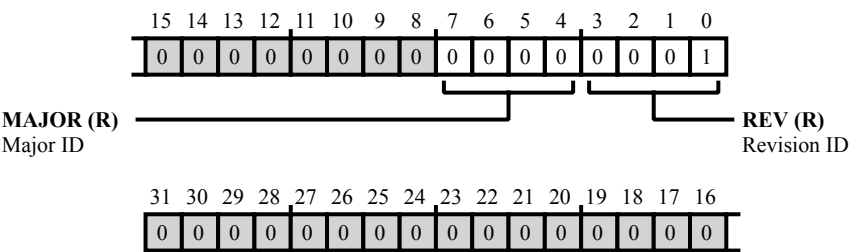


Figure 8-20: L2CTL_REVID Register Diagram

Table 8-24: L2CTL_REVID Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:4 (R/NW)	MAJOR	Major ID. The <code>L2CTL_REVID.MAJOR</code> bit indicates L2 Major ID.
3:0 (R/NW)	REV	Revision ID. The <code>L2CTL_REVID.REV</code> bit indicates the L2 Revision ID.

Read Priority Count Register

The `L2CTL_RPCR` register stores the count value to be used for priority elevation for bus read channels. If a bus channel is not granted access from the bank arbiter, the channel waits for the programmed number of SCLK cycles, before the request is elevated to a high priority request. If a priority count value is programmed as zero for a channel, that channel does not raise the urgent priority request.

This is a read/write register, but a new value in the corresponding field must be written only when there are no outstanding transactions on the corresponding bus read channel. A best practice is to program this register before initiating an L2 access.

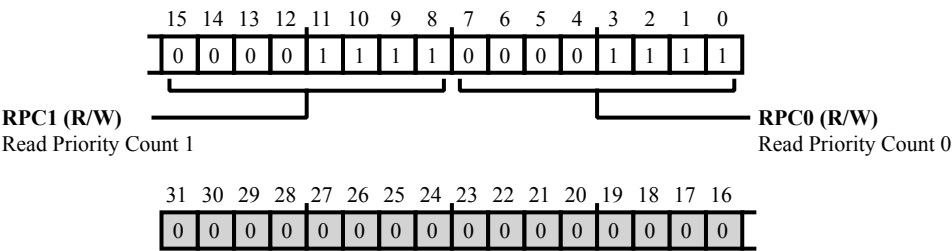


Figure 8-21: L2CTL_RPCR Register Diagram

Table 8-25: L2CTL_RPCR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:8 (R/W)	RPC1	Read Priority Count 1. The <code>L2CTL_RPCR.RPC1</code> bits hold the priority count for L2 bus read channel 1.
7:0 (R/W)	RPC0	Read Priority Count 0. The <code>L2CTL_RPCR.RPC0</code> bits hold the priority count for L2 bus read channel 0.

Status Register

The `L2CTL_STAT` register indicates ECC error status, refresh register status, and bus error status of Port0 and Port1.

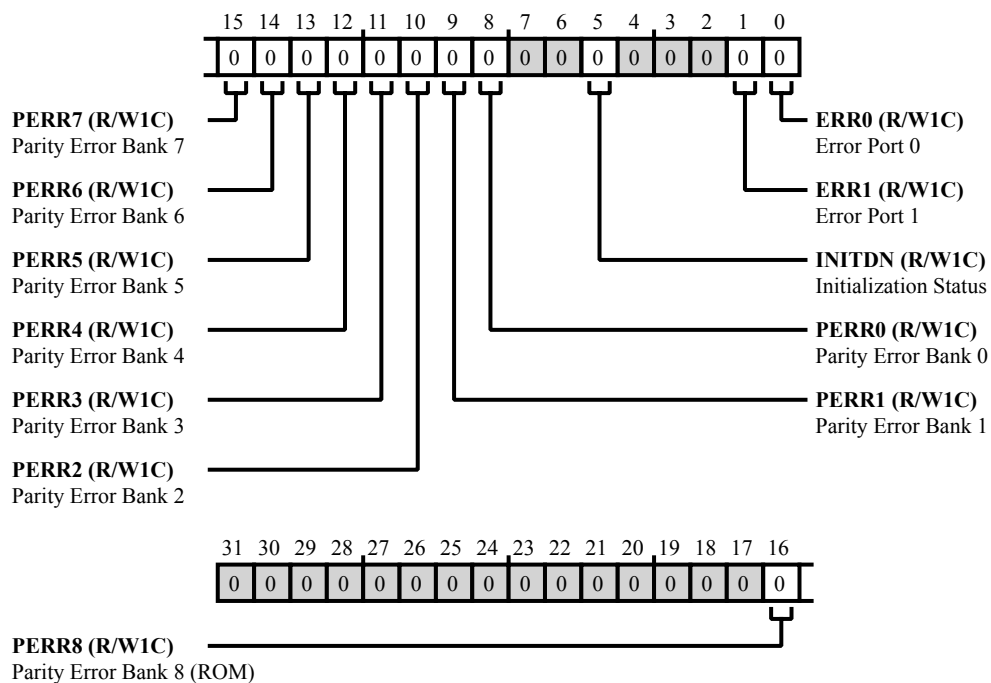


Figure 8-22: L2CTL_STAT Register Diagram

Table 8-26: L2CTL_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
16 (R/W1C)	PERR8	Parity Error Bank 8 (ROM). The <code>L2CTL_STAT.PERR8</code> bit indicates that an ECC double-bit error occurred inside L2 bank 8 (ROM).
		0 No Status
		1 ECC Double Bit Error
15 (R/W1C)	PERR7	Parity Error Bank 7. The <code>L2CTL_STAT.PERR7</code> bit indicates that an ECC double-bit error occurred inside L2 bank 7.
		0 No Status
		1 ECC Double Bit Error

Table 8-26: L2CTL_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
14 (R/W1C)	PERR6	Parity Error Bank 6. The L2CTL_STAT . PERR6 bit indicates that an ECC double-bit error occurred inside L2 bank 6.
		0 No Status
		1 ECC Double Bit Error
13 (R/W1C)	PERR5	Parity Error Bank 5. The L2CTL_STAT . PERR5 bit indicates that an ECC double-bit error occurred inside L2 bank 5.
		0 No Status
		1 ECC Double Bit Error
12 (R/W1C)	PERR4	Parity Error Bank 4. The L2CTL_STAT . PERR4 bit indicates that an ECC double-bit error occurred inside L2 bank 4.
		0 No Status
		1 ECC Double Bit Error
11 (R/W1C)	PERR3	Parity Error Bank 3. The L2CTL_STAT . PERR3 bit indicates that an ECC double-bit error occurred inside L2 bank 3.
		0 No Status
		1 ECC Double Bit Error
10 (R/W1C)	PERR2	Parity Error Bank 2. The L2CTL_STAT . PERR2 bit indicates that an ECC double-bit error occurred inside L2 bank 2.
		0 No Status
		1 ECC Double Bit Error
9 (R/W1C)	PERR1	Parity Error Bank 1. The L2CTL_STAT . PERR1 bit indicates that an ECC double-bit error occurred inside L2 bank 1.
		0 No Status
		1 ECC Double Bit Error

Table 8-26: L2CTL_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
8 (R/W1C)	PERR0	Parity Error Bank 0. The L2CTL_STAT.PERR0 bit indicates that an ECC double-bit error occurred inside L2 bank 0.
		0 No Status
		1 ECC Double Bit Error
5 (R/W1C)	INITDN	Initialization Status. The L2CTL_STAT.INITDN bit indicates whether initialization has been completed.
		0 Initialization Not Complete
		1 Initialization Completed
1 (R/W1C)	ERR1	Error Port 1. The L2CTL_STAT.ERR1 indicates whether the L2CTL has detected a bus access error on L2s bus port 1.
		0 No Error
		1 Bus Access Error
0 (R/W1C)	ERR0	Error Port 0. The L2CTL_STAT.ERR0 indicates whether the L2CTL has detected a bus access error on L2s bus port 0.
		0 No Error
		1 Bus Access Error

Write Priority Count Register

The `L2CTL_WPCR` register stores the count value to be used for priority elevation for bus write channels. If a bus channel is not granted access from the bank arbiter, the channel waits for the programmed number of SCLK cycles, before the request is elevated to a high priority request. If a priority count value is programmed as zero for a channel, that channel does not raise the urgent priority request.

This is a read/write register, but a new value in the corresponding field must be written only when there are no outstanding transactions on the corresponding bus write channel. A best practice is to program this register before initiating an L2 access.

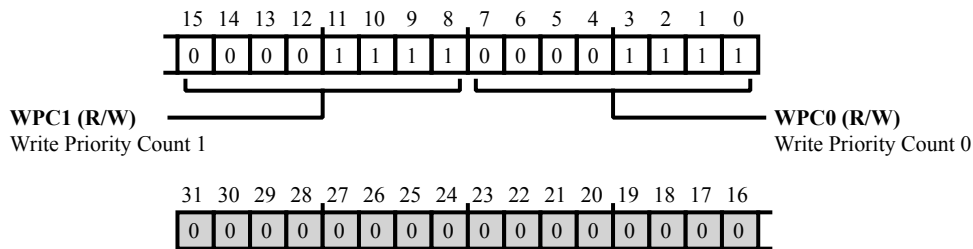


Figure 8-23: L2CTL_WPCR Register Diagram

Table 8-27: L2CTL_WPCR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:8 (R/W)	WPC1	Write Priority Count 1. The <code>L2CTL_WPCR.WPC1</code> bits hold the priority count for L2 bus write channel 1.
7:0 (R/W)	WPC0	Write Priority Count 0. The <code>L2CTL_WPCR.WPC0</code> bits hold the priority count for L2 bus write channel 0.

9 One-Time Programmable Memory Controller (OTPC)

This chapter describes the operation of the OTP controller. The OTP module is a complete system integrating an OTP memory core with a programming controller, charge pump, and voltage regulator. A built-in Hamming Code Error Correction (ECC), and a fully implemented double-redundant program or read scheme protect the OTP data.

CAUTION: OTP memory does not support burst transfers, which are required to support cache line fills. As such, OTP memory should not be made cacheable. If it is, the OTP controller returns an error when a read access is attempted.

OTPC Features

The OTP memory and controller have the following features:

- Built-in redundant read mode
- Built-in integrated power supply
- Built-in Hamming Code Error Correction (ECC)
- Full word serial (single bit at a time) programming with internal VPP

NOTE: The OTP module is protected by the System Memory Protection Unit (SMPU) and is always secure by default. Non secure access to the OTP by any module will fail unless allowed by the SMPU.

OTPC Functional Description

ADSP-21560_21561_21564_21568 OTPC Register List

The One-Time-Programmable Memory controller (OTPC) supports programming the OTP memory. A set of registers governs OTPC operations. For more information on OTPC functionality, see the OTPC register descriptions.

Table 9-1: ADSP-21560_21561_21564_21568 OTPC Register List

Name	Description
OTPC_SECU_STATE	OTP Security State Register
OTPC_STAT	OTP Status Register

ADSP-21560_21561_21564_21568 OTPC Interrupt List

Table 9-2: ADSP-21560_21561_21564_21568 OTPC Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
5	OTPC0_ERR	OTPC0 Dual-bit error	Level	

Error Correction

The OTP memory features a Hamming error correction implementation. Signal bit errors are automatically corrected, and dual-bit errors are detected. Refer to [OTPC Interrupt Signals](#).

ECC is always enabled. ECC applies to each 16-bit segment. Because of this functionality, each 16-bit location can only be written to once. Writing to a 16-bit location a second time results in unexpected behavior.

OTP Layout

This section details the memory layout of the OTP memory.

Table 9-3: ADSP-21560/21561/21564/21568 OTP Layout

Name	32-bit Aligned Byte Address	Size (bits)	Description
huk	0x5c	256	Hardware Unique Key
rkek	0x7c	128	Root Key Encryption Key
dek	0x8c	128	Local Encryption Key
oem_public_key	0x9c	512	OEM Public Key
pvt_128key0	0xdc	128	Customer Privatekey 0 128bits
pvt_128key1	0xec	128	Customer Privatekey 1 128bits
pvt_128key2	0xfc	128	Customer Privatekey 2 128bits
pvt_128key3	0x10c	128	Customer Privatekey 3 128 bits
ek	0x11c	256	Endorsement Key
secure_emu_key0	0x13c	256	Secure Emulation Key0
secure_emu_key1	0x15c	256	Secure Emulation Key1

Table 9-3: ADSP-21560/21561/21564/21568 OTP Layout (Continued)

Name	32-bit Aligned Byte Address	Size (bits)	Description
reserved	0x180	640	Reserved
emu_key_disable0	0x1d0	16	Secure Emulation Key0 Disable
emu_key_disable1	0x1d0	16	Secure Emulation Key1 Disable
public_key0	0x1d4	512	Customer Public Key0
public_key1	0x214	512	Customer Public Key1
watchdog_count	0x254	32	Watchdog Count
reserved	0x258	32	Reserved
cgu_ctl_WEN	0x25c	1	CGU Config ctl_WEN
cgu_div_WEN	0x25c	1	CGU Config div_WEN
cgu_reserved0	0x25c	1	CGU Config reserved0
cgu_div_DSEL	0x25c	5	CGU Config div_DSEL
cgu_div_CSEL	0x25c	5	CGU Config div_CSEL
cgu_div_S0SEL	0x25c	3	CGU Config div_S0SEL
cgu_div_SYSSSEL	0x25c	5	CGU Config div_SYSSSEL
cgu_div_S1SEL	0x25c	3	CGU Config div_S1SEL
cgu_div_OSEL	0x25c	7	CGU Config div_OSEL
cgu_ctl_DF	0x268	1	CGU Config ctl_DF
cgu_ctl_MSEL	0x260	7	CGU Config reserved1
cgu_reserved1	0x260	1	CGU Config ctl_S0SELEXEN
cgu_ctl_S1SELEX-EN	0x260	1	CGU Config ctl_S1SELEXEN
cgu_auto_disable	0x260	1	CGU Disable Auto Alignment
cgu_clkoutsel_CLKOUTSEL	0x260	5	CGU Config clkoutsel_CLKOUTSEL
cgu_clkoutsel_WEN	0x260	1	CGU Config clkoutsel_WEN
cgu_reserved1	0x260	8	CGU Config reserved2
cgu_divex_S1SELEX	0x260	8	CGU Config divex_S1SELEX
flashStartAddress	0x264	32	Flash Start Address Override
altflashStartAddress	0x268	32	Alternate Flash Start Address Override
spiMasterBootCmd	0x26c	32	SPI Master Boot Command Override
spiSlaveBootCmd	0x270	32	SPI Slave Boot Command Override

Table 9-3: ADSP-21560/21561/21564/21568 OTP Layout (Continued)

Name	32-bit Aligned Byte Address	Size (bits)	Description
uartBootCmd	0x274	32	UART Boot Command Override
xspiMasterBootCmd	0x278	32	xSPI Boot Command Override
altSlaveBootcmd	0x27c	32	Alternate Slave Boot Command Override
xspiMaster-BootCmd1	0x280	32	xSPI Boot Command 1 Override
spi1MasterBootCmd	0x284	32	SPI1 Master Boot Command Override
bcfg_pubkey0Inv	0x288	1	Boot Config pubkey0Inv
bcfg_reserved0	0x288	15	Boot Config reserved0
bcfg_pubkey1Inv	0x288	1	Boot Config pubkey1Inv
bcfg_reserved1	0x288	15	Boot Config reserved1
bcfg_privkey0Inv	0x28c	1	Boot Config privkey0Inv
bcfg_reserved2	0x28c	15	Boot Config reserved2
bcfg_privkey1Inv	0x28c	1	Boot Config privkey1Inv
bcfg_reserved3	0x28c	15	Boot Config reserved3
bcfg_privkey2Inv	0x290	1	Boot Config privkey2Inv
bcfg_reserved4	0x290	15	Boot Config reserved4
bcfg_privkey3Inv	0x290	1	Boot Config privkey3Inv
bcfg_reserved5	0x290	15	Boot Config reserved5
bcfg_xspiramEn	0x294	1	Boot Config xspiramEn
bcfg_reserved6	0x294	15	Boot Config reserved6
Bcfg_xspiramInv	0x294	1	Boot Config xspiramInv
bcfg_reserved7	0x294	15	Boot Config reserved7
bcfg_dmcDecDis	0x298	1	Boot Config dmcDecDis
bcfg_reserved8	0x298	15	Boot Config reserved8
bcfg_altMstrBootEn	0x298	1	Boot Config altMsterBootEn
bcfg_reserved9	0x298	15	Boot Config reserved9
bcfg_altMstrBootDis	0x29c	1	Boot Config altMsterBootDis
bcfg_reserved10	0x29c	15	Boot Config reserved10
bcfg_altSlaveBootEn	0x29c	1	Boot Config altSlaveBootEn
bcfg_reserved11	0x29c	15	Boot Config reserved11
bcfg_altSlaveBootDis	0x2A0	1	Boot Config altSlaveBootDis

Table 9-3: ADSP-21560/21561/21564/21568 OTP Layout (Continued)

Name	32-bit Aligned Byte Address	Size (bits)	Description
bcfg_reserved12	0x2A0	15	Boot Config reserved12
bcfg_benchmark_En	0x2A0	1	Boot Config benchmark_En
bcfg_reserved13	0x2A0	15	Boot Config reserved13
bcfg_wdtDis	0x2A4	1	Boot Config wdtDis
bcfg_reserved14	0x2A4	15	Boot Config reserved14
bcfg_fastcguDis	0x2A4	1	Boot Config fastcguDis
bcfg_reserved15	0x2A4	15	Boot Config reserved15
otpTiming	0x2A8	32	OTP timing override value
reserved	0x2ac	160	Reserved
xspi_phyconfig0	0x2c0	32	xSPI Phyconfig0
xspi_phyconfig1	0x2c4	32	xSPI Phyconfig1
xspi_phyconfig2	0x2c8	32	xSPI Phyconfig2
xspi_phyconfig3	0x2cc	32	xSPI Phyconfig3
antiroll_nv_cntr	0x2d0	512	AntiRollback NV Counter
gp1	0x310	384	General Purpose 1
reserved	0x340	24	Reserved
bootModeDisable	0x340	8	Boot Mode Disable Bits
xspi_sram_phyconfig0	0x344	32	xSPI SRAM Phyconfig0
xspi_sram_phyconfig1	0x348	32	xSPI SRAM Phyconfig1
xspi_sram_phyconfig2	0x34c	32	xSPI SRAM Phyconfig2
xspi_sram_phyconfig3	0x350	32	xSPI SRAM Phyconfig3
xspi_sram_remap-Config	0x354	2	xSPI SRAM remapConfig
xspi_sram_device-type	0x354	1	xSPI SRAM devicetype
xspi_sram_Address-bytes	0x354	2	xSPI SRAM Addressbytes
xspi_sram_bytesper-addr	0x354	1	xSPI SRAM bytesperaddr

Table 9-3: ADSP-21560/21561/21564/21568 OTP Layout (Continued)

Name	32-bit Aligned Byte Address	Size (bits)	Description
xspi_sram_byteswap	0x354	1	xSPI SRAM byteswap
xspi_sram_reserved0	0x354	1	Reserved0
xspi_sram_csda_min delay	0x354	8	xSPI SRAM csda_mindelay
xspi_sram_clockDiv	0x354	8	xSPI SRAM clockDiv
xspi_sram_readop- code1	0x354	8	xSPI SRAM readopcode1
xspi_sram_readop- code2	0x358	8	xSPI SRAM readopcode2
xspi_sram_writeop- code1	0x358	8	xSPI SRAM writeopcode1
xspi_sram_writeop- code2	0x358	8	xSPI SRAM writeopcode2
xspi_sram_readlaten- cy	0x358	8	xSPI SRAM readlatency
xspi_sram_writelat- ency	0x35c	8	xSPI SRAM writelatency
xspi_sram_busmode	0x35c	8	xSPI SRAM busmode
xspi_sram_slavesel	0x35c	2	xSPI SRAM slavesel
xspi_sram_wen_ena- ble	0x35c	1	xSPI SRAM wen_enable
xspi_sram_wen_op- codeexenable	0x35c	1	xSPI SRAM wen_opcodeexenable
xspi_sram_wen_op- codeex	0x35c	8	xSPI SRAM wen_opcodeex
xspi_sram_reserved1	0x35c	4	Reserved1
xspi_sram_dev_ac- tive_max	0x360	32	xSPI SRAM dev_active_max
reserved	0x364	128	Reserved
stageID	0x374	48	StageID
reserved	0x378	16	Reserved
fsn	0x380	128	Factory Serial Number
reserved	0x424	16	Reserved
lock	0x424	2	lockbit

Table 9-3: ADSP-21560/21561/21564/21568 OTP Layout (Continued)

Name	32-bit Aligned Byte Address	Size (bits)	Description
reserved	0x424	14	Reserved
reserved	0xC24	16	Reserved
FA_EN	0xC24	1	Field Analysis Enable
reserved	0xC24	15	Reserved

OTPC Event Control

The following sections provide information on OTP events and error management.

OTPC Interrupt Signals

When making 32-bit accesses to OTP memory, a double-bit error in any 16-bit segment triggers the OTPC_INT interrupt. The OTPC also has the OTPC dual bit error (OTPC0_ERR) with the SEC ID of 5.

OTPC Status and Error Signals

The OTP controller does not produce error signals.

OTP API Overview

The ROM/OTP SSLDD library provides a set of functions to facilitate OTP field access. The OTP memory is broken up into a set of specialized fields that are described in this section. The API removes the requirement of understanding the details of the layout or OTP access procedures.

All OTP accesses are made through the provided API.

OTP Programming

The OTP programming API provides a simple access, abstracting particulars of the OTP controller.

Any fields that contain zero or null pointers are skipped.

All addresses are assumed to be byte addresses unless otherwise noted.

OTP Reading

This API provides a unified source for retrieving OTP data fields.

All addresses are assumed to be byte addresses, unless otherwise noted.

A list of APIs follow:

	OTP Get Field
--	-------------------------------

<pre>bool adi_rom_otp_get(OTPCMD cmd, uint32_t* data);</pre>	
---	--

OTP Get Field

Retrieves indicated data from OTP memory.

Name	OTP Get Field	
Prototype	bool adi_rom_otp_get(OTPCMD cmd, uint32_t* data);	
Argument	cmd	Indicates what data to fetch, based on theOTPCMD enum.
Argument	data	memory location to write the data to
Return Value	bool	true for a successful read
Stack Requirements	valid stack	

```
bool res = adi_rom_otp_get(otpcmd_info,data);
```

The data specified by the OTPCMD enum parameter is fetched from OTP memory and placed in the location specified by data. The OTPCMD enum contains entries for each field defined in OTP memory, for the most current list, refer to the OTP header file.

ADSP-21560_21561_21564_21568 OTPC Register Descriptions

OTP Memory Controller (OTPC) contains the following registers.

Table 9-4: ADSP-21560_21561_21564_21568 OTPC Register List

Name	Description
OTPC_SECU_STATE	OTP Security State Register
OTPC_STAT	OTP Status Register

OTPC Security State Register

The `OTPC_SECU_STATE` register indicates the secure state. The register is updated only after the part is rebooted and locked.

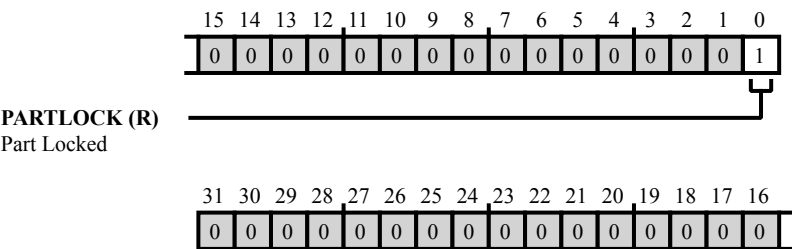


Figure 9-1: OTPC_SECU_STATE Register Diagram

Table 9-5: OTPC_SECU_STATE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/NW)	PARTLOCK	Part Locked. The <code>OTPC_SECU_STATE.PARTLOCK</code> field indicates a locked part.
		0 OPEN part
		2 Reserved
		1 Locked part
		3 Reserved

OTPC Status Register

The `OTPC_STAT` register bits indicate errors and flag status and control the protection bits.

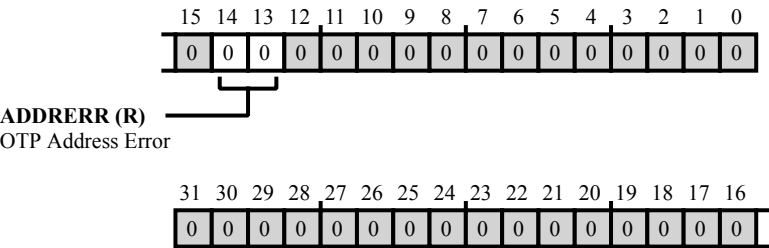


Figure 9-2: OTPC_STAT Register Diagram

Table 9-6: OTPC_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
14:13 (R/NW)	ADDRERR	OTP Address Error. The <code>OTPC_STAT.ADDRERR</code> bit field indicates errors which occur when the OTP programming address is out of range or tries to access protected space.
		0 No error - proper OTP address
		1 OTP address out of range
		2 8-bit OTP address
		3 Protected OTP address

10 System Memory Protection Unit (SMPU)

The SMPU provides a flexible way of protecting memory regions against read or write access from any or all bus requester in the system. In addition, it can guard against memory access depending on security privileges of the system bus requester.

SMPU Features

The system memory protection unit has the following features.

- After reset, the default state of the system is fully open. The SMPUs admit any access to memory spaces by any controller.
- Each SMPU instance can be configured to monitor multiple regions. Each can be individually enabled.
- Each region can be configured with its own protection settings.
- Provides general read or write protection.
- Read and write transactions are restricted or allowed depending on the transaction ID.

On the ADSP-2156x processor, four SMPU instances are available to protect the L2.

Table 10-1: SMPU Instances

Module	SMPU Instance
L2 Core Port 0	2
L2 DMA Port 0	3
L2 DMA Port 1	9
SPI1/2 and xSPI	11

All the SMPU instances can be configured up to eight regions.

- Up to eight outstanding read or write transactions supported on SMPU instances for Core_L2. Up to four outstanding read or write transactions supported on the SMPU instances for DMA_L2 and SPI flash address space.

SMPU Functional Description

The following sections provide details on the function of the SMPU module. If the region security settings allow transactions to go through, the ID in the ID-based region protection settings can still filter the transactions.

For the memory that an SMPU protects, programs can configure region-based settings with the `SMPU_RCTL[n]` registers. (There can be multiple SMPUs in a system.) The `SMPU_RCTL[n]` registers define the ID-based protection for memory regions.

If the target address does not reside in any configured memory region, the transaction permission resorts back to the global configuration setting.

Protection Units

Each SMPU provides two protection units, A and B for ID-based matching in the region-based memory protection. This feature provides a degree of flexibility for the user to match against multiple IDs.

Instruction Fetches

When the core executes instructions from memory, this operation is also considered a memory transaction. If the SMPU is configured to protect a memory region from read accesses that contain instructions, the core cannot fetch and execute these instructions.

Using Cache

When the processor uses both cache and the SMPU, there are a few issues to be aware of. If the SMPU is configured to protect a memory region from write accesses, instruction fetches from a core are still possible since instructions are not updated and replaced during run time.

NOTE: The debugger typically replaces an instruction with a breakpoint instruction for software breakpoints. If a memory region is protected against write accesses, software breakpoints are not possible unless the SMPU is configured with the appropriate system controller ID of the debugger. The configuration allows it to perform a write-access.

In the case where memory is used for data, a read access or cache fill is not possible if the memory is blocked from read accesses. If read accesses are allowed but write accesses are disallowed, then there is an issue with coherency. The cache is filled but when the cache is updated and must be written back to the SMPU protected memory, the write-access is blocked.

In general, exercise caution when using both the SMPU and cache.

Speculative Reads

If speculative reads are enabled (`SMPU_CTL.RSDIS = 0`), the SMPU forwards the read transaction directly to the memory before checking the protection setting corresponding to the addressed memory region. This functionality saves one clock cycle in the clock domain of the SMPU. The SMPU checks the protection setting while the read

transaction occurs with the memory. If the protection setting dictates that the target memory address is blocked, the SMPU blocks the read to the controller.

If speculative reads are disabled (`SMPU_CTL.RSDIS = 1`), the SMPU checks the protection settings first and forwards the transaction to memory only if it passes the configured protection settings. This functionality incurs a one-cycle latency per read.

NOTE: Reads affect certain memory operations such as automatic clearing of the memory (that is, FIFOs). When the SMPU protects this type of memory, disable read speculation since the blocking can occur without the read transaction reaching the target memory.

ADSP-21560_21561_21564_21568 SMPU Register List

The System Memory Protection Unit (SMPU) provides selective protection of the processor's memory resources. The SMPU includes a set of processor events that can be monitored during program execution. A set of registers governs SMPU operations. For more information on SMPU functionality, see the SMPU register descriptions.

Table 10-2: ADSP-21560_21561_21564_21568 SMPU Register List

Name	Description
<code>SMPU_BADDR</code>	Bus Error Address Register
<code>SMPU_BDTLS</code>	Bus Error Details Register
<code>SMPU_CTL</code>	SMPU Control Register
<code>SMPU_IADDR</code>	Interrupt Address Register
<code>SMPU_IDTLS</code>	Interrupt Details Register
<code>SMPU_RADDR[n]</code>	Region n Address Register
<code>SMPU_RCTL[n]</code>	Region n Control Register
<code>SMPU_REVID</code>	SMPU Revision ID Register
<code>SMPU_RIDA[n]</code>	Region n ID A Register
<code>SMPU_RIDB[n]</code>	Region n ID B Register
<code>SMPU_RIDMSKA[n]</code>	Region n ID Mask A Register
<code>SMPU_RIDMSKB[n]</code>	Region n ID Mask B Register
<code>SMPU_SECURECTL</code>	SMPU Control Secure Accesses Register
<code>SMPU_SECURERCTL[n]</code>	Region n Control Secure Accesses Register
<code>SMPU_STAT</code>	SMPU Status Register

SMPU Interrupts

The SMPU has one interrupt with the SEC ID = 187.

See the SEC chapter for complete information on interrupt generation and use.

Memory Writes

A write transaction to address n is prevented when the following is true.

Address n is in memory region m and memory region m is write-protected ($\text{SMPU_RCTL}[n].\text{WPROTEN} = 1$) and ID is not a match. (See [ID Comparison](#)). The block occurs because the memory region is configured for write-protection and the ID comparison does not result in a match. If an ID comparison results in a match, the write transaction is allowed through.

Memory Reads

A read transaction from address n is prevented when the following is true:

- Address n is in memory region and memory region m is read-protected ($\text{SMPU_RCTL}[n].\text{RPROTEN} = 1$) and
- ID is not a match

The block occurs because the memory region is configured for read protection and the ID comparison does not result in a match. If the ID comparison results in a match, the read transaction is permitted. (See [ID Comparison](#)).

ID Comparison

ID comparison automatically occurs during region-based memory protection. ID matches allow the transaction to bypass the configured memory protection for that region. The following sections describe the calculation of a write ID match and read ID match.

Write Transaction

The state of the following values determines the ID value that is compared with the ID of an incoming write transaction:

- The $\text{SMPU_RCTL}[n].\text{WIDCINV}$ bit
- The $\text{SMPU_RIDA}[n].\text{ID}$ and $\text{SMPU_RIDB}[n].\text{ID}$ bit fields
- The $\text{SMPU_RIDMSKA}[n].\text{MSK}$ and $\text{SMPU_RIDMSKB}[n].\text{MSK}$ bit fields

Write IDA match = $((\text{ID of incoming write transaction AND SMPU_RIDMSKA}[n].\text{MSK}) == (\text{SMPU_RIDA}[n].\text{ID AND SMPU_RIDMSKA}[n].\text{MSK}))$

Write IDB match = $((\text{ID of incoming write transaction AND SMPU_RIDMSKB}[n].\text{MSK}) == (\text{SMPU_RIDB}[n].\text{ID AND SMPU_RIDMSKB}[n].\text{MSK}))$

Write ID match = $(\text{Write IDA match OR Write IDB match}) \text{ XOR } \text{SMPU_RCTL}[n].\text{WIDCINV}$ bit

Read Transaction

The state of the following values determines the ID value that is compared with the ID of an incoming read transaction:

- The `SMPU_RCTL[n].RIDCINV` bit
- The `SMPU_RIDA[n].ID` and `SMPU_RIDB[n].ID` bit fields
- The `SMPU_RIDMSKA[n].MSK` and `SMPU_RIDMSKB[n].MSK` bit fields

Read IDA match = ((ID of incoming read transaction AND `SMPU_RIDMSKA[n].MSK`) == (`SMPU_RIDA[n].ID` AND `SMPU_RIDMSKA[n].MSK`))

Read IDB match = ((ID of incoming read transaction AND `SMPU_RIDMSKB[n].MSK`) == (`SMPU_RIDB[n].ID` AND `SMPU_RIDMSKB[n].MSK`))

Read ID match = (Read IDA match OR Read IDB match) XOR `SMPU_RCTL[n].RIDCINV`

In the two cases described above, the incoming transaction (either write or read) ID is AND'ed with the configured mask value in protection unit A. It is then compared to the value of the configured ID value which is also AND'ed with the configured mask value in protection unit A. The mask provides a method to allow a group of IDs to match. This process is also performed for protection unit B. The two outcomes (from A and B) are then OR'ed together.

Depending on the setting of the `SMPU_RCTL[n].RIDCINV` or the `SMPU_RCTL[n].WIDCINV` bits, the ID match comparison is inverted or not. The final result after applying the inversion, `SMPU_RCTL[n].RIDCINV`, or `SMPU_RCTL[n].WIDCINV`, determines whether the transaction bypasses the protection.

Usage

The masks, `SMPU_RIDMSKA[n]` and `SMPU_RIDMSKB[n]`, are AND'ed with both the incoming transaction ID and the configured ID in `SMPU_RIDA[n].ID` and `SMPU_RIDB[n].ID`, respectively. By default the masks are zero. If ID-based region protection is enabled by setting the `SMPU_RCTL[n].WPROTEN` or `SMPU_RCTL[n].RPROTEN` bit fields and the masks are not set, the ID comparison essentially compares zeros. The comparison allows all transactions to bypass (if the region-based security setting is also configured in a way to allow transactions to go through for the region). To have the ID-based region protection to function, the mask registers and ID registers must also be set.

System IDs

The *System controller IDs* table provides the IDs for the system controllers. An x means that the bit can be a 0 or a 1. There are multiple IDs associated with that system controller.

Table 10-3: System Controller IDs

ASIB Name	IID	SIID	ID
SP0A	0	0	11'b00000000x000
SP0B	0	1	11'b00000000x001
SP1A	0	2	11'b00000000x010
SP1B	0	3	11'b00000000x011

Table 10-3: System Controller IDs (Continued)

ASIB Name	IID	SIID	ID
SP2A	0	4	11'b0000000x100
SP2B	0	5	11'b0000000x101
SP3A	0	6	11'b0000000x110
SP3B	0	7	11'b0000000x111
CRC0_CH0	1	6	11'b0010000x110
CRC0_CH1	1	7	11'b0010000x111
MLB	2	4	11'b00100000100
UART0_TX	3	0	11'b0011000x000
UART0_RX	3	4	11'b0011000x100
SPI0TX	4	0	11'b0100000x000
SPI0RX	4	1	11'b0100000x001
SPI1TX	4	2	11'b0100000x010
SPI1RX	4	3	11'b0100000x011
SPI2TX	4	5	11'b0100000x101
SPI2RX	4	4	11'b0100000x100
UART1_TX	3	1	11'b0011000x001
UART1_RX	3	2	11'b0011000x010
CRYPTO	6	2	11'b01100000010
ACC_CH0	6	0	11'b0110xxxx000
ACC_CH1	6	1	11'b0110xxxx001
EMDMA0_CH0	7	0	11'b01110000000
EMDMA0_CH1	7	1	11'b01110000001
EMDMA1_CH0	8	1	11'b10000000001
EMDMA1_CH1	8	0	11'b10000000000
MSMDMA_CH0	9	0	11'b1001000x000
MSMDMA_CH1	9	1	11'b1001000x001
DBG	9	2	11'b10010000010
CRC1_CH0	10	4	11'b1010000x100
CRC1_CH1	10	5	11'b1010000x101
SH0_DPORT	11	0	11'b1011xxxx000
SH0_IPORT	11	1	11'b10110000001

Table 10-3: System Controller IDs (Continued)

ASIB Name	IID	SIID	ID
HSMDMA_CH0	12	0	11'b1100000x000
HSMDMA_CH1	12	1	11'b1100000x001
SP4A	13	0	11'b1101000x000
SP4B	13	1	11'b1101000x001
SP5A	13	2	11'b1101000x010
SP5B	13	3	11'b1101000x011
SP6A	13	4	11'b1101000x100
SP6B	13	5	11'b1101000x101
SP7A	13	6	11'b1101000x110
SP7B	13	7	11'b1101000x111
SH0_MMR	14	0	11'b1110xxxx000

Memory Region

Memory regions can start at address 0x00000000 or at any address that is a multiple of its size. The *Supported Memory Region Size and Alignment* table shows the memory region sizes that the processor supports and the alignment of the memory region. (X values are do-not-care).

SMPU supports a maximum of eight regions.

Table 10-4: Supported Memory Region Size and Alignment

Size	SMPU_RCTLn.SIZE	Address	Possible Values for N
4KB	0b00000	0xXXXXX000	-
8KB	0b00001	0xXXXXN000	0x0, 0x2, 0x4, 0x8, 0xA, 0xC, 0xE
16KB	0b00010	0xXXXXN000	0x0, 0x4, 0x8, 0xC
32KB	0b00011	0xXXXXN000	0x0, 0x8
64KB	0b00100	0xXXXX0000	-
128KB	0b00101	0xXXXN0000	0x0, 0x2, 0x4, 0x8, 0xA, 0xC, 0xE
256KB	0b00110	0xXXXN0000	0x0, 0x4, 0x8, 0xC
512KB	0b00111	0xXXXN0000	0x0, 0x8
1MB	0b01000	0xXXX00000	-
2MB	0b01001	0xXXN00000	0x0, 0x2, 0x4, 0x8, 0xA, 0xC, 0xE
4MB	0b01010	0xXXN00000	0x0, 0x4, 0x8, 0xC
8MB	0b01011	0xXXN00000	0x0, 0x8

Table 10-4: Supported Memory Region Size and Alignment (Continued)

Size	SMPU_RCTLn.SIZE	Address	Possible Values for N
16MB	0b01100	0xXX000000	-
32MB	0b01101	0xXN000000	0x0, 0x2, 0x4, 0x8, 0xA, 0xC, 0xE
64MB	0b01110	0xXN000000	0x0, 0x4, 0x8, 0xC
128MB	0b01111	0xXN000000	0x0, 0x8
256MB	0b10000	0xX0000000	-
512MB	0b10001	0xN0000000	0x0, 0x2, 0x4, 0x8, 0xA, 0xC, 0xE
1GB	0b10010	0xN0000000	0x0, 0x4, 0x8, 0xC
2GB	0b10011	0xN0000000	0x0, 0x8
4GB	0b10100	0x00000000	-

For the case where the region size is selected as 4 GB, the region address must be at address 0x00000000.

NOTE: If a memory region address is not aligned to its size, the memory region start address protected by the SMPU is the configured address with the corresponding least significant bits masked. For example, if the size is configured for 16 KB (SMPU_RCTL[n].SIZE = 0b00010), and the base address is configured for SMPU_RADDR[n].BADDR = 0x00005018, the actual base address used by the SMPU is 0x00004000. When SMPU_RADDR[n].BADDR is read back, the program reads 0x00005000. This functionality is because only bits [11:0] are reserved as 0's. Programs must use care when setting the base address as it is not always the true base address.

SMPU Definitions

To make the best use of the SMPU, it is useful to understand the terms in this section.

Global Protection

Guarding of the entire memory space for the SMPU instantiation.

Region-Based Protection

Guarding individual segments of memory inside the memory space for the SMPU instantiation.

ID Match

A successful comparison of the ID associated with the incoming transaction and the ID and MASK configured in the SMPU.

SMPU Block Diagram

The *SMPU Top-Level Block Diagram* shows the SMPU block.

As seen in the diagram, the SMPU sits between the memory port (SCB requester port) and the SCB fabric (SCB completer port). It acts as a gateway analyzing the transaction requests. It either rejects the transaction request or allows access based on the user-programmed configuration of the SMPU.

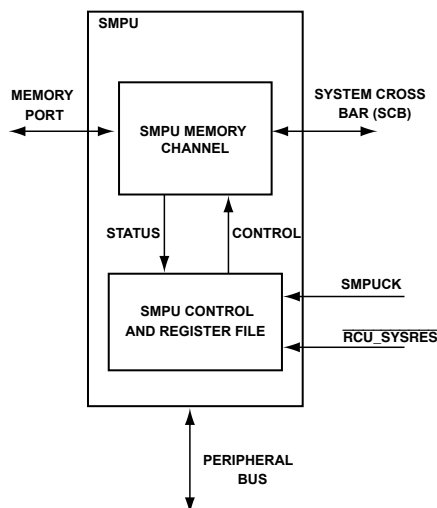


Figure 10-1: SMPU Top-Level Block Diagram

SMPU Architectural Concepts

The following sections provide brief descriptions of the architecture of the SMPU module.

Default Setting

At reset, the default state of the system is fully open. The SMPUs admit any access to memory spaces.

Latency

The SMPU adds latency to all the transactions to the memory except reads when read speculation is enabled (`SMPU_CTL.RSDIS = 0`). In this case, read accesses are always forwarded to the memory and read responses are generated according to the SMPU settings. If read speculation is disabled (`SMPU_CTL.RSDIS = 1`), reads are blocked if they cause a security or protection violation. The SMPU generates the SCB read response that corresponds to a blocked transaction.

If read speculation is enabled, the SMPU adds 1 clock cycle latency to the read transaction. If read speculation is disabled, the SMPU adds 2 clock cycles latency to the read transaction.

SMPU Operating Modes

The SMPU does not have any strict modes of operation. However, it can be configured for region-based protection where a controller with a particular ID can be blocked or allowed based on settings in the `SMPU_RCTL[n]` register.

Region-based protection is programmed with registers:

- [SMPU_RCTL\[n\]](#)
- [SMPU_RADDR\[n\]](#)
- [SMPU_RIDA\[n\]](#)
- [SMPU_RIDMSKA\[n\]](#)
- [SMPU_RIDB\[n\]](#)
- [SMPU_RIDMSKB\[n\]](#)

SMPU Interrupt Signals

There is one interrupt signal associated with the SMPU. If interrupts are enabled, the `SMPU_STAT.IRQ` bit is set. The `SMPU_IRQ` signal is asserted when the SMPU detects a memory access violation. The target address triggering the interrupt is found in the [SMPU_IADDR](#) register. The [SMPU_IDTLS](#) register provides further details about the cause of the interrupt.

Write errors are prioritized over read errors.

Protection violations (an ID-based violation) can trigger the SMPU interrupt and can be enabled independently. The protection violation interrupt is enabled by setting the `SMPU_CTL.PINTEN` bit.

The SMPU interrupt is asserted for any of the following conditions:

If a second memory access violation occurs while the `SMPU_STAT.IRQ` bit is set, the `SMPU_STAT.IOVR` (interrupt overrun) bit is set. The [SMPU_IADDR](#) and the [SMPU_IDTLS](#) registers are not updated until the `SMPU_STAT.IRQ` bit is cleared. Any information on the subsequent interrupt is lost. Once the `SMPU_STAT.IRQ` bit and the `SMPU_STAT.IOVR` bit are cleared, any new memory access violations can trigger an interrupt and its details can be captured.

NOTE: When a blocked access occurs, the SMPU triggers an interrupt when interrupt generation is enabled. The SMPU can also be configured to generate a bus error that propagates back to the system controller. The system controller can also trigger an interrupt due to this bus error.

NOTE: On the processor, each SMPU instance has an interrupt. All of the SMPU interrupts are OR'ed and mapped to a single SMPU interrupt on the SEC. While servicing the SMPU interrupt, check all of the [SMPU_STAT](#) registers to determine which triggered the interrupt. The interrupt service routine clears the `SMPU_STAT.IRQ` bit of all of the [SMPU_STAT](#) registers for which the interrupt is triggered.

SMPU Status and Error Signals

If bus errors are enabled (`SMPU_CTL.PBEDIS = 0`), the SMPU generates and returns a bus error to the controller initiating the blocked access. This bit also sets the `SMPU_STAT.BERR` bit. The [SMPU_BADDR](#) and [SMPU_BDTLS](#) registers can be read to get the address and details of the transaction that caused the SMPU to generate the error.

Write errors are prioritized over read errors.

A bus error status is returned to the system controller if:

- an ID-based violation happened and the `SMPU_CTL.PBEDIS` bit =0

If a second memory access violation occurs while the `SMPU_STAT.BERR` bit is set, the `SMPU_STAT.BEOVR` bit (bus error overrun) is set. The `SMPU_BADDR` and the `SMPU_BDTLS` registers are not updated until the `SMPU_STAT.IRQ` bit is cleared. The information about the transaction that caused the `SMPU_STAT.BEOVR` bit to be set is lost.

NOTE: If both the protection violation interrupt is not enabled (`SMPU_CTL.PINTEN` =0) and the protection bus error is disabled (`SMPU_CTL.PBEDIS` =1), the SMPU blocks invalid transactions. However, it does not provide any status or interrupt information indicating that a transaction is blocked.

ADSP-21560_21561_21564_21568 SMPU Register Descriptions

The System Memory Protection Unit (SMPU) contains the following registers.

Table 10-5: ADSP-21560_21561_21564_21568 SMPU Register List

Name	Description
<code>SMPU_BADDR</code>	Bus Error Address Register
<code>SMPU_BDTLS</code>	Bus Error Details Register
<code>SMPU_CTL</code>	SMPU Control Register
<code>SMPU_IADDR</code>	Interrupt Address Register
<code>SMPU_IDTLS</code>	Interrupt Details Register
<code>SMPU_RADDR[n]</code>	Region n Address Register
<code>SMPU_RCTL[n]</code>	Region n Control Register
<code>SMPU_REVID</code>	SMPU Revision ID Register
<code>SMPU_RIDA[n]</code>	Region n ID A Register
<code>SMPU_RIDB[n]</code>	Region n ID B Register
<code>SMPU_RIDMSKA[n]</code>	Region n ID Mask A Register
<code>SMPU_RIDMSKB[n]</code>	Region n ID Mask B Register
<code>SMPU_SECURECTL</code>	SMPU Control Secure Accesses Register
<code>SMPU_SECURERCTL[n]</code>	Region n Control Secure Accesses Register
<code>SMPU_STAT</code>	SMPU Status Register

Bus Error Address Register

Programs read the `SMPU_BADDR` and the `SMPU_BDTLS` registers to determine the cause of a bus error. Write errors are prioritized over read errors.

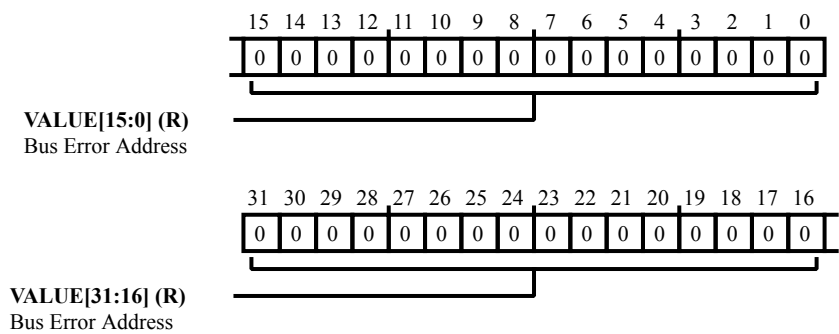


Figure 10-2: `SMPU_BADDR` Register Diagram

Table 10-6: `SMPU_BADDR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	Bus Error Address. The <code>SMPU_BADDR.VALUE</code> bit field contains the address of the bus error.

Bus Error Details Register

The `SMPU_BDTLS` register indicates the ID of the bus error transaction, whether the transaction that caused the last bus error was a read, a write, secure or non-secure.

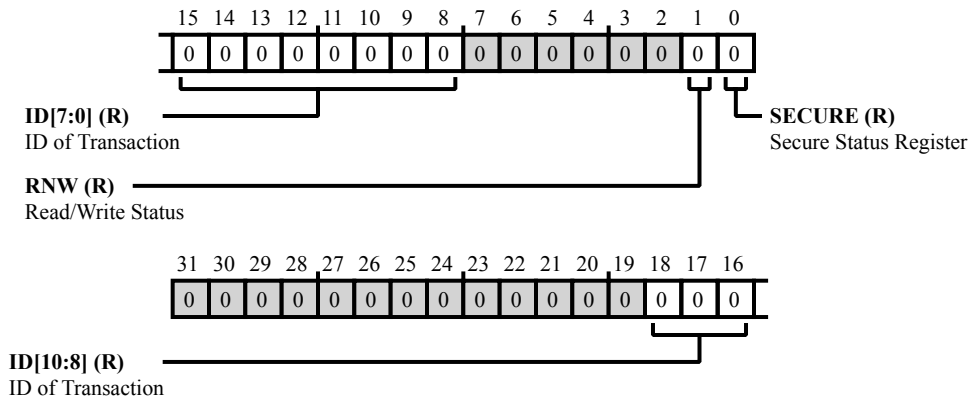


Figure 10-3: SMPU_BDTLS Register Diagram

Table 10-7: SMPU_BDTLS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
18:8 (R/NW)	ID	ID of Transaction. The <code>SMPU_BDTLS</code> .ID bit field provides the ID of the transaction that caused the bad address error.
1 (R/NW)	RNW	Read/Write Status. The <code>SMPU_BDTLS</code> .RNW bit indicates whether the last transaction that caused the bad address error was a read or write.
		0 Transaction that caused last bus error was a write
		1 Transaction that caused last bus error was a read
0 (R/NW)	SECURE	Secure Status Register. The <code>SMPU_BDTLS</code> .SECURE bit indicates whether the last transaction that caused the bad address error was secure or non-secure.
		0 Transaction that caused last bus error was non-secure
		1 Transaction that caused last bus error was secure

SMPU Control Register

The `SMPU_CTL` register provides access to the locking control, error interrupts and SMPU violations.

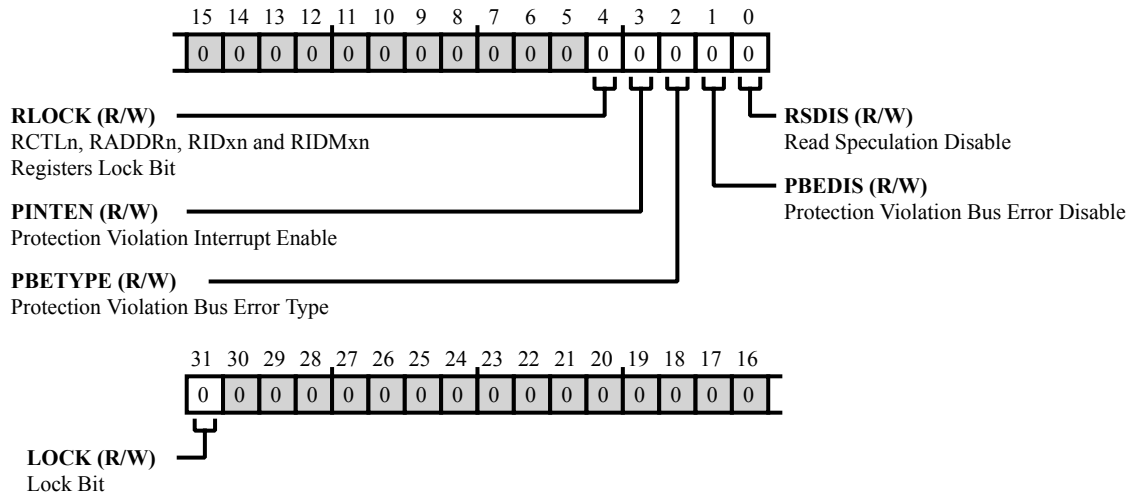


Figure 10-4: `SMPU_CTL` Register Diagram

Table 10-8: `SMPU_CTL` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock Bit. When the <code>SMPU_CTL</code> . <code>LOCK</code> bit is set and the global lock signal is asserted from the SPU, the <code>SMPU_CTL</code> register is write-protected. Write-protection is disabled only when the global lock signal becomes deasserted again.
		0 CTL Global Lock Disable. The <code>SMPU_CTL</code> register is not write-protected.
		1 CTL Global Lock Enable. The <code>SMPU_CTL</code> register is write-protected.
4 (R/W)	RLOCK	<code>RCTLn</code> , <code>RADDRn</code> , <code>RIDxn</code> and <code>RIDMxn</code> Registers Lock Bit. When the <code>SMPU_CTL</code> . <code>RLOCK</code> bit is set, all the registers associated with region-based control (<code>SMPU_RCTL[n]</code> , <code>SMPU_RADDR[n]</code> , <code>SMPU_RIDA[n]</code> , <code>SMPU_RIDB[n]</code> , <code>SMPU_RIDMSKA[n]</code> and <code>SMPU_RIDMSKB[n]</code>) are write-protected when the global lock signal is active from the SPU. Write access is allowed again when the global lock signal is deasserted.
		0 Region Registers Write-Protect Enable. All region registers are not write-protected.
		1 Region Registers Write-Protect Disable. All region registers are write-protected.

Table 10-8: SMPU_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
3 (R/W)	PINTEN	Protection Violation Interrupt Enable. The SMPU_CTL.PINTEN bit controls whether or not an interrupt is generated when a protection violation occurs.
		0 Protection Violation IRQ Disable. The protection violation interrupt is disabled.
		1 Protection Violation IRQ Enable. The protection violation interrupt is enabled.
2 (R/W)	PBETYPE	Protection Violation Bus Error Type. The SMPU_CTL.PBETYPE bit controls whether a protection violation produces a decode error or a completer error.
		0 Decode Error Type. Decode error for transactions that violate the configured protection.
		1 Completer Error Type. Completer error for transactions which violate the configured protection.
1 (R/W)	PBEDIS	Protection Violation Bus Error Disable. If set, the SMPU_CTL.PBEDIS bit blocks protection violations, but does not cause a bus error.
		0 Bus Error Generation Enable. Transactions which violate the configured protection are blocked and cause a bus error.
		1 Bus Error Generation Disable. Transactions which violate the configured protection are blocked but do not cause a bus error.
0 (R/W)	RSDIS	Read Speculation Disable. The SMPU_CTL.RSDIS bit controls whether or not the read addresses are checked before being sent to the completer.
		0 Read Speculation Enable. Read addresses are sent to the completer without checking.
		1 Read Speculation Disable. Read addresses are checked before being sent to the completer.

Interrupt Address Register

The `SMPU_IADDR` register indicates an attempt to make a read or write access to unimplemented addresses or accesses are non-aligned. The SMPU issues a bus error for this condition.

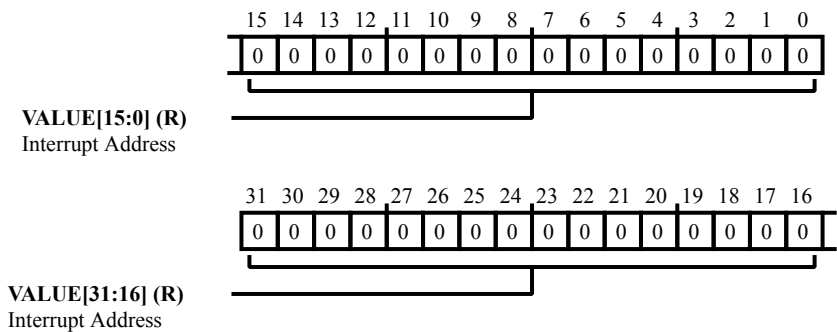


Figure 10-5: SMPU_IADDR Register Diagram

Table 10-9: SMPU_IADDR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	Interrupt Address. The <code>SMPU_IADDR.VALUE</code> bit field is the address where an attempt to access an unimplemented address or a non-aligned access has occurred.

Interrupt Details Register

The `SMPU_IDTLS` register provides the ID of the last signaled interrupt, whether the interrupt was caused by a read or write, and whether the transaction that caused the last signaled interrupt was secure.

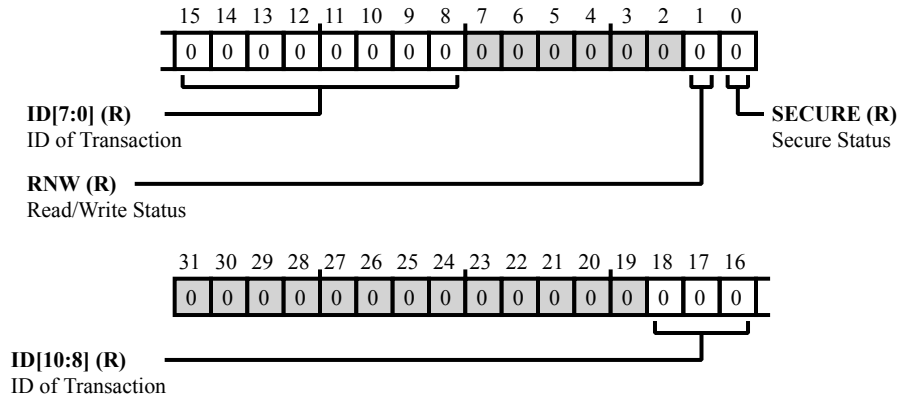


Figure 10-6: SMPU_IDTLS Register Diagram

Table 10-10: SMPU_IDTLS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
18:8 (R/NW)	ID	ID of Transaction. The <code>SMPU_IDTLS.ID</code> bit field provides the ID of the transaction that caused the interrupt.
1 (R/NW)	RNW	Read/Write Status. The <code>SMPU_IDTLS.RNW</code> bit indicates whether the last transaction that caused the interrupt was a read or write.
		0 Transaction that caused last signaled interrupt was a write
		1 Transaction that caused last signaled interrupt was a read
0 (R/NW)	SECURE	Secure Status. The <code>SMPU_IDTLS.SECURE</code> bit indicates whether the last transaction that caused the interrupt was secure or non-secure.
		0 Transaction that caused last signaled interrupt was non-secure
		1 Transaction that caused last signaled interrupt was secure

Region n Address Register

The `SMPU_RADDR[n]` register is used to define the base address for a memory region to be protected.

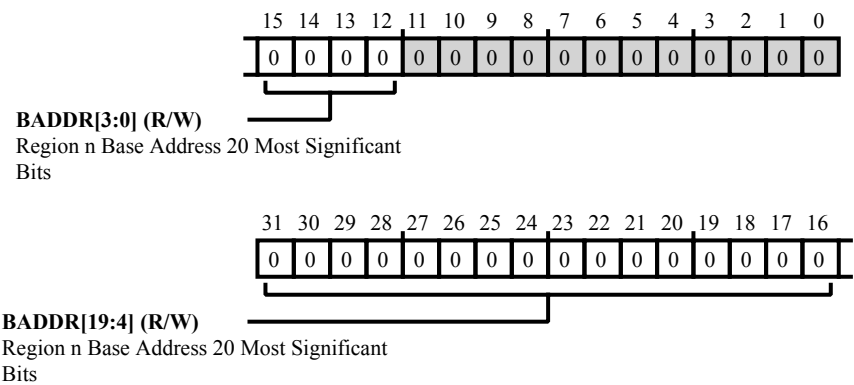


Figure 10-7: SMPU_RADDR[n] Register Diagram

Table 10-11: SMPU_RADDR[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:12 (R/W)	BADDR	Region n Base Address 20 Most Significant Bits. The <code>SMPU_RADDR[n].BADDR</code> bit field defines the base address for a memory region to be protected.

Region n Control Register

The `SMPU_RCTL[n]` register is used to define the level of protection for a region of memory. The protection of a region is controlled and defined by this register and the `SMPU_RADDR[n]`, `SMPU_RIDA[n]`, `SMPU_RIDB[n]`, `SMPU_RIDMSKA[n]`, and `SMPU_RIDMSKB[n]` registers.

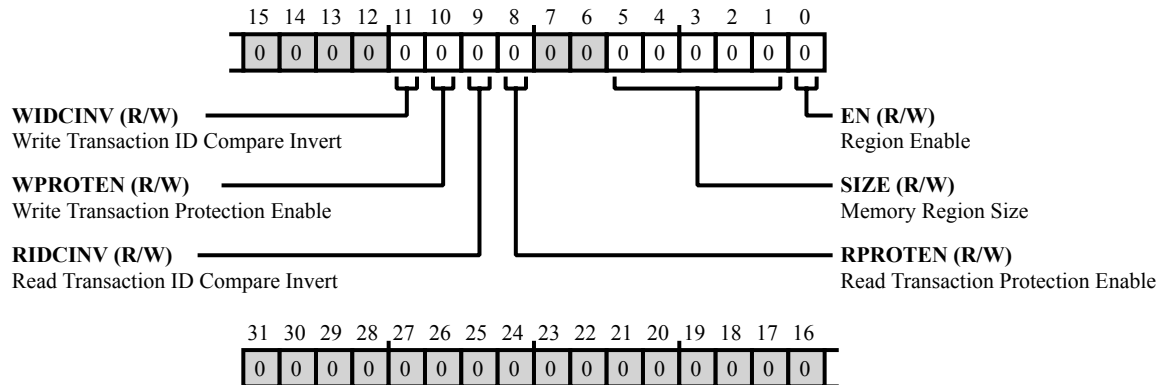


Figure 10-8: SMPU_RCTL[n] Register Diagram

Table 10-12: SMPU_RCTL[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
11 (R/W)	WIDCINV	Write Transaction ID Compare Invert. The <code>SMPU_RCTL[n].WIDCINV</code> bit inverts the write ID match result.
		0 Write transaction ID comparison result not inverted
		1 Write transaction ID comparison result inverted
10 (R/W)	WPROTEN	Write Transaction Protection Enable. The <code>SMPU_RCTL[n].WPROTEN</code> bit enables protection against ID-based write transactions for the memory region.
		0 Write transaction ID-based protection disabled
		1 Write transaction ID-based protection enabled
9 (R/W)	RIDCINV	Read Transaction ID Compare Invert. When the <code>SMPU_RCTL[n].RIDCINV</code> bit is set, the read ID match result is inverted.
		0 Read transaction ID comparison result not inverted
		1 Read transaction ID comparison result inverted

Table 10-12: SMPU_RCTL[n] Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
8 (R/W)	RPROTEN	Read Transaction Protection Enable. The SMPU_RCTL[n] . RPROTEN bit enable bit to turn on protection against ID-based read transactions for the memory region.
		0 Read transaction ID-based protection disabled
		1 Read transaction ID-based protection enabled
5:1 (R/W)	SIZE	Memory Region Size. The SMPU_RCTL[n] . SIZE bit defines the size of the memory region to be protected.
		0 4 KB
		1 8 KB
		2 16 KB
		3 32 KB
		4 64 KB
		5 128 KB
		6 256 KB
		7 512 KB
		8 1 MB
		9 2 MB
		10 4 MB
		11 8 MB
		12 16 MB
		13 32 MB
		14 64 MB
		15 128 MB
		16 256 MB
		17 512 MB
		18 1 GB
		19 2 GB
		20 4 GB
		21-31 Reserved

Table 10-12: SMPU_RCTL[n] Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	EN	Region Enable. The SMPU_RCTL[n] . EN bit enables the protection of a region.
		0 Disabled
		1 Enabled

SMPU Revision ID Register

The `SMPU_REVID` register provides the major and minor revision numbers of this module.

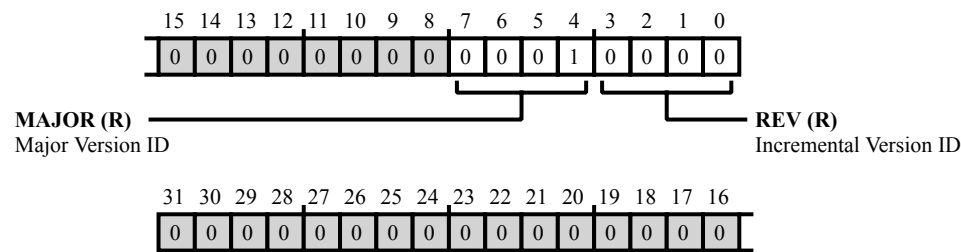


Figure 10-9: SMPU_REVID Register Diagram

Table 10-13: SMPU_REVID Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:4 (R/NW)	MAJOR	Major Version ID.
3:0 (R/NW)	REV	Incremental Version ID.

Region n ID A Register

The `SMPU_RIDA[n]` register is used for ID comparison 'A'. This comparison is performed after a mask is applied to both the transaction ID (from either the read or write IDs) and the register value. An ID match means that the ID is the exception to the rule and the read or write is allowed even if the region is read or write-protected. For more detail, refer to the ID Comparison section.

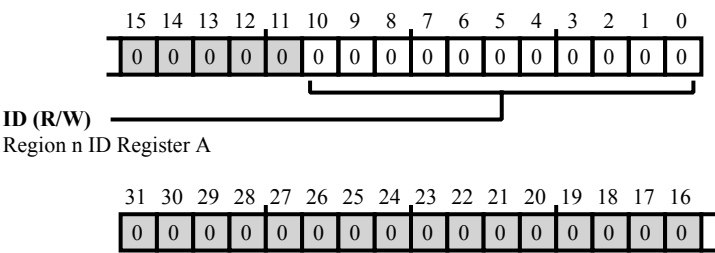


Figure 10-10: `SMPU_RIDA[n]` Register Diagram

Table 10-14: `SMPU_RIDA[n]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
10:0 (R/W)	ID	Region n ID Register A. The <code>SMPU_RIDA[n] . ID</code> bit field, combined with the mask provides the means to bypass the configured memory protection for a region.

Region n ID B Register

The `SMPU_RIDB[n]` register is used for ID comparison 'B'. This comparison is performed after a mask is applied to both the transaction ID (from either the read or write IDs) and the register value. An ID match means that the ID is the exception to the rule and the read or write is allowed even if the region is read or write-protected. For more details, refer to the ID Comparison section.

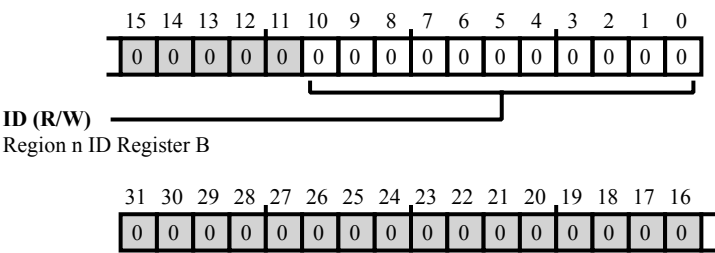


Figure 10-11: `SMPU_RIDB[n]` Register Diagram

Table 10-15: `SMPU_RIDB[n]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
10:0 (R/W)	ID	Region n ID Register B. The <code>SMPU_RIDB[n] . ID</code> bit field, combined with the mask provides the means to bypass the configured memory protection for a region.

Region n ID Mask A Register

The `SMPU_RIDMSKA[n]` register is used for ID comparison 'A'. The mask allows or disallows certain IDs from affecting the final result of the ID match. For more details, refer to the ID Comparison section.

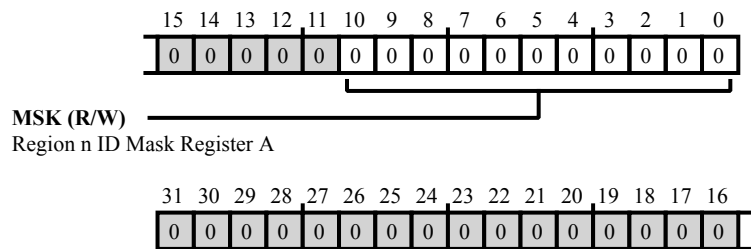


Figure 10-12: `SMPU_RIDMSKA[n]` Register Diagram

Table 10-16: `SMPU_RIDMSKA[n]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
10:0 (R/W)	MSK	Region n ID Mask Register A. The <code>SMPU_RIDMSKA[n].MSK</code> bit field, combined with the incoming transaction, provides the means to bypass the configured memory protection for a region.

Region n ID Mask B Register

The `SMPU_RIDMSKB[n]` register is used for ID comparison 'B'. The mask allows or disallows certain IDs from affecting the final result of the ID match. For more details, refer to the ID Comparison section.

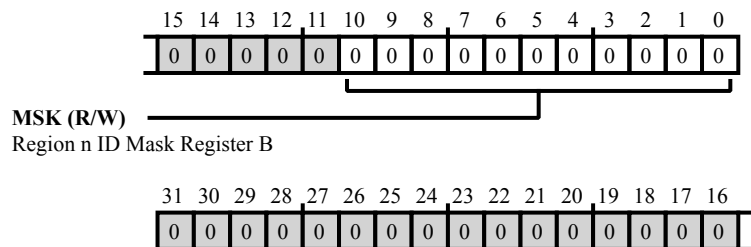


Figure 10-13: `SMPU_RIDMSKB[n]` Register Diagram

Table 10-17: `SMPU_RIDMSKB[n]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
10:0 (R/W)	MSK	Region n ID Mask Register B. The <code>SMPU_RIDMSKB[n].MSK</code> bit field, combined with the incoming transaction provides the means to bypass the configured memory protection for a region.

SMPU Control Secure Accesses Register

The `SMPU_SECURECTL` register provides the bits required to set up the security settings for the processor. These settings includes error generation and read/write security.

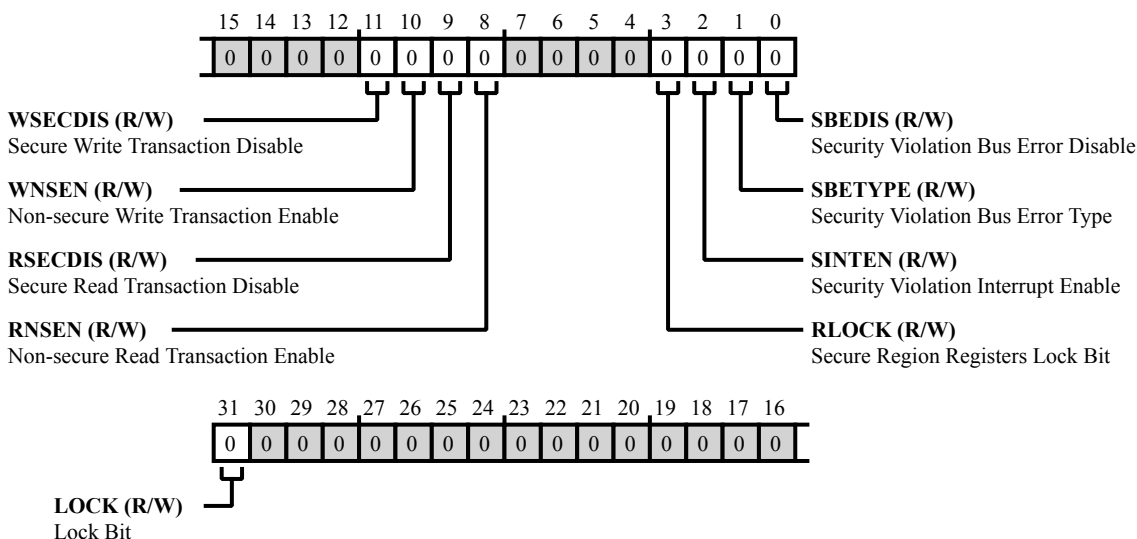


Figure 10-14: SMPU_SECURECTL Register Diagram

Table 10-18: SMPU_SECURECTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock Bit. When the <code>SMPU_SECURECTL.LOCK</code> bit is set and the global lock signal is asserted from the SPU, the <code>SMPU_SECURECTL</code> register is write-protected. Write-protection is disabled only when the global lock signal becomes deasserted again.
		0 <code>SMPU_SECURECTL</code> is not write-protected
		1 <code>SMPU_SECURECTL</code> is write-protected
11 (R/W)	WSECDIS	Secure Write Transaction Disable. The <code>SMPU_SECURECTL.WSECDIS</code> bit disables secure write transactions.
		0 Enable secure write transactions
		1 Disable secure write transactions
10 (R/W)	WNSEN	Non-secure Write Transaction Enable. The <code>SMPU_SECURECTL.WNSEN</code> bit enables non-secure write transactions.
		0 Disable non-secure writes
		1 Enable non-secure writes

Table 10-18: SMPU_SECURECTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
9 (R/W)	RSECDIS	Secure Read Transaction Disable. The SMPU_SECURECTL.RSECDIS bit disables secure read transactions.
		0 Enable secure read transactions
		1 Disable secure read transactions
8 (R/W)	RNSEN	Non-secure Read Transaction Enable. The SMPU_SECURECTL.RNSEN bit enables non-secure read transactions.
		0 Disable non-secure read transactions
		1 Enable non-secure read transactions
3 (R/W)	RLOCK	Secure Region Registers Lock Bit. When the SMPU_SECURECTL.RLOCK bit is set, the secure region control registers, SMPU_SECURECTL[n] , are write-protected when the global lock signal is active from the SPU. When the global lock signal is deasserted, write access is allowed again.
		0 Disable write-protection on secure region registers
		1 Enable write-protection on secure region registers
2 (R/W)	SINTEN	Security Violation Interrupt Enable. The SMPU_SECURECTL.SINTEN bit enables interrupt generation when a security violation occurs.
		0 Disable security settings violation interrupt
		1 Enable security settings violation interrupt
1 (R/W)	SBETYPE	Security Violation Bus Error Type. The SMPU_SECURECTL.SBETYPE bit controls whether a decode error or a completer error is returned when a security violation occurs.
		0 Decode error
		1 Completer error
0 (R/W)	SBEDIS	Security Violation Bus Error Disable. The SMPU_SECURECTL.SBEDIS bit controls whether or not a bus error is caused when a security violation occurs.
		0 Enable bus error
		1 Disable bus error

Region n Control Secure Accesses Register

The `SMPU_SECURERCTL[n]` register contains bits that configure read/write security for a specific region.

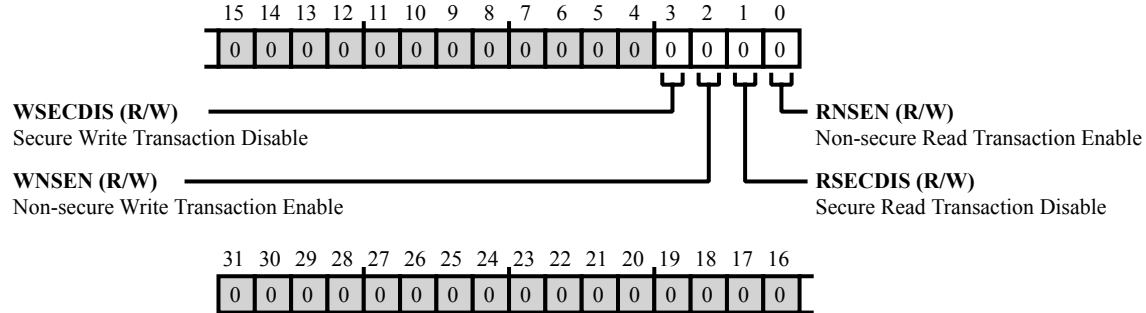


Figure 10-15: `SMPU_SECURERCTL[n]` Register Diagram

Table 10-19: `SMPU_SECURERCTL[n]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3 (R/W)	WSECDIS	Secure Write Transaction Disable. The <code>SMPU_SECURERCTL[n].WSECDIS</code> bit disables secure write transactions for the memory region.
		0 Enable secure write transactions to this region
		1 Disable secure write transactions to this region
2 (R/W)	WNSSEN	Non-secure Write Transaction Enable. This <code>SMPU_SECURERCTL[n].WNSSEN</code> bit enables non-secure write transactions for the memory region.
		0 Disable non-secure write transactions to this region
		1 Enable non-secure write transactions to this region
1 (R/W)	RSECDIS	Secure Read Transaction Disable. The <code>SMPU_SECURERCTL[n].RSECDIS</code> bit disables secure read transactions for the memory region.
		0 Enable secure read transactions to this region
		1 Disable secure read transactions to this region
0 (R/W)	RNSSEN	Non-secure Read Transaction Enable. The <code>SMPU_SECURERCTL[n].RNSSEN</code> bit enables non-secure read transactions for the memory region.
		0 Disable non-secure read transactions to this region
		1 Enable non-secure read transactions to this region

SMPU Status Register

The `SMPU_STAT` register provides the state of the SMPU and indicates various errors. All bits in this register are write 1 to clear.

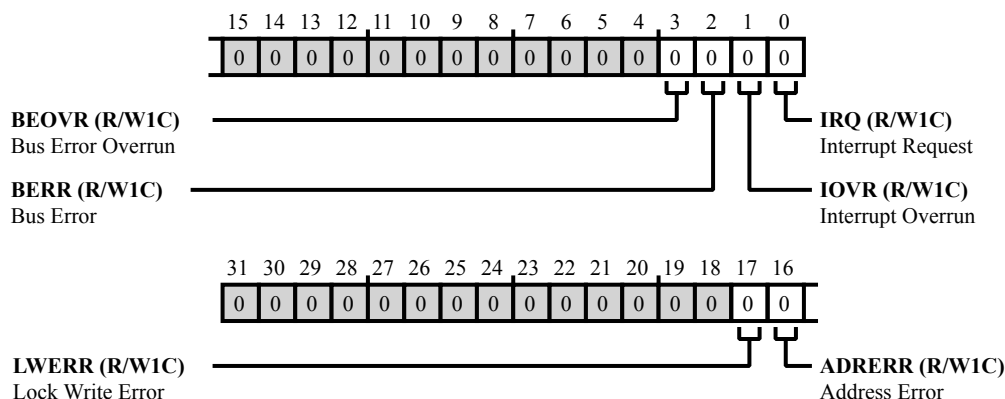


Figure 10-16: `SMPU_STAT` Register Diagram

Table 10-20: `SMPU_STAT` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
17 (R/W1C)	LWERR	Lock Write Error. The <code>SMPU_STAT.LWERR</code> bit is set when <code>SMPU_CTL.LOCK</code> bit =1, the global lock signal is asserted from the SPU and a read or write attempt was made to the <code>SMPU_CTL</code> MMR.
		0 No Lock Write Error
		1 Lock Write Error
16 (R/W1C)	ADRERR	Address Error. The <code>SMPU_STAT.ADRERR</code> bit is set when the SMPU MMR is accessed as an un-aligned address, or when a read-only MMR is written to.
		0 No Address Error
		1 Address Error
3 (R/W1C)	BEOVR	Bus Error Overrun. The <code>SMPU_STAT.BEOVR</code> bit indicates that another bus error had occurred. Any new information about the most recent violation which caused the bus error is not captured.
		0 No Bus Error overrun
		1 Bus Error overrun has occurred

Table 10-20: SMPU_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
2 (R/W1C)	BERR	Bus Error. This SMPU_STAT . BERR bit indicates if a bus error was generated.
		0 No Bus Error since this bit has been cleared
		1 Bus Error has been generated
1 (R/W1C)	IOVR	Interrupt Overrun. The SMPU_STAT . IOVR bit indicates if another violation occurred while the previous violation interrupt was not finished being serviced. Information about the most recent violation is then not captured.
		0 No Interrupt overrun
		1 Interrupt overrun has occurred
0 (R/W1C)	IRQ	Interrupt Request. The SMPU_STAT . IRQ bit provides an indication that an interrupt has been generated.
		0 No Interrupt since this bit has been cleared
		1 Interrupt has been generated

11 General-Purpose Ports (PORT)

This section describes general-purpose ports, pin multiplexing, general-purpose input/output (GPIO) functionality, and pin interrupts. The general-purpose ports provide the following three functions:

- Pin multiplexing scheme
- GPIO functionality
- Pin interrupt requests

NOTE: In this chapter, the naming convention for registers and bits omits the alphabetic group enumeration to refer to any and all of the ports. For example, `PORT_FER` represents registers `PORTA_FER`, `PORTB_FER`, and so on. Likewise `PORT_FER.PX1` represents bits `PA1`, `PB1`, and so on.

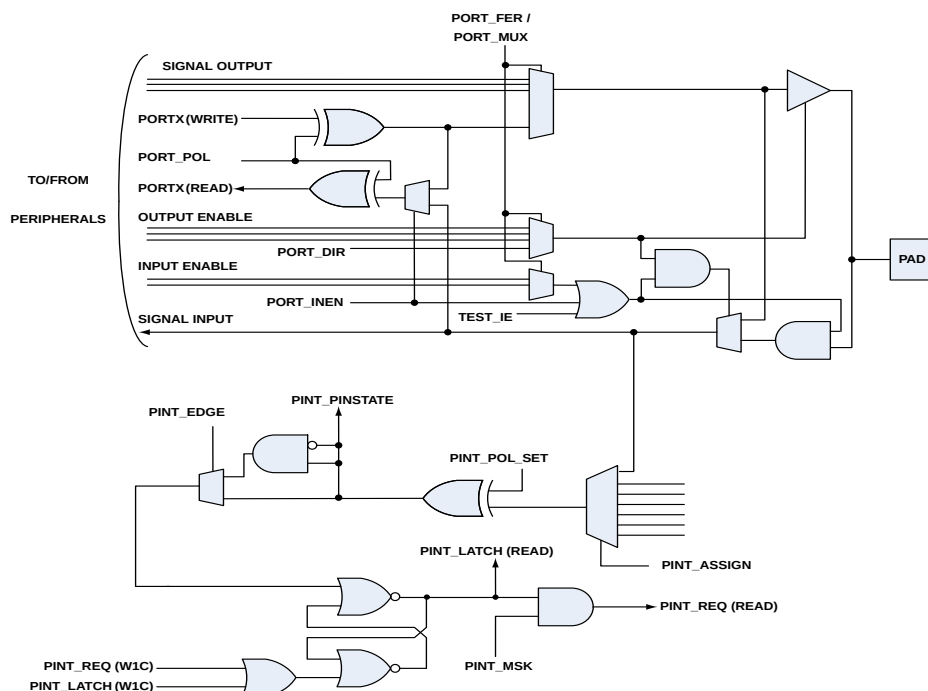


Figure 11-1: Simplified GPIO and Pin Interrupt Signal Flow

PORT Features

The PORTs include the following features:

- Input mode, output mode, and open-drain mode of GPIO operation
- Port multiplexing controlled on a pin-by-pin basis
- No external glue hardware required for unused pins
- All port pins provide interrupt request functionality
- Byte-wide pin-to-interrupt request assignment

PORT Functional Description

The number of ports and each's composition are defined in the processor data sheet. Each port has a dedicated set of MMR registers that control pin functions and operates in general-purpose I/O (GPIO) mode by default, as controlled by the port-specific `PORT_FER` register. Each bit in this register, as well as the other PORT MMRs, represents a specific GPIO pin on the specified port.

Input Mode, Output Mode, and Open-Drain Mode of GPIO Operation

At reset, every GPIO pin defaults to input mode with the input drivers disabled. To enable any GPIO input driver, set the bits corresponding to the individual pins in the appropriate input enable register (`PORT_INEN`).

The GPIO output drivers are enabled by setting the corresponding bits in the direction registers (`PORT_DIR`).

The PORT can use every GPIO in open-drain mode by clearing the respective bit in the `PORT_DATA` register or setting the respective bit in the `PORT_DATA_CLR` register. Then, set the corresponding bit in the `PORT_INEN` register. Read from the `PORT_DATA` register to obtain the status from the pin.

Port Multiplexing Controlled on Pin-by-Pin Basis

Each port has two dedicated MMRs that control the port multiplexing, the 16-bit function enable (`PORT_FER`) registers and the 32-bit port multiplexing (`PORT_MUX`) registers.

All Port Pins Provide Interrupt Functionality

Pin interrupts are completely decoupled from GPIO functionality. Pins are connected to the system event controller (SEC) via the PINTx modules, each of which is configurable in terms of which port pins are sensed for interrupt generation.

ADSP-21560_21561_21564_21568 PORT Register List

The PORT module (PORT) regulates the use of the multiplexable processor pins. Every port pin can operate in general-purpose I/O (GPIO) mode or as an alternate function. This GPIO operation is the default after processor reset and is controlled by a set of registers that control GPIO functionality. Every bit in these registers represents a certain GPIO pin of a specific port. For more information on PORT functionality, see the PORT register descriptions.

Table 11-1: ADSP-21560_21561_21564_21568 PORT Register List

Name	Description
PORT_DATA	Port x GPIO Data Register
PORT_DATA_CLR	Port x GPIO Data Clear Register
PORT_DATA_SET	Port x GPIO Data Set Register
PORT_DATA_TGL	Port x GPIO Output Toggle Register
PORT_DIR	Port x GPIO Direction Register
PORT_DIR_CLR	Port x GPIO Direction Clear Register
PORT_DIR_SET	Port x GPIO Direction Set Register
PORT_FER	Port x Function Enable Register
PORT_FER_CLR	Port x Function Enable Clear Register
PORT_FER_SET	Port x Function Enable Set Register
PORT_INEN	Port x GPIO Input Enable Register
PORT_INEN_CLR	Port x GPIO Input Enable Clear Register
PORT_INEN_SET	Port x GPIO Input Enable Set Register
PORT_LOCK	Port x GPIO Lock Register
PORT_MUX	Port x Multiplexer Control Register
PORT_POL	Port x GPIO Polarity Invert Register
PORT_POL_CLR	Port x GPIO Polarity Invert Clear Register
PORT_POL_SET	Port x GPIO Polarity Invert Set Register
PORT_TRIG_TGL	Port x GPIO Trigger Toggle Register

ADSP-21560_21561_21564_21568 PORT Trigger List

Table 11-2: ADSP-21560_21561_21564_21568 PORT Trigger List Generators

Trigger ID	Name	Description	Sensitivity
None			

Table 11-3: ADSP-21560_21561_21564_21568 PORT Trigger List Receivers

Trigger ID	Name	Description	Sensitivity
111	PORTA_TOGGLE	Port Toggle Trigger	Pulse
112	PORTB_TOGGLE	Port Toggle Trigger	Pulse

ADSP-21560_21561_21564_21568 PINT Register List

The Pin Interrupt module (PINT) controls the pin-to-interrupt assignment in a byte-wide manner. The pin-interrupt assignment registers do not consist of 32 individual bits. They consist of four control bytes, each functioning as a multiplexer control. For more information, see the PINT register descriptions.

All PINT registers are 32 bits wide and can be accessed by 32-bit load/store instructions. They also support 16-bit operation where the upper 16 bits are ignored and the application uses the lower 16 bits only. Consequently, all PINT registers support 32-bit accesses as well as 16-bit accesses for the lower half words. Applications may use faster 16-bit accesses as long as they do not require functionality of upper register halves.

Table 11-4: ADSP-21560_21561_21564_21568 PINT Register List

Name	Description
PINT_ASSIGN	PINT Assign Register
PINT_EDGE_CLR	PINT Edge Clear Register
PINT_EDGE_SET	PINT Edge Set Register
PINT_INV_CLR	PINT Invert Clear Register
PINT_INV_SET	PINT Invert Set Register
PINT_LATCH	PINT Latch Register
PINT_MSK_CLR	PINT Mask Clear Register
PINT_MSK_SET	PINT Mask Set Register
PINT_PINSTATE	PINT Pin State Register
PINT_REQ	PINT Request Register

ADSP-21560_21561_21564_21568 PINT Interrupt List

Table 11-5: ADSP-21560_21561_21564_21568 PINT Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
26	PINT0_BLOCK	PINT0 Pin Interrupt Block 0	Level	
27	PINT1_BLOCK	PINT1 Pin Interrupt Block 1	Level	

ADSP-21560_21561_21564_21568 PINT Trigger List

Table 11-6: ADSP-21560_21561_21564_21568 PINT Trigger List Generators

Trigger ID	Name	Description	Sensitivity
13	PINT0_BLOCK	PINT0 Pin Interrupt Block	Level
14	PINT1_BLOCK	PINT1 Pin Interrupt Block	Level

Table 11-7: ADSP-21560_21561_21564_21568 PINT Trigger List Receivers

Trigger ID	Name	Description	Sensitivity
None			

ADSP-21560_21561_21564_21568 PADS Register List

The PADS controls signal hysteresis and other system interface signal features for a number of module interfaces.

Table 11-8: ADSP-21560_21561_21564_21568 PADS Register List

Name	Description
PADS_DAI0_0_DS	DAI0 1 to 10 pins DS Control Register
PADS_DAI0_1_DS	DAI0 11 to 20 pins DS Control Register
PADS_DAI0_IE	DAI0 Port Input Enable Control Register
PADS_DAI1_0_DS	DAI1 1 to 10 pins DS Control Register
PADS_DAI1_1_DS	DAI1 11 to 20 pins DS Control Register
PADS_DAI1_IE	DAI1 Port Input Enable Control Register
PADS_DAI[n]_PDE	DAIx Pull-Down Enable Register
PADS_DAI[n]_PUE	DAIx Pull-Up Enable Register
PADS_NONPORTS_DS	Non-GPIO Drive Strength Register
PADS_PCFG0	Peripheral PAD Configuration 0 Register
PADS_PORTA0_DS	PORTA 0 to 7 pins DS Control Register
PADS_PORTA1_DS	PORTA 8 - 15 pins DS Control Register
PADS_PORTB0_DS	PORTB 0 to 7 pins DS Control Register
PADS_PORT[n]_PDE	PORTx Pull-Down Enable Register
PADS_PORT[n]_PUE	PORTx Pull-Up Enable Register

PORT Architectural Concepts

These sections describe in more detail how the PORT module connects externally to pins and internally to the MMR bus. Ports are named alphabetically beginning with A.

- [Internal Interfaces](#)
- [External Interfaces](#)
- [GPIO Pin Function](#)
- [Port Multiplexing Control](#)

Internal Interfaces

All of the pin multiplexing, GPIO, and pin interrupt control block MMRs can be accessed through the MMR bus. There is no DMA support. Each of the pin interrupt (PINTx) modules has its own dedicated interrupt request output signal that connects directly to the system event controller (SEC).

External Interfaces

The pin multiplexing hardware can be seen as a layer between the on-chip peripherals and the silicon pads connecting to the physical pins/balls or the package, as controlled by the PORT unit.

GPIO Pin Function

By default, the PORT sets every GPIO pin to input mode. The input drivers are not enabled, which avoids the need for unnecessary current sinks and external termination resistors on unused pins.

Input Mode

The default mode of every GPIO pin after reset is input mode, but the input drivers are not enabled. To enable GPIO input drivers, set the bits corresponding to the PORT pins in the appropriate input enable register ([PORT_INEN](#)). When enabled, a read from the [PORT_DATA](#) register returns the logical state of the input pins. However, the input signal does not overwrite the state of the internal flip-flop used for providing output to the same pin. Only software can alter the state. If the input driver is enabled, a write to the [PORT_DATA](#) register can alter the state of the flip-flop, but the change cannot be read back.

Output Mode

Any GPIO pin can be configured for output mode. The GPIO output drivers are enabled by setting the bits corresponding to the PORT pins in the appropriate direction register. The PORT implements direction registers as a pair of write-1-to-set (W1S) and write-1-to-clear (W1C) MMRs called [PORT_DIR_SET](#) and [PORT_DIR_CLR](#), respectively. As such, software can alter the direction of the signal flow on individual GPIO pins without impacting other GPIOs on the same port.

Both the [PORT_DIR_SET](#) and [PORT_DIR_CLR](#) registers return the same value when read, and a logical 1 indicates an enabled output. The [PORT_DATA](#) registers control the state of output pins. A logical 0 drives the output low while a logical 1 drives the output high.

While writes to the [PORT_DATA](#) register can alter all of the GPIOs on a specific port at once, there are also a pair of W1S and W1C MMRs called [PORT_DATA_SET](#) and [PORT_DATA_CLR](#), respectively. These registers enable the manipulation of individual GPIO outputs. The state of the outputs can be obtained by reading the [PORT_DATA](#) registers. Because the state of the GPIO output can be controlled before the output driver is enabled, set or clear the internal flip-flop first by programming this register to avoid volatile levels on the output pin.

Trigger Toggle Mode

Any GPIO pin that has been configured for output mode can be toggled using a trigger input from the Trigger Routing Unit (TRU). To enable this functionality for a particular GPIO, set the appropriate bit in the `PORT_TRIG_TGL` register. Any subsequent trigger for the designated port causes all GPIO outputs set in the `PORT_TRIG_TGL` register to toggle.

To avoid unpredictable behavior, do not set, clear, or toggle the `PORT_DATA`, `PORT_DATA_SET`, `PORT_DATA_CLR`, or `PORT_DATA_TGL` registers when the GPIO output has the corresponding `PORT_TRIG_TGL` bit set. To change the state of the GPIO output using one of these registers, first clear the corresponding bit in the `PORT_TRIG_TGL` register.

Open-Drain Mode

Every GPIO can also be used in open-drain mode. First, either clear the respective bit in the `PORT_DATA` register or set the respective bit in the `PORT_DATA_CLR` register. Then, set the appropriate bit in the `PORT_INEN` register. Read from the `PORT_DATA` register to return the status from the pin rather than the state of the internal flip-flop.

By toggling the output driver through the `PORT_DIR_SET` and `PORT_DIR_CLR` register pair, the output signal can be pulled low or three-stated, as required. The polarity of the driven signal can be inverted when the internal flip-flop is set. When using a GPIO port in open-drain mode, take care to not exceed the V_{IH} operating condition associated with the respective pins.

Port Multiplexing Control

To configure pins properly, consult the processor data sheet to determine which bits in the `PORT_FER` and `PORT_MUX` register map to the pin of interest, and then set these registers appropriately for the desired function.

After reset, all port pins default to GPIO input mode with their output and input drivers disabled. As a result, all unused port pins can be left unconnected. Disabled pins appear as a high impedance to external circuits.

Each port has two dedicated MMRs that control the port multiplexing, the 16-bit function enable (`PORT_FER`) registers and the 32-bit port multiplexing (`PORT_MUX`) registers.

The function enable register specifies whether the pin is used as a GPIO pin or allocated for use by a specific peripheral, but it does not specify what the peripheral function is. Each bit in the 16-bit `PORT_FER` register corresponds to an individual port pin. For example, if bit 1 (PA1) of the `PORTA_FER` register is cleared, the PA_01 pin is configured as a GPIO. When set, one of the available peripheral functions becomes active on the PA_01 pin instead.

Pairs of bits in the `PORT_MUX` register control the multiplexing between the peripheral functions available to an individual pin, as some PORT pins provide up to four possible peripheral functions.

Refer to the Signal Muxing table in the data sheet for the specific `PORT_MUX` settings.

PORT Event Control

The following sections describe event generation in the PORT module.

PORT Interrupt Signals

The pin interrupts are decoupled from GPIO functionality, providing the following advantages.

- Flexible mapping scheme enables pins from up to four different ports to be grouped into one common interrupt scheme.
- Interrupt requests work on input and output pins regardless of whether the pin is functioning as a GPIO or a peripheral.

The processor has a number of interrupt channels dedicated to pin interrupts, managed by a set of pin interrupt (PINTx) blocks. Each PINTx block can sense up to 32 GPIO pins, as described in the following list and figure.

- PINT0 can sense pin activity on PORTA and PORTB
- PINT1 can sense pin activity on PORTB

The processor supports both 32-bit and 16-bit peripheral bus accesses to PINTx registers.

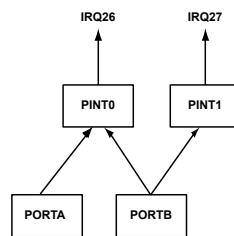


Figure 11-2: GPIO to PINTx Assignment

Pins connect to the PINTx module and then to the system event controller (SEC), as shown in the *PINTx Block Diagram*.

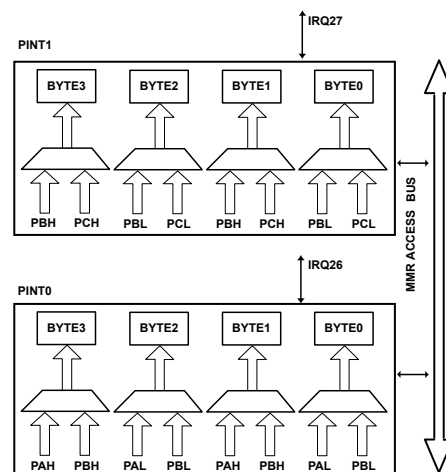


Figure 11-3: PINTx Block Diagram

As shown in the *PINTx Block Diagram*, each port is subdivided into two 8-pin half ports, upper (PxH) and lower (PxL). The `PINT_ASSIGN` registers control the 8-bit multiplexers associated with these half ports, where the lower half units (eight pins) can be forwarded to either byte 0 or byte 2 of the PINTx blocks, and the upper half units (eight pins) can be forwarded to either byte 1 or byte 3 of the PINTx blocks.

When a half port is assigned to a byte in any PINTx block, the state of the eight pins appears in the `PINT_PINSTATE` register, regardless of whether the pin is enabled for GPIO or peripheral functions (input or output). When neither the input nor output drivers of the pin are enabled, the pin state is read as zero. The `PINT_PINSTATE` register reports the inverted state of the pin when the `PINT_INV_SET` register activates the signal inverter. The inverter can be enabled on an individual bit-by-bit basis. Each bit in the `PINT_INV_SET/PINT_INV_CLR` register pair represents a pin signal.

By default, PORT interrupt request generation is level-sensitive, and an interrupt request occurs when the enabled pin is sensed as active high. Use the `PINT_EDGE_SET` register to change the interrupt request generation scheme to instead be edge-sensitive (rising edge generates the interrupt request). Use the `PINT_INV_SET` register to invert the polarity such that the PINTx block generates the interrupt request on active-low signals or falling edges.

The PINTx modules also assist when both signal edges must generate unique interrupt requests. If two different interrupt requests are required, the `PINT_ASSIGN` registers can route a single input signal to two different PINTx blocks, where one block inverts the signal in the `PINT_INV_SET` register and the other one does not. In this fashion, a unique software routine is associated with the hardware PINTx block that is generating the unique interrupt request for each signal edge. When both signal edges can be serviced by the same interrupt request, each half port can be routed to two separate bytes within a single PINTx block using the `PINT_ASSIGN` register, and then one of the half ports needs to have the inversion enabled in the `PINT_INV_SET` register. The servicing software routine can then detect from the `PINT_LATCH` register whether a falling, rising, or both edges have occurred.

Regardless of whether level-sensitive or edge-sensitive mode is used, the hardware always latches an interrupt request. Latched signals can be read from the `PINT_LATCH` registers. Only a software or hardware reset clears the latches. To clear the latch, a W1C operation must be performed to the `PINT_REQ` or `PINT_LATCH` register. When in level-sensitive mode, the interrupt request remains asserted if the pin state does not change by the time the interrupt service routine exits.

Because every PINTx block groups up to 32 pin signals, the `PINT_MSK_SET/PINT_MSK_CLR` register pair can control which of the signals can request an interrupt at the system level. Software can interrogate the `PINT_REQ` register for signaling pins. The `PINT_REQ` bits represent a logical AND between the mask and the latch. When any of these bits is set, the block output interrupt request is asserted.

PORT Programming Model

The *GPIO Programming Model Flow (Part 1)*, *GPIO Programming Model Flow (Part 2)*, and *GPIO Programming Model Flow (Part 3)* figures show the programming model for the general-purpose ports. This programming includes the GPIO input and output operation, open-drain mode, and the pin interrupt PINTx modules.

NOTE: These process flow diagrams connect where call-out letters appear. For example, call-out A in the *GPIO Programming Model Flow (Part 1)* diagram connects to call-out A in the *GPIO Programming Model Flow (Part 2)* diagram.

The following flowcharts describe the processes for setting up pins for various functions. Begin the process from the start label in the *GPIO Programming Model Flow (Part 1)* figure. The first decision (GPIO or peripheral) determines how to program the `PORT_FER` register. Set the bit(s) corresponding to the pin(s) to 1 to enable peripheral functionality or to 0 to enable GPIO mode. For more information on setting up for peripheral functions, refer to [Port Multiplexing Control](#).

If the pin is to be a GPIO pin, a subsequent series of decisions must be made that will impact how the `PORT_DATA`, `PORT_POL`, `PORT_DIR`, and `PORT_INEN` configuration registers must be programmed. Depending on the type of GPIO pin desired, some configurations do not apply and can have different meanings. The following paragraphs briefly describe the function of the different settings for each of the pin functions in the input, output, and open-drain GPIO modes. It is a best practice to use the SET or CLR versions of the PORT registers, where applicable, to effect changes on a pin-by-pin basis rather than on the full port.

For output mode, first clear the `PORT_DATA` register to set all the pins low. Then write the `PORT_DIR` register to define the direction of each pin (set the bits associated with the desired output pins to 1). In output mode, the other registers are not significant. The *GPIO Programming Model Flow (Part 1)* chart shows this flow starting at label 2.

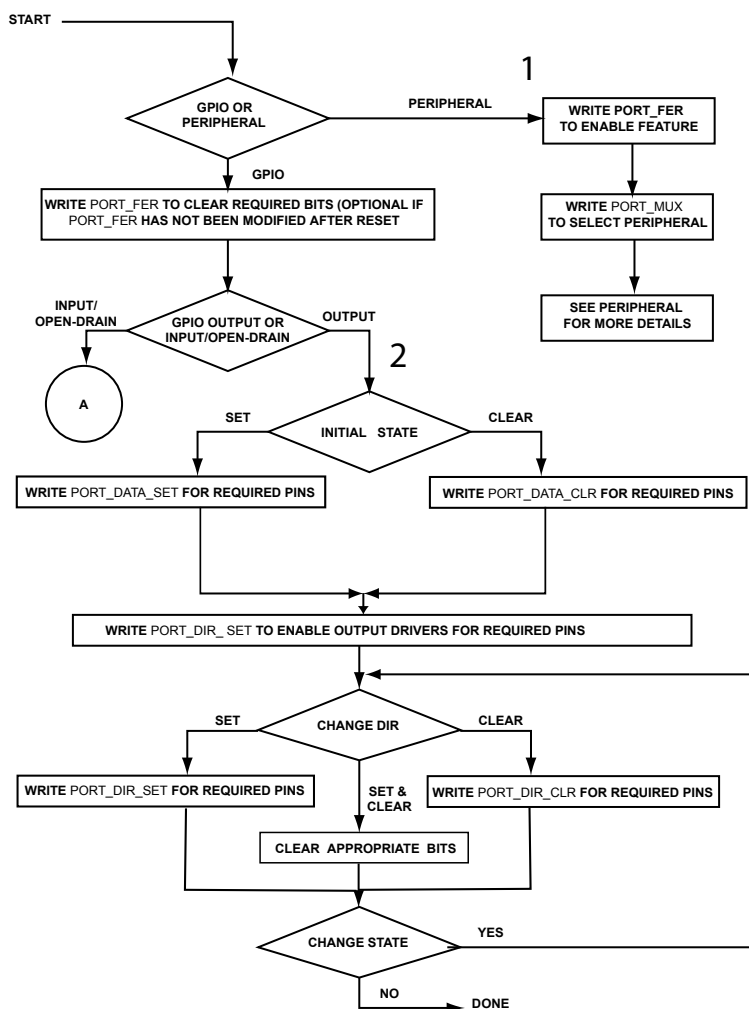


Figure 11-4: GPIO Programming Model Flow (Part 1)

For input mode, first decide the polarity for each pin using the `PORT_POL` register. Program the `PORT_DIR` register to define the appropriate pins as inputs (write a 0 to the bit location associated with the pin). If interrupt requests are desired, configure the PINT module as shown in the *GPIO Programming Model Flow (Part 3)* figure starting at label B. Finally, write the `PORT_INEN` register to enable the associated input drivers. The *GPIO Programming Model Flow (Part 2)* chart shows this entire flow starting at label 3.

For open-drain mode, set all pins low by clearing the `PORT_DATA` register. Then, use the `PORT_INEN` register to enable the appropriate input drivers. Set the `PORT_DIR` register in this mode to indicate whether the pin is in an active state or not (active being 0). The *GPIO Programming Model Flow (Part 2)* chart shows this flow starting at label 4.

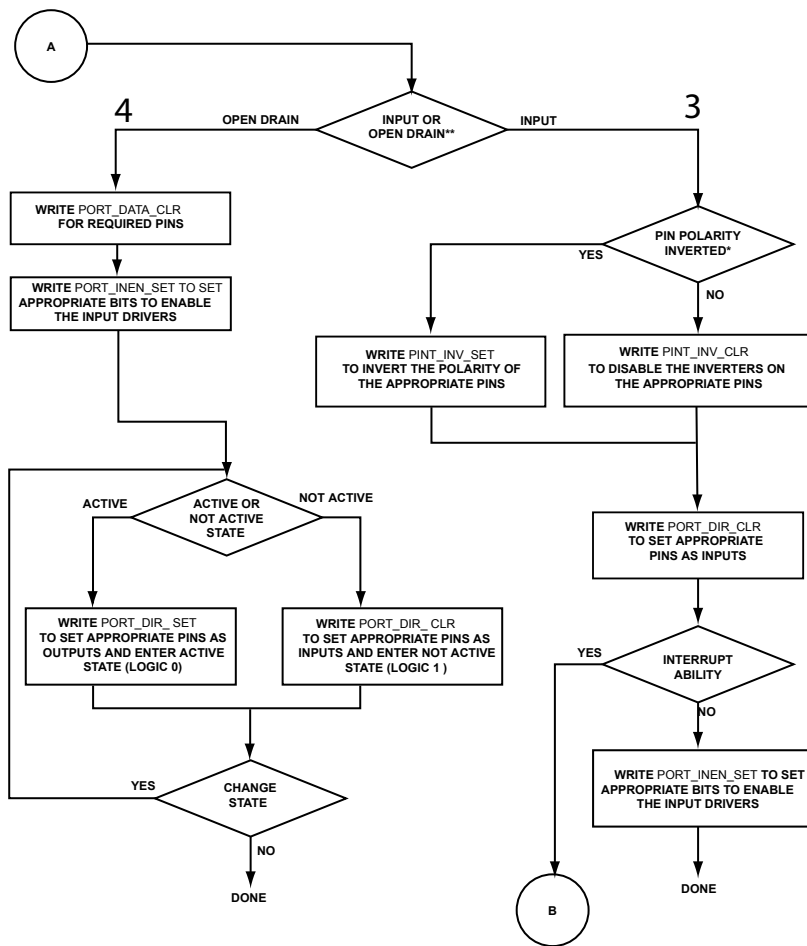


Figure 11-5: GPIO Programming Model Flow (Part 2)

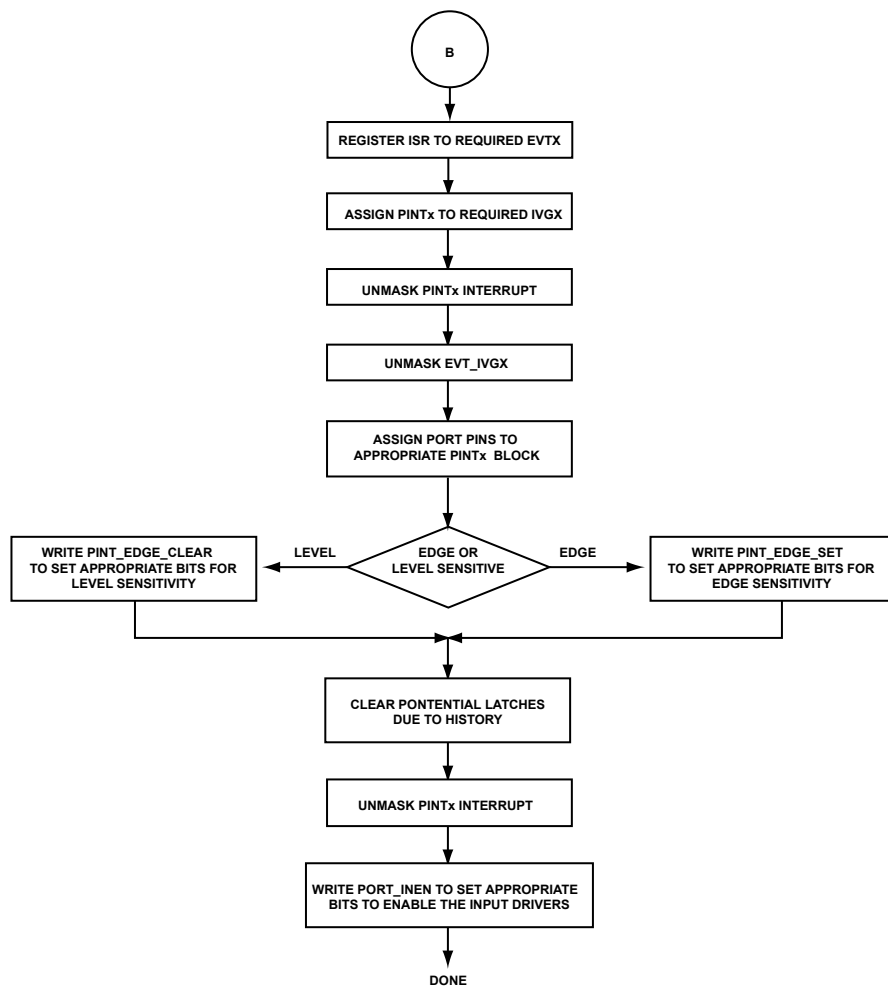


Figure 11-6: GPIO Programming Model Flow (Part 3)

Programmable Pull-up Resistors for PORT and DAI

There are programmable weak pull-up resistors for DAI and PORT pins. These resistors can be enabled or disabled by configuring the `PADS_DAI[n]_PUE`, `PADS_PORT[n]_PUE`, `PADS_DAI[n]_PDE.PUD` and `PADS_PORT[n]_PDE.PUD` registers.

For DAI pins, if both the `PADS_DAI[n]_PDE.PUD` and `PADS_DAI[n]_PUE` registers are set (=1) for the same pin, then the `PADS_DAI[n]_PDE.PUD` register takes priority over `PADS_DAI[n]_PUE` register.

NOTE: For unused PORT/DAI pins, ensure that any unused pin does not go to floating state. Clear (=0) the PUD bit of that pin.

Refer to the *DAI Pull-up Resistor States* table. To enable the pull-up on a DAI pin, the PADS_DAI[n]_PDE.PUDbit must be cleared (=0) **AND** the PADS_DAI[n]_PUE.PUE bit must be set (=1). To disable the pull-up on a DAI pin, the PADS_DAI[n]_PDE.PUD bit must be set (=1) and PADS_DAI[n]_PUE.PUE bit can be any value.

Table 11-9: DAI Pull-up Resistor States

PADS_DAI[n]_PUE.PUE	PADS_DAI[n]_PDE.PUD	Pull-up Resistor State
0	0	Disabled
0	1	Disabled
1	0	Enabled
1	1	Disabled

Similarly, for PORT pins, if both PADS_PORT[n]_PDE.PUD and PADS_PORT[n]_PUE registers are set (=1) for the same pin, then the PADS_PORT[n]_PDE.PUD register takes priority over PADS_PORT[n]_PUE register.

Refer to the *PORT Pull-up Resistor States* table. To enable the pull-up on a PORT pin, the PADS_PORT[n]_PDE.PUD bit must be cleared (=0) **AND** the PADS_PORT[n]_PUE.PUE bit must be set (=1). To disable the pull-up on a PORT pin, PADS_PORT[n]_PDE.PUD bit must be set (=1) and PADS_PORT[n]_PUE.PUE bit can be any value.

Table 11-10: PORT Pull-up Resistor States

PADS_PORT[n]_PUE.PUE	PADS_PORT[n]_PDE.PUD	Pull-up Resistor State
0	0	Disabled
0	1	Disabled
1	0	Enabled
1	1	Disabled

NOTE: Refer to the *Designer Quick Reference* section of the data sheet for handling unused PORT and DAI pins.

NOTE: The internal pull-up on GPIO pins are very weak. These pins hold a defined logic level when pins are left floating. The internal pull-up resistance varies over a wide range. If an application needs a specified pull-up on the pin, use an external resistor.

NOTE: For unused PORT/DAI pins, ensure that any unused pin does not go to floating state. Clear (=0) the PUD bit of that pin.

ADSP-21560_21561_21564_21568 PORT Register Descriptions

The General-Purpose Input/Output Port (PORT) contains the following registers.

Table 11-11: ADSP-21560_21561_21564_21568 PORT Register List

Name	Description
PORT_DATA	Port x GPIO Data Register
PORT_DATA_CLR	Port x GPIO Data Clear Register

Table 11-11: ADSP-21560_21561_21564_21568 PORT Register List (Continued)

Name	Description
PORT_DATA_SET	Port x GPIO Data Set Register
PORT_DATA_TGL	Port x GPIO Output Toggle Register
PORT_DIR	Port x GPIO Direction Register
PORT_DIR_CLR	Port x GPIO Direction Clear Register
PORT_DIR_SET	Port x GPIO Direction Set Register
PORT_FER	Port x Function Enable Register
PORT_FER_CLR	Port x Function Enable Clear Register
PORT_FER_SET	Port x Function Enable Set Register
PORT_INEN	Port x GPIO Input Enable Register
PORT_INEN_CLR	Port x GPIO Input Enable Clear Register
PORT_INEN_SET	Port x GPIO Input Enable Set Register
PORT_LOCK	Port x GPIO Lock Register
PORT_MUX	Port x Multiplexer Control Register
PORT_POL	Port x GPIO Polarity Invert Register
PORT_POL_CLR	Port x GPIO Polarity Invert Clear Register
PORT_POL_SET	Port x GPIO Polarity Invert Set Register
PORT_TRIG_TGL	Port x GPIO Trigger Toggle Register

Port x GPIO Data Register

The operation of the `PORT_DATA` register depends on whether the bit/pin is in output mode or input mode. In both modes, a set bit in the `PORT_DATA` register corresponds to a signal high on a GPIO pin. A cleared bit in the `PORT_DATA` register corresponds to a signal low on a GPIO pin.

The `PORT_DATA`, `PORT_DATA_SET`, and `PORT_DATA_CLR` registers control the state of GPIO pins in output mode. To enable output mode (and output drivers), use the `PORT_DIR_SET` and `PORT_DIR_CLR` registers.

Writes to the `PORT_DATA` register affect the state of all pins of the port that are in output mode. To set or clear specific pins without impacting other pins of the port, use the `PORT_DATA_SET` and `PORT_DATA_CLR` registers.

When the GPIO pins are in input mode (input driver is enabled with the `PORT_INEN` register), reads from the `PORT_DATA`, `PORT_DATA_SET`, and `PORT_DATA_CLR` registers return the state of the respective GPIO pins.

Note that when the input driver is not enabled, reads from the `PORT_DATA`, `PORT_DATA_SET`, and `PORT_DATA_CLR` registers return the value previously written to the registers.

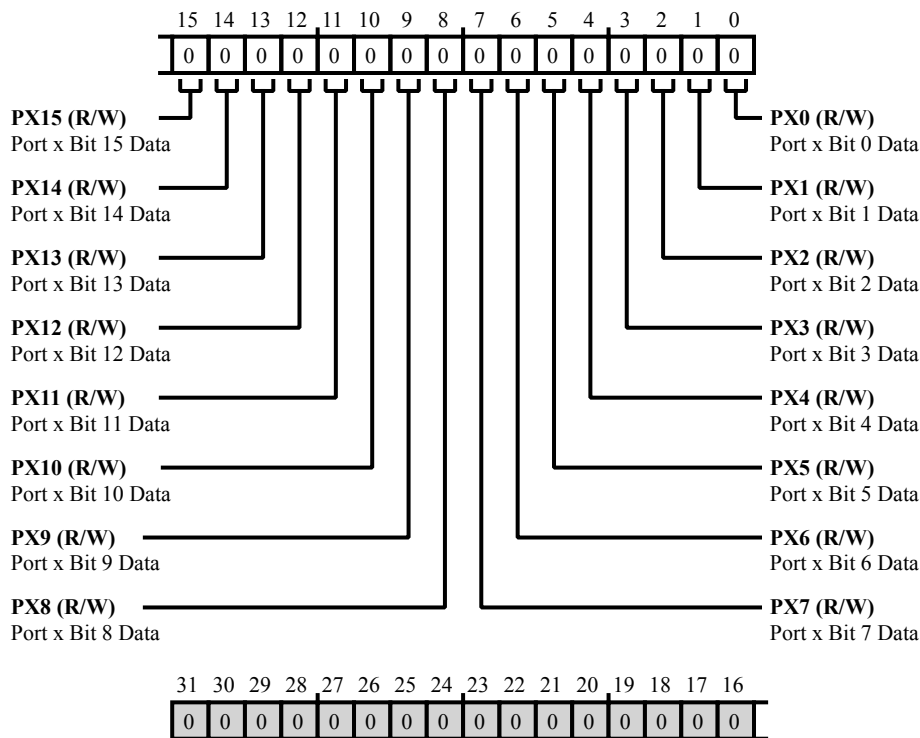


Figure 11-7: `PORT_DATA` Register Diagram

Table 11-12: PORT_DATA Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W)	PX15	Port x Bit 15 Data. The PORT_DATA.PX15 bit indicates a signal on a GPIO pin.
		0 Signal Low
		1 Signal High
14 (R/W)	PX14	Port x Bit 14 Data. The PORT_DATA.PX14 bit indicates a signal on a GPIO pin.
		0 Signal Low
		1 Signal High
13 (R/W)	PX13	Port x Bit 13 Data. The PORT_DATA.PX13 bit indicates a signal on a GPIO pin.
		0 Signal Low
		1 Signal High
12 (R/W)	PX12	Port x Bit 12 Data. The PORT_DATA.PX12 bit indicates a signal on a GPIO pin.
		0 Signal Low
		1 Signal High
11 (R/W)	PX11	Port x Bit 11 Data. The PORT_DATA.PX11 bit indicates a signal on a GPIO pin.
		0 Signal Low
		1 Signal High
10 (R/W)	PX10	Port x Bit 10 Data. The PORT_DATA.PX10 bit indicates a signal on a GPIO pin.
		0 Signal Low
		1 Signal High
9 (R/W)	PX9	Port x Bit 9 Data. The PORT_DATA.PX9 bit indicates a signal on a GPIO pin.
		0 Signal Low
		1 Signal High

Table 11-12: PORT_DATA Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
8 (R/W)	PX8	Port x Bit 8 Data. The PORT_DATA.PX8 bit indicates a signal on a GPIO pin.
		0 Signal Low
		1 Signal High
7 (R/W)	PX7	Port x Bit 7 Data. The PORT_DATA.PX7 bit indicates a signal on a GPIO pin.
		0 Signal Low
		1 Signal High
6 (R/W)	PX6	Port x Bit 6 Data. The PORT_DATA.PX6 bit indicates a signal on a GPIO pin.
		0 Signal Low
		1 Signal High
5 (R/W)	PX5	Port x Bit 5 Data. The PORT_DATA.PX5 bit indicates a signal on a GPIO pin.
		0 Signal Low
		1 Signal High
4 (R/W)	PX4	Port x Bit 4 Data. The PORT_DATA.PX4 bit indicates a signal on a GPIO pin.
		0 Signal Low
		1 Signal High
3 (R/W)	PX3	Port x Bit 3 Data. The PORT_DATA.PX3 bit indicates a signal on a GPIO pin.
		0 Signal Low
		1 Signal High
2 (R/W)	PX2	Port x Bit 2 Data. The PORT_DATA.PX2 bit indicates a signal on a GPIO pin.
		0 Signal Low
		1 Signal High

Table 11-12: PORT_DATA Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/W)	PX1	Port x Bit 1 Data. The PORT_DATA.PX1 bit indicates a signal on a GPIO pin.
		0 Signal Low
		1 Signal High
0 (R/W)	PX0	Port x Bit 0 Data. The PORT_DATA.PX0 bit indicates a signal on a GPIO pin.
		0 Signal Low
		1 Signal High

Port x GPIO Data Clear Register

The `PORT_DATA_CLR` register operates differently for port bits/pins, depending on whether the bit/pin is output mode or input mode. For more information, see the `PORT_DATA` register description.

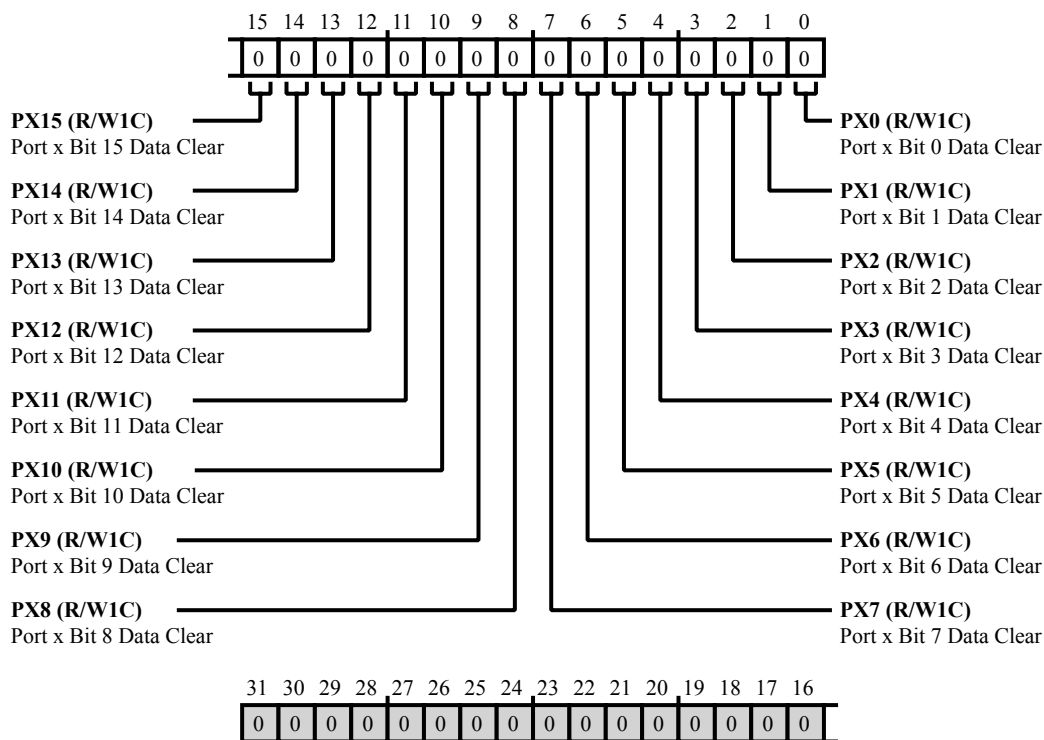


Figure 11-8: `PORT_DATA_CLR` Register Diagram

Table 11-13: `PORT_DATA_CLR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W1C)	PX15	Port x Bit 15 Data Clear. The <code>PORT_DATA_CLR</code> . PX15 bit clears the pin without impacting other pins of the port.
		0 No Effect
		1 Clear Bit. Write 1 for signal low in output mode.
14 (R/W1C)	PX14	Port x Bit 14 Data Clear. The <code>PORT_DATA_CLR</code> . PX14 bit clears the pin without impacting other pins of the port.
		0 No Effect. Write 0 has no effect in output mode.
		1 Clear Bit. Write 1 for signal low in output mode.

Table 11-13: PORT_DATA_CLR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
13 (R/W1C)	PX13	Port x Bit 13 Data Clear. The PORT_DATA_CLR.PX13 bit clears the pin without impacting other pins of the port.
		0 No Effect. Write 0 has no effect in output mode.
		1 Clear Bit. Write 1 for signal low in output mode.
12 (R/W1C)	PX12	Port x Bit 12 Data Clear. The PORT_DATA_CLR.PX12 bit clears the pin without impacting other pins of the port.
		0 No Effect. Write 0 has no effect in output mode.
		1 Clear Bit. Write 1 for signal low in output mode.
11 (R/W1C)	PX11	Port x Bit 11 Data Clear. The PORT_DATA_CLR.PX11 bit clears the pin without impacting other pins of the port.
		0 No Effect
		1 Clear Bit. Write 1 for signal low in output mode.
10 (R/W1C)	PX10	Port x Bit 10 Data Clear. The PORT_DATA_CLR.PX10 bit clears the pin without impacting other pins of the port.
		0 No Effect. Write 0 has no effect in output mode.
		1 Clear Bit. Write 1 for signal low in output mode.
9 (R/W1C)	PX9	Port x Bit 9 Data Clear. The PORT_DATA_CLR.PX9 bit clears the pin without impacting other pins of the port.
		0 No Effect. Write 0 has no effect in output mode.
		1 Clear Bit. Write 1 for signal low in output mode.
8 (R/W1C)	PX8	Port x Bit 8 Data Clear. The PORT_DATA_CLR.PX8 bit clears the pin without impacting other pins of the port.
		0 No Effect. Write 0 has no effect in output mode.
		1 Clear Bit. Write 1 for signal low in output mode.

Table 11-13: PORT_DATA_CLR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
7 (R/W1C)	PX7	Port x Bit 7 Data Clear. The PORT_DATA_CLR . PX7 bit clears the pin without impacting other pins of the port.
		0 No Effect. Write 0 has no effect in output mode.
		1 Clear Bit. Write 1 for signal low in output mode.
6 (R/W1C)	PX6	Port x Bit 6 Data Clear. The PORT_DATA_CLR . PX6 bit clears the pin without impacting other pins of the port.
		0 No Effect. Write 0 has no effect in output mode.
		1 Clear Bit. Write 1 for signal low in output mode.
5 (R/W1C)	PX5	Port x Bit 5 Data Clear. The PORT_DATA_CLR . PX5 bit clears the pin without impacting other pins of the port.
		0 No Effect. Write 0 has no effect in output mode.
		1 Clear Bit. Write 1 for signal low in output mode.
4 (R/W1C)	PX4	Port x Bit 4 Data Clear. The PORT_DATA_CLR . PX4 bit clears the pin without impacting other pins of the port.
		0 No Effect. Write 0 has no effect in output mode.
		1 Clear Bit. Write 1 for signal low in output mode.
3 (R/W1C)	PX3	Port x Bit 3 Data Clear. The PORT_DATA_CLR . PX3 bit clears the pin without impacting other pins of the port.
		0 No Effect. Write 0 has no effect in output mode.
		1 Clear Bit. Write 1 for signal low in output mode.
2 (R/W1C)	PX2	Port x Bit 2 Data Clear. The PORT_DATA_CLR . PX2 bit clears the pin without impacting other pins of the port.
		0 No Effect Write 0 has no effect in output mode.
		1 Clear Bit Write 1 for signal low in output mode.

Table 11-13: PORT_DATA_CLR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
1 (R/W1C)	PX1	Port x Bit 1 Data Clear. The PORT_DATA_CLR.PX1 bit clears the pin without impacting other pins of the port.	
		0	No Effect. Write 0 has no effect in output mode.
		1	Clear Bit. Write 1 for signal low in output mode.
0 (R/W1C)	PX0	Port x Bit 0 Data Clear. The PORT_DATA_CLR.PX0 bit clears the pin without impacting other pins of the port.	
		0	No Effect. Write 0 has no effect in output mode.
		1	Clear Bit. Write 1 for signal low in output mode.

Port x GPIO Data Set Register

The `PORT_DATA_SET` register operates differently for port bits/pins, depending on whether the bit/pin is output mode or input mode. For more information, see the `PORT_DATA` register description.

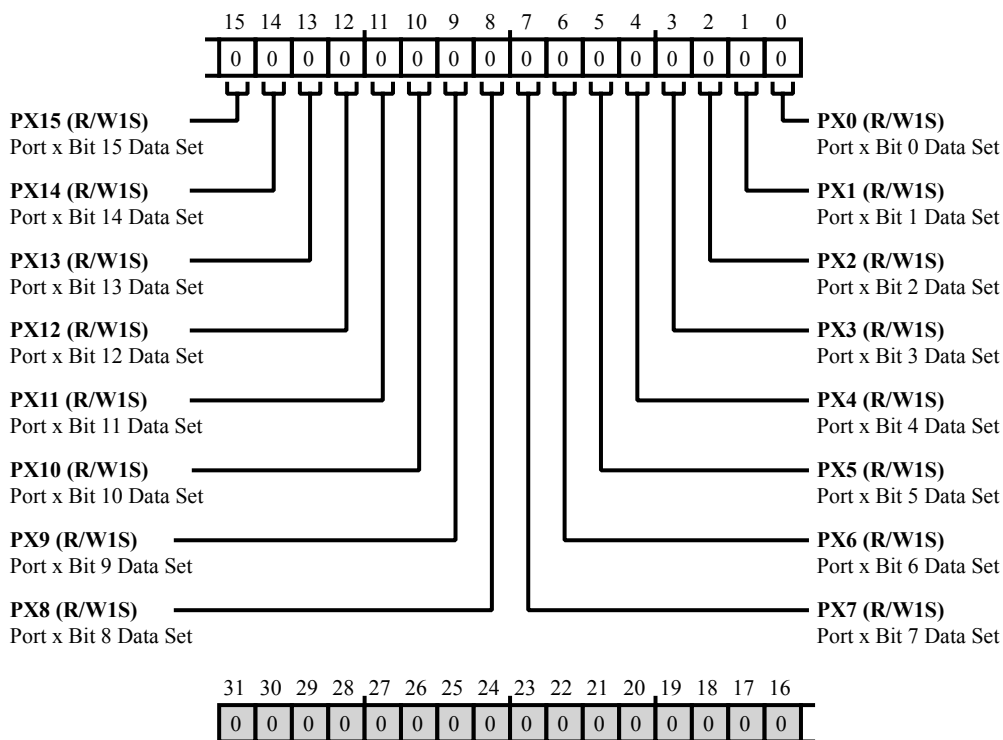


Figure 11-9: PORT_DATA_SET Register Diagram

Table 11-14: PORT_DATA_SET Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W1S)	PX15	Port x Bit 15 Data Set.
		0 No Effect. Write 0 has no effect in output mode.
		1 Set Bit. Write 1 for signal high in output mode.
14 (R/W1S)	PX14	Port x Bit 14 Data Set.
		0 No Effect. Write 0 has no effect in output mode.
		1 Set Bit. Write 1 for signal high in output mode.
13 (R/W1S)	PX13	Port x Bit 13 Data Set.
		0 No Effect. Write 0 has no effect in output mode.
		1 Set Bit. Write 1 for signal high in output mode.

Table 11-14: PORT_DATA_SET Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
12 (R/W1S)	PX12	Port x Bit 12 Data Set.	
		0	No Effect. Write 0 has no effect in output mode.
		1	Set Bit. Write 1 for signal high in output mode.
11 (R/W1S)	PX11	Port x Bit 11 Data Set.	
		0	No Effect. Write 0 has no effect in output mode.
		1	Set Bit.
10 (R/W1S)	PX10	Port x Bit 10 Data Set.	
		0	No Effect. Write 0 has no effect in output mode.
		1	Set Bit. Write 1 for signal high in output mode.
9 (R/W1S)	PX9	Port x Bit 9 Data Set.	
		0	No Effect. Write 0 has no effect in output mode.
		1	Set Bit. Write 1 for signal high in output mode.
8 (R/W1S)	PX8	Port x Bit 8 Data Set.	
		0	No Effect. Write 0 has no effect in output mode.
		1	Set Bit. Write 1 for signal high in output mode.
7 (R/W1S)	PX7	Port x Bit 7 Data Set.	
		0	No Effect. Write 0 has no effect in output mode.
		1	Set Bit. Write 1 for signal high in output mode.
6 (R/W1S)	PX6	Port x Bit 6 Data Set.	
		0	No Effect. Write 0 has no effect in output mode.
		1	Set Bit. Write 1 for signal high in output mode.
5 (R/W1S)	PX5	Port x Bit 5 Data Set.	
		0	No Effect. Write 0 has no effect in output mode.
		1	Set Bit. Write 1 for signal high in output mode.
4 (R/W1S)	PX4	Port x Bit 4 Data Set.	
		0	No Effect. Write 0 has no effect in output mode.
		1	Set Bit. Write 1 for signal high in output mode.
3 (R/W1S)	PX3	Port x Bit 3 Data Set.	
		0	No Effect. Write 0 has no effect in output mode.
		1	Set Bit. Write 1 for signal high in output mode.

Table 11-14: PORT_DATA_SET Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
2 (R/W1S)	PX2	Port x Bit 2 Data Set.	
		0	No Effect. Write 0 has no effect in output mode.
		1	Set Bit. Write 1 for signal high in output mode.
1 (R/W1S)	PX1	Port x Bit 1 Data Set.	
		0	No Effect. Write 0 has no effect in output mode.
		1	Set Bit. Write 1 for signal high in output mode.
0 (R/W1S)	PX0	Port x Bit 0 Data Set.	
		0	No Effect. Write 0 has no effect in output mode.
		1	Set Bit. Write 1 for signal high in output mode.

Port x GPIO Output Toggle Register

The `PORT_DATA_TGL` register permits toggling the state of output GPIO pins. Setting bits in the `PORT_DATA_TGL` register affects the state of specific pins without impacting other pins of the port.

Reading the `PORT_DATA_TGL` returns the state of the `PORT_DATA` register output pin state, but does not return the input pin/signal state.

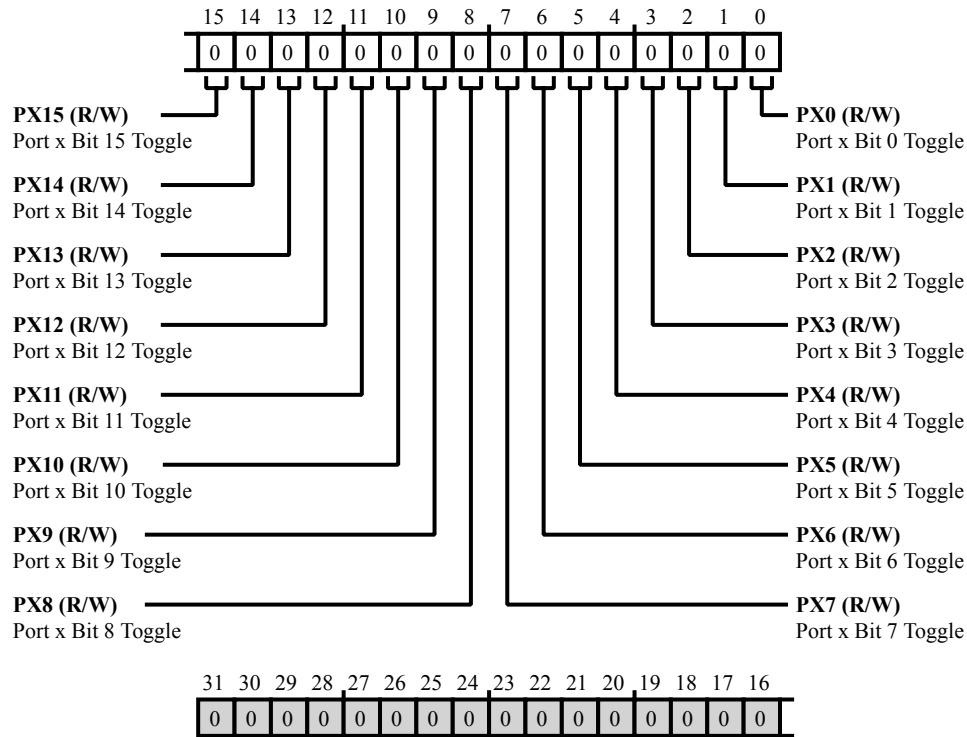


Figure 11-10: `PORT_DATA_TGL` Register Diagram

Table 11-15: `PORT_DATA_TGL` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W)	PX15	Port x Bit 15 Toggle. The <code>PORT_DATA_TGL</code> , PX15 bit toggles the output GPIO bit/pin state.
		0 No Effect
		1 Toggle Bit.
14 (R/W)	PX14	Port x Bit 14 Toggle. The <code>PORT_DATA_TGL</code> , PX14 bit toggles the output GPIO bit/pin state.
		0 No Effect
		1 Toggle Bit

Table 11-15: PORT_DATA_TGL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
13 (R/W)	PX13	Port x Bit 13 Toggle. The PORT_DATA_TGL.PX13 bit toggles the output GPIO bit/pin state.
		0 No Effect
		1 Toggle Bit
12 (R/W)	PX12	Port x Bit 12 Toggle. The PORT_DATA_TGL.PX12 bit toggles the output GPIO bit/pin state.
		0 No Effect
		1 Toggle Bit
11 (R/W)	PX11	Port x Bit 11 Toggle. The PORT_DATA_TGL.PX11 bit toggles the output GPIO bit/pin state.
		0 No Effect
		1 Toggle Bit
10 (R/W)	PX10	Port x Bit 10 Toggle. The PORT_DATA_TGL.PX10 bit toggles the output GPIO bit/pin state.
		0 No Effect
		1 Toggle Bit
9 (R/W)	PX9	Port x Bit 9 Toggle. The PORT_DATA_TGL.PX9 bit toggles the output GPIO bit/pin state.
		0 No Effect
		1 Toggle Bit
8 (R/W)	PX8	Port x Bit 8 Toggle. The PORT_DATA_TGL.PX8 bit toggles the output GPIO bit/pin state.
		0 No Effect
		1 Toggle Bit
7 (R/W)	PX7	Port x Bit 7 Toggle. The PORT_DATA_TGL.PX7 bit toggles the output GPIO bit/pin state.
		0 No Effect
		1 Toggle Bit

Table 11-15: PORT_DATA_TGL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
6 (R/W)	PX6	Port x Bit 6 Toggle. The PORT_DATA_TGL.PX6 bit toggles the output GPIO bit/pin state.
		0 No Effect
		1 Toggle Bit
5 (R/W)	PX5	Port x Bit 5 Toggle. The PORT_DATA_TGL.PX5 bit toggles the output GPIO bit/pin state.
		0 No Effect
		1 Toggle Bit
4 (R/W)	PX4	Port x Bit 4 Toggle. The PORT_DATA_TGL.PX4 bit toggles the output GPIO bit/pin state.
		0 No Effect
		1 Toggle Bit
3 (R/W)	PX3	Port x Bit 3 Toggle. The PORT_DATA_TGL.PX3 bit toggles the output GPIO bit/pin state.
		0 No Effect
		1 Toggle Bit
2 (R/W)	PX2	Port x Bit 2 Toggle. The PORT_DATA_TGL.PX2 bit toggles the output GPIO bit/pin state.
		0 No Effect
		1 Toggle Bit
1 (R/W)	PX1	Port x Bit 1 Toggle. The PORT_DATA_TGL.PX1 bit toggles the output GPIO bit/pin state.
		0 No Effect
		1 Toggle Bit
0 (R/W)	PX0	Port x Bit 0 Toggle. The PORT_DATA_TGL.PX0 bit toggles the output GPIO bit/pin state.
		0 No Effect
		1 Toggle Bit

Port x GPIO Direction Register

The `PORT_DIR`, `PORT_DIR_SET`, and `PORT_DIR_CLR` registers select output or input mode for GPIO pins and enable output drivers. Use the `PORT_INEN`, `PORT_INEN_SET`, and `PORT_INEN_CLR` registers to enable or disable input drivers.

Writes to the `PORT_DIR` register affect the state of all pins of the port. To select a direction for specific pins without impacting other pins of the port, use the `PORT_DIR_SET` and `PORT_DIR_CLR` registers.

Setting a bit in the `PORT_DIR` register enables output mode on the corresponding a GPIO pin. Clearing a bit in the `PORT_DIR` register disables output mode on the corresponding GPIO pin.

Input Mode - The default mode of every GPIO pin after reset is the input mode, but the input drivers are not enabled. To enable GPIO input drivers, set the corresponding bits in the `PORT_INEN` register. When enabled, a read from the `PORT_DATA` register returns the logical state of the input pin. The input signal does not overwrite the state of the bit used for the output case. That state can only be altered by software. If the input driver is enabled, a write to the `PORT_DATA` register can alter the state of the bit, but the change cannot be read back.

Output Mode - Any GPIO pin can be configured for output mode. The GPIO output drivers are enabled by setting the corresponding bits in the `PORT_DIR`, `PORT_DIR_SET`, or `PORT_DIR_CLR` registers. By using the `PORT_DIR_SET` and `PORT_DIR_CLR` registers, the direction of the signal flow of individual GPIO pins can be altered by separate software threads without mutually impacting other GPIOs on the same port. Both registers return the same value when read. Because the state of the GPIO output can already be controlled before the output driver is enabled, it is recommended to first set or clear the bit (using the `PORT_DATA`, `PORT_DATA_SET`, or `PORT_DATA_CLR` registers) to avoid any volatile levels on the output.

Open-Drain Mode - Every GPIO can also be used in open-drain mode. To accomplish this, first, clear the respective bit in the `PORT_DATA` or `PORT_DATA_CLR` register. Then, set the one bit in the `PORT_INEN` register. Reads from the `PORT_DATA` register then return the status from the pin and do not return the state of the internal flip-flop. By toggling the output driver through the `PORT_DIR_SET` and `PORT_DIR_CLR` register pair, the output signal can be pulled low or three-stated as required. Note that the polarity of the driven signal can be inverted when the internal flip-flop is set instead. When a GPIO port is used in open-drain mode, take care to not exceed the V_{IH} operating condition associated with the respective pin.

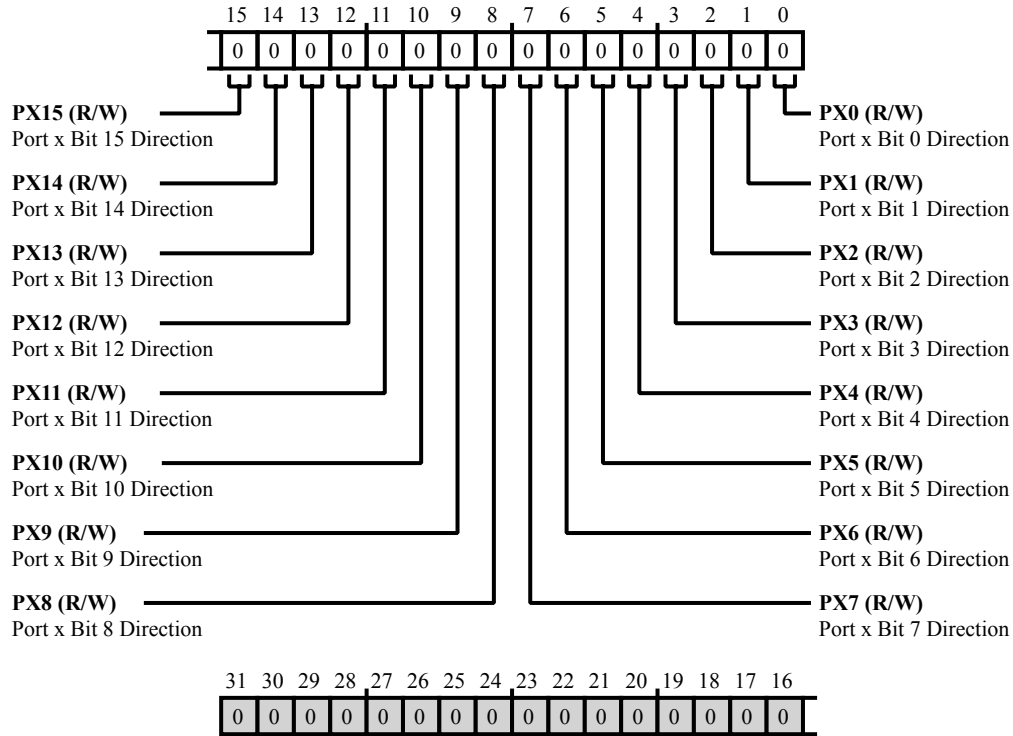


Figure 11-11: PORT_DIR Register Diagram

Table 11-16: PORT_DIR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W)	PX15	Port x Bit 15 Direction.
		0 Input mode. The output driver is disabled.
		1 Output mode. The output driver is enabled.
14 (R/W)	PX14	Port x Bit 14 Direction.
		0 Input mode. The output driver is disabled.
		1 Output mode. The output driver is enabled.
13 (R/W)	PX13	Port x Bit 13 Direction.
		0 Input mode. The output driver is disabled.
		1 Output mode. The output driver is enabled.
12 (R/W)	PX12	Port x Bit 12 Direction.
		0 Input mode. The output driver is disabled.
		1 Output mode. The output driver is enabled.

Table 11-16: PORT_DIR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
11 (R/W)	PX11	Port x Bit 11 Direction.	
		0	Input mode. The output driver is disabled.
		1	Output mode. The output driver is enabled.
10 (R/W)	PX10	Port x Bit 10 Direction.	
		0	Input mode. The output driver is disabled.
		1	Output mode. The output driver is enabled.
9 (R/W)	PX9	Port x Bit 9 Direction.	
		0	Input mode. The output driver is disabled.
		1	Output mode. The output driver is enabled.
8 (R/W)	PX8	Port x Bit 8 Direction.	
		0	Input mode. The output driver is disabled.
		1	Output mode. The output driver is enabled.
7 (R/W)	PX7	Port x Bit 7 Direction.	
		0	Input mode. The output driver is disabled.
		1	Output mode. The output driver is enabled.
6 (R/W)	PX6	Port x Bit 6 Direction.	
		0	Input mode. The output driver is disabled.
		1	Output mode. The output driver is enabled.
5 (R/W)	PX5	Port x Bit 5 Direction.	
		0	Input mode. The output driver is disabled.
		1	Output mode. The output driver is enabled.
4 (R/W)	PX4	Port x Bit 4 Direction.	
		0	Input mode. The output driver is disabled.
		1	Output mode. The output driver is enabled.
3 (R/W)	PX3	Port x Bit 3 Direction.	
		0	Input mode. The output driver is disabled.
		1	Output mode. The output driver is enabled.
2 (R/W)	PX2	Port x Bit 2 Direction.	
		0	Input mode. The output driver is disabled.
		1	Output mode. The output driver is enabled.

Table 11-16: PORT_DIR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
1 (R/W)	PX1	Port x Bit 1 Direction.	
		0	Input mode. The output driver is disabled.
		1	Output mode. The output driver is enabled.
0 (R/W)	PX0	Port x Bit 0 Direction.	
		0	Input mode. The output driver is disabled.
		1	Output mode. The output driver is enabled.

Port x GPIO Direction Clear Register

The `PORT_DIR_CLR` register disables output mode and disables the output drivers for GPIO pins. For more information, see the `PORT_DIR` register description.

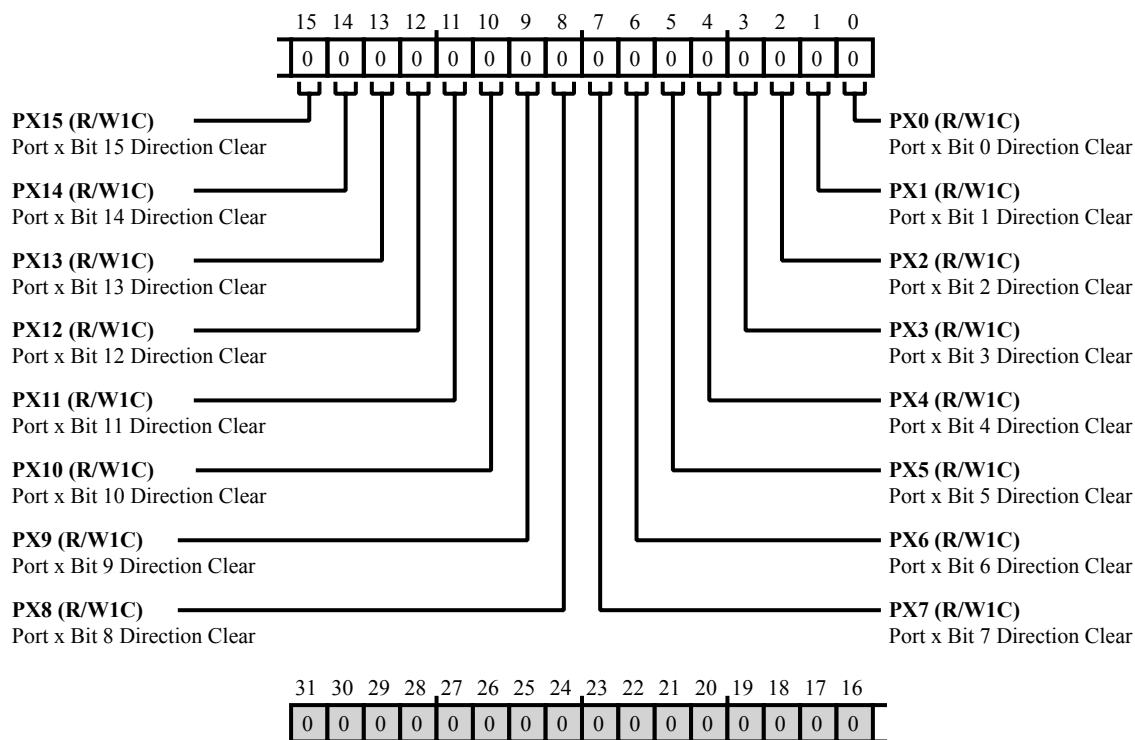


Figure 11-12: `PORT_DIR_CLR` Register Diagram

Table 11-17: `PORT_DIR_CLR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W1C)	PX15	Port x Bit 15 Direction Clear. The <code>PORT_DIR_CLR</code> . PX15 bit disables output mode and the output drivers for port x.
		0 No Effect
		1 Disable output mode/driver
14 (R/W1C)	PX14	Port x Bit 14 Direction Clear. The <code>PORT_DIR_CLR</code> . PX14 bit disables output mode and the output drivers for port x.
		0 No Effect
		1 Disable output mode/driver

Table 11-17: PORT_DIR_CLR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
13 (R/W1C)	PX13	Port x Bit 13 Direction Clear. The PORT_DIR_CLR.PX13 bit disables output mode and the output drivers for port x.
		0 No Effect
		1 Disable output mode/driver
12 (R/W1C)	PX12	Port x Bit 12 Direction Clear. The PORT_DIR_CLR.PX12 bit disables output mode and the output drivers for port x.
		0 No Effect
		1 Disable output mode/driver
11 (R/W1C)	PX11	Port x Bit 11 Direction Clear. The PORT_DIR_CLR.PX11 bit disables output mode and the output drivers for port x.
		0 No Effect
		1 Disable output mode/driver
10 (R/W1C)	PX10	Port x Bit 10 Direction Clear. The PORT_DIR_CLR.PX10 bit disables output mode and the output drivers for port x.
		0 No Effect
		1 Disable output mode/driver
9 (R/W1C)	PX9	Port x Bit 9 Direction Clear. The PORT_DIR_CLR.PX9 bit disables output mode and the output drivers for port x.
		0 No Effect
		1 Disable output mode/driver
8 (R/W1C)	PX8	Port x Bit 8 Direction Clear. The PORT_DIR_CLR.PX8 bit disables output mode and the output drivers for port x.
		0 No Effect
		1 Disable output mode/driver

Table 11-17: PORT_DIR_CLR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
7 (R/W1C)	PX7	Port x Bit 7 Direction Clear. The PORT_DIR_CLR.PX7 bit disables output mode and the output drivers for port x.
		0 No Effect
		1 Disable output mode/driver
6 (R/W1C)	PX6	Port x Bit 6 Direction Clear. The PORT_DIR_CLR.PX6 bit disables output mode and the output drivers for port x.
		0 No Effect
		1 Disable output mode/driver
5 (R/W1C)	PX5	Port x Bit 5 Direction Clear. The PORT_DIR_CLR.PX5 bit disables output mode and the output drivers for port x.
		0 No Effect
		1 Disable output mode/driver
4 (R/W1C)	PX4	Port x Bit 4 Direction Clear. The PORT_DIR_CLR.PX4 bit disables output mode and the output drivers for port x.
		0 No Effect
		1 Disable output mode/driver
3 (R/W1C)	PX3	Port x Bit 3 Direction Clear. The PORT_DIR_CLR.PX3 bit disables output mode and the output drivers for port x.
		0 No Effect
		1 Disable output mode/driver
2 (R/W1C)	PX2	Port x Bit 2 Direction Clear. The PORT_DIR_CLR.PX2 bit disables output mode and the output drivers for port x.
		0 No Effect
		1 Disable output mode/driver

Table 11-17: PORT_DIR_CLR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/W1C)	PX1	Port x Bit 1 Direction Clear. The PORT_DIR_CLR.PX1 bit disables output mode and the output drivers for port x.
		0 No Effect
		1 Disable output mode/driver
0 (R/W1C)	PX0	Port x Bit 0 Direction Clear. The PORT_DIR_CLR.PX0 bit disables output mode and the output drivers for port x.
		0 No Effect
		1 Disable output mode/driver

Port x GPIO Direction Set Register

The `PORT_DIR_SET` register enables output mode and output drivers for GPIO pins. For more information, see the `PORT_DIR` register description.

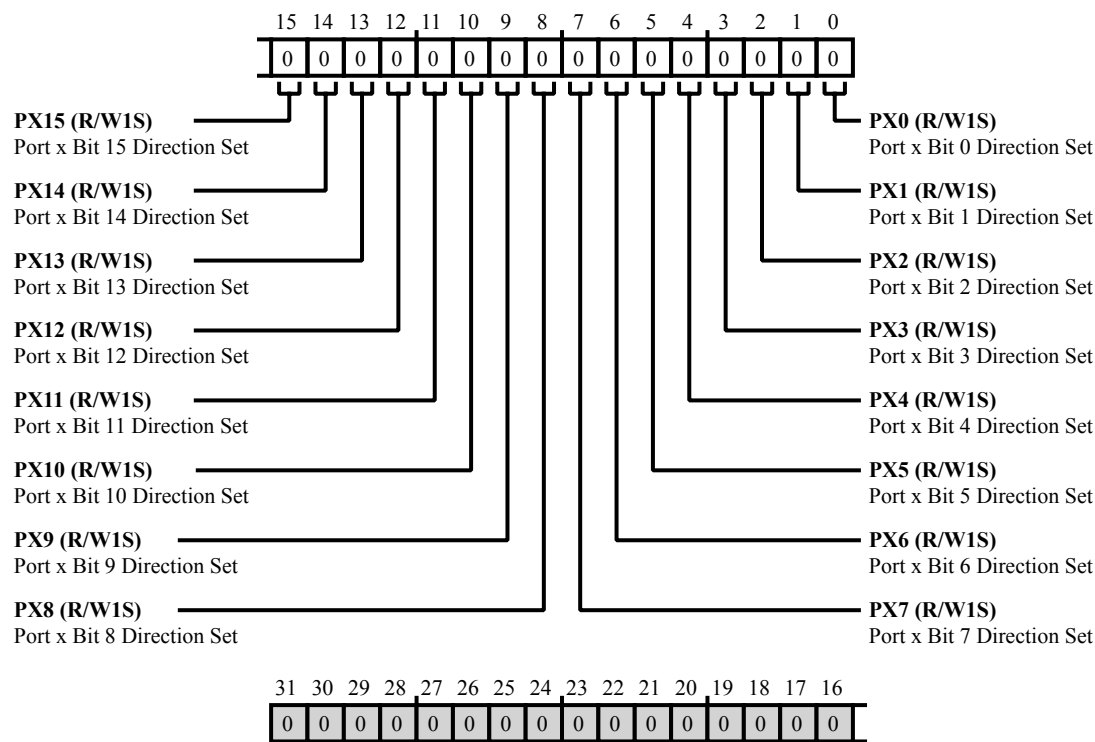


Figure 11-13: `PORT_DIR_SET` Register Diagram

Table 11-18: `PORT_DIR_SET` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W1S)	PX15	Port x Bit 15 Direction Set. The <code>PORT_DIR_SET</code> . PX15 bit enables the output mode/driver for port x.
		0 No Effect
		1 Enable output mode/driver
14 (R/W1S)	PX14	Port x Bit 14 Direction Set. The <code>PORT_DIR_SET</code> . PX14 bit enables the output mode/driver for port x.
		0 No Effect
		1 Enable output mode/driver

Table 11-18: PORT_DIR_SET Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
13 (R/W1S)	PX13	Port x Bit 13 Direction Set. The PORT_DIR_SET . PX13 bit enables the output mode/driver for port x.
		0 No Effect
		1 Enable output mode/driver
12 (R/W1S)	PX12	Port x Bit 12 Direction Set. The PORT_DIR_SET . PX12 bit enables the output mode/driver for port x.
		0 No Effect
		1 Enable output mode/driver
11 (R/W1S)	PX11	Port x Bit 11 Direction Set. The PORT_DIR_SET . PX11 bit enables the output mode/driver for port x.
		0 No Effect
		1 Enable output mode/driver
10 (R/W1S)	PX10	Port x Bit 10 Direction Set. The PORT_DIR_SET . PX10 bit enables the output mode/driver for port x.
		0 No Effect
		1 Enable output mode/driver
9 (R/W1S)	PX9	Port x Bit 9 Direction Set. The PORT_DIR_SET . PX9 bit enables the output mode/driver for port x.
		0 No Effect
		1 Enable output mode/driver
8 (R/W1S)	PX8	Port x Bit 8 Direction Set. The PORT_DIR_SET . PX8 bit enables the output mode/driver for port x.
		0 No Effect
		1 Enable output mode/driver
7 (R/W1S)	PX7	Port x Bit 7 Direction Set. The PORT_DIR_SET . PX7 bit enables the output mode/driver for port x.
		0 No Effect
		1 Enable output mode/driver

Table 11-18: PORT_DIR_SET Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
6 (R/W1S)	PX6	Port x Bit 6 Direction Set. The PORT_DIR_SET . PX6 bit enables the output mode/driver for port x.	
		0	No Effect
		1	Enable output mode/driver
5 (R/W1S)	PX5	Port x Bit 5 Direction Set. The PORT_DIR_SET . PX5 bit enables the output mode/driver for port x.	
		0	No Effect
		1	Enable output mode/driver
4 (R/W1S)	PX4	Port x Bit 4 Direction Set. The PORT_DIR_SET . PX4 bit enables the output mode/driver for port x.	
		0	No Effect
		1	Enable output mode/driver
3 (R/W1S)	PX3	Port x Bit 3 Direction Set. The PORT_DIR_SET . PX3 bit enables the output mode/driver for port x.	
		0	No Effect
		1	Enable output mode/driver
2 (R/W1S)	PX2	Port x Bit 2 Direction Set. The PORT_DIR_SET . PX2 bit enables the output mode/driver for port x.	
		0	No Effect
		1	Enable output mode/driver
1 (R/W1S)	PX1	Port x Bit 1 Direction Set. The PORT_DIR_SET . PX1 bit enables the output mode/driver for port x.	
		0	No Effect
		1	Enable output mode/driver
0 (R/W1S)	PX0	Port x Bit 0 Direction Set. The PORT_DIR_SET . PX0 bit enables the output mode/driver for port x.	
		0	No Effect
		1	Enable output mode/driver

Port x Function Enable Register

The `PORT_FER` register bits indicate each port bit's operating mode: general purpose I/O mode or peripheral mode. After reset, all pins default to GPIO mode. Setting a bit in the `PORT_FER` registers enables a peripheral module to take ownership of the pin. The function enable bits impact output control only. Regardless of the setting of the function enable bits, both GPIO and peripherals can still sense the pin input. After a function is enabled, it is up to the `PORT_MUX` registers as to which peripheral takes control.

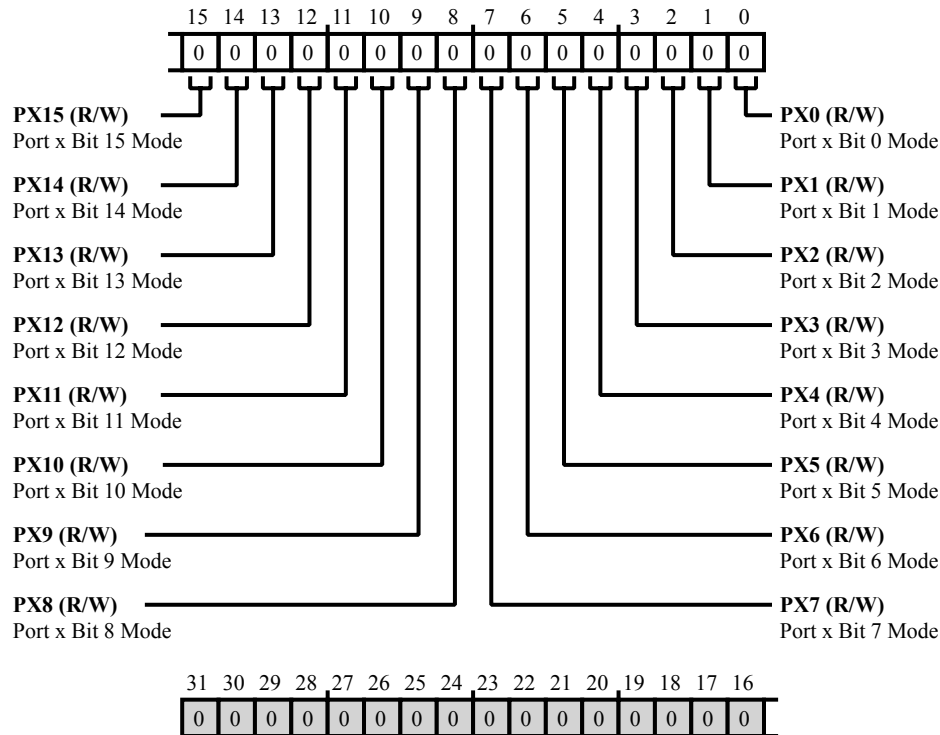


Figure 11-14: PORT_FER Register Diagram

Table 11-19: PORT_FER Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W)	PX15	Port x Bit 15 Mode. The <code>PORT_FER.PX15</code> bit indicates the operating mode for port x.
		0 GPIO Mode
		1 Peripheral Mode
14 (R/W)	PX14	Port x Bit 14 Mode. The <code>PORT_FER.PX14</code> bit indicates the operating mode for port x.
		0 GPIO Mode
		1 Peripheral Mode

Table 11-19: PORT_FER Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
13 (R/W)	PX13	Port x Bit 13 Mode. The PORT_FER.PX13 bit indicates the operating mode for port x.
		0 GPIO Mode
		1 Peripheral Mode
12 (R/W)	PX12	Port x Bit 12 Mode. The PORT_FER.PX12 bit indicates the operating mode for port x.
		0 GPIO Mode
		1 Peripheral Mode
11 (R/W)	PX11	Port x Bit 11 Mode. The PORT_FER.PX11 bit indicates the operating mode for port x.
		0 GPIO Mode
		1 Peripheral Mode
10 (R/W)	PX10	Port x Bit 10 Mode. The PORT_FER.PX10 bit indicates the operating mode for port x.
		0 GPIO Mode
		1 Peripheral Mode
9 (R/W)	PX9	Port x Bit 9 Mode. The PORT_FER.PX9 bit indicates the operating mode for port x.
		0 GPIO Mode
		1 Peripheral Mode
8 (R/W)	PX8	Port x Bit 8 Mode. The PORT_FER.PX8 bit indicates the operating mode for port x.
		0 GPIO Mode
		1 Peripheral Mode
7 (R/W)	PX7	Port x Bit 7 Mode. The PORT_FER.PX7 bit indicates the operating mode for port x.
		0 GPIO Mode
		1 Peripheral Mode

Table 11-19: PORT_FER Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
6 (R/W)	PX6	Port x Bit 6 Mode. The PORT_FER.PX6 bit indicates the operating mode for port x.
		0 GPIO Mode
		1 Peripheral Mode
5 (R/W)	PX5	Port x Bit 5 Mode. The PORT_FER.PX5 bit indicates the operating mode for port x.
		0 GPIO Mode
		1 Peripheral Mode
4 (R/W)	PX4	Port x Bit 4 Mode. The PORT_FER.PX4 bit indicates the operating mode for port x.
		0 GPIO Mode
		1 Peripheral Mode
3 (R/W)	PX3	Port x Bit 3 Mode. The PORT_FER.PX3 bit indicates the operating mode for port x.
		0 GPIO Mode
		1 Peripheral Mode
2 (R/W)	PX2	Port x Bit 2 Mode. The PORT_FER.PX2 bit indicates the operating mode for port x.
		0 GPIO Mode
		1 Peripheral Mode
1 (R/W)	PX1	Port x Bit 1 Mode. The PORT_FER.PX1 bit indicates the operating mode for port x.
		0 GPIO Mode
		1 Peripheral Mode
0 (R/W)	PX0	Port x Bit 0 Mode. The PORT_FER.PX0 bit indicates the operating mode for port x.
		0 GPIO Mode
		1 Peripheral Mode

Port x Function Enable Clear Register

The `PORT_FER_CLR` register permits enabling GPIO mode for each bit and corresponding GPIO pin. Writing 1 to a bit in `PORT_FER_CLR` enables GPIO mode for the corresponding pin.

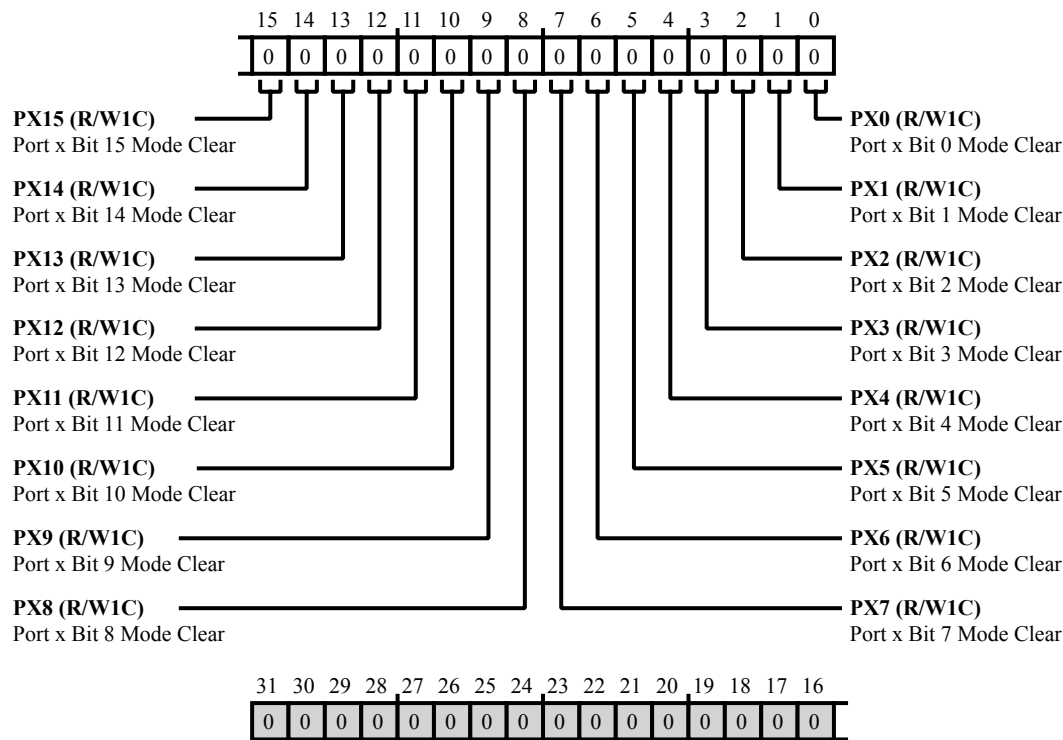


Figure 11-15: PORT_FER_CLR Register Diagram

Table 11-20: PORT_FER_CLR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration	
15 (R/W1C)	PX15	Port x Bit 15 Mode Clear. The <code>PORT_FER_CLR</code> . PX15 bit enables GPIO mode.	
		0	No Effect
		1	Set Bit for GPIO Mode
14 (R/W1C)	PX14	Port x Bit 14 Mode Clear. The <code>PORT_FER_CLR</code> . PX14 bit enables GPIO mode.	
		0	No Effect
		1	Set Bit for GPIO Mode

Table 11-20: PORT_FER_CLR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
13 (R/W1C)	PX13	Port x Bit 13 Mode Clear. The PORT_FER_CLR.PX13 bit enables GPIO mode.
		0 No Effect
		1 Set Bit for GPIO Mode
12 (R/W1C)	PX12	Port x Bit 12 Mode Clear. The PORT_FER_CLR.PX12 bit enables GPIO mode.
		0 No Effect
		1 Set Bit for GPIO Mode
11 (R/W1C)	PX11	Port x Bit 11 Mode Clear. The PORT_FER_CLR.PX11 bit enables GPIO mode.
		0 No Effect
		1 Set Bit for GPIO Mode
10 (R/W1C)	PX10	Port x Bit 10 Mode Clear. The PORT_FER_CLR.PX10 bit enables GPIO mode.
		0 No Effect
		1 Set Bit for GPIO Mode
9 (R/W1C)	PX9	Port x Bit 9 Mode Clear. The PORT_FER_CLR.PX9 bit enables GPIO mode.
		0 No Effect
		1 Set Bit for GPIO Mode
8 (R/W1C)	PX8	Port x Bit 8 Mode Clear. The PORT_FER_CLR.PX8 bit enables GPIO mode.
		0 No Effect
		1 Set Bit for GPIO Mode
7 (R/W1C)	PX7	Port x Bit 7 Mode Clear. The PORT_FER_CLR.PX7 bit enables GPIO mode.
		0 No Effect
		1 Set Bit for GPIO Mode

Table 11-20: PORT_FER_CLR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
6 (R/W1C)	PX6	Port x Bit 6 Mode Clear. The PORT_FER_CLR.PX6 bit enables GPIO mode.	
		0	No Effect
		1	Set Bit for GPIO Mode
5 (R/W1C)	PX5	Port x Bit 5 Mode Clear. The PORT_FER_CLR.PX5 bit enables GPIO mode.	
		0	No Effect
		1	Set Bit for GPIO Mode
4 (R/W1C)	PX4	Port x Bit 4 Mode Clear. The PORT_FER_CLR.PX4 bit enables GPIO mode.	
		0	No Effect
		1	Set Bit for GPIO Mode
3 (R/W1C)	PX3	Port x Bit 3 Mode Clear. The PORT_FER_CLR.PX3 bit enables GPIO mode.	
		0	No Effect
		1	Set Bit for GPIO Mode
2 (R/W1C)	PX2	Port x Bit 2 Mode Clear. The PORT_FER_CLR.PX2 bit enables GPIO mode.	
		0	No Effect
		1	Set Bit for GPIO Mode
1 (R/W1C)	PX1	Port x Bit 1 Mode Clear. The PORT_FER_CLR.PX1 bit enables GPIO mode.	
		0	No Effect
		1	Set Bit for GPIO Mode
0 (R/W1C)	PX0	Port x Bit 0 Mode Clear. The PORT_FER_CLR.PX0 bit enables GPIO mode.	
		0	No Effect
		1	Set Bit for GPIO Mode

Port x Function Enable Set Register

The `PORT_FER_SET` register permits enabling peripheral mode for each bit and corresponding GPIO pin. Writing 1 to a bit in `PORT_FER_SET` enables peripheral mode for the corresponding pin.

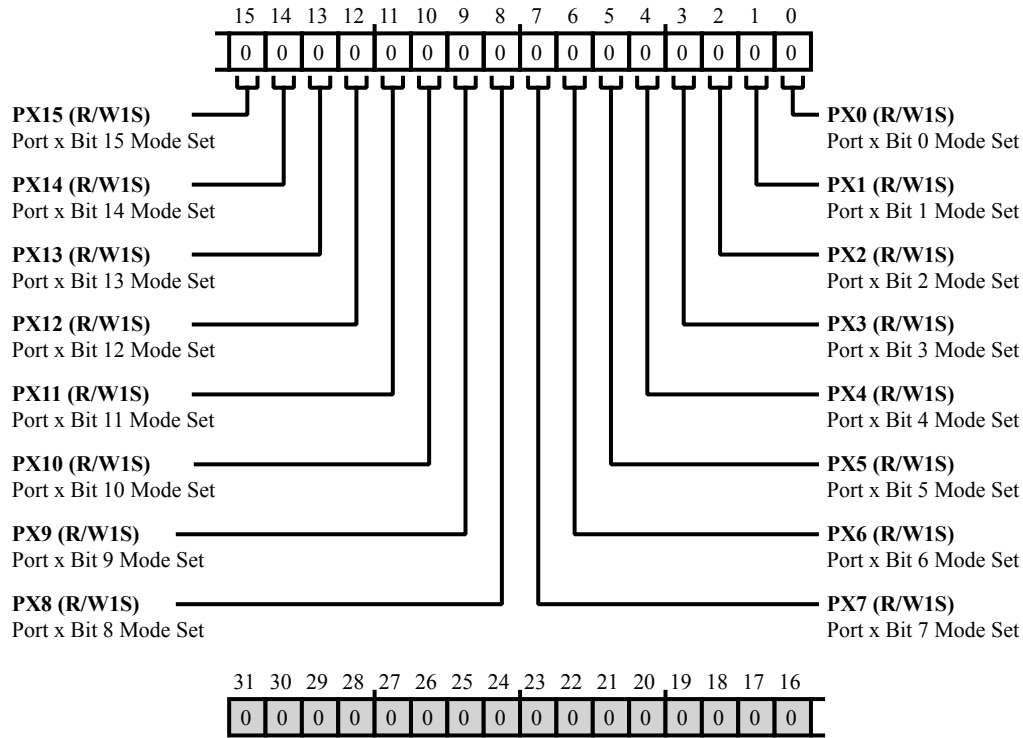


Figure 11-16: `PORT_FER_SET` Register Diagram

Table 11-21: `PORT_FER_SET` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W1S)	PX15	Port x Bit 15 Mode Set. The <code>PORT_FER_SET</code> . PX15 bit enables peripheral mode.
		0 No Effect
		1 Set Bit for Peripheral Mode
14 (R/W1S)	PX14	Port x Bit 14 Mode Set. The <code>PORT_FER_SET</code> . PX14 bit enables peripheral mode.
		0 No Effect
		1 Set Bit for Peripheral Mode

Table 11-21: PORT_FER_SET Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
13 (R/W1S)	PX13	Port x Bit 13 Mode Set. The PORT_FER_SET . PX13 bit enables peripheral mode.	
		0	No Effect
		1	Set Bit for Peripheral Mode
12 (R/W1S)	PX12	Port x Bit 12 Mode Set. The PORT_FER_SET . PX12 bit enables peripheral mode.	
		0	No Effect
		1	Set Bit for Peripheral Mode
11 (R/W1S)	PX11	Port x Bit 11 Mode Set. The PORT_FER_SET . PX11 bit enables peripheral mode.	
		0	No Effect
		1	Set Bit for Peripheral Mode
10 (R/W1S)	PX10	Port x Bit 10 Mode Set. The PORT_FER_SET . PX10 bit enables peripheral mode.	
		0	No Effect
		1	Set Bit for Peripheral Mode
9 (R/W1S)	PX9	Port x Bit 9 Mode Set. The PORT_FER_SET . PX9 bit enables peripheral mode.	
		0	No Effect
		1	Set Bit for Peripheral Mode
8 (R/W1S)	PX8	Port x Bit 8 Mode Set. The PORT_FER_SET . PX8 bit enables peripheral mode.	
		0	No Effect
		1	Set Bit for Peripheral Mode
7 (R/W1S)	PX7	Port x Bit 7 Mode Set. The PORT_FER_SET . PX7 bit enables peripheral mode.	
		0	No Effect
		1	Set Bit for Peripheral Mode

Table 11-21: PORT_FER_SET Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
6 (R/W1S)	PX6	Port x Bit 6 Mode Set. The PORT_FER_SET . PX6 bit enables peripheral mode.
		0 No Effect
		1 Set Bit for Peripheral Mode
5 (R/W1S)	PX5	Port x Bit 5 Mode Set. The PORT_FER_SET . PX5 bit enables peripheral mode.
		0 No Effect
		1 Set Bit for Peripheral Mode
4 (R/W1S)	PX4	Port x Bit 4 Mode Set. The PORT_FER_SET . PX4 bit enables peripheral mode.
		0 No Effect
		1 Set Bit for Peripheral Mode
3 (R/W1S)	PX3	Port x Bit 3 Mode Set. The PORT_FER_SET . PX3 bit enables peripheral mode.
		0 No Effect
		1 Set Bit for Peripheral Mode
2 (R/W1S)	PX2	Port x Bit 2 Mode Set. The PORT_FER_SET . PX2 bit enables peripheral mode.
		0 No Effect
		1 Set Bit for Peripheral Mode
1 (R/W1S)	PX1	Port x Bit 1 Mode Set. The PORT_FER_SET . PX1 bit enables peripheral mode.
		0 No Effect
		1 Set Bit for Peripheral Mode
0 (R/W1S)	PX0	Port x Bit 0 Mode Set. The PORT_FER_SET . PX0 bit enables peripheral mode.
		0 No Effect
		1 Set Bit for Peripheral Mode

Port x GPIO Input Enable Register

The `PORT_INEN`, `PORT_INEN_SET`, and `PORT_INEN_CLR` registers enable or disable input drivers, which are required for using a GPIO pin in input mode.

Writes to the `PORT_INEN` register affect the input drivers for all pins of the port. To set or clear specific pin drivers without impacting other pin drivers of the port, use the `PORT_INEN_SET` and `PORT_INEN_CLR` registers.

If the input is enabled, reads from the `PORT_DATA`, `PORT_DATA_SET`, or `PORT_DATA_CLR` registers return the state of the pins. However, the state of the output is not overwritten by the input. It is altered by software writes only. Input and output drivers can be enabled at the same time. In this case, a read of the data register returns the true value of the data register and not the pin state.

For more information, see the `PORT_DATA` register description and the `PORT_DIR` register description.

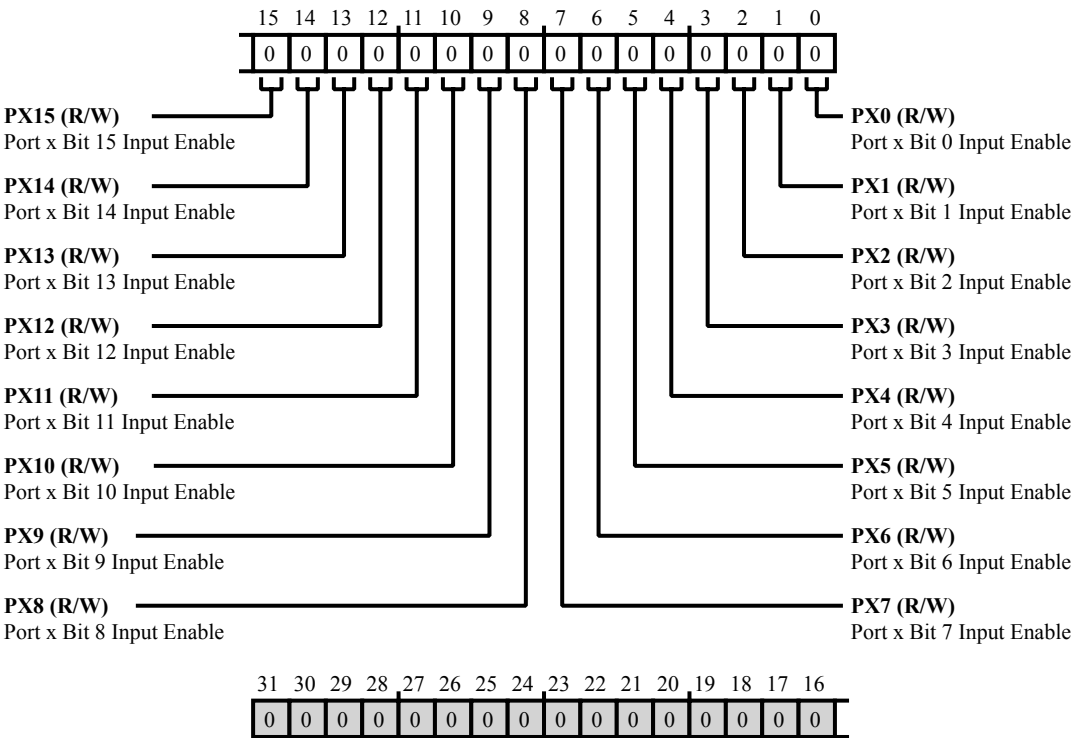


Figure 11-17: PORT_INEN Register Diagram

Table 11-22: PORT_INEN Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W)	PX15	Port x Bit 15 Input Enable.
		0 Disable Input Driver
		1 Enable Input Driver

Table 11-22: PORT_INEN Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
14 (R/W)	PX14	Port x Bit 14 Input Enable.	
		0	Disable Input Driver
		1	Enable Input Driver
13 (R/W)	PX13	Port x Bit 13 Input Enable.	
		0	Disable Input Driver
		1	Enable Input Driver
12 (R/W)	PX12	Port x Bit 12 Input Enable.	
		0	Disable Input Driver
		1	Enable Input Driver
11 (R/W)	PX11	Port x Bit 11 Input Enable.	
		0	Disable Input Driver
		1	Enable Input Driver
10 (R/W)	PX10	Port x Bit 10 Input Enable.	
		0	Disable Input Driver
		1	Enable Input Driver
9 (R/W)	PX9	Port x Bit 9 Input Enable.	
		0	Disable Input Driver
		1	Enable Input Driver
8 (R/W)	PX8	Port x Bit 8 Input Enable.	
		0	Disable Input Driver
		1	Enable Input Driver
7 (R/W)	PX7	Port x Bit 7 Input Enable.	
		0	Disable Input Driver
		1	Enable Input Driver
6 (R/W)	PX6	Port x Bit 6 Input Enable.	
		0	Disable Input Driver
		1	Enable Input Driver
5 (R/W)	PX5	Port x Bit 5 Input Enable.	
		0	Disable Input Driver
		1	Enable Input Driver

Table 11-22: PORT_INEN Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
4 (R/W)	PX4	Port x Bit 4 Input Enable.	
		0	Disable Input Driver
		1	Enable Input Driver
3 (R/W)	PX3	Port x Bit 3 Input Enable.	
		0	Disable Input Driver
		1	Enable Input Driver
2 (R/W)	PX2	Port x Bit 2 Input Enable.	
		0	Disable Input Driver
		1	Enable Input Driver
1 (R/W)	PX1	Port x Bit 1 Input Enable.	
		0	Disable Input Driver
		1	Enable Input Driver
0 (R/W)	PX0	Port x Bit 0 Input Enable.	
		0	Disable Input Driver
		1	Enable Input Driver

Port x GPIO Input Enable Clear Register

The `PORT_INEN_CLR` register disables the input drivers for GPIO pins. For more information, see the `PORT_INEN` register description.

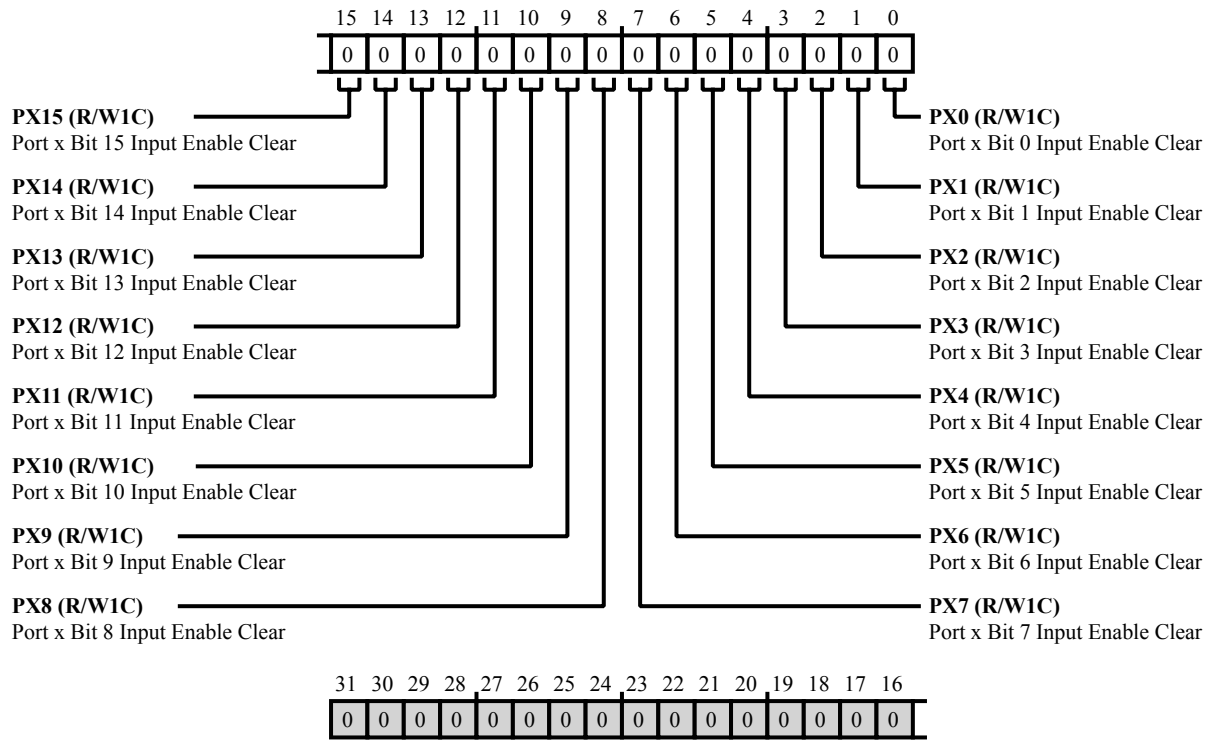


Figure 11-18: `PORT_INEN_CLR` Register Diagram

Table 11-23: `PORT_INEN_CLR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W1C)	PX15	Port x Bit 15 Input Enable Clear.
		0 No Effect
		1 Clear Bit. Set to disable the input driver.
14 (R/W1C)	PX14	Port x Bit 14 Input Enable Clear.
		0 No Effect
		1 Clear Bit. Set to disable the input driver.
13 (R/W1C)	PX13	Port x Bit 13 Input Enable Clear.
		0 No Effect
		1 Clear Bit. Set to disable the input driver.

Table 11-23: PORT_INEN_CLR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
12 (R/W1C)	PX12	Port x Bit 12 Input Enable Clear.	
		0	No Effect
		1	Clear Bit. Set to disable the input driver.
11 (R/W1C)	PX11	Port x Bit 11 Input Enable Clear.	
		0	No Effect
		1	Clear Bit. Set to disable the input driver.
10 (R/W1C)	PX10	Port x Bit 10 Input Enable Clear.	
		0	No Effect
		1	Clear Bit. Set to disable the input driver.
9 (R/W1C)	PX9	Port x Bit 9 Input Enable Clear.	
		0	No Effect
		1	Clear Bit. Set to disable the input driver.
8 (R/W1C)	PX8	Port x Bit 8 Input Enable Clear.	
		0	No Effect
		1	Clear Bit. Set to disable the input driver.
7 (R/W1C)	PX7	Port x Bit 7 Input Enable Clear.	
		0	No Effect
		1	Clear Bit. Set to disable the input driver.
6 (R/W1C)	PX6	Port x Bit 6 Input Enable Clear.	
		0	No Effect
		1	Clear Bit. Set to disable the input driver.
5 (R/W1C)	PX5	Port x Bit 5 Input Enable Clear.	
		0	No Effect
		1	Clear Bit. Set to disable the input driver.
4 (R/W1C)	PX4	Port x Bit 4 Input Enable Clear.	
		0	No Effect
		1	Clear Bit. Set to disable the input driver.
3 (R/W1C)	PX3	Port x Bit 3 Input Enable Clear.	
		0	No Effect
		1	Clear Bit. Set to disable the input driver.

Table 11-23: PORT_INEN_CLR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
2 (R/W1C)	PX2	Port x Bit 2 Input Enable Clear.	
		0	No Effect
		1	Clear Bit. Set to disable the input driver.
1 (R/W1C)	PX1	Port x Bit 1 Input Enable Clear.	
		0	No Effect
		1	Clear Bit. Set to disable the input driver.
0 (R/W1C)	PX0	Port x Bit 0 Input Enable Clear.	
		0	No Effect
		1	Clear Bit. Set to disable the input driver.

Port x GPIO Input Enable Set Register

The `PORT_INEN_SET` register enables input drivers for GPIO pins. For more information, see the `PORT_INEN` register description.

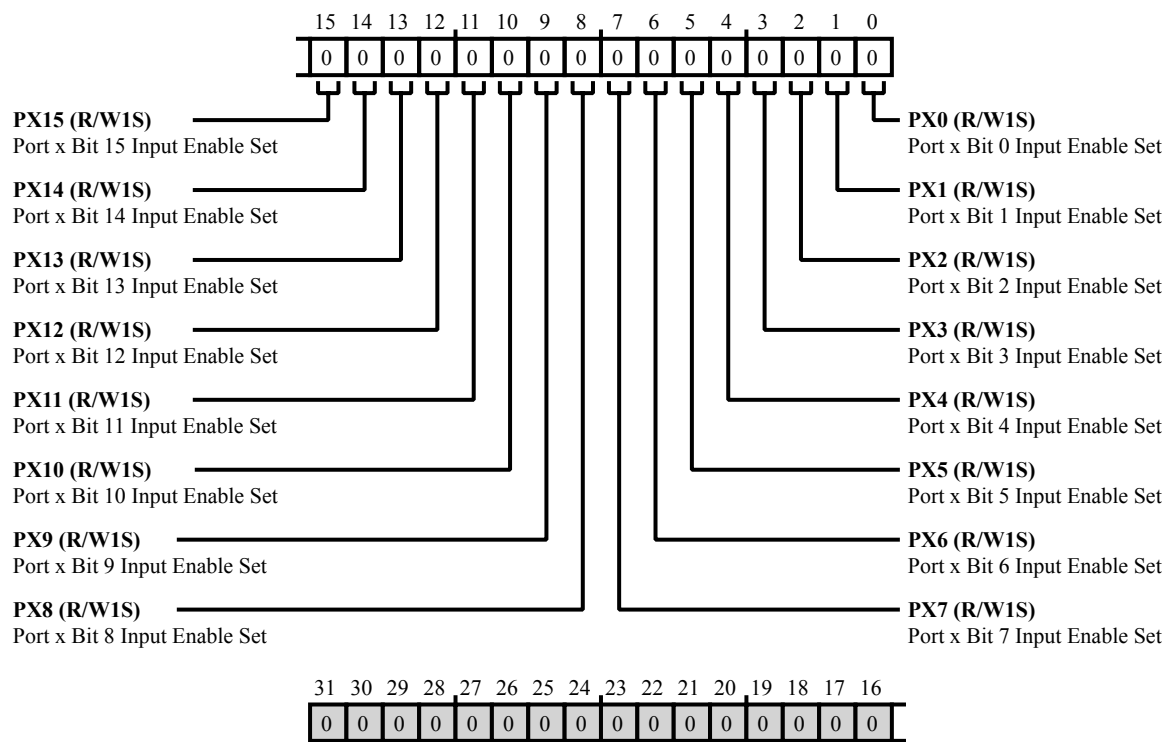


Figure 11-19: `PORT_INEN_SET` Register Diagram

Table 11-24: `PORT_INEN_SET` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W1S)	PX15	Port x Bit 15 Input Enable Set.
		0 No Effect
		1 Set Bit. Set to enable the input driver.
14 (R/W1S)	PX14	Port x Bit 14 Input Enable Set.
		0 No Effect
		1 Set Bit. Set to enable the input driver.
13 (R/W1S)	PX13	Port x Bit 13 Input Enable Set.
		0 No Effect
		1 Set Bit. Set to enable the input driver.

Table 11-24: PORT_INEN_SET Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
12 (R/W1S)	PX12	Port x Bit 12 Input Enable Set.	
		0	No Effect
		1	Set Bit. Set to enable the input driver.
11 (R/W1S)	PX11	Port x Bit 11 Input Enable Set.	
		0	No Effect
		1	Set Bit. Set to enable the input driver.
10 (R/W1S)	PX10	Port x Bit 10 Input Enable Set.	
		0	No Effect
		1	Set Bit. Set to enable the input driver.
9 (R/W1S)	PX9	Port x Bit 9 Input Enable Set.	
		0	No Effect
		1	Set Bit. Set to enable the input driver.
8 (R/W1S)	PX8	Port x Bit 8 Input Enable Set.	
		0	No Effect
		1	Set Bit. Set to enable the input driver.
7 (R/W1S)	PX7	Port x Bit 7 Input Enable Set.	
		0	No Effect
		1	Set Bit. Set to enable the input driver.
6 (R/W1S)	PX6	Port x Bit 6 Input Enable Set.	
		0	No Effect
		1	Set Bit. Set to enable the input driver.
5 (R/W1S)	PX5	Port x Bit 5 Input Enable Set.	
		0	No Effect
		1	Set Bit. Set to enable the input driver.
4 (R/W1S)	PX4	Port x Bit 4 Input Enable Set.	
		0	No Effect
		1	Set Bit. Set to enable the input driver.
3 (R/W1S)	PX3	Port x Bit 3 Input Enable Set.	
		0	No Effect
		1	Set Bit. Set to enable the input driver.

Table 11-24: PORT_INEN_SET Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
2 (R/W1S)	PX2	Port x Bit 2 Input Enable Set.	
		0	No Effect
		1	Set Bit. Set to enable the input driver.
1 (R/W1S)	PX1	Port x Bit 1 Input Enable Set.	
		0	No Effect
		1	Set Bit. Set to enable the input driver.
0 (R/W1S)	PX0	Port x Bit 0 Input Enable Set.	
		0	No Effect
		1	Set Bit. Set to enable the input driver.

Port x GPIO Lock Register

The `PORT_LOCK` register enables (unlocks) or disables (locks) write access selectively for the PORT control registers.

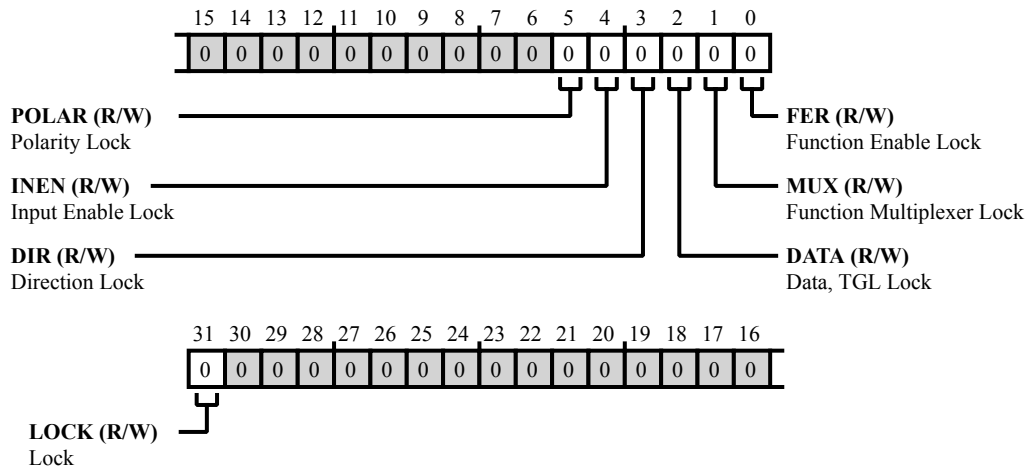


Figure 11-20: `PORT_LOCK` Register Diagram

Table 11-25: `PORT_LOCK` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock. If the global lock bit is set (<code>SPU_CTL.GLCK</code> bit =1) and the <code>PORT_LOCK.LOCK</code> bit is set, the <code>PORT_LOCK</code> register is read only (locked).
		0 Unlock
		1 Lock
5 (R/W)	POLAR	Polarity Lock. The <code>PORT_LOCK.POLAR</code> disables write access to the <code>PORT_POL</code> , <code>PORT_POL_SET</code> , and <code>PORT_POL_CLR</code> registers.
		0 Unlock POL
		1 Lock POL
4 (R/W)	INEN	Input Enable Lock. The <code>PORT_LOCK.INEN</code> disables write access to the <code>PORT_INEN</code> , <code>PORT_INEN_SET</code> , and <code>PORT_INEN_CLR</code> registers.
		0 Unlock INEN
		1 Lock INEN

Table 11-25: PORT_LOCK Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
3 (R/W)	DIR	Direction Lock. The PORT_LOCK.DIR disables write access to the PORT_DIR, PORT_DIR_SET, PORT_DIR_CLR registers.
		0 Unlock DIR
		1 Lock DIR
2 (R/W)	DATA	Data, TGL Lock. The PORT_LOCK.DATA disables write access to the PORT_DATA, PORT_DATA_SET, PORT_DATA_CLR, and PORT_DATA_TGL registers.
		0 Unlock DATA
		1 Lock DATA
1 (R/W)	MUX	Function Multiplexer Lock. The PORT_LOCK.MUX disables write accesses to the PORT_MUX register.
		0 Unlock MUX
		1 Lock MUX
0 (R/W)	FER	Function Enable Lock. The PORT_LOCK.FER disables write access to the PORT_FER, PORT_FER_SET, and PORT_FER_CLR registers.
		0 Unlock FER
		1 Lock FER

Port x Multiplexer Control Register

When a pin is in peripheral mode (not GPIO mode), the `PORT_MUX` register controls which peripheral takes ownership of a pin. Ports may have multiple, different peripheral functions. Two bits are required to describe every multiplexer on an individual pin-by-pin scheme. For example, bit 0 and bit 1 of the `PORT_MUX` register control the multiplexer of pin 0, bit 2 and bit 3 of `PORT_MUX` control the multiplexer of pin 1, and so on. The value of any `PORT_MUX` bit has no effect on the port pins when the associated bit in the `PORT_FER` register is 0 (selects GPIO mode). Even if a port has only one function, the `PORT_MUX` register is still present. For single function ports (no multiplexing is needed), leave the `PORT_MUX` bits at 0 (default). For all `PORT_MUX` bit fields: 00 = default/reset peripheral option, 01 = first alternate peripheral option, 10 = second alternate peripheral option, and 11 = third alternate peripheral option.

See the processor data sheet for details regarding the peripheral options associated with each port.

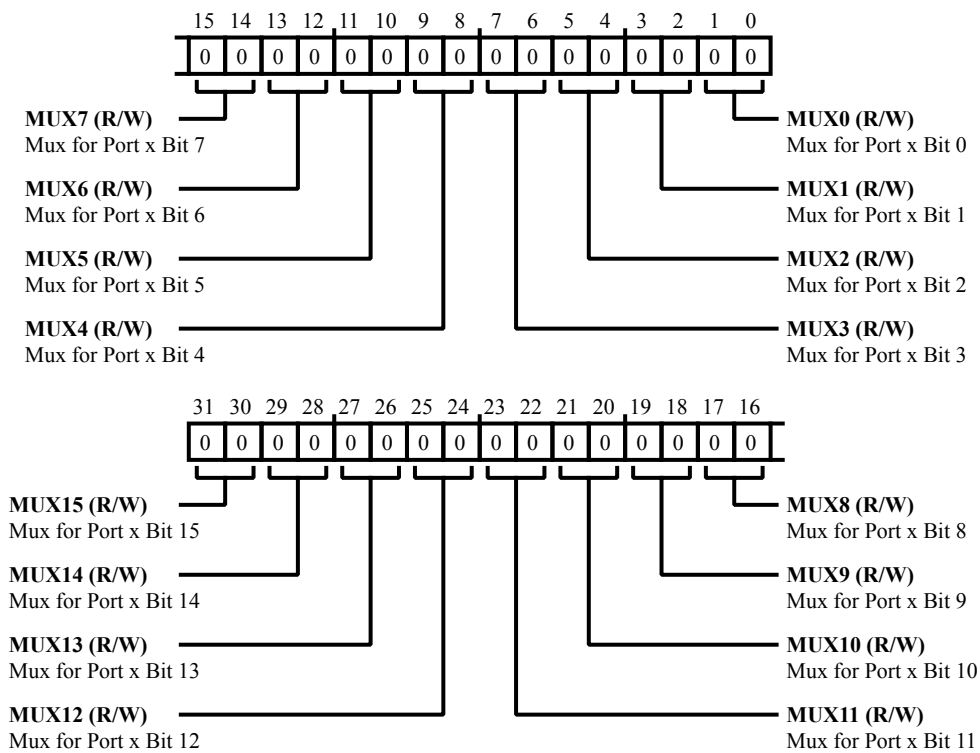


Figure 11-21: `PORT_MUX` Register Diagram

Table 11-26: `PORT_MUX` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:30 (R/W)	MUX15	Mux for Port x Bit 15. The <code>PORT_MUX.MUX15</code> bit provides multiplexer control for port x bit 15.

Table 11-26: PORT_MUX Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
29:28 (R/W)	MUX14	Mux for Port x Bit 14. The PORT_MUX.MUX14 bit provides multiplexer control for port x bit 14.
27:26 (R/W)	MUX13	Mux for Port x Bit 13. The PORT_MUX.MUX13 bit provides multiplexer control for port x bit 13.
25:24 (R/W)	MUX12	Mux for Port x Bit 12. The PORT_MUX.MUX12 bit provides multiplexer control for port x bit 12.
23:22 (R/W)	MUX11	Mux for Port x Bit 11. The PORT_MUX.MUX11 bit provides multiplexer control for port x bit 11.
21:20 (R/W)	MUX10	Mux for Port x Bit 10. The PORT_MUX.MUX10 bit provides multiplexer control for port x bit 10.
19:18 (R/W)	MUX9	Mux for Port x Bit 9. The PORT_MUX.MUX9 bit provides multiplexer control for port x bit 9.
17:16 (R/W)	MUX8	Mux for Port x Bit 8. The PORT_MUX.MUX8 bit provides multiplexer control for port x bit 8.
15:14 (R/W)	MUX7	Mux for Port x Bit 7. The PORT_MUX.MUX7 bit provides multiplexer control for port x bit 7.
13:12 (R/W)	MUX6	Mux for Port x Bit 6. The PORT_MUX.MUX6 bit provides multiplexer control for port x bit 6.
11:10 (R/W)	MUX5	Mux for Port x Bit 5. The PORT_MUX.MUX5 bit provides multiplexer control for port x bit 5.
9:8 (R/W)	MUX4	Mux for Port x Bit 4. The PORT_MUX.MUX4 bit provides multiplexer control for port x bit 4.
7:6 (R/W)	MUX3	Mux for Port x Bit 3. The PORT_MUX.MUX3 bit provides multiplexer control for port x bit 3.
5:4 (R/W)	MUX2	Mux for Port x Bit 2. The PORT_MUX.MUX2 bit provides multiplexer control for port x bit 2.
3:2 (R/W)	MUX1	Mux for Port x Bit 1. The PORT_MUX.MUX1 bit provides multiplexer control for port x bit 1.
1:0 (R/W)	MUX0	Mux for Port x Bit 0. The PORT_MUX.MUX0 bit provides multiplexer control for port x bit 0.

Port x GPIO Polarity Invert Register

The `PORT_POL`, `PORT_POL_SET`, and `PORT_POL_CLR` registers enable or disable inverting polarity of GPIO signals. To invert polarity of peripheral signals, use the inversion selection programming in the signal's corresponding module.

Writes to the `PORT_POL` register affect the polarity inversion selection of all pins of the port. To enable or disable polarity inversion for specific pins without impacting other pins of the port, use the `PORT_POL_SET` and `PORT_POL_CLR` registers.

Setting a bit in the `PORT_POL` register enables polarity inversion on the corresponding inversion GPIO pin, making the pin active-low or falling-edge sensitive. Clearing a bit in the `PORT_POL` register disables polarity (default state) on the corresponding GPIO pin, making it active-high or rising-edge sensitive.

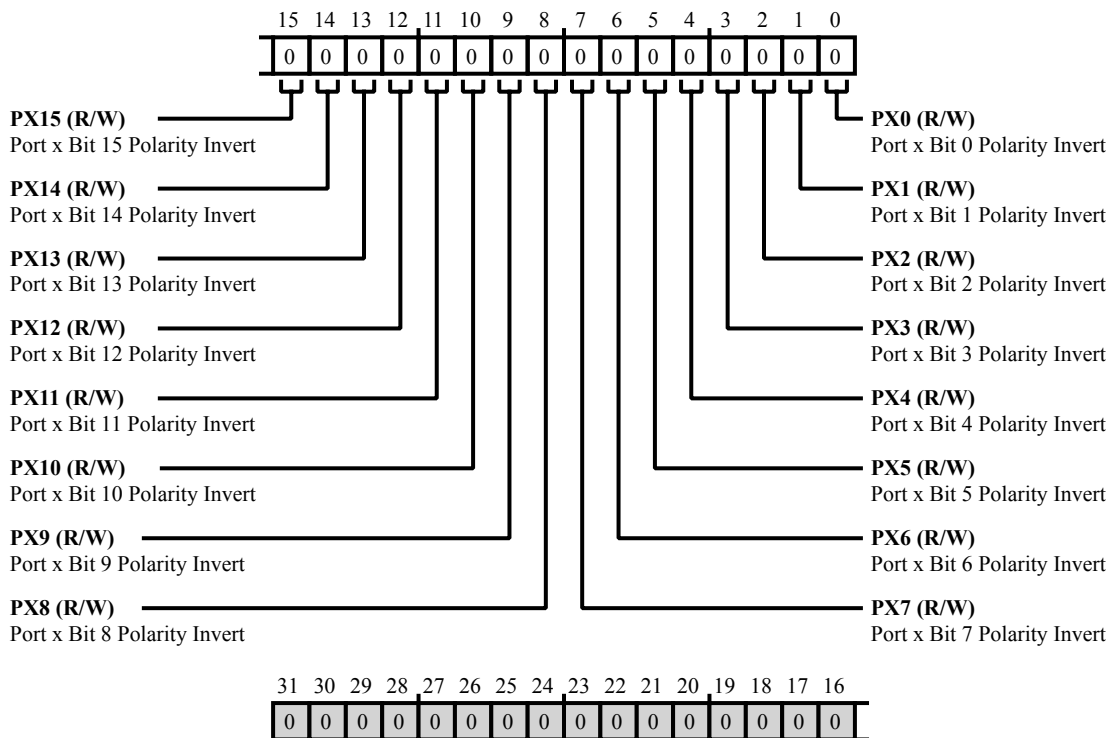


Figure 11-22: PORT_POL Register Diagram

Table 11-27: PORT_POL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W)	PX15	Port x Bit 15 Polarity Invert. The <code>PORT_POL</code> . PX15 bit enables polarity inversion.
		0 No Invert. GPIO is active high or rising edge sensitive.
		1 Invert. GPIO is active low or falling edge sensitive.

Table 11-27: PORT_POL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
14 (R/W)	PX14	Port x Bit 14 Polarity Invert. The PORT_POL.PX14 bit enables polarity inversion.
		0 No Invert. GPIO is active high or rising edge sensitive.
		1 Invert. GPIO is active low or falling edge sensitive.
13 (R/W)	PX13	Port x Bit 13 Polarity Invert. The PORT_POL.PX13 bit enables polarity inversion.
		0 No Invert. GPIO is active high or rising edge sensitive.
		1 Invert. GPIO is active low or falling edge sensitive.
12 (R/W)	PX12	Port x Bit 12 Polarity Invert. The PORT_POL.PX12 bit enables polarity inversion.
		0 No Invert. GPIO is active high or rising edge sensitive.
		1 Invert. GPIO is active low or falling edge sensitive.
11 (R/W)	PX11	Port x Bit 11 Polarity Invert. The PORT_POL.PX11 bit enables polarity inversion.
		0 No Invert. GPIO is active high or rising edge sensitive.
		1 Invert. GPIO is active low or falling edge sensitive.
10 (R/W)	PX10	Port x Bit 10 Polarity Invert. The PORT_POL.PX10 bit enables polarity inversion.
		0 No Invert. GPIO is active high or rising edge sensitive.
		1 Invert. GPIO is active low or falling edge sensitive.
9 (R/W)	PX9	Port x Bit 9 Polarity Invert. The PORT_POL.PX9 bit enables polarity inversion.
		0 No Invert. GPIO is active high or rising edge sensitive.
		1 Invert. GPIO is active low or falling edge sensitive.
8 (R/W)	PX8	Port x Bit 8 Polarity Invert. The PORT_POL.PX8 bit enables polarity inversion.
		0 No Invert. GPIO is active high or rising edge sensitive.
		1 Invert. GPIO is active low or falling edge sensitive.

Table 11-27: PORT_POL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
7 (R/W)	PX7	Port x Bit 7 Polarity Invert. The PORT_POL.PX7 bit enables polarity inversion.
		0 No Invert. GPIO is active high or rising edge sensitive.
		1 Invert. GPIO is active low or falling edge sensitive.
6 (R/W)	PX6	Port x Bit 6 Polarity Invert. The PORT_POL.PX6 bit enables polarity inversion.
		0 No Invert. GPIO is active high or rising edge sensitive.
		1 Invert. GPIO is active low or falling edge sensitive.
5 (R/W)	PX5	Port x Bit 5 Polarity Invert. The PORT_POL.PX5 bit enables polarity inversion.
		0 No Invert. GPIO is active high or rising edge sensitive.
		1 Invert. GPIO is active low or falling edge sensitive.
4 (R/W)	PX4	Port x Bit 4 Polarity Invert. The PORT_POL.PX4 bit enables polarity inversion.
		0 No Invert. GPIO is active high or rising edge sensitive.
		1 Invert. GPIO is active low or falling edge sensitive.
3 (R/W)	PX3	Port x Bit 3 Polarity Invert. The PORT_POL.PX3 bit enables polarity inversion.
		0 No Invert. GPIO is active high or rising edge sensitive.
		1 Invert. GPIO is active low or falling edge sensitive.
2 (R/W)	PX2	Port x Bit 2 Polarity Invert. The PORT_POL.PX2 bit enables polarity inversion.
		0 No Invert. GPIO is active high or rising edge sensitive.
		1 Invert. GPIO is active low or falling edge sensitive.
1 (R/W)	PX1	Port x Bit 1 Polarity Invert. The PORT_POL.PX1 bit enables polarity inversion.
		0 No Invert. GPIO is active high or rising edge sensitive.
		1 Invert. GPIO is active low or falling edge sensitive.

Table 11-27: PORT_POL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	PX0	Port x Bit 0 Polarity Invert. The PORT_POL.PX0 bit enables polarity inversion.
		0 No Invert. GPIO is active high or rising edge sensitive.
		1 Invert. GPIO is active low or falling edge sensitive.

Port x GPIO Polarity Invert Clear Register

The `PORT_POL_CLR` register disables polarity inversion for GPIO pins. For more information, see the `PORT_POL` register description.

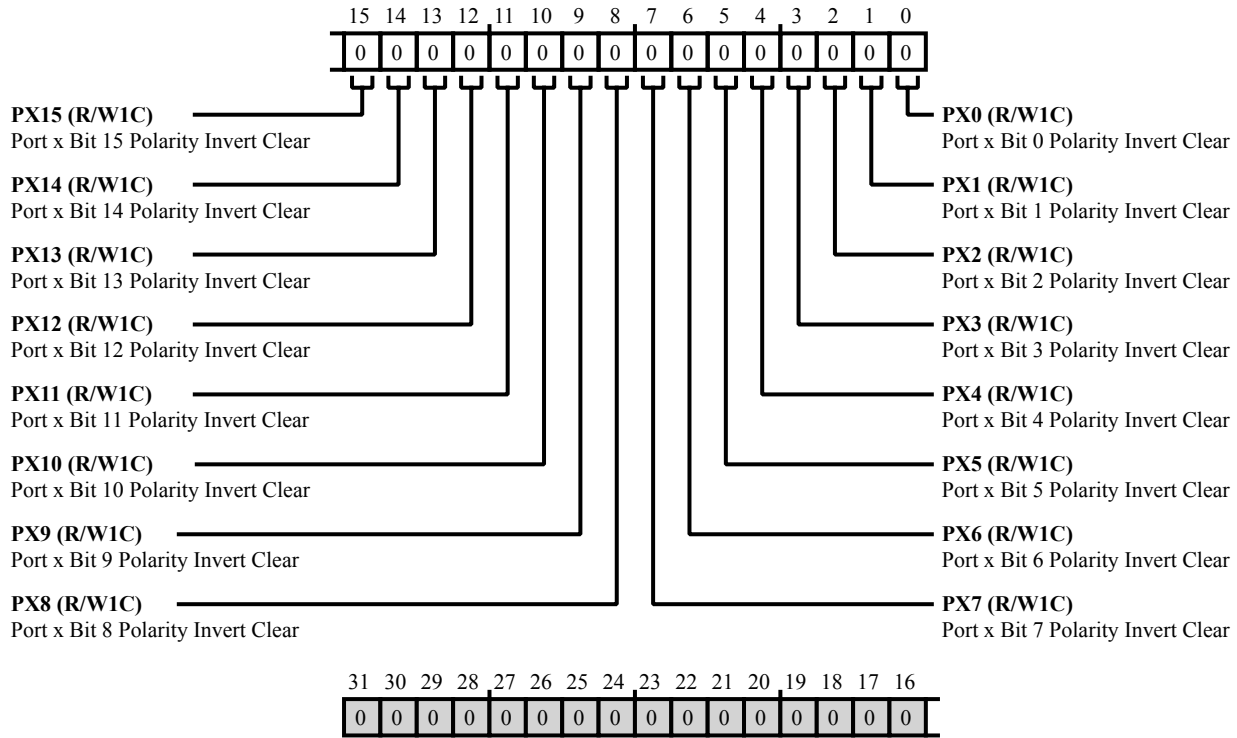


Figure 11-23: `PORT_POL_CLR` Register Diagram

Table 11-28: `PORT_POL_CLR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W1C)	PX15	Port x Bit 15 Polarity Invert Clear.
		0 No Effect
		1 Clear Bit. Set to disable GPIO pin polarity invert.
14 (R/W1C)	PX14	Port x Bit 14 Polarity Invert Clear.
		0 No Effect
		1 Clear Bit. Set to disable GPIO pin polarity invert.
13 (R/W1C)	PX13	Port x Bit 13 Polarity Invert Clear.
		0 No Effect
		1 Clear Bit. Set to disable GPIO pin polarity invert.

Table 11-28: PORT_POL_CLR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
12 (R/W1C)	PX12	Port x Bit 12 Polarity Invert Clear.	
		0	No Effect
		1	Clear Bit. Set to disable GPIO pin polarity invert.
11 (R/W1C)	PX11	Port x Bit 11 Polarity Invert Clear.	
		0	No Effect
		1	Clear Bit. Set to disable GPIO pin polarity invert.
10 (R/W1C)	PX10	Port x Bit 10 Polarity Invert Clear.	
		0	No Effect
		1	Clear Bit. Set to disable GPIO pin polarity invert.
9 (R/W1C)	PX9	Port x Bit 9 Polarity Invert Clear.	
		0	No Effect
		1	Clear Bit. Set to disable GPIO pin polarity invert.
8 (R/W1C)	PX8	Port x Bit 8 Polarity Invert Clear.	
		0	No Effect
		1	Clear Bit. Set to disable GPIO pin polarity invert.
7 (R/W1C)	PX7	Port x Bit 7 Polarity Invert Clear.	
		0	No Effect
		1	Clear Bit. Set to disable GPIO pin polarity invert.
6 (R/W1C)	PX6	Port x Bit 6 Polarity Invert Clear.	
		0	No Effect
		1	Clear Bit. Set to disable GPIO pin polarity invert.
5 (R/W1C)	PX5	Port x Bit 5 Polarity Invert Clear.	
		0	No Effect
		1	Clear Bit. Set to disable GPIO pin polarity invert.
4 (R/W1C)	PX4	Port x Bit 4 Polarity Invert Clear.	
		0	No Effect
		1	Clear Bit. Set to disable GPIO pin polarity invert.
3 (R/W1C)	PX3	Port x Bit 3 Polarity Invert Clear.	
		0	No Effect
		1	Clear Bit. Set to disable GPIO pin polarity invert.

Table 11-28: PORT_POL_CLR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
2 (R/W1C)	PX2	Port x Bit 2 Polarity Invert Clear.	
		0	No Effect
		1	Clear Bit. Set to disable GPIO pin polarity invert.
1 (R/W1C)	PX1	Port x Bit 1 Polarity Invert Clear.	
		0	No Effect
		1	Clear Bit. Set to disable GPIO pin polarity invert.
0 (R/W1C)	PX0	Port x Bit 0 Polarity Invert Clear.	
		0	No Effect
		1	Clear Bit. Set to disable GPIO pin polarity invert.

Port x GPIO Polarity Invert Set Register

The `PORT_POL_SET` register enables polarity inversion for GPIO pins. For more information, see the `PORT_POL` register description.

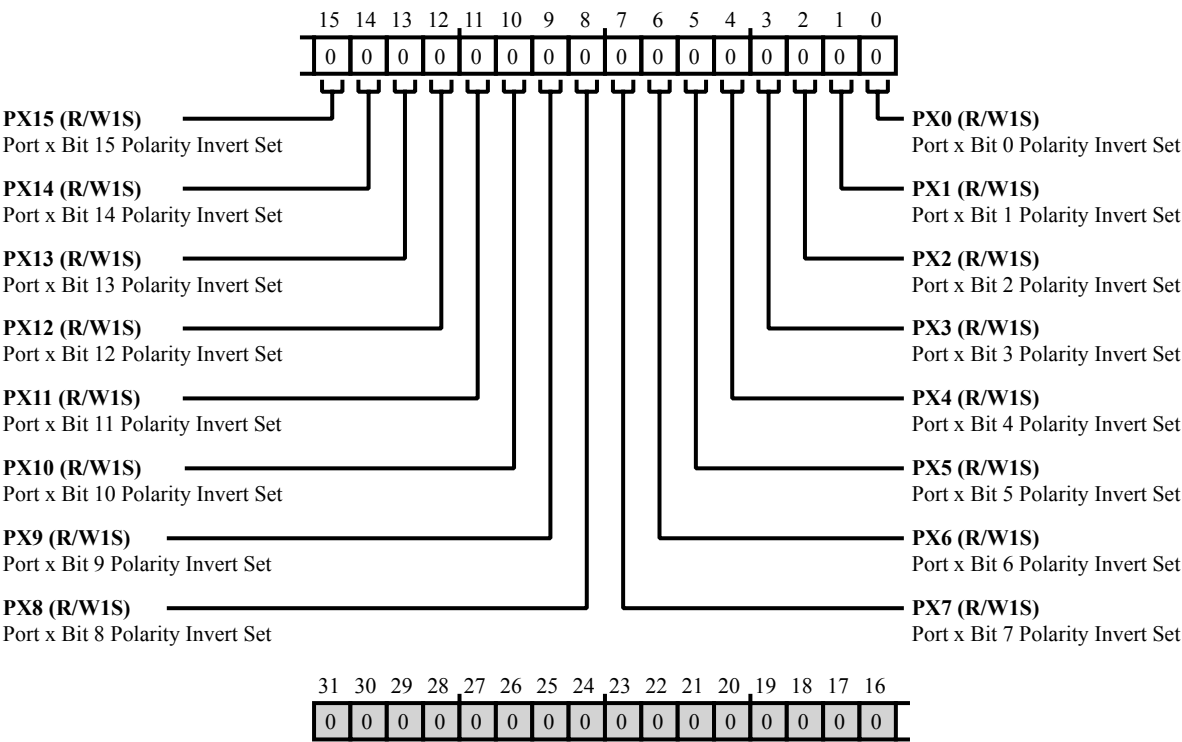


Figure 11-24: `PORT_POL_SET` Register Diagram

Table 11-29: `PORT_POL_SET` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W1S)	PX15	Port x Bit 15 Polarity Invert Set. The <code>PORT_POL_SET</code> . PX15 bit enables pin polarity inversion.
		0 No Effect
		1 Set Bit. Set to enable GPIO pin polarity invert.
14 (R/W1S)	PX14	Port x Bit 14 Polarity Invert Set. The <code>PORT_POL_SET</code> . PX14 bit enables pin polarity inversion.
		0 No Effect
		1 Set Bit. Set to enable GPIO pin polarity invert.

Table 11-29: PORT_POL_SET Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
13 (R/W1S)	PX13	Port x Bit 13 Polarity Invert Set. The PORT_POL_SET . PX13 bit enables pin polarity inversion.
		0 No Effect
		1 Set Bit. Set to enable GPIO pin polarity invert.
12 (R/W1S)	PX12	Port x Bit 12 Polarity Invert Set. The PORT_POL_SET . PX12 bit enables pin polarity inversion.
		0 No Effect
		1 Set Bit. Set to enable GPIO pin polarity invert.
11 (R/W1S)	PX11	Port x Bit 11 Polarity Invert Set. The PORT_POL_SET . PX11 bit enables pin polarity inversion.
		0 No Effect
		1 Set Bit. Set to enable GPIO pin polarity invert.
10 (R/W1S)	PX10	Port x Bit 10 Polarity Invert Set. The PORT_POL_SET . PX10 bit enables pin polarity inversion.
		0 No Effect
		1 Set Bit. Set to enable GPIO pin polarity invert.
9 (R/W1S)	PX9	Port x Bit 9 Polarity Invert Set. The PORT_POL_SET . PX9 bit enables pin polarity inversion.
		0 No Effect
		1 Set Bit. Set to enable GPIO pin polarity invert.
8 (R/W1S)	PX8	Port x Bit 8 Polarity Invert Set. The PORT_POL_SET . PX8 bit enables pin polarity inversion.
		0 No Effect
		1 Set Bit. Set to enable GPIO pin polarity invert.
7 (R/W1S)	PX7	Port x Bit 7 Polarity Invert Set. The PORT_POL_SET . PX7 bit enables pin polarity inversion.
		0 No Effect
		1 Set Bit. Set to enable GPIO pin polarity invert.

Table 11-29: PORT_POL_SET Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
6 (R/W1S)	PX6	Port x Bit 6 Polarity Invert Set. The PORT_POL_SET . PX6 bit enables pin polarity inversion.
		0 No Effect
		1 Set Bit. Set to enable GPIO pin polarity invert.
5 (R/W1S)	PX5	Port x Bit 5 Polarity Invert Set. The PORT_POL_SET . PX5 bit enables pin polarity inversion.
		0 No Effect
		1 Set Bit. Set to enable GPIO pin polarity invert.
4 (R/W1S)	PX4	Port x Bit 4 Polarity Invert Set. The PORT_POL_SET . PX4 bit enables pin polarity inversion.
		0 No Effect
		1 Set Bit. Set to enable GPIO pin polarity invert.
3 (R/W1S)	PX3	Port x Bit 3 Polarity Invert Set. The PORT_POL_SET . PX3 bit enables pin polarity inversion.
		0 No Effect
		1 Set Bit. Set to enable GPIO pin polarity invert.
2 (R/W1S)	PX2	Port x Bit 2 Polarity Invert Set. The PORT_POL_SET . PX2 bit enables pin polarity inversion.
		0 No Effect
		1 Set Bit. Set to enable GPIO pin polarity invert.
1 (R/W1S)	PX1	Port x Bit 1 Polarity Invert Set. The PORT_POL_SET . PX1 bit enables pin polarity inversion.
		0 No Effect
		1 Set Bit. Set to enable GPIO pin polarity invert.
0 (R/W1S)	PX0	Port x Bit 0 Polarity Invert Set. The PORT_POL_SET . PX0 bit enables pin polarity inversion.
		0 No Effect
		1 Set Bit. Set to enable GPIO pin polarity invert.

Port x GPIO Trigger Toggle Register

The `PORT_TRIG_TGL` register permits toggling the state of output GPIO pins in response to a trigger from the TRU for the corresponding port. Setting bits in the `PORT_TRIG_TGL` register enables triggers to toggle the state of those specific pins without impacting other pins of the port.

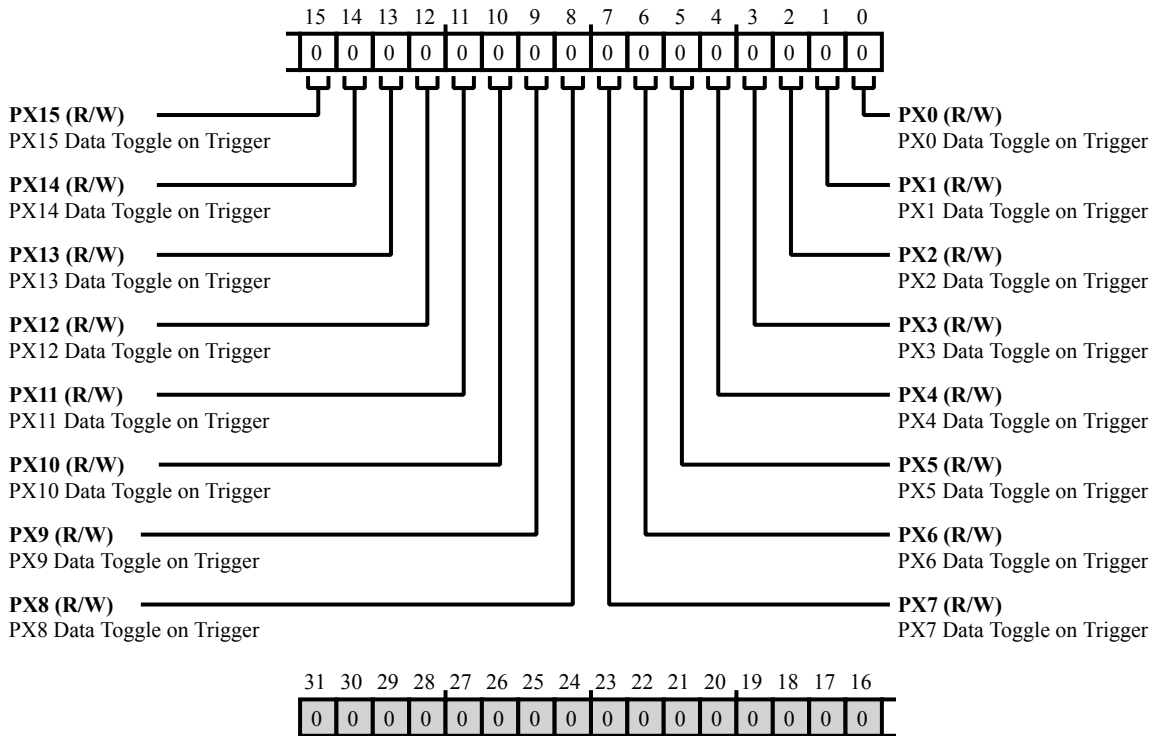


Figure 11-25: `PORT_TRIG_TGL` Register Diagram

Table 11-30: `PORT_TRIG_TGL` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W)	PX15	PX15 Data Toggle on Trigger. The <code>PORT_TRIG_TGL</code> . PX15 bit enables triggers to toggle the state of the pin.
14 (R/W)	PX14	PX14 Data Toggle on Trigger. The <code>PORT_TRIG_TGL</code> . PX14 bit enables triggers to toggle the state of the pin.
13 (R/W)	PX13	PX13 Data Toggle on Trigger. The <code>PORT_TRIG_TGL</code> . PX13 bit enables triggers to toggle the state of the pin.
12 (R/W)	PX12	PX12 Data Toggle on Trigger. The <code>PORT_TRIG_TGL</code> . PX12 bit enables triggers to toggle the state of the pin.
11 (R/W)	PX11	PX11 Data Toggle on Trigger. The <code>PORT_TRIG_TGL</code> . PX11 bit enables triggers to toggle the state of the pin.

Table 11-30: PORT_TRIG_TGL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
10 (R/W)	PX10	PX10 Data Toggle on Trigger. The PORT_TRIG_TGL.PX10 bit enables triggers to toggle the state of the pin.
9 (R/W)	PX9	PX9 Data Toggle on Trigger. The PORT_TRIG_TGL.PX9 bit enables triggers to toggle the state of the pin.
8 (R/W)	PX8	PX8 Data Toggle on Trigger. The PORT_TRIG_TGL.PX8 bit enables triggers to toggle the state of the pin.
7 (R/W)	PX7	PX7 Data Toggle on Trigger. The PORT_TRIG_TGL.PX7 bit enables triggers to toggle the state of the pin.
6 (R/W)	PX6	PX6 Data Toggle on Trigger. The PORT_TRIG_TGL.PX6 bit enables triggers to toggle the state of the pin.
5 (R/W)	PX5	PX5 Data Toggle on Trigger. The PORT_TRIG_TGL.PX5 bit enables triggers to toggle the state of the pin.
4 (R/W)	PX4	PX4 Data Toggle on Trigger. The PORT_TRIG_TGL.PX4 bit enables triggers to toggle the state of the pin.
3 (R/W)	PX3	PX3 Data Toggle on Trigger. The PORT_TRIG_TGL.PX3 bit enables triggers to toggle the state of the pin.
2 (R/W)	PX2	PX2 Data Toggle on Trigger. The PORT_TRIG_TGL.PX2 bit enables triggers to toggle the state of the pin.
1 (R/W)	PX1	PX1 Data Toggle on Trigger. The PORT_TRIG_TGL.PX1 bit enables triggers to toggle the state of the pin.
0 (R/W)	PX0	PX0 Data Toggle on Trigger. The PORT_TRIG_TGL.PX0 bit enables triggers to toggle the state of the pin.

ADSP-21560_21561_21564_21568 PINT Register Descriptions

The Pin Interrupt module (PINT) contains the following registers.

Table 11-31: ADSP-21560_21561_21564_21568 PINT Register List

Name	Description
PINT_ASSIGN	PINT Assign Register
PINT_EDGE_CLR	PINT Edge Clear Register
PINT_EDGE_SET	PINT Edge Set Register
PINT_INV_CLR	PINT Invert Clear Register

Table 11-31: ADSP-21560_21561_21564_21568 PINT Register List (Continued)

Name	Description
PINT_INV_SET	PINT Invert Set Register
PINT_LATCH	PINT Latch Register
PINT_MSK_CLR	PINT Mask Clear Register
PINT_MSK_SET	PINT Mask Set Register
PINT_PINSTATE	PINT Pin State Register
PINT_REQ	PINT Request Register

PINT Assign Register

The `PINT_ASSIGN` register controls the pin-to-interrupt request assignment in a byte-wide manner. This register consists of four control bytes that each function as a multiplexer control.

The PINT ports are subdivided into 8-bit half ports, resulting in lower and upper half 8-bit units. Using the multiplexers controlled by the `PINT_ASSIGN` register, the lower half units of eight pins can be forwarded to either byte 0 or byte 2 of either associated PINT block. The upper half units can be forwarded to either byte 1 or byte 3 of the PINT block, without further restrictions.

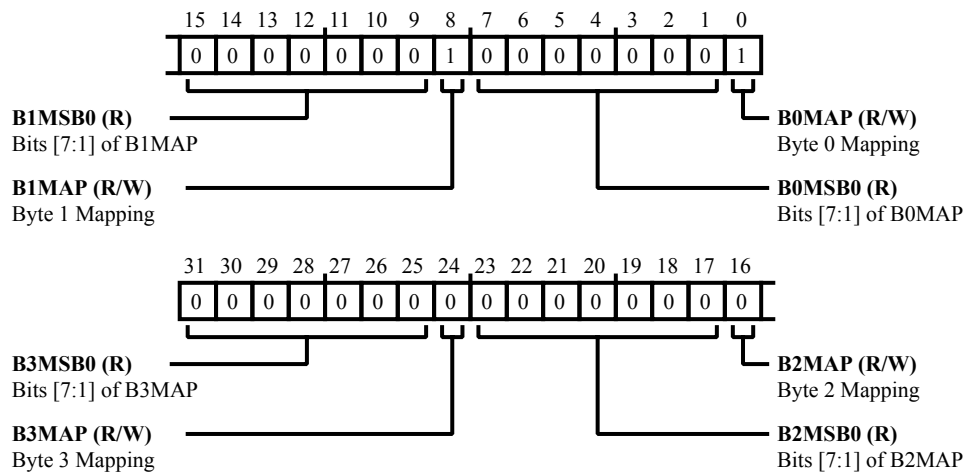


Figure 11-26: PINT_ASSIGN Register Diagram

Table 11-32: PINT_ASSIGN Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:25 (R/NW)	B3MSB0	Bits [7:1] of B3MAP.
24 (R/W)	B3MAP	Byte 3 Mapping.
		0 B3MAP_PAH. Byte 3 = PA.H
		1 B3MAP_PBH. Byte 3 = PB.H
23:17 (R/NW)	B2MSB0	Bits [7:1] of B2MAP.
16 (R/W)	B2MAP	Byte 2 Mapping.
		0 B2MAP_PAL. Byte 2 = PA.L
		1 B2MAP_PBL. Byte 2 = PB.L
15:9 (R/NW)	B1MSB0	Bits [7:1] of B1MAP.

Table 11-32: PINT_ASSIGN Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
8 (R/W)	B1MAP	Byte 1 Mapping.	
		0	B1MAP_PAH. Byte 1 = PA.H
		1	B1MAP_PBH. Byte 1 = PB.H
7:1 (R/NW)	B0MSB0	Bits [7:1] of B0MAP.	
0 (R/W)	B0MAP	Byte 0 Mapping.	
		0	B0MAP_PAL. Byte 0 = PA.L
		1	B0MAP_PBL. Byte 0 = PB.L

PINT Edge Clear Register

The `PINT_EDGE_CLR` register permits selecting level-sensitive interrupts. Writing 1 to a bit in `PINT_EDGE_CLR` enables level sensitivity for the corresponding pin interrupt.

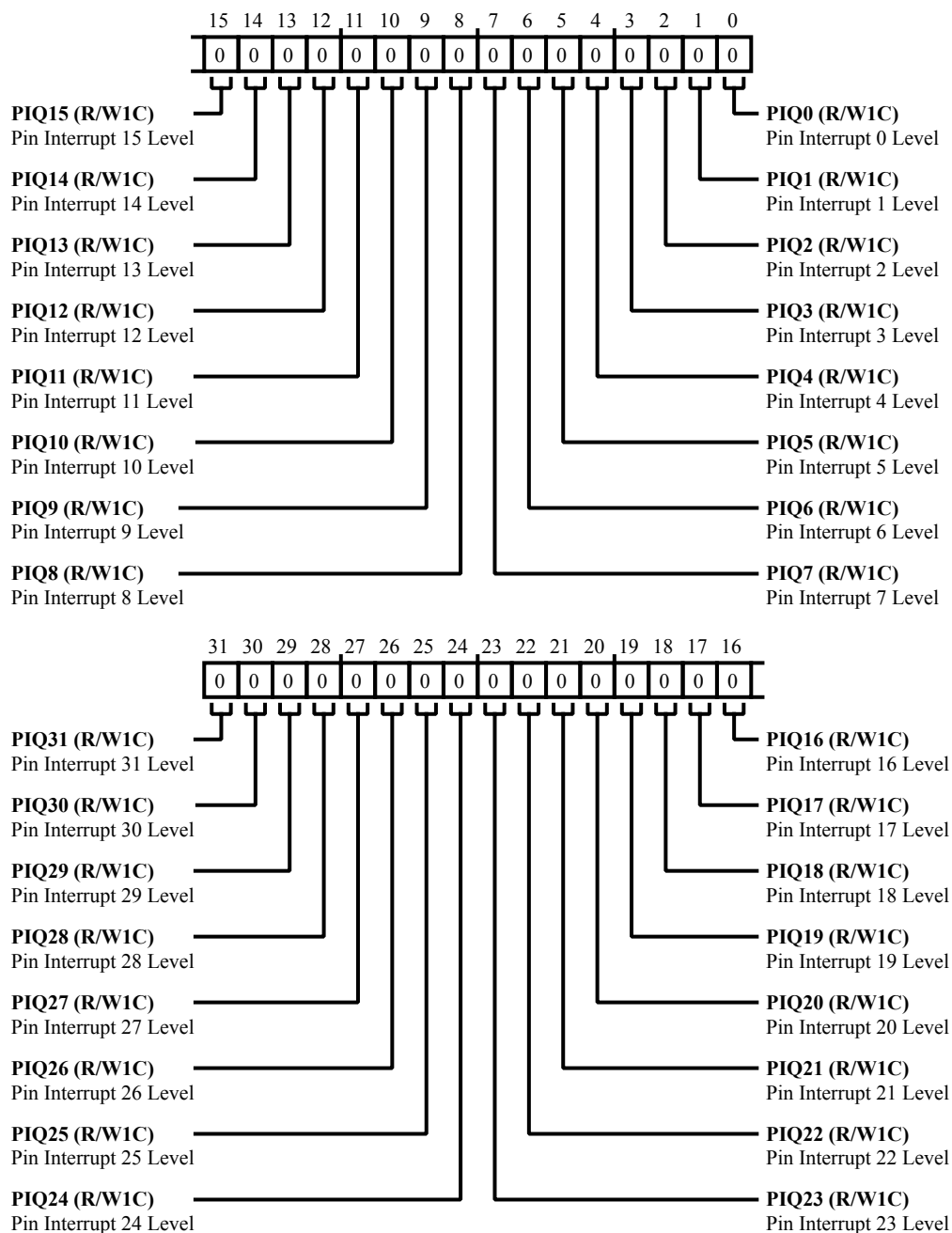


Figure 11-27: `PINT_EDGE_CLR` Register Diagram

Table 11-33: PINT_EDGE_CLR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W1C)	PIQ31	Pin Interrupt 31 Level. Set the PINT_EDGE_CLR.PIQ31 bit to enable level sensitivity.
30 (R/W1C)	PIQ30	Pin Interrupt 30 Level. Set the PINT_EDGE_CLR.PIQ30 bit to enable level sensitivity.
29 (R/W1C)	PIQ29	Pin Interrupt 29 Level. Set the PINT_EDGE_CLR.PIQ29 bit to enable level sensitivity.
28 (R/W1C)	PIQ28	Pin Interrupt 28 Level. Set the PINT_EDGE_CLR.PIQ28 bit to enable level sensitivity.
27 (R/W1C)	PIQ27	Pin Interrupt 27 Level. Set the PINT_EDGE_CLR.PIQ27 bit to enable level sensitivity.
26 (R/W1C)	PIQ26	Pin Interrupt 26 Level. Set the PINT_EDGE_CLR.PIQ26 bit to enable level sensitivity.
25 (R/W1C)	PIQ25	Pin Interrupt 25 Level. Set the PINT_EDGE_CLR.PIQ25 bit to enable level sensitivity.
24 (R/W1C)	PIQ24	Pin Interrupt 24 Level. Set the PINT_EDGE_CLR.PIQ24 bit to enable level sensitivity.
23 (R/W1C)	PIQ23	Pin Interrupt 23 Level. Set the PINT_EDGE_CLR.PIQ23 bit to enable level sensitivity.
22 (R/W1C)	PIQ22	Pin Interrupt 22 Level. Set the PINT_EDGE_CLR.PIQ22 bit to enable level sensitivity.
21 (R/W1C)	PIQ21	Pin Interrupt 21 Level. Set the PINT_EDGE_CLR.PIQ21 bit to enable level sensitivity.
20 (R/W1C)	PIQ20	Pin Interrupt 20 Level. Set the PINT_EDGE_CLR.PIQ20 bit to enable level sensitivity.
19 (R/W1C)	PIQ19	Pin Interrupt 19 Level. Set the PINT_EDGE_CLR.PIQ19 bit to enable level sensitivity.
18 (R/W1C)	PIQ18	Pin Interrupt 18 Level. Set the PINT_EDGE_CLR.PIQ18 bit to enable level sensitivity.
17 (R/W1C)	PIQ17	Pin Interrupt 17 Level. Set the PINT_EDGE_CLR.PIQ17 bit to enable level sensitivity.
16 (R/W1C)	PIQ16	Pin Interrupt 16 Level. Set the PINT_EDGE_CLR.PIQ16 bit to enable level sensitivity.

Table 11-33: PINT_EDGE_CLR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W1C)	PIQ15	Pin Interrupt 15 Level. Set the PINT_EDGE_CLR.PIQ15 bit to enable level sensitivity.
14 (R/W1C)	PIQ14	Pin Interrupt 14 Level. Set the PINT_EDGE_CLR.PIQ14 bit to enable level sensitivity.
13 (R/W1C)	PIQ13	Pin Interrupt 13 Level. Set the PINT_EDGE_CLR.PIQ13 bit to enable level sensitivity.
12 (R/W1C)	PIQ12	Pin Interrupt 12 Level. Set the PINT_EDGE_CLR.PIQ12 bit to enable level sensitivity.
11 (R/W1C)	PIQ11	Pin Interrupt 11 Level. Set the PINT_EDGE_CLR.PIQ11 bit to enable level sensitivity.
10 (R/W1C)	PIQ10	Pin Interrupt 10 Level. Set the PINT_EDGE_CLR.PIQ10 bit to enable level sensitivity.
9 (R/W1C)	PIQ9	Pin Interrupt 9 Level. Set the PINT_EDGE_CLR.PIQ9 bit to enable level sensitivity.
8 (R/W1C)	PIQ8	Pin Interrupt 8 Level. Set the PINT_EDGE_CLR.PIQ8 bit to enable level sensitivity.
7 (R/W1C)	PIQ7	Pin Interrupt 7 Level. Set the PINT_EDGE_CLR.PIQ7 bit to enable level sensitivity.
6 (R/W1C)	PIQ6	Pin Interrupt 6 Level. Set the PINT_EDGE_CLR.PIQ6 bit to enable level sensitivity.
5 (R/W1C)	PIQ5	Pin Interrupt 5 Level. Set the PINT_EDGE_CLR.PIQ5 bit to enable level sensitivity.
4 (R/W1C)	PIQ4	Pin Interrupt 4 Level. Set the PINT_EDGE_CLR.PIQ4 bit to enable level sensitivity.
3 (R/W1C)	PIQ3	Pin Interrupt 3 Level. Set the PINT_EDGE_CLR.PIQ3 bit to enable level sensitivity.
2 (R/W1C)	PIQ2	Pin Interrupt 2 Level. Set the PINT_EDGE_CLR.PIQ2 bit to enable level sensitivity.
1 (R/W1C)	PIQ1	Pin Interrupt 1 Level. Set the PINT_EDGE_CLR.PIQ1 bit to enable level sensitivity.
0 (R/W1C)	PIQ0	Pin Interrupt 0 Level. Set the PINT_EDGE_CLR.PIQ0 bit to enable level sensitivity.

PINT Edge Set Register

The `PINT_EDGE_SET` register permits selecting edge-sensitive interrupts. Writing 1 to a bit in `PINT_EDGE_SET` enables edge sensitivity for the corresponding pin interrupt.

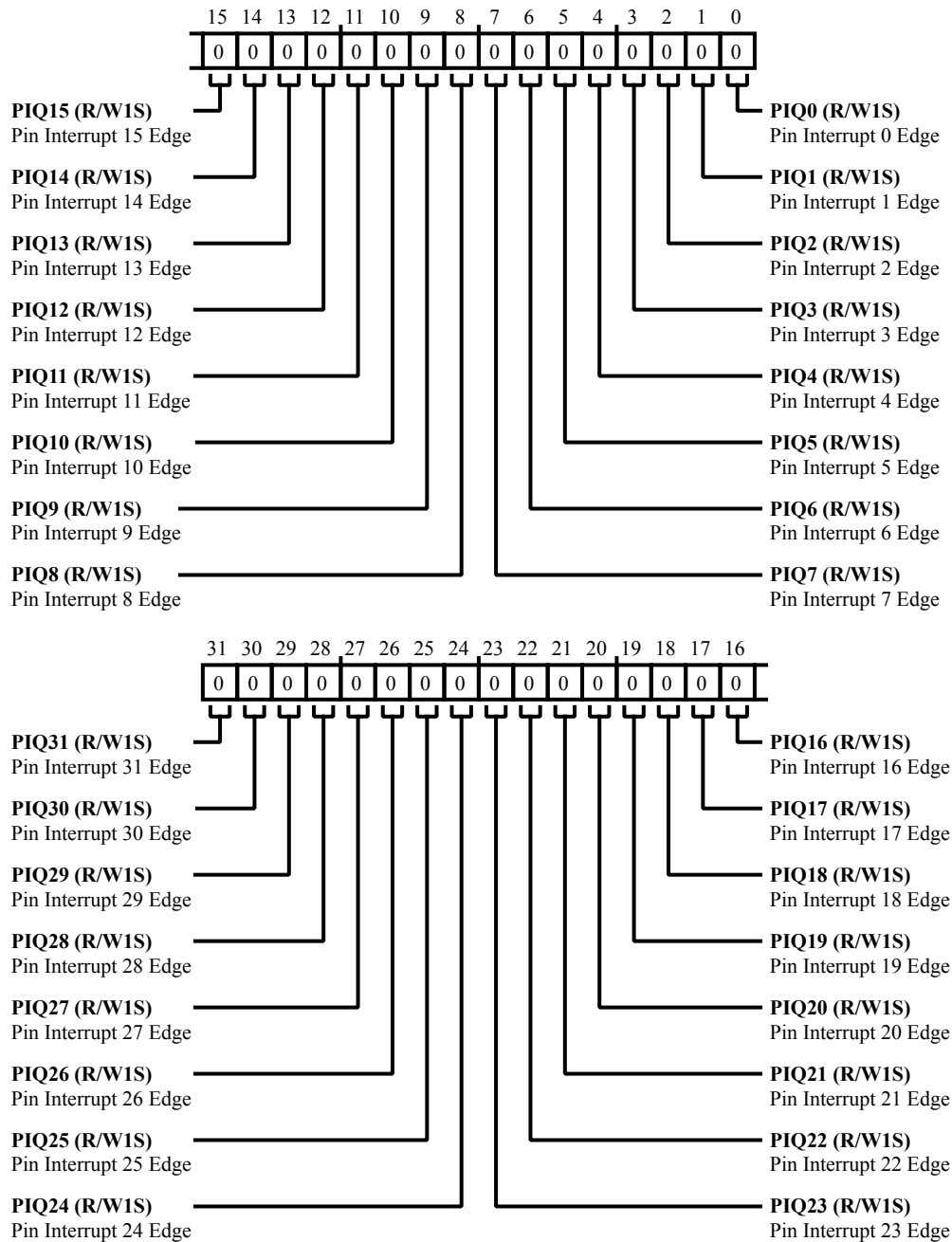


Figure 11-28: `PINT_EDGE_SET` Register Diagram

Table 11-34: PINT_EDGE_SET Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W1S)	PIQ31	Pin Interrupt 31 Edge. Set the PINT_EDGE_SET.PIQ31 bit to enable edge sensitivity.
30 (R/W1S)	PIQ30	Pin Interrupt 30 Edge. Set the PINT_EDGE_SET.PIQ30 bit to enable edge sensitivity.
29 (R/W1S)	PIQ29	Pin Interrupt 29 Edge. Set the PINT_EDGE_SET.PIQ29 bit to enable edge sensitivity.
28 (R/W1S)	PIQ28	Pin Interrupt 28 Edge. Set the PINT_EDGE_SET.PIQ28 bit to enable edge sensitivity.
27 (R/W1S)	PIQ27	Pin Interrupt 27 Edge. Set the PINT_EDGE_SET.PIQ27 bit to enable edge sensitivity.
26 (R/W1S)	PIQ26	Pin Interrupt 26 Edge. Set the PINT_EDGE_SET.PIQ26 bit to enable edge sensitivity.
25 (R/W1S)	PIQ25	Pin Interrupt 25 Edge. Set the PINT_EDGE_SET.PIQ25 bit to enable edge sensitivity.
24 (R/W1S)	PIQ24	Pin Interrupt 24 Edge. Set the PINT_EDGE_SET.PIQ24 bit to enable edge sensitivity.
23 (R/W1S)	PIQ23	Pin Interrupt 23 Edge. Set the PINT_EDGE_SET.PIQ23 bit to enable edge sensitivity.
22 (R/W1S)	PIQ22	Pin Interrupt 22 Edge. Set the PINT_EDGE_SET.PIQ22 bit to enable edge sensitivity.
21 (R/W1S)	PIQ21	Pin Interrupt 21 Edge. Set the PINT_EDGE_SET.PIQ21 bit to enable edge sensitivity.
20 (R/W1S)	PIQ20	Pin Interrupt 20 Edge. Set the PINT_EDGE_SET.PIQ20 bit to enable edge sensitivity.
19 (R/W1S)	PIQ19	Pin Interrupt 19 Edge. Set the PINT_EDGE_SET.PIQ19 bit to enable edge sensitivity.
18 (R/W1S)	PIQ18	Pin Interrupt 18 Edge. Set the PINT_EDGE_SET.PIQ18 bit to enable edge sensitivity.
17 (R/W1S)	PIQ17	Pin Interrupt 17 Edge. Set the PINT_EDGE_SET.PIQ17 bit to enable edge sensitivity.
16 (R/W1S)	PIQ16	Pin Interrupt 16 Edge. Set the PINT_EDGE_SET.PIQ16 bit to enable edge sensitivity.

Table 11-34: PINT_EDGE_SET Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W1S)	PIQ15	Pin Interrupt 15 Edge. Set the PINT_EDGE_SET.PIQ15 bit to enable edge sensitivity.
14 (R/W1S)	PIQ14	Pin Interrupt 14 Edge. Set the PINT_EDGE_SET.PIQ14 bit to enable edge sensitivity.
13 (R/W1S)	PIQ13	Pin Interrupt 13 Edge. Set the PINT_EDGE_SET.PIQ13 bit to enable edge sensitivity.
12 (R/W1S)	PIQ12	Pin Interrupt 12 Edge. Set the PINT_EDGE_SET.PIQ12 bit to enable edge sensitivity.
11 (R/W1S)	PIQ11	Pin Interrupt 11 Edge. Set the PINT_EDGE_SET.PIQ11 bit to enable edge sensitivity.
10 (R/W1S)	PIQ10	Pin Interrupt 10 Edge. Set the PINT_EDGE_SET.PIQ10 bit to enable edge sensitivity.
9 (R/W1S)	PIQ9	Pin Interrupt 9 Edge. Set the PINT_EDGE_SET.PIQ9 bit to enable edge sensitivity.
8 (R/W1S)	PIQ8	Pin Interrupt 8 Edge. Set the PINT_EDGE_SET.PIQ8 bit to enable edge sensitivity.
7 (R/W1S)	PIQ7	Pin Interrupt 7 Edge. Set the PINT_EDGE_SET.PIQ7 bit to enable edge sensitivity.
6 (R/W1S)	PIQ6	Pin Interrupt 6 Edge. Set the PINT_EDGE_SET.PIQ6 bit to enable edge sensitivity.
5 (R/W1S)	PIQ5	Pin Interrupt 5 Edge. Set the PINT_EDGE_SET.PIQ5 bit to enable edge sensitivity.
4 (R/W1S)	PIQ4	Pin Interrupt 4 Edge. Set the PINT_EDGE_SET.PIQ4 bit to enable edge sensitivity.
3 (R/W1S)	PIQ3	Pin Interrupt 3 Edge. Set the PINT_EDGE_SET.PIQ3 bit to enable edge sensitivity.
2 (R/W1S)	PIQ2	Pin Interrupt 2 Edge. Set the PINT_EDGE_SET.PIQ2 bit to enable edge sensitivity.
1 (R/W1S)	PIQ1	Pin Interrupt 1 Edge. Set the PINT_EDGE_SET.PIQ1 bit to enable edge sensitivity.
0 (R/W1S)	PIQ0	Pin Interrupt 0 Edge. Set the PINT_EDGE_SET.PIQ0 bit to enable edge sensitivity.

PINT Invert Clear Register

The `PINT_INV_CLR` register disables inverting input polarity. Writing 1 to a bit in `PINT_INV_CLR` disables an inverter for input on the corresponding pin.

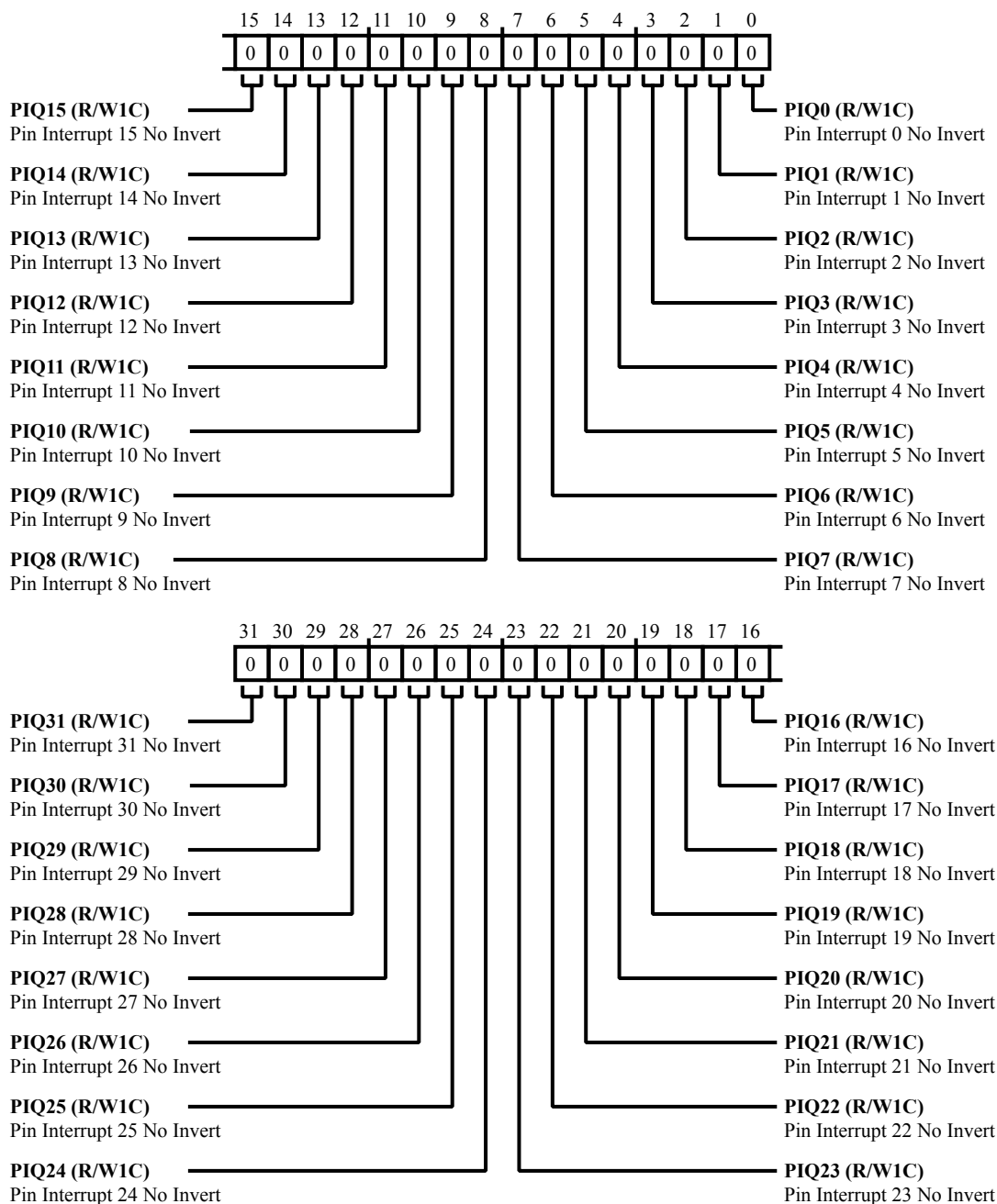


Figure 11-29: `PINT_INV_CLR` Register Diagram

Table 11-35: PINT_INV_CLR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W1C)	PIQ31	Pin Interrupt 31 No Invert. Set the PINT_INV_CLR.PIQ31 bit to disable inverted input.
30 (R/W1C)	PIQ30	Pin Interrupt 30 No Invert. Set the PINT_INV_CLR.PIQ30 bit to disable inverted input.
29 (R/W1C)	PIQ29	Pin Interrupt 29 No Invert. Set the PINT_INV_CLR.PIQ29 bit to disable inverted input.
28 (R/W1C)	PIQ28	Pin Interrupt 28 No Invert. Set the PINT_INV_CLR.PIQ28 bit to disable inverted input.
27 (R/W1C)	PIQ27	Pin Interrupt 27 No Invert. Set the PINT_INV_CLR.PIQ27 bit to disable inverted input.
26 (R/W1C)	PIQ26	Pin Interrupt 26 No Invert. Set the PINT_INV_CLR.PIQ26 bit to disable inverted input.
25 (R/W1C)	PIQ25	Pin Interrupt 25 No Invert. Set the PINT_INV_CLR.PIQ25 bit to disable inverted input.
24 (R/W1C)	PIQ24	Pin Interrupt 24 No Invert. Set the PINT_INV_CLR.PIQ24 bit to disable inverted input.
23 (R/W1C)	PIQ23	Pin Interrupt 23 No Invert. Set the PINT_INV_CLR.PIQ23 bit to disable inverted input.
22 (R/W1C)	PIQ22	Pin Interrupt 22 No Invert. Set the PINT_INV_CLR.PIQ22 bit to disable inverted input.
21 (R/W1C)	PIQ21	Pin Interrupt 21 No Invert. Set the PINT_INV_CLR.PIQ21 bit to disable inverted input.
20 (R/W1C)	PIQ20	Pin Interrupt 20 No Invert. Set the PINT_INV_CLR.PIQ20 bit to disable inverted input.
19 (R/W1C)	PIQ19	Pin Interrupt 19 No Invert. Set the PINT_INV_CLR.PIQ19 bit to disable inverted input.
18 (R/W1C)	PIQ18	Pin Interrupt 18 No Invert. Set the PINT_INV_CLR.PIQ18 bit to disable inverted input.
17 (R/W1C)	PIQ17	Pin Interrupt 17 No Invert. Set the PINT_INV_CLR.PIQ17 bit to disable inverted input.
16 (R/W1C)	PIQ16	Pin Interrupt 16 No Invert. Set the PINT_INV_CLR.PIQ16 bit to disable inverted input.

Table 11-35: PINT_INV_CLR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W1C)	PIQ15	Pin Interrupt 15 No Invert. Set the PINT_INV_CLR.PIQ15 bit to disable inverted input.
14 (R/W1C)	PIQ14	Pin Interrupt 14 No Invert. Set the PINT_INV_CLR.PIQ14 bit to disable inverted input.
13 (R/W1C)	PIQ13	Pin Interrupt 13 No Invert. Set the PINT_INV_CLR.PIQ13 bit to disable inverted input.
12 (R/W1C)	PIQ12	Pin Interrupt 12 No Invert. Set the PINT_INV_CLR.PIQ12 bit to disable inverted input.
11 (R/W1C)	PIQ11	Pin Interrupt 11 No Invert. Set the PINT_INV_CLR.PIQ11 bit to disable inverted input.
10 (R/W1C)	PIQ10	Pin Interrupt 10 No Invert. Set the PINT_INV_CLR.PIQ10 bit to disable inverted input.
9 (R/W1C)	PIQ9	Pin Interrupt 9 No Invert. Set the PINT_INV_CLR.PIQ9 bit to disable inverted input.
8 (R/W1C)	PIQ8	Pin Interrupt 8 No Invert. Set the PINT_INV_CLR.PIQ8 bit to disable inverted input.
7 (R/W1C)	PIQ7	Pin Interrupt 7 No Invert. Set the PINT_INV_CLR.PIQ7 bit to disable inverted input.
6 (R/W1C)	PIQ6	Pin Interrupt 6 No Invert. Set the PINT_INV_CLR.PIQ6 bit to disable inverted input.
5 (R/W1C)	PIQ5	Pin Interrupt 5 No Invert. Set the PINT_INV_CLR.PIQ5 bit to disable inverted input.
4 (R/W1C)	PIQ4	Pin Interrupt 4 No Invert. Set the PINT_INV_CLR.PIQ4 bit to disable inverted input.
3 (R/W1C)	PIQ3	Pin Interrupt 3 No Invert. Set the PINT_INV_CLR.PIQ3 bit to disable inverted input.
2 (R/W1C)	PIQ2	Pin Interrupt 2 No Invert. Set the PINT_INV_CLR.PIQ2 bit to disable inverted input.
1 (R/W1C)	PIQ1	Pin Interrupt 1 No Invert. Set the PINT_INV_CLR.PIQ1 bit to disable inverted input.
0 (R/W1C)	PIQ0	Pin Interrupt 0 No Invert. Set the PINT_INV_CLR.PIQ0 bit to disable inverted input.

PINT Invert Set Register

The `PINT_INV_SET` register enables inverting input polarity. Writing 1 to a bit in `PINT_INV_SET` enables an inverter for input on the corresponding pin.

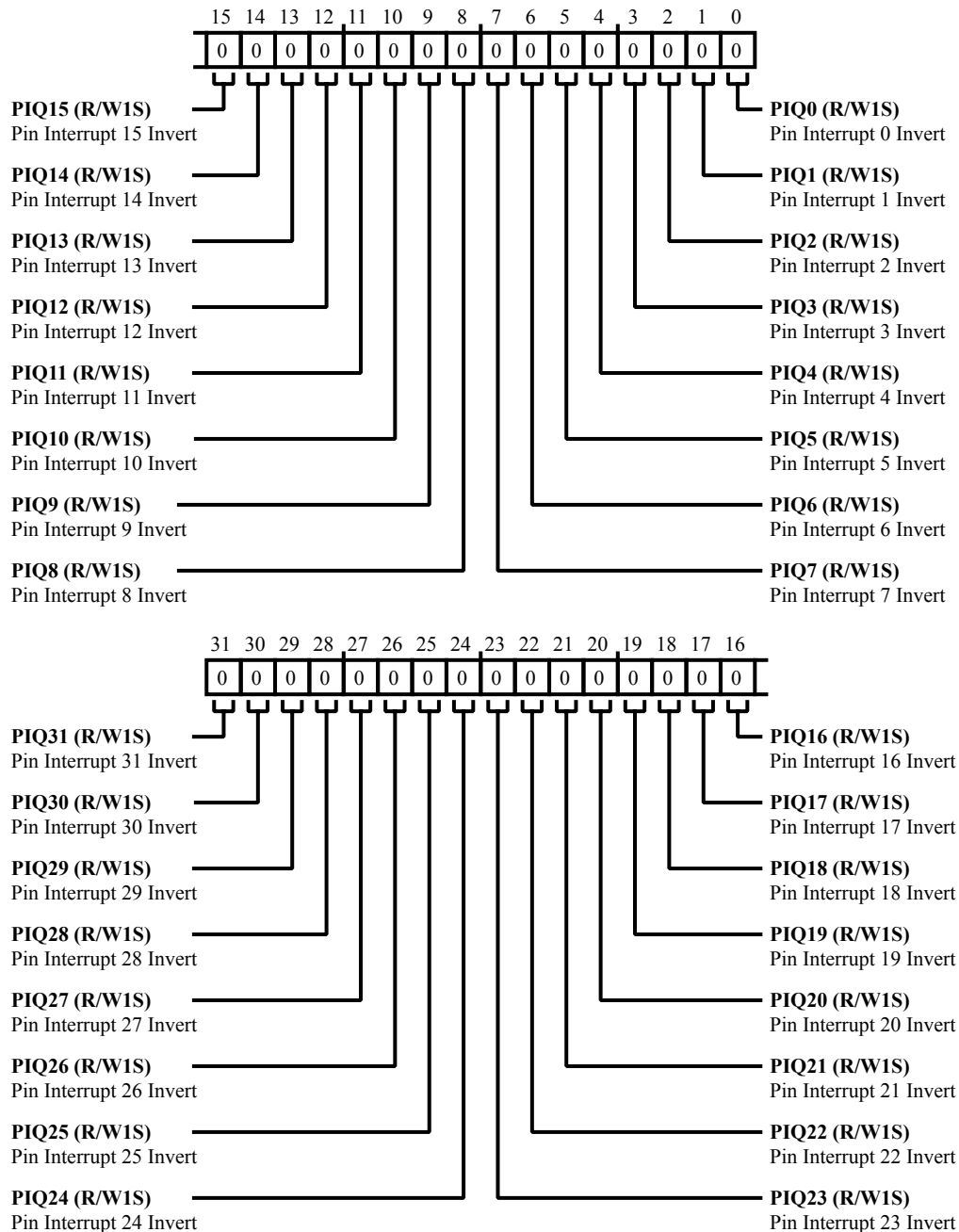


Figure 11-30: PINT_INV_SET Register Diagram

Table 11-36: PINT_INV_SET Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W1S)	PIQ31	Pin Interrupt 31 Invert. Set the PINT_INV_SET.PIQ31 bit to enable inverted input.
30 (R/W1S)	PIQ30	Pin Interrupt 30 Invert. Set the PINT_INV_SET.PIQ30 bit to enable inverted input.
29 (R/W1S)	PIQ29	Pin Interrupt 29 Invert. Set the PINT_INV_SET.PIQ29 bit to enable inverted input.
28 (R/W1S)	PIQ28	Pin Interrupt 28 Invert. Set the PINT_INV_SET.PIQ28 bit to enable inverted input.
27 (R/W1S)	PIQ27	Pin Interrupt 27 Invert. Set the PINT_INV_SET.PIQ27 bit to enable inverted input.
26 (R/W1S)	PIQ26	Pin Interrupt 26 Invert. Set the PINT_INV_SET.PIQ26 bit to enable inverted input.
25 (R/W1S)	PIQ25	Pin Interrupt 25 Invert. Set the PINT_INV_SET.PIQ25 bit to enable inverted input.
24 (R/W1S)	PIQ24	Pin Interrupt 24 Invert. Set the PINT_INV_SET.PIQ24 bit to enable inverted input.
23 (R/W1S)	PIQ23	Pin Interrupt 23 Invert. Set the PINT_INV_SET.PIQ23 bit to enable inverted input.
22 (R/W1S)	PIQ22	Pin Interrupt 22 Invert. Set the PINT_INV_SET.PIQ22 bit to enable inverted input.
21 (R/W1S)	PIQ21	Pin Interrupt 21 Invert. Set the PINT_INV_SET.PIQ21 bit to enable inverted input.
20 (R/W1S)	PIQ20	Pin Interrupt 20 Invert. Set the PINT_INV_SET.PIQ20 bit to enable inverted input.
19 (R/W1S)	PIQ19	Pin Interrupt 19 Invert. Set the PINT_INV_SET.PIQ19 bit to enable inverted input.
18 (R/W1S)	PIQ18	Pin Interrupt 18 Invert. Set the PINT_INV_SET.PIQ18 bit to enable inverted input.
17 (R/W1S)	PIQ17	Pin Interrupt 17 Invert. Set the PINT_INV_SET.PIQ17 bit to enable inverted input.
16 (R/W1S)	PIQ16	Pin Interrupt 16 Invert. Set the PINT_INV_SET.PIQ16 bit to enable inverted input.

Table 11-36: PINT_INV_SET Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W1S)	PIQ15	Pin Interrupt 15 Invert. Set the PINT_INV_SET.PIQ15 bit to enable inverted input.
14 (R/W1S)	PIQ14	Pin Interrupt 14 Invert. Set the PINT_INV_SET.PIQ14 bit to enable inverted input.
13 (R/W1S)	PIQ13	Pin Interrupt 13 Invert. Set the PINT_INV_SET.PIQ13 bit to enable inverted input.
12 (R/W1S)	PIQ12	Pin Interrupt 12 Invert. Set the PINT_INV_SET.PIQ12 bit to enable inverted input.
11 (R/W1S)	PIQ11	Pin Interrupt 11 Invert. Set the PINT_INV_SET.PIQ11 bit to enable inverted input.
10 (R/W1S)	PIQ10	Pin Interrupt 10 Invert. Set the PINT_INV_SET.PIQ10 bit to enable inverted input.
9 (R/W1S)	PIQ9	Pin Interrupt 9 Invert. Set the PINT_INV_SET.PIQ9 bit to enable inverted input.
8 (R/W1S)	PIQ8	Pin Interrupt 8 Invert. Set the PINT_INV_SET.PIQ8 bit to enable inverted input.
7 (R/W1S)	PIQ7	Pin Interrupt 7 Invert. Set the PINT_INV_SET.PIQ7 bit to enable inverted input.
6 (R/W1S)	PIQ6	Pin Interrupt 6 Invert. Set the PINT_INV_SET.PIQ6 bit to enable inverted input.
5 (R/W1S)	PIQ5	Pin Interrupt 5 Invert. Set the PINT_INV_SET.PIQ5 bit to enable inverted input.
4 (R/W1S)	PIQ4	Pin Interrupt 4 Invert. Set the PINT_INV_SET.PIQ4 bit to enable inverted input.
3 (R/W1S)	PIQ3	Pin Interrupt 3 Invert. Set the PINT_INV_SET.PIQ3 bit to enable inverted input.
2 (R/W1S)	PIQ2	Pin Interrupt 2 Invert. Set the PINT_INV_SET.PIQ2 bit to enable inverted input.
1 (R/W1S)	PIQ1	Pin Interrupt 1 Invert. Set the PINT_INV_SET.PIQ1 bit to enable inverted input.
0 (R/W1S)	PIQ0	Pin Interrupt 0 Invert. Set the PINT_INV_SET.PIQ0 bit to enable inverted input.

PINT Latch Register

The `PINT_LATCH` register indicates the interrupt latch status for pin interrupts. When set, an interrupt request is latched. When cleared, there is no interrupt request latched.

Both the `PINT_REQ` and `PINT_LATCH` registers indicate whether an interrupt request is latched on the respective pin. The `PINT_LATCH` register is a latch that operates regardless of the interrupt masks. Bits of the `PINT_REQ` register depend on the mask register. The `PINT_REQ` register is a logical AND of the `PINT_LATCH` register and the interrupt mask.

Having two separate registers here enables the user to interrogate certain pins in polling mode while others work in interrupt mode. The `PINT_LATCH` registers can be used for edge detection or pin activity detection.

Both registers have W1C behavior. Writing a 1 to either register clears the respective bits in both registers. For interrupt operation, the user may prefer to W1C the `PINT_REQ` register (address still loaded in Px pointer). In polling mode, it might be cleaner to W1C the `PINT_LATCH` register.

Whether in edge-sensitive mode or level-sensitive mode, `PINT_LATCH` bits are never cleared by hardware except at system reset. Even in level-sensitive mode, the `PINT_LATCH` register functions as latch.

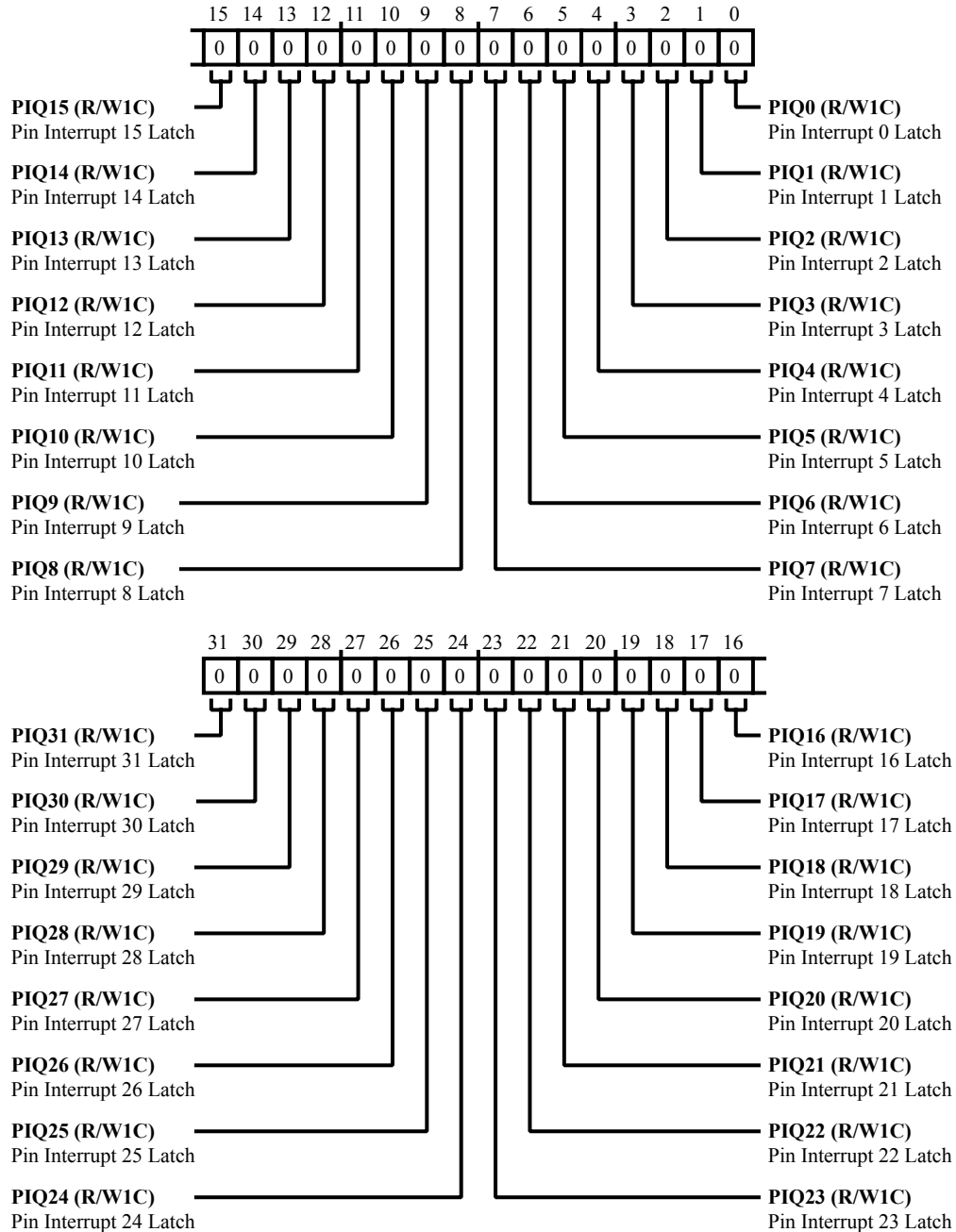


Figure 11-31: PINT_LATCH Register Diagram

Table 11-37: PINT_LATCH Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W1C)	PIQ31	Pin Interrupt 31 Latch. If the <code>PINT_LATCH.PIQ31</code> bit is set, the request is latched.

Table 11-37: PINT_LATCH Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
30 (R/W1C)	PIQ30	Pin Interrupt 30 Latch. If the PINT_LATCH.PIQ30 bit is set, the request is latched.
29 (R/W1C)	PIQ29	Pin Interrupt 29 Latch. If the PINT_LATCH.PIQ29 bit is set, the request is latched.
28 (R/W1C)	PIQ28	Pin Interrupt 28 Latch. If the PINT_LATCH.PIQ28 bit is set, the request is latched.
27 (R/W1C)	PIQ27	Pin Interrupt 27 Latch. If the PINT_LATCH.PIQ27 bit is set, the request is latched.
26 (R/W1C)	PIQ26	Pin Interrupt 26 Latch. If the PINT_LATCH.PIQ26 bit is set, the request is latched.
25 (R/W1C)	PIQ25	Pin Interrupt 25 Latch. If the PINT_LATCH.PIQ25 bit is set, the request is latched.
24 (R/W1C)	PIQ24	Pin Interrupt 24 Latch. If the PINT_LATCH.PIQ24 bit is set, the request is latched.
23 (R/W1C)	PIQ23	Pin Interrupt 23 Latch. If the PINT_LATCH.PIQ23 bit is set, the request is latched.
22 (R/W1C)	PIQ22	Pin Interrupt 22 Latch. If the PINT_LATCH.PIQ22 bit is set, the request is latched.
21 (R/W1C)	PIQ21	Pin Interrupt 21 Latch. If the PINT_LATCH.PIQ21 bit is set, the request is latched.
20 (R/W1C)	PIQ20	Pin Interrupt 20 Latch. If the PINT_LATCH.PIQ20 bit is set, the request is latched.
19 (R/W1C)	PIQ19	Pin Interrupt 19 Latch. If the PINT_LATCH.PIQ19 bit is set, the request is latched.
18 (R/W1C)	PIQ18	Pin Interrupt 18 Latch. If the PINT_LATCH.PIQ18 bit is set, the request is latched.
17 (R/W1C)	PIQ17	Pin Interrupt 17 Latch. If the PINT_LATCH.PIQ17 bit is set, the request is latched.
16 (R/W1C)	PIQ16	Pin Interrupt 16 Latch. If the PINT_LATCH.PIQ16 bit is set, the request is latched.
15 (R/W1C)	PIQ15	Pin Interrupt 15 Latch. If the PINT_LATCH.PIQ15 bit is set, the request is latched.

Table 11-37: PINT_LATCH Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
14 (R/W1C)	PIQ14	Pin Interrupt 14 Latch. If the PINT_LATCH.PIQ14 bit is set, the request is latched.
13 (R/W1C)	PIQ13	Pin Interrupt 13 Latch. If the PINT_LATCH.PIQ13 bit is set, the request is latched.
12 (R/W1C)	PIQ12	Pin Interrupt 12 Latch. If the PINT_LATCH.PIQ12 bit is set, the request is latched.
11 (R/W1C)	PIQ11	Pin Interrupt 11 Latch. If the PINT_LATCH.PIQ11 bit is set, the request is latched.
10 (R/W1C)	PIQ10	Pin Interrupt 10 Latch. If the PINT_LATCH.PIQ10 bit is set, the request is latched.
9 (R/W1C)	PIQ9	Pin Interrupt 9 Latch. If the PINT_LATCH.PIQ9 bit is set, the request is latched.
8 (R/W1C)	PIQ8	Pin Interrupt 8 Latch. If the PINT_LATCH.PIQ8 bit is set, the request is latched.
7 (R/W1C)	PIQ7	Pin Interrupt 7 Latch. If the PINT_LATCH.PIQ7 bit is set, the request is latched.
6 (R/W1C)	PIQ6	Pin Interrupt 6 Latch. If the PINT_LATCH.PIQ6 bit is set, the request is latched.
5 (R/W1C)	PIQ5	Pin Interrupt 5 Latch. If the PINT_LATCH.PIQ5 bit is set, the request is latched.
4 (R/W1C)	PIQ4	Pin Interrupt 4 Latch. If the PINT_LATCH.PIQ4 bit is set, the request is latched.
3 (R/W1C)	PIQ3	Pin Interrupt 3 Latch. If the PINT_LATCH.PIQ3 bit is set, the request is latched.
2 (R/W1C)	PIQ2	Pin Interrupt 2 Latch. If the PINT_LATCH.PIQ2 bit is set, the request is latched.
1 (R/W1C)	PIQ1	Pin Interrupt 1 Latch. If the PINT_LATCH.PIQ1 bit is set, the request is latched.
0 (R/W1C)	PIQ0	Pin Interrupt 0 Latch. If the PINT_LATCH.PIQ0 bit is set, the request is latched.

PINT Mask Clear Register

The `PINT_MSK_CLR` register permits masking (disabling) of interrupt requests. Writing 1 to a bit in `PINT_MSK_CLR` masks the corresponding pin interrupt.

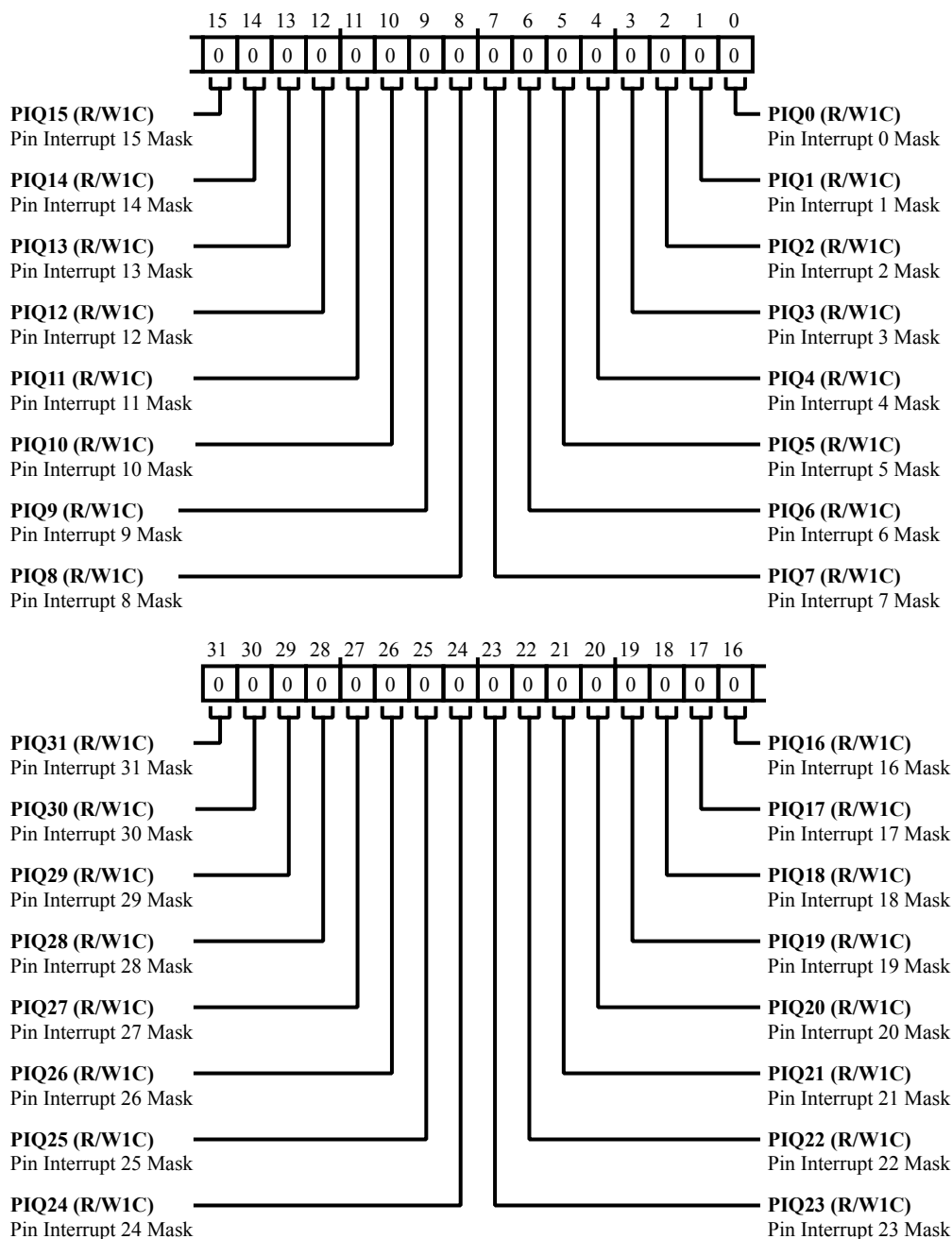


Figure 11-32: PINT_MSK_CLR Register Diagram

Table 11-38: PINT_MSK_CLR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W1C)	PIQ31	Pin Interrupt 31 Mask. Set the PINT_MSK_CLR.PIQ31 bit to disable the interrupt.
30 (R/W1C)	PIQ30	Pin Interrupt 30 Mask. Set the PINT_MSK_CLR.PIQ30 bit to disable the interrupt.
29 (R/W1C)	PIQ29	Pin Interrupt 29 Mask. Set the PINT_MSK_CLR.PIQ29 bit to disable the interrupt.
28 (R/W1C)	PIQ28	Pin Interrupt 28 Mask. Set the PINT_MSK_CLR.PIQ28 bit to disable the interrupt.
27 (R/W1C)	PIQ27	Pin Interrupt 27 Mask. Set the PINT_MSK_CLR.PIQ27 bit to disable the interrupt.
26 (R/W1C)	PIQ26	Pin Interrupt 26 Mask. Set the PINT_MSK_CLR.PIQ26 bit to disable the interrupt.
25 (R/W1C)	PIQ25	Pin Interrupt 25 Mask. Set the PINT_MSK_CLR.PIQ25 bit to disable the interrupt.
24 (R/W1C)	PIQ24	Pin Interrupt 24 Mask. Set the PINT_MSK_CLR.PIQ24 bit to disable the interrupt.
23 (R/W1C)	PIQ23	Pin Interrupt 23 Mask. Set the PINT_MSK_CLR.PIQ23 bit to disable the interrupt.
22 (R/W1C)	PIQ22	Pin Interrupt 22 Mask. Set the PINT_MSK_CLR.PIQ22 bit to disable the interrupt.
21 (R/W1C)	PIQ21	Pin Interrupt 21 Mask. Set the PINT_MSK_CLR.PIQ21 bit to disable the interrupt.
20 (R/W1C)	PIQ20	Pin Interrupt 20 Mask. Set the PINT_MSK_CLR.PIQ20 bit to disable the interrupt.
19 (R/W1C)	PIQ19	Pin Interrupt 19 Mask. Set the PINT_MSK_CLR.PIQ19 bit to disable the interrupt.
18 (R/W1C)	PIQ18	Pin Interrupt 18 Mask. Set the PINT_MSK_CLR.PIQ18 bit to disable the interrupt.
17 (R/W1C)	PIQ17	Pin Interrupt 17 Mask. Set the PINT_MSK_CLR.PIQ17 bit to disable the interrupt.
16 (R/W1C)	PIQ16	Pin Interrupt 16 Mask. Set the PINT_MSK_CLR.PIQ16 bit to disable the interrupt.

Table 11-38: PINT_MSK_CLR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W1C)	PIQ15	Pin Interrupt 15 Mask. Set the PINT_MSK_CLR.PIQ15 bit to disable the interrupt.
14 (R/W1C)	PIQ14	Pin Interrupt 14 Mask. Set the PINT_MSK_CLR.PIQ14 bit to disable the interrupt.
13 (R/W1C)	PIQ13	Pin Interrupt 13 Mask. Set the PINT_MSK_CLR.PIQ13 bit to disable the interrupt.
12 (R/W1C)	PIQ12	Pin Interrupt 12 Mask. Set the PINT_MSK_CLR.PIQ12 bit to disable the interrupt.
11 (R/W1C)	PIQ11	Pin Interrupt 11 Mask. Set the PINT_MSK_CLR.PIQ11 bit to disable the interrupt.
10 (R/W1C)	PIQ10	Pin Interrupt 10 Mask. Set the PINT_MSK_CLR.PIQ10 bit to disable the interrupt.
9 (R/W1C)	PIQ9	Pin Interrupt 9 Mask. Set the PINT_MSK_CLR.PIQ9 bit to disable the interrupt.
8 (R/W1C)	PIQ8	Pin Interrupt 8 Mask. Set the PINT_MSK_CLR.PIQ8 bit to disable the interrupt.
7 (R/W1C)	PIQ7	Pin Interrupt 7 Mask. Set the PINT_MSK_CLR.PIQ7 bit to disable the interrupt.
6 (R/W1C)	PIQ6	Pin Interrupt 6 Mask. Set the PINT_MSK_CLR.PIQ6 bit to disable the interrupt.
5 (R/W1C)	PIQ5	Pin Interrupt 5 Mask. Set the PINT_MSK_CLR.PIQ5 bit to disable the interrupt.
4 (R/W1C)	PIQ4	Pin Interrupt 4 Mask. Set the PINT_MSK_CLR.PIQ4 bit to disable the interrupt.
3 (R/W1C)	PIQ3	Pin Interrupt 3 Mask. Set the PINT_MSK_CLR.PIQ3 bit to disable the interrupt.
2 (R/W1C)	PIQ2	Pin Interrupt 2 Mask. Set the PINT_MSK_CLR.PIQ2 bit to disable the interrupt.
1 (R/W1C)	PIQ1	Pin Interrupt 1 Mask. Set the PINT_MSK_CLR.PIQ1 bit to disable the interrupt.
0 (R/W1C)	PIQ0	Pin Interrupt 0 Mask. Set the PINT_MSK_CLR.PIQ0 bit to disable the interrupt.

PINT Mask Set Register

The `PINT_MSK_SET` register permits unmasking (enabling) of interrupt requests. Writing 1 to a bit in `PINT_MSK_SET` unmask the corresponding pin interrupt.

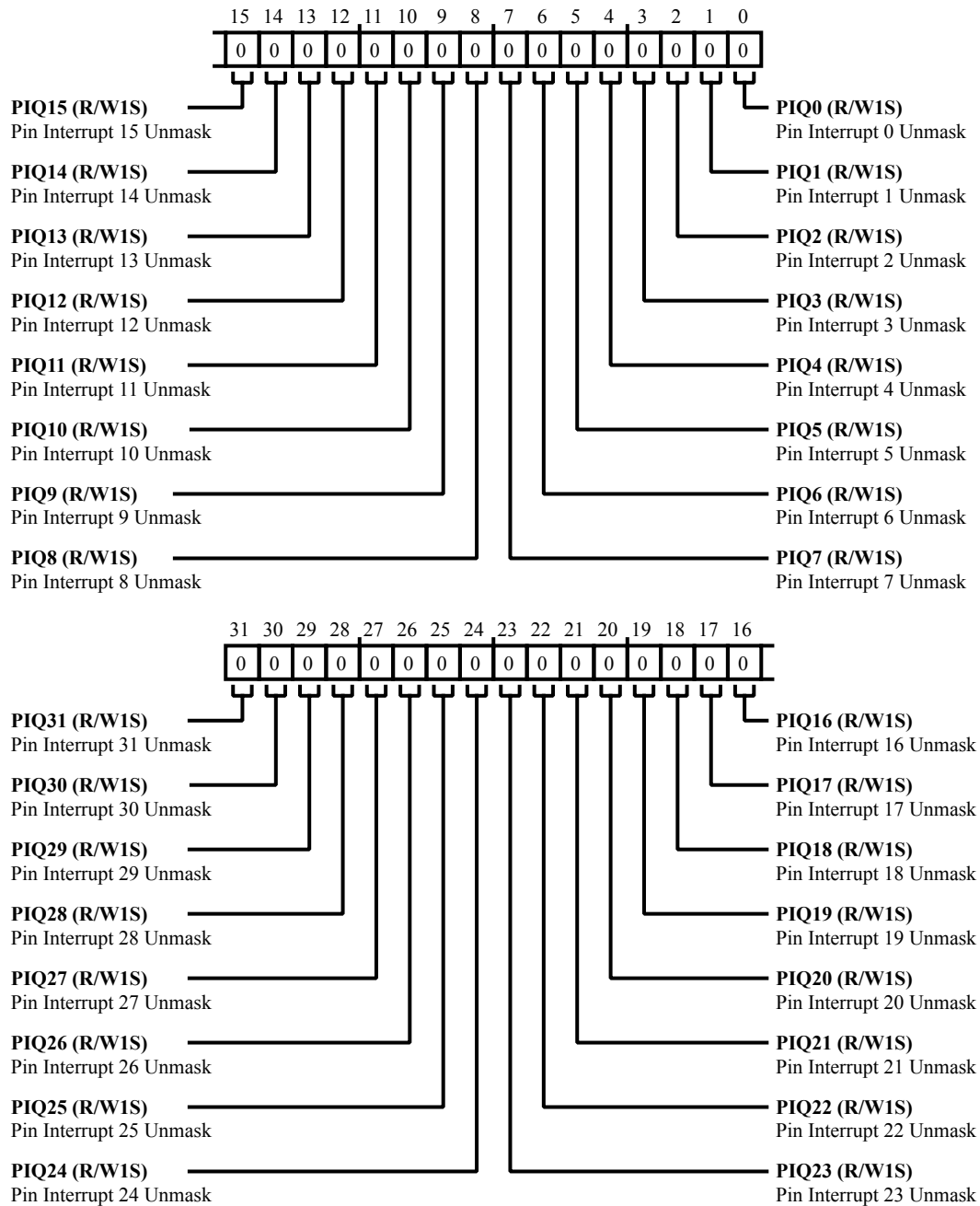


Figure 11-33: PINT_MSK_SET Register Diagram

Table 11-39: PINT_MSK_SET Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W1S)	PIQ31	Pin Interrupt 31 Unmask. Set the PINT_MSK_SET.PIQ31 bit to enable the interrupt.
30 (R/W1S)	PIQ30	Pin Interrupt 30 Unmask. Set the PINT_MSK_SET.PIQ30 bit to enable the interrupt.
29 (R/W1S)	PIQ29	Pin Interrupt 29 Unmask. Set the PINT_MSK_SET.PIQ29 bit to enable the interrupt.
28 (R/W1S)	PIQ28	Pin Interrupt 28 Unmask. Set the PINT_MSK_SET.PIQ28 bit to enable the interrupt.
27 (R/W1S)	PIQ27	Pin Interrupt 27 Unmask. Set the PINT_MSK_SET.PIQ27 bit to enable the interrupt.
26 (R/W1S)	PIQ26	Pin Interrupt 26 Unmask. Set the PINT_MSK_SET.PIQ26 bit to enable the interrupt.
25 (R/W1S)	PIQ25	Pin Interrupt 25 Unmask. Set the PINT_MSK_SET.PIQ25 bit to enable the interrupt.
24 (R/W1S)	PIQ24	Pin Interrupt 24 Unmask. Set the PINT_MSK_SET.PIQ24 bit to enable the interrupt.
23 (R/W1S)	PIQ23	Pin Interrupt 23 Unmask. Set the PINT_MSK_SET.PIQ23 bit to enable the interrupt.
22 (R/W1S)	PIQ22	Pin Interrupt 22 Unmask. Set the PINT_MSK_SET.PIQ22 bit to enable the interrupt.
21 (R/W1S)	PIQ21	Pin Interrupt 21 Unmask. Set the PINT_MSK_SET.PIQ21 bit to enable the interrupt.
20 (R/W1S)	PIQ20	Pin Interrupt 20 Unmask. Set the PINT_MSK_SET.PIQ20 bit to enable the interrupt.
19 (R/W1S)	PIQ19	Pin Interrupt 19 Unmask. Set the PINT_MSK_SET.PIQ19 bit to enable the interrupt.
18 (R/W1S)	PIQ18	Pin Interrupt 18 Unmask. Set the PINT_MSK_SET.PIQ18 bit to enable the interrupt.
17 (R/W1S)	PIQ17	Pin Interrupt 17 Unmask. Set the PINT_MSK_SET.PIQ17 bit to enable the interrupt.
16 (R/W1S)	PIQ16	Pin Interrupt 16 Unmask. Set the PINT_MSK_SET.PIQ16 bit to enable the interrupt.

Table 11-39: PINT_MSK_SET Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W1S)	PIQ15	Pin Interrupt 15 Unmask. Set the PINT_MSK_SET.PIQ15 bit to enable the interrupt.
14 (R/W1S)	PIQ14	Pin Interrupt 14 Unmask. Set the PINT_MSK_SET.PIQ14 bit to enable the interrupt.
13 (R/W1S)	PIQ13	Pin Interrupt 13 Unmask. Set the PINT_MSK_SET.PIQ13 bit to enable the interrupt.
12 (R/W1S)	PIQ12	Pin Interrupt 12 Unmask. Set the PINT_MSK_SET.PIQ12 bit to enable the interrupt.
11 (R/W1S)	PIQ11	Pin Interrupt 11 Unmask. Set the PINT_MSK_SET.PIQ11 bit to enable the interrupt.
10 (R/W1S)	PIQ10	Pin Interrupt 10 Unmask. Set the PINT_MSK_SET.PIQ10 bit to enable the interrupt.
9 (R/W1S)	PIQ9	Pin Interrupt 9 Unmask. Set the PINT_MSK_SET.PIQ9 bit to enable the interrupt.
8 (R/W1S)	PIQ8	Pin Interrupt 8 Unmask. Set the PINT_MSK_SET.PIQ8 bit to enable the interrupt.
7 (R/W1S)	PIQ7	Pin Interrupt 7 Unmask. Set the PINT_MSK_SET.PIQ7 bit to enable the interrupt.
6 (R/W1S)	PIQ6	Pin Interrupt 6 Unmask. Set the PINT_MSK_SET.PIQ6 bit to enable the interrupt.
5 (R/W1S)	PIQ5	Pin Interrupt 5 Unmask. Set the PINT_MSK_SET.PIQ5 bit to enable the interrupt.
4 (R/W1S)	PIQ4	Pin Interrupt 4 Unmask. Set the PINT_MSK_SET.PIQ4 bit to enable the interrupt.
3 (R/W1S)	PIQ3	Pin Interrupt 3 Unmask. Set the PINT_MSK_SET.PIQ3 bit to enable the interrupt.
2 (R/W1S)	PIQ2	Pin Interrupt 2 Unmask. Set the PINT_MSK_SET.PIQ2 bit to enable the interrupt.
1 (R/W1S)	PIQ1	Pin Interrupt 1 Unmask. Set the PINT_MSK_SET.PIQ1 bit to enable the interrupt.
0 (R/W1S)	PIQ0	Pin Interrupt 0 Unmask. Set the PINT_MSK_SET.PIQ0 bit to enable the interrupt.

PINT Pin State Register

When a half port is assigned to a byte in any PINT block, the state of the eight pins (regardless of GPIO or function, input or output) can be seen in the `PINT_PINSTATE` register. While neither input nor output drivers of the pin are enabled, reads of the pin state in `PINT_PINSTATE` return zero. The `PINT_PINSTATE` register reports the inverted state of the pin if the signal inverter is activated by the `PINT_INV_SET` register. The inverter can be enabled on an individual bit-by-bit basis. Every bit in the `PINT_INV_SET` and `PINT_INV_CLR` register pair represents a pin signal.

The pin interrupt pin state registers enable the service routine to read the current state of the pin without reading from GPIO space. If there was an edge-sensitive interrupt, the service routine can check whether the state of the pin is still high or turned low.

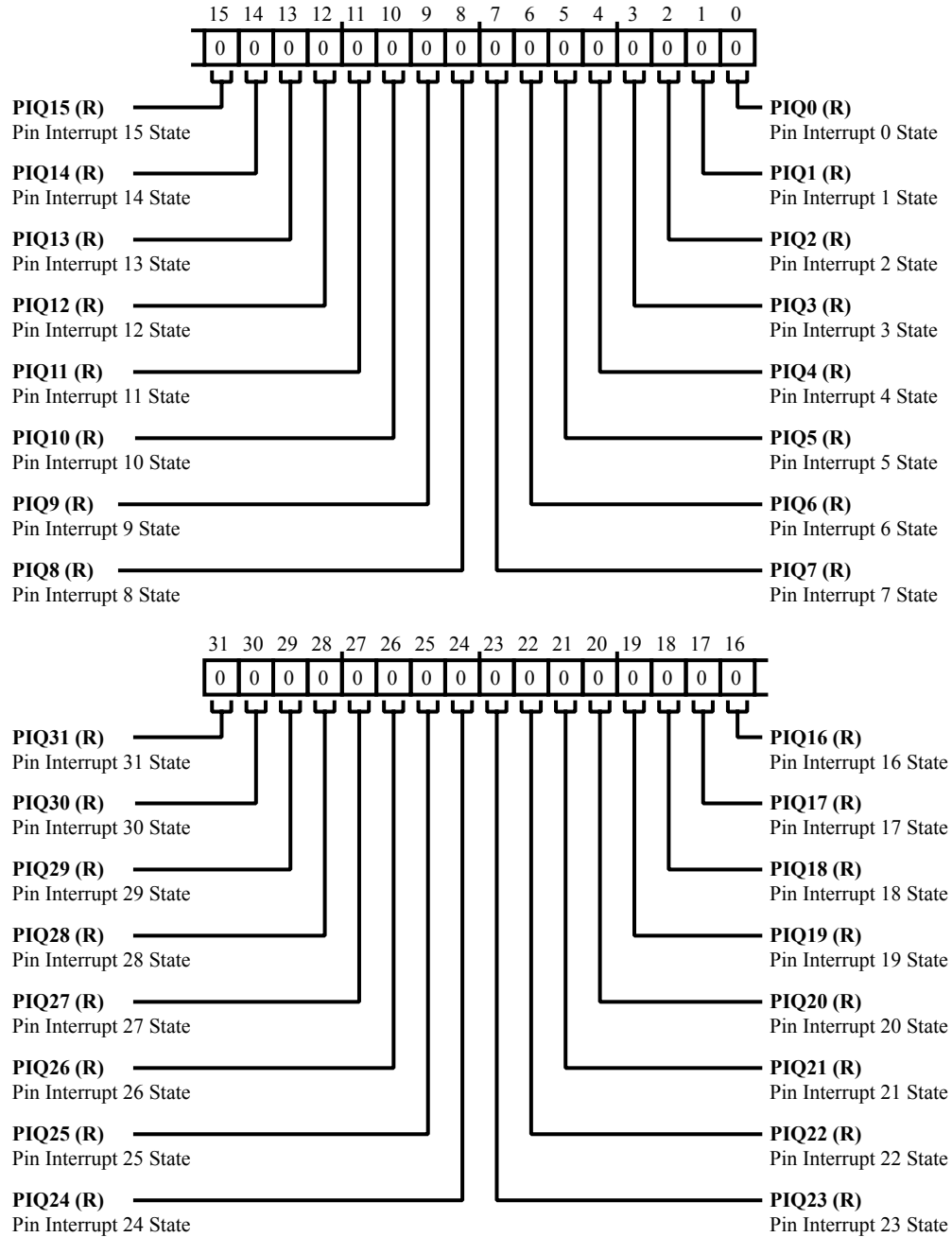


Figure 11-34: PINT_PINSTATE Register Diagram

Table 11-40: PINT_PINSTATE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/NW)	PIQ31	Pin Interrupt 31 State. A read of the <code>PINT_PINSTATE.PIQ31</code> bit returns the pin state.

Table 11-40: PINT_PINSTATE Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
30 (R/NW)	PIQ30	Pin Interrupt 30 State. A read of the PINT_PINSTATE.PIQ30 bit returns the pin state.
29 (R/NW)	PIQ29	Pin Interrupt 29 State. A read of the PINT_PINSTATE.PIQ29 bit returns the pin state.
28 (R/NW)	PIQ28	Pin Interrupt 28 State. A read of the PINT_PINSTATE.PIQ28 bit returns the pin state.
27 (R/NW)	PIQ27	Pin Interrupt 27 State. A read of the PINT_PINSTATE.PIQ27 bit returns the pin state.
26 (R/NW)	PIQ26	Pin Interrupt 26 State. A read of the PINT_PINSTATE.PIQ26 bit returns the pin state.
25 (R/NW)	PIQ25	Pin Interrupt 25 State. A read of the PINT_PINSTATE.PIQ25 bit returns the pin state.
24 (R/NW)	PIQ24	Pin Interrupt 24 State. A read of the PINT_PINSTATE.PIQ24 bit returns the pin state.
23 (R/NW)	PIQ23	Pin Interrupt 23 State. A read of the PINT_PINSTATE.PIQ23 bit returns the pin state.
22 (R/NW)	PIQ22	Pin Interrupt 22 State. A read of the PINT_PINSTATE.PIQ22 bit returns the pin state.
21 (R/NW)	PIQ21	Pin Interrupt 21 State. A read of the PINT_PINSTATE.PIQ21 bit returns the pin state.
20 (R/NW)	PIQ20	Pin Interrupt 20 State. A read of the PINT_PINSTATE.PIQ20 bit returns the pin state.
19 (R/NW)	PIQ19	Pin Interrupt 19 State. A read of the PINT_PINSTATE.PIQ19 bit returns the pin state.
18 (R/NW)	PIQ18	Pin Interrupt 18 State. A read of the PINT_PINSTATE.PIQ18 bit returns the pin state.
17 (R/NW)	PIQ17	Pin Interrupt 17 State. A read of the PINT_PINSTATE.PIQ17 bit returns the pin state.
16 (R/NW)	PIQ16	Pin Interrupt 16 State. A read of the PINT_PINSTATE.PIQ16 bit returns the pin state.
15 (R/NW)	PIQ15	Pin Interrupt 15 State. A read of the PINT_PINSTATE.PIQ15 bit returns the pin state.

Table 11-40: PINT_PINSTATE Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
14 (R/NW)	PIQ14	Pin Interrupt 14 State. A read of the PINT_PINSTATE.PIQ14 bit returns the pin state.
13 (R/NW)	PIQ13	Pin Interrupt 13 State. A read of the PINT_PINSTATE.PIQ13 bit returns the pin state.
12 (R/NW)	PIQ12	Pin Interrupt 12 State. A read of the PINT_PINSTATE.PIQ12 bit returns the pin state.
11 (R/NW)	PIQ11	Pin Interrupt 11 State. A read of the PINT_PINSTATE.PIQ11 bit returns the pin state.
10 (R/NW)	PIQ10	Pin Interrupt 10 State. A read of the PINT_PINSTATE.PIQ10 bit returns the pin state.
9 (R/NW)	PIQ9	Pin Interrupt 9 State. A read of the PINT_PINSTATE.PIQ9 bit returns the pin state.
8 (R/NW)	PIQ8	Pin Interrupt 8 State. A read of the PINT_PINSTATE.PIQ8 bit returns the pin state.
7 (R/NW)	PIQ7	Pin Interrupt 7 State. A read of the PINT_PINSTATE.PIQ7 bit returns the pin state.
6 (R/NW)	PIQ6	Pin Interrupt 6 State. A read of the PINT_PINSTATE.PIQ6 bit returns the pin state.
5 (R/NW)	PIQ5	Pin Interrupt 5 State. A read of the PINT_PINSTATE.PIQ5 bit returns the pin state.
4 (R/NW)	PIQ4	Pin Interrupt 4 State. A read of the PINT_PINSTATE.PIQ4 bit returns the pin state.
3 (R/NW)	PIQ3	Pin Interrupt 3 State. A read of the PINT_PINSTATE.PIQ3 bit returns the pin state.
2 (R/NW)	PIQ2	Pin Interrupt 2 State. A read of the PINT_PINSTATE.PIQ2 bit returns the pin state.
1 (R/NW)	PIQ1	Pin Interrupt 1 State. A read of the PINT_PINSTATE.PIQ1 bit returns the pin state.
0 (R/NW)	PIQ0	Pin Interrupt 0 State. A read of the PINT_PINSTATE.PIQ0 bit returns the pin state.

PINT Request Register

The `PINT_REQ` register indicates the interrupt request status for pin interrupts. When set, an interrupt request is pending. When cleared, there is no interrupt request pending.

Both the `PINT_REQ` and `PINT_LATCH` registers indicate whether an interrupt request is latched on the respective pin. The `PINT_LATCH` register is a latch that operates regardless of the interrupt masks. Bits of the `PINT_REQ` register depend on the mask register. The `PINT_REQ` register is a logical AND of the `PINT_LATCH` register and the interrupt mask.

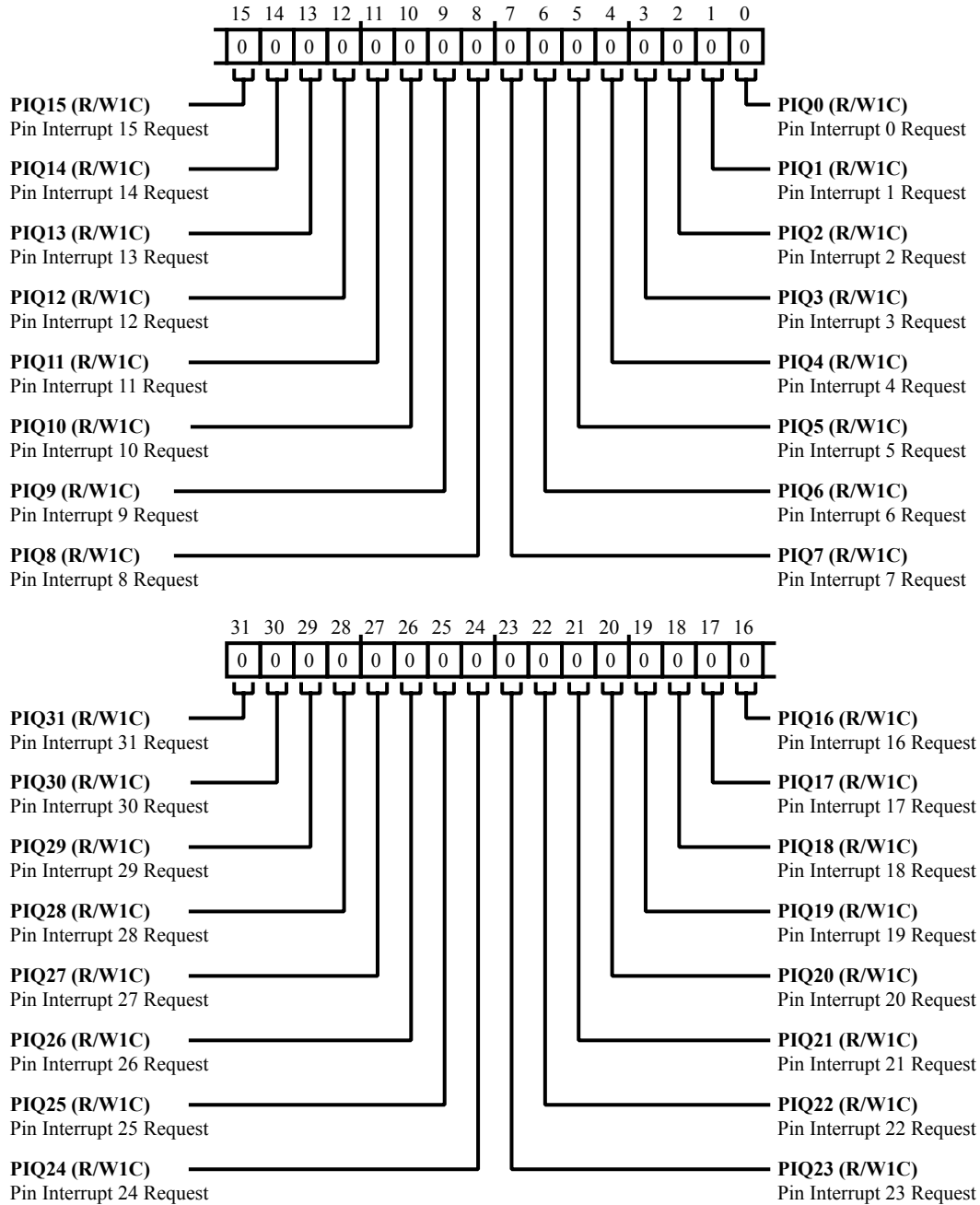


Figure 11-35: PINT_REQ Register Diagram

Table 11-41: PINT_REQ Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W1C)	PIQ31	Pin Interrupt 31 Request. If the PINT_REQ.PIQ31 bit is set, a request is pending.

Table 11-41: PINT_REQ Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
30 (R/W1C)	PIQ30	Pin Interrupt 30 Request. If the PINT_REQ.PIQ30 bit is set, a request is pending.
29 (R/W1C)	PIQ29	Pin Interrupt 29 Request. If the PINT_REQ.PIQ29 bit is set, a request is pending.
28 (R/W1C)	PIQ28	Pin Interrupt 28 Request. If the PINT_REQ.PIQ28 bit is set, a request is pending.
27 (R/W1C)	PIQ27	Pin Interrupt 27 Request. If the PINT_REQ.PIQ27 bit is set, a request is pending.
26 (R/W1C)	PIQ26	Pin Interrupt 26 Request. If the PINT_REQ.PIQ26 bit is set, a request is pending.
25 (R/W1C)	PIQ25	Pin Interrupt 25 Request. If the PINT_REQ.PIQ25 bit is set, a request is pending.
24 (R/W1C)	PIQ24	Pin Interrupt 24 Request. If the PINT_REQ.PIQ24 bit is set, a request is pending.
23 (R/W1C)	PIQ23	Pin Interrupt 23 Request. If the PINT_REQ.PIQ23 bit is set, a request is pending.
22 (R/W1C)	PIQ22	Pin Interrupt 22 Request. If the PINT_REQ.PIQ22 bit is set, a request is pending.
21 (R/W1C)	PIQ21	Pin Interrupt 21 Request. If the PINT_REQ.PIQ21 bit is set, a request is pending.
20 (R/W1C)	PIQ20	Pin Interrupt 20 Request. If the PINT_REQ.PIQ20 bit is set, a request is pending.
19 (R/W1C)	PIQ19	Pin Interrupt 19 Request. If the PINT_REQ.PIQ19 bit is set, a request is pending.
18 (R/W1C)	PIQ18	Pin Interrupt 18 Request. If the PINT_REQ.PIQ18 bit is set, a request is pending.
17 (R/W1C)	PIQ17	Pin Interrupt 17 Request. If the PINT_REQ.PIQ17 bit is set, a request is pending.
16 (R/W1C)	PIQ16	Pin Interrupt 16 Request. If the PINT_REQ.PIQ16 bit is set, a request is pending.
15 (R/W1C)	PIQ15	Pin Interrupt 15 Request. If the PINT_REQ.PIQ15 bit is set, a request is pending.

Table 11-41: PINT_REQ Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
14 (R/W1C)	PIQ14	Pin Interrupt 14 Request. If the PINT_REQ.PIQ14 bit is set, a request is pending.
13 (R/W1C)	PIQ13	Pin Interrupt 13 Request. If the PINT_REQ.PIQ13 bit is set, a request is pending.
12 (R/W1C)	PIQ12	Pin Interrupt 12 Request. If the PINT_REQ.PIQ12 bit is set, a request is pending.
11 (R/W1C)	PIQ11	Pin Interrupt 11 Request. If the PINT_REQ.PIQ11 bit is set, a request is pending.
10 (R/W1C)	PIQ10	Pin Interrupt 10 Request. If the PINT_REQ.PIQ10 bit is set, a request is pending.
9 (R/W1C)	PIQ9	Pin Interrupt 9 Request. If the PINT_REQ.PIQ9 bit is set, a request is pending.
8 (R/W1C)	PIQ8	Pin Interrupt 8 Request. If the PINT_REQ.PIQ8 bit is set, a request is pending.
7 (R/W1C)	PIQ7	Pin Interrupt 7 Request. If the PINT_REQ.PIQ7 bit is set, a request is pending.
6 (R/W1C)	PIQ6	Pin Interrupt 6 Request. If the PINT_REQ.PIQ6 bit is set, a request is pending.
5 (R/W1C)	PIQ5	Pin Interrupt 5 Request. If the PINT_REQ.PIQ5 bit is set, a request is pending.
4 (R/W1C)	PIQ4	Pin Interrupt 4 Request. If the PINT_REQ.PIQ4 bit is set, a request is pending.
3 (R/W1C)	PIQ3	Pin Interrupt 3 Request. If the PINT_REQ.PIQ3 bit is set, a request is pending.
2 (R/W1C)	PIQ2	Pin Interrupt 2 Request. If the PINT_REQ.PIQ2 bit is set, a request is pending.
1 (R/W1C)	PIQ1	Pin Interrupt 1 Request. If the PINT_REQ.PIQ1 bit is set, a request is pending.
0 (R/W1C)	PIQ0	Pin Interrupt 0 Request. If the PINT_REQ.PIQ0 bit is set, a request is pending.

ADSP-21560_21561_21564_21568 PADS Register Descriptions

Pads Controller (PADS) contains the following registers.

Table 11-42: ADSP-21560_21561_21564_21568 PADS Register List

Name	Description
PADS_DAI0_0_DS	DAI0 1 to 10 pins DS Control Register
PADS_DAI0_1_DS	DAI0 11 to 20 pins DS Control Register
PADS_DAI0_IE	DAI0 Port Input Enable Control Register
PADS_DAI1_0_DS	DAI1 1 to 10 pins DS Control Register
PADS_DAI1_1_DS	DAI1 11 to 20 pins DS Control Register
PADS_DAI1_IE	DAI1 Port Input Enable Control Register
PADS_DAI[n]_PDE	DAIx Pull-Down Enable Register
PADS_DAI[n]_PUE	DAIx Pull-Up Enable Register
PADS_NONPORTS_DS	Non-GPIO Drive Strength Register
PADS_PCFG0	Peripheral PAD Configuration 0 Register
PADS_PORTA0_DS	PORTA 0 to 7 pins DS Control Register
PADS_PORTA1_DS	PORTA 8 - 15 pins DS Control Register
PADS_PORTB0_DS	PORTB 0 to 7 pins DS Control Register
PADS_PORT[n]_PDE	PORTx Pull-Down Enable Register
PADS_PORT[n]_PUE	PORTx Pull-Up Enable Register

DAI0 1 to 10 pins DS Control Register

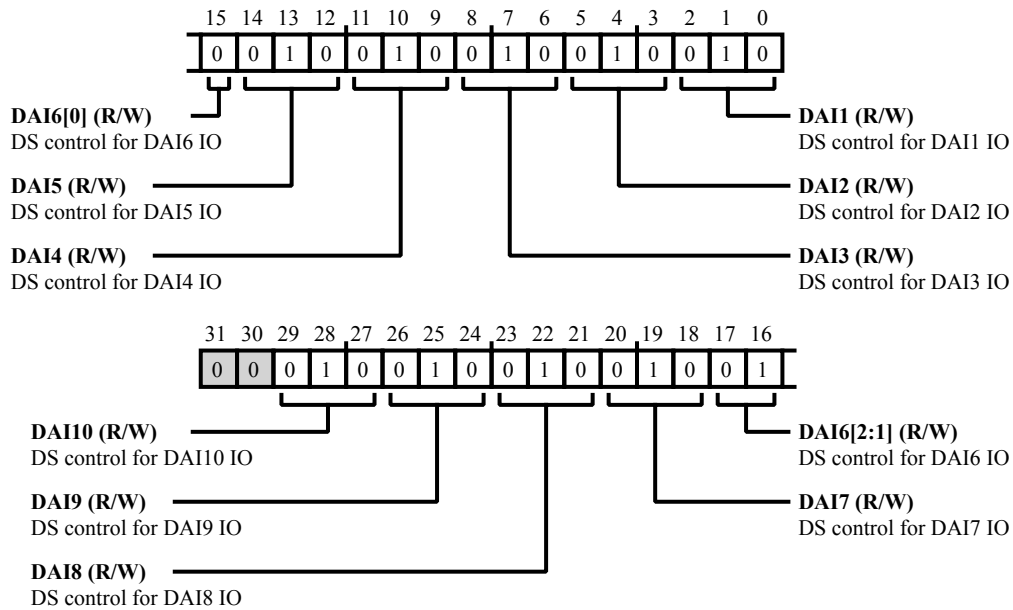


Figure 11-36: PADS_DAI0_0_DS Register Diagram

Table 11-43: PADS_DAI0_0_DS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:27 (R/W)	DAI10	DS control for DAI10 IO.
		1 Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2 Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
26:24 (R/W)	DAI9	DS control for DAI9 IO.
		1 Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2 Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.

Table 11-43: PADS_DAI0_0_DS Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
23:21 (R/W)	DAI8	DS control for DAI8 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
20:18 (R/W)	DAI7	DS control for DAI7 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
17:15 (R/W)	DAI6	DS control for DAI6 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
14:12 (R/W)	DAI5	DS control for DAI5 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
11:9 (R/W)	DAI4	DS control for DAI4 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.

Table 11-43: PADS_DAI0_0_DS Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
8:6 (R/W)	DAI3	DS control for DAI3 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
5:3 (R/W)	DAI2	DS control for DAI2 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
2:0 (R/W)	DAI1	DS control for DAI1 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.

DAI0 11 to 20 pins DS Control Register

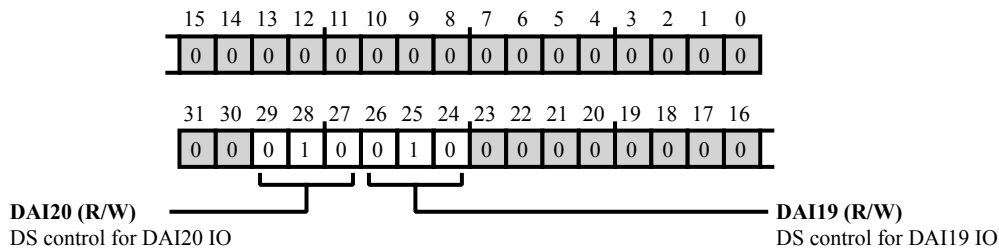


Figure 11-37: PADS_DAI0_1_DS Register Diagram

Table 11-44: PADS_DAI0_1_DS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:27 (R/W)	DAI20	DS control for DAI20 IO.
		1 Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2 Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
26:24 (R/W)	DAI19	DS control for DAI19 IO.
		1 Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2 Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.

DAI0 Port Input Enable Control Register

The `PADS_DAI0_IE` register configures input enable control of the DAI0 (20 pins) pads. If =0 implies input buffer disable and if =1 implies enable.

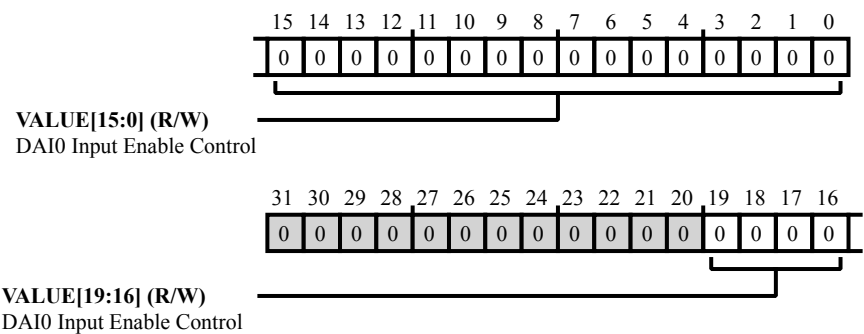


Figure 11-38: PADS_DAI0_IE Register Diagram

Table 11-45: PADS_DAI0_IE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
19:0 (R/W)	VALUE	DAI0 Input Enable Control.

DAI1 1 to 10 pins DS Control Register

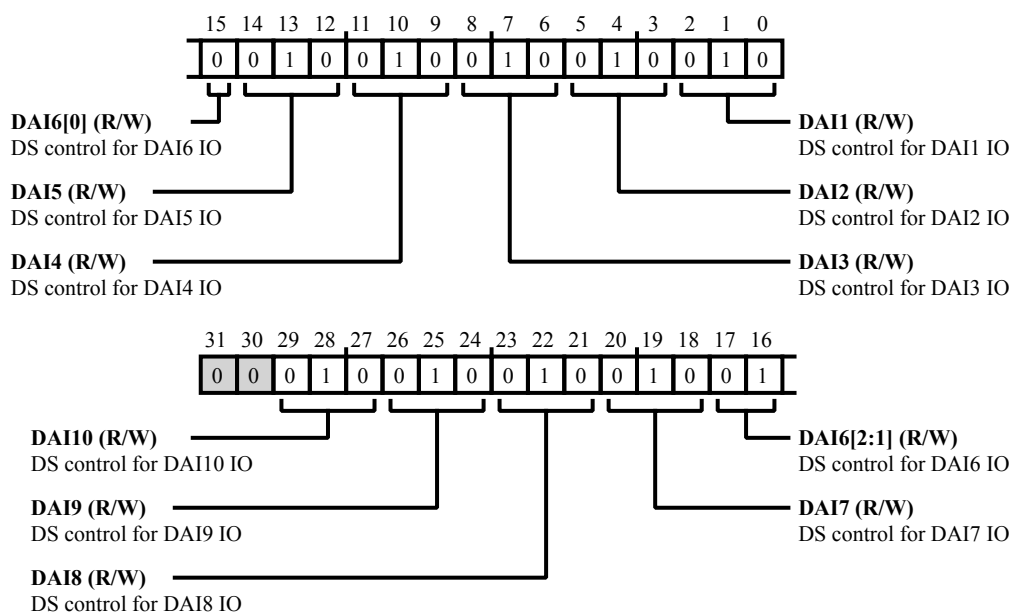


Figure 11-39: PADS_DAI1_0_DS Register Diagram

Table 11-46: PADS_DAI1_0_DS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:27 (R/W)	DAI10	DS control for DAI10 IO.
		1 Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2 Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
26:24 (R/W)	DAI9	DS control for DAI9 IO.
		1 Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2 Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.

Table 11-46: PADS_DAI1_0_DS Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
23:21 (R/W)	DAI8	DS control for DAI8 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
20:18 (R/W)	DAI7	DS control for DAI7 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
17:15 (R/W)	DAI6	DS control for DAI6 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
14:12 (R/W)	DAI5	DS control for DAI5 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
11:9 (R/W)	DAI4	DS control for DAI4 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.

Table 11-46: PADS_DAI1_0_DS Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
8:6 (R/W)	DAI3	DS control for DAI3 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
5:3 (R/W)	DAI2	DS control for DAI2 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
2:0 (R/W)	DAI1	DS control for DAI1 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.

DAI1 11 to 20 pins DS Control Register

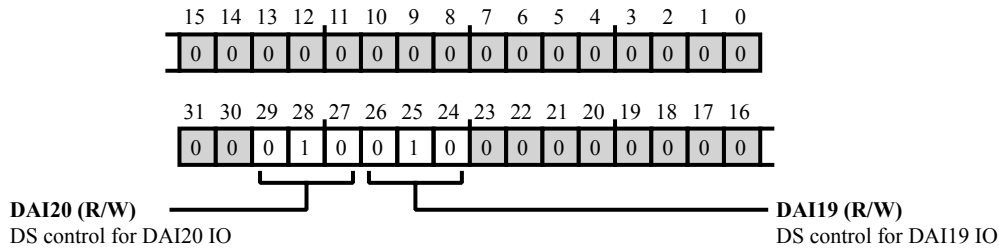


Figure 11-40: PADS_DAI1_1_DS Register Diagram

Table 11-47: PADS_DAI1_1_DS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:27 (R/W)	DAI20	DS control for DAI20 IO.
		1 Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2 Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
26:24 (R/W)	DAI19	DS control for DAI19 IO.
		1 Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2 Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.

DAI1 Port Input Enable Control Register

The `PADS_DAI1_IE` register configures input enable control of the DAI1 (20 pins) pads. If =0 implies input buffer disable and if =1 implies enable.

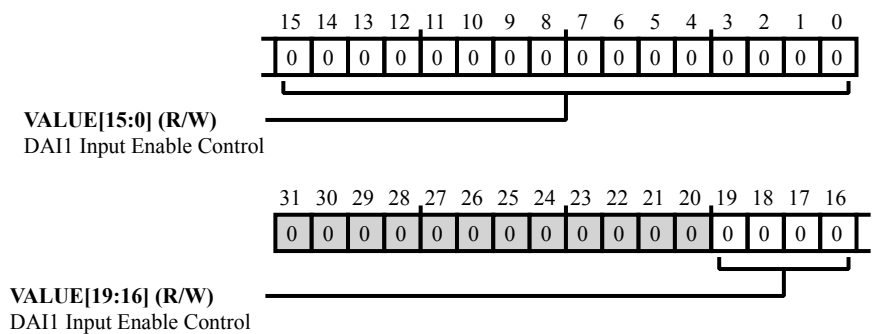


Figure 11-41: PADS_DAI1_IE Register Diagram

Table 11-48: PADS_DAI1_IE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
19:0 (R/W)	VALUE	DAI1 Input Enable Control.

DAIx Pull-Down Enable Register

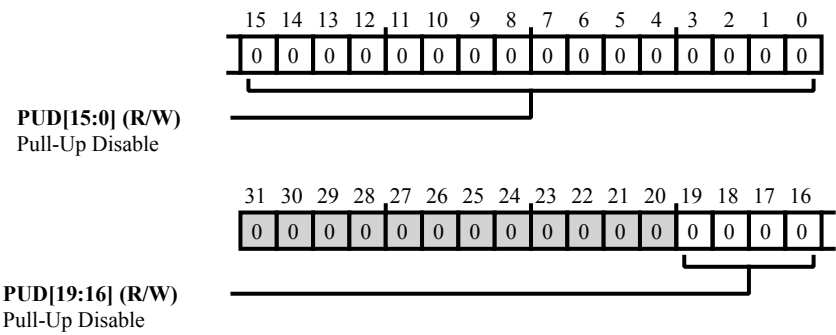


Figure 11-42: PADS_DAI[n]_PDE Register Diagram

Table 11-49: PADS_DAI[n]_PDE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
19:0 (R/W)	PUD	Pull-Up Disable.

DAIx Pull-Up Enable Register

The `PADS_DAI[n]_PUE` register enables the pull-up resistor for DAI pins.

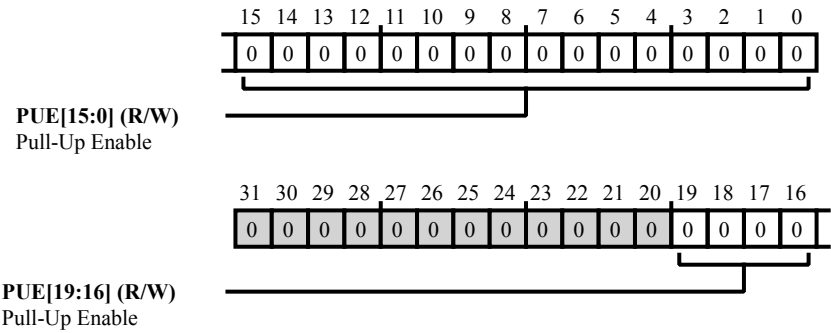


Figure 11-43: PADS_DAI[n]_PUE Register Diagram

Table 11-50: PADS_DAI[n]_PUE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
19:0 (R/W)	PUE	Pull-Up Enable.

Non-GPIO Drive Strength Register

The `PADS_NONPORTS_DS` register sets the drive strength and tolerance for the non-GPIO pins. The drive strength is only controlled for pin groups classified by the interface type.

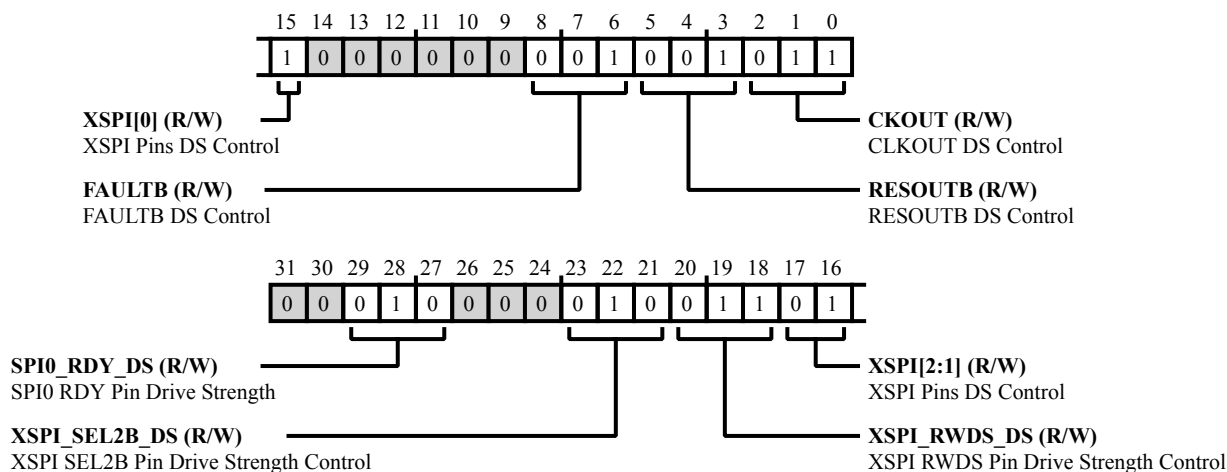


Figure 11-44: `PADS_NONPORTS_DS` Register Diagram

Table 11-51: `PADS_NONPORTS_DS` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:27 (R/W)	<code>SPI0_RDY_DS</code>	SPI0 RDY Pin Drive Strength. The <code>PADS_NONPORTS_DS.SPI0_RDY_DS</code> bit field indicates the drive strength of the SPI0_RDY pin.
23:21 (R/W)	<code>XSPI_SEL2B_DS</code>	XSPI SEL2B Pin Drive Strength Control. The <code>PADS_NONPORTS_DS.XSPI_SEL2B_DS</code> bit field indicates the drive strength of the XSPI_SEL2b pin.
20:18 (R/W)	<code>XSPI_RWDS_DS</code>	XSPI RWDS Pin Drive Strength Control. The <code>PADS_NONPORTS_DS.XSPI_RWDS_DS</code> bit field indicates the drive strength of the XSPI_SEL2b pin.
17:15 (R/W)	XSPI	XSPI Pins DS Control.
		1 Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2 Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.

Table 11-51: PADS_NONPORTS_DS Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
8:6 (R/W)	FAULTB	FAULTB DS Control.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
5:3 (R/W)	RESOUTB	RESOUTB DS Control.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
2:0 (R/W)	CKOUT	CLKOUT DS Control.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.

Peripheral PAD Configuration 0 Register

The `PADS_PCFG0` register provides several configuration options for the pads and multiplexing for peripherals.

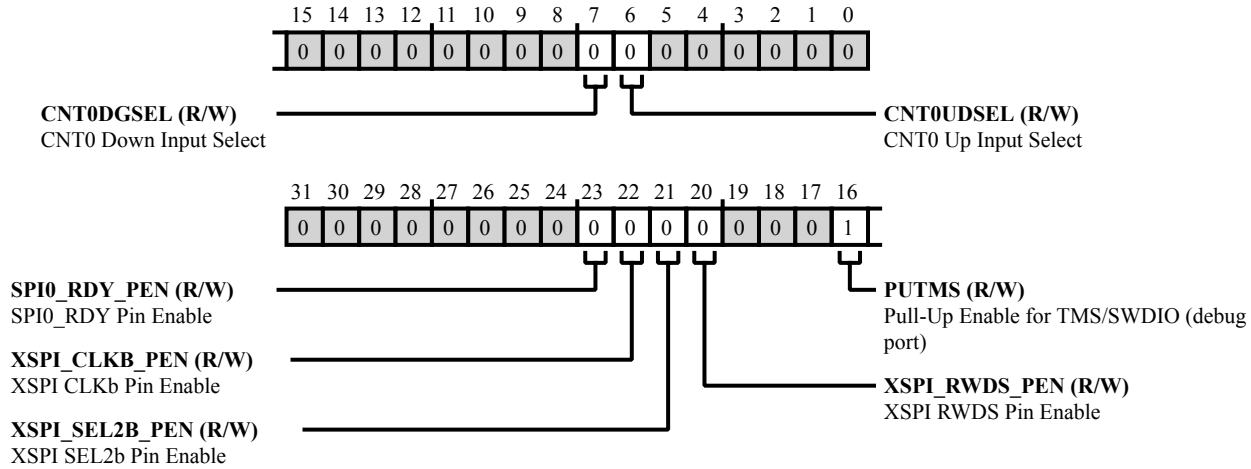


Figure 11-45: PADS_PCFG0 Register Diagram

Table 11-52: PADS_PCFG0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
23 (R/W)	SPI0_RDY_PEN	SPI0_RDY Pin Enable. The <code>PADS_PCFG0.SPI0_RDY_PEN</code> bit indicates the OE and IE gating of the SPI0_RDY pad.
		0 IE = 0 and OEN = 1, effectively disconnecting the SPI0_RDY pad.
		1 Pad is connected, and IE and OE are controlled from the SPI0 design.
22 (R/W)	XSPI_CLKB_PEN	XSPI_CLKb Pin Enable. The <code>PADS_PCFG0.XSPI_CLKB_PEN</code> bit indicates the OE and IE gating of the xSPI_CLKb pad.
		0 IE = 0 and OEN = 1, effectively disconnecting the xSPI_CLKb pad.
		1 Pad is connected, and IE and OE are controlled from the xSPI design.

Table 11-52: PADS_PCFG0 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
21 (R/W)	XSPI_SEL2B_PEN	XSPI SEL2b Pin Enable. The <code>PADS_PCFG0.XSPI_SEL2B_PEN</code> bit indicates the OE and IE gating of the <code>xSPI_SEL2b</code> pad.
		0 IE = 0 and OEN = 1, effectively disconnecting the <code>xSPI_SEL2b</code> pad.
		1 Pad is connected, and IE and OE are controlled from the xSPI design.
20 (R/W)	XSPI_RWDS_PEN	XSPI RWDS Pin Enable. The <code>PADS_PCFG0.XSPI_RWDS_PEN</code> bit indicates the OE and IE gating of the <code>xSPI_RWDS</code> pad.
		0 IE = 0 and OEN = 1, effectively disconnecting the <code>xSPI_RWDS</code> pad.
		1 Pad is connected, and IE and OE are controlled from the xSPI design.
16 (R/W)	PUTMS	Pull-Up Enable for TMS/SWDIO (debug port).
		0 Disable pull-up
		1 Enable pull-up
7 (R/W)	CNT0DGSEL	CNT0 Down Input Select. The <code>PADS_PCFG0</code> bit selects the input for the CNT0 counter down and gate.
		0 GPIO port pins
		1 Trigger connection
6 (R/W)	CNT0UDSEL	CNT0 Up Input Select. The <code>PADS_PCFG0</code> bit selects the input for the CNT0 counter up and direction.
		0 GPIO port pins
		1 Trigger connection

PORTA 0 to 7 pins DS Control Register

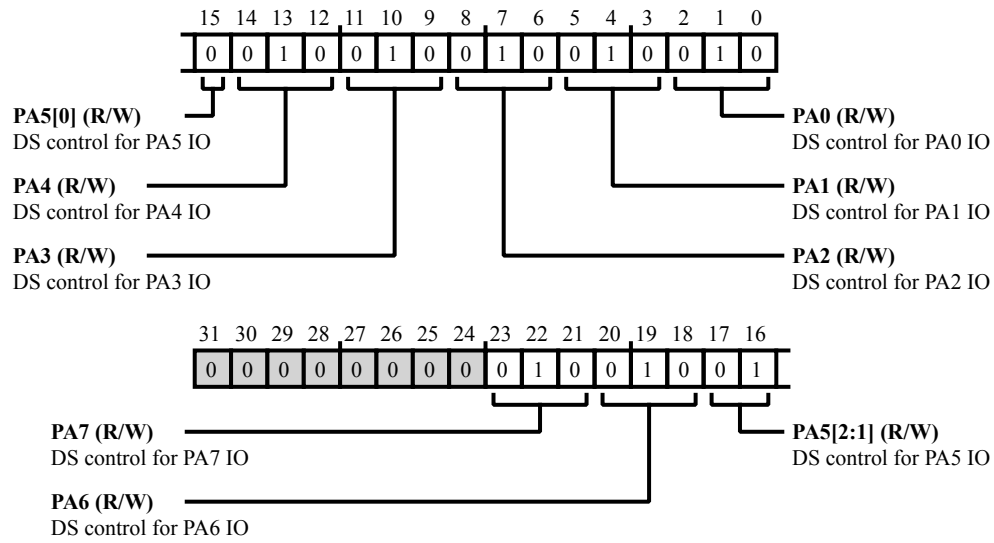


Figure 11-46: PADS_PORTA0_DS Register Diagram

Table 11-53: PADS_PORTA0_DS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
23:21 (R/W)	PA7	DS control for PA7 IO.
		1 Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2 Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
20:18 (R/W)	PA6	DS control for PA6 IO.
		1 Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2 Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.

Table 11-53: PADS_PORTA0_DS Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
17:15 (R/W)	PA5	DS control for PA5 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
14:12 (R/W)	PA4	DS control for PA4 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
11:9 (R/W)	PA3	DS control for PA3 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
8:6 (R/W)	PA2	DS control for PA2 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
5:3 (R/W)	PA1	DS control for PA1 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.

Table 11-53: PADS_PORTA0_DS Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
2:0 (R/W)	PA0	DS control for PA0 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.

PORTA 8 - 15 pins DS Control Register

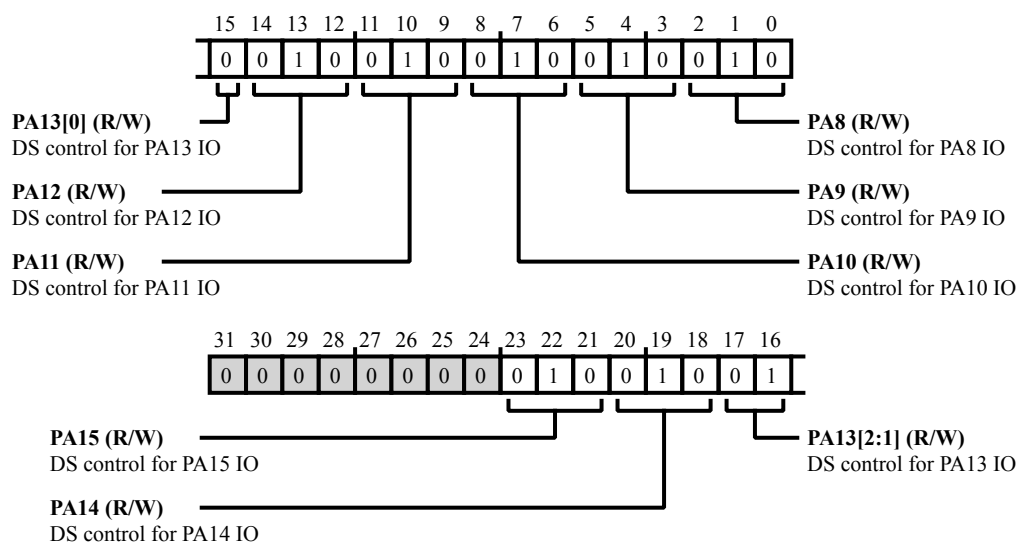


Figure 11-47: PADS_PORTA1_DS Register Diagram

Table 11-54: PADS_PORTA1_DS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
23:21 (R/W)	PA15	DS control for PA15 IO.
		1 Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2 Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
20:18 (R/W)	PA14	DS control for PA14 IO.
		1 Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2 Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.

Table 11-54: PADS_PORTA1_DS Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
17:15 (R/W)	PA13	DS control for PA13 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
14:12 (R/W)	PA12	DS control for PA12 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
11:9 (R/W)	PA11	DS control for PA11 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
8:6 (R/W)	PA10	DS control for PA10 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
5:3 (R/W)	PA9	DS control for PA9 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.

Table 11-54: PADS_PORTA1_DS Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
2:0 (R/W)	PA8	DS control for PA8 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.

PORTB 0 to 7 pins DS Control Register

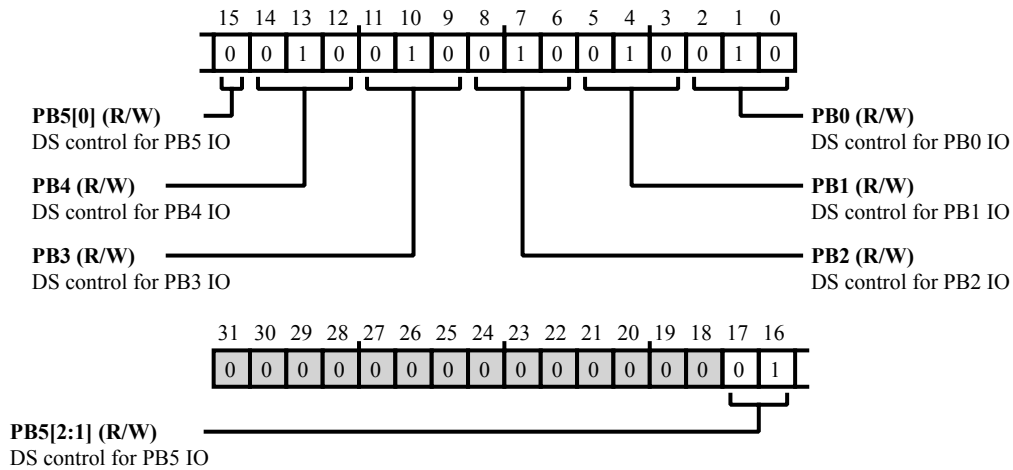


Figure 11-48: PADS_PORTB0_DS Register Diagram

Table 11-55: PADS_PORTB0_DS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
17:15 (R/W)	PB5	DS control for PB5 IO.
		1 Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2 Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
14:12 (R/W)	PB4	DS control for PB4 IO.
		1 Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2 Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
11:9 (R/W)	PB3	DS control for PB3 IO.
		1 Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2 Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.

Table 11-55: PADS_PORTB0_DS Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
8:6 (R/W)	PB2	DS control for PB2 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
5:3 (R/W)	PB1	DS control for PB1 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
2:0 (R/W)	PB0	DS control for PB0 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid.
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.

PORTx Pull-Down Enable Register

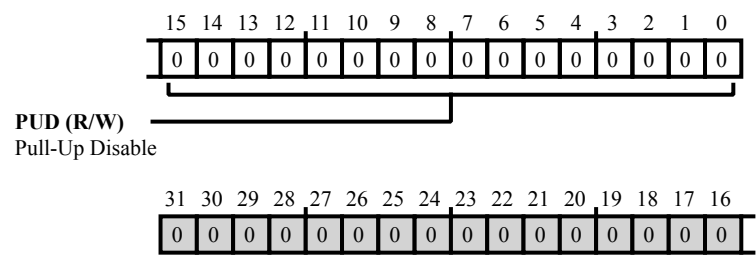


Figure 11-49: PADS_PORT[n]_PDE Register Diagram

Table 11-56: PADS_PORT[n]_PDE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	PUD	Pull-Up Disable.

PORTx Pull-Up Enable Register

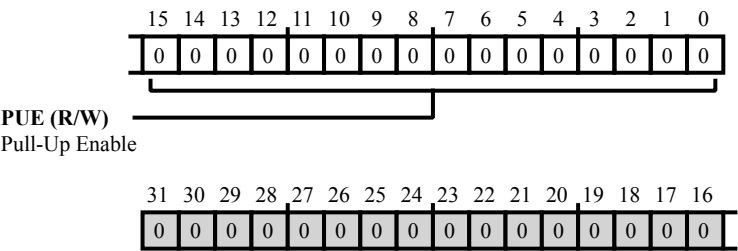


Figure 11-50: PADS_PORT[n]_PUE Register Diagram

Table 11-57: PADS_PORT[n]_PUE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	PUE	Pull-Up Enable.

12 Watchdog Timer (WDOG)

The processor has a 32-bit watchdog timer that can be used to verify system reliability by generating an event to the processor core when the watchdog expires before software updates it.

WDOG Features

The watchdog timer has the following features:

- Programmable 32-bit watchdog count value
- 8-bit disable bit pattern
- General-purpose core event generation

The watchdog timer can supervise system software stability by periodically reloading it to prevent expiration of the downward-counting timer (such that the count never becomes 0). When used in this fashion, an expiring timer can indicate the system software is not running normally.

Expiration of the WDOG counter generates a general-purpose interrupt, which can be used in a variety of ways:

- as an interrupt vector sent through the System Event Controller (SEC) to the core to be serviced by a handler function, providing full software control of device resources (for example, GPIO management and reset control).
- as a fault condition through the SEC to provide hardware-automated:
 - signaling of the fault condition on external pins to the system,
 - system reset requests to the Reset Control Unit (RCU), and/or
 - trigger outputs (SEC0_FAULT trigger generator) through the Trigger Routing Unit (TRU) to initiate activities in a variety of potential trigger receiver (for example, GPIO control).

To help debugging, the watchdog timer does not decrement (even when enabled) if the processor is in emulation mode.

WDOG Functional Description

When enabled, the 32-bit watchdog timer counts downward every SCLK0_0 cycle. If it reaches zero, the watchdog expiration event is generated, which can be used in many ways.

To start the watchdog timer:

1. Program the watchdog timeout period (in SCLK0_0 cycles) in the `WDOG_CNT` register. With the watchdog disabled, this write also preloads the `WDOG_STAT` register.
2. Enable the watchdog timer by writing any value other than 0xAD to the `WDOG_CTL.WDEN` field.

Once the watchdog is enabled, writes to the `WDOG_CNT` register are ignored. The counter begins decrementing, and the current counter value can be read from the 32-bit `WDOG_STAT` register at any time.

To prevent the counter from expiring, software must "kick" the watchdog by writing any value to the `WDOG_STAT` register while the current count is non-zero. While the value written is irrelevant and ignored, this action resets the current counter in the `WDOG_STAT` register to the programmed `WDOG_CNT` value, and decrementing continues. The internal counter continues decrementing until it reaches zero, at which point the expiration event is generated, and the `WDOG_CTL.WDRO` rollover bit is set.

Watchdog operation continues in this manner unless disabled by explicitly writing 0xAD to the `WDOG_CTL.WDEN` field.

The watchdog expiration event can be used in a variety of ways, as listed below.

- The watchdog expiration event itself is one of numerous interrupt sources that is managed by the System Event Controller. Like other peripheral sources, this event can be used to cause a vector to a handler function that executes based on interrupt priority.
- The watchdog expiration event can be used to initiate automated hardware response through the SEC Fault Interface (SFI).

For this, the WDOG expiry must be configured as the fault source in the SEC. Then the response to the WDOG expiry can be set to any of the following actions:

- Signaling through the external fault pin.
- System reset.
- Trigger outputs to numerous potential trigger completers.

ADSP-21560_21561_21564_21568 WDOG Register List

The Watchdog Timer unit (WDOG) provides a software-based watchdog timer that can improve system reliability by generating an event to the processor core if the watchdog expires before being updated by software. A set of registers governs WDOG operations. For more information on WDOG functionality, see the WDOG register descriptions.

Table 12-1: ADSP-21560_21561_21564_21568 WDOG Register List

Name	Description
WDOG_CNT	Count Register
WDOG_CTL	Control Register
WDOG_STAT	Watchdog Timer Status Register
WDOG_WIN	Watchdog Timer Window Register

ADSP-21560_21561_21564_21568 WDOG Interrupt List

Table 12-2: ADSP-21560_21561_21564_21568 WDOG Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
3	WDOG0_EXP	WDOG0 Expiration	Level	
4	WDOG1_EXP	WDOG1 Expiration	Level	

ADSP-21560_21561_21564_21568 WDOG Trigger List

Table 12-3: ADSP-21560_21561_21564_21568 WDOG Trigger List Generators

Trigger ID	Name	Description	Sensitivity
101	WDOG0_EXP	WDOG0 Expiration	Level
102	WDOG1_EXP	WDOG1 Expiration	Level

Table 12-4: ADSP-21560_21561_21564_21568 WDOG Trigger List Receivers

Trigger ID	Name	Description	Sensitivity
None			

WDOG Block Diagram

The *Watchdog Timer Block Diagram* figure shows the detailed block diagram for the watchdog timer.

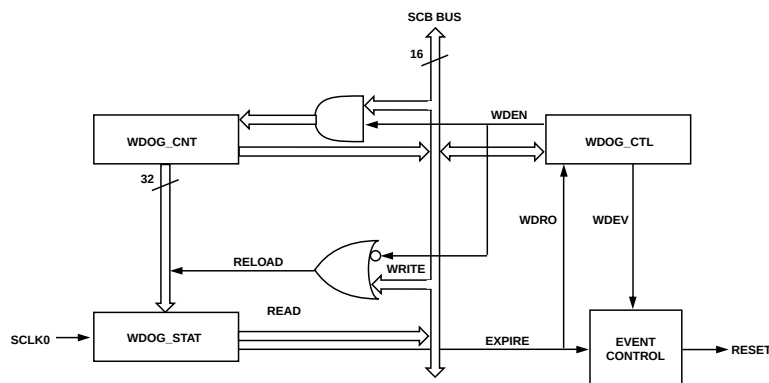


Figure 12-1: Watchdog Timer Block Diagram

Internal Interface

The registers are accessed through the peripheral MMR access bus. 32-bit read/write operations always access the 32-bit `WDOG_CNT` and `WDOG_STAT` registers. Hardware ensures that those accesses are atomic. When the counter expires, the WDOG expiration event is generated.

External Interface

The watchdog timer does not directly interact with any external pins.

ADSP-21560_21561_21564_21568 WDOG Register Descriptions

Watchdog Timer Unit (WDOG) contains the following registers.

Table 12-5: ADSP-21560_21561_21564_21568 WDOG Register List

Name	Description
<code>WDOG_CNT</code>	Count Register
<code>WDOG_CTL</code>	Control Register
<code>WDOG_STAT</code>	Watchdog Timer Status Register
<code>WDOG_WIN</code>	Watchdog Timer Window Register

Count Register

The `WDOG_CNT` register holds the programmable, unsigned count value. A valid write to this register also pre-loads the WDOG counter. For added safety, the `WDOG_CNT` register can be updated only when the WDOG timer is disabled. A write to the `WDOG_CNT` register while the timer is enabled does not modify the contents of this register. This register must be accessed with 32-bit read/writes only.

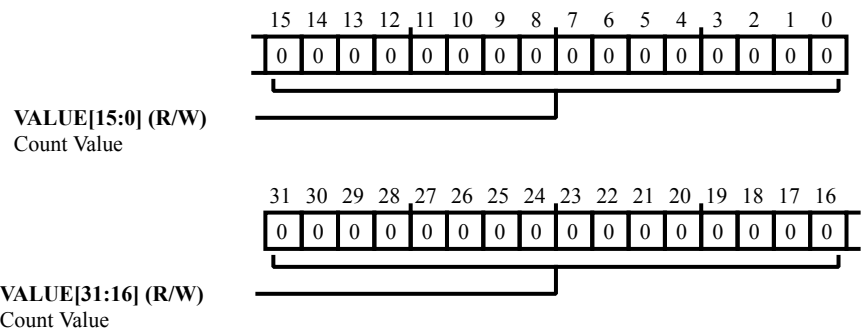


Figure 12-2: WDOG_CNT Register Diagram

Table 12-6: WDOG_CNT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Count Value. The <code>WDOG_CNT.VALUE</code> bit field holds the programmable, unsigned count value.

Control Register

The `WDOG_CTL` register controls the watchdog timer. This register supports enabling/disabling the watchdog timer and supports checking the timer rollover status. Note that when the processor is in emulation mode, the watchdog timer counter will not decrement even if it is enabled.

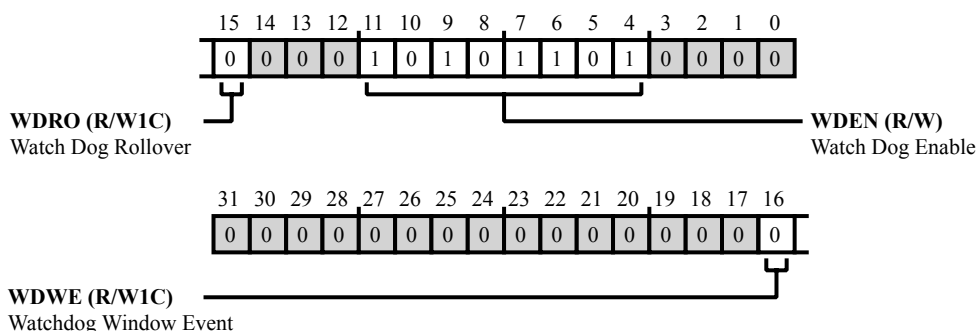


Figure 12-3: WDOG_CTL Register Diagram

Table 12-7: WDOG_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
16 (R/W1C)	WDWE	Watchdog Window Event. Software can determine whether the timer has been serviced unexpectedly early by interrogating the <code>WDOG_CTL.WDWE</code> status bit. This is a sticky bit that is set whenever the watchdog is serviced and the <code>WDOG_STAT</code> value is greater than the <code>WDOG_WIN</code> value. It is cleared only by disabling the watchdog timer and by writing a 1 to the bit.
		0 Window Event has not occurred
		1 Window Event has occurred
15 (R/W1C)	WDR0	Watch Dog Rollover. Software can determine whether the timer has rolled over by interrogating the <code>WDOG_CTL.WDR0</code> status bit. This is a sticky bit that is set whenever the watch dog timer count reaches 0 and cleared only by disabling the watch dog timer and then writing a 1 to the bit.
		0 WDT Has Not Expired
		1 WDT Has Expired
11:4 (R/W)	WDEN	Watch Dog Enable. The <code>WDOG_CTL.WDEN</code> field is used to enable and disable the watchdog timer. Writing any value other than the disable value into this field enables the watchdog timer. This multi-bit disable key minimizes the chance of inadvertently disabling the watchdog timer.
		173 Counter Disabled. All other values mean that the counter is enabled.

Watchdog Timer Status Register

The `WDOG_STAT` register contains the current count value of the watchdog timer. Reads of this register return the current count value. When the watchdog timer is enabled, the `WDOG_STAT` register is decremented by 1 on each SCLK cycle. When the count value reaches 0, the watchdog timer stops counting, and the expiry event is generated. The `WDOG_STAT` register is a 32-bit unsigned system MMR that must be accessed with 32-bit reads and writes.

Values cannot be stored directly in this register but are instead copied from the `WDOG_CNT` register. This copy process can happen in two ways:

- While the watchdog timer is disabled, writing the `WDOG_CNT` register pre-loads the `WDOG_STAT` register.
- While the watchdog timer is enabled, writing the `WDOG_STAT` register loads it with the value in the `WDOG_CNT` register.
- While the watchdog timer is disabled, writing to the `WDOG_STAT` register also reloads it with the value in the `WDOG_CNT` register.

When the processor executes a write (of an arbitrary value) to the `WDOG_STAT` register, the value in the `WDOG_CNT` register is copied into the `WDOG_STAT` register. Typically, software sets the value of `WDOG_CNT` at initialization, then periodically writes to the `WDOG_STAT` register before the watchdog timer expires. This reloads the watchdog timer with the value from `WDOG_CNT` and prevents generation of the expiry event.

If the program does not reload the counter before SCLK x count register cycles, an expiry event is generated, and the `WDOG_CTL.WDRO` bit is set. When this happens, the counter stops decrementing and remains at zero. If the counter is enabled with a zero loaded to the counter, the `WDOG_CTL.WDRO` bit is set immediately and the counter remains at zero and does not decrement.

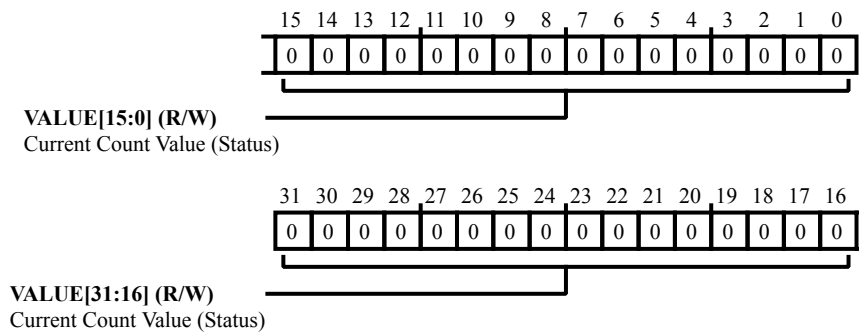


Figure 12-4: WDOG_STAT Register Diagram

Table 12-8: WDOG_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Current Count Value (Status). The <code>WDOG_STAT.VALUE</code> bit field contains the current count value of the watchdog timer.

Watchdog Timer Window Register

Watchdog window register holds the unsigned window value programmed. The window register can be programmed while the WDOG is disabled; any write to the register when WDOG is enabled doesn't alter the contents of the register. When the WDOG is enabled and the core services the WDOG by doing a write to `WDOG_STAT` register while the `WDOG_STAT` value is greater than `WDOG_WIN`, the WDWE event is generated and also the counter stops decrementing.

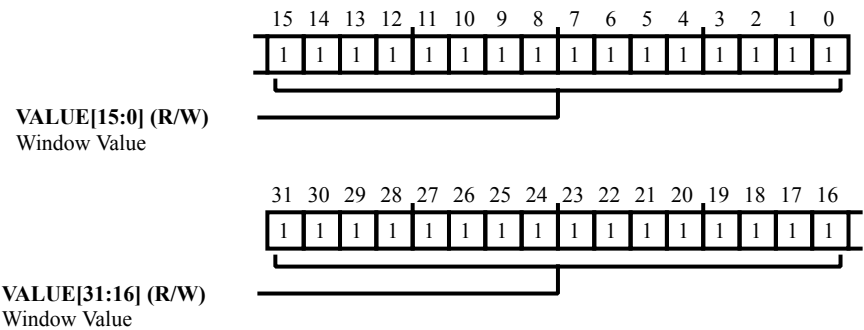


Figure 12-5: WDOG_WIN Register Diagram

Table 12-9: WDOG_WIN Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Window Value. The WDOG_WIN.VALUE bit field contains the unsigned window value.

13 Serial Peripheral Interface (SPI)

The serial peripheral interface is an industry-standard synchronous serial link that supports communication with multiple SPI-compatible devices. The baseline SPI peripheral is a synchronous, four-wire interface consisting of two data pins, one device select pin, and a gated clock pin. The two data pins allow full-duplex operation to other SPI-compatible devices. Two extra (optional) data pins are provided on specific SPIs to support quad SPI operation. Enhanced modes of operation such as flow control, fast mode, and dual-I/O mode (DIOM) are also supported. In addition, a direct memory access (DMA) mode allows for transferring several words with minimal CPU interaction.

With a range of configurable options, the SPI ports provide a glueless hardware interface with other SPI-compatible devices in controller mode, target mode, and multi-controller environments. The SPI peripheral includes programmable baud rates, clock phase, and clock polarity. The peripheral can operate in a multi-controller environment by interfacing with several other devices, acting as either a controller device or a target device. In a multi-controller environment, the SPI peripheral uses open-drain outputs to avoid data bus contention. The flow control features enable slow target devices to interface with fast controller devices by providing an SPI ready pin which flexibly controls the transfers.

NOTE: SPI peripherals on the processor operate in the CLKO6 domain. For more details on CLKO6 programming, refer the *Clock Generation Unit (CGU)* chapter.

SPI Features

The SPI module supports the following features:

- Full-duplex, synchronous serial interface
- 8, 16-bit and 32-bit word sizes
- Programmable baud rate, clock phase, and polarity
- Programmable interframe latency
- Flow control
- Support for Fast and DIOM modes
- SPI1 and SPI2 support quad mode and memory-mapped mode.
- Independent receive and transmit DMA channels

- Burst transfer mode for non-DMA write accesses

SPI Functional Description

This section provides information on the function of the SPI module.

Shift register functionality

The SPI is a shift register that serially transmits and receives data bits to or from other SPI devices. During an SPI transfer, data is simultaneously transmitted (shifted out serially) and received (shifted in serially). A serial clock line synchronizes shifting and sampling of the information on the two serial data lines.

Controller/target functionality

During a data transfer, one SPI system acts as the link controller which controls the data flow. The other system acts as the target, which has data shifted into and out of it by the controller. Different devices can take turn being controllers, and one controller can simultaneously shift data into multiple targets (broadcast mode). However, only one target can drive its output to write data back to the controller at any given time. This rule must be enforced in the broadcast mode. Several targets can be selected to receive data from the controller in this mode. But, only one target can be enabled to send data back to the controller.

Enhanced operating modes

SPI supports enhanced modes of operation like fast mode, DIOM, and Quad-SPI, and optional flow control. In fast mode, received data is sampled on the transmit edge instead of the standard receive edge, thus enabling a full-cycle path for the received data. In DIOM, both MOSI and MISO are configured as input or output pins, and 2 bits are shifted in or out on each receive or transmit edge. In Quad-SPI mode, SPI_D3:0 are configured as input or output pins and 4 bits are shifted in or out on each receive or transmit edge. A slower target can use flow control to stall a faster controller device.

Single and multi-controller use

The SPI can be used in a single controller as well as multi-controller environment. The `SPI_MOSI`, `SPI_MISO`, and the `SPI_CLK` signals are all tied together in both configurations. SPI transmission and reception can be enabled simultaneously or individually, depending on `SPI_RXCTL` and `SPI_TXCTL` settings. In broadcast mode, several target can be enabled to receive, but only one target must be in transmit mode and driving the `SPI_MISO` line.

ADSP-21560_21561_21564_21568 SPI Register List

The Serial Peripheral Interface (SPI) provides a full-duplex, synchronous serial interface, which supports both master/slave modes and multi-master environments. The SPI's baud rate and clock phase/polarities are programmable, and it has integrated DMA channels for both transmit and receive data streams. A set of registers governs SPI operations. For more information on SPI functionality, see the SPI register descriptions.

Table 13-1: ADSP-21560_21561_21564_21568 SPI Register List

Name	Description
SPI_CLK	Clock Rate Register
SPI_CTL	Control Register
SPI_DLY	Delay Register
SPI_ILAT	Masked Interrupt Condition Register
SPI_ILAT_CLR	Masked Interrupt Clear Register
SPI_IMSK	Interrupt Mask Register
SPI_IMSK_CLR	Interrupt Mask Clear Register
SPI_IMSK_SET	Interrupt Mask Set Register
SPI_MMRDH	Memory Mapped Read Header
SPI_MMTOP	SPI Memory Top Address
SPI_RFIFO	Receive FIFO Data Register
SPI_RWC	Received Word Count Register
SPI_RWCR	Received Word Count Reload Register
SPI_RXCTL	Receive Control Register
SPI_SLVSEL	Slave Select Register
SPI_STAT	Status Register
SPI_TFIFO	Transmit FIFO Data Register
SPI_TWC	Transmitted Word Count Register
SPI_TWCR	Transmitted Word Count Reload Register
SPI_TXCTL	Transmit Control Register

ADSP-21560_21561_21564_21568 SPI Interrupt List

Table 13-2: ADSP-21560_21561_21564_21568 SPI Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
69	SPI2_TXDMA	SPI2 TX DMA Channel	Level	26
70	SPI2_RXDMA	SPI2 RX DMA Channel	Level	27
71	SPI2_STAT	SPI2 Status	Level	
72	SPI2_ERR	SPI2 Error	Level	
85	SPI0_TXDMA	SPI0 TX DMA Channel	Level	22
86	SPI0_RXDMA	SPI0 RX DMA Channel	Level	23

Table 13-2: ADSP-21560_21561_21564_21568 SPI Interrupt List (Continued)

Interrupt ID	Name	Description	Sensitivity	DMA Channel
87	SPI0_STAT	SPI0 Status	Level	
88	SPI0_ERR	SPI0 Error	Level	
89	SPI1_TXDMA	SPI1 TX DMA Channel	Level	24
90	SPI1_RXDMA	SPI1 RX DMA Channel	Level	25
91	SPI1_STAT	SPI1 Status	Level	
92	SPI1_ERR	SPI1 Error	Level	
156	SPI2_TXDMA_ERR	SPI2 TX DMA Channel Error	Level	
157	SPI2_RXDMA_ERR	SPI2 RX DMA Channel Error	Level	
158	SPI0_TXDMA_ERR	SPI0 TX DMA Channel Error	Level	
159	SPI0_RXDMA_ERR	SPI0 RX DMA Channel Error	Level	
160	SPI1_TXDMA_ERR	SPI1 TX DMA Channel Error	Level	
161	SPI1_RXDMA_ERR	SPI1 RX DMA Channel Error	Level	

ADSP-21560_21561_21564_21568 SPI Trigger List

Table 13-3: ADSP-21560_21561_21564_21568 SPI Trigger List Generators

Trigger ID	Name	Description	Sensitivity
33	SPI0_TXDMA	SPI0 TX DMA Channel	Edge
34	SPI0_RXDMA	SPI0 RX DMA Channel	Edge
35	SPI1_TXDMA	SPI1 TX DMA Channel	Edge
36	SPI1_RXDMA	SPI1 RX DMA Channel	Edge
37	SPI2_TXDMA	SPI2 TX DMA Channel	Edge
38	SPI2_RXDMA	SPI2 RX DMA Channel	Edge

Table 13-4: ADSP-21560_21561_21564_21568 SPI Trigger List Receivers

Trigger ID	Name	Description	Sensitivity
36	SPI0_TXDMA	SPI0 TX DMA Channel	Pulse
37	SPI0_RXDMA	SPI0 RX DMA Channel	Pulse
38	SPI1_TXDMA	SPI1 TX DMA Channel	Pulse
39	SPI1_RXDMA	SPI1 RX DMA Channel	Pulse
40	SPI2_TXDMA	SPI2 TX DMA Channel	Pulse

Table 13-4: ADSP-21560_21561_21564_21568 SPI Trigger List Receivers (Continued)

Trigger ID	Name	Description	Sensitivity
41	SPI2_RXDMA	SPI2 RX DMA Channel	Pulse

ADSP-21560_21561_21564_21568 SPI DMA Channel List

Table 13-5: ADSP-21560_21561_21564_21568 SPI DMA Channel List

DMA ID	DMA Channel Name	Description
DMA22	SPI0_TXDMA	SPI0 TX DMA Channel
DMA23	SPI0_RXDMA	SPI0 RX DMA Channel
DMA24	SPI1_TXDMA	SPI1 Transmit DMA
DMA25	SPI1_RXDMA	SPI1 RX DMA Channel
DMA26	SPI2_TXDMA	SPI2 TX DMA Channel
DMA27	SPI2_RXDMA	SPI2 RX DMA Channel

SPI Block Diagram

The *SPI Controller Block Diagram* illustrates the blocks of the SPI module. The module is comprised of three primary parts:

- SPI core contains the receive and transmit FIFOs and their associated shift registers
- Control blocks contain the synchronizer and logic to control the data flow through the data pipelines
- Register block

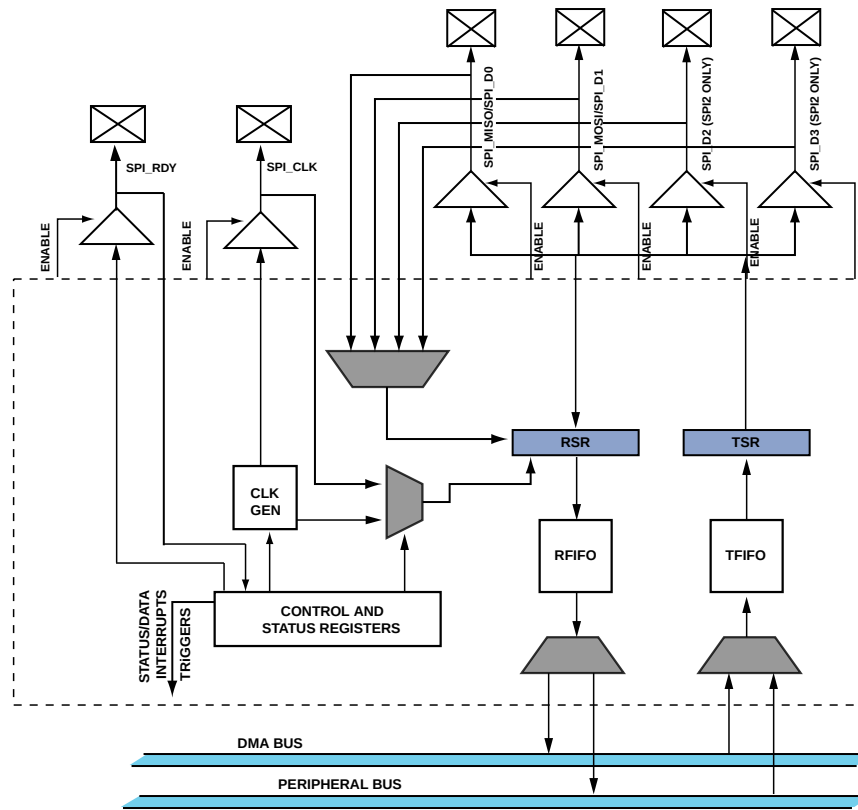


Figure 13-1: SPI Controller Block Diagram, Quad Mode

Transfer Protocol

The SPI module implements two channels that are independent of each other. The SPI module uses the [SPI_RXCTL](#) and [SPI_TXCTL](#) dedicated control registers to control these channels. Except in dual and quad modes, SPI can enable and use both channels simultaneously.

The SPI protocol supports four different combinations of serial clock phase and polarity. These combinations are selected through the [SPI_CTL.CPOL](#) and [SPI_CTL.CPHA](#) bits.

The *SPI Transfer Protocol* figures demonstrate the two basic transfer formats as defined by the [CPHA](#) bit. Two waveforms are shown for [SPI_CLK](#); one for [SPI_CTL.CPOL=0](#) and the other for [SPI_CTL.CPOL=1](#). The diagrams can be interpreted as controller or target timing diagrams since the [SPI_CLK](#), [SPI_MISO](#), and [SPI_MOSI](#) pins are directly connected between the controller and the target. The [SPI_MISO](#) signal is the output from the target (target transmission), and the [SPI_MOSI](#) signal is the output from the controller (controller transmission). The controller generates the [SPI_CLK](#) signal. The [SPI_SS](#) signal is the target device select input to the target from the controller. The diagrams represent an 8-bit transfer ([SPI_CTL.SIZE=0](#)) with the MSB first ([SPI_CTL.LSBF=0](#)). Any combination of the [SPI_CTL.SIZE](#) and [SPI_CTL.LSBF](#) bits is permissible. For example, a 16-bit transfer with the LSB first is another possible configuration.

The clock polarity and the clock phase could be identical for the controller device and the target device involved in the communication link. The transfer format from the controller can be changed between transfers to adjust for various requirements of a target device.

The SPI module uses the `SPI_CTL.ASSEL` bit to determine when the SPI hardware or software control the `SPI_SEL[n]` line. When `SPI_CTL.ASSEL=1`, the target select line must be set to the polarity set in the `SPI_CTL.SELST` field between each serial transfer. The actual behavior of `SPI_SEL[n]` also depends on the parameters programmed into the `SPI_DLY` register. The SPI hardware logic automatically controls this functionality. When `SPI_CTL.ASSEL=0`, `SPI_SEL[n]` can either remain active between successive transfers or be inactive. The software must control this activity through manipulation of the `SPI_SLVSEL` register.

The *SPI Transfer Protocol* pair of figures illustrates the case when `SPI_CTL.ASSEL = 1` and the `SPI_SEL[n]` line is inactive between frames. If `ASSEL = 0`, the `SPI_SEL[n]` line can remain active between frames; however, the first bit is only driven when an active transition of `SPI_CLK` occurs.

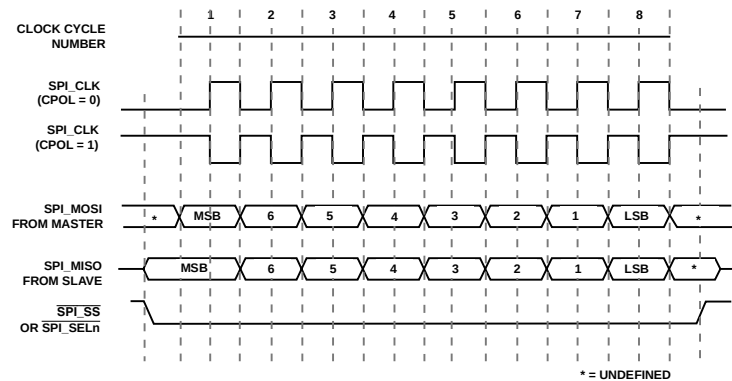


Figure 13-2: SPI Transfer Protocol for CPHA=0

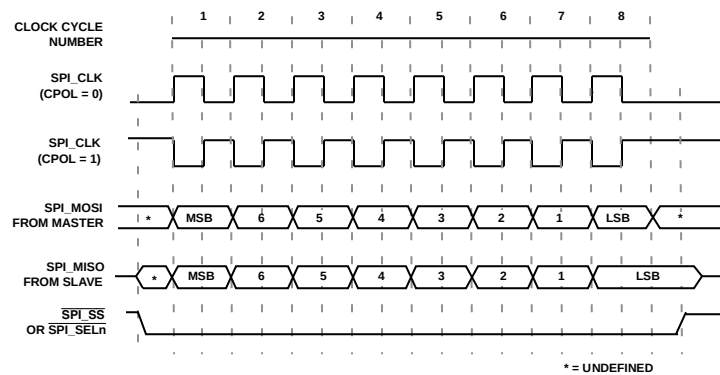


Figure 13-3: SPI Transfer Protocol for CPHA=1

Clock Considerations

The `SPI_CLK` signal is a gated clock that is only active during data transfers, for the time of the transferred word. In normal mode, the number of active edges is equal to the number of bits to be transmitted or received. In dual-I/O mode, it is half of the number of bits to be transmitted or received, and in quad-SPI mode it is one-fourth of the number. The clock rate can be derived using both even and odd dividers from `SCLK`.

For controller devices, the SPI uses the `SPI_CLK` register value to determine the clock rate, whereas this value is ignored for target devices.

When the SPI controller is a controller, `SPI_CLK` is an output signal. Conversely, when the SPI controller is a target, `SPI_CLK` is an input signal. Peripheral devices ignore the SPI clock when the target select input is driven inactive. The SPI uses the `SPI_CLK` signal to shift out and shift in the data driven onto the `SPI_MISO` and `SPI_MOSI` lines. The data is always shifted out on one edge of the clock (the active edge) and sampled on the opposite edge of the clock (the sampling edge). Clock polarity and clock phase relative to data are programmable through the `SPI_CTL` register and define the transfer format.

Controlling Delay Between Frames

The *SPI Timing with Lead and Lag Programming (Independent of SPI_CTL.CPHA Setting)* figure illustrates SPI timing using the `SPI_DLY.LEADX` and `SPI_DLY.LAGX` programming. The SPI uses the `SPI_DLY.LAGX` bits to control the timing between the chip select (`SPI_SS`) signal assertion and the first `SPI_CLK` edge. The SPI uses the `SPI_DLY.LEADX` bits to control the timing between the last `SPI_CLK` edge and deassertion of the `SPI_SS` signal. The lead and lag timing can be extended by a 1 `SPI_CLK` duration to ease timing restrictions on the target device.

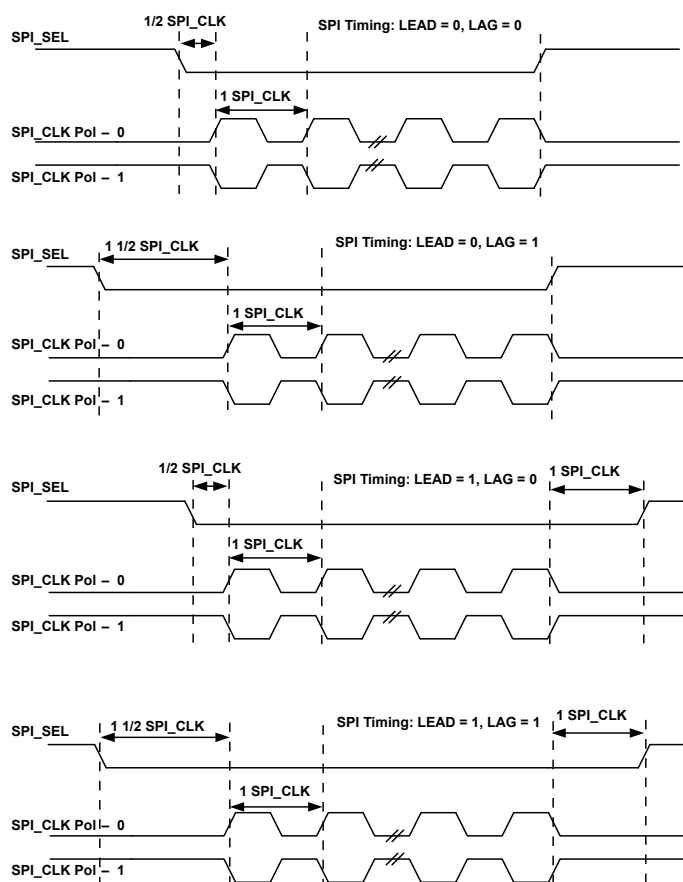


Figure 13-4: SPI Timing with Lead and Lag Programming (Independent of SPI_CTL.CPHA Setting)

The *SPI Timing with SPI_DLY.STOP Programming (Independent of SPI_CTL.CPHA Setting)* figure illustrates SPI timing with STOP programming. The SPI module uses this timing to insert multiples of SPI_CLK period delays between transfers. The $\overline{\text{SPI_SS}}$ line is deasserted for the duration specified in the SPI_DLY.STOP bit field, assuming the SPI_CTL.SELST bit is configured for deassertion between transfers.

If the SPI_DLY.STOP bit = 0, the controller operates in a *continuous mode*. This mode causes an immediate start of the second word after the last bit is transferred from the first word. During this mode of operation, the chip select line is continuously asserted.

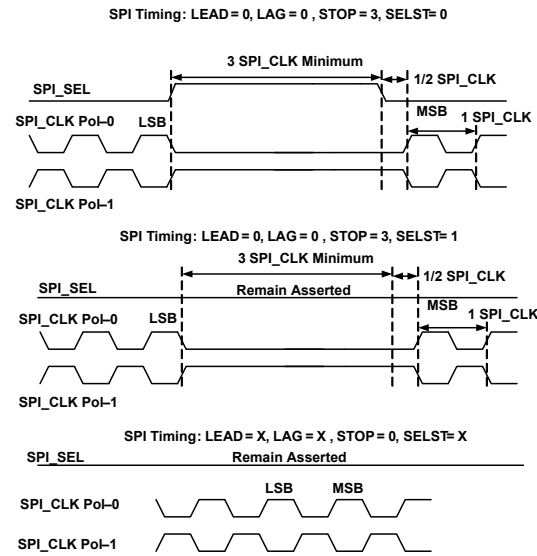


Figure 13-5: SPI Timing with SPI_DLY.STOP Programming (Independent of SPI_CTL.CPHA Setting)

When the `SPI_DLY.STOP` bit = 0 and initial conditions for a transfer are not met, the interface pauses before the next transfer. During this pause, the SPI uses the `SPI_CTL.SELST` bit to determine the state of the chip select pin. The SPI uses the `SPI_DLY.LEADX` and `SPI_DLY.LAGX` bits to determine the timing between `SPI_CLK` edges and the chip select line.

Flow Control

In controller mode, the target device must drive the `SPI_RDY` pin. The pin acts as an input signal. The target can deassert the `SPI_RDY` pin to stop the controller from initiating any new transfer. If `SPI_RDY` is deasserted in the middle of a transfer, the current transfer continues, and the next transfer will not start unless the target asserts the `SPI_RDY` signal. Whenever the target deasserts `SPI_RDY` and stalls the controller, the SPI controller goes into a waiting state, and the `SPI_STAT.FCS` bit is set. When the target asserts `SPI_RDY`, the SPI controller resumes operation, and the `SPI_STAT.FCS` bit is cleared.

In target mode, the `SPI_RDY` pin acts as an output signal. Flow control can be configured on either the TX channel or the RX channel. The SPI uses the `SPI_CTL.FCCH` bit to control this configuration. If flow control is configured on the TX channel, as the `SPI_TFIFO` status nears the empty condition, the `SPI_RDY` pin is deasserted. If flow control is configured on the RX channel, as the `SPI_RFIFO` status nears the full condition, the `SPI_RDY` pin is deasserted. The SPI uses the `SPI_CTL.FCWM` bits to control the FIFO status at which `SPI_RDY` deassertion takes place. Flow control in target mode is purely based on the FIFO status and does not depend on the word counters.

The *SPI Flow Control Timing in Controller Mode* figure illustrates this timing.

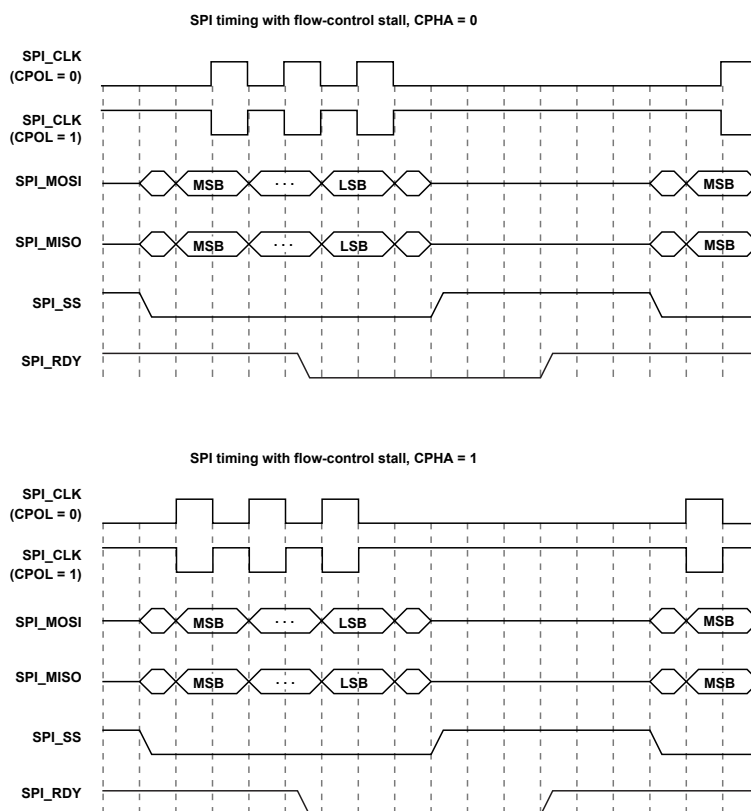


Figure 13-6: SPI Flow Control Timing in Controller Mode.

Target Select Operation

If the SPI is in target mode, `SPI_SS` acts as the target select input. When SPI is enabled as a controller, `SPI_SS` can serve as an error detection input for the SPI in a multi-controller environment. The `SPI_CTL.PSSE` bit enables this feature. When `SPI_CTL.PSSE=1`, the `SPI_SS` input is the controller mode error input. Otherwise, `SPI_SS` is ignored.

The `SPI_SS` signal is an active-low signal. The controller asserts the signal during the transfer. The signal can be deasserted or remain asserted between transfers. When `SPI_SS` is deasserted, `SPI_CLK` and inputs are ignored, and outputs are three-stated.

The target select bits (`SPI_SLVSEL.SSEL1 – SPI_SLVSEL.SSEL7`) are used in a multiple-target SPI environment. For example, if there are eight SPI devices in the system including a processor controller, the controller processor can support the SPI mode transactions across the other seven devices. This configuration requires only one controller processor in this multi-target environment.

For example, assume that the SPI of the processor is the controller. The `SPI_SLVSEL.SSEL1 – SPI_SLVSEL.SSEL7` bits on the processor can be connected to the target select pin of each target device. In this configuration, the target select bits can be used in three ways. In cases 1 and 2, the processor is the controller and the seven microcontrollers or targets with SPI interfaces are targets. The processor can do one of the following:

1. Transmit to all seven SPI devices at the same time in a broadcast mode. Here, all target select bits are set.

2. Receive and transmit from one SPI device by enabling only one target SPI device at a time.
3. If all the targets are also processors, then the requester can receive data from only one processor at a time. (The functionality is enabled by clearing the `SPI_CTL.EMISO` bit in the six other slave processors.) The requestor can transmit broadcast data to all seven at the same time. This MISO enabling feature is available in some other microcontrollers. Therefore, it is possible to use the MISO enabling feature with any other SPI device that includes this functionality.

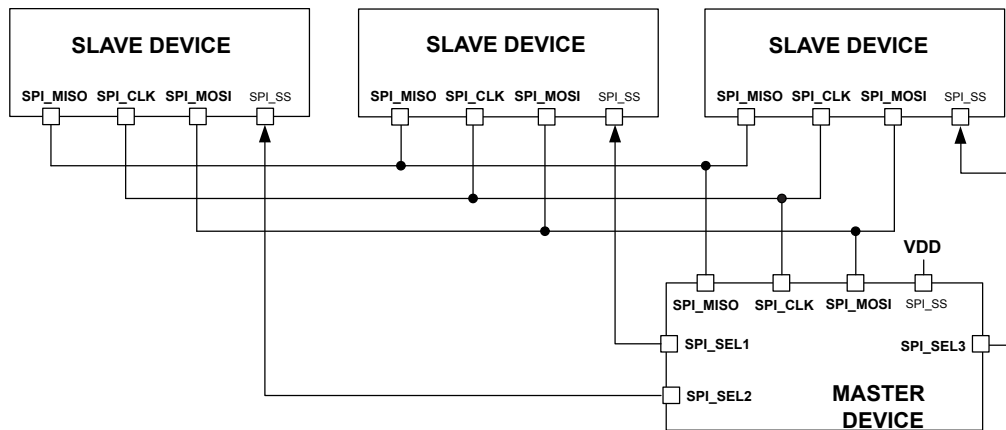


Figure 13-7: Single-Controller, Multiple-Target Configuration

Beginning and Ending a Non-DMA SPI Transfer

The start and finish of a non-DMA SPI transfer depend on the following settings.

1. Whether the device is configured as a controller or a target.
2. The state of the `SPI_CTL.ASSEL` bit, which selects between hardware and software control over `SPI_SLVSEL`.

When `SPI_CTL.CPHA=0`, the enabled target select outputs are driven active. However, the `SPI_CLK` signal remains inactive for the first half of the first cycle of `SPI_CLK`. For a target with `SPI_CTL.CPHA=0`, the transfer starts as soon as the `SPI_SS` input goes low.

When `SPI_CTL.CPHA=1`, a transfer starts with the first active edge of `SPI_CLK` for both target and controller devices. For a controller device, a transfer is complete after it sends the last data and simultaneously receives the last data bit. A transfer for a target device ends after the last sampling edge of `SPI_CLK`. If `SPI_CTL.ASSEL=1`, the hardware maintains responsibility for toggling `SPI_SS` between frames. If `SPI_CTL.ASSEL=0`, software controls the `SPI_SS` line and can keep it active between frames.

The `SPI_STAT.RFE` bit defines when the receive buffer can be read, indicating that `SPI_RFIFO` is not empty. The `SPI_STAT.TFF` bit defines when the transmit buffer can be written, indicating that the `SPI_TFIFO` is not full. The end of a single word transfer occurs when the `SPI_STAT.RFE` bit is cleared. The status indicates that a new word has been received and written into the receive FIFO. The `SPI_STAT.RFE` bit remains cleared when the receive FIFO has valid data.

To maintain software compatibility with other SPI devices, the `SPI_STAT.SPIF` bit is also available for polling. This bit can have a slightly different behavior from other commercially available devices.

In controller mode with the `SPI_CTL.ASSEL` bit cleared, software manually asserts the required target select signal before starting the transaction. After all data transfers, software typically releases the target select line.

When the receive or transmit word counters are enabled in the `SPI_TXCTL` or `SPI_RXCTL` registers, the SPI generates a finish interrupt at the end of the transfer. It signals the end of all transfers related to that transaction.

Transmit Operation in Non-DMA Mode

The transmit operation in non-DMA mode is enabled through the `SPI_TXCTL.TEN` bit. It can be enabled independently from the receive operation, and the transmit channel can become the initiating channel based on the `SPI_TXCTL.TTI` bit setting.

Transmit underrun is not possible in this mode, as no new transfer initiates unless the transmit FIFO is empty (in the case that `SPI_TXCTL.TTI = 1`). A receive overflow is detected when data from a new frame transfer replaces older data in a full receive FIFO. This event can occur if `SPI_TXCTL.TTI = 1` and the receive channel is enabled in a non-initiating capacity.

An SPI transmit interrupt request is signaled once the transmit channel has been enabled and the transmit FIFO is not full. The SPI uses the `SPI_TXCTL.TDR` bit setting to control the frequency of the interrupt request.

Receive Operation in Non-DMA Mode

The receive operation in non-DMA mode is enabled through the `SPI_RXCTL.REN` bit. It can be enabled independently from the transmit operation, and the receive channel can become the initiating channel based on the `SPI_RXCTL.RTI` bit setting.

Receive overflow is not possible in this mode, as no new transfer initiates when the receive FIFO is full (in the case of `SPI_RXCTL.RTI = 1`). A transmit underrun can occur (`SPI_TXCTL.TDU` bit) when no valid data is in the `SPI_TFIFO` register when a transfer is initiated. This event can occur if `SPI_RXCTL.RTI = 1` and the transmit channel is enabled in a non-initiating capacity.

An SPI receive interrupt request is signaled once the receive channel has been enabled and there is data waiting to be read. The SPI uses the `SPI_RXCTL.RDR` bit setting to control the frequency of the interrupt request.

Dual I/O Mode

In Dual I/O mode, the `SPI_MISO` and `SPI_MOSI` pins are configured to operate in the same direction which doubles bandwidth. The SPI uses the `SPI_CTL.SOSI` bit to determine the order of bits on the pins. When set, the processor sends the first bit on the `SPI_MOSI` pin and the second bit on the `SPI_MISO` pin. If the `SPI_CTL.SOSI` bit is cleared, the order is reversed. Since dual I/O mode uses both pins to transmit or receive data, only one channel can be enabled, either transmit or receive. Flow control through the `SPI_RDY` pin is supported. Interrupt request generation is unaffected by dual I/O mode. However, the interrupt service interval is reduced because the individual transfer latency is halved.

Changing to quad SPI mode must be done when the SPI is in a quiescent state.

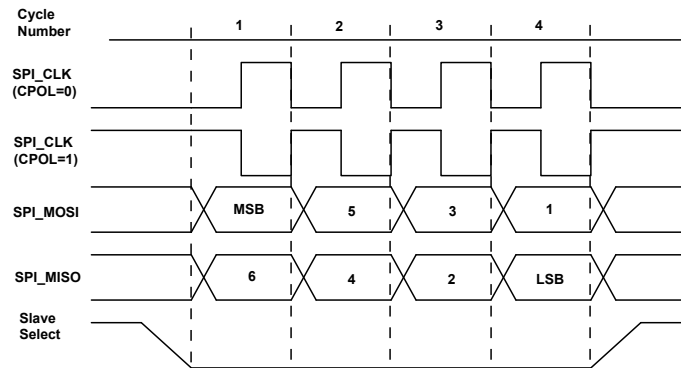


Figure 13-8: Dual I/O Mode Transfer Protocol for CPHA=0, SOSI=1, 8-Bit Transfer, LSBF=0.

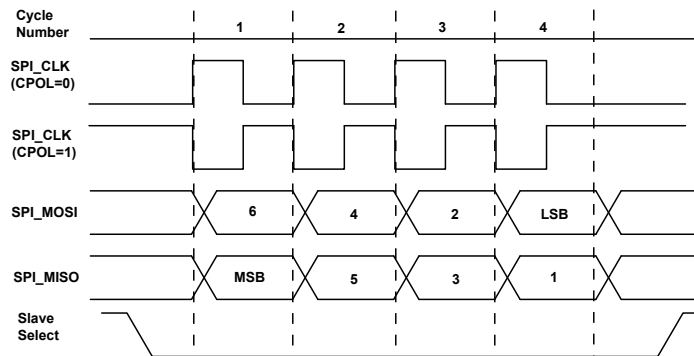


Figure 13-9: Dual I/O Mode Transfer Protocol for CPHA=1, SOSI=0, 8-Bit Transfer, LSBF=0.

Quad I/O Mode (SPI1/SPI2)

In quad SPI mode, the `SPI_MISO` and `SPI_MOSI` pins, in tandem with the `SPI_D2` and `SPI_D3` pins, are configured to operate in the same direction. The SPI uses the `SPI_CTL.SOSI` bit to determine the order of bits on the pins. When set, the processor sends:

- The first bit on the `SPI_MOSI` pin
- The second bit on the `SPI_MISO` pin
- The third bit on the `SPI_D2` pin
- The fourth bit on the `SPI_D3` pin

If the `SPI_CTL.SOSI` bit is cleared, the order is reversed. Since quad SPI mode uses all four pins to transmit or receive data, only one channel can be enabled as either transmit or receive. Flow control through the `SPI_RDY` pin is supported. Interrupt generation is unaffected by quad SPI mode.

Changing to quad SPI mode must be done when the SPI is in a quiescent state.

While using dual or quad I/O mode for communicating with flash devices, program the `SPI_CTL.CPHA` and the `SPI_CTL.CPOL` bits =1. This programming avoids bus contention during read operations, because the flash device starts driving out the bits immediately after dummy cycles in read header.

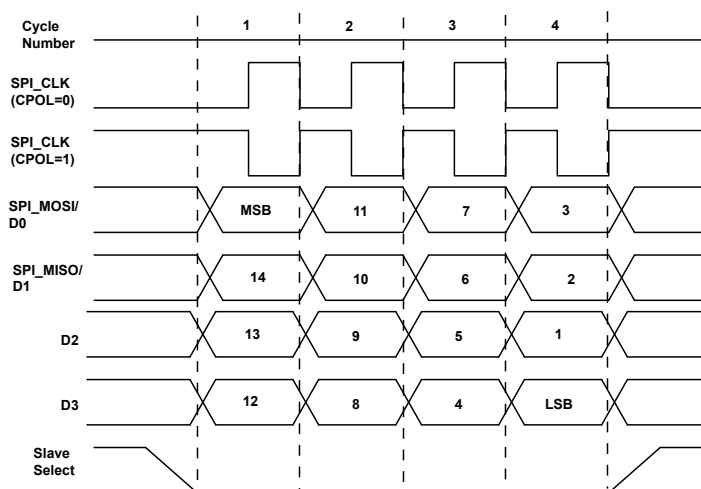


Figure 13-10: Quad Mode Timing for CPHA=0, SOSI=1, 16-Bit Transfer, LSBF=0.

NOTE: The SPI does support quad SPI 8-bit transfer in target continuous mode of operation with an SCLK:SPI_CLK ratio of less than 1:2. A minimum of 2 SCLK cycles is required between transfers in 8-bit quad SPI target mode with an SCLK:SPI_CLK ratio of less than 1:2.

Fast Mode

Fast mode is like the normal mode of operation when transmitting. When receiving, data is sampled at the next transmit edge allowing a full cycle of timing in the receive direction. This mode is valid in controller mode operation only. When the SPI operates in fast mode, the target drives the data for one full cycle.

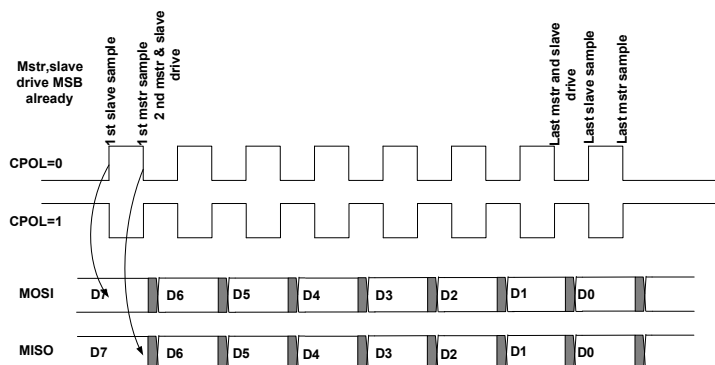


Figure 13-11: SPI Transfer Protocol in Fast Mode for SPI_CTL.CPHA = 0

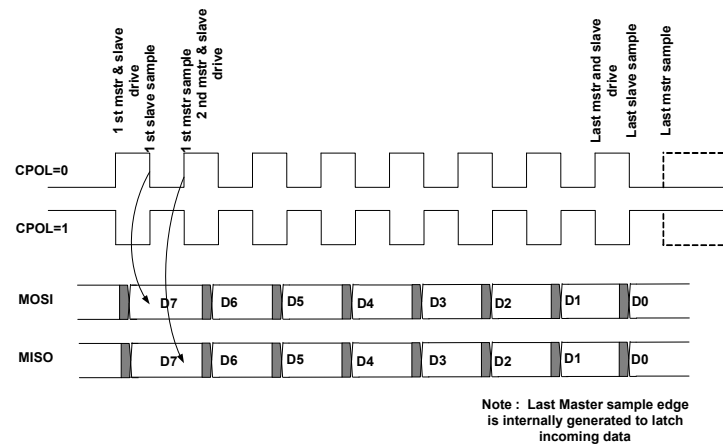


Figure 13-12: SPI Transfer Protocol in Fast Mode for SPI_CTL.CPHA = 1

Memory-Mapped Mode (SPI1/SPI2)

The SPI supports direct memory-mapped read accesses from a SPI memory device, enabled by setting the SPI_CTL.MMSE bit. This mode allows for direct execution of instructions from a SPI memory device without the need for a low-level software driver, as hardware handles all overhead tasks (for example, transmission of the read header, pin turnaround timing, and receive data sizing). The SPI features configurable options in the memory-mapped read header register (SPI_MMRDH) to provide compatibility with a wide range of SPI memory devices.

In non-memory-mapped mode, the software is responsible for providing the command and required dummy words for the read response, whereas this is all handled by hardware when the SPI is in memory-mapped mode. The memory of the SPI device is accessible directly through reads of the processor address space. The read accesses can be code or data accesses in core mode or when using memory DMA (MDMA). These accesses allow code to execute directly from SPI memory devices (true eXecute-In-Place operations), and the contents can be cached to improve performance. It is not necessary to access the SPI data buffer registers nor poll status bits; however, the hardware does not support peripheral DMA accesses nor write operations to the SPI memory space.

The *Types of Operations* table is a comparison of the permitted operations in the non-memory-mapped and memory-mapped modes supported by the SPI controller.

Table 13-6: Types of Operations

SPI Operation	Non-Memory-Mapped Mode	Memory-Mapped Mode
Core data write	Yes	No
Core data read	Yes	Yes
Code fetch: Execute-In-Place (XIP)	No	Yes
Read/Write accesses using SPI peripheral DMA	Yes	No
Read/Write accesses by other peripheral DMA channels	No	No
MDMA read	No	Yes

Table 13-6: Types of Operations (Continued)

SPI Operation	Non-Memory-Mapped Mode	Memory-Mapped Mode
MDMA write	No	No

NOTE: MDMA read* - Only standard bandwidth MDMA (MDMA0 and MDMA1) and Enhanced Bandwidth MDMA (MDMA2) can be used for accessing flash address space in memory mapped mode. The maximum bandwidth MDMA (MDMA3) cannot access flash address space.

Memory-Mapped Description of Operation

Memory-mapped mode is enabled by setting the `SPI_CTL.MMSE` bit. When enabled, the SPI (if ready) accepts the read requests through a dedicated on-chip target interface. The memory subsystem controller drives this dedicated interface through the SCB fabric.

In a typical scenario, the memory subsystem controller issues read requests to the fabric, and the fabric routes these requests to the target port of the SPI peripheral. The controller describes the read access using a number of parameters such as starting address, transfer size, and burst type. The SPI responds to this read access request when it is ready for a new transfer. It loads the opcode, a specified number of address bytes, and an optional mode byte into the transmit FIFO. The SPI memory state machine begins when both the transmit and receive channels of the SPI are enabled with:

- the transmit transfer initiation bit is set (`SPI_TXCTL.TTI=1`), and
- the receive initiation bit is cleared (`SPI_RXCTL.RTI=0`)

The SPI memory read sequence starts with the assertion of `SPI_SEL1`. If the SPI memory state machine is in the reset state, it looks for a command. The SPI hardware then sends the specific 8-bit read command (which can be optionally skipped), followed by the SPI memory read address. After this, a dummy period is inserted, in which a mode byte is optionally sent, and the pins are held or three-stated during the dummy clocking period.

NOTE: This read header is transmitted over the SPI standard protocol pins (`SPI_CLK`, `SPI_MOSI`, `SPI_MISO`, `SPI_SEL1`) or over the extended SPI protocol pins (`SPI_CLK`, `SPI_MOSI`, `SPI_MISO`, `SPI_D2`, `SPI_D3`, `SPI_SEL1`), based on the `SPI_MMRDH.CMDPINS`, `SPI_MMRDH.ADRPINS`, and `SPI_CTL.MIOM` bit settings. SPI memory devices usually support communication in MSB-first mode. In dual mode, the SPI typically uses `SPI_MISO` as IO1 and `SPI_MOSI` as IO0. In quad mode, the SPI typically uses `SPI_D3` pin as IO3, `SPI_D2` as IO2, `SPI_MISO` as IO1, and `SPI_MOSI` as IO0.

When all I/O data pins are three-stated, the SPI continues clocking the SPI memory device, which drives out the data bits at the addressed location, until all bytes are received. The SPI hardware reads the data as configured by the `SPI_CTL.MIOM` bit setting. Upon reception of the last byte, the SPI typically deasserts `SPI_SEL1` to prepare for the next requested read header.

Application code must ensure that the opcode sent is consistent with multiple I/O programming and that the parameters specified in the memory-mapped read header register are consistent with flash read access timing.

The *SPI Memory-Mapped Register Operations Flow* diagram shows how the fields of the `SPI_MMRDH` register determine the read header while initiating transfers in memory-mapped mode.

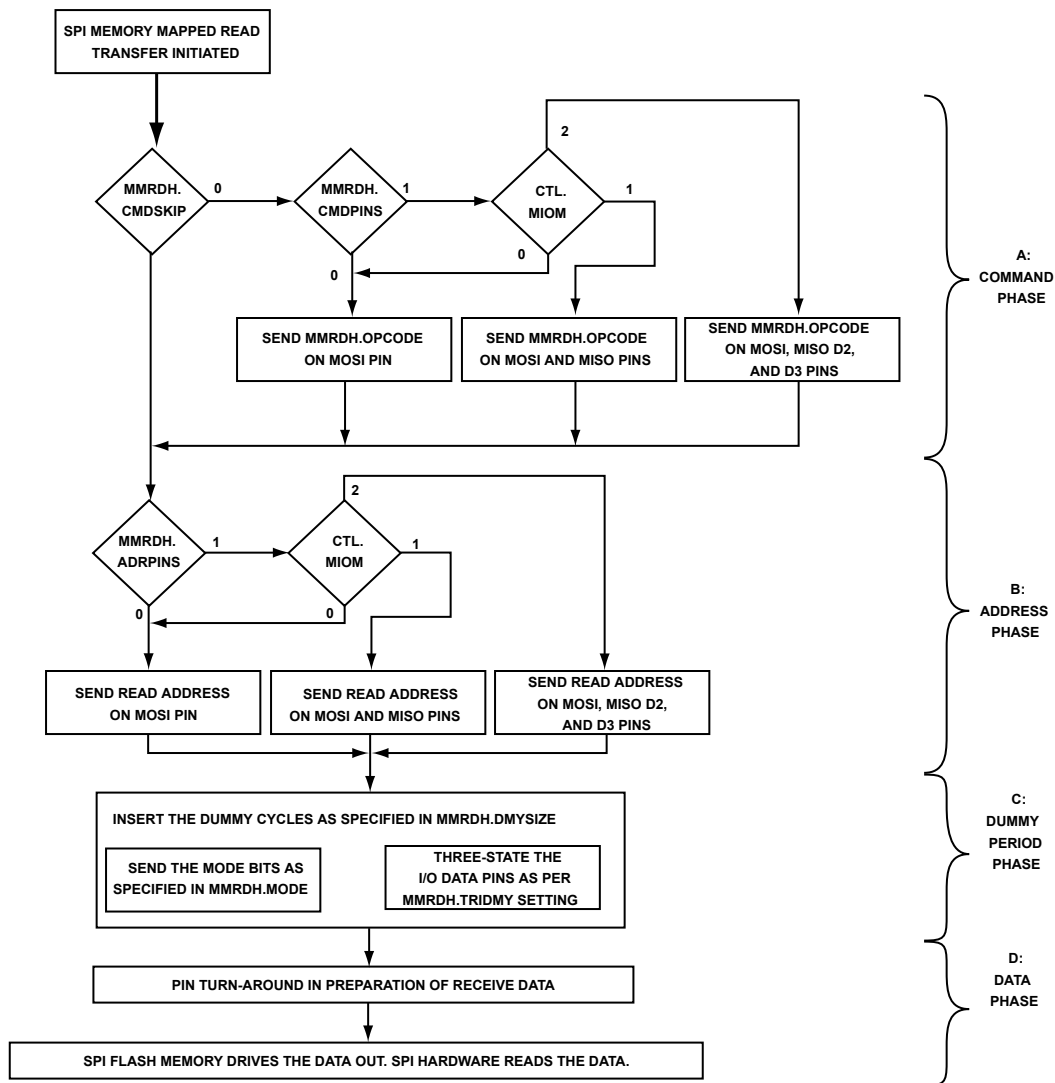


Figure 13-13: SPI Memory-Mapped Register Operations Flow

Memory-Mapped Architectural Concepts

In memory-mapped mode, the SPI accepts read requests through a dedicated on-chip slave interface. The SPI (when ready) accepts these requests and begins the process of assembling the read header based on access attributes described in both the `SPI_MMRDH` register and the internal bus request. After the read header transmission completes, a pin turnaround period is timed, and the receiver is enabled. The SPI continues clocking the SPI memory device until all bytes are received.

The SPI memory-mapped hardware accommodates various memory devices with different read timing. The capabilities include extra mode bits, flexible dummy period timing, and three-state control, as configured in the `SPI_MMRDH` register.

The *Memory-Mapped Protocol* figure shows the protocol for the SPI controller in memory-mapped mode.

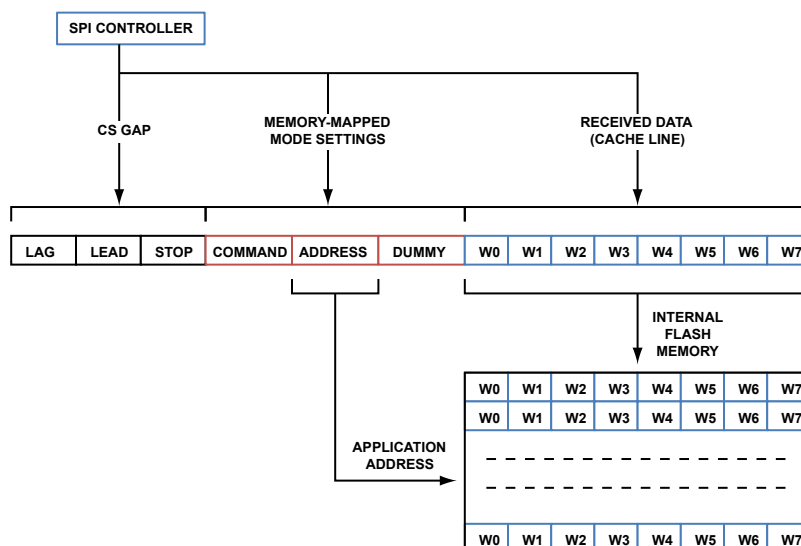


Figure 13-14: Memory-Mapped Protocol

As shown in the figure, the **COMMAND** field (`SPI_MMRDH.OPCODE`) is transmitted upon assertion of the `SPI_SEL[n]` signal. The SPI memory interprets this 8-bit value as a read command. Any 8-bit read opcode whose timing is compliant with the processor SPI and features provided by memory-mapped hardware is allowed, the most common being:

- Standard Read (0x03)
- Fast Read (0x0B)
- Fast Read Dual Output (0x3B)
- Fast Read Dual I/O (0x6B)
- Fast Read Quad Output (0xBB)
- Fast Read Quad I/O (0xEB)

NOTE: The SPI hardware does not validate the content of the `SPI_MMRDH.OPCODE` field prior to transmitting.

DMYSIZE (Number of Dummy Bytes)

When operating at a high clock frequency in multi-IO modes, most flash devices require some dummy clocks after the address bits. These dummy clock cycles allow the internal circuits of the device extra time for setting up the initial address. These bits specify the number of bytes separating address transmission and read data return.

The number of dummy cycles required varies per manufacturer, the read command used, and the SPI access time. The SPI hardware allows dummy cycles to be programmed in bytes in the `SPI_MMRDH.DMYSIZE` field, the value of which is a function of the number of pins used to transmit the address (`SPI_MMRDH.ADRPINS`), as shown in the *Pins Used to Transmit the Address (ADRPINS)* table.

Table 13-7: Pins Used to Transmit the Address (ADRPINS)

SPI_MMRDH. DMYSIZE	Dummy clock cycles		
	(SPI_MMRDH.ADRPINS=0, SPI_CTL.MIOM=x) Dummy bytes elapse over 1-pin	(SPI_MMRDH.ADRPINS=1, SPI_CTL.MIOM=1) Dummy bytes elapse over 2- pins	(SPI_MMRDH.ADRPINS=1, MIOM=2) Dummy bytes elapse over 4- pins
000	0	0	0
001	8	4	2
010	16	8	4
011	24	12	6
100	32	16	8
101	40	20	10
110	48	24	12
111	56	28	14

This dummy clocking period allows the mode bits to be sent, the pins to be three-stated, and the pins to be turned around in preparation for the receive data.

Memory-Mapped Read Accesses

The SPI hardware supports the most used read operations.

- Two standard SPI reads (read and read fast), which use the unidirectional SPI_MOSI and SPI_MISO pins in addition to SPI_SEL[n] and SPI_CLK
- Four extended SPI multiple I/O reads: dual output, quad output, dual I/O, and quad I/O reads

The *SPI Read Operations* table and *SPI Flash Fast Read Sequence* figures summarize the types of read operations. Program each read operation in a way that is compatible with the description given in the SPI flash data sheet.

Table 13-8: SPI Read Operations

Operation	Read Command (Opcode)	CMDPIN	ADRPIN	DMYSIZE	Three-state	Multiple I/O Mode	Data Pins
Read	0x03	1	1	Zero	No	No	1
Fast Read	0x0B	1	1	Non-Zero	Yes	No	1
Dual Output Read	0x3B	1	1	Non-Zero	Yes	Yes(IO0-1)	2
Quad Output Read	0x6B	1	1	Non-Zero	Yes	Yes(IO0-3)	4
Dual I/O Read	0xBB	1, 2	2	Non-Zero	Yes	Yes (IO0-1)	2
Quad I/O Read	0xEB	1, 4	4	Non-Zero	Yes	Yes (IO0-3)	4

Some memory devices also support word quad I/O read (0xE7) and octal quad I/O read (0xE3) operations. These operations require fewer dummy cycles than normal quad I/O read operations.

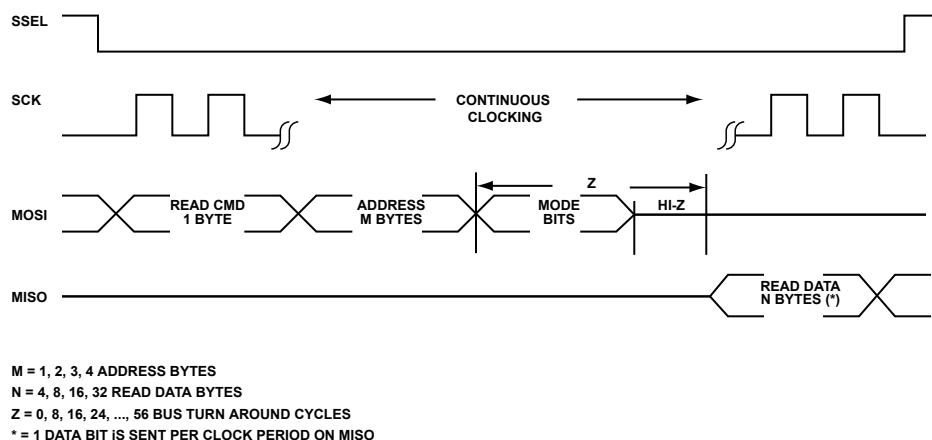


Figure 13-15: SPI Flash Fast Read Sequence

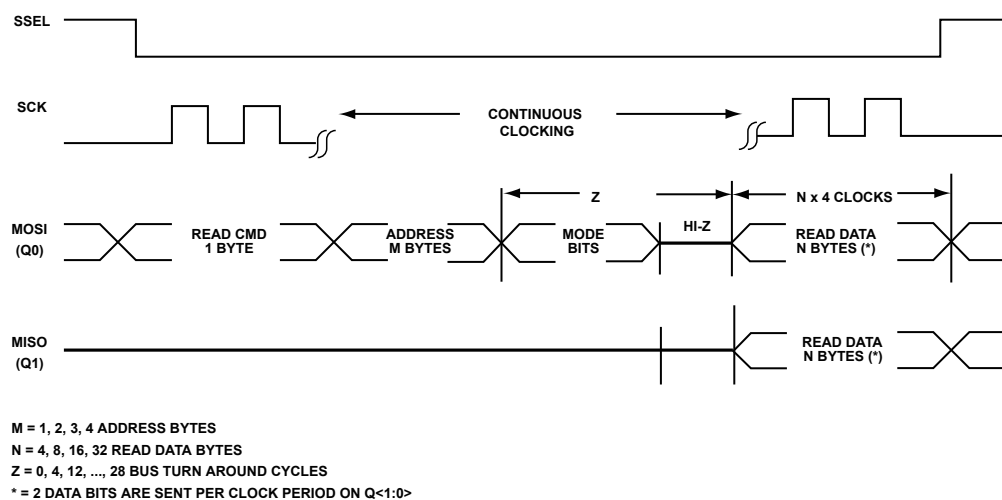


Figure 13-16: SPI Flash Fast Read (Dual Output) Sequence

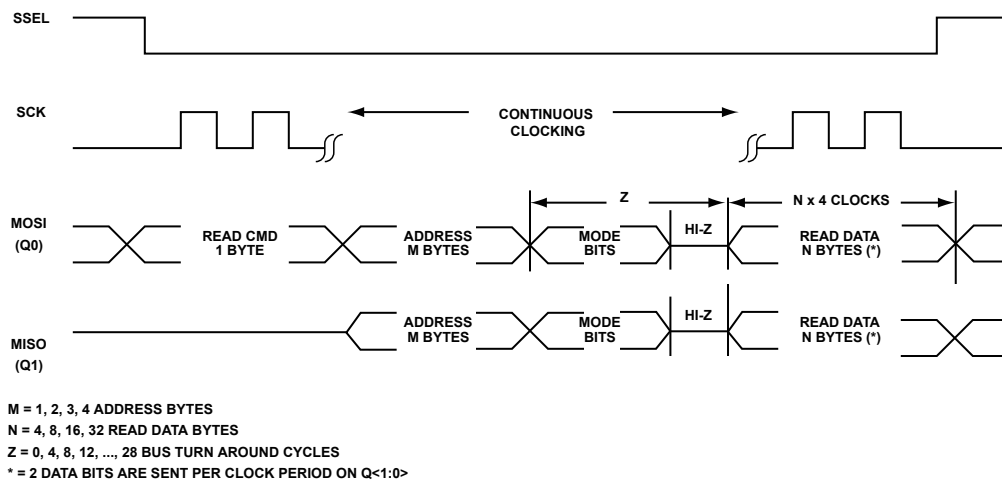


Figure 13-17: SPI Flash Fast Read (Dual I/O) Sequence

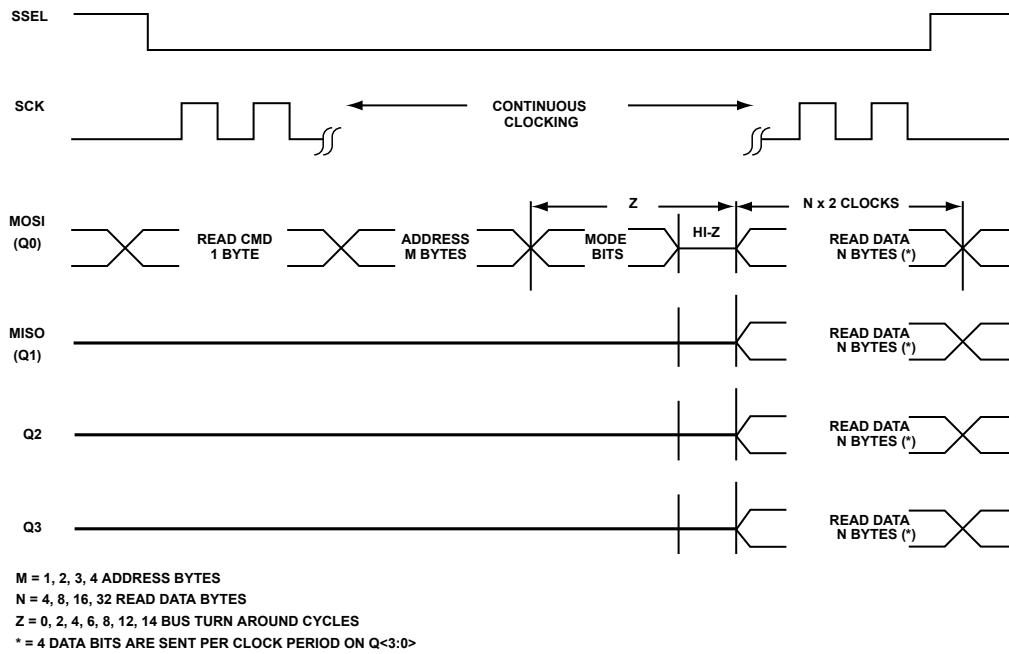


Figure 13-18: SPI Flash Quad Output Read Sequence

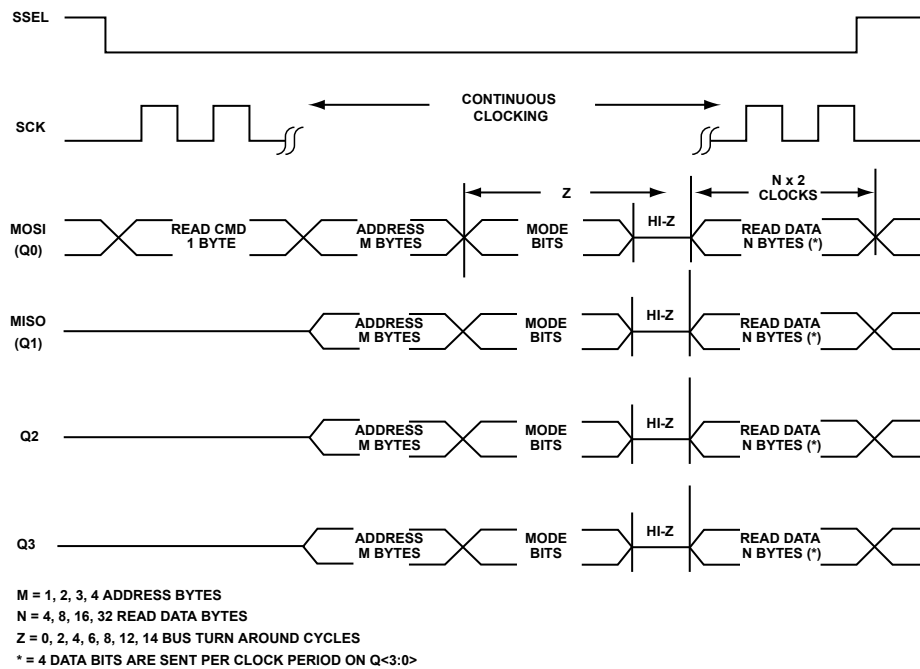


Figure 13-19: SPI Flash Quad I/O Read Sequence

SPI memory-mapped reads can be made cacheable in the core's internal memory by properly configuring the region as cacheable memory without bypass (see the related core's cache configuration documentation for details). In the figures, the number of read data bytes (N) is based on the following:

- For an instruction fetch by core (when in XIP mode); the number of instruction bytes to be fetched depends on the cache line size of the cache.

- For a data fetch by the core (data read), the number of data bytes to be fetched depends on the cache line size of the cache.

Although the minimum size of a memory-mapped data read transfer is 4 bytes, applications can fetch a single byte or a 2-byte data. (For example, it can fetch an unsigned char or short access in C code). In this case, only the required bytes are provided to the core and the other bytes are cached.

The on-chip memory subsystem controller provides a starting address for the burst and the SPI hardware issues this address as part of the read header. The address provided is N-byte aligned. For example, to read the 30th byte from SPI memory, then the typical address to provide is:

- 28 (0x0000_001C) for a 32-bit cache line
- 24 (0x0000_0018) for a 64-bit cache line
- 16 (0x0000_0010) for a 128-bit cache line
- 0 (0x0000_0000) for a 256-bit cache line

The read data is returned to the memory subsystem in the order provided by the SPI memory. There can be considerable delay for the expected data provided to the controller.

To minimize this delay, the wrap feature can be used where the memory subsystem provides the address of the critical word.

- For MDMA reads, the number of read data bytes (N) is always equal to 4 bytes. The MDMA read does not depend on the cache setting. For MDMA reads, limit the `DMA_CFG.MSIZE` field to one, two or four bytes. The address provided by the memory subsystem controller to the SPI hardware is always 4-byte aligned.

Memory-Mapped High-Performance Features

In addition to automating the SPI memory read accesses, the memory-mapped hardware also provides some features to improve SPI memory fetches and increase the system performance. The following sections describe these features.

Merged Read Accesses

It is common for the memory subsystem to fetch two or more cache lines from consecutive addresses (the address sequencing is linear without any jumps). To take advantage of this situation, the SPI memory-mapped hardware provides a feature called merging. Enable merging by setting the `SPI_MMRDH.MERGE` bit.

When enabled, the hardware compares the address of an incoming read request to the address of a request the SPI memory is actively servicing. It can decide to merge two accesses when the address for the second access is incremental. For example, if the first address of a 32-byte cache line fetch is 0x0000_0000 and the second fetch is to address 0x0000_0020, then these two accesses can be merged. Merging increases efficiency and overall fetch bandwidth by eliminating the read header for those accesses which only require continuation of the SPI clock.

Wrap Around Accesses

Many SPI flash memory devices support wrapping which is used to enhance critical word fetching of cache lines. In this mode, the SPI device automatically wraps the read address to the base of a cache line once the end of the cache line is reached.

Wrap around accesses are enabled by setting the `SPI_MMRDH.WRAP` bit.

Some flash devices require programs to send a *Set Wrap* command to place the device in wrap mode. Other flash devices provide a configuration register which must be programmed to set the flash in wrap mode. Since the SPI memory-mapped hardware does not support any write operations to flash, perform this step in non-memory-mapped mode (`SPI_CTL.MMSE=0`) by accessing the SPI registers.

Data access is limited to 8-byte, 16-byte, or 32-byte sections of flash page in wrap mode. The Arm core uses the Wrap 4 access (64-bit data) for L1 cache. The Arm core uses Wrap 4 and Wrap 8 accesses (64-bit data) for L2 cache. The cores use the Wrap 8 accesses for unaligned accesses. During the read request to the SPI memory-mapped hardware, the memory subsystem controller of the processor provides the address of a critical word instead of the line base. The read-data starts at the address specified in the instruction. Once it reaches the end boundary of the 8, 16, or 32-byte section, the output automatically wraps around to the beginning boundary to the line base address. The data fetch continues. It is not necessary to deassert the SPI `SPI_SEL[n]` signal or resend the read header to wrap to the cache line base when servicing misaligned cache fill requests.

The *Byte Sequence in Wrap Modes* table shows byte sequences in various wrap modes.

Table 13-9: Byte Sequence in Wrap Modes

Starting Address	8-Byte Wrap (cache_line = 8 byte)
0	0-1-2- . . . -6-7
1	1-2- 3- . . . -7-0
7	7-0-1- . . . -5-6
15	15-8-9- . . . -13-14
31	31-24-25- . . . -29-30

The burst with wrap feature allows applications to fetch a critical address quickly. Applications then fill the cache afterwards within a fixed length (8/16/32-byte) of data without issuing multiple read commands. Certain applications can benefit from this feature to improve cache fill efficiency and overall performance of system code execution.

NOTE: Do not use the merge and wrap feature together. Using wrap bursts can unintentionally disable merging (merging cannot occur for unaligned wrapping bursts). A wrap burst can start fetching data words in the middle of the cache line and cannot be merged with the next access.

Execute-In-Place (XIP, SPI2 only)

Execute-In-Place, known as XIP, permits software code to execute directly from an SPI flash device rather than downloading the code and executing it out of RAM. XIP, also known as Command Skip mode, is a general term and can be applied to fetching data as well.

There is a difference between XIP mode and standard mode. In XIP mode, after the SPI memory device is selected (`CS# =LOW`), the memory device does not decode the first input byte as command code. Instead, it expects the read header to directly start with address bytes. In standard mode, the memory decodes the first input byte it receives as a command code.

The XIP mode dramatically reduces random access time for applications that require fast code execution without shadowing the memory content on a RAM. The SPI memory-mapped hardware provides a control bit, `SPI_MMRDH.CMDSKIP` to skip the command from read header.

Some SPI memory devices require configuration of their control register to enable the XIP mode of operation, using the non-memory-mapped mode of the processor SPI. Typically, during the dummy cycle period, the mode bits are used to confirm the XIP operation and the `SPI_MMRDH.MODE` field must be set appropriately. A dummy memory-mapped access may be needed before setting the `SPI_MMRDH.CMDSKIP` bit to set the SPI memory device in Command Skip mode.

For more details about how to configure SPI memories into XIP mode, refer to the device data sheet.

NOTE: When configuring the flash to XIP mode from the SHARC+ core, ensure that the routine that configures flash to XIP is not routed through the L2CC. This is accomplished by first configuring the flash to XIP mode, then enabling the L2CC from the core.

Memory-Mapped Mode Error Status Bits

The SPI memory-mapped hardware provides bits in the `SPI_STAT` register to report errors. It provides these bits for notification only and their state has no effect on SPI operations. The status register bits are sticky. A W1C (write-1-to-clear) operation clears the bits.

- Memory-Mapped Write Error (`SPI_STAT.MMWE`). This bit is set (=1) if an attempt is made to write to address space that is reserved for memory-mapped SPI memory. The SPI memory-mapped hardware does not support automated write access to SPI memory space.
- Memory-Mapped Read Error (`SPI_STAT.MMRE`). This bit is set (=1) if an attempt is made to read address space reserved for memory-mapped SPI memory while memory mapping is disabled (`SPI_CTL.MMSE = 0`).
- Memory-Mapped Access Error (`SPI_STAT.MMAE`). This bit is set (=1) if an attempt is made to access either the TX or RX FIFO while memory-mapped access of SPI memory is enabled. In this case, attempts to communicate with the SPI device using legacy methods are blocked and receive fabric reports an error. Legacy methods include any direct access made to the TX and RX FIFOs, whether by DMA or processor MMR.
- Memory-Mapped Write Error Mask (`SPI_CTL.MMWEM`) bit specifies whether an error response is returned to the fabric on write attempts to address space that is reserved for memory-mapped SPI memory reads. Regardless of whether a write error response is masked using this bit, the memory-mapped write error (`SPI_STAT.MMWE`) sticky notification bit is still set.

NOTE: Unlike other bits in the `SPI_STAT` register, these memory-mapped mode error bits do not have associated bits in the SPI interrupt mask (`SPI_IMSK`) and SPI interrupt condition (`SPI_ILAT`) registers.

The memory-mapped top register (`SPI_MMTOP`) is used to specify the upper limit of the SPI memory address. The memory-mapped accesses to SPI memory addresses equal to or above this range are considered illegal. The accesses are blocked, and a bus error response is generated.

This register is useful to block the invalid SPI memory address accesses. Some SPI memory vendors do not clearly specify (guarantee) that overrange address bits are ignored (address spaces can be wrapped).

Memory-Mapped Programming Guidelines

Setting the `SPI_CTL.MMSE` bit enables SPI memory-mapped mode. When enabled, the SPI interface is forced to be consistent with SPI memory requirements regardless the settings of certain control bits. The following tables specify typical settings for configuring the SPI in memory-mapped mode:

Table 13-10: SPI Control (SPI_CTL) Register

Bits	Typical values to set	Description	Comments
<code>SPI_CTL.MSTR</code>	1	Controller mode enable	
<code>SPI_CTL.PSSE</code>	0	Protected target select enable	
<code>SPI_CTL.ODM</code>	0	Open-drain mode enable	
<code>SPI_CTL.CPHASPI_CTL.CPOL</code>	0–0 or 1–1	SPI mode of communication	Flash dependent, usually SPI flash supports mode-0 (<code>CPHA=CPOL=0</code>) and mode-3 (<code>CPHA=CPOL=1</code>)
<code>SPI_CTL.ASSEL</code>	1	Hardware target select pin control	
<code>SPI_CTL.SELST</code>	1	Target select asserted between transfers	
<code>SPI_CTL.EMISO</code>	1	MISO pin enable	
<code>SPI_CTL.SIZE</code>	2	32-bit transfer size	
<code>SPI_CTL.LSBF</code>	0	MSB bit first mode	Flash dependent, usually SPI flash communicates in MSB bit first mode
<code>SPI_CTL.FCEN</code> <code>SPI_CTL.FCCH</code> <code>SPI_CTL.FCPL</code> <code>SPI_CTL.FCWM</code>	0	Hardware flow control related bits	
<code>SPI_CTL.FMODE</code>	1	Fast mode enable	Typically set to 1 for full cycle timing, 0 only works at low speed
<code>SPI_CTL.SOSI</code>	0	Treat <code>SPI_MOSI</code> pin as IO0 pin.	

Table 13-11: SPI Receive Control Register

Bits	Typical values to set	Description
<code>SPI_RXCTL.REN</code>	1	Receive channel enable
<code>SPI_RXCTL.RTI</code>	0	Receive transfer initiation disable
<code>SPI_RXCTL.RWCEN</code>	0	Receive word counter disable

Table 13-11: SPI Receive Control Register (Continued)

Bits	Typical values to set	Description
SPI_RXCTL.RDR	0	Receive data request disable
SPI_RXCTL.RDO	0	Discard incoming data if RFIFO is full
SPI_RXCTL.RRWM	0	Receive FIFO regular watermark
SPI_RXCTL.RUWM	0	Receive FIFO urgent watermark disable

Table 13-12: SPI Transmit Control Register

Bits	Typical values to set	Description
SPI_TXCTL.TEN	1	Transmit channel enable
SPI_TXCTL.TTI	1	Transmit transfer initiation disable
SPI_TXCTL.TWCEN	0	Transmit word counter disable
SPI_TXCTL.TDR	0	Transmit data request disable
SPI_TXCTL.TDU	0	Send last word when TFIFO is empty
SPI_TXCTL.TRWM	0	Transmit FIFO regular watermark
SPI_TXCTL.TUWM	0	Transmit FIFO urgent watermark disable

Table 13-13: SPI DLY Control Register

Bits	Typical Values to Set	Description	Comments See Flash data sheet for CS (for example, SSEL) Timing Specs
SPI_DLY.LAGX	1	Extended lag timing	
SPI_DLY.LEADX	1	Extended lead timing	
SPI_DLY.STOP	3	Stop bit between the transfers	Can be set to 1 at lower SPI clock frequencies.

The multiple I/O mode (SPI_CTL.MIOM) bits are partially ignored:

- The command (opcode) is transmitted using either just one or the number of pins specified by the SPI_CTL.MIOM bits, depending on SPI_MMRDH.CMDPINS bit setting.
- The address is then transmitted using either just one or the number of pins specified by the SPI_CTL.MIOM bits, depending on SPI_MMRDH.ADRPINS bit setting.
- The data is always read with the number of pins specified by the SPI_CTL.MIOM bits.

NOTE: Set the SPI module enable bits SPI_CTL.EN last after configuring all registers.

Use the following programming guidelines for memory-mapped mode:

- The SPI memory-mapped hardware does not check the flash status before initiating the access. It assumes that SPI memory is always able to respond to a read access. Before enabling memory-mapped mode (for example, setting the `SPI_CTL.MMSE` bit) ensure that SPI flash is ready for a read access. When using non-memory-mapped mode, a write-complete status can be examined prior to enabling the SPI in memory-mapped mode. (See the write in progress bit in the SPI flash memory status register.) Also, immediately after initial power-up, SPI memory devices can be inaccessible for a vendor-specified period.
- When SPI is enabled in memory-mapped mode, attempts to communicate with the SPI device using legacy methods are blocked. Legacy methods include any direct access made to the transmit or receive FIFOs, whether initiated by DMA or by a processor MMR access.
- To use some of the features offered by SPI memory devices, programs can first configure the SPI memory device by setting its control word or sending some commands. Because SPI memory-mapped hardware does not allow any type of SPI write operations, configure the SPI in non-memory-mapped mode prior to enabling memory-mapped mode.
- The memory-mapped hardware does not interpret the opcode. It does not check the validity of the timing that is specified in the `SPI_MMRDH` register for a particular opcode. Programs must set the fields of the `SPI_MMRDH` register to be consistent with the read-type selected.
- When the core requests the data or code fetch, the memory-mapped transfer depends on cache settings. The cache configuration register in the SPI memory device must be appropriately configured before enabling memory-mapped mode. Some of the high performance modes like merge, wrap, and transfer size depend on cache parameters.
- SPI memory-mapped MDMA reads do not support wrapping. For MDMA reads, limit the `DMA_CFG.MSIZE` field to 1 byte, 2 bytes or 4 bytes.
- There is not always tool support to change the SPI memory-mapped hardware setting or cache settings on-the-fly. Changing these settings can optimize the performance of code that accesses SPI memory in memory-mapped mode. It is expected that the SPI memory, SPI peripheral, and cache are programmed to one specific set of control settings for the whole application. Profiling or benchmarking of the actual application can be done to find the setting that works best.

SPI Interrupt Signals

The SPI controller supports three types of interrupt request signals that correspond to data, status, and error conditions.

Data Interrupts

The SPI target supports two data interrupt channels – receive and transmit. These interrupt signals are multiplexed into the DMA request lines. Since the target interfaces with separate read and write interfaces with DMA, the read and write data interrupts are independent. When the DMA channels are not used, the interrupts are routed directly to the system event controller. The interrupts occupy the same vector locations as the corresponding DMA channels.

Each of the data interrupt requests can be individually controlled. Program the `SPI_RXCTL.RDR` and `SPI_TXCTL.TDR` bit fields for receive and transmit, respectively. When receive is enabled, the RX interrupt request is issued whenever there is data available in the receive datapath for reading. (The event occurs according to the `SPI_RXCTL.RDR` bit setting.) When transmit is enabled, the TX interrupt request is issued whenever the transmit datapath can be written. (The event occurs according to the `SPI_TXCTL.TDR` setting.) DMA data interrupts are compatible with second-generation DMA to incorporate urgent data requests and transfer finish interrupt requests apart from the usual data request interrupts. Transmit interrupt requests operate independently from the word counter-value in the `SPI_TWC` register.

Status Interrupts

The SPI controller supports several status interrupt requests to indicate different conditions of the receiver and transmitter. All status interrupt requests can be masked. Status interrupt requests are signaled directly through a single SPI status IRQ line. The line cannot be combined with the SPI error IRQ line for some processors. The *SPI Status Interrupts* table describes the status interrupt requests that are available for the SPI controller.

Table 13-14: SPI Status Interrupts

SPI_STAT Bit	Description
<code>SPI_STAT.RUWM</code>	Receive FIFO urgent watermark interrupt request. Issued when the level of the RFIFO breaches the watermark set in the <code>SPI_RXCTL.RUWM</code> field. It is cleared when the level of the RFIFO reaches the watermark set in the <code>SPI_RXCTL.RRWM</code> field. If the RX channel is configured in DMA mode, <code>SPI_RXCTL.RUWM</code> is multiplexed with the data request.
<code>SPI_STAT.TUWM</code>	Transmit FIFO urgent watermark interrupt request. Issued when the level of the TFIFO breaches the watermark set using the <code>SPI_TXCTL.TUWM</code> bit. It is cleared when the level of the TFIFO reaches the watermark set in the <code>SPI_TXCTL.TRWM</code> field. If the TX channel is configured in DMA mode, <code>SPI_STAT.TUWM</code> is multiplexed with the data request.
<code>SPI_STAT.TS</code>	Transmit start interrupt request. Issued when the start of a transmit burst is detected by loading of the <code>SPI_TWC</code> register with the contents of the <code>SPI_TWCR</code> register.
<code>SPI_STAT.RS</code>	Receive start interrupt request. Issued when the start of a receive burst is detected by the loading of <code>SPI_RWC</code> with the contents of <code>SPI_RWCR</code> .
<code>SPI_STAT.TF</code>	Transmit finish interrupt request. Issued when a transmit burst completes (<code>SPI_TWC</code> decrements to zero).
<code>SPI_STAT.RF</code>	Receive finish interrupt request. Issued when a receive burst completes (<code>SPI_RWC</code> decrements to zero).

Error Conditions

The SPI controller supports interrupt requests upon several different error conditions. All interrupt requests are maskable. The individual error indications combine into a single SPI error IRQ signal, which can be multiplexed on some processors with the aggregated SPI status IRQ signal. The *SPI Error Interrupts* table details the possible error indications.

Error conditions arise depending on which of the channels (transmit or receive) are enabled. If a channel is disabled, all errors related to it are ignored. When both channels are enabled, errors from both channels are enabled.

Table 13-15: SPI Error Interrupts

Bit	Description
<code>SPI_STAT.MF</code>	Mode fault. Signaled when another device also tries to be a controller in a multi-controller system and drives the <code>SPI_SS</code> input low. This error is signaled in controller mode operation.
<code>SPI_STAT.TUR</code>	Transmission error. Signaled when an underflow condition occurs on the transmit channel. This event occurs when a new transfer starts but <code>SPI_TFIFO</code> is empty. This error does not occur in controller transmit initiating mode since <code>SPI_TFIFO Not Empty</code> is one of the conditions for transfer initiation.
<code>SPI_STAT.ROR</code>	Reception error. Signaled when an overflow condition occurs on the receive channel. This event occurs when a new data word is received, but the <code>SPI_RFIFO</code> is full. This error condition does not occur in controller receive initiating mode since <code>SPI_RFIFO Not Full</code> is one of the conditions for transfer initiation.
<code>SPI_STAT.TC</code>	Transmit collision error. Signaled when loading data to the transmit shift register happens near the first transmitting edge of <code>SPI_CLK</code> . In target mode of operation, the SPI controller is unaware of when the next transfer starts. Loading of data to the transmit shift register can happen just after the transmitting edge. This event results in the setup time not being met for the first bit transmitted. The transmitted data is corrupt. In <code>SPI_CTL.CPHA 1</code> mode, the first <code>SPI_CLK</code> edge is taken as the first transmitting edge. If <code>SPI_CTL.CPHA = 0</code> , then the last <code>SPI_CLK</code> edge of the last transmission (<code>SPI_CTL.SELST = 1</code>) or target select deassertion (<code>SPI_CTL.SELST = 0</code>) is taken as the first transmitting edge. This error is signaled only in the target mode of operation. In controller mode of operation, loading of data happens before the first transmitting edge of <code>SPI_CLK</code> .

SPI Programming Concepts

The following sections provide general programming guidelines and procedures.

Programming Guidelines

It is acceptable to program `SPI_RXCTL` and `SPI_TXCTL` registers after programming the `SPI_CTL` register. However, program the initiating mode register and its counter-register, if enabled, after the non-initiating mode register. For example, if transmit is the initiating mode and receive is the non-initiating mode, then program the `SPI_RXCTL` and `SPI_RWC` registers before the `SPI_TXCTL` and `SPI_TWC` registers. If enabling both transmit and receive in initiating mode, enable the `SPI_CTL` register after programming both the `SPI_RXCTL` and `SPI_TXCTL` registers.

These programming guidelines prevent SPI from starting a transfer when SPI registers are not fully programmed. Other ways of programming are also allowed when the initiating conditions prevent the start of communication until after programming of SPI registers is complete.

Avoid data corruption when changing the SPI module configuration. Do not change the configuration during a data transfer. Additionally, change the clock polarity only when no target is selected. However, an exception to this rule exists. When an SPI communication link consists of a single controller and target, `SPI_CTL.ASSEL = 0`. The target select input of the slave is permanently tied low. In this case, the target is always selected. Avoid data corruption by enabling the target only after both the controller and target devices are configured.

The module supports 8, 16-bit and 32-bit word sizes. To ensure correct operation, configure both the controller and target with the same word size.

Controller Operation in Non-DMA Modes

This section describes the operation of the SPI as a controller in non-DMA mode.

1. Write to the `SPI_SLVSEL` register, setting one or more of the SPI select enable bits. This operation ensures that the desired targets are properly deselected while the controller is configured.
2. The `SPI_RXCTL.RTI` and `SPI_TXCTL.TTI` bits determine the SPI initiating mode. The initiating mode defines the primary transfer channel, and also the initiating condition for the transfer.
3. Write to the `SPI_CLK`, `SPI_CTL`, `SPI_RXCTL`, and `SPI_TXCTL` registers. This operation enables the device as a controller and configures the SPI system. It specifies the transfer modes and channels, appropriate word length, transfer format, baud rate, and other control information.

ADDITIONAL INFORMATION: If `SPI_RXCTL.RTI` is enabled and `SPI_TXCTL.TTI` is not, write to the `SPI_RXCTL` register after writing into `SPI_CTL`, `SPI_TXCTL`, and `SPI_TFIFO` registers to prevent a transmit underrun for the first transfer.

4. If `SPI_CTL.ASSEL=0`, activate the desired targets by clearing one or more of the `SPI_SLVSEL` flag bits. Otherwise, the SPI hardware performs target activation.
5. The SPI controller then generates the programmed clock pulses on `SPI_CLK` and simultaneously shifts data out of `SPI_MOSI` while shifting data in from `SPI_MISO`. Before a shift, the shift register is loaded with the contents of the `SPI_TFIFO` register. At the end of the transfer, the contents of the shift register are loaded into `SPI_RFIFO`.
6. Whenever the initiating conditions are satisfied, the SPI continues to send and receive words. If the transmit buffer remains empty or the receive buffer remains full, the device operates according to the states of the `SPI_TXCTL.TDU` and `SPI_RXCTL.RDO` bits.
7. It is possible to program a secondary channel in addition to the initiating channel. This feature allows usage of available channel resources for receives or transmits simultaneously with the initiating channel.

Target Operation in Non-DMA Modes

When a device is enabled as a target in a non-DMA mode, a transition of the `SPI_SS` select signal to the active state (low) triggers the start of a transfer. Or the first active edge of `SPI_CLK` triggers the start, depending on the state of `SPI_CTL.CPHA` bit. The interface operates in the following manner.

1. The core writes to the `SPI_CTL`, `SPI_RXCTL`, and `SPI_TXCTL` registers. The operation defines the mode of the serial link to be the same as the mode setup in the SPI controller.
2. To prepare for the data transfer, the core writes data to be transmitted into `SPI_TFIFO`.
3. Once the `SPI_SS` falling edge is detected, the target starts sending data on active `SPI_CLK` edges and sampling data on inactive `SPI_CLK` edges.
4. Reception or transmission continues until `SPI_SS` is released or until the target has received the proper number of clock cycles.

5. The target device continues to receive or transmit with each new falling edge transition on `SPI_SS` or active `SPI_CLK` edge. If the transmit buffer remains empty or the receive buffer remains full, the device operates according to the states of the `SPI_TXCTL.TDU` and `SPI_RXCTL.RDO` bits.

Configuring DMA Controller Mode

The SPI interface supports a write DMA channel and a read DMA channel. It can use these functions individually or in a lock-step manner in duplex mode (`SPI_TXCTL.TTI = SPI_RXCTL.RTI = 1`).

1. Write to the appropriate DMA registers to enable the SPI DMA channel and to configure the necessary work units, access direction, word count, and so on.
2. Write to the `SPI_SLVSEL` register, setting one or more of the SPI flag select bits.
3. Write to the `SPI_CLK` and `SPI_CTL` registers, enabling the device as a controller and configuring the SPI system by specifying the appropriate word length, transfer format, baud rate, and so forth.
4. Write to `SPI_RXCTL` to configure SPI controller receive mode, or write to `SPI_TXCTL` to configure SPI controller transmit mode.
5. Finally, write to the `SPI_RXCTL.REN` bit to enable the receive channel, or write to `SPI_TXCTL.TEN` to enable the transmit channel.
6. When the `SPI_RXCTL.RTI` bit is enabled, a receive transfer is initiated upon enabling `SPI_CTL.EN` bit. If the receive word counter is enabled (`SPI_RXCTL.RWCEN`), then the `SPI_RWC` register must be non-zero for a transfer to initiate.

ADDITIONAL INFORMATION: If enabling both receive and transmit DMA channels, but not enabling `SPI_TXCTL.TTI`, write to the `SPI_RXCTL` register after writing the `SPI_CTL` and `SPI_TXCTL` registers. In this way, a transmit underrun can be prevented for the first transfer. Subsequent transfers are initiated as the SPI reads data from the receive shift register and writes to the SPI receive FIFO. The SPI then requests a write from DMA to memory. Upon a DMA grant, the DMA engine reads a word from the SPI receive FIFO and writes to memory. New requests continue to be initiated as long as the receive FIFO does not fill up, when `SPI_RWC` does not become zero while `SPI_RXCTL.RWCEN = 1`.

7. When `SPI_TXCTL.TTI` is enabled, the SPI controller requests DMA reads from memory when there is space for more data in the transmit pipe. Upon a DMA grant, the DMA engine reads a word from memory and writes to the transmit FIFO. As long as transmit data is available in the FIFO, and the `SPI_TWC` register is non-zero when `SPI_TXCTL.TWCEN = 1`, the SPI continues to initiate transfers until disabled.
8. When both the `SPI_TXCTL.TTI` and `SPI_RXCTL.RTI` bits are enabled, the SPI controller requests a DMA read from memory. However, there must be space for more data in the transmit pipe and the number of words written into the SPI must be less than `SPI_TWC` if `SPI_TXCTL.TWCEN = 1`. Upon a DMA grant, the DMA engine reads a word from memory and writes to the transmit FIFO.

ADDITIONAL INFORMATION: As the SPI writes data from the transmit FIFO into the transmit shift register, it initiates a transfer on the SPI link. Data received from the transfer is moved from the SPI receive shift register to the receive FIFO. The SPI controller requests a write from DMA to memory. Upon a DMA grant,

the DMA engine reads a word from the receive FIFO and writes to memory. Transfer continues to be initiated when both receives and transmits can accommodate new data.

9. When the receive pipe fills up due to unavailability of DMA grants, the transmit pipe stalls until the pipe is drained. If the transmit pipe fills up, the SPI stops requesting for DMA writes. When the value in `SPI_RWC` expires, further write-requests to DMA stop. However, data already written into the transmit FIFO is sent, and read requests to DMA continue until the receive data is read from the receive FIFO.
10. The SPI then generates the programmed clock pulses on `SPI_CLK` and simultaneously shifts data out of `SPI_MOSI` while shifting data in from `SPI_MISO`. For receive transfers, the value in the shift register is loaded into the `SPI_RFIFO` register at the end of the transfer. For transmit transfers, the value in the `SPI_TFIFO` register is loaded into the shift register at the start of the transfer.

Configuring DMA Target Mode Operation

This mode occurs when the SPI is enabled as a target and the DMA engine is configured to transmit or receive data. A transition of the `SPI_SS` signal to the active-low state triggers the start of a transfer. Or, the first active edge of `SPI_CLK` triggers the start of a transfer, depending on the state of the `SPI_CTL.CPHA` bit. The following steps illustrate the SPI receive or transmit DMA sequence in an SPI target (in response to a controller command). The SPI supports a receive DMA channel and a transmit DMA channel.

1. Write to the appropriate DMA registers to enable the SPI DMA channel and configure the necessary work units, access direction, word count, and so on.
2. Write to the `SPI_CTL`, `SPI_RXCTL`, and `SPI_TXCTL` registers to define the mode of the serial link to be the same as the mode configured in the SPI controller.
3. When the receive channel is enabled (`SPI_RXCTL.REN` is asserted), the following actions occur:
 - a. Once the target select input is active, the target starts receiving and transmitting data on active `SPI_CLK` edges.
 - b. The value in the shift register is loaded into the `SPI_RFIFO` register at the end of the transfer.
 - c. Once `SPI_RFIFO` has valid data, it requests a write from DMA to memory.
 - d. Upon a DMA grant, the DMA engine reads a word from the receive FIFO and writes to memory.
 - e. When there is data in the receive FIFO, the SPI target continues to request a DMA write to memory. The DMA engine continues to read a word from the FIFO and writes to memory. The SPI target continues receiving words on active `SPI_CLK` edges when the `SPI_SS` input is active.
 - f. When the data collected in the receive pipe breaches the set level, and the DMA engine cannot keep up with the receive rate, the target can deassert the `SPI_RDY` signal. This signaling throttles the controller. The receive pipe level is set according to the `SPI_CTL.FCWM` field. The signal is deasserted as the DMA drains the receive FIFO. Alternatively, the SPI can use the `SPI_RXCTL.RDO` bit to decide when the incoming data is discarded or overwritten into the receive FIFO (when `SPI_CTL.FCEN` is inactive).
4. When the transmit channel is enabled (`SPI_TXCTL.TEN` is asserted), the following actions occur:

- a. The SPI requests a DMA read from memory.
 - b. Upon a DMA grant, the DMA engine reads a word from memory and writes to the transmit FIFO.
 - c. The SPI then reads DMA data from the transmit FIFO and writes to the transmit shift register, awaiting the start of the next transfer.
 - d. Once the target select input is active, the target starts receiving and transmitting data on active `SPI_CLK` edges.
 - e. When there is room in the transmit FIFO, the SPI target continues to request a DMA read from memory. The DMA engine continues to read a word from memory and write to the transmit FIFO. The SPI target continues transmitting words on active `SPI_CLK` edges as long as the `SPI_SS` input is active.
 - f. When the number of outstanding data entries in the transmit pipe breaches the level set and the DMA cannot keep up with the transmit rate, the target deasserts the `SPI_RDY` signal. This signaling throttles the controller. The transmit pipe level is set according to the `SPI_CTL.FCWM` field. The signal is deasserted as the DMA fills the transmit FIFO. Alternately, the `SPI_TXCTL.TDU` bit decides the state of the transmit data (when `SPI_CTL.FCEN` is deasserted).
5. When both receive and transmit channels are enabled, the following actions occur after the actions for each channel. Transfers continue when both receive and transmit channels can accommodate new data.
- a. When the receive pipe fills up due to the unavailability of DMA grant, the SPI interface stalls the controller by asserting the `SPI_RDY` pin. This signal is deasserted as the DMA drains the receive FIFO. Alternately, the SPI uses the `SPI_RXCTL.RDO` bit to decide when the incoming data is discarded or overwritten in the receive FIFO (when `SPI_CTL.FCEN` is deasserted).
 - b. When the transmit pipe fills up, the SPI stops requesting DMA writes until the pipe clears.
 - c. When there is an underflow problem in the transmit pipe, the target stalls the controller by deasserting `SPI_RDY` while the DMA fills the transmit FIFO. Alternately, the SPI uses the `SPI_TXCTL.TDU` bit to decide the state of the transmit data (when `SPI_CTL.FCEN` is deasserted).

ADSP-21560_21561_21564_21568 SPI Register Descriptions

Serial Peripheral Interface (SPI) contains the following registers.

Table 13-16: ADSP-21560_21561_21564_21568 SPI Register List

Name	Description
<code>SPI_CLK</code>	Clock Rate Register
<code>SPI_CTL</code>	Control Register
<code>SPI_DLY</code>	Delay Register
<code>SPI_ILAT</code>	Masked Interrupt Condition Register
<code>SPI_ILAT_CLR</code>	Masked Interrupt Clear Register

Table 13-16: ADSP-21560_21561_21564_21568 SPI Register List (Continued)

Name	Description
SPI_IMSK	Interrupt Mask Register
SPI_IMSK_CLR	Interrupt Mask Clear Register
SPI_IMSK_SET	Interrupt Mask Set Register
SPI_MMRDH	Memory Mapped Read Header
SPI_MMTOP	SPI Memory Top Address
SPI_RFIFO	Receive FIFO Data Register
SPI_RWC	Received Word Count Register
SPI_RWCR	Received Word Count Reload Register
SPI_RXCTL	Receive Control Register
SPI_SLVSEL	Slave Select Register
SPI_STAT	Status Register
SPI_TFIFO	Transmit FIFO Data Register
SPI_TWC	Transmitted Word Count Register
SPI_TWCR	Transmitted Word Count Reload Register
SPI_TXCTL	Transmit Control Register

Clock Rate Register

The `SPI_CLK` register selects the baud rate for SPI data transfers, relating this rate to the SPI serial clock (SPI clock) and the system clock (SCLK).

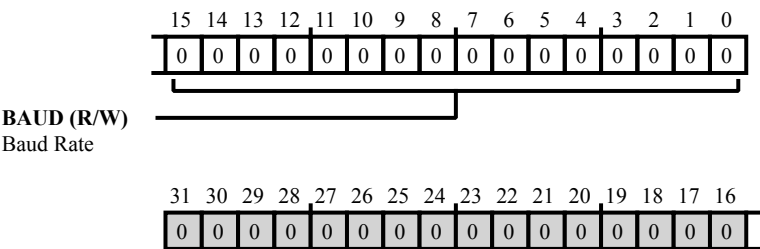


Figure 13-20: SPI_CLK Register Diagram

Table 13-17: SPI_CLK Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	BAUD	Baud Rate. The <code>SPI_CLK.BAUD</code> bits set the SPI baud rate according to the formula: $BAUD = (SCLK / SPI\ Clock) - 1$

Control Register

The `SPI_CTL` register enables the SPI and configures settings for operating modes, communication protocols, and buffer operations.

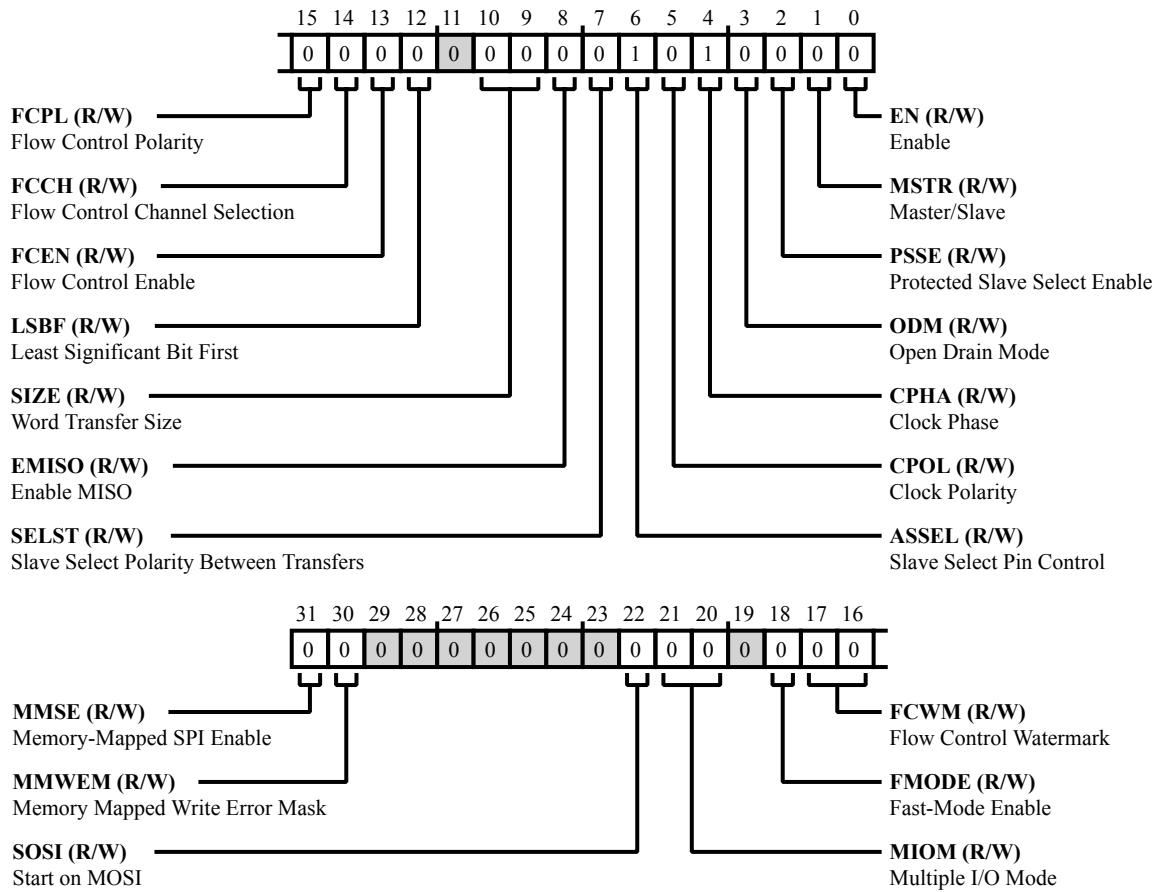


Figure 13-21: SPI_CTL Register Diagram

Table 13-18: SPI_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	MMSE	Memory-Mapped SPI Enable. When the <code>SPI_CTL.MMSE</code> bit is asserted, communication to an SPI memory device is automated such that the memory it contains is accessible directly through the read of processor address space assigned to it. (As far as the SPI peripheral is concerned, this includes all read accesses received by the SPI peripherals system crossbar slave port.) Note that when memory-mapped access of SPI memory is enabled, attempts to communicate with the SPI device using legacy methods are blocked and receive fabric error responses are generated. Legacy methods include any direct access made to the Tx and Rx FIFOs, whether initiated by DMA or processor MMR access.
		0 Hardware automated access of memory-mapped SPI memory disabled.
		1 Hardware-automated access of memory-mapped SPI memory enabled.
30 (R/W)	MMWEM	Memory Mapped Write Error Mask. The <code>SPI_CTL.MMWEM</code> bit specifies whether an error response is returned to the fabric upon write attempts to address space reserved for memory-mapped reads of SPI memory.
		0 Write error response returned upon write attempts to memory-mapped SPI memory
		1 Write error response masked (not returned) upon write attempts to memory-mapped SPI memory
22 (R/W)	SOSI	Start on MOSI. The <code>SPI_CTL.SOSI</code> bit is valid only when <code>SPI_CTL.MIOM</code> is enabled for either DIOM or QIOM, and this bit selects the starting pin and the bit placement on pins for these modes. In DIOM, by default, (<code>SPI_CTL.SOSI = 0</code>) SPI sends the first bit on the <code>SPI_MISO</code> pin and the second bit on the <code>SPI_MOSI</code> pin. In QIOM, by default, the SPI sends the first bit on the <code>SPI_D3</code> pin, the second bit on the <code>SPI_D2</code> pin, the third bit on the <code>SPI_MISO</code> pin and the fourth bit on the <code>SPI_MOSI</code> pin. This order can be reversed by setting the <code>SPI_CTL.SOSI</code> bit. When this bit is set, the SPI sends the first bit on the <code>SPI_MOSI</code> pin. The first bit referred to here depends on the <code>SPI_CTL.LSBF</code> bit setting (MSB bit or LSB bit).
		0 Start on MISO (DIOM) or start on SPI_D3
		1 Start on MOSI

Table 13-18: SPI_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
21:20 (R/W)	MIOM	Multiple I/O Mode. The <code>SPI_CTL.MIOM</code> bits enable SPI operation in dual I/O mode (DIOM) or quad I/O mode (QIOM). These bits can only be changed when the SPI is disabled (<code>SPI_CTL.EN = 0</code>).
		0 No MIOM (disabled)
		1 DIOM operation
		2 QIOM operation
		3 Reserved
18 (R/W)	FMODE	Fast-Mode Enable. The <code>SPI_CTL.FMODE</code> bit enables fast mode operation for SPI receive transfers. SPI transmit operations in fast mode are the same as normal mode.
		0 Disable
		1 Enable
17:16 (R/W)	FCWM	Flow Control Watermark. The <code>SPI_CTL.FCWM</code> bits select the watermark level of the transmit channel (<code>SPI_TFIFO</code> buffer) or receive channel (<code>SPI_RFIFO</code> buffer) that triggers flow control operation. These bits are applicable only when the SPI is a slave (<code>SPI_CTL.MSTR = 0</code>) and flow control is enabled (<code>SPI_CTL.FCEN = 1</code>). When the watermark condition is met, the SPI slave deasserts the <code>SPI_RDY</code> pin.
		0 TFIFO empty or RFIFO full
		1 TFIFO 75% or more empty, or RFIFO 75% or more full
		2 TFIFO 50% or more empty, or RFIFO 50% or more full
		3 Reserved
15 (R/W)	FCPL	Flow Control Polarity. The <code>SPI_CTL.FCPL</code> bit selects flow control polarity for the <code>SPI_RDY</code> pin when flow control is enabled. When the <code>SPI_RDY</code> pin is active, the SPI is indicating it is ready for data transfer.
		0 Active-low RDY
		1 Active-high RDY

Table 13-18: SPI_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
14 (R/W)	FCCH	Flow Control Channel Selection. The <code>SPI_CTL.FCCH</code> bit selects whether the SPI applies flow control to the transmit channel (<code>SPI_TFIFO</code> buffer) or receive channel (<code>SPI_RFIFO</code> buffer). This bit is applicable only when the SPI is a slave and flow control is enabled.
		0 Flow control on RX buffer
		1 Flow control on TX buffer
13 (R/W)	FCEN	Flow Control Enable. The <code>SPI_CTL.FCEN</code> bit enables SPI flow control operation, which permits slow slave devices to interface with fast master devices. This bit controls the operation of the <code>SPI_RDY</code> pin. Note that options for flow control operation are available using the <code>SPI_CTL.FCCH</code> , <code>SPI_CTL.FCPL</code> , and <code>SPI_CTL.FCWM</code> bits.
		0 Disable
		1 Enable
12 (R/W)	LSBF	Least Significant Bit First. The <code>SPI_CTL.LSBF</code> bit selects whether the SPI transmits/receives data as LSB first (little endian) or MSB first (big endian). This bit can only be changed when the SPI is disabled.
		0 MSB sent/received first (big endian)
		1 LSB sent/received first (little endian)
10:9 (R/W)	SIZE	Word Transfer Size. The <code>SPI_CTL.SIZE</code> bits select the SPI transfer word size as 8, 16 or 32 bits. To ensure correct operation, both the master and slave must be configured with the same word size. This bit can only be changed when the SPI is disabled (<code>SPI_CTL.EN = 0</code>).
		0 8-bit word
		1 16-bit word
		2 32-bit word
		3 Reserved
8 (R/W)	EMISO	Enable MISO. The <code>SPI_CTL.EMISO</code> bit enables master-in-slave-out (MISO) mode. This SPI mode is applicable only when the SPI is a slave.
		0 Disable
		1 Enable

Table 13-18: SPI_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
7 (R/W)	SELST	Slave Select Polarity Between Transfers. The SPI_CTL.SELST bit selects the state (polarity) for the $\overline{\text{SPI_SEL[n]}}$ pin between SPI transfers when the SPI is a master and hardware slave select assertion is enabled (SPI_CTL.ASSEL = 1). In slave mode, this bit affects the detection of both transmit collision (SPI_STAT.TC and underrun (SPI_STAT.TUR) errors.
		0 Deassert slave select (high)
		1 Assert slave select (low)
6 (R/W)	ASSEL	Slave Select Pin Control. The SPI_CTL.ASSEL bit selects whether the SPI hardware sets the $\overline{\text{SPI_SEL[n]}}$ pin output value (ignoring the slave select SPI_SLVSEL.SSEL1 - SPI_SLVSEL.SSEL7 bits) or whether software control of the slave select bits set the $\overline{\text{SPI_SEL[n]}}$ pin output value. This feature is applicable only when the SPI is a master. When hardware control is enabled, the $\overline{\text{SPI_SEL[n]}}$ pin output is asserted during the transfers, and the pin polarity between transfers is selected by the SPI_CTL.SELST bit. When software control is enabled, the $\overline{\text{SPI_SEL[n]}}$ pin output value is set through software control of the slave select bits, and as such, the pin may either remain asserted (low) or be deasserted between transfers.
		0 Software slave select control
		1 Hardware slave select control
5 (R/W)	CPOL	Clock Polarity. The SPI_CTL.CPOL bit selects whether the SPI uses an active-low or active-high signal for the SPI clock (SPI_CLK). This bit works with the SPI_CTL.CPHA bit to select combinations of clock phase and polarity for the SPI_CLK pin. This bit can only be changed when the SPI is disabled.
		0 Active-high SPI CLK
		1 Active-low SPI CLK
4 (R/W)	CPHA	Clock Phase. The SPI_CTL.CPHA bit selects whether the SPI starts toggling the signal for the SPI clock (SPI_CLK) from the start of the first data bit or from the middle of the first data bit. The SPI_CTL.CPHA bit works with the SPI_CTL.CPOL bit to select combinations of clock phase and polarity for the SPI_CLK pin. This bit can only be changed when the SPI is disabled.
		0 SPI CLK toggles from middle
		1 SPI CLK toggles from start

Table 13-18: SPI_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
3 (R/W)	ODM	Open Drain Mode. The SPI_CTL.ODM bit configures the data output pins (SPI_MOSI and SPI_MISO) to behave as open drain outputs, which prevents contention and possible damage to pin drivers in multi-master or multi-slave SPI systems. When SPI_CTL.ODM is enabled and the SPI is a master, the SPI three-states the SPI_MOSI pin when the data driven out on MOSI is a logic-high. The SPI does not three-state the SPI_MOSI pin when the driven data is a logic-low. When SPI_CTL.ODM is enabled and the SPI is a slave, the SPI three-states the SPI_MISO pin when the data driven out on SPI_MISO is a logic-high. Note that an external pull-up resistor is required on both the SPI_MOSI and SPI_MISO pins when SPI_CTL.ODM is enabled.
		0 Disable
		1 Enable
2 (R/W)	PSSE	Protected Slave Select Enable. The SPI_CTL.PSSE bit enables the <u>SPI_SS</u> pin to provide error detection input in a multi-master environment when the SPI is in master mode. If some other device in the system asserts the <u>SPI_SS</u> pin while SPI is enabled as master (and SPI_CTL.PSSE is enabled), this condition causes a mode fault error.
		0 Disable
		1 Enable
1 (R/W)	MSTR	Master/Slave. The SPI_CTL.MSTR bit toggles the SPI between master mode and slave mode. This bit can only be changed when the SPI is disabled.
		0 Slave
		1 Master
0 (R/W)	EN	Enable. The SPI_CTL.EN bit enables SPI operation.
		0 Disable SPI module
		1 Enable

Delay Register

The `SPI_DLY` register selects a transfer delay and the lead/lag timing between slave select signals and SPI clock edge assertion/deassertion.

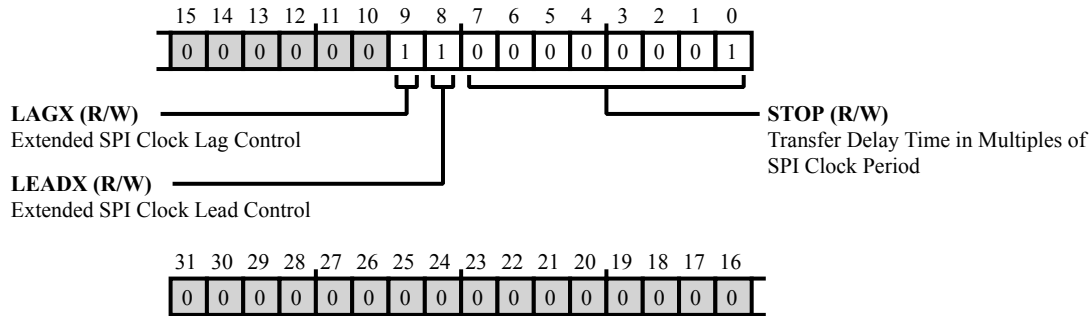


Figure 13-22: SPI_DLY Register Diagram

Table 13-19: SPI_DLY Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
9 (R/W)	LAGX	Extended SPI Clock Lag Control. The <code>SPI_DLY.LAGX</code> bit enables insertion of a 1-SPI_CLK cycle lag (extend lag) in the timing between the slave select (<code>SPI_SEL[n]</code>) assertion and first SPI clock edge.
		0 Disable
		1 Enable
8 (R/W)	LEADX	Extended SPI Clock Lead Control. The <code>SPI_DLY.LEADX</code> bit enables insertion of a 1-SPI_CLK cycle lead (extend lead) in the timing between the slave select (<code>SPI_SEL[n]</code>) deassertion and last SPI clock edge.
		0 Disable
		1 Enable
7:0 (R/W)	STOP	Transfer Delay Time in Multiples of SPI Clock Period. The <code>SPI_DLY.STOP</code> bits select a delay (number of stop bits in multiples of SPI clock duration) at the end of each SPI transfer. The default delay is the minimum value required to comply with the SPI protocol (1-bit duration). The <code>SPI_DLY.STOP</code> bits can be programmed with smaller delay values, resulting in continuous operation (for example, stop bits =0).

Masked Interrupt Condition Register

The `SPI_ILAT` register latches interrupts, queuing the interrupt requests for service. When a condition is indicated by a bit in the `SPI_STAT` register and the corresponding interrupt request is unmasked in `SPI_IMSK`, the SPI latches the interrupt request bit in `SPI_ILAT`.

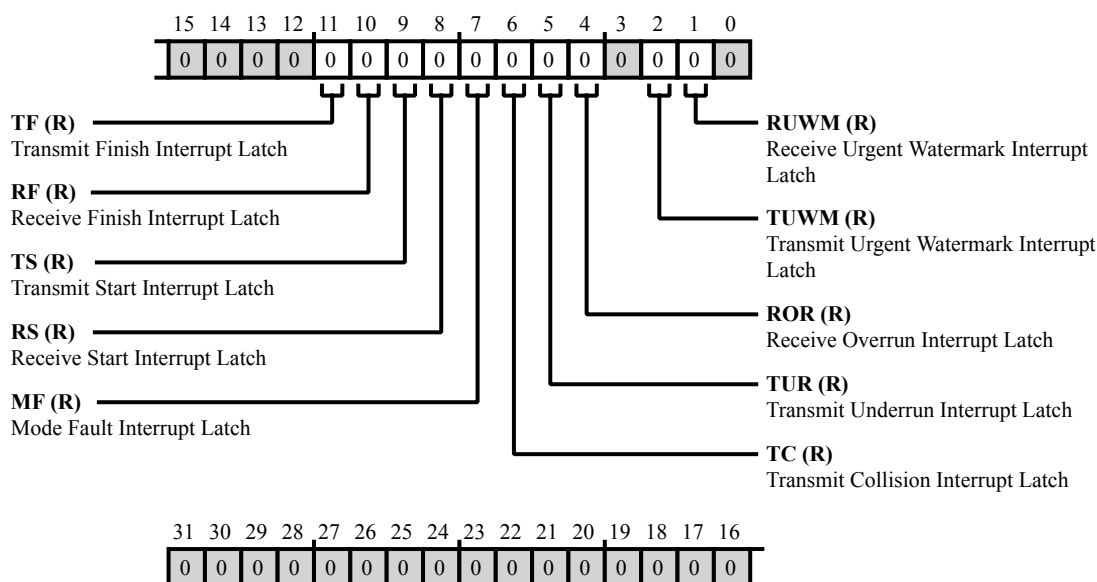


Figure 13-23: SPI_ILAT Register Diagram

Table 13-20: SPI_ILAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
11 (R/NW)	TF	Transmit Finish Interrupt Latch.
		0 No interrupt request
		1 Latched interrupt request
10 (R/NW)	RF	Receive Finish Interrupt Latch.
		0 No interrupt request
		1 Latched interrupt request
9 (R/NW)	TS	Transmit Start Interrupt Latch.
		0 No interrupt request
		1 Latched interrupt request
8 (R/NW)	RS	Receive Start Interrupt Latch.
		0 No interrupt request
		1 Latched interrupt request

Table 13-20: SPI_ILAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
7 (R/NW)	MF	Mode Fault Interrupt Latch.	
		0	No interrupt request
		1	Latched interrupt request
6 (R/NW)	TC	Transmit Collision Interrupt Latch.	
		0	No interrupt request
		1	Latched interrupt request
5 (R/NW)	TUR	Transmit Underrun Interrupt Latch.	
		0	No interrupt request
		1	Latched interrupt request
4 (R/NW)	ROR	Receive Overrun Interrupt Latch.	
		0	No interrupt request
		1	Latched interrupt request
2 (R/NW)	TUWM	Transmit Urgent Watermark Interrupt Latch.	
		0	No interrupt request
		1	Latched interrupt request
1 (R/NW)	RUWM	Receive Urgent Watermark Interrupt Latch.	
		0	No interrupt request
		1	Latched interrupt request

Masked Interrupt Clear Register

The `SPI_ILAT_CLR` register permits clearing individual mask bits in the `SPI_ILAT` register without affecting other bits in the register. Use write-1-to-clear on a bit in the `SPI_ILAT_CLR` register to clear the corresponding bit in the `SPI_ILAT` register.

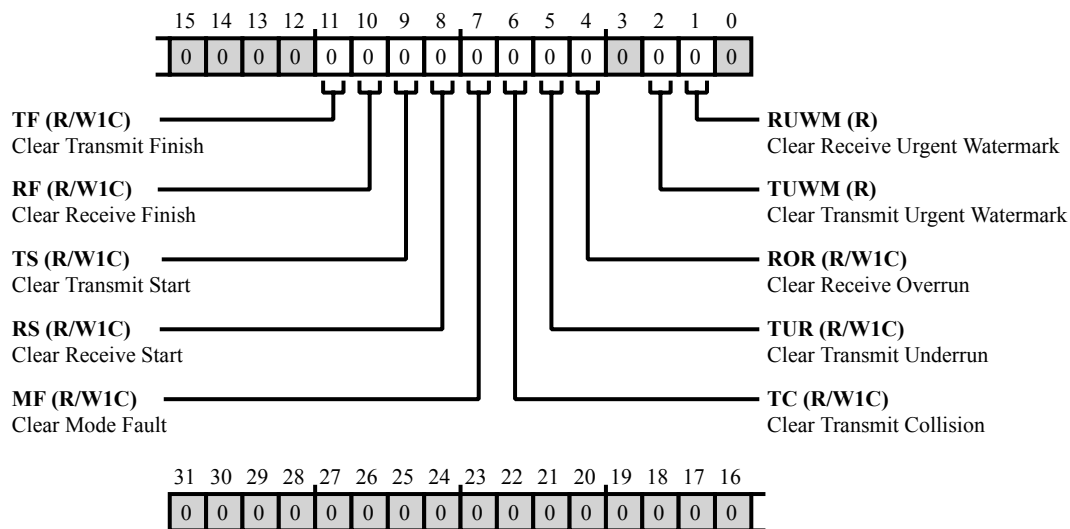


Figure 13-24: `SPI_ILAT_CLR` Register Diagram

Table 13-21: `SPI_ILAT_CLR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
11 (R/W1C)	TF	Clear Transmit Finish. The <code>SPI_ILAT_CLR.TF</code> bit clears the corresponding mask bit in the <code>SPI_ILAT</code> register.
		0 No effect
		1 Clear mask bit
10 (R/W1C)	RF	Clear Receive Finish. The <code>SPI_ILAT_CLR.RF</code> bit clears the corresponding mask bit in the <code>SPI_ILAT</code> register.
		0 No effect
		1 Clear mask bit
9 (R/W1C)	TS	Clear Transmit Start. The <code>SPI_ILAT_CLR.TS</code> bit clears the corresponding mask bit in the <code>SPI_ILAT</code> register.
		0 No effect
		1 Clear mask bit

Table 13-21: SPI_ILAT_CLR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
8 (R/W1C)	RS	Clear Receive Start. The <code>SPI_ILAT_CLR.RS</code> bit clears the corresponding mask bit in the <code>SPI_ILAT</code> register.
		0 No effect
		1 Clear mask bit
7 (R/W1C)	MF	Clear Mode Fault. The <code>SPI_ILAT_CLR.MF</code> bit clears the corresponding mask bit in the <code>SPI_ILAT</code> register.
		0 No effect
		1 Clear mask bit
6 (R/W1C)	TC	Clear Transmit Collision. The <code>SPI_ILAT_CLR.TC</code> bit clears the corresponding mask bit in the <code>SPI_ILAT</code> register.
		0 No effect
		1 Clear mask bit
5 (R/W1C)	TUR	Clear Transmit Underrun. The <code>SPI_ILAT_CLR.TUR</code> bit clears the corresponding mask bit in the <code>SPI_ILAT</code> register.
		0 No effect
		1 Clear mask bit
4 (R/W1C)	ROR	Clear Receive Overrun. The <code>SPI_ILAT_CLR.ROR</code> bit clears the corresponding mask bit in the <code>SPI_ILAT</code> register.
		0 No effect
		1 Clear mask bit
2 (R/NW)	TUWM	Clear Transmit Urgent Watermark. The <code>SPI_ILAT_CLR.TUWM</code> bit clears the corresponding mask bit in the <code>SPI_ILAT</code> register.
		0 No effect
		1 Clear mask bit

Table 13-21: SPI_ILAT_CLR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/NW)	RUWM	Clear Receive Urgent Watermark. The <code>SPI_ILAT_CLR.RUWM</code> bit clears the corresponding mask bit in the <code>SPI_ILAT</code> register.
		0 No effect
		1 Clear mask bit

Interrupt Mask Register

The `SPI_IMSK` register unmask (enables) or mask (disables) SPI interrupt requests. When a condition is indicated by a bit in the `SPI_STAT` register and the corresponding interrupt request is unmasked in `SPI_IMSK`, the SPI latches the interrupt request bit in the `SPI_ILAT` register, queuing the interrupt request for service.

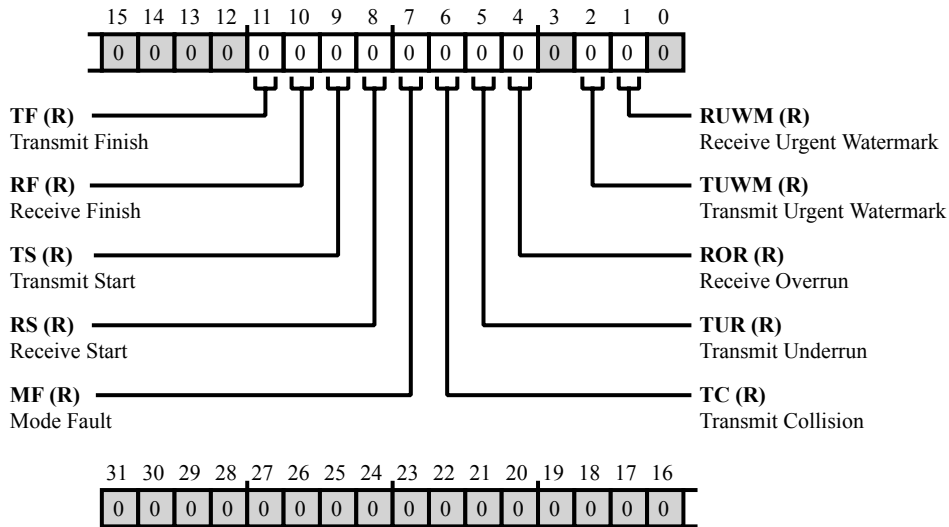


Figure 13-25: SPI_IMSK Register Diagram

Table 13-22: SPI_IMSK Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
11 (R/NW)	TF	Transmit Finish. The <code>SPI_IMSK.TF</code> bit unmask (enables) or mask (disables) the TF interrupt.
		0 Disable (mask) interrupt request
		1 Enable (unmask) interrupt request
10 (R/NW)	RF	Receive Finish. The <code>SPI_IMSK.RF</code> bit unmask (enables) or mask (disables) the RF interrupt.
		0 Disable (mask) interrupt request
		1 Enable (unmask) interrupt request
9 (R/NW)	TS	Transmit Start. The <code>SPI_IMSK.TS</code> bit unmask (enables) or mask (disables) the TS interrupt.
		0 Disable (mask) interrupt request
		1 Enable (unmask) interrupt request

Table 13-22: SPI_IMSK Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
8 (R/NW)	RS	Receive Start. The <code>SPI_IMSK.RS</code> bit unmask (enables) or masks (disables) the RS interrupt.
		0 Disable (mask) interrupt request
		1 Enable (unmask) interrupt request
7 (R/NW)	MF	Mode Fault. The <code>SPI_IMSK.MF</code> bit unmask (enables) or masks (disables) the MF interrupt.
		0 Disable (mask) interrupt request
		1 Enable (unmask) interrupt request
6 (R/NW)	TC	Transmit Collision. The <code>SPI_IMSK.TC</code> bit unmask (enables) or masks (disables) the TC interrupt.
		0 Disable (mask) interrupt request
		1 Enable (unmask) interrupt request
5 (R/NW)	TUR	Transmit Underrun. The <code>SPI_IMSK.TUR</code> bit unmask (enables) or masks (disables) the TUR interrupt.
		0 Disable (mask) interrupt request
		1 Enable (unmask) interrupt request
4 (R/NW)	ROR	Receive Overrun. The <code>SPI_IMSK.ROR</code> bit unmask (enables) or masks (disables) the ROR interrupt.
		0 Disable (mask) interrupt request
		1 Enable (unmask) interrupt request
2 (R/NW)	TUWM	Transmit Urgent Watermark. The <code>SPI_IMSK.TUWM</code> bit unmask (enables) or masks (disables) the TUWM interrupt.
		0 Disable (mask) interrupt request
		1 Enable (unmask) interrupt request
1 (R/NW)	RUWM	Receive Urgent Watermark. The <code>SPI_IMSK.RUWM</code> bit unmask (enables) or masks (disables) the RUWM interrupt.
		0 Disable (mask) interrupt request
		1 Enable (unmask) interrupt request

Interrupt Mask Clear Register

The `SPI_IMSK_CLR` register permits clearing individual mask bits in the `SPI_IMSK` register without affecting other bits in the register. Use write-1-to-clear on a bit in the `SPI_IMSK_CLR` register to clear the corresponding bit in the `SPI_IMSK` register.

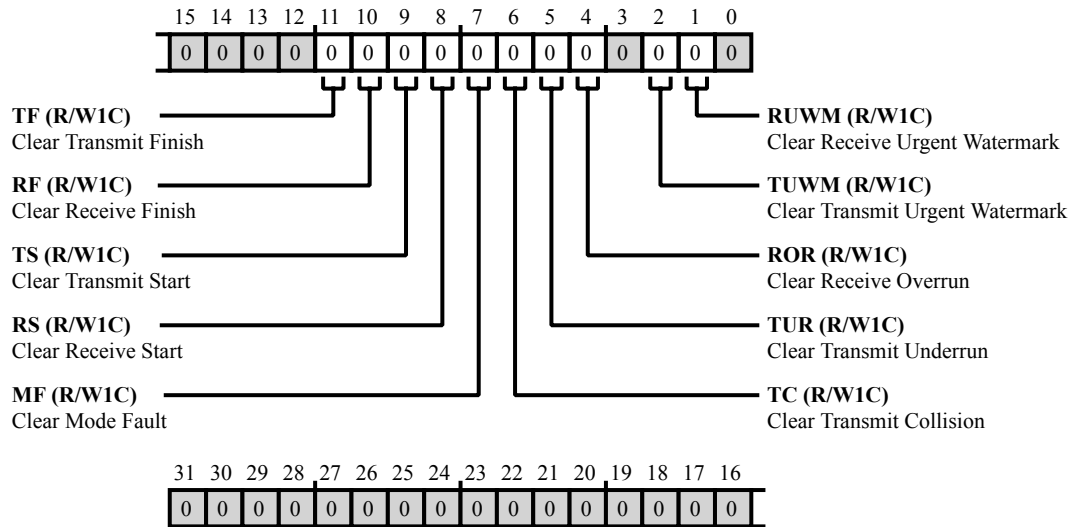


Figure 13-26: SPI_IMSK_CLR Register Diagram

Table 13-23: SPI_IMSK_CLR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
11 (R/W1C)	TF	Clear Transmit Finish. The <code>SPI_IMSK_CLR.TF</code> bit clears the corresponding mask bit in the <code>SPI_IMSK</code> register.
		0 No effect
		1 Clear mask bit
10 (R/W1C)	RF	Clear Receive Finish. The <code>SPI_IMSK_CLR.RF</code> bit clears the corresponding mask bit in the <code>SPI_IMSK</code> register.
		0 No effect
		1 Clear mask bit
9 (R/W1C)	TS	Clear Transmit Start. The <code>SPI_IMSK_CLR.TS</code> bit clears the corresponding mask bit in the <code>SPI_IMSK</code> register.
		0 No effect
		1 Clear mask bit

Table 13-23: SPI_IMSK_CLR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
8 (R/W1C)	RS	Clear Receive Start. The <code>SPI_IMSK_CLR.RS</code> bit clears the corresponding mask bit in the <code>SPI_IMSK</code> register.
		0 No effect
		1 Clear mask bit
7 (R/W1C)	MF	Clear Mode Fault. The <code>SPI_IMSK_CLR.MF</code> bit clears the corresponding mask bit in the <code>SPI_IMSK</code> register.
		0 No effect
		1 Clear mask bit
6 (R/W1C)	TC	Clear Transmit Collision. The <code>SPI_IMSK_CLR.TC</code> bit clears the corresponding mask bit in the <code>SPI_IMSK</code> register.
		0 No effect
		1 Clear mask bit
5 (R/W1C)	TUR	Clear Transmit Underrun. The <code>SPI_IMSK_CLR.TUR</code> bit clears the corresponding mask bit in the <code>SPI_IMSK</code> register.
		0 No effect
		1 Clear mask bit
4 (R/W1C)	ROR	Clear Receive Overrun. The <code>SPI_IMSK_CLR.ROR</code> bit clears the corresponding mask bit in the <code>SPI_IMSK</code> register.
		0 No effect
		1 Clear mask bit
2 (R/W1C)	TUWM	Clear Transmit Urgent Watermark. The <code>SPI_IMSK_CLR.TUWM</code> bit clears the corresponding mask bit in the <code>SPI_IMSK</code> register.
		0 No effect
		1 Clear mask bit

Table 13-23: SPI_IMSK_CLR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
1 (R/W1C)	RUWM	Clear Receive Urgent Watermark. The <code>SPI_IMSK_CLR.RUWM</code> bit clears the corresponding mask bit in the <code>SPI_IMSK</code> register.	
		0	No effect
		1	Clear mask bit

Interrupt Mask Set Register

The `SPI_IMSK_SET` register permits setting individual mask bits in the `SPI_IMSK` register without affecting other bits in the register. Use write-1-to-set on a bit in the `SPI_IMSK_SET` register to set the corresponding bit in the `SPI_IMSK` register.

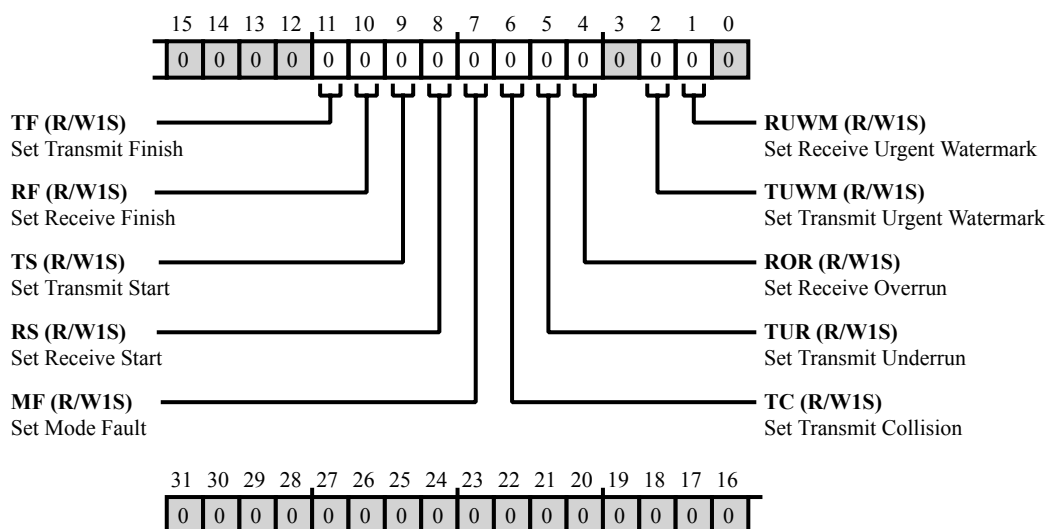


Figure 13-27: SPI_IMSK_SET Register Diagram

Table 13-24: SPI_IMSK_SET Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
11 (R/W1S)	TF	Set Transmit Finish. The <code>SPI_IMSK_SET.TF</code> bit sets the corresponding mask bit in the <code>SPI_IMSK</code> register.
		0 No effect
		1 Set mask bit
10 (R/W1S)	RF	Set Receive Finish. The <code>SPI_IMSK_SET.RF</code> bit sets the corresponding mask bit in the <code>SPI_IMSK</code> register.
		0 No effect
		1 Set mask bit
9 (R/W1S)	TS	Set Transmit Start. The <code>SPI_IMSK_SET.TS</code> bit sets the corresponding mask bit in the <code>SPI_IMSK</code> register.
		0 No effect
		1 Set mask bit

Table 13-24: SPI_IMSK_SET Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
8 (R/W1S)	RS	Set Receive Start. The <code>SPI_IMSK_SET.RS</code> bit sets the corresponding mask bit in the <code>SPI_IMSK</code> register.
		0 No effect
		1 Set mask bit
7 (R/W1S)	MF	Set Mode Fault. The <code>SPI_IMSK_SET.MF</code> bit sets the corresponding mask bit in the <code>SPI_IMSK</code> register.
		0 No effect
		1 Set mask bit
6 (R/W1S)	TC	Set Transmit Collision. The <code>SPI_IMSK_SET.TC</code> bit sets the corresponding mask bit in the <code>SPI_IMSK</code> register.
		0 No effect
		1 Set mask bit
5 (R/W1S)	TUR	Set Transmit Underrun. The <code>SPI_IMSK_SET.TUR</code> bit sets the corresponding mask bit in the <code>SPI_IMSK</code> register.
		0 No effect
		1 Set mask bit
4 (R/W1S)	ROR	Set Receive Overrun. The <code>SPI_IMSK_SET.ROR</code> bit sets the corresponding mask bit in the <code>SPI_IMSK</code> register.
		0 No effect
		1 Set mask bit
2 (R/W1S)	TUWM	Set Transmit Urgent Watermark. The <code>SPI_IMSK_SET.TUWM</code> bit sets the corresponding mask bit in the <code>SPI_IMSK</code> register.
		0 No effect
		1 Set mask bit

Table 13-24: SPI_IMSK_SET Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/W1S)	RUWM	Set Receive Urgent Watermark. The <code>SPI_IMSK_SET</code> .RUWM bit sets the corresponding mask bit in the <code>SPI_IMSK</code> register.
		0 No effect
		1 Set mask bit

Memory Mapped Read Header

The `SPI_MMRDH` register enables the use of memory-mapped mode. This mode allows direct memory-mapped read accesses of an SPI memory device and is primarily used to directly execute instructions from an SPI FLASH memory without using a low-level software driver. All overhead tasks such as transmission of the read header, pin turnaround timing and receive data sizing are handled in hardware.

The memory-mapped access mode is enabled by setting the `SPI_CTL.MMSE` bit. The features within the `SPI_MMRDH` register include a command skip mode, variable length byte addressing, and independent multi-pin support for command transmission, address transmission and data reception. In addition, the command opcode and mode bytes are fully programmable.

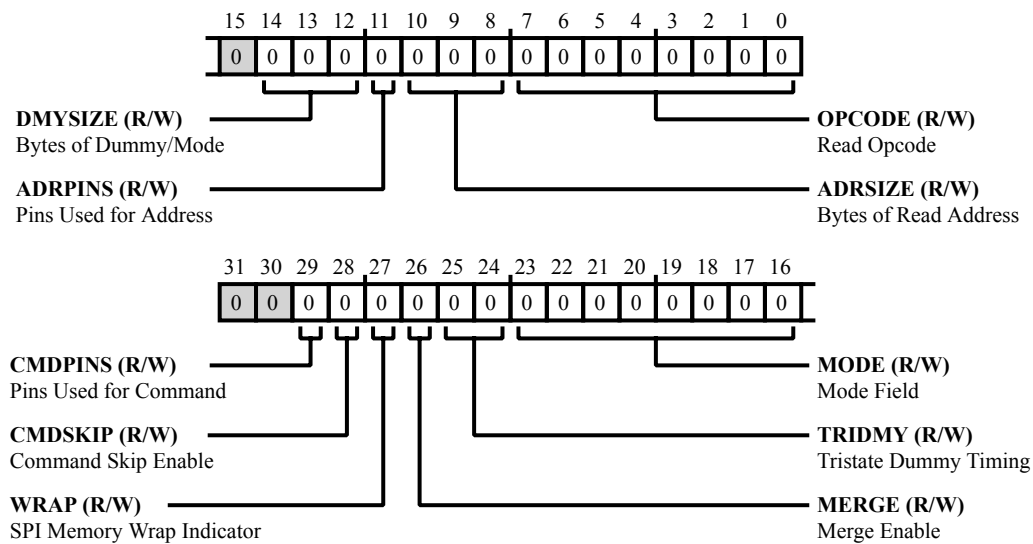


Figure 13-28: `SPI_MMRDH` Register Diagram

Table 13-25: `SPI_MMRDH` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29 (R/W)	CMDPINS	Pins Used for Command.
		The <code>SPI_MMRDH.CMDPINS</code> bit specifies the number of pins to be used for command transmission. This bit must be set consistent with the expectations established by the read opcode. Hardware does not interpret <code>SPI_MMRDH.OPCODE</code> , but rather relies on this bit to specify behavior. When cleared, it overrides the <code>SPI_CTL.MIOM</code> bits. When set, it uses bits specified by the <code>SPI_CTL.MIOM</code> bit setting.
		0 Use only one pin: MOSI (overrides <code>SPI_CTL.MIOM</code> bits)
		1 Use pins specified by <code>SPI_CTL.MIOM</code> bits

Table 13-25: SPI_MMRDH Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
28 (R/W)	CMDSKIP	Command Skip Enable. The <code>SPI_MMRDH.CMDSKIP</code> bit enables command skip mode where the address is sent first and the <code>OPCODE</code> field is not sent (<code>SPI_MMRDH.CMDSKIP</code> bit =1). This mode is useful for supporting XIP (Execute-In-Place) operation where only the address is sent and the same read command is assumed. The SPI flash device must be primed with an initial read command, before the <code>SPI_MMRDH.CMDSKIP</code> bit is set.
		0 <code>OPCODE</code> field is sent first followed by address
		1 <code>OPCODE</code> field is not sent; address is sent first
27 (R/W)	WRAP	SPI Memory Wrap Indicator. The <code>SPI_MMRDH.WRAP</code> bit must be set by software if software places a connected SPI memory device into a 8-byte, 16-byte or 32-byte wrap mode based on the <code>ILINE</code> and <code>DLINE</code> field setting of the cache configuration register address wrap mode. Software achieves this by transmitting a vendor specified command to the SPI memory device while the <code>SPI_CTL.MMSE</code> bit =0. If the <code>SPI_MMRDH.WRAP</code> bit =1, the SPI does not need to deassert the SPI slave select signal and resend the read header in order to wrap to the cache line base when servicing misaligned cache fill requests. Although this improves cache fill efficiency, it requires that the SPI deassert the SPI slave select pin and resend the read header whenever a DMA burst requests crosses 32 byte alignments. Setting this bit improves cache throughput but decreases DMA throughput.
		0 SPI Memory auto increments address purely sequentially
		1 SPI Memory auto increments address but wraps within 32 Byte lines
26 (R/W)	MERGE	Merge Enable. When the <code>SPI_MMRDH.MERGE</code> bit is set, SPI hardware combines the two successive transfers. This increases the throughput rate when accessing a large number of sequential memory locations. For more information refer to the Merged Read Accesses section.

Table 13-25: SPI_MMRDH Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
25:24 (R/W)	TRIDMY	Tristate Dummy Timing. The SPI_MMRDH.TRIDMY bits specify whether and when output pins are three-stated during the interval of time specified by the SPI_MMRDH.DMYSIZE bits. Output pins potentially three-stated include all pins which were used to transmit the address
		0 Tristate outputs immediately
		1 Tristate outputs after 4 bits of dummy/mode are transmitted
		2 Tristate outputs after 8 bits of dummy/mode are transmitted
		3 Never tristate outputs (previously specified output state is held)
23:16 (R/W)	MODE	Mode Field. These bits specify up to a leading byte to be transmitted during the interval of time specified by the SPI_MMRDH.DMYSIZE bit field. This first byte, or a portion of it, is interpreted as mode bits when certain opcodes are used in conjunction with certain SPI memory devices. Mode bits are sent using the same number of pins which were used to transmit the address. Once sent, output pins will be held in their final resultant state until the conclusion of all dummy byte periods, unless three-stating the outputs is specified first by the SPI_MMRDH.TRIDMY bits.
14:12 (R/W)	DMYSIZE	Bytes of Dummy/Mode. The SPI_MMRDH.DMYSIZE bit field specifies the number of bytes separating address transmission and read data return. Dummy bytes elapse assuming dummy bits are transmitted using the same number of pins which were used to transmit address.
		0 0 Bytes
		1 1 Bytes
		2 2 Bytes
		3 3 Bytes
		4 4 Bytes
		5 5 Bytes
		6 6 Bytes
		7 7 Bytes

Table 13-25: SPI_MMRDH Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
11 (R/W)	ADRPINS	Pins Used for Address. The <code>SPI_MMRDH.ADRPINS</code> bit specifies the number of pins to be used for address transmission. This bit must be set consistent with expectations established by read opcode. Hardware does not interpret the <code>SPI_MMRDH.OPCODE</code> , but rather relies on this bit to specify behavior.
		0 Use only one pin: MOSI (overrides <code>SPI_CTL.MIOM</code> bits)
		1 Use pins specified by <code>SPI_CTL.MIOM</code> bits
10:8 (R/W)	ADRSIZE	Bytes of Read Address. The <code>SPI_MMRDH.ADRSIZE</code> bit field defines the number of bytes used to specify the read address. The read address is sent immediately following the transmission of opcode. Unlike opcode bits, address bits may be sent using either one or multiple pins. The number of pins is selected using the <code>SPI_MMRDH.ADRPINS</code> bit. The address sent to a connected SPI memory device is an echo of the read address received by the SPI peripheral slave port. The least significant bytes of address are sent when the entire address is not sent.
		0 1 Byte
		1 1 Byte
		2 2 Bytes
		3 3 Bytes
		4 4 Bytes
7:0 (R/W)	OPCODE	Read Opcode. The <code>SPI_MMRDH.OPCODE</code> bit field specifies the initial bits transmitted in response to a read request of SPI memory. Although any opcode may be sent, values 0x03, 0x0B, 0x3B, 0x6B, 0xBB, and 0xEB are likely to be the most commonly used. <code>SPI_MMRDH.OPCODE</code> is sent by the SPI without interpretation; the states of these bits have no effect beyond specifying what is initially shifted across the SPI interface.

SPI Memory Top Address

The `SPI_MMTOP` register specifies the top populated address of a connected SPI memory device.

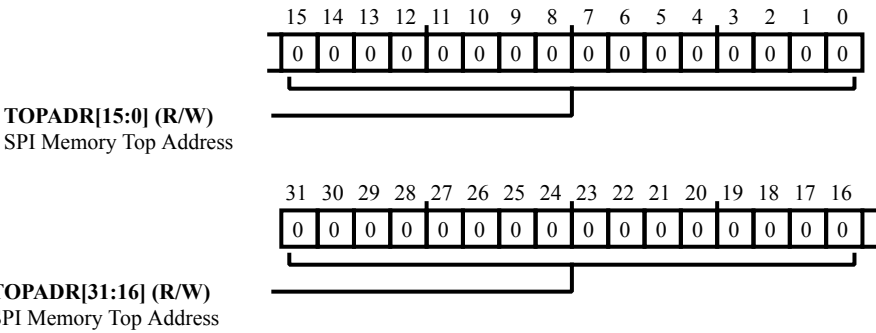


Figure 13-29: SPI_MMTOP Register Diagram

Table 13-26: SPI_MMTOP Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	TOPADR	SPI Memory Top Address. The <code>SPI_MMTOP.TOPADR</code> bit field specifies the top populated address of a connected SPI memory device. Attempts to access SPI memory are not blocked if this address is exceeded and an error is generated as part of the read response.

Receive FIFO Data Register

The `SPI_RFIFO` register has an interface to the receive shift register in the SPI and has an interface to the processor's data buses. The top level of the buffer is visible to programs as the 32-bit `SPI_RFIFO` register, but the size (number of word locations) of the receive FIFO is actually flexible with transfer word size. The size of the receive FIFO is 8 if the word size is 8-bit, or the size is 4 if the word size is 16-bit, or the size is 2 if the word size is 32-bit.

Both masters and slaves may stop or stall receive transfers based on FIFO status. When the receive FIFO is full, the SPI master stops initiating new transfers on the SPI if `SPI_RXCTL.RTI` is enabled. A slave may stall the SPI interface when the content of the FIFO crosses the selected watermark. If data reception continues after `SPI_RFIFO` is full, the data in the receive FIFO is invalid. The SPI indicates this condition with receive overrun (`SPI_STAT.ROR`) error. This condition is possible when `SPI_RXCTL.RTI` = 0 and `SPI_RXCTL.REN` = 1 for a master, or for a slave that does not exercise flow control.

Note that the receive FIFO is reset (cleared) when the SPI is disabled after being enabled.

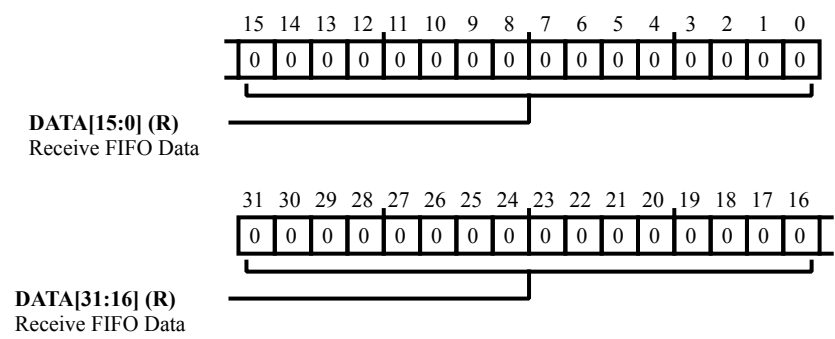


Figure 13-30: SPI_RFIFO Register Diagram

Table 13-27: SPI_RFIFO Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	DATA	Receive FIFO Data. The <code>SPI_RFIFO.DATA</code> bit field contains the FIFO receive data.

Received Word Count Register

The `SPI_RWC` register holds a count of the number of words remaining to be received by the SPI. To start the decrement of the word count in `SPI_RWC`, enable the receive word counter (`SPI_RXCTL.RWCEN = 1`). The SPI uses the word count to control the duration of transfers and to signal the completion of a burst of transfers with the receive finish interrupt (`SPI_ILAT.RF`). In DMA mode, the SPI uses the `SPI_RWC` register to ensure that the number of frames received during a DMA transfer is equal to the number of words programmed in the DMA channel controller. The values programmed into the `SPI_RWC` registers should match the word count in the DMA configuration. The `SPI_RWC` register maintains the number of frames to be received in a transfer. The `SPI_RWC` should only be changed when the counter is disabled.

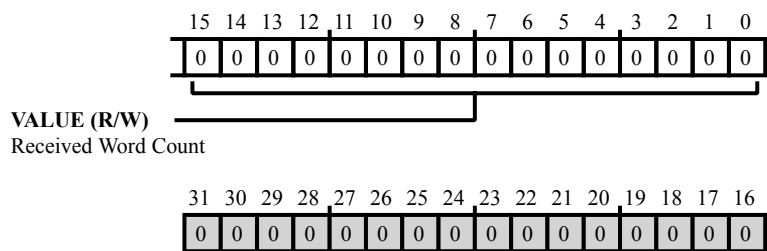


Figure 13-31: SPI_RWC Register Diagram

Table 13-28: SPI_RWC Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	VALUE	Received Word Count. The <code>SPI_RWC.VALUE</code> bits hold the receive transfer word count.

Received Word Count Reload Register

The `SPI_RWCR` register holds the receive word count value that the SPI loads into the `SPI_RWC` register when the transfer count decrements to zero. To prevent the SPI from reloading the counter, use zero for the reload count value. The `SPI_RWCR` register should only be changed when the counter is disabled.

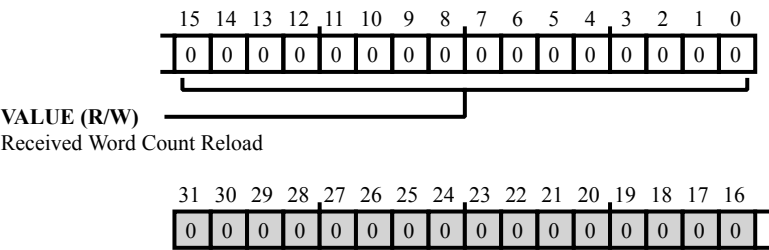


Figure 13-32: SPI_RWCR Register Diagram

Table 13-29: SPI_RWCR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	VALUE	Received Word Count Reload. The <code>SPI_RWCR.VALUE</code> bits hold the receive transfer word count reload value.

Receive Control Register

The `SPI_RXCTL` register enables the SPI receive channel, initiates receive transfers, and configures `SPI_RFIFO` buffer watermark settings.

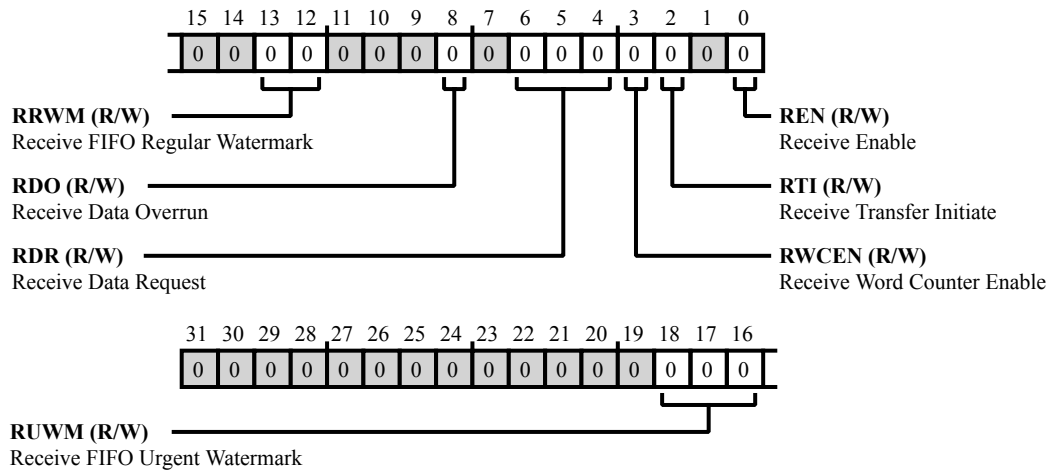


Figure 13-33: SPI_RXCTL Register Diagram

Table 13-30: SPI_RXCTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
18:16 (R/W)	RUWM	Receive FIFO Urgent Watermark. The <code>SPI_RXCTL.RUWM</code> bits select the receive FIFO (<code>SPI_RFIFO</code>) watermark level for urgent data bus requests. The SPI also uses this watermark level for generation of the <code>SPI_ILAT.RUWM</code> interrupt. When an urgent <code>SPI_RFIFO</code> watermark is enabled with <code>SPI_RXCTL.RUWM</code> , the <code>SPI_RXCTL.RRWM</code> selection is used as the deassertion condition for any <code>SPI_ILAT.RUWM</code> interrupts that are latched.
		0 Disabled
		1 25% full RFIFO
		2 50% full RFIFO
		3 75% full RFIFO
		4 Full RFIFO
		5 Reserved
		6 Reserved
		7 Reserved

Table 13-30: SPI_RXCTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
13:12 (R/W)	RRWM	Receive FIFO Regular Watermark. The SPI_RXCTL.RRWM bits select the receive FIFO (SPI_RFIFO) watermark level for regular data bus requests. When an urgent SPI_RFIFO watermark is enabled with SPI_RXCTL.RUWM, the SPI_RXCTL.RRWM selection is used as the deassertion condition for any SPI_ILAT.RUWM interrupts that are latched.
		0 Empty RFIFO
		1 RFIFO less than 25% full
		2 RFIFO less than 50% full
		3 RFIFO less than 75% full
8 (R/W)	RDO	Receive Data Overrun. The SPI_RXCTL.RDO bit selects handling for receive data requests when the receive buffer (SPI_RFIFO) is full. If enabled and SPI_RFIFO is full, the SPI overwrites old data in the buffer with incoming data. If disabled and SPI_RFIFO is full, the SPI keeps old data in the buffer and discards incoming data.
		0 Discard incoming data if SPI_RFIFO is full
		1 Overwrite old data if SPI_RFIFO is full
6:4 (R/W)	RDR	Receive Data Request. The SPI_RXCTL.RDR bits select receive FIFO (SPI_RFIFO) watermark conditions that direct the SPI to generate a receive data request.
		0 Disabled
		1 Not empty RFIFO
		2 25% full RFIFO
		3 50% full RFIFO
		4 75% full RFIFO
		5 Full RFIFO
		6 Reserved
		7 Reserved
3 (R/W)	RWCEN	Receive Word Counter Enable. The SPI_RXCTL.RWCEN bit enables the decrement of the SPI_RWC register when the count is not zero and SPI_RXCTL.RTI is enabled. Enabling SPI_RXCTL.RWCEN prevents receive overrun errors from occurring. The SPI_RXCTL.RWCEN bit is valid only when the SPI is a master.
		0 Disable
		1 Enable

Table 13-30: SPI_RXCTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
2 (R/W)	RTI	Receive Transfer Initiate. The <code>SPI_RXCTL.RTI</code> bit enables initiation of receive transfers if the receive FIFO (<code>SPI_RFIFO</code>) is not full. The bit also enables this initiation if <code>SPI_RWC</code> is not zero when <code>SPI_RXCTL.RWCEN</code> is enabled. Enabling <code>SPI_RXCTL.RTI</code> prevents receive overrun errors from occurring. The <code>SPI_RXCTL.RTI</code> bit is valid only when the SPI is a master.
		0 Disable
		1 Enable
0 (R/W)	REN	Receive Enable. The <code>SPI_RXCTL.REN</code> bit enables SPI receive channel operation.
		0 Disable
		1 Enable

Slave Select Register

The `SPI_SLVSEL` register enables the `SPI_SEL[n]` pins for output and indicates the state (high or low) of these pins when enabled.

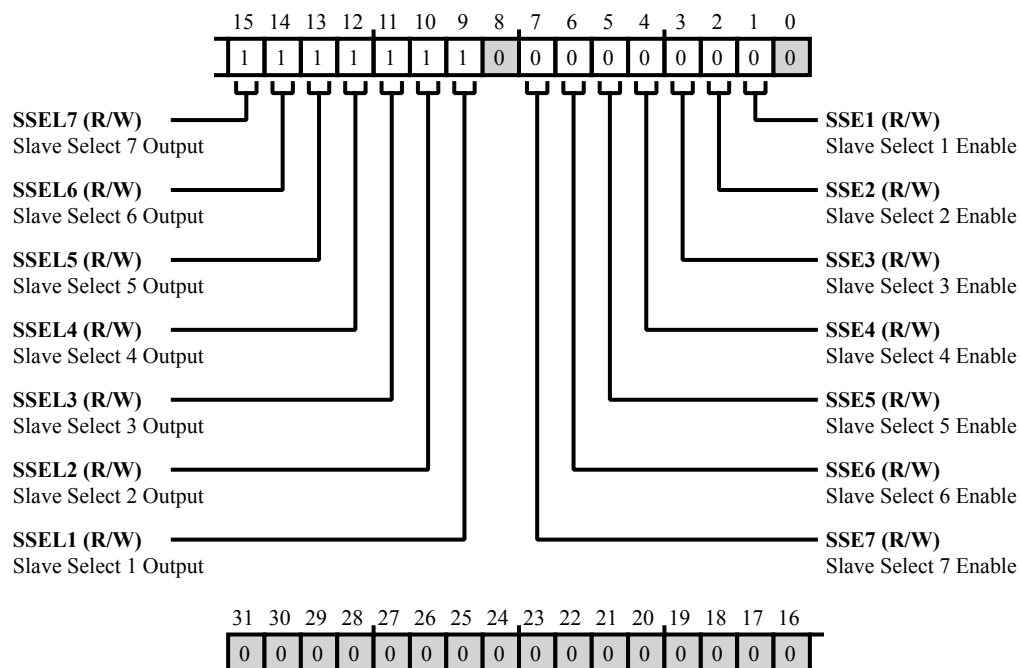


Figure 13-34: SPI_SLVSEL Register Diagram

Table 13-31: SPI_SLVSEL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W)	SSEL7	Slave Select 7 Output. The <code>SPI_SLVSEL.SSEL7</code> bit state indicates the value driven on the related <code>SPI_SEL[n]</code> pin.
		0 Low
		1 High
14 (R/W)	SSEL6	Slave Select 6 Output. The <code>SPI_SLVSEL.SSEL6</code> bit state indicates the value driven on the related <code>SPI_SEL[n]</code> pin.
		0 Low
		1 High

Table 13-31: SPI_SLVSEL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
13 (R/W)	SSEL5	Slave Select 5 Output. The <code>SPI_SLVSEL.SSEL5</code> bit state indicates the value driven on the related <code>SPI_SEL[n]</code> pin.
		0 Low
		1 High
12 (R/W)	SSEL4	Slave Select 4 Output. The <code>SPI_SLVSEL.SSEL4</code> bit state indicates the value driven on the related <code>SPI_SEL[n]</code> pin.
		0 Low
		1 High
11 (R/W)	SSEL3	Slave Select 3 Output. The <code>SPI_SLVSEL.SSEL3</code> bit state indicates the value driven on the related <code>SPI_SEL[n]</code> pin.
		0 Low
		1 High
10 (R/W)	SSEL2	Slave Select 2 Output. The <code>SPI_SLVSEL.SSEL2</code> bit state indicates the value driven on the related <code>SPI_SEL[n]</code> pin.
		0 Low
		1 High
9 (R/W)	SSEL1	Slave Select 1 Output. The <code>SPI_SLVSEL.SSEL1</code> bit state indicates the value driven on the related <code>SPI_SEL[n]</code> pin.
		0 Low
		1 High
7 (R/W)	SSE7	Slave Select 7 Enable. The <code>SPI_SLVSEL.SSE7</code> bit enables the related <code>SPI_SEL[n]</code> pin for output. If disabled, the SPI three-states the related <code>SPI_SEL[n]</code> pin.
		0 Disable
		1 Enable

Table 13-31: SPI_SLVSEL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
6 (R/W)	SSE6	Slave Select 6 Enable. The SPI_SLVSEL.SSE6 bit enables the related $\overline{\text{SPI_SEL}}[n]$ pin for output. See the SPI_SLVSEL.SSE7 bit description for more information.
		0 Disable
		1 Enable
5 (R/W)	SSE5	Slave Select 5 Enable. The SPI_SLVSEL.SSE5 bit enables the related $\overline{\text{SPI_SEL}}[n]$ pin for output. See the SPI_SLVSEL.SSE7 bit description for more information.
		0 Disable
		1 Enable
4 (R/W)	SSE4	Slave Select 4 Enable. The SPI_SLVSEL.SSE4 bit enables the related $\overline{\text{SPI_SEL}}[n]$ pin for output. See the SPI_SLVSEL.SSE7 bit description for more information.
		0 Disable
		1 Enable
3 (R/W)	SSE3	Slave Select 3 Enable. The SPI_SLVSEL.SSE3 bit enables the related $\overline{\text{SPI_SEL}}[n]$ pin for output. See the SPI_SLVSEL.SSE7 bit description for more information.
		0 Disable
		1 Enable
2 (R/W)	SSE2	Slave Select 2 Enable. The SPI_SLVSEL.SSE2 bit enables the related $\overline{\text{SPI_SEL}}[n]$ pin for output. See the SPI_SLVSEL.SSE7 bit description for more information.
		0 Disable
		1 Enable
1 (R/W)	SSE1	Slave Select 1 Enable. The SPI_SLVSEL.SSE1 bit enables the related $\overline{\text{SPI_SEL}}[n]$ pin for output. See the SPI_SLVSEL.SSE7 bit description for more information.
		0 Disable
		1 Enable

Status Register

The `SPI_STAT` register indicates SPI status including FIFO status, error conditions, and interrupt conditions. When an interrupt condition from this register is unmasked (enabled) by the corresponding bit in the `SPI_IMSK` register, the interrupt request is latched into the corresponding bit in the `SPI_ILAT` register.

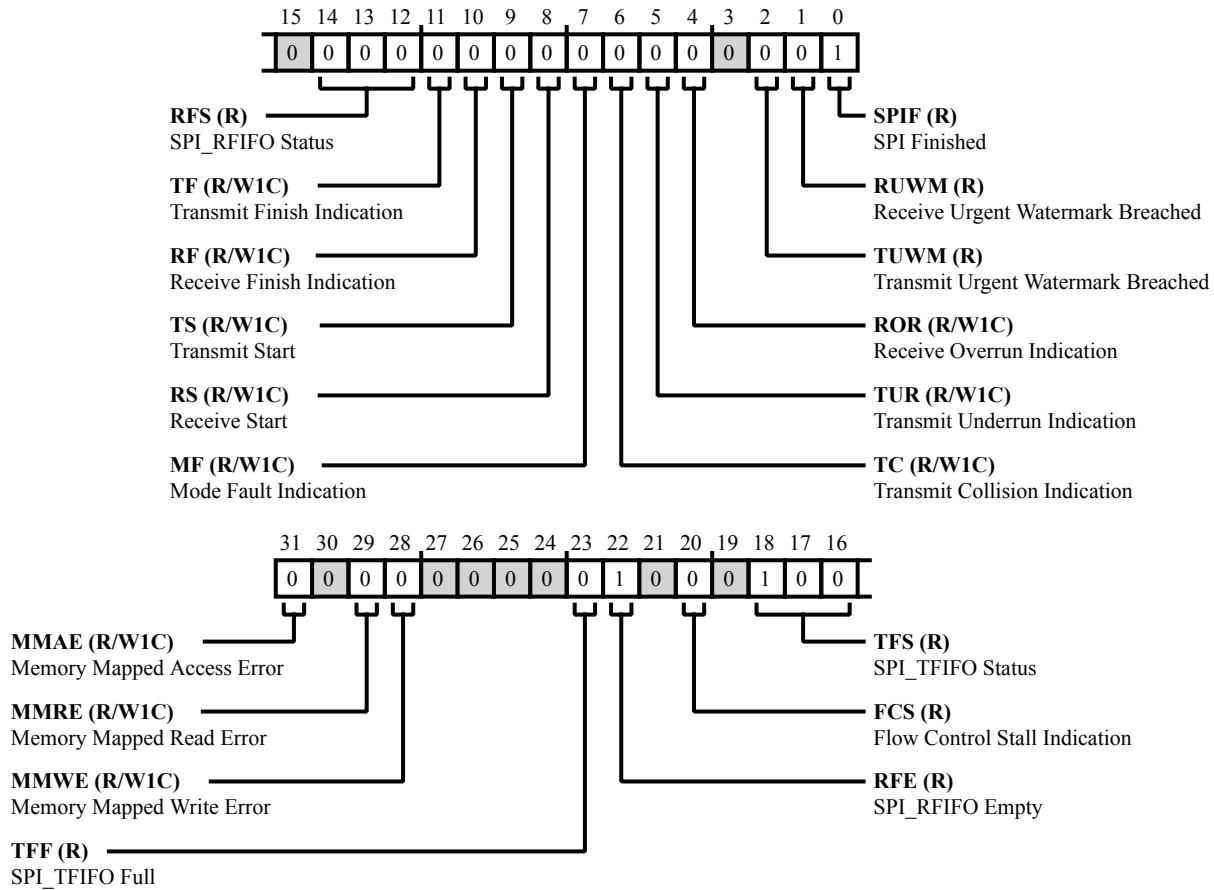


Figure 13-35: SPI_STAT Register Diagram

Table 13-32: SPI_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W1C)	MMAE	Memory Mapped Access Error. The <code>SPI_STAT.MMAE</code> bit =1 if an attempt is made to access either the Tx or Rx FIFO while memory-mapped access of SPI memory is enabled (see the <code>SPI_CTL.MMSE</code> bit). The <code>SPI_STAT.MMAE</code> bit =0 when a 1 is written to it. The <code>SPI_STAT.MMAE</code> bit is provided for software notification only. Its state has no further effect.

Table 13-32: SPI_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
29 (R/W1C)	MMRE	Memory Mapped Read Error. The <code>SPI_STAT.MMRE</code> bit =1 if an attempt is made to read address space reserved for memory-mapped SPI memory while memory mapping is disabled (see the <code>SPI_CTL.MMSE</code> bit). The <code>SPI_STAT.MMRE</code> bit =0 when a 1 is written to it. This bit is provided for software notification only. Its state has no further effect.
28 (R/W1C)	MMWE	Memory Mapped Write Error. The <code>SPI_STAT.MMWE</code> bit =1 if an attempt is made to write address space reserved for memory-mapped SPI memory. The <code>SPI_STAT.MMWE</code> bit =0 when a 1 is written to it. This bit is provided for software notification only. Its state has no further effect.
23 (R/NW)	TFF	SPI_TFIFO Full. The <code>SPI_STAT.TFF</code> bit indicates whether the <code>SPI_TFIFO</code> is full or not full.
		0 Not full Tx FIFO
		1 Full Tx FIFO
22 (R/NW)	RFE	SPI_RFIFO Empty. The <code>SPI_STAT.RFE</code> bit indicates whether the <code>SPI_RFIFO</code> is empty or not empty.
		0 Rx FIFO not empty
		1 Rx FIFO empty
20 (R/NW)	FCS	Flow Control Stall Indication. The <code>SPI_STAT.FCS</code> bit indicates whether a slave has deasserted the <code>SPI_RDY</code> pin to stall the SPI master while the slave is unable to service the SPI masters request. This bit is valid only when the SPI is a master (<code>SPI_CTL.MSTR</code> =1) and flow control is enabled (<code>SPI_CTL.FCEN</code> =1).
		0 No Stall (RDY pin asserted)
		1 Stall (RDY pin deasserted)
18:16 (R/NW)	TFS	SPI_TFIFO Status. The <code>SPI_STAT.TFS</code> bits indicate the status of the <code>SPI_TFIFO</code> . The SPI uses this status when evaluating transmit watermark conditions.
		0 Full TFIFO
		1 25% empty TFIFO
		2 50% empty TFIFO
		3 75% empty TFIFO
		4 Empty TFIFO

Table 13-32: SPI_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
14:12 (R/NW)	RFS	SPI_RFIFO Status. The SPI_STAT.RFS bits indicate the status of the SPI_RFIFO. The SPI uses this status when evaluating receive watermark conditions.
		0 Empty RFIFO
		1 25% full RFIFO
		2 50% full RFIFO
		3 75% full RFIFO
		4 Full RFIFO
11 (R/W1C)	TF	Transmit Finish Indication. The SPI_STAT.TF bit indicates that the SPI has detected the finish of a transmit burst transfer (the SPI_TWC count decrements to zero). This condition can only occur when SPI_TXCTL.TTI and SPI_TXCTL.TWCEN are enabled.
		0 No status
		1 Transmit finish detected
10 (R/W1C)	RF	Receive Finish Indication. The SPI_STAT.RF bit indicates that the SPI has detected the finish of a receive burst transfer (the SPI_RWC count decrements to zero). This condition can only occur when SPI_RXCTL.RTI and SPI_RXCTL.RWCEN are enabled.
		0 No status
		1 Receive finish detected
9 (R/W1C)	TS	Transmit Start. The SPI_STAT.TS bit indicates that the SPI has detected the start of a transmit burst transfer. A transmit bursts starts with the load of SPI_TWC from the SPI_TWCR. This condition can only occur when SPI_TXCTL.TTI and SPI_TXCTL.TWCEN are enabled.
		0 No status
		1 Transmit start detected
8 (R/W1C)	RS	Receive Start. The SPI_STAT.RS bit indicates that the SPI has detected the start of a receive burst transfer. A receive bursts starts with the load of SPI_RWC from the SPI_RWCR. This condition can only occur when SPI_RXCTL.RTI and SPI_RXCTL.RWCEN are enabled.
		0 No status
		1 Receive start detected

Table 13-32: SPI_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
7 (R/W1C)	MF	Mode Fault Indication. The <code>SPI_STAT.MF</code> bit, when SPI is a master and <code>SPI_CTL.PSSE</code> is enabled, indicates that multiple masters have asserted slave select inputs.
		0 No status
		1 Mode fault occurred
6 (R/W1C)	TC	Transmit Collision Indication. The <code>SPI_STAT.TC</code> bit, when SPI is a slave, indicates that the load of data into the shift register has occurred too close to the first transmitting edge of the SPI clock.
		0 No status
		1 Transmit collision occurred
5 (R/W1C)	TUR	Transmit Underrun Indication. The <code>SPI_STAT.TUR</code> bit, when the transmit FIFO (<code>SPI_TFIFO</code>) is empty, indicates that the last word in the transmit FIFO has been re-sent as transmit data. Alternately, it indicates that zero has been sent as transmit data.
		0 No status
		1 Transmit underrun occurred
4 (R/W1C)	ROR	Receive Overrun Indication. The <code>SPI_STAT.ROR</code> bit, when the receive FIFO (<code>SPI_RFIFO</code>) is full, indicates that a word in the receive FIFO has been overwritten with incoming receive data. Alternately, it indicates that incoming receive data has been discarded.
		0 No status
		1 Receive overrun occurred
2 (R/NW)	TUWM	Transmit Urgent Watermark Breached. The <code>SPI_STAT.TUWM</code> bit indicates that the transmit urgent watermark (<code>SPI_TXCTL.TUWM</code>) has been reached. This condition is cleared when the transmit FIFO fills enough to reach the transmit regular watermark (<code>SPI_TXCTL.TRWM</code>).
		0 Tx regular watermark reached
		1 Tx urgent watermark breached
1 (R/NW)	RUWM	Receive Urgent Watermark Breached. The <code>SPI_STAT.RUWM</code> bit indicates that the receive urgent watermark (<code>SPI_RXCTL.RUWM</code>) has been reached. This condition is cleared when the receive FIFO empties enough to reach the receive regular watermark (<code>SPI_RXCTL.RRWM</code>).
		0 Rx regular watermark reached
		1 Rx urgent watermark breached

Table 13-32: SPI_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/NW)	SPIF	SPI Finished. The <code>SPI_STAT.SPIF</code> bit indicates that a single word transfer is complete.
		0 No status
		1 Completed single word transfer

Transmit FIFO Data Register

The `SPI_TFIFO` register has an interface to the transmit shift register in the SPI and has an interface to the processor’s data buses. The top level of the buffer is visible to programs as the 32-bit `SPI_TFIFO` register, but the size (number of word locations) of the transmit FIFO is actually flexible with transfer word size. The size of the transmit FIFO is 8 if the word size is 8-bit, or the size is 4 if the word size is 16-bit, or the size is 2 if the word size is 32-bit.

Both masters and slaves may stop or stall transmit transfers based on FIFO status. When the transmit FIFO is empty, the SPI master stops initiating new transfers on the SPI if `SPI_TXCTL.TTI` is enabled. A slave may stall the SPI interface when the content of the FIFO crosses the selected watermark. If data transmit requests continue after `SPI_TFIFO` is empty, the data sent from the transmit FIFO is invalid, and the SPI indicates this condition with transmit underrun (`SPI_STAT.TUR`). This condition is possible when `SPI_TXCTL.TTI` =0 and `SPI_TXCTL.TEN` =1 for a master, or for a slave that does not exercise flow control.

Note that the transmit FIFO is reset (cleared) when the SPI is disabled after being enabled.

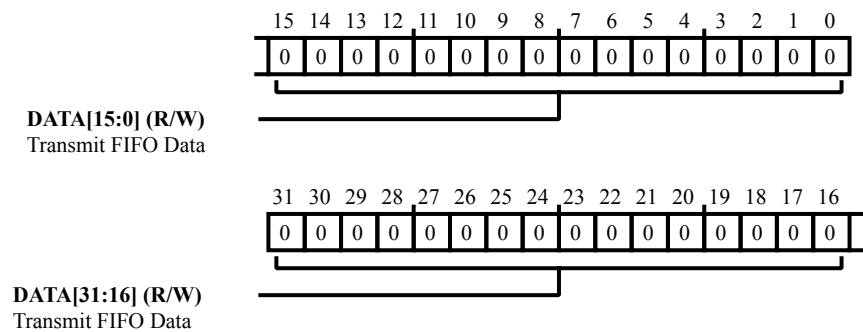


Figure 13-36: SPI_TFIFO Register Diagram

Table 13-33: SPI_TFIFO Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	DATA	Transmit FIFO Data. The <code>SPI_TFIFO.DATA</code> bit field contains the FIFO transmit data.

Transmitted Word Count Register

The `SPI_TWC` register holds a count of the number of words remaining to be transmitted by the SPI. To start the decrement of the word count in `SPI_TWC`, enable the transmit word counter (`SPI_TXCTL.TWCEN = 1`). The SPI uses the word count to control the duration of transfers and to signal the completion of a burst of transfers with the transmit finish interrupt request. In DMA mode, the SPI uses the `SPI_TWC` to ensure that the number of frames transmitted during a DMA transfer is equal to the number of words programmed in the DMA channel controller. The values programmed into the `SPI_TWC` registers should match the word count in the DMA configuration. The `SPI_TWC` maintains the number of frames to be transmitted in a transfer. The `SPI_TWC` should only be changed when the counter is disabled.

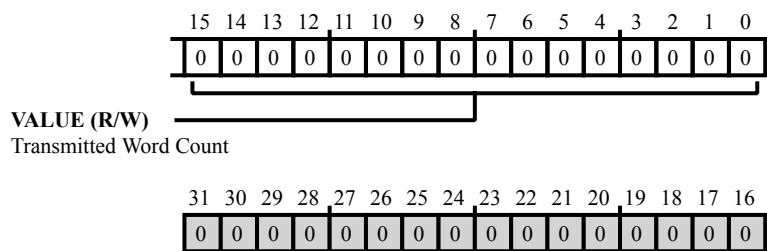


Figure 13-37: SPI_TWC Register Diagram

Table 13-34: SPI_TWC Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	VALUE	Transmitted Word Count. The <code>SPI_TWC.VALUE</code> bits hold the transmit transfer word count.

Transmitted Word Count Reload Register

The `SPI_TWCR` register holds the transmit word count value that the SPI loads into the `SPI_TWC` register when the transfer count decrements to zero. To prevent the SPI from reloading the counter, use zero for the reload count value. The `SPI_TWCR` should only be changed when the counter is disabled.

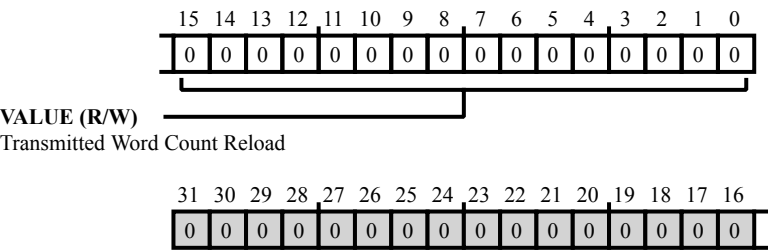


Figure 13-38: SPI_TWCR Register Diagram

Table 13-35: SPI_TWCR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	VALUE	Transmitted Word Count Reload. The <code>SPI_TWCR.VALUE</code> bits hold the transmit transfer word count reload value.

Transmit Control Register

The `SPI_TXCTL` register enables the SPI transmit channel, initiates transmit transfers, and configures `SPI_TFIFO` buffer watermark settings.

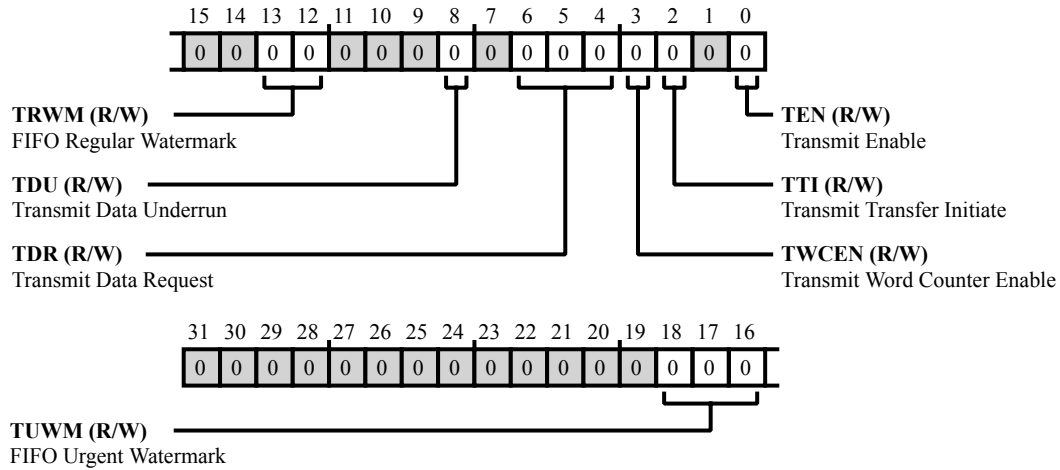


Figure 13-39: SPI_TXCTL Register Diagram

Table 13-36: SPI_TXCTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
18:16 (R/W)	TUWM	FIFO Urgent Watermark. The <code>SPI_TXCTL.TUWM</code> bits select the transmit FIFO (<code>SPI_TFIFO</code>) watermark level for urgent data bus requests. The SPI also uses this watermark level for generation of the <code>SPI_ILAT.TUWM</code> interrupt request. When an urgent <code>SPI_TFIFO</code> watermark is enabled with <code>SPI_TXCTL.TUWM</code>, the <code>SPI_TXCTL.TRWM</code> selection is used as the deassertion condition for any <code>SPI_ILAT.TUWM</code> interrupt requests that are latched.
		0 Disabled
		1 25% empty TFIFO
		2 50% empty TFIFO
		3 75% empty TFIFO
		4 Empty TFIFO

Table 13-36: SPI_TXCTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
13:12 (R/W)	TRWM	FIFO Regular Watermark. The SPI_TXCTL.TRWM bits select the transmit FIFO (SPI_TFIFO) watermark level for regular data bus requests. When an urgent SPI_TFIFO watermark is enabled with SPI_TXCTL.TUWM, the SPI_TXCTL.TRWM selection is used as the deassertion condition for any SPI_ILAT.TUWM interrupt requests that are latched.
		0 Full TFIFO
		1 TFIFO less than 25% empty
		2 TFIFO less than 50% empty
		3 TFIFO less than 75% empty
8 (R/W)	TDU	Transmit Data Underrun. The SPI_TXCTL.TDU bit selects handling for transmit data requests when the transmit buffer (SPI_TFIFO) is empty. If enabled and SPI_TFIFO is empty, the SPI transmits zero as data. If disabled and SPI_TFIFO is empty, the SPI transmits the last word in the buffer as data.
		0 Send last word when SPI_TFIFO is empty
		1 Send zeros when SPI_TFIFO is empty
6:4 (R/W)	TDR	Transmit Data Request. The SPI_TXCTL.TDR bits select transmit FIFO (SPI_TFIFO) watermark conditions that direct the SPI to generate a transmit status interrupt request.
		0 Disabled
		1 Not full TFIFO
		2 25% empty TFIFO
		3 50% empty TFIFO
		4 75% empty TFIFO
		5 Empty TFIFO
3 (R/W)	TWCEN	Transmit Word Counter Enable. The SPI_TXCTL.TWCEN bit enables the decrement of the transmit word count (SPI_TWC) register when the count is not zero and SPI_TXCTL.TTI is enabled. Enabling SPI_TXCTL.TWCEN prevents transmit underrun errors from occurring. The SPI_TXCTL.TWCEN bit is valid only when the SPI is a master.
		0 Disable
		1 Enable

Table 13-36: SPI_TXCTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
2 (R/W)	TTI	Transmit Transfer Initiate. The SPI_TXCTL.TTI bit enables initiation of transmit transfers if the transmit FIFO (SPI_TFIFO) is not empty. The bit also enables this initiation if SPI_TWC is not zero when SPI_TXCTL.TWCEN is enabled. Enabling SPI_TXCTL.TTI prevents transmit underrun errors from occurring. The SPI_TXCTL.TTI bit is valid only when the SPI is a master.
		0 Disable
		1 Enable
0 (R/W)	TEN	Transmit Enable. The SPI_TXCTL.TEN bit enables SPI transmit channel operation.
		0 Disable
		1 Enable

14 Extended Serial Peripheral Interface (xSPI)

The xSPI flash controller provides access to serial flash devices that support JESD216 and JESD251 standards. The controller accesses flash devices in modes that permit efficient communication with minimal software overhead. The supported HyperBus™ protocol enables seamless communication with HyperFlash™/HyperRAM™ devices. The standard SPI protocol is supported along with high performance dual, quad, and octal SPI modes, where data can be transferred on up to 8 data pins. In addition to the normal mode of operation in single transfer (STR) mode, the xSPI controller supports a dual transfer rate (DTR) mode. In DTR mode, data, address, and commands are transferred on both edges of the serial clock. With direct access mode support, any read/write to the xSPI memory-mapped space triggers a read/write to flash memory.

An integrated PHY handles the low-level timings of the data, address and control signals between the device and controller.

The xSPI controller also provides support for RWDS/DQS when sampling data. This feature improves read data capture, which, along with the PHY, enables high-speed data transfers.

xSPI Features

The xSPI controller supports the following features:

- Memory-mapped direct mode operation for performing flash data transfers
- Auto Command Mode (ACMD) to perform various memory operations with minimum software overhead
- An ACMD DMA engine that triggers transactions using descriptors fetched from the main memory. It automatically moves data between the device memory and system host memory.
- STIG mode operation to issue flash commands
- Automatic flash status polling support for flash program
- Execute in Place (XIP) support
- Device Discovery (DD) to identify the memory device
- Support for single, dual, quad, or octal modes of operation
- Support for DDR mode and DTR protocol (including octal DDR protocol)

- Data integrity features (CRC or ECC) for flash memories
- Programmable baud rate, clock phase, and polarity
- Programmable interrupt generation
- Programmable chip select control timing
- Programmable dummy cycles for read and write operations
- Support for up to 116.6 MHz in non-DQS mode
- Support for up to 155.6 MHz in DQS mode

Supported Memory Devices

The xSPI controller supports any device that is compatible with the JESD216 specification standard. In this standard, these devices are defined as profile 1 or profile 2 type devices. Traditional serial NOR flash devices tend to be compatible with the profile 1 type as defined in that standard. Devices supporting HyperBus (traditional HyperFlash and most HyperRAM or pSRAM devices) tend to be compatible with the profile 2 type. Sometimes, a pSRAM or HyperRAM device may support a profile 1 style interface; the device manufacturer selects the interface.

The supported devices include:

- *Traditional serial SPI (single/dual/quad/octal)* - Memory devices defined as profile 1 in the JEDEC JESD216 specification. The device may have been manufactured before the specification was released. Any device that specifies compliance with an xSPI profile 1, where profile 1 is described in the xSPI JEDEC JESD216 specification.
- *Traditional HyperFlash* - Memory devices defined as profile 2 in the JEDEC JESD216 specification. The device may have been manufactured before the specification was released. Any devices that specify compliance with an xSPI profile 2, where profile 2 is described in the xSPI JEDEC JESD216 specification.
- *Serial RAM and HyperRAM* - Serial SRAM devices (for example, OctaRAM[™]/OctalRAM, HyperRAM, pSRAM) devices. These volatile RAM devices tend to be interfaced similar to profile 1 or profile 2. However, they must be occasionally self-refreshed to maintain state and may not require some elements of the sequences used to read or program traditional flash devices.
- *Serial NAND* - also referred to as SPI NAND

xSPI Functional Description

This section provides information on the function of the xSPI module.

ADSP-21560_21561_21564_21568 XSPI Register List

The XSPI is an expanded SPI controller. It contains the following registers.

Table 14-1: ADSP-21560_21561_21564_21568 XSPI Register List

Name	Description
XSPI_BOOT_STAT	Boot Status Register
XSPI_CMD0	Command Register 0
XSPI_CMD1	Command Register 1
XSPI_CMD2	Command Register 2
XSPI_CMD3	Command Register 3
XSPI_CMD4	Command Register 4
XSPI_CMD5	Command Register 5
XSPI_CMD_STAT	Command Status Register
XSPI_CMD_STAT_PTR	Command Status Pointer Register
XSPI_DAC_CFG	DAC Configuration Register
XSPI_DAC_REMAPADDR0	Address Remapping Register 0
XSPI_DAC_REMAPADDR1	Address Remapping Register 1
XSPI_DISCOVERY_GCTL	Device Discovery Control Register
XSPI_DMA_CTL	DMA Interface Control Register
XSPI_DMA_ERRADDR_HI	DMA Error Address High Register
XSPI_DMA_ERRADDR_LO	DMA Error Address Low Register
XSPI_ERS_SEQ_CFG0	Erase Sequence Configuration Register 0
XSPI_ERS_SEQ_CFG1	Erase Sequence Configuration Register 1
XSPI_ERS_SEQ_CFG2	Erase Sequence Configuration Register 2
XSPI_FEATURES	Controller Features Register
XSPI_GSTAT	General Controller Status Register
XSPI_INT_EN	Interrupt Enable Register
XSPI_ISTAT	Interrupt Status Register
XSPI_LONGPOL_GCTL	Long Polling Count Register
XSPI_MINICTL_DEV_ACTIVE_MAX	Device Active Maximum Clock Cycle Register
XSPI_MINICTL_CLKMODE	Clock Mode Control Register
XSPI_MINICTL_DEV_DLY	Device Delay Register
XSPI_MINICTL_HF_OFFSET	HyperFlash Offset Register
XSPI_MINICTL_JEDEC_RST_TR	JEDEC Reset Delay Register
XSPI_MINICTL_RST_PIN_CTL	Hardware Reset Control Register
XSPI_MINICTL_RST_RECOV	Reset Recovery Delay Register

Table 14-1: ADSP-21560_21561_21564_21568 XSPI Register List (Continued)

Name	Description
XSPI_MINICTL_WP_CTL	Write Protect Register
XSPI_PHY_DLLOB0	PHY DLL Observable Points 0 Register
XSPI_PHY_DLLOB1	PHY DLL Observable Points 1 Register
XSPI_PHY_DLL_CTL	PHY DLL Control Register
XSPI_PHY_DLL_MSTR_CTL	Master DLL Control Register
XSPI_PHY_DLL_SLAVE_CTL	Slave DLL Control Register
XSPI_PHY_DLL_UPDT_CNT	PHY DLL Resynchronization Register
XSPI_PHY_DQS_TR	PHY DQS Timing Register
XSPI_PHY_DQ_TR	PHY DQ Timing Register
XSPI_PHY_FEATURES	PHY Features Register
XSPI_PHY_GATE_LPBK_CTL	PHY Gate Loopback Control Register
XSPI_PHY_GCTL	PHY Global Control Register
XSPI_PHY_GPIO_CTL0	PHY GPIO Control Register 0
XSPI_PHY_GPIO_CTL1	PHY GPIO Control Register 1
XSPI_PHY_GPIO_STAT0	PHY GPIO Status Register 0
XSPI_PHY_GPIO_STAT1	PHY GPIO Status Register 1
XSPI_PHY_GTSEL	PHY Global Termination Control Register
XSPI_PHY_IE_TR	PHY DQS Input Enable Timing Register
XSPI_PHY_OB0	PHY Observable Points Register
XSPI_PHY_REVID	PHY Revision ID Register
XSPI_PHY_STATIC_TGL	PHY Static Aging Register
XSPI_PHY_WR_DESKEW_PAD_CTL0	PHY Deskew Write Register
XSPI_PROG_SEQ_CFG0	Program Sequence Configuration Register 0
XSPI_PROG_SEQ_CFG1	Program Sequence Configuration Register 1
XSPI_PROG_SEQ_CFG2	Program Sequence Configuration Register 2
XSPI_READ_SEQ_CFG0	Read Sequence Configuration Register 0
XSPI_READ_SEQ_CFG1	Read Sequence Configuration Register 1
XSPI_READ_SEQ_CFG2	Read Sequence Configuration Register 2
XSPI_REVID	Revision ID Register
XSPI_RST_SEQ_CFG0	Reset Sequence Configuration Register 0
XSPI_RST_SEQ_CFG1	Reset Sequence Configuration Register 1

Table 14-1: ADSP-21560_21561_21564_21568 XSPI Register List (Continued)

Name	Description
XSPI_SDMA_ADDR0	Host DMA Buffer Address Register 0
XSPI_SDMA_ADDR1	Host DMA Buffer Address Register 1
XSPI_SDMA_SIZ	Host DMA Block Size Register
XSPI_SDMA_TRD_STAT	Host DMA Thread Status Register
XSPI_SEQ_GCTL0	Sequence Configuration Register 0
XSPI_SEQ_GCTL1	Sequence Configuration Register 1
XSPI_SHORTPOL_GCTL	Short Polling Count Register
XSPI_STAT_SEQ_CFG0	Status Checking Sequence Configuration Register 0
XSPI_STAT_SEQ_CFG1	Status Checking Sequence Configuration Register 1
XSPI_STAT_SEQ_CFG10	Status Checking Sequence Configuration Register 10
XSPI_STAT_SEQ_CFG2	Status Checking Sequence Configuration Register 2
XSPI_STAT_SEQ_CFG3	Status Checking Sequence Configuration Register 3
XSPI_STAT_SEQ_CFG4	Status Checking Sequence Configuration Register 4
XSPI_STAT_SEQ_CFG5	Status Checking Sequence Configuration Register 5
XSPI_STAT_SEQ_CFG7	Status Checking Sequence Configuration Register 7
XSPI_STAT_SEQ_CFG8	Status Checking Sequence Configuration Register 8
XSPI_STAT_SEQ_CFG9	Status Checking Sequence Configuration Register 9
XSPI_TRD_COMP_ISTAT	Auto Command Engine Interrupt Status Thread Register
XSPI_TRD_ERR_INT_EN	Thread Error Interrupt Enable Register
XSPI_TRD_ERR_ISTAT	Thread Error Interrupt Status Register
XSPI_TRD_STAT	Auto Command Engine Thread Status Register
XSPI_WE_SEQ_CFG0	WEL Sequence Configuration Register
XSPI_WORKMODE_CTL	Device Control Register
XSPI_XIP_GCTL	XIP Configuration Register

xSPI Block Diagram

The xSPI module is comprised of:

- Main command sequencer
- PHY block
- Internal transmit/receive FIFOs
- DAC/STIG controller

- Register interface

The *xSPI Controller Block Diagram* shows the xSPI controller functional blocks.

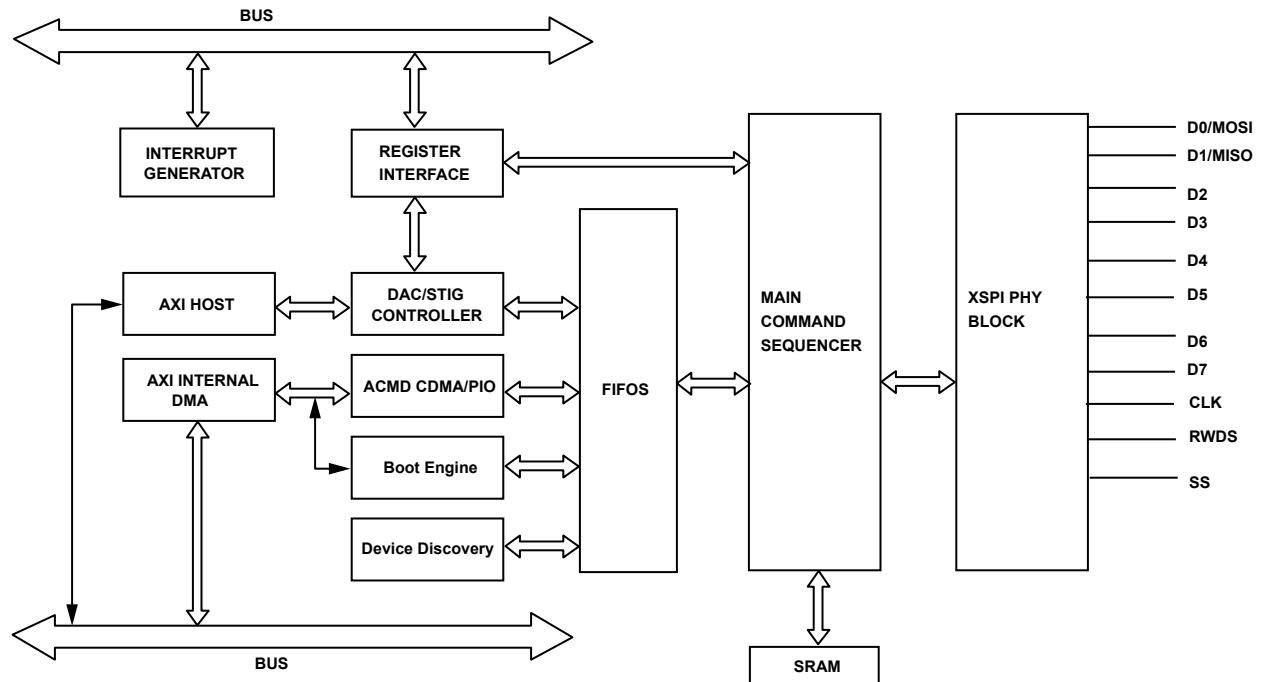


Figure 14-1: xSPI Controller Block Diagram

Architectural Concepts

There are two clock sources for the xSPI controller: CLK010 and SCLK0_0. CLK010 (from the CDU) is the main system clock used to transfer data over the bus between the controller (core/DMA) and the xSPI controller. This clock is also driven on the external XSPI_CLK pin to transfer the data to/from external devices. The user can choose among SCLK0_0, OCLK_0, and SCLK1_1 signals for this transaction. Similarly, SCLK0_0 is used to access the xSPI controller registers to perform basic configuration of the controller and interrupt handling.

The xSPI controller supports independent modes/controllers for different memory devices:

- Direct mode
- STIG mode
- Auto Command Mode (ACMD, CDMA or PIO)

The bus interface provides access to controller status and configuration registers. The AXI host interface is used in direct mode, STIG mode (when necessary), and optionally in ACMD mode.

The main command sequencer holds the main sequencing logic. It is responsible for building the command structures and meeting the required device timings. It also interfaces with the PHY module and regulates data flow between the physical layer and the system interface. PHY is responsible for handling the low-level timings of the data, address, and control signals between the device and controller.

Operating Modes

There are three main usage modes for the xSPI controller: direct mode, STIG mode, and ACMD mode. These modes can perform standard read/write transactions to a device memory. However, only one mode can be active at a time.

One active work mode is selected at a time to communicate with an attached device. ACMD or direct modes are the primary operational modes. STIG mode is used as an accompanying mode to send commands or a sequences of commands to the device that may not be directly supported in ACMD or direct mode. STIG mode is a highly flexible mode that can also be used to write or read data to the device and perform other operations. However, STIG mode relies more on software to construct the necessary command structures.

ACMD mode requires that software either manage a descriptor table with high-level sequences defined within, or manage some command registers within the controller to issue these same high-level sequences. The controller can automatically translate high-level sequences (reset, erase, erase_all, program, read) into subsequences that are tied to the device type.

Direct mode, as the name suggests, directly translates read and write operations detected on the AXI host slave interface to equivalent low-level read and write subsequences at the device. As such, direct mode can only support read or program operations. It relies on STIG mode to perform other operations such as erase. It can also be used to directly boot from the device, when required.

Direct Mode

In direct mode, reads and writes that are issued by the processor (either by the core or MDMA controller) enter the controller using the AXI slave interface. The read/write operations are directly translated to read/write at the memory device. The shaded portion of the *Direct Mode Block Diagram* shows the flow. Direct mode allows the processor to execute code directly, or have direct access from/to the attached memory device. This flow is sometimes referred to as eXecute-In-Place or XIP. The controller support devices operating in XIP mode (in all work modes, not just direct). However, there are steps the host must take to trigger entry to and exit from XIP mode.

See the topic [Operating in Direct Mode](#) for details on operating in direct mode, programming the mode, and usage restrictions.

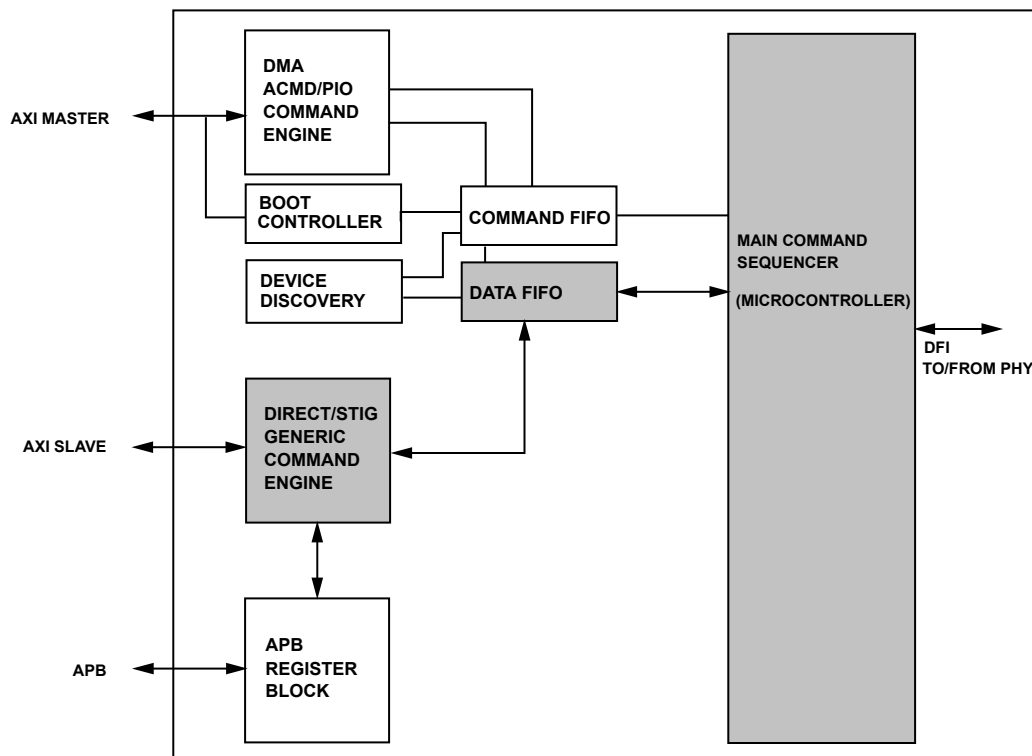


Figure 14-2: Direct Mode Block Diagram

STIG Mode

Software Triggered Instruction Generator (STIG) mode is configured and triggered using the programming interface. This mode is very flexible and configurable. In addition to performing standard reads and writes, it can also be used to access memory device registers and/or trigger functions in the memory device such as erase, read JEDEC ID, etc. STIG mode uses data paths that are similar to direct mode. However, the commands are configured and triggered through access to registers using the APB interface. Data can optionally be passed from the AXI slave interface, or through other registers in the APB register block. The shaded portion of the *STIG Mode Block Diagram* shows the operational flow.

See the topic [Operating in STIG Mode](#) for details on operating in STIG mode, programming the mode, and usage restrictions.

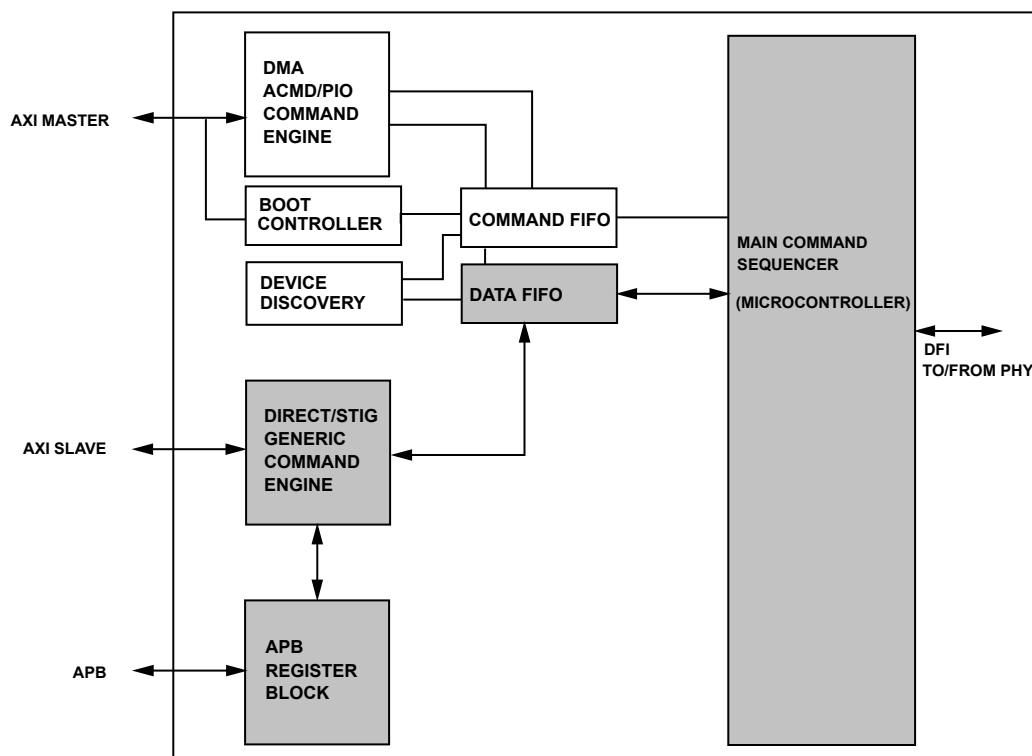


Figure 14-3: STIG Mode Block Diagram

ACMD Mode

In Auto Command Mode (ACMD) mode, an alternative to direct mode, the user configures the system DMA engine using the AXI slave interface. The shaded portion of the *ACMD Mode Block Diagram* shows the operational flow.

In this mode, the controller signals to the processor using an interrupt or status register when it requires the host to send write data on the slave AXI write ports, or when it needs the processor to issue an AXI read to fetch read data.

See the topic [Operating in ACMD Mode](#) for details on operating in ACMD mode, programming the mode, and usage restrictions.

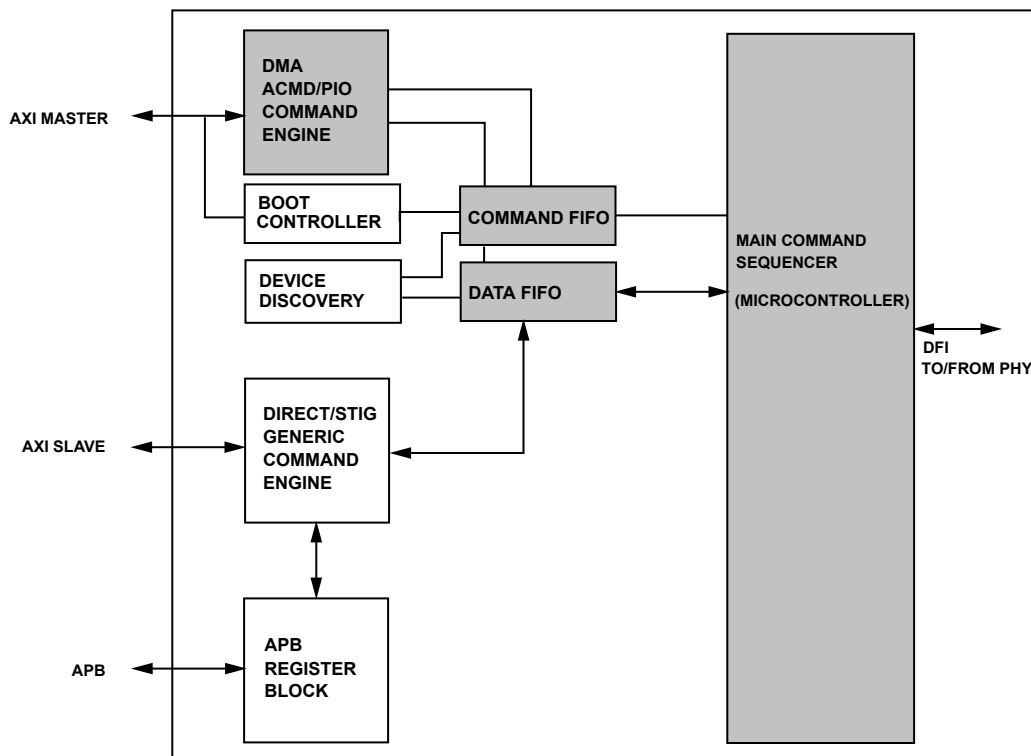


Figure 14-4: ACMD Block Diagram

Integrated Soft PHY

The xSPI PHY controller is responsible for handling the low-level timings of the data, address, and control signals between the device and controller. It provides additional logic, which operates as a bridge between formulated xSPI transactions and the timing-aware off-chip xSPI transfer to the external device. PHY has its own programmable registers. Accessing these registers occurs through the standard controller APB address space. Before the processor can communicate with the memory device, the PHY must be initialized and locked.

NOTE: In ADSP-2156x processors, the internal DMA of the xSPI controller is not supported.

See the section [xSPI PHY Controller](#) for details on how to use PHY, programming the PHY, and usage restrictions.

XIP Mode

The eXecute-In-Place (XIP) mode is also referred to as continuous mode. When the controller is in XIP mode, the processor executes code directly from the attached memory device or from where the processor has direct access to the attached memory device. The controller supports this function using the direct work mode. If executing code directly is needed, enable direct work mode.

The device uses XIP mode to improve read performance by removing the need for the controller to send the read opcode for any read operation. If the read opcode is typically sent over eight device clock cycles (SDR, serial transmission of the opcode), the read operation latency is reduced by eight cycles. This difference can significantly improve performance when executing code directly from the device. On the downside, when XIP mode is entered, the device can only perform read operations. If a different operation is required, XIP mode must be exited. Although the

controller supports XIP mode, there are steps the host must take to trigger entry to and exit from XIP mode. These steps depend on the selected work mode. See the section [Entering and Exiting XIP Mode](#).

Configuring the Controller

To use the controller in direct, ACMD, or STIG mode, there are global configuration registers that must be configured. These are defined in the common sequence registers.

Use the following steps to configure the controller.

1. Configure the `XSPI_SEQ_GCTL0` register to select the correct xSPI profile as per the memory device, to enable or disable the data swap feature, to select single or dual bytes per address, device page program size, device page read size, and other desired features.
2. Configure the `XSPI_SEQ_GCTL1` register. This register is used only for SPI NAND profile to configure plane count, pages per block, extended page size, etc.
3. Configure the erase/read/program sequence. Configure the read or program sequence that the controller must send to the memory device. This sequence depends on the attached memory device. Refer to the device data sheet for details.

Some devices require a write enable sub sequence to be issued prior to a program operation (for example, to configure the write enable latch or WEL in the device). By default, it is assumed to be memory device requirement. However, the sequence can be disabled using the `XSPI_WE_SEQ_CFG0.P1_EN` bit. By default, the command extension of the write enable sequence is disabled. It is sent in a serial SDR format with an opcode of 0x6. If any of the parameters need changes, configure the `XSPI_WE_SEQ_CFG0` register accordingly.

Some devices require the device to be polled for correctness and readiness after issuing an operation. Due to the differences between how device types interpret and respond to status requests, and different types of status requests, the controller requires several programmable registers that must be tuned to the device requirements. For direct mode, the controller can optionally check the device fail or busy status following a program or erase operation (or additionally for soft_reset/read for SPI NAND devices). For SPI NAND devices, the controller can also check the ECC status following a read page command. All of these requirements are configured using the `XSPI_STAT_SEQ_CFGn` registers (`XSPI_STAT_SEQ_CFG0` to `XSPI_STAT_SEQ_CFG10`).

NOTE: When using the device discovery engine, the registers are updated automatically after power on reset. Refer to section [Device Discovery](#) for details.

The *Global Configurations Registers* table shows the registers used to configure the controller for various operations and types of flash devices. These registers must be initialized and configured when operating in direct or ACMD mode.

Table 14-2: Global Configurations Registers

	Profile 1 Devices	Profile 2 – HyperFlash Devices	Profile 2 – HyperRAM Devices	SPI NAND Devices
Program	XSPI_PROG_SEQ_CFG0 XSPI_PROG_SEQ_CFG1	XSPI_PROG_SEQ_CFG2	XSPI_PROG_SEQ_CFG2	XSPI_PROG_SEQ_CFG0 XSPI_PROG_SEQ_CFG1
Read	XSPI_READ_SEQ_CFG0 XSPI_READ_SEQ_CFG1	XSPI_READ_SEQ_CFG2	XSPI_READ_SEQ_CFG2	XSPI_READ_SEQ_CFG0 XSPI_READ_SEQ_CFG1
Write	XSPI_WE_SEQ_CFG0	N/A	N/A	XSPI_WE_SEQ_CFG0
Device Fail Status Check Device Busy Status Check	XSPI_STAT_SEQ_CFG0 XSPI_STAT_SEQ_CFG1 XSPI_STAT_SEQ_CFG2 XSPI_STAT_SEQ_CFG3 XSPI_STAT_SEQ_CFG5 XSPI_STAT_SEQ_CFG7 XSPI_STAT_SEQ_CFG8 XSPI_STAT_SEQ_CFG9	XSPI_STAT_SEQ_CFG4 XSPI_STAT_SEQ_CFG5 XSPI_STAT_SEQ_CFG7 XSPI_STAT_SEQ_CFG8 XSPI_STAT_SEQ_CFG9	N/A	XSPI_STAT_SEQ_CFG0 XSPI_STAT_SEQ_CFG1 XSPI_STAT_SEQ_CFG2 XSPI_STAT_SEQ_CFG3 XSPI_STAT_SEQ_CFG5 XSPI_STAT_SEQ_CFG7 XSPI_STAT_SEQ_CFG8 XSPI_STAT_SEQ_CFG9
ECC Status Check	N/A	N/A	N/A	XSPI_STAT_SEQ_CFG10

Operating in Direct Mode

In direct mode, reads and writes that are issued by the processor enter the controller through the AXI slave interface. The read/write operations are directly translated to reads and writes at the memory device. Along with standard code execution, this mode can also be used immediately after power on reset to boot data from the attached memory device.

NOTE: This mode is different from the dedicated boot feature. The boot engine is specifically used to transfer boot code from the memory device to an area of host memory following power on reset (from which the host can then boot). There is a dedicated xSPI boot mode for the processor to support this feature. See the xSPI Controller Boot Mode topic in the Booting chapter for details.

Direct work mode is enabled by default (`XSPI_WORKMODE_CTL.WORKMODE`). The `XSPI_WORKMODE_CTL` register can only be changed when the controller is in an idle state. This state can be determined by polling the `CRTL_BUSY` pin of the xSPI flash controller or by polling the `XSPI_GSTAT_CTL_BUSY` bit. When direct mode is enabled, access to the `XSPI_CMDn` registers (which are only required for other work modes) are ignored and should be avoided.

Read and program/write transactions to the memory device occur only as a direct consequence of the processor applying read or write AXI transactions to the AXI slave interface through memory-mapped xSPI addresses.

NOTE: In direct mode, the configured page size for write or read transfers must be configured to a value wider than the selected word size of the AXI transaction ($sA \times \text{SIZE}$). Also, the xSPI page boundaries must be aligned to the AXI transaction word size ($sA \times \text{SIZE}$).

In addition to configuring the global registers described in [Configuring the Controller](#), configure the direct mode specific registers such as `XSPI_DAC_CFG` and `XSPI_DAC_REMAPADDR0/XSPI_DAC_REMAPADDR1`:

1. Write to the `XSPI_DAC_CFG` register to select the appropriate bank number in the attached memory device, to enable or disable AXI slave address remap, XIP mode, and other desired features.
2. Configure the remap address offset in the `XSPI_DAC_REMAPADDR0` register.

Write Limitations

By default, the xSPI controller does not support the following functions:

- Byte writes (for example, an AXI write with an `AWSIZE` value of 0) to a device that is 16-bit addressed, or to an octal DDR configured device
- Issuing a write burst to an odd address to a device that is 16-bit addressed, or to an octal DDR configured device
- Using AXI write strobes to mask bytes on the write bus

These write operations can optionally be supported by setting the bit `XSPI_DAC_CFG.RWDS_CAP_EN` if the connected device supports byte masking using the `DQS` signal. `DQS` is traditionally used by a memory device to transfer a strobe with read data to help read data sampling. However, some devices also use the same signal as a byte mask for writes (in which case the limitations can be avoided).

AXI Wrapping Bursts

The following wrapping bursts restrictions are taken from the AXI standard:

- The start address must be aligned to the size of each transfer
- The length of the burst must be 2, 4, 8, or 16 transfers

Wrapping bursts are used for cache line accesses issued after a cache miss, and are, therefore, more relevant to the direct mode of operation than `ACMD/STIG` work modes. Internally, the xSPI controller handles AXI wrapping bursts by splitting them into separate incrementing bursts, automatically. Wrapping bursts on the AXI interface are supported even when the attached memory device does not support wrapping.

There is no restriction on using wrapping read bursts. However, there is one known scenario where wrapping write bursts received by the controller operating in direct mode through the AXI slave interface are not be supported. This scenario is described below:

1. The wrapping write burst restriction applies only to attached memory devices that require time to execute any program (write) operation. Flash devices, typically, operate in this manner.
2. While any program (write) operation is ongoing within the memory device, the controller has a function (which, by default, is enabled) to automatically check the status of the memory device. It does not allow any other read or write operation from the controller to proceed until the memory device has indicated that it is ready for a new operation. This function is programmable and can be disabled.
3. If the user has chosen to disable the function above (through the `XSPI_STAT_SEQ_CFG1` register), the splitting of the AXI wrapping write transaction into separate write incrementing transactions results in the second incrementing write being discarded by the memory device.

Therefore, wrapping write bursts *cannot* be supported when the attached memory device requires time to process write operations and the controller function to check ready/busy status of the device has been disabled (using the `XSPI_STAT_SEQ_CFG1` register). Otherwise, wrapping write bursts in direct mode are fully supported.

Entering and Exiting XIP Mode

The `XSPI_DAC_CFG.XIP_EN_MB_VAL` and `XSPI_DAC_CFG.XIP_DIS_MB_VAL` bit fields control how and when XIP mode in the device is entered or exited when using direct mode. By default, XIP mode is disabled. The *XIP Mode Register Configurations* table shows the registers used to configure XIP mode operations.

Most memory device use either of the one or both of the following steps to enter the XIP mode.

- Configure a specific bit in the memory configuration/status register (this step is not mandatory for all flash devices, but it which can be achieved easily using STIG mode).
- Send a read command with the correct value applied in the mode byte; the info is located immediately after the address phase of the read transaction

Set the `XSPI_DAC_CFG.XIP_EN_MB_VAL` bit to enter XIP mode while operating in direct mode. Follow up with a normal direct mode read transaction. As the read transaction is executed and the read sequence is sent to the device, the xSPI controller sends the value encoded in the `XSPI_XIP_GCTL.XIP_EN_MB_VALUE` register bit, immediately following the address phase of the transaction. Once the read transaction completes, the `XSPI_XIP_GCTL.XIP_EN` bit is set (=1) to indicate that both the controller and the memory device are now configured to work in XIP mode.

IMPORTANT: A direct read transfer to enable or disable XIP mode in the controller and memory device must not have more than 64B of data and must not cause crossing between pages.

To enable XIP mode during power on reset (assuming that the memory device is already configured to work in XIP mode in a non-volatile manner from reset), the `XSPI_XIP_GCTL.XIP_EN` bit should be set (=1). Then, each read transaction is performed in XIP mode. For example, the controller issues the read operation without sending any opcode and additionally setting the enable XIP mode bit following the address phase (when

XSPI_DAC_CFG.XIP_EN_MB_VAL = 1). The value of the XSPI_DAC_CFG.XIP_EN_MB_VAL field is sent following the address phase of the transaction.

To exit XIP mode in both the xSPI controller and the memory device, a new direct read transaction must be issued to the device with the disable XIP mode bit following address phase. Set (=1) the XSPI_DAC_CFG.XIP_DIS_MB_VAL bit to disable XIP mode. Issue a new direct read. As the read is executed and the read sequence is sent to the device, the controller sends the value encoded in the XSPI_XIP_GCTL.XIP_DIS_MB_VALUE field immediately following the address phase of the transaction. Once completed, the XSPI_XIP_GCTL.XIP_EN bit is cleared (=0) to indicate that the controller and memory device have exited XIP mode and are configured to work in normal mode.

Table 14-3: XIP Mode Register Configurations

XSPI_XIP_GCTL.XIP_EN	XSPI_DAC_CFG.XIP_EN_MB_VAL	XSPI_DAC_CFG.XIP_DIS_MB_VAL	XSPI_READ_SEQ_CFG1.P1_MB_EN	Description
0	0	0	0	Normal read mode without mode byte. (Number of dummy cycles = XSPI_READ_SEQ_CFG1.P1_MB_DMY_CNT)
0	0	1	0	Normal read mode with the additional disable mode byte following the address phase. (Number of dummy cycles = XSPI_READ_SEQ_CFG1.P1_MB_DMY_CNT)
0	0	0	1	Normal read mode with the additional disable mode byte following the address phase (number of dummy cycles = XSPI_READ_SEQ_CFG1.P1_MB_DMY_CNT)

Table 14-3: XIP Mode Register Configurations (Continued)

XSPI_XIP_GCTL. XIP_EN	XSPI_DAC_CFG.XIP _EN_MB_VAL	XSPI_DAC_CFG.XIP _DIS_MB_VAL	XSPI_READ_SEQ_ CFG1.P1_MB_EN	Description
0	1	N/A	N/A	Enables XIP mode in the controller and memory device on the next read transfer – the enable mode byte is sent after the opcode and address phases. Then, XSPI_XIP_GCTL.XIP_EN =1. (Number of dummy cycles = XSPI_READ_SEQ_CFG1.P1_MB_DMY_CNT)
1	0	0	N/A	Controller reports DIR_CMD_ERR error. Mode byte must be sent when controller is configured to work in XIP mode.
1	0	1	N/A	Disables XIP mode in the controller and memory device on the next read transfer - the disable mode byte is sent after the address phase. Then, XSPI_XIP_GCTL.XIP_EN =0. (Number of dummy cycles = XSPI_READ_SEQ_CFG1.P1_MB_DMY_CNT)
1	1	N/A	N/A	Controller works in XIP mode - the enable mode byte is sent after the address phase. (Number of dummy cycles = XSPI_READ_SEQ_CFG1.P1_MB_DMY_CNT)

Table 14-3: XIP Mode Register Configurations (Continued)

XSPI_XIP_GCTL. XIP_EN	XSPI_DAC_CFG.XIP _EN_MB_VAL	XSPI_DAC_CFG.XIP _DIS_MB_VAL	XSPI_READ_SEQ_ CFG1.P1_MB_EN	Description
0	0	1	1	Normal read mode with the additional disable mode byte following the address phase. (Number of dummy cycles = XSPI_READ_SEQ_CFG1.P1_MB_DMY_CNT)

Operating in ACMD Mode

The ACMD work mode relieves the host from issuing read and write commands directly. Instead, the host uses a dedicated engine to translate high-level commands into a series of low-level commands on the xSPI interface. It selects the optimal command sequence to send and to follow all transfer restrictions defined for the targeted memory device.

The auto command engine receives high-level commands either through a DMA descriptor format or directly through programmable registers. It is capable of operating multiple parallel threads that are configured using the controller registers and descriptors held in the main memory. It can operate in one of two modes – CDMA mode or PIO mode. Data transfers can be performed using host DMA mode.

CDMA mode uses an embedded DMA to trigger various memory transactions using descriptors it fetches from main memory, automatically moving data between device memory and the host memory. In PIO work mode, dedicated controller registers are used in place of the descriptors to trigger the operation in the selected execution thread. The xSPI controller signals the host using an interrupt or status register when it requires the host to send write data on the host DMA write ports, or when it needs the host to issue a read command to fetch read data.

The auto command engine operates in the system (mACLK) clock domain. This mode uses system SRAM to buffer descriptor content locally; it is synchronous to mACLK.

NOTE: mACLK — This clock is SCLK0. It drives the front end of the controller. The host DMA interface is synchronous to this clock.

The auto command engine can handle the following high-level sequences:

- Reset — sequence which allows the reset command to be sent to the selected device
- Erase — sequence to erase part of device or a whole device at once
- Program — sequence to transfer a specified amount of data from the host to the xSPI device
- Read — sequence to transfer a specified amount of data from the xSPI device to the Host

When operating in CDMA mode, the high-level sequence selection is made using a command descriptor. The DMA uses a standard structure where the head of the descriptor list is stored in APB register space. Each queued descriptor is chained together (one descriptor points to the next descriptor). For extra flexibility, up to eight descriptor chains can be managed simultaneously by the controller, and this is referred to as execution threads. Refer to section, [Operating in ACMD Mode](#) on for further details.

In PIO mode, the sequence is defined directly in specific command registers. Refer to section, [Using the Auto Command in PIO Mode](#) for further details.

NOTE: The high-level sequence the controller sends often must be broken up into multiple sub-sequences applied at the low-level xSPI interface. For example, it is common that prior to a program operation, a write enable sequence must be issued to instruct the memory device that a write operation is imminent, and, therefore, to configure the write enable latch of the device. This, obviously, depends on the attached memory type and controller configuration. The auto command engine selects the most appropriate low-level interface.

Using the Auto Command in CDMA Mode

To enable the auto command engine in CDMA mode, configure `XSPI_WORKMODE_CTL.WORKMODE = 3`. This operation can be done when the controller is in an idle state (poll the `XSPI_GSTAT_CTL_BUSY` bit). If software tries to access the `XSPI_WORKMODE_CTL` register when the controller is in the busy state, the access is ignored. To trigger the auto command engine in CDMA work mode, the `XSPI_CMD0`, `XSPI_CMD2` and `XSPI_CMD3` registers must be configured to pass the descriptor chain start address and the thread number associated with the descriptor chain. The command format for these registers are shown in the following tables.

When the ACMD mode is enabled, each write to the `XSPI_CMD0` register starts execution on a new command. Therefore, the `XSPI_CMD0` register must be written last; a write access to this register triggers the execution of the programmed operation.

Table 14-4: `XSPI_CMD0` Format for ACMD mode

<code>XSPI_CMD0</code> (offset 0x0)			
31:30	29:27	26:24	23:0
CDMA/PIO = 00	Reserved	TRD_NUM (thread number)	Reserved

Table 14-5: `XSPI_CMD2` Format for ACMD mode

<code>XSPI_CMD2</code> (offset 0x2) [31:0]
Lower 32 bits of the descriptor head address

Table 14-6: `XSPI_CMD3` Format for ACMD mode

<code>XSPI_CMD3</code> (offset 0x3) [31:0]
Upper 32 bits of the descriptor head address

The host must write zero to the CDMA/PIO field of the `XSPI_CMD0` register and provide the address of the head of the linked descriptor list using the `XSPI_CMD3` and `XSPI_CMD2` registers.

The controller issues a bus transaction to fetch the complete descriptor from main memory. The *Command Descriptor Structure* and *Command Descriptor Layout* tables present the details of the descriptor format.

Table 14-7: Command Descriptor Structure

No	63:48	47:32	31:16	15:0
0	Next pointer			
1	System memory pointer			
2	xSPI pointer			
3	Reserved			
4	Command counter		Command flags	Command type
5	Reserved		Status	

Table 14-8: Command Descriptor Layout

Field	Description
Next pointer	Next descriptor address. Address needs to follow system bus restrictions.
System memory pointer	System memory address required for data DMA commands. For read operations, this field is the address where the start of the read data is placed. For write operations, this field is the location of the start of the write data.
xSPI pointer	The field indicates the xSPI address for sequences that require addressing of the xSPI address space.
Command type	This field identifies the operation that the controller must perform. The encoding is: • 0x1000 – Erase the number of sequential sectors specified in the command counter field • 0x1001 – Full chip erase • 0x1100 – Device reset • 0x1101 – Device JEDEC reset • 0x2100 – Command to program the number of data bytes specified in the command counter field • 0x2200 – Command to read the number of data bytes specified in the command counter field • Other values are reserved
Command flags	This field contains different control flags for the operation of the command (See the <i>Valid Command Flag</i> table).

Table 14-8: Command Descriptor Layout (Continued)

Field	Description
Command counter	For read/program operations, the field indicates the number of data bytes to transfer minus one. For example, to transfer 1024 bytes, the value encoded is 1023. For an erase operation, this field represents the number of sectors to erase. For other operations, this field is ignored.
Status	The controller updates this field with the command status once the command operation is complete, and writes the status back to the descriptor. Refer to the Table 14-10 Descriptor Status Field table for bit descriptions.

On receipt of a complete descriptor, the controller constructs a low-level command or a series of low-level commands.

NOTE: If XIP mode is enabled in the controller (that is, `XSPI_XIP_GCTL.XIP_EN = 1`, which is supported by some devices to improve read performance), the connected device can only be read. Therefore, any other command will fail. As such, invoking the CDMA descriptor with a non-read command type results in the controller ignoring it and raising a `DSC_ERROR` fail status.

The *Valid Command Flag* table describes the allowed values in the command flags field of the descriptor.

Table 14-9: Valid Command Flag

Bits	Name	Description
15:11	Reserved	Reserved
10	DMA_SEL	This bit must be zero. In the ADSP-2156x processors, the internal DMA of the xSPI controller is not supported. Refer to section, Selecting the DMA Interface in ACMD mode for details on how to configure this bit.
9	CONT	Set (=1) the bit to indicate that another valid descriptor after this has been queued and is ready for the controller to fetch and process. Clear (=0) the bit when this descriptor is last in the descriptor chain.
8	INT	Set (=1) the bit to instruct the controller to issue an interrupt after the completion of descriptor processing. The triggered interrupt is identified in the XSPI_TRD_COMP_ISTAT register. The bit of the interrupt status register that is set (=1) maps to the thread of the descriptor (as

Table 14-9: Valid Command Flag (Continued)

Bits	Name	Description
		indicated originally in the descriptor by the TRD[n] field).
7	MB_XIP_DIS	When XIP mode is enabled in the controller and the attached memory device, setting this bit (=1) indicates to the controller that it must send mode bits to disable and exit XIP mode. When set, the controller sends mode bits as specified in the XSPI_XIP_GCTL.XIP_DIS_MB_VALUE field. Additionally, the XSPI_XIP_GCTL.XIP_EN_MB_VALUE bit is cleared to disable XIP mode in the controller. This bit is valid only for read commands. If XSPI_READ_SEQ_CFG1.P1_MB_EN = 1 and XSPI_XIP_GCTL.XIP_EN = 0 the controller ignores the value of this field and it is treated as set (=1) (for example, mode bits are sent regardless of the value of this field).
6	MB_XIP_EN	This bit triggers entry to XIP mode. Setting this bit indicates to the controller that it must send mode bits to the device to enable XIP mode. The mode bits that are sent depend on the XSPI_XIP_GCTL.XIP_EN_MB_VALUE field. Additionally, the XSPI_XIP_GCTL.XIP_EN bit is set to enable XIP mode in the controller. This bit is valid only for read commands.

Table 14-9: Valid Command Flag (Continued)

Bits	Name	Description
5	xSPI_PTR_CONT	If this bit is set, the xSPI pointer field of current descriptor is ignored, and command engine uses the existing xSPI pointer value. Essentially, setting this bit allows the host to continue a data transfer from the address on the flash interface where a previously completed descriptor finished. This feature can only be triggered if a previous descriptor in the chain had this bit cleared and the xSPI pointer was initialized. It must also only be used for a command of the same type (for example, a series of read descriptors or series of write descriptors). For SPI NAND devices, a transfer is always continued from the beginning of the next page even if the current page was not filled completely (column address = 0, row address incremented).
4	SYS_PTR_CONT	If this bit is set (=1), the system memory pointer field of the current descriptor is ignored. The command engine uses the existing memory pointer value. This bit allows the data transfer to continue on the host interface where the previously completed descriptor finished. This feature can only be triggered if a previous descriptor in the chain had this bit cleared and the system memory pointer was initialized. It must also only be used for commands of the same type (for example, a series of read descriptors or series of write descriptors).
3	Reserved	Reserved
2:0	BANK	The field indicates the device number (bank/CS) that is selected for sequence execution.

NOTE: If the INT bit is not set in the current descriptor and the following descriptor chain is dropped from execution due to an error, the controller does not set the TRD[n] interrupt flag in the [XSPI_TRD_COMP_ISTAT](#) register. The error condition is, however, always reported by setting the TRD[n] bit in the [XSPI_TRD_ERR_ISTAT](#) register. The bit can be used to detect that the descriptor chain execution was interrupted due to an error. The n in the field names represent the thread number for which the error or complete condition occurred.

Opon completion of a command, the controller writes back the status into the descriptor status field. This field value is ignored during descriptor read operation. The status word is described in the *Descriptor Status Field* table.

Table 14-10: Descriptor Status Field

Bits	Name	Description
31:24	Reserved	Reserved
23:16	ECC_STAT	Value of the ECC status returned by the SPI NAND device. The value of this field applies to last read operation. It is updated only when fail checking is enabled.
15	COMPLETE	When set, the bit denotes that the controller has updated status information and the operation is complete. This bit is set even when the operation ended as a failure. This bit should be in a cleared state while descriptor is constructed.
14	FAIL	When set, the bit denotes that the operation failed to complete successfully.
13:6	Reserved	Reserved
5	ECC_CORR_ERROR	The bit indicates that an ECC correctable error was detected during page read. This bit is valid only for SPI NAND devices when <code>XSPI_STAT_SEQ_CFG10.ECC_FAIL_EN=1</code> .
4	DEVICE_ERROR	The bit indicates that a device error was detected during a read status operation for erase/program/read.
3	DQS_ERROR	This bit is set when an incorrect number of DQS pulses were detected during a direct mode read or status check. Essentially, this status is passed from the soft PHY and could mean: The PHY is configured badly such that it expects DQS strobes but never received them, or, The value <code>(XSPI_PHY_GATE_LPBK_CTL.RD_DEL_SEL</code> field) is incorrect and has caused data corruption and pointer misalignment in the PHY. To resolve this issue, the PHY must be reset to clear the DQS_OVF and DQS_UR flags and reset the read data pointers.

Table 14-10: Descriptor Status Field (Continued)

Bits	Name	Description
2	CRC_ERROR	When set, the bit indicates that the controller detected a CRC error.
1	BUS_ERROR	When set, the bit indicates that the controller got an error response on the system DMA interface.
0	DSC_ERROR	This bit indicates that an invalid descriptor sequence has been detected.

The auto command engine accept new commands only when the thread selected is in an idle state. The host software should check the thread status before it queues any new commands. The thread status is available in the `XSPI_TRD_STAT` register. Commands sent to a busy thread are ignored and `XSPI_ISTAT.CMD_IGNORED = 1`.

Using the Auto Command in PIO Mode

In PIO work mode, dedicated command registers are used to trigger operation in a selected execution thread of the auto command engine. In this mode it is only possible to trigger a single command in each of the execution threads. This mode is useful when host software wants to trigger a single operation. In this case it is much simpler and faster to program a few controller registers than prepare a descriptor list in system memory. Functionality of this work mode is similar to the CDMA work mode for cases where the descriptors chain contains only a single descriptor. The auto command engine executes the programmed command and returns a status word separately for each execution thread. Status words are mapped into controller status registers, where it remains valid until a new command is triggered on this thread.

Depending on the selected operation, the number of `XSPI_CMD*` required to be programmed. The registers described as unused for given operation do not need to be set. `XSPI_CMD1` to `XSPI_CMD5` registers retain the contents from previous writes meaning that if the current register configuration matches what is needed for the next operation, it can be left unchanged. The only exception is the `XSPI_CMD0` register. This register must always be written on each operation. Write access to this register triggers operation execution. Host software needs to initialize all required command registers before it writes to command register 0.

- The reset and chip erase sequences only requires `XSPI_CMD0` to be written to. `XSPI_CMD1` through `XSPI_CMD5` registers are unused.
- The sector erase sequence requires `XSPI_CMD0`, `XSPI_CMD1`, `XSPI_CMD4` and `XSPI_CMD5` to be written, `XSPI_CMD2` and `XSPI_CMD3` are unused.
- The program and read sequences require all `XSPI_CMD*` registers to be written.

The `XSPI_CMD0` - `XSPI_CMD5` tables below show the layout for the available high-level sequences that may be triggered in PIO mode. The operation is always triggered when `XSPI_CMD0` is written to, so configure `XSPI_CMD1`-`XSPI_CMD5` registers first.

Table 14-11: `XSPI_CMD0` Layout for PIO Sequences

Bits	Name	Description
31:30	CDMA/PIO	Set (=1) for PIO work mode.
29:27	-	Reserved
26:24	TRD_NUM	This field selects the destination thread number for command. Software can select any available thread. Commands can be issued in parallel to all threads.
23	-	Reserved
22:20	BANK/CS	Selects device targeted by the sequence
19	DMA_SEL	This bit must be zero. In the ADSP-2156x processors, the internal DMA of the xSPI controller is not supported.
18	INT	If this bit is set, an interrupt is issued after the operation has finished. The triggered interrupt is indicated in the <code>XSPI_TRD_COMP_ISTAT</code> register, where interrupt bit is selected by the thread number in the TRD[n] field.
17	MB_XIP_DIS	When XIP mode is enabled in the controller and the attached memory device, setting this bit indicates to the controller that it must send mode bits to disable and exit XIP mode. When set, the controller sends mode bits as specified in the <code>XSPI_XIP_GCTL.XIP_DIS_MB_VALUE</code> field. Additionally, the <code>XSPI_XIP_GCTL.XIP_EN_MB_VALUE</code> bit is cleared to disable XIP mode in the controller. This bit is valid only for read operations. If <code>XSPI_READ_SEQ_CFG1.P1_MB_EN = 1</code> and <code>XSPI_XIP_GCTL.XIP_EN_MB_VALUE = 0</code> the controller ignores the value of this field. It is treated as set (= 1) (for example, mode bits are sent regardless of the value of this field).

Table 14-11: [XSPI_CMD0](#) Layout for PIO Sequences (Continued)

Bits	Name	Description
16	MB_XIP_EN	This bit triggers entry to XIP mode. Setting this bit indicates to the controller that it must send mode bits to the device to enable XIP mode. The mode bits sent depends on the <code>XSPI_XIP_GCTL.XIP_EN_MB_VALUE</code> field. Additionally, the <code>XSPI_XIP_GCTL.XIP_EN_MB_VALUE</code> bit is set to enable XIP mode in the controller. This bit is valid only for read operations.
15:0	CMD_TYPE	This field identifies the kind of operation that is executed by the controller. It is configured as follows: 0x1100 : Reset sequence (soft reset) 0x1101 : Reset sequence (JEDEC reset) 0x1000 : Sector erase sequence 0x1001 : Chip erase sequence 0x2100 : Program sequence 0x2200 : Read sequence

The [XSPI_CMD1](#) register is required for all operations other than reset and chip erase. It holds the lower half of the memory device starting address where the operation takes place.

Table 14-12: [XSPI_CMD1](#) Layout for PIO Sequences

Bits	Name	Description
31:0	xSPIAddress (Lower)	This field contains the lower half of the starting address of the operation in the xSPI device.

The [XSPI_CMD2](#) is only required for read and program operations. It holds the lower half of the host side address where the operation fetches data for writes, or places data for reads.

Table 14-13: [XSPI_CMD2](#) Layout for PIO Sequences

Bits	Name	Description
31:0	SYS_ADDR_PTR_L	Host memory address required for DMA transfers. It is lower half of the address.

The [XSPI_CMD3](#) is only required for read and program operations. It holds the upper half of the host side address where the operation should fetch data for writes, or place data for reads.

Table 14-14: [XSPI_CMD3](#) Layout for PIO Sequences

Bits	Name	Description
31:0	SYS_ADDR_PTR_H	Host memory address required for DMA transfers. It is upper half of the address.

The [XSPI_CMD4](#) is required for sector erase operations as well as read and program operations. For sector erase, it holds the number of sectors that must be erased. For read/program operations, it holds the number of data bytes that should be transferred.

Table 14-15: [XSPI_CMD4](#) Layout for PIO SECTOR ERASE Sequences

Bits	Name	Description
31:0	SECT_CNT	Sector counter. Determines how many sectors should be erased. Number of sectors is decoded as a value of this field plus 1 (that is, to erase 1 sector this field should have value 0, and so on).

Table 14-16: [XSPI_CMD4](#) Layout for PIO READ/PROGRAM Sequences

Bits	Name	Description
31:0	DATA_CNT	Data counter. Determines how many data bytes should be transferred minus 1. For example, if 1024 bytes should be transferred, the value to be programmed should be 1023.

The [XSPI_CMD5](#) is required for all operations other than reset and chip erase. It holds the upper half of the memory device starting address where the operation should take place.

Table 14-17: [XSPI_CMD5](#) Layout for PIO Sequences

Bits	Name	Description
31:0	xSPI Address (Upper)	This field contains additional address space for the xSPI address. It is utilized for SPI NAND devices to hold both the row and column address as a single xSPI address. For sector erase operations, bits that belong to column address must be cleared (=0).

Status Checking

The host can check the operating status of each active thread. After the thread ID is written to the [XSPI_CMD_STAT_PTR](#) register, the status can be read from the [XSPI_CMD_STAT](#) register. The COMPLETE bit is cleared when a new operation is triggered, and all other fields within the [XSPI_CMD_STAT](#) register should at this point be ignored. When the selected operation is complete, the controller sets the COMPLETE bit and all other register fields reflect the status of that operation.

The *Status Checking* table describes the status fields.

Table 14-18: Status Checking

Bits	Name	Description
31:24	Reserved	Reserved
23:16	ECC_STAT	Value of the ECC status returned by the SPI NAND device. Value of this field applies to last read operation and is updated only when fail checking is enabled.
15	COMPLETE	When set, denotes that the controller has updated status information and the operation is complete. This bit shall be set even if the operation ended as a failure.
14	FAIL	When set, denotes that operation failed to complete successfully.
13:6	Reserved	Reserved
5	ECC_CORR_ERROR	ECC correctable error was detected during page read. This field is valid only for SPI NAND devices when the <code>XSPI_STAT_SEQ_CFG10.ECC_FAIL_EN</code> = 1.
4	DEVICE_ERROR	Device error was detected during read status operation for erase/program/read operation.
3	DQS_ERROR	Incorrect DQS pulses number was detected during the data read operation. This is information from the PHY that either the DQS strobe did not appear during read (for example, device is not connected to the controller) or <code>RD_DEL_SEL</code> signal value is incorrect (<code>XSPI_PHY_GATE_LPBK_CTL.RD_DEL_SEL</code>) and data read from the flash device are corrupted and read FIFO pointers in the PHY are misaligned. The <code>DLL_RST</code> signals to clear <code>DQS_OVF</code> and <code>DQS_UR</code> flags in the PHY and clear all PHY read data pointers. One of these is required by the PHY before continuing to work after the <code>DQS_ERROR</code> signal assertion.
2	CRC_ERROR	When set, it indicates that the controller detected a CRC error.

Table 14-18: Status Checking (Continued)

Bits	Name	Description
1	BUS_ERROR	When set, it indicates that the controller got an error response on the system DMA interface.
0	CMD_ERROR	This bit indicates that an invalid command sequence has been detected.

Selecting the DMA Interface

ACMD mode can be configured using only the host DMA interface in ADSP-2156x family of processors.

- To use the ACMD in CDMA mode with data transfers passing via the host DMA interface, follow the process as described in the section, [Using the Auto Command in CDMA Mode](#), clearing the DMA_SEL bit in the command flags.
- To use the ACMD in PIO mode with data transfers passing via the host DMA interface, follow the process as described in the section, [Using the Auto Command in PIO Mode](#), clearing the DMA_SEL bit of the [XSPI_CMD0](#) register.

When transfers using the slave interface are enabled, the following sequence also needs to be followed by the external host to transfer the required data:

1. Poll the `XSPI_ISTAT.SDMA_TRIGG` bit. Or, if an interrupt is preferable, unmask the interrupt associated with `SDMA_TRIGG` by setting the `XSPI_INT_EN.SDMA_TRIGG_EN` bit. When the bit is asserted, the host can proceed.
2. The external host reads the `XSPI_SDMA_SIZ` register, the `XSPI_SDMA_TRD_STAT` register and the `XSPI_SDMA_ADDR0/XSPI_SDMA_ADDR1` registers. The `XSPI_SDMA_SIZ` register reports the size of the data transfer that must be transferred. The `XSPI_SDMA_TRD_STAT` register reports the thread ID of the operation that triggered SDMA data transfer and also reports the data transfer direction. The `XSPI_SDMA_ADDR0/XSPI_SDMA_ADDR1` registers indicate the address (awaddr/araddr) the external host must use on the write or read transaction when it issues it.
3. Before starting data transmission, the external host must clear the `XSPI_ISTAT.SDMA_TRIGG` register by writing 1.
4. Host can now transfer data. For program operations, it is expected that the host will issue one or more write requests with a total size matching the data transfer indicated in step 2. For read operations, it is expected that the host will issue one or more read requests with a total size matching the data transfer indicated in step 2. The address issued by the host should match that being reported in the `XSPI_SDMA_ADDR0/XSPI_SDMA_ADDR1` registers.

If the host ignores any of the requirements and starts the data transfer when the host DMA is not ready, the `XSPI_ISTAT.SDMA_ERR` flag is set. If the `XSPI_INT_EN.SDMA_ERR_EN` bit is set, an interrupt is also triggered. When the `XSPI_DMA_CTL.SDMA_ERR_RSP` bit is set, an error response is returned; if it is cleared, the OK response is returned.

If the host sends an unsupported transaction to a slave interface, the host DMA ignores this access and the `XSPI_ISTAT.SDMA_ERR` flag is set. If an error response is detected on the system bus during host DMA transfer, the fail and bus error bits are set in the last operation status. The last operation status can check in:

- The `XSPI_CMD_STAT` register. After selecting thread in which command was executed (for ACMD PIO work mode). Thread is selected by value of the `XSPI_CMD_STAT_PTR` register. This method is valid for the ACMD PIO and STIG work modes.
- The status field of the command descriptor. This method is valid for the CDMA work mode.

Performing transactions in wrong direction (that is, the host issuing a write transaction to the slave port in case of sending read operation command to an xSPI device) is forbidden and causes the `XSPI_ISTAT.SDMA_ERR` flag to be set. If the `XSPI_INT_EN.SDMA_ERR_EN` bit is set, an interrupt is also triggered.

Rules and Limitations

The host must follow the following rules when using ACMD mode:

- For SPI NAND devices, the initial column address must be specified within the range of the page size.
- For profile 2 or profile 1, octal DDR devices, the number of bytes to be transferred must be even.
- For profile 2 or profile 1, octal DDR devices, the start address of the transfer in the xSPI device must be even.
- For profile 1 and octal DDR devices, the command extension must be enabled.
- For `DATA_PER_ADDR` (when the device is 16-bit addressed), the start address must be even and the number of bytes to transfer must be even.
- If XIP mode is enabled, only read operations are permitted.
- Only the supported command types as illustrated in this section should be attempted.

Violating any of the above rules results in the controller returning the `DSC_ERROR/CMD_ERROR` status.

NOTE: `DSC_ERROR` and `CMD_ERROR` status bits are only intended to ease debug and integration and should never be triggered for correct controller operation. Using the controller outside of the descriptor and command rules may lead to undefined behavior.

In automatic work modes, the controller writes/reads data consecutively from the specified address. If a page boundary is crossed, the transaction continues from the beginning of the next page.

Operating in STIG Mode

The Software-Triggered Instruction Generator (STIG) mode allows software to define the low-level commands and sequences sent to the memory device. Relative to direct or ACMD mode, it is more flexible because software can control the low-level characteristics of each operation carried out on the device. STIG mode is normally used with the direct or ACMD modes of operation.

To enable the STIG work mode, the `XSPI_WORKMODE_CTL.WORKMODE` field must be set (= 1) . This can be done only when the controller is in an idle state. The *STIG Mode Registers* table shows the registers used to configure STIG mode operations.

Table 14-19: STIG Mode Registers

Bits	Name	Description
31:0	XSPI_CMD0	A write to this register triggers the STIG operation and has no other purpose. Reading this register is ignored.
31:0	XSPI_CMD1	Holds bits [31:0] of the STIG instruction
31:0	XSPI_CMD2	Holds bits [63:32] of the STIG instruction
31:0	XSPI_CMD3	Holds bits [95:64] of the STIG instruction
31:0	XSPI_CMD4	Holds bits [127:96] of the STIG instruction

Instruction Variants

The main instruction variants for STIG mode include the generic command instruction, legacy HyperFlash or profile 2 instructions, and legacy SPI xSPI profile 1 or SPI NAND instructions. The following tables show the structure of the STIG command instruction (`XSPI_CMDn`) for each variant.

Table 14-20: STIG Command Instruction - Legacy SPI, xSPI Profile 1 or SPI NAND

Name	Bits	Description
Reserved	127:126	N/A
t _{CMS} /t _{CEM} Enable (XSPI_SEQ_GCTL0.TCMS_EN)	125	Setting this bit enables the feature that automatically forces the controller to pause activity to the memory device to allow it to perform a periodic refresh operation. Some volatile memory devices require this functionality (for example, HyperRAM). This functionality is referred to as t _{CMS} timing. When set, the controller pauses activity as specified in the XSPI_MINICTL_DEV_ACTIVE_MAX register.
Instruction Termination (INSTR_LINK)	124	This bit is only relevant when the instruction type field of the instruction = 0, 32, 36 or 96. See the section Instruction Type Variants for details on instruction type configuration. The field assigned to this bit indicates whether the instruction is to be terminated (CS is deactivated after the instruction finishes) or the instruction is glued with the next instruction (providing it is ready to be fetched from the command FIFO). 0 – Current sequence is terminated and not glued with another instruction of the same type 1 – Current sequence is glued with the next STIG instruction
Reserved	123:119	N/A

Table 14-20: STIG Command Instruction - Legacy SPI, xSPI Profile 1 or SPI NAND (Continued)

Name	Bits	Description
CRC Variant (CRC_VARIANT)	118	Determines when the CRC is calculated on an outbound command. When cleared (=0), calculate for the address bytes only. When set (=1), calculate for all bytes in the sequence.
CRC Enable (CRC_EN)	117	Set to enable dynamic CRC calculation and output the value on xSPI flash interface in the transfer modes as specified by ADDR_EDGE_MODE and ADDR_NO_OF_IOS. If the octal DDR (8D) variant is configured, the calculated CRC byte is maintained for a full flash clock cycle.
XIP Enable (XIP_EN)	116	Setting this bit instructs the controller to block the sending of the opcode, which is needed when the device is operating in XIP mode 0 – Opcode transfer phase is sent 1 – Opcode transfer phase is not sent (XIP enabled)
Address Shift (ADDR_SHIFT)	115	When the ADDR_EDGE_MODE and ADDR_NO_OF_IOS configuration determines the octal DDR variant, two bytes are required to be sent in a single flash clock cycle. Therefore, the host must configure an even number of bytes in the quoted case. However, there are specific exceptions such as the Adesto octal DTR SFDP command which requires a {A2,A1,A0,do not care} flash interface address alignment rather than a common {A3,A2,A1,A0} alignment. The ADDR_SHIFT bit shifts the address byte chain by one byte towards the left and fills the least-significant byte with zeroes (when an octal DDR address structure is required).
Reserved	114:111	N/A
Bank Number (BANK_NUM)	110:108	This field informs the controller on which memory device the sequences should be executed. This information directly maps to the chip select (CS) pin.
Opcode Edge Mode (OPCODE_EDGE_MODE)	107	This bit selects SDR or DDR operation at the memory device 0 – SDR 1 – DDR
Reserved	106	N/A

Table 14-20: STIG Command Instruction - Legacy SPI, xSPI Profile 1 or SPI NAND (Continued)

Name	Bits	Description
Opcode IO Mode (OPCODE_NO_OF_IOS)	105:104	Determines number of active IOs for opcode phase of the instruction 0 – Single 1 – Dual 2 – Quad 3 – Octal
Reserved	103:101	N/A
Opcode Extension Enable (OPCODE_EXT_EN)	100	Enables dual byte opcode for the instruction 0 – Only CMD[87:80] is transmitted in the opcode phase of the instruction. 1 – CMD[87:80] followed by CMD_EXT[79:72] is transmitted in the opcode phase of the instruction. If the OPCODE_EDGE_MODE and OPCODE_NO_OF_IOS configuration determines the octal DDR variant, two bytes are required to be sent in a single flash clock cycle. The host must also set OPCODE_EXT_EN =1. Failure to do this results in an error being reported.
Address Edge Mode (ADDR_EDGE_MODE)	99	Determines edge mode for the address phase of the instruction. 0 – SDR 1 – DDR
Reserved	98	N/A
Address IO Mode (ADDR_NO_OF_IOS)	97:96	Determines the number of active IOs for the address phase of the instruction 0 – Single 1 – Dual 2 – Quad 3 – Octal
Reserved	95	N/A

Table 14-20: STIG Command Instruction - Legacy SPI, xSPI Profile 1 or SPI NAND (Continued)

Name	Bits	Description
Address Number of bytes (ADDR_NO_OF_BYTES)	94:92	Determines the number of address bytes to be sent in the instruction. Up to six address bytes can be sent in one sequence. A value of 7 encoded here is not permitted. A value of 0 encoded implies that no address bytes are sent (for example, address phase is disabled). If ADDR_EDGE_MODE and ADDR_NO_OF_IOS configuration determines the octal DDR variant, two bytes are required to be sent in a single flash clock cycle. The host must configure an even number of address. If the host configures ADDR_NO_OF_BYTES = 1 or 3 or 5, the controller generates an error.
Reserved	91:90	N/A
Number of data bytes (DATA_NO_OF_BYTES) or Number of mode bytes (MODE_NO_OF_BYTES)	89:88	For cases where the number of data (or mode) bytes is ≤ 2, this field allows that data to be included within this instruction, directly avoiding the need to glue a separate data sequence instruction. If there are more than two data bytes required to be transferred, the instruction must be glued with a data sequence. Note that if up to two data (or mode) bytes are transferred directly from this instruction, the format in which they are presented to the device depends on the ADDR_EDGE_MODE and ADDR_NO_OF_IOS configuration. If the host needs the data to be sent in a different format, this field must be cleared (=0). Instead, transfer the data using a glued data sequence instruction, or generic instruction where the format of those data bytes can be individually configured. 0 – zero data/mode bytes are sent to device (data phase disabled) 1 – One byte is transferred to the device 2 – Two bytes are transferred to the device 3 – N/A If ADDR_EDGE_MODE and ADDR_NO_OF_IOS configuration determines the octal DDR variant, two bytes are required to be sent in a single flash clock cycle. The host must configure an even number of bytes. Failure to do this results in an error being reported.
CMD	87:80	Opcode mnemonic
CMD_EXT	79:72	Opcode extension mnemonic
ADDR5	71:64	Byte 5 of the address phase
ADDR4	63:56	Byte 4 of the address phase
ADDR3	55:48	Byte 3 of the address phase

Table 14-20: STIG Command Instruction - Legacy SPI, xSPI Profile 1 or SPI NAND (Continued)

Name	Bits	Description
ADDR2	47:40	Byte 2 of the address phase
ADDR1	39:32	Byte 1 of the address phase
ADDR0	31:24	Byte 0 of the address phase
DATA1/MODE1	23:16	Byte 1 of the data/mode phase
DATA0/MODE0	15:8	Byte 0 of the data/mode phase
Reserved	7	N/A
Instruction Type	6:0	This field defines the type (index) of the instruction. A description of each value and its corresponding instruction can be found in the section, Instruction Type Variants .

Table 14-21: STIG Instruction - Legacy HyperFlash or xSPI Profile 2

Name	Bits	Description
Reserved	127:126	N/A
t _{CMS} /t _{CEM} Enable (TCMS_EN)	125	Setting this bit enables the feature that automatically forces the controller to pause activity to the memory device to allow it to perform a periodic refresh operation. Some volatile memory devices require this functionality (for example, HyperRAM) and is referred to as t _{CMS} timing. When set, the controller pauses activity as specified in the XSPI_MINICTL_DEV_ACTIVE_MAX register.
Instruction Termination (INSTR_LINK)	124	This bit is only relevant when the instruction_type field of the instruction = 0, 32, 36 or 96. Refer to the Instruction Type Variants section for details on instruction type configuration. The field assigned to this bit indicates whether the instruction is to be terminated (CS is to be deactivated after the instruction finishes) or the instruction is to be glued with the next instruction providing it is ready to be fetched from the command FIFO. 0 – Current sequence is terminated and not glued with another instruction of the same type 1 – Current sequence is glued with the next STIG instruction
Reserved	123:119	N/A
CRC Variant (CRC_VARIANT)	118	When the CRC is calculated on the outbound command, this bit indicates when cleared (=0) to calculate for the address bytes only, or when set (=1) to calculate for all bytes in the sequence.

Table 14-21: STIG Instruction - Legacy HyperFlash or xSPI Profile 2 (Continued)

Name	Bits	Description
CRC Enable (CRC_EN)	117	Set to enable dynamic CRC calculation and output this value on xSPI flash interface in the transfer modes specified by ADDR_EDGE_MODE and ADDR_NO_OF_IOS configuration. If an octal DDR (8D) variant is configured, the calculated CRC byte is maintained for full flash clock cycle.
XIP Enable (XIP_EN)	116	Setting this bit instructs the controller to block the sending of the opcode, which is needed when the device is operating in XIP mode. 0 – Opcode transfer phase is sent 1 – Opcode transfer phase is not sent (XIP enabled)
Address Shift (ADDR_SHIFT)	115	If ADDR_EDGE_MODE and ADDR_NO_OF_IOS configuration determine the octal DDR variant, two bytes are required to be sent in a single flash clock cycle. The host must configure an even number of bytes in the quoted case. However, there are specific exceptions. For example, the Adesto octal DTR SFDP command requires a {A2,A1,A0,do not care} flash interface address alignment rather than the common {A3,A2,A1,A0} alignment. The ADDR_SHIFT bit shifts the address byte chain by one byte towards the left and fills the least-significant byte with zeroes providing the octal DDR address is required.
Reserved	114:111	N/A
Bank Number (BANK_NUM)	110:108	This field informs the controller on which memory device the sequence should be executed. This information directly maps to the chip select (CS) pin.
Opcode Edge Mode (OPCODE_EDGE_MODE)	107	This bit selects SDR or DDR operation at the memory device 0 – SDR 1 – DDR
Reserved	106	NA
Opcode IO Mode (OPCODE_NO_OF_IOS)	105:104	Indicates the number of active IOs for opcode phase of the instruction 0 – Single 1 – Dual 2 – Quad 3 – Octal
Reserved	103:101	N/A

Table 14-21: STIG Instruction - Legacy HyperFlash or xSPI Profile 2 (Continued)

Name	Bits	Description
Opcode Extension Enable (OPCODE_EXT_EN)	100	Enables dual byte opcode for the instruction 0 – Only CMD[87:80] is transmitted in the opcode phase of the instruction 1 – CMD[87:80] followed by CMD_EXT[79:72] is transmitted in the opcode phase of the instruction If OPCODE_EDGE_MODE and OPCODE_NO_OF_IOS configuration determines the octal DDR variant, two bytes are required to be sent in a single flash clock cycle. It is therefore required that the host also sets OPCODE_EXT_EN. Failure to do this results in an error being reported.
Address Edge Mode (ADDR_EDGE_MODE)	99	Determines edge mode for address phase of the instruction 0 – SDR 1 – DDR
Reserved	98	N/A
Address IO Mode (ADDR_NO_OF_IOS)	97:96	Determines number of active IOs for the address phase of the instruction 0 – Single 1 – Dual 2 – Quad 3 – Octal
Reserved	95	N/A
Address Number of bytes (ADDR_NO_OF_BYTES)	94:92	Determines number of address bytes to be sent in the instruction. Up to 6 address bytes can be sent in one sequence. A value of 7 encoded here is not permitted. A value of 0 encoded implies that no address bytes is sent (for example, the address phase is disabled). If ADDR_EDGE_MODE and ADDR_NO_OF_IOS configuration determines the octal DDR variant, two bytes are required to be sent in a single flash clock cycle. It is, therefore, required for the host to configure an even number of address. If the host configures ADDR_NO_OF_BYTES = 1, 3 or 5, the controller generates an error.
Reserved	91:90	N/A

Table 14-21: STIG Instruction - Legacy HyperFlash or xSPI Profile 2 (Continued)

Name	Bits	Description
Number of data bytes (DATA_NO_OF_BYTES) or Number of mode bytes (MODE_NO_OF_BYTES)	89:88	For cases where the number of data (or mode) bytes is ≤ 2, this field allows that data be included within this instruction, directly avoiding the need to glue a separate data sequence instruction. If there are more than two data bytes required to be transferred, the instruction must be glued with a data sequence. Note that if up to two data (or mode) bytes are transferred directly from this instruction, the format in which they are presented to the device depends on the ADDR_EDGE_MODE and ADDR_NO_OF_IOS configuration. If the host needs the data to be sent in a different format, the field must be cleared (=0). Instead, transfer the data using a glued data sequence instruction, or generic instruction, where the format of the data bytes can be individually configured. 0 – Zero data/mode bytes are sent to device (data phase disabled) 1 – One byte is transferred to the device 2 – Two bytes are transferred to the device 3 – N/A If ADDR_EDGE_MODE and ADDR_NO_OF_IOS configuration determines the octal DDR variant, two bytes are required to be sent in a single flash clock cycle. The host must configure an even number of bytes. Failure to do this results in an error being reported.
CMD	87:80	Opcode mnemonic
CMD_EXT	79:72	Opcode extension mnemonic
ADDR5	71:64	Byte 5 of the address phase
ADDR4	63:56	Byte 4 of the address phase
ADDR3	55:48	Byte 3 of the address phase
ADDR2	47:40	Byte 2 of the address phase
ADDR1	39:32	Byte 1 of the address phase
ADDR0	31:24	Byte 0 of the address phase
DATA1/MODE1	23:16	Byte 1 of the data/mode phase
DATA0/MODE0	15:8	Byte 0 of the data/mode phase
Reserved	7	N/A
Instruction Type	6:0	This field defines the type (index) of the instruction. A description of each value and its corresponding instruction can be found in the section, Instruction Type Variants .

Table 14-22: Generic Command STIG Instruction

Name	Bits	Description
Reserved	127:125	N/A
Instruction Termination (INSTR_LINK)	124	This bit is only relevant when the instruction_type field of the instruction=0, 32, 36 or 96. Refer to section Instruction Type Variants for details on the instruction type setting. The field assigned to this bit indicates whether the instruction is to be terminated (CS must be deactivated after the instruction finishes) or glued with the next instruction (providing it is ready to be fetched from the command FIFO). 0 – Current sequence is terminated and not glued with another instruction of the same type 1 – Current sequence is glued with the next STIG instruction
Reserved	123:118	N/A
CRC Enable (CRC_EN)	117	Set to enable dynamic CRC calculation and output this value on the xSPI flash interface. The transfer modes are specified by ADDR_EDGE_MODE and ADDR_NO_OF_IOS configuration. If the octal DDR (8D) variant is configured, the calculated CRC byte is maintained for a full flash clock cycle.
Reserved	116:111	N/A
Bank Number (BANK_NUM)	110:108	This field informs the controller on which memory device the sequence should be executed. This information directly maps to the chip select (CS) pin.
Edge Mode (EDGE_MODE)	107	This bit selects SDR or DDR operation at the memory device. 0 – SDR 1 – DDR
Reserved	106	N/A
IO Mode (NO_OF_IOS)	105:104	Determines number of active IOs for the instruction 0 – Single 1 – Dual 2 – Quad 3 – Octal
Reserved	103:92	N/A

Table 14-22: Generic Command STIG Instruction (Continued)

Name	Bits	Description
Number of bytes (NO_OF_BYTES)	91:88	Determines the number of bytes to be sent in the instruction At least one byte must be sent, but no more than 10 bytes in one sequence. If EDGE_MODE and NO_OF_IOS configuration determines the octal DDR variant, two bytes must be sent in a single flash clock cycle. The host must configure an even number of bytes in the quoted case (allowable NO_OF_BYTES = 0 or 2 or 4 or 6 or 8 or 10). If the host configures NO_OF_BYTES = 1 or 3 or 5 or 7 or 9, the controller does not accept the transfer request and generates an error.
BYTE9	87:80	The 10th byte sent when NO_OF_BYTES = 10
BYTE8	79:72	The 9th byte sent when NO_OF_BYTES ≥ 9
BYTE7	71:64	The 8th byte sent when NO_OF_BYTES ≥ 8
BYTE6	63:56	The 7th byte sent when NO_OF_BYTES ≥ 7
BYTE5	55:48	The 6th byte sent when NO_OF_BYTES ≥ 6
BYTE4	47:40	The 5th byte sent when NO_OF_BYTES ≥ 5
BYTE3	39:32	The 4th byte sent when NO_OF_BYTES ≥ 4
BYTE2	31:24	The 3rd byte sent when NO_OF_BYTES ≥ 3
BYTE1	23:16	The 2nd byte sent when NO_OF_BYTES ≥ 2
BYTE0	15:8	The 1st byte sent
Reserved	7	N/A
Instruction Type	6:0	This field defines the type (index) of the instruction. A description of each value and its corresponding instruction can be found in the section Instruction Type Variants .

Glued Data Instruction

The *Data Instruction Structure* table outlines the structure of the main data instruction variant that the host issues as the final glued instruction of a sequence of instructions. This variant cannot be used by itself and must be glued to one of the other variants. It is typically used to define how data is transferred in accordance with the details specified in the other three main command variants. As such, this variant must never be the first instruction issued by the host; it is always the last instruction.

Table 14-23: Data Instruction Structure

Name	Bits	Description
Reserved	127:123	N/A

Table 14-23: Data Instruction Structure (Continued)

Name	Bits	Description
CRC Chunk Size (CRC_CHUNK_SIZE)	122:120	This field presents the size of the CRC chunk as configured in the memory device. It is the number of bytes after which the CRC byte is placed onto the bus. 1 – N/A 2 – 8B chunk size 3 – 16B chunk size 4 – 32B chunk size 5 – 64B chunk size 6 – 128B chunk size 7 – 256B chunk size 8 – 512B chunk size
CRC Read Status Inversion Enable (CRC_INV)	119	This bit controls whether the controller requests one more status slice to check its data integrity. The expectation is that the redundant data chunk will be inverted (as applicable for Macronix MX25 octal devices). If redundant, data from flash is not inverted. A CRC error is generated. This bit must only be set when CRC_EN = 1, DATA_EDGE_MODE = 1 (DDR), DATA_NO_OF_IOS = 3 (octal) and DATA_NO_OF_BYTES = 2 because this set of parameters matches the implementation of Macronix MX25 octal devices. If CRC_INV = 1, the CRC_CHUNK_SIZE configuration is ignored because these two sequence parameters are mutually exclusive.
CRC Output Enable (CRC_OE)	118	This bit specifies whether the CDC is an output on both clock edges, with the CRC being inverted on the second clock edge. 0 – Inverted CRC is not output 1 – Inverted CRC is output
CRC Enable (CRC_EN)	117	This bit is set to enable dynamic CRC calculation and output this value on the xSPI flash interface. The transfer modes are specified by ADDR_EDGE_MODE and ADDR_NO_OF_IOS configurations. If an octal DDR (8D) variant is configured, the calculated CRC byte is maintained for a full flash clock cycle.

Table 14-23: Data Instruction Structure (Continued)

Name	Bits	Description
CRC Unaligned Chunk Enable (CRC_UAL_CHUNK_EN)	116	Setting this bit instructs the controller to take into consideration the command address in determining where the CRC bytes are expected to be returned from the memory device. This bit can be set (=1) only when the sequence (DATA_SEQ) is glued with the READ_PROFILE_1 sequence. It must be cleared (=0), otherwise. For example, if the original flash address is 0x4 and CRC_CHUNK_SIZE is 16B, the first CRC from the device is expected to occur after 12 bytes (CRC_CHUNK_SIZE - (device address)). Only even addresses are supported (when CRC_UAL_CHUNK_EN = 1) as required by Macronix MX25 octal devices.
CRC Unaligned Chunk Check (CRC_UAL_CHUNK_CHK)	115	Set this bit to have the controller check the correctness of a CRC unaligned chunk from a flash device. It can be set (=1) only when CRC_UAL_CHUNK_EN = 1. It must be cleared (=0), otherwise. Because Macronix MX25 octal devices do not guarantee the correctness of CRC calculated for unaligned chunk, the controller provides control of this check. When configured, DATA_NO_OF_BYTES accommodates many CRC chunks; the CRC correctness is checked for all chunk-aligned CRC slices regardless of the CRC_UAL_CHUNK_CHK value.
Reserved	114:113	N/A
Data Swap	112	When set (=1), this bit swaps the order of bytes transferred in a single octal DDR clock cycle. When DATA_EDGE_MODE and DATA_NO_OF_IOS do not reflect the octal DDR mode, this bit should be cleared (=0).
Reserved	111	N/A
Bank Number (BANK_NUM)	110:108	This field informs the controller on which memory device the sequence should be executed. This value directly maps to the chip select (CS) pin.
Data Edge Mode (DATA_EDGE_MODE)	107	This bit selects SDR or DDR operation at the memory device 0 – SDR 1 – DDR
Reserved	106	N/A

Table 14-23: Data Instruction Structure (Continued)

Name	Bits	Description
Data IO Mode (DATA_NO_OF_IOS)	105:104	Determines number of active IOs for all data bytes of the instruction Data bytes associated with the instruction are taken from an internal data FIFO (for write operations) and put into the internal data FIFO (for read operations) 0 – Single 1 – Dual 2 – Quad 3 – Octal
Reserved	103:101	N/A
Transfer Direction (DIR)	100	Determines the direction of the data stream 0 – Read from memory device 1 – Write to memory device
Reserved	99:93	N/A
Data Size per Flash Address Location (DATA_PER_ADDR)	92	Indicates how many bytes of data are assigned to a single flash memory address. The controller uses this information to internally calculate consecutive addresses of all data bytes in a sequence. The address incrementation step is assumed to be 1. 0 – The memory device is byte addressable 1 – The memory device is two byte addressable
Reserved	91:90	N/A
Number of dummy (latency) cycles (NO_OF_DUMMY)	89:84	Indicates the number of dummy cycles before the data phase Note that the definition of latency cycles for legacy HyperFlash, and xSPI profile 2 devices overlaps with the address transfer phase by one clock cycle. The number of required latency cycles is one less than programmed. If an external board delay incurs a few more clock cycle long additional latency (for read commands), compensation is handled by the PHY module.
HyperFlash Read Crossing Boundary Enable (HF_READ_BOUND_EN)	83	The controller uses this bit to calculate a read transaction crossing a page boundary phase in which data is not valid (based on the number of dummy cycles (NO_OF_DUMMY) and the start address of the associated instruction). During the latency period, the device does not generate DQS.
Reserved	82:80	N/A

Table 14-23: Data Instruction Structure (Continued)

Name	Bits	Description
Number of data bytes (DATA_NO_OF_BYTES)	79:48	Indicates the number of data bytes to be sent in the instruction. All data bytes are taken from the data FIFO of the controller (for write operations) and put into the data FIFO (for read operations). When DATA_EDGE_MODE and DATA_NO_OF_IOS configuration indicates the octal DDR variant, two bytes are required to be sent in a single flash clock cycle. Therefore, the host must configure an even number of data bytes. Failure to do this causes the controller to generate an error.
Reserved	47:25	N/A
Status Data Source (STATUS_SOURCE)	24	The bit is valid only when DIR = 0 (read transaction) and there are ≤ 2 bytes to be read. When there are >2 bytes to be read, this bit should be zero. This bit determines how the host eventually receives the data. If there are ≤ 2 bytes to be read, the host can choose to have the two bytes reported in the STIG status register at offset 0x44. Otherwise, the read data must be fetched using the slave AXI interface. 0 – Read data is sent to the AXI slave interface 1 – Read data is sent to the STIG status register at offset 0x44
Reserved	23:7	N/A
Instruction Type	6:0	This value is always fixed at 127 for the glued data instruction.

Instruction Type Variants

The *Typical Device Operations* table shows typical operations and the associated value in the instruction type field that the host can carry out on the device using instruction variant 1. The table also outlines whether an instruction supports instruction gluing.

NOTE: If a data instruction is glued to the back of one of the instructions, the data instruction must have an instruction type field configured to 128.

Table 14-24: Typical Device Operations - Variant 1

Operation	Instruction Type	Assumed Instruction Gluing
Read	1	Yes – Glue with data instruction
Read In XIP	1	Yes – Glue with data instruction
Write	0	Yes, if data length is greater than two bytes
Set WEL	0	No
Clear WEL	0	No

Table 14-24: Typical Device Operations - Variant 1 (Continued)

Operation	Instruction Type	Assumed Instruction Gluing
Erase Sector	0	No
Chip Erase	0	No
Program Suspend	0	No
Program Resume	0	No
Read Device Register	1	No
Write Device Register	0	No
Setup Read Burst	0	No
Clear Flag Register	0	No
Enter Deep Powerdown	0	No
Exit Deep Powerdown	0	No
Reset Enable	0	No
Soft Reset	2	No
Read JEDEC ID	1	No
Read SFDP	1	Yes – Glue with data instruction
Enter Default Protocol Mode	0	No
Enter SDR Octal	0	No
Global Protection	0	No
Global Unprotection	0	No
Protect Sector	0	No
Unprotect Sector	0	No
Enter 4-Byte Address Mode	0	No
Exit 4-Byte Address Mode	0	No
SPI_NAND Page Read	0	Yes – Glue with data instruction
SPI_NAND Read From Cache	0	Yes – Glue with data instruction
SPI_NAND Page Read Page Cache Random	0	No
SPI_NAND Page Read Page Cache Last	0	No
SPI_NAND Program Load	0	Yes, if data length is greater than two bytes
SPI_NAND Program Load Random	0	Yes, if data length is greater than two bytes
SPI_NAND Program Execute	0	Yes, if data length is greater than two bytes
SPI_NAND Set Feature	0	Yes, if data length is greater than two bytes
SPI_NAND Get Feature	1	Yes – Glue with data instruction

Table 14-24: Typical Device Operations - Variant 1 (Continued)

Operation	Instruction Type	Assumed Instruction Gluing
SPI_NAND Read ID	32	Yes – Glue with data or variant instruction

The *Typical Device Operations* table shows typical operations and the associated value in the instruction type field that the host can carry out on the device using instruction variant 2. The table outlines some of the instruction fields that must be forced to a fixed value to carry out the operation, and whether an instruction supports instruction gluing.

The variant should be used when trying to formulate a generic or custom command sequence that is not a known protocol-specific instruction. For this instruction, select an instruction type field of 96. This instruction is permitted to be glued with another variant 3 instruction or with a data sequence instruction.

Table 14-25: Typical Device Operations - Variant 2

Operation	Instruction Type	Assumed Instruction Gluing
Write/Read for HyperRAM Devices	32	Yes – Glue with data or variant instruction
WREN 1	33	No
WREN 2	34	No
Profile 2 Read	35	Yes – Glue with data instruction
Profile 2 Program	36	Yes, if the data length is greater than two bytes. Can glue with data or another variant
Erase Sector	37	No
Chip Erase	38	No
Erase Suspend	39	No
Erase Resume	40	No
Program Suspend	41	No
Program Resume	42	No
Read Status	43	Yes – Glue with data instruction
Read Volatile Memory Register	44	Yes – Glue with data instruction
Read Non-Volatile Memory Register	45	Yes – Glue with data instruction
Read Customer Memory Register	47	Yes – Glue with data instruction
Status Register Clear	48	No
Write Volatile Memory Register	49	No
Write Non-Volatile Memory Register Enable	50	No
Write Custom Memory Register Enable	51	Yes – Complete with another variant 2 instruction with instruction type = 32
Enter Deep Powerdown	52	No

Table 14-25: Typical Device Operations - Variant 2 (Continued)

Operation	Instruction Type	Assumed Instruction Gluing
Reset/ASO Exit	53	No
Enter Default Protocol Mode	54	No
Read SFDP	55	Yes – Glue with data instruction

Switching Work Modes

STIG work mode provides an interface to execute low-level transactions on the xSPI flash interface. It is expected to be used in conjunction with ACMD or direct work mode. Therefore, switching into and out of STIG mode can become a common requirement. The host software must enter and exit STIG mode when the target xSPI flash device is in the idle state.

Before the host switches to STIG mode, it must wait for all operations that address the same xSPI flash device to be finished. Before the host exits STIG mode, it must check whether the device is in an idle state. Waiting for the last STIG command to complete execution may not be enough for cases where the command caused the xSPI device go into a busy state. In such cases, the host can poll the status register of the device. It can issue STIG instructions to execute read status commands and continue to do this until the memory device has reached an idle state. Once the device is in the idle state, the host can exit STIG mode.

Programming STIG Mode

Complete the following steps to program STIG mode.

1. Wait for xSPI flash controller to be in the idle state, then, poll the `XSPI_GSTAT.CTL_BUSY` bit.
2. Configure the `XSPI_WORKMODE_CTL` field to 2'b01 to enable STIG work mode.
3. Configure the command registers (`XSPI_CMD1` to `XSPI_CMD4`).
4. Enable the STIG DONE interrupt when completion of the requested STIG transaction must raise external interrupt or `sdma_trigg` for the AXI slave interface transactions. Set the `XSPI_INT_EN.STIG_DONE_EN`, `XSPI_INT_EN.SDMA_TRIGG_EN`, and `XSPI_INT_EN.GINT_EN` bits to enable these interrupts.
5. Start the STIG operation by writing to the `XSPI_CMD0` register. Set `XSPI_GSTAT.GCMD_ENG_MC_BUSY = 1`.
6. Based on the instruction type field, the host must select one of the following:
 - Gluing with a data sequence, proceed to the next step.
 - For other instructions without the gluing condition, skip to step 9.
 - Otherwise, return to configuring the command register step.
7. When the instruction that contains the data sequence command indicates a read direction, and the number of data to be read is one or two bytes and the `STATUS_SOURCE` field in the instruction = 1, then skip to step 9.

In all other cases, proceed to the next step.

8. The host transfers data through the AXI host interface of the controller to complete the STIG operation. In this step, `XSPI_GSTAT.GCMD_ENG_MC_BUSY = 1`.
 - a. Poll the `XSPI_ISTAT.SDMA_TRIGG` bit, or alternatively wait for the interrupt if the `SDMA_TRIGG` interrupt was unmasked. When `XSPI_ISTAT.SDMA_TRIGG = 1`, proceed.
 - b. Read the `XSPI_SDMA_SIZ` and `XSPI_SDMA_TRD_STAT` registers to determine the number of bytes to transfer, the thread ID and transfer direction.
 - c. Clear the `XSPI_ISTAT.SDMA_TRIGG` flag.
 - d. The external host can now transfer data. For program operations, it is expected that the external host will issue one or more AXI write requests with a total size matching the data transfer indicated in step b. For read operations, it is expected that the external host will issue one or more AXI read requests with a total size matching the data transfer indicated in step b. The external host should issue an address that matches the one reported in the `XSPI_SDMA_ADDR0` and `XSPI_SDMA_ADDR1` registers.
 - e. If the host ignores any of the requirements described above and starts the data transfer when the host DMA is not ready, the `XSPI_ISTAT.SDMA_ERR` flag is set. If `XSPI_INT_EN.SDMA_ERR_EN = 1`, an interrupt is also triggered. When `XSPI_DMA_CTL.SDMA_ERR_RSP = 1`, an error response is returned. When `XSPI_DMA_CTL.SDMA_ERR_RSP = 0`, an OK response is returned.
 - f. If the host sends an unsupported transaction to the slave interface, the slave DMA ignores the access and the `SDMA_ERR` flag in the `XSPI_ISTAT` register is set. If an AXI error response is detected on the system bus during host DMA transfer, the fail and bus error bits are set in the last operation status. The last operation status can be checked in the `XSPI_CMD_STAT` register.
 - g. Performing transactions in wrong direction (that is, a write transaction to host DMA port in case of sending read operation command to xSPI device) is forbidden and causes the `XSPI_ISTAT.SDMA_ERR` flag to be set. When `XSPI_INT_EN.SDMA_ERR_EN = 1`, an interrupt is also triggered. When `XSPI_DMA_CTL.SDMA_ERR_RSP = 1`, an error response is returned. When `XSPI_DMA_CTL.SDMA_ERR_RSP = 0`, an OK response is returned.
9. Wait for the STIG transaction completion status. Poll the `COMPLETE` bit in the `XSPI_CMD_STAT` register or view the status through the external interrupt from the `STIG_DONE` (when correctly enabled by the host). In this step, `XSPI_GSTAT.GCMD_ENG_BUSY = 1`.
10. Check the STIG status by reading the `XSPI_CMD_STAT` register. In this step, the generic command engine busy flags are programmed to a low value.
11. If another STIG operation must be executed, go to step 3. Otherwise, if the work mode must be changed, wait for the controller idle state by polling the `XSPI_GSTAT.CTL_BUSY` bit.

STIG Mode Status

The host can check the latest STIG operation status by reading the `XSPI_CMD_STAT` register.

After every new operation is triggered, the `COMPLETE` bit is cleared and other bits in the status register are not valid. When the operation completes, the `COMPLETE` bit is set and other bits in the status register become valid.

Table 14-26: STIG Mode XSPI_CMD_STAT Bit Descriptions

Name	Bits	Description
DATA_FROM_DEV	31:16	This field holds one or two bytes read from memory device. Fields of the DATA_SEQ instruction must be configured as follow: DATA_NO_OF_BYTES = 1 – one byte, or 2 – two bytes STATUS_SOURCE = 1. In other cases, data from the flash device is available on the slave data interface.
COMPLETE	15	When set, it denotes that the controller has updated status information and the operation is complete. This bit is set even when the operation ended as a failure.
FAIL	14	When set, it denotes that the operation failed to complete successfully.
Reserved	13:4	N/A
DQS_ERROR	3	The bit indicates that an incorrect DQS pulses number was detected during the data read operation. This is information from the PHY that either the DQS strobe did not appear during read (for example, the device is not connected to the controller) or the RD_DEL_SEL signal value is incorrect (field in the XSPI_PHY_GATE_LPBK_CTL) and data read from flash device are corrupted and read FIFO pointers in the PHY are misaligned. The DLL RESET or RESET signals clear the underrun/overflow flags in the PHY and clear all PHY read data pointers. One of these flags is required by the PHY before continuing to work after a DQS_ERROR assertion.
CRC_ERROR	2	This bit indicates that a CRC error has been received
BUS_ERROR	1	When set, it indicates that the controller received an error response on the system DMA interface.
CMD_ERROR	0	This bit indicates that an invalid command sequence has been detected.

STIG Limitations

STIG mode has the following limitations:

- The HOLD# functionality (alternative function of DQ3 pin of some legacy SPI devices) is not supported in STIG mode. Since STIG mode is fully software controlled, the HOLD# function is not particularly useful.

Transfer management can instead be handled by higher-level layers (system layers) throttling when STIG transfers are triggered.

- The xSPI flash controller does not support wrap data flash transfer commands (all data to/from device are considered linear).
- The maximum number of bytes that can be read when using a generic STIG instruction type is limited to 64 bytes. Similarly, a maximum of 64 bytes can be read for all legacy HyperFlash and xSPI profile 2 instructions other than the READ_PROFILE_2 and READ_SFDP_P2.
- If XIP mode is used, the read operation that is used to either enter or exit XIP mode should not attempt to transfer more than 64B.

Sequencing STIG Operation

Internally, the STIG instructions are held in the main data FIFO. Based on the instruction type, glued instructions are sent and buffered in a second-level internal cache FIFO. This cache FIFO can accommodate up to 16 glued sequences (if more glued sequences are requested, an incorrect sequence error is generated). The instructions are processed together when the INSTR_LINK field = 0 of the last instruction in the instruction chain.

Internally, the controller translates the instructions and breaks them down into the correct sequence required by the memory device based on the type of the command. The general structure of xSPI frame is shown in the *STIG Frame Structure* diagram.

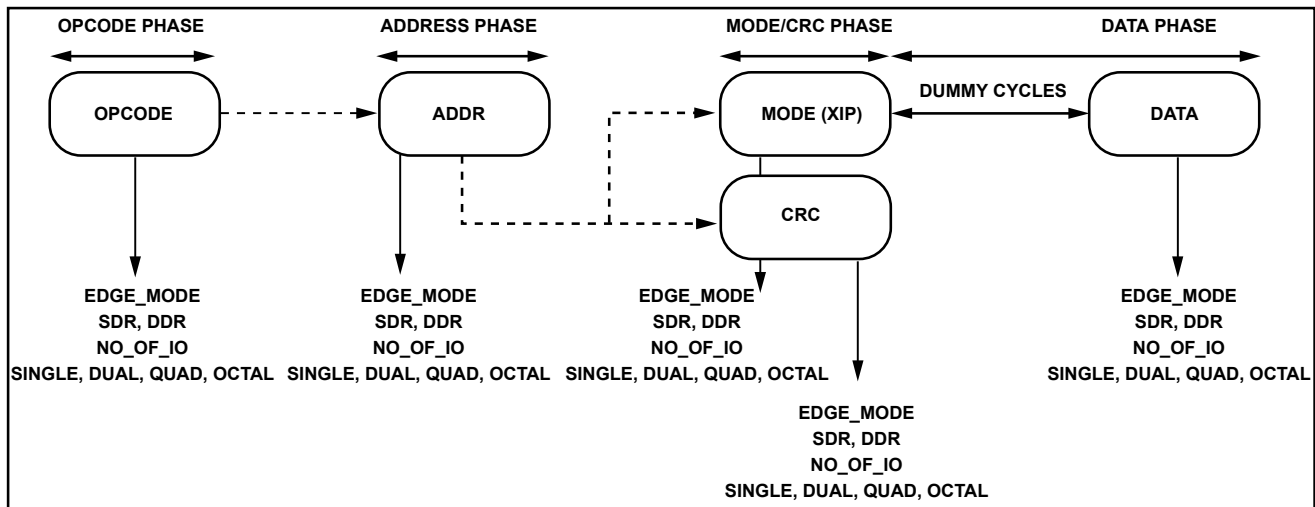


Figure 14-5: STIG Frame Structure

The blocks correspond to xSPI transfer phases. Depending on the individual command layout, some phases may be dropped; however, the order is always maintained. (Refer to JESD216 or a particular xSPI device specification). The EDGE_MODE and NO_OF_IOS can also differ in between phases. Per JESD216, only incremental increases over the number of IOs used or number of clock edges used is permitted. For example, the (opcode=1S, addr=8D, data=8D) transfer variant may be supported by current devices, but, a variant like (opcode=1S, addr=8D, data=4S) is not. JESD216 does not allow for transfers with a number of active IO lines that differs from one another more than once. Also, note that if any of the phases are configured to be 8D and the associated NO_OF_BYTES corresponds

to an odd value, the controller generates an error and does not accept the instruction. The 8D variant always requires an even number of bytes. The flexible command sequence structure of the controller allows any configuration of `EDGE_MODE` and `NO_OF_IOS` that provides a future proven solution for new variants of future revisions of JESD216.

Event Handling

The controller reports errors slightly differently depending on the selected work mode:

- For STIG or ACMD PIO work modes, the status is reported using the `XSPI_CMD_STAT` register at offset 0x44.
- For ACMD CDMA work mode, the status is reported using the status field of descriptor associated with a specified thread and the `XSPI_CMD_STAT` register.
- For direct work mode - the `DIR_*` bits in the `XSPI_ISTAT` register reflect the status.

The *Error Descriptions* table identifies the errors that can occur.

Table 14-27: Error Descriptions

Error	Description
DQS Error	This type of error is reported when the attached PHY detects a DQS underrun/overflow situation. When the DQS error is detected then data stream transferred from the xSPI flash memory can be corrupted. The host should assert <code>DLL_RST_N</code> or <code>RST_N</code> to clear DQS underrun/overflow flags in the PHY and clear all PHY read data pointers.
System Bus Error	For ACMD or STIG work modes, for the AXI slave interface, this error is reported when: 1. The host attempts to access the slave interface before it is permitted (refer to the required steps in STIG and ACMD sections of this document). 2. The host issues an unsupported burst type. The controller only supports incremental (non-wrapping) bursts. For the AXI master interface (not applicable for STIG or direct mode) the error may occur when an AXI error response is returned on <code>RRESP</code> or <code>BRESP</code>.

Table 14-27: Error Descriptions (Continued)

Error	Description
CMD Error	STIG mode: This error occurs when the specific STIG sequence as defined in this document is not followed properly. To rectify, review the sequence carefully and repeat. Direct mode: In this mode, the DIR_CMD_ERR interrupt is triggered when the host has issued an AXI transaction that cannot be executed by the controller for one of the following reasons. • The command extension is not enabled when the opcode phase reflects octal DDR mode. • The number of address bytes is not even when the address phase reflects octal DDR mode. • The address value is not even when the address phase reflects octal DDR mode. • The number of data bytes is not even when the data phase reflects octal DDR mode, unless RWDS byte masking functionality is enabled through the direct access configuration register (only available for direct work modes). • The number of bytes for the status checking sequence is not even when data phase reflects octal DDR mode. • Device and controller are configured to work in XIP mode but, the write access was made from the host. • Device and controller are configured to work in XIP mode but, the sending of mode byte (enable/disable) is disabled. • The DATA_PER_ADDR parameter is set and transfer address or transfer size is not even, unless RWDS byte masking functionality is enabled through the direct access configuration register (only available for direct work modes). • Word size (sAWSIZE or sARSize) is greater than the selected page size in profile 1 or profile 2 - HF. ACMD mode: Conditions which cause the CMD_ERROR being triggered are described in the section, Rules and Limitations.
CRC Error	When CRC checking is enabled, this error is reported when the controller has detected a CRC error.
Device Error	A device error is reported when the attached memory device returned fail after checking the status of the last erase/program/read operation. It is applicable only for ACMD and direct work modes.

Table 14-27: Error Descriptions (Continued)

Error	Description
	The status of a read operation is only checked for SPI NAND devices. Detecting an error in this case means that the device returned an uncorrectable ECC error.
ECC Error	An ECC correctable error is reported when the attached memory device returns a correctable error during a read operation and when the <code>XSPI_STAT_SEQ_CFG10.ECC_FAIL_EN</code> field is set in the register.

Device Discovery

Device discovery (DD) is used to identify the memory device and pre-configure controller registers to the required state automatically following power on reset. Automated device discovery is implemented in the controller using the SFDP as defined in JEDEC standard JESD216. The SFDP standard provides a consistent method of describing the functional and feature capabilities of serial flash devices in a standard set of parameter tables. The SFDP must be implemented in the attached memory device for discovery to be successful.

Device discovery can be triggered in two ways — automatically, following power on reset (by default, this always occurs, unless the controller input `discovery_inhibit` is set (=1), or manually using the discovery control `XSPI_DISCOVERY_GCTL` register.

Discovery Modes

Device discovery can automatically identify the memory device at power on reset and initialize many of the controller registers. However, the user can alternatively choose to trigger device discovery manually. There are three modes of operation for device discovery: full automatic, preconfigured SFDP, and preconfigured non-SFDP.

Full Automatic Mode Detection - In this default mode, the controller must first determine how to communicate with an unknown device. Per JESD216, if the memory device supports device discovery, it must also support the read SFDP command. The controller, therefore, attempts to communicate by sending read SFDP commands in different transfer formats. Each time it waits for an indication that the device has interpreted the command successfully. When a valid SFDP header is recognized, the controller knows which transfer format has been successful. It proceeds automatically to fetch and decode the SFDP database residing within the device. As the database is fetched from the device, the controller updates the controller registers accordingly.

To trigger device discovery using this mode as part of power on reset initialization, ensure inputs `XSPI_DISCOVERY_GCTL.INHIBIT = 0` and `XSPI_DISCOVERY_GCTL.NUM_LINES = 0`.

To trigger device discovery using this mode from the registers, write the following values to the `XSPI_DISCOVERY_GCTL` register.

- `XSPI_DISCOVERY_GCTL.REQ_TYP = 0`
- `XSPI_DISCOVERY_GCTL.NUM_LINES = 0`
- `XSPI_DISCOVERY_GCTL.REQ = 1`

When operating in full automatic mode detection, the initial querying of the memory device to identify the correct transfer mode is performed as part of the controller's initialization process. Once completed, the `XSPI_DISCOVERY_GCTL` register (at 0x260) is automatically updated.

Preconfigured SFDP mode - If the user already knows what memory device the controller is connected to and that SFDP is supported, the controller does not need to query the device with different transfer formats of the read SFDP command. Device discovery is triggered as in full automatic detection mode, but it bypasses the initial sequence to identify the required transfer mode. This can speed up the device identification process.

To trigger device discovery using this mode as part of power on reset initialization, ensure inputs `XSPI_DISCOVERY_GCTL.INHIBIT = 0` and `XSPI_DISCOVERY_GCTL.NUM_LINES != 0`. To trigger device discovery using this mode from registers, write the following values to the `XSPI_DISCOVERY_GCTL` register:

- `XSPI_DISCOVERY_GCTL.REQ_TYP = 0`
- `XSPI_DISCOVERY_GCTL.NUM_LINES != 0` (choose the value suited to the attached device, refer to the `XSPI_DISCOVERY_GCTL` register definition for details)
- `XSPI_DISCOVERY_GCTL.REQ = 1`

Preconfigured non-SFDP mode - In this mode, the controller does not attempt to fetch the SFDP database from the memory device. This alternative configuration method allows the controller to operate with a pre-known memory device. It is optional, and the user can alternatively program each controller register manually.

To trigger device discovery using this mode as part of power on reset initialization, ensure inputs `XSPI_DISCOVERY_GCTL.INHIBIT = 1` and `XSPI_DISCOVERY_GCTL.NUM_LINES != 0`. To trigger device discovery using this mode from the registers, write the following values to the `XSPI_DISCOVERY_GCTL` register:

- `XSPI_DISCOVERY_GCTL.REQ_TYP = 1`
- `XSPI_DISCOVERY_GCTL.NUM_LINES != 0` (choose the value suited to the attached device, refer to the `XSPI_DISCOVERY_GCTL` register definition for details)
- `XSPI_DISCOVERY_GCTL.REQ = 1`

Discovery Flow

The *Discovery Flow Diagram* shows the operations and decisions that are part of the device discovery process. If device discovery is inhibited, or no valid SFDP header or parameter page header is recognized during the flow, the SFDP database/parameter page cannot be obtained from the device and all controller registers retain the reset values.

The host must not trigger any operation while device discovery is in progress. The discovery process completion is marked by the controller setting the `XSPI_DISCOVERY_GCTL.PASS` bit. When full automatic mode is in use, discovery is complete when either the `INIT_COMP` pin is high, or `XSPI_GSTAT.INIT_PASS = 1`.

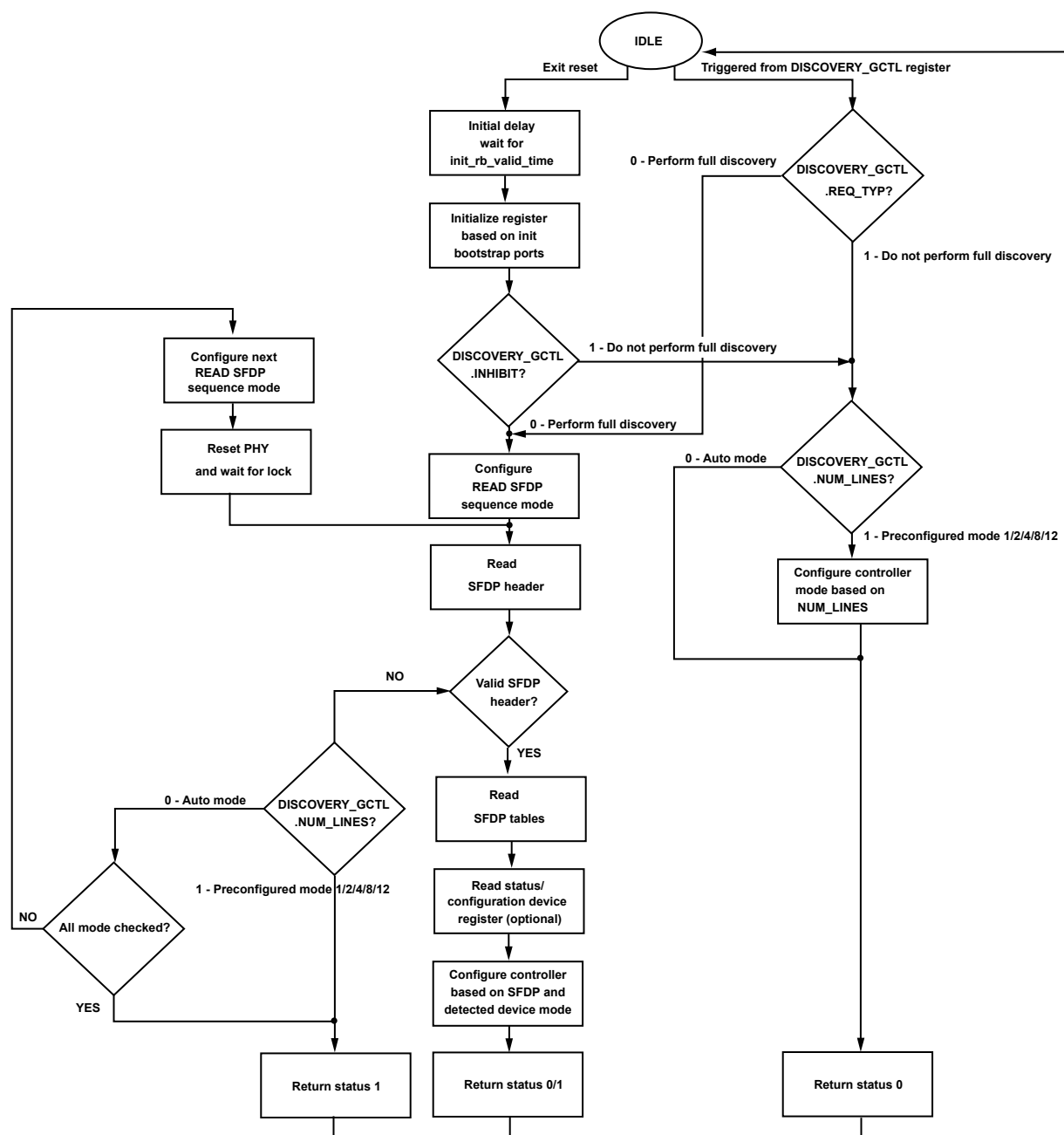


Figure 14-6: Device Discovery Flow

SFDP Command Structure

When full automatic mode detection is active, the controller must determine how to communicate with the device. It communicates by repeatedly sending read SFDP commands in different transfer formats. This mode is selected when `XSPI_DISCOVERY_GCTL.NUM_LINES` signals is configured to 4'd0. The algorithm that the controller executes differs from the suggested algorithm presented in the JESD216 specification document. The algorithm starts with single mode, goes through dual, quad, and octal modes, and finishes with the read SFDP command (compliant with legacy HyperFlash and xSPI profile 2.0).

The read SFDP commands are sent in the following order. If the device does not successfully interpret any of these steps, the device discovery is deemed to have failed.

1. 1S-1S-1S — opcode, address and data phases used for reading SFDP database are sent on one line in SDR mode. Command uses three address bytes and eight dummy clock cycles.
2. 1D-1D-1D — opcode, address and data phases used for reading SFDP database are sent on one line in DDR mode. Command uses three address bytes and eight dummy clock cycles.
3. 2S-2S-2S — opcode, address and data phases used for reading SFDP database are sent on two lines in SDR mode. Command uses three address bytes and eight dummy clock cycles.
4. 2D-2D-2D — opcode, address and data phases used for reading SFDP database are sent on two lines in DDR mode. Command uses three address bytes and eight dummy clock cycles.
5. 4S-4S-4S — opcode, address and data phases used for reading SFDP database are sent on four lines in SDR mode. Command uses three address bytes and eight dummy clock cycles.
6. 4D-4D-4D (Quad DDR* - presented as 4S-4D-4D in JESD216 specification) — opcode, address, and data phases used for reading SFDP database are sent on four lines. The opcode phase is sent in DDR mode with enabled command extension value. Address and data phases are sent in DDR mode. Command uses 3 address bytes and eight dummy clock cycles.
7. 4D-4D-4D (Quad DDR) — opcode, address and data phases used for reading SFDP database are sent on four lines in DDR mode. Command uses three address bytes and eight dummy clock cycles.
8. 8S-8S-8S — opcode, address and data phases used for reading SFDP database are sent on eight lines in SDR mode. Command uses three address bytes and eight dummy clock cycles.
9. 8S-8S-8S — opcode, address and data phases used for reading SFDP database are sent on eight lines in SDR mode. Command uses four address bytes and 20 dummy clock cycles. The opcode is sent twice and the second one is logical negation of the first one.
10. 8D-8D-8D — opcode, address and data phases used for reading SFDP database are sent on eight lines in DDR mode. Command uses four address bytes and 8 dummy clock cycles. The opcode extension is repetition of the opcode.
11. 8D-8D-8D — opcode, address and data phases used for reading SFDP database are sent on eight lines in DDR mode. Command sends the address over two clock cycles (meaning four bytes are sent as this is octal DDR), but only three address bytes are valid. The opcode extension is repetition of the opcode.
12. 8D-8D-8D — opcode, address and data phases used for reading SFDP database are sent on eight lines in DDR mode. Command uses four address bytes and 20 dummy clock cycles. The opcode extension is logical negation of the opcode.
13. 8D-8D-8D — read SFDP command for legacy HyperFlash and xSPI profile 2.0 memory devices with 15 latency clock cycles. When preconfigured mode is selected, the above step is not required.

To use preconfigured mode in device discovery, configure the parameters of the read SFDP (READ_SFDP) instructions using [XSPI_DISCOVERY_GCTL](#) register fields shown in the following table.

Table 14-28: Read SFDP Command Variations - XSPI_DISCOVERY_GCTL Register

NUM_LINES	ABNUM	DMY_CNT	CMD_TYP[1]	CMD_TYP[0]	OE_EN	OE_VAL	Comment
1	N/A	N/A	N/A	0	N/A	N/A	Read SFDP command in 1S-1S-1S mode with three address bytes and eight dummy clock cycles.
1	N/A	N/A	N/A	1	N/A	N/A	Read SFDP command in 1D-1D-1D mode with three address bytes and eight dummy clock cycles.
2	N/A	N/A	N/A	0	N/A	N/A	Read SFDP command in 2S-2S-2S mode with three address bytes and eight dummy clock cycles.
2	N/A	N/A	N/A	1	N/A	N/A	Read SFDP command in 2D-2D-2D mode with three address bytes and eight dummy clock cycles.
4	N/A	N/A	0	0	N/A	N/A	Read SFDP command in 4S-4S-4S mode with three address bytes and eight dummy clock cycles.
4	N/A	N/A	1	0	N/A	N/A	Read SFDP command in 4D-4D-4D mode (quad DDR* - presented as

Table 14-28: Read SFDP Command Variations - XSPI_DISCOVERY_GCTL Register (Continued)

NUM_LINES	ABNUM	DMY_CNT	CMD_TYP[1]	CMD_TYP[0]	OE_EN	OE_VAL	Comment
							4S-4D-4D in JESD216 specification) with three address bytes and with eight dummy clock cycles.
4	N/A	N/A	N/A	1	N/A	N/A	Read SFDP command in 4D-4D-4D mode (quad DDR) with three address bytes and with eight dummy clock cycles.
8	0/1	0/1	N/A	0	0	N/A	Read SFDP command in 8S-8S-8S mode with three or four address bytes and with eight or 20 dummy clock cycles. The op-code is sent one time.
8	0/1	0/1	N/A	0	1	1	Read SFDP command in 8S-8S-8S mode with three or four address bytes and with eight or 20 dummy clock cycles. The op-code is sent two times and the second (command extension) is a logical negation of the first one.

Table 14-28: Read SFDP Command Variations - XSPI_DISCOVERY_GCTL Register (Continued)

NUM_LINES	ABNUM	DMY_CNT	CMD_TYP[1]	CMD_TYP[0]	OE_EN	OE_VAL	Comment
8	0/1	0/1	N/A	0	1	0	Read SFDP command in 8S-8S-8S mode with three or four address bytes and with eight or 20 dummy clock cycles. The opcode is sent two times and the second (command extension) is a logical repetition of the first one.
8	0/1	0/1	N/A	1	N/A	0	Read SFDP command in 8D-8D-8D mode with four address bytes and with eight or 20 dummy clock cycles. The opcode extension is a repetition of the opcode. If three address bytes are selected, the controller sends the address over two clock cycles (meaning four bytes are sent due to octal DDR). But, only the first three address bytes are valid.
8	0/1	0/1	N/A	1	N/A	1	Read SFDP command in 8D-8D-8D mode with four address bytes and with eight

Table 14-28: Read SFDP Command Variations - XSPI_DISCOVERY_GCTL Register (Continued)

NUM_LINES	ABNUM	DMY_CNT	CMD_TYP[1]	CMD_TYP[0]	OE_EN	OE_VAL	Comment
							or 20 dummy clock cycles. The opcode extension is the logical negation of the opcode. If three address bytes are selected controller sends the address over two clock cycles (meaning four bytes are sent due to octal DDR), but only the first three address bytes are valid.

Table 14-28: Read SFDP Command Variations - XSPI_DISCOVERY_GCTL Register (Continued)

NUM_LINES	ABNUM	DMY_CNT	CMD_TYP[1]	CMD_TYP[0]	OE_EN	OE_VAL	Comment
12	hf_bound_en	latency_cnt[4]	latency_cnt[3]	latency_cnt[2]	latency_cnt[1]	latency_cnt[0]	Read SFDP command in 8D-8D-8D mode for xSPI profile 2.0 with variable number of latency clock cycles and enabled or disabled detection of the inter pages crossing latency clock cycles. This value should be configured to N-1, where N is the value expected by the memory device. If device expects 16 latency clock cycles then set 15 latency clock cycles as follow: DMY_CNT = 0 CMD_TYP = 3 OE_EN = 1 OE_VAL = 1
14	N/A	N/A	0/1	0/1	N/A	0	Read parameter page command in 1-1-1 mode for legacy SPI NAND device (at POR command is sent to device first). CMD_TYP field/port is

Table 14-28: Read SFDP Command Variations - XSPI_DISCOVERY_GCTL Register (Continued)

NUM_LINES	ABNUM	DMY_CNT	CMD_TYP[1]	CMD_TYP[0]	OE_EN	OE_VAL	Comment
							used to set XSPI_SEQ_GCTL1 . PLANE_CNT parameter value.
14	N/A	N/A	0/1	0/1	N/A	1	Read parameter page command in 1-1-1 mode for legacy SPI NAND device (at PoR command is not send to device). CMD_TYP field/port is used to set XSPI_SEQ_GCTL1 . PLANE_CNT parameter value.

SPI NAND Workflow

When connected to an SPI NAND memory device, the device discovery executes the following commands to read the parameter page database from the flash device:

1. Wait for MISCREG_XSPI_CTL1 .RB_VLD_TM- this step is done only when device discovery is enabled at PoR. The RB_VALID_TIME is taken from the MISCREG_XSPI_CTL1 32-bit controller input.
2. Send 8'hFF reset command if MISCREG_XSPI_CTL0 .EXTP_VL bootstrap port is 0, otherwise the reset command is not sent - this step is done only when device discovery is enabled at PoR.
3. Wait for the device ready state by reading the OIP bit in the flash device (get feature command 8'h0F with one address byte 8'hC0) - this step is done only when device discovery is enabled at PoR and if MISCREG_XSPI_CTL0 .EXTP_VL - bootstrap port is 1'b0.
4. Read configuration register (get feature command 8'h0F with one address byte 8'hB0).
5. Configure bit 6th in the configuration register to enter parameter page read mode (configure feature command 8'h1F with one address byte 8'hB0).
6. Send read page 13h command with the block address of 24'h0001.
7. Wait for device ready state by reading the OIP bit in the flash device (get feature command 8'h0F with one address byte 8'hC0).

8. Send the read from cache 8'h0B command with the page address of 16'd0 and eight dummy clock cycles to fetch the parameter page signature.
9. Send the read from cache 8'h0B command with the page address of 16'd80 and eight dummy clock cycles to fetch the page size, page size extension and block size.
10. Restore the value in the configuration register read in step 3 (configure feature command 8'h1F with one address byte 8'hB0).
11. If a valid parameter page header was received, then configure the controller to SPI NAND mode.
12. Return status.

Discovery Status

When device discovery is performed as part of controller initialization, the status can be interpreted by looking at the INIT_FAIL controller output as follows.

NOTE: In the *Discovery Status Parameters* table, the controller's parameters related to CRC are updated to the values from the bootstrap ports (discovery_crc_*) except where invalid values of XSPI_DISCOVERY_GCTL.NUM_LINES are configured, or where the controller failed to detect the device accurately.

Table 14-29: Discovery Status Parameters

XSPI_DISCOVERY_GCTL. INHIBIT	XSPI_DISCOVERY_GCTL. NUM_LINES[3:0]	Description (INIT_FAIL status is valid when INIT_PASS = 1)
1	0	INIT_FAIL = 2'b00 – Device discovery is disabled Note that the PHY registers are still updated separately – see the section xSPI PHY Controller .
1	1, 2, 4, 8 or 12	INIT_FAIL = 2'b00 – The SFDP database was not read. The controller was, however, configured to the selected mode.
1	14	INIT_FAIL = 2'b00 – The SFDP database was not read. The controller was, however, configured to SPI NAND mode.
1	3, 5, 6, 7, 9, 10, 11, 13 or 15	INIT_FAIL = 2'b00 – The SFDP database was not read. Invalid configuration of XSPI_DISCOVERY_GCTL.NUM_LINES. The controller registers remain unchanged.
0	0	INIT_FAIL = 2'b00 – Device discovery enabled and detected an xSPI profile 1 device or profile 2 device during auto-detection mode. INIT_FAIL = 2'b10 – Device discovery enabled and detected a legacy SPI device during auto-detection mode INIT_FAIL = 2'b01 – Device discovery enabled but did not detect a device during auto-detection mode.

Table 14-29: Discovery Status Parameters (Continued)

XSPI_DISCOVERY_GCTL. INHIBIT	XSPI_DISCOVERY_GCTL. NUM_LINES[3:0]	Description (INIT_FAIL status is valid when INIT_PASS = 1)
0	1, 2, 4, 8 or 12	INIT_FAIL = 2'b00 – Device discovery enabled and detected an xSPI profile 1 device or profile 2 device during preconfigured detection mode. INIT_FAIL = 2'b10 – Device discovery enabled and detected a legacy SPI device during pre-configured detection mode. INIT_FAIL = 2'b01 – Device discovery enabled but did not detect a device during preconfigured-detection mode.
0	14	INIT_FAIL = 2'b00 – Device discovery enabled and detected a legacy SPI NAND device during pre-configured detection mode. INIT_FAIL = 2'b01 – Device discovery enabled but did not detect a legacy SPI NAND device during preconfigureddetection mode.
0	3, 5, 6, 7, 9, 10, 11, 13 or 15	INIT_FAIL = 2'b01 – Invalid configuration on XSPI_DISCOVERY_GCTL.NUM_LINES. The controller registers remain unchanged.

When device discovery is triggered from the registers, the same status can be interpreted using the XSPI_DISCOVERY_GCTL.FAIL field of the discovery control register.

XSPI_DISCOVERY_GCTL. INHIBIT	XSPI_DISCOVERY_GCTL. NUM_LINES[3:0]	Description (XSPI_DISCOVERY_GCTL.FAIL status is valid when XSPI_DISCOVERY_GCTL.PASS = 1)
0	0	XSPI_DISCOVERY_GCTL.FAIL = 2'b00 – Device discovery enabled and detected an xSPI profile 1 device or profile 2 device during auto-detection mode. XSPI_DISCOVERY_GCTL.FAIL = 2'b10 – Device discovery enabled and detected a legacy SPI device during auto-detection mode. XSPI_DISCOVERY_GCTL.FAIL = 2'b01 – Device discovery enabled but did not detect a device during auto-detection mode.

XSPI_DISCOVERY_GCTL. INHIBIT	XSPI_DISCOVERY_GCTL. NUM_LINES[3:0]	Description (XSPI_DISCOVERY_GCTL.FAIL status is valid when XSPI_DISCOVERY_GCTL.PASS = 1)
0	1, 2, 4, 8 or 12	XSPI_DISCOVERY_GCTL.FAIL = 2'b00 – Device discovery enabled and detected an xSPI profile 1 device or profile 2 device during preconfigured detection mode. XSPI_DISCOVERY_GCTL.FAIL = 2'b10 – Device discovery enabled and detected a legacy SPI device during pre-configured detection mode. XSPI_DISCOVERY_GCTL.FAIL = 2'b01 – Device discovery enabled but did not detect device during preconfigured detection mode. Presents how to configure device discovery module in pre-configured detection mode.
0	14	XSPI_DISCOVERY_GCTL.FAIL = 2'b00 – Device discovery enabled and detected a legacy SPI NAND device during pre-configured detection mode. XSPI_DISCOVERY_GCTL.FAIL = 2'b01 – Device discovery enabled but did not detect a legacy SPI NAND device during preconfigured detection mode.
0	3, 5, 6, 7, 9, 10, 11, 13 or 15	XSPI_DISCOVERY_GCTL.FAIL = 2'b01 – Invalid configuration of XSPI_DISCOVERY_GCTL.NUM_LINES. The controller registers remain unchanged.
1	1, 2, 4, 8 or 12	XSPI_DISCOVERY_GCTL.FAIL = 2'b00 – The SFDP database was not read. The controller is configured to the selected mode.
1	14	XSPI_DISCOVERY_GCTL.FAIL = 2'b00 – The SFDP database was not read. The controller is configured to SPI NAND mode.

Discovery Restrictions

As defined in the JESD216 standard, flash devices can implement various parameter tables. But, only the JEDEC basic flash parameter table is mandatory. The following restrictions apply to xSPI profile 1 and profile 2 memory devices with a SFDP revision number is greater than or equal to 16'h0107:

- For xSPI profile 1 memory devices that are configured to any mode other than single, the SFDP database must implement the SCCR (status, control and configuration register) map for the xSPI memory devices parameter table. This implementation allows the controller to update its registers accurately. If the device does not implement the map fully, some of the sequence-related registers of the controller can get commonly used values instead.

- For xSPI profile 2 memory device, the SFDP database must implement the SCCR map for xSPI profile 2 memory devices parameter table. This implementation allows the controller to update its registers accurately. If the device does not implement the map fully, some of the sequence-related registers of the controller can get commonly used values instead.

Discovery Limitations

Note the following device discovery limitations:

- The device discovery module must not be used to detect a device that is configured to operate in XIP mode.
- The device discovery module must not be used to detect the HyperRAM device. Only profile 1, profile 2 - HF and SPI NAND devices are supported for device discovery.

Legacy Devices

When the number of dummy clock cycles for a read command (READ_SEQ_PL_DUMMY_CNT) parameter cannot be read from the device status/configuration register, the controller configures a default value for single, dual, quad modes or 16/20 for octal modes. The device discovery module ends with a 2'b10 status, but, with the wrong number of dummy clock cycles. In a similar way, if the number of dummy clock cycles for a status checking command (READ_SEQ_PL_DEV_RDY_DUMMY_CNT) cannot be read from the from the SFDP database, the device discovery module configures a default value. See the section [Controller Reconfiguration](#).

Device Specific Configuration

Macronix MX25 octal devices - To configure the controller to STR-OPI mode without trying to read SFDP database, device discovery must be run in not full discovery pre-configuration mode:

- `XSPI_DISCOVERY_GCTL.INHIBIT = 1` or `XSPI_DISCOVERY_GCTL.REQ_TYP = 1`
- `XSPI_DISCOVERY_GCTL.NUM_LINES = 8`
- `XSPI_DISCOVERY_GCTL.ABNUM = 1`
- `XSPI_DISCOVERY_GCTL.DMY_CNT = 1`
- `XSPI_DISCOVERY_GCTL.CMD_TYP = 0`
- `XSPI_DISCOVERY_GCTL.OE_EN = 1`
- `XSPI_DISCOVERY_GCTL.OE_VAL = 1.`

To configure the controller to DTR-OPI mode without trying to read SFDP database, device discovery must be run in not full discovery pre-configuration mode:

- `XSPI_DISCOVERY_GCTL.INHIBIT = 1` or `XSPI_DISCOVERY_GCTL.REQ_TYP = 1`
- `XSPI_DISCOVERY_GCTL.NUM_LINES = 8`
- `XSPI_DISCOVERY_GCTL.ABNUM = 1`
- `XSPI_DISCOVERY_GCTL.DMY_CNT = 1`

- `XSPI_DISCOVERY_GCTL.CMD_TYP = 1`
- `XSPI_DISCOVERY_GCTL.OE_EN = 0 or 1`
- `XSPI_DISCOVERY_GCTL.OE_VAL = 1`

Controller Reconfiguration

When device discovery completes successfully, the controller automatically updates various registers in the controller. The registers depend on the type of memory device attached. For profile 1 or SPI NAND devices, the registers used to define how the command (opcode), address, and data are transferred between the controller and device. The *Device Discovery Parameter* tables show the parameters in the device sequence registers that require updates.

Table 14-30: Device Discovery Parameters - Profile 1 or SPI NAND devices

Detected/Selected Mode	Command Phase (*cmd_ios, *cmd_edge, *cmd_ext_en, *cmd_ext_val)	Address Phase (*addr_ios, *addr_edge,*addr_cnt)	Data Phase (*data_ios, *data_edge)
Single SDR	0,0, 0, I	0*,0, 3(4)**	0*,0
Single DDR	0,1, 0, I	0*,1, 3(4)**	0*,1
Dual SDR	1,0, 0, I	1,0, 3(4)**	1,0
Dual	1,1, 0, I	1,1, 3(4)**	1,1
Quad SDR	2,0, 0, I	2,0, 3(4)**	2,0
Quad DDR	2,1, 0, I	2,1, 3(4)**	2,1
Quad DDR	2,1, 1, N	2,1, 3(4)**	2,1
Octal SDR without command extension	3,0, 0, I	3,0, 3(4)**	3,0
Octal SDR with command extension repeated	3,0, 1, R	3,0, 3(4)**	3,0
Octal SDR with command extension inverted	3, 0, 1, I	3, 0, 3(4)**	3, 0
Octal DDR with command extension repeated	3, 1, 1, R	3, 1, 4**	3, 1
Octal DDR with command extension inverted	3, 1, 1, I	3, 1, 4**	3, 1
SPI NAND	0, 0, 0, R	0, 0, 2	0, 0

NOTE: • * = single SDR/DDR mode, the number of IOs for address and data phases (for read and program commands) can, in many devices, be reprogrammed higher than what is shown in the table; but, the lower value is chosen as a result of device discovery.

- ** = the number of address bytes for read/program/erase sector commands is configured to four when:
 - the device operates with four address bytes, or
 - when SFDP indicates support for four address byte commands, or
 - when octal DDR mode is detected (in which case only an even number of address bytes can be transferred during the address phase).
 - In all other cases, the device discovery module configures the number of address bytes based on the number of address bytes that is used to detect the device.
- R = the command extension is repeated
- I = the command extension is inverted
- N = command value consists of the first half of the opcode (bits[7:4]) repeated twice, and command extension value consists of second half of the opcode (bits[3:0]) repeated twice. For example, if the attached memory requires 0x12 opcode for a program page operation, then PROG_SEQ_PL_CMD_VAL is configured to 8'h11 and PROG_SEQ_PL_CMD_EXT_VAL is configured to 8'h22. This mode is presented as 4S-4D-4D in the JESD216 specification.

Common Sequence Register Updates

For profile 1, profile 2 (HF) or SPI NAND devices, the *Device Discovery Parameters* table shows the common sequence parameters in the `XSPI_SEQ_GCTL0` and `XSPI_SEQ_GCTL1` registers. The registers are updated after discovery.

Table 14-31: Device Discovery Parameters - Profile 1, Profile 2(HF) or SPI NAND devices

Parameter Name	Full Discovery Process	Not Full Discovery Process
TYP	• 0 when a legacy SPI or xSPI profile 1 device is detected • 1 when a legacy HyperFlash or xSPI profile 2 device is detected • 3 when a legacy SPI NAND device is detected.	• 0 for legacy SPI or xSPI profile 1 configuration • 1 for legacy HyperFlash or xSPI profile 2 configuration • 3 for SPI NAND configuration
DATA_PER_ADDR	• 0 when legacy SPI, xSPI profile 1 and SPI NAND devices are detected • 1 when a legacy HyperFlash or xSPI profile 2 device is detected	• 0 for legacy SPI, xSPI profile 1, or SPI NAND configuration • 1 for legacy HyperFlash or xSPI profile 2 configuration

Table 14-31: Device Discovery Parameters - Profile 1, Profile 2(HF) or SPI NAND devices (Continued)

Parameter Name	Full Discovery Process	Not Full Discovery Process
PSIZ_PGM	Value is read from the SFDP database or parameter page	4'd8 for legacy SPI or xSPI profile 1 configuration 4'd9 for legacy HyperFlash or xSPI profile 2 configuration 4'd11 for SPI NAND configuration if XSPI_DISCOVERY_GCTL.ABNUM bootstrap port/register field is 1'b0. Otherwise, 4'd12.
PSIZ_RD	For SPI NAND devices, the value is read from the parameter page. Otherwise, 4'd15 value is configured.	4'd15 for profile 1 and profile 2 - HF 4'd11 for SPI NAND configuration if XSPI_DISCOVERY_GCTL.ABNUM bootstrap port/register field is 1'b0. Otherwise, 4'd12.
DATA_SWAP	1 when the SFDP header is swapped in octal DDR mode. Otherwise, 0.	1 for legacy SPI or xSPI profile 1 when octal DDR mode is configured, number of address bytes is 4, and number of dummy cycles is 20 1 for legacy HyperFlash or xSPI profile 2 configuration. Otherwise, 0.
CRC_EN	Parameter is updated from bootstrap port signal DISCOVERY_SEQ_CRC_EN after controller initialization process only	Parameter is updated from bootstrap port signal DISCOVERY_SEQ_CRC_EN after controller initialization process only
CRC_VAR	Parameter is updated from bootstrap port signal DISCOVERY_SEQ_CRC_VARIANT after controller initialization process only	Parameter is updated from bootstrap port signal DISCOVERY_SEQ_CRC_VARIANT after controller initialization process only
CRC_OE	Parameter is updated from bootstrap port signal DISCOVERY_SEQ_CRC_CHUNK after controller initialization process only	Parameter is updated from bootstrap port signal DISCOVERY_SEQ_CRC_OE after controller initialization process only
CHUNK_SIZ	Parameter is updated from bootstrap port signal DISCOVERY_SEQ_CRC_CHUNK_SIZE after controller initialization process only	Parameter is updated from bootstrap port signal DISCOVERY_SEQ_CRC_CHUNK_SIZE after controller initialization process only
UAL_CHUNK_EN	Parameter is updated from bootstrap port signal DISCOVERY_SEQ_CRC_UAL_CHUNK_EN after controller initialization process only	Parameter is updated from bootstrap port signal DISCOVERY_SEQ_CRC_UAL_CHUNK_EN after controller initialization process only
UAL_CHUNK_CHK	Parameter is updated from bootstrap port signal DISCOVERY_SEQ_CRC_UAL_CHUNK_CHK after controller initialization process only	Parameter is updated from bootstrap port signal DISCOVERY_SEQ_CRC_UAL_CHUNK_CHK after controller initialization process only

Table 14-31: Device Discovery Parameters - Profile 1, Profile 2(HF) or SPI NAND devices (Continued)

Parameter Name	Full Discovery Process	Not Full Discovery Process
PSIZ_EXT	For SPI NAND devices, the value is read from the parameter page	64 value is configured if <code>XSPI_DISCOVERY_GCTL.DMY_CNT = 1'b0</code> . Otherwise, 128.
PCA_SIZ	For SPI NAND devices when page size equals 4KB, 1'b1 value is set. Otherwise, 1'b0 value is configured.	Value is configured to the value of <code>XSPI_DISCOVERY_GCTL.ABNUM</code> field/ bootstrap port
PLANE_CNT	Value is configured to the value of <code>XSPI_DISCOVERY_GCTL.CMD_TYP</code> register field/ bootstrap port	Value is configured to the value of <code>XSPI_DISCOVERY_GCTL.CMD_TYP</code> register field/ bootstrap port
PPER_BLOCK	Value is read from the parameter page	6 (64 pages per block) value is configured
TCMS_EN	1'd0 value is configured	1'd0 value is configured

Reset Sequence Parameters for Profile 1 Devices

For profile 1 devices, the reset sequence registers are used when a reset command is issued to the memory device. The parameters are identified in the `XSPI_RST_SEQ_CFG0` and `XSPI_RST_SEQ_CFG1` registers. The registers are updated as shown in the *Reset Sequence Parameters - Profile 1 Devices* table.

Table 14-32: Reset Sequence Parameters - Profile 1 Devices

Parameter Name	Full Discovery Process	Not Full Discovery Process
<code>XSPI_RST_SEQ_CFG0.CMD0_EN</code>	0 when the device supports the soft reset command - 0xF0	Always 1
<code>XSPI_RST_SEQ_CFG0.CMD0_EN</code>	Always 8'h66 command is configured	Always 8'h66 command is configured
<code>XSPI_RST_SEQ_CFG0.CMD1_VALUE</code>	When the device supports soft reset command, 8'hF0. Otherwise, 8'h99 command is configured.	Always 8'h99 command is configured
<code>XSPI_RST_SEQ_CFG1.P1_DAT_VALUE</code>	Always 8'hD0 value is configured	Always 8'hD0 value is configured
<code>XSPI_RST_SEQ_CFG0.DAT_EN</code>	Always 0	Always 0
<code>XSPI_ERS_SEQ_CFG0.CMD_VALUE</code>	Read from the SFDP database	8'h20 command when <code>XSPI_DISCOVERY_GCTL.ABNUM = 0</code> , 8'h21 command when <code>XSPI_DISCOVERY_GCTL.ABNUM = 1</code>
<code>XSPI_ERS_SEQ_CFG1.P1_SECT_SIZ_VALUE</code>	Read from the SFDP database	Always 4'd12 (4kB)
<code>XSPI_ERS_SEQ_CFG2.ERSA_P1_CMD_VALUE</code>	Always 8'hC7 command is configured	Always 8'hC7 command is configured

Erase Sequence Register Updates

For profile 1 devices, the erase sequence registers are used when an erase_sector or erase_all command is issued to the memory device. The parameters are identified in the [XSPI_ERASE_SEQ_CFG0](#), [XSPI_ERASE_SEQ_CFG1](#), and [XSPI_ERASE_SEQ_CFG2](#) registers. The registers are updated as shown in the *Erase_Sector / Erase_All Sequence Parameters* table.

Table 14-33: Erase_Sector / Erase_All Sequence Parameters - Profile 1 Devices

Parameter Name	Full Discovery Process	Not Full Discovery Process
XSPI_ERASE_SEQ_CFG0.CMD_VALUE	8'hD8 command is configured	8'hD8 command is configured
XSPI_ERASE_SEQ_CFG1.P1_SECT_SIZ_VALUE	Calculated as follow: 2^N , where N=column address size + pages row address size	5'd18 value is configured
XSPI_ERASE_SEQ_CFG2.ERASE_P1_CMD_VALUE	Always 8'hC7 command is configured	Always 8'hC7 command is configured

For SPI NAND devices, the erase sequence registers are used when an erase_sector command is issued to the memory device. The parameters are identified in the [XSPI_ERASE_SEQ_CFG0](#) and [XSPI_ERASE_SEQ_CFG1](#) registers. The registers are updated as shown in the *Erase_Sector Sequence Parameters* table.

Table 14-34: Erase_Sector Sequence Parameters - SPI NAND Devices

Parameter Name	Full Discovery Process	Not Full Discovery Process
XSPI_ERASE_SEQ_CFG0.CMD_VALUE	8'hD8 command is configured	8'hD8 command is configured
XSPI_ERASE_SEQ_CFG1.P1_SECT_SIZ_VALUE	Calculated as follow: 2^N , where N = column address size + pages row address size	5'd18 value is configured
XSPI_ERASE_SEQ_CFG2.ERASE_P1_CMD_VALUE	Always 8'hC7 command is configured	Always 8'hC7 command is configured

Write Enable Sequence Register Updates

For profile 1 or SPI NAND devices, the write enable sequence parameters are identified in the [XSPI_WE_SEQ_CFG0](#) register. The registers are used when the controller needs to prepare the device for a program operation. The *Write Enable Sequence Parameters* table shows the parameters for full and not full discovery.

Table 14-35: Write Enable Sequence Parameters - Profile 1 and SPI NAND devices

Parameter Name	Full Discovery Process	Not Full Discovery Process
XSPI_WE_SEQ_CFG0.P1_CMD_VALUE	Always 8'h06 command is configured	Always 8'h06 command is configured
XSPI_WE_SEQ_CFG0.P1_EN	Always 1'b1 value is set	Always 1'b1 value is set

Program Sequence Register Updates

For profile 1, profile 2 HyperFlash, or SPI NAND devices, the program sequence parameters are identified in the [XSPI_PROG_SEQ_CFG0](#), [XSPI_PROG_SEQ_CFG1](#), and [XSPI_PROG_SEQ_CFG2](#) registers. The registers

used when the controller needs to perform a program (write) operation. The parameters are shown in the *Program Sequence Parameters* tables.

Table 14-36: Program Sequence Parameters - Profile 1

Detected/Selected Mode	Parameter Name	Full Discovery Process	Not Full Discovery Process
Single	XSPI_PROG_SEQ_CFG0.P1_CMD_VALUE	8'h8E command is configured (1-8-8 with 4 address bytes) when the SFDP database indicates support for this command by device 8'h84 command is configured (1-1-8 with 4 address bytes) when SFDP database indicates support for this command by device 8'h3E command is configured (1-4-4 with 4 address bytes) when SFDP database indicates support for command by device 8'h34 command is configured (1-1-4 with 4 address bytes) when SFDP database indicates support for this command by device 8'h12 command is configured (1-1-1 with 4 address bytes) when SFDP database indicates support for this command by device Otherwise, 8'h02 command is configured (1-1-1 with 3 or 4 address bytes)	8'h02 command is configured when XSPI_DISCOVERY_GCTL.ABNUM = 0 8'h12 command is configured when XSPI_DISCOVERY_GCTL.ABNUM = 1
Single		6'd0 value is configured	6'd0 value is configured
Dual, Quad, Octal	XSPI_PROG_SEQ_CFG0.P1_CMD_VALUE	8'h12 command is configured when the SFDP database indicates support for this command by device Otherwise, 8'h02 command is configured	8'h02 command is configured when XSPI_DISCOVERY_GCTL.ABNUM = 0 8'h12 command is configured when XSPI_DISCOVERY_GCTL.ABNUM = 1
Dual, Quad, Octal	XSPI_PROG_SEQ_CFG0.P1_DMY_CNT	6'd0 value is configured	6'd0 value is configured

Table 14-37: Program Sequence Parameters - Profile 2 HF Devices

Parameter Name	Full Discovery Process	Not Full Discovery Process
XSPI_PROG_SEQ_CFG2.P2_MSK_CMD_MOD	Always 0 is configured	Always 0 is configured
XSPI_PROG_SEQ_CFG2.P2_BURST_TYP	Always 1 (linear burst) is set	Always 1 (linear burst) is set
XSPI_PROG_SEQ_CFG2.P2_TARGET	Always 0 (memory space) is configured	Always 0 (memory space) is configured

Table 14-37: Program Sequence Parameters - Profile 2 HF Devices (Continued)

Parameter Name	Full Discovery Process	Not Full Discovery Process
XSPI_PROG_SEQ_CFG2.P2_LATENCY_CNT	Parameter is not updated after discovery procedure since it is related with profile 2 - HR which is not supported by the device discovery module	Parameter is not updated after discovery procedure since it is related with profile 2 - HR which is not supported by the device discovery module

Table 14-38: Program Sequence Parameters - SPI NAND Devices

Parameter Name	Parameter Value
XSPI_PROG_SEQ_CFG0.P1_CMD_VALUE	8'h02 command is configured

Read Sequence Register Updates

For profile 1, profile HyperFlash, or SPI NAND devices, read sequence registers are identified in [XSPI_READ_SEQ_CFG0](#), [XSPI_READ_SEQ_CFG1](#) and [XSPI_READ_SEQ_CFG2](#) registers. The registers used when the controller needs to perform a read operation. The parameters are shown in the *Read Sequence Parameters* table.

Table 14-39: Read Sequence Parameters - Profile 1 Devices

Detected / Selected Mode	Parameter Name	Full Discovery Process	Not Full Discovery Process
Single	P1_CMD_VALUE	When the SFDP database indicates supporting the command by device: 8'hCC command is configured (1-8-8 with 4 address bytes) 8'h7C command is configured (1-1-8 with 4 address bytes) 8'hEC command is configured (1-4-4 with 4 address bytes) 8'h6C command is configured (1-1-4 with 4 address bytes) 8'hBC command is configured (1-2-2 with 4 address bytes) 8'h3C command is configured (1-1-2 with 4 address bytes) 8'h0C command is configured (1-1-1 with 4 address bytes) Read command value for 1-8-8 mode with 3 or 4 address bytes is read from SFDP database and configured Read command value for 1-1-8 mode with 3 or 4 address bytes is read from SFDP database and configured Read command value for 1-4-4 mode with 3 or 4 address bytes is read from SFDP database and configured Read command value for 1-1-4 mode with 3 or 4 address bytes is read from SFDP database and configured Read command value for 1-2-2 mode with 3 or 4 address bytes is read from SFDP database and configured Read command value for 1-1-2 mode with 3 or 4 address bytes is read from SFDP database and configured Otherwise, 8'h0B command value (1-1-1 with 3 or 4 address bytes)	8'h03 command is configured when XSPI_DISCOVERY_GCTL.ABNUM = 0 and XSPI_DISCOVERY_GCTL.DMY_CNT = 0 8'h13 command is configured when XSPI_DISCOVERY_GCTL.ABNUM = 1 and XSPI_DISCOVERY_GCTL.DMY_CNT = 0 8'h0B command is configured when XSPI_DISCOVERY_GCTL.ABNUM = 0 and XSPI_DISCOVERY_GCTL.DMY_CNT = 1 8'h0C command is configured when XSPI_DISCOVERY_GCTL.ABNUM = 1 and XSPI_DISCOVERY_GCTL.DMY_CNT = 1
Single	P1_DMY_CNT	Number of dummy clock cycles is read from the SFDP database. For 8'0B and 8'h0C read commands 8 number of dummy cycles are set.	0 dummy clock cycles are configured when XSPI_DISCOVERY_GCTL.DMY_CNT = 0, 8 dummy clock cycles are configured when XSPI_DISCOVERY_GCTL.DMY_CNT = 1.

Table 14-39: Read Sequence Parameters - Profile 1 Devices (Continued)

Detected / Selected Mode	Parameter Name	Full Discovery Process	Not Full Discovery Process
Dual	P1_CMD_VALUE	Read command is read from the SFDP database.	8'h0B command is configured when XSPI_DISCOVERY_GCTL.ABNUM = 0 8'h0C command is configured when XSPI_DISCOVERY_GCTL.ABNUM = 1
Dual	P1_DMY_CNT	If possible, number of dummy clock cycles is read from the memory configuration/status register. Otherwise, the number of dummy clock cycles is read from the SFDP database.	8 dummy clock cycles are configured when XSPI_DISCOVERY_GCTL.DMY_CNT = 0 10 dummy clock cycles are configured when XSPI_DISCOVERY_GCTL.DMY_CNT = 1
Quad	P1_CMD_VALUE	Read command is read from the SFDP database.	8'h0B command is configured when XSPI_DISCOVERY_GCTL.ABNUM= 0, 8'h0C command is configured when XSPI_DISCOVERY_GCTL.ABNUM = 1.
Quad	P1_DMY_CNT	If possible, the number of dummy clock cycles is read from the memory configuration/status register. Otherwise, the number of dummy clock cycles is read from the SFDP database.	8 dummy clock cycles are configured when XSPI_DISCOVERY_GCTL.DMY_CNT = 0
Octal	P1_CMD_VALUE	8'h0C command is configured when the SFDP database indicates supporting this command by device. Otherwise, 8'h0B command is configured	8'hEC command is configured when XSPI_DISCOVERY_GCTL.ABNUM= 1 XSPI_DISCOVERY_GCTL.DMY_CNT = 1, XSPI_DISCOVERY_GCTL.CMD_TYP[0] = 0 and XSPI_DISCOVERY_GCTL.OE_EN = 1 8'h EE command is configured when XSPI_DISCOVERY_GCTL.ABNUM = 1, XSPI_DISCOVERY_GCTL.DMY_CNT = 1 XSPI_DISCOVERY_GCTL.CMD_TYP[0] = 1 8'h 0B command is configured when XSPI_DISCOVERY_GCTL.ABNUM = 0 8'h0C command is configured when XSPI_DISCOVERY_GCTL.ABNUM= 1

Table 14-39: Read Sequence Parameters - Profile 1 Devices (Continued)

Detected / Selected Mode	Parameter Name	Full Discovery Process	Not Full Discovery Process
Octal	P1_DMY_CNT	If possible, number of dummy clock cycles is read from the memory configuration/status register. Otherwise, 16 dummy clock cycles are configured when 8 dummy clock cycles have been used for device detection, or, 20 dummy clock cycles are configured when 20 dummy clock cycles have been used for device detection	16 dummy clock cycles are configured when <code>XSPI_DISCOVERY_GCTL.DMY_CNT = 0</code> 20 dummy clock cycles is configured when <code>XSPI_DISCOVERY_GCTL.DMY_CNT = 1</code>

Table 14-40: Read Sequence Parameters - Profile 1 Devices (mode byte)

Parameter Name	Full Discovery Process	Not Full Discovery Process
<code>XSPI_READ_SEQ_CFG1.P1_MB_DMY_CNT</code>	If possible, (number of clock cycles to send the mode byte (<code>mb_factor</code>) is not greater than <code>XSPI_READ_SEQ_CFG1.P1_MB_DMY_CNT</code>), the value equals the <code>P1_MB_DMY_CNT - mb_factor</code>. Otherwise, 0 is configured. $mb_factor = 2 * (3 - P1_ADDR_IOS - P1_ADDR_EDGE)$	Value is computed in the same way as for full discovery process
<code>XSPI_READ_SEQ_CFG1.P1_MB_EN</code>	If the basic flash parameter table indicates support for mode byte for the selected read command, then 1. Otherwise, 0.	Always 1'b0
<code>XSPI_XIP_GCTL.XIP_DIS_MB_VALUE</code>	Always 8'hFF value	Always 8'hFF value

*When the address phase is in octal DDR mode, then `mb_factor = 1`.

Table 14-41: Read Sequence Parameters - Profile 2 HF Devices

Parameter Name	Full Discovery Process	Not Full Discovery Process
<code>XSPI_READ_SEQ_CFG2.P2_LATENCY_CNT</code>	If possible, the number of latency clock cycles is read from the device status/configuration register. Otherwise, the number of latency clock cycles is configured to the number that is used to read the SFDP database.	Number of latency clock cycles is configured to number that is used to read the SFDP database

Table 14-41: Read Sequence Parameters - Profile 2 HF Devices (Continued)

Parameter Name	Full Discovery Process	Not Full Discovery Process
XSPI_READ_SEQ_CFG2.P2_HF_BOUND_EN	Value is equal to the XSPI_DISCOVERY_GCTL.ABNUM for preconfigured mode, or, 1 for auto-detection mode	Value is equal to the XSPI_DISCOVERY_GCTL.ABNUM
XSPI_READ_SEQ_CFG2.P2_MSK_CMD_MOD	Always 0 is configured	Always 0 is configured
XSPI_READ_SEQ_CFG2.P2_BURST_TYP	Always 1 (linear burst) is configured	Always 1 (linear burst) is configured
XSPI_READ_SEQ_CFG2.P2_TARGET	Always 0 (memory space) is configured	Always 0 (memory space) is configured

Table 14-42: Read Sequence Parameters - SPI NAND Devices

Parameter Name	Parameter Value
XSPI_READ_SEQ_CFG0.P1_CMD_VALUE	8'h0B command is configured
XSPI_READ_SEQ_CFG0.P1_DMY_CNT	8 dummy clock cycles are configured

Table 14-43: Read Sequence Parameters for SPI-NAND Devices (mode byte)

Parameter Name	Parameter Value
XSPI_READ_SEQ_CFG1.P1_MB_EN	0
XSPI_READ_SEQ_CFG1.P1_MB_DMY_CNT	0 dummy clock cycles are configured
XSPI_XIP_GCTL.XIP_DIS_MB_VALUE	Always 8'hFF value
XSPI_READ_SEQ_CFG1.P1_CACHE_RANDOM_READ_EN	0

Status Sequence Register Updates

For profile 1, profile HyperFlash, or NAND devices, the status sequence registers are identified in the XSPI_STAT_SEQ_CFG register. The registers are used when the controller needs to check the status of the memory device. The parameters are shown in the *Status Sequence Parameters* table.

Table 14-44: Status Sequence Parameters - Profile 1 Devices

Parameter Name	Full Discovery Process	Not Full Discovery Process
XSPI_ERS_SEQ_CFG0.ADDR_CNT	When possible, the number of address bytes is read from the SFDP database. Otherwise, 2 address bytes are configured for octal DDR mode and 1 address byte is configured.	Four address bytes are configured when XSPI_DISCOVERY_GCTL.ABNUM= 1 and XSPI_DISCOVERY_GCTL.DMY_CNT=1
XSPI_STAT_SEQ_CFG1.P1_ERS_FAIL_ADDR_EN	Address phase is enabled when the SFDP database indicates a requirement of an address phase	Address phase is always disabled

Table 14-44: Status Sequence Parameters - Profile 1 Devices (Continued)

Parameter Name	Full Discovery Process	Not Full Discovery Process
	for reading the erase error bit. Otherwise, the address phase is disabled.	
XSPI_STAT_SEQ_CFG1.P1_ERS_FAIL_DMY_CNT	Number of dummy clock cycles is read from the SFDP database.	8 dummy clock cycles are configured for octal modes. Otherwise, 1 dummy clock cycle is configured when the data phase is in DDR mode and 0 for other cases.
XSPI_STAT_SEQ_CFG2.P1_ERS_FAIL_CMD_VALUE	Command is read from the SFDP database when it indicates support for an erase error bit. Otherwise, the 8'h05 command is configured.	8'h05 command is configured
XSPI_STAT_SEQ_CFG1.P1_PROG_FAIL_ADDR_EN	Address phase is enabled when the SFDP database indicates the requirement of an address phase for reading the program error bit. Otherwise, the address phase is disabled.	Address phase is always disabled
XSPI_STAT_SEQ_CFG1.P1_PROG_FAIL_DMY_CNT	Number of dummy clock cycles is read from the SFDP database	8 dummy clock cycles are configured for octal modes. Otherwise, 1 dummy clock cycle is configured when the data phase is in DDR mode and 0 for other cases.
XSPI_STAT_SEQ_CFG2.P1_PROG_FAIL_CMD_VALUE	Command is read from the SFDP database when it indicates support of the program error bit. Otherwise, the 8'h05 command is configured.	8'h05 command is configured
XSPI_STAT_SEQ_CFG1.P1_DEV_RDY_ADDR_EN	Address phase is enabled when the SFDP database indicates the requirement of an address phase for reading the RDY/ BUSY bit. Otherwise, the address phase is disabled.	Address phase is enabled when XSPI_DISCOVERY_GCTL.ABNUM = 1, XSPI_DISCOVERY_GCTL.DMY_CNT = 1, XSPI_DISCOVERY_GCTL.CMD_TYP[0] = 0, and XSPI_DISCOVERY_GCTL.OE_EN = 1
XSPI_STAT_SEQ_CFG1.P1_ERS_FAIL_ADDR_EN	Address phase is enabled when the SFDP database indicates the requirement of an address phase for reading the erase error bit. Otherwise, the address phase is disabled.	Address phase is always disabled
XSPI_STAT_SEQ_CFG1.P1_ERS_FAIL_DMY_CNT	Number of dummy clock cycles is read from the SFDP database	8 dummy clock cycles are configured for octal modes. Otherwise, 1 dummy clock cycle is configured when the data phase is in DDR mode and 0 for other cases.
XSPI_STAT_SEQ_CFG2.P1_ERS_FAIL_CMD_VALUE	Command is read from the SFDP database when it indicates support for the erase error bit. Otherwise, the 8'h05 command is configured.	8'h05 command is configured
XSPI_STAT_SEQ_CFG1.P1_PROG_FAIL_ADDR_EN	Address phase is enabled when the SFDP database indicates the requirement of an address phase for reading the program error bit. Otherwise, the address phase is disabled.	Address phase is always disabled

Table 14-44: Status Sequence Parameters - Profile 1 Devices (Continued)

Parameter Name	Full Discovery Process	Not Full Discovery Process
<code>XSPI_STAT_SEQ_CFG1.P1_PROG_FAIL_DMY_CNT</code>	Number of dummy clock cycles is read from the SFDP database	8 dummy clock cycles are configured for octal modes. Otherwise, 1 dummy clock cycle is configured when the data phase is in DDR mode and 0 for other cases.
<code>XSPI_STAT_SEQ_CFG2.P1_PROG_FAIL_CMD_VALUE</code>	Command is read from the SFDP database when it indicates support for the program error bit. Otherwise, the 8'h05 command is configured.	8'h05 command is configured
<code>XSPI_STAT_SEQ_CFG1.P1_DEV_RDY_ADDR_EN</code>	Address phase is enabled when the SFDP database indicates the requirement of an address phase for reading the RDY/ BUSY bit. Otherwise, the address phase is disabled.	Address phase is enabled when <code>XSPI_DISCOVERY_GCTL.ABNUM= 1</code> , <code>XSPI_DISCOVERY_GCTL.DMY_CNT = 1</code> , <code>XSPI_DISCOVERY_GCTL.CMD_TYP = 0</code> and <code>XSPI_DISCOVERY_GCTL.OE_EN = 1</code> or when <code>XSPI_DISCOVERY_GCTL.ABNUM= 1</code> , <code>XSPI_DISCOVERY_GCTL.DMY_CNT=1</code> and <code>XSPI_DISCOVERY_GCTL.CMD_TYP[0] = 1</code> Otherwise, the address phase is disabled.
<code>XSPI_STAT_SEQ_CFG1.P1_DEV_RDY_DMY_CNT</code>	Number of dummy clock cycles is read from the SFDP database.	4 dummy clock cycles are set if <code>XSPI_DISCOVERY_GCTL.ABNUM= 1</code> , <code>XSPI_DISCOVERY_GCTL.DMY_CNT= 1</code> , <code>XSPI_DISCOVERY_GCTL.CMD_TYP = 0</code> and <code>XSPI_DISCOVERY_GCTL.OE_EN</code> is 1 or if <code>XSPI_DISCOVERY_GCTL.ABNUM= 1</code> , <code>XSPI_DISCOVERY_GCTL.DMY_CNT= 1</code> and <code>XSPI_DISCOVERY_GCTL.CMD_TYP[0] = 1</code> else 8 dummy clock cycles are configured for other octal modes Otherwise, 1 dummy clock cycles is configured when data phase in DDR mode and 0 in other all other modes
<code>XSPI_STAT_SEQ_CFG2.P1_DEV_RDY_CMD_VALUE</code>	Command is read from the SFDP database when it supports the RDY/BUSY bit. Otherwise, the 8'h05 command is configured.	8'h05 command is configured

Table 14-45: Status Sequence Parameters - Profile 2 HF Devices

Parameter Name	Parameter Value	Not Full Discovery Process
XSPI_STAT_SEQ_CFG4.P2_LATENCY_CNT	When possible, the number of latency clock cycles is read from the device status/configuration register. Otherwise, the number of latency clock cycles is configured to a number that is used to read the SFDP database.	Number of latency clock cycles is configured to a number that is used to read the SFDP database
XSPI_STAT_SEQ_CFG4.P2_MSK_CMD_MOD	Always 0	Always 0

Table 14-46: Status Sequence Parameters

Parameter Name	Parameter Value	Not Full Discovery Process
XSPI_STAT_SEQ_CFG5.PROG_FAIL_EN	1 when the SFDP database indicates support for the program error bit. Otherwise, 0.	Always 0
XSPI_STAT_SEQ_CFG5.PROG_FAIL_SIZ	1 when the data phase reflects octal DDR mode. Otherwise, 0.	1 when the data phase reflects octal DDR mode. Otherwise, 0.
XSPI_STAT_SEQ_CFG5.PROG_FAIL_VALUE	Value is read from the SFDP database when it indicates support for the program error bit. Otherwise, 0.	Always 0
XSPI_STAT_SEQ_CFG5.PROG_FAIL_IDX	Value is read from the SFDP database when it indicates support for the program error bit. Otherwise, 0.	Always 0
XSPI_STAT_SEQ_CFG8.PROG_FAIL_ADDR_VALUE	Value is read from the SFDP database when it indicates support for the program error bit. Otherwise, 0x00000000 address is configured.	Always 0x00000000 address is configured.
XSPI_STAT_SEQ_CFG5.ERS_FAIL_EN	1 when the SFDP database indicates support for the program error bit. Otherwise, 0.	Always 0
XSPI_STAT_SEQ_CFG5.ERS_FAIL_SIZ	1 when the data phase reflects octal DDR mode. Otherwise, 0.	1 when the data phase reflects octal DDR mode. Otherwise, 0.
XSPI_STAT_SEQ_CFG5.ERS_FAIL_VALUE	Value is read from the SFDP database when it indicates support for the erase error bit. Otherwise, 0.	Always 0
XSPI_STAT_SEQ_CFG5.ERS_FAIL_IDX	Value is read from the SFDP database when it indicates support for the erase error bit. Otherwise, 0.	Always 0
XSPI_STAT_SEQ_CFG9.ERS_FAIL_ADDR_VALUE	Value is read from the SFDP database if it indicates support for the erase error bit, Otherwise, 0x00000000 address is configured.	Always 0x00000000 address is configured
XSPI_STAT_SEQ_CFG5.DEV_RDY_EN	Always 1	Always 1

Table 14-46: Status Sequence Parameters (Continued)

Parameter Name	Parameter Value	Not Full Discovery Process
XSPI_STAT_SEQ_CFG5.DEV_RDY_SIZ	1 when the data phase reflects octal DDR mode. Otherwise, 0.	1 when the data phase reflects octal DDR mode. Otherwise, 0.
XSPI_STAT_SEQ_CFG5.DEV_RDY_VALUE	Value is read from the SFDP database if it indicates support for the RDY/BUSY bit, Otherwise, 0 for legacy SPI and xSPI profile 1 or 1 for legacy HyperFlash and xSPI profile 2.	0 for legacy SPI and xSPI profile 1 1 for legacy HyperFlash and xSPI profile 2
XSPI_STAT_SEQ_CFG5.DEV_RDY_IDX	Value is read from the SFDP database if it indicates support for the RDY/BUSY bit, Otherwise, 0 for legacy SPI and xSPI profile 1 or 7 for legacy HyperFlash and xSPI profile 2.	0 for legacy SPI and xSPI profile 1 7 for legacy HyperFlash and xSPI profile 2
XSPI_STAT_SEQ_CFG7.DEV_RDY_ADDR_VALUE	Value is read from the SFDP database if it indicates support for the RDY/BUSY bit, Otherwise, 0x00000000 address is configured.	Always 0x00000000 address is configured.

The following status sequence parameters are updated in the XSPI_STAT_SEQ_CFGn registers for SPI NAND devices following device discovery.

Table 14-47: Status Sequence Parameters for SPI NAND Devices

Parameter Name	Parameter Value
XSPI_ERS_SEQ_CFG0.ADDR_CNT	0 (one address byte)
XSPI_STAT_SEQ_CFG1.P1_ERS_FAIL_ADDR_EN	1
XSPI_STAT_SEQ_CFG1.P1_ERS_FAIL_DMY_CNT	0 dummy clock cycles are configured
XSPI_STAT_SEQ_CFG2.P1_ERS_FAIL_CMD_VALUE	8'h0F command is configured
XSPI_STAT_SEQ_CFG1.P1_PROG_FAIL_ADDR_EN	1
XSPI_STAT_SEQ_CFG1.P1_PROG_FAIL_DMY_CNT	0 dummy clock cycles are configured
XSPI_STAT_SEQ_CFG2.P1_PROG_FAIL_CMD_VALUE	8'h0F command is configured
XSPI_STAT_SEQ_CFG1.P1_DEV_RDY_ADDR_EN	1
XSPI_STAT_SEQ_CFG1.P1_DEV_RDY_DMY_CNT	0 dummy clock cycles are configured
XSPI_STAT_SEQ_CFG2.P1_DEV_RDY_CMD_VALUE	8'h0F command is configured
XSPI_STAT_SEQ_CFG5.PROG_FAIL_SIZ	0
XSPI_STAT_SEQ_CFG5.PROG_FAIL_VALUE	1
XSPI_STAT_SEQ_CFG5.ERS_FAIL_IDX	3
XSPI_STAT_SEQ_CFG8.PROG_FAIL_ADDR_VALUE	32'h00000C0 value is configured

Table 14-47: Status Sequence Parameters for SPI NAND Devices (Continued)

Parameter Name	Parameter Value
XSPI_STAT_SEQ_CFG5.ERS_FAIL_EN	1
XSPI_STAT_SEQ_CFG5.ERS_FAIL_SIZ	0
XSPI_STAT_SEQ_CFG5.ERS_FAIL_VALUE	1
XSPI_STAT_SEQ_CFG5.ERS_FAIL_IDX	2.
XSPI_STAT_SEQ_CFG9.ERS_FAIL_ADDR_VALUE	32'h00000C0 value is configured
XSPI_STAT_SEQ_CFG5.DEV_RDY_EN	1
XSPI_STAT_SEQ_CFG5.DEV_RDY_SIZ	0
XSPI_STAT_SEQ_CFG5.DEV_RDY_VALUE	0
XSPI_STAT_SEQ_CFG5.DEV_RDY_IDX	0
XSPI_STAT_SEQ_CFG7.DEV_RDY_ADDR_VALUE	32'h00000C0 value is configured
XSPI_STAT_SEQ_CFG10.CRDY_VALUE	0
XSPI_STAT_SEQ_CFG10.CRDY_IDX	7
XSPI_STAT_SEQ_CFG10.ECC_FAIL_EN	0
XSPI_STAT_SEQ_CFG10.ECC_FAIL_VALUE	0
XSPI_STAT_SEQ_CFG10.ECC_FAIL_MSK	0

Boot Engine

xSPI controller provides an option to boot directly from the attached memory device using direct mode following controller reset. Alternatively, the controller can use a dedicated boot engine to transfer the boot code from the memory device to an area of host memory. The boot engine automatically triggers a page read command and probes the status of the transfer.

The boot engine assumes that the boot image is stored in the xSPI device connected to bank 0. The boot image is stored in the xSPI memory. It is composed of a configuration record and main data. The configuration record contains the following information:

- Main data size — the overall data size that need to be transferred during boot process
- Host address — the address on the host side where the boot image is transferred
- Image offset — the address inside the xSPI device from where the boot image is transferred
- SPI NAND page size — For NAND devices, this provides the SPI NAND device page size that is used to determine number of read iterations

After the boot process completes, the boot engine module writes the boot operation status. Only a single boot image copy is stored in the xSPI memory. This kind of memory is less prone to errors. So, no additional copies are required.

The figure shows the boot engine module functional diagram.

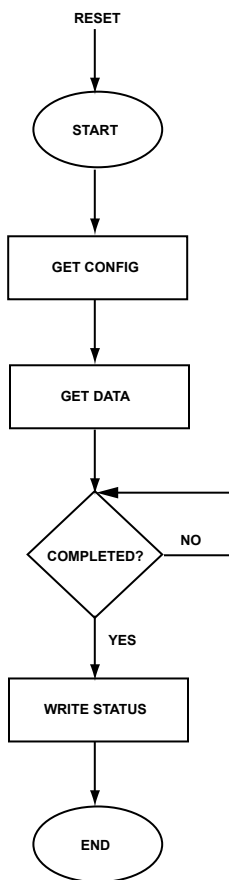


Figure 14-7: Boot Engine Flow

Configuring the Boot Engine

The boot engine is enabled by writing 1 to `MISCREG_XSPI_CTL0.BT_EN` field before doing a controller reset. Once reset is done, the controller fetches a 256-bit configuration record stored in the memory device. This record should have been previously stored in the memory device from address 0x0. The Boot Engine Configuration Record Layout figure provides the details of the required layout of the configuration record.

- Main data size — exists at memory device address 0x00. The lower 32 bits contain the size of the boot record information, the higher 32 bits are reserved.
- Image offset — exists at memory device address 0x08. The lower 48 bits contain the starting address of the boot image within the memory device. The higher 16 bits are reserved.
- Host address — exists at memory device address 0x10. The whole 64 bits contain the host address indicating where the boot image is copied.
- SPI NAND page size — exists at memory device address 0x18. It provides SPI NAND device page size which is used to determine the number of read iterations. Information is stored in the first 16 bits. The upper 48 bits are reserved.

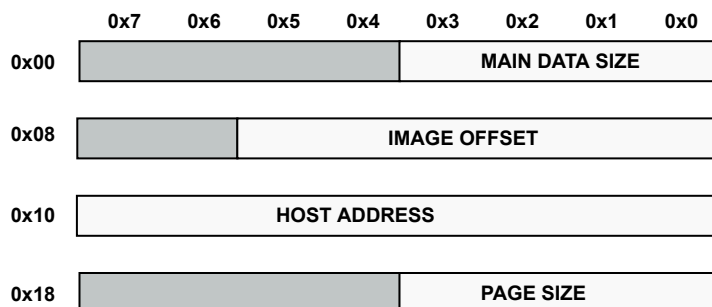


Figure 14-8: Boot Engine Configuration Record Layout

The register fields table shows the bits used by the boot engine to select valid commands and manage data transfer between the xSPI device and host memory. Register content is established by the device discovery process or register field reset vectors. The boot engine assumes that the information in these control registers are valid when boot process is triggered.

The following configuration register fields are used in the boot process:

Table 14-48: Registers for Boot

Register	Bit
XSPI_SEQ_GCTL0	- XSPI_SEQ_GCTL0.TYP - XSPI_SEQ_GCTL0.DAT_SWAP - XSPI_SEQ_GCTL0.UAL_CHUNK_CHK - XSPI_SEQ_GCTL0.UAL_CHUNK_EN - XSPI_SEQ_GCTL0.CHUNK_SIZ - XSPI_SEQ_GCTL0.CRC_OE - XSPI_SEQ_GCTL0.CRC_VAR - XSPI_SEQ_GCTL0.CRC_EN - XSPI_SEQ_GCTL0.DAT_PER_ADDR
XSPI_XIP_GCTL	- XSPI_XIP_GCTL.XIP_DIS_MB_VALUE
XSPI_READ_SEQ_CFG0	- XSPI_READ_SEQ_CFG0.P1_DMY_CNT - XSPI_READ_SEQ_CFG0.P1_DAT_EDGE - XSPI_READ_SEQ_CFG0.P1_DAT_IOS - XSPI_READ_SEQ_CFG0.P1_ADDR_EDGE - XSPI_READ_SEQ_CFG0.P1_ADDR_IOS - XSPI_READ_SEQ_CFG0.P1_ADDR_CNT - XSPI_READ_SEQ_CFG0.P1_CMD_EDGE - XSPI_READ_SEQ_CFG0.P1_CMD_IOS - XSPI_READ_SEQ_CFG0.P1_CMD_VALUE

Table 14-48: Registers for Boot (Continued)

Register	Bit
XSPI_READ_SEQ_CFG1	-XSPI_READ_SEQ_CFG1.P1_CMD_EXT_EN -XSPI_READ_SEQ_CFG1.P1_CMD_EXT_VALUE -XSPI_READ_SEQ_CFG1.P1_MB_EN -XSPI_READ_SEQ_CFG1.P1_MB_DMY_CNT
XSPI_READ_SEQ_CFG2	-XSPI_READ_SEQ_CFG2.P2_LATENCY_CNT -XSPI_READ_SEQ_CFG2.P2_HF_BOUND_EN -XSPI_READ_SEQ_CFG2.P2_MSK_CMD_MOD -XSPI_READ_SEQ_CFG2.P2_BURST_TYP -XSPI_READ_SEQ_CFG2.P2_TARGET
XSPI_STAT_SEQ_CFG0 (only for SPI NAND device)	-XSPI_STAT_SEQ_CFG0.P1_DAT_EDGE -XSPI_STAT_SEQ_CFG0.P1_DAT_IOS -XSPI_STAT_SEQ_CFG0.P1_CMD_EXT_EN -XSPI_STAT_SEQ_CFG0.P1_CMD_EDGE -XSPI_STAT_SEQ_CFG0.P1_CMD_IOS -XSPI_STAT_SEQ_CFG0.P1_ADDR_EDGE -XSPI_STAT_SEQ_CFG0.P1_ADDR_IOS -XSPI_STAT_SEQ_CFG0.P1_ADDR_CNT
XSPI_STAT_SEQ_CFG1 (only for SPI NAND device)	-XSPI_STAT_SEQ_CFG1.P1_DEV_RDY_ADDR_EN -XSPI_STAT_SEQ_CFG1.P1_DEV_RDY_DMY_CNT
XSPI_STAT_SEQ_CFG2 (only for SPI NAND device)	-XSPI_STAT_SEQ_CFG2.P1_DEV_RDY_CMD_VALUE
XSPI_STAT_SEQ_CFG3 (only for SPI NAND device)	-XSPI_STAT_SEQ_CFG3.P1_DEV_RDY_CMD_EXT_VALUE
XSPI_STAT_SEQ_CFG5 (only for SPI NAND device)	-XSPI_STAT_SEQ_CFG5.DEV_RDY_SIZ -XSPI_STAT_SEQ_CFG5.DEV_RDY_VALUE -XSPI_STAT_SEQ_CFG5.DEV_RDY_IDX
XSPI_STAT_SEQ_CFG7 (only for SPI NAND device)	-XSPI_STAT_SEQ_CFG7.DEV_RDY_ADDR_VALUE

xSPI PHY Controller

The PHY handles the low-level timings of the data, address, and control signals between the device and controller. The PHY has its own programmable register group.

PHY Initialization

Before the host can communicate with the memory device, software must initialize the PHY and lock it. The soft PHY contains three synthesizable delay lines (DLLs) that are built from 256 delay elements. The first delay line is

the controller delay line and it measures how many delay elements are needed to match the `XSPI_CLK` clock period. The other two delay lines are targets, one for the transmission TX outbound path and the other for the reception RX inbound path.

The TX target delay line can use the controller delay line measurements to influence how much delay to add to transmitted data based on a percentage of the clock period. This improves the eye diagram (a common indicator of the quality of signals in high-speed digital transmissions) of the memory device. Similarly, the RX target delay line can shift the read sampling clock as a percentage of the clock period to find the ideal sampling point for the inbound read data. Before this can happen however, the controller delay line must be configured and this is done immediately after power on reset (PoR). This is PHY locking, a fully internal PHY function that attempts to lock the master delay line. Until the PHY is locked, there can be no access to or from the memory device.

When choosing to disable the controller delay line (that is, saturation mode), the target delay line cannot be programmed based on a percentage of the clock period and instead must be manually configured. This is suitable for when operating the PHY at slower data rates, when the controller delay line does not have enough delay elements to measure the complete clock period. Running at these slower rates does not require fine tuning of the target delay lines (when running at SDR) and manual configured is acceptable.

NOTE: PHY locking should not be confused with PHY training. PHY training is a software application (not part of the controller deliverables) that can be implemented to help tune the PHY to find the pbest sampling point and allow the highest possible `XSPI_CLK` frequency. The controller does not implement PHY training.

To lock the controller delay line and have the general PHY ready to transfer data, some of the PHY registers must be programmed with appropriate values that are dependent on the `XSPI_CLK` frequency. The PHY runs at the same frequency as the attached memory device clock and uses system parameters such as board skew, pad delays, and so forth. Failure to program suitable values can result in the PHY inability to lock and unable to properly wake up.

Initial PHY Registers to Configure

The PHY registers that must be initially configured before attempting to lock the PHY are described in the following sections:

- [XSPI_PHY_DLL_CTL Register](#)
- [XSPI_PHY_DQ_TR Register](#)
- [XSPI_PHY_DQS_TR Register](#)
- [XSPI_PHY_GATE_LPBK_CTL Register](#)
- [XSPI_PHY_DLL_MSTR_CTL Register](#)
- [XSPI_PHY_DLL_SLAVE_CTL Register](#)

XSPI_PHY_DLL_CTL Register

This register generally controls how the PHY target delay lines are routinely updated to compensate for drift caused by dynamic factors such as temperature or open-circuit voltage (OCV). It also contains register settings that are dependent on the attached memory device and operating mode.

The `XSPI_PHY_DLL_CTL.SDR_EDGE_ACT[21:21]` bit controls on which clock edge the PHY controller transmits write data. It is only required for SDR operation and when the PHY controller defaults to propagating write data on the clock rising edge. Generally keep at the default state of zero, unless the memory device (in SDR operation) is unable to sample the data.

The `XSPI_PHY_DLL_CTL.DQS_LST_DAT_DRP_EN[20:20]` bit must only be set to one, when all of the following conditions are met:

- The attached memory device issues read data on the negative clock edge.
- The memory device supports DQS (transfer two data words per clock cycle) and the PHY has been set up to sample read data using that DQS signal. The DQS that the memory device sends contains one extra edge than normal and the PHY must compensate for the added edge.

XSPI_PHY_DQ_TR Register

This register controls the timing of the main data (DQ) bus as it switches from the PHY to the memory device. Adjustments can be made to control when the attached DQ pads switch direction (that is, to add delay when the output enable switches from input to output and from output to input). There is also an option to define additional latency on the data bus itself, in units of one-half clock cycle.

NOTE: The `XSPI_PHY_DQ_TR.DAT_OE_END` field (which defaults to 0x2 and controls when the output enable switches from output to input) must always be configured to a minimum value of 0x1. This register does not influence the locking procedure.

XSPI_PHY_DQS_TR Register

This register determines the read data sampling method used by the PHY. See [PHY Read Sampling Method Selection](#) for more details.

CAUTION: This register must be updated to reflect the read data sampling method to successfully read any data from the device.

XSPI_PHY_GATE_LPBK_CTL Register

This register controls the gate for read data sampling to occur and enables the loopback BIST test modes within the PHY. There are fields that must be set up prior to locking. When the register fields are not mentioned in this section, the field can be left in its default state.

- `XSPI_PHY_GATE_LPBK_CTL.GATE_CFG` – Use when the PHY is not using DQS from the device to sample read data. (Determined by the read data sampling method described in [PHY Read Sampling Method Selection](#).) PHY uses information from the controller to determine which cycle the read data should be returning from the device, and when it should be sampling the DQ bus. When the round-trip delay of the entire read operation (based on `WRITE_CLK_PAD` delay + PCB trace to device + PCB trace from device + `DQ_PAD` delay) exceeds one clock cycle, then the PHY must also delay the cycle in which it samples that data.

When DQS from the device is not used to sample read data, the

`XSPI_PHY_GATE_LPBK_CTL.GATE_CFG` field = `floor` (round-trip delay in ns / clock period in ns)

- $\text{XSPI_PHY_GATE_LPBK_CTL.RD_DEL_SEL} = \text{ceiling}(\text{round-trip delay in ns} / \text{clock period in ns}) + 2$

XSPI_PHY_DLL_MSTR_CTL Register

CAUTION: With a small value, the initial lock time is longer. Values less than 0x04 may cause no lock by the delay line.

Use these register fields to configure and direct the controller delay line, prior to the PHY locking.

- `XSPI_PHY_DLL_MSTR_CTL.DLL_BYPASS_MODE` – When set (=1) (default), it disables the controller delay line and the PHY always locks immediately. This forces saturation mode. When the user wants the PHY to operate in saturation mode, this field should be set and the PHY requires no further effort to lock.
- `XSPI_PHY_DLL_MSTR_CTL.PHASE_DT_SEL` – This defaults to zero, but Analog Devices recommends it be configured to 0x2.
- `XSPI_PHY_DLL_MSTR_CTL.DLL_START_PT` – The controller delay line can take some time to measure the clock period and then signal that the lock was achieved. It must start from a fixed number of delay elements and either add or remove delay elements until it finds the appropriate delay matching the clock period. Configuring this field can speed up the locking time. This field should be configured such that it is not greater than $7/8^{\text{th}}$ of the clock period (use the worst-case element delay).

Example: When the clock frequency is 200 MHz (5 ns cycle time) and a worst-case element delay of 80 ps, this field equals $[5 * (7/8) / .080 = 54]$ elements. When the locking time is not a concern, a small value such as 0x04 can be used instead to ensure that the delay line does not lock on a harmonic.

XSPI_PHY_DLL_SLAVE_CTL Register

This register controls the TX and RX target delay lines. When the PHY is operating in saturation mode, the number of delay elements can be manually programmed for both delay lines. Otherwise, each delay line can be precisely delayed by steps ($1/256^{\text{th}}$ of the clock period). The `XSPI_PHY_DLL_SLAVE_CTL` register does not need to be updated to initially lock the PHY, but it must be given appropriate values for the memory device to be able to sample the data sent by the PHY, and for the PHY to sample the data sent by the device.

On the transmit side, the DQ line can be delayed using the TX target delay line. On the receive side, the sampling clock (which depends on the chosen read data sampling method described in [PHY Read Sampling Method Selection](#)) is delayed by the RX target delay line. Choose initial values for these fields—setup times, skew between DQS and the data, board delays, and so forth—all affect where the eye diagram begins.

Initial values for the RX target delay line can be calculated using the steps found in [Initial Value Selection for XSPI_PHY_DLL_SLAVE_CTL Register](#). The delivered script automatically calculates the required values, but these are for additional guidance.

IMPORTANT: Performing a full training sequence is the only way to calculate the most optimum delay value to find the center of the eye diagram.

Initial Value Selection for XSPI_PHY_DLL_SLAVE_CTL Register

1. Express the `quarter_cycle_period` in picoseconds. For example, when the clock period is 10 ns, configure this field equal to 2500 ps.
2. For the `delay_of_each_element` variable, calculate the expected delay through each implemented PHY delay element in ps. The delay elements within the PHY are made up of two NAND2 gates as selected in the hand instantiated cells. For example, when the delay through each NAND2 is 12 ps, the delay element = 24 ps.
3. Determine the `max_dqs_dq_skew` (in ns) maximum skew between the DQS strobe and the DQ data obtained from the memory device datasheet `tdqsq` value.
4. The `max_dqs_dq_skew_perclk` skew as a percentage of the clock equals $(\text{max_dqs_dq_skew}) / \text{clock_period}$.
5. The `rx_setup` time in ns is the time the device takes to drive the read data following the edge of the clock. This time is specified in the memory device datasheet. Note there are specific times for SDR and for each edge to valid data in DDR. Pick the largest value (worst case).
6. Calculate the `rx_setup_gt_half` value when `rx_setup` is greater than one-half the clock cycle. Determine the difference between these two values in nanoseconds. Formula: $\text{rx_setup_gt_half} = \text{rx_setup} - (\text{clock_period} / 2)$.
7. Calculate the `rx_setup_gt_full` value when `rx_setup` + 4 is greater than a clock cycle. Determine the difference between these two values in nanoseconds. Formula: $\text{rx_setup_gt_full} = \text{rx_setup} + 4 - \text{clock_period}$.
8. Calculate the `tx_setup_de` value. Let `tx_setup` be the setup time in ps, that the target memory device requires before it samples write data. This is often referred to as `tdvch` in the device data sheet. Divide the `tx_setup` time by the delay of each element to get the number of delay elements needed to cover the set up.
9. Calculate the `tx_skew_margin_de` value for the TX skew margin. Let `quarter_cycle_de` be the `quarter_cycle_period` variable divided by the delay of each element to get the number of delay elements needed to cover the 1/4 of the clock period. Divide again by five to get the number of delay elements needed to cover 1/20th of the clock period. This value is chosen for the TX skew margin.

Value Selection for RX Target Delay Lines

Use the *RX Target Delay Line Parameters* table to configure initial values for the RX target delay line.

Table 14-49: RX Target Delay Line Parameters

Parameters	Initial Value to Store in RX Delay Line Field
Saturation Mode = 1 Using DQS from device	$\text{ceiling}((\text{max_dqs_dq_skew} * 1000) / \text{delay_of_each_element}) + 1$

Table 14-49: RX Target Delay Line Parameters (Continued)

Parameters	Initial Value to Store in RX Delay Line Field
Saturation Mode = 1 Using Internal or External Loopback DQS $rx_setup \leq \text{half clock period}$	1
Saturation Mode = 1 Using Internal or External Loopback DQS $rx_setup > \text{half clock period}$ $rx_setup + 4 \leq \text{clock period}$	Setup time is quite large relative to the clock period. Need to delay initial sample point accordingly. $\text{ceiling}((rx_setup_gt_half * 1000) / \text{delay_of_each_element}) + 1$
Saturation Mode = 1 Using Internal or External Loopback DQS $rx_setup + 4 > \text{clock period}$	Setup time is very large relative to the clock period. For SDR modes, Analog Devices recommends setting SDR_EDGE_ACT to force the capture edge to be negative. $\text{ceiling}((rx_setup_gt_full * 1000) / \text{delay_of_each_element}) + 1$
Saturation Mode = 0 Using DQS from device $\text{max_dqs_dq_skew_perclk} > 1$	255
Saturation Mode = 0 Using DQS from device $\text{max_dqs_dq_skew_perclk} \leq 1$	$\text{max_dqs_dq_skew_perclk} * 256 + 0.01 * 256$
Saturation Mode = 0 Using Internal or External Loopback DQS $rx_setup \leq \text{half clock period}$	0
Saturation Mode = 0 Using Internal or External Loopback DQS $rx_setup > \text{half clock period}$ $rx_setup + 4 \leq \text{clock period}$	$\text{ceiling}(256 * (rx_setup_gt_half / \text{clk_period})) + 25$; Adding 25 extra elements to guarantee minimal skew.
Saturation Mode = 0 Using Internal or External Loopback $rx_setup + 4 > \text{clock period}$	$\text{ceiling}(256 * (rx_setup_gt_full / \text{clk_period})) + 25$; Adding 25 extra elements to guarantee minimal skew.

CAUTION: The maximum value that can be programmed for the controller delay line is 255. When any calculation exceeds 255, configure the value to 255.

Value Selection for TX Target Delay Lines

Use the *TX Target Delay Line Parameters* table to configure initial values for the TX target delay line.

Table 14-50: TX Target Delay Line Parameters

Parameters	Initial Value to Store in TX Delay Line Field
Saturation Mode = 1 flash setup is greater than quarter of cycle - skew margin ($\text{tx_setup_de} > (\text{quarter_cycle_de} - \text{tx_skew_margin_de})$)	0x5
Saturation Mode = 1 flash setup is less than or equal to quarter of cycle - skew margin ($\text{tx_setup_de} \leq (\text{quarter_cycle_de} - \text{tx_skew_margin_de})$)	Set to 25% That is, 25% ceiling(quarter_cycle_de)
Saturation Mode = 0 flash setup is greater than quarter of a cycle ($\text{tx_setup_de} > \text{quarter_cycle_de}$)	0x15
Saturation Mode = 0 flash setup is less than or equal to quarter of cycle ($\text{tx_setup_de} = \text{quarter_cycle_de}$)	0x33

CAUTION: The maximum value that can be programmed for the controller delay line is 255. When any calculation exceeds 255, configure the value to 255.

PHY Re-initialization

Once the host is operational, it can update the PHY registers and trigger a re-initialization or locking procedure using bits 24 and 25 of the `XSPI_PHY_DLL_CTL` register. A re-initialization must be performed whenever the PHY registers are modified or when the user has changed the `XSPI_CLK` frequency.

Resynchronization of Target Delay Lines

While the PHY is running, the target delay lines can still occasionally require resynchronization as a result of temperature changes, OCV, and how much that change alters the delay through the delay elements. The frequency at which this resync needs to happen depends on how much change those dynamic factors can make on the delay through the delay line and whether that change is sufficient that the sampling point is no longer within the valid data window.

There is no need to resync when the `DLL_LOCK_VALUE` has not changed or has only changed by a small amount. Analog Devices recommends that software monitors the `XSPI_PHY_DLLOB0.DLL_LOCK_VALUE` field and triggers a resync when the `XSPI_PHY_DLLOB0.DLL_LOCK_VALUE` has changed by a significant amount since the last resync (for example, 10%). Alternatively, a user can choose to resync periodically, often enough to mitigate these changes in delay, but which does not impact overall performance.

During a resynchronization, the memory device cannot be accessed. For automatic recurring delay line resynchronizations, configure the `XSPI_PHY_DLL_UPDT_CNT.RESYNC_CNT` field to a value other than 0x0. The value defines the time interval between each resync in `xspi_clk` cycles.

CAUTION: The memory interface must be idle during resync operation.

When a resync happens while the controller is communicating with the memory device, the resync is automatically withheld until the memory operation completes. Similarly, when a resync is already in progress, the controller withholds any normal operation triggered by the host until that resync completes. The `XSPI_PHY_DLL_CTL.RESYNC_IDLE_CNT` field defines the wait time after a resync operation completes when normal traffic can resume.

PHY Read Sampling Method Selection

To use the PHY, decide which read sampling mechanism to use. Essentially, there are two options the user can select from when using the PHY:

- [DQS Mode for PHY Read Sampling](#)
- [Non-DQS Mode \(Loopback\) for PHY Read Sampling](#)

DQS Mode for PHY Read Sampling

The highest device clock frequencies and, therefore, performance is achieved when the memory device drives DQS. DQS is an independent signal that is driven parallel to the read data and acts as a source synchronous clock from the device to the PHY. The PHY takes the DQS from the device, passes it through the receive delay line (for programmable incremental delay steps) and uses the resulting signal as a strobe to sample incoming read data. To support this option, the memory device being targeted must support DQS. Additionally, a physical hardware DQS pin connection must be provided. Ensure that bit 20 of the `XSPI_PHY_DQS_TR` register is cleared (=0).

Non-DQS Mode (Loopback) for PHY Read Sampling

The memory device sometimes does not supply DQS or lacks DQS hardware support on board. The alternative option is to mimic the data sampling strobe, referred to as the DQS loopback options of the PHY. Generally, this mode is referred as on-DQS (loopback) mode.

Non-DQS Loopback Mode With a Pad

Internal loopback through the `pad_mem_rebar_t pad`. This still requires the integrator to implement the `pad_mem_rebar_t pad`, although it is not bonded out to a pin. This option can be chosen when dynamic factors like OCV are not important (perhaps because the operating frequency is slower or only SDR operation is needed). This loopback function is performed within the pad and includes pad delays. Depending on the chosen pads, pad delays can be significant, so including these in the loopback path can be important. To support this option, the user must implement the `pad_mem_rebar_t pad`, but it is not connected to anything on the board. Bits 20 and 21 of the `XSPI_PHY_DQS_TR` register must be set (=1). Bit 22 of the `XSPI_PHY_DQS_TR` register must be cleared (=0).

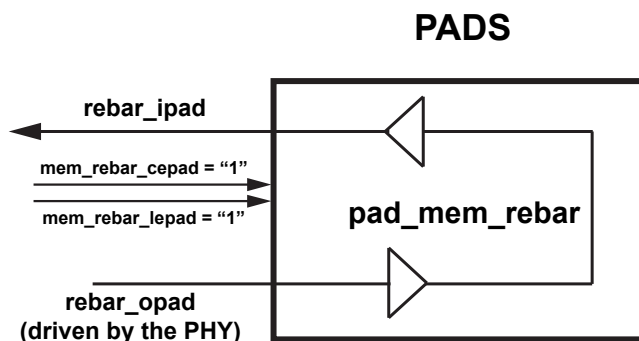


Figure 14-9: Logic Diagram for Non-DQS (Loopback) Mode With Pad

Memory Device Data Integrity

Some memory devices support advanced data integrity checking functions (for example, Macronix MX25). The controller is compatible with the following device capabilities.

ECC Data Integrity

Some memory devices have built-in Error Checking and Correcting (ECC). During a page program operation, the device automatically creates the ECC. During a read operation, the device checks and when necessary, automatically corrects errors. These devices can output an ECC status signal (ECS#) to signal the ECC correction status. The status is optionally passed to the controller using the `XSPI_DFI_GP_OPEN_DRAIN` input, which can be visible with the interrupt status register.

CRC Parity Check

Some memory devices have built-in cyclic redundancy check (CRC) capabilities. For write operations, the controller must send the CRC code to the device for checking. For read operations, the device must send the CRC code and the code must be validated using the controller.

The controller can generate the CRC on the address and calculate it with an exclusive-OR of the address bytes. This is sent to the device immediately after the address bytes. The controller can also generate a CRC on a chunk (programmable number of bytes ranging from 8 bytes through 512 bytes) of the write data. A CRC code is inserted into the outbound data stream immediately following that data chunk. Configuring the controller to use CRC depends on the active work mode. The CRC on the data bytes is calculated by exclusively-ORing the data bytes within a chunk.

CRC Using Direct or ACMD Mode

CRC is enabled using the `XSPI_SEQ_GCTL0[8:8]` bit. It can be configured to always assume that a CRC code is immediately present after the address bytes, or when it is to be inserted into the data stream after a chunk of bytes (independent of whether the stream is sending address bytes or data bytes). This operation is controlled using bit nine of the same register. Some devices require that the CRC be sent first with the CRC value and then repeated with an inverted value. Enable this option using the `XSPI_SEQ_GCTL0[10:10]` bit. Control the chunk size (number of bytes protected by the CRC) with the `XSPI_SEQ_GCTL0[14:12]` field. When the first CRC code protects

the address and is part of the data bytes, bit 16 should be set (=1). When checking that the first CRC code for read data is required, then bit 17 must also be set.

CRC Using STIG Mode

The parameters defined in the [XSPI_SEQ_GCTL0](#) register apply also for the direct and ACMD modes available directly in the glued data STIG instruction described in the [Glued Data Instruction](#) section. Enable the CRC using the [XSPI_SEQ_GCTL0.CRC_EN](#) field. Control the chunk size (number of bytes protected by the CRC) using the [XSPI_SEQ_GCTL0.CHUNK_SIZ](#) field. When the first CRC code protects the address and part of the data bytes, the [XSPI_SEQ_GCTL0.UAL_CHUNK_EN](#) field must be cleared (=0). When needing to check the first CRC code for read data, the [XSPI_SEQ_GCTL0.UAL_CHUNK_CHK](#) bit must be set (=1).

Miscellaneous Low-level Configurations

The following configurations are supported using the ACMD, direct, and STIG modes.

Write Protection

Some devices that operate in single or dual mode support the use of a write protect signal using the DQ2 pin. The controller can support this feature by using the [XSPI_MINICTL_WP_CTL](#) register. See the [XSPI_MINICTL_WP_CTL](#) register description for configuration details.

Device Reset

When operating in single or dual mode, some devices have a dedicated #RESET pin or support the hardware reset using the DQ3 pin. The controller supports the driving of reset on any bank (chip select) with the [XSPI_MINICTL_WP_CTL](#) register.

SPI Clock CPOL and CPHA Configuration

When operating in Single Data Rate (SDR), the transfers to and from the SPI memory device can operate in SPI mode 0 (polarity CPOL=0, phase CPHA=0, neither checked) or SPI mode 3 (CPOL=1, CPHA=1, both checked). It is unlikely this default configuration needs changing, but the option is available using the [XSPI_MINICTL_CLKMODE](#) register.

JEDEC Reset Delay Settings

When issuing a JEDEC reset instruction to the memory device, the Joint Electron Device Engineering Council Solid State Technology Association (JEDEC) standard requires the controller to adhere to the t_{CSL} and t_{CSH} timing parameters. When non-default settings are required, a user programs the [XSPI_MINICTL_JEDEC_RST_TR](#) register with specific values.

Device Delay Settings

The [XSPI_MINICTL_DEV_DLY](#) register can provide minimum delays between transactions at the memory device interface.

Reset Recovery Delay Settings

Following a hardware reset, the user can force a minimum time delay using the `XSPI_MINICTL_RST_RECOV` register, before the chip select moves to an active state.

t_{CMS} Timing Pause Refresh Settings

Some volatile memory devices can require a periodic refresh (for example, HyperRAM). This refresh action is referred to as t_{CMS} timing. Use the `XSPI_MINICTL_DEV_ACTIVE_MAX` register and the `XSPI_SEQ_GCTL0.TCMS_EN` bit for the controller to configure a maximum time that the chip select remains in an active state to perform a periodic refresh operation. The controller automatically breaks up larger transactions. This action is transparent to the host interface.

Legacy HyperFlash or Profile 2 Address Settings

By default, the host address must be rearranged to form the Command/Address (CA) structure for many legacy HyperFlash and profile 2 devices. Typically, the CA field has a reserved section; some bits of the host address must be shifted. The `XSPI_MINICTL_HF_OFFSET` register provides the user a method to configure the mapping of these reserved bits.

Multiple Device Connections to Controller

The xSPI controller can connect to a maximum of eight memory devices using the chip select (CS). Throughout this hardware reference, each device connected to an individual chip select is referred to as a bank. Mixed device types are permitted, but, cannot be used simultaneously. For example, when bank zero is connected to a quad SPI NOR flash device and bank one is connected to a HyperBus style device, then some reconfiguration of the xSPI register space is required when the host switches between accessing these devices.

SPI NAND Device Operation Settings

For SPI NAND devices, the memory device address contains both row and column addresses. The *SPI NAND Device Memory Address* figure shows the field layout. The boundary between the row and column addresses can be configured using the `XSPI_SEQ_GCTL1.PCA_SIZ` field. When the selected sequences require a number of bits aligned to 8 (for example, when a sequence requires 16 bits and contains 4 dummy bits before the 12 bits of the column address), the added dummy bits are internally cleared (=0).

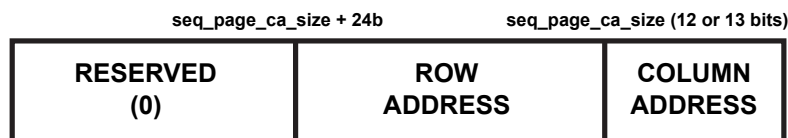


Figure 14-10: SPI NAND Device Memory Address

The SPI NAND devices require a specific page layout. The host software must program the controller to adhere to this field layout. It is especially important not to overwrite the reserved areas as bad block data.

ADSP-21560_21561_21564_21568 XSPI Register Descriptions

Expanded Serial Peripheral Interface (XSPI) contains the following registers.

Table 14-51: ADSP-21560_21561_21564_21568 XSPI Register List

Name	Description
XSPI_BOOT_STAT	Boot Status Register
XSPI_CMD0	Command Register 0
XSPI_CMD1	Command Register 1
XSPI_CMD2	Command Register 2
XSPI_CMD3	Command Register 3
XSPI_CMD4	Command Register 4
XSPI_CMD5	Command Register 5
XSPI_CMD_STAT	Command Status Register
XSPI_CMD_STAT_PTR	Command Status Pointer Register
XSPI_DAC_CFG	DAC Configuration Register
XSPI_DAC_REMAPADDR0	Address Remapping Register 0
XSPI_DAC_REMAPADDR1	Address Remapping Register 1
XSPI_DISCOVERY_GCTL	Device Discovery Control Register
XSPI_DMA_CTL	DMA Interface Control Register
XSPI_DMA_ERRADDR_HI	DMA Error Address High Register
XSPI_DMA_ERRADDR_LO	DMA Error Address Low Register
XSPI_ERS_SEQ_CFG0	Erase Sequence Configuration Register 0
XSPI_ERS_SEQ_CFG1	Erase Sequence Configuration Register 1
XSPI_ERS_SEQ_CFG2	Erase Sequence Configuration Register 2
XSPI_FEATURES	Controller Features Register
XSPI_GSTAT	General Controller Status Register
XSPI_INT_EN	Interrupt Enable Register
XSPI_ISTAT	Interrupt Status Register
XSPI_LONGPOL_GCTL	Long Polling Count Register
XSPI_MINICTL_DEV_ACTIVE_MAX	Device Active Maximum Clock Cycle Register
XSPI_MINICTL_CLKMODE	Clock Mode Control Register
XSPI_MINICTL_DEV_DLY	Device Delay Register
XSPI_MINICTL_HF_OFFSET	HyperFlash Offset Register
XSPI_MINICTL_JEDEC_RST_TR	JEDEC Reset Delay Register

Table 14-51: ADSP-21560_21561_21564_21568 XSPI Register List (Continued)

Name	Description
XSPI_MINICTL_RST_PIN_CTL	Hardware Reset Control Register
XSPI_MINICTL_RST_RECOV	Reset Recovery Delay Register
XSPI_MINICTL_WP_CTL	Write Protect Register
XSPI_PHY_DLLOB0	PHY DLL Observable Points 0 Register
XSPI_PHY_DLLOB1	PHY DLL Observable Points 1 Register
XSPI_PHY_DLL_CTL	PHY DLL Control Register
XSPI_PHY_DLL_MSTR_CTL	Master DLL Control Register
XSPI_PHY_DLL_SLAVE_CTL	Slave DLL Control Register
XSPI_PHY_DLL_UPDT_CNT	PHY DLL Resynchronization Register
XSPI_PHY_DQS_TR	PHY DQS Timing Register
XSPI_PHY_DQ_TR	PHY DQ Timing Register
XSPI_PHY_FEATURES	PHY Features Register
XSPI_PHY_GATE_LPBK_CTL	PHY Gate Loopback Control Register
XSPI_PHY_GCTL	PHY Global Control Register
XSPI_PHY_GPIO_CTL0	PHY GPIO Control Register 0
XSPI_PHY_GPIO_CTL1	PHY GPIO Control Register 1
XSPI_PHY_GPIO_STAT0	PHY GPIO Status Register 0
XSPI_PHY_GPIO_STAT1	PHY GPIO Status Register 1
XSPI_PHY_GTSEL	PHY Global Termination Control Register
XSPI_PHY_IE_TR	PHY DQS Input Enable Timing Register
XSPI_PHY_OB0	PHY Observable Points Register
XSPI_PHY_REVID	PHY Revision ID Register
XSPI_PHY_STATIC_TGL	PHY Static Aging Register
XSPI_PHY_WR_DESKEW_PAD_CTL0	PHY Deskew Write Register
XSPI_PROG_SEQ_CFG0	Program Sequence Configuration Register 0
XSPI_PROG_SEQ_CFG1	Program Sequence Configuration Register 1
XSPI_PROG_SEQ_CFG2	Program Sequence Configuration Register 2
XSPI_READ_SEQ_CFG0	Read Sequence Configuration Register 0
XSPI_READ_SEQ_CFG1	Read Sequence Configuration Register 1
XSPI_READ_SEQ_CFG2	Read Sequence Configuration Register 2
XSPI_REVID	Revision ID Register

Table 14-51: ADSP-21560_21561_21564_21568 XSPI Register List (Continued)

Name	Description
XSPI_RST_SEQ_CFG0	Reset Sequence Configuration Register 0
XSPI_RST_SEQ_CFG1	Reset Sequence Configuration Register 1
XSPI_SDMA_ADDR0	Host DMA Buffer Address Register 0
XSPI_SDMA_ADDR1	Host DMA Buffer Address Register 1
XSPI_SDMA_SIZ	Host DMA Block Size Register
XSPI_SDMA_TRD_STAT	Host DMA Thread Status Register
XSPI_SEQ_GCTL0	Sequence Configuration Register 0
XSPI_SEQ_GCTL1	Sequence Configuration Register 1
XSPI_SHORTPOL_GCTL	Short Polling Count Register
XSPI_STAT_SEQ_CFG0	Status Checking Sequence Configuration Register 0
XSPI_STAT_SEQ_CFG1	Status Checking Sequence Configuration Register 1
XSPI_STAT_SEQ_CFG10	Status Checking Sequence Configuration Register 10
XSPI_STAT_SEQ_CFG2	Status Checking Sequence Configuration Register 2
XSPI_STAT_SEQ_CFG3	Status Checking Sequence Configuration Register 3
XSPI_STAT_SEQ_CFG4	Status Checking Sequence Configuration Register 4
XSPI_STAT_SEQ_CFG5	Status Checking Sequence Configuration Register 5
XSPI_STAT_SEQ_CFG7	Status Checking Sequence Configuration Register 7
XSPI_STAT_SEQ_CFG8	Status Checking Sequence Configuration Register 8
XSPI_STAT_SEQ_CFG9	Status Checking Sequence Configuration Register 9
XSPI_TRD_COMP_ISTAT	Auto Command Engine Interrupt Status Thread Register
XSPI_TRD_ERR_INT_EN	Thread Error Interrupt Enable Register
XSPI_TRD_ERR_ISTAT	Thread Error Interrupt Status Register
XSPI_TRD_STAT	Auto Command Engine Thread Status Register
XSPI_WE_SEQ_CFG0	WEL Sequence Configuration Register
XSPI_WORKMODE_CTL	Device Control Register
XSPI_XIP_GCTL	XIP Configuration Register

Boot Status Register

The `XSPI_BOOT_STAT` register provides the status of the most recent boot operation, triggered by the automated boot controller.

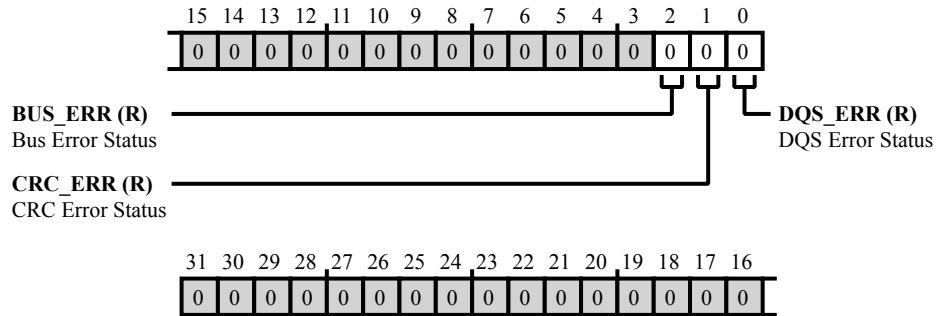


Figure 14-11: XSPI_BOOT_STAT Register Diagram

Table 14-52: XSPI_BOOT_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
2 (R/NW)	BUS_ERR	Bus Error Status. The <code>XSPI_BOOT_STAT.BUS_ERR</code> field describes bus status during the boot process. When set, the boot process failed due to the bus interface receiving an error response from the target.
		0 No Error Detected
		1 Error Detected
1 (R/NW)	CRC_ERR	CRC Error Status. The <code>XSPI_BOOT_STAT.CRC_ERR</code> field describes CRC status during the boot process. When set, the boot process failed due to the CRC error on the xSPI interface.
		0 No Error Detected
		1 Error Detected
0 (R/NW)	DQS_ERR	DQS Error Status. The <code>XSPI_BOOT_STAT.DQS_ERR</code> bit field describes the DQS status during the boot process. When set, the boot process failed due to the DQS error on the xSPI interface.
		0 No Error Detected
		1 Error Detected

Command Register 0

Writing data to the `XSPI_CMD0` register initiates a new transaction of the xSPI flash controller in CDMA/PIO and STIG work mode.

Encoding of the register depends on the selected work mode.

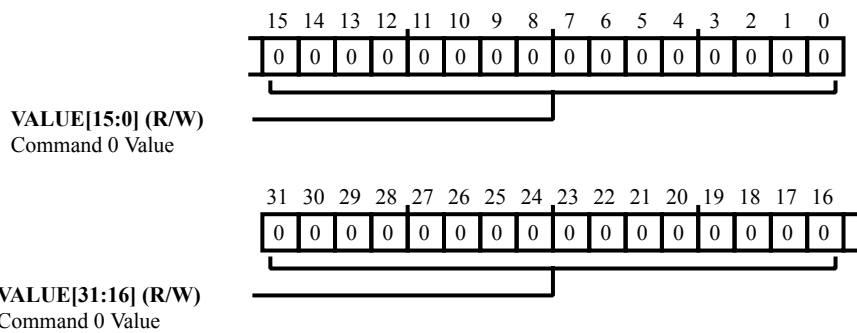


Figure 14-12: XSPI_CMD0 Register Diagram

Table 14-53: XSPI_CMD0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Command 0 Value. Command 0 register field.

Command Register 1

The `XSPI_CMD1` register is only used in CDMA/PIO and STIG work mode. Field encoding of the register depends on the selected work mode.

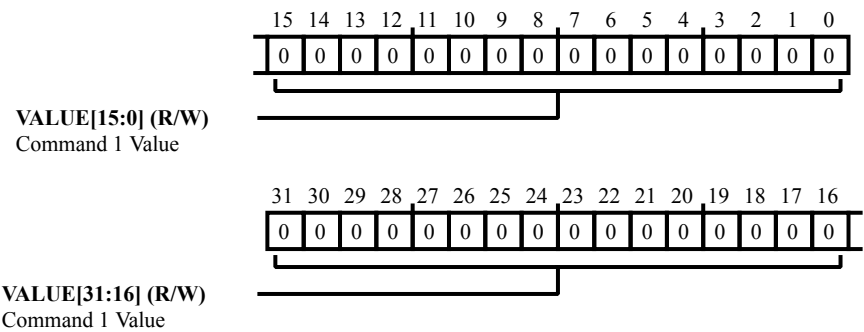


Figure 14-13: XSPI_CMD1 Register Diagram

Table 14-54: XSPI_CMD1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Command 1 Value. Command 1 register field.

Command Register 2

The `XSPI_CMD2` register is only used in CDMA/PIO and STIG work mode. Field encoding of the register depends on the selected work mode.

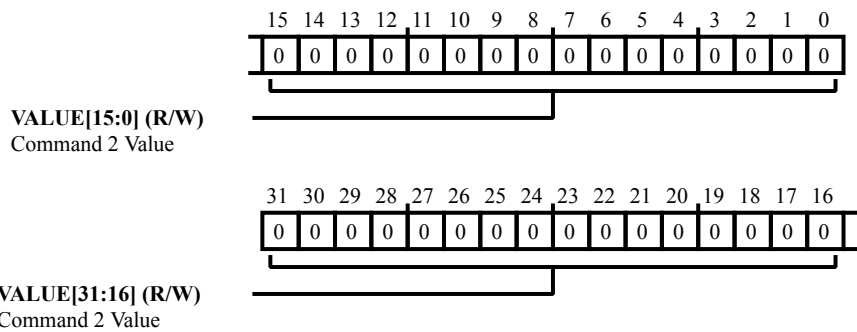


Figure 14-14: XSPI_CMD2 Register Diagram

Table 14-55: XSPI_CMD2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Command 2 Value. Command 2 register field.

Command Register 3

The `XSPI_CMD3` register is only used in CDMA/PIO and STIG work mode; the definition changes depending on the work mode.

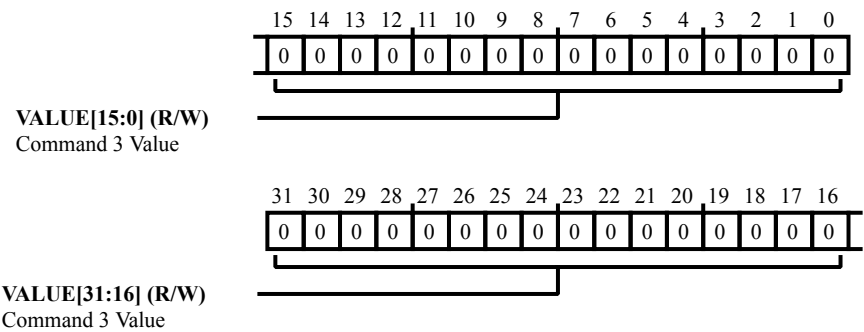


Figure 14-15: XSPI_CMD3 Register Diagram

Table 14-56: XSPI_CMD3 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Command 3 Value. Command 3 register field.

Command Register 4

The `XSPI_CMD4` register is only used in CDMA/PIO and STIG work mode; the definition changes depending on the work mode.

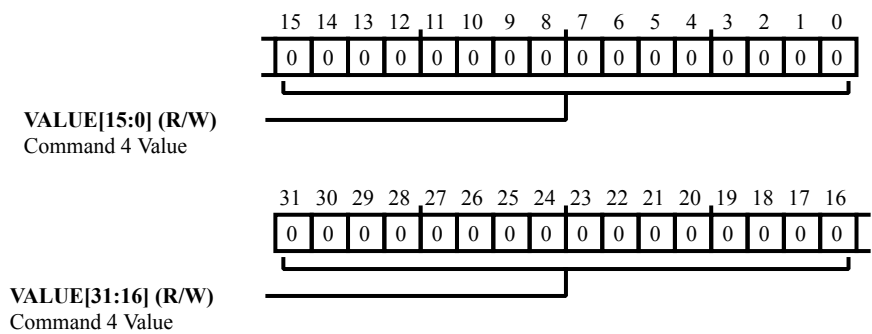


Figure 14-16: XSPI_CMD4 Register Diagram

Table 14-57: XSPI_CMD4 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Command 4 Value. Command 4 register field.

Command Register 5

The `XSPI_CMD5` register is only used in CDMA/PIO and STIG work mode; the definition changes depending on the work mode.

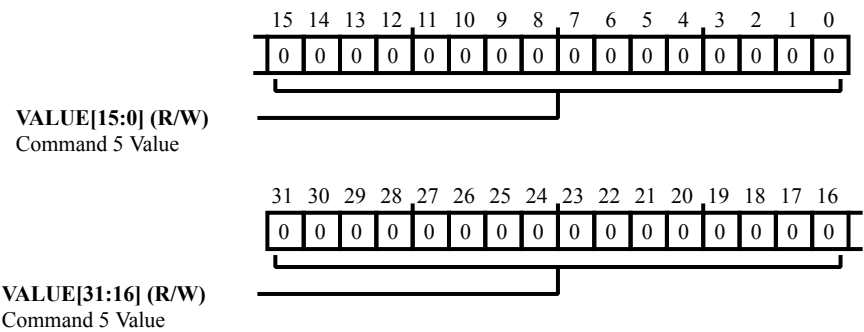


Figure 14-17: XSPI_CMD5 Register Diagram

Table 14-58: XSPI_CMD5 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Command 5 Value. Command 5 register field.

Command Status Register

The `XSPI_CMD_STAT` register reports the command status for the selected thread in ACMD and STIG work mode, when an xSPI flash transaction has completed.

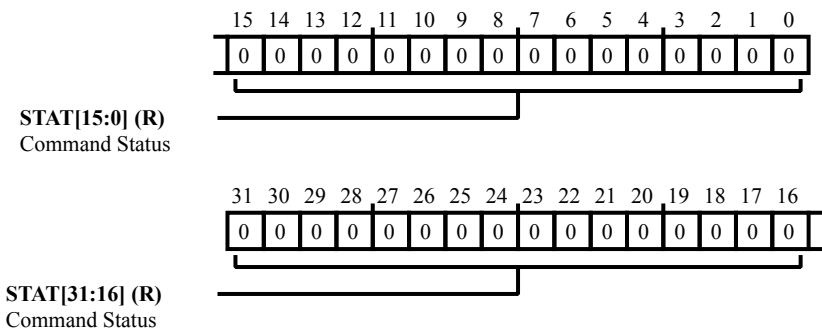


Figure 14-18: XSPI_CMD_STAT Register Diagram

Table 14-59: XSPI_CMD_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	STAT	Command Status. When operating in ACMD mode, the <code>XSPI_CMD_STAT.STAT</code> bit field reports the descriptor status associated with the selected thread (the thread to which this status belongs is selected via the <code>XSPI_CMD_STAT_PTR</code> register). When operating in STIG mode, this register reports the STIG completion status. Refer to section "STIG Mode Status" for a full bitwise definition.

Command Status Pointer Register

The `XSPI_CMD_STAT_PTR` register selects the thread that is used for outputting the command status in the `XSPI_CMD_STAT` register.

This register is relevant for ACMD mode only.

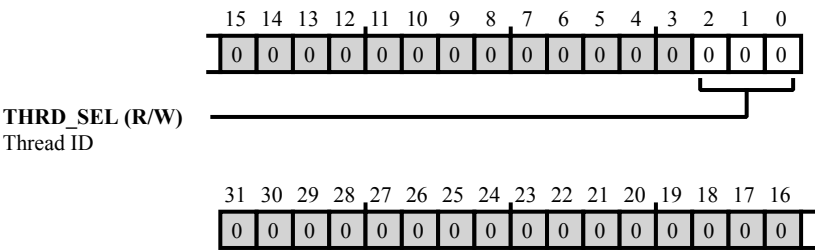


Figure 14-19: XSPI_CMD_STAT_PTR Register Diagram

Table 14-60: XSPI_CMD_STAT_PTR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
2:0 (R/W)	THRD_SEL	Thread ID. The <code>XSPI_CMD_STAT_PTR.THRD_SEL</code> bit field indicates the thread ID status; the status is available in the <code>XSPI_CMD_STAT</code> register.

DAC Configuration Register

The `XSPI_DAC_CFG` register holds specific configuration information required while operating in direct work mode.

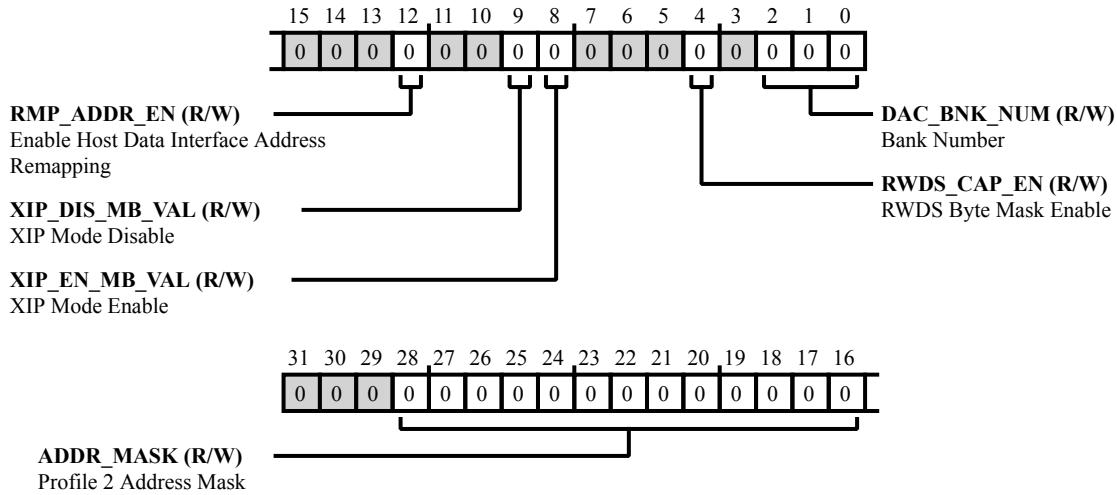


Figure 14-20: XSPI_DAC_CFG Register Diagram

Table 14-61: XSPI_DAC_CFG Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
28:16 (R/W)	ADDR_MASK	Profile 2 Address Mask. The <code>XSPI_DAC_CFG.ADDR_MASK</code> bit field enables masking bits [44:32] of the system address for read/write transfers for profile 2.
12 (R/W)	RMP_ADDR_EN	Enable Host Data Interface Address Remapping. When the <code>XSPI_DAC_CFG.RMP_ADDR_EN</code> bit is set (=1), the incoming AXI host address is adapted and sent to the flash device as (address - N), where N is the value stored in the remap address register (<code>XSPI_DAC_REMAPADDRn</code>).
9 (R/W)	XIP_DIS_MB_VAL	XIP Mode Disable. The <code>XSPI_DAC_CFG.XIP_DIS_MB_VAL</code> bit is used to trigger exit from XIP mode within the device. When <code>XSPI_DAC_CFG.XIP_DIS_MB_VAL</code> is set (=1), the controller sends mode bits specified in the <code>XSPI_XIP_GCTL.XIP_DIS_MB_VALUE</code> on the next read transaction. This disables XIP work mode for both the device and controller.
8 (R/W)	XIP_EN_MB_VAL	XIP Mode Enable. The <code>XSPI_DAC_CFG.XIP_EN_MB_VAL</code> bit is used to trigger entry to XIP mode within the device. When <code>XSPI_DAC_CFG.XIP_EN_MB_VAL</code> is set (=1), the controller sends mode bits as specified in <code>XSPI_XIP_GCTL.XIP_EN_MB_VAL</code> on the next read transaction. This switches both device and controller into XIP work mode.

Table 14-61: XSPI_DAC_CFG Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
4 (R/W)	RWDS_CAP_EN	RWDS Byte Mask Enable. When enabled, the controller supports byte masking in the connected device by automatically translating the incoming AXI host interface write strobes as needed. If the connected device does not support RWDS byte masking, the XSPI_DAC_CFG.RWDS_CAP_EN bit must be cleared. Note that when the XSPI_DAC_CFG.RWDS_CAP_EN bit is cleared, the controller assumes all bytes on the AXI write channel have an equivalent AXI write strobe set to 1. Also note that when this bit is cleared, the user cannot send byte writes to an octal DDR device or to a device that is 16-bit addressed.
2:0 (R/W)	DAC_BNK_NUM	Bank Number. The XSPI_DAC_CFG.DAC_BNK_NUM bit field indicates the bank (the attached memory device) targeted by the controller while operating in direct mode.

Address Remapping Register 0

When XSPI_DAC_CFG.RMP_ADDR_EN =1, the incoming AXI host interface address is adapted and sent to the flash device as (address - N), where N[31:0] is the value stored in this register.

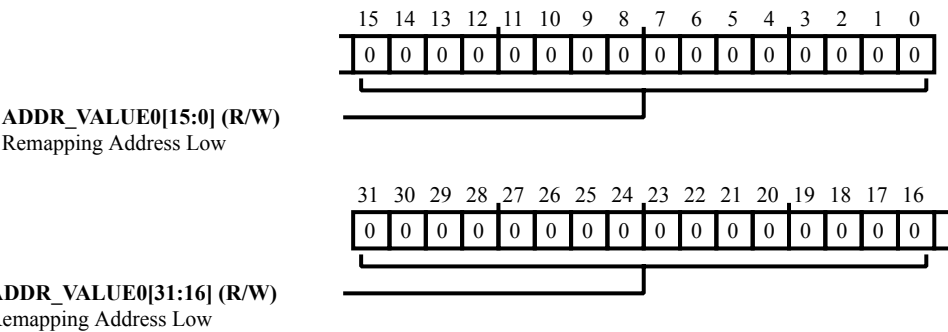


Figure 14-21: XSPI_DAC_REMAPADDR0 Register Diagram

Table 14-62: XSPI_DAC_REMAPADDR0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	ADDR_VALUE0	Remapping Address Low. The XSPI_DAC_REMAPADDR0.ADDR_VALUE0 bit field indicates the remapping of the incoming address on the AXI host interface to a different address used by the flash device. The value of this register must be aligned to 8 bytes.

Address Remapping Register 1

When XSPI_DAC_CFG.RMP_ADDR_EN =1, the incoming host interface address is adapted and sent to the flash device as (address - N), where N[63:32] is the value stored in this register.

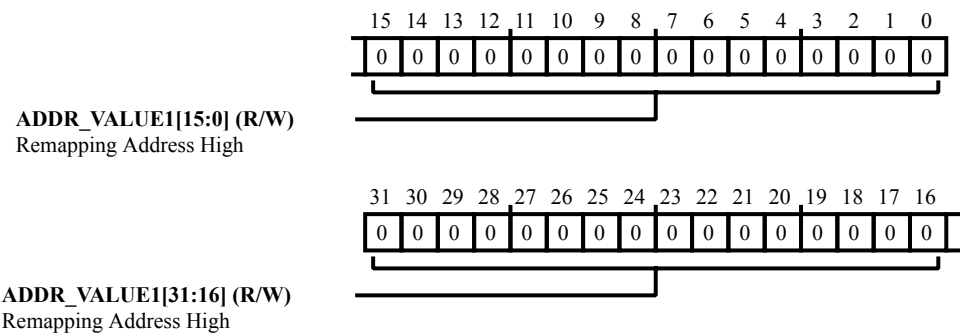


Figure 14-22: XSPI_DAC_REMAPADDR1 Register Diagram

Table 14-63: XSPI_DAC_REMAPADDR1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	ADDR_VALUE1	Remapping Address High. The XSPI_DAC_REMAPADDR1.ADDR_VALUE1 bit field indicates the remapping of the incoming address on the host interface to a different address used by the flash device.

Device Discovery Control Register

The `XSPI_DISCOVERY_GCTL` register controls device discovery.

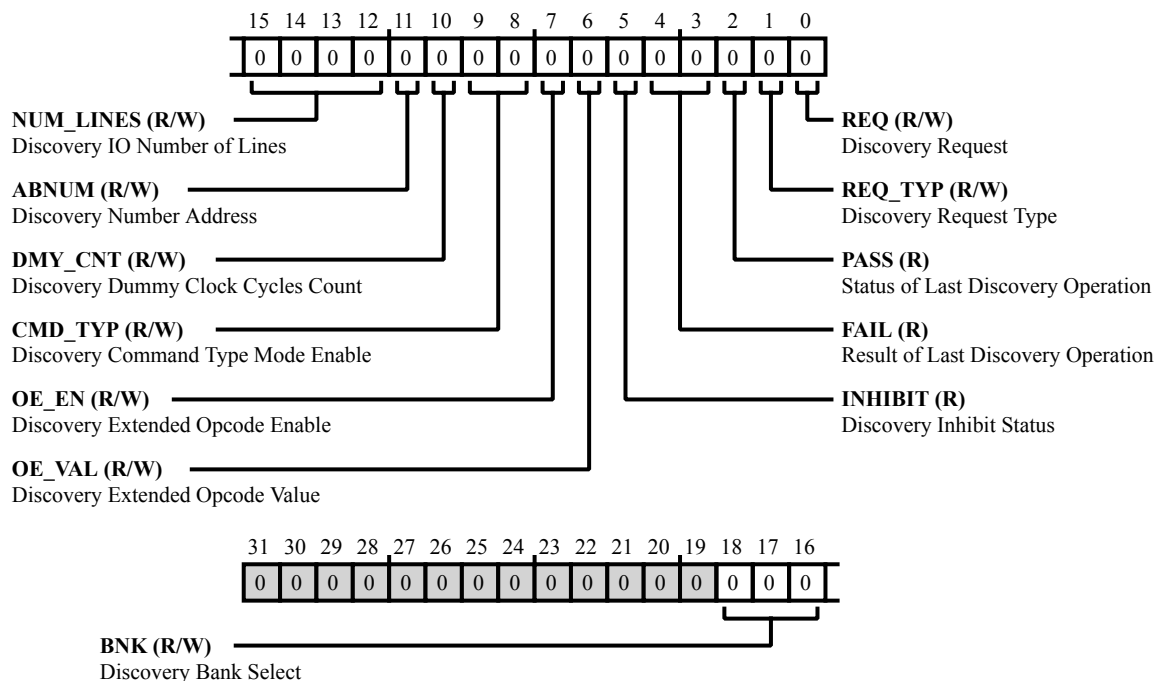


Figure 14-23: XSPI_DISCOVERY_GCTL Register Diagram

Table 14-64: XSPI_DISCOVERY_GCTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
18:16 (R/W)	BNK	Discovery Bank Select. The <code>XSPI_DISCOVERY_GCTL.BNK</code> bit field indicates the discovery bank. Writing a value of 0x0-0x7 selects a bank. This field is updated after the initialization process.
15:12 (R/W)	NUM_LINES	Discovery IO Number of Lines. This is a 4-bit value used in discovery mode. Writing a value to the <code>XSPI_DISCOVERY_GCTL.NUM_LINES</code> bit field selects number of xSPI I/Os used by device discovery. This field is updated after the initialization process.
	0	Auto
	1	1 line
	2	2 lines
	3	Reserved
	4	4 lines

Table 14-64: XSPI_DISCOVERY_GCTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
		5	Reserved
		6	Reserved
		7	Reserved
		8	8 lines
		9	Reserved
		10	Reserved
		11	Reserved
		12	8 lines for legacy HyperFlash and xSPI profile
		13	Reserved
		14	1 line for legacy SPI NAND
		15	Reserved
11 (R/W)	ABNUM	Discovery Number Address. The XSPI_DISCOVERY_GCTL.ABNUM bit enables discovery 4-bit addressing. This field is updated after the initialization process.	
		0	3-bit addressing
		1	4-bit addressing
10 (R/W)	DMY_CNT	Discovery Dummy Clock Cycles Count. The XSPI_DISCOVERY_GCTL.DMY_CNT bit indicates the discovery number of dummy clock cycles. This bit is updated after the initialization process.	
		0	8 dummy clock cycles
		1	20 dummy clock cycles
9:8 (R/W)	CMD_TYP	Discovery Command Type Mode Enable. The XSPI_DISCOVERY_GCTL.CMD_TYP bit selects the discovery command type mode. This field is updated after the initialization process.	
		0	SDR mode enabled
		1	DDR mode enabled
		2	DTR mode enabled (quad mode only)
		3	Reserved

Table 14-64: XSPI_DISCOVERY_GCTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
7 (R/W)	OE_EN	Discovery Extended Opcode Enable. The XSPI_DISCOVERY_GCTL.OE_EN bit enables the discovery extended opcode. This field is updated after the initialization process.
		0 Disable
		1 Enable
6 (R/W)	OE_VAL	Discovery Extended Opcode Value. The XSPI_DISCOVERY_GCTL.OE_VAL bit field indicates the discovery extended opcode value. This field is updated after the initialization process.
		0 Extended opcode is 8'hA5
		1 Extended opcode is 8'h5A
5 (R/NW)	INHIBIT	Discovery Inhibit Status. The XSPI_DISCOVERY_GCTL.INHIBIT bit indicates whether device discovery is inhibited at power on.
		0 Discovery allow
		1 Discovery inhibit
4:3 (R/NW)	FAIL	Result of Last Discovery Operation. The XSPI_DISCOVERY_GCTL.FAIL bit field indicates the result of the last discovery operation. Valid when PASS = 1.
		0 xSPI or SPI NAND device detected
		1 Fail
		2 Legacy SPI device detected
		3 N/A
2 (R/NW)	PASS	Status of Last Discovery Operation. The XSPI_DISCOVERY_GCTL.PASS bit indicates the status of the last discovery operation. XSPI_DISCOVERY_GCTL.PASS = 1 when the device discovery operation has finished. The result can be read from the FAIL field.
1 (R/W)	REQ_TYP	Discovery Request Type. The XSPI_DISCOVERY_GCTL.REQ_TYP bit indicates the discovery request type.
		0 Full discovery. Perform full discovery process (try to detect device)
		1 Selected mode. Configure registers only to selected mode (not full discovery process)

Table 14-64: XSPI_DISCOVERY_GCTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	REQ	Discovery Request. Writing 1 to the XSPI_DISCOVERY_GCTL.REQ bit triggers the device discovery operation. The XSPI_DISCOVERY_GCTL.REQ bit is cleared by hardware when the device discovery operation completes.

DMA Interface Control Register

The `XSPI_DMA_CTL` register is a common register for both the host and internal DMA interface.

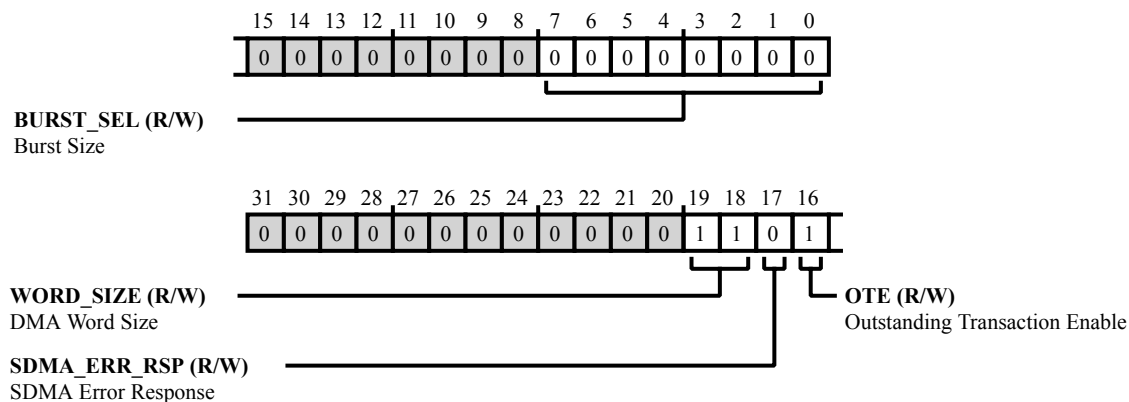


Figure 14-24: XSPI_DMA_CTL Register Diagram

Table 14-65: XSPI_DMA_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
19:18 (R/W)	WORD_SIZE	DMA Word Size. The <code>XSPI_DMA_CTL.WORD_SIZE</code> bit field indicates the AXI transaction size used by the internal DMA interface to transfer data.
		0 Byte
		1 16-bit word
		2 32-bit word
		3 64-bit word
17 (R/W)	SDMA_ERR_RSP	SDMA Error Response. AXI error responses can only occur in ACMD or STIG work modes. When <code>XSPI_DMA_CTL.SDMA_ERR_RSP</code> is set (=1), an AXI error response is returned if one of the following is true. 1. The host attempts to access the target interface before it is permitted (refer to the required steps in STIG and ACMD sections of this document). 2. The host issues an unsupported burst type. The controller only supports incremental bursts (non-wrapping) bursts currently. When <code>XSPI_DMA_CTL.SDMA_ERR_RSP</code> is cleared (=0), an OK response is returned.
16 (R/W)	OTE	Outstanding Transaction Enable. The <code>XSPI_DMA_CTL.OTE</code> bit enables an outstanding transaction. This only applies to the internal DMA interface. The AXI host interface ignores this bit and accepts all incoming transactions.

Table 14-65: XSPI_DMA_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/W)	BURST_SEL	Burst Size. The XSPI_DMA_CTL.BURST_SEL bit field indicates the maximum AXI burst size used by the internal DMA interface. The maximum burst size can be calculated as XSPI_DMA_CTL.BURST_SEL+1. Note that this field must be changed only when controller is in an idle state.

DMA Error Address High Register

Internal DMA interface error address [63:32]. The `XSPI_DMA_ERRADDR_HI` register holds the upper 32 bits of the address of the AXI request on the system master data interface that caused the `CDMA_TERR` or `DDMA_TERR` bits in the `XSPI_ISTAT` register to be set. This register is overwritten when further error responses are detected.

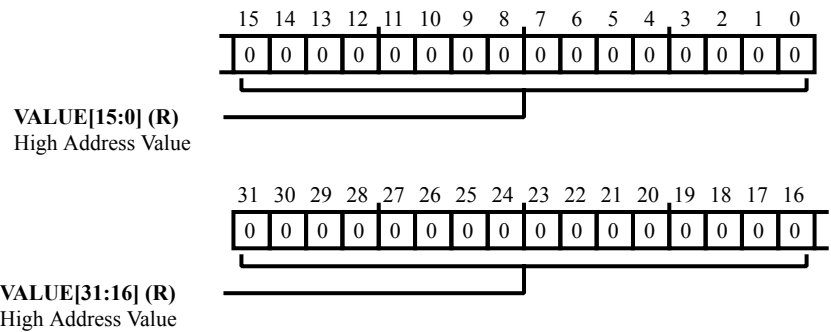


Figure 14-25: XSPI_DMA_ERRADDR_HI Register Diagram

Table 14-66: XSPI_DMA_ERRADDR_HI Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	High Address Value. The <code>XSPI_DMA_ERRADDR_HI.VALUE</code> bit field indicates the address of the first request on the internal DMA interface that returned an error response.

DMA Error Address Low Register

Internal DMA interface error address [31:0]. The `XSPI_DMA_ERRADDR_LO` register holds the lower 32 bits of the address of the AXI request on the system master data interface that caused the `CDMA_TERR` or `DDMA_TERR` bits in the `XSPI_ISTAT` register to be set. The `XSPI_DMA_ERRADDR_LO` register is overwritten when further error responses are detected.

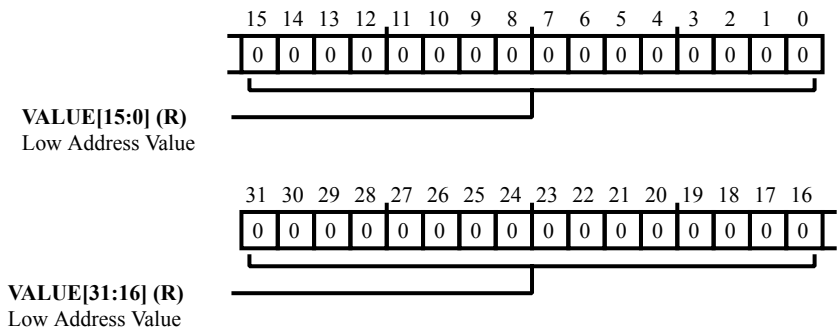


Figure 14-26: XSPI_DMA_ERRADDR_LO Register Diagram

Table 14-67: XSPI_DMA_ERRADDR_LO Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	Low Address Value. The <code>XSPI_DMA_ERRADDR_LO.VALUE</code> bit field indicates the address of the first request on the internal DMA interface that returned an error response.

Erase Sequence Configuration Register 0

The `XSPI_ERS_SEQ_CFG0` register configures the erase_sector sequence for profile 1 and SPI NAND in ACMD work mode.

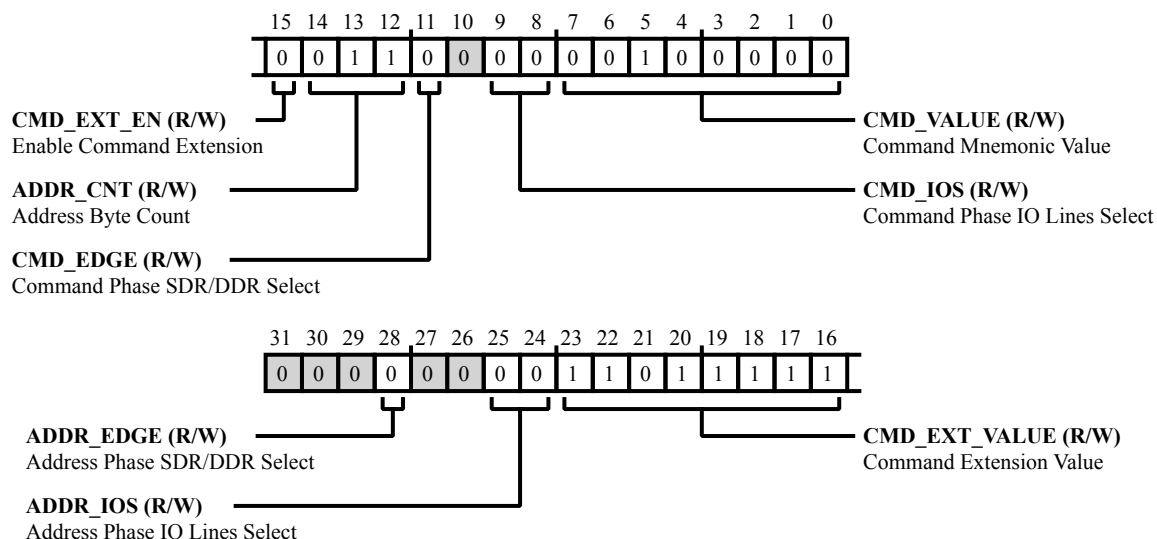


Figure 14-27: XSPI_ERS_SEQ_CFG0 Register Diagram

Table 14-68: XSPI_ERS_SEQ_CFG0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
28 (R/W)	ADDR_EDGE	Address Phase SDR/DDR Select. The <code>XSPI_ERS_SEQ_CFG0.ADDR_EDGE</code> bit selects between SDR/DDR mode for the address phase.
25:24 (R/W)	ADDR_IOS	Address Phase IO Lines Select. The <code>XSPI_ERS_SEQ_CFG0.ADDR_IOS</code> bit field indicates the number of data lines used to send the address phase.
	0	One data line used (for example, serial)
	1	Two data lines used
	2	Four data lines used
	3	Eight data lines used
23:16 (R/W)	CMD_EXT_VALUE	Command Extension Value. The <code>XSPI_ERS_SEQ_CFG0.CMD_EXT_VALUE</code> bit field indicates the command extension value, when enabled.
15 (R/W)	CMD_EXT_EN	Enable Command Extension. The <code>XSPI_ERS_SEQ_CFG0.CMD_EXT_EN</code> bit enables command extension.

Table 14-68: XSPI_ERS_SEQ_CFG0 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
14:12 (R/W)	ADDR_CNT	Address Byte Count. The XSPI_ERS_SEQ_CFG0.ADDR_CNT bit field indicates the number of address bytes.
11 (R/W)	CMD_EDGE	Command Phase SDR/DDR Select. The XSPI_ERS_SEQ_CFG0.CMD_EDGE bit field selects between SDR/DDR mode for the command phase.
9:8 (R/W)	CMD_IOS	Command Phase IO Lines Select. The XSPI_ERS_SEQ_CFG0.CMD_IOS bit field indicates the number of data lines used to send the command phase.
		0 One data line used (for example, serial)
		1 Two data lines used
		2 Four data lines used
		3 Eight data lines used
7:0 (R/W)	CMD_VALUE	Command Mnemonic Value. The XSPI_ERS_SEQ_CFG0.CMD_VALUE bit field indicates the command mnemonic value.

Erase Sequence Configuration Register 1

The `XSPI_ERS_SEQ_CFG1` register configures the `erase_sector` sequence for profile 1 and SPI NAND in ACMD work mode.

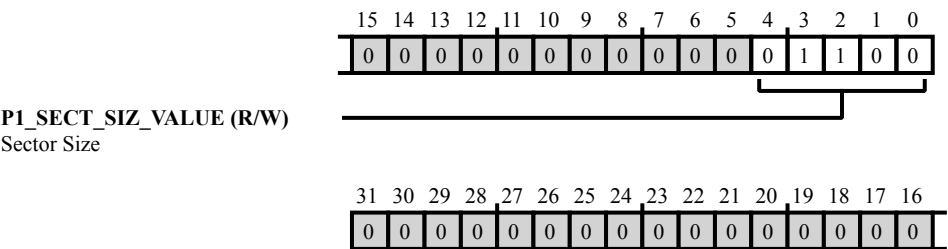


Figure 14-28: XSPI_ERS_SEQ_CFG1 Register Diagram

Table 14-69: XSPI_ERS_SEQ_CFG1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
4:0 (R/W)	P1_SECT_SIZ_VALUE	Sector Size. The <code>XSPI_ERS_SEQ_CFG1.P1_SECT_SIZ_VALUE</code> bit field indicates the sector size. Value encoded as $2^{\text{erss_seq_p1_sect_size}}$: 8'h00 - 1B 8'h01 - 2B 8'h02 - 4B ... 8'h0f - 32kB 8'h10 - 64kB ... 8'h1f - (2^{31})B

Erase Sequence Configuration Register 2

The `XSPI_ERS_SEQ_CFG2` register configures the erase_all sequence for profile 1 in ACMD work mode.

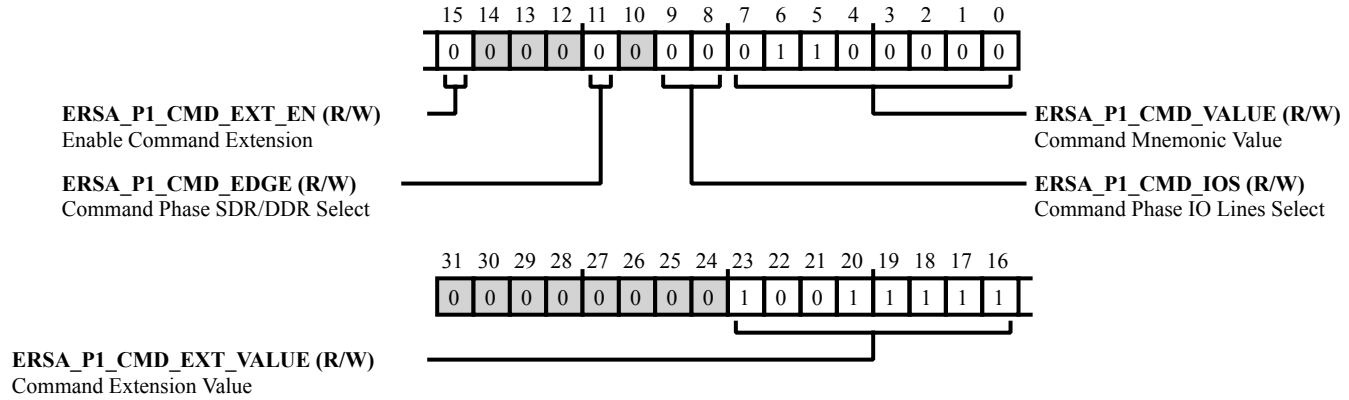


Figure 14-29: XSPI_ERS_SEQ_CFG2 Register Diagram

Table 14-70: XSPI_ERS_SEQ_CFG2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
23:16 (R/W)	ERSA_P1_CMD_EXT_VALUE	Command Extension Value. The <code>XSPI_ERS_SEQ_CFG2.ERSA_P1_CMD_EXT_VALUE</code> bit field indicates the command extension value.
15 (R/W)	ERSA_P1_CMD_EXT_EN	Enable Command Extension. The <code>XSPI_ERS_SEQ_CFG2.ERSA_P1_CMD_EXT_EN</code> bit field enables the command extension.
11 (R/W)	ERSA_P1_CMD_EDGE	Command Phase SDR/DDR Select. The <code>XSPI_ERS_SEQ_CFG2.ERSA_P1_CMD_EDGE</code> bit selects between SDR/DDR mode for the command phase.
9:8 (R/W)	ERSA_P1_CMD_IOS	Command Phase IO Lines Select. The <code>XSPI_ERS_SEQ_CFG2.ERSA_P1_CMD_IOS</code> bit field indicates the number of data lines used to send the command phase.
	0	One data line used (for example, serial)
	1	Two data lines used
	2	Four data lines used
	3	Eight data lines used
7:0 (R/W)	ERSA_P1_CMD_VALUE	Command Mnemonic Value. The <code>XSPI_ERS_SEQ_CFG2.ERSA_P1_CMD_VALUE</code> bit field indicates the command mnemonic value.

Controller Features Register

The `XSPI_FEATURES` register shows the available hardware features of the controller.

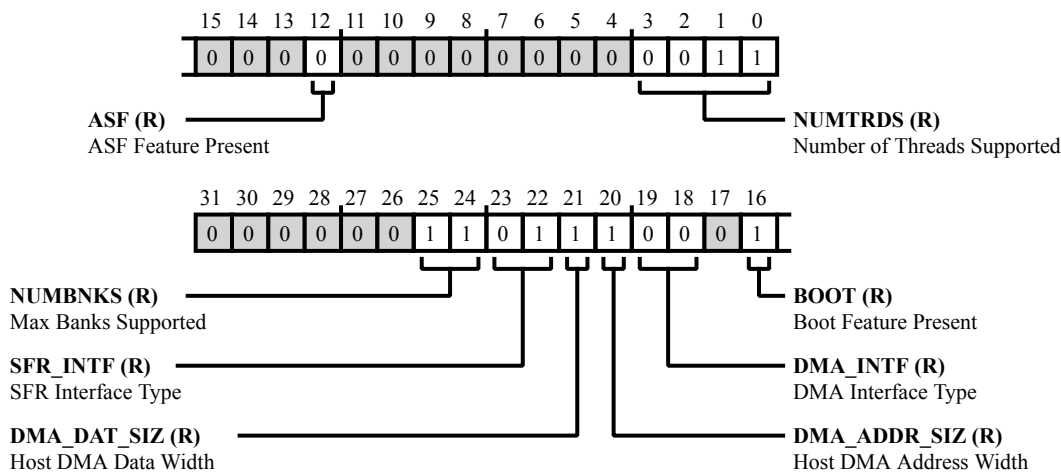


Figure 14-30: XSPI_FEATURES Register Diagram

Table 14-71: XSPI_FEATURES Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
25:24 (R/NW)	NUMBNKS	Max Banks Supported. The <code>XSPI_FEATURES.NUMBNKS</code> bit field indicates the maximum number of banks supported by hardware.
		0 One bank
		1 Two banks
		2 Four banks
		3 Eight banks
23:22 (R/NW)	SFR_INTF	SFR Interface Type. The <code>XSPI_FEATURES.SFR_INTF</code> bit field indicates the SFR interface type 1-APB, other values reserved.
21 (R/NW)	DMA_DAT_SIZ	Host DMA Data Width. The <code>XSPI_FEATURES.DMA_DAT_SIZ</code> bit indicates the host DMA data width.
		0 32 bit
		1 64 bit

Table 14-71: XSPI_FEATURES Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
20 (R/NW)	DMA_ADDR_SIZ	Host DMA Address Width. The XSPI_FEATURES.DMA_ADDR_SIZ bit indicates the host DMA address width.
		0 32 bit
		1 64 bit
19:18 (R/NW)	DMA_INTF	DMA Interface Type. The XSPI_FEATURES.DMA_INTF bit field indicates the DMA interface type (0-AXI4 other values reserved).
16 (R/NW)	BOOT	Boot Feature Present. The XSPI_FEATURES.BOOT bit indicates whether the boot feature is present.
12 (R/NW)	ASF	ASF Feature Present. The XSPI_FEATURES.ASF bit indicates whether the ASF feature is present.
3:0 (R/NW)	NUMTRDS	Number of Threads Supported. The XSPI_FEATURES.NUMTRDS bit field indicates the number of threads available in the controller.
		0 One thread
		1 Two threads
		2 Four threads
		3 Eight threads
		4-15 Reserved

General Controller Status Register

The `XSPI_GSTAT` register indicates the status of the controller.

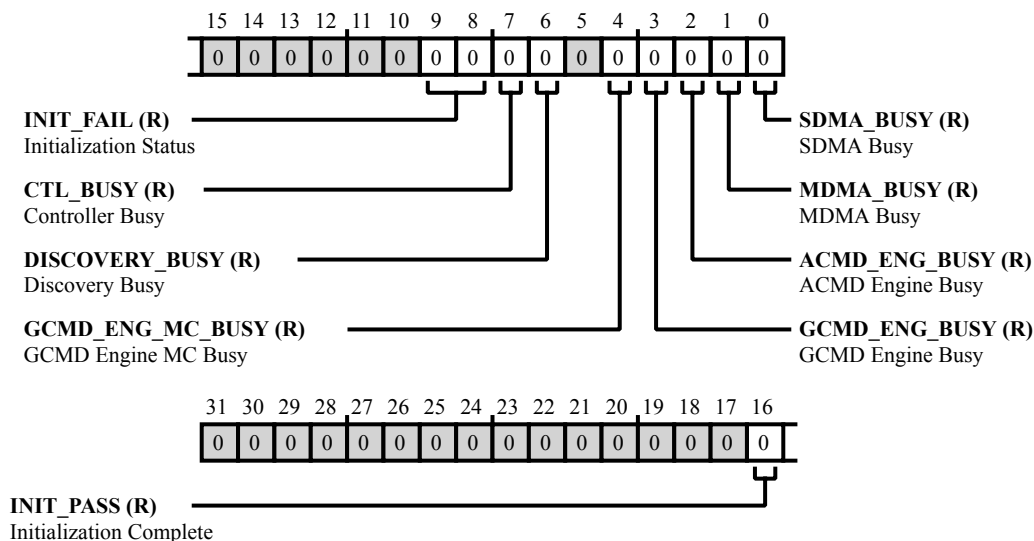


Figure 14-31: XSPI_GSTAT Register Diagram

Table 14-72: XSPI_GSTAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
16 (R/NW)	INIT_PASS	Initialization Complete. When <code>XSPI_GSTAT.INIT_PASS</code> is set (=1), the xSPI controller has completed its reset and initialization process.
9:8 (R/NW)	INIT_FAIL	Initialization Status. The <code>XSPI_GSTAT.INIT_FAIL</code> bit field indicates the initialization process status.
		0 xSPI device detected
		1 Initialization has failed
		2 Legacy SPI device detected
		3 N/A
7 (R/NW)	CTL_BUSY	Controller Busy. The <code>XSPI_GSTAT.CTL_BUSY</code> bit indicates whether controller is in the busy state. Note that this bit is also routed to the controller interface via the <code>CTRL_BUSY</code> pin.
		0 Idle
		1 Busy

Table 14-72: XSPI_GSTAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
6 (R/NW)	DISCOVERY_BUSY	Discovery Busy. When XSPI_GSTAT.DISCOVERY_BUSY is set (=1), device discovery is in progress. When device discovery is inhibited, this bit is also set during the PHY initialization procedure.
4 (R/NW)	GCMD_ENG_MC_BUSY	GCMD Engine MC Busy. The bit XSPI_GSTAT.GCMD_ENG_MC_BUSY is relevant for STIG mode only. This bit indicates when the controller is waiting for next/last instruction in a glued instruction chain or it is executing a requested sequence on the xSPI interface.
3 (R/NW)	GCMD_ENG_BUSY	GCMD Engine Busy. When operating in direct work mode, the XSPI_GSTAT.GCMD_ENG_BUSY is set (=1), when the direct CMD generator is busy. Note that no new requests on the host interface are accepted while XSPI_GSTAT.GCMD_ENG_BUSY is set. When operating in STIG work mode, the XSPI_GSTAT.GCMD_ENG_BUSY is set while the STIG engine is busy.
2 (R/NW)	ACMD_ENG_BUSY	ACMD Engine Busy. When XSPI_GSTAT.ACMD_ENG_BUSY is set (=1), the auto command engine is busy (ACMD mode only).
1 (R/NW)	MDMA_BUSY	MDMA Busy. When XSPI_GSTAT.MDMA_BUSY is set (=1), the internal DMA interface is busy.
0 (R/NW)	SDMA_BUSY	SDMA Busy. When XSPI_GSTAT.SDMA_BUSY is set (=1), the host interface is busy.

Interrupt Enable Register

If the selected bit of the `XSPI_INT_EN` register is set (=1), the rising edge of the corresponding bit in the `XSPI_ISTAT` register triggers an interrupt.

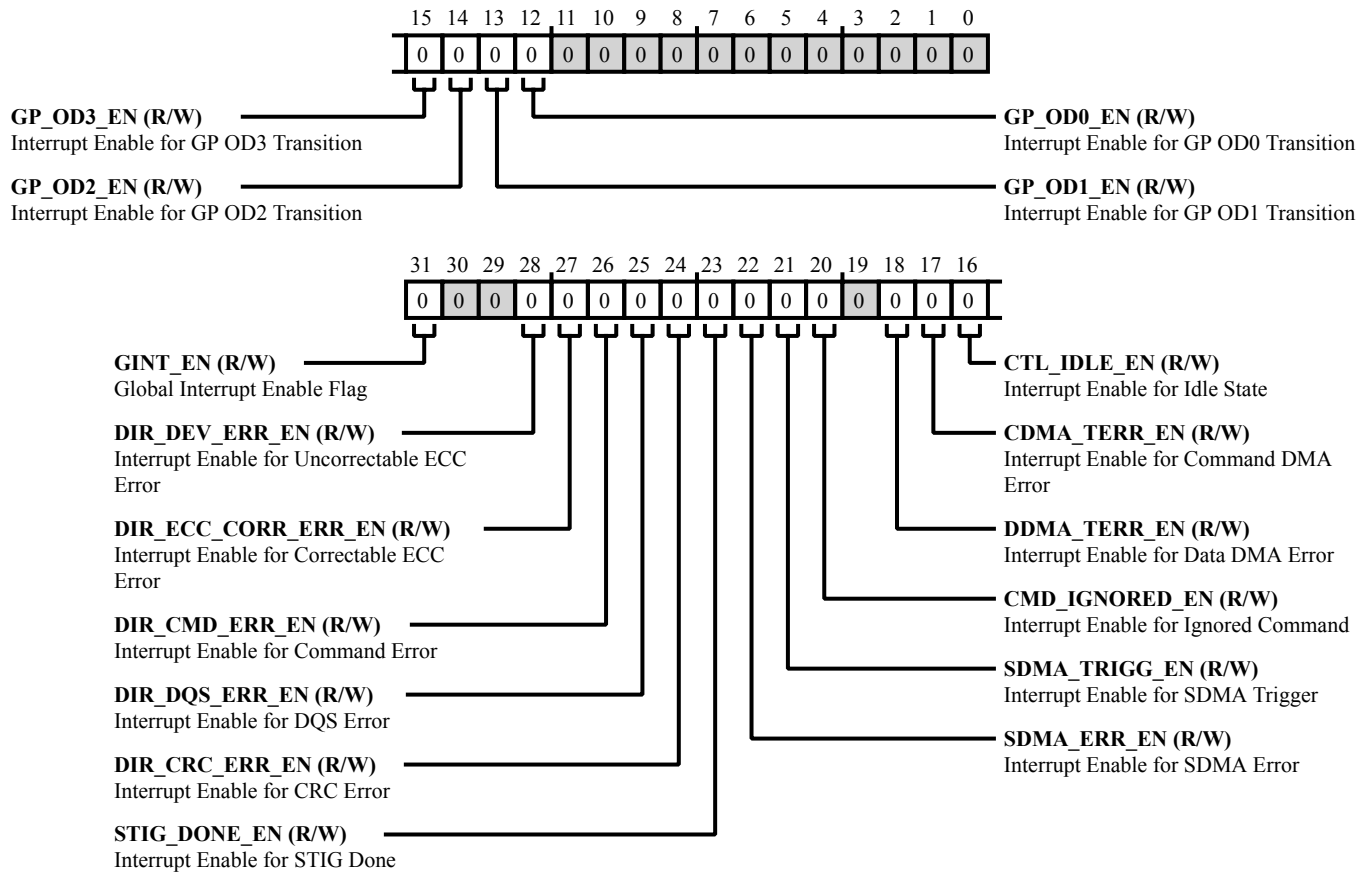


Figure 14-32: XSPI_INT_EN Register Diagram

Table 14-73: XSPI_INT_EN Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	GINT_EN	Global Interrupt Enable Flag. The <code>XSPI_INT_EN.GINT_EN</code> bit enables a global interrupt flag.
28 (R/W)	DIR_DEV_ERR_EN	Interrupt Enable for Uncorrectable ECC Error. The <code>XSPI_INT_EN.DIR_DEV_ERR_EN</code> bit enables an interrupt when an uncorrectable ECC or program fail error occurred in direct work mode.
27 (R/W)	DIR_ECC_CORR_ERR_EN	Interrupt Enable for Correctable ECC Error. The <code>XSPI_INT_EN.DIR_ECC_CORR_ERR_EN</code> bit enables an interrupt when a correctable ECC error occurred in direct work mode.

Table 14-73: XSPI_INT_EN Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
26 (R/W)	DIR_CMD_ERR_EN	Interrupt Enable for Command Error. The XSPI_INT_EN.DIR_CMD_ERR_EN bit enables an interrupt when an invalid command sequence has been detected in direct work mode.
25 (R/W)	DIR_DQS_ERR_EN	Interrupt Enable for DQS Error. The XSPI_INT_EN.DIR_DQS_ERR_EN bit enables an interrupt when the controller returns a DQS error after a read or status checking command in direct work mode.
24 (R/W)	DIR_CRC_ERR_EN	Interrupt Enable for CRC Error. The XSPI_INT_EN.DIR_CRC_ERR_EN bit enables an interrupt when the controller returns a CRC error after a read or status checking command in direct work mode.
23 (R/W)	STIG_DONE_EN	Interrupt Enable for STIG Done. The XSPI_INT_EN.STIG_DONE_EN bit enables an interrupt when an instruction in a glued chain is complete.
22 (R/W)	SDMA_ERR_EN	Interrupt Enable for SDMA Error. The XSPI_INT_EN.SDMA_ERR_EN bit enables an interrupt when an illegal access to the host DMA interface is detected.
21 (R/W)	SDMA_TRIGG_EN	Interrupt Enable for SDMA Trigger. The XSPI_INT_EN.SDMA_TRIGG_EN bit enables an interrupt when the trigger condition for the host DMA is met.
20 (R/W)	CMD_IGNORED_EN	Interrupt Enable for Ignored Command. The XSPI_INT_EN.CMD_IGNORED_EN bit enables an interrupt for detection of an ignored command.
18 (R/W)	DDMA_TERR_EN	Interrupt Enable for Data DMA Error. The XSPI_INT_EN.DDMA_TERR_EN bit enables an interrupt for detecting a data internal DMA error.
17 (R/W)	CDMA_TERR_EN	Interrupt Enable for Command DMA Error. The XSPI_INT_EN.CDMA_TERR_EN bit enables an interrupt for detecting an auto CMD engine target error.
16 (R/W)	CTL_IDLE_EN	Interrupt Enable for Idle State. The XSPI_INT_EN.CTL_IDLE_EN bit enables an interrupt for detecting that the controller has returned to the idle state.
15 (R/W)	GP_OD3_EN	Interrupt Enable for GP OD3 Transition. The XSPI_INT_EN.GP_OD3_EN bit enables an interrupt for detecting the high-to-low or low-to-high transition on the XSPI_DFI_GP_OPEN_DRAIN[3] input pin.

Table 14-73: XSPI_INT_EN Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
14 (R/W)	GP_OD2_EN	Interrupt Enable for GP OD2 Transition. The XSPI_INT_EN.GP_OD2_EN bit enables an interrupt for detecting the high-to-low or low-to-high transition on the XSPI_DFI_GP_OPEN_DRAIN[2] input pin.
13 (R/W)	GP_OD1_EN	Interrupt Enable for GP OD1 Transition. The XSPI_INT_EN.GP_OD1_EN bit enables an interrupt for detecting the high-to-low or low-to-high transition on the XSPI_DFI_GP_OPEN_DRAIN[1] input pin.
12 (R/W)	GP_OD0_EN	Interrupt Enable for GP OD0 Transition. The XSPI_INT_EN.GP_OD0_EN bit enables an interrupt for detecting the high-to-low or low-to-high transition on the XSPI_DFI_GP_OPEN_DRAIN[0] input pin.

Interrupt Status Register

The `XSPI_ISTAT` register contains information about functional areas requiring servicing. Many of the bits serve as an indicator to further read and service various status registers. After servicing the interrupt source associated with a bit, the user must clear that interrupt source bit by writing a 1 to it.

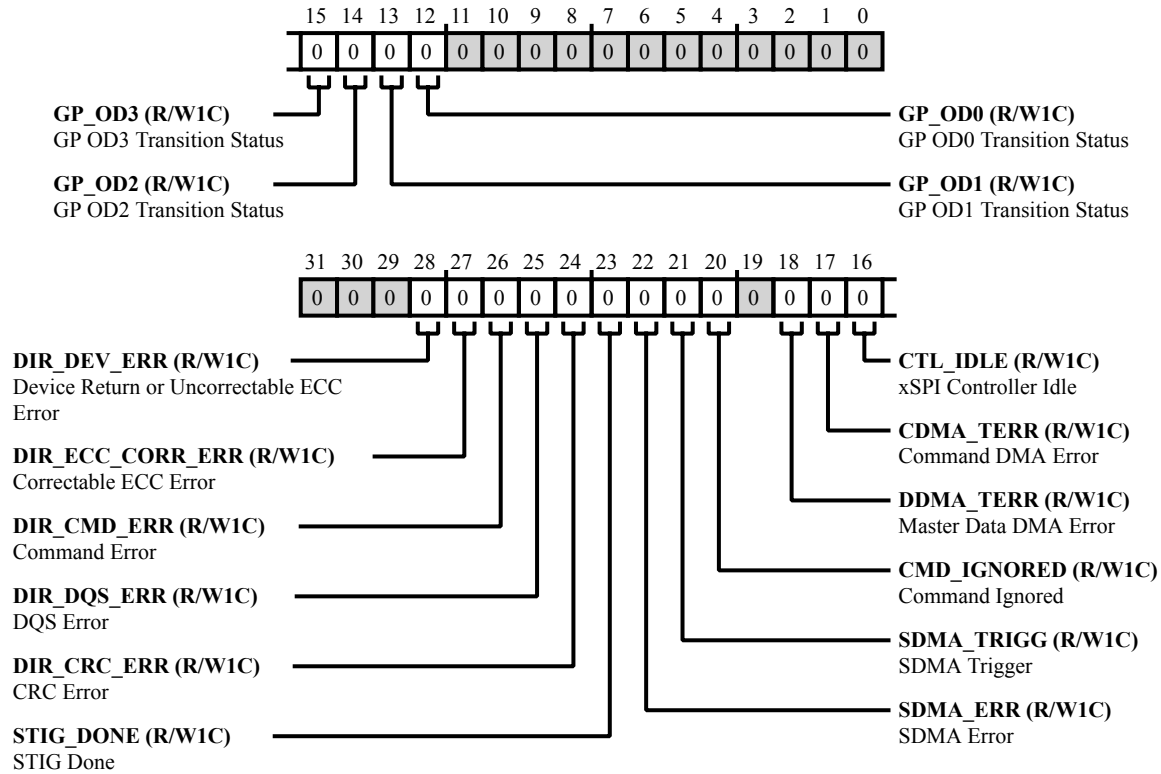


Figure 14-33: XSPI_ISTAT Register Diagram

Table 14-74: XSPI_ISTAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
28 (R/W1C)	DIR_DEV_ERR	Device Return or Uncorrectable ECC Error. The <code>XSPI_ISTAT.DIR_DEV_ERR</code> bit is set (=1) when a program operation in direct mode failed. That is, the device returned the program fail bit when the status was checked. This bit is also set when an uncorrectable ECC error occurs in direct mode.
27 (R/W1C)	DIR_ECC_CORR_ERR	Correctable ECC Error. The <code>XSPI_ISTAT.DIR_ECC_CORR_ERR</code> bit is set (=1) when a correctable ECC error occurs in direct mode.

Table 14-74: XSPI_ISTAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
26 (R/W1C)	DIR_CMD_ERR	Command Error. The XSPI_ISTAT.DIR_CMD_ERR bit is set (=1) when the host has attempted to do something that is unsupported or incorrect. For example, the host has triggered a write when the device is operating in XIP mode. The full list of error conditions is described in the Event Handling section.
25 (R/W1C)	DIR_DQS_ERR	DQS Error. The XSPI_ISTAT.DIR_DQS_ERR bit is set (=1) when an incorrect number of DQS pulses were detected during a direct mode read or status check. The status is passed from the soft PHY and could mean that the PHY is configured badly such that it expects DQS strobes but never received them or that the XSPI_PHY_GATE_LPBK_CTL.RD_DEL_SEL value is incorrect and has caused data corruption and pointer misalignment in the PHY. To resolve this issue, the PHY must be reset to clear the XSPI_PHY_OB0.DQS_UR and XSPI_PHY_OB0.DQS_OV flags and reset the read data pointers.
24 (R/W1C)	DIR_CRC_ERR	CRC Error. The XSPI_ISTAT.DIR_CRC_ERR bit is set (=1) when CRC checking is enabled and a CRC error after a read or status checking command in direct work mode has been detected.
23 (R/W1C)	STIG_DONE	STIG Done. The XSPI_ISTAT.STIG_DONE bit is set (=1) when the last instruction in a glued chain has completed.
22 (R/W1C)	SDMA_ERR	SDMA Error. The XSPI_ISTAT.SDMA_ERR bit is set (=1) when an illegal access to the target DMA interface is detected.
21 (R/W1C)	SDMA_TRIGG	SDMA Trigger. The XSPI_ISTAT.SDMA_TRIGG bit indicates when the trigger condition for the target DMA is met. It is relevant to STIG and ACMD mode only and used to instruct the host when an access on the host interface is to be performed. For details, see the Operating in ACMD or STIG mode section.
20 (R/W1C)	CMD_IGNORED	Command Ignored. In ACMD work mode, the controller detected that a command that was sent by the host was to an already busy thread and ignored it. In STIG work mode, the controller detected that a command was sent when the STIG engine was already busy and ignored it.
18 (R/W1C)	DDMA_TERR	Master Data DMA Error. ACMD mode only. The XSPI_ISTAT.DDMA_TERR bit is set (=1) when a system bus error was detected on the internal DMA interface during data reads or writes.

Table 14-74: XSPI_ISTAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
17 (R/W1C)	CDMA_TERR	Command DMA Error. Command DMA target error. ACMD mode only. The XSPI_ISTAT.CDMA_TERR bit is set (=1) when a system bus error was detected on the internal DMA interface during descriptor reads or writes.
16 (R/W1C)	CTL_IDLE	xSPI Controller Idle. This is general status. The XSPI_ISTAT.CTL_IDLE bit indicates that the xSPI controller has returned to an idle state.
15 (R/W1C)	GP_OD3	GP OD3 Transition Status. The XSPI_ISTAT.GP_OD3 bit indicates a high-to-low or low-to-high transition was detected on the XSPI_DFI_GP_OPEN_DRAIN[3] input pin.
14 (R/W1C)	GP_OD2	GP OD2 Transition Status. The XSPI_ISTAT.GP_OD2 bit indicates a high-to-low or low-to-high transition detected on the XSPI_DFI_GP_OPEN_DRAIN[2] input pin.
13 (R/W1C)	GP_OD1	GP OD1 Transition Status. The XSPI_ISTAT.GP_OD1 bit indicates a high-to-low or low-to-high transition detected on the XSPI_DFI_GP_OPEN_DRAIN[1] input pin.
12 (R/W1C)	GP_OD0	GP OD0 Transition Status. The XSPI_ISTAT.GP_OD0 bit indicates a high-to-low or low-to-high transition detected on the XSPI_DFI_GP_OPEN_DRAIN[0] input pin.

Long Polling Count Register

The `XSPI_LONGPOL_GCTL` register contains the wait count value for long polling.

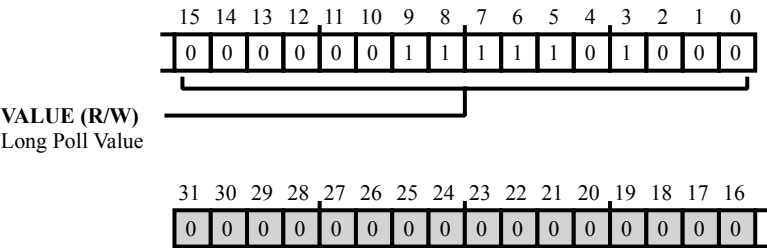


Figure 14-34: XSPI_LONGPOL_GCTL Register Diagram

Table 14-75: XSPI_LONGPOL_GCTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	VALUE	Long Poll Value. The <code>XSPI_LONGPOL_GCTL.VALUE</code> bit field indicates the number of system clock cycles after an erase/write operation has been issued before the controller starts to check device status (ready/busy and fail/pass). First status checking polling happens after at least this many number of system clock cycles. The next status checking will happen every <code>XSPI_SHORTPOLL_GCTL.VALUE</code> cycles. The long polling value should be significantly larger the short polling value.

Device Active Maximum Clock Cycle Register

The `XSPI_MINICTL_DEV_ACTIVE_MAX` register is used to introduce a maximum number of xSPI clock cycles through which the CS number is kept active (low) on the memory interface. It is sometimes referred to as tCMS or tCEM timing. It is generally only relevant for RAM devices that must be periodically refreshed. The refresh is typically handled internal to the device. But, to fit this refresh operation along with normal controller requests, the controller may need to limit its access time. The user can program this register to force the controller to obey that requirement.

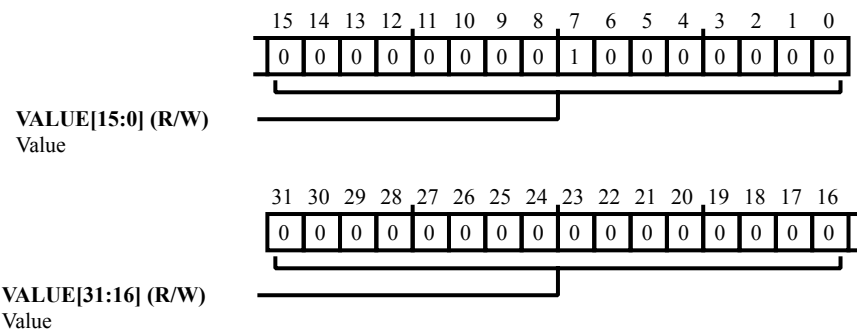


Figure 14-35: XSPI_MINICTL_DEV_ACTIVE_MAX Register Diagram

Table 14-76: XSPI_MINICTL_DEV_ACTIVE_MAX Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Value. The <code>XSPI_MINICTL_DEV_ACTIVE_MAX.VALUE</code> bit field is only valid when <code>XSPI_SEQ_GTL0.TCMS_EN</code> is set (=1). When using the STIG work mode, this is also only valid when the <code>TCMS_EN</code> bit of the STIG instruction is set. The <code>XSPI_MINICTL_DEV_ACTIVE_MAX.VALUE</code> bit field must only be enabled while working with RAM devices that require timing constraint for chip select low pulse width (the most common name is tCMS or tCEM in device specification).

Clock Mode Control Register

The `XSPI_MINICTL_CLKMODE` register defines the CPOL/CPHA settings for legacy SPI mode (defaulting to mode 0).

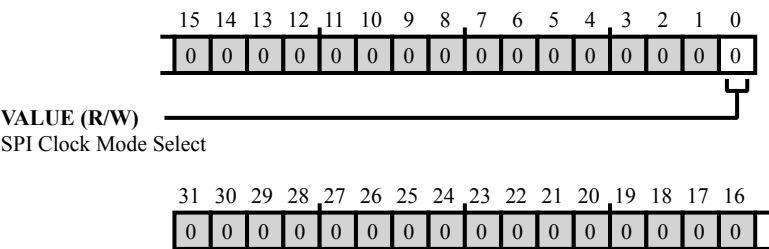


Figure 14-36: XSPI_MINICTL_CLKMODE Register Diagram

Table 14-77: XSPI_MINICTL_CLKMODE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	VALUE	SPI Clock Mode Select. The <code>XSPI_MINICTL_CLKMODE.VALUE</code> bit indicates the SPI clock mode. For DDR transfers, this bit must be cleared (=0) to meet DDR flash timings. For SDR transfers, allowable values are as follows: 0 - SPI mode 0 (clock is low when SPI bus is in idle) 1 - SPI mode 3 (clock is high when SPI bus is in idle)

Device Delay Register

The `XSPI_MINICTL_DEV_DLY` register is used to introduce relative device selection delays with respect to generated xSPI flash interface.

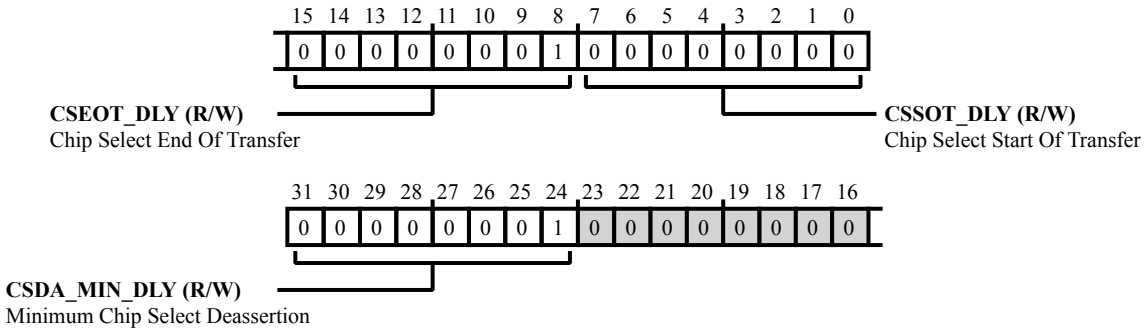


Figure 14-37: XSPI_MINICTL_DEV_DLY Register Diagram

Table 14-78: XSPI_MINICTL_DEV_DLY Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	CSDA_MIN_DLY	Minimum Chip Select Deassertion. The <code>XSPI_MINICTL_DEV_DLY.CSDA_MIN_DLY</code> bit field indicates the Minimum Chip Select (CSDA_MIN) deassertion timing.
15:8 (R/W)	CSEOT_DLY	Chip Select End Of Transfer. The <code>XSPI_MINICTL_DEV_DLY.CSEOT_DLY</code> bit field indicates the Chip Select End Of Transfer (CSEOT). It improves the CS deassertion device timing to the last active clock edge.
7:0 (R/W)	CSSOT_DLY	Chip Select Start Of Transfer. The <code>XSPI_MINICTL_DEV_DLY.CSSOT_DLY</code> bit field indicates the Chip Select Start Of Transfer (CSSOT). It improves the CS deassertion device timing to the first active clock edge.

HyperFlash Offset Register

The `XSPI_MINICTL_HF_OFFSET` register is only applicable for legacy HyperFlash or xSPI profile 2 devices and can be used to configure the reserved area of the Command/Address (CA) field.

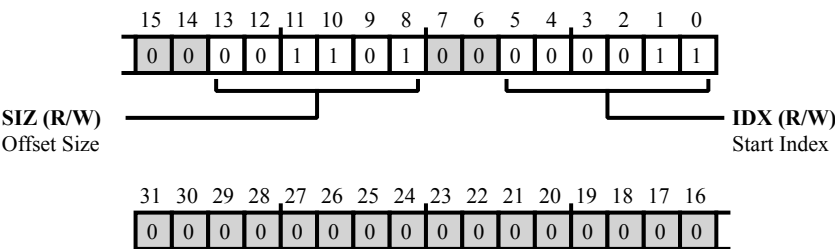


Figure 14-38: XSPI_MINICTL_HF_OFFSET Register Diagram

Table 14-79: XSPI_MINICTL_HF_OFFSET Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
13:8 (R/W)	SIZ	Offset Size. The <code>XSPI_MINICTL_HF_OFFSET.SIZ</code> bit field indicates the offset size of the reserved area in the command format.
5:0 (R/W)	IDX	Start Index. The <code>XSPI_MINICTL_HF_OFFSET.IDX</code> bit field indicates the starting index of the reserved area in the command format.

JEDEC Reset Delay Register

The `XSPI_MINICTL_JEDEC_RST_TR` register is used to introduce relative device selection delays applicable for JEDEC reset instruction.

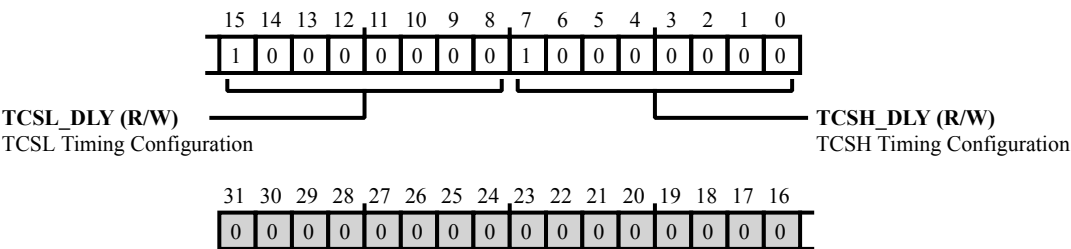


Figure 14-39: XSPI_MINICTL_JEDEC_RST_TR Register Diagram

Table 14-80: XSPI_MINICTL_JEDEC_RST_TR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:8 (R/W)	TCSL_DLY	TCSL Timing Configuration. The <code>XSPI_MINICTL_JEDEC_RST_TR.TCSL_DLY</code> bit field indicates how many xSPI clock cycles constitute tCSL timing of JEDEC reset instruction.
7:0 (R/W)	TCSH_DLY	TCSH Timing Configuration. The <code>XSPI_MINICTL_JEDEC_RST_TR.TCSH_DLY</code> bit field indicates how many xSPI clock cycles constitute tCSH timing of JEDEC reset instruction.

Hardware Reset Control Register

The `XSPI_MINICTL_RST_PIN_CTL` register configures the software controlled hardware reset.

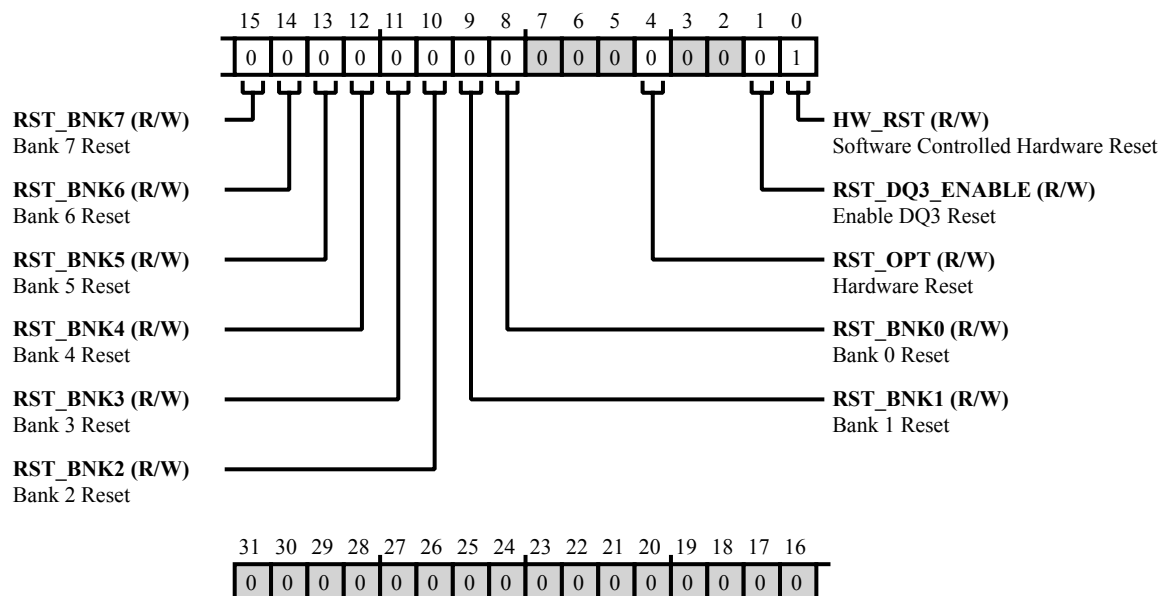


Figure 14-40: XSPI_MINICTL_RST_PIN_CTL Register Diagram

Table 14-81: XSPI_MINICTL_RST_PIN_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W)	RST_BNK7	Bank 7 Reset. The <code>XSPI_MINICTL_RST_PIN_CTL.RST_BNK7</code> bit activates software controlled hardware reset signal on bank 7 (such as a device connected to chip select bit 7). It is applicable only for devices that support the RESET pin.
		0 Disable
		1 Enable
14 (R/W)	RST_BNK6	Bank 6 Reset. The <code>XSPI_MINICTL_RST_PIN_CTL.RST_BNK6</code> bit activates software controlled hardware reset signal on bank 6 (such as a device connected to chip select bit 6). It is applicable only for devices that support the RESET pin.
		0 Disable
		1 Enable

Table 14-81: XSPI_MINICTL_RST_PIN_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
13 (R/W)	RST_BNK5	Bank 5 Reset. The XSPI_MINICTL_RST_PIN_CTL.RST_BNK5 bit activates software controlled hardware reset signal on bank 5 (such as a device connected to chip select bit 5). It is applicable only for devices that support the RESET pin.
		0 Disable
		1 Enable
12 (R/W)	RST_BNK4	Bank 4 Reset. The XSPI_MINICTL_RST_PIN_CTL.RST_BNK4 bit activates software controlled hardware reset signal on bank 0 (such as a device connected to chip select bit 4). It is applicable only for devices that support the RESET pin.
		0 Disable
		1 Enable
11 (R/W)	RST_BNK3	Bank 3 Reset. The XSPI_MINICTL_RST_PIN_CTL.RST_BNK3 bit activates software controlled hardware reset signal on bank 3 (such as a device connected to chip select bit 3). It is applicable only for devices that support the RESET pin.
		0 Disable
		1 Enable
10 (R/W)	RST_BNK2	Bank 2 Reset. The XSPI_MINICTL_RST_PIN_CTL.RST_BNK2 bit activates software controlled hardware reset signal on bank 2 (such as a device connected to chip select bit 2). It is applicable only for devices that support the RESET pin.
		0 Disable
		1 Enable
9 (R/W)	RST_BNK1	Bank 1 Reset. The XSPI_MINICTL_RST_PIN_CTL.RST_BNK1 bit activates software controlled hardware reset signal on bank 1 (such as a device connected to chip select bit 1). It is applicable only for devices that support the RESET pin.
		0 Disable
		1 Enable

Table 14-81: XSPI_MINICTRL_RST_PIN_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
8 (R/W)	RST_BNK0	Bank 0 Reset. The XSPI_MINICTRL_RST_PIN_CTL.RST_BNK0 bit activates the software controlled hardware reset signal on bank 0 (such as a device connected to chip select bit 0). It is applicable only for devices that support the RESET pin.
		0 Disable
		1 Enable
4 (R/W)	RST_OPT	Hardware Reset. The XSPI_MINICTRL_RST_PIN_CTL.RST_OPT bit indicates the hardware reset option.
		0 RESET pin. Device RESET# pin is used for toggling device hardware reset functionality.
		1 DQ3 pin. Device DQ3 pin is used for toggling device hardware reset functionality.
1 (R/W)	RST_DQ3_ENABLE	Enable DQ3 Reset. The XSPI_MINICTRL_RST_PIN_CTL.RST_DQ3_ENABLE bit enables passing RESET to the DQ3 port of the device (by switching direction of DQ3 pad).
0 (R/W)	HW_RST	Software Controlled Hardware Reset. The value of this field is directly routed to the DQ3 or RESET# output signal (depending on XSPI_MINICTRL_RST_PIN_CTL.RST_OPT). The value can be changed only when xSPI flash interface is in an idle state. The value of the software controlled hardware reset signal is overwritten when DQ3 is a valid transaction pin. The controller does not check the device hardware reset setup/hold timings - this must be ensured by the host. The host is also responsible for triggering a suitable reset method by selecting the corresponding bank number and reset method (as defined in this register). The controller does not drive RESET value during read data phase of any active transfer (as transfer direction switches in this transfer part).

Reset Recovery Delay Register

The `XSPI_MINICTL_RST_RECOV` register is used to introduce relative reset recovery delay with respect to generated xSPI flash interface.

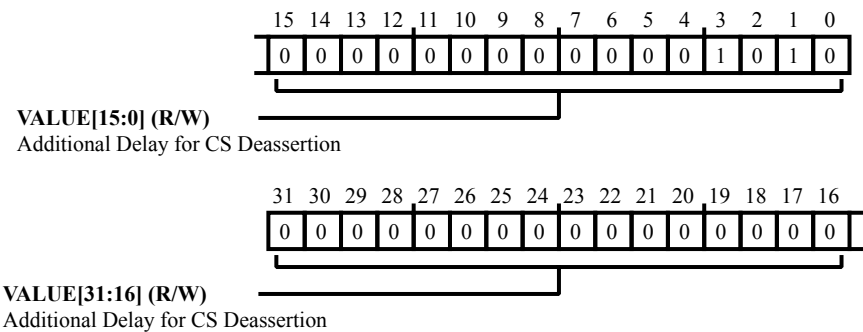


Figure 14-41: XSPI_MINICTL_RST_RECOV Register Diagram

Table 14-82: XSPI_MINICTL_RST_RECOV Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Additional Delay for CS Deassertion. The <code>XSPI_MINICTL_RST_RECOV.VALUE</code> bit field defines the additional delay for CS deassertion to accommodate device reset recovery timing.

Write Protect Register

The `XSPI_MINICTL_WP_CTL` register controls write protection.

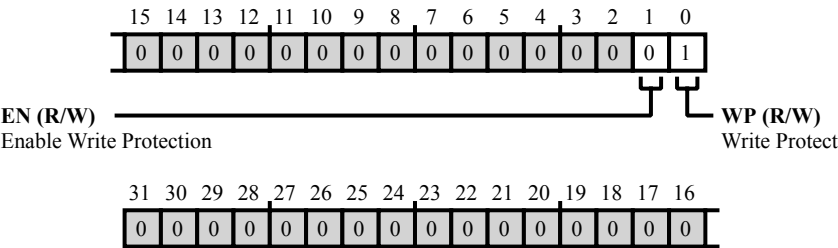


Figure 14-42: XSPI_MINICTL_WP_CTL Register Diagram

Table 14-83: XSPI_MINICTL_WP_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/W)	EN	Enable Write Protection. The <code>XSPI_MINICTL_WP_CTL.EN</code> bit enables passing the write protect signal to the device (by switching direction of DQ2 pad).
0 (R/W)	WP	Write Protect. The <code>XSPI_MINICTL_WP_CTL.WP</code> bit indicates the write protect signal for all devices. Value of this register is directly routed to the DQ2 output signal. The value can be changed only when xSPI flash interface is in an idle state. Value of the write protection signal is overwritten in case DQ2 is a valid transaction pin. The controller does not check the write protect setup/hold timings - this must be ensured by the host. The controller does not drive the write protect value during the read data phase of any active transfer. Write protection on DQ2 functionality is only supported by flash devices and when the controller is configured in single and dual SPI modes.

PHY DLL Observable Points 0 Register

The `XSPI_PHY_DLLOB0` register holds observable points in the PHY.

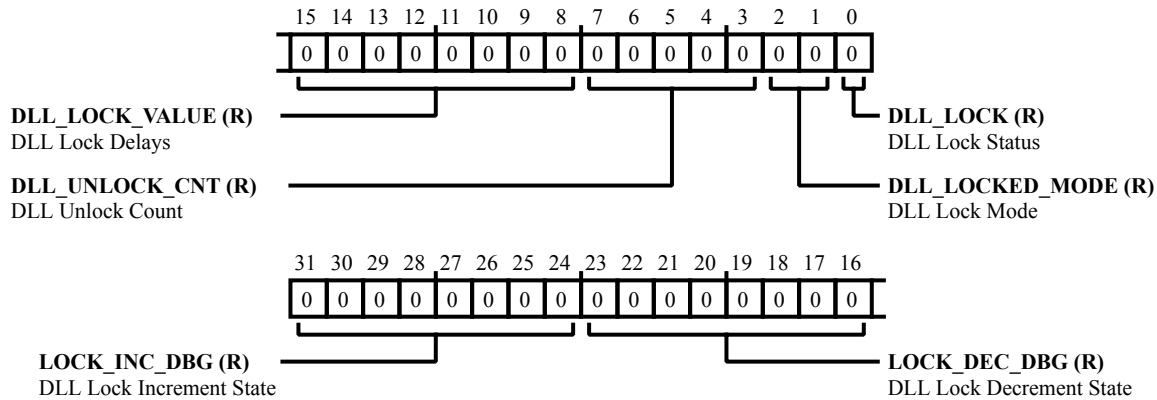


Figure 14-43: XSPI_PHY_DLLOB0 Register Diagram

Table 14-84: XSPI_PHY_DLLOB0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/NW)	LOCK_INC_DBG	DLL Lock Increment State. The <code>XSPI_PHY_DLLOB0.LOCK_INC_DBG</code> bit field holds the state of the cumulative <code>DLL_LOCK_INC</code> register when the <code>DLL_UNLOCK_CNT</code> field (bits [7:3]) of this parameter was triggered to increment or was last saturated at a value of 0x1f.
23:16 (R/NW)	LOCK_DEC_DBG	DLL Lock Decrement State. The <code>XSPI_PHY_DLLOB0.LOCK_DEC_DBG</code> bit field holds the state of the cumulative <code>DLL_LOCK_DEC</code> register when the <code>DLL_UNLOCK_CNT</code> field (bits [7:3]) of this parameter was triggered to decrement or was last saturated at a value of 0x1f.
15:8 (R/NW)	DLL_LOCK_VALUE	DLL Lock Delays. The <code>XSPI_PHY_DLLOB0.DLL_LOCK_VALUE</code> bit field indicates the number of delay elements that the DLL has determined for lock in either full clock or half clock mode. In full clock mode, this value equals the number of delay elements in one cycle. In half clock mode, this value equals the number of delay elements in one half clock cycle. In saturation mode, this value equals the maximum number of delay elements. The slaves use this value to configure their delays for the clock write and read DQS signals. This value is valid only when locking mechanism is done.
7:3 (R/NW)	DLL_UNLOCK_CNT	DLL Unlock Count. The <code>XSPI_PHY_DLLOB0.DLL_UNLOCK_CNT</code> bit field indicates the number of times that the master DLL consecutive increment or decrement value programmed in the <code>DLL_LOCK_NUM</code> field (bits [18:16]) of the <code>XSPI_PHY_DLL_MSTR_CTL</code> register has been triggered. The <code>DLL_UNLOCK_CNT</code> saturates at a value of 0x1f. Asserting the <code>XSPI_PHY_DLL_CTL.DLL_RST</code> resets this counter to 0.

Table 14-84: XSPI_PHY_DLLOB0 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
2:1 (R/NW)	DLL_LOCKED_MODE	DLL Lock Mode. The XSPI_PHY_DLLOB0.DLL_LOCKED_MODE bit field indicates the mode in which the DLL has achieved the lock.
		0 Full clock mode. The master delay line was long enough to lock on one full clock cycle of delay. In this mode, the DLL_LOCK_VALUE field (bits [15:8]) of this parameter indicates the number of delays in full clock cycles.
		1 Reserved
		2 Half clock mode. The master delay line was not long enough to lock one full cycle of delay but could lock on a half-cycle of delay. In this mode, the DLL_LOCK_VALUE field (bits [15:8]) of this parameter indicates the number of delays in one half clock cycles.
		3 Saturation mode. The master delay line was not long enough to lock on a full or a half-clock cycle. In this mode, the encoder value is fixed at the maximum delay line setting and the master DLL will be disabled. The slave delay lines continue to use the fractional delays based upon the fixed saturation value of the delay line.
0 (R/NW)	DLL_LOCK	DLL Lock Status. The XSPI_PHY_DLLOB0.DLL_LOCK bit indicates the status of the DLL locking when the DLL lock logic is found (not inc AND not dec) OR (an inc then dec) OR (a dec then inc). When XSPI_PHY_DLL_MSTR_CTL.DLL_START_PT is configured to be smaller than half a clock period, the first found (a dec then inc) is not the DLL locking point, but DLL_LOCK is asserted. 0 - DLL has not locked 1 - DLL is locked
		0 Not locked
		1 Locked

PHY DLL Observable Points 1 Register

The `XSPI_PHY_DLLOB1` register holds observable points in the PHY.

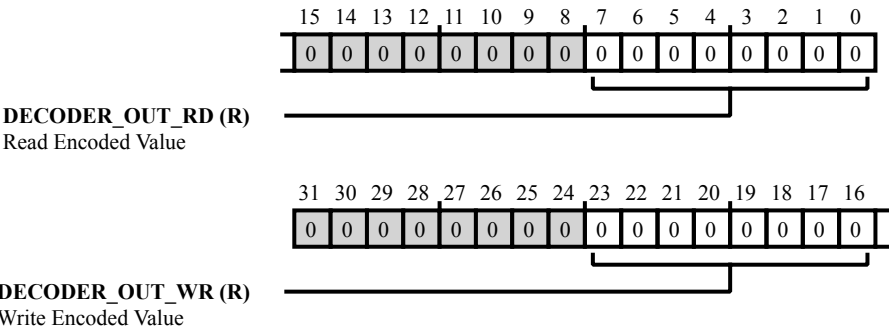


Figure 14-44: XSPI_PHY_DLLOB1 Register Diagram

Table 14-85: XSPI_PHY_DLLOB1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
23:16 (R/NW)	DECODER_OUT_WR	Write Encoded Value. The <code>XSPI_PHY_DLLOB1.DECODER_OUT_WR</code> bit field holds the encoded value for the clock write delay line for this slice.
7:0 (R/NW)	DECODER_OUT_RD	Read Encoded Value. The <code>XSPI_PHY_DLLOB1.DECODER_OUT_RD</code> bit field holds the encoded value for the read delay line for this slice.

PHY DLL Control Register

The `XSPI_PHY_DLL_CTL` register configures the resynchronization of the slave DLL of the PHY. When the PHY is used with the xSPI controller, this register is automatically updated by device discovery during initialization.

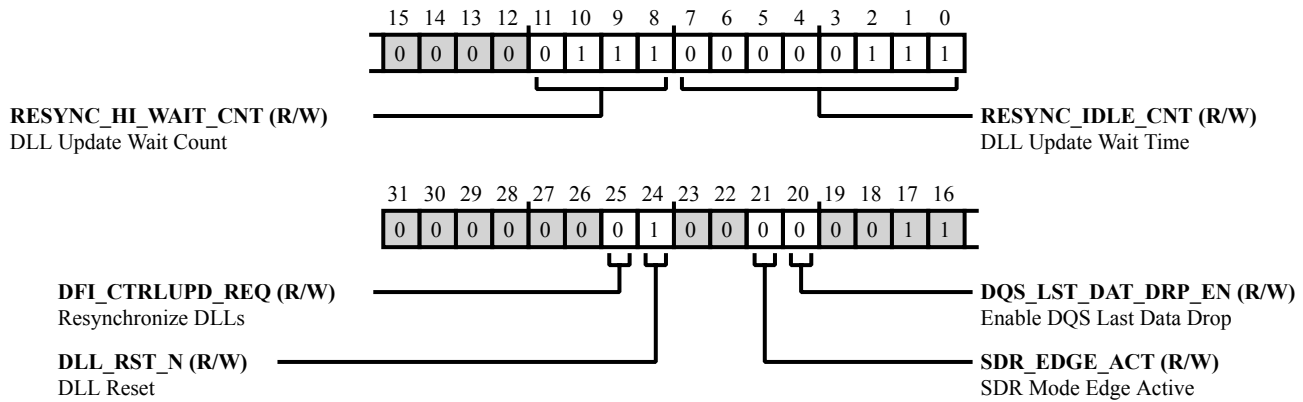


Figure 14-45: XSPI_PHY_DLL_CTL Register Diagram

Table 14-86: XSPI_PHY_DLL_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
25 (R/W)	DFI_CTRLUPD_REQ	Resynchronize DLLs. The <code>XSPI_PHY_DLL_CTL.DFI_CTRLUPD_REQ</code> bit signals to resynchronize the DLLs and read and write FIFO pointers. To send the update request to the PHY, the host must first set this bit (=1), then, wait until the bit is cleared (=0). Do not use this signal when automatic resynchronization is enabled (<code>XSPI_PHY_DLL_UPDT_CNT</code> is not zero).
24 (R/W)	DLL_RST_N	DLL Reset. The <code>XSPI_PHY_DLL_CTL.DLL_RST_N</code> bit resets the DLLs of the PHY and starts searching for lock again.
21 (R/W)	SDR_EDGE_ACT	SDR Mode Edge Active. The <code>XSPI_PHY_DLL_CTL.SDR_EDGE_ACT</code> bit indicates how the PHY samples data on the edges of the sampling clock. In SDR mode, only one sample is needed. When <code>XSPI_PHY_DLL_CTL.SDR_EDGE_ACT</code> is cleared (=0), the controller propagates data from the positive edge of the PHY sampling clock. When <code>XSPI_PHY_DLL_CTL.SDR_EDGE_ACT</code> is set (=1), the controller propagates data from the negative edge of PHY sampling clock. In DDR mode, this bit must be cleared.

Table 14-86: XSPI_PHY_DLL_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
20 (R/W)	DQS_LST_DAT_DRP_EN	Enable DQS Last Data Drop. XSPI_PHY_DLL_CTL.DQS_LST_DAT_DRP_EN is set (=1) when the flash device issues data on the negative edge of the flash clock and returns them with DQS, and the PHY is configured to sample data in DQS mode. In this case, the number of DQS edges equals to number of requested data + 1. When XSPI_PHY_DLL_CTL.DQS_LST_DAT_DRP_EN is set, the controller internally requests this redundant data at the end of the transfer cleaning up the PHY FIFO.
11:8 (R/W)	RESYNC_HI_WAIT_CNT	DLL Update Wait Count. The XSPI_PHY_DLL_CTL.RESYNC_HI_WAIT_CNT bit field defines the number of xSPI clock cycles for which the DLL update request (XSPI_PHY_DLL_CTL.DFI_CTRLUPD_REQ) must be asserted to resynchronize the DLLs and read and write FIFO pointers.
7:0 (R/W)	RESYNC_IDLE_CNT	DLL Update Wait Time. The XSPI_PHY_DLL_CTL.RESYNC_IDLE_CNT bit field defines the wait time (in terms of xSPI clock cycles) between the deassertion of the DLL update request (XSPI_PHY_DLL_CTL.DFI_CTRLUPD_REQ) and resuming traffic to the PHY.

Master DLL Control Register

The `XSPI_PHY_DLL_MSTR_CTL` register controls the master DLL logic.

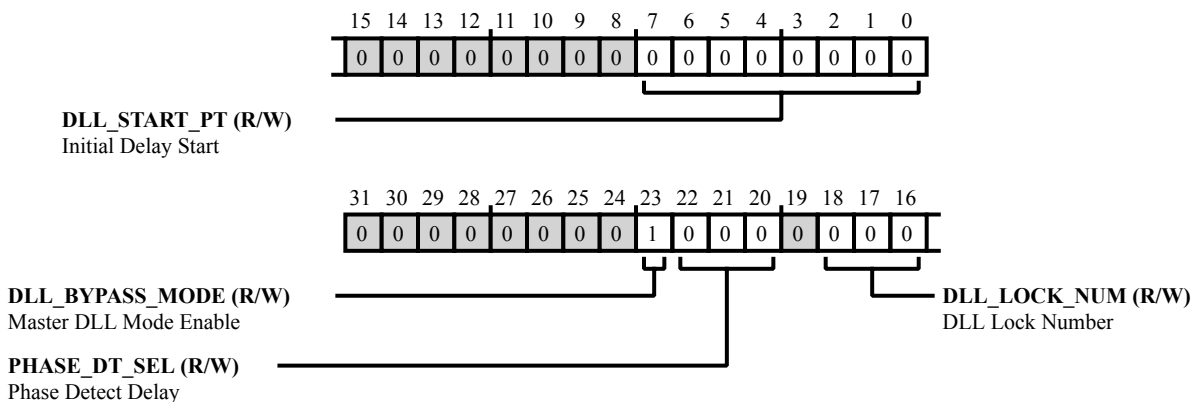


Figure 14-46: XSPI_PHY_DLL_MSTR_CTL Register Diagram

Table 14-87: XSPI_PHY_DLL_MSTR_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
23 (R/W)	DLL_BYPASS_MODE	Master DLL Mode Enable. The <code>XSPI_PHY_DLL_MSTR_CTL.DLL_BYPASS_MODE</code> bit enables DLL bypass mode control. It controls the bypass mode of the master and slave DLLs. The <code>XSPI_PHY_DLL_MSTR_CTL.DLL_BYPASS_MODE</code> bit is intended to be used only for debug.
		0 Normal operational mode. DLL functioning in normal mode where the slave delay line settings are used as fractional delay of the master delay line encoder reading of the number of delays in one cycle.
		1 Bypass mode on. Delays are defined in the <code>XSPI_PHY_DLL_SLAVE_CTL</code> register. Master DLL is disabled with only 1 delay element in its delay line. The slave delay lines decode delays in absolute delay elements rather than as fractional delays. The <code>XSPI_PHY_DLLOB0.DLL_LOCK</code> is forced high.
22:20 (R/W)	PHASE_DT_SEL	Phase Detect Delay. The <code>XSPI_PHY_DLL_MSTR_CTL.PHASE_DT_SEL</code> bit field indicates the number of delay elements to be inserted between the phase detect flip-flops. Defaults to 0x0, although the recommended value is 2 elements. But, if a lock condition is not detected, the user should increase the number of delay elements.
		0 One delay element
		1 Two delay elements

Table 14-87: XSPI_PHY_DLL_MSTR_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
		2	Three delay elements
		3	Four delay elements
		4	Five delay elements
		5	Six delay elements
		6	Seven delay elements
		7	Eight delay elements
18:16 (R/W)	DLL_LOCK_NUM	DLL Lock Number. The XSPI_PHY_DLL_MSTR_CTL.DLL_LOCK_NUM bit field holds the number of consecutive increment or decrement indications that trigger an unlock condition and increment the XSPI_PHY_DLLOB0.DLL_UNLOCK_CNT field and either the XSPI_PHY_DLLOB0.LOCK_DEC_DBG or XSPI_PHY_DLLOB0.LOCK_DEC_DBG fields.	
7:0 (R/W)	DLL_START_PT	Initial Delay Start. The XSPI_PHY_DLL_MSTR_CTL.DLL_START_PT bit field indicates the initial delay value for the DLL. This value is also used as the increment value if the initial value is less than a half-clock cycle. This field must be set such that it is not greater than 7/8ths of a clock period given the worst case element delay. For example, if the frequency is 200MHz (5ns cycle time) with a worst case element 80ps delay, this field must be configured to $= 5 * (7/8) / .080 = 54$ elements. This calculation helps determine the start point which achieves the fastest lock. However, a small value such as 0x04 may be used instead to ensure that the DLL does not lock on a harmonic. Note that with a small value like this, the initial lock time is longer. Value smaller than 0x04 may cause no lock by DLL.	

Slave DLL Control Register

The `XSPI_PHY_DLL_SLAVE_CTL` register controls the slave DLL.

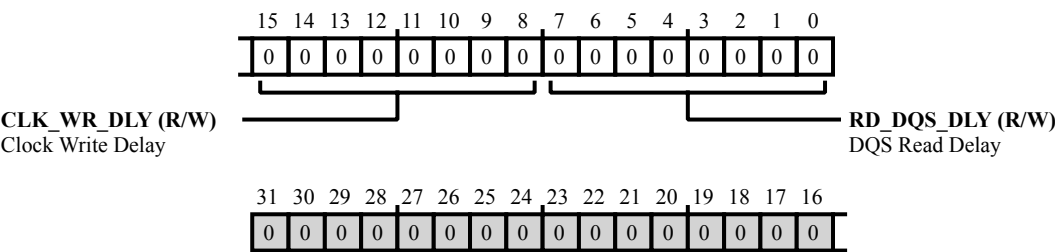


Figure 14-47: XSPI_PHY_DLL_SLAVE_CTL Register Diagram

Table 14-88: XSPI_PHY_DLL_SLAVE_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:8 (R/W)	CLK_WR_DLY	Clock Write Delay. The <code>XSPI_PHY_DLL_SLAVE_CTL.CLK_WR_DLY</code> bit field controls the clock write delay line which adjusts the write DQ bit timing in 1/256th steps of the clock period in normal DLL locked mode. In bypass mode, this field directly programs the number of delay elements.
7:0 (R/W)	RD_DQS_DLY	DQS Read Delay. The <code>XSPI_PHY_DLL_SLAVE_CTL.RD_DQS_DLY</code> bit field controls the read DQS delay which adjusts the timing in 1/256th of the clock period when in normal DLL locked mode. In bypass mode, this field directly programs the number of delay elements.

PHY DLL Resynchronization Register

The `XSPI_PHY_DLL_UPDT_CNT` register configures the resynchronization of the slave DLL of the PHY.

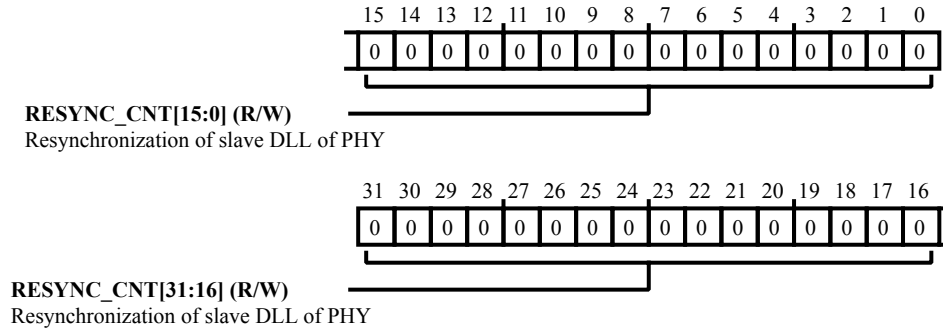


Figure 14-48: XSPI_PHY_DLL_UPDT_CNT Register Diagram

Table 14-89: XSPI_PHY_DLL_UPDT_CNT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	RESYNC_CNT	Resynchronization of slave DLL of PHY. The <code>XSPI_PHY_DLL_UPDT_CNT.RESYNC_CNT</code> bit field defines the time interval (in terms of xSPI clock) to send an update (<code>XSPI_PHY_DLL_CTL.DFI_CTRLUPD_REQ=1</code>) to the PHY to re-synchronize the slave DLL values with that of the master DLL and to also re-synchronize the read and write FIFO pointers in the read path. If the value in this field is zero, the controller will not further DLL update requests to the PHY. <code>DFI_CTRLUPD_REQ</code> signal can be controlled directly by the host using <code>XSPI_PHY_DLL_CTL.DFI_CTRLUPD_REQ</code>. Note: While this feature is enabled, access to the PHY registers shall not be performed.

PHY DQS Timing Register

The `XSPI_PHY_DQS_TR` register controls the DQS related timing.

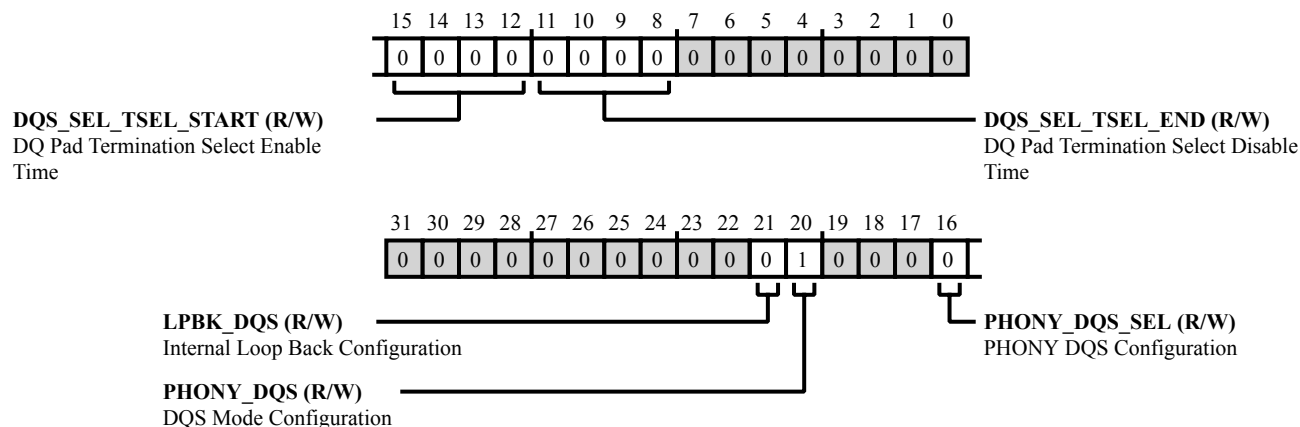


Figure 14-49: XSPI_PHY_DQS_TR Register Diagram

Table 14-90: XSPI_PHY_DQS_TR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
21 (R/W)	LPBK_DQS	Internal Loop Back Configuration. The <code>XSPI_PHY_DQS_TR.LPBK_DQS</code> bit is used in conjunction with bits 22 and 20 to control how read data is sampled by the PHY. The <code>XSPI_PHY_DQS_TR.LPBK_DQS</code> bit is only valid when bit 20 is set (=1), meaning it is only relevant when read data is not being sampled by DQS from the memory device. If bit 20 is cleared (=0), the <code>XSPI_PHY_DQS_TR.LPBK_DQS</code> bit should also be cleared. When using the PHY with the xSPI controller, this bit must be set to the same value as bit 20. This bit selects which internal source is used by the PHY to sample the read data. This is internally generated and is passed out of the PHY via <code>rebar_opad</code>. If bit 22 of this register is '0', then it will be internally looped back within the rebar pad. If bit 22 of this register is '1', the <code>rebar_opad</code> will be passed through the rebar pad and it will be the responsibility of the integrator to loop that back to the PHY into the <code>lpbk_dqs</code> pin. 0 - Use phony DQS for data capture 1 - Use <code>lpbk_dqs</code> for data capture. [/list].

Table 14-90: XSPI_PHY_DQS_TR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
20 (R/W)	PHONY_DQS	DQS Mode Configuration. The XSPI_PHY_DQS_TR.PHONY_DQS bit is used in conjunction with bits 22 and 21 to control how read data is sampled by the PHY. The XSPI_PHY_DQS_TR.PHONY_DQS bit selects whether the read data sent by the memory device is sampled by DQS supplied by the memory device, or by a signal locally generated within the PHY.
		0 Use DQS from device
		1 Use internally generated DQS
16 (R/W)	PHONY_DQS_SEL	PHONY DQS Configuration. When XSPI_PHY_DQS_TR.PHONY_DQS_SEL is cleared (=0), the PHONY_DQS is synchronous with rising edge of the CLK_PHY before sending to the entry flops. When XSPI_PHY_DQS_TR.PHONY_DQS_SEL is set (=1), the PHONY_DQS signal is synchronous with the falling edge of CLK_PHY before sending to the entry flops.
15:12 (R/W)	DQS_SEL_TSEL_START	DQ Pad Termination Select Enable Time. The XSPI_PHY_DQS_TR.DQS_SEL_TSEL_START bit field defines the DQ pad dynamic termination select enable time. Larger values add greater delay to when tsel turns on. Each bit changes the output enable time by a 1/2 cycle resolution.
11:8 (R/W)	DQS_SEL_TSEL_END	DQ Pad Termination Select Disable Time. The XSPI_PHY_DQS_TR.DQS_SEL_TSEL_END bit field defines the DQ pad dynamic termination select disable time. Larger values increase the delay to when tsel turns off. Each bit changes the output enable time by a 1/2 cycle resolution.

PHY DQ Timing Register

The `XSPI_PHY_DQ_TR` register controls the DQ related timing.

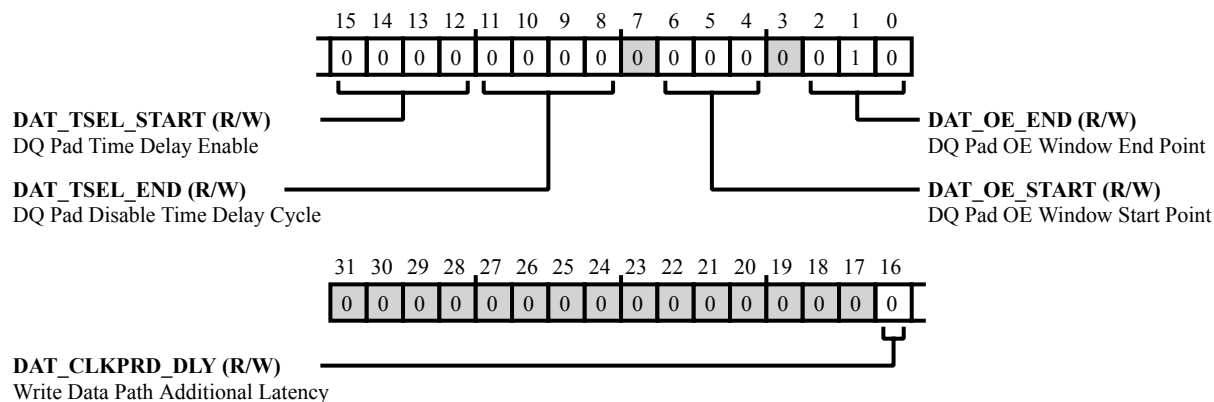


Figure 14-50: XSPI_PHY_DQ_TR Register Diagram

Table 14-91: XSPI_PHY_DQ_TR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
16 (R/W)	DAT_CLKPRD_DLY	Write Data Path Additional Latency. The <code>XSPI_PHY_DQ_TR.DAT_CLKPRD_DLY</code> bit field defines additional latency on the write data path. It also adds a clock cycle delay for the data OE path which is equivalent of adding 2 to <code>DAT_OE_END</code> and <code>DAT_OE_START</code> .
15:12 (R/W)	DAT_TSEL_START	DQ Pad Time Delay Enable. The <code>XSPI_PHY_DQ_TR.DAT_TSEL_START</code> bit field defines the DQ pad dynamic termination select enable time. Larger values add greater delay to when <code>tsel</code> turns on. Each bit changes the output enable time by a 1/2 cycle resolution.
11:8 (R/W)	DAT_TSEL_END	DQ Pad Disable Time Delay Cycle. The <code>XSPI_PHY_DQ_TR.DAT_TSEL_END</code> bit field defines the DQ pad dynamic termination select disable time. Larger values increase the delay to when <code>tsel</code> turns off. Each bit changes the output enable time by a 1/2 cycle resolution.
6:4 (R/W)	DAT_OE_START	DQ Pad OE Window Start Point. The <code>XSPI_PHY_DQ_TR.DAT_OE_START</code> bit field adjusts the starting point of the DQ pad output enable window. Lower numbers pull the rising edge earlier in time and larger numbers cause the rising edge to be delayed. Each bit changes the output enable time by a 1/2 cycle resolution.
2:0 (R/W)	DAT_OE_END	DQ Pad OE Window End Point. The <code>XSPI_PHY_DQ_TR.DAT_OE_END</code> bit field adjusts the ending point of the DQ pad output enable window. Lower numbers pull the falling edge earlier in time and larger numbers cause the falling edge to be delayed. Each bit changes the output enable time by a 1/2 cycle resolution.

PHY Features Register

The `XSPI_PHY_FEATURES` register shows the available hardware features.

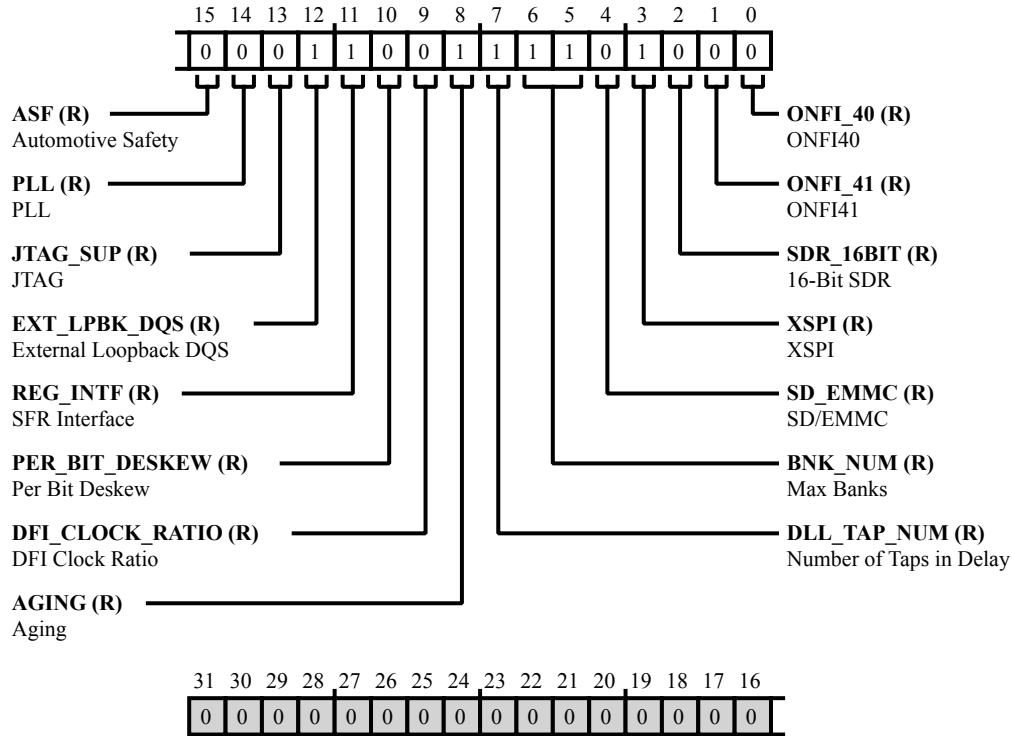


Figure 14-51: XSPI_PHY_FEATURES Register Diagram

Table 14-92: XSPI_PHY_FEATURES Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/NW)	ASF	Automotive Safety. The <code>XSPI_PHY_FEATURES.ASF</code> bit indicates support for an automotive safety feature.
14 (R/NW)	PLL	PLL. The <code>XSPI_PHY_FEATURES.PLL</code> bit indicates support for PLL.
13 (R/NW)	JTAG_SUP	JTAG. The <code>XSPI_PHY_FEATURES.JTAG_SUP</code> bit indicates support for JTAG multi-plexing.
12 (R/NW)	EXT_LPBK_DQS	External Loopback DQS. The <code>XSPI_PHY_FEATURES.EXT_LPBK_DQS</code> bit indicates support for an external LPBK_DQS IO pad.

Table 14-92: XSPI_PHY_FEATURES Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
11 (R/NW)	REG_INTF	SFR Interface. The <code>XSPI_PHY_FEATURES.REG_INTF</code> bit indicates the SFR interface type. This is an encoded value.
		0 DFI
		1 APB
10 (R/NW)	PER_BIT_DESKEW	Per Bit Deskew. The <code>XSPI_PHY_FEATURES.PER_BIT_DESKEW</code> bit indicates support for per-bit deskew.
9 (R/NW)	DFI_CLOCK_RATIO	DFI Clock Ratio. The <code>XSPI_PHY_FEATURES.DFI_CLOCK_RATIO</code> bit indicates support for a clock ratio on the DFI interface. This is an encoded value.
		0 1:1
		1 1:2
8 (R/NW)	AGING	Aging. The <code>XSPI_PHY_FEATURES.AGING</code> bit indicates support for aging in delay lines.
7 (R/NW)	DLL_TAP_NUM	Number of Taps in Delay. The <code>XSPI_PHY_FEATURES.DLL_TAP_NUM</code> bit indicates the number of taps in the delay line.
		0 128
		1 256
6:5 (R/NW)	BNK_NUM	Max Banks. The <code>XSPI_PHY_FEATURES.BNK_NUM</code> bit field indicates the maximum number of banks supported by hardware. This is an encoded value.
		0 One bank
		1 Two banks
		2 Four banks
		3 Eight banks
4 (R/NW)	SD_EMMC	SD/EMMC. The <code>XSPI_PHY_FEATURES.SD_EMMC</code> bit indicates support for SD/eMMC.
3 (R/NW)	XSPI	XSPI. The <code>XSPI_PHY_FEATURES.XSPI</code> bit indicates support for xSPI.

Table 14-92: XSPI_PHY_FEATURES Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
2 (R/NW)	SDR_16BIT	16-Bit SDR. The XSPI_PHY_FEATURES.SDR_16BIT bit indicates support for 16-bit in ONFI SDR work mode.
1 (R/NW)	ONFI_41	ONFI41. The XSPI_PHY_FEATURES.ONFI_41 bit indicates support for ONFI4.1 - NAND flash.
0 (R/NW)	ONFI_40	ONFI40. The XSPI_PHY_FEATURES.ONFI_40 bit indicates support for ONFI4.0 - NAND flash.

PHY Gate Loopback Control Register

The `XSPI_PHY_GATE_LPBK_CTL` register controls the gate and loopback control related timing.

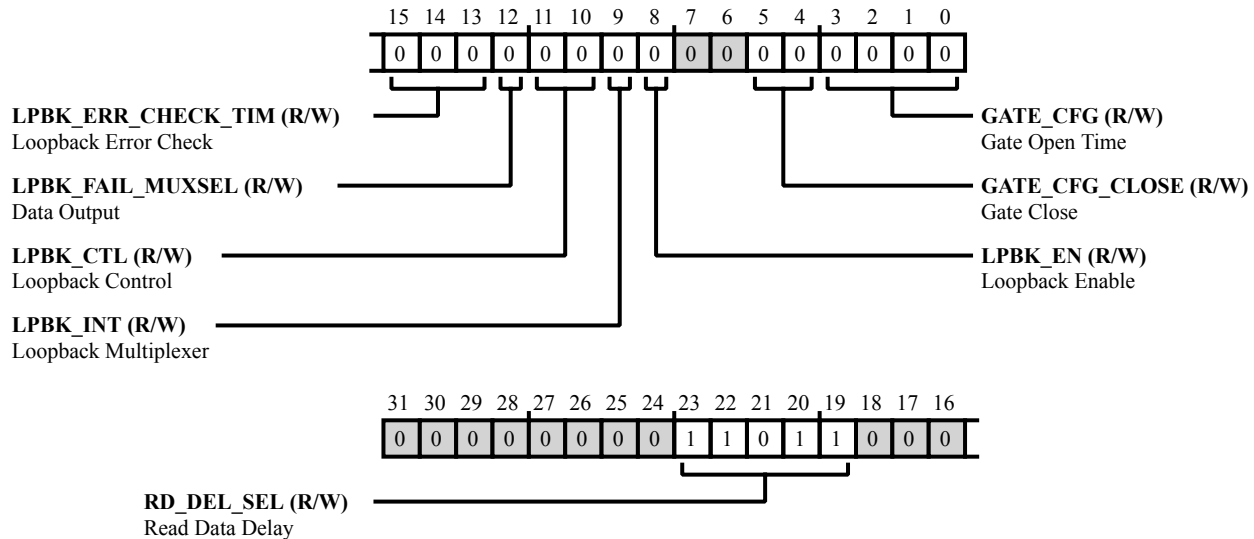


Figure 14-52: XSPI_PHY_GATE_LPBK_CTL Register Diagram

Table 14-93: XSPI_PHY_GATE_LPBK_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
23:19 (R/W)	RD_DEL_SEL	Read Data Delay. The <code>XSPI_PHY_GATE_LPBK_CTL.RD_DEL_SEL</code> bit defines the read data delay. It indicates the number of cycles to delay the <code>dfi_rddata_en</code> signal prior to enabling the read FIFO. After this delay, the read pointers begin incrementing the read FIFO. If <code>sync_method</code> is set (=1) the value of this field must take into account the synchronization time of the pointers in the entry FIFO (adding three clock cycles should be sufficient).
15:13 (R/W)	LPBK_ERR_CHECK_TIM	Loopback Error Check. The <code>XSPI_PHY_GATE_LPBK_CTL.LPBK_ERR_CHECK_TIM</code> bit field indicates the cycle delay between the LFSR and loopback error check logic to ensure that the LFSR sourced data and data being looped back arrive at the same clock cycle for comparison. This value is related to the <code>XSPI_PHY_GATE_LPBK_CTL.RD_DEL_SEL</code> field, and is equal to $7 - RD_DEL_SEL$.
12 (R/W)	LPBK_FAIL_MUXSEL	Data Output. The <code>XSPI_PHY_GATE_LPBK_CTL.LPBK_FAIL_MUXSEL</code> bit selects the data output type for <code>XSPI_PHY_OB0.LPBK_DQ_DAT</code> .
		0 Return the expected data
		1 Return the actual data

Table 14-93: XSPI_PHY_GATE_LPBK_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
11:10 (R/W)	LPBK_CTL	Loopback Control. The XSPI_PHY_GATE_LPBK_CTL.LPBK_CTL bit field indicates the loopback control.
		0 Normal operation mode
		1 Loopback start. Enable loopback write mode.
		2 Loopback stop. Stop loopback to check error register.
		3 Clear. Clear loopback registers.
9 (R/W)	LPBK_INT	Loopback Multiplexer. The XSPI_PHY_GATE_LPBK_CTL.LPBK_INT bit controls the loopback read multiplexer.
		0 External loopback
		1 Internal loopback
8 (R/W)	LPBK_EN	Loopback Enable. The XSPI_PHY_GATE_LPBK_CTL.LPBK_EN bit controls the internal write multiplexer.
		0 Normal operation
		1 Enable loopback
5:4 (R/W)	GATE_CFG_CLOSE	Gate Close. The XSPI_PHY_GATE_LPBK_CTL.GATE_CFG_CLOSE bit field allows extending the closing of the DQS gate. Recommended value is zero. rebar_dfi - Read enable output signal. It is an internal signal between the xSPI PHY and xSPI controller. dfi_cebar - Peripheral select output (CS) signal. It is an internal signal between the xSPI PHY and xSPI controller.
3:0 (R/W)	GATE_CFG	Gate Open Time. Coarse adjust of gate open time. The XSPI_PHY_GATE_LPBK_CTL.GATE_CFG bit field is the number of cycles to delay the dfi_rddata_en signal prior to opening the gate in full cycle increments. Decreasing this value pulls the gate earlier in time. This field must be programmed such that the gate signal lands in the valid DQS gate window.

PHY Global Control Register

The `XSPI_PHY_GCTL` register handles the global control settings for the PHY.

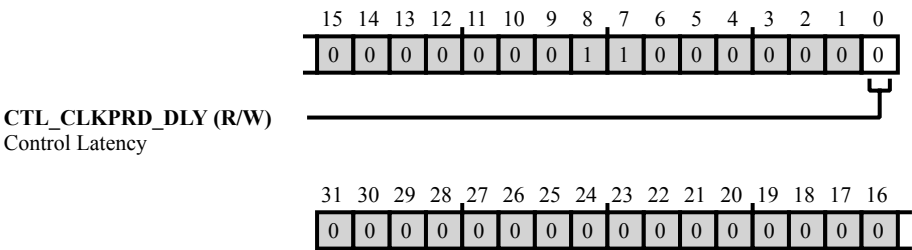


Figure 14-53: XSPI_PHY_GCTL Register Diagram

Table 14-94: XSPI_PHY_GCTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	CTL_CLKPRD_DLY	Control Latency. The <code>XSPI_PHY_GCTL.CTL_CLKPRD_DLY</code> bit indicates an additional latency on the control signals WE/RE/CE/WP.

PHY GPIO Control Register 0

The `XSPI_PHY_GPIO_CTL0` register is a general purpose register. The [31:0] vector is brought to the PHY I/Os. These pins can be used to control any static settings that may be required for the connected I/O pads.

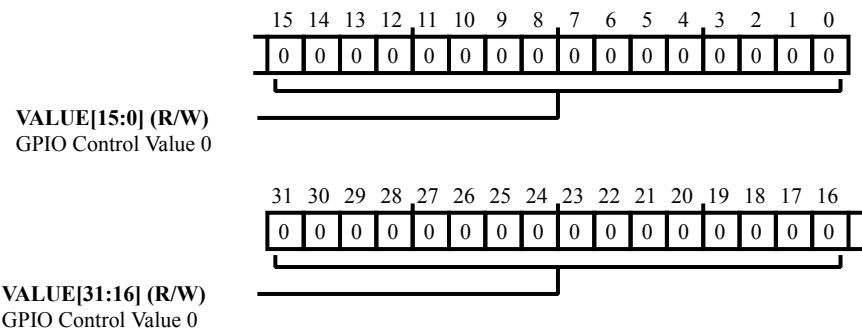


Figure 14-54: XSPI_PHY_GPIO_CTL0 Register Diagram

Table 14-95: XSPI_PHY_GPIO_CTL0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	GPIO Control Value 0. The [31:0] vector is brought to the PHY I/Os. The <code>XSPI_PHY_GPIO_CTL0.VALUE</code> bit field can be used to control any static settings that may be required for the connected IO pads.

PHY GPIO Control Register 1

The `XSPI_PHY_GPIO_CTL1` register is a general purpose register. The [31:0] vector is brought to the PHY I/Os. These pins can be used to control any static settings that may be required for the connected I/O pads.

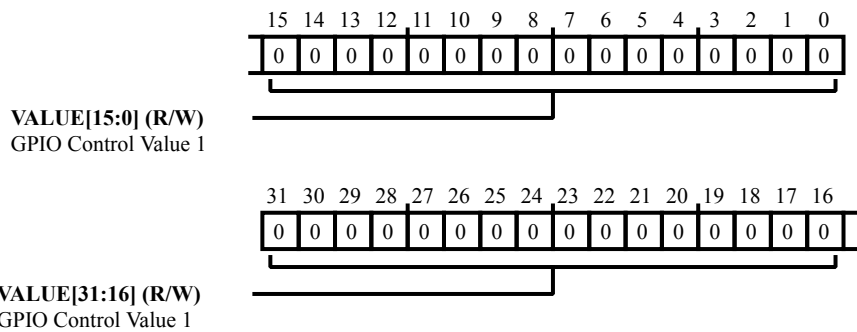


Figure 14-55: XSPI_PHY_GPIO_CTL1 Register Diagram

Table 14-96: XSPI_PHY_GPIO_CTL1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	GPIO Control Value 1. The [31:0] vector is brought to the PHY I/Os. The <code>XSPI_PHY_GPIO_CTL1.VALUE</code> bit field can be used to control any static settings that may be required for the connected IO pads.

PHY GPIO Status Register 0

The `XSPI_PHY_GPIO_STAT0` register is a general purpose register. A [31:0] vector is brought from the PHY I/Os to this register. This can be used as a status register.

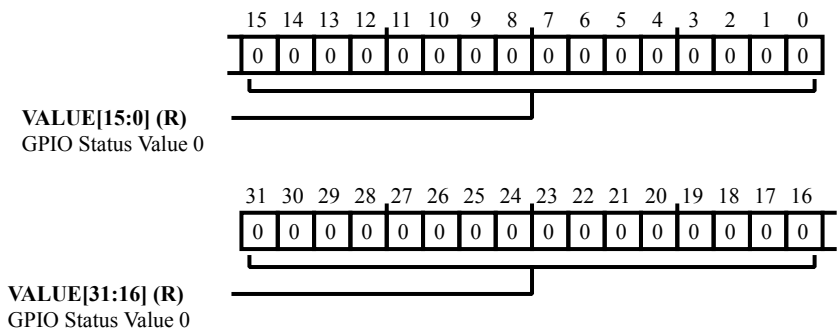


Figure 14-56: XSPI_PHY_GPIO_STAT0 Register Diagram

Table 14-97: XSPI_PHY_GPIO_STAT0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	GPIO Status Value 0. A [31:0] vector is brought from the PHY I/Os to this register. The <code>XSPI_PHY_GPIO_STAT0.VALUE</code> bit field can be used as a status register.

PHY GPIO Status Register 1

The `XSPI_PHY_GPIO_STAT1` register is a general purpose register. A [31:0] vector is brought from the PHY I/Os to this register. This can be used as a status register.

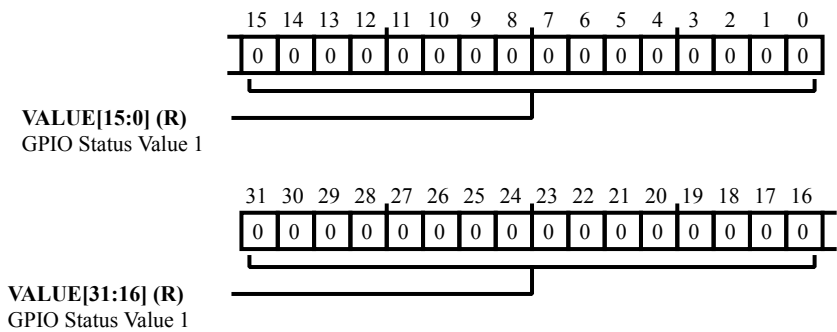


Figure 14-57: XSPI_PHY_GPIO_STAT1 Register Diagram

Table 14-98: XSPI_PHY_GPIO_STAT1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	GPIO Status Value 1. A [31:0] vector is brought from the PHY I/Os to this register. The <code>XSPI_PHY_GPIO_STAT1.VALUE</code> bit field can be used as a status register.

PHY Global Termination Control Register

The `XSPI_PHY_GTSEL` register handles the global control settings for the termination selects for reads.

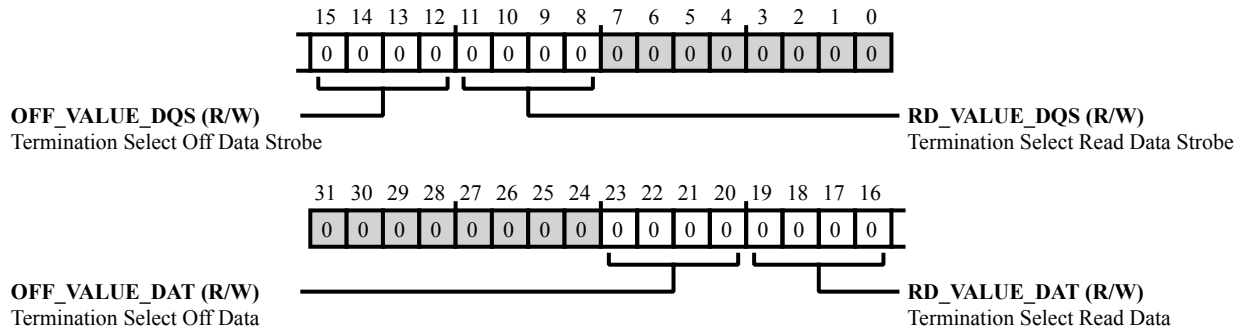


Figure 14-58: XSPI_PHY_GTSEL Register Diagram

Table 14-99: XSPI_PHY_GTSEL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
23:20 (R/W)	OFF_VALUE_DAT	Termination Select Off Data. The <code>XSPI_PHY_GTSEL.OFF_VALUE_DAT</code> bit field indicates the termination select off value for the data.
19:16 (R/W)	RD_VALUE_DAT	Termination Select Read Data. The <code>XSPI_PHY_GTSEL.RD_VALUE_DAT</code> bit field indicates the termination select read value for the data.
15:12 (R/W)	OFF_VALUE_DQS	Termination Select Off Data Strobe. The <code>XSPI_PHY_GTSEL.OFF_VALUE_DQS</code> bit field indicates the termination select off value for the data strobe.
11:8 (R/W)	RD_VALUE_DQS	Termination Select Read Data Strobe. The <code>XSPI_PHY_GTSEL.RD_VALUE_DQS</code> bit field indicates the termination select read value for the data strobe.

PHY DQS Input Enable Timing Register

The `XSPI_PHY_IE_TR` register controls the DQS related timing.

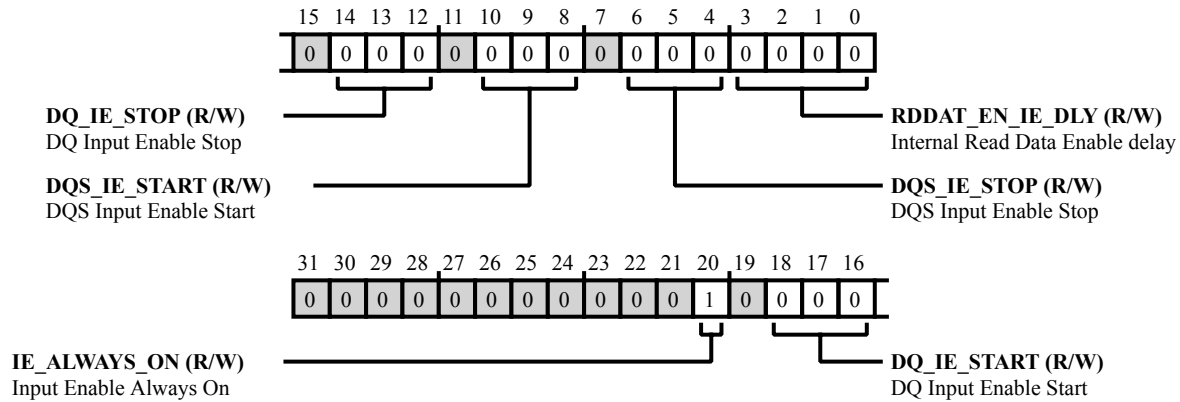


Figure 14-59: XSPI_PHY_IE_TR Register Diagram

Table 14-100: XSPI_PHY_IE_TR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
20 (R/W)	IE_ALWAYS_ON	Input Enable Always On. The <code>XSPI_PHY_IE_TR.IE_ALWAYS_ON</code> bit forces the input enable(s) to be on always.
18:16 (R/W)	DQ_IE_START	DQ Input Enable Start. The <code>XSPI_PHY_IE_TR.DQ_IE_START</code> bit field indicates the start position for the DQ input enable.
14:12 (R/W)	DQ_IE_STOP	DQ Input Enable Stop. The <code>XSPI_PHY_IE_TR.DQ_IE_STOP</code> bit field indicates the stop position for the DQ input enable.
10:8 (R/W)	DQS_IE_START	DQS Input Enable Start. The <code>XSPI_PHY_IE_TR.DQS_IE_START</code> bit field indicates the start position for the DQS input enable.
6:4 (R/W)	DQS_IE_STOP	DQS Input Enable Stop. The <code>XSPI_PHY_IE_TR.DQS_IE_STOP</code> bit field indicates the stop position for the DQS input enable.
3:0 (R/W)	RDDAT_EN_IE_DLY	Internal Read Data Enable delay. The <code>XSPI_PHY_IE_TR.RDDAT_EN_IE_DLY</code> bit field indicates the number of clocks of delay for the <code>DFI_RDDATA_EN</code> signal to line it up with the true (normal) DFI read data position. The controller must deliver an early version of the read data enable to allow time for the input pads to turn on and this field allows the PHY to create the original timing.

PHY Observable Points Register

The `XSPI_PHY_OB0` register holds observable points in the PHY.

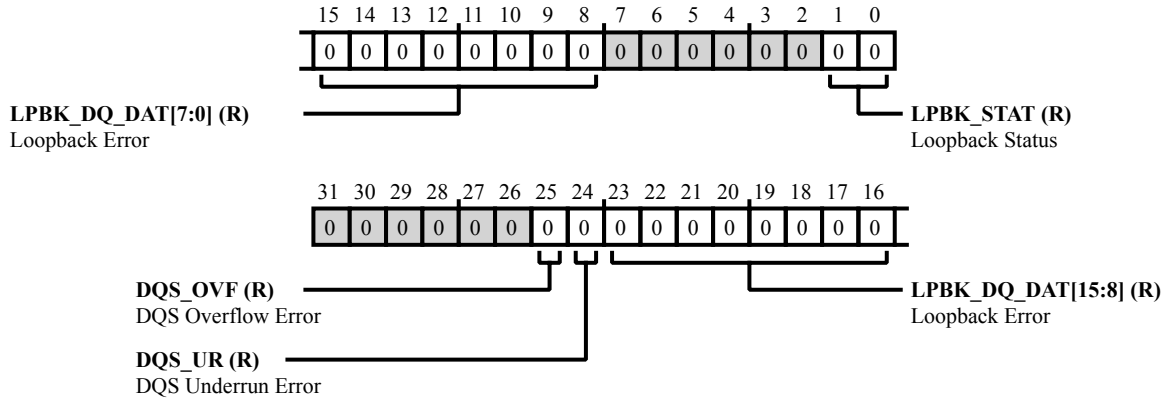


Figure 14-60: XSPI_PHY_OB0 Register Diagram

Table 14-101: XSPI_PHY_OB0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
25 (R/NW)	DQS_OVF	DQS Overflow Error. The <code>XSPI_PHY_OB0.DQS_OVF</code> bit indicates that the logic gate was closed too late. For example, the number of DQS strobes exceed the capacity of the entry FIFO. It indicates that <code>XSPI_PHY_GATE_LPBK_CTL.RD_DEL_SE</code> signal value is too high and <code>DFI_RDDATA</code> are corrupted. It is possible that overflow status is asserted with underrun status - in such case the overflow takes the precedence. The <code>XSPI_PHY_DLL_CTL.DLL_RST_N</code> bit or <code>RST_N</code> signal clears this flag.
24 (R/NW)	DQS_UR	DQS Underrun Error. The <code>XSPI_PHY_OB0.DQS_UR</code> bit indicates that the logic gate had to be forced closed. It indicates that either the DQS strobe did not appear during read or <code>XSPI_PHY_GATE_LPBK_CTL.RD_DEL_SEL</code> signal value is too low and <code>DFI_RDDATA</code> are corrupted. The <code>XSPI_PHY_DLL_CTL.DLL_RST_N</code> bit or <code>RST_N</code> signal clears this flag.
23:8 (R/NW)	LPBK_DQ_DAT	Loopback Error. If errors are encountered in loopback test, the <code>XSPI_PHY_OB0.LPBK_DQ_DAT</code> bit field reports the actual data or the expected data, depending on the setting of the <code>XSPI_PHY_GATE_LPBK_CTL.LPBK_FAIL_MUXSEL</code> bit. This field is not clear by the clear state of the loopback. If there are no errors in loopback test, the value is zero (or value from previous state).

Table 14-101: XSPI_PHY_OB0 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
1:0 (R/NW)	LPBK_STAT	Loopback Status. The XSPI_PHY_OB0.LPBK_STAT bit field indicates the loopback status. Bit0 - loopback start; Defines the status of the loopback mode. 0 = Not in loopback mode; 1 = In loopback mode. Bit1 - loopback status; Defines the status of the loopback mode. 0 = Last Loopback test had no errors; 1 = Last loopback test contained data errors.

PHY Revision ID Register

The `XSPI_PHY_REVID` register contains the release identification number.

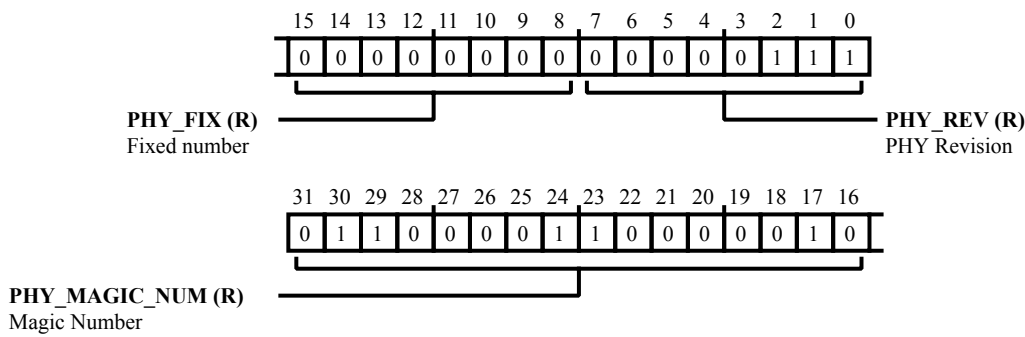


Figure 14-61: XSPI_PHY_REVID Register Diagram

Table 14-102: XSPI_PHY_REVID Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:16 (R/NW)	PHY_MAGIC_NUM	Magic Number. The <code>XSPI_PHY_REVID.PHY_MAGIC_NUM</code> bit field indicates the magic number.
15:8 (R/NW)	PHY_FIX	Fixed number. The <code>XSPI_PHY_REVID.PHY_FIX</code> bit field indicates the fixed number (minor revision number).
7:0 (R/NW)	PHY_REV	PHY Revision. The <code>XSPI_PHY_REVID.PHY_REV</code> bit field indicates the PHY revision number.

PHY Static Aging Register

The `XSPI_PHY_STATIC_TGL` register controls the static aging feature of the PHY.

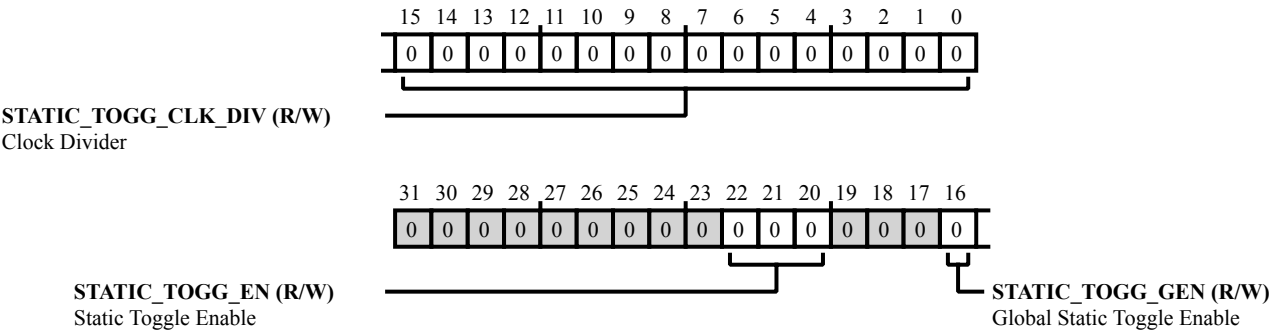


Figure 14-62: XSPI_PHY_STATIC_TGL Register Diagram

Table 14-103: XSPI_PHY_STATIC_TGL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
22:20 (R/W)	STATIC_TOGG_EN	Static Toggle Enable. The <code>XSPI_PHY_STATIC_TGL.STATIC_TOGG_EN</code> bit field enables the toggle signal during static activity. When cleared (=0), the feature is disabled. Bit 0 - Master delay line enable Bit 1 - Read path delay line enable Bit 2 - Write path delay line enable
16 (R/W)	STATIC_TOGG_GEN	Global Static Toggle Enable. The <code>XSPI_PHY_STATIC_TGL.STATIC_TOGG_GEN</code> bit enables the toggle signal during static activity.
15:0 (R/W)	STAT- IC_TOGG_CLK_DIV	Clock Divider. The <code>XSPI_PHY_STATIC_TGL.STATIC_TOGG_CLK_DIV</code> bit field indicates the clock divider used to create the toggle signal.

PHY Deskew Write Register

The `XSPI_PHY_WR_DESKEW_PAD_CTL0` register holds the values of the phase detect block for each DQ bit on the write path.

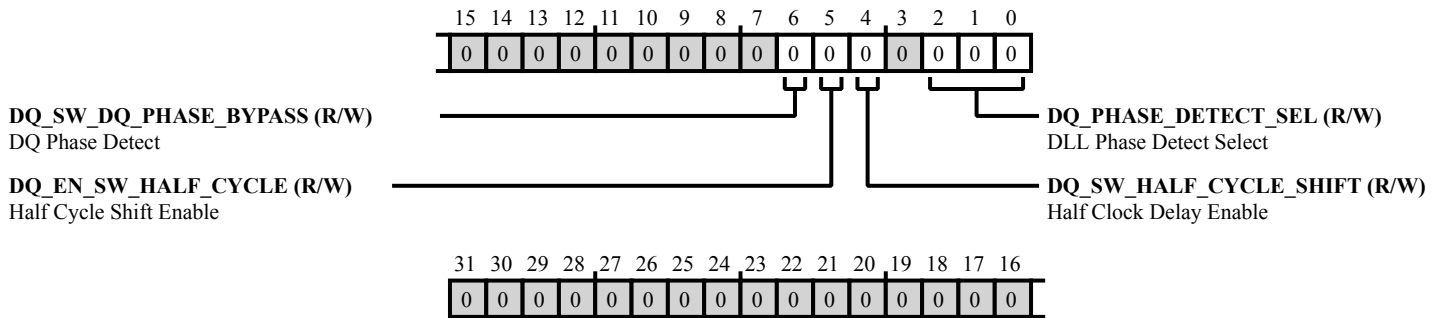


Figure 14-63: XSPI_PHY_WR_DESKEW_PAD_CTL0 Register Diagram

Table 14-104: XSPI_PHY_WR_DESKEW_PAD_CTL0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
6 (R/W)	DQ_SW_DQ_PHASE_BYPASS	DQ Phase Detect. When <code>XSPI_PHY_WR_DESKEW_PAD_CTL0.DQ_SW_DQ_PHASE_BYPASS</code> is set (=1), the clock write delay line configuration is used to determine the <code>HALF_CYCLE_SHIFT</code> . When <code>XSPI_PHY_WR_DESKEW_PAD_CTL0.DQ_SW_DQ_PHASE_BYPASS</code> is cleared (=0), the phase detect circuit is used to determine the <code>HALF_CYCLE_SHIFT</code> .
		0 Use phase detect circuit
		1 Use the clock write delay line configuration. A delay line configuration of 0x00-0x7f means <code>HALF_CYCLE_SHIFT</code> = 0 and a delay line configuration of 0x80-0xff means <code>HALF_CYCLE_SHIFT</code> = 1.
5 (R/W)	DQ_EN_SW_HALF_CYCLE	Half Cycle Shift Enable. When <code>XSPI_PHY_WR_DESKEW_PAD_CTL0.DQ_EN_SW_HALF_CYCLE</code> is set (=1), it enables the software half cycle shift. This determines whether the write data is transferred to the clock write domain on the positive or negative edge of the PHY clock. This field is valid when <code>DQ_SW_DQ_PHASE_BYPASS</code> =0. Note: If the user chooses to control the half cycle shift manually, it is important that <code>DQ_SW_HALF_CYCLE_SHIFT</code> = 0 if the delay is less than a 1/2 cycle and <code>DQ_SW_HALF_CYCLE_SHIFT</code> = 1 if the delay is greater than a 1/2 cycle. It is recommended to allow the hardware to control this automatically.
		0 Hardware automatically controls shifting needed for the write level delay line.
		1 <code>DQ_SW_HALF_CYCLE_SHIFT</code> field defines the shift

Table 14-104: XSPI_PHY_WR_DESKEW_PAD_CTL0 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
4 (R/W)	DQ_SW_HALF_CYCLE_SHIFT	Half Clock Delay Enable. When XSPI_PHY_WR_DESKEW_PAD_CTL0.DQ_SW_HALF_CYCLE_SHIFT is set (=1), a half clock delay is added to the write data path.
		0 No effect
		1 Adds a half clock delay to the write data path
2:0 (R/W)	DQ_PHASE_DETECT_SEL	DLL Phase Detect Select. The XSPI_PHY_WR_DESKEW_PAD_CTL0.DQ_PHASE_DETECT_SEL bit field indicates the DLL phase detect selector for DQ generation to handle the clock domain crossing between the clock and clock write signal. It indicates the number of delay elements to be inserted between the phase detect flip-flops. Defaults to 0x0.
		0 One delay element
		1 Two delay elements
		2 Three delay elements
		3 Four delay elements
		4 Five delay elements
		5 Six delay elements
		6 Seven delay elements
		7 Eight delay elements

Program Sequence Configuration Register 0

The `XSPI_PROG_SEQ_CFG0` register configures the program sequence for profile 1 and SPI NAND in ACMD and direct work modes.

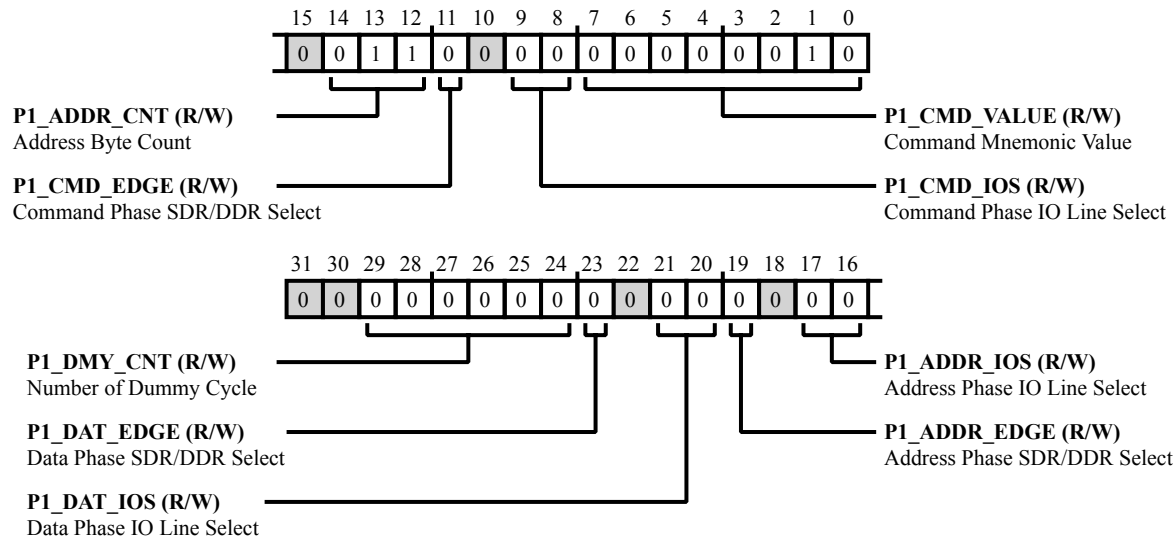


Figure 14-64: XSPI_PROG_SEQ_CFG0 Register Diagram

Table 14-105: XSPI_PROG_SEQ_CFG0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:24 (R/W)	P1_DMY_CNT	Number of Dummy Cycle. The <code>XSPI_PROG_SEQ_CFG0.P1_DMY_CNT</code> bit field indicates the number of dummy cycles in profile 1. If 0, dummy cycles are disabled.
23 (R/W)	P1_DAT_EDGE	Data Phase SDR/DDR Select. The <code>XSPI_PROG_SEQ_CFG0.P1_DAT_EDGE</code> bit selects between SDR/DDR mode for the data phase.
21:20 (R/W)	P1_DAT_IOS	Data Phase IO Line Select. The <code>XSPI_PROG_SEQ_CFG0.P1_DAT_IOS</code> bit field indicates the number of data lines used to send the data phase.
		0 One data line used (for example, serial)
		1 Two data lines used
		2 Four data lines used
		3 Eight data lines used
19 (R/W)	P1_ADDR_EDGE	Address Phase SDR/DDR Select. The <code>XSPI_PROG_SEQ_CFG0.P1_ADDR_EDGE</code> bit selects between SDR/DDR mode for the address phase.

Table 14-105: XSPI_PROG_SEQ_CFG0 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
17:16 (R/W)	P1_ADDR_IOS	Address Phase IO Line Select. The XSPI_PROG_SEQ_CFG0.P1_ADDR_IOS bit field indicates the number of data lines used to send the address phase.
		0 One data line used (for example, serial)
		1 Two data lines used
		2 Four data lines used
		3 Eight data lines used
14:12 (R/W)	P1_ADDR_CNT	Address Byte Count. The XSPI_PROG_SEQ_CFG0.P1_ADDR_CNT bit field indicates the number of address bytes.
11 (R/W)	P1_CMD_EDGE	Command Phase SDR/DDR Select. The XSPI_PROG_SEQ_CFG0.P1_CMD_EDGE bit selects between SDR/DDR mode for the command phase.
9:8 (R/W)	P1_CMD_IOS	Command Phase IO Line Select. The XSPI_PROG_SEQ_CFG0.P1_CMD_IOS bit field indicates the number of data lines used to send the command phase.
		0 One data line used (for example, serial)
		1 Two data lines used
		2 Four data lines used
		3 Eight data lines used
7:0 (R/W)	P1_CMD_VALUE	Command Mnemonic Value. The XSPI_PROG_SEQ_CFG0.P1_CMD_VALUE bit field indicates the command mnemonic value.

Program Sequence Configuration Register 1

The `XSPI_PROG_SEQ_CFG1` register configures the program sequence for profile 1 and SPI NAND in ACMD and direct work modes.

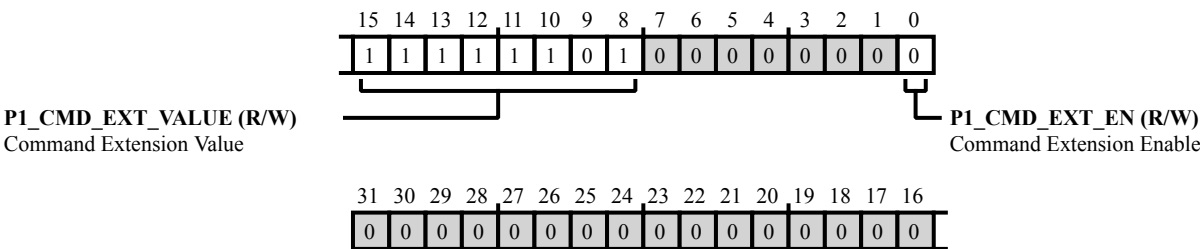


Figure 14-65: XSPI_PROG_SEQ_CFG1 Register Diagram

Table 14-106: XSPI_PROG_SEQ_CFG1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:8 (R/W)	P1_CMD_EXT_VALUE	Command Extension Value. The <code>XSPI_PROG_SEQ_CFG1.P1_CMD_EXT_VALUE</code> bit field indicates the command extension value.
0 (R/W)	P1_CMD_EXT_EN	Command Extension Enable. The <code>XSPI_PROG_SEQ_CFG1.P1_CMD_EXT_EN</code> bit enables the command extension.

Program Sequence Configuration Register 2

The `XSPI_PROG_SEQ_CFG2` register configures the program sequence for profile 2 in ACMD and direct work modes.

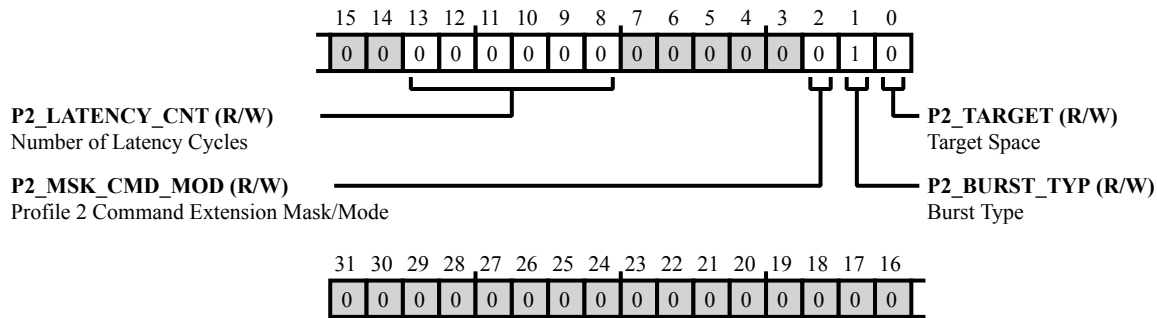


Figure 14-66: XSPI_PROG_SEQ_CFG2 Register Diagram

Table 14-107: XSPI_PROG_SEQ_CFG2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration				
13:8 (R/W)	P2_LATENCY_CNT	Number of Latency Cycles. The XSPI_PROG_SEQ_CFG2.P2_LATENCY_CNT bit field indicates the number of latency cycles for profile 2 - HR only. When XSPI_PROG_SEQ_CFG2.P2_LATENCY_CNT is cleared (=0), latency cycles are disabled. Configure the XSPI_PROG_SEQ_CFG2.P2_LATENCY_CNT bit field to N-1, where N is the number of latency clock cycles expected by the memory device.				
2 (R/W)	P2_MSK_CMD_MOD	Profile 2 Command Extension Mask/Mode. The XSPI_PROG_SEQ_CFG2.P2_MSK_CMD_MOD bit indicates the profile 2 command extension variant. This influences bits [44:40] of the CA. When XSPI_PROG_SEQ_CFG2.P2_MSK_CMD_MOD is set (=1), CA[44:40] is configured to all ones. In direct work mode, when XSPI_PROG_SEQ_CFG2.P2_MSK_CMD_MOD is cleared (=0), bits [44:40] of the CA are configured to (sAWADDR[45:41] logically AND'ed with DAC_ADDR_MASK[12:8]). In ACMD work mode, when XSPI_PROG_SEQ_CFG2.P2_MSK_CMD_MOD is cleared (=0), bits [44:40] of the CA are cleared (= 0).				
1 (R/W)	P2_BURST_TYP	Burst Type. The XSPI_PROG_SEQ_CFG2.P2_BURST_TYP bit indicates the burst type; it corresponds to the 45th Command/Address (CA) bit assignment. Although the user can set wrapped burst, the controller itself does not currently support wrapping bursts and therefore must always have this bit set (=1). <table><tr><td>0</td><td>Wrapped burst</td></tr><tr><td>1</td><td>Linear burst</td></tr></table>	0	Wrapped burst	1	Linear burst
0	Wrapped burst					
1	Linear burst					

Table 14-107: XSPI_PROG_SEQ_CFG2 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	P2_TARGET	Target Space. The XSPI_PROG_SEQ_CFG2 . P2_TARGET bit indicates the target space, it corresponds to the 46th Command/Address (CA) bit assignment.
		0 Memory space
		1 Register space

Read Sequence Configuration Register 0

The `XSPI_READ_SEQ_CFG0` register configures the read sequence for profile 1 and SPI NAND in ACMD and direct work modes.

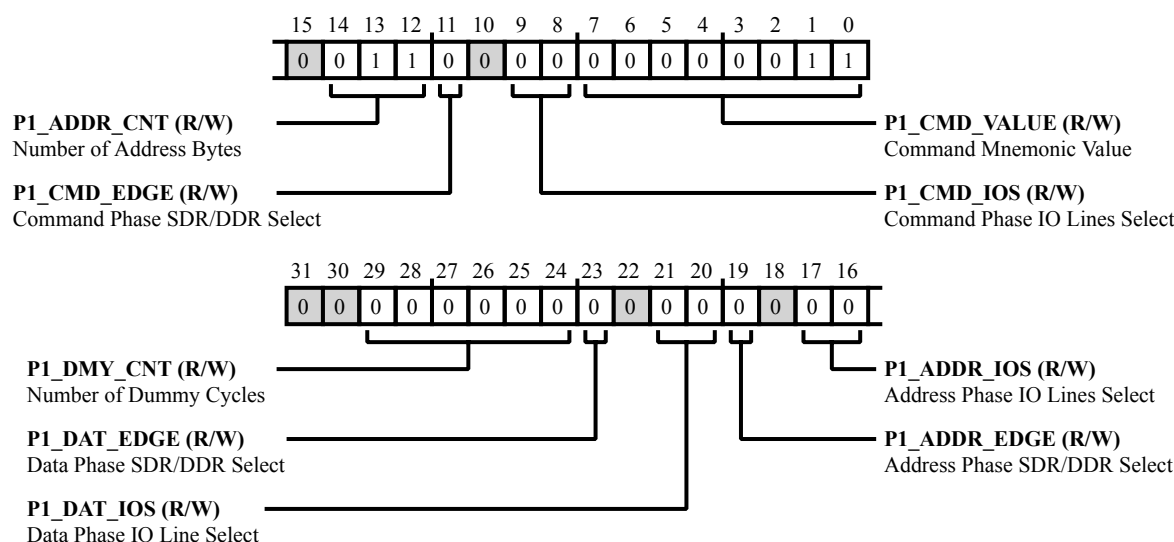


Figure 14-67: XSPI_READ_SEQ_CFG0 Register Diagram

Table 14-108: XSPI_READ_SEQ_CFG0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:24 (R/W)	P1_DMY_CNT	Number of Dummy Cycles. The <code>XSPI_READ_SEQ_CFG0.P1_DMY_CNT</code> bit field indicates the number of dummy cycles. If 0 - dummy cycles disabled. This field is used when sending mode-bits is disabled. Otherwise, the <code>read_seq_p1_mb_dummy_cnt</code> must be used.
23 (R/W)	P1_DAT_EDGE	Data Phase SDR/DDR Select. The <code>XSPI_READ_SEQ_CFG0.P1_DAT_EDGE</code> bit selects between SDR/DDR mode for data phase.
21:20 (R/W)	P1_DAT_IOS	Data Phase IO Line Select. The <code>XSPI_READ_SEQ_CFG0.P1_DAT_IOS</code> bit field indicates the number of data lines used to send the data phase.
	0	One data line used (for example, serial)
	1	Two data lines used
	2	Four data lines used
	3	Eight data lines used

Table 14-108: XSPI_READ_SEQ_CFG0 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
19 (R/W)	P1_ADDR_EDGE	Address Phase SDR/DDR Select. The XSPI_READ_SEQ_CFG0.P1_ADDR_EDGE bit selects between SDR/DDR mode for address phase.
17:16 (R/W)	P1_ADDR_IOS	Address Phase IO Lines Select. The XSPI_READ_SEQ_CFG0.P1_ADDR_IOS bit field indicates the number of data lines used to send the address phase.
		0 One data line used (for example, serial)
		1 Two data lines used
		2 Four data lines used
		3 Eight data lines used
14:12 (R/W)	P1_ADDR_CNT	Number of Address Bytes. The XSPI_READ_SEQ_CFG0.P1_ADDR_CNT bit field indicates the number of address bytes.
11 (R/W)	P1_CMD_EDGE	Command Phase SDR/DDR Select. The XSPI_READ_SEQ_CFG0.P1_CMD_EDGE bit selects between SDR/DDR mode for command phase.
9:8 (R/W)	P1_CMD_IOS	Command Phase IO Lines Select. The XSPI_READ_SEQ_CFG0.P1_CMD_IOS bit field indicates the number of data lines used to send the command phase.
		0 One data line used (for example, serial)
		1 Two data lines used
		2 Four data lines used
		3 Eight data lines used
7:0 (R/W)	P1_CMD_VALUE	Command Mnemonic Value. The XSPI_READ_SEQ_CFG0.P1_CMD_VALUE bit field indicates the command mnemonic value.

Read Sequence Configuration Register 1

The `XSPI_READ_SEQ_CFG1` register configures the read sequence for profile 1 and SPI NAND in ACMD and direct work modes.

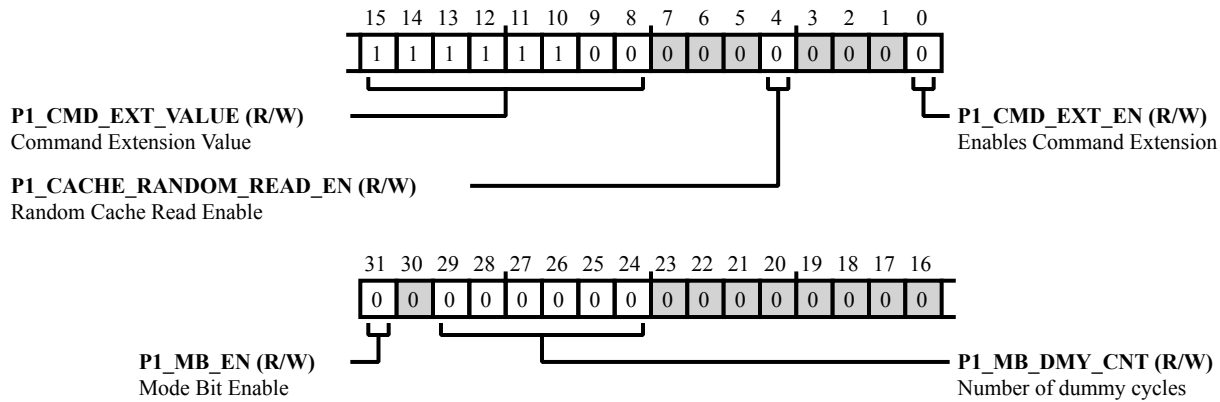


Figure 14-68: XSPI_READ_SEQ_CFG1 Register Diagram

Table 14-109: XSPI_READ_SEQ_CFG1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	P1_MB_EN	Mode Bit Enable. When <code>XSPI_READ_SEQ_CFG1.P1_MB_EN</code> is set (=1), the mode bits as defined in the <code>XSPI_XIP_GCTL.XIP_DIS_MB_VAL</code> field, are sent following the address bytes.
29:24 (R/W)	P1_MB_DMY_CNT	Number of dummy cycles. The <code>XSPI_READ_SEQ_CFG1.P1_MB_DMY_CNT</code> bit field indicates the number of dummy cycles. When <code>XSPI_READ_SEQ_CFG1.P1_MB_DMY_CNT</code> is cleared (=0), dummy cycles are disabled. This field is used when the sending mode bits function is enabled.
15:8 (R/W)	P1_CMD_EXT_VALUE	Command Extension Value. The <code>XSPI_READ_SEQ_CFG1.P1_CMD_EXT_VALUE</code> bit enables the command extension value.
4 (R/W)	P1_CACHE_RANDOM_READ_EN	Random Cache Read Enable. The <code>XSPI_READ_SEQ_CFG1.P1_CACHE_RANDOM_READ_EN</code> bit changes behavior of the read sequence to utilize the read page cache random/read page cache random last commands. This field is not used in direct mode. This bit is only applicable to the SPI NAND profile.

Table 14-109: XSPI_READ_SEQ_CFG1 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	P1_CMD_EXT_EN	Enables Command Extension. The XSPI_READ_SEQ_CFG1.P1_CMD_EXT_EN bit enables command extension.

Read Sequence Configuration Register 2

The `XSPI_READ_SEQ_CFG2` register configures the read sequence for profile 2 in ACMD and direct work modes.

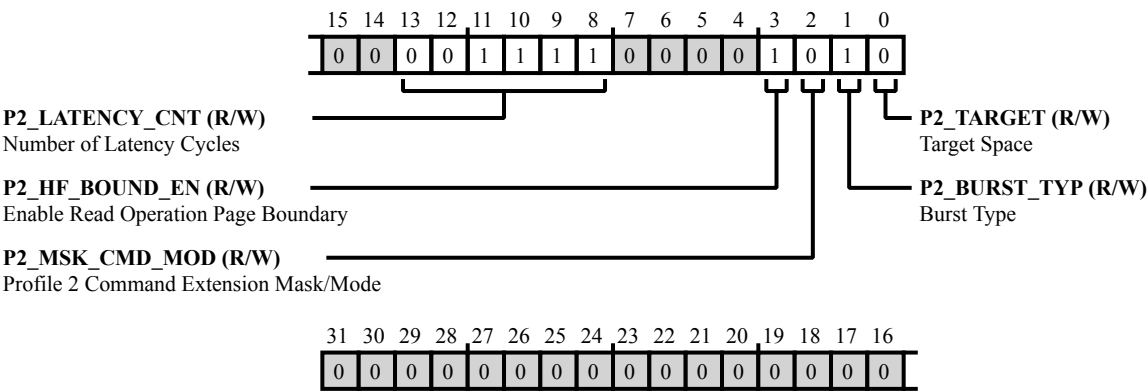


Figure 14-69: XSPI_READ_SEQ_CFG2 Register Diagram

Table 14-110: XSPI_READ_SEQ_CFG2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
13:8 (R/W)	P2_LATENCY_CNT	Number of Latency Cycles. The <code>XSPI_READ_SEQ_CFG2.P2_LATENCY_CNT</code> bit field indicates the number of latency cycles. When <code>XSPI_READ_SEQ_CFG2.P2_LATENCY_CNT</code> is cleared (=0), latency clock cycles are disabled. Configure <code>XSPI_READ_SEQ_CFG2.P2_LATENCY_CNT</code> to N-1, where N is the number of latency clock cycles expected by the memory device.
3 (R/W)	P2_HF_BOUND_EN	Enable Read Operation Page Boundary. The <code>XSPI_READ_SEQ_CFG2.P2_HF_BOUND_EN</code> bit field is used by the controller to calculate read transaction crossing page boundary. It is valid only when profile 2 - HF is selected.
		0 Disable
		1 Enable

Table 14-110: XSPI_READ_SEQ_CFG2 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
2 (R/W)	P2_MSK_CMD_MOD	Profile 2 Command Extension Mask/Mode. The XSPI_READ_SEQ_CFG2.P2_MSK_CMD_MOD bit field indicates the profile 2 command extension variant. This bit influences bits [44:40] of Command/Address. When XSPI_READ_SEQ_CFG2.P2_MSK_CMD_MOD is set (=1), bits [44:40] of Command/Address is set to 1. In direct work mode, when XSPI_READ_SEQ_CFG2.P2_MSK_CMD_MOD is cleared (=0), bits [44:40] of Command/Address is set to (sARADDR[45:41] logically AND'ed with dac_addr_mask[12:8]). In ACMD work mode, when XSPI_READ_SEQ_CFG2.P2_MSK_CMD_MOD is cleared (=0), bits [44:40] of Command/Address is cleared.
1 (R/W)	P2_BURST_TYP	Burst Type. The XSPI_READ_SEQ_CFG2.P2_BURST_TYP bit indicates the burst type; it corresponds to 45th Command/Address (CA) bit assignment. Although the user can set wrapped burst, the controller itself does not currently support wrapping bursts; therefore, configure XSPI_READ_SEQ_CFG2.P2_BURST_TYP = 1.
		0 Wrapped burst
		1 Linear burst
0 (R/W)	P2_TARGET	Target Space. The XSPI_READ_SEQ_CFG2.P2_TARGET bit indicates the target space; it corresponds to 46th Command/Address (CA) bit assignment.
		0 Memory space
		1 Register space

Revision ID Register

The `XSPI_REVID` register contains the release identification number.

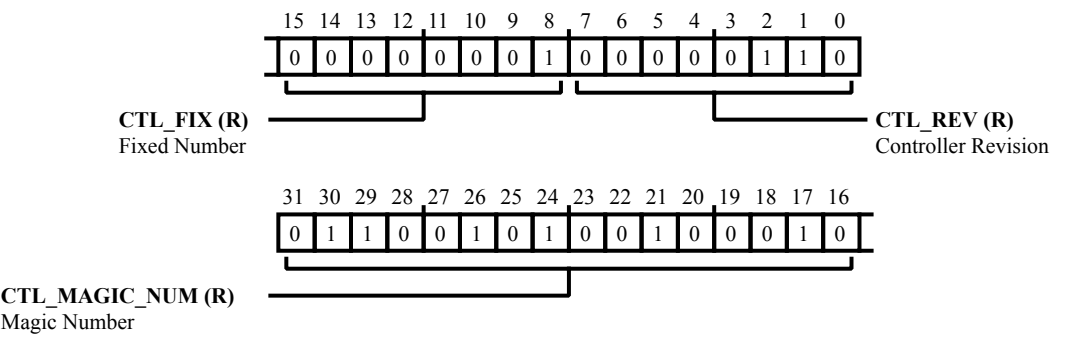


Figure 14-70: XSPI_REVID Register Diagram

Table 14-111: XSPI_REVID Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:16 (R/NW)	CTL_MAGIC_NUM	Magic Number. The <code>XSPI_REVID.CTL_MAGIC_NUM</code> bit field indicates the controller's magic number. It is a unique number characteristic for the xSPI controller.
15:8 (R/NW)	CTL_FIX	Fixed Number. The <code>XSPI_REVID.CTL_FIX</code> bit field indicates the fixed number (minor revision number).
7:0 (R/NW)	CTL_REV	Controller Revision. The <code>XSPI_REVID.CTL_REV</code> bit field indicates the controller revision number.

Reset Sequence Configuration Register 0

The `XSPI_RST_SEQ_CFG0` register configures the reset sequence for profile 1 and SPI NAND in ACMD work mode.

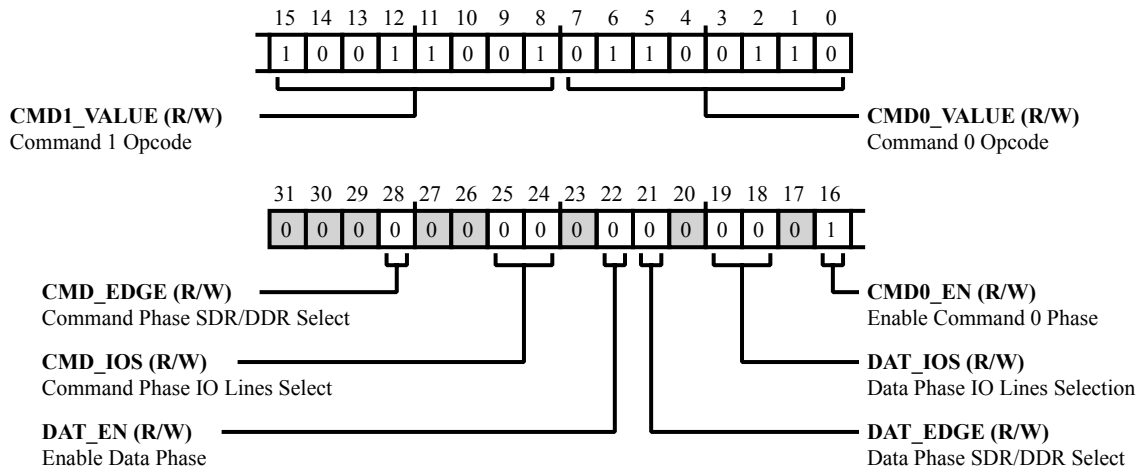


Figure 14-71: XSPI_RST_SEQ_CFG0 Register Diagram

Table 14-112: XSPI_RST_SEQ_CFG0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
28 (R/W)	CMD_EDGE	Command Phase SDR/DDR Select. The <code>XSPI_RST_SEQ_CFG0.CMD_EDGE</code> bit selects between SDR/DDR mode for both command phases.
25:24 (R/W)	CMD_IOS	Command Phase IO Lines Select. The <code>XSPI_RST_SEQ_CFG0.CMD_IOS</code> bit field indicates the number of data lines used to send commands for both command phases.
		0 One data line used (for example, serial)
		1 Two data lines used
		2 Four data lines used
		3 Eight data lines used
22 (R/W)	DAT_EN	Enable Data Phase. The <code>XSPI_RST_SEQ_CFG0.DAT_EN</code> bit enables sending the data phase (confirmation byte in) following the CMD1 phase.
21 (R/W)	DAT_EDGE	Data Phase SDR/DDR Select. The <code>XSPI_RST_SEQ_CFG0.DAT_EDGE</code> bit selects between SDR/DDR mode for the data phase (confirmation byte in) following the CMD1 phase.

Table 14-112: XSPI_RST_SEQ_CFG0 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
19:18 (R/W)	DAT_IOS	Data Phase IO Lines Selection. The XSPI_RST_SEQ_CFG0.DAT_IOS bit field indicates the number of lines used to send the data phase (confirmation byte in) following the CMD1 phase.
		0 One data line used (for example, serial)
		1 Two data lines used
		2 Four data lines used
		3 Eight data lines used
16 (R/W)	CMD0_EN	Enable Command 0 Phase. The XSPI_RST_SEQ_CFG0.CMD0_EN bit enables the CMD0 phase.
15:8 (R/W)	CMD1_VALUE	Command 1 Opcode. The XSPI_RST_SEQ_CFG0.CMD1_VALUE bit field indicates the command mnemonic value for the CMD1 phase.
7:0 (R/W)	CMD0_VALUE	Command 0 Opcode. The XSPI_RST_SEQ_CFG0.CMD0_VALUE bit field indicates the command mnemonic value for the CMD0 phase.

Reset Sequence Configuration Register 1

The `XSPI_RST_SEQ_CFG1` register configures the reset sequence for profile 1 and SPI NAND in ACMD work mode.

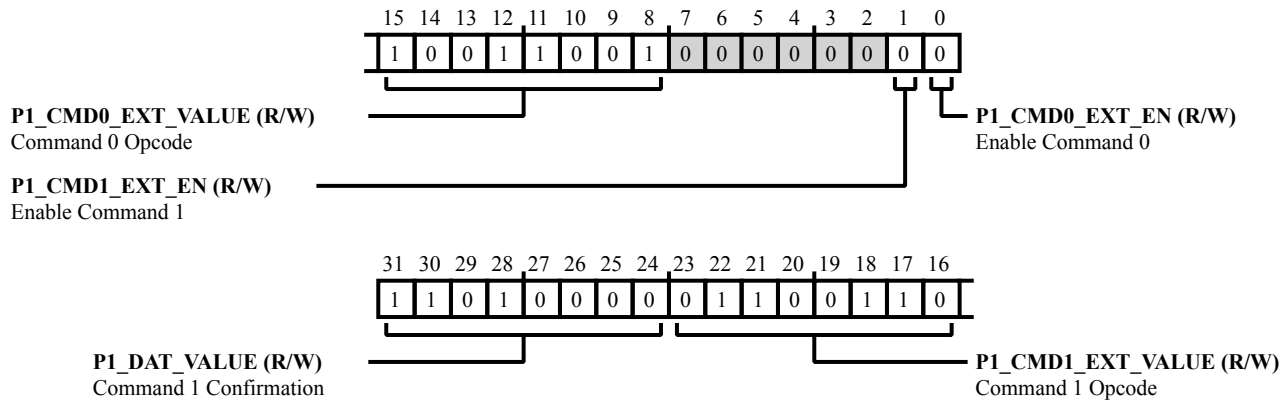


Figure 14-72: XSPI_RST_SEQ_CFG1 Register Diagram

Table 14-113: XSPI_RST_SEQ_CFG1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	P1_DAT_VALUE	Command 1 Confirmation. Value of confirmation byte in (if enabled) the following CMD1 phase.
23:16 (R/W)	P1_CMD1_EXT_VALUE	Command 1 Opcode. Command extension value of the CMD1 phase (if enabled).
15:8 (R/W)	P1_CMD0_EXT_VALUE	Command 0 Opcode. Command extension value of the CMD0 phase (if enabled).
1 (R/W)	P1_CMD1_EXT_EN	Enable Command 1. Command extension enable for the CMD1 phase.
0 (R/W)	P1_CMD0_EXT_EN	Enable Command 0. Command extension enable for the CMD0 phase.

Host DMA Buffer Address Register 0

The `XSPI_SDMA_ADDR0` register stores the buffer address in the host memory that is used as a sink/source for the host DMA transfer. The host DMA address is based on the memory pointer field that was programmed by the host as part of the CDMA/PIO command.

A single CDMA/PIO command can trigger multiple transfers on the host interface, so the host DMA address value is automatically incremented and updated before each DMA transfer.

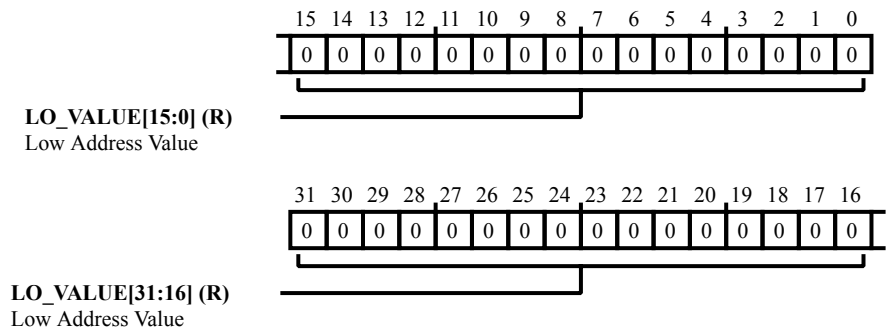


Figure 14-73: XSPI_SDMA_ADDR0 Register Diagram

Table 14-114: XSPI_SDMA_ADDR0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	LO_VALUE	Low Address Value. The <code>XSPI_SDMA_ADDR0.LO_VALUE</code> bit field indicates the lower part of the SDMA destination/source address.

Host DMA Buffer Address Register 1

The `XSPI_SDMA_ADDR1` register stores the buffer address in the host memory that is used as a sink/source for the host DMA transfer. The host DMA address is based on the memory pointer field that was programed by the host as part of the CDMA/PIO command.

A single CDMA/PIO command can trigger multiple transfers on the host interface, so the host DMA address value is automatically incremented and updated before each DMA transfer.

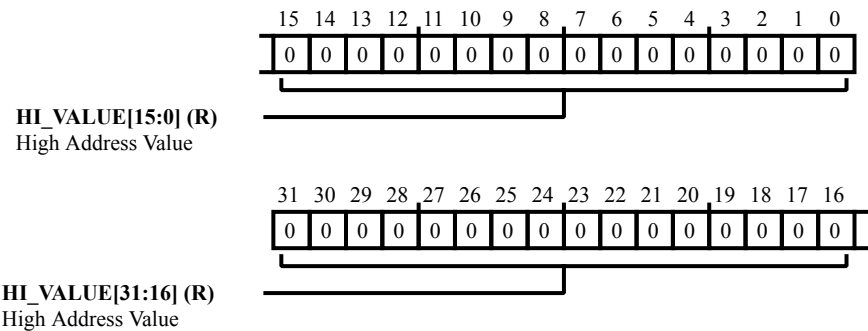


Figure 14-74: XSPI_SDMA_ADDR1 Register Diagram

Table 14-115: XSPI_SDMA_ADDR1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	HI_VALUE	High Address Value. The XSPI_SDMA_ADDR1 . HI_VALUE bit field indicates the higher part of the SDMA destination/source address.

Host DMA Block Size Register

The `XSPI_SDMA_SIZ` register contains the transferred data block size for the host DMA module.

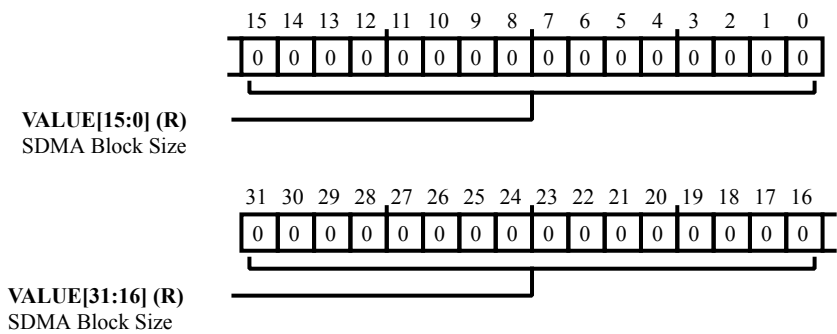


Figure 14-75: XSPI_SDMA_SIZ Register Diagram

Table 14-116: XSPI_SDMA_SIZ Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	SDMA Block Size. The <code>XSPI_SDMA_SIZ.VALUE</code> bit field indicates the transferred data block size in bytes for the host DMA module. Data size is rounded up to the data bus word size.

Host DMA Thread Status Register

The `XSPI_SDMA_TRD_STAT` register contains information for the current host DMA transaction as related to the execution thread.

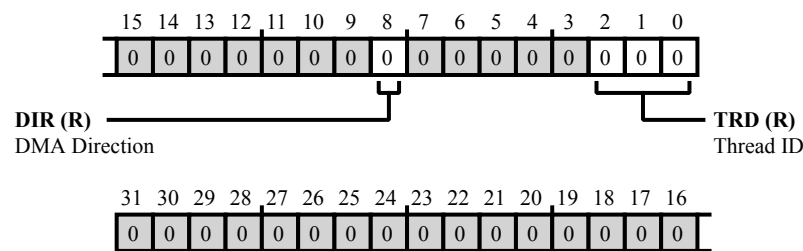


Figure 14-76: XSPI_SDMA_TRD_STAT Register Diagram

Table 14-117: XSPI_SDMA_TRD_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
8 (R/NW)	DIR	DMA Direction. The <code>XSPI_SDMA_TRD_STAT.DIR</code> bit field indicates the transfer direction related to current host DMA transfer (0-read; 1-write).
2:0 (R/NW)	TRD	Thread ID. The <code>XSPI_SDMA_TRD_STAT.TRD</code> bit field indicates the thread number associated with transferred data block for the host DMA module.

Sequence Configuration Register 0

The `XSPI_SEQ_GCTL0` register configures common values for sequences in CDMA, PIO and direct work modes.

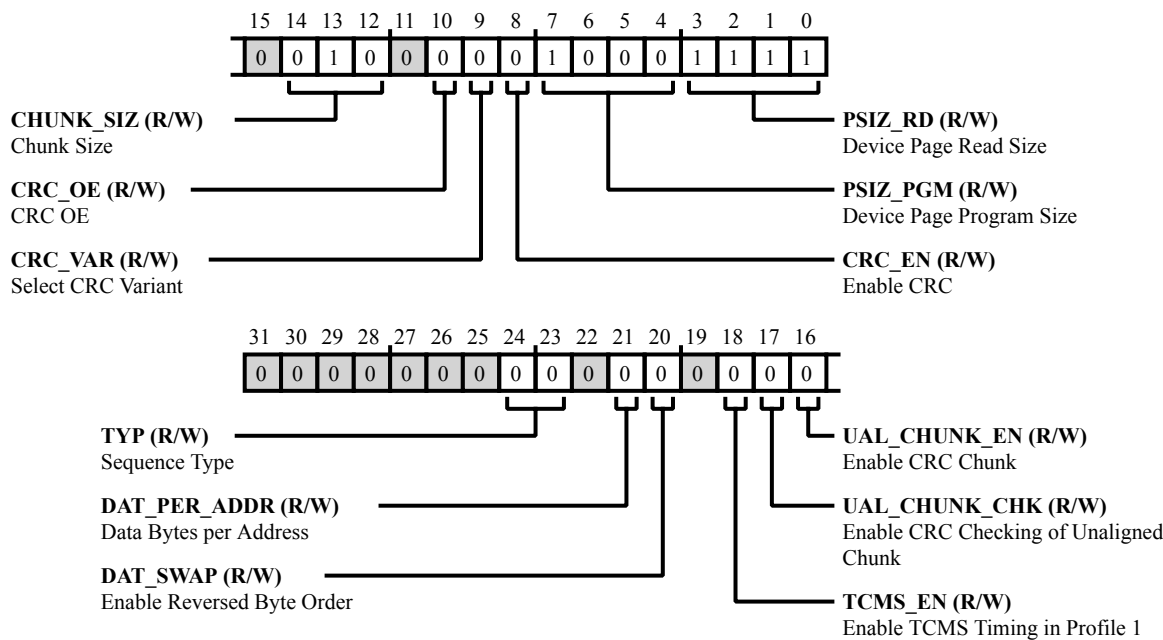


Figure 14-77: XSPI_SEQ_GCTL0 Register Diagram

Table 14-118: XSPI_SEQ_GCTL0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
24:23 (R/W)	TYP	Sequence Type. The <code>XSPI_SEQ_GCTL0</code> . TYP bit field indicates the sequence type (common for all sequences).
		0 Profile 1
		1 Profile 2 - HF (HyperFlash)
		2 Profile 2 - HR (HyperRAM)
		3 SPI NAND

Table 14-118: XSPI_SEQ_GCTL0 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
21 (R/W)	DAT_PER_ADDR	Data Bytes per Address. The XSPI_SEQ_GCTL0.DAT_PER_ADDR bit indicates the data organization of the xSPI memory. Note that the xSPI address/pointer specified during sending CDMA/PIO and direct command is always byte-aligned (for example, when this field is set, the xSPI address must be even).
		0 1B per single memory address. No translation of xSPI address.
		1 2B per single memory address. Translation of xSPI address from byte address to word address is performed automatically.
20 (R/W)	DAT_SWAP	Enable Reversed Byte Order. When set (=1), the XSPI_SEQ_GCTL0.DAT_SWAP bit enables reverse byte order. The XSPI_SEQ_GCTL0.DAT_SWAP bit can be set only when data phase reflects octal DDR mode. In other modes the XSPI_SEQ_GCTL0.DAT_SWAP bit must be cleared (=0).
		0 Disable
		1 Enable
18 (R/W)	TCMS_EN	Enable TCMS Timing in Profile 1. When set (=1), the XSPI_SEQ_GCTL0.TCMS_EN bit enables tCMS timing in profile 1. Refer to the XSPI_MINICTL_DEV_ACTIVE_MAX register at offset 0x1018 for further details.
		0 Disable
		1 Enable
17 (R/W)	UAL_CHUNK_CHK	Enable CRC Checking of Unaligned Chunk. Setting the XSPI_SEQ_GCTL0.UAL_CHUNK_CHK bit enables the checking of a CRC unaligned chunk from the flash device. It can be set (=1) only if crc_ual_chunk_en = 1. Otherwise, the XSPI_SEQ_GCTL0.UAL_CHUNK_CHK bit must be cleared (=0).
		0 Disable
		1 Enable
16 (R/W)	UAL_CHUNK_EN	Enable CRC Chunk. Setting the XSPI_SEQ_GCTL0.UAL_CHUNK_EN bit enables taking into consideration the command address to determine after how many bytes CRC data slice is expected to be returned by the flash device.
		0 Disable
		1 Enable

Table 14-118: XSPI_SEQ_GCTL0 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
14:12 (R/W)	CHUNK_SIZ	Chunk Size. The XSPI_SEQ_GCTL0.CHUNK_SIZ field indicates the number of bytes after which CRC occurs.
		0 N/A
		1 8B
		2 16B
		3 32B
		4 64B
		5 128B
		6 256B
		7 512B
10 (R/W)	CRC_OE	CRC OE. The XSPI_SEQ_GCTL0.CRC_OE field determines whether the controller expects the xSPI device to toggle CRC data on both SPI clock edges in CRC->CRC# sequence.
		0 Disable
		1 Enable
9 (R/W)	CRC_VAR	Select CRC Variant. The XSPI_SEQ_GCTL0.CRC_VAR bit indicates the CRC variant.
		0 Address Transfer Phase Only. CRC is calculated for all bytes of address transfer phase only and put on the bus after the address transfer phase.
		1 Sequence. CRC is calculated for all bytes in sequence and put on the bus after all bytes in the sequence.
8 (R/W)	CRC_EN	Enable CRC. When set (=1), the XSPI_SEQ_GCTL0.CRC_EN bit enables dynamic CRC calculation that is based on all previous bytes in the current sequence. Software puts this value on the xSPI flash interface. Not required by Legacy Hyper flash and xSPI Profile 2.0 Devices but can be useful for external flash Monitor to control data integrity.
		0 Disable
		1 Enable

Table 14-118: XSPI_SEQ_GCTL0 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
7:4 (R/W)	PSIZ_PGM	Device Page Program Size. The XSPI_SEQ_GCTL0 . PSIZ_PGM bit field indicates the page size of device being used for program operations. Number of bytes in page = $2^{\text{page_size}}$. Allowed values are: 4'b0000 - 1B 4'b0001 - 2B ... 4'b1000 - 256B 4'b1001 - 512B 4'b1010 - 1024B 4'b1011 - 2048B 4'b1100 - 4096B 4'b1111 - N/A This field is not used in direct mode for profile 2 - HR or when SPI NAND device is selected.
3:0 (R/W)	PSIZ_RD	Device Page Read Size. The XSPI_SEQ_GCTL0 . PSIZ_RD bit field indicates the page size of the device used for read operations. Number of bytes in page = $2^{\text{page_size}}$. Allowed values are: 4'b0000 - 1B 4'b0001 - 2B ... 4'b1000 - 256B 4'b1001 - 512B 4'b1010 - 1024B 4'b1011 - 2048B 4'b1100 - 4096B 4'b1101 - N/A ... 4'b1111 - unlimited (controller sends all data with single xSPI command)

Sequence Configuration Register 1

The `XSPI_SEQ_GCTL1` register configures common values for sequences in CDMA, PIO and direct work modes.

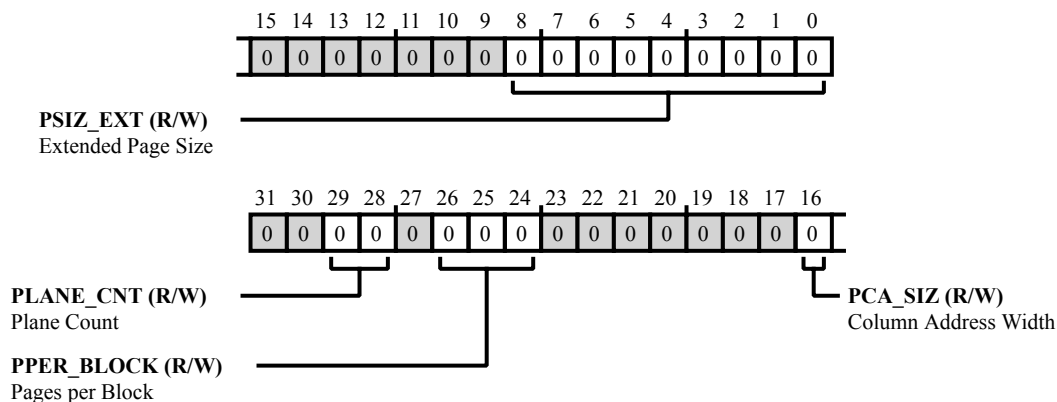


Figure 14-78: XSPI_SEQ_GCTL1 Register Diagram

Table 14-119: XSPI_SEQ_GCTL1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:28 (R/W)	PLANE_CNT	Plane Count. The <code>XSPI_SEQ_GCTL1 . PLANE_CNT</code> bit field indicates the number of planes in the SPI NAND device (encoded as 2^N).
		0 Single plane
		1 Two planes
		2 Four planes
		3 Reserved
26:24 (R/W)	PPER_BLOCK	Pages per Block. The <code>XSPI_SEQ_GCTL1 . PPER_BLOCK</code> bit field indicates the number of pages per blocks for SPI NAND device (encoded as 2^N):
		0 1 page per block
		1 2 pages per block
		2 4 pages per block
		3 8 pages per block
		4 16 pages per block
		5 32 pages per block
		6 64 pages per block
		7 128 pages per block

Table 14-119: XSPI_SEQ_GCTL1 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
16 (R/W)	PCA_SIZ	Column Address Width. The XSPI_SEQ_GCTL1.PCA_SIZ bit indicates the width of the column address for SPI NAND devices. The value of this field is used to calculate the next page address when the data size specified in the sequence exceeds the current page capacity.
		0 12 bit address width
		1 13 bit address width
8:0 (R/W)	PSIZ_EXT	Extended Page Size. The XSPI_SEQ_GCTL1.PSIZ_EXT bit field indicates the extended page size area (spare area size) for SPI NAND devices. The number of data bytes transmitted to/from each page is extended by a value of this field. This field is not used in direct mode.

Short Polling Count Register

The `XSPI_SHORTPOL_GCTL` register contains the status monitor cycle count value.

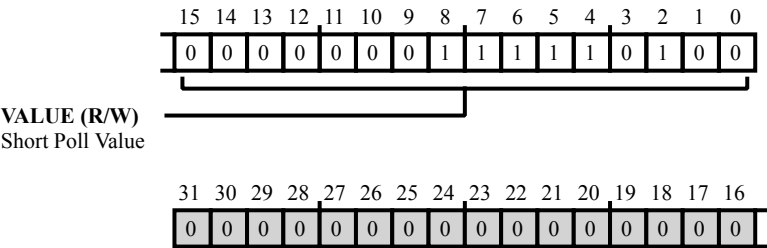


Figure 14-79: XSPI_SHORTPOL_GCTL Register Diagram

Table 14-120: XSPI_SHORTPOL_GCTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	VALUE	Short Poll Value. The <code>XSPI_SHORTPOL_GCTL.VALUE</code> bit field indicates the minimum number of system clocks after a long polling delay before the controller starts to poll for status if the controller was busy during the first status poll attempt. The long polling value should be significantly larger the short polling value.

Status Checking Sequence Configuration Register 0

The `XSPI_STAT_SEQ_CFG0` register configures the status checking sequence for profile 1 and SPI NAND in ACMD and direct work modes.

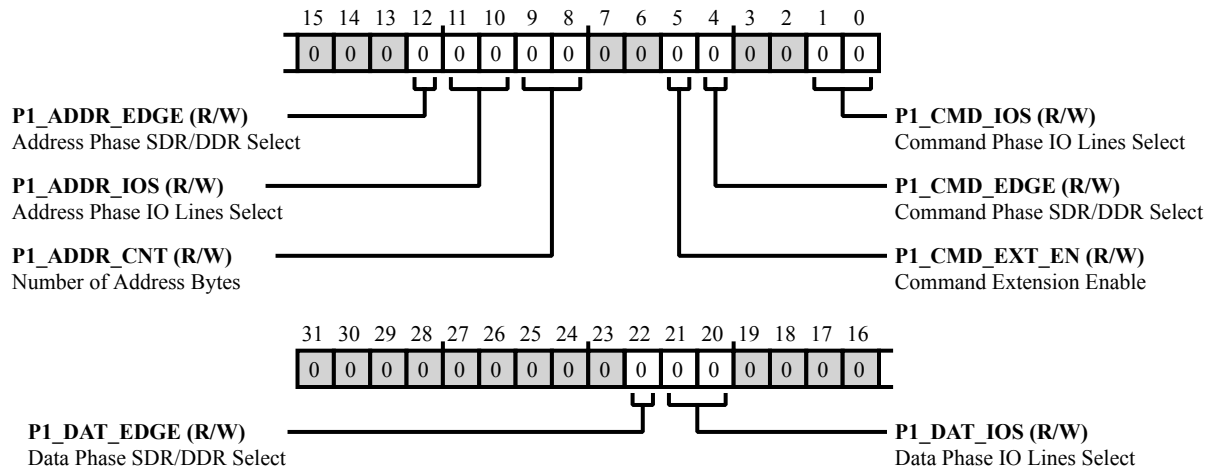


Figure 14-80: XSPI_STAT_SEQ_CFG0 Register Diagram

Table 14-121: XSPI_STAT_SEQ_CFG0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
22 (R/W)	P1_DAT_EDGE	Data Phase SDR/DDR Select. The <code>XSPI_STAT_SEQ_CFG0.P1_DAT_EDGE</code> bit selects between SDR/DDR mode for data phase.
21:20 (R/W)	P1_DAT_IOS	Data Phase IO Lines Select. The <code>XSPI_STAT_SEQ_CFG0.P1_DAT_IOS</code> bit field indicates the number of data lines used to send the data phase.
	0	One data line used (for example, serial)
	1	Two data lines used
	2	Four data lines used
	3	Eight data lines used
12 (R/W)	P1_ADDR_EDGE	Address Phase SDR/DDR Select. The <code>XSPI_STAT_SEQ_CFG0.P1_ADDR_EDGE</code> bit selects between SDR/DDR mode for address phase.

Table 14-121: XSPI_STAT_SEQ_CFG0 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
11:10 (R/W)	P1_ADDR_IOS	Address Phase IO Lines Select. The XSPI_STAT_SEQ_CFG0.P1_ADDR_IOS bit field indicates the number of data lines used to send the address phase.
		0 One data line used (for example, serial)
		1 Two data lines used
		2 Four data lines used
		3 Eight data lines used
9:8 (R/W)	P1_ADDR_CNT	Number of Address Bytes. The XSPI_STAT_SEQ_CFG0.P1_ADDR_CNT bit field indicates the number of address bytes for all status sequences.
		0 One address byte
		1 Two address bytes
		2 Three address bytes
		3 Four address bytes
5 (R/W)	P1_CMD_EXT_EN	Command Extension Enable. The XSPI_STAT_SEQ_CFG0.P1_CMD_EXT_EN bit enables the command extension.
4 (R/W)	P1_CMD_EDGE	Command Phase SDR/DDR Select. The XSPI_STAT_SEQ_CFG0.P1_CMD_EDGE bit selects between SDR/DDR mode for command phase.
1:0 (R/W)	P1_CMD_IOS	Command Phase IO Lines Select. The XSPI_STAT_SEQ_CFG0.P1_CMD_IOS bit field indicates the number of data lines used to send the command.
		0 One data line used (for example, serial)
		1 Two data lines used
		2 Four data lines used
		3 Eight data lines used

Status Checking Sequence Configuration Register 1

The `XSPI_STAT_SEQ_CFG1` register configures the status checking sequence for profile 1 and SPI NAND ACMD and direct work modes.

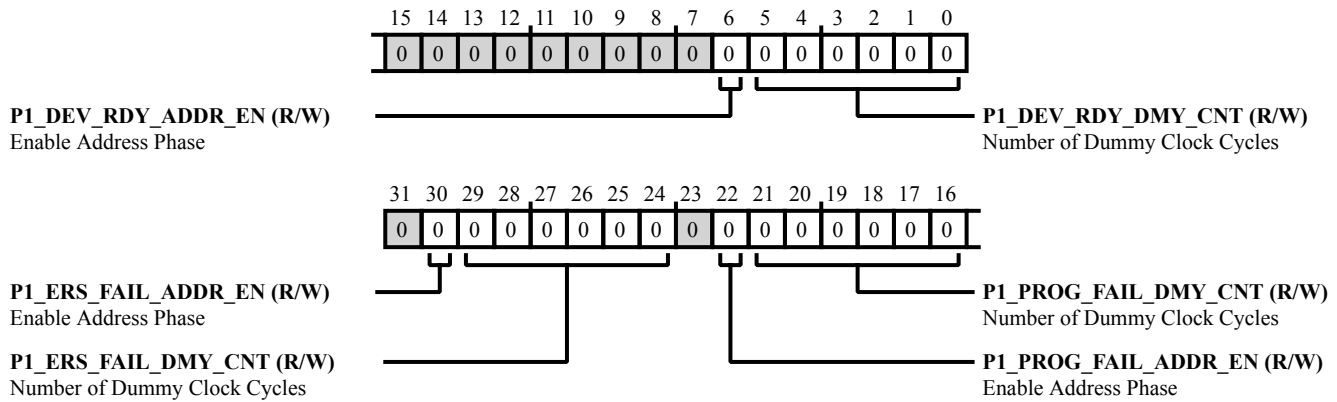


Figure 14-81: XSPI_STAT_SEQ_CFG1 Register Diagram

Table 14-122: XSPI_STAT_SEQ_CFG1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
30 (R/W)	<code>P1_ERS_FAIL_ADDR_EN</code>	Enable Address Phase. The <code>XSPI_STAT_SEQ_CFG1.P1_ERS_FAIL_ADDR_EN</code> bit enables the address phase for checking the fail status after the erase operation. This field is not used in direct work mode.
29:24 (R/W)	<code>P1_ERS_FAIL_DMY_CNT</code> T	Number of Dummy Clock Cycles. The <code>XSPI_STAT_SEQ_CFG1.P1_ERS_FAIL_DMY_CNT</code> bit field indicates the number of dummy clock cycles used to check the fail status after the erase operation. This field is not used in direct work mode.
22 (R/W)	<code>P1_PROG_FAIL_ADDR_EN</code>	Enable Address Phase. The <code>XSPI_STAT_SEQ_CFG1.P1_PROG_FAIL_ADDR_EN</code> bit enables the address phase for checking the fail status after the program operation.
21:16 (R/W)	<code>P1_PROG_FAIL_DMY_CNT</code> NT	Number of Dummy Clock Cycles. The <code>XSPI_STAT_SEQ_CFG1.P1_PROG_FAIL_DMY_CNT</code> bit field indicates the number of dummy clock cycles used to check the fail status after the program operation.
6 (R/W)	<code>P1_DEV_RDY_ADDR_EN</code> N	Enable Address Phase. The <code>XSPI_STAT_SEQ_CFG1.P1_DEV_RDY_ADDR_EN</code> bit enables the address phase for checking the ready/busy status after the program/erase and soft reset/read (only for SPI NAND) operation.

Table 14-122: XSPI_STAT_SEQ_CFG1 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
5:0 (R/W)	P1_DEV_RDY_DMY_CNT T	Number of Dummy Clock Cycles. The XSPI_STAT_SEQ_CFG1.P1_DEV_RDY_DMY_CNT bit field indicates the number of dummy clock cycles needed to check the ready/busy status after program/erase or soft reset/read (only for SPI NAND) operation.

Status Checking Sequence Configuration Register 10

The `XSPI_STAT_SEQ_CFG10` register configures the status checking sequence for SPI NAND devices in ACMD and direct work modes.

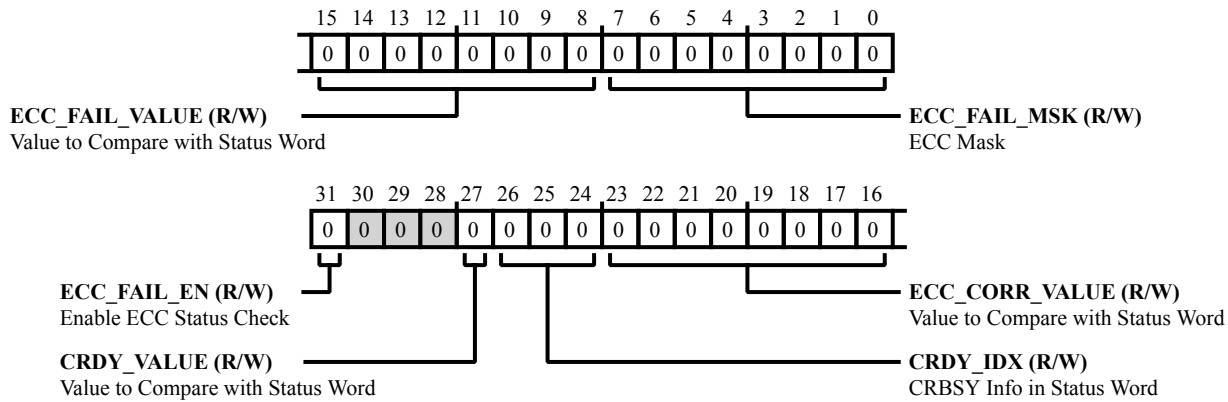


Figure 14-82: XSPI_STAT_SEQ_CFG10 Register Diagram

Table 14-123: XSPI_STAT_SEQ_CFG10 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	ECC_FAIL_EN	Enable ECC Status Check. The <code>XSPI_STAT_SEQ_CFG10.ECC_FAIL_EN</code> bit enables checking ECC status after a read page operation.
27 (R/W)	CRDY_VALUE	Value to Compare with Status Word. The <code>XSPI_STAT_SEQ_CFG10.CRDY_VALUE</code> bit indicates the value to be compared with the selected status bit in order to detect if the device is in a ready state after the read page cache random operation (CRBSY bit). This field is not used in direct work mode.
26:24 (R/W)	CRDY_IDX	CRBSY Info in Status Word. The <code>XSPI_STAT_SEQ_CFG10.CRDY_IDX</code> bit field indicates which bit of the status word contains the Cache Read Busy (CRBSY) bit information for the SPI NAND read page cache random operation. This field is not used in direct work mode.
23:16 (R/W)	ECC_CORR_VALUE	Value to Compare with Status Word. The <code>XSPI_STAT_SEQ_CFG10.ECC_CORR_VALUE</code> bit field indicates the value to be compared with the status word masked by the <code>XSPI_STAT_SEQ_CFG10.ECC_FAIL_MSK</code> field in order to detect if the device returned a correctable ECC error during SPI NAND page read operation. This can be used to detect a single range of correctable errors returned by the xSPI device.

Table 14-123: XSPI_STAT_SEQ_CFG10 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
15:8 (R/W)	ECC_FAIL_VALUE	Value to Compare with Status Word. The XSPI_STAT_SEQ_CFG10.ECC_FAIL_VALUE bit field indicates the value to be compared with the status word masked by the XSPI_STAT_SEQ_CFG10.ECC_FAIL_MSK field in order to detect if the device returned an uncorrectable ECC error during SPI NAND page read operation.
7:0 (R/W)	ECC_FAIL_MSK	ECC Mask. The XSPI_STAT_SEQ_CFG10.ECC_FAIL_MSK bit field indicates the mask used to select which bits of status word carries the ECC status.

Status Checking Sequence Configuration Register 2

The `XSPI_STAT_SEQ_CFG2` register configures the status checking sequence for profile 1 and SPI NAND in ACMD and direct work modes.

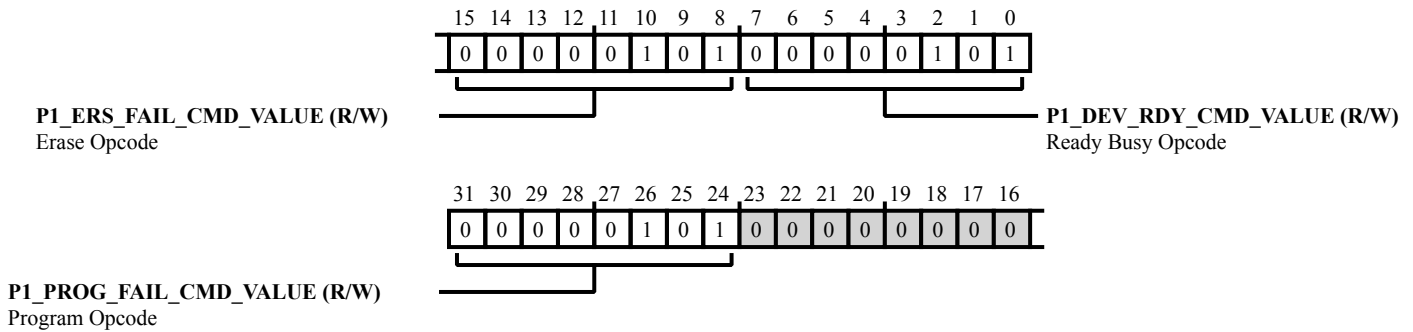


Figure 14-83: XSPI_STAT_SEQ_CFG2 Register Diagram

Table 14-124: XSPI_STAT_SEQ_CFG2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	P1_PROG_FAIL_CMD_VALUE	Program Opcode. The <code>XSPI_STAT_SEQ_CFG2.P1_PROG_FAIL_CMD_VALUE</code> bit field contains the command mnemonic value that is used to check the fail status after program operation.
15:8 (R/W)	P1_ERS_FAIL_CMD_VALUE	Erase Opcode. The <code>XSPI_STAT_SEQ_CFG2.P1_ERS_FAIL_CMD_VALUE</code> bit field contains the command mnemonic value that is used to check the fail status after the erase operation. This field is not used in direct work mode.
7:0 (R/W)	P1_DEV_RDY_CMD_VALUE	Ready Busy Opcode. The <code>XSPI_STAT_SEQ_CFG2.P1_DEV_RDY_CMD_VALUE</code> bit field contains the command mnemonic value that is used to check the ready/busy status after program/erase and soft erase/read (only for SPI NAND) operation.

Status Checking Sequence Configuration Register 3

The `XSPI_STAT_SEQ_CFG3` register configures the status checking sequence for profile 1 and SPI NAND in ACMD and direct work modes.

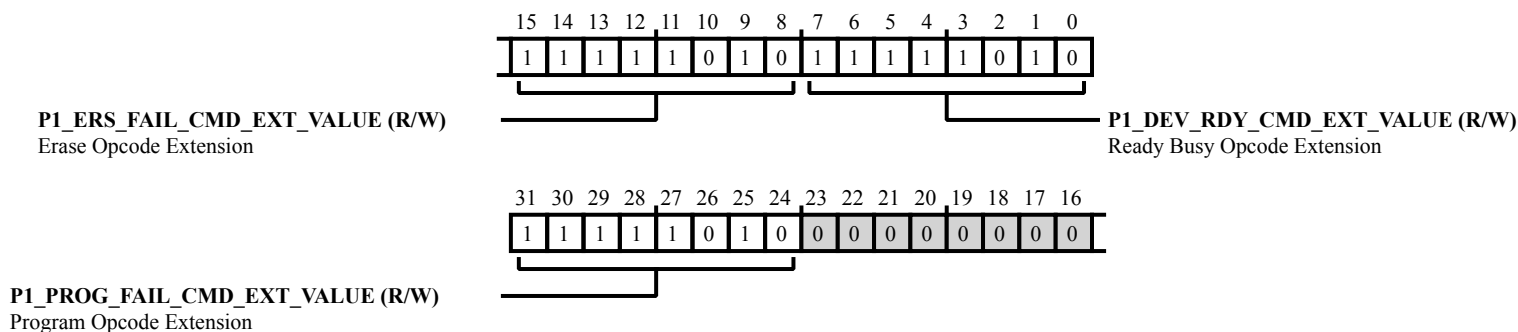


Figure 14-84: XSPI_STAT_SEQ_CFG3 Register Diagram

Table 14-125: XSPI_STAT_SEQ_CFG3 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	P1_PROG_FAIL_CMD_EXT_VALUE	Program Opcode Extension. The <code>XSPI_STAT_SEQ_CFG3.P1_PROG_FAIL_CMD_EXT_VALUE</code> bit field contains the command mnemonic value that is used to check the fail status after the program operation.
15:8 (R/W)	P1_ERS_FAIL_CMD_EXT_VALUE	Erase Opcode Extension. The <code>XSPI_STAT_SEQ_CFG3.P1_ERS_FAIL_CMD_EXT_VALUE</code> bit field contains the command mnemonic value that is used to check the fail status after the erase operation. This field is not used in direct work mode.
7:0 (R/W)	P1_DEV_RDY_CMD_EXT_VALUE	Ready Busy Opcode Extension. The <code>XSPI_STAT_SEQ_CFG3.P1_DEV_RDY_CMD_EXT_VALUE</code> bit field contains the command mnemonic value that is used to check the ready/busy status after the program/erase and soft reset/read (only for SPI NAND) operation.

Status Checking Sequence Configuration Register 4

The `XSPI_STAT_SEQ_CFG4` register configures the status checking sequence for profile 2 - HF in ACMD and direct work modes.

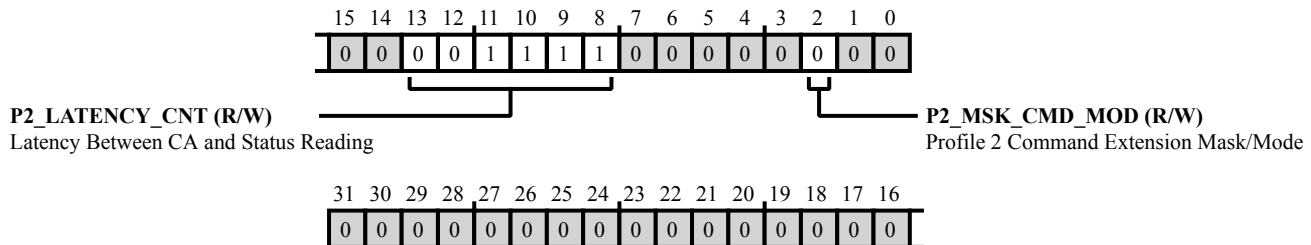


Figure 14-85: XSPI_STAT_SEQ_CFG4 Register Diagram

Table 14-126: XSPI_STAT_SEQ_CFG4 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
13:8 (R/W)	P2_LATENCY_CNT	Latency Between CA and Status Reading. The <code>XSPI_STAT_SEQ_CFG4.P2_LATENCY_CNT</code> bit field indicates the number of latency cycles between the CA and status reading. Configure <code>XSPI_STAT_SEQ_CFG4.P2_LATENCY_CNT</code> to N-1, where N is the number of latency clock cycles expected by the memory device.
2 (R/W)	P2_MSK_CMD_MOD	Profile 2 Command Extension Mask/Mode. The <code>XSPI_STAT_SEQ_CFG4.P2_MSK_CMD_MOD</code> bit field indicates the profile 2 command extension variant. The value of this field influences bits[44:40] of the Command/Address (CA). If this bit is set (=1), bits [44:40] of the CA are set (=1). Otherwise, bits [44:40] of the CA are cleared (=0).

Status Checking Sequence Configuration Register 5

The `XSPI_STAT_SEQ_CFG5` register configures the status checking sequence for profile 1, SPI NAND and profile 2 - HF in ACMD and direct work modes.

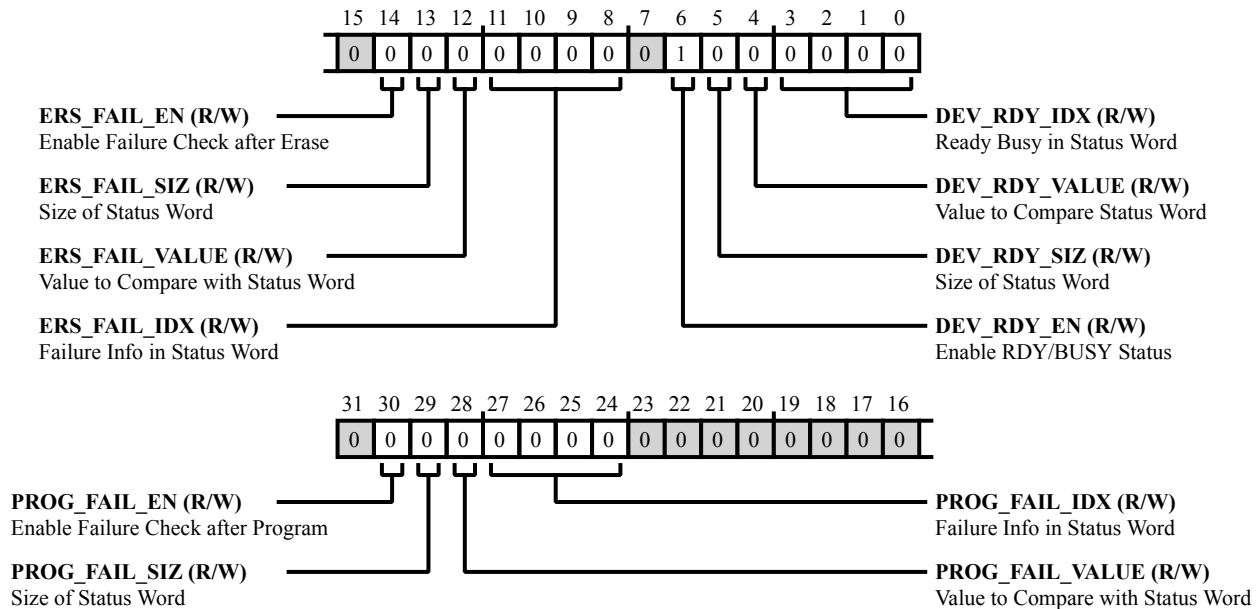


Figure 14-86: XSPI_STAT_SEQ_CFG5 Register Diagram

Table 14-127: XSPI_STAT_SEQ_CFG5 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
30 (R/W)	PROG_FAIL_EN	Enable Failure Check after Program. The <code>XSPI_STAT_SEQ_CFG5.PROG_FAIL_EN</code> bit enables checking the fail status after program operation.
29 (R/W)	PROG_FAIL_SIZ	Size of Status Word. The <code>XSPI_STAT_SEQ_CFG5.PROG_FAIL_SIZ</code> bit indicates the size of the status word (0-1B, 1-2B).
28 (R/W)	PROG_FAIL_VALUE	Value to Compare with Status Word. The <code>XSPI_STAT_SEQ_CFG5.PROG_FAIL_VALUE</code> bit indicates the value to be compared with selected status bit in order to detect that the device is in fail state after program operation.
27:24 (R/W)	PROG_FAIL_IDX	Failure Info in Status Word. The <code>XSPI_STAT_SEQ_CFG5.PROG_FAIL_IDX</code> bit field indicates which bit of the status word contains the failure information after the program command.

Table 14-127: XSPI_STAT_SEQ_CFG5 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
14 (R/W)	ERS_FAIL_EN	Enable Failure Check after Erase. The XSPI_STAT_SEQ_CFG5.ERS_FAIL_EN bit enables checking the fail status after an erase operation. This field is not used in direct work mode.
13 (R/W)	ERS_FAIL_SIZ	Size of Status Word. The XSPI_STAT_SEQ_CFG5.ERS_FAIL_SIZ bit indicates the size of status word (0-1B, 1-2B). This field is not used in direct work mode.
12 (R/W)	ERS_FAIL_VALUE	Value to Compare with Status Word. The XSPI_STAT_SEQ_CFG5.ERS_FAIL_VALUE bit indicates the value to be compared with the selected status bit in order to detect that the device is in the ready state after erase operation. This field is not used in direct work mode.
11:8 (R/W)	ERS_FAIL_IDX	Failure Info in Status Word. The XSPI_STAT_SEQ_CFG5.ERS_FAIL_IDX bit field indicates which bit of the status word contains the failure information after the erase command. This field is not used in direct work mode.
6 (R/W)	DEV_RDY_EN	Enable RDY/BUSY Status. The XSPI_STAT_SEQ_CFG5.DEV_RDY_EN bit enables checking the ready/busy status after program/erase operations.
5 (R/W)	DEV_RDY_SIZ	Size of Status Word. The XSPI_STAT_SEQ_CFG5.DEV_RDY_SIZ bit indicates the size of status word (0-1B, 1-2B).
4 (R/W)	DEV_RDY_VALUE	Value to Compare Status Word. The XSPI_STAT_SEQ_CFG5.DEV_RDY_VALUE bit indicates the value to be compared with selected status bit in order to detect that the device is ready after a program/erase or soft reset/read (only for SPI NAND) operation.
3:0 (R/W)	DEV_RDY_IDX	Ready Busy in Status Word. The XSPI_STAT_SEQ_CFG5.DEV_RDY_IDX bit field indicates which bit of the status word contains the ready/busy information to be polled after a program/erase or soft erase/read (only for SPI NAND) operation.

Status Checking Sequence Configuration Register 7

The `XSPI_STAT_SEQ_CFG7` register configures the status checking sequence for profile 1, SPI NAND and profile 2 - HF in ACMD and direct work modes.

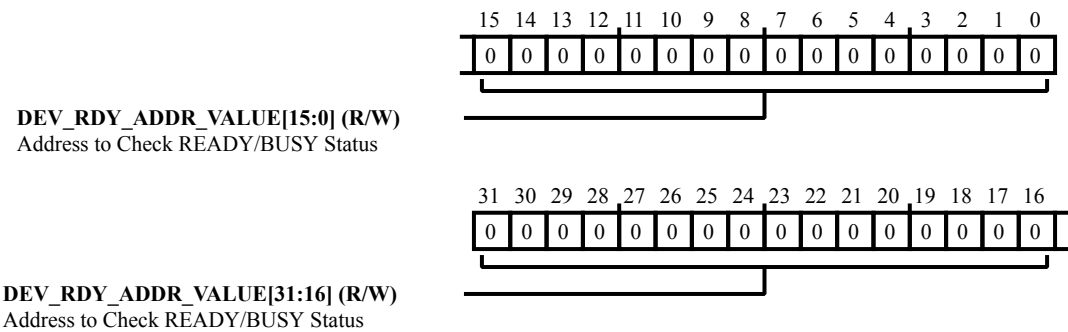


Figure 14-87: XSPI_STAT_SEQ_CFG7 Register Diagram

Table 14-128: XSPI_STAT_SEQ_CFG7 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	DEV_RDY_ADDR_VAL- UE	Address to Check READY/BUSY Status. The <code>XSPI_STAT_SEQ_CFG7.DEV_RDY_ADDR_VALUE</code> bit field indicates the value of the address used to check the ready/busy status after the program/erase and soft reset/read (only for SPI NAND) operation.

Status Checking Sequence Configuration Register 8

The `XSPI_STAT_SEQ_CFG8` register configures the status checking sequence for profile 1, SPI NAND and profile 2 - HF in ACMD and direct work modes.

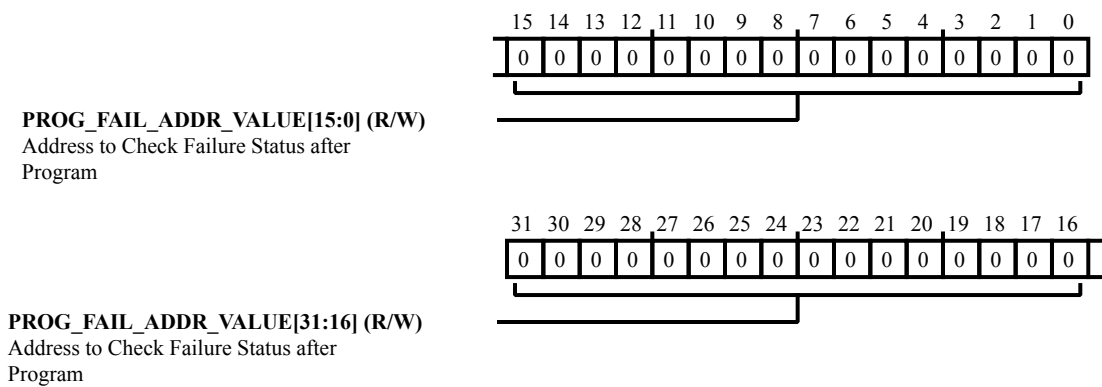


Figure 14-88: XSPI_STAT_SEQ_CFG8 Register Diagram

Table 14-129: XSPI_STAT_SEQ_CFG8 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	PROG_FAIL_ADDR_VAL- UE	Address to Check Failure Status after Program. The <code>XSPI_STAT_SEQ_CFG8.PROG_FAIL_ADDR_VALUE</code> bit field indicates the value of the address used to check the fail status after the program operation.

Status Checking Sequence Configuration Register 9

The `XSPI_STAT_SEQ_CFG9` register configures the status checking sequence for profile 1, SPI NAND and profile 2 - HF in ACMD work mode.

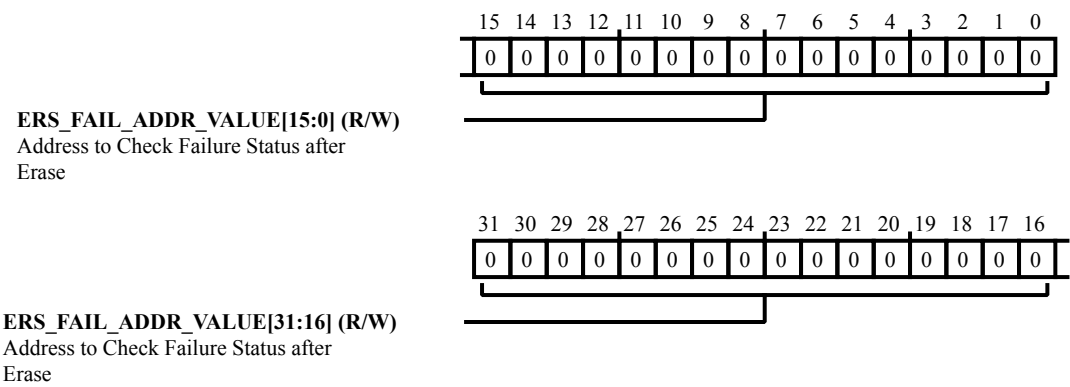


Figure 14-89: XSPI_STAT_SEQ_CFG9 Register Diagram

Table 14-130: XSPI_STAT_SEQ_CFG9 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	ERS_FAIL_ADDR_VAL- UE	Address to Check Failure Status after Erase. The <code>XSPI_STAT_SEQ_CFG9.ERS_FAIL_ADDR_VALUE</code> bit field indicates the value of the address used to check fail status after erase operation.

Auto Command Engine Interrupt Status Thread Register

Each bit of the field corresponds to an auto command engine thread and holds the descriptor status for that selected thread. It is set only when INT bit of descriptor is set.

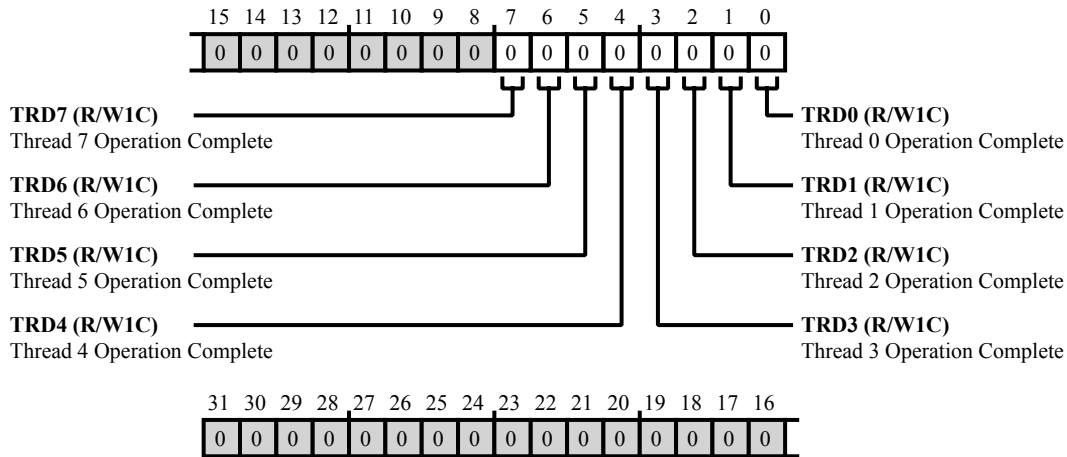


Figure 14-90: XSPI_TRD_COMP_ISTAT Register Diagram

Table 14-131: XSPI_TRD_COMP_ISTAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7 (R/W1C)	TRD7	Thread 7 Operation Complete. Thread 7 operation complete flag.
6 (R/W1C)	TRD6	Thread 6 Operation Complete. Thread 6 operation complete flag.
5 (R/W1C)	TRD5	Thread 5 Operation Complete. Thread 5 operation complete flag.
4 (R/W1C)	TRD4	Thread 4 Operation Complete. Thread 4 operation complete flag.
3 (R/W1C)	TRD3	Thread 3 Operation Complete. Thread 3 operation complete flag.
2 (R/W1C)	TRD2	Thread 2 Operation Complete. Thread 2 operation complete flag.
1 (R/W1C)	TRD1	Thread 1 Operation Complete. Thread 1 operation complete flag.
0 (R/W1C)	TRD0	Thread 0 Operation Complete. Thread 0 operation complete flag.

Thread Error Interrupt Enable Register

If the selected bit of the `XSPI_TRD_ERR_INT_EN` register is set, the rising edge of the corresponding bit in the `XSPI_TRD_COMP_ISTAT` register causes the setting of the external interrupt line.

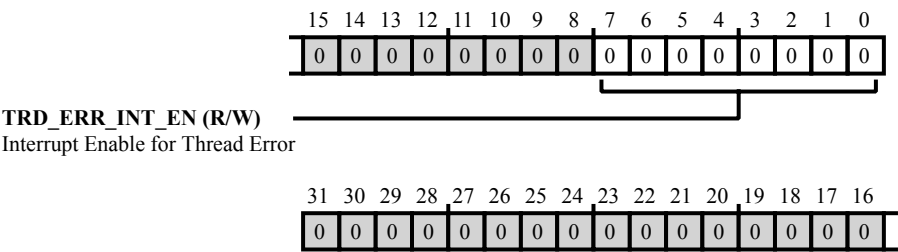


Figure 14-91: XSPI_TRD_ERR_INT_EN Register Diagram

Table 14-132: XSPI_TRD_ERR_INT_EN Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/W)	TRD_ERR_INT_EN	Interrupt Enable for Thread Error. Interrupt enable for detecting a thread error.

Thread Error Interrupt Status Register

The `XSPI_TRD_ERR_ISTAT` register can be read to determine whether a particular thread (in the auto command engine) has encountered an error condition.

To get more information on the error, software must read the status field of the descriptor (CDMA mode) or the appropriate status register (PIO mode).

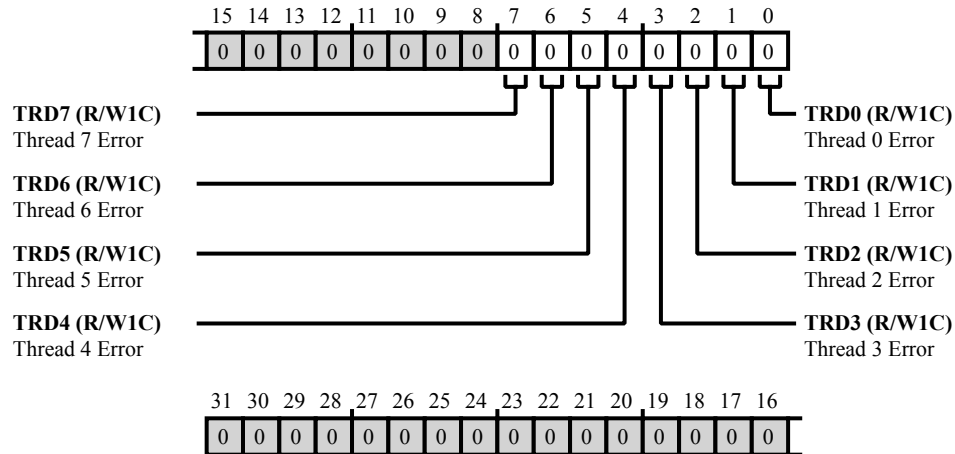


Figure 14-92: XSPI_TRD_ERR_ISTAT Register Diagram

Table 14-133: XSPI_TRD_ERR_ISTAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7 (R/W1C)	TRD7	Thread 7 Error. Thread 7 error.
6 (R/W1C)	TRD6	Thread 6 Error. Thread 6 error.
5 (R/W1C)	TRD5	Thread 5 Error. Thread 5 error.
4 (R/W1C)	TRD4	Thread 4 Error. Thread 4 error.
3 (R/W1C)	TRD3	Thread 3 Error. Thread 3 error.
2 (R/W1C)	TRD2	Thread 2 Error. Thread 2 error.
1 (R/W1C)	TRD1	Thread 1 Error. Thread 1 error.

Table 14-133: XSPI_TRD_ERR_ISTAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W1C)	TRD0	Thread 0 Error. Thread 0 error.

Auto Command Engine Thread Status Register

The `XSPI_TRD_STAT` register indicates the auto command engine thread status (ACMD mode only).

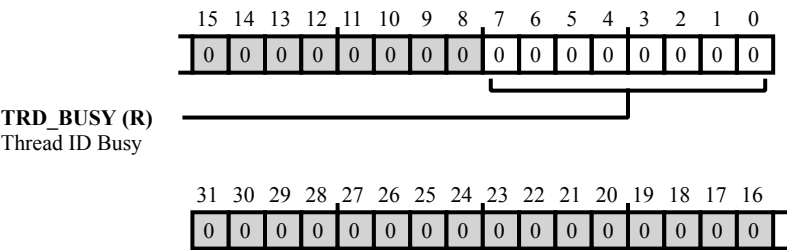


Figure 14-93: XSPI_TRD_STAT Register Diagram

Table 14-134: XSPI_TRD_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	TRD_BUSY	Thread ID Busy. The <code>XSPI_TRD_STAT.TRD_BUSY</code> bit indicates the auto command engine thread busy status. When set, the corresponding thread is currently busy.

WEL Sequence Configuration Register

The `XSPI_WE_SEQ_CFG0` register configures the Write Enable Latch (WEL) sequence for profile 1 and SPI NAND in ACMD and direct work modes.

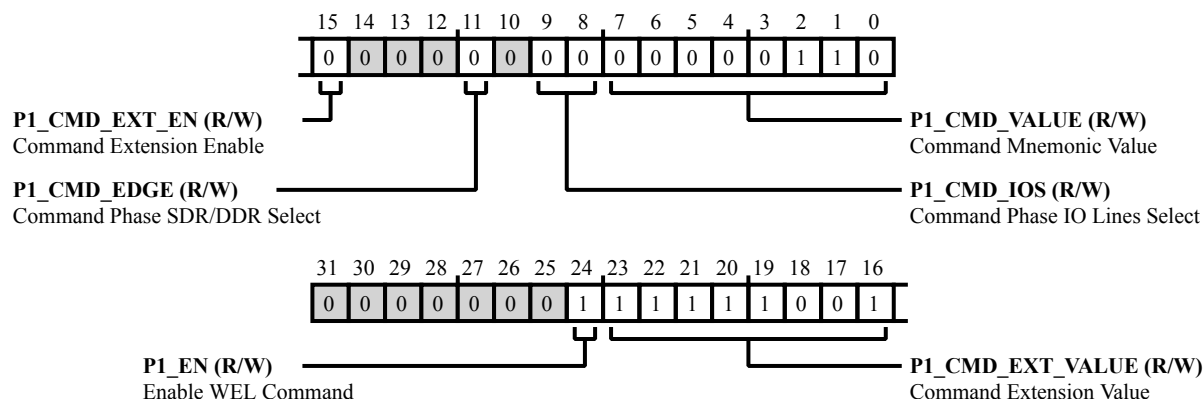


Figure 14-94: XSPI_WE_SEQ_CFG0 Register Diagram

Table 14-135: XSPI_WE_SEQ_CFG0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
24 (R/W)	P1_EN	Enable WEL Command. The <code>XSPI_WE_SEQ_CFG0.P1_EN</code> bit enables the WEL command.
23:16 (R/W)	P1_CMD_EXT_VALUE	Command Extension Value. The <code>XSPI_WE_SEQ_CFG0.P1_CMD_EXT_VALUE</code> bit field indicates the command extension value.
15 (R/W)	P1_CMD_EXT_EN	Command Extension Enable. The <code>XSPI_WE_SEQ_CFG0.P1_CMD_EXT_EN</code> bit enables the command extension.
11 (R/W)	P1_CMD_EDGE	Command Phase SDR/DDR Select. The <code>XSPI_WE_SEQ_CFG0.P1_CMD_EDGE</code> bit selects between SDR/DDR mode for command phase.
9:8 (R/W)	P1_CMD_IOS	Command Phase IO Lines Select. The <code>XSPI_WE_SEQ_CFG0.P1_CMD_IOS</code> bit field indicates the number of data lines used to send the command phase.
	0	One data line used (for example, serial)
	1	Two data lines used
	2	Four data lines used
	3	Eight data lines used

Table 14-135: XSPI_WE_SEQ_CFG0 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/W)	P1_CMD_VALUE	Command Mnemonic Value. The XSPI_WE_SEQ_CFG0.P1_CMD_VALUE bit field indicates the command mnemonic value.

Device Control Register

The `XSPI_WORKMODE_CTL` register controls the device working modes.

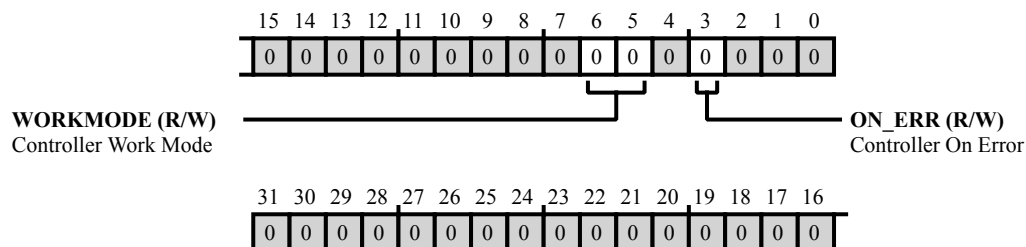


Figure 14-95: XSPI_WORKMODE_CTL Register Diagram

Table 14-136: XSPI_WORKMODE_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
6:5 (R/W)	WORKMODE	Controller Work Mode. The <code>XSPI_WORKMODE_CTL.WORKMODE</code> bit field indicates the work mode of the controller. Allowed values are 00 - Direct mode 01 - STIG mode 11 - ACMD mode
3 (R/W)	ON_ERR	Controller On Error. This control bit is relevant only for ACMD work mode. If the <code>XSPI_WORKMODE_CTL.ON_ERR</code> bit is cleared (=0) and an error occurs, the controller drops execution of all further operations programmed in a single thread at the page (for read or program commands) or sector (for the sector erase command) boundary. The <code>XSPI_WORKMODE_CTL.ON_ERR</code> bit applies to both command DMA and PIO work modes. When <code>XSPI_WORKMODE_CTL.ON_ERR</code> is set (=1), execution continues.

XIP Configuration Register

The `XSPI_XIP_GCTL` register configures the controller in XIP work mode in CDMA, PIO and direct work modes.

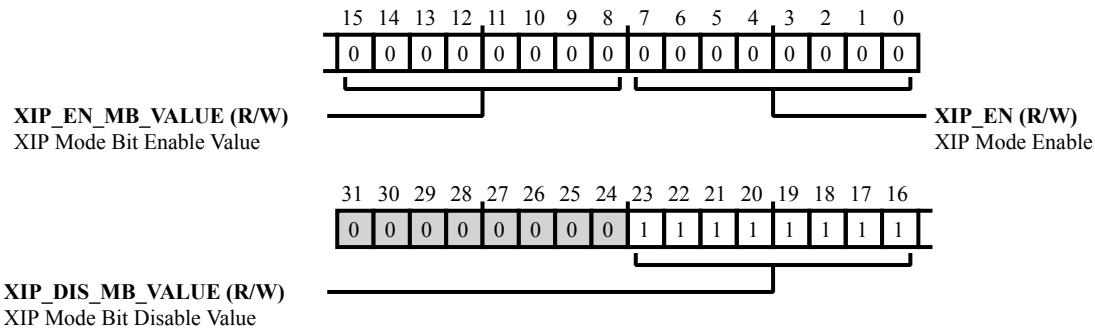


Figure 14-96: XSPI_XIP_GCTL Register Diagram

Table 14-137: XSPI_XIP_GCTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
23:16 (R/W)	XIP_DIS_MB_VALUE	XIP Mode Bit Disable Value. The <code>XSPI_XIP_GCTL.XIP_DIS_MB_VALUE</code> bit field indicates the value of the mode bits required to disable XIP mode.
15:8 (R/W)	XIP_EN_MB_VALUE	XIP Mode Bit Enable Value. The <code>XSPI_XIP_GCTL.XIP_EN_MB_VALUE</code> bit field indicates the value of the mode bits required to enable XIP mode.
7:0 (R/W)	XIP_EN	XIP Mode Enable. The <code>XSPI_XIP_GCTL.XIP_EN</code> bit field indicates the XIP mode enabled for a selected memory bank. When XIP mode is enabled, only read sequences are valid. Invoking any other command sequence is ignored and <code>CMD_ERROR/DSC_ERROR/XSPI_ISTAT.DIR_CMD_ERR = 1</code> .

15 Universal Asynchronous Receiver/Transmitter (UART)

The UART module is a full-duplex peripheral compatible with PC-style industry-standard UARTs. The UART converts data between serial and parallel formats. The serial communication follows an asynchronous protocol that supports various word lengths, stop bits, bit rates, and parity-generation options. Multiple events can generate interrupts.

The UART is logically compliant to EIA-232E, EIA-422, EIA-485 and LIN standards, but usually requires external transceiver devices to meet electrical requirements. In IrDA (Infrared Data Association) mode, the UART meets the half-duplex IrDA SIR (9.6/115.2 Kbps rate) protocol. In multi-drop bus mode, the UART meets the full-duplex MDB/ICP v2.0 protocol.

The UART module supports partial modem status and control functionality to allow for hardware flow control.

The UART is a DMA-capable peripheral with separate transmit and receive DMA controller channels. The use of DMA requires minimal software intervention as the DMA engine moves the data. The UART can also use a programmed core mode of operation. The core mode requires software management of the data flow using either interrupts or polling.

The UART can use one of the peripheral timers for a hardware-assisted auto-baud detection mechanism. The timers are external to the UART.

NOTE: The UARTs (UART0, UART1, and UART2) operate in the SCLK0 domain.

UART Features

Each UART includes the following features.

- 5–8 data bits
- Programmable extra stop bit and programmable extra half-stop bit
- Even, odd, and sticky parity bit options
- Extra 8-stage receive FIFO with programmable threshold interrupt request
- Flexible transmit and receive interrupt request timing

- Three interrupt outputs for receive, transmit, and status
- Independent DMA operation for receive and transmit
- Programmable automatic request to send (RTS)/clear to send (CTS) hardware flow control
- False start bit detection
- SIR IrDA operation mode
- MDB/ICP v2.0 operation mode
- Internal loopback
- Improved bit rate granularity
- LIN break command/Inter-frame gap transmission support

Table 15-1: UART Specifications

Feature	Availability
<i>Protocol</i>	
Controller-Capable	Yes
Target-Capable	Yes
Transmission Simplex	Yes
Transmission Half-Duplex	Yes
Transmission Full-Duplex	Yes
<i>Access Type</i>	
Data Buffer	Yes
Core Data Access	Yes
DMA Data Access	Yes
DMA Channels	2 (per UART Port)
DMA Descriptor	Yes
Boot Capable	Yes (Target Mode)
Local Memory	No
Clock Operation	SCLK/16

UART Functional Description

The following sections provide details on the UARTs functionality.

ADSP-21560_21561_21564_21568 UART Register List

The Universal Asynchronous Receiver/Transmitter module (UART) is a full-duplex peripheral compatible with PC-style industry-standard UARTs. The UART converts data between serial and parallel formats. The serial communication follows an asynchronous protocol that supports various word length, stop bit, parity, and interrupt generation options. A set of registers governs UART operations. For more information on UART functionality, see the UART register descriptions.

Table 15-2: ADSP-21560_21561_21564_21568 UART Register List

Name	Description
UART_CLK	Clock Rate Register
UART_CTL	Control Register
UART_IMSK	Interrupt Mask Register
UART_IMSK_CLR	Interrupt Mask Clear Register
UART_IMSK_SET	Interrupt Mask Set Register
UART_RBR	Receive Buffer Register
UART_RSR	Receive Shift Register
UART_RXCNT	Receive Counter Register
UART_SCR	Scratch Register
UART_STAT	Status Register
UART_TAIP	Transmit Address/Insert Pulse Register
UART_THR	Transmit Hold Register
UART_TSR	Transmit Shift Register
UART_TXCNT	Transmit Counter Register

ADSP-21560_21561_21564_21568 UART Interrupt List

Table 15-3: ADSP-21560_21561_21564_21568 UART Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
93	UART0_TXDMA	UART0 Transmit DMA	Level	20
94	UART0_RXDMA	UART0 Receive DMA	Level	21
95	UART0_STAT	UART0 Status	Level	
96	UART1_TXDMA	UART1 Transmit DMA	Level	34
97	UART1_RXDMA	UART1 Receive DMA	Level	35
98	UART1_STAT	UART1 Status	Level	
162	UART0_TXDMA_ERR	UART0 Transmit DMA Error	Level	

Table 15-3: ADSP-21560_21561_21564_21568 UART Interrupt List (Continued)

Interrupt ID	Name	Description	Sensitivity	DMA Channel
163	UART0_RXDMA_ERR	UART0 Receive DMA Error	Level	
164	UART1_TXDMA_ERR	UART1 Transmit DMA Error	Level	
165	UART1_RXDMA_ERR	UART1 Receive DMA Error	Level	

ADSP-21560_21561_21564_21568 UART Trigger List

Table 15-4: ADSP-21560_21561_21564_21568 UART Trigger List Generators

Trigger ID	Name	Description	Sensitivity
43	UART0_TXDMA	UART0 Transmit DMA	Edge
44	UART0_RXDMA	UART0 Receive DMA	Edge
45	UART1_TXDMA	UART1 Transmit DMA	Edge
46	UART1_RXDMA	UART1 Receive DMA	Edge

Table 15-5: ADSP-21560_21561_21564_21568 UART Trigger List Receivers

Trigger ID	Name	Description	Sensitivity
82	UART0_TXDMA	UART0 Transmit DMA	Pulse
83	UART0_RXDMA	UART0 Receive DMA	Pulse
84	UART1_TXDMA	UART1 Transmit DMA	Pulse
85	UART1_RXDMA	UART1 Receive DMA	Pulse

ADSP-21560_21561_21564_21568 UART DMA Channel List

Table 15-6: ADSP-21560_21561_21564_21568 UART DMA Channel List

DMA ID	DMA Channel Name	Description
DMA20	UART0_TXDMA	UART0 Transmit DMA
DMA21	UART0_RXDMA	UART0 Receive DMA
DMA34	UART1_TXDMA	UART1 Transmit DMA
DMA35	UART1_RXDMA	UART1 Receive DMA

UART Block Diagram

The *UART Block Diagram* figure shows a simplified block diagram of one UART module and how it interconnects to the processor system.

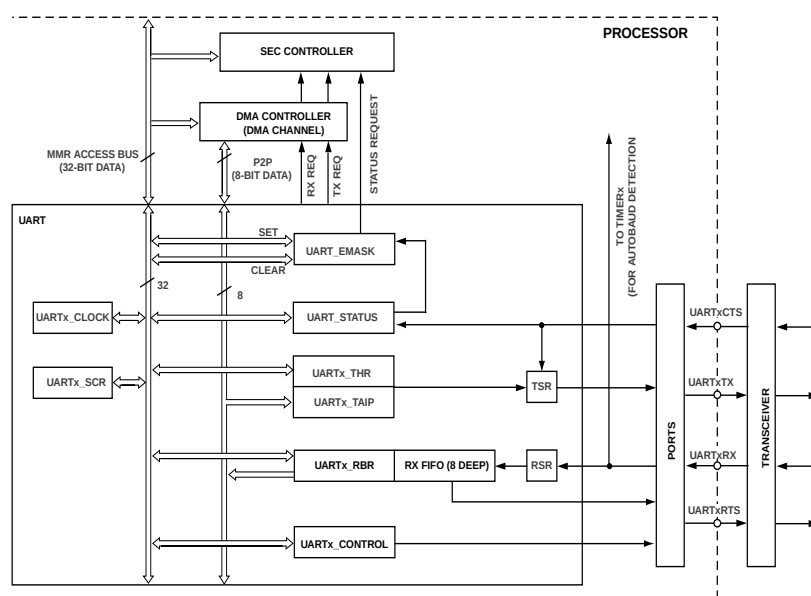


Figure 15-1: UART Block Diagram

UART Architectural Concepts

The following sections provide information about the UART architecture.

Internal Interface

The UART is a DMA-capable peripheral with support for separate transmit and receive DMA controller channels. It operates in either DMA or programmed core modes. The core mode requires software management of the data flow using either interrupts or polling. The DMA method requires minimal software intervention, as the DMA engine itself moves the data. The `UART_RBR` and `UART_THR` registers also connect to one of the peripheral DMA buses.

All UART registers are 32 bits, and the registers connect to the peripheral MMR bus. Not all MMRs can be used, and unused bits are zero-filled. The UART has three interrupt outputs described as follows.

- The transmit and receive request outputs can function as DMA requests and connect to the DMA controller. Therefore, if the DMA is not enabled, the DMA controller simply forwards the request to the system event controller (SEC).
- The status interrupt output connects directly to the SEC. On many processors, the alternative capture input () of one of the GP timers also senses the `UART_RX` pin. When configured in capture mode, the processor can then use the GP timer to detect the bit rate of the received signal.

External Interface

Each UART features a `UART_RX` (receive) pin and a `UART_TX` (transmit) pin available through the general-purpose ports. These two pins usually connect to an external transceiver device that meets the electrical requirements of full-duplex or half-duplex standards. For example, EIA-232, EIA-422, 4-wire EIA-485 for full-duplex or 2-wire EIA-485, LIN for half-duplex. Additionally, the UART features a pair of clear-to-send, input pins (`UART_CTS`),

and request-to-send, output pins (UART_RTS) for hardware flow control. UART signals are multiplexed with other functions at the pin level.

Hardware Flow Control

To prevent the UART transmitter from sending data while the receiving counterpart is not ready, the UART features a UART_RTS/UART_CTS hardware flow control mechanism. The UART_RTS signal is an output that connects to the UART_CTS input of the communication partner. If data transfer is bidirectional, the figure shows the *UART Hardware Flow Control* handshake.

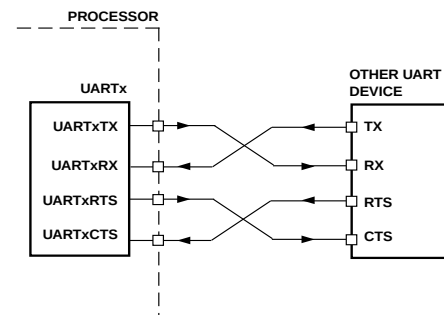


Figure 15-2: UART Hardware Flow Control

In both DMA and core mode, the receiver can deassert the UART_RTS signal to indicate that its receive buffer is almost full. Continued data transfers can cause an overrun error. The transmitter pauses when the UART_CTS input is in a deasserted state. In this state, the transmitter completes transmission of the data currently held in the transmit shift register (UART_TSR) but it does not continue with the data in the transmit hold register (UART_THR). If the UART_CTS pin is asserted again, the transmitter resumes and loads the content of UART_THR register into the UART_TSR register.

Bit Rate Generation

The peripheral clock (SCLK) and the 16-bit divisor in the UART_CLK register characterize the sample clock. The UART uses the UART_CTL.EN bit to enable the clock. By default, every serial bit is oversampled 16 times. The bit clock is 1/16th of the sample clock. If not in IrDA mode, the bit clock can equal the sample clock if the UART_CLK.EDBO bit is set, so that the following equation applies:

$$\text{Bit Rate} = \text{SCLK} / (16^{(1-\text{EDBO})} \times \text{Divisor})$$

The *UART Bit Rate Examples with 100 MHz SCLK* table provides example divide factors required to support standard baud rates at an SCLK of 100 MHz.

Table 15-7: UART Bit Rate Examples with 100 MHz SCLK

Bit Rate (bits/sec)	EDBO = 0			EDBO = 1		
	DL	Actual	% Error	DL	Actual	% Error
2400	2604	2400.15	0.006	41667	2399.98	0.001
4800	1302	4800.31	0.006	20833	4800.08	0.002

Table 15-7: UART Bit Rate Examples with 100 MHz SCLK (Continued)

Bit Rate (bits/sec)	EDBO = 0			EDBO = 1		
	DL	Actual	% Error	DL	Actual	% Error
9600	651	9600.61	0.006	10417	9599.69	0.003
19200	326	19171.78	0.147	5208	19201.23	0.006
38400	163	38343.56	0.147	2604	38402.46	0.006
57600	109	57339.45	0.452	1736	57603.69	0.006
115200	54	115740.74	0.469	868	115207.37	0.006
921600	7	892857.14	3.119	109	917431.19	0.452
1500000	4	1562500	4.167	67	1492537.31	0.498
3000000	2	3125000	4.167	33	3030303.03	1.01
6250000	1	6250000	0	16	6250000	0

NOTE: Carefully select the SCLK frequencies. Even multiples of the bit rate can decrease the error percentage.

Setting the bit clock equal to the sample clock (UART_CLK.EDBO=1) improves the bit rate granularity and the bit clock matches with the bit rate of the communication partner. Use UART_CLK.EDBO mode only when bit rate accuracy is not acceptable in the UART_CLK.EDBO=0 mode.

The UART_CLK.EDBO=1 mode is not intended to increase the speed of operation beyond the electrical limitations of the UART transfer protocol.

Autobaud Detection

At the chip level, the `UART_RX` pin is typically routed to an alternate capture input (`TIMER_ACI[n]`) of a general-purpose timer. When working in width capture mode, the processor uses this general-purpose timer to detect the bit rate applied to the `UART_RX` pin automatically by an external device. It often uses the capture capabilities of the timer to supervise the bit rate at run time. If the UART communicates with any device supplied by a weak clock oscillator that drifts over time, the processor can then readjust its UART bit rate dynamically, as required.

Often, the processor uses autobaud detection for initial bit rate negotiations where it is most likely a completer device waiting for the host to send a predefined autobaud character. This situation is common for UART booting. Do not enable the `UART_CTL.EN` bit while autobaud detection is in-process, to prevent the UART from starting a receive operation with incorrect bit rate matching. Alternatively, set the `UART_CTL.LOOP_EN` bit to disconnect the UART from its `UART_RX` pin.

A software routine can detect the pulse widths of serial stream bit cells. The sample base of the timer is synchronous with the UART operation (all derived from the same SCLK). The UART uses pulse widths to calculate the bit rate divider as follows:

$$\text{Divisor} = \text{TIMER_TMR}[n] _ \text{WID} / (16^{(1-\text{EDBO})} \times \text{Number of captured UART bits})$$

To increase the number of timer counts and the resolution of the captured signal, do not measure just the pulse width of a single bit. Instead, enlarge the pulse of interest over more bits. Traditionally, a NULL character (ASCII 0x00) is used in autobaud detection, as shown in the *Autobaud Detection* figure.

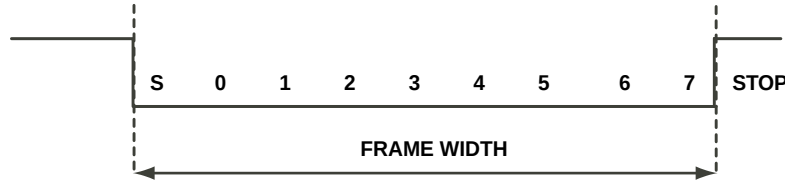


Figure 15-3: Autobaud Detection

Because the example frame encloses 8 data bits and 1 start bit, apply the following formula:

$$\text{Divisor} = \text{TIMER_TMR}[n]_WID / (16^{(1-\text{EDBO})} \times 9)$$

NOTE: For processor-specific mapping of timer alternate capture inputs to the UARTs of the processor, see "Width Capture (WIDCAP) Mode" in the "General-Purpose Timer (TIMER)" chapter.

Real receive signals often have asymmetrical falling and rising edges, and the sampling logic level is not exactly in the middle of the signal voltage range. At higher bit rates, such pulse-width-based autobaud detection does not always return adequate results without extra conditioning of the analog signal. Measure signal periods to work around this issue.

For example, predefine the ASCII character "@" (0x40) as the autobaud detection character and measure the period between two subsequent falling edges. As shown in the *Autobaud Detection Character 0x40* figure, measure the period between the falling edge of the start bit and the falling edge after bit 6. Since this period encloses 8 bits, apply the following formula:

$$\text{Divisor} = \text{TIMER_TMR}[n]_PER / (16^{(1-\text{EDBO})} \times 8)$$

or:

- Divisor = `TIMER_TMR[n]_PER >> 7`, if `UART_CLK.EDBO=0`
- Divisor = `TIMER_TMR[n]_PER >> 3`, if `UART_CLK.EDBO=1`

The *Autobaud Detection Character 0x40* figure shows the ASCII "@" (0x40) detection character.

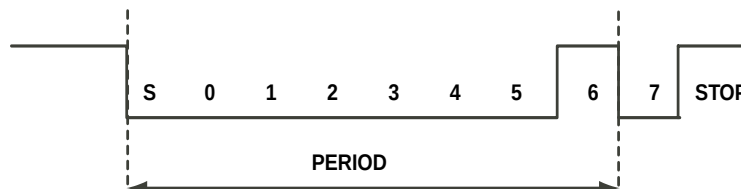


Figure 15-4: Autobaud Detection Character 0x40

UART Debug Features

The UART can automatically calculate and transmit a parity bit. The *UART Parity* table summarizes parity behavior assuming 8-bit data words (`UART_CTL.WLS=b#11`).

Table 15-8: UART Parity

PEN	STP	EPS	Data (hex)	Data (binary, LSB first)	Parity
0	x	x	x	x	None
1	0	0	0x60	0000 0110	1
1	0	0	0x57	1110 1010	0
1	0	1	0x60	0000 0110	0
1	0	1	0x57	1110 1010	1
1	1	0	x	x	1
1	1	1	x	x	0

The two force error bits, `UART_CTL.FPE` and `UART_CTL.FFE`, are intended for test purposes. They are useful for debugging software, especially in loopback mode.

The UART can be set to internal loopback mode (`UART_CTL.LOOP_EN=1`). Loopback mode disconnects the input of the receiver from the receive pin and internally redirects the transmit output to the receiver. The transmit pin remains active and continues to transmit data externally as well. Loopback mode also forces the `UART_RTS` pin to deassert, disconnects the `UART_STAT.CTS` bit from the `UART_CTS` input pin, and connects the internal version of `UART_RTS` to the `UART_STAT.CTS` bit.

Additionally, the `UART_TX` pin can be forced to zero asynchronously using the `UART_CTL.SB` bit.

UART Operating Modes

The following sections describe the main operating modes of the UART.

- [UART Mode](#)
- [IrDA SIR Mode](#)
- [Multi-Drop Bus Mode](#)

UART Mode

The UART mode follows an asynchronous serial communication protocol with these options:

- 1 start bit
- 5–8 data bits
- Address bit (available in MDB mode only)
- None, even, odd or sticky parity
- One, 1½, or two stop bits (1½ stop bits valid only in 5-bit word length)

The `UART_CTL` register controls the format of received and transmitted character frames. Data is always transmitted and received with the least significant bit (LSB) first.

The *Bit Stream on a UART TX Pin Transmitting an “S” Character (0x53)* figure shows a typical physical bit stream measured on a `UART_TX` pin.

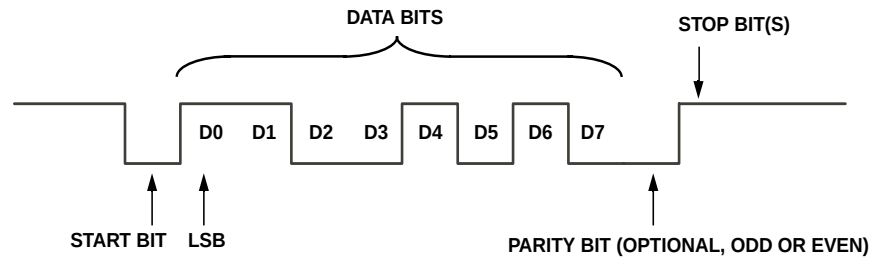


Figure 15-5: Bit Stream on a UART TX Pin Transmitting an “S” Character (0x53)

IrDA SIR Mode

The UART also supports serial data communication by way of infrared signals, according to the recommendations of the Infrared Data Association (IrDA). The physical layer known as IrDA SIR (9.6/115.2 Kbps rate) is based on return-to-zero-inverted (RZI) modulation. The UART does not support pulse position modulation.

Using the 16x data rate clock, RZI modulation is achieved by inverting and modulating the non-return-to-zero (NRZ) code normally transmitted by the UART. On the receive side, the UART uses a 16x clock to determine an IrDA pulse sample window, from which it recovers the RZI modulated NRZ code.

NOTE: The `UART_CLK.EDBO` bit is not valid in IrDA mode. Clear (=0) this bit in this mode.

Multi-Drop Bus Mode

The UART uses a protocol for point-to-point connections as well as in networks where the EIA-485 standard is representative of UART-based bus systems. The EIA-232 standard defines point-to-point connections. In such networks, node addressing is important.

In a multidrop bus (MDB) network, for example, an address bit enhances the UART frame. The address bit is inserted between the data bits and the optional parity bit. To configure the UART for MDB mode, set the mode of operation bits (`UART_CTL.MOD [5:4]`) to 01.

By convention, the address bit is transmitted low for regular data bytes. When set, it marks special address bytes that require the attention of all nodes on the network.

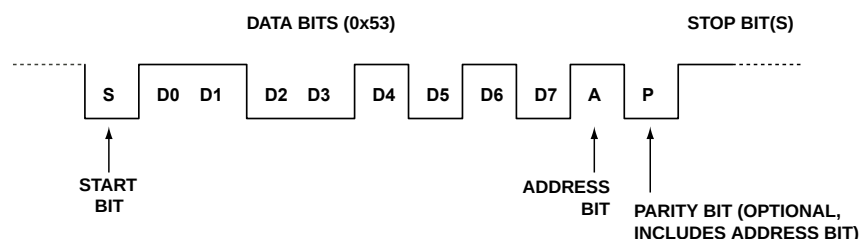


Figure 15-6: UART Frame with Address Bit

All transmit operations are processed through the transmit buffer register ([UART_THR](#)), so all DMA data transmissions clear the address bit. If data is written to the transmit address or insert pulse register ([UART_TAIP](#)) instead, the same transmit operation is initiated with the only exception that the address bit is sent high.

The UART uses the `UART_STAT.ADDR` bit of the receiver to signal whether the previously received frame had the address bit set or not. Hardware updates it every time a new frame is received. When the enable address word interrupt bit (`UART_IMSK.EAWI`) is set, the reception of an address byte triggers a special status interrupt request.

The address sticky bit (`UART_STAT.ASTKY`) is the sticky version of the `UART_STAT.ADDR` bit. Hardware sets it whenever the `UART_STAT.ADDR` bit is set. Software can clear the `UART_STAT.ASTKY` bit with a W1C operation.

In MDB mode, only address bytes progress to the receive FIFO by default. Data bytes are gated unless the `UART_STAT.ASTKY` bit is set. The receiver ignores all traffic on the UART bus. This way, the processor can go into low-power mode and interrupt activity does not load the processor every time a frame is transmitted on the UART bus. If, however, an address frame is transmitted, the receiver immediately samples all further traffic. A software routine can analyze the received data, decide whether it was of relevance for the local network node, and W1C the `UART_STAT.ASTKY` bit if it was not.

Software can overrule of the hardware address frame detection by setting the `UART_STAT.ADDR` bit and (indirectly) the `UART_STAT.ASTKY` bit with a W1S operation.

The MDB mode follows an asynchronous serial communication protocol with the following options.

- 1 start bit
- 5–8 data bits
- Address bit
- None, even, odd or sticky parity
- One, 1½, or two stop bits (1½ stop bits are valid only in 5-bit word length)

NOTE: If the address bit and parity bit are both enabled, the parity check and generation includes the address bit in its parity calculation.

UART Data Transfer Modes

The UART can transfer data using both the core and DMA. Receive and transmit paths operate independently except that the bit rate and the frame format are identical for both transfer directions. Transmit and receive channels are both buffered. The [UART_THR](#) register buffers the transmit shift register ([UART_TSR](#)) and the [UART_RBR](#) register buffers the receive shift register ([UART_RSR](#)).

UART Mode Transmit Operation (Core)

In core mode, the processor core moves data to and from the UART. A write to the [UART_THR](#) register initiates the transmit operation. If no former operation is pending, the [UART_THR](#) register passes the data immediately to the

`UART_TSR` register. There, it is shifted out at the bit rate characterized by the `UART_CLK` register, with start, stop, and parity bits appended as defined by the `UART_CTL` register.

The `UART_THR` register and the `UART_TSR` register can be modeled as a two-stage transmit buffer. The least significant bit (LSB) always transmits first. This bit is bit 0 of the value written to the `UART_THR` register.

UART Mode LIN Break Command

Some UART-based protocols demand synchronization methods that are not native to standard UART implementations. For example, the Local Interconnect Network (LIN) protocol requires a low-pulse of well-defined transmit length as a prologue to every multi-byte message. Its length must be at least 13 bit-times.

With previous UARTs, there were two options to implement this protocol:

- A null byte is transmitted with a temporarily lowered bit rate, or
- A software counter generates the period, and the asynchronous set break (SB) mechanisms pull the transmit pin low

Since both methods have their disadvantages, the newer UART introduces a new inter-frame gap technique.

The feature is not available in MDB or IrDA operating modes. However, in standard UART mode (bits `UART_CTL.MOD[5:4] = 00`), a write to the `UART_TAIP` register initiates the transmission of an inter-frame pulse. If the transmit buffer is not empty, the UART first transmits all bytes in the queue. It only initiates with pulse generation after the last stop bit of the last byte has been shifted out.

The value written into the `UART_TAIP` register defines the nature and the duration of the transmitted pulse. Bits [6:0] control the duration in bit-times and bit [7] controls the value (duration = `UART_TAIP[6:0] / UART_CLK[15:0]`). If `UART_TAIP[7]` is set, and an active high pulse is issued, the number of stop bits is extended. If `UART_TAIP[7]` is cleared, a low pulse is generated. Invert the polarity using the `UART_CTL.FCPOL` bit. Writing a value of 13 into the `UART_TAIP` register generates the break command as required by the LIN protocol.

NOTE: If the `UART_CTL.TPOLC` bit is enabled, an inverted most-significant bit can be transmitted.

NOTE: If another transmission is pending (in the `UART_TSR` register), the `UART_TAIP` initiated pulse is queued until after all pending operations have finished and all stop bits are transmitted.

The transmission of break command/inter-frame gap precedes transmission of the number of stop bits as set in the `UART_CTL.STB` and `UART_CTL.STBH` bit fields.

The UART receiver can detect break commands through the break indicator (`UART_STAT.BI`) flag. This flag reports that an entire UART frame has been received in low state. It does not report whether the duration of the received low pulse was exact or at least 13 bit-times as LIN controllers transmit. Typically, the break indicator meets LIN requirements. The processor can use GP timers to determine the pulse width more precisely, if necessary.

Each `UART_RX` pin is also routed to a GP timer through its alternate capture input (TACI). This functionality is not only useful for bit rate detection (*autobaud*) but also helps to measure the pulse widths precisely on the `UART_RX` input. Additionally, the GP timers can issue an interrupt request or a fault condition when the received

pulse width is shorter than a bit time or longer than the worst-case break condition. The windowed watchdog width mode of the GP timers controls this functionality.

UART Mode Receive Operation (Core)

The receive operation uses the same data format as the transmit configuration except that one valid stop bit is always sufficient. The `UART_CTL.STB` and `UART_CTL.STBH` bits have no impact on the receiver.

The UART receiver senses the falling edges of the receive input. When it detects an edge, the receiver starts sampling the input according to settings in the `UART_CLK` register. The receiver samples the start bit (majority sampling) close to its midpoint. If sampled low, it assumes a valid start condition. Otherwise, it discards the detected falling edge.

After detection of the start bit, the received word is shifted into the `UART_RSR` register.

After the corresponding stop bit is received, the content of the `UART_RSR` register is transferred to the 8-deep receive FIFO and is accessible by reading the `UART_RBR` register.

The receive FIFOs and the `UART_RBR` register act as a 9-stage receive buffer. If the stop bit of the ninth word is received before software reads the `UART_RBR` register, an overrun error is reported. Overruns protect data in the `UART_RBR` register and the receive FIFO from being overwritten by further data until the software clears the `UART_STAT.OE` bit. However, the data in the `UART_RSR` register is immediately destroyed as soon as the overrun occurs.

The sampling clock is 16 times faster than the bit clock. The receiver oversamples every bit 16 times and makes a majority-decision based on the middle three samples. This functionality improves immunity against noise and hazards on the line. The receiver disregards spurious pulses of less than two times the sampling clock period.

Normally, the receiver samples every incoming bit at exactly the 7th, 8th, and 9th sample clock. If, however, the `UART_CLK.EDBO` bit is set to 1, the receiver samples bits approximately at 7/16th, 8/16th, and 9/16th of their period. This configuration achieves better bit rate granularity and accuracy as required at high operation speeds. Hardware design must ensure that the incoming signal is stable between 6/16th and 10/16th of the nominal bit period.

Reception starts when the UART receiver detects a falling edge on the `UART_RX` input pin. The receiver attempts to see a start bit. The data is shifted into the `UART_RSR` register. After the ninth sample of the first, the receiver processes the stop bit and copies the received data to the 8-stage receive FIFO. The `UART_RSR` recovers for further data reception.

The receiver samples data bits close to their midpoint. Because the receiver clock is typically asynchronous to the data rate of the transmitter, the sampling point can drift relative to the center of the data bits. The sampling point is synchronized again with each start bit, so the error accumulates only over the length of a single word. The polarity of received data is selectable, using the `UART_CTL.RPOLC` bit.

NOTE: The receiver checks for only a single stop bit. After the third sample of the first stop bit has been received (at time 9/16th of the stop bit duration), the receiver immediately acts (status update). It then prepares for new falling edge detection (start detection).

IrDA Transmit Operation

To generate the IrDA pulse transmitted by the UART, the normal NRZ output of the transmitter is first inverted if the `UART_CTL.TPOLC` bit is configured for active-low operation. In this configuration, a zero is transmitted as a high pulse of 16 UART clock periods and a one is transmitted as a low pulse for 16 UART clock periods. Then, six UART clock periods delay the leading edge of the pulse. Similarly, eight UART clock periods truncate the trailing edge of the pulse. For a 16-cycle UART clock period, this operation results in the final representation of the original zero as a high pulse of only 3/16 clock periods. The *IrDA Transmit Pulse* figure shows how the pulse is centered around the middle of the bit time. The final IrDA pulse is fed to the off-chip infrared driver.

This modulation approach ensures a pulse width output from the UART of three cycles high out of every 16 UART clock cycles. As shown in the *IrDA Transmit Pulse* figure, the error terms associated with the bit rate generator are small and well within the tolerance of most infrared transceiver specifications.

NOTE: In IrDA mode, writes to the `UART_TAIP` register are equivalent to writes to the `UART_THR` register.

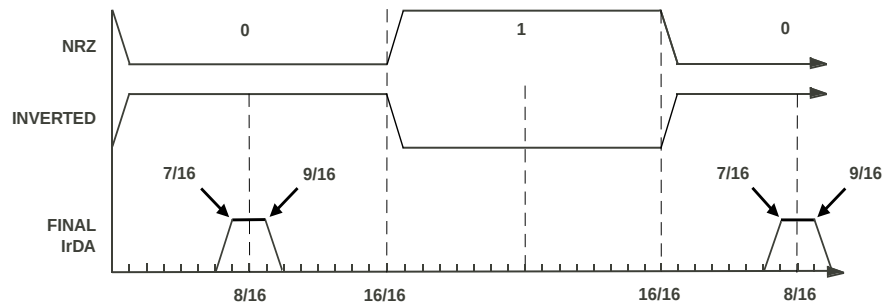


Figure 15-7: IrDA Transmit Pulse

IrDA Receive Operation

The IrDA receiver function is more complex than the transmit function. The receiver must discriminate the IrDA pulse and reject noise. The receiver looks for the IrDA pulse in a narrow window centered around the middle of the expected pulse.

Glitch filtering is accomplished by counting 16 system clocks from the time the receiver detects an initial pulse. If the pulse is absent when the counter expires, the receiver interprets it as a glitch. Otherwise, the receiver interprets it as a zero. This assessment is acceptable because glitches originating from on-chip capacitive cross-coupling typically do not last for more than a fraction of the system clock (SCLK) period. Appropriate shielding avoids sources outside of the chip and not part of the transmitter. The only other source of a glitch is the transmitter itself. The processor relies on the transmitter to perform within specification. If the transmitter violates the specification, unpredictable results can occur. The 4-bit counter adds an extra level of protection at a minimal cost.

NOTE: Because SCLK can change across systems, the longest glitch tolerated is inversely proportional to the SCLK frequency.

A counter that is clocked at the 16x bit-time sample clock determines the receive sampling window. The sampling window is resynchronized with each start bit by centering the sampling window around the start bit.

The polarity of receive data is selectable, using the `UART_CTL.RPOLC` bit. The *IrDA Receiver Pulse Detection* figure provides examples of each polarity type.

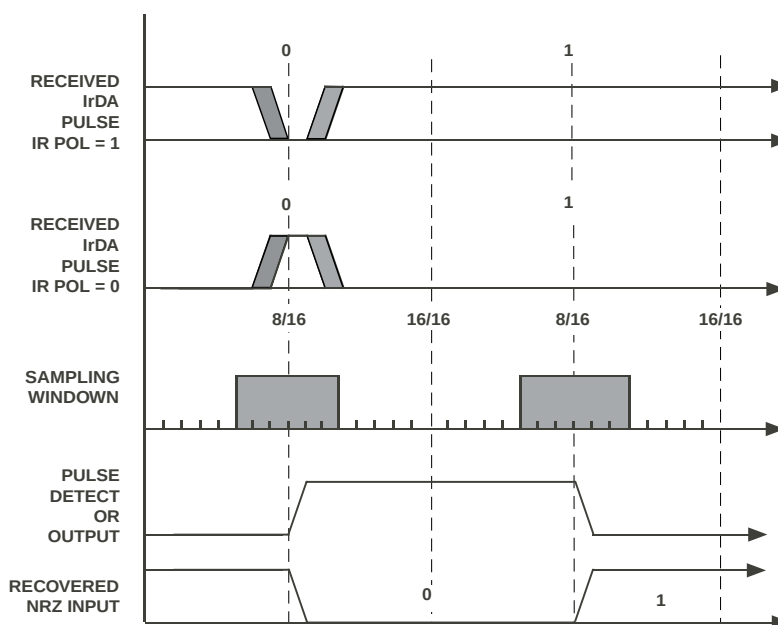


Figure 15-8: IrDA Receiver Pulse Detection

MDB Transmit Operation

In MDB mode, receive and transmit paths operate independently from each other, except for sharing bit rate and frame formats for both transfer directions.

Transmit operation is initiated by writing the `UART_THR` or `UART_TAIP` registers. A write to the `UART_THR` register transmits the written word with the appending address bit set low. A write to the `UART_TAIP` register transmits the written word with the appended address bit set high. The data is moved into the `UART_TSR` register, where it is shifted out at the bit rate programmed by the `UART_CLK` register, with start, stop, address, and parity bits appended, as required.

If DMA is enabled, the DMA engine always writes the data into the `UART_THR` register, and the written word is transmitted with the appending address bit set low.

The polarity of transmit data is selectable, using the `UART_CTL.TPOLC` bit.

MDB Receive Operation

Receive operations use the same data format as the transmit configuration, except that the number of stop bits is always assumed to be 1. After detection of the start bit, the received word is shifted into the `UART_RSR` register at the programmed bit.

Normally, the receiver samples every incoming bit at exactly the 7th, 8th, and 9th sample clock. If, however, the `UART_CLK.EDBO` bit is set, the receiver samples the bits approximately at 7/16th, 8/16th, and 9/16th of their period. This configuration achieves better bit rate granularity and accuracy needed at high operation speeds. Hardware design must ensure that the incoming signal is stable between 6/16th and 10/16th of the nominal bit period.

After the appropriate number of bits (including address, parity, and stop bits) is received, the `UART_RSR` register is transferred to the receive FIFO and accessible through the `UART_RBR` register.

The polarity of receive data is selectable, using the `UART_CTL.RPOLC` bit.

DMA Mode

In DMA mode, separate receive and transmit DMA channels move data between the UART and memory. The software does not have to move data; it just must set up the appropriate transfers either through the descriptor mechanism or through autobuffer mode.

DMA channels provide a 4-deep FIFO, resulting in total buffer capabilities of 6 words at the transmit side and 9 words at the receive side. In DMA mode, the bus activity and arbitration mechanism determine the latency. The processor loading and interrupt priorities do not determine the latency.

To enable UART DMA, first set up the system DMA control registers. Then, enable the `UART_IMSK.ERBFI` or `UART_IMSK.ETBEI` interrupts. This sequence is necessary because these interrupt request lines double as DMA request lines. With DMA enabled, once these requests are received, the DMA control unit generates a direct memory access. If DMA is not enabled, the UART interrupt is passed on to the system interrupt handling unit. The status interrupt for the UART goes directly to the system event controller (SEC), bypassing the DMA unit completely.

For transmit DMA, programs must set the `DMA_CFG.SYNC` bit. With this bit set, interrupt generation is delayed until the entire DMA FIFO is drained to the UART module. The UART transmit DMA interrupt service routine can disable the DMA or to clear the `UART_IMSK.ETBEI` control bit only when the `DMA_CFG.SYNC` bit is set. Otherwise, up to four data bytes can be lost.

When the `UART_IMSK.ETBEI` bit is set, an initial transmit DMA request is issued immediately. The program then clears the `UART_IMSK.ETBEI` bit through the DMA service routine.

In DMA transmit mode, the `UART_IMSK.ETBEI` bit enables the peripheral request to the DMA FIFO. The `DMA_CFG.EN` bit enables the strobe on the memory side. If the DMA count is less than the DMA FIFO depth, which is 4, then the DMA interrupt can be requested before the `UART_IMSK.ETBEI` bit is set. If this behavior is unwanted, set the `DMA_CFG.SYNC` bit.

Regardless of the `DMA_CFG.SYNC` setting, the DMA stream has not left the UART transmitter completely at the time the interrupt request is generated. Transmission can abort in the middle of the stream, causing data loss, when the UART clock was disabled without extra synchronization with the `UART_STAT.TEMT` bit.

The UART provides functionality to avoid resource-consuming polling of the `UART_STAT.TEMT` bit. The `UART_IMSK_SET.EDTPTI` bit enables the `UART_STAT.TEMT` bit to trigger a DMA interrupt. To delay the DMA completion interrupt until the last data word of a STOP DMA has left the UART, keep the `DMA_CFG.INT` bit cleared and set the `UART_IMSK_SET.EDTPTI` bit instead. Then, the normal DMA completion interrupt is suppressed. Later, the `UART_STAT.TEMT` event triggers a DMA interrupt after the last word of the DMA has left the UART transmit buffers. If `DMA_CFG.INT` and `UART_IMSK.EDTPTI` are set, when finishing STOP mode, the DMA requests two interrupts.

The DMA of the UART module supports 8-bit and 16-bit operation, but not 32-bit operation. It does not support sign-extension.

Mixing DMA and Core Modes

Switching from DMA mode to core operation dynamically requires some consideration, especially for transmit operations. By default, the interrupt timing of the DMA is synchronized with the memory side of the DMA FIFOs. Normally, the transmit DMA completion interrupt is generated after the last byte is copied from the memory into the DMA FIFO. The transmit DMA interrupt service routine is not yet permitted to disable the `DMA_CFG.EN` bit. The interrupt is requested when the `DMA_STAT.IRQDONE` bit is set. The `DMA_STAT.RUN` bit, however, remains set until the data has completely left the transmit DMA FIFO.

When planning to switch from a DMA to core mode, set the `DMA_CFG.SYNC` bit in the word of the last descriptor or work unit before handing over control. Then, after the interrupt request occurs, software can write new data into the `UART_THR` register as soon as the `UART_STAT.THRE` bit permits. If the `DMA_CFG.SYNC` bit cannot be set, software can poll the `DMA_STAT.RUN` bit instead. Alternatively, using the `UART_IMSK.EDTPTI` bit can avoid expensive status bit polling.

When switching from core to DMA operation, ensure that the first DMA request is issued properly. If the DMA is enabled while the UART is still transmitting, no precaution is required. If, however, the DMA is enabled after the `UART_STAT.TEMT` bit is high, pulse the `UART_IMSK.ETBEI` bit to initiate DMA transmission.

Setting Up Hardware Flow Control

The following steps show how to set up UART hardware flow control:

1. Configure automatic or manual hardware flow control for the receiver through the `UART_CTL.ARTS` bit, or the transmitter through the `UART_CTL.ACTS` bit.
2. Configure `UART_CTS` and `UART_RTS` polarity through the `UART_CTL.FCPOL` bit.

On reset, when the UART is not yet enabled and the port multiplexing has not been programmed, the `UART_RTS` pin is not driven. Some applications require a resistor to pull the `UART_RTS` signal to either state during reset.

UART Event Control

Status flags in the `UART_STAT` register are available to signal data reception, parity, and error conditions, if necessary.

DMA and Interrupt Multiplexing

See the *Direct Memory Access (DMA)* chapter for information on DMA multiplexing. Several interrupts and DMA channels in the UART can be multiplexed.

NOTE: To operate in interrupt mode without using DMA channels, set the `UART_IMSK.ELSI` bit. This configuration redirects receive and transmit requests to the status interrupt output. The status interrupt goes directly to the SEC without going through the DMA controller.

Interrupt Masks

Each UART features a set of interrupt mask registers: `UART_IMSK`, `UART_IMSK_SET`, and `UART_IMSK_CLR`. The `UART_IMSK` register supports read/write operations. Writing ones to the `UART_IMSK_SET` register enables interrupts, writing ones to the `UART_IMSK_CLR` register disables them. Reads from either register return the enabled bits. This way, different interrupt service routines can control transmit, receive, and status interrupt requests independently and easily.

The UART module uses the `UART_IMSK` registers to enable requests for system handling of empty or full states of data registers. Unless polling is used as a means of action, the `UART_IMSK.ERBFI` and `UART_IMSK.ETBEI` bits in this register are normally set.

Each UART module has three interrupt outputs. It uses one for transmission, one for reception, and one for reporting status events. The UART module routes transmit and receive requests through the DMA controller. The status request goes directly to the system event controller (SEC).

If the associated DMA channel is enabled, the request functions as a DMA request. If the DMA channel is disabled, it simply forwards the request to the SEC. A DMA channel must be associated with the UART module to enable transmit and receive interrupts. Otherwise, transmit and receive requests cannot be forwarded.

NOTE: To operate in interrupt mode without using DMA channels, set the `UART_IMSK.ELSI` bit. This configuration redirects receive and transmit requests to the status interrupt request output. The status interrupt goes directly to the SEC without going through the DMA controller.

Interrupt Servicing

Interrupt service routines (ISRs) perform UART writes and reads. Separate interrupt lines are provided for transmit, receive, and status. The `UART_IMSK` register group enables the independent interrupts individually. To enable UART transmit interrupts, set the `UART_CTL.EN` bit.

The ISRs evaluate the status bits in the `UART_STAT` register to determine the signaling interrupt source. The system event controller for the processor assigns and unmask interrupts. The ISRs must clear the interrupt latches explicitly. To reduce interrupt frequency on the receive side in core mode, use the `UART_IMSK.ERFCI` status interrupt as an alternative to the regular `UART_IMSK.ERBFI` receive interrupt. Hardware must ensure that at least two (if `UART_CTL.RFIT=0`) or four (if `UART_CTL.RFIT=1`) words are available in the receive buffer by the time the interrupt is requested.

Transmit Interrupts

The UART module uses the `UART_IMSK_SET.ETBEI` bit to enable transmit interrupt requests.

The `UART_THR` and `UART_TAIP` registers are the same physical register, and both affect the signaling of the `UART_STAT.TEMT`, `UART_STAT.TFI`, and `UART_STAT.THRE` bits similarly.

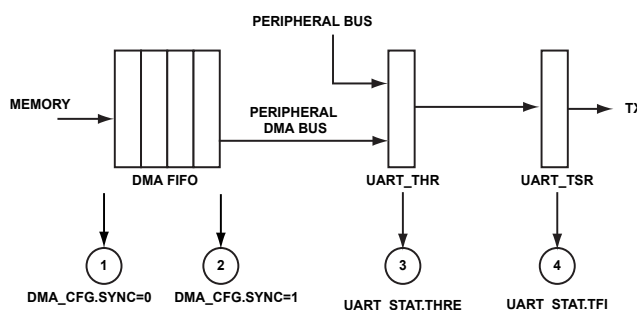


Figure 15-9: Transmit Interrupts

The UART module asserts the transmit request along with the `UART_STAT.THRE` bit, indicating that the transmit buffer is ready for new data. The `UART_STAT.THRE` bit resets to 1. When the `UART_IMSK_SET.ETBEI` bit is set, the UART module immediately issues an interrupt or DMA request. This way, no special handling of the first character is required when transmission of a string is initiated. Set the `UART_IMSK_SET.ETBEI` bit and let the interrupt service routine load the first character from memory and write it to the `UART_THR` register in the normal manner. ISRs can clear the `UART_IMSK.ETBEI` bit through the `UART_IMSK_CLR` register when the string transmission has completed.

Hardware clears the `UART_STAT.THRE` bit when new data is written to the `UART_THR` register. These write operations also clear the transmit interrupt request. However, they also initiate further transmission. If continued transmission is undesirable, the UART module can alternatively clear the transmit request through the `UART_IMSK_CLR.ETBEI` bit register. Transfers of data from the `UART_THR` register to the `UART_TSR` register reset this status flag in the `UART_STAT` register.

ISRs can interrogate the `UART_STAT.TEMT` bit to discover any ongoing transmission. The sticky counterpart of the `UART_STAT.TEMT` bit, `UART_STAT.TFI`, indicates when the transmit buffer has drained and can trigger a status interrupt. When data is pending in either one of these registers, the `UART_STAT.TEMT` flag is low. As soon as all data has left the `UART_TSR` register, the `UART_STAT.TEMT` bit goes high again and indicates that all pending transmit operations (including stop bits) have finished. Then, it is safe to disable the `UART_CTL.EN` bit or to three-state off-chip line drivers. Then, the UART module can generate an interrupt either through the status interrupt channel when the `UART_IMSK.ETFI` bit is set, or through the DMA controller when enabled by the `UART_IMSK.EDTPTI` bit.

When enabled by the `UART_IMSK.ETBEI` bit, the `UART_STAT.THRE` flag requests data along the peripheral command lines to the DMA controller (referred to as TXREQ). This signal is routed through the DMA controller. If the associated DMA channel is enabled, the TXREQ signal functions as a DMA request, otherwise the DMA controller simply forwards it to the SEC. Alternatively the `UART_IMSK.ETXS` bit can redirect the transmit interrupts to the UART status interrupt.

With interrupts disabled, the UART module can poll the status flags to determine when data is ready to move. Because polling is processor intensive; it is not typically used in real-time signal processing environments. Since read operations from `UART_STAT` registers have no side effects, different software threads can interrogate these registers without mutual impacts.

Polling the `SEC_SSTAT[n]` register without enabling the interrupts by the `SEC_CCTL[n]` register is an alternate method of operation to consider. Software can write up to two words into the `UART_THR` register before enabling the UART clock. As soon as the `UART_CTL.EN` bit is set, the UART module sends those two words.

Receive Interrupts

The UART module uses the `UART_IMSK_SET.ERBFI` bit to enable receive interrupt requests. If set, the `UART_STAT.DR` flag requests an interrupt on the dedicated RXREQ output, indicating that new data is available in the `UART_RBR` register. This signal is routed through the DMA controller. If the associated DMA channel is enabled, the RXREQ signal functions as a DMA request; otherwise the DMA controller simply forwards it to the SEC. Alternatively, if no DMA channel is assigned to the UART, the `UART_IMSK.ERXS` bit can redirect the receive interrupts to the UART status interrupt. When software reads the `UART_RBR` register, hardware clears the `UART_STAT.DR` bit again, which, in turn, clears the receive interrupt request.

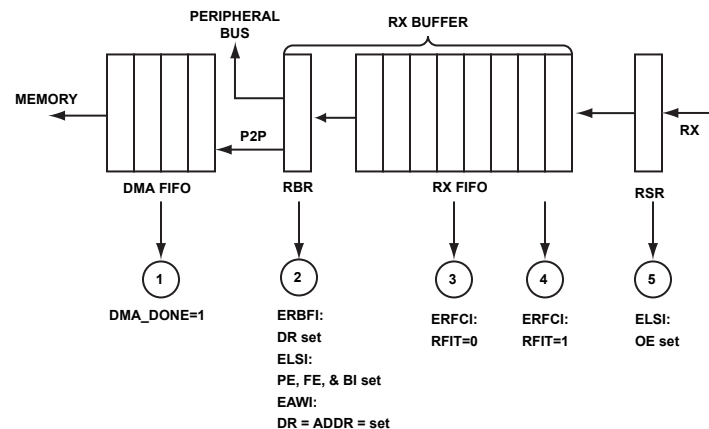


Figure 15-10: Receive Interrupts

Hardware updates the following:

- `UART_STAT.DR` bits
- `UART_STAT.ADDR` bits
- `UART_STAT.ASTKY` bits
- `UART_STAT.PE` bits
- `UART_STAT.FE` bits
- `UART_STAT.BI` bits
- `UART_RBR` register

The `UART_STAT.OE` bit is updated as soon as an overflow condition occurs (for example when a stop bit for a frame is received and the receive FIFO is full). When software does not read the `UART_RBR` register in time, the received data is protected from being overwritten by new data until software clears the `UART_STAT.OE` bit. Only the content of the `UART_RSR` register can be overwritten in the overrun case.

The UART module uses the `UART_STAT.RFCS` bit to monitor the state of the 8-deep receive FIFO. It uses the `UART_CTL.RFIT` bit to control the behavior of the buffer. If `UART_CTL.RFIT` is zero, the `UART_STAT.RFCS` bit is set when the receive buffer holds four or more words. If `UART_CTL.RFIT` is set, the `UART_STAT.RFCS` bit is set when the receive buffer holds seven or more words. Hardware clears the `UART_STAT.RFCS` bit when a core or DMA reads the `UART_RBR` register and when the buffer is flushed below the level of four (`UART_CTL.RFIT=0`) or seven (`UART_CTL.RFIT=1`). If the associated interrupt bit `UART_IMSK.ERFCI` is enabled, a status interrupt request is reported when the `UART_STAT.RFCS` bit is set.

If errors are detected during reception, an interrupt can be requested from the status interrupt output. This status interrupt request goes directly to the SEC. The bit enables status interrupt requests.

The controller detects the following error conditions, shown with their associated bits in the `UART_STAT` register.

- Overrun error (`UART_STAT.OE` bit)
- Parity error (`UART_STAT.PE` bit)
- Framing error or invalid stop bit (`UART_STAT.FE` bit)
- Break indicator (`UART_STAT.BI` bit)

Status Interrupts

The UART module uses status interrupt channels for the following purposes:

- Line status interrupt requests
- Flow control interrupt requests
- Receive FIFO threshold interrupt requests
- Transmission finished interrupt request

The UART module uses the `UART_IMSK.ELSI` bit to enable the line status interrupts. If set, the status interrupt request is asserted with one of the `UART_STAT.BI`, `UART_STAT.FE`, `UART_STAT.PE`, or `UART_STAT.OE` receive errors bits. A W1C operation to the `UART_STAT` register clears the error bits. Once all error conditions are cleared, the interrupt request deasserts.

The UART module uses the `UART_IMSK_SET.ERFCI` bit to enable the receive FIFO count interrupt. If set, a status interrupt request is generated when the `UART_STAT.RFCS` is active. The `UART_STAT.RFCS` bit indicates a receive buffer threshold level. If the `UART_CTL.RFIT` bit is cleared, software can safely read two words out of the `UART_RBR` register by the time the `UART_STAT.RFCS` interrupt occurs.

If the `UART_CTL.RFIT` bit is set, software can safely read four words. The interrupt request and the `UART_STAT.RFCS` bit are cleared when the `UART_RBR` is read enough of times, so that the receive buffer drains below the threshold of two (`UART_CTL.RFIT=0`) or four (`UART_CTL.RFIT=1`). Because in DMA mode a status service routine may not be permitted to read `UART_RBR`, this interrupt is only recommended in core mode. In DMA mode, use this functionality for error recovery only.

The UART module uses the `UART_IMSK_SET.EDSSI` bit to enable the flow control interrupts. If active, a status interrupt is generated when the sticky `UART_STAT.SCTS` bit register is set, indicating that the `UART_CTS` input of the transmitter been reasserted. A W1C operation to the `UART_STAT.SCTS` bit clears the interrupt request.

The UART module uses the `UART_IMSK_SET.ETFI` bit to enable the transmission finished interrupt. If active, a status interrupt request is asserted when the `UART_STAT.TFI` bit is set. The `UART_STAT.TFI` is the sticky version of the `UART_STAT.TEMT` bit, indicating that a byte that started transmission has finished. A W1C operation to the `UART_STAT.TFI` bit clears the interrupt request.

Multi-Drop Bus Events

Several status bits and interrupt features in the `UART_STAT` and `UART_IMSK` registers facilitate efficient data handling in multi-drop bus mode. These features include the address (`UART_STAT.ADDR`) bit, the address sticky (`UART_STAT.ASTKY`) bit and the enable address word interrupt (`UART_IMSK.EAWI`). One of the key features of the multi-drop bus protocol is its address bit. The address bit signifies to the completer that the requester is transmitting an address word (all read it) or a data word (only the addressed target reads its). The UART hardware provides an efficient method of handling the situation described with the use of `UART_STAT.ASTKY` bit.

NOTE: The UART module uses the `UART_STAT.ASTKY` bit in multi-drop bus mode to indicate when an address operation for a peripheral is occurring. The `UART_STAT.ASTKY` bit is a sticky version of the `UART_STAT.ADDR` bit. Hardware sets the bit whenever the `UART_STAT.ADDR` bit is set. Only software clears it with a W1C operation. With the `UART_STAT.ASTKY` bit set, words are received irregardless of the mode bit or address bit setting. With the `UART_STAT.ASTKY` bit cleared, only address words (mode bit =1) are received and words with mode bit =0 are ignored in MDB mode. This bit does not affect reception in non-MDB modes. (Words with mode bit =0 are not moved from the `UART_RSR` register to the receive FIFO.)

UART Programming Model

The following sections provide basic procedures for configuring various UART operations.

Detecting Autobaud

To detect Autobaud:

1. Ensure that the timer is disabled.
2. Configure the following bits: `UART_CTL.MOD = 00`, `UART_CTL.LOOP_EN = 1`, `UART_CTL.WLS = 11` (8-bit data), and `UART_CTL.EN = 1`
3. Configure the following bits: `TIMER_TMR[n]_CFG.TMODE = 1101`, `TIMER_TMR[n]_CFG.OUTDIS = 1`, `TIMER_TMR[n]_CFG.IRQMODE = 10` and enable the timer.
4. Send test data through the host device and wait for the timer interrupt and disable the timer.

The bit rate can be derived from the timer period register value according to the formula provided in the [Autobaud Detection](#) section.

Using Common Initialization Steps

When using the core or the DMA to execute transfers, the following steps are common to all UART modes.

1. All UART signals are multiplexed and compete with other functions at pin level. First, program the port registers according to the guidelines in the PORT chapter.
2. Program the `UART_CLK` register. Refer to [Bit Rate Generation](#) topic.
3. Program the `UART_CTL` register and enable the UART clock.

Using Core Transfers

Write data into the `UART_THR` register, when the `UART_STAT.THRE` bit is set, to initiate a core transmit operation. If the `UART_STAT.DR` bit is set, received data can be read from the `UART_RBR` register.

Using DMA Transfers

1. Make sure that the `UART_IMSK.ETBEI` or the `UART_IMSK.ERBFI` bits are cleared before configuring the DMA.
2. Configure the dedicated DMA channel.
3. Set the `UART_IMSK.ETBEI` or `UART_IMSK.ERBFI` bits to start the transfer.

Using Interrupts

Each UART features three interrupt signal outputs.

1. Enable individual interrupts in the system event controller (SEC)
2. Register IRQ handlers.
3. Use the interrupts mask registers to enable specific IRQ events.

Setting Up Hardware Flow Control

1. Configure automatic or manual hardware flow control for the receiver through the `UART_CTL.ARTS` bit, or the transmitter through the `UART_CTL.ACTS` bit.
2. Configure `UART_CTS` and `UART_RTS` polarity through the `UART_CTL.FCPOL` bit.

ADSP-21560_21561_21564_21568 UART Register Descriptions

UART (UART) contains the following registers.

Table 15-9: ADSP-21560_21561_21564_21568 UART Register List

Name	Description
<code>UART_CLK</code>	Clock Rate Register

Table 15-9: ADSP-21560_21561_21564_21568 UART Register List (Continued)

Name	Description
UART_CTL	Control Register
UART_IMSK	Interrupt Mask Register
UART_IMSK_CLR	Interrupt Mask Clear Register
UART_IMSK_SET	Interrupt Mask Set Register
UART_RBR	Receive Buffer Register
UART_RSR	Receive Shift Register
UART_RXCNT	Receive Counter Register
UART_SCR	Scratch Register
UART_STAT	Status Register
UART_TAIP	Transmit Address/Insert Pulse Register
UART_THR	Transmit Hold Register
UART_TSR	Transmit Shift Register
UART_TXCNT	Transmit Counter Register

Clock Rate Register

The `UART_CLK` register divides the system clock (SCLK) down to the bit clock.

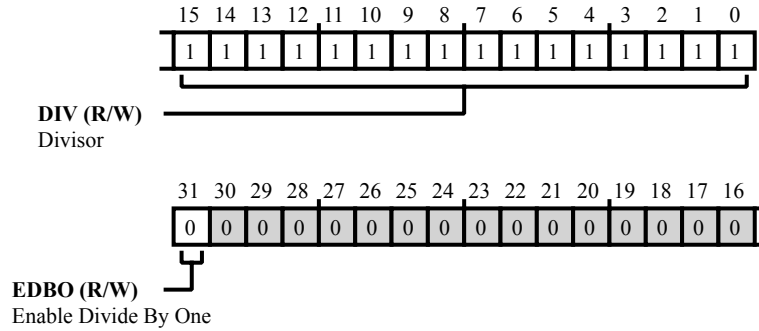


Figure 15-11: UART_CLK Register Diagram

Table 15-10: UART_CLK Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	EDBO	Enable Divide By One. The <code>UART_CLK.EDBO</code> bit enables the bypassing of the divide-by-16 prescaler in bit clock generation. This functionality improves bit rate granularity, especially at high bit rates. Do not set this bit in IrDA mode. Note: Properly select the SCLK frequencies. Even multiples of bit rate decrease the error percentage. Setting the bit clock equal to the sample clock (<code>UART_CLK.EDBO = 1</code>) improves the bit rate granularity and the bit clock matches with the bit rate of the communication partner. The disadvantage is that the power dissipation is higher and sample points are not always accurate. Therefore, use <code>UART_CLK.EDBO = 1</code> mode only when bit rate accuracy is not acceptable in the <code>UART_CLK.EDBO = 0</code> mode. The <code>UART_CLK.EDBO = 1</code> mode is not intended to increase the speed of operation beyond the electrical limitations of the UART transfer protocol.
		0 Bit clock prescaler = 16
		1 Bit clock prescaler = 1
15:0 (R/W)	DIV	Divisor. The <code>UART_CLK.DIV</code> provides the divisor for the UART's clock bit rate calculation. The bit rate is defined by: $\text{Bit Rate} = \text{SCLK} / (16^{(1-\text{EDBo})} \times \text{UART_CLK.DIV})$

Control Register

The `UART_CTL` register provides enable and disable control for UART and IrDA mode of operation. It also provides UART line control, permitting selection of the format of received and transmitted character frames. Modem feature control also is available from this register, including partial modem functionality to allow for hardware flow control and loopback mode.

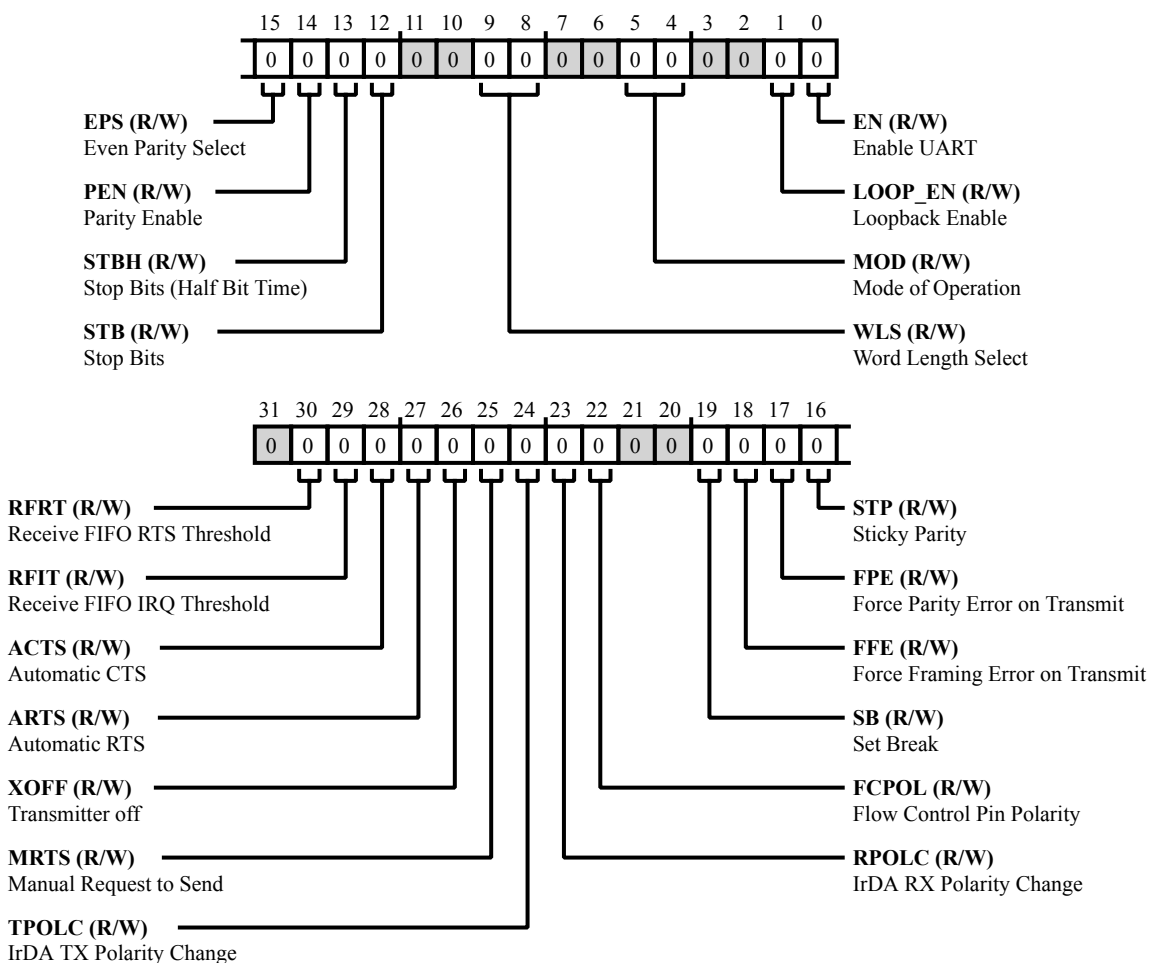


Figure 15-12: `UART_CTL` Register Diagram

Table 15-11: UART_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
30 (R/W)	RFRT	Receive FIFO RTS Threshold. The <code>UART_CTL.RFRT</code> bit controls <code>UART_RTS</code> pin assertion and deassertion timing. This bit is ignored if <code>UART_CTL.ARTS</code> is cleared. If set, the <code>UART_RTS</code> pin is deasserted when the receive buffer already holds seven words and an eighth start bit is detected. It is reasserted when the FIFO contains seven words or less. If cleared, the <code>UART_RTS</code> pin is deasserted when the RX buffer already holds four words and a fifth start bit is detected. The <code>UART_RTS</code> pin is reasserted when the RX buffer contains no more than 4 words.
		0 Deassert RTS if RX FIFO word count > 4; assert if <= 4
		1 Deassert RTS if RX FIFO word count > 7; assert if <= 7
29 (R/W)	RFIT	Receive FIFO IRQ Threshold. The <code>UART_CTL.RFIT</code> bit controls the timing of the <code>UART_STAT.RFCS</code> bit. If <code>UART_CTL.RFIT</code> is cleared, the receive threshold is two. If <code>UART_CTL.RFIT</code> is set, the threshold is four words in the receive buffer.
		0 Set RFCS=1 if RX FIFO count >= 4
		1 Set RFCS=1 if RX FIFO count >= 7
28 (R/W)	ACTS	Automatic CTS. The <code>UART_CTL.ACTS</code> bit must be set to enable the <code>UART_CTS</code> input pin for <code>UART_TX</code> handshaking. If enabled, the <code>UART_STAT.CTS</code> bit holds the value (if <code>UART_CTL.FCPOL</code> is set) or complement value (if <code>UART_CTL.FCPOL</code> is cleared) of the <code>UART_CTS</code> input pin. The <code>UART_STAT.CTS</code> bit can be used to determine whether the external device is ready to receive data (if <code>UART_STAT.CTS</code> is set) or whether it is busy (if <code>UART_STAT.CTS</code> is cleared). If <code>UART_CTL.ACTS</code> is cleared, the <code>UART_TX</code> handshaking protocol is disabled, and the <code>UART_TX</code> line transmits data whenever there is data to send, regardless of the value of <code>UART_CTS</code> . Software can pause ongoing transmission by setting the <code>UART_CTL.XOFF</code> bit.
		0 Disable TX handshaking protocol
		1 Enable TX handshaking protocol
27 (R/W)	ARTS	Automatic RTS. The <code>UART_CTL.ARTS</code> bit must be set to enable the <code>UART_RTS</code> input pin for <code>UART_TX</code> handshaking. If set, the hardware guarantees a minimal <code>UART_RTS</code> pin deassertion pulse width of at least the number of data bits defined by the <code>UART_CTL.WLS</code> bit field. If cleared, the <code>UART_RTS</code> pin is not generated automatically by hardware. The <code>UART_RTS</code> pin can still be manually controlled by the <code>UART_CTL.MRTS</code> bit, and software is responsible for <code>UART_RTS</code> pulse width control (if needed).
		0 Disable RX handshaking protocol.
		1 Enable RX handshaking protocol.

Table 15-11: UART_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
26 (R/W)	XOFF	Transmitter off. The <code>UART_CTL.XOFF</code> bit (if set) turns off transmission (XOFF) by preventing the content of <code>THR</code> from being continued to <code>TSR</code> . When set, this bit turns on transmission (XON). The state of the <code>UART_CTL.XOFF</code> bit is ignored if the <code>UART_CTL.ACTS</code> bit is set.
		0 Transmission ON, if <code>ACTS</code> =0
		1 Transmission OFF, if <code>ACTS</code> =0
25 (R/W)	MRTS	Manual Request to Send. The <code>UART_CTL.MRTS</code> bit controls the state of the <code>UART_RTS</code> output pin when the <code>UART_CTL.ARTS</code> bit is cleared. When <code>UART_CTL.MRTS</code> is cleared, the UART deasserts the <code>UART_RTS</code> pin, signaling to the external device that the UART is not ready to receive. When <code>UART_CTL.MRTS</code> is set, the UART asserts the <code>UART_RTS</code> pin, signaling to the external device that the UART is ready to receive.
		0 Deassert RTS pin when <code>ARTS</code> =0
		1 Assert RTS pin when <code>ARTS</code> =0
24 (R/W)	TPOLC	IrDA TX Polarity Change. The <code>UART_CTL.TPOLC</code> bit selects the active low or high polarity for IrDA communications. This bit is effective only in IrDA mode. If set, in IrDA mode, the <code>UART_TX</code> pin idles high. In UART or MDB mode, it is inverted-NRZ. If cleared, in IrDA mode, the <code>UART_TX</code> pin idles low. In UART or MDB mode, it is NRZ.
		0 Active-low TX polarity setting
		1 Active-high TX polarity setting
23 (R/W)	RPOLC	IrDA RX Polarity Change. The <code>UART_CTL.RPOLC</code> bit selects the active low or high polarity for IrDA communications. This bit is effective only in IrDA mode. If set, in IrDA mode, the <code>UART_RX</code> pin idles high. In UART or MDB mode, it is inverted-NRZ. If cleared, in IrDA mode, the <code>UART_RX</code> pin idles low. In UART or MDB mode, it is NRZ.
		0 Active-low RX polarity setting
		1 Active-high RX polarity setting

Table 15-11: UART_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
22 (R/W)	FCPOL	Flow Control Pin Polarity. The <code>UART_CTL.FCPOL</code> bit selects the polarities of the <code>UART_CTS</code> and <code>UART_RTS</code> pins. When the <code>UART_CTL.FCPOL</code> bit is cleared, the <code>UART_RTS</code> and <code>UART_CTS</code> pins are active low, and the UART is halted when the <code>UART_RTS</code> and <code>UART_CTS</code> pin state is high. When <code>UART_CTL.FCPOL</code> bit is set, the <code>UART_RTS</code> and <code>UART_CTS</code> pins are active high, and the UART is halted when the <code>UART_RTS</code> and <code>UART_CTS</code> pin state is low.
		0 Active low CTS/RTS
		1 Active high CTS/RTS
19 (R/W)	SB	Set Break. If set, the <code>UART_CTL.SB</code> bit forces the <code>UART_TX</code> pin to low asynchronously, regardless of whether or not data is currently transmitted. This bit functions even when the UART clock is disabled. Because the <code>UART_TX</code> pin normally drives high, it can be used as a flag output pin, if the UART is not used. (For example, if <code>UART_CTL.TPOLC</code> is cleared, drive <code>UART_TX</code> pin low; or if <code>UART_CTL.TPOLC</code> is set, drive <code>UART_TX</code> pin high.)
		0 No force
		1 Force TX pin to 0
18 (R/W)	FFE	Force Framing Error on Transmit. The <code>UART_CTL.FFE</code> bit is intended for test purposes. This bit is useful for debugging software, especially in loopback mode.
		0 Normal operation
		1 Force error
17 (R/W)	FPE	Force Parity Error on Transmit. The <code>UART_CTL.FPE</code> bit is intended for test purposes. This bit is useful for debugging software, especially in loopback mode.
		0 Normal operation
		1 Force parity error

Table 15-11: UART_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
16 (R/W)	STP	Sticky Parity. The <code>UART_CTL.STP</code> bit controls whether the parity is generated by hardware based on the data bits or whether it is set to a fixed value. If this bit is cleared, the hardware calculates the parity bit value based on the data bits. Then, the <code>EPS</code> bit determines whether odd or even parity mode is chosen. If this bit is set, odd parity is used. That means that the total count of logical-1 data bits including the parity bit must be an odd value. Even parity is chosen by <code>UART_CTL.STP</code> cleared and <code>UART_CTL.EPS</code> set. Then, the count of logical-1 bits must be an even value. If the <code>UART_CTL.STP</code> bit is set, hardware parity calculation is disabled. In this case, the sent and received parity equals the inverted <code>UART_CTL.EPS</code> bit.
		0 No forced parity
		1 Force (Stick) parity to defined value (if <code>PEN=1</code>)
15 (R/W)	EPS	Even Parity Select.
		0 Odd parity
		1 Even parity
14 (R/W)	PEN	Parity Enable. The <code>UART_CTL.PEN</code> bit enables parity transmission and parity check. The <code>UART_CTL.PEN</code> bit inserts one additional bit between the most significant data bit and the first stop bit. The polarity of this so-called parity bit depends on data and the <code>UART_CTL.STP</code> and <code>UART_CTL.EPS</code> control bits. Both transmitter and receiver calculate the parity value. The receiver compares the received parity bit with the expected value and issues a parity error if they do not match. If the <code>UART_CTL.PEN</code> bit is cleared, the <code>UART_CTL.STP</code> and the <code>UART_CTL.EPS</code> bits are ignored.
		0 Disable
		1 Enable parity transmit and check
13 (R/W)	STBH	Stop Bits (Half Bit Time).
		0 0 half-bit-time stop bit
		1 1 half-bit-time stop bit
12 (R/W)	STB	Stop Bits. The <code>UART_CTL.STB</code> bit controls how many stop bits are appended to transmitted data.
		0 1 stop bit
		1 2 stop bits

Table 15-11: UART_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
9:8 (R/W)	WLS	Word Length Select. The <code>UART_CTL.WLS</code> field determines whether the transmitted and received UART word consists of 5, 6, 7, or 8 data bits.
		0 5-bit word
		1 6-bit word
		2 7-bit word
		3 8-bit word
5:4 (R/W)	MOD	Mode of Operation. The <code>UART_CTL.MOD</code> selects the UART operation mode (UMOD).
		0 UART mode
		1 MDB mode
		2 IrDA SIR mode
1 (R/W)	LOOP_EN	Loopback Enable. The <code>UART_CTL.LOOP_EN</code> bit enables UART loopback mode. When set, this bit disconnects the input of the receiver from the <code>UART_RX</code> pin, and internally redirects the transmit output to the receiver. The <code>UART_TX</code> pin remains active and continues to transmit data externally as well. Loopback mode also forces the <code>UART_RTS</code> pin to its deassertive state, disconnects the <code>UART_CTS</code> bit from the <code>UART_CTS</code> input pin, and directly connects the <code>UART_CTL.MRTS</code> bit to the <code>UART_STAT.CTS</code> bit. In loopback mode, setting the <code>UART_CTL.MRTS</code> bit sets the <code>UART_STAT.CTS</code> bit and enables the transmitter of the UART. Clearing the <code>UART_CTL.MRTS</code> bit clears the <code>UART_STAT.CTS</code> bit and disables the transmitter of the UART.
		0 Disable
		1 Enable
0 (R/W)	EN	Enable UART. The <code>UART_CTL.EN</code> enables the UART clocks. This bit also resets the state machine and control registers when cleared. Using this bit to disable the UART -- when not used -- reduces power consumption.
		0 Disable
		1 Enable

Interrupt Mask Register

The `UART_IMSK` register indicates the interrupt mask status (unmasked, if set, or masked, if cleared) of the UART status interrupt requests. This register is not a data register. Instead, it is controlled by the `UART_IMSK_SET` and `UART_IMSK_CLR` register pair. Writing ones to the `UART_IMSK_SET` register enables (unmasks) interrupt requests, and writing ones to the `UART_IMSK_CLR` register disables (masks) them. Reads from either register return the enabled bits.

The `UART_IMSK` register is used to enable requests for system handling of empty or full states of UART data registers. Unless polling is used as a means of action, the `UART_IMSK.ERBFI` and `UART_IMSK.ETBEI` bits are normally set. Setting this register without enabling system DMA causes the UART to notify the processor of the data inventory state using interrupts. For proper operation in this mode, system interrupts must be enabled, and appropriate interrupt handling routines must be present.

Each UART features three separate interrupt channels to handle the data transmit, data receive, and line status events independently, regardless of whether DMA is enabled or not. If no DMA channels are assigned to the UART, set the `UART_IMSK.ELSI` bit to reroute the transmit and receive interrupts to the status interrupt request output.

With system DMA enabled, the UART uses DMA to transfer data to or from the processor. Dedicated DMA channels are available for receive and transmit operations. Line error handling can be configured independently from the receive or transmit setup.

The DMA of the UART is enabled by first setting up the system DMA control registers and then enabling the `UART_IMSK.ERBFI` and `UART_IMSK.ETBEI` interrupts. This configuration is because the interrupt request lines double as DMA request lines. Depending on whether DMA is enabled or not, upon receiving these requests, the DMA control unit either generates a direct memory access or passes the UART interrupt on to the system interrupt handling unit(s). However, the error interrupt for the UART goes directly to the system interrupt handling unit(s), bypassing the DMA unit completely.

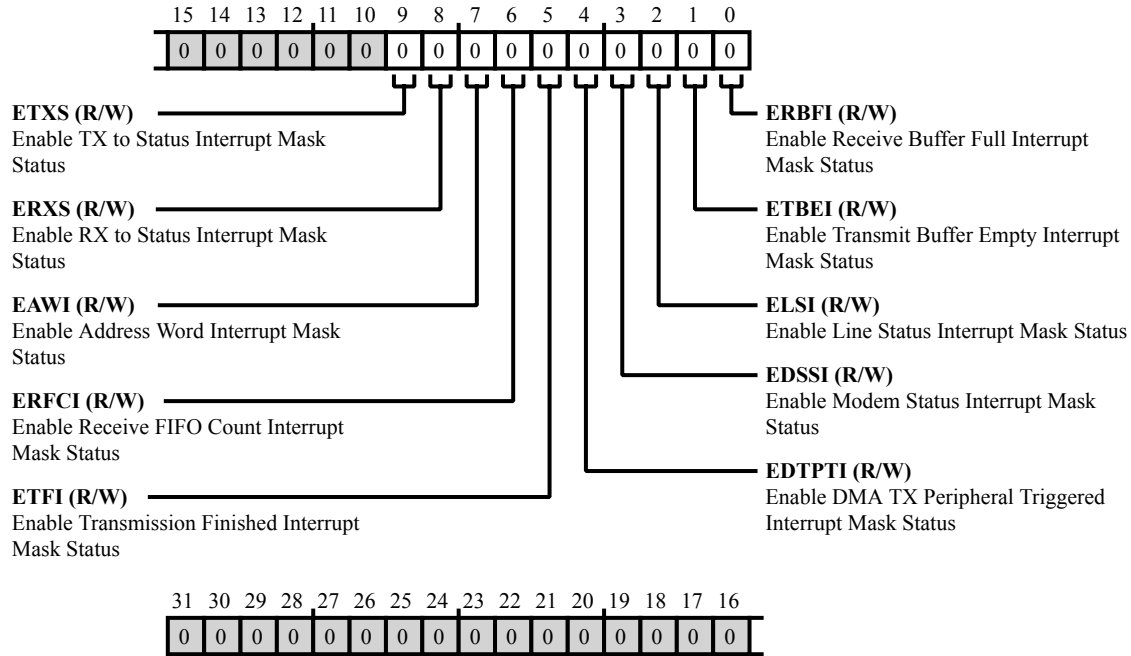


Figure 15-13: UART_IMSK Register Diagram

Table 15-12: UART_IMSK Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
9 (R/W)	ETXS	Enable TX to Status Interrupt Mask Status. If set (interrupt unmasked), the <code>UART_IMSK.ETXS</code> bit indicates re-direction of the TX interrupt requests to status interrupt output. If cleared, TX interrupt requests are routed to normal interrupt outputs.
		0 Interrupt is masked
		1 Interrupt is unmasked
8 (R/W)	ERXS	Enable RX to Status Interrupt Mask Status. If set (interrupt unmasked), the <code>UART_IMSK.ERXS</code> bit indicates re-direction of RX interrupt requests to status interrupt output. If cleared, RX interrupt requests are routed to normal interrupt outputs.
		0 Interrupt is masked
		1 Interrupt is unmasked

Table 15-12: UART_IMSK Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
7 (R/W)	EAWI	Enable Address Word Interrupt Mask Status. If set (interrupt unmasked), the <code>UART_IMSK.EAWI</code> bit indicates generation of a status interrupt request when an Address word in MDB-mode is present in the <code>UART_RBR</code> . A received word is an address word if the <code>UART_STAT.ADDR</code> bit is set.
		0 Interrupt is masked
		1 Interrupt is unmasked
6 (R/W)	ERFCI	Enable Receive FIFO Count Interrupt Mask Status. If set (interrupt unmasked), the <code>UART_IMSK.ERFCI</code> bit indicates enabling of the receive buffer threshold interrupt request if signaled by the <code>UART_STAT.RFCS</code> bit. Read the <code>UART_RBR</code> register sufficient times to clear the interrupt request.
		0 Interrupt is masked
		1 Interrupt is unmasked
5 (R/W)	ETFI	Enable Transmission Finished Interrupt Mask Status. If set (interrupt unmasked) the <code>UART_IMSK.ETFI</code> bit indicates enabling of interrupt generation on the status interrupt channel when the transmit buffer register, the transmit address register, and the transmit shift register are all empty as indicated by the <code>UART_STAT.TFI</code> . The <code>UART_IMSK.ETFI</code> interrupt can be used to avoid expensive polling of the <code>UART_STAT.TEMT</code> bit, when the UART clock or line drivers should be disabled after transmission has completed. With the <code>UART_STAT.TFI</code> bit to clear the interrupt request. In DMA operation, the <code>UART_IMSK.ETFI</code> bits functionality might be preferred.
		0 Interrupt is masked
		1 Interrupt is unmasked
4 (R/W)	EDTPTI	Enable DMA TX Peripheral Triggered Interrupt Mask Status. If set (interrupt unmasked), the <code>UART_IMSK.EDTPTI</code> bit indicates enabling of the DMA completion interrupt request to be delayed until the data has left the UART completely. This bit is required for DMA transmit operation only. If set, the UART can generate a DMA interrupt request by the time the <code>UART_STAT.TEMT</code> bit goes high after the last DMA data word is transmitted. When <code>UART_IMSK.EDTPTI</code> is set, usually the <code>DMA_CFG.INT</code> field is cleared to 00 in a STOP mode DMA. This set up suppresses the normal completion interrupt request, and the <code>UART_STAT.TEMT</code> event is signaled through the DMA controller and triggers the DMA interrupt. If both (<code>DMA_CFG.INT</code> not 00 and <code>UART_IMSK.EDTPTI</code> set), two interrupts are requested at the end of a STOP mode DMA.
		0 Interrupt is masked
		1 Interrupt is unmasked

Table 15-12: UART_IMSK Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
3 (R/W)	EDSSI	Enable Modem Status Interrupt Mask Status. If set (interrupt unmasked), the <code>UART_IMSK.EDSSI</code> bit indicates enabling of a modem status interrupt request on the same status interrupt channel when the <code>UART_STAT.SCTS</code> bit is set. This indicates <code>UART_CTS</code> pin re-assertion. Write-1-to-clear (W1C) the <code>UART_STAT.SCTS</code> bit to clear the interrupt request.
		0 Interrupt is masked
		1 Interrupt is unmasked
2 (R/W)	ELSI	Enable Line Status Interrupt Mask Status. If set (interrupt unmasked), the <code>UART_IMSK.ELSI</code> bit indicates that redirection of TX and RX interrupt requests to the status interrupt output of the UART by OR'ing them with the <code>UART_STAT.OE</code> , <code>UART_STAT.PE</code> , <code>UART_STAT.FE</code> , and <code>UART_STAT.BI</code> interrupt requests. Set this bit when no DMA channel is associated with the UART. Enabling <code>UART_IMSK.ELSI</code> disables the RX/TX interrupt channels and negates the <code>UART_IMSK.EDTPTI</code> bit.
		0 Interrupt is masked
		1 Interrupt is unmasked
1 (R/W)	ETBEI	Enable Transmit Buffer Empty Interrupt Mask Status. If set (interrupt unmasked), the <code>UART_IMSK.ETBEI</code> bit indicates generation of a TX interrupt request if the <code>UART_STAT.THRE</code> bit is set.
		0 Interrupt is masked
		1 Interrupt is unmasked
0 (R/W)	ERBFI	Enable Receive Buffer Full Interrupt Mask Status. If set (interrupt unmasked), the <code>UART_IMSK.ERBFI</code> indicates generation of an RX interrupt request if the <code>UART_STAT.DR</code> bit is set.
		0 Interrupt is masked
		1 Interrupt is unmasked

Interrupt Mask Clear Register

The `UART_IMSK` indicates interrupt mask status (unmasked if set, masked if cleared) of UART status interrupts. This register is not a data register. Instead it is controlled by the `UART_IMSK_SET` and `UART_IMSK_CLR` register pair. Writing ones to `UART_IMSK_SET` enables (unmasks) interrupt requests, and writing ones to `UART_IMSK_CLR` disables (masks) them. Reads from either register return the enabled bits. For more information, see the `UART_IMSK` register description.

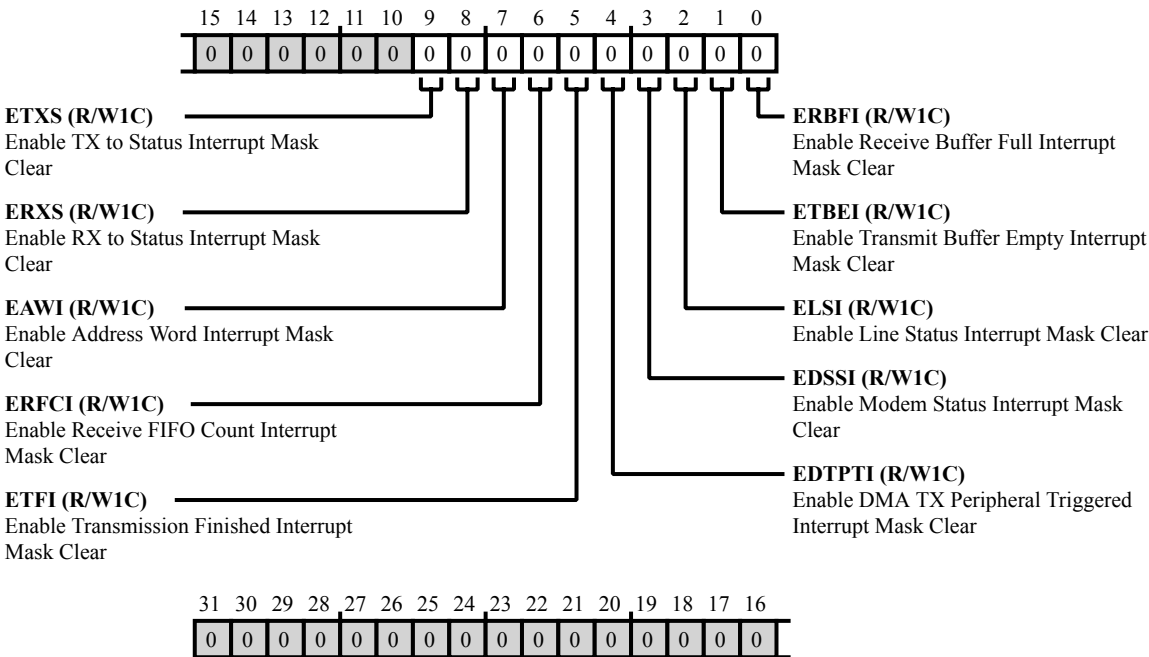


Figure 15-14: UART_IMSK_CLR Register Diagram

Table 15-13: UART_IMSK_CLR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
9 (R/W1C)	ETXS	Enable TX to Status Interrupt Mask Clear.
		0 No action
		1 Mask interrupt
8 (R/W1C)	ERXS	Enable RX to Status Interrupt Mask Clear.
		0 No action
		1 Mask interrupt
7 (R/W1C)	EAWI	Enable Address Word Interrupt Mask Clear.
		0 No action
		1 Mask interrupt

Table 15-13: UART_IMSK_CLR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
6 (R/W1C)	ERFCI	Enable Receive FIFO Count Interrupt Mask Clear.	
		0	No action
		1	Mask interrupt
5 (R/W1C)	ETFI	Enable Transmission Finished Interrupt Mask Clear.	
		0	No action
		1	Mask interrupt
4 (R/W1C)	EDTPTI	Enable DMA TX Peripheral Triggered Interrupt Mask Clear.	
		0	No action
		1	Mask interrupt
3 (R/W1C)	EDSSI	Enable Modem Status Interrupt Mask Clear.	
		0	No action
		1	Mask interrupt
2 (R/W1C)	ELSI	Enable Line Status Interrupt Mask Clear.	
		0	No action
		1	Mask interrupt
1 (R/W1C)	ETBEI	Enable Transmit Buffer Empty Interrupt Mask Clear.	
		0	No action
		1	Mask interrupt
0 (R/W1C)	ERBFI	Enable Receive Buffer Full Interrupt Mask Clear.	
		0	No action
		1	Mask interrupt

Interrupt Mask Set Register

The `UART_IMSK` indicates interrupt request mask status (unmasked if set, masked if cleared) of UART status interrupts. This register is not a data register. Instead it is controlled by the `UART_IMSK_SET` and `UART_IMSK_CLR` register pair. Writing ones to `UART_IMSK_SET` enables (unmasks) interrupt requests, and writing ones to `UART_IMSK_CLR` disables (masks) them. Reads from either register return the enabled bits. For more information, see the `UART_IMSK` register description.

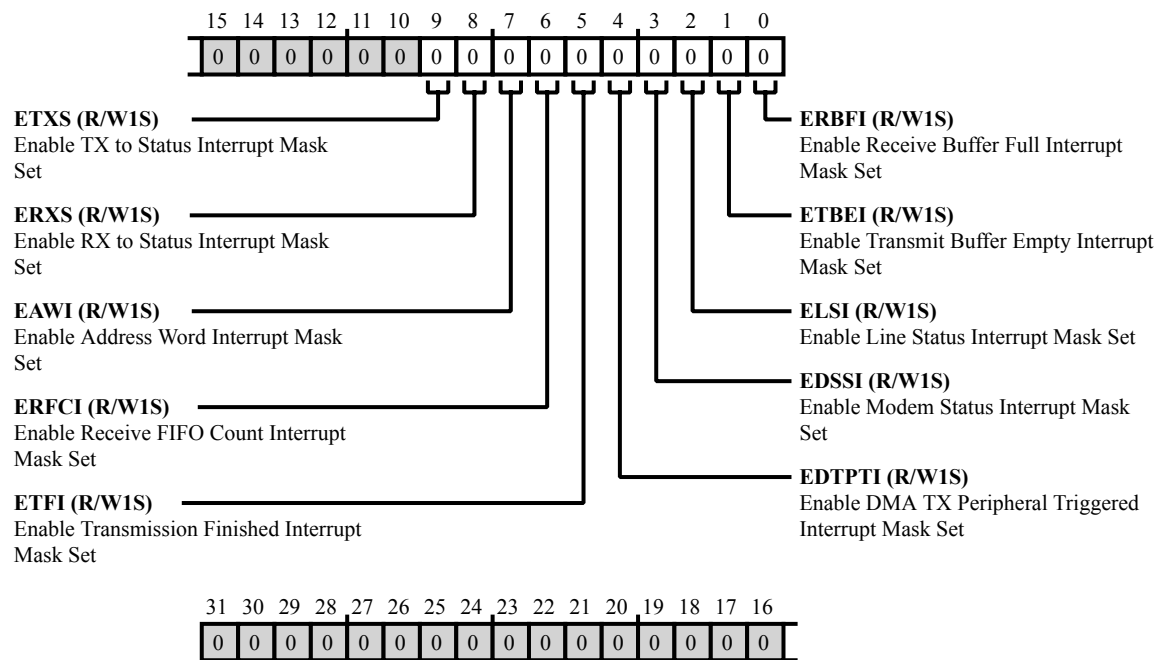


Figure 15-15: UART_IMSK_SET Register Diagram

Table 15-14: UART_IMSK_SET Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
9 (R/W1S)	ETXS	Enable TX to Status Interrupt Mask Set.
		0 No action
		1 Unmask interrupt
8 (R/W1S)	ERXS	Enable RX to Status Interrupt Mask Set.
		0 No action
		1 Unmask interrupt
7 (R/W1S)	EAWI	Enable Address Word Interrupt Mask Set.
		0 No action
		1 Unmask interrupt

Table 15-14: UART_IMSK_SET Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
6 (R/W1S)	ERFCI	Enable Receive FIFO Count Interrupt Mask Set.	
		0	No action
		1	Unmask interrupt
5 (R/W1S)	ETFI	Enable Transmission Finished Interrupt Mask Set.	
		0	No action
		1	Unmask interrupt
4 (R/W1S)	EDTPTI	Enable DMA TX Peripheral Triggered Interrupt Mask Set.	
		0	No action
		1	Unmask interrupt
3 (R/W1S)	EDSSI	Enable Modem Status Interrupt Mask Set.	
		0	No action
		1	Unmask interrupt
2 (R/W1S)	ELSI	Enable Line Status Interrupt Mask Set.	
		0	No action
		1	Unmask interrupt
1 (R/W1S)	ETBEI	Enable Transmit Buffer Empty Interrupt Mask Set.	
		0	No action
		1	Unmask interrupt
0 (R/W1S)	ERBFI	Enable Receive Buffer Full Interrupt Mask Set.	
		0	No action
		1	Unmask interrupt

Receive Buffer Register

The read-only `UART_RBR` register is the UART’s receive buffer. It is updated when there is pending data in the receive FIFO. Newly available data is signaled by the `UART_STAT.DR` bit.

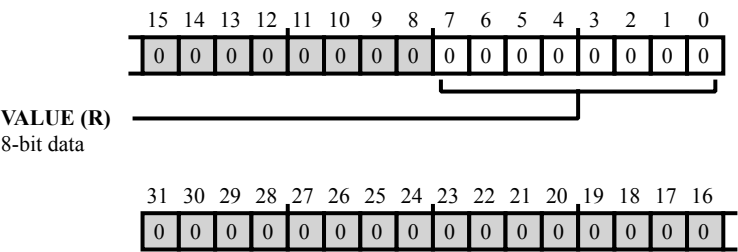


Figure 15-16: UART_RBR Register Diagram

Table 15-15: UART_RBR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	VALUE	8-bit data.

Receive Shift Register

The read only `UART_RSR` register which returns the content of the UART’s receive shift register.

The frame data is moved into this shift register after polarity inversion, if any (including the native polarity inversion in the IrDA case).

In the case of the longest frame (MDB, with parity mode, and 8 bit data word-length), the start bit may be shifted out and not available for reading at the end of the frame reception. This register is NOT reset at the start of frame. If read, in the middle of a frame reception, data corresponding the previous frame may not have entirely shifted out (for example, the read data that have been read may NOT correspond entirely to the frame being received).

Because the UART is receiving only 1 stop bit, the `UART_RSR` contains only 1 stop bit even if more than one stop bit is present in the actual transfer. This register may be considered as storing the 10 most recently received bits (taking into consideration the stop bit receive limitation above).

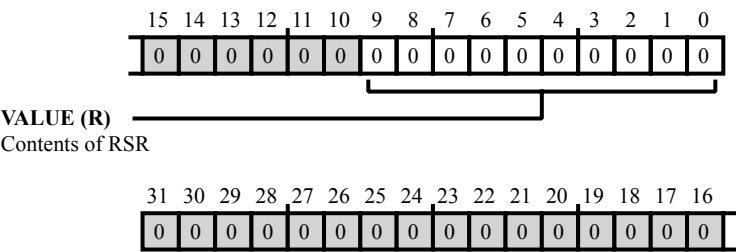


Figure 15-17: UART_RSR Register Diagram

Table 15-16: UART_RSR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
9:0 (R/NW)	VALUE	Contents of RSR.

Receive Counter Register

The `UART_RXCNT` register returns the content of 16-bit counter in the UART receiver. This count is used for baud rate clock generation (the lower [15:0] is the count data).

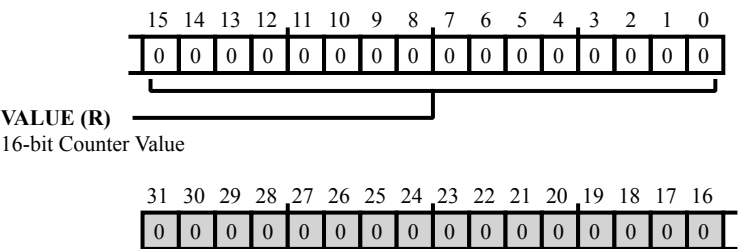


Figure 15-18: UART_RXCNT Register Diagram

Table 15-17: UART_RXCNT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/NW)	VALUE	16-bit Counter Value.

Scratch Register

The `UART_SCR` registers contain 8-bit scratch pad data. These registers are used for general purpose data storage and do not control the UART hardware in any way.

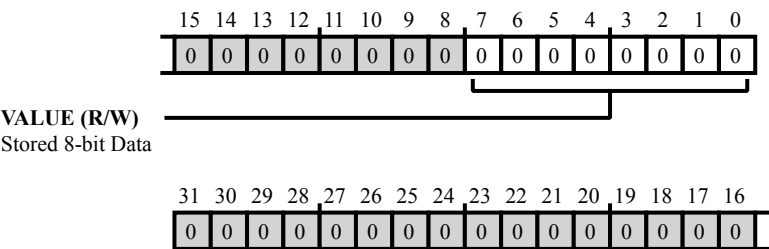


Figure 15-19: UART_SCR Register Diagram

Table 15-18: UART_SCR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/W)	VALUE	Stored 8-bit Data.

Status Register

The `UART_STAT` register contains the UART line status and UART modem status, as indicated by the current states of the UART's `UART_CTS` pin and internal receive buffers. Writes to this register can perform write-one-to-clear (W1C) operations on most status bits. Reading this register has no side effects.

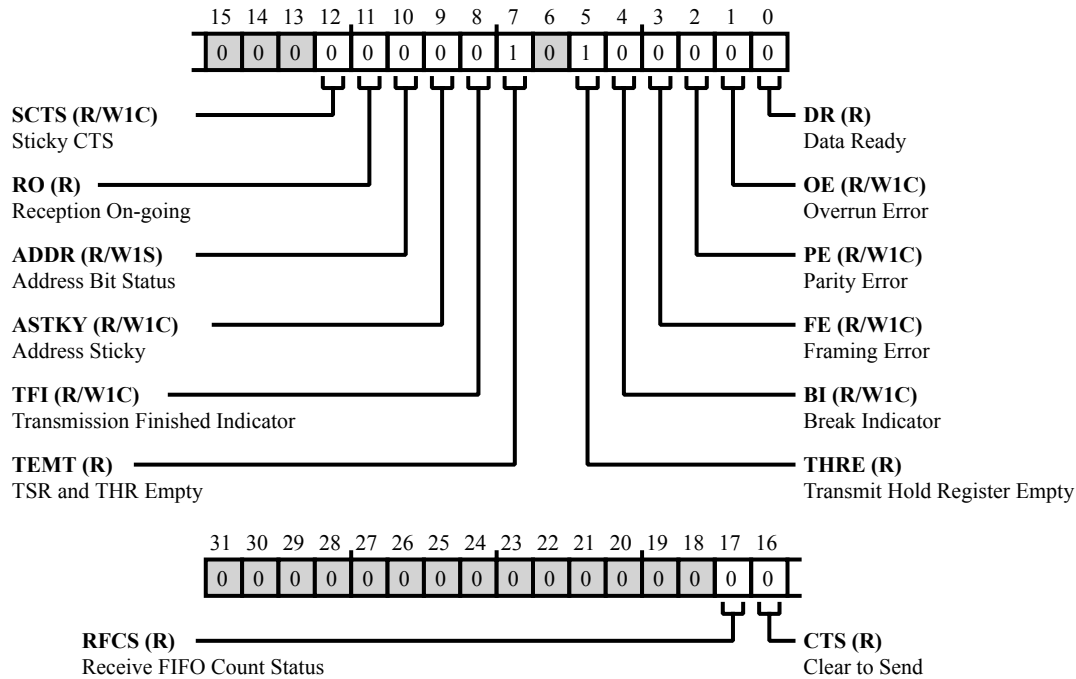


Figure 15-20: UART_STAT Register Diagram

Table 15-19: UART_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
17 (R/NW)	RFCS	Receive FIFO Count Status.
		The <code>UART_STAT.RFCS</code> bit is set when the receive buffer holds more or equal entries than a certain threshold. The threshold is controlled by the <code>UART_CTL.RFIT</code> bit. If <code>UART_CTL.RFIT</code> is cleared, the threshold is four entries. If <code>UART_CTL.RFIT</code> is set, the threshold is seven entries. The <code>UART_STAT.RFCS</code> bit is cleared when the <code>UART_RBR</code> register is read sufficient times until the buffer is drained below the threshold. The <code>UART_STAT.RFCS</code> bit can trigger a status interrupt request if enabled by the <code>UART_IMSK_SET.ERFCI</code> bit.
		0 RX FIFO has less than 4 (7) entries when <code>RFIT=0</code> (1)
		1 RX FIFO has at least 4 (7) entries when <code>RFIT=0</code> (1)

Table 15-19: UART_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
16 (R/NW)	CTS	Clear to Send. The <code>UART_STAT.CTS</code> bit holds the value (if <code>UART_CTL.FCPOL</code> set) or the complement value (if <code>UART_CTL.FCPOL</code> cleared) of the <code>UART_CTS</code> input pin. The <code>UART_CTL.ACTS</code> bit must be set to enable this feature. The core can read the value of the <code>UART_STAT.CTS</code> bit to determine whether the external device is ready to receive (<code>UART_STAT.CTS</code> set) or if it is busy (<code>UART_STAT.CTS</code> cleared). If <code>UART_CTL.ACTS</code> is cleared, the <code>UART_TX</code> handshaking protocol is disabled, and the UART transmits data as long as there is data to transmit, regardless of the value of <code>UART_STAT.CTS</code> . When <code>UART_CTL.ACTS</code> is cleared, the software can pause transmission temporarily by setting the <code>XOFF</code> bit. Note that in loopback mode (<code>UART_CTL.LOOP_EN</code> set), the <code>UART_STAT.CTS</code> bit is disconnected from the <code>UART_CTS</code> input pin. Instead, the bit is directly connected to the <code>UART_CTL.MRTS</code> bit.
		0 Not clear to send (External device not ready to receive)
		1 Clear to send (External device ready to receive)
12 (R/W1C)	SCTS	Sticky CTS. The <code>UART_STAT.SCTS</code> bit is a sticky bit that is set when <code>UART_STAT.CTS</code> transitions from 0 to 1. The <code>UART_STAT.SCTS</code> bit is cleared by software with a W1C operation. This bit can trigger a line status interrupt request if enabled by the <code>UART_IMSK_SET.EDSSI</code> bit.
		0 CTS has not transitioned from low to high
		1 CTS has transitioned from low to high
11 (R/NW)	RO	Reception On-going.
		0 No data reception in progress
		1 Data reception in progress
10 (R/W1S)	ADDR	Address Bit Status. The <code>UART_STAT.ADDR</code> bit is used to mirror the address bit of the word in <code>UART_RBR</code> in multi-drop bus protocol, and is enabled only in MDB mode. The <code>UART_STAT.ADDR</code> bit is updated by hardware upon detecting a received word with the address bit in <code>UART_RBR</code> set or cleared. Additionally, software can set the <code>ADDR</code> bit with a write-1-to-set (W1S) operation.
		0 Address bit is low
		1 Address bit is high

Table 15-19: UART_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
9 (R/W1C)	ASTKY	Address Sticky. The <code>UART_STAT.ASTKY</code> bit is used in multi-drop bus mode to indicate whether a peripheral is currently being addressed. This bit is a sticky version of the <code>UART_STAT.ADDR</code> bit and is set by hardware when setting the <code>UART_STAT.ADDR</code> bit. The <code>UART_STAT.ASTKY</code> bit can only be cleared by software with a write-one-to-clear (W1C) operation. With the <code>UART_STAT.ASTKY</code> bit set, words will be received irrespective of the <code>UART_CTL.MOD</code> bit or <code>UART_STAT.ADDR</code> bit selection. With the <code>UART_STAT.ASTKY</code> bit cleared, only address words (<code>UART_CTL.MOD</code> bit set) will be received and words with <code>UART_CTL.MOD</code> bit cleared are ignored (not moved from the RSR to the RX FIFO) in MDB mode. The <code>UART_STAT.ASTKY</code> bit does not affect reception in non-MDB modes.
		0 ADDR bit has not been set
		1 ADDR bit has been set
8 (R/W1C)	TFI	Transmission Finished Indicator. The <code>UART_STAT.TFI</code> bit is a sticky version of the <code>UART_STAT.TEMT</code> bit. While <code>UART_STAT.TEMT</code> is automatically cleared by hardware when new data is written to the <code>UART_THR</code> register, the sticky <code>UART_STAT.TFI</code> bit remains set, until it is cleared by software (W1C). The <code>UART_STAT.TFI</code> bit enables more flexible transmit interrupt request timing.
		0 TEMT did not transition from 0 to 1
		1 TEMT transition from 0 to 1
7 (R/NW)	TEMT	TSR and THR Empty. The <code>UART_STAT.TEMT</code> bit indicates that the <code>UART_THR</code> and <code>UART_TAIP</code> registers and the <code>UART_TSR</code> register are empty. In this case, the program is permitted to write to the <code>UART_THR</code> and <code>UART_TAIP</code> registers twice without losing data. The <code>UART_STAT.TEMT</code> bit can also be used as indicator that pending UART transmission is completed. At that time, it is safe to disable the <code>UART_CTL.EN</code> bit or to three-state the off-chip line driver.
		0 Not empty TSR/THR
		1 TSR/THR Empty

Table 15-19: UART_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
5 (R/NW)	THRE	Transmit Hold Register Empty. The <code>UART_STAT.THRE</code> bit indicates that the UART transmit channel is ready for new data and software can write to the <code>UART_THR</code> and <code>UART_TAIP</code> registers. Writes to the <code>UART_THR</code> and <code>UART_TAIP</code> registers clear the <code>UART_STAT.THRE</code> . The bit is set again when the <code>UART_THR</code> and <code>UART_TAIP</code> registers are empty and ready to accept data.
		0 Not empty THR/TAIP
		1 Empty THR/TAIP
4 (R/W1C)	BI	Break Indicator. The <code>UART_STAT.BI</code> bit indicates that the first stop bit is sampled low and the entire <u>data word</u> , including parity bit, consists of low bits only. (This condition indicates that <code>UART_RX</code> was held low for more than the maximum word length.) The <code>UART_STAT.BI</code> bit is updated simultaneously with the <code>UART_STAT.DR</code> bit, that is, by the time the first stop bit is received or when data is loaded from the receive FIFO to the <code>UART_RBR</code> register. The bit is sticky and can be cleared by W1C operations.
		0 No break interrupt
		1 Break interrupt this indicates UARTxRX was held low(RPOLC=0) / high (RPOLC=1) for more than the maximum word length
3 (R/W1C)	FE	Framing Error. The <code>UART_STAT.FE</code> bit indicates that the first stop bit is sampled. This bit is updated simultaneously with the <code>UART_STAT.DR</code> bit, that is, by the time the first stop bit is received or when data is loaded from the receive FIFO to the <code>UART_RBR</code> register. The <code>UART_STAT.FE</code> bit is sticky and can be cleared by W1C operations. Note that invalid stop bits can be simulated by setting the <code>UART_CTL.FFE</code> bit.
		0 No error
		1 Invalid stop bit error
2 (R/W1C)	PE	Parity Error. The <code>UART_STAT.PE</code> bit indicates that the received parity bit does not match the expected value. This bit is updated simultaneously with the <code>UART_STAT.DR</code> bit, that is, by the time the first stop bit is received or when data is loaded from the receive FIFO to the <code>UART_RBR</code> register. The <code>UART_STAT.PE</code> bit is sticky and can be cleared by W1C operations. Note that invalid parity bits can be simulated by setting the <code>UART_CTL.FPE</code> bit.
		0 No parity error
		1 Parity error

Table 15-19: UART_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/W1C)	OE	Overrun Error. The <code>UART_STAT.OE</code> bit indicates that further data is received while the internal receive buffer was full. This bit is set when sampling the stop bit of the sixth data word. To avoid overruns, read the <code>UART_RBR</code> register in time. In DMA receive mode, overruns are very unlikely to happen ever. After an overrun occurs, the <code>UART_RBR</code> and receive FIFO are protected from being overwritten by new data until the <code>UART_STAT.OE</code> bit is cleared by software. The content of the <code>UART_RSR</code> register is lost as soon as the overrun occurs. The <code>UART_STAT.OE</code> bit is sticky and can be cleared by W1C operations.
		0 No overrun
		1 Overrun error
0 (R/NW)	DR	Data Ready. The <code>UART_STAT.DR</code> bit indicates that data is available in the receiver and can be read from the <code>UART_RBR</code> register. The bit is set by hardware when the receiver detects the first valid stop bit. The bit is cleared by hardware when the <code>UART_RBR</code> register is read.
		0 No new data
		1 New data in RBR

Transmit Address/Insert Pulse Register

The `UART_TAIP` register and the `UART_THR` register share the same physical register, but `UART_TAIP` has different effect than the `UART_THR` register when `UART_TAIP` is written to in MDB and UART modes.

In MDB mode, data written to the `UART_TAIP` register is transmitted as an address frame (as with the `UART_CTL.MOD` bit set).

In UART mode, a write to `UART_TAIP` causes a pulse of value `UART_TAIP` [7] for a duration of `UART_TAIP` [6:0] x bit time. (There is additional inversion if the `UART_CTL.TPOLC` bit is set).

Bit time is defined by the `UART_CLK` register. The transmission of the pulse is followed by stop bit transmission as specified by the `UART_CTL.STB` and `UART_CTL.STBH` bits. This could be used for supporting line break command and inter-frame gap.

In IrDA mode, writes to `UART_TAIP` is treated the same as writes to `UART_THR`.

Accesses to the `UART_TAIP` register have the same affects as the `UART_THR` register with respect to the `UART_STAT.THRE`, `UART_STAT.TEMT`, and `UART_STAT.TFI` flags.

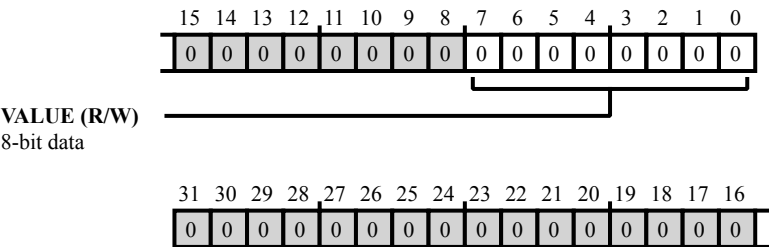


Figure 15-21: UART_TAIP Register Diagram

Table 15-20: UART_TAIP Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/W)	VALUE	8-bit data.

Transmit Hold Register

The write-only `UART_THR` register is the UART’s transmit buffer. The `UART_STAT.THRE` bit indicates whether data can be written to `UART_THR`. Writes to this register automatically propagate to the internal `UART_TSR` register as soon as `UART_TSR` is ready. Then, transmit operation is initiated immediately.

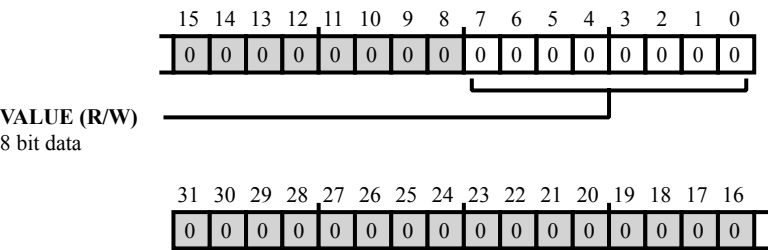


Figure 15-22: UART_THR Register Diagram

Table 15-21: UART_THR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/W)	VALUE	8 bit data.

Transmit Shift Register

The read only `UART_TSR` register which returns the content of the UART’s transmit shift register.

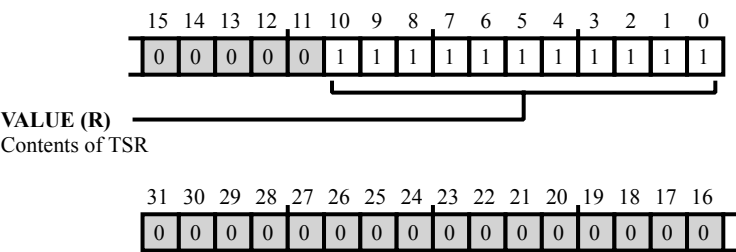


Figure 15-23: UART_TSR Register Diagram

Table 15-22: UART_TSR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
10:0 (R/NW)	VALUE	Contents of TSR.

Transmit Counter Register

The `UART_TXCNT` read only register returns the content of 16-bit counter in the UART transmitter. This count is used for baud rate clock generation (the lower [15:0] is the count data).

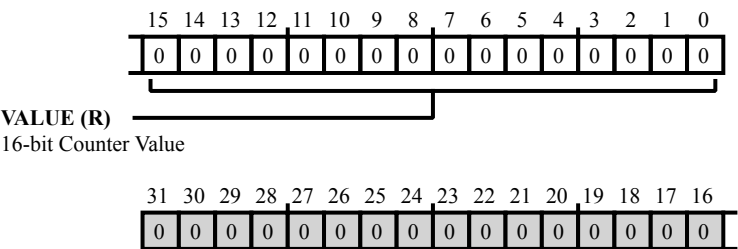


Figure 15-24: UART_TXCNT Register Diagram

Table 15-23: UART_TXCNT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/NW)	VALUE	16-bit Counter Value.

16 General-Purpose Timer (TIMER)

The general-purpose timer (GP Timer) module serves as a collection of system timers that support various system-level functions. These functions include:

- Synchronized PWM waveform output capability
- External signal capture
- External event count
- General time-base functionality

Additionally, interrupt requests can be generated upon completion of timer events. Moreover, GP timers can act both as trigger requesters and trigger completers.

GP Timer Features

Each timer can be individually configured in any of these modes:

- Pin interrupt capture mode
- Windowed watchdog mode
- Pulse-width count and capture (WDTH_CAP) mode
- External Event (EXT_CLK) mode
- Pulse-width modulation (PWM_OUT) mode

Other features include:

- Synchronous operation
- Consistent management of period and pulse width values
- Autobaud detection for UART module (where available)
- Graceful bit pattern termination when stopping
- Support for center-aligned PWM patterns
- Error detection on implausible pattern values

- All read and write accesses to 32-bit registers are atomic
- Every timer has its dedicated interrupt request output

ADSP-21560_21561_21564_21568 TIMER Register List

The General-Purpose Timer block (TIMER) provides timers that may be used for external event capture and measurement, system timing, and PWM waveform generation. A set of registers governs TIMER operations. For more information on TIMER functionality, see the TIMER register descriptions.

Table 16-1: ADSP-21560_21561_21564_21568 TIMER Register List

Name	Description
TIMER_BCAST_DLY	Broadcast Delay Register
TIMER_BCAST_PER	Broadcast Period Register
TIMER_BCAST_WID	Broadcast Width Register
TIMER_DATA_ILAT	Data Interrupt Latch Register
TIMER_DATA_IMSK	Data Interrupt Mask Register
TIMER_ERR_TYPE	Error Type Status Register
TIMER_RUN	Run Register
TIMER_RUN_CLR	Run Clear Register
TIMER_RUN_SET	Run Set Register
TIMER_STAT_ILAT	Status Interrupt Latch Register
TIMER_STAT_IMSK	Status Interrupt Mask Register
TIMER_STOP_CFG	Stop Configuration Register
TIMER_STOP_CFG_CLR	Stop Configuration Clear Register
TIMER_STOP_CFG_SET	Stop Configuration Set Register
TIMER_TMR[n]_CFG	Timer n Configuration Register
TIMER_TMR[n]_CNT	Timer n Counter Register
TIMER_TMR[n]_DLY	Timer n Delay Register
TIMER_TMR[n]_PER	Timer n Period Register
TIMER_TMR[n]_WID	Timer n Width Register
TIMER_TRG_IE	Trigger Receiver Enable Register
TIMER_TRG_MSK	Trigger Generator Mask Register

ADSP-21560_21561_21564_21568 TIMER Interrupt List

Table 16-2: ADSP-21560_21561_21564_21568 TIMER Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
22	TIMER0_TMR00	TIMER0 Timer 0	Level	
23	TIMER0_TMR01	TIMER0 Timer 1	Level	
24	TIMER0_TMR02	TIMER0 Timer 2	Level	
25	TIMER0_TMR03	TIMER0 Timer 3	Level	
74	TIMER0_TMR04	TIMER0 Timer 4	Level	
75	TIMER0_TMR05	TIMER0 Timer 5	Level	
76	TIMER0_TMR06	TIMER0 Timer 6	Level	
77	TIMER0_TMR07	TIMER0 Timer 7	Level	
78	TIMER0_TMR08	TIMER0 Timer 8	Level	
79	TIMER0_TMR09	TIMER0 Timer 9	Level	
80	TIMER0_STAT	TIMER0 Status	Level	

ADSP-21560_21561_21564_21568 TIMER Trigger List

Table 16-3: ADSP-21560_21561_21564_21568 TIMER Trigger List Generators

Trigger ID	Name	Description	Sensitivity
3	TIMER0_TMR00_MST	TIMER0 Timer 0	Edge
4	TIMER0_TMR01_MST	TIMER0 Timer 1	Edge
5	TIMER0_TMR02_MST	TIMER0 Timer 2	Edge
6	TIMER0_TMR03_MST	TIMER0 Timer 3	Edge
7	TIMER0_TMR04_MST	TIMER0 Timer 4	Edge
8	TIMER0_TMR05_MST	TIMER0 Timer 5	Edge
9	TIMER0_TMR06_MST	TIMER0 Timer 6	Edge
10	TIMER0_TMR07_MST	TIMER0 Timer 7	Edge
11	TIMER0_TMR08_MST	TIMER0 Timer 8	Edge
12	TIMER0_TMR09_MST	TIMER0 Timer 9	Edge

Table 16-4: ADSP-21560_21561_21564_21568 TIMER Trigger List Receivers

Trigger ID	Name	Description	Sensitivity
0	TIMER0_TMR00_SLV0	TIMER0 Timer 0	Pulse
1	TIMER0_TMR00_SLV1	TIMER0 Timer 0	Pulse

Table 16-4: ADSP-21560_21561_21564_21568 TIMER Trigger List Receivers (Continued)

Trigger ID	Name	Description	Sensitivity
2	TIMER0_TMR01_SLV0	TIMER0 Timer 1	Pulse
3	TIMER0_TMR01_SLV1	TIMER0 Timer 1	Pulse
4	TIMER0_TMR02_SLV0	TIMER0 Timer 2	Pulse
5	TIMER0_TMR02_SLV1	TIMER0 Timer 3	Pulse
6	TIMER0_TMR03_SLV0	TIMER0 Timer 3	Pulse
7	TIMER0_TMR03_SLV1	TIMER0 Timer 3	Pulse
8	TIMER0_TMR04_SLV0	TIMER0 Timer 4	Pulse
9	TIMER0_TMR04_SLV1	TIMER0 Timer 4	Pulse
10	TIMER0_TMR05_SLV0	TIMER0 Timer 5	Pulse
11	TIMER0_TMR05_SLV1	TIMER0 Timer 5	Pulse
12	TIMER0_TMR06_SLV0	TIMER0 Timer 6	Pulse
13	TIMER0_TMR06_SLV1	TIMER0 Timer 6	Pulse
14	TIMER0_TMR07_SLV0	TIMER0 Timer 7	Pulse
15	TIMER0_TMR07_SLV1	TIMER0 Timer 7	Pulse
16	TIMER0_TMR08_SLV0	TIMER0 Timer 8	Pulse
17	TIMER0_TMR08_SLV1	TIMER0 Timer 8	Pulse
18	TIMER0_TMR09_SLV0	TIMER0 Timer 9	Pulse
19	TIMER0_TMR09_SLV1	TIMER0 Timer 9	Pulse

Timer Block Diagram

The *Timer Block Diagram* shows all of the possible clock sources.

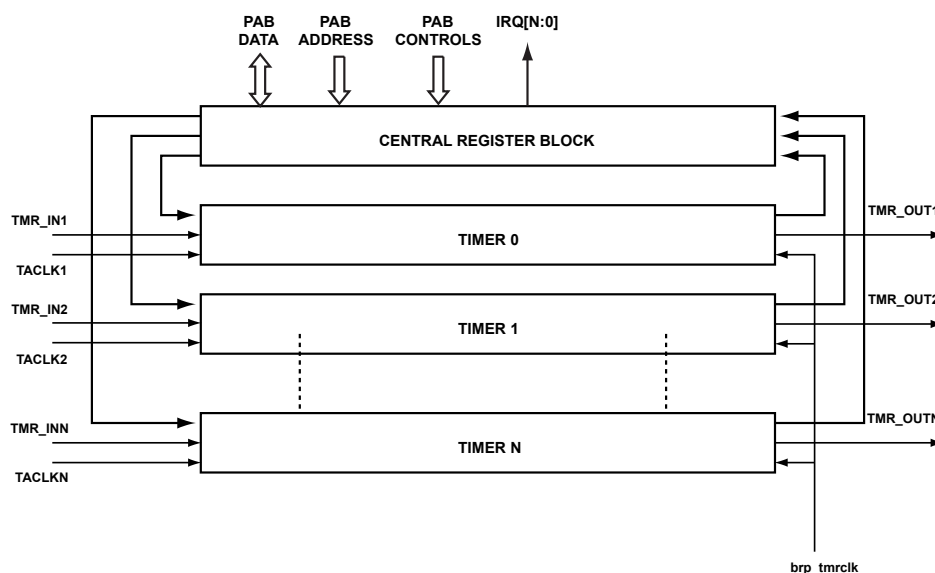


Figure 16-1: Timer Block Diagram

Internal Interface

The processor core always accesses the timer registers through the MMR access bus. Hardware ensures that all read and write operations from and to 32-bit timer registers are atomic. Every timer has a dedicated data interrupt request. There is also one common timer status and error interrupt request output that connects to the system event controller. Whenever a data interrupt request is generated, a data trigger generator pulse is also driven out, if enabled. Each timer has an individual trigger input line, and each timer can be either started or stopped as a trigger receiver.

In total, the GP timer module can have up to $(N + 1)$ interrupt request output lines and N data trigger lines.

Internal Timer Connections

The Timers support alternate inputs for the clock/capture (see [External Interface](#)). Some signals have internal default alternate connections according to the *Timer Signal Routing* table.

Table 16-5: Timer Signal Routing

Timer Signal	Connection
TM0_ACLK0	SYS_CLKIN0
TM0_ACIS	DAI0_CRS_PB04
TM0_ACLK5	DAI0_CRS_PB03
TM0_ACIS6	DAI1_CRS_PB04
TM0_ACLK6	DAI1_CRS_PB03
TM0_ACIS7	CNT0_TO signal
TM0_ACLK7	SYS_CLKIN0

Table 16-5: Timer Signal Routing (Continued)

Timer Signal	Connection
TM0_ACI8	DAI0_PB06
TM0_ACLK8	DAI0_PB05
TM0_ACI9	DAI1_PB06
TM0_ACLK9	DAI1_PB05

External Interface

Each GP timer module can support up to 16 individual timers. However, most processors have less than this number. The exact number of timers available on a given processor is available in the data sheet for the processor.

Every timer has one main input/output signal (`TIMER_TMR[nn]`) and, usually, one auxiliary input pin, used as an alternate capture input (`TIMER_ACI[nn]`). Each timer can either run with a time base of `SCLK` or can reference an external clock on one of two (`TIMER_ACLK[nn]`) pins. The `TMR_ALT_CLK0` signal maps to individual alternate clock (`TIMER_ACLK[nn]`) pins for one or more timers. For instance, a `TM_ACLK3` pin would provide an alternate site to supply an external signal that would serve as reference clock for `TMR3`. Likewise, the `TMR_ALT_CLK1` signal from each timer unit connects internally to provide a single global timer clock pin (`TIMER_CLK`) for the GP timer module. It is used as an additional time base.

GP Timer Operating Modes

The following sections provide information on the various operating modes of the GP timer.

General Operation

The core of every timer is a 32-bit counter that can be interrogated through the read-only `TIMER_TMR[n]_CNT` register. Once the module enables a timer, it loads the timer `TIMER_TMR[n]_CNT` register with a starting value.

A timer can operate in one of several different modes, configured through the `TIMER_TMR[n]_CFG` register for that timer. These modes are: `PWMOUT`, `EXTCLK`, `WIDCAP`, `WATCHDOG`, `PININT`, and `IDLE`. The *Timer Mode Descriptions* table summarizes the modes.

Table 16-6: Timer Mode Descriptions

Timer Mode	Description
PWMOUT	Generates single or continuous PWM waveforms with programmable pulse width, period, and delay
EXTCLK	Counts edges of an externally applied waveform
WIDCAP	Captures pulse width or period of an externally applied waveform
WATCHDOG	Monitors pulse width or period of an external signal and compares against a window of acceptable values, optionally generating an interrupt when it falls inside or outside of that window

Table 16-6: Timer Mode Descriptions (Continued)

Timer Mode	Description
PININT	Can generate an interrupt request on an active edge applied to a timer pin
IDLE	Idle; no activity

Period, Width and Delay Register Interaction

When the timer is started, writes to the buffer registers are immediately copied through to the double-buffered period, pulse width, and delay registers. These values are then ready for use in the first timer period. When a timer is already running, software can write new values to the `TIMER_TMR[n]_PER`, `TIMER_TMR[n]_WID`, and `TIMER_TMR[n]_DLY` registers. The written values are buffered and do not update into the registers until the end of the current period. (The update occurs when the value in the `TIMER_TMR[n]_CNT` register equals the value in the `TIMER_TMR[n]_PER` register.)

If new values are not written to these registers, the value from the previous period is reused. Writes to these registers are atomic; it is not possible for the high word to be written without the low word also being written. Values written to the period, pulse width, and delay registers are always stored in the buffer registers. Reads from the same register always return the current, active value of period, pulse width, or delay value. Written values are not readback until they become active.

The usage of the `TIMER_TMR[n]_PER`, `TIMER_TMR[n]_WID`, and `TIMER_TMR[n]_DLY` registers varies, depending on the mode of the timer specified by the `TIMER_TMR[n]_CFG.TMODE` bits. See the *Usage of the Period, Width, and Delay Registers in Different Timer Modes* table for more information.

Table 16-7: Usage of the Period, Width, and Delay Registers in Different Timer Modes

Timer Mode	<code>TIMER_TMR[n]_PER</code>	<code>TIMER_TMR[n]_WID</code>	<code>TIMER_TMR[n]_DLY</code>
IDLE	Not writable	Not writable	Not writable
WATCHDOG	Update on-the-fly. New value takes effect either at timer start or when an asserting edge on the input signal is sensed.	Read-only. Retains value of last measured width or period of the input signal.	Update on-the-fly. New value takes effect either at timer start or when an asserting edge on the input signal is sensed.
WIDCAP	Read-only. Period value captured at the appropriate time and updated from its buffer register simultaneously with the Width register.	Read-only. Width value captured at the appropriate time and updated from its buffer register simultaneously with the Period register.	Not used
PWMOUT	Update on-the-fly. New value takes effect either at timer start or at the end of the current period. A write followed by immediate read returns the current operating values.	Update on-the-fly. New value takes effect either at timer start or at the end of the current period. A write followed by immediate read returns the current operating values.	Update on-the-fly. New value takes effect either at timer start or at the end of the current period. A write followed by immediate read returns the current operating values.
EXTCLK	Can be updated on-the-fly.	Not used	Not used
PININT	Not used	Not used	Not used

If any of the period, pulse width, and delay registers are not used, then programs cannot write into that register. For example, in WIDCAP mode, the delay registers are not used. So, the program is not allowed to write any value to the `TIMER_TMR[n]_DLY` register. To prevent undesired operation, program the `TIMER_TMR[n]_CFG.TMODE` bits before programming the period, width, or delay registers.

If a program changes the `TIMER_TMR[n]_CFG.TMODE` bits from a status register to writable register (for example in PWMOUT mode), hardware does not clear these registers. These values are automatically overwritten by new values specified by software.

In PWMOUT mode with small periods, there may not be enough time between updates from the buffer registers to write these registers. The next period can use one old value and one new value. To prevent (width + pulse delay) > period errors, write the width and delay registers before the period register when decreasing the values. Write the period register before the width and delay registers when increasing the value.

Single-Pulse PWMOUT Mode

In single-pulse PWMOUT mode, the timer generates a single pulse on the `TIMER_TMR[nn]` pin. This mode is frequently used to implement a precise delay, often with generating an output trigger. The timer module uses the value in the `TIMER_TMR[n]_DLY` register to control the assertion of a pulse. The value in the `TIMER_TMR[n]_WID` register defines the pulse width. The `TIMER_TMR[n]_PER` is not used and cannot be written in this mode. After completion of the pulse, the timer is automatically stopped, and optionally generates an interrupt. The timer uses the `TIMER_TMR[n]_CFG.PULSEHI` bit to control pulse polarity.

The timer can be configured to generate a data interrupt request after satisfying various conditions specified by the `TIMER_TMR[n]_CFG.IRQMODE` bits.

It is not necessary to clear the relevant `TIMER_RUN` bit to stop the timer cleanly. At the end of the pulse, the timer stops automatically and the corresponding `TIMER_RUN` bit is cleared. To generate multiple discrete pulses (as opposed to a continuous PWM waveform), write a 1 to the appropriate `TIMER_RUN` bit, and wait for the timer to stop. Then, write another 1 to the same `TIMER_RUN` bit.

Continuous PWMOUT Mode

In continuous PWMOUT mode, the timer generates a repetitive pulse with a well-defined period, duty cycle, and pulse position. The `TIMER_TMR[n]_DLY`, `TIMER_TMR[n]_PER`, and `TIMER_TMR[n]_WID` registers are programmed with the values of the required PWM pulse. After the timer is started, the counter counts towards the value programmed in the `TIMER_TMR[n]_PER` register. Initially, the `TIMER_TMR[nn]` pin remains in a deasserted state. The pin toggles to an asserted state when the value in the `TIMER_TMR[n]_CNT` register equals the value in the `TIMER_TMR[n]_DLY` register.

The timer can control the assertion sense of the `TIMER_TMR[nn]` pin with the `TIMER_TMR[n]_CFG.PULSEHI` bit. The `TIMER_TMR[nn]` pin holds this value for the number of clock cycles specified in the `TIMER_TMR[n]_WID` register. Then, the pin deasserts and holds this value until the completion of the programmed period. The same waveform is generated repeatedly until the timer is disabled.

The timer can be configured to generate a data interrupt request after satisfying any of various conditions specified by the `TIMER_TMR[n]_CFG.IRQMODE` bits.

It is important to guarantee that the programmed period is greater than or equal to the sum of width and delay. Violating either of these criteria results in an unpredictable waveform on the `TIMER_TMR[nn]` pin until the situation is rectified by writing proper values to these registers.

The maximum frequency possible to generate on the `TIMER_TMR[nn]` pin is achieved by setting `TIMER_TMR[n]_PER` to 2 and `TIMER_TMR[n]_WID` to 1. This operation makes the `TIMER_TMR[nn]` pin toggle each SCLK clock cycle (assuming the timer is configured to clock internally), producing a duty cycle of 50%.

When the `TIMER_STOP_CFG.TMR[nn]` bit of a timer is 0, the timer treats a stop operation as a stop-is-pending condition. When terminated with this setting, the timer automatically completes the current waveform and then stops cleanly, remaining in a deasserted state. This functionality prevents truncation of the current pulse and unwanted PWM patterns at the `TIMER_TMR[nn]` pin. The processor can determine when the timer stops running by polling the corresponding `TIMER_RUN.TMR[nn]` bit until it reads 0 or by waiting for the last interrupt (if enabled).

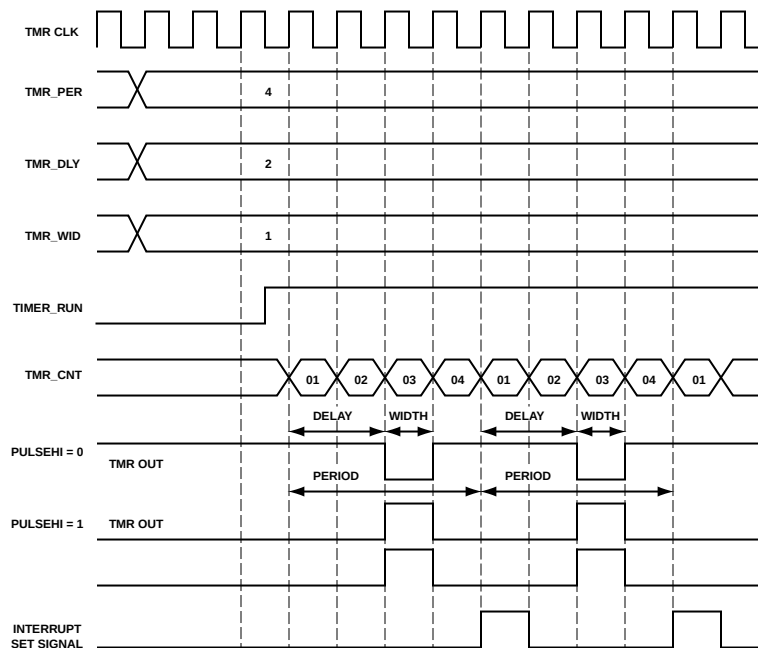


Figure 16-2: Signal Generation in Continuous PWMOUT Mode

The `TIMER_TMR[n]_CFG` register cannot be reconfigured until after the timer stops and the `TIMER_RUN` register reads 0.

Programs can force a timer to stop immediately in PWMOUT mode by writing a 1 to the `TIMER_STOP_CFG` register followed by writing a 1 to the `TIMER_RUN_CLR` register. (Or a program can stop a timer by writing a 0 to the appropriate `TIMER_RUN.TMR[nn]` bit.) This operation stops the timer whether the pending stop is waiting for the end of the current period or the end of the current pulse width. The timer can use this feature to regain immediate control of a timer during an error recovery sequence.

Use this feature carefully, as it can corrupt the PWM pattern generated at the `TIMER_TMR[nn]` pin, though after such a stop the pin deasserts automatically. Each timer samples its `TIMER_RUN.TMR[nn]` bit at the end of each period. It stops cleanly at the end of the first period after the `TIMER_RUN.TMR[nn]` bit is low. A timer that is disabled and then restarted (before the end of the current period), continues to run as if nothing happened. Typically, the program disables a PWMOUT timer and then waits for it to stop itself.

Width Capture (WIDCAP) Mode

The timer uses WIDCAP mode, often called capture mode, to measure pulse widths on the `TIMER_TMR[nn]` or inputs. The polarity (active high or low) of the input signal can be selected with the `TIMER_TMR[n]_CFG.PULSEHI` bit. The *Timer Signal Flow in Width Capture Mode* figure shows the control signal flow for WIDCAP_CAP mode.

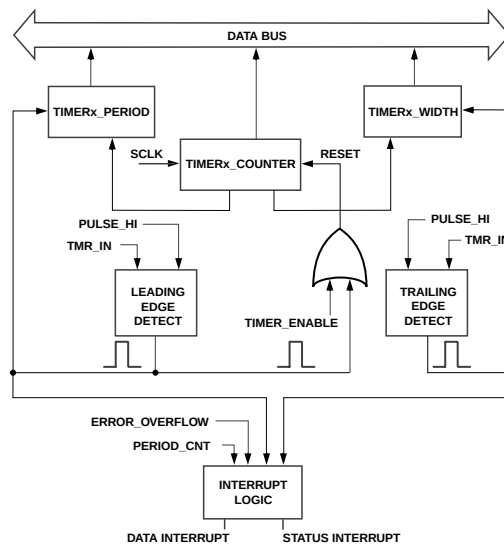


Figure 16-3: Timer Signal Flow in Width Capture Mode

NOTE: SCLK in the *Timer Signal Flow in Width Capture Mode* figure is SCLK.

In this mode, the timer uses the `TIMER_TMR[n]_CFG.TINSEL` bit to select between the `TIMER_TMR[nn]` or input. The internally clocked timer is used to determine the period and pulse width of the externally applied rectangular waveforms.

When a timer is enabled in this mode, the timer resets the count in its `TIMER_TMR[n]_CNT` register to 0x0000 0001. It does not start counting until it detects a leading edge on the selected input pin.

When the timer detects the first leading edge, it starts incrementing. When it detects a trailing edge of a waveform, it captures the current 32-bit value of its `TIMER_TMR[n]_CNT` register into its width buffer register. At the next leading edge, the timer transfers the current 32-bit value of its `TIMER_TMR[n]_CNT` register into its period buffer register. The `TIMER_TMR[n]_CNT` register is reset to 0x0000 0001 again, and the timer continues counting and capturing until it is disabled.

In this mode, programs can measure both the pulse width and the pulse period of a waveform. The timer does not use the `TIMER_TMR[n]_DLY` register in this mode. The timer uses the `TIMER_TMR[n]_CFG.PULSEHI` bit to control the definition of leading edge and trailing edge of the `TIMER_TMR[nn] / pin`.

In WIDCAP mode, the following events always occur at the same time as one unit:

1. The `TIMER_TMR[n]_PER` register is updated from the period buffer register.
2. The `TIMER_TMR[n]_WID` register is updated from the width buffer register.
3. The `TIMER_DATA_ILAT.TMR[nn]` bit is set (if enabled).
4. A timer data trigger pulse is generated (if enabled).

The `TIMER_TMR[n]_CFG.TMODE` bit 0 controls the point in time at which this set of events is executed. Taken together, these four events are called a measurement report. The `TIMER_STAT_ILAT` register is not set at a measurement report. A measurement report occurs, at most, once per input signal period. The current `TIMER_TMR[n]_CNT` value is always copied to the width buffer and period buffer registers at the trailing and leading edges of the input signal, respectively. But these values are not visible to software. A measurement report event samples the captured values into visible registers and sets the timer interrupt request to signal that the `TIMER_TMR[n]_PER` and the `TIMER_TMR[n]_WID` registers are ready to be read.

When the `TIMER_TMR[n]_CFG.TMODE` bit = `b#1011`, the measurement report occurs just after the width buffer register captures its value at a falling edge. Then, the `TIMER_TMR[n]_WID` register reports the pulse width measured in the pulse that has ended, but the `TIMER_TMR[n]_PER` register reports the pulse period measured at the end of the previous period. If only the first trailing edge has occurred, then the first period value has not yet been measured at the first measurement report. So, the period value is not valid. A read of the `TIMER_TMR[n]_PER` value in this case returns 0. See the *Example of Width Capture Deasserted Mode (TMODE=b#1011)* figure for more information.

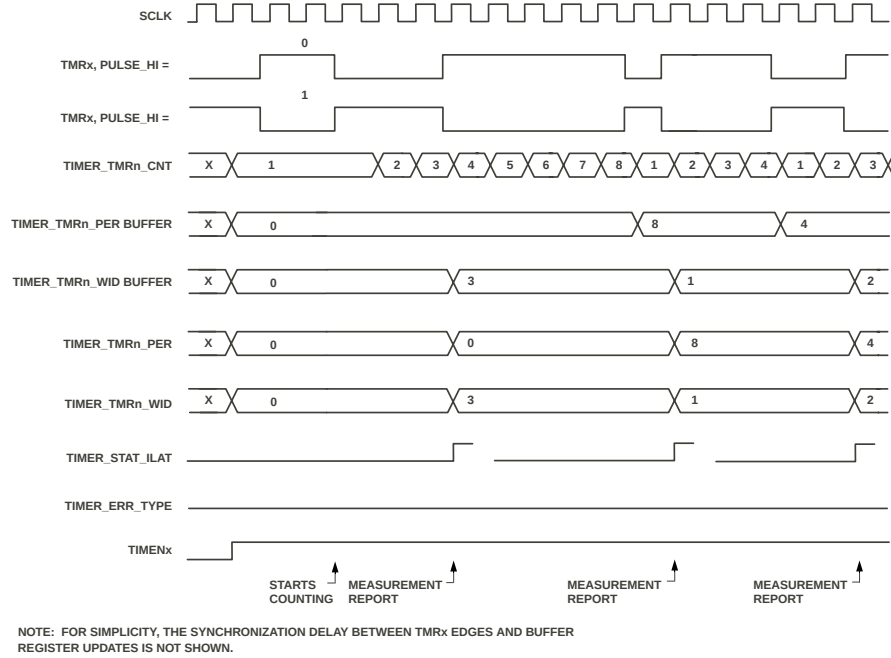


Figure 16-4: Example of Width Capture Deasserted Mode (TMODE=b#1011)

NOTE: SCLK in the *Example of Width Capture Deasserted Mode (TMODE=b#1011)* figure is SCLK.

When the `TIMER_TMR[n]_CFG.TMODE` bit =b#1010, the measurement report occurs just after the period buffer register captures its value at a leading edge. Then, the `TIMER_TMR[n]_PER` and `TIMER_TMR[n]_WID` registers report the pulse period and pulse width measured in the period that has ended. Refer to the *Example of Width Capture Asserted Mode (TMODE=b#1010)* figure for more information.

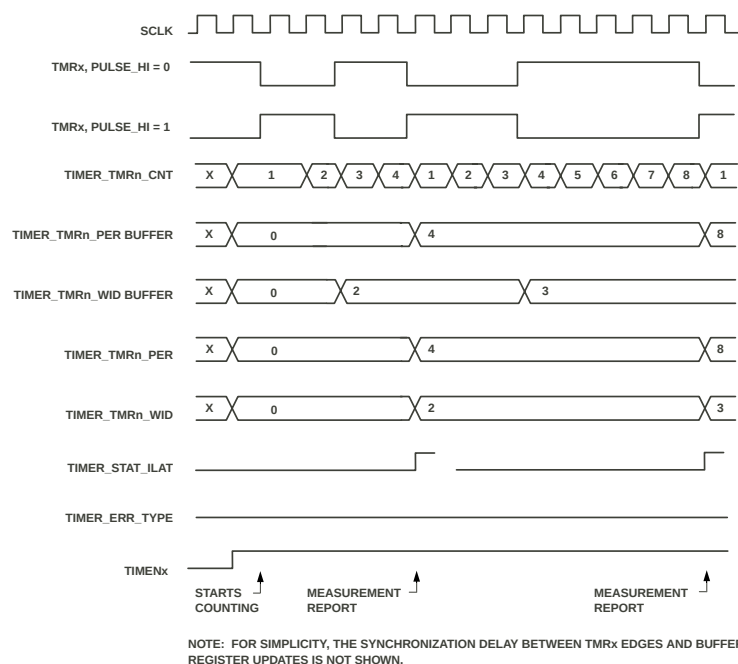


Figure 16-5: Example of Width Capture Asserted Mode (TMODE=b#1010)

NOTE: SCLK in the *Example of Width Capture Asserted Mode (TMODE=b#1010)* figure is SCLK.

To measure the pulse width of a waveform that has only one leading edge and one trailing edge, set `TIMER_TMR[n]_CFG.TMODE = b#1011`. When `TIMER_TMR[n]_CFG.TMODE = b#1010`, no period value is captured in the period buffer register. Instead, the timer generates an error report interrupt request (if enabled) when the `TIMER_TMR[n]_CNT` range is exceeded and the counter wraps around. In this case, both the `TIMER_TMR[n]_PER` and `TIMER_TMR[n]_WID` registers read 0 (because no measurement report occurred to copy the value captured in the width buffer register to the `TIMER_TMR[n]_WID` register).

When using the `TIMER_TMR[n]_CFG.TMODE` bit =b#1010 mode to measure the width of a single pulse, programs can disable the timer after taking the interrupt that ends the measurement interval. When desired, restart the timer as appropriate in preparation for another measurement. This procedure prevents the timer from freely running after the width measurement and logging errors generated by the timer count overflowing.

Width Capture Mode Overflow

A timer status interrupt request (when enabled) is generated when the `TIMER_TMR[n]_CNT` register wraps around from 0xFFFF FFFF to 0 in the absence of a leading edge. At that point, the `TIMER_STAT_ILAT` bit is set and the `TIMER_ERR_TYPE` bits change to indicate a count overflow due to a period greater than the range of the counter. This indication is referred to as an error report. A data interrupt request in WIDCAP mode indicates that a new measurement is ready to be read (a measurement report). Similarly, an interrupt request on the timer status interrupt line (shared interrupt request for all timers) indicates an overflow error when generated in this mode.

The `TIMER_TMR[n]_PER` and `TIMER_TMR[n]_WID` registers are never updated at the time an overflow error is signaled. If the timer overflows and the `TIMER_TMR[n]_CFG.TMODE` bit =b#1010, the `TIMER_TMR[n]_PER` and `TIMER_TMR[n]_WID` registers are not updated. If the timer overflows and the

`TIMER_TMR[n]_CFG.TMODE` bit = `b#1011`, the `TIMER_TMR[n]_PER` and `TIMER_TMR[n]_WID` registers are updated only if a trailing edge is detected at a previous measurement report.

Software can count the number of error reports between measurement report interrupt requests to measure input signal periods longer than `0xFFFF FFFF`. Each error report interrupt request adds a full 2^{32} SCLK counts to the total for the period, but the width is ambiguous. Ensure that if software monitors only the status interrupt request, then status interrupt requests from all other timers are masked.

Refer to the *Example Timing for Width Capture Followed by Period Overflow (TMR_CFG.TMODE=b#1010)* figure. The period is `0x1 0000 0004`, but the pulse width could be either `0x0 0000 0002` or `0x1 0000 0002`.

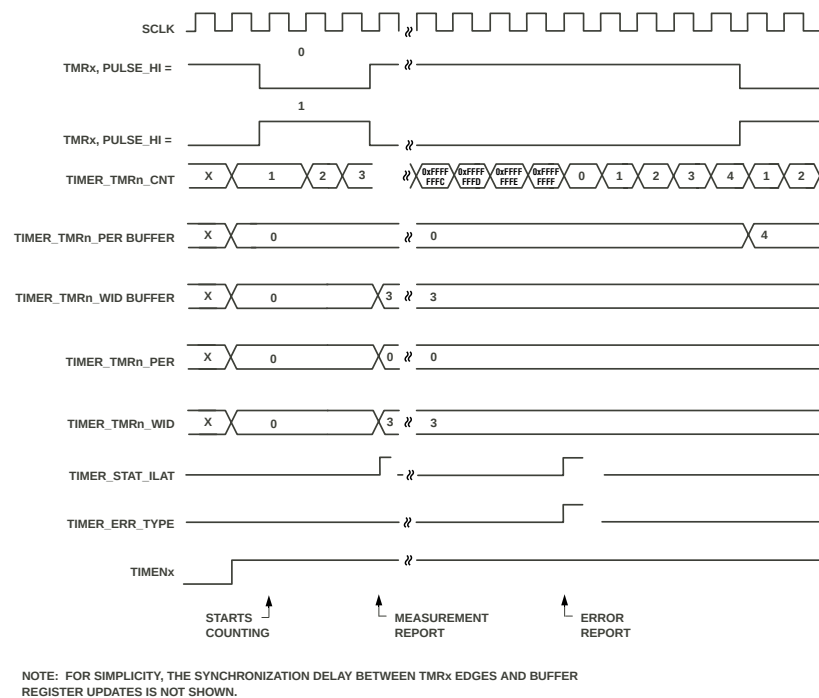


Figure 16-6: Example Timing for Width Capture Followed by Period Overflow (TMR_CFG.TMODE=b#1010)

NOTE: SCLK in the *Example Timing for Width Capture Followed by Period Overflow* figure is SCLK.

The waveform applied to the `TIMER_TMR[nn]` (or) pin is not required to have a 50% duty cycle. The minimum input low time is little more than one SCLK period. The minimum input high time is a little more than one SCLK period. (Refer to the product data sheet for details.) The maximum `TIMER_TMR[nn]` input frequency is less than $\text{SCLK}/2$, with a 50% duty cycle. Under these conditions, the WIDCAP mode timer measures: period = 2 and pulse width = 1.

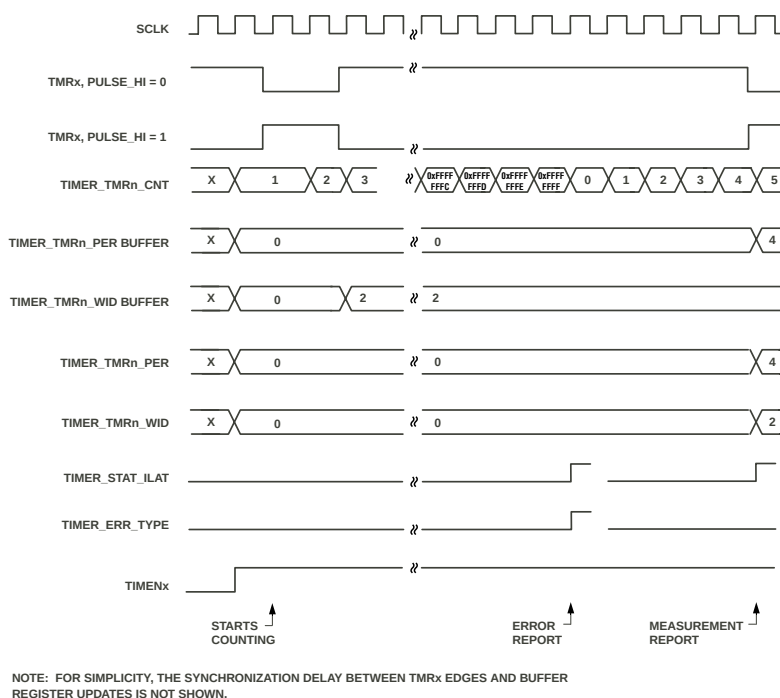


Figure 16-7: Example Timing for Width Capture Followed by Period Overflow (TMR_CFG.TMODE=b#1011)

NOTE: SCLK in the *Example Timing for Width Capture Followed by Period Overflow* figure is SCLK.

Windowed Watchdog (WATCHDOG) Modes

The waveform applied to the input pin in watchdog mode is not required to have a 50% duty cycle. The minimum input pulse low time, high time, and total period specifications are available in the product data sheet.

Windowed Watchdog Width Mode

- Inside the window ($p_{MIN} < \text{pulse width} \leq p_{MAX}$), or
- Outside the window ($\text{pulse width} \leq p_{MIN}$ or $\text{pulse width} > p_{MAX}$)

After enabling the timer in this mode, it always starts counting at the asserting edge of the input signal. Any pulse that is already active when the timer is enabled is ignored.

With the `TIMER_TMR[n]_CFG.IRQMODE` bit =b#11, the timer generates an interrupt if the timed pulse width exceeds p_{MAX} , or if the pulse width is less than p_{MIN} . After attaining p_{MAX} , the pulse stays at an active level, and the counter keeps on counting until it sees a deasserting edge. When the input pulse is not active, the counter holds its current value until it again sees an asserting edge, or it restarts. An interrupt can also be generated for when the pulse occurs within the specified window condition, by setting `TIMER_TMR[n]_CFG.IRQMODE` =b#10.

In this mode, a trailing edge on the input pin triggers capturing of pulse width into the `TIMER_TMR[n]_WID` register. During the inactive portion of the input signal, the internal counter does not increment. The *Watchdog Width Mode Timing* figure shows the signal flow in this mode.

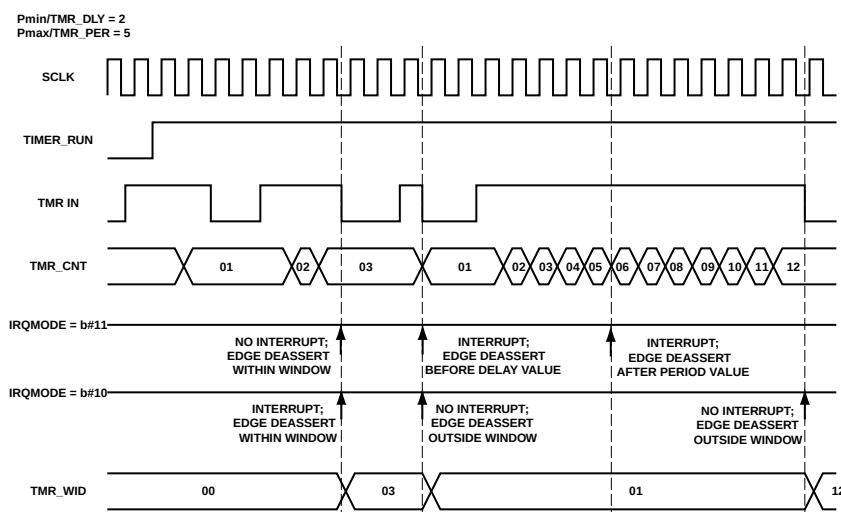


Figure 16-8: Watchdog Width Mode Timing

NOTE: SCLK in the *Watchdog Width Mode Timing* figure is SCLK.

To check only the upper limit on pulse width (P_{MAX} but not P_{MIN}), then program P_{MIN} as 0 or 1. In such a case, it is better to use `TIMER_TMR[n]_CFG.IRQMODE = b#11`. With `TIMER_TMR[n]_CFG.IRQMODE = b#10`, a pulse width of 1 clock cycle results in an interrupt. For details, see the *Windowed Watchdog Width Mode Interpretation* table.

Table 16-8: Windowed Watchdog Width Mode Interpretation

Timer Delay	Timer Period	Incoming Pulse Width	IRQMODE= b#10	IRQMODE= b#11	Error Interrupt?
0 or 1	Anything ≥ 1	$PW = 1$	Interrupt at deasserting edge of input signal	No Interrupt	No Error Interrupt
		$PW \leq TMR_PER$	Interrupt at deasserting edge of input signal	No Interrupt	No Error Interrupt
		$PW > TMR_PER$	No Interrupt	Interrupt when pulse width exceeds P_{max} (Period Register) Value	No Error Interrupt

Table 16-8: Windowed Watchdog Width Mode Interpretation (Continued)

Timer Delay	Timer Period	Incoming Pulse Width	IRQMODE= b#10	IRQMODE= b#11	Error Interrupt?
> 1 but $\leq$ (Period -1)	Anything > 1	$PW \leq TMR_DLY$	No Interrupt	Interrupt at deasserting edge of input signal	No Error Interrupt
		$TMR_DLY < PW \leq TMR_PER$	Interrupt at deasserting edge of input signal	No Interrupt	No Error Interrupt
		$PW > TMR_PER$	No Interrupt	Interrupt when pulse width exceeds Pmax (Period Register) Value	No Error Interrupt
$\geq$ Period	-	$PW \leq TMR_PER$	Undefined	Undefined	No Error Interrupt
	-	$PW > TMR_PER$	Undefined	Undefined	b#11 Error Type
-	0	-	Undefined	Undefined	b#10 Error Type

Windowed Watchdog Period Mode

In this mode, the timer monitors the number of clock cycles between two consecutive rising or falling edges of an input signal on one of the `TIMER_TMR[nn]` or `TIMER_ACI[nn]` pins. Program the required minimum number of clock cycles (t_{MIN}) in the `TIMER_TMR[n]_DLY` register and the required maximum allowed number of clock cycles (t_{MAX}) in the `TIMER_TMR[n]_PER` register. Both values are programmed in terms of number of clock cycles (SCLK) or alternate time clock (`TIMER_ACLK[nn]`). The timer can generate an interrupt when two consecutive occurrences of an active edge are:

- Within a specified window ($t_{MIN} < \text{Pulse Period} \leq t_{MAX}$), or
- Outside a specified window ($\text{pulse width} \leq (t_{MIN} \text{ or } t_{MAX} < \text{pulse width})$)

When the `TIMER_TMR[n]_CFG.IRQMODE` bit =b#11 and the pulse period $> t_{MAX}$ or is $\leq t_{MIN}$, the timer generates an interrupt (if unmasked). After attaining the t_{MAX} value, the counter keeps on counting until it sees an active edge on the input line. An interrupt can also be generated for when the pulse occurs within the specified window condition, by setting `TIMER_TMR[n]_CFG.IRQMODE` =b#10. Refer to the *Watchdog Period Mode Timing* figure for timer functionality in period watchdog mode.

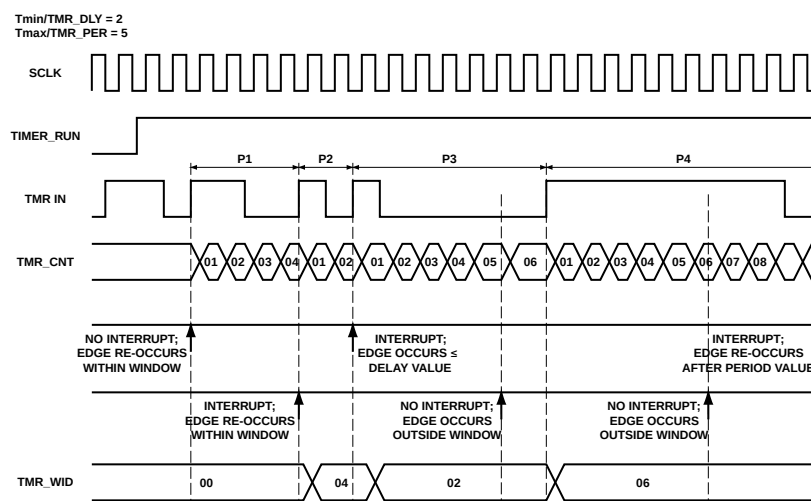


Figure 16-9: Watchdog Period Mode Timing

NOTE: SCLK in the *Watchdog Period Mode Timing* figure is SCLK.

To check only the upper limit on period (the t_{MAX} value, not the t_{MIN} value), program t_{MIN} as 0 or 1. For details, refer to the *Windowed Watchdog Period Mode Interpretation* table.

Table 16-9: Windowed Watchdog Period Mode Interpretation

Timer Delay	Timer Period	Incoming Pulse Width	IRQMODE=b#10	IRQMODE =b#11	Error Interrupt?
0 or 1	Anything ≥ 2	Pulse Period $\leq$ TMR_PER	Interrupt at deasserting edge of input signal	No Interrupt	No Error Interrupt
		Pulse Period $>$ TMR_PER	No Interrupt	Interrupt when pulse period crosses Pmax (Period Register) value	No Error Interrupt
> 1 but $\leq$ Period -1	Anything ≥ 2	Pulse Period $\leq$ TMR_DLY	No Interrupt	Interrupt at deasserting edge of input signal	No Error Interrupt
		TMR_DLY $<$ Pulse Period $\leq$ TMR_PER	Interrupt at deasserting edge of input signal	No Interrupt	No Error Interrupt
		Pulse Period $>$ TMR_PER	No Interrupt	Interrupt when pulse width exceeds Pmax (Period Register) value	No Error Interrupt
$\geq$ Period	-	Pulse Period $<$ TMR_PER	Undefined	Undefined	No Error Interrupt

Table 16-9: Windowed Watchdog Period Mode Interpretation (Continued)

Timer Delay	Timer Period	Incoming Pulse Width	IRQMODE=b#10	IRQMODE =b#11	Error Interrupt?
		Pulse Period $\geq$ TMR_PER	Undefined	Undefined	b#11 Error Type
-	0 or 1	-	Undefined	Undefined	b#10 Error Type

Pin Interrupt (PININT) Mode

In PININT mode, any active edges on either the `TIMER_TMR[nn]` pin or the pin can cause an edge-based interrupt, if enabled. (The timer uses the `TIMER_TMR[n]_CFG.TINSEL` register to select the pin). The event on the input pin can set the `TIMER_DATA_ILAT.TMR[nn]` bit and issue a system interrupt request. Program the `TIMER_TMR[n]_CFG.PULSEHI` bit to change active edge polarity.

Since the interrupt request is generated in the SCLK clock domain, the width of the input signal must be more than one SCLK period. Along with generating the interrupt request, the timer also generates a trigger pulse (configured using the `TIMER_TRG_MSK` register). Due to the configuration of polarity, glitches can cause the generation of an undesired interrupt request at the input. To avoid this problem, programs must ensure that interrupt requests are unmasked only after configuring the desired polarity.

External Clock (EXTCLK) Mode

The timer uses EXTCLK mode, sometimes referred to as the counter mode, to count external events (signal edges), on either the `TIMER_TMR[nn]` or input pin. The timer works as a counter clocked by an external source (the signal at the pin), which can be asynchronous to SCLK. The current count in the `TIMER_TMR[n]_CNT` register represents the number of leading-edge events detected. The `TIMER_TMR[n]_PER` register is programmed with the value of the maximum timer external count before stopping or issuing an interrupt request or trigger.

The `TIMER_TMR[n]_CFG.PULSEHI` bit determines the polarity of the leading edge on the input pin. The timer uses the `TIMER_TMR[n]_CFG.TINSEL` bit to select whether the event is counted on the `TIMER_TMR[nn]` or on the pin. The `TIMER_STAT_ILAT.TMR[nn]` and `TIMER_ERR_TYPE` bits are set if *one* of these conditions is met:

- `TIMER_TMR[n]_CNT` wraps around from 0xFFFF FFFF to 0
- The period = 0 at startup
- `TIMER_TMR[n]_CNT` register rolls over (from count = period to count = 0x1)

The `TIMER_TMR[n]_WID` and `TIMER_TMR[n]_DLY` registers are unused in this mode and must not be written.

The *EXTCLK Mode Control Flow* figure shows a flow diagram for EXTCLK mode.

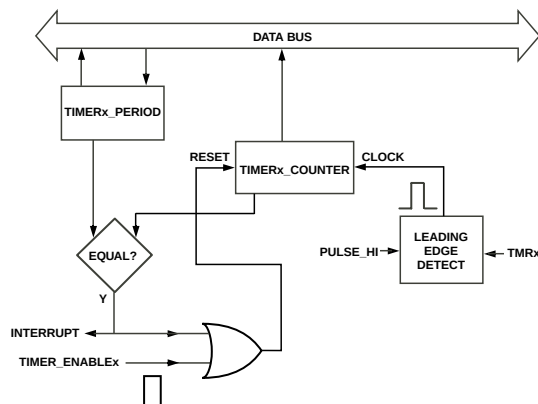


Figure 16-10: EXTCLK Mode Control Flow

The waveform applied to the input pin is not required to have a 50% duty cycle. The minimum input pulse low time, high time, and total period specifications are available in the product data sheet. Program the period to any value from 1 to $(2^{32} - 1)$, inclusive.

After the timer has started, it resets the `TIMER_TMR[n]_CNT` register to 0x0 and then waits for the first leading edge on the input pin. This edge causes `TIMER_TMR[n]_CNT` to be incremented to the value 0x1, and every subsequent leading edge increments it by one. After the `TIMER_TMR[n]_CNT` register reaches the value programmed in the `TIMER_TMR[n]_PER` register, the corresponding `TIMER_DATA_ILAT` bit is set, and an interrupt and trigger are both generated (if enabled). The next leading-edge reloads the `TIMER_TMR[n]_CNT` register with 0x1, and the timer continues counting until it is disabled.

GP Timer Programming Concepts

Using the features, operating modes, and event control for the GP timer to their greatest potential requires an understanding of some GP timer-related concepts.

Setting Up Constantly Changing Timer Conditions

This task shows how to use different period, pulse width, and delay settings for each of the first three timer periods after the timer starts.

1. Program the first set of `TIMER_TMR[n]_PER`, `TIMER_TMR[n]_WID`, and `TIMER_TMR[n]_DLY` register values.
2. Enable the timer using the `TIMER_RUN` register.
3. Immediately program the second set of `TIMER_TMR[n]_PER`, `TIMER_TMR[n]_WID`, and `TIMER_TMR[n]_DLY` register values, as needed.
4. Wait for the first timer interrupt request.
5. Program the third set of `TIMER_TMR[n]_PER`, `TIMER_TMR[n]_WID`, and `TIMER_TMR[n]_DLY` register values.

Each new setting is then programmed when the preceding timer interrupt request is received.

Configuring, Enabling, and Disabling One or More Timers

1. Configure the relevant timers for the operating mode and other properties using the `TIMER_TMR[n]_CFG` register.
2. Write a 1 to the representative `TIMER_RUN.TMR[nn]` bit. Or use the `TIMER_RUN_SET` register to avoid disturbing the settings of other timers that are not going through configuration.

The timer is enabled and operating.

3. To stop one or more timers, first program the `TIMER_STOP_CFG` register to determine whether to stop immediately or gracefully upon receiving a stop command.

ADDITIONAL INFORMATION: PWMOUT modes are the only modes where a timer can be configured for graceful termination.

4. Write a 0 to the representative `TIMER_RUN.TMR[nn]` bits to stop the timer according to their `TIMER_STOP_CFG` settings. Alternately, write a 1 to the appropriate `TIMER_RUN_CLR.TMR[nn]` bits to avoid disturbing the settings of other timers that are not terminating.

The timers stop.

Configuring Timer Data and Status Interrupts

1. Configure the `TIMER_TMR[n]_CFG.IRQMODE` bit field with the desired interrupt properties.
2. Unmask the interrupt source.
3. Set the `TIMER_TMR[n]_CFG.IRQMODE` field but leave the interrupt masked at the system level to poll the `TIMER_DATA_ILAT.TMR[nn]` bit of the timer without generating an interrupt.
4. Use the `TIMER_STAT_IMSK` register to generate interrupt requests by overflow or error conditions (incorrect programming values). The timer uses the `TIMER_STAT_ILAT.TMR[nn]` bits to report interrupt errors, when the timer status interrupt source is unmasked.
5. To poll the `TIMER_STAT_ILAT.TMR[nn]` bit of the timer without generating an interrupt, unmask the corresponding bit in the `TIMER_STAT_IMSK` register, but leave the interrupt masked at the system level.

Configuring the Timer as a Trigger Receiver

The timer can be configured to either start or stop or toggle between these two states on the input trigger pulse depending on the configuration of the `TIMER_TMR[n]_CFG.TGLTRIG` and `TIMER_TMR[n]_CFG.SLAVETRIG` bits.

- If `TIMER_TMR[n]_CFG.TGLTRIG` bit =0 and `TIMER_TMR[n]_CFG.SLAVETRIG` bit =1 then the trigger pulse starts timer, if it is stopped.

- If `TIMER_TMR[n]_CFG.TGLTRIG` bit =0 and `TIMER_TMR[n]_CFG.SLAVETRIG` bit =0 then the trigger pulse stops timer, if it is running.
- When `TIMER_TMR[n]_CFG.TGLTRIG` bit =0, the trigger pulse has no effect when the timer is already in the requested state.

If `TIMER_TMR[n]_CFG.TGLTRIG` bit is 1, the trigger pulse starts the timer if it is stopped, or stops the timer if it is running. The `TIMER_TMR[n]_CFG.SLAVETRIG` bit has no effect on trigger mechanism. In continuous PWMOUT mode, the timer stops gracefully or abruptly depending on the stop mechanism programmed in the `TIMER_STOP_CFG_CLR` register. In other modes, the timer stops immediately.

Ordered Trigger Toggle Mode

The timer can be configured to either start or stop the timer on the input trigger pulse depending on the configuration of the `TIMER_TMR[n]_CFG.TGLTRIG` and `TIMER_TMR[n]_CFG.ORDTGLTRIG` bits.

- If `TIMER_TMR[n]_CFG.TGLTRIG` bit is 1 and `TIMER_TMR[n]_CFG.ORDTGLTRIG` bit is 0, then the input trigger pulse on trigger receiver 0 or trigger receiver 1 starts the timer if it is stopped, or stops the timer if it is running.
- If `TIMER_TMR[n]_CFG.TGLTRIG` bit is 1 and `TIMER_TMR[n]_CFG.ORDTGLTRIG` bit is 1, then the input trigger pulse on trigger receiver 0 starts timer if it is stopped. If the timer is running, it stops when the input trigger pulse on trigger receiver 1 is detected.

Trigger receiver 0 cannot toggle the timer from the run to halt state. Trigger receiver 1 cannot toggle the timer from the halt to run state.

NOTE: The timer state is not toggled by writes to the `TIMER_RUN`, `TIMER_RUN_SET`, and `TIMER_RUN_CLR` registers in this mode.

Using the Timer Broadcast Feature

The broadcast feature provides a means to update period, width, and delay registers simultaneously across more than one timer.

1. Enable the appropriate broadcast bits (`TIMER_TMR[n]_CFG.BPEREN`, `TIMER_TMR[n]_CFG.BWIDEN` are `TIMER_TMR[n]_CFG.BDLYEN`) for the timers involved in the broadcast. The use of these bits depends on which broadcast registers the timer uses (`TIMER_BCAST_PER`, `TIMER_BCAST_WID`, or `TIMER_BCAST_DLY`).
2. Program the `TIMER_BCAST_PER` register (for example), to broadcast the period setting across the multiple timers enabled.

The enabled timers load their `TIMER_TMR[n]_PER` registers with the value specified in the `TIMER_BCAST_PER` register.

3. Repeat Step 2 as needed for the `TIMER_BCAST_WID` and `TIMER_BCAST_DLY` register settings.

Timer Illegal States

The following sections use these definitions:

- Startup. The first clock period during which the timer counter is running after the timer is started by writing the `TIMER_RUN` register.
- Rollover. The time when the current count in `TIMER_TMR[n]_CNT` matches the value in `TIMER_TMR[n]_PER` and the counter is reloaded with the value 1.
- Overflow. The timer counter was incremented instead of doing a rollover when it was holding the maximum count value of 0xFFFF FFFF. The counter does not have a large enough range to express the next greater value and so it erroneously loads a new value of 0x0000 0000.
- Unchanged. No new error.

When the `TIMER_ERR_TYPE` register is designated unchanged, it displays the previously reported error code orb# 00 when there has been no error since this timer was enabled.

When the `TIMER_STAT_ILAT` register is unchanged, it reads 0 when there has been no error or overflow since this timer was enabled. Or, it reads 0 if software has performed a W1C to clear any previous error. If software has not acknowledged a previous error, the `TIMER_STAT_ILAT` register reads 1. Software can read the `TIMER_STAT_ILAT` register to check for errors. If a particular bit of a timer is set in this register, software can then read the `TIMER_ERR_TYPE` register for more information. Once detected, software can W1C the appropriate `TIMER_STAT_ILAT` bit to acknowledge the error.

Read the following tables as:

- In mode __ at event __,
- if `TIMER_TMR[n]_PER` is __ and `TIMER_TMR[n]_WID` is __ and `TIMER_TMR[n]_DLY` is __,
- then `TIMER_ERR_TYPE` is __ and `TIMER_STAT_ILAT` is __.

Startup error conditions do not prevent the timer from starting. Similarly, overflow and rollover error conditions do not stop the timer. Illegal cases can cause unwanted behavior of the `TIMER_TMR[nn]` pins.

NOTE: For PININT mode, the timer does not use error functionality.

Continuous PWMOUT Mode

Table 16-10: Startup Event

<code>TIMER_TMR[n]_PER</code>	<code>TIMER_TMR[n]_DLY</code>	<code>TIMER_TMR[n]_WID</code>	<code>TIMER_TMR[n]_WID + TIMER_TMR[n]_DLY</code>	<code>TIMER_ERR_TYPE</code>	<code>TIMER_STAT_ILAT</code> (if enabled)
≤ 1	Anything other than period ^{*1}	Anything	Anything	b#10	Set

Table 16-10: Startup Event (Continued)

TIMER_TMR[n]_PER	TIMER_TMR[n]_DLY	TIMER_TMR[n]_WID	TIMER_TMR[n]_WID + TIMER_TMR[n]_DLY	TIMER_ERR_TYPE	TIMER_STAT_ILAT (if enabled)
≥ 2	Anything including 0, excluding TMR_PER value	Anything including 0	$\leq \text{PERIOD}$	Unchanged	Unchanged
	Anything including 0	Anything including 0	$> \text{PERIOD}$	Unchanged *2 (Detected at rollover)	Unchanged (Detected at rollover)
	Anything	Anything	$> 2^{32} - 1$	b#11	Set
	=Period	=0	=Period	No error	Unchanged (Detected at rollover)

*1 If delay = period, the #10 error is rolled to a #11 error.

*2 This case is not detected at startup, but when the timer counts to the complete period.

Table 16-11: Rollover Event

TIMER_TMR[n]_PER	TIMER_TMR[n]_DLY	TIMER_TMR[n]_WID	TIMER_TMR[n]_WID + TIMER_TMR[n]_DLY	TIMER_ERR_TYPE	TIMER_STAT_ILAT (if enabled)
≥ 1	Anything	Anything	Anything	b#10[timer running at SCLK] b#11 [timer running at ALT_CLKx]	Set
≥ 2	Anything including 0, excluding TMR_PER value	Anything including 0	$\leq \text{PERIOD}$	Unchanged	Unchanged
	Anything including 0, excluding TMR_PER value	Anything > 0	$> \text{PERIOD}$	b#11	Set
	Anything	Anything	$> 2^{32} - 1$	b#11	Set
	= Period*1	=0	=Period	b#11	Set
	>Period	=0	>Period	Unchanged	Unchanged

*1 In case of graceful termination, this error is not generated.

Table 16-12: Overflow Event (On TMR_PER Register Programming Error Only)

TIMER_TMR[n]_PER	TIMER_TMR[n]_DLY	TIMER_TMR[n]_WID	TIMER_TMR[n]_WID + TIMER_TMR[n]_DLY	TIMER_ERR_TYPE	TIMER_STAT_ILAT (if enabled)
Anything	Anything	Anything	Anything	b#01	Set

Single Pulse PWMOUT Mode

For single pulse PWMOUT mode, there are no rollover events.

Table 16-13: Startup Event

<code>TIMER_TMR[n]_PER</code>	<code>TIMER_TMR[n]_DLY</code>	<code>TIMER_TMR[n]_WID</code>	<code>TIMER_TMR[n]_WID + TIMER_TMR[n]_DLY</code>	<code>TIMER_STAT_ILAT</code> (if enabled)	<code>TIMER_STAT_ILAT</code> (if enabled)
N/A	Anything	$== 0$	Anything	b#11 ^{*1}	Set
N/A	Anything including 0	≥ 1	$\leq 2^{32} - 1$	Unchanged	Unchanged
N/A	Anything including 0	≥ 1	$> 2^{32} - 1$	b#11	Set

*1 The timer does not run in this case.

Table 16-14: Overflow Event (On another error, such as $\text{DELAY} + \text{WIDTH} \geq 2^{32} - 1$)

	<code>TIMER_TMR[n]_DLY</code>		<code>TIMER_TMR[n]_WID + TIMER_TMR[n]_DLY</code>		<code>TIMER_STAT_ILAT</code> (if enabled)
Anything	Anything	Anything	Anything	b#01	Set

WIDCAP Mode

For WIDCAP mode, the `TIMER_TMR[n]_PER` and `TIMER_TMR[n]_WID` registers are read-only and the `TIMER_TMR[n]_DLY` register is not used. Therefore, no startup or rollover errors are possible.

Table 16-15: Overflow Event

<code>TIMER_TMR[n]_PER</code>	<code>TIMER_TMR[n]_DLY</code>	<code>TIMER_TMR[n]_WID</code>	<code>TIMER_TMR[n]_WID + TIMER_TMR[n]_DLY</code>	<code>TIMER_ERR_TYPE</code>	<code>TIMER_STAT_ILAT</code> (if enabled)
Anything	N/A	Anything	N/A	b#01	Set

EXTCLK Mode

Table 16-16: Startup Event

<code>TIMER_TMR[n]_PER</code>	<code>TIMER_TMR[n]_DLY</code>	<code>TIMER_TMR[n]_WID</code>	<code>TIMER_TMR[n]_WID + TIMER_TMR[n]_DLY</code>	<code>TIMER_ERR_TYPE</code>	<code>TIMER_STAT_ILAT</code> (if enabled)
$= 0$	N/A	N/A	N/A	b#01	Set

Table 16-16: Startup Event (Continued)

<code>TIMER_TMR[n]_PER</code>	<code>TIMER_TMR[n]_DLY</code>	<code>TIMER_TMR[n]_WID</code>	<code>TIMER_TMR[n]_WID + TIMER_TMR[n]_DLY</code>	<code>TIMER_ERR_TYPE</code>	<code>TIMER_STAT_ILAT</code> (if enabled)
≥ 1	N/A	N/A	N/A	Unchanged	Unchanged

Table 16-17: Rollover Event

<code>TIMER_TMR[n]_PER</code>	<code>TIMER_TMR[n]_DLY</code>	<code>TIMER_TMR[n]_WID</code>	<code>TIMER_TMR[n]_WID + TIMER_TMR[n]_DLY</code>	<code>TIMER_ERR_TYPE</code>	<code>TIMER_STAT_ILAT</code> (if enabled)
$= 0$	N/A	N/A	N/A	b#01	Set
≥ 1	N/A	N/A	N/A	Unchanged	Unchanged

Table 16-18: Overflow Event (On TMR_PER Register = 0 Only)

<code>TIMER_TMR[n]_PER</code>	<code>TIMER_TMR[n]_DLY</code>	<code>TIMER_TMR[n]_WID</code>	<code>TIMER_TMR[n]_WID + TIMER_TMR[n]_DLY</code>	<code>TIMER_ERR_TYPE</code>	<code>TIMER_STAT_ILAT</code> (if enabled)
Anything	N/A	N/A	N/A	b#01	Set

WATCHDOG Events

Table 16-19: Startup Event

<code>TIMER_TMR[n]_PER</code>	<code>TIMER_TMR[n]_DLY</code>	<code>TIMER_TMR[n]_WID</code>	<code>TIMER_TMR[n]_WID + TIMER_TMR[n]_DLY</code>	<code>TIMER_ERR_TYPE</code>	<code>TIMER_STAT_ILAT</code> (if enabled)
$\leq$ Allowed MIN ^{*1}	Anything < PERIOD	N/A	N/A	b#01	Set
> Allowed MIN	Anything < PERIOD	N/A	N/A	Unchanged	Unchanged
> Allowed MIN	Anything $\geq$ PERIOD	Refer to WATCHDOG Mode tables			

*1 Allowed MIN=0 for width watchdog mode. Allowed MIN=1 for period watchdog mode.

Table 16-20: Rollover Event

<code>TIMER_TMR[n]_PER</code>	<code>TIMER_TMR[n]_DLY</code>	<code>TIMER_TMR[n]_WID</code>	<code>TIMER_TMR[n]_WID + TIMER_TMR[n]_DLY</code>	<code>TIMER_ERR_TYPE</code>	<code>TIMER_STAT_ILAT</code> (if enabled)
$\leq$ Allowed MIN ^{*1}	Anything < PERIOD	N/A	N/A	b#01	Set

Table 16-20: Rollover Event (Continued)

TIMER_TMR[n]_PER	TIMER_TMR[n]_DLY	TIMER_TMR[n]_WID	TIMER_TMR[n]_WID + TIMER_TMR[n]_DLY	TIMER_ERR_TYPE	TIMER_STAT_ILAT (if enabled)
> Allowed MIN	Anything	N/A	N/A	Unchanged	Unchanged
> Allowed MIN	Anything $\geq$ PERIOD	Refer to WATCHDOG Mode tables			

*1 In case of graceful termination, this error is not generated.

Table 16-21: Overflow Event

TIMER_TMR[n]_PER	TIMER_TMR[n]_DLY	TIMER_TMR[n]_WID	TIMER_TMR[n]_WID + TIMER_TMR[n]_DLY	TIMER_ERR_TYPE	TIMER_STAT_ILAT (if enabled)
Anything	Anything	N/A	N/A	b#01	Set

ADSP-21560_21561_21564_21568 TIMER Register Descriptions

General-Purpose Timer Block (TIMER) contains the following registers.

Table 16-22: ADSP-21560_21561_21564_21568 TIMER Register List

Name	Description
TIMER_BCAST_DLY	Broadcast Delay Register
TIMER_BCAST_PER	Broadcast Period Register
TIMER_BCAST_WID	Broadcast Width Register
TIMER_DATA_ILAT	Data Interrupt Latch Register
TIMER_DATA_IMSK	Data Interrupt Mask Register
TIMER_ERR_TYPE	Error Type Status Register
TIMER_RUN	Run Register
TIMER_RUN_CLR	Run Clear Register
TIMER_RUN_SET	Run Set Register
TIMER_STAT_ILAT	Status Interrupt Latch Register
TIMER_STAT_IMSK	Status Interrupt Mask Register
TIMER_STOP_CFG	Stop Configuration Register
TIMER_STOP_CFG_CLR	Stop Configuration Clear Register
TIMER_STOP_CFG_SET	Stop Configuration Set Register
TIMER_TMR[n]_CFG	Timer n Configuration Register

Table 16-22: ADSP-21560_21561_21564_21568 TIMER Register List (Continued)

Name	Description
TIMER_TMR[n]_CNT	Timer n Counter Register
TIMER_TMR[n]_DLY	Timer n Delay Register
TIMER_TMR[n]_PER	Timer n Period Register
TIMER_TMR[n]_WID	Timer n Width Register
TIMER_TRG_IE	Trigger Receiver Enable Register
TIMER_TRG_MSK	Trigger Generator Mask Register

Broadcast Delay Register

For timers with `TIMER_TMR[n]_CFG.BDLYEN` enabled, a write to the `TIMER_BCAST_DLY` register concurrently updates the delay (`TIMER_TMR[n]_DLY`) registers of only those timers. A read of the `TIMER_BCAST_DLY` register returns 0x00000000, and no bus error is generated. To read back a written value, read that TMR's `TIMER_TMR[n]_DLY` register.

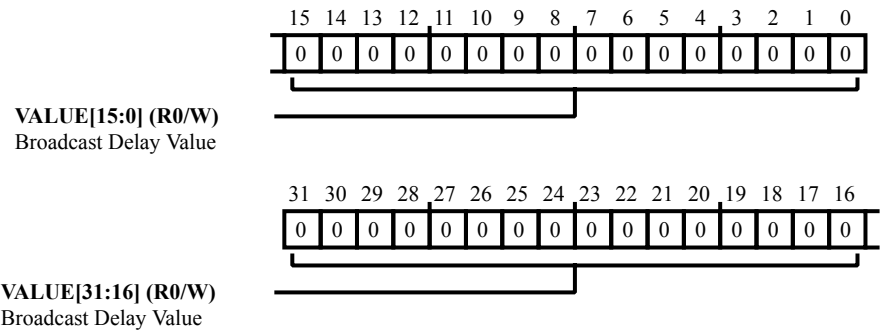


Figure 16-11: TIMER_BCAST_DLY Register Diagram

Table 16-23: TIMER_BCAST_DLY Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R0/W)	VALUE	Broadcast Delay Value. A write to the <code>TIMER_BCAST_DLY.VALUE</code> bit field concurrently updates the delay (<code>TIMER_TMR[n]_DLY</code>) registers of only those timers. A read of the <code>TIMER_BCAST_DLY.VALUE</code> bit field returns 0x0000 0000, and no bus error is generated.

Broadcast Period Register

For timers with `TIMER_TMR[n]_CFG.BPEREN` enabled, a write to the `TIMER_BCAST_PER` register concurrently updates the period (`TIMER_TMR[n]_PER`) registers of only those timers. A read of `TIMER_BCAST_PER` returns 0x00000000, and no bus error is generated. To read back a written value, read that TMR's `TIMER_TMR[n]_PER` register.

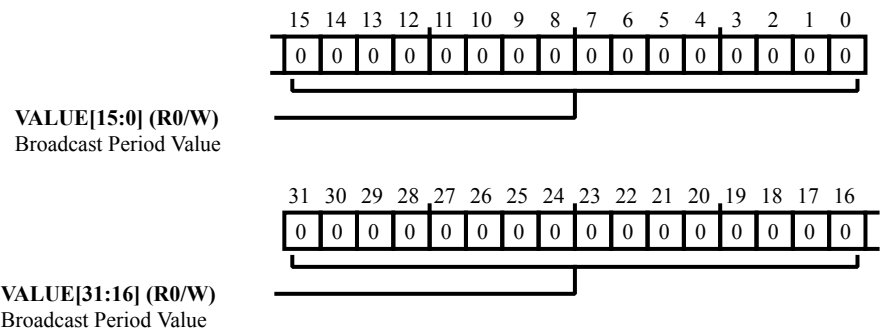


Figure 16-12: `TIMER_BCAST_PER` Register Diagram

Table 16-24: `TIMER_BCAST_PER` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R0/W)	VALUE	Broadcast Period Value. A write to the <code>TIMER_BCAST_PER.VALUE</code> bit field concurrently updates the period (<code>TIMER_TMR[n]_PER</code>) registers of only those timers. A read of the <code>TIMER_BCAST_PER.VALUE</code> bit fields returns 0x0000 0000, and no bus error is generated.

Broadcast Width Register

For timers with `TIMER_TMR[n]_CFG.BWIDEN` enabled, a write to the `TIMER_BCAST_WID` register concurrently updates the width (`TIMER_TMR[n]_WID`) registers of only those timers. A read of the `TIMER_BCAST_WID` register returns 0x00000000, and no bus error is generated. To read back a written value, read that TMR's `TIMER_TMR[n]_WID` register.

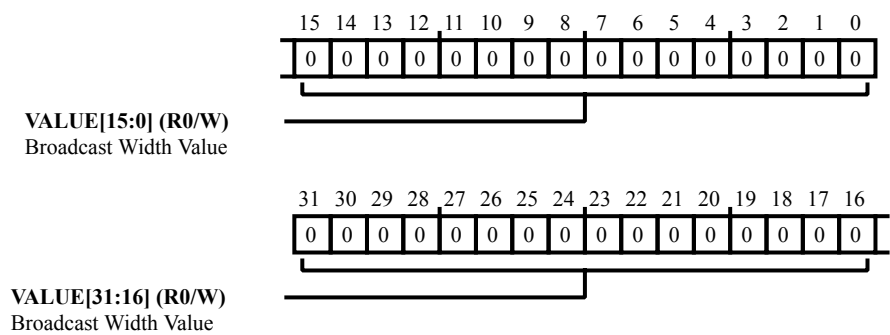


Figure 16-13: `TIMER_BCAST_WID` Register Diagram

Table 16-25: `TIMER_BCAST_WID` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R0/W)	VALUE	Broadcast Width Value. A write to the <code>TIMER_BCAST_WID.VALUE</code> bit field concurrently updates the width (<code>TIMER_TMR[n]_WID</code>) registers of only those timers. A read of the <code>TIMER_BCAST_WID.VALUE</code> bit field returns 0x0000 0000, and no bus error is generated.

Data Interrupt Latch Register

The `TIMER_DATA_ILAT` holds the latched interrupt status for interrupt requests that have been unmasked (enabled) by the `TIMER_DATA_IMSK` register and generated according to the conditions selected by the `TIMER_TMR[n]_CFG.IRQMODE` bits. If a bit in `TIMER_DATA_ILAT` is already set and the corresponding interrupt is masked in `TIMER_DATA_IMSK`, the latch holds its old value, leaving the interrupt request asserted until it is reset by software with a `W1C` operation.

Note that interrupt service routines (ISRs) should clear the appropriate bits in `TIMER_DATA_ILAT` before returning from the ISR, to ensure that the interrupt is not re-issued. To make sure that no timer event is missed, the latch should be reset at the very beginning of the ISR when in `EXTCLK` mode.

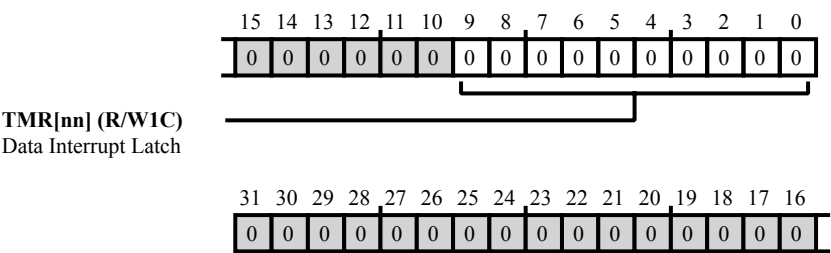


Figure 16-14: `TIMER_DATA_ILAT` Register Diagram

Table 16-26: `TIMER_DATA_ILAT` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
9:0 (R/W1C)	TMR[nn]	Data Interrupt Latch. For all <code>TIMER_DATA_ILAT.TMR[nn]</code> bits, status of =0 indicates no interrupt is latched, and status of =1 indicates a latched interrupt (indicating an unmasked interrupt request from a timer with a condition matching the one selected with corresponding <code>TIMER_TMR[n]_CFG.IRQMODE</code> bit has occurred).

Data Interrupt Mask Register

Each timer may generate a unique processor data interrupt request signal. The `TIMER_DATA_IMSK` register contains an interrupt mask for these requests, masking (disabling) or unmasking (enabling) the interrupts as programmed. The reset value of the `TIMER_DATA_IMSK` register is 0xFFFF, masking these interrupts after reset.

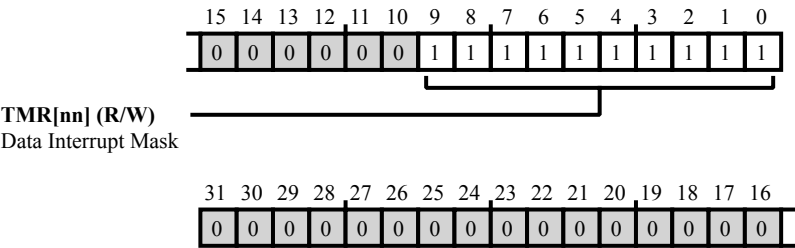


Figure 16-15: `TIMER_DATA_IMSK` Register Diagram

Table 16-27: `TIMER_DATA_IMSK` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
9:0 (R/W)	TMR[nn]	Data Interrupt Mask. For all <code>TIMER_DATA_IMSK.TMR[nn]</code> bits, write =0 unmask (enables) the corresponding data interrupt request, and write =1 masks (disables) the corresponding data interrupt request.

Error Type Status Register

The `TIMER_ERR_TYPE` register contains error type status bits for each timer. These bits indicate the type of error (if any) in a running timer. This register is read-only. These status bits are cleared at reset and when a particular timer is enabled.

Each time an error request interrupt is latched in the `TIMER_STAT_ILAT` register, the corresponding `TERRx` bits in the `TIMER_ERR_TYPE` register are loaded with a code that identifies the type of error that was detected. This status value is held until the next error or until a particular timer is restarted. No bus error is generated if a write is performed on the `TIMER_ERR_TYPE` register.

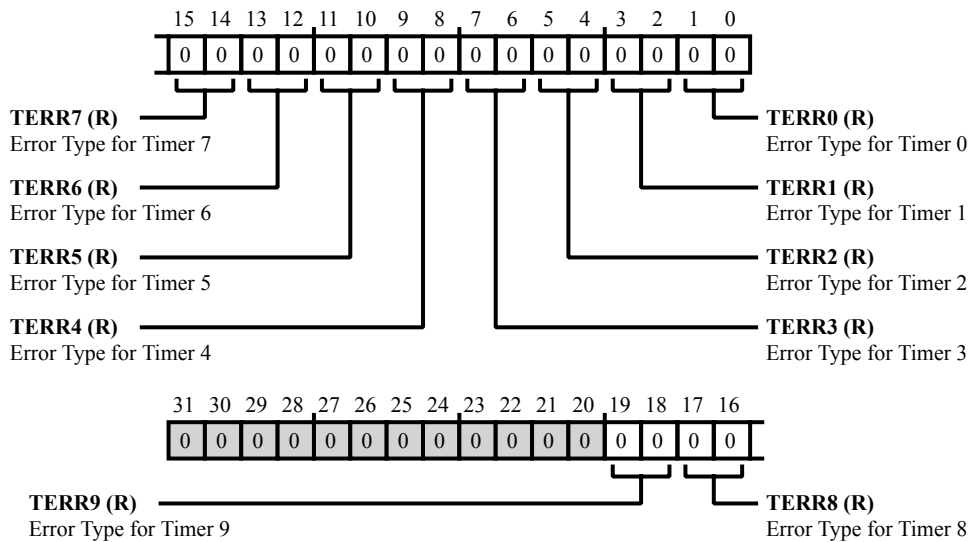


Figure 16-16: `TIMER_ERR_TYPE` Register Diagram

Table 16-28: `TIMER_ERR_TYPE` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
19:18 (R/NW)	TERR9	Error Type for Timer 9.
		0 No Error
		1 Counter Overflow Error
		2 PER Register Programming Error
		3 WID or DLY Register Programming Error
17:16 (R/NW)	TERR8	Error Type for Timer 8.
		0 No Error
		1 Counter Overflow Error
		2 PER Register Programming Error
		3 WID or DLY Register Programming Error

Table 16-28: TIMER_ERR_TYPE Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
15:14 (R/NW)	TERR7	Error Type for Timer 7.	
		0	No Error
		1	Counter Overflow Error
		2	PER Register Programming Error
		3	WID or DLY Register Programming Error
13:12 (R/NW)	TERR6	Error Type for Timer 6.	
		0	No Error
		1	Counter Overflow Error
		2	PER Register Programming Error
		3	WID or DLY Register Programming Error
11:10 (R/NW)	TERR5	Error Type for Timer 5.	
		0	No Error
		1	Counter Overflow Error
		2	PER Register Programming Error
		3	WID or DLY Register Programming Error
9:8 (R/NW)	TERR4	Error Type for Timer 4.	
		0	No Error
		1	Counter Overflow Error
		2	PER Register Programming Error
		3	WID or DLY Register Programming Error
7:6 (R/NW)	TERR3	Error Type for Timer 3.	
		0	No Error
		1	Counter Overflow Error
		2	PER Register Programming Error
		3	WID or DLY Register Programming Error
5:4 (R/NW)	TERR2	Error Type for Timer 2.	
		0	No Error
		1	Counter Overflow Error
		2	PER Register Programming Error
		3	WID or DLY Register Programming Error

Table 16-28: TIMER_ERR_TYPE Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
3:2 (R/NW)	TERR1	Error Type for Timer 1.	
		0	No Error
		1	Counter Overflow Error
		2	PER Register Programming Error
		3	WID or DLY Register Programming Error
1:0 (R/NW)	TERR0	Error Type for Timer 0.	
		0	No Error
		1	Counter Overflow Error
		2	PER Register Programming Error
		3	WID or DLY Register Programming Error

Run Register

The `TIMER_RUN` allows all timers to be enabled simultaneously, permitting them to run synchronously. For each timer, there is a single start/stop control bit. Writing a 1 to this bit starts the corresponding timer; writing a 0 stops the timer with mechanism specified in the timer stop configuration `TIMER_STOP_CFG` register.

The start/stop control bits can be set/reset individually or in any combination. While starting or stopping one particular timer directly with this register, software must perform a read-modify write, so the bits corresponding to other timers remain unchanged. To avoid this need, software can use the `TIMER_RUN_CLR` register.

Reading the `TIMER_RUN` register shows the start status for the corresponding timer. A 1 indicates that the timer is running.

If a timer is in run state (corresponding run bit is =1), a software write of 1 in this bit does not have any effect on the timer state. The write does not result in restarting the timer.

Note that the `TIMER_RUN` register is not used in PININT mode. PININT mode starts as soon as the `TIMER_TMR[n]_CFG.TMODE` bits are set to 111.

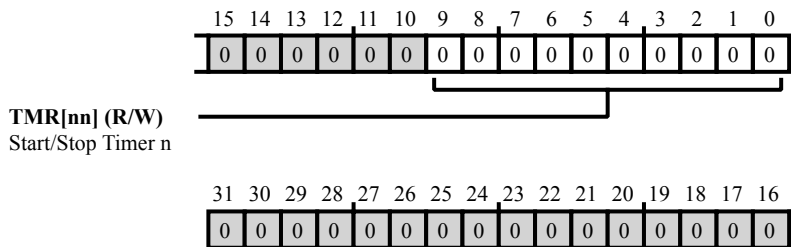


Figure 16-17: `TIMER_RUN` Register Diagram

Table 16-29: `TIMER_RUN` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
9:0 (R/W)	<code>TMR[nn]</code>	Start/Stop Timer n. For all <code>TIMER_RUN.TMR[nn]</code> bits, write =0 for stop, and write =1 for start. Read =1 when timer is running.

Run Clear Register

The `TIMER_RUN_CLR` register is an alias register, providing a mechanism to clear a specific start/stop bit in the `TIMER_RUN` register without affecting other bits in `TIMER_RUN`. To stop a particular timer, software must write a 1 into the corresponding `TIMER_RUN_CLR` bit. Writing a 0 has no effect. Because `TIMER_RUN_CLR` is a write-only register, the result of any write to this register must be checked by reading the `TIMER_RUN` register. A read of the `TIMER_RUN_CLR` returns 0x0000.

Note that the stopping mechanism of a timer may be abrupt or graceful (after completion of current waveform period) depending on the selection in the `TIMER_STOP_CFG` register.

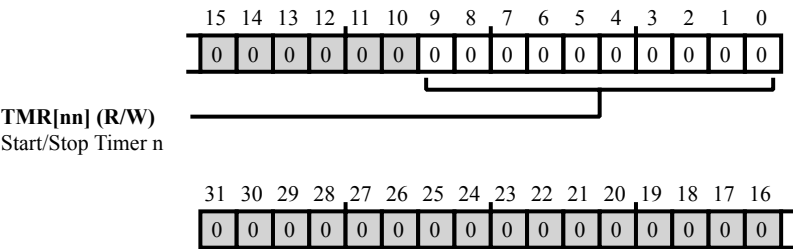


Figure 16-18: `TIMER_RUN_CLR` Register Diagram

Table 16-30: `TIMER_RUN_CLR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
9:0 (R0/W1C)	TMR[nn]	RUN Clear Alias. For all <code>TIMER_RUN_CLR.TMR[nn]</code> bits, write =0 has no effect, and write =1 for stop (clearing the corresponding in start/stop bit in the <code>TIMER_RUN</code> register). Using <code>TIMER_RUN_CLR</code> to clear start/stop bits permits stopping specific timers without influencing run status of other timers.

Run Set Register

The `TIMER_RUN_SET` register is an alias register, providing a mechanism to set a specific start/stop bit in the `TIMER_RUN` register without affecting other bits in `TIMER_RUN`. To start a particular timer, software must write a 1 into the corresponding `TIMER_RUN_SET` bit. Writing a zero has no effect. For an example, to start timer 3 without affecting any other timer, write 0x0008 into `TIMER_RUN_SET`. Because `TIMER_RUN_SET` is a write-only register, the result of any write to this register must be checked by reading the `TIMER_RUN` register. A read of the `TIMER_RUN_SET` returns 0x0000.

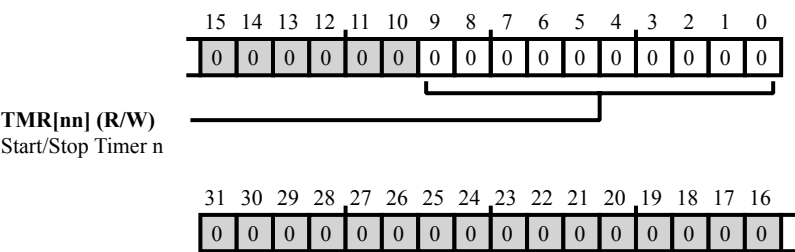


Figure 16-19: `TIMER_RUN_SET` Register Diagram

Table 16-31: `TIMER_RUN_SET` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
9:0 (R0/W1S)	TMR[nn]	RUN Set Alias. For all <code>TIMER_RUN_SET.TMR[nn]</code> bits, write =0 has no effect, and write =1 for start (setting the corresponding start/stop bit in the <code>TIMER_RUN</code> register). Using <code>TIMER_RUN_SET</code> to set start/stop bits permits starting specific timers without influencing the run status of other timers.

Status Interrupt Latch Register

The `TIMER_STAT_ILAT` holds the latched interrupt status for error interrupt requests, indicating a timer overflow condition or indicating that prohibited programming has occurred for a timer. These interrupt status bits are sticky and are W1C. The bits in the `TIMER_STAT_ILAT` register provide information regarding each timer interrupt request source.

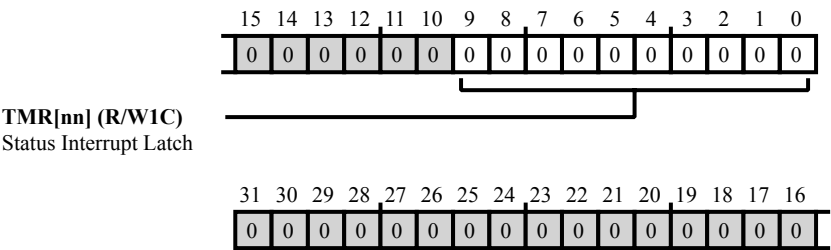


Figure 16-20: TIMER_STAT_ILAT Register Diagram

Table 16-32: TIMER_STAT_ILAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
9:0 (R/W1C)	TMR[nn]	Status Interrupt Latch. For all <code>TIMER_STAT_ILAT.TMR[nn]</code> bits, status of 0 indicates no error interrupt request is latched, and status of 1 indicates a timer counter overflow or programming error interrupt request is latched.

Status Interrupt Mask Register

While each timer may generate a status interrupt request, these requests are OR'ed to generate a single status interrupt signal to the system event controller. The `TIMER_STAT_IMSK` register contains an interrupt mask for these requests, masking (disabling) or unmasking (enabling) the interrupts as programmed. The reset value of the `TIMER_STAT_IMSK` register is 0xFFFF, masking these interrupts after reset.

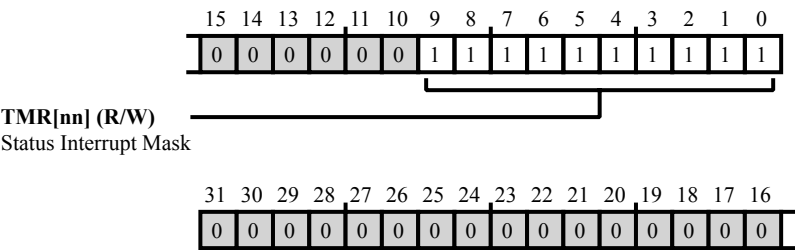


Figure 16-21: `TIMER_STAT_IMSK` Register Diagram

Table 16-33: `TIMER_STAT_IMSK` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
9:0 (R/W)	TMR[nn]	Status Interrupt Mask. For all <code>TIMER_STAT_IMSK.TMR[nn]</code> bits, write =0 unmask (enables) the corresponding status interrupt request, and write =1 masks (disables) the corresponding status interrupt request.

Stop Configuration Register

The `TIMER_STOP_CFG` register selects the stop mode for each timer. Timers may be stopped abruptly (immediate halt - all modes) or gracefully in PWMOUT modes (single pulse and continuous). The halt is achieved through either a write =0 to the corresponding bit in `TIMER_RUN` or a write =1 to the corresponding bit in `TIMER_RUN_CLR`. A read of `TIMER_STOP_CFG` returns the last value written.

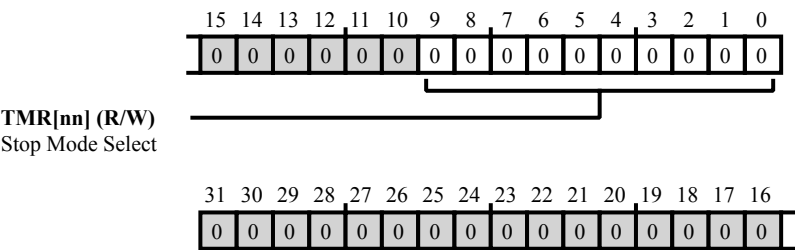


Figure 16-22: TIMER_STOP_CFG Register Diagram

Table 16-34: TIMER_STOP_CFG Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
9:0 (R/W)	TMR[nn]	Stop Mode Select. For all <code>TIMER_STOP_CFG.TMR[nn]</code> bits, write =0 for graceful termination (PWMOUT modes only), and write =1 for abrupt (immediate halt) on stop.

Stop Configuration Clear Register

This is an alias register, providing a mechanism to clear a specific bit in the `TIMER_STOP_CFG` register without affecting other bits in `TIMER_STOP_CFG`. To clear a bit in `TIMER_STOP_CFG`, software must write a 1 to the corresponding bit of `TIMER_STOP_CFG_CLR` register. Writing a zero has no effect. Because the `TIMER_STOP_CFG_CLR` register is a write-only register, the result of any write to this register must be checked by reading the `TIMER_STOP_CFG` register. A read of the `TIMER_STOP_CFG_CLR` register returns 0x0000.

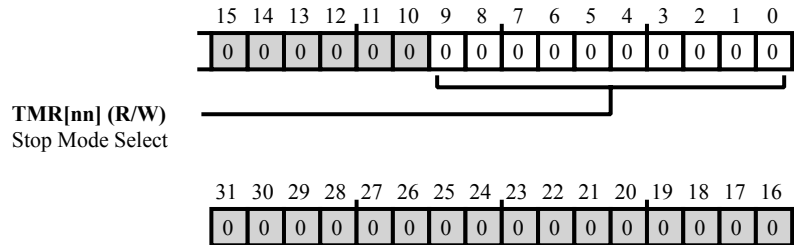


Figure 16-23: `TIMER_STOP_CFG_CLR` Register Diagram

Table 16-35: `TIMER_STOP_CFG_CLR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
9:0 (R0/W1C)	<code>TMR[nn]</code>	<code>STOP_CFG</code> Clear Alias. For all <code>TIMER_STOP_CFG_CLR.TMR[nn]</code> bits, write =0 has no effect, and write =1 for graceful stop in <code>PWMOUT</code> modes (clearing the corresponding stop mode select bit in the <code>TIMER_STOP_CFG</code> register). Using <code>TIMER_STOP_CFG_CLR</code> to clear stop mode bits permits configuring specific timers without influencing the stop mode configuration of other timers.

Stop Configuration Set Register

This is an alias register, providing a mechanism to set a specific bit in the `TIMER_STOP_CFG` register without affecting other bits in `TIMER_STOP_CFG`. To set a bit in the `TIMER_STOP_CFG` register, software must write a 1 to the corresponding bit of the `TIMER_STOP_CFG_SET` register. Writing a zero has no effect. Because the `TIMER_STOP_CFG_SET` register is a write-only register, the result of any write to this register must be checked by reading the `TIMER_STOP_CFG` register. A read of the `TIMER_STOP_CFG_SET` register returns 0x0000.

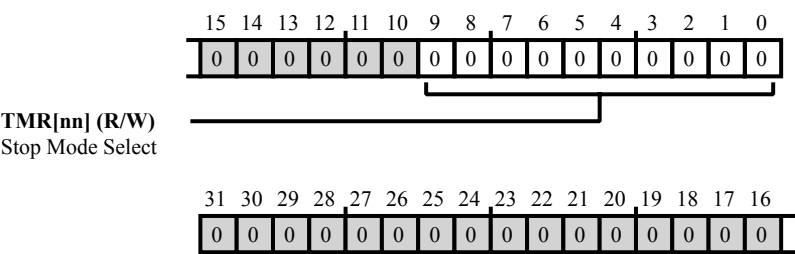


Figure 16-24: TIMER_STOP_CFG_SET Register Diagram

Table 16-36: TIMER_STOP_CFG_SET Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
9:0 (R0/W1S)	TMR[nn]	STOP_CFG Set Alias. For all <code>TIMER_STOP_CFG_SET.TMR[nn]</code> bits, write =0 has no effect, and write =1 for abrupt stop (setting the corresponding stop mode select bit in the <code>TIMER_STOP_CFG</code> register). Using <code>TIMER_STOP_CFG_SET</code> to set stop mode bits permits configuring specific timers without influencing the stop mode configuration of other timers.

Timer n Configuration Register

Each timer has a `TIMER_TMR[n]_CFG` register that specifies its operating mode. Only write to a `TIMER_TMR[n]_CFG` register when the corresponding timer is not running.

After disabling a timer operating in PWMOUT mode, verify that the timer has stopped running by checking the start/stop status of the timer in the `TIMER_RUN` register before writing to the timer's `TIMER_TMR[n]_CFG` register.

Note that a timer's `TIMER_TMR[n]_CFG` register may be read at any time.

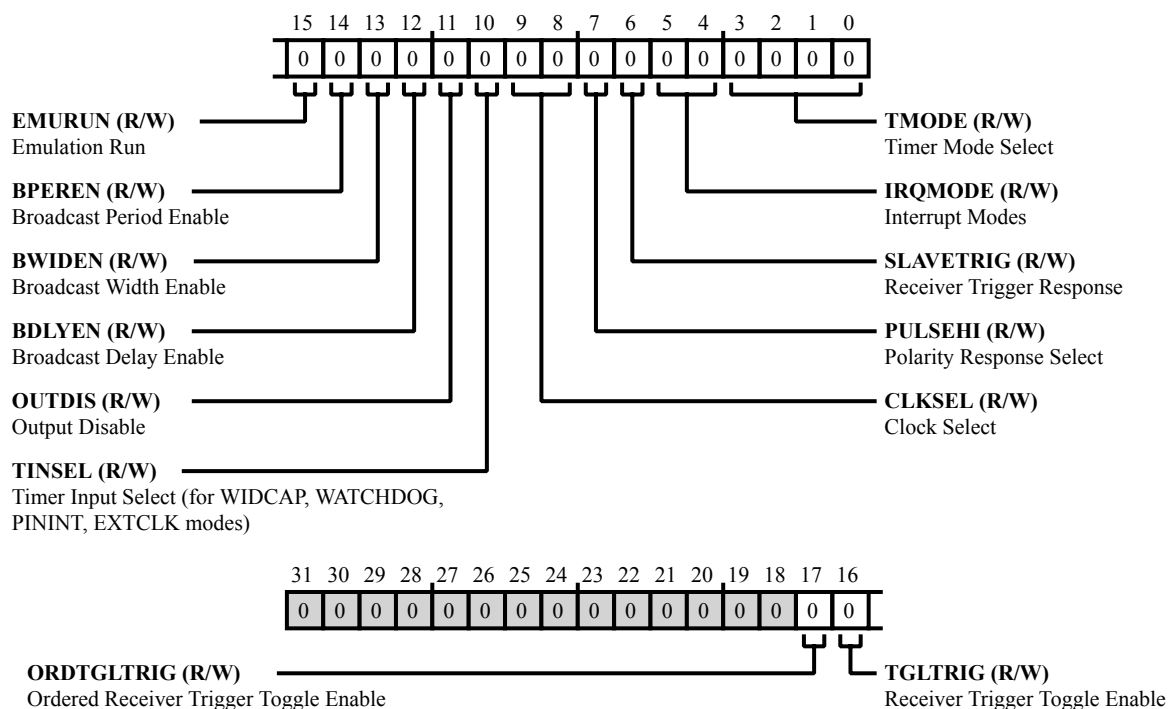


Figure 16-25: `TIMER_TMR[n]_CFG` Register Diagram

Table 16-37: TIMER_TMR[n]_CFG Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
17 (R/W)	ORDTGLTRIG	Ordered Receiver Trigger Toggle Enable. The <code>TIMER_TMR[n]_CFG.ORDTGLTRIG</code> bit controls ordered receive trigger toggle mode. The timer(receiver) state is not toggled by writes to the <code>TIMER_RUN</code> , <code>TIMER_RUN_SET</code> and <code>TIMER_RUN_CLR</code> registers when this bit is enabled.
		0 No effect
		1 Trigger pulse on trigger receiver 0 starts the timer if it is in the halt state. Trigger pulse in trigger receiver 1 stops the timer if it is running.
16 (R/W)	TGLTRIG	Receiver Trigger Toggle Enable. The <code>TIMER_TMR[n]_CFG.TGLTRIG</code> bit stops the timer if it is running and starts the timer if it is halted (in the stop state). If the <code>TIMER_TMR[n]_CFG.TGLTRIG</code> bit is set, then the setting of the <code>TIMER_TMR[n]_CFG.SLAVETRIG</code> bit is ignored.
		0 Receiver trigger response depends on SLAVETRIG bit
		1 Receiver trigger toggles timer state
15 (R/W)	EMURUN	Emulation Run. The <code>TIMER_TMR[n]_CFG.EMURUN</code> bit causes the timer to run (count) during emulation.
		0 Stop Timer During Emulation
		1 Run Timer During Emulation
14 (R/W)	BPEREN	Broadcast Period Enable. The <code>TIMER_TMR[n]_CFG.BPEREN</code> bit enables updates to the <code>TIMER_TMR[n]_PER</code> register simultaneously across more than one timer.
		0 Disable Broadcast to PER Register
		1 Enable Broadcast to PER Register
13 (R/W)	BWIDEN	Broadcast Width Enable. The <code>TIMER_TMR[n]_CFG.BWIDEN</code> bit enables updates to the <code>TIMER_TMR[n]_WID</code> register simultaneously across more than one timer.
		0 Disable Broadcast to WID Register
		1 Enable Broadcast to WID Register

Table 16-37: TIMER_TMR[n]_CFG Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
12 (R/W)	BDLYEN	Broadcast Delay Enable. The <code>TIMER_TMR[n]_CFG.BDLYEN</code> bit enables updates to the <code>TIMER_TMR[n]_DLY</code> register simultaneously across more than one timer.
		0 Disable Broadcast to DLY Register
		1 Enable Broadcast to DLY Register
11 (R/W)	OUTDIS	Output Disable. The <code>TIMER_TMR[n]_CFG.OUTDIS</code> bit enables or disables the timer pin output buffer.
		0 Enable TMR Pin Output Buffer
		1 Disable TMR Pin Output Buffer
10 (R/W)	TINSEL	Timer Input Select (for WIDCAP, WATCHDOG, PININT, EXTCLK modes).
		0 Use TMR Pin Input
		1 Use TMR Alternate Capture Input
9:8 (R/W)	CLKSEL	Clock Select. The <code>TIMER_TMR[n]_CFG.CLKSEL</code> bit field selects the TIMER clock to use.
		0 Use SCLK
		1 Use TMR_ALT_CLK0 as TMR Clock
		3 Use TMR_ALT_CLK1 as TMR Clock
7 (R/W)	PULSEHI	Polarity Response Select. The <code>TIMER_TMR[n]_CFG.PULSEHI</code> bit defines specific behaviors of the timer based on the operating mode. For more information, see the specific operating mode in the Programming Concepts section.
		0 Negative Response or Pulse. A Negative Edge Response or Negative Action Pulse on the TMR pin.
		1 Positive Response or Pulse. A Positive Edge Response or Positive Action Pulse on the TMR pin.
6 (R/W)	SLAVETRIG	Receiver Trigger Response. The <code>TIMER_TMR[n]_CFG.SLAVETRIG</code> bit controls the trigger response. The trigger pulse has no effect (to stop or start the timer) if the timer is already in the requested state.
		0 Pulse Stops Timer if it is Running
		1 Pulse Starts Timer if it is Stopped

Table 16-37: TIMER_TMR[n]_CFG Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
5:4 (R/W)	IRQMODE	Interrupt Modes. The <code>TIMER_TMR[n]_CFG.IRQMODE</code> bit field selects the interrupt request mode. Note that any mismatched combination of the <code>TIMER_TMR[n]_CFG.IRQMODE</code> and the <code>TIMER_TMR[n]_CFG.TMODE</code> bits results in no interrupt being generated. In WIDCAP modes, the position of the interrupt is controlled with the <code>TIMER_TMR[n]_CFG.TMODE</code> bit, and the <code>TIMER_TMR[n]_CFG.IRQMODE</code> bit is ignored. Whenever an interrupt is generated, a trigger generator pulse is also occurs, when enabled in the <code>TIMER_TRG_MSK</code> register.
		0 Active Edge Mode. The timer generates an interrupt at every active edge. The active edge polarity depends on the state of the <code>TIMER_TMR[n]_CFG.PULSEHI</code> bit. Valid for PININT mode only.
		1 Delay Expired Mode. The timer generates an interrupt when the <code>TIMER_TMR[n]_CNT</code> value reaches the value in the <code>TIMER_TMR[n]_DLY</code> register. This mode is valid for all PWMOUT modes.
		2 Width Plus Delay Expired Mode. The timer generates an interrupt when the <code>TIMER_TMR[n]_CNT</code> value reaches the value in the <code>TIMER_TMR[n]_WID</code> register plus the value in the <code>TIMER_TMR[n]_DLY</code> register. (PWMOUT modes only)
		3 Period Expired Mode. The timer generates an interrupt when the <code>TIMER_TMR[n]_CNT</code> value reaches the value in the <code>TIMER_TMR[n]_PER</code> register. (Continuous PWMOUT and EXTCLK modes only)
3:0 (R/W)	TMODE	Timer Mode Select. The <code>TIMER_TMR[n]_CFG.TMODE</code> bit field selects the operating mode of each timer.
		0-7 Idle Mode
		8 Period Watchdog Mode
		9 Width Watchdog Mode

Table 16-37: TIMER_TMR[n]_CFG Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
		10	Measurement Report at Asserting Edge of Waveform
		11	Measurement Report at Deasserting Edge of Waveform
		12	Continuous PWMOUT Mode
		13	Single Pulse PWMOUT Mode
		14	EXTCLK Mode
		15	PININT (pin interrupt) Mode

Timer n Counter Register

The `TIMER_TMR[n]_CNT` register holds the current timer count. After enabling, the count is re-initialized to either 0x0 or 0x1, depending on the configuration and mode. The `TIMER_TMR[n]_CNT` register is read-only and may be read at any time (whether the timer is running or stopped). Reading the `TIMER_TMR[n]_CNT` register returns an atomic 32-bit value.

Depending on the timer operation mode, the counter increment can be clocked by a number of sources, including SCLK, the TMR or alternate capture input pins, ACLK[n]. The counter retains its value after the timer is disabled.

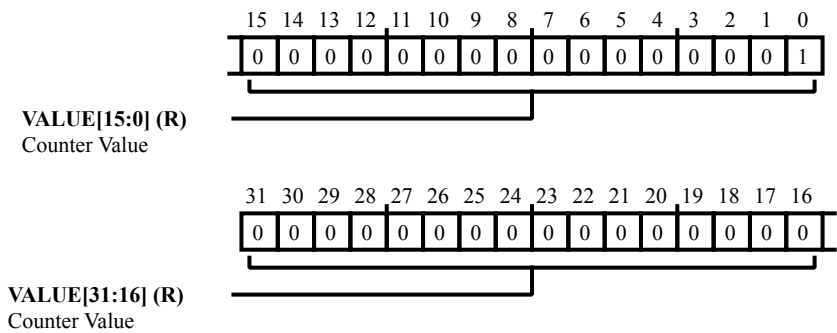


Figure 16-26: TIMER_TMR[n]_CNT Register Diagram

Table 16-38: TIMER_TMR[n]_CNT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	Counter Value. The <code>TIMER_TMR[n]_CNT.VALUE</code> bit field holds the current timer count.

Timer n Delay Register

The `TIMER_TMR[n]_DLY` register holds the delay value for the corresponding timer. This register's use is based on the selected timer mode.

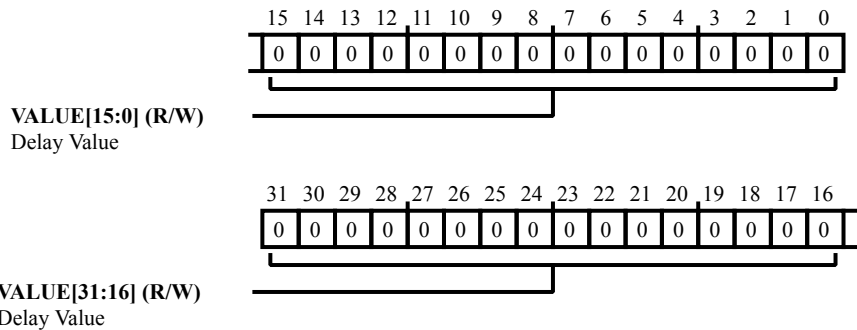


Figure 16-27: TIMER_TMR[n]_DLY Register Diagram

Table 16-39: TIMER_TMR[n]_DLY Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Delay Value. The <code>TIMER_TMR[n]_DLY.VALUE</code> bit field holds the delay value for the corresponding timer.

Timer n Period Register

The `TIMER_TMR[n]_PER` register holds the period value for the corresponding timer. This register's use is based on the selected timer mode.

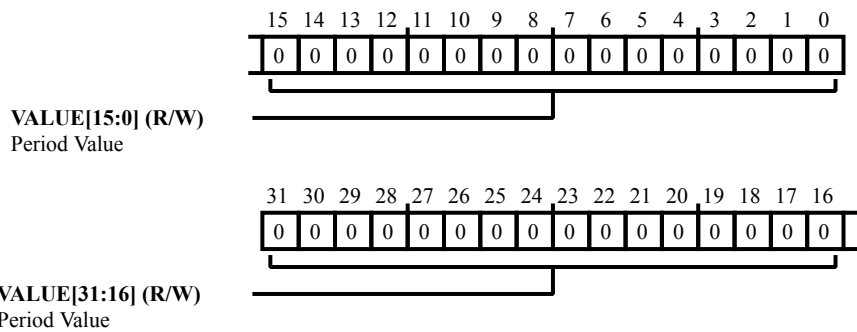


Figure 16-28: `TIMER_TMR[n]_PER` Register Diagram

Table 16-40: `TIMER_TMR[n]_PER` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Period Value. The <code>TIMER_TMR[n]_PER.VALUE</code> bit field holds the period value for the corresponding timer.

Timer n Width Register

The `TIMER_TMR[n]_WID` register holds the width value for the corresponding timer. This register's use is based on the selected timer mode.

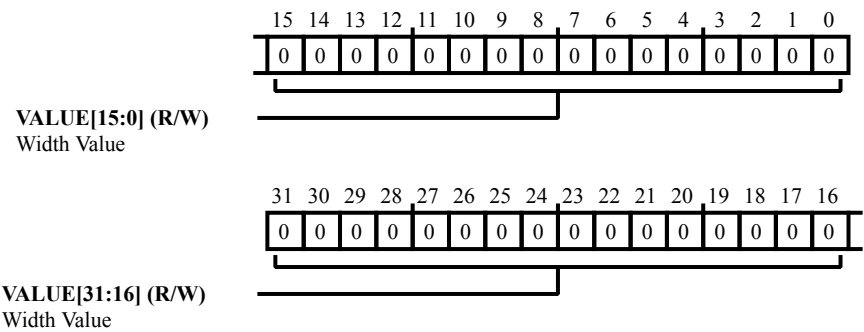


Figure 16-29: `TIMER_TMR[n]_WID` Register Diagram

Table 16-41: `TIMER_TMR[n]_WID` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Width Value. The <code>TIMER_TMR[n]_WID.VALUE</code> bit field holds the width value for the corresponding timer.

Trigger Receiver Enable Register

As a trigger receiver, each timer can generate a unique data trigger pulse signal. The `TIMER_TRG_IE` contains trigger input enable bits for these signals, disabling or enabling the triggers as programmed. The reset value of the `TIMER_TRG_IE` register is 0xFFFF, masking these triggers after reset.

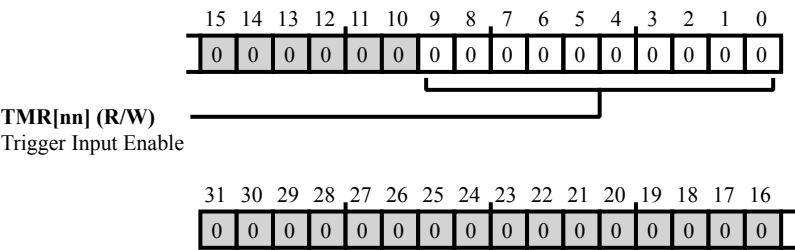


Figure 16-30: `TIMER_TRG_IE` Register Diagram

Table 16-42: `TIMER_TRG_IE` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
9:0 (R/W)	TMR[nn]	Trigger Input Enable. For all <code>TIMER_TRG_IE.TMR[nn]</code> bits, write =0 disables the corresponding trigger input, and write =1 enables the corresponding trigger input.

Trigger Generator Mask Register

As a trigger generator, each timer can generate a unique data trigger pulse signal. The `TIMER_TRG_MSK` register contains a trigger mask for these outputs, masking (disabling) or unmasking (enabling) the triggers as programmed. The reset value of the `TIMER_TRG_MSK` register is 0xFFFF, masking these triggers after reset.

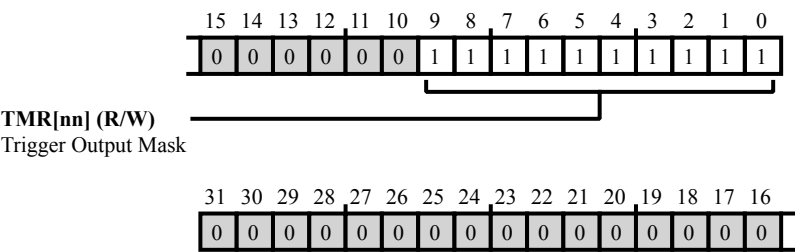


Figure 16-31: `TIMER_TRG_MSK` Register Diagram

Table 16-43: `TIMER_TRG_MSK` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
9:0 (R/W)	TMR[nn]	Trigger Output Mask. For all <code>TIMER_TRG_MSK.TMR[nn]</code> bits, write =0 unmask (enables) the corresponding data trigger output, and write =1 masks (disables) the corresponding data trigger output.

17 General-Purpose Counter (CNT)

The GP counter converts pulses from incremental position encoders into data that is representative of the actual position of the pulse. This conversion is done by integrating (counting) pulses on one or two inputs. Since integration provides relative position, some devices also feature a zero-position input (zero marker). The GP counter can use the zero position input feature to establish a reference point for verifying that the acquired position does not drift over time. In addition, the GP counter can use the incremental position information to determine speed, if the time intervals are measured.

The GP counter provides flexible ways to establish position information. When used with the GP timer block, the GP counter can allow for the acquisition of coherent position or time stamp information that enables speed calculation.

GP Counter Features

The GP counter includes the following features:

- 32-bit up or down counter
- Quadrature encode mode (Gray code)
- Binary encoder mode
- Alternative frequency-direction mode
- Timed direction and up or down counting modes
- Zero marker or push-button support
- Capture event timing in association with GP Timer
- Boundary comparison and boundary setting features

GP Counter Functional Description

The *GP Counter Block Diagram* shows a block diagram of the GP counter. The CNT_UD and CNT_DG pins accept various forms of incremental inputs. The 32-bit counter processes the inputs. The GP counter uses the CNT_ZM pin to sense the pressing of a push button.

NOTE: When enabled, the GP counter requires 3 SCLK cycles of initialization before recognizing valid toggles on its input pins.

The three input pins can be filtered (debounced) before the GP counter evaluates them.

The GP counter features a flexible boundary comparison. In all of the operating modes, the counter can be compared to an upper and lower limit. It takes various actions when reaching these limits.

The GP counter has a flexible input selection. Apart from accepting inputs from the CNT0_UD and CNT0_DG and PORT pins, the counter can be configured to accept trigger inputs by setting and bits. Refer to the register description in General-Purpose Ports (PORT) chapter for details.

The module can optionally generate an interrupt request to the system through its IRQ line. On many processors, a GP timer module can use an output to generate time stamps on certain events.

ADSP-21560_21561_21564_21568 CNT Register List

The GP Counter (CNT) provides support for manually controlled rotary controllers, such as the volume wheel on a radio device. This unit also supports industrial encoders.

The CNT converts pulses from incremental position encoders into data that is representative of the actual position. To complete this task, the CNT integrates (counts) pulses on one or two inputs. Because integration provides relative position, a zero position input (zero marker) is usually provided that establishes a reference point, verifying that the acquired position does not drift over time. The incremental position information may also be used to determine speed, if the relevant time intervals are measured. The CNT provides flexible ways to establish position information. When used in conjunction with the General-purpose Timer (TIMER), the CNT allows acquisition of coherent position/time stamp information, enabling speed calculation.

A set of registers govern CNT operations. For more information on CNT functionality, see the CNT register descriptions.

Table 17-1: ADSP-21560_21561_21564_21568 CNT Register List

Name	Description
CNT_CFG	Configuration Register
CNT_CMD	Command Register
CNT_CNTR	Counter Register
CNT_DEBNCE	Debounce Register
CNT_IMSK	Interrupt Mask Register
CNT_MAX	Maximum Count Register
CNT_MIN	Minimum Count Register
CNT_STAT	Status Register

ADSP-21560_21561_21564_21568 CNT Interrupt List

Table 17-2: ADSP-21560_21561_21564_21568 CNT Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
108	CNT0_STAT	CNT0 Status	Level	

ADSP-21560_21561_21564_21568 CNT Trigger List

Table 17-3: ADSP-21560_21561_21564_21568 CNT Trigger List Generators

Trigger ID	Name	Description	Sensitivity
16	CNT0_STAT	CNT0 Status	Level
98	CNT0_UD	CNT0 CNT0 Count Up and Direction	Level
99	CNT0_DG	CNT0 CNT0 Count Down and Gate	Level
100	CNT0_TO	CNT0 CNT0 Output to Timer Block	Level

Table 17-4: ADSP-21560_21561_21564_21568 CNT Trigger List Receivers

Trigger ID	Name	Description	Sensitivity
114	CNT0_UD	CNT0 CNT0 Count Up and Direction Input	Pulse
115	CNT0_DG	CNT0 CNT0 Count Down and Gate Input	Pulse

GP Counter Operating Modes

The GP counter has the following modes of operation.

1. Quadrature Encoder
2. Binary Encoder
3. Up/Down Counter
4. Direction Counter
5. Timed Direction

With the exception of the timed direction mode, the GP counter can operate with the GP timer block to capture additional timing information (time stamps) associated with events detected by this block.

Quadrature Encoder Mode

In this mode, the CNT_UD and CNT_DG inputs expect a quadrature-encoded signal that is interpreted as a two-bit Gray code. The order of transitions of the CNT_UD and CNT_DG inputs determines whether the counter increments or decrements. The CNT_CNTR register contains the number of transitions that have occurred as shown in the *Quadrature Events and Counting Mechanism* table. Optionally, an interrupt is generated when both inputs

change within one SCLK cycle. Gray coding prohibits such transitions. Therefore, the `CNT_CNTR` register remains unchanged, and an error condition is signaled.

Table 17-5: Quadrature Events and Counting Mechanism

CNT_COUNTER Register Value	-4	-3	-2	-1	0	+1	+2	+3	+4
CDG, CUD Inputs	00	01	11	10	00	01	11	10	00

It is possible to reverse the count direction of the Gray-coded signal by enabling the polarity inverter of either the `CNT_UD` pin or the `CNT_DG` pin. Inverting both pins does not alter the behavior. The GP counter can enable this feature with the `CNT_CFG.CDGINV` and `CNT_CFG.CUDINV` bits.

As an example, the `CNT_DG` and `CNT_UD` inputs are 00 and the next transition is to 01. These inputs normally change the counter in increments as shown in the table. If the `CNT_UD` polarity is inverted, this condition generates a received input of 01 followed by 00. The results is a decrement of the counter, altering the behavior of the connected hardware.

Binary Encoder Mode

This mode is almost identical to quadrature encoder mode, with the exception that the `CNT_UD`: `CNT_DG` inputs expect a binary-encoded signal. The order of transitions of the `CNT_UD` and `CNT_DG` inputs determines whether the counter increments or decrements. The `CNT_CNTR` register contains the number of transitions that have occurred as shown in the *Binary Events and Counting Mechanism* table. Optionally, an interrupt is generated when the detected code steps by more than 1 (in binary arithmetic) within one SCLK cycle. Such transitions are erroneous. Therefore, the `CNT_CNTR` register remains unchanged, and an error condition is signaled.

Table 17-6: Binary Events and Counting Mechanism

CNT_COUNTER Register Value	-4	-3	-2	-1	0	+1	+2	+3	+4
CDG:CUD Inputs	00	01	10	11	00	01	10	11	00

Reversing the `CNT_UD` and `CNT_DG` pin polarity has a different effect in binary encoder mode than for the quadrature encoder mode. Inverting the polarity of the `CNT_UD` pin only or inverting both the `CNT_UD` and `CNT_DG` pins, results in reversing the count direction.

Up/Down Counter Mode

In this mode, the counter increments or decrements at every active edge of the input pins. The GP counter uses the `CNT_CFG.CUDINV` bit to select an active edge and has the following results.

- If the GP counter module detects an active edge at the `CNT_UD` input, the counter increments.
- If the GP counter module detects an active edge at the `CNT_DG` input, the counter decrements.
- If simultaneous edges occur on the `CNT_DG` and `CNT_UD` pins, the counter remains unchanged, and both up-count and down-count events are signaled in the `CNT_STAT` register.

Direction Counter Mode

In this mode, the counter is incremented or decremented at every active edge of the `CNT_DG` input pin. The state of the `CNT_UD` input determines whether the counter increments or decrements. The GP counter uses the `CNT_CFG.CUDINV` bit to select the polarity.

If the GP counter detects an active edge at the `CNT_DG` input, the counter value changes by one in the selected direction.

Timed Direction Mode

In this mode, the counter is incremented or decremented at each `SCLK` cycle. The state of the `CNT_UD` input determines whether the counter increments or decrements. The GP counter uses the `CNT_CFG.CUDINV` bit to select the polarity. The `CNT_DG` pin can be used to gate the clock. The GP counter uses the `CNT_CFG.CDGINV` bit to select the polarity.

GP Counter Programming Model

The following sections provide information for programming the interface.

GP Counter General Programming Flow

The following are general guidelines for configuring and enabling the GP counter.

1. Initialize (but do not enable) the GP counter for the desired mode and settings through the `CNT_CFG` register.
2. Usually, events of interest are processed using interrupts rather than by polling status bits. In this case, clear all status bits and activate the interrupt generation requests with the `CNT_IMSK` register.
3. Configure interrupts at the system level to insure desired interrupt signaling to the system.
4. If timing information is required, set up the relevant GP Timer in width capture mode.
5. Finally, enable interrupt requests and the GP Counter itself using the `CNT_IMSK` and `CNT_CFG` registers, respectively.

GP Counter Mode Configuration

The GP counter can use the `CNT_ZM` input pin to sense the zero marker output of a rotary device or to detect the pressing of a push button. There are four programming schemes, which are functional in all counter modes:

- Push-button mode
- Zero-marker-zeros-counter mode
- Zero-marker-error mode
- Zero-once mode

Configuring GP Counter Push-Button Operation

Use the following procedure to configure push-button operation:

1. Set `CNT_IMSK.CZM` to enable (unmask) the zero marker interrupt.
2. Select the active edge polarity through the `CNT_CFG.CZMINV` bit.
3. Proceed with any other desired configuration steps and enable the peripheral.

An active edge at the `CNT_ZM` input sets the `CNT_IMSK.CZM` bit.

Configuring Zero-Marker-Zeros-Counter Mode

The following provides information on configuring zero-marker-zeros-counter mode for the GP counter.

1. Set `CNT_IMSK.CZMZ` to enable `CNT_CNTR`. The zero marker interrupt zeroes the counter.
2. Set `CNT_CFG.ZMZC` to enable ZMZC mode.
3. Select the active edge polarity through the `CNT_CFG.CZMINV` bit.
4. Proceed with any other desired configuration steps and enable the peripheral.

This configuration causes an active level at the `CNT_ZM` pin to clear the `CNT_CNTR` register and keep it cleared until the `CNT_ZM` pin is deactivated. In addition, the `CNT_STAT.CZMZ` bit is set.

Configuring Zero-Marker-Error Mode

The GP counter uses this mode to detect discrepancies between counter-value and the zero marker output of certain rotary encoder devices.

1. Set the `CNT_STAT.CZME` bit to enable this mode.
2. Select the active edge of the `CNT_ZM` pin through the `CNT_CFG.CZMINV` bit.
3. Proceed with any other desired configuration steps and enable the peripheral.

When the GP counter detects an active edge at the `CNT_ZM` input pin, it compares the four LSBs of the `CNT_CNTR` register to zero. If they are not zero, the GP counter uses `CNT_STAT.CZME` bit to signal a mismatch.

Configuring Zero-Once Mode

The GP counter uses this mode to perform an initial reset of the counter-value when it detects an active zero marker. After that, the zero marker is ignored (the counter is no longer reset).

1. Set the `CNT_CMD.W1ZMONCE` bit to enable this mode.
2. Select the active edge of the `CNT_ZM` pin through the `CNT_CFG.CZMINV` bit.
3. Ensure that at least one of the following bits is enabled: `CNT_IMSK.CZM`, `CNT_IMSK.CZME`, `CNT_IMSK.CZMZ`.

4. Proceed with any other desired configuration steps and enable the peripheral.

The `CNT_CNTR` register and the `CNT_CMD.W1ZMONCE` bit are cleared on the next active edge of the `CNT_ZM` pin. Now the `CNT_CMD.W1ZMONCE` bit can be read to check whether the event has already occurred.

Configuring Boundary Auto-Extend Mode

In this mode, hardware modifies the boundary registers (`CNT_MIN` and `CNT_MAX`) whenever the `CNT_CNTR` value reaches either of them. The GP counter uses this mode to monitor the widest angle a thumb wheel even if the software did not generate interrupts.

1. Initialize `CNT_CNTR` with the desired value.
2. Set both `CNT_MIN` and `CNT_MAX` to this same value.
3. Configure the `CNT_CFG.BNDMODE` field for auto extend mode.
4. Proceed with any other desired configuration steps and enable the peripheral.

The `CNT_MAX` register is loaded with the current `CNT_CNTR` value when the latter increments beyond the `CNT_MAX` value. Similarly, the `CNT_MIN` register is loaded with the `CNT_CNTR` value when the latter decrements below the `CNT_MIN` value. The `CNT_STAT.MAXC` and `CNT_STAT.MINC` status bits are set when the `CNT_CNTR` value matches the respective boundary register value.

Configuring Boundary Capture Mode

In this mode, the `CNT_CNTR` value is latched into the `CNT_MIN` register at one detected edge of the `CNT_ZM` input pin and latched into the `CNT_MAX` boundary register at the opposite edge.

1. To capture the `CNT_ZM` pin rising edge into `CNT_MIN` and the falling edge into `CNT_MAX`, program `CNT_CFG.CZMINV` for active high polarity. Conversely, to capture the `CNT_ZM` pin falling edge into `CNT_MIN` and the rising edge into `CNT_MAX`, program `CNT_CFG.CZMINV` for active low polarity.
2. Program the `CNT_IMSK.MAXC` and `CNT_IMSK.MINC` interrupt mask bits according to interrupt generation requirements.
3. Configure the `CNT_CFG.BNDMODE` field for boundary capture mode.
4. Proceed with any other desired configuration steps and enable the peripheral.

The `CNT_STAT.MAXC` and `CNT_STAT.MINC` status bits report the capture event, depending on how interrupt masks are configured.

Configuring Boundary Compare and Boundary Zero Modes

In these modes, the two boundary registers (`CNT_MAX` and `CNT_MIN`) are compared to the value in the `CNT_CNTR` register.

1. Program `CNT_MAX` and `CNT_MIN` registers with appropriate upper and lower range values.

2. Program the `CNT_IMSK.MAXC` and `CNT_IMSK.MINC` interrupt mask bits according to interrupt generation requirements.
3. Configure the `CNT_CFG.BNDMODE` field for boundary compare mode.
4. Proceed with any other desired configuration steps and enable the peripheral.

If after incrementing, `CNT_CNTR = CNT_MAX`, then the `CNT_STAT.MAXC` bit is set. Similarly if after decrementing, `CNT_CNTR = CNT_MIN`, then the `CNT_STAT.MINC` bit is set.

Additionally, for boundary zero mode, the counter-value in `CNT_CNTR` is set to zero. The `CNT_STAT.MAXC` and `CNT_STAT.MINC` bits are not set when software updates the `CNT_MAX` or `CNT_MIN` registers.

Configuring GP Counter Push-Button Operation

Use the following procedure to configure push-button operation:

1. Set `CNT_IMSK.CZM` to enable (unmask) the zero marker interrupt.
2. Select the active edge polarity through the `CNT_CFG.CZMINV` bit.
3. Proceed with any other desired configuration steps and enable the peripheral.

An active edge at the `CNT_ZM` input sets the `CNT_IMSK.CZM` bit.

GP Counter Programming Concepts

Using the features, operating modes, and event control for the GP counter to their greatest potential requires an understanding of some GP counter-related concepts. Some key aspects to consider are input noise filtering and capturing timing information.

CNT Input Noise Filtering

In all modes, the three input pins can be filtered to present clean signals to the GP counter logic. The GP counter uses the `CNT_CFG.DEBEN` bit to enable or disable this filtering. The *Programmable Noise Filtering* figure shows the filtering operation for the `CNT_UD` pin.

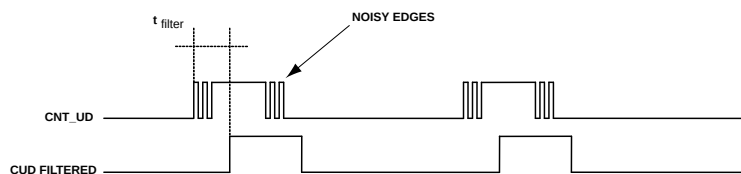


Figure 17-1: Programmable Noise Filtering

The CNT module implements the filtering mechanism using counters for each GP counter pin, where each counter is initialized from the `CNT_DEBNCE.DPRESCALE` field. When a transition is detected on a pin, the corresponding counter starts counting up to the programmed number of SCLK cycles. The state of the pin is latched after time t_{filter} and passed on to the GP counter logic.

The following formula determines the time t_{filter} , given SCLK and the CNT_DEBNCE.DPRESCALE value, where lower values of CNT_DEBNCE.DPRESCALE result in shorter debounce delays:

$$t_{\text{filter}} = 128 \times (2^{\text{DPPRESCALE}} \times \text{SCLK})$$

Capturing Counter Interval and CNT_CNTR Read Timing

When the count speed is low, it is often useful to capture the time elapsed since the last count event. Program the `TIMER TMR[n] CFG` register of the associated GP timer in a width capture mode with the following bit settings.

- `TIMER_TMR[n]_CFG.PULSEHI = 0`
- `TIMER_TMR[n]_CFG.TMODE = b#1011`
- `TIMER_TMR[n]_CFG.TINSEL = 1`

The *Capture Intervals* figure shows and the following list describes the operation of the GP counter and the GP timer in this mode.

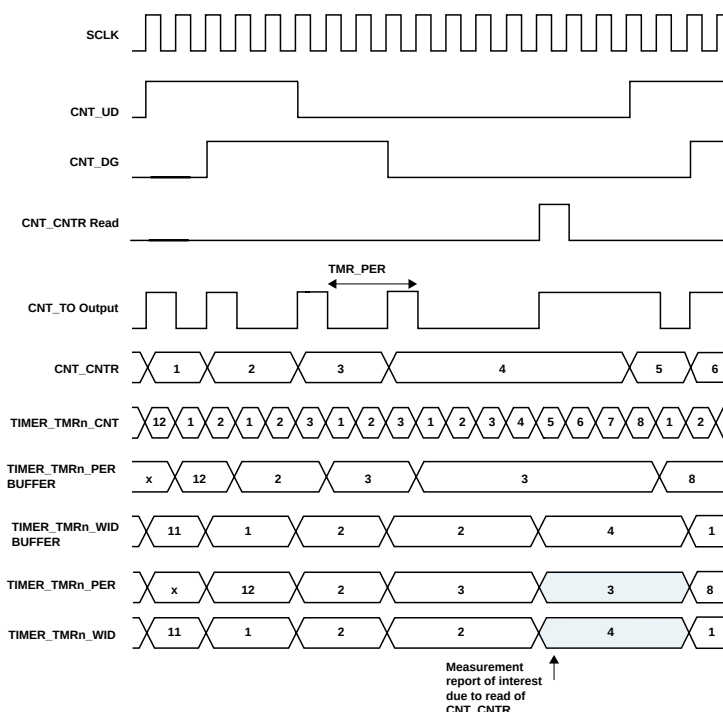


Figure 17-2: Capture Intervals

NOTE: SCLK in the *Capture Intervals* figure is SCLK.

1. The CNT_TO signal generates a pulse every time a count event occurs. In addition, when the processor reads the CNT_CNTR register, the CNT_TO signal presents a pulse which is extended (high) until the next count event.
2. The GP timer updates its TIMER_TMR[n]_PER register with the period (measured from falling edge to falling edge, because TIMER_TMR[n]_CFG.PULSEHI = 0) of the CNT_TO signal.

3. The `TIMER_TMR[n]_WID` register is updated with the pulse width (the portion where `CNT_TO` is low, again because `TIMER_TMR[n]_CFG.PULSEHI = 0`).
4. Both registers are updated at every rising edge of the `CNT_TO` signal (because `TIMER_TMR[n]_CFG.TMODE = b#011`).

The `TIMER_TMR[n]_PER` register contains the period between the last two count events. The `TIMER_TMR[n]_WID` register contains the time since the last count event and the read of the `CNT_CNTR` register, both measured in SCLK cycles.

Read the `CNT_CNTR` register to latch the two time measurements, providing a coherent triplet of information to calculate speed and position.

NOTE: Speed restrictions apply to the use of the `CNT_TO` signal. Therefore, programs must not operate at count event rates that are high. For instance, if `CNT_CNTR` is incremented or decremented every SCLK cycle (timed direction mode), the `CNT_TO` signal is not valid.

Capturing Time Interval Between Successive Counter Events

When the required timing information is the interval between successive count events, program the associated timer in a width capture mode. Set the `TIMER_TMR[n]_CFG` bit of `TIMER_TMR[n]_CFG.PULSEHI = 1`, `TIMER_TMR[n]_CFG.TMODE = b#1010` and `TIMER_TMR[n]_CFG.TINSEL = 1`. Typically, this information is sufficient if the speed of GP counter events does not reach low values.

The *Period Register Timing* figure shows the operation of the GP counter and the GP timer in this mode.

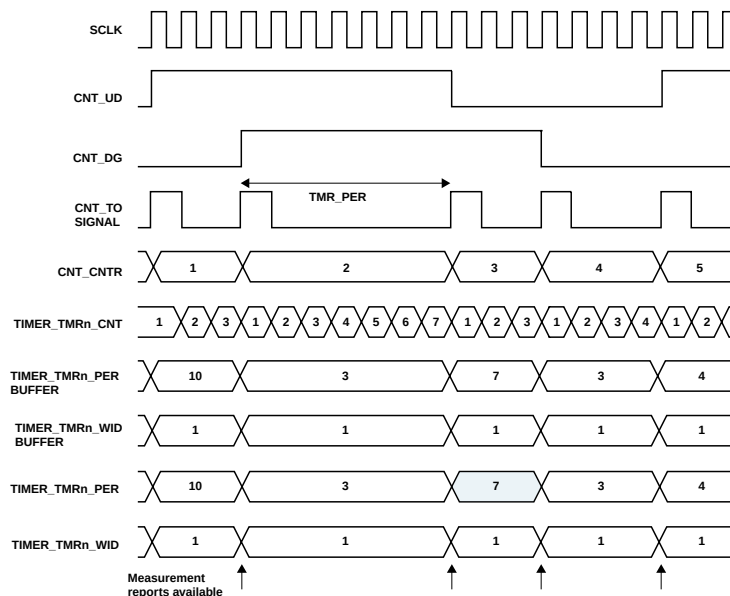


Figure 17-3: Period Register Timing

The `CNT_TO` signal generates a pulse every time a count event occurs. The GP timer updates its `TIMER_TMR[n]_PER` register with the period (measured from rising edge to rising edge) of the `CNT_TO` signal.

The `TIMER_TMR[n]_PER` register is updated at every rising edge of the `CNT_TO` signal and contains the number of SCLK cycles that have elapsed since the previous rising edge.

Incidentally, the `TIMER_TMR[n]_WID` register is also updated at the same time, but is of no interest in this mode of operation. If no reads of the `CNT_CNTR` register occur between counter events, the `TIMER_TMR[n]_WID` register only contains the width of the `CNT_TO` pulse. If a read of `CNT_CNTR` has occurred between events, the `TIMER_TMR[n]_WID` register contains the time between the read of `CNT_CNTR` and the next event.

This mode can also be used with `TIMER_TMR[n]_CFG.PULSEHI = 0`. In this case, the period of `CNT_TO` is measured between falling edges. It results in the same values as in the previous case, only the latching occurs one SCLK cycle later.

GP Counter Event Control

Eleven events can be signaled to the processor using status information and optional interrupt requests. The GP counter uses the respective bits in the `CNT_IMSK` register to enable the interrupt requests. It uses dedicated bits in the `CNT_STAT` register to report events. When an interrupt request from the GP counter is serviced, the application software is responsible for correct interpretation of the events. It is recommended to logically AND the content of the `CNT_IMSK` and `CNT_STAT` registers to identify pending interrupt requests.

Perform a write-one-to-clear (W1C) operation to the `CNT_STAT` register to clear the interrupt requests. Hardware does not clear the status bits automatically unless the counter module is disabled.

The following sections describe the events associated with the GP counter.

Illegal Gray and Binary Code Events

When illegal transitions occur in quadrature encoder or binary encoder modes, the `CNT_STAT.IC` bit is set. If enabled by the `CNT_STAT.IC` bit, the counter module generates an interrupt request. Set the `CNT_STAT.IC` bit only in the quadrature encoder or binary encoder modes.

Up/Down Count Events

The GP counter uses the `CNT_STAT.UC` bit to indicate whether the counter has been incremented. Similarly, the `CNT_STAT.DC` bit reports decrements. The two events are independent. For instance, if the counter increments by one and then decrements by two, both bits remain set, even though the resulting counter-value shows a decrement by one.

In up/down counter mode, hardware can detect simultaneous active edges on the `CNT_UD` and `CNT_DG` inputs. In that case, the `CNT_CNTR` remains unchanged, but both the `CNT_STAT.UC` and `CNT_STAT.DC` bits are set. Interrupt requests for these events can be enabled through the `CNT_IMSK.UC` and `CNT_IMSK.DC` bits. Use this feature carefully when the counter is clocked at high rates. This suggestion is especially critical when the counter operates in `DIR_TMR` mode, as interrupts are generated every SCLK cycle.

These events can also be used for more push buttons, when GP counter features are unnecessary. When up/down counter mode is enabled, the GP counter can use these count events to report interrupts from push buttons that connect to the CNT_UD and CNT_DG inputs.

Zero-Count Events

The CNT_STAT.CZERO status bit indicates that the CNT_CNTR has reached a value equal to 0x0000 0000 after an increment or decrement. This bit is not set when the counter value is set to zero by a write to CNT_CNTR or by setting the CNT_CMD.W1LCNTZERO bit. If enabled by the CNT_IMSK.CZERO bit, the GP counter module generates an interrupt request.

Overflow Events

There are two status bits that indicate whether the signed counter-register has overflowed from a positive to a negative value or conversely. The CNT_STAT.COV31 bit reports that the 32-bit CNT_CNTR register has either incremented from 0x7FFF FFFF to 0x8000 0000, or decremented from 0x8000 0000 to 0x7FFF FFFF.

If enabled by the CNT_IMSK.COV31 bit, an interrupt request is generated. Similarly, in applications where only the lower 16 bits of the counter are of interest, the CNT_STAT.COV15 status bit reports counter transitions from 0xFFFF 7FFF to 0xFFFF 8000, or from 0xFFFF 8000 to 0xFFFF 7FFF. If enabled by the CNT_IMSK.COV15 bit, an interrupt request is generated.

Boundary Match Events

The CNT_STAT.MINC and CNT_STAT.MAXC status bits report boundary events as described in [Configuring Boundary Capture Mode](#). These bits are not set if the software updates the CNT_CNTR, CNT_MAX, or CNT_MIN registers or writes to the CNT_CMD register. The CNT_IMSK.MINC and CNT_IMSK.MAXC bits enable interrupt request generation on boundary events.

Zero Marker Events

The CNT_STAT.CZM, CNT_STAT.CZME, and CNT_STAT.CZMZ bits are associated with zero marker events, as described in [Configuring GP Counter Push-Button Operation](#). Each of these events can optionally generate an interrupt request, when enabled by the corresponding CNT_IMSK.CZM, CNT_IMSK.CZME and CNT_IMSK.CZMZ bits.

ADSP-21560_21561_21564_21568 CNT Register Descriptions

General-Purpose Counter (CNT) contains the following registers.

Table 17-7: ADSP-21560_21561_21564_21568 CNT Register List

Name	Description
CNT_CFG	Configuration Register
CNT_CMD	Command Register

Table 17-7: ADSP-21560_21561_21564_21568 CNT Register List (Continued)

Name	Description
CNT_CNTR	Counter Register
CNT_DEBNCE	Debounce Register
CNT_IMSK	Interrupt Mask Register
CNT_MAX	Maximum Count Register
CNT_MIN	Minimum Count Register
CNT_STAT	Status Register

Configuration Register

The `CNT_CFG` register configures counter modes, configures input pins, and enables the CNT.

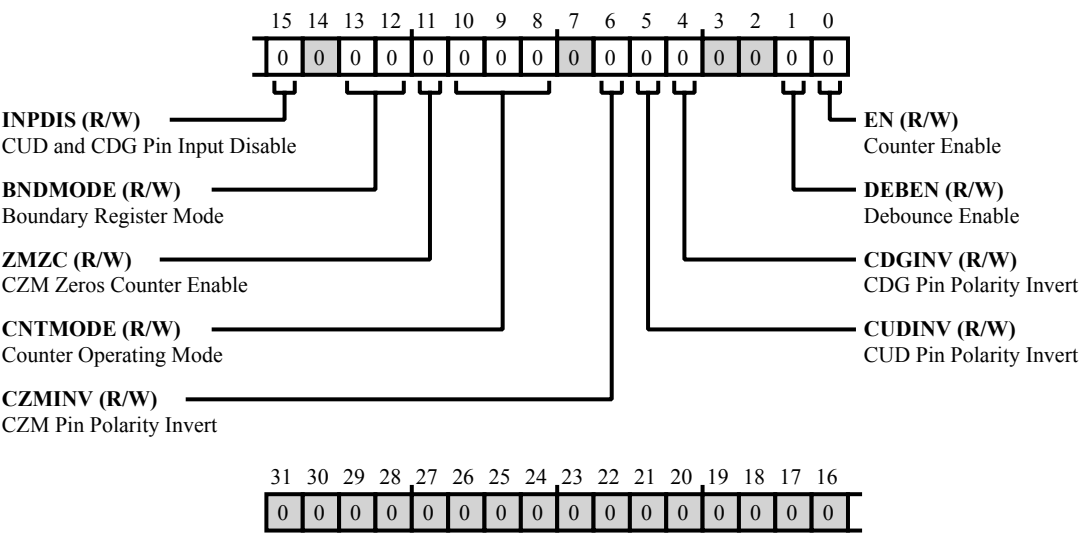


Figure 17-4: CNT_CFG Register Diagram

Table 17-8: CNT_CFG Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W)	INPDIS	CUD and CDG Pin Input Disable. The <code>CNT_CFG</code> . <code>INPDIS</code> disables or enables the <code>CNT_UD</code> input pin and the <code>CNT_DG</code> pin.
		0 Enable
		1 Pin Input Disable
13:12 (R/W)	BNDMODE	Boundary Register Mode. The <code>CNT_CFG</code> . <code>BNDMODE</code> bit field selects the mode for the <code>CNT_MIN</code> and <code>CNT_MAX</code> boundary registers.
		0 BND_COMP. Boundary Compare Mode
		1 BND_ZERO. Boundary Zero Mode
		2 BND_CAPT. Boundary Capture Mode
		3 BND_AEXT. Boundary Auto-extend Mode

Table 17-8: CNT_CFG Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
11 (R/W)	ZMZC	CZM Zeros Counter Enable. The CNT_CFG.ZMZC bit enables or disables level sensitive - active CNT_ZM pin operation to zero the CNT_CNTR register.
		0 Disable
		1 Enable
10:8 (R/W)	CNTMODE	Counter Operating Mode. The CNT_CFG.CNTMODE bit field selects the operating mode for the CNT_UD input pin and the CNT_DG pin.
		0 QUAD_ENC. Quadrature Encoder Mode
		1 BIN_ENC. Binary Encoder Mode
		2 UD_CNT. Rotary Counter Mode
		4 DIR_CNT. Direction Counter Mode
		5 DIR_TMR. Direction Timer Mode
6 (R/W)	CZMINV	CZM Pin Polarity Invert. The CNT_CFG.CZMINV bit selects the polarity for the CNT_ZM pin. This polarity must be configured before the counter is enabled. It must not change on-the-fly while the counter is enabled.
		0 Active High, Rising Edge
		1 Active Low, Falling Edge
5 (R/W)	CUDINV	CUD Pin Polarity Invert. The CNT_CFG.CUDINV bit selects the polarity for the CNT_UD pin. This polarity must be configured before the counter is enabled. It must not change on-the-fly while the counter is enabled.
		0 Active High, Rising Edge
		1 Active Low, Falling Edge
4 (R/W)	CDGINV	CDG Pin Polarity Invert. The CNT_CFG.CDGINV bit selects the polarity for the CNT_DG pin. This polarity must be configured before the counter is enabled. It must not change on-the-fly while the counter is enabled.
		0 Active High, Rising Edge
		1 Active Low, Falling Edge

Table 17-8: CNT_CFG Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/W)	DEBEN	Debounce Enable. The CNT_CFG.DEBEN bit enables or disables CNT input debounce filtering operation selected with the CNT_DEBNCE register.
		0 Disable
		1 Enable
0 (R/W)	EN	Counter Enable. The CNT_CFG.EN bit enables or disables CNT operation.
		0 Counter Disable
		1 Counter Enable

Command Register

The `CNT_CMD` register configures the CNT, enabling operations such as zeroing a counter register and copying or swapping boundary registers. These actions are taken by setting the appropriate bit.

Read operations from this register do not return meaningful values, with the exception of the `CNT_CMD.W1ZMONCE` bit, where a set bit indicates that the bit has been set by software before, but a zero marker event has not yet been detected on the `CNT_ZM` pin yet. For more information, see the CNT functional description.

The `CNT_CNTR`, `CNT_MIN`, and `CNT_MAX` registers can be initialized to zero by setting the `CNT_CMD.W1LCNTZERO`, `CNT_CMD.W1LMINZERO`, and `CNT_CMD.W1LMAXZERO` bits. In addition to clearing registers, the `CNT_CMD` register permits modifying the `CNT_MIN` and `CNT_MAX` boundary registers in a number of ways. The current counter value in the `CNT_CNTR` register can be captured and loaded into either of the two boundary registers to create new boundary limits. This operation is performed by setting the `CNT_CMD.W1LMAXCNT` and `CNT_CMD.W1LMINCNT` bits. Alternatively, the counter can be loaded from `CNT_MAX` or `CNT_MIN` using the `CNT_CMD.W1LCNTMAX` and `CNT_CMD.W1LCNTMIN` bits. It is also possible to transfer the current `CNT_MAX` value into `CNT_MIN` (or conversely) through the `CNT_CMD.W1LMINMAX` and `CNT_CMD.W1LMAXMIN` bits.

Another counter operation is the ability to only have the zero marker clear the `CNT_CNTR` register once. For more information, see the CNT functional description.

It is possible for multiple actions to be performed simultaneously by setting multiple bits in the `CNT_CMD` register. However, there are restrictions. The bits associated with each command have been grouped together such that all bits that involve a write to the `CNT_CNTR`, `CNT_MAX`, or `CNT_MIN` registers are located within bits 4-bit groups of the `CNT_CMD` register.

Note that a maximum of three commands can be issued at any one time, excluding the `CNT_CMD.W1ZMONCE` command. Also, note that `CNT_CMD.W1LCNTMIN`, `CNT_CMD.W1LCNTMAX`, and `CNT_CMD.W1LCNTZERO` bits have to be used exclusively. Never set more than one of them at the same time.

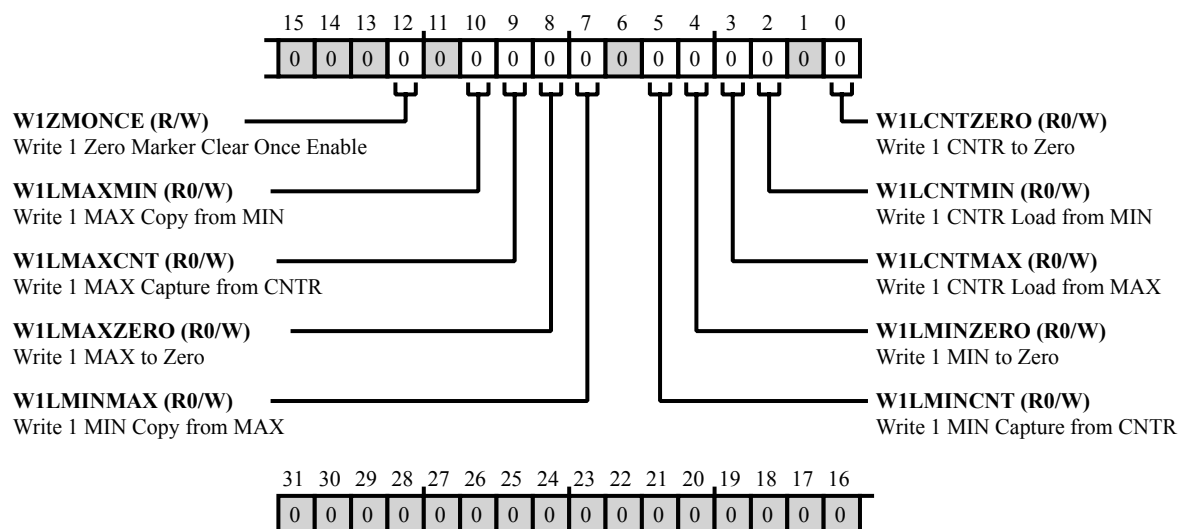


Figure 17-5: CNT_CMD Register Diagram

Table 17-9: CNT_CMD Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
12 (R/W)	W1ZMONCE	Write 1 Zero Marker Clear Once Enable. The CNT_CMD.W1ZMONCE enables a single zero marker clear of the CNT_CNTR register. Reading a 1 in this bit indicates that the bit has been set by software before, but no zero marker event has been detected on the CNT_ZM pin yet.
10 (R0/W)	W1LMAXMIN	Write 1 MAX Copy from MIN. The CNT_CMD.W1LMAXMIN bit transfers the current CNT_MIN register value into CNT_MAX register.
9 (R0/W)	W1LMAXCNT	Write 1 MAX Capture from CNTR. The CNT_CMD.W1LMAXCNT bit loads the current value in the CNT_CNTR register into the CNT_MAX register to create a new boundary limit.
8 (R0/W)	W1LMAXZERO	Write 1 MAX to Zero. Writing a 1 to the CNT_CMD.W1LMAXZERO bit clears the CNT_MAX register.
7 (R0/W)	W1LMINMAX	Write 1 MIN Copy from MAX. The CNT_CMD.W1LMINMAX bit transfers the current CNT_MAX register value into CNT_MIN register.
5 (R0/W)	W1LMINCNT	Write 1 MIN Capture from CNTR. The CNT_CMD.W1LMINCNT bit loads the current value in the CNT_CNTR register into the CNT_MIN register to create a new boundary limit.
4 (R0/W)	W1LMINZERO	Write 1 MIN to Zero. Writing a 1 to the CNT_CMD.W1LMINZERO bit clears the CNT_MIN register.

Table 17-9: CNT_CMD Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
3 (R0/W)	W1LCNTMAX	Write 1 CNTR Load from MAX. The CNT_CMD.W1LCNTMAX bit loads the current value in the CNT_MAX register into the CNT_CNTR register to create a new boundary limit.
2 (R0/W)	W1LCNTMIN	Write 1 CNTR Load from MIN. The CNT_CMD.W1LCNTMIN bit loads the current value in the CNT_MIN register into the CNT_CNTR register to create a new boundary limit.
0 (R0/W)	W1LCNTZERO	Write 1 CNTR to Zero. Writing a 1 to the CNT_CMD.W1LCNTZERO bit clears the CNT_CNTR register.

Counter Register

The `CNT_CNTR` register holds the 32-bit, two's-complement count value. It can be read and written at any time. Hardware ensures that reads and write are atomic, by providing respective shadow registers. This register can be accessed with either 32-bit or 16-bit operations. This allows use of the CNT as a 16-bit counter if sufficient for the application.

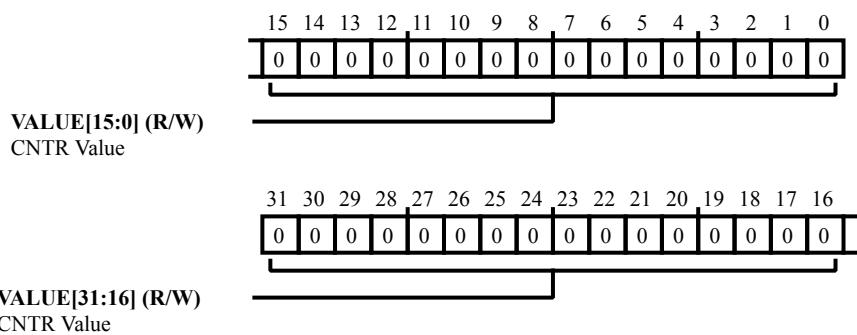


Figure 17-6: CNT_CNTR Register Diagram

Table 17-10: CNT_CNTR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	CNTR Value. The <code>CNT_CNTR.VALUE</code> bit field holds the 32-bit, two's-complement count value.

Debounce Register

The `CNT_DEBNCE` register selects the noise filtering characteristic of the three input pins according to the formula:

$$t_{\text{filter}} = 128 \times (2^{\text{DPRESCALE}} / \text{SCLK})$$

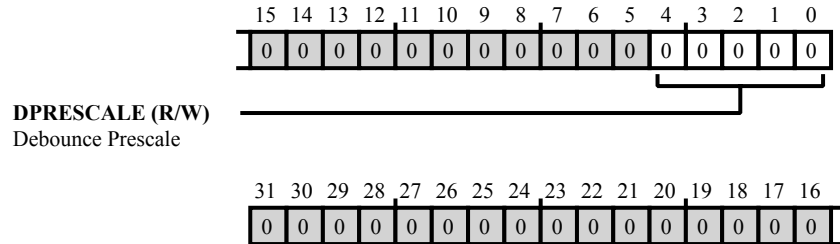


Figure 17-7: CNT_DEBNCE Register Diagram

Table 17-11: CNT_DEBNCE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
4:0 (R/W)	DPRESCALE	Debounce Prescale. The <code>CNT_DEBNCE.DPRESCALE</code> selects the desired number of input filtering cycles (and resulting input debounce time) in multiples of SCLK.
		0 1x Cycles = 128 SCLK Cycles
		1 2x Cycles
		2 4x Cycles
		3 8x Cycles
		4 16x Cycles
		5 32x Cycles
		6 64x Cycles
		7 128x Cycles
		8 256x Cycles
		9 512x Cycles
		10 1024x Cycles
		11 2048x Cycles
		12 4096x Cycles
		13 8192x Cycles
		14 16384x Cycles
		15 32768x Cycles
		16 65536x Cycles

Table 17-11: CNT_DEBNCE Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
		17	131072x Cycles
		18	Reserved from This Value. The values 10010 - 11111 are reserved.
		31	Reserved Until This Value

Interrupt Mask Register

The `CNT_IMSK` register supports enabling (unmasking) interrupt request generation from each of the CNT events.

All bits in `CNT_IMSK` either disable/mask an interrupt request (if bit cleared) or enable/unmask an interrupt request (if bit set).

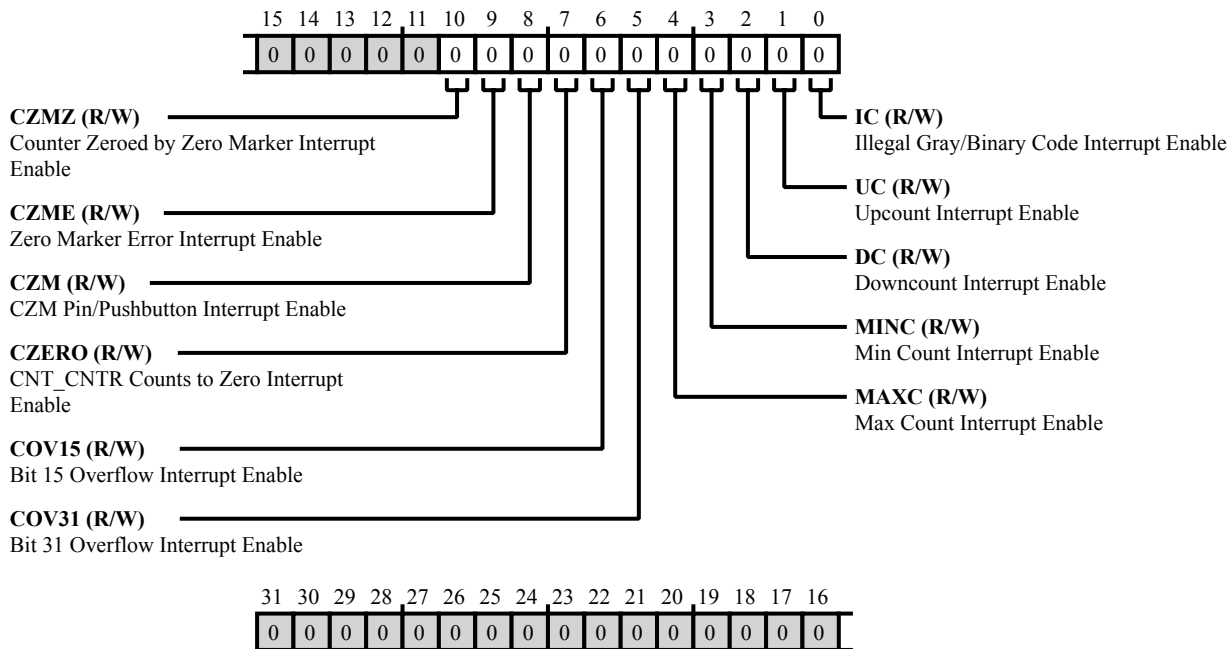


Figure 17-8: CNT_IMSK Register Diagram

Table 17-12: CNT_IMSK Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
10 (R/W)	CZMZ	Counter Zeroed by Zero Marker Interrupt Enable. The <code>CNT_IMSK.CZMZ</code> bit enables (unmasks) the counter zeroed by zero marker interrupt request.
		0 Mask Interrupt
		1 Unmask Interrupt
9 (R/W)	CZME	Zero Marker Error Interrupt Enable. The <code>CNT_IMSK.CZME</code> bit enables (unmasks) the zero marker error interrupt request.
		0 Mask Interrupt
		1 Unmask Interrupt

Table 17-12: CNT_IMSK Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
8 (R/W)	CZM	CZM Pin/Pushbutton Interrupt Enable. The CNT_IMSK.CZM bit enables (unmasks) the CZM pin/pushbutton interrupt request.
		0 Mask Interrupt
		1 Unmask Interrupt
7 (R/W)	CZERO	CNT_CNTR Counts to Zero Interrupt Enable. The CNT_IMSK.CZERO bit enables (unmasks) the counts to zero interrupt request.
		0 Mask Interrupt
		1 Unmask Interrupt
6 (R/W)	COV15	Bit 15 Overflow Interrupt Enable. The CNT_IMSK.COV15 bit enables (unmasks) the bit 15 overflow interrupt request.
		0 Mask Interrupt
		1 Unmask Interrupt
5 (R/W)	COV31	Bit 31 Overflow Interrupt Enable. The CNT_IMSK.COV31 bit enables (unmasks) the bit 31 overflow interrupt request.
		0 Mask Interrupt
		1 Unmask Interrupt
4 (R/W)	MAXC	Max Count Interrupt Enable. The CNT_IMSK.MAXC bit enables (unmasks) the max count interrupt request.
		0 Mask Interrupt
		1 Unmask Interrupt
3 (R/W)	MINC	Min Count Interrupt Enable. The CNT_IMSK.MINC bit enables (unmasks) the min count interrupt request.
		0 Mask Interrupt
		1 Unmask Interrupt
2 (R/W)	DC	Downcount Interrupt Enable. The CNT_IMSK.DC bit enables (unmasks) the down count interrupt request.
		0 Mask Interrupt
		1 Unmask Interrupt

Table 17-12: CNT_IMSK Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/W)	UC	Upcount Interrupt Enable. The CNT_IMSK. UC bit enables (unmasks) the up count interrupt request.
		0 Mask Interrupt
		1 Unmask Interrupt
0 (R/W)	IC	Illegal Gray/Binary Code Interrupt Enable. The CNT_IMSK. IC bit enables (unmasks) the illegal Gray/Binary Code interrupt request and should only be used in these modes.
		0 Mask Interrupt
		1 Unmask Interrupt

Maximum Count Register

The `CNT_MAX` register holds the 32-bit, two's-complement, higher boundary value. It can be read and written at any time. Hardware ensures that reads and write are atomic, by providing respective shadow registers. This register can be accessed with either 32-bit or 16-bit operations. This allows for using the CNT as a 16-bit counter if sufficient for the application.

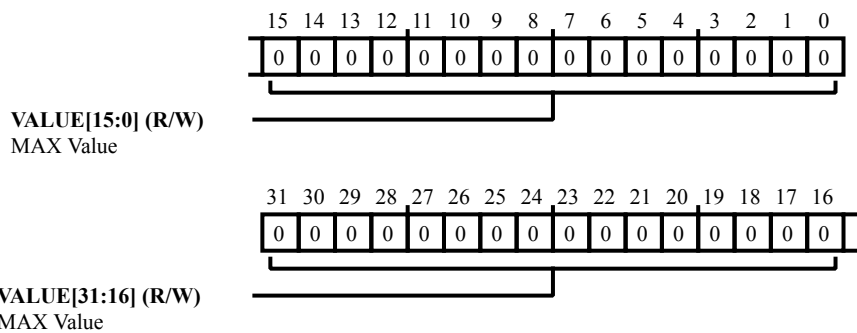


Figure 17-9: CNT_MAX Register Diagram

Table 17-13: CNT_MAX Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	MAX Value. The <code>CNT_MAX.VALUE</code> bit field holds the 32-bit, two's-complement, higher boundary value.

Minimum Count Register

The `CNT_MIN` register holds the 32-bit, two's-complement, lower boundary value. It can be read and written at any time. Hardware ensures that reads and write are atomic, by providing respective shadow registers. This register can be accessed with either 32-bit or 16-bit operations. This allows for using the CNT as a 16-bit counter if sufficient for the application.

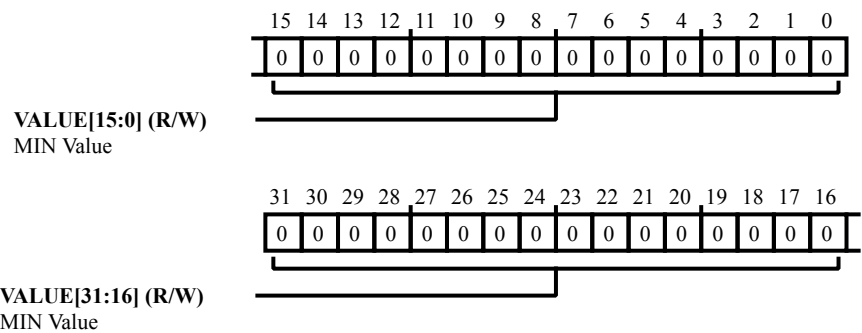


Figure 17-10: CNT_MIN Register Diagram

Table 17-14: CNT_MIN Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	MIN Value. The <code>CNT_MIN.VALUE</code> bit field holds the 32-bit, two's-complement, lower boundary value.

Status Register

The **CNT_STAT** register provides status information for each of the CNT events as configured in the **CNT_IMSK** register. When a CNT event is detected, the corresponding bit in this register is set. It remains set until either software writes a 1 to the bit (write-1-to-clear) or the CNT is disabled.

All bits in the **CNT_STAT** register indicate either no interrupt request pending (if bit cleared) or an interrupt request pending (if bit set).

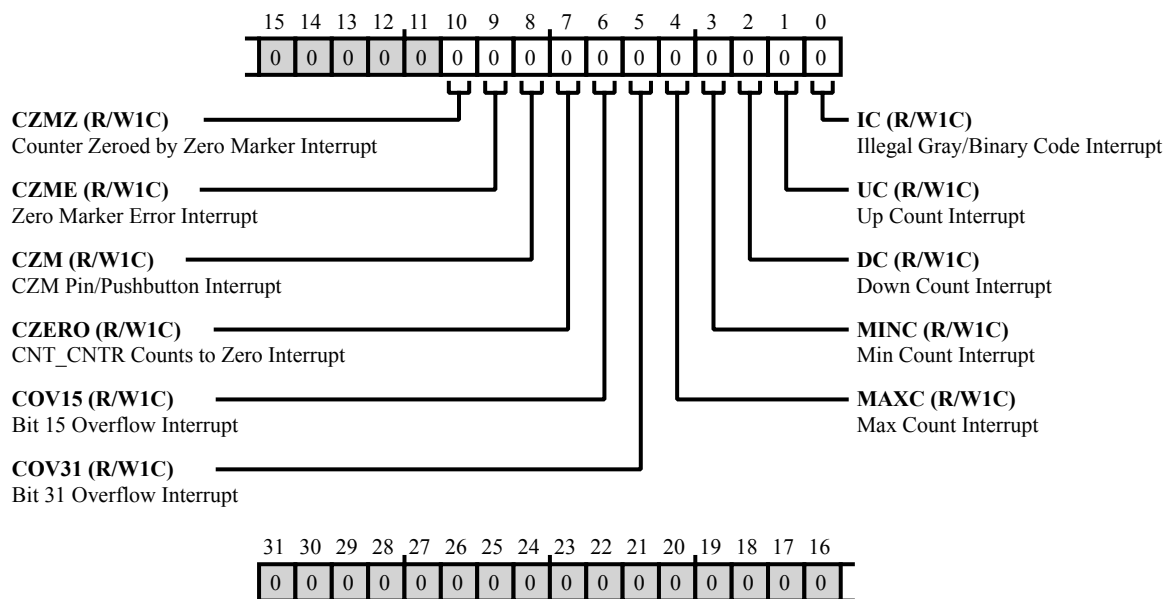


Figure 17-11: CNT_STAT Register Diagram

Table 17-15: CNT_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
10 (R/W1C)	CZMZ	Counter Zeroed by Zero Marker Interrupt. The CNT_STAT.CZMZ bit indicates a zero marker error. If the CNT_CFG.ZMZC bit =1, this interrupt request is generated when the CZMII latch reports a significant edge on the CZM input. Once cleared by software the CNT_STAT.CZM bit is not set again when the CZM input remains active without pulsing.
		0 No Error
		1 Error Occurred

Table 17-15: CNT_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
9 (R/W1C)	CZME	Zero Marker Error Interrupt. The CNT_STAT.CZME bit behaves similarly to the CNT_STAT.CZM bit, with the exception that CNT_STAT.CZME is not set on the CZM edge when the lower four bits of the CNT_CNTR are not zero. In many applications this indicates an error condition, as the zero marker might be out of sync with the counter.
		0 No Error
		1 Error Occurred
8 (R/W1C)	CZM	CZM Pin/Pushbutton Interrupt. The CNT_STAT.CZM bit indicates a CZM pin/pushbutton error. This interrupt request is generated when a significant edge is seen on the CZM pin, regardless what mode the counter is operating in. This is often used to sense push buttons (especially with the debouncing circuit enabled).
		0 No Error
		1 Error Occurred
7 (R/W1C)	CZERO	CNT_CNTR Counts to Zero Interrupt. The CNT_STAT.CZERO bit indicates a counts to zero error. This error is generated when the CNT_CNTR register has incremented or decremented toward 0x0000.0000. The latch is not set when software writes to the CNT_CNTR register directly or when the counter is zeroed by writes to the CNT_CMD register.
		0 No Error
		1 Error Occurred
6 (R/W1C)	COV15	Bit 15 Overflow Interrupt. The CNT_STAT.COV15 bit indicates a bit 15 overflow error. This error is generated when the 16-bit twos-complement CNT_CNTR register has incremented from 0xxxxx.7FFF to 0xxxxx.8000 or decremented from 0xxxxx.8000 to 0xxxxx.7FFE.
		0 No Error
		1 Error Occurred
5 (R/W1C)	COV31	Bit 31 Overflow Interrupt. The CNT_STAT.COV31 bit indicates a bit 31 overflow error. This error is generated when the 32-bit twos-complement CNT_CNTR register has incremented from 0x7FFF.FFFF to 0x8000.0000 or decremented from 0x8000.0000 to 0x7FFF.FFFF.
		0 No Error
		1 Error Occurred

Table 17-15: CNT_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
4 (R/W1C)	MAXC	Max Count Interrupt. The CNT_STAT.MAXC bit indicates a max count error. This interrupt is used in boundary compare (BND_COMP) mode. If after incrementing the CNT_CNTR register equals CNT_MAX, the CNT_STAT.MAXC bit is set.
		0 No Error
		1 Error Occurred
3 (R/W1C)	MINC	Min Count Interrupt. The CNT_STAT.MINC bit indicates a minimum count error. This interrupt is used in boundary compare (BND_COMP) mode. If, after decrementing, the CNT_CNTR register equals CNT_MIN, the CNT_STAT.MINC bit is set.
		0 No Error
		1 Error Occurred
2 (R/W1C)	DC	Down Count Interrupt. The CNT_STAT.DC bit indicates a down count error. This interrupt is generated when the CNT_CNTR register decrements.
		0 No Error
		1 Error Occurred
1 (R/W1C)	UC	Up Count Interrupt. The CNT_STAT.UC bit indicates an up count interrupt. This interrupt is generated when the CNT_CNTR register increments.
0 (R/W1C)	IC	Illegal Gray/Binary Code Interrupt. The CNT_STAT.IC bit indicates a illegal Gray/Binary Code interrupt and should only be used in these modes. In normal operation those codes can increment or decrement the CNT_CNTR register by one at a time. If the sensed inputs instruct the counter to increment or decrement by two, the CNT_STAT.IC bit is set. Hardware sets the CNT_STAT.IC bit in QUAD_ENC and BIN_ENC encoder modes only.
		0 No Error
		1 Error Occurred

18 Media Local Bus (MLB)

Media Local Bus (MediaLB[®]) is an on-PCB or inter-chip communication bus, which allows an application to access the MOST network data. Media Local Bus supports all the MOST network data transport methods including synchronous stream data, asynchronous packet data, control message data and isochronous data. The MLB topology supports communication among the MLB controller and MLB devices, where the MLB controller is the interface between the MLB devices and the MOST network.

The MLB module serves as an interface between the MediaLB and the processor, implementing the requirements of the physical layer and the link layer outlined in the MediaLB specification. It supports up to 64 logical channels with up to 128 bytes of data per MediaLB frame. Transmit and receive data can be transferred between MediaLB and on-chip memory with DMA block transfers.

The MLB supports the MOST25, MOST50 and MOST150 standards and this document assumes familiarity with these standards. For more information, refer to the Media Local Bus specification version 4.2.

- Copyright 1998-2016 Microchip Technology Inc. All rights reserved. Portions of this chapter are included with permission from Microchip Technology, Inc.

Features

The objective of MLB is to map all the MOST Network data types (transport methods) into a single low-cost, scalable, and standardized hardware interface between a MediaLB controller and at least one other MediaLB Device. The adoption of MediaLB simplifies the hardware interface, reduces the pin count, and facilitates the design of modular reusable hardware. From a software development perspective, the use of MediaLB relieves the system developer from the complexity of the MOST network, which simplifies software development and enables the design of reusable software for different applications. This simplified, standardized interface shortens time to market and makes software maintenance effortless.

- Compliant to Media Local Bus specification version 4.2
- Support MOST25, MOST50, MOST150 standards
- 3-pin mode support various data rates of $256 \times FS$, $512 \times FS$, $1024 \times FS$
- Support 64 logic channels
- Shared pins for 3-pin mode

- Dedicated internal RAM for data buffering and channel table
- MLB 3-pin clock available out on a shared pin for DAI

MLB Definitions

The following are standard MLB and MOST terms that are used in this chapter.

AGU

Address Generation Units. To access a particular HBI channel the HC must first configure one of two HBI address generation units. AGUs can be configured by writing the HBI command registers, HCMD0 and HCMD1.

CAT

Channel Allocation Table. The Channel Allocation Table (CAT) is comprised of 16 CTR entries. Each 16-bit CAT entry represents a logical connection to or from a transmit/receive device (for example MediaLB or HBI channel).

CDT

Channel Descriptor Table.

FCE

Flow Control Enable bit. The FCE bit is used by MediaLB isochronous Rx channels only.

HBI

Host Bus Interface. The HBI block provides 16-bit parallel completer access to all MOST channels and data types for the external Host Controller (HC). The HBI supports up to 64 independent channels.

HC

Host Controller (external).

HCMDx

HBI Command registers.

HSTSx

HBI status registers.

MFE

Multi-Frame per sub-buffer enable bit. The MFE bit is used by MediaLB synchronous channels only.

PML

Packet Message Length.

Clocking

The MLB controller provides an external clock pin—the media local bus clock. The MLB controller generates the clock. It is synchronized to the MOST network and provides the timing for the entire MLB interface at FS = 48 kHz.

Functional Description

The *MediaLB Block Diagram* figure shows the MLB high-level architecture. The MLB core serves as an interface between the MediaLB and the processor, implementing the requirements of the physical layer and the link layer outlined in the MediaLB specification. The MLB core has the following responsibilities.

- Transmit commands and data when functioning as the transmitting device associated with a *Channel Address*
- Receive data and transmit Rx status responses when functioning as the receiving device associated *Channel Address*
- MLB lock detection
- System channel command handling

The MediaLB interfaces are capable of exchanging data at speeds up to $1024 \times F_s$ in 3-pin mode.

A set of physical channels for exchanging data over the MediaLB bus is supported. These physical channels (4 bytes in length, or a quadlet) can be grouped into logical channels, where each logical channel is referenced using a channel address and represents a uni-directional datapath between a specific MediaLB device transmitting the data and the MediaLB device(s) receiving the data. The MediaLB 3-pin interface provides support for up to 128 bytes of data per frame. The logical channels, configured by system software, can be any combination of channel types (synchronous, asynchronous, isochronous, or control) and direction (transmit or receive).

ADSP-21560_21561_21564_21568 MLB Register List

The MediaLB Device Interface Macro 2 (MediaLB DIM 2), also referred to as OS62420, implements the required functionality of a Media Local Bus (MediaLB) device. This logic serves as an interface between the inter-chip MediaLB bus and a customer IC, implementing the physical- and link-layer requirements outlined in the MediaLB Specification.

Table 18-1: ADSP-21560_21561_21564_21568 MLB Register List

Name	Description
MLB_ACMR0	Peripheral Channel Mask 0 Register

Table 18-1: ADSP-21560_21561_21564_21568 MLB Register List (Continued)

Name	Description
MLB_ACMR1	Peripheral Channel Mask 1 Register
MLB_ACSR0	Peripheral Channel Status 0 Register
MLB_ACSR1	Peripheral Channel Status 1 Register
MLB_ACTL	Bus Control Register
MLB_CTL0	MediaLB Control 0 Register
MLB_CTL1	Control 1 Register
MLB_GCTL	MLB Global Control Register
MLB_HCBR0	HBI Channel Busy 0 Register
MLB_HCBR1	HBI Channel Busy 1 Register
MLB_HCER0	HBI Channel Error 0 Register
MLB_HCER1	HBI Channel Error 1 Register
MLB_HCMR0	HBI Channel Mask 0 Register
MLB_HCMR1	HBI Channel Mask 1 Register
MLB_HCTL	HBI Control Register
MLB_MADR	Memory Interface Address Register
MLB_MCTL	Memory Interface Control Register
MLB_MDAT0	Memory Interface Control Data 0 Register
MLB_MDAT1	Memory Interface Control Data 1 Register
MLB_MDAT2	Memory Interface Control Data 2 Register
MLB_MDAT3	Memory Interface Control Data 3 Register
MLB_MDWE0	Memory Interface Control Data Write Enable 0 Register
MLB_MDWE1	Memory Interface Control Data Write Enable 1 Register
MLB_MDWE2	Memory Interface Control Data Write Enable 2 Register
MLB_MDWE3	Memory Interface Control Data Write Enable 3 Register
MLB_MIEN	Interrupt Enable Register
MLB_MS0	Channel Status 0 Register
MLB_MS1	Channel Status 1 Register
MLB_MSD	System Data Register
MLB_MSS	System Status Register

ADSP-21560_21561_21564_21568 MLB Interrupt List

Table 18-2: ADSP-21560_21561_21564_21568 MLB Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
123	MLB0_INT0	MLB0 Interrupt 0		
124	MLB0_INT1	MLB0 Interrupt 1		
125	MLB0_STAT	MLB0 Status		

MediaLB Protocol

The MediaLB topology supports communication among all MediaLB devices, including the MediaLB controller. The bus interface consists of a uni-directional line for clock (MLBC), a bidirectional line for signal information (MLBS), and a bidirectional line for data transfer (MLBD). The MediaLB topology supports one controller connected to one or more devices, where the controller is the interface between the MediaLB devices and the MOST network.

The MediaLB controller includes MediaLB device functionality, and also generates the MediaLB clock (MLBC) that is synchronized to the MOST Network. This generated clock provides the timing for the entire MediaLB interface. The MLBS line is a multiplexed signal which carries channel addresses generated by the MediaLB controller, as well as command and RxStatus bytes from MediaLB devices. The MLBD line is driven by the transmitting MediaLB device and is received by all other MediaLB devices, including the MediaLB controller. The MLBD line carries the actual data (synchronous, asynchronous, control, or isochronous).

Once per MOST network frame, the MLB controller generates a unique frame sync pattern on the MLB_SIG line. The end of the frame sync pattern defines the byte boundary and the channel boundary for the MLB_SIG and MLB_DAT lines of all MLB devices.

The MLB controller manages the arbitration for all the channels on the MLB and grants bandwidth for all the MLB devices. An MLB *physical channel* is defined as four bytes wide, or a quadlet. Physical channels can be grouped into multiple quadlets (which do not have to be consecutive) to form an MLB *logical channel*, which is defined by a unique channel address.

As shown in *MLB Data Structure*, the MLB controller initiates communication by sending out a channel address on the MLB_SIG line for each physical channel. The channel address indicates which MLB device is transmitting and which MLB devices are receiving in the following physical channel. Therefore, four bytes after the controller outputs the channel address on the MLB_SIG line, the transmitting device outputs a command byte command on the MLB_SIG line and outputs the respective data on the MLB_DAT line, concurrently. The MLB command byte contains the type of data currently being transmitted (for example synchronous, asynchronous or control).

The MLB device receiving the channel data outputs a status byte, RxStatus, on the MLB_SIG line immediately after the transmitting device outputs the command byte. The status response can indicate that the receiving device is busy and cannot receive the data at present, or the device is ready to receive the data. Since synchronous stream data is sent in a broadcast fashion, receiving devices cannot return a busy status and should not drive RxStatus onto the MLB_SIG line.

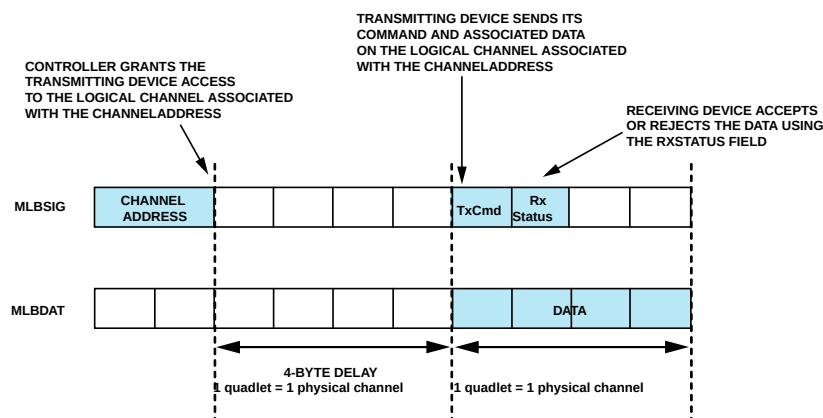


Figure 18-1: MLB Data Structure

MLB Architectural Concepts

The following sections provide information about the MLB architecture.

MediaLB Block Diagram

The *MediaLB Block Diagram* shows the various blocks within the interface its connections to the processor.

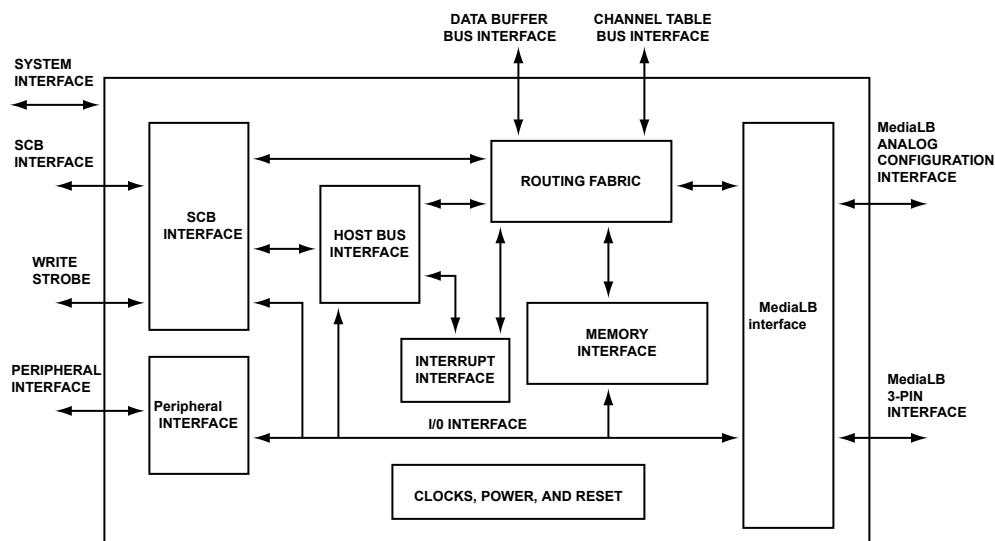


Figure 18-2: MediaLB Block Diagram

MediaLB Interface

The MediaLB interfaces provide real-time access to all network data types - synchronous, asynchronous, control, and isochronous data.

- MediaLB 3-pin interface—supports the MediaLB protocol for single-ended 3-pin mode, with a maximum data rate of 1024 FS (49.152 MHz at FS = 48 kHz).

MediaLB Channel Address to Logical Channel Mapping

The MediaLB channel addresses are mapped to the logical channels as shown in the *MediaLB Channel Address to Logical Channel Mapping* table.

Table 18-3: MediaLB Channel Address to Logical Channel Mapping

Channel Address	Logical Address
0x0002	1
0x0004	2
0x0006	3
...	...
0x007C	62
0x007E	63
0x01FE	0 (Logical channel 0 is the system channel and is reserved)

Routing Fabric

The Routing Fabric (RF) block manages the flow of data between the MediaLB port and the HBI port. It manages accessing the Channel Table RAM (CTR) and Data Buffer RAM (DBR), which are explained in the following sub-sections. The routing fabric uses channel descriptors (stored in the CTR) to manage access to dynamic buffers in the DBR.

Data Buffer RAM

The Data Buffer RAM (DBR) is an 8-bit x 16k entry buffer, single-port synchronous SRAM, and provides dynamic circular buffering between the transmit and receive devices. The size and location of each data buffer is defined by software in the Channel Descriptor Table (CDT), which is in the Channel Table RAM (CTR), described in following sections.

Channel Table RAM

The Channel Table RAM (CTR) is a 128-bit x 144-entry table that allows system software to dynamically configure channel routing and allocate data buffers in the DBR. The CTR is logically divided into three tables:

- [Channel Descriptor Tables](#)
- [AHB Descriptor Table \(ADT\)](#)
- [Channel Allocation Table](#)

Address Mapping

The *CTR Address Mapping* table shows the address mapping for the CTR.

Table 18-4: CTR Address Mapping

Label	Address	Bits [127:96]	Bits [95:64]		Bits [63:32]		Bits [31:0]		
Channel Descriptor Table (CDT)									
CDT	0x00	CDT0[127:0], CL = 0							
	0x01	CDT1[127:0], CL = 1							
	0x02	CDT2[127:0], CL = 2							
	...								
	0x3D	CDT61[127:0], CL = 61							
	0x3E	CDT62[127:0], CL = 62							
	0x3F	CDT63[127:0], CL = 63							
AHB Descriptor Table (ADT)									
ADT	0x40	ADT0[127:0], CL = 0							
	0x41	ADT1[127:0], CL = 1							
	0x42	ADT2[127:0], CL = 2							
	...								
	0x7D	ADT61[127:0], CL = 61							
	0x7E	ADT62[127:0], CL = 62							
	0x7F	ADT63[127:0], CL = 63							
Channel Allocation Table (CAT)									
CAT for Medi-aLB	0x80	CAT7	CAT6	CAT5	CAT4	CAT3	CAT2	CAT1	CAT0
	...	...	...	...	...	...	...	...	...
	0x87	CAT63	CAT62	CAT61	CAT60	CAT59	CAT58	CAT57	CAT56
CAT for HBI* ¹	0x88	CAT71	CAT70	CAT69	CAT68	CAT67	CAT66	CAT65	CAT64
	...	...	...	...	...	...	...	...	...
	0x8F	CAT127	CAT126	CAT125	CAT124	CAT123	CAT122	CAT121	CAT120

^{*1} A fixed relationship exists between ADT entries and HBI CAT entries. When using HBI channel 0 (CAT64), program ADT0. When using HBI channel 1 (CAT65), program ADT1, and so on.

Channel Allocation Table

The *Channel Allocation Table (CAT)* table is comprised of 16 CTR entries (addresses 0x80 0x8F) as shown in the *CTR Entry Map* table. Each 16-bit CAT entry represents a logical connection to or from a transmit or receive device. (for example, MediaLB channel). All entries are indexed according to a fixed physical address assigned to every RX/TX channel as shown in the *CAT Entry Formats* table. The value stored in a CAT entry includes a 6-bit connection label, which provides a pointer to the CDT. To complete a logical channel and form a routing connection, system software must assign the same connection label to both the RX and TX channels.

Table 18-5: CAT Entry Map

Peripheral	TX Channels	RX Channels	CAT Start Index	CAT End Index	Entries
MediaLB	0 to 64	64 TX Channels	0	63	64
HBI	0 to 64	64 TX Channels	64	127	64

The format of a full CAT entry is shown in the *CAT Entry Formats* table, with field descriptions described in the *CAT Field Definitions* table. All reserved bits of a CAT entry field should be written as zero.

Table 18-6: CAT Entry Formats

Channel Type	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Isochronous	rsvd	FCE	rsvd	RNW	CE	CT[2:0] = 011			rsvd		CL[5:0]					
Asynchronous	rsvd		MT	RNW	CE	CT[2:0] = 010			rsvd		CL[5:0]					
Control	rsvd		MT	RNW	CE	CT[2:0] = 001			rsvd		CL[5:0]					
Synchronous	rsvd	MFE	MT	RNW	CE	CT[2:0] = 000			rsvd		CL[5:0]					

Table 18-7: CAT Field Definitions

Field	Description
CL[5:0]	Connection Label (offset into CDT)
CT[2:0]	Channel Type (others) 111 = Reserved 110 = Reserved 101 = Reserved 100 = Reserved 011 = Isochronous 010 = Asynchronous 001 = Control 000 = Synchronous
CE	Channel Enable. 0 = Disabled, 1 = Enabled
RNW	Read Not Write. 0 = Write, 1 = Read
MT	Mute Enable. 0 = Disabled When set for synchronous channels, the MT bit forces RX channels to write zeros into the channel data buffer, and TX channels to output zeros on the physical interface. When set for asynchronous and control channels, the MT bit causes DMA to halt at a packet boundary. Not valid for isochronous channels.
FCE	Flow Control Enable. 0 = Disabled, 1 = Enabled The FCE bit is used by MediaLB isochronous RX channels only.
MFE	Multi-Frame per sub-buffer enable. 0 = Disabled, 1 = Enabled The MFE bit is used by MediaLB synchronous channels only.

Table 18-7: CAT Field Definitions (Continued)

Field	Description
rsvd	Reserved. Software writes a 0 to all reserved bits when the entry is initialized. These bits are read-only after initialization.

Channel Set Up

Data direction is in reference to the DBR. The data direction of CAT entries corresponding to the same channel is reversed for the HBI CAT and the MediaLB CAT.

- For a Tx channel (from the HC to the MediaLB interface):
 - HBI CAT entry: RNW = 0 (write)
 - MediaLB CAT entry: RNW = 1 (read)
- Conversely, for an Rx channel (data from MediaLB to HC):
 - HBI CAT entry: RNW = 1 (read)
 - MediaLB CAT entry: RNW = 0 (write)

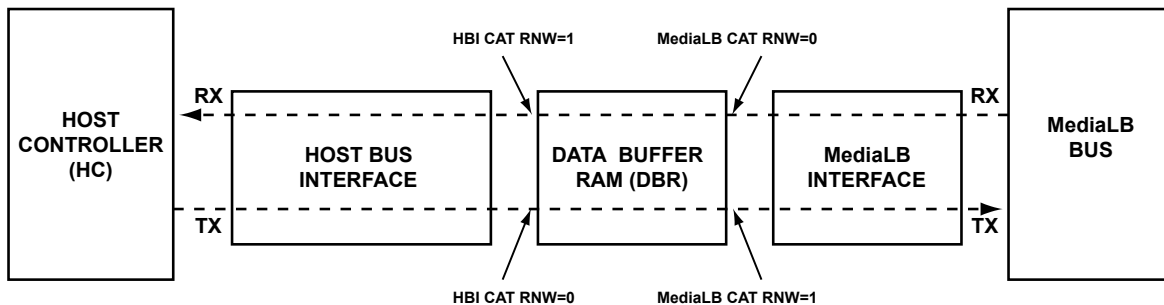


Figure 18-3: DBR Directional Relationship

Channel Descriptor Tables

The Channel Descriptor Table (CDT) is comprised of 64 CTR entries (addresses 0x00 - 0x3F), as shown in the [Table 18-4 CTR Address Mapping](#) table. Each 128-bit CDT entry (also referred to as a channel descriptor) is referenced by a connection label and contains information about a data buffer in the DBR (for example buffer size, address pointers). The format of each CDT entry is dependent on the channel type (synchronous, isochronous, asynchronous, or control).

NOTE: All reserved channel descriptor bits must be written to 0 by software when initialized.

Synchronous Channel Descriptors

The format and field definitions for a synchronous CDT entry are shown in the *Synchronous CDT Entry Format* and *Synchronous CDT Entry Field Definitions* tables.

Table 18-8: Synchronous CDT Entry Format

Bit Offset	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	WSBC[1:0]		Reserved													
16	RSBC[1:0]		Reserved													
32	Reserved															
48	Reserved															
64	WSTS[3:0]				WPTR[11:0]											
80	RSTS[3:0]				RPTR[11:0]											
96	Reserved				BD[11:0]											
112	Reserved		BA[13:0]													

Table 18-9: Synchronous CDT Entry Field Definitions

Field	Description	Details	Access
BA	Buffer Base Address	Can start at any byte in the 16k DBR	RW
BD	Buffer Depth	BD = size of buffer in bytes 1. Buffer end address = BA + BD. BD = 4 m bpf 1 where: m = frames per sub-buffer (for MFE = 0, m = 1) bpf = bytes per frame.	RW
RPTR	Read Pointer	Software initializes to 0, hardware updates. Counts the read address offset within a buffer. DMA read address = BA + RPTR.	RWU
WPTR	Write Pointer	Software initializes to 0, hardware updates. Counts the write address offset within a buffer. DMA read address = BA + WPTR.	RWU
RSBC	Read Sub-buffer Counter	Software initializes to 0, hardware updates. Counts the read sub-buffer offset. DMA uses for pointer management.	RWU
WSBC	Write Sub-buffer Counter	Software initializes to 0, hardware updates. Counts the write sub-buffer offset. DMA uses for pointer management.	RWU
RSTS	Read Status	Software initializes to 0, hardware updates. RSTS States: xxx0 = normal operation (no mute) xxx1 = normal operation (mute) xx0x = idle	RWU

Table 18-9: Synchronous CDT Entry Field Definitions (Continued)

Field	Description	Details	Access
WSTS	Write Status	Software initializes to 0, hardware updates. WSTS States: xxx0 = normal operation (no mute) xxx1 = normal operation (mute) xx0x = idle 1xxx = command protocol error	RWU
Reserved. Software writes a 0 to all these bits when the entry is initialized. Reserved bits are RO after initialization.			RWU

Isochronous Channel Descriptors

The format and field definitions for a synchronous CDT entry are shown in the *Isochronous Entry Formats* and *Isochronous CDT Entry Field Definitions* tables.

Table 18-10: Isochronous Entry Formats

Bit Offset	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	Reserved															
16	Reserved															
32	Reserved								BS[8:0]							
48	Reserved															
64	WSTS[3:0]				WPTR[12:0]											
80	RSTS[3:0]				RPTR[12:0]											
96	Reserved				BD[12:0]											
112	BF	Rsvd	BA[13:0]													

Table 18-11: Isochronous CDT Entry Field Definitions

Field	Description	Details	Access
BA	Buffer Base Address	Can start at any byte in the 16k DBR.	RW
BD	Buffer Depth	BD = size of buffer in bytes 1. Buffer end address = BA + BD. Isochronous buffers must be large enough to hold at least 3 blocks (packets) of data. BD Must be an integer multiple of blocks.	RW

Table 18-11: Isochronous CDT Entry Field Definitions (Continued)

Field	Description	Details	Access
BF	Buffer Full	Software initializes to 0, hardware updates. DMA write hardware sets BF when the buffer is full. DMA read hardware clears BF when the buffer is empty. BF is valid only when buffer is full or empty, otherwise ignore.	RWU
BS	Block Size	BS defines when to begin the DMA to the data buffer. BS = buffer block size in bytes 1. For RX channels, the DMA writes start when the number of empty bytes in the data buffer the block size. For TX channels, the DMA reads start when the number of valid bytes in the data buffer the block size.	RWU
RPTR	Read Pointer	Software initializes to 0, hardware updates. Counts the read address offset within a buffer. DMA read address = BA + RPTR.	RWU
WPTR	Write Pointer	Software initializes to 0, hardware updates. Counts the write address offset within a buffer. DMA write address = BA + WPTR.	RWU
RSTS	Read Status	Software initializes to 0, hardware updates. RSTS States: xx1 = active xx0 = idle	RWU
WSTS	Write Status	Software initializes to 0, hardware updates. WSTS States: xxx0 = active xxx1 = idle xx0x = command protocol error 1xxx = buffer overflow (FCE = 0 only)	RWU
Reserved. Software writes a 0 to all these bits when the entry is initialized. Reserved bits are RO after initialization.			RWU

Asynchronous/Control Channel Descriptors

The format and field definitions for an Asynchronous/Control CDT entry are shown in the *Asynchronous/Control CDT Entry Format* and *Asynchronous/Control CDT Entry Field Definitions* tables.

Table 18-12: Asynchronous/Control CDT Entry Format

Bit Offset	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	WPC[4:0]					Reserved										
16	RPC[4:0]					Reserved										
32	Rsvd	WPC[7:5]			Reserved											
48	Rsvd	RPC[7:5]			Reserved											
64	WSTS[3:0]				WPTR[11:0]											
80	RSTS[3:0]				RPTR[11:0]											
96	RSTS[4]	WSTS[4]	Rsvd		BD[11:0]											
112	Rsvd		BA[13:0]													

Table 18-13: Asynchronous/Control CDT Entry Field Definitions

Field	Description	Details	Access
BA	Buffer Base Address	Can start at any byte in the 16k DBR.	RW
BD	Buffer Depth	BD = size of buffer in bytes 1. Buffer end address = BA + BD. BD = max packet length 1.	RW
RPC	Read Packet Count	Software initializes to 0, hardware updates. Used with RPC, RPTR and WPTR to determine if the buffer is empty or full.	RWU
WPC	Write Packet Count	Software initializes to 0, hardware updates. Used with RPC, RPTR and WPTR to determine if the buffer is empty or full.	RWU
RPTR	Read Pointer	Software initializes to 0, hardware updates. Counts the read address offset within a buffer. DMA read address = BA + RPTR.	RWU
WPTR	Write Pointer	Software initializes to 0, hardware updates. Counts the write address offset within a buffer. DMA read address = BA + WPTR.	RWU
RSTS	Read Status	Software initializes to 0, hardware updates. RSTS States: x0x00 = idle xx1xx = <i>ReceiverProtocolError</i> response received from RX device 1xxxx = <i>ReceiverBreak</i> command received from RX device	RWU

Table 18-13: Asynchronous/Control CDT Entry Field Definitions (Continued)

Field	Description	Details	Access
WSTS	Write Status	Software initializes to 0, hardware updates. Status States (only valid for DMA pointers associated with the MLB block, not HBI block pointers): xxx0 = idle xxx1 = command protocol error detected xx0x = <i>AsyncBreak/ControlBreak</i> command received from TX device	RWU
Reserved. Software writes a 0 to all these bits when the entry is initialized. Reserved bits are RO after initialization.			RWU

AHB Descriptor Table (ADT)

The AHB block manages data exchange between local channel data buffers within MLB module and the system memory buffer. To support system memory buffering, a ping-pong memory structure is implemented on a per channel basis using 128-bit descriptors for AHB Descriptor Table (ADT) entries. The [Table 18-4 CTR Address Mapping](#) table provides a complete address map of the CTR, including the location of the ADT.

Each logical channel is assigned a separate 128-bit descriptor, defining the data buffers in the system memory used by the DMA interface for that channel. The descriptors are stored at fixed addresses in the CTR as described in previous section. The *ADT Field Definitions* table provides an overview of field definitions for ADT entries.

Table 18-14: ADT Field Definitions

Field	No. of Bits	Description	Access
CE	1	Channel Enable. 0 = Disabled	RW, U
LE	1	Endianness Select. 0 = Big Endian, 1 = Little Endian	RW
PG	1	Page pointer. Software initializes to 0, hardware writes thereafter. 0 = Ping buffer, 1 = Pong buffer	RW, U
RDY1	1	Buffer Ready bit for ping buffer page. 0 = Not ready, 1 = Ready	RW
RDY2	1	Buffer Ready bit for pong buffer page. 0 = Not ready, 1 = Ready	RW
DNE1	1	Buffer Done bit for ping buffer page. 0 = Not done, 1 = Done	R, U, c0
DNE2	1	Buffer Done bit for pong buffer page. 0 = Not done, 1 = Done	R, U, c0
ERR1	1	Error Response detected for ping buffer page. 0 = No error, 1 = Error	R, U, c0
ERR2	1	Error Response detected for pong buffer page. 0 = No error, 1 = Error	R, U, c0
PS1	1	Packet Start bit for ping buffer page. 0 = No packet start, 1 = Packet start Reserved for synchronous and isochronous channels.	RW, U both TX and RX
PS2	1	Packet Start bit for pong buffer page. 0 = No packet start, 1 = Packet start Reserved for synchronous and isochronous channels.	RW, U both TX and RX

Table 18-14: ADT Field Definitions (Continued)

Field	No. of Bits	Description	Access
MEP1	1	Most Ethernet Packet indicator for ping buffer page. 0 = Not MEP, 1 = MEP. MEP1 only valid for the first page of a segmented buffer. Reserved for control synchronous and isochronous channels.	Rsvd for TX, R, U, c0 for RX
MEP2	1	Most Ethernet Packet indicator for pong buffer page. 0 = Not MEP, 1 = MEP. MEP2 only valid for the first page of a segmented buffer. Reserved for control synchronous and isochronous channels.	Rsvd for TX, R, U, c0 for RX
BD1	11 to 13	Buffer Depth for ping buffer page. 11 or 12 bits for asynchronous and control channels. 13 bits for synchronous and isochronous channels.	RW
BD2	11 to 13	Buffer Depth for pong buffer page. 11 or 12 bits for asynchronous and control channels. 13 bits for synchronous and isochronous channels.	RW
BA1	32	Buffer Base Address for ping buffer page	RW
BA2	32	Buffer Base Address for pong buffer page.	RW
Reserved	varies	Reserved. Software writes a 0 to all these bits when the entry is initialized. Reserved bits are RO after initialization.	RW, U

The *Ping-Pong System Memory Structure* figure shows that this system memory structure is similar for all channel types and shows the relationship between the BAn, BDn, and PG descriptor fields.

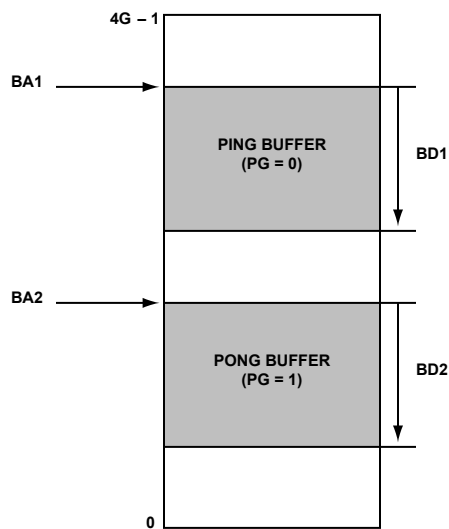


Figure 18-4: Ping-Pong System Memory Structure

Each ADT entry (also referred to as a *Channel Descriptor*) holds a 32-bit BAn field which defines the start of each ping or pong buffer within system memory. The BDn field is used to indicate the size for the respective ping or pong page. The maximum size is 2k entries for asynchronous and control channels and 8k entries for isochronous and synchronous channels.

Synchronous Channel Descriptors

The synchronous buffering scheme allows each ping or pong buffer to contain a single frame or a multiple number of frames. For this reason, the synchronous buffer depth (BDn) must be defined in terms of an integer number (n), frames per sub-buffer (m) and bytes per frame (bpf) of data (for example $BDn = n \cdot m \cdot bpf$). The *Synchronous ADT Entry Format* table shows the format for a synchronous ADT entry. The field definitions are defined in the *ADT Field Definitions* table. Each synchronous channel buffer can be up to 8k-bytes deep.

Table 18-15: Synchronous ADT Entry Format

Bit Offset	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	CE	LE	PG	Reserved												
16	Reserved															
32	RDY1	DNE1	ERR1	BD1[12:0]												
48	RDY2	DNE2	ERR2	BD2[12:0]												
64	BA1[15:0]															
80	BA1[31:16]															
96	BA2[15:0]															
112	BA2[31:16]															

Isochronous Channel Descriptors

The isochronous buffering scheme allows each ping or pong buffer to contain a single block or a multiple number of blocks. For this reason, the isochronous buffer depth (BDn) must be defined in terms of an integer number (n) and block size (BS) (for example $BDn = n \cdot (BS + 1)$).

The *Isochronous ADT Entry Format* table shows the format for an isochronous ADT entry. The field definitions are defined in the *ADT Field Definitions* table. Each isochronous channel buffer can be up to 8k-bytes deep.

Table 18-16: Isochronous ADT Entry Format

Bit Offset	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	CE	LE	PG	Reserved												
16	Reserved															
32	RDY1	DNE1	ERR1	BD1[12:0]												
48	RDY2	DNE2	ERR2	BD2[12:0]												
64	BA1[15:0]															
80	BA1[31:16]															
96	BA2[15:0]															
112	BA2[31:16]															

Asynchronous and Control Channel Descriptors

Every asynchronous and control packet adheres to the Port Message Protocol (PMP), which designates the first two bytes of each packet as the packet length (PML). Each packet must be no more than 2048-bytes (PML 2048).

Software must set the buffer ready bit (RDYn) for each buffer as it programs the DMA. As hardware processes each buffer, it sets the done bit (DNEn) and generates an interrupt to inform HC. When hardware finishes processing a buffer, it can begin processing another buffer if RDYn is set. The application is responsible for setting up and configuring the channel buffer descriptor prior to every DMA access on the channel.

Two packet buffering modes are supported by hardware for programming the DMA, single-packet mode (MLB_ACTL.MPB =0) and multiple-packet mode (MLB_ACTL.MPB =1). The MPB is written prior to enabling the channel DMA.

Single Packet Mode. The single-packet mode asynchronous and control buffering scheme supports a maximum of one packet per buffer (for example, ping or pong). Both non-segmented and segmented data packets are allowed while using single-packet mode. Non-segmented packets are exchanged when only one buffer (for example, ping or pong) is needed for packet transfer. Segmented packets are exchanged when a single packet is too long for one buffer and the packet must span multiple buffers.

The *Single-Packet Asynchronous and Control Entry Format* table shows the format for single-packet mode asynchronous and control ADT entries. The field definitions are defined in the *ADT Field Definitions* table.

Table 18-17: Single-Packet Asynchronous and Control Entry Format

Bit Offset	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	CE	LE	PG	Reserved												
16	Reserved															
32	RDY1	DNE1	ERR1	PS1	MEP1	BD1[10:0]										
48	RDY2	DNE2	ERR2	PS2	MEP2	BD2[10:0]										
64	BA1[15:0]															
80	BA1[31:16]															
96	BA2[15:0]															
112	BA2[31:16]															

Multiple Packet Mode. The multiple-packet mode asynchronous and control buffering scheme supports more than one packet per system memory buffer, as shown in the *Asynchronous/Control CDT Entry Format* table. Multiple-packet mode reduces the interrupt rate for packet channels at the cost of increasing buffering and latency.

For TX packet channels in multiple-packet mode, software sets the packet start bit (PSn) for every buffer. Setting PSn informs hardware that the first two bytes of the buffer contains the port message length (PML) of the first packet. After the first packet, hardware keeps track of where packets start and end within the current buffer. Software should not write to PSn while the buffer is active (RDYn = 1 and DNEn = 0). For TX packet channels, the buffer is done (DNEn = 1) when the last byte of the last packet in the buffer is read from system memory. Software should

set the buffer depth to contain the exact number of complete packets for that buffer. Segmented buffers are not supported for TX packet channels in multiple-packet mode.

NOTE: The PS1 and PS2 bits are only valid for TX channels. Set PS1 and PS2 = 1 at the start of the buffer.

Table 18-18: Multiple-Packet Asynchronous and Control Entry Format

Bit Offset	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	CE	LE	PG	Reserved												
16	Reserved															
32	RDY1	DNE1	ERR1	PS1	BD1[11:0]											
48	RDY2	DNE2	ERR2	PS2	BD2[11:0]											
64	BA1[15:0]															
80	BA1[31:16]															
96	BA2[15:0]															
112	BA2[31:16]															

Interrupt Interface Block

The interrupt interface raises an interrupt when specific changes to HBI channel descriptors occur, including:

- For asynchronous and control read/write channels:
 - a packet is available to read in the channel buffer, or
 - sufficient empty space is available in the channel buffer to accept a requested packet write
- For isochronous read/write channels:
 - the number of valid bytes in the channel buffer exceeds the block size, or
 - the number of empty bytes in the channel buffer exceeds the block size

Operating Modes

The following sections describe the operating modes of the MLB interface. The channel type selection enables the logical channels to operate in synchronous, asynchronous, isochronous, or control channels.

NOTE: The logical channels can be any combination of channel type (for example synchronous, asynchronous, or control) and direction (transmit or receive).

Isochronous Data Exchange

An isochronous HBI channel is initially opened and synchronized with HCMD0.CMD[2:0]= 010. For isochronous channels, no further synchronization is required from the HBI perspective; however, an optional resynchronization command is available for HC flexibility. Setting HCMD0.CMD[2:0]= 011 reinitializes the address pointer within

the data buffer, ensuring that subsequent data exchange with the channel is aligned at an isochronous packet boundary. When the HC must close an isochronous channel before it has read or written an entire data packet, setting HCMD0.CMD[2:0]= 000 reopens the channel without synchronizing the address pointer in the buffer. This action allows reading and writing to continue where the HC previously stopped.

For isochronous data transmission, the *Exchanging Isochronous Data on an HBI Channel* figure shows the flow control.

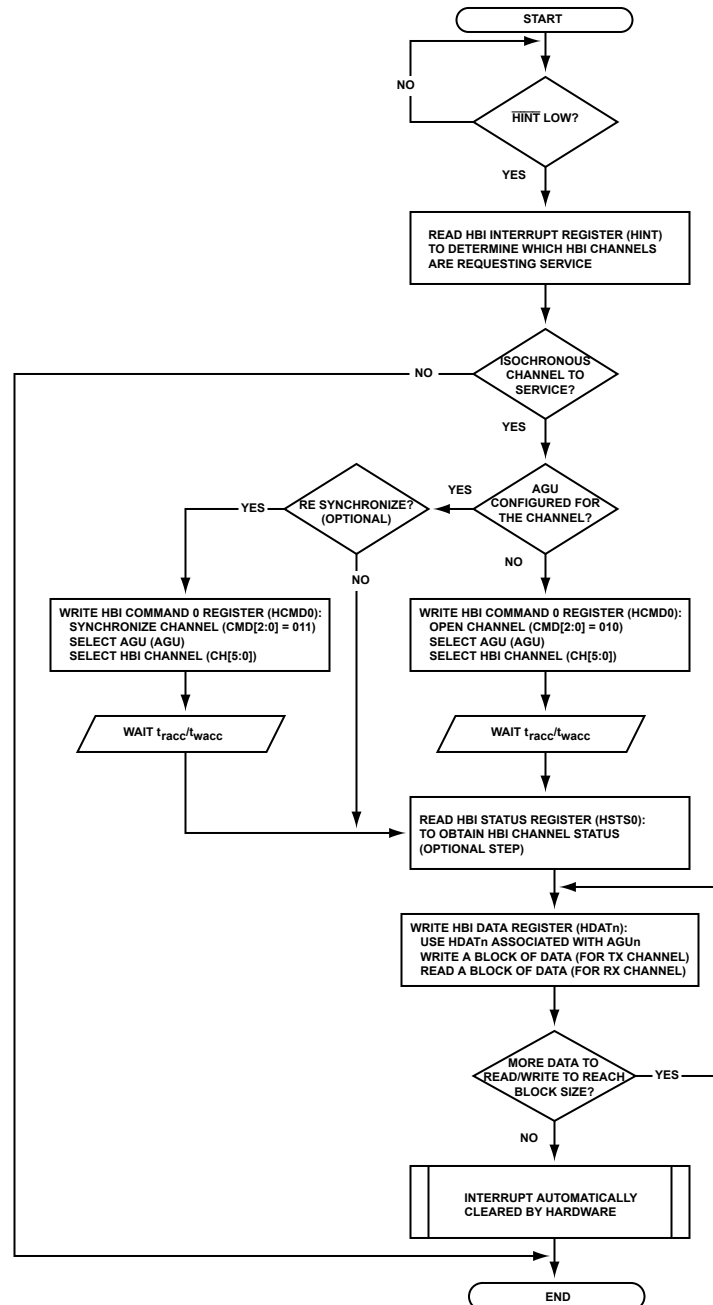


Figure 18-5: Exchanging Isochronous Data on an HBI Channel

Asynchronous and Control Data Exchange

An asynchronous or control HBI channel is initially opened using $\text{HCMD0.CMD}[2:0] = 010$. Occasionally, the HC may need to close a packet channel before it has completed writing or reading the current packet of data (for example, the HC needs to use this AGU to service another HBI channel). In this case, the HC can reopen the previous channel with $\text{HCMD0.CMD}[2:0] = 000$. This situation allows the HC to continue writing or reading a packet from the point it left off. In this situation, the PML is already known, and the packet length is not reread or rewritten by the HC.

During the reading of a packet, if the HC sets $\text{HCMD0.CMD}[2:0] = 010$ for the channel before the last byte of the packet is read, an internal hardware pointer is reset to the beginning of the packet buffer. This situation requires that the HC reread the PML (from HSTS1) and reread the packet data (from HDATn) from the beginning. In the same manner, if the HC resets $\text{HCMD0.CMD}[2:0] = 010$ for the channel before the last byte of a packet is written, an internal hardware pointer is reset to the beginning of the packet buffer. This situation requires that the HC rewrite the PML (to HCMD1) and rewrite the packet data (to HDATn) from the beginning. Any previous packet data in the buffer is overwritten.

Frame synchronization is not supported for asynchronous channels.

Synchronous Data Exchange

The MLB core provides two modes of operation; standard and multi-frame per sub-buffer which provide flexibility for implementing synchronous channels. Channels configured for standard mode require less buffer space but have higher interrupt rates and more stringent latency requirements. Channels setup for each multi-frame per sub-buffer mode requires more buffer space, but has lower interrupt rates and less stringent latency requirements.

To set up a channel in multi-frame per sub-buffer mode:

1. Program the MLB_CTL0.FCNT bit field to select the number of frames per sub-buffer.
2. Program the CAT to enable multi-frame sub-buffering ($\text{MFE} = 1$) for each channel.
3. Set the buffer depth in the CDT: $\text{BD} = 4 \times m \times \text{bpf}$ where: m = frames per sub-buffer, bpf = bytes per frame.
4. Repeat for additional synchronous channels

A sample synchronous data buffer is shown in the *Synchronous Data Buffer Structure* figure. Each data buffer contains four sub-buffers and each sub-buffer contains space for 1 to 64 frames of data, determined by the MLB_CTL0.FCNT bits.

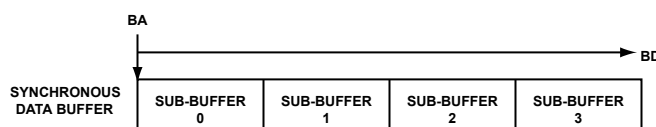


Figure 18-6: Synchronous Data Buffer Structure

Data Transfer

Two modes of operation are supported for transferring channel data between the MLB and internal memory. DMA allows the multi-channel DMA engine to manage data transfers without core intervention. Core driven mode (I/O mode) allows software to manage the transfer of data between MLB and internal memory.

NOTE: All hardware channels must use the same data transfer method. Mixed mode operation where hardware channels operate in both I/O mode and DMA mode is not supported.

DMA

The processor supports DMA mode which uses only INCR8, INCR4 and SINGLE beat bursts. Program the `MLB_ACTL.DMAMODE = 1` to use DMA mode.

Programming Model

This section provides general guidelines for programming the MediaLB interface.

Channel Initialization

The software flow required to initialize a channel must be performed to ensure proper operation.

Configure the Hardware

1. Initialize the CTR and registers.
 - a. Set all the CTR (CAT, CDT, and ADT) bits to 0.
 - b. Set all bits of all registers to 0.
2. Configure the MediaLB interface.
 - a. Select 3-pin operation by setting the `MLB_CTL0.PEN = 0`.
 - b. Select MediaLB clock speed via `MLB_CTL0.CLK`.
 - c. Set MediaLB enable via `MLB_CTL0.EN`.
3. Configure the HBI interface.
 - a. Set `MLB_HCMR0` and `MLB_HCMR1 = 0xFFFFFFFF` to activate all channels.
 - b. Set the HBI enable bit: `MLB_HCTL.EN = 1`.

Program the CAT and the CDT

1. Initialize all bits of the CAT to 0.

2. Select a logical channel: $N = 1 - 63$.
3. Program the CDT for channel N .
 - a. Set the 14-bit base address (BA)
 - b. Set the 12-bit or 13-bit buffer depth (BD): BD = buffer depth in bytes 1
 - For synchronous channels: $(BD + 1) = 4 \text{ frames per sub-buffer (m) bytes-per-frame (bpf)}$
 - For isochronous channels: $(BD + 1) \bmod (BS + 1) = 0$
 - For asynchronous channels: (BD + 1) max packet length (1024 for a MOST Data Packet (MDP)
 - 1536 for a MOST Ethernet Packet (MEP)
 - For control channels: (BD + 1) max packet length (64)
4. Program the CAT for the inbound DMA.
 - a. For TX channels (to MediaLB) HBI is the inbound DMA
 - b. For RX channels (from MediaLB) MediaLB is the inbound DMA
 - c. Set the channel direction: $RNW = 0$
 - d. Set the channel type: $CT[2:0] = 010$ (asynchronous), 001 (control), 011 (isochronous), or 000 (synchronous)
 - e. Set the connection label: $CL[5:0] = N$
 - f. If $CT[2:0] = 000$ (synchronous), set the mute bit ($MT = 1$)
 - g. Set the channel enable: $CE = 1$
 - h. Set all other bits of the CAT to 0
5. Program the CAT for the outbound DMA.
 - a. For TX channels (to MediaLB) HBI is the outbound DMA
 - b. For RX channels (from MediaLB) MediaLB is the outbound DMA
 - c. Set the channel direction: $RNW = 1$
 - d. Set the channel type: $CT[2:0] = 010$ (asynchronous), 001 (control), 011 (isochronous), or 000 (synchronous)
 - e. Set the channel label: $CL[5:0] = N$
 - f. If $CT[2:0] = 000$ (synchronous), set the mute bit ($MT = 1$)
 - g. Set the channel enable: $CE = 1$
 - h. Set all other bits of the CAT to 0
6. Repeat steps 2 through 5 to initialize all logical channels.

Program the ADT

1. Initialize all bits of the ADT to 0
2. Select a logical channel: $N = 1 - 63$
3. Program the AMBA AHB block ping page for channel N
 - a. Set the 32-bit base address (BA1)
 - b. Set the 11-bit buffer depth (BD1): $BD1 = \text{buffer depth in bytes} - 1$
 - For synchronous channels: $(BD1 + 1) = n \text{ frames per sub-buffer (m) bytes-per-frame (bpf)}$
 - For isochronous channels: $(BD1 + 1) \bmod (BS + 1) = 0$
 - For asynchronous channels: $5 (BD1 + 1) \leq 4096$ (max packet length)
 - For control channels: $5 (BD1 + 1) \leq 4096$ (max packet length)
 - c. For asynchronous and control Tx channels set the packet start bit (PS1) when the page contains the start of the packet
 - d. Clear the page done bit (DNE1)
 - e. Clear the error bit (ERR1)
 - f. Set the page ready bit (RDY1)
4. Program the AMBA AHB block pong page for channel N
 - a. Set the 32-bit base address (BA2)
 - b. Set the 11-bit buffer depth (BD2): $BD2 = \text{buffer depth in bytes} - 1$
 - For synchronous channels: $(BD2 + 1) = n \text{ frames per sub-buffer (m) bytes-per-frame (bpf)}$
 - For isochronous channels: $(BD2 + 1) \bmod (BS + 1) = 0$
 - For asynchronous channels: $5 (BD2 + 1) \leq 4096$ (max packet length)
 - For control channels: $5 (BD2 + 1) \leq 4096$ (max packet length)
 - c. For asynchronous and control TX channels set the packet start bit (PS2) when the page contains the start of the packet
 - d. Clear the page done bit (DNE2)
 - e. Clear the error bit (ERR2)
 - f. Set the page ready bit (RDY2)
5. Select Big Endian ($LE = 0$) or Little Endian ($LE = 1$)
6. Select the active page: $PG = 0$ (ping), $PG = 1$ (pong)

7. Set the channel enable (CE) bit for all active logical channels
8. Repeat steps 2 through 7 for all active logical channels.

Service

After initialization, each channel will require periodic servicing. Use the procedures in the following sections to service DMA and MLB interrupts and to poll for MLB system commands.

Servicing the DMA Channel Interrupts

1. Program the [MLB_ACMR0/MLB_ACMR1](#) registers to enable interrupts from all active DMA channels.
2. Select the status clear method. `MLB_ACTL.SCE = 0` (hardware clears on read), `MLB_ACTL.SCE = 1` (software writes a 1 to clear).
3. Select 1 or 2 interrupt signals. Configure the `MLB_ACTL.SMX` bit = 0 (one interrupt for channels 0 through 31 and another interrupt for channels 32 through 63 on). Configure the `MLB_ACTL.SMX` bit = 1 (single interrupt for all channels).
4. Wait for an interrupt.
5. Read the [MLB_ACSR0/MLB_ACSR1](#) registers to determine which channel or channels are causing the interrupt.
6. If the `MLB_ACTL.SCE` bit = 1, write the results of step 5 back to the [MLB_ACSR0](#) and [MLB_ACSR1](#) registers to clear the interrupt.
7. Select a logical channel ($N = 0 - 63$) with an interrupt to service.
8. Read the ADT entry for channel N to:
 - a. Determine the active page (ping or pong) via the PG bit
 - b. Determine which page(s) are done via the DNEn bits
 - c. Determine which channels encountered an AHB error via the ERRn bit
 - d. Determine which asynchronous and control Rx channel pages contain a packet start via the PSn bit (extract the PML)
9. Repeat steps 6 through 8 for all channels with pending interrupts
10. Repeat steps 4 through 9 while there are active channels.

Servicing the MediaLB Status Interrupts

1. Select the MediaLB channel status register ([MLB_MS0/MLB_MS1](#)) to be cleared by software, writing a 0 to the appropriate bits.

2. Program the [MLB_MIEN](#) register to enable protocol error interrupts for all active MediaLB channels. (MLB_MIEN.CTXPE =1, MLB_MIEN.CRXPE =1, MLB_MIEN.ATXPE =1, MLB_MIEN.ARXPE =1, MLB_MIEN.SYNCPE =1, and MLB_MIEN.ISOCPE =1).
3. Wait for an interrupt on the MLB_INT0/MLB_INT1 signal.
4. Read the [MLB_MS0/MLB_MS1](#) registers to determine which channel(s) are causing the interrupt.
5. Read the RSTS/WSTS of the appropriate CDT(s) to determine the interrupt type.
6. Clear the RSTS/WSTS errors to ensure that the status of channel operations is reflected in the register:

Option	Description
For synchronous RX channels	WSTS[3] = 0
For synchronous TX channels	RSTS[3] = 0
For isochronous RX channels	WSTS[2:1] = 00
For isochronous TX channels	RSTS[2:1] = 00
For asynchronous and control RX channels	WSTS[4] = 0 and WSTS[2] = 0
For asynchronous and control TX channels	RSTS[4] = 0 and RSTS[2] = 0

Polling for MediaLB System Commands

The MediaLB System status ([MLB_MSS](#)) register is used to detect a system command received from the MediaLB controller. The processor's peripheral automatically sends the appropriate system response to the MediaLB controller. The procedure for the application is:

1. The application periodically polls the [MLB_MSS](#) register.
2. Clear by writing a 0 to the appropriate bit in the [MLB_MSS](#) register after the application finishes the service.
3. If MLB_MSS.SWSYSCMD = 1, read the [MLB_MSD](#) register to receive the system data sent from MediaLB controller.

ADSP-21560_21561_21564_21568 MLB Register Descriptions

MediaLB Device Interface Macro 2 (MLB) contains the following registers.

Table 18-19: ADSP-21560_21561_21564_21568 MLB Register List

Name	Description
MLB_ACMR0	Peripheral Channel Mask 0 Register
MLB_ACMR1	Peripheral Channel Mask 1 Register
MLB_ACSR0	Peripheral Channel Status 0 Register
MLB_ACSR1	Peripheral Channel Status 1 Register

Table 18-19: ADSP-21560_21561_21564_21568 MLB Register List (Continued)

Name	Description
MLB_ACTL	Bus Control Register
MLB_CTL0	MediaLB Control 0 Register
MLB_CTL1	Control 1 Register
MLB_GCTL	MLB Global Control Register
MLB_HCBR0	HBI Channel Busy 0 Register
MLB_HCBR1	HBI Channel Busy 1 Register
MLB_HCER0	HBI Channel Error 0 Register
MLB_HCER1	HBI Channel Error 1 Register
MLB_HCMR0	HBI Channel Mask 0 Register
MLB_HCMR1	HBI Channel Mask 1 Register
MLB_HCTL	HBI Control Register
MLB_MADR	Memory Interface Address Register
MLB_MCTL	Memory Interface Control Register
MLB_MDAT0	Memory Interface Control Data 0 Register
MLB_MDAT1	Memory Interface Control Data 1 Register
MLB_MDAT2	Memory Interface Control Data 2 Register
MLB_MDAT3	Memory Interface Control Data 3 Register
MLB_MDWE0	Memory Interface Control Data Write Enable 0 Register
MLB_MDWE1	Memory Interface Control Data Write Enable 1 Register
MLB_MDWE2	Memory Interface Control Data Write Enable 2 Register
MLB_MDWE3	Memory Interface Control Data Write Enable 3 Register
MLB_MIEN	Interrupt Enable Register
MLB_MS0	Channel Status 0 Register
MLB_MS1	Channel Status 1 Register
MLB_MSD	System Data Register
MLB_MSS	System Status Register

Peripheral Channel Mask 0 Register

The `MLB_ACMR0` register allows control over which channel(s) generate interrupts on `MLB_INT[1:0]`. All of the bits in this register default to 0 (masked) so the HC must initially write to the `MLB_ACMR0` register to enable interrupts. Each bit of this register is read/write accessible.

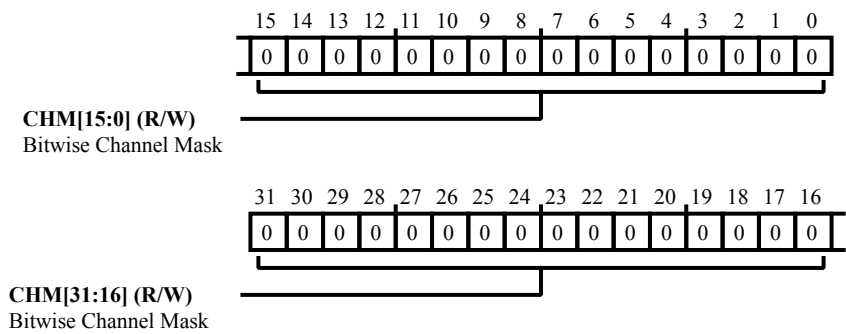


Figure 18-7: MLB_ACMR0 Register Diagram

Table 18-20: MLB_ACMR0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	CHM	Bitwise Channel Mask. The <code>MLB_ACMR0.CHM</code> bit field masks or unmasks channels 31-0.
		0 Mask interrupt for Channel 31:0 (bitwise; all channels shown)
		4294967295 Unmask interrupt for Channel 31:0 (bitwise; all channels shown)

Peripheral Channel Mask 1 Register

The [MLB_ACMR1](#) register allows control over which channel(s) generate interrupts on MLB_INT[1:0]. All of the bits in this register default to 0 (masked) so the HC must initially write to the [MLB_ACMR1](#) register to enable interrupts. Each bit of this register is read/write accessible.

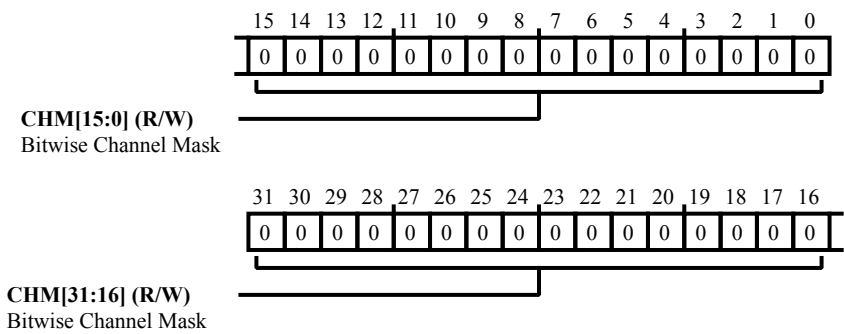


Figure 18-8: MLB_ACMR1 Register Diagram

Table 18-21: MLB_ACMR1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	CHM	Bitwise Channel Mask. The <code>MLB_ACMR1.CHM</code> bit field masks or unmasks channels 63-32.
		0 Mask interrupt for Channel 63:32 (bitwise; all channels shown)
		4294967295 Unmask interrupt for Channel 63:32 (bitwise; all channels shown)

Peripheral Channel Status 0 Register

The `MLB_ACSR0` register contains interrupt bits for each of the 64 physical channels. When a bit in this register is set, it indicates that the corresponding physical channel has an interrupt pending.

A peripheral interrupt is triggered when either `DNEn` or `ERRn` is set within the Bus Channel Descriptor. The HC is notified of the channel interrupt via `MLB_INT[1:0]`. When an interrupt occurs in `ACCUSER` (for channels 31 to 0) `MLB_INT[0]` is set. When an interrupt occurs in `MLB_ACSR1` (for channels 63 to 32) `MLB_INT[1]` is set.

Interrupts in the `MLB_ACSR0` and `MLB_ACSR1` registers can be optionally multiplexed onto a single interrupt signal, `MLB_INT[0]`, if the `MLB_ACTL.SMX` bit = 1.

If the `MLB_ACTL.SCE` bit =0, hardware automatically clears the interrupt bit(s) after the HC reads the peripheral channel status registers. Alternatively, if the `MLB_ACTL.SCE` bit =1, software must write a 1 to the appropriate bit(s) of the peripheral channel status registers to clear the interrupt(s).

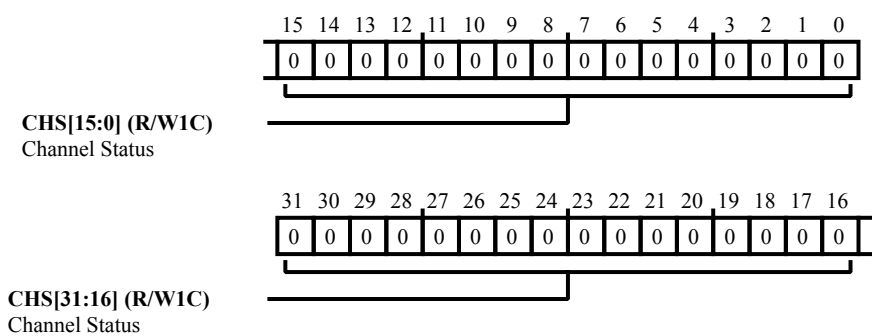


Figure 18-9: MLB_ACSR0 Register Diagram

Table 18-22: MLB_ACSR0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration	
31:0 (R/W1C)	CHS	Channel Status. The <code>MLB_ACSR0.CHS</code> bit field indicates channel status for channels 31-0.	
		0	No interrupt on Channel 31:0 (bitwise; all channels shown)
		4294967295	Interrupt on Channel 31:0 (bitwise; all channels shown)

Peripheral Channel Status 1 Register

The `MLB_ACSR1` register contains interrupt bits for each of the 64 physical channels. When a bit in this register is set, it indicates that the corresponding physical channel has an interrupt pending.

A peripheral interrupt is triggered when either `DNEn` or `ERRn` is set within the Bus Channel Descriptor. The HC is notified of the channel interrupt via `MLB_INT[1:0]`. When an interrupt occurs in `ACCUSER` (for channels 31 to 0) `MLB_INT[0]` is set. When an interrupt occurs in `MLB_ACSR1` (for channels 63 to 32) `MLB_INT[1]` is set.

Interrupts in the `MLB_ACSR1` and `MLB_ACSR0` registers can be optionally multiplexed onto a single interrupt signal, `MLB_INT[0]`, if the `MLB_ACTL.SMX` bit = 1.

If the `MLB_ACTL.SCE` bit = 0, hardware automatically clears the interrupt bit(s) after the HC reads the peripheral channel status registers. Alternatively, if the `MLB_ACTL.SCE` bit = 1, software must write a 1 to the appropriate bit(s) of peripheral channel status registers to clear the interrupt(s).

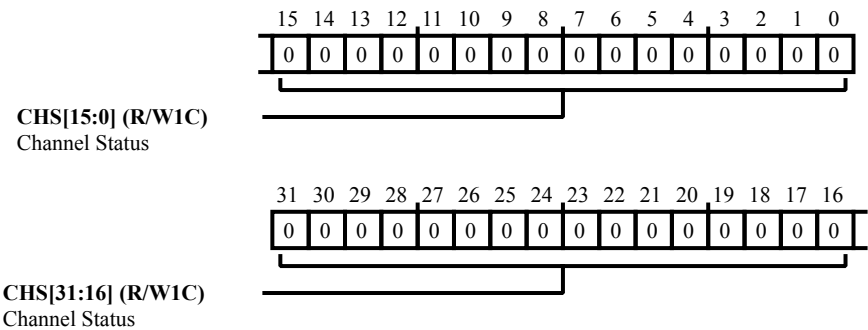


Figure 18-10: `MLB_ACSR1` Register Diagram

Table 18-23: `MLB_ACSR1` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W1C)	CHS	Channel Status. The <code>MLB_ACSR1.CHS</code> bit field indicates channel status for channels 63-32.
		0 No interrupt on Channel 63:32 (bitwise; all channels shown)
		4294967295 Interrupt on Channel 63:32 (bitwise; all channels shown)

Bus Control Register

The `MLB_ACTL` register is written by the HC to configure the block for channel interrupts. This register contains bits to select the packet buffering mode and the DMA mode. This register also contains bits that are used to multiplex channel interrupts onto a single interrupt signal and to select the method of clearing channel interrupts.

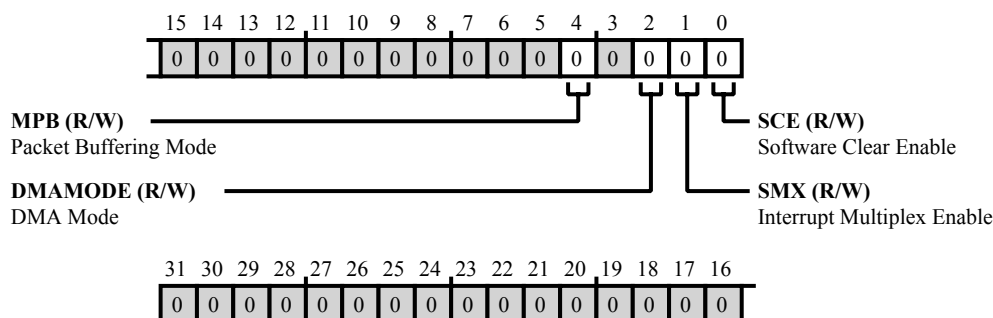


Figure 18-11: `MLB_ACTL` Register Diagram

Table 18-24: `MLB_ACTL` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
4 (R/W)	MPB	Packet Buffering Mode. The <code>MLB_ACTL.MPB</code> bit selects whether the buffering mode is single-packet or multiple-packet.
		0 single-packet mode
		1 multiple-packet mode
2 (R/W)	DMAMODE	DMA Mode. The <code>MLB_ACTL.DMAMODE</code> bit selects between DMA mode 1 and 0. DMA Mode 0 uses incrementing bursts of an unspecified length. This allows the block to perform single beat transfers as well as an incrementing (INCR) burst of unspecified length up to the maximum specified burst length (8 beats). DMA Mode 1 uses only INCR8, INCR4, and SINGLE beat bursts. The <code>hburst[2:0]</code> signal encodes the burst transfer used for a DMA operation.
		0 DMA Mode 0
		1 DMA Mode 1

Table 18-24: MLB_ACTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/W)	SMX	Interrupt Multiplex Enable. The MLB_ACTL.SMX bit selects whether ACSR0 generates an interrupt on MLB_INT[0] and ACSR1 generates an interrupt on MLB_INT[1] or ACSR0 and ACSR1 generate an interrupt on MLB_INT[0] only.
		0 ACSR0 generates an interrupt on MLB_INT[0]; ACSR1 generates an interrupt on MLB_INT[1]
		1 ACSR0 and ACSR1 generate an interrupts on MLB_INT[0] only
0 (R/W)	SCE	Software Clear Enable. The MLB_ACTL.SCE bit selects whether hardware clears the interrupt after a ACSRn register read or software clears the interrupt.
		0 Hardware clears interrupt after a ACSRn register read
		1 Software clears interrupt

MediaLB Control 0 Register

The `MLB_CTL0` register contains bit that enable the module and provide module status. Note that the maximum speed for ML3-pin mode is 1024 x FS

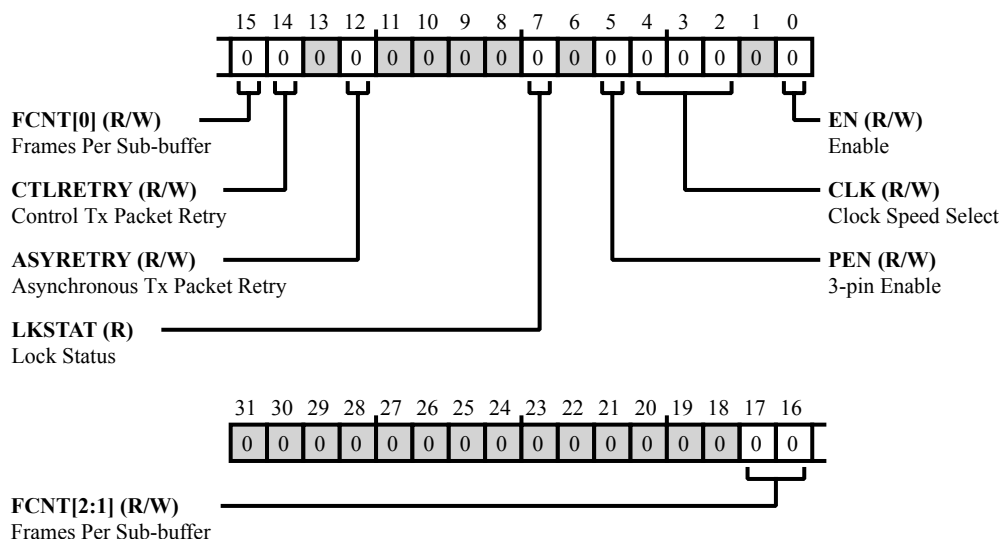


Figure 18-12: MLB_CTL0 Register Diagram

Table 18-25: MLB_CTL0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
17:15 (R/W)	FCNT	Frames Per Sub-buffer. The <code>MLB_CTL0 . FCNT</code> bit field configures the frames per sub-buffer on synchronous channels.
		0 1 Frame per sub-buffer (Operation is the same as Standard mode)
		1 2 frames per sub-buffer
		2 4 frames per sub-buffer
		3 8 frames per sub-buffer
		4 16 frames per sub-buffer
		5 32 frames per sub-buffer
		6 64 frames per sub-buffer
14 (R/W)	CTLRETRY	Control Tx Packet Retry. When the <code>MLB_CTL0 . CTLRETRY</code> bit is set, a control packet that is flagged with a Break or Protocol error by the receiver is retransmitted. When cleared, a control packet that is flagged with a Break or Protocol error by the receiver is skipped.

Table 18-25: MLB_CTL0 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
12 (R/W)	ASYRETRY	Asynchronous Tx Packet Retry. When the MLB_CTL0 . ASYRETRY bit is set, an asynchronous packet that is flagged with a Break or Protocol error by the receiver is retransmitted. When cleared, an asynchronous packet that is flagged with a Break or ProtocolError by the receiver is skipped.
7 (R/NW)	LKSTAT	Lock Status. When the MLB_CTL0 . LKSTAT bit is set (=0), the MediaLB block is synchronized to the incoming MediaLB frame with the following conditions. • If MLB_CTL1 . LOCK =0 (unlocked), MLB_CTL1 . LOCK is set after a FRAME-SYNC is detected at the same position for three consecutive frames. • If MLB_CTL1 . LOCK =1 (locked), MLB_CTL1 . LOCK is cleared after not receiving a FRAMESYNC at the expected time for two consecutive frames. In this case FRAMESYNC patterns occurring at locations other than the expected one are ignored.
5 (R/W)	PEN	3-pin Enable. The MLB_CTL0 . PEN bit configures the 3-pin mode.
		0 3-pin interface enabled
		1 invalid condition
4:2 (R/W)	CLK	Clock Speed Select. The MLB_CTL0 . CLK bit field sets the clock speed.
		0 256 x Fs (MLB_CTL.PEN = 0)
		1 512 x Fs (MLB_CTL.PEN = 0)
		2 1024 x Fs (MLB_CTL.PEN = 0)
0 (R/W)	EN	Enable. When the MLB_CTL0 . EN bit is set (=1), MediaLB clock, signal, and data are received and transmitted on the appropriate MediaLB pins.

Control 1 Register

The `MLB_CTL1` register contains bits that provide lock status and control system commands.

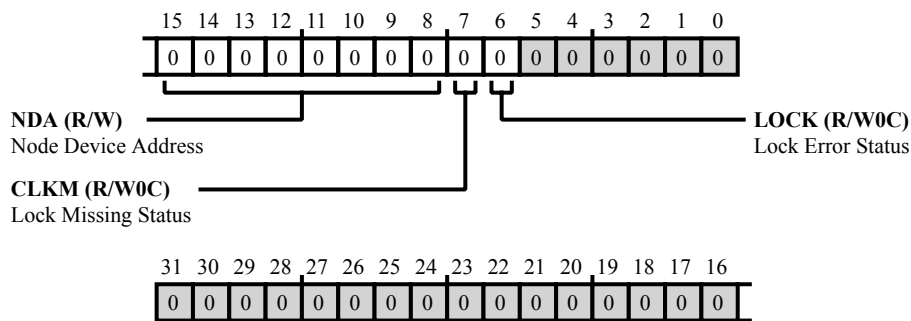


Figure 18-13: MLB_CTL1 Register Diagram

Table 18-26: MLB_CTL1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:8 (R/W)	NDA	Node Device Address. The <code>MLB_CTL1</code> . <code>NDA</code> bit field is used for system commands directed to individual MediaLB nodes.
7 (R/W0C)	CLKM	Lock Missing Status. The <code>MLB_CTL1</code> . <code>CLKM</code> bit is set when the MediaLB clock is not toggling at the pin. This bit is cleared by software.
6 (R/W0C)	LOCK	Lock Error Status. The <code>MLB_CTL1</code> . <code>LOCK</code> bit is set when the MediaLB is unlocked. This bit is cleared by software.

MLB Global Control Register

The `MLB_GCTL` register contains bits that manage the MLB clock.

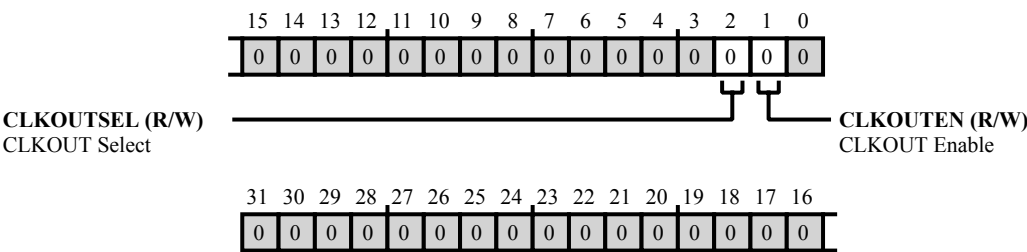


Figure 18-14: MLB_GCTL Register Diagram

Table 18-27: MLB_GCTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
2 (R/W)	CLKOUTSEL	CLKOUT Select. The <code>MLB_GCTL.CLKOUTSEL</code> bit selects MLB 3 pin clock
		0 MLB 3 pin clock is selected for <code>MLBCLKOUT</code>
1 (R/W)	CLKOUTEN	CLKOUT Enable.

HBI Channel Busy 0 Register

The HC can determine which channel(s) are busy by reading the [MLB_HCBR0](#) register. An HBI channel is busy if:

- it is currently loaded into one of the two AGUs
- the channel is enabled, CE = 1 from the Channel Allocation Table
- the DMA is active

When an HBI channel is busy, hardware may write back its local copy of the channel descriptor at any time. System software should not write a CDT descriptor for a channel that is busy. Only two HBI channels can be busy at any given time. Each bit of this register is read-only.

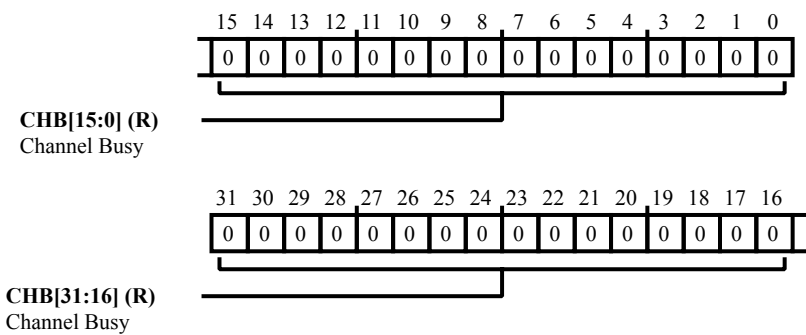


Figure 18-15: MLB_HCBR0 Register Diagram

Table 18-28: MLB_HCBR0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CHB	Channel Busy. The MLB_HCBR0 . CHB bit field contains the bitwise channel busy bit for channels 31:0. When a bit is cleared (=0) the channel is idle. When a bit is set (=1) the channel is busy.

HBI Channel Busy 1 Register

The HC can determine which channel(s) are busy by reading the [MLB_HCBR1](#) register. An HBI channel is busy if:

- it is currently loaded into one of the two AGUs
- the channel is enabled, CE = 1 from the Channel Allocation Table
- the DMA is active

When an HBI channel is busy, hardware may write back its local copy of the channel descriptor at any time. System software should not write a CDT descriptor for a channel that is busy. Only two HBI channels can be busy at any given time. Each bit of this register is read-only.

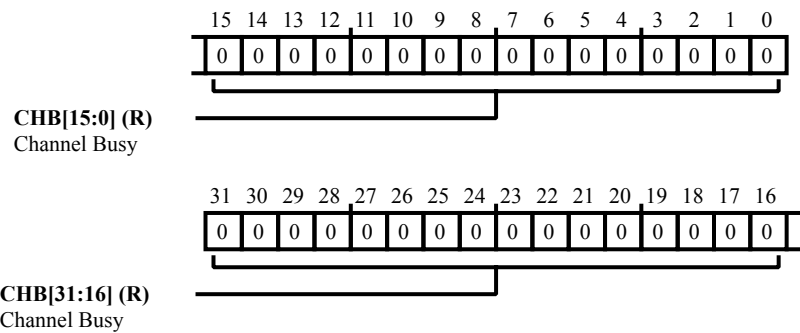


Figure 18-16: MLB_HCBR1 Register Diagram

Table 18-29: MLB_HCBR1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CHB	Channel Busy. The MLB_HCBR1 . CHB bit field contains the bitwise channel busy bit for channels 63:32. When a bit is cleared (=0) the channel is idle. When a bit is set (=1) the channel is busy.

HBI Channel Error 0 Register

The `MLB_HCER0` register indicates which channels (channels 31:0) have encountered fatal errors.

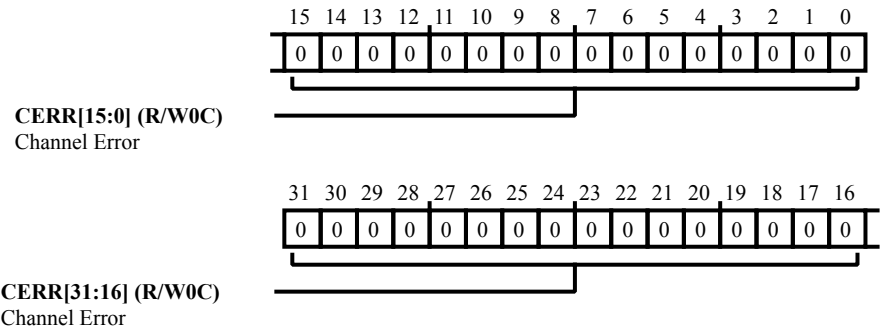


Figure 18-17: MLB_HCER0 Register Diagram

Table 18-30: MLB_HCER0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W0C)	CERR	Channel Error. The <code>MLB_HCER0.CERR</code> bit field reports bitwise errors for channels 31:0.

HBI Channel Error 1 Register

The `MLB_HCER1` register indicates which channel(s) have encountered fatal errors.

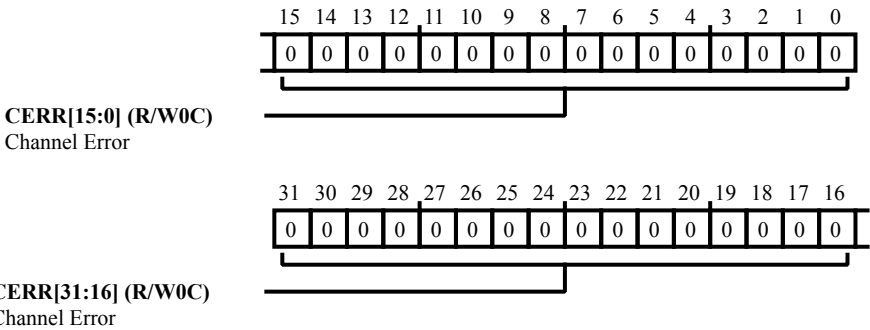


Figure 18-18: `MLB_HCER1` Register Diagram

Table 18-31: `MLB_HCER1` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W0C)	CERR	Channel Error. The <code>MLB_HCER1</code> . CERR bite field reports bitwise errors for channels 63:32.

HBI Channel Mask 0 Register

The `MLB_HCMR0` register controls which channels (for channels 31:0) are able to generate an HBI interrupt.

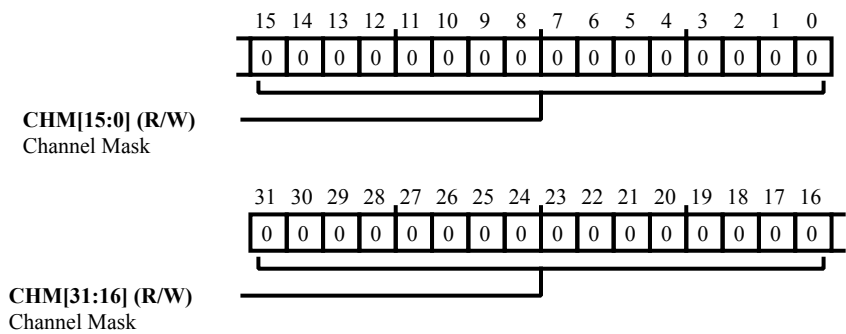


Figure 18-19: `MLB_HCMR0` Register Diagram

Table 18-32: `MLB_HCMR0` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	CHM	Channel Mask. The <code>MLB_HCMR0.CHM</code> bit field contains the bitwise channel mask bit for channels 31:0. When a bit is cleared (=0) the channel is masked. When set (=1) the channel is unmasked.

HBI Channel Mask 1 Register

The `MLB_HCMR1` register controls which channels (for channels 63:32) are able to generate an HBI interrupt.

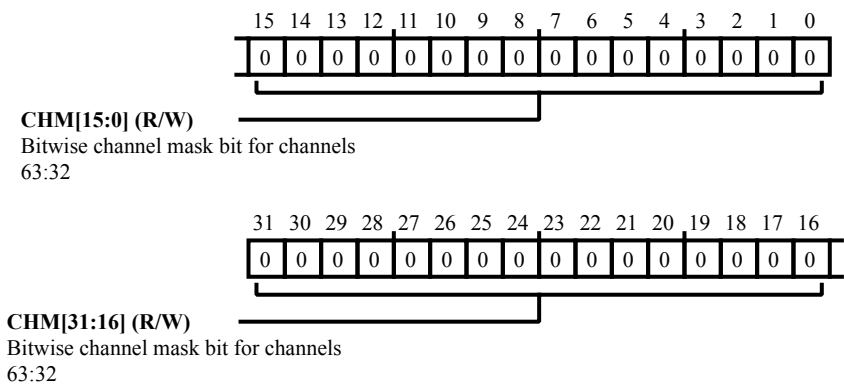


Figure 18-20: MLB_HCMR1 Register Diagram

Table 18-33: MLB_HCMR1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	CHM	Bitwise channel mask bit for channels 63:32. The <code>MLB_HCMR1.CHM</code> bit field contains the bitwise channel mask bit for channels 63:32. When a bit is cleared (=0) the channel is masked. When set (=1) the channel is unmasked.

HBI Control Register

The `MLB_HCTL` register controls and monitors general operation of the HBI block through the Address Generation Units) by reading and writing the register through the I/O interface. Each bit of this register is read/write.

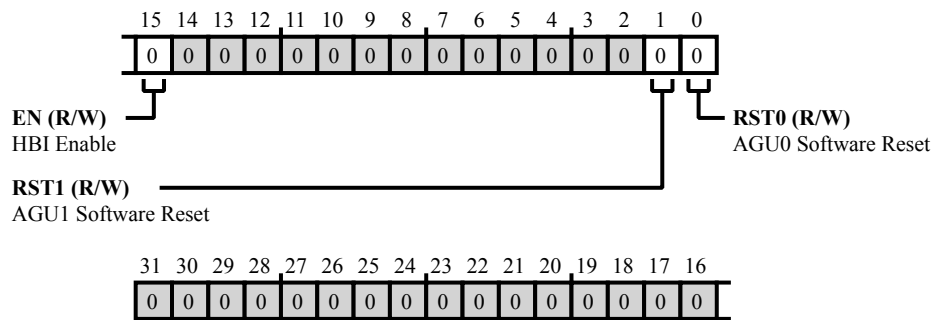


Figure 18-21: `MLB_HCTL` Register Diagram

Table 18-34: `MLB_HCTL` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W)	EN	HBI Enable. Setting the <code>MLB_HCTL.EN</code> bit enables the HBI.
1 (R/W)	RST1	AGU1 Software Reset. Setting the <code>MLB_HCTL.RST1</code> bit resets AGU1.
0 (R/W)	RST0	AGU0 Software Reset. Setting the <code>MLB_HCTL.RST0</code> bit resets AGU0.

Memory Interface Address Register

The `MLB_MADR` register contains bit fields that contain the Channel Table RAM (CTR) or Data Buffer RAM (DBR) addresses. It also contains bits that set target location and read-not-write parameters.

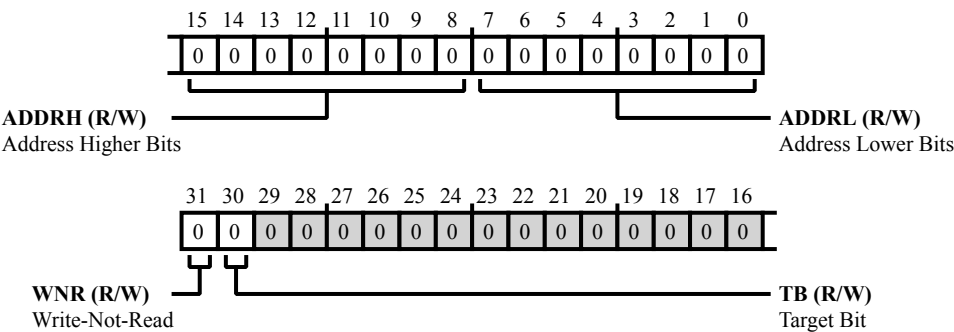


Figure 18-22: `MLB_MADR` Register Diagram

Table 18-35: `MLB_MADR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	WNR	Write-Not-Read. The <code>MLB_MADR.WNR</code> bit selects Write-Not-Read.
		0 Read
		1 Write
30 (R/W)	TB	Target Bit. The <code>MLB_MADR.TB</code> bit sets the target location.
		0 Channel Table RAM (CTR)
		1 Data Buffer RAM (DBR)
15:8 (R/W)	ADDRH	Address Higher Bits. The <code>MLB_MADR.ADDRH</code> bit field contains the DBR address of the 8-bit entry (bits [13:8]).
7:0 (R/W)	ADDRL	Address Lower Bits. The <code>MLB_MADR.ADDRL</code> bit field contains the CTR address of the 128-bit entry or the DBR address of the 8-bit entry (bits [7:0]).

Memory Interface Control Register

The `MLB_MCTL` register contains a bit that indicates that the data transfer is complete.

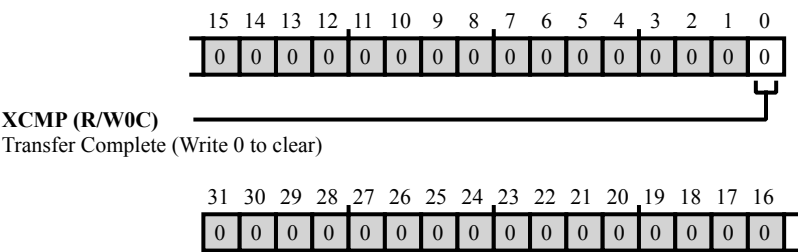


Figure 18-23: `MLB_MCTL` Register Diagram

Table 18-36: `MLB_MCTL` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W0C)	XCMP	Transfer Complete (Write 0 to clear). The <code>MLB_MCTL.XCMP</code> bit indicates that the data transfer is complete.

Memory Interface Control Data 0 Register

The `MLB_MDAT0` register contains Channel Table RAM (CTR) data (bits [31:0] of 128-bit entry) or Data Buffer RAM (DBR) data (bits [7:0] of 8-bit entry).

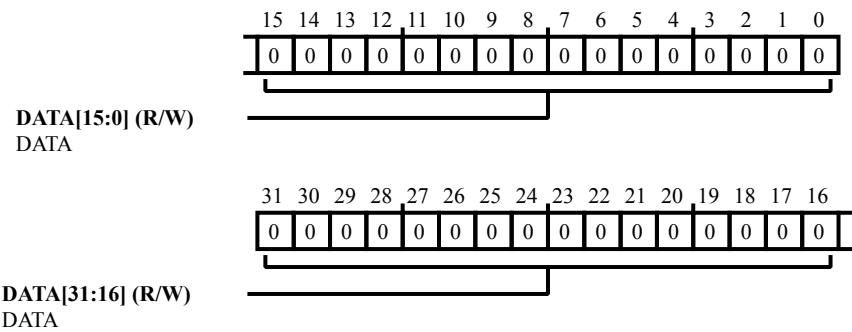


Figure 18-24: `MLB_MDAT0` Register Diagram

Table 18-37: `MLB_MDAT0` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	DATA	DATA. The <code>MLB_MDAT0</code> . <code>DATA</code> bit field contains CTR data or DBR data.

Memory Interface Control Data 1 Register

The `MLB_MDAT1` register contains Channel Table RAM (CTR) data (bits [63:32] of 128-bit entry).

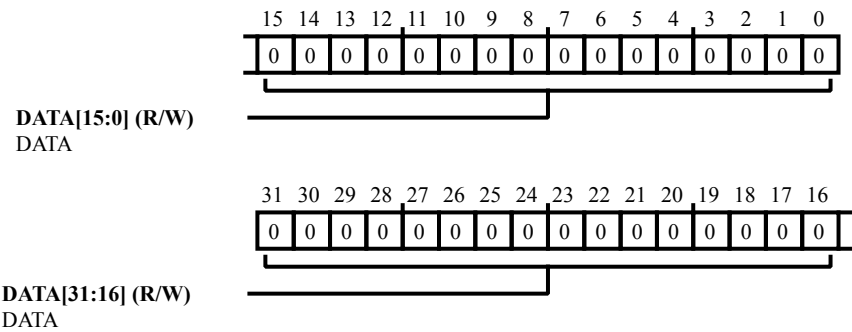


Figure 18-25: `MLB_MDAT1` Register Diagram

Table 18-38: `MLB_MDAT1` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	DATA	DATA. The <code>MLB_MDAT1</code> . DATA bit field contains CTR data.

Memory Interface Control Data 2 Register

The `MLB_MDAT2` register contains Channel Table RAM (CTR) data (bits [95:64] of 128-bit entry).

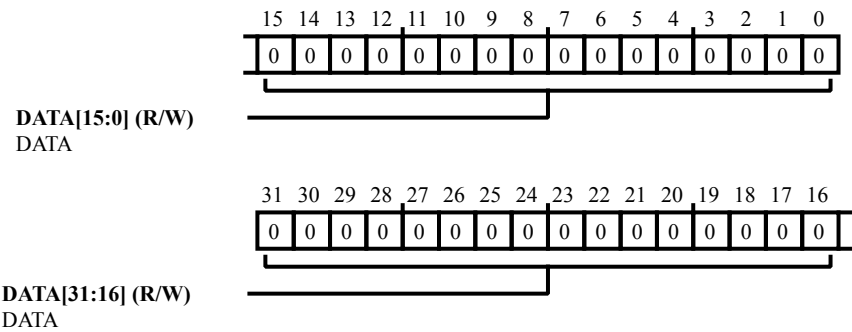


Figure 18-26: MLB_MDAT2 Register Diagram

Table 18-39: MLB_MDAT2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	DATA	DATA. The <code>MLB_MDAT2</code> . DATA bit field contains CTR data.

Memory Interface Control Data 3 Register

The `MLB_MDAT3` register contains Channel Table RAM (CTR) data (bits [127:96] of 128-bit entry).

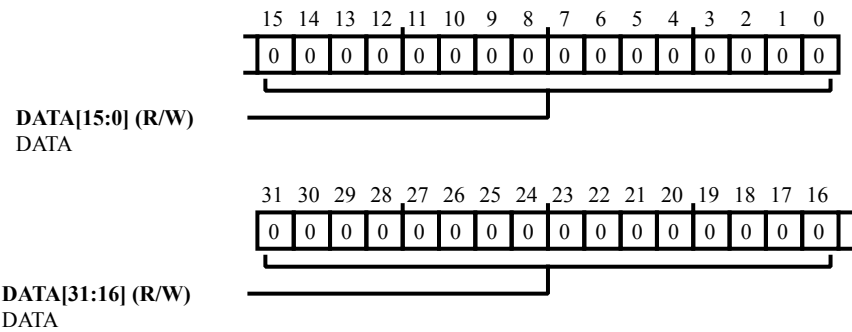


Figure 18-27: MLB_MDAT3 Register Diagram

Table 18-40: MLB_MDAT3 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	DATA	DATA. The MLB_MDAT3 . DATA bit field contains CTR data.

Memory Interface Control Data Write Enable 0 Register

The `MLB_MDWE0` register contains the bitwise write enable for Channel Table RAM (CTR) data bits [31:0]. When cleared (=0) the bit is disabled, when set (=1) the bit is enabled.

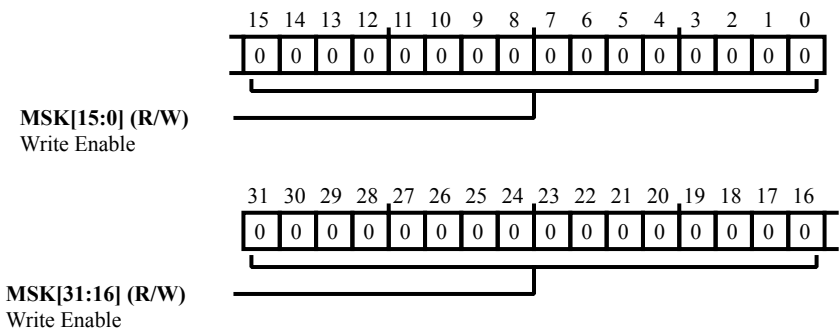


Figure 18-28: MLB_MDWE0 Register Diagram

Table 18-41: MLB_MDWE0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	MSK	Write Enable. The <code>MLB_MDWE0</code> .MSK bit field contains the bitwise write enable for CTR data.

Memory Interface Control Data Write Enable 1 Register

The `MLB_MDWE1` register contains the bitwise write enable for Channel Table RAM (CTR) data bits [63:32]. When cleared (=0), the bit is disabled. When set (=1), the bit is enabled.

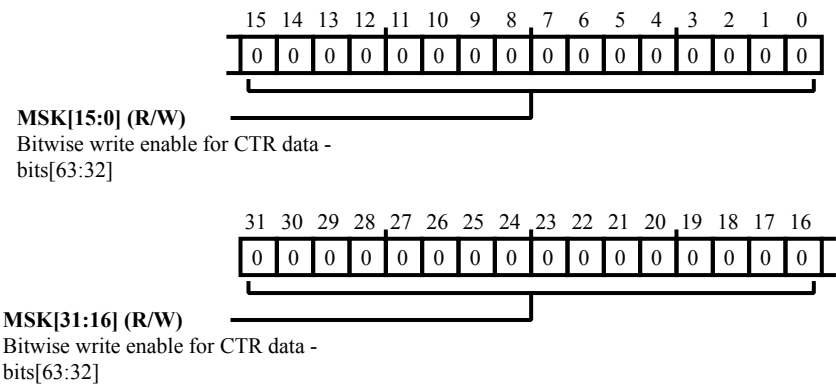


Figure 18-29: MLB_MDWE1 Register Diagram

Table 18-42: MLB_MDWE1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	MSK	Bitwise write enable for CTR data - bits[63:32]. The <code>MLB_MDWE1</code> . MSK bit field contains the bitwise write enable for CTR data.

Memory Interface Control Data Write Enable 2 Register

The `MLB_MDWE2` register contains the bitwise write enable for Channel Table RAM (CTR) data bits [95:64]. When cleared (=0), the bit is disabled. When set (=1), the bit is enabled.

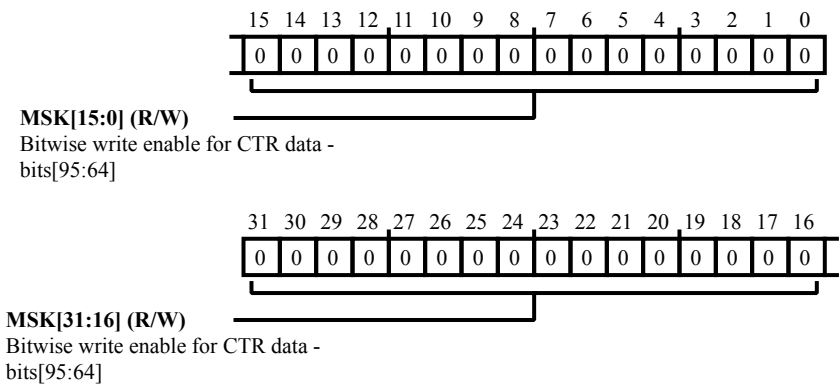


Figure 18-30: MLB_MDWE2 Register Diagram

Table 18-43: MLB_MDWE2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	MSK	Bitwise write enable for CTR data - bits[95:64]. The <code>MLB_MDWE2</code> . MSK bit field contains the bitwise write enable for CTR data.

Memory Interface Control Data Write Enable 3 Register

The [MLB_MDWE3](#) register contains the bitwise write enable for Channel Table RAM (CTR) data bits [127:96]. When cleared (=0), the bit is disabled. When set (=1), the bit is enabled.

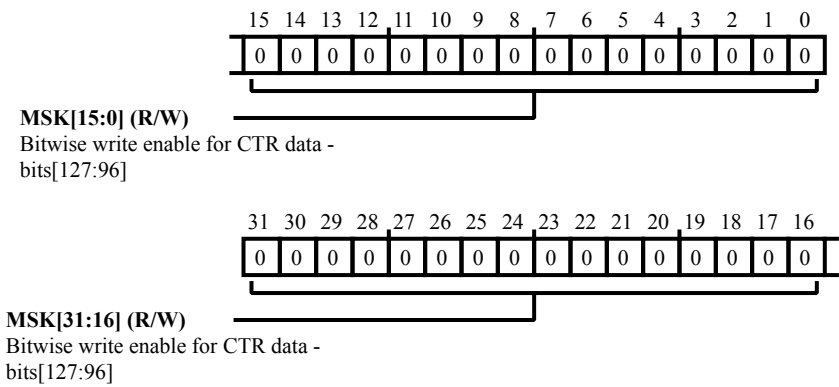


Figure 18-31: MLB_MDWE3 Register Diagram

Table 18-44: MLB_MDWE3 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	MSK	Bitwise write enable for CTR data - bits[127:96]. The MLB_MDWE3 . MSK bit field contains the bitwise write enable for CTR data.

Interrupt Enable Register

The `MLB_MIEN` register is used to enable various interrupt conditions.

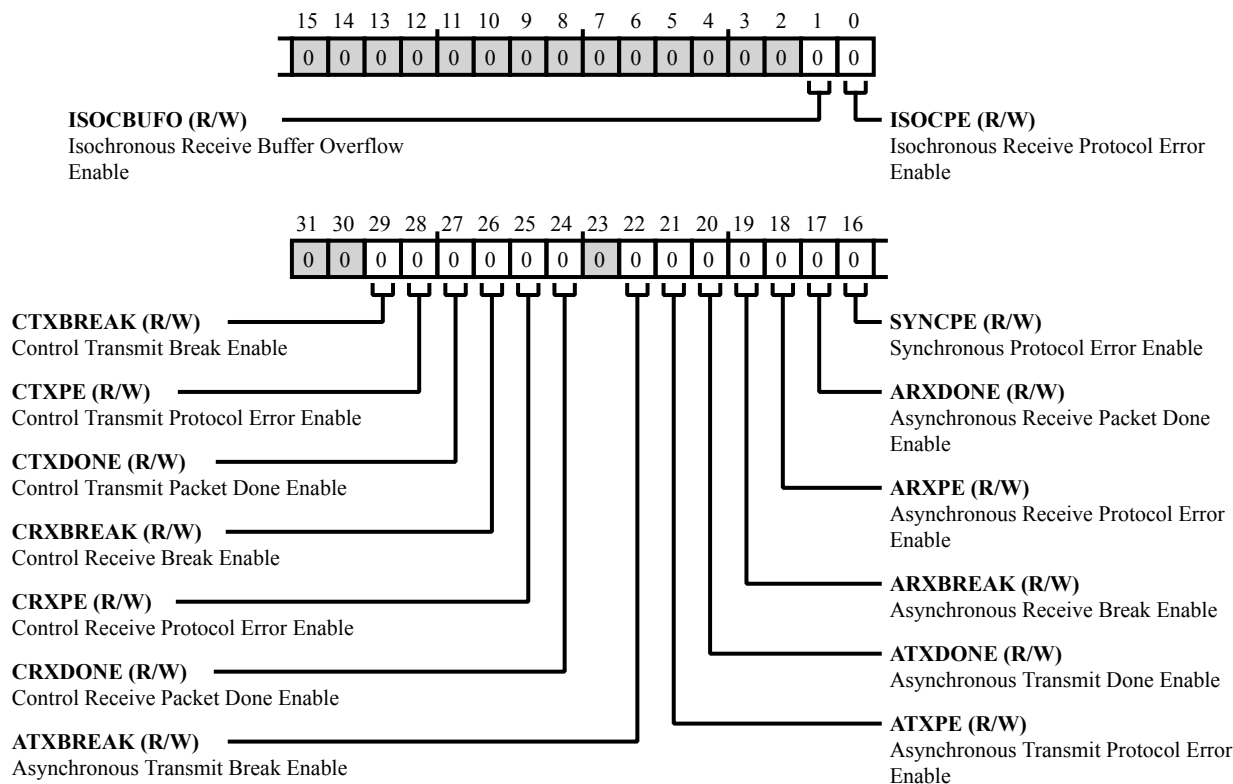


Figure 18-32: `MLB_MIEN` Register Diagram

Table 18-45: `MLB_MIEN` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29 (R/W)	CTXBREAK	Control Transmit Break Enable. When the <code>MLB_MIEN.CTXBREAK</code> bit is set, a ReceiverBreak response received from the receiver on a control Tx channel causes the appropriate channel bit in the <code>MLB_MS0</code> or <code>MLB_MS1</code> registers to be set.
28 (R/W)	CTXPE	Control Transmit Protocol Error Enable. When the <code>MLB_MIEN.CTXPE</code> bit is set, a ProtocolError generated by the receiver on a control Tx channel causes the appropriate channel bit in the <code>MLB_MS0</code> or <code>MLB_MS1</code> registers to be set.
27 (R/W)	CTXDONE	Control Transmit Packet Done Enable. When the <code>MLB_MIEN.CTXDONE</code> bit is set, a packet transmitted with no errors on a control Tx channel causes the appropriate channel bit in the <code>MLB_MS0</code> or <code>MLB_MS1</code> registers to be set.

Table 18-45: MLB_MIEN Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
26 (R/W)	CRXBREAK	Control Receive Break Enable. When the MLB_MIEN.CRXBREAK bit is set, a ControlBreak command received from the transmitter on a control Rx channel causes the appropriate channel bit in the MLB_MS0 or MLB_MS1 registers to be set.
25 (R/W)	CRXPE	Control Receive Protocol Error Enable. When the MLB_MIEN.CRXPE bit is set, a ProtocolError detected on a control Rx channel causes the appropriate channel bit in the MLB_MS0 or MLB_MS1 registers to be set.
24 (R/W)	CRXDONE	Control Receive Packet Done Enable. When the MLB_MIEN.CRXDONE bit is set, a packet received with no errors on a control Rx channel causes the appropriate channel bit in the MLB_MS0 or MLB_MS1 registers to be set.
22 (R/W)	ATXBREAK	Asynchronous Transmit Break Enable. When the MLB_MIEN.ATXBREAK bit is set, a ReceiverBreak response received from the receiver on an asynchronous Tx channel causes the appropriate channel bit in the MLB_MS0 or MLB_MS1 registers to be set.
21 (R/W)	ATXPE	Asynchronous Transmit Protocol Error Enable. When the MLB_MIEN.ATXPE bit is set, a ProtocolError generated by the receiver on an asynchronous Tx channel causes the appropriate channel bit in the MLB_MS0 or MLB_MS1 registers to be set.
20 (R/W)	ATXDONE	Asynchronous Transmit Done Enable. When the MLB_MIEN.ATXDONE bit is set, a packet transmitted with no errors on an asynchronous Tx channel causes the appropriate channel bit in the MLB_MS0 or MLB_MS1 registers to be set.
19 (R/W)	ARXBREAK	Asynchronous Receive Break Enable. When the MLB_MIEN.ARXBREAK bit is set, a AsyncBreak command received from the transmitter on an asynchronous Rx channel causes the appropriate channel bit in the MLB_MS0 or MLB_MS1 registers to be set.
18 (R/W)	ARXPE	Asynchronous Receive Protocol Error Enable. When the MLB_MIEN.ARXPE bit is set, a protocol error detected on an asynchronous Rx channel causes the appropriate channel bit in the MLB_MS0 or MLB_MS1 registers to be set.
17 (R/W)	ARXDONE	Asynchronous Receive Packet Done Enable. When the MLB_MIEN.ARXDONE bit is set, a packet received with no errors on an asynchronous Rx channel causes the appropriate channel bit in the MLB_MS0 or MLB_MS1 registers to be set.

Table 18-45: MLB_MIEN Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
16 (R/W)	SYNCPE	Synchronous Protocol Error Enable. When the MLB_MIEN . SYNCPE bit is set, a protocol error detected on a synchronous Rx channel causes the appropriate channel bit in the MLB_MS0 or MLB_MS1 registers to be set. This occurs only when isochronous flow control is disabled.
1 (R/W)	ISOCBUFO	Isochronous Receive Buffer Overflow Enable. When the MLB_MIEN . ISOCBUFO bit is set, a buffer overflow on an isochronous Rx channel causes the appropriate channel bit in the MLB_MS0 or MLB_MS1 registers to be set. This occurs only when isochronous flow control is disabled.
0 (R/W)	ISOCPE	Isochronous Receive Protocol Error Enable. When the MLB_MIEN . ISOCPE bit is set, a protocol error detected on an isochronous Rx channel causes the appropriate channel bit in the MLB_MS0 or MLB_MS1 registers to be set.

Channel Status 0 Register

The `MLB_MS0` register indicates the channel status for MediaLB channels 31 to 0. Channel status bits are set by hardware and cleared by software. Status is only set if the appropriate bits in the `MLB_MIEN` register are set.

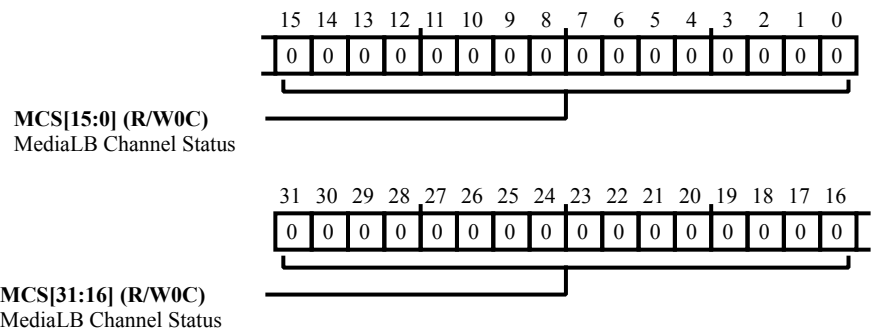


Figure 18-33: MLB_MS0 Register Diagram

Table 18-46: MLB_MS0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W0C)	MCS	MediaLB Channel Status. The <code>MLB_MS0.MCS</code> bit field indicates the MediaLB channel status for channels 31 to 0.

Channel Status 1 Register

The [MLB_MS1](#) register indicates the channel status for MediaLB channels 32 to 63. Channel status bits are set by hardware and cleared by software. Status is only set if the appropriate bits in the [MLB_MIEN](#) register are set.

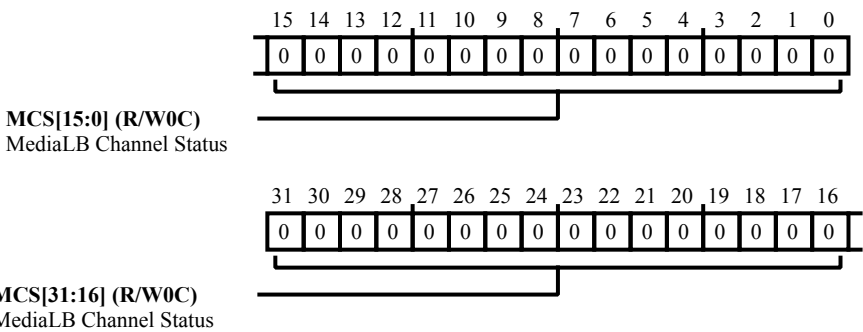


Figure 18-34: MLB_MS1 Register Diagram

Table 18-47: MLB_MS1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W0C)	MCS	MediaLB Channel Status. The MLB_MS1 .MCS bit field indicates the MediaLB channel status for channels 63 to 32.

System Data Register

The `MLB_MSD` register allows system software to receive control information from the MLB controller. The `MLB_MSD` register is updated once per frame by the hardware during the MLB system channel.

The `MLB_MSD` register is loaded with the data from the `MLBDAT_IN` signal during the system channel quadlet. System software must read this register before the start of the next MLB frame to prevent the current data from being lost.

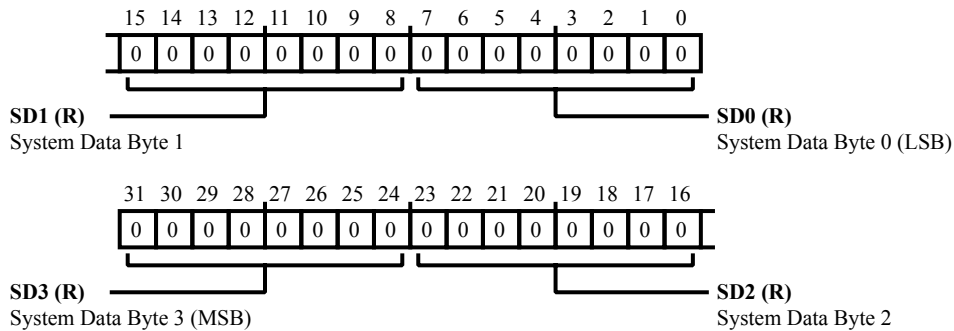


Figure 18-35: MLB_MSD Register Diagram

Table 18-48: MLB_MSD Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/NW)	SD3	System Data Byte 3 (MSB). The <code>MLB_MSD.SD3</code> bits are updated with MediaLB Data [31:24] when a MediaLB software system command is received in the system quadlet. If the <code>MLB_MSS.SWSYSCMD</code> bit is already set, then SD3 is not updated.
23:16 (R/NW)	SD2	System Data Byte 2. The <code>MLB_MSD.SD2</code> bits are updated with MediaLB Data [23:16] when a MediaLB software system command is received in the system quadlet. If the <code>MLB_MSS.SWSYSCMD</code> bit is already set, then SD2 is not updated.
15:8 (R/NW)	SD1	System Data Byte 1. The <code>MLB_MSD.SD1</code> bits are updated with MediaLB Data [15:8] when a MediaLB software system command is received in the system quadlet. If the <code>MLB_MSS.SWSYSCMD</code> bit is already set, then SD1 is not updated.
7:0 (R/NW)	SD0	System Data Byte 0 (LSB). The <code>MLB_MSD.SD0</code> bits are updated with MediaLB Data [7:0] when a MediaLB software system command is received in the system quadlet. If the <code>MLB_MSS.SWSYSCMD</code> bit is already set, then SD0 is not updated.

System Status Register

The `MLB_MSS` register allows system software to monitor and control the status of the MLB network. The register is updated once per frame by hardware during the MLB system channel. The bits of this register are not valid until the processor is locked to the MLB interface (except for the bits associated with MLB lock and unlock, SDMU and SDML). System software must service events before the start of the next MLB frame to prevent the current frame status from being lost.

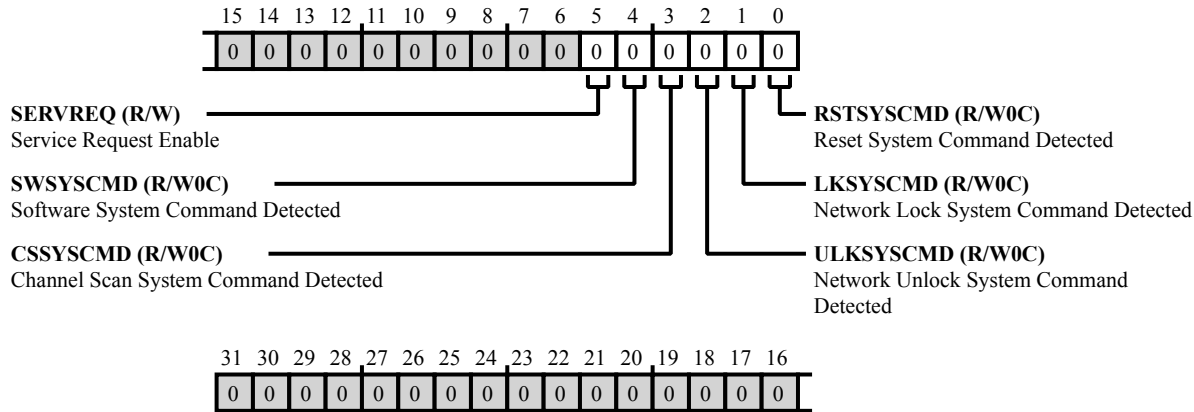


Figure 18-36: MLB_MSS Register Diagram

Table 18-49: MLB_MSS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
5 (R/W)	SERVREQ	Service Request Enable. When the <code>MLB_MSS.SERVREQ</code> bit set, the MediaLB block responds with a device present, request service system response if a matching channel scan system command is detected. When cleared, the MediaLB block responds with a device present system response.
4 (R/W0C)	SWSYSCMD	Software System Command Detected. The <code>MLB_MSS.SWSYSCMD</code> bit indicates that a software system command was detected (in the system quadlet). The <code>MLB_MSS.SWSYSCMD</code> bit is set by hardware, cleared by software. Data is stored in the <code>MLB_MSD</code> register for this command.
3 (R/W0C)	CSSYSCMD	Channel Scan System Command Detected. The <code>MLB_MSS.CSSYSCMD</code> bit indicates that a channel scan system command was detected (in the system quadlet). The <code>MLB_MSS.CSSYSCMD</code> bit is set by hardware, cleared by software. If the node address specified in the data quadlet matches the value in the <code>MLB_CTL1.NDA</code> bit field, the device responds either device present or device present, request service system response in the next system quadlet.

Table 18-49: MLB_MSS Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
2 (R/W0C)	ULKSYSCMD	Network Unlock System Command Detected. The MLB_MSS.ULKSYSCMD bit indicates a network unlock system command was detected (in the system quadlet). The MLB_MSS.ULKSYSCMD bit is by hardware, cleared by software.
1 (R/W0C)	LKSYSCMD	Network Lock System Command Detected. The MLB_MSS.LKSYSCMD bit indicates a network lock system command was detected (in the system quadlet). The MLB_MSS.LKSYSCMD bit is set by hardware and cleared by software.
0 (R/W0C)	RSTSYSCMD	Reset System Command Detected. The MLB_MSS.RSTSYSCMD bit indicates a reset system command was detected (in the system quadlet). The MLB_MSS.RSTSYSCMD bit is set by hardware, cleared by software.

19 Two-Wire Interface (TWI)

The processor has a two-wire interface (TWI), that provides a simple exchange method of control data between multiple devices. The TWI module is compatible with the widely used I²C bus standard. Additionally, the TWI module is fully compatible with serial camera control bus (SCCB) functionality for easier control of various CMOS camera sensor devices.

The TWI module offers the capabilities of simultaneous requester and completer operation and support for both 7-bit addressing and multimedia data arbitration. The TWI interface uses two pins for transferring clock (TWI_SCL) and data (TWI_SDA) and supports the protocol at speeds up to 400K bits/s. The TWI interface pins are compatible with 3.3V logic levels.

To preserve processor bandwidth, the TWI module can be set up with transfer-initiated interrupts to only service FIFO buffer data reads and writes. Protocol-related interrupts are optional. The TWI externally moves 8-bit data while maintaining compliance with the I²C bus protocol.

NOTE: All TWI instances operate in the SCLK0 domain.

TWI Features

The TWI is fully compatible with the widely used I²C bus standard.

The TWI controller includes the following features.

- Simultaneous requester and completer operation on multiple device systems
- Support for multi-controller bus arbitration
- 7-bit addressing
- 100K bits/second and 400K bits/second data rates
- General call address support
- Controller clock synchronization and support for clock low extension
- Separate multiple-byte receive and transmit FIFOs
- Low interrupt rate

- Individual override control of data and clock lines in the event of bus lock-up
- Input filter for spike suppression
- Serial camera control bus support as specified in the *OmniVision Serial Camera Control Bus (SCCB) Functional Specification*
- Programmable drive/tolerance of TWI pins. Refer to the register description in General-Purpose Ports (PORT) chapter for details.

TWI Functional Description

The TWI interface is a shift register that serially transmits and receives data bits. It moves data 1 bit at a time at the SCL rate, to and from other TWI devices. The SCL signal synchronizes the shifting and sampling of the data on the serial data pin.

ADSP-21560_21561_21564_21568 TWI Register List

The Two-Wire Interface controller TWI allows a device to interface to an inter-IC bus as specified by the Philips I²C Bus Specification version 2.1, dated January 2000. A set of registers governs TWI operations. For more information on TWI functionality, see the TWI register descriptions.

Table 19-1: ADSP-21560_21561_21564_21568 TWI Register List

Name	Description
TWI_CLKDIV	SCL Clock Divider Register
TWI_CTL	Control Register
TWI_FIFOCTL	FIFO Control Register
TWI_FIFOSTAT	FIFO Status Register
TWI_IMSK	Interrupt Mask Register
TWI_ISTAT	Interrupt Status Register
TWI_MSTRADDR	Controller Mode Address Register
TWI_MSTRCTL	Controller Mode Registers
TWI_MSTRSTAT	Controller Mode Status Register
TWI_RXDATA16	Rx Data Double-Byte Register
TWI_RXDATA8	Rx Data Single-Byte Register
TWI_SLVADDR	Target Mode Address Register
TWI_SLVCTL	Target Mode Control Register
TWI_SLVSTAT	Target Mode Status Register
TWI_TXDATA16	Tx Data Double-Byte Register
TWI_TXDATA8	Tx Data Single-Byte Register

ADSP-21560_21561_21564_21568 TWI Interrupt List

Table 19-2: ADSP-21560_21561_21564_21568 TWI Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
102	TWI0_DATA	TWI0 Data Interrupt	Level	
103	TWI1_DATA	TWI1 Data Interrupt	Level	
104	TWI2_DATA	TWI2 Data Interrupt	Level	
105	TWI3_DATA	TWI3 Data Interrupt	Level	

TWI Block Diagram

The *TWI Block Diagram* figure shows the basic blocks of the TWI interface.

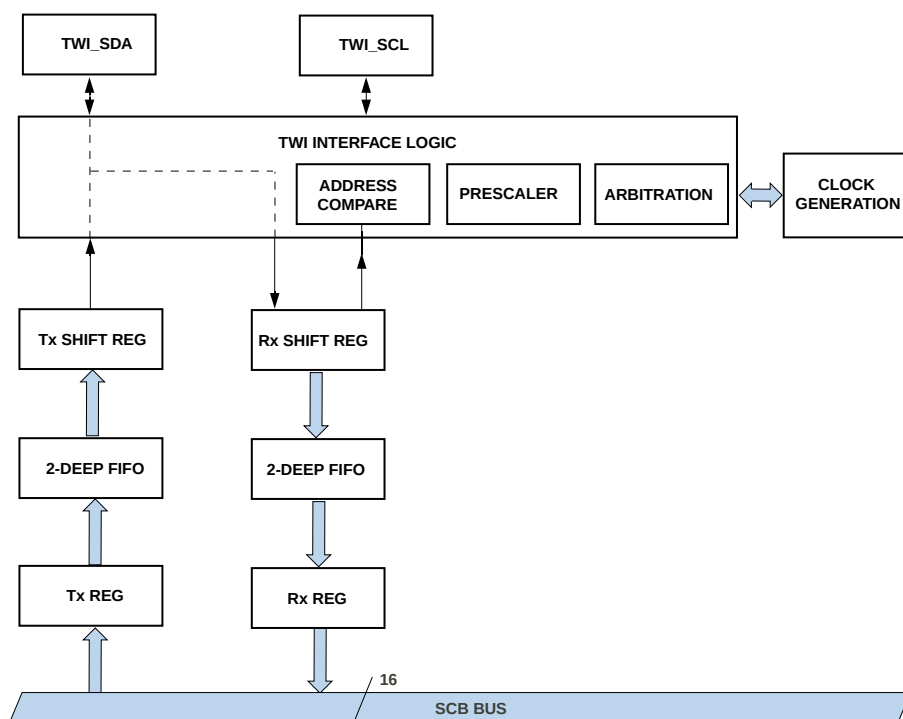


Figure 19-1: TWI Block Diagram

External Interface

The TWI_SDA (serial data) and TWI_SCL (serial clock) signals are open drain and require pull-up resistors. These bidirectional signals externally interface the TWI controller to the I²C bus, and no other external connections or logic are necessary.

Serial Clock Signal (SCL)

The serial clock signal (TWI_SCL) is an input in completer mode. In requester mode, the TWI controller must set this signal to the desired frequency.

The TWI controller supports the standard mode of operation (up to 100 kHz) or fast mode (up to 400 kHz). The TWI control register (TWI_CTL) sets the TWI_CTL.PRESCALE value which sets the relationship between the system clock (SCLK) and the internally timed events of the TWI controller. The internal time reference is derived from SCLK using a prescaled value. The prescale value is the number of SCLK periods used in the generation of one internal time reference. Set the value of prescale to create an internal time reference with a period of 10 MHz. It is represented as a 7-bit binary value as follows.

$$\text{PRESCALE} = f_{\text{SCLK}}/10\text{MHz}$$

NOTE: It is not always possible to achieve 10-MHz accuracy. In such cases, it is safe to round up the PRESCALE value to the next highest integer. For example, if SCLK is 100 MHz, the PRESCALE value is calculated as $100\text{ MHz}/10\text{ MHz} = 10$. A prescale value of 14 in this case ensures that all timing requirements are met.

During requester mode operation, the TWI module uses the TWI_CLKDIV register values to create the minimum TWI_CLKDIV.CLKHI and TWI_CLKDIV.CLKLO durations of the TWI_SCL signal. The TWI_CLKDIV.CLKHI field specifies the minimum number of 10-MHz time reference periods the TWI_SCL waits before a new clock low period begins, assuming a single requester. (The 10-MHz time reference periods are represented as an 8-bit binary value). The TWI uses the TWI_CLKDIV.CLKLO field to specify the minimum number of internal time reference periods (represented as an 8-bit binary value); the TWI_SCL signal is held low.

Serial clock frequencies can vary from 400 kHz to less than 20 kHz. The resolution of the clock generated is 1/10 MHz or 100 ns. The following equation describes the frequency.

$$\text{CLKDIV} = \text{TWI_SCL period}/10\text{ MHz time reference.}$$

For example, for an TWI_SCL of 400 kHz (period = $1/400\text{ kHz} = 2500\text{ ns}$) and an internal time reference of 10 MHz (period = 100 ns), the following equation applies:

$$\text{CLKDIV} = 2500\text{ ns}/100\text{ ns} = 25$$

Therefore, a TWI_SCL signal with a 30% duty cycle has TWI_CLKDIV.CLKLO=17 and TWI_CLKDIV.CLKHI=8. Adding TWI_CLKDIV.CLKLO and TWI_CLKDIV.CLKHI equals CLKDIV.

NOTE: The TWI_CLKDIV.CLKHI and TWI_CLKDIV.CLKLO fields are not intended to guarantee a certain frequency. Rather, they guarantee a certain minimum high and low duration for the TWI_SCL signal. Slew rate controls falling edges. The RC time constant governs the rising edges. The pull-up resistor and the TWI_SCL capacitance form the time constant. See the “Register Descriptions” section for more details.

Serial Data Signal (SDA)

The TWI transmits and receives serial data, depending on the direction of the transfer, on the bidirectional serial data signal (SDA).

Internal Interface

The peripheral bus interface supports the transfer of 16-bit wide data. The processor uses the interface in the support of register and FIFO buffer reads and writes. The TWI internal interface is comprised of the blocks described as follows.

Register block. Contains all control and status bits and reflects what can be written or read as outlined by the programming model. Each function block updates their corresponding status bits.

FIFO buffer. Configured as a 1-byte-wide, 2-deep transmit FIFO buffer and a 1-byte-wide, 2-deep receive FIFO buffer.

Transmit shift register. Serially shifts its data out externally off chip. The output can be controlled for generation of acknowledgments, or it can be manually overwritten.

Receive shift register. Receives its data serially from off chip. The receive shift register is 1 byte wide and data received can either be transferred to the FIFO buffer or used in an address comparison.

Address compare block. Supports address comparison in the event the TWI controller module is accessed as a completer.

Prescaler block. Must be programmed to generate a 10-MHz time reference relative to the system clock. The block uses this time base for filtering of data and timing events specified by the electrical data sheet (See the Philips specification). The block uses the time base to generate the TWI_SCL clock as well.

Clock generation module. Generates an external TWI_SCL clock when in requester mode. It includes the logic necessary for synchronization in a multi-requester clock configuration and clock stretching when configured in completer mode.

NOTE: The TWI does not support DMA based operation.

TWI Architectural Concepts

The TWI controller follows the transfer protocol of the Philips I²C Bus specification version 2.1 dated January 2000.

NOTE: The TWI unit does not support DMA-based operation.

TWI Protocol

The *Data Transfer* figure shows a simple complete transfer.

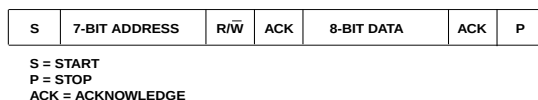


Figure 19-2: Data Transfer

The TWI controller register contents maps to a basic transfer. The *Data Transfer with Bit Illustration* figure details the same transfer from the *Data Transfer* figure noting the corresponding TWI controller bit names. In this

illustration, the TWI controller successfully transmits 1 byte of data. The completer has acknowledged both address and data.



Figure 19-3: Data Transfer with Bit Illustration

Clock Generation and Synchronization

The TWI controller implementation only issues a clock during requester mode operation and only at the time a transfer initiates. If arbitration for the bus is lost, the serial clock output immediately three-states. If multiple clocks attempt to drive the serial clock line, the TWI controller synchronizes its clock with the other remaining clocks. The *Clock Synchronization* figure shows this functionality.

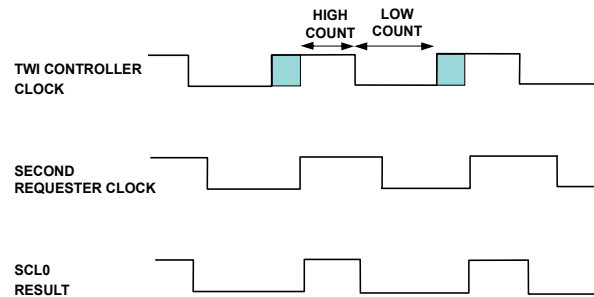


Figure 19-4: Clock Synchronization

The TWI controller serial clock (TWI_SCL) output follows these rules:

- Once the clock high (TWI_CLKDIV.CLKHI) count is complete, the serial clock output is driven low and the clock low (TWI_CLKDIV.CLKLO) count begins.
- Once the clock low count is complete, the serial clock line is three-stated. This state allows the external pull-up resistor to pull the TWI_SCL signal high. The clock synchronization logic enters a delay mode (shaded area) until the TWI_SCL signal is detected at logic 1 level. Now, the clock high count begins.

Bus Arbitration

The TWI controller initiates a requester mode transmission only when the bus is idle. When the bus is idle and two requesters initiate a transfer, arbitration for the bus begins. The *Bus Arbitration* figure shows the arbitration.

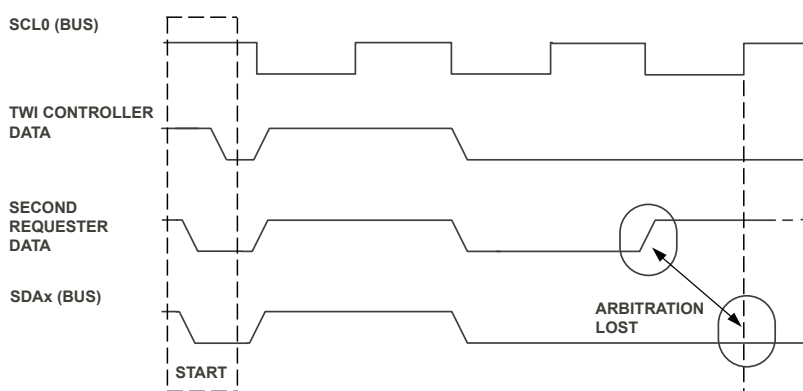


Figure 19-5: Bus Arbitration

The TWI controller monitors the serial data bus (SDA) while the `TWI_SCL` signal is high. If the `TWI_SDA` signal is determined to be an active logic 0 level while the data of the TWI controller is a logic 1 level, the TWI controller has lost arbitration. It stops generating the clock and data signals. Arbitration is not only performed at the serial clock edges, but also during the entire time the `TWI_SCL` signal is high.

Start and Stop Conditions

Start and stop conditions involve serial data transitions while the serial clock is a logic 1 level. The TWI controller generates and recognizes these transitions. Typically, start and stop conditions occur at the beginning and at the conclusion of a transmission, except repeated start combined transfers. The *Start and Stop Conditions* figure shows the transitions.

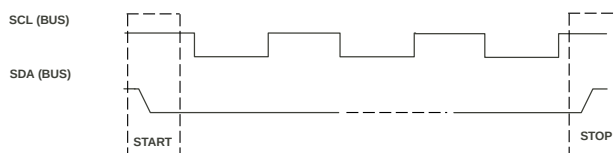


Figure 19-6: Start and Stop Conditions

The TWI special case start and stop conditions of the TWI controller include the following.

- Controller addressed as a completer-receiver. If the requester asserts a stop condition during the data phase of a transfer, the TWI controller concludes the transfer (`TWI_ISTAT.SCOMP`).
- Controller addressed as a completer-transmitter. If the requester asserts a stop condition during the data phase of a transfer, the TWI controller concludes the transfer (`TWI_ISTAT.SCOMP`) and indicates a completer transfer error (`TWI_ISTAT.SERR`).
- Controller as a requester-transmitter or requester-receiver. If the stop bit (`TWI_MSTRCTL.STOP`) is set during an active requester transfer, the TWI controller issues a stop condition as soon as possible avoiding any error conditions. The TWI controller operates as if data transfer count had been reached.

General Call Support

The TWI controller always decodes and acknowledges a general call address if:

- The TWI controller is enabled as a target
- General call is enabled

The `TWI_SLVCTL.GEN` bit configures general call addressing (0x00) only when the TWI controller is a target-receiver.

If the data associated with the transfer is (NAK) not acknowledged, the `TWI_SLVCTL.NAK` bit can be set. If the TWI controller issues a general call as a requester-transmitter, set the appropriate address (`TWI_MSTRADDR` register) and transfer direction (`TWI_MSTRCTL.DIR` bit) and load the transmit FIFO data.

NOTE: The byte following the general call address usually defines the completer's response to the call. The interpretation of the command in the second byte is based on the value of its LSB. For a TWI completer device, the bytes received after the general call address are considered data.

Fast Mode

Fast mode uses the same mechanics as the standard mode of operation. Fast mode affects electrical specifications and timing. When fast mode is enabled, (FAST) timing is modified to meet the following electrical requirements.

- Serial data rise times before arbitration evaluation (t_r)
- Stop condition set-up time from serial clock to serial data (t_{SUSTO})
- Bus free time between a stop and start condition (t_{BUF})

TWI Operating Modes

The TWI has two modes of operation: repeated start and clock stretching. The following sections describe the operating modes.

Repeated Start

A repeated start condition is the absence of a stop condition between two transfers. The two transfers can be of any direction type. Examples include a transmit followed by a receive, or a receive followed by a transmit. The following sections guide the programmer in developing a service routine.

Transmit Receive Repeated Start

The *Repeated Start Followed by Data Receive* figure shows a repeated start followed by a data receive sequence. The shading in the figure indicates that the completer has control of the bus.

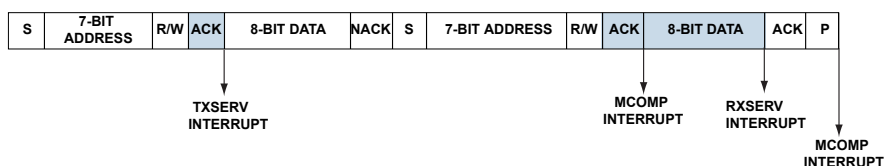


Figure 19-7: Repeated Start Followed by Data Receive

The tasks performed at each interrupt are:

- Transmit FIFO service (TWI_ISTAT.TXSERV) interrupt request. This interrupt is generated due to a FIFO access. Since this byte is the last of this transfer, the TWI uses the TWI_FIFOSTAT register to indicate that the transmit FIFO is empty. When read, TWI_MSTRCTL.DCNT bit field=0. Set the TWI_MSTRCTL.RSTART bit to indicate a repeated start and set the TWI_MSTRCTL.DIR bit if the following transfer is a data receive.
- Requester transfer complete (TWI_ISTAT.MCOMP) interrupt. This interrupt request is generated when all data transfers (TWI_MSTRCTL.DCNT bit field=0). If no errors occur, a start condition initiates. Clear the TWI_MSTRCTL.RSTART bit and program the TWI_MSTRCTL.DCNT bits with the desired number of bytes to receive.
- Receive FIFO service (TWI_ISTAT.RXSERV) interrupt. This interrupt request is generated due to the arrival of a byte in the receive FIFO. Simple data handling is the only requirement.
- Requester transfer complete (TWI_ISTAT.MCOMP) interrupt. This interrupt request is generated when the transfer completes.

Receive Transmit Repeated Start

The *Repeated Start Data Receive Followed by Data Transmit* figure illustrates a repeated start data receive followed by a data transmit sequence. The shading in the figure indicates that the completer has control of the bus.

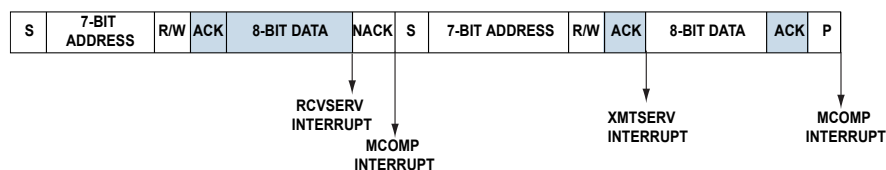


Figure 19-8: Repeated Start Data Receive Followed by Data Transmit

The tasks performed at each interrupt are:

- Receive FIFO service (TWI_ISTAT.RXSERV) interrupt. This interrupt request is generated due to the arrival of a data byte in the receive FIFO. Set the TWI_MSTRCTL.RSTART bit to indicate a repeated start and clear the TWI_MSTRCTL.DIR bit if the following transfer is a data transmit.
- Requester transfer complete (TWI_ISTAT.MCOMP) interrupt. This interrupt request has occurred due to the completion of the data receive transfer. If no errors occur, a start condition initiates. Clear the TWI_MSTRCTL.RSTART bit and program the TWI_MSTRCTL.DCNT bits with the desired number of bytes to transmit.
- Transmit FIFO service (TWI_ISTAT.TXSERV) interrupt. This interrupt request is generated due to a FIFO access. Simple data handling is the only requirement.
- Requester transfer complete (TWI_ISTAT.MCOMP) interrupt. This interrupt request is generated when the transfer completes.

NOTE: There is no timing constraint to meet the conditions—program the bits as required. Refer to [Clock Stretching During Repeated Start](#) section for more on how the controller stretches the clock during repeated start transfers.

Clock Stretching

Clock stretching is an added function of the TWI controller in requester mode operation. This behavior uses self-induced stretching of the I²C clock while waiting to service interrupts. Hardware initiates stretching automatically. No programming is necessary. The TWI controller as a requester supports three modes of clock stretching:

- [Clock Stretching During FIFO Underflow](#)
- [Clock Stretching During FIFO Overflow](#)
- [Clock Stretching During Repeated Start](#)

Clock Stretching During FIFO Underflow

During a requester mode transmit, an interrupt request occurs the instant the transmit FIFO becomes empty. The most recent byte begins transmission. If the `TWI_ISTAT.TXSERV` interrupt request is not serviced, the concluding acknowledge phase of the transfer stretches.

Stretching of the clock continues until new data bytes are written to the transmit FIFO ([TWI_TXDATA8](#) or [TWI_TXDATA16](#) registers). No other action is required to release the clock and continue the transmission. This behavior continues until the transmission completes (`TWI_MSTRCTL.DCNT=0`). The transmission concludes (`TWI_ISTAT.MCOMP`). The *Clock Stretching during FIFO Underflow* figure and table show the stretching.

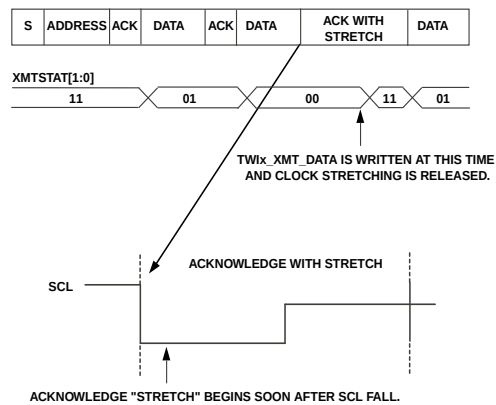


Figure 19-9: Clock Stretching during FIFO Underflow

TWI Controller	Processor
Interrupt: XMTSERV – Transmit FIFO buffer is empty	Acknowledge: Clear the interrupt request source bits. Write to the transmit FIFO buffer
...	...
Interrupt: MCOMP – requester transmit complete (DCNT= 0x00)	Acknowledge: Clear the interrupt request source bits

Clock Stretching During FIFO Overflow

During a requester mode receive operation, an interrupt occurs at the instant the receive FIFO becomes full. It is during the acknowledge phase of this received byte that clock stretching begins. The TWI module makes no attempt to initiate the reception of another byte. Stretching of the clock continues until the data bytes previously received are read from the receive FIFO buffer (`TWI_RXDATA8` or `TWI_RXDATA16` registers). No other action is required to release the clock and continue the reception of data. This behavior continues until the reception is complete (`TWI_MSTRCTL.DCNT=0`). Reception concludes (`TWI_ISTAT.MCOMP`). The *Clock Stretching During FIFO Overflow* figure and table show the clock stretching.

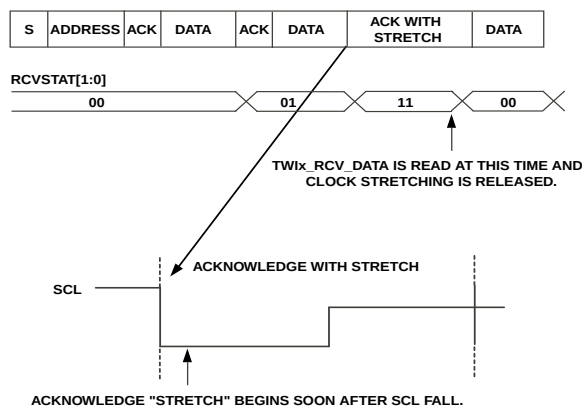


Figure 19-10: Clock Stretching During FIFO Overflow

TWI Controller	Processor
Interrupt: RCVSERV – Receive FIFO buffer is full	Acknowledge: Clear the interrupt request source bits. Read the receive FIFO buffer
...	...
Acknowledge: Clear the interrupt source bits	Interrupt: MCOMP – requester receive complete

Clock Stretching During Repeated Start

The repeated start feature in I²C protocol requires a transition between two subsequent transfers. With the use of clock stretching, the task of managing transitions becomes simpler and common to all transfer types.

Once an initial TWI requester transfer completes (transmit or receive), the clock initiates a stretch during the repeated start phase between transfers. Concurrent with this event, the initial transfer generates a `TWI_ISTAT.MCOMP` interrupt to signify the initial transfer has completed (`TWI_MSTRCTL.DCNT=0`). This initial transfer is handled without any special bit setting sequences or timing.

The clock stretching logic described applies here. With no system-related timing constraints, the subsequent transfer (receive or transmit) is set up and activated. This sequence can repeat as many times as required to string a series of repeated start transfers together. The *Clock Stretching during Repeated Start Condition* figure and table show the clock stretching.

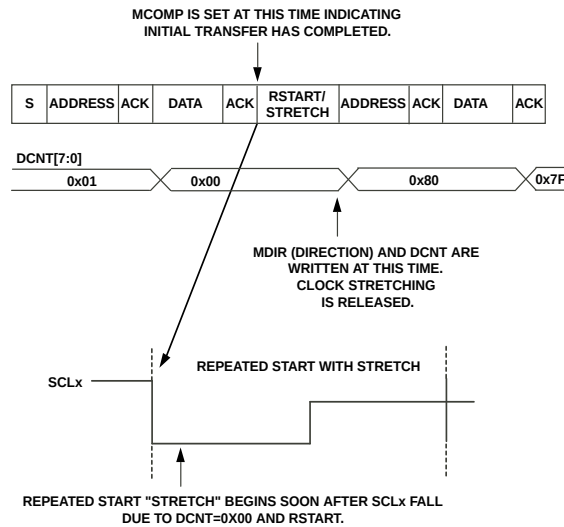


Figure 19-11: Clock Stretching during Repeated Start Condition

TWI Controller	Processor
Interrupt: MCOMP – Initial transmit has completed and DCNT = 0x00. Note: transfer in progress, RSTART previously set.	Acknowledge: Clear the interrupt request source bits. Write to TWIx_MASTER_CTL, setting MDIR (receive), clearing RSTART, and setting new DCNT value (nonzero).
Interrupt: RCVSERV – Receive FIFO is full	Acknowledge: Clear the interrupt request source bits. Read the receive FIFO buffer
...	...
Interrupt: MCOMP – requester receive complete	Acknowledge: Clear the interrupt request source bits

TWI Programming Model

The topics in this section provide information on the basic programming steps required to set up and run the two wire interface.

General Setup

General setup refers to register writes that are required for both completer mode and requester mode operations.

Perform general setup before setting either the requester or completer enable bits.

1. Program the `TWI_CTL.EN` bit to enable the TWI controller and set the prescale value (`TWI_CTL.PRESCALE` bit).
2. Program the prescale value to the binary representation of $f_{SCLK0}/10$ MHz. Round up all values to the next whole number.
3. Set the `TWI_CTL.EN` bit to enable the controller.

Once the TWI controller is enabled, a bus busy condition can be detected. This condition clears after t_{BUF} has expired, assuming no additional bus activity has been detected.

Completer Mode

When enabled, completer mode operation supports both receive and transmit data transfers.

It is not possible to enable only one data transfer direction and not acknowledge (NAK) the other. The following setup reflects this functionality.

1. Program the `TWI_SLVADDR` register. The TWI uses the appropriate 7 bits in determining a match during the address phase of the transfer.
2. Program the `TWI_TXDATA8.VALUE` or `TWI_TXDATA16` registers. These values are the initial data values for transmission when the completer is addressed, and transmission is needed. This step is optional. If no data is written when the completer is addressed and transmission is needed, the serial clock (`TWI_SCL`) stretches. An interrupt is generated until data is written to the transmit FIFO.
3. Program the `TWI_IMSK` register. There are enable-bits associated with the desired interrupt sources. For example, programming the value 0x000F results in an interrupt request output to the processor, when the TWI module detects a valid address match. An interrupt request also occurs when a valid completer transfer completes or has an error, or a subsequent transfer has begun, and the previous transfer has not been serviced.
4. Program the `TWI_SLVCTL` register. This step prepares and enables completer mode operation. For example, programming the value 0x0005 enables completer mode operation and requires 7-bit addressing. It indicates that data in the transmit FIFO buffer is for completer mode transmission.

The *Completer Mode Interaction* table and *TWI Completer Mode Program Flow* diagram represent the interaction between the TWI controller and the processor using this example.

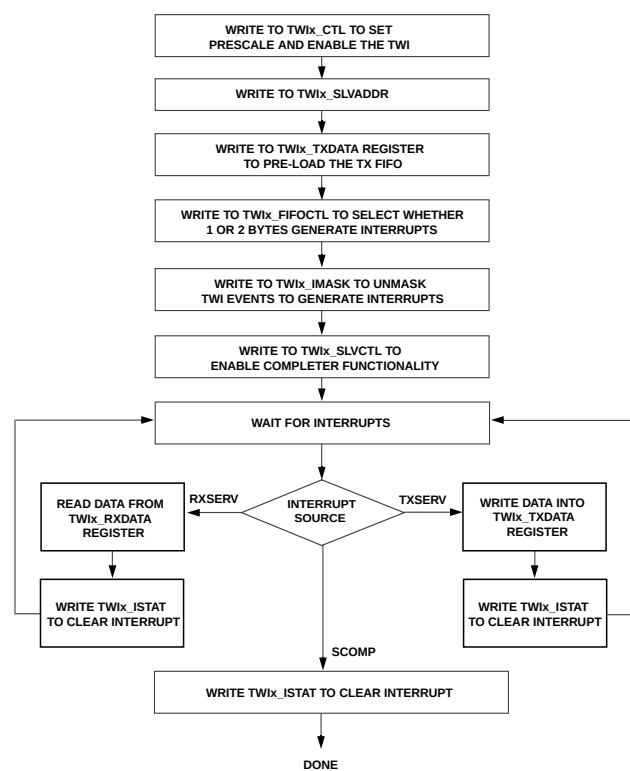


Figure 19-12: TWI Completer Mode Program Flow

Table 19-3: Completer Mode Interaction

TWI Controller	Processor
Interrupt: SINIT – completer transfer in progress.	Acknowledge: Clear the interrupt source bits.
Interrupt: RCVSERV – Receive buffer is full.	Acknowledge: Clear the interrupt source bits. Read TWIx_FIFO_STAT. Read the receive FIFO buffer.
...	...
Interrupt: SCOMP – completer transfer complete.	Acknowledge: Clear the interrupt source bits. Read the receive FIFO buffer.

Requester Mode Program Flow

The *Requester Mode Program Flow* figure shows the program for the TWI in requester mode.

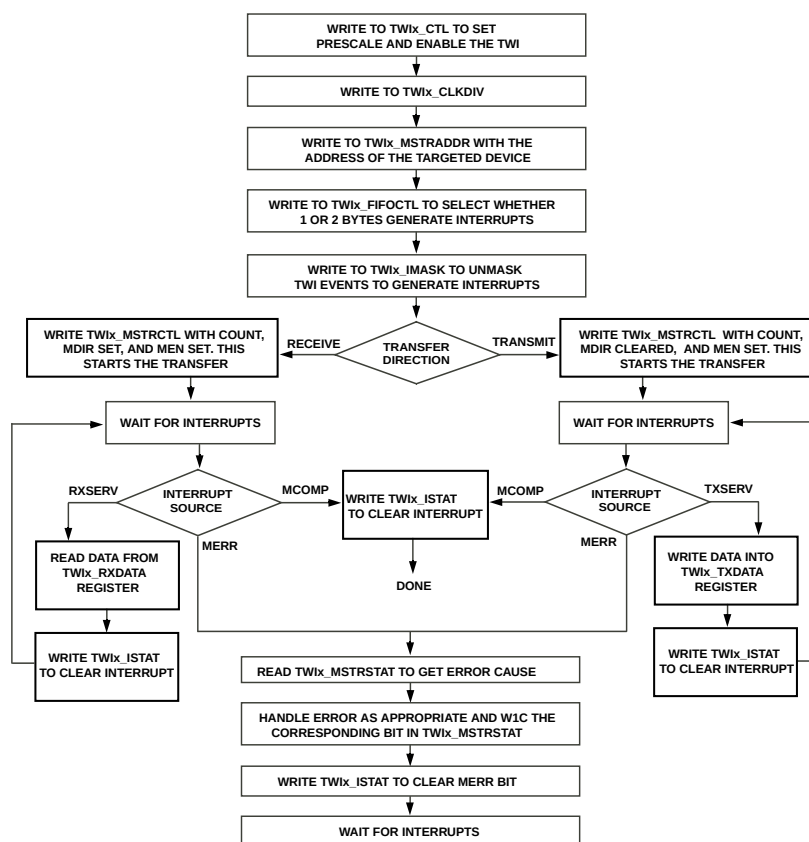


Figure 19-13: Requester Mode Program Flow

Requester Mode Clock Setup

Requester mode operation is set up and executed on a per-transfer basis.

An example of programming steps for a receive and for a transmit is given separately in following sections. The programming step for clock setup listed here is common to both transfer types.

1. Program the `TWI_CLKDIV` register to define the minimum high and minimum low duration for the clock.

The `TWI_CLKDIV.CLKHI` and `TWI_CLKDIV.CLKLO` fields do not guarantee a certain frequency. Rather, they guarantee a certain minimum high and low duration for `TWI_SCL`. The slew rate controls falling edges. The RC time constant formed by the pull-up resistor and the SCL capacitance govern rising edges. See the “Register Descriptions” section for more details.

Requester Mode Transmit

Follow these programming steps for a single requester mode transmission:

1. Program the `TWI_MSTRADDR` register. This step defines the address transmitted during the address phase of the transfer.

2. Program the `TWI_TXDATA8` or `TWI_TXDATA16` register. This step configures the initial data transmitted. It is an error to complete the address phase of the transfer and not have data available in the transmit FIFO buffer.
3. Program the `TWI_FIFOCTL` register. The programming indicates if the transmit FIFO buffer interrupt requests occur with each byte transmitted (8-bits) or with every 2 bytes transmitted (16-bits).
4. Program the `TWI_IMSK` register. This step enables the bits associated with the desired interrupt request sources. For example, programming the value 0x0030 results in an interrupt output to the processor when the requester transfer completes, and the requester transfer has an error.
5. Program the `TWI_MSTRCTL` register. This step prepares and enables requester mode operation. For example, programming the value 0x0201: enables requester mode operation, generates a 7-bit address, sets the direction to requester-transmit, uses standard mode timing, and transmits 8 data bytes before generating a stop condition.

The *Requester Mode Transmit Setup Interaction* table represents the interaction between the TWI controller and the processor using this example.

Table 19-4: Requester Mode Transmit Setup Interaction

TWI Controller	Processor
Interrupt: XMTSERV – Transmit buffer is empty	Acknowledge: Clear the interrupt request source bits and write to the transmit FIFO buffer
...	...
Interrupt: MCOMP – requester transfer complete	Acknowledge: Clear the interrupt request source bits

Requester Mode Receive

Follow these programming steps for a single requester mode receive.

1. Program the `TWI_MSTRADDR` register. This step defines the address transmitted during the address phase of the transfer.
2. Program the `TWI_FIFOCTL` register. This step indicates if the receive FIFO buffer interrupt requests occur with each byte received (8-bits) or with every 2 bytes received (16-bits).
3. Program the `TWI_IMSK` register. This step configures the enable bits associated with the desired interrupt sources. For example, programming the value 0x0030 results in an interrupt request output to the processor when the requester transfer completes, and the requester transfer has an error.
4. Program the `TWI_MSTRCTL` register. This step prepares and enables requester mode operation. For example, programming the value 0x0205: enables requester mode operation, generates a 7-bit address, sets the direction to requester-receive, uses standard mode timing, and receives 8 data bytes before generating a stop condition.

The *Requester Mode Receive Setup Interaction* table shows the interaction between the TWI controller and the processor using this example.

Table 19-5: Requester Mode Receive Setup Interaction

TWI Requester	Processor
Interrupt: RCVSERV – receive buffer is full	Acknowledge: Clear the interrupt request source bits. Read the receive FIFO buffer.
...	...
Interrupt: MCOMP – requester transfer complete	Acknowledge: Clear the interrupt request source bits and read the receive FIFO buffer

NOTE: After the `TWI_MSTRCTL.DCNT` bit decrements to zero, the TWI controller device sends a NAK to indicate to the completer transmitter to release the bus. This operation allows the requester to send the stop signal to terminate the transfer.

ADSP-21560_21561_21564_21568 TWI Register Descriptions

Two-Wire Interface (TWI) contains the following registers.

Table 19-6: ADSP-21560_21561_21564_21568 TWI Register List

Name	Description
<code>TWI_CLKDIV</code>	SCL Clock Divider Register
<code>TWI_CTL</code>	Control Register
<code>TWI_FIFCTL</code>	FIFO Control Register
<code>TWI_FIFOSTAT</code>	FIFO Status Register
<code>TWI_IMSK</code>	Interrupt Mask Register
<code>TWI_ISTAT</code>	Interrupt Status Register
<code>TWI_MSTRADDR</code>	Controller Mode Address Register
<code>TWI_MSTRCTL</code>	Controller Mode Registers
<code>TWI_MSTRSTAT</code>	Controller Mode Status Register
<code>TWI_RXDATA16</code>	Rx Data Double-Byte Register
<code>TWI_RXDATA8</code>	Rx Data Single-Byte Register
<code>TWI_SLVADDR</code>	Target Mode Address Register
<code>TWI_SLVCTL</code>	Target Mode Control Register
<code>TWI_SLVSTAT</code>	Target Mode Status Register
<code>TWI_TXDATA16</code>	Tx Data Double-Byte Register
<code>TWI_TXDATA8</code>	Tx Data Single-Byte Register

SCL Clock Divider Register

During controller mode operation, the `TWI_CLKDIV` holds values, which the TWI uses to create the high and low durations of the serial clock (SCL). The clock signal SCL is an output in controller mode and an input in target mode. The values in the `TWI_CLKDIV.CLKLO` and `TWI_CLKDIV.CLKHI` fields add up to the `CLKDIV` value the following equation.

$$\text{CLKDIV} = \text{TWI SCL period} / 10 \text{ MHz time reference}$$

Serial clock frequencies can vary from 400 KHz to less than 20 KHz. The resolution of the clock generated is 1/10 MHz or 100 ns. For example, for an SCL of 400 KHz (period = 1/400 KHz = 2500 ns) and an internal time reference of 10 MHz (period = 100 ns):

$$\text{CLKDIV} = 2500 \text{ ns} / 100 \text{ ns} = 25$$

For an SCL with a 30% duty cycle, use `TWI_CLKDIV.CLKLO` = 17 and `TWI_CLKDIV.CLKHI` = 8.

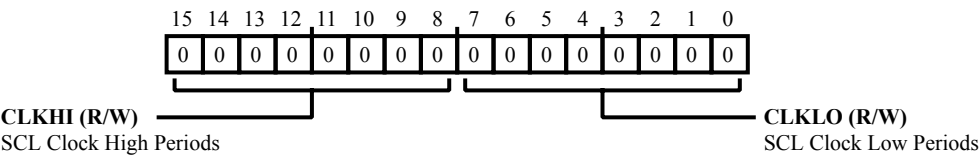


Figure 19-14: TWI_CLKDIV Register Diagram

Table 19-7: TWI_CLKDIV Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:8 (R/W)	CLKHI	SCL Clock High Periods. The <code>TWI_CLKDIV.CLKHI</code> specifies the number of 10 MHz time reference periods the serial clock (SCL) waits before a new clock low period begins, assuming a single requester.
7:0 (R/W)	CLKLO	SCL Clock Low Periods. The <code>TWI_CLKDIV.CLKLO</code> specifies the number of internal time reference periods the serial clock (SCL) is held low.

Control Register

The `TWI_CTL` enables the TWI, establishes a relationship between the system clock (SCLK) and the TWI controller's internally timed events, and enables SCCB compatibility.

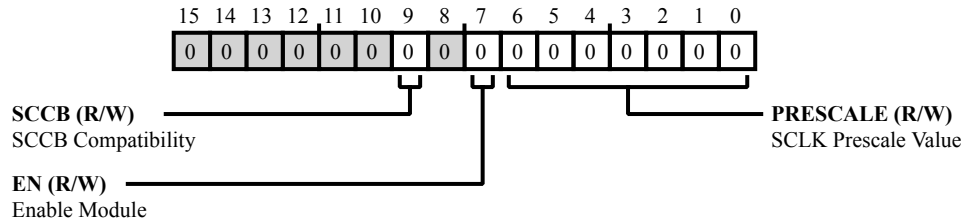


Figure 19-15: TWI_CTL Register Diagram

Table 19-8: TWI_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
9 (R/W)	SCCB	SCCB Compatibility. The <code>TWI_CTL.SCCB</code> enables SCCB compatible operation for the TWI. SCCB compatibility is an optional feature and should not be used in an I ² C bus system. When this feature is enabled, all completer asserted acknowledgement bits are ignored by this requester. This feature is valid only during transfers where the TWI is requesting an SCCB bus. Completer mode transfers should be avoided when this feature is enabled because the TWI controller always generates an acknowledge in completer mode.
		0 Disable SCCB compatibility. When disabled, requester transfers are not SCCB compatible.
		1 Enable SCCB compatibility. When enabled, requester transfers are SCCB compatible. All completer-asserted acknowledgment bits are ignored by this requester.
7 (R/W)	EN	Enable Module. The <code>TWI_CTL.EN</code> enables TWI controller operation for either requester and/or completer mode of operation. It is recommended that this bit be set at the time <code>TWI_CTL.PRESCALE</code> is initialized and remain set. This method guarantees accurate operation of bus busy detection logic.
		0 Disable
		1 Enable

Table 19-8: TWI_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
6:0 (R/W)	PRESCALE	SCLK Prescale Value. The <code>TWI_CTL.PRESCALE</code> holds the pre-scaled value for the TWI internal time reference. This reference is derived from SCLK according to the formula: $\text{TWI_CTL.PRESCALE} = f_{\text{SCLK}}/10\text{MHz}$ The <code>TWI_CTL.PRESCALE</code> specifies the number of system clock (SCLK) periods used in the generation of one internal time reference. The value of <code>TWI_CTL.PRESCALE</code> must be set to create an internal time reference with a period of 10 MHz. It is represented as a 7-bit binary value.

FIFO Control Register

The `TWI_FIFCTL` control bits affect only the FIFO and are not tied in any way with controller or target mode operation.

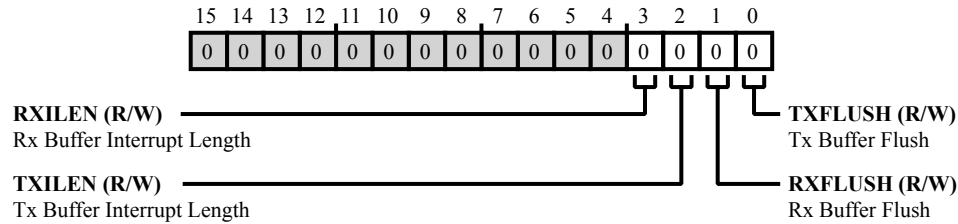


Figure 19-16: `TWI_FIFCTL` Register Diagram

Table 19-9: `TWI_FIFCTL` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3 (R/W)	RXILEN	Rx Buffer Interrupt Length. The <code>TWI_FIFCTL.RXILEN</code> determines the rate at which receive buffer interrupts are to be generated. Interrupts may be generated with each byte received or after two bytes are received. Interrupt status is available in <code>TWI_FIFOSTAT.RXSTAT</code> .
		0 RXSERVI on 1 or 2 Bytes in FIFO
		1 RXSERVI on 2 Bytes in FIFO
2 (R/W)	TXILEN	Tx Buffer Interrupt Length. The <code>TWI_FIFCTL.TXILEN</code> determines the rate at which transmit buffer interrupts are to be generated. Interrupts may be generated with each byte transmitted or after two bytes are transmitted. Interrupt status is available in <code>TWI_FIFOSTAT.TXSTAT</code> .
		0 TXSERVI on 1 Byte of FIFO Empty
		1 TXSERVI on 2 Bytes of FIFO Empty
1 (R/W)	RXFLUSH	Rx Buffer Flush. The <code>TWI_FIFCTL.RXFLUSH</code> directs the TWI to flush the contents of the receive buffer and update <code>TWI_FIFOSTAT.RXSTAT</code> to indicate the buffer is empty. This state is held until this bit is cleared. During an active receive, the receive buffer in this state responds to the receive logic as if it is full.
		0 Normal Operation of Rx Buffer
		1 Flush Rx Buffer

Table 19-9: TWI_FIFOCTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	TXFLUSH	Tx Buffer Flush. The <code>TWI_FIFOCTL.TXFLUSH</code> directs the TWI to flush the contents of the transmit buffer and update <code>TWI_FIFOSTAT.TXSTAT</code> to indicate the buffer is empty. This state is held until this bit is cleared. During an active transmit, the transmit buffer in this state responds to the transmit logic as if it is empty.
		0 Normal Operation of Tx Buffer
		1 Flush Tx Buffer

FIFO Status Register

The `TWI_FIFOSTAT` fields indicate the state of the FIFO buffers' receive and transmit contents. The FIFO buffers do not discriminate between controller data and target data. By using the status and control bits provided, the FIFO can be managed to allow simultaneous controller and target operation.

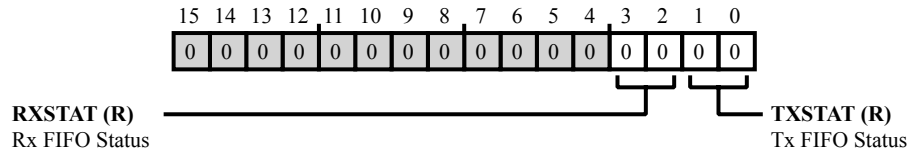


Figure 19-17: TWI_FIFOSTAT Register Diagram

Table 19-10: TWI_FIFOSTAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:2 (R/NW)	RXSTAT	Rx FIFO Status. The read-only <code>TWI_FIFOSTAT.RXSTAT</code> indicates the number of valid data bytes in the receive FIFO buffer. The status is updated with each FIFO buffer read using the peripheral data bus or write access by the receive shift register. Simultaneous accesses are allowed.
		0 Empty. The FIFO is empty.
		1 Contains 1 Byte. The FIFO contains one byte of data. A single byte peripheral read of the FIFO is allowed.
		2 Reserved
		3 Full. The FIFO is full and contains two bytes of data. Either a single or double byte peripheral read of the FIFO is allowed.
1:0 (R/NW)	TXSTAT	Tx FIFO Status. The read-only <code>TWI_FIFOSTAT.TXSTAT</code> field indicates the number of valid data bytes in the FIFO buffer. The status is updated with each FIFO buffer write using the peripheral data bus or read access by the transmit shift register. Simultaneous accesses are allowed.
		0 Empty. The FIFO is empty. Either a single or double byte peripheral write of the FIFO is allowed.
		1 Contains 1 Byte. The FIFO contains one byte of data. A single byte peripheral write of the FIFO is allowed.
		2 Reserved
		3 Full. The FIFO is full and contains two bytes of data.

Interrupt Mask Register

The **TWI_IMSK** enables interrupt sources to assert the interrupt output. Each mask bit corresponds with one interrupt request source bit in **TWI_ISTAT**. Reading and writing **TWI_IMSK** does not affect the contents of the **TWI_ISTAT**.

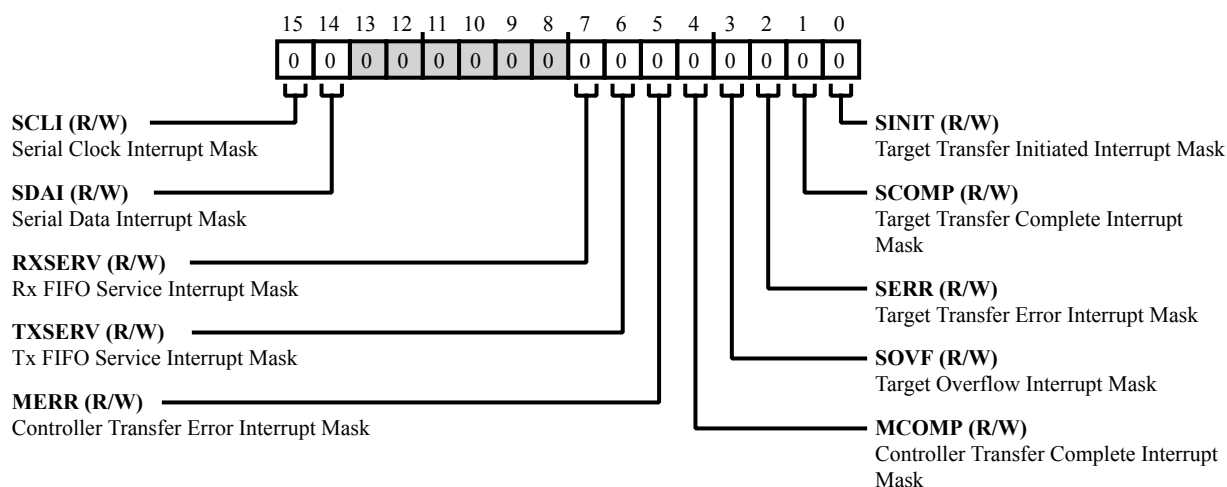


Figure 19-18: TWI_IMSK Register Diagram

Table 19-11: TWI_IMSK Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W)	SCLI	Serial Clock Interrupt Mask.
		0 Mask (Disable) Interrupt
		1 Unmask (Enable) Interrupt
14 (R/W)	SDAI	Serial Data Interrupt Mask.
		0 Mask (Disable) Interrupt
		1 Unmask (Enable) Interrupt
7 (R/W)	RXSERV	Rx FIFO Service Interrupt Mask.
		0 Mask (Disable) Interrupt
		1 Unmask (Enable) Interrupt
6 (R/W)	TXSERV	Tx FIFO Service Interrupt Mask.
		0 Mask (Disable) Interrupt
		1 Unmask (Enable) Interrupt

Table 19-11: TWI_IMSK Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
5 (R/W)	MERR	Controller Transfer Error Interrupt Mask.	
		0	Mask (Disable) Interrupt
		1	Unmask (Enable) Interrupt
4 (R/W)	MCOMP	Controller Transfer Complete Interrupt Mask.	
		0	Mask (Disable) Interrupt
		1	Unmask (Enable) Interrupt
3 (R/W)	SOVF	Target Overflow Interrupt Mask.	
		0	Mask (Disable) Interrupt
		1	Unmask (Enable) Interrupt
2 (R/W)	SERR	Target Transfer Error Interrupt Mask.	
		0	Mask (Disable) Interrupt
		1	Unmask (Enable) Interrupt
1 (R/W)	SCOMP	Target Transfer Complete Interrupt Mask.	
		0	Mask (Disable) Interrupt
		1	Unmask (Enable) Interrupt
0 (R/W)	SINIT	Target Transfer Initiated Interrupt Mask.	
		0	Mask (Disable) Interrupt
		1	Unmask (Enable) Interrupt

Interrupt Status Register

The `TWI_ISTAT` contains information about functional areas requiring servicing. Many of the bits serve as an indicator to further read and service various status registers. After servicing the interrupt source associated with a bit, the user must clear that interrupt source bit by writing a 1 to it.

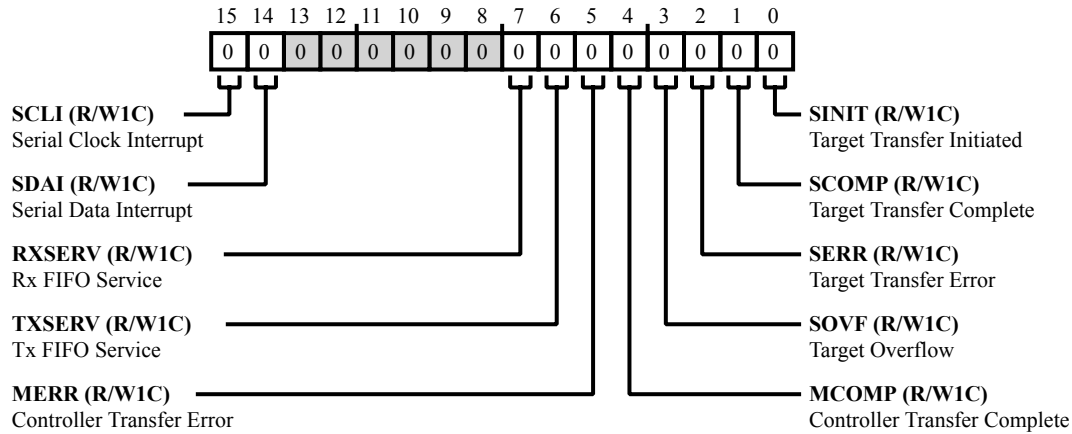


Figure 19-19: TWI_ISTAT Register Diagram

Table 19-12: TWI_ISTAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W1C)	SCLI	Serial Clock Interrupt. If the TWI is enabled (<code>TWI_CTL.EN</code>), SCLI is set on a high-to-low transition of the serial clock pin (<code>SCLx</code>). Normally, this bit is not required for I ² C bus transfers. It will be initially set on an I ² C transfer and does not require clearing.
		0 No Interrupt. No transition was detected on the <code>SCLx</code> pin.
		1 Interrupt Detected. A high-to-low transition was detected on the <code>SCLx</code> pin. This bit is W1C.
14 (R/W1C)	SDAI	Serial Data Interrupt. If the TWI is enabled (<code>TWI_CTL.EN</code>), SDAI is set on a high-to-low transition of the serial data pin (<code>SDAx</code>). Normally, this bit is not required for I ² C bus transfers. It will be initially set on an I ² C transfer and does not require clearing.
		0 No Interrupt. No transition was detected on the <code>SDAx</code> pin.
		1 Interrupt Detected. A high-to-low transition was detected on the <code>SDAx</code> pin. This bit is W1C.

Table 19-12: TWI_ISTAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
7 (R/W1C)	RXSERV	Rx FIFO Service. If <code>TWI_FIFCTL.RXILEN = 0</code> , the <code>TWI_ISTAT.RXSERV</code> is set each time the <code>TWI_FIFOSTAT.RXSTAT</code> field is updated to either 01 or 11. If <code>TWI_FIFCTL.RXILEN = 1</code> , the <code>TWI_ISTAT.RXSERV</code> is set each time <code>TWI_FIFOSTAT.RXSTAT</code> is updated to 11.
		0 No Interrupt. The FIFO does not require servicing, or the <code>TWI_FIFOSTAT.RXSTAT</code> field has not changed since this bit was last cleared.
		1 Interrupt Detected. The receive FIFO buffer has one or two 8-bit words of data available to be read.
6 (R/W1C)	TXSERV	Tx FIFO Service. If <code>TWI_FIFCTL.TXILEN = 0</code> , the <code>TWI_ISTAT.TXSERV</code> is set each time the <code>TWI_FIFOSTAT.TXSTAT</code> field is updated to either 01 or 00. If <code>TWI_FIFCTL.TXILEN = 1</code> , the <code>TWI_ISTAT.TXSERV</code> is set each time <code>TWI_FIFOSTAT.TXSTAT</code> is updated to 00.
		0 No Interrupt. FIFO does not require servicing, or the <code>TWI_FIFOSTAT.TXSTAT</code> field has not changed since this bit was last cleared.
		1 Interrupt Detected. The transmit FIFO buffer has one or two 8-bit locations available to be written.
5 (R/W1C)	MERR	Controller Transfer Error. The <code>TWI_ISTAT.MERR</code> indicates that a requester error has occurred. The conditions surrounding the error are indicated by the requester status register (TWI_MSTRSTAT).
		0 No Interrupt
		1 Interrupt Detected
4 (R/W1C)	MCOMP	Controller Transfer Complete. The <code>TWI_ISTAT.MCOMP</code> indicates that the initiated requester transfer has completed. In the absence of a repeat start, the bus has been released.
		0 No Interrupt
		1 Interrupt Detected

Table 19-12: TWI_ISTAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
3 (R/W1C)	SOVF	Target Overflow. The <code>TWI_ISTAT.SOVF</code> indicates that the <code>TWI_ISTAT.SCOMP</code> bit was set at the time a subsequent transfer has acknowledged an address phase. The transfer continues, however, it may be difficult to delineate data of one transfer from another.
		0 No Interrupt
		1 Interrupt Detected
2 (R/W1C)	SERR	Target Transfer Error. The <code>TWI_ISTAT.SERR</code> indicates that a completer error has occurred. A restart or stop condition has occurred during the data receive phase of a transfer.
		0 No Interrupt
		1 Interrupt Detected
1 (R/W1C)	SCOMP	Target Transfer Complete. The <code>TWI_ISTAT.SCOMP</code> indicates that the transfer is complete and either a stop, or a restart was detected.
		0 No Interrupt
		1 Interrupt Detected
0 (R/W1C)	SINIT	Target Transfer Initiated. The <code>TWI_ISTAT.SINIT</code> indicates whether or not a completer transfer is in progress.
		0 No Interrupt. A transfer is not in progress, or an address match has not occurred since the last time this bit was cleared.
		1 Interrupt Detected. The completer has detected an address match, and a transfer has been initiated.

Controller Mode Address Register

During the addressing phase of a transfer, the TWI controller, with its controller enabled, transmits the contents of `TWI_MSTRADDR`. When programming this register, omit the read/write bit. That is, only the upper 7 bits that make up the target address should be written to this register. For example, if the target address is `b#1010000X`, where `X` is the read/write bit, the `TWI_MSTRADDR` is programmed with `b#1010000`, which corresponds to `0x50`. When sending out the address on the bus, the TWI controller appends the read/write bit as appropriate based on the state of the `TWI_MSTRCTL.DIR` bit.

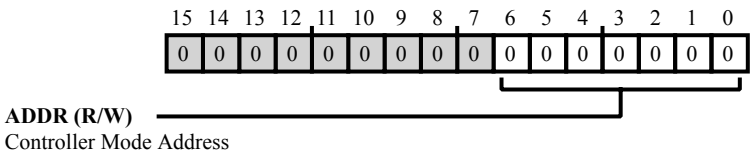


Figure 19-20: TWI_MSTRADDR Register Diagram

Table 19-13: TWI_MSTRADDR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
6:0 (R/W)	ADDR	Controller Mode Address.

Controller Mode Registers

The `TWI_MSTRCTL` controls the logic associated with controller mode operation. Bits in this register do not affect target mode operation and should not be modified to control target mode functionality.

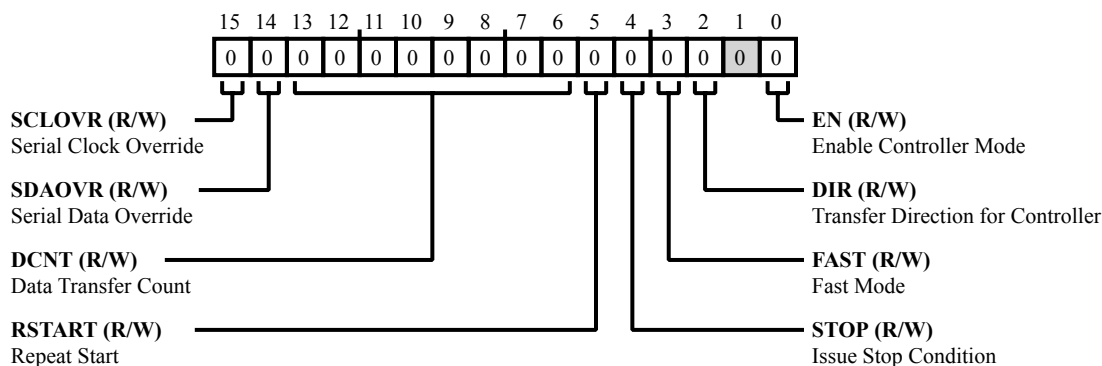


Figure 19-21: TWI_MSTRCTL Register Diagram

Table 19-14: TWI_MSTRCTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W)	SCLOVR	Serial Clock Override. The <code>TWI_MSTRCTL.SCLOVR</code> provides direct control of the serial clock line when required. Normal requester and completer mode operation should not require override operation. When <code>TWI_MSTRCTL.SCLOVR</code> is set, the TWI overrides normal serial clock output, driving it to an active 0 level and overriding all other logic. This state is held until this bit is cleared. When <code>TWI_MSTRCTL.SCLOVR</code> is cleared, the TWI permits normal serial clock operation under the control of requester mode clock generation and completer mode clock stretching logic.
		0 Permit Normal SCL Operation
		1 Override Normal SCL Operation
14 (R/W)	SDAOVR	Serial Data Override. The <code>TWI_MSTRCTL.SDAOVR</code> provides direct control of the serial data line when required. Normal requester and completer mode operation should not require override operation. When <code>TWI_MSTRCTL.SDAOVR</code> is set, the TWI overrides normal serial data operation under the control of the transmit shift register and acknowledge logic, driving serial data output to an active 0 level and overriding all other logic. This state is held until this bit is cleared. When <code>TWI_MSTRCTL.SDAOVR</code> is cleared, the TWI permits normal serial data operation.
		0 Permit Normal SDA Operation
		1 Override Normal SDA Operation

Table 19-14: TWI_MSTRCTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
13:6 (R/W)	DCNT	Data Transfer Count. The <code>TWI_MSTRCTL.DCNT</code> indicates the number of data bytes to transfer. As each data word is transferred, the TWI decrements this counter. When <code>TWI_MSTRCTL.DCNT</code> decrements to 0, a stop condition is generated. Setting <code>TWI_MSTRCTL.DCNT</code> to 0xFF disables the counter. In this transfer mode, data continues to be transferred until it is concluded by setting the <code>TWI_MSTRCTL.STOP</code> bit. In the event a requester transmit is aborted due to a completer data NAK, the value of <code>TWI_MSTRCTL.DCNT</code> equals the number of bytes not sent. The byte which was NAK'ed by the completer is counted as a sent byte.
5 (R/W)	RSTART	Repeat Start. The <code>TWI_MSTRCTL.RSTART</code> enables the TWI to issue a repeat start condition at the conclusion of the current transfer (<code>TWI_MSTRCTL.DCNT = 0</code>) and begin the next transfer. The current transfer concludes with updates to the appropriate status and interrupt bits. If errors occurred during the previous transfer, a repeat start does not occur. In the absence of any errors, requester enable (<code>TWI_MSTRCTL.EN</code>) does not self clear on a repeat start.
		0 Disable Repeat Start
		1 Enable Repeat Start
4 (R/W)	STOP	Issue Stop Condition. The <code>TWI_MSTRCTL.STOP</code> directs the TWI to issue a stop condition. The transfer concludes as soon as possible avoiding any error conditions (as if data transfer count had been reached). At that time, the <code>TWI_IMSK</code> is updated along with any associated status bits.
		0 Permit Normal Operation
		1 Issue Stop
3 (R/W)	FAST	Fast Mode. The <code>TWI_MSTRCTL.FAST</code> selects whether the TWI operates in fast mode or standard mode. In fast mode, the TWI uses timing specifications for transfers at up to 400K bits/s. In standard mode, the TWI uses timing specifications for transfers at up to 100K bits/s.
		0 Select Standard Mode
		1 Select Fast Mode
2 (R/W)	DIR	Transfer Direction for Controller. The <code>TWI_MSTRCTL.DIR</code> selects the transfer direction for the TWI as requester initiated receive or transmit.
		0 Requester Transmit
		1 Requester Receive

Table 19-14: TWI_MSTRCTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	EN	Enable Controller Mode. The <code>TWI_MSTRCTL.EN</code> enables requester mode functionality. A start condition is generated if the bus is idle. This bit self clears at the completion of a transfer (after <code>TWI_MSTRCTL.DCNT</code> decrements to zero), including transfers terminated due to errors. If disabled (=0) during operation, the transfer is aborted, and all logic associated with requester mode transfers are reset. Serial data and serial clock (SDA, SCL) are no longer driven. Write-1-to-clear status bits are not affected.
		0 Disable
		1 Enable

Controller Mode Status Register

The `TWI_MSTRSTAT` holds information during controller mode transfers and at their conclusion. Generally, controller mode status bits are not directly associated with the generation of interrupt requests, but these bits offer information on the current transfer. Target mode operation does not affect controller mode status bits.

Note that while `TWI_MSTRSTAT.SCLSEN` is set (this condition could be due to having no pull-up resistor on `TWI_SCL` or another agent is driving `TWI_SCL` low), the acknowledge bits (`TWI_MSTRSTAT.ANAK` and `TWI_MSTRSTAT.DNAK`) do not update. This result occurs because the acknowledge conditions are sampled during the high phase of `TWI_SCL`.

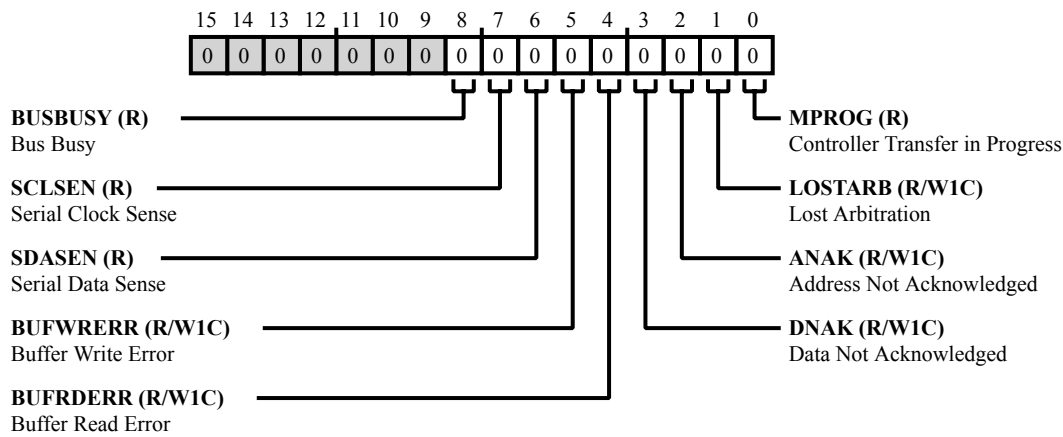


Figure 19-22: `TWI_MSTRSTAT` Register Diagram

Table 19-15: `TWI_MSTRSTAT` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
8 (R/NW)	BUSBUSY	Bus Busy. The <code>TWI_MSTRSTAT.BUSBUSY</code> indicates whether the bus is currently busy or free. This indication is not limited to only this device but is for all devices. On a start condition, the setting of the register value is delayed due to the input filtering. On a stop condition the clearing of the register value occurs after t_{BUF} .
		0 Bus Free. The bus is free. The clock and data bus signals have been inactive for the appropriate bus free time.
		1 Bus Busy. The bus is busy. Clock or data activity has been detected.

Table 19-15: TWI_MSTRSTAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
7 (R/NW)	SCLSEN	Serial Clock Sense. The TWI_MSTRSTAT.SCLSEN indicates the active or inactive state of the serial clock. Use this status bit when direct sensing of the serial clock line is required. The register value is delayed due to the input filter (nominally 50 ns). Normal requester and completer mode operation should not require this feature.
		0 SCL Inactive "One". An inactive "one" is being sensed on the serial clock.
		1 SCL Active "Zero". An active "zero" is being sensed on the serial clock. The source of the active driver is not known and can be internal or external.
6 (R/NW)	SDASEN	Serial Data Sense. The TWI_MSTRSTAT.SDASEN indicates the active or inactive status of the serial data. Use this status bit when direct sensing of the serial data line is required. The register value is delayed due to the input filter (nominally 50 ns). Normal requester and completer mode operation should not require this feature.
		0 SDA Inactive "One". An inactive "one" is currently being sensed on the serial data line.
		1 SDA Active "Zero". An active "zero" is currently being sensed on the serial data line. The source of the active driver is not known and can be internal or external.
5 (R/W1C)	BUFWREERR	Buffer Write Error. The TWI_MSTRSTAT.BUFWREERR indicates whether the current requester transfer was aborted due to a receive buffer write error. The receive buffer and receive shift register were both full at the same time. This bit is W1C.
		0 No Status
		1 Buffer Write Error
4 (R/W1C)	BUFRDERR	Buffer Read Error. The TWI_MSTRSTAT.BUFRDERR indicates whether the current requester transfer was aborted due to a transmit buffer read error. The error occurs if the buffer is empty when data is required by the transmit shift register. This bit is W1C.
		0 No Status
		1 Buffer Read Error

Table 19-15: TWI_MSTRSTAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
3 (R/W1C)	DNAK	Data Not Acknowledged. The <code>TWI_MSTRSTAT.DNAK</code> indicates whether the current requester transfer was aborted due to the detection of a NAK during data transmission. This bit is W1C.
		0 No Status
		1 Data NAK
2 (R/W1C)	ANAK	Address Not Acknowledged. The <code>TWI_MSTRSTAT.ANAK</code> indicates whether the current requester transfer was aborted due to the detection of a NAK during the address phase of the transfer. This bit is W1C.
		0 No Status
		1 Address NAK
1 (R/W1C)	LOSTARB	Lost Arbitration. The <code>TWI_MSTRSTAT.LOSTARB</code> indicates whether the current transfer was aborted due to the loss of arbitration with another requester. This bit is W1C.
		0 No Status
		1 Lost Arbitration
0 (R/NW)	MPROG	Controller Transfer in Progress. The <code>TWI_MSTRSTAT.MPROG</code> indicates whether or not a requester transfer is in progress. If clear (<code>TWI_MSTRSTAT.MPROG = 0</code>), currently no transfer is taking place. This can occur after a transfer is complete or while an enabled requester is waiting for an idle bus.
		0 No Status
		1 Requester Transfer in Progress

Rx Data Double-Byte Register

The `TWI_RXDATA16` holds a 16-bit data value read from the FIFO buffer. To reduce interrupt output rates and peripheral bus access times, a double byte receive data access can be performed. Two data bytes can be read, effectively emptying the receive FIFO buffer with a single access.

The data is read in little endian byte order, where byte 0 is the first byte received and byte 1 is the second byte received. With each access, the receive status (`TWI_FIFOSTAT.RXSTAT`) field is updated to indicate it is empty. If an access is performed while the FIFO buffer is not full, the read data is unknown and the existing FIFO buffer data and its status remains unchanged.

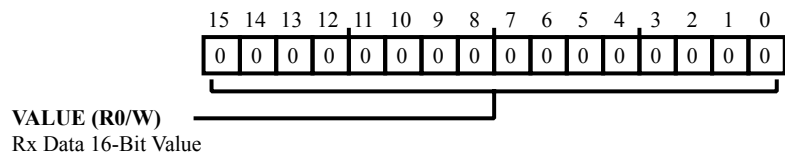


Figure 19-23: TWI_RXDATA16 Register Diagram

Table 19-16: TWI_RXDATA16 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R0/W)	VALUE	Rx Data 16-Bit Value.

Rx Data Single-Byte Register

The `TWI_RXDATA8` holds an 8-bit data value read from the FIFO buffer. Receive data is read from the corresponding receive buffer in a first-in first-out order. Although peripheral bus reads are 16 bits, a read access to `TWI_RXDATA8` accesses only one transmit data byte from the FIFO buffer. With each access, the receive status (`TWI_FIFOSTAT.RXSTAT`) field is updated. If an access is performed while the FIFO buffer is empty, the data is unknown and the FIFO buffer status remains indicating it is empty.

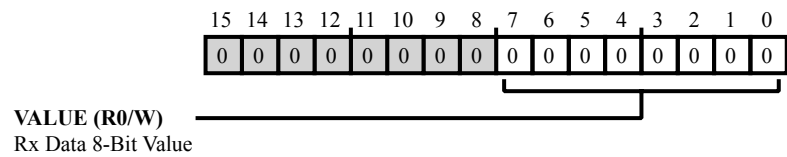


Figure 19-24: TWI_RXDATA8 Register Diagram

Table 19-17: TWI_RXDATA8 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R0/W)	VALUE	Rx Data 8-Bit Value.

Target Mode Address Register

The `TWI_SLVADDR` holds the target mode address, which is the valid address to which the target-enabled TWI controller responds. The TWI controller compares this value with the received address during the addressing phase of a transfer.

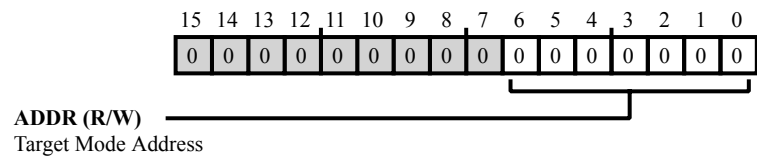


Figure 19-25: TWI_SLVADDR Register Diagram

Table 19-18: TWI_SLVADDR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
6:0 (R/W)	ADDR	Target Mode Address.

Target Mode Control Register

The `TWI_SLVCTL` controls the logic associated with target mode operation. Settings in this register do not affect controller mode operation and should not be modified to manage controller mode functionality.

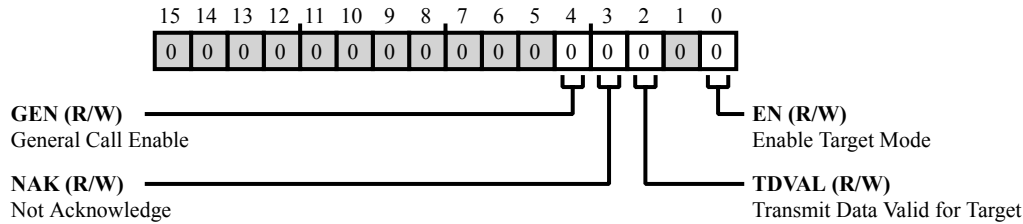


Figure 19-26: TWI_SLVCTL Register Diagram

Table 19-19: TWI_SLVCTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
4 (R/W)	GEN	General Call Enable. The <code>TWI_SLVCTL.GEN</code> enables general call address matching. When enabled, a general call completer receive transfer is accepted. All status and interrupt source bits associated with transfers are updated. Note that general call address detection is available only when completer mode is enabled.
		0 Disable General Call Matching
		1 Enable General Call Matching
3 (R/W)	NAK	Not Acknowledge. The <code>TWI_SLVCTL.NAK</code> directs the TWI to generate a NAK (if set) or an ACK (if cleared) at the conclusion of data transfer for completer receive. For NAK, the completer is still considered to be addressed at the conclusion of transfer.
		0 Generate ACK
		1 Generate NAK
2 (R/W)	TDVAL	Transmit Data Valid for Target. The <code>TWI_SLVCTL.TDVAL</code> selects whether the data in the transmit FIFO is available (valid) for completer transmission (<code>TWI_SLVCTL.TDVAL</code> set). If the FIFO data is not available (invalid) for completer transmission (<code>TWI_SLVCTL.TDVAL</code> cleared), the data in the transmit FIFO is for requester mode transmits, and the data is not allowed to be used during a completer transmit; the transmit FIFO is treated as if it is empty.
		0 Data Invalid for Completer Tx
		1 Data Valid for Completer Tx

Table 19-19: TWI_SLVCTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	EN	Enable Target Mode. The <code>TWI_SLVCTL.EN</code> enables completer operation. Enabling completer and requester modes of operation concurrently is allowed. If disabled, no attempt is made to identify a valid address. If <code>TWI_SLVCTL.EN</code> is cleared during a valid transfer, clock stretching ceases, the serial data line is released, and the current byte is not acknowledged.
		0 Disable
		1 Enable

Target Mode Status Register

During and at the conclusion of register target mode transfers, the `TWI_SLVSTAT` holds information on the current transfer. Generally target mode status bits are not associated with the generation of interrupt requests. Controller mode operation does not affect target mode status bits.

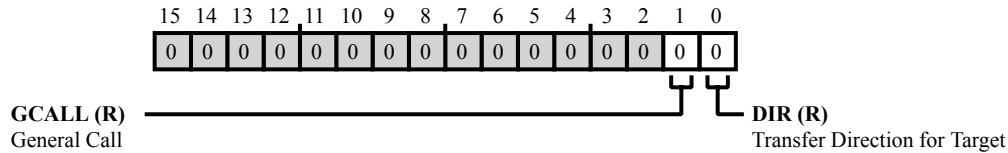


Figure 19-27: TWI_SLVSTAT Register Diagram

Table 19-20: TWI_SLVSTAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/NW)	GCALL	General Call. The <code>TWI_SLVSTAT.GCALL</code> indicates whether or not--at the time of addressing--the address was determined to be a general call. This bit self clears if completer mode is disabled (<code>TWI_SLVCTL.EN = 0</code>).
		0 Not a General Call Address
		1 General Call Address
0 (R/NW)	DIR	Transfer Direction for Target. The <code>TWI_SLVSTAT.DIR</code> indicates whether--at the time of addressing--the transfer direction was determined to be completer transmit or receive. This bit self clears if completer mode is disabled (<code>TWI_SLVCTL.EN = 0</code>).
		0 Completer Receive
		1 Completer Transmit

Tx Data Double-Byte Register

The `TWI_TXDATA16` register holds a 16-bit data value written into the FIFO buffer. To reduce interrupt latency output rates and peripheral bus access times, a double byte transfer data access can be done. Two data bytes can be written, effectively filling the transmit FIFO buffer with a single access.

The data is written in little endian byte order, where byte 0 is the first byte to be transferred and byte 1 is the second byte to be transferred. With each access, the transmit status (`TWI_FIFOSTAT.TXSTAT`) field is updated. If an access is performed while the FIFO buffer is not empty, the write is ignored and the existing FIFO buffer data and its status remains unchanged. This register when read back returns zero.

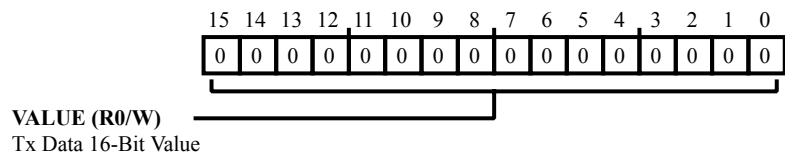


Figure 19-28: TWI_TXDATA16 Register Diagram

Table 19-21: TWI_TXDATA16 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R0/W)	VALUE	Tx Data 16-Bit Value.

Tx Data Single-Byte Register

The `TWI_TXDATA8` register holds an 8-bit data value written into the FIFO buffer. Transmit data is entered into the corresponding transmit buffer in a first-in first-out order. For 16-bit peripheral bus writes, a write access to this register adds only one transmit data byte to the FIFO buffer. With each access, the transmit status (`TWI_FIFOSTAT.TXSTAT`) field is updated. If an access is performed while the FIFO buffer is full, the write is ignored and the existing FIFO buffer data and its status remains unchanged. This register returns zero when read back.

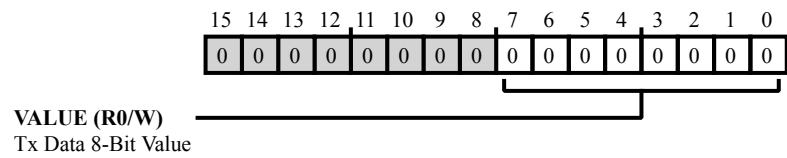


Figure 19-29: TWI_TXDATA8 Register Diagram

Table 19-22: TWI_TXDATA8 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R0/W)	VALUE	Tx Data 8-Bit Value.

20 Digital Audio Interface (DAI)

The Digital Audio Interfaces (DAI_n) are comprised of groups of identical peripherals and their respective Signal Routing Units (SRU_n). The SRU connects inputs and outputs of the DAI peripherals with each other and to the external pins. This configuration allows peripherals to be interconnected to accommodate a wide variety of systems without making external pin connections.

In a typical processor, static (multiplexed) pins are assigned to specific peripherals. When certain peripherals are not required for an application, these pins are unnecessary and expensive. The pins may need to be defined as high or low to prevent any illegal conditions. The signal routing units on the SHARC processors addresses this situation by controlling a number of general-purpose pins which can be assigned flexibly (a virtual connectivity between peripherals) depending on system requirements. This virtual connectivity includes pin buffers and routing logic (multiplexer). It also allows the SHARC processors to include an arbitrary number and variety of peripherals while retaining high levels of compatibility without increasing pin count.

SRU Features

The SRU has the following features and capabilities.

- Flexible connections can be made through software and during run time; no hard wiring is required.
- At reset, a default routing scheme is already programmed.
- Connectivity can be made internally between peripherals, externally between pin buffers, or a mix of both.
- Status of the pin buffers can be programmed for conditional execution or interrupts.
- Some pin buffers allow control of signal polarity changes.
- No fan-out limitation, a peripheral or pin buffer output that can be routed to multiple peripheral or pin buffer inputs.
- Two independent routing systems are available the DAI0 and the DAI1. Signals cannot be interconnected between both routing units with the exception of the following modules under some specific scenarios.
 - The precision clock generator (PCG)
 - A pair of DAI pin buffers from each DAI unit
 - Asynchronous Sample Rate Converter (ASRC) data for daisy chaining across the DAI units

NOTE: This processor has 2 x 12 DAI pins bonded off-chip (DAIx_PIN01 through DAIx_PIN10, DAIx_PIN19, and DAIx_PIN20).

Functional Description

The fundamental timing clock of the DAI modules is SCLK.

The [DAI Block Diagram](#) shows how the DAI pin buffers are connected through the SRUn. This configuration allows for flexible signal routing.

The DAI units are comprised of four primary blocks:

- Peripherals (A/B/C) associated with DAI_n
- Signal Routing Units (SRUn)
- DAI_n I/O pin buffers
- Miscellaneous buffers

The peripherals shown in [DAI Block Diagram](#) can have up to three connections (if requester or completer capable); one acts as a signal input, one as a signal output and the third as an output enable. The SRUs are based on a group of multiplexers which are controlled by registers to establish the desired interconnections. The DAI pin buffers have three signals which are used for input and output to or from off-chip and the third for output enable.

The miscellaneous buffers have an input and an output and are used for group interconnection.

The figures are a simplified representation of a DAI system. In a real representation, the SRU and DAI would show several types of data being routed from several sources including the following:

- Serial ports (SPORT)
- Precision clock generators (PCG)
- Asynchronous sample rate converters (SRC)
- S/PDIF transmitter
- S/PDIF receiver
- DAI interrupts (miscellaneous)
- PDM interface

ADSP-21560_21561_21564_21568 DAI Register List

The Digital Audio Interfaces (DAI_n) contain groups of identical peripherals which can be connected internally between peripherals, externally between pin buffers, or a mix of both. This module contains the following registers.

Table 20-1: ADSP-21560_21561_21564_21568 DAI Register List

Name	Description
DAI_CLK0	Clock Routing Control Register 0
DAI_CLK1	Clock Routing Control Register 1
DAI_CLK2	Clock Routing Control Register 2
DAI_CLK3	Clock Routing Control Register 3
DAI_CLK4	Clock Routing Control Register 4
DAI_CLK5	Clock Routing Control Register 5
DAI_DAT0	Serial Data Routing Control Register 0
DAI_DAT1	Serial Data Routing Control Register 1
DAI_DAT2	Serial Data Routing Control Register 2
DAI_DAT3	Serial Data Routing Control Register 3
DAI_DAT4	Serial Data Routing Control Register 4
DAI_DAT5	Serial Data Routing Control Register 5
DAI_DAT6	Serial Data Routing Control Register 6
DAI_FS0	Frame Sync Routing Control Register 0
DAI_FS1	Frame Sync Routing Control Register 1
DAI_FS2	Frame Sync Routing Control Register 2
DAI_FS4	Frame Sync Routing Control Register 4
DAI_GBL_INT_EN	Global SPORT Interrupt Grouping Register
DAI_GBL_SP_EN	Global SPORT Enable Register
DAI_IMSK_FE	Falling-Edge Interrupt Mask Register
DAI_IMSK_PRI	Core Interrupt Priority Assignment Register
DAI_IMSK_RE	Rising-Edge Interrupt Mask Register
DAI_IRPTL_H	High Priority Interrupt Latch Register
DAI_IRPTL_HS	Shadow High Priority Interrupt Latch Register
DAI_IRPTL_L	Low Priority Interrupt Latch Register
DAI_IRPTL_LS	Shadow Low Priority Interrupt Latch Register
DAI_MISC0	Miscellaneous Control Register 0
DAI_MISC1	Miscellaneous Control Register 1
DAI_PBEN0	Pin Buffer Enable Register 0
DAI_PBEN1	Pin Buffer Enable Register 1
DAI_PBEN2	Pin Buffer Enable Register 2

Table 20-1: ADSP-21560_21561_21564_21568 DAI Register List (Continued)

Name	Description
DAI_PBEN3	Pin Buffer Enable Register 3
DAI_PIN0	Pin Buffer Assignment Register 0
DAI_PIN1	Pin Buffer Assignment Register 1
DAI_PIN2	Pin Buffer Assignment Register 2
DAI_PIN4	Pin Buffer Assignment Register 4
DAI_PIN_STAT	Pin Status Register

ADSP-21560_21561_21564_21568 DAI Interrupt List

Table 20-2: ADSP-21560_21561_21564_21568 DAI Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
20	DAI0_IRQH	DAI0 High priority	None	
21	DAI1_IRQH	DAI1 High priority	None	
116	DAI0_IRQL	DAI0 Low Priority Interrupt	None	
117	DAI1_IRQL	DAI1 Low Priority Interrupt	None	
189	DAI0_GBL_SPORT_INT0	DAI0 DAI0 Global SPORT interrupt0	None	
190	DAI0_GBL_SPORT_INT1	DAI0 DAI0 Global SPORT interrupt1	None	
191	DAI1_GBL_SPORT_INT0	DAI1 DAI1 Global SPORT interrupt0	None	
192	DAI1_GBL_SPORT_INT1	DAI1 DAI1 Global SPORT interrupt1	None	

ADSP-21560_21561_21564_21568 DAI Trigger List

Table 20-3: ADSP-21560_21561_21564_21568 DAI Trigger List Generators

Trigger ID	Name	Description	Sensitivity
105	DAI0_GBL_SPORT_TRG_00	DAI0 SPORT GROUP0 Trigger Output	None
106	DAI0_GBL_SPORT_TRG_01	DAI0 SPORT GROUP1 Trigger Output	None
107	DAI1_GBL_SPORT_TRG_00	DAI1 SPORT GROUP2 Trigger Output	None
108	DAI1_GBL_SPORT_TRG_01	DAI1 SPORT Group3 Trigger Output	None

Table 20-4: ADSP-21560_21561_21564_21568 DAI Trigger List Receivers

Trigger ID	Name	Description	Sensitivity
None			

DAI Block Diagram

The *DAI Functional Block Diagram* and the *Digital Audio Interconnect Unit* figures show the functional blocks within the DAI and the unit connections to the peripherals.

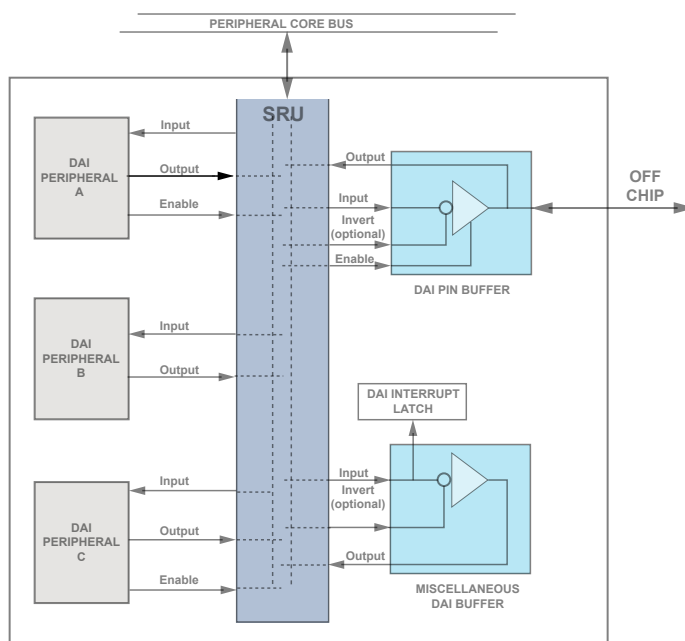


Figure 20-1: DAI Functional Block Diagram

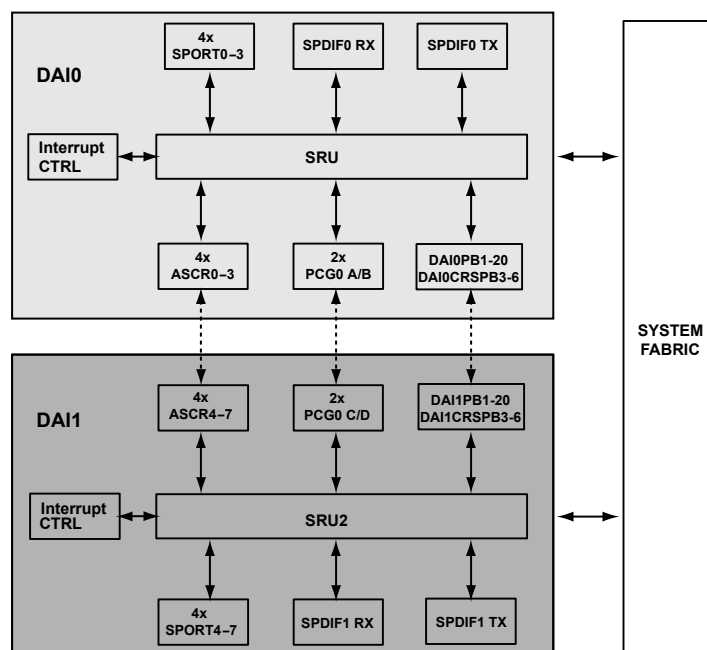


Figure 20-2: Digital Audio Interconnect Unit

DAI Signal Naming Conventions

The peripherals associated with the DAI do not have any dedicated I/O pins for off-chip communication. Instead, the I/O pin is only accessible in the chip internally and is known as an *internal node*. Every internal node of a DAI peripheral (input or output) is given a unique mnemonic. The convention is to begin the name with an identifier for the peripheral that the signal is coming to or from, followed by the function of the signal.

A number is included if the DAI contains more than one peripheral type (for example, serial ports), or if the peripheral has more than one signal that performs this function (for example, serial ports). The mnemonic always ends with *_I* if the signal is an input, or with *_O* if the signal is an output. An example is shown in the *Example DAI Mnemonics* figure.

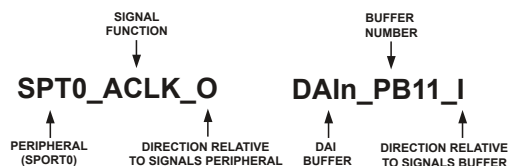


Figure 20-3: Example DAI Mnemonics

I/O Pin Buffers

Within the context of the SRU, physical connections to the DAI pins are replaced by a logical interface known as a *pin buffer*. This three terminal active device is capable of sourcing or sinking output current when its driver is enabled, and passing external input signals when disabled. Each pin has an input, an output, and an enable as shown in the *Pin Buffer Example* figure. The inputs and the outputs are defined with respect to the pin, like a peripheral device. This naming convention is consistent with the SRU naming convention.

Pin Buffer Signals

The pin buffer is based on three signals shown in the *Pin Buffer Example* figure and described in the following sections.

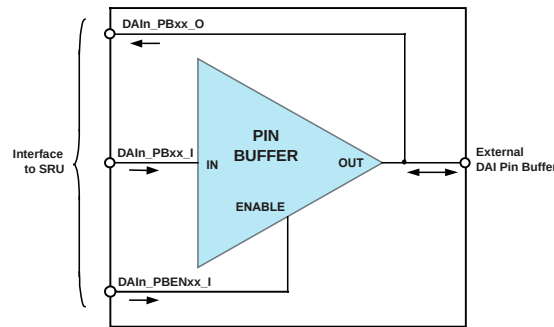


Figure 20-4: Pin Buffer Example

Pin Buffer Input Signal

A pin buffer input (DAIn_PBxx_I) is driven as an output from the processor when the pin buffer enable is set (=1). Each physical pin (connected to a bonded pad) can be connected through the SRU to any of the outputs of the DAI peripherals, based on the bit field values. The SRU can also be used to route signals that control the pins in other ways. Many signals can be configured for use as control signals.

Pin Buffer Enable Signal

When a pin buffer enable (DAIn_PBENxx_I) is set (=1), the signal present at the corresponding pin buffer input (DAIn_PBxx_I) is driven off-chip as an output. When a pin buffer enable is cleared (=0), the signal present at the corresponding pin buffer input is ignored. The pin enable control registers activate the drive buffer for each of the DAI pins. When the pins are not enabled (driven), they can be used as inputs. There are two options to control the pin buffer enable signal; setting the level high for a static solution or connecting the dedicated peripheral's pin buffer output enable signal to its pin buffer, which automatically enables the pin buffer.

Pin Buffer Functions

Pin buffers can be configured as inputs or outputs as described in the following sections.

Pin Buffers as Signal Input

When the DAI pin is used only as an input, connect the corresponding pin buffer enable to logic low as shown in the *Pin Buffer as Input* figure. This configuration disables the buffer amplifier and allows an off-chip source to drive the value present on the DAI pin and at the pin buffer output. When the pin buffer enable (for example, DAI_PBEN0.PB01) is cleared (= 0), the pin buffer output (DAIn_PBxx_O) is the signal driven onto the DAI pin by an external source, and the pin buffer input signal (DAIn_PBxx_I) is not used.

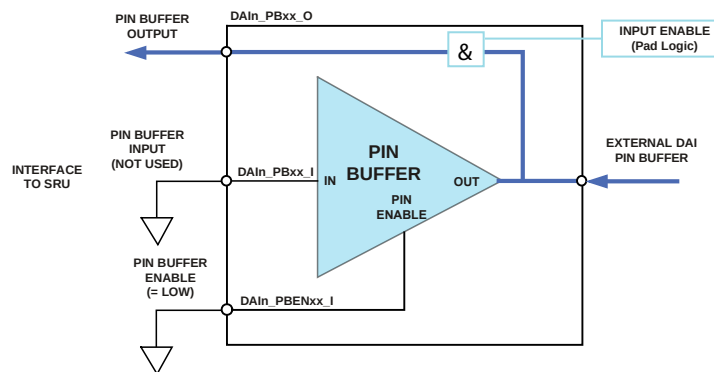


Figure 20-5: Pin Buffer as Input

NOTE: Whether programmed as input or output, a DAI buffer input always routes the same signal to an output internally. DAI pins have programmable internal pull-up resistors. For more information, see the General-Purpose Ports (PORT) chapter for details.

Pin Buffers As Signal Output

In a typical embedded system, most pins are designated as either inputs or outputs when the circuit is designed, even though they can be used in either direction. Each of the DAI pins can be used as either an output or an input. Although the direction of a DAI pin is set simply by writing to a memory-mapped register, most often the direction of the pin is dictated by the designated use of that pin.

When the DAI pin is used only as an output, connect the corresponding pin buffer enable to logic high as shown in the *Pin Buffer as Output* figure. This configuration enables the buffer amplifier to operate as a current source and to drive the value present at the pin buffer input onto the DAI pin and off-chip. When the pin buffer enable bits are set (in the `DAIn_PBxx_I` registers) ($=1$), the pin buffer output (`DAIn_PBxx_O`) is the same signal as the pin buffer input (`DAIn_PBxx_I`), and this signal is driven as an output.

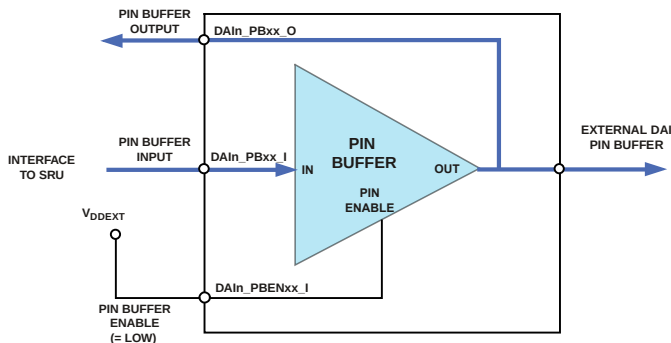


Figure 20-6: Pin Buffer as Output

DAI Pin Buffer Status

The signal levels on the DAI pins can be read with the `DAI_PIN_STAT` registers.

DAIn Peripherals

There are two categories of peripherals associated with the DAI units. These are described in the following sections.

Output Signals With Pin Buffer Enable Control

Many peripherals within the DAI that have bidirectional pins generate a corresponding pin enable signal. Typically, the settings within the control registers of a peripheral determine if a bidirectional pin is an input or an output. The pin is then driven accordingly.

Though most peripherals can operate bidirectionally, it is not required that all of the `_I` and `_O` signals of a peripheral be connected to the pin buffer. When the system design only uses a signal in one direction, it is simpler to connect the pin buffer accordingly.

NOTE: All available pin buffer output enables must be routed to their pin buffer input enable signals in cases where data streaming connections are used. This arrangement guarantees timing requirements.

NOTE: In some cases, it is necessary to use a peripheral's dedicated pin buffer enable signal instead of static levels. For example, SPORT TDM mode requires the SPORT's dedicated data pin buffer enable signal to be used for the SPORT's data pin to three-state the data pins on inactive channels.

Output Signals Without Pin Buffer Enable Control

Some peripherals have signal outputs without an automated pin buffer control enable signal. The operation of these peripherals is simplified. The routing to a DAI pin buffer enable input requires a static high from the SRUn. To disable the pin buffer output, software must clear the pin buffer enable input.

Signal Routing Units (SRUs)

The following sections provide details specific to the SRUs.

Cross Mode Connections

The DAI allows a few signals to be interconnected across DAI units. These connections are commonly referred as *cross mode connections*. These cross mode connections are useful for the system designer to share signals across both DAIs so that synchronization can be achieved on the peripheral modules across DAI0 and DAI1.

NOTE: This processor has 2 x 12 DAI pins bonded off-chip (DAIx_PIN01 through DAIx_PIN10, DAIx_PIN19, and DAIx_PIN20).

Shared Clock

This cross mode connection (see [Figure 20-7 Cross Connect Clock Example](#)) can be used to share a clock signal across DAIs. DAI0_PIN03/DAI0_PIN05 and DAI1_PIN03/DAI1_PIN05 can be used to share clock signal across DAI. When DAI0_PIN03/DAI0_PIN05 is configured as input clock pin then this clock signal is available in Group A of both DAIs. This source signal is DAI0_PB03_O/DAI0_PB05_O in group A of DAI 0 and DAI1_CRS_PB03_O/DAI1_CRS_PB05_O in group A of DAI1.

Peripherals routed through DAI0 like PCG A and SPORT 0A can use DAI0_PB03_O/DAI0_PB05_O as the input clock signal. Peripherals routed through DAI1 like PCG C, SPORT 4A can use DAI1_CRS_PB03/DAI1_CRS_PB05 as the input clock signal. Similarly when DAI1_PIN03 is configured as the input clock pin, then this clock signal is available as DAI1_PB03_O/DAI1_PB05_O in group A of DAI1 and DAI0_CRS_PB03/DAI0_CRS_PB05 in group A of DAI0.

Shared Frame Sync

This cross mode connection can be used to share a frame sync signal cross DAIs. DAI0_PIN04/DAI0_PIN06 and DAI1_PIN04/DAI1_PIN06 can be used to share the frame sync signal across the DAIs. When DAI0_PIN04/DAI0_PIN06 is configured as the input frame sync pin, then this frame sync signal is available in Group C of both DAIs. This source signal is DAI0_PB04_O/DAI0_PB06_O in group C of DAI 0 and DAI1_CRS_PB04_O/DAI1_CRS_PB06_O in group C of DAI1.

Peripherals on DAI0 like SPORT 0A can use DAI0_PB04_O/DAI0_PB06_O as the input frame sync signal. The peripherals on DAI1 like SPORT 4A can use DAI1_CRS_PB04/DAI1_CRS_PB06 as the input sync signal. Similarly, when DAI1_PIN04 /DAI1_PIN06 is configured as input frame sync pin, then this frame sync signal is available as DAI1_PB04_O/DAI1_PB06_O in group A of DAI1 and DAI0_CRS_PB04/DAI0_CRS_PB06 in group C of DAI0.

Shared Pin Buffer

This cross mode connection can be used to connect input signals on DAI0_PIN03/DAI0_PIN05, DAI0_PIN04/DAI0_PIN06, DAI1_PIN03/DAI1_PIN05 and DAI1_PIN04 /DAI1_PIN06 to other DAI pins across DAI. When DAI0_PIN03/DAI0_PIN05 and DAI0_PIN04 are configured as input signals, these signals can be routed to the pin buffers of DAI1 in Group D using DAI1_CRS_PIN03/DAI1_CRS_PIN05 and DAI1_CRS_PIN04. When DAI1_PIN03/DAI1_PIN05 and DAI1_PIN04/DAI1_PIN06 are configured as input signals, they can be routed to the pin buffers of DAI0 in Group D using DAI0_CRS_PIN03/DAI0_CRS_PIN05 and DAI0_CRS_PIN04/DAI0_CRS_PIN06 signals.

PCGs and Pin Buffers

This cross mode connection (see [Figure 20-8 Cross Connect PCG Example](#)) can be used to connect PCG output signals of one DAI to pin buffers of other DAI. For example, PCG C outputs of DAI 1 can be connected to pin buffers of DAI 0 using PCG0_CRS_CLKC_O of Group D.

ASRC TDM and Data Signal

This cross mode connection is available in Group B (Data Signals). This allows programs to connect the Data and TDM signals of SRC3 and SRC 7 across DAIs so that eight SRCs can be daisy chained for TDM 16 operation. For example SRC3_DAT_OP_O and SRC3_TDM_IP_O of DAI0 are available as SRC3_CRS_DAT_OP_O and SRC3_CRS_IP_O in DAI1 so that DAI0 SRC3 can be connected to DAI1's SRC for daisy chaining.

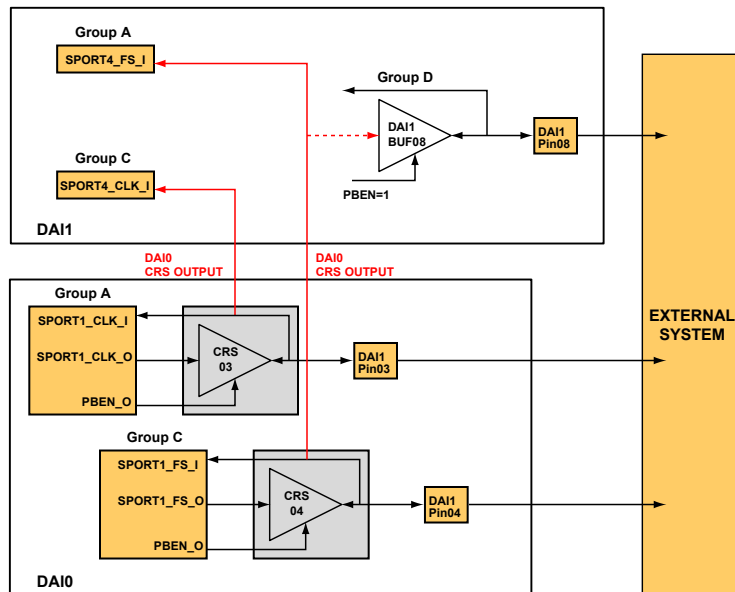


Figure 20-7: Cross Connect Clock Example

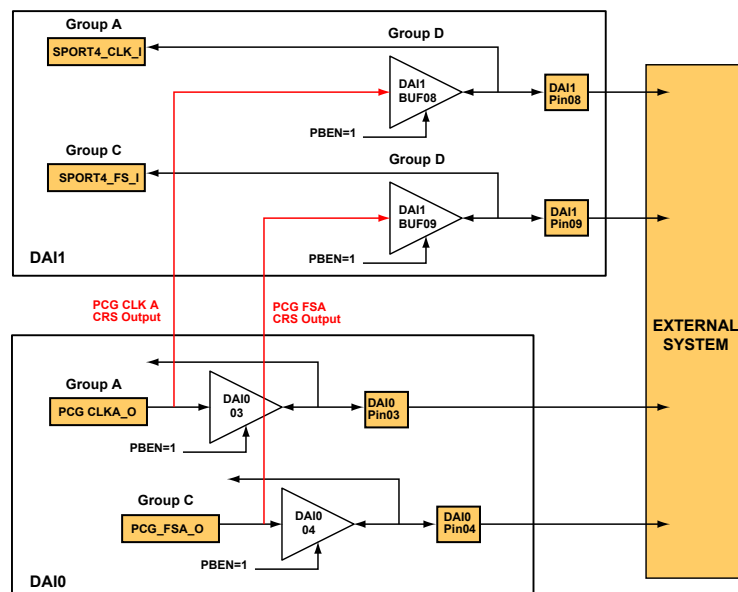


Figure 20-8: Cross Connect PCG Example

Refer to the [DAI Group Routing](#) section for more details on DAI routing.

Signal Routing Matrix by Groups

The SRU is similar to a set of patch bays, which contain a bank of inputs and a bank of outputs. For each input (destination), there is a set of permissible output (source) options. Outputs can feed to any number of inputs in parallel, but every input must be patched to exactly one valid output source. Together, the set of inputs and outputs are called a group. The signal's inputs and outputs that comprise each group all serve similar purposes. They are

compatible such that almost any output-to-input patch makes functional sense. With the grouping, the multiplexing scheme becomes highly efficient since it does not make sense (for example) to route a frame sync signal to a data signal.

The SRU for the DAI contains six groups named A through F. Each group routes a unique set of signals with a specific purpose:

- Group A routes clock signals
- Group B routes serial data signals
- Group C routes frame sync signals
- Group D routes pin signals
- Group E routes miscellaneous signals
- Group F routes pin output enable signals

Together, the six groups of the SRU include all of the inputs and outputs of the DAI peripherals, a number of additional signals from the core, and all of the connections to the DAI pins.

NOTE: It is not possible to connect a signal in one group directly to a signal in a different group (analogous to wiring from one patch bay to another). However, group D (DAI) is largely devoted to routing in this vein.

DAI Group Routing

Each group has a unique encoding for its associated output signals and a set of configuration registers. For example, DAI group A is used to route clock signals. The memory-mapped group A registers, `DAI_CLK0` through `DAI_CLK5`, contain bit fields corresponding to the clock inputs of various peripherals. The values written to these bit fields specify a signal source that is an output from another peripheral. All of the possible encodings represent sources that are clock signals (or at least could be clock signals in some systems). The *Example DAI Group A Multiplexing (DAI_CLKx)* diagrams the input signals that are controlled by the group A registers. All bit fields in the SRU configuration registers correspond to inputs. The value written to the bit field specifies the signal source. This value is also an output from some other component within the SRU.

The SRU is similar to a set of patch bays. Each bay routes a distinct set of outputs to compatible inputs. These connections are implemented as a set of memory-mapped registers with a bit field for each input. The outputs are implemented as a set of bit encodings. Conceptually, a patch cord is used to connect an output to an input. In the SRU, a bit pattern that is associated with a signal output (shown in the *Example DAI Group A Multiplexing (DAI_CLKx)* figure) is written to a bit field corresponding to a signal input.

The same encoding can be written to any number of bit fields in the same group. It is not possible to run out of patch points for an output signal.

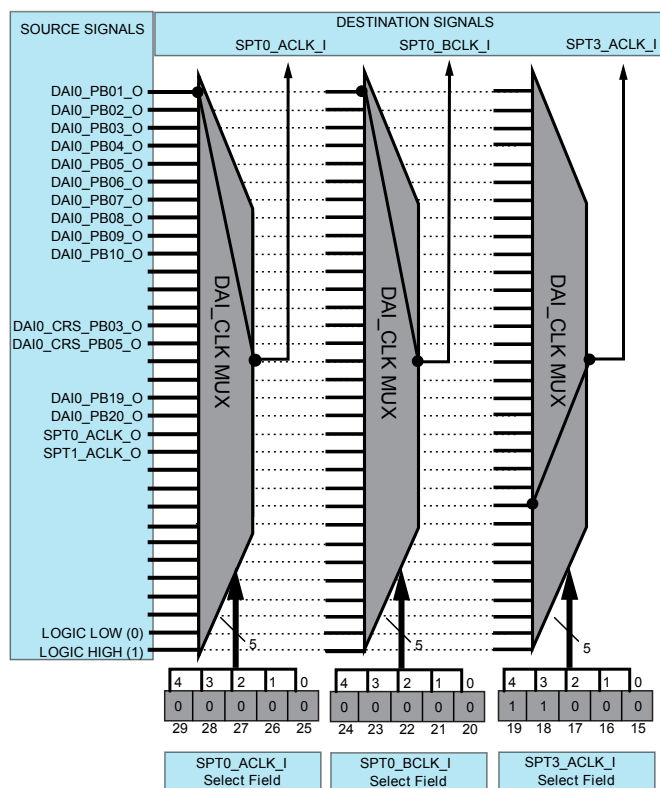


Figure 20-9: Example DAI Group A Multiplexing (DAI_CLKx)

Just as group A routes clock signals, each of the other groups route a collection of compatible signals. Group B routes serial data streams while group C routes frame sync signals. All of the groups have an encoding that allow a signal to flow from a pin output to the input being specified by the bit field.

Group D routes signals to pins so that they may be driven off-chip (required to route a signal to the pin input). Group F routes signals to the pin enables, and the value of these signals determines if a DAI pin is used as an output or an input. The input of one pin can be patched to the output of another pin, allowing board-level routing under software control.

Rules for SRU Connections

There are two rules which apply to all routing:

1. One source (output node) can drive different destinations (input nodes).
2. One destination (input node) can only be assigned to one source (output node).

As an example from the *Example DAI Group A Multiplexing (DAI_CLKx)* figure:

- DAI0_PB02_O is routed to SPT0_ACLK_I
- DAI0_PB02_O is routed to SPT0_BCLK_I

NOTE: Inputs may only be connected to outputs.

Miscellaneous Buffers and Functions

The SRU group E provides miscellaneous buffers used for group interconnect.

DAI group E connections are slightly different from the others in that the inputs and outputs being routed vary considerably in function. This group routes control signals and provides a means of connecting signals between groups.

In the *DAI MISCAx SRU Signal Connections* table, the `DAIn_MISCAx_I` signals appear as inputs in group E (also connected to the DAI interrupt logic), but do not directly feed any peripheral. Rather, the `MISCAx_O` signals reappear as outputs in group F.

Table 20-5: DAI MISCAx SRU Signal Connections

MISCA Source	DAI Connection	MISCA Destination
	Group E	DAIn_MISCA5-0_I
DAIn_MISCA5-0_O	Group F	

Additional connections among groups provide a great amount of utility. Since the output groups F (DAI) dictate pin direction, these few signal paths enable a number of possible uses and connections for the DAI pins. Other examples include:

- A pin input can be patched to another pin's enable, allowing an off-chip signal to gate an output from the processor.
- Any of the DAI pins can be used as interrupt sources or general-purpose I/O (GPIO) signals.

In summary, the SRU enables many possible functional changes, both within the processor as well as externally. Used creatively, it allows system designers to radically change functionality at run time, and to potentially reuse circuit boards across many products.

DAI Routing Capabilities

This section describes the routing options to aid in designing a system using the DAI units. The [DAI Default Routing](#) section provides diagrams that show how that DAIs connect at default. The DAI Group tables provide the source signals and selections codes. Finally, the provides information about configuring destinations.

The *DAI Routing Capabilities* tables provide an overview of the different routing capabilities for the DAI unit. For information on an individual peripherals routing, see the “SRU Programming” section of the specific peripheral chapter.

Table 20-6: DAI0 Routing Capabilities

Source Signals - Output (xxxx_O)		DAI0 Group	Destination Signals - Input (xxxx_I)
SPORT2B-0A (clocks) PCG A, B S/PDIF0 Rx (clock, TDM clock)	DAI0 pin buffer 10-1 and 20-19 DAI0 CRS pin buffer (buffer 3 and 5 from other DAI) Logic level high Logic level low	A-clocks	SPORT3B-0A (clocks) SRC3-0 (clocks) PCG A-B ext. clock, ext. sync S/PDIF0 Tx (clock, HF clock)
SPORT3B-0A (data) SRC3-0 (data, TDM data) S/PDIF0 Tx/Rx (data) SRC7 from other DAI (data, TDM data)	DAI0 pin buffer 10-1 and 20-19 Logic level high Logic level low	B-data	SPORT3B-0A (data) SRC3-0 (data, TDM data) S/PDIF0 Tx/Rx (data)
SPORT2B-0A (FS) PCG A, B (FS) S/PDIF0 Rx (FS)	DAI0 pin buffer 10-1 and 20-19 DAI0 CRS pin buffer (buffer 4 and 6 from other DAI) Logic level high Logic level low	C-frame sync	SPORT3B-0A (FS) SRC3-0 (FS)
SPORT3B-0A (clock, FS, TDV, data) PCG A,B (clock, FS) S/PDIF0 Rx (clock, TDM clock, FS, data) S/PDIF0 Tx (data, block start) PCG C, D from the other DAI (clock, FS) SRC3-0	DAI0 pin buffer 10-1 and 20-19 DAI0 pin buffer 3, 4, 5, and 6 from other DAI Logic level high Logic level low	D-pin buffer inputs	DAI0 pin buffer 10-1 and 20-19 Options: DAI0 pin buffer 20-19 Polarity change
SPORT2B-0A (FS) PCG A (clock) PCG B (clock, FS) S/PDIF0 Tx (block start)	DAI0 pin buffer 10-1 and 20-19 Logic level high Logic level low	E-miscellaneous signals	<i>DAI Interrupt 31-22</i> MISCA5-0 Options: MISCA5-4 Polarity change
SPORT3B-0A (clock, FS, data, TDV) MISCA5-0	DAI0 pin buffer 10-1 and 20-19 Logic level high Logic level low	F-pin buffer enable	DAI0 pin buffer enable 10-1 and 20-19

Table 20-7: DAI1 Routing Capabilities

Source Signals - Output (xxxx_O)		DAI1 Group	Destination Signals - Input (xxxx_I)
SPORT6B–4A (clocks) PCG C, D (clocks) S/PDIF1 Rx (clock, TDM clock)	DAI1 Pin Buffer 10-1 and 20–19 DAI1 CRS pin buffer (buffer 3 and 5 from other DAI) Logic level high Logic level low	A-Clocks	SPORT7B–4A (clocks) SRC4–7 (clocks) PCG C–D Ext. clock, Ext. sync S/PDIF1 Tx (clock, HF clock)
SPORT7B–4A (data) SRC4–7 (data, TDM data) S/PDIF1 Tx/Rx (data) SRC3 from other DAI (data, TDM data)	DAI1 Pin Buffer 10-1 and 20–19 Logic level high Logic level low	B-Data	SPORT7B–4A (data) SRC4–7 (data, TDM data) S/PDIF1 Tx/Rx (data)
SPORT6B–4A (FS) PCG C, D (FS) S/PDIF1 Rx (FS)	DAI1 Pin Buffer 10-1 and 20–19 DAI1 CRS pin buffer (buffer 4 and 6 from other DAI) Logic level high Logic level low	C-Frame Sync	SPORT7B–4A (FS) SRC4–7 (FS)
SPORT7B–4A (clock, FS, TDV, data) PCG C, D (clock, FS) S/PDIF1 Rx (clock, TDM clock, FS, data) S/PDIF1 Tx (data, block start) PCG A, B from the other DAI (clock, FS) SRC4–7	DAI1 Pin Buffer 10-1 and 20–19 DAI1 Pin Buffer 3, 4, 5, and 6 from other DAI Logic level high Logic level low	D-Pin Buffer Inputs	DAI1 Pin Buffer 10-1 and 20–19 Options: DAI1 Pin Buffer 20–19 Polarity Change
SPORT6B–4A (FS) PCG C (clock) PCG D (clock, FS) S/PDIF1 Tx (block start)	DAI1 Pin Buffer 10-1 and 20–19 Logic level high Logic level low	E-Miscellaneous Signals	<i>DAI Interrupt 31-22</i> MISCA5–0 Options: MISCA5–4 Polarity Change
SPORT7B–4A (clock, FS, data, TDV) MISCA5-0	Logic level high Logic level low	F-Pin Buffer Enable	DAI1 Pin Buffer Enable 10-1 and 20–19

DAI Default Routing

When the processor comes out of reset, the SPORT junctions are bidirectional to the DAI pin buffers. This configuration allows systems to use the SPORTs as either requester or completer (without changing the routing scheme). Therefore, programs only need to use the SPORT control register settings to configure requester or completer operations. Note that all DAI inputs which are not routed by default are tied to signal low.

NOTE: All DAI input buffers which are not routed by default are driven low and all DAI pin buffer enable signals are driven low.

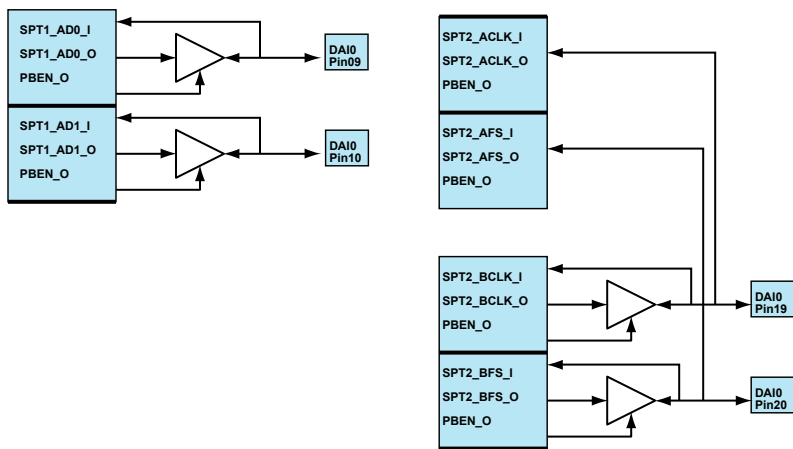


Figure 20-10: DAI0 Default Routing Pins 09-20

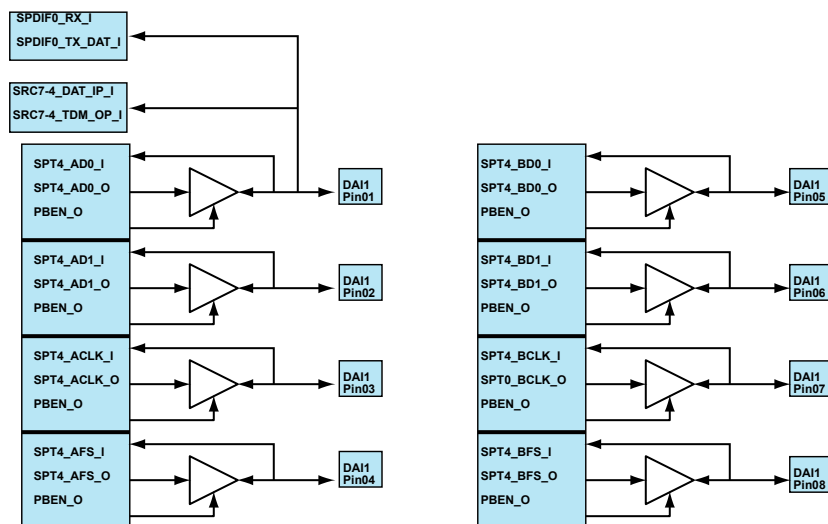


Figure 20-11: DAI1 Default Routing Pins 01-08

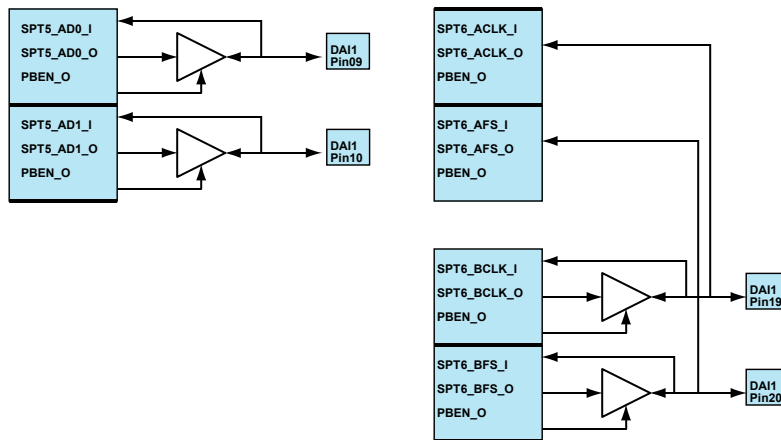


Figure 20-12: DAI1 Default Routing Pins 09-20

Unused DAI Connections

The SRUs have a default general-purpose routing scheme which can be modified to accommodate any number of different system designs. Regardless of the system design, it is good practice to tie all unused inputs to a high or low level to reduce dynamic power consumption.

DAI Operating Modes

Some buffers allow polarity changes, described as follows.

DAI Pin Buffer Polarity

As shown in the *Pin Buffer Polarity* figure, the DAI pin buffer 20-19 can change the polarity of the input signal if the corresponding control bits (DAI_PIN4 . INV20, DAI_PIN4 . INV19) are set. These bits can be set during run time and the buffer should not loopback to itself.

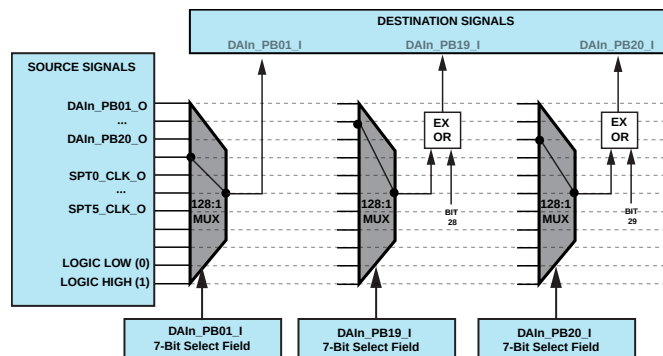


Figure 20-13: Pin Buffer Polarity

DAI Miscellaneous Buffer Polarity

As shown in the *Pin Buffer Polarity* figure, the A5-4 miscellaneous buffers can change the polarity of the input signal if the corresponding control bits (DAI_MISC1.IN5, DAI_MISC1.IN4) are set. Both buffers are not connected to the DAI interrupt latch register. These bits can be set during run time.

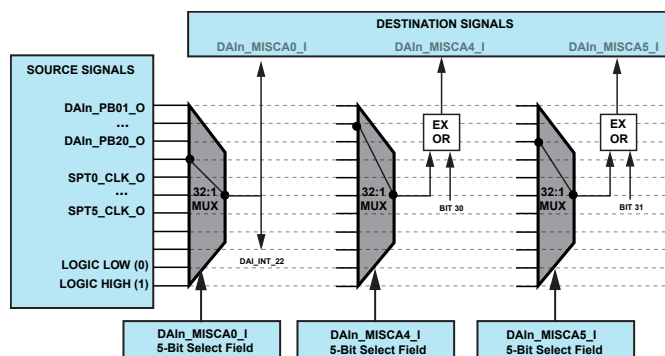


Figure 20-14: Miscellaneous Buffer Polarity

DAI System Interrupt Controller (SIC)

The DAI module incorporates a system interrupt controller (SIC) which is connected to the SEC as seen in the *DAI System Interrupt Controller* figure.

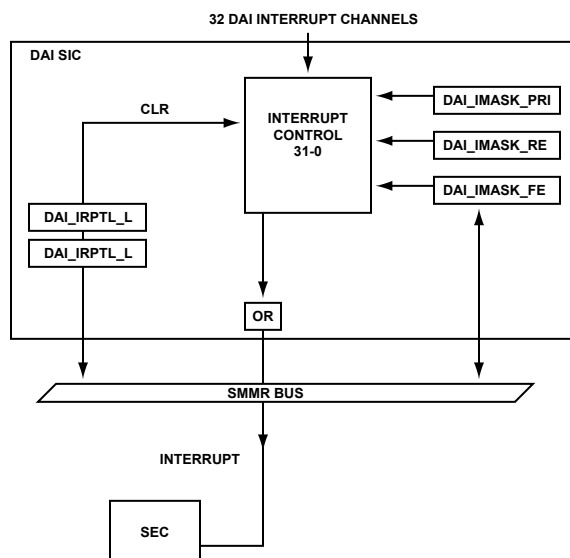


Figure 20-15: DAI System Interrupt Controller

The DAI has its own system interrupt controllers that indicate to the core when DAI audio peripheral-related events have occurred. Since audio events occur infrequently relative to the SHARC core, the DAI interrupt controller reduces all of its interrupts onto two interrupt signals within the core's primary interrupt systems. One interrupt is mapped with DAI low priority. The second interrupt is mapped with DAI high priority. This configuration allows programs to broadly indicate priority. In this way, the DAI SIC provides 32 independently configurable sources or

channels. The output bus interrupt signals are logically OR'ed into one interrupt line and fed to the interrupt controller logic of the core.

Three registers are used to configure the DAI interrupt controller. Each of the 32 interrupt sources can be independently configured to trigger on a rising edge, falling edge, both edges, or neither edge of an incoming signal. All DAI interrupt control registers are memory-mapped registers and are accessed through the peripheral bus.

Interrupt Sources

The DAI's five peripheral sources are multiplexed into 32 interrupt sources and are labeled DAI_INT31-0 (*DAI Interrupt Sources* table).

NOTE: There are two naming conventions. The DAI interrupt controller register bits are labeled DAI_31-0_INT. Their corresponding SRU routing signals are labeled DAI_INT_31-0_I.

Table 20-8: DAI Interrupt Sources

Interrupt Source	Description	Signal Response
DAI_INT2-0, DAI_INT4	S/PDIF RX, 4 channels	Event
DAI_INT2-0, DAI_INT4	S/PDIF RX, 4 channels	Waveform
DAI_INT21-18	ASRC, 4 channels	
DAI_INT31-22	Miscellaneous, S/PDIF TX, 9 channels	

Interrupt Latch Priority Option

The [DAI_IMSK_PRI](#) register specifies the priority for the DAI interrupt channels. DAI system interrupt controller has a pair of interrupt latch registers, [DAI_IRPTL_H](#) and [DAI_IRPTL_L](#). The configuration of the [DAI_IMSK_PRI](#) register also determines the interrupt latch mapping for a particular DAI interrupt. When a DAI interrupt is configured as low priority ([DAI_IMSK_PRI](#) bit cleared, default setting), interrupts are mapped to the DAI_INTR_IRQL signal and when an interrupt occurs, the corresponding bit of the [DAI_IRPTL_L](#) register is set. When a DAI interrupt is configured as high priority ([DAI_IMSK_PRI](#) bit set), interrupts are mapped to the DAI_INTR_IRQH signal and the interrupt is latched to the [DAI_IRPTL_H](#) register. The low priority DAI interrupt (INTR_DAI_IRQL) and high priority DAI interrupt (INTR_DAI_IRQH) are connected to the SEC .

Interrupt Mask for Waveforms

The [DAI_IMSK_RE](#) and [DAI_IMSK_FE](#) registers allow programs to mask or unmask interrupts for specific edges of a signal mapped to the channel. It can be configured for rising edges, falling edges, both rising and falling edges, or neither rising nor falling edges by masking them separately. Signals from the SRU can be used to generate interrupts. For example, when the [DAI_IMSK_FE.MISCINT9](#) bit is set to one, any falling edge signals from the external channel generate an interrupt and the interrupt latch is set.

Interrupt Mask for Events

The system interrupt controller needs information about a peripheral's interrupt sources that correspond to event signals (see the *DAI Interrupt Sources* table). As a result, the rising edge is used as an interrupt source only. For DAI peripherals marked as events, programs may unmask an interrupt source on the rising edge only.

Shadow Interrupt Register

The DAI interrupt controller has shadow registers to simplify debug activities since these registers do not manipulate status control. Any read of the shadow registers (`DAI_IRPTL_HS`, `DAI_IRPTL_LS`), provides the same data as a read of the `DAI_IRPTL_H` and `DAI_IRPTL_L` registers. However, reads of the DAI shadow registers do not change the interrupt acknowledge status to the core interrupt controller.

Interrupt Service

The interrupt acknowledge operates differently when multiple channels are multiplexed into one interrupt output signal. When an interrupt from the DAI must be serviced, any of the two interrupt service routines (`INTR_DAI_IRQL`, `INTR_DAI_IRQH`) must query the SIC to determine the source(s). Sources can be any one or more of the DAI channels (`DAI_INT31-0`).

- When the `DAI_IRPTL_H` register is read, the high priority latched interrupts are cleared.
- When the `DAI_IRPTL_L` register is read, the low priority latched interrupts are cleared.

If an interrupt occurs in the same cycle as a latch register is cleared, the clear mechanism has lower priority and the new interrupt is registered.

Signal Routing Unit Effect Latency

After the DAI registers are configured the effect latency is 2 *SCLK0* cycles minimum and 3 *SCLK0* cycles maximum.

DAI Programming Model

As discussed in the previous sections, the signal routing unit is controlled by writing values that provide a plug-in tool in CCES so that configuring the SRU is done graphically. Analog Devices offers macros that are included with the CrossCore or VisualDSP++ tools, greatly easing code development in the SRU.

There is a macro that has been created to connect peripherals used in a DAI configuration. This code can be used in both assembly and C code. See the INCLUDE file `SRU.H`. In practice the macro is provided and forms the style `SRU(source_O, destination_I)` for DAI0. Example appears as: `SRU(DAI0_PB12_O, SPT0_ACLK_I);`

Debug Features

The following section describes the feature that can be used to help in debugging the DAI.

Loopback Routing

The SPORT serial peripheral supports an internal loopback mode. If the loopback bit for each peripheral is enabled, it connects the transmitter with the receiver block internally (does not signal off-chip). The SRU can be used for this purpose. The *Loopback Routing* table describes the different possible routings based on the peripheral.

NOTE: The peripheral's loopback mode for debug is independent from both signal routing units.

Table 20-9: Loopback Routing

Peripheral	Loopback Mode	SRU1-0 Internal Routing for Loopback	SRU1-0 External Routing for Loopback
SPORT	Yes	SPTx_xx_O → SPTx_xx_I	SPTx_xx_O → DAI _n PB _{xx} _I DAI _n PB _{xx} _O → SPTx_xx_I
S/PDIF Tx/Rx	No	SPDIF _n _TX_O → SPDIF _n _RX_I	SPDIF _n _TX_O → DAI _n PB _{xx} _I DAI _n PB _{xx} _O → SPDIF _n _RX_I
SRC	No	SRCx_DAT_OP_O → SRCx_DAT_IP_I	SRCx_DAT_OP_O → DAI _n PB _{xx} _I DAI _n PB _{xx} _O → SRCx_DAT_IP_I

DAI Sources Overview

The following tables provide information of the various DAI sources sorted by group.

DAI0 Group A – Clock Routing Source Signals

The following table lists Group A clock sources.

Table 20-10: DAI0 Group A – Clock Routing

Selection Code	Source Signal	Description (Source Selection)
00000 (0x0)	DAI0_PB01_O	Pin Buffer 1
00001 (0x1)	DAI0_PB02_O	Pin Buffer 2
00010 (0x2)	DAI0_PB03_O	Pin Buffer 3
00011 (0x3)	DAI0_PB04_O	Pin Buffer 4
00100 (0x4)	DAI0_PB05_O	Pin Buffer 5
00101 (0x5)	DAI0_PB06_O	Pin Buffer 6
00110 (0x6)	DAI0_PB07_O	Pin Buffer 7
00111 (0x7)	DAI0_PB08_O	Pin Buffer 8
01000 (0x8)	DAI0_PB09_O	Pin Buffer 9
01001 (0x9)	DAI0_PB10_O	Pin Buffer 10
01100 (0xC)	Reserved	
01101 (0xD)	Reserved	
01110 (0xE)	DAI0_CRS_PB03_O	Pin Buffer 3 (DAI1)
01111 (0xF)	DAI0_CRS_PB05_O	Pin Buffer 5 (DAI1)
10000 (0x10)	Reserved	
10001 (0x11)	Reserved	
10010 (0x12)	DAI0_PB19_O	Pin Buffer 19
10011 (0x13)	DAI0_PB20_O	Pin Buffer 20
10100 (0x14)	SPT0_ACLK_O	SPORT 0 Clock A
10101 (0x15)	SPT0_BCLK_O	SPORT 0 Clock B
10110 (0x16)	SPT1_ACLK_O	SPORT 1 Clock A
10111 (0x17)	SPT1_BCLK_O	SPORT 1 Clock B
11000 (0x18)	SPT2_ACLK_O	SPORT 2 Clock A
11001 (0x19)	SPT2_BCLK_O	SPORT 2 Clock B
11010 (0x1A)	SPDIF0_RX_CLK_O	SPDIF 0 Receive Clock Output
11011 (0x1B)	SPDIF0_RX_TDMCLK_O	SPDIF 0 Receive TDM Clock Output
11100 (0x1C)	PCG0_CLKA_O	Precision Clock A Output
11101 (0x1D)	PCG0_CLKB_O	Precision Clock B Output

Table 20-10: DAI0 Group A – Clock Routing (Continued)

Selection Code	Source Signal	Description (Source Selection)
11110 (0x1E)	LOW	Logic Level Low (0)
11111 (0x1F)	HIGH	Logic Level High (1)

DAI0 Group B – Serial Data Source Signals

The group B data sources are listed in the following table. The group B data destinations are configured using [DAI_PBEN0](#) through [DAI_PBEN3](#).

Table 20-11: DAI0 Group B – Serial Data Signals

Selection Code	Source Signal	Description (Source Selection)
000000 (0x0)	DAI0_PB01_O	Pin Buffer 1
000001 (0x1)	DAI0_PB02_O	Pin Buffer 2
000010 (0x2)	DAI0_PB03_O	Pin Buffer 3
000011 (0x3)	DAI0_PB04_O	Pin Buffer 4
000100 (0x4)	DAI0_PB05_O	Pin Buffer 5
000101 (0x5)	DAI0_PB06_O	Pin Buffer 6
000110 (0x6)	DAI0_PB07_O	Pin Buffer 7
000111 (0x7)	DAI0_PB08_O	Pin Buffer 8
001000 (0x8)	DAI0_PB09_O	Pin Buffer 9
001001 (0x9)	DAI0_PB10_O	Pin Buffer 10
001100 (0xC)	Reserved	
001101 (0xD)	Reserved	
001110 (0xE)	Reserved	
001111 (0xF)	Reserved	
010000 (0x10)	Reserved	
010001 (0x11)	Reserved	
010010 (0x12)	DAI0_PB19_O	Pin Buffer 19
010011 (0x13)	DAI0_PB20_O	Pin Buffer 20
010100 (0x14)	SPT0_AD0_O	SPORT 0 Data AD0
010101 (0x15)	SPT0_AD1_O	SPORT 0 Data AD1
010110 (0x16)	SPT0_BD0_O	SPORT 0 Data BD0
010111 (0x17)	SPT0_BD1_O	SPORT 0 Data BD1
011000 (0x18)	SPT1_AD0_O	SPORT 1 Data AD0
011001 (0x19)	SPT1_AD1_O	SPORT 1 Data AD1
011010 (0x1A)	SPT1_BD0_O	SPORT 1 Data BD0
011011 (0x1B)	SPT1_BD1_O	SPORT 1 Data BD1
011100 (0x1C)	SPT2_AD0_O	SPORT 2 Data AD0
011101 (0x1D)	SPT2_AD1_O	SPORT 2 Data AD1

Table 20-11: DAI0 Group B – Serial Data Signals (Continued)

Selection Code	Source Signal	Description (Source Selection)
011110 (0x1E)	SPT2_BD0_O	SPORT 2 Data BD0
011111 (0x1F)	SPT2_BD1_O	SPORT 2 Data BD1
100000 (0x20)	SRC0_DAT_OP_O	SRC0 Data Out
100001 (0x21)	SRC1_DAT_OP_O	SRC1 Data Out
100010 (0x22)	SRC2_DAT_OP_O	SRC2 Data Out
100011 (0x23)	SRC3_DAT_OP_O	SRC3 Data Out
100100 (0x24)	SRC0_TDM_IP_O	SRC0 Data Out
100101 (0x25)	SRC1_TDM_IP_O	SRC1 Data Out
100110 (0x26)	SRC2_TDM_IP_O	SRC2 Data Out
100111 (0x27)	SRC3_TDM_IP_O	SRC3 Data Out
101000 (0x28)	SPDIF0_RX_DAT_O	SPDIF 0 RX Serial Data Out
101100 (0x2C)	SPT3_AD0_O	SPORT 3 Data AD0
101101 (0x2D)	SPT3_AD1_O	SPORT 3 Data AD1
101110 (0x2E)	SPT3_BD0_O	SPORT 3 Data BD0
101111 (0x2F)	SPT3_BD1_O	SPORT 3 Data BD1
110000 (0x30)	SPDIF0_TX_O	SPDIF 0 TX Biphase Stream
110001 (0x31)	SRC7_CRS_DAT_OP_O	SRC7 data out (DAI1)
110010 (0x32)	SRC7_CRS_TDM_IP_O	SRC7 data out (DAI1)
110011 (0x33)–111101 (0x3D)	Reserved	
111110 (0x3E)	LOW	Logic Level Low (0)
111111 (0x3F)	HIGH	Logic Level High (1)

DAI0 Group C – Frame Sync Source Signals

The group C frame sync signal sources are listed in the following table.

Table 20-12: DAI0 Group C – Frame Sync Signals

Selection Code	Source Signal	Description (Source Selection)
00000 (0x0)	DAI0_PB01_O	Pin Buffer 1
00001 (0x1)	DAI0_PB02_O	Pin Buffer 2
00010 (0x2)	DAI0_PB03_O	Pin Buffer 3
00011 (0x3)	DAI0_PB04_O	Pin Buffer 4
00100 (0x4)	DAI0_PB05_O	Pin Buffer 5

Table 20-12: DAI0 Group C – Frame Sync Signals (Continued)

Selection Code	Source Signal	Description (Source Selection)
00101 (0x5)	DAI0_PB06_O	Pin Buffer 6
00110 (0x6)	DAI0_PB07_O	Pin Buffer 7
00111 (0x7)	DAI0_PB08_O	Pin Buffer 8
01000 (0x8)	DAI0_PB09_O	Pin Buffer 9
01001 (0x9)	DAI0_PB10_O	Pin Buffer 10
01100 (0xC)	Reserved	
01101 (0xD)	Reserved	
01110 (0xE)	DAI0_CRS_PB04_O	Pin Buffer 4 (DAI1)
01111 (0xF)	DAI0_CRS_PB06_O	Pin Buffer 6 (DAI1)
10000 (0x10)	Reserved	
10001 (0x11)	Reserved	
10010 (0x12)	DAI0_PB19_O	Pin Buffer 19
10011 (0x13)	DAI0_PB20_O	Pin Buffer 20
10100 (0x14)	SPT0_AFS_O	SPORT 0 Frame Sync A
10101 (0x15)	SPT0_BFS_O	SPORT 0 Frame Sync B
10110 (0x16)	SPT1_AFS_O	SPORT 1 Frame Sync A
10111 (0x17)	SPT1_BFS_O	SPORT 1 Frame Sync B
11000 (0x18)	SPT2_AFS_O	SPORT 2 Frame Sync A
11001 (0x19)	SPT2_BFS_O	SPORT 2 Frame Sync B
11010 (0x1A)	SPDIF0_FS_O	SPDIF 0 RX Frame Sync Output
11011 (0x1B)	Reserved	
11100 (0x1C)	PCG0_FSA_O	Precision Frame Sync A Output
11101 (0x1D)	PCG0_FSB_O	Precision Frame Sync B Output
11110 (0x1E)	LOW	Logic Level Low (0)
11111 (0x1F)	HIGH	Logic Level High (1)

DAI0 Group D – Pin Signal Assignment Source Signals

The group D pin signal assignment sources are listed in the following table.

Table 20-13: DAI0 Group D – Pin Signal Assignments

Selection Code	Source Signal	Description (Source Selection)
0000000 (0x0)	DAI0_PB01_O	Pin Buffer 1

Table 20-13: DAI0 Group D – Pin Signal Assignments (Continued)

Selection Code	Source Signal	Description (Source Selection)
0000001 (0x1)	DAI0_PB02_O	Pin Buffer 2
0000010 (0x2)	DAI0_PB03_O	Pin Buffer 3
0000011 (0x3)	DAI0_PB04_O	Pin Buffer 4
0000100 (0x4)	DAI0_PB05_O	Pin Buffer 5
0000101 (0x5)	DAI0_PB06_O	Pin Buffer 6
0000110 (0x6)	DAI0_PB07_O	Pin Buffer 7
0000111 (0x7)	DAI0_PB08_O	Pin Buffer 8
0001000 (0x8)	DAI0_PB09_O	Pin Buffer 9
0001001 (0x9)	DAI0_PB10_O	Pin Buffer 10
0001100 (0xC)	Reserved	
0001101 (0xD)	Reserved	
0001110 (0xE)	Reserved	
0001111 (0xF)	Reserved	
0010000 (0x10)	Reserved	
0010001 (0x11)	Reserved	
0010010 (0x12)	DAI0_PB19_O	Pin Buffer 19
0010011 (0x13)	DAI0_PB20_O	Pin Buffer 20
0010100 (0x14)	SPT0_AD0_O	SPORT 0 Data AD0
0010101 (0x15)	SPT0_AD1_O	SPORT 0 Data AD1
0010110 (0x16)	SPT0_BD0_O	SPORT 0 Data BD0
0010111 (0x17)	SPT0_BD1_O	SPORT 0 Data BD1
0011000 (0x18)	SPT1_AD0_O	SPORT 1 Data AD0
0011001 (0x19)	SPT1_AD1_O	SPORT 1 Data AD1
0011010 (0x1A)	SPT1_BD0_O	SPORT 1 Data BD0
0011011 (0x1B)	SPT1_BD1_O	SPORT 1 Data BD1
0011100 (0x1C)	SPT2_AD0_O	SPORT 2 Data AD0
0011101 (0x1D)	SPT2_AD1_O	SPORT 2 Data AD1
0011110 (0x1E)	SPT2_BD0_O	SPORT 2 Data BD0
0011111 (0x1F)	SPT2_BD1_O	SPORT 2 Data BD1
0100000 (0x20)	SPT0_ACLK_O	SPORT 0 Clock A
0100001 (0x21)	SPT0_BCLK_O	SPORT 0 Clock B

Table 20-13: DAI0 Group D – Pin Signal Assignments (Continued)

Selection Code	Source Signal	Description (Source Selection)
0100010 (0x22)	SPT1_ACLK_O	SPORT 1 Clock A
0100011 (0x23)	SPT1_BCLK_O	SPORT 1 Clock B
0100100 (0x24)	SPT2_ACLK_O	SPORT 2 Clock A
0100101 (0x25)	SPT2_BCLK_O	SPORT 2 Clock B
0100110 (0x26)	SPT0_AFS_O	SPORT 0 Frame Sync A
0100111 (0x27)	SPT0_BFS_O	SPORT 0 Frame Sync B
0101000 (0x28)	SPT1_AFS_O	SPORT 1 Frame Sync A
0101001 (0x29)	SPT1_BFS_O	SPORT 1 Frame Sync B
0101010 (0x2A)	SPT2_AFS_O	SPORT 2 Frame Sync A
0101011 (0x2B)	SPT2_BFS_O	SPORT 2 Frame Sync B
0101100 (0x2C)	SPT3_AD0_O	SPORT 3 Data AD0
0101101 (0x2D)	SPT3_AD1_O	SPORT 3 Data AD1
0101110 (0x2E)	SPT3_BD0_O	SPORT 3 Data BD0
0101111 (0x2F)	SPT3_BD1_O	SPORT 3 Data BD1
0110000 (0x30)	MLB0_CLKOUT	MLB PLL clock output
0110001 (0x31)	SPDIF0_TX_BLKSTART_O	SPDIF 0 TX Block Start Output
0110100 (0x34)	SPT3_ACLK_O	SPORT 3 Clock A
0110101 (0x35)	SPT3_BCLK_O	SPORT 3 Clock B
0110110 (0x36)	SPT3_AFS_O	SPORT 3 Frame Sync A
0110111 (0x37)	SPT3_BFS_O	SPORT 3 Frame Sync B
0111000 (0x38)	PCG0_CLKA_O	Precision Clock A
0111001 (0x39)	PCG0_CLKB_O	Precision Clock B
0111010 (0x3A)	PCG0_FSA_O	Precision Frame Sync A
0111011 (0x3B)	PCG0_FSB_O	Precision Frame Sync B
0111100 (0x3C)	Reserved	
0111101 (0x3D)	SRC0_DAT_OP_O	SRC0 Data Output
0111110 (0x3E)	SRC1_DAT_OP_O	SRC1 Data Output
0111111 (0x3F)	SRC2_DAT_OP_O	SRC2 Data Output
1000000 (0x40)	SRC3_DAT_OP_O	SRC3 Data Output
1000001 (0x41)	SPDIF0_RX_DAT_O	SPDIF 0 RX Data Output
1000010 (0x42)	SPDIF0_FS_O	SPDIF 0 RX Frame Sync Output

Table 20-13: DAI0 Group D – Pin Signal Assignments (Continued)

Selection Code	Source Signal	Description (Source Selection)
1000011 (0x43)	SPDIF0_RXCLK_O	SPDIF 0 RX Clock Output
1000100 (0x44)	SPDIF0_RX_TDMCLK_O	SPDIF 0 RX TDM Clock Output
1000101 (0x45)	SPDIF0_TX_O	SPDIF 0 TX Biphase Encoded Data Output
1000110 (0x46)	SPT0_ATDV_O	SPORT0 Transmit A Data Valid Output
1000111 (0x47)	SPT0_BTDTV_O	SPORT0 Transmit B Data Valid Output
1001000 (0x48)	SPT1_ATDV_O	SPORT1 Transmit A Data Valid Output
1001001 (0x49)	SPT1_BTDTV_O	SPORT1 Transmit B Data Valid Output
1001010 (0x4A)	SPT2_ATDV_O	SPORT2 Transmit A Data Valid Output
1001011 (0x4B)	SPT2_BTDTV_O	SPORT2 Transmit B Data Valid Output
1001100 (0x4C)	SPT3_ATDV_O	SPORT3 Transmit A Data Valid Output
1001101 (0x4D)	SPT3_BTDTV_O	SPORT3 Transmit B Data Valid Output
1001110 (0x4E)	DAI0_CRS_PB05_O	Pin Buffer 5 (DAI1)
1001111 (0x4F)	DAI0_CRS_PB06_O	Pin Buffer 6 (DAI1)
1010000 (0x50)	PCG0_CRS_CLKC_O	Precision Clock C (DAI1)
1010001 (0x51)	PCG0_CRS_CLKD_O	Precision Clock D (DAI1)
1010010 (0x52)	PCG0_CRS_FSC_O	Precision Frame Sync C (DAI1)
1010011 (0x53)	PCG0_CRS_FSD_O	Precision Frame Sync D (DAI1)
1010100 (0x54)	DAI0_CRS_PB03_O	Pin Buffer 3 (DAI1)
1010101 (0x55)	DAI0_CRS_PB04_O	Pin Buffer 4 (DAI1)
1010110 – 1111101	Reserved	
1111110 (0x7E)	LOW	Logic Level Low (0)
1111111 (0x7F)	HIGH	Logic Level High (1)

DAI0 Group E – Miscellaneous Source Signals

The following table lists the group E miscellaneous signal sources.

Table 20-14: DAI0 Group E – Miscellaneous Signals

Selection Code	Source Signal	Description (Source Selection)
00000 (0x0)	DAI0_PB01_O	Pin Buffer 1 Output
00001 (0x1)	DAI0_PB02_O	Pin Buffer 2 Output
00010 (0x2)	DAI0_PB03_O	Pin Buffer 3 Output

Table 20-14: DAI0 Group E – Miscellaneous Signals (Continued)

Selection Code	Source Signal	Description (Source Selection)
00011 (0x3)	DAI0_PB04_O	Pin Buffer 4 Output
00100 (0x4)	DAI0_PB05_O	Pin Buffer 5 Output
00101 (0x5)	DAI0_PB06_O	Pin Buffer 6 Output
00110 (0x6)	DAI0_PB07_O	Pin Buffer 7 Output
00111 (0x7)	DAI0_PB08_O	Pin Buffer 8 Output
01000 (0x8)	DAI0_PB09_O	Pin Buffer 9 Output
01001 (0x9)	DAI0_PB10_O	Pin Buffer 10 Output
01100 (0xC)	Reserved	
01101 (0xD)	Reserved	
01110 (0xE)	Reserved	
01111 (0xF)	Reserved	
10000 (0x10)	Reserved	
10001 (0x11)	Reserved	
10010 (0x12)	DAI0_PB19_O	Pin Buffer 19 Output
10011 (0x13)	DAI0_PB20_O	Pin Buffer 20 Output
10100 (0x14)	SPT0_AFS_O	SPORT 0 Frame Sync A
10101 (0x15)	SPT0_BFS_O	SPORT 0 Frame Sync B
10110 (0x16)	SPT1_AFS_O	SPORT 1 Frame Sync A
10111 (0x17)	SPT1_BFS_O	SPORT 1 Frame Sync B
11000 (0x18)	SPT2_AFS_O	SPORT 2 Frame Sync A
11001 (0x19)	SPT2_BFS_O	SPORT 2 Frame Sync B
11010 (0x1A)	SPDIF0_TX_BLKSTART_O	SPDIF 0 TX Block Start Output
11011 (0x1B)	PCG0_FSA_O	Precision Frame Sync A
11100 (0x1C)	PCG0_CLKB_O	Precision Clock B
11101 (0x1D)	PCG0_FSB_O	Precision Frame Sync B
11110 (0x1E)	LOW	Logic Level Low (0) as a Source
11111 (0x1F)	HIGH	Logic Level High (1) as a Source

DAI0 Group F – Pin Output Enable Source Signals

The group F pin output enable signal sources are listed in the following table. The group F destinations are configured using .

Table 20-15: DAI0 Group F – Pin Output Enable

Selection Code	Source Signal	Description (Source Selection)
000000 (0x0)	LOW	Logic Level Low (0)
000001 (0x1)	HIGH	Logic Level High (1)
000010 (0x2)	DAI0_MISCA0_O	DAI0 Miscellaneous Control A0 Output
000011 (0x3)	DAI0_MISCA1_O	DAI0 Miscellaneous Control A1 Output
000100 (0x4)	DAI0_MISCA2_O	DAI0 Miscellaneous Control A2 Output
000101 (0x5)	DAI0_MISCA3_O	DAI0 Miscellaneous Control A3 Output
000110 (0x6)	DAI0_MISCA4_O	DAI0 Miscellaneous Control A4 Output
000111 (0x7)	DAI0_MISCA5_O	DAI0 Miscellaneous Control A5 Output
001000 (0x8)	SPT0_ACLK_PBEN_O	SPORT 0 Clock A Output Enable
001001 (0x9)	SPT0_AFS_PBEN_O	SPORT 0 Frame Sync A Output Enable
001010 (0xA)	SPT0_AD0_PBEN_O	SPORT 0 Data AD0 Output Enable
001011 (0xB)	SPT0_AD1_PBEN_O	SPORT 0 Data AD1 Output Enable
001100 (0xC)	SPT0_BCLK_PBEN_O	SPORT 0 Clock B Output Enable
001101 (0xD)	SPT0_BFS_PBEN_O	SPORT 0 Frame Sync B Output Enable
001110 (0xE)	SPT0_BD0_PBEN_O	SPORT 0 Data BD0 Output Enable
001111 (0xF)	SPT0_BD1_PBEN_O	SPORT 0 Data BD1 Output Enable
010000 (0x10)	SPT1_ACLK_PBEN_O	SPORT 1 Clock A Output Enable
010001 (0x11)	SPT1_AFS_PBEN_O	SPORT 1 Frame Sync A Output Enable
010010 (0x12)	SPT1_AD0_PBEN_O	SPORT 1 Data AD0 Output Enable
010011 (0x13)	SPT1_AD1_PBEN_O	SPORT 1 Data AD1 Output Enable
010100 (0x14)	SPT1_BCLK_PBEN_O	SPORT 1 Clock B Output Enable
010101 (0x15)	SPT1_BFS_PBEN_O	SPORT 1 Frame Sync B Output Enable
010110 (0x16)	SPT1_BD0_PBEN_O	SPORT 1 Data BD0 Output Enable
010111 (0x17)	SPT1_BD1_PBEN_O	SPORT 1 Data BD1 Output Enable
011000 (0x18)	SPT2_ACLK_PBEN_O	SPORT 2 Clock A Output Enable
011001 (0x19)	SPT2_AFS_PBEN_O	SPORT 2 Frame Sync A Output Enable
011010 (0x1A)	SPT2_AD0_PBEN_O	SPORT 2 Data AD0 Output Enable
011011 (0x1B)	SPT2_AD1_PBEN_O	SPORT 2 Data AD1 Output Enable

Table 20-15: DAI0 Group F – Pin Output Enable (Continued)

Selection Code	Source Signal	Description (Source Selection)
011100 (0x1C)	SPT2_BCLK_PBEN_O	SPORT 2 Clock B Output Enable
011101 (0x1D)	SPT2_BFS_PBEN_O	SPORT 2 Frame Sync B Output Enable
011110 (0x1E)	SPT2_BD0_PBEN_O	SPORT 2 Data BD0 Output Enable
011111 (0x1F)	SPT2_BD1_PBEN_O	SPORT 2 Data BD1 Output Enable
100000 (0x20)	SPT3_ACLK_PBEN_O	SPORT 3 Clock A Output Enable
100001 (0x21)	SPT3_AFS_PBEN_O	SPORT 3 Frame Sync A Output Enable
100010 (0x22)	SPT3_AD0_PBEN_O	SPORT 3 Data AD0 Output Enable
100011 (0x23)	SPT3_AD1_PBEN_O	SPORT 3 Data AD1 Output Enable
100100 (0x24)	SPT3_BCLK_PBEN_O	SPORT 2 Clock B Output Enable
100101 (0x25)	SPT3_BFS_PBEN_O	SPORT 3 Frame Sync B Output Enable
100110 (0x26)	SPT3_BD0_PBEN_O	SPORT 3 Data BD0 Output Enable
100111 (0x27)	SPT3_BD1_PBEN_O	SPORT 3 Data BD1 Output Enable
101000 (0x28)	SPT0_ATDV_PBEN_O	SPORT 0 A Transmit Data Valid Output
101001 (0x29)	SPT0_BTDTV_PBEN_O	SPORT 0 B Transmit Data Valid Output
101010 (0x2A)	SPT1_ATDV_PBEN_O	SPORT 1 A Transmit Data Valid Output
101011 (0x2B)	SPT1_BTDTV_PBEN_O	SPORT 1 B Transmit Data Valid Output
101100 (0x2C)	SPT2_ATDV_PBEN_O	SPORT 2 A Transmit Data Valid Output
101101 (0x2D)	SPT2_BTDTV_PBEN_O	SPORT 2 B Transmit Data Valid Output
100111 (0x2E)	SPT3_ATDV_PBEN_O	SPORT 3 A Transmit Data Valid Output
101110 (0x2F)	SPT3_BTDTV_PBEN_O	SPORT 3 B Transmit Data Valid Output
110000 (0x30)–1111111 (0x3F)	Reserved	

DAI1 Group A – Clock Routing Source Signals

The group A clock routing signal sources are listed in the following table.

Table 20-16: DAI1 Group A – Clock Routing Signals

Selection Code	Source Signal	Description (Source Selection)
00000 (0x0)	DAI1_PB01_O	Pin Buffer 1
00001 (0x1)	DAI1_PB02_O	Pin Buffer 2
00010 (0x2)	DAI1_PB03_O	Pin Buffer 3
00011 (0x3)	DAI1_PB04_O	Pin Buffer 4
00100 (0x4)	DAI1_PB05_O	Pin Buffer 5

Table 20-16: DAI1 Group A – Clock Routing Signals (Continued)

Selection Code	Source Signal	Description (Source Selection)
00101 (0x5)	DAI1_PB06_O	Pin Buffer 6
00110 (0x6)	DAI1_PB07_O	Pin Buffer 7
00111 (0x7)	DAI1_PB08_O	Pin Buffer 8
01000 (0x8)	DAI1_PB09_O	Pin Buffer 9
01001 (0x9)	DAI1_PB10_O	Pin Buffer 10
01100 (0xC)	Reserved	
01101 (0xD)	Reserved	
01110 (0xE)	DAI1_CRS_PB03_O	Pin Buffer 3 (DAI0)
01111 (0xF)	DAI1_CRS_PB05_O	Pin Buffer 5 (DAI0)
10000 (0x10)	Reserved	
10001 (0x11)	Reserved	
10010 (0x12)	DAI1_PB19_O	Pin Buffer 19
10011 (0x13)	DAI1_PB20_O	Pin Buffer 20
10100 (0x14)	SPT4_ACLK_O	SPORT 4 Clock A
10101 (0x15)	SPT4_BCLK_O	SPORT 4 Clock B
10110 (0x16)	SPT5_ACLK_O	SPORT 5 Clock A
10111 (0x17)	SPT5_BCLK_O	SPORT 5 Clock B
11000 (0x18)	SPT6_ACLK_O	SPORT 6 Clock A
11001 (0x19)	SPT6_BCLK_O	SPORT 6 Clock B
11010 (0x1A)	SPDIF1_RX_CLK_O	SPDIF 1 Receive Clock Output
11011 (0x1B)	SPDIF1_RX_TDMCLK_O	SPDIF 1 Receive TDM Clock Output
11100 (0x1C)	PCG0_CLKC_O	Precision Clock C Output
11101 (0x1D)	PCG0_CLKD_O	Precision Clock D Output
11110 (0x1E)	LOW	Logic Level Low (0)
11111 (0x1F)	HIGH	Logic Level High (1)

DAI1 Group B – Serial Data Source Signals

The group B data sources are listed in the following table.

Table 20-17: DAI1 Group B – Serial Data Signals

Selection Code	Source Signal	Description (Source Selection)
000000 (0x0)	DAI1_PB01_O	Pin Buffer 1

Table 20-17: DAI1 Group B – Serial Data Signals (Continued)

Selection Code	Source Signal	Description (Source Selection)
000001 (0x1)	DAI1_PB02_O	Pin Buffer 2
000010 (0x2)	DAI1_PB03_O	Pin Buffer 3
000011 (0x3)	DAI1_PB04_O	Pin Buffer 4
000100 (0x4)	DAI1_PB05_O	Pin Buffer 5
000101 (0x5)	DAI1_PB06_O	Pin Buffer 6
000110 (0x6)	DAI1_PB07_O	Pin Buffer 7
000111 (0x7)	DAI1_PB08_O	Pin Buffer 8
001000 (0x8)	DAI1_PB09_O	Pin Buffer 9
001001 (0x9)	DAI1_PB10_O	Pin Buffer 10
001100 (0xC) -010001 (0x11)	Reserved	
010010 (0x12)	DAI1_PB19_O	Pin Buffer 19
010011 (0x13)	DAI1_PB20_O	Pin Buffer 20
010100 (0x14)	SPT4_AD0_O	SPORT 4 Data AD0
010101 (0x15)	SPT4_AD1_O	SPORT 4 Data AD1
010110 (0x16)	SPT4_BD0_O	SPORT 4 Data BD0
010111 (0x17)	SPT4_BD1_O	SPORT 4 Data BD1
011000 (0x18)	SPT5_AD0_O	SPORT 5 Data AD0
011001 (0x19)	SPT5_AD1_O	SPORT 5 Data AD1
011010 (0x1A)	SPT5_BD0_O	SPORT 5 Data BD0
011011 (0x1B)	SPT5_BD1_O	SPORT 5 Data BD1
011100 (0x1C)	SPT6_AD0_O	SPORT 6 Data AD0
011101 (0x1D)	SPT6_AD1_O	SPORT 6 Data AD1
011110 (0x1E)	SPT6_BD0_O	SPORT 6 Data BD0
011111 (0x1F)	SPT6_BD1_O	SPORT 6 Data BD1
100000 (0x20)	SRC4_DAT_OP_O	SRC4 Data Out
100001 (0x21)	SRC5_DAT_OP_O	SRC5 Data Out
100010 (0x22)	SRC6_DAT_OP_O	SRC6 Data Out
100011 (0x23)	SRC7_DAT_OP_O	SRC7 Data Out
100100 (0x24)	SRC4_TDM_IP_O	SRC4 Data Out
100101 (0x25)	SRC5_TDM_IP_O	SRC5 Data Out
100110 (0x26)	SRC6_TDM_IP_O	SRC6 Data Out

Table 20-17: DAI1 Group B – Serial Data Signals (Continued)

Selection Code	Source Signal	Description (Source Selection)
100111 (0x27)	SRC7_TDM_IP_O	SRC7 Data Out
101000 (0x28)	SPDIF1_RX_DAT_O	SPDIF 1 RX Serial Data Out
101100(0x2C)	SPT7_AD0_O	SPORT 7 Data AD0
101101(0x2D)	SPT7_AD1_O	SPORT 7 Data AD1
101110(0x2E)	SPT7_BD0_O	SPORT 7 Data BD0
101111(0x2F)	SPT7_BD1_O	SPORT 7 Data BD1
110000(0x30)	SPDIF1_TX_O	SPDIF 1 TX Biphase Stream
110001 (0x31)	SRC3_CRS_DAT_OP_O	SRC3 Data Out (DAI0)
110010 (0x32)	SRC3_CRS_TDM_IP_O	SRC3 Data Out (DAI0)
110001(0x33)–111101(0x3D)	Reserved	
111110 (0x3E)	LOW	Logic Level Low (0)
111111 (0x3F)	HIGH	Logic Level High (1)

DAI1 Group C – Frame Sync Source Signals

The group C frame sync signals are listed in the following table.

Table 20-18: DAI1 Group C – Frame Sync Signals

Selection Code	Source Signal	Description (Source Selection)
00000 (0x0)	DAI0_PB01_O	Pin Buffer 1
00001 (0x1)	DAI1_PB02_O	Pin Buffer 2
00010 (0x2)	DAI1_PB03_O	Pin Buffer 3
00011 (0x3)	DAI1_PB04_O	Pin Buffer 4
00100 (0x4)	DAI1_PB05_O	Pin Buffer 5
00101 (0x5)	DAI1_PB06_O	Pin Buffer 6
00110 (0x6)	DAI1_PB07_O	Pin Buffer 7
00111 (0x7)	DAI1_PB08_O	Pin Buffer 8
01000 (0x8)	DAI1_PB09_O	Pin Buffer 9
01001 (0x9)	DAI1_PB10_O	Pin Buffer 10
01100 (0xC)	Reserved	
01101 (0xD)	Reserved	
01110 (0xE)	DAI1_CRS_PB04_O	Pin Buffer 4 (DAI0)
01111 (0xF)	DAI1_CRS_PB06_O	Pin Buffer 6 (DAI0)

Table 20-18: DAI1 Group C – Frame Sync Signals (Continued)

Selection Code	Source Signal	Description (Source Selection)
10000 (0x10)	Reserved	
10001 (0x11)	Reserved	
10010 (0x12)	DAI1_PB19_O	Pin Buffer 19
10011 (0x13)	DAI1_PB20_O	Pin Buffer 20
10100 (0x14)	SPT4_AFS_O	SPORT 4 Frame Sync A
10101 (0x15)	SPT4_BFS_O	SPORT 4 Frame Sync B
10110 (0x16)	SPT5_AFS_O	SPORT 5 Frame Sync A
10111 (0x17)	SPT5_BFS_O	SPORT 5 Frame Sync B
11000 (0x18)	SPT6_AFS_O	SPORT 6 Frame Sync A
11001 (0x19)	SPT6_BFS_O	SPORT 6 Frame Sync B
11010 (0x1A)	SPDIF1_FS_O	SPDIF 1 RX Frame Sync Output
11011 (0x1B)	Reserved	
11100 (0x1C)	PCG0_FSC_O	Precision Frame Sync C Output
11101 (0x1D)	PCG0_FSD_O	Precision Frame Sync D Output
11110 (0x1E)	LOW	Logic Level Low (0)
11111 (0x1F)	HIGH	Logic Level High (1)

DAI1 Group D – Pin Signal Assignment Source Signals

The group D pin signal assignments are listed in the following table.

Table 20-19: DAI1 Group D – Pin Signal Assignments

Selection Code	Source Signal	Description (Source Selection)
0000000 (0x0)	DAI1_PB01_O	Pin Buffer 1
0000001 (0x1)	DAI1_PB02_O	Pin Buffer 2
0000010 (0x2)	DAI1_PB03_O	Pin Buffer 3
0000011 (0x3)	DAI1_PB04_O	Pin Buffer 4
0000100 (0x4)	DAI1_PB05_O	Pin Buffer 5
0000101 (0x5)	DAI1_PB06_O	Pin Buffer 6
0000110 (0x6)	DAI1_PB07_O	Pin Buffer 7
0000111 (0x7)	DAI1_PB08_O	Pin Buffer 8
0001000 (0x8)	DAI1_PB09_O	Pin Buffer 9
0001001 (0x9)	DAI1_PB10_O	Pin Buffer 10

Table 20-19: DAI1 Group D – Pin Signal Assignments (Continued)

Selection Code	Source Signal	Description (Source Selection)
0001100 (0xC) - 0010001 (0x11)	Reserved	
0010010 (0x12)	DAI1_PB19_O	Pin Buffer 19
0010011 (0x13)	DAI1_PB20_O	Pin Buffer 20
0010100 (0x14)	SPT4_AD0_O	SPORT 4 Data AD0
0010101 (0x15)	SPT4_AD1_O	SPORT 4 Data AD1
0010110 (0x16)	SPT4_BD0_O	SPORT 4 Data BD0
0010111 (0x17)	SPT4_BD1_O	SPORT 4 Data BD1
0011000 (0x18)	SPT5_AD0_O	SPORT 5 Data AD0
0011001 (0x19)	SPT5_AD1_O	SPORT 5 Data AD1
0011010 (0x1A)	SPT5_BD0_O	SPORT 5 Data BD0
0011011 (0x1B)	SPT5_BD1_O	SPORT 5 Data BD1
0011100 (0x1C)	SPT6_AD0_O	SPORT 6 Data AD0
0011101 (0x1D)	SPT6_AD1_O	SPORT 6 Data AD1
0011110 (0x1E)	SPT6_BD0_O	SPORT 6 Data BD0
0011111 (0x1F)	SPT6_BD1_O	SPORT 6 Data BD1
0100000 (0x20)	SPT4_ACLK_O	SPORT 4 Clock A
0100001 (0x21)	SPT4_BCLK_O	SPORT4 Clock B
0100010 (0x22)	SPT5_ACLK_O	SPORT 5 Clock A
0100011 (0x23)	SPT5_BCLK_O	SPORT 5 Clock B
0100100 (0x24)	SPT6_ACLK_O	SPORT 6 Clock A
0100101 (0x25)	SPT6_BCLK_O	SPORT 6 Clock B
0100110 (0x26)	SPT4_AFS_O	SPORT 4 Frame Sync A
0100111 (0x27)	SPT4_BFS_O	SPORT 4 Frame Sync B
0101000 (0x28)	SPT5_AFS_O	SPORT 5 Frame Sync A
0101001 (0x29)	SPT5_BFS_O	SPORT 5 Frame Sync B
0101010 (0x2A)	SPT6_AFS_O	SPORT 5 Frame Sync A
0101011 (0x2B)	SPT6_BFS_O	SPORT 5 Frame Sync B
0101100 (0x2C)	SPT7_AD0_O	SPORT 7 Data AD0
0101101 (0x2D)	SPT7_AD1_O	SPORT 7 Data AD1
0101110 (0x2E)	SPT7_BD0_O	SPORT 7 Data BD0
0101111 (0x2F)	SPT7_BD1_O	SPORT 7 Data BD1

Table 20-19: DAI1 Group D – Pin Signal Assignments (Continued)

Selection Code	Source Signal	Description (Source Selection)
0110000 (0x30)	MLB0_CLKOUT	MLB PLL clock output
0110001 (0x31)	SPDIF1_TX_BLKSTART_O	SPDIF 1 TX Block Start Output
0110100 (0x34)	SPT7_ACLK_O	SPORT 7 Clock A
0110101 (0x35)	SPT7_BCLK_O	SPORT 7 Clock B
0110110 (0x36)	SPT7_AFS_O	SPORT 7 Frame Sync A
0110111 (0x37)	SPT7_BFS_O	SPORT 7 Frame Sync B
0111000 (0x38)	PCG0_CLKC_O	Precision Clock C
0111001 (0x39)	PCG0_CLKD_O	Precision Clock D
0111010 (0x3A)	PCG0_FSC_O	Precision Frame Sync C
0111011 (0x3B)	PCG0_FSD_O	Precision Frame Sync D
0111100 (0x3C)	Reserved	
0111101 (0x3D)	SRC4_DAT_OP_O	SRC4 Data Output
0111110 (0x3E)	SRC5_DAT_OP_O	SRC5 Data Output
0111111 (0x3F)	SRC6_DAT_OP_O	SRC6 Data Output
1000000 (0x40)	SRC7_DAT_OP_O	SRC7 Data Output
1000001 (0x41)	SPDIF1_RX_DAT_O	SPDIF 1 RX Data Output
1000010 (0x42)	SPDIF1_FS_O	SPDIF 1 RX Frame Sync Output
1000011 (0x43)	SPDIF1_RXCLK_O	SPDIF 1 RX Clock Output
1000100 (0x44)	SPDIF1_RX_TDMCLK_O	SPDIF 1 RX TDM Clock Output
1000101 (0x45)	SPDIF1_TX_O	SPDIF 1 TX Biphase Encoded Data Output
1000110 (0x46)	SPT4_ATDV_O	SPORT4 Transmit A Data Valid Output
1000111 (0x47)	SPT4_BTDTV_O	SPORT4 Transmit B Data Valid Output
1001000 (0x48)	SPT5_ATDV_O	SPORT5 Transmit A Data Valid Output
1001001 (0x49)	SPT5_BTDTV_O	SPORT5 Transmit B Data Valid Output
1001010 (0x4A)	SPT6_ATDV_O	SPORT6 Transmit A Data Valid Output
1001011 (0x4B)	SPT6_BTDTV_O	SPORT6 Transmit B Data Valid Output
1001100 (0x4C)	SPT7_ATDV_O	SPORT7 Transmit A Data Valid Output
1001101 (0x4D)	SPT7_BTDTV_O	SPORT7 Transmit B Data Valid Output
1001110 (0x4E)	DAI1_CRS_PB05_O	Pin Buffer 5 (DAI0)
1001111 (0x4F)	DAI1_CRS_PB06_O	Pin Buffer 6 (DAI0)
1010000 (0x50)	PCG0_CRS_CLKA_O	Precision Clock A (DAI0)

Table 20-19: DAI1 Group D – Pin Signal Assignments (Continued)

Selection Code	Source Signal	Description (Source Selection)
1010001 (0x51)	PCG0_CRS_CLKB_O	Precision Clock B (DAI0)
1010010 (0x52)	PCG0_CRS_FSA_O	Precision Frame Sync A (DAI0)
1010011 (0x53)	PCG0_CRS_FSB_O	Precision Frame Sync B (DAI0)
1010100 (0x54)	DAI1_CRS_PB03_O	Pin Buffer 3 (DAI0)
1010101 (0x55)	DAI1_CRS_PB04_O	Pin Buffer 4 (DAI0)
1010110 – 1111101	Reserved	
1111110 (0x7E)	LOW	Logic Level Low (0)
1111111 (0x7F)	HIGH	Logic Level High (1)

DAI1 Group E – Miscellaneous Source Signals

The group E miscellaneous signals are listed in the following table.

Table 20-20: DAI1 Group E – Miscellaneous Signals

Selection Code	Source Signal	Description (Source Selection)
00000 (0x0)	DAI1_PB01_O	Pin Buffer 1 Output
00001 (0x1)	DAI1_PB02_O	Pin Buffer 2 Output
00010 (0x2)	DAI1_PB03_O	Pin Buffer 3 Output
00011 (0x3)	DAI1_PB04_O	Pin Buffer 4 Output
00100 (0x4)	DAI1_PB05_O	Pin Buffer 5 Output
00101 (0x5)	DAI1_PB06_O	Pin Buffer 6 Output
00110 (0x6)	DAI1_PB07_O	Pin Buffer 7 Output
00111 (0x7)	DAI1_PB08_O	Pin Buffer 8 Output
01000 (0x8)	DAI1_PB09_O	Pin Buffer 9 Output
01001 (0x9)	DAI1_PB10_O	Pin Buffer 10 Output
01100 (0xC) - 10001 (0x11)	Reserved	
10010 (0x12)	DAI1_PB19_O	Pin Buffer 19 Output
10011 (0x13)	DAI1_PB20_O	Pin Buffer 20 Output
10100 (0x14)	SPT4_AFS_O	SPORT 4 Frame Sync A
10101 (0x15)	SPT4_BFS_O	SPORT 4 Frame Sync B
10110 (0x16)	SPT5_AFS_O	SPORT 5 Frame Sync A
10111 (0x17)	SPT5_BFS_O	SPORT 5 Frame Sync B
11000 (0x18)	SPT6_AFS_O	SPORT 6 Frame Sync A

Table 20-20: DAI1 Group E – Miscellaneous Signals (Continued)

Selection Code	Source Signal	Description (Source Selection)
11001 (0x19)	SPT6_BFS_O	SPORT 6 Frame Sync B
11010 (0x1A)	SPDIF1_TX_BLKSTART_O	SPDIF 1 TX Block Start Output
11011 (0x1B)	PCG0_FSC_O	Precision Frame Sync C
11100 (0x1C)	PCG0_CLKD_O	Precision Clock D
11101 (0x1D)	PCG0_FSD_O	Precision Frame Sync D
11110 (0x1E)	LOW	Logic Level Low (0) as a Source
11111 (0x1F)	HIGH	Logic Level High (1) as a Source

DAI1 Group F – Pin Output Enable Source Signals

The group F pin output enable signals are listed in the following table.

Table 20-21: DAI1 Group F – Pin Output Enable

Selection Code	Source Signal	Description (Source Selection)
000000 (0x0)	LOW	Logic Level Low (0)
000001 (0x1)	HIGH	Logic Level High (1)
000010 (0x2)	DAI1_MISCA0_O	DAI1 Miscellaneous Control A0 Output
000011 (0x3)	DAI1_MISCA1_O	DAI1 Miscellaneous Control A1 Output
000100 (0x4)	DAI1_MISCA2_O	DAI1 Miscellaneous Control A2 Output
000101 (0x5)	DAI1_MISCA3_O	DAI1 Miscellaneous Control A3 Output
000110 (0x6)	DAI1_MISCA4_O	DAI1 Miscellaneous Control A4 Output
000111 (0x7)	DAI1_MISCA5_O	DAI1 Miscellaneous Control A5 Output
001000 (0x8)	SPT4_ACLK_PBEN_O	SPORT 4 Clock A Output Enable
001001 (0x9)	SPT4_AFS_PBEN_O	SPORT 4 Frame Sync A Output Enable
001010 (0xA)	SPT4_AD0_PBEN_O	SPORT 4 Data AD0 Output Enable
001011 (0xB)	SPT4_AD1_PBEN_O	SPORT 4 Data AD1 Output Enable
001100 (0xC)	SPT4_BCLK_PBEN_O	SPORT 4 Clock B Output Enable
001101 (0xD)	SPT4_BFS_PBEN_O	SPORT 4 Frame Sync B Output Enable
001110 (0xE)	SPT4_BD0_PBEN_O	SPORT 4 Data BD0 Output Enable
001111 (0xF)	SPT4_BD1_PBEN_O	SPORT 4 Data BD1 Output Enable
010000 (0x10)	SPT5_ACLK_PBEN_O	SPORT 5 Clock A Output Enable
010001 (0x11)	SPT5_AFS_PBEN_O	SPORT 5 Frame Sync A Output Enable
010010 (0x12)	SPT5_AD0_PBEN_O	SPORT 5 Data AD0 Output Enable

Table 20-21: DAI1 Group F – Pin Output Enable (Continued)

Selection Code	Source Signal	Description (Source Selection)
010011 (0x13)	SPT5_AD1_PBEN_O	SPORT 5 Data AD1 Output Enable
010100 (0x14)	SPT5_BCLK_PBEN_O	SPORT 5 Clock B Output Enable
010101 (0x15)	SPT5_BFS_PBEN_O	SPORT 5 Frame Sync B Output Enable
010110 (0x16)	SPT5_BD0_PBEN_O	SPORT 5 Data BD0 Output Enable
010111 (0x17)	SPT5_BD1_PBEN_O	SPORT 5 Data BD1 Output Enable
011000 (0x18)	SPT6_ACLK_PBEN_O	SPORT 6 Clock A Output Enable
011001 (0x19)	SPT6_AFS_PBEN_O	SPORT 6 Frame Sync A Output Enable
011010 (0x1A)	SPT6_AD0_PBEN_O	SPORT 6 Data AD0 Output Enable
011011 (0x1B)	SPT6_AD1_PBEN_O	SPORT 6 Data AD1 Output Enable
011100 (0x1C)	SPT6_BCLK_PBEN_O	SPORT 6 Clock B Output Enable
011101 (0x1D)	SPT6_BFS_PBEN_O	SPORT 6 Frame Sync B Output Enable
011110 (0x1E)	SPT6_BD0_PBEN_O	SPORT 6 Data BD0 Output Enable
011111 (0x1F)	SPT6_BD1_PBEN_O	SPORT 6 Data BD1 Output Enable
100000 (0x20)	SPT7_ACLK_PBEN_O	SPORT 7 Clock A Output Enable
100001 (0x21)	SPT7_AFS_PBEN_O	SPORT 7 Frame Sync A Output Enable
100010 (0x22)	SPT7_AD0_PBEN_O	SPORT 7 Data AD0 Output Enable
100011 (0x23)	SPT7_AD1_PBEN_O	SPORT 7 Data AD1 Output Enable
100100 (0x24)	SPT7_BCLK_PBEN_O	SPORT 6 Clock B Output Enable
100101 (0x25)	SPT7_BFS_PBEN_O	SPORT 7 Frame Sync B Output Enable
100110 (0x26)	SPT7_BD0_PBEN_O	SPORT 7 Data BD0 Output Enable
100111 (0x27)	SPT7_BD1_PBEN_O	SPORT 7 Data BD1 Output Enable
101000 (0x28)	SPT4_ATDV_PBEN_O	SPORT 4 A Transmit Data Valid Output
101001 (0x29)	SPT4_BTDTV_PBEN_O	SPORT 4 B Transmit Data Valid Output
101010 (0x2A)	SPT5_ATDV_PBEN_O	SPORT 5 A Transmit Data Valid Output
101011 (0x2B)	SPT5_BTDTV_PBEN_O	SPORT 5 B Transmit Data Valid Output
101100 (0x2C)	SPT6_ATDV_PBEN_O	SPORT 6 A Transmit Data Valid Output
101101 (0x2D)	SPT6_BTDTV_PBEN_O	SPORT 6 B Transmit Data Valid Output
100111 (0x2E)	SPT7_ATDV_PBEN_O	SPORT 7 A Transmit Data Valid Output
101110 (0x2F)	SPT7_BTDTV_PBEN_O	SPORT 7 B Transmit Data Valid Output
110000 (0x30)–111111 (0x3F)	Reserved	

DAI Destination Registers Overview

The tables in [DAI Routing Capabilities](#) provide high level descriptions that illustrate source (output) connections to destinations (inputs) depending on the routing groups A through F. This sections lists the various input fields (INx), which are described in detail in the [ADSP-21560_21561_21564_21568 DAI Register Descriptions](#) section.

Table 20-22: Clock Destination Registers (Group A)

DAI Register	Bit Field Name	DAI0 Mapping	DAI1 Mapping	Description
DAI_CLK0	IN0	SPT0_ACLK_I	SPT4_ACLK_I	SPORT0A or SPORT4A Clock
	IN1	SPT0_BCLK_I	SPT4_BCLK_I	SPORT0B or SPORT4B Clock
	IN2	SPT1_ACLK_I	SPT5_ACLK_I	SPORT1A or SPORT5A Clock
	IN3	SPT1_BCLK_I	SPT5_BCLK_I	SPORT1B or SPORT5B Clock
	IN4	SPT2_ACLK_I	SPT6_ACLK_I	SPORT2A or SPORT6A Clock
	IN5	SPT2_BCLK_I	SPT6_BCLK_I	SPORT2B or SPORT6B Clock
DAI_CLK1	IN0	SRC0_CLK_IP_I	SRC4_CLK_IP_I	SRC0 or SRC4 Clock Input
	IN1	SRC0_CLK_OP_I	SRC4_CLK_OP_I	SRC0 or SRC4 Clock Output
	IN2	SRC1_CLK_IP_I	SRC5_CLK_IP_I	SRC1 or SRC5 Clock Input
	IN3	SRC1_CLK_OP_I	SRC5_CLK_OP_I	SRC1 or SRC5 Clock Output
	IN4	SRC2_CLK_IP_I	SRC6_CLK_IP_I	SRC2 or SRC6 Clock Input
	IN5	SRC2_CLK_OP_I	SRC6_CLK_OP_I	SRC2 or SRC6 Clock Output
DAI_CLK2	IN0	SRC3_CLK_IP_I	SRC7_CLK_IP_I	SRC3 or SRC7 Clock Input
	IN1	SRC3_CLK_OP_I	SRC7_CLK_OP_I	SRC3 or SRC7 Clock Output
	IN2	SPDIF0_TX_CLK_I	SPDIF1_TX_CLK_I	SPDIF0 or SPDIF1 TX Clock
DAI_CLK3	IN5	SPDIF0_TX_HFCLK_I	SPDIF1_TX_HFCLK_I	SPDIF0 or SPDIF1 TX HF Clock
DAI_CLK4	IN0	PCG0_EXTCLKA_I	PCG0_EXTCLKC_I	PCG0 External Clock A or C
	IN1	PCG0_EXTCLKB_I	PCG0_EXTCLKD_I	PCG0 External Clock B or D
	IN3	SPDIF0_TX_EXT_SYNC_I	SPDIF1_TX_EXT_SYNC_I	SPDIF0 or SPDIF1 TX External Sync
	IN4	PCG0_SYNC_CLKA_I	PCG0_SYNC_CLKC_I	PCG0 Sync Clock A or C
	IN5	PCG0_SYNC_CLKB_I	PCG0_SYNC_CLKD_I	PCG0 Sync Clock B or D
DAI_CLK5	IN0	SPT3_ACLK_I	SPT7_ACLK_I	SPORT3A or SPORT 7A Clock
	IN1	SPT3_BCLK_I	SPT7_BCLK_I	SPORT3B or SPORT 7B Clock

Table 20-23: Data Destination Registers (Group B)

DAI Register	Bit Field Name	DAI0 Mapping	DAI1 Mapping	Description
DAI_DAT0	IN0	SPT0_AD0_I	SPT4_AD0_I	SPORT0A or SPORT4A Primary Data
	IN1	SPT0_AD1_I	SPT4_AD1_I	SPORT0A or SPORT4A Secondary Data
	IN2	SPT0_BD0_I	SPT4_BD0_I	SPORT0B or SPORT4B Primary Data
	IN3	SPT0_BD1_I	SPT4_BD1_I	SPORT0B or SPORT4B Secondary Data
	IN4	SPT1_AD0_I	SPT5_AD0_I	SPORT1A or SPORT5A Primary Data
DAI_DAT1	IN0	SPT1_AD1_I	SPT5_AD1_I	SPORT1A or SPORT5A Secondary Data
	IN1	SPT1_BD0_I	SPT5_BD0_I	SPORT1B or SPORT5B Primary Data
	IN2	SPT1_BD1_I	SPT5_BD1_I	SPORT1B or SPORT5B Secondary Data
	IN3	SPT2_AD0_I	SPT6_AD0_I	SPORT2A or SPORT6A Primary Data
	IN4	SPT2_AD1_I	SPT6_AD1_I	SPORT2A or SPORT6A Secondary Data
DAI_DAT2	IN0	SPT2_BD0_I	SPT6_BD0_I	SPORT2B or SPORT6B Primary Data
	IN1	SPT2_BD1_I	SPT6_BD1_I	SPORT2B or SPORT6B Secondary Data
	IN2	SRC0_DAT_IP_I	SRC4_DAT_IP_I	SRC0 or SRC4 Data
	IN3	SRC1_DAT_IP_I	SRC5_DAT_IP_I	SRC1 or SRC5 Data
	IN4	SRC2_DAT_IP_I	SRC6_DAT_IP_I	SRC2 or SRC6 Data
DAI_DAT3	IN0	SRC3_DAT_IP_I	SRC7_DAT_IP_I	SRC3 or SRC7 Data
	IN1	SRC0_DAT_TDM_OP_I	SRC4_DAT_TDM_OP_I	SRC0 or SRC4 TDM output port
	IN2	SRC1_DAT_TDM_OP_I	SRC5_DAT_TDM_OP_I	SRC1 or SRC5 TDM Output port Data
	IN3	SRC2_DAT_TDM_OP_I	SRC6_DAT_TDM_OP_I	SRC2 or SRC6 TDM Output port Data
	IN4	SRC3_DAT_TDM_OP_I	SRC7_DAT_TDM_OP_I	SRC3 or SRC7 TDM Output port Data
DAI_DAT4	IN0	SPDIF0_TX_DAT_I	SPDIF1_TX_DAT_I	SPDIF0 or SPDIF1 serial transmitter Data
DAI_DAT5	IN4	SPDIF0_RX_I	SPDIF1_RX_I	SPDIF0 or SPDIF1 receiver bi-phase Data

Table 20-23: Data Destination Registers (Group B) (Continued)

DAI Register	Bit Field Name	DAI0 Mapping	DAI1 Mapping	Description
DAI_DAT6	IN0	SPT3_AD0_I	SPT7_AD0_I	SPORT3A or SPORT7A Primary Data
	IN1	SPT3_AD1_I	SPT7_AD1_I	SPORT3A or SPORT7A Secondary Data
	IN2	SPT3_BD0_I	SPT7_BD0_I	SPORT3B or SPORT7B Primary Data
	IN3	SPT3_BD1_I	SPT7_BD1_I	SPORT3B or SPORT7B Secondary Data
	IN4	Reserved		

Table 20-24: Frame Sync Destination Registers (Group C)

DAI Register	Bit Field Name	DAI0 Mapping	DAI1 Mapping	Description
DAI_FS0	IN0	SPT0_AFS_I	SPT4_AFS_I	SPORT0A or SPORT4A Frame Sync
	IN1	SPT0_BFS_I	SPT4_BFS_I	SPORT0B or SPORT4B Frame Sync
	IN2	SPT1_AFS_I	SPT5_AFS_I	SPORT1A or SPORT5A Frame Sync
	IN3	SPT1_BFS_I	SPT5_BFS_I	SPORT1B or SPORT5B Frame Sync
	IN4	SPT2_AFS_I	SPT6_AFS_I	SPORT2A or SPORT6A Frame Sync
	IN5	SPT2_BFS_I	SPT6_BFS_I	SPORT2B or SPORT6B Frame Sync
DAI_FS1	IN0	SRC0_FS_IP_I	SRC4_FS_IP_I	SRC0 or SRC4 Frame Sync Input
	IN1	SRC0_FS_OP_I	SRC4_FS_OP_I	SRC0 or SRC4 Frame Sync Output
	IN2	SRC1_FS_IP_I	SRC5_FS_IP_I	SRC1 or SRC5 Frame Sync Input
	IN3	SRC1_FS_OP_I	SRC5_FS_OP_I	SRC1 or SRC5 Frame Sync Output
	IN4	SRC2_FS_IP_I	SRC6_FS_IP_I	SRC2 or SRC6 Frame Sync Input
	IN5	SRC2_FS_OP_I	SRC6_FS_OP_I	SRC2 or SRC6 Frame Sync Output
DAI_FS2	IN0	SRC3_FS_IP_I	SRC7_FS_IP_I	SRC3 or SRC7 Frame Sync Input
	IN1	SRC3_FS_OP_I	SRC7_FS_OP_I	SRC3 or SRC7 Frame Sync Output
	IN2	SPDIF0_TX_FS_I	SPDIF1_TX_FS_I	SPDIF0 or SPDIF1 Transmit Frame Sync
DAI_FS4	IN0	SPT3_AFS_I	SPT7_AFS_I	SPORT3A or SPORT7A Frame Sync
	IN1	SPT3_BFS_I	SPT7_BFS_I	SPORT3B or SPORT7B Frame Sync

Table 20-25: Pin Buffer Assignment Destination Registers (Group D)

DAI Register	Bit Field Name	DAI0 Mapping	DAI1 Mapping	Description
DAI_PIN0	IN0	DAI0_PB01_I	DAI1_PB01_I	Pin Buffer 1
	IN1	DAI0_PB02_I	DAI1_PB02_I	Pin Buffer 2
	IN2	DAI0_PB03_I	DAI1_PB03_I	Pin Buffer 3
	IN3	DAI0_PB04_I	DAI1_PB04_I	Pin Buffer 4
DAI_PIN1	IN0	DAI0_PB05_I	DAI1_PB05_I	Pin Buffer 5
	IN1	DAI0_PB06_I	DAI1_PB06_I	Pin Buffer 6
	IN2	DAI0_PB07_I	DAI1_PB07_I	Pin Buffer 7
	IN3	DAI0_PB08_I	DAI1_PB08_I	Pin Buffer 8
DAI_PIN2	IN0	DAI0_PB09_I	DAI1_PB09_I	Pin Buffer 9
	IN1	DAI0_PB10_I	DAI1_PB10_I	Pin Buffer 10
	IN2	Reserved		
	IN3	Reserved		
DAI_PIN3	IN0	Reserved		
	IN1	Reserved		
	IN2	Reserved		
	IN3	Reserved		
DAI_PIN4	IN0	Reserved		
	IN1	Reserved		
	IN2	DAI0_PB19_I	DAI1_PB19_I	Pin Buffer 19
	IN3	DAI0_PB20_I	DAI1_PB20_I	Pin Buffer 20
	IN4	INV_DAI0_PB19_I	INV_DAI1_PB19_I	Inverted Pin Buffer 19
	IN5	INV_DAI0_PB20_I	INV_DAI1_PB20_I	Inverted Pin Buffer 20

Table 20-26: Pin Buffer Enable Destination Registers (Group E)

DAI Register	Bit Field Name	DAI0 Mapping	DAI1 Mapping	Description
DAI_PBEN0	IN0	DAI0_PBEN01_I	DAI1_PBEN01_I	Pin Buffer 1 Enable
	IN1	DAI0_PBEN02_I	DAI1_PBEN02_I	Pin Buffer 2 Enable
	IN2	DAI0_PBEN03_I	DAI1_PBEN03_I	Pin Buffer 3 Enable
	IN3	DAI0_PBEN04_I	DAI1_PBEN04_I	Pin Buffer 4 Enable
	IN4	DAI0_PBEN05_I	DAI1_PBEN05_I	Pin Buffer 5 Enable

Table 20-26: Pin Buffer Enable Destination Registers (Group E) (Continued)

DAI Register	Bit Field Name	DAI0 Mapping	DAI1 Mapping	Description
DAI_PBEN1	IN0	DAI0_PBEN06_I	DAI1_PBEN06_I	Pin Buffer 6 Enable
	IN1	DAI0_PBEN07_I	DAI1_PBEN07_I	Pin Buffer 7 Enable
	IN2	DAI0_PBEN08_I	DAI1_PBEN08_I	Pin Buffer 8 Enable
	IN3	DAI0_PBEN09_I	DAI1_PBEN09_I	Pin Buffer 9 Enable
	IN4	DAI0_PBEN10_I	DAI1_PBEN10_I	Pin Buffer 10 Enable
DAI_PBEN2	IN0	Reserved		
	IN1	Reserved		
	IN2	Reserved		
	IN3	Reserved		
	IN4	Reserved		
DAI_PBEN3	IN0	Reserved		
	IN1	Reserved		
	IN2	Reserved		
	IN3	DAI0_PBEN19_I	DAI1_PBEN19_I	Pin Buffer 19 Enable
	IN4	DAI0_PBEN20_I	DAI1_PBEN20_I	Pin Buffer 20 Enable

The the *Miscellaneous Control Registers* table, DAI_MISCAx_I is an alias name for DAI_INT_x_I.

Table 20-27: Miscellaneous Control Destination Registers (Group F)

DAI Register	Bit Field Name	DAI0 Mapping	DAI1 Mapping	Description
DAI_MISC0	IN6	DAI0_MISCA0_I/ DAI0_INT_6_I	DAI1_MISCA0_I/ DAI1_INT_6_I	DAIx miscellaneous A0 input, or DAIx interrupt 28
	IN7	DAI0_MISCA1_I/ DAI0_INT_7_I	DAI1_MISCA1_I/ DAI1_INT_7_I	DAIx miscellaneous A1 input, or DAIx interrupt 29
	IN8	DAI0_MISCA2_I/ DAI0_INT_8_I	DAI1_MISCA2_I/ DAI1_INT_8_I	DAIx miscellaneous A2 input, or DAIx interrupt 30
	IN9	DAI0_MISCA3_I/ DAI0_INT_9_I	DAI1_MISCA3_I/ DAI1_INT_9_I	DAIx miscellaneous A3 input, or DAIx interrupt 31
	IN10	DAI0_MISCA4_I	DAI1_MISCA4_I	DAIx miscellaneous A4 input
	IN11	DAI0_MISCA5_I	DAI1_MISCA5_I	DAIx miscellaneous A5 input
	INV10	DAI0_INV_MISCA4_I	DAI1_INV_MISCA4_I	DAIx inverted miscellaneous A4 input

Table 20-27: Miscellaneous Control Destination Registers (Group F) (Continued)

DAI Register	Bit Field Name	DAI0 Mapping	DAI1 Mapping	Description
	INV11	DAI0_INV_MISCA5_I	DAI1_INV_MISCA5_I	DAIx inverted miscellaneous A5 input
DAI_MISC1	IN0	DAI0_INT_0	DAI1_INT_0	DAIx interrupt 22
	IN1	DAI0_INT_1	DAI1_INT_1	DAIx interrupt 23
	IN2	DAI0_INT_2	DAI1_INT_2	DAIx interrupt 24
	IN3	DAI0_INT_3	DAI1_INT_3	DAIx interrupt 25
	IN4	DAI0_INT_4	DAI1_INT_4	DAIx interrupt 26
	IN5	DAI0_INT_5	DAI1_INT_5	DAIx interrupt 27

The *Interrupt Events* table presents the information for the following registers.

- DAI_IMSK_PRI (Core Interrupt Priority Assignment) register
- DAI_IMSK_FE (Falling-Edge Interrupt Mask) and DAI_IMSK_RE (Rising-Edge Interrupt Mask) registers
- DAI_IRPTL_H (High-Priority Interrupt Latch) and DAI_IRPTL_L (Low-Priority Interrupt Latch) registers
- DAI_IRPTL_HS (Shadow High Interrupt Latch) and DAI_IRPTL_LS (Shadow Low Interrupt Latch) registers

Table 20-28: Interrupt Events

Bit Field Name	DAI0 Mapping	DAI1 Mapping	Description
RXVALID	SPDIF0_RXVALID_INT	SPDIF1_RXVALID_INT	SPDIF0 or SPDIF1 RX Valid interrupt
RXLOCK	SPDIF0_RXLOCK_INT	SPDIF1_RXLOCK_INT	SPDIF0 or SPDIF1 RX Lock interrupt
RXLOSSOF-LOCK	SPDIF0_RXLOSSOFLOCK_INT	SPDIF1_RXLOSSOFLOCK_INT	SPDIF0 or SPDIF1 RX Loss of Lock interrupt
RXNONAUDIO	SPDIF0_RXNONAUDIO_INT	SPDIF1_RXNONAUDIO_INT	SPDIF0 or SPDIF1 RX Non-audio interrupt
SRC0MUTE	SRC0_MUTE_INT	SRC4_MUTE_INT	DAI0 or DAI1 ASRC0 Mute interrupt
SRC1MUTE	SRC1_MUTE_INT	SRC5_MUTE_INT	DAI0 or DAI1 ASRC1 Mute interrupt
SRC2MUTE	SRC2_MUTE_INT	SRC6_MUTE_INT	DAI0 or DAI1 ASRC2 Mute interrupt
SRC3MUTE	SRC3_MUTE_INT	SRC7_MUTE_INT	DAI0 or DAI1 ASRC3 Mute interrupt

Table 20-28: Interrupt Events (Continued)

Bit Field Name	DAI0 Mapping	DAI1 Mapping	Description
MISCINT0	DAI0_INT_00	DAI1_INT_00	DAI0 or DAI1 Miscellaneous Interrupt 0
MISCINT1	DAI0_INT_01	DAI1_INT_01	DAI0 or DAI1 Miscellaneous Interrupt 1
MISCINT2	DAI0_INT_02	DAI1_INT_02	DAI0 or DAI1 Miscellaneous Interrupt 2
MISCINT3	DAI0_INT_03	DAI1_INT_03	DAI0 or DAI1 Miscellaneous Interrupt 3
MISCINT4	DAI0_INT_04	DAI1_INT_04	DAI0 or DAI1 Miscellaneous Interrupt 4
MISCINT5	DAI0_INT_05	DAI1_INT_05	DAI0 or DAI1 Miscellaneous Interrupt 5
MISCINT6	DAI0_EXTMISCA0_INT/ DAI0_INT_06	DAI1_EXTMISCA0_INT/ DAI1_INT_06	DAI0 or DAI1 Miscellaneous Interrupt 6 / External Miscellaneous A0 Interrupt
MISCINT7	DAI0_EXTMISCA1_INT/ DAI0_INT_07	DAI1_EXTMISCA1_INT/ DAI1_INT_07	DAI0 or DAI1 Miscellaneous Interrupt 7 / External Miscellaneous A1 Interrupt
MISCINT8	DAI0_EXTMISCA2_INT/ DAI0_INT_08	DAI1_EXTMISCA2_INT/ DAI1_INT_08	DAI0 or DAI1 Miscellaneous Interrupt 8 / External Miscellaneous A2 Interrupt
MISCINT9	DAI0_EXTMISCA3_INT/ DAI0_INT_09	DAI1_EXTMISCA3_INT/ DAI1_INT_09	DAI0 or DAI1 Miscellaneous Interrupt 9 / External Miscellaneous A3 Interrupt

SPORT Grouping with DAI

DAI supports two registers [DAI_GBL_SP_EN](#) and [DAI_GBL_INT_EN](#) to group enabling/disabling and interrupt/trigger generation/reception for multiple SPORTs.

For more information about this, refer to the [Grouping of SPORTs](#) section.

ADSP-21560_21561_21564_21568 DAI Register Descriptions

The Digital Audio Interface (DAI_n) registers are used to configure DAI destinations for the DAI sources shown in the Group A - F Routing tables. (DAI) contains the following registers.

Table 20-29: ADSP-21560_21561_21564_21568 DAI Register List

Name	Description
DAI_CLK0	Clock Routing Control Register 0
DAI_CLK1	Clock Routing Control Register 1
DAI_CLK2	Clock Routing Control Register 2
DAI_CLK3	Clock Routing Control Register 3
DAI_CLK4	Clock Routing Control Register 4
DAI_CLK5	Clock Routing Control Register 5
DAI_DAT0	Serial Data Routing Control Register 0
DAI_DAT1	Serial Data Routing Control Register 1
DAI_DAT2	Serial Data Routing Control Register 2
DAI_DAT3	Serial Data Routing Control Register 3
DAI_DAT4	Serial Data Routing Control Register 4
DAI_DAT5	Serial Data Routing Control Register 5
DAI_DAT6	Serial Data Routing Control Register 6
DAI_FS0	Frame Sync Routing Control Register 0
DAI_FS1	Frame Sync Routing Control Register 1
DAI_FS2	Frame Sync Routing Control Register 2
DAI_FS4	Frame Sync Routing Control Register 4
DAI_GBL_INT_EN	Global SPORT Interrupt Grouping Register
DAI_GBL_SP_EN	Global SPORT Enable Register
DAI_IMSK_FE	Falling-Edge Interrupt Mask Register
DAI_IMSK_PRI	Core Interrupt Priority Assignment Register
DAI_IMSK_RE	Rising-Edge Interrupt Mask Register
DAI_IRPTL_H	High Priority Interrupt Latch Register
DAI_IRPTL_HS	Shadow High Priority Interrupt Latch Register
DAI_IRPTL_L	Low Priority Interrupt Latch Register
DAI_IRPTL_LS	Shadow Low Priority Interrupt Latch Register
DAI_MISC0	Miscellaneous Control Register 0
DAI_MISC1	Miscellaneous Control Register 1
DAI_PBEN0	Pin Buffer Enable Register 0
DAI_PBEN1	Pin Buffer Enable Register 1
DAI_PBEN2	Pin Buffer Enable Register 2

Table 20-29: ADSP-21560_21561_21564_21568 DAI Register List (Continued)

Name	Description
DAI_PBEN3	Pin Buffer Enable Register 3
DAI_PIN0	Pin Buffer Assignment Register 0
DAI_PIN1	Pin Buffer Assignment Register 1
DAI_PIN2	Pin Buffer Assignment Register 2
DAI_PIN4	Pin Buffer Assignment Register 4
DAI_PIN_STAT	Pin Status Register

Clock Routing Control Register 0

The `DAI_CLK0` register provides clock routing connections for the serial ports (SPORTs).

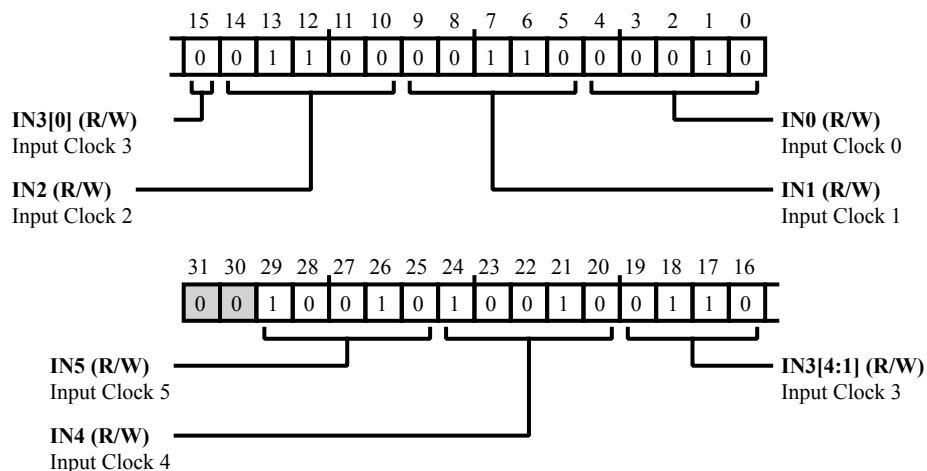


Figure 20-16: DAI_CLK0 Register Diagram

Table 20-30: DAI_CLK0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:25 (R/W)	IN5	Input Clock 5. DAI_CLK0 . IN5 holds the source signal assignment that will be routed to the DAI_CLK0 . IN5 destination. Refer to the Group A Signals table for source and destination mappings.
24:20 (R/W)	IN4	Input Clock 4. DAI_CLK0 . IN4 holds the source signal assignment that will be routed to the DAI_CLK0 . IN4 destination. Refer to the Group A Signals table for source and destination mappings.
19:15 (R/W)	IN3	Input Clock 3. DAI_CLK0 . IN3 holds the source signal assignment that will be routed to the DAI_CLK0 . IN3 destination. Refer to the Group A Signals table for source and destination mappings.
14:10 (R/W)	IN2	Input Clock 2. DAI_CLK0 . IN2 holds the source signal assignment that will be routed to the DAI_CLK0 . IN2 destination. Refer to the Group A Signals table for source and destination mappings.
9:5 (R/W)	IN1	Input Clock 1. DAI_CLK0 . IN1 holds the source signal assignment that will be routed to the DAI_CLK0 . IN1 destination. Refer to the Group A Signals table for source and destination mappings.

Table 20-30: DAI_CLK0 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
4:0 (R/W)	IN0	Input Clock 0. DAI_CLK0.IN0 holds the source signal assignment that will be routed to the DAI_CLK0.IN0 destination. Refer to the Group A Signals table for source and destination mappings.

Clock Routing Control Register 1

The `DAI_CLK1` register provides clock routing connections for the asynchronous sample rate converters (ASRC).

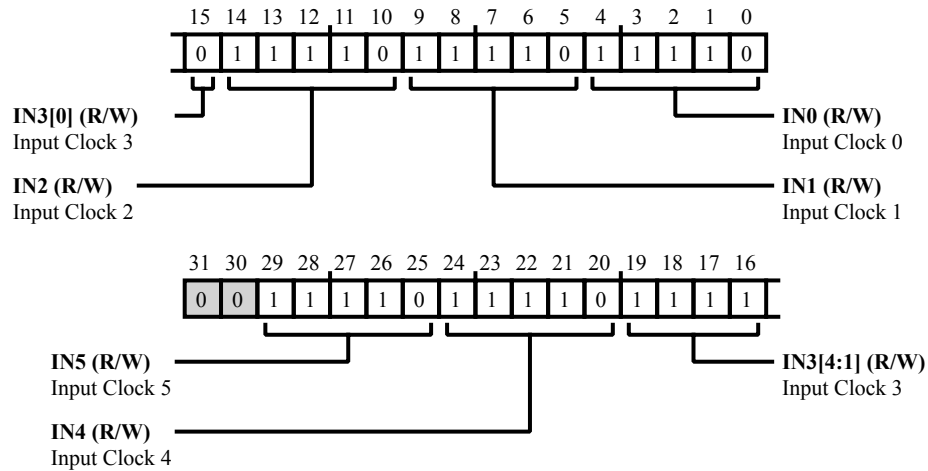


Figure 20-17: DAI_CLK1 Register Diagram

Table 20-31: DAI_CLK1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:25 (R/W)	IN5	Input Clock 5. DAI_CLK1 . IN5 holds the source signal assignment that will be routed to the DAI_CLK1 . IN5 destination. Refer to the Group A Signals table for source and destination mappings.
24:20 (R/W)	IN4	Input Clock 4. DAI_CLK1 . IN4 holds the source signal assignment that will be routed to the DAI_CLK1 . IN4 destination. Refer to the Group A Signals table for source and destination mappings.
19:15 (R/W)	IN3	Input Clock 3. DAI_CLK1 . IN3 holds the source signal assignment that will be routed to the DAI_CLK1 . IN3 destination. Refer to the Group A Signals table for source and destination mappings.
14:10 (R/W)	IN2	Input Clock 2. DAI_CLK1 . IN2 holds the source signal assignment that will be routed to the DAI_CLK1 . IN2 destination. Refer to the Group A Signals table for source and destination mappings.
9:5 (R/W)	IN1	Input Clock 1. DAI_CLK1 . IN1 holds the source signal assignment that will be routed to the DAI_CLK1 . IN1 destination. Refer to the Group A Signals table for source and destination mappings.

Table 20-31: DAI_CLK1 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
4:0 (R/W)	IN0	Input Clock 0. DAI_CLK1 . IN0 holds the source signal assignment that will be routed to the DAI_CLK1 . IN0 destination. Refer to the Group A Signals table for source and destination mappings.

Clock Routing Control Register 2

The `DAI_CLK2` register provides clock routing connections for the S/PDIF and ASRC.

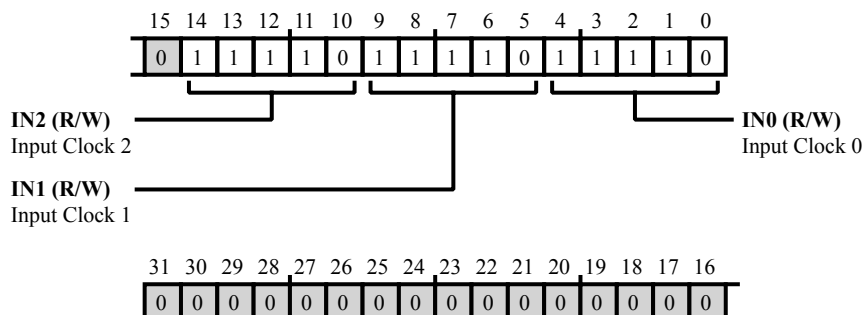


Figure 20-18: DAI_CLK2 Register Diagram

Table 20-32: DAI_CLK2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
14:10 (R/W)	IN2	Input Clock 2. DAI_CLK2 . IN2 holds the source signal assignment that will be routed to the DAI_CLK2 . IN2 destination. Refer to the Group A Signals table for source and destination mappings.
9:5 (R/W)	IN1	Input Clock 1. DAI_CLK2 . IN1 holds the source signal assignment that will be routed to the DAI_CLK2 . IN1 destination. Refer to the Group A Signals table for source and destination mappings.
4:0 (R/W)	IN0	Input Clock 0. DAI_CLK2 . IN0 holds the source signal assignment that will be routed to the DAI_CLK2 . IN0 destination. Refer to the Group A Signals table for source and destination mappings.

Clock Routing Control Register 3

The `DAI_CLK3` register provides clock routing connections for the S/PDIF.

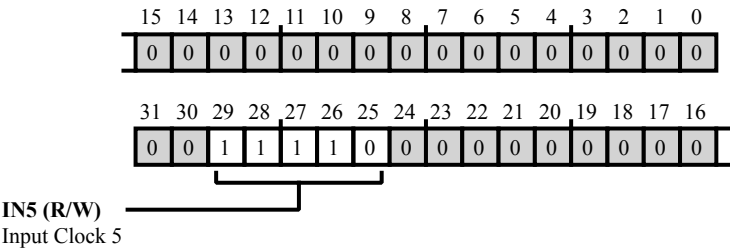


Figure 20-19: DAI_CLK3 Register Diagram

Table 20-33: DAI_CLK3 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:25 (R/W)	IN5	Input Clock 5. The <code>DAI_CLK3</code> . IN5 bit field provides the S/PDIF oversampling clock.

Clock Routing Control Register 4

The `DAI_CLK4` register provides clock routing connections for the precision clock generators (PCGs).

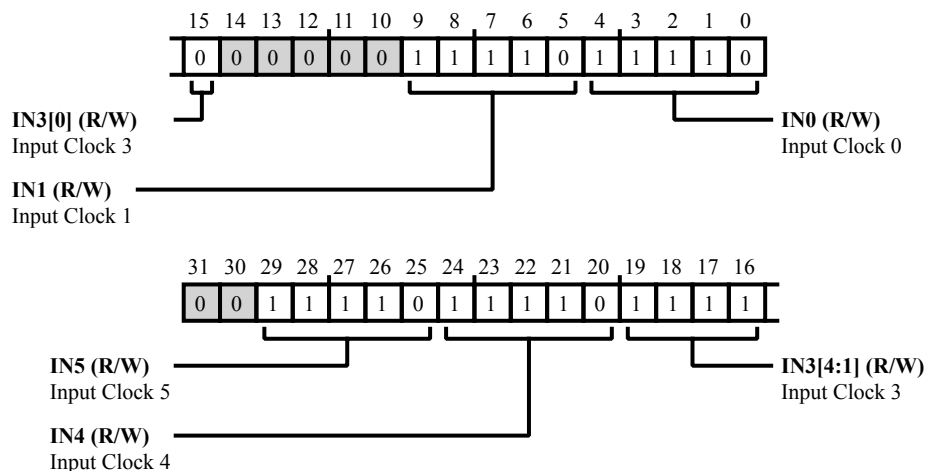


Figure 20-20: DAI_CLK4 Register Diagram

Table 20-34: DAI_CLK4 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:25 (R/W)	IN5	Input Clock 5. DAI_CLK4 . IN5 holds the source signal assignment that will be routed to the DAI_CLK4 . IN5 destination. Refer to the Group A Signals table for source and destination mappings.
24:20 (R/W)	IN4	Input Clock 4. DAI_CLK4 . IN4 holds the source signal assignment that will be routed to the DAI_CLK4 . IN4 destination. Refer to the Group A Signals table for source and destination mappings.
19:15 (R/W)	IN3	Input Clock 3. DAI_CLK4 . IN3 holds the source signal assignment that will be routed to the DAI_CLK4 . IN3 destination. Refer to the Group A Signals table for source and destination mappings.
9:5 (R/W)	IN1	Input Clock 1. DAI_CLK4 . IN1 holds the source signal assignment that will be routed to the DAI_CLK4 . IN1 destination. Refer to the Group A Signals table for source and destination mappings.
4:0 (R/W)	IN0	Input Clock 0. DAI_CLK4 . IN0 holds the source signal assignment that will be routed to the DAI_CLK4 . IN0 destination. Refer to the Group A Signals table for source and destination mappings.

Clock Routing Control Register 5

The `DAI_CLK5` register provides clock routing connections for the serial ports.

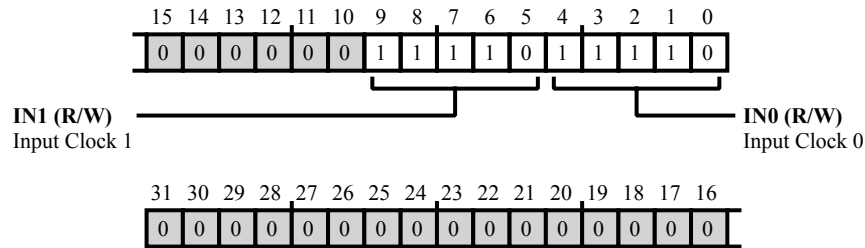


Figure 20-21: DAI_CLK5 Register Diagram

Table 20-35: DAI_CLK5 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
9:5 (R/W)	IN1	Input Clock 1. DAI_CLK5.IN1 holds the source signal assignment that will be routed to the DAI_CLK5.IN1 destination. Refer to the Group A Signals table for source and destination mappings.
4:0 (R/W)	IN0	Input Clock 0. DAI_CLK5.IN0 holds the source signal assignment that will be routed to the DAI_CLK5.IN0 destination. Refer to the Group A Signals table for source and destination mappings.

Serial Data Routing Control Register 0

The `DAI_DAT0` register routes serial data to the serial ports.

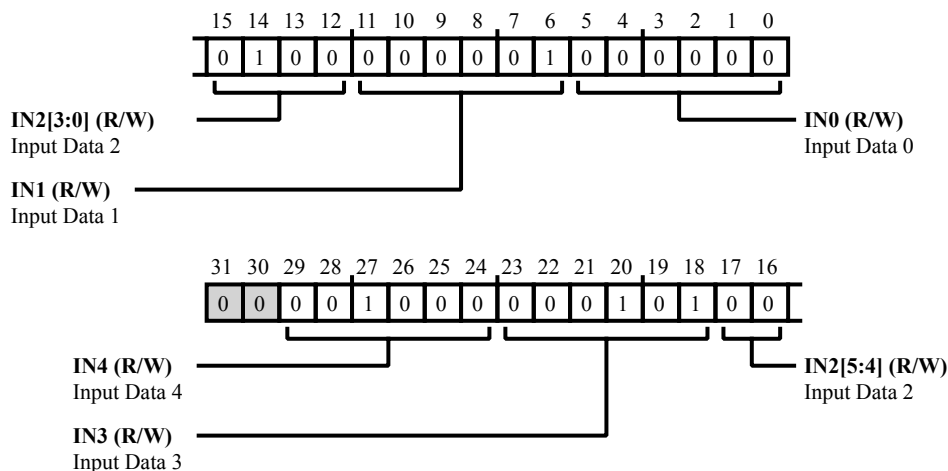


Figure 20-22: DAI_DAT0 Register Diagram

Table 20-36: DAI_DAT0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:24 (R/W)	IN4	Input Data 4. DAI_DAT0 . IN4 holds the source signal assignment that will be routed to the DAI_DAT0 . IN4 destination. Refer to the Group B Signals table for source and destination mappings.
23:18 (R/W)	IN3	Input Data 3. DAI_DAT0 . IN3 holds the source signal assignment that will be routed to the DAI_DAT0 . IN3 destination. Refer to the Group B Signals table for source and destination mappings.
17:12 (R/W)	IN2	Input Data 2. DAI_DAT0 . IN2 holds the source signal assignment that will be routed to the DAI_DAT0 . IN2 destination. Refer to the Group B Signals table for source and destination mappings.
11:6 (R/W)	IN1	Input Data 1. DAI_DAT0 . IN1 holds the source signal assignment that will be routed to the DAI_DAT0 . IN1 destination. Refer to the Group B Signals table for source and destination mappings.
5:0 (R/W)	IN0	Input Data 0. DAI_DAT0 . IN0 holds the source signal assignment that will be routed to the DAI_DAT0 . IN0 destination. Refer to the Group B Signals table for source and destination mappings.

Serial Data Routing Control Register 1

The `DAI_DAT1` register routes serial data to the serial ports.

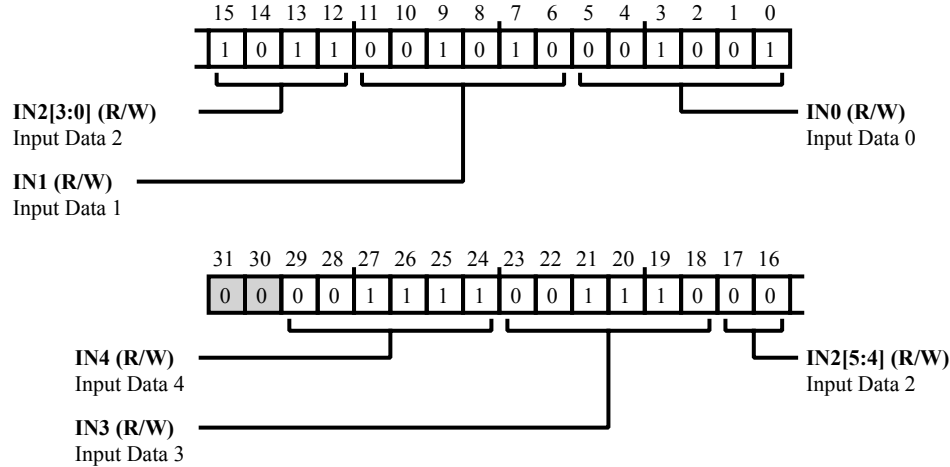


Figure 20-23: DAI_DAT1 Register Diagram

Table 20-37: DAI_DAT1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:24 (R/W)	IN4	Input Data 4. DAI_DAT1 . IN4 holds the source signal assignment that will be routed to the DAI_DAT1 . IN4 destination. Refer to the Group B Signals table for source and destination mappings.
23:18 (R/W)	IN3	Input Data 3. DAI_DAT1 . IN3 holds the source signal assignment that will be routed to the DAI_DAT1 . IN3 destination. Refer to the Group B Signals table for source and destination mappings.
17:12 (R/W)	IN2	Input Data 2. DAI_DAT1 . IN2 holds the source signal assignment that will be routed to the DAI_DAT1 . IN2 destination. Refer to the Group B Signals table for source and destination mappings.
11:6 (R/W)	IN1	Input Data 1. DAI_DAT1 . IN1 holds the source signal assignment that will be routed to the DAI_DAT1 . IN1 destination. Refer to the Group B Signals table for source and destination mappings.
5:0 (R/W)	IN0	Input Data 0. DAI_DAT1 . IN0 holds the source signal assignment that will be routed to the DAI_DAT1 . IN0 destination. Refer to the Group B Signals table for source and destination mappings.

Serial Data Routing Control Register 2

The `DAI_DAT2` register routes serial data to the serial ports and the asynchronous sample rate converter (ASRC).

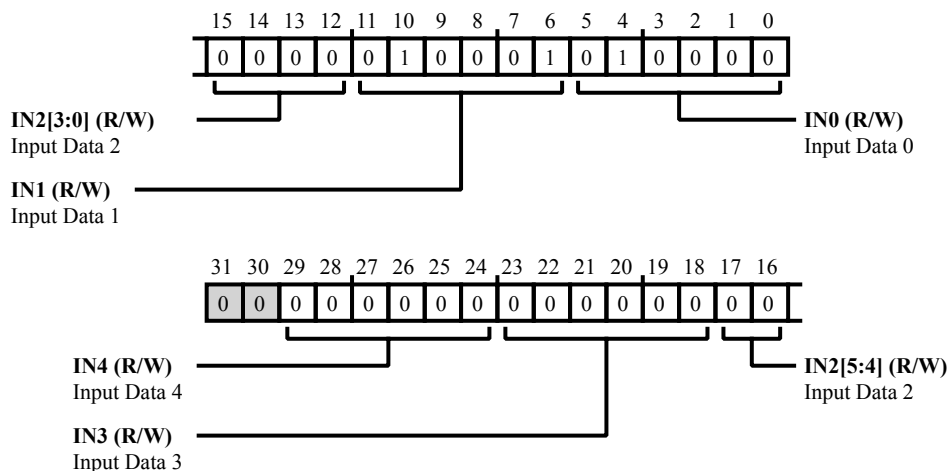


Figure 20-24: DAI_DAT2 Register Diagram

Table 20-38: DAI_DAT2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:24 (R/W)	IN4	Input Data 4. DAI_DAT2 . IN4 holds the source signal assignment that will be routed to the DAI_DAT2 . IN4 destination. Refer to the Group B Signals table for source and destination mappings.
23:18 (R/W)	IN3	Input Data 3. DAI_DAT2 . IN3 holds the source signal assignment that will be routed to the DAI_DAT2 . IN3 destination. Refer to the Group B Signals table for source and destination mappings.
17:12 (R/W)	IN2	Input Data 2. DAI_DAT2 . IN2 holds the source signal assignment that will be routed to the DAI_DAT2 . IN2 destination. Refer to the Group B Signals table for source and destination mappings.
11:6 (R/W)	IN1	Input Data 1. DAI_DAT2 . IN1 holds the source signal assignment that will be routed to the DAI_DAT2 . IN1 destination. Refer to the Group B Signals table for source and destination mappings.
5:0 (R/W)	IN0	Input Data 0. DAI_DAT2 . IN0 holds the source signal assignment that will be routed to the DAI_DAT2 . IN0 destination. Refer to the Group B Signals table for source and destination mappings.

Serial Data Routing Control Register 3

The `DAI_DAT3` register routes serial data to the asynchronous sample rate converter (ASRC).

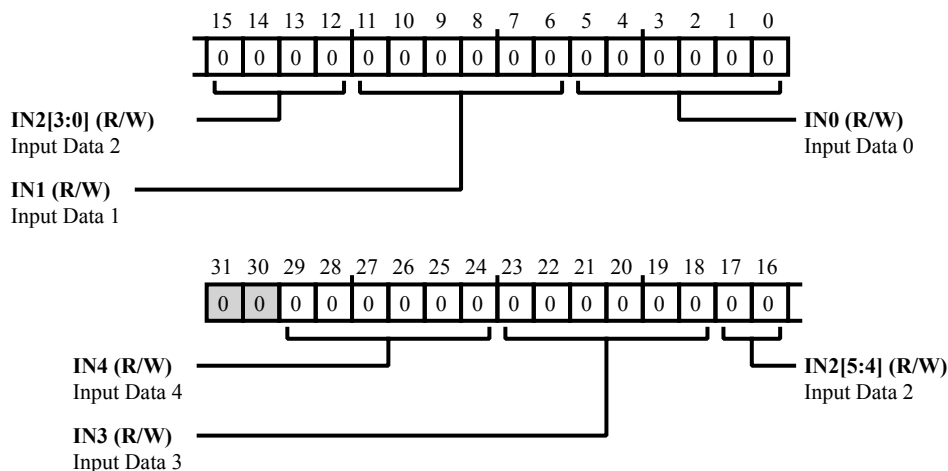


Figure 20-25: DAI_DAT3 Register Diagram

Table 20-39: DAI_DAT3 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:24 (R/W)	IN4	Input Data 4. DAI_DAT3 . IN4 holds the source signal assignment that will be routed to the DAI_DAT3 . IN4 destination. Refer to the Group B Signals table for source and destination mappings.
23:18 (R/W)	IN3	Input Data 3. DAI_DAT3 . IN3 holds the source signal assignment that will be routed to the DAI_DAT3 . IN3 destination. Refer to the Group B Signals table for source and destination mappings.
17:12 (R/W)	IN2	Input Data 2. DAI_DAT3 . IN2 holds the source signal assignment that will be routed to the DAI_DAT3 . IN2 destination. Refer to the Group B Signals table for source and destination mappings.
11:6 (R/W)	IN1	Input Data 1. DAI_DAT3 . IN1 holds the source signal assignment that will be routed to the DAI_DAT3 . IN1 destination. Refer to the Group B Signals table for source and destination mappings.
5:0 (R/W)	IN0	Input Data 0. DAI_DAT3 . IN0 holds the source signal assignment that will be routed to the DAI_DAT3 . IN0 destination. Refer to the Group B Signals table for source and destination mappings.

Serial Data Routing Control Register 4

The `DAI_DAT4` register routes serial data to the S/PDIF.

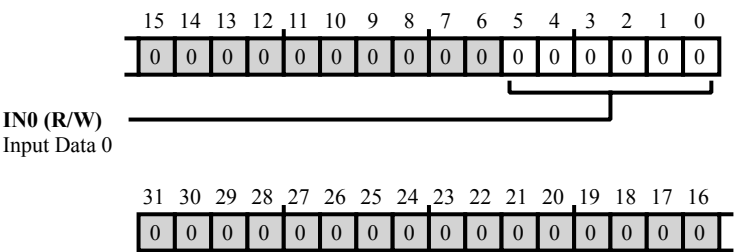


Figure 20-26: `DAI_DAT4` Register Diagram

Table 20-40: `DAI_DAT4` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
5:0 (R/W)	IN0	Input Data 0. <code>DAI_DAT4</code> . IN0 holds the source signal assignment that will be routed to the <code>DAI_DAT4</code> . IN0 destination. Refer to the Group B Signals table for source and destination mappings.

Serial Data Routing Control Register 5

The `DAI_DAT5` register routes serial data to the S/PDIF.

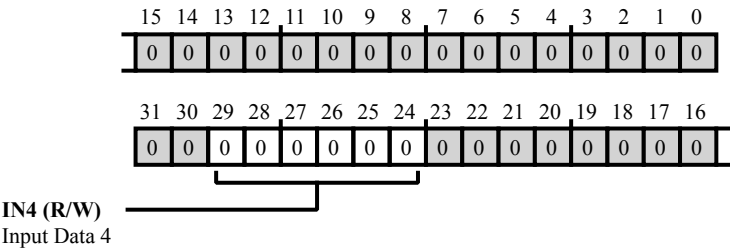


Figure 20-27: `DAI_DAT5` Register Diagram

Table 20-41: `DAI_DAT5` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:24 (R/W)	IN4	Input Data 4. The <code>DAI_DAT5</code> . IN4 bit field routes input data to the S/PDIF Biphase receiver stream.

Serial Data Routing Control Register 6

The `DAI_DAT6` register routes serial data to the serial ports.

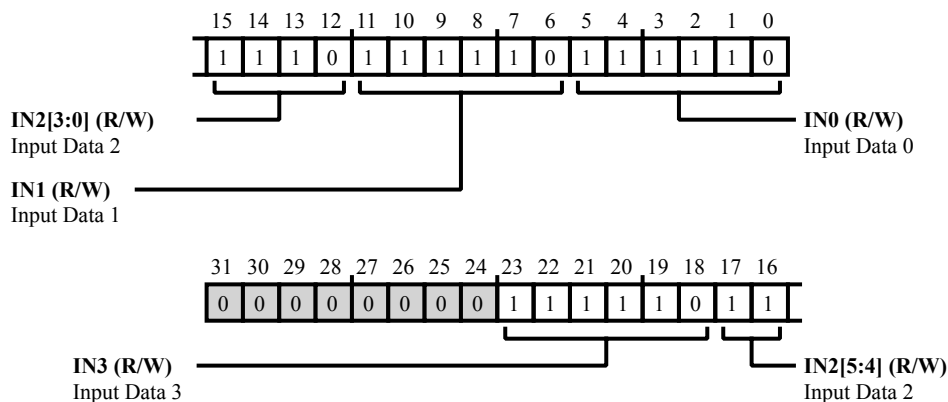


Figure 20-28: DAI_DAT6 Register Diagram

Table 20-42: DAI_DAT6 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
23:18 (R/W)	IN3	Input Data 3. DAI_DAT6 . IN3 holds the source signal assignment that will be routed to the DAI_DAT6 . IN3 destination. Refer to the Group B Signals table for source and destination mappings.
17:12 (R/W)	IN2	Input Data 2. DAI_DAT6 . IN2 holds the source signal assignment that will be routed to the DAI_DAT6 . IN2 destination. Refer to the Group B Signals table for source and destination mappings.
11:6 (R/W)	IN1	Input Data 1. DAI_DAT6 . IN1 holds the source signal assignment that will be routed to the DAI_DAT6 . IN1 destination. Refer to the Group B Signals table for source and destination mappings.
5:0 (R/W)	IN0	Input Data 0. DAI_DAT6 . IN0 holds the source signal assignment that will be routed to the DAI_DAT6 . IN0 destination. Refer to the Group B Signals table for source and destination mappings.

Frame Sync Routing Control Register 0

The `DAI_FS0` register routes frame syncs to the serial ports.

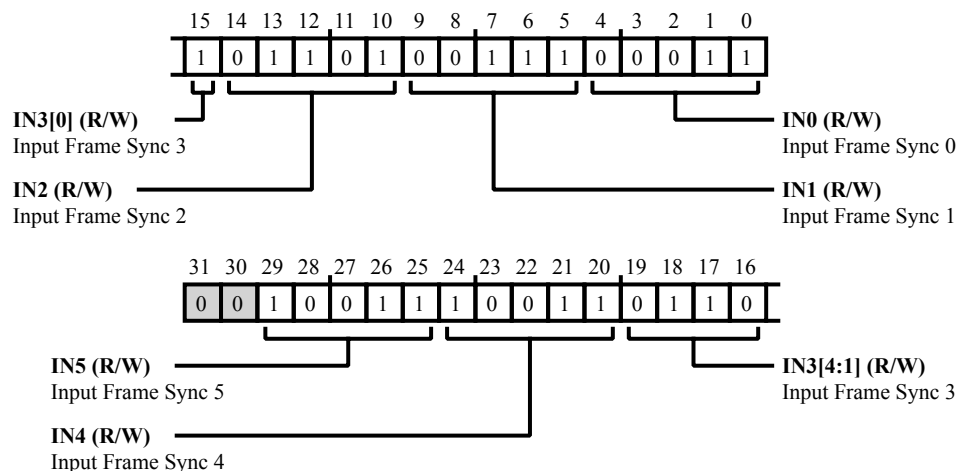


Figure 20-29: DAI_FS0 Register Diagram

Table 20-43: DAI_FS0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:25 (R/W)	IN5	Input Frame Sync 5. DAI_FS0 . IN5 holds the source signal assignment that will be routed to the DAI_FS0 . IN5 destination. Refer to the Group C Signals table for source and destination mappings.
24:20 (R/W)	IN4	Input Frame Sync 4. DAI_FS0 . IN4 holds the source signal assignment that will be routed to the DAI_FS0 . IN4 destination. Refer to the Group C Signals table for source and destination mappings.
19:15 (R/W)	IN3	Input Frame Sync 3. DAI_FS0 . IN3 holds the source signal assignment that will be routed to the DAI_FS0 . IN3 destination. Refer to the Group C Signals table for source and destination mappings.
14:10 (R/W)	IN2	Input Frame Sync 2. DAI_FS0 . IN2 holds the source signal assignment that will be routed to the DAI_FS0 . IN2 destination. Refer to the Group C Signals table for source and destination mappings.
9:5 (R/W)	IN1	Input Frame Sync 1. DAI_FS0 . IN1 holds the source signal assignment that will be routed to the DAI_FS0 . IN1 destination. Refer to the Group C Signals table for source and destination mappings.

Table 20-43: DAI_FS0 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
4:0 (R/W)	IN0	Input Frame Sync 0. DAI_FS0.IN0 holds the source signal assignment that will be routed to the DAI_FS0.IN0 destination. Refer to the Group C Signals table for source and destination mappings.

Frame Sync Routing Control Register 1

The `DAI_FS1` register routes frame syncs to the asynchronous sample rate converter (ASRC).

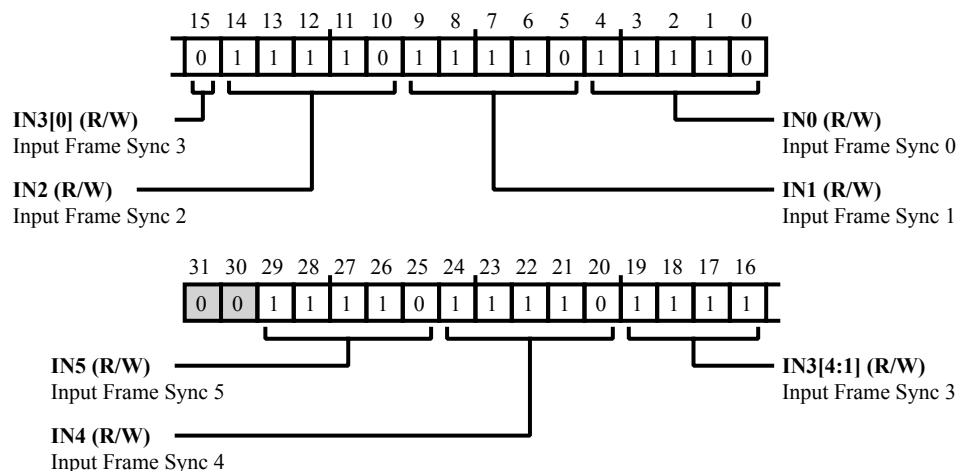


Figure 20-30: DAI_FS1 Register Diagram

Table 20-44: DAI_FS1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:25 (R/W)	IN5	Input Frame Sync 5. DAI_FS1 . IN5 holds the source signal assignment that will be routed to the DAI_FS1 . IN5 destination. Refer to the Group C Signals table for source and destination mappings.
24:20 (R/W)	IN4	Input Frame Sync 4. DAI_FS1 . IN4 holds the source signal assignment that will be routed to the DAI_FS1 . IN4 destination. Refer to the Group C Signals table for source and destination mappings.
19:15 (R/W)	IN3	Input Frame Sync 3. DAI_FS1 . IN3 holds the source signal assignment that will be routed to the DAI_FS1 . IN3 destination. Refer to the Group C Signals table for source and destination mappings.
14:10 (R/W)	IN2	Input Frame Sync 2. DAI_FS1 . IN2 holds the source signal assignment that will be routed to the DAI_FS1 . IN2 destination. Refer to the Group C Signals table for source and destination mappings.
9:5 (R/W)	IN1	Input Frame Sync 1. DAI_FS1 . IN1 holds the source signal assignment that will be routed to the DAI_FS1 . IN1 destination. Refer to the Group C Signals table for source and destination mappings.

Table 20-44: DAI_FS1 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
4:0 (R/W)	IN0	Input Frame Sync 0. DAI_FS1 . IN0 holds the source signal assignment that will be routed to the DAI_FS1 . IN0 destination. Refer to the Group C Signals table for source and destination mappings.

Frame Sync Routing Control Register 2

The `DAI_FS2` register routes frame syncs to the asynchronous sample rate converter (ASRC).

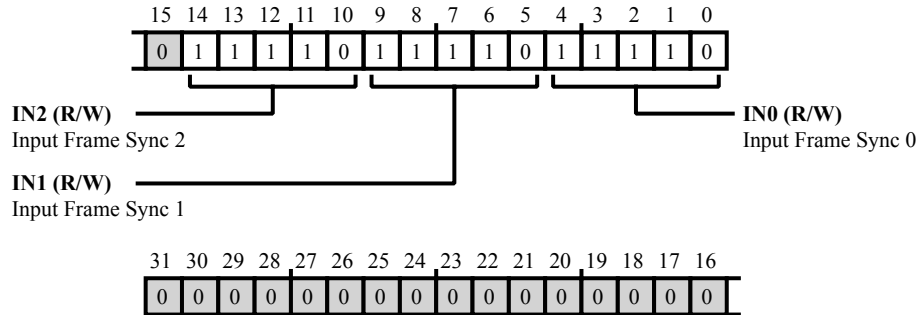


Figure 20-31: DAI_FS2 Register Diagram

Table 20-45: DAI_FS2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
14:10 (R/W)	IN2	Input Frame Sync 2. DAI_FS2 . IN2 holds the source signal assignment that will be routed to the DAI_FS2 . IN2 destination. Refer to the Group C Signals table for source and destination mappings.
9:5 (R/W)	IN1	Input Frame Sync 1. The DAI_FS2 . IN1 bit field routes the frame sync output to the ASRC3 frame sync input port.
4:0 (R/W)	IN0	Input Frame Sync 0. DAI_FS2 . IN0 holds the source signal assignment that will be routed to the DAI_FS2 . IN0 destination. Refer to the Group C Signals table for source and destination mappings.

Frame Sync Routing Control Register 4

The `DAI_FS4` register routes frame syncs to the asynchronous sample rate converter (ASRC).

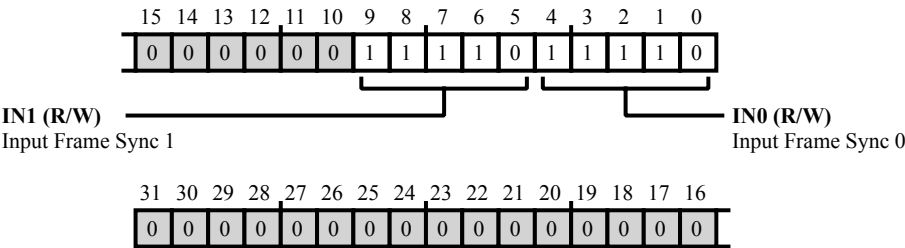


Figure 20-32: DAI_FS4 Register Diagram

Table 20-46: DAI_FS4 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
9:5 (R/W)	IN1	Input Frame Sync 1. DAI_FS4 . IN1 holds the source signal assignment that will be routed to the DAI_FS4 . IN1 destination. Refer to the Group C Signals table for source and destination mappings.
4:0 (R/W)	IN0	Input Frame Sync 0. DAI_FS4 . IN0 holds the source signal assignment that will be routed to the DAI_FS4 . IN0 destination. Refer to the Group C Signals table for source and destination mappings.

Global SPORT Interrupt Grouping Register

The `DAI_GBL_INT_EN` register contains bits to configure the group interrupts and triggers of the SPORTs when using the SPORT global enable feature.

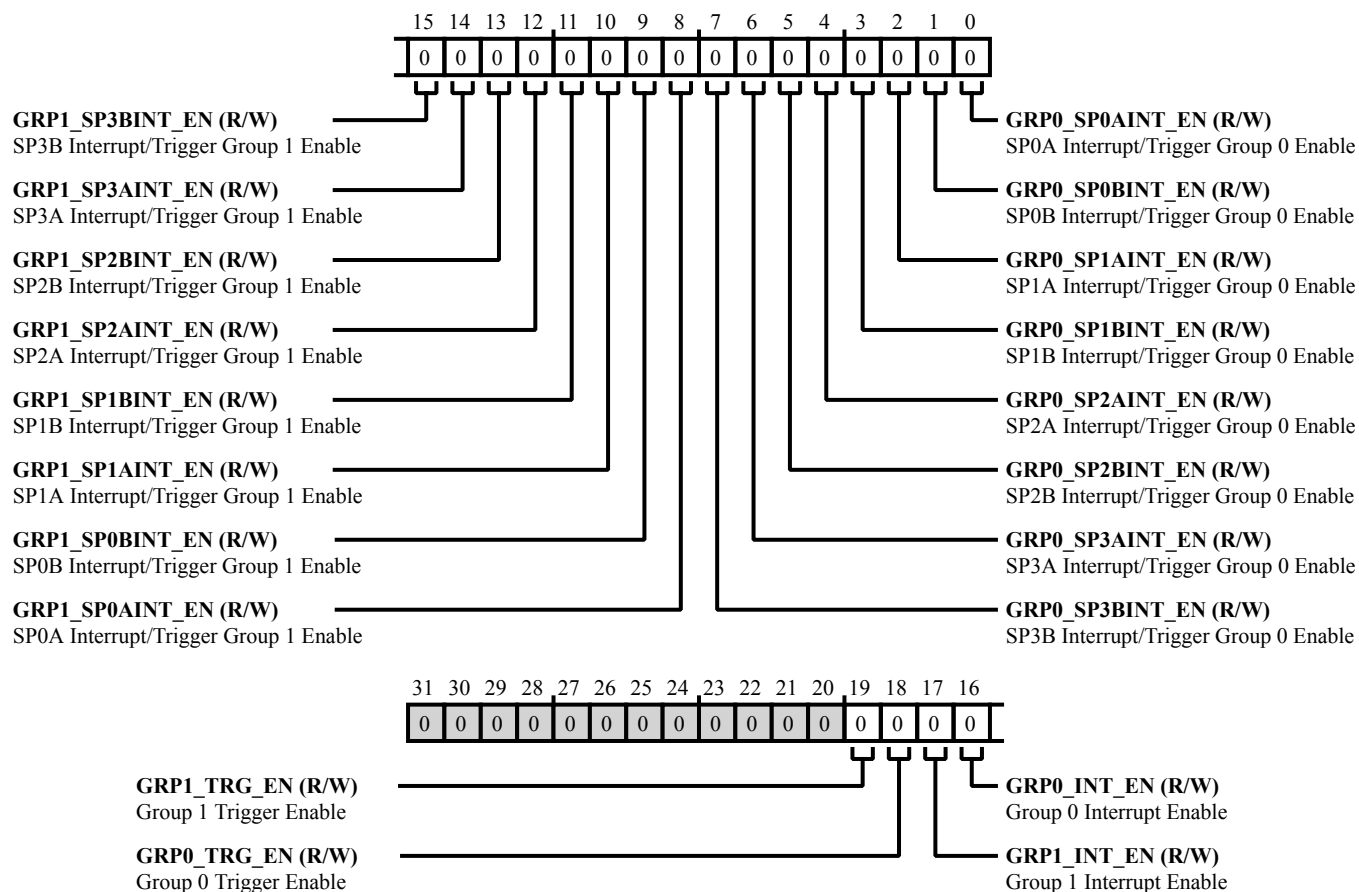


Figure 20-33: `DAI_GBL_INT_EN` Register Diagram

Table 20-47: `DAI_GBL_INT_EN` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
19 (R/W)	GRP1_TRG_EN	Group 1 Trigger Enable. The <code>DAI_GBL_INT_EN.GRP1_TRG_EN</code> bit enables the trigger-out for group 1.
18 (R/W)	GRP0_TRG_EN	Group 0 Trigger Enable. The <code>DAI_GBL_INT_EN.GRP0_TRG_EN</code> bit enables the trigger-out for group 0.
17 (R/W)	GRP1_INT_EN	Group 1 Interrupt Enable. The <code>DAI_GBL_INT_EN.GRP1_INT_EN</code> bit enables the interrupt for group 1.

Table 20-47: DAI_GBL_INT_EN Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
16 (R/W)	GRP0_INT_EN	Group 0 Interrupt Enable. The DAI_GBL_INT_EN.GRP0_INT_EN bit enables the interrupt for group 0.
15 (R/W)	GRP1_SP3BINT_EN	SP3B Interrupt/Trigger Group 1 Enable. The DAI_GBL_INT_EN.GRP1_SP3BINT_EN bit enables the interrupt or trigger-out for SPORT3 B when it is part of group 1.
14 (R/W)	GRP1_SP3AINT_EN	SP3A Interrupt/Trigger Group 1 Enable. The DAI_GBL_INT_EN.GRP1_SP3AINT_EN bit enables the interrupt or trigger-out for SPORT3 A when it is part of group 1.
13 (R/W)	GRP1_SP2BINT_EN	SP2B Interrupt/Trigger Group 1 Enable. The DAI_GBL_INT_EN.GRP1_SP2BINT_EN bit enables the interrupt or trigger-out for SPORT2 B when it is part of group 1.
12 (R/W)	GRP1_SP2AINT_EN	SP2A Interrupt/Trigger Group 1 Enable. The DAI_GBL_INT_EN.GRP1_SP2AINT_EN bit enables the interrupt or trigger-out for SPORT2 A when it is part of group 1.
11 (R/W)	GRP1_SP1BINT_EN	SP1B Interrupt/Trigger Group 1 Enable. The DAI_GBL_INT_EN.GRP1_SP1BINT_EN bit enables the interrupt or trigger-out for SPORT1 B when it is part of group 1.
10 (R/W)	GRP1_SP1AINT_EN	SP1A Interrupt/Trigger Group 1 Enable. The DAI_GBL_INT_EN.GRP1_SP1AINT_EN bit enables the interrupt or trigger-out for SPORT1 A when it is part of group 1.
9 (R/W)	GRP1_SP0BINT_EN	SP0B Interrupt/Trigger Group 1 Enable. The DAI_GBL_INT_EN.GRP1_SP0BINT_EN bit enables the interrupt or trigger-out for SPORT0 B when it is part of group 1.
8 (R/W)	GRP1_SP0AINT_EN	SP0A Interrupt/Trigger Group 1 Enable. The DAI_GBL_INT_EN.GRP1_SP0AINT_EN bit enables the interrupt or trigger-out for SPORT0 A when it is part of group 1.
7 (R/W)	GRP0_SP3BINT_EN	SP3B Interrupt/Trigger Group 0 Enable. The DAI_GBL_INT_EN.GRP0_SP3BINT_EN bit enables the interrupt or trigger-out for SPORT3 B when it is part of group 0.
6 (R/W)	GRP0_SP3AINT_EN	SP3A Interrupt/Trigger Group 0 Enable. The DAI_GBL_INT_EN.GRP0_SP3AINT_EN bit enables the interrupt or trigger-out for SPORT3 A when it is part of group 0.
5 (R/W)	GRP0_SP2BINT_EN	SP2B Interrupt/Trigger Group 0 Enable. The DAI_GBL_INT_EN.GRP0_SP2BINT_EN bit enables the interrupt or trigger-out for SPORT2 B when it is part of group 0.

Table 20-47: DAI_GBL_INT_EN Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
4 (R/W)	GRP0_SP2AINT_EN	SP2A Interrupt/Trigger Group 0 Enable. The DAI_GBL_INT_EN.GRP0_SP2AINT_EN bit enables the interrupt or trigger-out for SPORT2 A when it is part of group 0.
3 (R/W)	GRP0_SP1BINT_EN	SP1B Interrupt/Trigger Group 0 Enable. The DAI_GBL_INT_EN.GRP0_SP1BINT_EN bit enables the interrupt or trigger-out for SPORT1 B when it is part of group 0.
2 (R/W)	GRP0_SP1AINT_EN	SP1A Interrupt/Trigger Group 0 Enable. The DAI_GBL_INT_EN.GRP0_SP1AINT_EN bit enables the interrupt or trigger-out for SPORT1 A when it is part of group 0.
1 (R/W)	GRP0_SP0BINT_EN	SP0B Interrupt/Trigger Group 0 Enable. The DAI_GBL_INT_EN.GRP0_SP0BINT_EN bit enables the interrupt or trigger-out for SPORT0 B when it is part of group 0.
0 (R/W)	GRP0_SP0AINT_EN	SP0A Interrupt/Trigger Group 0 Enable. The DAI_GBL_INT_EN.GRP0_SP0AINT_EN bit enables the interrupt or trigger-out for SPORT0 A when it is part of group 0.

Global SPORT Enable Register

The `DAI_GBL_SP_EN` register indicate the status (signal high or low) for each pin. The individual bit reads 1 if the signal to this pin is high and reads 0 if signal to this pin is low.

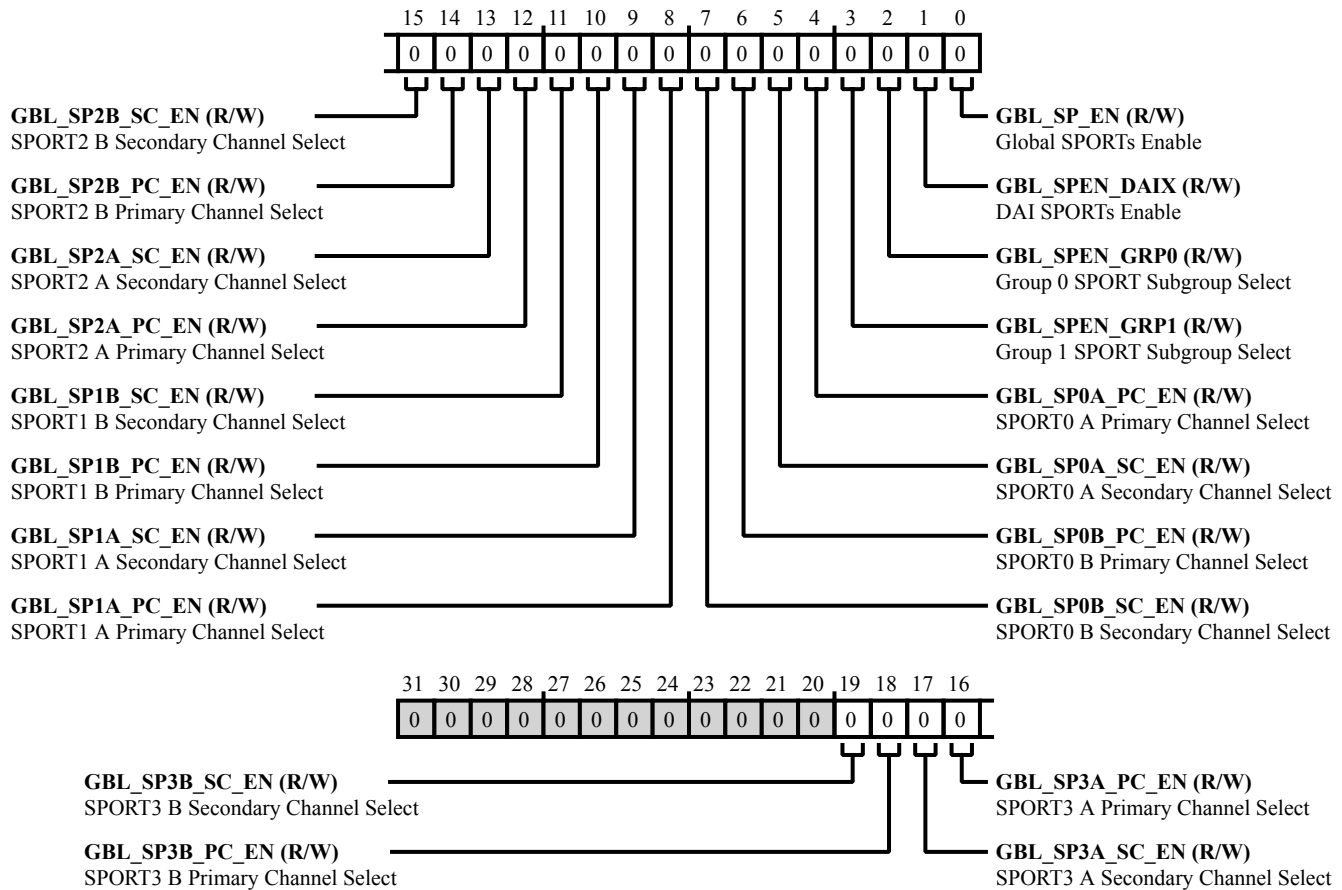


Figure 20-34: `DAI_GBL_SP_EN` Register Diagram

Table 20-48: `DAI_GBL_SP_EN` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
19 (R/W)	<code>GBL_SP3B_SC_EN</code>	SPORT3 B Secondary Channel Select. The <code>DAI_GBL_SP_EN.GBL_SP3B_SC_EN</code> bit selects SPORT3 B as the secondary channel for global enable.
18 (R/W)	<code>GBL_SP3B_PC_EN</code>	SPORT3 B Primary Channel Select. The <code>DAI_GBL_SP_EN.GBL_SP3B_PC_EN</code> bit selects SPORT3 B as the primary channel for global enable.

Table 20-48: DAI_GBL_SP_EN Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
17 (R/W)	GBL_SP3A_SC_EN	SPORT3 A Secondary Channel Select. The DAI_GBL_SP_EN.GBL_SP3A_SC_EN bit selects SPORT3 A as the secondary channel for global enable.
16 (R/W)	GBL_SP3A_PC_EN	SPORT3 A Primary Channel Select. The DAI_GBL_SP_EN.GBL_SP3A_PC_EN bit selects SPORT3 A as the primary channel for global enable.
15 (R/W)	GBL_SP2B_SC_EN	SPORT2 B Secondary Channel Select. The DAI_GBL_SP_EN.GBL_SP2B_SC_EN bit selects SPORT2 B as the secondary channel for global enable.
14 (R/W)	GBL_SP2B_PC_EN	SPORT2 B Primary Channel Select. The DAI_GBL_SP_EN.GBL_SP2B_PC_EN bit selects SPORT2 B as the primary channel for global enable.
13 (R/W)	GBL_SP2A_SC_EN	SPORT2 A Secondary Channel Select. The DAI_GBL_SP_EN.GBL_SP2A_SC_EN bit selects SPORT2 A as the secondary channel for global enable.
12 (R/W)	GBL_SP2A_PC_EN	SPORT2 A Primary Channel Select. The DAI_GBL_SP_EN.GBL_SP2A_PC_EN bit selects SPORT2 A as the primary channel for global enable.
11 (R/W)	GBL_SP1B_SC_EN	SPORT1 B Secondary Channel Select. The DAI_GBL_SP_EN.GBL_SP1B_SC_EN bit selects SPORT1 B as the secondary channel for global enable.
10 (R/W)	GBL_SP1B_PC_EN	SPORT1 B Primary Channel Select. The DAI_GBL_SP_EN.GBL_SP1B_PC_EN bit selects SPORT1 B as the primary channel for global enable.
9 (R/W)	GBL_SP1A_SC_EN	SPORT1 A Secondary Channel Select. The DAI_GBL_SP_EN.GBL_SP1A_SC_EN bit selects SPORT1 A as the secondary channel for global enable.
8 (R/W)	GBL_SP1A_PC_EN	SPORT1 A Primary Channel Select. The DAI_GBL_SP_EN.GBL_SP1A_PC_EN bit selects SPORT1 A as the primary channel for global enable.
7 (R/W)	GBL_SP0B_SC_EN	SPORT0 B Secondary Channel Select. The DAI_GBL_SP_EN.GBL_SP0B_SC_EN bit selects SPORT0 B as the secondary channel for global enable.
6 (R/W)	GBL_SP0B_PC_EN	SPORT0 B Primary Channel Select. Selects SPORT0 B primary channel for global enable.

Table 20-48: DAI_GBL_SP_EN Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
5 (R/W)	GBL_SP0A_SC_EN	SPORT0 A Secondary Channel Select. The DAI_GBL_SP_EN.GBL_SP0A_SC_EN bit selects SPORT0 A as the secondary channel for global enable.
4 (R/W)	GBL_SP0A_PC_EN	SPORT0 A Primary Channel Select. The DAI_GBL_SP_EN.GBL_SP0A_PC_EN bit selects SPORT0 A as the primary channel for global enable.
3 (R/W)	GBL_SPEN_GRP1	Group 1 SPORT Subgroup Select. The DAI_GBL_SP_EN.GBL_SPEN_GRP1 bit creates a subgroup of selected SPORTs.
2 (R/W)	GBL_SPEN_GRP0	Group 0 SPORT Subgroup Select. The DAI_GBL_SP_EN.GBL_SPEN_GRP0 bit creates a subgroup of selected SPORTs.
1 (R/W)	GBL_SPEN_DAIX	DAI SPORTs Enable. The DAI_GBL_SP_EN.GBL_SPEN_DAIX bit enables the SPORTs selected within a DAI.
0 (R/W)	GBL_SP_EN	Global SPORTs Enable. The DAI_GBL_SP_EN.GBL_SP_EN bit enables the selected SPORTs.

Falling-Edge Interrupt Mask Register

The `DAI_IMSK_FE` register masks and unmasks interrupts generated on the falling edge of a waveform. Note that any of the Group E signals can be mapped to any of the Miscellaneous interrupts (9-0).

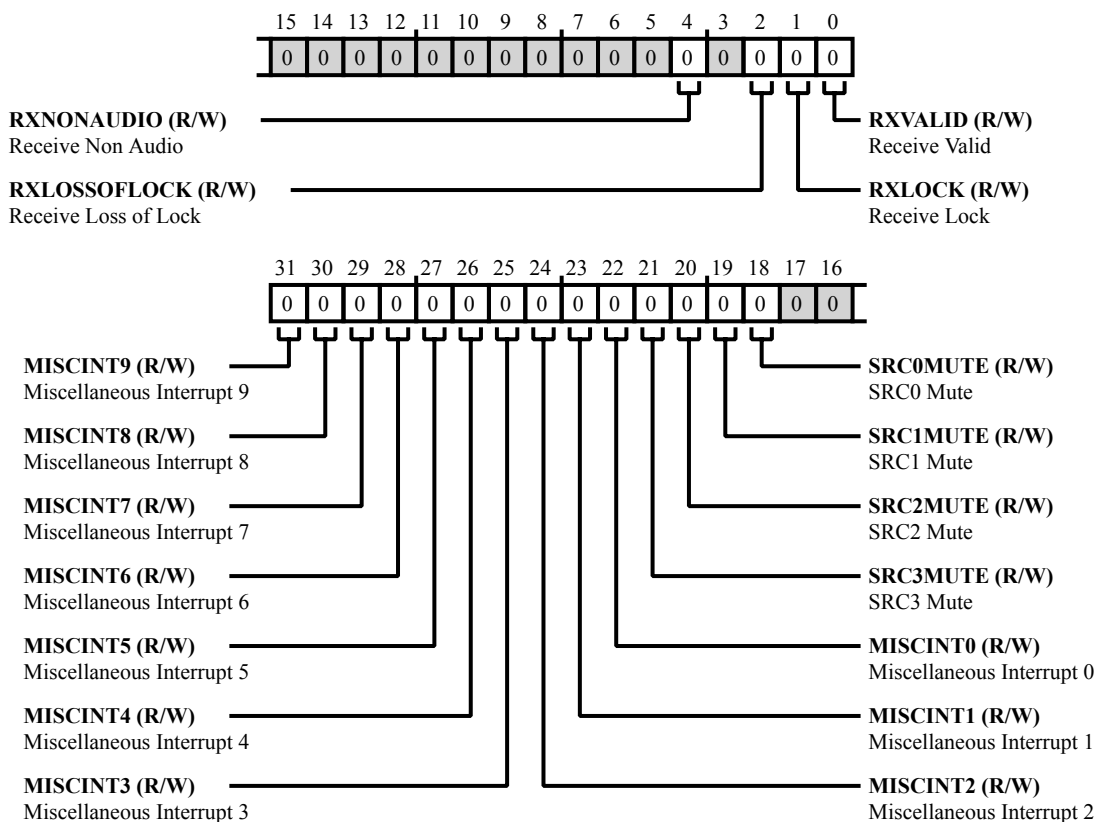


Figure 20-35: DAI_IMSK_FE Register Diagram

Table 20-49: DAI_IMSK_FE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	MISCINT9	Miscellaneous Interrupt 9. Setting the <code>DAI_IMSK_FE.MISCINT9</code> bit unmasks the interrupt for the falling edge of the routed signal.
30 (R/W)	MISCINT8	Miscellaneous Interrupt 8. Setting the <code>DAI_IMSK_FE.MISCINT8</code> bit unmasks the interrupt for the falling edge of the routed signal.
29 (R/W)	MISCINT7	Miscellaneous Interrupt 7. Setting the <code>DAI_IMSK_FE.MISCINT7</code> bit unmasks the interrupt for the falling edge of the routed signal.

Table 20-49: DAI_IMSK_FE Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
28 (R/W)	MISCINT6	Miscellaneous Interrupt 6. Setting the DAI_IMSK_FE.MISCINT6 bit unmask the interrupt for the falling edge of the routed signal.
27 (R/W)	MISCINT5	Miscellaneous Interrupt 5. Setting the DAI_IMSK_FE.MISCINT5 bit unmask the interrupt for the falling edge of the routed signal.
26 (R/W)	MISCINT4	Miscellaneous Interrupt 4. Setting the DAI_IMSK_FE.MISCINT4 bit unmask the interrupt for the falling edge of the routed signal.
25 (R/W)	MISCINT3	Miscellaneous Interrupt 3. Setting the DAI_IMSK_FE.MISCINT3 bit unmask the interrupt for the falling edge of the routed signal.
24 (R/W)	MISCINT2	Miscellaneous Interrupt 2. Setting the DAI_IMSK_FE.MISCINT2 bit unmask the interrupt for the falling edge of the routed signal.
23 (R/W)	MISCINT1	Miscellaneous Interrupt 1. Setting the DAI_IMSK_FE.MISCINT1 bit unmask the interrupt for the falling edge of the routed signal.
22 (R/W)	MISCINT0	Miscellaneous Interrupt 0. Setting the DAI_IMSK_FE.MISCINT0 bit unmask the interrupt for the falling edge of the routed signal.
21 (R/W)	SRC3MUTE	SRC3 Mute. The DAI_IMSK_FE.SRC3MUTE bit masks or unmask the corresponding SRC mute out interrupt for the falling edge of this interrupt. This interrupt can be generated either entering mute, exiting muting or both.
20 (R/W)	SRC2MUTE	SRC2 Mute. The DAI_IMSK_FE.SRC2MUTE bit masks or unmask the corresponding SRC mute out interrupt for the falling edge of this interrupt. This interrupt can be generated either entering mute, exiting muting or both.
19 (R/W)	SRC1MUTE	SRC1 Mute. The DAI_IMSK_FE.SRC1MUTE bit masks or unmask the corresponding SRC mute out interrupt for the falling edge of this interrupt. This interrupt can be generated either entering mute, exiting muting or both.

Table 20-49: DAI_IMSK_FE Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
18 (R/W)	SRC0MUTE	SRC0 Mute. The DAI_IMSK_FE.SRC0MUTE bit masks or unmasks the corresponding SRC mute out interrupt for the falling edge of this interrupt. This interrupt can be generated either entering mute, exiting muting or both.
4 (R/W)	RXNONAUDIO	Receive Non Audio. The DAI_IMSK_FE.RXNONAUDIO bit masks or unmasks the non audio frame mode interrupt for the falling edge of this interrupt. If the channel status indicates non-PCM audio, the NONAUDIO bit flag is set.
2 (R/W)	RXLOSSOFLOCK	Receive Loss of Lock. The DAI_IMSK_FE.RXLOSSOFLOCK bit masks or unmasks the emphasis loss of lock interrupt for the falling edge of this interrupt. The loss of lock status is set by the S/PDIF receiver if receiver loses the lock of biphase stream.
1 (R/W)	RXLOCK	Receive Lock. The DAI_IMSK_FE.RXLOCK bit masks or unmasks the S/PDIF receiver lock for the falling edge of this interrupt. This interrupt occurs when the S/PDIF receiver locks to the S/PDIF stream.
0 (R/W)	RXVALID	Receive Valid. Setting the DAI_IMSK_FE.RXVALID bit unmasks the interrupt for the falling edge of the routed signal. This interrupt is set based on whether data received by the S/PDIF is linear PCM or non-linear audio data.

Core Interrupt Priority Assignment Register

The `DAI_IMSK_PRI` register masks interrupts for DAI high or DAI low interrupt priority.

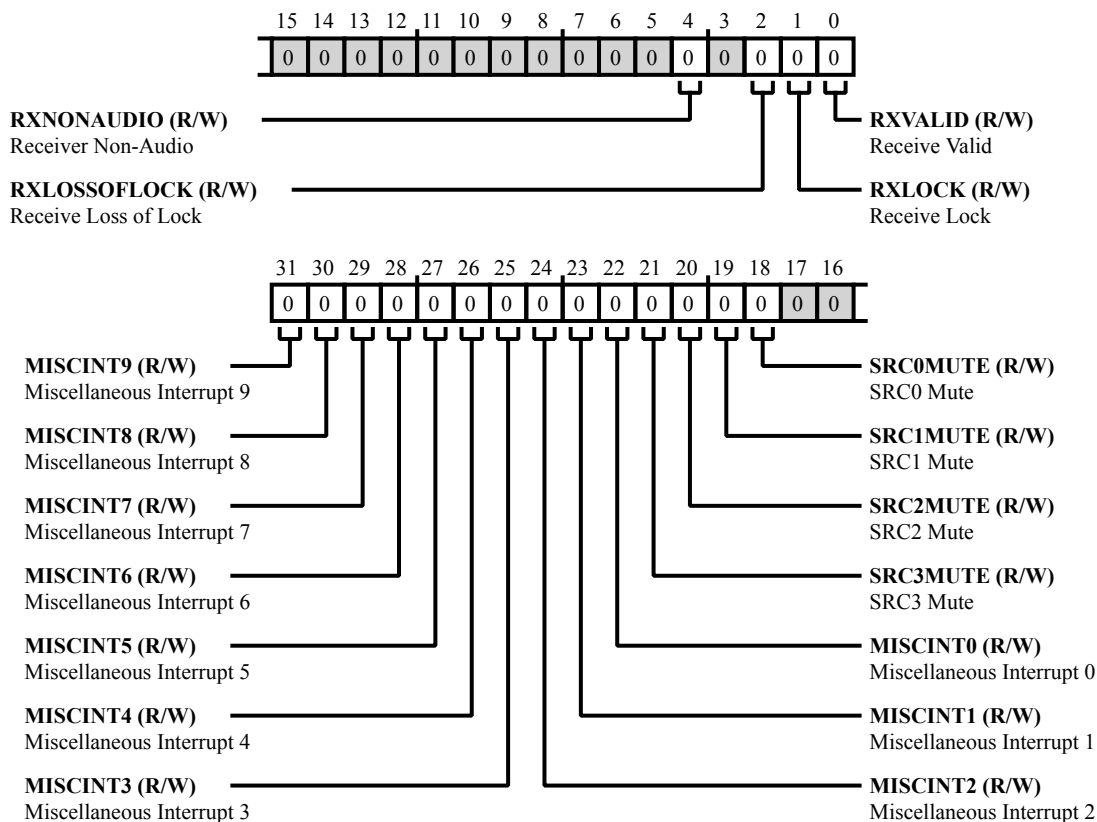


Figure 20-36: DAI_IMSK_PRI Register Diagram

Table 20-50: DAI_IMSK_PRI Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	MISCINT9	Miscellaneous Interrupt 9. If the <code>DAI_IMSK_PRI.MISCINT9</code> bit is cleared (=0), the interrupt is mapped to <code>INTR_DAI_IRQL</code> signal from the SEC. If the <code>DAI_IMSK_PRI.MISCINT9</code> bit is set (=1) the interrupt is mapped to <code>INTR_DAI_IRQH</code> signal from the SEC.
30 (R/W)	MISCINT8	Miscellaneous Interrupt 8. If the <code>DAI_IMSK_PRI.MISCINT8</code> bit is cleared (=0), the interrupt is mapped to <code>INTR_DAI_IRQL</code> signal from the SEC. If the <code>DAI_IMSK_PRI.MISCINT8</code> bit is set (=1) the interrupt is mapped to <code>INTR_DAI_IRQH</code> signal from the SEC.

Table 20-50: DAI_IMSK_PRI Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
29 (R/W)	MISCINT7	Miscellaneous Interrupt 7. If the DAI_IMSK_PRI.MISCINT7 bit is cleared (=0), the interrupt is mapped to INTR_DAI_IRQL signal from the SEC. If the DAI_IMSK_PRI.MISCINT7 bit is set (=1) the interrupt is mapped to INTR_DAI_IRQH signal from the SEC.
28 (R/W)	MISCINT6	Miscellaneous Interrupt 6. If the DAI_IMSK_PRI.MISCINT6 bit is cleared (=0), the interrupt is mapped to INTR_DAI_IRQL signal from the SEC. If the DAI_IMSK_PRI.MISCINT6 bit is set (=1) the interrupt is mapped to INTR_DAI_IRQH signal from the SEC.
27 (R/W)	MISCINT5	Miscellaneous Interrupt 5. If the DAI_IMSK_PRI.MISCINT5 bit is cleared (=0), the interrupt is mapped to INTR_DAI_IRQL signal from the SEC. If the DAI_IMSK_PRI.MISCINT5 bit is set (=1) the interrupt is mapped to INTR_DAI_IRQH signal from the SEC.
26 (R/W)	MISCINT4	Miscellaneous Interrupt 4. If the DAI_IMSK_PRI.MISCINT4 bit is cleared (=0), the interrupt is mapped to INTR_DAI_IRQL signal from the SEC. If the DAI_IMSK_PRI.MISCINT4 bit is set (=1) the interrupt is mapped to INTR_DAI_IRQH signal from the SEC.
25 (R/W)	MISCINT3	Miscellaneous Interrupt 3. If the DAI_IMSK_PRI.MISCINT3 bit is cleared (=0), the interrupt is mapped to INTR_DAI_IRQL signal from the SEC. If the DAI_IMSK_PRI.MISCINT3 bit is set (=1) the interrupt is mapped to INTR_DAI_IRQH signal from the SEC.
24 (R/W)	MISCINT2	Miscellaneous Interrupt 2. If the DAI_IMSK_PRI.MISCINT2 bit is cleared (=0), the interrupt is mapped to INTR_DAI_IRQL signal from the SEC. If the DAI_IMSK_PRI.MISCINT2 bit is set (=1) the interrupt is mapped to INTR_DAI_IRQH signal from the SEC.
23 (R/W)	MISCINT1	Miscellaneous Interrupt 1. If the DAI_IMSK_PRI.MISCINT1 bit is cleared (=0), the interrupt is mapped to INTR_DAI_IRQL signal from the SEC. If the DAI_IMSK_PRI.MISCINT1 bit is set (=1) the interrupt is mapped to INTR_DAI_IRQH signal from the SEC.
22 (R/W)	MISCINT0	Miscellaneous Interrupt 0. If the DAI_IMSK_PRI.MISCINT0 bit is cleared (=0), the interrupt is mapped to INTR_DAI_IRQL signal from the SEC. If the DAI_IMSK_PRI.MISCINT0 bit is set (=1) the interrupt is mapped to INTR_DAI_IRQH signal from the SEC.
21 (R/W)	SRC3MUTE	SRC3 Mute. If the DAI_IMSK_PRI.SRC3MUTE bit is cleared (=0), the interrupt is mapped to INTR_DAI_IRQL signal from the SEC. If the DAI_IMSK_PRI.SRC3MUTE bit is set (=1) the interrupt is mapped to INTR_DAI_IRQH signal from the SEC.

Table 20-50: DAI_IMSK_PRI Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
20 (R/W)	SRC2MUTE	SRC2 Mute. If the DAI_IMSK_PRI.SRC2MUTE bit is cleared (=0), the interrupt is mapped to INTR_DAI_IRQL signal from the SEC. If the DAI_IMSK_PRI.SRC2MUTE bit is set (=1) the interrupt is mapped to INTR_DAI_IRQH signal from the SEC.
19 (R/W)	SRC1MUTE	SRC1 Mute. If the DAI_IMSK_PRI.SRC1MUTE bit is cleared (=0), the interrupt is mapped to INTR_DAI_IRQL signal from the SEC. If the DAI_IMSK_PRI.SRC1MUTE bit is set (=1) the interrupt is mapped to INTR_DAI_IRQH signal from the SEC.
18 (R/W)	SRC0MUTE	SRC0 Mute. If the DAI_IMSK_PRI.SRC0MUTE bit is cleared (=0), the interrupt is mapped to INTR_DAI_IRQL signal from the SEC. If the DAI_IMSK_PRI.SRC0MUTE bit is set (=1) the interrupt is mapped to INTR_DAI_IRQH signal from the SEC.
4 (R/W)	RXNONAUDIO	Receiver Non-Audio. If the DAI_IMSK_PRI.RXNONAUDIO bit is cleared (=0), the interrupt is mapped to INTR_DAI_IRQL signal from the SEC. If the DAI_IMSK_PRI.RXNONAUDIO bit is set (=1) the interrupt is mapped to INTR_DAI_IRQH signal from the SEC.
2 (R/W)	RXLOSSOFLOCK	Receive Loss of Lock. The DAI_IMSK_PRI.RXLOSSOFLOCK bit masks or unmasks the emphasis loss of lock interrupt for the falling edge of this interrupt. The loss of lock status is set by the S/PDIF receiver if receiver loses the lock of biphase stream.
1 (R/W)	RXLOCK	Receive Lock. If the DAI_IMSK_PRI.RXLOCK bit is cleared (=0), the interrupt is mapped to INTR_DAI_IRQL signal from the SEC. If the DAI_IMSK_PRI.RXLOCK bit is set (=1) the interrupt is mapped to INTR_DAI_IRQH signal from the SEC.
0 (R/W)	RXVALID	Receive Valid. If the DAI_IMSK_PRI.RXVALID bit is cleared (=0), the interrupt is mapped to the INTR_DAI_IRQL from the SEC. If the DAI_IMSK_PRI.RXVALID bit is set (=1) the interrupt is mapped to INTR_DAI_IRQH signal from the SEC.

Rising-Edge Interrupt Mask Register

The `DAI_IMSK_RE` register masks and unmasks interrupts generated on the rising edge of a waveform. Note that any of the Group E signals can be mapped to any of the miscellaneous interrupts (9-0).

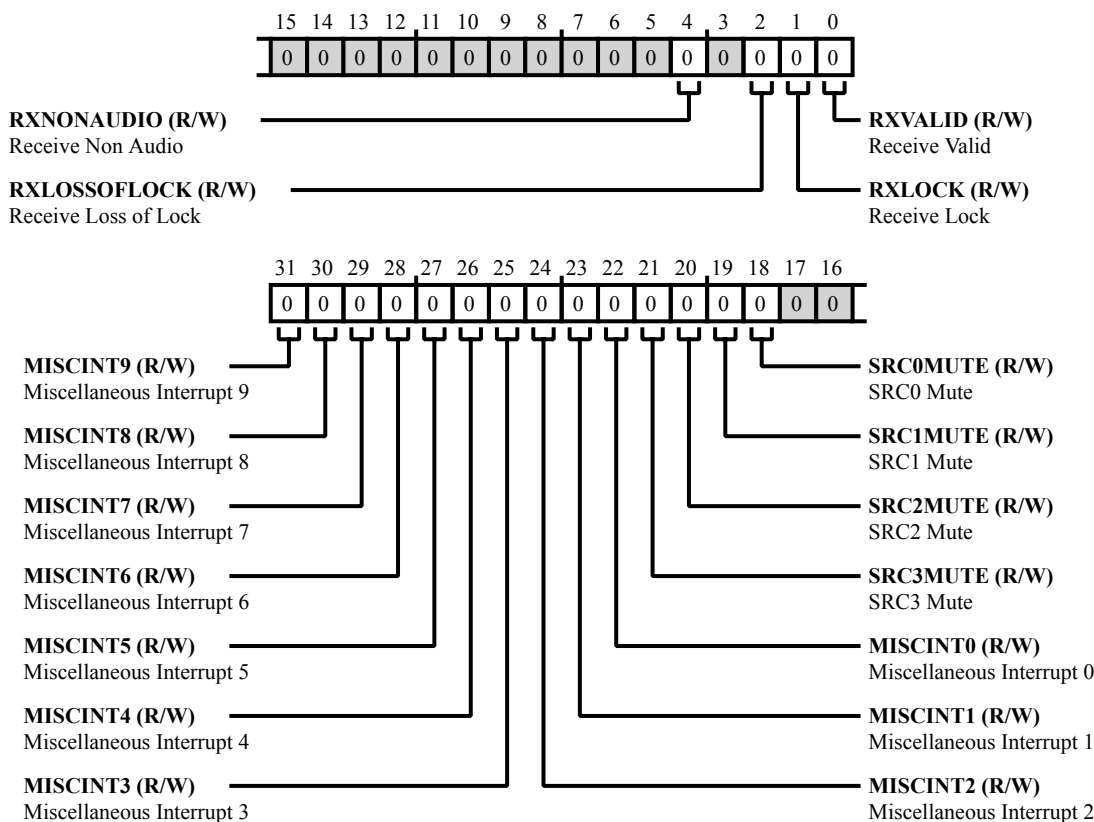


Figure 20-37: DAI_IMSK_RE Register Diagram

Table 20-51: DAI_IMSK_RE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	MISCINT9	Miscellaneous Interrupt 9. Setting the <code>DAI_IMSK_RE.MISCINT9</code> bit unmask the interrupt for the rising edge of the routed signal.
30 (R/W)	MISCINT8	Miscellaneous Interrupt 8. Setting the <code>DAI_IMSK_RE.MISCINT8</code> bit unmask the interrupt for the rising edge of the routed signal.
29 (R/W)	MISCINT7	Miscellaneous Interrupt 7. Setting the <code>DAI_IMSK_RE.MISCINT7</code> bit unmask the interrupt for the rising edge of the routed signal.

Table 20-51: DAI_IMSK_RE Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
28 (R/W)	MISCINT6	Miscellaneous Interrupt 6. Setting the DAI_IMSK_RE.MISCINT6 bit unmask the interrupt for the rising edge of the routed signal.
27 (R/W)	MISCINT5	Miscellaneous Interrupt 5. Setting the DAI_IMSK_RE.MISCINT5 bit unmask the interrupt for the rising edge of the routed signal.
26 (R/W)	MISCINT4	Miscellaneous Interrupt 4. Setting the DAI_IMSK_RE.MISCINT4 bit unmask the interrupt for the rising edge of the routed signal.
25 (R/W)	MISCINT3	Miscellaneous Interrupt 3. Setting the DAI_IMSK_RE.MISCINT3 bit unmask the interrupt for the rising edge of the routed signal.
24 (R/W)	MISCINT2	Miscellaneous Interrupt 2. Setting the DAI_IMSK_RE.MISCINT2 bit unmask the interrupt for the rising edge of the routed signal.
23 (R/W)	MISCINT1	Miscellaneous Interrupt 1. Setting the DAI_IMSK_RE.MISCINT1 bit unmask the interrupt for the rising edge of the routed signal.
22 (R/W)	MISCINT0	Miscellaneous Interrupt 0. Setting the DAI_IMSK_RE.MISCINT0 bit unmask the interrupt for the rising edge of the routed signal.
21 (R/W)	SRC3MUTE	SRC3 Mute. The DAI_IMSK_RE.SRC3MUTE bit masks or unmask the corresponding SRC mute out interrupt for the rising edge of this interrupt. This interrupt can be generated either entering mute, exiting muting or both.
20 (R/W)	SRC2MUTE	SRC2 Mute. The DAI_IMSK_RE.SRC2MUTE bit masks or unmask the corresponding SRC mute out interrupt for the rising edge of this interrupt. This interrupt can be generated either entering mute, exiting muting or both.
19 (R/W)	SRC1MUTE	SRC1 Mute. The DAI_IMSK_RE.SRC1MUTE bit masks or unmask the corresponding SRC mute out interrupt for the rising edge of this interrupt. This interrupt can be generated either entering mute, exiting muting or both.

Table 20-51: DAI_IMSK_RE Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
18 (R/W)	SRC0MUTE	SRC0 Mute. The DAI_IMSK_RE.SRC0MUTE bit masks or unmasks the corresponding SRC mute out interrupt for the rising edge of this interrupt. This interrupt can be generated either entering mute, exiting muting or both.
4 (R/W)	RXNONAUDIO	Receive Non Audio. The DAI_IMSK_RE.RXNONAUDIO bit masks or unmasks the non audio frame mode interrupt for the rising edge of this interrupt. If the channel status indicates non-PCM audio, the NONAUDIO bit flag is set.
2 (R/W)	RXLOSSOFLOCK	Receive Loss of Lock. The DAI_IMSK_RE.RXLOSSOFLOCK bit masks or unmasks the emphasis loss of lock interrupt for the rising edge of this interrupt. The loss of lock status is set by the S/PDIF receiver if receiver loses the lock of biphase stream.
1 (R/W)	RXLOCK	Receive Lock. The DAI_IMSK_RE.RXLOCK bit masks or unmasks the S/PDIF receiver lock for the rising edge of this interrupt. This interrupt occurs when the S/PDIF receiver locks to the S/PDIF stream.
0 (R/W)	RXVALID	Receive Valid. Setting the DAI_IMSK_RE.RXVALID bit unmasks the interrupt for the rising edge of the routed signal. This interrupt is set based on whether data received by the S/PDIF is linear PCM or non-linear audio data.

High Priority Interrupt Latch Register

The `DAI_IRPTL_H` register holds the high priority latched interrupt status for interrupt requests that have been unmasked (enabled) by the `DAI_IMSK_FE`, `DAI_IMSK_RE` registers and mapped to the `INTR_DAI_IRQH` signal in the SEC by `DAI_IMSK_PRI` registers. If a bit in this register is already set and the corresponding interrupt is masked in the `DAI_IMSK_FE`, `DAI_IMSK_RE` or `DAI_IMSK_PRI` registers, the latch holds its old value, leaving the interrupt asserted until its mask registers (`DAI_IMSK_FE`, `DAI_IMSK_RE` or `DAI_IMSK_PRI`) are reset by software with a W1C operation.

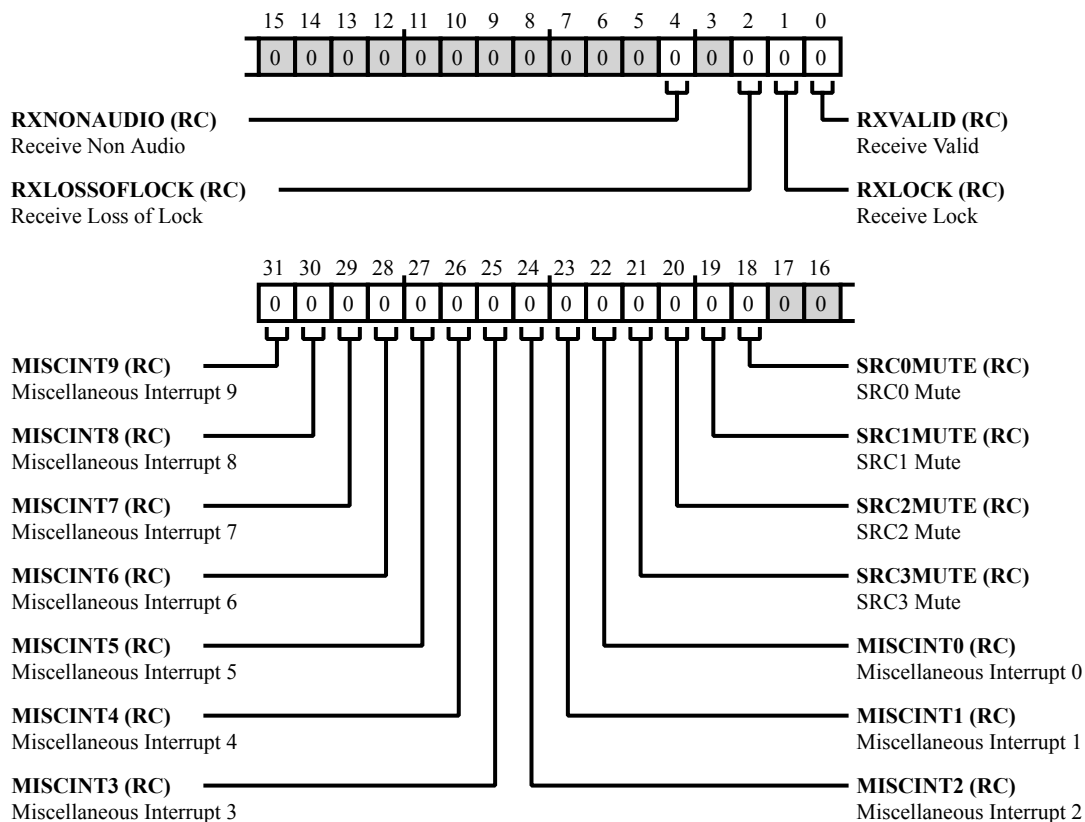


Figure 20-38: `DAI_IRPTL_H` Register Diagram

Table 20-52: `DAI_IRPTL_H` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (RC/NW)	MISCINT9	Miscellaneous Interrupt 9. The <code>DAI_IRPTL_H.MISCINT9</code> bit is set if the interrupt is mapped to the <code>INTR_DAI_IRQH</code> signal in the SEC using the <code>DAI_IMSK_PRI</code> register.
30 (RC/NW)	MISCINT8	Miscellaneous Interrupt 8. The <code>DAI_IRPTL_H.MISCINT8</code> bit is set if the interrupt is mapped to the <code>INTR_DAI_IRQH</code> signal in the SEC using the <code>DAI_IMSK_PRI</code> register.

Table 20-52: DAI_IRPTL_H Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
29 (RC/NW)	MISCINT7	Miscellaneous Interrupt 7. The DAI_IRPTL_H.MISCINT7 bit is set if the interrupt is mapped to the INTR_DAI_IRQH signal in the SEC using the DAI_IMSK_PRI register.
28 (RC/NW)	MISCINT6	Miscellaneous Interrupt 6. The DAI_IRPTL_H.MISCINT6 bit is set if the interrupt is mapped to the INTR_DAI_IRQH signal in the SEC using the DAI_IMSK_PRI register.
27 (RC/NW)	MISCINT5	Miscellaneous Interrupt 5. The DAI_IRPTL_H.MISCINT5 bit is set if the interrupt is mapped to the INTR_DAI_IRQH signal in the SEC using the DAI_IMSK_PRI register.
26 (RC/NW)	MISCINT4	Miscellaneous Interrupt 4. The DAI_IRPTL_H.MISCINT4 bit is set if the interrupt is mapped to the INTR_DAI_IRQH signal in the SEC using the DAI_IMSK_PRI register.
25 (RC/NW)	MISCINT3	Miscellaneous Interrupt 3. The DAI_IRPTL_H.MISCINT3 bit is set if the interrupt is mapped to the INTR_DAI_IRQH signal in the SEC using the DAI_IMSK_PRI register.
24 (RC/NW)	MISCINT2	Miscellaneous Interrupt 2. The DAI_IRPTL_H.MISCINT2 bit is set if the interrupt is mapped to the INTR_DAI_IRQH signal in the SEC using the DAI_IMSK_PRI register.
23 (RC/NW)	MISCINT1	Miscellaneous Interrupt 1. The DAI_IRPTL_H.MISCINT1 bit is set if the interrupt is mapped to the INTR_DAI_IRQH signal in the SEC using the DAI_IMSK_PRI register.
22 (RC/NW)	MISCINT0	Miscellaneous Interrupt 0. The DAI_IRPTL_H.MISCINT0 bit is set if the interrupt is mapped to the INTR_DAI_IRQH signal in the SEC using the DAI_IMSK_PRI register.
21 (RC/NW)	SRC3MUTE	SRC3 Mute. The DAI_IRPTL_H.SRC3MUTE bit is set if the interrupt is mapped to the INTR_DAI_IRQH signal in the SEC using the DAI_IMSK_PRI register.
20 (RC/NW)	SRC2MUTE	SRC2 Mute. The DAI_IRPTL_H.SRC2MUTE bit is set if the interrupt is mapped to the INTR_DAI_IRQH signal in the SEC using the DAI_IMSK_PRI register.
19 (RC/NW)	SRC1MUTE	SRC1 Mute. The DAI_IRPTL_H.SRC1MUTE bit is set if the interrupt is mapped to the INTR_DAI_IRQH signal in the SEC using the DAI_IMSK_PRI register.

Table 20-52: DAI_IRPTL_H Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
18 (RC/NW)	SRC0MUTE	SRC0 Mute. The DAI_IRPTL_H.SRC0MUTE bit is set if the interrupt is mapped to the INTR_DAI_IRQH signal in the SEC using the DAI_IMSK_PRI register.
4 (RC/NW)	RXNONAUDIO	Receive Non Audio. The DAI_IRPTL_H.RXNONAUDIO bit is set if the interrupt is mapped to the INTR_DAI_IRQH signal in the SEC using the DAI_IMSK_PRI register.
2 (RC/NW)	RXLOSSOFLOCK	Receive Loss of Lock. The DAI_IRPTL_H.RXLOSSOFLOCK bit is set if the interrupt is mapped to the INTR_DAI_IRQH signal in the SEC using the DAI_IMSK_PRI register.
1 (RC/NW)	RXLOCK	Receive Lock. The DAI_IRPTL_H.RXLOCK bit is set if the interrupt is mapped to the INTR_DAI_IRQH signal in the SEC using the DAI_IMSK_PRI register.
0 (RC/NW)	RXVALID	Receive Valid. The DAI_IRPTL_H.RXVALID bit is set if the interrupt is mapped to the INTR_DAI_IRQH signal in the SEC using the DAI_IMSK_PRI register.

Shadow High Priority Interrupt Latch Register

The `DAI_IRPTL_HS` register is the shadow register of the `DAI_IRPTL_H` register. Its content is the same as the `DAI_IRPTL_H` register. Reading the `DAI_IRPTL_HS` register does not affect its contents while reading the contents of the `DAI_IRPTL_H` registers clears it.

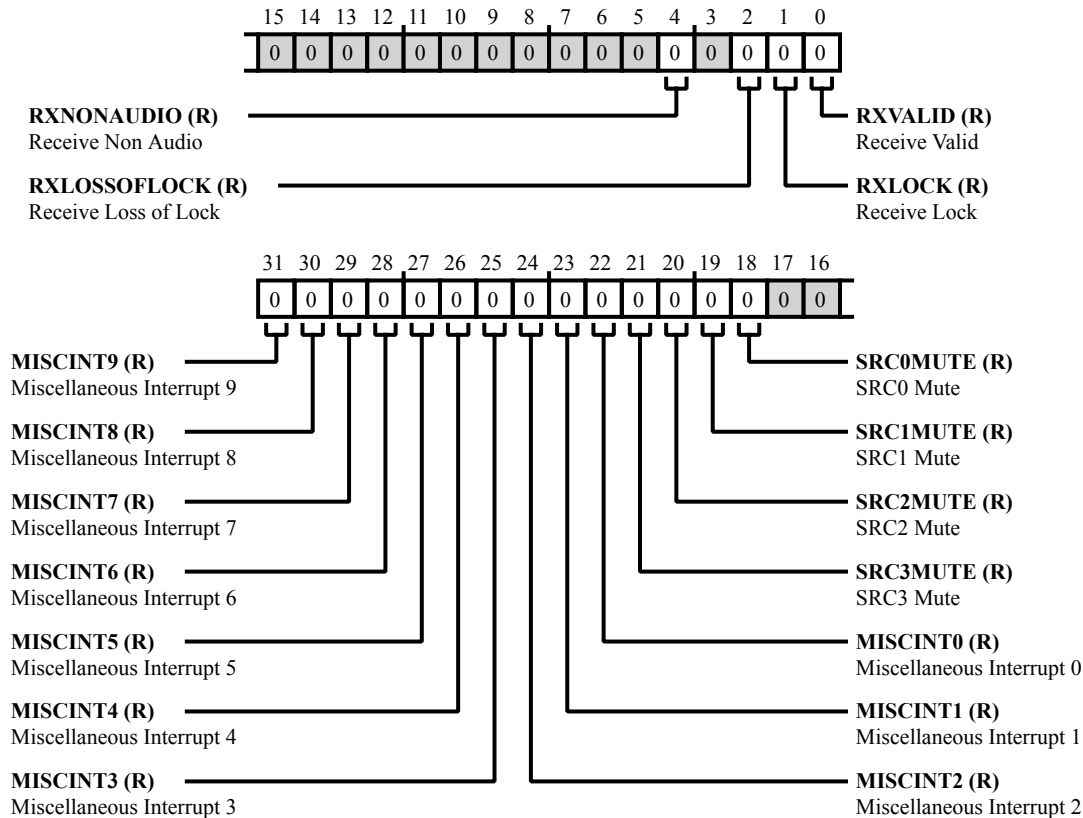


Figure 20-39: `DAI_IRPTL_HS` Register Diagram

Table 20-53: `DAI_IRPTL_HS` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/NW)	MISCINT9	Miscellaneous Interrupt 9. The <code>DAI_IRPTL_HS.MISCINT9</code> bit is the shadow of the <code>DAI_IRPTL_H.MISCINT9</code> bit and contains the same content. Reading the <code>DAI_IRPTL_HS.MISCINT9</code> bit does not affect its contents while reading the <code>DAI_IRPTL_H.MISCINT9</code> bit clears it.
30 (R/NW)	MISCINT8	Miscellaneous Interrupt 8. The <code>DAI_IRPTL_HS.MISCINT8</code> bit is the shadow of the <code>DAI_IRPTL_H.MISCINT8</code> bit and contains the same content. Reading the <code>DAI_IRPTL_HS.MISCINT8</code> bit does not affect its contents while reading the <code>DAI_IRPTL_H.MISCINT8</code> bit clears it.

Table 20-53: DAI_IRPTL_HS Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
29 (R/NW)	MISCINT7	Miscellaneous Interrupt 7. The DAI_IRPTL_HS.MISCINT7 bit is the shadow of the DAI_IRPTL_H.MISCINT7 bit and contains the same content. Reading the DAI_IRPTL_HS.MISCINT7 bit does not affect its contents while reading the DAI_IRPTL_H.MISCINT7 bit clears it.
28 (R/NW)	MISCINT6	Miscellaneous Interrupt 6. The DAI_IRPTL_HS.MISCINT6 bit is the shadow of the DAI_IRPTL_H.MISCINT6 bit and contains the same content. Reading the DAI_IRPTL_HS.MISCINT6 bit does not affect its contents while reading the DAI_IRPTL_H.MISCINT6 bit clears it.
27 (R/NW)	MISCINT5	Miscellaneous Interrupt 5. The DAI_IRPTL_HS.MISCINT5 bit is the shadow of the DAI_IRPTL_H.MISCINT5 bit and contains the same content. Reading the DAI_IRPTL_HS.MISCINT5 bit does not affect its contents while reading the DAI_IRPTL_H.MISCINT5 bit clears it.
26 (R/NW)	MISCINT4	Miscellaneous Interrupt 4. The DAI_IRPTL_HS.MISCINT4 bit is the shadow of the DAI_IRPTL_H.MISCINT4 bit and contains the same content. Reading the DAI_IRPTL_HS.MISCINT4 bit does not affect its contents while reading the DAI_IRPTL_H.MISCINT4 bit clears it.
25 (R/NW)	MISCINT3	Miscellaneous Interrupt 3. The DAI_IRPTL_HS.MISCINT3 bit is the shadow of the DAI_IRPTL_H.MISCINT3 bit and contains the same content. Reading the DAI_IRPTL_HS.MISCINT3 bit does not affect its contents while reading the DAI_IRPTL_H.MISCINT3 bit clears it.
24 (R/NW)	MISCINT2	Miscellaneous Interrupt 2. The DAI_IRPTL_HS.MISCINT2 bit is the shadow of the DAI_IRPTL_H.MISCINT2 bit and contains the same content. Reading the DAI_IRPTL_HS.MISCINT2 bit does not affect its contents while reading the DAI_IRPTL_H.MISCINT2 bit clears it.
23 (R/NW)	MISCINT1	Miscellaneous Interrupt 1. The DAI_IRPTL_HS.MISCINT1 bit is the shadow of the DAI_IRPTL_H.MISCINT1 bit and contains the same content. Reading the DAI_IRPTL_HS.MISCINT1 bit does not affect its contents while reading the DAI_IRPTL_H.MISCINT1 bit clears it.

Table 20-53: DAI_IRPTL_HS Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
22 (R/NW)	MISCINT0	Miscellaneous Interrupt 0. The DAI_IRPTL_HS.MISCINT0 bit is the shadow of the DAI_IRPTL_H.MISCINT0 bit and contains the same content. Reading the DAI_IRPTL_HS.MISCINT0 bit does not affect its contents while reading the DAI_IRPTL_H.MISCINT0 bit clears it.
21 (R/NW)	SRC3MUTE	SRC3 Mute. The DAI_IRPTL_HS.SRC3MUTE bit is the shadow of the DAI_IRPTL_H.SRC3MUTE bit and contains the same content. Reading the DAI_IRPTL_HS.SRC3MUTE bit does not affect its contents while reading the DAI_IRPTL_H.SRC3MUTE bit clears it.
20 (R/NW)	SRC2MUTE	SRC2 Mute. The DAI_IRPTL_HS.SRC2MUTE bit is the shadow of the DAI_IRPTL_H.SRC2MUTE bit and contains the same content. Reading the DAI_IRPTL_HS.SRC2MUTE bit does not affect its contents while reading the DAI_IRPTL_H.SRC2MUTE bit clears it.
19 (R/NW)	SRC1MUTE	SRC1 Mute. The DAI_IRPTL_HS.SRC1MUTE bit is the shadow of the DAI_IRPTL_H.SRC1MUTE bit and contains the same content. Reading the DAI_IRPTL_HS.SRC1MUTE bit does not affect its contents while reading the DAI_IRPTL_H.SRC1MUTE bit clears it.
18 (R/NW)	SRC0MUTE	SRC0 Mute. The DAI_IRPTL_HS.SRC0MUTE bit is the shadow of the DAI_IRPTL_H.SRC0MUTE bit and contains the same content. Reading the DAI_IRPTL_HS.SRC0MUTE bit does not affect its contents while reading the DAI_IRPTL_H.SRC0MUTE bit clears it.
4 (R/NW)	RXNONAUDIO	Receive Non Audio. The DAI_IRPTL_HS.RXNONAUDIO bit is the shadow of DAI_IRPTL_H.RXNONAUDIO bit and contains the same content. Reading this bit does not affect its content.
2 (R/NW)	RXLOSSOFLOCK	Receive Loss of Lock. The DAI_IRPTL_HS.RXLOSSOFLOCK bit is the shadow of DAI_IRPTL_H.RXLOSSOFLOCK bit and contains the same content. Reading this bit does not affect its content.
1 (R/NW)	RXLOCK	Receive Lock. The DAI_IRPTL_HS.RXLOCK bit is the shadow of DAI_IRPTL_H.RXLOCK bit and contains the same content. Reading this bit does not affect its content.

Table 20-53: DAI_IRPTL_HS Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/NW)	RXVALID	Receive Valid. Setting the DAI_IRPTL_HS.RXVALID bit is the shadow of DAI_IRPTL_H.RXVALID bit and contains the same content. Reading this bit does not affect its content.

Low Priority Interrupt Latch Register

The `DAI_IRPTL_L` register holds the low priority latched interrupt status for interrupt requests that have been unmasked (enabled) by the `DAI_IMSK_FE`, `DAI_IMSK_RE` or `DAI_IMSK_PRI` registers. If a bit in `DAI_IRPTL_L` is already set and the corresponding interrupt is masked in `DAI_IMSK_FE`, `DAI_IMSK_RE` or `DAI_IMSK_PRI` registers, the latch holds its old value, leaving the interrupt asserted until its masked registers (`DAI_IMSK_FE`, `DAI_IMSK_RE` or `DAI_IMSK_PRI`) are reset by software with a W1C operation.

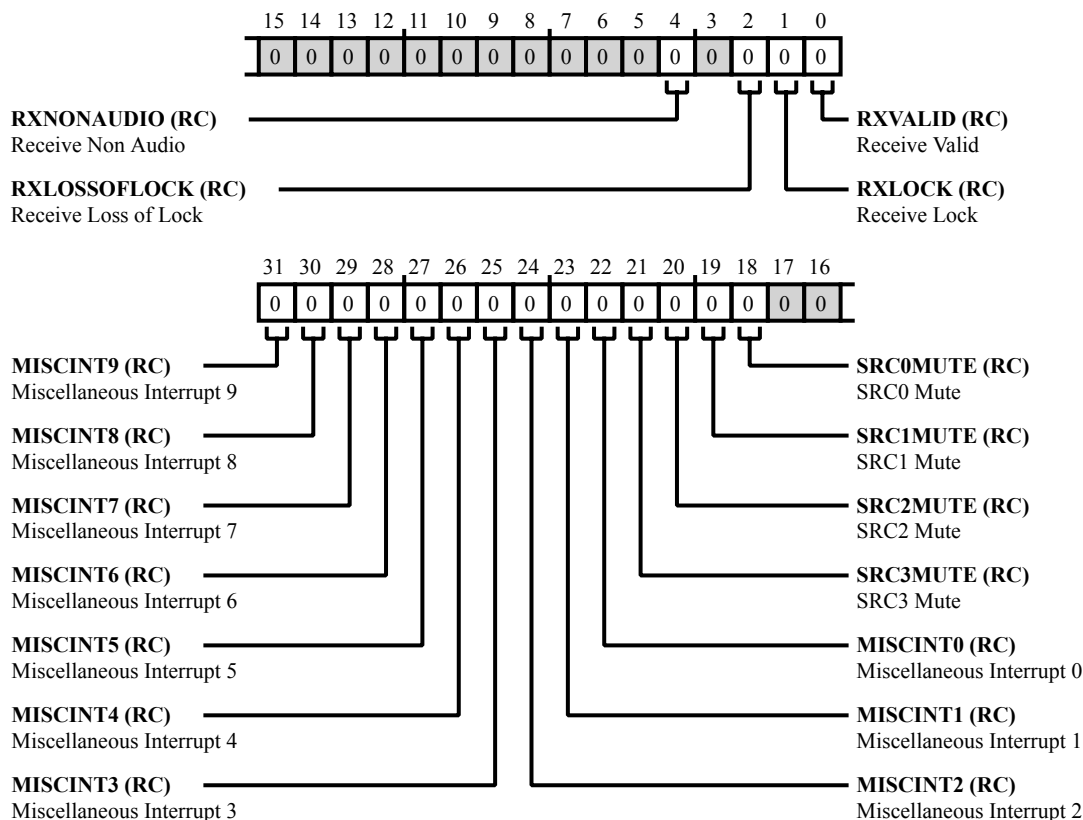


Figure 20-40: DAI_IRPTL_L Register Diagram

Table 20-54: DAI_IRPTL_L Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (RC/NW)	MISCINT9	Miscellaneous Interrupt 9. The <code>DAI_IRPTL_L.MISCINT9</code> bit is set if the interrupt is mapped to the <code>INTR_DAI_IRQL</code> signal in the SEC using the <code>DAI_IMSK_PRI</code> register.
30 (RC/NW)	MISCINT8	Miscellaneous Interrupt 8. The <code>DAI_IRPTL_L.MISCINT8</code> bit is set if the interrupt is mapped to the <code>INTR_DAI_IRQL</code> signal in the SEC using the <code>DAI_IMSK_PRI</code> register.

Table 20-54: DAI_IRPTL_L Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
29 (RC/NW)	MISCINT7	Miscellaneous Interrupt 7. The DAI_IRPTL_L.MISCINT7 bit is set if the interrupt is mapped to the INTR_DAI_IRQL signal in the SEC using the DAI_IMSK_PRI register.
28 (RC/NW)	MISCINT6	Miscellaneous Interrupt 6. The DAI_IRPTL_L.MISCINT6 bit is set if the interrupt is mapped to the INTR_DAI_IRQL signal in the SEC using the DAI_IMSK_PRI register.
27 (RC/NW)	MISCINT5	Miscellaneous Interrupt 5. The DAI_IRPTL_L.MISCINT5 bit is set if the interrupt is mapped to the INTR_DAI_IRQL signal in the SEC using the DAI_IMSK_PRI register.
26 (RC/NW)	MISCINT4	Miscellaneous Interrupt 4. The DAI_IRPTL_L.MISCINT4 bit is set if the interrupt is mapped to the INTR_DAI_IRQL signal in the SEC using the DAI_IMSK_PRI register.
25 (RC/NW)	MISCINT3	Miscellaneous Interrupt 3. The DAI_IRPTL_L.MISCINT3 bit is set if the interrupt is mapped to the INTR_DAI_IRQL signal in the SEC using the DAI_IMSK_PRI register.
24 (RC/NW)	MISCINT2	Miscellaneous Interrupt 2. The DAI_IRPTL_L.MISCINT2 bit is set if the interrupt is mapped to the INTR_DAI_IRQL signal in the SEC using the DAI_IMSK_PRI register.
23 (RC/NW)	MISCINT1	Miscellaneous Interrupt 1. The DAI_IRPTL_L.MISCINT1 bit is set if the interrupt is mapped to the INTR_DAI_IRQL signal in the SEC using the DAI_IMSK_PRI register.
22 (RC/NW)	MISCINT0	Miscellaneous Interrupt 0. The DAI_IRPTL_L.MISCINT0 bit is set if the interrupt is mapped to the INTR_DAI_IRQL signal in the SEC using the DAI_IMSK_PRI register.
21 (RC/NW)	SRC3MUTE	SRC3 Mute. The DAI_IRPTL_L.SRC3MUTE bit is set if the interrupt is mapped to the INTR_DAI_IRQL signal in the SEC using the DAI_IMSK_PRI register.
20 (RC/NW)	SRC2MUTE	SRC2 Mute. The DAI_IRPTL_L.SRC2MUTE bit is set if the interrupt is mapped to the INTR_DAI_IRQL signal in the SEC using the DAI_IMSK_PRI register.
19 (RC/NW)	SRC1MUTE	SRC1 Mute. The DAI_IRPTL_L.SRC1MUTE bit is set if the interrupt is mapped to the INTR_DAI_IRQL signal in the SEC using the DAI_IMSK_PRI register.

Table 20-54: DAI_IRPTL_L Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
18 (RC/NW)	SRC0MUTE	SRC0 Mute. The DAI_IRPTL_L.SRC0MUTE bit is set if the interrupt is mapped to the INTR_DAI_IRQL signal in the SEC using the DAI_IMSK_PRI register.
4 (RC/NW)	RXNONAUDIO	Receive Non Audio. The DAI_IRPTL_L.RXNONAUDIO bit is set if the interrupt is mapped to the INTR_DAI_IRQL signal in the SEC using the DAI_IMSK_PRI register.
2 (RC/NW)	RXLOSSOFLOCK	Receive Loss of Lock. The DAI_IRPTL_L.RXLOSSOFLOCK bit is set if the interrupt is mapped to the INTR_DAI_IRQL signal in the SEC using the DAI_IMSK_PRI register.
1 (RC/NW)	RXLOCK	Receive Lock. The DAI_IRPTL_L.RXLOCK bit is set if the interrupt is mapped to the INTR_DAI_IRQL signal in the SEC using the DAI_IMSK_PRI register.
0 (RC/NW)	RXVALID	Receive Valid. The DAI_IRPTL_L.RXVALID bit is set if the interrupt is mapped to the INTR_DAI_IRQL signal in the SEC using the DAI_IMSK_PRI register.

Shadow Low Priority Interrupt Latch Register

The `DAI_IRPTL_LS` register is the shadow register of the `DAI_IRPTL_L` register. Its content is the same as the `DAI_IRPTL_L` register. Reading the `DAI_IRPTL_LS` register does not affect its contents while reading the contents of the `DAI_IRPTL_L` registers clears it.

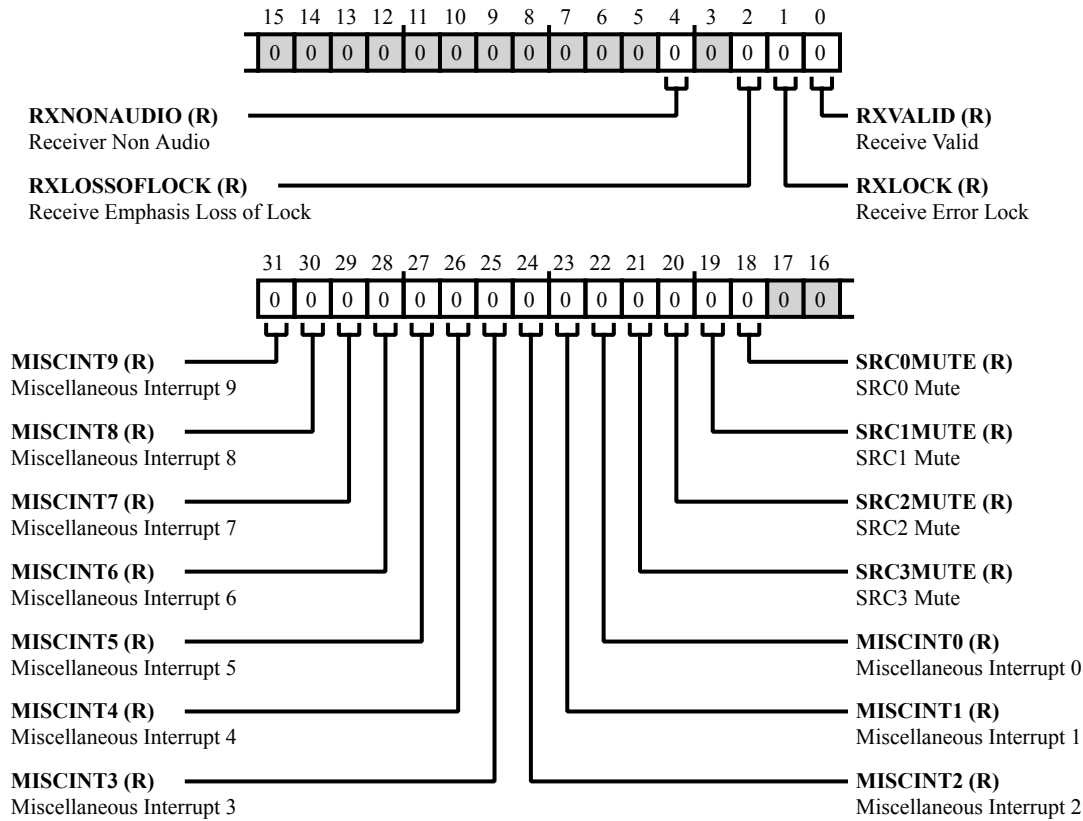


Figure 20-41: `DAI_IRPTL_LS` Register Diagram

Table 20-55: `DAI_IRPTL_LS` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/NW)	MISCINT9	Miscellaneous Interrupt 9. The <code>DAI_IRPTL_LS.MISCINT9</code> bit is the shadow of the <code>DAI_IRPTL_L.MISCINT9</code> bit and contains the same content. Reading the <code>DAI_IRPTL_LS.MISCINT9</code> bit does not affect its contents while reading the <code>DAI_IRPTL_L.MISCINT9</code> bit clears it.
30 (R/NW)	MISCINT8	Miscellaneous Interrupt 8. The <code>DAI_IRPTL_LS.MISCINT8</code> bit is the shadow of the <code>DAI_IRPTL_L.MISCINT8</code> bit and contains the same content. Reading the <code>DAI_IRPTL_LS.MISCINT8</code> bit does not affect its contents while reading the <code>DAI_IRPTL_L.MISCINT8</code> bit clears it.

Table 20-55: DAI_IRPTL_LS Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
29 (R/NW)	MISCINT7	Miscellaneous Interrupt 7. The DAI_IRPTL_LS.MISCINT7 bit is the shadow of the DAI_IRPTL_L.MISCINT7 bit and contains the same content. Reading the DAI_IRPTL_LS.MISCINT7 bit does not affect its contents while reading the DAI_IRPTL_L.MISCINT7 bit clears it.
28 (R/NW)	MISCINT6	Miscellaneous Interrupt 6. The DAI_IRPTL_LS.MISCINT6 bit is the shadow of the DAI_IRPTL_L.MISCINT6 bit and contains the same content. Reading the DAI_IRPTL_LS.MISCINT6 bit does not affect its contents while reading the DAI_IRPTL_L.MISCINT6 bit clears it.
27 (R/NW)	MISCINT5	Miscellaneous Interrupt 5. The DAI_IRPTL_LS.MISCINT5 bit is the shadow of the DAI_IRPTL_L.MISCINT5 bit and contains the same content. Reading the DAI_IRPTL_LS.MISCINT5 bit does not affect its contents while reading the DAI_IRPTL_L.MISCINT5 bit clears it.
26 (R/NW)	MISCINT4	Miscellaneous Interrupt 4. The DAI_IRPTL_LS.MISCINT4 bit is the shadow of the DAI_IRPTL_L.MISCINT4 bit and contains the same content. Reading the DAI_IRPTL_LS.MISCINT4 bit does not affect its contents while reading the DAI_IRPTL_L.MISCINT4 bit clears it.
25 (R/NW)	MISCINT3	Miscellaneous Interrupt 3. The DAI_IRPTL_LS.MISCINT3 bit is the shadow of the DAI_IRPTL_L.MISCINT3 bit and contains the same content. Reading the DAI_IRPTL_LS.MISCINT3 bit does not affect its contents while reading the DAI_IRPTL_L.MISCINT3 bit clears it.
24 (R/NW)	MISCINT2	Miscellaneous Interrupt 2. The DAI_IRPTL_LS.MISCINT2 bit is the shadow of the DAI_IRPTL_L.MISCINT2 bit and contains the same content. Reading the DAI_IRPTL_LS.MISCINT2 bit does not affect its contents while reading the DAI_IRPTL_L.MISCINT2 bit clears it.
23 (R/NW)	MISCINT1	Miscellaneous Interrupt 1. The DAI_IRPTL_LS.MISCINT1 bit is the shadow of the DAI_IRPTL_L.MISCINT1 bit and contains the same content. Reading the DAI_IRPTL_LS.MISCINT1 bit does not affect its contents while reading the DAI_IRPTL_L.MISCINT1 bit clears it.

Table 20-55: DAI_IRPTL_LS Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
22 (R/NW)	MISCINT0	Miscellaneous Interrupt 0. The DAI_IRPTL_LS.MISCINT0 bit is the shadow of the DAI_IRPTL_L.MISCINT0 bit and contains the same content. Reading the DAI_IRPTL_LS.MISCINT0 bit does not affect its contents while reading the DAI_IRPTL_L.MISCINT0 bit clears it.
21 (R/NW)	SRC3MUTE	SRC3 Mute. The DAI_IRPTL_LS.SRC3MUTE bit is the shadow of the DAI_IRPTL_L.SRC3MUTE bit and contains the same content. Reading the DAI_IRPTL_LS.SRC3MUTE bit does not affect its contents while reading the DAI_IRPTL_L.SRC3MUTE bit clears it.
20 (R/NW)	SRC2MUTE	SRC2 Mute. The DAI_IRPTL_LS.SRC2MUTE bit is the shadow of the DAI_IRPTL_L.SRC2MUTE bit and contains the same content. Reading the DAI_IRPTL_LS.SRC2MUTE bit does not affect its contents while reading the DAI_IRPTL_L.SRC2MUTE bit clears it.
19 (R/NW)	SRC1MUTE	SRC1 Mute. The DAI_IRPTL_LS.SRC1MUTE bit is the shadow of the DAI_IRPTL_L.SRC1MUTE bit and contains the same content. Reading the DAI_IRPTL_LS.SRC1MUTE bit does not affect its contents while reading the DAI_IRPTL_L.SRC1MUTE bit clears it.
18 (R/NW)	SRC0MUTE	SRC0 Mute. The DAI_IRPTL_LS.SRC0MUTE bit is the shadow of the DAI_IRPTL_L.SRC0MUTE bit and contains the same content. Reading the DAI_IRPTL_LS.SRC0MUTE bit does not affect its contents while reading the DAI_IRPTL_L.SRC0MUTE bit clears it.
4 (R/NW)	RXNONAUDIO	Receiver Non Audio. The DAI_IRPTL_LS.RXNONAUDIO bit is the shadow of the DAI_IRPTL_L.RXNONAUDIO bit and contains the same content. Reading the DAI_IRPTL_LS.RXNONAUDIO bit does not affect its contents while reading the DAI_IRPTL_L.RXNONAUDIO bit clears it.
2 (R/NW)	RXLOSSOFLOCK	Receive Emphasis Loss of Lock. The DAI_IRPTL_LS.RXLOSSOFLOCK bit is the shadow of DAI_IRPTL_L.RXLOSSOFLOCK bit and contains the same content. Reading this bit does not affect its content. Reading the DAI_IRPTL_L.RXLOSSOFLOCK bit clears it.
1 (R/NW)	RXLOCK	Receive Error Lock. The DAI_IRPTL_LS.RXLOCK bit is the shadow of DAI_IRPTL_L.RXLOCK bit and contains the same content. Reading this bit does not affect its content. Reading the DAI_IRPTL_L.RXLOCK bit clears it.

Table 20-55: DAI_IRPTL_LS Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/NW)	RXVALID	Receive Valid. The <code>DAI_IRPTL_LS.RXVALID</code> bit is the shadow of <code>DAI_IRPTL_L.RXVALID</code> bit and contains the same content. Reading this bit does not affect its content. Reading the <code>DAI_IRPTL_L.RXVALID</code> bit clears it.

Miscellaneous Control Register 0

The `DAI_MISC0` register allows programs to route to the DAI interrupt latch, PBEN input routing, or input signal inversion. This register belongs to group E which routes control signals and provides a means of connecting signals between groups.

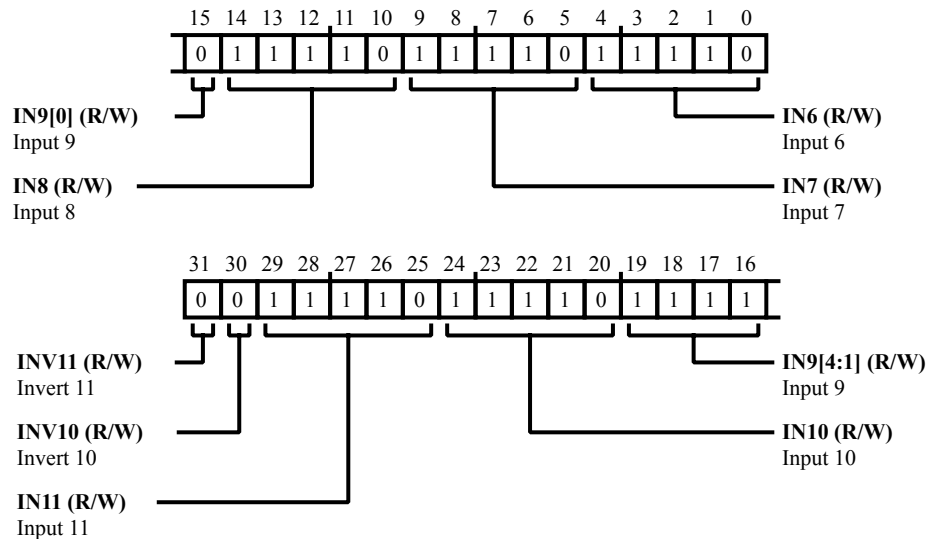


Figure 20-42: DAI_MISC0 Register Diagram

Table 20-56: DAI_MISC0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	INV11	Invert 11. The <code>DAI_MISC0.INV11</code> bit field inverts miscellaneous A5 input.
30 (R/W)	INV10	Invert 10. The <code>DAI_MISC0.INV10</code> bit field inverts miscellaneous A4 input.
29:25 (R/W)	IN11	Input 11. The <code>DAI_MISC0.IN11</code> bit field configures miscellaneous A5 input.
24:20 (R/W)	IN10	Input 10. The <code>DAI_MISC0.IN10</code> bit field configures miscellaneous A4 input.
19:15 (R/W)	IN9	Input 9. The <code>DAI_MISC0.IN9</code> bit field configures miscellaneous A3 input/miscellaneous Interrupt 9(DAI interrupt 31).
14:10 (R/W)	IN8	Input 8. The <code>DAI_MISC0.IN8</code> bit field configures miscellaneous A2 input/miscellaneous Interrupt 8(DAI interrupt 30).

Table 20-56: DAI_MISC0 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
9:5 (R/W)	IN7	Input 7. The <code>DAI_MISC0.IN7</code> bit field configures miscellaneous A1 input/miscellaneous Interrupt 7(DAI interrupt 29).
4:0 (R/W)	IN6	Input 6. The <code>DAI_MISC0.IN6</code> bit field configures miscellaneous A0 input/miscellaneous Interrupt 6(DAI interrupt 28).

Miscellaneous Control Register 1

The `DAI_MISC1` register allows programs to route to the DAI interrupt latch, PBEN input routing, or input signal inversion. This register belongs to group E which routes control signals and provides a means of connecting signals between groups.

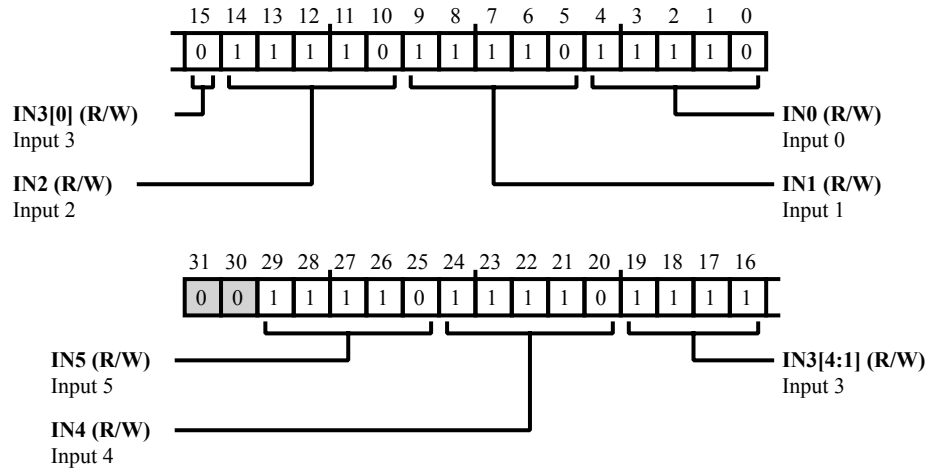


Figure 20-43: DAI_MISC1 Register Diagram

Table 20-57: DAI_MISC1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:25 (R/W)	IN5	Input 5. The <code>DAI_MISC1</code> . IN5 bit field configures miscellaneous Interrupt 5(DAI interrupt 27).
24:20 (R/W)	IN4	Input 4. The <code>DAI_MISC1</code> . IN4 bit field configures miscellaneous Interrupt 4(DAI interrupt 26).
19:15 (R/W)	IN3	Input 3. The <code>DAI_MISC1</code> . IN3 bit field configures miscellaneous Interrupt 3(DAI interrupt 25).
14:10 (R/W)	IN2	Input 2. The <code>DAI_MISC1</code> . IN2 bit field configures miscellaneous Interrupt 2(DAI interrupt 24).
9:5 (R/W)	IN1	Input 1. The <code>DAI_MISC1</code> . IN1 bit field configures miscellaneous Interrupt 1(DAI interrupt 23).

Table 20-57: DAI_MISC1 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
4:0 (R/W)	IN0	Input 0. The <code>DAI_MISC1.IN0</code> bit field configures miscellaneous interrupt 0(DAI interrupt 22).

Pin Buffer Enable Register 0

The `DAI_PBEN0` register routes signals to the pin enables, and the value of these signals determines if a DAI pin is used as an output or an input.

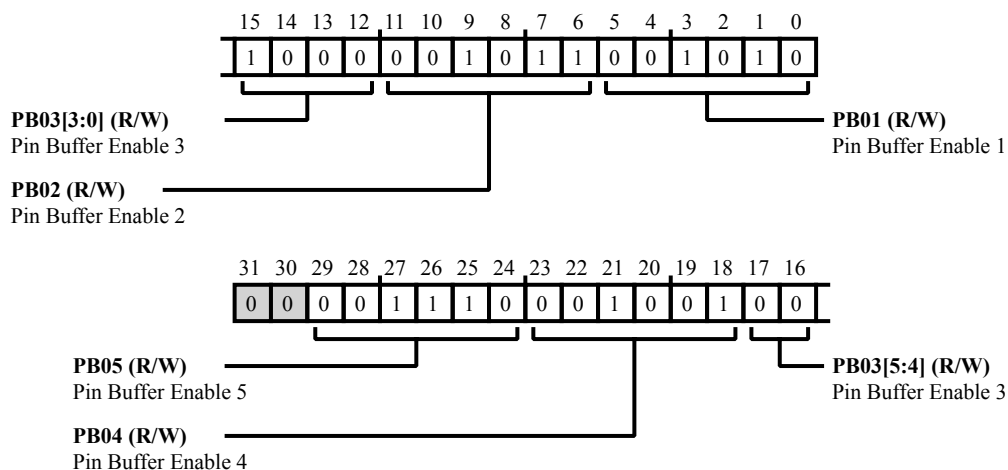


Figure 20-44: `DAI_PBEN0` Register Diagram

Table 20-58: `DAI_PBEN0` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:24 (R/W)	PB05	Pin Buffer Enable 5. The <code>DAI_PBEN0 . PB05</code> bit field is the pin buffer enable for DAI port 5.
23:18 (R/W)	PB04	Pin Buffer Enable 4. The <code>DAI_PBEN0 . PB04</code> bit field is the pin buffer enable for DAI port 4.
17:12 (R/W)	PB03	Pin Buffer Enable 3. The <code>DAI_PBEN0 . PB03</code> bit field is the pin buffer enable for DAI port 3.
11:6 (R/W)	PB02	Pin Buffer Enable 2. The <code>DAI_PBEN0 . PB02</code> bit field is the pin buffer enable for DAI port 2.
5:0 (R/W)	PB01	Pin Buffer Enable 1. The <code>DAI_PBEN0 . PB01</code> bit field is the pin buffer enable for DAI port 1.

Pin Buffer Enable Register 1

The `DAI_PBEN1` register routes signals to the pin enables, and the value of these signals determines if a DAI pin is used as an output or an input.

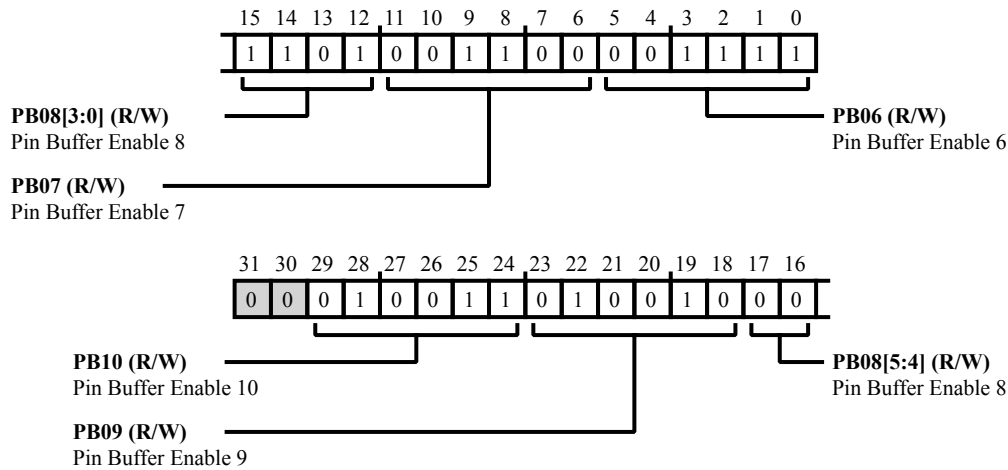


Figure 20-45: DAI_PBEN1 Register Diagram

Table 20-59: DAI_PBEN1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:24 (R/W)	PB10	Pin Buffer Enable 10. The <code>DAI_PBEN1 . PB10</code> bit field is the pin buffer enable for DAI port 10.
23:18 (R/W)	PB09	Pin Buffer Enable 9. The <code>DAI_PBEN1 . PB09</code> bit field is the pin buffer enable for DAI port 9.
17:12 (R/W)	PB08	Pin Buffer Enable 8. The <code>DAI_PBEN1 . PB08</code> bit field is the pin buffer enable for DAI port 8.
11:6 (R/W)	PB07	Pin Buffer Enable 7. The <code>DAI_PBEN1 . PB07</code> bit field is the pin buffer enable for DAI port 7.
5:0 (R/W)	PB06	Pin Buffer Enable 6. The <code>DAI_PBEN1 . PB06</code> bit field is the pin buffer enable for DAI port 6.

Pin Buffer Enable Register 2

The `DAI_PBEN2` register routes signals to the pin enables, and the value of these signals determines if a DAI pin is used as an output or an input.

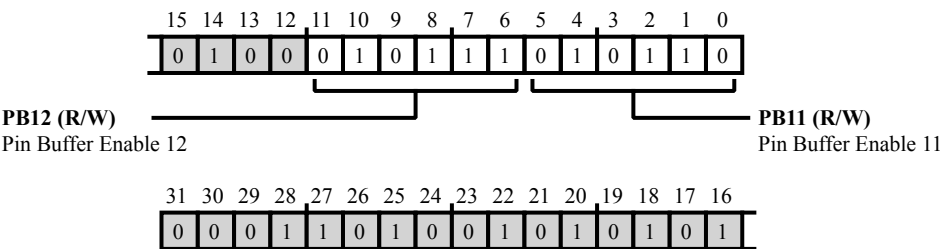


Figure 20-46: `DAI_PBEN2` Register Diagram

Table 20-60: `DAI_PBEN2` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
11:6 (R/W)	PB12	Pin Buffer Enable 12. The <code>DAI_PBEN2 . PB12</code> bit field is the pin buffer enable for DAI port 12.
5:0 (R/W)	PB11	Pin Buffer Enable 11. The <code>DAI_PBEN2 . PB11</code> bit field is the pin buffer enable for DAI port 11.

Pin Buffer Enable Register 3

The `DAI_PBEN3` register routes signals to the pin enables, and the value of these signals determines if a DAI pin is used as an output or an input.

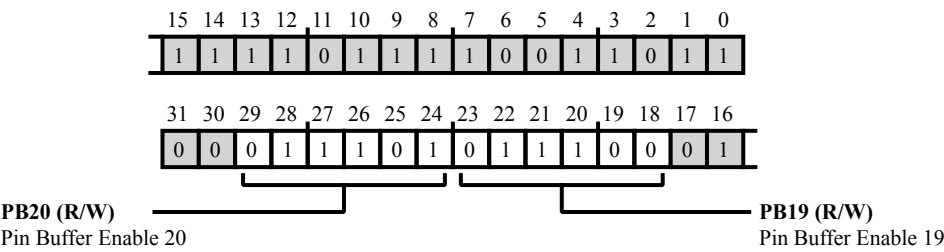


Figure 20-47: DAI_PBEN3 Register Diagram

Table 20-61: DAI_PBEN3 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:24 (R/W)	PB20	Pin Buffer Enable 20. The <code>DAI_PBEN3.PB20</code> bit field is the pin buffer enable for DAI port 20.
23:18 (R/W)	PB19	Pin Buffer Enable 19. The <code>DAI_PBEN3.PB19</code> bit field is the pin buffer enable for DAI port 19.

Pin Buffer Assignment Register 0

The `DAI_PIN0` register routes physical pins that are connected to a bonded pad.

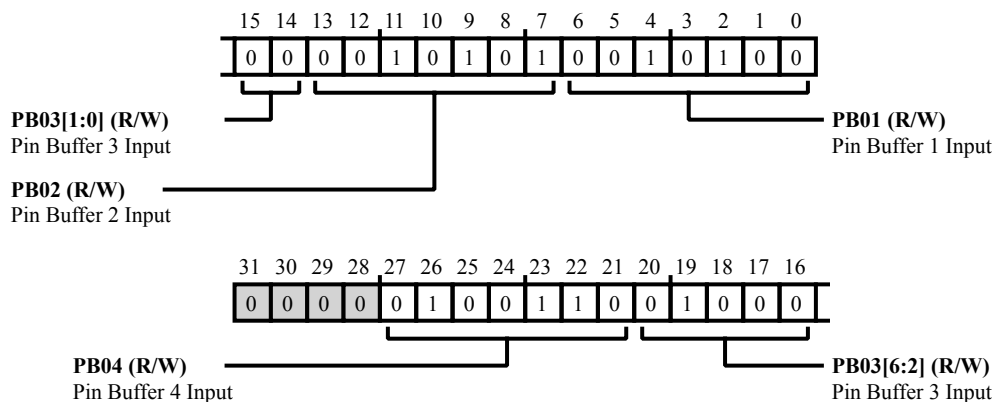


Figure 20-48: `DAI_PIN0` Register Diagram

Table 20-62: `DAI_PIN0` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
27:21 (R/W)	PB04	Pin Buffer 4 Input. <code>DAI_PIN0.PB04</code> holds the source signal assignment that will be routed to the <code>DAI_PIN0.PB04</code> destination. Refer to the Group D Signals table for source and destination mappings.
20:14 (R/W)	PB03	Pin Buffer 3 Input. <code>DAI_PIN0.PB03</code> holds the source signal assignment that will be routed to the <code>DAI_PIN0.PB03</code> destination. Refer to the Group D Signals table for source and destination mappings.
13:7 (R/W)	PB02	Pin Buffer 2 Input. <code>DAI_PIN0.PB02</code> holds the source signal assignment that will be routed to the <code>DAI_PIN0.PB02</code> destination. Refer to the Group D Signals table for source and destination mappings.
6:0 (R/W)	PB01	Pin Buffer 1 Input. <code>DAI_PIN0.PB01</code> holds the source signal assignment that will be routed to the <code>DAI_PIN0.PB01</code> destination. Refer to the Group D Signals table for source and destination mappings.

Pin Buffer Assignment Register 1

The `DAI_PIN1` register routes physical pins that are connected to a bonded pad.

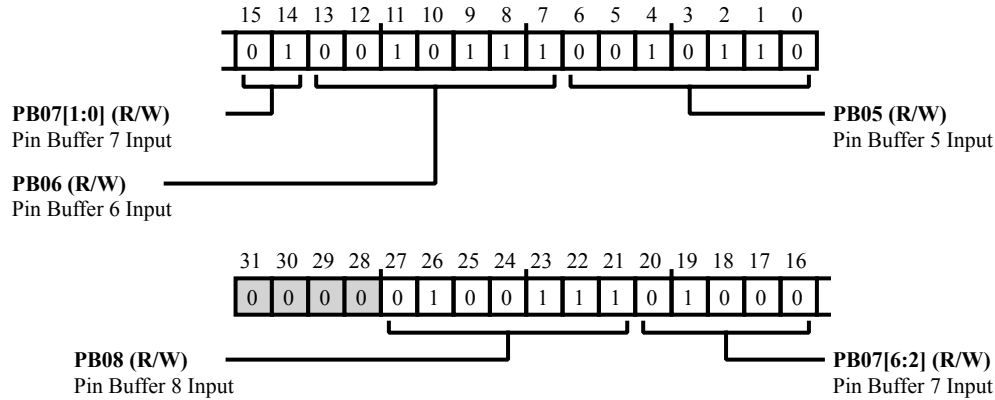


Figure 20-49: DAI_PIN1 Register Diagram

Table 20-63: DAI_PIN1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
27:21 (R/W)	PB08	Pin Buffer 8 Input. <code>DAI_PIN1 . PB08</code> holds the source signal assignment that will be routed to the <code>DAI_PIN1 . PB08</code> destination. Refer to the Group D Signals table for source and destination mappings.
20:14 (R/W)	PB07	Pin Buffer 7 Input. <code>DAI_PIN1 . PB07</code> holds the source signal assignment that will be routed to the <code>DAI_PIN1 . PB07</code> destination. Refer to the Group D Signals table for source and destination mappings.
13:7 (R/W)	PB06	Pin Buffer 6 Input. <code>DAI_PIN1 . PB06</code> holds the source signal assignment that will be routed to the <code>DAI_PIN1 . PB06</code> destination. Refer to the Group D Signals table for source and destination mappings.
6:0 (R/W)	PB05	Pin Buffer 5 Input. <code>DAI_PIN1 . PB05</code> holds the source signal assignment that will be routed to the <code>DAI_PIN1 . PB05</code> destination. Refer to the Group D Signals table for source and destination mappings.

Pin Buffer Assignment Register 2

The `DAI_PIN2` register routes physical pins that are connected to a bonded pad.

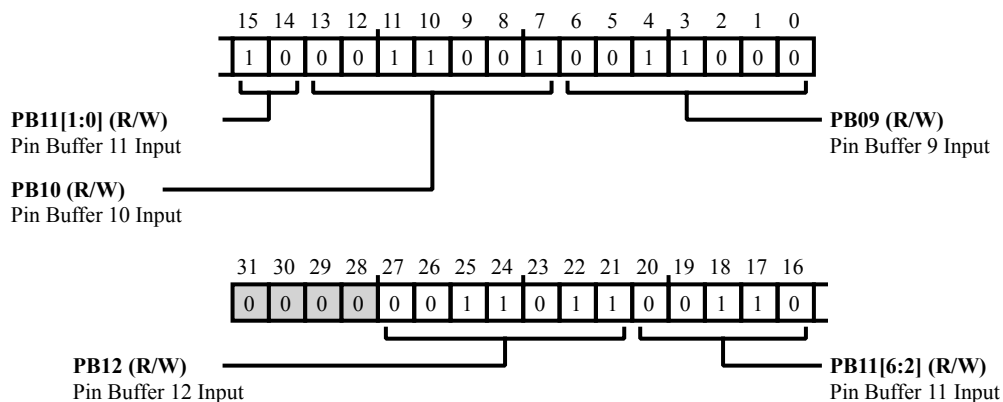


Figure 20-50: DAI_PIN2 Register Diagram

Table 20-64: DAI_PIN2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
27:21 (R/W)	PB12	Pin Buffer 12 Input. <code>DAI_PIN2.PB12</code> holds the source signal assignment that will be routed to the <code>DAI_PIN2.PB12</code> destination. Refer to the Group D Signals table for source and destination mappings.
20:14 (R/W)	PB11	Pin Buffer 11 Input. <code>DAI_PIN2.PB11</code> holds the source signal assignment that will be routed to the <code>DAI_PIN2.PB11</code> destination. Refer to the Group D Signals table for source and destination mappings.
13:7 (R/W)	PB10	Pin Buffer 10 Input. <code>DAI_PIN2.PB10</code> holds the source signal assignment that will be routed to the <code>DAI_PIN2.PB10</code> destination. Refer to the Group D Signals table for source and destination mappings.
6:0 (R/W)	PB09	Pin Buffer 9 Input. <code>DAI_PIN2.PB09</code> holds the source signal assignment that will be routed to the <code>DAI_PIN2.PB09</code> destination. Refer to the Group D Signals table for source and destination mappings.

Pin Buffer Assignment Register 4

The `DAI_PIN4` register routes physical pins that are connected to a bonded pad.

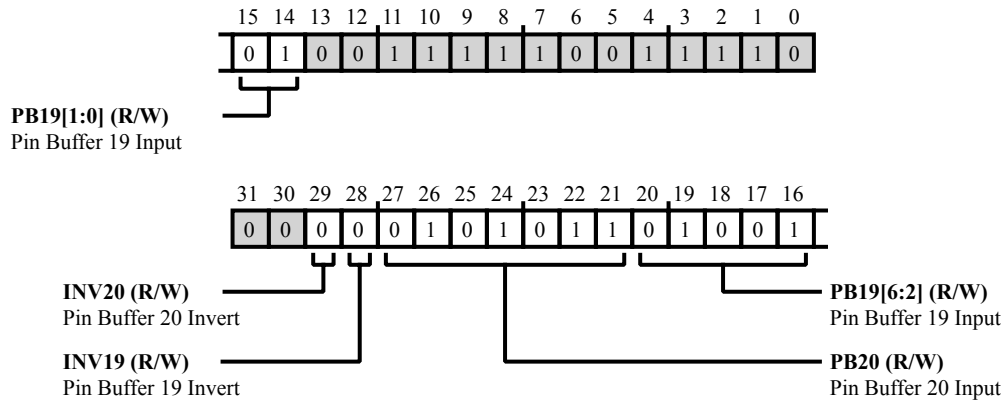


Figure 20-51: DAI_PIN4 Register Diagram

Table 20-65: DAI_PIN4 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29 (R/W)	INV20	Pin Buffer 20 Invert. <code>DAI_PIN4</code> . <code>INV20</code> holds the source signal assignment that will be routed to the <code>DAI_PIN4</code> . <code>INV20</code> destination. Refer to the Group D Signals table for source and destination mappings.
28 (R/W)	INV19	Pin Buffer 19 Invert. <code>DAI_PIN4</code> . <code>INV19</code> holds the source signal assignment that will be routed to the <code>DAI_PIN4</code> . <code>INV19</code> destination. Refer to the Group D Signals table for source and destination mappings.
27:21 (R/W)	PB20	Pin Buffer 20 Input. <code>DAI_PIN4</code> . <code>PB20</code> holds the source signal assignment that will be routed to the <code>DAI_PIN4</code> . <code>PB20</code> destination. Refer to the Group D Signals table for source and destination mappings.
20:14 (R/W)	PB19	Pin Buffer 19 Input. <code>DAI_PIN4</code> . <code>PB19</code> holds the source signal assignment that will be routed to the <code>DAI_PIN4</code> . <code>PB19</code> destination. Refer to the Group D Signals table for source and destination mappings.

Pin Status Register

The `DAI_PIN_STAT` register bits indicate the status (signal high or low) for each pin. The individual bit reads 1 if the signal to this pin is high and reads 0 if signal to this pin is low.

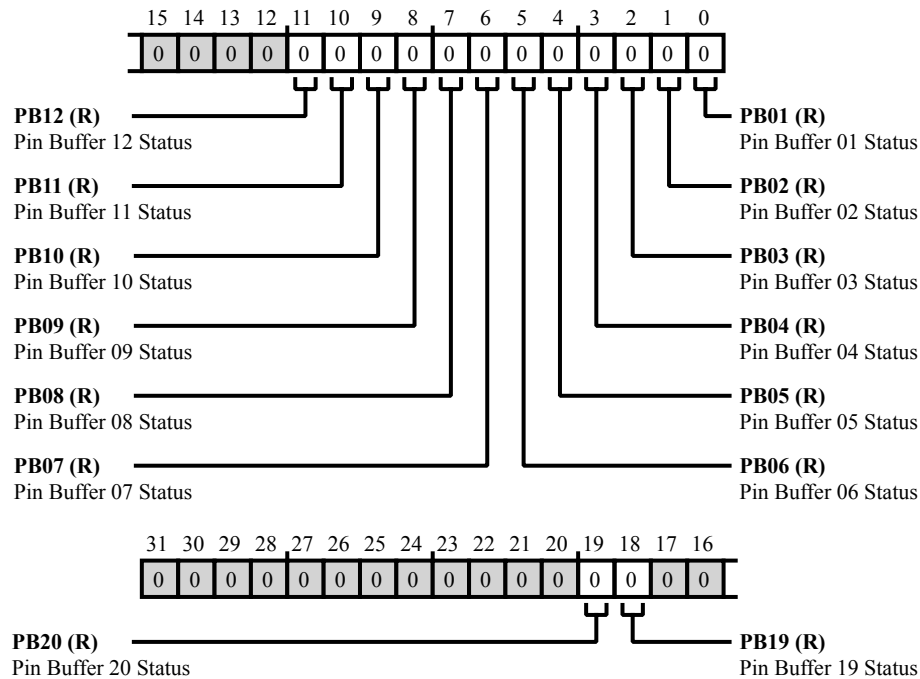


Figure 20-52: `DAI_PIN_STAT` Register Diagram

Table 20-66: `DAI_PIN_STAT` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
19 (R/NW)	PB20	Pin Buffer 20 Status. The <code>DAI_PIN_STAT</code> . PB20 bit reads 1 if the signal to this pin is high and reads 0 if signal to this pin is low.
18 (R/NW)	PB19	Pin Buffer 19 Status. The <code>DAI_PIN_STAT</code> . PB19 bit reads 1 if the signal to this pin is high and reads 0 if signal to this pin is low.
11 (R/NW)	PB12	Pin Buffer 12 Status. The <code>DAI_PIN_STAT</code> . PB12 bit reads 1 if the signal to this pin is high and reads 0 if signal to this pin is low.
10 (R/NW)	PB11	Pin Buffer 11 Status. The <code>DAI_PIN_STAT</code> . PB11 bit reads 1 if the signal to this pin is high and reads 0 if signal to this pin is low.

Table 20-66: DAI_PIN_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
9 (R/NW)	PB10	Pin Buffer 10 Status. The DAI_PIN_STAT.PB10 bit reads 1 if the signal to this pin is high and reads 0 if signal to this pin is low.
8 (R/NW)	PB09	Pin Buffer 09 Status. The DAI_PIN_STAT.PB09 bit reads 1 if the signal to this pin is high and reads 0 if signal to this pin is low.
7 (R/NW)	PB08	Pin Buffer 08 Status. The DAI_PIN_STAT.PB08 bit reads 1 if the signal to this pin is high and reads 0 if signal to this pin is low.
6 (R/NW)	PB07	Pin Buffer 07 Status. The DAI_PIN_STAT.PB07 bit reads 1 if the signal to this pin is high and reads 0 if signal to this pin is low.
5 (R/NW)	PB06	Pin Buffer 06 Status. The DAI_PIN_STAT.PB06 bit reads 1 if the signal to this pin is high and reads 0 if signal to this pin is low.
4 (R/NW)	PB05	Pin Buffer 05 Status. The DAI_PIN_STAT.PB05 bit reads 1 if the signal to this pin is high and reads 0 if signal to this pin is low.
3 (R/NW)	PB04	Pin Buffer 04 Status. The DAI_PIN_STAT.PB04 bit reads 1 if the signal to this pin is high and reads 0 if signal to this pin is low.
2 (R/NW)	PB03	Pin Buffer 03 Status. The DAI_PIN_STAT.PB03 bit reads 1 if the signal to this pin is high and reads 0 if signal to this pin is low.
1 (R/NW)	PB02	Pin Buffer 02 Status. The DAI_PIN_STAT.PB02 bit reads 1 if the signal to this pin is high and reads 0 if signal to this pin is low.
0 (R/NW)	PB01	Pin Buffer 01 Status. The DAI_PIN_STAT.PB01 bit reads 1 if the signal to this pin is high and reads 0 if signal to this pin is low.

21 Serial Port (SPORT)

The programmable serial ports (SPORTs) support various protocols for serial data communication and provide a glueless hardware interface to many industry-standard data converters and codecs. They have high data rates and dual half-duplex datapaths and are ideal for establishing a direct serial connection among two or more processors in a multiprocessor system, as many processors provide compatible serial interfaces.

The SPORT top module consists of two half SPORTs with identical functionality and programming requirements. Each half SPORT can be independently configured as either a transmitter or receiver and can be coupled with the other half SPORT within the same SPORT top module. Further, each half SPORT provides two synchronous half-duplex data lines to double the total supported throughput. As such, a single SPORT top module can be used to provide up to four unidirectional or up to two full-duplex data streams. Further channels are possible as well, but utilization of multiple SPORT top modules is required, thus requiring external connections to provide a common time base.

Features

An individual SPORT top module consists of two independently configurable SPORT halves with identical functionality. These SPORT halves offer the following features:

- Up to two bidirectional data lines - each half SPORT supports up to two transmit or receive channels, thus allowing two unidirectional streams into or out of each half SPORT and providing greater flexibility for serial communications. If full-duplex functionality is desired, two SPORT halves can be combined to enable dual-stream bidirectional communication.
- Six operating modes:
 1. Standard DSP serial mode
 2. I²S mode
 3. Left-Justified mode
 4. Right-Justified mode
 5. Multichannel (TDM) mode
 6. Packed mode

- Supports internally or externally generated clock.
- Support for both even and odd SCLK to SPORT clock (SPORT_CLK) ratios. If both data lines of a half SPORT are active, the maximum throughput is 2 x SPORT_CLK bps.
- Configurable rising or falling edge of the SPORT_CLK for driving and sampling data and frame syncs.
- Gated clock mode support for internally or externally generated clocks in DSP serial mode and stereo modes (left-justified and I²S mode).
- Supports frameless operation.
- Supports internally or externally generated frame sync signals.
- Programmable frame sync polarity.
- Programmable frame sync timing (synchronous to data or 1 SPORT clock in advance of it).
- Detection of prematurely received external frame syncs (with optional interrupt request generation).
- Programmable level-/edge-sensitivity for external frame syncs.
- Programmable (4–32-bit) data length, either in most significant bit (MSB) first or in least significant bit (LSB) first format, with optional sign-extension on received data.
- Optional 16-bit to 32-bit word packing (as receiver) and 32-bit to 16-bit word unpacking (as transmitter).
- Support for A-law and μ -law compression/decompression hardware companding, according to the G.711 specification, on transmitted/received words in all operating modes.
- Transmit underrun and receive overflow detection (with optional interrupt request generation).
- TDM mode transfers data on 128 contiguous channels from a stream of up to 1024 total channels (useful for H.100/H.110 and other telephony interfaces as a network communication scheme for multiple processors).
- Performs interrupt-driven, single word core transfers to and from on-chip or off-chip memory.
- Dedicated DMA channel for each SPORT half supporting autobuffer (for a repeated, identical range of transfers) and numerous descriptor-based (individual or repeated ranges of transfers with differing DMA parameters) modes.
- Generator and receiver trigger functionality.
- Unique transfer finish interrupt (TFI) signaling when the last transmit word is fully out of the transmit shift register.
- Multiplexer to internally connect critical timing signals between SPORT halves.
- Support for Global Sport Enable functionality, where enabling multiple SPORTs need to be synchronized so that all of them start and/or end at the same time.

Signal Descriptions

Each half SPORT module has five dedicated signals, as described in the *SPORT Signal Descriptions* table. The actual pin name varies with different SPORT halves. Individual SPORT halves do not share any of its signals across the pair that comprises the SPORT top module; however, it is possible to connect the clock and frame sync signals between the SPORT half pair, as explained in the [Multiplexer Logic](#) section.

All of the SPORT signals are multiplexed on the PORT pins, possibly sharing functionality with other peripherals on the device. By default, the PORT pins are in GPIO mode and must be reconfigured for SPORT functionality by setting the appropriate bits in the [PORT_FER](#) and [PORT_MUX](#) registers. Consult the processor data sheet for details regarding which ports the SPORT signals are available and to ensure to configure the [PORT_MUX](#) register before the [PORT_FER](#) register.

Table 21-1: SPORT Signal Descriptions

Internal Node	Direction	Description
SPORTx_CLK	I/O	Transmit or receive serial clock. Data and frame syncs are driven or sampled on this clock's edges. This signal can be either internally or externally generated.
SPORTx_FS	I/O	Transmit or receive frame sync. The frame sync pulse initiates shifting of serial data. This signal is either internally or externally generated.
SPORTx_D0	I/O	Primary transmit or receive data channel. This signal can be configured as an output to transmit serial data or as an input to receive serial data.
SPORTx_D1	I/O	Secondary transmit or receive data channel. This signal can be configured as an output to transmit serial data or as an input to receive serial data.
SPORTx_TDV	O	Multichannel transmit data valid. This signal is only active in multichannel transmit mode and is asserted during enabled slots, as defined by the channel selection registers (SPORT_CS0_A through SPORT_CS3_B).

The data channel signals are transmit signals when the serial port is configured in transmit mode ([SPORT_CTL_A.SPTRAN](#) = 1). They are receive signals when the serial port is configured in receive mode ([SPORT_CTL_A.SPTRAN](#) = 0). The following sections further describe the SPORT signals.

NOTE: These sections explicitly refer to the registers associated with half SPORT A, but the same concepts also apply to half SPORT B.

Serial Clock

The serial port clock ([SPORT_ACLK](#)) is either a receive serial clock or a transmit serial clock, depending on the transfer direction ([SPORT_CTL_A.SPTRAN](#)), governing when the data bits are serially shifted into or out of the SPORT and when the frame sync signal is driven (in internal frame sync mode) or sampled (in external frame sync mode). It can be internally generated from the processor's system clock ([SCLK0_0](#)) or externally provided. If the half SPORT is configured in internal clock mode ([SPORT_CTL_A.ICLK](#) = 1), then the [SPORT_DIV_A.CLKDIV](#) field specifies the divisor applied to [SCLK0_0](#) to generate the SPORT clock. As it is a

16-bit divisor, a wide range of serial clock rates is possible. Use the following equation to calculate the serial clock frequency:

$$\text{SPORT_ACLK} = [\text{SCLK0_0} \div (\text{SPORT_DIV_A.CLKDIV} + 1)]$$

From this, the following equation can be used to determine the value of `SPORT_DIV_A.CLKDIV`, given the `SCLK0_0` frequency and the desired frequency of the SPORT clock:

$$\text{SPORT_DIV_A.CLKDIV} = [(\text{SCLK0_0} \div \text{SPORT_ACLK}) - 1]$$

The half SPORT also supports a 1:1 `SPORT_ACLK` to `SCLK0_0` ratio (per the equations above, program the clock divisor field to zero). In this case, the resulting SPORT clock frequency is equal to `SCLK0_0`.

NOTE: Be careful not to exceed the maximum `SPORT_ACLK` frequency specified in the processor data sheet.

In certain operating modes, the SPORT can be configured to generate a gated clock, which is active only during valid data. In some applications, a SPORT uses it to generate a general-purpose clock in the system. In this case, enable the SPORT with the appropriate `SPORT_DIV_A.CLKDIV` divisor field in internal clock mode.

If a SPORT is configured in external clock mode (`SPORT_CTL_A.ICLK = 0`), the serial clock is an input signal, thus making the SPORT operate in target mode. In this mode, the `SPORT_DIV_A.CLKDIV` is irrelevant and is ignored. The optional loopback capability provided by the internal SPORT multiplexer (SPMUX) block allows the target SPORT to use the serial clock from the neighboring SPORT half in the same SPORT top module.

An externally-supplied serial clock does not need to be synchronous with the processor clocks. Further, the external clock can be a gated clock, but it must comply with the requirements described in the [Gated Clock Mode](#) section.

Refer to the product data sheet for exact AC timing specifications.

Frame Sync

The SPORT frame sync (`SPORT_AFS`) signal is either a receive frame sync or a transmit frame sync, depending on the transfer direction (`SPORT_CTL_A.SPTRAN`), which is used to determine the start of a new word or frame. When this signal goes active, the serial port starts serially shifting data into or out of the SPORT. The frame sync signal can be internally generated based on its serial clock (`SPORT_ACLK`) or externally provided, as configured by the `SPORT_CTL_A.IFS` bit.

If the half SPORT is configured to generate frame syncs (`SPORT_CTL_A.IFS = 1`), then the `SPORT_DIV_A.FSDIV` field specifies the divisor used to generate the periodic `SPORT_AFS` signal from the SPORT clock. As this is a 16-bit divisor, a wide range of frame sync rates to initiate periodic transfers is possible. Whether the serial clock is internally or externally generated, this divisor is a count of SPORT clock cycles between frame sync pulses, the formula for which is:

$$\text{Number of serial clocks between frame syncs} = (\text{SPORT_DIV_A.FSDIV} + 1)$$

From this, the following equation can be used to determine the value of `SPORT_DIV_A.FSDIV`, given the serial clock frequency and the desired frame sync frequency:

$$\text{SPORT_DIV_A.FSDIV} = [(\text{SPORT_ACLK} \div \text{SPORT_AFS}) - 1]$$

The frame sync is continuously active when `SPORT_DIV_A.FSDIV = 0`. The value of `SPORT_DIV_A.FSDIV` cannot be less than the serial word length minus one (the value of the `SPORT_CTL_A.SLEN` bit field). Failure to adhere to this guideline can cause an external device to abort the current operation or cause other unpredictable results.

NOTE: After enabling the SPORT, the first internal frame sync appears after a delay of `SPORT_DIV_A.FSDIV + 3` SPORT clock cycles.

If a SPORT is configured for external frame syncs (`SPORT_CTL_A.IFS = 0`), then `SPORT_AFS` is an input signal and the `SPORT_DIV_A.FSDIV` field of the [SPORT_DIV_A](#) register is irrelevant and ignored. By default, this external signal is level-sensitive, but it can be configured as an edge-sensitive signal by setting the `SPORT_CTL_A.FSED` bit. The frame sync is expected to be synchronous with the serial clock. If not, it must meet the timing requirements that appear in the data sheet.

The SPORT can be used as a counter for dividing an external clock to generate periodic pulses or periodic interrupts. To do so, enable the SPORT with the appropriate `SPORT_DIV_A.FSDIV` divisor field with the SPORT configured for an external clock and internal data-independent frame syncs.

In some of the operating modes, the SPORT can be programmed to treat the frame sync signal as an optional signal by clearing the `SPORT_CTL_A.FSR` bit. Even with this bit cleared, the SPORT requires a single frame sync assertion to start the continuous transfers, after which it is ignored (for externally supplied frame syncs) or not generated (for internally-generated frame syncs). Characteristics of the frame sync depend on the settings in the SPORT control registers and the operating mode of the SPORT. For more information, refer to the [SPORT_CTL_A](#) register.

Data Signals

Each half SPORT has two bidirectional data lines known as the primary (`SPORT_AD0`) and secondary (`SPORT_AD1`) data channels. Both data lines can be configured as either transmitters or receivers using the `SPORT_CTL_A.SPTRAN` bit, thus permitting dual unidirectional data streams to increase the data throughput of the SPORT.

NOTE: Configuring one transmit data channel and one receive data channel on a single half SPORT is not supported.

The primary and secondary data lines can be individually enabled or disabled using the `SPORT_CTL_A.SPENPRI` and the `SPORT_CTL_A.SPENSEC` bits, respectively. However, if using both, enable or disable them concurrently. These data lines operate in a synchronous manner (sharing a clock and frame sync) but have separate datapaths with unique data buffers, shift registers and optional companding logic. All of the SPORT control settings are common for both channels, but the single DMA channel per half SPORT serves both the primary and secondary data channels.

When a SPORT is configured in multichannel transmit mode, the data pins three-state during inactive channel slots, thus allowing multiple transmitters to operate on the same bus with different active channels.

See the [Architectural Concepts](#) section for more details about data transfer operation.

Transmit Data Valid Signal

The transmit data valid (`SPORT_ATDV`) signal is available only in transmit multichannel modes (including packed mode). It is driven active during enabled multichannel slots, and it is driven inactive during the disabled channels. In other words, the `SPORT_ATDV` signal is active when data is being driven to the data pins and inactive when the data pins are being three-stated. As such, the `SPORT_ATDV` signal can serve as an output-enable signal for the data transmit pin.

SRU Programming

The SPORT uses the SRU (signal routing unit) to connect the SPORT data, serial clock, frame sync, and external sync (if external synchronization is required). Inputs also must be routed through the SRU. Program the corresponding SRU registers to connect the outputs to the required destinations. For details of the routing, see [DAI Routing Capabilities](#) in the *Digital Audio Interface (DAI)* chapter.

SRU SPORT Receive Controller

If the SPORT is receiver, it must feed its controller output clock back to its input clock. This is required to trigger the SPORT's state machine. Using SPORT 0A as an example receiver, programs should route `SPT0_ACLK_O` to `SPT0_ACLK_I`. This is not required if the SPORT is operating as a transmitter in controller mode.

Functional Description

The following sections provide general information about the functionality of the processor's serial ports:

- [Architectural Concepts](#)
- [Data Types and Companding](#)
- [Transmit Path](#)
- [Receive Path](#)

ADSP-21560_21561_21564_21568 SPORT Register List

The Serial Port (SPORT) controller, with its range of clock and frame synchronization options, supports a variety of serial communication protocols and provides a glueless hardware interface to many industry-standard data converters and CODECs. Each SPORT has two independent halves (A and B), and each half contains two channels (primary and secondary). A set of registers governs SPORT operations. For more information on SPORT functionality, see the SPORT register descriptions.

Table 21-2: ADSP-21560_21561_21564_21568 SPORT Register List

Name	Description
SPORT_CS0_A	Half SPORT 'A' Multichannel 0-31 Select Register
SPORT_CS0_B	Half SPORT 'B' Multichannel 0-31 Select Register

Table 21-2: ADSP-21560_21561_21564_21568 SPORT Register List (Continued)

Name	Description
SPORT_CS1_A	Half SPORT 'A' Multichannel 32-63 Select Register
SPORT_CS1_B	Half SPORT 'B' Multichannel 32-63 Select Register
SPORT_CS2_A	Half SPORT 'A' Multichannel 64-95 Select Register
SPORT_CS2_B	Half SPORT 'B' Multichannel 64-95 Select Register
SPORT_CS3_A	Half SPORT 'A' Multichannel 96-127 Select Register
SPORT_CS3_B	Half SPORT 'B' Multichannel 96-127 Select Register
SPORT_CTL2_A	Half SPORT 'A' Control 2 Register
SPORT_CTL2_B	Half SPORT 'B' Control 2 Register
SPORT_CTL_A	Half SPORT 'A' Control Register
SPORT_CTL_B	Half SPORT 'B' Control Register
SPORT_DIV_A	Half SPORT 'A' Divisor Register
SPORT_DIV_B	Half SPORT 'B' Divisor Register
SPORT_ERR_A	Half SPORT 'A' Error Register
SPORT_ERR_B	Half SPORT 'B' Error Register
SPORT_MCTL_A	Half SPORT 'A' Multichannel Control Register
SPORT_MCTL_B	Half SPORT 'B' Multichannel Control Register
SPORT_MSTAT_A	Half SPORT 'A' Multichannel Status Register
SPORT_MSTAT_B	Half SPORT 'B' Multichannel Status Register
SPORT_RXPRI_A	Half SPORT 'A' Rx Buffer (Primary) Register
SPORT_RXPRI_B	Half SPORT 'B' Rx Buffer (Primary) Register
SPORT_RXSEC_A	Half SPORT 'A' Rx Buffer (Secondary) Register
SPORT_RXSEC_B	Half SPORT 'B' Rx Buffer (Secondary) Register
SPORT_TXPRI_A	Half SPORT 'A' Tx Buffer (Primary) Register
SPORT_TXPRI_B	Half SPORT 'B' Tx Buffer (Primary) Register
SPORT_TXSEC_A	Half SPORT 'A' Tx Buffer (Secondary) Register
SPORT_TXSEC_B	Half SPORT 'B' Tx Buffer (Secondary) Register

ADSP-21560_21561_21564_21568 SPORT Interrupt List

Table 21-3: ADSP-21560_21561_21564_21568 SPORT Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
37	SPORT0_A_DMA	SPORT0 Channel A DMA	Level	0
38	SPORT0_A_STAT	SPORT0 Channel A Status	Level	
39	SPORT0_B_DMA	SPORT0 Channel B DMA	Level	1
40	SPORT0_B_STAT	SPORT0 Channel B Status	Level	
41	SPORT1_A_DMA	SPORT1 Channel A DMA	Level	2
42	SPORT1_A_STAT	SPORT1 Channel A Status	Level	
43	SPORT1_B_DMA	SPORT1 Channel B DMA	Level	3
44	SPORT1_B_STAT	SPORT1 Channel B Status	Level	
45	SPORT2_A_DMA	SPORT2 Channel A DMA	Level	4
46	SPORT2_A_STAT	SPORT2 Channel A Status	Level	
47	SPORT2_B_DMA	SPORT2 Channel B DMA	Level	5
48	SPORT2_B_STAT	SPORT2 Channel B Status	Level	
49	SPORT3_A_DMA	SPORT3 Channel A DMA	Level	6
50	SPORT3_A_STAT	SPORT3 Channel A Status	Level	
51	SPORT3_B_DMA	SPORT3 Channel B DMA	Level	7
52	SPORT3_B_STAT	SPORT3 Channel B Status	Level	
53	SPORT4_A_DMA	SPORT4 Channel A DMA	Level	10
54	SPORT4_A_STAT	SPORT4 Channel A Status	Level	
55	SPORT4_B_DMA	SPORT4 Channel B DMA	Level	11
56	SPORT4_B_STAT	SPORT4 Channel B Status	Level	
57	SPORT5_A_DMA	SPORT5 Channel A DMA	Level	12
58	SPORT5_A_STAT	SPORT5 Channel A Status	Level	
59	SPORT5_B_DMA	SPORT5 Channel B DMA	Level	13
60	SPORT5_B_STAT	SPORT5 Channel B Status	Level	
61	SPORT6_A_DMA	SPORT6 Channel A DMA	Level	14
62	SPORT6_A_STAT	SPORT6 Channel A Status	Level	
63	SPORT6_B_DMA	SPORT6 Channel B DMA	Level	15
64	SPORT6_B_STAT	SPORT6 Channel B Status	Level	
65	SPORT7_A_DMA	SPORT7 Channel A DMA	Level	16
66	SPORT7_A_STAT	SPORT7 Channel A Status	Level	
67	SPORT7_B_DMA	SPORT7 Channel B DMA	Level	17

Table 21-3: ADSP-21560_21561_21564_21568 SPORT Interrupt List (Continued)

Interrupt ID	Name	Description	Sensitivity	DMA Channel
68	SPORT7_B_STAT	SPORT7 Channel B Status	Level	
140	SPORT0_A_DMA_ERR	SPORT0 Channel A DMA Error	Level	
141	SPORT0_B_DMA_ERR	SPORT0 Channel B DMA Error	Level	
142	SPORT1_A_DMA_ERR	SPORT1 Channel A DMA Error	Level	
143	SPORT1_B_DMA_ERR	SPORT1 Channel B DMA Error	Level	
144	SPORT2_A_DMA_ERR	SPORT2 Channel A DMA Error	Level	
145	SPORT2_B_DMA_ERR	SPORT2 Channel B DMA Error	Level	
146	SPORT3_A_DMA_ERR	SPORT3 Channel A DMA Error	Level	
147	SPORT3_B_DMA_ERR	SPORT3 Channel B DMA Error	Level	
148	SPORT4_A_DMA_ERR	SPORT4 Channel A DMA Error	Level	
149	SPORT4_B_DMA_ERR	SPORT4 Channel B DMA Error	Level	
150	SPORT5_A_DMA_ERR	SPORT5 Channel A DMA Error	Level	
151	SPORT5_B_DMA_ERR	SPORT5 Channel B DMA Error	Level	
152	SPORT6_A_DMA_ERR	SPORT6 Channel A DMA Error	Level	
153	SPORT6_B_DMA_ERR	SPORT6 Channel B DMA Error	Level	
154	SPORT7_A_DMA_ERR	SPORT7 Channel A DMA Error	Level	
155	SPORT7_B_DMA_ERR	SPORT7 Channel B DMA Error	Level	

ADSP-21560_21561_21564_21568 SPORT Trigger List

Table 21-4: ADSP-21560_21561_21564_21568 SPORT Trigger List Generators

Trigger ID	Name	Description	Sensitivity
17	SPORT0_A_DMA	SPORT0 Channel A DMA	Edge
18	SPORT0_B_DMA	SPORT0 Channel B DMA	Edge
19	SPORT1_A_DMA	SPORT1 Channel A DMA	Edge
20	SPORT1_B_DMA	SPORT1 Channel B DMA	Edge
21	SPORT2_A_DMA	SPORT2 Channel A DMA	Edge
22	SPORT2_B_DMA	SPORT2 Channel B DMA	Edge
23	SPORT3_A_DMA	SPORT3 Channel A DMA	Edge
24	SPORT3_B_DMA	SPORT3 Channel B DMA	Edge
25	SPORT4_A_DMA	SPORT4 Channel A DMA	Edge

Table 21-4: ADSP-21560_21561_21564_21568 SPORT Trigger List Generators (Continued)

Trigger ID	Name	Description	Sensitivity
26	SPORT4_B_DMA	SPORT4 Channel B DMA	Edge
27	SPORT5_A_DMA	SPORT5 Channel A DMA	Edge
28	SPORT5_B_DMA	SPORT5 Channel B DMA	Edge
29	SPORT6_A_DMA	SPORT6 Channel A DMA	Edge
30	SPORT6_B_DMA	SPORT6 Channel B DMA	Edge
31	SPORT7_A_DMA	SPORT7 Channel A DMA	Edge
32	SPORT7_B_DMA	SPORT7 Channel B DMA	Edge

Table 21-5: ADSP-21560_21561_21564_21568 SPORT Trigger List Receivers

Trigger ID	Name	Description	Sensitivity
20	SPORT0_A_DMA	SPORT0 Channel A DMA	Pulse
21	SPORT0_B_DMA	SPORT0 Channel B DMA	Pulse
22	SPORT1_A_DMA	SPORT1 Channel A DMA	Pulse
23	SPORT1_B_DMA	SPORT1 Channel B DMA	Pulse
24	SPORT2_A_DMA	SPORT2 Channel A DMA	Pulse
25	SPORT2_B_DMA	SPORT2 Channel B DMA	Pulse
26	SPORT3_A_DMA	SPORT3 Channel A DMA	Pulse
27	SPORT3_B_DMA	SPORT3 Channel B DMA	Pulse
28	SPORT4_A_DMA	SPORT4 Channel A DMA	Pulse
29	SPORT4_B_DMA	SPORT4 Channel B DMA	Pulse
30	SPORT5_A_DMA	SPORT5 Channel A DMA	Pulse
31	SPORT5_B_DMA	SPORT5 Channel B DMA	Pulse
32	SPORT6_A_DMA	SPORT6 Channel A DMA	Pulse
33	SPORT6_B_DMA	SPORT6 Channel B DMA	Pulse
34	SPORT7_A_DMA	SPORT7 Channel A DMA	Pulse
35	SPORT7_B_DMA	SPORT7 Channel B DMA	Pulse

ADSP-21560_21561_21564_21568 SPORT DMA Channel List

Table 21-6: ADSP-21560_21561_21564_21568 SPORT DMA Channel List

DMA ID	DMA Channel Name	Description
DMA0	SPORT0_A_DMA	SPORT0 Channel A DMA

Table 21-6: ADSP-21560_21561_21564_21568 SPORT DMA Channel List (Continued)

DMA ID	DMA Channel Name	Description
DMA1	SPORT0_B_DMA	SPORT0 Channel B DMA
DMA2	SPORT1_A_DMA	SPORT1 Channel A DMA
DMA3	SPORT1_B_DMA	SPORT1 Channel B DMA
DMA4	SPORT2_A_DMA	SPORT2 Channel A DMA
DMA5	SPORT2_B_DMA	SPORT2 Channel B DMA
DMA6	SPORT3_A_DMA	SPORT3 Channel A DMA
DMA7	SPORT3_B_DMA	SPORT3 Channel B DMA
DMA10	SPORT4_A_DMA	SPORT4 Channel A DMA
DMA11	SPORT4_B_DMA	SPORT4 Channel B DMA
DMA12	SPORT5_A_DMA	SPORT5 Channel A DMA
DMA13	SPORT5_B_DMA	SPORT5 Channel B DMA
DMA14	SPORT6_A_DMA	SPORT6 Channel A DMA
DMA15	SPORT6_B_DMA	SPORT6 Channel B DMA
DMA16	SPORT7_A_DMA	SPORT7 Channel A DMA
DMA17	SPORT7_B_DMA	SPORT7 Channel B DMA

Block Diagram

Each SPORT top module consists of two separate blocks, known as half SPORTs (HSPORT) A and B, each with identical functionality and programming models. The *Half Serial Port Block Diagram* shows a detailed block diagram of a half SPORT.

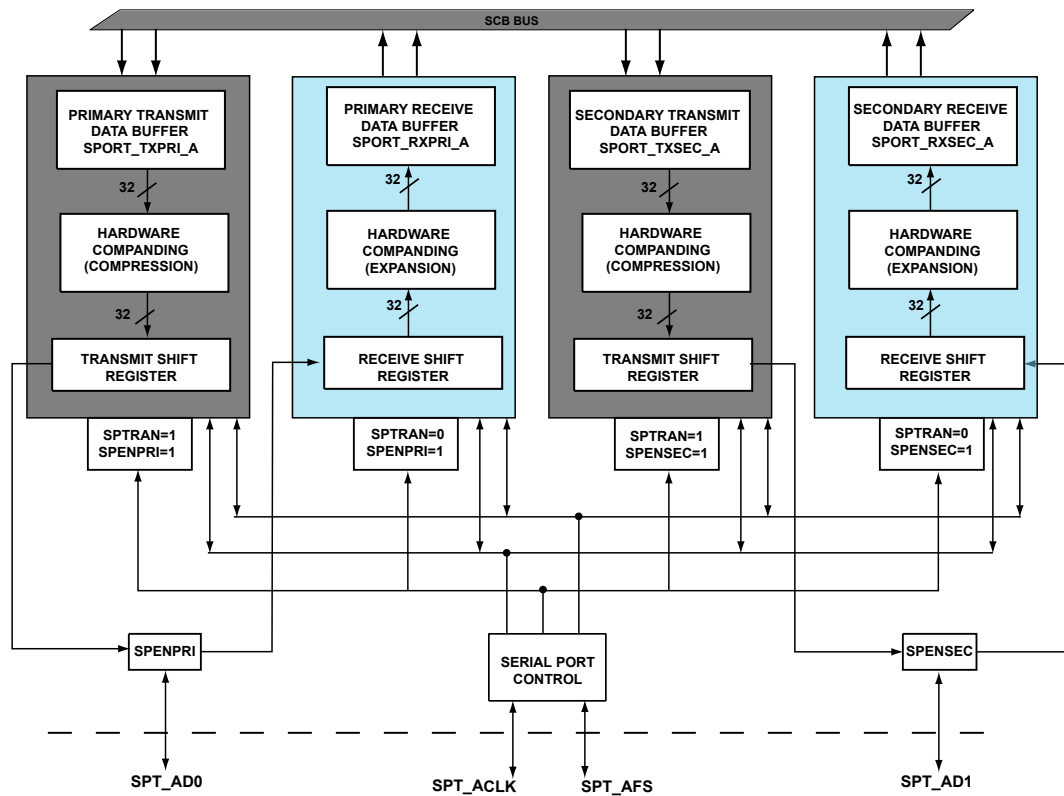


Figure 21-1: Half Serial Port Block Diagram

Architectural Concepts

Each half SPORT (HSPORT) block has its own set of control registers and data buffers, grouped per SPORT module. The HSPORT A and B blocks can be independently configured as either a transmitter or a receiver, with the option to be coupled together internally within the single SPORT top module. Each HSPORT also supports two synchronous bidirectional datapaths, referred to as the primary (D0) and secondary (D1) data lines, as shown in the *Top-Level SPORT Diagram* figure.

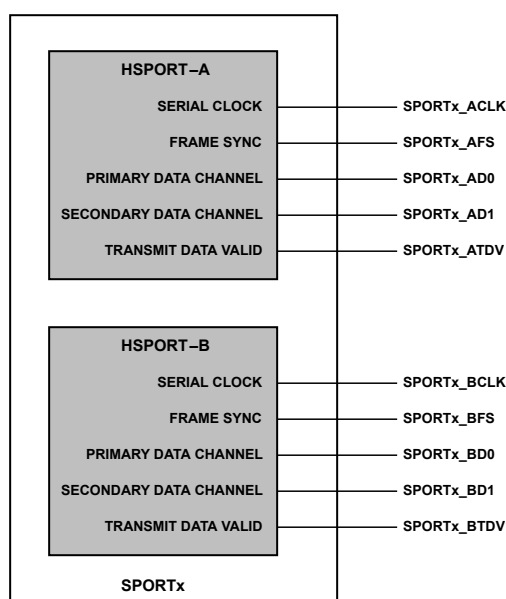


Figure 21-2: Top-Level SPORT Diagram

The `SPORT_CTL_A.SPTRAN` bit controls the direction for both datapaths of the HSPORT. Depending on whether the HSPORT is a transmitter or a receiver, the pair of data signals respectfully transmit or receive data bits synchronously. The dual data signals of each HSPORT cannot transmit and receive the data simultaneously in support of full-duplex operation, however, two HSPORTs can be combined to achieve this.

Serial communications are synchronized to the serial clock signal, where a valid clock pulse must accompany each data bit. Each HSPORT can take its clock from an external source or internally generate it from the processor's system clock using the `SPORT_DIV_A.CLKDIV` clock divisor bit field. Both primary and secondary data channels shift data based on the `SPORT_CLK` rate and the clock polarity defined by the `SPORT_CTL_A.CKRE` bit.

In addition to the serial clock signal, a frame synchronization signal is used to signify the beginning of an individual data word or a multichannel data stream (block of words). Each SPORT can take the frame sync signal from an external source or generate it (`SPORT_FS`), depending on the `SPORT_CTL_A.IFS` bit. An internally generated frame sync is derived from the SPORT clock using the `SPORT_DIV_A.FSDIV` divisor field. Both primary and secondary datapaths start shifting data either synchronous to or one serial clock in advance of detecting/generating a valid frame sync signal, as determined by the `SPORT_CTL_A.LAFS` bit. Various communication protocols for serial data can be emulated according to the frame sync format, and all frame sync options are available whether the signal is generated internally or externally.

NOTE: These SPORTs are not UARTs and cannot communicate with an RS-232 device or any other asynchronous communications protocol.

Multiplexer Logic

The SPORT multiplexing block (SPMUX) is situated between the SPORT hardware block and the processor's pin multiplexing logic. It allows the flexibility to route and share the clock and frame sync signals between the HSPORT A and B halves within each SPORT top module, which can double the data throughput (if both SPORT halves are transmitters or both are receivers) or provide full-duplex capabilities (if one HSPORT is a receiver and the other a

transmitter) without the need to allocate pins for the peripheral or make physical connections outside the processor. The `SPORT_CTL2_A` register is used to configure this loopback feature.

NOTE: Throughout this section, HSPORT A is used as a reference, but all concepts also apply to HSPORT B.

The multiplexing depends on the configuration of the `SPORT_CTL_A.IFS` and `SPORT_CTL_A.ICLK` bits, and the `SPORT_CTL2_A.CKMUXSEL` and `SPORT_CTL2_A.FSMUXSEL` bit settings control the multiplexing. The *Frame Sync Combinations* and *Clock Combinations* tables show the valid combinations for the bit settings.

NOTE: All other settings are illegal. However, hardware does not check or prevent the illegal settings. Ensure that programs use only legal combinations.

The column headers in the *Frame Sync Combinations* table are defined as follows:

- FS Combination = Frame sync combination, referenced in the notes that follow the *Clock Combinations* table.
- HSA_IFS = the setting of the `SPORT_CTL_A.IFS` configuration bit.
- HSB_IFS = the setting of the `SPORT_CTL_B.IFS` configuration bit.
- FSAMUX = the setting of the `SPORT_CTL2_A.FSMUXSEL` configuration bit.
- FSBMUX = the setting of the `SPORT_CTL2_B.FSMUXSEL` configuration bit.

The Routing column in the *Frame Sync Combinations* table defines how the signals are used between the SPORT halves and which pin is used for the frame sync (whether it is an input or an output). Within the column, the inequality characters ($\leq$ and $\geq$) are used to show the direction of the signal, and the following abbreviations are used (where x = A or B):

- HSx_FI = Frame sync input signal, provided by an external device or the complementing HSPORT.
- HSx_FO = Frame sync output signal.
- SPx_FS = HSPORT's frame sync pin, where the signal is either:
 - Provided by an external source and distributed to both HSPORT frame sync signals.
 - Internally generated by one HSPORT and routed to both the pin and to the complementary HSPORT frame sync signal.

Table 21-7: Frame Sync Combinations

FS Combination	HSA_IFS	HSB_IFS	FSAMUX	FSBMUX	Routing
1	0	0	0	0	Native FS Operation
2	0	1	0	0	Native FS Operation
3	1	0	0	0	Native FS Operation
4	1	1	0	0	Native FS Operation

Table 21-7: Frame Sync Combinations (Continued)

FS Combination	HSA_IFS	HSB_IFS	FSAMUX	FSBMUX	Routing
5	0	0	1	0	$HSA_FI \leq SPB_FS$; $HSB_FI \leq SPB_FS$
6	0	1	1	0	$HSA_FI \leq HSB_FO \geq SPB_FS$
7	0	0	0	1	$HSB_FI \leq SPA_FS$; $HSA_FI \leq SPA_FS$
8	1	0	0	1	$HSB_FI \leq HSA_FO \geq SPA_FS$

The column headers in the *Clock Combinations* table are defined as follows:

- CLK Combination = Clock combination, referenced in the notes that follow the table.
- HSA_ICLK = the setting of the SPORT_CTL_A.ICLK configuration bit.
- HSB_ICLK = the setting of the SPORT_CTL_B.ICLK configuration bit.
- CKAMUX = the setting of the SPORT_CTL2_A.CKMUXSEL configuration bit.
- CKBMUX = the setting of the SPORT_CTL2_B.CKMUXSEL configuration bit.

The Routing column in the *Clock Combinations* table defines how the signals are used between the SPORT halves and which pin is used for the serial clock (whether it is an input or an output). Within the column, the inequality characters ($\leq$ and $\geq$) are used to show the direction of the signal, and the following abbreviations are used (x = A or B):

- HSx_CI = Serial clock input signal, provided by an external device or the complementing HSPORT.
- HSx_CO = Serial clock output signal.
- SPx_CLK = HSPORT's serial clock pin, where the signal is either:
 - provided by an external source and distributed to both HSPORT serial clock signals, or
 - internally generated by one HSPORT and routed to both the pin and to the complementary HSPORT serial clock signal.

Table 21-8: Clock Combinations

CLK Combination	HSA_ICLK	HSB_ICLK	CKAMUX	CKBMUX	Routing
9	0	0	0	0	Native CLK Operation
10	0	1	0	0	Native CLK Operation
11	1	0	0	0	Native CLK Operation
12	1	1	0	0	Native CLK Operation

Table 21-8: Clock Combinations (Continued)

CLK Combination	HSA_ICLK	HSB_ICLK	CKAMUX	CKBMUX	Routing
13	0	0	1	0	$HSA_CI \leq SPB_CLK$; $HSB_CI \leq SPB_CLK$
14	0	1	1	0	$HSA_CI \leq HSB_CO \geq SPB_CLK$
15	0	0	0	1	$HSB_CI \leq SPA_CLK$; $HSA_CI \leq SPA_CLK$
16	1	0	0	1	$HSB_CI \leq HSA_CO \geq SPA_CLK$

The following is a comprehensive list of the legal combinations for the above described frame sync and clock multiplexing configurations:

- FS Combinations 1–4 are compatible with all CLK Combinations (9–16)
- CLK Combinations 9–12 are compatible with all FS Combinations (1–8)
- FS Combination 5 is only compatible with CLK Combination 13 (and vice versa)
- FS Combination 6 is only compatible with CLK Combination 14 (and vice versa)
- FS Combination 7 is only compatible with CLK Combination 15 (and vice versa)
- FS Combination 8 is only compatible with CLK Combination 16 (and vice versa)

NOTE: Program only the `SPORT_CTL2` register of the HSPORT that is accepting the signal from the other HSPORT. However, be sure to set the `SPORT_CTL_A.CKRE` and `SPORT_CTL_A.LFS` polarity bits to have identical settings between the HSPORTs when making internal connections via the SPMUX block.

Data Types and Companding

The SPORT uses the data type select field `SPORT_CTL_A.DTYPE` bit to specify one of the four data formats supported by serial ports. These formats apply to any of the operating modes of serial port.

Table 21-9: Data Type Bit Field Settings

DTYPE field	SPORT Receiver	SPORT Transmitter
00	Right-justify, zero-fill unused most significant bits	Normal operation
01	Right-justify, sign-extend unused most significant bits	Reserved
10	Expand using μ -law	Compress using μ -law
11	Expand using A-law	Compress using A-law

These formats apply to data words loaded into the SPORT transmit or receive data buffers. The first two data formats (00 and 01 values of `SPORT_CTL_A.DTYPE`) are applicable only when SPORT is configured as receiver.

When SPORT is configured as transmitter, only the significant bits are transmitted (per the field defined in control register). Therefore, the transmit data buffers are not actually zero-filled or sign-extended.

The other two data formats enable the companding logic on the transmit or receive path. Companding (compressing or expanding) is the process of logarithmically encoding and decoding data to minimize the number of bits sent. The SPORTs of the processor support the two most widely used companding algorithms, A-law, and μ -law. The algorithms are performed according to the CCITT G.711 specification.

If selected, companding applies to both the enabled data channels. When enabled as SPORT transmitter, writes to transmit buffer make the content compressed to 8 bits according to algorithm selected. (The content is zero filled to the width of the transmit word.) Similarly, if configured in receive mode, the 8 bits in the receive data buffers expand in right-justified, zero fill format per the algorithm selected. If companding is enabled in multichannel mode, it applies to all the active channels.

The compression for transmit data requires a minimum word length of 8 for proper function. If `SPORT_CTL_A.SLEN` is less than 7, then expansion does not work correctly. Also, if the data value is greater than 13-bit A-law or 14-bit μ -law maximum, it automatically compresses to the maximum value.

NOTE: The processor companding logic supports in-place companding feature. So, companding can be used for debug without enabling SPORT.

Companding as a Function

Since the values in the transmit and receive buffers are companded in place, the SPORT can use the companding hardware without transmitting or receiving data, which can be useful during testing or debugging. For companding to execute properly, program the SPORT registers prior to loading data values into the SPORT buffers.

To compress data in place without transmitting, use the following procedure:

1. Set the SPORT as a transmitter (`SPORT_CTL_A.SPTRAN = 1`) with both primary and secondary data channels disabled (`SPORT_CTL_A.SPENPRI = SPORT_CTL_A.SPENSEC = 0`).
2. Enable companding in the `SPORT_CTL_A.DTYPE` field.
3. Write a 32-bit data word to the transmit buffer.
4. Wait two system clock cycles to allow the SPORT companding hardware to reload the transmit buffer with the companded value. Any instructions that do not access the transmit buffer can be used to cause this delay.
5. Read the 8-bit compressed value from the transmit buffer.

To expand data in place, use the same sequence of operations with the receive buffer instead of the transmit buffer.

Transmit Path

When the `SPORT_CTL_A.SPTRAN` control bit is set, the HSPORT is in transmit mode. Primary and secondary transmit data paths are then enabled using the `SPORT_CTL_A.SPENPRI` and `SPORT_CTL_A.SPENSEC` bits, respectively. The primary and secondary datapaths are unique and identical, each including its own transmit data buffer, optional companding logic, and transmit shift register.

The data buffer on the primary transmit data path is `SPORT_TXPRI_A`, and the data buffer on the secondary transmit data path is `SPORT_TXSEC_A`. The transmit data buffer and output shift register form a FIFO type of structure. When packing is disabled (`SPORT_CTL_A.PACK = 0`), the SPORT can hold as many as three data words. If packing is enabled (`SPORT_CTL_A.PACK = 1`), the serial port can hold two packed data words at any time.

The transmit data for primary and secondary channels is written to the `SPORT_TXPRI_A` and `SPORT_TXSEC_A` transmit data buffers, respectively. The transmit data buffers can be accessed in core mode through the peripheral bus or in DMA mode through the DMA bus. When a SPORT is configured in transmit mode, the receive paths are deactivated and do not respond to serial clock or frame sync signals. Because the receive data buffers and receive shift registers are also deactivated, reading from an empty and inactive receive data buffer can cause the core to hang indefinitely.

NOTE: Be sure to avoid accesses to inactive data buffers. Such accesses can cause unpredictable SPORT behavior or a hang condition and are not reported in any status register.

This data can optionally be compressed in hardware according to the selected algorithm (μ -law or A-law) and then automatically transferred to the transmit shift register. The shift register, clocked by the `SPORT_ACLK` signal, then serially outputs this data on the `SPORT_AD0` and/or `SPORT_AD1` pins (if both are enabled, these output data bits are transmitted synchronously). If the SPORT uses a framing signal, the `SPORT_AFS` signal indicates the start of the serial word transmission.

When using DMA mode, a single DMA feeds the data buffers of the enabled channels (primary and/or secondary). When using both channels, interleave the data of these channels starting with the primary channel in the transmit buffer.

When the SPORT is configured in non-multichannel mode as a transmitter, the enabled SPORT data pins (`SPORT_AD0` and/or `SPORT_AD1`) are always driven. When a SPORT channel is enabled, data from the transmit data buffer is loaded into the transmit shift register. The shift register then immediately latches the first bit of data (either the LSB or MSB, depending on the `SPORT_CTL_A.LSBF` configuration bit) and drives it to the respective data pin such that it is ready when the frame sync signal asserts. Similarly, if the frame sync period exceeds the serial word length, then the data pins are driven with the first bit of the next word for transmission immediately after the active word completes, and the outputs are held during the inactive serial clock cycles (clock cycles between frame sync pulses).

When the SPORT is configured in multichannel mode, the data pins are driven only during active transmit channels and are always three-stated during inactive channel slots.

The SPORT provides status of transmit data buffers and also error detection logic for transmit errors such as an underrun condition. See the [Error Detection \(Status\) Interrupt](#) section for more details.

Receive Path

When the `SPORT_CTL_A.SPTRAN` bit is cleared, the SPORT is in receive mode. Primary and/or secondary receive data paths can be enabled by setting the `SPORT_CTL_A.SPENPRI` and `SPORT_CTL_A.SPENSEC` configuration bits, respectively. These data paths are unique but identical, each with a receive shift register, optional companding logic, and a receive data buffer.

The data buffer on the primary receive path is `SPORT_RXPRI_A`, and the data buffer on the secondary receive path is `SPORT_RXSEC_A`. The receive data paths act like a three-deep (32-bit words) FIFO because they have two data registers plus an input shift register.

Upon enabling the SPORT data channels, the input shift register shifts in data bits on the `SPORT_AD0` and/or `SPORT_AD1` pins, synchronous to the SPORT clock signal. If the SPORT uses a framing signal, the `SPORT_AFS` signal indicates the beginning of the serial word (or frame) to be received. When an entire word is shifted into the primary and secondary channels, the data can be optionally expanded in hardware according to a selected algorithm (μ -law or A-law) and then automatically transferred to the `SPORT_RXPRI_A` and/or `SPORT_RXSEC_A` data buffers.

The receive data buffers can be read in core mode through the peripheral bus or in DMA mode through the DMA bus. When the SPORT uses DMA mode, a single DMA reads the data buffers of the enabled channels (primary and/or secondary) and interleaves them in memory beginning with the primary channel when both channels are enabled. When using both channels, software must de-interleave the data of these channels.

The SPORT provides the status of receive data buffers and also error detection logic for receive errors such as overflow. See the Error Detection (Status) Interrupt section for more details.

When a SPORT is configured in receive mode, the transmit paths are deactivated and do not respond to serial clock or frame sync signals. As the transmit data buffers and transmit shift registers in the data paths are also deactivated, programs must not try to access them.

NOTE: Be sure to avoid accesses to inactive data buffers. Such accesses can cause unpredictable SPORT behavior or a hang condition and are not reported in any status register.

Operating Modes and Options

The SPORT has a number of operating modes:

- Standard DSP Serial mode
- I²S mode
- Left-Justified mode
- Right-Justified mode
- Multichannel (TDM) mode
- Packed I²S mode

The SPORT halves within a SPORT top module can be independently configured in any of these operating modes unless they are coupled together using SPMUX logic, in which case they must be configured identically. Each half SPORT has its own set of control and data registers and is programmed similarly.

The *Control Bits for SPORT Operating Modes* table lists all the programmable configuration bits in the [SPORT_CTL_A](#) control register, which combine to determine the overall function and operating mode of the SPORT. The columns are arranged according to the setting of the `SPORT_CTL_A.OPMODE` bit that selects between standard DSP/multichannel modes and the various I²S modes, and the cell contents are defined as follows:

- Yes – bit is programmable for this mode of operation and may be written
- Reserved – bit is not programmable for this mode of operation and must not be written
- = value – bit must be set to this value to enable this mode of operation
- FUNCTION – indicates alternate function for this bit in this mode

NOTE: When changing operating modes, first clear the [SPORT_CTL_A](#) register before again writing the register with the new configuration settings.

Table 21-10: Control Bits for SPORT Operating Modes

Name (Bit #)	Standard DSP Serial Mode	I ² S and Left-Justified Mode	Right-Justified Mode	Multichannel (TDM) Mode	Packed I ² S Mode
SPENPRI (0)	Valid				
DTYPE (2:1)	Valid	Reserved		Valid	
LSBF (3)	Valid	Reserved		Valid	
SLEN (8:4)	Valid				
PACK (9)	Valid				
ICLK (10)	Valid				
OPMODE (11)	= 0	= 1	= 1	= 0	= 1
CKRE (12)	Valid				
FSR (13)	Valid	Reserved			
IFS (14)	Valid				
DIFS (15)	Valid			Reserved	
LFS (16)	Valid	L_FIRST/PLFS		Valid	L_FIRST/PLFS
LAFS (17)	Valid	OPMODE2	Valid	Reserved	
RJUST (18)	Reserved		= 1	Reserved	
FSED (19)	Valid	Reserved		Valid	Reserved
TFIEN (20)	Valid				
GCLKEN (21)	Valid		Reserved		

Table 21-10: Control Bits for SPORT Operating Modes (Continued)

Name (Bit #)	Standard DSP Serial Mode	I ² S and Left-Justified Mode	Right-Justified Mode	Multichannel (TDM) Mode	Packed I ² S Mode
SPENSEC (24)	Valid				
SPTRAN (25)	Valid				

Serial Word Length

The SPORT uses the `SPORT_CTL_A.SLEN` field to determine the word length of the serial data to transmit or receive. Each half SPORT can independently handle word lengths up to 32 bits, and the value that must be programmed to the `SPORT_CTL_A.SLEN` field is obtained from:

$$\text{SLEN} = \text{Desired SPORT word length} - 1$$

The minimal word length depends on the selected operating mode. Words smaller than 32 bits are right justified in the transmit or receive buffers; however, data can be shifted in or out in MSB or LSB first format, as configured by the `SPORT_CTL_A.LSBF` bit. The received word can also be sign-extended or zero-filled when storing the data to processor memory, as governed by the `SPORT_CTL_A.DTYPE` bit.

The *Data Lengths for SPORT Operating Modes* table shows the range of valid word lengths for each of the supported SPORT operating modes.

Table 21-11: Data Lengths for SPORT Operating Modes

Mode	SPORT Word Length (SLEN+1)
Standard DSP Serial	4–32
I ² S	5–32
Left-Justified	5–32
Right-Justified	5–32
Multichannel (TDM)	5–32
Packed I ² S	5–32

NOTE: When the companding feature is enabled on the datapath, it limits the word length settings. See [Data Types and Companding](#) for more details about word lengths required for companding. When more than 32 bits per frame sync are required to transmit or receive, use the multichannel mode to spread the data across numerous continuous channels.

Clock Sample and Drive Edges

The SPORT uses two control signals to sample or drive the serial data:

1. Serial clock (`SPORT_ACLK`) - bit clock for the serial data.
2. Frame sync (`SPORT_AFS`) - divides the incoming data stream into frames.

These control signals can be internally generated or externally provided, as determined by the `SPORT_CTL_A.ICLK` and `SPORT_CTL_A.IFS` bit settings, respectively.

Data and frame syncs can be sampled on the rising or falling edges of the SPORT clock signal, as determined by the `SPORT_CTL_A.CKRE` bit. By default, the `SPORT_CTL_A.CKRE = 0` setting configures the falling edge of the `SPORT_ACLK` signal as the sampling edge for receive data and externally supplied frame syncs. The receive data and frame syncs can be sampled on the rising edges of `SPORT_ACLK` when `SPORT_CTL_A.CKRE = 1`.

NOTE: The SPORT drives transmit data and internal frame sync signals on the opposite serial clock edge of the sampling edge. Be sure to select the same value for `SPORT_CTL_A.CKRE` for transmit and receive functions for any two HSPORTs that are connected, and always verify the correct polarity for any external device connected to the SPORT.

The *Frame Sync and Data Driven on Rising Edge* figure provides an example of the drive and sample edges when two HSPORTs are connected, each with `SPORT_CTL_A.CKRE = 0`. In this example, the HSPORT that is configured as the transmitter drives the serial clock and frame sync signals, and both HSPORTs are configured for early, active high frame syncs and a word length of eight bits.

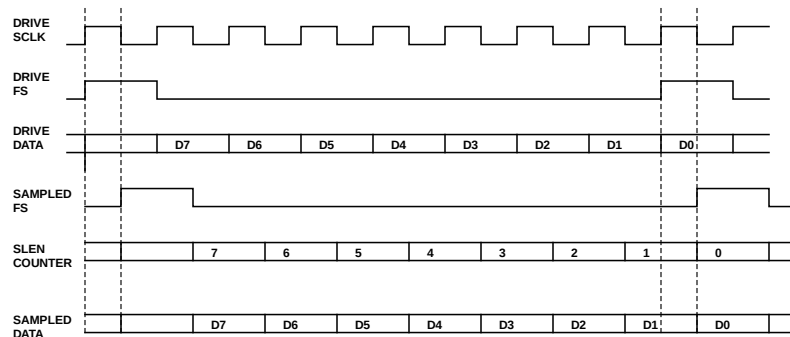


Figure 21-3: Frame Sync and Data Driven on Rising Edge

NOTE: The SCLK in the *Frame Sync and Data Driven on Rising Edge* figure is SCLK.

As shown, the transmitting HSPORT provides the clock and generates the frame sync. Because the HSPORTs are configured for early frame mode, the first bit of data is driven one serial clock later, with subsequent bits being driven on the following rising clock edges in the signal train. When the receiving HSPORT samples the frame sync signal (as indicated in the SAMPLED FS waveform), the `SPORT_CTL_A.SLEN` bit counter is loaded with the `SPORT_CTL_A.SLEN` setting, after which each `SPORT_ACLK` decrements the `SPORT_CTL_A.SLEN` counter until the full word is received. In this figure, the DRIVE FS and SAMPLED FS waveforms show the frame sync required for the next word in a continuous data stream. Note that it is legal for this frame sync to be sampled synchronous to the last bit of the previous data being sampled, as the early frame mode means that the data lags the frame sync by one serial clock cycle. If the frame sync were sampled as asserted before the D0 bit is sampled, the frame sync error is logged in the receiver's status register.

Since the transmitter drives the internally-generated frame sync and data on the rising edge of the serial clock, the receiver must use the falling edge to sample the externally-supplied frame sync and data.

Frame Sync Options

The following sections provide details regarding the programmable aspects of the SPORT frame sync signal. See the specific operating mode sections for more information regarding frame sync requirements and behavior for each specific operating mode.

Data-Dependent versus Data-Independent Frame Syncs

By default, the generation of a frame sync signal is data-dependent:

- When the SPORT is configured as a transmitter (`SPORT_CTL_A.SPTRAN = 1`), an internally generated transmit frame sync is output when a new data word has been loaded into the channel transmit buffer of the SPORT (by either the core or the DMA engine).
- When the SPORT is configured as a receiver (`SPORT_CTL_A.SPTRAN = 0`), an internally-generated receive frame sync is output only when the receive data buffer is not full.

The data-independent frame sync option, enabled by setting the `SPORT_CTL_A.DIFS` bit, allows for the generation of a periodic framing signal, regardless of the status of the data buffers. When this bit is set, the frame sync output will be continuous and periodic, according to the setting of the `SPORT_DIV_A.FSDIV` field.

Support for Edge-Detected and Level-Sensitive Frame Syncs

The level-sensitive nature of frame sync signals operates well in a noise-free environment. However, if noise corrupts the signals coming into the SPORT, the internal logic can lose synchronization. For example, excessive noise on the frame sync signal may cause the frame sync to be sampled as inactive on the clock edge that it is intended to be synchronous to, but then be sampled at the correct active level one cycle later. Similarly, a noisy clock signal can cause an unintended clock edge, resulting in potential premature sampling of the frame sync signal being applied to the pin.

The *Level-Sensitive Frame Sync versus Edge Sensitive Frame Sync* figure describes a scenario where an external frame sync signal is corrupted due to noise, causing the receiving SPORT module to incorrectly sample the signal. If the frame sync is driven on the rising edge of the serial clock at t_A , the SPORT would normally sample the signal on the falling edge of the serial clock at t_B . Due to the noise, however, the SPORT misses the first edge of the frame sync and instead samples it at t_C .

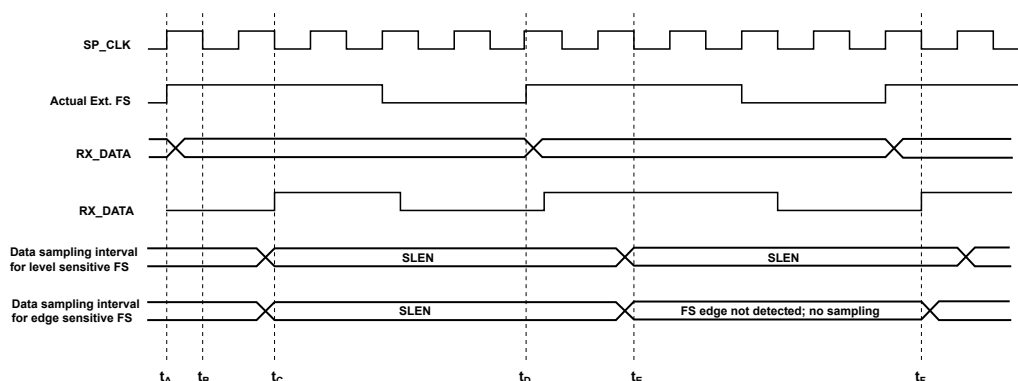


Figure 21-4: Level-Sensitive Frame Sync versus Edge Sensitive Frame Sync

NOTE: SCLK in the *Level-Sensitive Frame Sync versus Edge Sensitive Frame Sync* figure is SCLK.

When the above occurs, the internal word length counter runs for a period equal to the `SPORT_CTL_A.SLEN` field of the control register, but it erroneously expires at t_E rather than at the appropriate point at t_D , thus receiving incorrect data. Further, if a new level-sensitive frame sync edge arrives at time t_D , the SPORT samples this framing signal again at t_E . As such, the frame sync sampling continues to be misaligned with the external data.

To help address this, the SPORT module provides an option to configure the frame sync signal to instead be edge-sensitive via the `SPORT_CTL_A.FSED` configuration bit. When this bit is set with active high frame syncs enabled (`SPORT_CTL_A.LFS = 0`), the rising edge of the frame sync is valid. Conversely, when the frame sync is active low (`SPORT_CTL_A.LFS = 1`), the falling edge is defined to be valid.

NOTE: `SPORT_CTL_A.FSED` is valid only in external frame sync mode. In internal frame sync mode, the setting of this bit is irrelevant and ignored.

In the above example, an edge-sensitive frame sync signal is not detected at t_E because the edge of the framing signal already occurred in the previous cycle (t_D) and there is no new edge to detect at t_E . As a result, the internal word length counter remains idle for this frame, thus ignoring the incorrect data, and the counter correctly resumes operation at t_F when a new frame sync edge is detected.

This activity sets the `SPORT_ERR_A.FSERRSTAT` bit and optionally generates a premature frame sync error interrupt.

Frame sync edge detection is used by default for stereo modes. MCM mode and DSP serial mode choose between edge detection and normal mode of FS detection.

NOTE: When the SPORT is first enabled, an already active externally applied frame sync will not commence operation. The SPORT will wait for a valid change in the frame sync's state from inactive to active before operation begins.

Early versus Late Frame Syncs

Frame sync signals can occur in the same serial clock cycle as the first bit of the data word (late) or one serial clock cycle before the first bit (early), as controlled by the `SPORT_CTL_A.LAFS` bit.

By default, the frame sync signal is configured to be early (`SPORT_CTL_A.LAFS = 0`). The first bit of the transmit data word will be driven one serial clock cycle after the frame sync is asserted (whether sensed externally or internally provided), and the first bit of the receive data word is expected to lag the frame sync by one serial clock cycle. The frame sync is not checked again until the entire word has been transferred.

If data transmission is continuous in early framing mode, then an internally-generated frame sync signal will be asserted (pulsed active for one serial clock cycle) synchronous to the last data bit of the current transfer, as the first bit of the next transfer will be immediately driven in the next serial clock cycle (no clocks are wasted). This event is not a premature frame sync error, so the `SPORT_ERR_A.FSERRSTAT` bit is not set.

The frame sync can alternatively be configured as late (`SPORT_CTL_A.LAFS = 1`), in which case the first bit of the transmit data word is available in the same serial clock cycle that the frame sync is asserted (whether sensed

externally or internally provided), and the first bit of the receive data word is also latched in the same cycle. Serial clock edges latch the receive data bits, but the frame sync signal is checked only during the first bit of each word. Internally generated frame syncs remain asserted for the entire length of the data word in late framing mode.

The *Normal Framing (Early Frame Sync) Versus Alternate Framing (Late Frame Sync)* figure illustrates these concepts.

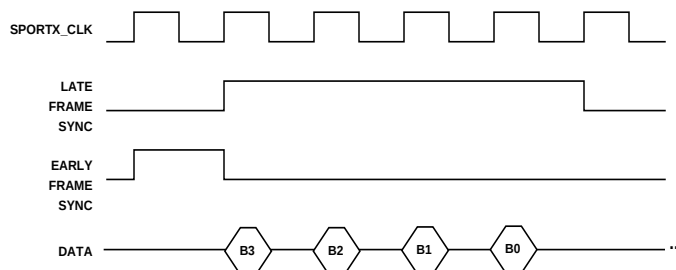


Figure 21-5: Normal Framing (Early Frame Sync) Versus Alternate Framing (Late Frame Sync)

Framed versus Unframed Frame Syncs

The use of a frame sync signal is optional for SPORT operation, as controlled by the `SPORT_CTL_A.FSR` bit. When the frame sync is configured to be required (`SPORT_CTL_A.FSR = 1`), the data is defined to be framed (a frame sync signal must accompany every data word). To allow continuous transmission from the processor, ensure that a new data word is loaded into the transmit buffer before the ongoing transfer is completed (this is automatically cared for when DMA is used to transmit blocks of data).

Data words can be transferred continuously in what is referred to as unframed data mode, which is appropriate for continuous reception, by setting `SPORT_CTL_A.FSR = 0`. In this configuration, a single frame sync is still required to initiate communication, but it is subsequently unrequired once the communication begins. From that point onward, externally provided frame syncs are ignored and internally generated frame syncs are not driven. The *Framed versus Unframed Data Stream* figure shows the differences in SPORT operation between framed and unframed data modes with the frame sync configured to be early (`SPORT_CTL_A.LAFS = 0`).

NOTE: When DMA is enabled in a mode where frame syncs are not required, chaining can delay DMA requests. DMA requests are not always serviced frequently enough to guarantee continuous unframed data flow. Monitor status bits or check for a SPORT error interrupt to detect underflow or overflow of data.

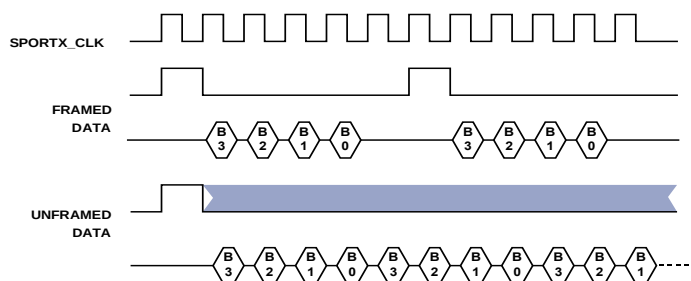


Figure 21-6: Framed versus Unframed Data Stream

Frame Sync Polarity

The framing signals can be active high or active low, as governed by the `SPORT_CTL_A.LFS` bit:

- When `SPORT_CTL_A.LFS = 0`, the corresponding frame sync signal is active high.
- When `SPORT_CTL_A.LFS = 1`, the corresponding frame sync signal is active low.

Active high is the default polarity of the frame sync signal.

Premature Frame Sync Error Detection

A SPORT framing signal is used to synchronize transmit or receive data. In external frame sync mode, any frame sync received during an active frame is premature and invalid. When this occurs, the `SPORT_ERR_A.FSERRSTAT` bit is set to indicate the framing error, and an optional error interrupt request can be generated for this event by setting the `SPORT_ERR_A.FSERRMSK` bit.

NOTE: The `SPORT_ERR_A.FSERRSTAT` bit is not set in the presence of uncleared underflow or overflow errors.

Refer to the *Frame Sync Error Detection* figure. The frame sync error bit gets set when an unexpected frame sync occurs during the ongoing data transfer (transmission or reception).

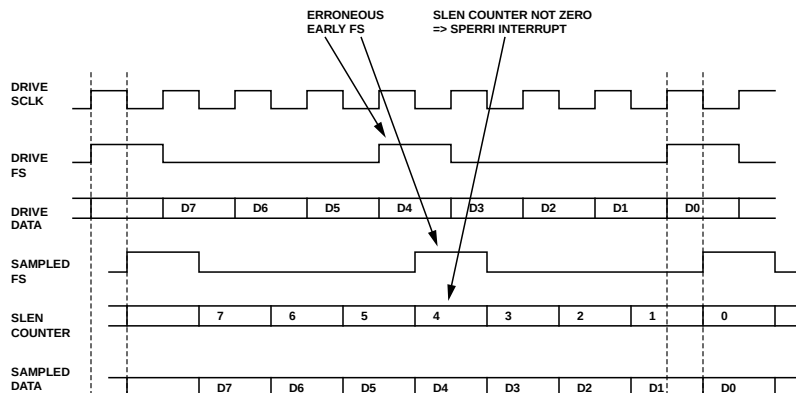


Figure 21-7: Frame Sync Error Detection

NOTE: SCLK in the *Frame Sync Error Detection* figure is SCLK.

Whether a SPORT is receiving or transmitting, its bit count is set to the programmed serial word length when the frame sync is sampled, which is then decremented every subsequent serial clock cycle until the transfer has completed. At this point, the bit count reaches zero and will be reset to the programmed serial length when the next frame sync is sampled. As such, the bit count value is non-zero during an active transfer, and the frame sync error is asserted if a frame sync is sampled when this count is non-zero.

Mode Selection

The SPORT's operating mode is configured in the `SPORT_CTL_A` and `SPORT_MCTL_A` registers. The *SPORT Operating Modes* table provides specific guidance to properly program these SPORT control registers for the desired mode of operation.

Table 21-12: SPORT Operating Modes

Operating Modes	<code>SPORT_CTL_A.OPMODE</code>	<code>SPORT_CTL_A.LAFS</code>	<code>SPORT_CTL_A.RJUST</code>	<code>SPORT_MCTL_A.MCE</code>
Standard DSP Serial	0	Programmable	Reserved	0
I ² S	1	0	Reserved	0
Left-Justified	1	1	Reserved	0
Right-Justified	1	1	1	0
Multichannel	0	Reserved	Reserved	1
Packed I ² S mode	1	Reserved	Reserved	1

The following sections provide detailed information for each of the supported SPORT modes of operation.

Standard DSP Serial Mode

The SPORT can be configured in standard DSP serial mode by clearing the `SPORT_CTL_A.OPMODE` and `SPORT_MCTL_A.MCE` bits. This mode provides great flexibility in terms of programmable options to configure the SPORTs to communicate with various serial devices such as serial data converters and audio codecs. In order to properly connect to such devices, various clocking, framing, and data formatting options are available.

Timing Control Bits

Several bits in the `SPORT_CTL_A` control register define the configuration of the SPORT in standard DSP serial mode:

- SLEN: serial word length (4–32 bits)
- LSBF: shift LSB or MSB first
- ICLK: internally generated or externally provided serial clock
- CKRE: sample on rising or falling edge of the serial clock
- IFS: internally generated or externally provided frame sync
- FSR: framed or continuous operation
- DIFS: data-dependent or data-independent frame sync
- LFS: active high or active low frame sync
- LAFS: frame sync synchronous to data or one clock cycle before it
- PACK: 16-bit to 32-bit packing option

- GCLKEN: free-running or gated clock

Clocking Options

In standard DSP serial mode, the SPORTs can either accept an external serial clock or generate one internally, as controlled by the `SPORT_CTL_A.ICLK` bit. For internally generated serial clocks (`SPORT_CTL_A.ICLK = 1`), the `SPORT_DIV_A.CLKDIV` field configures the serial clock rate from the system clock.

The SPORT clock can also be gated, where it is only valid during an active transfer, as controlled by the `SPORT_CTL_A.GCLKEN` bit.

The SPORT clock edge used for driving and sampling of serial data and frame syncs is configured using the `SPORT_CTL_A.CKRE` bit:

- If `SPORT_CTL_A.CKRE = 0`, input data and frame sync signals are sampled on the falling edge of the serial clock, and output data and frame sync signals are driven on the rising edge.
- If `SPORT_CTL_A.CKRE = 1`, input data and frame sync signals are sampled on the rising edge of the serial clock, and output data and frame sync signals are driven on the falling edge.

Stereo Modes

The SPORTs support three widely used stereo modes of operation:

- I²S mode
- Left-Justified mode
- Right-Justified mode

In these modes, the serial data stream consists of left and right channels. The following sections describe these modes in more detail.

Channel Order

The active low frame sync (`SPORT_CTL_A.LFS`) bit is used to determine the polarity of the frame sync signal in the non-stereo modes of operation. For the stereo modes of operation, it instead controls whether the right or left channel is first in the data transfer. The *Channel Order Bit Settings* table shows which word is transmitted or received first, based on the setting of the `SPORT_CTL_A.LFS` bit.

Table 21-13: Channel Order Bit Settings

Mode	<code>SPORT_CTL_A.LFS=0</code>	<code>SPORT_CTL_A.LFS=1</code>
Left-Justified or Right-Justified	Left channel first	Right channel first
I ² S or Packed I ² S	Right channel first	Left channel first

I²S Mode

I²S mode is a commonly used stereo mode, where left and right channel data words are interleaved in the serial data stream and each transition of the frame sync signal is associated with one of the channels. The left channel data is transferred during the low segment of the frame sync signal, and the right channel data is transferred during the high segment of the frame sync signal. As such, the frame sync signal is treated as a left-right (L/R) clock in this mode.

To set up the SPORT in I²S mode, the following configuration is required:

- `SPORT_CTL_A.OPMODE = 1`
- `SPORT_CTL_A.LFS = 0`
- `SPORT_MCTL_A.MCE = 0`

Protocol Configuration Options

Several bits in the `SPORT_CTL_A` control register must be configured to be compliant with the I²S standard, but they can be otherwise configured to support non-standard operations

- `SLEN`—programmable (allowable word lengths are 5–32 bits)
- `LSBF`—set to 0 (MSB first)
- `ICLK`—programmable (serial bit clock can be internally generated or externally provided)
- `IFS`—programmable (serial L/R clock source must match serial bit clock source)
- `LFS`—set to 1 (left channel first)
- `CKRE`—set to 1 (sample L/R clock and data on rising edge of bit clock)

Serial Bit Clock and L/R Clock Rates

When the SPORT is configured to generate the bit clock and the L/R clock (`SPORT_CTL_A.ICLK = SPORT_CTL_A.IFS = 1`), set the serial bit clock rate using the `SPORT_DIV_A.CLKDIV` bit field and the L/R clock rate using the `SPORT_DIV_A.FSDIV` bit field.

The *Word Select Timing in I²S Mode* figure shows the SPORT timing in I²S mode. The data lags the L/R clock transition by one SCLK cycle, and the transfer begins with the left channel data word first.

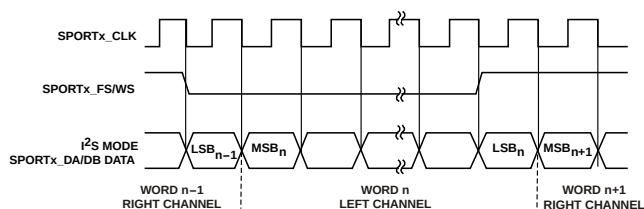


Figure 21-8: Word Select Timing in I²S Mode

Left-Justified Mode

Left-justified mode is a stereo mode subset of the I²S standard. As in I²S mode, the frame sync signal acts as a left-right clock (L/R clock), where left and right data samples are transferred each L/R clock period. The left channel is associated with the high segment of the frame sync, and the right channel aligns with the low segment of the frame sync. The difference between left-justified mode and standard I²S mode is that the channel data is driven in the same bit clock cycle as the L/R clock transition (rather than one bit clock cycle later), such that the MSB is synchronous with the leading edge of the frame sync transition.

To set the SPORT up in left-justified mode, the following configuration is required:

- `SPORT_CTL_A.OPMODE = 1`
- `SPORT_CTL_A.LAFS = 1`
- `SPORT_MCTL_A.MCE = 0`

Protocol Configuration Options

Several bits in the `SPORT_CTL_A` control register must be configured to operate the SPORT in left-justified mode, but they can be otherwise configured as well:

- **SLEN**: programmable (allowable word lengths are 5–32 bits)
- **LSBF**: set to 0 (MSB first)
- **ICLK**: programmable (serial bit clock can be internally generated or externally provided)
- **IFS**: programmable (serial L/R clock source must match serial bit clock source)
- **LFS**: set to 0 (left channel first)
- **CKRE**: set to 1 (sample L/R clock and data on rising edge of bit clock)

Serial Bit Clock and L/R Clock Rates

If the SPORT is configured to generate the bit clock and the L/R clock (`SPORT_CTL_A.ICLK = SPORT_CTL_A.IFS = 1`), set the serial bit clock rate using the `SPORT_DIV_A.CLKDIV` bit field and the L/R clock rate using the `SPORT_DIV_A.FSDIV` bit field.

The *Word Select Timing in Left-Justified Mode* figure shows the SPORT timing in left-justified mode. The start of a data sample is synchronous to the L/R clock transition, and the transfer begins with the left channel data word first.

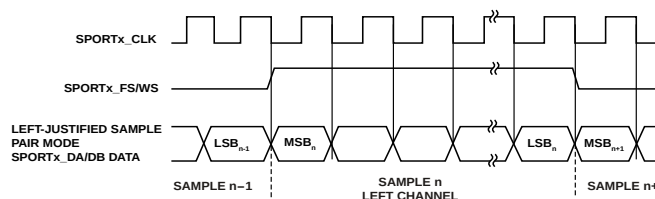


Figure 21-9: Word Select Timing in Left-Justified Mode

Right-Justified Mode

Right-justified mode is a stereo mode subset of the I²S standard. As in I²S mode and left-justified mode, the frame sync signal acts as a left-right clock (L/R clock), where left and right data samples are transferred each L/R clock period. The left channel is associated with the high segment of the frame sync, and the right channel aligns with the low segment of the frame sync. The difference between right-justified mode and standard I²S mode is that the LSB of the channel data ends at the point that the L/R clock transitions to frame the next sample (rather than one bit clock cycle after the L/R clock transition).

To set the SPORT up in right-justified mode, the following configuration is required:

- `SPORT_CTL_A.OPMODE = 1`
- `SPORT_CTL_A.RJUST = 1`
- `SPORT_MCTL_A.MCE = 0`

Timing Control Bits

Several bits in the `SPORT_CTL_A` control register must be configured to operate the SPORT in right-justified mode, but they can be otherwise configured as well:

- **SLEN**: programmable (allowable word lengths are 5–32 bits)
- **LSBF**: set to 0 (MSB first)
- **ICLK**: programmable (serial bit clock can be internally generated or externally provided)
- **IFS**: programmable (serial L/R clock source must match serial bit clock source)
- **LFS**: set to 0 (left channel first)
- **CKRE**: set to 1 (sample L/R clock and data on rising edge of bit clock)

Serial Bit Clock and L/R Clock Rates

If the SPORT is configured to generate the bit clock and the L/R clock (`SPORT_CTL_A.ICLK = SPORT_CTL_A.IFS = 1`), set the serial bit clock rate using the `SPORT_DIV_A.CLKDIV` bit field and the L/R clock rate using the `SPORT_DIV_A.FSDIV` bit field.

The *Word Select Timing in Right-Justified Mode* figure shows the SPORT timing in right-justified mode. The transmitter aligns the transmit data such that the last bit of the serial word is sent in the last clock cycle of the L/R clock (frame sync) signal marking the channels.

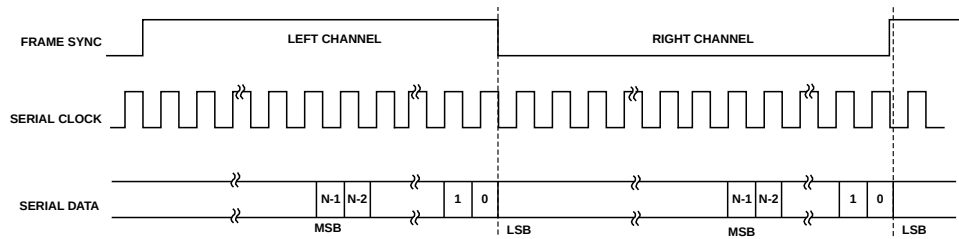


Figure 21-10: Word Select Timing in Right-Justified Mode

NOTE: For some SPORT-compatible ADCs or DACs such as the AD1871, right-justified mode is limited to commonly used ratios such as 64 FS and 128 FS. FS is the sampling frequency of ADCs and DACs, referred to as the SPORT's L/R clock (frame sync) signal.

Consider the SPORT timing for right-justified mode, as shown in the *Timing Comparison Between Different Stereo Modes* figure. The frame sync width is limited to 32 SPORT clock periods (or 32 bits per channel) if:

- the SPORT's frame sync (L/R clock) runs at the FS rate, and
- the SPORT's serial bit clock runs at the 64 FS rate

The limitation applies to the frame sync width of either channel. If the data is confined to 24 bits, the SPORT introduces a $32 - 24 = 8$ -bit clock delay before it starts to transmit or capture data.

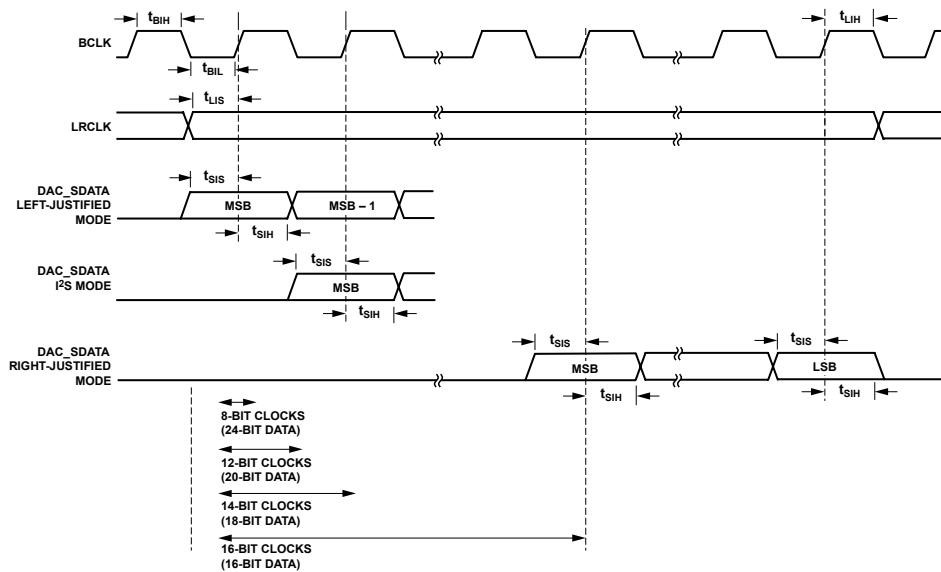


Figure 21-11: Timing Comparison Between Different Stereo Modes

Similarly, to support the 128 FS bit clock frequency, the frame sync width becomes 64 serial bit clock periods per channel. The delay can be a maximum of 59 bit clocks ($64 - 5$, which is the minimum serial data length in right-justified mode).

The starting point of the first bit is delayed so that the LSB of the serial data aligns properly with the end of the channel. A 6-bit counter is added for this purpose in the stereo mode counter, which is programmed by writing the least significant six bits of the `SPORT_MCTL_A.WOFFSET` field. Though this is a multichannel mode configuration register, the SPORT uses these bits in right-justified mode to configure the offset from the transition of the L/R

clock to where the first data bit must be driven to have the end of the last bit align properly with the next L/R clock transition. The software must program this register with the appropriate delay.

Multichannel (TDM) Mode

The multichannel mode of SPORT operation allows the SPORT to communicate as part of a time division multiplexed (TDM) serial system. In TDM communications, a large frame of streamed serial data words is defined to be a particular length. It consists of a specific number of channels, and each channel contains one serial data word of the defined data length. For example, a 24-word block of 24-bit data can be defined to be a window within a frame, having a duration of 576 bit clocks and comprised of 24 continuous channels. The SPORT is configured to transfer on specific channels within a defined window in this frame.

To set the SPORT up in multichannel mode, the following configuration is required:

- `SPORT_CTL_A.OPMODE = 0`
- `SPORT_MCTL_A.MCE = 1`

In multichannel mode, the SPORT can selectively transfer data on up to a maximum window size of 128 continuous channels out of a maximum 1024-channel frame while ignoring all the disabled channels within the window and all the channels outside the window. The SPORT can do any of the following on each channel:

- Transmit data (`SPORT_CTL_A.SPTRAN = 1`)
- Receive data (`SPORT_CTL_A.SPTRAN = 0`)
- Do nothing (during inactive channels)

Channel selection is configured in the half SPORT multichannel select registers (`SPORT_CS0_A - SPORT_CS3_A`) before enabling SPORT operation for multichannel mode. Programming of these registers is especially important in DMA data unpacked mode, since the SPORT data buffers begin operation immediately after the SPORT data lines are enabled. Be sure to enable multichannel operation (set the `SPORT_MCTL_A.MCE` bit) prior to enabling the SPORT itself.

Clocking Options

In multichannel mode, the SPORTs can either accept an external serial clock or generate one internally, as governed by the `SPORT_CTL_A.ICLK` bit. For an internally-generated serial clock (`SPORT_CTL_A.ICLK = 1`), the `SPORT_DIV_A.CLKDIV` bit field is used to configure the serial clock rate, as derived from the system clock.

The serial clock edges used to drive and sample data and frame syncs are also configurable using the `SPORT_CTL_A.CKRE` bit:

- If `SPORT_CTL_A.CKRE = 0`, input data and frame sync signals are sampled on the falling edge of the serial clock, and output data and frame sync signals are driven on the rising edge.
- If `SPORT_CTL_A.CKRE = 1`, input data and frame sync signals are sampled on the rising edge of the serial clock, and output data and frame sync signals are driven on the falling edge.

Frame Sync Options

The frame sync signal synchronizes the channels and restarts each multichannel sequence, starting with the channel 0 data word. For internally-generated frame syncs (`SPORT_CTL_A.IFS = 1`), the frame sync period in multichannel mode is defined as:

$$\text{FS period} = [(\text{SPORT_CTL_A.SLEN} + 1) \times \text{number of channels}] - 1$$

The active level for the frame sync signal is also configurable by programming the `SPORT_CTL_A.LFS` bit. Set this bit to make the frame sync an active low signal, and clear it to make it active high.

In multichannel mode, frame sync timing resembles late framing mode (although the `SPORT_CTL_A.LAFS` bit is reserved in this mode). The first bit of the transmit data word is driven and the first bit of the receive data word is sampled in the same serial clock cycle as the frame sync, provided there is no programmed frame delay (`SPORT_MCTL_A.MFD = 0`).

Once the frame sync signal is asserted, word transfers are performed continuously for the duration of the active window, and no further frame syncs are required for different channels within the window. As such, internally-generated frame syncs are always data-independent, and the `SPORT_CTL_A.DIFS` bit is reserved.

Transmit Data Valid (TDV)

Each SPORT features a transmit data valid signal (`SPORT_ATDV`), which is driven high during enabled transmit channels. Because the SPORT output data signals are three-stated during inactive channels, the `SPORT_ATDV` signal signifies when the processor is actively driving the SPORT data outputs, thus serving as an output-enable signal for the data transmit pin(s).

Active Channel Selection Registers

In multichannel mode, the SPORT supports a window size of up to 128 channels for transmitting or receiving data, where it can selectively receive or transmit data in any of these 128 channels. Each channel can be individually enabled or disabled using the multichannel selection registers (`SPORT_CS0_A` to `SPORT_CS3_A`) to select the channels in which to transfer data during a multichannel communication stream. Data words associated with enabled channels are transmitted or received in the respective channels, while disabled channels cause a transmit SPORT to three-state the data output pins and a receive SPORT to ignore the data.

The four 32-bit multichannel selection registers combine to form up to a 128-bit meta-register to accommodate the maximum window size of 128 channels. Setting any bit within these registers enables the associated channel. The 128 channels are sequentially numbered from bit 0 in the `SPORT_CS0_A` register (corresponding to channel 0 of the window) to bit 31 of the `SPORT_CS3_A` register (corresponding to channel 127 of the window). For example, setting bit 13 of the `SPORT_CS1_A` register enables channel number 45 (add 32 for the channels in the `SPORT_CS0_A`). Likewise, setting bit 5 of the `SPORT_CS3_A` register enables channel number 101 (add 96 for the 32 channels in each of the `SPORT_CS0_A`, `SPORT_CS1_A`, and `SPORT_CS2_A` registers).

Multichannel Frame Delay (MFD)

The multichannel frame delay (`SPORT_MCTL_A.MFD`) field specifies the delay in serial bit clocks between the frame sync pulse and the first data bit in the frame. This configurability allows the processor to work with different types of telephony interface devices.

As `SPORT_MCTL_A.MFD` is a 4-bit field, the maximum value allowed for the frame delay is 15 serial clock cycles. When set to 0, the frame sync is concurrent with the first data bit. If `SPORT_MCTL_A.MFD > 0`, a new frame sync can occur during the last channel(s) of a previous frame and still be valid (does not cause a frame sync error).

NOTE: When the required frame delay exceeds 15 serial clocks, use the window offset field (`SPORT_MCTL_A.WOFFSET`) to delay the start of channel 0 in increments of the serial word length, and then adjust `SPORT_MCTL_A.MFD` accordingly. For example, when the serial word length is 12 bits and the desired frame delay is 16 serial clock cycles, set the `SPORT_MCTL_A.WOFFSET` field to 1 to insert a 12-bit delay after the frame sync to where the channel 0 data begins, and then program `SPORT_MCTL_A.MFD` to four (that is, $16 - 12$).

NOTE: When the ASRC module is used with SPORTs, note the following:

- In ASRC TDM mode, the delay between the frame sync (FS) and the data is one clock cycle.
- For the SPORT MFD feature with respect to ASRC TDM mode, select the MFD (`SPORT_MCTL_A.MFD / SPORT_MCTL_B.MFD`) bit as follows:
 - The ASRC input is expecting one clock cycle between the FS and the data. Configure the MFD transmit bit to 1 (the SPORT which transmits the data to ASRC serial in)
 - The ASRC output has one clock cycle delay between the FS and the data. Configure the MFD receive bit to 1 (for the SPORT which receives the ASRC serial out data).

Window Size (WSIZE)

Select the number of channels used in multichannel operation by programming the 7-bit `SPORT_MCTL_A.WSIZE` field. This field must be set to the actual number of channels minus one (`SPORT_MCTL_A.WSIZE = Number of channels - 1`).

The 10-bit `SPORT_MSTAT_A.CURCHAN` field holds the channel number currently being serviced during multi-channel operation.

Window Offset (WOFFSET)

The window offset (`SPORT_MCTL_A.WOFFSET`) field specifies where in the 1024-channel frame to place the start of the active window (up to 128 channels long). A value of 0 specifies no channel offset from the frame sync (channel 0 immediately follows it). Any non-zero value indicates the number of channels that come between the frame sync and the start of channel 0 of the active frame, with 896 (for example, $1024 - 128$) being the largest value that permits using all 128 channels.

As an example, a program could define an active window comprised of eight channels (`SPORT_MCTL_A.WSIZE = 7`) with a window offset of 93 (`SPORT_MCTL_A.WOFFSET = 93`). If configured in this fashion, the 8-channel window that the SPORT will transfer within resides in the channel range from 93 to 100 in the up-to-1024-channel frame.

Do not change the window offset or the number of multichannel slots (`SPORT_MCTL_A.WSIZE`) while the SPORT is enabled. If the combination of the window size and offset place any portion of the window out-of-range relative to the channel counter, none of the channels are enabled.

Companding Selection

Like the other operating modes, companding logic can optionally be applied to serial data (compression logic for transmit mode or expansion logic for receive mode). The two widely used companding algorithms, A-law and μ -law, are selectable using the `SPORT_CTL_A.DTYPE` field.

If companding is enabled, the companding algorithm is applied to both the primary and secondary datapaths. In multichannel mode, companding can be applied to either all or none of the enabled channels (companding cannot be selected on a per-channel basis).

Multichannel DMA Data Packing (MCPDE)

Multichannel DMA data packing and unpacking are enabled using the `SPORT_MCTL_A.MCPDE` bit.

When set, data is packed, and the SPORT expects the data in the DMA buffer to correspond only with enabled SPORT channels. For example, if only channels 1 and 9 are enabled in a 10-channel window (`SPORT_MCTL_A.WSIZE = 9`), the SPORT expects the buffer to be exactly two words in length, where channel 1 is associated with the first element in the buffer and channel 9 is associated with the second.

When cleared, data is unpacked, and the SPORT expects the DMA buffer to have a word for each of the channels in the active window, whether the channel is enabled or not. As such, the DMA buffer size must be exactly the size of the window. Using the same example as the packed case above, if only channels 1 and 9 are enabled in a 10-channel window (`SPORT_MCTL_A.WSIZE = 9`), then the DMA buffer size is ten words. The data at offsets 1 and 9 within the buffer are associated with the data transfers of channels 1 and 9, respectively. The rest of the words in the buffer are unused.

Packed I²S Mode

The SPORT supports a packed I²S mode, which can be used for audio codec communications using multiple channels. This mode allows applications to send more than the standard 32 bits per channel available through standard I²S mode. Packed mode is implemented using standard multichannel mode (and is therefore programmed similarly to multichannel mode).

To set the SPORT up in packed I²S mode, the following configuration is required:

- `SPORT_CTL_A.OPMODE = 1`
- `SPORT_MCTL_A.MCE = 1`

Like multichannel mode, packed I²S mode also supports a maximum of 128 channels, where up to 128 channels of data can be transferred for every transition of the frame sync signal acting as an L/R clock (for example, up to 128 left-channel words transfer during the high portion of the L/R clock, and up to 128 right-channel words transfer during the low portion).

As shown in the *Packed I²S Mode 128 Operation* figure, the packed waveforms are the same as those waveforms used in multichannel mode, except the frame sync is toggled for every frame and emulates I²S mode.

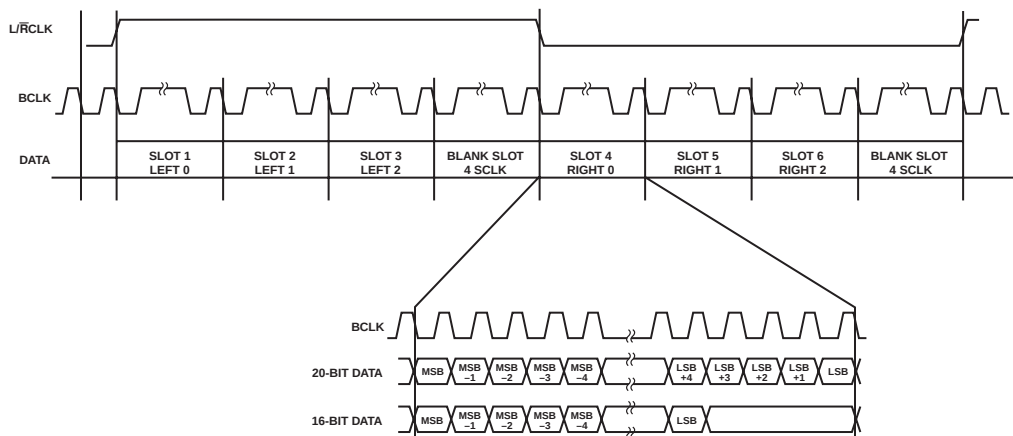


Figure 21-12: Packed I²S Mode 128 Operation

Serial Bit Clock Options

In packed I²S mode, the SPORTs can either accept an external serial bit clock or generate one internally, as governed by the `SPORT_CTL_A.ICLK` configuration bit. For an internally-generated serial bit clock (`SPORT_CTL_A.ICLK = 1`), use the `SPORT_DIV_A.CLKDIV` bit field to configure the serial bit clock rate from the system clock.

The serial bit clock edge that is used for sampling or driving data and frame syncs is programmable using the `SPORT_CTL_A.CKRE` bit.

L/R Clock (Frame Sync) Options

The frame sync period in packed I²S mode is defined as:

$$\text{FS period} = [(\text{SPORT_CTL_A.SLEN} + 1) \times \text{number of channels}] - 1.$$

The L/R clock can be supplied externally or internally generated depending on the `SPORT_CTL_A.IFS` bit setting. The logic level of the L/R clock associated with the left and right channel data can be changed using the `SPORT_CTL_A.LFS` configuration bit.

Gated Clock Mode

Some system components such as ADCs and DACs utilize a SPI-compatible protocol for the interface. To communicate with such devices, the SPORT must support a gated clock, where the data valid information is embedded in

the clock (for example, the clock only toggles when data is valid). This gated clock feature is enabled using the `SPORT_CTL_A.GCLKEN` bit.

To enable the gated clock mode of operation, program the SPORT to comply with the following requirements.

- Do not enable gated clock functionality in right-justified or multichannel mode
- Gated clock mode has the following requirements for other control bits:
 - The serial clock and frame sync signals must have the same source (`SPORT_CTL_A.ICLK = SPORT_CTL_A.IFS`)
 - Unframed mode is not supported (`SPORT_CTL_A.FSR` must be set)
 - Clear the `SPORT_CTL_A.DIFS` bit in transmit mode; set it in receive mode
- Satisfy the following necessary conditions when gated clock mode is enabled:
 - Seven serial clock cycles are required between enabling the SPORT and the first frame sync. If this requirement is not met, the SPORT can drop the first data (for subsequent data, this requirement is not applicable)
 - For externally-provided clock and frame sync, the frame sync must be inactive during clock synchronization after the SPORT has been enabled
 - For an edge-detected frame sync (`SPORT_CTL_A.FSED = 1`), the frame sync must transition back to the inactive state before the current word transfer is complete (or when the clock is still running). If this requirement is not met, the SPORT does not recognize the next valid frame sync and skips the channel. The SPORT continues to skip the frame syncs until the frame sync transitions back to an inactive state while the clock is active.

Data Transfers and Interrupts

SPORT data can be transferred to or from internal or external memory by two methods:

- Core-driven, single-word transfers
- DMA-driven, multiple-word transfers (optionally with multiple work units)

Core-driven transfers use SPORT interrupts to signal the processor core to perform MMR-based single-word transfers to or from the SPORT data buffers. DMA can be set up to automatically transfer a configurable number of serial words between the SPORT transmit/receive data buffers and memory, and then generate a data completion interrupt request when a work unit or a series of work units completes, thus signaling to the processor core that a block of data has been transferred.

The following sections provide information on core-driven and DMA-driven data transfers.

Data Buffers

When programming the serial port data channels (primary or secondary) as a transmitter by setting `SPORT_CTL_A.SPTRAN = 1`, only the corresponding transmit data buffers (`SPORT_TXPRI_A` and `SPORT_TXSEC_A`) become active. The receive data buffers (`SPORT_RXPRI_A` and `SPORT_RXSEC_A`) remain inactive. Similarly, when the SPORT data channels are programmed for receive operation (`SPORT_CTL_A.SPTRAN = 0`), then only corresponding receive data buffers (`SPORT_RXPRI_A` and `SPORT_RXSEC_A`) are active. Do not attempt to read or write inactive data buffers. When the processor operates on the inactive transmit or receive buffers while the SPORT is enabled, unpredictable results can occur.

Each of these buffers is 32-bit wide (corresponds to maximum serial data word length). When using word lengths less than 32 bits for SPORT operation, the data in these buffers is automatically right justified. (The LSB bit of data is at the bit 0 location of the buffer). The upper unused bits can be zero-filled or sign-extended depending on `SPORT_CTL_A.DTYPE` field.

Transmit Data Buffers (`SPORT_TXPRI_A` and `SPORT_TXSEC_A`)

When enabled as a transmitter (`SPORT_CTL_A.SPTRAN = 1`), each SPORT half has its own set of transmit data buffers. The primary (0) and secondary (1) datapaths of each SPORT half have separate data buffers, referred to as `SPORT_TXPRI_A` and `SPORT_TXSEC_A`, respectively.

These transmit data buffers are 32 bits wide. Load these buffers with the data for transmission on the primary and secondary data channels. The DMA controller loads the data automatically. Or, the program running on the processor core loads the data manually.

Together with the output shift register, transmit data buffers act like a two-location FIFO. If data packing is disabled (`SPORT_CTL_A.PACK = 0`), the transmit path can hold as many as three data words. If data packing is enabled (`SPORT_CTL_A.PACK = 1`), it can hold two packed data words at any given time.

When the transmit shift register becomes empty (transfer out all the bits of previous word), data in the transmit data buffer is automatically loaded into it. An interrupt occurs when the output transmit shift register has been loaded, signifying that the transmit data buffer is empty and ready to accept the next word. This interrupt does not occur when serial port is operating in DMA mode or when the corresponding interrupt enable mask bit is set.

If only the primary datapath of a SPORT half is enabled, programs must not write to the inactive secondary transmit data buffer and conversely. If the core keeps writing to the inactive buffer, the status of that transmit buffer becomes full. This state can cause the core to hang indefinitely, since data is never transmitted to the output shift register.

Receive Data Buffers (`SPORT_RXPRI_A` and `SPORT_RXSEC_A`)

When enabled as receiver (`SPORT_CTL_A.SPTRAN = 0`), each SPORT half has its own set of receive data buffers. The primary (0) and secondary (1) datapaths of each SPORT half have separate data buffers, referred as `SPORT_RXPRI_A` and `SPORT_RXSEC_A`, respectively. Together with input shift register, the receive data buffers act like a three-location FIFO, as the receive path has two data registers.

These receive data buffers are the 32 bits wide. These buffers are automatically loaded from the receive shift register when a complete word has been received into it. An interrupt occurs when the receive data buffer is loaded, signifying that new data is available in the receive data buffer and is ready to read. This interrupt does not occur when the serial port is operating in DMA mode or when the corresponding interrupt enable mask bit is set.

If only the primary datapath of a SPORT half is enabled, programs must not read from the inactive secondary receive data buffer and conversely. If the core keeps reading from the inactive buffer, the status of that receive buffer becomes empty. This state can cause the core to hang indefinitely since new data is never received through the input shift register.

Data Buffer Status

The SPORT provides status information about its primary and secondary data buffers through the `SPORT_CTL_A.DXSPRI` and `SPORT_CTL_A.DXSSEC` bits, respectively. It also provides error status information through the corresponding `SPORT_CTL_A.DERRPRI` and `SPORT_CTL_A.DERRSEC` bits, respectively. Depending on the `SPORT_CTL_A.SPTRAN` bit setting, these bits reflect the status of either the pair of transmit (`SPORT_TXPRI_A` and `SPORT_TXSEC_A`) or receive (`SPORT_RXPRI_A` and `SPORT_RXSEC_A`) buffers, indicating whether the buffer is full, partially full, or empty.

When attempting to read from an empty receive buffer or write to a full transmit buffer, the SPORT delays access until the buffer is ready, potentially resulting in excessive MMR bus response times. To avoid this when doing core-driven transfers, always check the buffer status to determine if the access can be made. The SPORT updates the status bits in the `SPORT_CTL_A` register during reads and writes by the core processor.

NOTE: These status bits are updated during reads and writes from the core processor even when the SPORT is disabled.

Two complete 32-bit words can be stored in the receive buffer while a third word shifts in. Therefore, almost three complete words can be received without the receive buffer being read before an overflow occurs. After receiving the third word completely, the shift register contents overwrite the second word, which will occur if the first word has not yet been read by the processor core or the DMA controller. This receive overflow condition is flagged through the error status bits of the `SPORT_CTL_A` register on the last bit of the third word.

Data Buffer Packing

When the SPORT is configured as a receiver with a serial data word length of 16 or less, the received data words can be packed into a 32-bit word. Similarly, if the SPORT is configured as a transmitter with a serial data word length of 16 or less, then 32-bit words being transmitted can be unpacked into 16-bit words. The `SPORT_CTL_A.PACK` bit is used to select this packing or unpacking feature.

When `SPORT_CTL_A.PACK = 1`, two consecutive received words are packed into a single 32-bit word, or each 32-bit word is unpacked and transmitted as two 16-bit words. The first 16-bit (or smaller) word is right-justified in bits 15–0 of the packed word, and the second 16-bit (or smaller) word is right-justified in bits 31–16. This packing method applies to both receive (packing) and transmit (unpacking) operations. In this case, the transmit and receive interrupt requests are generated for the 32-bit packed words, not for each 16-bit word.

NOTE: When 16-bit received data is packed into 32-bit words and stored in normal word space in the processor's internal memory, the 16-bit words can be read or written using short word space addressing.

Single-Word (Core) Transfers

The SPORTs can transmit or receive individual data words with interrupt requests occurring as each data word is transferred. When a SPORT is enabled with the corresponding DMA channel disabled, interrupt requests are generated when:

- a complete word has been received in the receive data buffer or
- the transmit data buffer is not full

When performing core transfers, be sure to access only those buffers that are associated with enabled datapaths, as governed by the transfer direction (`SPORT_CTL_A.SPTRAN`) bit and the primary/secondary (`SPORT_CTL_A.SPENPRI`/`SPORT_CTL_A.SPENSEC`) data enable bits. If inactive SPORT data buffers are read from or written to by the core while the SPORT is enabled, the core can hang. For example, if a half SPORT is programmed to be a transmitter and the core reads from one of the receive buffers associated with that half SPORT, the core can hang as if it were reading an empty buffer that is active and awaiting new data to arrive. Because this is a transmitting HSPORT, that data will never arrive, thus locking the core up until the SPORT is reset. To avoid such a situation, be sure to check the status of the appropriate data buffer before attempting a core access to it by interrogating the `SPORT_CTL_A.DXSPRI` or `SPORT_CTL_A.DXSSEC` status bits.

DMA Transfers

Direct memory access (DMA) provides a mechanism for transferring an entire block of serial data before an interrupt is generated. The processor's on-chip DMA controller automatically handles the DMA transfer, thus allowing the processor core to run in parallel until the entire block of data is transferred. When the interrupt request occurs, a service routine can then process the entire block of data (rather than react to single words), thus significantly reducing overhead.

Each half SPORT has a dedicated DMA channel that serves both the primary and secondary datapaths. When configured as a transmitter (`SPORT_CTL_A.SPTRAN = 1`) with both the primary and secondary datapaths enabled (`SPORT_CTL_A.SPENPRI = SPORT_CTL_A.SPENSEC = 1`), the DMA channel requires that the source DMA buffer interleave the data beginning with the primary channel, as it will alternately load to the primary and secondary transmit data buffers once it is enabled. The complementary operation is true in receive mode (`SPORT_CTL_A.SPTRAN = 0`) when both datapaths are enabled, as the DMA channel alternately reads from the primary and secondary receive data buffers and interleaves them in the destination DMA buffer. As such, software must de-interleave the data corresponding to the primary and secondary channels from the receive DMA buffer.

If the SPORT is configured in stereo mode, the same DMA channel handles both the left and right channels of both datapaths (primary and/or secondary). Therefore, for a transmit DMA with only one datapath enabled, the source buffer must be populated such that the left- and right-channel data is interleaved. If both datapaths are enabled, the DMA channel alternately loads to the primary and secondary transmit data buffers once it is enabled. As such, the interleaving requirement is for the primary left-channel data to be followed by the secondary left-channel data, then the primary right-channel data, and finally the secondary right-channel data. The complementary operation is true

in receive mode, where the DMA channel alternately reads from the primary and secondary receive data buffers and interleave them in the destination DMA buffer. For the stereo modes of operation, the destination DMA buffer is interleaved as left-right data for a single data input. If both datapaths are enabled, the destination DMA buffer is written with the primary and secondary left-channel data followed by the primary and secondary right-channel data. As such, software must de-interleave the primary and secondary left- and right-channel data from the receive DMA buffer, as defined by this scheme.

Since both the primary and secondary datapaths share the single DMA channel, each half SPORT has a single interrupt request for data completion, as well as an error interrupt request. The DMA controller can generate an interrupt request at the end of a chain of DMA work units (when using multiple descriptors) or at the end of individual DMA work unit.

The SPORT DMA channels are assigned a higher priority than all the other DMA channels (for example, the SPI port). Having higher priority causes the SPORT DMA transfers to execute first when multiple DMA requests occur in the same cycle. The SPORT DMA channels are numbered and prioritized in the DMA channel list table in the DMA chapter.

Although the most efficient DMA transfers execute with 32-bit words, the SPORTs can handle word sizes from 4 to 32 bits (as defined by `SPORT_CTL_A.SLEN` field). If the serial data length is 16 bits or smaller, two pieces of data can be packed into 32-bit words for each DMA transfer, as selected by setting the `SPORT_CTL_A.PACK` bit. When this bit is set, the SPORT generates the transmit and receive interrupts for the 32-bit packed words, not for each 16-bit word. For more information, see the [Data Buffer Status](#) section.

NOTE: The SPORT DMA channel can access both internal memory and external memory of the processor without any core overhead.

Data Transfer Interrupt

Each half SPORT features a data transfer interrupt request that is shared by both the primary and secondary data channels in both transmit and receive modes. To determine the source of the data transfer interrupt request, applications can check the primary and secondary data buffer status bits (`SPORT_CTL_A.DXSPRI` and `SPORT_CTL_A.DXSSEC`, respectively).

When using core-driven transfers, this interrupt's meaning depends on the direction of the SPORT:

- As transmitter (`SPORT_CTL_A.SPTRAN = 1`) - the transmit data buffer is empty
- As receiver (`SPORT_CTL_A.SPTRAN = 0`) - new data is available in the receive data buffer

NOTE: When data packing is enabled (`SPORT_CTL_A.PACK = 1`), the core-driven transmit and receive interrupt requests are generated for 32-bit packed words, not for each 16-bit word.

In both cases, the interrupt request can be used to signal the core that an individual transfer has completed. For transmit operations, it indicates that the transmit data buffer can be safely loaded (either the buffer is already empty, or the last data has moved from the data buffer to the shift register). For receive operations, it indicates that new data has arrived and can be read (or must be read before a subsequent word overwrites it).

When the SPORT is configured to use DMA to move data between memory and the peripheral (the most generic way to use dedicated DMA for sport data transfers), the same data transfer interrupt request instead indicates the completion of the transfer of a block of serial data (rather than a single word). When DMA is used, the DMA count register must be initialized to specify the number of words to transfer. This count decrements after each DMA transfer on the channel, and the data transfer interrupt request signal is asserted when the word count reaches zero (for example, a DMA work unit has finished).

For transmit DMA, the interrupt request is raised when the last word in the DMA work unit is loaded from the source memory to the HSPORT FIFO. This interrupt request can signal to the core that a new DMA work unit can be configured or that other software threads can now run. The transmit interrupt request can optionally be deferred until the last word of the work unit has fully shifted out of the shift register (see the Transfer Finish Interrupt (TFI) section for details).

For receive DMA, the interrupt request is raised when the last word is loaded to the destination memory. In addition to that described for transmit DMA, this interrupt request also serves as an indication to the core that there is a buffer of newly acquired data that is ready to be processed.

See the DMA chapter for further details regarding enabling of the DMA interrupt requests associated with the various modes of DMA operation.

NOTE: As a single DMA channel services both the primary and secondary datapaths associated with the SPORT, there is a single DMA completion interrupt request.

Transfer Finish Interrupt (TFI)

When configured for transmit DMA (`SPORT_CTL_A.SPTRAN = 1`), the data transfer interrupt request gets generated by the DMA engine itself when it decrements its count register upon loading the last element from memory to the HSPORT hardware. Alternately, the SPORT can use a Transmit Finish Interrupt (TFI) to signal the actual end of the transmission (for example, when the last bit of the last data word of the buffer has shifted out of the SPORT to the system) by setting the `SPORT_CTL_A.TFIEN` bit. When this bit is set, then DMA signal that would normally assert the data transfer interrupt request instead signals the SPORT that the DMA work unit is complete. The SPORT then waits until all the data in the FIFO is shifted out (including the transmit shift register) and asserts the TFI interrupt request upon completion.

NOTE: To enable this functionality in the DMA engine, be sure to configure the interrupt type field in the DMA configuration register for Peripheral interrupt. See the DMA chapter for further details.

Error Detection (Status) Interrupt

In addition to the dedicated data transfer interrupt request, each half SPORT also features an optional error status interrupt request that can be triggered when error conditions occur relative to data or frame syncs associated with the half SPORT.

Data-related errors depend on the direction of the SPORT and reflect overflow or underflow conditions, which are depicted in the [SPORT_CTL_A](#) control register as read-only sticky bits `SPORT_CTL_A.DERRPRI` and `SPORT_CTL_A.DERRSEC` (for the primary and secondary channels, respectively).

- When the SPORT is configured as a transmitter, these bits provide transmit data buffer underflow status. When the frame sync signal occurs when the transmit data buffer is empty, the underflow bit corresponding with the offending transmit data buffer is set, as the SPORT will transmit data whenever it detects a valid frame sync signal, whether new data is present or not.
- When the SPORT is configured as a receiver, these bits provide receive overflow status. When a channel receives new data while the receive buffer is already full, the new data overwrites the existing data, thus causing an overflow. When this occurs, the overflow bit corresponding with the offending receive data buffer is set, as the SPORT receives data whenever it detects a valid frame sync signal, whether there is room in the receive buffer or not.

Each half SPORT also features an error register ([SPORT_ERR_A](#)), which is the source for the assertion of the described data-related error status bits. When a data-related error occurs on the primary or secondary datapaths, the error is logged in the `SPORT_ERR_A.DERRPSTAT` or `SPORT_ERR_A.DERRSSTAT` bits, respectively. To enable these status bits to generate the HSPORT status interrupt request in the SEC, the corresponding `SPORT_ERR_A.DERRPMSK` and `SPORT_ERR_A.DERRSMSK` bits must be set (for the primary and secondary datapaths, respectively).

The `SPORT_CTL_A.DERRPRI` and `SPORT_CTL_A.DERRSEC` channel error status bits are sticky read-only bits that can be cleared in two ways:

- Reset the error detection logic by disabling the channel associated with the error condition (clear the `SPORT_CTL_A.SPENPRI` or `SPORT_CTL_A.SPENSEC` control bit).
- Clear the source of the interrupt by writing-1-to-clear the `SPORT_ERR_A.FSERRSTAT`, `SPORT_ERR_A.DERRPSTAT`, or `SPORT_ERR_A.DERRSSTAT` status bits.

In addition to data-related errors, [SPORT_ERR_A](#) also tracks frame sync errors in the `SPORT_ERR_A.FSERRSTAT` status bit. Similar to the data-related errors, the frame sync error can be enabled as a source for raising the error status interrupt request via the SEC by setting the `SPORT_ERR_A.FSERRMSK` bit. A frame sync error occurs when the frame sync is detected prematurely, as explained in the [Premature Frame Sync Error Detection](#) section.

A frame sync error is not detected in the following cases:

- When there is no active transmit or receive data, and the frame sync pulse occurs due to noise on the input signal—when there is no active transfer, a noise-induced frame sync pulse is valid.
- When there is an active underflow or overflow error—frame sync errors cannot be detected because the SPORT error logic does not run after one of the data errors has occurred and remains un-serviced.
- When the frame sync pulse does not meet minimum timing requirements—when the frame sync pulse is shorter than a SPORT clock period, there is no guarantee that it gets sampled and may go unnoticed.

Grouping of SPORTs

For some applications, enabling multiple SPORTs need to be synchronized so that all of them start and/or end at the same time. It may also be required to generate/receive a single interrupt/trigger for the DMA transfer of all the SPORTs in the same group.

The `DAI_GBL_SP_EN` register is used to control enabling/disabling. The `DAI_GBL_INT_EN` register can be used to control interrupt/trigger generation/reception by multiple SPORTs in a group.

NOTE: All references to SPORT 0-3 in the context of DAI1 must be read as SPORT4-7.

SPORT Enable Grouping

The figure shows the block diagram of possible group enabling options for the SPORTs in DAI0 and DAI1.

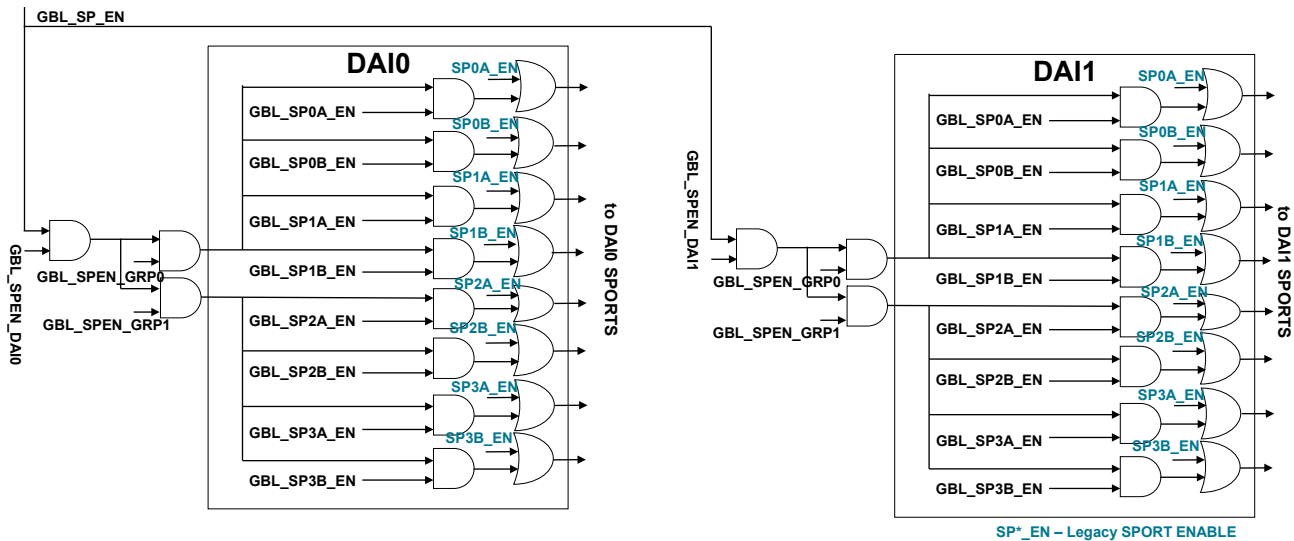


Figure 21-13: Group Enabling Options in DAI0 and DAI1

It shows one of the channels in each half sport, same is applicable for other channel as well. For example, `GBL_SP0A_EN` means `GBL_SP0A_PC_EN` or `GBL_SP0A_SC_EN`.

Each DAI has a `DAI_GBL_SP_EN` register. To enable the SPORTs grouped within the same DAI at the same time, write to the corresponding `DAI_GBL_SP_EN` register is required. When a group of SPORTs in different DAIs must be enabled simultaneously, writing to the `DAI_GBL_SP_EN.GBL_SP_EN` field is required. This bit is reserved for the DAI1.

The following group enable options are possible:

- Up to 16 channels in 8 half SPORTs in DAI0
- Up to 16 channels in 8 half SPORTs in DAI1
- Up to 8 channels in 4 half SPORTs in each DAI
- Up to 32 channels in 16 half SPORTs (8 half SPORTs in DAI0 and 8 half SPORTs in DAI1)

NOTE: While grouping SPORTs across DAIs, two interrupts (one per DAI) are generated and managed by software. Grouping can be done only once as the `DAI_GBL_SP_EN.GBL_SP_EN` field is used.

NOTE: The grouping of SPORTs and its interrupts/triggers are two separate entities. When the interrupts and triggers must be grouped similar to the SPORT grouping, use the group interrupt/trigger enable register.

SPORT Interrupt/Trigger Grouping

Each DAI has a `DAI_GBL_INT_EN` register. Each of these registers has two group interrupt enable bits `DAI_GBL_INT_EN.GRP0_INT_EN` and `DAI_GBL_INT_EN.GRP1_INT_EN`, two group trigger (controller) enable bits `DAI_GBL_INT_EN.GRP0_TRG_EN` and `DAI_GBL_INT_EN.GRP1_TRG_EN`. There are individual `GRPx_SPyINT_EN` bits for each group to enable both interrupt/trigger for up to 8 channels (4 half SPORTs). Enable these bits as per the grouping in the `DAI_GBL_SP_EN` register.

NOTE: To enable or to disable the SPORT interrupt/trigger grouping feature, the individual sport interrupt/trigger enable bits must be programmed along with the respective group interrupt/trigger enable bits in the `DAI_GBL_INT_EN` register.

Regarding grouping of SPORT interrupts/triggers:

- All the interrupts/triggers in a single group are ANDed and a single interrupt/trigger goes to SEC/TRU.
- To generate group interrupt/trigger, all the SPORTs in the corresponding group should be configured to generate interrupt (`DMA_CFG.INT`)/trigger (`DMA_CFG.TRIG`).
- The interrupts/triggers generated by the SPORTs in a group can either be used as a single group interrupt/trigger (enabled by the `DAI_GBL_INT_EN` register) or the individual SPORT DMA done interrupts/triggers.
- As there are no separate status registers for global interrupts, to clear the global interrupts, clear (W1C) the individual SPORT `DMA_STAT.IRQDONE` bits. Theoretically, the global interrupt gets cleared by clearing the `DMA_STAT.IRQDONE` bit of at least one SPORT in the group. However, it is recommended to clear the `DMA_STAT.IRQDONE` bits of all the SPORTs in the group to achieve a clean state for the next global interrupt generation.
- The `DAI_GBL_INT_EN.GRP1_TRG_EN/DAI_GBL_INT_EN.GRP0_TRG_EN` bits enables group controller trigger functionality for SPORTs in a group.

SPORT Programming Model

The following sections provide programming guidance for setting up the SPORTs for use in an application:

- [Initializing Core-Driven \(Non-MCM\) Transfers](#)
- [Initializing Multichannel Transfers](#)
- [Using DMA for SPORT Transfers](#)
- [Using Companding as a Function](#)

Initializing Core-Driven (Non-MCM) Transfers

The following programming model applies to all of [Standard DSP Serial Mode](#), [I²S Mode](#), [Left-Justified Mode](#), and [Right-Justified Mode](#) for core-driven transfers. More steps are required to properly initialize the SEC to service the SPORT interrupts (see the SEC chapter for details).

NOTE: This example uses half SPORT A registers. With appropriate changes to register names, this example also applies to half SPORT B.

1. Clear the [SPORT_CTL_A](#) and [SPORT_MCTL_A](#) configuration registers.

ADDITIONAL INFORMATION: Clearing these registers ensures that the SPORT logic (including the multi-channel logic) is fully reset before attempting to reprogram it.

2. Optionally program the [SPORT_DIV_A](#) clock divisor register.

ADDITIONAL INFORMATION: This step is only required for internally-generated timing signals. Configure the serial bit clock and/or frame sync (or L/R clock, for stereo modes) rates according to the guidance in the [Serial Clock](#) and [Frame Sync](#) sections.

3. Program the [SPORT_CTL_A](#) primary configuration register.

ADDITIONAL INFORMATION: Set the SPORT operating mode along with the configurable clock, frame sync, word length, direction, and data format options (see the [Operating Modes and Options](#) section for details). Do not set the [SPORT_CTL_A](#).SPENPRI and/or [SPORT_CTL_A](#).SPENSEC buffer enable bits in this step.

4. Optionally program the [SPORT_CTL2_A](#) secondary configuration register.

ADDITIONAL INFORMATION: This step is required only if internal multiplexing logic must be enabled to share clock and frame sync signals between a top SPORT module's A and B halves (see the [Multiplexer Logic](#) section for details).

5. Optionally program the [SPORT_ERR_A](#) error register.

ADDITIONAL INFORMATION: This step is required only if a separate SPORT error interrupt is desired (see the [Error Detection \(Status\) Interrupt](#) section for details).

6. For Right-Justified mode only, program the [SPORT_MCTL_A](#).WOFFSET field.

ADDITIONAL INFORMATION: In Right-Justified mode, this field serves as the delay count (DCNT) required to align the LSB of each stereo channel with the L/R clock transition and must be programmed manually (see the [Right-Justified Mode](#) section for details).

7. Enable the primary/secondary datapath(s) in the [SPORT_CTL_A](#) register.

ADDITIONAL INFORMATION: This should be performed in a read-modify-write operation setting the [SPORT_CTL_A](#).SPENPRI and/or [SPORT_CTL_A](#).SPENSEC bits, as appropriate.

8. Write data to be transmitted to the transmit buffer ([SPORT_TXPRI_A](#) and/or [SPORT_TXSEC_A](#)) or read data that has been received from the receive buffer ([SPORT_RXPRI_A](#) and/or [SPORT_RXSEC_A](#)).

ADDITIONAL INFORMATION: These accesses are typically performed in the context of an interrupt service routine. See the SEC chapter for further information. Do not attempt to read or write inactive data buffers. If the core attempts to access inactive transmit or receive buffers while the SPORT is enabled, unpredictable results may occur.

Initializing Multichannel Transfers

When in [Multichannel \(TDM\) Mode](#) or [Packed I²S Mode](#), the SPORT is in a multichannel operational mode. Follow the steps below to properly initialize the SPORT for multichannel modes of operation. More steps are required to properly (see the SEC chapter for details).

NOTE: This example uses half SPORT A registers. With appropriate register changes, this example also applies to half SPORT B.

1. Clear the [SPORT_CTL_A](#) and [SPORT_MCTL_A](#) registers.

ADDITIONAL INFORMATION: Clearing these registers ensures that the SPORT logic (including the multichannel logic) is fully reset before attempting to reprogram it.

2. Optionally program the [SPORT_DIV_A](#) clock divisor register.

ADDITIONAL INFORMATION: This step is only required for internally-generated timing signals. Configure the serial bit clock and/or frame sync (or L/R clock, for stereo modes) rates according to the guidance in the [Serial Clock](#) and [SPORT_CTL2_A](#) sections.

3. Program the [SPORT_CS0_A](#) - [SPORT_CS3_A](#) channel select registers.

4. Program the [SPORT_MCTL_A](#) multichannel configuration register.

ADDITIONAL INFORMATION: The SPORT supports many multichannel options. For more information, see the [Multichannel \(TDM\) Mode](#) section. Do not set the [SPORT_MCTL_A.MCE](#) enable bit in this step.

5. Program the [SPORT_CTL_A](#) primary configuration register.

ADDITIONAL INFORMATION: Set the SPORT operating mode along with the configurable clock, frame sync, word length, direction, and data format options (see the [Operating Modes and Options](#) section for details). Do not set the [SPORT_CTL_A.SPENPRI](#) and/or [SPORT_CTL_A.SPENSEC](#) buffer enable bits in this step.

6. Optionally program the [SPORT_CTL2_A](#) secondary configuration register.

ADDITIONAL INFORMATION: This step is required only if internal multiplexing logic must be enabled to share clock and frame sync signals between a top SPORT module's A and B halves (see the [Multiplexer Logic](#) section for details).

7. Optionally program the [SPORT_ERR_A](#) error register.

ADDITIONAL INFORMATION: This step is required only if a separate SPORT error interrupt request is desired (see the [Error Detection \(Status\) Interrupt](#) section for details).

8. Set the `SPORT_MCTL_A.MCE` bit to enable multichannel mode.
9. Enable the primary/secondary datapath(s) in the `SPORT_CTL_A` register.

ADDITIONAL INFORMATION: This should be performed in a read-modify-write operation setting the `SPORT_CTL_A.SPENPRI` and/or `SPORT_CTL_A.SPENSEC` bits, as appropriate. DMA mode is recommended for multichannel modes of operation. For more information, see the [Using DMA for SPORT Transfers](#) programming model.

Using DMA for SPORT Transfers

DMA is supported in all SPORT operating modes ([Standard DSP Serial Mode](#), [I²S Mode](#), [Left-Justified Mode](#), [Right-Justified Mode](#), [Multichannel \(TDM\) Mode](#) or [Packed I²S Mode](#)). To enable DMA operation with the SPORT, execute the steps described in this section after initializing and enabling the SPORT. Instead of using the single word read or write operations described in the referenced programming models, the DMA engine automates accesses to the enabled SPORT data buffers.

NOTE: This example uses half SPORT A registers. With appropriate changes to register names, it also applies to half SPORT B.

1. Follow the guidance in the multichannel ([Initializing Multichannel Transfers](#)) or non-multichannel ([Initializing Core-Driven \(Non-MCM\) Transfers](#)) programming models to properly initialize and enable the SPORT hardware.
2. Prepare the data buffers in memory.

ADDITIONAL INFORMATION: Ensure that the DMA buffer is defined according to the [DMA Transfers](#) section. For the multichannel modes of operation, be sure to also consider the setting of the `SPORT_MCTL_A.MCPDE` bit, as described in the [Multichannel DMA Data Packing \(MCPDE\)](#) section.

3. Initialize and enable the DMA channel allocated for the SPORT, as described in the Direct Memory Access (DMA) chapter.

Using Companding as a Function

The data in the transmit and receive buffers are companded in place. The following programming model can be used to exercise the companding hardware without transferring data, which is useful for test and debug purposes.

NOTE: This example uses half SPORT A registers. With appropriate changes to register names, this example also applies to half SPORT B.

1. Configure the SPORT as a transmitter (`SPORT_CTL_A.SPTRAN=1`) with both the primary and secondary data channels disabled (`SPORT_CTL_A.SPENPRI=0` and `SPORT_CTL_A.SPENSEC=0`).

2. Enable the desired companding scheme in the `SPORT_CTL_A.DTYPE` field.
3. Write a 32-bit word to one of the transmit buffers.
4. Wait two system clock cycles.

ADDITIONAL INFORMATION: This delay is required to allow the SPORT companding hardware to reload the transmit buffer with the companded result. Any instructions that do not access the transmit buffer can be used to cause this delay.

5. Read the 8-bit compressed value from the transmit buffer written above.

To expand data in place, use the same sequence of operations with the receive buffer instead of the transmit buffer. When expanding data in this way, set the appropriate serial word length (`SPORT_CTL_A.SLEN`).

Programming Global SPORT Groups

Complete the following steps to program global SPORT groups.

NOTE: To complete all of the SPORT group transfers simultaneously, all of the SPORTs in the group must have a similar data/clock/frame sync configuration.

1. For each SPORT, program the following registers for the desired SPORT mode of operation.
 - `SPORT_DIV_A/SPORT_DIV_B`
 - `SPORT_CTL2_A/SPORT_CTL2_B`
 - `SPORT_CTL_A/SPORT_CTL_B`
 - `SPORT_MCTL_A/SPORT_MCTL_B`
 - `SPORT_CS0_A/SPORT_CS0_B`
 - `SPORT_CS1_A/SPORT_CS1_B`
 - `SPORT_CS2_A/SPORT_CS2_B`
 - `SPORT_CS3_A/SPORT_CS3_B`

ADDITIONAL INFORMATION: The sport enable control bits (in `SPORT_CTL2_A/SPORT_CTL2_B` registers) must be cleared (disabled).

2. For a global group trigger as the controller, program each SPORT DMA in the group as the controller trigger.
3. Program the sport enable and group enable bits in the `DAI_GBL_SP_EN` register for the desired global sport grouping.

ADDITIONAL INFORMATION: The grouping of SPORTs (enable and disable) and the associated interrupts/triggers are separate activities. If the interrupts and triggers must be grouped similar to the SPORTs (enable and disable) grouping, use the `DAI_GBL_INT_EN` register.

4. Program the `DAI_GBL_INT_EN` register to enable the grouped interrupt or trigger generation for the SPORT group.

ADDITIONAL INFORMATION: To enable the SPORT grouped interrupt and trigger feature, each sport interrupt or trigger enable bit must be programmed with the respective group interrupt and trigger enable bits in the `DAI_GBL_INT_EN` register.

5. Program the `DAI_GBL_SP_EN.GBL_SP_EN` bit to enable the global sport groups at the same instance.

Disabling Global SPORT Groups

Complete the following steps to disable global SPORT groups.

1. Use the individual SPORT DMA status register to clear the group interrupt. There is no specific status register for global interrupts.
2. Disable the global SPORT group interrupt and trigger bits for the group in the `DAI_GBL_INT_EN` register.

ADDITIONAL INFORMATION: To disable the SPORT grouped interrupt and trigger feature, each sport interrupt and trigger enable bit must be programmed with the respective group interrupt and trigger enable bits in the `DAI_GBL_INT_EN` register.

3. Disable the global individual SPORT enable and the global SPORT group enable bits in the `DAI_GBL_SP_EN` register.
4. Disable the DMAs for the SPORTs when the transfer is complete.

ADSP-21560_21561_21564_21568 SPORT Register Descriptions

Serial Port (SPORT) contains the following registers.

Table 21-14: ADSP-21560_21561_21564_21568 SPORT Register List

Name	Description
<code>SPORT_CS0_A</code>	Half SPORT 'A' Multichannel 0-31 Select Register
<code>SPORT_CS0_B</code>	Half SPORT 'B' Multichannel 0-31 Select Register
<code>SPORT_CS1_A</code>	Half SPORT 'A' Multichannel 32-63 Select Register
<code>SPORT_CS1_B</code>	Half SPORT 'B' Multichannel 32-63 Select Register
<code>SPORT_CS2_A</code>	Half SPORT 'A' Multichannel 64-95 Select Register
<code>SPORT_CS2_B</code>	Half SPORT 'B' Multichannel 64-95 Select Register
<code>SPORT_CS3_A</code>	Half SPORT 'A' Multichannel 96-127 Select Register
<code>SPORT_CS3_B</code>	Half SPORT 'B' Multichannel 96-127 Select Register
<code>SPORT_CTL2_A</code>	Half SPORT 'A' Control 2 Register
<code>SPORT_CTL2_B</code>	Half SPORT 'B' Control 2 Register

Table 21-14: ADSP-21560_21561_21564_21568 SPORT Register List (Continued)

Name	Description
SPORT_CTL_A	Half SPORT 'A' Control Register
SPORT_CTL_B	Half SPORT 'B' Control Register
SPORT_DIV_A	Half SPORT 'A' Divisor Register
SPORT_DIV_B	Half SPORT 'B' Divisor Register
SPORT_ERR_A	Half SPORT 'A' Error Register
SPORT_ERR_B	Half SPORT 'B' Error Register
SPORT_MCTL_A	Half SPORT 'A' Multichannel Control Register
SPORT_MCTL_B	Half SPORT 'B' Multichannel Control Register
SPORT_MSTAT_A	Half SPORT 'A' Multichannel Status Register
SPORT_MSTAT_B	Half SPORT 'B' Multichannel Status Register
SPORT_RXPRI_A	Half SPORT 'A' Rx Buffer (Primary) Register
SPORT_RXPRI_B	Half SPORT 'B' Rx Buffer (Primary) Register
SPORT_RXSEC_A	Half SPORT 'A' Rx Buffer (Secondary) Register
SPORT_RXSEC_B	Half SPORT 'B' Rx Buffer (Secondary) Register
SPORT_TXPRI_A	Half SPORT 'A' Tx Buffer (Primary) Register
SPORT_TXPRI_B	Half SPORT 'B' Tx Buffer (Primary) Register
SPORT_TXSEC_A	Half SPORT 'A' Tx Buffer (Secondary) Register
SPORT_TXSEC_B	Half SPORT 'B' Tx Buffer (Secondary) Register

Half SPORT 'A' Multichannel 0-31 Select Register

Each of the bits (when set, =1) of the `SPORT_CS0_A` register correspond to an active channel for the half SPORT in multichannel mode. When the register activates a channel (corresponding bit =1), the half SPORT transmits or receives the word in that channel's position of the data stream. When the register deactivates a channel (corresponding bit =0), the half SPORT either three-states its data transmit pin (during the channel's transmit time slot) or ignores incoming data (during the channel's receive time slot).

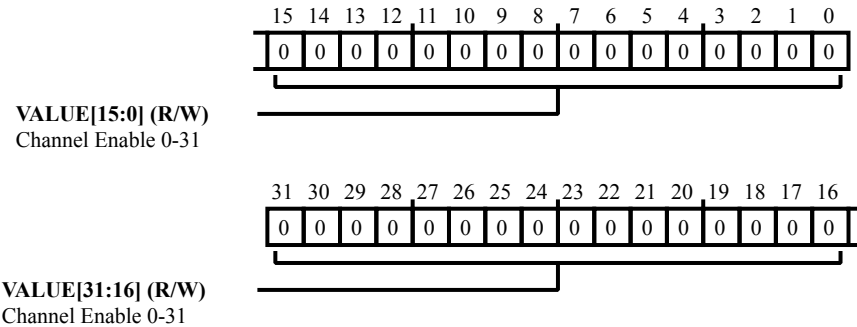


Figure 21-14: `SPORT_CS0_A` Register Diagram

Table 21-15: `SPORT_CS0_A` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Channel Enable 0-31.

Half SPORT 'B' Multichannel 0-31 Select Register

Each of the bits (when set, =1) of the `SPORT_CS0_B` register correspond to an active channel for the half SPORT in multichannel mode. When the register activates a channel (corresponding bit =1), the half SPORT transmits or receives the word in that channel's position of the data stream. When the register deactivates a channel (corresponding bit =0), the half SPORT either three-states its data transmit pin (during the channel's transmit time slot) or ignores incoming data (during the channel's receive time slot).

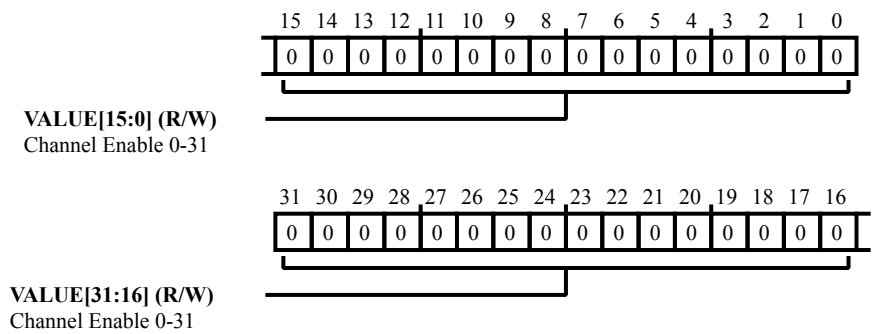


Figure 21-15: `SPORT_CS0_B` Register Diagram

Table 21-16: `SPORT_CS0_B` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Channel Enable 0-31.

Half SPORT 'A' Multichannel 32-63 Select Register

Each of the bits (when set, =1) of the `SPORT_CS1_A` register correspond to an active channel for the half SPORT in multichannel mode. When the register activates a channel (corresponding bit =1), the half SPORT transmits or receives the word in that channel's position of the data stream. When the register deactivates a channel (corresponding bit =0), the half SPORT either three-states its data transmit pin (during the channel's transmit time slot) or ignores incoming data (during the channel's receive time slot).

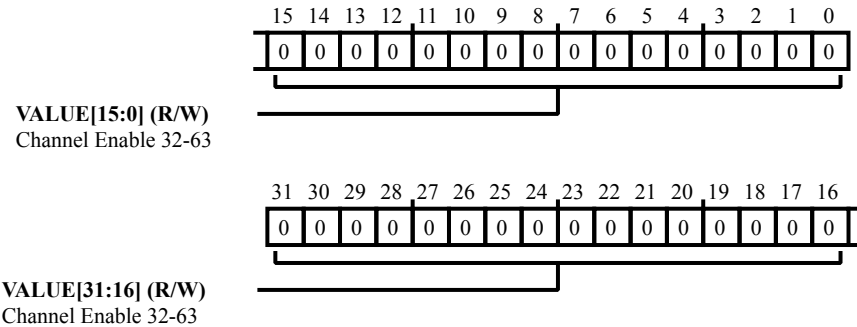


Figure 21-16: `SPORT_CS1_A` Register Diagram

Table 21-17: `SPORT_CS1_A` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Channel Enable 32-63.

Half SPORT 'B' Multichannel 32-63 Select Register

Each of the bits (when set, =1) of the `SPORT_CS1_B` register correspond to an active channel for the half SPORT in multichannel mode. When the register activates a channel (corresponding bit =1), the half SPORT transmits or receives the word in that channel's position of the data stream. When the register deactivates a channel (corresponding bit =0), the half SPORT either three-states its data transmit pin (during the channel's transmit time slot) or ignores incoming data (during the channel's receive time slot).

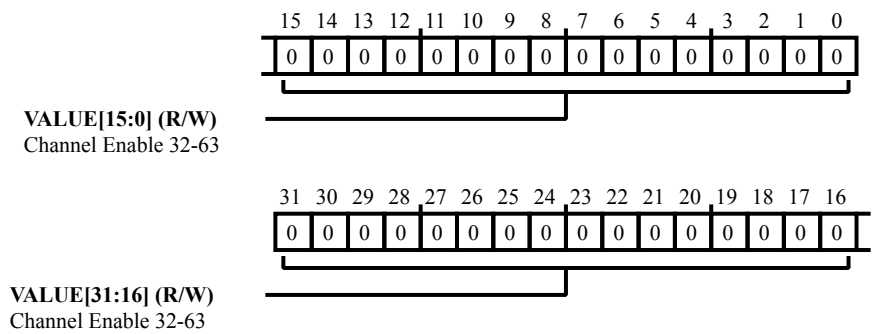


Figure 21-17: `SPORT_CS1_B` Register Diagram

Table 21-18: `SPORT_CS1_B` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Channel Enable 32-63.

Half SPORT 'A' Multichannel 64-95 Select Register

Each of the bits (when set, =1) of the `SPORT_CS2_A` register correspond to an active channel for the half SPORT in multichannel mode. When the register activates a channel (corresponding bit =1), the half SPORT transmits or receives the word in that channel's position of the data stream. When the register deactivates a channel (corresponding bit =0), the half SPORT either three-states its data transmit pin (during the channel's transmit time slot) or ignores incoming data (during the channel's receive time slot).

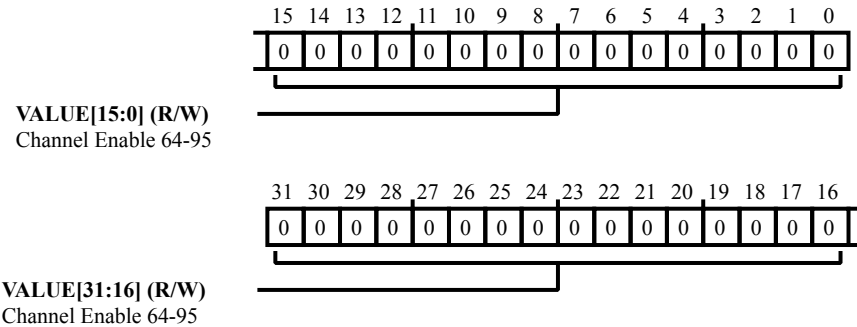


Figure 21-18: `SPORT_CS2_A` Register Diagram

Table 21-19: `SPORT_CS2_A` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Channel Enable 64-95.

Half SPORT 'B' Multichannel 64-95 Select Register

Each of the bits (when set, =1) of the `SPORT_CS2_B` register correspond to an active channel for the half SPORT in multichannel mode. When the register activates a channel (corresponding bit =1), the half SPORT transmits or receives the word in that channel's position of the data stream. When the register deactivates a channel (corresponding bit =0), the half SPORT either three-states its data transmit pin (during the channel's transmit time slot) or ignores incoming data (during the channel's receive time slot).

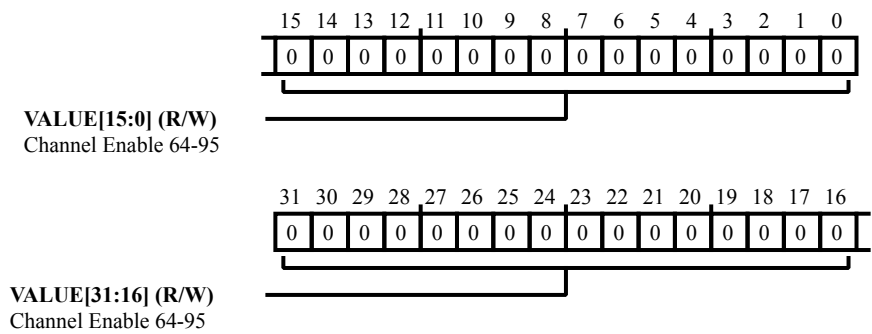


Figure 21-19: `SPORT_CS2_B` Register Diagram

Table 21-20: `SPORT_CS2_B` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Channel Enable 64-95.

Half SPORT 'A' Multichannel 96-127 Select Register

Each of the bits (when set, =1) of the `SPORT_CS3_A` register correspond to an active channel for the half SPORT in multichannel mode. When the register activates a channel (corresponding bit =1), the half SPORT transmits or receives the word in that channel's position of the data stream. When the register deactivates a channel (corresponding bit =0), the half SPORT either three-states its data transmit pin (during the channel's transmit time slot) or ignores incoming data (during the channel's receive time slot).

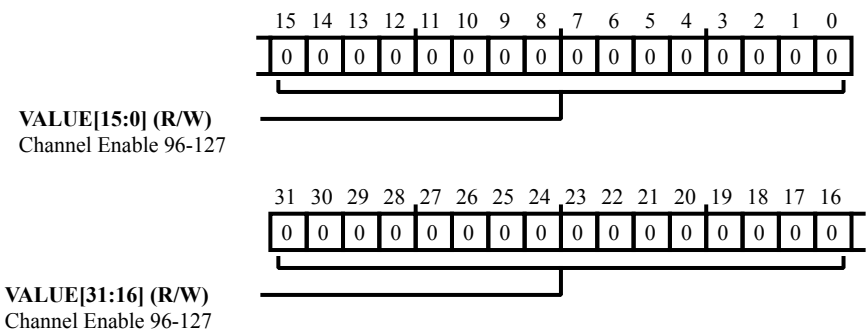


Figure 21-20: `SPORT_CS3_A` Register Diagram

Table 21-21: `SPORT_CS3_A` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Channel Enable 96-127.

Half SPORT 'B' Multichannel 96-127 Select Register

Each of the bits (when set, =1) of the `SPORT_CS3_B` register correspond to an active channel for the half SPORT in multichannel mode. When the register activates a channel (corresponding bit =1), the half SPORT transmits or receives the word in that channel's position of the data stream. When the register deactivates a channel (corresponding bit =0), the half SPORT either three-states its data transmit pin (during the channel's transmit time slot) or ignores incoming data (during the channel's receive time slot).

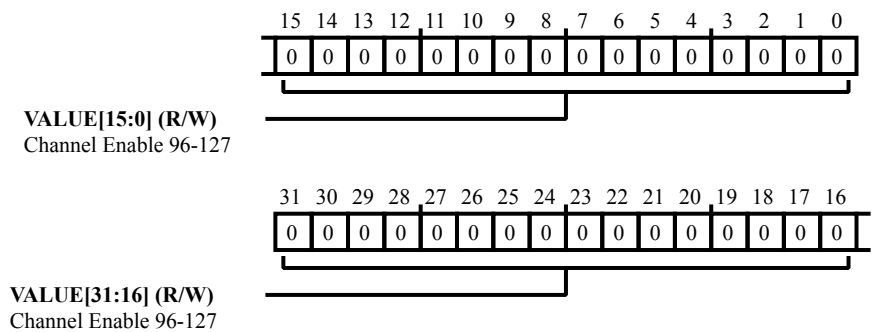


Figure 21-21: `SPORT_CS3_B` Register Diagram

Table 21-22: `SPORT_CS3_B` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Channel Enable 96-127.

Half SPORT 'A' Control 2 Register

The `SPORT_CTL2_A` register controls multiplexing options for sharing serial clock and frame sync signals across the related half SPORTs.

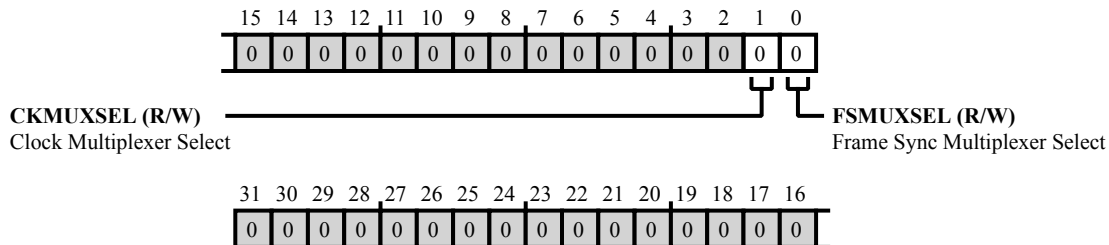


Figure 21-22: `SPORT_CTL2_A` Register Diagram

Table 21-23: `SPORT_CTL2_A` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/W)	CKMUXSEL	Clock Multiplexer Select. The <code>SPORT_CTL2_A.CKMUXSEL</code> bit enables multiplexing of the half SPORT's serial clock. In this mode, the serial clock of the related half SPORT is used instead of the half SPORT's own serial clock. For example, if the <code>SPORT_CTL2_A.CKMUXSEL</code> bit is enabled, half SPORT 'A' uses <code>SPORT_BCLK</code> instead of <code>SPORT_ACLK</code> .
		0 Disable serial clock multiplexing
		1 Enable serial clock multiplexing
0 (R/W)	FSMUXSEL	Frame Sync Multiplexer Select. The <code>SPORT_CTL2_A.FSMUXSEL</code> bit enables multiplexing of the half SPORT's frame sync. In this mode, the frame sync of the related half SPORT is used instead of the half SPORT's own frame sync. For example, if the <code>SPORT_CTL2_A.FSMUXSEL</code> bit is enabled, half SPORT 'A' uses <code>SPORT_BFS</code> instead of <code>SPORT_AFS</code> .
		0 Disable frame sync multiplexing
		1 Enable frame sync multiplexing

Half SPORT 'B' Control 2 Register

The `SPORT_CTL2_B` register controls multiplexing options for sharing serial clock and frame sync signals across the related half SPORTs.

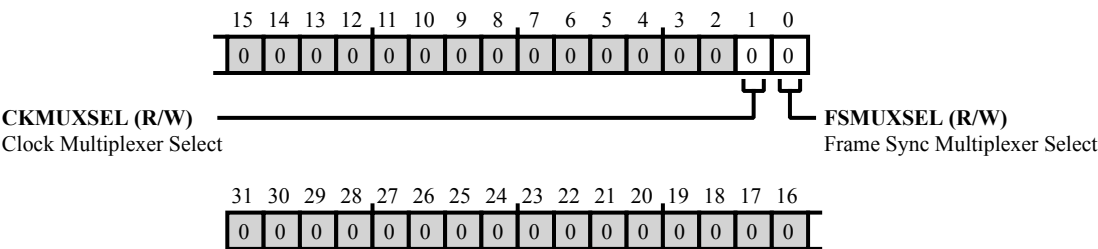


Figure 21-23: SPORT_CTL2_B Register Diagram

Table 21-24: SPORT_CTL2_B Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/W)	CKMUXSEL	Clock Multiplexer Select. The <code>SPORT_CTL2_B.CKMUXSEL</code> bit enables multiplexing of the half SPORT's serial clock. In this mode, the serial clock of the related half SPORT is used instead of the half SPORT's own serial clock. For example, if the <code>SPORT_CTL2_B.CKMUXSEL</code> bit is enabled, half SPORT 'B' uses <code>SPORT_ACLK</code> instead of <code>SPORT_BCLK</code> .
		0 Disable serial clock multiplexing
		1 Enable serial clock multiplexing
0 (R/W)	FSMUXSEL	Frame Sync Multiplexer Select. The <code>SPORT_CTL2_B.FSMUXSEL</code> bit enables multiplexing of the half SPORT's frame sync. In this mode, the frame sync of the related half SPORT is used instead of the half SPORT's own frame sync. For example, if the <code>SPORT_CTL2_B.FSMUXSEL</code> bit is enabled, half SPORT 'B' uses <code>SPORT_AFS</code> instead of <code>SPORT_BFS</code> .
		0 Disable frame sync multiplexing
		1 Enable frame sync multiplexing

Half SPORT 'A' Control Register

The `SPORT_CTL_A` register contains transmit and receive control bits for SPORT half 'A', including serial port mode selection for the half SPORT's primary and secondary channels. The function of some bits in the `SPORT_CTL_A` register vary depending on the SPORT's operating mode. For more information, see the SPORT operating modes description. If reading reserved bits, the read value is the last written value to these bits or is the reset value of these bits.

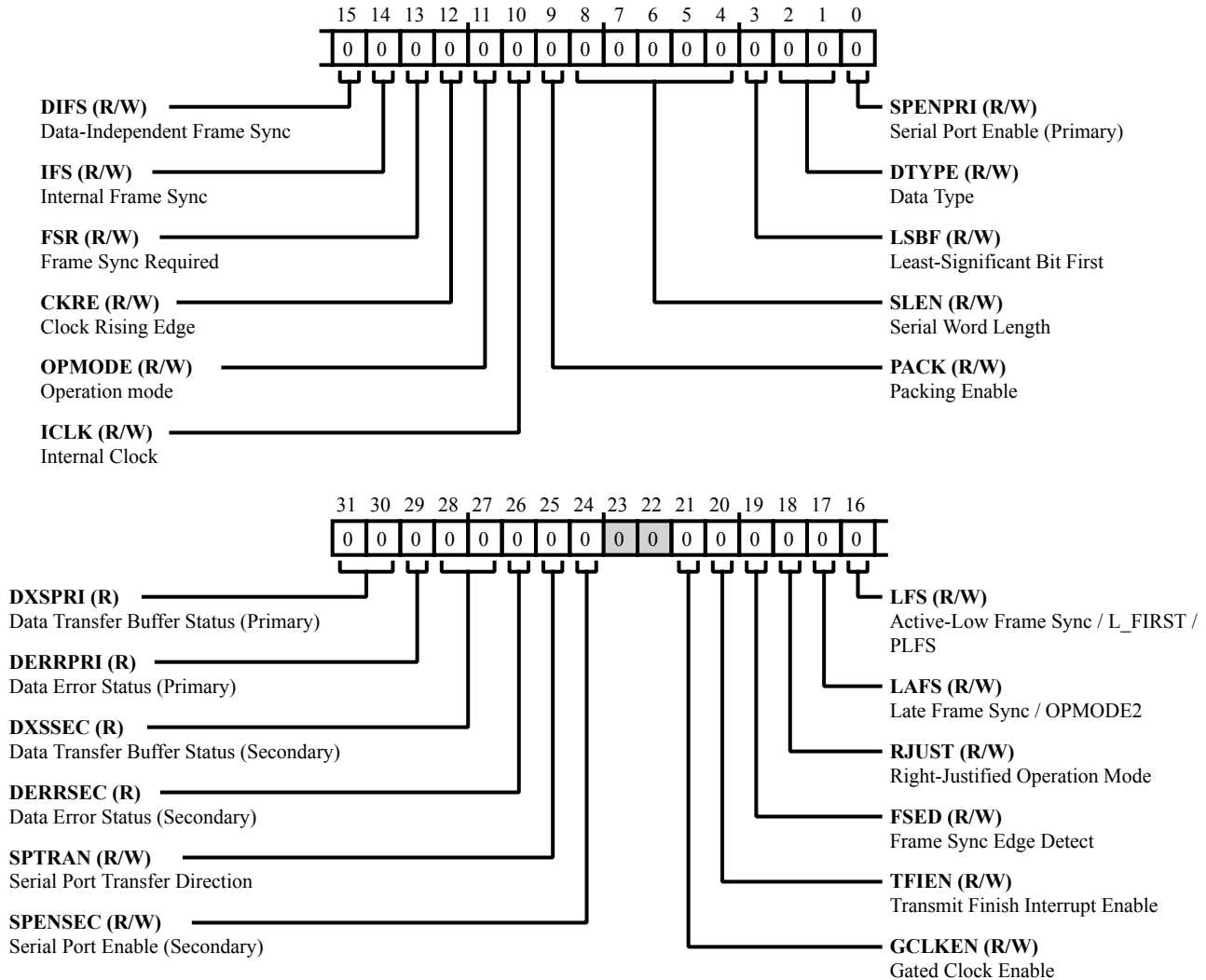


Figure 21-24: `SPORT_CTL_A` Register Diagram

Table 21-25: SPORT_CTL_A Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:30 (R/NW)	DXSPRI	Data Transfer Buffer Status (Primary). The <code>SPORT_CTL_A.DXSPRI</code> bit field indicates the status of the half SPORT's primary channel data buffer.
		0 Empty
		1 Reserved
		2 Partially full
		3 Full
29 (R/NW)	DERRPRI	Data Error Status (Primary). The <code>SPORT_CTL_A.DERRPRI</code> bit reports the half SPORT's primary channel transmit underflow status or receive overflow status, depending on the SPORT transfer direction. If the <code>SPORT_CTL_A.FSR</code> bit =1, the <code>SPORT_CTL_A.DERRPRI</code> bit indicates whether the <code>SPORT_AFS</code> signal (from an internal or external source) occurred while the <code>SPORT_TXPRI_A</code> data buffer was empty (during transmit) or the <code>SPORT_RXPRI_A</code> data buffer was full (during receive). The SPORT transmits or receives data whenever it detects the <code>SPORT_AFS</code> signal. It is important to note that, as a receiver, the <code>SPORT_CTL_A.DERRPRI</code> bit indicates when the channel has received new data while the <code>SPORT_RXPRI_A</code> receive buffer is full. This new data overwrites existing data. If the <code>SPORT_CTL_A.FSR</code> bit =0, the <code>SPORT_CTL_A.DERRPRI</code> bit is set whenever the SPORT is required to transmit while the <code>SPORT_TXPRI_A</code> transmit buffer is empty. It is also set whenever the SPORT is required to receive while the <code>SPORT_RXPRI_A</code> receive buffer is full. The SPORT clears the <code>SPORT_CTL_A.DERRPRI</code> bit if the <code>SPORT_ERR_A.DERRPSTAT</code> bit is cleared.
		0 No error
		1 Error (Tx underflow or Rx overflow)
28:27 (R/NW)	DXSSEC	Data Transfer Buffer Status (Secondary). The <code>SPORT_CTL_A.DXSSEC</code> bit field indicates the status of the half SPORT's secondary channel data buffer.
		0 Empty
		1 Reserved
		2 Partially full
		3 Full

Table 21-25: SPORT_CTL_A Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
26 (R/NW)	DERRSEC	Data Error Status (Secondary). The <code>SPORT_CTL_A.DERRSEC</code> bit reports the half SPORT's secondary channel transmit underflow status or receive overflow status, depending on the SPORT transfer direction. If the <code>SPORT_CTL_A.FSR</code> bit =1, the <code>SPORT_CTL_A.DERRSEC</code> bit indicates whether the <code>SPORT_AFS</code> signal (from an internal or external source) occurred while the <code>SPORT_TXSEC_A</code> data buffer was empty (during transmit) or the <code>SPORT_RXSEC_A</code> data buffer was full (during receive). The SPORT transmits or receives data whenever it detects the <code>SPORT_AFS</code> signal. It is important to note that, as a receiver, the <code>SPORT_CTL_A.DERRSEC</code> bit indicates when the channel has received new data while the <code>SPORT_RXSEC_A</code> receive buffer is full. This new data overwrites existing data. If the <code>SPORT_CTL_A.FSR</code> bit =0, the <code>SPORT_CTL_A.DERRSEC</code> bit is set whenever the SPORT is required to transmit while the <code>SPORT_TXSEC_A</code> transmit buffer is empty. It is also set whenever the SPORT is required to receive while the <code>SPORT_RXSEC_A</code> receive buffer is full. The SPORT clears the <code>SPORT_CTL_A.DERRSEC</code> bit if the <code>SPORT_ERR_A.DERRSSTAT</code> bit is cleared.
		0 No error
		1 Error (Tx underflow or Rx overflow)
25 (R/W)	SPTRAN	Serial Port Transfer Direction. The <code>SPORT_CTL_A.SPTRAN</code> bit selects the transfer direction (receive or transmit) for the half SPORT's primary and secondary channels. When the direction is receive, the half SPORT activates the receive buffers, and the <code>SPORT_ACLK</code> and <code>SPORT_AFS</code> pins control the receive buffers. The transmit buffers are inactive when the half SPORT's transfer direction is receive. When the direction is transmit, the half SPORT activates the transmit buffers, and the <code>SPORT_ACLK</code> and <code>SPORT_AFS</code> pins control the transmit shift registers. The receive buffers are inactive when the half SPORT's transfer direction is transmit.
		0 Receive
		1 Transmit
24 (R/W)	SPENSEC	Serial Port Enable (Secondary). The <code>SPORT_CTL_A.SPENSEC</code> bit enables the half SPORT's secondary channel. When this bit is cleared (changes from =1 to =0), the half SPORT automatically flushes the channel's data buffers.
		0 Disable
		1 Enable

Table 21-25: SPORT_CTL_A Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
21 (R/W)	GCLKEN	Gated Clock Enable. The <code>SPORT_CTL_A.GCLKEN</code> bit enables gated clock operation for the half SPORT when in DSP serial mode or left-justified stereo modes (<code>SPORT_CTL_A.OPMODE = 0</code> or <code>1</code>). This bit is ignored when the half SPORT is in right-justified mode (<code>SPORT_CTL_A.RJUST = 1</code>) or multichannel mode (<code>SPORT_MCTL_A.MCE = 1</code>). When the <code>SPORT_CTL_A.GCLKEN</code> bit is enabled, the SPORT clock is active when the SPORT is transferring data or when the frame sync changes (transitions to active state).
		0 Disable
		1 Enable
20 (R/W)	TFIEN	Transmit Finish Interrupt Enable. The <code>SPORT_CTL_A.TFIEN</code> bit selects when the half SPORT issues its transmission complete interrupt request, if a DMA complete interrupt request is enabled by the <code>DMA_CFG.INT</code> configuration. When enabled (<code>SPORT_CTL_A.TFIEN = 1</code>), the DMA complete peripheral interrupt request is generated when the last bit of last word in the DMA is shifted out. When disabled (<code>SPORT_CTL_A.TFIEN = 0</code>), the DMA interrupt request is generated when the DMA counter expires (the last word goes to the transmit buffer).
		0 Last word sent (DMA count done) interrupt
		1 Last bit sent (Tx buffer done) interrupt
19 (R/W)	FSED	Frame Sync Edge Detect. The <code>SPORT_CTL_A.FSED</code> bit enables the half SPORT to start transmitting or receiving after detecting an active edge of an external frame sync. The <code>SPORT_CTL_A.FSED</code> bit may be enabled even during an active frame sync, and the half SPORT starts the transfer on the next valid rising or falling edge of external frame sync. If disabled (<code>SPORT_CTL_A.FSED = 0</code>), the half SPORT operates in the standard level-sensitive detection mode for external frame sync.
		0 Level detect frame sync
		1 Edge detect frame sync

Table 21-25: SPORT_CTL_A Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
18 (R/W)	RJUST	Right-Justified Operation Mode. The SPORT_CTL_A.RJUST bit enables the half SPORT (if SPORT_CTL_A.OPMODE =1) to transfer data in right-justified operation mode. In this mode, the half SPORT aligns data to the end of the frame sync, rather than the start of the frame sync. When using right-justified mode, systems should program an appropriate delay count to introduce a clock delay before the half SPORT state machine starts to capture data. This value is set in the DCNT field (right-justified mode usage of the SPORT_MCTL_A.WOFFSET field). For information about appropriate delay selections, see the SPORT operating modes section.
		0 Disable
		1 Enable
17 (R/W)	LAFS	Late Frame Sync / OPMODE2. When the half SPORT is in DSP standard mode (SPORT_CTL_A.OPMODE =0) or in right-justified mode (SPORT_CTL_A.RJUST =1), the SPORT_CTL_A.LAFS bit selects whether the half SPORT generates a late frame sync (SPORT_AFS during first data bit) or generates an early frame sync signal (SPORT_AFS during serial clock cycle before first data bit). When the half SPORT is in I ² S / left-justified mode (SPORT_CTL_A.OPMODE =1), the SPORT_CTL_A.LAFS bit acts as OP-MODE2, selecting whether the half SPORT is in left-justified mode or I ² S mode. When the half SPORT is in multichannel mode (SPORT_MCTL_A.MCE =1), the SPORT_CTL_A.LAFS bit is reserved.
		0 Early frame sync (or I ² S mode)
		1 Late frame sync (or left-justified mode)
16 (R/W)	LFS	Active-Low Frame Sync / L_FIRST / PLFS. When the half SPORT is in DSP standard mode and multichannel mode (SPORT_CTL_A.OPMODE =0), the SPORT_CTL_A.LFS bit selects whether the half SPORT uses active low or active high frame sync. When the half SPORT is in I ² S / packed / left-justified mode (SPORT_CTL_A.OPMODE =1), the SPORT_CTL_A.LFS bit acts as L_FIRST, selecting whether the half SPORT transfers data first for the left or right channel.
		0 Active high frame sync (DSP standard mode) or rising edge frame sync (multichannel mode) or right channel first (I ² S/packed mode) or left channel first (left-justified mode)
		1 Active low frame sync (DSP standard mode) or falling edge frame sync (multichannel mode) or left channel first (I ² S/packed mode) or right channel first (left-justified mode)

Table 21-25: SPORT_CTL_A Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W)	DIFS	Data-Independent Frame Sync. The <code>SPORT_CTL_A.DIFS</code> bit selects whether the half SPORT uses a data-independent or data-dependent frame sync. When using a data-independent frame sync, the half SPORT generates the sync at the interval selected by the <code>SPORT_DIV_A.FSDIV</code> bit. When using a data-dependent frame sync, the half SPORT generates the sync on the selected interval when the transmit buffer is not empty or when the receive buffer is not full. Note that the <code>SPORT_CTL_A.DIFS</code> bit is automatically set when the half SPORT is in packed or multichannel modes.
		0 Data-dependent frame sync
		1 Data-independent frame sync
14 (R/W)	IFS	Internal Frame Sync. The <code>SPORT_CTL_A.IFS</code> bit selects whether the half SPORT uses an internal frame sync or uses an external frame sync. Note that the externally-generated frame sync does not need to be synchronous with the processor's system clock.
		0 External frame sync
		1 Internal frame sync
13 (R/W)	FSR	Frame Sync Required. The <code>SPORT_CTL_A.FSR</code> bit selects whether or not the half SPORT requires frame sync for data transfer. This bit is automatically set when the half SPORT is in I ² S / packed / left-justified mode (<code>SPORT_CTL_A.OPMODE</code> = 1) or is in multichannel mode (<code>SPORT_MCTL_A.MCE</code> = 1).
		0 No frame sync required
		1 Frame sync required
12 (R/W)	CKRE	Clock Rising Edge. The <code>SPORT_CTL_A.CKRE</code> bit selects the rising or falling edge of the <code>SPORT_ACLK</code> clock for the half SPORT to sample receive data and frame sync. Note that the half SPORT changes the state of transmit data and frame sync signals on the non-selected edge of the <code>SPORT_ACLK</code> . Also, note that the transmit and receive related SPORT halves (A and B) should be programmed with the same value for the <code>SPORT_CTL_A.CKRE</code> bit. This programming drives the internally-generated signals on one edge of <code>SPORT_ACLK</code> and samples the received signals on the opposite edge.
		0 Clock falling edge
		1 Clock rising edge

Table 21-25: SPORT_CTL_A Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
11 (R/W)	OPMODE	Operation mode. The SPORT_CTL_A.OPMODE bit selects whether the half SPORT operates in DSP standard/multichannel mode or operates in I ² S/packed/left-justified mode. The mode selection affects the operation of the SPORT_CTL_A.LAFS and SPORT_CTL_A.LFS bits. Also, the SPORT_CTL_A.OPMODE bit enables or disables operation of the SPORT_CTL_A.GCLKEN, SPORT_CTL_A.FSED, SPORT_CTL_A.RJUST, SPORT_CTL_A.DIFS, SPORT_CTL_A.FSR, and SPORT_CTL_A.CKRE bits.
		0 DSP standard/multichannel mode
		1 I ² S/packed/left-justified mode
10 (R/W)	ICLK	Internal Clock. When the half SPORT is in DSP standard mode (SPORT_CTL_A.OPMODE =0), the SPORT_CTL_A.ICLK bit selects whether the half SPORT uses an internal or external clock. For internal clock enabled, the half SPORT generates the SPORT_ACLK clock signal, and SPORT_ACLK is an output. The SPORT_DIV_A.CLKDIV serial clock divisor value determines the clock frequency. For internal clock disabled, the SPORT_ACLK clock signal is an input, and the serial clock divisor is ignored. Note that the externally-generated serial clock does not need to be synchronous with the processor's system clock.
		0 External clock
		1 Internal clock
9 (R/W)	PACK	Packing Enable. The SPORT_CTL_A.PACK bit enables the half SPORT to perform 16- to 32-bit packing on received data and to perform 32- to 16-bit unpacking on transmitted data. The receive packing operation packs two successive received words into a single 32-bit word. The transmit unpacking operation unpacks each 32-bit word and transmits it as two 16-bit words. The first 16-bit (or smaller) word is right-justified in bits 15:0 of the packed word, and the second 16-bit (or smaller) word is right-justified in bits 31:16. This format applies to both receive (packing) and transmit (unpacking) operations. Companding may be used with word packing or unpacking. The half SPORT generates data transfer related interrupts when packing is enabled. The transmit and receive interrupts are generated for the 32-bit packed words, not for each 16-bit word.
		0 Disable
		1 Enable

Table 21-25: SPORT_CTL_A Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
8:4 (R/W)	SLEN	Serial Word Length. The SPORT_CTL_A.SLEN bits selects word length in bits for the half SPORT's data transfers. Word may be from 4- to 32-bits in length. The formula for selecting the word length in bits is: SPORT_CTL_A.SLEN = (serial word length in bits) - 1 For DSP standard mode (SPORT_CTL_A.OPMODE =0), use SPORT_CTL_A.SLEN of 3 to 31 bits. For I ² S / packed / left-justified mode (SPORT_CTL_A.OPMODE =1), use SPORT_CTL_A.SLEN of 4 to 31 bits.	
3 (R/W)	LSBF	Least-Significant Bit First. The SPORT_CTL_A.LSBF bit selects whether the half SPORT transmits or receives data LSB first or MSB first.	
		0	MSB first sent/received (big endian)
		1	LSB first sent/received (little endian)
2:1 (R/W)	DTYPE	Data Type. The SPORT_CTL_A.DTYPE bits selects the data type formatting for the half SPORT's data transfers in DSP standard mode (SPORT_CTL_A.OPMODE =0).	
		0	Right-justify data, zero-fill unused MSBs
		1	Right-justify data, sign-extend unused MSBs
		2	u-law compand data
		3	A-law compand data
0 (R/W)	SPENPRI	Serial Port Enable (Primary). The SPORT_CTL_A.SPENPRI bit enables the half SPORT's primary channel. When this bit is cleared (changes from =1 to =0), the half SPORT automatically flushes the channel's data buffers.	
		0	Disable
		1	Enable

Half SPORT 'B' Control Register

The `SPORT_CTL_B` register contains transmit and receive control bits for SPORT half 'B', including serial port mode selection for the half SPORT's primary and secondary channels. The function of some bits in the `SPORT_CTL_B` register vary, depending on the SPORT's operating mode. For more information, see the SPORT operating modes description. If reading reserved bits, the read value is the last written value to these bits or is the reset value of these bits.

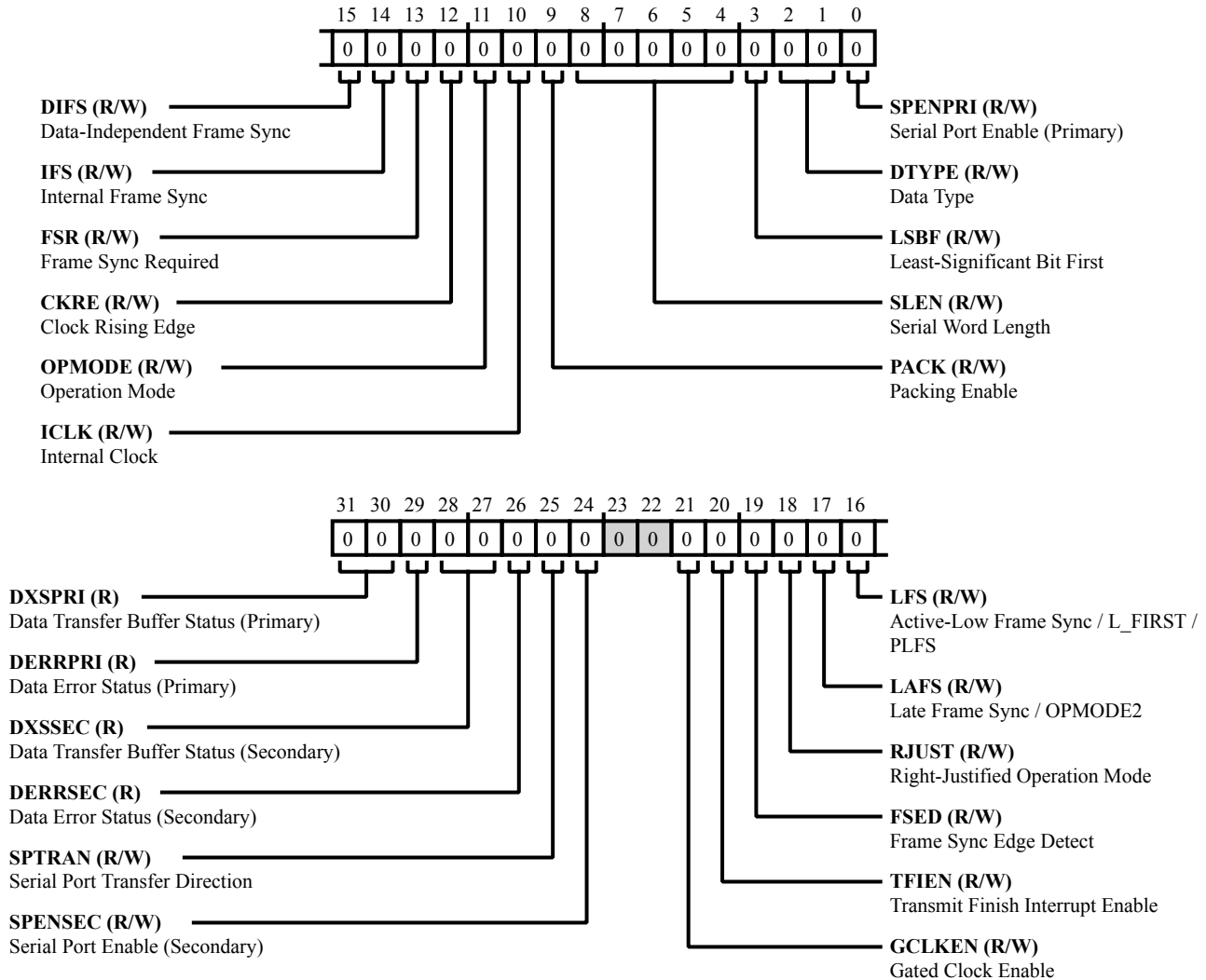


Figure 21-25: `SPORT_CTL_B` Register Diagram

Table 21-26: SPORT_CTL_B Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:30 (R/NW)	DXSPRI	Data Transfer Buffer Status (Primary). The <code>SPORT_CTL_B.DXSPRI</code> bit field indicates the status of the half SPORT's primary channel data buffer.
		0 Empty
		1 Reserved
		2 Partially full
		3 Full
29 (R/NW)	DERRPRI	Data Error Status (Primary). The <code>SPORT_CTL_B.DERRPRI</code> bit reports the half SPORT's primary channel transmit underflow status or receive overflow status, depending on the SPORT transfer direction. If the <code>SPORT_CTL_B.FSR</code> bit =1, the <code>SPORT_CTL_B.DERRPRI</code> bit indicates whether the <code>SPORT_BFS</code> signal (from an internal or external source) occurred while the <code>SPORT_TXPRI_B</code> data buffer was empty (during transmit) or the <code>SPORT_RXPRI_B</code> data buffer was full (during receive). The SPORT transmits or receives data whenever it detects the <code>SPORT_BFS</code> signal. It is important to note that, as a receiver, the <code>SPORT_CTL_B.DERRPRI</code> bit indicates when the channel has received new data while the <code>SPORT_RXPRI_B</code> receive buffer is full. This new data overwrites existing data. If the <code>SPORT_CTL_B.FSR</code> bit =0, the <code>SPORT_CTL_B.DERRPRI</code> bit is set whenever the SPORT is required to transmit while the <code>SPORT_TXPRI_B</code> transmit buffer is empty and is set whenever the SPORT is required to receive while the <code>SPORT_RXPRI_B</code> receive buffer is full. The SPORT clears the <code>SPORT_CTL_B.DERRPRI</code> bit if the <code>SPORT_ERR_B.DERRPSTAT</code> bit is cleared.
		0 No error
		1 Error (Tx underflow or Rx overflow)
28:27 (R/NW)	DXSSEC	Data Transfer Buffer Status (Secondary). The <code>SPORT_CTL_B.DXSSEC</code> bit field indicates the status of the half SPORT's secondary channel data buffer.
		0 Empty
		1 Reserved
		2 Partially full
		3 Full

Table 21-26: SPORT_CTL_B Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
26 (R/NW)	DERRSEC	Data Error Status (Secondary). The <code>SPORT_CTL_B.DERRSEC</code> bit reports the half SPORT's secondary channel transmit underflow status or receive overflow status, depending on the SPORT transfer direction. If the <code>SPORT_CTL_B.FSR</code> bit =1, the <code>SPORT_CTL_B.DERRSEC</code> bit indicates whether the <code>SPORT_BFS</code> signal (from an internal or external source) occurred while the <code>SPORT_TXSEC_B</code> data buffer was empty (during transmit) or the <code>SPORT_RXSEC_B</code> data buffer was full (during receive). The SPORT transmits or receives data whenever it detects the <code>SPORT_BFS</code> signal. It is important to note that, as a receiver, the <code>SPORT_CTL_B.DERRSEC</code> bit indicates when the channel has received new data while the <code>SPORT_RXSEC_B</code> receive buffer is full. This new data overwrites existing data. If the <code>SPORT_CTL_B.FSR</code> bit =0, the <code>SPORT_CTL_B.DERRSEC</code> bit is set whenever the SPORT is required to transmit while the <code>SPORT_TXSEC_B</code> transmit buffer is empty. It is also set whenever the SPORT is required to receive while the <code>SPORT_RXSEC_B</code> receive buffer is full. The SPORT clears the <code>SPORT_CTL_B.DERRSEC</code> bit if the <code>SPORT_ERR_B.DERRSSTAT</code> bit is cleared.
		0 No error
		1 Error (Tx underflow or Rx overflow)
25 (R/W)	SPTRAN	Serial Port Transfer Direction. The <code>SPORT_CTL_B.SPTRAN</code> bit selects the transfer direction (receive or transmit) for the half SPORT's primary and secondary channels. When the direction is receive, the half SPORT activates the receive buffers, and the <code>SPORT_BCLK</code> and <code>SPORT_BFS</code> pins control the receive buffers. The transmit buffers are inactive when the half SPORT's transfer direction is receive. When the direction is transmit, the half SPORT activates the transmit buffers, and the <code>SPORT_BCLK</code> and <code>SPORT_BFS</code> pins control the transmit shift registers. The receive buffers are inactive when the half SPORT's transfer direction is transmit.
		0 Receive
		1 Transmit
24 (R/W)	SPENSEC	Serial Port Enable (Secondary). The <code>SPORT_CTL_B.SPENSEC</code> bit enables the half SPORT's secondary channel. When this bit is cleared (changes from =1 to =0), the half SPORT automatically flushes the channel's data buffers.
		0 Disable
		1 Enable

Table 21-26: SPORT_CTL_B Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
21 (R/W)	GCLKEN	Gated Clock Enable. The <code>SPORT_CTL_B.GCLKEN</code> bit enables gated clock operation for the half SPORT when in DSP serial mode or left-justified stereo modes (<code>SPORT_CTL_B.OPMODE = 0</code> or <code>1</code>). This bit is ignored when the half SPORT is in right-justified mode (<code>SPORT_CTL_B.RJUST = 1</code>) or multichannel mode (<code>SPORT_MCTL_B.MCE = 1</code>). When <code>SPORT_CTL_B.GCLKEN</code> is enabled, the SPORT clock is active when the SPORT is transferring data or when the frame sync changes (transitions to active state).
		0 Disable
		1 Enable
20 (R/W)	TFIEN	Transmit Finish Interrupt Enable. The <code>SPORT_CTL_B.TFIEN</code> bit selects when the half SPORT issues its transmission complete interrupt request, if a DMA complete interrupt request is enabled by the <code>DMA_CFG.INT</code> configuration. When enabled (<code>SPORT_CTL_B.TFIEN = 1</code>), the DMA complete peripheral interrupt request is generated when the last bit of last word in the DMA is shifted out. When disabled (<code>SPORT_CTL_B.TFIEN = 0</code>), the DMA interrupt request is generated when the DMA counter expires (the last word goes to the transmit buffer).
		0 Last word sent (DMA count done) interrupt
		1 Last bit sent (Tx buffer done) interrupt
19 (R/W)	FSED	Frame Sync Edge Detect. The <code>SPORT_CTL_B.FSED</code> bit enables the half SPORT to start transmitting or receiving after detecting an active edge of an external frame sync. The <code>SPORT_CTL_B.FSED</code> may be enabled even during an active frame sync, and the half SPORT starts the transfer on the next valid rising or falling edge of external frame sync. If disabled (<code>SPORT_CTL_B.FSED = 0</code>), the half SPORT operates in the standard level-sensitive detection mode for external frame sync.
		0 Level detect frame sync
		1 Edge detect frame sync

Table 21-26: SPORT_CTL_B Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
18 (R/W)	RJUST	Right-Justified Operation Mode. The SPORT_CTL_B.RJUST bit enables the half SPORT (if SPORT_CTL_B.OPMODE =1) to transfer data in right-justified operation mode. In this mode, the half SPORT aligns data to the end of the frame sync, rather than the start of the frame sync. When using right-justified mode, systems should program an appropriate delay count to introduce a clock delay before the half SPORT state machine starts to capture data. This value is set in the DCNT field (right-justified mode usage of the SPORT_MCTL_B.WOFFSET field). For information about appropriate delay selections, see the SPORT operating modes section.
		0 Disable
		1 Enable
17 (R/W)	LAFS	Late Frame Sync / OPMODE2. When the half SPORT is in DSP standard mode (SPORT_CTL_B.OPMODE =0) or in right-justified mode (SPORT_CTL_B.RJUST =1), the SPORT_CTL_B.LAFS bit selects whether the half SPORT generates a late frame sync (SPORT_BFS during first data bit) or generates an early frame sync signal (SPORT_BFS during serial clock cycle before first data bit). When the half SPORT is in I ² S / left-justified mode (SPORT_CTL_B.OPMODE =1), the SPORT_CTL_B.LAFS bit acts as OPMODE2, selecting whether the half SPORT is in left-justified mode or I ² S mode. When the half SPORT is in multichannel mode (SPORT_MCTL_B.MCE =1), the SPORT_CTL_B.LAFS bit is reserved.
		0 Early frame sync (or I ² S mode)
		1 Late frame sync (or left-justified mode)

Table 21-26: SPORT_CTL_B Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
16 (R/W)	LFS	Active-Low Frame Sync / L_FIRST / PLFS. When the half SPORT is in DSP standard mode and multichannel mode (SPORT_CTL_B.OPMODE = 0), the SPORT_CTL_B.LFS bit selects whether the half SPORT uses active low or active high frame sync. When the half SPORT is in I ² S / packed / left-justified mode (SPORT_CTL_B.OPMODE = 1), the SPORT_CTL_B.LFS bit acts as L_FIRST, selecting whether the half SPORT transfers data first for the left or right channel.
		0 Active high frame sync (DSP standard mode) or rising edge frame sync (multichannel mode) or right channel first (I ² S/packed mode) or left channel first (left-justified mode)
		1 Active low frame sync (DSP standard mode) or falling edge frame sync (multichannel mode) or left channel first (I ² S/packed mode) or right channel first (left-justified mode)
15 (R/W)	DIFS	Data-Independent Frame Sync. The SPORT_CTL_B.DIFS bit selects whether the half SPORT uses a data-independent or data-dependent frame sync. When using a data-independent frame sync, the half SPORT generates the sync at the interval selected by SPORT_DIV_B.FSDIV. When using a data-dependent frame sync, the half SPORT generates the sync on the selected interval when the transmit buffer is not empty or when the receive buffer is not full. Note that the SPORT_CTL_B.DIFS bit is automatically set when the half SPORT is in packed or multichannel modes.
		0 Data-dependent frame sync
		1 Data-independent frame sync
14 (R/W)	IFS	Internal Frame Sync. The SPORT_CTL_B.IFS bit selects whether the half SPORT uses an internal frame sync or uses an external frame sync. Note that the externally-generated frame sync does not need to be synchronous with the processor's system clock.
		0 External frame sync
		1 Internal frame sync

Table 21-26: SPORT_CTL_B Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
13 (R/W)	FSR	Frame Sync Required. The SPORT_CTL_B.FSR selects whether or not the half SPORT requires frame sync for data transfer. This bit is automatically set when the half SPORT is in I ² S / packed / left-justified mode (SPORT_CTL_B.OPMODE =1) or is in multichannel mode (SPORT_MCTL_B.MCE =1).
		0 No frame sync required
		1 Frame sync required
12 (R/W)	CKRE	Clock Rising Edge. The SPORT_CTL_B.CKRE selects the rising or falling edge of the SPORT_BCLK clock for the half SPORT to sample receive data and frame sync. Note that the half SPORT changes the state of transmit data and frame sync signals on the non-selected edge of the SPORT_BCLK. Also note that the transmit and receive related SPORT halves (A and B) should be programmed with the same value for SPORT_CTL_B.CKRE. This programming drives the internally-generated signals on one edge of SPORT_BCLK and samples the received signals on the opposite edge.
		0 Clock falling edge
		1 Clock rising edge
11 (R/W)	OPMODE	Operation Mode. The SPORT_CTL_B.OPMODE bit selects whether the half SPORT operates in DSP standard / multichannel mode or operates in I ² S / packed / left-justified mode. The mode selection affects the operation of the SPORT_CTL_B.LAFS and SPORT_CTL_B.LFS bits. Also, the SPORT_CTL_B.OPMODE bit enables or disables operation of the SPORT_CTL_B.GCLKEN, SPORT_CTL_B.FSED, SPORT_CTL_B.RJUST, SPORT_CTL_B.DIFS, SPORT_CTL_B.FSR, and SPORT_CTL_B.CKRE bits.
		0 DSP standard/multichannel mode
		1 I ² S/packed/left-justified mode
10 (R/W)	ICLK	Internal Clock. When the half SPORT is in DSP standard mode (SPORT_CTL_B.OPMODE =0), the SPORT_CTL_B.ICLK bit selects whether the half SPORT uses an internal or external clock. For internal clock enabled, the half SPORT generates the SPORT_BCLK clock signal, and the SPORT_BCLK is an output. The SPORT_DIV_B.CLKDIV serial clock divisor value determines the clock frequency. For internal clock disabled, the SPORT_BCLK clock signal is an input, and the serial clock divisor is ignored. Note that the externally-generated serial clock does not need to be synchronous with the processor's system clock.
		0 External clock
		1 Internal clock

Table 21-26: SPORT_CTL_B Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
9 (R/W)	PACK	Packing Enable. The SPORT_CTL_B.PACK bit enables the half SPORT to perform 16- to 32-bit packing on received data and to perform 32- to 16-bit unpacking on transmitted data. The receive packing operation packs two successive received words into a single 32-bit word. The transmit unpacking operation unpacks each 32-bit word and transmits it as two 16-bit words. The first 16-bit (or smaller) word is right-justified in bits 15:0 of the packed word, and the second 16-bit (or smaller) word is right-justified in bits 31:16. This format applies to both receive (packing) and transmit (unpacking) operations. Companding may be used with word packing or unpacking. The half SPORT generates data transfer related interrupts when packing is enabled. The transmit and receive interrupts are generated for the 32-bit packed words, not for each 16-bit word.
		0 Disable
		1 Enable
8:4 (R/W)	SLEN	Serial Word Length. The SPORT_CTL_B.SLEN bits selects word length in bits for the half SPORT's data transfers. Word may be from 4- to 32-bits in length. The formula for selecting the word length in bits is: $\text{SPORT_CTL_B.SLEN} = (\text{serial word length in bits}) - 1$ For DSP standard mode (SPORT_CTL_B.OPMODE =0), use SPORT_CTL_B.SLEN of 3 to 31 bits. For I ² S / packed / left-justified mode (SPORT_CTL_B.OPMODE =1), use SPORT_CTL_B.SLEN of 4 to 31 bits.
3 (R/W)	LSBF	Least-Significant Bit First. The SPORT_CTL_B.LSBF bit selects whether the half SPORT transmits or receives data LSB first or MSB first.
		0 MSB first sent/received (big endian)
		1 LSB first sent/received (little endian)
2:1 (R/W)	DTYPE	Data Type. The SPORT_CTL_B.DTYPE bits selects the data type formatting for the half SPORT's data transfers in DSP standard mode (SPORT_CTL_B.OPMODE =0).
		0 Right-justify data, zero-fill unused MSBs
		1 Right-justify data, sign-extend unused MSBs
		2 u-law compand data
		3 A-law compand data

Table 21-26: SPORT_CTL_B Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	SPENPRI	Serial Port Enable (Primary). The SPORT_CTL_B.SPENPRI bit enables the half SPORT's primary channel. When this bit is cleared (changes from =1 to =0), the half SPORT automatically flushes the channel's data buffers.
		0 Disable
		1 Enable

Half SPORT 'A' Divisor Register

The `SPORT_DIV_A` register contains divisor values that determine frequencies of internally-generated clocks and frame syncs for half SPORT 'A'.

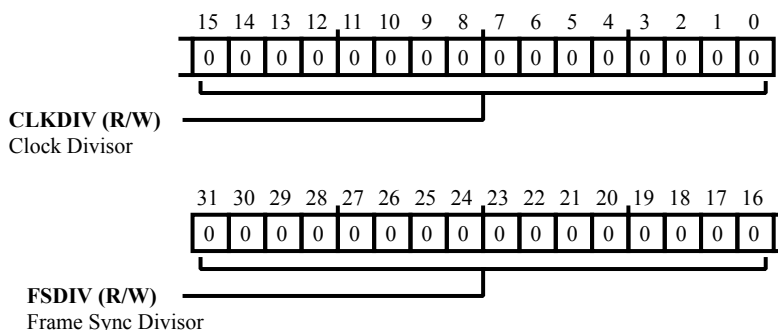


Figure 21-26: `SPORT_DIV_A` Register Diagram

Table 21-27: `SPORT_DIV_A` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:16 (R/W)	FSDIV	Frame Sync Divisor. The <code>SPORT_DIV_A.FSDIV</code> bits select the number of transmit or receive clock cycles that the half SPORT counts before generating a frame sync pulse. The half SPORT counts serial clock cycles whether these are from an internally- or an externally-generated serial clock. The formula relating <code>SPORT_DIV_A.FSDIV</code> to the number of cycles between frame sync pulses is: $\text{SPORT_DIV_A.FSDIV} = (\text{number of serial clocks between frame syncs}) - 1$ Use the following equation to determine the value of <code>SPORT_DIV_A.FSDIV</code>, given the serial clock frequency and desired frame sync frequency: $\text{FSDIV} = (\text{SPORT_ACLK} / \text{SPORT_AFS}) - 1$ Note that the frame sync is continuously active when <code>SPORT_DIV_A.FSDIV = 0</code>. The value of <code>SPORT_DIV_A.FSDIV</code> should not be less than the serial word length (<code>SPORT_CTL_A.SLEN</code>), as this may cause an external device to abort the current operation or cause other unpredictable results.
15:0 (R/W)	CLKDIV	Clock Divisor. The <code>SPORT_DIV_A.CLKDIV</code> bits select the divisor that the half SPORT uses to calculate the serial clock (<code>SPORT_ACLK</code>) from the processor system clock (<code>SCLK</code>). The divisor is a 16-bit value, allowing a wide range of serial clock rates. When configured for internal clock (<code>SPORT_CTL_A.ICLK = 1</code>), legal <code>SPORT_DIV_A.CLKDIV</code> values are 0 to 65535. Given the processor system clock frequency and desired serial clock frequency, use the following formula to calculate the value of <code>SPORT_DIV_A.CLKDIV</code>: $\text{CLKDIV} = (\text{SCLK} / \text{SPORT_ACLK}) - 1$ For the maximum serial clock frequency, see the processor data sheet.

Half SPORT 'B' Divisor Register

The `SPORT_DIV_B` contains divisor values that determine frequencies of internally-generated clocks and frame syncs for SPORT half 'B'.

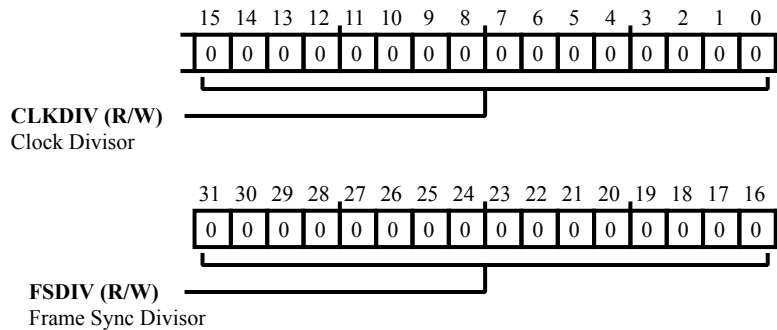


Figure 21-27: SPORT_DIV_B Register Diagram

Table 21-28: SPORT_DIV_B Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:16 (R/W)	FSDIV	Frame Sync Divisor. The <code>SPORT_DIV_B.FSDIV</code> bits select the number of transmit or receive clock cycles that the half SPORT counts before generating a frame sync pulse. The half SPORT counts serial clock cycles whether these are from an internally- or an externally-generated serial clock. The formula relating <code>SPORT_DIV_B.FSDIV</code> to the number of cycles between frame sync pulses is: $\text{SPORT_DIV_B.FSDIV} = (\text{number of serial clocks between frame syncs}) - 1$ Use the following equation to determine the value of <code>SPORT_DIV_B.FSDIV</code>, given the serial clock frequency and desired frame sync frequency: $\text{FSDIV} = (\text{SPORT_BCLK} / \text{SPORT_BFS}) - 1$ Note that the frame sync is continuously active when <code>SPORT_DIV_B.FSDIV = 0</code>. The value of <code>SPORT_DIV_B.FSDIV</code> should not be less than the serial word length (<code>SPORT_CTL_B.SLEN</code>), as this may cause an external device to abort the current operation or cause other unpredictable results.
15:0 (R/W)	CLKDIV	Clock Divisor. The <code>SPORT_DIV_B.CLKDIV</code> bits select the divisor that the half SPORT uses to calculate the serial clock (<code>SPORT_BCLK</code>) from the processor system clock (<code>SCLK</code>). The divisor is a 16-bit value, allowing a wide range of serial clock rates. When configured for internal clock (<code>SPORT_CTL_B.ICLK = 1</code>), legal <code>SPORT_DIV_B.CLKDIV</code> values are 0 to 65535. Given the processor system clock frequency and desired serial clock frequency, use the following formula to calculate the value of <code>SPORT_DIV_B.CLKDIV</code>: $\text{CLKDIV} = (\text{SCLK} / \text{SPORT_BCLK}) - 1$ For the maximum serial clock frequency, see the processor data sheet.

Half SPORT 'A' Error Register

The `SPORT_ERR_A` register contains error status and error interrupt mask bits for SPORT half 'A', including error handling bits for the half SPORT's primary and secondary channels and frame sync. Detected errors are frame sync violations or buffer over/underflow conditions.

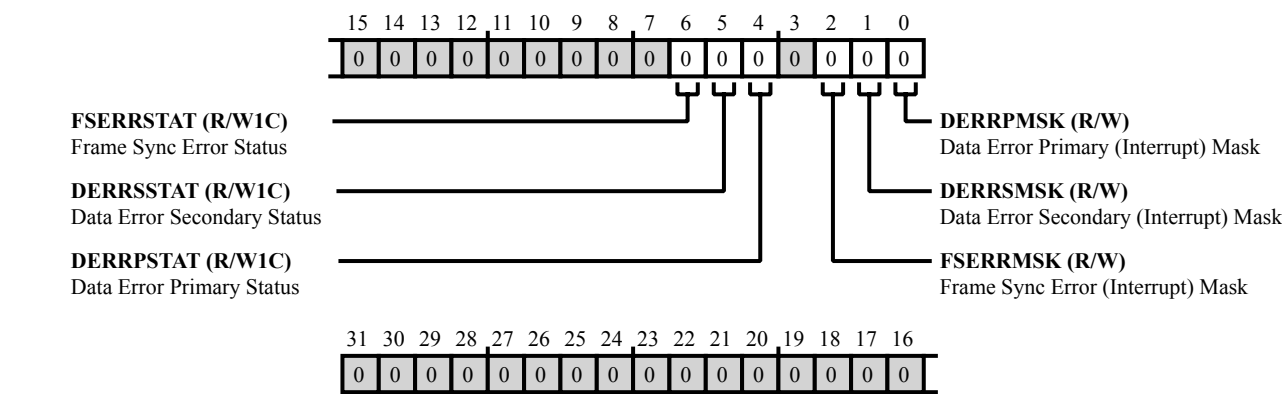


Figure 21-28: `SPORT_ERR_A` Register Diagram

Table 21-29: `SPORT_ERR_A` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
6 (R/W1C)	FSERRSTAT	Frame Sync Error Status. The <code>SPORT_ERR_A.FSERRSTAT</code> bit indicates that the half SPORT has detected a frame sync when the bit count (bits remaining in the frame) is non-zero. When a half SPORT is receiving or transmitting, its bit count is set to a word length (for example, <code>SPORT_CTL_A.SLEN = 31</code>). After each serial clock edge, the half SPORT decrements the transfer's bit count. After the word is received or transmitted, the transfer's bit count reaches zero, and the half SPORT resets it (for example, to 32) on next frame sync. Normal SPORT data transfers always have a non-zero bit count value when active transmission or reception is occurring. Normal SPORT frame syncs occur after the bit count becomes zero.
		0 No error
		1 Error (non-zero bit count at frame sync)

Table 21-29: SPORT_ERR_A Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
5 (R/W1C)	DERRSSTAT	Data Error Secondary Status. The SPORT_ERR_A.DERRSSTAT bit indicates the error status for the half SPORT's secondary channel data buffers. During transmit (SPORT_CTL_A.SPTRAN =1), the SPORT_ERR_A.DERRSSTAT bit indicates the transmit underflow status. During receive (SPORT_CTL_A.SPTRAN =0), the SPORT_ERR_A.DERRSSTAT bit indicates the receive overflow status. This bit is used to clear the latch of SPORT status interrupt request when triggered by a secondary data error. This bit can also be used to clear the read-only SPORT_CTL_A.DERRSEC status bit.
		0 No error
		1 Error (transmit underflow or receive overflow)
4 (R/W1C)	DERRPSTAT	Data Error Primary Status. The SPORT_ERR_A.DERRPSTAT bit indicates the error status for the half SPORT's primary channel data buffers. During transmit (SPORT_CTL_A.SPTRAN =1), the SPORT_ERR_A.DERRPSTAT bit indicates the transmit underflow status. During receive (SPORT_CTL_A.SPTRAN =0), the SPORT_ERR_A.DERRPSTAT bit indicates the receive overflow status. This bit is used to clear the latch of SPORT status interrupt request when triggered by a primary data error. This bit can also be used to clear the read-only SPORT_CTL_A.DERRPRI status bit.
		0 No error
		1 Error (transmit underflow or receive overflow)
2 (R/W)	FSERRMSK	Frame Sync Error (Interrupt) Mask. The SPORT_ERR_A.FSERRMSK unmask (enables) the half SPORT to generate the frame sync error interrupt request.
		0 Mask (disable)
		1 Unmask (enable)
1 (R/W)	DERRSMSK	Data Error Secondary (Interrupt) Mask. The SPORT_ERR_A.DERRSMSK unmask (enables) the half SPORT to generate the data error interrupt request for the secondary channel.
		0 Mask (disable)
		1 Unmask (enable)
0 (R/W)	DERRPMSK	Data Error Primary (Interrupt) Mask. The SPORT_ERR_A.DERRPMSK unmask (enables) the half SPORT to generate the data error interrupt request for the primary channel.
		0 Mask (disable)
		1 Unmask (enable)

Half SPORT 'B' Error Register

The `SPORT_ERR_B` register contains error status and error interrupt mask bits for SPORT half 'B', including error handling bits for the half SPORT's primary and secondary channels and frame sync. Detected errors are frame sync violations or buffer over/underflow conditions.

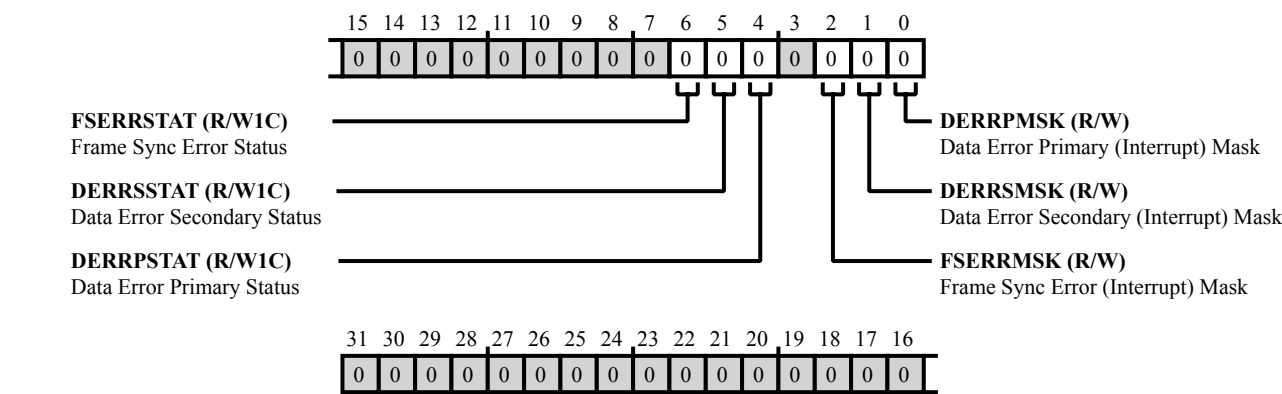


Figure 21-29: SPORT_ERR_B Register Diagram

Table 21-30: SPORT_ERR_B Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
6 (R/W1C)	FSERRSTAT	Frame Sync Error Status. The <code>SPORT_ERR_B.FSERRSTAT</code> bit indicates that the half SPORT has detected a frame sync when the bit count (bits remaining in the frame) is non-zero. When a half SPORT is receiving or transmitting, its bit count is set to a word length (for example, <code>SPORT_CTL_B.SLEN = 31</code>). After each serial clock edge, the half SPORT decrements the transfer's bit count. After the word is received or transmitted, the transfer's bit count reaches zero, and the half SPORT resets it (for example, to 32) on next frame sync. Normal SPORT data transfers always have a non-zero bit count value when active transmission or reception is occurring. Normal SPORT frame syncs occur after the bit count becomes zero.
		0 No error
		1 Error (non-zero bit count at frame sync)

Table 21-30: SPORT_ERR_B Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
5 (R/W1C)	DERRSSTAT	Data Error Secondary Status. The SPORT_ERR_B.DERRSSTAT bit indicates the error status for the half SPORT's secondary channel data buffers. During transmit (SPORT_CTL_B.SPTRAN =1), SPORT_ERR_B.DERRSSTAT indicates the transmit underflow status. During receive (SPORT_CTL_B.SPTRAN =0), SPORT_ERR_B.DERRSSTAT indicates the receive overflow status. This bit is used to clear the latch of SPORT status interrupt request when triggered by a secondary data error. This bit can also be used to clear the read-only SPORT_CTL_B.DERRSEC status bit.
		0 No error
		1 Error (transmit underflow or receive overflow)
4 (R/W1C)	DERRPSTAT	Data Error Primary Status. The SPORT_ERR_B.DERRPSTAT bit indicates the error status for the half SPORT's primary channel data buffers. During transmit (SPORT_CTL_B.SPTRAN =1), the SPORT_ERR_B.DERRPSTAT bit indicates the transmit underflow status. During receive (SPORT_CTL_B.SPTRAN =0), the SPORT_ERR_B.DERRPSTAT bit indicates the receive overflow status. This bit is used to clear the latch of SPORT status interrupt request when triggered by a primary data error. This bit can also be used to clear the read-only SPORT_CTL_B.DERRPRI status bit.
		0 No error
		1 Error (transmit underflow or receive overflow)
2 (R/W)	FSERRMSK	Frame Sync Error (Interrupt) Mask. The SPORT_ERR_B.FSERRMSK unmask (enables) the half SPORT to generate the frame sync error interrupt request.
		0 Mask (disable)
		1 Unmask (enable)
1 (R/W)	DERRSMSK	Data Error Secondary (Interrupt) Mask. The SPORT_ERR_B.DERRSMSK unmask (enables) the half SPORT to generate the data error interrupt request for the secondary channel.
		0 Mask (disable)
		1 Unmask (enable)
0 (R/W)	DERRPMSK	Data Error Primary (Interrupt) Mask. The SPORT_ERR_B.DERRPMSK unmask (enables) the half SPORT to generate the data error interrupt request for the primary channel.
		0 Mask (disable)
		1 Unmask (enable)

Half SPORT 'A' Multichannel Control Register

The `SPORT_MCTL_A` register controls the half SPORT's multichannel operations. This register enables multichannel operation, enables multichannel data packing, selects the multichannel frame delay, selects the number of multichannel slots, and selects the multichannel window offset size.

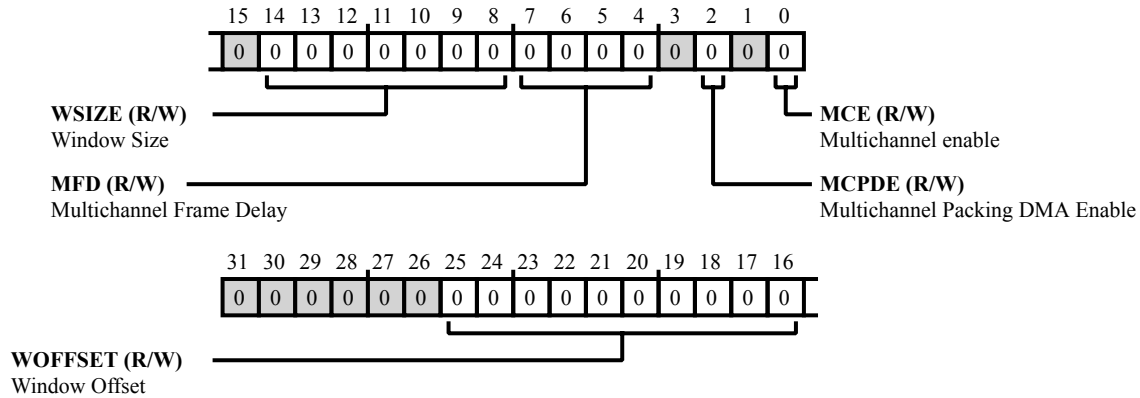


Figure 21-30: SPORT_MCTL_A Register Diagram

Table 21-31: SPORT_MCTL_A Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
25:16 (R/W)	WOFFSET	Window Offset. The <code>SPORT_MCTL_A.WOFFSET</code> bits select the start location for the half SPORT's active window of channels within the 1024-channel range. A value of 0 specifies no offset and 896 is the largest value that permits using all 128 channels. When multichannel mode is disabled (<code>SPORT_MCTL_A.MCE = 0</code>) and the right-justified mode is enabled (<code>SPORT_CTL_A.RJUST = 1</code>), the least significant 6 bits of <code>SPORT_MCTL_A.WOFFSET</code> serve as the delay count (DCNT) field. These bits introduce a clock delay before the half SPORT state machine starts to capture data. For information about appropriate delay selections, see the SPORT operating modes section.
14:8 (R/W)	WSIZE	Window Size. The <code>SPORT_MCTL_A.WSIZE</code> bits select the window size for the half SPORT's active window of channels. Use the following formula to calculate the window size value: $\text{SPORT_MCTL_A.WSIZE} = (\text{number of channel slots}) - 1$
7:4 (R/W)	MFD	Multichannel Frame Delay. The <code>SPORT_MCTL_A.MFD</code> bits select the delay (in serial clock cycles) between the half SPORT's multichannel frame sync pulse and channel 0. The 4-bit field allows selecting multichannel frame delay of 0-15 serial clocks.

Table 21-31: SPORT_MCTL_A Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
2 (R/W)	MCPDE	Multichannel Packing DMA Enable. The SPORT_MCTL_A.MCPDE bit enables DMA data packing for transmit and enables DMA data unpacking for the half SPORT's multichannel data transfers.
		0 Disable
		1 Enable
0 (R/W)	MCE	Multichannel enable. The SPORT_MCTL_A.MCE bit enables multichannel operations for the half SPORT. The half SPORT is configured in normal multichannel mode if SPORT_CTL_A.OPMODE=0; while it is configured in packed mode if SPORT_CTL_A.OPMODE=1. When configuring in these modes, the multichannel enable bit (SPORT_MCTL_A.MCE) should be set before enabling the SPORT data channel enable bits (SPORT_CTL_A.SPENPRI and/or SPORT_CTL_A.SPENSEC). When these channel bits transition from 1 to 0, note that the half SPORT's data transfer buffers are cleared, and the SPORT_CTL_A.DERRPRI and SPORT_CTL_A.DERRSEC bits are cleared.
		0 Disable
		1 Enable

Half SPORT 'B' Multichannel Control Register

The `SPORT_MCTL_B` register controls the half SPORT's multichannel operations. This register enables multichannel operation, enables multichannel data packing, selects the multichannel frame delay, selects the number of multichannel slots, and selects the multichannel window offset size.

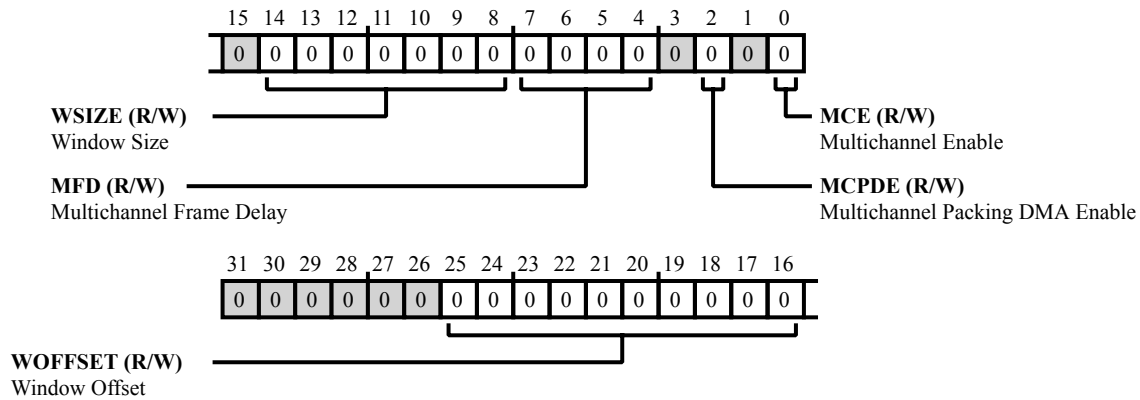


Figure 21-31: `SPORT_MCTL_B` Register Diagram

Table 21-32: `SPORT_MCTL_B` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
25:16 (R/W)	<code>WOFFSET</code>	Window Offset. The <code>SPORT_MCTL_B.WOFFSET</code> bits select the start location for the half SPORT's active window of channels within the 1024-channel range. A value of 0 specifies no offset and 896 is the largest value that permits using all 128 channels. When multichannel mode is disabled (<code>SPORT_MCTL_B.MCE = 0</code>) and right-justified mode is enabled (<code>SPORT_CTL_B.RJUST = 1</code>), the least significant 6 bits of <code>SPORT_MCTL_B.WOFFSET</code> serve as the delay count (DCNT) field. These bits introduce a clock delay before the half SPORT state machine starts to capture data. For information about appropriate delay selections, see the SPORT operating modes section.
14:8 (R/W)	<code>WSIZE</code>	Window Size. The <code>SPORT_MCTL_B.WSIZE</code> bits select the window size for the half SPORT's active window of channels. Use the following formula to calculate the window size value: $\text{SPORT_MCTL_B.WSIZE} = (\text{number of channel slots}) - 1$
7:4 (R/W)	<code>MFD</code>	Multichannel Frame Delay. The <code>SPORT_MCTL_B.MFD</code> bits select the delay (in serial clock cycles) between the half SPORT's multichannel frame sync pulse and channel 0. The 4-bit field allows selecting a multichannel frame delay of 0-15 serial clocks.

Table 21-32: SPORT_MCTL_B Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
2 (R/W)	MCPDE	Multichannel Packing DMA Enable. The SPORT_MCTL_B.MCPDE bit enables DMA data packing for transmit and enables DMA data unpacking for the half SPORT's multichannel data transfers.
		0 Disable
		1 Enable
0 (R/W)	MCE	Multichannel Enable. The SPORT_MCTL_B.MCE bit enables multichannel operations for the half SPORT. The half SPORT is configured in normal multichannel mode if SPORT_CTL_B.OPMODE=0; while it is configured in packed mode if SPORT_CTL_B.OPMODE=1. When configuring in these modes, the multichannel enable bit (SPORT_MCTL_B.MCE) should be set before enabling SPORT data channel enable bits (SPORT_CTL_B.SPENPRI and/or SPORT_CTL_B.SPENSEC). When these channel bits transition from 1 to 0, note that the half SPORT's data transfer buffers are cleared, and the SPORT_CTL_B.DERRPRI and SPORT_CTL_B.DERRSEC bits are cleared.
		0 Disable
		1 Enable

Half SPORT 'A' Multichannel Status Register

The `SPORT_MSTAT_A` register indicates the current multichannel being serviced among the half SPORT's active channels in multichannel mode. The half SPORT increments the value by one in this register as each channel is serviced. The value in the `SPORT_MSTAT_A` register restarts at 0 at each frame sync.

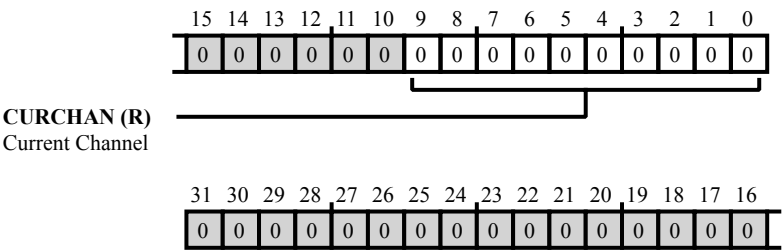


Figure 21-32: `SPORT_MSTAT_A` Register Diagram

Table 21-33: `SPORT_MSTAT_A` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
9:0 (R/NW)	CURCHAN	Current Channel. The <code>SPORT_MSTAT_A.CURCHAN</code> bits indicate the half SPORT's current channel being serviced in multichannel mode.

Half SPORT 'B' Multichannel Status Register

The `SPORT_MSTAT_B` register indicates the current multichannel being serviced among the half SPORT's active channels in multichannel mode. The half SPORT increments the value by one in this register as each channel is serviced. The value in the `SPORT_MSTAT_B` register restarts at 0 at each frame sync.

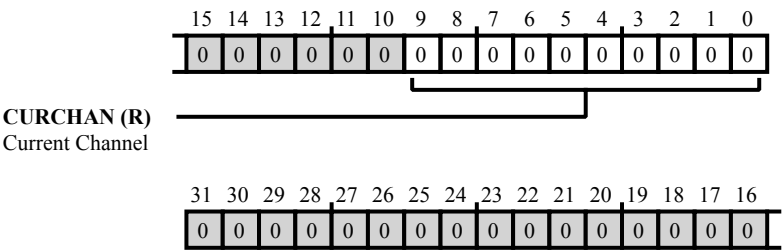


Figure 21-33: SPORT_MSTAT_B Register Diagram

Table 21-34: SPORT_MSTAT_B Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
9:0 (R/NW)	CURCHAN	Current Channel. The <code>SPORT_MSTAT_B.CURCHAN</code> bits indicate the half SPORT's current channel being serviced in multichannel mode.

Half SPORT 'A' Rx Buffer (Primary) Register

The `SPORT_RXPRI_A` register buffers the half SPORT's primary channel receive data. This buffer becomes active when the half SPORT is configured to receive data on the primary channel. After a complete word has been received in the receive shifter, it is placed into the `SPORT_RXPRI_A` register. This data can be read in core mode (in interrupt-based or polling-based mechanism) or directly transferred into processor memory using the DMA controller. With a data buffer and an input shift register, the `SPORT_RXPRI_A` register acts as a two-location buffer. So, the SPORT can keep a maximum of two 32-bit received words at any given time (independent of the `SPORT_CTL_A.PACK` bit setting).

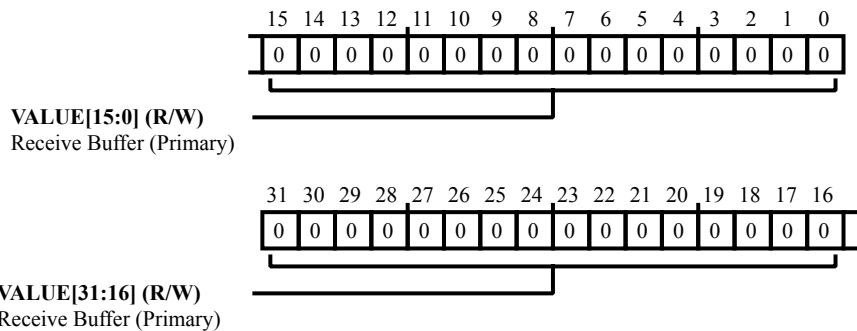


Figure 21-34: SPORT_RXPRI_A Register Diagram

Table 21-35: SPORT_RXPRI_A Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Receive Buffer (Primary). The <code>SPORT_RXPRI_A.VALUE</code> bits hold the half SPORT's primary channel receive data. Note that changes to the half SPORT operation mode (for example, toggling the <code>SPORT_MCTL_A.MCE</code>) empty the contents of this data buffer. For more information, see the <code>SPORT_CTL_A</code> and <code>SPORT_MCTL_A</code> register descriptions.

Half SPORT 'B' Rx Buffer (Primary) Register

The `SPORT_RXPRI_B` register buffers the half SPORT's primary channel receive data. This buffer becomes active when the half SPORT is configured to receive data on the primary channel. After a complete word has been received in the receive shifter, it is placed into the `SPORT_RXPRI_B` register. This data can be read in core mode (in interrupt-based or polling-based mechanism) or directly transferred into processor memory using the DMA controller. With a data buffer and an input shift register, the `SPORT_RXPRI_B` register acts as a two-location buffer. So, the SPORT can keep a maximum of two 32-bit received words at any given time (independent of the `SPORT_CTL_A.PACK` bit setting).

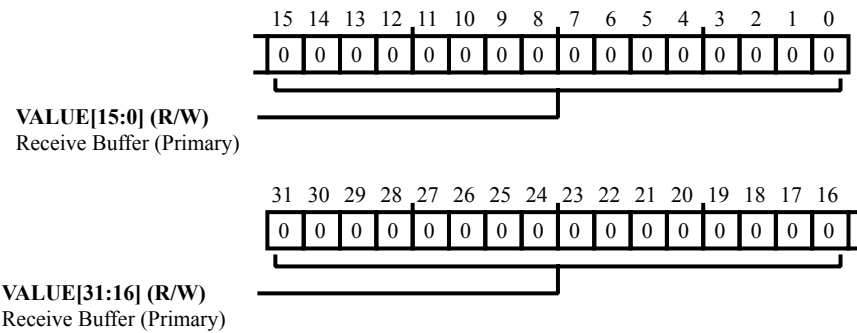


Figure 21-35: `SPORT_RXPRI_B` Register Diagram

Table 21-36: `SPORT_RXPRI_B` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Receive Buffer (Primary). The <code>SPORT_RXPRI_B.VALUE</code> bits hold the half SPORT's primary channel receive data. Note that changes to the half SPORT operation mode (for example, toggling the <code>SPORT_MCTL_B.MCE</code>) empty the contents of this data buffer. For more information, see the <code>SPORT_CTL_B</code> and <code>SPORT_MCTL_B</code> register descriptions.

Half SPORT 'A' Rx Buffer (Secondary) Register

The `SPORT_RXSEC_A` register buffers the half SPORT's secondary channel receive data. This buffer becomes active when the half SPORT is configured to receive data on the secondary channel. After a complete word has been received in the receive shifter, it is placed into the `SPORT_RXSEC_A` register. This data can be read in core mode (in interrupt-based or polling-based mechanism) or directly transferred into processor memory using the DMA controller. With a data buffer and an input shift register, the `SPORT_RXSEC_A` register acts as a two-location buffer. So, the SPORT can keep a maximum of two 32-bit received words at any given time (independent of the `SPORT_CTL_A.PACK` bit setting).

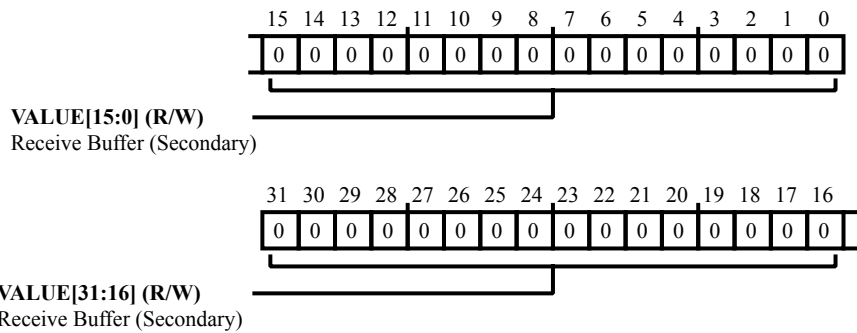


Figure 21-36: SPORT_RXSEC_A Register Diagram

Table 21-37: SPORT_RXSEC_A Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Receive Buffer (Secondary). The <code>SPORT_RXSEC_A.VALUE</code> bits hold the half SPORT's secondary channel receive data. Note that changes to the half SPORT operation mode (for example, toggling the <code>SPORT_MCTL_A.MCE</code>) empty the contents of this data buffer. For more information, see the <code>SPORT_CTL_A</code> and <code>SPORT_MCTL_A</code> register descriptions.

Half SPORT 'B' Rx Buffer (Secondary) Register

The `SPORT_RXSEC_B` register buffers the half SPORT's secondary channel receive data. This buffer becomes active when the half SPORT is configured to receive data on the secondary channel. After a complete word has been received in the receive shifter, it is placed into the `SPORT_RXSEC_B` register. This data can be read in core mode (in interrupt-based or polling-based mechanism) or directly transferred into processor memory using the DMA controller. With a data buffer and an input shift register, the `SPORT_RXSEC_B` register acts as a two-location buffer. So, the SPORT can keep a maximum of two 32-bit received words at any given time (independent of the `SPORT_CTL_A.PACK` bit setting).

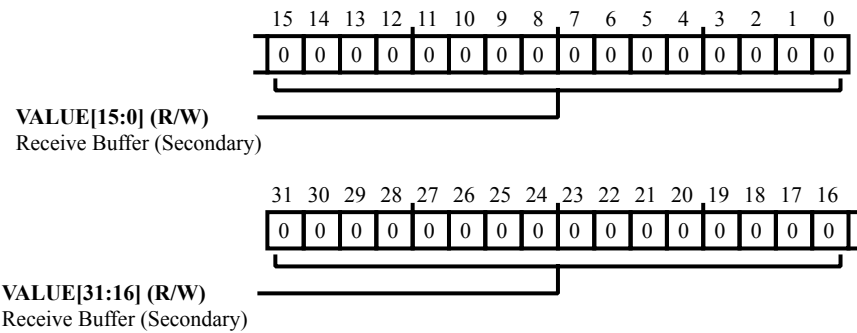


Figure 21-37: `SPORT_RXSEC_B` Register Diagram

Table 21-38: `SPORT_RXSEC_B` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Receive Buffer (Secondary). The <code>SPORT_RXSEC_B.VALUE</code> bits hold the half SPORT's secondary channel receive data. Note that changes to the half SPORT operation mode (for example, toggling the <code>SPORT_MCTL_B.MCE</code>) empty the contents of this data buffer. For more information, see the <code>SPORT_CTL_B</code> and <code>SPORT_MCTL_B</code> register descriptions.

Half SPORT 'A' Tx Buffer (Primary) Register

The `SPORT_TXPRI_A` register buffers the half SPORT's primary channel transmit data. This register must be loaded with the data to be transmitted if the half SPORT is configured to transmit on the primary channel. Either a program running on the processor core loads the data into the buffer (word-by-word process) or the DMA controller automatically loads the data into the buffer (DMA process).

The `SPORT_TXPRI_A` register acts as a three-location buffer if SPORT data packing is disabled (`SPORT_CTL_A.PACK = 0`); while it acts as a two-location buffer when packing is enabled (`SPORT_CTL_A.PACK = 1`). So, depending on the `PACK` bit setting, two 32-bit words or three 32-bit words can be stored in the transmit queue at any time. When the transmit register is loaded and any previous word has been transmitted, the `SPORT_TXPRI_A` register contents are automatically loaded into the output shifter. The half SPORT can issue an interrupt request (transmit buffer is not full) when it has loaded the output transmit shifter, signifying that the transmit buffer is ready to accept the next word. This interrupt request does not occur when the half SPORT is executing a DMA-based transfer.

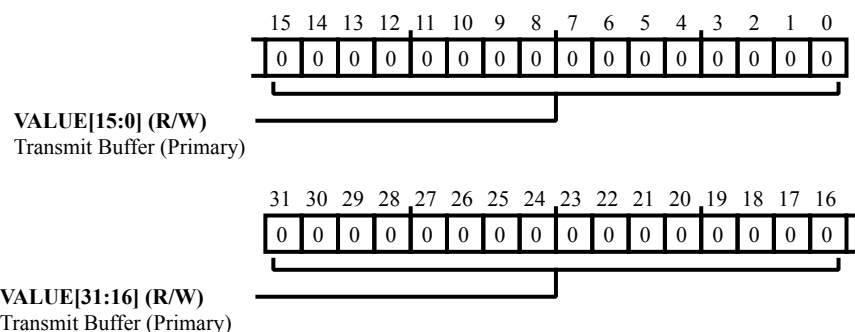


Figure 21-38: `SPORT_TXPRI_A` Register Diagram

Table 21-39: `SPORT_TXPRI_A` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Transmit Buffer (Primary). The <code>SPORT_TXPRI_A.VALUE</code> bits hold the half SPORT's primary channel transmit data. Note that changes to the half SPORT operation mode (for example, toggling the <code>SPORT_MCTL_A.MCE</code>) empty the contents of this data buffer. For more information, see the <code>SPORT_CTL_A</code> and <code>SPORT_MCTL_A</code> register descriptions.

Half SPORT 'B' Tx Buffer (Primary) Register

The `SPORT_TXPRI_B` register buffers the half SPORT's primary channel transmit data. This register must be loaded with the data to be transmitted if the half SPORT is configured to transmit on the primary channel. Either a program running on the processor core loads the data into the buffer (word-by-word process) or the DMA controller automatically loads the data into the buffer (DMA process).

The `SPORT_TXPRI_B` register acts as a three-location buffer if SPORT data packing is disabled (`SPORT_CTL_B.PACK = 0`); while it acts as a two-location buffer when packing is enabled (`SPORT_CTL_B.PACK = 1`). So, depending on the `PACK` bit setting, two 32-bit words or three 32-bit words can be stored in the transmit queue at any time. When the transmit register is loaded and any previous word has been transmitted, the `SPORT_TXPRI_B` register contents are automatically loaded into the output shifter. The half SPORT can issue an interrupt request (transmit buffer is not full) when it has loaded the output transmit shifter, signifying that the transmit buffer is ready to accept the next word. This interrupt request does not occur when the half SPORT is executing a DMA-based transfer.

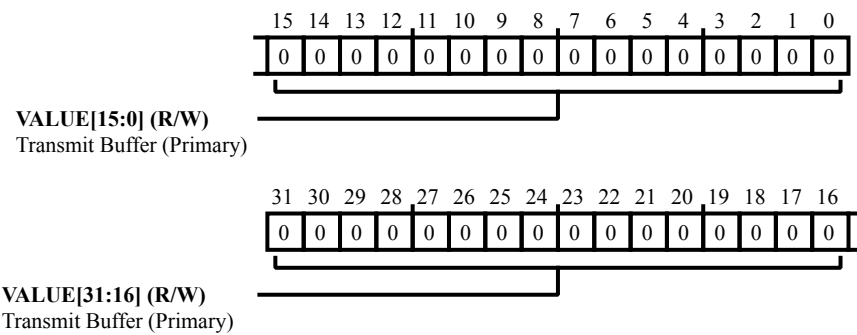


Figure 21-39: `SPORT_TXPRI_B` Register Diagram

Table 21-40: `SPORT_TXPRI_B` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Transmit Buffer (Primary). The <code>SPORT_TXPRI_B.VALUE</code> bits hold the half SPORT's primary channel transmit data. Note that changes to the half SPORT operation mode (for example, toggling the <code>SPORT_MCTL_B.MCE</code>) empty the contents of this data buffer. For more information, see the <code>SPORT_CTL_B</code> and <code>SPORT_MCTL_B</code> register descriptions.

Half SPORT 'A' Tx Buffer (Secondary) Register

The `SPORT_TXSEC_A` register buffers the half SPORT's secondary channel transmit data. This register must be loaded with the data to be transmitted if the half SPORT is configured to transmit on the secondary channel. Either a program running on the processor core loads the data into the buffer (word-by-word process) or the DMA controller automatically loads the data into the buffer (DMA process).

The `SPORT_TXSEC_A` register acts as a three-location buffer if SPORT data packing is disabled (`SPORT_CTL_A.PACK = 0`); while it acts as a two-location buffer when packing is enabled (`SPORT_CTL_A.PACK = 1`). So, depending on the `PACK` bit setting, two 32-bit words or three 32-bit words can be stored in the transmit queue at any time. When the transmit register is loaded and any previous word has been transmitted, the `SPORT_TXSEC_A` register contents are automatically loaded into the output shifter. The half SPORT can issue an interrupt request (transmit buffer is not full) when it has loaded the output transmit shifter, signifying that the transmit buffer is ready to accept the next word. This interrupt request does not occur when the half SPORT is executing a DMA-based transfer.

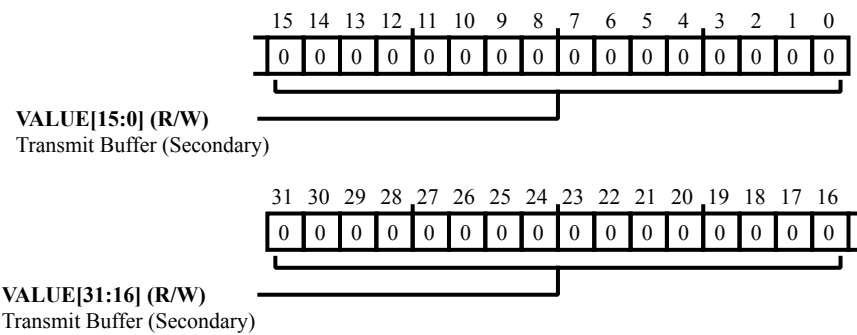


Figure 21-40: `SPORT_TXSEC_A` Register Diagram

Table 21-41: `SPORT_TXSEC_A` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Transmit Buffer (Secondary). The <code>SPORT_TXSEC_A.VALUE</code> bits hold the half SPORT's secondary channel transmit data. Note that changes to the half SPORT operation mode (for example, toggling the <code>SPORT_MCTL_A.MCE</code>) empty the contents of this data buffer. For more information, see the <code>SPORT_CTL_A</code> and <code>SPORT_MCTL_A</code> register descriptions.

Half SPORT 'B' Tx Buffer (Secondary) Register

The `SPORT_TXSEC_B` register buffers the half SPORT's secondary channel transmit data. This register must be loaded with the data to be transmitted if the half SPORT is configured to transmit on the secondary channel. Either a program running on the processor core loads the data into the buffer (word-by-word process) or the DMA controller automatically loads the data into the buffer (DMA process).

The `SPORT_TXSEC_B` register acts as a three-location buffer if SPORT data packing is disabled (`SPORT_CTL_B.PACK = 0`); while it acts as two-location buffer when packing is enabled (`SPORT_CTL_B.PACK = 1`). So, depending on the `PACK` bit setting, two 32-bit words or three 32-bit words can be stored in the transmit queue at any time. When the transmit register is loaded and any previous word has been transmitted, the `SPORT_TXSEC_B` register contents are automatically loaded into the output shifter. The half SPORT can issue an interrupt request (transmit buffer is not full) when it has loaded the output transmit shifter, signifying that the transmit buffer is ready to accept the next word. This interrupt request does not occur when the half SPORT is executing a DMA-based transfer.

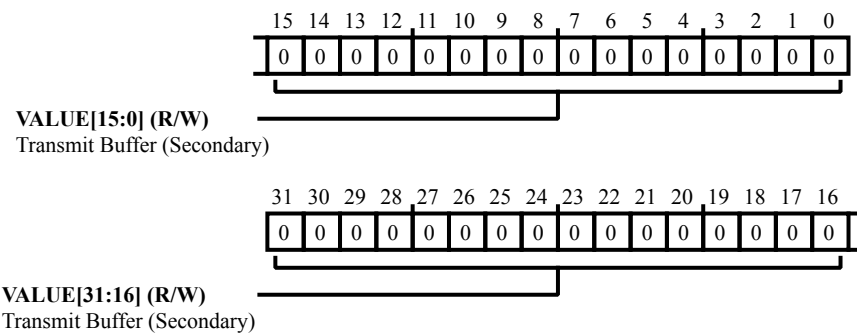


Figure 21-41: `SPORT_TXSEC_B` Register Diagram

Table 21-42: `SPORT_TXSEC_B` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Transmit Buffer (Secondary). The <code>SPORT_TXSEC_B.VALUE</code> bits hold the half SPORT's secondary channel transmit data. Note that changes to the half SPORT operation mode (for example, toggling the <code>SPORT_MCTL_B.MCE</code>) empty the contents of this data buffer. For more information, see the <code>SPORT_CTL_B</code> and <code>SPORT_MCTL_B</code> register descriptions.

22 Precision Clock Generators (PCG)

The precision clock generators are used to produce a pair of signals from a clock input signal. The two signals generated are normally used as a serial bit clock and frame sync pair. The PCG is part of the DAI. There are two PCG signals in each DAI: *PCGA* and *PCGB* belong to DAI0, *PCGC* and *PCGD* belong to DAI1.

Features

The following list describes the features of the precision clock generators.

- SRU allows the routing of all of the PCG signals in one DAI (four PCG signals in DAI0 or DAI1)
- Input clock selection: *SYS_CLKIN0* can be used as the input clock for PCG, when configured to use *CLKIN* as clock source.
- Provides four different clock dividers for serial clock, frame sync, phase (20-bit), and pulse width (16-bit)
- Phase shift allows adjustment of the frame sync relative to the serial clock and can be shifted the full period and wrapped around
- Provides pulse width control for arbitrary frame sync signal generation
- Bypass mode for external frame sync manipulation
- External trigger mode starts PCG operation
- No additional jitter is introduced when using off-chip clocks

Functional Description

The *PCG Block Diagram* shows the blocks within the module and its connection to the DAI. The following sections provide information on the function of these blocks.

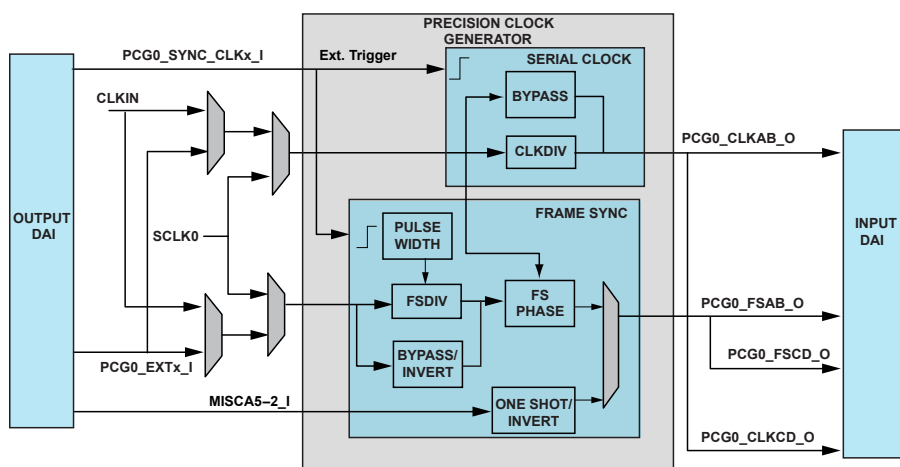


Figure 22-1: PCG Block Diagram

ADSP-21560_21561_21564_21568 PCG Register List

Precision Clock Generator

Table 22-1: ADSP-21560_21561_21564_21568 PCG Register List

Name	Description
PCG_CTLA0	Precision Clock A Control 0 Register
PCG_CTLA1	Precision Clock A Control 1 Register
PCG_CTLB0	Precision Clock B Control 0 Register
PCG_CTLB1	Precision Clock B Control 1 Register
PCG_CTLC0	Precision Clock C Control 0 Register
PCG_CTLC1	Precision Clock C Control 1 Register
PCG_CTLD0	Precision Clock D Control 0 Register
PCG_CTLD1	Precision Clock D Control 1 Register
PCG_PW1	Precision Clock Pulse Width Control 1 Register
PCG_PW2	Precision Clock Pulse Width Control 2 Register
PCG_SYNC1	Precision Clock Frame Sync Synchronization 1 Register
PCG_SYNC2	Precision Clock Frame Sync Synchronization 2 Register

22.2.2

Internal Interface

The fundamental clock of the PCG is *SCLK0*. The clock to this module can be shut off for power savings.

Serial Clock

Each of the four units (A, B, C, and D) produces a clock output. Serial clock generation from a unit is independently enabled and controlled. Sources for the serial clock generation can be either from the CLKIN, SCLK0, or a DAI pin source.

When CLKIN is chosen as input clock in PCG, the clock source is SYS_CLKIN0.

Note that the divider is working in normal mode for $PCG0_CTLx1.CLKDIV > 1$. For $PCG_CTLA1.CLKDIV/PCG_CTLB1.CLKDIV = 0$ or 1 , the divider operates in bypass mode (input clock is fed directly to its output). In bypass mode, the clock at the output can theoretically run at up to the SCLK0 frequency. Check the data sheet for the specified maximum operation speed of the DAI pin buffers.

Note that the clock output is always set (as closely as possible) to a 50% duty cycle. If the clock divisor is even, the duty cycle of the clock output is exactly 50%. If the clock divisor is odd, then the duty cycle is slightly less than 50%. The low period of the output clock is one input clock period more than the high period of the output clock. For higher values of an odd divisor, the duty cycle is close to 50%.

NOTE: A PCG clock output cannot be fed to its own input.

Frame Sync

The following sections describe the use of frame syncs in the PCGs.

Frame Sync Output

Each of the four units (A through D) produces a synchronization signal for framing serial data. The frame sync outputs are much more flexible since they must accommodate the wide variety of serial protocols used by peripherals.

Frame sync generation from a unit is independently enabled and controlled. Sources for the frame sync generation can be either from the crystal buffer output, SCLK0, or an external pin source. There is only one external source pin for both frame sync and clock output for a unit.

If an external source is selected for both frame sync and clock output for a unit, then they operate on the same input signal. Apart from enable and source select control bits, a 20-bit divisor controls frame sync generation.

Divider Mode Selection

If a frame sync divisor is greater than 1, the PCG frame sync output frequency is equal to the input clock frequency, divided by a 20-bit integer. This integer is specified in the $PCG_CTLA0.FSDIV/PCG_CTLB0.FSDIV$ bit field (bits 19:0).

However, if the frame sync divisor is 0 or 1, the PCG's frame sync clock generation unit is bypassed, and the frame sync input is connected directly to the frame sync output. For $PCG_CTLB0.FSDIV = 0, 1$ [PCG_PW1/PCG_PW2](#) registers function differently than in normal mode.

Phase Shift

Phase shift is a frame sync parameter that defines the phase shift of the frame sync relative to the input clock of the same unit. This feature allows the shifting of the frame sync signal in time relative to the clock input signal. Frame sync phase shifting is often required by peripherals that need a frame sync signal to lead or lag a clock signal.

For example, the I²S protocol specifies that the frame sync transition from high-to-low occurs one clock cycle before the beginning of a frame. Since an I²S frame is 64 clock cycles long, delaying the frame sync by 63 cycles produces the required framing.

Phase shifting is represented as a full 20-bit value. Even when the frame sync is divided by the maximum amount, the phase can be shifted to the full range, from zero to one input clock short of the period.

NOTE: Phase shifting is specified as a 2 x 10-bit divider value in the `PCG_CTLA0.FSPHASEHI` bit field (bits 29:20) and in the `PCG_CTLA1.FSPHASELO` bit field (bits 29:20).

A single 20-bit value spans these two-bit fields. The upper half of the word (bits 19:10) is in the `PCG_CTLA0` register, and the lower half (bits 9:0) is in the `PCG_CTLA1` register.

The phase shift between clock and frame sync outputs can be programmed using the `PCG_PW1/PCG_PW2` registers and all of the control registers under these conditions:

- The input clock source for the clock generator output and the frame sync generator output is the same.
- The clock and frame sync are enabled at the same time using a single atomic instruction.
- The frame sync divisor is an integral multiple of the clock divisor.

NOTE: When using a clock and frame sync as a synchronous pair, the units must be enabled in a single atomic instruction before their parameters are modified. Both units must also be disabled in a single atomic instruction.

NOTE: If the phase shift is 0 (see the *Phase and Pulse Width Settings* figure), the clock and frame sync outputs rise at the same time.

If the phase shift is 1, the frame sync output transitions one input clock period ahead of the clock transition.

If the phase shift is $divisor - 1$, the frame sync transitions $divisor - 1$ input clock periods ahead of the clock transitions.

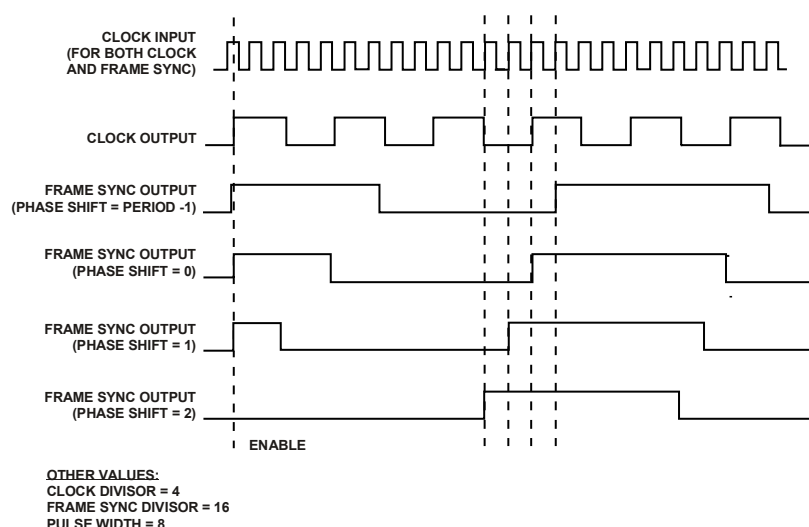


Figure 22-2: Phase and Pulse Width Settings

NOTE: When generating single frame sync pulses (the length of one SPORT clock cycle), take care with respect to the drive and sampling edges. If the rules are violated, for example if the SPORT is not driving data, the module cannot detect a valid sample edge.

Pulse Width

Pulse width is the number of input clock periods for which the frame sync output is high.

A 16-bit value determines the width of the framing pulse. Settings for pulse width can range from zero to $DIV - 1$. The pulse width should be less than the divisor of the frame sync. The pulse width of frame sync is specified in the `PCG_PW1.FSA`, `PCG_PW1.FSB`, `PCG_PW2.FSC`, and `PCG_PW2.FSD` bits (15-0) and (31-16).

Default Pulse Width

If the pulse width count is equal to 0 and if the `PCG_CTLA0.FSDIV/PCG_CTLB0.FSDIV` bit field is even, then the actual pulse width of the frame sync output is equal to:

For even divisors: frame sync divisor/2

If the pulse width count is equal to 0 and if the `PCG_CTLA0.FSDIV/PCG_CTLB0.FSDIV` bit field is odd, then the actual pulse width of the frame sync output is equal to:

For odd divisors: frame sync divisor – 1/2

Input Clock Source Considerations

The core Phase-Locked Loop (PLL) has been designed to provide clocking for the processor core. The performance specifications of this PLL are appropriate for the core. But they have not been optimized or specified for precision data converters where jitter directly translates into time quantization errors and distortion.

The PCG allows the routing of external clock sources which are independent of the core PLL.

Timing Example for I²S Mode

For I²S mode, the frame sync should be driven at the falling edge of SPORT clock. In other words, the frame sync edge must coincide with the falling edge of the SPORT clock. To satisfy this requirement, program the phase of the frame sync accordingly in the PCG control registers.

For example, assume that the input clock source for both clock and frame sync are the same and both the clock and frame sync are enabled at the same time. Also, assume that the clock divisor value for generating the required SPORT clock is `PCG_CTLA1.CLKDIV = 4`. Then, for a 32-bit word length, the frame sync divisor value is:

By default, for phase = 0, the rising edge of both SPORT clock and frame sync coincide. To make sure that the frame sync edges coincide with the falling edge of the SPORT clock, program the phase value as:

`PCG_CTLA1.CLKDIV/2 = 2`.

Cross Mode Connections

The symmetric dual DAI architecture allows cross connections between both PCGs (A,B) and (C,D) to the other DAI. Each PCG (A through D) supports an alternative input clock (`PCG0_EXTx_I`) (see [Figure 22-1 PCG Block Diagram](#)) which can be sourced via a DAI pin buffer from the other DAI. Note however if routing a source (clock or FS) only DAI pin buffer 2 to 20 can be used (DAI pin buffer 1 is no longer available and is replaced by the DAI CRS buffer for the other DAI).

See [DAI Routing Capabilities](#) for more information.

Operating Modes

The following sections provide information on the operating modes of the precision clock generator.

Normal Mode

When the frame sync divisor is set to any value other than zero or one, the PCGs operate in normal mode. In normal mode, the divisor determines the frequency of the frame sync output where:

Frequency of Frame Sync Output = Input Frequency/Divisor

The value of the pulse width control determines the high period of the frame sync output. The value of the pulse width control must be less than the value of the divisor.

The value of the phase control determines the phase of the frame sync output. When the phase is zero, then the positive edges of the clock and frame sync coincide when:

- the clock and frame sync dividers are enabled at the same time using an atomic instruction
- the divisors of the clock and frame sync are the same
- the source for the clock and frame sync is the same

The number of input clock cycles that have already elapsed before the frame sync is enabled is equal to the difference between the divisor and the phase values. When the phase is a small fraction of the divisor, then the frame sync leads

the clock. When the phase is only slightly less than the frame sync divisor, then the frame sync lags the clock. The frame sync phase must not be greater than the divisor.

Bypass Mode

When the frame sync divisor for the frame sync has a value of zero or one, the frame sync is in bypass mode and the `PCG_PW1/PCG_PW2` register has different functionality than in normal mode.

NOTE: In normal mode, bits 15:0 and 31:16 of the `PCG_PW1/PCG_PW2` register are used to program the pulse width count. In bypass mode, bits 15:2 and 31:18 are ignored.

If the `PCG_PW1.STROBEA` field is cleared, then the input is directly passed (see the *Bypass and Inverted Bypass* figure) to the frame sync output either inverted or not inverted, depending on the `PCG_PW1.INVFSA` bits.

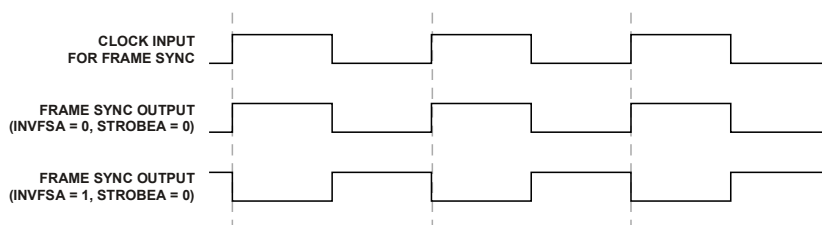


Figure 22-3: Bypass and Inverted Bypass

One-Shot Mode

In the one-shot mode operation shown in the *One Shot Mode PCG A (MISCA2_I Input)* figure, the PCG produces a series of periods but does not run continuously.

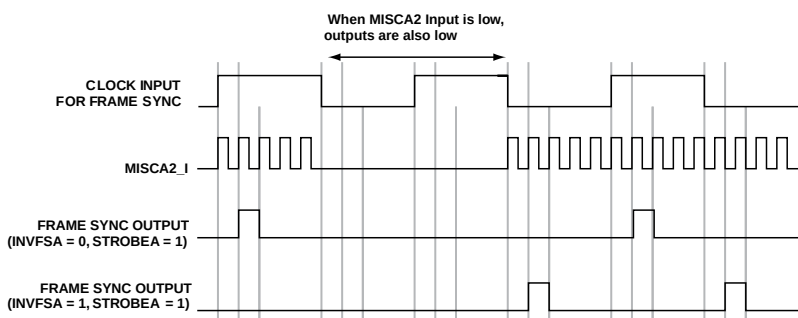


Figure 22-4: One Shot Mode PCG A (MISCA2_I Input)

Bypass mode also enables the generation of a strobe pulse (one-shot frame sync). Strobe usage ignores the divider counters and looks to the SRU to provide the input signal.

The following are the input signals for the respective PCG instances in One-shot mode:

- PCG A: DAI0_MISCA2_I
- PCG B: DAI0_MISCA3_I
- PCG C: DAI1_MISCA2_I

- PCG D: DAI1_MISCA3_I

NOTE: A strobe period is defined to be the period of the FS input clock signal as specified by the `PCG_CTLA1.FSSRC` bit.

Audio System Example

The *PCG Setup for I²S or Left-Justified DAI* figure shows an example of the interconnections between the S/PDIF receiver, ASRC, and the PCGs. The interconnections are made by programming the signal routing unit. It shows how to set up two precision clock generators using the S/PDIF receiver and an asynchronous sample rate converter (ASRC) to interface to an external audio DAC. The PCG is configured to provide a fixed ASRC/DAC output sample rate of 65.098 kHz. The input to the S/PDIF receiver is typically 44.1 kHz if supplied by a CD player, but can also be from other source at any nominal sample rates.

Similarly, the phase shift for frame syncs B, C, and D is specified in the corresponding control registers (`PCG_CTLA0` through `PCG_CTLB1`).

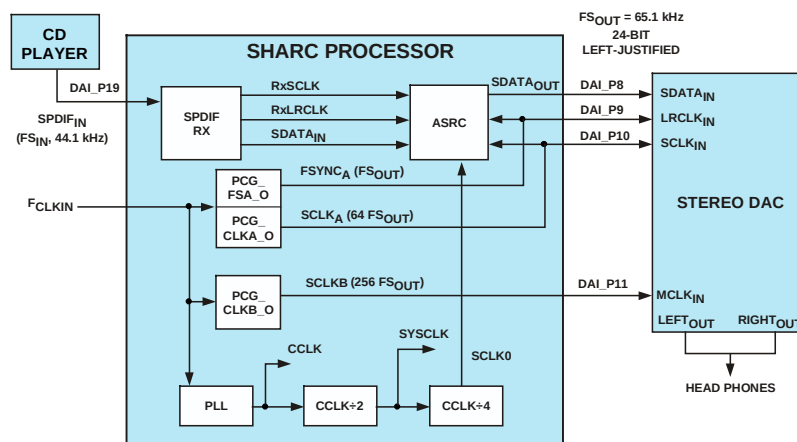


Figure 22-5: PCG Setup for I²S or Left-Justified DAI

Three synchronous clocks are required in audio systems:

1. Frame sync (FS)
2. Serial bit clock (64 FS)
3. Controller DAC clock (256 FS)

Since each PCG has only two outputs, this example requires two PCGs. Furthermore, because the digital audio interface requires a fixed-phase relation between serial clock and FS, these two outputs should come from one PCG (PCG A). The controller clock comes from the second (PCG B).

The combined PCGs can provide a selection of synchronous clock frequencies to support alternate sample rates for the ASRCs and external DACs. However, the range of choices is limited by CLKIN and the ratio of `PCG_CLKx_O`: serial clock:FS. The ratio is normally fixed at 256:64:1 to support digital audio left-justified, I²S and right-justified interface modes.

Many DACs also support 384, 512, and 786x FS for PCG_CLKx_O, which allows some additional flexibility in choosing serial clock.

Note that the falling edge of serial clock must always be synchronous with both edges of FS. This condition requires that the phase of the serial clock and FS signals for a common PCG (PCG A) be adjustable.

While the frequency of the controller DAC clock (PCG_CLKx_O) must be synchronous with the sample rate supplied to the external DAC, there is no fixed-phase requirement.

Configure the clock divisor and source and low-phase word first, followed by the control register enable bits, which are set together. When the PCG_PW1/PCG_PW2 registers are configured to zero (default), the FS pulse width is (divisor /2) for even divisors and (divisor-1)/2 for odd divisors. Alternatively, the PCG_PW1/PCG_PW2 registers can be set high for exactly one-half the period of CLKIN cycles for a 50% duty cycle, provided the FS divisor is an even number.

Clock Configuration Examples

For a CLKIN = 33.330 MHz, the two PCGs provide the three synchronous clocks PCGx_CLK, serial clock and FS for the SRCs and external DAC. These divisors are stored in the PCG_CTLA1.CLKDIV/PCG_CTLB1.CLKDIV bit fields.

The integer divisors for several sample rates based on 33.330 MHz CLKIN are shown in the *Precision Clock Generator Division Ratios* table.

Table 22-2: Precision Clock Generator Division Ratios (33.330 CLKIN)

Sample Rate kHz)	PCG Divisors		FSDIV A* ¹
	CLKDIV B	CLKDIV A	
130.195	1	4	256
65.098	2	8	512
43.398	3	12	768
32.549	4	16	1024
26.039	5	20	1280
21.699	6	24	1536
18.599	7	28	1792

*1 The frame sync divisor must be an even integer to produce a 50% duty cycle waveform. See [Frame Sync](#).

PCG Event Control

The following sections describe the generation and control of PCG events.

External Event Trigger

The trigger with the external clock is enabled by setting bits 0 and 16 of the `PCG_SYNC1/PCG_SYNC2` registers.

Refer to the *FS Output Synchronization With External Trigger Input* figure. Since the rising edge of the external clock is used to synchronize with the frame sync, the frame sync output is not generated until a rising edge of the external clock is sensed.

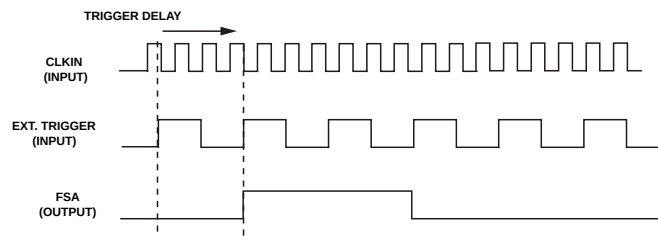


Figure 22-6: FS Output Synchronization With External Trigger Input

External Event Trigger Delay

The time delay between the rising trigger edge and the start of serial clock and frame sync varies between 2.5 to 3.5 input clock periods. If the input clock and the trigger signal are synchronous, the delay is 3 input clock periods.

Consider the following cases:

- SCLK0 is the input source. In this case, if the given trigger event is synchronous to SCLK0, the delay is 3 SCLK0 periods. If the trigger signal is asynchronous with SCLK0, the delay varies from 2.5 SCLK0 periods to 3.5 SCLK0 periods. (It depends on whether the trigger edge occurs in the positive half cycle or negative half cycle of SCLK0.)
- The SYS_CLKIN0 is the input source for PCG A, B, C, D. In this case, if the given trigger signal is synchronous to CLKIN, the delay is 3 CLKIN periods. But if they are asynchronous to CLKIN, the delay can vary between 2.5 CLKIN periods to 3.5 CLKIN periods.
- SRU DAI0 is the input source for PCG A, B. SRU DAI1 is the input source for PCG C, D. If the input clock and trigger signal are synchronous, the delay is exactly 3 input clock periods. If asynchronous, it varies between 2.5 to 3.5 input clock periods depending on the phase difference between the input clock and trigger signal.

Programming Model

This section describes the sequence of software steps required for successful PCG operation.

If the PCG is disabled to reprogram a parameter, use a delay after writing to the disable bit. This delay in core clock (*CCLK*) cycles = (PCG source clock period/*CCLK* period). In summary, use the following general procedure:

1. Clear the PCG enable bits without modifying any other settings.
2. Wait for N *CCLK* cycles ($N = \text{PCG source clock period} / \text{processor clock period}$).
3. Program all new parameters without setting the PCG enable bit.
4. Enable the PCG.

Frame Sync Phase Setting

The phase unit requires that the clock and FS are enabled simultaneously in an atomic instruction.

1. Write the clock divider/low 10-bit phase divider to the `PCG_CTLA1/PCG_CTLB1` registers.
2. Program the FS divider/high 10-bit phase divider, enabling both the `PCG_CTLA0.CLKEN/PCG_CTLA0.FSEN` and the `PCG_CTLB0.CLKEN/PCG_CTLB0.FSEN` bits.

Note that both units must be disabled in the same way.

External Event Trigger

The trigger with the external clock is enabled by setting bits 0 and 16 of the `PCG_SYNC` register. Program the phase to 3, so that the rising edge of the external clock is in-sync with the frame sync (*FS Output Synchronization With External Trigger Input*).

Use the following steps.

1. Program the `PCG_SYNC1/PCG_SYNC2` registers and the `PCG_CTLA0` through `PCG_CTLB1` registers appropriately.
2. Enable the clock or frame sync, or both.

Since the rising edge of the external clock is used to synchronize with the frame sync, the frame sync output is not generated until a rising edge of the external clock is sensed.

Debug Features

Take care in cases where any input to the phase unit is modified. Any individual change of the `PCG_CTLA1.CLKDIV` or `PCG_CTLA0.FSDIV` dividers can cause a failure in the PCG sync operation between the serial clock and the frame sync. Only the programming model ensures a correct setup for phase settings.

ADSP-21560_21561_21564_21568 PCG Register Descriptions

Precision Clock Generator (PCG) contains the following registers.

Table 22-3: ADSP-21560_21561_21564_21568 PCG Register List

Name	Description
<code>PCG_CTLA0</code>	Precision Clock A Control 0 Register
<code>PCG_CTLA1</code>	Precision Clock A Control 1 Register
<code>PCG_CTLB0</code>	Precision Clock B Control 0 Register
<code>PCG_CTLB1</code>	Precision Clock B Control 1 Register
<code>PCG_CTLC0</code>	Precision Clock C Control 0 Register
<code>PCG_CTLC1</code>	Precision Clock C Control 1 Register

Table 22-3: ADSP-21560_21561_21564_21568 PCG Register List (Continued)

Name	Description
PCG_CTLD0	Precision Clock D Control 0 Register
PCG_CTLD1	Precision Clock D Control 1 Register
PCG_PW1	Precision Clock Pulse Width Control 1 Register
PCG_PW2	Precision Clock Pulse Width Control 2 Register
PCG_SYNC1	Precision Clock Frame Sync Synchronization 1 Register
PCG_SYNC2	Precision Clock Frame Sync Synchronization 2 Register

Precision Clock A Control 0 Register

The `PCG_CTLA0` register enables the clock, frame sync, and select divisor for the PCG0 clock A signal.

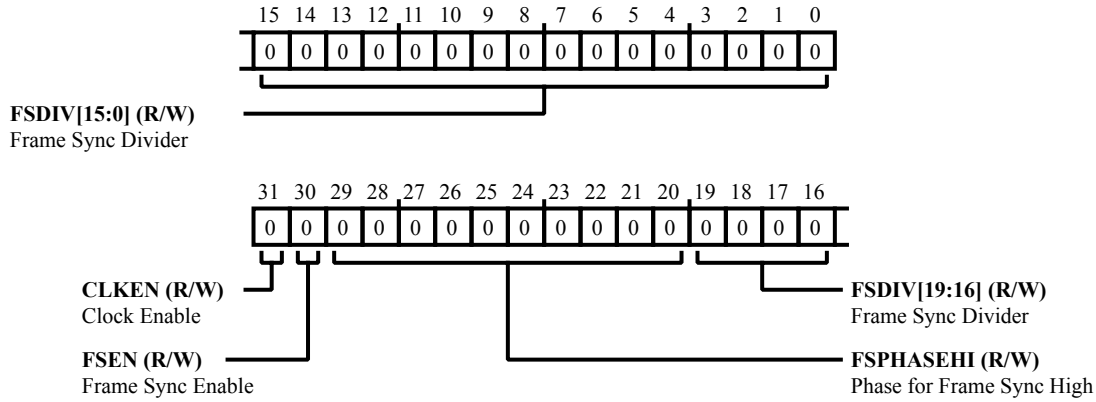


Figure 22-7: PCG_CTLA0 Register Diagram

Table 22-4: PCG_CTLA0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	CLKEN	Clock Enable. The <code>PCG_CTLA0 . CLKEN</code> bit enables the clock.
		0 Clock generation disabled
		1 Clock generation enabled
30 (R/W)	FSEN	Frame Sync Enable. The <code>PCG_CTLA0 . FSEN</code> bit enables the frame sync.
		0 Frame sync generation disabled
		1 Frame sync generation enabled
29:20 (R/W)	FSPHASEHI	Phase for Frame Sync High. The <code>PCG_CTLA0 . FSPHASEHI</code> bit field represents the upper half of the 20-bit value for the channel A/B/C/D frame sync phase.
19:0 (R/W)	FSDIV	Frame Sync Divider. The <code>PCG_CTLA0 . FSDIV</code> bit field provides the frame sync divider value. This 20-bit field frame sync divider is multiplexed where: <code>PCG_CTLA0 . FSDIV > 1</code> PCGx is in normal mode, <code>PCG_CTLA0 . FSDIV = 0, 1</code> PCGx is in bypass mode.
		0 PCG is in bypass mode
		1 PCG is in bypass mode
		2-1048575 PCG is in normal mode

Precision Clock A Control 1 Register

The `PCG_CTLA1` register sets the clock divisor, frame sync source, and clock source for the PCG1 clock A signal.

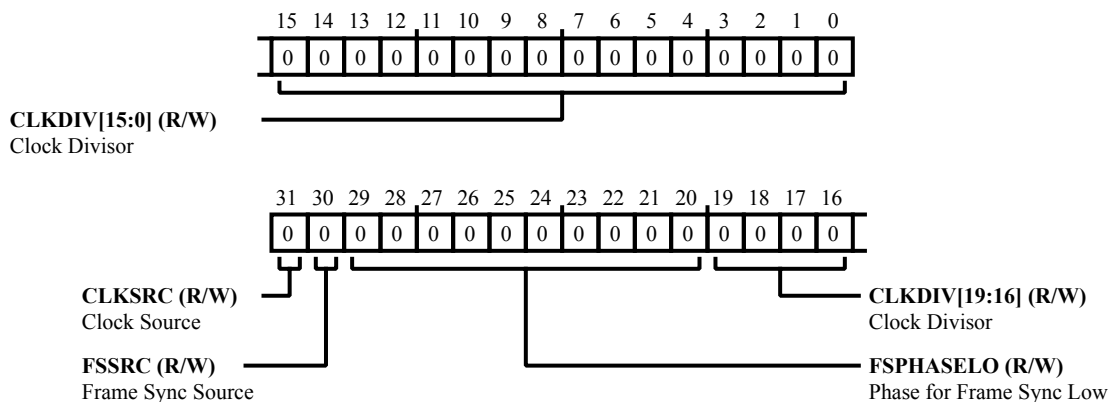


Figure 22-8: `PCG_CTLA1` Register Diagram

Table 22-5: `PCG_CTLA1` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	CLKSRC	Clock Source. The <code>PCG_CTLA1 . CLKSRC</code> bit specifies the clock source.
		0 CLKIN0 pin selected for clock
		1 <code>PCG_EXT_DAI0</code> selected for clock
30 (R/W)	FSSRC	Frame Sync Source. The <code>PCG_CTLA1 . FSSRC</code> bit specifies the frame sync source.
		0 CLKIN0 pin selected for frame sync
		1 <code>PCG_EXTX_DAI0</code> selected for frame sync
29:20 (R/W)	FSPHASELO	Phase for Frame Sync Low. The <code>PCG_CTLA1 . FSPHASELO</code> bit field represents the lower half of the 20-bit value for the channel A/B/C/D frame sync phase.
19:0 (R/W)	CLKDIV	Clock Divisor. The <code>PCG_CTLA1 . CLKDIV</code> bit field contains the clock divisor value.

Precision Clock B Control 0 Register

The `PCG_CTLB0` register enables the clock, frame sync, and select divisor for the PCG0 clock B signal.

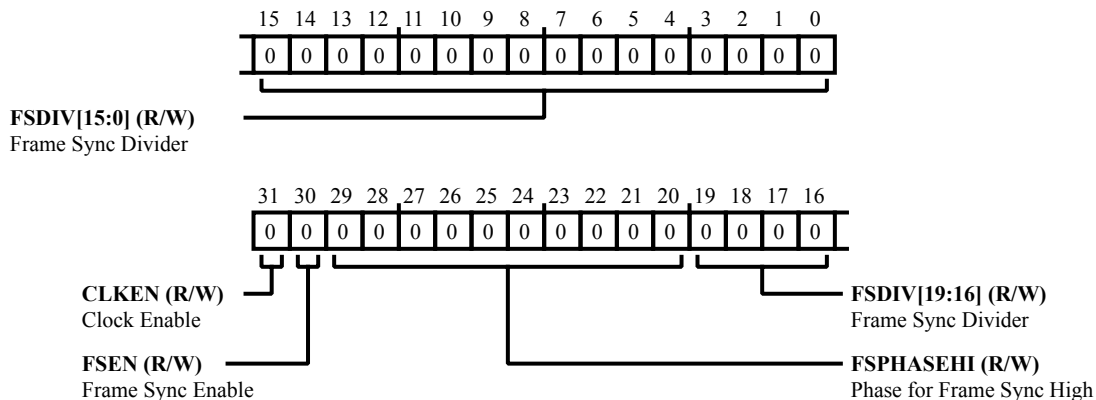


Figure 22-9: PCG_CTLB0 Register Diagram

Table 22-6: PCG_CTLB0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	CLKEN	Clock Enable. The <code>PCG_CTLB0.CLKEN</code> bit enables the clock.
		0 Clock generation disabled
		1 Clock generation enabled
30 (R/W)	FSEN	Frame Sync Enable. The <code>PCG_CTLB0.FSEN</code> bit enables the frame sync.
		0 Frame sync generation disabled
		1 Frame sync generation enabled
29:20 (R/W)	FSPHASEHI	Phase for Frame Sync High. The <code>PCG_CTLB0.FSPHASEHI</code> bit field represents the upper half of the 20-bit value for the channel A/B/C/D frame sync phase.
19:0 (R/W)	FSDIV	Frame Sync Divider. The <code>PCG_CTLB0.FSDIV</code> bit field provides the frame sync divider value. This 20-bit field frame sync divider is multiplexed where: <code>PCG_CTLB0.FSDIV > 1</code> PCGx is in normal mode, <code>PCG_CTLB0.FSDIV = 0, 1</code> PCGx is in bypass mode.
		0 PCG is in bypass mode
		1 PCG is in bypass mode
		2-1048575 FSDIV > 1 PCG is in normal mode

Precision Clock B Control 1 Register

The `PCG_CTLB1` register sets the clock divisor, frame sync source, and clock source for the PCG1 clock B signal.

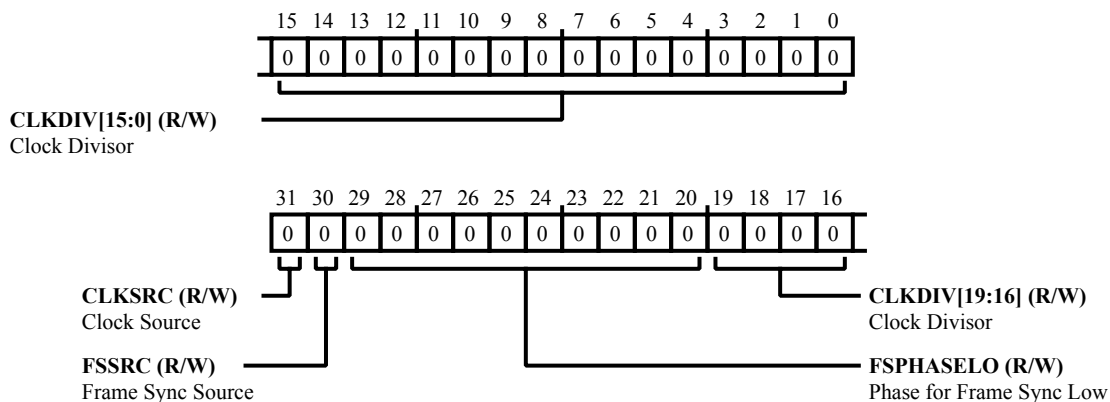


Figure 22-10: `PCG_CTLB1` Register Diagram

Table 22-7: `PCG_CTLB1` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	CLKSRC	Clock Source. The <code>PCG_CTLB1</code> . <code>CLKSRC</code> bit specifies the clock source.
		0 CLKIN0 pin selected for clock
		1 PCG_EXT_DAI0 selected for clock
30 (R/W)	FSSRC	Frame Sync Source. The <code>PCG_CTLB1</code> . <code>FSSRC</code> bit specifies the frame sync source.
		0 CLKIN0 pin selected for frame sync
		1 PCG_EXT_DAI0 selected for frame sync
29:20 (R/W)	FSPHASELO	Phase for Frame Sync Low. The <code>PCG_CTLB1</code> . <code>FSPHASELO</code> bit field represents the lower half of the 20-bit value for the channel A/B/C/D frame sync phase.
19:0 (R/W)	CLKDIV	Clock Divisor. The <code>PCG_CTLB1</code> . <code>CLKDIV</code> bit field contains the clock divisor value.

Precision Clock C Control 0 Register

The `PCG_CTLC0` register enables the clock, frame sync, and select divisor for the PCG0 clock C signal.

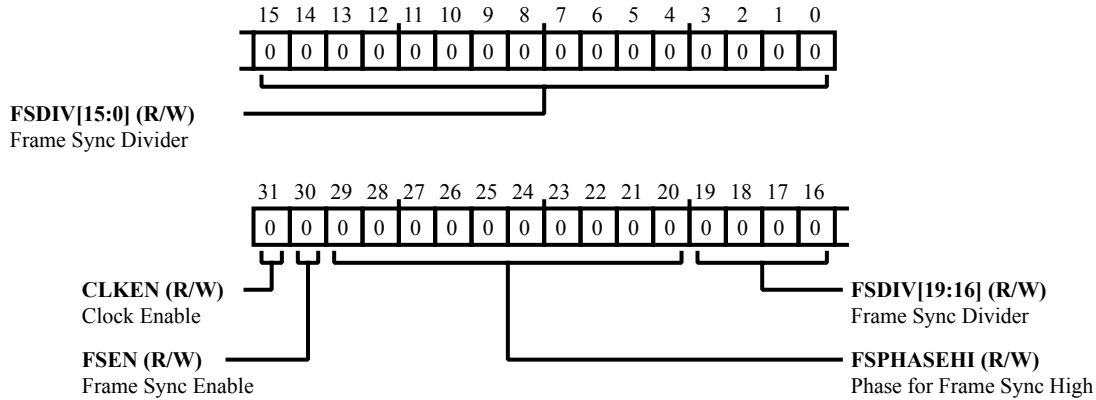


Figure 22-11: PCG_CTLC0 Register Diagram

Table 22-8: PCG_CTLC0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	CLKEN	Clock Enable. The <code>PCG_CTLC0.CLKEN</code> bit enables the clock.
		0 Clock generation disabled
		1 Clock generation enabled
30 (R/W)	FSEN	Frame Sync Enable. The <code>PCG_CTLC0.FSEN</code> bit enables the frame sync.
		0 Frame sync generation disabled
		1 Frame sync generation enabled
29:20 (R/W)	FSPHASEHI	Phase for Frame Sync High. The <code>PCG_CTLC0.FSPHASEHI</code> bit field represents the upper half of the 20-bit value for the channel A/B/C/D frame sync phase.
19:0 (R/W)	FSDIV	Frame Sync Divider. The <code>PCG_CTLC0.FSDIV</code> bit field provides the frame sync divider value. This 20-bit field frame sync divider is multiplexed where: <code>PCG_CTLC0.FSDIV > 1</code> PCGx is in normal mode, <code>PCG_CTLC0.FSDIV = 0, 1</code> PCGx is in bypass mode.
		0 PCG is in bypass mode
		1 PCG is in bypass mode
		2-1048575 FSDIV > 1 PCG is in normal mode

Precision Clock C Control 1 Register

The `PCG_CTL1` register sets the clock divisor, frame sync source, and clock source for the PCG1 clock C signal.

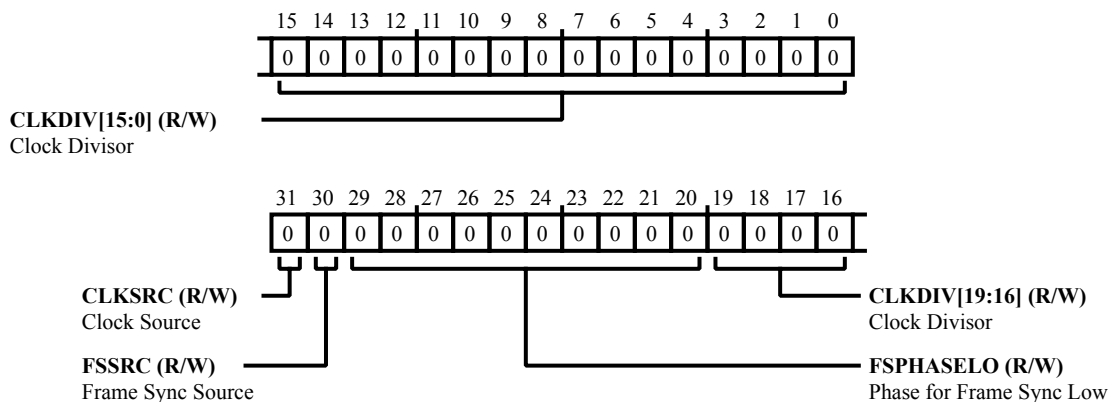


Figure 22-12: `PCG_CTL1` Register Diagram

Table 22-9: `PCG_CTL1` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	CLKSRC	Clock Source. The <code>PCG_CTL1</code> . <code>CLKSRC</code> bit specifies the clock source.
		0 CLKIN1 pin selected for clock
		1 PCG_EXT_DAI1 selected for clock
30 (R/W)	FSSRC	Frame Sync Source. The <code>PCG_CTL1</code> . <code>FSSRC</code> bit specifies the frame sync source.
		0 CLKIN1 pin selected for frame sync
		1 PCG_EXT_DAI1 selected for frame sync
29:20 (R/W)	FSPHASELO	Phase for Frame Sync Low. The <code>PCG_CTL1</code> . <code>FSPHASELO</code> bit field represents the lower half of the 20-bit value for the channel A/B/C/D frame sync phase.
19:0 (R/W)	CLKDIV	Clock Divisor. The <code>PCG_CTL1</code> . <code>CLKDIV</code> bit field contains the clock divisor value.

Precision Clock D Control 0 Register

The `PCG_CTLD0` register enables the clock, frame sync, and select divisor for the PCG0 clock D signal.

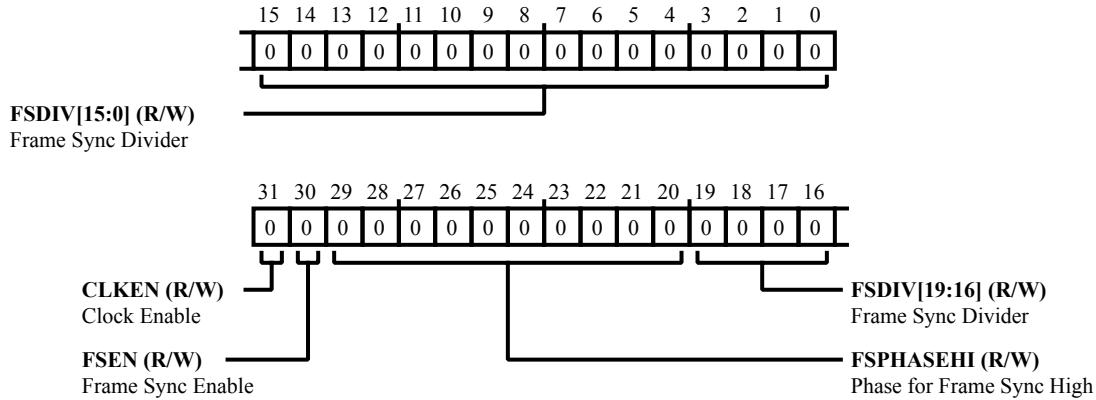


Figure 22-13: `PCG_CTLD0` Register Diagram

Table 22-10: `PCG_CTLD0` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	CLKEN	Clock Enable. The <code>PCG_CTLD0.CLKEN</code> bit enables the clock.
		0 Clock generation disabled
		1 Clock generation enabled
30 (R/W)	FSEN	Frame Sync Enable. The <code>PCG_CTLD0.FSEN</code> bit enables the frame sync.
		0 Frame sync generation disabled
		1 Frame sync generation enabled
29:20 (R/W)	FSPHASEHI	Phase for Frame Sync High. The <code>PCG_CTLD0.FSPHASEHI</code> bit field represents the upper half of the 20-bit value for the channel A/B/C/D frame sync phase.
19:0 (R/W)	FSDIV	Frame Sync Divider. The <code>PCG_CTLD0.FSDIV</code> bit field provides the frame sync divider value. This 20-bit field frame sync divider is multiplexed where: <code>PCG_CTLD0.FSDIV > 1</code> PCGx is in normal mode, <code>PCG_CTLD0.FSDIV = 0, 1</code> PCGx is in bypass mode.
		0 PCG is in bypass mode
		1 PCG is in bypass mode
		2-1048575 FSDIV > 1 PCG is in normal mode

Precision Clock D Control 1 Register

The `PCG_CTLD1` register sets the clock divisor, frame sync source, and clock source for the PCG1 clock D signal.

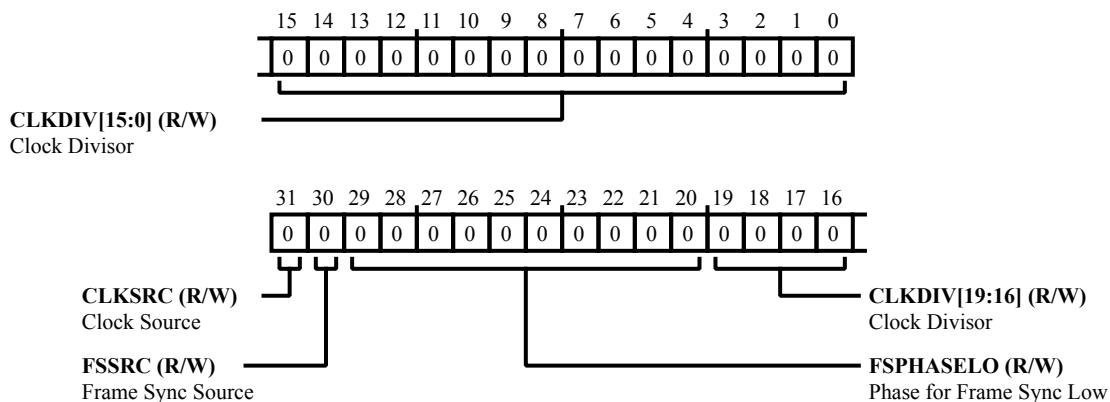


Figure 22-14: PCG_CTLD1 Register Diagram

Table 22-11: PCG_CTLD1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	CLKSRC	Clock Source. The <code>PCG_CTLD1</code> . <code>CLKSRC</code> bit specifies the clock source.
		0 CLKIN1 pin selected for clock
		1 PCG_EXT_DAI1 selected for clock
30 (R/W)	FSSRC	Frame Sync Source. The <code>PCG_CTLD1</code> . <code>FSSRC</code> bit specifies the frame sync source.
		0 CLKIN1 pin selected for frame sync
		1 PCG_EXT_DAI1 selected for frame sync
29:20 (R/W)	FSPHASELO	Phase for Frame Sync Low. The <code>PCG_CTLD1</code> . <code>FSPHASELO</code> bit field represents the lower half of the 20-bit value for the channel A/B/C/D frame sync phase.
19:0 (R/W)	CLKDIV	Clock Divisor. The <code>PCG_CTLD1</code> . <code>CLKDIV</code> bit field contains the clock divisor value.

Precision Clock Pulse Width Control 1 Register

The `PCG_PW1` register sets the one shot frame sync and the active low frame sync select for PCG A and PCG B.

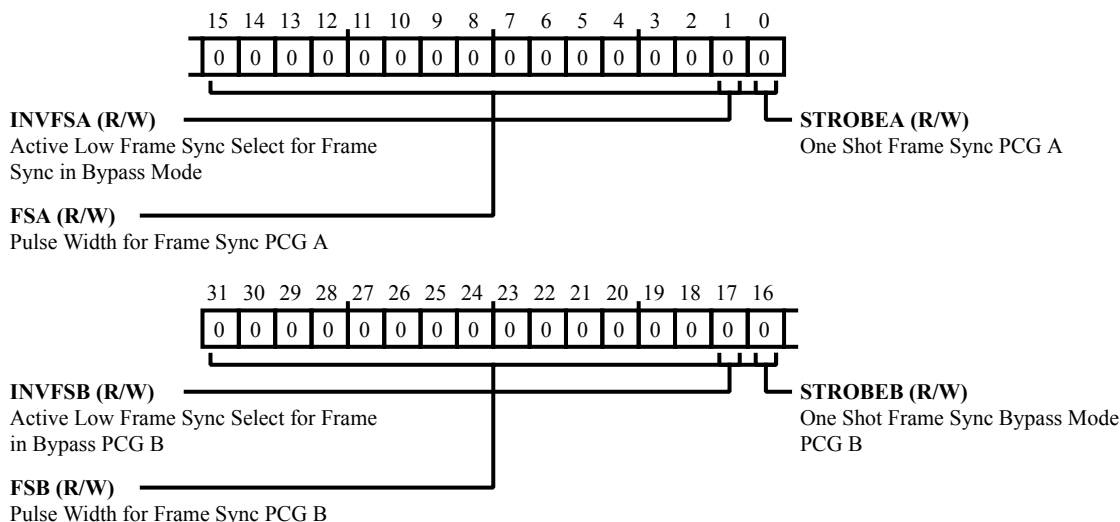


Figure 22-15: `PCG_PW1` Register Diagram

Table 22-12: `PCG_PW1` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
17 (R/W)	INVFSB	Active Low Frame Sync Select for Frame in Bypass PCG B. The <code>PCG_PW1</code> . INVFSB bit selects active low or active high frame sync in bypass mode for PCG B.
16 (R/W)	STROBEB	One Shot Frame Sync Bypass Mode PCG B. The <code>PCG_PW1</code> . STROBEB bit sets the frame sync pulse in bypass mode for PCG B. This is the duration equal to one period of the DAI_MISCA3_I signal (PCG B) repeating at the beginning of every frame.
31:16 (R/W)	FSB	Pulse Width for Frame Sync PCG B. The <code>PCG_PW1</code> . FSB bit field sets the number of input clock periods for which the frame sync output is high. Pulse width should be less than the divisor of the frame sync.
1 (R/W)	INVFSB	Active Low Frame Sync Select for Frame Sync in Bypass Mode. The <code>PCG_PW1</code> . INVFSB bit selects active low or active high frame sync for PCG A in bypass mode.
15:0 (R/W)	FSA	Pulse Width for Frame Sync PCG A. The <code>PCG_PW1</code> . FSA bit field sets the number of input clock periods for which the frame sync output is high. Pulse width should be less than the divisor of the frame sync.

Table 22-12: PCG_PW1 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	STROBEA	One Shot Frame Sync PCG A. The PCG_PW1 . STROBEA bit sets the frame sync pulse for PCG A in bypass mode. This is the duration equal to one period of the DAI_MISCA2_I signal (PCG A) repeating at the beginning of every frame.

Precision Clock Pulse Width Control 2 Register

The `PCG_PW2` register sets the one shot frame sync and the active low frame sync select for PCG C and PCG D.

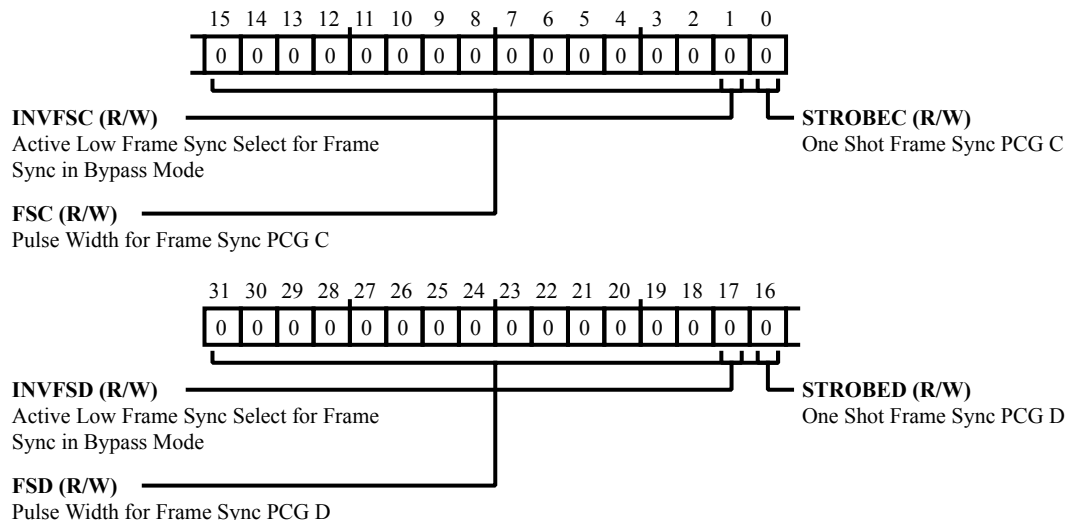


Figure 22-16: `PCG_PW2` Register Diagram

Table 22-13: `PCG_PW2` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
17 (R/W)	INVFS	Active Low Frame Sync Select for Frame Sync in Bypass Mode. The <code>PCG_PW2</code> . <code>INVFS</code> bit selects active low or active high frame sync for PCG D in bypass mode.
16 (R/W)	STROBED	One Shot Frame Sync PCG D. The <code>PCG_PW2</code> . <code>STROBED</code> bit sets the frame sync pulse for PCG D in bypass mode. This is the duration equal to one period of the <code>DAI_MISCA3_I</code> signal (PCG D) repeating at the beginning of every frame.
31:16 (R/W)	FSD	Pulse Width for Frame Sync PCG D. The <code>PCG_PW2</code> . <code>FSD</code> bit field sets the number of input clock periods for which the frame sync output is high for PCG D. Pulse width should be less than the divisor of the frame sync.
1 (R/W)	INVFS	Active Low Frame Sync Select for Frame Sync in Bypass Mode. The <code>PCG_PW2</code> . <code>INVFS</code> bit selects active low or active high frame sync for PCG C in bypass mode.
15:0 (R/W)	FSC	Pulse Width for Frame Sync PCG C. The <code>PCG_PW2</code> . <code>FSC</code> bit field sets the number of input clock periods for which the frame sync output is high for PCG C. Pulse width should be less than the divisor of the frame sync.

Table 22-13: PCG_PW2 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	STROBEC	One Shot Frame Sync PCG C. The PCG_PW2 . STROBEC bit sets the frame sync pulse for PCG C in bypass mode. This is the duration equal to one period of the DAI_MISCA2_I signal (PCG C) repeating at the beginning of every frame.

Precision Clock Frame Sync Synchronization 1 Register

The `PCG_SYNC1` register allows programs to synchronize the clock frame sync units with external frame syncs. Note that the `PCG_CTLA1.CLKSRC` bit is overridden if `PCG_SYNC1.CLKASRC` bit in the `PCG_SYNC1` register is set.

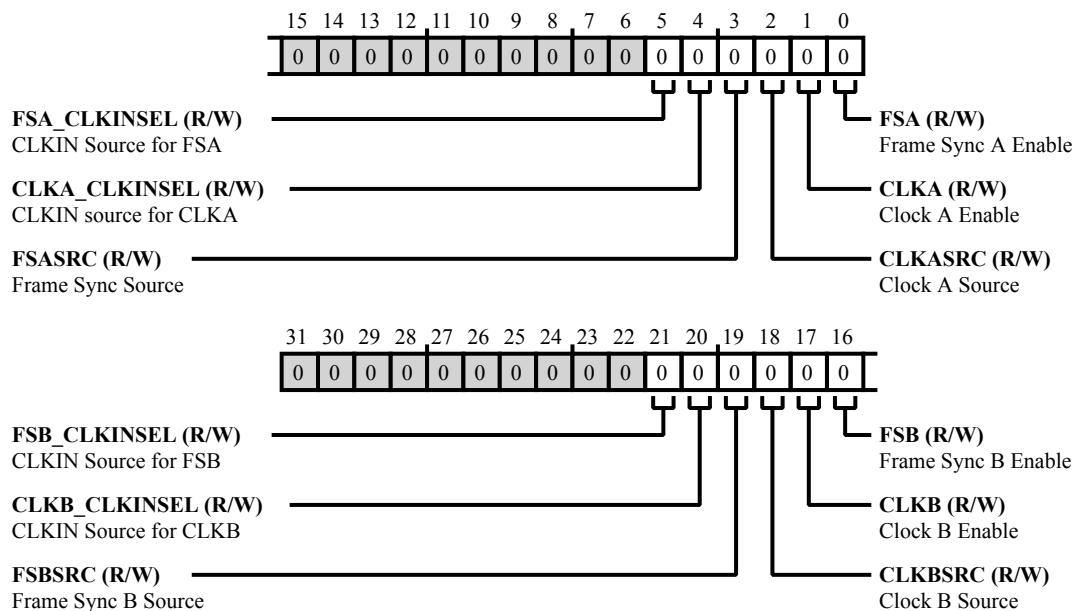


Figure 22-17: PCG_SYNC1 Register Diagram

Table 22-14: PCG_SYNC1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
21 (R/W)	FSB_CLKINSEL	CLKIN Source for FSB. The <code>PCG_SYNC1.FSB_CLKINSEL</code> bit enables the CLKIN input source for FSB.
		0 CLKIN0
		1 CLKIN1
20 (R/W)	CLKB_CLKINSEL	CLKIN Source for CLKB. The <code>PCG_SYNC1.CLKB_CLKINSEL</code> bit enables the CLKIN input source for CLKB.
		0 CLKIN0
		1 CLKIN1

Table 22-14: PCG_SYNC1 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
19 (R/W)	FSBSRC	Frame Sync B Source. The PCG_SYNC1 . FSBSRC bit enables the frame sync B input source.
		0 Output selected by FSBSOURCE bit
		1 Clock derived from core PLL selected for frame sync B
18 (R/W)	CLKBSRC	Clock B Source. The PCG_SYNC1 . CLKBSRC bit enables the clock B input source.
		0 Output selected by CLKBSOURCE bit
		1 Clock derived from core PLL selected for clock B
17 (R/W)	CLKB	Clock B Enable. The PCG_SYNC1 . CLKB bit enables synchronization of clock B with the external frame sync.
		0 Clock disabled
		1 Clock enabled
16 (R/W)	FSB	Frame Sync B Enable. The PCG_SYNC1 . FSB bit enables synchronization of frame sync B with the external frame sync.
		0 Frame sync disabled
		1 Frame sync enabled
5 (R/W)	FSA_CLKINSEL	CLKIN Source for FSA. The PCG_SYNC1 . FSA_CLKINSEL bit enables the CLKIN input source for FSA.
		0 CLKIN0
		1 CLKIN1
4 (R/W)	CLKA_CLKINSEL	CLKIN source for CLKA. The PCG_SYNC1 . CLKA_CLKINSEL bit enables the CLKIN input source for CLKA
		0 Select CLKIN0 source for CLKA CLKIN
		1 Select CLKIN0 source for CLKA CLKIN
3 (R/W)	FSASRC	Frame Sync Source. The PCG_SYNC1 . FSASRC bit enables the frame sync A input source.
		0 Output selected by FSASOURCE bit
		1 Clock derived from core PLL selected for frame sync A

Table 22-14: PCG_SYNC1 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
2 (R/W)	CLKASRC	Clock A Source. The PCG_SYNC1 . CLKASRC bit enables the clock A input source.
		0 Output selected by CLKASOURCE bit
		1 Clock derived from core PLL selected for clock A
1 (R/W)	CLKA	Clock A Enable. The PCG_SYNC1 . CLKA bit enables synchronization of clock A with the external frame sync.
		0 Clock disabled
		1 Clock enabled
0 (R/W)	FSA	Frame Sync A Enable. The PCG_SYNC1 . FSA bit enables synchronization of frame sync A with the external frame sync.
		0 Frame Sync Disabled
		1 Frame Sync Enabled

Precision Clock Frame Sync Synchronization 2 Register

The `PCG_SYNC2` register allows programs to synchronize the clock frame sync units with external frame syncs. Note that the `PCG_CTLD1.CLKSRC` bit is overridden if `PCG_SYNC2.CLKDSRC` bit in the `PCG_SYNC2` register is set.

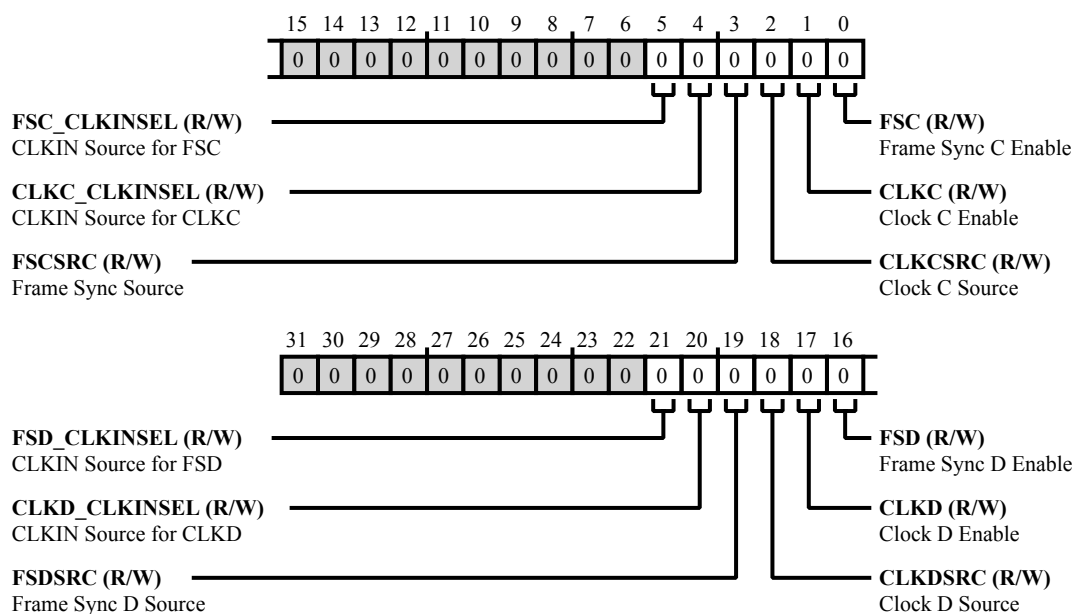


Figure 22-18: `PCG_SYNC2` Register Diagram

Table 22-15: `PCG_SYNC2` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
21 (R/W)	FSD_CLKINSEL	CLKIN Source for FSD. The <code>PCG_SYNC2.FSD_CLKINSEL</code> bit enables the CLKIN input source for FSD.
		0 Selects CLKIN0 source for FSB CLKIN
		1 Selects CLKIN0 source for FSB CLKIN
20 (R/W)	CLKD_CLKINSEL	CLKIN Source for CLKD. The <code>PCG_SYNC2.CLKD_CLKINSEL</code> bit enables the CLKIN input source for CLKD.
		0 Select CLKIN0 source for CLKB CLKIN
		1 Select CLKIN0 source for CLKB CLKIN

Table 22-15: PCG_SYNC2 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
19 (R/W)	FSDSRC	Frame Sync D Source. The PCG_SYNC2.FSDSRC bit enables the frame sync D input source.
		0 Output selected by FSDSOURCE bit
		1 Clock derived from core PLL selected for frame sync D
18 (R/W)	CLKDSRC	Clock D Source. The PCG_SYNC2.CLKDSRC bit enables the clock D input source.
		0 Output selected by CLKDSOURCE bit
		1 Clock derived from core PLL selected for clock D
17 (R/W)	CLKD	Clock D Enable. The PCG_SYNC2.CLKD bit enables synchronization of clock D with the external frame sync.
		0 Clock disabled
		1 Clock enabled
16 (R/W)	FSD	Frame Sync D Enable. The PCG_SYNC2.FSD bit enables synchronization of frame sync D with the external frame sync.
		0 Frame sync disabled
		1 Frame sync enabled
5 (R/W)	FSC_CLKINSEL	CLKIN Source for FSC. The PCG_SYNC2.FSC_CLKINSEL bit enables the CLKIN input source for FSC.
		0 Selects CLKIN0 as source for FSA CLKIN
		1 Selects CLKIN0 as source for FSA CLKIN
4 (R/W)	CLKC_CLKINSEL	CLKIN Source for CLKC. The PCG_SYNC2.CLKC_CLKINSEL bit enables the CLKIN input source for CLKC.
		0 Select CLKIN0 source for CLKA CLKIN
		1 Select CLKIN0 source for CLKA CLKIN
3 (R/W)	FSCSRC	Frame Sync Source. The PCG_SYNC2.FSCSRC bit enables the frame sync C input source.
		0 Output selected by FSCSOURCE bit
		1 Clock derived from core PLL selected for frame sync C

Table 22-15: PCG_SYNC2 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
2 (R/W)	CLKSRC	Clock C Source. The PCG_SYNC2.CLKSRC bit enables the clock C input source.
		0 Output selected by CLKCSOURCE bit
		1 Clock derived from core PLL selected for clock C
1 (R/W)	CLKC	Clock C Enable. The PCG_SYNC2.CLKC bit enables synchronization of clock C with the external frame sync.
		0 Clock disabled
		1 Clock enabled
0 (R/W)	FSC	Frame Sync C Enable. The PCG_SYNC2.FSC bit enables synchronization of frame sync C with the external frame sync.
		0 Frame sync disabled
		1 Frame sync enabled

23 Asynchronous Sample Rate Converter (ASRC)

Sample rate converters (SRC) are frequently used in digital signal processing audio applications. The most frequently used sample rate conversions are off-loaded into hardware modules that are dedicated for filter processing and reduce the instruction processing load on the core, freeing it up for other tasks.

Features

The ASRC has the following features and capabilities:

- 4 asynchronous stereo SRCs operating in completer mode are available in each DAI.
- Simple programming model
- Controllable muting options (hardware, software and automatic)
- Automatically senses input and output sample frequencies
- Supports left-justified, I²S, right-justified (16-, 18-, 20-, 24-bits), and TDM serial port modes
- Daisy-chain configuration in TDM modes (including between DAI0 and DAI1) for input and output ports to create a serial frame
- Different protocols on input/output port allow format conversions
- De-emphasis filter for 32, 44.1 and 48 kHz sampling frequencies
- Up to 192 kHz sample rate input/output continuous sample ratios from 7.5:1 to 1:8
- Group delay (latency of interpolation filter) is 16 samples
- SNR from 128 to 140 dB (depending on processor model)
- Matched phase mode available to compensate for group delays
- Can be used to de-jitter clocks in systems

Functional Description

Conceptually, the sample rate converter interpolates the serial input data at a rate of 220 and samples the interpolated data stream by the output sample rate. In practice, a 64-tap FIR filter with 220 polyphases, a FIFO, a digital servo loop that measures the time difference between the input and output samples within 5 ps, and a digital circuit to track the sample rate ratio are used to perform the interpolation and output sampling.

ADSP-21560_21561_21564_21568 ASRC Register List

Sample Rate Converter Module

Table 23-1: ADSP-21560_21561_21564_21568 ASRC Register List

Name	Description
ASRC_CTL01	Control Register for ASRC 0 and 1
ASRC_CTL23	Control Register for ASRC 2 and 3
ASRC_MUTE	Mute Register
ASRC_RAT01	Ratio Register for ASRC 0 and 1
ASRC_RAT23	Ratio Register for ASRC 2 and 3

ASRC Interrupt List

The ASRC interrupts are controlled through the DAI.

ASRC Block Diagram

The *ASRC Block Diagram* figure shows a top level block diagram of the ASRC module and the *Core Architecture* figure shows architecture details.

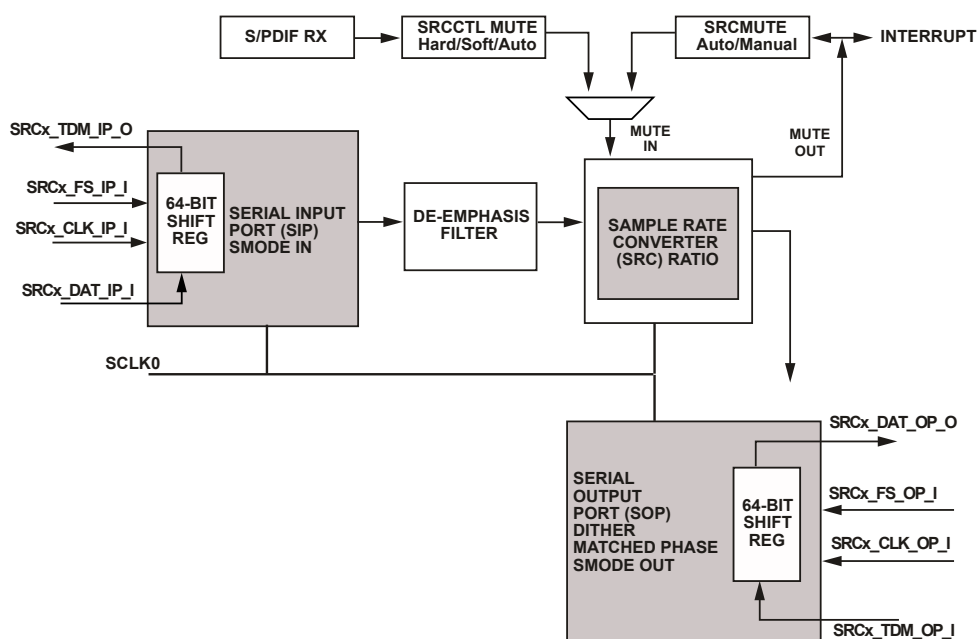


Figure 23-1: ASRC Block Diagram

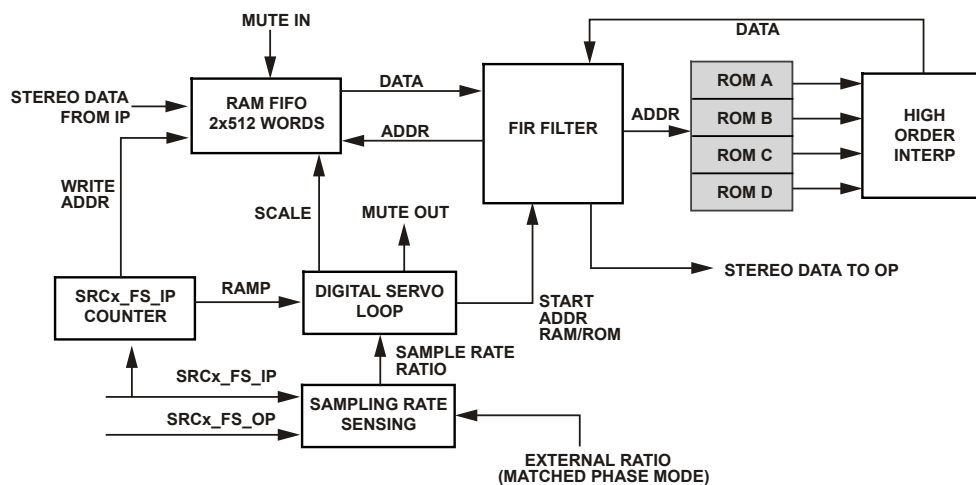


Figure 23-2: ASRC Core Architecture

SRU Programming

The SRU (signal routing unit) needs to be programmed to connect the ASRCs to the output pins or any other peripherals. For more information, see the [Digital Audio Interface \(DAI\)](#) chapter.

Clocking

The ASRC module is in the SCLK0 clock domain. An internal divided version of the SCLK0 clock is generated and used as the fundamental clock for the ASRC module.

I/O Ports

The I/O ports provide the interface through which data is transferred asynchronously into and out of the SRC modules. The SRC has a 3-wire interface for the serial input and output ports that supports left-justified, I²S, and right-justified (16-, 18-, 20-, 24-bit) modes. Additionally, the serial interfaces support TDM mode for daisy-chaining multiple SRCs to form a frame. The serial output data is dithered down to 20, 18, or 16 bits when 20-, 18-, or 16-bit output data is selected.

NOTE: The SRC converts the data from the serial input port to the sample rate of the serial output port. The sample rate at the serial input port can be asynchronous with respect to the output sample rate of the output serial port.

De-Emphasis Filter

The de-emphasis filter is used to de-emphasize audio data that has been emphasized.

Mute Control

When either the SRC starts up (or there is a change in sample ratio), the mute out signal (SRCx_MUTEOUT) is asserted (=1). The mute out signal stays high until the SRC settles on the new sample rates. While mute out is asserted high, the mute in signal should be asserted high as well. The mute in signal performs a soft mute of the audio input data when asserted and un-mutes the input audio data softly when deasserted.

Note that it takes 4096 input port FS samples until the audio input data is completely muted and 4096 FS samples until the audio input data is completely unmuted.

SRC Core

As shown in the *ASRC Core Architecture* figure, the sample rate converter's RAM FIFO block adjusts the left and right input samples and stores them for the FIR filter's convolution cycle. The ASRCx_FS_IP counter provides the write address (for scaling) to the FIFO block and the ramp input to the digital-servo loop. The ROM stores the coefficients for the FIR filter convolution and performs a high-order interpolation between the stored coefficients. The sample rate ratio block measures the sample rate by dynamically altering the ROM coefficients and scaling the FIR filter length and input data. The digital-servo loop automatically tracks the SRCx_FS_IP and SRCx_FS_OP sample rates and provides the RAM and ROM start addresses for the start of the FIR filter convolution.

NOTE: Unlike other peripherals, the sample rate converters own local memories (RAM and ROM) which are dedicated for the purpose of sample rate conversion only.

The sample rate converter only operates asynchronously and is always a completer to the input and output ports.

RAM FIFO

The RAM FIFO receives the left and right input data and adjusts the amplitude of the data for both the soft muting of the SRC and the scaling of the input data by the sample rate ratio before storing the samples in RAM. The input data is scaled by the sample rate ratio because as the FIR filter length of the convolution increases, so does the

amplitude of the convolution output. To keep the output of the FIR filter from saturating, the input data is scaled down by multiplying it by $(\text{SRCx_FS_OP})/(\text{SRCx_FS_IP})$ when $\text{SRCx_FS_OP} < \text{SRCx_FS_IP}$. The FIFO also scales the input data to mute and stop muting the SRC.

Digital Servo Loop

The digital-servo loop is a ramp filter that provides the initial pointer to the address in RAM and ROM for the start of the FIR convolution. The RAM pointer is the integer output of the ramp filter while the ROM pointer is the fractional part. The digital-servo loop must be able to provide excellent rejection of jitter on the SRCx_FS_IP and SRCx_FS_OP clocks as well as measure the arrival of the SRCx_FS_OP clock within 5 ps. The digital-servo loop also divides the fractional part of the ramp output by the ratio of $(\text{SRCx_FS_IP})/(\text{SRCx_FS_OP})$ for the case when $\text{SRCx_FS_IP} > \text{SRCx_FS_OP}$, to dynamically alter the ROM coefficients.

The digital-servo loop is implemented with a multi-rate filter. To settle the digital-servo loop filter quickly at startup or at a change in the sample rate, a fast mode has been added to the filter. When the digital-servo loop starts up or the sample rate is changed, the digital-servo loop kicks into fast mode to adjust and settle on the new sample rate. Upon sensing the digital-servo loop settling down to some reasonable value, the digital-servo loop kicks into normal or slow mode. During fast mode, the SRCx_MUTE_OUT bit of the ASRC is asserted to mute the ASRC input which avoids clicks and pops.

FIR Filter

The FIR filter is a 64-tap filter in the case of $\text{SRCx_FS_OP} < \text{SRCx_FS_IP}$ and is $(\text{SRCx_FS_IP})/(\text{SRCx_FS_OP}) \times 64$ taps for the case when $\text{SRCx_FS_IP} > \text{SRCx_FS_OP}$. The FIR filter performs its convolution by loading in the starting address of the RAM address pointer and the ROM address pointer from the digital-servo loop at the start of the SRCx_FS_OP period. The FIR filter then steps through the RAM by decrementing its address by 1 for each tap, and the ROM pointer increments its address by the $(\text{SRCx_FS_OP}/\text{SRCx_FS_IP}) \times 2^{20}$ ratio for $\text{SRCx_FS_IP} > \text{SRCx_FS_OP}$ or 2^{20} for $\text{SRCx_FS_OP} < \text{SRCx_FS_IP}$. Once the ROM address rolls over, the convolution is complete. The convolution is performed for both the left and right channels, and the multiply/accumulate circuit used for the convolution is shared between the channels.

Sample Rate Sensing

The $(\text{SRCx_FS_IP})/(\text{SRCx_FS_OP})$ sample rate ratio circuit is used to dynamically alter the coefficients in the ROM for the case when $\text{SRCx_FS_IP} > \text{SRCx_FS_OP}$. The ratio is calculated by comparing the output of an SRCx_FS_OP counter to the output of an SRCx_FS_IP counter. If $\text{ASRCx_FS_OP} > \text{SRCx_FS_IP}$, the ratio is held at one. If $\text{SRCx_FS_IP} > \text{SRCx_FS_OP}$, the sample rate ratio is updated if it is different by more than two SRCx_FS_OP periods from the previous SRCx_FS_OP to SRCx_FS_IP comparison. This is done to provide some hysteresis to prevent the filter length from oscillating and causing distortion.

Digital Filter Group Delay

The RAM in the FIFO is 512 words deep for both left and right channels. An offset of 16 samples to the write address, provided by the SRCx_FS_IP counter, is added to prevent the RAM read pointer from overlapping the write address. The maximum decimation rate can be calculated from the RAM word: depth = $(512 - 16) \div 64$ taps = 7.75:1.

The 64 samples effect latency in the interpolation filter. This latency (group delay) depends on interpolation or decimation ratio and is determined as follows:

Interpolation or Decimation Ratio (1): $GDL = 16/f_{S_IN} + 32/f_{S_IN}$ seconds for $SRC_FS_OP > SRC_FS_IP$

Interpolation or Decimation Ratio (2): $GDL = 16/f_{S_IN} + 32/f_{S_IN} \times f_{S_IN}/f_{S_OUT}$ seconds for $SRC_FS_OP < SRC_FS_IP$

Data Format

The *ASRC Data Frame Format by Protocol* figure shows the data input format for a frame (stereo data). The frame format is valid for all protocols. For models which do not support matched phase mode the 8-bit data field is ignored.

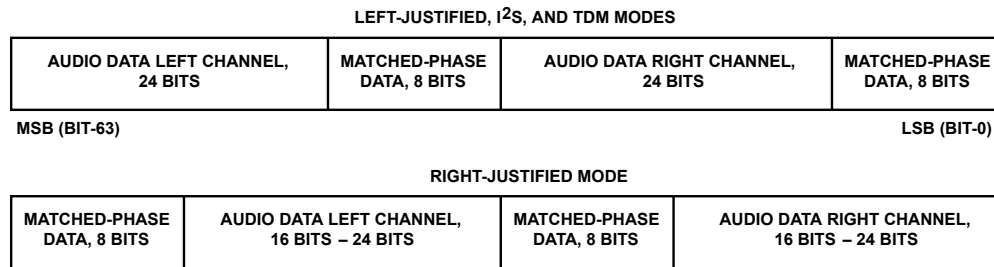


Figure 23-3: ASRC Data Frame Format by Protocol

Operating Modes

The ASRC can operate in TDM, I²S, left-justified, right-justified, and bypass modes. The serial ports of the processor can be used for moving the ASRC data to/from the internal memory.

In I²S, left-justified and right-justified modes, the ASRCs operate individually. The serial data provided in the input port is converted to the sample rate of the output port.

TDM Input Mode

In TDM input port, several ASRCs can be daisy-chained together and connected to the serial input port of a SHARC processor or other processor (see the *TDM Input/Output Modes* figure). The ASRC IP contains a 64-bit parallel load shift register. When the $SRCx_FS_IP_I$ pulse arrives, each ASRC parallel loads its left and right data into the 64-bit shift register. The input to the shift register is connected to $SRCx_DATA_IP_I$, while the output is connected to $SRCx_TDM_IP_O$. By connecting the $SRCx_TDM_IP_O$ to the $SRCx_DATA_IP_I$ of the next ASRC, a large shift register is created, which is clocked by $SRCx_CLK_IP_I$.

NOTE: In TDM mode, the ASRC drives at the rising edge and samples at the falling edge of the serial clock. In all other modes, the serial clock rising edge is the sampling edge, and the falling edge is the driving edge.

NOTE: The number of ASRCs that can be daisy-chained together is limited by the maximum frequency of $SRCx_CLK_xx_I$, refer to the data sheet for exact values. For example, if the maximum frequency of

$\text{SRC}_x_CLK_xx_I$ is x MHz, and the output sample rate is f_s , then number of ASRCs (n) that can be connected in daisy chained fashion is: $n \cdot 64 \text{ FS} \leq x \text{ MHz}$.

TDM Output Mode

As shown in the *TDM Input/Output Modes* figure, using the TDM output port several ASRCs can be daisy-chained together and connected to the SPORT of this or another processor. The ASRC OP contains a 64-bit parallel load shift register. When the $\text{SRC}_x_FS_OP_I$ pulse arrives, each ASRC loads its left and right data into the 64-bit shift register. The input to the shift register is connected to $\text{SRC}_x_TDM_OP_I$, and the output is connected to $\text{SRC}_x_DAT_OP_O$. By connecting the $\text{SRC}_x_DAT_OP_O$ to the $\text{SRC}_x_TDM_OP_I$ of the next ASRC, a large shift register is created, which is clocked by $\text{SRC}_x_CLK_OP_I$.

As shown in *TDM Input/Output Modes*, with three ASRCs in a daisy-chain connection, the serial clock for input/or output port is defined as: $\text{SCLK} = 3 \times 64 \text{ FS} = 192 \text{ FS}$.

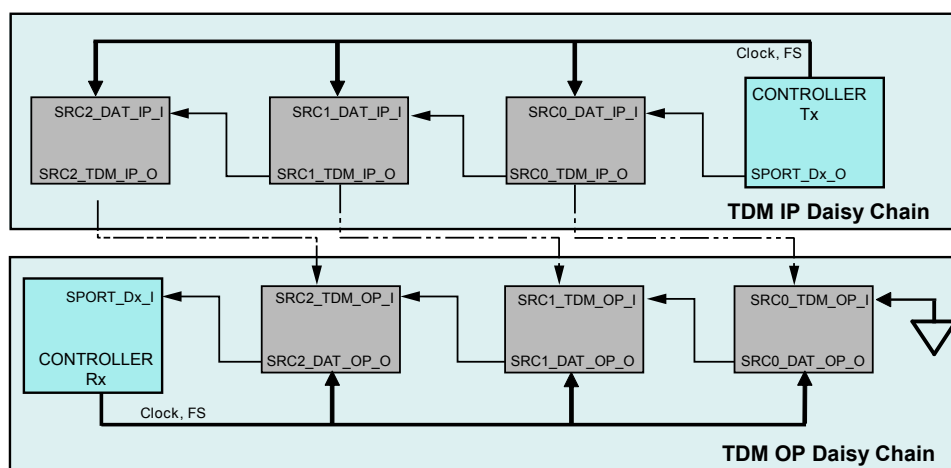


Figure 23-4: TDM Input/Output Modes

Matched-Phase Mode

The matched-phase mode of the sample rate converter, shown in *Typical Configuration for Matched-Phase Mode Operation*, is enabled by the $\text{ASRC_CTL01.MPHASE0}$, $\text{ASRC_CTL01.MPHASE1}$, $\text{ASRC_CTL23.MPHASE2}$ and $\text{ASRC_CTL23.MPHASE3}$ bits. This mode is used to match the phase (group delay) between two or more adjacent sample rate converters that are operating with the same input and output clocks.

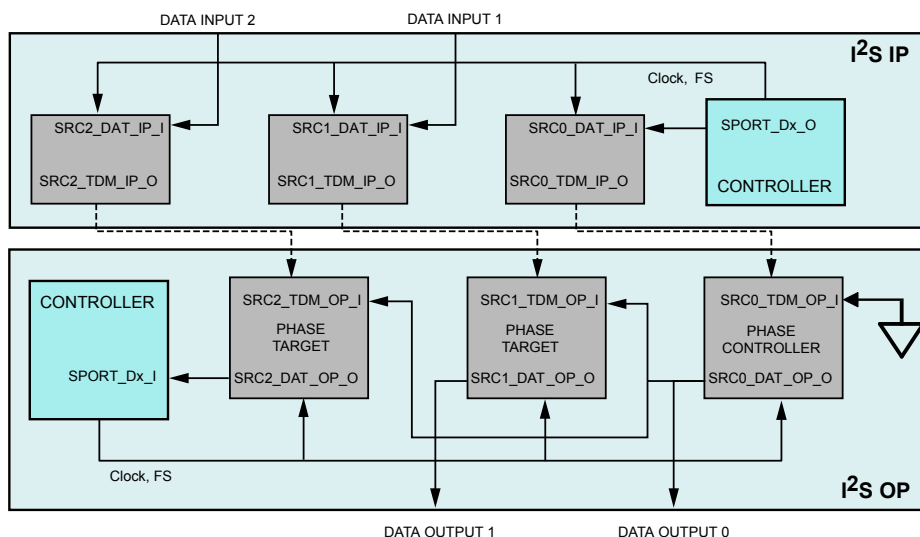


Figure 23-5: Typical Configuration for Matched-Phase Mode Operation

Hysteresis of the $(SRCx_FS_OP)/(SRCx_FS_IP)$ ratio circuit can cause phase mismatching between two ASRCs operating with the same input and output clocks. Since the hysteresis requires a difference of more than two $SRCx_FS_OP$ periods to update the $SRCx_FS_OP$ and $SRCx_FS_IP$ ratios, two ASRCs may have differences in their ratios from 0 to 4 $SRCx_FS_OP$ period counts. The $(SRCx_FS_OP)/(SRCx_FS_IP)$ ratio adjusts the filter length of the ASRC, which corresponds directly with the group delay. Thus, the magnitude in the phase difference depends upon the resolution of the $SRCx_FS_OP$ and $SRCx_FS_IP$ counters. The greater the resolution of the counters, the smaller the phase difference error.

When the completer SRC matched-phase mode bit is set (=1), it accepts the sample rate ratio transmitted by another SRC, (the matched-phase requester) which has its matched-phase mode bit cleared (=0), through its serial output.

The phase requester ASRC device transmits its $SRCx_FS_OP/SRCx_FS_IP$ ratio through the data output pin ($SRCx_DAT_OP_O$) to the completer's ASRC's data input pins ($SRCx_TDM_OP_I$). The transmitted data (32-bit subframe) contains 24-bit data and 8-bits matched phase (see the *ASRC Data Frame Format by Protocol* figure).

The completer SRCs receive the 8-bit matched phase bits (instead of their own internally-derived ratio) if their $SRCx_MPHASE$ bits are set to 1, respectively. The $SRCx_FS_IP$ and $SRCx_FS_OP$ signals may be asynchronous with respect to each other in this mode. Note that there must be 64 $SRCx_CLK_OP$ cycles per frame in matched-phase mode (two 24-bits data and two 8-bits phase match).

NOTE: By default, matched phased data is sent on the $SRCx_DAT_OP_O$ pin, but only if the $SRCx_TDM_OP_I$ pin is tied low. The completers' simply ignore the matched phased data if their $ASRC_CTL01.MPHASE1$ through $ASRC_CTL23.MPHASE3$ bits are cleared (= 0).

Bypass Mode

When the $ASRC_CTL01.BYP0$, $ASRC_CTL01.BYP1$, $ASRC_CTL23.BYP2$ and $ASRC_CTL23.BYP3$ bits are set (=1), the input data bypasses the sample rate converter and is sent directly to the serial output port. Dithering is disabled. This mode is ideal when the input and output sample rates are the same and the $SRCx_FS_IP_I$ and

`SRCx_FS_OP_I` signals are synchronous with respect to each other. In matched phase bypass mode, the `SRCx_FS_OP_I` signal should come at least one `SRCx_CLK_xx_I` period before `SRCx_FS_IP_I`. Cases where this is not met could result in data loss. For example, if internal SPORTS are used then the `SRCx_FS_OP_I` and `SRCx_FS_IP_I` signals could be driven by different SPORTs so that the timing of these signals can be controlled by enabling them at different times. This mode can also be used for passing through non-audio data since no processing is performed on the input data.

De-Emphasis Mode

The `ASRC_CTL01.DEEMPHASIS0`, `ASRC_CTL01.DEEMPHASIS1`, `ASRC_CTL23.DEEMPHASIS2` and `ASRC_CTL23.DEEMPHASIS3` bits choose the type of de-emphasis filter based on the input sample rate for 32, 44.1 or 48 kHz sampling rates.

Dithering Mode

The `ASRC_CTL01.DITHER0`, `ASRC_CTL01.DITHER1`, `ASRC_CTL23.DITHER2`, and `ASRC_CTL23.DITHER3` control this mode of operation. Serial output data is dithered down to 20, 18, or 16 bits when 20-, 18-, or 16-bit output data is selected. In the case of 20-, 18-, and 16-bit word lengths, the least significant bits of the 24-bit word coming from the SRC into the serial output port are truncated. The `DITHER_EN` signal (not user configurable) automatically adds dithering to the 24-bit word before truncating to the appropriate output word length. The `21BIT_DITHER` signal is used for the consumer version of the SRC to reduce the dynamic range performance to approximately 128 dB.

NOTE: The ASRC can be programmed to add the triangular Probability Distribution Function (PDF) dither to the digital audio samples. It is advisable to add dither when the input word width exceeds the output word width, for example the input word is 20 bits, and the output word is 16 bits. A triangular PDF is considered to create the most favorable noise shaping of the residual quantization noise.

Muting Modes

The mute feature of the ASRC can be controlled automatically in hardware using the `MUTE_IN` signal by connecting it to the `MUTE_OUT` signal. Automatic muting can be disabled by setting (=1) the `ASRC_MUTE.MUTE0` through `ASRC_MUTE.MUTE3` bits.

NOTE: Note that by default, the `ASRC_MUTE` register connects the `MUTE_IN` signal to the `MUTE_OUT` signal, but not conversely.

Soft Mute

When the `ASRC_CTL01.SOFTMUTE0`, `ASRC_CTL01.SOFTMUTE1`, `ASRC_CTL23.SOFTMUTE2` and `ASRC_CTL23.SOFTMUTE3` bits are set, the `MUTE_IN` signal is asserted, and the ASRC performs a soft mute by linearly decreasing the input data to the ASRC FIFO to zero, (-144 dB) attenuation as described for automatic hardware muting.

A 12-bit counter, clocked by `SRCx_FS_IP_I`, is used to control the mute attenuation. Therefore, the time it takes from the assertion of the `MUTE_IN` signal to -144 dB, full mute attenuation is 4096 FS cycles. Likewise, the time it takes to reach 0 dB mute attenuation from the deassertion of the `MUTE_IN` signal is 4096 FS cycles.

Hard Mute

When the `ASRC_CTL01.HARDMUTE0`, `ASRC_CTL01.HARDMUTE1`, `ASRC_CTL23.HARDMUTE2` and `ASRC_CTL23.HARDMUTE3` bits are set, the ASRC immediately mutes the input data to the ASRC FIFO to zero, (-144 dB) attenuation.

Auto Mute

When the `ASRC_CTL01.AUTOMUTE0`, `ASRC_CTL01.AUTOMUTE1`, `ASRC_CTL23.AUTOMUTE2` and `ASRC_CTL23.AUTOMUTE3` bits are set, the ASRC communicates with the S/PDIF receiver peripheral to determine when the input should mute.

This mode is useful for automatic detection of non-PCM audio data received from the S/PDIF receiver.

Interrupts

The following sections provide information about interrupt sources, masking, and servicing.

Sources

Each ASRC module drives one interrupt signal (mute out asserted). All these signals are connected into the `DAI_IRPTL_H` or `DAI_IRPTL_L` latch registers. The ASRC ports generate interrupts as described below.

SRC Mute Out

The SRC mute out signal can be used to generate interrupts on their rising edge, falling edge, or both, depending on how the DAI interrupt mask registers (`DAI_IMSK_RE/DAI_IMSK_FE`) are programmed. This programming allows the generation of `DAI_IRPTL_H/DAI_IRPTL_L` interrupts either entering mute, exiting muting or both. The `SRCx_MUTE_OUT` interrupt is generated only once when the SRC is locked (after 4096 FS input samples) and after changes to the sample ratio. Hard mute, soft mute, and auto mute only control the muting of the input data to the SRC.

Masking

The `DAI_IMSK_FE`, `DAI_IMSK_RE`, and `DAI_IMSK_PRI` registers must be unmasked accordingly. The `DAI_IRQH` and `DAI_IRQL` signals are routed to the system event controller (SEC).

Service

The ISR reads the `DAI_IRPTL_H` and `DAI_IRPTL_L` registers to clear the interrupt request.

Programming Model

The following is basic information on programming the ASRC module.

1. Program the [ASRC_CTL01](#) and [ASRC_CTL23](#) registers and keep the `ASRC_CTL01.EN0` through `ASRC_CTL23.EN3` bits cleared.
2. Set the `ASRC_CTL01.EN0` through `ASRC_CTL23.EN3` bits. After 4096 input port FS cycles the ASRC has un-muted.

Debug Features

The asynchronous sample rate converter allow the bypass mode. When the `ASRC_CTL01.BYP0` through `ASRC_CTL23.BYP3` bits are set (=1), the input data bypasses the sample rate converter and is sent directly to the serial output port. This mode can be used for testing both ports when the input and output sample rates are at the same frequency, therefore both input and output ports can be routed to the same serial clock and frame sync.

ADSP-21560_21561_21564_21568 ASRC Register Descriptions

Sample Rate Converter Module (ASRC) contains the following registers.

Table 23-2: ADSP-21560_21561_21564_21568 ASRC Register List

Name	Description
ASRC_CTL01	Control Register for ASRC 0 and 1
ASRC_CTL23	Control Register for ASRC 2 and 3
ASRC_MUTE	Mute Register
ASRC_RAT01	Ratio Register for ASRC 0 and 1
ASRC_RAT23	Ratio Register for ASRC 2 and 3

Control Register for ASRC 0 and 1

The `ASRC_CTL01` register (read/write) controls the operating modes, filters, and data formats used in the ASRC modules 0 and 1.

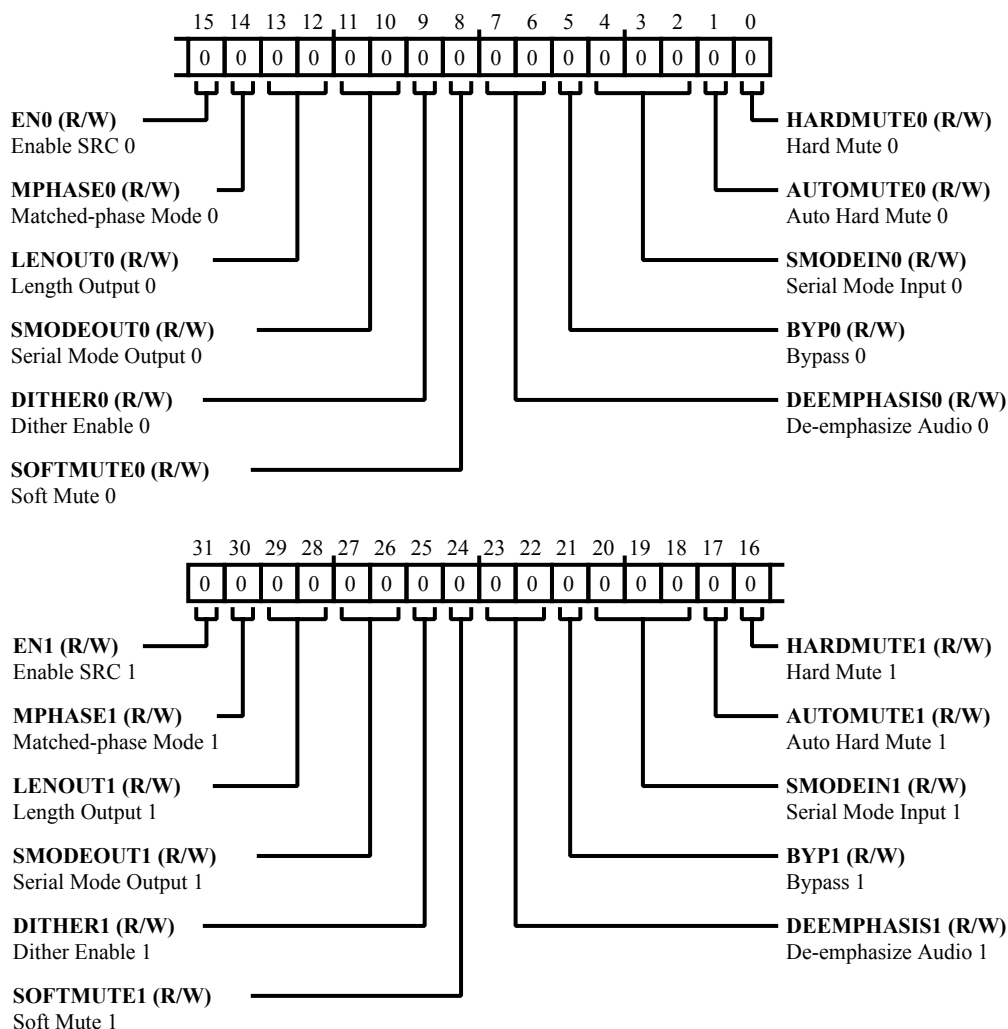


Figure 23-6: `ASRC_CTL01` Register Diagram

Table 23-3: ASRC_CTL01 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration	
31 (R/W)	EN1	Enable SRC 1.	
		The ASRC_CTL01.EN1 bit enables SRC 1. When (set = 1), or when the sample rate (frame sync) between the input and output changes, the SRC begins its initialization routine where; 1) MUTE_OUT is asserted, 2) soft mute control counter for input samples is set to maximum attenuation (144 dB).	
		Note that SRC power-up completion is finished by clearing the ASRC_RAT01.MUTEOUT1 bit.	
		Writes to the ASRC_CTL01 register should be at least one cycle before setting the ASRC_CTL01.EN1 bit. When setting and clearing this bit, it should be held low for a minimum of 5 SCLK cycles.	
30 (R/W)	MPHASE1	Matched-phase Mode 1.	
		The ASRC_CTL01.MPHASE1 bit configures SRC1 to not use its own internally-generated sample rate ratio but use an externally-generated ratio. Used with TDM data.	
		0	Matched phase completer disabled
	1	Matched phase completer enabled	
29:28 (R/W)	LENOUT1	Length Output 1.	
		The ASRC_CTL01.LENOUT1 bit field selects the serial output word length on SRC1.	
		0	24 bits
		1	20 bits
		2	18 bits
	3	16 bits	
27:26 (R/W)	SMODEOUT1	Serial Mode Output 1.	
		The ASRC_CTL01.SMODEOUT1 bit field selects the serial output format on SRC1.	
		0	Left-justified
		1	I2S
		2	TDM
	3	Right-justified	
25 (R/W)	DITHER1	Dither Enable 1.	
		The ASRC_CTL01.DITHER1 bit enables dithering before truncation on SRC1 when a word length less than 24 bits is selected.	
		0	Truncation only
	1	Dithering before truncation	

Table 23-3: ASRC_CTL01 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
24 (R/W)	SOFTMUTE1	Soft Mute 1. The ASRC_CTL01 . SOFTMUTE1 bit enables soft mute on SRC1.
		0 Unmute
		1 Mute
23:22 (R/W)	DEEMPHASIS1	De-emphasize Audio 1. The ASRC_CTL01 . DEEMPHASIS1 bits are used to de-emphasize audio data that has been emphasized. The type of de-emphasis filter is based on the input sample rate (SRCx_FS_IP_I signal).
		0 No de-emphasis
		1 32 kHz
		2 44.1 kHz
		3 48 kHz
21 (R/W)	BYP1	Bypass 1. The ASRC_CTL01 . BYP1 bit makes the output of SRC1 the same as the input.
20:18 (R/W)	SMODEIN1	Serial Mode Input 1. The ASRC_CTL01 . SMODEIN1 bit field selects the serial input format for SRC1.
		0 left-justified
		1 I2S
		2 TDM
		4 24-bit right-justified
		5 20-bit right-justified
		6 18-bit right-justified
		7 16-bit right-justified
17 (R/W)	AUTOMUTE1	Auto Hard Mute 1. The ASRC_CTL01 . AUTOMUTE1 bit auto hard mutes SRC1 when non audio is asserted by the SPDIF receiver.
		0 Unmute
		1 Mute
16 (R/W)	HARDMUTE1	Hard Mute 1. The ASRC_CTL01 . HARDMUTE1 bit hard mutes SRC1.
		0 Unmute
		1 Mute

Table 23-3: ASRC_CTL01 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration									
15 (R/W)	EN0	Enable SRC 0. The ASRC_CTL01.EN0 bit enables SRC 0. When (set =1), or when the sample rate (frame sync) between the input and output changes, the SRC begins its initialization routine where; 1) MUTE_OUT is asserted, 2) soft mute control counter for input samples is set to maximum attenuation (144 dB). Note that SRC power-up completion is finished by clearing the ASRC_RATE01.MUTEOUT0 bit. Writes to the ASRC_CTL01 register should be at least one cycle before setting the ASRC_CTL01.EN0 bit. When setting and clearing this bit, it should be held low for a minimum of 5 CLK cycles.									
14 (R/W)	MPHASE0	Matched-phase Mode 0. The ASRC_CTL01.MPHASE0 bit configures SRC0 to not use its own internally-generated sample rate ratio but use an externally-generated ratio. Used with TDM data. <table><tr><td>0</td><td>Matched phase completer disabled</td></tr><tr><td>1</td><td>Matched phase completer enabled</td></tr></table>		0	Matched phase completer disabled	1	Matched phase completer enabled				
0	Matched phase completer disabled										
1	Matched phase completer enabled										
13:12 (R/W)	LENOUT0	Length Output 0. The ASRC_CTL01.LENOUT0 bit field selects the serial output word length on SRC0. <table><tr><td>0</td><td>24 bits</td></tr><tr><td>1</td><td>20 bits</td></tr><tr><td>2</td><td>18 bits</td></tr><tr><td>3</td><td>16 bits</td></tr></table>		0	24 bits	1	20 bits	2	18 bits	3	16 bits
0	24 bits										
1	20 bits										
2	18 bits										
3	16 bits										
11:10 (R/W)	SMODEOUT0	Serial Mode Output 0. The ASRC_CTL01.SMODEOUT0 bit field selects the serial output format on SRC0. <table><tr><td>0</td><td>Left-justified</td></tr><tr><td>1</td><td>I2S</td></tr><tr><td>2</td><td>TDM</td></tr><tr><td>3</td><td>Right-justified</td></tr></table>		0	Left-justified	1	I2S	2	TDM	3	Right-justified
0	Left-justified										
1	I2S										
2	TDM										
3	Right-justified										
9 (R/W)	DITHER0	Dither Enable 0. The ASRC_CTL01.DITHER0 bit enables dithering before truncation on SRC0 when a word length less than 24 bits is selected. <table><tr><td>0</td><td>Truncation only</td></tr><tr><td>1</td><td>Dithering before truncation</td></tr></table>		0	Truncation only	1	Dithering before truncation				
0	Truncation only										
1	Dithering before truncation										

Table 23-3: ASRC_CTL01 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
8 (R/W)	SOFTMUTE0	Soft Mute 0. The ASRC_CTL01 . SOFTMUTE0 bit enables soft mute on SRC0.
		0 Unmute
		1 Mute
7:6 (R/W)	DEEMPHASIS0	De-emphasize Audio 0. The ASRC_CTL01 . DEEMPHASIS0 bits are used to de-emphasize audio data that has been emphasized. The type of de-emphasis filter is based on the input sample rate (SRCx_FS_IP_I signal).
		0 No de-emphasis
		1 32 kHz
		2 44.1 kHz
		3 48 kHz
5 (R/W)	BYP0	Bypass 0. The ASRC_CTL01 . BYP0 bit makes the output of SRC0 the same as the input.
4:2 (R/W)	SMODEIN0	Serial Mode Input 0. The ASRC_CTL01 . SMODEIN0 bit field selects the serial input format for SRC0.
		0 left-justified
		1 I2S
		2 TDM
		4 24-bit right-justified
		5 20-bit right-justified
		6 18-bit right-justified
		7 16-bit right-justified
1 (R/W)	AUTOMUTE0	Auto Hard Mute 0. The ASRC_CTL01 . AUTOMUTE0 bit auto hard mutes SRC0 when non audio is asserted by the SPDIF receiver.
		0 Unmute
		1 Mute
0 (R/W)	HARDMUTE0	Hard Mute 0. The ASRC_CTL01 . HARDMUTE0 bit hard mutes SRC0.
		0 Unmute
		1 Mute (default)

Control Register for ASRC 2 and 3

The [ASRC_CTL23](#) register (read/write) controls the operating modes, filters, and data formats used in the sample rate converter modules 2 and 3.

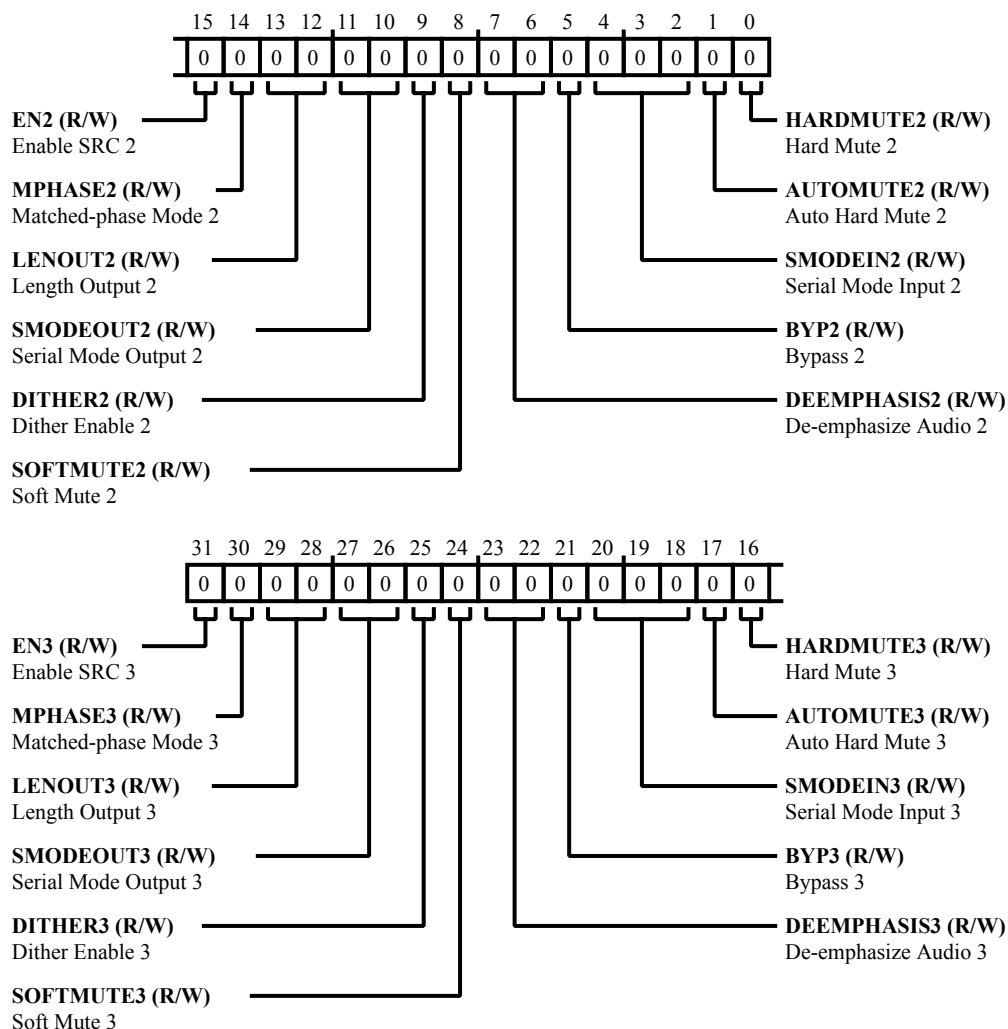


Figure 23-7: ASRC_CTL23 Register Diagram

Table 23-4: ASRC_CTL23 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration								
31 (R/W)	EN3	Enable SRC 3. The ASRC_CTL23.EN3 bit enables SRC 3. When (set =1), or when the sample rate (frame sync) between the input and output changes, the SRC begins its initialization routine where; 1) MUTE_OUT is asserted, 2) soft mute control counter for input samples is set to maximum attenuation (144 dB). Note that SRC power-up completion is finished by clearing the ASRC_RAT23.MUTEOUT3 bit. Writes to the ASRC_CTL23 register should be at least one cycle before setting the ASRC_CTL23.EN3 bit. When setting and clearing this bit, it should be held low for a minimum of 5 CLK cycles.								
30 (R/W)	MPHASE3	Matched-phase Mode 3. The ASRC_CTL23.MPHASE3 bit configures SRC3 to not use its own internally-generated sample rate ratio but use an externally-generated ratio. Used with TDM data. <table><tr><td>0</td><td>Matched phase completer disabled</td></tr><tr><td>1</td><td>Matched phase completer enabled</td></tr></table>	0	Matched phase completer disabled	1	Matched phase completer enabled				
0	Matched phase completer disabled									
1	Matched phase completer enabled									
29:28 (R/W)	LENOUT3	Length Output 3. The ASRC_CTL23.LENOUT3 bit field selects the serial output word length on SRC3. <table><tr><td>0</td><td>24 bits</td></tr><tr><td>1</td><td>20 bits</td></tr><tr><td>2</td><td>18 bits</td></tr><tr><td>3</td><td>16 bits</td></tr></table>	0	24 bits	1	20 bits	2	18 bits	3	16 bits
0	24 bits									
1	20 bits									
2	18 bits									
3	16 bits									
27:26 (R/W)	SMODEOUT3	Serial Mode Output 3. The ASRC_CTL23.SMODEOUT3 bit field selects the serial output format on SRC3. <table><tr><td>0</td><td>Left-justified</td></tr><tr><td>1</td><td>I2S</td></tr><tr><td>2</td><td>TDM</td></tr><tr><td>3</td><td>Right-justified</td></tr></table>	0	Left-justified	1	I2S	2	TDM	3	Right-justified
0	Left-justified									
1	I2S									
2	TDM									
3	Right-justified									
25 (R/W)	DITHER3	Dither Enable 3. The ASRC_CTL23.DITHER3 bit enables dithering before truncation on SRC3 when a word length less than 24 bits is selected. <table><tr><td>0</td><td>Truncation only</td></tr><tr><td>1</td><td>Dithering before truncation</td></tr></table>	0	Truncation only	1	Dithering before truncation				
0	Truncation only									
1	Dithering before truncation									

Table 23-4: ASRC_CTL23 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
24 (R/W)	SOFTMUTE3	Soft Mute 3. The ASRC_CTL23 . SOFTMUTE3 bit enables soft mute on SRC3.
		0 Unmute
		1 Mute
23:22 (R/W)	DEEMPHASIS3	De-emphasize Audio 3. The ASRC_CTL23 . DEEMPHASIS3 bits are used to de-emphasize audio data that has been emphasized. The type of de-emphasis filter is based on the input sample rate (SRCx_FS_IP_I signal).
		0 No de-emphasis
		1 32 kHz
		2 44.1 kHz
		3 48 kHz
21 (R/W)	BYP3	Bypass 3. The ASRC_CTL23 . BYP3 bit makes the output of SRC3 the same as the input.
20:18 (R/W)	SMODEIN3	Serial Mode Input 3. The ASRC_CTL23 . SMODEIN3 bit field selects the serial input format for SRC3.
		0 left-justified
		1 I2S
		2 TDM
		4 24-bit right-justified
		5 20-bit right-justified
		6 18-bit right-justified
		7 16-bit right-justified
17 (R/W)	AUTOMUTE3	Auto Hard Mute 3. The ASRC_CTL23 . AUTOMUTE3 bit auto hard mutes SRC3 when non audio is asserted by the SPDIF receiver.
		0 Unmute
		1 Mute
16 (R/W)	HARDMUTE3	Hard Mute 3. The ASRC_CTL23 . HARDMUTE3 bit hard mutes SRC3.
		0 Unmute
		1 Mute

Table 23-4: ASRC_CTL23 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
15 (R/W)	EN2	Enable SRC 2. The ASRC_CTL23.EN2 bit enables SRC 2. When (set =1), or when the sample rate (frame sync) between the input and output changes, the SRC begins its initialization routine where; 1) MUTE_OUT is asserted, 2) soft mute control counter for input samples is set to maximum attenuation (144 dB). Note that SRC power-up completion is finished by clearing the ASRC_RATE23.MUTEOUT2 bit. Writes to the ASRC_CTL23 register should be at least one cycle before setting the ASRC_CTL23.EN2 bit. When setting and clearing this bit, it should be held low for a minimum of 5 CLK cycles.	
14 (R/W)	MPHASE2	Matched-phase Mode 2. The ASRC_CTL23.MPHASE2 bit configures SRC2 to not use its own internally-generated sample rate ratio but use an externally-generated ratio. Used with TDM data.	
		0	Matched phase completer disabled
		1	Matched phase completer enabled
13:12 (R/W)	LENOUT2	Length Output 2. The ASRC_CTL23.LENOUT2 bit field selects the serial output word length on SRC2.	
		0	24 bits
		1	20 bits
		2	18 bits
		3	16 bits
11:10 (R/W)	SMODEOUT2	Serial Mode Output 2. The ASRC_CTL23.SMODEOUT2 bit field selects the serial output format on SRC2.	
		0	Left-justified
		1	I2S
		2	TDM
		3	Right-justified
9 (R/W)	DITHER2	Dither Enable 2. The ASRC_CTL23.DITHER2 bit enables dithering before truncation on SRC2 when a word length less than 24 bits is selected.	
		0	Truncation only
		1	Dithering before truncation

Table 23-4: ASRC_CTL23 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
8 (R/W)	SOFTMUTE2	Soft Mute 2. The ASRC_CTL23.SOFTMUTE2 bit enables soft mute on SRC2.
		0 Unmute
		1 Mute
7:6 (R/W)	DEEMPHASIS2	De-emphasize Audio 2. The ASRC_CTL23.DEEMPHASIS2 bits are used to de-emphasize audio data that has been emphasized. The type of de-emphasis filter is based on the input sample rate (SRCx_FS_IP_I signal).
		0 No de-emphasis
		1 32 kHz
		2 44.1 kHz
		3 48 kHz
5 (R/W)	BYP2	Bypass 2. The ASRC_CTL23.BYP2 bit makes the output of SRC2 the same as the input.
4:2 (R/W)	SMODEIN2	Serial Mode Input 2. The ASRC_CTL23.SMODEIN2 bit field selects the serial input format for SRC2.
		0 left-justified
		1 I2S
		2 TDM
		4 24-bit right-justified
		5 20-bit right-justified
		6 18-bit right-justified
		7 16-bit right-justified
1 (R/W)	AUTOMUTE2	Auto Hard Mute 2. The ASRC_CTL23.AUTOMUTE2 bit auto hard mutes SRC2 when non audio is asserted by the SPDIF receiver.
		0 Unmute
		1 Mute
0 (R/W)	HARDMUTE2	Hard Mute 2. The ASRC_CTL23.HARDMUTE2 bit hard mutes SRC2.
		0 Unmute
		1 Mute

Mute Register

This register connects an ASRCx mute input and output when the mute bit is cleared (=0). This allows ASRCx to automatically mute input while the ASRC is initializing (0 = automatic muting and 1 = manual muting). Bit 0 controls ASRC0, bit 1 controls ASRC1, bit 2 controls ASRC2, and bit 3 controls ASRC3.

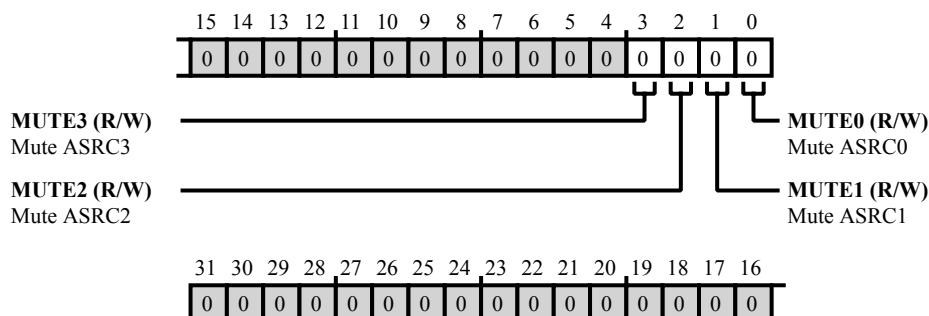


Figure 23-8: ASRC_MUTE Register Diagram

Table 23-5: ASRC_MUTE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3 (R/W)	MUTE3	Mute ASRC3. The ASRC_MUTE.MUTE3 bit automatically mutes ASRC3 output when cleared (=0).
2 (R/W)	MUTE2	Mute ASRC2. The ASRC_MUTE.MUTE2 bit automatically mutes ASRC2 output when cleared (=0).
1 (R/W)	MUTE1	Mute ASRC1. The ASRC_MUTE.MUTE1 bit automatically mutes ASRC1 output when cleared (=0).
0 (R/W)	MUTE0	Mute ASRC0. The ASRC_MUTE.MUTE0 bit automatically mutes ASRC0 output when cleared (=0).

Ratio Register for ASRC 0 and 1

The `ASRC_RAT01` register reports the mute and I/O sample ratio for ASRC0 and ASRC1.

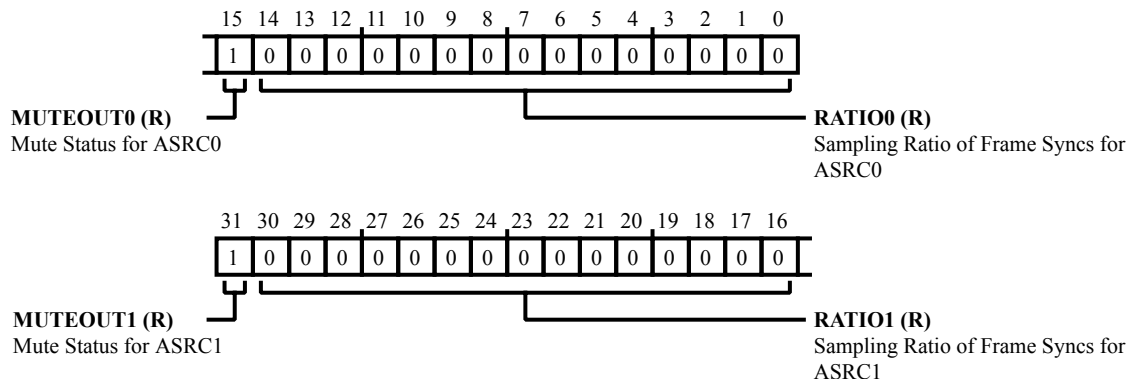


Figure 23-9: ASRC_RAT01 Register Diagram

Table 23-6: ASRC_RAT01 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/NW)	MUTEOUT1	Mute Status for ASRC1. The <code>ASRC_RAT01.MUTEOUT1</code> bit field reports the status of the <code>MUTE_OUT</code> signal. Once the <code>SRCx_MUTEOUT</code> signal is cleared, the ratio can be read. When ASRC1 is enabled or there is a change in the sample ratio, the <code>MUTE_OUT</code> signal is asserted. The <code>MUTE_OUT</code> signal remains asserted until the digital servo loops internal fast settling mode is complete. When the digital servo loop has switched to slow settling mode, the <code>MUTE_OUT</code> signal is deasserted.
30:16 (R/NW)	RATIO1	Sampling Ratio of Frame Syncs for ASRC1. The <code>ASRC_RAT01.RATIO1</code> bit field is read to find the ratio of output to input sampling frequency for ASRC1 (<code>SRCx_FS_OP_I/SRCx_FS_IP_I</code>). This ratio is reported in 4.11 (integer.fraction) format where the 15-bit value of the normal binary number is comprised of 4 bits for the integer and 11 bits for the fraction.
15 (R/NW)	MUTEOUT0	Mute Status for ASRC0. The <code>ASRC_RAT01.MUTEOUT0</code> bit field reports the status of the <code>MUTE_OUT</code> signal. Once the <code>SRCx_MUTEOUT</code> signal is cleared, the ratio can be read. When ASRC0 is enabled or there is a change in the sample ratio, the <code>MUTE_OUT</code> signal is asserted. The <code>MUTE_OUT</code> signal remains asserted until the digital servo loops internal fast settling mode is complete. When the digital servo loop has switched to slow settling mode, the <code>MUTE_OUT</code> signal is deasserted.

Table 23-6: ASRC_RAT01 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
14:0 (R/NW)	RATIO0	Sampling Ratio of Frame Syncs for ASRC0. The ASRC_RAT01 . RATIO0 bit field is read to find the ratio of output to input sampling frequency for ASRC0 (SRCx_FS_OP_I/SRCx_FS_IP_I). This ratio is reported in 4.11 (integer.fraction) format where the 15-bit value of the normal binary number is comprised of 4 bits for the integer and 11 bits for the fraction.

Ratio Register for ASRC 2 and 3

The `ASRC_RAT23` register reports the mute and I/O sample ratio for ASRC0 and ASRC1.

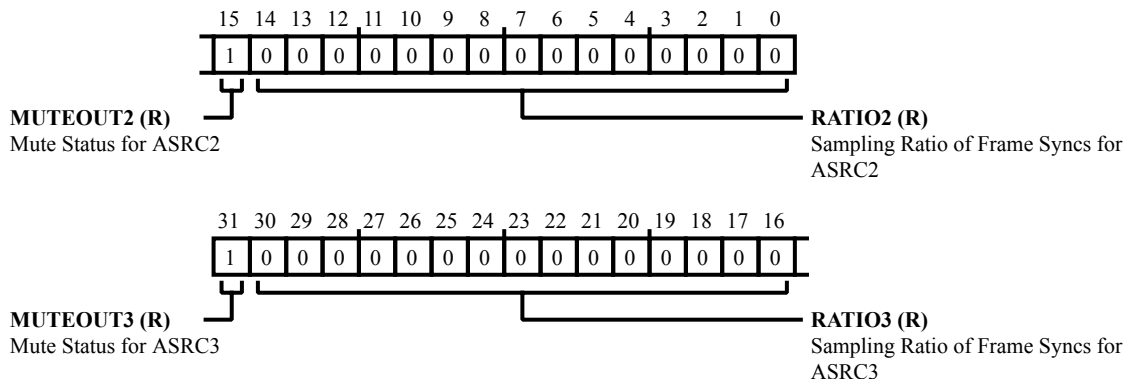


Figure 23-10: ASRC_RAT23 Register Diagram

Table 23-7: ASRC_RAT23 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/NW)	MUTEOUT3	Mute Status for ASRC3. The <code>ASRC_RAT23.MUTEOUT3</code> bit field reports the status of the <code>MUTE_OUT</code> signal. Once the <code>SRCx_MUTEOUT</code> signal is cleared, the ratio can be read. When ASRC3 is enabled or there is a change in the sample ratio, the <code>MUTE_OUT</code> signal is asserted. The <code>MUTE_OUT</code> signal remains asserted until the digital servo loops internal fast settling mode is complete. When the digital servo loop has switched to slow settling mode, the <code>MUTE_OUT</code> signal is deasserted.
30:16 (R/NW)	RATIO3	Sampling Ratio of Frame Syncs for ASRC3. The <code>ASRC_RAT23.RATIO3</code> bit field is read to find the ratio of output to input sampling frequency for ASRC3 (<code>SRCx_FS_OP_I/SRCx_FS_IP_I</code>). This ratio is reported in 4.11 (integer.fraction) format where the 15-bit value of the normal binary number is comprised of 4 bits for the integer and 11 bits for the fraction.
15 (R/NW)	MUTEOUT2	Mute Status for ASRC2. The <code>ASRC_RAT23.MUTEOUT2</code> bit field reports the status of the <code>MUTE_OUT</code> signal. Once the <code>SRCx_MUTEOUT</code> signal is cleared, the ratio can be read. When ASRC2 is enabled or there is a change in the sample ratio, the <code>MUTE_OUT</code> signal is asserted. The <code>MUTE_OUT</code> signal remains asserted until the digital servo loops internal fast settling mode is complete. When the digital servo loop has switched to slow settling mode, the <code>MUTE_OUT</code> signal is deasserted.

Table 23-7: ASRC_RAT23 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
14:0 (R/NW)	RATIO2	Sampling Ratio of Frame Syncs for ASRC2. The <code>ASRC_RAT23.RATIO2</code> bit field is read to find the ratio of output to input sampling frequency for ASRC2 (<code>SRCx_FS_OP_I/SRCx_FS_IP_I</code>). This ratio is reported in 4.11 (integer.fraction) format where the 15-bit value of the normal binary number is comprised of 4 bits for the integer and 11 bits for the fraction.

24 Sony/Philips Digital Interface (S/PDIF)

The Sony/Philips Digital Interface (S/PDIF) is a standard audio data transfer format that allows the transfer of digital audio signals from one device to another without having to convert them to an analog signal. The digital audio interface carries three types of information; audio data, non-audio data (compressed data), and timing information.

Features

The S/PDIF interface has the following features.

- Supports one stereo channel or compressed audio streams.
- AES3-compatible S/PDIF transmitter and receiver.
- Transmitting a biphase mark encoded signal that may contain any number of audio channels (compressed or linear pulse code modulation) or non-audio data.
- S/PDIF receiver managing clock recovery with separate S/PDIF on-chip PLL.
- S/PDIF receiver supports the detection of DTS frames of 256, 512, 1024, 2048, and 4096.
- Manage user status information and provide error-handling capabilities in both the transmitter and receiver.
- DAI allows interactions over DAI by serial ports and the external DAI pins to interface to other S/PDIF devices. This includes using the receiver to decode incoming biphase encoded audio streams and passing them via the SPORTs to internal memory for processing-or using the transmitter to encode audio or digital data and transfer it to another S/PDIF receiver in the audio system.

It is important to be familiar with serial digital audio interface standards IEC-60958, EIAJ CP-340, AES3 and AES11.

ADSP-21560_21561_21564_21568 SPDIF Register List

The S/PDIF module is a standard audio data transfer format that allows the transfer of digital audio signals from one device to another without having to convert them to an analog signal. A set of registers govern S/PDIF operations. For more information on S/PDIF functionality, see the S/PDIF register descriptions.

Table 24-1: ADSP-21560_21561_21564_21568 SPDIF Register List

Name	Description
SPDIF_RX_CTL	Receive Control
SPDIF_RX_STAT	Receive Status Register
SPDIF_RX_STAT0_A	Receive Status A0 Register
SPDIF_RX_STAT0_B	Receive Status B0 Register
SPDIF_RX_STAT1_A	Receive Status A1 Register
SPDIF_RX_STAT1_B	Receive Status B1 Register
SPDIF_TX_CTL	Transmit Control Register
SPDIF_TX_STAT_A0	Transmit Status A0 Register
SPDIF_TX_STAT_A1	Transmit Status A1 Register
SPDIF_TX_STAT_A2	Transmit Status A2 Register
SPDIF_TX_STAT_A3	Transmit Status A3 Register
SPDIF_TX_STAT_A4	Transmit Status A4 Register
SPDIF_TX_STAT_A5	Transmit Status A5 Register
SPDIF_TX_STAT_B0	Transmit Status B0 Register
SPDIF_TX_STAT_B1	Transmit Status B1 Register
SPDIF_TX_STAT_B2	Transmit Status B2 Register
SPDIF_TX_STAT_B3	Transmit Status B3 Register
SPDIF_TX_STAT_B4	Transmit Status B4 Register
SPDIF_TX_STAT_B5	Transmit Status B5 Register
SPDIF_TX_UBUFF_A0	Transmit User Buffer A0 Register
SPDIF_TX_UBUFF_A1	Transmit User Buffer A1 Register
SPDIF_TX_UBUFF_A2	Transmit User Buffer A2 Register
SPDIF_TX_UBUFF_A3	Transmit User Buffer A3 Register
SPDIF_TX_UBUFF_A4	Transmit User Buffer A4 Register
SPDIF_TX_UBUFF_A5	Transmit User Buffer A5 Register
SPDIF_TX_UBUFF_B0	Transmit User Buffer B0 Register
SPDIF_TX_UBUFF_B1	Transmit User Buffer B1 Register
SPDIF_TX_UBUFF_B2	Transmit User Buffer B2 Register
SPDIF_TX_UBUFF_B3	Transmit User Buffer B3 Register
SPDIF_TX_UBUFF_B4	Transmit User Buffer B4 Register
SPDIF_TX_UBUFF_B5	Transmit User Buffer B5 Register

Table 24-1: ADSP-21560_21561_21564_21568 SPDIF Register List (Continued)

Name	Description
SPDIF_TX_USRUPDT	User Bit Update Register

SRU Programming

The SRU (signal routing unit) is used to connect the S/PDIF transmitter biphasic data out to the output pins or to the S/PDIF receiver. The serial clock, frame sync, data, and external sync (if external synchronization is required) inputs also need to be routed through the SRU. For details of the routing, see [DAI Routing Capabilities](#) in the *Digital Audio Interface (DAI)* chapter.

The SRU needs to be programmed in order to connect the S/PDIF receiver to the output pins or any other peripherals and also for the connection to the input biphasic stream.

Program the corresponding SRU registers to connect the outputs to the required destinations (see [DAI Routing Capabilities](#) inputs to the receiver are routed through the SRU. The extracted clock, frame sync, and data are also routed through the SRU.

S/PDIF Interrupt List

Table 24-2: S/PDIF Interrupt List

Interrupt Name	Interrupt Condition	Return DAI Register	Return SEC Register	SEC ID
DAI0_IRQH	Block Start	DAIx_IRPTL	SEC_ID	24
DAI0_IRQL	Non Audio			145

Clocking

The fundamental timing clock of the S/PDIF receiver is SCLK1_0 from the CDU. When SCLK1_0 is configured, it supports sampling frequencies of 32 kHz to 192 kHz. The fundamental timing clock of the S/PDIF transmitter is *SCLK0/4*.

For information on clock programming, see [CDU Programming Model](#).

S/PDIF Transmitter

The following sections provide information on the S/PDIF transmitter.

Functional Description

The S/PDIF transmitter, shown in the *S/PDIF Transmitter Block Diagram*, resides within the DAI, and its inputs and outputs can be routed via the SRU. It receives audio data in serial format, encloses the specified user status information, and converts it into the biphasic encoded signal. The serial data input to the transmitter can be

formatted as left-justified, I²S, or right-justified with word widths of 16, 18, 20 or 24 bits. *AES3 Output Block* shows the detail of the AES block.

The serial data, clock, and frame sync inputs to the S/PDIF transmitter are routed through the signal routing unit (SRU).

The S/PDIF transmitter output may be routed to an output pin via the SRU and then routed to another S/PDIF receiver or to components for off-board connections to other S/PDIF receivers. The output is also available to the S/PDIF receiver for loop-back testing through SRU.

In addition to encoding the audio data in the bi-phase format, the transmitter also provides a way to easily add the channel status information to the outgoing bi-phase stream. There are status/user registers for a frame (192-bits/24 bytes) in the transmitter that correspond to each channel or subframe.

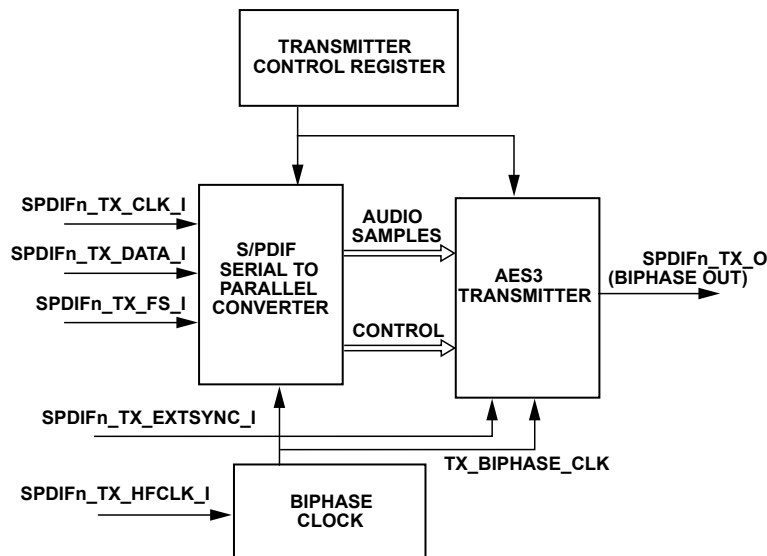


Figure 24-1: S/PDIF Transmitter Block Diagram

Validity bits for both channels may also be controlled by the transmitter control register. Optionally, the user bit, validity bit, and channel status bit are sent to the transmitter with each left/right sample. For each subframe the parity bit is automatically generated and inserted into the biphase encoded data.

A mute control and support for double-frequency single-channel mode are also provided. The serial data input format may be selected as left-justified, I²S, or right-justified with 16-, 18-, 20-, or 24-bit word widths. The over sampling clock is also selected by the transmitter control register.

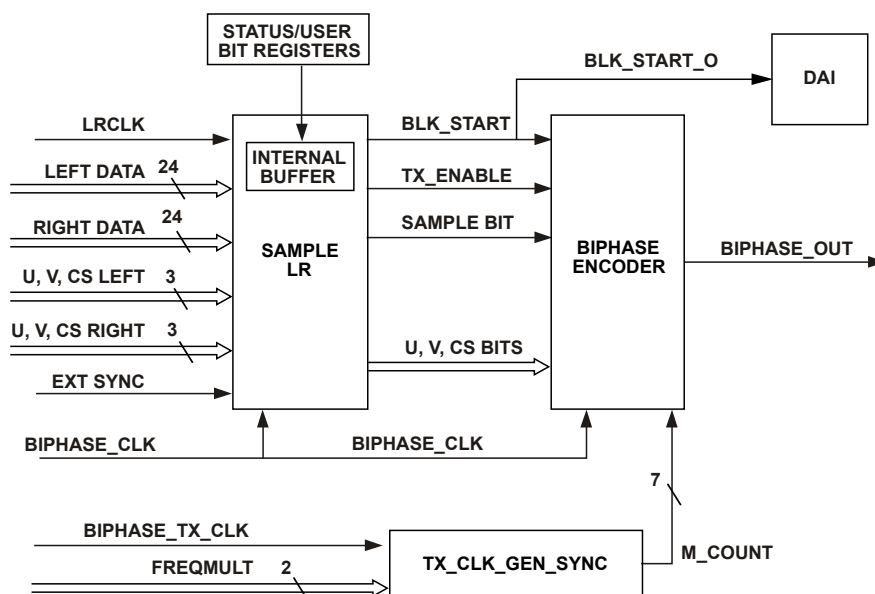
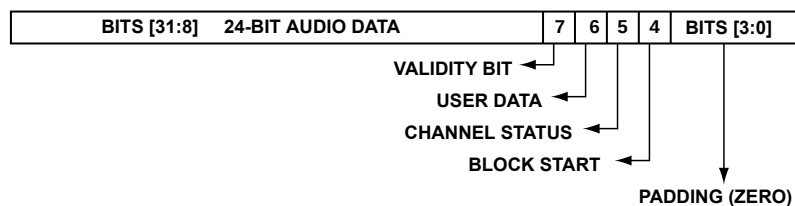


Figure 24-2: AES3 Output Block

Input Data Formats

The *I2S and Left-Justified Formats* and *Right-Justified Formats* figures show the format of data that is sent to the S/PDIF transmitter using a variety of protocol standards.

Figure 24-3: I²S and Left-Justified Formats

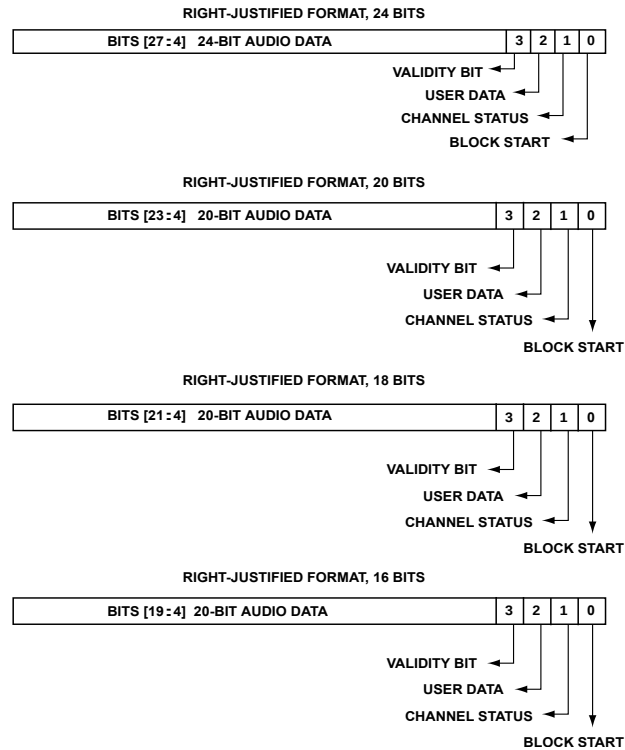


Figure 24-4: Right-Justified Formats

Operating Modes

The S/PDIF transmitter can operate in standalone and full serial modes. The following sections describe these modes in detail.

Full Serial Mode

This mode is selected by clearing the `SPDIF_TX_CTL.AUTO` bit. In this mode all the status bits, audio data and the block start bit (indicating start of a frame), come through the serial data stream (`SPDIF_TX_DATA_I`) pin. The transmitter should be enabled after or at the same time as all of the other control bits.

Standalone Mode

This mode is selected by setting the `SPDIF_TX_CTL.AUTO` bit. In this mode, the block start bit (indicating the start of a frame) is generated internally. The channel status bits come from the channel status buffer registers. The user status bits come from the user bits buffers as shown in the *AES3 Output Block* figure.

The validity bits are the `SPDIF_TX_CTL.VALIDR` and `SPDIF_TX_CTL.VALIDL`. In this mode only audio data comes from the `SPDIF_TX_DATA_I` pin. All other data, including the status bit and block start bit is either generated internally or taken from the internal register.

Once the user bits buffer registers (`SPDIF_TX_UBUFF_A0` - `SPDIF_TX_UBUFF_B5`) are programmed, they are used only for the next block of data. This allows programs to change the user bit information in every block of data.

To allow user bit updates, write a 0x1 to the `SPDIF_TX_USRUPDT` register that is used for further processing. If the `SPDIF_TX_CTL.AUTO` bit is set:

- and if `SPDIF_TX_USRUPDT` =1, at every 192nd frame end the user status bits are taken from user bits buffers and transmitted. Simultaneously, the `SPDIF_TX_USRUPDT` register is cleared automatically by hardware.
- and if `SPDIF_TX_USRUPDT` =0, at every 192nd frame end the user status bits are updated as zeros and transmitted. The `SPDIF_TX_USRUPDT` register remains low.

In general, for the next block, programs can update user bits buffers at any time during the transfer of the current block (1 block = 192 frames). There are internal buffers to store the user status bits of the current block of transfer. In other words, at the beginning of every new block, the user status bit (`SPDIF_TX_CTL.USRPEND` bit) from user bits buffers are copied to internal buffers and transmitted in each frame during the transfer.

Note that since a frame contains $192 \text{ bits} / 8 = 24 \text{ bytes}$, six status/user registers are required to store each four bytes.

Data Output Mode

Two output data formats are supported by the transmitter; *two channel mode* and *single-channel double-frequency* (SCDF) mode. The output format is determined by the transmitter control register (`SPDIF_TX_CTL`).

In two channel mode, the left channel (channel A) is transmitted when the `SPDIF_TX_FS_I` is high, and the right channel (channel B) is transmitted when the `SPDIF_TX_FS_I` is low.

In SCDF mode, the transmitter sends successive audio samples of the same signal across both sub frames, instead of channel A and B. The transmitter transmits at half the sample rate of the input bit stream. The `SPDIF_TX_CTL.SCDF` bit selects SCDF mode. When in SCDF mode, the `SPDIF_TX_CTL.SCDFLR` bit determines whether left or right channel data is transmitted.

S/PDIF Receiver

The S/PDIF receiver (*S/PDIF Receiver Block Diagram*) is compliant with all common serial digital audio interface standards including IEC-60958, IEC-61937, AES3, and AES11. For the IEC-60958 standard, all the user-data and channel-status bits (as outlined in this document) are not decoded by the S/PDIF receiver. The interface does make all 192 user-data and channel-status pairs available as an output of the block, for post-decoding.

For the IEC-61937 standard, the S/PDIF only detects compressed AC-3 and DTS formats. No decompression is performed.

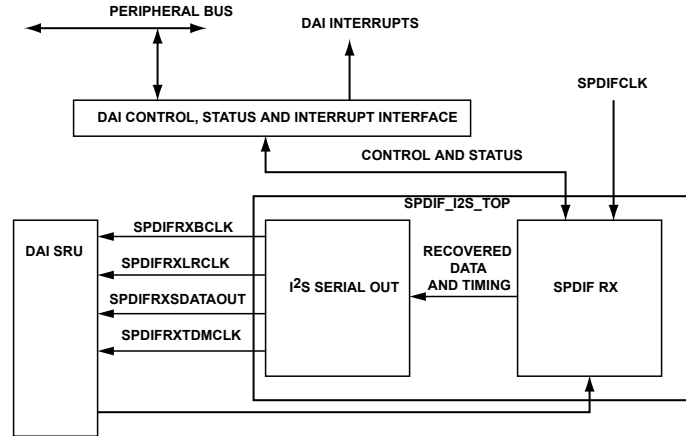


Figure 24-5: S/PDIF Receiver Block Diagram

Functional Description

If the receiver is used, programs need to enable it using the `SPDIF_RX_CTL` register. After the SRU programming is complete, write to the register with control values. At this point, the receiver attempts to lock.

NOTE: The S/PDIF receiver is disabled at default. If the receiver is used in an application, programs should enable the receiver.

The input to the receiver (`SPDIFn_RX_I`) is a biphase encoded signal that may contain two audio channels (compressed or linear PCM) or non-audio data. The receiver decodes the single biphase encoded stream, producing an I²S compatible serial data output that consists of a serial clock, a left-right frame sync, and data (channel A/B). It provides the programmer with several methods of managing the incoming status bit information.

The S/PDIF receiver receives any S/PDIF stream with a sampling frequency range of 24 kHz to 192 kHz. Refer to [Clocking](#) for more details.

The channel status bits are collected into memory-mapped registers, while other channel status and user bytes must be handled manually. The block start bit, which replaces the parity bit in the serial I²S stream, indicates the reception of the Z preamble and the start of a new block of channel status and data bits.

Clock Recovery

The S/PDIF receiver recovers the clock that generated the AES3/SPDIF biphase encoded stream from the incoming S/PDIF stream.

This clock is used by the receiver to clock in the biphase encoded data stream and also to provide clocks for either the SPORTs, sample rate converter, or the AES3 and S/PDIF transmitter. The recovered clock may also be used externally to the chip for clocking D/A and A/D converters.

In order to maintain performance, jitter on the clock is sourced to several peripherals.

To comply with the AES11 standard, the recovered left or right clock must be aligned with the preambles within a + or - 5% of the frame period. Since the PLL clock generates a clock 512 times the frame rate clock ($512 f_{\text{SCLK}}$), this clock can be used and divided down to create the phase aligned jitter-free left or right clock.

TDM Clock Output

The SPDIF receiver generates the TDMCLK clock signal which is 256 times the sampling rate. For proper generation of TDMCLK, program the SPDIF receiver as shown in the *TDMCLK Configuration* table.

IMPORTANT: The SPDIF receiver guarantees 256 clocks in one frame sync. It does not guarantee the duty cycle of the generated clocks. Therefore, do not use this clock in the applications that have duty cycle requirements.

Table 24-3: TDMCLK Configuration

SPDIF_RX_CTL.TDMSEL	Recommended Sampling Rate
0000	$\leq 96\text{K}$
0001	192K
0011	96K - 192K
0101	96K

Output Data Format

The extracted 24-bit audio data, V, U, C, and block start bits are sent on the SPDIF_RX_DAT pin in 32-bit I²S format as shown in *I²S and Left-Justified Formats*. The frame sync is transmitted on the SPDIF_RX_FS pin and serial clock is transmitted on the SPDIF_RX_CLK pin. All three pins are routed through the SRU.

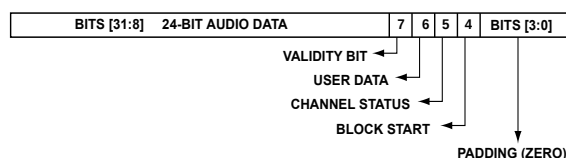


Figure 24-6: I²S Format

Channel Status

The channel status for the first bytes 4-0 (consumer mode) are collected into memory-mapped registers (`SPDIF_RX_STAT0_A`, `SPDIF_RX_STAT0_B`, `SPDIF_RX_STAT1_A` and `SPDIF_RX_STAT1_B`). All other channel status bytes 23-5 (professional mode) must be manually extracted from the receiver data stream.

NOTE: Only the first 5 channel status bytes (40-bit) for consumer mode of a frame are stored into the S/PDIF receiver status registers.

Operating Modes

This section describes the receiver channel status for the different modes.

Compressed or Non-linear Audio Data

The S/PDIF receiver processes compressed as well as non-linear audio data according to the supported standards. The following sections describe how this peripheral handles different data.

MPEG-2, AC-3, DTS, and AAC compressed data may be transmitted without setting either the `SPDIF_RX_STAT.VALID` bit or bit 1 of byte 0. To detect this data, the IEC61937 and SPMTE 337M standards dictate that there be a 96-bit sync code in the 16-, 20- or 24-bit audio data stream. This sync code consists of four words of zeros followed by a word consisting of 0xF872 and another word consisting of 0x4E1F. When this sync code is detected, the `SPDIF_RX_STAT.COMPMODE` bits hold the information regarding type of compression.

The last two words of the sync code, 0xF872 and 0x4E1F, are called the preamble-A and preamble-B of the burst preamble. Preamble-C of the burst preamble contains burst information and is captured and stored by the receiver. Preamble-D of the burst preamble contains the length code and is captured by the receiver. Even if the validity bit or bit 1 of byte 0 has been set, the receiver still looks for the sync code in order to record the preamble-C and D values. Once the sync code has not been detected in 4096 frames, the preamble-C and D registers are set to zero.

Emphasized Audio Data

Determination as to whether the received audio data is emphasized or not is done in software using the channel status bits as detailed below.

- In professional mode, (bit 0 of byte 0 = 1), channel status bits 2-4 of byte 0 indicate the audio data is emphasized if they are equal to 110 or 111.
- In consumer mode, (bit 0 of byte 0 = 0), channel status bits 3-5 indicate the audio data is emphasized if they are equal to 100, 010 or 110.

Single-Channel Double-Frequency Mode

Unlike previous processors, support for single-channel, double-frequency mode (SCDF) is not supported through specific bits within the `SPDIF_RX_CTL` register, but must be implemented in software using the information provided by the CS (channel status) bits.

- 0111—single channel double frequency mode
- 1000—single channel double frequency mode-stereo left
- 1001—single channel double frequency mode-stereo right

Clock Recovery Modes

The S/PDIF receiver extracts audio data, channel status, and user bits from the biphase encoded AES3 and S/PDIF stream. In addition, a 50% duty cycle reference clock running at the sampling rate of the audio input data is generated for the receiver to recover the oversampling clock.

Number Controlled Oscillator

The receiver can recover the clock from the biphase encoded stream using an on-chip NCO shown in the following figure. The dedicated NCO is separate from the PLL that supplies the clock to the processor core, which is the default operation of the receiver.

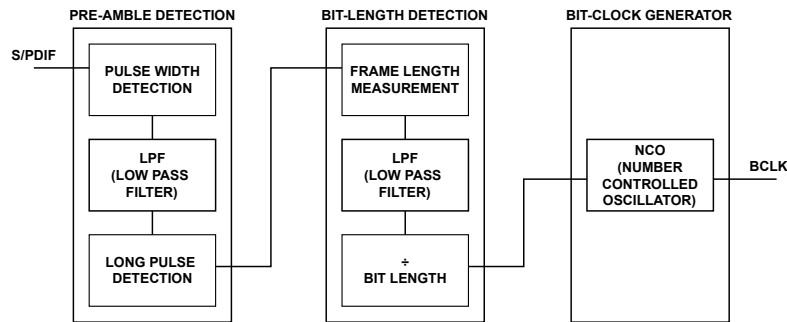


Figure 24-7: S/PDIF Clock Recovery Mechanism

The left/right frame reference clock for the NCO is generated using the preambles. The recovered low jitter left/right frame clock from the NCO attempts to align with the reference clock. However, this recovered left/right clock, like the reference clock, is not phase aligned with the preambles.

Interrupts

The following sections provide information about interrupt sources, masking, and servicing.

Sources

The S/PDIF module of each DAI drives five interrupt signals. Four are status signals driven from SPDIFn_RX and one signal is driven from SPDIFn_TX (block start). These signals are connected into the [DAI_IRPTL_L/DAI_IRPTL_H](#) latch register.

Transmit Block Start

The SPDIFn_TX_BLKSTART output signal, if routed to any miscellaneous interrupt bits (DAIn_INT_31-22 in the [DAI_MISCO/DAI_MISC1](#) registers), triggers a block start interrupt during the last frame of current block.

Receiver Status

The following receiver status bits generate an interrupt.

- Validity ([SPDIF_RX_STAT.VALID](#))
- Receiver locked ([SPDIF_RX_STAT.LOCK](#))
- No audio ([SPDIF_RX_STAT.AUDIOTYPE](#))

Receiver Error

The loss of lock ([SPDIF_RX_STAT.LOCKLOSS](#)) bit generates an interrupt.

Masking

For the S/PDIF receiver the [DAI_IMSK_RE](#) register must be unmasked accordingly. For the S/PDIF transmit the [DAIn_IMASK_x](#) register must be unmasked accordingly.

The `INTR_DAI_IRQH` and `INTR_DAI_IRQL` signals are routed to the SEC.

Service

The ISR reads the `DAI_IRPTL_H` and `DAI_IRPTL_L` registers to clear the interrupt request.

Programming Model

The following sections provide information on programming the transmitter and receiver.

Programming the Transmitter

Since the S/PDIF transmitter data input is not available to the core, programming the transmitter is as simple as: 1) connecting the SRU to the on-chip (serial ports or input data port) or off-chip (DAI pins) serial devices that provide the clock and data to be encoded, and 2) selecting the desired mode in the transmitter control register. This setup can be accomplished in three steps.

1. Connect the transmitter's four required input signals and one biphase encoded output in the SRU. The four input signals are the serial clock (`SPDIF_TX_CLK_I`), the serial frame sync (`SPDIF_TX_FS_I`), the serial data (`SPDIF_TX_DAT_I`), and the high frequency clock (`SPDIF_TX_HFCLK_I`) used for the encoding. The only output of the transmitter is `SPDIF_TX_O`.
2. If user bits are required, write 0x1 to the `SPDIF_TX_USRUPDT` register for the first block of transfer. Also route the `SPDIF_TX_BLK_START_O` signal to the `DAI_INT_31-22` (`DAI_IRPTLx` register). This generates interrupts during the last frame of the block (192), allowing changes of user bits for the next block.
3. Initialize the `SPDIF_TX_CTL` register to enable the data encoding.
4. Manually set the block start bit in the data stream once per block (every 384 words). This is necessary if automatic generation of block start information is not enabled using the `SPDIF_TX_CTL.AUTO` bit = 0.

NOTE: For more information, see the "DAI Routing Capabilities" section of the *Digital Audio Interface (DAI)* chapter.

Programming the Receiver

Since the S/PDIF receiver data output is not available to the core, programming the peripheral is as simple as connecting the SRU to the on-chip (serial ports) or off-chip (DAI pins) serial devices that provide the clock and data to be decoded, and selecting the desired mode in the receiver control register. This setup can be accomplished in two steps.

1. Connect the input signal and three output signals in the SRU. The only input of the receiver is the biphase encoded stream, `SPDIFn_RX_I`. The three required output signals are the serial clock (`SPDIFn_RX_CLK`), the serial frame sync (`SPDIFn_RX_FS`), and the serial data (`SPDIFn_RX_DAT`). The high frequency clock (`SPDIFn_RX_TDMCLK`) derived from the encoded stream is also available if the system requires it.

2. Initialize the `SPDIF_RX_CTL` register to enable the data decoding. Note that this peripheral is disabled by default.

NOTE: For more information, see the "DAI Routing Capabilities" section of the *Digital Audio Interface (DAI)* chapter.

Interrupted Data Streams on the Receiver

When using the S/PDIF receiver with data streams that are likely to be interrupted, (in other words unplugged and reconnected), it is necessary to take some extra steps to ensure that the S/PDIF receiver's digital PLL will relock to the stream. The steps to accomplish this are described below.

1. Set up interrupts within the DAI so that the S/PDIF receiver can generate an interrupt when the stream is reconnected.
2. Within the interrupt service routine (ISR), stop and restart the NCO. This is accomplished by setting and then clearing the `SPDIF_RX_CTL.RST` bit.
3. Return from the ISR and continue normal operation.

This method of resetting the NCO has been shown to provide extremely reliable performance when the S/PDIF inputs are interrupted or unplugged momentarily.

Debug Features

The following feature supports S/PDIF debugging.

Loopback Routing

The S/PDIF supports an internal loopback mode by using the SRU. For more information about loopback, see "Loopback Routing" in the *Digital Audio Interface (DAI)* chapter.

ADSP-21560_21561_21564_21568 SPDIF Register Descriptions

The S/PDIF module (SPDIF) contains the following registers.

Table 24-4: ADSP-21560_21561_21564_21568 SPDIF Register List

Name	Description
<code>SPDIF_RX_CTL</code>	Receive Control
<code>SPDIF_RX_STAT</code>	Receive Status Register
<code>SPDIF_RX_STAT0_A</code>	Receive Status A0 Register
<code>SPDIF_RX_STAT0_B</code>	Receive Status B0 Register
<code>SPDIF_RX_STAT1_A</code>	Receive Status A1 Register
<code>SPDIF_RX_STAT1_B</code>	Receive Status B1 Register

Table 24-4: ADSP-21560_21561_21564_21568 SPDIF Register List (Continued)

Name	Description
SPDIF_TX_CTL	Transmit Control Register
SPDIF_TX_STAT_A0	Transmit Status A0 Register
SPDIF_TX_STAT_A1	Transmit Status A1 Register
SPDIF_TX_STAT_A2	Transmit Status A2 Register
SPDIF_TX_STAT_A3	Transmit Status A3 Register
SPDIF_TX_STAT_A4	Transmit Status A4 Register
SPDIF_TX_STAT_A5	Transmit Status A5 Register
SPDIF_TX_STAT_B0	Transmit Status B0 Register
SPDIF_TX_STAT_B1	Transmit Status B1 Register
SPDIF_TX_STAT_B2	Transmit Status B2 Register
SPDIF_TX_STAT_B3	Transmit Status B3 Register
SPDIF_TX_STAT_B4	Transmit Status B4 Register
SPDIF_TX_STAT_B5	Transmit Status B5 Register
SPDIF_TX_UBUFF_A0	Transmit User Buffer A0 Register
SPDIF_TX_UBUFF_A1	Transmit User Buffer A1 Register
SPDIF_TX_UBUFF_A2	Transmit User Buffer A2 Register
SPDIF_TX_UBUFF_A3	Transmit User Buffer A3 Register
SPDIF_TX_UBUFF_A4	Transmit User Buffer A4 Register
SPDIF_TX_UBUFF_A5	Transmit User Buffer A5 Register
SPDIF_TX_UBUFF_B0	Transmit User Buffer B0 Register
SPDIF_TX_UBUFF_B1	Transmit User Buffer B1 Register
SPDIF_TX_UBUFF_B2	Transmit User Buffer B2 Register
SPDIF_TX_UBUFF_B3	Transmit User Buffer B3 Register
SPDIF_TX_UBUFF_B4	Transmit User Buffer B4 Register
SPDIF_TX_UBUFF_B5	Transmit User Buffer B5 Register
SPDIF_TX_USRUPDT	User Bit Update Register

Receive Control

The `SPDIF_RX_CTL` register is used to enable and control the S/PDIF receiver.

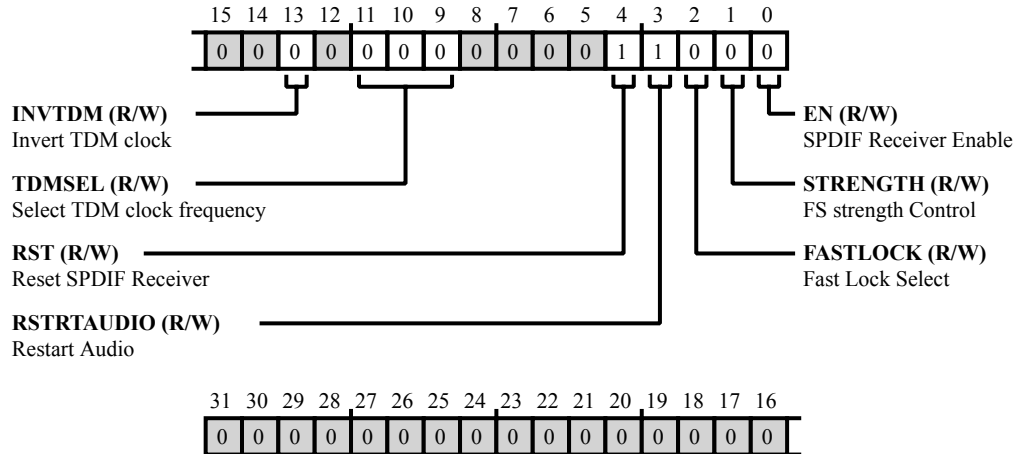


Figure 24-8: SPDIF_RX_CTL Register Diagram

Table 24-5: SPDIF_RX_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
13 (R/W)	INVTDM	Invert TDM clock. When set to 0, inverted TDM clock or non-inverted Bit clock is selected. When set to 1, non-inverted TDM clock or inverted Blit clock is selected.
11:9 (R/W)	TDMSEL	Select TDM clock frequency. Select TDM clock frequency
4 (R/W)	RST	Reset SPDIF Receiver. The <code>SPDIF_RX_CTL.RST</code> bit resets the receiver.
		0 Takes the receiver out of reset
		1 Resets the SPDIF receiver
3 (R/W)	RSTRTAUDIO	Restart Audio. The <code>SPDIF_RX_CTL.RSTRTAUDIO</code> bit restarts the audio once a re-lock has occurred. When the S/PDIF receiver loses lock the audio output is set to 0. This bit determines the behavior of the audio once lock is re-established. Audio can be manually restarted by toggling this bit high and then low.
		0 Manually restart audio
		1 Automatically restart audio

Table 24-5: SPDIF_RX_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
2 (R/W)	FASTLOCK	Fast Lock Select. The SPDIF_RX_CTL.FASTLOCK bit allows the lock mechanism to lock at normal speed or at faster speed. This has the advantage of recovering very quickly whenever the S/PDIF receiver loses lock due to glitches in the signal. In normal mode the S/PDIF receiver locks after 64 consecutive valid samples, in fast mode the S/PDIF receiver locks after 8 consecutive valid samples
		0 Enable normal mode
		1 Enable fast mode
1 (R/W)	STRENGTH	FS strength Control. The SPDIF_RX_CTL.STRENGTH bit controls the strength of the bit clock and Frame sync outputs from the SPDIF receiver. In strong mode these output signals are continued (as best possible) when the receiver notices a loss-of-lock condition. Note that 'as best possible' refers to the fact that this recovered signal may not be accurate, given the loss-of-lock condition. In weak mode these output signals are interrupted as soon as the receiver notices a loss-of-lock condition.
		0 Enable strong mode
		1 Enable weak mode
0 (R/W)	EN	SPDIF Receiver Enable. When the SPDIF_RX_CTL.EN bit =0 the clock to SPDIF is switched off for power savings.
		0 Disable receiver
		1 Enable receiver

Receive Status Register

The `SPDIF_RX_STAT` register consists of bits that indicate the status of various functions supported by S/PDIF receiver.

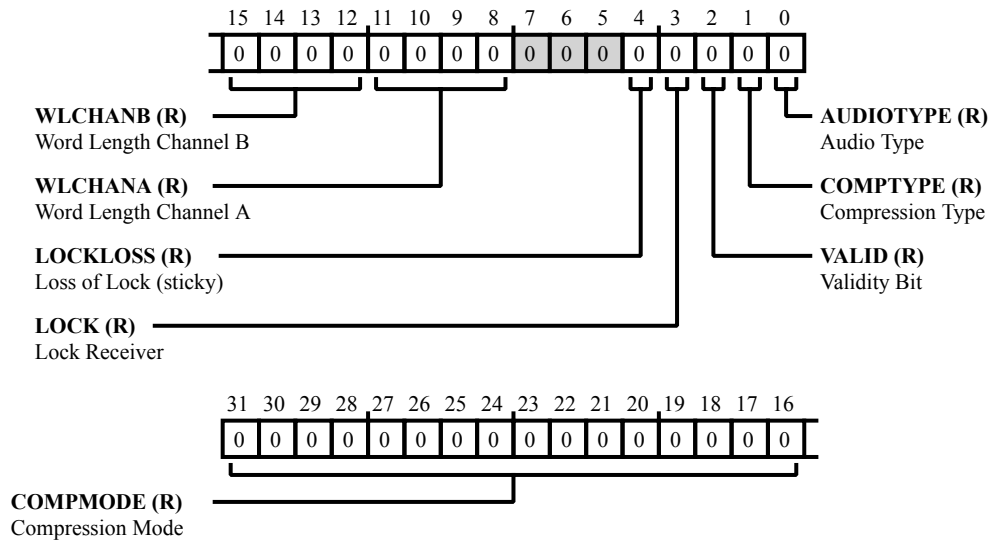


Figure 24-9: SPDIF_RX_STAT Register Diagram

Table 24-6: SPDIF_RX_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:16 (R/NW)	COMPMODE	Compression Mode. The <code>SPDIF_RX_STAT.COMPMODE</code> bit field indicates the type of compression mode used in the Digital audio stream. The value in this field indicates the 16-bit burst information as specified by the IEC 62937-2 standard. Use this document to decode the value in this bit field.
15:12 (R/NW)	WLCHANB	Word Length Channel B. The <code>SPDIF_RX_STAT.WLCHANB</code> bit field indicates the S/PDIF word length for channel B. May be decoded as follows (from the S/PDIF standard).
		0 Reserved
		1 Reserved
		2 SPDIF_LENGTH_16
		3 Reserved
		4 SPDIF_LENGTH_18
		5 SPDIF_LENGTH_22
		6 Reserved

Table 24-6: SPDIF_RX_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
		7	Reserved
		8	SPDIF_LENGTH_19
		9	SPDIF_LENGTH_23
		10	SPDIF_LENGTH_20
		11	SPDIF_LENGTH_24
		12	SPDIF_LENGTH_17
		13	SPDIF_LENGTH_21
		14	Reserved
		15	Reserved
11:8 (R/NW)	WLCHANA	Word Length Channel A. The SPDIF_RX_STAT.WLCHANA bit indicates the S/PDIF word length for channel A. May be decoded as follows (from the S/PDIF standard).	
		0	Reserved
		1	Reserved
		2	SPDIF_LENGTH_16
		3	Reserved
		4	SPDIF_LENGTH_18
		5	SPDIF_LENGTH_22
		6	Reserved
		7	Reserved
		8	SPDIF_LENGTH_19
		9	SPDIF_LENGTH_23
		10	SPDIF_LENGTH_20
		11	SPDIF_LENGTH_24
		12	SPDIF_LENGTH_17
		13	SPDIF_LENGTH_21
		14	Reserved
		15	Reserved

Table 24-6: SPDIF_RX_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
4 (R/NW)	LOCKLOSS	Loss of Lock (sticky). The <code>SPDIF_RX_STAT.LOCKLOSS</code> bit indicates that the system has lost lock. This is different to the lock register, as it is sticky it goes high when system loses lock, but returns to low once <code>SPDIF_RX_CTL.RSTRTAUDIO</code> bit is toggled. This is to allow the programs to poll the lock status.
		0 SPDIF receiver locked
		1 SPDIF Receiver lost lock
3 (R/NW)	LOCK	Lock Receiver. The <code>SPDIF_RX_STAT.LOCK</code> bit indicates the S/PDIF receiver has successfully locked to the S/PDIF stream and is outputting valid data.
		0 Receiver not locked
		1 Receiver locked
2 (R/NW)	VALID	Validity Bit. The <code>SPDIF_RX_STAT.VALID</code> bit indicates the ORed bits of channel A and B.
		0 Linear PCM data
		1 Non-linear audio data
1 (R/NW)	COMPTYPE	Compression Type. The <code>SPDIF_RX_STAT.COMPTYPE</code> bit indicates AC3 or DTS compression. Valid only if <code>SPDIF_RX_STAT.AUDIOTYPE</code> indicates compressed data.
		0 AC3 compressed data
		1 DTS compressed data
0 (R/NW)	AUDIOTYPE	Audio Type. The <code>SPDIF_RX_STAT.AUDIOTYPE</code> bit indicates PCM or compressed data.
		0 PCM data
		1 Compressed Data

Receive Status A0 Register

The `SPDIF_RX_STAT0_A` register holds the receive channel 0 status for bytes 0-3 for sub frame A.

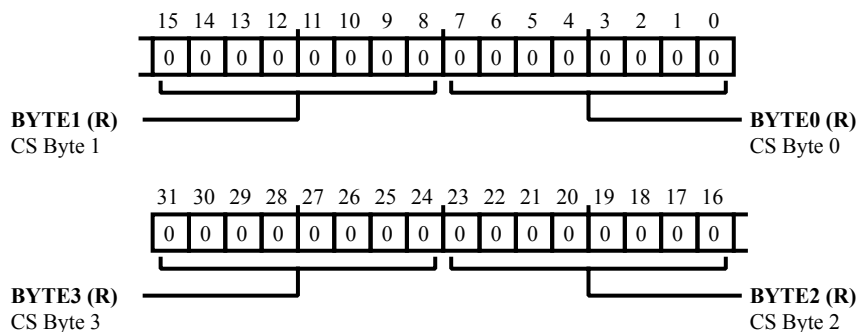


Figure 24-10: `SPDIF_RX_STAT0_A` Register Diagram

Table 24-7: `SPDIF_RX_STAT0_A` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/NW)	BYTE3	CS Byte 3. The <code>SPDIF_RX_STAT0_A.BYTE3</code> bit field contains byte 3 of received channel A status.
23:16 (R/NW)	BYTE2	CS Byte 2. The <code>SPDIF_RX_STAT0_A.BYTE2</code> bit field contains byte 2 of received channel A status.
15:8 (R/NW)	BYTE1	CS Byte 1. The <code>SPDIF_RX_STAT0_A.BYTE1</code> bit field contains byte 1 of received channel A status.
7:0 (R/NW)	BYTE0	CS Byte 0. The <code>SPDIF_RX_STAT0_A.BYTE0</code> bit field contains the status of byte 0-3 of received channel A.

Receive Status B0 Register

The `SPDIF_RX_STAT0_B` register holds the receive channel 0 status for bytes 0-3 for sub frame B.

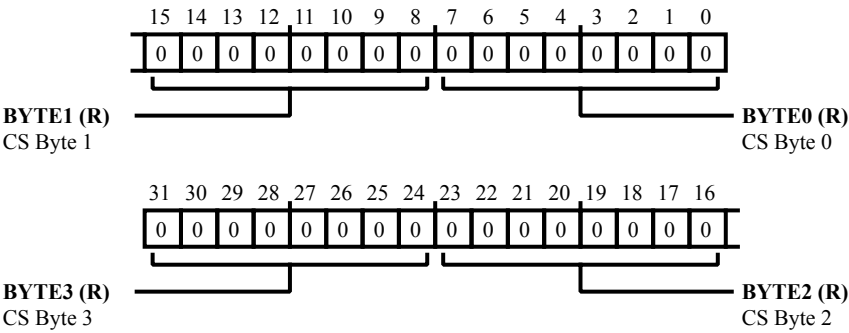


Figure 24-11: SPDIF_RX_STAT0_B Register Diagram

Table 24-8: SPDIF_RX_STAT0_B Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/NW)	BYTE3	CS Byte 3. The <code>SPDIF_RX_STAT0_B.BYTE3</code> bit field contains byte 3 of received channel B status.
23:16 (R/NW)	BYTE2	CS Byte 2. The <code>SPDIF_RX_STAT0_B.BYTE2</code> bit field contains byte 2 of received channel B status.
15:8 (R/NW)	BYTE1	CS Byte 1. The <code>SPDIF_RX_STAT0_B.BYTE1</code> bit field contains byte 1 of received channel B status.
7:0 (R/NW)	BYTE0	CS Byte 0. The <code>SPDIF_RX_STAT0_B.BYTE0</code> bit field contains byte 0 of received channel B status.

Receive Status A1 Register

The `SPDIF_RX_STAT1_A` register holds the receive channel 1 status for byte 4 for sub frame A.

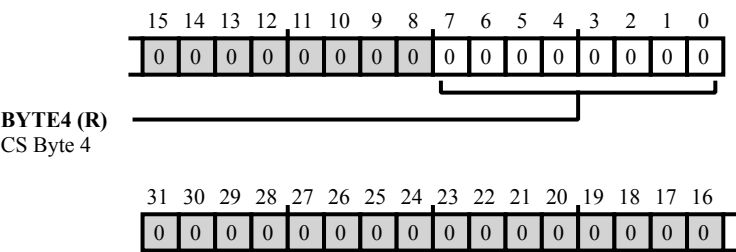


Figure 24-12: SPDIF_RX_STAT1_A Register Diagram

Table 24-9: SPDIF_RX_STAT1_A Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	BYTE4	CS Byte 4. The <code>SPDIF_RX_STAT1_A.BYTE4</code> bit field contains byte 4 of received channel A status.

Receive Status B1 Register

The `SPDIF_RX_STAT1_B` register holds the receive channel 1 status for byte 4 for sub frame B.

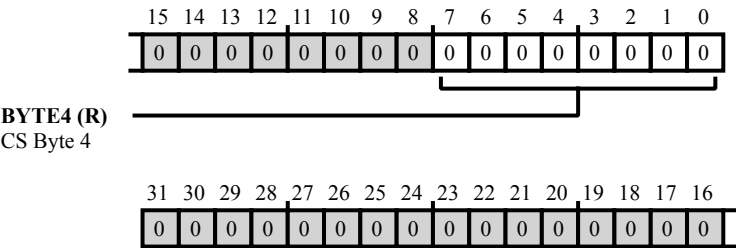


Figure 24-13: SPDIF_RX_STAT1_B Register Diagram

Table 24-10: SPDIF_RX_STAT1_B Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	BYTE4	CS Byte 4. The <code>SPDIF_RX_STAT1_B.BYTE4</code> bit field contains byte 4 of received channel B status.

Transmit Control Register

The `SPDIF_TX_CTL` register provides bits to enable/disable the transmitter and configure several options related to data transmission.

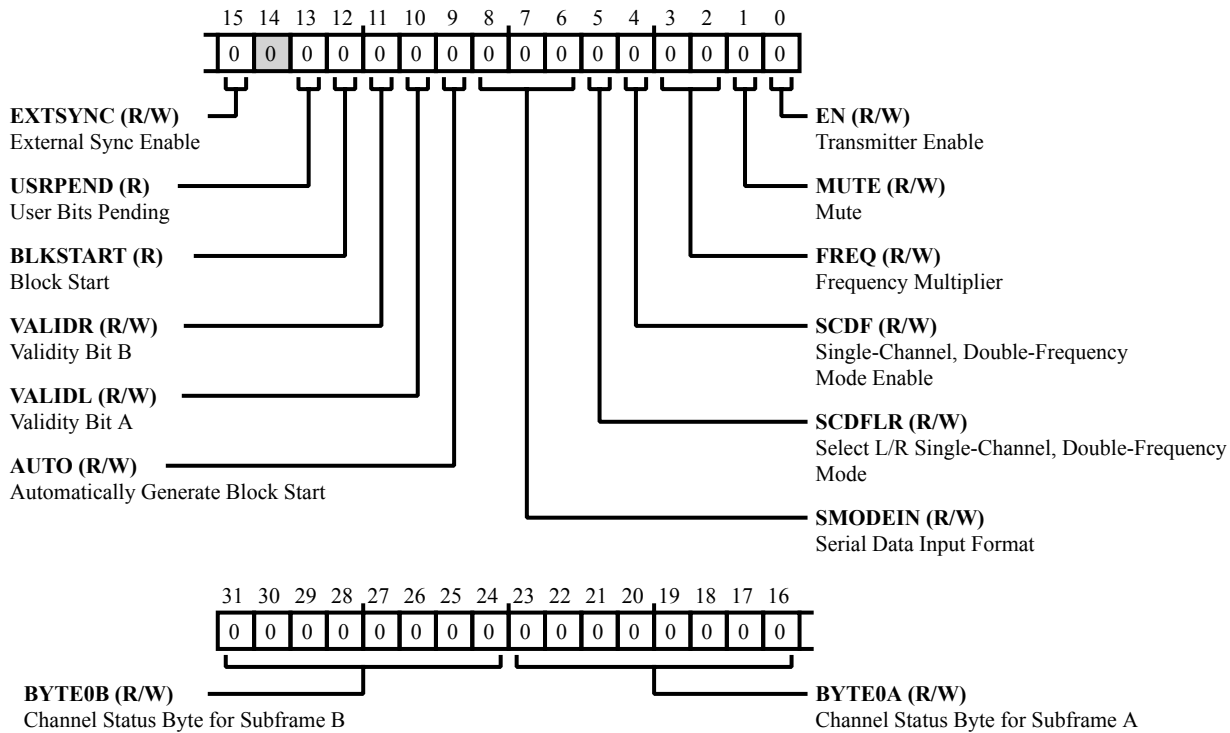


Figure 24-14: SPDIF_TX_CTL Register Diagram

Table 24-11: SPDIF_TX_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	BYTE0B	Channel Status Byte for Subframe B. The <code>SPDIF_TX_CTL.BYTE0B</code> bit field contains the channel status for the second bytes 95.
23:16 (R/W)	BYTE0A	Channel Status Byte for Subframe A. The <code>SPDIF_TX_CTL.BYTE0A</code> bit field contains the channel status for the first bytes 40.
15 (R/W)	EXTSYNC	External Sync Enable. When the <code>SPDIF_TX_CTL.EXTSYNC</code> bit is set (regardless of the <code>SPDIF_TX_CTL.AUTO</code> bit setting) the internal frame counter is set to zero at an internal LRCLK rising edge followed by an <code>EXTSYNC_I</code> rising edge.

Table 24-11: SPDIF_TX_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
13 (R/NW)	USRPEND	User Bits Pending. The SPDIF_TX_CTL.USRPEND bit is set if the update of the internal buffer from the Transmit User Bits Buffer registers has not yet completed.
12 (R/NW)	BLKSTART	Block Start. The SPDIF_TX_CTL.BLKSTART bit is a status bit that indicates block start (when the SPDIF_TX_CTL.AUTO bit = 1).
		0 Current word is not block start
		1 Current word is block start
11 (R/W)	VALIDR	Validity Bit B. Use the SPDIF_TX_CTL.VALIDR bit with the channel status buffer.
10 (R/W)	VALIDL	Validity Bit A. The SPDIF_TX_CTL.VALIDL bit either manually start block transfer according to input stream status bits or automatically start block transfer. Use the SPDIF_TX_CTL.VALIDL bit with the channel status buffer.
		0 Manually start block transfer
		1 Automatically start block transfer
9 (R/W)	AUTO	Automatically Generate Block Start. When enabled, the transmitter is in standalone mode where it inserts block start, channel status, and validity bits on its own. If the channel status or validity buffer needs to be enabled (after the DAI programming is complete), first write to the buffers with the required data and then enable the buffers by setting the SPDIF_TX_CTL.AUTO bit.
8:6 (R/W)	SMODEIN	Serial Data Input Format. The SPDIF_TX_CTL.SMODEIN bit selects the data input format.
		0 Left-justified
		1 I2S
		2 Reserved
		3 Reserved
		4 Right-justified, 24 bits
		5 Right-justified, 20 bits
		6 Right-justified, 18 bits
		7 Right-justified, 16 bits

Table 24-11: SPDIF_TX_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
5 (R/W)	SCDFLR	Select L/R Single-Channel, Double-Frequency Mode. The <code>SPDIF_TX_CTL.SCDFLR</code> bit selects the left or right channel in SCDF mode.
		0 Left channel
		1 Right channel
4 (R/W)	SCDF	Single-Channel, Double-Frequency Mode Enable. The <code>SPDIF_TX_CTL.SCDF</code> bit enables single-channel, double-frequency mode.
		0 Two-channel mode
		1 SCDF mode
3:2 (R/W)	FREQ	Frequency Multiplier. The <code>SPDIF_TX_CTL.FREQ</code> bit field sets the oversampling ratio.
		0 256 x frame sync oversampling
		1 384 x frame sync oversampling
		2-3 Reserved
1 (R/W)	MUTE	Mute. The <code>SPDIF_TX_CTL.MUTE</code> bit mutes the serial data output.
		0 Disable Mute
		1 Enable Mute
0 (R/W)	EN	Transmitter Enable. The <code>SPDIF_TX_CTL.EN</code> bit enables the transmitter and resets the control registers to their defaults.
		0 Transmitter disabled
		1 Transmitter enabled

Transmit Status A0 Register

The `SPDIF_TX_STAT_A0` register holds the transmit channel 0 status for bytes 1-4 for sub frame A.

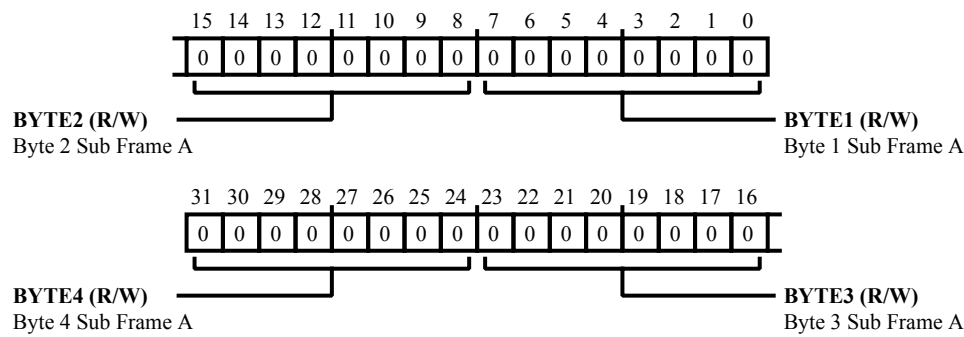


Figure 24-15: `SPDIF_TX_STAT_A0` Register Diagram

Table 24-12: `SPDIF_TX_STAT_A0` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	BYTE4	Byte 4 Sub Frame A. The <code>SPDIF_TX_STAT_A0</code> . <code>BYTE4</code> bit field holds the transmit channel 0 status for byte 4 for sub frame A.
23:16 (R/W)	BYTE3	Byte 3 Sub Frame A. The <code>SPDIF_TX_STAT_A0</code> . <code>BYTE3</code> bit field holds the transmit channel 0 status for byte 3 for sub frame A.
15:8 (R/W)	BYTE2	Byte 2 Sub Frame A. The <code>SPDIF_TX_STAT_A0</code> . <code>BYTE2</code> bit field holds the transmit channel 0 status for byte 2 for sub frame A.
7:0 (R/W)	BYTE1	Byte 1 Sub Frame A. The <code>SPDIF_TX_STAT_A0</code> . <code>BYTE1</code> bit field holds the transmit channel 0 status for byte 1 for sub frame A.

Transmit Status A1 Register

The `SPDIF_TX_STAT_A1` register holds the transmit status for bytes 5-8 for sub frame A.

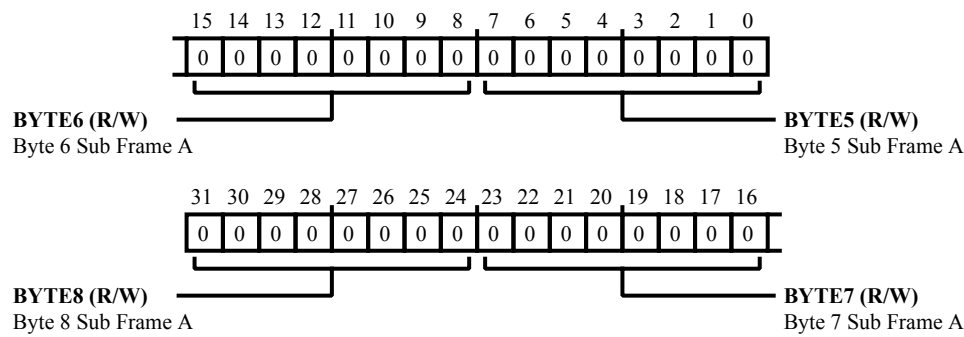


Figure 24-16: `SPDIF_TX_STAT_A1` Register Diagram

Table 24-13: `SPDIF_TX_STAT_A1` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	BYTE8	Byte 8 Sub Frame A. The <code>SPDIF_TX_STAT_A1</code> . <code>BYTE8</code> bit field contains transmit status for byte 8 of sub frame A.
23:16 (R/W)	BYTE7	Byte 7 Sub Frame A. The <code>SPDIF_TX_STAT_A1</code> . <code>BYTE7</code> bit field contains transmit status for byte 7 of sub frame A.
15:8 (R/W)	BYTE6	Byte 6 Sub Frame A. The <code>SPDIF_TX_STAT_A1</code> . <code>BYTE6</code> bit field contains transmit status for byte 6 of sub frame A.
7:0 (R/W)	BYTE5	Byte 5 Sub Frame A. The <code>SPDIF_TX_STAT_A1</code> . <code>BYTE5</code> bit field contains transmit status for byte 5 of sub frame A.

Transmit Status A2 Register

The `SPDIF_TX_STAT_A2` register holds the transmit status for bytes 9-12 for sub frame A.

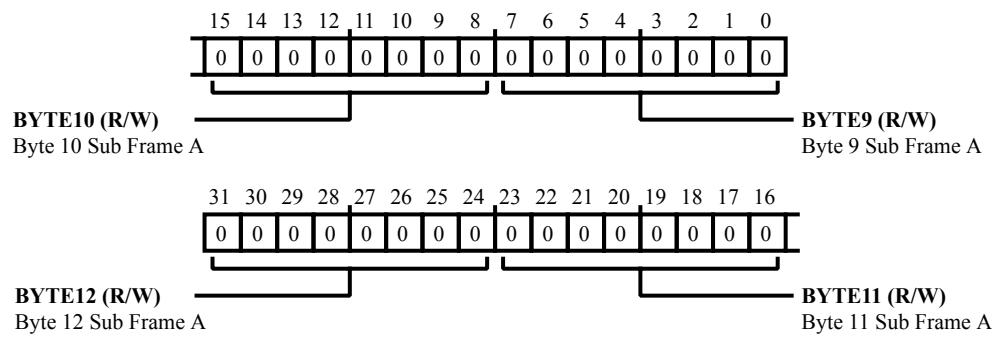


Figure 24-17: SPDIF_TX_STAT_A2 Register Diagram

Table 24-14: SPDIF_TX_STAT_A2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	BYTE12	Byte 12 Sub Frame A. The <code>SPDIF_TX_STAT_A2.BYTE12</code> bit field contains transmit status for byte 12 of sub frame A.
23:16 (R/W)	BYTE11	Byte 11 Sub Frame A. The <code>SPDIF_TX_STAT_A2.BYTE11</code> bit field contains transmit status for byte 11 of sub frame A.
15:8 (R/W)	BYTE10	Byte 10 Sub Frame A. The <code>SPDIF_TX_STAT_A2.BYTE10</code> bit field contains transmit status for byte 10 of sub frame A.
7:0 (R/W)	BYTE9	Byte 9 Sub Frame A. The <code>SPDIF_TX_STAT_A2.BYTE9</code> bit field contains transmit status for byte 9 of sub frame A.

Transmit Status A3 Register

The `SPDIF_TX_STAT_A3` register holds the transmit status for bytes 13-16 for sub frame A.

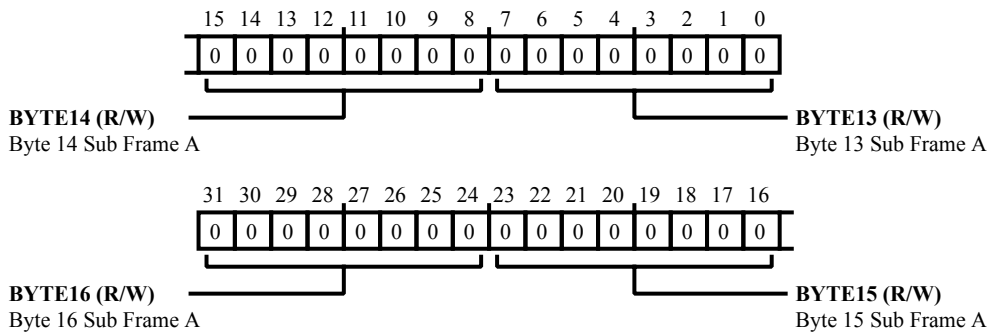


Figure 24-18: SPDIF_TX_STAT_A3 Register Diagram

Table 24-15: SPDIF_TX_STAT_A3 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	BYTE16	Byte 16 Sub Frame A. The <code>SPDIF_TX_STAT_A3.BYTE16</code> bit field contains transmit status for byte 16 of sub frame A.
23:16 (R/W)	BYTE15	Byte 15 Sub Frame A. The <code>SPDIF_TX_STAT_A3.BYTE15</code> bit field contains transmit status for byte 15 of sub frame A.
15:8 (R/W)	BYTE14	Byte 14 Sub Frame A. The <code>SPDIF_TX_STAT_A3.BYTE14</code> bit field contains transmit status for byte 14 of sub frame A.
7:0 (R/W)	BYTE13	Byte 13 Sub Frame A. The <code>SPDIF_TX_STAT_A3.BYTE13</code> bit field contains transmit status for byte 13 of sub frame A.

Transmit Status A4 Register

The `SPDIF_TX_STAT_A4` register holds the transmit status for bytes 17-20 for sub frame A.

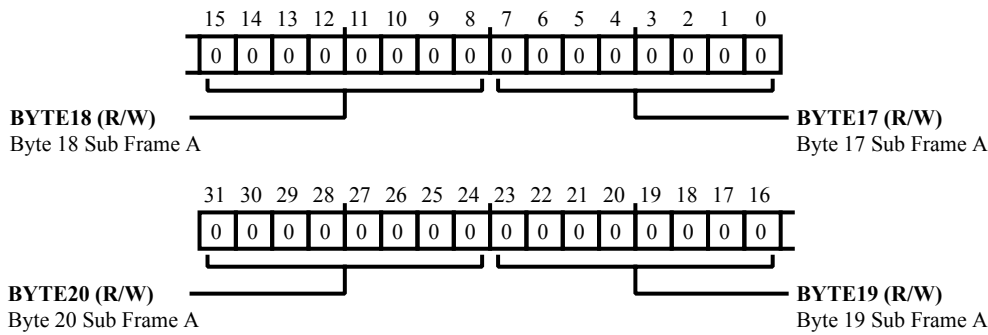


Figure 24-19: SPDIF_TX_STAT_A4 Register Diagram

Table 24-16: SPDIF_TX_STAT_A4 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	BYTE20	Byte 20 Sub Frame A. The <code>SPDIF_TX_STAT_A4.BYTE20</code> bit field contains transmit status for byte 20 of sub frame A.
23:16 (R/W)	BYTE19	Byte 19 Sub Frame A. The <code>SPDIF_TX_STAT_A4.BYTE19</code> bit field contains transmit status for byte 19 of sub frame A.
15:8 (R/W)	BYTE18	Byte 18 Sub Frame A. The <code>SPDIF_TX_STAT_A4.BYTE18</code> bit field contains transmit status for byte 18 of sub frame A.
7:0 (R/W)	BYTE17	Byte 17 Sub Frame A. The <code>SPDIF_TX_STAT_A4.BYTE17</code> bit field contains transmit status for byte 17 of sub frame A.

Transmit Status A5 Register

The `SPDIF_TX_STAT_A5` register holds the transmit status for bytes 21-23 for sub frame A.

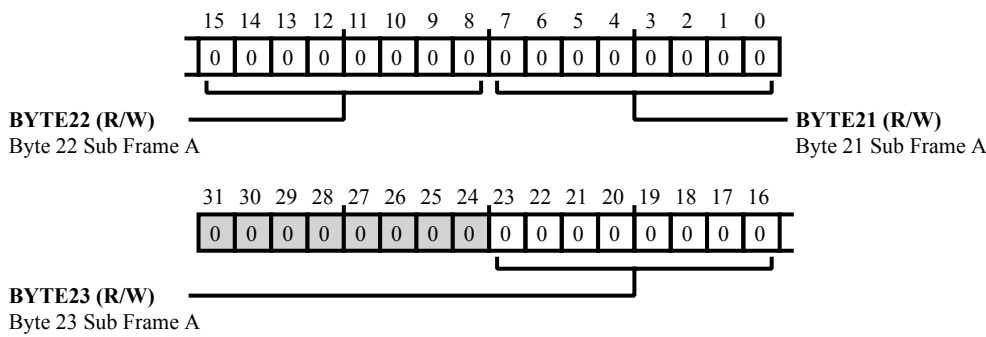


Figure 24-20: `SPDIF_TX_STAT_A5` Register Diagram

Table 24-17: `SPDIF_TX_STAT_A5` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
23:16 (R/W)	BYTE23	Byte 23 Sub Frame A. The <code>SPDIF_TX_STAT_A5.BYTE23</code> bit field contains transmit status for byte 23 of sub frame A.
15:8 (R/W)	BYTE22	Byte 22 Sub Frame A. The <code>SPDIF_TX_STAT_A5.BYTE22</code> bit field contains transmit status for byte 22 of sub frame A.
7:0 (R/W)	BYTE21	Byte 21 Sub Frame A. The <code>SPDIF_TX_STAT_A5.BYTE21</code> bit field contains transmit status for byte 21 of sub frame A.

Transmit Status B0 Register

The `SPDIF_TX_STAT_B0` register holds the transmit channel 0 status for bytes 1-4 for sub frame B.

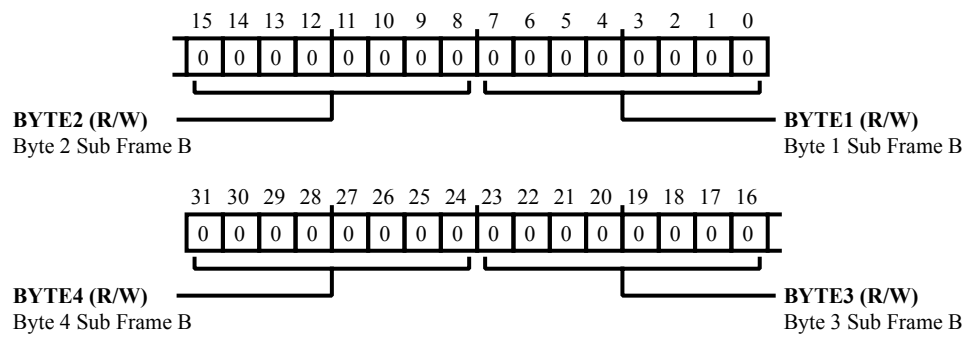


Figure 24-21: `SPDIF_TX_STAT_B0` Register Diagram

Table 24-18: `SPDIF_TX_STAT_B0` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	BYTE4	Byte 4 Sub Frame B. The <code>SPDIF_TX_STAT_B0.BYTE4</code> bit field holds the transmit channel 0 status for byte 4 for sub frame B.
23:16 (R/W)	BYTE3	Byte 3 Sub Frame B. The <code>SPDIF_TX_STAT_B0.BYTE3</code> bit field holds the transmit channel 0 status for byte 3 for sub frame B.
15:8 (R/W)	BYTE2	Byte 2 Sub Frame B. The <code>SPDIF_TX_STAT_B0.BYTE2</code> bit field holds the transmit channel 0 status for byte 2 for sub frame B.
7:0 (R/W)	BYTE1	Byte 1 Sub Frame B. The <code>SPDIF_TX_STAT_B0.BYTE1</code> bit field holds the transmit channel 0 status for byte 1 for sub frame B.

Transmit Status B1 Register

The `SPDIF_TX_STAT_B1` register holds the transmit status for bytes 5-8 for sub frame B.

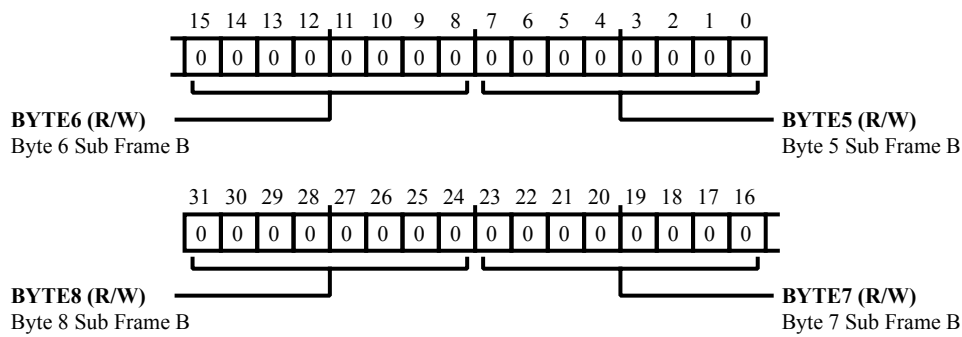


Figure 24-22: `SPDIF_TX_STAT_B1` Register Diagram

Table 24-19: `SPDIF_TX_STAT_B1` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	BYTE8	Byte 8 Sub Frame B. The <code>SPDIF_TX_STAT_B1</code> . <code>BYTE8</code> bit field contains transmit status for byte 8 of sub frame B.
23:16 (R/W)	BYTE7	Byte 7 Sub Frame B. The <code>SPDIF_TX_STAT_B1</code> . <code>BYTE7</code> bit field contains transmit status for byte 7 of sub frame B.
15:8 (R/W)	BYTE6	Byte 6 Sub Frame B. The <code>SPDIF_TX_STAT_B1</code> . <code>BYTE6</code> bit field contains transmit status for byte 6 of sub frame B.
7:0 (R/W)	BYTE5	Byte 5 Sub Frame B. The <code>SPDIF_TX_STAT_B1</code> . <code>BYTE5</code> bit field contains transmit status for byte 5 of sub frame B.

Transmit Status B2 Register

The `SPDIF_TX_STAT_B2` register holds the transmit status for bytes 9-12 for sub frame B.

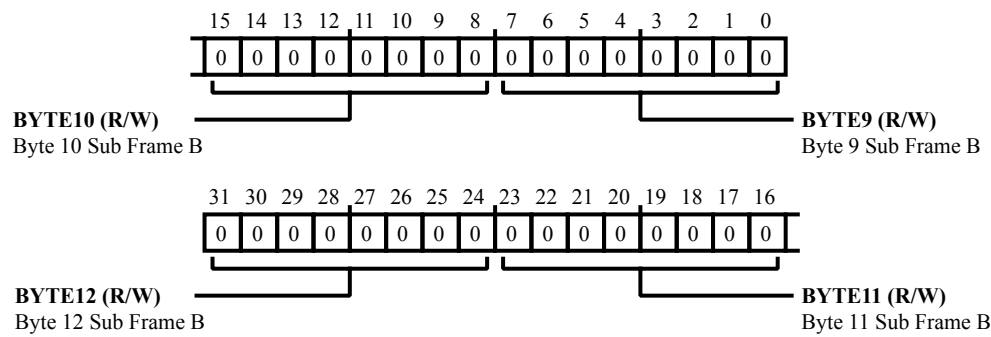


Figure 24-23: `SPDIF_TX_STAT_B2` Register Diagram

Table 24-20: `SPDIF_TX_STAT_B2` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	BYTE12	Byte 12 Sub Frame B. The <code>SPDIF_TX_STAT_B2.BYTE12</code> bit field contains transmit status for byte 12 of sub frame B.
23:16 (R/W)	BYTE11	Byte 11 Sub Frame B. The <code>SPDIF_TX_STAT_B2.BYTE11</code> bit field contains transmit status for byte 11 of sub frame B.
15:8 (R/W)	BYTE10	Byte 10 Sub Frame B. The <code>SPDIF_TX_STAT_B2.BYTE10</code> bit field contains transmit status for byte 10 of sub frame B.
7:0 (R/W)	BYTE9	Byte 9 Sub Frame B. The <code>SPDIF_TX_STAT_B2.BYTE9</code> bit field contains transmit status for byte 9 of sub frame B.

Transmit Status B3 Register

The `SPDIF_TX_STAT_B3` register holds the transmit status for bytes 13-16 for sub frame B.

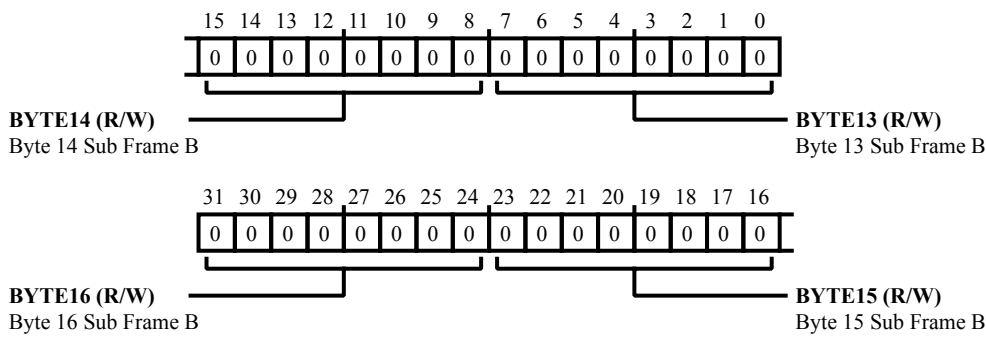


Figure 24-24: `SPDIF_TX_STAT_B3` Register Diagram

Table 24-21: `SPDIF_TX_STAT_B3` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	BYTE16	Byte 16 Sub Frame B. The <code>SPDIF_TX_STAT_B3.BYTE16</code> bit field contains transmit status for byte 16 of sub frame B.
23:16 (R/W)	BYTE15	Byte 15 Sub Frame B. The <code>SPDIF_TX_STAT_B3.BYTE15</code> bit field contains transmit status for byte 15 of sub frame B.
15:8 (R/W)	BYTE14	Byte 14 Sub Frame B. The <code>SPDIF_TX_STAT_B3.BYTE14</code> bit field contains transmit status for byte 14 of sub frame B.
7:0 (R/W)	BYTE13	Byte 13 Sub Frame B. The <code>SPDIF_TX_STAT_B3.BYTE13</code> bit field contains transmit status for byte 13 of sub frame B.

Transmit Status B4 Register

The `SPDIF_TX_STAT_B4` register holds the transmit status for bytes 17-20 for sub frame B.

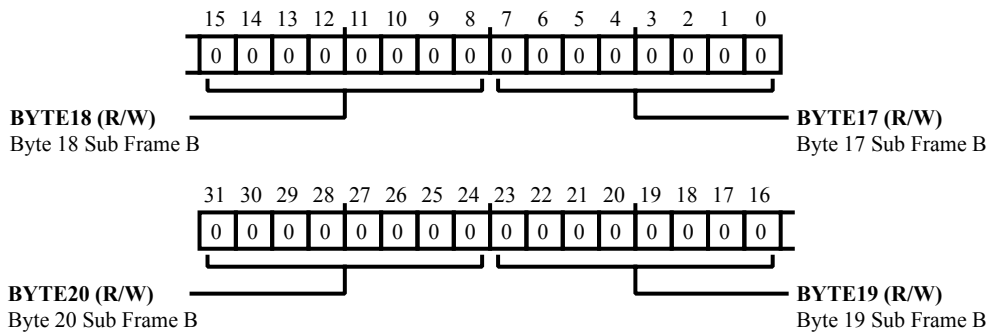


Figure 24-25: `SPDIF_TX_STAT_B4` Register Diagram

Table 24-22: `SPDIF_TX_STAT_B4` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	BYTE20	Byte 20 Sub Frame B. The <code>SPDIF_TX_STAT_B4.BYTE20</code> bit field contains transmit status for byte 20 of sub frame B.
23:16 (R/W)	BYTE19	Byte 19 Sub Frame B. The <code>SPDIF_TX_STAT_B4.BYTE19</code> bit field contains transmit status for byte 19 of sub frame B.
15:8 (R/W)	BYTE18	Byte 18 Sub Frame B. The <code>SPDIF_TX_STAT_B4.BYTE18</code> bit field contains transmit status for byte 18 of sub frame B.
7:0 (R/W)	BYTE17	Byte 17 Sub Frame B. The <code>SPDIF_TX_STAT_B4.BYTE17</code> bit field contains transmit status for byte 17 of sub frame B.

Transmit Status B5 Register

The `SPDIF_TX_STAT_B5` register holds the transmit status for bytes 21-23 for sub frame B.

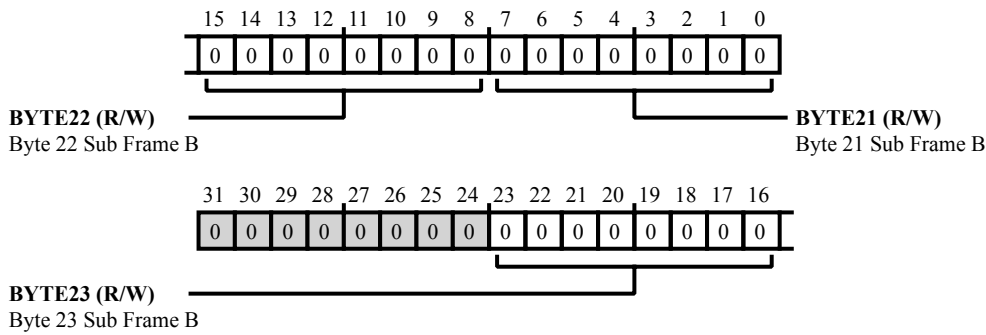


Figure 24-26: `SPDIF_TX_STAT_B5` Register Diagram

Table 24-23: `SPDIF_TX_STAT_B5` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
23:16 (R/W)	BYTE23	Byte 23 Sub Frame B. The <code>SPDIF_TX_STAT_B5.BYTE23</code> bit field contains transmit status for byte 23 of sub frame B.
15:8 (R/W)	BYTE22	Byte 22 Sub Frame B. The <code>SPDIF_TX_STAT_B5.BYTE22</code> bit field contains transmit status for byte 22 of sub frame B.
7:0 (R/W)	BYTE21	Byte 21 Sub Frame B. The <code>SPDIF_TX_STAT_B5.BYTE21</code> bit field contains transmit status for byte 21 of sub frame B.

Transmit User Buffer A0 Register

The `SPDIF_TX_UBUFF_A0` register holds the transmit user buffer data for bytes 0-3 for sub frame A.

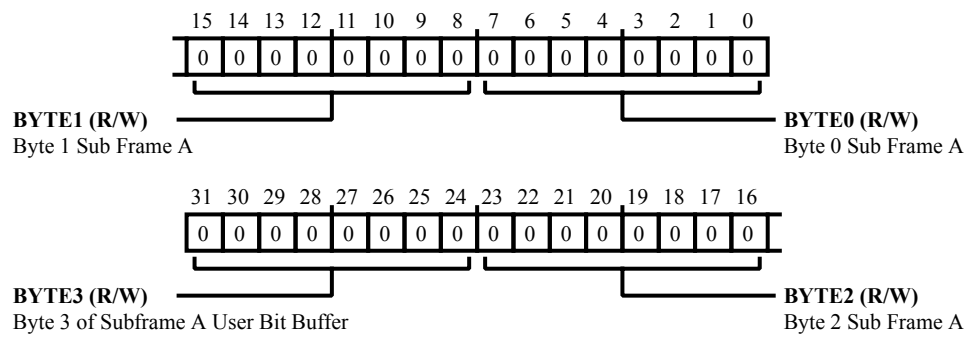


Figure 24-27: SPDIF_TX_UBUFF_A0 Register Diagram

Table 24-24: SPDIF_TX_UBUFF_A0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	BYTE3	Byte 3 of Subframe A User Bit Buffer. The <code>SPDIF_TX_UBUFF_A0.BYTE3</code> bit field contains user bit data for byte 0 of sub frame A.
23:16 (R/W)	BYTE2	Byte 2 Sub Frame A. The <code>SPDIF_TX_UBUFF_A0.BYTE2</code> bit field contains user bit data for byte 2 of sub frame A.
15:8 (R/W)	BYTE1	Byte 1 Sub Frame A. The <code>SPDIF_TX_UBUFF_A0.BYTE1</code> bit field contains user bit data for byte 1 of sub frame A.
7:0 (R/W)	BYTE0	Byte 0 Sub Frame A. The <code>SPDIF_TX_UBUFF_A0.BYTE0</code> bit field contains user bit data for byte 0 of sub frame A.

Transmit User Buffer A1 Register

The `SPDIF_TX_UBUFF_A1` register holds the transmit user buffer data for bytes 4-7 for sub frame A.

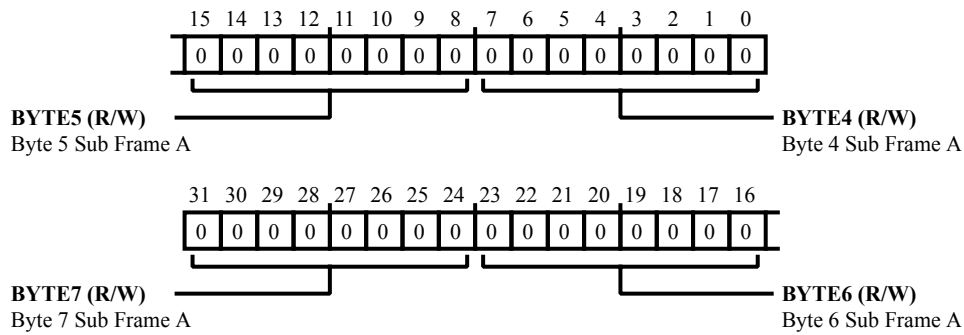


Figure 24-28: SPDIF_TX_UBUFF_A1 Register Diagram

Table 24-25: SPDIF_TX_UBUFF_A1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	BYTE7	Byte 7 Sub Frame A. The <code>SPDIF_TX_UBUFF_A1</code> .BYTE7 bit field contains user bit data for byte 7 of sub frame A.
23:16 (R/W)	BYTE6	Byte 6 Sub Frame A. The <code>SPDIF_TX_UBUFF_A1</code> .BYTE6 bit field contains user bit data for byte 6 of sub frame A.
15:8 (R/W)	BYTE5	Byte 5 Sub Frame A. The <code>SPDIF_TX_UBUFF_A1</code> .BYTE5 bit field contains user bit data for byte 5 of sub frame A.
7:0 (R/W)	BYTE4	Byte 4 Sub Frame A. The <code>SPDIF_TX_UBUFF_A1</code> .BYTE4 bit field contains user bit data for byte 0 of sub frame A.

Transmit User Buffer A2 Register

The `SPDIF_TX_UBUFF_A2` register holds the transmit user buffer data for bytes 8-11 for sub frame A.

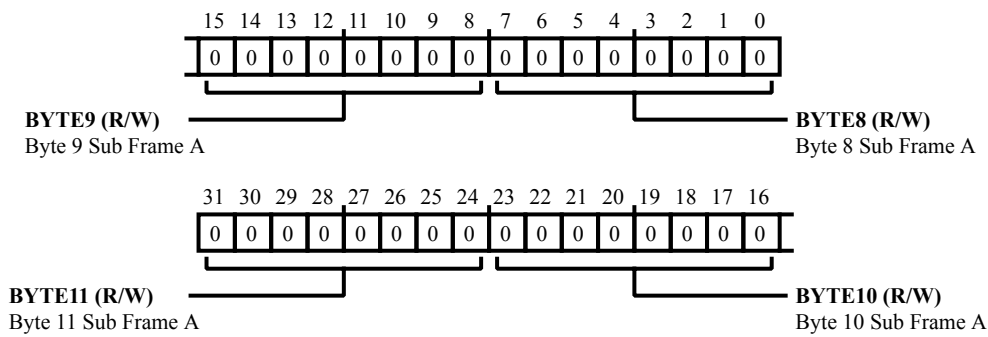


Figure 24-29: SPDIF_TX_UBUFF_A2 Register Diagram

Table 24-26: SPDIF_TX_UBUFF_A2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	BYTE11	Byte 11 Sub Frame A. The <code>SPDIF_TX_UBUFF_A2.BYTE11</code> bit field contains user bit data for byte 11 of sub frame A.
23:16 (R/W)	BYTE10	Byte 10 Sub Frame A. The <code>SPDIF_TX_UBUFF_A2.BYTE10</code> bit field contains user bit data for byte 10 of sub frame A.
15:8 (R/W)	BYTE9	Byte 9 Sub Frame A. The <code>SPDIF_TX_UBUFF_A2.BYTE9</code> bit field contains user bit data for byte 9 of sub frame A.
7:0 (R/W)	BYTE8	Byte 8 Sub Frame A. The <code>SPDIF_TX_UBUFF_A2.BYTE8</code> bit field contains user bit data for byte 8 of sub frame A.

Transmit User Buffer A3 Register

The `SPDIF_TX_UBUFF_A3` register holds the transmit user buffer data for bytes 12-15 for sub frame A.

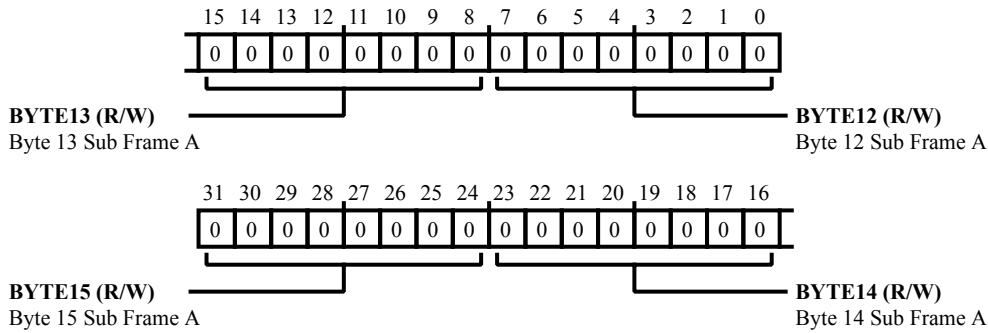


Figure 24-30: SPDIF_TX_UBUFF_A3 Register Diagram

Table 24-27: SPDIF_TX_UBUFF_A3 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	BYTE15	Byte 15 Sub Frame A. The <code>SPDIF_TX_UBUFF_A3.BYTE15</code> bit field contains user bit data for byte 15 of sub frame A.
23:16 (R/W)	BYTE14	Byte 14 Sub Frame A. The <code>SPDIF_TX_UBUFF_A3.BYTE14</code> bit field contains user bit data for byte 14 of sub frame A.
15:8 (R/W)	BYTE13	Byte 13 Sub Frame A. The <code>SPDIF_TX_UBUFF_A3.BYTE13</code> bit field contains user bit data for byte 13 of sub frame A.
7:0 (R/W)	BYTE12	Byte 12 Sub Frame A. The <code>SPDIF_TX_UBUFF_A3.BYTE12</code> bit field contains user bit data for byte 12 of sub frame A.

Transmit User Buffer A4 Register

The `SPDIF_TX_UBUFF_A4` register holds the transmit user buffer data for bytes 16-19 for sub frame A.

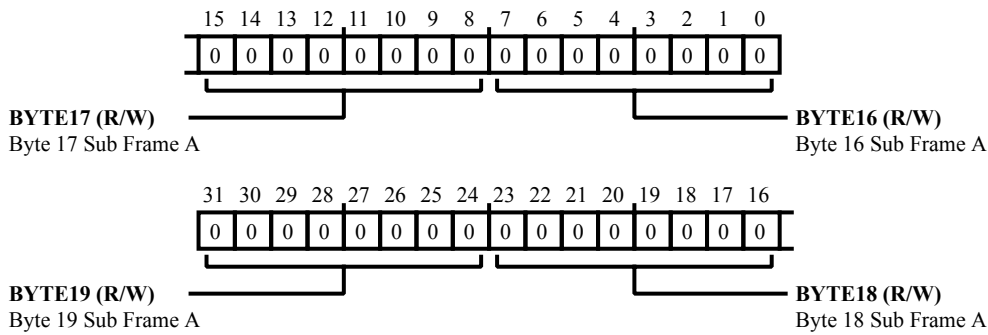


Figure 24-31: SPDIF_TX_UBUFF_A4 Register Diagram

Table 24-28: SPDIF_TX_UBUFF_A4 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	BYTE19	Byte 19 Sub Frame A. The <code>SPDIF_TX_UBUFF_A4.BYTE19</code> bit field contains user bit data for byte 19 of sub frame A.
23:16 (R/W)	BYTE18	Byte 18 Sub Frame A. The <code>SPDIF_TX_UBUFF_A4.BYTE18</code> bit field contains user bit data for byte 18 of sub frame A.
15:8 (R/W)	BYTE17	Byte 17 Sub Frame A. The <code>SPDIF_TX_UBUFF_A4.BYTE17</code> bit field contains user bit data for byte 17 of sub frame A.
7:0 (R/W)	BYTE16	Byte 16 Sub Frame A. The <code>SPDIF_TX_UBUFF_A4.BYTE16</code> bit field contains user bit data for byte 16 of sub frame A.

Transmit User Buffer A5 Register

The `SPDIF_TX_UBUFF_A5` register holds the transmit user buffer data for bytes 20-23 for sub frame A.

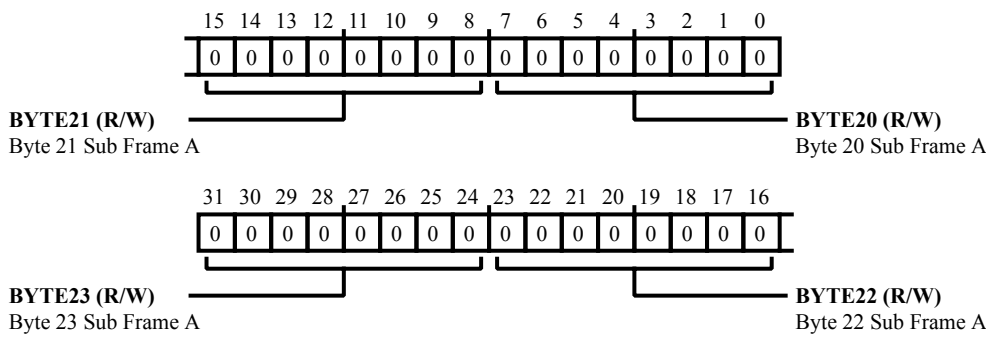


Figure 24-32: SPDIF_TX_UBUFF_A5 Register Diagram

Table 24-29: SPDIF_TX_UBUFF_A5 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	BYTE23	Byte 23 Sub Frame A. The <code>SPDIF_TX_UBUFF_A5.BYTE23</code> bit field contains user bit data for byte 23 of sub frame A.
23:16 (R/W)	BYTE22	Byte 22 Sub Frame A. The <code>SPDIF_TX_UBUFF_A5.BYTE22</code> bit field contains user bit data for byte 22 of sub frame A.
15:8 (R/W)	BYTE21	Byte 21 Sub Frame A. The <code>SPDIF_TX_UBUFF_A5.BYTE21</code> bit field contains user bit data for byte 21 of sub frame A.
7:0 (R/W)	BYTE20	Byte 20 Sub Frame A. The <code>SPDIF_TX_UBUFF_A5.BYTE20</code> bit field contains user bit data for byte 20 of sub frame A.

Transmit User Buffer B0 Register

The `SPDIF_TX_UBUFF_B0` register holds the transmit user buffer data for bytes 0-3 for sub frame B.

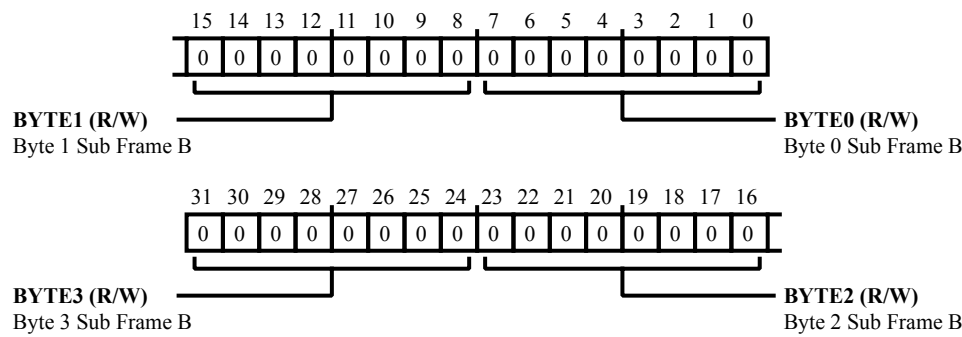


Figure 24-33: SPDIF_TX_UBUFF_B0 Register Diagram

Table 24-30: SPDIF_TX_UBUFF_B0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	BYTE3	Byte 3 Sub Frame B. The <code>SPDIF_TX_UBUFF_B0.BYTE3</code> bit field contains user bit data for byte 3 of sub frame B.
23:16 (R/W)	BYTE2	Byte 2 Sub Frame B. The <code>SPDIF_TX_UBUFF_B0.BYTE2</code> bit field contains user bit data for byte 2 of sub frame B.
15:8 (R/W)	BYTE1	Byte 1 Sub Frame B. The <code>SPDIF_TX_UBUFF_B0.BYTE1</code> bit field contains user bit data for byte 1 of sub frame B.
7:0 (R/W)	BYTE0	Byte 0 Sub Frame B. The <code>SPDIF_TX_UBUFF_B0.BYTE0</code> bit field contains user bit data for byte 0 of sub frame B.

Transmit User Buffer B1 Register

The `SPDIF_TX_UBUFF_B1` register holds the transmit user buffer data for bytes 4-7 for sub frame B.

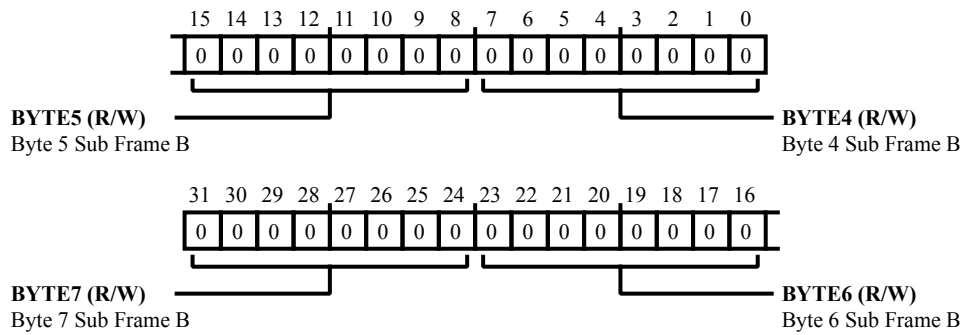


Figure 24-34: SPDIF_TX_UBUFF_B1 Register Diagram

Table 24-31: SPDIF_TX_UBUFF_B1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	BYTE7	Byte 7 Sub Frame B. The <code>SPDIF_TX_UBUFF_B1</code> . <code>BYTE7</code> bit field contains user bit data for byte 7 of sub frame B.
23:16 (R/W)	BYTE6	Byte 6 Sub Frame B. The <code>SPDIF_TX_UBUFF_B1</code> . <code>BYTE6</code> bit field contains user bit data for byte 6 of sub frame B.
15:8 (R/W)	BYTE5	Byte 5 Sub Frame B. The <code>SPDIF_TX_UBUFF_B1</code> . <code>BYTE5</code> bit field contains user bit data for byte 5 of sub frame B.
7:0 (R/W)	BYTE4	Byte 4 Sub Frame B. The <code>SPDIF_TX_UBUFF_B1</code> . <code>BYTE4</code> bit field contains user bit data for byte 4 of sub frame B.

Transmit User Buffer B2 Register

The `SPDIF_TX_UBUFF_B2` register holds the transmit user buffer data for bytes 8-11 for sub frame B.

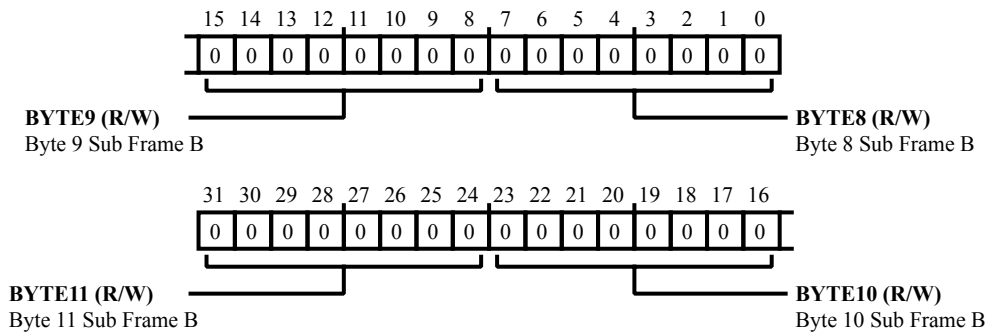


Figure 24-35: SPDIF_TX_UBUFF_B2 Register Diagram

Table 24-32: SPDIF_TX_UBUFF_B2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	BYTE11	Byte 11 Sub Frame B. The <code>SPDIF_TX_UBUFF_B2.BYTE11</code> bit field contains user bit data for byte 11 of sub frame B.
23:16 (R/W)	BYTE10	Byte 10 Sub Frame B. The <code>SPDIF_TX_UBUFF_B2.BYTE10</code> bit field contains user bit data for byte 10 of sub frame B.
15:8 (R/W)	BYTE9	Byte 9 Sub Frame B. The <code>SPDIF_TX_UBUFF_B2.BYTE9</code> bit field contains user bit data for byte 9 of sub frame B.
7:0 (R/W)	BYTE8	Byte 8 Sub Frame B. The <code>SPDIF_TX_UBUFF_B2.BYTE8</code> bit field contains user bit data for byte 8 of sub frame B.

Transmit User Buffer B3 Register

The `SPDIF_TX_UBUFF_B3` register holds the transmit user buffer data for bytes 12-15 for sub frame B.

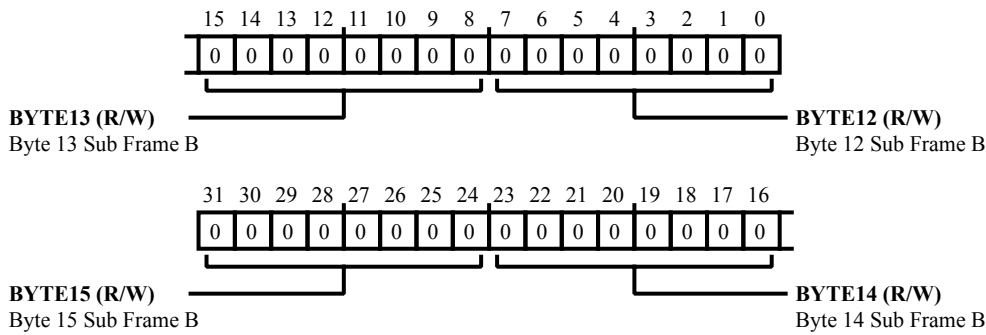


Figure 24-36: SPDIF_TX_UBUFF_B3 Register Diagram

Table 24-33: SPDIF_TX_UBUFF_B3 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	BYTE15	Byte 15 Sub Frame B. The <code>SPDIF_TX_UBUFF_B3.BYTE15</code> bit field contains user bit data for byte 15 of sub frame B.
23:16 (R/W)	BYTE14	Byte 14 Sub Frame B. The <code>SPDIF_TX_UBUFF_B3.BYTE14</code> bit field contains user bit data for byte 14 of sub frame B.
15:8 (R/W)	BYTE13	Byte 13 Sub Frame B. The <code>SPDIF_TX_UBUFF_B3.BYTE13</code> bit field contains user bit data for byte 13 of sub frame B.
7:0 (R/W)	BYTE12	Byte 12 Sub Frame B. The <code>SPDIF_TX_UBUFF_B3.BYTE12</code> bit field contains user bit data for byte 12 of sub frame B.

Transmit User Buffer B4 Register

The `SPDIF_TX_UBUFF_B4` register holds the transmit user buffer data for bytes 16-19 for sub frame B.

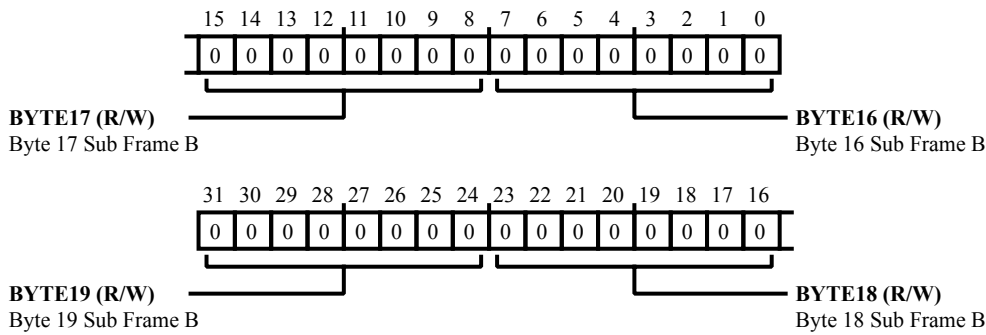


Figure 24-37: SPDIF_TX_UBUFF_B4 Register Diagram

Table 24-34: SPDIF_TX_UBUFF_B4 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	BYTE19	Byte 19 Sub Frame B. The <code>SPDIF_TX_UBUFF_B4.BYTE19</code> bit field contains user bit data for byte 19 of sub frame B.
23:16 (R/W)	BYTE18	Byte 18 Sub Frame B. The <code>SPDIF_TX_UBUFF_B4.BYTE18</code> bit field contains user bit data for byte 18 of sub frame B.
15:8 (R/W)	BYTE17	Byte 17 Sub Frame B. The <code>SPDIF_TX_UBUFF_B4.BYTE17</code> bit field contains user bit data for byte 17 of sub frame B.
7:0 (R/W)	BYTE16	Byte 16 Sub Frame B. The <code>SPDIF_TX_UBUFF_B4.BYTE16</code> bit field contains user bit data for byte 16 of sub frame B.

Transmit User Buffer B5 Register

The `SPDIF_TX_UBUFF_B5` register holds the transmit user buffer data for bytes 20-23 for sub frame B.

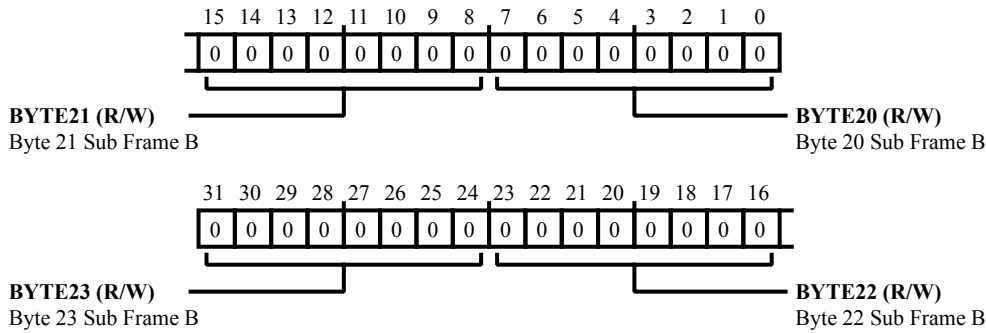


Figure 24-38: SPDIF_TX_UBUFF_B5 Register Diagram

Table 24-35: SPDIF_TX_UBUFF_B5 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	BYTE23	Byte 23 Sub Frame B. The <code>SPDIF_TX_UBUFF_B5.BYTE23</code> bit field contains user bit data for byte 23 of sub frame B.
23:16 (R/W)	BYTE22	Byte 22 Sub Frame B. The <code>SPDIF_TX_UBUFF_B5.BYTE22</code> bit field contains user bit data for byte 22 of sub frame B.
15:8 (R/W)	BYTE21	Byte 21 Sub Frame B. The <code>SPDIF_TX_UBUFF_B5.BYTE21</code> bit field contains user bit data for byte 21 of sub frame B.
7:0 (R/W)	BYTE20	Byte 20 Sub Frame B. The <code>SPDIF_TX_UBUFF_B5.BYTE20</code> bit field contains user bit data for byte 20 of sub frame B.

User Bit Update Register

After writing to the transmit user buffer registers, a value of 0x1 must be written into the `SPDIF_TX_USRUPDT` register to enable the use of these user buffer bits in the next transfer block.

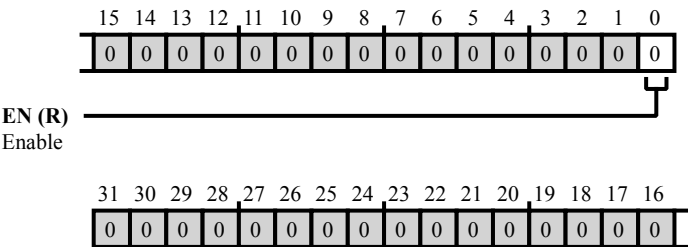


Figure 24-39: SPDIF_TX_USRUPDT Register Diagram

Table 24-36: SPDIF_TX_USRUPDT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/NW)	EN	Enable. After writing to the transmit user buffer registers, a value of 0x1 must be written into the <code>SPDIF_TX_USRUPDT</code> register to enable the use of these <code>SPDIF_TX_USRUPDT</code> .EN bits in the next transfer block.

25 Direct Memory Access (DMA)

The processor architecture distributes the DMA channels throughout the infrastructure. Often, the channels cluster together through system crossbars (SCB), sharing a single interface with the main system crossbar.

The DMA channels can perform transfers between memory and a peripheral or between one memory and another memory. Memory-to-memory DMA transfers (MDMA) require two DMA channels. One channel is the source channel, and the second, the destination channel.

All DMA channels can transport data to and from virtually all on-chip and off-chip memories.

DMA transfers on the processor use either a descriptor-based method or register-based method. Register-based DMA allows the processor directly to program DMA controller registers to initiate a DMA transfer. On completion, the controller registers can automatically update with their original setup values for continuous transfer, if needed. Descriptor-based DMA transfers require a set of parameters stored within memory to initiate a DMA sequence. Descriptor-based transfers allow the chaining together of multiple DMA sequences. In descriptor-based DMA operations, DMA channel programming can automatically set up and start another DMA transfer after the current sequence completes.

The DMA channel does not connect external memories and devices directly. Rather, data passes through an external-memory interface port. DMA operations can access any device the external memory interface supports. These interfaces typically include:

- Flash memory
- SRAM
- FIFOs
- Memory-mapped peripheral devices
- Dynamic Memory (if present)

DMA Channel Features

The processor uses Direct Memory Access (DMA) to transfer data within memory spaces or between a memory space and a peripheral. The processor can specify data transfer operations and return to normal processing while the fully integrated DMA channel carries out the data transfers independent of processor activity. The DMA channels are dispersed throughout the infrastructure and interface with the system crossbar unit (SCB).

The following is a list of DMA interface features.

- Supports integer byte strides including byte strides of 0 and negative byte strides
- Register-based configuration
 - Core writes DMA configuration
 - Supports automatic reloading for continuous operation
- Flexible descriptor-based configuration
 - DMA descriptors are fetched from memory
 - Support for variable descriptor sizes
- Flexible flow control – Transitions between the various descriptor-based modes and for DMA termination
- Orthogonal transfers
 - Support for three transfer dimensions
 - One and two dimensional (1D and 2D) transfers supported per descriptor set
 - Three dimensional (3D) support provided by chained descriptor sets
- Configurable memory and peripheral-transfer word sizes
 - Memory interface supports 8-bit, 16-bit, 32-bit, 64-bit, and 128-bit transfers
 - Peripheral interface supports for 8-bit, 16-bit, and 32-bit transfers
- Interrupt notification
 - Row or work unit completion
 - Error conditions
- Incoming and outgoing trigger support
 - Trigger generation for row or work unit completion
 - Work unit can wait for incoming trigger
- MMR access bus – Provides access to memory-mapped registers for configuration, monitoring, and debug
- SCB crossbar interface connects the DMA channel to the system crossbar
- Peripheral DMA bus – Interfaces the DMA channel to a peripheral or another DMA channel
- Peripheral data-request interrupt support
- Bandwidth monitoring and limiting for MDMA channels

DMA Channel Functional Description

This section provides a functional description of the DMA channel interface.

NOTE: There are two types of peripherals that use DMA. The first have dedicated DMA channels controlled by the Dedicated DMA Engine (DDE) and are described in this chapter. The second type is not controlled by the DDE. These peripherals have their own operating modes and programming models (see the peripheral chapter for this information). The complete list of DMA supported peripherals are shown in [Table 37-5 SCB Controlled DMA Channel Peripherals](#).

ADSP-21560_21561_21564_21568 DMA Register List

The DMA channel controller (DMA) supports data transfers within memory spaces or between a memory space and a peripheral. The processor can specify data transfer operations and return to normal processing while the fully integrated DMA channel carries out the data transfers independent of processor activity. The DMA channels are dispersed throughout the infrastructure, as DMA's. A set of registers governs DMA operations. For more information on DMA functionality, see the DMA register descriptions.

Table 25-1: ADSP-21560_21561_21564_21568 DMA Register List

Name	Description
DMA_ADDRSTART	Start Address of Current Buffer Register
DMA_ADDR_CUR	Current Address Register
DMA_BWLCNT	Bandwidth Limit Count Register
DMA_BWLCNT_CUR	Bandwidth Limit Count Current Register
DMA_BWMCNT	Bandwidth Monitor Count Register
DMA_BWMCNT_CUR	Bandwidth Monitor Count Current Register
DMA_CFG	Configuration Register
DMA_DSCPTR_CUR	Current Descriptor Pointer Register
DMA_DSCPTR_NXT	Pointer to Next Initial Descriptor Register
DMA_DSCPTR_PRV	Previous Initial Descriptor Pointer Register
DMA_STAT	Status Register
DMA_XCNT	Inner Loop Count Start Value Register
DMA_XCNT_CUR	Current Count (1D) or Intra-row XCNT (2D) Register
DMA_XMOD	Inner Loop Address Increment Register
DMA_YCNT	Outer Loop Count Start Value (2D only) Register
DMA_YCNT_CUR	Current Row Count (2D only) Register
DMA_YMOD	Outer Loop Address Increment (2D only) Register

ADSP-21560_21561_21564_21568 DMA Channel List

Table 25-2: ADSP-21560_21561_21564_21568 DMA Channel List

DMA ID	DMA Channel Name	Description
DMA0	SPORT0_A_DMA	SPORT0 Channel A DMA
DMA1	SPORT0_B_DMA	SPORT0 Channel B DMA
DMA2	SPORT1_A_DMA	SPORT1 Channel A DMA
DMA3	SPORT1_B_DMA	SPORT1 Channel B DMA
DMA4	SPORT2_A_DMA	SPORT2 Channel A DMA
DMA5	SPORT2_B_DMA	SPORT2 Channel B DMA
DMA6	SPORT3_A_DMA	SPORT3 Channel A DMA
DMA7	SPORT3_B_DMA	SPORT3 Channel B DMA
DMA8	MDMA0_SRC	Memory DMA Stream n Source Channel
DMA9	MDMA0_DST	Memory DMA Stream n Destination Channel
DMA10	SPORT4_A_DMA	SPORT4 Channel A DMA
DMA11	SPORT4_B_DMA	SPORT4 Channel B DMA
DMA12	SPORT5_A_DMA	SPORT5 Channel A DMA
DMA13	SPORT5_B_DMA	SPORT5 Channel B DMA
DMA14	SPORT6_A_DMA	SPORT6 Channel A DMA
DMA15	SPORT6_B_DMA	SPORT6 Channel B DMA
DMA16	SPORT7_A_DMA	SPORT7 Channel A DMA
DMA17	SPORT7_B_DMA	SPORT7 Channel B DMA
DMA18	MDMA1_SRC	Memory DMA Stream n Source Channel
DMA19	MDMA1_DST	Memory DMA Stream n Destination Channel
DMA20	UART0_TXDMA	UART0 Transmit DMA
DMA21	UART0_RXDMA	UART0 Receive DMA
DMA22	SPI0_TXDMA	SPI0 TX DMA Channel
DMA23	SPI0_RXDMA	SPI0 RX DMA Channel
DMA24	SPI1_TXDMA	SPI1 Transmit DMA
DMA25	SPI1_RXDMA	SPI1 RX DMA Channel
DMA26	SPI2_TXDMA	SPI2 TX DMA Channel

Table 25-2: ADSP-21560_21561_21564_21568 DMA Channel List (Continued)

DMA ID	DMA Channel Name	Description
DMA27	SPI2_RXDMA	SPI2 RX DMA Channel
DMA34	UART1_TXDMA	UART1 Transmit DMA
DMA35	UART1_RXDMA	UART1 Receive DMA
DMA39	MDMA2_SRC	Memory DMA Stream n Source Channel
DMA40	MDMA2_DST	Memory DMA Stream n Destination Channel
DMA43	MDMA3_SRC	Memory DMA Stream n Source Channel
DMA44	MDMA3_DST	Memory DMA Stream n Destination Channel

DMA Definitions

To make the best use of the DMA controller, it is useful to understand the following terms.

Descriptor

A single element of a descriptor set that maps to a specific register of a DMA channel.

Descriptor Fetch

The action of retrieving descriptors from memory through memory read operations and loading them into the DMA channel registers upon their read return.

Descriptor Set

An array of descriptors associated with a single work unit. The user can configure the size of the descriptor set. See [Descriptor-Based Flow Modes](#).

Disabled State

The channel is disabled because the enable bit = 0 or because of an error.

DMAC

An acronym used for a DMA cluster.

DMA Channel

A single DMA engine that has all the capabilities and registers as defined for a given processor. A DMA channel or engine is connected to a single peripheral.

DMA Cluster

A grouping of multiple DMA channels with a shared SCB crossbar interface, controller, and arbiter. Also known as a DMAC.

Initial Descriptor

The first descriptor in the descriptor set.

MDMA

Memory-to-Memory DMA data transfer. Two DMA channels are paired to perform a memory read from one address location and a memory write of that data to another address location.

Stop State

A time where the channel is enabled but not currently programmed to perform a data transfer. Programming the flow to STOP causes the channel to enter the stop state at the end of the work unit.

User

Any person, debug, emulator, software routine, or action taken by the core that accesses the MMR registers of the DMA channel or peripherals, or sets up data and descriptors in memory.

Wait State

If instructed to wait for a trigger, the channel enters this state once it has completed a work unit. The channel remains in this state until a trigger occurs. If a trigger came in before reaching the wait state, the channel skips over the wait state upon completion of the work unit.

Work Unit

A single data transaction or series of data transactions performed based on the configuration of the DMA channel. For autobuffer mode, a new work unit is defined at the time all current count registers are initialized to start values. Once all the current count registers count down to zero, the work unit has completed.

Work Unit Chain

A single work unit or a series of work units separated by a STOP or disabled state. The work units in the chain (except the last one) are programmed to the required descriptor flow. The last work unit in the chain is programmed

to a flow of STOP or AUTO. STOP terminates the state at the end of that work unit. AUTO must be terminated by disabling the DMA channel. A work unit chain is also known as a descriptor chain.

Block Diagram

The *DMA Channel Block Diagram* shows the functional blocks within the DMA interface.

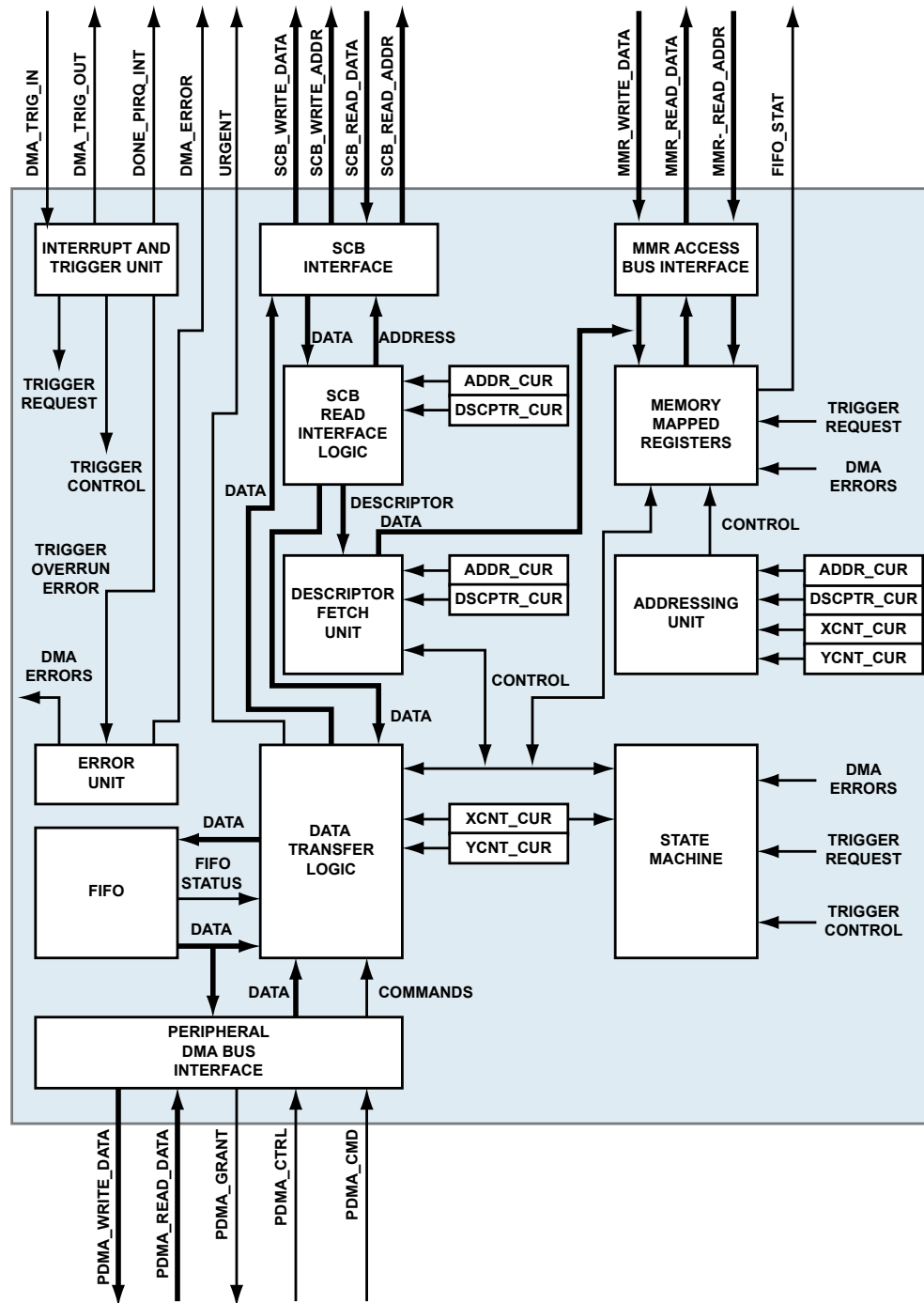


Figure 25-1: DMA Channel Block Diagram

For more information on the interfaces, see:

- [DMA Channel Peripheral DMA Bus](#)
- [Medium Band Width DMA Channel MMR Access Bus](#)
- [DMA Channel Event Control](#)
- [DMA Channel SCB Interface](#)

Architectural Concepts

The DMA channel provides a method to transfer data between memory spaces or between memory and a peripheral using a number of system interfaces. The DMA channel provides an efficient method of distributing data throughout the system, freeing up the processor core for other operations. Each peripheral that supports DMA transfers has its own dedicated DMA channel or channels with its own register set. The register set configures and controls the operating modes of the DMA transfers.

DMA Channel SCB Interface

The SCB interface connects the DMA channel to the SCB crossbar allowing for transfers to and from the processors internal memory and other suitable system resources.

The DMA channel connects to the system interconnect through the SCB interface. This connection lets the DMA channel perform work-unit data transfers with memories such as L1 and L2 (internal). In addition to work unit data transfers, the SCB interface also is used for fetching descriptor sets for all the descriptor-based transfer modes.

The DMA channel can support data bus widths of 16, 32, 64, or 128 bits. The data bus widths for a given DMA channel on a specific processor can vary and are not configurable. Read the `DMA_STAT.MBWID` field to determine the assigned bus widths.

SCB Interface Signals

The DMA channel operates at one of the *SCLK_n* frequencies, as does the SCB interface. *SCLK0* clocks all but four DMA channels which are clocked by *SCLK1*. The SCB crossbar handles the internal arbitration of the transfer requests of all the controllers interfaced to the SCB crossbar instance as shown in the *SCB Interface Signals* table.

Table 25-3: SCB Interface Signals

Signal	Width (bits)	Description
SCB_WRITE_DATA	16, 32, 64, or 128	Data bus used for write operations. The width of the bus can be determined from <code>DMA_STAT.MBWID</code> .
SCB_WRITE_ADDRESS	32	Write address bus. Provides the address of the first transfer in a burst transaction.
SCB_READ_DATA	16, 32, 64, or 128	Data bus used for read operations. The width of the bus can be determined from <code>DMA_STAT.MBWID</code> .
SCB_READ_ADDRESS	32	Read address bus. Provides the address of the first transfer in a burst transaction.

SCB Burst Transfers

The SCB interface supports burst transfers for memory read and write operations. The burst length is a function of the configurable memory size of the DMA channel for the work unit and the fixed bus width of the SCB data bus of the DMA channel.

- If the DMA channel configuration selects a memory transfer size less than or equal to the DMA channels bus width, the burst length is always 1.
- If the configured memory size is greater than the SCB interface bus width, the burst length is sufficient to transfer a transaction as specified by the configured memory size.

Table 25-4: DMA Channel SCB Burst Lengths

Configured Memory Size	Burst Length			
	<i>16-bit Bus</i>	<i>32-bit Bus</i>	<i>64-Bit Bus</i>	<i>128-bit Bus</i>
1 Byte	1	1	1	1
2 Bytes	1	1	1	1
4 Bytes	2	1	1	1
8 Bytes	4	2	1	1
16 Bytes	8	4	2	1
32 Bytes	16	8	4	2

Data Address Alignment

To prevent addressing errors and to maximize bandwidth of the SCB interface to the DMA channel, data addresses align with a multiple of the programmable memory size of the DMA channels configuration. These configuration options appear in the [Descriptor Set Address Alignment](#) table.

There are situations in which entire work units may not transfer at the maximum configurable memory size. In this case, the entire work unit can transfer by reducing the configured memory size at the expense of bus bandwidth using descriptor sets as follows:

- The first descriptor set can be configured to transfer data until the larger memory size alignments are met.
- A second descriptor set with a larger memory size configuration then can be used to transfer the bulk of the data in the work unit.
- Finally, a third descriptor set can be used with a smaller memory size to complete any final data transfers that cannot meet the alignment requirements of the previous descriptor set configuration.

Table 25-5: DMA Channel Address Alignment Requirements

Configured Memory Size	Address Restriction
1 Byte	No restriction
2 Bytes	ADDR[0] == 0

Table 25-5: DMA Channel Address Alignment Requirements (Continued)

Configured Memory Size	Address Restriction
4 Bytes	ADDR[1:0] == 0
8 Bytes	ADDR[2:0] == 0
16 Bytes	ADDR[3:0] == 0
32 Bytes	ADDR[4:0] == 0

Descriptor Set Address Alignment

All descriptor set addresses and descriptors within a descriptor set must align to a 32-bit address. For descriptor set fetches, the DMA engine ignores the memory-size configuration of the DMA channel. This feature avoids the need to align descriptor sets based on the memory width configuration of the previous descriptor set.

For descriptor sets containing only a single descriptor, the transfer takes place as a single 32-bit transfer. For descriptor sets containing multiple descriptors, the DMA engine fetches each 32-bit descriptor individually and treats it as multiple 32-bit transfers.

Peripheral Control Commands

The peripheral DMA bus of the DMA channel provides a means for peripherals on the processor to issue commands to the DMA channel. These commands provide greater control over the DMA channel operation. This control improves real-time performance and relieves control and interrupt demands on the core. Peripherals can send commands to the DMA controller over the 3-bit PERI_CMD bus. The DMA control commands extend the set of operations available to the peripheral beyond the simple “request data” command used by peripherals in general. Refer to the appropriate peripheral chapter for a description on how that peripheral uses DMA control commands.

These DMA control commands (see the *PDMA_CMD Peripheral DMA Control Commands* table) are not visible to or controlled by the program. But, their use by a peripheral has implications for the structure of the DMA transfers that the peripheral can support. It is important to write application software such that it complies with certain restrictions, regarding work units and descriptor chains. Complying with this guideline makes the peripheral operate properly whenever it issues DMA control commands.

The *PDMA_CMD Peripheral DMA Control Commands* table describes the commands the DMA controller issues. The following sections describe these commands in more detail.

Table 25-6: PDMA_CMD Peripheral DMA Control Commands

Command	Name	Description
b#000	NOP	No operation
b#001	Restart	Restarts the current work unit from the beginning
b#010	Finish	Finishes the current work unit and starts the next
b#011	Interrupt	Immediately sets the DMA completion interrupt in the DMA channel

Table 25-6: PDMA_CMD Peripheral DMA Control Commands (Continued)

Command	Name	Description
b#100	Request Data	Typical DMA data request
b#101	Request Data Urgent	Urgent DMA data request
b#110	Reserved	Reserved
b#111	Reserved	Reserved

Idle Command

The DMA channel drives this command when the enabled peripheral has no data requests required.

Request-Data Command

The request data command is a request for data transfers between the DMA channel and the peripheral. The request is held by the peripheral until granted or acknowledged by the DMA channel.

Request-Data Urgent Command

The request-data urgent command behaves identically to the request data command, except that---during the commands assertion---the DMA channel performs its memory accesses with urgent priority. This priority includes both data and descriptor fetch memory accesses. For example, a DMA management capable peripheral can use this control command if an internal FIFO approaches a critical condition.

The request is held by the peripheral until granted or acknowledged by the DMA channel.

Peripheral-Control Command Restrictions

The proper operation of the DMA channel FIFO leads to certain restrictions in the sequence of DMA peripheral control commands issued by a peripheral. The following sections describe these restrictions.

DMA Channel Peripheral DMA Bus

The DMA channel connects to peripherals or other DMA channels through the peripheral DMA bus. This bus is a dedicated point-to-point interface supporting data bus widths of 8, 16, 32, or 64 bits. The data bus widths for a given DMA channel on a particular processor can vary and are not configurable. Reading the `DMA_STAT.PBWID` field permits determining the assigned bus width.

The DMA channel operates at one of the SCLK frequencies, as does the peripheral DMA bus. The *Peripheral DMA Bus Signals* table provides descriptions of the peripheral DMA bus signals.

Table 25-7: Peripheral DMA Bus Signals

Signal	Width (bits)	Description
PDMA_WRITE_DATA	8, 16, 32, or 64	Data bus used for write operations. The width of the bus can be determined from <code>DMA_STAT.PBWID</code> .
PDMA_READ_DATA	8, 16, 32, or 64	Data bus used for read operations. The width of the bus can be determined from <code>DMA_STAT.PBWID</code> .

Table 25-7: Peripheral DMA Bus Signals (Continued)

Signal	Width (bits)	Description
PDMA_DMA_GRANT		Control signals to indicate that data is valid for DMA channel read operations (peripheral transmit). These signals indicate that the DMA channel is ready to receive data for write operations (peripheral receive).
PDMA_CMD	3	The peripheral uses the signal for issuing DMA channel control commands.
PDMA_CTRL		The peripheral uses the control signals to send various commands to the DMA channel and control the direction of flow.

Memory DMA and Triggering

A memory DMA (MDMA) channel provides a means of doing memory-to-memory DMA transfers among the various memory spaces that have DMA support.

The DMA controller implements memory DMA (MDMA) channels by interfacing two DMA channels through the peripheral DMA bus interface. One DMA channel serves for memory read operations, and the second channel serves for memory writes. Depending on the processor, a memory DMA channel can have an additional peripheral, such as a CRC peripheral. The additional peripheral is inserted into the peripheral DMA bus that optionally can be enabled.

MDMA channel configurations that do not involve an additional peripheral impose no restrictions on which of the DMA channels is used for the read operation or the write operation. But, the configuration of both channels cannot have the same transfer direction. For MDMA channel configurations that enable a peripheral between the read and write channels, be aware of possible restrictions imposed on which channel can be used for a given transfer direction.

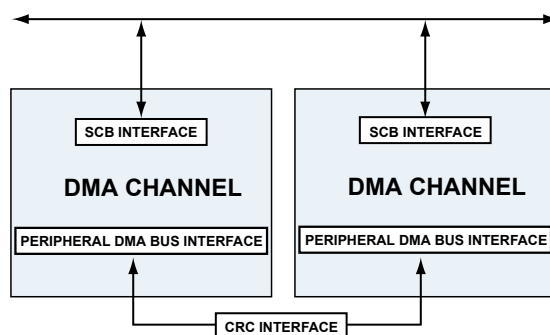


Figure 25-2: MDMA Channel Dedicated Pair

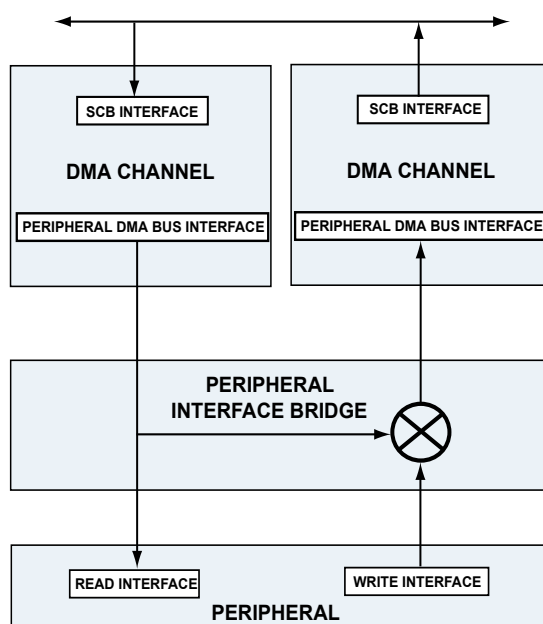


Figure 25-3: MDMA Channel Pair with Peripheral

A memory-to-memory transfer always requires enabled source and destination channels. Because the channels interface through the peripheral DMA bus and can have an additional peripheral inserted into the peripheral DMA bus, programs must make sure to set the same values in the `DMA_CFG.PSIZE` of both the source and destination channels.

The memory DMA channels support the full range of the `DMA_CFG.MSIZE` options for the DMA transfers to and from the memories.

Because the MDMA channel consists of two DMA channels, the entire MDMA channel has two sets of FIFOs, one in the read channel and one in the write channel. This FIFO usage allows for more efficient bursting of both read and write transactions using the available bandwidth. While the `DMA_CFG.PSIZE` configuration must be identical for both source and destination DMA channels, this restriction does not apply for the `DMA_CFG.MSIZE` configuration.

Configure the `DMA_CFG.PSIZE` bits to a value no larger than the supported bus width of the peripheral DMA bus. From a performance perspective, use the largest possible `DMA_CFG.PSIZE` value (for example, equal to the supported peripheral bus width (`DMA_STAT.PBWID`)). However, ensure that the number of bytes in the work unit is a multiple of the `PSIZE` value used.

NOTE: This is applicable for all of the DMA channels, except the MDMA channels 2 and 3. For the MDMA 2 and 3, the minimum *PSIZE* and *MSIZE* is 4 bytes (32 bits).

The independent source and destination DMA channels also have their own dedicated interrupt and trigger events. While it is normal practice to have only event generation performed at destination DMA completion, programs also can use other means of interrupt generation.

Configuration of an MDMA transfer is done in a similar manner to peripheral DMA transfers, except for writing two DMA channel registers instead of one.

To control the pace of data transfers, use triggers on either the memory read or the memory write channel pair used in an MDMA operation. Setting the `DMA_CFG.TWAIT` bit in the memory read channel prevents both channels from transferring data before the system is ready. However, only configuring the memory write channel to wait for a trigger allows for data fetch from the memory in anticipation of the memory write operation.

The processor supports these categories of MDMA channels:

Enhanced Bandwidth MDMA. The channels run in SYSCLK domain and support 32-bit memory and peripheral bus width. The maximum theoretical bandwidth of this MDMA stream is up to $4 \times 500 = 2000$ MB/s.

Maximum Bandwidth MDMA. The channels run in SYSCLK domain and support 64-bit memory and peripheral bus width. The maximum theoretical bandwidth of these MDMA streams is up to $8 \times 500 = 4000$ MB/s.

Medium Band Width DMA Channel MMR Access Bus

The MMR access bus provides access to all the DMA channels memory-mapped registers for DMA channel configuration, monitoring, and debug. The interface has a fixed 32-bit data bus for read and write accesses.

The *MMR Access Bus Signals* table provides descriptions of the MMR access bus signals.

Table 25-8: MMR Access Bus Signals

Signal	Width (bits)	Description
MMR_WRITE_DATA	32	Data bus used for write operations to the MMRs from the core
MMR_READ_DATA	32	Data bus used to return read data from the MMRs
MMR_READ_ADDR	7	Address used to select the MMR to access

DMA Channel Operation Flow

A detailed description of the flow of operation of the DMA channel appears in the following topics:

- [Startup Flow](#)
- [Refresh Flow](#)
- [DMA Operating Modes](#)
- [Stop Mode](#)
- [DMA Channel Errors](#)

Startup Flow

Enabling a DMA operation on a given channel first requires directly writing some or all of the DMA parameter registers. The minimum set of register required to be initialized depends on the desired mode of operation as described in the following sections.

Startup Minimum-Enable Requirements

To start a DMA operation on a given channel, some or all of the DMA parameter registers must first be initialized and configured to the desired DMA channels operating mode.

- For descriptor-array-based flow modes, at minimum, write the `DMA_DSCPTR_CUR` register prior to writing to the `DMA_CFG` register, which is the special action required to start the DMA channel.
- For descriptor-list-based flow modes, at minimum, write the `DMA_DSCPTR_NXT` register prior to writing to the `DMA_CFG` register, which is the special action required to start the DMA channel.
- For non-descriptor-based flow modes, write the `DMA_ADDRSTART`, `DMA_XCNT`, and `DMA_XMOD` registers prior to writing the `DMA_CFG` register.

Programs can write other registers that can remain static throughout the course of the DMA activity. The write to the `DMA_CFG` register begins the DMA operation.

ATTENTION: When software directly writes the `DMA_CFG` register, the DMA controller recognizes this action as the special startup condition. This condition occurs when starting the DMA controller for the first time on this channel or occurs after the DMA channel stops. It is possible for the channel to flag a DMA error condition regardless of the `DMA_CFG.EN` bit setting.

Startup Operation

The startup operation is initiated by software directly writing the `DMA_CFG` register when starting DMA for the first time on a channel or after the channel has entered to the stop state.

When the descriptor fetch is complete and the DMA channel is enabled, the `DMA_CFG` descriptor element replaces the `DMA_CFG` register content and assumes control. Before this point, the direct write to the `DMA_CFG` register had control.

At startup, the selected flow mode and the descriptor size determine the course of the DMA initialization process. The `DMA_CFG.FLOW` field determines whether to load more current registers from descriptor sets in memory. The `DMA_CFG.NDSIZE` field details how many descriptor elements to fetch before starting the DMA operation. This process does not affect DMA registers that are not in the descriptor; no modifications are made to their prior values.

For descriptor-list flow modes, the channel copies the `DMA_DSCPTR_NXT` register value into the `DMA_DSCPTR_CUR` register. Then, the channel fetches new descriptor elements from memory. The `DMA_DSCPTR_CUR` register indexes each fetch, and the channel increments the index after each fetch. After completion of the descriptor fetch, the `DMA_DSCPTR_CUR` register points to the next 32-bit word in memory past the end of the descriptor.

If the descriptor fetch is for a descriptor-array mode transfer, the channel does *not* copy the `DMA_DSCPTR_NXT` register into the `DMA_DSCPTR_CUR` register. *Instead*, the descriptor fetch indexing begins with the value in the `DMA_DSCPTR_CUR` register.

If `DMA_CFG` is not part of the fetched descriptor set, the previous value (originally as written on startup) controls the work unit operation. If the `DMA_CFG` register is part of the fetched descriptor set, the value programmed by the

MMR access controls only the loading of the first descriptor set fetched from memory. The configuration of the `DMA_CFG` register controls the subsequent DMA work units of the fetched descriptor set.

After the descriptor fetch is complete or if the flow configuration was originally for one of the register-based flow modes, the DMA operation begins. The DMA channel immediately fills its FIFO. For a memory-write operation, the DMA channel begins accepting data from the peripheral. For a memory-read operation, the DMA channel begins memory reads when the SCB bus grants access to the DMA channel.

When the DMA channel performs its first data-memory access, its address and count computations take their input operands from the start registers. These registers can include `DMA_ADDRSTART`, `DMA_XCNT`, and `DMA_YCNT`, if necessary. The channel writes results back to the current registers. These registers include `DMA_ADDR_CUR`, `DMA_XCNT_CUR`, and `DMA_YCNT_CUR`. Note that the current registers are not valid until the channel performs the first memory access, which can be some time after the write to the `DMA_CFG` register starts the channel. Once started, the channel automatically loads the current registers from the appropriate descriptor elements, overwriting their previous contents. These automatic-load operations include:

- The channel copies the `DMA_ADDRSTART` value to `DMA_ADDR_CUR`.
- The channel copies the `DMA_XCNT` value to `DMA_XCNT_CUR`.
- The channel copies the `DMA_YCNT` to `DMA_YCNT_CUR`.

Refresh Flow

When the channel completes processing of a work unit, the DMA channel performs the following operations:

- Completes the transfer of all data between memory and the DMA channel.
- Performs a synchronized transition (if the DMA channel configuration is a memory read operation with the `DMA_CFG.SYNC` bit enabled) *and* transfers all data to the peripheral before continuing.
- Forwards the signals from the DMA channel (if interrupts or triggers are enabled) *and* updates the `DMA_STAT` register to indicate the interrupt request or trigger events.
- Clears the `DMA_STAT.RUN` bit field to stop DMA operation (if the flow was set to stop mode) *and* transfers any remaining data in the FIFO of the DMA channel to the peripheral.
- Loads a new descriptor from memory into the DMA registers by way of the contents of the `DMA_DSCPTR_CUR` register (for descriptor-array mode) *and* increments the `DMA_DSCPTR_CUR` register. The channel takes the descriptor size from the `DMA_CFG.NDSIZE` value before the fetch.
- Copies the `DMA_DSCPTR_NXT` register into the `DMA_DSCPTR_CUR` register (for descriptor-list mode), fetches the descriptor from the new contents of the `DMA_DSCPTR_CUR` register, *and* places these contents into the DMA registers while incrementing the `DMA_DSCPTR_CUR` register.
- Checks for detection of an incoming trigger event (for descriptor-on-demand array mode):
 - If the channel detects a trigger event, the DMA channel loads a new descriptor from memory into the DMA registers from the contents of the `DMA_DSCPTR_CUR` register, while incrementing the

`DMA_DSCPTR_CUR` register. The channel takes the descriptor size from the `DMA_CFG.NDSIZE` value before the fetch.

- If the channel detects no trigger event, the DMA channel begins the next work unit by reloading the current registers.
- Checks for detection of an incoming trigger event (for descriptor-on-demand list mode):
 - If the channel detects a trigger event, the DMA channel copies the `DMA_DSCPTR_NXT` register value to the `DMA_DSCPTR_CUR` register, fetches the descriptor memory from the `DMA_DSCPTR_CUR` register, *and* places the contents into the DMA registers while incrementing the `DMA_DSCPTR_CUR` register.
 - If the channel detects no trigger event, the DMA channel begins the next work unit by reloading the current registers as described in the next step.
- Begins the next work unit (if flow configuration is anything other than stop mode) by reloading the current registers (`DMA_ADDR_CUR`, `DMA_XCNT_CUR`, and `DMA_YCNT_CUR`) from their descriptor registers (`DMA_ADDRSTART`, `DMA_XCNT`, and `DMA_YCNT`)

Work Unit Transition Flow

The `DMA_CFG.SYNC` bit controls transitions from one work unit to the next work unit. In general, continuous transitions have lower latency at the cost of restrictions on changes of data format or addressed memory space in the two work units. These latency gains and data restrictions arise from the way the channel handles the DMA FIFO while fetching the next descriptor.

In continuous transitions, with disabled synchronization, the DMA FIFO pipeline continues to transfer data to and from the peripheral or destination memory. These transfers continue during the descriptor fetch and during the DMA channel pause between descriptor chains. By comparison, synchronized transitions provide better real-time synchronization of interrupts and triggers with a given peripheral state. Synchronized transitions also provide greater flexibility in the data formats and memory spaces of the two work units. This flexibility comes at the cost of higher latency in the transition. In synchronized transitions, the DMA FIFO pipeline drains to the destination or flushes (received data discarded) between work units.

NOTE: The `DMA_CFG.SYNC` bit of the MDMA source channel controls work unit transitions for MDMA streams. Clear this reserved bit of the MDMA destination channel, placing it in the disabled state. In transmit (memory read) channels, the `DMA_CFG.SYNC` bit of the last descriptor before the transition controls the transition behavior. In contrast, in receive channels, the `DMA_CFG.SYNC` bit of the first descriptor of the next descriptor chain controls the transition.

Work Unit Transmit and MDMA Source Transitions

In DMA transmit (memory read) and MDMA source channels, the `DMA_CFG.SYNC` bit controls the interrupt timing at the end of the work unit. This bit also controls the handling of the DMA FIFO between the current and the next work unit.

If the `DMA_CFG.SYNC` bit configuration disables synchronization, the DMA channel operates in continuous transition. In a continuous transition, just after reading the last data item from memory, the DMA channel starts all of the following operations parallel:

- Signals the interrupt request or trigger
- Updates the `DMA_STAT` register to indicate DMA completion status
- Begins fetching the next descriptor
- Delivers the final data items from the DMA FIFO to the destination memory or peripheral

This process lets the DMA channel provide data from the FIFO to the peripheral continuously during the descriptor fetch latency period.

If the configuration disables synchronization, the final interrupt request or trigger (if enabled) occurs when the channel reads the last data from memory. This event occurs at the earliest time that the channel safely can modify the output memory buffer without affecting the previous data transmission. There can be a number of data items remaining in the FIFO and not yet at the peripheral. This number depends on the FIFO depth of the DMA channel. In this configuration, do not use the DMA interrupt request as the sole means of synchronizing the shutdown or reconfiguration of the peripheral following a transmission.

NOTE: If the configuration selects continuous transition on a transmit (memory read) descriptor, the next descriptor must have the same:

- Peripheral transfer size (`DMA_CFG.PSIZE`)
- Read or write direction
- Source memory (internal versus external) as the current descriptor

It is possible to disable synchronization by selecting continuous transition on a work unit with configuration for stop-flow mode and with enabled interrupts or triggers. This approach can result in the execution of the event service routine while draining of the final data is ongoing from the FIFO to the peripheral. If data transfers are in-progress, the FIFO is not yet empty. The `DMA_STAT.RUN` bits of the DMA channels indicate this status. Do not start a new work unit with a different peripheral transfer size or direction while data transfers are in-progress.

CAUTION: Disabling the channel with the `DMA_CFG.EN` bit while data transfers are in-progress causes the loss of the data in the FIFO.

A synchronized transition configuration directs the channel to drain the DMA FIFO to the destination memory or peripheral. This FIFO operation occurs before the channel signals any interrupt and before the channel fetches any subsequent descriptor or data. This operation incurs greater latency, but provides direct synchronization between the DMA interrupt and the state of the data at the peripheral.

If the configuration enables synchronization and enables interrupts, on the last descriptor in a work unit, the interrupt occurs when the channel transfers the final data to the peripheral. This event allows the service routine to switch properly to non-DMA transmit operation. When the event vectors to the interrupt service routine, the DMA channel FIFO is empty, and the DMA channel is no longer running (indicated by the `DMA_STAT.RUN` bits).

A synchronized transition also allows greater flexibility in the format of the DMA descriptor chain. When enabled, the next descriptor can have any `DMA_CFG.PSIZE` configuration or read/write direction supported by the peripheral and can come from either memory space (internal or external). This feature can be useful in managing MDMA work unit queues, since it is no longer necessary to interrupt the queue between dissimilar work units.

Work Unit Receive and MDMA Destination Transitions

In DMA receive channels (memory write operations), the `DMA_CFG.SYNC` bit controls the handling of the DMA FIFO between descriptor chains (not individual descriptor sets), during the DMA channel pause. The DMA channel pauses after the descriptor sets configured with stop flow mode are complete. Restart the channel (for example, after an interrupt) by writing the `DMA_CFG` register of the channel with a value that enables the DMA channel. If the configuration disables synchronization in the `DMA_CFG` value of the new work unit, the configuration selects a continuous transition. In this mode, the DMA FIFO retains any data items received during the channel pause, and they are the first items written to memory in the new work unit. This mode of operation provides lower latency at work unit transitions and ensures no dropping of data items during a DMA pause. The channel provides this operation at the cost of certain restrictions on the DMA descriptors.

NOTE: If the `DMA_CFG.SYNC` bit disables synchronization on the first descriptor of a chain after a DMA pause, do not change the configuration of the `DMA_CFG.PSIZE` field of the new chain from the previous descriptor chain (active before the pause). This restriction applies unless the DMA channel is reset between chains by disabling and then re-enabling the DMA channel.

If the `DMA_CFG.SYNC` bit configuration enables synchronization, the channel uses a synchronized transition. In this mode, only the data that the DMA channel receives from the peripheral after the write to the `DMA_CFG` register gets to memory. The channel discards any prior data items transferred from the peripheral to the DMA FIFO before this register write occurs. This operation provides direct synchronization between the data stream received from the peripheral and the timing of the channel restart, which occurs on the write to the `DMA_CFG` register.

For receive DMA operations, the synchronization has no effect in transitions between work units in the same descriptor chain. When the flow mode of previous descriptor was not stopped, the DMA channel did not pause.

If a descriptor chain begins with synchronization enabled, there is no restriction on the `DMA_CFG.PSIZE` of the new chain in comparison with the previous chain.

NOTE: The peripheral transfer size (`DMA_CFG.PSIZE`) must not change between one descriptor and the next in any DMA receive (memory write) channel within a single descriptor chain, regardless of the `DMA_CFG.SYNC` bit setting. In other words, all memory write descriptor sets in a descriptor chain must have the same `DMA_CFG.PSIZE` value. For any DMA receive channel (memory write operation), there is no restriction on changes of peripheral transfer size (internal versus external) between descriptors or descriptor chains.

Transfer Termination and Shutdown Flow

This section describes channel transfer termination and shutdown in stop flow mode and in autobuffer flow mode.

Stop Flow Mode

In stop flow mode, the DMA channel stops automatically after the work unit is complete. If using a list or array of descriptors to control DMA transfers and if every descriptor contains a `DMA_CFG` descriptor element, configure the flow of the final `DMA_CFG` descriptor element to stop mode, stopping the channel gracefully. After completion, the DMA channel remains in the stop state. Do not confuse this state with the disabled state, which either occurs due to a DMA error or occurs through disabling the DMA channel by configuring the `DMA_CFG.EN` bit.

The intention of disabling the DMA channel through a write to the `DMA_CFG.EN` bit is to shut down the DMA channel and to enter the disabled state. All memory and peripheral data transfers cease, and only peripheral interrupts pass through the DMA channels interrupt signals. However, the DMA channel maintains the `DMA_STAT.RUN` bits. For a write to memory, the outstanding memory-transaction counter tracks returning memory write acknowledgments and updates as required.

For memory reads, the outstanding memory-transaction count also tracks returning memory reads. The channel does not write the memory reads into the FIFO. The channel updates the counter to reflect the completion of the transaction, but the channel ignores the data. The `DMA_STAT.RUN` bits remain in the *waiting for write ACK or FIFO drain to peripheral* state and do not change to *stop or idle state* until the return of all outstanding transactions.

When the `DMA_CFG.EN` bit again enables the DMA channel, the channel performs a full reset and clears all counters. If an outstanding memory transaction returns an acknowledgment or read data after this event, a memory transaction error occurred, which generates an error event. Programs must ensure that all outstanding memory transactions complete before reconfiguring the DMA channel. For example, programs can poll the `DMA_STAT.RUN` bits to return to the stop or idle state before proceeding.

Autobuffer Flow Mode

In this mode, the flow does not use any descriptors in stored memory. Instead, the channel performs DMA in a continuous circular buffer fashion, based on user-programmed DMA register settings. On completion of the work unit, the channel reloads the parameter registers into the current registers, and the DMA controller resumes immediately with zero overhead. Consider this mode as a succession of automatically restarted work units.

For autobuffer-flow modes, the only way to cease operations is to disable the DMA channel through the `DMA_CFG.EN` bit. One method of changing to a new work unit is:

- Disable the DMA channel
- Set up all the registers (and descriptors in memory, if used) except for `DMA_CFG`
- Poll `DMA_STAT.RUN` to wait for the status to reflect stop or idle state, and
- Write `DMA_CFG` to the new configuration to begin the next work unit

In autobuffer-flow mode or for a list or array of descriptor sets without `DMA_CFG` descriptors, use an MMR write to the `DMA_CFG` register to terminate the DMA transfer process. Configure the value of the `DMA_CFG.EN` bit in this register to disable the DMA channel.

CAUTION: When the configuration disables a DMA channel, the DMA controller disables interrupt logic that is based on work unit transitions. Be aware of the system environment and current actions, so that additional interrupts are not required from the DMA channel.

CAUTION: If disabled through `DMA_CFG.EN` in the middle of a transaction, the DMA channel completes any transactions that have begun and avoids generating bus errors. However, the channel considers the action of re-enabling the DMA as a hard reset for all internal DMA channel components. Therefore, pay attention to that particular action to avoid unexpected results.

DMA Channel Errors

When an error occurs, the DMA channel maintains all the state and register values that allow programs to diagnose error causes more thoroughly. The greatest benefit to the programmer is to know exactly what operational state the DMA channel was in at the exact moment the error occurred.

Take care to address the root cause of the error, whether or not the problem originated in the DMA channel. If not properly resolved, the error can result in an additional error shortly after operations resume. The problem can cause other errors elsewhere in the DMA channel or associated modules and circuitry. So, take care also to address those potential problems. Ensure that all outstanding memory reads and writes are complete or cleared before resuming DMA channel operation.

After addressing all issues and neutralizing all side effects of any errors, clear the `DMA_STAT.ERRC` status field and restart the DMA channel by disabling then re-enabling the DMA channel through the `DMA_CFG.EN` bit.

The following sections describe the error types.

Status and Debug Errors

DMA channel error conditions can cause the DMA process to end abnormally. The DMA channel provides error detection as a tool for system development and debug, helping to identify DMA-related programming errors. When the DMA channel detects an error, the channel immediately stops and discards any returned memory-read transactions. The `DMA_STAT.RUN` field of the DMA channel indicates the idle state after acknowledging all outstanding memory transactions. In addition, the channel asserts an error interrupt request and updates the `DMA_STAT.IRQERR` field. Also, the channel updates the `DMA_STAT.ERRC` field, indicating the error cause of the first detected error. Unless the error occurs at the exact moment that modification of register values occurs, the registers contain the error values.

All the DMA error interrupt requests are combined into a single shared interrupt request output. Combined error signals require reading the `DMA_STAT` register of each DMA channel associated with a combined error interrupt request to determine the DMA channel responsible for the generation of the interrupt.

The DMA channel error interrupt handler performs the following actions:

- Read the `DMA_STAT` register of each DMA channel, seeking a channel with the `DMA_STAT.IRQERR` set to indicate an error.
- Read the `DMA_STAT.ERRC` field of each DMA channel, determining the cause of the error.
- Clear the problem with the DMA channel. For example, fix the register values.

- Clear the error in the DMA channel through a write-1-to-clear operation to the `DMA_STAT.IRQERR` bit.

If the channel flags any uncleared error other than a bandwidth monitor error, the channel reports no other error. If the channel reports an uncleared bandwidth monitor error, the channel reports any newly detected error through updating the `DMA_STAT.ERRC` field.

DMA Configuration Register Errors

The channel only flags these configuration errors when the `DMA_CFG.EN` bit enables the DMA channel. Error flagging occurs when the configuration:

- Uses a reserved setting
- Enables `DMA_CFG.TWAIT` in descriptor on-demand flow mode
- Uses an illegal `DMA_CFG.NDSIZE`
- Uses an illegal `DMA_CFG.MSIZE`
- Configures `DMA_XCNT = 0` or, when `DMA_YCNT = 0` in 2D DMA mode
- Uses non-zero value in `DMA_CFG.NDSIZE` when DMA is configured in stop mode or auto mode
- Enables interrupt or outgoing triggers on `DMA_YCNT` when DMA is configured in 1D mode
- Use a `DMA_CFG.MSIZE` that exceeds the FIFO size of the DMA channel
- Uses an illegal `DMA_CFG.PSIZE`
- Uses a `DMA_CFG.PSIZE` that exceeds the FIFO size
- Uses a `DMA_CFG.PSIZE` that exceeds the bus width
- Attempts to change from a transmit operation (memory read) to a receive operation without properly synching in the previous work unit or when it is the first work unit in a new chain
- Attempts to change `DMA_CFG.PSIZE` of a transmit operation (memory read) without properly synching in previous work unit or when it is the first work unit in a new chain
- Attempts to change from receive operation (memory write) to a transmit operation during a descriptor chain.
The channel only can change from receive to transmit if the new transmit is synchronized and is the first work unit.
- Attempts to change `DMA_CFG.PSIZE` of a receive operation (memory write) when the operation was not the first work unit (with `DMA_CFG.SYNC` enabled)

Illegal Register Write During Run

The channel generates an error when a write occurs to writable registers of an enabled, running DMA channel. The channel blocks the write. The `DMA_STAT`, `DMA_BWLCNT`, and `DMA_BWMCNT` registers are exempt from this behavior.

Address Alignment Error

The channel generates an address alignment error when any of the following apply:

- Alignment of a descriptor address is not on a 32-bit boundary.
- The current `DMA_CFG.MSIZE` configuration contains an unaligned transfer address. The `DMA_ADDRSTART` register is not aligned according to the `DMA_CFG.MSIZE` field.

Memory Access Error

The channel generates a memory access error when the DMA process:

- attempts to access an unpopulated address,
- attempts to access a location that provokes a security violation

The error returned from the memory triggers the memory access error.

Trigger Overrun Error

A trigger overrun error is generated when a new trigger input occurred while an outstanding trigger is waiting. This error is only generated if `DMA_CFG.TOVEN` is enabled.

Bandwidth-Monitor Error

The channel generates this error when the bandwidth-monitor count expires. This error is not fatal, and the DMA channel continues operation.

Control Interface Error

The channel reports control-interface errors as bus errors to the bus controller. This error can result from:

- An address error
- A register write error (write to a read-only register)

DMA Operating Modes

The DMA channel supports a number of different flow modes that control how the DMA channel progresses from one work unit to the next.

The flow mode of a DMA channel is not a global setting. A DMA descriptor set can include the descriptor responsible for configuring the flow of the work unit. There is no restriction, limiting the flow configuration to be the same for the entire descriptor chain. If the descriptor chain is not endless, the last descriptor set configures the flow to stop mode, which results in termination of the descriptor chain after the work unit completes. Another example for mixing flow modes is to create an endless descriptor-array. The configuration of the last descriptor set in the array selects the descriptor-list mode. The next descriptor pointer in this set of descriptors points to the first descriptor in the array.

Register-Based Flow Modes

Register-based DMA operations require configuration by directly writing to the memory-mapped registers of the DMA channel.

Register-based DMA is the traditional method of DMA operation. Software writes all of the configuration of the DMA channel into the memory-mapped registers. This configuration includes information such as the source or destination address and length of the data in the transfer. The DMA controller then starts channel operation. The DMA channel supports the following register-based flow modes.

- [Stop Mode](#)
- [Autobuffer Mode](#)

The DMA channel supports variable descriptor set sizes within the configuration. The size of a descriptor set can contain as little as a single descriptor. The supported descriptor set sizes can differ between the various descriptor-based flow modes. In addition to the descriptor set size being configurable, descriptor-based DMA also allows altering the flow mode of the next descriptor set. This feature allows for the transition from descriptor-array mode to descriptor-list mode and permits configuring the flow to stop or autobuffer mode.

Stop Mode

In stop mode, the DMA operation executes only once. If started, the DMA channel transfers the desired number of data words and stops itself again when finished. If the DMA channel is no longer used, software configures the enable bit to disable a paused channel. The channel also can generate interrupts and triggers for each row or work unit completion, depending on the desired operation.

Autobuffer Mode

In autobuffer mode, the DMA operates repeatedly in a circular manner. If the transfer of all data words completes, the channel reloads the address pointer ([DMA_ADDR_CUR](#)) automatically with the [DMA_ADDRSTART](#) value. The channel also can generate an interrupt.

The `DMA_CFG.FLOW` field enables autobuffer mode. The configuration must load the `DMA_CFG.NDSIZE` field value, such that the next descriptor size is zero.

Descriptor-Based Flow Modes

Descriptor-based DMA operations fetch descriptor sets from memory allowing for autonomous loading of work units on other work units. Software does not need to set up the DMA sequences directly by writing into the DMA controller registers. Rather, software keeps DMA descriptor sets in memory.

Descriptor-based DMA operations have the following additional attributes.

- The DMA controller autonomously loads the descriptor set from memory to the affected DMA controller registers on demand.
- The channel can fetch descriptor sets from any memory space that supports DMA read operations.
- The descriptor set describes the next operation that the DMA controller performs.

- The descriptor set can include information such as the DMA configuration word as well as data source or destination address, transfer count, and address modify values.

A descriptor set describes a single work unit. The next work unit can reuse some values from the previous one descriptor set. But, this reuse is possible only if they are not overwritten in the subsequent descriptor set fetches and only if the work unit requires the use of this descriptor.

The DMA channel supports the following flow modes with descriptor-based operations.

- [Descriptor-Array Mode](#)
- [Descriptor-List Mode](#)
- [Descriptor-On-Demand Modes](#)

The DMA channel supports variable descriptor set sizes within the configuration. The size of a descriptor set can contain as little as a single descriptor and the supported descriptor set sizes can differ between the various descriptor-based flow modes. In addition to configurable descriptor set size, descriptor-based DMA also allows for altering of the flow mode of the next descriptor set. Programs can transition from one descriptor-based mode to another descriptor-based mode and can also transition to any of the register-based flow modes.

Descriptor-Array Mode

When configured in this mode, the descriptor sets do not contain further descriptor pointers. Software writes the initial descriptor-pointer value, which points to an array of descriptors. This operation assumes that the individual descriptors reside next to each other and assumes that their addresses are known.

The *Offsets for Descriptor-Array Mode Parameters and Descriptors* table illustrates how to structure a descriptor set in memory. The descriptor sets must reside in a contiguous block of memory in the format shown in the table. Locate the first descriptor of the next descriptor set in the memory location immediately following the last descriptor of the current descriptor set. The values have the same order as the corresponding offset addresses of the memory-mapped register.

Table 25-9: Offsets for Descriptor -Array Mode Parameters and Descriptors

Descriptor Offset	Parameter Register
0x00	DMA_ADDRSTART
0x04	DMA_CFG
0x08	DMA_XCNT
0x0C	DMA_XMOD
0x10	DMA_YCNT
0x14	DMA_YMOD

All other DMA channel registers that were not loaded as a result of the descriptor set fetch, retain their previous values. The channel reloads all of the current registers between the descriptor set fetch and the start of the DMA operation for the work unit.

NOTE: At a minimum, write the [DMA_DSCPTR_CUR](#) register prior to writing to the [DMA_CFG](#) register, which is the special action required to start the DMA channel.

Descriptor-List Mode

In this flow mode, multiple descriptors form a chained list in which each descriptor set contains a pointer to the next descriptor set, allowing greater flexibility in memory layout options. When the channel fetches the descriptor set, the operation loads this pointer value into the next descriptor pointer register of the DMA channel.

Descriptor Sets

The *Offsets for Descriptor-List Mode Parameters and Descriptors* table shows how to structure a descriptor set in memory. Disperse the placement of the descriptor sets throughout memory, having sets reside in different memory blocks. But, each descriptor of the descriptor set must reside in a contiguous section of memory in the format shown in the table. The values have the same order as the corresponding offset addresses of the memory-mapped registers.

Table 25-10: Offsets for Descriptor-List Mode Parameters and Descriptors

Descriptor Offset	Parameter Register
0x00	DMA_DSCPTR_NXT
0x04	DMA_ADDRSTART
0x08	DMA_CFG
0x0C	DMA_XCNT
0x10	DMA_XMOD
0x14	DMA_YCNT
0x18	DMA_YMOD

All other DMA channel registers that were not loaded as a result of the descriptor set fetch, retain their previous values. The channel reloads all of the current values of the registers between the descriptor set fetch and the start of the DMA operation for the work unit.

Minimum Startup Requirements

At a minimum, write the [DMA_DSCPTR_NXT](#) register prior to write to the [DMA_CFG](#) register, which is the special action required to start the DMA channel.

Descriptor-On-Demand Modes

The [Descriptor-Array Mode](#) and [Descriptor-List Mode](#) each have an on-demand mode of operation.

In on-demand mode, at the end of the work unit, if the DMA channel has not detected an incoming trigger event, the channel repeats the current work unit. If the DMA channel receives an incoming trigger before completion of the work unit, the channel fetches a new descriptor set.

The *Offsets for Descriptor-Array Mode Parameters and Descriptors* and *Offsets for Descriptor-List Mode Parameters and Descriptors* tables illustrate how to structure each descriptor set in memory.

Table 25-11: Offsets for Descriptor-Array Mode Parameters and Descriptors

Descriptor Offset	Parameter Register
0x00	DMA_ADDRSTART
0x04	DMA_CFG
0x08	DMA_XCNT
0x0C	DMA_XMOD
0x10	DMA_YCNT
0x14	DMA_YMOD

NOTE: For descriptor-array mode, at a minimum, write the [DMA_DSCPTR_CUR](#) register prior to writing to the [DMA_CFG](#) register, which is the special action required to start the DMA channel.

Table 25-12: Offsets for Descriptor-List Mode Parameters and Descriptors

Descriptor Offset	Parameter Register
0x00	DMA_DSCPTR_NXT
0x04	DMA_ADDRSTART
0x08	DMA_CFG
0x0C	DMA_XCNT
0x10	DMA_XMOD
0x14	DMA_YCNT
0x18	DMA_YMOD

NOTE: For descriptor-list mode, at a minimum, write the [DMA_DSCPTR_NXT](#) register prior to write to the [DMA_CFG](#) register, which is the special action required to start the DMA channel.

Data Transfer Modes

In addition to supporting basic one-dimensional DMA transfers, the DMA channel also supports two-dimensional functionality.

Two-Dimensional DMA

Register-based flow modes and descriptor-based flow modes support two-dimensional data transfers.

In two-dimensional (2D) mode, the X-direction count ([DMA_XCNT](#)), the X-direction modifier ([DMA_XMOD](#)), the Y-direction count ([DMA_YCNT](#)), and the Y-direction modifier ([DMA_YMOD](#)) support arbitrary row and column sizes. Also, the modify values can be negative, allowing implementation of interleaved data streams. The [DMA_XCNT](#) value specifies the row size, and the [DMA_YCNT](#) value specifies the column size; where the [DMA_XCNT](#) value must be 2 or greater.

The DMA start address (`DMA_ADDRSTART`), the X-direction modifier (`DMA_XMOD`), and the Y-direction modifier (`DMA_YMOD`) specifications all are in bytes. The alignment must be a multiple of the DMA transfer word size; configured using the `DMA_CFG.MSIZE` bit. Misalignment results in a DMA channel error.

The `DMA_XMOD` register value is the byte-address increment that the channel applies after each transfer, decrementing the `DMA_XCNT` register. The channel does not apply the `DMA_XCNT` when the inner loop count ends with the `DMA_XCNT_CUR` register decrementing to 0 from 1. Except, the channel does apply the `DMA_XCNT` on the final transfer, when the `DMA_YCNT` register is 1 and the `DMA_XCNT` register decrements from 1 to 0.

The `DMA_YMOD` register value is the byte-address increment that the channel applies after each decrement of the value in `DMA_YCNT_CUR`. However, the channel does not apply the `DMA_YMOD` value to the last item in the array on which the outer loop count (`DMA_YCNT_CUR`) also expires by decrementing from 1 to 0.

After the last transfer completes, `DMA_YCNT_CUR` is 1 and the `DMA_XCNT_CUR` register is 0. The DMA channels current address points to the last items address plus the `DMA_XMOD` register value. If the DMA channel programming selects automatic refresh (such as in autobuffer mode), the channel reloads the `DMA_XCNT_CUR`, `DMA_YCNT_CUR`, and `DMA_ADDR_CUR` for the first data transfer of the next work unit.

Interrupt notification is configurable for end-of-row or end-of-work unit completion.

For example, two-dimensional DMA can be used to extract interleaved data (such as RGB values for a video frame) by modifying both `DMA_XMOD` and `DMA_YMOD` values. The *Capturing a Video Data Stream 2D DMA Example* depicts the process of receiving a stream of the R, G, B values from an $N \times M$ frame. The inner loop of the 2D DMA configuration has three values (`DMA_XCNT` = 3) and a stride (`DMA_XMOD`) of $N \times M$, chosen such that successive elements in each row are 1-2-3, 4-5-6 and so forth. The outer loop of the 2D DMA configuration has $N \times M$ values (`DMA_YCNT` = $N \times M$) and a negative stride (`DMA_YMOD`) of $1 - 2 \times N \times M$ chosen to instruct the DMA controller to jump from element 3 to 4, 6 to 7 and so forth at the end of each inner loop.

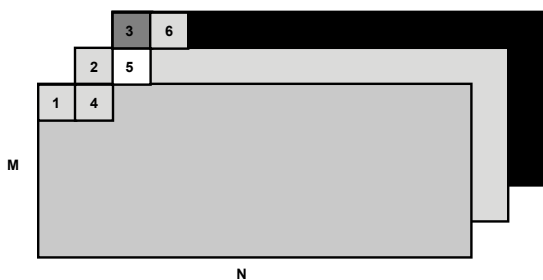


Figure 25-4: Capturing a Video Data Stream 2D DMA Example

DMA Channel Event Control

The DMA channel supports a number of events that provide notification of work unit state, peripheral data request, peripheral interrupt request and completion events, and DMA channel error conditions. In addition to flexible interrupt configuration, the DMA channel also supports incoming and outgoing triggers which are useful in synchronizing the DMA channel with other system resources.

The DMA channel has two interrupt signals for support of a number of events such as work-unit state events, peripheral interrupt request (PIRQ) events, peripheral data request (PDR) events, and DMA channel errors. The channel reports DMA channel errors on a dedicated interrupt signal. All other interrupt sources share an interrupt signal. In addition to flexible interrupt configuration, the DMA channel also supports incoming and outgoing triggers which are useful in synchronizing the DMA channel with other system resources.

The channel can signal the processor on DMA channel events using status information and optional interrupt requests. Programs can use these events to update the progress of data transfers and to request intervention from the processor core. Configure most DMA channel interrupts using bits in the [DMA_CFG](#) register. Dedicated bits in the [DMA_STAT](#) register report the occurrence of various events. Use write-one-to-clear (W1C) operations to clear interrupt requests from the status register.

NOTE: Hardware does not clear the interrupt status bits automatically, even when programs disable then reenables the DMA channel. In this situation, the channel deasserts the interrupt signal, after the program disables the DMA channel. But, the status bit remains set until software either re-enables the DMA channel or clears the status bit.

The DMA channel supports the following categories of events on the interrupt signals:

- Work-unit state events generate interrupts on row or on work unit DMA completion.
- A peripheral uses peripheral interrupt request (PIRQ) events to signal when it has completed the transfer of all data.
- A peripheral uses peripheral data request (PDR) events to request data from a disabled or idle DMA channel.
- Error events signal a failure in the work unit.

ATTENTION: While in an error state, the DMA channel does not generate an interrupt to the processor for a work-unit state event or a PIRQ event, nor does the channel forward a PDR event.

Event Signals

The *Event Signals* table provides descriptions of DMA channel events.

Table 25-13: Event Signals

Signal	Width (bits)	Description
DMA_ERROR	1	Used to signal an error condition in the DMA channel. The source of the error can be determined by reading the DMA_STAT .ERRC bit.
DONE_PIRQ_INT	1	Signal used to indicate DMA completions events, PIRQ events and also for forwarding PDR events based on configuration. Read the corresponding fields in DMA_STAT to determine the source of the event.
DMA_TRIG_OUT	1	Trigger output that gets routed to the TRU and can be configured to provide notification on row or work unit completion.
DMA_TRIG_IN	1	Trigger input from the TRU that can be used to control the start of a work unit.

Work Unit State Events

Completing a row or a work unit generates a work-unit state event. For either of these events to generate an interrupt request, the configuration of the interrupt of the DMA channel must select one of the available work-unit completion modes.

- Current X count reaching 0 for row completion or 1D DMA work unit completion
- Current Y count reaching 0 for work unit completion of 2D DMA

NOTE: For 1D DMA, a DMA channel configuration error results if the configuration generates the interrupt request when the current Y counter reaches 0.

The DMA channel issues the last memory read or write transaction for the row or work unit, then pauses until the return of the read or write acknowledge. After successful acknowledge of the transfer, the DMA channel issues the interrupt request and continues to process the next row or work unit.

Waiting for acknowledgement of the memory access results in a delay. However, programs can read or modify data in the memory without adversely affecting or being affected by the DMA transfer.

NOTE: While the DMA channel pauses waiting for acknowledgement of the memory transfer, the DMA channel is still capable of fetching the next descriptor set. This fetch gets the channel ready to process the next work unit as soon as the memory access completes.

The channel configuration of the synchronization feature also affects interrupt timing. For memory-read operations with synchronization enabled, the channel delays the interrupt request until the completion of the last transfer from the DMA channel FIFO to the peripheral. The synchronization feature does not affect interrupt timing for memory write operations.

Peripheral Interrupt Request Events

For peripheral-transmit operations, a peripheral connected to the DMA channel can use peripheral interrupt request (PIRQ) events to indicate that data has left the channel FIFO and to indicate transfer completion.

In order to support PIRQ interrupts, correctly configure the interrupt of the DMA channel. This configuration disables the generation of interrupt requests based on the work unit state and, instead, results in generating an interrupt request when the DMA channel receives the command from the peripheral.

The channel only generates the interrupt request under the following conditions:

- The configuration enables the DMA channel
- The DMA channel is in the stop state
- The configuration of the DMA channel interrupt selects PIRQ operation

Peripheral Data Request Events

Peripheral data request (PDR) events occur when an interfaced peripheral requests data from the DMA channel and the DMA channel (either disabled or enabled) is in the stop state.

When a peripheral sends a data request command to a disabled DMA channel, the DMA channel generates an interrupt to the System Event Controller (SEC). There is no status information reported about this event in the status register of the DMA channel. Instead, the channel identifies the PDR event from the fact that the DMA channel generated an interrupt while disabled. It is possible to further confirm event status by verifying the status of the peripheral interfaced to the DMA channel.

This operation forwards data requests as interrupts when the DMA channel is in the disabled state. The DMA channel can forward PDR events as an interrupt request when the DMA channel is in the stop state after the completion of a work unit. The forwarding of this interrupt when the DMA channel is in the stop state is optional and configured by the program during DMA channel configuration.

DMA Channel Triggers

DMA channel triggers are useful for synchronizing the DMA channel with other events in the system. One usage is to combine channel triggers with each other to create ping-pong buffers. Another usage is to combine the triggers with interrupt requests to notify the processor on reaching a particular milestone that requires service. The channel also can use triggers to enforce a handshake DMA operation in which the trigger acts as a signal for a DMA request.

NOTE: Using the trigger to control the pace of data transfers, such as for handshake DMA, requires that all the data for the entire work unit is ready for transfer.

The DMA channel has a single incoming trigger that can control the pace of the data transfers performed by the DMA channel. The configuration can direct the DMA channel to wait for the incoming trigger before starting the work unit transfer or fetching a descriptor set from memory.

The DMA channel also has a single outgoing trigger signal. This configuration can direct this trigger to signal the end of row or an entire work unit. The DMA channel issues the last memory read or memory write transaction for the row or work unit, then pauses until return of the transfer acknowledge. After acknowledgement of the transfer, the DMA channel issues the trigger before processing the next row or work unit.

NOTE: The peripheral interrupt (PIRQ) and trigger out are independent functions. PIRQ is asserted when the respective peripheral completes the transaction. Trigger out is asserted when the DMA module completes the transaction.

Issuing Triggers

The DMA channel configuration can direct the channel to generate an outgoing trigger signal at the end of row or the end of a work unit. The DMA channel issues the last memory read or memory write transaction for the row or work unit, then pauses until the return of the transfer acknowledge. After acknowledgement of the transfer, the DMA channel issues the trigger before processing the next row or work unit.

NOTE: While the DMA channel pauses waiting for acknowledgement of the memory transfer, the DMA channel is still capable of fetching the next descriptor set. This fetch gets the channel ready to process the next work unit as soon as the memory access completes.

Waiting For Triggers

Programs can use triggering to control the pace of data transfers performed by the DMA channel. The DMA channel enters a wait state before beginning the next work unit if the configuration enables `DMA_CFG.TWAIT` and either of the following apply:

- The channel receives a trigger since the last time the DMA channel left the wait state.
- The channel receives a trigger since its transition from disable to enable.

In the wait state, the DMA channel also does not perform a descriptor fetch. After receiving a trigger, the DMA channel leaves the wait state and begins the next work unit or fetches the next descriptor if configured for a descriptor-based mode of operation.

If a memory-mapped register write operation programs the channel with stop flow mode enabled (`DMA_CFG.TWAIT` bit) and the channel has not already received a trigger, the DMA channel enters a wait state before performing the data transfer. On receiving the trigger, the DMA channel begins the data transfer portion of the work unit. Once the data transfer is complete, the DMA channel enters the stop state.

If a memory-mapped register write operation programs the DMA channel with the flow mode configured to one of the descriptor-based modes, the DMA channel enters the wait state before performing the descriptor fetch. After completing the descriptor fetch, the DMA channel immediately proceeds to the data transfer, regardless of the value of the `DMA_CFG.TWAIT` bit. If another (next) descriptor fetch follows the descriptor fetch, the DMA channel enters a wait state before fetching the next descriptor.

If the descriptor fetch returns a descriptor with stop flow mode, the `DMA_CFG.TWAIT` value for that descriptor does not affect the DMA as the channel enters the stop state after completing the data transfer. The DMA channel only enters the wait state based on `DMA_CFG.TWAIT` before the next work unit or descriptor fetch.

If the descriptor fetch returns a descriptor configured for autobuffer flow mode, the `DMA_CFG.TWAIT` for that descriptor does not affect the DMA for the first work unit of the autobuffer transfer. After completing the first work unit and not receiving another trigger, the DMA channel enters the wait state before reinitializing its counters and address registers (if not configured for current addressing). The channel performs the next work unit after receiving the trigger.

The incoming trigger can occur when the DMA channel has not entered the wait state. The trigger can occur while the DMA channel is executing a work unit, is performing descriptor fetch, or is in the stop state. The trigger is held internally. After the work unit is complete, the DMA channel skips the wait state and proceeds directly to executing the following work unit. If the `DMA_CFG.TWAIT` bit is not enabled, the DMA channel also skips the wait state. However, the trigger is held internally and is used the next time the configuration enables `DMA_CFG.TWAIT`. This trigger retention allows programs to enable the `DMA_CFG.TWAIT` functionality several work units apart without concern for losing a trigger. The DMA channels trigger-overrun enable functionality can be enabled in all work units to ensure that multiple triggers do not occur between the work units with the `DMA_CFG.TWAIT` bit enabled.

DMA Channel Programming Model

Several synchronization and control methods are available for use in development of software tasks which manage peripheral DMA and memory DMA. Software must accept requests for new DMA transfers from other software tasks, integrate these transfers into existing transfer queues, and reliably notify other tasks when the transfers are complete.

In the processor, it is possible to manage each peripheral DMA and memory DMA stream with a separate task or to manage them together with any other stream. Each DMA channel has independent, orthogonal control registers, resources, and interrupts. So, the selection of the control scheme for one channel does not affect the choice of control scheme on other channels. For example, one peripheral can use a linked-descriptor-list, interrupt-driven scheme while another peripheral can simultaneously use a demand-driven, buffer-at-a-time scheme synchronized by polling DMA events.

The topics that follow describe the steps required to configure the DMA channel for the various modes in addition to the programming concepts required for software synchronization.

Mode Configuration

Use the step-by-step directions that follow to set up the DMA channel for operating modes.

Register-Based Linear-Buffer Stop Flow Mode

This procedure configures the DMA channel of a peripheral to read data from internal memory and to send it to the peripheral for transmission. Upon DMA completion, the DMA channel enters the idle state until either disabled or reconfigured for a new transfer.

Assume that the peripheral is in a state where it is ready to transmit data received from the DMA channel.

The task involves writing to a number of DMA channel MMR registers to configure a DMA channel to:

- Read data from internal memory, and
- Send it to a peripheral connected to the peripheral DMA bus.

1. Write the `DMA_ADDRSTART` register.

ADDITIONAL INFORMATION: Software can use the address to calculate the most optimum possible `DMA_CFG.MSIZE`.

2. Calculate the optimum `DMA_CFG.MSIZE` based on the `DMA_ADDRSTART` register and number of bytes in work unit.

ADDITIONAL INFORMATION: The number of bytes in the work unit must be a multiple of the selected `DMA_CFG.MSIZE`, and the calculation also must consider the start address alignment.

3. Write the `DMA_XCNT` register based on the calculated `DMA_CFG.MSIZE`.

ADDITIONAL INFORMATION: The `DMA_XCNT` value is the number of `DMA_CFG.MSIZE` transfers to make up the entire work unit.

4. Write the `DMA_XMOD` register.

ADDITIONAL INFORMATION: For a linear buffer transfer, determine the value in `DMA_XMOD` from the selected `DMA_CFG.MSIZE`. Always specify this register as a number of bytes.

5. Write the `DMA_CFG` register with `DMA_CFG.EN` configured to enable the DMA channel.

ADDITIONAL INFORMATION: Set the `DMA_CFG.FLOW` bit for STOP mode. Configure the `DMA_CFG.WNR` bit for memory read operation. Configure the `DMA_CFG.PSIZE` bits to a value no larger than the supported bus width of the peripheral DMA bus. From performance perspective, it is recommended to use the largest possible `DMA_CFG.PSIZE` value (for example, equal to the supported peripheral bus width (`DMA_STAT.PBWID`)). However, ensure that the number of bytes in the work unit is a multiple of the `DMA_CFG.PSIZE` value used.

- The `DMA_CFG.SYNC` bit can be configured to control DMA completion notification timing.
- Interrupts and triggers also can be configured at this step depending on requirements.

Now, the DMA channel is enabled, and the buffer is transferred. The DMA channel enters the IDLE state upon completion of the work unit.

Register-Based Autobuffer Flow Mode

This procedure configures the DMA channel of a peripheral to read data from internal memory and send it to the peripheral for transmission. The transmission of the buffer repeats endlessly. Upon DMA completion, the DMA channel restarts the DMA operation, creating an endless circular buffer transfer.

Assume the peripheral is in a state where it is ready to transmit data received from the DMA channel.

The task involves writing to a number of DMA channel MMR registers to configure a DMA channel to:

- Read data from internal memory, and
- Send it to a peripheral connected to the peripheral DMA bus.

1. Write the `DMA_ADDRSTART` register.

ADDITIONAL INFORMATION: Use the address to calculate the optimum possible `DMA_CFG.MSIZE`.

2. Calculate the optimum `DMA_CFG.MSIZE` based on the `DMA_ADDRSTART` register and number of bytes in work unit.

ADDITIONAL INFORMATION: The number of bytes in the work unit must be a multiple of the selected `DMA_CFG.MSIZE`, and the calculation must consider the start address alignment.

3. Write the `DMA_XCNT` register based on calculated `DMA_CFG.MSIZE`.

ADDITIONAL INFORMATION: The `DMA_XCNT` register value is the number of `DMA_CFG.MSIZE` transfers to make up the entire work unit.

4. Write the `DMA_XMOD` register.

ADDITIONAL INFORMATION: For a linear buffer transfer, determine the value in `DMA_XMOD` from the selected `DMA_CFG.MSIZE`. Always specify this register as a number of bytes.

5. Write the `DMA_CFG` register with the `DMA_CFG.EN` bit configured to enable the DMA channel.

ADDITIONAL INFORMATION: Set the `DMA_CFG.FLOW` bit for autobuffer mode. Configure the `DMA_CFG.WNR` bit for memory read operation. Configure the `DMA_CFG.PSIZE` bit to a value no larger than the supported bus width of the peripheral DMA bus. From performance perspective, it is recommended to use the largest possible `DMA_CFG.PSIZE` value (for example, equal to the supported peripheral bus width (`DMA_STAT.PBWID`)). However, ensure that the number of bytes in the work unit is a multiple of the `DMA_CFG.PSIZE` value used.

- The `DMA_CFG.SYNC` bit can be configured to control DMA completion notification timing.
- Interrupts and triggers also can be configured at this step depending on requirements.

Now, the DMA channel is enabled, and the buffer transfers until the DMA channel is disabled.

Descriptor-Array Flow Mode

This procedure configures the DMA channel of a peripheral to:

- Read data from memory as described by the descriptor sets in the array, and
- Send the data to the peripheral for transmission.

Descriptor sets are read from an array in memory to configure the individual work units.

Assume the peripheral is in a state where it is ready to transmit data received from the DMA channel. Assume that the array of descriptors is to be initialized with the last descriptor set configured for STOP flow mode.

The task involves writing to a number of DMA channel MMR registers to:

- Configure a DMA channel to read the array in memory, containing the first descriptor set that configured the DMA channel to retrieve, and
- Send the data to a peripheral connected to the peripheral DMA bus.

On DMA completion, the DMA channel enters the idle state until either disabled or reconfigured for a new transfer.

1. Write the `DMA_DSCPTR_CUR` register with the address of the array in which the descriptor sets are stored.

ADDITIONAL INFORMATION: The array address must meet any processor alignments restrictions imposed by descriptor fetches.

2. Write the `DMA_CFG` register with the `DMA_CFG.EN` bit configured to enable the DMA channel.

ADDITIONAL INFORMATION: Configure the `DMA_CFG.PSIZE` bits to a value no larger than the supported bus width of the peripheral DMA bus. From performance perspective, it is recommended to use the largest possible `DMA_CFG.PSIZE` value (for example, equal to the supported peripheral bus width (`DMA_STAT.PBWID`)). However, ensure that the number of bytes in the work unit is a multiple of the `DMA_CFG.PSIZE` value used. Set the `DMA_CFG.FLOW` bit for descriptor-array mode. Configure the `DMA_CFG.NDSIZE` bits to describe the number of descriptor elements contained within the first descriptor set. Configure the `DMA_CFG.WNR` bit for memory read operation.

- The descriptor set that is fetched controls the `DMA_CFG.SYNC` configuration and the interrupt or trigger configurations.

The first descriptor set is fetched from memory location provided by the `DMA_DSCPTR_CUR` register and loaded to the MMR registers of the DMA channel.

Now, the DMA channel is processing all the work units provided in the descriptor array. The DMA channel enters the IDLE state on completion of the final work unit that was configured for STOP flow mode.

Descriptor-List Flow Mode

This procedure configures the DMA channel of a peripheral to:

- Read data from memory as described by the descriptor sets in the list, and
- Send it to the peripheral for transmission.

The DMA controller reads the descriptor sets from a list of descriptors. With the list, each descriptor set has a descriptor that points to the next descriptor set location in memory.

Assume the peripheral must be in a state where it is ready to transmit data received from the DMA channel. Assume that the list of descriptors must be initialized with the last descriptor set in the list configured for Stop flow mode.

The task involves writing to a number of DMA channel MMR registers to:

- Configure a DMA channel to read the list in memory, containing the first descriptor set that configured the DMA channel to retrieve, and
- Send the data to a peripheral connected to the peripheral DMA bus.

On DMA completion, the DMA channel enters the idle state until either disabled or reconfigured for a new transfer.

1. Write the `DMA_DSCPTR_NXT` register with the address of the first descriptor in the list to be processed.

ADDITIONAL INFORMATION: The array address must meet any processor alignments restrictions imposed by descriptor fetches.

2. Write the `DMA_CFG` register with the `DMA_CFG.EN` configured to enable the DMA channel.

ADDITIONAL INFORMATION: Set the `DMA_CFG.FLOW` for descriptor-list mode. Configure the `DMA_CFG.NDSIZE` bit to describe the number of descriptor elements contained within the first descriptor set. Configure the `DMA_CFG.WNR` bit for memory read operation. Configure the `DMA_CFG.PSIZE` bit to a value no larger than the supported bus width of the peripheral DMA bus. From performance perspective, it is recommended to use the largest possible `DMA_CFG.PSIZE` value (for example, equal to the supported peripheral bus width (`DMA_STAT.PBWID`)). However, ensure that the number of bytes in the work unit is a multiple of the `DMA_CFG.PSIZE` value used.

- The descriptor set that is fetched controls the `DMA_CFG.SYNC` configuration and controls the interrupt or trigger configurations.

The first descriptor set is fetched from the memory location provided by `DMA_DSCPTR_NXT` and is loaded to the MMR registers of the DMA channel.

Now, the DMA channel is processing all the work units provided in the descriptor list. The DMA channel enters the idle state when the final work unit that was configured for stop-flow mode is complete.

Register-Based Memory-to-Memory Transfer in Stop Flow Mode

This procedure configures a memory DMA channel pair in stop flow mode. One DMA channel is configured for memory read operations, while the other DMA channel is configured for memory write.

The task involves writing to a number of DMA channels on two DMA channels that create a memory DMA pair. On DMA completion, the DMA channel enters the idle state, until either the DMA channel is disabled or is reconfigured for a new transfer.

1. Write the `DMA_ADDRSTART` register of the source DMA channel.

ADDITIONAL INFORMATION: The address can be used to calculate the optimum `DMA_CFG.MSIZE` possible.

2. Calculate the optimum `DMA_CFG.MSIZE` based on the `DMA_ADDRSTART` register and number of bytes in work unit.

ADDITIONAL INFORMATION: The number of bytes in the work unit must be a multiple of the selected `DMA_CFG.MSIZE` and the start address alignment must also be considered.

3. Write the `DMA_XCNT` register of the source DMA channel based on calculated `DMA_CFG.MSIZE`.

ADDITIONAL INFORMATION: `DMA_XCNT` is the number of `DMA_CFG.MSIZE` transfers to make up the entire work unit.

4. Write the `DMA_XMOD` register of the source DMA channel.

ADDITIONAL INFORMATION: For a linear buffer transfer, determine the value in `DMA_XMOD` from the selected `DMA_CFG.MSIZE`. This register is always specified in the number of bytes.

5. Write the `DMA_ADDRSTART` register of the destination DMA channel.

ADDITIONAL INFORMATION: The address can be used to calculate the most optimum `DMA_CFG.MSIZE` possible.

6. Calculate the optimum `DMA_CFG.MSIZE` based on the `DMA_ADDRSTART` register and number of bytes in work unit.

ADDITIONAL INFORMATION: The number of bytes in the work unit must be a multiple of the selected `DMA_CFG.MSIZE` and the start address alignment must also be considered.

7. Write the `DMA_XCNT` register of the destination DMA channel based on the calculated `DMA_CFG.MSIZE`.

ADDITIONAL INFORMATION: `DMA_XCNT` is the number of `DMA_CFG.MSIZE` transfers to make up the entire work unit.

8. Write the `DMA_XMOD` register of the destination DMA channel.

ADDITIONAL INFORMATION: For a linear buffer transfer, determine the value in `DMA_XMOD` from the selected `DMA_CFG.MSIZE`. This register is always specified in the number of bytes.

9. Write the `DMA_CFG` register of the source DMA channel with `DMA_CFG.EN` configured to enable the DMA channel.

ADDITIONAL INFORMATION: The `DMA_CFG.FLOW` bit field must be configured for stop mode. The `DMA_CFG.WNR` bit must be cleared for memory read operation. The `DMA_CFG.PSIZE` bits must be configured to a value no larger than the supported bus width of the peripheral DMA bus. From performance perspective, it is recommended to use the largest possible `DMA_CFG.PSIZE` value (for example, equal to the supported peripheral bus width (`DMA_STAT.PBWID`)). However, ensure that the number of bytes in the work unit is a multiple of the `DMA_CFG.PSIZE` value used.

- The `DMA_CFG.SYNC` bit can be configured to control DMA completion notification timing.
- Interrupts and triggers also can be configured at this step, depending on requirements. The interrupts and triggers are enabled within the destination DMA channel configuration.

The memory read DMA transfer begins.

10. Write the `DMA_CFG` register of the destination DMA channel with `DMA_CFG.EN` configured to enable the DMA channel.

ADDITIONAL INFORMATION: The `DMA_CFG.FLOW` bit field must be configured for stop mode. The `DMA_CFG.WNR` bit must be set for memory write operation. The `DMA_CFG.PSIZE` bits must be configured to a value no larger than the supported bus width of the peripheral DMA bus. This value must also match the value written for the source DMA channel configuration.

- Interrupts and triggers also can be configured at this step depending on requirements.

The memory write DMA transfer begins.

Both memory DMA channels are now running and the data is transferred from the source address to the destination address. The DMA channel enters the IDLE state upon completion of the work unit.

Programming Concepts

Using the features, operating modes, and event control for the DMA channel to their greatest potential requires an understanding of some DMA channel-related concepts.

Synchronization of Software and DMA

A critical element of software DMA management is the synchronization of DMA work unit completion with software. This synchronization can be achieved using DMA channel interrupt request and trigger events and using a poll of the status bits of these events within the DMA channel registers, or combining these techniques. Processor polling of DMA address/count/status for completion is not a recommended programming practice. The requirements and limitations of processor polling place significant responsibility onto the code developer to be deeply aware of the underlying hardware. The interrupt requests and triggers are designed for efficient code development and reuse.

Interrupt and Trigger Event-Based Synchronization

Interrupt and trigger based synchronization methods must avoid overrun. An overrun occurs when some events fail to invoke the event handler of a DMA channel for every event due to excessive latency in processing of events. The system design must ensure to either:

- Schedule only one event per channel (for example, at the end of a descriptor list), or
- Space the generated events sufficiently far apart in time that system processing budgets can guarantee service of every event.

The DMA channel issues status information through an interrupt request or trigger event or changes event status bits in the [DMA_STAT](#) register. This status guarantees that the last memory operation of the work unit is complete. For memory read DMA transactions, this status means that the FIFO of the DMA channel safely receives the final memory read data. For DMA transactions writing to memory, this status indicates that the DMA channel received an acknowledge of completion of the last write transfer of the work unit.

Register Polling Based Synchronization

Do not poll the DMA channel registers ([DMA_ADDR_CUR](#), [DMA_DSCPTR_CUR](#), [DMA_XCNT_CUR](#), or [DMA_YCNT_CUR](#)) as a method of precisely synchronizing DMA with data processing. This approach is inaccurate due to the operation of the DMA channel FIFOs and DMA or memory pipelining. The current address, pointer, and count registers change several cycles in advance of the completion of the corresponding memory operation. This timing is measurable from the time at which the results of the operation are first visible to the core by memory read or write instructions.

For example, in a DMA channel memory write operation to external memory, assume DMA channel *A* initiates a DMA channel write operation. For memories with access latency, this operation requires many system-clock cycles. Meanwhile, DMA channel *B* (which does not in itself incur latency) initiates a transfer, which stalls behind the slow operation of channel *A*. Software monitoring channel *B* could not safely conclude whether the memory location pointed to by the [DMA_ADDR_CUR](#) of channel *B*. Also, the software cannot conclude whether the register has been written based solely on the contents of this register.

Polling of the current address, pointer, and count registers can permit loose synchronization of DMA with software. But, the software must allow for the lengths of the DMA or memory pipeline. Also, software must consider the length of the DMA FIFO for a particular peripheral. If the FIFOs are filled with incomplete work, the DMA channel does not advance current address, pointer, or count registers. The incomplete work includes reads that have been started but have not yet finished.

Additionally, software must consider the length of the pipelines to the destination memory. If the DMA FIFO length and channel memory-pipeline length are added, software can estimate the maximum number of incomplete memory operations in progress.

NOTE: The estimate would be a maximum, as the DMA or memory pipeline can include traffic from other DMA channels.

Descriptor Queues

A system designer may want to write a DMA manager facility which accepts DMA requests from other software. The DMA manager software does not know in advance when new work requests are received or what these requests contain. The software could manage these transfers using a circular linked list of DMA descriptors. In such a list, each descriptor sets the `DMA_DSCPTR_NXT` descriptor, which points to the next descriptor set. And the last descriptor set in the list points to the first descriptor set.

The code that writes into this descriptor list could use the circular addressing modes of the processor. This approach does not need to use comparison and conditional instructions to manage the circular structure. In this case, the `DMA_DSCPTR_NXT` descriptor of each descriptor set can be written once at startup, and skipped over as new contents are written for each descriptor.

The recommended method for synchronization of a descriptor queue is to use an interrupt or trigger. The descriptor queue is structured, such that (at least) the final valid descriptor set is always programmed to generate an interrupt or trigger event upon completion. More detail is provided in the following sections.

- [Queues Using Event Generation for Every Descriptor Set](#)
- [Queues Using Minimal Events](#)

Queues Using Event Generation for Every Descriptor Set

In this system, the DMA manager software synchronizes with the DMA channel by enabling an interrupt request or trigger on every descriptor set. Only use this method if the system design can guarantee that each work unit completion event is serviced separately (no interrupt or trigger overrun).

To maintain synchronization of the descriptor set queue, the non-interrupt software maintains a count of descriptor sets added to the queue. The event handler (either interrupt or trigger) maintains a count of completed descriptor sets removed from the queue. The counts are equal only when the DMA channel is paused after having processed all the descriptor sets.

When each new work unit event is received, the DMA manager software initializes a new descriptor set, taking care to set the flow to stop mode. Next, the software compares the descriptor set counts to determine whether the DMA channel is running. If the DMA channel is paused (counts equal), the software increments its count. Then, the software starts the DMA channel by writing the `DMA_CFG` of the new descriptor set.

If the counts are unequal, the software instead modifies the `DMA_CFG` of the next-to-last descriptor set, such that it now describes the newly queued descriptor set. This operation does not disrupt the DMA channel provided the rest of the descriptors of the set are initialized in advance. It is necessary to synchronize the software to the DMA to determine whether the DMA channel read the new or the old `DMA_CFG` value.

The event handler performs the synchronization operation. When an event is detected, the handler reads the `DMA_STAT` register of the DMA channel. If the `DMA_STAT.RUN` bit indicates that the DMA channel is running, the channel has moved on to processing another descriptor. The event handler can increment its count and exit. If the `DMA_STAT.RUN` bit indicates that the channel is not running, the channel is paused because either:

- There are no more descriptor sets to process, or
- The last descriptor set was queued too late

Where *too late* means that the modification of the `DMA_CFG` of the next-to-last descriptor set occurred *after* that descriptor was read into the DMA channel. In this case, the event handler does the following:

- Writes the `DMA_CFG` value appropriate for the last descriptor set to `DMA_CFG` register of the DMA channel,
- Increments the completed descriptor count, and
- Exits

If the event latencies of the system are large enough to cause any of the events to be dropped, this system can fail. An event handler capable of safely synchronizing multiple descriptor set interrupt requests is complex, performing several MMR accesses to ensure robust operation. In such a system environment, a minimal event synchronization method is preferred.

Queues Using Minimal Events

In this system, only one DMA interrupt request or trigger event is generated in the queue at any time. The DMA event handler for this system can also be extremely short. Here, the descriptor queue is organized into an *active* and a *waiting* portion, where events are enabled only on the last descriptor set in each portion.

When each new DMA request is processed, the software fills in the content of a new descriptor set and adds it to the waiting portion of the queue. The `DMA_CFG` descriptor of the descriptor set must have the flow set to stop mode. If more than one request is received before the DMA queue completion event occurs, the non-interrupt code queues later descriptor sets. It forms a waiting portion of the queue separate from the active portion of the queue that the DMA channel is processing. In other words, all but the last active descriptor sets contain flow values for a descriptor-based mode and have no event enable set.

The last active descriptor set has the stop flow mode and an event generation enabled. Also, all but the last waiting descriptor sets are configured for descriptor-based flow modes with no event generation. Only the last waiting descriptor set is configured for stop flow mode and event generation enabled. This configuration ensures that the DMA channel can automatically process the whole active queue before then issuing one event. Also, this arrangement makes it easy to start the waiting queue within the event handler by a single `DMA_CFG` register write.

After queuing a new waiting descriptor, the non-interrupt software leaves a message for its interrupt handler in a memory mailbox location. The location contains the desired `DMA_CFG` value for starting the first waiting descriptor set in the waiting queue (or 0, indicating no waiting descriptors).

The software must not modify the contents of the active descriptor set queue directly once processing by the DMA channel has started, unless careful synchronization measures are taken. In the most straightforward implementation of a descriptor set queue, the DMA manager software never modifies descriptors on the active queue. Instead, the DMA manager waits until the DMA queue completion event indicates that the processing of the entire active queue is complete.

When a DMA queue completion event is received, the event handler reads the mailbox from the non-interrupt software and writes the value to the `DMA_CFG` register of the DMA channel. This write to a register restarts the queue, effectively transforming the waiting queue to an active queue. The event handler then passes a message back to the non-interrupt software indicating the location of the last descriptor set accepted into the active queue.

However, the event handler can read its mailbox and find a `DMA_CFG` value of zero, indicating there is no more work to perform. It then passes an appropriate message back to the non-interrupt software indicating that the queue has stopped.

The non-interrupt software which accepts new DMA work unit requests must synchronize the activation of a new work unit with the interrupt handler. If the queue has stopped (the mailbox from the event handler is zero), the non-interrupt software must start the queue. (The queue starts by writing the first descriptor sets `DMA_CFG` value to the `DMA_CFG` register of the channel). If the queue is not stopped, the non-interrupt software must not write the `DMA_CFG` register. (This write causes a DMA error). Instead, it must queue the descriptor onto the waiting queue and update its mailbox directed to the event handler.

ADSP-21560_21561_21564_21568 DMA Register Descriptions

The Direct Memory Access module (DMA) contains the following registers.

Table 25-14: ADSP-21560_21561_21564_21568 DMA Register List

Name	Description
<code>DMA_ADDRSTART</code>	Start Address of Current Buffer Register
<code>DMA_ADDR_CUR</code>	Current Address Register
<code>DMA_BWLCNT</code>	Bandwidth Limit Count Register
<code>DMA_BWLCNT_CUR</code>	Bandwidth Limit Count Current Register
<code>DMA_BWMCNT</code>	Bandwidth Monitor Count Register
<code>DMA_BWMCNT_CUR</code>	Bandwidth Monitor Count Current Register
<code>DMA_CFG</code>	Configuration Register
<code>DMA_DSCPTR_CUR</code>	Current Descriptor Pointer Register
<code>DMA_DSCPTR_NXT</code>	Pointer to Next Initial Descriptor Register

Table 25-14: ADSP-21560_21561_21564_21568 DMA Register List (Continued)

Name	Description
DMA_DSCPTR_PRV	Previous Initial Descriptor Pointer Register
DMA_STAT	Status Register
DMA_XCNT	Inner Loop Count Start Value Register
DMA_XCNT_CUR	Current Count (1D) or Intra-row XCNT (2D) Register
DMA_XMOD	Inner Loop Address Increment Register
DMA_YCNT	Outer Loop Count Start Value (2D only) Register
DMA_YCNT_CUR	Current Row Count (2D only) Register
DMA_YMOD	Outer Loop Address Increment (2D only) Register

Start Address of Current Buffer Register

The `DMA_ADDRSTART` register contains the start address of the work unit currently targeted for DMA. This register is read/write prior to enabling the channel, but is read-only after enabling channel.

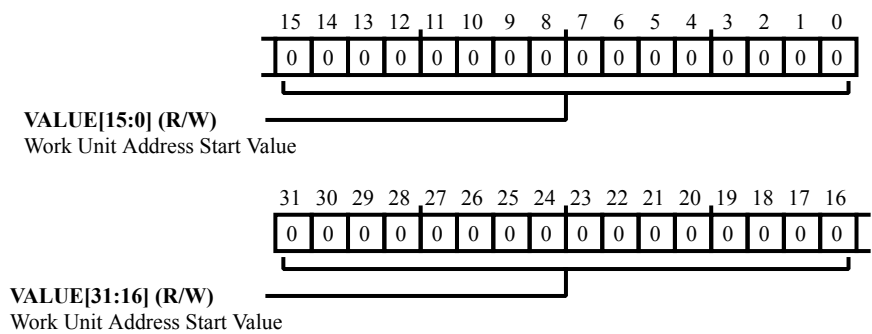


Figure 25-5: DMA_ADDRSTART Register Diagram

Table 25-15: DMA_ADDRSTART Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Work Unit Address Start Value. The <code>DMA_ADDRSTART.VALUE</code> bit field contains the start address of the work unit currently targeted for DMA.

Current Address Register

The `DMA_ADDR_CUR` register contains the present memory transfer address for a given work unit. At the start of a work unit, the `DMA_ADDR_CUR` register is loaded from the `DMA_ADDRSTART` register, and the `DMA_ADDR_CUR` register is incremented as each transfer occurs. The `DMA_ADDR_CUR` register is read/write prior to enabling the channel, but is read-only after enabling the channel.

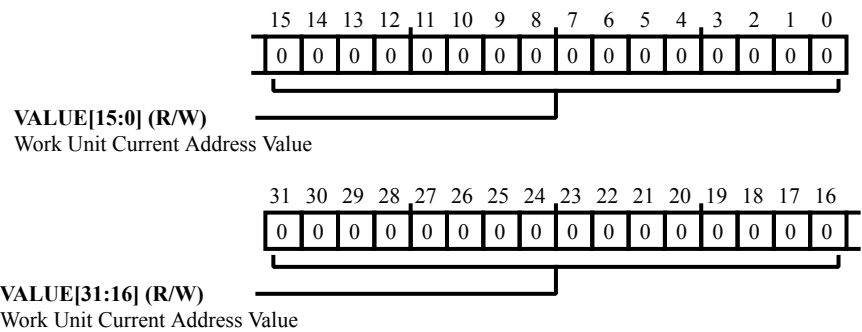


Figure 25-6: DMA_ADDR_CUR Register Diagram

Table 25-16: DMA_ADDR_CUR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Work Unit Current Address Value. The <code>DMA_ADDR_CUR.VALUE</code> bit field contains the present memory transfer address for a given work unit.

Bandwidth Limit Count Register

The `DMA_BWLCNT` register contains a count that determines how often the DMA issues memory transactions. The DMA loads the value from the `DMA_BWLCNT` register into the `DMA_BWLCNT_CUR` register and decrements the current value each SCLK cycle. When `DMA_BWLCNT_CUR` reaches 0x0000, the next request is issued, and the DMA reloads `DMA_BWLCNT_CUR`. This bandwidth limit functionality is not applied to descriptor fetch requests. Programming 0x0000 allows the DMA to request as often as possible. 0xFFFF is a special case and causes requests to stop.

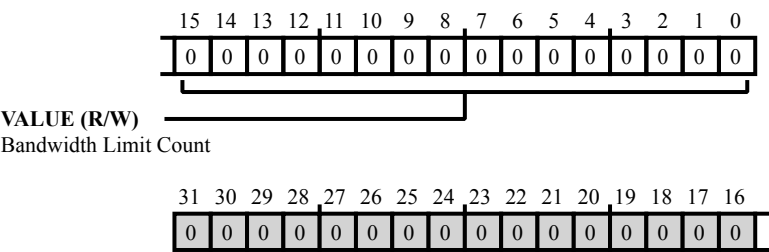


Figure 25-7: DMA_BWLCNT Register Diagram

Table 25-17: DMA_BWLCNT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	VALUE	Bandwidth Limit Count. The <code>DMA_BWLCNT.VALUE</code> bit field contains a count that determines how often the DMA issues memory transactions.

Bandwidth Limit Count Current Register

The `DMA_BWLCNT_CUR` register contains the number of SCLK count cycles remaining before the next request is issued.

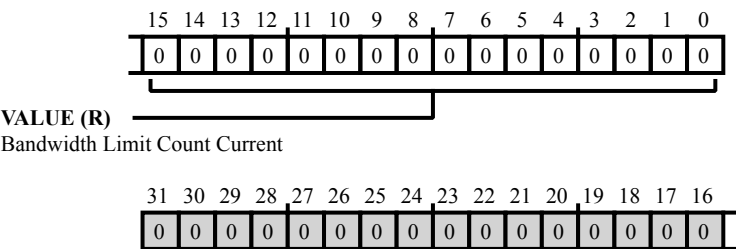


Figure 25-8: DMA_BWLCNT_CUR Register Diagram

Table 25-18: DMA_BWLCNT_CUR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/NW)	VALUE	Bandwidth Limit Count Current. The <code>DMA_BWLCNT_CUR.VALUE</code> bit field contains the number of SCLK count cycles remaining before the next request is issued.

Bandwidth Monitor Count Register

The `DMA_BWMCNT` register contains the maximum number of SCLK cycles allowed for a work unit to complete. Each time the `DMA_CFG` register is written (MMR access only), a work unit ends or an autobuffer wraps. The DMA loads the value in this register into the `DMA_BWMCNT_CUR` register.

The DMA decrements `DMA_BWMCNT_CUR` every SCLK a work unit is active. If the `DMA_BWMCNT_CUR` register reaches 0x0000_0000, the `DMA_STAT.IRQERR` bit is set, and the `DMA_STAT.ERRC` bit field is set to 0x6. The `DMA_BWMCNT_CUR` remains at 0x0000_0000 until it is reloaded when the work unit completes.

Unlike other errors, a bandwidth monitor error does not stop work unit processing. Programming 0x0000_0000 disables bandwidth monitor functionality.

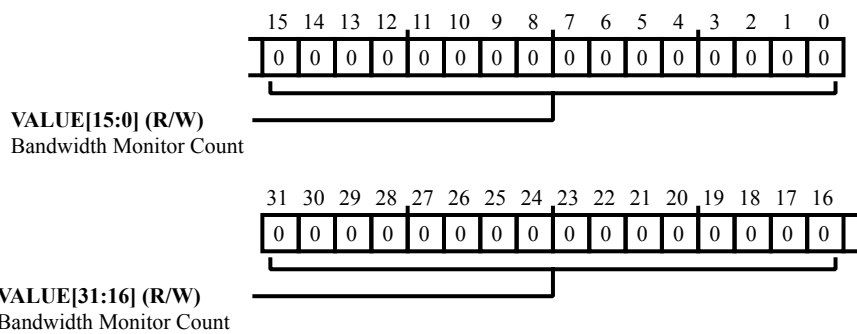


Figure 25-9: `DMA_BWMCNT` Register Diagram

Table 25-19: `DMA_BWMCNT` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Bandwidth Monitor Count. The <code>DMA_BWMCNT.VALUE</code> bit field contains the maximum number of SCLK cycles allowed for a work unit to complete.

Bandwidth Monitor Count Current Register

The `DMA_BWMCNT_CUR` register contains the number of cycles remaining for the current descriptor to complete.

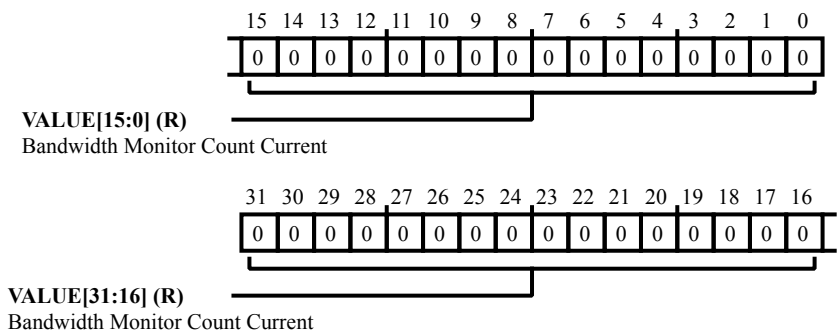


Figure 25-10: `DMA_BWMCNT_CUR` Register Diagram

Table 25-20: `DMA_BWMCNT_CUR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	Bandwidth Monitor Count Current. The <code>DMA_BWMCNT_CUR.VALUE</code> bit field contains the number of cycles remaining for the current descriptor to complete.

Configuration Register

The `DMA_CFG` register sets up DMA parameters and operation modes. Writing to the `DMA_CFG` register while a DMA process is already running causes a DMA error (except when clearing the `DMA_CFG.EN` bit).

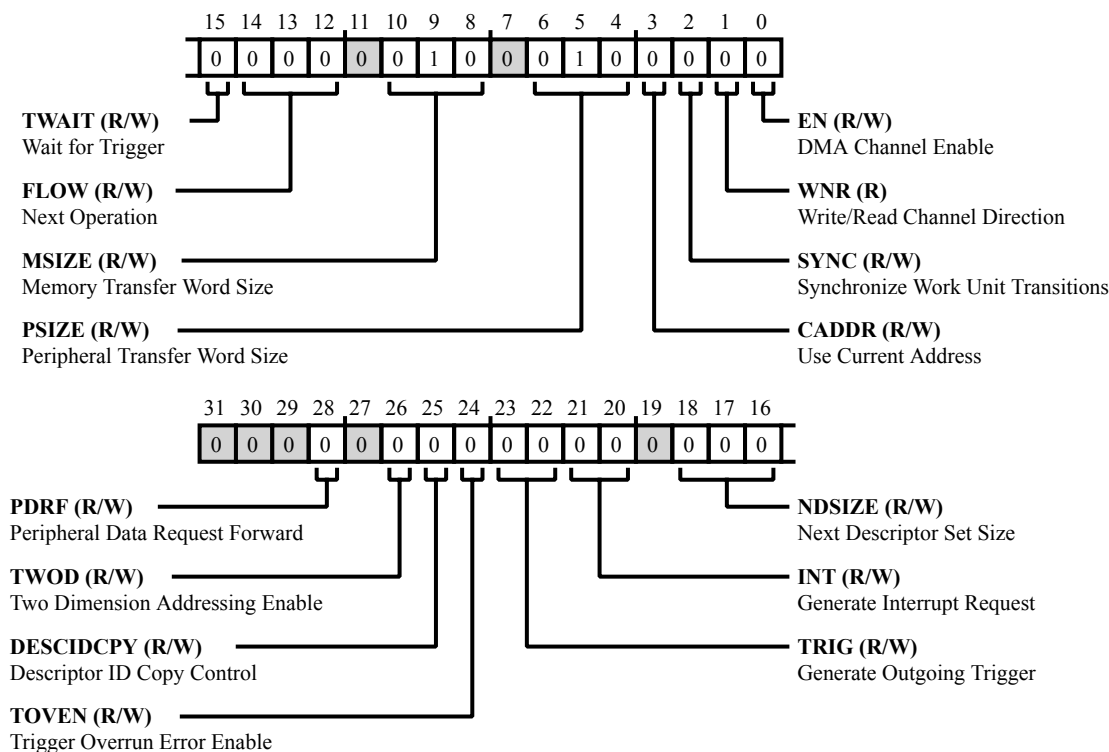


Figure 25-11: `DMA_CFG` Register Diagram

Table 25-21: `DMA_CFG` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
28 (R/W)	PDRF	Peripheral Data Request Forward. The <code>DMA_CFG.PDRF</code> bit defines how the DMA handles data requests from the peripheral while in idle state after a stop mode or memory read work unit. If set, the DMA forwards the peripheral data request as an interrupt. This bit applies only to the <code>DMA_CFG.FLOW</code> bits configured for stop and <code>DMA_CFG.WNR</code> bits configured for memory read.
		0 Peripheral Data Request Not Forwarded
		1 Peripheral Data Request Forwarded

Table 25-21: DMA_CFG Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
26 (R/W)	TWOD	Two Dimension Addressing Enable. The DMA_CFG.TWOD bit selects whether the DMA addressing involves only DMA_XCNT and DMA_XMOD (one-dimensional DMA) or also involves DMA_YCNT and DMA_YMOD (two-dimensional DMA).
		0 One-Dimensional Addressing
		1 Two-Dimensional Addressing
25 (R/W)	DESCIDCPY	Descriptor ID Copy Control. The DMA_CFG.DESCIDCPY bit specifies when to copy the initial descriptor pointer to the DMA_DSCPTR_PRV register. A bus write to the DMA_CFG register to clear the DMA_CFG.EN bit causes the DMA to immediately use the new value of the DMA_CFG.DESCIDCPY bit. To preserve consistency (if required by application), match the new value of this bit to the previous value.
		0 Never Copy
		1 Copy on Work Unit Complete
24 (R/W)	TOVEN	Trigger Overrun Error Enable. A trigger overrun occurs if more than one trigger was received before the DMA reached the trigger wait state. If DMA_CFG.TOVEN is set, a trigger overrun causes the DMA to flag an error. In cases where a trigger overrun is not a problem, clearing DMA_CFG.TOVEN prevents the overrun from causing an error and halting the DMA. The DMA_CFG.TOVEN operates independently of the DMA_CFG.TWAIT bit selection.
		0 Ignore Trigger Overrun
		1 Error on Trigger Overrun

Table 25-21: DMA_CFG Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
23:22 (R/W)	TRIG	Generate Outgoing Trigger. The DMA_CFG.TRIG selects whether the DMA issues an outgoing trigger, based on the work unit counter values. In one-dimensional mode, the only options are to trigger at the end of the whole work unit (trigger when DMA_XCNT_CUR reaches 0) or not to trigger at all. If in one-dimensional addressing mode, programming the DMA_CFG.TRIG bit field to trigger when DMA_YCNT_CUR reaches 0 (or to reserved) causes the DMA to flag a configuration error. In two-dimensional addressing mode, the trigger options are: at the end of each row of the inner loop (when DMA_XCNT_CUR reaches 0), only after completing the whole work unit (when DMA_YCNT_CUR reaches 0), or no trigger. If in two-dimensional mode and set to trigger when DMA_XCNT_CUR reaches 0, the DMA also issues a trigger at the end of the work unit. If in two-dimensional addressing mode, programming DMA_CFG.TRIG to reserved causes the DMA to flag a configuration error. If DMA_CFG.TRIG is non-zero and the peripheral issues a finish command, the DMA issues a trigger after the finish procedure is complete. For more information about trigger generation timing, see the trigger section of the DMA functional description.
		0 Never Assert Trigger
		1 Trigger When XCNTCUR Reaches 0
		2 Trigger When YCNTCUR Reaches 0
		3 Reserved

Table 25-21: DMA_CFG Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
21:20 (R/W)	INT	Generate Interrupt Request. The DMA_CFG . INT bit field selects whether an interrupt request goes to the core based on work unit status or a peripheral interrupt request. For one-dimensional mode, setting the DMA_CFG . INT bits to generate an interrupt request when the DMA_YCNT_CUR register reaches 0 causes the DMA to flag a configuration error. The peripheral interrupt setting lets the DMA generate the last grant indication and to accept and or forward the peripheral interrupt command. The peripheral interrupt selection applies only to the DMA_CFG . FLOW bits set for stop and the DMA_CFG . WNR bits set for memory read. If the DMA_CFG . INT is set for interrupt request on count completion (DMA_XCNT_CUR or DMA_YCNT_CUR reach 0) and the peripheral issues a finish command, the DMA issues an interrupt request after the finish procedure is complete. For more information, see the sections on interrupt generation and peripheral control in the DMA functional description.
		0 Never Assert Interrupt
		1 Interrupt When X Count Expires
		2 Interrupt When Y Count Expires
		3 Peripheral Interrupt request
18:16 (R/W)	NDSIZE	Next Descriptor Set Size. The DMA_CFG . NDSIZE bit field specifies the number of descriptor elements in memory to load during the next descriptor fetch. The DMA loads the descriptors in a specific order. The descriptor set contains the next descriptor pointer when it is a descriptor list. The descriptor set does not contain the next descriptor pointer when it is a descriptor array.
		0 Fetch One Descriptor Element
		1 Fetch Two Descriptor Elements
		2 Fetch Three Descriptor Elements
		3 Fetch Four Descriptor Elements
		4 Fetch Five Descriptor Elements
		5 Fetch Six Descriptor Elements
		6 Fetch Seven Descriptor Elements
		7 Reserved

Table 25-21: DMA_CFG Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W)	TWAIT	Wait for Trigger. The DMA_CFG.TWAIT bit controls whether the DMA waits for an incoming trigger from another channel or user. If the DMA_CFG.TWAIT bit is set, the DMA enters the wait state before starting the next work unit, including descriptor fetch when in descriptor mode. Do not use the wait for trigger control for descriptor-on-demand mode which causes an error. For more information, see the trigger section of the DMA functional description.
		0 Begin Work Unit Automatically (No Wait)
		1 Wait for Trigger (Halt before Work Unit)
14:12 (R/W)	FLOW	Next Operation. The DMA_CFG.FLOW bit field selects the descriptor handling options.
		0 STOP. See the Stop Flow Mode section.
		1 AUTO. See the Autobuffer Flow Mode section.
		2 Reserved
		3 Reserved
		4 DSCL. See the Descriptor List Mode section.
		5 DSCA. See the Descriptor Array Mode section.
		6 Descriptor On-Demand List. See the Descriptor List Mode section.
		7 Descriptor On Demand Array. See the Descriptor On Demand section.
10:8 (R/W)	MSIZE	Memory Transfer Word Size. The DMA_CFG.MSIZE bits select memory transfer sizes of 8-, 16-, 32-, 64-, 128-, or 256-bit words. The transfer start address (DMA_ADDRSTART) and transfer increment values (DMA_XMOD, and, if needed, DMA_YMOD) must be a multiple of the memory transfer unit size.
		2 4 Bytes
		3 8 Bytes
		4 16 Bytes
		5 32 Bytes

Table 25-21: DMA_CFG Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
6:4 (R/W)	PSIZE	Peripheral Transfer Word Size. The DMA_CFG.PSIZE bits select peripheral transfer sizes as 8, 16, 32, or 64 bits wide. Each request and grant results in a single peripheral access. There are no bursts on the peripheral bus, so the DMA_CFG.PSIZE selection must be less than or equal to the width of the bus. If the selection is greater than the bus width, a configuration error occurs. The peripheral bus of the processor is dedicated to DMA and peripheral accesses.
		2 4 Bytes
		3 8 Bytes
3 (R/W)	CADDR	Use Current Address. When the DMA_CFG.CADDR bit is cleared, the DMA loads the DMA_ADDRSTART register on the first access of the work unit. When the DMA_CFG.CADDR bit is set, the DMA uses the DMA_ADDR_CUR register value for the starting address for the work unit and writes the same value to the DMA_ADDRSTART register. This operation permits continuation of a previous work unit. When DMA uses this mode, the fetched start address value (as part of the descriptor set at the end of a descriptor list or array) is ignored.
		0 Load Starting Address
		1 Use Current Address

Table 25-21: DMA_CFG Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
2 (R/W)	SYNC	Synchronize Work Unit Transitions. Setting the DMA_CFG.SYNC bit clears the DMA FIFO and pointers before starting the first work unit of a work unit chain. When the transfer direction is memory read/transmit (DMA_CFG.WNR =0), the DMA waits until all data transmits to a peripheral before moving on to the next work unit, clearing the FIFO and pointers. When the transfer direction is memory write/receive (DMA_CFG.WNR =1), the DMA ignores the DMA_CFG.SYNC bit value after processing the first work unit of a work unit chain. As a channel can receive data when turned on but idle, data from the peripheral can still be in the FIFO even though the channel was not programmed. When the DMA_CFG.SYNC bit field is set at the beginning of a work unit chain (during the first work unit), the DMA clears the FIFO, erasing the data put into the FIFO while the channel was idle. Syncing lets programs change the DMA_CFG.PSIZE between individual work units and (in some cases) work unit chains. The sync resets the pointers in the FIFO, preventing misaligned FIFO access. Programs can change the DMA_CFG.MSIZE field between consecutive work units, independent of the DMA_CFG.SYNC bit setting. Syncing also permits changes to transfer direction. Because the data in the FIFO is eliminated, the data that went into the FIFO from one direction (transmit or receive) is not sent back in the other direction after the direction change.
		0 No Synchronization
		1 Synchronize Channel
1 (R/NW)	WNR	Write/Read Channel Direction. The DMA_CFG.WNR selects receive (write to memory) or transmit (read from memory) channel direction.
		0 Transmit (Read from memory)
		1 Receive (Write to memory)

Table 25-21: DMA_CFG Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	EN	DMA Channel Enable. The <code>DMA_CFG.EN</code> bit enables the selected DMA channel. When a peripheral DMA channel is enabled, data requests from the peripheral denote DMA requests. When a channel is disabled, the DMA unit ignores the peripheral data request and passes it directly to the system event controller. To avoid unexpected results, enable the DMA channel before enabling the peripheral, and disable the peripheral before disabling the DMA channel. A transition of the <code>DMA_CFG.EN</code> bit from 0 to 1 creates a hard reset of all internal counters and states, including the <code>DMA_STAT</code> register. (All other register values remain unaffected.) A transition from 1 to 0 maintains all counters and registers for reading and analysis. Note that if a descriptor loads when this bit is cleared (see the <code>DMA_CFG.FLOW</code> field), the DMA transitions to the off or idle state after the descriptor load is complete.
		0 Disable
		1 Enable

Current Descriptor Pointer Register

The `DMA_DSCPTR_CUR` register contains the memory address for the next descriptor to be loaded. The `DMA_DSCPTR_CUR` register is read/write prior to enabling the channel, but is read-only after enabling the channel. For `DMA_CFG.FLOW` mode settings that involve descriptor fetches, this register is used to read descriptors into appropriate MMRs before a work unit begins. For descriptor list mode, the `DMA_DSCPTR_CUR` register is initialized from the `DMA_DSCPTR_NXT` register before fetching each descriptor set. Then, the address in the `DMA_DSCPTR_CUR` register increments as each descriptor is read.

When the entire descriptor set has been read, the `DMA_DSCPTR_CUR` register contains this value:

`DMA_DSCPTR_CUR` = Descriptor Start Address + Descriptor Size (# of elements)

For descriptor array mode, the `DMA_DSCPTR_CUR` register, and not the `DMA_DSCPTR_NXT` register, must be programmed by a MMR access before starting DMA operation.

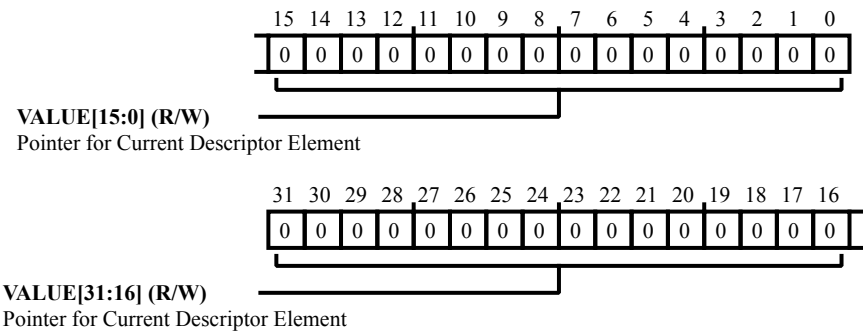


Figure 25-12: `DMA_DSCPTR_CUR` Register Diagram

Table 25-22: `DMA_DSCPTR_CUR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Pointer for Current Descriptor Element. The <code>DMA_DSCPTR_CUR.VALUE</code> bit field contains the memory address for the next descriptor to be loaded.

Pointer to Next Initial Descriptor Register

The `DMA_DSCPTR_NXT` register specifies the start location of the next descriptor set, which begins when the DMA activity specified by the current descriptor set completes. This register is read/write prior to enabling the channel, but is read-only after enabling channel.

The `DMA_DSCPTR_NXT` register is only used in descriptor list mode. At the start of a descriptor fetch in this mode, the `DMA_DSCPTR_NXT` register is copied into the `DMA_DSCPTR_CUR` register. During descriptor fetch, the DMA increments the `DMA_DSCPTR_CUR` register value after reading each element of the descriptor set.

In descriptor list mode, the `DMA_DSCPTR_NXT` register (not the `DMA_DSCPTR_CUR` register) must be programmed directly through MMR access, before the DMA operation is started. In descriptor array mode, the DMA disregards the `DMA_DSCPTR_NXT` register and uses the `DMA_DSCPTR_CUR` register to control descriptor fetch.

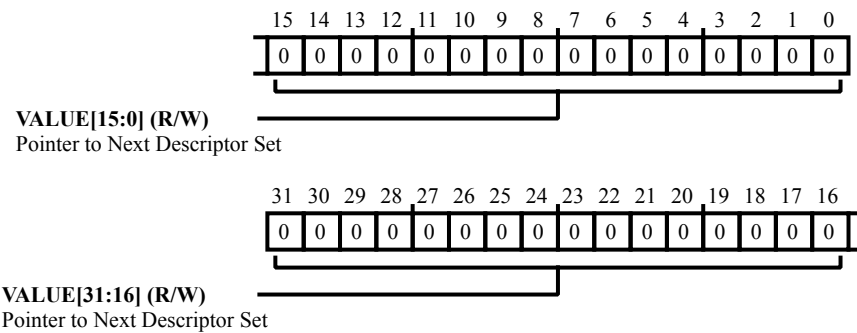


Figure 25-13: `DMA_DSCPTR_NXT` Register Diagram

Table 25-23: `DMA_DSCPTR_NXT` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Pointer to Next Descriptor Set. The <code>DMA_DSCPTR_NXT.VALUE</code> bit field specifies the start location of the next descriptor set.

Previous Initial Descriptor Pointer Register

The `DMA_DSCPTR_PRV` register contains the initial descriptor pointer for the previous work unit. If `DMA_CFG.DESCIDCPY` is set, the DMA copies the initial descriptor pointer to `DMA_DSCPTR_PRV` after the work unit completes. Otherwise, the value is not updated.

To indicate an overrun, bit 0 of the `DMA_DSCPTR_PRV` register is used as a previous descriptor pointer overrun (PDPO) status bit. Due to aligned addressing combined with all descriptors being 32 bits in width, bits 0 and 1 of all descriptor pointers must be zero. Otherwise, an alignment error occurs when used for descriptor fetches. As a result, bit 1 and 0 of the `DMA_DSCPTR_PRV` register can be used for status. For more information, see the section on descriptor pointer capture in the DMA functional description.

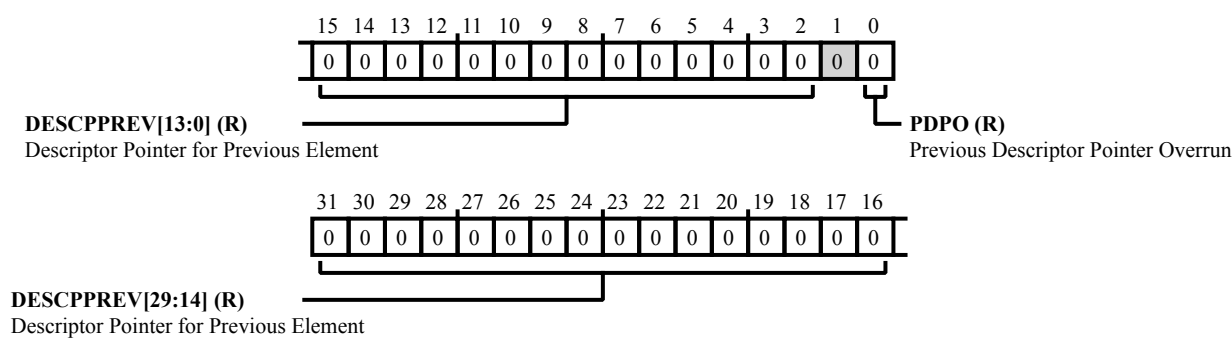


Figure 25-14: DMA_DSCPTR_PRV Register Diagram

Table 25-24: DMA_DSCPTR_PRV Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:2 (R/NW)	DESCPPREV	Descriptor Pointer for Previous Element. The <code>DMA_DSCPTR_PRV.DESCPPREV</code> bit field contains the initial descriptor pointer for the previous work unit.
0 (R/NW)	PDPO	Previous Descriptor Pointer Overrun. The <code>DMA_DSCPTR_PRV.PDPO</code> bit signifies an alignment error. Due to aligned addressing combined with all descriptors being 32 bits in width, bits 0 and 1 of all descriptor pointers must be zero. Otherwise, an alignment error would occur when used for descriptor fetches.
		0 No Error Occurred
		1 Error Occurred

Status Register

The `DMA_STAT` register indicates the status of DMA work units, the FIFO, errors, interrupts, and triggers.

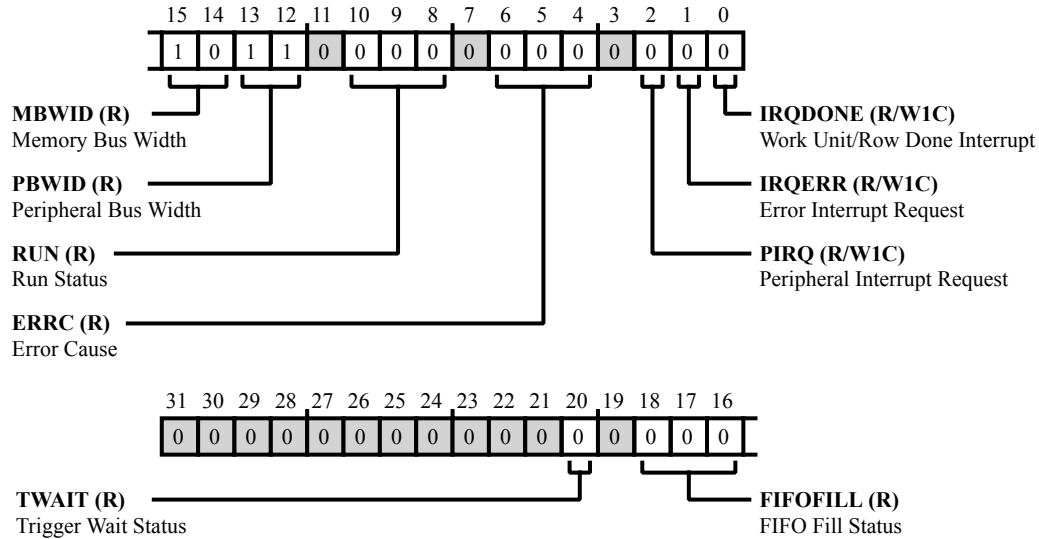


Figure 25-15: `DMA_STAT` Register Diagram

Table 25-25: `DMA_STAT` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
20 (R/NW)	TWAIT	Trigger Wait Status. The <code>DMA_STAT.TWAIT</code> bit indicates whether the DMA has or has not received a trigger. This bit is set until it reaches the next wait state. At that point, the bit is cleared, the DMA stops processing that work unit, and the following work unit processes. The DMA does not distinguish between one or more triggers received.
		0 No Trigger Received
		1 Trigger Received
18:16 (R/NW)	FIFOFILL	FIFO Fill Status. The <code>DMA_STAT.FIFOFILL</code> bit field reports the quantity of data in the FIFO relative to available space.
		0 Empty
		1 Empty < FIFO = 1/4 Full
		2 1/4 Full < FIFO = 1/2 Full
		3 1/2 Full < FIFO = 3/4 Full
		4 3/4 Full < FIFO = Full
		5 Reserved

Table 25-25: DMA_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
		6	Reserved
		7	Full
15:14 (R/NW)	MBWID	Memory Bus Width. The DMA_STAT.MBWID bit field indicates the width of the memory bus connected to this DMA.	
		0	2 Bytes
		1	4 Bytes
		2	8 Bytes
		3	16 Bytes
13:12 (R/NW)	PBWID	Peripheral Bus Width. The DMA_STAT.PBWID bit field indicates the width of the peripheral bus connected to this DMA.	
		0	1 Byte
		1	2 Bytes
		2	4 Bytes
		3	8 Bytes
10:8 (R/NW)	RUN	Run Status. The DMA_STAT.RUN bit field reports the DMA's current operational state. If the DMA is in idle or stop state, the DMA_CFG.EN bit is either 0 or 1. This bit field does not clear when a transition of the DMA_CFG.EN bit from 0 to 1 occurs. The DMA_STAT.RUN clears automatically when the DMA completes.	
		0	Idle/Stop State
		1	Descriptor Fetch
		2	Data Transfer
		3	Waiting for Trigger
		4	Waiting for Write ACK/FIFO Drain to Peripheral
		5	Reserved
		6	Reserved
		7	Reserved
6:4 (R/NW)	ERRC	Error Cause. When an interrupt request error occurs (DMA_STAT.IRQERR), the DMA updates the DMA_STAT.ERRC bit field to identify the type of error. For more information, see the errors section of the DMA functional description.	

Table 25-25: DMA_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
		0	Configuration Error
		1	Illegal Write Occurred While Channel Running
		2	Address Alignment Error
		3	Memory Access or Fabric Error
		4	Reserved
		5	Trigger Overrun
		6	Bandwidth Monitor Error
		7	Reserved
2 (R/W1C)	PIRQ	Peripheral Interrupt Request. The DMA_STAT . PIRQ bit indicates a peripheral interrupt request. Programs can use the DMA_STAT . PIRQ bit status to help determine which DMA asserted the interrupt request. This bit also helps to distinguish between an interrupt request caused by the state of the work unit and an interrupt request caused by the peripheral.	
		0	No Interrupt request
		1	Interrupt Request signaled by peripheral
1 (R/W1C)	IRQERR	Error Interrupt Request. The DMA_STAT . IRQERR bit indicates that the DMA has detected a documented rule violation during DMA programming or operation. The DMA cannot, however, flag all possible programming or operation issues to indicate errors. Programmers can use the DMA_STAT . IRQERR bit to help determine which DMA issued the error interrupt request. The DMA_STAT . IRQERR does not clear a transition of the DMA_CFG . EN bit from 0 to 1. Clear the DMA_STAT . IRQERR bit with a write-1-to-clear operation prior to the DMA_CFG . EN transition for the fields to be reset.	
		0	No Error
		1	Error Occurred
0 (R/W1C)	IRQDONE	Work Unit/Row Done Interrupt. The DMA_STAT . IRQDONE bit indicates that the DMA has detected the completion of a work unit or row (inner loop count) and has issued an interrupt request. Programs can use the DMA_STAT . IRQDONE status to help determine which DMA asserted the interrupt request. Programs can also use these bits to help distinguish between an interrupt request based on the state of the work unit and an interrupt request made by the peripheral. For more information, see the interrupts section of the DMA functional description.	
		0	Inactive
		1	Active

Inner Loop Count Start Value Register

For 2D DMA, the `DMA_XCNT` register contains the inner loop count. This value selects the number of `DMA_CFG.MSIZE` size data transfers that make up the length of a row. For 1D DMA, the `DMA_XCNT` register specifies the number of `DMA_CFG.MSIZE` size data transfers for the entire work unit. The `DMA_XCNT` register is read/write prior to enabling the channel, but is read-only after enabling channel. Note that the DMA generates a configuration error if this register is 0x0 when a work unit begins.

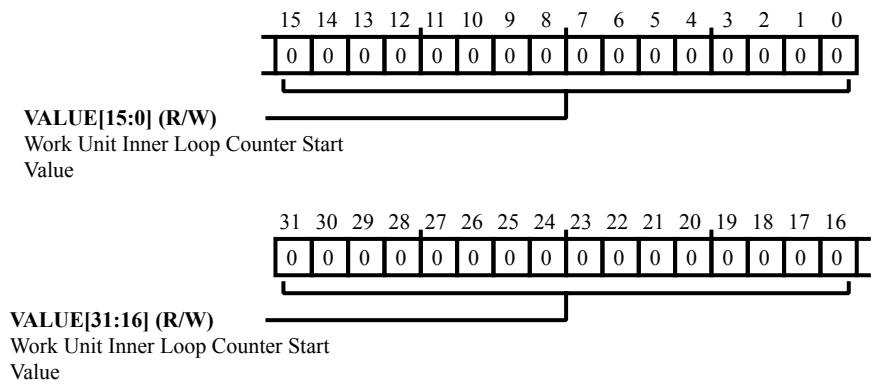


Figure 25-16: DMA_XCNT Register Diagram

Table 25-26: DMA_XCNT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Work Unit Inner Loop Counter Start Value. For 2D DMA, the <code>DMA_XCNT.VALUE</code> bit field contains the inner loop count. For 1D DMA, <code>DMA_XCNT.VALUE</code> specifies the number of <code>DMA_CFG.MSIZE</code> size data transfers for the entire work unit.

Current Count (1D) or Intra-row XCNT (2D) Register

For 1D DMA, the DMA loads the `DMA_XCNT_CUR` register from the `DMA_XCNT` register at the beginning of each work unit. For 2D DMA, the DMA loads the `DMA_XCNT_CUR` register from the `DMA_XCNT` register after the end of each row. The DMA decrements the value in `DMA_XCNT_CUR` register each time a `DMA_CFG.MSIZE` size data transfer occurs. When the count in `DMA_XCNT_CUR` register expires, the work unit is complete. In 2D DMA, the `DMA_XCNT_CUR` value is 0 only when the entire transfer is complete.

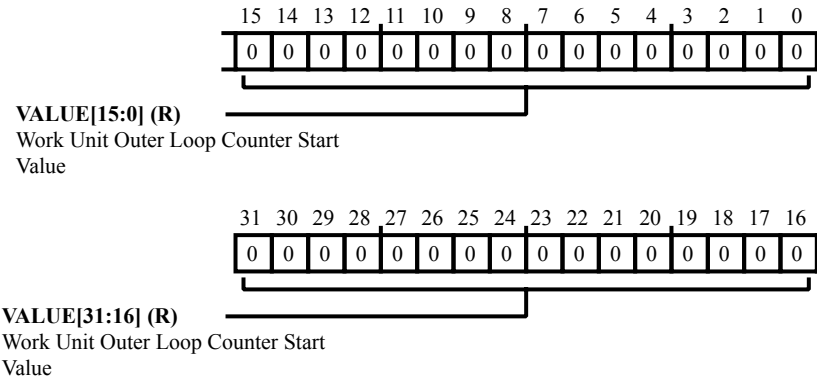


Figure 25-17: DMA_XCNT_CUR Register Diagram

Table 25-27: DMA_XCNT_CUR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	Work Unit Outer Loop Counter Start Value. For 1D DMA, the <code>DMA_XCNT_CUR.VALUE</code> bit field holds the <code>DMA_XCNT</code> value at the beginning of each work unit. For 2D DMA, the <code>DMA_XCNT_CUR.VALUE</code> bit field holds the <code>DMA_XCNT</code> value after the end of each row.

Inner Loop Address Increment Register

The `DMA_XMOD` register contains a signed, two's-complement byte address increment. In 1D DMA, this increment is the stride that is applied after each `DMA_CFG.MSIZE` size data transfer. The `DMA_XMOD` register is read/write prior to enabling the channel, but is read-only after enabling the channel.

The `DMA_XMOD` register value is specified in bytes, regardless of the work unit size. In 2D DMA, this increment is applied after each `DMA_CFG.MSIZE` size data transfer in the inner loop, up to but not including the last `DMA_CFG.MSIZE` size data transfer in each inner loop. After the last `DMA_CFG.MSIZE` size data transfer in each inner loop, the `DMA_YMOD` register is applied instead, including the last `DMA_CFG.MSIZE` size data transfer of a work unit.

The `DMA_XMOD` field can be set to 0. In this case, DMA is performed repeatedly to or from the same address. This approach can be useful for transferring data between a data register and an external memory-mapped peripheral.

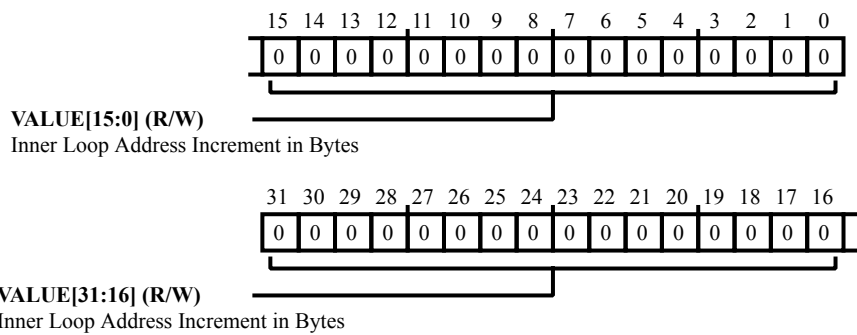


Figure 25-18: DMA_XMOD Register Diagram

Table 25-28: DMA_XMOD Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Inner Loop Address Increment in Bytes. The <code>DMA_XMOD.VALUE</code> bit field contains a signed, two's-complement byte address increment.

Outer Loop Count Start Value (2D only) Register

For 2D DMA, the DMA_YCNT register contains the outer loop count. This register is not used in 1D DMA mode. The DMA_YCNT register specifies the number of rows in the outer loop of a 2D DMA sequence. The DMA_YCNT register is read/write prior to enabling the channel, but is read-only after enabling channel. Note that the DMA generates a configuration error if this register is 0x0 when a work unit begins.

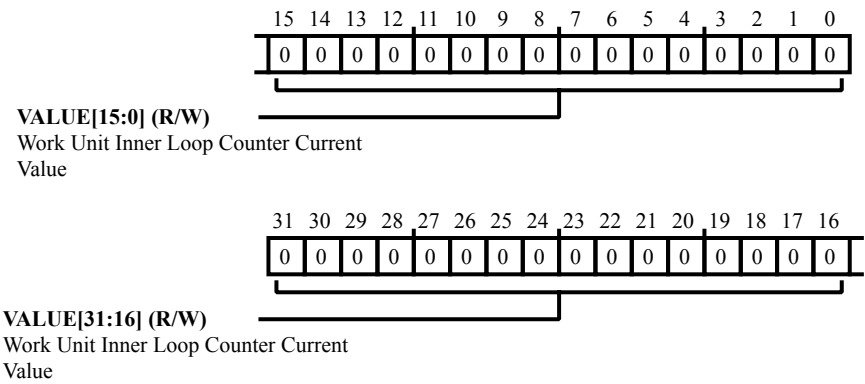


Figure 25-19: DMA_YCNT Register Diagram

Table 25-29: DMA_YCNT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Work Unit Inner Loop Counter Current Value. For 2D DMA, the DMA_YCNT . VALUE bit field contains the outer loop count.

Current Row Count (2D only) Register

For 2D DMA, the DMA loads the `DMA_YCNT_CUR` register from the `DMA_YCNT` register at the beginning of each 2D DMA session. The `DMA_YCNT_CUR` register is not used for 1D DMA. The DMA decrements the `DMA_YCNT_CUR` register each time the `DMA_XCNT_CUR` register expires during 2D DMA operation, signifying the completion of an entire row transfer.

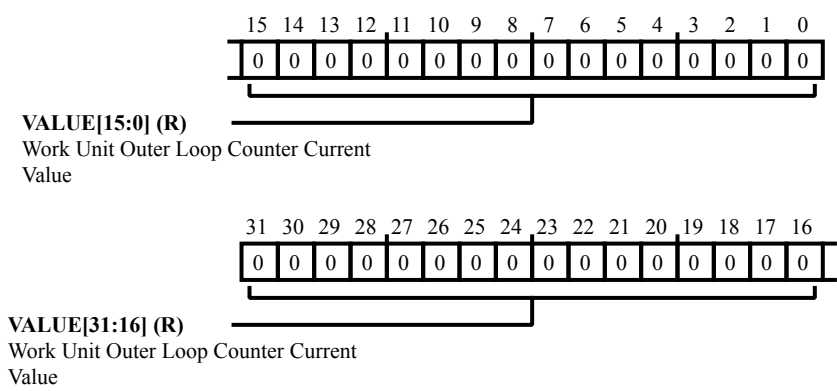


Figure 25-20: `DMA_YCNT_CUR` Register Diagram

Table 25-30: `DMA_YCNT_CUR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	Work Unit Outer Loop Counter Current Value. For 2D DMA, the <code>DMA_YCNT_CUR.VALUE</code> bit field holds the value from the <code>DMA_YCNT</code> register at the beginning of each 2D DMA session.

Outer Loop Address Increment (2D only) Register

The `DMA_YMOD` register contains a signed, two's-complement value. This byte address increment is applied after each decrement of the `DMA_YCNT_CUR` register. The value is the offset between the last word of one row and the first word of the next row. Note that `DMA_YMOD` is specified in bytes, regardless of the work unit size. The `DMA_YMOD` register is read/write prior to enabling the channel, but is read-only after enabling the channel.

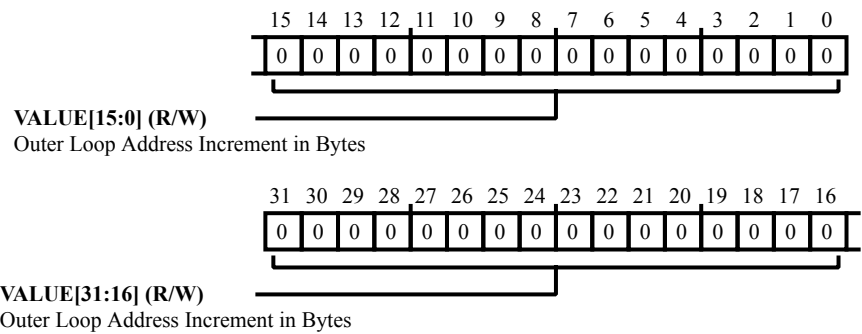


Figure 25-21: DMA_YMOD Register Diagram

Table 25-31: DMA_YMOD Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Outer Loop Address Increment in Bytes. The <code>DMA_YMOD.VALUE</code> bit field contains a signed, two's-complement value.

26 Extended Memory DMA (EMDMA)

The Extended Memory DMA engine can be used in applications that copy data in a non-sequential manner. This includes delay lines, scatter and gather, and circular access types.

NOTE: Previous SHARC processors featured external port DMA. Current SHARC products use EMDMA that can access all memory locations for source and destination DMA operations.

EMDMA Features

EMDMA frees the processor core, allowing it to perform other operations while the data transfers between memories occurs as a background task.

EMDMA has the following features and capabilities.

- Standard mode DMA Transfer
- Chained mode with direction on-the-fly
- Tap list mode (scatter/gather)
- Delay line mode (write to read)
- All the DMA modes can operate in circular fashion
- In circular operation, some modes allow a write back of the index pointer for correct addressing of the next Transfer Control Block (TCB)

EMDMA Functional Description

The following sections provide information on the functional operations and operating modes of EMDMA.

Internal-to-internal DMA

This is accomplished by indexing all external parameter registers with internal addresses.

DMA bursting

DMA supports burst transfers only when the modifier is 1, for any other modifier values, single accesses are performed. The burst size is not user configurable and is chosen by the processor for optimal performance. The maximum burst size is 8 and the minimum is 1.

Transfer control blocks

The structure of a TCB is conceptually the same as that of a traditional linked list. Each TCB has several data values and a pointer to the next TCB. Further, the chain pointer of a TCB may point to itself to continuously re-run the same DMA.

Chain pointer DMA

In chained DMA operations, the processor automatically initializes and then begins another DMA transfer when the current DMA transfer is complete.

ADSP-21560_21561_21564_21568 EMDMA Register List

The EMDMA controllers support a variety of direct memory access operations which can access any system memory and transfer the entire block of data. A set of registers govern EMDMA operations. For more information on EMDMA functionality, see the EMDMA register descriptions.

Table 26-1: ADSP-21560_21561_21564_21568 EMDMA Register List

Name	Description
EMDMA_BASE	External Base Address Register
EMDMA_BUFLEN	Circular Buffer Length Register
EMDMA_CHNPTR	Chain Pointer Register
EMDMA_CNT0	Internal Count Register
EMDMA_CNT1	External Count Register
EMDMA_CTL	External Memory DMA Control Register
EMDMA_INDX0	Internal Index Register
EMDMA_INDX1	External Index Register
EMDMA_MOD0	Internal Modifier Register
EMDMA_MOD1	External Modifier Register
EMDMA_TCNT	Delay Line Tap Count Register
EMDMA_TPTR	Tap List Pointer Register

ADSP-21560_21561_21564_21568 EMDMA Interrupt List

Table 26-2: ADSP-21560_21561_21564_21568 EMDMA Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
130	EMDMA0_DONE	EMDMA0 Transfer Done	Edge	
131	EMDMA1_DONE	EMDMA1 Transfer Done	Edge	

ADSP-21560_21561_21564_21568 EMDMA Trigger List

Table 26-3: ADSP-21560_21561_21564_21568 EMDMA Trigger List Generators

Trigger ID	Name	Description	Sensitivity
57	EMDMA0_DONE	EMDMA0 EMDMA0 DMA Done	Edge
58	EMDMA1_DONE	EMDMA1 EMDMA1 DMA Done	Edge

Table 26-4: ADSP-21560_21561_21564_21568 EMDMA Trigger List Receivers

Trigger ID	Name	Description	Sensitivity
None			

DMA Addressing

Besides the traditional internal-to-external addressing type, the DMA module also supports internal-to-internal transfers. This is accomplished by indexing all external parameter registers with internal addresses. The DMA controller recognizes the transfer by addresses and not by an additional control bit setting.

All the DMA addresses given by EMDMA are word-aligned byte addresses. The programming for the index registers is provided in the *Register Descriptions* section.

DMA Burst Transfers

DMA supports burst transfers only when the modifier is 1. For any other modifier values, single accesses are performed.

The burst size is not user-configurable and is chosen by the processor for optimal performance. The maximum burst size is 8 and the minimum is 1.

The EMDMA uses appropriate burst transfer sizes for optimal throughput. For example, if the word count is 15, then 5 bursts are performed with burst sizes of 8+4+1+1+1 transfers.

Transfer Control Block (TCB) Memory Storage

The location of the DMA parameters for the next sequence comes from the chain pointer register that points to the next set of DMA parameters stored in the processor's internal memory. In chained DMA operations, the processor automatically initializes and then begins another DMA transfer when the current DMA transfer is complete. Each new set of parameters is stored in a user-initialized memory buffer or TCB for a chosen peripheral.

Chain Assignment

The structure of a TCB is conceptually the same as that of a traditional linked list. Each TCB has several data values and a pointer to the next TCB. Further, the chain pointer of a TCB may point to itself to continuously rerun the same DMA. The EMDMA reads each word of the TCB and loads it into the corresponding register. The end of the chain (no further TCBs are loaded) is indicated by a TCB with a chain pointer register value of zero.

The address field of the chain pointer registers is only 30 bits wide. If a program writes a symbolic address to bit 30 of the chain pointer there may be a conflict with the `EMDMA_CHNPTR.PCI` bit. Programs should clear the upper bits of the address then AND the `EMDMA_CHNPTR.PCI` bit separately, if needed.

Starting Chain Loading

A DMA sequence is defined as the sum of the DMA transfer from when the parameter registers initialize to when the count register decrements to zero. The EMDMA module has a chaining enable bit (`EMDMA_CTL.CHEN`).

To start the chain, write the internal (channel 0) index address of the first TCB to the chain pointer register (`EMDMA_CHNPTR`). When chaining is enabled, DMA transfers are initiated by writing a memory address to the chain pointer register. This is also an easy way to start a single DMA sequence, with no subsequent chained DMAs.

During TCB chain loading, the EMDMA loads the DMA channel parameter registers with values retrieved from system memory.

The address in the chain pointer register points to the highest address of the TCB. This contains the internal (channel 0) index parameter. This means that if a program declares an array to hold the TCB, the chain pointer register should point to the last location of the array and not to the first TCB location.

Buffered Chain Loading Register

The chain pointer register (`EMDMA_CHNPTR`) is buffered. Before the chain loading starts, the buffer is copied into the chain pointer register and is decremented after each register is loaded.

The chain pointer register can be loaded with a new address at any time during the DMA sequence (`EMDMA_CTL.CHEN = 1`). This allows a DMA channel to have chaining status deactivated (chain pointer register = 0x0) until some event occurs that loads the chain pointer register with a non-zero value. Writing all zeros to the address field of the chain pointer register also deactivates chaining for the next TCB.

TCB Storage

The EMDMA supports several types of DMA, resulting in different lengths of TCBs. The TCB size varies from six locations (chained DMA) to 13 locations (delay line DMA). The *EMDMA TCBs* table shows the required TCBs for chained DMA.

In the following tables, TCB refers to the start address of the TCB array.

Table 26-5: EMDMA TCBs for Standard DMA

Address	Register
TCB	EMDMA_CHNPTR
TCB + 0x1	EMDMA_MOD1
TCB + 0x2	EMDMA_INDX1
TCB + 0x3	EMDMA_CNT0
TCB + 0x4	EMDMA_MOD0
TCB + 0x5	EMDMA_INDX0

The order the descriptors are fetched with circular buffering enabled is shown in the *EMDMA TCBs for Standard Circular DMA* table.

Table 26-6: EMDMA TCBs for Standard Circular DMA

Address	Register
TCB	EMDMA_CHNPTR
TCB + 0x1	EMDMA_BUFLen
TCB + 0x2	EMDMA_BASE
TCB + 0x3	EMDMA_MOD1
TCB + 0x4	EMDMA_INDX1
TCB + 0x5	EMDMA_CNT0
TCB + 0x6	EMDMA_MOD0
TCB + 0x7	EMDMA_INDX0

For delay line DMA, TCB loading is split into two sequences to improve overall priority. The first TCB loads the write parameters (EMDMA_INDX0 – EMDMA_BUFLen) and the second loads the read parameters (EMDMA_INDX0 – EMDMA_CHNPTR). This two stage loading is transparent to the application. The order the descriptors are fetched for delay line DMA, as shown in the *EMDMA TCBs for Delay Line DMA* table.

Table 26-7: EMDMA TCBs for Delay Line DMA

Address	Register
Delay Line Read	
TCB	EMDMA_CHNPTR
TCB + 0x1	EMDMA_TPTR
TCB + 0x2	EMDMA_TCNT
TCB + 0x3	EMDMA_MOD1
TCB + 0x4	EMDMA_CNT0

Table 26-7: EMDMA TCBs for Delay Line DMA (Continued)

Address	Register
TCB + 0x5	EMDMA_INDX0
Delay Line Write	
TCB + 0x6	EMDMA_BUFLN
TCB + 0x7	EMDMA_BASE
TCB + 0x8	EMDMA_MOD1
TCB + 0x9	EMDMA_INDX1
TCB + 0xA	EMDMA_CNT0
TCB + 0xB	EMDMA_MOD0
TCB + 0xC	EMDMA_INDX0

The order the descriptors are fetched for scatter/gather DMA with circular buffering disabled is shown in the *EMDMA TCBs for Scatter/Gather DMA* table.

Table 26-8: EMDMA TCBs for Scatter/Gather DMA

Address	Register
TCB	EMDMA_CHNPTR
TCB + 0x1	EMDMA_TPTR
TCB + 0x2	EMDMA_TCNT
TCB + 0x3	EMDMA_MOD1
TCB + 0x4	EMDMA_INDX1
TCB + 0x5	EMDMA_CNT0
TCB + 0x6	EMDMA_MOD0
TCB + 0x7	EMDMA_INDX0

The order the descriptors are fetched for scatter/gather DMA with circular buffering enabled is shown in the *EMDMA TCBs for Circular Scatter/Gather DMA* table.

Table 26-9: EMDMA TCBs for Circular Scatter/Gather DMA

Address	Register
TCB	EMDMA_CHNPTR
TCB + 0x1	EMDMA_BUFLN
TCB + 0x2	EMDMA_BASE
TCB + 0x3	EMDMA_TPTR
TCB + 0x4	EMDMA_TCNT

Table 26-9: EMDMA TCBs for Circular Scatter/Gather DMA (Continued)

Address	Register
TCB + 0x5	EMDMA_MOD1
TCB + 0x6	EMDMA_INDX1
TCB + 0x7	EMDMA_CNT0
TCB + 0x8	EMDMA_MOD0
TCB + 0x9	EMDMA_INDX0

EMDMA Operating Modes

This section and the *EMDMA_CTL Register Bit to Operating Modes* table show the different DMA modes which can be used. The complete register bit descriptions are in the *External Memory DMA Control Registers (EMDMA_CTL)*.

Table 26-10: EMDMA_CTL Register Bit to Operating Modes

Bit (Name)	Standard	Chained	Scatter/Gather	Delay Line
Control Bits				
0 (EN)	Valid			
1 (TRAN)	Valid			N/A
2 (CHEN)	Valid			
3 (DLEN)	N/A			Valid
4 (CBEN)	Valid			
5 (DFLSH)	Valid			
6	N/A			
7 (WRBEN)	N/A	Valid	(=0)	(=1)
8 (OFCEN)	Valid			N/A
9 (TLEN)	N/A		Valid	N/A
11–10	N/A			
12 (INTDONE0)	Valid			
15–13	N/A			
Status Bits				
17–16 (DFS)	Valid			
19–18	N/A			
20 (DMAS0)	Valid			
21 (CHS)	N/A	Valid	N/A	Valid

Table 26-10: EMDMA_CTL Register Bit to Operating Modes (Continued)

Bit (Name)	Standard	Chained	Scatter/Gather	Delay Line
22 (TLS)	N/A		Valid	N/A
23 (WBS)	N/A			Valid
24 (DMAS1)	Valid			
25 (DIRS)	Valid			
31–26	N/A			

NOTE: Reading additional bit-field information from N/A (Not applicable) bits does not generate a meaningful result.

A program sets up a DMA channel by writing the transfer's parameters to the DMA parameter registers.

NOTE: The `EMDMA_CNT1` parameter register (read only) is a copy of the `EMDMA_CNT0` register. If `EMDMA_CNT0` is written, the `EMDMA_CNT1` register is updated automatically.

Standard DMA

A standard DMA (once it is configured) transfers data from location A to location B. An interrupt can be used to indicate the end of the transfer. To start a new DMA sequence after the current one is finished, a program must first clear the DMA enable bit (`EMDMA_CTL.EN`), write new parameters to the index, modify, and count registers (parameter registers), then set the DMA enable bit to re-enable DMA.

This DMA type resembles the traditional DMA type to initialize the different internal and external parameters (channel0 and channel1) (index, modify and count) registers and configuration of the DMA control registers.

Circular Buffered DMA

Circular buffered DMA resembles the traditional core DAG circular buffered mode by using registers for circular buffering. In this mode, the DMA needs two additional registers (base and length) to support reads and writes to a circular buffer. The *Circular Buffering Write DMA* and *Circular Buffering Read DMA* figures illustrate circular buffered DMA, in contrast with the *Standard Write DMA* figure.

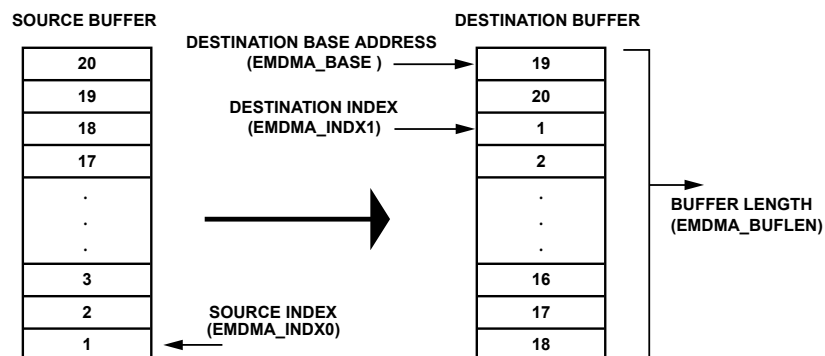


Figure 26-1: Circular Buffering Write DMA

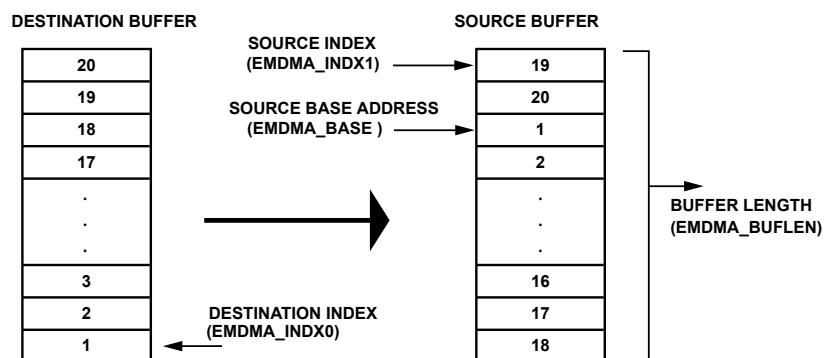


Figure 26-2: Circular Buffering Read DMA

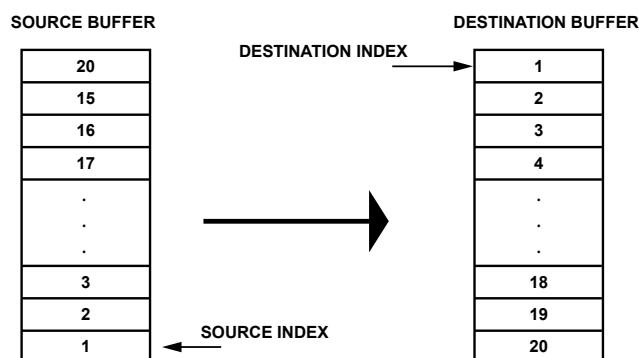


Figure 26-3: Standard Write DMA

NOTE: Circular buffering is available for all operating modes (standard, chained, tap list and delay line DMA).

Chained DMA Mode

Chained DMA sequences are a set of multiple DMA operations, each auto-initializing the next in line. It is used to support automated access by a linked list (repetitive reads and writes to a defined location defined by the individual TCBs). To start a new DMA sequence after the current one is finished, the EMDMA automatically loads new index, modify, and count values pointed to by that channel's [EMDMA_CHNPTR](#) register. Using chaining, programs can set up consecutive DMA operations and each operation can have different attributes.

DMA data transfers can be set up as continuous or periodic. With chained DMA, the attributes of a specific DMA are stored in internal memory and are referred to as a Transfer Control Block or TCB. Extended Memory DMA loads these attributes in chains for execution. This allows for multiple chains that are finite or infinite.

NOTE: When chaining is enabled, polling should not be used to determine DMA status only because the DMA appears inactive if it is sampled while the next TCB is loading. In such cases where chaining is enabled, along with the polling of DMA status bit, polling of the chaining status bit should also occur so that the correct status of the DMA is known. For example, the `EMDMA_CTL.CHS` bit should be polled as well as the `EMDMA_CTL.DMAS0` and `EMDMA_CTL.DMAS1` bits when EMDMA is configured in DMA chaining mode.

Data Direction On-the-Fly

A change of external memory data direction for each individual TCB in a chain sequence is allowed.

The `EMDMA_CHNPTR.CPDR` bit changes the data flow direction. If the `EMDMA_CHNPTR.CPDR = 0`, writes through channel 0 are performed; if `EMDMA_CHNPTR.CPDR = 1`, channel 0 reads are performed. This works similarly to the `EMDMA_CHNPTR.PCI` bit. The `EMDMA_CTL.OFCEN` and `EMDMA_CTL.CHEN` bits must be set (=1) to enable this functionality.

NOTE: If chaining is enabled with the `EMDMA_CTL.OFCEN = 1`, then the `EMDMA_CTL.TRAN` bit has no effect, and the data direction is determined by the `EMDMA_CHNPTR.CPDR` bit.

Write Back Circular Index Pointer

Operating the DMA in circular mode requires some special considerations. The index pointer of start address within the buffer may wrap around for the case when $IC \times IM > EL$ or it does not finish if $IC \times IM < EL$ where:

IC is the value of the `EMDMA_INDX0` register

IM is the value of the `EMDMA_MOD0` register

EL is the value of the `EMDMA_BUFLN` register

In both cases, the TCB start address is no longer valid.

Setting the `EMDMA_CTL.WRBEN` bit writes (at the end of current TCB block) the current index address + 1 into the TCB memory which is the start address for the next TCB. This bit is only selectable for chained DMA mode. For tap list and delay line modes, this bit is hardwired to 0 or 1.

Scatter/Gather DMA

The purpose of scatter/gather DMA is the transfer of data from/to non-contiguous memory blocks.

The scatter/gather DMA type is a fixed block size scatter/gather DMA that relies on tap list entries in system memory to calculate the (Channel 1 DMA Address) to scatter/gather the DMA. When the DMA direction is Channel 1 write (`EMDMA_CTL.TRAN = 1`) it is a scatter DMA. When (`EMDMA_CTL.TRAN = 0`), it is a gather DMA. This mode also supports chained and circular buffer chained DMAs.

See the *Scatter DMA (Writes)*, *Gather DMA (Reads)*, *Circular Buffering Scatter DMA (Writes)*, and *Circular Gather DMA (Reads)* figures.

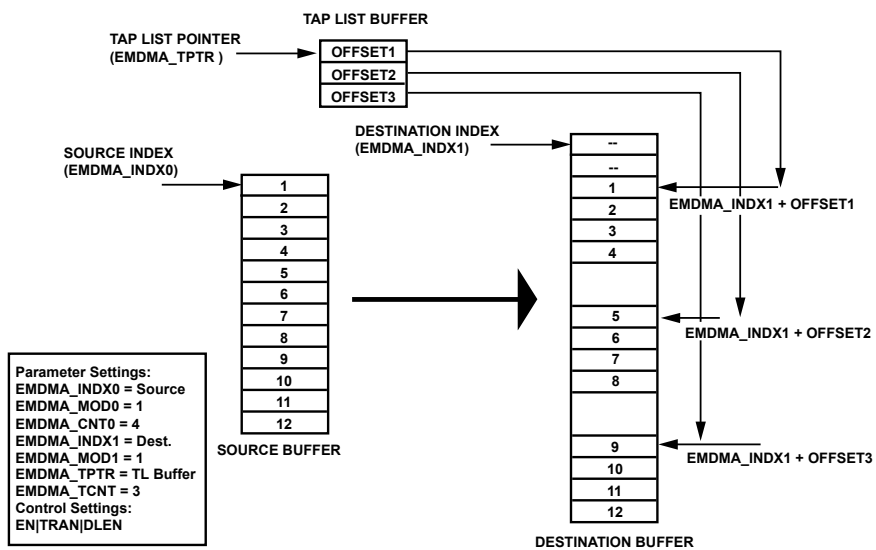


Figure 26-4: Scatter DMA (Writes)

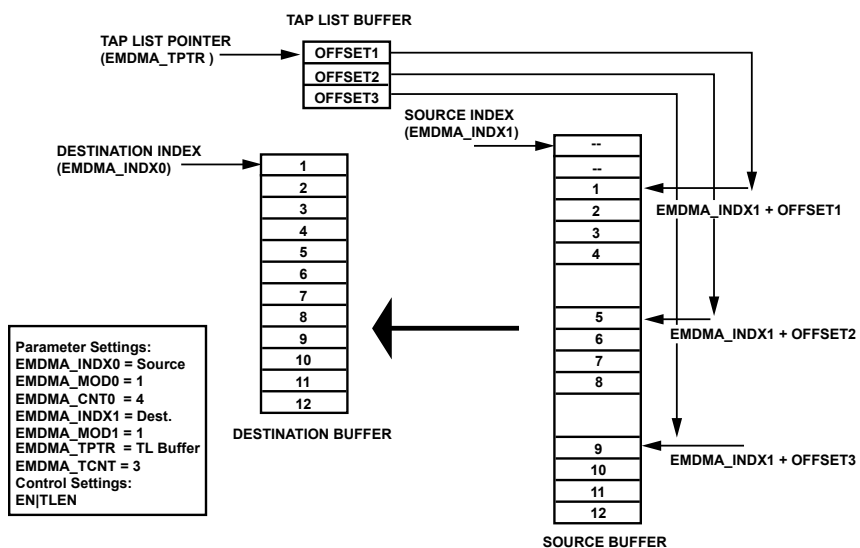


Figure 26-5: Gather DMA (Reads)

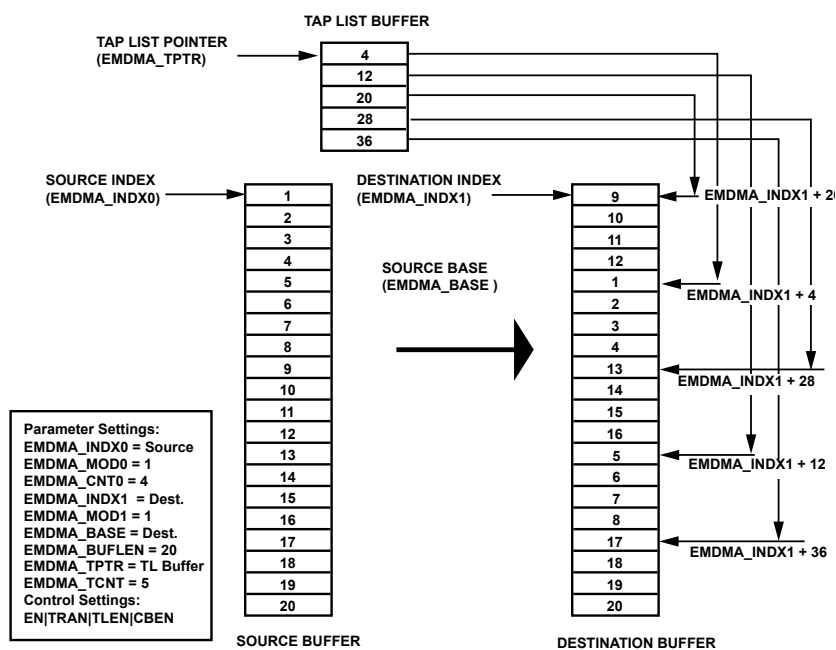


Figure 26-6: Circular Buffering Scatter DMA (Writes)

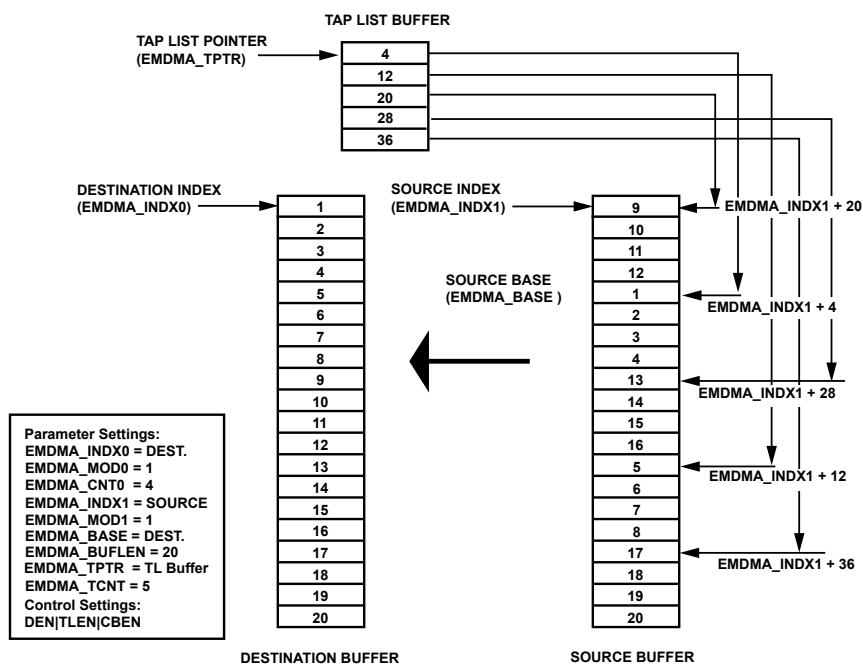


Figure 26-7: Circular Gather DMA (Reads)

For each 32-bit tap read, the Channel 1 read index is shown in the *Read/Write Index Pre-Modify (Scatter/Gather DMA)* table. Note that one tap list entry starts multiple reads.

Table 26-11: Read/Write Index Pre-Modify (Scatter/Gather DMA)

Pre-Modify Address Equation	Result	
	Block Size	Tap
$\text{EMDMA_INDX0} + \text{EMDMA_TPTR}[\text{EMDMA_TCNT}] + (\text{EMDMA_MOD1} \times \text{EMDMA_CNT1})$	N	0
$\text{EMDMA_INDX0} + \text{EMDMA_TPTR}[0] + \text{EMDMA_MOD1} \times 0$		
$\text{EMDMA_INDX0} + \text{EMDMA_TPTR}[0] + \text{EMDMA_MOD1} \times 1$		
$\text{EMDMA_INDX0} + \text{EMDMA_TPTR}[0] + \text{EMDMA_MOD1} \times 2$		
$\text{EMDMA_INDX0} + \text{EMDMA_TPTR}[0] + \text{EMDMA_MOD1} \times N$	N	1
$\text{EMDMA_INDX0} + \text{EMDMA_TPTR}[1] + \text{EMDMA_MOD1} \times 0$		
$\text{EMDMA_INDX0} + \text{EMDMA_TPTR}[1] + \text{EMDMA_MOD1} \times 1$		
$\text{EMDMA_INDX0} + \text{EMDMA_TPTR}[1] + \text{EMDMA_MOD1} \times 2$		
$\text{EMDMA_INDX0} + \text{EMDMA_TPTR}[1] + \text{EMDMA_MOD1} \times N$	N	M
$\text{EMDMA_INDX0} + \text{EMDMA_TPTR}[M] + \text{EMDMA_MOD1} \times 0$		
$\text{EMDMA_INDX0} + \text{EMDMA_TPTR}[M] + \text{EMDMA_MOD1} \times 1$		
$\text{EMDMA_INDX0} + \text{EMDMA_TPTR}[M] + \text{EMDMA_MOD1} \times 2$		
$\text{EMDMA_INDX0} + \text{EMDMA_TPTR}[M] + \text{EMDMA_MOD1} \times N$		

Pre Modified Read/Write Index

For scatter/gather DMA, the tap list modifiers are employed, and the number of taps is determined by the tap list count register ([EMDMA_TCNT](#)). The number of sequential reads (block size) from every tap is determined by the internal count register ([EMDMA_CNT0](#)), and is the same for every tap. The read/write pointer in external index register ([EMDMA_INDX1](#)) serves as the index address for these read/writes.

$TL[N]$ is the first tap list entry in the memory as pointed to by the tap list pointer register ([EMDMA_TPTR](#)). The tap list entries are 27-bit signed integers. For each read/write block, the DMA state machine fetches the offset from the tap list. The offset is added to the [EMDMA_INDX1](#) register value to get the start address of the next block. The Channel 1 addresses are circular buffered if circular buffering is enabled (see the *Circular Buffering Scatter DMA (Writes)* and *Circular Buffering Gather DMA (Reads)* figures in [Scatter/Gather DMA](#)).

Once the [EMDMA_CNT0](#) register for the final tap decrements to zero ([EMDMA_TCNT](#) and [EMDMA_CNT0](#) are zero), then the tap list DMA access is complete, and the DMA completion interrupt is generated (when chaining is enabled the interrupt depends on the [EMDMA_CHNPTR.PCI](#) bit setting).

The write back mode ([EMDMA_CTL.WRBEN](#) bit) is hardwired to zero for tap list based DMA (as the addressing is pre-modify, and therefore the [EMDMA_INDX1](#) value coincides with the TCB value even at the end of the DMA).

Delay Line DMA

Delay line DMA is used to support reads and writes to delay line buffers with limited core interaction. In this sense, delay line DMA is a quantity of integrated writes followed by reads from channel 1 (a delay line DMA access). Delay line DMA is described in the following sections.

NOTE: Delay line DMA can only operate by using chained DMA mode (EMDMA_CTL.CHEN bit set). In order to use delay line DMA for a single DMA sequence, initialize the EMDMA_CHNPTR register to zero in the TCB.

NOTE: Delay line DMA can be used in any system memory.

The delay line DMA operation follows these steps:

1. Load the first half of TCB for writing (seven parameters).
2. DMA writes to the delay line buffer until IC = 0.
3. Update the EI index pointer if circular mode is enabled.
4. Load the second half of TCB for reading (six parameters).
5. DMA tap based reads from the delay line buffer until RC = 0.

Jump to step 1.

Writes to delay line. The number of writes is determined by the EMDMA_CNT0 register. The data is fetched from the EMDMA_INDX0 register and the EMDMA_MOD0 register is used as the internal modifier. The EMDMA_INDX1 register serves as the external index and is incremented by the EMDMA_MOD1 register after each write. These writes are circular buffered if circular buffering is enabled. See the *Write to Delay Line Buffer* figure.

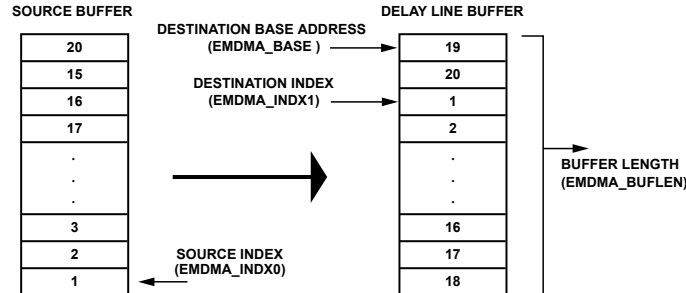


Figure 26-8: Write to Delay Line Buffer

When the writes are complete, (EMDMA_CNT0 = 0) the EMDMA_INDX1 register, which serves as the write pointer of the delay line, is written back (EMDMA_CTL.WRBEN is hardwired to 1) to the TCB location from where it was fetched.

Reads from the delay line. For reads, the tap list (TL) modifiers are used, and the number of reads is determined by the EMDMA_CNT0 register. The write pointer in the EMDMA_INDX1 register serves as the index address for these reads (reads start from where writes end). The EMDMA_INDX1 register, along with tap list modifiers, are used in a pre-modify addressing mode to create the external address for the reads. Therefore, for each read, the DMA controller fetches the external modifier (EMDMA_TCNT register) from the tap list and the reads are circular buffered (if enabled). See the *Read From Delay Line Buffer* figure.

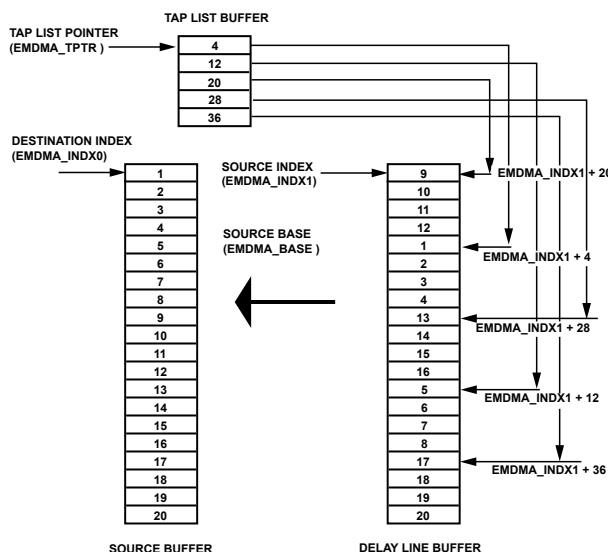


Figure 26-9: Read From Delay Line Buffer

Pre-Modified Read Index

Note that $TL[N]$ is the first tap list entry in memory pointed to by the tap list pointer register (`EMDMA_TPTR`). Tap list entries are 27-bit signed integers. Therefore, for each read-block, the DMA state machine fetches the offset external modifier from the tap list. The reads are circular buffered if circular buffering is enabled.

NOTE: The channel 1 DMA address generation follows pre-modify addressing for reads in delay line DMA and therefore the `EMDMA_INDX1` register values are not updated. Also the `EMDMA_MOD1` register does not have any effect during these delay line reads. Once the read count completes, the `EMDMA_CNT0` register decrements to zero (both `EMDMA_CNT0` and `EMDMA_TCNT` are zero) for the final tap. Finally, the delay line DMA access completes, and the DMA completion interrupt is generated. If chaining is enabled, the interrupt is dependent on the `EMDMA_CHNPTR.PCI` bit setting. The delay line DMA can only be initialized using the TCB. To use the delay line DMA for a single DMA sequence, initialize the `EMDMA_CHNPTR` register to zero in the TCB.

For each 32-bit tap read, the channel 1 read index is shown in the *Read/Write Index Pre-Modify (Scatter/Gather DMA)* table. Note that one tap list entry starts multiple reads.

Table 26-12: Read/Write Index Pre-Modify (Scatter/Gather DMA)

Pre-Modify Address Equation	Result	
	Block Size	Tap
$EMDMA_INDX1 + EMDMA_TPTR[EMDMA_TCNT] + (EMDMA_MOD1 \times EMDMA_CNT1)$	N	0
$EMDMA_INDX1 + EMDMA_TPTR[0] + EMDMA_MOD1 \times 0$		
$EMDMA_INDX1 + EMDMA_TPTR[0] + EMDMA_MOD1 \times 1$		
$EMDMA_INDX1 + EMDMA_TPTR[0] + EMDMA_MOD1 \times 2$		
$EMDMA_INDX1 + EMDMA_TPTR[0] + EMDMA_MOD1 \times N$		

Table 26-12: Read/Write Index Pre-Modify (Scatter/Gather DMA) (Continued)

Pre-Modify Address Equation	Result	
	Block Size	Tap
$\text{EMDMA_INDX1} + \text{EMDMA_TPTR}[\text{EMDMA_TCNT}] + (\text{EMDMA_MOD1} \times \text{EMDMA_CNT1})$	N	1
$\text{EMDMA_INDX1} + \text{EMDMA_TPTR}[1] + \text{EMDMA_MOD1} \times 0$		
$\text{EMDMA_INDX1} + \text{EMDMA_TPTR}[1] + \text{EMDMA_MOD1} \times 1$		
$\text{EMDMA_INDX1} + \text{EMDMA_TPTR}[1] + \text{EMDMA_MOD1} \times 2$		
$\text{EMDMA_INDX1} + \text{EMDMA_TPTR}[1] + \text{EMDMA_MOD1} \times \text{N}$	N	M
$\text{EMDMA_INDX1} + \text{EMDMA_TPTR}[\text{M}] + \text{EMDMA_MOD1} \times 0$		
$\text{EMDMA_INDX1} + \text{EMDMA_TPTR}[\text{M}] + \text{EMDMA_MOD1} \times 1$		
$\text{EMDMA_INDX1} + \text{EMDMA_TPTR}[\text{M}] + \text{EMDMA_MOD1} \times 2$		
$\text{EMDMA_INDX1} + \text{EMDMA_TPTR}[\text{M}] + \text{EMDMA_MOD1} \times \text{N}$		

ADSP-21560_21561_21564_21568 EMDMA Register Descriptions

Extended Memory DMA (EMDMA) contains the following registers.

Table 26-13: ADSP-21560_21561_21564_21568 EMDMA Register List

Name	Description
EMDMA_BASE	External Base Address Register
EMDMA_BUFLN	Circular Buffer Length Register
EMDMA_CHNPTR	Chain Pointer Register
EMDMA_CNT0	Internal Count Register
EMDMA_CNT1	External Count Register
EMDMA_CTL	External Memory DMA Control Register
EMDMA_INDX0	Internal Index Register
EMDMA_INDX1	External Index Register
EMDMA_MOD0	Internal Modifier Register
EMDMA_MOD1	External Modifier Register
EMDMA_TCNT	Delay Line Tap Count Register
EMDMA_TPTR	Tap List Pointer Register

External Base Address Register

The `EMDMA_BASE` register contains the external base address of the delay line buffer. This is used for maintaining circular buffered read/writes to the delay line.

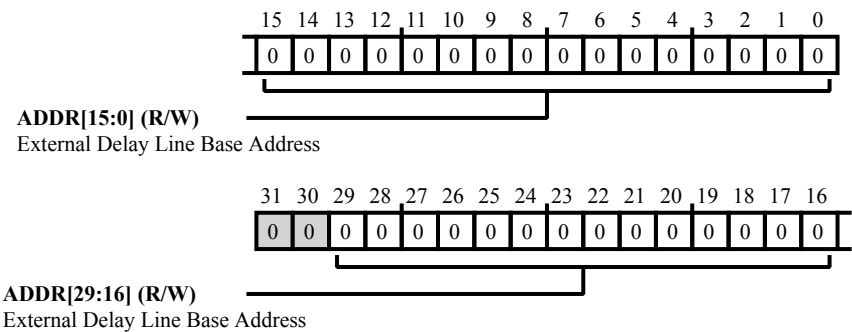


Figure 26-10: EMDMA_BASE Register Diagram

Table 26-14: EMDMA_BASE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:0 (R/W)	ADDR	External Delay Line Base Address. The <code>EMDMA_BASE.ADDR</code> bit field contains the external base address of the delay line buffer.

Circular Buffer Length Register

The `EMDMA_BUFLEN` register holds the circular buffer length for the delay line DMA.

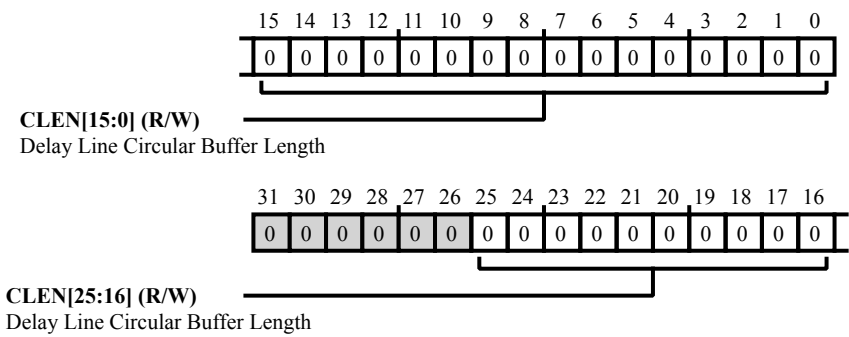


Figure 26-11: EMDMA_BUFLEN Register Diagram

Table 26-15: EMDMA_BUFLEN Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
25:0 (R/W)	CLEN	Delay Line Circular Buffer Length. The <code>EMDMA_BUFLEN.CLEN</code> bit field holds the circular buffer length for the delay line DMA.

Chain Pointer Register

The `EMDMA_CHNPTR` register contains the address of the next descriptor in memory when the `EMDMA_CTL.CHEN` bit =1. This register also has bits to change DMA directions for each descriptor and the PCI bit.

Note that the lower 30-bits of this register are to be written with 30 MSB's of the word-aligned byte addresses of the corresponding next descriptor address.

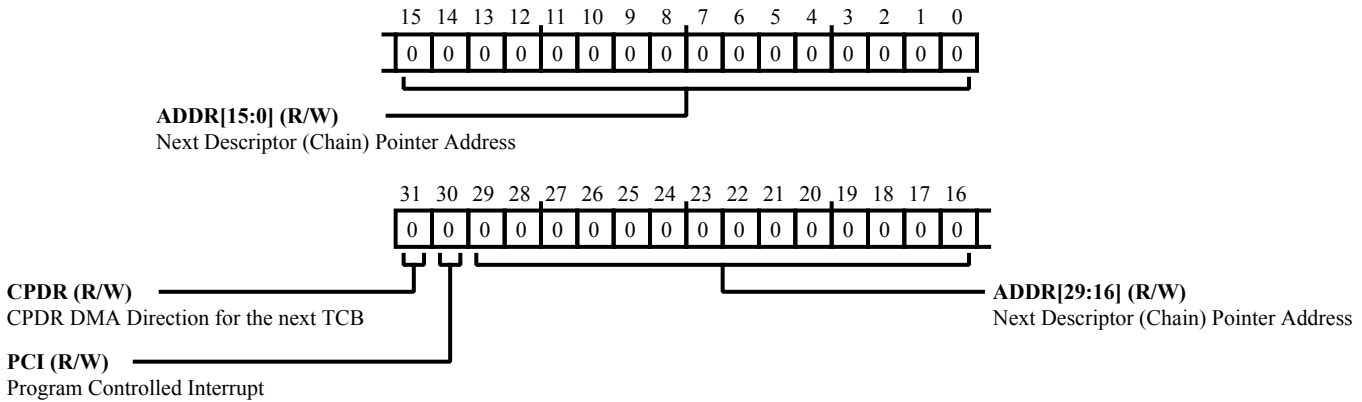


Figure 26-12: EMDMA_CHNPTR Register Diagram

Table 26-16: EMDMA_CHNPTR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	CPDR	CPDR DMA Direction for the next TCB. The <code>EMDMA_CHNPTR.CPDR</code> bit configures whether the DMA is a write to internal memory or a read from internal memory. Note: this setting is applicable only if <code>EMDMA_CTL.OFCEN</code> =1 and is not applicable for delay line DMA.
		0 Write to Channel 0 (Channel 1 reads)
		1 Read from Channel 0 (Channel 1 Writes)
30 (R/W)	PCI	Program Controlled Interrupt. The <code>EMDMA_CHNPTR.PCI</code> bit PCI sets whether an interrupt is generated after the current TCB or if no interrupt is generated. (Only affects DMA if chaining is enabled).
		0 Enable DMA Channel interrupt to occur at the completion of the entire DMA chained transfer
		1 Enable DMA Channel interrupt to occur at the completion of current DMA sequence

Table 26-16: EMDMA_CHNPTR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
29:0 (R/W)	ADDR	Next Descriptor (Chain) Pointer Address. The EMDMA_CHNPTR.ADDR bit field provides the address of the next descriptor in memory.

Internal Count Register

The `EMDMA_CNT0` register contains the number of words to be transferred for channel 0 DMA.

Note: If delay line DMA is enabled then the `EMDMA_CNT0` register serves as the count register for the delay line writes.

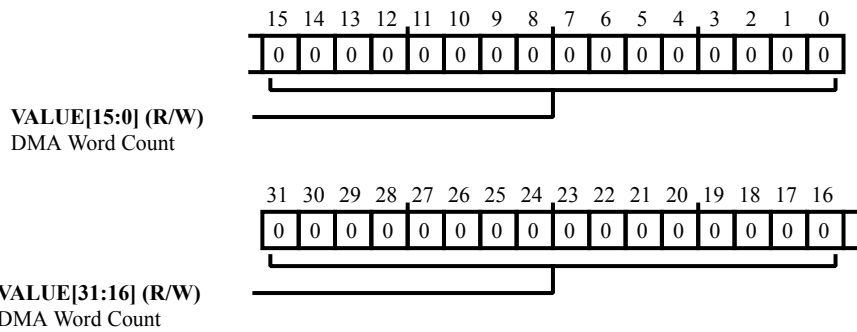


Figure 26-13: EMDMA_CNT0 Register Diagram

Table 26-17: EMDMA_CNT0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	DMA Word Count. The <code>EMDMA_CNT0.VALUE</code> bit field is the DMA word count.

External Count Register

The `EMDMA_CNT1` register contains the number of words to be transferred for channel 1 DMA.

Note: If delay line DMA is enabled then the `EMDMA_CNT1` register serves as the count register for the delay line writes.

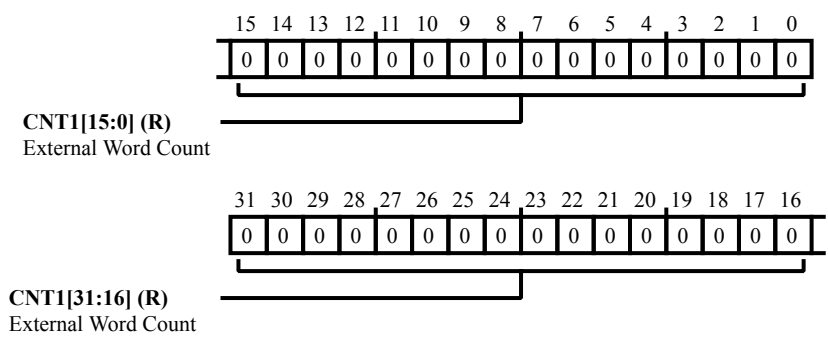


Figure 26-14: EMDMA_CNT1 Register Diagram

Table 26-18: EMDMA_CNT1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT1	External Word Count. The <code>EMDMA_CNT1 . CNT1</code> bit field is the external word count.

External Memory DMA Control Register

The `EMDMA_CTL` register contains bits that enable and configure EMDMA and indicate DMA transfer status.

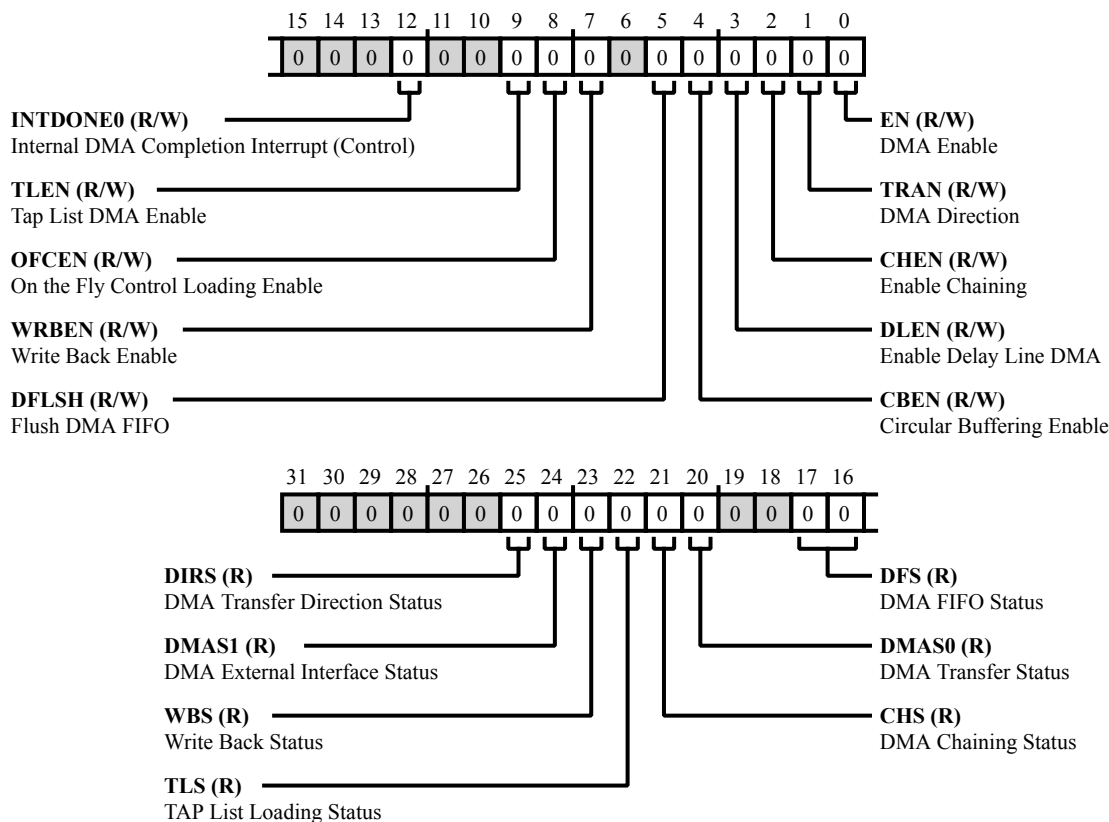


Figure 26-15: EMDMA_CTL Register Diagram

Table 26-19: EMDMA_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
25 (R/NW)	DIRS	DMA Transfer Direction Status.
		The <code>EMDMA_CTL.DIRS</code> bit provides the DMA transfer status direction. This is useful for delay line DMA where the transfer direction changes with the state of the DMA state machine. For standard DMA, the <code>EMDMA_CTL.DIRS</code> bit reflects the state of the <code>EMDMA_CTL.TRAN</code> bit.
		0 DMA direction is Channel 1 Reads
		1 DMA direction is Channel 1 Writes

Table 26-19: EMDMA_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
24 (R/NW)	DMAS1	DMA External Interface Status. The EMDMA_CTL.DMAS1 bit provides the DMA channel 1 transfer status.
		0 Channel 1 DMA does not have any access pending
		1 Channel 1 DMA access are pending
23 (R/NW)	WBS	Write Back Status. The EMDMA_CTL.WBS bit provides the delay line write pointer write back status.
		0 Write pointer write back is not active
		1 Write pointer write back is active
22 (R/NW)	TLS	TAP List Loading Status. The EMDMA_CTL.TLS bit provides the DMA tap list loading status.
		0 TAP list loading is not active
		1 TAP list loading is active
21 (R/NW)	CHS	DMA Chaining Status. The EMDMA_CTL.CHS bit provides the DMA chaining status.
		0 DMA chain loading is not active
		1 DMA chain loading is active
20 (R/NW)	DMAS0	DMA Transfer Status. The EMDMA_CTL.DMAS0 bit provides the DMA channel 0 transfer status.
		0 DMA idle
		1 DMA in progress
17:16 (R/NW)	DFS	DMA FIFO Status. The EMDMA_CTL.DFS bit field provides the DMA FIFO status.
		0 FIFO EMPTY
		1 FIFO Partially Full
		2 Reserved
		3 FIFO Full

Table 26-19: EMDMA_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
12 (R/W)	INTDONE0	Internal DMA Completion Interrupt (Control). The EMDMA_CTL.INTDONE0 bit configures when the DMA complete interrupt is generated. The EMDMA_CTL.INTDONE0 = 1 setting is provided for backward compatibility with older SHARC processors.
		0 Interrupt on access completion (Channel 0 or Channel 1 DMA completion)
		1 Interrupt on Channel 0 DMA completion.
9 (R/W)	TLEN	Tap List DMA Enable. The EMDMA_CTL.TLEN bit enables scatter/gather tap list DMA.
		0 Disables the tap list based scatter/gather DMA
		1 Enables the tap list based scatter/gather DMA
8 (R/W)	OFCEN	On the Fly Control Loading Enable. The control bits in EMDMA_CHNPTR register are used to describe the next TCB behavior if the EMDMA_CTL.OFCEN bit is set. Therefore, the DMA controls can be changed from TCB to TCB. 0 = disables the control bits in the EMDMA_CHNPTR register. 1 = enables the control bits in the EMDMA_CHNPTR register. If chaining is enabled with EMDMA_CTL.OFCEN bit set, the EMDMA_CTL.TRAN bit has no effect, and direction is determined by the EMDMA_CHNPTR.CPDR bit.
7 (R/W)	WRBEN	Write Back Enable. The EMDMA_CTL.WRBEN bit enables write back of the EIEP register after reads and or writes. Write back is automatically enabled for delay line DMA. WRBEN is applicable only if chaining is enabled (EMDMA_CTL.CHEN = 1).
5 (R/W)	DFLSH	Flush DMA FIFO. The EMDMA_CTL.DFLSH bit flushes the DMA FIFO. The buffer is only flushed when this bit is set. It can be set with the enable bit. Setting this bit also clears the EMDMA_CTL.DFS bit.
4 (R/W)	CBEN	Circular Buffering Enable. The EMDMA_CTL.CBEN bit enables circular buffering. Circular buffering can be used with normal DMA as well. If circular buffering is enabled with chaining for normal DMA, then ELEP and EBEP should be part of the TCB.
		0 Disables circular buffering with delay line DMA
		1 Enables circular buffering with delay line DMA

Table 26-19: EMDMA_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
3 (R/W)	DLEN	Enable Delay Line DMA. The EMDMA_CTL.DLEN bit enables delay line DMA. This bit is applicable only when the EMDMA_CTL.CHEN bit =1.
		0 Delay-line DMA disabled
		1 Delay-line DMA enabled
2 (R/W)	CHEN	Enable Chaining. The EMDMA_CTL.CHEN bit enables DMA chaining.
		0 Chaining disabled
		1 Chaining enabled
1 (R/W)	TRAN	DMA Direction. The EMDMA_CTL.TRAN bit determines the DMA data direction. Note: If the delay line DMA is enabled, this bit does not have any effect. For delay line DMA, transfer direction depends on the state of delay line transfers. For internal-internal or external-external DMA, this bit must be set.
		0 Write through Channel 0 (Channel 1 reads)
		1 Read from Channel 0(Channel 1 writes)
0 (R/W)	EN	DMA Enable. The EMDMA_CTL.EN bit enables DMA.
		0 Disable DMA
		1 Enable DMA

Internal Index Register

The `EMDMA_INDXX0` register contains the start address of the buffer for channel 0 DMA.

Note: For delay line DMA the `EMDMA_INDXX0` register serves as the delay line write index which is the start address of the channel 0 DMA buffer for the channel 1 write data.

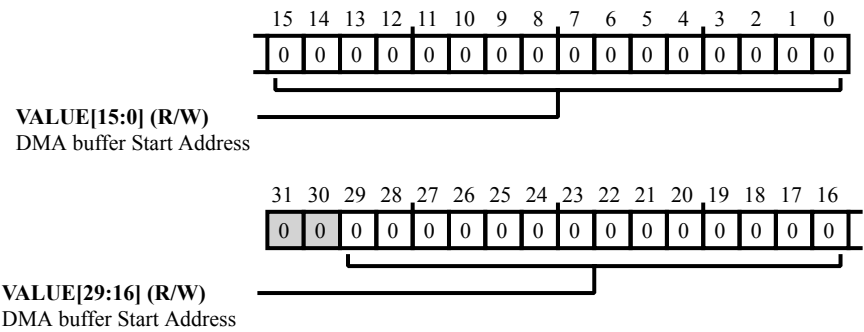


Figure 26-16: EMDMA_INDXX0 Register Diagram

Table 26-20: EMDMA_INDXX0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:0 (R/W)	VALUE	DMA buffer Start Address. The <code>EMDMA_INDXX0.VALUE</code> bit field is written with the 30 MSBs of the word - aligned byte addresses.

External Index Register

The `EMDMA_IND1` register contains the start address of the buffer for channel 1 DMA.

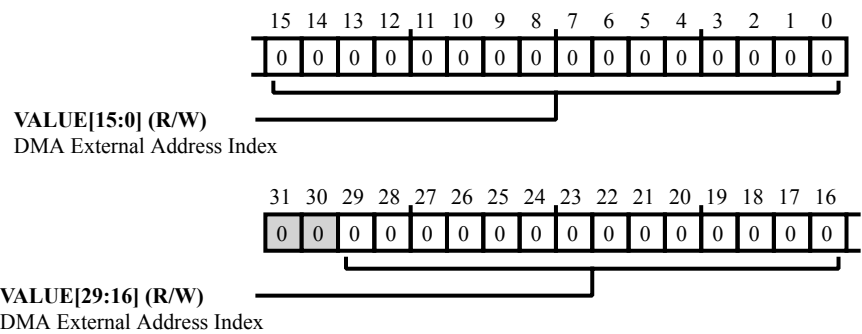


Figure 26-17: EMDMA_IND1 Register Diagram

Table 26-21: EMDMA_IND1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:0 (R/W)	VALUE	DMA External Address Index. The <code>EMDMA_IND1.VALUE</code> bit field is the DMA external address index.

Internal Modifier Register

The `EMDMA_MOD0` register contains the channel 0 DMA address modifier.

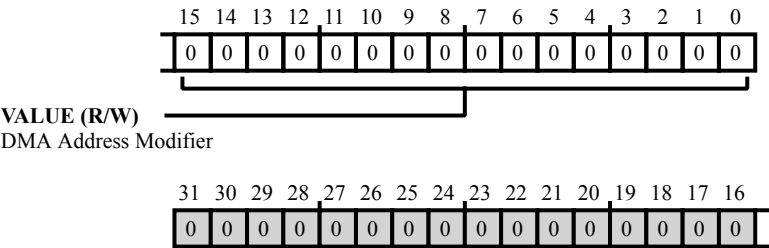


Figure 26-18: EMDMA_MOD0 Register Diagram

Table 26-22: EMDMA_MOD0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	VALUE	DMA Address Modifier. The <code>EMDMA_MOD0.VALUE</code> bit field is the DMA address modifier.

External Modifier Register

The `EMDMA_MOD1` register contains the external (channel 1 DMA) address modifier.

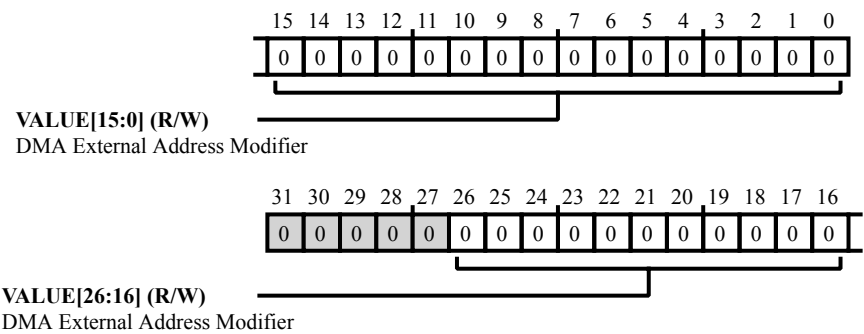


Figure 26-19: EMDMA_MOD1 Register Diagram

Table 26-23: EMDMA_MOD1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
26:0 (R/W)	VALUE	DMA External Address Modifier. The <code>EMDMA_MOD1.VALUE</code> bit field is the DMA external address modifier.

Delay Line Tap Count Register

The `EMDMA_TCNT` register is the tap count register for delay line DMA. This register holds the length of the tap list (the number of taps). The total number of words read from the delay line is equal to the `EMDMA_TCNT` (tap count) multiplied by the `EMDMA_CNT1` (read block size).

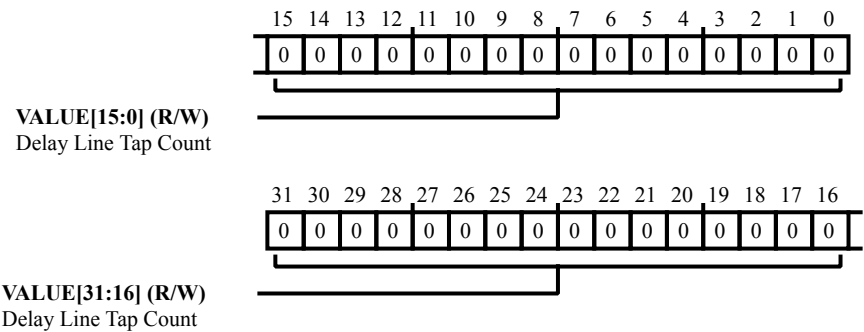


Figure 26-20: EMDMA_TCNT Register Diagram

Table 26-24: EMDMA_TCNT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Delay Line Tap Count. The <code>EMDMA_TCNT.VALUE</code> bit field is the tap count register for delay line DMA.

Tap List Pointer Register

The `EMDMA_TPTR` register holds the address of an array in memory which holds offsets to be used when accessing a delay line in system memory. The offset represents the first address of each read block.

Note: the lower 30-bits of this register are to be written with 30 MSBs of the word-aligned byte address of the array.

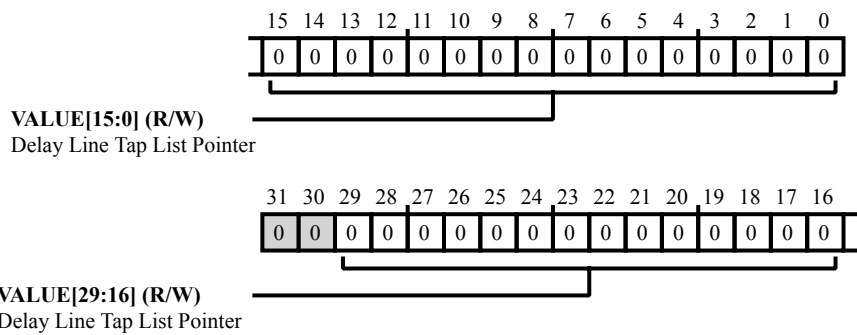


Figure 26-21: EMDMA_TPTR Register Diagram

Table 26-25: EMDMA_TPTR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:0 (R/W)	VALUE	Delay Line Tap List Pointer. The <code>EMDMA_TPTR.VALUE</code> bit field contains the offsets to be used when accessing a delay line in system memory.

27 Cyclic Redundancy Check (CRC)

The CRC peripheral performs the cyclic redundancy check (CRC) of the block of data that is presented to the peripheral. The peripheral provides a means to verify periodically the integrity of the system memory, the contents of memory-mapped registers (MMRs), or communication message objects. It is based on a CRC32 engine that computes the signature of 32-bit data presented to the peripheral.

The dedicated hardware compares the calculated signature of the operation to a pre-loaded expected signature. If the two signatures fail to match, the peripheral generates an error.

The source channel of the memory-to-memory DMA channels can provide data. The CRC optionally forwards data to memory through the destination DMA channel. Alternatively, the peripheral supports data presented by any qualified controller of the CRC peripheral bus.

The CRC peripheral implements a reduced table-look-up algorithm to compute the signature of the data. The CRC uses a programmable 32-bit CRC polynomial to generate the look-up table (LUT) contents automatically.

More CRC peripheral modes allow for initializing large memory sections with a constant value, or for verifying that sections of memory are equal to a constant value.

NOTE: CRC is supported by MDMA0 and MDMA1 channels only.

CRC Features

The CRC peripheral supports a number of key features.

- Memory scan modes for memory verification
- Memory transfer modes for on-the-fly CRC calculations while transferring data from one memory to another
- A programmable 32-bit CRC polynomial with automatic LUT generation
- Data mirroring options

The CRC module also includes the following features.

- CRC checksum computation and comparison modes
- 32-bit programmable CRC polynomial with bit reverse option
- Automatic look-up table (LUT) generation

- Data mirroring options for endian and reflected polynomial cases
- Automatic clear and preset of results
- Fault and error interrupt reporting
- DMA and MMR based operation

Because the CRC module is closely tied to memory-to-memory DMA (MDMA) channel pairs, the use cases include the following features.

- Memory scan mode with CRC compute or compare
- Memory transfer mode with CRC compute or compare
- Memory fill operation with 32-bit data patterns
- Memory verify operation
- MMR write access to FIFO of destination DMA
- MMR read access to FIFO of source DMA
- Profiting from advanced DMA features, like descriptor mode and bandwidth control or monitor

CRC Functional Description

The CRC peripheral supports a number of modes of operation that allow for the initialization and verification of regions of memory. The peripheral supports efficient memory-fill and verification operations on regions of memory with or against a constant value. These modes of operation do not require the CRC engine to calculate a signature. Other modes of operation allow for the calculation of CRC signature and verification for a memory region. The modes allow for on-the-fly CRC calculation when performing memory-to-memory DMA transfers from one memory region to another.

To minimize the need for core accesses, the peripheral interfaces with one or more (depending on processor features) memory-to-memory DMA (MDMA) channels. This connectivity permits flexible configuration, in which data can be written-to or read-from the peripheral using DMA transactions, core transactions, or a combination of both.

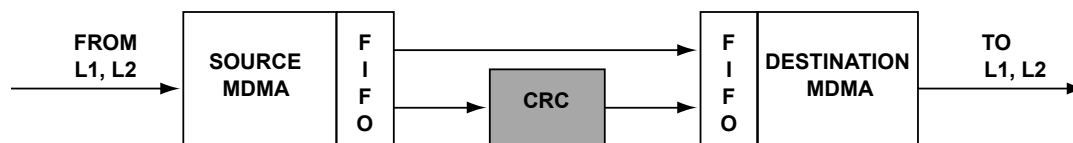


Figure 27-1: Memory Flow

ADSP-21560_21561_21564_21568 CRC Register List

The Cyclic Redundancy Check (CRC) unit includes the data comparison, polynomial operation, and look up table generation features needed for CRC operation. The CRC provides CRC protection as specified by many functional safety requirements. This unit facilitates the system software's ability to periodically check the correctness of the

code/data available in memory. A set of registers govern CRC operations. For more information on CRC functionality, see the CRC register descriptions.

Table 27-1: ADSP-21560_21561_21564_21568 CRC Register List

Name	Description
CRC_COMP	Data Compare Register
CRC_CTL	Control Register
CRC_DCNT	Data Word Count Register
CRC_DCNTCAP	Data Count Capture Register
CRC_DCNTRLD	Data Word Count Reload Register
CRC_DFIFO	Data FIFO Register
CRC_FILLVAL	Fill Value Register
CRC_INEN	Interrupt Enable Register
CRC_INEN_CLR	Interrupt Enable Clear Register
CRC_INEN_SET	Interrupt Enable Set Register
CRC_POLY	Polynomial Register
CRC_RESULT_CUR	CRC Current Result Register
CRC_RESULT_FIN	CRC Final Result Register
CRC_STAT	Status Register

ADSP-21560_21561_21564_21568 CRC Interrupt List

Table 27-2: ADSP-21560_21561_21564_21568 CRC Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
136	CRC0_DCNTEXP	CRC0 Datacount expiration	Level	
137	CRC1_DCNTEXP	CRC1 Datacount expiration	Level	
138	CRC0_ERR	CRC0 Error	Level	
139	CRC1_ERR	CRC1 Error	Level	

CRC Definitions

To make the best use of the CRC, it is useful to understand the following terms.

CRC

Acronym for Cyclic Redundancy Check. An error detection code that can detect changes within a block of data.

CRC Polynomial

The 32-bit polynomial used by the CRC engine to generate the look-up table required for the CRC implementation

LUT

Acronym for the Look-up Table. The look-up table is automatically generated from the supplied 32-bit CRC polynomial.

DMA

Acronym for Direct Memory Access. Used to describe a data transfer that takes place through a DMA channel allowing data distribution around a system without intervention from the core.

MDMA

Acronym for Memory-To-Memory DMA transfer that often requires the use of two DMA channels to transfer data from one memory region to another memory region. One DMA channel is configured as a source channel and the second as a destination channel.

CRC Block Diagram

The *CRC Block Diagram* shows the functional block diagram of the CRC. The following sections describe the blocks.

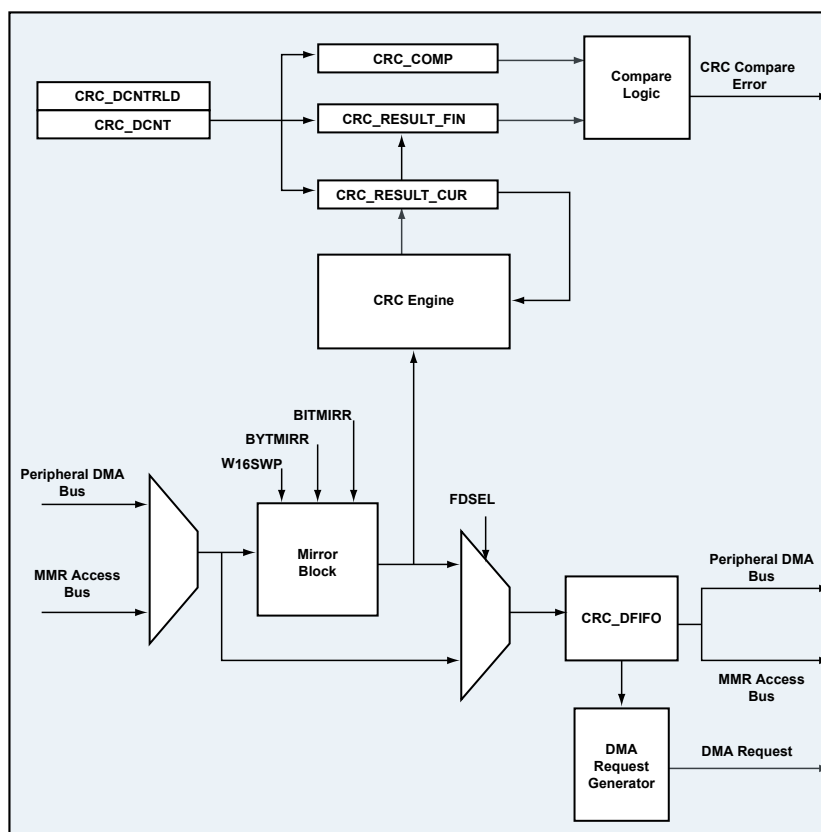


Figure 27-2: CRC Block Diagram

Peripheral DMA Bus

The CRC peripheral provides both an incoming and outgoing datapath to the peripheral DMA bus. The MDMA source channel is interfaced to the incoming datapath providing data to the CRC peripheral. For memory transfer and data fill modes, the CRC uses the MDMA destination channel to either output the data from the CRC FIFO or use the data for the fill operation.

MMR Access Bus

The core uses the MMR access bus to access all the memory-mapped registers of the peripheral for configuration, status, and debug purposes. The core can also use the MMR access bus to feed data to the CRC peripheral or read data from the FIFO of the CRC peripheral. The CRC operation is an alternative to the DMA channel operation to read data from the FIFO.

Data received by MMR writes can transfer to destination DMA. Similarly, data received by source DMA can be output through the MMR interface. Optionally, intermediate results can be made available to the MMR interface.

Mirror Block

The mirror block individually controls bit-reversing of the polynomial, the computation results, and the expected result. Bit mirroring, byte mirroring, word swapping, and any combination of these operations can control endian and the reflection of processed data.

Data FIFO

The CRC data FIFO is a 32-bit-wide 4-entry FIFO. The FIFO is accessible to both the peripheral DMA bus and the MMR access bus. The FIFO status is accessible from the [CRC_STAT](#) register.

DMA Request Generator

The DMA request generator is responsible for granting incoming DMA requests from the source DMA channel and issuing outgoing DMA requests to the destination DMA channel.

CRC Engine

The CRC engine is a 32-bit CRC engine that implements the reduced table look-up scheme. The CRC engine provides support for a user-programmable 32-bit polynomial that the CRC uses to load the look-up table parameters required for the CRC calculation. The CRC engine is a single cycle implementation operating on 32 bits of data per cycle.

Compare Logic

The compare logic takes the final CRC signature and compares it to the expected CRC signature, generating a CRC compare error when the signatures do not match.

CRC Architectural Concepts

A 32-bit polynomial is required before calculation of the CRC signature can occur. The CRC uses the polynomial to generate the contents of an internal look-up table that the reduced table look-up implementation requires. The look-up table is automatically generated when the polynomial is written. It must be initialized prior to any operation that requires the use of the CRC engine.

The mirror block logic can manipulate the data presented to the CRC engine before the CRC uses the data in the calculation of the CRC signature. The data mirror operation is configurable to allow for bit reversing, byte reversing, and 16-bit word swapping operations on the incoming data. For memory transfer compute-and-compare operations, programs can configure the peripheral to output the data in the same form in which it was received. Or, the operation can output the mirrored data in the same manner that it is presented to the CRC engine.

While the CRC peripheral is in operation, the status of the FIFO is continually updated and reflected in the [CRC_STAT](#) register. The FIFO status is required for core-based accesses to the CRC peripheral. The status indicates when:

- The CRC peripheral can receive data

- Data is available for reading from the FIFO
- The result of the `CRC_RESULT_CUR` register has been updated

The status of the `CRC_RESULT_CUR` register indicates that the current CRC calculation has completed and the result is available.

Look-up Table

The look-up table consists of a set of sixteen 32-bit registers that hardware populates automatically when a write access takes place to the `CRC_POLY` register. 16 clock cycles are required to generate all 16 look-up table entries. The status of the process for generating the look-up table is reflected in `CRC_STAT.LUTDONE` allowing for software to poll on the completion of the event or for generation of an interrupt.

NOTE: Hardware must populate the look-up table before any operation using the CRC peripheral can take place, even if the operation does not use the CRC engine. The peripheral does not issue any data requests until the table generation process is complete. In addition, the `CRC_STAT.IBR` field, that indicates the input buffer status as required for core-based transfers, is only valid upon completion of the process for generating the look-up table.

Data Mirroring

The data mirror block can be configured to manipulate the incoming data before the data passes to the CRC engine and, optionally, to the FIFO. This configuration allows the peripheral to handle various forms of endianness and to function with reflected polynomials.

There are three configuration bits that control the data mirroring process: `CRC_CTL.BITMIRR`, `CRC_CTL.BYTMIRR`, and `CRC_CTL.W16SWP`. The *Data Mirroring Options* table details how these options affect the incoming data and the output generated by the mirror block.

Table 27-3: Data Mirroring Options

W16SWP	BYTMIRR	BITMIRR	Output Data
0	0	0	Dout[31:0] = Din[31:0]
0	0	1	Dout[31:0] = Din[24:31],Din[16:23],Din[8:15],Din[0:7]
0	1	0	Dout[31:0] = Din[7:0],Din[15:8],Din[23:16],Din[31:24]
0	1	1	Dout[31:0] = Din[0:7],Din[8:15],Din[16:23],Din[24:31]
1	0	0	Dout[31:0] = Din[15:0], D[31:16]
1	0	1	Dout[31:0] = Din[8:15],Din[0:7], Din[24:31],Din[16:23]
1	1	0	Dout[31:0] = Din[23:16],Din[31:24], Din[7:0],Din[15:8]
1	1	1	Dout[31:0] = Din[16:23],Din[24:31], Din[0:7],Din[8:15]

When the CRC is configured to operate in the memory transfer compute-and-compare mode, the bit-reversed output data can be written to the FIFO. This feature is controlled through the `CRC_CTL.FDSEL` field.

In addition to providing bit swapping and mirror options to the incoming data, the CRC peripheral also supports bit mirroring on the following registers.

- `CRC_RESULT_CUR` and `CRC_RESULT_FIN`, controlled through the `CRC_CTL.RSLTMIRR` field. When mirroring is enabled, the values to be written to these registers are fully bit-reversed before the write operation occurs.
- `CRC_POLY`, controlled through the `CRC_CTL.POLYMIRR` field. When mirroring is enabled, the 32-bit polynomial is fully bit-reversed before the write operation to the register occurs.
- `CRC_COMP`, controlled through the `CRC_CTL.CMPMIRR` field. When mirroring is enabled, the contents to be loaded to this register are fully bit-reversed before the write operation occurs.

FIFO Status and Data Requests

The CRC peripheral provides indication of the input and output buffer status through `CRC_STAT.IBR` and `CRC_STAT.OBR` respectively. For core-based operations, software must monitor these status fields prior to writing to or reading from the CRC FIFO. No write to the CRC FIFO can occur while `CRC_STAT.IBR` indicates that the buffer is not ready to accept data. Similarly, the CRC FIFO cannot be read until `CRC_STAT.OBR` indicates that data is available.

The memory scan modes of operation only require the monitoring of the input buffer status. The memory transfer, compute-and-compare mode uses both input and output buffer status. If the current result of the CRC computation is required, then software must verify that the current operation has completed and that the intermediate result is ready. The `CRC_STAT.IRR` indicates the status.

NOTE: The memory transfer fill mode of operation requires the use of a DMA channel. The CRC does not support core reads from the CRC FIFO for this mode of operation.

Memory transfer, compute-and-compare mode uses burst transactions to make the most efficient use of the available resources. In this mode, when the FIFO is initially empty and the peripheral is enabled, the `CRC_STAT.IBR` bit indicates that the CRC is ready to accept data. The peripheral generates data requests to the source DMA channel (if the CRC uses DMA). While the number of words remaining in the `CRC_DCNT` register is greater than the FIFO depth, the peripheral issues data requests or accepts incoming data in bursts. The peripheral continues until the CRC FIFO becomes full.

Once full, the `CRC_STAT.IBR` and `CRC_STAT.OBR` bits are updated, and then the CRC issues outgoing data requests. Only when the FIFO is empty can the peripheral accept further incoming data, and the `CRC_STAT.IBR` and `CRC_STAT.OBR` bits are updated once again.

Once `CRC_DCNT` is decremented such that the number of words waiting for processing is less than the number required to fill the FIFO, the burst mode of operation is disabled. Incoming data is accepted when the FIFO is not full. Outgoing data is available when the FIFO is not empty. Therefore, there are no restrictions requiring the word count to be a multiple of the FIFO depth.

All other CRC modes of operation indicate that incoming data can be accepted when the FIFO is not full. Outgoing data is available when the FIFO is not empty.

The `CRC_CTL.OBRSTALL` and `CRC_CTL.IRRSTALL` bit configurations also influence how the CRC generates data requests and status bits. The following list describes the bits.

- The `CRC_CTL.OBRSTALL` bit can be configured such that the CRC peripheral stalls as soon as there is output data available in the FIFO. Use this mode of operation only in memory transfer, compute-and-compare mode. This mode results in the processing of one 32-bit word at a time. The peripheral does not request or accept incoming data until the current value being processed is read from the peripheral.
- The `CRC_CTL.IRRSTALL` bit can be configured so that the CRC peripheral stalls all further incoming data requests until the `CRC_RESULT_CUR` register is read after being updated. Use this mode of operation for CRC signature generation. It is not applicable to memory transfer data-fill mode or memory scan data-verify mode of operation.

CRC Operating Modes

The following sections describe the various operating modes of the CRC interface.

Data Transfer Modes

The CRC peripheral supports two main categories of operation involving data transfers:

- Memory scan mode
- Memory transfer mode

Memory scan modes are read-only operations that allow the contents of memory to be read into the peripheral and verified for correctness. There are two forms of memory scan mode:

- CRC compute-and-compare performs a CRC calculation on data presented to the peripheral and compares the CRC result with a pre-determined and pre-loaded result. An error is generated when the results differ.
- Data verify compares each 32-bit data word presented to the CRC peripheral to a pre-loaded 32-bit value and generates an error when the data differs.

Both modes of operation require at the most, a single DMA channel to read the data from memory into the peripheral. No data is forwarded to the data output or destination DMA. The CRC can also use core-driven transfers for either of these modes of operation.

The memory transfer modes involve memory write or memory read-and-write operations allowing for memory to be initialized or transferred from one region of memory to another. There are two forms of memory transfer mode:

- CRC compute-and-compare performs a full data transfer from one memory region to another memory region. The CRC generates a signature on the data presented and compares it with a pre-determined and pre-loaded result. An error is generated when the results differ.
- Data fill initializes a region of memory with a pre-loaded 32-bit constant value.

The CRC compute-and-compare mode of operation requires both incoming and outgoing data channels. The operation occurs either using DMA channels, using core-driven write or read operations to and from the FIFO or using

a combination of both. The data fill mode of operation requires only a memory write DMA destination channel—this mode does not support core driven operations.

Memory Scan Compute-and-Compare Mode

In this mode of operation, the CRC engine of the peripheral is enabled. The mode is configured through the `CRC_CTL.OPMODE` field and the CRC engine performs a 32-bit CRC operation on the incoming data stream.

The length of the data stream is configured through the `CRC_DCNT` register. The accumulated result of the CRC operation is contained in the `CRC_RESULT_CUR` register. As the CRC engine processes each 32-bit word, the `CRC_DCNT` register is decremented and `CRC_RESULT_CUR` is updated.

Once `CRC_DCNT` decrements to zero, the contents of the `CRC_RESULT_CUR` register are copied to `CRC_RESULT_FIN` and `CRC_STAT.DCNTEXP` is updated accordingly. The CRC uses the `CRC_COMP` register to store the expected result of the operation. After the CRC calculation, `CRC_COMP` is compared with `CRC_RESULT_FIN` and `CRC_STAT.CMPERR` is updated to reflect the status of the compare operation. `CRC_STAT.CMPERR` must be cleared before the next CRC operation is performed.

The CRC peripheral also contains the `CRC_DCNTRLRD` register. The CRC uses this register to reload `CRC_DCNT` upon completion of the CRC operation in preparation for the next transfer.

The initial seed of the CRC computation can be configured through `CRC_CTL.AUTOCLRZ` and `CRC_CTL.AUTOCLRF`. This configuration provides a way to reset `CRC_RESULT_CUR` to 0x00000000, 0xFFFFFFFF or to leave the current register contents untouched for the next operation.

The peripheral can be configured to allow for the compare error and data expiration events to generate an interrupt.

Memory Scan Data Verify

In this mode of operation, the CRC engine of the peripheral is not required. The mode is enabled through the `CRC_CTL.OPMODE` field. Each 32-bit word of the data stream is compared with a constant value that is stored in the `CRC_COMP` register. The `CRC_DCNT` register contains the number of words for comparison. The `CRC_DCNT` register is decremented upon receiving a new 32-bit word from the data stream. If the compare operation fails, the `CRC_STAT.CMPERR` bit is updated and the contents of `CRC_DCNT` are captured in the `CRC_DCNTPCAP` register. This information can be used to identify the location in the data stream where the error occurred. Clear the `CRC_STAT.CMPERR` field to reenables capturing of further errors.

Once `CRC_DCNT` decrements to zero, `CRC_STAT.DCNTEXP` is updated accordingly to signal the end of the operation. The peripheral can be configured to allow for the compare error and data expiration events to generate an interrupt.

Memory Transfer Compute-and-Compare Mode

In this mode of operation, the CRC engine of the peripheral is enabled. The mode is configured through the `CRC_CTL.OPMODE` field and the CRC engine performs a 32-bit CRC operation on the incoming data stream.

The length of the data stream is configured through the `CRC_DCNT` register. The accumulated result of the CRC operation is contained in the `CRC_RESULT_CUR` register. As the CRC engine processes each 32-bit word, the `CRC_DCNT` register is decremented and `CRC_RESULT_CUR` is updated.

Once `CRC_DCNT` decrements to zero, the contents of the `CRC_RESULT_CUR` register are copied to `CRC_RESULT_FIN` and `CRC_STAT.DCNTEXP` is updated accordingly. The CRC uses the `CRC_COMP` register to store the expected result of the operation. Upon completion of the CRC calculation, `CRC_COMP` is compared with `CRC_RESULT_FIN` and `CRC_STAT.CMPERR` is updated to reflect the status of the compare operation. Clear `CRC_STAT.CMPERR` before the next CRC operation is performed.

The CRC peripheral also contains `CRC_DCNTRLD` register. The CRC uses this register to reload `CRC_DCNT` upon completion of the CRC operation in preparation for the next transfer.

The initial seed of the CRC computation can be configured through `CRC_CTL.AUTOCLRZ` and `CRC_CTL.AUTOCLRF`. This configuration provides a means to reset `CRC_RESULT_CUR` to 0x00000000, 0xFFFFFFFF or to leave the current register contents untouched for the next operation.

The peripheral can be configured to allow for the compare error and data expiration events to generate an interrupt.

Memory Transfer Data Fill Mode

In this mode of operation, the CRC engine of the peripheral is not required. The mode is enabled through the `CRC_CTL.OPMODE` field. The `CRC_FILLVAL` register is written with a 32-bit value. The CRC uses this value to initialize a block memory through the memory-to-memory DMA destination channel. When the CRC peripheral and the DMA destination channel are enabled, the contents of the `CRC_FILLVAL` register is written to the DMA channel to initialize the memory region. The `CRC_DCNT` register contains the number of words for the write operation.

Once `CRC_DCNT` decrements to zero, `CRC_STAT.DCNTEXP` is updated accordingly to signal the end of the operation. The peripheral can be configured to allow for the data expiration event to generate an interrupt.

CRC Event Control

The CRC peripheral can enable certain CRC status operations to generate an interrupt event to the system event controller. There, a CRC error can be qualified as a system fault.

Interrupt Signals

The `CRC_STAT.CMPERR` status bit can be configured as an interrupt and is signaled through the CRC error interrupt signal. The `CRC_STAT.CMPERR` status field is set whenever the CRC peripheral performs a compare operation that fails. This status can be the result of a failed memory scan data-verify operation that compares the contents of a memory range with a constant 32-bit value. Or, it can be the result of a CRC signature calculated for a memory region that does not match the expected pre-programmed result for a memory-compare operation.

The `CRC_STAT.DCNTEXP` status bit is set when the `CRC_DCNT` register has decremented to zero. The status indicates that the CRC peripheral has now processed all the data requested for the current CRC operation. The CRC can also use this signal to generate an interrupt. The interrupt is signaled on the CRC status interrupt signal.

Both these status bits can be configured to generate an interrupt through the `CRC_INEN` register. The `CRC_INEN` register also has bit set, `CRC_INEN_SET`, and bit clear `CRC_INEN_CLR` equivalent registers that the CRC uses for the enabling and disabling of these interrupt sources.

The `CRC_STAT` register has two write one to clear (W1C) fields for clearing the two interrupt sources.

NOTE: Disabling the CRC peripheral through the `CRC_CTL.BLKEN` bit does not result in the clearing of interrupt sources. Clear the interrupt sources using a W1C operation to the `CRC_STAT` register.

CRC Programming Model

It is important to note the following restrictions when using the CRC peripheral with the DMA channels:

1. When enabling the CRC peripheral and the DMA channels, enable the CRC peripheral prior to enabling the DMA channels.
2. When disabling the CRC peripheral and the DMA channels, disable the DMA channels prior to disabling the CRC peripheral.

CRC Mode Configuration

Describes a number of tasks showing the various operation modes of the CRC peripheral.

- [Look-up Table Generation](#)
- [Core Driven Memory Scan Compute-and-Compare Mode](#)
- [DMA Driven Memory Scan Compute-and-Compare Mode](#)
- [Core Driven Memory Scan Data Verify Mode](#)
- [DMA Driven Memory Scan Data Verify Mode](#)
- [Core Driven Memory Transfer Compute-and-Compare Mode](#)
- [DMA Driven Memory Transfer Compute-and-Compare Mode](#)
- [DMA Driven Memory Transfer Data Fill Mode](#)

Look-up Table Generation

Describes the steps required to initialize the CRC peripheral LUT.

1. Write the 32-bit CRC polynomial of choice to the `CRC_POLY` register.

ADDITIONAL INFORMATION: This operation results in the CRC peripheral starting the LUT initialization process. The `CRC_STAT.LUTDONE` bit is updated to reflect the operation is in progress.

2. Poll the `CRC_STAT.LUTDONE` bit until the status bit indicates that the operation is completed.

The CRC peripheral has completed initialization of all the LUT registers and is now ready for data operations. The `CRC_STAT.LUTDONE` bit remains in the current state until the `CRC_POLY` register is written again, or the peripheral or processor are reset.

Core Driven Memory Scan Compute-and-Compare Mode

Performs CRC signature calculation and verification for a region of memory using core transactions. The CRC peripheral is configured such that it operates in burst mode due to the stalling options configured through disabling the `CRC_CTL` register.

The task assumes the following:

- The polynomial has been loaded and the look-up table is fully initialized
- All CRC interrupts have been serviced (none pending)
- The CRC block is disabled per `CRC_CTL.BLKEN`

1. Initialize the `CRC_DCNT` register.

ADDITIONAL INFORMATION: The value loaded must represent the number of 32-bit words in the memory region for which the software calculates and verifies the signature.

2. Initialize the `CRC_DCNTRL` register.

ADDITIONAL INFORMATION: This value is used to reload the `CRC_DCNT` register upon completion of current CRC operation. If no further operation is needed, then this register can be initialized to zero.

3. Initialize the `CRC_RESULT_CUR` register.

ADDITIONAL INFORMATION: This register can be initialized to provide an initial seed for the CRC operation that is about to take place.

4. Initialize the `CRC_COMP` register.

ADDITIONAL INFORMATION: This register contains the pre-calculated final CRC signature result for the memory region that the software uses in the final compare operation.

5. Initialize the `CRC_INEN` register.

ADDITIONAL INFORMATION: The CRC uses this register to enable the generation of the CRC interrupts for notification of compare errors and block completion. Configure these interrupts. If enabled, ensure that the corresponding interrupt handlers are also configured.

6. Initialize `CRC_CTL` register with the `CRC_CTL.OPMODE` bit set to memory scan compute-and-compare mode and the `CRC_CTL.BLKEN` bit configured to enable the CRC peripheral.

- Disable the `CRC_CTL.OBRSTALL` and `CRC_CTL.IRRSTALL` bit options for this task example.
- Configure all mirroring and bit reversal options.
- Configure CRC auto-clear options.

The CRC peripheral is now enabled and ready for the core or DMA channel to write data.

7. Write memory region data to the CRC peripheral.

- a. While `CRC_STAT.IBR` bit indicates that the input buffer is ready, write the `CRC_DFIFO` register with 32-bit data.

ADDITIONAL INFORMATION: Repeat this step until all data has been written.

8. Poll the `CRC_STAT.DCNTEXP` bit if the interrupt was disabled.

ADDITIONAL INFORMATION: Perform this step only if counter expired interrupt is disabled. Polling ensures that all the data has been processed.

9. Poll the `CRC_STAT.CMPERR` bit if the interrupt was disabled to check for a compare error.

ADDITIONAL INFORMATION: Perform this step only if the compare error interrupt is not enabled.

10. Write to the `CRC_STAT` register to clear both the `CRC_STAT.DCNTEXP` and `CRC_STAT.CMPERR` bits.

ADDITIONAL INFORMATION: If interrupts were enabled, then clear of these status bits within the interrupt handlers for the respective interrupts.

The CRC compute-and-compare operation is now complete. The CRC peripheral is ready to be configured for the next CRC operation.

The integrity check of the memory through the expected CRC signature has completed. The final result is indicated through the `CRC_STAT.CMPERR` bit and the corresponding interrupt when enabled.

Clear any W1C CRC status bits before performing more CRC operations.

DMA Driven Memory Scan Compute-and-Compare Mode

Performs CRC signature calculation and verification for a region of memory using DMA transactions. The CRC peripheral is configured such that it operates in the burst mode of operation due to the stalling options configured through disabling `CRC_CTL`.

The task assumes the following:

- The polynomial has been loaded and the look-up table is fully initialized
- All CRC interrupts have been serviced (none pending)
- The CRC block is disabled per the `CRC_CTL.BLKEN` bit.

1. Initialize the `CRC_DCNT` register.

ADDITIONAL INFORMATION: The value loaded must represent the number of 32-bit words in the memory region for which the software calculates and verifies the signature.

2. Initialize the `CRC_DCNTRLD` register.

ADDITIONAL INFORMATION: This value is used to reload the `CRC_DCNT` register upon completion of current operation. If no further operation is needed, then this register can be initialized to zero.

3. Initialize the `CRC_RESULT_CUR` register.

ADDITIONAL INFORMATION: This register can be initialized to provide an initial seed for the CRC operation that is about to take place.

4. Initialize the `CRC_COMP` register.

ADDITIONAL INFORMATION: This register contains the pre-calculated final CRC signature result for the memory region that the software uses in the final operation.

5. Initialize the `CRC_INEN` register.

ADDITIONAL INFORMATION: The CRC module uses this register to enable the generation of the CRC interrupts for notification of compare errors and block completion. Configure these interrupts, as needed. If enabled, ensure that the corresponding interrupt handlers are also configured.

6. Initialize the `CRC_CTL` register with the `CRC_CTL.OPMODE` bit set to memory scan compute compare mode and the `CRC_CTL.BLKEN` bit configured to enable the CRC peripheral.

- Disable the `CRC_CTL.OBRSTALL` and `CRC_CTL.IRRSTALL` bit options for this task example.
- Configure all mirroring and bit reversal options.
- Configure all CRC auto clear options.

The CRC peripheral is now enabled and ready for the core or DMA channel to write data.

7. Configure and enable the memory-to-memory source DMA channel for memory read STOP mode.

ADDITIONAL INFORMATION: This step starts the data transfer from the memory region and writes the data to the CRC peripheral.

8. Poll the `CRC_STAT.DCNTEXP` bit if the interrupt was disabled.

ADDITIONAL INFORMATION: Perform this step only if the counter expired interrupt is disabled. Polling ensures all the data has been processed.

9. Poll the `CRC_STAT.CMPERR` bit if the interrupt was disabled to check for a compare error.

ADDITIONAL INFORMATION: Perform this step only if the compare error interrupt is not enabled.

10. Write the `CRC_STAT` register to clear both the `CRC_STAT.DCNTEXP` and `CRC_STAT.CMPERR` bits.

ADDITIONAL INFORMATION: If interrupts were enabled, then clear these status bits within the interrupt handlers for the respective interrupts.

The CRC compute-and-compare operation is now complete. The CRC peripheral is ready to be configured for the next CRC operation.

The integrity check of the memory through the expected CRC signature has completed and the final result indicated is through `CRC_STAT.CMPERR` and the corresponding interrupt, when enabled.

Clear any W1C CRC status bits before performing a further CRC operation. Clear any W1C status bits of the memory-to-memory source DMA channel before the next CRC operation.

Core Driven Memory Scan Data Verify Mode

Reads a region of memory using core transactions and performs a compare operation on each 32-bit word against a single pre-loaded 32-bit constant. The compare error interrupt is enabled to capture and log the location of any compare errors.

The task assumes the following:

- The polynomial has been loaded and the look-up table is fully initialized
- All CRC interrupts have been serviced (none pending)
- The CRC block is disabled per `CRC_CTL.BLKEN`

The interrupt service routine for the compare error interrupt reads and stores the contents of `CRC_DCNTCAP` register to a buffer before clearing the compare error interrupt.

1. Initialize the `CRC_DCNT` register.

ADDITIONAL INFORMATION: The value loaded must represent the number of 32-bit words in the memory region for which the software calculates and verifies the signature.

2. Initialize the `CRC_DCNTRLD` register.

ADDITIONAL INFORMATION: This value is used to reload the `CRC_DCNT` register upon completion of current CRC operation. If no further operation is needed, then this register can be initialized to zero.

3. Initialize the `CRC_COMP` register.

ADDITIONAL INFORMATION: This register contains the 32-bit constant that the memory region is expected to be filled with. Each 32 bit of data presented to the peripheral is compared with this value.

4. Initialize the `CRC_INEN` register.

ADDITIONAL INFORMATION: The CRC module uses this register to enable the generation of the CRC interrupts for notification of compare errors and block completion. Configure these interrupts. If enabled, ensure that the corresponding interrupt handlers are also configured.

5. Initialize the `CRC_CTL` register with the `CRC_CTL.OPMODE` bit set to memory scan data verify mode and the `CRC_CTL.BLKEN` bit configured to enable the CRC peripheral.

The CRC peripheral is now enabled and ready for the core or DMA channel to write data.

6. Write memory region data to the CRC peripheral.
 - a. Poll the `CRC_STAT.IBR` bit until input buffer is ready.
 - b. Write the `CRC_DFIFO` register with 32-bit data.

ADDITIONAL INFORMATION: Repeat these two steps until the entire memory region has been written to the CRC peripheral.

7. Poll the `CRC_INEN_SET.DCNTEXP` bit if the interrupt was disabled.

ADDITIONAL INFORMATION: Perform this step only if counter expired interrupt is disabled. Polling ensures all the data has been processed.

8. Check if the buffer used to capture the `CRC_DCNTCAP` register upon a compare error has any new entries.

ADDITIONAL INFORMATION: The values captures in the buffer provide a means to locate where in the memory region the failures occurred.

9. Write to the `CRC_STAT` to clear both the `CRC_INEN_SET.DCNTEXP` and `CRC_INEN.CMPERR` bits.

ADDITIONAL INFORMATION: If interrupts were enabled, the clear these status bits within the interrupt handlers for the respective interrupts.

The CRC memory scan-verify operation is now complete. The CRC peripheral is ready to be configured for the next CRC operation.

The result of the integrity check of the memory with the 32-bit constant is indicated through the `CRC_INEN.CMPERR` bit and the corresponding interrupt, when enabled. Each comparison error is traceable due to the logging of `CRC_DCNTCAP` from within the compare error interrupt handler.

Clear any W1C CRC status bits before performing a further CRC operation.

DMA Driven Memory Scan Data Verify Mode

The memory scan data verify mode reads a region of memory using DMA transactions and performs a compare operation on each 32-bit word against a single pre-loaded 32-bit constant. The compare error interrupt is enabled to capture and log the location of any compare errors.

The task assumes the following:

- The polynomial has been loaded and the look-up table is fully initialized
- All CRC interrupts have been serviced (none pending)
- The CRC block is disabled per the `CRC_CTL.BLKEN` bit

The interrupt service routine for the compare error interrupt reads and stores the contents of the `CRC_DCNTCAP` register to a buffer before clearing the compare error interrupt.

1. Initialize the `CRC_DCNT` register.

ADDITIONAL INFORMATION: The value loaded must represent the number of 32-bit words in the memory region for which the software calculates and verifies the signature.

2. Initialize the `CRC_DCNTRLD` register.

ADDITIONAL INFORMATION: The CRC module uses this register to reload the `CRC_DCNT` register upon completion of current CRC operation. If no further operation is needed, then this register can be initialized to zero.

3. Initialize the `CRC_COMP` register.

ADDITIONAL INFORMATION: This register contains the 32-bit constant that the memory region is expected to be filled with. Each 32 bit of data presented to the peripheral is compared with this value.

4. Initialize the `CRC_INEN` register.

ADDITIONAL INFORMATION: The CRC module uses this register to enable the generation of the CRC interrupts for notification of compare errors and block completion. Configure these interrupts, as needed. If enabled, ensure that the corresponding interrupt handlers are also configured.

5. Initialize the `CRC_CTL` register with the `CRC_CTL.OPMODE` bit set to memory scan data verify mode and `CRC_CTL.BLKEN` configured to enable the CRC peripheral.

The CRC peripheral is now enabled and ready for the core or DMA channel to write the data.

6. Configure and enable the memory-to-memory source DMA channel for memory read STOP mode.

ADDITIONAL INFORMATION: This step starts the data transfer from the memory region and writes the data to the CRC peripheral.

7. Poll the `CRC_STAT.DCNTEXP` bit if the interrupt was disabled.

ADDITIONAL INFORMATION: Perform this step only if counter expired interrupt is disabled. Polling ensures all the data has been processed.

8. Check if the buffer used to capture the `CRC_DCNTCAP` register upon a compare error has any new entries.

ADDITIONAL INFORMATION: The values captures in the buffer provide a means to locate where in the memory region the failures occurred.

9. Write the `CRC_STAT` register to clear both the `CRC_STAT.DCNTEXP` and `CRC_STAT.CMPERR` bits.

ADDITIONAL INFORMATION: If interrupts were enabled, then clear these status bits within the interrupt handlers for the respective interrupts.

The CRC memory scan-verify operation is now complete. The CRC peripheral is ready to be configured for the next CRC operation.

The result of the integrity check of the memory with the 32-bit constant is indicated through the `CRC_STAT.CMPERR` bit and the corresponding interrupt when enabled. Each comparison error is traceable due to the logging of the `CRC_DCNTCAP` register from within the compare error interrupt handler.

Clear any W1C CRC status bits and DMA status bits before performing a further CRC operation.

Core Driven Memory Transfer Compute-and-Compare Mode

The memory transfer compute-and-compare mode performs CRC signature calculation and verification for a region of memory using core transactions while copying the contents to another memory region. The CRC peripheral is configured such that it operates in the burst mode of operation due to the stalling options configured through disabling the `CRC_CTL` register.

The task assumes the following:

- The polynomial has been loaded and the look-up table is fully initialized
- All CRC interrupts have been serviced (none pending)
- The CRC block is disabled per the `CRC_CTL.BLKEN` bit

1. Initialize the `CRC_DCNT` register.

ADDITIONAL INFORMATION: The value loaded must represent the number of 32-bit words in the memory region for which the software calculates and verifies the signature.

2. Initialize the `CRC_DCNTRLD` register.

ADDITIONAL INFORMATION: This value is used to reload the `CRC_DCNT` register upon completion of the current CRC operation. If no further operation is needed, then this register can be initialized to zero.

3. Initialize the `CRC_RESULT_CUR` register.

ADDITIONAL INFORMATION: This register can be initialized to provide an initial seed for the CRC operation that is about to take place.

4. Initialize the `CRC_COMP` register.

ADDITIONAL INFORMATION: This register contains the pre-calculated final CRC signature result for the memory region that the software uses in the final compare operation.

5. Initialize the `CRC_INEN` register.

ADDITIONAL INFORMATION: The CRC module uses this register to enable the generation of the CRC interrupts for notification of compare errors and block completion. Configure these interrupts, as needed. If enabled, ensure that the corresponding interrupt handlers are also configured.

6. Initialize the `CRC_CTL` register with the `CRC_CTL.OPMODE` bit set to memory scan compute-and-compare mode and the `CRC_CTL.BLKEN` bit configured to enable the CRC peripheral.
 - a. Disable the `CRC_CTL.OBRSTALL` bit and the `CRC_CTL.IRRSTALL` bit options for this task example.
 - b. Configure all mirroring and bit reversal options.
 - c. Configure CRC auto clear options

The CRC peripheral is now enabled and ready for the core or DMA channel to write data.

7. Write memory region data to the CRC peripheral and read it back to the new destination.
 - a. While the `CRC_STAT.IBR` bit indicates that the input buffer is ready, write the `CRC_DFIFO` register with 32-bit data.
 - b. While the `CRC_STAT.OBR` bit indicates that the output buffer is ready, read the `CRC_DFIFO` register and store data to new destination.

ADDITIONAL INFORMATION: Repeat these two steps until all required data has been processed through the CRC peripheral and copied to the new destination.

8. Poll the `CRC_STAT.DCNTEXP` bit if the interrupt was disabled.

ADDITIONAL INFORMATION: Perform this step only if the counter expired interrupt is disabled. Polling ensures all the data has been processed.

9. Poll the `CRC_STAT.CMPERR` bit if the interrupt was disabled to check for a compare error.

ADDITIONAL INFORMATION: Perform this step only if the compare error interrupt is not enabled.

10. Write the `CRC_STAT` register to clear both `CRC_STAT.DCNTEXP` and `CRC_STAT.CMPERR` bits.

ADDITIONAL INFORMATION: If interrupts were enabled, then clear these status bits within the interrupt handlers for the respective interrupts.

The CRC compute-and-compare operation is now complete. The CRC peripheral is ready to be configured for the next CRC operation. The memory region has also been copied to its new destination.

The memory region has been copied to a new location and an integrity check of the memory through the expected CRC signature has also completed. The final result is indicated through the `CRC_STAT.CMPERR` bit and the corresponding interrupt when enabled.

Clear any W1C CRC status bits before performing a further CRC operation.

DMA Driven Memory Transfer Compute-and-Compare Mode

The memory transfer compute-and-compare mode performs CRC signature calculation and verification for a region of memory using DMA transactions. The memory region is also copied to another memory region using memory-to-memory DMA transfers. The CRC peripheral is configured such that it operates in burst mode due to the stalling options configured through disabling `CRC_CTL`.

The task assumes the following:

- The polynomial has been loaded and the look-up table is fully initialized
- All CRC interrupts have been serviced (none pending)
- The CRC block is disabled per the `CRC_CTL.BLKEN` register.

1. Initialize the `CRC_DCNT` register.

ADDITIONAL INFORMATION: The value loaded must represent the number of 32-bit words in the memory region for which the software calculates and verifies the signature.

2. Initialize the `CRC_DCNTRLD` register.

ADDITIONAL INFORMATION: This value is used to reload the `CRC_DCNT` register upon completion of current CRC operation. If no further operation is needed, then this register can be initialized to zero.

3. Initialize the `CRC_RESULT_CUR` register.

ADDITIONAL INFORMATION: This register can be initialized to provide an initial seed for the CRC operation that is about to take place.

4. Initialize the `CRC_COMP` register.

ADDITIONAL INFORMATION: This register contains the pre-calculated final CRC signature result for the memory region that the software uses in the final compare operation.

5. Initialize the `CRC_INEN` register.

ADDITIONAL INFORMATION: The CRC module uses this register to enable the generation of the CRC interrupts for notification of compare errors and block completion. Configure these interrupts, as needed. If enabled, ensure that the corresponding interrupt handlers are also configured.

6. Initialize the `CRC_CTL` register with the `CRC_CTL.OPMODE` bit set to memory scan compute compare mode and `CRC_CTL.BLKEN` configured to enable the CRC peripheral.
 - a. Disable the `CRC_CTL.OBRSTALL` and the `CRC_CTL.IRRSTALL` bit options for this task example.
 - b. Configure all mirroring and bit reversal options
 - c. Configure CRC auto clear options

The CRC peripheral is now enabled and ready for the core or DMA channel to write data.

7. Configure and enable the memory-to-memory source DMA channel for memory read STOP mode and destination DMA channel for memory write STOP mode.

ADDITIONAL INFORMATION: This step starts the data transfer from one memory region to another through the memory-to-memory DMA channels and the CRC peripheral.

8. Poll the `CRC_STAT.DCNTEXP` bit if the interrupt was disabled.

ADDITIONAL INFORMATION: Perform this step only if counter expired interrupt is disabled. Polling ensures all the data has been processed.

9. Poll the `CRC_STAT.CMPERR` bit if the interrupt was disabled to check for a compare error.

ADDITIONAL INFORMATION: Perform this step only if the compare error interrupt is not enabled.

10. Write the `CRC_STAT` register to clear both the `CRC_STAT.DCNTEXP` and the `CRC_STAT.CMPERR` bits.

ADDITIONAL INFORMATION: If interrupts were enabled, then clear these status bits within the interrupt handlers for the respective interrupts.

The CRC compute-and-compare operation is now complete. The CRC peripheral is ready to be configured for the next CRC operation. The memory region has also been copied to its new destination.

The integrity check of the memory through the expected CRC signature has completed and the final result is indicated through the `CRC_STAT.CMPERR` bit and the corresponding interrupt when enabled. The memory region has also been copied to its final destination.

Clear any W1C CRC status bits before performing a further CRC operation. Also, clear any W1C status bits of the memory-to-memory source and destination DMA channels before the next CRC operation.

DMA Driven Memory Transfer Data Fill Mode

This mode initializes a region of memory to a constant 32-bit value using DMA transactions.

The task assumes the following:

- The polynomial has been loaded and the look-up table is fully initialized
- All CRC interrupts have been serviced (none pending)
- The CRC block is disabled per the `CRC_CTL.BLKEN` bit

1. Initialize the `CRC_DCNT` register.

ADDITIONAL INFORMATION: The value loaded must represent the number of 32-bit words in the memory region for which the software calculates and verifies the signature.

2. Initialize the `CRC_DCNTRLD` register.

ADDITIONAL INFORMATION: This value is used to reload the `CRC_DCNT` register upon completion of current CRC operation. If no further operation is needed, then this register can be initialized to zero.

3. Initialize the `CRC_FILLVAL` register.

ADDITIONAL INFORMATION: This register contains the 32-bit constant that the CRC module uses to fill the memory region.

4. Initialize the `CRC_INEN` register.

ADDITIONAL INFORMATION: The CRC module uses this register to enable the generation of the CRC interrupts for notification of block completion. Configure these interrupts as required. If enabled, ensure that the corresponding interrupt handlers are also configured.

5. Initialize the `CRC_CTL` register with the `CRC_CTL.OPMODE` bit set to memory transfer fill mode and the `CRC_CTL.BLKEN` bit configured to enable the CRC peripheral.

The CRC peripheral is now enabled and is ready for the DMA channel to write data.

6. Configure and enable the memory-to-memory destination DMA channel for memory write STOP mode.

ADDITIONAL INFORMATION: This step starts the data transfer taking the constant 32-bit value from the CRC peripheral and writing the data to the DMA channel.

7. Poll the `CRC_STAT.DCNTEXP` bit if the interrupt was disabled.

ADDITIONAL INFORMATION: Perform this step only if counter expired interrupt is disabled. Polling ensures that all the data has been processed.

8. Write the `CRC_STAT` register to clear the `CRC_STAT.DCNTEXP` bit.

ADDITIONAL INFORMATION: If interrupts were enabled, then clear this status bit within the interrupt handlers for the respective interrupts.

The CRC memory transfer fill operation is complete, and the CRC peripheral is ready to be configured for the next CRC operation.

The memory region is filled with the constant data and the CRC peripheral is ready to be configured for a new operation.

Clear any W1C CRC status bits and DMA status bits before performing a further CRC operation.

ADSP-21560_21561_21564_21568 CRC Register Descriptions

Cyclic Redundancy Check Unit (CRC) contains the following registers.

Table 27-4: ADSP-21560_21561_21564_21568 CRC Register List

Name	Description
<code>CRC_COMP</code>	Data Compare Register
<code>CRC_CTL</code>	Control Register
<code>CRC_DCNT</code>	Data Word Count Register
<code>CRC_DCNTCAP</code>	Data Count Capture Register
<code>CRC_DCNTRLD</code>	Data Word Count Reload Register
<code>CRC_DFIFO</code>	Data FIFO Register
<code>CRC_FILLVAL</code>	Fill Value Register
<code>CRC_INEN</code>	Interrupt Enable Register
<code>CRC_INEN_CLR</code>	Interrupt Enable Clear Register
<code>CRC_INEN_SET</code>	Interrupt Enable Set Register
<code>CRC_POLY</code>	Polynomial Register
<code>CRC_RESULT_CUR</code>	CRC Current Result Register
<code>CRC_RESULT_FIN</code>	CRC Final Result Register

Table 27-4: ADSP-21560_21561_21564_21568 CRC Register List (Continued)

Name	Description
CRC_STAT	Status Register

Data Compare Register

The `CRC_COMP` register contains the value corresponding to the expected CRC result or signature for the current data stream. At the end of the operation, the content of this register is used to compare against the result produced by the CRC operation. In data verify mode, each incoming data value is compared with the content of this register.

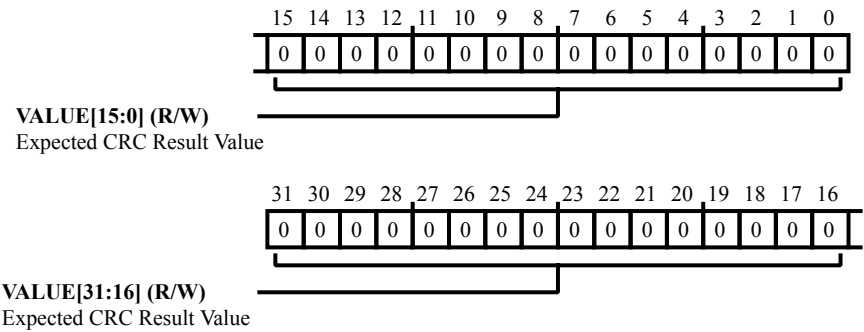


Figure 27-3: CRC_COMP Register Diagram

Table 27-5: CRC_COMP Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Expected CRC Result Value. The <code>CRC_COMP.VALUE</code> bit field contains the value corresponding to the expected CRC result or signature for the current data stream.

Control Register

The `CRC_CTL` register configures the operation modes and settings for the CRC.

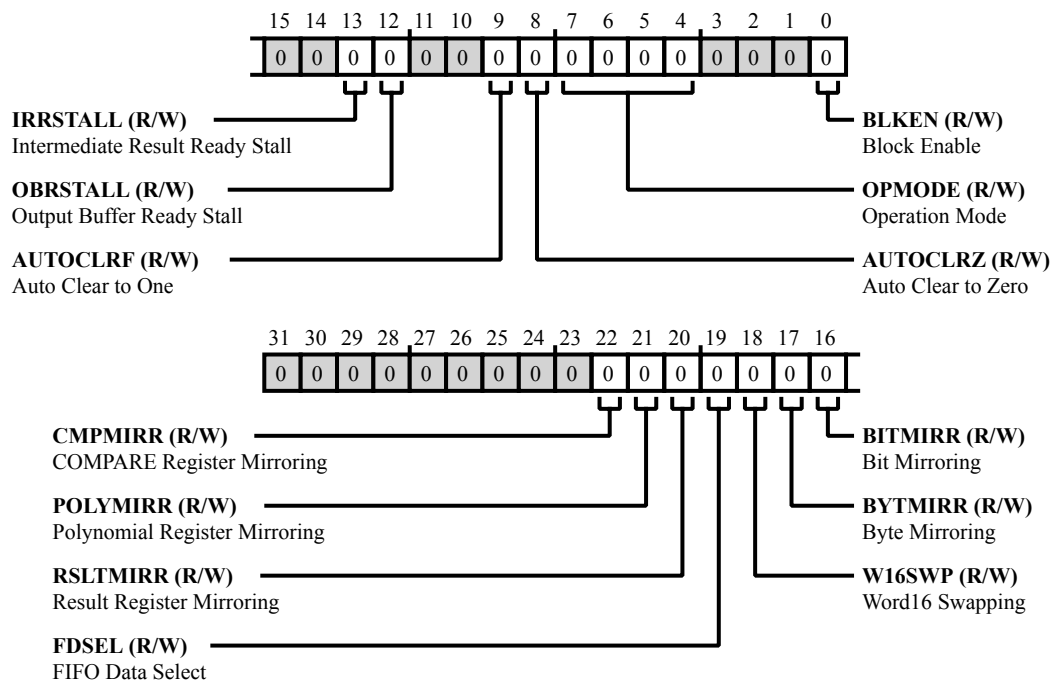


Figure 27-4: CRC_CTL Register Diagram

Table 27-6: CRC_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
22 (R/W)	CMPMIRR	COMPARE Register Mirroring. The <code>CRC_CTL.CMPMIRR</code> bit enables data mirroring for the <code>CRC_COMP</code> compare register. When enabled, the 32-bit value in this register is fully bit mirrored (reversed). The bit-reversed value is used for comparison with the <code>CRC_RESULT_FIN</code> register.
		0 Disable compare mirroring
		1 Enable compare mirroring
21 (R/W)	POLYMIRR	Polynomial Register Mirroring. The <code>CRC_CTL.POLYMIRR</code> bit enables data mirroring for the <code>CRC_POLY</code> polynomial register. When enabled, the 32-bit value in this register is fully bit mirrored (reversed). The bit-reversed value is used for CRC computations.
		0 Disable polynomial mirroring
		1 Enable polynomial mirroring

Table 27-6: CRC_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
20 (R/W)	RSLTMIRR	Result Register Mirroring. The <code>CRC_CTL.RSLTMIRR</code> bit enables data mirroring for the <code>CRC_RESULT_CUR</code> and <code>CRC_RESULT_FIN</code> result registers. When enabled, the 32-bit values in these registers are fully bit mirrored (reversed).
		0 Disable result mirroring
		1 Enable result mirroring
19 (R/W)	FDSEL	FIFO Data Select. The <code>CRC_CTL.FDSEL</code> bit selects whether the CRC writes modified or unmodified data to the FIFO in memory transfer mode. If enabled, the data written is affected by the state of the data mirroring selections (<code>CRC_CTL.BITMIRR</code> , <code>CRC_CTL.BYTMIRR</code> , and <code>CRC_CTL.W16SWP</code>) before being written to the FIFO.
		0 Write unmodified data to FIFO
		1 Write modified data to FIFO
18 (R/W)	W16SWP	Word16 Swapping. The <code>CRC_CTL.W16SWP</code> bit enables the CRC's data mirror block to swap the upper and lower 16-bit words within the 32-bit input data, before further processing.
		0 Disable word16 swapping
		1 Enable word16 swapping
17 (R/W)	BYTMIRR	Byte Mirroring. The <code>CRC_CTL.BYTMIRR</code> bit enables the CRC's data mirror block to mirror the bytes within the 32-bit input data, before further processing.
		0 Disable byte mirroring
		1 Enable byte mirroring
16 (R/W)	BITMIRR	Bit Mirroring. The <code>CRC_CTL.BITMIRR</code> bit enables the CRC's data mirror block to mirror the bits within each byte of the 32-bit input data, before further processing.
		0 Disable bit mirroring
		1 Enable bit mirroring
13 (R/W)	IRRSTALL	Intermediate Result Ready Stall. The <code>CRC_CTL.IRRSTALL</code> bit enables stalling the state machine for input data when there is a valid intermediate result to be read in the <code>CRC_RESULT_CUR</code> register. This feature should be used only in CRC computation modes (for example, <code>CRC_CTL.OPMODE = 1</code> or <code>=3</code>).
		0 Do not stall
		1 Stall on IRR

Table 27-6: CRC_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
12 (R/W)	OBRSTALL	Output Buffer Ready Stall. The CRC_CTL.OBRSTALL bit enables stalling the state machine for input data when there is valid data in the output buffer. This feature should be used only in memory-to-memory transfer modes (for example, CRC_CTL.OPMODE =1).
		0 Do not stall
		1 Stall on OBR
9 (R/W)	AUTOCLRF	Auto Clear to One. The CRC_CTL.AUTOCLRF bit enables auto clear to one when the CRC is in intermediate results ready stall mode (CRC_CTL.IRRSTALL=1) and the CRC data count expires (CRC_DCNT=0). Note that the CRC_CTL.AUTOCLRZ bit must be disabled, or the CRC_CTL.AUTOCLRF bit has no effect.
		0 No auto clear
		1 Auto clear
8 (R/W)	AUTOCLRZ	Auto Clear to Zero. The CRC_CTL.AUTOCLRZ bit enables auto clear to zero when the CRC is in intermediate results ready stall mode (CRC_CTL.IRRSTALL=1) and the CRC data count expires (CRC_DCNT=0). Note that CRC_CTL.AUTOCLRF must be disabled, or the CRC_CTL.AUTOCLRZ has no effect.
		0 No auto clear
		1 Auto clear
7:4 (R/W)	OPMODE	Operation Mode. The CRC_CTL.OPMODE bit field selects the memory transfer or scan mode.
		0 Reserved
		1 CRC compute/compare memory transfer
		2 Data fill memory transfer
		3 CRC compute/compare memory scan
		4 Data verify memory scan
0 (R/W)	BLKEN	Block Enable. The CRC_CTL.BLKEN bit enables and disables the CRC operation.
		0 Disable
		1 Enable

Data Word Count Register

The `CRC_DCNT` register holds the word count that is used for the CRC operation. On transfer of every 32-bit word, the CRC decrements by 1 the content of this register. When the count decrements to zero, this event triggers a CRC compare action, and the `CRC_DCNT` register is automatically loaded from the `CRC_DCNTRL` register for the next CRC operation.

Note that the initial value programmed into the `CRC_DCNT` register may be different from what is programmed in the `CRC_DCNTRL` register.

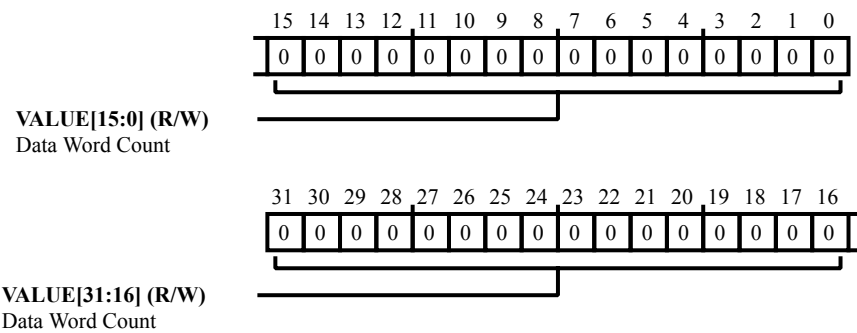


Figure 27-5: CRC_DCNT Register Diagram

Table 27-7: CRC_DCNT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Data Word Count. The <code>CRC_DCNT.VALUE</code> bit field holds the word count that is used for the CRC operation.

Data Count Capture Register

The `CRC_DCNTCAP` register captures the `CRC_DCNT` value when a compare operation fails in data verify mode. This capture can be used to track the position of an error in the data stream. The capture operation is enabled only if the `CRC_STAT.CMPERR` bit indicates no compare error. After an error occurs and the data count is captured, no further errors are logged until the `CRC_STAT.CMPERR` bit is cleared. To obtain the position of an error in the data stream, subtract the `CRC_DCNTCAP` register value from the initial `CRC_DCNT`.

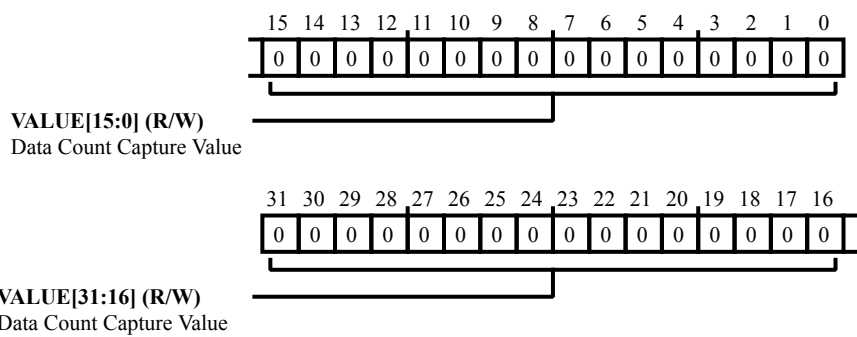


Figure 27-6: `CRC_DCNTCAP` Register Diagram

Table 27-8: `CRC_DCNTCAP` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Data Count Capture Value. The <code>CRC_DCNTCAP.VALUE</code> bit field contains the <code>CRC_DCNT</code> value when a compare operation fails in data verify mode.

Data Word Count Reload Register

The `CRC_DCNTRLD` register holds the value that the CRC automatically loads into `CRC_DCNT` when the `CRC_DCNT` decrements to 0. At startup, the value programmed in `CRC_DCNT` and the `CRC_DCNTRLD` register could be different. So, for the first iteration, the CRC operation happens for the count initially programmed in the `CRC_DCNT` register. While for subsequent CRC operations, the count is taken from the `CRC_DCNTRLD` register.

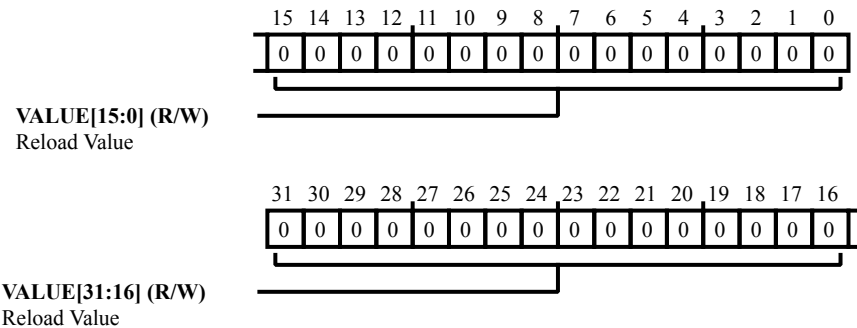


Figure 27-7: `CRC_DCNTRLD` Register Diagram

Table 27-9: `CRC_DCNTRLD` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Reload Value. The <code>CRC_DCNTRLD.VALUE</code> bit field holds the value that automatically loads into <code>CRC_DCNT</code> when the <code>CRC_DCNT</code> decrements to 0.

Data FIFO Register

In memory transfer mode (non-data fill mode), the data from the DMA or processor core buses is written into the `CRC_DFIFO` on each input data grant (DMA grant or core write). Data is read from this FIFO on each output data grant (DMA grant or core read). FIFO status information is available in the `CRC_STAT` register. Whenever, the FIFO has valid data, output data requests are generated.

Note that in non-memory transfer mode and in data fill mode, the input data does not get written into this FIFO. So, this register should not be read in these modes.

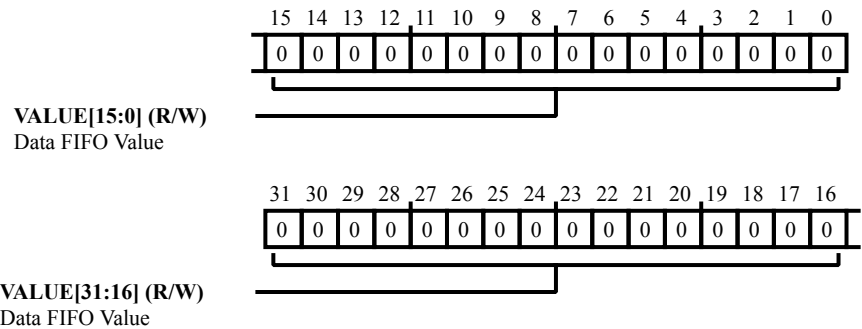


Figure 27-8: CRC_DFIFO Register Diagram

Table 27-10: CRC_DFIFO Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Data FIFO Value. The <code>CRC_DFIFO.VALUE</code> bit field is the data from the DMA or processor core buses.

Fill Value Register

The `CRC_FILLVAL` register holds the value that the CRC uses for the memory fill operation. In data fill mode, the value programmed in this register is used for the memory fill operation.

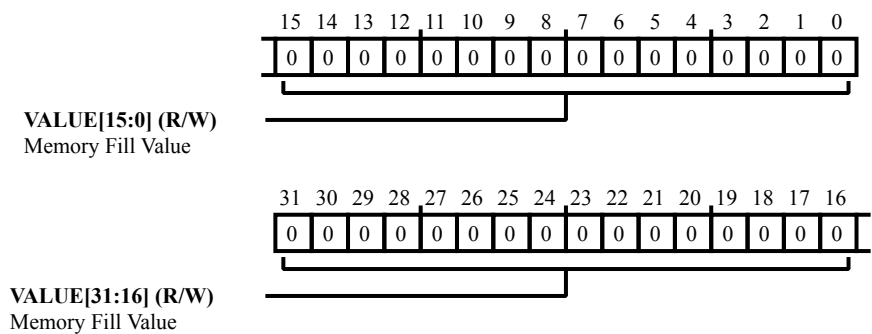


Figure 27-9: CRC_FILLVAL Register Diagram

Table 27-11: CRC_FILLVAL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Memory Fill Value. The <code>CRC_FILLVAL.VALUE</code> bit field holds the value that the CRC uses for the memory fill operation.

Interrupt Enable Register

The `CRC_INEN` register unmask (enables) or mask (disables) interrupt requests generated in the CRC from going to the processor core.

Note that CRC interrupts are not disabled when the CRC is disabled (`CRC_CTL.BLKEN = 0`).

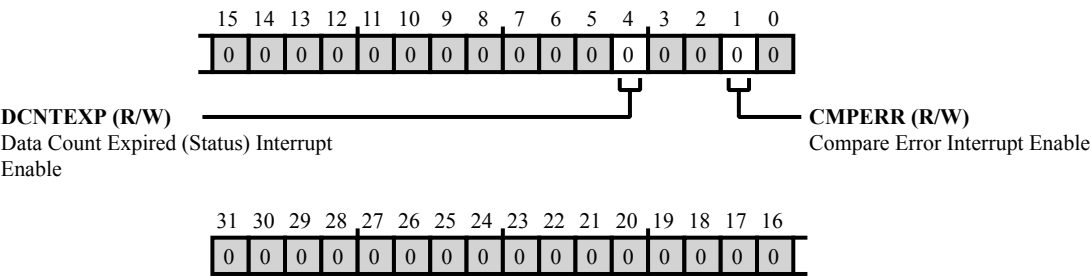


Figure 27-10: CRC_INEN Register Diagram

Table 27-12: CRC_INEN Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
4 (R/W)	DCNTEXP	Data Count Expired (Status) Interrupt Enable. The <code>CRC_INEN.DCNTEXP</code> enables (unmasks) the data count expired (CRC status) interrupt.
		0 Disable (mask) interrupt
		1 Enable (unmask) interrupt
1 (R/W)	CMPERR	Compare Error Interrupt Enable. The <code>CRC_INEN.CMPERR</code> enables (unmasks) the data compare interrupt, which is generated when CRC data comparison fails.
		0 Disable (mask) interrupt
		1 Enable (unmask) interrupt

Interrupt Enable Clear Register

The `CRC_INEN_CLR` register permits clearing individual bits in the `CRC_INEN` register without affecting other bits in the register.

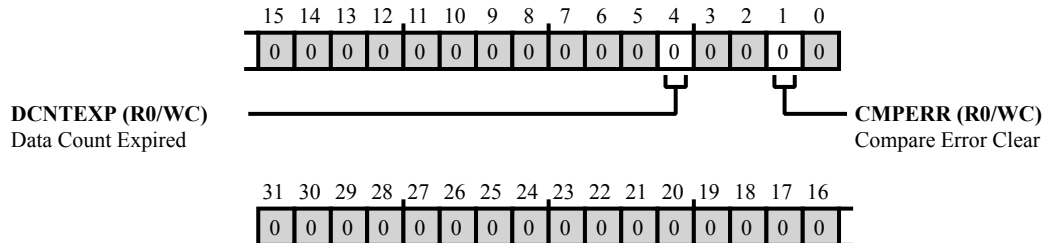


Figure 27-11: `CRC_INEN_CLR` Register Diagram

Table 27-13: `CRC_INEN_CLR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
4 (R0/WC)	DCNTEXP	Data Count Expired. The <code>CRC_INEN_CLR.DCNTEXP</code> bit clears the data count expired (status) interrupt.
		0 No effect
		1 Clear bit
1 (R0/WC)	CMPERR	Compare Error Clear. The <code>CRC_INEN_CLR.CMPERR</code> bit clears the compare error interrupt.
		0 No effect
		1 Clear bit

Interrupt Enable Set Register

The `CRC_INEN_SET` register permits setting individual bits in the `CRC_INEN` register without affecting other bits in the register.

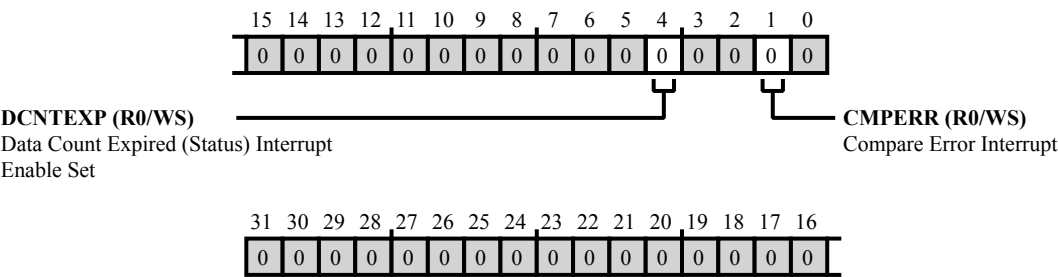


Figure 27-12: `CRC_INEN_SET` Register Diagram

Table 27-14: `CRC_INEN_SET` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
4 (R0/WS)	DCNTEXP	Data Count Expired (Status) Interrupt Enable Set. The <code>CRC_INEN_SET.DCNTEXP</code> bit sets the data count expired (status) interrupt.
		0 No effect
		1 Set bit
1 (R0/WS)	CMPERR	Compare Error Interrupt. The <code>CRC_INEN_SET.CMPERR</code> bit sets the compare error interrupt.
		0 No effect
		1 Set bit

Polynomial Register

The `CRC_POLY` register holds a 32-bit polynomial for CRC operations. Bit 31 corresponds to the coefficient of x^{31} of the CRC polynomial, bit 30 corresponds to the coefficient of x^{30} , and so on through bit 0. A coefficient of x^{32} is assumed to be "1" for any polynomial that is selected. Based on the polynomial in the `CRC_POLY` register, the CRC generates a look-up table (LUT), which is used to compute the CRC of the incoming data stream.

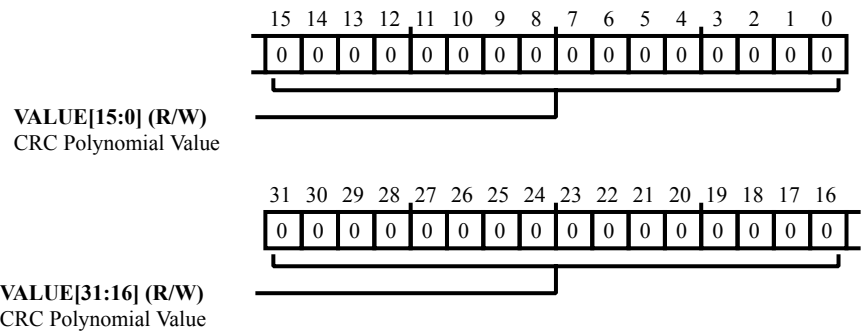


Figure 27-13: CRC_POLY Register Diagram

Table 27-15: CRC_POLY Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	CRC Polynomial Value. The <code>CRC_POLY.VALUE</code> bit field holds the 32-bit polynomial for CRC operations.

CRC Current Result Register

The `CRC_RESULT_CUR` register holds the current or intermediate CRC result. It is updated when new data is written into the CRC. Each time the `CRC_DCNT` expires, the CRC loads the value from this register into the `CRC_RESULT_FIN` register. The `CRC_RESULT_CUR` register may be set to auto clear to zero or auto clear to ones when `CRC_DCNT` expires by configuring the `CRC_CTL.AUTOCLRZ` and `CRC_CTL.AUTOCLR1` bits. Before starting a CRC operation, the `CRC_RESULT_CUR` register should be programmed to the desired value.

Note that this register can be read by the processor core at any time.

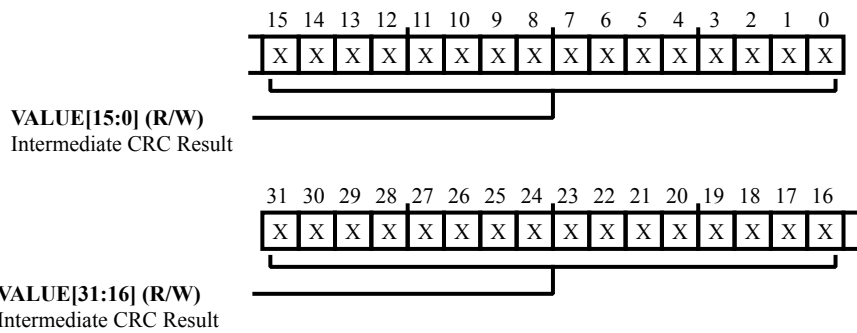


Figure 27-14: `CRC_RESULT_CUR` Register Diagram

Table 27-16: `CRC_RESULT_CUR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Intermediate CRC Result. The <code>CRC_RESULT_CUR.VALUE</code> bit field holds the current or intermediate CRC result.

CRC Final Result Register

The `CRC_RESULT_FIN` register holds the final CRC computed for a data stream. A data stream is a DMA of `CRC_DCNT` number of words into the CRC. When `CRC_DCNT` decrements to zero for each data stream, the CRC loads the `CRC_RESULT_FIN` register with the value from the `CRC_RESULT_CUR` register.

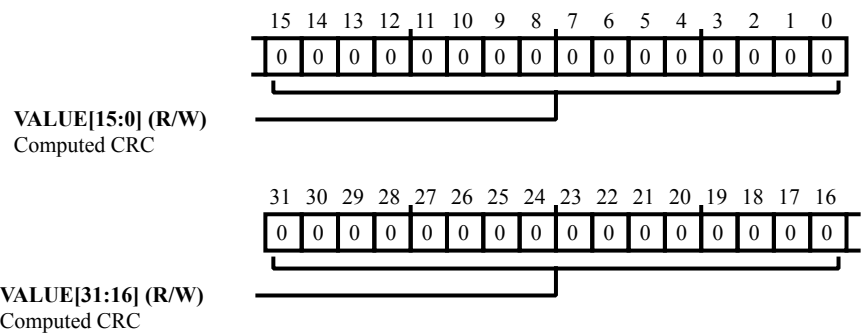


Figure 27-15: CRC_RESULT_FIN Register Diagram

Table 27-17: CRC_RESULT_FIN Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Computed CRC. The <code>CRC_RESULT_FIN.VALUE</code> bit field holds the final CRC computed for a data stream.

Status Register

The `CRC_STAT` register indicates the status for CRC operations and interrupt generation.

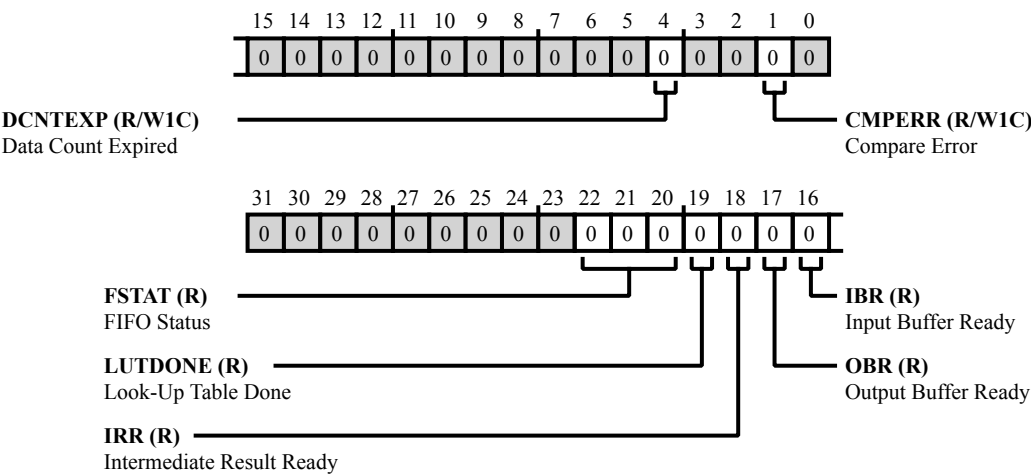


Figure 27-16: CRC_STAT Register Diagram

Table 27-18: CRC_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
22:20 (R/NW)	FSTAT	FIFO Status. The <code>CRC_STAT.FSTAT</code> indicates the current FIFO status. This field is read-only.
		0 FIFO empty
		1 FIFO has 1 data
		2 FIFO has 2 data
		3 FIFO has 3 data
		4 FIFO has 4 data (full)
19 (R/NW)	LUTDONE	Look-Up Table Done. The <code>CRC_STAT.LUTDONE</code> bit indicates that the CRC has generated the look-up table for the current polynomial. This read-only bit is cleared at reset and cleared when the <code>CRC_POLY</code> is written.
		0 No status
		1 LUT generation done

Table 27-18: CRC_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
18 (R/NW)	IRR	Intermediate Result Ready. The CRC_STAT . IRR bit indicates that the CRC has updated the CRC_RESULT_CUR register with intermediate CRC results for the new data written to the CRC. The processor core should read from the CRC_RESULT_CUR register only after detecting CRC_STAT . IRR =1. This read-only bit is cleared by CRC hardware and is valid when the CRC_CTL . IRRSTALL bit is enabled.
		0 No status
		1 Intermediate results ready
17 (R/NW)	OBR	Output Buffer Ready. The CRC_STAT . OBR bit indicates that the CRC has data ready for the processor core to read. The processor core should read from the CRC only after detecting CRC_STAT . OBR =1. This read-only bit is cleared by CRC hardware.
		0 No status
		1 Output buffer ready
16 (R/NW)	IBR	Input Buffer Ready. The CRC_STAT . IBR bit indicates that the CRC is ready to accept a processor core write. The processor core should write to the input register only after detecting that CRC_STAT . IBR =1. This read-only bit is cleared by CRC hardware.
		0 No status
		1 Input buffer ready
4 (R/W1C)	DCNTEXP	Data Count Expired. The CRC_STAT . DCNTEXP bit indicates that the CRC_DCNT has expired. This W1C bit is not automatically cleared when the CRC is disabled (CRC_CTL . BLKEN =0). When the CRC sets this bit on CRC_DCNT expiry, the CRC generates the CRC_INEN . DCNTEXP interrupt.
		0 No status
		1 Data counter expired
1 (R/W1C)	CMPERR	Compare Error. The CRC_STAT . CMPERR bit indicates that a CRC mismatch or data mismatch has been detected. This W1C bit is not automatically cleared when the CRC is disabled (CRC_CTL . BLKEN =0). When the CRC sets this bit on detecting a mismatch, the CRC generates the CRC_INEN . CMPERR interrupt. While this bit is set, the CRC_DCNTCAP register is disabled from capturing the data count values.
		0 No status
		1 Compare error

28 System Security

The requirement to protect content, keys, intellectual property, and other sensitive information have become increasingly prevalent. The processor contains several modules and system elements that contribute to creating a secure operating environment for trusted code to execute.

The modules and system elements that can be used are:

- Boot Kernel
- Secure Core
- System Protection Unit (SPU)
- System Memory Protection Unit (SMPU)
- DEBUG through the Test Access Port Controller (TAPC)
- One-Time Programmable (OTP) memory
- Cryptographic Accelerators (optionally)

NOTE: This product includes security features that can be used to protect embedded non-volatile memory contents and prevent execution of non-authorized code. When security is enabled on this device (either by the ordering party or the subsequent receiving parties), Analog Devices' ability to conduct Failure Analysis on returned devices will be limited. Contact Analog Devices, Inc. for details on the Failure Analysis limitations for this device.

Security Features

The security infrastructure in the system provides the following features:

- Secure operating environment for secure code execution
- Protect sensitive IP from theft by malicious users or competitors
- Protect sensitive data (for example cipher keys)
- Allow debugging while still maintaining security

Security Functional Description

In order to provide secure operating environment in a system, it requires the involvement of multiple elements.

Boot Kernel

The Boot kernel is the root of trust for a secure system. Since the boot kernel is developed by Analog Devices, Inc and is stored in ROM and can't be changed, it can be trusted. The boot kernel validates the authenticity of the application binary image that needs to be booted in. It also handles decrypting the binary image if it's encrypted.

Verifying the authenticity of the application binary asserts that

1. The image is not tampered with or altered
2. The image came from a trusted developer

If the image is encrypted, it ensures confidentiality since the boot image is stored on an external storage device and can be more easily read or stolen than in the part.

Once the boot kernel can verify and optionally decrypt the boot image, it can be loaded into the processor for execution. At this point, the *chain of trust* is continued with the verified application.

Secure Booting

Secure booting is when the boot kernel uses cryptographic algorithms to perform checks on the application binary and to decrypt it. When security is enabled, signature verification and optional decryption on the application binary are both performed by secure boot. See [Security Mode Configuration](#).

Secure Core

In a system with multiple requester and completer resources, not everything is considered secure. There are secure and non-secure peripherals and secure and non-secure segments of memory.

For a system to be considered secure, a secure core that can access and execute instructions (verified application) from secure memory must be configured. Typically, in a single core processor, either the core security is hardwired, or the core can switch between secure and non-secure modes.

System Protection Unit (SPU)

The SPU in the system serves two functions. First, it acts as a gatekeeper, guarding against non-secure accesses to secure resources (peripherals). Second, it is used to define which resources in the system are secure or non-secure requesters and which resources in the system are secure or non-secure completers.

NOTE: Though the SPU can be configured by secure or non-secure requester, the first steps that the verified secure application must perform are:

1. Configure the SPU as a secure completer itself so only other secure requesters can configure it
2. Define and configure the secure requesters and completers using the SPU

This way, once the SPU is secure and other secure requesters and completers are configured, non-secure requesters cannot tamper with the security privileges of secure requesters and completers nor can non-secure resources be changed to a secure resource.

System Memory Protection Unit (SMPU)

Similar to the SPU protecting the MMR address range for peripherals, the SMPU can guard memory ranges or pages. Memory pages can be configured as secure or non-secure. Again, like in the case of the SPU, the SMPU can guard against non-secure transaction attempts to secure memory.

NOTE: A verified application should reside in memory configured as secure. By default the SMPU has all memory configured as secure. When the program needs to update the memory protections using the SMPU, the application and its data should remain in secure memory.

Debug

The typical way of accessing a system is through the debug port via a JTAG or serial wire interface. When these access points are not secure, sensitive IP such as cipher keys can be exposed and code can be changed to disable other security settings. To guard against this type of attack or security hole and still provide debugging capabilities for a developer, a debug unit with security features is used.

When the developer first receives the part, they can define a JTAG/DEBUG key that is programmed into OTP memory. Once security is enabled, the debug unit compares the key sent from the host debugger with the key inside the system. If a match occurs, debug access to secure resources are allowed. There is also a provision to bypass the loading of the JTAG/DEBUG key from the OTP memory.

See [Boot ROM and Booting the Processor](#)

One-Time-Programmable (OTP) Memory

Customer programmable OTP memory is used to safely and securely store sensitive information such as cipher keys.

In a public key algorithm (for example, ECDSA which is used in secure boot), the public key is used to verify the digital signature that accompanies the application binary. The private key is used by the developer on the host development machine to create the digital signature. The public and private key pair is unique, plus the public key needs to be stored in non-volatile, one-time-programmable memory. When the public key can be changed then users can generate their own public/private key pairs and successfully boot in malicious code. This can either re-purpose the part or change security configurations to allow easier access to sensitive information stored elsewhere in the OTP memory.

Cryptographic Accelerators

Cryptographic algorithms are mathematical tools to help provide security. Hardware engines provide some advantages but are not necessarily required. For computationally expensive operations like those used in Elliptic Curve Cryptography, like ECDSA used in secure boot, the operations can be accelerated while the core performs other tasks. Also, it's less likely that the hardware engine can be hacked to change the results.

Security Mode Configuration

Security as a feature does not necessarily be employed by the user. There are no steps to disable security, if security protection is not required. The part does not have security enabled.

To use the security features, perform the following steps:

- Generate the public/private key pair on the host development machine.¹
- Program OTP memory with the public key.²
- Program OTP memory with the decryption key if the application binary needs to be encrypted.²
- Program OTP memory with the debug/JTAG key.²
- Develop the application and sign it, creating the digital signature with the private key.¹
- If confidentiality is required, encrypt the application binary before signing it.
- Set the LOCK bit in OTP memory to enable security. After this, subsequent boots are secure boots.^{2, 3}

¹ Software tools are provided with developments tools to generate keys, sign boot streams, and also perform encryption. Refer to the development tools manuals for details on how to use.

²Refer to the OTP Chapter for programming the OTP and other related information.

³Refer to the Booting Chapter for more information on Secure Booting and other related information.

Status and Error Signals

In a fully functional secure system, non-secure resources should not even attempt to access a secure resource. When this happens, either the code has been altered or replaced with malicious code or the system contains a coding error.

Errors or error events are dependent on the configuration of the SPU and SMPU, the protection units which guard against security violations. In the case of the SMPU, the error can simply be captured, captured and interrupt generated, or the access prevented without capturing any error. It is the developer's responsibility to:

1. Determine when there was an error because of a blocked access.
2. Determine how to handle a blocked access. For example fix the bug (offline) or try to use a different resource (run time).

29 System Protection Unit (SPU)

In a system with multiple system MMR requesters, peripheral configurations can be changed unintentionally, leading to bad data or even system malfunctions. The peripherals are shared resources in the system. The SPU restricts access to certain MMRs, similar to the functionality of a semaphore.

The SPU also protects peripherals based on security settings. It is part of the overall security infrastructure of the processor.

SPU Features

The SPU has the following features:

- Write-protect system MMR from certain system requesters and core requesters.
- Simultaneously lock multiple peripheral configuration registers through a global lock mechanism.
- Write-protect and block access to its own write-protection registers from other system requesters.
- Defined security privileges to peripherals and system resources.
- Security protection to guard secure peripheral MMRs against non-secure accesses.

SPU Functional Description

The following sections provide information on the function of the SPU.

ADSP-21560_21561_21564_21568 SPU Register List

The System Protection Unit (SPU) provides a set of registers that can protect system resources from errant writes. The protection categories are global lock (protects configuration registers) and write protect register lock (protects the write protect register). For more information on SPU functionality, see the SPU register descriptions.

Table 29-1: ADSP-21560_21561_21564_21568 SPU Register List

Name	Description
SPU_CTL	Control Register
SPU_SECURECHK	Secure Check Register

Table 29-1: ADSP-21560_21561_21564_21568 SPU Register List (Continued)

Name	Description
SPU_SECURECTL	Secure Control Register
SPU_SECUREC[n]	Secure Core Registers
SPU_SECUREP[n]	Secure Peripheral Register
SPU_STAT	Status Register
SPU_WP[n]	Write Protect Register n

ADSP-21560_21561_21564_21568 SPU Interrupt List

Table 29-2: ADSP-21560_21561_21564_21568 SPU Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
186	SPU0_INT	SPU0 Interrupt	Level	

Peripheral Register Write Protection

The SPU has a write-protection register ([SPU_WP\[n\]](#)) associated with each peripheral. Each of these write-protection registers has the exact same bits that correspond to a particular SMMR requester (for example, Core 0, MDMA). When the bits are set, the SPU locks the corresponding SMMR requesters from accessing the register address space of the associated peripheral. The bits in the register can be cleared to allow access to the registers of the peripheral again. When the SPU initializes the write-protection register, any writes that are in-progress complete before the SPU blocks subsequent writes.

In the *SPU Write Protect Registers* figure, each write-protect register in the SPU is associated with a particular peripheral.

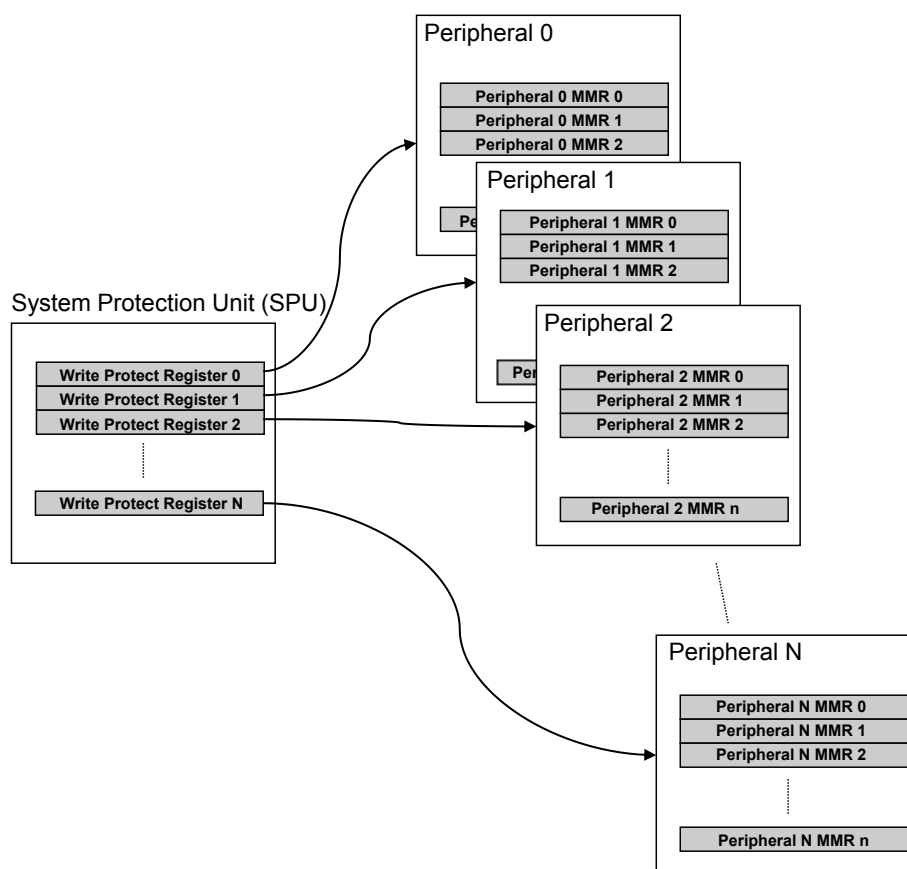


Figure 29-1: SPU Write Protect Registers

In the figure, a write-protect register in the SPU module blocks write-attempts to the MMR space of the associated peripheral. The bits in the write-protect register specify from which requesters to block write-access.

NOTE: A SPU write protection register (`SPU_WP[n]`) exists for the SPU alone. If all defined bits are set in this register for the SPU, any configurations in the SPU are locked and cannot be changed. Only a system reset can restore access to the SPU.

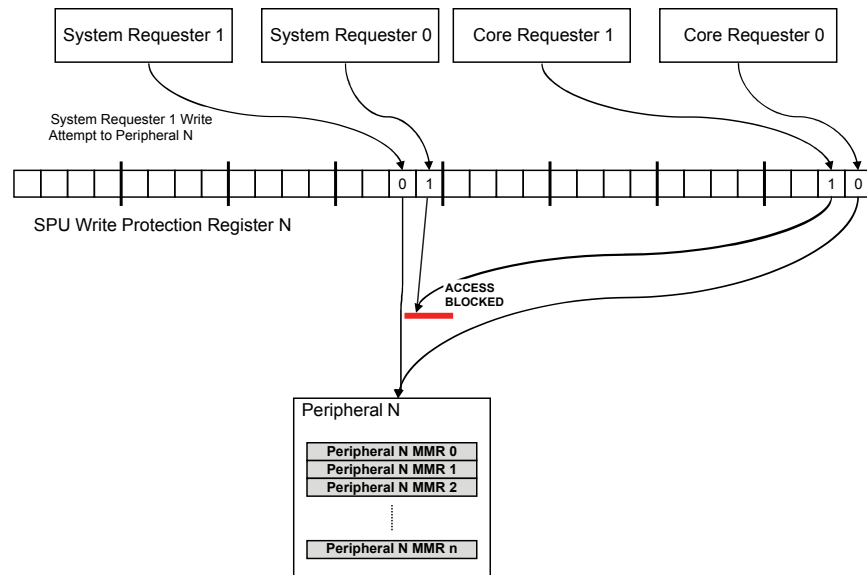


Figure 29-2: SPU Write-Protect Register Blocking Access from System Requester 0 and Core Requester 1

Global Locking

The SPU also has global locking capability. When enabled by setting `SPU_CTL.GLCK` bit field to a value other than `0xAD`, a system-wide global lock signal is active. Some peripherals have a lock enable bit in their control register. When this bit is set, the peripheral recognizes the global lock signal and blocks further write-accesses to its own control register. Access to the configuration register of the peripheral is enabled when the global lock is turned off in the SPU.

The *Global Locking* figure is a conceptual diagram. The diagram shows how the SPU module (or any peripheral) blocks any write attempts to its control register when:

- The global lock signal from the SPU is active, and
- The global lock enable bit is set in the control register of the peripheral

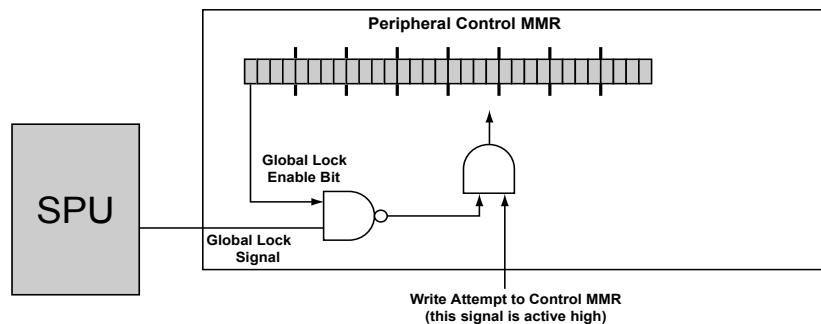


Figure 29-3: Global Locking

The SPU can write-protect its own registers. When the `SPU_CTL.WPLCK` bit is set and global locking is enabled, the SPU blocks accesses to the SPU write-protection registers. To enable write access to the write-protection registers in the SPU, disable the global locking.

SPU Block Diagram

The *SPU System-Level Block Diagram* shows a system-level block diagram of where the SPU is located in the system. It resides between the SMMR interface and the system crossbar. Depending on the configuration of the SPU write-protect registers, it can block access to some peripherals from certain SMMR requesters.

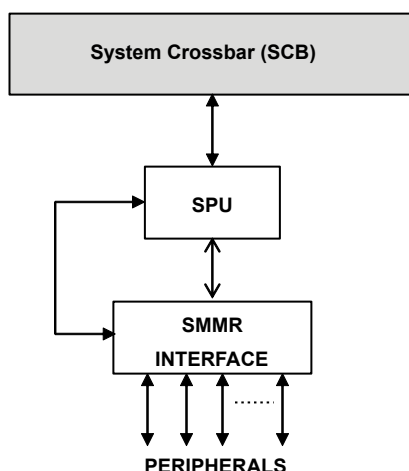


Figure 29-4: SPU System-Level Block Diagram

SPU Architectural Concepts

As shown in the block diagram, the SPU sits between the system crossbar (SCB) and the SMMR interface to the peripherals. The SPU gates any MMR access to any peripheral from any requester that comes through the SCB. Depending on the configuration of the write-protection registers in the SPU, the SPU does or does not allow the MMR write to go through.

The SPU also checks whether the transaction is a secure or non-secure transaction and blocks it according to the configured security setting for the target destination. A secure requester can generate secure read or secure write transactions which can access secure or non-secure completers. A non-secure requester can generate non-secure read or non-secure write transactions and can only access non-secure completers.

SPU Event Control

The system protection unit provides write-protection against MMRs peripherals and its own write-protect registers. If a write attempt is made to any locked MMR peripheral the SPU has write-protected, it blocks the write. The SPU generates a bus error to the requester that attempted the write. That requester does or does not generate an event, based on the returned error.

The SPU can be configured to generate an interrupt for the write-protection violation by setting the `SPU_CTL.PINTEN` bit. The SPU can also be configured to generate an interrupt for a security violation by setting the `SPU_SECURECTL.SINTEN` bit. If either one or both bits is triggered, the `SPU_STAT.VIRQ` bit is set.

The SPU can also lock its own registers from write attempts. If a write-attempt is made to a locked register in the SPU, the SPU blocks it and records it as an error in the `SPU_STAT.LWERR` bit. Again, the SPU generates a bus error to the requester that attempted the write.

The requester does or does not generate an event, based on the returned error.

The SPU does not generate an event for a blocked write access to an SPU register. If the `SPU_CTL.PINTEN` bit is set, the SPU triggers an interrupt for this blocked access attempt.

The global lock is enabled by setting the `SPU_CTL.GLCK` bit to something other than 0xAD. If the lock bit is set in that same configuration register, a peripheral can block write access to its configuration register. When the SPU blocks a write attempt, the peripheral logs and reports the failed attempt. The SPU is unaware and therefore does not provide any indication of a failed write attempt to the configuration register of the peripheral.

SPU Programming Model

The system protection unit (SPU) consists of write-protect and access-protect registers. Each one corresponds to a different peripheral instance. Bits in the write-protect registers correspond to system requesters that can modify the MMR contents of the peripherals. By writing to these write-protect registers, the corresponding memory-mapped registers of the peripheral are write-protected against requesters whose bits in the write-protect register are set.

The SPU globally locks the control register of the peripheral. Peripherals that support this feature have a lock enable bit in their control register. The peripheral blocks any additional write attempts to its control register from any requester when:

- The global lock signal is active from the SPU, and
- The lock enable bit of the peripheral is set

If the lock enable bit of a peripheral is not set and the global lock signal is active, access to that control register of the peripheral is still allowed. To grant access again, disable the global lock signal from the SPU by writing the value 0xAD into the `SPU_CTL.GLCK` bit field.

Another protection mechanism that the SPU offers is write-protection against the write-protection registers. If the write protect register lock bit (`SPU_CTL.WPLCK`) is set and the global lock signal is active, writes to the write-protect registers of the SPU are blocked. To reenabling access to the write-protect registers in the SPU, deactivate the global lock signal by writing 0xAD into the `SPU_CTL.GLCK` bit field.

For security, the SPU provides a set of `SPU_SECUREC[n]` registers (one for each processor core from Analog Devices) to configure their security settings. The SPU also provides a set of `SPU_SECUREP[n]` registers (one for each peripheral instance) to configure their security settings.

Enabling and Disabling the SPU

The SPU is always operating. There are no bits to enable or disable the SPU. The SPU configuration can be updated at any time. Any ongoing transactions finish before a new configuration is in effect. By default, the SPU does not write-protect any of the MMRs.

Write-Protecting the SPU

The SPU is treated like any other peripheral in the system. As such, the SPU also has an associated write-protection register. If this write-protection register is configured to block all writes from all requesters, any SPU configuration remains the same until the next system reset.

Checking the Security State

In some cases while running a peripheral, an application system requester does not know whether they are a secure requester generating secure transactions or not. The SPU provides a means for checking the security state of the requester through the `SPU_SECURECHK` register. When read by a secure requester, the register reads 0xFFFFFFFF and when read by a non-secure requester, the value is 0x00000000.

SPU Mode Configuration

The SPU can provide address range-wide protection by write-protecting the peripherals MMR address range from system MMR requesters. It can also provide register wide protection using global locking. Peripherals that support this feature can enable it in their respective configuration register. When the SPU enables the global lock signal, all subsequent writes to the configuration register of the peripheral are blocked until the global lock signal is deasserted. Similarly, the write-protection registers of the SPU can be write-protected using the global lock signal as well. The SPU uses all these modes of operation together.

Locking Write-Protect Registers

Use the following steps to lock (write-protect) a register.

1. Set the `SPU_CTL.WPLCK` bit and configure the `SPU_CTL.GLCK` field to something other than 0xAD.

The SPU write-protect registers are blocked from further write accesses.

Protecting a Peripheral

Use the following procedure to protect a peripheral.

1. Determine which peripheral needs protection and locate the corresponding write-protect register (`SPU_WP[n]`) in the SPU. See the Write-Protect and Secure Peripheral Registers section.
2. Determine the SMMR requesters from which the peripheral needs protection. Then, set the corresponding bit or bits in the write-protect register (`SPU_WP[n]`) for the peripheral. See the Write-Protect and Secure Peripheral Registers section.

After setting the write-protect register for the particular peripheral, the identified SMMR requesters are blocked from writing to any MMR in the address space of the peripheral. This block remains in place until the bits in the write-protect register are cleared.

Configuring Security Privileges of a Peripheral

Use the following procedure to configure the security privileges of a peripheral.

1. Determine the peripheral and its corresponding secure peripheral register (`SPU_SECUREP[n]`) in the SPU. See the Write-Protect and Secure Peripheral Registers section.
2. If the peripheral is to be a secure completer that only accepts secure transactions, set bit 0 (`SPU_SECUREP[n].SSEC`).
3. If the peripheral is to be a secure requester that generates secure transactions (keeping in mind not all peripherals can be requesters), set bit 1 (`SPU_SECUREP[n].MSEC`).

This procedure sets the security privileges of a peripheral.

NOTE: Only a secure requester can set security privileges, keeping the chain of trust intact. If a non-secure requester configures the security privileges, it can undermine security protection.

ADSP-21560_21561_21564_21568 SPU Register Descriptions

System Protection Unit (SPU) contains the following registers.

Table 29-3: ADSP-21560_21561_21564_21568 SPU Register List

Name	Description
<code>SPU_CTL</code>	Control Register
<code>SPU_SECURECHK</code>	Secure Check Register
<code>SPU_SECURECTL</code>	Secure Control Register
<code>SPU_SECUREC[n]</code>	Secure Core Registers
<code>SPU_SECUREP[n]</code>	Secure Peripheral Register
<code>SPU_STAT</code>	Status Register
<code>SPU_WP[n]</code>	Write Protect Register n

Control Register

The SPU control register (`SPU_CTL`) provides a global lock for configuration registers as well as control for locking the write protect (`SPU_WP[n]`) registers. It also controls the generation of an interrupt to report blocked accesses.

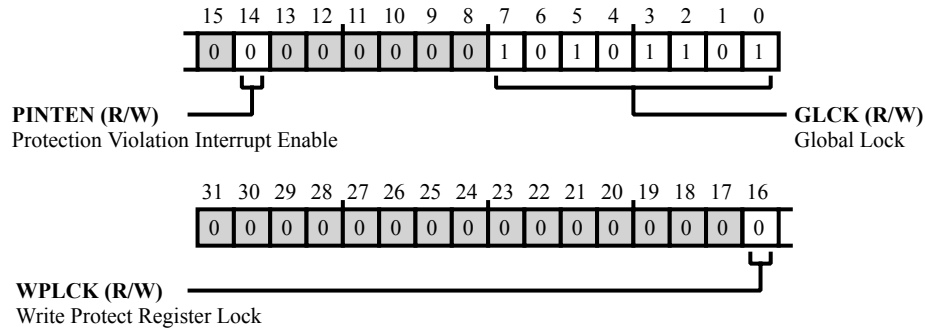


Figure 29-5: SPU_CTL Register Diagram

Table 29-4: SPU_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
16 (R/W)	WPLCK	Write Protect Register Lock.
		When the <code>SPU_CTL.WPLCK</code> bit is set in combination with the <code>SPU_CTL.GLCK</code> bit, writes to the SPU's write protect registers are blocked and return an error.
		0 Disable 1 Enable
14 (R/W)	PINTEN	Protection Violation Interrupt Enable.
		When the <code>SPU_CTL.PINTEN</code> bit is set (=1), a block of any transaction according to the configured settings produces an interrupt.
		0 Disable 1 Enable
7:0 (R/W)	GLCK	Global Lock. The <code>SPU_CTL.GLCK</code> controls the global lock signal. The global lock signal provides register-based write protection. Writing 0xAD to this field disables the lock, and writing any other value enables the lock.

Secure Check Register

The `SPU_SECURECHK` register reads by secure requesters return 0xFFFFFFFF. Reads by non-secure requesters return 0x00000000.

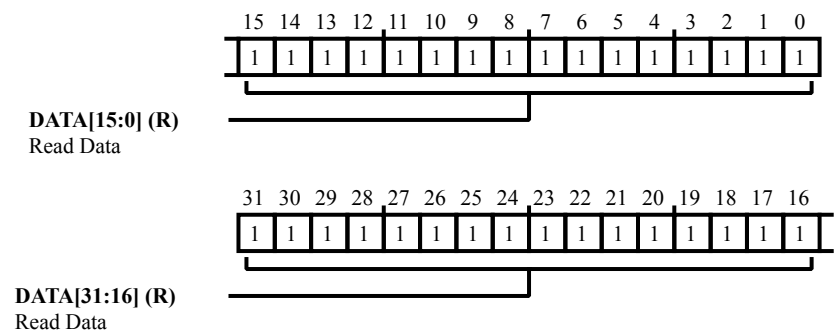


Figure 29-6: SPU_SECURECHK Register Diagram

Table 29-5: SPU_SECURECHK Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	DATA	Read Data. The <code>SPU_SECURECHK.DATA</code> bit field performs reads. Reads by secure requesters return 0xFFFFFFFF. Reads by non-secure requesters return 0x00000000.

Secure Control Register

The SPU Secure Control Register (`SPU_SECURECTL`) allows the user to lock write access to all the `SPU_SECUREC[n]` and `SPU_SECUREP[n]` registers as well as configure the interrupt generation in an event of a security error. It also allows bulk clear of the SSEC bits and/or MSEC bits in the `SPU_SECUREP[n]` registers.

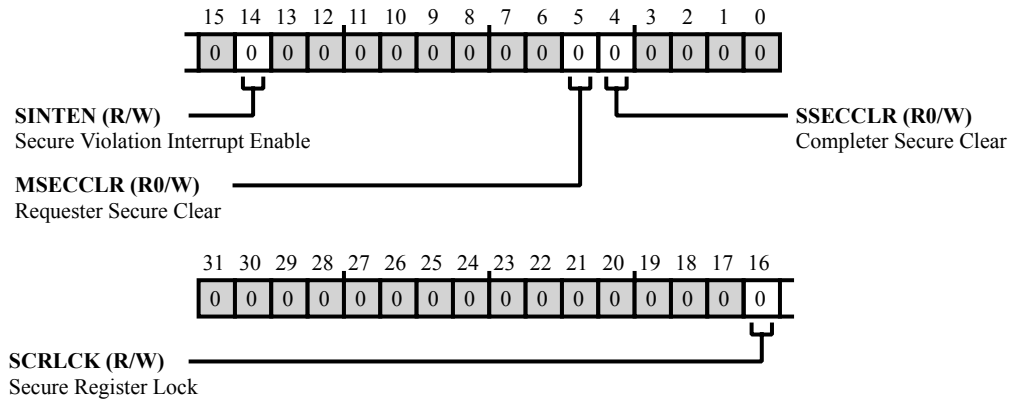


Figure 29-7: SPU_SECURECTL Register Diagram

Table 29-6: SPU_SECURECTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
16 (R/W)	SCRLCK	Secure Register Lock. When the <code>SPU_SECURECTL.SCRLCK</code> bit is set in combination with the <code>SPU_CTL.GLCK</code> bit, writes to the Security Configuration registers (<code>SPU_SECUREC[n]</code> and <code>SPU_SECUREP[n]</code>) are blocked and return an error which is captured in the <code>SPU_STAT.LWERR</code> bit.
		0 Disable
		1 Enable
14 (R/W)	SINTEN	Secure Violation Interrupt Enable. The <code>SPU_SECURECTL.SINTEN</code> bit generates an interrupt if a security violation was captured. Interrupt status is provided in the <code>SPU_STAT.VIRQ</code> bit.
		0 Disable
		1 Enable
5 (R0/W)	MSECCLR	Requester Secure Clear. When the <code>SPU_SECURECTL.MSECCLR</code> bit is set, the <code>SPU_SECUREP[n].MSEC</code> bits in all <code>SPU_SECUREP[n]</code> registers are cleared. The <code>SPU_SECURECTL.MSECCLR</code> bit always reads back as a 0.
		0 No Action
		1 Clear All Requester Secure Control Bits

Table 29-6: SPU_SECURECTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
4 (R0/W)	SSECCLR	Completer Secure Clear. When the SPU_SECURECTL.SSECCLR bit is set, the SPU_SECUREP[n].SSEC bits in all SPU_SECUREP[n] registers are cleared. The SPU_SECURECTL.SSECCLR bit always reads back as a 0.
		0 No Action
		1 Clear All Completer Secure Control Bits

Secure Core Registers

A SPU register exists for every DSP core in the system. The bits enable or disable security for features in the core.

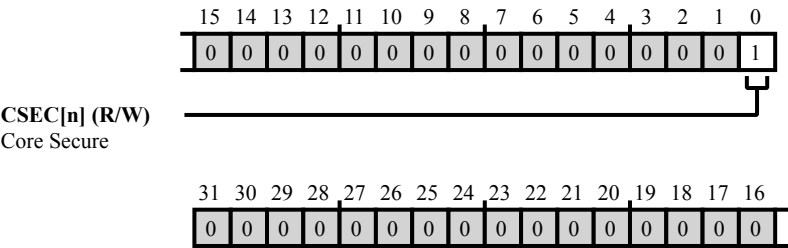


Figure 29-8: SPU_SECUREC[n] Register Diagram

Table 29-7: SPU_SECUREC[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	CSEC[n]	Core Secure. The SPU_SECUREC [n] .CSEC [n] bit controls whether non-secure accesses are allowed to L1 memory of the processor core. When =1, the core (as a completer) is set as secure meaning only secure transactions are allowed to L1.
		0 Disable
		1 Enable

Secure Peripheral Register

In the system, each `SPU_SECUREP[n]` register is assigned to a specific MMR address range associated with one peripheral. Each `SPU_SECUREP[n]` has a Completer Secure (SSEC) bit and a Requester Secure (MSEC) bit. When the SSEC bit is set, the SPU will only allow secure requesters generating secure transactions to access the peripheral's MMR address space. When the MSEC bit is set, the associated peripheral will be secure and will generate secure transactions.

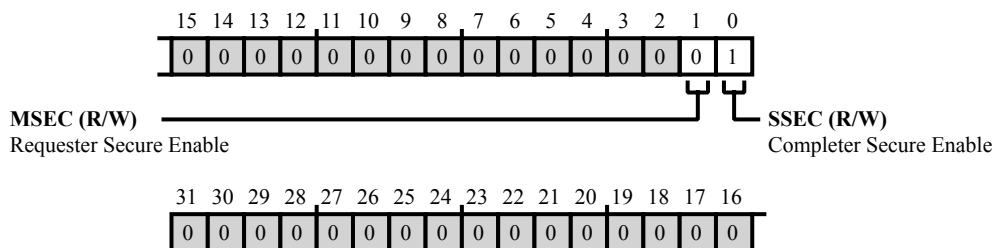


Figure 29-9: SPU_SECUREP[n] Register Diagram

Table 29-8: SPU_SECUREP[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/W)	MSEC	Requester Secure Enable. The <code>SPU_SECUREP[n].MSEC</code> bit controls whether the peripheral generates secure transactions as a requester. When clear (=0), the peripheral generates non-secure transactions as a requester (if applicable). When set (=1), the peripheral generates secure transactions as a requester.
		0 Disable
		1 Enable
0 (R/W)	SSEC	Completer Secure Enable. The <code>SPU_SECUREP[n].SSEC</code> bit controls whether the peripheral is protected from non-secure transactions. When clear (=0), the security status of the transaction is ignored. When set (=1), only secure transactions are allowed to access the address space of the peripheral and non-secure transactions are blocked.
		0 Disable
		1 Enable

Status Register

The `SPU_STAT` register indicates if there have been any errors, active interrupts and global lock status.

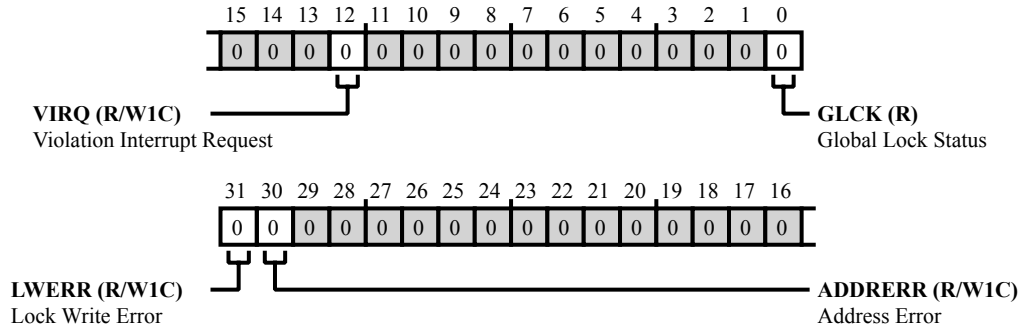


Figure 29-10: `SPU_STAT` Register Diagram

Table 29-9: `SPU_STAT` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W1C)	LWERR	Lock Write Error. The <code>SPU_STAT.LWERR</code> indicates whether there has been an attempted write to a register in the SPU with its lock bit (<code>SPU_CTL.WPLCK</code> or <code>SCRLOCK</code>) set while <code>SPU_CTL.GLCK</code> was asserted. This bit is W1C.
		0 Inactive
		1 Active
30 (R/W1C)	ADDRERR	Address Error. The <code>SPU_STAT.ADDRERR</code> indicates whether there has been an attempted write to a read-only register or an access an invalid address in the SPU MMR address range. This bit is W1C.
		0 Inactive
		1 Active
12 (R/W1C)	VIRQ	Violation Interrupt Request. The <code>SPU_STAT.VIRQ</code> bit indicates that a security and/or protection violation has been detected and interrupt asserted. This is a W1C bit.
		0 Inactive
		1 Active
0 (R/NW)	GLCK	Global Lock Status. The <code>SPU_STAT.GLCK</code> indicates whether the global lock is enabled or disabled.
		0 Disabled (<code>global_lock=0</code>)
		1 Enabled (<code>global_lock=1</code>)

Write Protect Register n

In the system, each `SPU_WP[n]` register is assigned to a specific MMR address range associated with one peripheral. When the appropriate bits are set, writes to the peripheral from a specific requester are blocked and an error is returned to the requester. For more information, see the processor specific additional information for the `SPU_WP[n]` register.

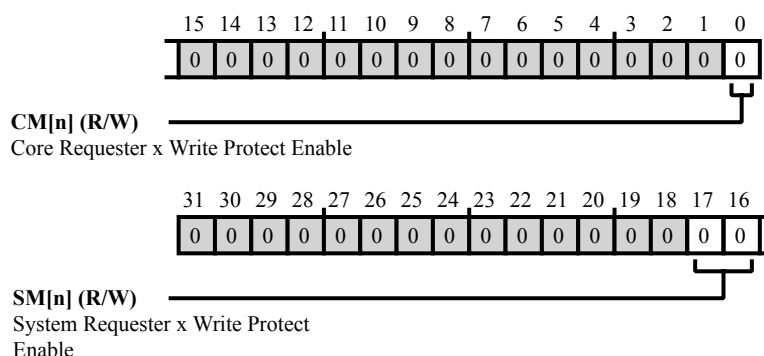


Figure 29-11: `SPU_WP[n]` Register Diagram

Table 29-10: `SPU_WP[n]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
17:16 (R/W)	SM[n]	System Requester x Write Protect Enable. The <code>SPU_WP[n].SM[n]</code> bits correspond to different system requesters in the system. When a particular bit is set in this field, the corresponding system requester cannot write to the corresponding peripheral's MMR address space. The write attempt is blocked by the SPU.
0 (R/W)	CM[n]	Core Requester x Write Protect Enable. The <code>SPU_WP[n].CM[n]</code> bits correspond to different cores in the system. When a particular bit is set in this field, the corresponding core cannot write to the corresponding peripheral's MMR address space. The write attempt is blocked by the SPU.

Write-Protect, Secure Peripheral, and Secure Core Registers

The SPU consists of a collection of write-protect registers each of which are associated with a specific peripheral or target. The SPU also has a collection of secure peripheral registers which are also associated with specific peripherals. The table gives the write-protect register and secure peripheral number for each of the peripherals that are provided with write protection and security through the SPU. The SPU for the processor is configured with 136 write-protect registers and also 136 secure peripheral registers. The number corresponding to a peripheral correlates to both the Write-Protect register and the Secure Peripheral register.

For each processor, there are controllers that are able to access the SMMR space. The *SPU_WPn.CMn and SPU_WPn.SMn Bits* table shows which bits enable the protection against which controller.

The `SPU_WP[n]` register shows which bits enable the protection against which controller.

For each peripheral, there is a corresponding write-protect register, `SPU_WP[n]`, and secure peripheral register, `SPU_SECUREP[n]`. The *Write-Protect Register and Secure Peripheral Number(n)* table shows the write-protect register and secure peripheral number for each peripheral.

Table 29-11: Write-Protect Register and Secure Peripheral Number(n)

No.	Peripheral/Block Name	SPU WPn Mapping	SECUREPn Mapping	Controller Capable
0	MMRG GPV SPACE	WP0	SECUREP0	N/A
2	SPIF GPV SPACE	WP2	SECUREP2	N/A
7	TWI3	WP7	SECUREP7	N/A
10	TWI0	WP10	SECUREP10	N/A
11	TWI1	WP11	SECUREP11	N/A
12	TWI2	WP12	SECUREP12	N/A
13	SPORT0A	WP13	SECUREP13	N/A
14	SPORT0B	WP14	SECUREP14	N/A
15	SPORT1A	WP15	SECUREP15	N/A
16	SPORT1B	WP16	SECUREP16	N/A
17	SPORT2A	WP17	SECUREP17	N/A
18	SPORT2B	WP18	SECUREP18	N/A
19	SPORT3A	WP19	SECUREP19	N/A
20	SPORT3B	WP20	SECUREP20	N/A
21	SPORT4A	WP21	SECUREP21	N/A
22	SPORT4B	WP22	SECUREP22	N/A
23	SPORT5A	WP23	SECUREP23	N/A
24	SPORT5B	WP24	SECUREP24	N/A
25	SPORT6A	WP25	SECUREP25	N/A
26	SPORT6B	WP26	SECUREP26	N/A
27	SPORT7A	WP27	SECUREP27	N/A
28	SPORT7B	WP28	SECUREP28	N/A
29	UART0	WP29	SECUREP29	N/A
30	UART1	WP30	SECUREP30	N/A
32	PORTA	WP32	SECUREP32	N/A
33	PORTB	WP33	SECUREP33	N/A
35	PADS	WP35	SECUREP35	N/A

Table 29-11: Write-Protect Register and Secure Peripheral Number(n) (Continued)

No.	Peripheral/Block Name	SPU WPn Mapping	SECUREPn Mapping	Controller Capable
36	PINT0	WP36	SECUREP36	N/A
37	PINT1	WP37	SECUREP37	N/A
39	SMC0	WP39	SECUREP39	N/A
40	SMPU SMC	WP40	SECUREP40	N/A
41	WDT0	WP41	SECUREP41	N/A
42	WDT1	WP42	SECUREP42	N/A
43	CNT0	WP43	SECUREP43	N/A
44	OTP MMR	WP44	SECUREP44	N/A
47	MEM REPAIR	WP47	SECUREP47	N/A
48	TMR	WP48	SECUREP48	N/A
49	SPORT0A DDE	WP49	SECUREP49	Yes
50	SPORT0B DDE	WP50	SECUREP50	Yes
51	SPORT1A DDE	WP51	SECUREP51	Yes
52	SPORT1B DDE	WP52	SECUREP52	Yes
53	SPORT2A DDE	WP53	SECUREP53	Yes
54	SPORT2B DDE	WP54	SECUREP54	Yes
55	SPORT3A DDE	WP55	SECUREP55	Yes
56	SPORT3B DDE	WP56	SECUREP56	Yes
57	SPORT4A DDE	WP57	SECUREP57	Yes
58	SPORT4B DDE	WP58	SECUREP58	Yes
59	SPORT5A DDE	WP59	SECUREP59	Yes
60	SPORT5B DDE	WP60	SECUREP60	Yes
61	SPORT6A DDE	WP61	SECUREP61	Yes
62	SPORT6B DDE	WP62	SECUREP62	Yes
63	SPORT7A DDE	WP63	SECUREP63	Yes
64	SPORT7B DDE	WP64	SECUREP64	Yes
65	UART0RX DDE	WP65	SECUREP65	Yes
66	UART0TX DDE	WP66	SECUREP66	Yes
67	UART1RX DDE	WP67	SECUREP67	Yes
68	UART1TX DDE	WP68	SECUREP68	Yes
71	XSPI	WP71	SECUREP71	N/A

Table 29-11: Write-Protect Register and Secure Peripheral Number(n) (Continued)

No.	Peripheral/Block Name	SPU WPn Mapping	SECUREPn Mapping	Controller Capable
72	CORE MBIST	WP72	SECUREP72	N/A
73	SPI0TX DDE	WP73	SECUREP73	Yes
74	SPI0RX DDE	WP74	SECUREP74	Yes
75	SPI1TX DDE	WP75	SECUREP75	Yes
76	SPI1RX DDE	WP76	SECUREP76	Yes
77	SPI2TX DDE	WP77	SECUREP77	Yes
78	SPI2RX DDE	WP78	SECUREP78	Yes
79	SPI0	WP79	SECUREP79	N/A
80	SPI1	WP80	SECUREP80	N/A
81	SPI2	WP81	SECUREP81	N/A
82	L2CTL1	WP82	SECUREP82	N/A
85	L2CTL0	WP85	SECUREP85	N/A
86	SMPU L2CTL0 CL2	WP86	SECUREP86	N/A
87	SMPU L2CTL0 DL2	WP87	SECUREP87	N/A
88	SEC0	WP88	SECUREP88	N/A
89	TRU0	WP89	SECUREP89	N/A
90	SPU0	WP90	SECUREP90	N/A
91	RCU0	WP91	SECUREP91	N/A
92	CGU0	WP92	SECUREP92	N/A
94	CDU0	WP94	SECUREP94	N/A
95	DPM0	WP95	SECUREP95	N/A
96	PLL0	WP96	SECUREP96	N/A
101	Enhanced BW MDMA	WP101	SECUREP101	Yes
102	Max BW MDMA	WP102	SECUREP102	Yes
103	MLB0	WP103	SECUREP103	Yes
105	SMPU L2CTL1 CL2	WP105	SECUREP105	N/A
106	SMPU SPI2	WP106	SECUREP106	N/A
107	MEC0	WP107	SECUREP107	N/A
108	CRC0	WP108	SECUREP108	N/A
109	CRC1	WP109	SECUREP109	N/A
110	MDMA0 DDE0 CRC0	WP110	SECUREP110	Yes

Table 29-11: Write-Protect Register and Secure Peripheral Number(n) (Continued)

No.	Peripheral/Block Name	SPU WPn Mapping	SECUREPn Mapping	Controller Capable
111	MDMA0 DDE1 CRC0	WP111	SECUREP111	Yes
112	MDMA1 DDE0 CRC1	WP112	SECUREP112	Yes
113	MDMA1 DDE1 CRC1	WP113	SECUREP113	Yes
115	MISC REG	WP115	SECUREP115	N/A
117	DAI0	WP117	SECUREP117	N/A
118	DAI1	WP118	SECUREP118	N/A
119	CRYPTO SPE	WP119	SECUREP119	Yes
120	CRYPTO PKP	WP120	SECUREP120	N/A
121	EMDMA0	WP121	SECUREP121	Yes
122	DAP ROM	WP122	SECUREP122	N/A
123	SHARC0 DBG	WP123	SECUREP123	N/A
124	SHARC0 CTI	WP124	SECUREP124	N/A
125	SHARC0 PTM	WP125	SECUREP125	N/A
132	CTI System	WP132	SECUREP132	N/A
133	TAPC MMR	WP133	SECUREP133	N/A
134	Debug Control	WP134	SECUREP134	N/A

Processor-Specific Information

Global Locking

The global lock signal from the SPU along with the peripheral lock bit can be used to provide lock functionality for the control MMR of the peripheral. The Global Lock (SPU_CTL . GLCK) field determines whether global lock is enabled or not. Global Lock is disabled if the SPU_CTL . GLCK field is 0xAD (default value), otherwise it is enabled. The following is a list of peripherals that have the global lock bit in their control MMR.

- General-Purpose IO (GPIO)
- System Event Controller (SEC0)
- Trigger Routing Unit (TRU0)
- Clock Generation Unit (CGU0)
- Clock Generation Unit1 (CGU1)
- Clock Distribution Unit (CDU0)
- Dynamic Power Management (DPM)

- Reset Control Unit (RCU0)
- System Protection Unit (SPU0)
- L2 Memory Controller (L2CTL0)

30 Security Packet Engine (PKTE)

The PKTE is a security packet engine designed to off-load the host processor to improve the speed of applications requiring cryptographic processing. The packet engine contains a set of modules for encryption and decryption, hashing, and pseudo-random number generation.

PKTE Features

The PKTE has the following features.

- Hardware assisted processing for the cryptographic ciphers, hashes, and pseudo-random number generation
- Header and trailer processing for Internet security protocols
- DMA capability to move data in and out of the engine efficiently and to allow the engine to run autonomously while moving data
- Interrupt controller to signal module status and errors
- Clock manager for enabling or disabling different features to save power

NOTE: Not all algorithms, decrypt, and hash functions and extra features are available on all product models. For complete information on included features, see the product-specific data sheet.

NOTE: This packet engine provides support for various network security protocols by processing headers and trailers as well as accelerating cryptographic functions. Not all processors have direct support for Ethernet. As such, the packet engine can still be used if Ethernet is indirectly used.

PKTE Functional Description

The packet engine contains a set of modules for encryption and decryption, hashing, and pseudo-random number generation. The following sections describe these functional blocks.

ADSP-21560_21561_21564_21568 PKTE Register List

The Security Packet Engine (PKTE) provides security-related features. A set of registers governs PKTE operations. For more information on PKTE functionality, see the PKTE register descriptions.

Table 30-1: ADSP-21560_21561_21564_21568 PKTE Register List

Name	Description
PKTE_ARC4STATE_ADDR	Packet Engine ARC4 State Record Address
PKTE_ARC4STATE_BUF	Starting Entry of 256-byte ARC4 State Buffer
PKTE_BUF_PTR	Packet Engine Buffer Pointer Register
PKTE_BUF_THRESH	Packet Engine Buffer Threshold Register
PKTE_CDRBASE_ADDR	Packet Engine Command Descriptor Ring Base Address
PKTE_CDSC_CNT	Packet Engine Command Descriptor Count Register
PKTE_CDSC_INCR	Packet Engine Command Descriptor Count Increment Register
PKTE_CFG	Packet Engine Configuration Register
PKTE_CLK_CTL	PE Clock Control Register
PKTE_CONT	PKTE Continue Register
PKTE_CTL_STAT	Packet Engine Control Register
PKTE_DATAIO_BUF	Starting Entry of 256-byte Data Input/Output Buffer
PKTE_DEST_ADDR	Packet Engine Destination Address
PKTE_DMA_CFG	Packet Engine DMA Configuration Register
PKTE_ENDIAN_CFG	Packet Engine Endian Configuration Register
PKTE_HLT_CTL	Packet Engine Halt Control Register
PKTE_HLT_STAT	Packet Engine Halt Status Register
PKTE_IMSK_DIS	Interrupt Mask Disable Register
PKTE_IMSK_EN	Interrupt Mask Enable Register
PKTE_IMSK_STAT	Interrupt Masked Status Register
PKTE_INBUF_CNT	Packet Engine Input Buffer Count Register
PKTE_INBUF_INCR	Packet Engine Input Buffer Count Increment Register
PKTE_INT_CFG	Interrupt Configuration Register
PKTE_INT_CLR	Interrupt Clear Register
PKTE_INT_EN	Interrupt Enable Register
PKTE_IUMSK_STAT	Interrupt Unmasked Status Register
PKTE_LEN	Packet Engine Length Register
PKTE_OUTBUF_CNT	Packet Engine Output Buffer Count Register
PKTE_OUTBUF_DECR	Packet Engine Output Buffer Count Decrement Register
PKTE_RDRBASE_ADDR	Packet Engine Result Descriptor Ring Base Address
PKTE_RDSC_CNT	Packet Engine Result Descriptor Count Registers

Table 30-1: ADSP-21560_21561_21564_21568 PKTE Register List (Continued)

Name	Description
PKTE_RDSC_DECR	Packet Engine Result Descriptor Count Decrement Registers
PKTE_RING_CFG	Packet Engine Ring Configuration
PKTE_RING_PTR	Packet Engine Ring Pointer Status
PKTE_RING_STAT	Packet Engine Ring Status
PKTE_RING_THRESH	Packet Engine Ring Threshold Registers
PKTE_SA_ADDR	Packet Engine SA Address
PKTE_SA_ARC4IJPTR	ARC4 i and j Pointer Register
PKTE_SA_CMD0	SA Command 0
PKTE_SA_CMD1	SA Command 1
PKTE_SA_IDIGEST[n]	SA Inner Hash Digest Registers
PKTE_SA_KEY[n]	SA Key Registers
PKTE_SA_NONCE	SA Initialization Vector Register
PKTE_SA_ODIGEST[n]	SA Outer Hash Digest Registers
PKTE_SA_RDY	SA Ready Indicator
PKTE_SA_SEQNUM[n]	SA Sequence Number Register
PKTE_SA_SEQNUM_MSK[n]	SA Sequence Number Mask Registers
PKTE_SA_SPI	SA SPI Register
PKTE_SRC_ADDR	Packet Engine Source Address
PKTE_STAT	Packet Engine Status Register
PKTE_STATE_ADDR	Packet Engine State Record Address
PKTE_STATE_BYTE_CNT[n]	State Hash Byte Count Registers
PKTE_STATE_IDIGEST[n]	State Inner Digest Registers
PKTE_STATE_IV[n]	State Initialization Vector Registers
PKTE_USERID	Packet Engine User ID

ADSP-21560_21561_21564_21568 PKTE Interrupt List

Table 30-2: ADSP-21560_21561_21564_21568 PKTE Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
111	PKTE0_IRQ	PKTE0 Interrupt	Level	

PKTE Definitions

Command Descriptor

An 8-word structure that is either written directly into the packet command MMR set or is placed in a Command Descriptor Ring (CDR) in the processor memory. The packet engine sequentially processes the structure. The command descriptor contains the information that varies for every packet. This information includes pointers to the SA record, the state information, the source packet, and the destination packet.

Command Descriptor Ring (CDR)

A circular contiguous portion of memory which is used to manage one or more command descriptions for the packet engine.

Result Descriptor

When the packet engine completes the processing of a packet, it writes a result descriptor with the state information. The result descriptor can be read directly from the result register set or from the Result Descriptor Ring (RDR) in the processor memory.

Result Descriptor Ring (RDR)

A circular contiguous portion of memory which holds the mirror or copy of the CDR but contains the result descriptors. The RDR and CDR can be overlaid on top of each other.

Security Association (SA) Record

A structure that contains the remainder of the information the packet engine requires to process a packet. Most of the information fields in the SA record such as the key and encryption mode are static for the lifetime of the association. The fields do not require frequent manipulation by the processor core. The SA record non-static fields are the sequence number and sequence number mask. The SA record can have a corresponding state record for saving results from the current operations that are useful for future operations. The state record can hold the IV, the hash byte count, and the intermediate hash digest.

Cipher

A method or algorithm to encrypt or decrypt information

Hash

A cryptographic hash is a function that takes an arbitrary block of data and returns a fixed-size bit string. Four main properties define the function:

- It is easy to compute a hash value for any given input

- It is infeasible to generate the original input from a given hash
- It is infeasible to modify the input without changing the resulting hash
- It is infeasible to find two different inputs that result in the same hash

Autonomous Ring Mode (ARM)

Mode of operation in which most of the parameters as well as the data are set up in memory and moved to the engine for configuration and processing through DMA.

Target Command Mode (TCM)

Mode of operation where some parameters are set up in memory and moved into the packet engine through DMA while the other parameters are directly written to the registers. DMA moves the input and output data in and out of the engine.

Direct Host Mode (DHM)

Mode of operation that does not use DMA. All parameters are directly written to and read from the MMRs. The input and output are written to and read from the FIFO buffers.

Cipher Module

The cipher module does the symmetric encrypt or decrypt operations for:

- Data Encryption Standard (DES)
- Triple-DES
- ARC4
- Advanced Encryption Standard (AES) algorithms

The cipher module supports standard modes for DES and AES that include Electronic Code Book (ECB) and Cipher Block Chaining (CBC). The key size for DES is 56 bits, for Triple-DES is 168 bits. The AES module also provides support for AES counter-modes for IPsec and SRTP. All AES modes can use key sizes of 128 bits and 192/256 bits. Key scheduling is automatic and done in parallel with the encrypt or decrypt operation.

Hash Module

The hash module is tightly coupled with the encrypt or decrypt module and provides hardware accelerated one-way hash functions. Operations that combine both hash and encrypt or decrypt functions are provided to reduce processing time for data that needs both applied. For hash-then-decrypt operations, the packet engine performs parallel execution of both functions from the input buffer. For encrypt-then-hash operations, the processing is pipelined from the input buffer to provide minimum latency. An offset can be specified between the start of hashing and the start of encryption to support protocols such as IPsec or SRTP. The HMAC keyed hashing mechanism is supported

for MD5, SHA-1, SHA-2-224 and SHA-2-256. The SSL-MAC is supported for MD5 and SHA-1. A second AES-CBC module for the hash function enables parallel processing for AES-CCM, a combined hash and encrypt algorithm.

Pseudo-Random Number Generator

Cipher algorithms that operate in CBC mode or counter-mode, require an IV. This IV must not be secret; however the IV must be unpredictable and unique for each execution of the encryption process. Pseudo-random number generators are deterministic algorithms that output statistically independent and unbiased numbers. True random number generators are non-deterministic and use the randomness that occurs in a physical process. The packet engine incorporates an ANSI X9.31 compliant Pseudo Random Number Generator (PRNG) that it can use to generate unique IVs using strong encryption. The ANSI X9.31 PRNG is defined as part of the ANSI X9 standards that are used to secure financial transactions. The function can also be used for pseudo-random number generation as part of an implementation of the digital signature standard described in NIST FIPS PUB 186-2.

The PRNG function, as defined by ANSI X9.31, is based on the AES cipher. This section describes the function to promote understanding of the different inputs and outputs of the PRNG function itself.

NOTE: The PRNG in the packet engine is only based on the AES cipher with 128-bit keys. Other ciphers and key lengths are not supported for the PRNG based on ANSI X9.31.

Let $e \times K(Y)$ represent the AES encryption of Y under the key K.

The PRNG function uses three inputs:

- K, a 128-bit key
- V, a 128-bit seed value
- DT, a 128-bit date/ time vector which is updated on each iteration

The intermediate value I is the result of an AES encryption of the data and time vector under key K.

$$I = e \times K(DT)$$

That value I is then XOR-ed with the seed V and AES encrypted under key K. The result R is the output of the PRNG function.

$$R = e \times K(I \text{ XOR } V)$$

A new seed value V is generated from the AES encryption of the result R XOR'ed with the intermediate value I under the key K.

$$V = e \times K(R \text{ XOR } I)$$

The PRNG function is deeply integrated inside the datapath of the packet engine. The function is controlled indirectly through the PRNG mode bits in the `PKTE_CTL_STAT.PRNGMD` bit field of the command descriptor and the IV source selection bits in the `PKTE_SA_CMD0.IVSRC` bit field of the SA record.

The PKTE module supports four different modes.

1. Load IV from PRNG for the current operation: `PKTE_CTL_STAT.PRNGMD = 0b00` and `PKTE_SA_CMD0.IVSRC = 0b11`.
2. PRNG init mode initializes the PRNG with a key, seed, and date/time value: `PKTE_CTL_STAT.PRNGMD = 0b01`.

Before the PRNG function can be used, it must be initialized with a key, seed, and date/time value. At initialization, the key, seed, and date/time values are programmed. Other PRNG operations do not change the key, however the seed, and date/time values are updated (not re-programmed). The date/time is updated every 128 system clock cycles.

The date/time is a 128-bit value with randomly distributed number of ones and zeros. It must not be all zeros.

The *SA Record for PRNG Init Operation* table shows how the key, seed and date/time values are loaded into the PKTE registers for initialization.

Table 30-3: SA Record for PRNG Init Operation

Parameter	SA Field	Description
K	SA_KEY0	PRNG key [127:96]
	SA_KEY1	PRNG key [95:64]
	SA_KEY2	PRNG key [63:32]
	SA_KEY3	PRNG key [31:0]
V	SA_IDIGEST0	PRNG seed [127:96]
	SA_IDIGEST1	PRNG seed [95:64]
	SA_IDIGEST2	PRNG seed [63:32]
	SA_IDIGEST3	PRNG seed [31:0]
DT	SA_ODIGEST0	PRNG date/time [127:96]
	SA_ODIGEST1	PRNG date/time [95:64]
	SA_ODIGEST2	PRNG date/time [63:32]
	SA_ODIGEST3	PRNG date/time [31:0]

3. PRNG generate mode generates pseudo-random data on initialized key, seed, and date/time value: `PKTE_CTL_STAT.PRNGMD = 0b10`.

The PRNG function can be used to generated pseudo-random data for other purposes than IVs. For this mode, the PRNG must have been initialized once with the PRNG init mode.

The PRNG generate mode uses the initialized key and unique changing date/time value as inputs. The pseudo-random data is output to the output buffer of the packet engine.

The `LEN` field in the command descriptor indicates the amount of pseudo random data that is generated in multiples of 16 bytes. The maximum is $255 \times 16 = 4080$ bytes.

In autonomous ring mode, the output data is copied to the host memory at the destination address in the command descriptor. In direct host mode, the host must read the data directly from the output buffer. No SA record is used for this function.

Directly after the PRNG generate mode, a new pseudo-random number is generated and available for the next operation that uses the option `PKTE_SA_CMD0.IVSRC = PRNG`.

4. PRNG test mode generates pseudo-random data on initialized key, seed, and input (test) data:

`PKTE_CTL_STAT.PRNGMD = 0b11`.

The PRNG test mode can be used to test the correctness of the PRNG function. This mode is similar to the PRNG generate mode, except that the data is read from the input buffer of the packet engine, instead of the date/time value.

For this mode, the PRNG must have been initialized once with the PRNG Init mode.

The `LEN` field in the command descriptor indicates the amount of pseudo-random data to be generated in multiples of 16 bytes. The maximum is limited to the `LEN` field in bytes.

In autonomous ring mode, the output data is copied to the host memory at the destination address in the command descriptor. In the direct host mode, the host must read the data directly from the output buffer. No SA record is used for this function.

Directly after the PRNG test mode, a new pseudo-random number is generated and available for the next operation that uses the option `PKTE_SA_CMD0.IVSRC = PRNG`.

Packet Engine Processing Details

This section describes data processing through the packet engine. It describes padding and supported algorithms for each protocol.

A valid Security Association (SA) must be created before packet processing can start. A formatted SA record must reside in memory and be accessible to the packet engine. The host processor application is responsible for these tasks.

Crypto Padding

Padding is the process of adding data to fill-out a fixed-size plain text data structure. Three factors determine when to use a pad field:

1. If a block cipher encryption algorithm is used, a pad field is used to expand the plain text to a multiple of the block size.
2. Padding can be used to ensure that the cipher text terminates on an n-byte boundary.
3. Padding can conceal the actual length of the payload.

To facilitate peak encrypt or decrypt performance, the packet engine supports the following most commonly used padding functions in hardware:

1. Pad generation and insertion of pad bytes to the end of plain text prior to encryption, for outbound operations.

2. Pad verification to check for correct padding after decrypting a packet for inbound operations.
3. Pad consumption to strip the pad bytes from the plain text data after decrypting a packet, for inbound operations.

Pad Generation and Insertion

The pad type and quantity of bytes the packet engine inserts depends on the plain text length and the value of the following fields:

- `PKTE_SA_CMD0.PADTYPE` and `PKTE_SA_CMD0.ETHPAD` defines the type of padding
- `PKTE_CTL_STAT.PADVAL` defines a value that is inserted in the pad
- `PKTE_SA_CMD0.CIPHER` enforces a certain pad alignment
- `PKTE_SA_CMD1.CIPHERMD` enforces a certain pad alignment
- `PKTE_SA_CMD0.SCPAD` allows stream ciphers to be padded
- `PKTE_CTL_STAT.PADCTLSTAT` controls the pad alignment

The `PKTE_CTL_STAT.PADCTLSTAT` bit field of the result descriptor returns the total number of inserted pad bytes.

Pad Types

The pad type bit field (`PKTE_SA_CMD0.PADTYPE`) and the extended pad bit (`PKTE_SA_CMD0.ETHPAD`) select the pad type for the extended protocol group. The packet engine can generate different pad types in hardware as described in the *Pad Examples* table.

The `PKTE_CTL_STAT.PADVAL` bit field, together with the number of pad bytes, defines the value that is inserted in the pad. The format of the pad and the use of this field is best explained in an example (see the *Pad Examples* table).

For the IPsec pad type, this field holds the value that is inserted into the next header field (in the ESP trailer) of the innermost operation's header. For the Constant pad type or the Constant SSL pad type, this field holds the inserted fixed constant pad value. For all other pad types, this field is not used and must be zero.

Table 30-4: Pad Types

Pad Type	Value	Description
IPSec	0b000	Append 0 to 255 pad bytes, followed by a pad length byte and a next header byte. The first pad byte appended to the plain text is numbered 1, with subsequent pad bytes making up a monotonically increasing sequence: 1, 2, 3 and up. Append the pad length field that indicates the number of pad bytes (0–255), where a value of zero indicates no pad bytes present. Append the next header byte as specified in the <code>PKTE_CTL_STAT.PADVAL</code> field of the command descriptor. A minimum of 2 bytes are appended; zero pad bytes plus the pad length byte plus the next header byte, in which case the <code>PKTE_CTL_STAT.PADCTLSTAT</code> field in the result descriptor returns 0x02. A maximum of 257 bytes can be appended, in which case the <code>PKTE_CTL_STAT.PADCTLSTAT</code> field in the descriptor returns 0x01.
PKCS#7	0b001	Appends 1–128 pad bytes with a pad byte value equal to the pad length (1–128).
Constant	0b010	Appends 0–255 pad bytes of a user-specified character to the plain text data. This character is specified in the <code>PKTE_CTL_STAT.PADVAL</code> field of the command descriptor.
Zero	0b011	Appends 0–255 pad bytes of 0x00 to the plain text data.
Constant SSL	0b110	Appends 0–255 pad bytes of a user-specified character to the plain text data, followed by a ‘pad length’ byte (0–255). This character is specified in the <code>PKTE_CTL_STAT.PADVAL</code> field of the command descriptor. A total of 256 bytes can be appended, in which case the pad field returns 0x00.

For example, the *Pad Examples* table shows the appended pad for any of the pad types for an outbound (encrypt) operation. The table shows a plain text input of 2 bytes using the 8-byte block cipher crypto-algorithm DES-ECB and a `PKTE_CTL_STAT.PADVAL` field value of 0xAA.

Table 30-5: Pad Examples

Pad Type	Pad field (extended to Plain Text)	<code>PKTE_CTL_STAT</code>
IPSec	0x01, 0x02, 0x03, 0x04, 0x04, 0xAA	0x06
PKCS#7	0x06, 0x06, 0x06, 0x06, 0x06, 0x06	0x06
Constant	0xAA, 0xAA, 0xAA, 0xAA, 0xAA, 0xAA	0x06
Zero	0x00, 0x00, 0x00, 0x00, 0x00, 0x00	0x06
Constant SSL	0xAA, 0xAA, 0xAA, 0xAA, 0xAA, 0x05	0x06

Pad Length

The *Pad Alignment* table lists the alignment (boundaries) to which the packet engine will pad, based on:

- The selected crypto algorithm
- Crypto mode
- The value of the pad stream cipher bit
- The value of pad control

The minimum number of inserted pad bytes depends on the cipher algorithm, selected using the `PKTE_SA_CMD0.CIPHER` bit field, and the cipher mode, selected using the `PKTE_SA_CMD1.CIPHERMD` bit field.

For block ciphers, the plain text data is always (as a minimum) padded to the next block boundary. More pad bytes beyond the algorithm or protocol alignment requirements can be inserted using the pad control (`PKTE_CTL_STAT.PADCTLSTAT`) in the command descriptor. This feature can be used for *traffic flow security* to conceal the number of plain text bytes in an encrypted packet.

Encrypt operations that use block ciphers have minimum pad requirements based on their block size. The packet engine enforces a minimum pad alignment for block ciphers according to the *Pad Alignment* table. For ESP out-bound operations, the minimum pad alignment is forced to 4 bytes.

For stream ciphers and null crypto the data is never padded when the stream cipher pad bit (`PKTE_SA_CMD0.SCPAD`) = 0. When `PKTE_SA_CMD0.SCPAD` = 1 the plain text data is padded to the length as defined by the `PKTE_CTL_STAT.PADCTLSTAT` bits in the command descriptor.

NOTE: The SSL protocol does not allow padding to exceed the ciphers block length. This length is 8 bytes for DES/Triple-DES and 16 bytes for AES. For SSL, the packet engine does not enforce this pad alignment value. The host processor must ensure that the `PKTE_CTL_STAT.PADCTLSTAT` bit field is configured correctly.

Table 30-6: Pad Alignment

Pad Control PADCTLSTAT = PKTE_CTL_STAT[31:24]			0x00	0x01 ^{*1}	0x02	0x04	0x08	0x10	0x20	0x40	0x80 ^{*2}
Crypto Algo- rithm	Crypto Mode	Pad Stream Ciphers	%8	%1	%4	%8	%16	%32	%64	%128	%256
DES	ECB, CBC	N/A	8	8	8	8	16	32	64	128	256
AES	ECB, CBC	N/A	16	16	16	16	16	32	64	128	256
	CTR, ICM with ESP	N/A	8	4	4	8	16	32	64	128	256
	CTR, ICM no ESP	no	0	0	0	0	0	0	0	0	0
		yes	8	0	4	8	16	32	64	128	256
NULL	with ESP	N/A	8	4	4	8	16	32	64	128	256
	no ESP	no	8	0	0	0	0	0	0	0	0
		yes	8	0	4	8	16	32	64	128	256
ARC4	with ESP	N/A	8	4	4	8	16	32	64	128	256
	no ESP	no	0	0	0	0	0	0	0	0	0
		yes	8	0	4	8	16	32	64	128	256

*1 When `PKTE_CTL_STAT.PADCTLSTAT` is configured for no padding (0x01), it does not mean that no padding bytes are inserted. When PKCS#7 padding is selected, a pad length field with a value =1 is inserted. When SSL or TLS padding is selected, a pad length field with a value =0 is inserted. When IPsec padding is selected, a pad length field is forced to (`PKTE_CTL_STAT.PADCTLSTAT=0x20`). When zero pad and constant pad are selected, no pad bytes are inserted.

*2 Pad type PKCS#7 supports a maximum length of 128 pad bytes, so the packet engine overrules a 256-byte alignment (`PKTE_CTL_STAT.PADCTLSTAT=0x80`) to a 128-byte boundary (`PKTE_CTL_STAT.PADCTLSTAT=0x40`).

The packet engine does not constrain the pad type that is used for an operation; any pad type can be used for each operation. The user must be aware that some protocol specifications only allow specific pad types. The SRTP specification does not have padding defined as padding performed by RTP. The host must pad the RTP packet.

The host software can implement padding that is not supported in hardware. In this case, the host must select the *zero pad* type and set the `PKTE_CTL_STAT.PADCTLSTAT` bit field in the packet descriptor to zero (no padding). The hardware padding engine does not add any bytes. When using a block cipher, the host must insert pad bytes. Then, the data to be encrypted (plain text and pad bytes) are a multiple of the block ciphers boundary. For stream ciphers, any number of pad bytes can be added.

Pad Verification and Consumption

The packet engine can validate a pad type against the expected values. The value of the following bits controls the pad verify function:

- `PKTE_SA_CMD0.PADTYPE` and `PKTE_SA_CMD0.EXTPAD` define the type of padding
- `PKTE_SA_CMD0.CIPHER` enforces a certain pad alignment
- `PKTE_SA_CMD1.CIPHERMD` enforces a certain pad alignment
- `PKTE_SA_CMD0.SCPAD` allows stream ciphers to be padded

When packet processing is complete, the status byte in the first word of the result descriptor reports the pad verification status. Refer to the [Table 30-28 Extended Error Codes - Status Encoding](#) table.

The `PKTE_CTL_STAT.PADCTLSTAT` bits in the first word of the result descriptor return the total number of detected pad bytes and returns zero for a pad verify error.

When the IPsec pad type is selected, the `PKTE_CTL_STAT.PADVAL` bit field of the result descriptor returns the next header field. For IPsec ESP outbound operations, this field returns the decimal value 50. For IPsec inbound operations and basic inbound operations that use the IPsec pad mode, the packet engine returns the next header field it detects on the header of the innermost operation. This value is typical for the payload protocol, such as TCP or UDP. However, in bundling scenarios or in IPv6 with destination option headers, another header value could be seen. For all other inbound operations, the returned pad value is zero.

Pad verification is performed for inbound (decrypt) operations that use IPsec, TLS/DTLS or PKCS#7 pad type:

- In combination with a block cipher algorithm: DES-ECB, DES-CBC, AES-ECB, AES-CBC,
- In combination with a stream cipher and stream cipher padding `PKTE_SA_CMD0.SCPAD` enabled: AES-CTR, AES-ICM and ARC4
- In combination with null-crypto (no encryption).

Pad Types

The `PKTE_SA_CMD0.PADTYPE` bit field and the extended pad `PKTE_SA_CMD0.ETPAD` bit selects the pad type for the extended protocol group pad type. The packet engine can verify the different pad types in hardware as described in the *Pad Types* table.

The constant and zero pad types are not verified since they do not include a pad length field. The SSL pad type is not verified since it does not have a defined pattern.

Table 30-7: Pad Types

Pad Type	SA_CMD0[18, 7:6]	Description
IPSec	0b000	Verify that the pad field includes 0–255 pad bytes, followed by a correct pad length and a next header byte. Verify that pad bytes appended to the plain text are an incremental count, starting at one. Verify that the pad length field is the number of pad bytes (0–255), where a value of zero indicates no pad bytes present. Verify that a next header byte is present as the last byte of the packet, the value is not verified. This is after removal of the ICV. The total number of detected pad bytes is returned in the <code>PKTE_CTL_STAT.PADCTLSTAT</code> field in the result descriptor. NOTE: A minimum of 2 bytes must be present, zero pad bytes plus the pad length byte plus the next header byte. In this case the <code>PKTE_CTL_STAT.PADCTLSTAT</code> field in the descriptor returns 0x02. A maximum of 257 bytes can be present, in which case the <code>PKTE_CTL_STAT.PADCTLSTAT</code> field in the result descriptor returns 0x01. The value of the next header byte is returned in the <code>PKTE_CTL_STAT.PADVAL</code> field in the result descriptor.
PKCS#7	0b001	Verify that the pad field includes 1–128 pad bytes, with a pad byte value equal to the pad length (1–128).

When a block cipher is used and the payload is not padded to the nearest block size boundary, as required by the protocol, a *block size error* is generated for all pad types.

Pad Consumption

The packet engine can optionally consume the decrypted pad bytes for an inbound operation that uses the IPsec, SSL, TLS/DTLS, or PKCS#7 pad types. The pad types constant and zero are not consumed since they do not include a pad length field.

Pad consumption (or stripping) is selected on a flow-by-flow basis in the SA-record with the `PKTE_SA_CMD1.CPYPAD` bit. When this bit is set, the length returned in the `LEN` field of the result descriptor is the total length of the plain text including the pad. When the `PKTE_SA_CMD1.CPYPAD` is disabled, the detected pad length, as returned in the `PKTE_CTL_STAT.PADCTLSTAT` bits, is subtracted from the total length and then returned in the `LEN` field of the result descriptor.

The pad is always written to the result packet buffer in memory. When the `PKTE_SA_CMD1.CPYPAD` bit is disabled, only the result length is corrected.

Crypto and Hash Algorithms

The packet engine supports a wide range of crypto and hash algorithms to accelerate basic operations and protocol operations. These algorithms are:

- Basic Encrypt and Basic Decrypt Operations
- Basic Hash Operations
- Basic Encrypt-Hash and Basic Hash-Decrypt Operations
- IPSec ESP Operations
- SRTP Operations

The following tables provide allowed algorithm combinations. Those algorithms not listed in the tables are invalid and can give unexpected results.

NOTE: Not all crypto and hash algorithms are available on all product models. For information on algorithm availability, see the product-specific data sheet.

Table 30-8: Algorithms for Basic Encrypt and Basic Decrypt Operations

Crypto Algorithm	Crypto Mode
DES, Triple-DES	ECB, CBC
AES	ECB, CBC, CRT, ICM
NULL	—

Table 30-9: Algorithms for Basic Encrypt and Basic Decrypt Operations

Crypto Algorithm	Crypto Mode
SHA-1	Basic hash, HMAC, SSL-MAC
SHA-224	Basic hash, HMAC
SHA-256	Basic hash, HMAC
NULL	—

Table 30-10: Algorithms for Basic Encrypt-Hash and Basic Hash-Decrypt Operations

Crypto Algorithm	Crypto Mode	Hash Algorithm	Hash Mode
DES, Triple-DES	ECB, CBC	SHA-1	Basic hash, HMAC, SSL-MAC
		SHA-224	Basic hash, HMAC
		SHA-256	Basic hash, HMAC
		MD5	Basic hash, HMAC, SSL-MAC
		NULL	—

Table 30-10: Algorithms for Basic Encrypt-Hash and Basic Hash-Decrypt Operations (Continued)

Crypto Algorithm	Crypto Mode	Hash Algorithm	Hash Mode
AES	ECB, CBC, CRT, ICM	SHA-1	Basic hash, HMAC, SSL-MAC
		SHA-224	Basic hash, HMAC
		SHA-256	Basic hash, HMAC
		MD5	Basic hash, HMAC, SSL-MAC
		NULL	—
NULL	—	SHA-1	Basic hash, HMAC, SSL-MAC
		SHA-224	Basic hash, HMAC
		SHA-256	Basic hash, HMAC
		MD5	Basic hash, HMAC, SSL-MAC

Table 30-11: Algorithms for IPsec ESP Operations

Crypto Algorithm	Crypto Mode	Hash Algorithm	Hash Mode
DES, Triple-DES	CBC	SHA-1	HMAC
		SHA-224	HMAC
		SHA-256	HMAC
		MD5	HMAC
		NULL	—
AES	CBC, CTR	SHA-1	HMAC
		SHA-224	HMAC
		SHA-256	HMAC
		MD5	HMAC
		NULL	—
NULL	CBC	SHA-1	HMAC
		SHA-224	HMAC
		SHA-256	HMAC
		MD5	—

Table 30-12: Algorithms for Basic SSL and Extended SSL Operations

Crypto Algorithm	Crypto Mode	Hash Algorithm	Hash Mode
DES, Triple-DES	CBC	SHA-1	SSL-MAC
		MD5	SSL-MAC
		NULL	—

Table 30-12: Algorithms for Basic SSL and Extended SSL Operations (Continued)

Crypto Algorithm	Crypto Mode	Hash Algorithm	Hash Mode
AES	CBC	SHA-1	SSL-MAC
		MD5	SSL-MAC
		NULL	—
ARC4	Stateful	SHA-1	SSL-MAC
		MD5	SSL-MAC
		NULL	—
NULL	—	SHA-1	SSL-MAC
	—	MD5	SSL-MAC
	—	NULL	—

Table 30-13: Algorithms for Basic TLS, Extended TLS and DTLS Operations

Crypto Algorithm	Crypto Mode	Hash Algorithm	Hash Mode
DES, Triple-DES	CBC	SHA-1	HMAC
		SHA-224	HMAC
		SHA-256	HMAC
		MD5	HMAC
		NULL	—
AES	CBC, CTR	SHA-1	HMAC
		SHA-224	HMAC
		SHA-256	HMAC
		MD5	HMAC
		NULL	—
ARC4 ¹	Stateful	SHA-1	HMAC
		SHA-224	HMAC
		SHA-256	HMAC
		MD5	HMAC
		NULL	—
NULL	—	SHA-1	HMAC
		SHA-224	HMAC
		SHA-256	HMAC
		MD5	HMAC
		NULL	—

1 Only for Basic TLS and Extended TLS

Table 30-14: Algorithms for SRTP Operations

Crypto Algorithm	Crypto Mode	Hash Algorithm	Hash Mode
AES	ICM	SHA-1	HMAC
NULL	—	SHA-1	HMAC

IV Processing

An initialization vector (IV) is necessary to start a cipher stream or a block cipher in any of the streaming modes of operation. The IV must be unique for each packet. The IV ensures that all cipher texts are unique even if produced by the same encryption key. This functionality prevents every packet from needing a unique encryption key.

Depending on the packet engine operation, the IV can be loaded from different sources. The IV format depends on the algorithm and the source of the IV. The *Format of the IV* table provides an overview of all IV formats.

Table 30-15: Format of the IV

Algorithm	IV Source (<code>PKTE_SA_CMD0.IVSR</code>)	Format	IV Offset (<code>PKTE_SA_CMD1.HSHCOFFST</code>)
DES/Triple-DES (CBC)	Previous Result of IV	Internal IV register [63:0]	0x00
	Input Buffer	Input buffer [63:0]	0x02
	Saved IV	State Record Saved IV [63:0]	0x00
	Automatic	PRNG output [63:0]	0x00
AES (CBC)	Previous Result of IV	Internal IV register [127:0]	0x00
	Input Buffer	Input Buffer [127:0]	0x04
	Saved IV	State Record Saved IV [127:0]	0x00
	Automatic	PRNG output [127:0]	0x00
AES (ICM) for Basic and SRTP	Input Buffer	Input Buffer [127:0]	0x04
	Saved IV	State Record Saved IV [127:0]	0x00
AES (CTR) for Basic and IP-sec	Input Buffer	SA_NONCE Input Buffer [63:0] 0x00000001	0x02
	Saved IV	State Record Saved IV [127:0]	0x00
	Automatic	SA_NONCE PRNG output [95:32] 0x00000001	0x00

Notes:

1. The `PKTE_SA_CMD1.HSHCOFFST` bit field provides the IV offset. The offset is only applicable for basic hash or encrypt operations. For protocol operations, the offset is automatically enforced.

2. AES-CTR: The Nonce value as described in RFC 3686, is mapped to the `PKTE_SA_NONCE` register. This Nonce value remains constant for the lifetime of the security association.
3. The host processor controls the IV update using the save-IV bit (`PKTE_SA_CMD0.SVIV`). When part of a packet is processed using a stream cipher and the encrypt or decrypt data is not an integer multiple of the block size, the saved IV is invalid. It must not be used to resume processing the packet.
4. The packet engine supports automatic IV generation for outbound operations. A new IV is generated for every packet with the internal PRNG. This automatic IV generation can be used for all DES, Triple-DES, and AES modes that use an IV, except for AES-ICM mode.
5. For outbound operations, automatic IV generation is the most efficient. No additional I/O is required, and the host processor does not need to provide the IV. When the saved IV option supplies the IV, the IV must be changed for each packet sent. This activity happens when the packet engine writes back the IV to the state record after processing.
6. Outbound IPsec operations put the IV explicitly at the front of the packet. For an inbound IPsec operation, loading from the input buffer is most efficient and always used.
7. CBC processing must not use a predictable IV. Do not use the saved IV and previous result IV options for CBC processing. Refer to RFC 3602 for more details.

ARC4 Processing

The ARC4 algorithm supports two modes of operation: stateless and stateful. For SSL, TLS and DTLS operations that use the ARC4 algorithm, the mode must be set to stateful.

Stateless Mode. Each packet is processed with a newly initialized ARC4 key taken from the key field of the SA record. In this mode, the state information from the SA is never to be read.

CAUTION: When an ARC4 operation in stateless mode is interrupted by an *Interface Error* and the ARC4 state building process is aborted, the next packet fails. The ARC4 is not reset between two packets. For the next packet the ARC4 proceeds from the internal state that it was aborted. A soft reset *does not* reset the ARC4 internal state, a hardware reset is required.

Stateful Mode. When the `PKTE_CTL_STAT.INITARC4` bit =1, the ARC4 algorithm initializes using the key specified in the SA record. When the `PKTE_CTL_STAT.INITARC4` bit =0, the ARC4 context is read from the ARC4 state field of the SA record and the *i* and *j* pointer field of the SA record. The encrypt and decrypt processing continues from this algorithm state.

The packet engine supports ARC4 key sizes of 40 bits to 128 bits. Longer keys can be used, but they cannot be made inside the packet engine.

The host processor applies ARC4 key scheduling function to the s-box and puts the 256-byte result into the ARC4 state record. It writes the *i* and *j* pointers in the SA (initial *i* = 1, *j* = 0). The host programs the packet engine and specifies stateful operation to continue the ARC4 algorithm.

Hash State Loading

The hash state can be loaded from various sources, depending on the selected protocol and hash algorithm. The *Different Sources for Loading the Hash State* table provides a list of all the options.

Table 30-16: Different Sources for Loading the Hash State

Hash Algorithm PKTE_SA_CMD0.HASHSRC =	From SA 0b00	RESERVED 0b01	From State 0b10	No Load 0b11
SHA-1	yes	-	yes	Yes
SHA-224	Yes	-	Yes	Yes
SHA-256	Yes	-	Yes	Yes
Null hash	x	x	x	x

Sequence Number Processing

The packet engine supports sequence number generation and verification for IPsec and extended SSL, TLS, and DTLS protocol operations.

A Sequence Number (SN) is an unsigned number that a sender must implement, and a receiver can use to support anti-replay service (replay attacks) for a specific SA. This processing includes detection of the same packet that arrives more than once and detection of packets that arrive in an incorrect sequence and is outside an accepted level of order relative to the last received packet.

The packet engine supports the following options.

- Sequence number loaded from SA for outbound operations and is retrieved from the input packet for inbound operations.

The sequence number and sequence number mask fields are part of the SA record.

Sequence Number Processing in Extended SSL/TLS

SSL and TLS use an implicit sequence number of 64 bits that is not send in the packet but part of the hash.

For inbound operations, no sequence number verification is performed; instead, an incorrect sequence number results in an authentication error (PKTE_CTL_STAT.AUTHERR).

For outbound operations, with PKTE_SA_CMD0.HDRPROC enabled and PKTE_SA_CMD1.ENSQNCHK enabled, the packet engine generates a sequence number error when the 64-bit sequence number counter overflows (counter is $2^{64}-1$ and increments to 0). The host processor must not send the packet.

For both outbound and inbound operations, with PKTE_SA_CMD0.HDRPROC enabled and PKTE_SA_CMD1.ENSQNCHK enabled, the packet engine reads the sequence number from the PKTE_SA_SEQNUM[n] registers in the SA. When the operation is finished, the packet engine stores the incremented sequence number in the same SA fields. The sequence number mask PKTE_SA_SEQNUM_MSK[n] registers in the SA are not used.

For both outbound and inbound operations, with `PKTE_SA_CMD0.HDRPROC` disabled, the packet engine does not increment the sequence number. It expects the host to provide the sequence number through the input buffer as part of the header.

Sequence Number Processing in DTLS

DTLS uses an explicit sequence number of 64 bits that is sent in the packet. The DTLS sequence number is composed of the epoch (16 bits) and packet sequence number (48 bits) that together form the 64-bit number, like the TLS sequence number. The epoch is incremented after each *Change Cipher Spec* message. The packet sequence number is incremented per packet starting from zero after each change cipher spec message.

For outbound operations, with `PKTE_SA_CMD0.HDRPROC` enabled, the packet engine reads the sequence number from the `PKTE_SA_SEQNUM[n]` fields in the SA, then inserts the sequence number in the packet. Then the packet engine stores the incremented sequence number in the `PKTE_SA_SEQNUM[n]` fields in the SA. The sequence number mask `PKTE_SA_SEQNUM_MSK[n]` fields in the SA are not used.

For outbound operations, with `PKTE_SA_CMD0.HDRPROC` enabled and `PKTE_SA_CMD1.ENSQNCHK` enabled, the packet engine generates a sequence number error when the 48-bit sequence number counter overflows (counter is $2^{48} - 1$ and increments to 0). The host must not send the packet.

For outbound operations, with `PKTE_SA_CMD0.HDRPROC` disabled or `PKTE_SA_CMD1.ENSQNCHK` disabled, the packet engine does not increment and verify a sequence number counter overflow, and therefore never generates a sequence number error. With `PKTE_SA_CMD0.HDRPROC` disabled the Packet Engine does not update the sequence number and sequence number mask fields in the SA and expects the host to provide the sequence number through the input buffer as part of the header.

For inbound operations, with `PKTE_SA_CMD0.HDRPROC` enabled and `PKTE_SA_CMD1.ENSQNCHK` enabled, the packet engine verifies the `PKTE_SA_SEQNUM[n]` fields against the sequence number in the packet using the `PKTE_SA_SEQNUM_MSK[n]` from the SA. Three situations can occur:

1. If the received sequence number falls outside and above the 64-bit sequence number mask, the mask is shifted. The packet engine updates the SA with the received sequence number and the shifted sequence number mask.
2. If the received sequence number falls inside the 64-bit sequence number mask and is not a duplicate sequence number (the same as received before), the corresponding bit in the mask is set. The packet engine updates the SA with the received sequence number and the updated sequence number mask.
3. If the received sequence number falls outside the 64-bit sequence number mask or matches an earlier received number a sequence number error is generated. The packet engine does not update the sequence number and sequence number mask fields in the SA. The host must discard the packet.

For inbound operations, with `PKTE_SA_CMD0.HDRPROC` disabled or `PKTE_SA_CMD1.ENSQNCHK` disabled, the packet engine does not verify the sequence number against the sequence number in the packet and therefore never generates a sequence number error.

With `PKTE_SA_CMD0.HDRPROC` disabled the packet engine does not update the sequence number and sequence number mask fields in the SA and expects the host to provide the sequence number through the input buffer as part of the header.

The following tables provide details of sequence number processing.

Table 30-17: Sequence Number Generation and Verification Control (Outbound)

Header Processing SA_CMD0[19]	Anti-Replay Service SA_CMD1[29]	Description
Outbound		
1	1	Sequence number generation (increment)
1	1	Sequence number overflow check ($2^{32}-1$) to zero for IPsec, ($2^{48}-1$) to zero for DTLS and ($2^{64}-1$) to zero for Extended SSL/TLS.
1	0/1	Sequence number update in SA
0/1	0/1	Extended sequence number update in SA

Table 30-18: Sequence Number Generation and Verification Control (Inbound)

Header Processing SA_CMD0[19]	Anti-Replay Service SA_CMD1[29]	Description
Inbound		
1	1	Sequence number verification (check against sequence number and sequence number mask in SA). Applicable only for IPsec and DTLS.
1	0/1	Sequence number update in SA, except on authentication error
1	1	Sequence number mask update in SA. Applicable only for IPsec and DTLS.
0/1	0/1	Extended sequence number update in SA

Table 30-19: Header Processing Enabled, Anti-replay Service Enabled

	IPsec ESP	Ext. SSL	Ext. TLS	DTLS
Header processing enabled, anti-replay service enabled				
Inbound				
Initial value in the SA	B3 B0	B3 B0	B3 B0	B3 B0
SA_SEQNUM1[31:0]	0x00000000	0x00000000	0x00000000	0xAA550000
SA_SEQNUM0[31:0]	0x00000000	0x00000000	0x00000000	0x00000000
SA_SEQNUM_MSK	not used	not used	not used	not used
Value in the first packet or hashbyte stream, highest byte is B0	B0 B3 0x00000000 B4 B7 0x00000001	B0 B3 0x00000000 B4 B7 0x00000000	B0 B3 0x00000000 B4 B7 0x00000001	B0 B3 0xAA550000 B4 B7 0x00000001

Table 30-19: Header Processing Enabled, Anti-replay Service Enabled (Continued)

	IPsec ESP	Ext. SSL	Ext. TLS	DTLS
Header processing enabled, anti-replay service enabled				
Inbound				
Initial value in the SA after first packet	B3 B0	B3 B0	B3 B0	B3 B0
SA_SEQNUM1[31:0]	0x00000000	0x00000000	0x00000000	0xAA550000
SA_SEQNUM0[31:0]	0x00000001	0x00000001	0x00000001	0x00000001
SA_SEQNUM_MSK	not used	not used	not used	not used
Initial value in the SA	B3 B0	B3 B0	B3 B0	B3 B0
SA_SEQNUM1[31:0]	0xFEDCBA98	0xFEDCBA98	0xFEDCBA98	0xAA55BA98
SA_SEQNUM0[31:0]	0x76543210	0x76543210	0x76543210	0x76543210
SA_SEQNUM_MSK	not used	not used	not used	not used
Value in the first packet or hashbyte stream, highest byte is B0	B0 B3 0xFEDCBA98 B4 B7 0x76543211	B0 B3 0xFEDCBA98 B4 B7 0x76543211	B0 B3 0xFEDCBA98 B4 B7 0x76543211	B0 B3 0xAA55BA98 B4 B7 0x76543211
Initial value in the SA after the packet	B3 B0	B3 B0	B3 B0	B3 B0
SA_SEQNUM_1[31:0]	0xFEDCBA98	0xFEDCBA98	0xFEDCBA98	0xAA55BA98
SA_SEQNUM_0[31:0]	0x76543211	0x76543211	0x76543211	0x76543211
SA_SEQNUM_MASK	not used	not used	not used	not used
Highest value before overflow	B0 B3	B0 B3	B0 B3	B0 B3
SA_SEQNUM_1[31:0]	0xFFFFFFFF	0xFFFFFFFF	0xFFFFFFFF	0xAA55FFFF
SA_SEQNUM_0[31:0]	0xFFFFFFFF	0xFFFFFFFF	0xFFFFFFFF	0xFFFFFFFF
SA_SEQNUM_MASK	not used	not used	not used	not used
Value in the packet or hashbyte stream, highest byte is B0	B0 B3 0x00000000B4 B7 0x00000000	B0 B3 0xFFFFFFFF B4 B7 0xFFFFFFFF	B0 B3 0xFFFFFFFF B4 B7 0xFFFFFFFF	B0 B3 0xAA55FFFF B4 B7 0xFFFFFFFF
Value after overflow in SA	B0 B3	B0 B3	B0 B3	B0 B3
SA_SEQNUM_1[31:0]	0x00000000	0x00000000	0x00000000	0xAA550000
SA_SEQNUM_0[31:0]	0x00000000	0x00000000	0x00000000	0x00000000
SA_SEQNUM_MASK	not used	not used	not used	not used

Table 30-19: Header Processing Enabled, Anti-replay Service Enabled (Continued)

	IPsec ESP	Ext. SSL	Ext. TLS	DTLS
Header processing enabled, anti-replay service enabled				
Inbound				
Initial value in SA	B3 B0	B3 B0	B3 B0	B3 B0
SA_SEQNUM1[31:0]	0x00000000	0x00000000	0x00000000	0xAA550000
SA_SEQNUM0[31:0]	0x00000000	0x00000000	0x00000000	0x00000000
SA_SEQNUM_MSK1[31:0]	0x00000000	0x00000000	0x00000000	0x00000000
SA_SEQNUM_MSK0[31:0]	0x00000000	0x00000000	0x00000000	0x00000000
Expected value in the first packet or hash-byte stream, highest byte is B0	B0 B3 0x00000000 B4 B7 0x00000001	B0 B3 0x00000000 B4 B7 0x00000001	B0 B3 0x00000000 B4 B7 0x00000001	B0 B3 0xAA550000 B4 B7 0x00000001
Value in SA after first packet	B3 B0	B3 B0	B3 B0	B3 B0
SA_SEQNUM1[31:0]	0x00000000	0x00000000	0x00000000	0xAA550000
SA_SEQNUM0[31:0]	0x00000001	0x00000001	0x00000001	0x00000001
SA_SEQNUM_MSK1[31:0]	0x00000000	0x00000000	0x00000000	0x00000000
SA_SEQNUM_MSK0[31:0]	0x00000001	0x00000001	0x00000001	0x00000001

PKTE Block Diagram

The *PKTE Block Diagram* shows the functional blocks within the PKTE.

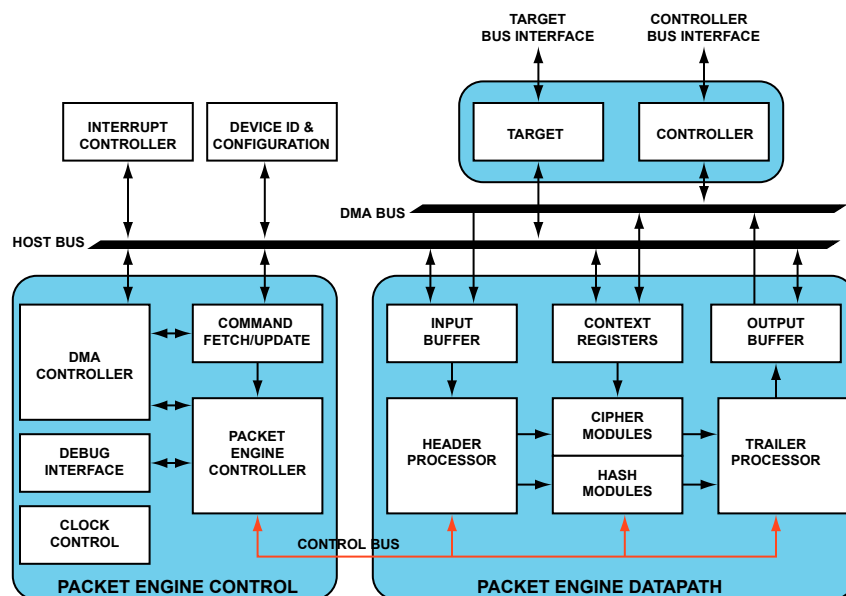


Figure 30-1: PKTE Block Diagram

PKTE Architectural Concepts

The following descriptions provide details on the functional blocks within the security packet engine.

Packet Engine

The packet engine contains symmetric cipher and hash engines. It is optimized to off-load intensive cryptographic operations from the host processor. It can perform parallel and pipelined encryption and hashing operations, reducing the latency, and processing time for packets that need both operations applied. The processor core provides the command information and packet data for the packet engine. The packet engine can run autonomously, using its local DMA controller to perform DMA transfers across the main bus to access the memory of the processor core. The DMA process incorporates flow-control to guarantee proper data flow. Two elements provide the command information that defines the processing for each packet:

- Command descriptor
- SA record

When the packet engine finishes the operation, it updates the SA record, when needed, and provides a result descriptor.

The packet engine has four different modes of operation. The modes give the processor core various levels of control over the command information and packet data transfers to and from the packet engine.

- Autonomous Ring Mode (ARM)
- Target Command Mode (with and without result descriptor ring) (TCM)
- Direct Host Mode (DHM)

Input/Output FIFO Buffers

The data for the packet engine is buffered at both input and output. These buffers decouple the DMA I/O process from the cipher and hash modules inside the packet engine. This functionality enables large DMA burst sizes and allows the crypto engines to process data during I/O latency periods. Data moves automatically from the input buffer through the encryption and hash engines to the output buffer. When the output buffer is full, the process stops until the data is read and space is available in the output buffer. Each buffer is a 256-byte dual-port RAM.

Parallel Operations

The hash functionality and encrypt or decrypt functionality are tightly coupled. Operations that combine both hash and encrypt or decrypt functions are available to reduce processing time for data that must apply both. For hash-then-decrypt operations, the packet engine performs parallel execution of both functions from the input buffer. For encrypt-then-hash operations, the processing is pipelined from the input buffer to provide minimum latency. An offset can be specified between the start of the hashing and the start of the encryption to support protocols such as IPSec and SRTP.

DMA Controller

The packet engine uses a high-performance DMA controller for autonomous data transfers for:

- Command descriptor reads
- SA record and state record reads
- Packet data read
- Result packet writes
- SA record and state record writes
- Result descriptor writes

Interrupt Controller

The packet engine includes an interrupt controller that, under programmable configuration control, can generate an interrupt on completion of certain operations. Individual interrupts can be masked and cleared. The interrupt registers show both the raw and masked interrupt status of the internal interrupts. The processor core can use interrupts, together with their associated threshold settings, to optimize the overall packet processing in the system. One interrupt can inform the processor core that the input side of the packet engine is almost empty to avoid a stall-on-empty condition. One interrupt can inform the host that the output side is almost full to avoid a stall-on-full condition. The controller uses several interrupts to inform the processor core about errors inside the packet engine. All available interrupts are combined into a single output port as either a level- or edge-active programmable interrupt output.

Clock Controller

The packet engine includes a clock controller that generates clock enable signals. A clock manager external to the packet engine uses the clock enable signals to switch the clocks to modules in the packet engine, reducing power consumption. The power saving can be significant, depending on the crypto-operation and the idle time of the packet engine. The clock controller generates the clock enable signals dynamically depending on the current crypto-operation. A clock control register provides the processor core the possibility to override this dynamic process.

PKTE Operating Modes

The packet engine can be configured in one of three command modes. For all modes the packet engine can generate an interrupt at completion of packet processing:

- **Autonomous Ring Mode (ARM)**. The core prepares descriptors in the CDR and then initiates a descriptor fetch by triggering the packet engine. When a packet operation is complete, the packet engine writes the result descriptor out into a ring in host memory using the system requester bus interface.
- **Target Command Mode (TCM)**. The core directly writes the command descriptors to the packet command register set to initiate a packet operation. This process eliminates an extra DMA transfer to fetch a descriptor, but requires the core to synchronously initiate packet processing. This mode can be configured both without RDR or with RDR. In the latter case, the packet engine writes the result descriptor out into the RDR in host memory using the system requester bus interface.
- **Direct Host Mode (DHM)**. The core has full control over the packet engine and uses the system completer bus interface. The core provides all the command descriptors, SA record and state record, and packet input data.

When the packet engine completes processing, the core must read the packet output data, the SA record, the state record, and the result descriptors.

Autonomous Ring Mode (ARM)

The *Autonomous Ring Mode* allows the packet engine and the host processor to operate asynchronously. A queue of multiple packets in the host processor memory can be processed continuously to provide the highest possible throughput. The packet engine autonomously fetches the command descriptor, the SA record, and optionally the state record and the input data from host processor memory. After the packet engine finishes processing, it autonomously writes the output data, updates the SA and state record, and writes the result descriptor in the host processor memory. It accesses the host processor memory through DMA read transfers across the system bus requester.

This mode uses both command descriptor ring (CDR) and result descriptor ring (RDR).

Physically the CDR, RDR, SA record, source packet, and result packet can all be in different memories depending on the system memory architecture. The host processor writes command descriptors to the CDR in host processor memory. Then, it writes to the `PKTE_CDSC_INCR` register with the number of command descriptors that it prepared in the CDR. This write to the `PKTE_CDSC_INCR` register is the trigger for the packet engine to fetch the command descriptors sequentially from the CDR. When a command descriptor is fetched and written to the internal packet command register set, the descriptor is validated. If the ownership bits are set for the packet engine and the command is valid, processing starts. If not, that command is discarded, and a result descriptor is written to the RDR with the error code *invalid command descriptor*. In this mode, the host processor can set a threshold on the CDR and enable an associated interrupt. The packet engine generates an interrupt when the number of command descriptors in the CDR is equal or below the threshold value.

The SA record and state record that contains the crypto context information are stored in a memory area. The packet engine autonomously accesses the memory area through DMA transfers across the system bus requester. Also, the source packet and result packet are stored in a memory area that the packet engine autonomously accesses through the same bus requester interface.

After decoding the command descriptor, the packet engine fetches the SA record and then, optionally, the state record.

Then, the source packet is fetched and stored in the input buffer. Packets less than the size of the input buffers are fetched entirely at once. Larger packets are fetched in parts that completely fill the input buffer. The packet engine initiates a new fetch each time the number of empty spaces in the input buffer reaches its threshold value. When the first packet data is available in the input buffer, the crypto engines start processing the data. After processing, the crypto engines write the result packet to the output buffer.

The packet engine writes the result packet from the output buffer to host processor memory when the number of bytes in the output buffer reaches its threshold value. Packets less than the threshold value are written entirely at once. Larger packets are written in parts that completely empty the output buffer.

The source packet data fetching, data processing, and result packet data writing are parallel processes that continue until the last result packet is written to host processor memory. Then, the packet engine optionally writes the SA record and the state record to update the crypto context information. As a final step, the packet engine writes the

result descriptor to the RDR. The host processor must either poll the RDR or wait for an interrupt from the packet engine to determine when packet processing is complete.

Target Command Mode (TCM)

This mode provides a synchronous interface between the processor core and the packet engine. The Command Descriptor Ring (CDR) is disabled and the processor core initiates packet processing by writing the command descriptor directly to the internal command descriptor MMRs of the packet engine. The Result Descriptor Ring (RDR) is optional.

- For `PKTE_CFG.MODE = 01`, the RDR is disabled and the processor core reads the result descriptor directly from the internal result descriptor register set for the packet engine.
- For `PKTE_CFG.MODE = 10`, the RDR is enabled and stored in a memory area that the packet engine can access through its requester bus interface as in autonomous ring mode.

In target command mode, the packet engine autonomously fetches the SA record, state record, source packet data as in autonomous ring mode. Also, as in ARM, after processing, the packet engine updates state fields of the SA record and state record in the host processor memory.

Direct Host Mode (DHM)

This mode provides a synchronous interface between the processor core and the packet engine. The packet engine is under full control of the processor core. The host processor writes the command descriptors, SA record, and state record directly to the packet engine registers. Then, the processor core writes the source packet data into the input buffer. When processing is complete, the processor core reads back the result packet data from the output buffer. Finally, it reads the result descriptor, the updated SA record, and state record directly from the packet engine registers.

PKTE Event Control

The following section provides information about interrupts in the PKTE module.

PKTE Interrupt Signals

The Packet Engine has an internal Interrupt Controller with 9 interrupt sources. There are 7 registers associated with the interrupt controller:

1. Interrupt Unmasked Status - [PKTE_IUMSK_STAT](#)
2. Interrupt Mask Status - [PKTE_IMSK_STAT](#)
3. Interrupt Clear Register - [PKTE_INT_CLR](#)
4. Interrupt Enable Register - [PKTE_INT_EN](#)
5. Interrupt Mask Disable - [PKTE_IMSK_DIS](#)
6. Interrupt Mask Enable - [PKTE_IMSK_EN](#)

7. Interrupt Configuration - `PKTE_INT_CFG`

The *Packet Engine Interrupt Controller Block Diagram* shows the blocks of the interrupt controller.

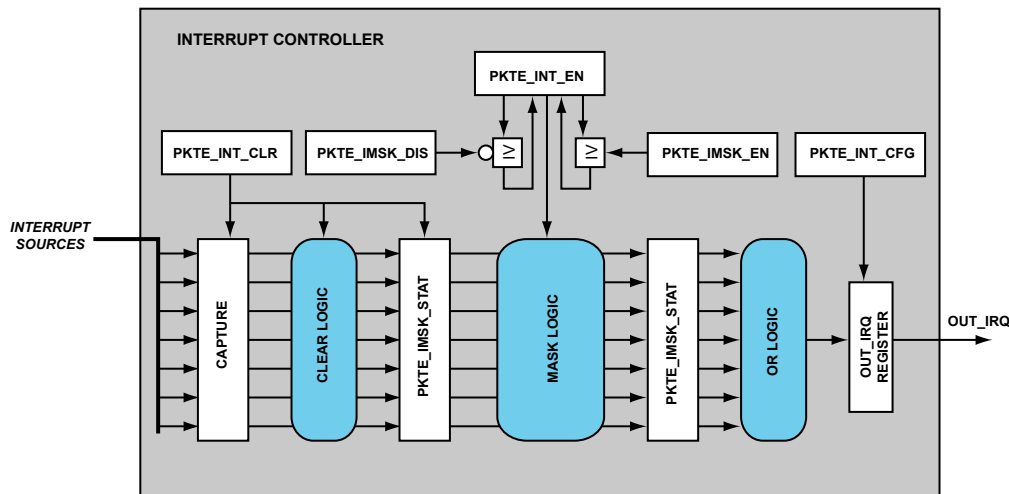


Figure 30-2: Packet Engine Interrupt Controller Block Diagram

All of the interrupt sources are pulse or level events in their native form.

These interrupts are captured and stored at their unmasked and masked status in their respective `PKTE_IUMSK_STAT` and `PKTE_IMSK_STAT` registers. This allows the host processor to read the status of any interrupt source either before or after the mask is applied.

The `PKTE_INT_EN` register provides a mask to select what interrupt source are enabled to the output interrupt request. Writing a one to the `PKTE_INT_CLR` register resets both the masked and unmasked interrupt.

The `PKTE_IMSK_EN` and `PKTE_IMSK_DIS` registers can be set to enable and disable individual interrupts respectively in the `PKTE_INT_EN` register. This avoids the need for read-modify-write operations from the host processor.

Ring Interrupts

Two interrupts are provided for efficient ring management: The CDR threshold interrupt (*cdrthrsh*) and the RDR threshold interrupt (*rdrthrsh*).

Command Descriptor Ring

The CDR threshold interrupt (*cdrthrsh*) is a level-based interrupt and connects to the threshold value in the `PKTE_RING_THRESH.CDRTHRSH` bit field. It enables the host processor to efficiently fill the CDR. The host processor writes command descriptors to the CDR with this interrupt masked until the CDR is full. Then the host processor enables the CDR threshold interrupt. When the interrupt is activated, the host processor clears the interrupt and it is guaranteed that it can put CDR threshold number of descriptors in the CDR.

Example Configuration:

```

PKTE_RING_CFG.RINGSZ=256,
PKTE_RING_THRESH.CDRTHRSH=224,
PKTE_INT_EN.CDRTHRSH=0 /*(IRQ disabled)*/

```

1. The host writes 8 Command Descriptors at once, then writes the `PKTE_CDSC_INCR` register to 8.
2. This is repeated until there are less than 8 empty entries in the CDR.
3. The fill level is now equal to the threshold (224)
4. The host enables the CDR threshold IRQ (`PKTE_INT_EN.CDRTHRSH=1`)
5. The packet engine processes packets and the fill level (`PKTE_CDSC_CNT` register) decreases.
6. Then the CDR threshold IRQ is activated as the fill level equals 224.
7. The host handles the interrupt, clears it, and continues with step #1.

Result Descriptor Ring

The RDR threshold interrupt (`rdrthrsh`) is a level-based interrupt and connects to the threshold value in the `PKTE_RING_THRESH.RDRTHRSH` bit field and the timeout value in the RD timeout (`PKTE_RING_THRESH.RDTO`) bit field. It enables the host processor to efficiently empty the RDR. The timeout reminds the host processor that when the threshold kicks in result descriptors stay long in the RDR and must be processed to reduce latency. The timeout counts when the ring is not empty, regardless of the fill level and restarts when the host processor writes the `PKTE_RDSC_CNT` register. Initially the host processor enables the RDR threshold interrupt. When the interrupt is activated the host processor reads result descriptors until the RDR is empty or contains less than the `PKTE_RING_THRESH.RDRTHRSH` number of descriptors.

Example Configuration:

```

PKTE_RING_CFG.RINGSZ=256,
PKTE_RING_THRESH.RDRTHRSH=32, timeout=1ms
PKTE_INT_EN.RDRTHRSH=1 /*(IRQ enabled)*/

```

1. The Packet Engine writes the Result Descriptor, timeout counter starts.
2. The Packet Engine writes 32 more Result Descriptors, fill level (`PKTE_RDSC_CNT` register) increases to 33.
3. The fill level exceeds threshold within 1ms, the RDR threshold IRQ is activated.
4. The host handles the interrupt, reads 8 Result Descriptors at once, then writes the `PKTE_RDSC_DECR` register with 8. The write to the `PKTE_RDSC_DECR` register restarts the timeout counter. The fill level is now under the threshold but there are still 25 descriptors left.
5. The Packet Engine writes 8 more Result Descriptors, fill level increases to 33.
6. The fill level exceeds threshold within 1 ms, the RDR threshold IRQ is activated.
7. The host handles the interrupt, reads 8 Result Descriptors at once, then writes the `PKTE_RDSC_DECR` register with 8. The write to the `PKTE_RDSC_DECR` register restarts the timeout counter. The fill level is now under the threshold but there are still 25 descriptors left.

8. After 1 ms, the timeout counter interrupt is activated.
9. The host handles the interrupt, reads 8 Result Descriptors at once, then writes the `PKTE_RDSC_DECR` register with 8. This is repeated until there are less than 8 full entries in the RDR. The fill level is now under the threshold and the RDR threshold IRQ interrupt is inactive. Each write to the `PKTE_RDSC_DECR` register restarts the timeout counter.

PKTE Programming Model

The host processor must always follow a pre-defined sequence of five phases required by the packet engine on a per packet basis when using direct host mode. The following sections describe the five phases.

Phase One. Write the Command Descriptor

1. Write the first command descriptor word with status and control information to the `PKTE_CTL_STAT` register.
2. Optionally, write the user ID to the `PKTE_USERID` register.
3. Write the last descriptor word to the `PKTE_LEN` register.
4. Write the value 0x1 to the `PKTE_CDSC_CNT` register. This operation triggers the packet engine to validate the command descriptor. When the command descriptor is invalid, an error is generated. (See the `PKTE_CTL_STAT` section in the Register Descriptions). When the command descriptor is valid, the packet engine waits for an `PKTE_SA_RDY` register write.

Phase Two. Write the State Registers, (ARC4 Buffer) and SA Registers

All required fields of the SA record and state record must be written. The fields required depend on the operation. The last field to be written is the `PKTE_SA_RDY` register. This register triggers the packet engine to start processing.

1. Write the required state record fields.
 - ARC4 state
 - IV
 - Digest count
 - State digest
2. Write the required SA record data.
3. To complete the SA record and state record, write the `PKTE_SA_RDY` register.

Phase Three. Write the Source Packet Data and Read Result Packet Data

The packet engine has input and output buffers. If a source packet is smaller than the size of the input buffer, then the packet can be written in one part. Otherwise, it must be written in multiple parts. The same applies to the output data. If the result packet size is smaller than the size of the output buffer, then the packet can be read in one part. Otherwise, it must be read in multiple parts.

NOTE: An outbound packet that is smaller than the size of the input buffer can increase in size due to padding and does not always fit in the output buffer. Conversely, an inbound packet that is larger than the size of the input buffer can decrease in size, and due to de-padding, can fit in the output buffer. If the input buffer becomes empty or the output buffer becomes full, the engine stalls.

Two following steps describe different situations:

- Source packet smaller than the size of the input buffer, start at step #1.
- Source packet larger than the size of the input buffer, start at step #3.

The host processor must follow these steps:

1. Write the source packet data. Write the full source packet to the input buffer. Go to step #4.
2. Write the input buffer count register (`PKTE_INBUF_CNT`) with the number of valid bytes that are written to the input buffer. This value must correspond to the value in the `PKTE_LEN.TOTLEN` field of the command descriptor rounded up to the next 4-byte multiple. Go to step #5 to check the packet engine status.
3. Write part of the source packet data. The `PKTE_STAT.IBUFEMPTYCNT` field indicates the amount of free space in the input buffer. Programs write the number of bytes determined by the setting in the `PKTE_BUF_THRESH` register. Write the (partial) source packet to the input buffer. The host processor must resume where it ended the previous write operation. Do not write more than the buffer size at once. Go to step 4.
4. Write the `PKTE_INBUF_CNT` register with the number of valid bytes written to the input buffer. Go to step 5 to check the packet engine status.
5. Check packet engine status. Wait for an interrupt or poll the `PKTE_STAT` register for any of the following conditions:
 - Condition 1—an error interrupt or any of the bits [7:5] in the `PKTE_STAT` register becomes active to indicate a packet processing error. Depending on the type of error, the host processor must take appropriate action. Usually, the result packet is not valid after a processing error has occurred. Go to phase 5 to read the result descriptor.
 - Condition 2—an operation done interrupt or the `PKTE_STAT.OPDN` bit becomes active (the packet engine completed processing). Go to step 8 to read the remaining output data.
 - Condition 3—an output buffer threshold interrupt or the `PKTE_STAT.OBUFREQ` bit becomes active. Go to step #6 to read a block of output data.

- Condition 4—an input buffer threshold interrupt or the `PKTE_STAT.IBUFREQ` bit becomes active. Go to step #3 to write a block of input data.
6. Read part of the output data. The `PKTE_STAT.OBUFFULLCNT` bit field indicates the number of bytes in the output buffer. This value is rounded up to full words. Programs read the number of bytes indicated in the `PKTE_BUF_THRESH` register. Read the (partial) output packet from the output buffer. The host processor must resume where it ended the previous read operation. Do not read more than the input buffer size at once. Go to step #7.
 7. Write the `PKTE_OUTBUF_CNT` register with the number of valid bytes read from the output buffer. Go to step #5 to check the packet engine status.
 8. Read the remaining output data. The `PKTE_STAT.OBUFFULLCNT` bit field indicates the number of bytes in the output buffer. This value is rounded up to words. Read the (partial) packet output data from the output buffer. The host processor must resume where it ended the previous read operation. Go to step #9.
 9. Write the `PKTE_OUTBUF_CNT` register with the number of valid bytes read from the output buffer. Go to phase four.

Phase Four. Read the Result Descriptor

1. Read the first result descriptor word from the `PKTE_CTL_STAT`.
2. Optionally read the user ID from the `PKTE_USERID` register.
3. Read the last result descriptor word from the `PKTE_LEN` register.
4. Write the value 0x1 to the `PKTE_RDSC_DECR` register. This operation allows the packet engine to accept new command descriptors. Go to phase five.

Phase Five. Read the SA Record and State Record

Depending on the operation, the SA record or state record is updated. Check the bit fields [23:16] in the `PKTE_CTL_STAT` register for the following conditions:

- Condition 1 - At least one error bit in the bit fields [23:16] of the `PKTE_CTL_STAT` register is set. Do not update the local host processor maintained version of the SA record and state record but take any required action.
- Condition 2 - None of the error bits in the bit fields [23:16] of the `PKTE_CTL_STAT` register is set, the packet is processed normally (without errors). Update the host processor maintained version of the SA record and state record with the result read from the packet engine registers:
 - ARC4 state
 - Sequence number
 - Sequence number mask
 - Result IV

- Result digest count
- Result digest

PKTE Mode Configuration

Before using the packet engine, it must be configured. The mode of the packet engine must be defined and the PRNG (if used) must be initialized.

Configure the packet engine in one of three command modes:

- Autonomous Ring Mode: `PKTE_CFG.MODE = b'11`
- Target Command Mode: `PKTE_CFG.MODE = b'10`
- Direct Host Mode: `PKTE_CFG.MODE = b'00`

PKTE Programming Concepts

The following sections provide conceptual information for programming the PKTE.

Packet Engine Descriptor

IMPORTANT: Depending on the mode, ARM, TCM, or DHM, the descriptor is either:

- in the memory of the host processor in the command descriptor ring, or
- written directly to the descriptor registers in the packet engine

References to descriptor registers are for either the register that is mirrored in the descriptor structure in memory or for the actual register itself.

Command descriptors are host-supplied commands that control the real-time operation of the packet engine. The packet engine returns result descriptors at the end of an operation that provide the status information to the host. The *Command Descriptor Structure* and the *Result Descriptor Structure* tables show these descriptors.

Table 30-20: Command Descriptor Structure

Word Off- set	31:24	23:20	19:16	15:8	7:0	Address Offset
0	Pad Control	—		Next Header/ Pad Value	Control	0x000
1	Source Address					0x004
2	Destination Address					0x008
3	SA Address					0x00C
4	SA State Address					0x010
5	Reserved/ARC4 State Address					0x014

Table 30-20: Command Descriptor Structure (Continued)

Word Off-set	31:24	23:20	19:16	15:8	7:0	Address Offset
6	User ID					0x018
7	Bypass (words)	Control	Reserved	Input Packet Length (bytes)		0x01C

Table 30-21: Result Descriptor Structure

Word Off- set	31:24	23:20		19:16	15:8	7:0	Address Offset
0	Pad Status	Status			Next Header/ Pad Value	Control	0x000
1	Source Address						0x004
2	Destination Address						0x008
3	SA Address						0x00C
4	SA State Address						0x010
5	Reserved/ARC4 State Address						0x014
6	User ID						0x018
7	Bypass (words)	Control	Reserved	Input Packet Length (bytes)			0x01C

When the packet engine is configured for autonomous ring mode, command descriptors and result descriptors reside in a ring in host memory. Command descriptors are automatically fetched from the Command Descriptor Ring (CDR) through DMA into the command descriptor registers. When an operation is complete, the result descriptors are automatically read from the packet engine and through DMA to the Result Descriptor Ring (RDR).

When the packet engine is configured for direct host mode, the host processor manually writes the command descriptor directly to the internal command descriptor MMR set. When an operation is complete, the host processor manually reads the result descriptor directly from the result descriptor MMR set.

The target command mode is a combination of the direct host mode and the autonomous ring mode. The host processor writes the command descriptor directly to the internal command descriptor register set. When an operation is complete there are two options.

1. The host processor can read the result descriptor directly from the result descriptor registers.
2. The result descriptors reside in a ring in host memory and the result descriptors are automatically DMA'd from the packet engine to the RDR.

When the host processor writes a command descriptor to the command descriptor registers, the packet engine is triggered when the host processor updates the `PKTE_CDSC_CNT` register. This functionality guarantees that all fields in the command descriptor are valid before the command is executed.

Descriptor Processing

This section describes the functional steps of the packet engine while processing the command descriptors.

Descriptor Ring Configuration

At initialization, the host processor specifies the size of the Command Descriptor Ring (CDR). The Result Descriptor Ring (RDR) has the same size.

NOTE: In some configurations, these two rings overlay each other with the results written on top of the command descriptors. This configuration is called overlaid ring mode.

When the packet engine is configured and enabled, it fetches the descriptors from the CDR using system bus requester reads.

Descriptor Ring Processing

To validate the descriptor exchange between the host processor and the packet engine, the ownership bits `PKTE_CTL_STAT.PERDY` and `PKTE_CTL_STAT.HOSTRDY` are used. One pair of ownership bits is in the first word of the descriptor (`PKTE_CTL_STAT`), and one pair is in the last word (`PKTE_LEN`). The ‘consumer’ of a descriptor must verify that both ownership pairs match to ensure that a race condition did not occur between one party writing and the other party reading the descriptor. A race condition can occur when a memory locking scheme is not used.

Each pair [`PKTE_CTL_STAT.PERDY`, `PKTE_CTL_STAT.HOSTRDY`] of ownership bits provide 3 states:

- b'00 = idle or null descriptor
- b'01 = host processor has written a descriptor in the CDR and passed ownership to the packet engine
- b'10 = packet engine processing complete: packet engine has written the descriptor in the RDR and passed ownership back to the host.
- b'11 = Reserved

At initialization, the host sets the entire CDR memory area to zero, when the CDR is used.

1. The host processor writes one or more command descriptors to the CDR. The host processor must set the `PKTE_CTL_STAT.HOSTRDY` bit to 1 and the `PKTE_CTL_STAT.PERDY` bit to 0 to indicate that ownership has passed to the packet engine. These bits are mirrored in the `PKTE_LEN` descriptor word.
2. The host processor must write the `PKTE_CDSC_INCR` register with the number of new valid command descriptors in the CDR.
3. The packet engine reads and validates one command descriptor.
4. The packet engine reads the SA record and state record, processes the packet, and updates the SA record and state record.

5. If the rings are not overlaid and the `PKTE_CFG.ENCDRUPDT` bit is 1, the packet engine writes the result descriptor to the CDR with the `PKTE_CTL_STAT.HOSTRDY` bit set to 0 and `PKTE_CTL_STAT.PERDY` bit set to 1. These bits are mirrored in the `PKTE_LEN` descriptor word.
6. The packet engine writes the result descriptor to the RDR. The packet engine sets the `PKTE_CTL_STAT.PERDY` bit to 1 and the `PKTE_CTL_STAT.HOSTRDY` bit to 0 to indicate that the ownership has passed to the host. These bits are mirrored in the `PKTE_LEN` descriptor word.
7. The packet engine decrements the value in the `PKTE_CDSC_CNT` register. If the value is not zero, the packet engine reads with the next command descriptor (step 3).
8. The packet engine increments the value in the `PKTE_RDSC_CNT` register.
9. The host processor reads one or more result descriptors from the RDR and processes the results.
10. The host processor must write the `PKTE_RDSC_DECR` register with the number of processed result descriptors in the RDR.

Descriptor Ownership

The ownership of the command descriptor and result descriptor is set by ownership bits in the first and last word of the respective descriptor, in the `PKTE_CTL_STAT` register and the `PKTE_LEN` register. For the command descriptor, it is the processor core that sets the ownership to the packet engine. For the result descriptor, it is the packet engine that sets the ownership to the host processor.

The packet engine reads the ownership bits before processing, irrespective of the mode of the packet engine. The ownership bits are used to validate and identify the descriptor. When two separate rings are used, the packet engine can be programmed to clear the ownership bits of the command descriptors in the CDR, so the host processor knows which descriptors have been processed.

NOTE: This update of the ownership bits can be disabled in the `PKTE_CFG` register when the host processor actively counts the number of descriptors in the CDR to prevent ring wrapping.

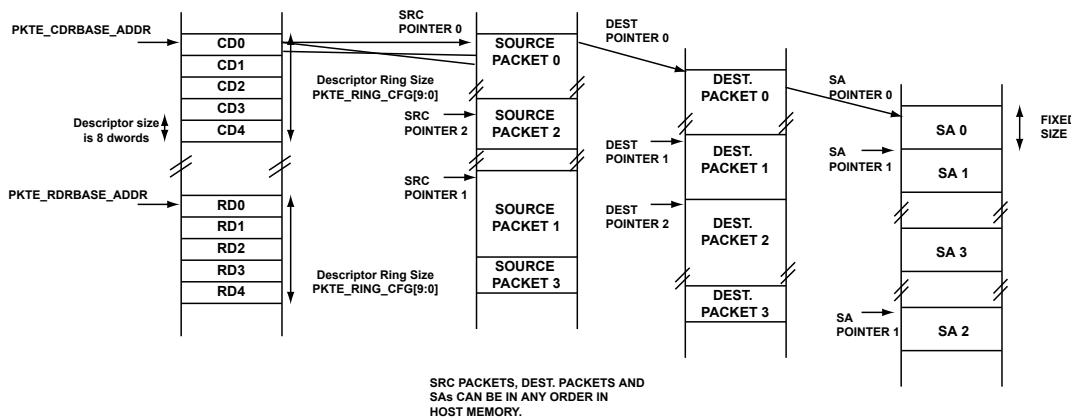


Figure 30-3: Descriptor Rings in Autonomous Ring Mode

SA Record and State Record Structure

The SA record is a packed structure that contains the remainder of the information needed by the packet engine to process a packet. Most of the information fields in the SA record, such as the key and encryption mode, are static for the lifetime of the association. The fields do not require frequent manipulation by the host processor. The SA record non-static fields are the sequence number and sequence number mask.

The SA record can have a corresponding state record that is used to save results from the current operations that can be used for future operations. The state record can hold the IV, the hash byte count, and the intermediate hash digest.

If an SA record is used for operations that use ARC4 processing in stateful mode, it has a corresponding ARC4 state record that holds the ARC4 State.

In this manual, the state record and the ARC4 state record are referred to as state record.

There is no practical limit to how many SA records and corresponding state records the packet engine can support.

In the autonomous ring mode and target command mode, once the packet engine has validated a command descriptor, it automatically fetches the SA record and optional state record. After processing, the packet engine updates the stateful fields in the SA record and state record in the host processor memory.

In direct host mode, after the descriptor is validated, the host must write the SA record directly into the internal registers of the packet engine. After processing, the host reads the stateful fields from the SA registers of the packet engine and saves them back to the SA record in the host processor memory.

SA Record Structure

The *SA Record Structure* table shows the structure for an SA Record. When using direct host mode, the corresponding elements are accessed directly with the registers. When using autonomous ring mode or target command mode, the SA Record is defined, configured, and accessed in host memory.

Table 30-22: SA Record Structure

Word Offset	Description (name)	Use
0	PKTE_SA_CMD0[31:0]	SA Control word 0 (all operations)
1	PKTE_SA_CMD1[31:0]	SA Control word 1 (all operations)
2	PKTE_SA_KEY0[31:0]	Key word (DES, Triple-DES, AES-128/192/256,)
3	PKTE_SA_KEY1[63:32]	Key word (DES, Triple-DES, AES-128/192/256,)
4	PKTE_SA_KEY2[95:64]	Key word (Triple-DES, AES-128/192/256,)

Table 30-22: SA Record Structure (Continued)

Word Offset	Description (name)	Use
5	PKTE_SA_KEY3[127:96]	Key word (Triple-DES, AES-128/192/256,)
6	PKTE_SA_KEY4[159:128]	Key word (Triple-DES, AES-192/256)
7	PKTE_SA_KEY5[191:160]	Key word (Triple-DES, AES-192/256)
8	PKTE_SA_KEY6[223:192]	Key word (AES-256)
9	PKTE_SA_KEY7[255:224]	Key word (AES-256)
10	PKTE_SA_IDIGEST0[31:0]	Inner Hash digest (Basic Hash and HMAC with SHA-1, SHA-224, SHA-256)
11	PKTE_SA_IDIGEST1[63:32]	Inner Hash digest (Basic Hash and HMAC with SHA-1, SHA-224, SHA-256)
12	PKTE_SA_IDIGEST2[95:64]	Inner Hash digest (Basic Hash and HMAC with SHA-1, SHA-224, SHA-256)
13	PKTE_SA_IDIGEST3[127:96]	Inner Hash digest (Basic Hash and HMAC with SHA-1, SHA-224, SHA-256)
14	PKTE_SA_IDIGEST4[159:128]	Inner Hash digest (Basic Hash and HMAC with SHA-1, SHA-224, SHA-256)
15	PKTE_SA_IDIGEST5[191:160]	Inner Hash digest (Basic Hash and HMAC with SHA-224, SHA-256)
16	PKTE_SA_IDIGEST6[223:192]	Inner Hash digest (Basic Hash and HMAC with SHA-224, SHA-256)
17	PKTE_SA_IDIGEST7[255:224]	Inner Hash digest (Basic Hash and HMAC with SHA-256)
18	PKTE_SA_ODIGEST0[31:0]	Outer Hash digest (HMAC with SHA-1, SHA-224, SHA-256)

Table 30-22: SA Record Structure (Continued)

Word Offset	Description (name)	Use
19	PKTE_SA_ODIGEST1[63:32]	Outer Hash digest (HMAC with SHA-1, SHA-224, SHA-256)
20	PKTE_SA_ODIGEST2[95:64]	Outer Hash digest (HMAC with SHA-1, SHA-224, SHA-256)
21	PKTE_SA_ODIGEST3[127:96]	Outer Hash digest (HMAC with SHA-1, SHA-224, SHA-256)
22	PKTE_SA_ODIGEST4[159:128]	Outer Hash digest (HMAC with SHA-1, SHA-224, SHA-256)
23	PKTE_SA_ODIGEST5[191:160]	Outer Hash digest (HMAC with SHA-224, SHA-256)
24	PKTE_SA_ODIGEST6[223:192]	Outer Hash digest (HMAC with SHA-224, SHA-256)
25	PKTE_SA_ODIGEST7[255:224]	Outer Hash digest (HMAC with SHA-256)
26	PKTE_SA_SPI[31:0]	SPI (IPsec), Type[23:16] / Version [15:0] (SSL, TLS, DTLS)
27	PKTE_SA_SEQNUM0[31:0]	Sequence Number (IPsec, SSL, TLS, DTLS with Header Processing)
28	PKTE_SA_SEQNUM1[63:32]	
29	PKTE_SA_SEQNUM_MSK0[31:0]	Sequence Number Mask (IPsec, DTLS inbound with Header Processing)
30	PKTE_SA_SEQNUM_MSK1[63:32]	
31	PKTE_SA_NONCE[31:0] / PKTE_SA_READY	Nonce value (AES-CTR, AES-ICM)/ i and j pointers ()/SA ready indicator (Direct Host Mode)

Some of these fields may be updated by the packet engine. These include:

- PKTE_SA_SEQNUM0
- PKTE_SA_SEQNUM1
- PKTE_SA_SEQNUM_MSK0
- PKTE_SA_SEQNUM_MSK1

All the other fields remain unchanged.

SA State Structure

The security association state structure contains information that may be updated after each packet, such as the IV and the intermediate hash result. The *SA State Structure* table shows the SA state structure and usage. In direct host mode, the elements are accessed directly using the PKTE registers. In target command mode and autonomous ring mode, this structure is defined and updated in host memory.

Table 30-23: SA State Structure

Word Offset	Description (name)	Use
0	PKTE_STATE_IV0[31:0]	Initialization Vector (DES, Triple DES, AES)
1	PKTE_STATE_IV1[63:32]	Initialization Vector (DES, Triple DES, AES)
2	PKTE_STATE_IV2[95:64]	Initialization Vector (AES)
3	PKTE_STATE_IV3[127:96]	Initialization Vector (AES)
4	PKTE_STATE_BYTE_CNT0[31:0]	Current hash byte count (MD5, SHA-1, SHA-224, SHA-256)
5	PKTE_STATE_BYTE_CNT1[63:32]	Current hash byte count (MD5, SHA-1, SHA-224, SHA-256)
6	PKTE_STATE_IDIGEST0[31:0]	Inner Hash digest (mirror of PKTE_SA_IDIGEST0)
7	PKTE_STATE_IDIGEST1[63:32]	Inner Hash digest (mirror of PKTE_SA_IDIGEST1)
8	PKTE_STATE_IDIGEST2[95:64]	Inner Hash digest (mirror of PKTE_SA_IDIGEST2)
9	PKTE_STATE_IDIGEST3[127:96]	Inner Hash digest (mirror of PKTE_SA_IDIGEST3)
10	PKTE_STATE_IDIGEST4[159:128]	Inner Hash digest (mirror of PKTE_SA_IDIGEST4)
11	PKTE_STATE_IDIGEST5[191:160]	Inner Hash digest (mirror of PKTE_SA_IDIGEST5)
12	PKTE_STATE_IDIGEST6[223:192]	Inner Hash digest (mirror of PKTE_SA_IDIGEST6)
13	PKTE_STATE_IDIGEST7[255:224]	Inner Hash digest (mirror of PKTE_SA_IDIGEST7)

ARC4 State Structure

The *ARC4 State Structure* table describes the state structure used with ARC4. When using the PKTE in direct host mode, these fields are accessed with the registers, starting at the value in the [PKTE_ARC4STATE_BUF](#) register. When using the PKTE in autonomous ring mode or target command mode, these fields are defined and accessed in a structure in host memory.

Table 30-24: ARC4 State Structure

Word Offset	Description (name) ^{*1}	Use
0	PKTE_ARC4_STATE0[3:0] ^{*2}	ARC4 (Basic, SSL and TLS)
1	PKTE_ARC4_STATE1[7:4]	
...	...	
62	PKTE_ARC4_STATE2[251:248]	
63	PKTE_ARC4_STATE3[255:252]	

*1 The indices in these fields indicate bytes.

*2 There are no corresponding named registers for these fields. PKTE_ARC4_STATE0 corresponds to [PKTE_ARC4STATE_BUF](#) and PKTE_ARC4_STATE1 corresponds to the 32-bit register following [PKTE_ARC4STATE_BUF](#) and so on.

Configuring Operations in the PKTE

The operation (cipher, hash function, and others) that the PKTE performs is configured primarily in the [PKTE_SA_CMD0](#) register. The following sections include a series of tables to help configure the least significant 16 bits of the [PKTE_SA_CMD0](#) register. These fields include:

- The operation code field (PKTE_SA_CMD0.OPCD)
- The direction field (PKTE_SA_CMD0.DIR)
- The operation group field (PKTE_SA_CMD0.OPGRP)
- The padding type (PKTE_SA_CMD0.PADTYPE)
- The cipher selection (PKTE_SA_CMD0.CIPHER)
- The hash selection (PKTE_SA_CMD0.HASH)

Basic Operations and Decoding

Table 30-25: Basic Operation Decoding

Outbound				Inbound			
OpGroup	Dir	OpCode	Operation	OpGroup	Dir	OpCode	Operation
0b00	0	0b000	Encrypt	0b00	1	0b000	Decrypt
0b00	0	0b001	Encrypt - Hash	0b00	1	0b001	Hash - Decrypt
0b00	0	0b010	Reserved	0b00	1	0b010	Reserved
0b00	0	0b011	Hash	0b00	1	0b011	Hash
0b00	0	0b100... 0b110	Reserved	0b00	1	0b100... 0b110	Reserved
0b00	0	0b111	PRNG	0b00	1	0b111	Reserved

Table 30-26: Protocol Operation Decoding

Outbound				Inbound			
OpGroup	Dir	OpCode	Operation	OpGroup	Dir	OpCode	Operation
0b01	0	0b000	ESP Outbound	0b01	1	0b000	ESP Inbound
0b01	0	0b001... 0b011	Reserved	0b01	1	0b001	Reserved
0b01	0	0b100	Basic SSL Out-bound	0b01	1	0b010	Basic SSL In-bound
0b01	0	0b101	Basic TLS Out-bound	0b01	1	0b011	Basic TLS In-bound
0b01	0	0b110	Reserved	0b01	1	0b100... 0b110	Reserved
0b01	0	0b111	SRTP Out-bound	0b01	1	0b111	SRTP Inbound

NOTE: For SSL/TLS and SRTP, no header processing is performed in hardware.

Table 30-27: Extended Protocol Operation Decoding

Outbound				Inbound			
OpGroup	Dir	OpCode	Operation	OpGroup	Dir	OpCode	Operation
0b11 0	0	0b000	Reserved	0b11	1	0b000	Reserved
0b11 0	0	0b001	DTLS Out-bound	0b11	1	0b001	DTLS Inbound
0b11 0	0	0b010... 0b011	Reserved	0b11	1	0b010... 0b011	Reserved
0b11 0	0	0b100	Ext. SSL Out-bound	0b11	1	0b100	Ext. SSL In-bound
0b11 0	0	0b101	Ext. TLS v1.0 Outbound	0b11	1	0b101	Ext. TLS v1.0 Inbound
0b11 0	0	0b110	Ext. TLS v1.1 Outbound	0b11	1	0b110	Ext. TLS v1.1 Inbound
0b11 0	0	0b111	Reserved	0b11	1	0b111	Reserved

Error Code Description

The `PKTE_CTL_STAT` register is used to configure the packet engine for processing in Direct Host Mode (DHM) or Target Command Mode (TCM). The `PKTE_CTL_STAT` structure element in memory is used when the packet

engine is configured for Autonomous Ring Mode (ARM). In both cases, when an operation is started, errors are reported in the status field (bits [23:16]) of this register or structure element. The *Extended Error Codes - Status Encoding* table provides a guide on how to decipher the meaning of the bits that are set when an error occurs.

Extended Error Codes

The following table provides information about the extended errors associated with the PKTE module.

Table 30-28: Extended Error Codes - Status Encoding

STATUS bits [23:16]	Hex Value	Priority	Description	Processing Result
0b0000_0000	0x00	NA	Successful completion. No errors occurred during processing of the packet.	Packet fully processed
0b----_---1	0x-1	NA	Authentication Error. For an inbound IPsec ESP operation, the Integrity Check Value (ICV) does not match the computed value. For an inbound SRTP operation, the authentication tag does not match the computed value. For a basic SSL/TLS, Extended SSL/TLS or DTLS operation the Message Authentication Code (MAC) does not match the computed value.	Packet fully processed
0b----_--1-	0x-2	NA	Pad Verify Error. For inbound operations that use pad type Constant TLS, IPsec or PKCS#7, the decrypted pad does not match the expected values for the selected pad type.	Packet fully processed
0b----_-1--	0x-4	NA	Sequence Number Error. For an inbound IPsec or DTLS operation, there was a fault in the Anti-Replay Sequence Number. For an outbound IPsec packet, the sequence number overflows; count is $2^{32}-1$ and increments to 0. For an outbound DTLS operation, the sequence number overflows; count is $2^{48}-1$ and increments to 0. For an outbound SSL or TLS operation, the sequence number overflows; count is $2^{64}-1$ and increments to 0.	Packet fully processed
0b0000_1---	0x08	1	System Bus error. The requester bus interface generates an error due to ERROR response from system completer. The completer bus interface generates an error due to request for non-word (32-bit) access.	Packet is stopped. The host must reject the packet and apply a hardware reset to the system.

Table 30-28: Extended Error Codes - Status Encoding (Continued)

STATUS bits [23:16]	Hex Value	Priority	Description	Processing Result
0b0001_1---	0x18	2	Invalid Command Descriptor Error. The ownership bits in the command descriptor are not set to the packet engine, after the <code>PKTE_CDSC_CNT</code> register is incremented.	Command descriptor is ignored, no packet is processed. The packet must be re-queued or discarded.
0b0010_1---	0x28	3	Invalid Crypto Operation Error. A reserved operation is selected.	The SA record is ignored, no packet is processed. The packet must be re-queued or discarded.
0b0011_1---	0x38	4	Invalid Crypto Algorithm Error. A reserved cipher is selected, refer to <code>PKTE_SA_CMD0.CIPHER</code> . A reserved hash is selected, refer to <code>PKTE_SA_CMD0.HASH</code> .	The SA record is ignored, no packet is processed. The packet must be re-queued or discarded.
0b0100_1---	0x48	5	SPI Error. On an inbound packet, the 32-bit SPI value in the packet does not match the value in the SA while header processing is enabled. Note: A failure caused by an SPI mismatch, in general should not occur because the host checks the SPI and does not send an incorrect SPI to the packet engine.	Packet is fully processed. The host must reject the packet.
0b0101_1---	0x58	3	Zero Length Error. The packet length defined in the command descriptor <code>PKTE_LEN.TOTLEN</code> is zero, which is illegal.	Packet command is ignored, no packet processed. The host must reject the packet.

Table 30-28: Extended Error Codes - Status Encoding (Continued)

STATUS bits [23:16]	Hex Value	Priority	Description	Processing Result
0b0110_1xxx	0x68	6	Invalid Packet Length Error. For Basic Encrypt-Hash and Hash-Decrypt operations: $PKTE_LEN.TOTLEN < \text{Hash/Encrypt Offset}$ For IPsec ESP inbound operations: $PKTE_LEN.TOTLEN < \text{ICV length}$ or $PKTE_LEN.TOTLEN$ is non-4 byte aligned For SRTP inbound operations: $PKTE_LEN.TOTLEN \leq \text{IV (opt.)} + \text{Bypass Offset} + \text{ROC}$ For SSL inbound operations: $PKTE_LEN.TOTLEN \leq 1$ or packet length > 65535 bytes (SSL packet-bypass length) For TLS and DTLS inbound operations: $PKTE_LEN.TOTLEN \leq 13$ or payload length > 65535 bytes (data to be hashed) Note: For IPsec ESP the ICV is stripped before the length is checked.	Packet processing is stopped Result packet length is zero. The host must reject the packet and apply a software reset of the packet engine.
0b0111_1xxx	0x78	7	Block Size Error. The length of the inbound packet defined in the Command Descriptor $PKTE_LEN.TOTLEN$ is not a multiple of the DES or AES block cipher length. For outbound packets the size is always automatically aligned (padded) to the correct block size. The hashed packet length is not a multiple of the hash block size for intermediate hash operation. For a final hash operation no error is generated. Note: For IPsec ESP operations the ICV is stripped before the block size is checked.	Packet is fully processed. The host must reject the packet.
0b1000_1xxx	0x88	8	Processing Error. The number of bytes in the input buffer is more than defined in the $PKTE_LEN.TOTLEN$ field. The number of bytes written to the output buffer is less than processed in the datapath.	Packet processing stops. Result packet length is zero. The host must reject the packet and apply a software reset.
0b1010_1xxx 0b1111_1xxx	Reserved			

Number Format

When dealing with cryptographic functions, data and keys are large vectors. For instance, AES supports keys of sizes 128, 192, and 256 bits. When a key needs to be loaded or read, multiple 32-bit key registers are used, namely

`PKTE_SA_KEY[n]` registers. The first key register `PKTE_SA_KEY[n] 0` holds bits 31:0, and `PKTE_SA_KEY[n] 1` holds the next thirty two bits 63:32, and so on.

Generally, for large vectors defined as Byte0, Byte1, Byte2, Byte3 and so on, the values are stored in the PKTE registers as `PKTE_REG0 = Byte3Byte2Byte1Byte0`, followed by `PKTE_REG1 = Byte7Byte6Byte5Byte4` and so on.

PKTE Programming Examples

Use these examples to extend your understanding of PKTE features, operating modes, event control, and programming modes.

Calculating SHA in Direct Host Mode

This section describes how to configure the packet engine to calculate a hash digest using one of supported SHA algorithms in direct host mode. This configuration follows the procedure outlined in the *PKTE Programming Model* section.

1. Configure the packet engine for direct host mode by setting the `PKTE_CFG.MODE` bit =0
2. Set the ownership back to the packet engine to process a command descriptor by setting the `PKTE_CTL_STAT.PERDY` bit =0 and the `PKTE_CTL_STAT.HOSTRDY` bit =1.
3. Set the `PKTE_CTL_STAT.HASHFINAL` bit to indicate this command descriptor handles all the input data for the hash calculation. This configuration is needed for the packet engine because the last block requires special handling (see FIPS 180-4 for details).
4. Set size of the input data in bytes in the `PKTE_LEN.TOTLEN` bit field.
5. Also set the `PKTE_LEN.PEDONE` bit =0 and the `PKTE_LEN.HSTRDY` bit =1. These bits must be the same as the `PKTE_CTL_STAT.PERDY` and `PKTE_CTL_STAT.HOSTRDY` bits to guarantee ownership.
6. Set the `PKTE_CDSC_CNT` register =1 to trigger the packet engine to start validating the command descriptor. In this case, the `PKTE_CTL_STAT`, `PKTE_LEN` and `PKTE_CDSC_CNT` registers are the only command descriptor registers modified.
7. Configure the `PKTE_SA_CMD0` and `PKTE_SA_CMD1` registers to define the operation. For an SHA, set the `PKTE_SA_CMD0.OPCD` bit field =0b011 for hash operation and the `PKTE_SA_CMD0.OPGRP` bit field =0b00 for basic operation.
8. Select the specific SHA function using the `PKTE_SA_CMD0.HASH` bit field as follows.
 - For SHA-1, `PKTE_SA_CMD0.HASH` =0b0001
 - SHA-224, `PKTE_SA_CMD0.HASH` =0b0010
 - for SHA-256, `PKTE_SA_CMD0.HASH` =0b0011
9. Depending on the SHA selected, the appropriate digest length must be chosen for the `PKTE_SA_CMD0.DIGESTLEN` bit field as follows.

- For SHA-1, `PKTE_SA_CMD0.DIGESTLEN = 0b0101` (5 words)
 - For SHA-224, `PKTE_SA_CMD0.DIGESTLEN = 0b0111` (7 words)
 - For SHA-256, `PKTE_SA_CMD0.DIGESTLEN = 0b1000` (8 words)
10. The SHA specifies initial constants. These constants can be pre-loaded or read from memory. In this example, by setting the `PKTE_SA_CMD0.HASHSRC` bit field = 0b11, the packet engine provides the correct initial constants depending on the SHA chosen.
 11. Next, set the `PKTE_SA_CMD1.CPYDGST` bit = 1 and `PKTE_SA_CMD1.CPYPAD` bit = 1 to move the result to the output buffer of the packet engine at the `PKTE_DATAIO_BUF` location.
 12. At this point, write to the `PKTE_SA_RDY` register with any value to trigger the operation.
 13. Start writing the input to the data buffer of the packet engine starting at the `PKTE_DATAIO_BUF` location.
 14. Write the `PKTE_INBUF_CNT` register with the length of the input rounded up to the next multiple of 4. For example, if the input length is 30 bytes, set this register to 32.
 15. Poll the `PKTE_STAT` register to see if any errors occurred or if the operation completed without errors.

Once the operation is done, the digest is available in the packet engine data I/O buffer.

NOTE: The input data or message is input into the packet engine data buffer in big endian format while the result or digest is little endian format.

Performing AES Decryption in Direct Host Mode

This section describes how to configure the packet engine to decrypt using AES-128 in direct host mode. This configuration follows the procedure outlined in the *PKTE Programming Model* section.

1. Configure the packet engine for direct host mode by setting the `PKTE_CFG.MODE` bit = 0
2. Start configuring the command descriptor registers. Set the ownership back to the packet engine to process a command descriptor by setting the `PKTE_CTL_STAT.PERDY` bit = 0 and the `PKTE_CTL_STAT.HOSTRDY` bit = 1.
3. Next, configure the `PKTE_LEN.TOTLEN` bit field with the size of the packet or message to decrypt. If the entire input message (cipher text) fits into the 256-byte data I/O buffer of the packet engine, the process can be done in one shot.
4. Set the `PKTE_LEN.PEDONE` bit = 0 and the `PKTE_LEN.HSTRDY` bit = 1. These bits must have the same setting as the `PKTE_CTL_STAT.PERDY` and `PKTE_CTL_STAT.HOSTRDY` bits to guarantee ownership.
5. Set the `PKTE_CDSC_CNT` register = 1 to trigger the packet engine to start validating the command descriptor. In this case, the `PKTE_CTL_STAT`, `PKTE_LEN`, and `PKTE_CDSC_CNT` registers are the only command descriptor registers modified.
6. Next, configure the `PKTE_SA_CMD0` and `PKTE_SA_CMD1` registers to define the operation.

- For a AES decrypt inbound cipher operation, set the `PKTE_SA_CMD0.OPCD` bit field =0b000 and the `PKTE_SA_CMD0.DIR` bit field =0b1.
 - Set the `PKTE_SA_CMD0.OPGRP` bit field =0b00 for basic operation.
 - To choose the AES cipher, set the `PKTE_SA_CMD0.CIPHER` bit field =0b0011. Set the `PKTE_SA_CMD0.HASH` bit field to 0b1111 to choose the NULL function.
7. Next, set the `PKTE_SA_CMD1.AESKEYLEN` bit field to select the appropriate key length. In this case, setting it to 0b10 select 128 bits. Also, set `PKTE_SA_CMD1.CIPHERMD` bit field to select the mode. In this case, setting it to 0b01 select CBC mode.
 8. Continue configuring the Security Association (SA) record by loading the key in the `PKTE_SA_KEY[n]` registers.
 9. Next load the initialization vector in the SA state registers (`PKTE_STATE_IV[n]`).
 10. Finally, write anything in to the `PKTE_SA_RDY` register to trigger the operation.
 11. The input data can now be written into the data I/O buffer starting at `PKTE_DATAIO_BUF`.
 12. After the data is written, write the length (or next multiple of 4) into `PKTE_INBUF_CNT` register.
 13. Poll `PKTE_STAT` to see if any errors occurred or if the operation completed without errors.

Once the operation is done, the result can be found in the same data I/O buffer.

ADSP-21560_21561_21564_21568 PKTE Register Descriptions

Security Packet Engine (PKTE) contains the following registers.

Table 30-29: ADSP-21560_21561_21564_21568 PKTE Register List

Name	Description
<code>PKTE_ARC4STATE_ADDR</code>	Packet Engine ARC4 State Record Address
<code>PKTE_ARC4STATE_BUF</code>	Starting Entry of 256-byte ARC4 State Buffer
<code>PKTE_BUF_PTR</code>	Packet Engine Buffer Pointer Register
<code>PKTE_BUF_THRESH</code>	Packet Engine Buffer Threshold Register
<code>PKTE_CDRBASE_ADDR</code>	Packet Engine Command Descriptor Ring Base Address
<code>PKTE_CDSC_CNT</code>	Packet Engine Command Descriptor Count Register
<code>PKTE_CDSC_INCR</code>	Packet Engine Command Descriptor Count Increment Register
<code>PKTE_CFG</code>	Packet Engine Configuration Register
<code>PKTE_CLK_CTL</code>	PE Clock Control Register
<code>PKTE_CONT</code>	PKTE Continue Register
<code>PKTE_CTL_STAT</code>	Packet Engine Control Register

Table 30-29: ADSP-21560_21561_21564_21568 PKTE Register List (Continued)

Name	Description
PKTE_DATAIO_BUF	Starting Entry of 256-byte Data Input/Output Buffer
PKTE_DEST_ADDR	Packet Engine Destination Address
PKTE_DMA_CFG	Packet Engine DMA Configuration Register
PKTE_ENDIAN_CFG	Packet Engine Endian Configuration Register
PKTE_HLT_CTL	Packet Engine Halt Control Register
PKTE_HLT_STAT	Packet Engine Halt Status Register
PKTE_IMSK_DIS	Interrupt Mask Disable Register
PKTE_IMSK_EN	Interrupt Mask Enable Register
PKTE_IMSK_STAT	Interrupt Masked Status Register
PKTE_INBUF_CNT	Packet Engine Input Buffer Count Register
PKTE_INBUF_INCR	Packet Engine Input Buffer Count Increment Register
PKTE_INT_CFG	Interrupt Configuration Register
PKTE_INT_CLR	Interrupt Clear Register
PKTE_INT_EN	Interrupt Enable Register
PKTE_IUMSK_STAT	Interrupt Unmasked Status Register
PKTE_LEN	Packet Engine Length Register
PKTE_OUTBUF_CNT	Packet Engine Output Buffer Count Register
PKTE_OUTBUF_DECR	Packet Engine Output Buffer Count Decrement Register
PKTE_RDRBASE_ADDR	Packet Engine Result Descriptor Ring Base Address
PKTE_RDSC_CNT	Packet Engine Result Descriptor Count Registers
PKTE_RDSC_DECR	Packet Engine Result Descriptor Count Decrement Registers
PKTE_RING_CFG	Packet Engine Ring Configuration
PKTE_RING_PTR	Packet Engine Ring Pointer Status
PKTE_RING_STAT	Packet Engine Ring Status
PKTE_RING_THRESH	Packet Engine Ring Threshold Registers
PKTE_SA_ADDR	Packet Engine SA Address
PKTE_SA_ARC4IJPTR	ARC4 i and j Pointer Register
PKTE_SA_CMD0	SA Command 0
PKTE_SA_CMD1	SA Command 1
PKTE_SA_IDIGEST[n]	SA Inner Hash Digest Registers
PKTE_SA_KEY[n]	SA Key Registers

Table 30-29: ADSP-21560_21561_21564_21568 PKTE Register List (Continued)

Name	Description
PKTE_SA_NONCE	SA Initialization Vector Register
PKTE_SA_ODIGEST[n]	SA Outer Hash Digest Registers
PKTE_SA_RDY	SA Ready Indicator
PKTE_SA_SEQNUM[n]	SA Sequence Number Register
PKTE_SA_SEQNUM_MSK[n]	SA Sequence Number Mask Registers
PKTE_SA_SPI	SA SPI Register
PKTE_SRC_ADDR	Packet Engine Source Address
PKTE_STAT	Packet Engine Status Register
PKTE_STATE_ADDR	Packet Engine State Record Address
PKTE_STATE_BYTE_CNT[n]	State Hash Byte Count Registers
PKTE_STATE_IDIGEST[n]	State Inner Digest Registers
PKTE_STATE_IV[n]	State Initialization Vector Registers
PKTE_USERID	Packet Engine User ID

Packet Engine ARC4 State Record Address

The `PKTE_ARC4STATE_ADDR` register holds the start address of the SA ARC4 state record.

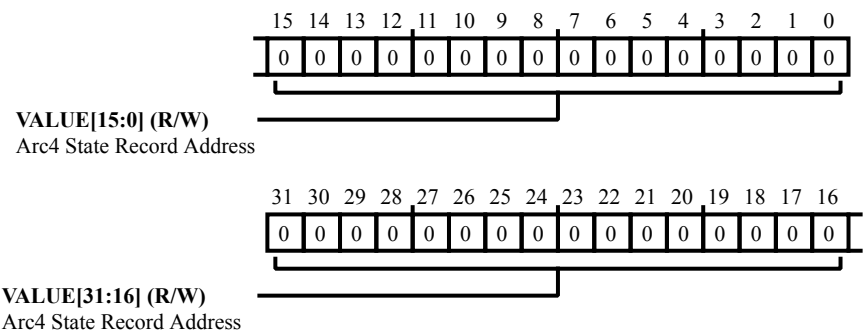


Figure 30-4: `PKTE_ARC4STATE_ADDR` Register Diagram

Table 30-30: `PKTE_ARC4STATE_ADDR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Arc4 State Record Address.

Starting Entry of 256-byte ARC4 State Buffer

The `PKTE_ARC4STATE_BUF` register is used to store the pre-processed key that initializes the ARC4 module. In direct host mode, before processing starts, the Host must write the ARC4 state, starting from the base address and increment the address pointer for each write. When processing completes the Host must read the ARC4 state and copy it to the local Host maintained state record. After a reset, a read from any address in the address range of the ARC4 buffer returns an undefined value.

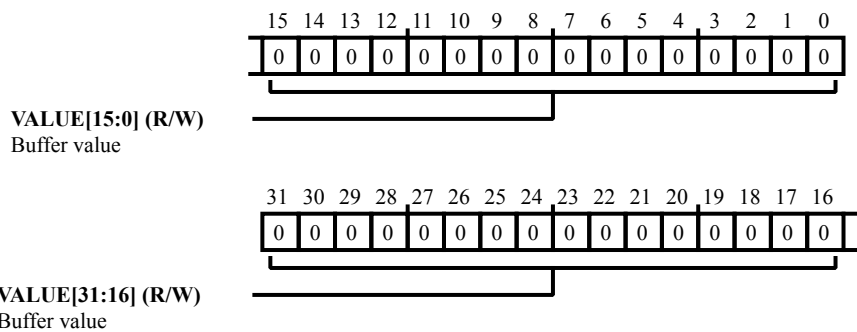


Figure 30-5: `PKTE_ARC4STATE_BUF` Register Diagram

Table 30-31: `PKTE_ARC4STATE_BUF` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Buffer value. The <code>PKTE_ARC4STATE_BUF.VALUE</code> bit field stores the pre-processed key that initializes the ARC4 module.

Packet Engine Buffer Pointer Register

The `PKTE_BUF_PTR` register contains the offset of the next buffer address (entry) to be read or written by the packet engine. This register is used in direct host mode only.

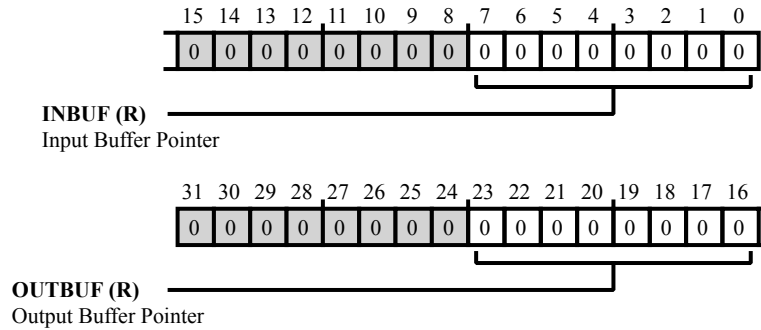


Figure 30-6: `PKTE_BUF_PTR` Register Diagram

Table 30-32: `PKTE_BUF_PTR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
23:16 (R/NW)	OUTBUF	Output Buffer Pointer. The <code>PKTE_BUF_PTR.OUTBUF</code> bit field indicates the offset of the next address (entry) in the output buffer that will be written next by the packet engine. This bit field is reset to zero after starting up and decremented by 4 at every output buffer write operation. Pointers wrap around; the maximum value this field can have equals the output buffer size minus 4.
7:0 (R/NW)	INBUF	Input Buffer Pointer. The <code>PKTE_BUF_PTR.INBUF</code> bit field indicates the offset of the next address (entry) in the input buffer that will be read next by the packet engine. The bit field is reset to zero after starting up and incremented by 4 at every input buffer read operation. Pointers wrap around; the maximum value this field can have equals the input buffer size minus 4.

Packet Engine Buffer Threshold Register

When in autonomous ring mode or target command mode, the `PKTE_BUF_THRESH` register defines the high- and low-level value at which the packet engine starts to transfer packet data in or out of the internal packet buffers. These parameters can be used to control the DMA burst size for packet data input and output from the packet engine. In direct host mode, this register contains both threshold values to reduce the amount of packet engine interrupts.

The input buffer threshold (`ibufthrsh`) interrupt indicates that the input buffer counter is less than or equal to the input buffer threshold value set in this register - this interrupt can be used to wake up a process that stalled on a full input buffer.

The output buffer threshold (`obufthrsh`) interrupt indicates that the output buffer counter exceeds the output buffer threshold setting. The output buffer interrupt remains active until the output buffer counter is decremented to zero again.

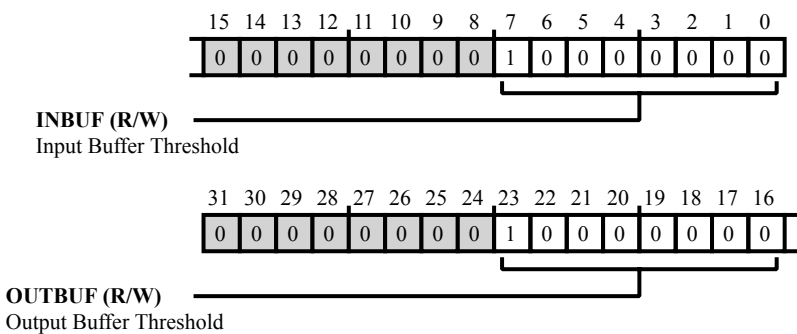


Figure 30-7: `PKTE_BUF_THRESH` Register Diagram

Table 30-33: `PKTE_BUF_THRESH` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
23:16 (R/W)	OUTBUF	Output Buffer Threshold. The <code>PKTE_BUF_THRESH.OUTBUF</code> bit field specifies how many bytes must be available in the packet engine output buffer before an output transfer starts. Valid values range from 0 to 252, in multiples of 4. In autonomous ring mode, a value of 128 generally gives a good performance, but the optimal value depends on the system and application. In direct host mode, the output buffer threshold (<code>obufthrsh</code>) interrupt activates when the output buffer counter for the output buffer exceeds the value set in this field. A value of 128 generally gives a good performance, but the optimal value depends on the system and application.

Table 30-33: PKTE_BUF_THRESH Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/W)	INBUF	Input Buffer Threshold. The <code>PKTE_BUF_THRESH.INBUF</code> bit field specifies how many bytes must be free in the packet engine input buffer before an input transfer starts. Valid values range from 0 to 252, in multiples of 4. In autonomous ring mode, a value of 128 generally gives a good performance, but the optimal value depends on the system and application. In direct host mode, the input buffer threshold (<code>ibufthrsh</code>) interrupt activates when the input buffer counter for the input buffer is below or equal the value set in this field. A value of 128 generally gives a good performance, but the optimal value depends on the system and application.

Packet Engine Command Descriptor Ring Base Address

The `PKTE_CDRBASE_ADDR` register holds the command descriptor ring base address in host memory. It is only applicable in autonomous ring mode. The `PKTE_CDRBASE_ADDR` register is ignored for all other modes when command descriptors are directly written into the descriptor registers.

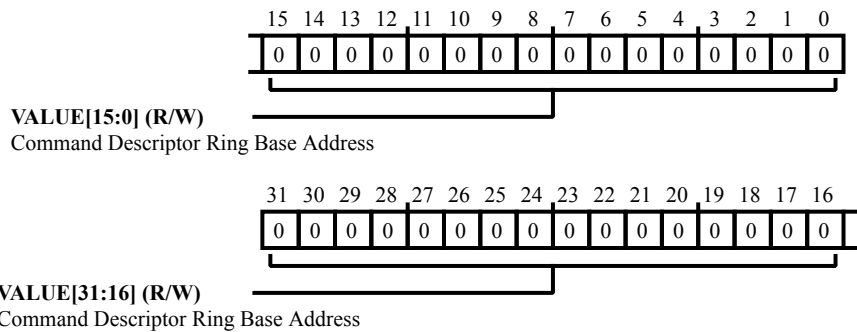


Figure 30-8: `PKTE_CDRBASE_ADDR` Register Diagram

Table 30-34: `PKTE_CDRBASE_ADDR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Command Descriptor Ring Base Address. The <code>PKTE_CDRBASE_ADDR.VALUE</code> bit field specifies the base location of the command descriptor ring in the host memory space.

Packet Engine Command Descriptor Count Register

The `PKTE_CDSC_CNT` register holds the counter for the number of descriptors in the Command Descriptor Ring (CDR). It is decremented by the packet engine each time a valid descriptor is read from the CDR and processed.

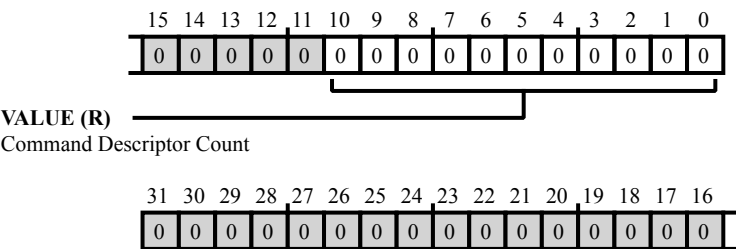


Figure 30-9: PKTE_CDSC_CNT Register Diagram

Table 30-35: PKTE_CDSC_CNT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
10:0 (R/NW)	VALUE	Command Descriptor Count. The <code>PKTE_CDSC_CNT.VALUE</code> bit field provides the number of command descriptors in the command descriptor ring. The packet engine decrements the counter when a valid command descriptor is read from the CDR and processed.

Packet Engine Command Descriptor Count Increment Register

The `PKTE_CDSC_INCR` register is accessible by the host connected through the system completer bus. The host can increment the command descriptor counter by writing a value between 1 and 255 to the lowest byte of this register.

In autonomous ring mode, the host must prepare 1 to 255 valid command descriptors in the CDR and then write this register with a value between 1 and 255. The write triggers the packet engine to fetch the command descriptors from the CDR. In direct host mode or target command mode, the host must write one valid command descriptor to the internal descriptor registers and then write this register with the value 1, to indicate that one valid descriptor is available.

A CDR threshold interrupt is activated when the command descriptor counter is less than or equal to the threshold value set in the `PKTE_RING_THRESH` register. This interrupt can be used to wake up a process that stalled on a full CDR.

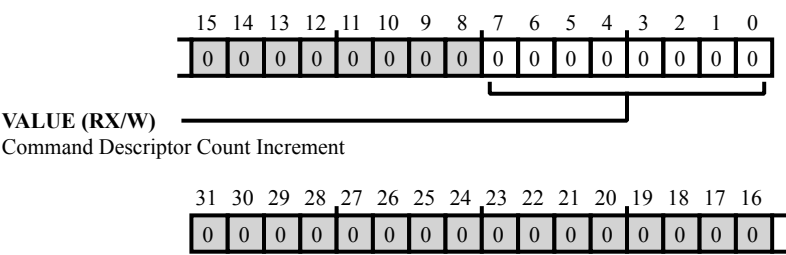


Figure 30-10: PKTE_CDSC_INCR Register Diagram

Table 30-36: PKTE_CDSC_INCR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (RX/W)	VALUE	Command Descriptor Count Increment. The value written to the <code>PKTE_CDSC_INCR.VALUE</code> bit field is added to the command descriptor counter. The counter is protected against overflow (see the <code>PKTE_RING_STAT</code> register description). Note that bits[10:8] should be written with zeros.

Packet Engine Configuration Register

The `PKTE_CFG` register is used to select static settings that control the packet-processing path. This register is typically the last one to be written during the initialization sequence. These settings are typically set at initialization and not changed again.

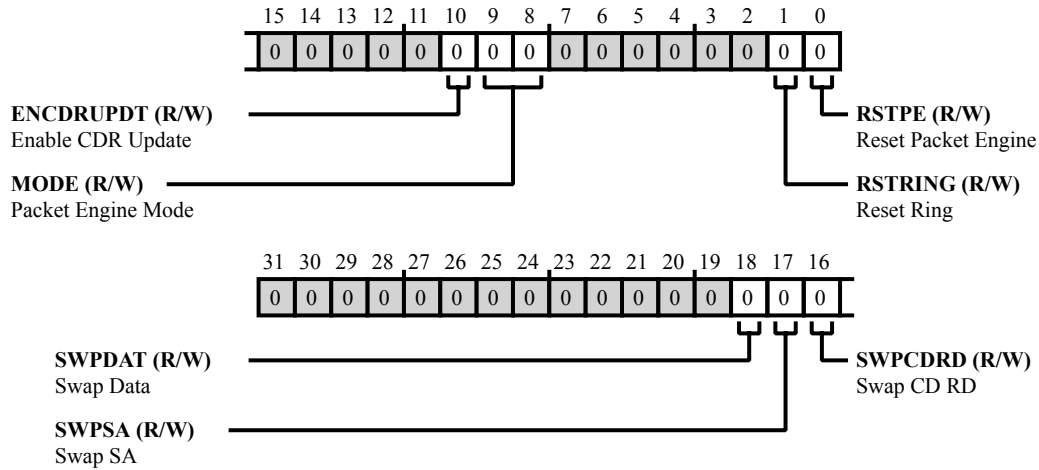


Figure 30-11: PKTE_CFG Register Diagram

Table 30-37: PKTE_CFG Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
18 (R/W)	SWPDAT	Swap Data. The <code>PKTE_CFG.SWPDAT</code> bit enables endian swap for packet data as configured in the <code>PKTE_ENDIAN_CFG.MSTRBSWP</code> bits for the packet data DMA read and write.
		0 No Endian Swap
		1 Apply Endian Swap
17 (R/W)	SWPSA	Swap SA. The <code>PKTE_CFG.SWPSA</code> bit enables endian swap for a SA record as configured in the <code>PKTE_ENDIAN_CFG.MSTRBSWP</code> bits for the SA record and state record DMA read and write. If the <code>PKTE_ENDIAN_CFG.MSTRBSWP</code> bits specify no endian swap, this bit is ignored.
		0 No Endian Swap
		1 Apply Endian Swap

Table 30-37: PKTE_CFG Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
16 (R/W)	SWPCDRD	Swap CD RD. The PKTE_CFG.SWPCDRD bit enables endian swap for descriptors as configured in the PKTE_ENDIAN_CFG.MSTRBSWP bits for the command descriptor DMA read and result descriptor DMA write. If the PKTE_ENDIAN_CFG.MSTRBSWP bits specify no endian swap, this bit is ignored.
		0 No Endian Swap
		1 Apply Endian Swap
10 (R/W)	ENCDRUPDT	Enable CDR Update. The PKTE_CFG.ENCDRUPDT bit enables the packet engine to update, (clear the ownership bits) in the command descriptor in the CDR.
		0 Do Not Clear Ownership Bits. The packet engine does not clear the ownership bits in the command descriptor when it completes an operation. The host application must clear the ownership bits in "old descriptors" before the packet engine is allowed to wrap around the CDR to re-encounter these "old descriptors". This setting has the advantage of eliminating a separate DMA write to the CDR.
		1 Clear Ownership Bits. The packet engine clears (set to zero) the ownership bits in the current command descriptor in the CDR. This prevents the packet engine from re-processing an "old descriptor" when it wraps around the CDR.
9:8 (R/W)	MODE	Packet Engine Mode. The PKTE_CFG.MODE bit field selects how the packet engine receives commands.
		0 Direct Host Mode.
		1 Target Command Mode with Result Descriptor Ring Disabled.
		2 Target Command Mode with Result Descriptor Ring Enabled.
		3 Autonomous Ring Mode

Table 30-37: PKTE_CFG Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/W)	RSTRING	Reset Ring. The PKTE_CFG.RSTRING bit resets the internal counters for the CDR and RDR, PKTE_CDSC_CNT and PKTE_RDSC_CNT registers) to zero. Resets the PKTE_RING_PTR register to the base address. After the reset the rings are empty. This bit must be written with a '1' to reset the descriptor ring manager and then re-written with a '0' to release the reset. Note that this bit can remain in the reset state if the CDR ring is disabled (PKTE_CFG.MODE is not 0b11). Note that this reset must be coordinated with the 'owner' of the descriptor ring to ensure that the pointers are in sync after the reset.
		0 Release the Descriptor Ring Manager Reset
		1 Reset the Descriptor Ring Manager
0 (R/W)	RSTPE	Reset Packet Engine. The PKTE_CFG.RSTPE bit resets the packet engine and the state machine logic that drives header processing, DMA, and context management. The PKTE_CFG.RSTPE bit resets the PKTE_CTL_STAT and PKTE_LEN internal registers. This bit must be written with a 1 to reset the packet engine and then re-written with a 0 to release the reset. Note that this bit should not be used by a typical application. It is provided to use during development testing or to recover from critical errors. Note that the PKTE_CTL_STAT.PADVAL and PKTE_CTL_STAT.PADCTLSTAT bit fields are only reset when in autonomous ring mode, but the PKTE_CTL_STAT.PRNGMD bit is not reset. Halt mode is not affected by this reset as well. When exiting out of halt mode, a HW reset is required or a write to the PKTE_CONT register.
		0 Release the Packet Engine Reset
		1 Reset the Packet Engine

PE Clock Control Register

The `PKTE_CLK_CTL` register controls the clock enable signals. This register can be used to enable the clock for read and write access to SA registers or to enable the required clock signals for certain crypto functions. The setting of this register overrides the packet engine dynamic clock enable.

In autonomous ring mode and target command modes, this register can be all zeros; the packet engine dynamically requests the external clock manager to activate the module clocks. This register can be used in combination with the debugging interface for internal register access.

In direct host mode, the clock enable bits for the packet engine (`PKTE_CLK_CTL.ENPECLK`) and for ARC4 (`PKTE_CLK_CTL.ENARC4CLK`) must be enabled to write and read the SA record and state record registers. All module clocks that are required for the current operation must be enabled during processing.

Note that all the clocks are enabled by default to reset all the registers within the packet engine. After a system reset the host can program this register to disable clocks for power reduction.

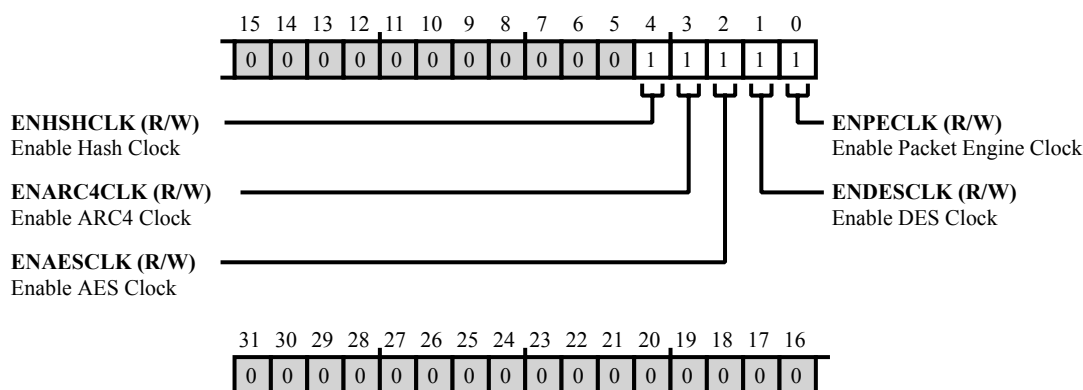


Figure 30-12: PKTE_CLK_CTL Register Diagram

Table 30-38: PKTE_CLK_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
4 (R/W)	ENSHCLK	Enable Hash Clock. The <code>PKTE_CLK_CTL.ENSHCLK</code> bit enables the clock to the hash functions.
		0 Do not enable the hash clock
		1 Enable the hash clock
3 (R/W)	ENARC4CLK	Enable ARC4 Clock. The <code>PKTE_CLK_CTL.ENARC4CLK</code> bit enables the clock to the ARC4 function.
		0 Do not enable the ARC4 clock
		1 Enable the ARC4 clock

Table 30-38: PKTE_CLK_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
2 (R/W)	ENAESCLK	Enable AES Clock. The PKTE_CLK_CTL.ENAESCLK bit enables the clock to the AES encrypt/decrypt function.
		0 Do not enable the AES clock
		1 Enable the AES clock
1 (R/W)	ENDESCLK	Enable DES Clock. The PKTE_CLK_CTL.ENDESCLK bit enables the clock to the DES function.
		0 Do not enable the DES clock
		1 Enable the DES clock
0 (R/W)	ENPECLK	Enable Packet Engine Clock. The PKTE_CLK_CTL.ENPECLK bit enables the clock in the PKTE data path.
		0 Do not enable the PKTE data path clock
		1 Enable the PKTE data path clock

PKTE Continue Register

A write to the `PKTE_CONT` register (with any value) releases the packet engine from a halt state when in halt mode.

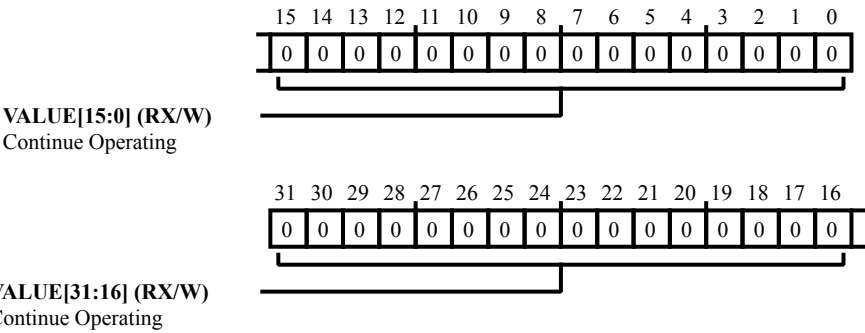


Figure 30-13: PKTE_CONT Register Diagram

Table 30-39: PKTE_CONT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (RX/W)	VALUE	Continue Operating. The <code>PKTE_CONT.VALUE</code> bit field releases the packet engine from a halt state when written with any value in halt mode.

Packet Engine Control Register

The `PKTE_CTL_STAT` register has a dual function. Together with the data in the SA, this register provides the basic command information for the packet engine to process a packet. When the packet engine successfully or unsuccessfully completes an operation, the packet engine control/status register provides the result status for the host.

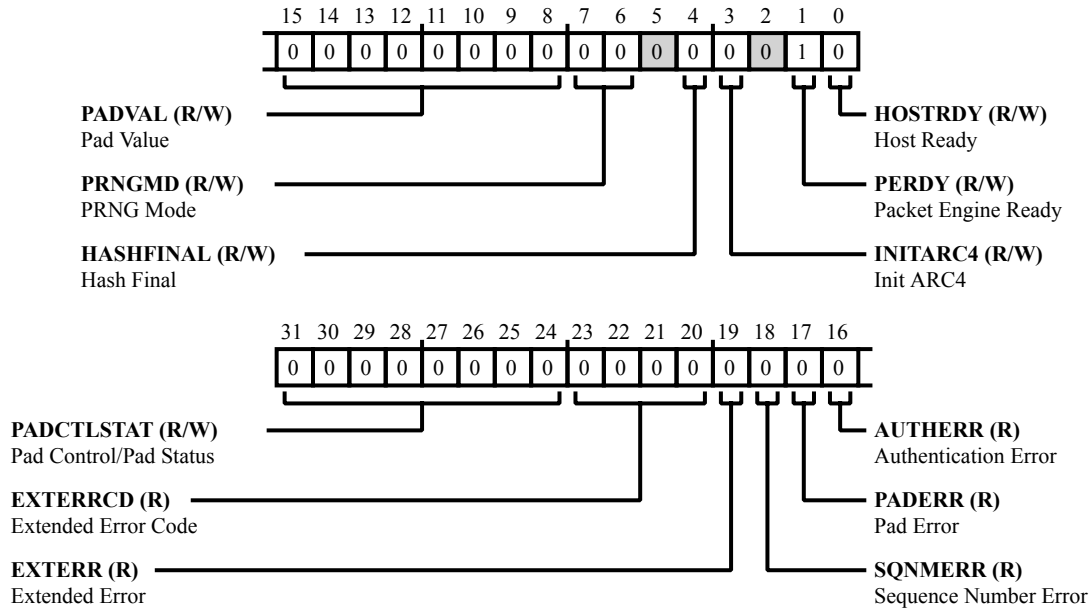


Figure 30-14: PKTE_CTL_STAT Register Diagram

Table 30-40: PKTE_CTL_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	PADCTLSTAT	Pad Control/Pad Status. The <code>PKTE_CTL_STAT.PADCTLSTAT</code> bit field is used to control the pad boundary for pad insertion (outbound) and after processing returns the number of inserted (outbound) or detected (inbound) pad bytes. For the command descriptor, the enumerations below provide the codes for the pad boundary for the outbound operations. This can be used for traffic flow security to conceal the number of payload bytes in an encrypted packet. For the result descriptor inbound operations that use pad types SSL, TLS, IPsec or PKCS#7, it returns the number of detected pad bytes. For all other inbound operations, it returns zero since the other pad modes do not allow implicit determination of pad count. If a pad verify failure occurs, it returns zero. For an outbound operation, it returns the number of inserted pad bytes for all pad types. The pad value includes added bytes such as the pad length and the next header field in an IPsec ESP pad type.
		0 Align packet end to modulo 8-byte boundary
		1 Align packet end to modulo 1-byte boundary

Table 30-40: PKTE_CTL_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
		2	Align packet end to modulo 4-byte boundary
		4	Align packet end to modulo 8-byte boundary
		8	Align packet end to modulo 16-byte boundary
		16	Align packet end to modulo 32-byte boundary
		32	Align packet end to modulo 64-byte boundary
		64	Align packet end to modulo 128-byte boundary
		128	Align packet end to modulo 256-byte boundary
23:20 (R/NW)	EXTERRCD	Extended Error Code. The PKTE_CTL_STAT.EXTERRCD bit field represents an encoded error condition.	
19 (R/NW)	EXTERR	Extended Error. The PKTE_CTL_STAT.EXTERR bit field provides an extended error code.	
		0	No Extended Error
		1	Extended Error
18 (R/NW)	SQNMERR	Sequence Number Error. The PKTE_CTL_STAT.SQNMERR bit indicates that for an inbound operation, there was a fault in the anti-replay sequence number. For an outbound operation, there was a sequence number overflow condition.	
		0	No Sequence Number Error
		1	Sequence Number Error
17 (R/NW)	PADERR	Pad Error. The PKTE_CTL_STAT.PADERR bit indicates that for an inbound operation the decrypted pad does not match the expected values.	
		0	No Pad Error
		1	Pad Error
16 (R/NW)	AUTHERR	Authentication Error. The PKTE_CTL_STAT.AUTHERR bit indicates that for an inbound operation the authentication value in the packet does not match the computed value.	
		0	No Authentication Error
		1	Authentication Error

Table 30-40: PKTE_CTL_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration								
15:8 (R/W)	PADVAL	Pad Value. The <code>PKTE_CTL_STAT . PADVAL</code> bit field is used to pass the pad value between the host and the packet engine. Command Descriptor: (write-only) For outbound operations that use pad type IPsec, the host must populate this field with the value that is to be inserted into the next header field. For the IPsec ESP operation, this next header is part of the ESP trailer of the innermost operation's header and the value must be 50 decimal. For outbound encrypt operations that use the pad type constant or constant SSL, the host must specify the fixed constant value in this field. For all other outbound and inbound operations, this field is not used. Result Descriptor: (read only) For inbound operations that use pad type IPsec, the packet engine returns the next header field that it detects. For IPsec ESP inbound operations, this is the next header field in the innermost operation's header, which will typically be the value for the payload protocol, such as TCP or UDP. However, in bundling scenarios or in IPv6 with destination option headers, another header value could be seen. For all other outbound operations, the packet engine will not update this field. For all other inbound operations, the returned pad value is zero.								
7:6 (R/W)	PRNGMD	PRNG Mode. The <code>PKTE_CTL_STAT . PRNGMD</code> bits select the pseudo-random number generator mode. <table><tr><td>0</td><td>Operation does not use the PRNG function. Operation does not use the PRNG function.</td></tr><tr><td>1</td><td>PRNG Init. PRNG is initialized with a SEED, KEY and an LFSR value as defined in the SA.</td></tr><tr><td>2</td><td>PRNG Generate. Pseudo-random data is generated with the LFSR as input value. Before this mode can be used, the PRNG must be initialized with a valid SEED, KEY and LFSR using PRNG Init (<code>PKTE_CTL_STAT . PRNGMD=b'01</code>).</td></tr><tr><td>3</td><td>PRNG Test. It can be used to test the PRNG function with custom input data. Before this mode can be used, the PRNG must be initialized once with a valid SEED using PRNG Init (<code>PKTE_CTL_STAT . PRNGMD=b'01</code>).</td></tr></table>	0	Operation does not use the PRNG function. Operation does not use the PRNG function.	1	PRNG Init. PRNG is initialized with a SEED, KEY and an LFSR value as defined in the SA.	2	PRNG Generate. Pseudo-random data is generated with the LFSR as input value. Before this mode can be used, the PRNG must be initialized with a valid SEED, KEY and LFSR using PRNG Init (<code>PKTE_CTL_STAT . PRNGMD=b'01</code>).	3	PRNG Test. It can be used to test the PRNG function with custom input data. Before this mode can be used, the PRNG must be initialized once with a valid SEED using PRNG Init (<code>PKTE_CTL_STAT . PRNGMD=b'01</code>).
0	Operation does not use the PRNG function. Operation does not use the PRNG function.									
1	PRNG Init. PRNG is initialized with a SEED, KEY and an LFSR value as defined in the SA.									
2	PRNG Generate. Pseudo-random data is generated with the LFSR as input value. Before this mode can be used, the PRNG must be initialized with a valid SEED, KEY and LFSR using PRNG Init (<code>PKTE_CTL_STAT . PRNGMD=b'01</code>).									
3	PRNG Test. It can be used to test the PRNG function with custom input data. Before this mode can be used, the PRNG must be initialized once with a valid SEED using PRNG Init (<code>PKTE_CTL_STAT . PRNGMD=b'01</code>).									

Table 30-40: PKTE_CTL_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
4 (R/W)	HASHFINAL	Hash Final. When the <code>PKTE_CTL_STAT.HASHFINAL</code> bit is zero, the data to be hashed must be a multiple of the hash block size, 64 bytes for SHA-1, MD5, SHA-224, SHA-256. This bit is only applicable for Basic Hash, Basic Encrypt-Hash and Basic Hash-De-encrypt operations that use the SHA-1, MD5, SHA-224, SHA-256 hash algorithm. The <code>PKTE_CTL_STAT.HASHFINAL</code> bit is overruled for HMAC operations that always completes the hash and always returns the last written value on a read by the host.
		0 Perform Intermediate Hash Operation. The packet engine performs an intermediate hash operation by generating an intermediate hash digest on the data presented on the input. No hash pad is applied.
		1 Perform Final Hash Operation. The packet engine appends the required final hash pad and generates the final hash digest on the data presented on the input. This completes the hash operation.
3 (R/W)	INITARC4	Init ARC4. The <code>PKTE_CTL_STAT.INITARC4</code> bit initializes the ARC4 crypto algorithm with a new key. This bit always returns the last written value on a read by the Host. This bit is only applicable for operations that use the ARC4 algorithm and must be zero for all other operations.
		0 Load ARC4 State and ARC4 i/j pointer from the SA The ARC4 State and ARC4 i/j pointer are loaded from the SA to continue the encrypt/decrypt processing from the previous algorithm state.
		1 Read ARC4 key from the SA-record and initialize ARC4 S-boxes using this key The ARC4 key is read from the SA-record, the ARC4 S-boxes are initialized using this key, prior to the encryption/decryption of data. This bit overrules Stateful mode as defined in bits [9:8] of <code>PKTE_SA_CMD1</code> .
1 (R/W)	PERDY	Packet Engine Ready. The <code>PKTE_CTL_STAT.PERDY</code> bit indicates that the packet engine has completed processing the command descriptor and returns the result descriptor with ownership set to the host. This bit can be reset to 0 by the host and the packet engine, but only the packet engine can set this bit. When the packet engine is idle (not processing), this bit always returns '1' on a read by the host.

Table 30-40: PKTE_CTL_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	HOSTRDY	Host Ready. The <code>PKTE_CTL_STAT.HOSTRDY</code> bit indicates that the host has populated the command descriptor. This bit can be reset to 0 by the host and the packet engine, but only the host can set this bit. When the packet engine is idle (not processing), this bit always returns '0' on a read by the host.

Starting Entry of 256-byte Data Input/Output Buffer

When in direct host mode, the source packet data is written here to be transferred to the packet engine. The host can monitor the available space in the input buffer through the [PKTE_STAT](#) register. This is also the location in the packet engine from where output data is read when in direct host mode. The host can monitor the available bytes in the output buffer through the [PKTE_STAT](#) register.

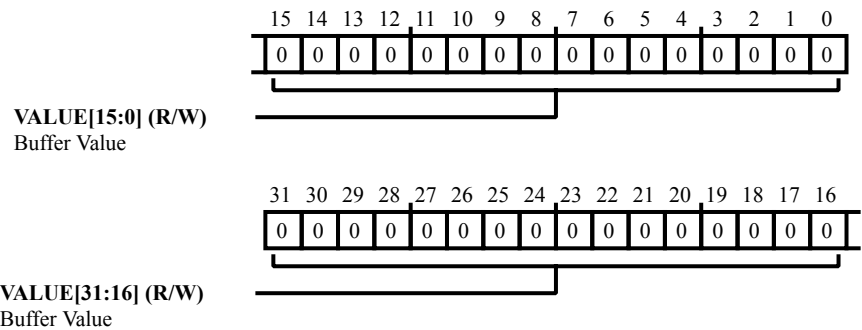


Figure 30-15: PKTE_DATAIO_BUF Register Diagram

Table 30-41: PKTE_DATAIO_BUF Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Buffer Value.

Packet Engine Destination Address

The `PKTE_DEST_ADDR` register holds the starting (byte) address to write the result packet from the requested operation.

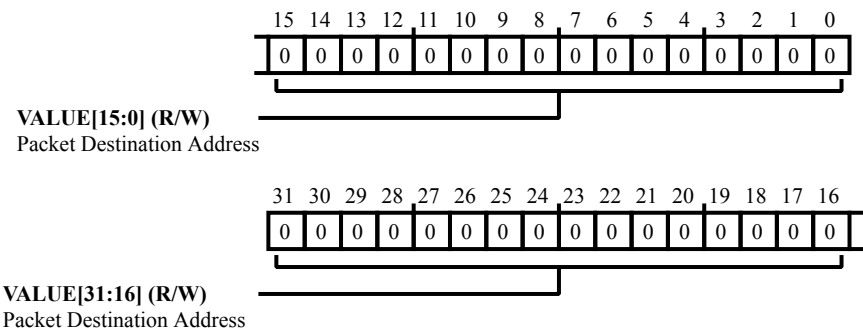


Figure 30-16: PKTE_DEST_ADDR Register Diagram

Table 30-42: PKTE_DEST_ADDR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Packet Destination Address.

Packet Engine DMA Configuration Register

The `PKTE_DMA_CFG` register configures the maximum burst transfer size, enables incremental transfers, and inserts IDLE cycles between two bus transfers.

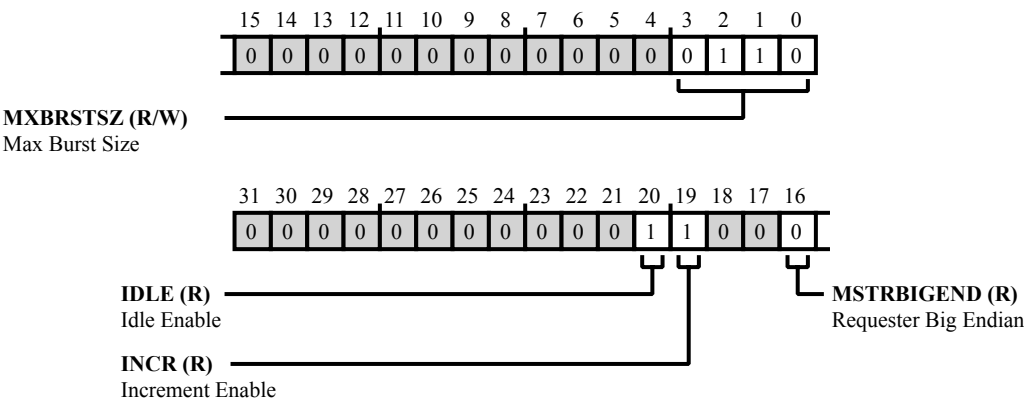


Figure 30-17: PKTE_DMA_CFG Register Diagram

Table 30-43: PKTE_DMA_CFG Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
20 (R/NW)	IDLE	Idle Enable. The <code>PKTE_DMA_CFG.IDLE</code> bit allows the peripheral bus requester to insert one additional IDLE transfer between two successive peripheral bus requester burst operations. This provides the arbiter one additional cycle to hand over the grant to another peripheral bus requester.
		0 The peripheral bus requester inserts no IDLE cycle between two successive burst operations
		1 The peripheral bus requester inserts one additional IDLE transfer between two successive burst operations

Table 30-43: PKTE_DMA_CFG Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
19 (R/NW)	INCR	Increment Enable. The PKTE_DMA_CFG.INCR bit lets the peripheral bus requester generate INC4, INC8 and INC16 type of burst transfers. By default, the peripheral bus requester generates the largest possible incremental burst of unspecified length (INCR) with a maximum length (in bytes) as configured by the PKTE_DMA_CFG.MXBRSTSZ bit field. In case there are less than 4 bytes of data available or the 1kB boundary will be crossed using a burst operation, then a single transfer of size byte is generated. When the PKTE_DMA_CFG.INCR bit is set, the peripheral bus requester generates one or more incremental burst of specified length (INC4, INC8, INC16). In case there is less data available then the smallest possible burst (INC4) or the 1kB boundary will be crossed using a burst operation, then an unspecified length burst or a single transfer of size byte is generated.
		0 The bus requester will generate only INCR burst types
		1 The bus requester will generate INC4, INC8 and INC16 burst types
16 (R/NW)	MSTRBIGEND	Requester Big Endian. The PKTE_DMA_CFG.MSTRBIGEND bit determines whether the engine is used in a little or big endian system.
		0 Little endian
		1 Big endian
3:0 (R/W)	MXBRSTSZ	Max Burst Size. The PKTE_DMA_CFG.MXBRSTSZ bit field configures the maximum size of an unspecified length burst (INC) at the bus in bytes. When there is less data available than the PKTE_DMA_CFG.MXBRSTSZ bit field setting or the 1kB boundary will be crossed using a burst operation, then the length of the burst can be less than PKTE_DMA_CFG.MXBRSTSZ. Any requested transfers larger than this size are broken up in to multiple burst transfers of this size or less.

Packet Engine Endian Configuration Register

The packet engine incorporates a powerful interface specific endian handler. This endian handler allows byte lane swapping in each direction for data passing through the host interface.

The `PKTE_ENDIAN_CFG` register configures the byte order function for the peripheral bus requester and peripheral bus completer interface. The bits for the peripheral bus requester are combined in four sets of two bits; each group configures a byte swap function for a particular DMA transfer. The same applies for the peripheral bus completer interface.

The `PKTE_ENDIAN_CFG` register also defines the endian swapping that occurs for host-initiated target transfers and for packet engine requester DMA read and write transfers. Individual endian swap enable bits in the configuration (`PKTE_CFG`) register can enable the endian swap for various transaction types: command descriptors and result descriptors, SA records and state records, packet data.

In direct host mode, only target operations are supported. Only the target endian configuration of this register is applicable.

Note: This register is typically programmed once during the initialization phase, although software is allowed to dynamically change the setting in this register just before initiating a data transfer. The developer will have to analyze the benefit of the cycles needed to write the endian register dynamically versus handling endian swapping for some data structures in the host system (most modern processors support a byte swap in zero cycles). Certainly the endian swap should be set correctly for the packet data, since this represents the majority of the data transferred.

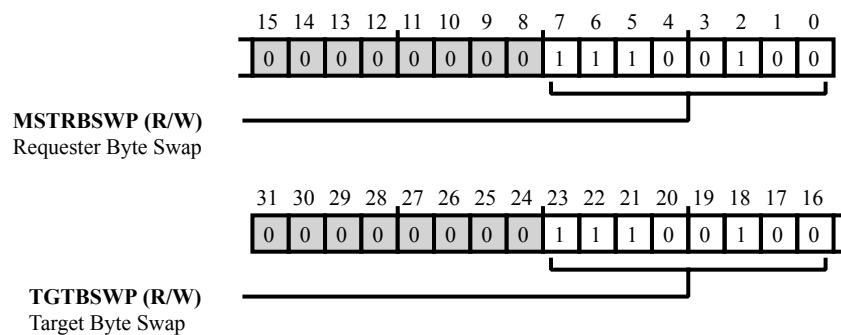


Figure 30-18: `PKTE_ENDIAN_CFG` Register Diagram

Table 30-44: PKTE_ENDIAN_CFG Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
23:16 (R/W)	TGTBSWP	Target Byte Swap. The <code>PKTE_ENDIAN_CFG.TGTBSWP</code> bit field configures the byte swap for peripheral bus target transfers. Note that only target word transfers are supported. Each double-bit field in this register specifies the source of the indicated byte lane. The field values are interpreted as follows: 00 = byte 0, 01 = byte 1, 10 = byte 2, 11 = byte 3. Note: Setting the value 0xE4 defines no swap (little endian) and setting value 0x1B defines a full byte swap within a 32-bit word (big endian).
7:0 (R/W)	MSTRBSWP	Requester Byte Swap. The <code>PKTE_ENDIAN_CFG.MSTRBSWP</code> bit field configures the byte swap for peripheral bus requester multi-byte transfers, including command descriptors, result descriptors, SA records, state records and packet data. Separate controls in the <code>PKTE_CFG</code> register can enable this swap individually for each of the 4 types of data. Each double-bit field in this register specifies the source of the indicated peripheral bus byte lane. The field values are interpreted as follows: 00=byte 0, 01=byte 1, 10=byte 2, 11=byte 3. Note: Setting the value 0xE4 defines no swap (little endian) and setting value 0x1B defines a full byte swap within a 32-bit word (big endian).

Packet Engine Halt Control Register

The `PKTE_HLT_CTL` register controls the packet engine halt mode. This register can be used for debugging purposes while processing in autonomous ring mode or target command mode. During the halt mode, the host can read all internal registers for examination without side-effects. When halted, the host should not write to any registers. To continue packet engine operation, the host must write to the `PKTE_CONT` (PKTE continue) register.

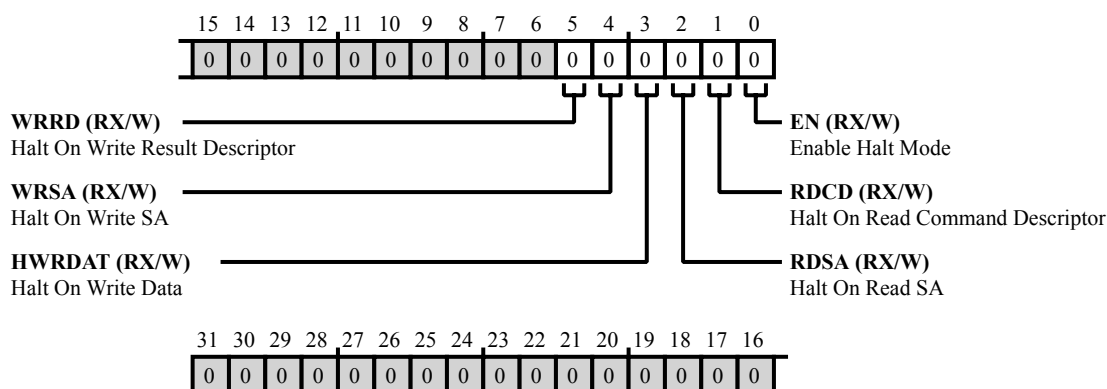


Figure 30-19: PKTE_HLT_CTL Register Diagram

Table 30-45: PKTE_HLT_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
5 (RX/W)	WRRD	Halt On Write Result Descriptor. The <code>PKTE_HLT_CTL.WRRD</code> bit halts the packet engine in the <code>HALT_WRITE_STATUS</code> state after it completes a result descriptor write operation to the result descriptor ring. The host can use this bit to examine the result descriptor that is currently in the host memory.
		0 Do not halt the Packet Engine operation
		1 Halt the Packet Engine operation
4 (RX/W)	WRSA	Halt On Write SA. The <code>PKTE_HLT_CTL.WRSA</code> bit halts the packet engine in the <code>HALT_WRITE_SA</code> state after it completes an SA write operation to the host memory. The host can use this bit to examine the security context that is currently in the host memory.
		0 Do not halt the Packet Engine operation
		1 Halt the Packet Engine operation

Table 30-45: PKTE_HLT_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
3 (RX/W)	HWRDAT	Halt On Write Data. The <code>PKTE_HLT_CTL.HWRDAT</code> bit halts the packet engine in the <code>HALT_DATA</code> state after it completes writing the result packet data to the host memory. The host can use this bit to examine the result packet that is currently in the host memory.
		0 Do not halt the Packet Engine operation
		1 Halt the Packet Engine operation
2 (RX/W)	RDSA	Halt On Read SA. The <code>PKTE_HLT_CTL.RDSA</code> bit halts the packet engine in the <code>HALT_READ_SA</code> state after it completes an SA read operation from the host memory. The host can use this bit to examine the security context that is currently in the SA registers.
		0 Do not halt the Packet Engine operation
		1 Halt the Packet Engine operation
1 (RX/W)	RD CD	Halt On Read Command Descriptor. The <code>PKTE_HLT_CTL.RD CD</code> bit halts the packet engine in the <code>HALT_READ_DESCR</code> state after it completes a command descriptor read operation from the command descriptor ring. It will halt whether the descriptor is valid or invalid. The host can use this bit to examine the command descriptor that is currently in the internal command descriptor registers.
		0 Do not halt the Packet Engine operation
		1 Halt the Packet Engine operation
0 (RX/W)	EN	Enable Halt Mode. The <code>PKTE_HLT_CTL.EN</code> bit enables halt mode where the packet engine can halt processing at any processing state as indicated by bits [5:1]. When halted, the packet engine continues operation on a write to the <code>PKTE_CONT</code> register.
		0 Do not enable halt mode
		1 Enable halt mode

Packet Engine Halt Status Register

The `PKTE_HLT_STAT` register reflects the status of the packet engine in halt mode. This register can be used for debugging purposes while processing in autonomous ring mode or target command mode. When the packet engine is halted, the host can read all internal registers for examination without side effects. The host should not write to any registers.

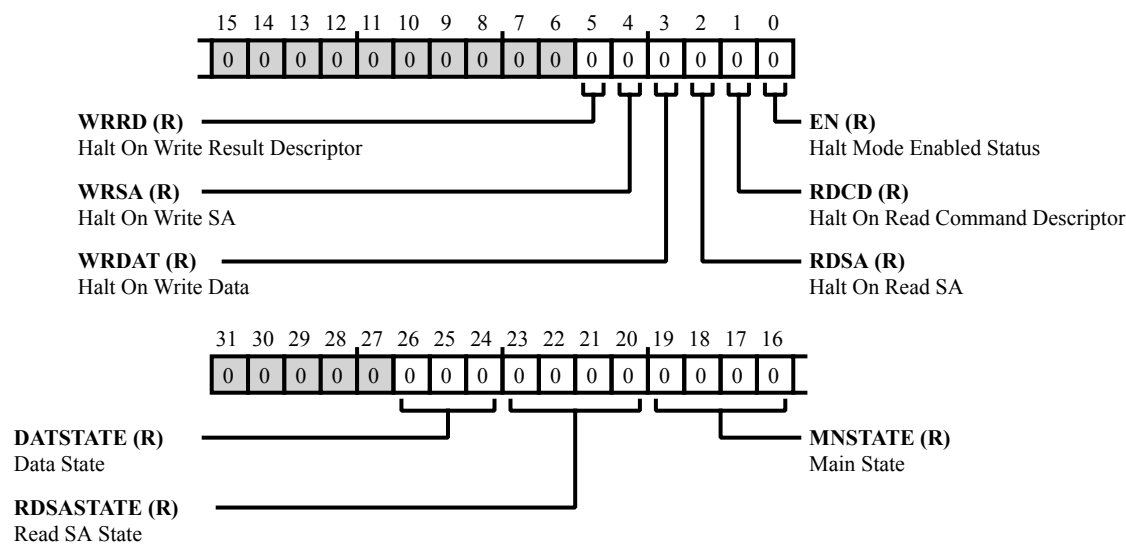


Figure 30-20: PKTE_HLT_STAT Register Diagram

Table 30-46: PKTE_HLT_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
26:24 (R/NW)	DATSTATE	Data State. The <code>PKTE_HLT_STAT.DATSTATE</code> bit field indicates the state of the packet engine read data FSM.
		0 DATA_IDLE, no operation
		1 DATA_READ
		2 DATA_WRITE
		3 DATA_WAIT
		5 DATA_PAD_READ
		6 DATA_BYP_READ
		7 RESERVED
23:20 (R/NW)	RDSASTATE	Read SA State. The <code>PKTE_HLT_STAT.RDSASTATE</code> bit field indicates the state of the packet engine read SA FSM.

Table 30-46: PKTE_HLT_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
		0	SA_IDLE, no operation
		1	SA_READ_CMD
		2	SA_READ_STATE_IV
		3-5	RESERVED
		6	SA_READ_ARC4_STATE
		7	SA_READ_WAIT
		8	RESERVED
		9	SA_WRITE_PROT_HDR
		10	RESERVED
		11	SA_WRITE_IV
		12	SA_WRITE_DIGEST
		13	SA_WRITE_ARC4_IJ_PNTR
		14	SA_WRITE_ARC4_STATE
		15	SA_WRITE_WAIT
19:16 (R/NW)	MNSTATE	Main State. The PKTE_HLT_STAT.MNSTATE bit field indicates the state of the packet engine main FSM.	
		0	MAIN_IDLE, no operation
		1	MAIN_READ_CD, reading command descriptor
		2	MAIN_READ_SA, reading SA
		3	MAIN_DATA, processing data
		4	MAIN_WRITE_SA, writing SA
		5	MAIN_WRITE_STATUS, writing status
		6	MAIN_WRITE_CD, updating command descriptor
		7	MAIN_WRITE_RD, updating result descriptor
		8	MAIN_INIT_WAIT, wait single clock
		9	MAIN_HALT_READ_CD, halt after read command descriptor
		10	MAIN_HALT_READ_SA, halt after read SA
		11	MAIN_HALT_DATA, halt after processing data
		12	MAIN_HALT_WRITE_SA, halt after write SA

Table 30-46: PKTE_HLT_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
		13	MAIN_WAIT_FOR_CLOCK, wait for clocks to be active
		15	MAIN_HALT_WRITE_RD, halt after write result descriptor
5 (R/NW)	WRRD	Halt On Write Result Descriptor. The PKTE_HLT_STAT.WRRD bit reflects the value in the PKTE_HLT_CTL.WRRD bit.	
4 (R/NW)	WRSA	Halt On Write SA. The PKTE_HLT_STAT.WRSA bit reflects the value in the PKTE_HLT_CTL.WRSA bit.	
3 (R/NW)	WRDAT	Halt On Write Data. The PKTE_HLT_STAT.WRDAT bit reflects the value in the PKTE_HLT_CTL.HWRDAT bit.	
2 (R/NW)	RDSA	Halt On Read SA. The PKTE_HLT_STAT.RDSA bit reflects the value in the PKTE_HLT_CTL.RDSA bit.	
1 (R/NW)	RDCD	Halt On Read Command Descriptor. The PKTE_HLT_STAT.RDCD bit reflects the value in the PKTE_HLT_CTL.RDCD bit.	
0 (R/NW)	EN	Halt Mode Enabled Status.	
		0	Halt mode not enabled
		1	Halt mode enabled

Interrupt Mask Disable Register

The host can use the `PKTE_IMSK_DIS` register to clear individual bits in the `PKTE_INT_EN` register for the host interrupt. This register is a bitmap for each of the possible interrupt sources: A 1 clears the interrupt enable bit, a 0 does not affect the interrupt enable bit in the `PKTE_INT_EN` register. Clearing the enable bits through this register avoids the time-consuming read-modify-write operation on the host.

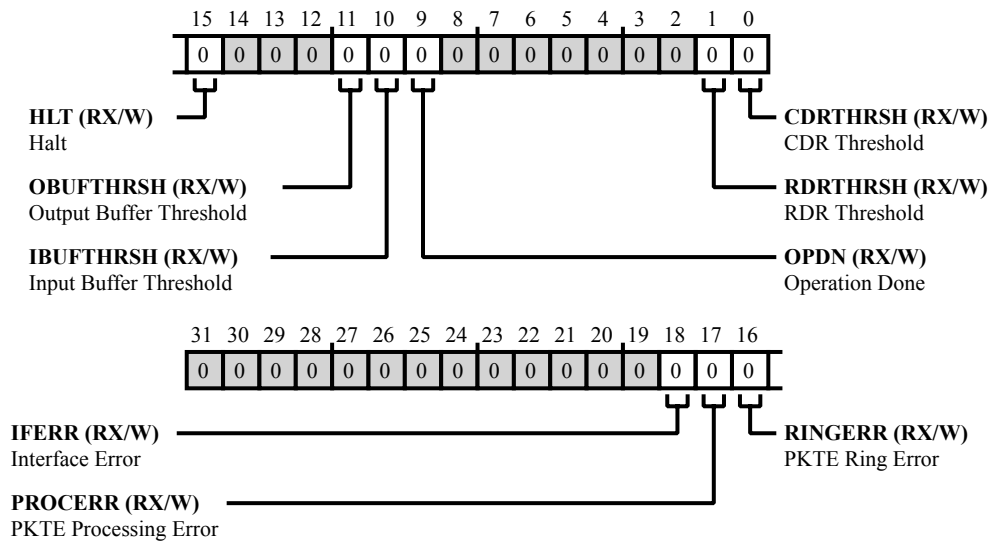


Figure 30-21: PKTE_IMSK_DIS Register Diagram

Table 30-47: PKTE_IMSK_DIS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
18 (RX/W)	IFERR	Interface Error. Write the <code>PKTE_IMSK_DIS</code> .IFERR bit to clear when the host requests a non 32-bit access to the packet engine or when the packet engine receives an error writing data back out to the host memory system.
17 (RX/W)	PROCERR	PKTE Processing Error. Write the <code>PKTE_IMSK_DIS</code> .PROCERR bit to clear an extended error that occurred before, during or after processing the current packet in the packet engine.
16 (RX/W)	RINGERR	PKTE Ring Error. Write the <code>PKTE_IMSK_DIS</code> .RINGERR bit to clear a CDR overflow or an RDR underflow.
15 (RX/W)	HLT	Halt. Write the <code>PKTE_IMSK_DIS</code> .HLT bit to clear when the packet engine is in the halt state.

Table 30-47: PKTE_IMSK_DIS Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
11 (RX/W)	OBUFTHRSH	Output Buffer Threshold. Write the PKTE_IMSK_DIS.OBUFTHRSH bit to clear the output buffer counter exceeds the output buffer threshold value defined in PKTE_BUF_THRESH.OUTBUF bit.
10 (RX/W)	IBUFTHRSH	Input Buffer Threshold. Write the PKTE_IMSK_DIS.IBUFTHRSH bit to clear when the input buffer counter is less than or equal to the input buffer threshold value defined in PKTE_BUF_THRESH.INBUF bit.
9 (RX/W)	OPDN	Operation Done.
1 (RX/W)	RDRTHRSH	RDR Threshold. Write the PKTE_IMSK_DIS.RDRTHRSH bit to clear when the number of result descriptors for the host in the RDR exceeds the RD threshold value in the PKTE_RING_THRESH.RDRTHRSH bit, or the RD counter for the RDR in the PKTE_RDSC_CNT register is non-zero for more than $2^{(N+10)}$ internal system clock cycles.
0 (RX/W)	CDRTHRSH	CDR Threshold. Write the PKTE_IMSK_DIS.CDRTHRSH bit to clear when the number of command descriptors for the packet engine in the CDR is less than or equal to the CD threshold value in the PKTE_RING_THRESH.CDRTHRSH bit.

Interrupt Mask Enable Register

The host can use the `PKTE_IMSK_EN` register to set individual bits in the `PKTE_INT_EN` register for the host interrupt. This register is a bitmap for each of the possible interrupt sources: A 1 sets the interrupt enable bit, a 0 does not affect the interrupt enable bit in the `PKTE_INT_EN` register. Setting the enable bits through this register avoids the time-consuming read-modify-write operation on the host.

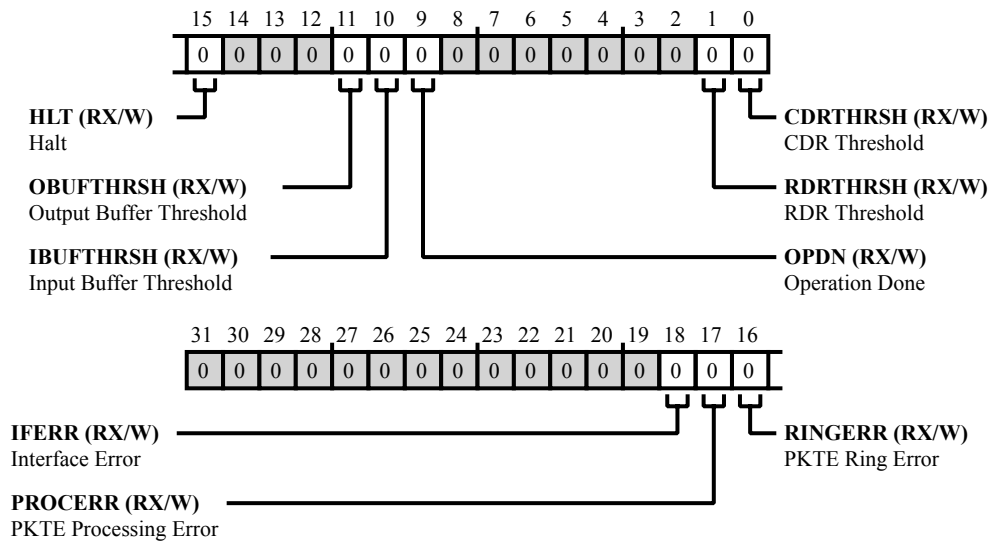


Figure 30-22: PKTE_IMSK_EN Register Diagram

Table 30-48: PKTE_IMSK_EN Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
18 (RX/W)	IFERR	Interface Error. Set the <code>PKTE_IMSK_EN</code> . <code>IFERR</code> bit to indicate a host request for a non 32-bit access to the packet engine or when the packet engine receives an error writing data back out to the host memory system.
17 (RX/W)	PROCERR	PKTE Processing Error. Set the <code>PKTE_IMSK_EN</code> . <code>PROCERR</code> bit to indicate an extended error occurred before, during or after processing the current packet in the packet engine.
16 (RX/W)	RINGERR	PKTE Ring Error.
15 (RX/W)	HLT	Halt. Set the <code>PKTE_IMSK_EN</code> . <code>HLT</code> bit to indicate when the packet engine is in the halt state.

Table 30-48: PKTE_IMSK_EN Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
11 (RX/W)	OBUFTHRSH	Output Buffer Threshold. Set the PKTE_IMSK_EN.OBUFTHRSH bit to indicate that the output buffer counter exceeds the output buffer threshold value defined in the PKTE_BUF_THRESH.OUTBUF bit.
10 (RX/W)	IBUFTHRSH	Input Buffer Threshold. Set the PKTE_IMSK_EN.IBUFTHRSH bit to indicate the input buffer counter is less than or equal to the input buffer threshold value defined in PKTE_BUF_THRESH.INBUF bit.
9 (RX/W)	OPDN	Operation Done.
1 (RX/W)	RDRTHRSH	RDR Threshold. Set the PKTE_IMSK_EN.RDRTHRSH bit to indicate when the number of result descriptors for the host in the RDR exceeds the RD threshold value in the PKTE_RING_THRESH.RDRTHRSH bit, or the RD counter for the RDR in PKTE_RDSC_CNT register is non-zero for more than $2^{(N+10)}$ internal system clock cycles.
0 (RX/W)	CDRTHRSH	CDR Threshold. Set the PKTE_IMSK_EN.CDRTHRSH bit to indicate when the number of command descriptors for the packet engine in the CDR is less than or equal to the CD threshold value in the PKTE_RING_THRESH.CDRTHRSH bit.

Interrupt Masked Status Register

The `PKTE_IMSK_STAT` register provides interrupt status visibility to the host, after the interrupt mask is applied. This lets the host view the selected sources of interrupts that are directed to the interrupt output signal, that is connected to the system interrupt controller. As with the unmasked status register, all interrupt bits are latched and must be cleared using the `PKTE_INT_CLR` register in order to capture a subsequent event. A 1 indicates that the associated interrupt is present.

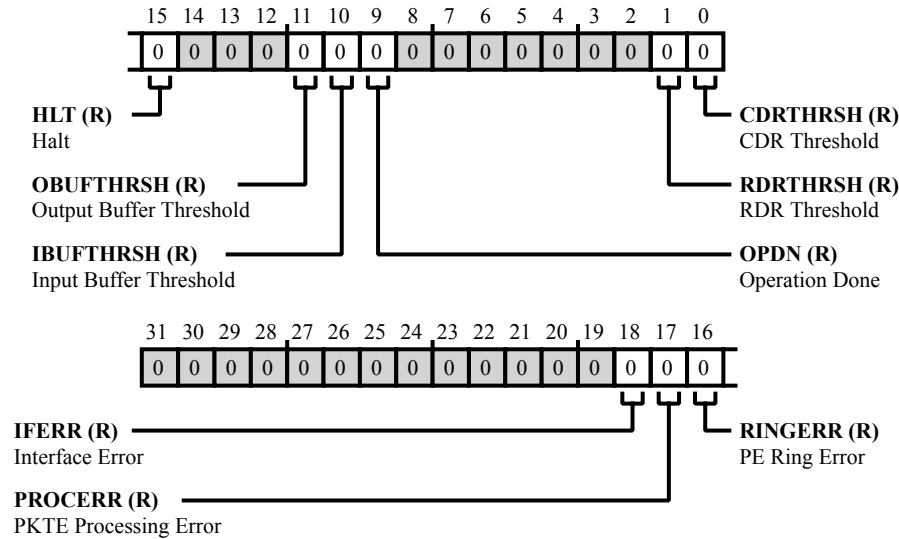


Figure 30-23: PKTE_IMSK_STAT Register Diagram

Table 30-49: PKTE_IMSK_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
18 (R/NW)	IFERR	Interface Error. The <code>PKTE_IMSK_STAT</code> . <code>IFERR</code> bit is set when the host requests a non 32-bit access to the packet engine or when the packet engine receives an error writing data back out to the host memory system.
17 (R/NW)	PROCERR	PKTE Processing Error. The <code>PKTE_IMSK_STAT</code> . <code>PROCERR</code> bit is when an extended error occurred before, during or after processing the current packet in the packet engine.
16 (R/NW)	RINGERR	PE Ring Error. The <code>PKTE_IMSK_STAT</code> . <code>RINGERR</code> bit is set on a CDR overflow or an RDR underflow.
15 (R/NW)	HLT	Halt. The <code>PKTE_IMSK_STAT</code> . <code>HLT</code> bit is set when the packet engine is in the HALT state.

Table 30-49: PKTE_IMSK_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
11 (R/NW)	OBUFTHRSH	Output Buffer Threshold. The <code>PKTE_IMSK_STAT.OBUFTHRSH</code> bit is set when the output buffer counter exceeds the output buffer threshold value defined in <code>PKTE_BUF_THRESH.OUTBUF</code> bit.
10 (R/NW)	IBUFTHRSH	Input Buffer Threshold. The <code>PKTE_IMSK_STAT.IBUFTHRSH</code> bit is set when the input buffer counter is less than or equal to the input buffer threshold value defined in <code>PKTE_BUF_THRESH.INBUF</code> bit.
9 (R/NW)	OPDN	Operation Done.
1 (R/NW)	RDRTHRSH	RDR Threshold. The <code>PKTE_IMSK_STAT.RDRTHRSH</code> bit is set when the number of result descriptors for the host in the RDR exceeds the RD threshold value in the <code>PKTE_RING_THRESH.RDRTHRSH</code> bit, or the RD counter for the RDR in <code>PKTE_RDSC_CNT</code> register is non-zero for more than $2^{(N+10)}$ internal system clock cycles.
0 (R/NW)	CDRTHRSH	CDR Threshold. The <code>PKTE_IMSK_STAT.CDRTHRSH</code> bit is set when the number of command descriptors for the packet engine in the CDR is less than or equal to the CD threshold value in the <code>PKTE_RING_THRESH.CDRTHRSH</code> bit.

Packet Engine Input Buffer Count Register

The `PKTE_INBUF_CNT` register provides the number of bytes available in the input buffer. The `PKTE_INBUF_CNT` register is used in direct host mode only.

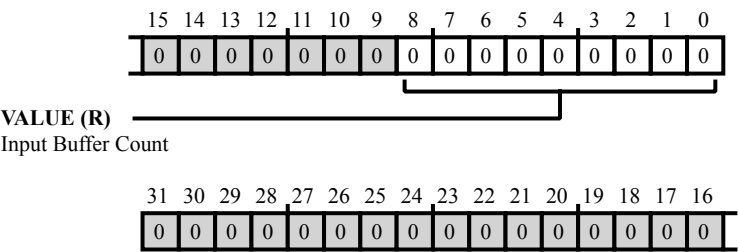


Figure 30-24: `PKTE_INBUF_CNT` Register Diagram

Table 30-50: `PKTE_INBUF_CNT` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
8:0 (R/NW)	VALUE	Input Buffer Count. The <code>PKTE_INBUF_CNT.VALUE</code> bit field provides the number of bytes in the input buffer. The packet engine decrements the counter by 4 when a 32-bit word is read from the input buffer.

Packet Engine Input Buffer Count Increment Register

A host connected through the system completer bus can increment the input buffer counter by writing a value between 4 and 256, in multiples of 4, to the lowest bits of this register. The `PKTE_INBUF_INCR` register is used in direct host mode only.

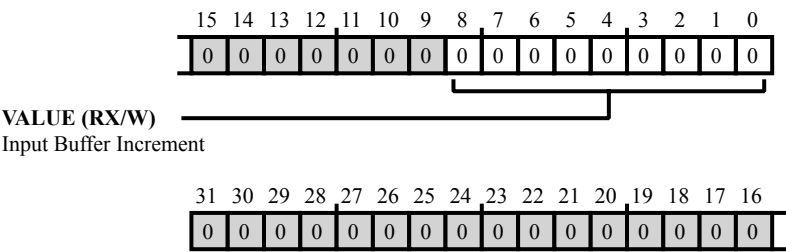


Figure 30-25: `PKTE_INBUF_INCR` Register Diagram

Table 30-51: `PKTE_INBUF_INCR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
8:0 (RX/W)	VALUE	Input Buffer Increment. The value written is added to the input buffer counter. Valid values range from 4 to 256, in multiples of 4.

Interrupt Configuration Register

The `PKTE_INT_CFG` register configures the interrupt type that is sent to the interrupt line connected to the system interrupt controller. (Note that this only effects the final output of the interrupt subsystem).

Configuring the interrupt output type for pulse causes the interrupt signal to pulse low for two clock cycles when activated. When set for level, the interrupt signal is set low until cleared by the host (it follows the bit in the masked status register). For the host, this is typically set to level.

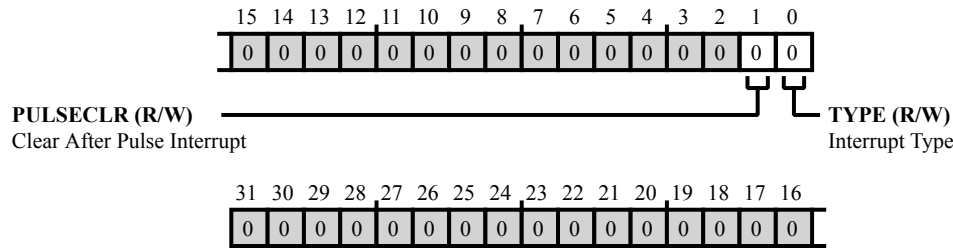


Figure 30-26: `PKTE_INT_CFG` Register Diagram

Table 30-52: `PKTE_INT_CFG` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/W)	PULSECLR	Clear After Pulse Interrupt. The <code>PKTE_INT_CFG.PULSECLR</code> bit clears the latched interrupt source after the pulse interrupt.
		0 Manually clear pulse interrupt source. Do not automatically clear the interrupt sources after pulsing the interrupt output. Clear the source by writing to the <code>PKTE_INT_CLR</code> register.
		1 Automatically clear pulse interrupt source. After pulsing the interrupt output, automatically clear the sources.
0 (R/W)	TYPE	Interrupt Type. The <code>PKTE_INT_CFG.TYPE</code> bit selects the type, pulse or level, for the interrupt output to the system.
		0 Level. The interrupt output is a level signal that is set low when an enabled interrupt is active until the interrupt is cleared.
		1 Pulse. The interrupt output is a two clock cycle low-active pulse, activated when an enabled interrupt is active.

Interrupt Clear Register

The `PKTE_INT_CLR` register allows the host processor to clear pending interrupts. A 1 written to a given bit in this register clears the corresponding interrupt. A 0 leaves the interrupt latch unchanged for that position.

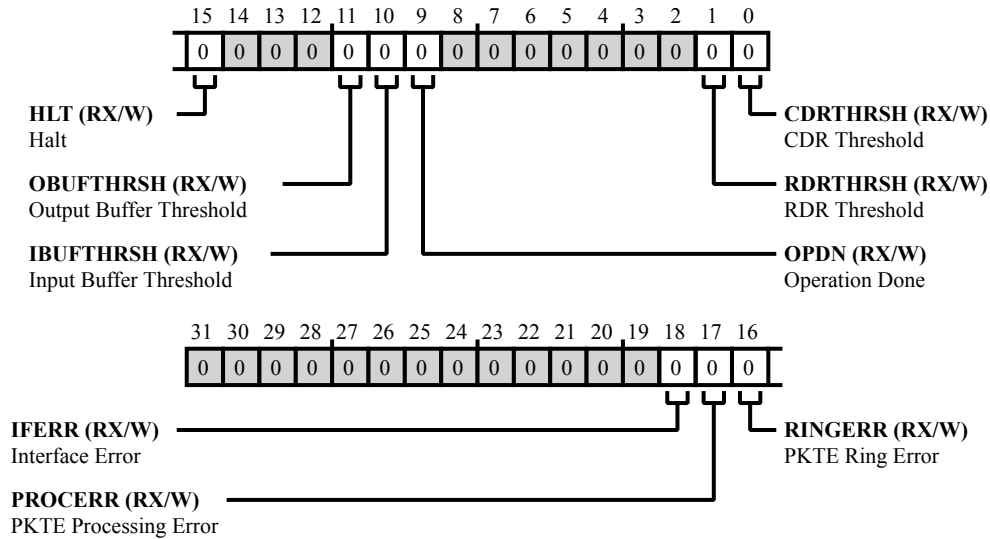


Figure 30-27: `PKTE_INT_CLR` Register Diagram

Table 30-53: `PKTE_INT_CLR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
18 (RX/W)	IFERR	Interface Error. The <code>PKTE_INT_CLR</code> . <code>IFERR</code> bit is set when the host requests a non 32-bit access to the packet engine or when the packet engine receives an error writing data back out to the host memory system.
17 (RX/W)	PROCERR	PKTE Processing Error. The <code>PKTE_INT_CLR</code> . <code>PROCERR</code> bit is set when an extended error occurred before, during or after processing the current packet in the packet engine.
16 (RX/W)	RINGERR	PKTE Ring Error. The <code>PKTE_INT_CLR</code> . <code>RINGERR</code> bit is set on a CDR overflow or an RDR under-flow.
15 (RX/W)	HLT	Halt. The <code>PKTE_INT_CLR</code> . <code>HLT</code> bit is set when the packet engine is in the HALT state.
11 (RX/W)	OBUFTHRSH	Output Buffer Threshold. The <code>PKTE_INT_CLR</code> . <code>OBUFTHRSH</code> bit is set when the output buffer counter exceeds the output buffer threshold value defined in <code>PKTE_BUF_THRESH</code> . <code>OUTBUF</code> bit.

Table 30-53: PKTE_INT_CLR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
10 (RX/W)	IBUFTHRSH	Input Buffer Threshold. The PKTE_INT_CLR.IBUFTHRSH bit is set when the input buffer counter is less than or equal to the input buffer threshold value defined in PKTE_BUF_THRESH.INBUF bit.
9 (RX/W)	OPDN	Operation Done.
1 (RX/W)	RDRTHRSH	RDR Threshold. The PKTE_INT_CLR.RDRTHRSH bit is set when the number of result descriptors for the host in the RDR exceeds the RD threshold value in the PKTE_RING_THRESH.RDRTHRSH bit, or the RD counter for the RDR in PKTE_RDSC_CNT register is non-zero for more than $2^{(N+10)}$ internal system clock cycles.
0 (RX/W)	CDRTHRSH	CDR Threshold. The PKTE_INT_CLR.CDRTHRSH bit is set when the number of command descriptors for the packet engine in the CDR is less than or equal to the CD threshold value in the PKTE_RING_THRESH.CDRTHRSH bit.

Interrupt Enable Register

The `PKTE_INT_EN` register configures the interrupt mask for the host interrupt. This register is a bitmap for each of the possible interrupt sources. A 1 enables the interrupt source and a 0 disables the source. If an interrupt source is disabled, a cleared bit also clears the matching interrupt in the `PKTE_IMSK_STAT` register.

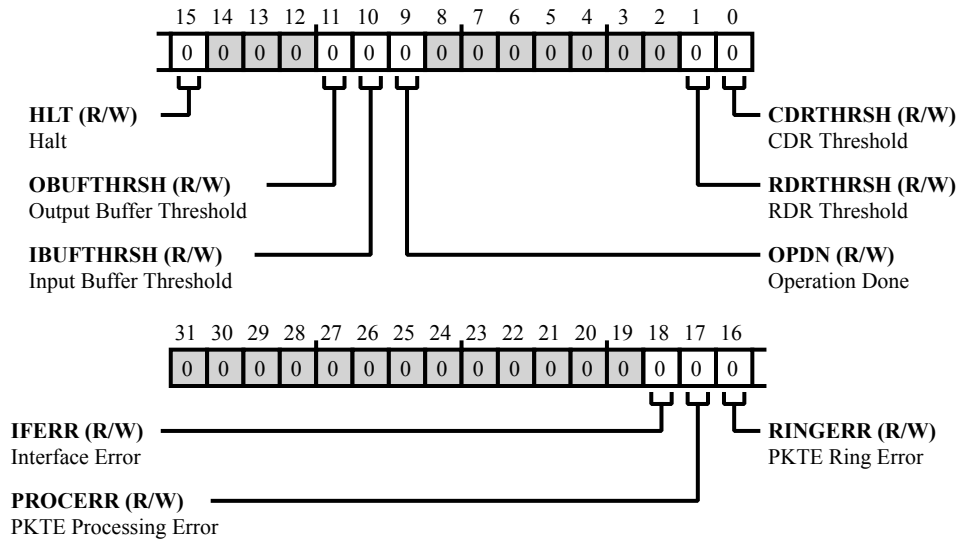


Figure 30-28: `PKTE_INT_EN` Register Diagram

Table 30-54: `PKTE_INT_EN` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
18 (R/W)	IFERR	Interface Error. Set the <code>PKTE_INT_EN</code> .IFERR bit for host requests for a non 32-bit access to the packet engine interrupt or when the packet engine receives an error writing data back out to the host memory system.
17 (R/W)	PROCERR	PKTE Processing Error. Set the <code>PKTE_INT_EN</code> .PROCERR bit to enable the extended error occurred before, during or after processing the current packet in the packet engine interrupt.
16 (R/W)	RINGERR	PKTE Ring Error. Set the <code>PKTE_INT_EN</code> .RINGERR bit to enable the CDR overflow or RDR under-flow interrupt.
15 (R/W)	HLT	Halt. Set the <code>PKTE_INT_EN</code> .HLT bit for when the packet engine is in the HALT state.
11 (R/W)	OBUFTHRS	Output Buffer Threshold. Set the <code>PKTE_INT_EN</code> .OBUFTHRS bit for to trigger an interrupt when the output buffer counter exceeds the output buffer threshold value defined in the <code>PKTE_BUF_THRESH</code> .OUTBUF bit.

Table 30-54: PKTE_INT_EN Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
10 (R/W)	IBUFTHRSH	Input Buffer Threshold. Set the PKTE_INT_EN.IBUFTHRSH bit for to trigger an interrupt when the input buffer counter is less than or equal to the input buffer threshold value defined in the PKTE_BUF_THRESH.INBUF bit.
9 (R/W)	OPDN	Operation Done.
1 (R/W)	RDRTHRSH	RDR Threshold. Set the PKTE_INT_EN.RDRTHRSH bit for to trigger an interrupt when the number of result descriptors for the host in the RDR exceeds the RD threshold value in the PKTE_RING_THRESH.RDRTHRSH bit, or the RD counter for the RDR in PKTE_RDSC_CNT register is non-zero for more than $2^{(N+10)}$ internal system clock cycles.
0 (R/W)	CDRTHRSH	CDR Threshold. Set the PKTE_INT_EN.CDRTHRSH bit for to trigger an interrupt when the number of command descriptors for the packet engine in the CDR is less than or equal to the CD threshold value in the PKTE_RING_THRESH.CDRTHRSH bit.

Interrupt Unmasked Status Register

The `PKTE_IUMSK_STAT` register provides interrupt status visibility to the host, prior to the interrupt mask being applied. Using this register, the host can view all potential sources of incoming interrupts. All of these sources, whether masked in or out, are latched in this register and must be cleared using the `PKTE_INT_CLR` register in order to capture a subsequent event. A 1 indicates that the associated interrupt is present.

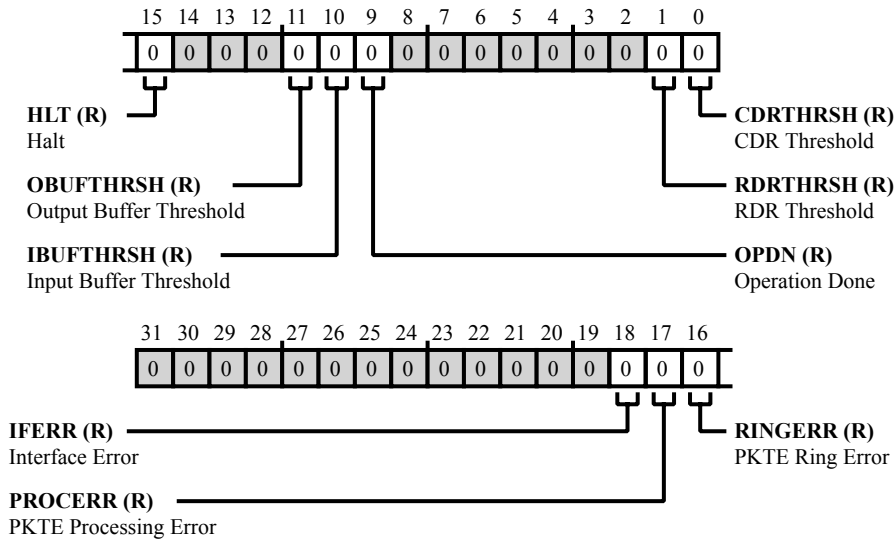


Figure 30-29: `PKTE_IUMSK_STAT` Register Diagram

Table 30-55: `PKTE_IUMSK_STAT` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
18 (R/NW)	IFERR	Interface Error. The <code>PKTE_IUMSK_STAT</code> .IFERR bit is set when the host requests a non 32-bit access to the packet engine or when the packet engine receives an error writing data back out to the host memory system.
17 (R/NW)	PROCERR	PKTE Processing Error. The <code>PKTE_IUMSK_STAT</code> .PROCERR bit is set when an extended error occurred before, during or after processing the current packet in the packet engine.
16 (R/NW)	RINGERR	PKTE Ring Error. The <code>PKTE_IUMSK_STAT</code> .RINGERR bit is set on a CDR overflow or an RDR underflow.
15 (R/NW)	HLT	Halt. The <code>PKTE_IUMSK_STAT</code> .HLT bit is set when the packet engine is in the HALT state.

Table 30-55: PKTE_IUMSK_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
11 (R/NW)	OBUFTHRSH	Output Buffer Threshold. The <code>PKTE_IUMSK_STAT.OBUFTHRSH</code> interrupt is triggered when the output buffer counter exceeds the output buffer threshold value defined in <code>PKTE_BUF_THRESH.OUTBUF</code> bit.
10 (R/NW)	IBUFTHRSH	Input Buffer Threshold. The <code>PKTE_IUMSK_STAT.IBUFTHRSH</code> interrupt is triggered when the input buffer counter is less than or equal to the input buffer threshold value defined in <code>PKTE_BUF_THRESH.INBUF</code> bit.
9 (R/NW)	OPDN	Operation Done.
1 (R/NW)	RDRTHRSH	RDR Threshold. The <code>PKTE_IUMSK_STAT.RDRTHRSH</code> bit is set when the number of result descriptors for the host in the RDR exceeds the RD threshold value in the <code>PKTE_RING_THRESH.RDRTHRSH</code> , or the RD counter for the RDR in <code>PKTE_RDSC_CNT</code> register is non-zero for more than $2^{(N+10)}$ internal system clock cycles.
0 (R/NW)	CDRTHRSH	CDR Threshold. The <code>PKTE_IUMSK_STAT.CDRTHRSH</code> bit is set when the number of command descriptors for the packet engine in the CDR is less than or equal to the CD threshold value in the <code>PKTE_RING_THRESH.CDRTHRSH</code> bit.

Packet Engine Length Register

The `PKTE_LEN` register gives the length of the packet, the bypass data and a second set of ownership bits.

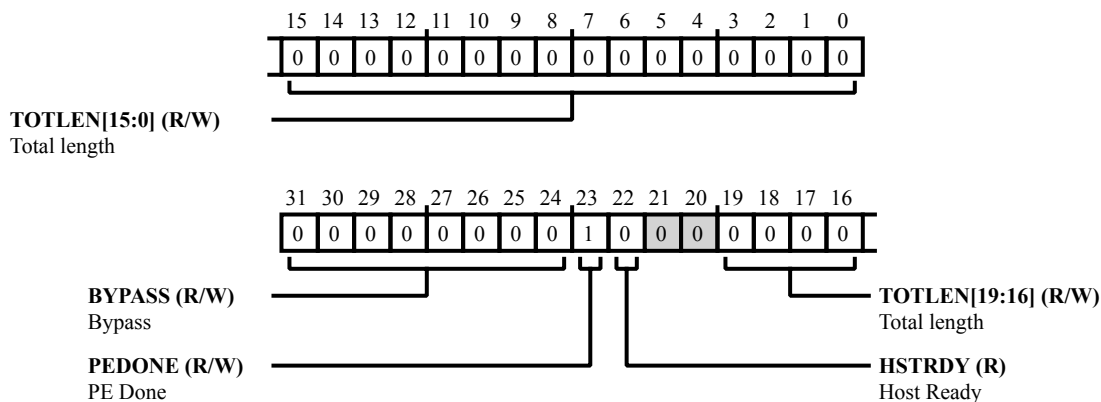


Figure 30-30: `PKTE_LEN` Register Diagram

Table 30-56: `PKTE_LEN` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	BYPASS	Bypass. The <code>PKTE_LEN.BYPASS</code> bit field indicates the length of data in words that must bypass the packet engine and are directly copied from the source buffer to the destination buffer. The packet engine does not process this data. Valid bypass offsets range from 0 (0x00) to 255 (0xFF) words. For SRTP operations, this field specifies the offset in words between the hash and encrypt/decrypt data.
23 (R/W)	PEDONE	PE Done. The <code>PKTE_LEN.PEDONE</code> bit is a mirrored bit from the <code>PKTE_CTL_STAT.PERDY</code> bit. The bit is repeated here to guarantee ownership consistency between the first and last word. When the packet engine fetches a descriptor, these bits must match or the descriptor is discarded and fetched again.
22 (R/NW)	HSTRDY	Host Ready. The <code>PKTE_LEN.HSTRDY</code> bit is a mirrored bit of the <code>PKTE_CTL_STAT.HOSTRDY</code> bit. The bit is repeated here to guarantee ownership consistency between the first and last word. It should also be set along with the <code>PKTE_CTL_STAT.HOSTRDY</code> bit when the command descriptor is finished being populated. When the packet engine fetches a descriptor, these bits must match or the descriptor is discarded and fetched again.

Table 30-56: PKTE_LEN Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
19:0 (R/W)	TOTLEN	Total length. Command Descriptor: The <code>PKTE_LEN.TOTLEN</code> bit field indicates the total length (in bytes) of all data to be passed to the packet engines input buffer for an operation. Exceptions are the PRNG init and PRNG generate operations. The PRNG init operation does not require any input data; this field must be zero. For the PRNG generate operation, this field indicates the number of pseudo-random bytes to be generated. Valid lengths range from 16 (0x00010) to $255 \times 16 = 4080$ (0x00FF0) bytes in multiples of 16 bytes. Valid lengths for the basic operation range from 1 (0x00001) to 1,048,575 (0xFFFFF) bytes. This is the length of the data to be encrypted or hashed and includes the bypass data and padding bytes. Valid lengths for IPsec ESP range from 1 (0x00001) to 65535 (0xFFFFF) bytes. This is the length of the IP payload. Valid lengths for SSL v3.0, TLS v1.x and DTLS range from 1 (0x00001) to 16383 (0x03FFF). This is the length of the payload. Valid lengths for SRTP range from 1 (0x00001) to 65535 (0xFFFFF). This is the length of the payload. Note: A length of zero bytes is illegal and will result in an error status code in the result descriptor. Result Descriptor: Upon completion of an operation, the <code>PKTE_LEN.TOTLEN</code> field indicates the result length of the result packet. Valid lengths range from 1 (0x001) to 1,048,575 (0xFFFFF) bytes. This includes the bypass data and padding bytes. Note: When an extended error (<code>PKTE_CTL_STAT[18]=1</code>) is reported in the result descriptor and no packet data is processed, this field returns zero.

Packet Engine Output Buffer Count Register

The `PKTE_OUTBUF_CNT` register provides the number of data bytes there are in the output buffer. The `PKTE_OUTBUF_CNT` register is used in direct host mode only.

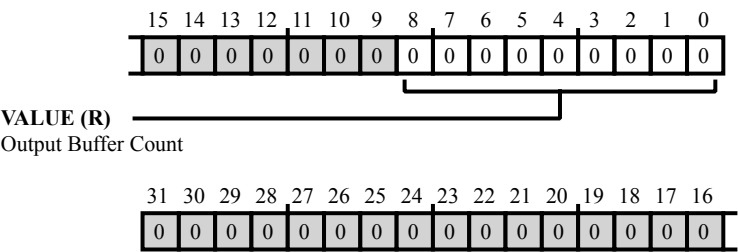


Figure 30-31: `PKTE_OUTBUF_CNT` Register Diagram

Table 30-57: `PKTE_OUTBUF_CNT` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
8:0 (R/NW)	VALUE	Output Buffer Count. The <code>PKTE_OUTBUF_CNT.VALUE</code> bit field provides the number of bytes in the output buffer. The packet engine increments the counter by 4 when a 32-bit word is written to the output buffer.

Packet Engine Output Buffer Count Decrement Register

A host connected via the system completer bus can decrement the output buffer counter by writing a value between 4 and 256, in multiples of 4, to the lowest bits of this register. The `PKTE_OUTBUF_DECR` register is used in direct host mode only.

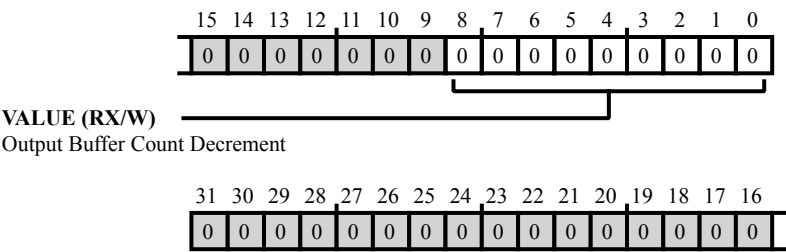


Figure 30-32: PKTE_OUTBUF_DECR Register Diagram

Table 30-58: PKTE_OUTBUF_DECR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
8:0 (RX/W)	VALUE	Output Buffer Count Decrement. The <code>PKTE_OUTBUF_DECR.VALUE</code> bit field is the value written is subtracted to the output buffer counter. Valid values range from 4 to 256, in multiples of 4.

Packet Engine Result Descriptor Ring Base Address

The `PKTE_RDRBASE_ADDR` register holds the result descriptor ring base address in host memory. It is only applicable in autonomous ring mode and target command mode with RDR enabled. Note that in target command mode, the CDR is not used, but the RDR must be configured when enabled so that the packet engine knows where to write the result descriptors.

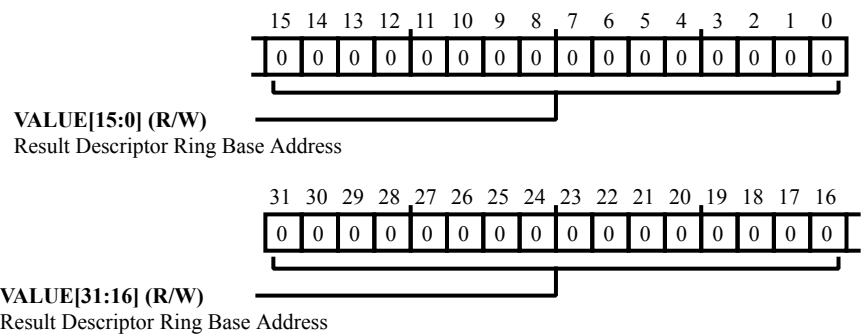


Figure 30-33: PKTE_RDRBASE_ADDR Register Diagram

Table 30-59: PKTE_RDRBASE_ADDR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Result Descriptor Ring Base Address. The <code>PKTE_RDRBASE_ADDR.VALUE</code> bit field specifies the base location of the result descriptor ring in the host memory space.

Packet Engine Result Descriptor Count Registers

The `PKTE_RDSC_CNT` register holds the counter for the number of descriptors in the Result Descriptor Ring (RDR). It is incremented by the packet engine each time a valid result descriptor is written to the RDR.

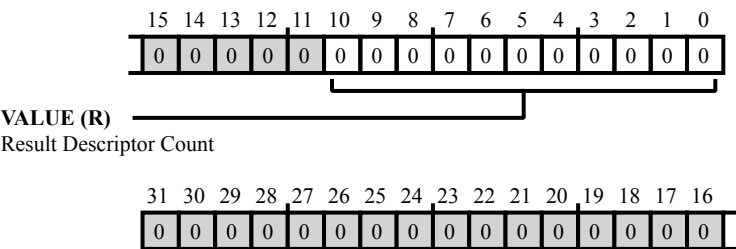


Figure 30-34: PKTE_RDSC_CNT Register Diagram

Table 30-60: PKTE_RDSC_CNT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
10:0 (R/NW)	VALUE	Result Descriptor Count. The <code>PKTE_RDSC_CNT.VALUE</code> bit field provides the number of result descriptors in the result descriptor ring. The packet engine increments the counter when a valid result descriptor is written to the RDR.

Packet Engine Result Descriptor Count Decrement Registers

The `PKTE_RDSC_DECR` register is accessible by the host connected through the system completer bus can decrement the result descriptor counter by writing a value between 1 and 255 to the lowest byte of this register.

With an RDR enabled, this is the number of result descriptors that have been read by the host. With an RDR disabled, this indicates that the host has read one valid result descriptor.

In autonomous ring mode or target command mode with the RDR enabled, the host must process 1 to 255 result descriptors from the RDR and then write this register with the number of result descriptors that have been processed by the host.

In direct host mode or target command mode with the RDR disabled, the host must read one result descriptor from the internal descriptor registers and then write this register with the value 1, to indicate that one valid descriptor is read. An RDR threshold interrupt is activated when the result descriptor counter exceeds the threshold value set in the `PKTE_RING_THRESH` register. This interrupt can be used to wake up a process that stalled on an empty RDR.

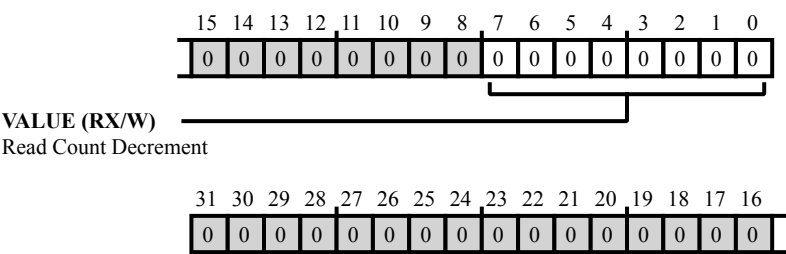


Figure 30-35: PKTE_RDSC_DECR Register Diagram

Table 30-61: PKTE_RDSC_DECR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (RX/W)	VALUE	Read Count Decrement. The value written to the <code>PKTE_RDSC_DECR.VALUE</code> bit field is subtracted from the result descriptor counter. The counter is protected against underflow (See the <code>PKTE_RING_STAT</code> register). Note that bits [10:8] should be written with zeros.

Packet Engine Ring Configuration

The `PKTE_RING_CFG` register configures the size (in number of descriptor ring entries minus 1) for both the command descriptor ring and result descriptor ring in host memory. This register is only applicable for autonomous ring mode and target command mode with RDR enabled.

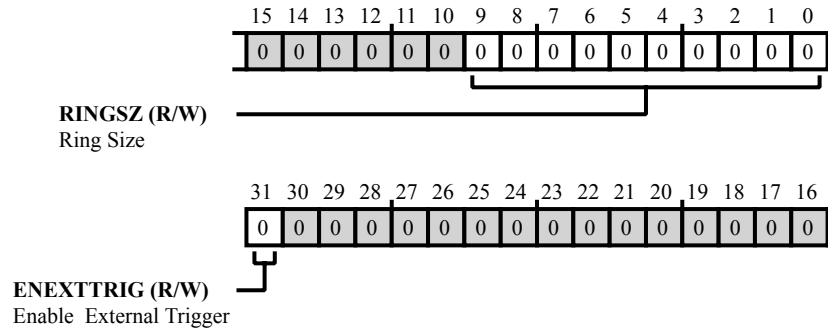


Figure 30-36: PKTE_RING_CFG Register Diagram

Table 30-62: PKTE_RING_CFG Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	ENEXTTRIG	Enable External Trigger. The <code>PKTE_RING_CFG.ENEXTTRIG</code> signal enables the increment of the <code>PKTE_CDSC_CNT</code> register through the external input pin <code>ext_cd_cnt_incr</code> and enables the decrement of the <code>PKTE_RDSC_CNT</code> fields through the external input pin <code>ext_rd_cnt_decr</code> .
9:0 (R/W)	RINGSZ	Ring Size. The <code>PKTE_RING_CFG.RINGSZ</code> bit field specifies the size of the command ring in number of descriptors, minus 1. Valid sizes range from 1 (for 2 descriptors) to 1023 (for 1024 descriptors). The accompanying result ring will have the same size.

Packet Engine Ring Pointer Status

The `PKTE_RING_PTR` register holds the pointers to the current entry of the Command Descriptor Ring (CDR) and Result Descriptor Ring (RDR).

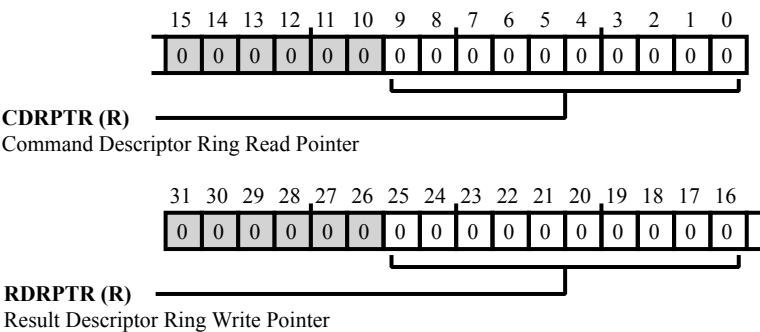


Figure 30-37: `PKTE_RING_PTR` Register Diagram

Table 30-63: `PKTE_RING_PTR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
25:16 (R/NW)	RDRPTR	Result Descriptor Ring Write Pointer. The <code>PKTE_RING_PTR.RDRPTR</code> bit field indicates the entry number in the RDR that will be written next by the packet engine. The <code>PKTE_RING_PTR.RDRPTR</code> bit field is reset to zero after starting up and updated after every result descriptor write DMA operation. Pointers wrap around; the maximum value this field can have equals the contents of the ring size (<code>PKTE_RING_CFG.RINGSZ</code>) bit field.
9:0 (R/NW)	CDRPTR	Command Descriptor Ring Read Pointer. The <code>PKTE_RING_PTR.CDRPTR</code> bit field indicates the entry number in the CDR that will be read next by the packet engine. The <code>PKTE_RING_PTR.CDRPTR</code> bit field is reset to zero after starting up and updated after every command descriptor read DMA operation. Pointers wrap around; the maximum value this field can have equals the contents of the ring size (<code>PKTE_RING_CFG.RINGSZ</code>) field.

Packet Engine Ring Status

The `PKTE_RING_STAT` register gives indication of either a Command Descriptor Ring (CDR) overflow or a Result Descriptor Ring (RDR) underflow. A ring error (ringerr) interrupt in the interrupt controller is activated on a command descriptor ring overflow or a result descriptor ring underflow. This type of error can occur when the host and the packet engine get out-of-sync. The host can read this register to retrieve information on which ring is corrupted. The corrupted ring must be reset and reinitialized. See the `PKTE_CFG.RSTRING` bit.

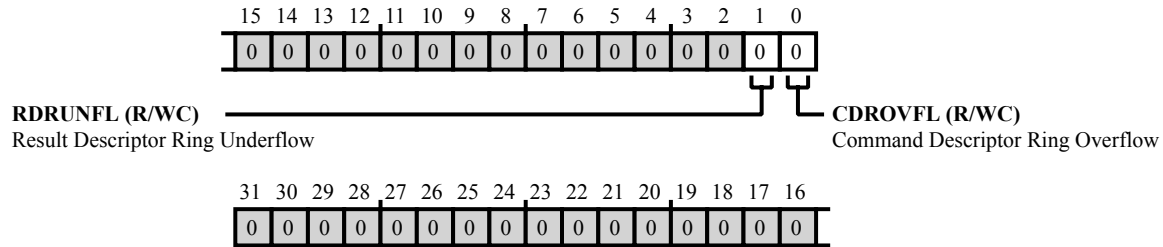


Figure 30-38: `PKTE_RING_STAT` Register Diagram

Table 30-64: `PKTE_RING_STAT` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/WC)	RDRUNFL	Result Descriptor Ring Underflow. The <code>PKTE_RING_STAT.RDRUNFL</code> bit is set when the command descriptor count (<code>PKTE_RDSC_CNT</code>) register is decremented below zero. This bit is reset with a write of any value.
0 (R/WC)	CDROVFL	Command Descriptor Ring Overflow. The <code>PKTE_RING_STAT.CDROVFL</code> bit is set when the command descriptor count (<code>PKTE_CDSC_CNT</code>) register is incremented above the ring size (<code>PKTE_RING_CFG.RINGSZ</code>) bits. This bit is reset with a write of any value.

Packet Engine Ring Threshold Registers

To reduce the amount of packet engine result interrupts, the `PKTE_RING_THRESH` register contains threshold and time-out values.

The CDR threshold (`cdrthrsh`) interrupt indicates that the command descriptor counter is less than or equal to the CDR threshold (`cdrthrsh`) value set in this register. This interrupt can be used to wake up a process that stalled on a full CDR.

The RDR threshold (`rdrthrsh`) interrupt indicates that the result descriptor counter exceeds the result descriptor threshold set here, or that the result descriptor counter is non-zero for a time longer than the result descriptor time-out setting. The RDR result interrupt remains active until the result descriptor counter is decremented below the RDR threshold (`rdrthrsh`) value. In case the interrupt is the result of a time-out and the result descriptor counter is below the threshold value, the result descriptor counter must be decremented once before the interrupt can be cleared in the interrupt controller.

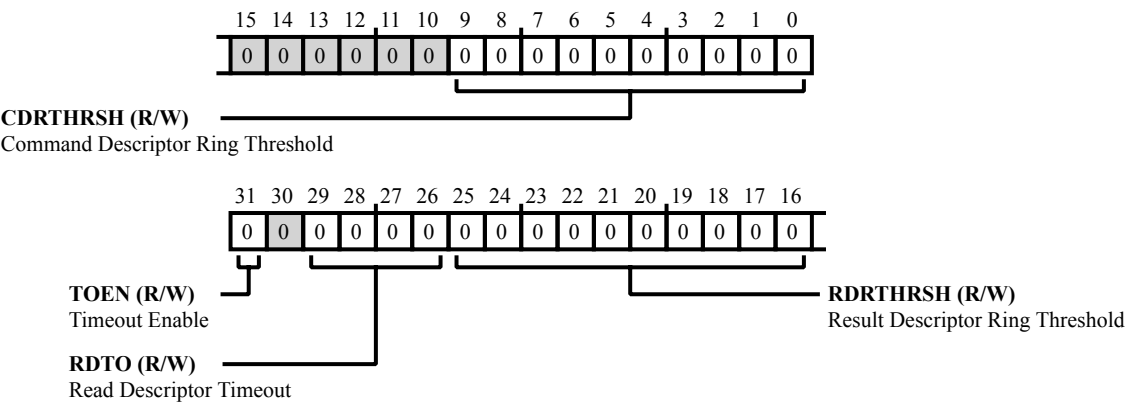


Figure 30-39: `PKTE_RING_THRESH` Register Diagram

Table 30-65: `PKTE_RING_THRESH` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	TOEN	Timeout Enable. A 1 in the <code>PKTE_RING_THRESH</code> . <code>TOEN</code> bit indicates the result descriptor timeout counter is enabled. This bit can be used to de-activate the timeout counter to save power.

Table 30-65: PKTE_RING_THRESH Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
29:26 (R/W)	RDTO	Read Descriptor Timeout. The timeout enable (PKTE_RING_THRESH.TOEN) bit in this register must be set to activate this PKTE_RING_THRESH.RDTO result descriptor timeout counter. The rdrthrsh interrupt activates when the RD counter for the RDR is non-zero for more than $2^{(N+10)}$ internal system clock cycles, where 'N' is the value set in this field. Valid settings range from 0 to 15. The minimum time-out value for N=0 is 1024 clock cycles and the maximum time-out value for N=15 is 33554432 clock cycles. At 100 MHz, this is 5.12 us for N=0 and ~335.55 ms for N=15. Note: The time-out delay may not be exact - expect a variation on the order of 1024 system clock cycles (just more than one microsecond at 100 MHz system clock frequency).
25:16 (R/W)	RDRTHRSH	Result Descriptor Ring Threshold. The rdrthrsh interrupt activates when the RD counter for the RDR exceeds the value set in the PKTE_RING_THRESH.RDRTHRSH field. Valid settings range from 0 to 1023.
9:0 (R/W)	CDRTHRSH	Command Descriptor Ring Threshold. The cdrthrsh interrupt activates when CD counter for the CDR is below or equal the value set in the PKTE_RING_THRESH.CDRTHRSH field. Valid settings range from 0 to 1023.

Packet Engine SA Address

The `PKTE_SA_ADDR` register holds the start address of the SA record.

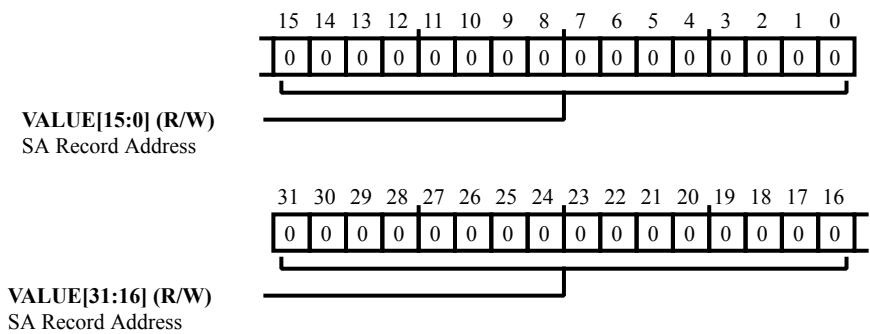


Figure 30-40: PKTE_SA_ADDR Register Diagram

Table 30-66: PKTE_SA_ADDR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	SA Record Address. The <code>PKTE_SA_ADDR.VALUE</code> bit field holds the start address of the SA record.

ARC4 i and j Pointer Register

When starting a new ARC4 operation the `PKTE_SA_ARC4IJPTR` register contains the initialization value, which is zeros. After processing the ARC4 algorithm it contains the latest status of the ARC4_IJ_PNTR.

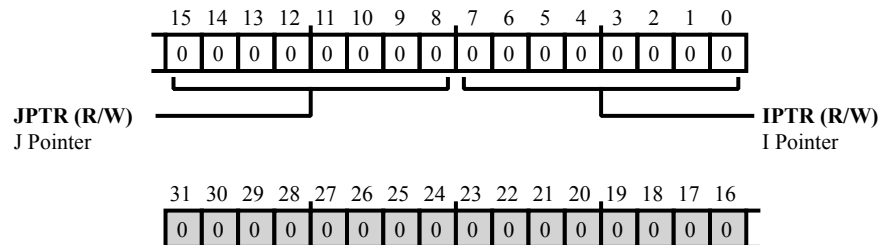


Figure 30-41: PKTE_SA_ARC4IJPTR Register Diagram

Table 30-67: PKTE_SA_ARC4IJPTR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:8 (R/W)	JPTR	J Pointer. The <code>PKTE_SA_ARC4IJPTR</code> . JPTR bit field contains the j pointer into s-box array for swapping bytes with i pointer.
7:0 (R/W)	IPTR	I Pointer. The <code>PKTE_SA_ARC4IJPTR</code> . IPTR bit field contains the i pointer into s-box array for swapping bytes with j pointer.

SA Command 0

The two SA command registers, `PKTE_SA_CMD0` and `PKTE_SA_CMD1`, are used to control the cryptographic operation of the packet engine. The `PKTE_SA_CMD0` register contains the major control bits to define an operation while the `PKTE_SA_CMD1` register contains the minor control bits. In direct host mode, this is a write-only register.

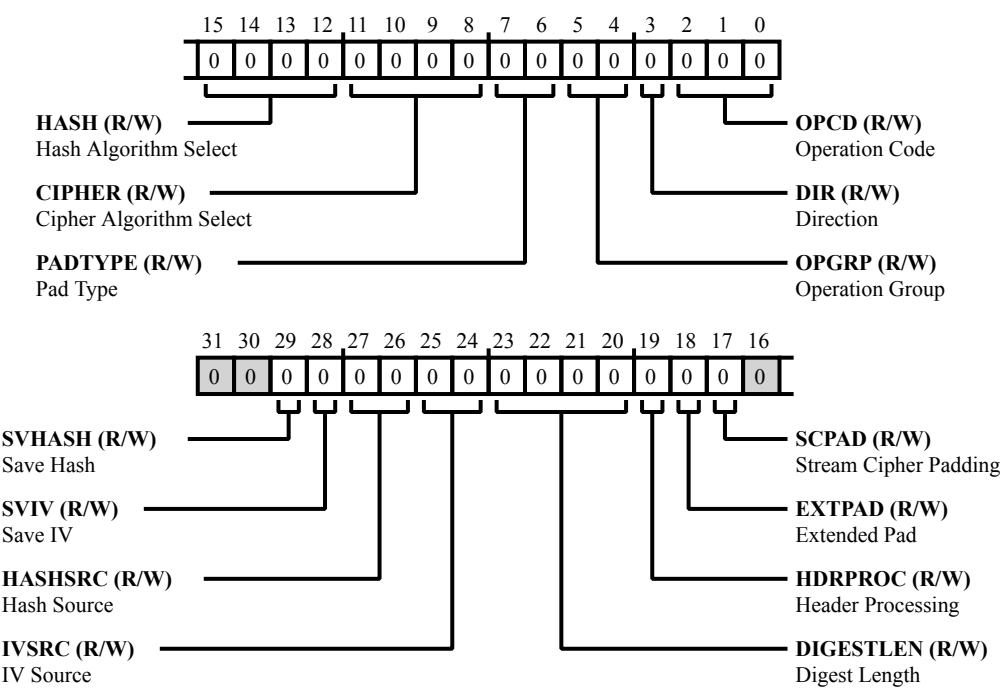


Figure 30-42: PKTE_SA_CMD0 Register Diagram

Table 30-68: PKTE_SA_CMD0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29 (R/W)	SVHASH	Save Hash. The <code>PKTE_SA_CMD0.SVHASH</code> bit indicates that the Hash State is saved to the <code>STATE_BYTE_CNT_X</code> and <code>STATE_IDIGEST_X</code> fields in the SA record in memory after completion of a crypto operation.
		0 Hash state is not saved
		1 Hash state is saved

Table 30-68: PKTE_SA_CMD0 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
28 (R/W)	SVIV	Save IV. The PKTE_SA_CMD0 . SVIV bit field indicates that for DES or the AES the Initialization Vector (IV) is saved to the STATE_IV_X fields in the state record , or the ARC4 state is saved to the ARC4 state record, after completion of the crypto operation.
		0 ARC4 State is not saved.
		1 ARC4 State is not saved.
27:26 (R/W)	HASHSRC	Hash Source. The PKTE_SA_CMD0 . HASHSRC bit field selects the source of the hash digest used by the algorithm.
		0 From SA. Digest only hash byte count is forced to 0x40.
		1 Reserved
		2 From State. Read saved inner hash digest and saved hash byte count.
		3 No Load. Use the hash algorithm defined constants for the initial hash. Hash byte count is 0x00.
25:24 (R/W)	IVSRC	IV Source. The PKTE_SA_CMD0 . IVSRC bit field selects the source of the initialization vector used by the crypto algorithm.
		0 No load. Use previous result IV, not applicable for inbound data. This option should never be used for operations with DES-CBC or AES-CBC, (see RFC3602) or any AES counter modes
		1 From input buffer. The IV is provided as part of the input data stream.
		2 From State. Read STATE_IV_X, from the SA structure. Refer to inner hash digest register structure. Useful for resume operations.
		3 From internal PRNG. Not applicable for inbound operations.
23:20 (R/W)	DIGESTLEN	Digest Length. The PKTE_SA_CMD0 . DIGESTLEN bit field defines the length of the hash digest in words as put in the output buffer.
		0 3 Words (96-bit output)
		1 1 Word
		2 2 Words

Table 30-68: PKTE_SA_CMD0 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
		3	3 Words (IPsec)
		4	4 Words (MD5 and AES-based hash)
		5	5 Words (SHA-1)
		6	6 Words
		7	7 Words (SHA-224)
		8	8 Words (SHA-256)
		9	Reserved
		10	10 bytes (SRTP and TLS)
		11-15	Reserved
19 (R/W)	HDRPROC	Header Processing. The PKTE_SA_CMD0.HDRPROC bit enables header processing for protocol operations. There is no header-processing support for basic SSL, basic TLS and SRTP protocol operations as defined in the protocol group (see the Crypto and Hash Algorithms section). This bit must be zero for these operations; however, the protocol header must be supplied to the packet engine since it is part of the hash calculation. Refer to the protocol specifications for more information about header-processing support for a protocol.	
		0	No header processing
		1	Header processing; insert the protocol header for out-bound operations, verify the protocol header for in-bound operations.
18 (R/W)	EXTPAD	Extended Pad. The PKTE_SA_CMD0.EXTPAD bit extends the number of padding types. Used in combination with PKTE_SA_CMD0.PADTYPE.	
17 (R/W)	SCPAD	Stream Cipher Padding. The PKTE_SA_CMD0.SCPAD bit enables padding for stream ciphers algorithms.	
15:12 (R/W)	HASH	Hash Algorithm Select. The PKTE_SA_CMD0.HASH bit field selects the hash algorithm.	
		0	MD5
		1	SHA-1
		2	SHA-224
		3	SHA-256
		4-14	Reserved
		15	Null

Table 30-68: PKTE_SA_CMD0 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
11:8 (R/W)	CIPHER	Cipher Algorithm Select. The PKTE_SA_CMD0.CIPHER bit field selects the cipher algorithm to be used for encryption and decryption. Note: Each type of protocol operation supports different sets of crypto algorithms. Refer to the Crypto and Hash Algorithms general processing section for details of the supported algorithms.
		0 DES
		1 Triple-DES
		2 ARC4
		3 AES
		4-14 Reserved
		15 Null
7:6 (R/W)	PADTYPE	Pad Type. The PKTE_SA_CMD0.PADTYPE bit field indicates the type of crypto that must be generated for outbound packets or checked for inbound packets.
		0 Select IPsec operation (if Bit 18=0); Reserved (if Bit 18=1)
		1 PKCS#7 (if Bit 18=0); Select TLS/DTLS Pad, required for TLS/DTLS operation (if Bit 18=1)
		2 Constant pad (if Bit 18=0); Select Constant SSL Pad, required for SSL operation (if Bit 18=1)
		3 Zero pad (if Bit 18=0), Reserved (if Bit 18=1)
5:4 (R/W)	OPGRP	Operation Group. The PKTE_SA_CMD0.OPGRP bit field defines the operation groups. Refer to the Basic Operations and Decoding section for more information.
		0 Basic operation group
		1 Protocol operation group
		2 Extended protocol operations group
		3 Reserved
3 (R/W)	DIR	Direction. The PKTE_SA_CMD0.DIR bit field selects the direction of operation.
		0 Outbound operations
		1 Inbound operations

Table 30-68: PKTE_SA_CMD0 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
2:0 (R/W)	OPCD	Operation Code. The PKTE_SA_CMD0.OPCD bit field selects the operation within the operation group.

SA Command 1

The `PKTE_SA_CMD1` register contains the minor control bits that define an operation. In direct host mode, this is a write-only register.

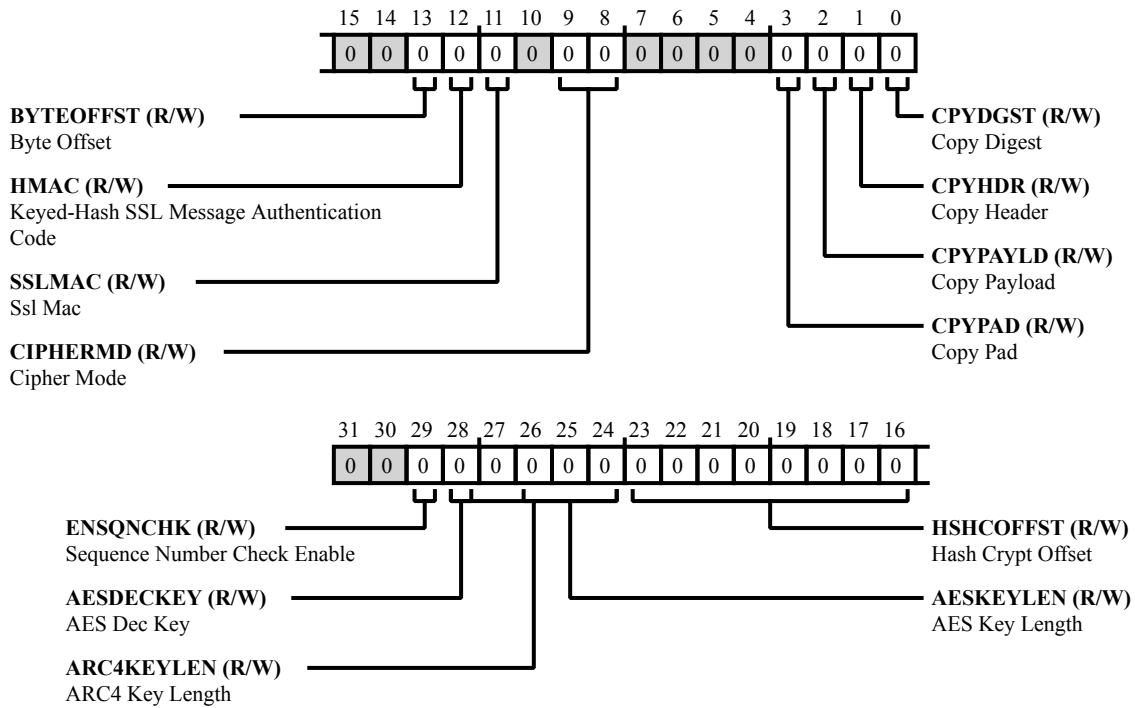


Figure 30-43: PKTE_SA_CMD1 Register Diagram

Table 30-69: PKTE_SA_CMD1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29 (R/W)	ENSQNCHK	Sequence Number Check Enable. The <code>PKTE_SA_CMD1.ENSQNCHK</code> bit defines that the key in the SA key field is an AES encrypt key or an AES decrypt key.
		0 Disable sequence number check
		1 Enable sequence number check
28 (R/W)	AESDECKEY	AES Dec Key. If the <code>PKTE_SA_CMD1.AESDECKEY</code> bit is set, the key in loaded in the <code>PKTE_SA_KEY[n]</code> registers are expected to be the key from the last round from key expansion. If not set, the key loaded in the <code>PKTE_SA_KEY[n]</code> registers are expected to be the same key used during the encryption process.
		0 AES key is an encrypt key.
		1 AES key is a decrypt key.

Table 30-69: PKTE_SA_CMD1 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
26:24 (R/W)	AESKEYLEN	AES Key Length. The PKTE_SA_CMD1.AESKEYLEN bit field select the size of the key used for the AES algorithm in increments of 64 bits.
		0-1 Reserved
		2 128 Bits
		3 192 Bits
		4 256 Bits
		5-7 Reserved
28:24 (R/W)	ARC4KEYLEN	ARC4 Key Length.
23:16 (R/W)	HSHCOFFST	Hash Crypt Offset. For Basic Encrypt-Hash and Basic Hash-Decrypt operations, the PKTE_SA_CMD1.HSHCOFFST bit field specifies the offset between the hash data and the encrypt/decrypt data. The data to be hashed is assumed to come first, with an offset to the beginning of encrypt/decrypt data. When PKTE_SA_CMD1.BYTEOFFST, bit 13, is zero, then the offset is defined in 32-bit words. When an initialization vector is loaded through the input buffer, valid values range from IV size to 255. In all other cases, valid values range from 0 to 255. When PKTE_SA_CMD1.BYTEOFFST, bit 13, is one, then the offset is defined in 8-bit bytes. When an initialization vector is loaded through the input buffer, valid values range from IV size to 255. In all other cases, valid values range from 4 to 255. (The IV size is two words for DES, Triple-DES and AES-CTR and four words for AES-CBC and AES-ICM operations). Other operations do not use these bits (a default value is applied by the packet engine).
13 (R/W)	BYTEOFFST	Byte Offset. The PKTE_SA_CMD1.BYTEOFFST bit defines how the PKTE_SA_CMD1.HSHCOFFST, bits of this register are used.
		0 HASH_CRYPT_OFFSET is defined in 32-bit words
		1 HASH_CRYPT_OFFSET is defined in 8-bit bytes
12 (R/W)	HMAC	Keyed-Hash SSL Message Authentication Code. For basic operations that include hashing, the PKTE_SA_CMD1.HMAC bit enables the HMAC processing, which calls for an extra outer hash operation.
		0 Standard Hash
		1 HMAC Processing

Table 30-69: PKTE_SA_CMD1 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
11 (R/W)	SSLMAC	Ssl Mac.
		0 Standard Hash
		1 SSL-MAC processing
9:8 (R/W)	CIPHERMD	Cipher Mode. The PKTE_SA_CMD1.CIPHERMD bit field selects the crypto mode to be used for the cipher algorithm.
		0 Electronic Code Book (ECB) used for DES and AES
		1 Cipher Block Chaining (CBC) used for DES and AES
		2 AES Counter Mode (CTR) for IPsec using a 32-bit counter
		3 AES Integer Counter Mode (ICM) for SRTP using a 16-bit counter. Note: This is implemented as a 32-bit counter, the Host must check for an overflow from the value 0xFF to zero and then refresh the IV.
3 (R/W)	CPYPAD	Copy Pad. The PKTE_SA_CMD1.CPYPAD bit indicates that the padding data for an inbound operation is copied to the output buffer and saved in memory.
		0 Do not copy the padding to output
		1 Copy padding to output
2 (R/W)	CPYPAYLD	Copy Payload. The PKTE_SA_CMD1.CPYPAYLD bit indicates that the payload data is copied to the output buffer and saved in memory.
		0 Do not copy the payload to output
		1 Copy payload to output
1 (R/W)	CPYHDR	Copy Header. The PKTE_SA_CMD1.CPYHDR bit indicates that the protocol header is copied to the output buffer and saved in memory. For Basic Encrypt-Hash and Basic Hash-Decrypt operations, the header is defined as the Hash/Crypt Offset data (authenticated only).
		0 Do not copy the header to output
		1 Copy header to output

Table 30-69: PKTE_SA_CMD1 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	CPYDGST	Copy Digest. The PKTE_SA_CMD1.CPYDGST bit copies the hash result is to the output buffer and saves in memory. The length of the hash result is defined by the PKTE_SA_CMD0.DIGESTLEN field.
		0 Do not copy hash result to output
		1 Copy hash result to output, when the command descriptor PKTE_CTL_STAT.HASHFINAL bit is set.

SA Inner Hash Digest Registers

The `PKTE_SA_IDIGEST[n]` registers are a set of eight 32-bit read/write registers.

For MD5, SHA-1, SHA-224 and SHA-256, these read/write registers are used to enter a start hash state, and to read the interim or final hash digest.

For IPsec, TLS and DTLS operations that make use of MD5, SHA-1, SHA-224 or SHA-256 with basic hash or HMAC authentication with the `PKTE_SA_CMD0.HASHSRC` bits = 00 (from SA), these registers hold the pre-computed inner hash digest. This is the hash of the hash-key padded with 0x36 hex. The starting hash byte count is automatically set to 64 decimal / 0x40 hex (to indicate that 64 bytes have already been processed through the hash).

For SSL operations that make use of SSL-MAC-MD5 with the `PKTE_SA_CMD0.HASHSRC` bits = 00 (from SA), these registers hold the inner hash pre-compute; this is the hash of the `MAC_WRITE_SECRET` padded with 0x36 hex. The starting hash byte count is automatically set to 64 decimal / 0x40 hex (to indicate that 64 bytes have already been processed through the hash).

For SSL operations that make use of SSL-MAC-SHA-1 with the `PKTE_SA_CMD0.HASHSRC` bits = 00 (from SA), these registers hold the `MAC_WRITE_SECRET`. Note that it is not possible to calculate a hash pre-compute for SHA-1 in combination with SSL-MAC (specification flaw). The packet engine appends the hash-key pad (0x36 hex) and sets the starting hash byte count automatically to 60 decimal / 0x3C hex (to indicate that 60 bytes have already been prepared for the hash).

The reset value for these registers is zero.

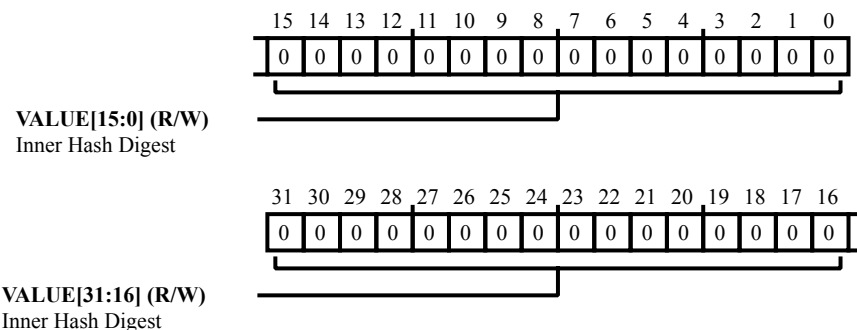


Figure 30-44: `PKTE_SA_IDIGEST[n]` Register Diagram

Table 30-70: `PKTE_SA_IDIGEST[n]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Inner Hash Digest.

SA Key Registers

These are the `PKTE_SA_KEY[n]` registers for DES, Triple-DES, ARC4 and AES: A set of eight 32-bit write only registers. The reset value of these registers is zero.

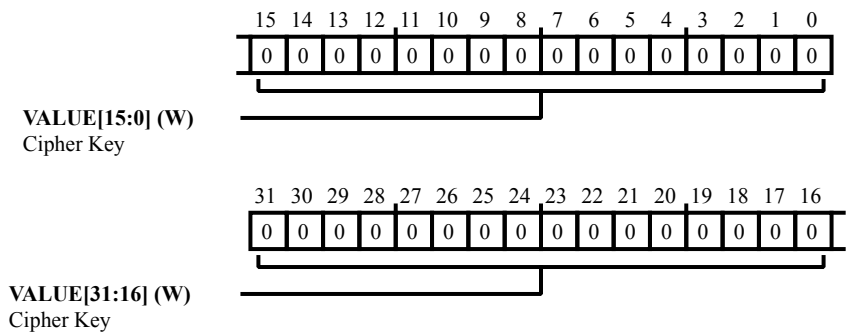


Figure 30-45: `PKTE_SA_KEY[n]` Register Diagram

Table 30-71: `PKTE_SA_KEY[n]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (RX/W)	VALUE	Cipher Key.

SA Initialization Vector Register

The `PKTE_SA_NONCE` register is used for operations that make use of the IV value loaded from the SA record. This register is used both to enter a starting IV state, as well as for reading the interim or final IV. For IPsec out-bound operations, it is recommended that the automatic IV insertion mode be used, this register is not needed. For IPsec inbound operations, the IV is extracted from the header of the packet.

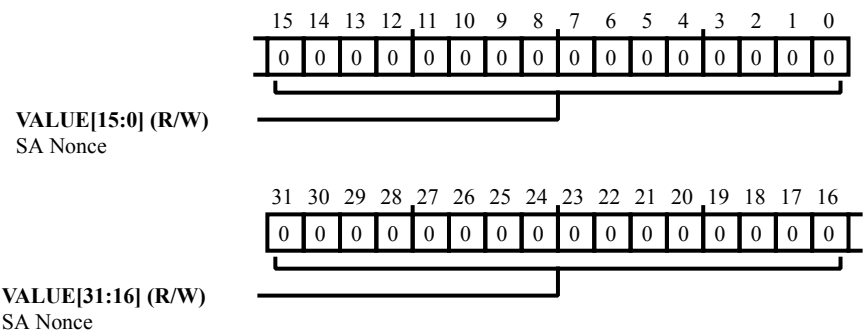


Figure 30-46: `PKTE_SA_NONCE` Register Diagram

Table 30-72: `PKTE_SA_NONCE` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	SA Nonce.

SA Outer Hash Digest Registers

The `PKTE_SA_ODIGEST[n]` registers are a set of five eight 32-bit write-only registers.

For write operations, these registers contain the pre-computed outer hash digest for IPsec operations with basic HMAC operations with the `PKTE_SA_CMD0.HASHSRC` bits = 00 (from SA).

For MD5, SHA-1, SHA-224 and SHA-256, these read/write registers hold a start hash state, or the interim outer hash digest. They are only used for HMAC processing.

For IPsec, SSL, TLS, DTLS and SRTP operations that make use of MD5, SHA-1, SHA-224 or SHA-256 with HMAC authentication with the `PKTE_SA_CMD0.HASHSRC` bits = 00 (from SA), these registers hold the pre-computed outer hash digest. This is the hash of the hash-key padded with 0x5C hex. The starting hash byte count is automatically set to 64 decimal / 0x40 hex (to indicate that 64 bytes have already been processed through the hash).

For SSL operations that make use of SSL-MAC-MD5 with the `PKTE_SA_CMD0.HASHSRC` bits = 00 (from SA), these registers hold the outer hash pre-compute; this is the hash of the `MAC_WRITE_SECRET` padded with 0x5C hex. The starting hash byte count is automatically set to 64 decimal / 0x40 hex (to indicate that 64 bytes have already been processed through the hash).

For SSL operations that make use of SSL-MAC-SHA-1 with the `PKTE_SA_CMD0.HASHSRC` bits = 00 (from SA), these registers hold the `MAC_WRITE_SECRET`. Note that it is not possible to calculate a hash pre-compute for SHA-1 in combination with SSL-MAC (specification flaw). The packet engine appends the required hash-key pad (0x5C hex) and sets the starting hash byte count automatically to 60 decimal / 0x3C hex (to indicate that 60 bytes have already been prepared for the hash).

The reset value for these registers is zero.

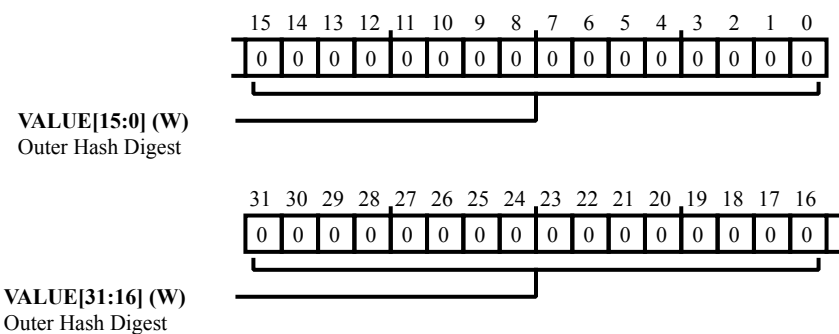


Figure 30-47: `PKTE_SA_ODIGEST[n]` Register Diagram

Table 30-73: `PKTE_SA_ODIGEST[n]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (RX/W)	VALUE	Outer Hash Digest.

SA Ready Indicator

In direct host mode, a write to the `PKTE_SA_RDY` register triggers the packet engine to start processing using the command descriptor, SA record and state record in the packet engine registers. This register **MUST** be written for all direct host mode packet operations. It is intended that this register is written in sequence; as the entire SA record is written.

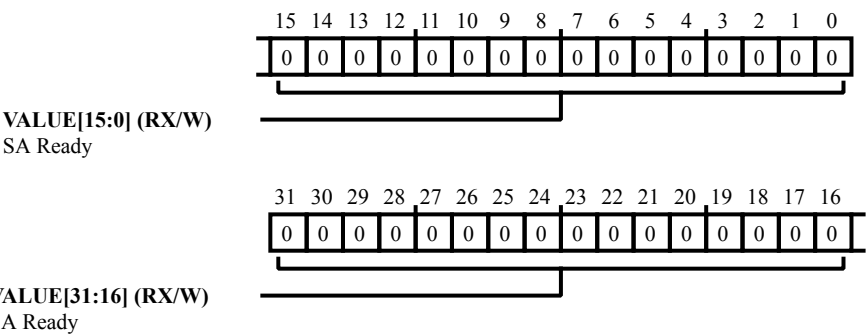


Figure 30-48: PKTE_SA_RDY Register Diagram

Table 30-74: PKTE_SA_RDY Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (RX/W)	VALUE	SA Ready.

SA Sequence Number Register

The `PKTE_SA_SEQNUM[n]` registers are a set of two read/write registers and are used for IPsec ESP, SSL, TLS, DTLS operations to specify the anti-replay sequence number value that is to be placed in the ESP header (out-bound), or to be checked against for inbound packets. The packet engine manages this counter value for both in-bound and outbound operations.

Outbound: The host writes the counter value stored in the SA record to this register to start an IPsec, SSL, TLS, DTLS operation. The packet engine automatically increments the count if header processing is selected. Upon successful completion, the host reads back this value and writes it to the SA record.

Inbound: The host writes the counter value stored in the SA record to this register to start an IPsec or DTLS operation. The packet engine automatically performs the specified inbound processing (per RFC 4303) as it processes the packet. As a result, the expected count value may or may not be updated during processing. Upon successful completion, the host should read back this value and write it to the SA record.

Note: The description is only for the direct host mode. The sequence number for autonomous ring mode and target command mode are updated by the packet engine.

The reset value of this register is zero.

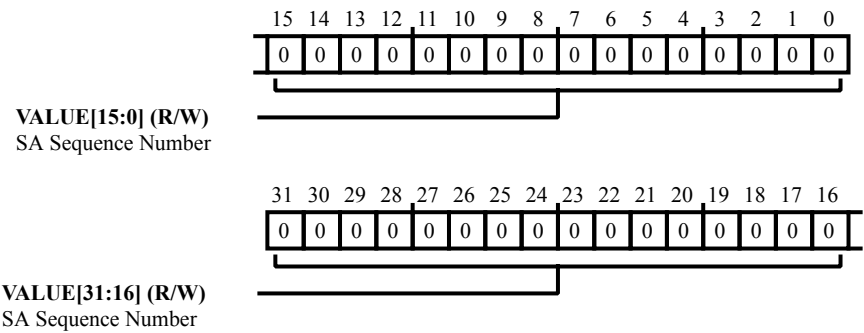


Figure 30-49: `PKTE_SA_SEQNUM[n]` Register Diagram

Table 30-75: `PKTE_SA_SEQNUM[n]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	SA Sequence Number.

SA Sequence Number Mask Registers

The `PKTE_SA_SEQNUM_MSK[n]` registers are a set of two read/write registers and are used for IPsec ESP and DTLS operations to specify the anti-replay sequence number mask value for inbound operations. The packet engine manages this counter value automatically.

Inbound: The host writes the counter value stored in the SA record into this register upon starting an IPsec, DTLS operation. The packet engine automatically performs the specified inbound processing (per RFC 4303) as it processes the packet. As a result, the new mask value may or may not be updated during processing. Upon successful completion, the host should read back this value and write it to the SA record.

Outbound: not used.

Note that the above description only applies to the direct host mode, for autonomous ring mode and target command mode the packet engine extracts the sequence number mask from the SA record.

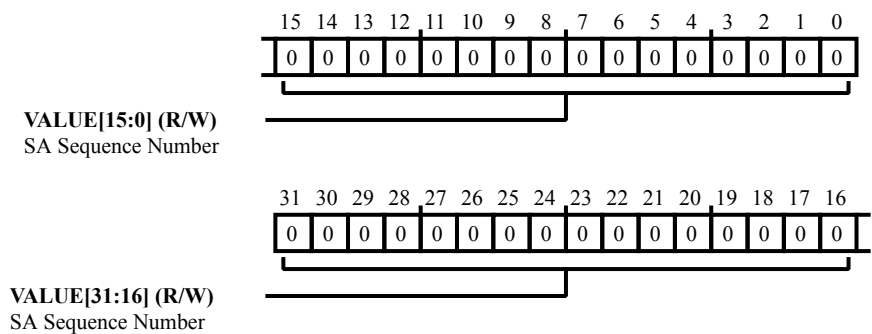


Figure 30-50: `PKTE_SA_SEQNUM_MSK[n]` Register Diagram

Table 30-76: `PKTE_SA_SEQNUM_MSK[n]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	SA Sequence Number.

SA SPI Register

For IPsec operations, the `PKTE_SA_SPI` register is written with the SPI (Security Parameters Index) associated with the inbound or outbound flow.

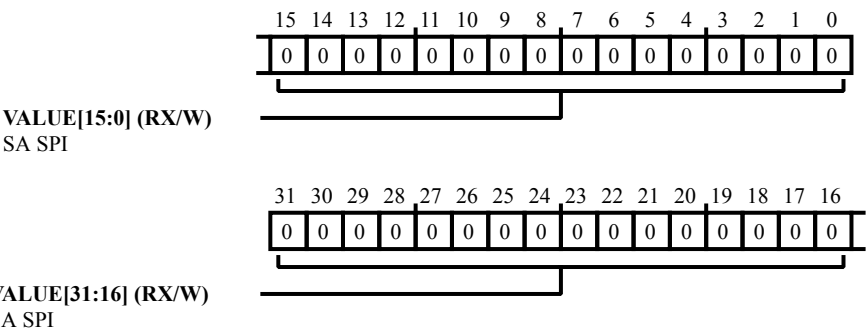


Figure 30-51: PKTE_SA_SPI Register Diagram

Table 30-77: PKTE_SA_SPI Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (RX/W)	VALUE	SA SPI. The <code>PKTE_SA_SPI.VALUE</code> bit field is used for IPsec ESP operations to specify the Security. Parameters Index (SPI) value that is to be placed in the ESP header. There is no need to read back this value at the end of an operation, since the Packet Engine does not change it. For SSL, TLS and DTLS this register stores the 8-bit TYPE field in bits [23:16], and the 16-bit Version field in bits [15:0] that are part of the protocol header.

Packet Engine Source Address

The `PKTE_SRC_ADDR` register holds the starting (byte) address for the packet to be processed.

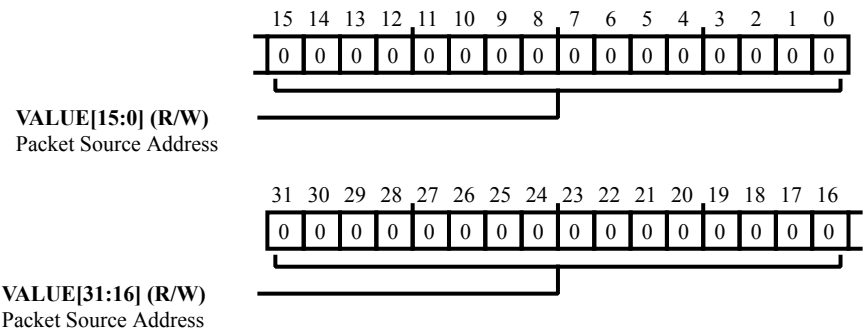


Figure 30-52: PKTE_SRC_ADDR Register Diagram

Table 30-78: PKTE_SRC_ADDR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Packet Source Address. The <code>PKTE_SRC_ADDR.VALUE</code> bit field holds the starting (byte) address for the packet to be processed.

Packet Engine Status Register

The `PKTE_STAT` register is used to provide the status of the packet engine. This register is useful in the direct host mode to determine when data must be written to or read from the packet engine, or for debugging the software when errors occur. This register can be ignored in autonomous ring mode and target command mode where the DMA engine controls the packet data I/O. This is a read-only register. A write to any of the bits has no effect.

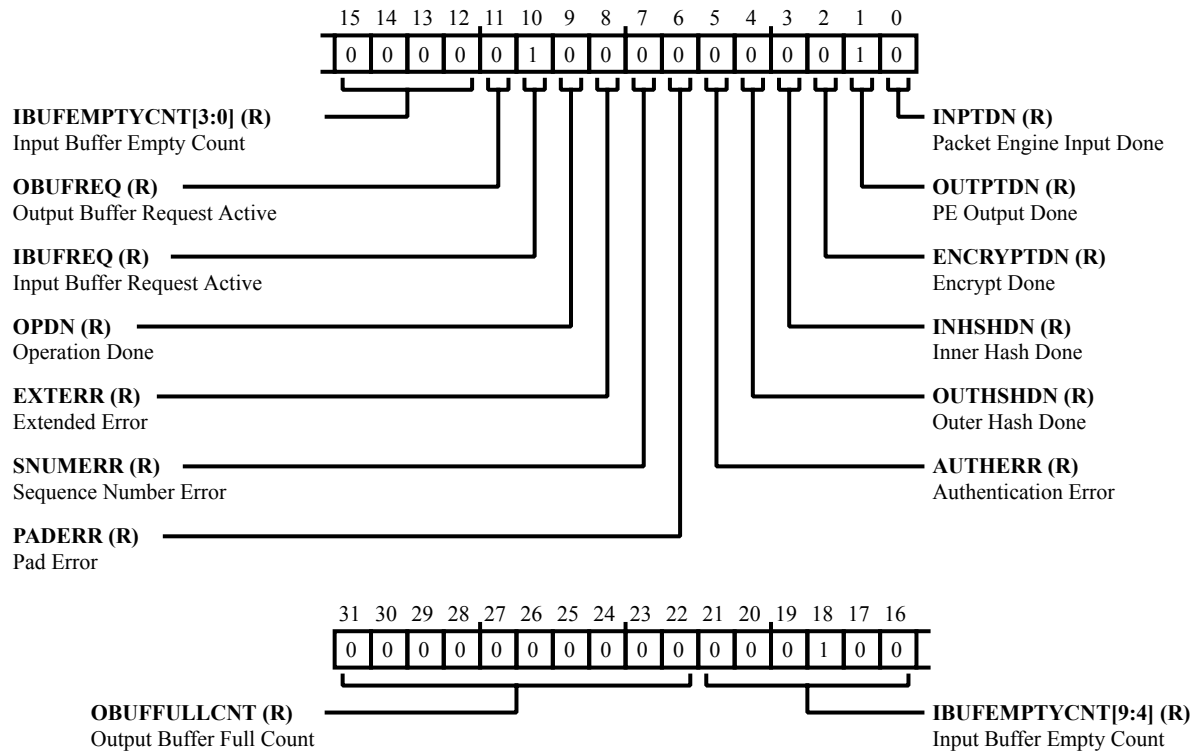


Figure 30-53: PKTE_STAT Register Diagram

Table 30-79: PKTE_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:22 (R/NW)	OBUFFULLCNT	Output Buffer Full Count. The <code>PKTE_STAT.OBUFFULLCNT</code> bit field indicates the number of 32-bit words that are available in the packet engine output buffer. It works in conjunction with bit 11 from this register. When bit 11 is asserted, to indicate a request for output, the word count matches the specified output buffer threshold setting in the <code>PKTE_BUF_THRESH</code> register. For the last output for a given packet, any value from 1 dword to the full output buffer threshold can be seen. Transfers must be a multiple of full dwords. The application must read the <code>PKTE_LEN</code> field in the result descriptor to determine the exact byte-length of the result.

Table 30-79: PKTE_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
21:12 (R/NW)	IBUFEMPTYCNT	Input Buffer Empty Count. The <code>PKTE_STAT . IBUFEMPTYCNT</code> bit field indicates the number of 32-bit empty spaces that are available in the packet engine input buffer. It works in conjunction with the <code>PKTE_STAT . IBUFREQ</code> bit (10) from this register. The value in the register is deducted from the specified packet length, so will never exceed the number of dwords that remain in the packet. For packets smaller than the buffer size, this register typically indicates that buffer space is available for the entire packet (rounded up to the nearest dword). For very large packets, these bits usually have a value around the maximum buffer size, indicating that the full input buffer is available.
11 (R/NW)	OBUFREQ	Output Buffer Request Active. The <code>PKTE_STAT . OBUFREQ</code> bit indicates that the packet engine requests output data to be read from the output buffer.
		0 No request for output data
		1 Request for output data
10 (R/NW)	IBUFREQ	Input Buffer Request Active. The <code>PKTE_STAT . IBUFREQ</code> bit indicates that the packet engine requests input data to be written to the input buffer.
		0 No request for input data
		1 Request for input data
9 (R/NW)	OPDN	Operation Done. The <code>PKTE_STAT . OPDN</code> bit indicates that the packet engine has finished processing a packet when in direct host mode. This bit is zero in autonomous ring mode and target command mode.
		0 Packet engine is idle
		1 Packet engine has finished processing a packet
8 (R/NW)	EXTERR	Extended Error. The <code>PKTE_STAT . EXTERR</code> bit indicates that an extended error occurred for this packet. For more information, refer to table Extended Error Codes - Status Encoding.
		0 No extended error
		1 Extended error

Table 30-79: PKTE_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
7 (R/NW)	SNUMERR	Sequence Number Error. For an inbound operation, the PKTE_STAT . SNUMERR bit indicates that there was a fault in the anti-replay sequence number. For an outbound operation, there was a sequence number overflow condition. For more information, refer to table Extended Error Codes - Status Encoding.
		0 No sequence number error
		1 Input done, all bytes written to input buffer
6 (R/NW)	PADERR	Pad Error. The PKTE_STAT . PADERR bit indicates that an inbound crypto pad fault is detected. For more information about pad verification, refer to the Pad Verification and Consumption section.
		0 No pad error
		1 Pad error
5 (R/NW)	AUTHERR	Authentication Error. The PKTE_STAT . AUTHERR bit indicates that an inbound ICV (for IPsec) or TAG (for SRTP) or MAC (for SSL/TLS/DTLS) fault is detected; the value carried within the packet did not match the value just computed.
		0 No authentication error
		1 Authentication error
4 (R/NW)	OUTHSHDN	Outer Hash Done. The PKTE_STAT . OUTHSHDN bit indicates that the outer hash processing for this packet is finished.
		0 Outer hash busy
		1 Outer hash done
3 (R/NW)	INHSHDN	Inner Hash Done. The PKTE_STAT . INHSHDN bit indicates that the inner hash processing for this packet is finished.
		0 Inner hash busy
		1 Inner hash done
2 (R/NW)	ENCRYPTDN	Encrypt Done. The PKTE_STAT . ENCRYPTDN bit indicates that the encryption or decryption for this packet is finished.
		0 Encryption or decryption busy
		1 Encryption or decryption done

Table 30-79: PKTE_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/NW)	OUTPTDN	PE Output Done. The <code>PKTE_STAT.OUTPTDN</code> bit indicates that the output data for the current packet is read from the packet engine output buffer.
		0 Output not done, more output bytes available
		1 Output done, all bytes read from the output buffer
0 (R/NW)	INPTDN	Packet Engine Input Done. The <code>PKTE_STAT.INPTDN</code> bit indicates that the number of bytes specified in the command descriptor <code>PKTE_LEN</code> field is written into the packet engine input buffer.
		0 Input not done, more input bytes expected
		1 Input done, all bytes written to input buffer

Packet Engine State Record Address

The `PKTE_STATE_ADDR` register holds the start address of the SA state record.

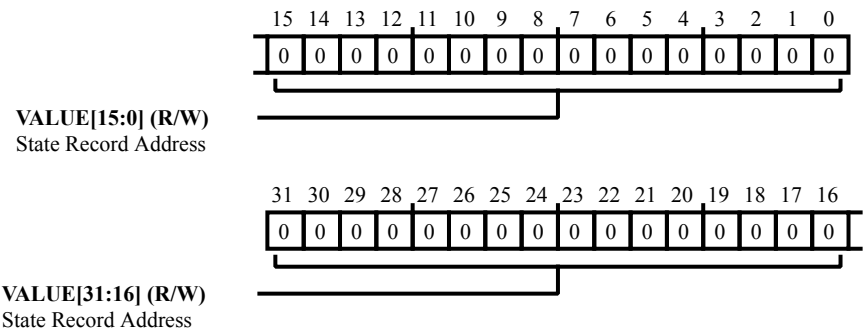


Figure 30-54: PKTE_STATE_ADDR Register Diagram

Table 30-80: PKTE_STATE_ADDR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	State Record Address. The <code>PKTE_STATE_ADDR.VALUE</code> bit field holds the start address of the SA state record.

State Hash Byte Count Registers

The `PKTE_STATE_BYTE_CNT[n]` registers are used to enter a starting hash byte count, as well as to read the interim or final byte count.

For some hash operations, these registers are ignored and the byte count is internally set to 64 (0x40 hex) to indicate that the first 64 bytes (512 bits) hash block has been processed using a pre-computed hash state. These operations are:

All IPsec, SSL, TLS, DTLS and SRTP operations that use authentication; the "pre-computed" inner and outer hash digests are loaded from SA words 10 - 19.

Basic operations with `PKTE_SA_CMD0.HASHSRC` bits = 00 (from SA) specified. For Basic Hash with no HMAC, a pre-computed digest is loaded from SA words 10 - 14. For Basic Hash with HMAC, the inner and outer digests are loaded from SA words 10 - 19.

Note: Protocol operations can not be suspended in mid-packet and resumed later, therefore protocol operations do not use these registers.

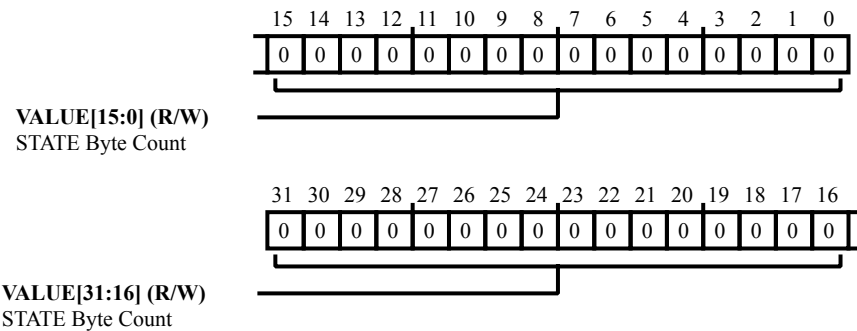


Figure 30-55: `PKTE_STATE_BYTE_CNT[n]` Register Diagram

Table 30-81: `PKTE_STATE_BYTE_CNT[n]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	STATE Byte Count.

State Inner Digest Registers

The `PKTE_STATE_IDIGEST[n]` registers consist of eight 32-bit registers. These read/write registers are used to read the interim or final hash digest. The `PKTE_STATE_IDIGEST[n]` registers are only used with basic operations involving basic hash, and are typically used for operations that must be suspended and resumed in the middle of a hash. The interim hash state can be read from these registers along with the hash byte-count from the previous register. Both can be restored when resuming the hash. The appropriate save hash state (`PKTE_SA_CMD0.SVHASH=1`) and load hash from state (`PKTE_SA_CMD0.HASHSRC=0b10`) settings must be used. These registers are a mirror of the SA record inner hash digest register.

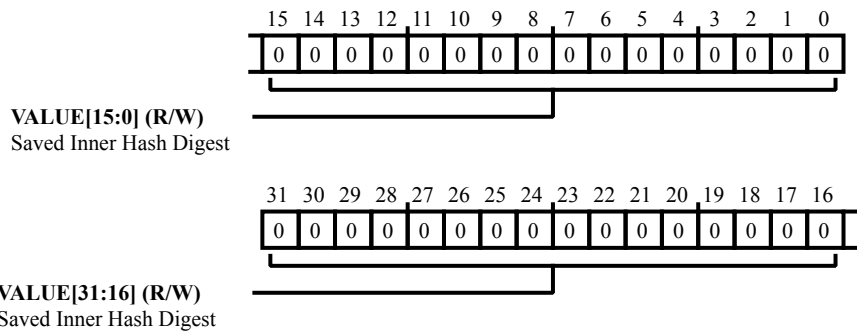


Figure 30-56: `PKTE_STATE_IDIGEST[n]` Register Diagram

Table 30-82: `PKTE_STATE_IDIGEST[n]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Saved Inner Hash Digest.

State Initialization Vector Registers

The `PKTE_STATE_IV[n]` consists of four 32-bit registers. These registers are used to enter a starting IV state and to read the interim or final IV. `PKTE_STATE_IV0` and `PKTE_STATE_IV1` are used with DES/3DES cipher while `PKTE_STATE_IV0` to `PKTE_STATE_IV3` are used with AES cipher. The reset value of these registers is zero.

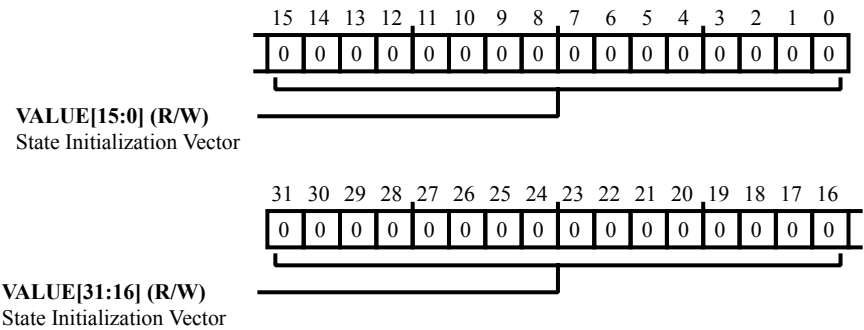


Figure 30-57: `PKTE_STATE_IV[n]` Register Diagram

Table 30-83: `PKTE_STATE_IV[n]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	State Initialization Vector. The <code>PKTE_STATE_IV[n].VALUE</code> bit field is used to enter a starting IV state and to read the interim or final IV.

Packet Engine User ID

The `PKTE_USERID` register is a read/write register that gives identification to a command descriptor and the resultant result descriptor. The host is free to use this field for its own purpose. The host can write a unique identifier to the register in direct host mode or includes it as part of the command descriptor in autonomous ring mode. The `PKTE_USERID` register value passes through the packet engine without alteration to the result descriptor to be read back by the host.

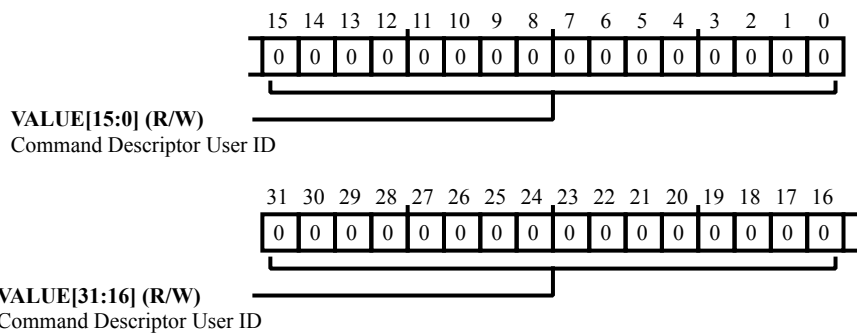


Figure 30-58: PKTE_USERID Register Diagram

Table 30-84: PKTE_USERID Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Command Descriptor User ID. The <code>PKTE_USERID.VALUE</code> bit field gives identification to a command descriptor and the resultant result descriptor.

31 Public Key Accelerator (PKA)

The PKA helps offload computationally-intensive operations commonly found in public key cryptography algorithms.

PKA Features

The PKA engine provides the following basic operations:

- Large vector addition, subtraction, and combined addition/subtraction
- Large vector shift right or left
- Large vector multiplication, division (with and without quotient)
- Large vector compare and copy

The PKA engine provides the following complex operations:

- Large vector unsigned value modular exponentiation
- Large vector unsigned value modular exponentiation using the ‘Chinese Remainders Theorem’ (CRT) method with pre-calculated Q inverse vector
- Modular inversion: Given A and M, calculate B such that $((A \times B) \text{ MOD } M) = 1$
- ECC point addition/doubling on elliptic curve $y^2 = x^3 + ax + b \pmod{p}$ with prime number p and input values a and b to the operation. Adding two identical points automatically performs point doubling.
- ECC point multiplication on elliptic curve $y^2 = x^3 + ax + b \pmod{p}$ with prime number p and input values a and b to the operation. A version of the ‘Montgomery ladder’ algorithm is used to provide side channel attack resistance.

The PKA also contains hardware logic to automatically zero out the PKA RAM buffer to clear out any information that is considered sensitive or secure.

PKA Functional Description

The following sections provide details on the function of the PKA module.

ADSP-21560_21561_21564_21568 PKA Register List

The Public Key Accelerator module (PKA) provides security-related features. A set of registers governs PKA operations. For more information on PKA functionality, see the PKA register descriptions.

Table 31-1: ADSP-21560_21561_21564_21568 PKA Register List

Name	Description
PKA_ALEN	PKA Vector_A Length
PKA_APTR	PKA Vector_A Address
PKA_BLEN	PKA Vector_B Length
PKA_BPTR	PKA Vector_B Address
PKA_COMPARE	PKA Compare Result
PKA_CPTR	PKA Vector_C Address
PKA_DIVMSW	PKA Most-Significant-Word of Divide Remainder
PKA_DPTR	PKA Vector_D Address
PKA_FUNC	PKA Function
PKA_RAM	Start of PKA RAM space
PKA_RESULTMSW	PKA Most-Significant-Word of Result Vector
PKA_SHIFT	PKA Bit Shift Value

PKA Definitions

The following definitions are helpful when using the PKA module.

Elliptic Curve Cryptography (ECC)

A form of public key cryptography based on elliptic curves over finite fields.

RSA

An acronym for Ron Rivest, Adi Shamir, and Leonard Adleman. It is another form of a public key cryptosystem.

Chinese Remainder Theorem (CRT)

A mathematical theorem used for simplifying time-consuming arithmetic used in public key algorithm computations.

Addition Chaining Table (ACT)

A method of speeding up exponentiation by repeatedly squaring the input, storing the result, and reusing the result as input. ACT2 uses a table with 2 address bits (4 entries) and ACT4 uses a table with 4 address bits (16 entries).

PKA Architectural Concepts

The following sections describe the PKA architecture.

Public Key Co-Processor (PKCP)

The Public Key Co-Processor (PKCP) handles the basic large vector processing such as addition, subtraction, multiplication, etc.

Sequencer

The sequencer is small processor that is part of the PKA which handles the more complicated vector processing for public key algorithms. Algorithms include modular exponentiation and the ECC addition and ECC multiply used in Elliptic Curve Cipher algorithms. It executes instructions stored from an internal pre-programmed ROM that handles these operations.

RAM

Input and output vectors are stored in a 4 kB RAM buffer that is part of the MMR space. The address of PKA_RAM is the beginning of the RAM space. This memory is also used as a scratchpad or workspace for the sequencer and PKCP. Programs must place the vectors appropriately following the constraints described in the *Functional Description* section of this chapter.

PKA Block Diagram

The *PKA Block Diagram* shows the top-level block diagram of the PKA engine. The PKA engine is comprised of five parts:

1. Registers for input, output, status, and control
2. Public Key Co-processor (PKCP) module which performs the basic suite of big number (vector) operations typically found in public key cryptography applications
3. Sequencer which controls modular exponentiation, elliptic curve cryptography, and modular inversion operations.
4. Program ROM associated with the PKA engine exclusively for the sequencer
5. PKA RAM holds the large input and output values as well as the workspace/scratchpad required from the sequencer and PKCP for operations.

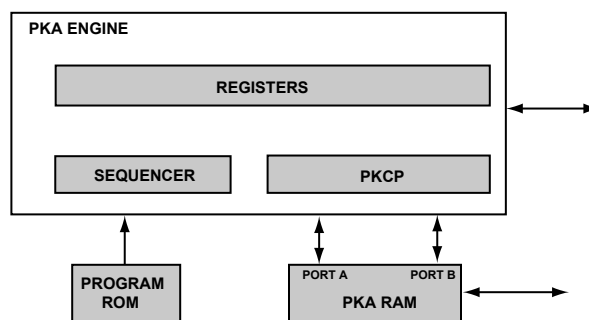


Figure 31-1: PKA Block Diagram

PKCP Vector Operations

The *Summary of PKCP Vector Operations* table lists the arguments and results for each PKCP vector operation.

Table 31-2: Summary of PKCP Vector Operations

Function	Mathematical Operation	Vector A	Vector B	Vector C	Vector D
Multiply	$A \times B \rightarrow C$	Multiplicand	Multiplier	Product	N/A
Add	$A + B \rightarrow C$	Addend	Addend	Sum	N/A
Subtract	$A - B \rightarrow C$	Minuend	Subtracthend	Difference	N/A
AddSub	$A + C - B \rightarrow D$	Addend	Subtracthend	Addend	Result
Right Shift	$A \gg \text{Shift} \rightarrow C$	Input	N/A	Result	N/A
Left Shift	$A \ll \text{Shift} \rightarrow C$	Input	N/A	Result	N/A
Divide	$A \bmod B \rightarrow C$, $A \div B \rightarrow D$	Dividend	Divisor	Remainder	Quotient
Modulo	$A \bmod B \rightarrow C$	Dividend	Divisor	Remainder	N/A
Compare	$A = B$, $A < B$, $A > B$	Input 1	Input 2	N/A	N/A
Copy	$A \rightarrow C$	Input	N/A	Result	N/A

To obtain correct result, the input vectors must meet the requirements presented in the *Operational Restrictions on Input Vectors for PKCP Operations* table.

Note the following:

- The PKCP does not check input restrictions
- A_Len and B_Len indicate the size of vectors A and B in 32-bit words
- Max_Len equals 128 (32-bit) words, for example, the standard maximum vector size is 4096 bits

Table 31-3: Operational Restrictions on Input Vectors for PKCP Operations

Function	Requirement
Multiply	$0 < A_Len, B_Len \leq Max_Len$
Add	$0 < A_Len, B_Len \leq Max_Len$
Subtract	$0 < A_Len, B_Len \leq Max_Len$ Result must be positive ($A > B$)
AddSub	$0 < A_Len \leq Max_Len$ (B and C operands have A_Len as length, B_Len ignored) Result must be positive ($(A + C) > B$)
Right Shift	$0 < A_Len \leq Max_Len$
Left Shift	$0 < A_Len \leq Max_Len$
Divide, Modulo	$1 < B_Len \leq A_Len \leq Max_Len$ Most significant 32-bit word of B operand cannot be zero
Compare	$0 < A_Len \leq Max_Len$ (B operand has A_Len as length, B_Len ignored)
Copy	$0 < A_Len \leq Max_Len$

The host processor is responsible for allocating a block of contiguous memory in PKA RAM for the result vectors. The *PKCP Result Vector Memory Allocation* table indicates how much memory is allocated for the result vectors.

Table 31-4: PKCP Result Vector Memory Allocation

Function	Result Vector	Result Vector Length (in 32-bit words)
Multiply	C	$A_Len + B_Len + 6$ (the 6 'scratchpad' words should be discarded)
Add	C	$Max(A_Len, B_Len) + 1$
Subtract	C	$Max(A_Len, B_Len)$
AddSub	D	$A_Len + 1$
Right Shift	C	A_Len
Left Shift	C	$A_Len + 1$ (when Shift Value is non-zero) A_Len (when Shift Value is zero)
Divide	C	Remainder $\rightarrow B_Len + 1$ (one 'scratchpad' word should be discarded)
	D	Quotient $\rightarrow A_Len - B_Len + 1$
Modulo	C	Remainder $\rightarrow B_Len + 1$ (one 'scratchpad' word should be discarded)
Compare	None	Compare updates the PKA_COMPARE register
Copy	C	A_Len

Input vectors for an operation are always allowed to overlap in memory (partially or completely). The *PKCP Result Vector/Input Overlap Restrictions* table gives restrictions for the overlap of output and input vectors of the operations.

Table 31-5: PKCP Result Vector/Input Overlap Restrictions

Function	Result Vector	Restrictions
Multiply	C	No overlap with A or B vectors allowed
Add, Subtract	C	May overlap with A and/or B vector, provided the start address of the C vector does not lie above the start address of the vectors with which it overlaps
AddSub	D	May overlap with A, B and/or C vector, provided the start address of the D vector does not lie above the start address of the vectors with which it overlaps
Right Shift, Left Shift	C	May overlap with A vector, provided the start address of the C vector does not lie above the start address of the A vector
Divide	C	No overlap with A, B, or D vectors allowed
	D	No overlap with A, B, or C vectors allowed
Modulo	C	No overlap with A or B vectors allowed
Compare	None	Compare does not write a result vector
Copy	None	Same restrictions as for right or left shift, copy of a vector to a lower address is always allowed even if source and destination overlap†

†The copy operation can be used to fill memory by breaking the overlap restrictions, but it requires setting up TWO initial (32-bit) words. To zero a block of memory, set the A vector pointer to the block start, set the C vector pointer two words higher and the A vector length to the block length minus two (words). Fill the first two words of the block with constant zero and perform a PKCP copy operation to zero the remainder of the block.

Modular Exponentiation Operations

The *Summary of ExpMod Operations* table summarizes the modular exponentiation operations that the PKA supports.

Table 31-6: Summary of ExpMod Operations

Function	Mathematical Operation	Vector A	Vector B	Vector C	Vector D
ExpMod-ACT2 ExpMod-ACT4 ExpMod-variable	$C^A \bmod B \rightarrow D$	Exponent, length = A_Len	Modulus, length = B_Len	Base, length = B_Len	Result and Workspace
ExpMod-CRT	See below	Exp P followed by Exp Q at next higher even word address†, both A_Len long	Mod P + buffer word followed by Mod Q at next higher even word address†, both B_Len long	Q inverse, length = B_Len	Input, Result (both 2xB_Len long) and Workspace

† If A_Len is even, Exp Q follows Exp P immediately – if A_Len is odd, there is one empty word between Exp Q and Exp P.

‡ If B_Len is even, there are two empty words between Mod P and Mod Q – if B_Len is odd, there is one empty (buffer) word between Mod Q and Mod P. Note that the engine may zero the words following Mod P and Mod Q.

The ExpMod-CRT operation performs the following computation steps. (These steps implement Garner's recombination algorithm after the basic exponentiations.)

- $X \leftarrow (\text{Input mod Mod P}) \text{Exp P mod Mod P}$
- $Y \leftarrow (\text{Input mod Mod Q}) \text{Exp Q mod Mod Q}$
- $Z \leftarrow (((X - Y) \text{ mod Mod P}) \cdot Q \text{ inverse}) \text{ mod Mod P} \cdot \text{Mod Q}$
- $\text{Result} \leftarrow Y + Z$

The ExpMod-ACT2, -ACT4, and -variable functions implement the same mathematical operation but with a differently sized table with pre-calculated *odd powers*. The ExpMod-ACT2 function uses a table with two entries whereas ExpMod-ACT4 uses a table with eight entries. The ACT4 version gives better performance but needs more memory. ExpMod-variable and ExpMod-CRT operations allow the selection of a variable number (from 1 up to and including 16) of odd powers through the register normally used to specify the number of bits to shift for shift operations.

The exponentiation functions are extensions of the set of PKA functions. Input and result vectors are passed the same way as basic PKCP operations. The *Restrictions on Input Vectors for ExpMod Operations* table shows the restrictions on the input and result vectors for the exponentiation operations.

Table 31-7: Restrictions on Input Vectors for ExpMod Operations

Function	Requirements
ExpMod-ACT2 ExpMod-ACT4 ExpMod-variable	$0 < A_Len \leq \text{Max_Len}$ $1 < B_Len \leq \text{Max_Len}$ Modulus B must be odd (for example, the least significant bit must be ONE) Modulus $B > 2^{32}$ Base $C < \text{Modulus B}$ Vectors B and C must be followed by an empty 32-bit <i>buffer</i> word
ExpMod-CRT	$0 < A_Len \leq \text{Max_Len}$ $1 < B_Len \leq \text{Max_Len}$ Mod P and Mod Q must be odd (for example, the least significant bits must be ONE) $\text{Mod P} > \text{Mod Q} > 2^{32}$ (note that Mod P must be larger than Mod Q) Mod P and Mod Q must be co-prime (their GCD must be 1) $0 < \text{Exp P} < (\text{Mod P} - 1)$ $0 < \text{Exp Q} < (\text{Mod Q} - 1)$ $(Q \text{ inverse} \cdot \text{Mod Q}) \cdot 1 \pmod{\text{Mod P}}$ Input $< (\text{Mod P} \cdot \text{Mod Q})$ Mod P and Mod Q must be followed by an empty 32-bit <i>buffer</i> word

The *ExpMod Result Vector/Scratchpad Area Memory Allocation Starting at PKA_DPTR* table shows the required scratchpad sizes for the exponentiation operations. These sizes depend on the PKA type. The ‘M_Len’ used in the table is the ‘real’ Modulus length in 32-bit words, for example, without trailing zero words at the end. (This description also applies to Mod P in an ExpMod-CRT operation and Modulus B in the other operations.) If the last word of the modulus vector as given is non-zero, ‘M_Len’ equals B_Len.

Table 31-8: ExpMod Result Vector/Scratchpad Area Memory Allocation Starting at PKA_DPTR

Function	Scratchpad Area Size (in 32-bit words), Result Vector is either M_Len or 2xM_Len 32-bit words long
ExpMod-ACT2	5 x (M_Len + 2)
ExpMod-ACT4	11 x (M_Len + 2)
ExpMod-variable	(# odd powers + 3) x (M_Len + 2)
ExpMod-CRT	(# odd powers + 3) x (M_Len + 2) + (M_Len + 2 – (M_Len MOD 2))

NOTE: During execution of an ExpMod-ACT2, -ACT4 or -variable operation, the last 34 bytes of the PKA RAM are used as the general scratchpad for the sequencer program execution. The ExpMod-CRT operation requires the last 72 bytes of the PKA RAM as the scratchpad. These (fixed location) areas may not overlap with any of the input vectors and/or the D vector scratchpad area. They can be used freely when executing basic PKCP operations.

Table 31-9: ExpMod Scratchpad Area / Input Vector Overlap Restrictions

Function	Result Vector	Restrictions
ExpMod-ACT2 ExpMod-ACT4 ExpMod-variable	D	Scratchpad area starting at D may not overlap with any of the other vectors, except that base C may be co-located with result vector D to save space (for example, PKA_CPTR = PKA_DPTR is allowed).
ExpMod-CRT	D	Scratchpad area starting at D may not overlap with any of the other vectors. This area is also the location of the main input vector (with length 2 x B_Len)

The *Maximum Number of Odd Powers* table indicates the maximum number of odd powers that can be used for different standard PKA RAM sizes and PKA types (non-CRT operations using [PKA_CPTR](#) = [PKA_DPTR](#)). As a rule of thumb, for optimal performance, use one odd power for *Verify* operations and 4 (or as many as the implemented PKA RAM size allows) for *Sign* operations. Note the following points about odd powers:

- Using more than eight odd powers is not advisable as the speed advantage for each extra odd power decreases rapidly (and can even become negative for short exponent vector lengths due to the extra pre-processing required).
- The maximum number of odd powers is 16 (limited by the firmware). All ‘16 odd powers’ entries in the table above hit this limit – they are not limited by the PKA RAM size.

Table 31-10: Maximum Number of Odd Powers

Operation	Modulus and Exponent Sizes	Maximum Number of Odd Powers
Non-CRT	1024 bits	16
	2048 bits	10
	4096 bits	2
CRT	2 × 512 bits	16
	2 × 1024 bits	16
	2 × 2048 bits	6

The *2K-bit Modular Exponentiation PKA RAM Allocation Examples* table shows example PKA RAM vector allocations for modular exponentiation operations with and without using CRT. The *free space* start address is the first free byte following the vector workspace. The sequencer execution scratchpad of 34 bytes (non-CRT) or 72 bytes (using CRT) must fit between this address and the end of the PKA RAM. Note that the non-CRT operations use `PKA_CPTR = PKA_DPTR` to save space.

Table 31-11: 2K-bit Modular Exponentiation PKA RAM Allocation Examples

Operation	(sub-)vector	Start address (Byte Offset)	Size (words)	Buffer (words)
non-CRT (<code>PKA_ALEN</code> = 0x040, <code>PKA_BLEN</code> = 0x040, 4 odd-powers)	Exponent	0x000 (<code>PKA_APTR</code> = 0x000)	64	0
	Modulus	0x100 (<code>PKA_BPTR</code> = 0x040)	64	2
	Base	0x208 (<code>PKA_CPTR</code> = 0x082)	64	2
	Result	0x208 (<code>PKA_DPTR</code> = 0x082)	64	2
	Vector Workspace	0x208 (= Result)	7 × (64+2)=462	0
	Free space	0x940 (2368 bytes used)	-	-
using CRT (<code>PKA_ALEN</code> = 0x020, <code>PKA_BLEN</code> = 0x020, 4 odd-powers)	Exp P	0x000 (<code>PKA_APTR</code> = 0x000)	32	0
	Exp Q	0x080	32	0
	Mod P	0x100 (<code>PKA_BPTR</code> = 0x040)	32	2
	Mod Q	0x188	32	2
	Q inverse	0x210 (<code>PKA_CPTR</code> = 0x084)	32	0
	Input, Result	0x290 (<code>PKA_DPTR</code> = 0x0A4)	64	0
	Vector workspace	0x290 (= Result)	7 × (32+2)+32+2-0 = 272	0
	Free space	0x6D0 (1744 bytes used)	-	-

The following example in pseudo-code describes the execution of a non-CRT modular exponentiation operation using a 512-bit modulus and a 160-bit exponent, using actual test vectors:

```
// Perform a 512/160 bit
modular exponentiation without CRT (using 4 'odd-powers')
// Exponent equals value 0x8FD84098_8A0930CC_9CDC1E8A_B246EB46_2D39F064
// write as vector A to PKA
```

```

RAM Byte offset 0x000:
Write PKA_RAM_BASE+0x000+0x00
0x2D39F064
Write PKA_RAM_BASE+0x000+0x04
0xB246EB46
Write PKA_RAM_BASE+0x000+0x08
0x9CDC1E8A
Write PKA_RAM_BASE+0x000+0x0C
0x8A0930CC
Write PKA_RAM_BASE+0x000+0x10
0x8FD84098
// Modulus equals value 0xF42F559D
1877CA5F_449492B9_42DC7C01_...
// A3C9085B_7236A085_2102B000_A093C6B4_...
// 9D0EDA0C_292DE841_29C23723_4048BDA3_...
// 373C4C9F_45CF15A7_5F049ABF_D8A01B9B
// write as vector B to PKA
RAM Byte offset 0x018 (following exp at next aligned 64-bit word):
Write PKA_RAM_BASE+0x018+0x00
0xD8A01B9B
Write PKA_RAM_BASE+0x018+0x04
0x5F049ABF
Write PKA_RAM_BASE+0x018+0x08
0x45CF15A7
Write PKA_RAM_BASE+0x018+0x0C
0x373C4C9F
Write PKA_RAM_BASE+0x018+0x10
0x4048BDA3
Write PKA_RAM_BASE+0x018+0x14
0x29C23723
Write PKA_RAM_BASE+0x018+0x18
0x292DE841
Write PKA_RAM_BASE+0x018+0x1C
0x9D0EDA0C
Write PKA_RAM_BASE+0x018+0x20
0xA093C6B4
Write PKA_RAM_BASE+0x018+0x24
0x2102B000
Write PKA_RAM_BASE+0x018+0x28
0x7236A085
Write PKA_RAM_BASE+0x018+0x2C
0xA3C9085B
Write PKA_RAM_BASE+0x018+0x30
0x42DC7C01
Write PKA_RAM_BASE+0x018+0x34
0x449492B9
Write PKA_RAM_BASE+0x018+0x38
0x1877CA5F
Write PKA_RAM_BASE+0x018+0x3C
0xF42F559D

```

```

// Base equals value 0x3D291F48_49064887_1149594B_67935110_...
// 14EB8FF0_AB291F3A_54A1B4D1_5E611E44_...
// C989251B_44904B45_0B060482_317F8352_...
// 18CE440E_9BF509F1_6EAF26F2_95F19F12
// write as vector C to PKA
RAM Byte offset 0x060 (following mod after buffer + align words):
Write PKA_RAM_BASE+0x060+0x00
0x95F19F12
Write PKA_RAM_BASE+0x060+0x04
0x6EAF26F2
Write PKA_RAM_BASE+0x060+0x08
0x9BF509F1
Write PKA_RAM_BASE+0x060+0x0C
0x18CE440E
Write PKA_RAM_BASE+0x060+0x10
0x317F8352
Write PKA_RAM_BASE+0x060+0x14
0x0B060482
Write PKA_RAM_BASE+0x060+0x18
0x44904B45
Write PKA_RAM_BASE+0x060+0x1C
0xC989251B
Write PKA_RAM_BASE+0x060+0x20
0x5E611E44
Write PKA_RAM_BASE+0x060+0x24
0x54A1B4D1
Write PKA_RAM_BASE+0x060+0x28
0xAB291F3A
Write PKA_RAM_BASE+0x060+0x2C
0x14EB8FF0
Write PKA_RAM_BASE+0x060+0x30
0x67935110
Write PKA_RAM_BASE+0x060+0x34
0x1149594B
Write PKA_RAM_BASE+0x060+0x38
0x49064887
Write PKA_RAM_BASE+0x060+0x3C
0x3D291F48
// The result value and scratchpad
(vector D) may be co-located with the base vector C for
// a normal modular exponentiation,
so these are located at PKA RAM Byte offset 0x060 too.
// Load pointer and length
registers:
Write PKA_APTR 0x000>>2 //
Exponent pointer
Write PKA_BPTR 0x018>>2 //
Modulus pointer
Write PKA_CPTR 0x060>>2 //
Base pointer

```

```

Write PKA_DPTR 0x060>>2 //
Result/scratchpad pointer
Write PKA_ALENGTH 0x00000005
// Exponent length in 32-bit words
Write PKA_BLENGTH 0x00000010
// Mod/base/result length in 32-bit words
// Start modular exponentiation
and wait until it's done:
Write PKA_SHIFT 0x00000004
// Number of 'odd powers'
Write PKA_FUNCTION 0x0000E000
// 'Run' bit set, 'Sequencer Operations' = 0b110
Wait PKA_FUNCTION[15] == '0'
// 'Run' bit clears itself - Host can also use interrupt!
// Result value equals 0xA497BF8B_DB729088_954005B0_B5CA6691_...
// A3EC491B_091A3D62_03C24214_0863A389_...
// 0C7C03CD_2333E231_35EC10ED_8F91281C_...
// 30F4253B_FE38FAFB_BB4A39DB_C14F2661
// written as vector D at
PKA RAM Byte offset 0x060:
Check PKA_RAM_BASE+0x060+0x00
== 0xC14F2661
Check PKA_RAM_BASE+0x060+0x04
== 0xBB4A39DB
Check PKA_RAM_BASE+0x060+0x08
== 0xFE38FAFB
Check PKA_RAM_BASE+0x060+0x0C
== 0x30F4253B
Check PKA_RAM_BASE+0x060+0x10
== 0x8F91281C
Check PKA_RAM_BASE+0x060+0x14
== 0x35EC10ED
Check PKA_RAM_BASE+0x060+0x18
== 0x2333E231
Check PKA_RAM_BASE+0x060+0x1C
== 0x0C7C03CD
Check PKA_RAM_BASE+0x060+0x20
== 0x0863A389
Check PKA_RAM_BASE+0x060+0x24
== 0x03C24214
Check PKA_RAM_BASE+0x060+0x28
== 0x091A3D62
Check PKA_RAM_BASE+0x060+0x2C
== 0xA3EC491B
Check PKA_RAM_BASE+0x060+0x30
== 0xB5CA6691
Check PKA_RAM_BASE+0x060+0x34
== 0x954005B0
Check PKA_RAM_BASE+0x060+0x38
== 0xDB729088

```

```
Check PKA_RAM_BASE+0x060+0x3C
== 0xA497BF8B
```

Modular Inversion

Besides modular exponentiation, the sequencer also controls modular inversion operations.

Table 31-12: Summary of ModInv Operation

Function	Mathematical Operation	Vector A	Vector B	Vector C	Vector D
ModInv	$A^{-1} \bmod B \rightarrow D$	NumToInvert, length = A_Len	Modulus, length = B_Len	Not Used	Result and Workspace

The above function is an extension of the set of basic PKCP functions with the following exceptions:

- Vector D not only addresses the result, but also a workspace.
- The `PKA_SHIFT` register field is used to return information on the operation's result.

Table 31-13: PKA_SHIFT Result Values for ModInv Operation

Function	PKA_SHIFT Register Field Value At Conclusion
ModInv	0 → success; VectorD holds result 7 → no inverse exists ($\text{GCD}(A, B) \neq 1$, for example, A and B have common factors); result undefined 31 → error, modulus even; result undefined other values are reserved

The following tables list the restrictions on the input and result vectors for the ModInv operation.

Table 31-14: Operational Restrictions on Input Vectors for the ModInv Operation

Function	Requirements
ModInv	$0 < A_Len \leq \text{Max_Len}$ $0 < B_Len \leq \text{Max_Len}$ Modulus B must be odd (for example, the least significant bit must be ONE) Modulus B may not have value 1 (result is undefined, no error indicated) The highest word of the modulus vector, as indicated by B_Len, may not be zero.

Table 31-15: ModInv Scratchpad Area/Input Vector Overlap Restrictions

Function	Result Vector	Restrictions
ModInv	D	Scratchpad area starting at D may not overlap with any of the other vectors

The following table shows the required scratchpad sizes for the ModInv Operation.

Table 31-16: ModInv Result Vector/Scratchpad Area Memory Allocation (Both Starting at PKA_DPTR)

Function	Scratchpad area size (in 32-bit words), Result Vector is B_Length 32-bit words long
ModInv	$5 \times (M + \varepsilon(M))$, with $M = \text{Max}(A_Length, B_Length)$ $\varepsilon(n) = 2 + (n \text{ MOD } 2)$, for example, 2 (for n even) or 3 (for n odd)

NOTE: During execution of a ModInv operation, the last 34 bytes of the PKA RAM are used as general scratchpad for the sequencer program execution. This (fixed location) area may not overlap with any of the input vectors and/or the D vector scratchpad area during execution.

Modular Inversion with an Even Modulus

The ModInv operation requires the modulus to be odd. At first, this requirement appears to make the operation useless in the case of RSA key generation where the private key exponent d is derived from a chosen public exponent e as follows:

$$d = \text{ModInv}(e, \phi); \text{ where } \phi = (p-1) \times (q-1) \text{ and } p \text{ and } q \text{ both prime}$$

Note that ϕ is even. However, since e must be odd (otherwise no inverse exists), d can be calculated as:

$$d = (1 + (\phi \times (e - \text{ModInv}(\phi, e)))) / e$$

With four more basic PKCP operations, ModInv can also be used to find inverse values in case the modulus is even.

Modular Inversion with a Prime Modulus

Modular inversion can be performed with a modular exponentiation using the modulus value minus two as exponent, provided that the modulus value is a prime. This is due to the following:

$$(A^M) \bmod M = A \Rightarrow$$

$$(A^{M-1}) \bmod M = 1 \Rightarrow$$

$$(A^{M-2}) \bmod M = A^{-1} \bmod M$$

Under the constraint that M is a prime value.

Especially with the large PKA engines containing an LNME, it is worthwhile to check whether this method is faster than using the ModInv operation directly. The modulus values for the ECC curves supported by this PKA engine must be prime, so this method can be used in ECDSA operations.

ECC Operations

Besides modular exponentiation and modular inversion, the sequencer also controls ECC operations.

Table 31-17: Summary of ECC Operations

Function	Mathematical Operation	Vector A	Vector B	Vector C	Vector D
ECC-ADD	Point addition/ doubling [†] on elliptic curve: $y^2 = x^3 + ax + b \pmod{p}$ $\text{pntA} + \text{pntC} \rightarrow \text{pntD}$	pntA.x followed [‡] by pntA.y both B_Len long (A_Len <i>not</i> used)	Curve parameter p fol- lowed [‡] by a (b is not needed) all B_Len long	pntC.x followed by pntC.y both B_Len long	Result, for example, pntD.x followed [‡] by pntD.y and work- space
ECC-MUL	Point multiplication on elliptic curve: $y^2 = x^3 + ax + b \pmod{p}$ $k \times \text{pntC} \rightarrow \text{pntD}$	Scalar k A_Len long	Curve parameter p fol- lowed [‡] by a and b all B_Len long.	pntC.x followed [‡] by pntC.y both B_Len long	Result, for example pntD.x followed [‡] by pntD.y and work- space

[†] If $\text{pntA} = \text{pntC}$, a point doubling operation is performed automatically.

[‡] All input components must be located on a 64-bit boundary and must have extra 'buffer' words (of 32 bits each) after their most significant word. ϵ must be 3 (B_Len odd) or 2 (B_Len even). Each result component (for example, pntD.x , pntD.y) is followed by ϵ buffer (zero) words.

The above functions are extensions of the set of PKCP basic functions with the following exceptions:

- Input and result vectors can now be composite (for example, consist of two or three equal-sized subvectors).
- Vector D not only addresses the result, but also a workspace.
- The [PKA_SHIFT](#) register is used to return information on the result of the operation.

Table 31-18: PKA_SHIFT Result Values for ECC Operations

Function	PKA_SHIFT Register Field Value at Conclusion
ECC-ADD	0 → success; VectorD holds result point
ECC-MUL	7 → result is point-at-infinity; VectorD result point undefined
	31 → error, (p not odd, p too short, etc); VectorD result point undefined
	Other values are reserved

The following tables below list the restrictions on the input and result vectors for the ECC operations.

Table 31-19: Operational Restrictions on Input Vectors for ECC Operations

Function	Requirements
ECC-ADD	$1 < B_Len \leq 24$ (maximum vector length is 768 bits) Modulus p must be a prime $> 2^{63}$ Effective modulus size (in bits) must be a multiple of 32 [†] The highest word of the modulus vector, as indicated by B_Len , may not be zero. $a < p$ and $b < p$ $pntA$ and $pntC$ must be on the curve (this condition is not checked) Neither $pntA$ nor $pntC$ can be the point-at-infinity, although ECC-ADD can return this point as a result
ECC-MUL	$0 < A_Len \leq 24$ (maximum vector length is 768 bits) $1 < B_Len \leq 24$ (maximum vector length is 768 bits) Modulus p must be a prime $> 2^{63}$ Effective modulus size (in bits) must be a multiple of 32 [†] The highest word of the modulus vector, as indicated by B_Len , may not be zero. $a < p$ and $b < p$ $pntC$ must be on the curve (this condition is not checked) $pntC$ cannot be the point-at-infinity, although ECC-MUL can return this point as a result

[†] Modulus lengths of 112 and 521 bits are exceptions to this rule.

Table 31-20: ECC Scratchpad Area/Input Vector Overlap Restrictions

Function	Result Vector	Restrictions
ECC-ADD ECC-MUL	D	Scratchpad area starting at D may not overlap with any of the other vectors

The *>ECC Result Vector/Scratchpad Area Memory Allocation* table shows the required scratchpad sizes for the ECC operations:

Table 31-21: ECC Result Vector/Scratchpad Area Memory Allocation (Both Starting at PKA_DPTR)

Function	Scratchpad area size (in 32-bit words), Result Vector is $2 \times (B_Length + \epsilon(B_Length))$ 32-bit words long
ECC-ADD	$2 \times L + 5 \times M$, where $L = B_Length + \epsilon(B_Length)$ $M = B_Length + 1 + \epsilon(B_Length + 1)$
ECC-MUL	$18 \times L + \text{Max}(8, L)$, where $L = (B_Length + \epsilon(B_Length))$

[†] $\epsilon(n) = 2 + (n \text{ MOD } 2)$, for example, 2 (for n even) or 3 (for n odd)

NOTE: During execution of an ECC-ADD or ECC-MUL operation, the last 72 bytes of the PKA RAM are used as a general scratchpad for the sequencer program execution. These (fixed location) areas must not overlap with any of the input vectors and the D vector scratchpad area during execution.

The *ECC Point Multiplication PKA RAM Allocation Examples* table shows example PKA RAM vector allocations for ECC point multiplication operations. The *free space* start address is the first free byte following the vector scratchpad. The sequencer execution scratchpad of 72 bytes must fit between this address and the end of the PKA RAM. Because of this requirement, a 521-bit ECC point multiplication cannot be performed with 2K byte PKA RAM.

Table 31-22: ECC Point Multiplication PKA RAM Allocation Examples

Modulus Length	(sub-)vector	Start Address (byte offset)	Size (words)	Buffer (words)
192 bits (=6 words, <code>PKA_ALEN=0x006</code> , <code>PKA_BLEN=0x006</code>)	Scalar k	0x000 (<code>PKA_APTR = 0x000</code>)	6	0
	p	0x018 (<code>PKA_BPTR = 0x006</code>)	6	2
	a	0x038	6	2
	b	0x058	6	2
	PntC.x (base)	0x078 (<code>PKA_CPTR = 0x01E</code>)	6	2
	PntC.y (base)	0x098	6	2
	PntD.x (result)	0x0B8 (<code>PKA_DPTR = 0x02E</code>)	6	2
	PntD.y (result)	0x0D8	6	0
	Vector scratchpad	0x0B8 (= PntD.x)	$(18 \times 8) + 8 = 152$	0
	Free space	0x318 (792 bytes used)	-	-
384 bits (=12 words, <code>ALENGTH=0x00C</code> , <code>BLENGTH=0x00C</code>)	Scalar k	0x000 (<code>PKA_APTR = 0x000</code>)	12	0
	p	0x030 (<code>PKA_BPTR = 0x00C</code>)	12	2
	a	0x068	12	2
	b	0x0A0	12	2
	PntC.x (base)	0x0D8 (<code>PKA_CPTR = 0x036</code>)	12	2
	PntC.y (base)	0x110	12	2
	PntD.x (result)	0x148 (<code>PKA_DPTR = 0x052</code>)	12	2
	PntD.y (result)	0x180	12	0
	Vector scratchpad	0x148 (= PntD.x)	$(18 \times 14) + 14 = 266$	0
	Free space	0x570 (1392 bytes used)	-	-
521 bits (=17 words, <code>ALENGTH=0x011</code> , <code>BLENGTH=0x011</code>)	Scalar k	0x000 (<code>PKA_APTR = 0x000</code>)	17	1 (to align p)
	p	0x048 (<code>PKA_BPTR = 0x012</code>)	17	3
	a	0x098	17	3
	b	0x0E8	17	3
	PntC.x (base)	0x138 (<code>PKA_CPTR = 0x04E</code>)	17	3
	PntC.y (base)	0x188	17	3
	PntD.x (result)	0x1D8 (<code>PKA_DPTR = 0x076</code>)	17	3

Table 31-22: ECC Point Multiplication PKA RAM Allocation Examples (Continued)

Modulus Length	(sub-)vector	Start Address (byte offset)	Size (words)	Buffer (words)
	PntD.y (result)	0x228	17	0
	Vector scratchpad	0x1D8 (= PntD.x)	$(18 \times 20) + 20 = 380$	0
	Free space	0x7C8 (1992 bytes used)	-	-

The following example in pseudo-code describes the execution of a 192 bits ECC point multiplication, using actual test vectors (the curve parameters and generator point are from standard curve 'secp192r1').

```
// Perform a 192 bits ECC point multiplication using PKA RAM layout from table
// above.
// Scalar 'k' equals value 0x8D98D058_9EFD018A_C9BCF3CF_2C33AEC0_24867D7F_6ADACBFF
// write as vector A to PKA RAM Byte offset 0x000:

    Write PKA_RAM_BASE+0x000+0x00 0x6ADACBFF
    Write PKA_RAM_BASE+0x000+0x04 0x24867D7F
    Write PKA_RAM_BASE+0x000+0x08 0x2C33AEC0
    Write PKA_RAM_BASE+0x000+0x0C 0xC9BCF3CF
    Write PKA_RAM_BASE+0x000+0x10 0x9EFD018A
    Write PKA_RAM_BASE+0x000+0x14 0x8D98D058
// Curve parameter 'p' equals value
0xFFFFFFFF_FFFFFFFF_FFFFFFFF_FFFFFFFFE_FFFFFFFF_FFFFFFFF
// write as 1st part of vector B immediately following vector A at PKA RAM Byte
// offset 0x018
// (no buffer word needed after 'k' vector, 64-bit alignment is OK):
    Write PKA_RAM_BASE+0x018+0x00 0xFFFFFFFF
    Write PKA_RAM_BASE+0x018+0x04 0xFFFFFFFF
    Write PKA_RAM_BASE+0x018+0x08 0xFFFFFFFFFE
    Write PKA_RAM_BASE+0x018+0x0C 0xFFFFFFFF
    Write PKA_RAM_BASE+0x018+0x10 0xFFFFFFFF
    Write PKA_RAM_BASE+0x018+0x14 0xFFFFFFFF
// Curve parameter 'a' equals value
0xFFFFFFFF_FFFFFFFF_FFFFFFFF_FFFFFFFFE_FFFFFFFF_FFFFFFFC
// write as 2nd part of vector B after one buffer word and one re-alignment word
// at 0x038:
    Write PKA_RAM_BASE+0x038+0x00 0xFFFFFFFFC
    Write PKA_RAM_BASE+0x038+0x04 0xFFFFFFFF
    Write PKA_RAM_BASE+0x038+0x08 0xFFFFFFFFFE
    Write PKA_RAM_BASE+0x038+0x0C 0xFFFFFFFF
    Write PKA_RAM_BASE+0x038+0x10 0xFFFFFFFF
    Write PKA_RAM_BASE+0x038+0x14 0xFFFFFFFF
// Curve parameter 'b' equals value
0x64210519_E59C80E7_0FA7E9AB_72243049_FEB8DEEC_C146B9B1
// write as 3rd part of vector B after one buffer word and one re-alignment word
// at 0x058:
    Write PKA_RAM_BASE+0x058+0x00
0xC146B9B1
```

```

Write PKA_RAM_BASE+0x058+0x04 0xFEB8DEEC
Write PKA_RAM_BASE+0x058+0x08 0x72243049
Write PKA_RAM_BASE+0x058+0x0C 0x0FA7E9AB
Write PKA_RAM_BASE+0x058+0x10 0xE59C80E7
Write PKA_RAM_BASE+0x058+0x14 0x64210519
// X-coord of generator point is value
0x188DA80E_B03090F6_7CBF20EB_43A18800_F4FF0AFD_82FF1012
// write as 1st part of vector C following vector B after buffer + alignment
words at 0x078:
Write PKA_RAM_BASE+0x078+0x00 0x82FF1012
Write PKA_RAM_BASE+0x078+0x04 0xF4FF0AFD
Write PKA_RAM_BASE+0x078+0x08 0x43A18800
Write PKA_RAM_BASE+0x078+0x0C 0x7CBF20EB
Write PKA_RAM_BASE+0x078+0x10 0xB03090F6
Write PKA_RAM_BASE+0x078+0x14 0x188DA80E
// Y-coord of generator point is value
0x07192B95_FFC8DA78_631011ED_6B24CDD5_73F977A1_1E794811
// write as 2nd part of vector C after one buffer word and one re-alignment word
at 0x098:
Write PKA_RAM_BASE+0x098+0x00 0x1E794811
Write PKA_RAM_BASE+0x098+0x04 0x73F977A1
Write PKA_RAM_BASE+0x098+0x08 0x6B24CDD5
Write PKA_RAM_BASE+0x098+0x0C 0x631011ED
Write PKA_RAM_BASE+0x098+0x10 0xFFC8DA78
Write PKA_RAM_BASE+0x098+0x14 0x07192B95
// The result point and scratchpad (vector D) follow vector C after one buffer
word and one
// re-alignment word, so these are located at PKA RAM Byte offset 0x0B8.
// Load pointer and length registers:
Write PKA_APTR 0x000>>2 // Scalar 'k' pointer
Write PKA_BPTR 0x018>>2 // Curve parameters 'p', 'a' & 'b' pointer
Write PKA_CPTR 0x078>>2 // Generator point X & Y coordinates pointer
Write PKA_DPTR 0x0B8>>2 // Result point X & Y coordinates/scratchpad pointer
Write PKA_ALENGTH 0x00000006 // Scalar 'k' length in 32-bit words
Write PKA_BLENGTH 0x00000006 // Curve parameters and coordinate lengths in 32-
bit words
// Start ECC point multiplication and wait until it's done:
Write PKA_FUNCTION 0x0000D000
// 'Run' bit set, 'Sequencer Operations' = 0b101
Wait PKA_FUNCTION[15] == '0'
// 'Run' bit clears itself - Host can also use interrupt!
Check PKA_SHIFT == 0x00000000
// Shift field value 0 indicates success - check this
// X-coord of result point is value
0x759B9F39_0E81D268_18C82BB9_CB42BCF5_0E0AE958_85BA3097
// written as 1st part of vector D at PKA RAM Byte offset 0x0B8:
Check PKA_RAM_BASE+0x0B8+0x00== 0x85BA3097
Check PKA_RAM_BASE+0x0B8+0x04== 0x0E0AE958
Check PKA_RAM_BASE+0x0B8+0x08== 0xCB42BCF5
Check PKA_RAM_BASE+0x0B8+0x0C== 0x18C82BB9

```

```

    Check PKA_RAM_BASE+0x0B8+0x10== 0x0E81D268
    Check PKA_RAM_BASE+0x0B8+0x14== 0x759B9F39
// Y-coord of result point is value
0xECA14640_F92EFF07_CAF2BD55_3FBE28EF_D043F28E_1CC3D238
// written as 2nd part of vector D at PKA RAM Byte offset 0x0D8:
    Check PKA_RAM_BASE+0x0D8+0x00== 0x1CC3D238
    Check PKA_RAM_BASE+0x0D8+0x04== 0xD043F28E
    Check PKA_RAM_BASE+0x0D8+0x08== 0x3FBE28EF
    Check PKA_RAM_BASE+0x0D8+0x0C== 0xCAF2BD55
    Check PKA_RAM_BASE+0x0D8+0x10== 0xF92EFF07
    Check PKA_RAM_BASE+0x0D8+0x14== 0xECA14640

```

ADSP-21560_21561_21564_21568 PKA Register Descriptions

Public Key Accelerator (PKA) contains the following registers.

Table 31-23: ADSP-21560_21561_21564_21568 PKA Register List

Name	Description
PKA_ALEN	PKA Vector_A Length
PKA_APTR	PKA Vector_A Address
PKA_BLEN	PKA Vector_B Length
PKA_BPTR	PKA Vector_B Address
PKA_COMPARE	PKA Compare Result
PKA_CPTR	PKA Vector_C Address
PKA_DIVMSW	PKA Most-Significant-Word of Divide Remainder
PKA_DPTR	PKA Vector_D Address
PKA_FUNC	PKA Function
PKA_RAM	Start of PKA RAM space
PKA_RESULTMSW	PKA Most-Significant-Word of Result Vector
PKA_SHIFT	PKA Bit Shift Value

PKA Vector_A Length

During execution of basic PKCP operations, the `PKA_ALEN` register is double buffered and can be written with a new value for the next operation. When not written, the value remains intact. During the execution of sequencer controlled complex operations, the `PKA_ALEN` register may not be written and its value is undefined at the conclusion of the operation. The driver software cannot rely on the written value to remain intact.

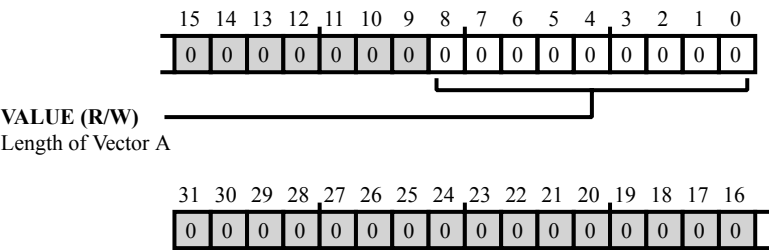


Figure 31-2: PKA_ALEN Register Diagram

Table 31-24: PKA_ALEN Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
8:0 (R/W)	VALUE	Length of Vector A. Length (in 32-bit words) of Vector A.

PKA Vector_A Address

During execution of basic PKCP operations, the `PKA_APTR` register is double buffered and can be written with a new value for the next operation. When not written, the value remains intact. During the execution of sequencer controlled complex operations, the `PKA_APTR` register may not be written and its value is undefined at the conclusion of the operation. The driver software cannot rely on the written value to remain intact.

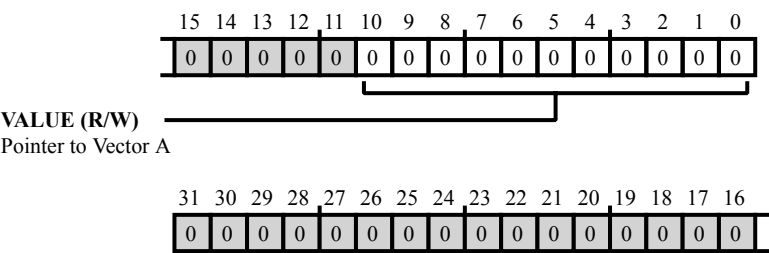


Figure 31-3: PKA_APTR Register Diagram

Table 31-25: PKA_APTR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
10:0 (R/W)	VALUE	Pointer to Vector A. The <code>PKA_APTR.VALUE</code> bit field is the location of Vector A within the PKA RAM. Vectors are identified through the location of their least-significant 32-bit word. Note that bit [0] must be zero to ensure that the vector starts at an 8-byte boundary.

PKA Vector_B Length

During execution of basic PKCP operations, the `PKA_BLEN` register is double buffered and can be written with a new value for the next operation. When not written, the value remains intact. During the execution of sequencer controlled complex operations, the `PKA_BLEN` register may not be written and its value is undefined at the conclusion of the operation. The driver software cannot rely on the written value to remain intact.

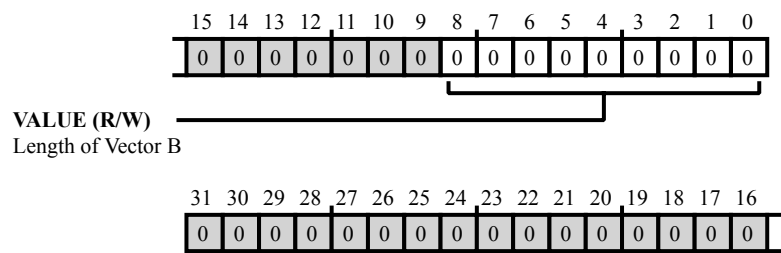


Figure 31-4: PKA_BLEN Register Diagram

Table 31-26: PKA_BLEN Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
8:0 (R/W)	VALUE	Length of Vector B. Length (in 32-bit words) of Vector B.

PKA Vector_B Address

During execution of basic PKCP operations, the `PKA_BPTR` register is double buffered and can be written with a new value for the next operation. When not written, the value remains intact. During the execution of sequencer controlled complex operations, the `PKA_BPTR` register may not be written and its value is undefined at the conclusion of the operation. The driver software cannot rely on the written value to remain intact.

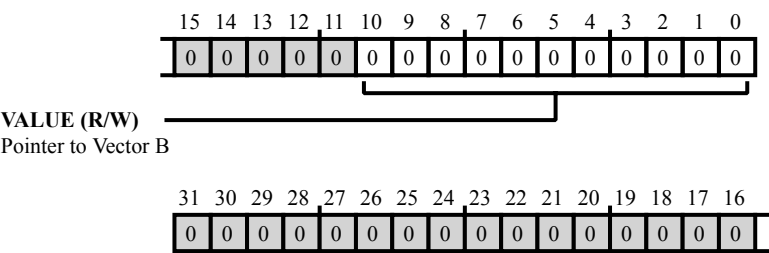


Figure 31-5: PKA_BPTR Register Diagram

Table 31-27: PKA_BPTR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
10:0 (R/W)	VALUE	Pointer to Vector B. The <code>PKA_BPTR.VALUE</code> bit field is the location of Vector B within the PKA RAM. Vectors are identified through the location of their least-significant 32-bit word. Note that bit [0] must be zero to ensure that the vector starts at an 8-byte boundary.

PKA Compare Result

The `PKA_COMPARE` register provides the result of a basic PKCP Compare operation. It is updated when the `PKA_FUNC.RUN` bit is reset at the end of that operation. The status after a complex sequencer operation is unknown.

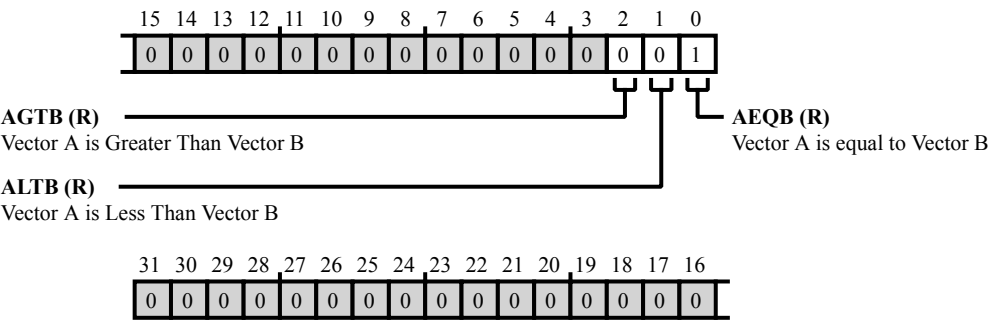


Figure 31-6: PKA_COMPARE Register Diagram

Table 31-28: PKA_COMPARE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
2 (R/NW)	AGTB	Vector A is Greater Than Vector B. The <code>PKA_COMPARE.AGTB</code> bit shows the result of the basic compare operation is PKCP <code>Vector_A</code> is greater than <code>Vector_B</code> .
1 (R/NW)	ALTB	Vector A is Less Than Vector B. The <code>PKA_COMPARE.ALTB</code> bit shows the result of the basic compare operation is <code>Vector_A</code> is less than <code>Vector_B</code> .
0 (R/NW)	AEQB	Vector A is equal to Vector B. The <code>PKA_COMPARE.AEQB</code> bit shows the result of the basic compare operation is <code>Vector_A</code> is equal to <code>Vector_B</code> .

PKA Vector_C Address

During execution of basic PKCP operations, the `PKA_CPTR` register is double buffered and can be written with a new value for the next operation. When not written, the value remains intact. During the execution of sequencer controlled complex operations, the `PKA_CPTR` register may not be written and its value is undefined at the conclusion of the operation. The driver software cannot rely on the written value to remain intact.

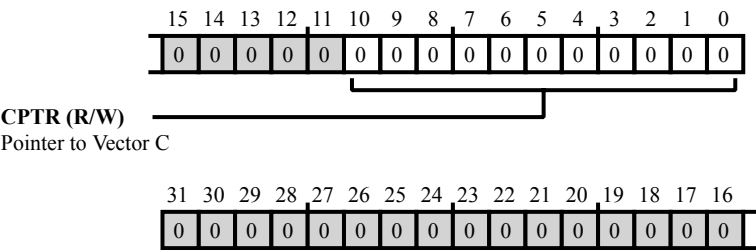


Figure 31-7: PKA_CPTR Register Diagram

Table 31-29: PKA_CPTR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
10:0 (R/W)	CPTR	Pointer to Vector C. The <code>PKA_CPTR.CPTR</code> bit field is the location of Vector C within the PKA RAM. Vectors are identified through the location of their least-significant 32-bit word. Note that bit [0] must be zero to ensure that the vector starts at an 8-byte boundary.

PKA Most-Significant-Word of Divide Remainder

The `PKA_DIVMSW` register indicates the (32-bit word) address in the PKA RAM where the most significant non-zero 32-bit word of the Remainder result for the basic Divide and Modulo operations is stored. Bits [4:0] are loaded with the bit number of the most significant non-zero bit in the most significant non-zero word when MS one control bit is set. For Divide, Modulo and MS one reporting, this register is updated when the `PKA_FUNC.RUN` bit is reset at the end of the operation.

For the complex sequencer controlled operations, updating bits [4:0] of this register with the actual result's most significant bit location is done near the end of the operation. Note that the result is only meaningful if no errors were detected and that for ECC operations, the `PKA_DIVMSW` register provides information for the x-coordinate of the result point only.

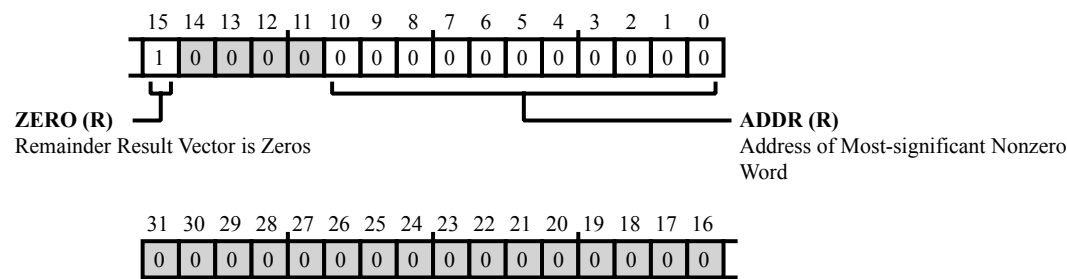


Figure 31-8: PKA_DIVMSW Register Diagram

Table 31-30: PKA_DIVMSW Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/NW)	ZERO	Remainder Result Vector is Zeros. The <code>PKA_DIVMSW.ZERO</code> bit shows the remainder result vector is all zeros, ignore the address returned in bits [10:0].
10:0 (R/NW)	ADDR	Address of Most-significant Nonzero Word. The <code>PKA_DIVMSW.ADDR</code> bit shows the address of the most significant non-zero 32-bit word of the remainder result vector in PKA RAM.

PKA Vector_D Address

During execution of basic PKCP operations, the `PKA_DPTR` register is double buffered and can be written with a new value for the next operation. When not written, the value remains intact. During the execution of sequencer controlled complex operations, the `PKA_DPTR` register may not be written and its value is undefined at the conclusion of the operation. The driver software cannot rely on the written value to remain intact.

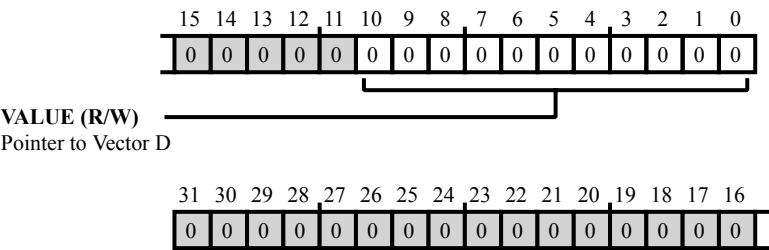


Figure 31-9: PKA_DPTR Register Diagram

Table 31-31: PKA_DPTR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
10:0 (R/W)	VALUE	Pointer to Vector D. The <code>PKA_DPTR.VALUE</code> bit field is the location of Vector D within the PKA RAM. Vectors are identified through the location of their least-significant 32-bit word. Note that bit [0] must be zero to ensure that the vector starts at an 8-byte boundary.

Table 31-32: PKA_FUNC Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration																
24 (R/W)	STALLRSLT	Stall Result. When the <code>PKA_FUNC . STALLRSLT</code> bit is set, updating the <code>PKA_COMPARE</code>, <code>PKA_RESULTMSW</code> and <code>PKA_DIVMSW</code> registers, as well as resetting the Run bit, is stalled beyond the point that a running operation is actually finished. Use this to allow software enough time to read results from a previous operation when the newly started operation is known to take only a short amount of time. If a result is waiting, the result registers is updated and the Run bit is reset in the clock cycle following writing the Stall Result bit back to 0. The Stall result function may only be used for basic PKCP operations.																
15 (R/W)	RUN	Run. Set the <code>PKA_FUNC . RUN</code> bit to instruct the PKA module to begin processing the basic PKCP or complex Sequencer operation. This bit is reset low automatically when the operation is complete. The complement of this bit is output as the <code>pkaint1</code> interrupt. After a reset, the Run bit is always set to 1b but the first Sequencer firmware instruction sets this bit to 0 immediately after the hardware reset is released. A few clock cycles are needed before the first instruction is executed and the Run bit state has been propagated.																
14:12 (R/W)	SEQOPS	Sequencer Operation Select. The <code>PKA_FUNC . SEQOPS</code> bit field select the complex Sequencer operation to perform. <table><tr><td>0</td><td>None</td></tr><tr><td>1</td><td>ExpMod-CRT</td></tr><tr><td>2</td><td>ExpMod-ACT4</td></tr><tr><td>3</td><td>ECC-ADD</td></tr><tr><td>4</td><td>ExpMod-ACT2</td></tr><tr><td>5</td><td>ECC-MUL</td></tr><tr><td>6</td><td>ExpMod-variable</td></tr><tr><td>7</td><td>ModInv</td></tr></table>	0	None	1	ExpMod-CRT	2	ExpMod-ACT4	3	ECC-ADD	4	ExpMod-ACT2	5	ECC-MUL	6	ExpMod-variable	7	ModInv
0	None																	
1	ExpMod-CRT																	
2	ExpMod-ACT4																	
3	ECC-ADD																	
4	ExpMod-ACT2																	
5	ECC-MUL																	
6	ExpMod-variable																	
7	ModInv																	
11 (R/W)	CPY	Copy. Setting the <code>PKA_FUNC . CPY</code> bit performs the copy operation. For more information, see the "PKCP Vector Operations" section.																
10 (R/W)	CMP	Perform Compare Operation. Setting the <code>PKA_FUNC . CMP</code> bit performs the compare operation. For more information, see the "PKCP Vector Operations" section.																

Table 31-32: PKA_FUNC Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
9 (R/W)	MODULO	Perform Modulo Operation. Setting the PKA_FUNC . MODULO bit performs the modulo operation. For more information, see the "PKCP Vector Operations" section.
8 (R/W)	DIV	Perform Divide Operation. Setting the PKA_FUNC . DIV bit performs the divide operation. For more information, see the "PKCP Vector Operations" section.
7 (R/W)	LSHFT	Perform Left Shift Operation. Setting the PKA_FUNC . LSHFT bit performs the Left shift operation. For more information, see the "PKCP Vector Operations" section.
6 (R/W)	RSHFT	Perform Right Shift Operation. Setting the PKA_FUNC . RSHFT bit performs the right shift operation. For more information, see the "PKCP Vector Operations" section.
5 (R/W)	SUB	Perform Subtract Operation. Setting the PKA_FUNC . SUB bit performs the subtract operation. For more information, see the "PKCP Vector Operations" section.
4 (R/W)	ADD	Perform Add Operation. Setting the PKA_FUNC . ADD bit performs the add operation. For more information, see the "PKCP Vector Operations" section.
3 (R/W)	MSONE	Most Significant One. Setting the PKA_FUNC . MSONE bit loads the location of the Most Significant one bit within the result word indicated in the PKA_RESULTMSW register into bits [4:0] of the PKA_DIVMSW register can only be used with basic PKCP operations, except for Divide, Modulo and Compare.
1 (R/W)	ADDSUB	Perform Combined Add/Subtract Operation. Setting the PKA_FUNC . ADDSUB bit performs the combined Add/Subtract operation. For more information, see the "PKCP Vector Operations" section.
0 (R/W)	MULT	Perform Multiply Operation. Setting the PKA_FUNC . MULT bit performs the multiply operation. For more information, see the "PKCP Vector Operations" section.

Start of PKA RAM space

The `PKA_RAM` register provides the starting location of the RAM space to hold the input, output and other vectors.

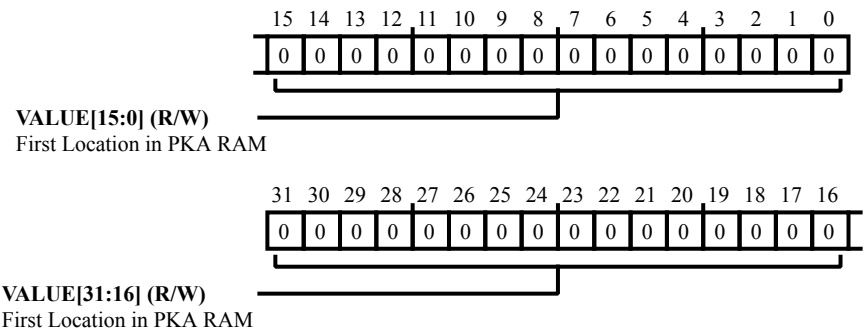


Figure 31-11: PKA_RAM Register Diagram

Table 31-33: PKA_RAM Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	First Location in PKA RAM. The <code>PKA_RAM.VALUE</code> bit field provides the starting location of the RAM space to hold the input, output and other vectors.

PKA Most-Significant-Word of Result Vector

The `PKA_RESULTMSW` register indicates the (word) address in the PKA RAM where the most significant non-zero 32-bit word of the result is stored and should be ignored for modulo operations. For basic PKCP operations, the `PKA_RESULTMSW` register is updated when the `PKA_FUNC.RUN` bit is reset at the end of the operation.

For the complex sequencer controlled operations, updating the final value matching the actual result is done near the end of the operation. Note that the result is only meaningful if no errors are detected and that for ECC operations, the `PKA_DIVMSW` register provides information for the x-coordinate of the result point only.

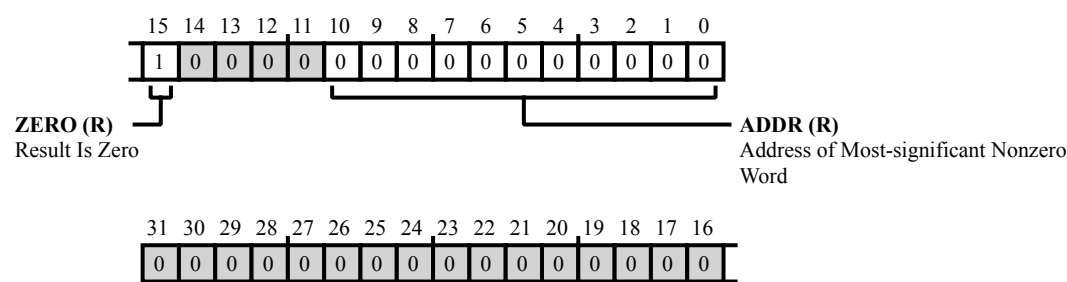


Figure 31-12: `PKA_RESULTMSW` Register Diagram

Table 31-34: `PKA_RESULTMSW` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/NW)	ZERO	Result Is Zero. The <code>PKA_RESULTMSW.ZERO</code> bit indicates the result vector is all zeros, ignore the address returned in bits [10:0].
10:0 (R/NW)	ADDR	Address of Most-significant Nonzero Word. The <code>PKA_RESULTMSW.ADDR</code> bit is the address of the most significant non-zero 32-bit word of the result vector in PKA RAM.

PKA Bit Shift Value

For basic PKCP operations, modifying the contents of the `PKA_SHIFT` register is made impossible while the operation is being performed. For the ExpMod-variable and ExpMod-CRT operations, the `PKA_SHIFT` register is used to indicate the number of odd powers to use (directly as a value in the range 1-16). For the ModInv and ECC operations, this register is used to hold a completion code.

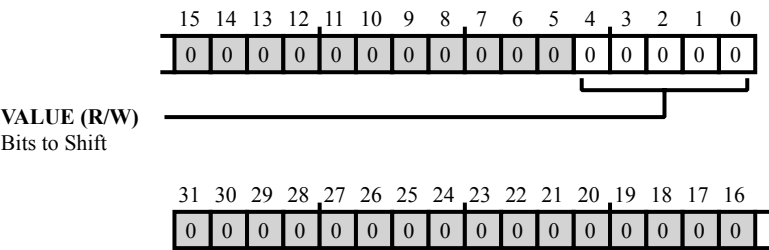


Figure 31-13: PKA_SHIFT Register Diagram

Table 31-35: PKA_SHIFT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
4:0 (R/W)	VALUE	Bits to Shift. The <code>PKA_SHIFT.VALUE</code> bit field is the number of bits to shift the input vector (in the range 0-31) during a Rshift or Lshift operation.

32 Public Key Interrupt Controller (PKIC)

The Public Key Accelerator (PKA) and the True Random Number Generator (TRNG) share a common interrupt controller, the Public Key Interrupt Controller (PKIC). The host processor configures the PKIC to generate interrupts when certain operations are complete, or interrupts are caused by errors.

PKIC Functional Description

The main purpose and function of the PKIC is to capture the interrupts from different sources, either from the PKA or the TRNG and combine them to one interrupt output. The interrupt controller is managed using the following register groups:

- Control for polarity, edge, and level detection and enabling of individual interrupts
- Acknowledgment (to clear edge detected interrupts)
- Status:
 - A raw source status register after edge detection, if edge selected.
 - A status register after masking with the interrupt enable control bits.

ADSP-21560_21561_21564_21568 PKIC Register List

The Public Key Processor Interrupt Controller (PKIC) provides security-related features. A set of registers governs PKIC operations. For more information on PKIC functionality, see the PKIC register descriptions.

Table 32-1: ADSP-21560_21561_21564_21568 PKIC Register List

Name	Description
PKIC_ACK	Acknowledge Register
PKIC_EN_CLR	Enable Clear Register
PKIC_EN_CTL	Enable Control Register
PKIC_EN_SET	Enable Set Register
PKIC_EN_STAT	Enabled Status Register
PKIC_POL_CTL	Polarity Control Register

Table 32-1: ADSP-21560_21561_21564_21568 PKIC Register List (Continued)

Name	Description
PKIC_RAW_STAT	Raw Status Register
PKIC_TYPE_CTL	Type Control Register

ADSP-21560_21561_21564_21568 PKIC Interrupt List

Table 32-2: ADSP-21560_21561_21564_21568 PKIC Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
110	PKIC0_IRQ	PKIC0 Interrupt	Level	

PKIC Programming Model

The following sections provide information on how to program the PKIC.

Enabling/Disabling and Status

The [PKIC_EN_STAT](#) register provides the mask to which interrupt source is enabled. There are two status registers, [PKIC_RAW_STAT](#) and [PKIC_EN_STAT](#). They allow the host processor to read the status of the interrupt source before and after the mask is applied.

Level or Edge

All of the interrupt sources are level or edge events. The [PKIC_TYPE_CTL](#) register configures each interrupt to either level or edge. The [PKIC_POL_CTL](#) register controls the polarity of the signal.

These interrupts are latched at both status registers in case edge detection is selected. The edge detectors are reset by clearing the interrupts using the [PKIC_ACK](#) registers.

PKIC Programming Concepts

The following concepts help with proper programming for the PKIC module.

Interrupt Handling

When an interrupt is triggered, the handler must first examine this module, the PKIC to determine what triggered the interrupt. By reading the [PKIC_EN_STAT](#), the bits that are set are the pending interrupts of the ones that were not masked. After determining the source of the interrupt, the appropriate action must be taken to service the interrupt from the corresponding module, the PKA, or the TRNG. After handling the interrupt in a particular module, the corresponding interrupt must be acknowledged and cleared in the PKIC to allow further interrupts.

While handling an interrupt, any events that would cause another interrupt would happen without triggering another interrupt.

Overlapping Registers

There are two sets of overlapping registers in the PKIC. The `PKIC_EN_STAT` and `PKIC_ACK` registers share one address. If read, the register tells which enabled interrupts are pending. If written to (W1C), the interrupt is acknowledged and cleared. The `PKIC_RAW_STAT` and `PKIC_EN_SET` registers are another pair that share the address. When read, the register tells which interrupts are pending and if written to will enable certain interrupts. This register cannot be used to disable any interrupts.

ADSP-21560_21561_21564_21568 PKIC Register Descriptions

Public Key Processor Interrupt Controller (PKIC) contains the following registers.

Table 32-3: ADSP-21560_21561_21564_21568 PKIC Register List

Name	Description
<code>PKIC_ACK</code>	Acknowledge Register
<code>PKIC_EN_CLR</code>	Enable Clear Register
<code>PKIC_EN_CTL</code>	Enable Control Register
<code>PKIC_EN_SET</code>	Enable Set Register
<code>PKIC_EN_STAT</code>	Enabled Status Register
<code>PKIC_POL_CTL</code>	Polarity Control Register
<code>PKIC_RAW_STAT</code>	Raw Status Register
<code>PKIC_TYPE_CTL</code>	Type Control Register

Acknowledge Register

The `PKIC_ACK` register is used to acknowledge the interrupt and clear the corresponding interrupt bit in the status register.

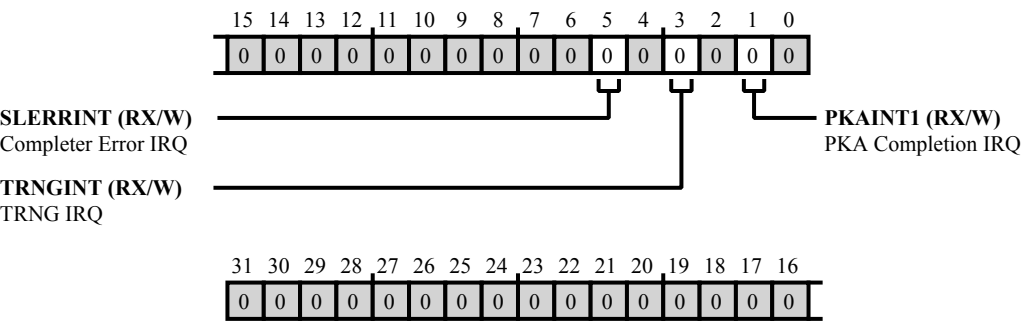


Figure 32-1: PKIC_ACK Register Diagram

Table 32-4: PKIC_ACK Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
5 (RX/W)	SLERRINT	Completer Error IRQ. The <code>PKIC_ACK.SLERRINT</code> bit is the acknowledge bit for the completer error interrupt. When set =1 the <code>PKIC_ACK.SLERRINT</code> bit acknowledges the interrupt signal and clears the status bit (and is cleared automatically).
		0 Do not acknowledge interrupt and clear status bit
		1 Acknowledge interrupt and clear status bit
3 (RX/W)	TRNGINT	TRNG IRQ. The <code>PKIC_ACK.TRNGINT</code> bit is the acknowledge bit for the TRNG interrupt. When set =1 the <code>PKIC_ACK.TRNGINT</code> bit acknowledges the interrupt signal and clears the status bit (and is cleared automatically).
		0 Do not acknowledge interrupt and clear status bit
		1 Acknowledge interrupt and clear status bit
1 (RX/W)	PKAINT1	PKA Completion IRQ. The <code>PKIC_ACK.PKAINT1</code> bit is the acknowledge bit for the PKA completion interrupt. When set =1 the <code>PKIC_ACK.PKAINT1</code> bit acknowledges the interrupt signal and clears the status bit (and is cleared automatically).

Enable Clear Register

The `PKIC_EN_CLR` register allows the user to disable certain interrupts without enabling others. The disabled interrupts are also reflected in `PKIC_EN_CTL` register.

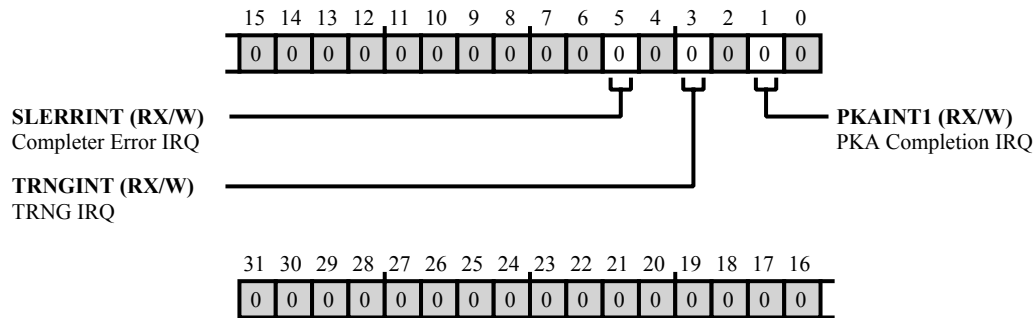


Figure 32-2: PKIC_EN_CLR Register Diagram

Table 32-5: PKIC_EN_CLR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
5 (RX/W)	SLERRINT	Completer Error IRQ. The <code>PKIC_EN_CLR.SLERRINT</code> bit is the individual disable for the completer error interrupt. When set =1 this bit clears/resets the corresponding bit in the <code>PKIC_EN_CTL</code> register to 0 (disables the interrupt). This bit is cleared automatically.
		0 No action
		1 Clear/reset corresponding CTL bit
3 (RX/W)	TRNGINT	TRNG IRQ. The <code>PKIC_EN_CLR.TRNGINT</code> bit is the individual disable for the TRNG interrupt. When set =1 this bit clears/resets the corresponding bit in the <code>PKIC_EN_CTL</code> register to 0 (disables the interrupt). This bit is cleared automatically. 0= no effect.
		0 No action
		1 Clear/reset corresponding CTL bit
1 (RX/W)	PKAINT1	PKA Completion IRQ. The <code>PKIC_EN_CLR.PKAINT1</code> bit is the individual disable for the PKA Completion interrupt. When set =1 this bit clears/resets the corresponding bit in the <code>PKIC_EN_CTL</code> register to 0 (disables the interrupt). This bit is cleared automatically. 0= no effect.
		0 No action
		1 Clear/reset corresponding CTL bit

Enable Control Register

The `PKIC_EN_CTL` register provides individual enable bits for the interrupt sources.

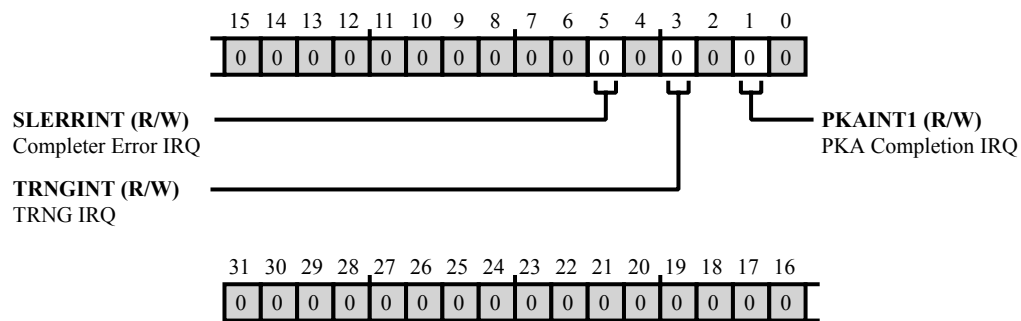


Figure 32-3: `PKIC_EN_CTL` Register Diagram

Table 32-6: `PKIC_EN_CTL` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
5 (R/W)	SLERRINT	Completer Error IRQ. The <code>PKIC_EN_CTL.SLERRINT</code> bit enables control for the completer error interrupt.
		0 Disable interrupt
		1 Enable interrupt
3 (R/W)	TRNGINT	TRNG IRQ. The <code>PKIC_EN_CTL.TRNGINT</code> bit enables control for the TRNG interrupt.
		0 Disable interrupt
		1 Enable interrupt
1 (R/W)	PKAINT1	PKA Completion IRQ. The <code>PKIC_EN_CTL.PKAINT1</code> bit enables control for the PKA completion interrupt.
		0 Disable interrupt
		1 Enable interrupt

Enable Set Register

The `PKIC_EN_SET` register allows the user to only set certain interrupt sources without disabling any others. The enabled interrupts are reflected in `PKIC_EN_CTL` register.

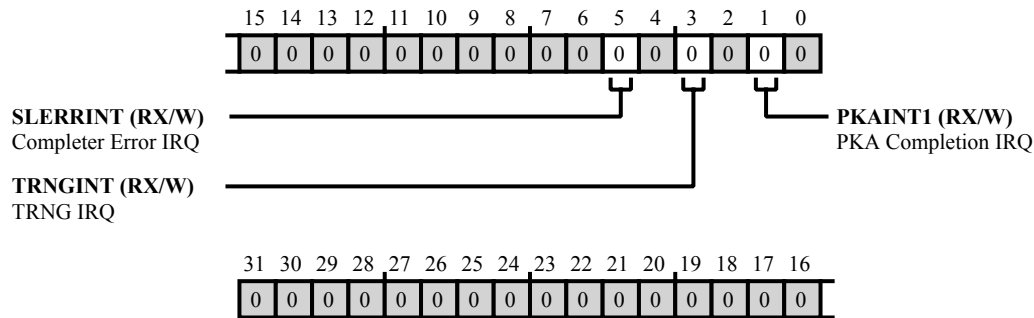


Figure 32-4: PKIC_EN_SET Register Diagram

Table 32-7: PKIC_EN_SET Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
5 (RX/W)	SLERRINT	Completer Error IRQ. The <code>PKIC_EN_SET.SLERRINT</code> bit is the individual enable for the completer error interrupt. If =1, sets the corresponding bit in the <code>PKIC_EN_CTL</code> register to 1 (enables interrupt). This bit is cleared automatically. 0=no effect
		0 No effect
		1 Enable interrupt
3 (RX/W)	TRNGINT	TRNG IRQ. The <code>PKIC_EN_SET.TRNGINT</code> bit is the individual enable for the TRNG interrupt. If =1, sets the corresponding bit in the <code>PKIC_EN_CTL</code> register to 1 (enables interrupt). This bit is cleared automatically.
		0 No effect
		1 Enable interrupt
1 (RX/W)	PKAINT1	PKA Completion IRQ. The <code>PKIC_EN_SET.PKAINT1</code> bit is the individual enable for the PKA Completion interrupt. If =1, sets the corresponding bit in the <code>PKIC_EN_CTL</code> register to 1 (enables interrupt). This bit is cleared automatically.
		0 No effect
		1 Enable interrupt

Enabled Status Register

The `PKIC_EN_STAT` register is used to tell the status of the interrupts after the gating with the `PKIC_EN_CTL` register.

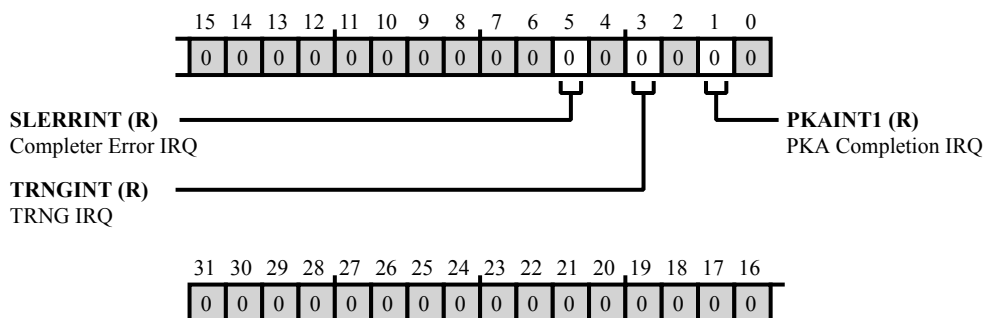


Figure 32-5: PKIC_EN_STAT Register Diagram

Table 32-8: PKIC_EN_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
5 (R/NW)	SLERRINT	Completer Error IRQ. The <code>PKIC_EN_STAT.SLERRINT</code> bit provides the status of the completer error interrupt (after masking from the <code>PKIC_EN_CTL</code> register).
		0 Interrupt is inactive
		1 Interrupt is pending
3 (R/NW)	TRNGINT	TRNG IRQ. The <code>PKIC_EN_STAT.TRNGINT</code> bit provides the status of the TRNG interrupt (after masking from the <code>PKIC_EN_CTL</code> register).
		0 Interrupt is inactive
		1 Interrupt is pending
1 (R/NW)	PKAINT1	PKA Completion IRQ. The <code>PKIC_EN_STAT.PKAINT1</code> bit provides the status of the PKA Completion interrupt (after masking from the <code>PKIC_EN_CTL</code> register).
		0 Interrupt is inactive
		1 Interrupt is pending

Polarity Control Register

The `PKIC_POL_CTL` register is used to configure the signal polarity for each individual interrupt. During the initialization phase of the PKA the host processor must set each interrupt in this register to (high level/rising edge or low level/falling edge).

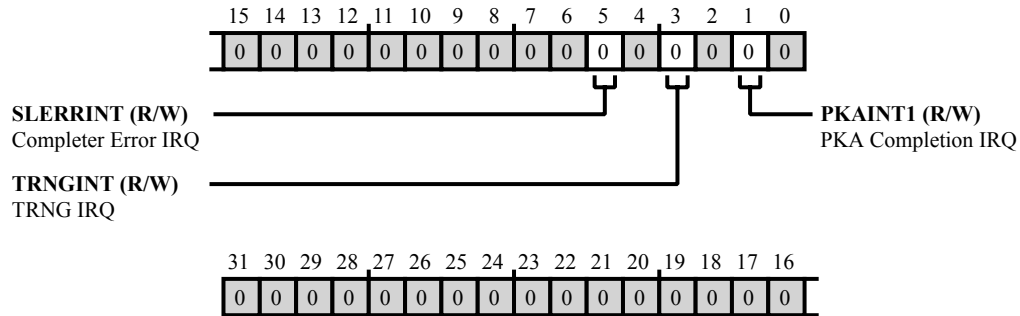


Figure 32-6: PKIC_POL_CTL Register Diagram

Table 32-9: PKIC_POL_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
5 (R/W)	SLERRINT	Completer Error IRQ. The <code>PKIC_POL_CTL.SLERRINT</code> bit provides polarity control for the completer error interrupt.
		0 Low level/falling edge
		1 High level/rising edge
3 (R/W)	TRNGINT	TRNG IRQ. The <code>PKIC_POL_CTL.TRNGINT</code> bit provides polarity control for the TRNG interrupt.
		0 Low level/falling edge
		1 High level/rising edge
1 (R/W)	PKAINT1	PKA Completion IRQ. The <code>PKIC_POL_CTL.PKAINT1</code> bit provides polarity control for PKA completion interrupt.
		0 Low level/falling edge
		1 High level/rising edge

Raw Status Register

The `PKIC_RAW_STAT` register reflects the status of the individual interrupts before masking with the `PKIC_EN_CTL` register.

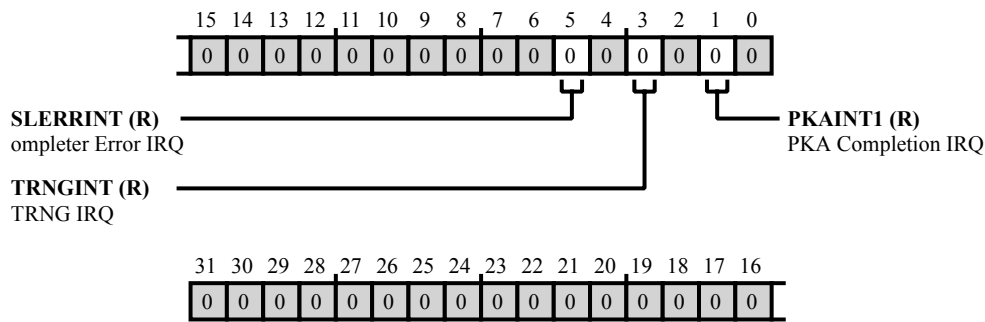


Figure 32-7: PKIC_RAW_STAT Register Diagram

Table 32-10: PKIC_RAW_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
5 (R/NW)	SLERRINT	ompleter Error IRQ. The <code>PKIC_RAW_STAT.SLERRINT</code> bit provides the raw status of the completer error interrupt.
		0 Inactive interrupt
		1 Pending interrupt
3 (R/NW)	TRNGINT	TRNG IRQ. The <code>PKIC_RAW_STAT.TRNGINT</code> bit provides the raw status of the TRNG interrupt where 1=pending and 0=inactive.
		0 Inactive interrupt
		1 Pending interrupt
1 (R/NW)	PKAINT1	PKA Completion IRQ. The <code>PKIC_RAW_STAT.PKAINT1</code> bit provides raw status of the PKA completion interrupt where 1=pending and 0=inactive.
		0 Inactive interrupt
		1 Pending interrupt

Type Control Register

The `PKIC_TYPE_CTL` register is used to configure the signal type for each individual interrupt. During the initialization phase of the PKA the host processor must set each interrupt in this register to level or edge.

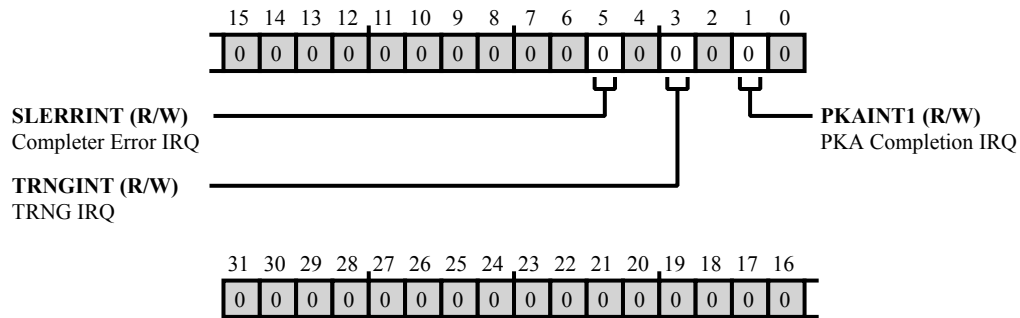


Figure 32-8: PKIC_TYPE_CTL Register Diagram

Table 32-11: PKIC_TYPE_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
5 (R/W)	SLERRINT	Completer Error IRQ. The <code>PKIC_TYPE_CTL.SLERRINT</code> bit provides signal type control for the completer error interrupt.
		0 Level
		1 Edge
3 (R/W)	TRNGINT	TRNG IRQ. The <code>PKIC_TYPE_CTL.TRNGINT</code> bit provides signal type control for the TRNG interrupt.
		0 Level
		1 Edge
1 (R/W)	PKAINT1	PKA Completion IRQ. The <code>PKIC_TYPE_CTL.PKAINT1</code> bit provides signal type control for the PKA completion interrupt.
		0 Level
		1 Edge

33 True Random Number Generator (TRNG)

The TRNG engine provides a true, non-deterministic, noise source for generating keys, Initialization Vectors (IVs), and other random number requirements. Other non-cryptographic purposes include statistical sampling, retry timers for communications protocols and noise generation.

TRNG Features

The TRNG features include:

- Hardware-based non-deterministic random number generator
- ANSI X9.31 postprocessing (depending on processor)
- Redundant 'fail-safe' design with self-test circuits
- Reliable shot noise oscillator implementation with auto-tuning
- Debug output to allow monitoring of internal operation
- Alarm count overflow and auto-tuning error interrupts
- Buffer to allow generation of large blocks of random data in the background

TRNG Functional Description

The following sections provide details on the function of the TRNG module.

ADSP-21560_21561_21564_21568 TRNG Register List

The True Random Number Generator (TRNG) provides random number generation, intended mainly for security-related applications. A set of registers governs TRNG operations. For more information on TRNG functionality, see the TRNG register descriptions.

Table 33-1: ADSP-21560_21561_21564_21568 TRNG Register List

Name	Description
TRNG_ALMCNT	TRNG Alarm Counter Register

Table 33-1: ADSP-21560_21561_21564_21568 TRNG Register List (Continued)

Name	Description
TRNG_ALMMSK	TRNG Alarm Mask Register
TRNG_ALMSTP	TRNG Alarm Stop Register
TRNG_BLKCNT	TRNG Block Count Register
TRNG_CFG	TRNG Configuration Register
TRNG_CNT	Counter Register
TRNG_CTL	TRNG Control Register
TRNG_FRODETUNE	TRNG FRO De-tune Register
TRNG_FROEN	TRNG FRO Enable Register
TRNG_INPUT[n]	TRNG Input Registers
TRNG_INTACK	TRNG Interrupt Acknowledge Register
TRNG_KEY[n]	Post-Process Key Registers
TRNG_LFSR_H	TRNG LFSR Access Register
TRNG_LFSR_L	TRNG LFSR Access Register
TRNG_LFSR_M	TRNG LFSR Access Register
TRNG_MONOBITCNT	TRNG Monobit Test Result Register
TRNG_OUTPUT[n]	TRNG Output Registers
TRNG_POKER[n]	TRNG Poker Test Result Registers
TRNG_RUNCNT	TRNG Run Count Registers
TRNG_RUN[n]	TRNG Run Test State and Result Registers
TRNG_STAT	TRNG Status Register
TRNG_TEST	TRNG Test Register
TRNG_V[n]	TRNG Post-Process "V" Value Registers

Random Number Generation

The random numbers that the TRNG generates are produced by sampling Free Running Oscillators (FRO). The (TRNG_CTL.STARTUPCYC) bit field along with TRNG_CFG.MINREFCYC (minimum refill cycles) and TRNG_CFG.MAXREFCYC (maximum refill cycles) bit fields determine the number of samples taken to generate the first random value and subsequent random values.

1. After enabling the TRNG (TRNG_CTL.TRNGEN bit =1), a number of FRO output samples defined by the TRNG_CTL.STARTUPCYC bit field are gathered in the main linear-feedback shift register (LFSR) before taking a snapshot of the LFSR and storing that snapshot in the random data buffer (after optional post-processing).

2. After taking a snapshot of the LFSR, a number of FRO output samples defined by the `TRNG_CFG.MINREFCYC` bit field, are gathered in the main LFSR. If the random data buffer is full, sample-taking continues until the number of samples (counting from the snapshot) matches the number of samples defined by the `TRNG_CFG.MAXREFCYC` bit field. At that point, the TRNG switches off the FROs and powers down.
3. If, after the `TRNG_CFG.MINREFCYC` sampling period, the random data buffer is not completely filled, a new snapshot of the LFSR is taken and stored in the random data buffer (after optional postprocessing). Control branches back to point step 2 above and a new `TRNG_CFG.MINREFCYC` sample period starts.

Locking Detection and Prevention

Lock detection in functional mode uses the sampled outputs of the individual FROs. A FRO alarm event is declared when a repeating pattern (of up to four sample lengths) is detected continuously for the number of samples defined in the alarm threshold `TRNG_ALMCNT.ALMTHRESH` bit field. The alarm event is logged by setting the bit that corresponds to the FRO that caused the alarm in the `TRNG_ALMMSK` register. If that bit was already set, the corresponding bit in the `TRNG_ALMSTP` register is set. The FRO is switched off to prevent further alarm events from that FRO. If the `TRNG_ALMMSK` register bit was not yet set, the FRO is restarted automatically in an attempt to break locking.

The shutdown count field in the alarm count `TRNG_ALMCNT.SHDNCNT` register monitors the number of FROs switched off. (It counts the number of 1 bits in the `TRNG_ALMSTP` register.) The shutdown threshold field (`TRNG_ALMCNT.SHDNTHRESH`) can be configured to generate the shutdown overflow interrupt (`TRNG_STAT.SHDNOVR`). When the shutdown count in the `TRNG_ALMCNT.SHDNCNT` bit field exceeds the shutdown threshold in the `TRNG_ALMCNT.SHDNTHRESH` bit field, the shutdown overflow bit (`TRNG_STAT.SHDNOVR`) is set to 1 (which can be used to generate an interrupt).

Software can use two strategies for the TRNG operation:

- **Monitored Operation.** Software checks the `TRNG_ALMMSK` register at regular intervals (on the order of seconds). If a bit is set in that register, then the program must also check the `TRNG_ALMSTP` register to determine if a FRO was shut down due to multiple alarm events. If no FROs are shut down, the program clears the `TRNG_ALMMSK` register to remove the incidental alarm events. If one or more FROs are shut down, the host processor can modify the delay selection of those FROs using the `TRNG_FRODETUNE` register to prevent further locking. For this type of operation, the shutdown threshold is normally set to a low value (two, for example). The shutdown overflow interrupt can then be used to signal abnormal operation conditions or the breakdown of FROs.
- **Unmonitored Operation.** Software sets the shutdown threshold to the acceptable number of FROs to be shut down before taking corrective actions. It then uses the shutdown overflow interrupt to initiate corrective actions (clearing the `TRNG_ALMMSK` and `TRNG_ALMSTP` registers, toggling bits in the `TRNG_FRODETUNE` register). The software must monitor the time interval between these interrupts. If they occur too often (for example, within a minute after each other), this frequency indicates abnormal operating conditions or the breakdown of FROs.

Run Testing

Run Test

The TRNG block counts the number of consecutive zeros and ones (runs) in the data stream shifted into the main LFSR. The run length and bit value is then used to increment a specific bucket counter for these values. After 20,000 bits, the bucket counters must be within specified limits for this test to pass. If not, a run fail interrupt (RUNFAIL) is generated.

Table 33-2: Allowable Limits on Runs of 0's and 1's

Run Count	Bit Value	Min (inclusive)	Max (inclusive)
1	0	2267	2733
	1	2267	2733
2	0	1079	1421
	1	1079	1421
3	0	502	748
	1	502	748
4	0	233	402
	1	233	402
5	0	90	223
	1	90	223
6 and up	0	90	233
	1	90	233

Long Run Test

The long run test fails immediately when a run longer than 33 bits is found and a long run fail (TRNG_STAT.LRUNFAIL) interrupt is generated.

Noise Source Test

A noise source failure is declared when a run of 48 or more identical bits is found and a noise fail interrupt (TRNG_STAT.NOISEFAIL) is generated.

The status and counts are stored in the [TRNG_RUNCNT](#) and [TRNG_RUN\[n\]](#) registers. Unless otherwise indicated, all counters and state bits in these registers are reset when writing a 1 to either the monobit fail acknowledge (TRNG_INTACK.MBITFAIL), the run fail acknowledge (TRNG_INTACK.RUNFAIL), or the poker fail acknowledge (TRNG_INTACK.PKRFail) bits.

Monobit Testing

The TRNG block performs the monobit test on blocks of 20,000 bits (in parallel with the run test and poker test). It monitors the number of zeros and ones in the data stream shifted into the main LFSR. At the start of the block,

the counter is initialized to 10,000. Each 1 value increments the counter, each 0 value decrements the counter. After 20,000 bits, the counter value must be within 9310–10690 (inclusive) for this test to pass. If not, a monobit fail interrupt (`TRNG_STAT.MBITFAIL`) is generated. The AIS-31 standard (test T1, ref 0) specifies this run time testing of the TRNG and the parameters.

NOTE: The actual limits stated here are different than the limits stated in the AIS-31 standard due to the implementation. The circuitry in the TRNG uses an up-down counter while the standard just evaluates the sum of the 20,000 bits.

When the continue poker (`TRNG_TEST.CONTPKR`) bit is set to 1, the test is not stopped after 20,000 bits. The counter keeps incrementing and decrementing (protected against overflow and underflow). But, no actual limit checking happens. The offset from starting value 10,000 indicates the balance of 0 and 1 bits that were checked since the start of the continuous test. The offset is twice the number of missing or extra 1 bits. (An extra 1 bit adds an increment operation and removes one decrement operation. So, having 10,001 1 bits in the block gives a counter value of 10,002.)

Poker Testing

The poker test is run in parallel with the monobit test and run test. Counters in the `TRNG_POKER[n]` registers are used to count the occurrences of one specific 4-bit value in the data stream fed into the main LFSR. All of the counters are decremented by one every 64 data bits and reset to their start value every 20,000 bits. All counters start at a value of –1 and are decremented 312 times during the 20,000-bit test run.

Each 8-bit counter holds a two's complement value and does not overflow past the range of –128 to +127. At the end of the 20,000-bits block, the values of the counters that contain a single 1 bit appended at the least significant bit side are individually squared and then added. The poker test fails with a poker fail (`PKRFAIL`) interrupt when:

- the resulting sum (accumulated in the `TRNG_MONOBITCNT` register) is outside the range 1288 – 71750 (inclusive), or
- one of the counters tries to increment or decrement outside its limit range

NOTE: The poker test fails when the 4-bit values of the data stream are distributed too evenly (with eight counters having incremented 312 times and the others incremented 313 times). This failure is intentional. The minimum mean deviation from the expected value of 312.5 is 4.5. Failure at counter overflow is not an official part of the poker test as specified in the AIS-31 standard (ref 0). It can be shown that the maximum deviation for one counter's value from the mean value of 312.5 (without the poker test failing) is 129.5. Since this deviation is more than 40% of the mean value, it indicates that something is wrong. Here, the counter overflow failure is combined with the official poker test failure.

When the continue poker (`TRNG_TEST.CONTPKR`) bit is set to 1, the test does not stop after 20,000 bits. The counters keep incrementing and decrementing (the latter every 64 bits). A `PKRFAIL` interrupt is generated when one of the counters tries to increment or decrement outside its limit range.

Data for Tests

For the monobit test, run test and poker test circuits self-test, the test data written to the `TRNG_INPUT[n]0` register is used for the monobit test and run test bit-by-bit. It executes from bit 31 down to bit 0. For the poker test, the written test data is used in eight blocks of 4 bits, starting from bits 31:28 down to bits 3:0.

When the `TRNG_TEST.CONTPKR` bit =0 during run test and poker test circuits self-tests, the state of the poker and runs test status registers is frozen after all calculations are made and after inputting 20,000 test bits. This state allows time to read the test results. These calculations take around 20 clock cycles to complete. The status registers are reset to their starting states when the first word for the next test block of test bits is written to the `TRNG_INPUT[n]0` register. Then, the contents of this word are processed.

X9.31 Postprocessing

Postprocessing is available on some parts using the TRNG. If available, the postprocessing block is situated after the main LFSRs and before the output buffer that stores the random numbers for consumption. The online test logic for monobit, run, and poker testing is before the postprocessor block. The bits used for testing are from the main LFSR.

The postprocessor is based on the ANSI X9.31 specification using 3-Key 3DES cipher algorithm. It does not provide any more entropy than is what is already achieved from sampling the Free Running Oscillators (FROs). The use of the postprocessor only helps with applications requiring the use of an X9.31 compliant RNG.

The following section is an example of postprocessing.

The variables include:

- DT is a 64-bit date/time vector. This value is the input coming from random bits from the main LFSR.
- I is an intermediate value, a 64-bit value
- R is the final result, a 64-bit value
- V is a 64-bit seed value that is to be kept secret
- K is the 3-key for 3DES, each being 64-bits

The postprocessing uses the steps:

1. The intermediate value I is calculated: $I = 3DES_{EDE}(K, DT)$ with 3DES.
2. The result R is calculated: $R = 3DES_{EDE}(K, I \text{ XOR } V)$.
3. Finally, V is updated: $V = 3DES_{EDE}(K, R \text{ XOR } I)$.

TRNG Block Diagram

The *System Block Diagram of the TRNG* diagram shows the system block for the TRNG. The system includes:

- Free Running Oscillators (FROs) that are the source of sampled bits
- A post-processing unit (processor dependent) for standards compliance

- Test circuitry to detect non-randomness due to failures in the system

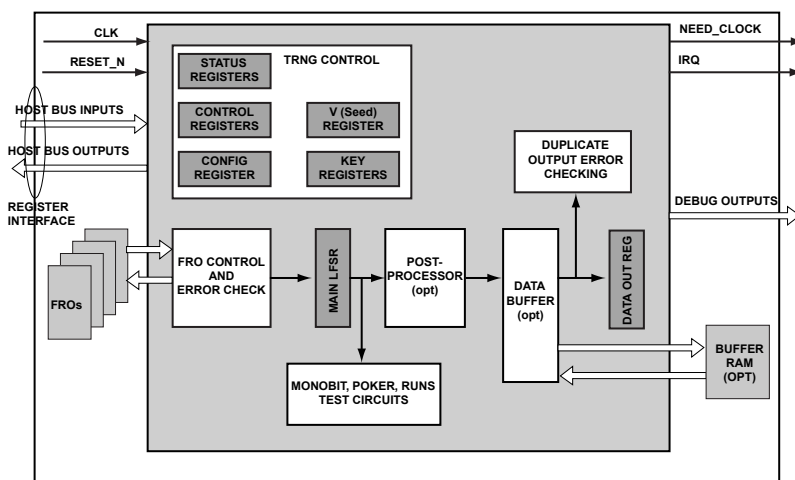


Figure 33-1: System Block Diagram of the TRNG

TRNG Architectural Concepts

The random numbers are accessible to the host processor in four 32-bit registers allowing a single burst read of a 128-bit random number. Acknowledging the data ready (interrupt) state causes the TRNG to move a new value (when available in the data buffer) to the TRNG output register. The TRNG always tries to keep the data buffer completely full. Pulling out data starts the regeneration of a new number by:

- Enabling the FROs
- Capturing their outputs in the LFSR, and
- Cryptographically postprocessing the values from the LFSR

The process produces new random values to replenish the buffer.

The major functional blocks of the TRNG module are:

- The actual TRNG core with control and test circuits, optional post-processor, and optional data buffer control logic
- Free Running Oscillators (FROs) instantiated outside the TRNG core
- A 128-byte buffer RAM instantiated outside the TRNG core

In the TRNG core, the true entropy source uses FROs as the basic building block. The accumulation of timing jitter, caused (for the largest part) by shot noise, creates uncertainty intervals for the output transitions of each FRO. Sampling within an uncertainty interval generates a single bit of entropy, which is ‘accumulated’ in a LFSR. As the uncertainty interval is very narrow compared to the cycle time of a FRO, the mean amount of entropy generated per sample is small (less than 1/100 bit per sample). To increase the entropy generation rate, multiple FROs are used in parallel.

The FROs are asynchronous to one another and asynchronous to the sampling clock to make their behavior truly non-deterministic.

The output signals of the FROs are sampled at regular intervals (in general, at the TRNG core module clock frequency). The samples feed into an error detection circuit that checks for repeating patterns coming out of a FRO. If a repeating pattern persists for a configurable number of samples, the FRO is suspect of having synchronized to (a harmonic of) the sampling interval. This activity drastically reduces the amount of entropy generated by that FRO. The error detection circuit signals this activity as a FRO error event.

Error events can occur during normal operation. The FRO control circuit attempts to restart a FRO that on a first error event. A second error event causes an automatic shutdown of the FRO. Because there are multiple FROs, shutting down a FRO reduces the amount of entropy generated, but it does not immediately jeopardize the TRNG operation. A limit can be configured below which the number of operational FROs does not drop. If this limit is crossed, an interrupt can be generated on the host processor. Software on the host processor can then attempt to prevent frequent locking of a FRO by *de-tuning* it to a slightly different frequency.

An XOR tree combines the sampled outputs and feeds them into an 81-bit LFSR to accumulate entropy and whiten the random bits stream.

NOTE: Here, *whitening* means balancing the number of one and zero bits in the stream.

TRNG Operating Modes

The TRNG has the following operating modes.

Normal Reading Mode

In normal reading mode, random data can only be read out of the output registers of the TRNG module when the ready bit (`TRNG_STAT.RDY`) = 1. Acknowledging the data by writing a 1 to the ready acknowledge bit in the `TRNG_INTACK.RDY` register clears the ready bit from the status register and clears the output registers. The registers remain at zero until the next 128-bit data block is available.

Secure Reading Mode

An attacker can try to read the output registers (without acknowledging the data) to obtain a copy of data to be read later by an application. To block this attack, secure reading mode is used. In this mode, enable reading from the output registers (by writing 0x00 to the open read gate bits (`TRNG_INTACK.OPENRDGATE`)) before it is possible to access the output registers. Enabling the reading starts a timeout (controlled by the `TRNG_CFG.RDTIMEOUT` bit field). When this timeout expires, reading is disabled, and the offered data is acknowledged so that it is not offered again. The host processor must set this timeout such that there is enough time to read the output registers and perform a normal data acknowledge (which stops the timeout).

Test Mode

In addition to the test circuitry that operates during normal operation, the TRNG has a test mode that allows further diagnosis when errors occur. In test mode, programs have access to the main LFSR through the

`TRNG_LFSR_L`, `TRNG_LFSR_M`, and `TRNG_LFSR_H` registers. Programs can also control the finite state machine sample counter using the `TRNG_CNT` register.

In test mode, the TRNG can be configured to test individual or a chosen set of FROs. It can also be configured to feed test patterns to the delay chain.

TRNG Data Transfer Modes

The host processor reads four 32-bit registers to access the random numbers. Once the registers are read and the data ready interrupt has been acknowledged, the TRNG moves more data from the internal buffer to the output registers (`TRNG_OUTPUT[n]`).

TRNG Event Control

There are eight events that the TRNG generates. The events are common error events from the run-time testing of the TRNG. While the TRNG is operating and generating random bits, test circuitry is also running statistical tests. The tests determine if the sources of the random bits have started to fail and are not truly producing random bits. There is also a single event to signal when data is ready in the output registers.

These events are captured in the `TRNG_STAT` register and can generate a single interrupt in the Public Key Interrupt Controller (PKIC). The individual events can be masked so the interrupt is not triggered. This configuration uses the `TRNG_CTL` register. The event and associated interrupt can be acknowledged in the `TRNG_INTACK` register.

The *TRNG Interrupt Signals* table lists all events or interrupts that the TRNG can generate.

TRNG Interrupt Signals

The TRNG provides a total of eight interrupts multiplexed into one output.

Table 33-3: TRNG Interrupt Signals

Interrupt	Name	Description
0	RDY	Ready. Random number is ready to be read from registers
1	SHDNOVR	Shutdown Overflow. The number of FRO's automatically shut down due to failures or errors have gone above the threshold specified in <code>TRNG_ALMCNT.SHDNTHRESH</code> .
2	STUCKOUT	Stuck Out. Logic circuitry around the output registers has detected the same output has been provided twice.
3	NOISEFAIL	Noise Fail. Logic circuitry monitoring the data shifted into the main LFSR detected 48 identical bits, which is considered a noise source failure.
4	RUNFAIL	Run Fail. Logic circuitry monitoring the data shifted into the main LFSR detected an out-of-bounds value for at least one of the <code>TRNG_RUN[n]</code> counters after checking 20,000 bits.

Table 33-3: TRNG Interrupt Signals (Continued)

Interrupt	Name	Description
5	LRUNFAIL	Long Run Fail. Logic circuitry monitoring the data shifted into the main LFSR detected 34 identical bits.
6	PKRFAIL	Poker Fail. Logic circuitry monitoring the data shifted into the main LFSR detected an out-of-bounds value in at least one of the 16 <code>TRNG_POKER[n]</code> counters or an out-of-bounds sum of squares values after checking 20,000 bits.
7	MBITFAIL	Monobit Fail. Logic circuitry monitoring the data shifted into the main LFSR detected an out-of-bounds number of 1's after checking 20,000 bits.

ADSP-21560_21561_21564_21568 TRNG Register Descriptions

True Random Number Generator (TRNG) contains the following registers.

Table 33-4: ADSP-21560_21561_21564_21568 TRNG Register List

Name	Description
<code>TRNG_ALMCNT</code>	TRNG Alarm Counter Register
<code>TRNG_ALMMSK</code>	TRNG Alarm Mask Register
<code>TRNG_ALMSTP</code>	TRNG Alarm Stop Register
<code>TRNG_BLKCNT</code>	TRNG Block Count Register
<code>TRNG_CFG</code>	TRNG Configuration Register
<code>TRNG_CNT</code>	Counter Register
<code>TRNG_CTL</code>	TRNG Control Register
<code>TRNG_FRODETUNE</code>	TRNG FRO De-tune Register
<code>TRNG_FROEN</code>	TRNG FRO Enable Register
<code>TRNG_INPUT[n]</code>	TRNG Input Registers
<code>TRNG_INTACK</code>	TRNG Interrupt Acknowledge Register
<code>TRNG_KEY[n]</code>	Post-Process Key Registers
<code>TRNG_LFSR_H</code>	TRNG LFSR Access Register
<code>TRNG_LFSR_L</code>	TRNG LFSR Access Register
<code>TRNG_LFSR_M</code>	TRNG LFSR Access Register
<code>TRNG_MONOBITCNT</code>	TRNG Monobit Test Result Register
<code>TRNG_OUTPUT[n]</code>	TRNG Output Registers
<code>TRNG_POKER[n]</code>	TRNG Poker Test Result Registers
<code>TRNG_RUNCNT</code>	TRNG Run Count Registers
<code>TRNG_RUN[n]</code>	TRNG Run Test State and Result Registers

Table 33-4: ADSP-21560_21561_21564_21568 TRNG Register List (Continued)

Name	Description
TRNG_STAT	TRNG Status Register
TRNG_TEST	TRNG Test Register
TRNG_V[n]	TRNG Post-Process "V" Value Registers

TRNG Alarm Counter Register

The `TRNG_ALMCNT` register, together with the `TRNG_ALMMSK` and `TRNG_ALMSTP` registers, can be used by the host processor to determine if the FRO/sample cycle locking is a problem. Note that incidental alarm events are expected to occur during normal operation. This register also controls the way the monobit test and poker test circuits operate (using the standard 20,000 bit blocks or running continuously).

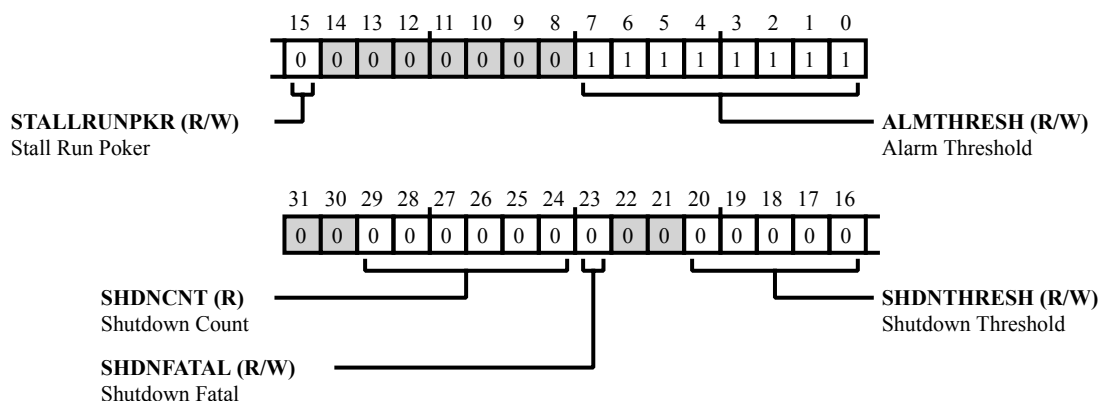


Figure 33-2: TRNG_ALMCNT Register Diagram

Table 33-5: TRNG_ALMCNT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:24 (R/NW)	SHDNCNT	Shutdown Count. This read-only field indicates the number of 1 bits in the <code>TRNG_ALMSTP</code> register, the number of FRO's that's been turned off.
23 (R/W)	SHDNFATAL	Shutdown Fatal. When the <code>TRNG_ALMCNT.SHDNFATAL</code> bit field is set, the shutdown overflow (SHDNOVR) interrupt is considered a fatal error requiring taking the complete TRNG engine off-line.
20:16 (R/W)	SHDNTHRESH	Shutdown Threshold. The <code>TRNG_ALMCNT.SHDNTHRESH</code> bit field provides the threshold setting for generating the shutdown overflow (SHDNOVR) interrupt, which is activated when the shutdown count (<code>TRNG_ALMCNT.SHDNCNT</code>) value in this register exceeds the threshold value set here.
15 (R/W)	STALLRUNPKR	Stall Run Poker. When the <code>TRNG_ALMCNT.STALLRUNPKR</code> bit is set, stalls the Monobit Test, Run Test and Poker Test circuits when either the <code>TRNG_STAT.MBITFAIL</code> , <code>TRNG_STAT.RUNFAIL</code> or <code>TRNG_STAT.PKRFAIL</code> bits =1. This allows inspection of the state of the result counters (which would otherwise be reset immediately for the next 20,000 bits block to test).

Table 33-5: TRNG_ALMCNT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/W)	ALMTHRESH	Alarm Threshold. The <code>TRNG_ALMCNT.ALMTHRESH</code> bit field sets the alarm detection threshold for the repeating pattern detectors on each FRO. A FRO alarm event is declared when a repeating pattern (of up to four samples length) is detected continuously for the number of samples defined by this fields value. Reset value 255 (decimal) should keep the number of alarm events to a manageable level.

TRNG Alarm Mask Register

A set bit (=1) in the `TRNG_ALMMSK` register signifies an alarm event and is used by the host processor to determine which of the individual FROs generated the alarm. If a bit in this register is set, the corresponding bit in the `TRNG_ALMSTP` register is set and the FRO is turned off by clearing the corresponding bit in the `TRNG_FROEN` register. If a bit is not set, the FRO restarts automatically to try to break sample cycle locking that could have caused the alarm event.

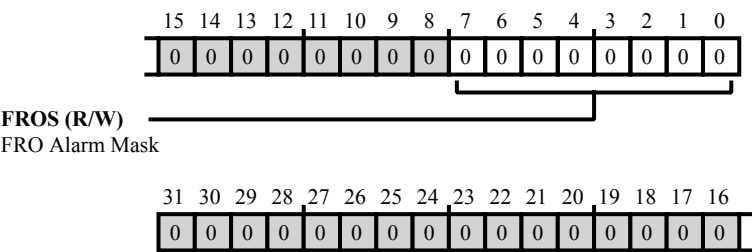


Figure 33-3: TRNG_ALMMSK Register Diagram

Table 33-6: TRNG_ALMMSK Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/W)	FROS	FRO Alarm Mask. The <code>TRNG_ALMMSK.FROS</code> bit field provides logging for the alarm events of individual FROs. A 1 in bit [n] indicates FRO n experienced an alarm event.

TRNG Alarm Stop Register

The `TRNG_ALMSTP` register is used by the host processor to determine which of the individual FROs generated more than one alarm event in quick succession. If a FRO generates an alarm event while a previous event is still logged in the `TRNG_ALMMSK` register, the corresponding bit in this register is set (=1) and the FRO is turned off by clearing (=0) the corresponding bit in the `TRNG_FROEN` register. The `TRNG_ALMCNT.SHDNCNT` bit field keeps track of the number of bits that are set in this register.

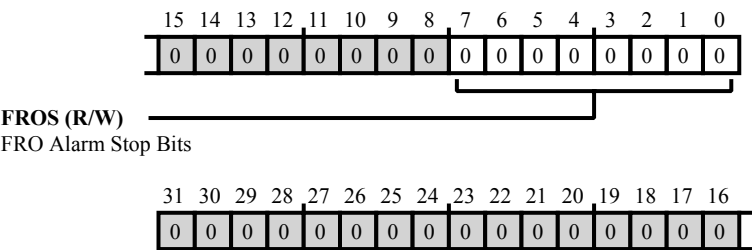


Figure 33-4: TRNG_ALMSTP Register Diagram

Table 33-7: TRNG_ALMSTP Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/W)	FROS	FRO Alarm Stop Bits. The <code>TRNG_ALMSTP.FROS</code> bit field provides logging for the alarm events of individual FROs. A 1 in bit [n] indicates FRO n experienced more than one alarm event in quick succession and has been turned off. A 1 in this field forces the corresponding bit in the <code>TRNG_FROEN</code> register to 0.

TRNG Block Count Register

The `TRNG_BLKCNT` register is the counter for the 128-bit blocks generated by the post-processor. These bits are forced to zero when the post-processor is disabled and are cleared to zero when an internal re-seed operation has finished. This register can be used by driver software to determine when to re-seed the post-processor.

The whole 32 bits of this register represent the amount of data (in bytes) generated since a re-seed. The `TRNG_BLKCNT` register is only present when a post-processor is available.

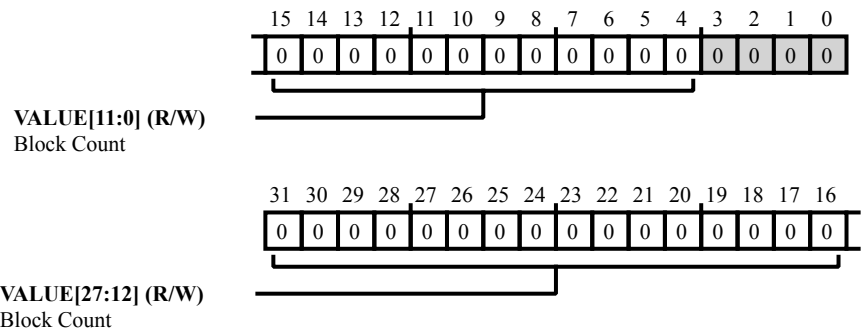


Figure 33-5: TRNG_BLKCNT Register Diagram

Table 33-8: TRNG_BLKCNT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:4 (R/W)	VALUE	Block Count. The <code>TRNG_BLKCNT.VALUE</code> bit field is the counter for the 128-bit blocks generated by the post-processor. These bits are forced to zero when the post-processor is disabled and are cleared to zero when an internal re-seed operation has finished.

TRNG Configuration Register

The `TRNG_CFG` register holds the lower and upper limits of the samples taken from the FROs in order to refill the random data buffer. This register also holds the time out value used for secure reading mode.

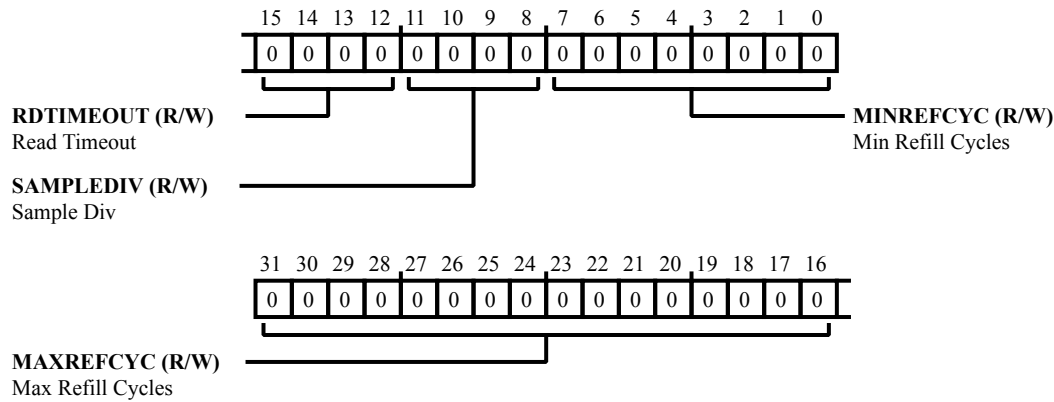


Figure 33-6: TRNG_CFG Register Diagram

Table 33-9: TRNG_CFG Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:16 (R/W)	MAXREFCYC	Max Refill Cycles. The <code>TRNG_CFG.MAXREFCYC</code> bit field determines the maximum number of samples (between 2^8 and 2^{24}) taken to re-generate entropy from the FROs after reading out a 64 bit random number. If the written value of this field is zero, the number of samples is 2^{24}, otherwise the number of samples equals the written value times 2^8. This field can only be modified while the <code>TRNG_CTL.TRNGEN</code> bit =0.
15:12 (R/W)	RDTIMEOUT	Read Timeout. The <code>TRNG_CFG.RDTIMEOUT</code> bit field controls the Secure Reading Mode. When this field is 0, Secure Reading Mode is disabled. Values in the range 115 enable Secure Reading and set a read gate closure timeout of approximately $(\text{read_timeout} + 1) \times 16$ clock input cycles. This field can only be modified while the <code>TRNG_CTL.TRNGEN</code> bit =0.
11:8 (R/W)	SAMPLEDIV	Sample Div. The <code>TRNG_CFG.SAMPLEDIV</code> bit field directly controls the number of input cycles between samples taken from the FROs. The default value 0 indicates that samples are taken every cycle, maximum value 15 (decimal) takes one sample every 16 cycles. This field must be set to a value such that the slowest FRO (even under worst-case conditions) has a cycle time less than twice the sample period. The default configuration of the FROs allows this field to remain 0. This field can only be modified while <code>TRNG_CTL.TRNGEN</code>=0.

Table 33-9: TRNG_CFG Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/W)	MINREFCYC	Min Refill Cycles. The <code>TRNG_CFG.MINREFCYC</code> bit field determines the minimum number of samples (between 2^6 and 2^{24}) taken to re-generate entropy from the FROs after reading out a 64 bit random number. If the value of this field is zero, the number of samples is fixed to the value determined by the maximum refill cycles (<code>TRNG_CFG.MAXREFCYC</code>) field, otherwise the minimum number of samples equals the written value times 64 (which can be up to 2^{14}). The number of samples defined here cannot be higher than the number defined by the <code>TRNG_CFG.MAXREFCYC</code> field (i.e. that field takes precedence). This field can only be modified while the <code>TRNG_CTL.TRNGEN</code> bit =0.

Counter Register

The `TRNG_CNT` register is used to access the main control Finite State Machine's (FSM) sample counter while the `TRNG_CTL.TSTMODE` bit =1.

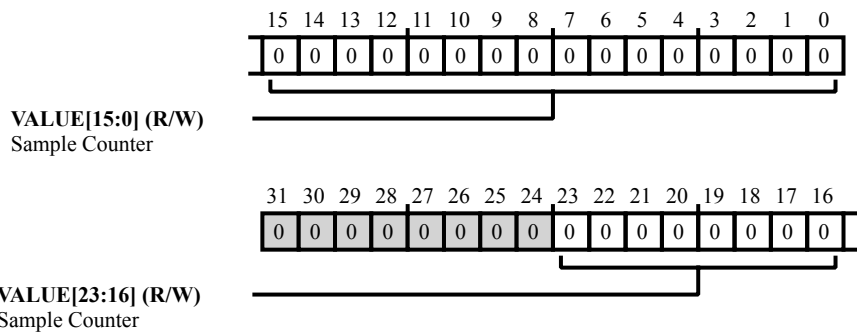


Figure 33-7: TRNG_CNT Register Diagram

Table 33-10: TRNG_CNT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
23:0 (R/W)	VALUE	Sample Counter. The <code>TRNG_CNT.VALUE</code> bit field is the sample counter used by control finite state machine. This counter can only be accessed when the <code>TRNG_CTL.TSTMODE</code> bit =1.

TRNG Control Register

The `TRNG_CTL` register must be written to start accumulating entropy before random numbers can be generated. In most cases, the `TRNG_CFG` register must also be written prior to writing the `TRNG_CTL` register. To enable the TRNG, set the `TRNG_CTL.TRNGEN` bit. This register also controls post-processing (if available). Note that when the `TRNG_CTL.TRNGEN` bit =1, the start up cycles field (`TRNG_CTL.STARTUPCYC`) and the post-processing enable bit (if available) are locked. Any writes to these fields are ignored.

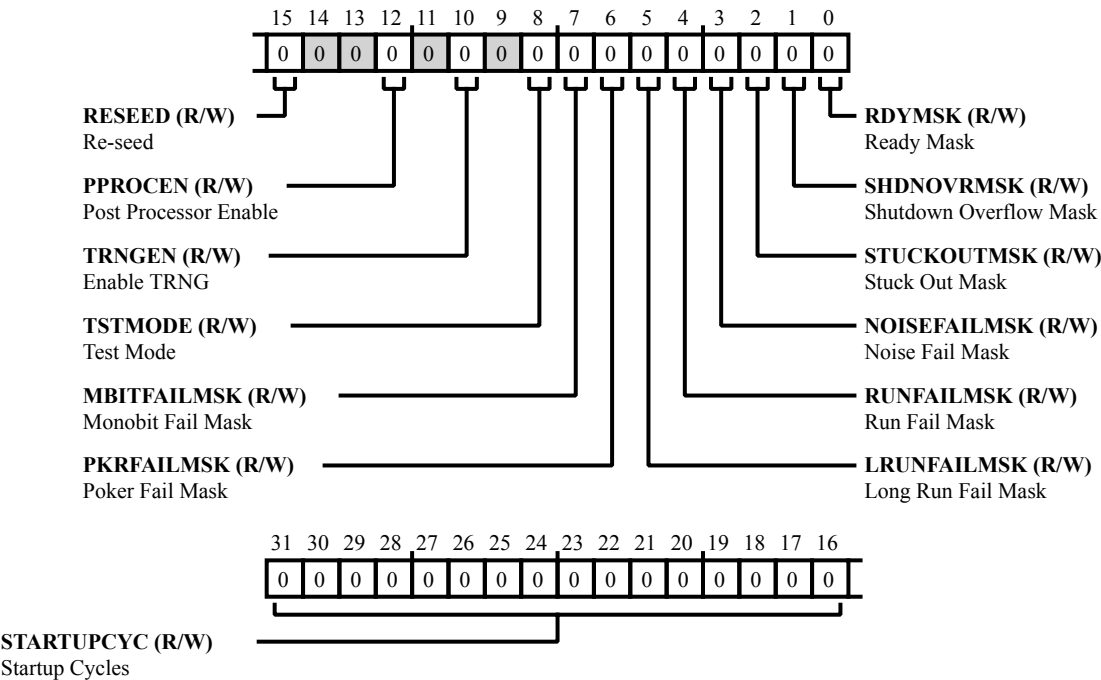


Figure 33-8: TRNG_CTL Register Diagram

Table 33-11: TRNG_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:16 (R/W)	STARTUPCYC	Startup Cycles. The <code>TRNG_CTL.STARTUPCYC</code> bit field determines the number of samples (between 2^8 and 2^{24}) taken to gather entropy from the FROs during startup. If the written value of this field is zero, the number of samples is 2^{24} , otherwise the number of samples equals the written value times 2^8 . This field can only be written when <code>TRNG_CTL.TRNGEN=0</code> before the write.

Table 33-11: TRNG_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W)	RESEED	Re-seed. The TRNG_CTL.RESEED bit is set-only, writing a 1 starts a re-seed cycle and writing a 0 has no effect. A re-seed cycle entails loading the TRNG_KEY[n] and TRNG_V[n] registers with random values generated internally these values are not visible outside the TRNG core. This bit falls back to 0 automatically after the re-seed operation is complete at that time the TRNG_BLKCNT register is reset to zero and the random data buffer is zero-ized so that any new data read from the TRNG will use the new seed values. This bit is only present when post-processing is available and can only be set to 1 when TRNG_CTL.TRNGEN=1 before the write. Note that re-seeding can be done with the post-processor disabled (normally used to seed the post-processor before enabling it). When writing a 1 to this bit, all other bits in this register remain unchanged.
12 (R/W)	PPROCEN	Post Processor Enable. Setting the TRNG_CTL.PPROCEN bit enables the FIPS post-processor. If this bit is reset to 0, the post-processor is forced back into the idle state immediately. This bit is only present when post-processing is available and can only be changed when the TRNG_CTL.TRNGEN bit (enable TRNG) was 0 before the write. To change the TRNG_CTL.PPROCEN bit during operation, first put the TRNG into reset (TRNG_CTL.TRNGEN =0). If the post-processor is enabled, it can be disabled by a subsequent write of 0 to this bit (writing this bit when the TRNG_CTL.TRNGEN bit is still 1 has no effect). The post-processor then stops immediately. To enable it, this bit must be written with 1. Changing the enabled/disabled state does not affect the contents of the TRNG_KEY[n] and TRNG_V[n] registers. After changing the state, the TRNG must be started again by setting the TRNG_CTL.TRNGEN bit to 1. Note that it is required to re-gather entropy, so the same number of start-up cycles must be used as when starting the TRNG out of a system reset state.
10 (R/W)	TRNGEN	Enable TRNG. Setting the TRNG_CTL.TRNGEN bit to 1 starts the TRNG, gathering entropy from the FROs for the number of samples determined by the value in the TRNG_CTL.STARTUPCYC (Startup Cycles) field. Resetting this bit to 0 forces all TRNG logic back into the idle state immediately. Resetting this bit to 0 also performs the Un-instantiate operation, clearing all internal post-processor registers.
8 (R/W)	TSTMODE	Test Mode. When the TRNG_CTL.TSTMODE bit is set, access is enabled to the TRNG_CNT and TRNG_LFSR_L, TRNG_LFSR_M and TRNG_LFSR_H registers (the latter are cleared before enabling access) and sets the TRNG_STAT.NEEDCLK bit for testing purposes. This bit must be set to 1 before various test modes in the TRNG_TEST register can be enabled.

Table 33-11: TRNG_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
7 (R/W)	MBITFAILMSK	Monobit Fail Mask. When the TRNG_CTL.MBITFAILMSK bit is set, this mask allows the TRNG_STAT.MBITFAIL bit to activate the (active HIGH) interrupt output.
6 (R/W)	PKRFAILMSK	Poker Fail Mask. When the TRNG_CTL.PKRFAILMSK bit is set, this mask allows the TRNG_STAT.PKRFAIL bit to activate the (active HIGH) interrupt output.
5 (R/W)	LRUNFAILMSK	Long Run Fail Mask. When the TRNG_CTL.LRUNFAILMSK bit is set, this mask allows the TRNG_STAT.LRUNFAIL bit to activate the (active HIGH) interrupt output.
4 (R/W)	RUNFAILMSK	Run Fail Mask. When the TRNG_CTL.RUNFAILMSK bit is set, this mask allows the TRNG_STAT.RUNFAIL bit to activate the (active HIGH) interrupt output.
3 (R/W)	NOISEFAILMSK	Noise Fail Mask. When the TRNG_CTL.NOISEFAILMSK bit is set, this mask allows the TRNG_STAT.NOISEFAIL bit to activate the (active HIGH) interrupt output.
2 (R/W)	STUCKOUTMSK	Stuck Out Mask. When the TRNG_CTL.STUCKOUTMSK bit is set, this mask allows the TRNG_STAT.STUCKOUT bit to activate the (active HIGH) interrupt output.
1 (R/W)	SHDNOVRMSK	Shutdown Overflow Mask. When the TRNG_CTL.SHDNOVRMSK bit is set, this mask allows the TRNG_STAT.SHDNOVR bit to activate the (active HIGH) interrupt output.
0 (R/W)	RDYMSK	Ready Mask. When the TRNG_CTL.RDYMSK bit is set, this mask allows the TRNG_STAT.RDY bit to activate the (active HIGH) interrupt output.

TRNG FRO De-tune Register

The `TRNG_FRODETUNE` register is used by the host processor to change the frequencies of individual FROs. This can reduce the number of alarm events generated by a specific FRO.

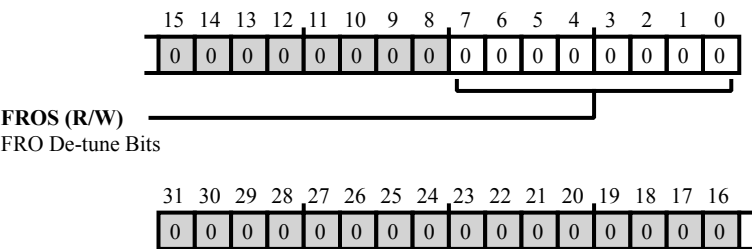


Figure 33-9: TRNG_FRODETUNE Register Diagram

Table 33-12: TRNG_FRODETUNE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/W)	FROS	FRO De-tune Bits. The <code>TRNG_FRODETUNE.FROS</code> bits De-tune the FROs. A 1 in bit [n] lets FRO n run approximately 5% faster. The value of one of these bits may only be changed while the corresponding FRO is turned off (by temporarily writing a 0 in the corresponding bit of the <code>TRNG_FROEN</code> register).

TRNG FRO Enable Register

The `TRNG_FROEN` register can be used by the host processor to enable and disable FROs individually. Only enabled FROs contribute to entropy generation, but require power to do so. Disabled FROs cannot generate alarm events.

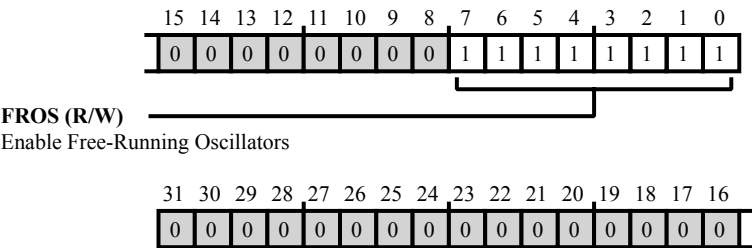


Figure 33-10: TRNG_FROEN Register Diagram

Table 33-13: TRNG_FROEN Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/W)	FROS	Enable Free-Running Oscillators. The <code>TRNG_FROEN.FROS</code> bits are the enables for the individual FROs. A 1 in bit [n] enables FRO n. The default state is all ones to enable all FROs after power-up. Note that the FROs are not actually started up before the <code>TRNG_CTL.TRNGEN</code> bit is set to 1. These bits are automatically forced to 0 (and cannot be written to 1) when the corresponding bit in the <code>TRNG_ALMSTP</code> register has value 1.

TRNG Input Registers

The `TRNG_INPUT[n]` registers are used as input for post-processor testing (if post processing is available) and as input for Monobit Test, Run Test and Poker Test functionality tests (`TRNG_INPUT0` only). They share their addresses with the corresponding `TRNG_OUTPUT[n]` registers. The least significant word is contained in the `TRNG_INPUT0` register.

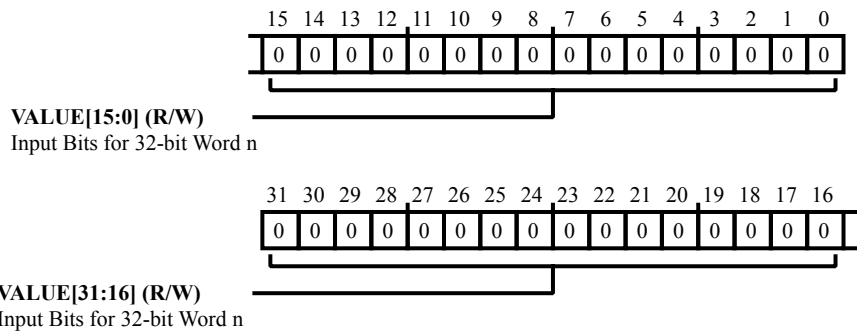


Figure 33-11: TRNG_INPUT[n] Register Diagram

Table 33-14: TRNG_INPUT[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Input Bits for 32-bit Word n. The <code>TRNG_INPUT[n].VALUE</code> bit field is used to hold 32-bits of the 64-bit word of test data for 3-DES post-processor (if available). Or, the <code>TRNG_INPUT[n].VALUE</code> bit field is used to hold 32-bit data word for Run Test and Poker Test circuits self test. Can only be written to when the <code>TRNG_STAT.TSTRDY</code> bit =1.

TRNG Interrupt Acknowledge Register

The `TRNG_INTACK` register is written to acknowledge interrupts indicated in bits [7:0] of the `TRNG_STAT` register. Writing a 1 to any of the bits [7:2] has side effects in resetting various parts of the TRNG core logic which can also be used even if no interrupts are actually active.

When acknowledging the interrupts, these bits are write '1' to clear the associated bit in `TRNG_STAT` register. The bit in this register will also automatically be reset to zero.

When secure reading mode is enabled, write bits [7:0] of this register with zeros to enable TRNG data reads from the `TRNG_OUTPUT[n]` registers. Writing bits [7:0] also starts the (configurable) timeout counter that automatically acknowledges the TRNG data (and disables reads) if the `TRNG_INTACK.RDY` bit is not written with a 1 within that timeout period.

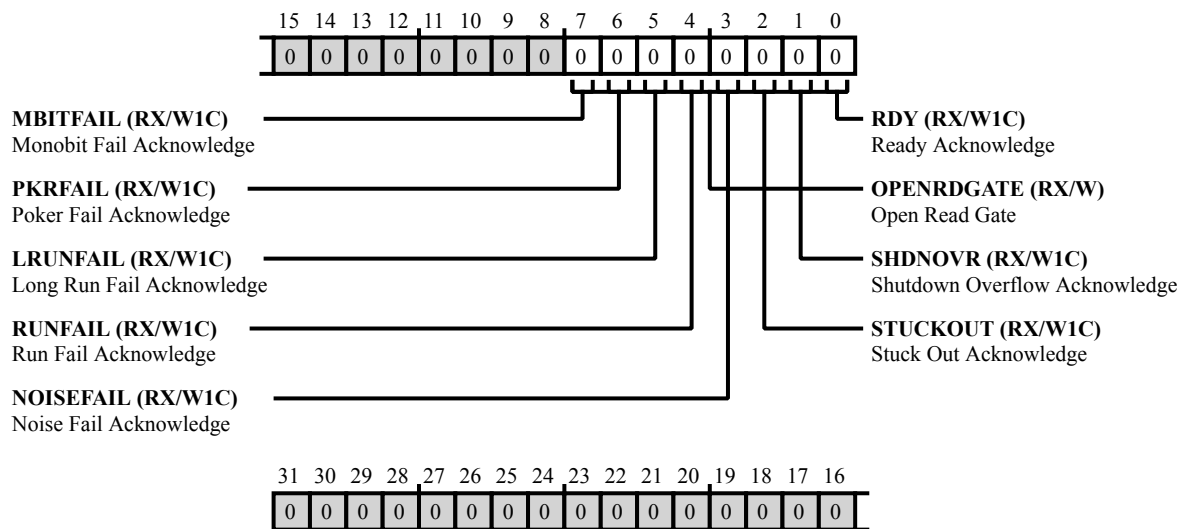


Figure 33-12: TRNG_INTACK Register Diagram

Table 33-15: TRNG_INTACK Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7 (RX/W1C)	MBITFAIL	Monobit Fail Acknowledge. Set the <code>TRNG_INTACK.MBITFAIL</code> bit to acknowledge the Monobit Fail Interrupt. This also resets all counter and state bits in the <code>TRNG_RUN[n]</code> , <code>TRNG_MONOBITCNT</code> and the <code>TRNG_POKER[n]</code> registers (except for the <code>TRNG_RUNCNT.LENMAX</code> field).
6 (RX/W1C)	PKRFAIL	Poker Fail Acknowledge. Set the <code>TRNG_INTACK.PKRFAIL</code> bit to acknowledge the Poker Fail Interrupt. This also resets all counter and state bits in the <code>TRNG_RUN[n]</code> , <code>TRNG_MONOBITCNT</code> and the <code>TRNG_POKER[n]</code> registers (except for the <code>TRNG_RUNCNT.LENMAX</code> field).

Table 33-15: TRNG_INTACK Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
5 (RX/W1C)	LRUNFAIL	Long Run Fail Acknowledge. Set the TRNG_INTACK.LRUNFAIL bit to acknowledge the Long Run Fail Interrupt. Also clears the TRNG_RUNCNT.LENMAX field.
4 (RX/W1C)	RUNFAIL	Run Fail Acknowledge. Set the TRNG_INTACK.RUNFAIL bit to acknowledge the Run Fail Interrupt. Also resets all counter and state bits in the TRNG_RUN[n], TRNG_MONOBITCNT and the TRNG_POKER[n] registers (except for the TRNG_RUNCNT.LENMAX field).
3 (RX/W1C)	NOISEFAIL	Noise Fail Acknowledge. Set the TRNG_INTACK.NOISEFAIL bit to acknowledge the Noise Fail Interrupt. Setting this bit also clears the TRNG_RUNCNT.LENMAX (Run Length Max) field, the random data buffer, the TRNG_OUTPUT[n] registers and the TRNG_STAT.RDY bit.
2 (RX/W1C)	STUCKOUT	Stuck Out Acknowledge. Set the TRNG_INTACK.STUCKOUT bit to acknowledge the Stuck Out Interrupt. Setting this bit also clears the random data buffer, the TRNG_OUTPUT[n] registers and the TRNG_STAT.RDY bit.
1 (RX/W1C)	SHDNOVR	Shutdown Overflow Acknowledge. Set the TRNG_INTACK.SHDNOVR bit to acknowledge the Shutdown Overflow Interrupt.
0 (RX/W1C)	RDY	Ready Acknowledge. The TRNG_INTACK.RDY bit allows a new number (if it is ready in the random data buffer), to directly move into the result register. Once done, the TRNG_STAT.RDY bit is reset, after at most size clock cycles.
7:0 (RX/W)	OPENRDGATE	Open Read Gate. In Secure Reading Mode, the TRNG_INTACK.OPENRDGATE bit writes an all zeros value to bits [7:0] to enable reading of TRNG data from the TRNG_OUTPUT[n] registers. This starts the timeout counter that automatically acknowledges the TRNG data (and disables reading) if the TRNG_INTACK.RDY bit is not written with a 1 within that timeout period.

Post-Process Key Registers

The `TRNG_KEY[n]` registers are used to load the key used for post-processing (if available). These registers are write-only. Reads return the values of the other registers mapped at the same addresses.

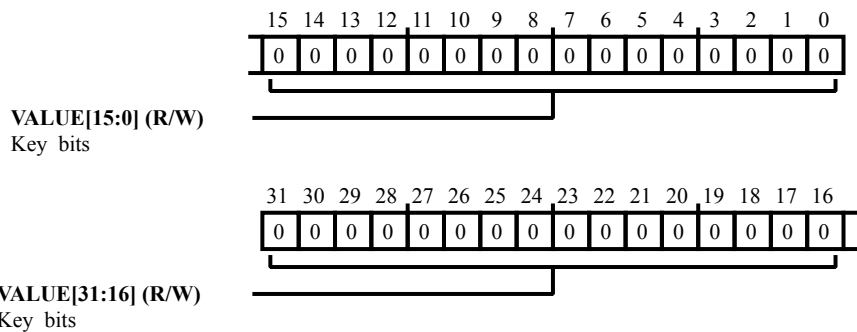


Figure 33-13: TRNG_KEY[n] Register Diagram

Table 33-16: TRNG_KEY[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Key bits. Bits for cipher key used in the post-processor.

TRNG LFSR Access Register

The `TRNG_LFSR_H` register is used to access bits [80:64] of the main entropy accumulation LFSR while in test mode (`TRNG_CTL.TSTMODE = 1`).

For security reasons, the LFSR contents are zeroed before enabling access.

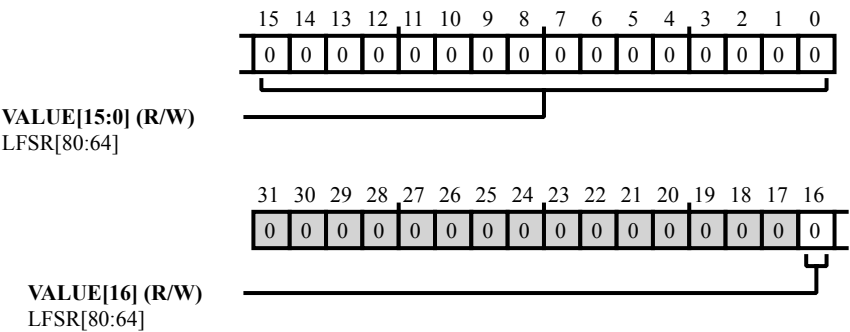


Figure 33-14: `TRNG_LFSR_H` Register Diagram

Table 33-17: `TRNG_LFSR_H` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
16:0 (R/W)	VALUE	LFSR[80:64]. The <code>TRNG_LFSR_H.VALUE</code> bit field contains bits [80:64] of the main entropy accumulation LFSR. This field can only be accessed when the <code>TRNG_CTL.TSTMODE</code> bit =1. Contents are cleared (=0) before access is enabled.

TRNG LFSR Access Register

The `TRNG_LFSR_L` register is used to access bits [31:0] of the main entropy accumulation LFSR while in test mode (`TRNG_CTL.TSTMODE` =1).

For security reasons, the LFSR contents are zeroed before enabling access.

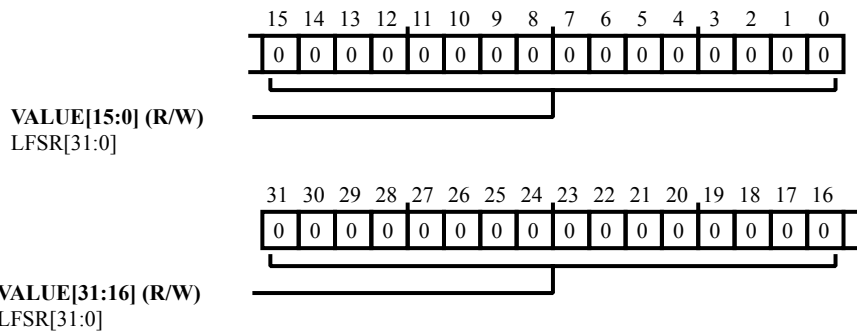


Figure 33-15: TRNG_LFSR_L Register Diagram

Table 33-18: TRNG_LFSR_L Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	LFSR[31:0]. The <code>TRNG_LFSR_L.VALUE</code> bit field contains bits [31:0] of the main entropy accumulation LFSR. This field can only be accessed when the <code>TRNG_CTL.TSTMODE</code> bit =1. Contents are cleared (=0) before access is enabled.

TRNG LFSR Access Register

The `TRNG_LFSR_M` register is used to access bits [63:32] of the main entropy accumulation LFSR while in test mode (`TRNG_CTL.TSTMODE` = 1).

For security reasons, the LFSR contents are zeroed before enabling access.

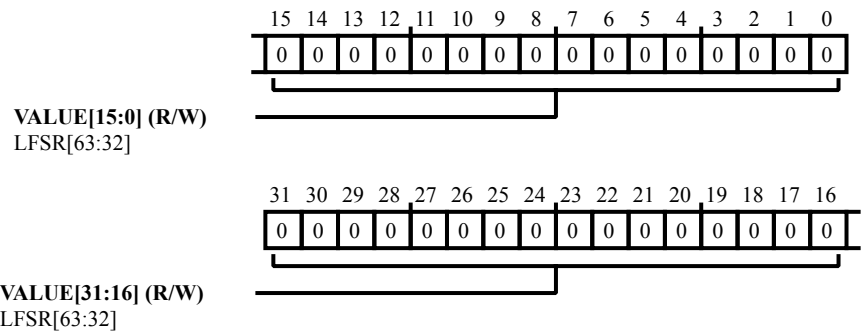


Figure 33-16: TRNG_LFSR_M Register Diagram

Table 33-19: TRNG_LFSR_M Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	LFSR[63:32]. The <code>TRNG_LFSR_M.VALUE</code> bit field contains bits [63:32] of the main entropy accumulation LFSR. This field can only be accessed when the <code>TRNG_CTL.TSTMODE</code> bit = 1. Contents are cleared (=0) before access is enabled.

TRNG Monobit Test Result Register

The `TRNG_MONOBITCNT` register accesses the counter used perform a Monobit Test as specified by the AIS-31 standard (test T1, ref 4). This test is performed on blocks of 20,000 bits (in parallel to the run test and Poker Test).

Note: Immediately after performing the actual Monobit Test at the end of the 20,000 bits block, the counter is used to accumulate the Poker Test results. As a result, the actual Monobit Test count result value can only be read in the `TRNG_MONOBITCNT` register if the test fails and the stall run Poker (`TRNG_ALMCNT.STALLRUNPKR`) bit =1.

The monobit test result register is read-only; writing it accesses the registers mapped at the same address. The counter in this register is reset when writing a 1 to either the monobit fail acknowledge (`TRNG_INTACK.MBITFAIL`), run fail acknowledge (`TRNG_INTACK.RUNFAIL`) or the poker fail acknowledge (`TRNG_INTACK.PKRFAIL`) bits.

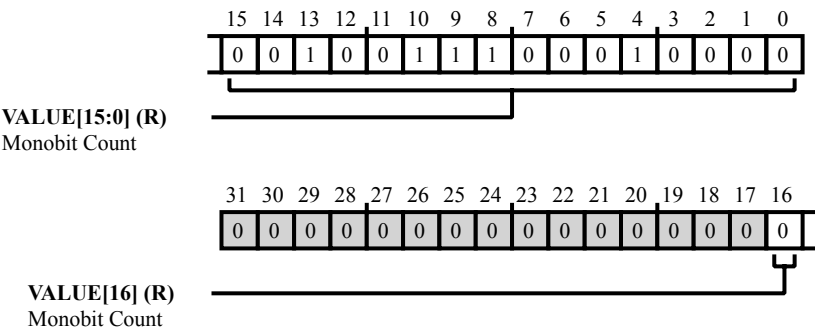


Figure 33-17: `TRNG_MONOBITCNT` Register Diagram

Table 33-20: `TRNG_MONOBITCNT` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
16:0 (R/NW)	VALUE	Monobit Count. The <code>TRNG_MONOBITCNT.VALUE</code> bit field is the up/down counter which monitors 1 and 0 bits. After 20,000 bits, this counter should have a value in the range 9310 through 10690 (inclusive) to pass the Monobit Test. This counter is protected against overflow and underflow.

TRNG Output Registers

The `TRNG_OUTPUT[n]` registers provide read access to the 128-bit random number output. A subset of these registers are also used as output for post-processor testing (if available). They share their addresses with the `TRNG_INPUT0` through `TRNG_INPUT3` registers. The least significant word is contained in the `TRNG_OUTPUT0` register.

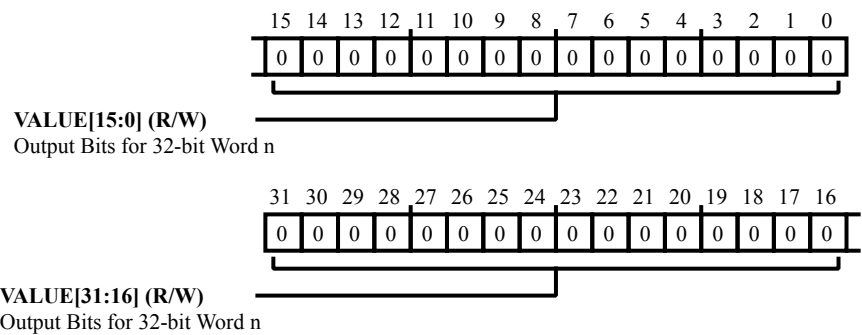


Figure 33-18: TRNG_OUTPUT[n] Register Diagram

Table 33-21: TRNG_OUTPUT[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Output Bits for 32-bit Word n. The <code>TRNG_OUTPUT[n].VALUE</code> bit field is used to holds 32 bits of the 128-bit word of random data. Only valid when the <code>TRNG_STAT.RDY</code> bit =1. Alternatively, this register holds the 32-bits of the 42-bit word of result data for 3-DES post-processing testing. Only valid when the <code>TRNG_STAT.TSTRDY</code> bit =1.

TRNG Poker Test Result Registers

The `TRNG_POKER[n]` registers are used to access the 16 counters used perform a poker test on blocks of 20,000 bits (in parallel to the monobit and run tests).

Poker test result registers are read-only; writing them accesses the registers mapped at these same addresses. All counters in these registers are reset when writing a 1 to either the monobit fail acknowledge (`TRNG_INTACK.MBITFAIL`), run fail acknowledge (`TRNG_INTACK.RUNFAIL`) or the poker fail acknowledge (`TRNG_INTACK.PKRFAIL`) bits.

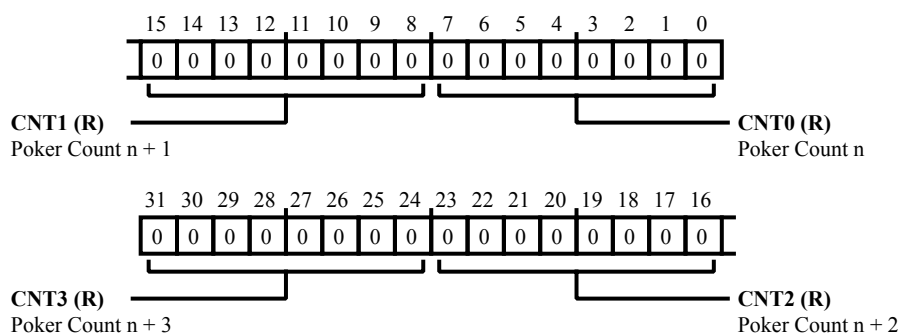


Figure 33-19: `TRNG_POKER[n]` Register Diagram

Table 33-22: `TRNG_POKER[n]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/NW)	CNT3	Poker Count $n + 3$. The <code>TRNG_POKER[n].CNT3</code> bit field provides the counter for 4-bit value 0x3, 0x7, 0xB, 0xF in <code>TRNG_POKER0</code> , <code>TRNG_POKER1</code> , <code>TRNG_POKER2</code> , <code>TRNG_POKER3</code> , respectively.
23:16 (R/NW)	CNT2	Poker Count $n + 2$. The <code>TRNG_POKER[n].CNT2</code> bit field provides the counter for 4-bit value 0x2, 0x6, 0xA, 0xE in <code>TRNG_POKER0</code> , <code>TRNG_POKER1</code> , <code>TRNG_POKER2</code> , <code>TRNG_POKER3</code> , respectively.
15:8 (R/NW)	CNT1	Poker Count $n + 1$. The <code>TRNG_POKER[n].CNT1</code> bit field provides the counter for 4-bit value 0x1, 0x5, 0x9, 0xD in <code>TRNG_POKER0</code> , <code>TRNG_POKER1</code> , <code>TRNG_POKER2</code> , <code>TRNG_POKER3</code> , respectively.
7:0 (R/NW)	CNT0	Poker Count n . The <code>TRNG_POKER[n].CNT0</code> bit field provides the counter for 4-bit value 0x0, 0x4, 0x8, 0xC in the <code>TRNG_POKER0</code> , <code>TRNG_POKER1</code> , <code>TRNG_POKER2</code> , <code>TRNG_POKER3</code> , respectively.

TRNG Run Count Registers

The `TRNG_RUNCNT` registers are used to access the 10 counters that perform a run test and long run test as specified by the AIS-31 standard (tests T3 and T4, ref 4). They are also used to perform the noise source failure test proposed in section E.5 of that same standard.

The `TRNG_RUNCNT` registers are read-only; writing them accesses the other registers which are mapped at the same addresses. Unless otherwise indicated, all counters and state bits in these registers are reset when writing a 1 to either the Monobit Fail acknowledge (`TRNG_INTACK.MBITFAIL`), Run Fail acknowledge (`TRNG_INTACK.RUNFAIL`) or the Poker Fail acknowledge (`TRNG_INTACK.PKRFAIL`) bits.

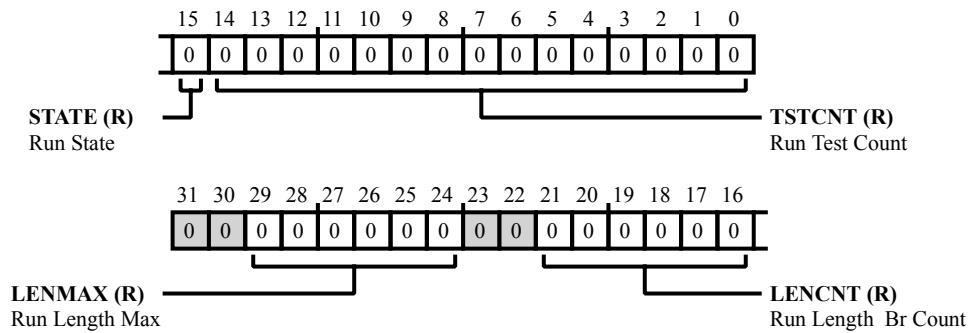


Figure 33-20: TRNG_RUNCNT Register Diagram

Table 33-23: TRNG_RUNCNT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:24 (R/NW)	LENMAX	Run Length Max. The <code>TRNG_RUNCNT.LENMAX</code> bit field configures the maximum run length count value encountered since start of test. This value is reset back to zero when writing a 1 to either the Noise Fail acknowledge (<code>TRNG_INTACK.NOISEFAIL</code>) or the Long Run Fail acknowledge (<code>TRNG_INTACK.LRUNFAIL</code>) bits.
21:16 (R/NW)	LENCNT	Run Length Br Count. The <code>TRNG_RUNCNT.LENCNT</code> bit field configures the counter for the current run of consecutive 0/1 bits; cannot increment past its maximum value of 63.
15 (R/NW)	STATE	Run State. The <code>TRNG_RUNCNT.STATE</code> bit field provides the state of bits in the current run.
14:0 (R/NW)	TSTCNT	Run Test Count. The <code>TRNG_RUNCNT.TSTCNT</code> bit field configures the block length counter for the run and poker tests - counts up for 20,000 tested bits and then controls testing of the <code>run_X_count...</code> and <code>poker_count_X</code> counters to contain expected values, after which they - and this counter - are reset for the next block.

TRNG Run Test State and Result Registers

The `TRNG_RUN[n]` registers holds the counts for the associated run bucket for 1's and 0's.

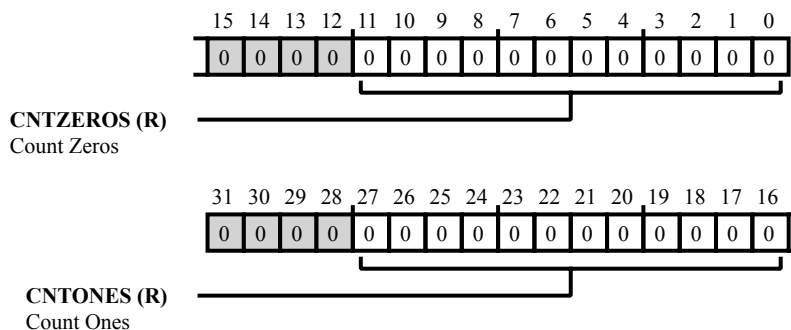


Figure 33-21: TRNG_RUN[n] Register Diagram

Table 33-24: TRNG_RUN[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
27:16 (R/NW)	CNTONES	Count Ones. In TRNG_RUN1, this counter is for single bit runs of value one bits. After 20,000 bits, this counter should have a value in the range 2267 to 2733 (inclusive) to pass the run test. This counter cannot increment past its maximum value of 4095. In TRNG_RUN2, this counter is for two bit runs of value one bits. After 20,000 bits, this counter should have a value in the range 1079 to 1421 (inclusive) to pass the run test. This counter cannot increment past its maximum value of 2047. In TRNG_RUN3, this counter is for three bit runs of value one bits. After 20,000 bits, this counter should have a value in the range 502 to 748 (inclusive) to pass the run test. This counter cannot increment past its maximum value of 1023. In TRNG_RUN4, this counter for four bit runs of value one bits. After 20,000 bits, this counter should have a value in the range 233 to 402 (inclusive) to pass the run test. This counter cannot increment past its maximum value of 511. In TRNG_RUN5, this counter is for five bit runs of value one bits. After 20,000 bits, this counter should have a value in the range 90 to 223 (inclusive) to pass the run test. This counter cannot increment past its maximum value of 255. In TRNG_RUN6, this counter for six and higher bit runs of value one bits. After 20,000 bits, this counter should have a value in the range 90 to 233 (inclusive) to pass the run test. This counter cannot increment past its maximum value of 255.

Table 33-24: TRNG_RUN[n] Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
11:0 (R/NW)	CNTZEROS	Count Zeros. In TRNG_RUN1, this counter is for single bit runs of value zero bits. After 20,000 bits, this counter should have a value in the range 2267 to 2733 (inclusive) to pass the run test. This counter cannot increment past its maximum value of 4095. In TRNG_RUN2, this counter is for two bit runs of value zero bits. After 20,000 bits, this counter should have a value in the range 1079 to 1421 (inclusive) to pass the run test. This counter cannot increment past its maximum value of 2047. In TRNG_RUN3, this counter is for three bit runs of value zero bits. After 20,000 bits, this counter should have a value in the range 502 to 748 (inclusive) to pass the run test. This counter cannot increment past its maximum value of 1023. In TRNG_RUN4, this counter is for four bit runs of value zero bits. After 20,000 bits, this counter should have a value in the range 233 to 402 (inclusive) to pass the run test. This counter cannot increment past its maximum value of 511. In TRNG_RUN5, this counter is for five bit runs of value zero bits. After 20,000 bits, this counter should have a value in the range 90 to 223 (inclusive) to pass the run test. This counter cannot increment past its maximum value of 255. In TRNG_RUN6, this counter is for six and higher bit runs of value zero bits. After 20,000 bits, this counter should have a value in the range 90 to 233 (inclusive) to pass the run test. This counter cannot increment past its maximum value of 255.

TRNG Status Register

The `TRNG_STAT` register provides status results. This register shares the same address as the Interrupt Acknowledge (`TRNG_INTACK`) register.

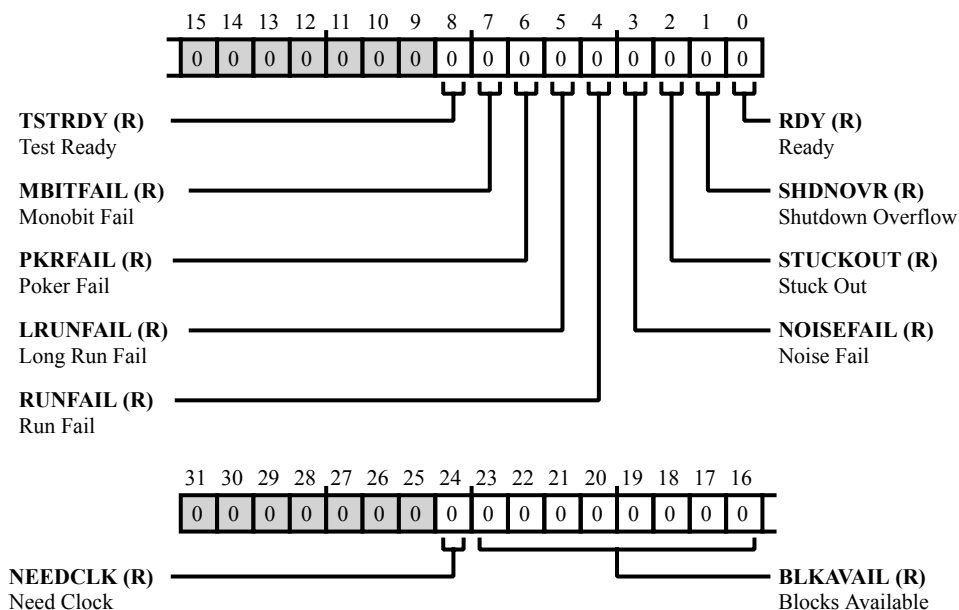


Figure 33-22: TRNG_STAT Register Diagram

Table 33-25: TRNG_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
24 (R/NW)	NEEDCLK	Need Clock. When the <code>TRNG_STAT.NEEDCLK</code> bit is set, it indicates that the TRNG is busy generating entropy or is in one of its test modes the module clock may not be turned off.
23:16 (R/NW)	BLKAVAIL	Blocks Available. This field indicates the number of 128 bits blocks of random data that are available in the random data buffer. If this value is non-zero, the output registers will be re-filled from the random data buffer immediately after acknowledging the <code>TRNG_STAT.RDY</code> by writing a '1' to <code>TRNG_INTACK.RDY</code> .
8 (R/NW)	TSTRDY	Test Ready. When the <code>TRNG_STAT.TSTRDY</code> bit is set, it indicates that data for known-answer tests on the Monobit Test, Run Test, Poker Test and post-processor functions can be written to the <code>TRNG_INPUT[n]</code> registers. When testing the post-processor, result data can be read from those same registers when this bit has become 1 again (after dropping to 0).

Table 33-25: TRNG_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
7 (R/NW)	MBITFAIL	Monobit Fail. When the <code>TRNG_STAT.MBITFAIL</code> bit is set, the Monobit Test logic monitoring data, shifted into the main LFSR, detected an out-of-bounds number of 1s after checking 20,000 bits (test T1 as specified in the AIS-31 standard).
6 (R/NW)	PKRFAIL	Poker Fail. When the <code>TRNG_STAT.PKRFAIL</code> bit is set, the Poker Test logic monitoring data shifted into the main LFSR detected an out-of-bounds value in at least one of the 16 Poker Counters or an out of bounds sum of squares value after checking 20,000 bits (test T2 as specified in the AIS-31 standard).
5 (R/NW)	LRUNFAIL	Long Run Fail. When the <code>TRNG_STAT.LRUNFAIL</code> bit is set, the Run Test logic monitoring data shifted into the main LFSR detected a sequence of 34 identical bits (test T4 as specified in the AIS-31 standard).
4 (R/NW)	RUNFAIL	Run Fail. When the <code>TRNG_STAT.RUNFAIL</code> bit is set, the Run Test logic monitoring data shifted into the main LFSR detected an out-of-bounds value for at least one of the <code>TRNG_RUN[n].CNTZEROS</code> or <code>TRNG_RUN[n].CNTONES</code> counters after checking 20,000 bits (test T3 as specified in the AIS-31 standard).
3 (R/NW)	NOISEFAIL	Noise Fail. When the <code>TRNG_STAT.NOISEFAIL</code> bit is set, the Run Test logic monitoring data shifted into the main LFSR detected a sequence of 48 identical bits, which is considered a noise source failure as proposed in section E.5 of the AIS-31 standard.
2 (R/NW)	STUCKOUT	Stuck Out. When the <code>TRNG_STAT.STUCKOUT</code> bit is set, the logic around the output data registers detected that the TRNG generates the same value twice in a row.
1 (R/NW)	SHDNOVR	Shutdown Overflow. When the <code>TRNG_STAT.SHDNOVR</code> bit is set, the number of FROs shut down after a second error event (the number of 1 bits in the TRNG_ALMSTP register) has exceeded the threshold set by the <code>TRNG_ALMCNT.SHDNTHRESH</code> bit field.
0 (R/NW)	RDY	Ready. When the <code>TRNG_STAT.RDY</code> bit is set, data is available in the <code>TRNG_OUTPUT0</code> to <code>TRNG_OUTPUT3</code> registers. If a new number is already available in the random data buffer, that number is directly moved into the result register. In this case the ready status bit is asserted again, after at most six module clock cycles.

TRNG Test Register

The `TRNG_TEST` register can be used by the host processor to perform a number of tests on the TRNG logic including:

- Register controlled characterization by connecting the `tst_fro_clk_out` output to a selected FRO clock output
- FRO logic connectivity and error event detection checking by feeding known patterns through the FRO delay line and error event detection circuits
- Direct XOR-ed FRO outputs capture by disabling the main LFSR feedback logic
- Extend the Monobit Test and Poker Test by not resetting the Monobit count and Poker Test X counters after each 20,000 bits block
- Perform known answer tests on the Run Test, Poker Test and post-processor functions.

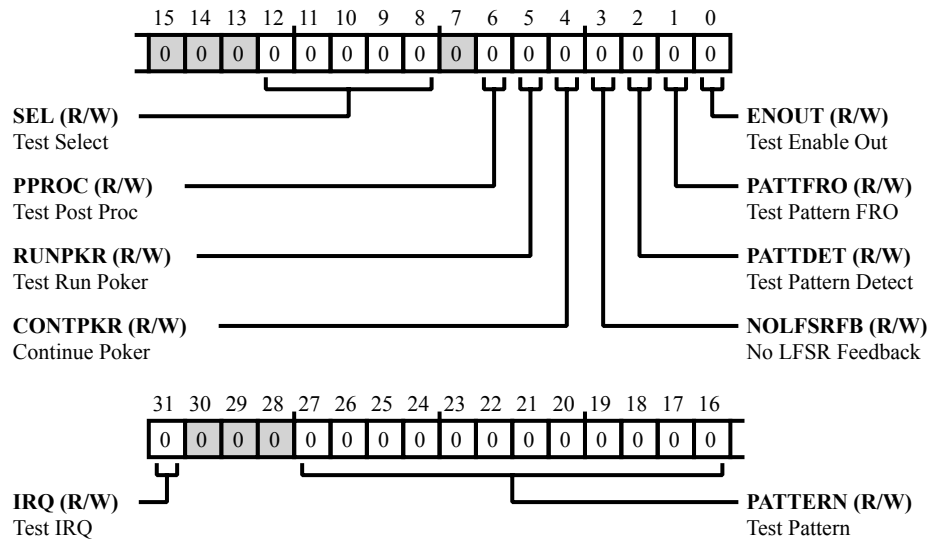


Figure 33-23: TRNG_TEST Register Diagram

Table 33-26: TRNG_TEST Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	IRQ	Test IRQ. When the <code>TRNG_TEST . IRQ</code> bit is set force irq output HIGH for interrupt signal connectivity testing.
		0 Do not force IRQ high
		1 Force IRQ output high

Table 33-26: TRNG_TEST Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
27:16 (R/W)	PATTERN	Test Pattern. The TRNG_TEST . PATTERN bit field sets up a repeating sequence of bits to be fed into the selected FRO delay chain TRNG_TEST . PATTFRO =1 and/or the selected FRO error detection circuit TRNG_TEST . PATTDET =1. This field is rotated right over one bit, once every sample period, when either of these control bits is 1. Therefore, bit [16] is the actual pattern bit fed into the test target.
12:8 (R/W)	SEL	Test Select. The TRNG_TEST . SEL bit field configures the number of the FRO to be tested, the value should be in the range of 0 to 7.
6 (R/W)	PPROC	Test Post Proc. When the TRNG_TEST . PPROC bit is set, it provides direct access to the post-processor for known-answer tests (writing input data to the TRNG_INPUT[n] registers). While this bit is set, the TRNG can continue to generate entropy in the main LFSR and any buffered random data is preserved, to be loaded into the output registers as soon as this bit is reset to 0 again. The need clock output is forced active while this bit is set. For X9.31 post-processors, it is advisable to re-seed the post-processor after running known-answer tests as the original key and V values are modified to known values. This bit is only present when post-processing is available and can only be set to 1 when the TRNG_CTL . PPROCEN bit =1 and the test run poker (TRNG_TEST . RUNPKR) bit in this register is 0.
5 (R/W)	RUNPKR	Test Run Poker. When the TRNG_TEST . RUNPKR bit is set, it provides direct access to the inputs of the Monobit, Run and Poker Test circuits (writing input data in chunks of 32 bits to the TRNG_INPUT0 register). While this bit is 1, the TRNG is not allowed to generate entropy but any buffered random data is preserved, to be loaded into the output registers as soon as this bit is reset to 0 again. The TRNG_STAT . NEEDCLK bit is forced active while this bit is 1. The Monobit, Run and Poker Test circuits are reset to their initial states on any change of this bit.
4 (R/W)	CONTPKR	Continue Poker. When the TRNG_TEST . CONTPKR bit is set, Monobit Test and Poker Test keep running continuously by not resetting the Monobit count (TRNG_MONOBITCNT) and poker counters (TRNG_POKER[n] register) at the end of each 20,000 bits test block. This bit can only be set to 1 when TRNG_CTL . TSTMODE =1.
		0 Do not continue poker test
		1 Continue poker test

Table 33-26: TRNG_TEST Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
3 (R/W)	NOLFSRFB	No LFSR Feedback. When the TRNG_TEST.NOLFSRFB bit is set, it removes XNOR feedback from the main LFSR, converting it into a normal shift register for the XOR-ed outputs of the FROs (shifting data in on the LSB side). A 1 also forces the LFSR to sample continuously. This bit can only be set to 1 when TRNG_CTL.TSTMODE = 1.
		0 Keep XNOR feedback
		1 Remove XNOR feedback
2 (R/W)	PATTDDET	Test Pattern Detect. When the TRNG_TEST.PATTDDET bit is set, it repeatedly feeds test pattern (PATTERN) into the error detection circuit of the FRO selected by the test select (TRNG_TEST.SEL) field. This bit can only be set to 1 when TRNG_CTL.TSTMODE = 1.
		0 Do not repeat feed test pattern
		1 Repeat feed test pattern
1 (R/W)	PATTFRO	Test Pattern FRO. When the TRNG_TEST.PATTFRO bit is set, it repeatedly feeds test pattern (PATTERN) into the delay chain of the FRO selected by the test select (TRNG_TEST.SEL) field by forcing the corresponding FRO enable (FROEN) output LOW. This bit can only be set to 1 when TRNG_CTL.TSTMODE = 1.
		0 Do not repeat feed test pattern
		1 Repeat feed test pattern
0 (R/W)	ENOUT	Test Enable Out. When the TRNG_TEST.ENOUT bit is set, it enables the tst_fro_clk_out output, connecting to the FRO selected by the test select (TRNG_TEST.SEL) field. This bit can only be set to 1 when TRNG_CTL.TSTMODE = 1.
		0 Disable tst_fro_clk_out
		1 Enable tst_fro_clk_out

TRNG Post-Process "V" Value Registers

The `TRNG_V[n]` registers are used to load the V value used for post-processing (if available). These registers are write-only. Reads return the values of the other registers mapped at the same addresses.

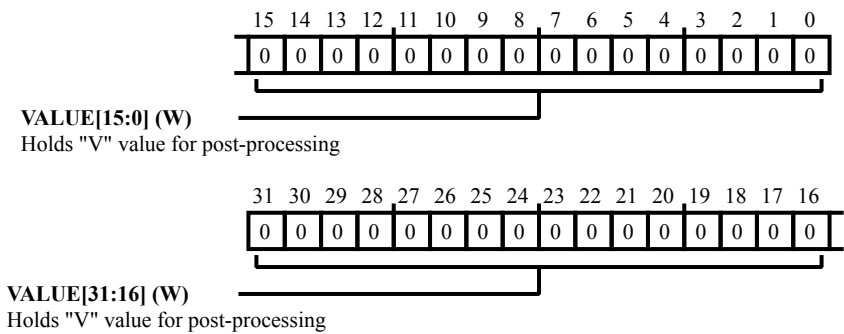


Figure 33-24: TRNG_V[n] Register Diagram

Table 33-27: TRNG_V[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (RX/W)	VALUE	Holds "V" value for post-processing. Bits of the post-processing 'V' value.

34 FIR Accelerator (FIR)

Finite Impulse Response (FIR) filters are frequently used in digital signal processing applications. The FIR accelerator is a dedicated hardware interface used to perform filter processing to reduce the instruction processing load on the core. FIR filters are used in a wide array of applications including multi-rate processing with an interpolator or decimator.

Features

This hardware module can perform FIR filters without core intervention. This allows programs to use the core to implement complex algorithms, effectively adding more bandwidth to the processor.

The FIR supports the following features:

- Fixed-point and 32-bit IEEE floating-point format
- Four SHARC+ core compatible MAC units that operate in parallel
- Rounding modes compatible with SHARC+ core MACs
- Single rate or multi-rate window processing
- Programmable rates with decimation or interpolation mode
- Up to 32 filter channels available in TDM in legacy mode
- Burst transfers on data or coefficient loads

NOTE: The FIR accelerator module has a local memory that the core cannot access during regular operation. Unlike previous SHARC processors, the FIR accelerator modules each have access to the system memory (on-chip or off-chip).

Also, unlike the previous SHARC processors, where only one of the FIR or IIR accelerators can be used at a time, the SHARC+ processor can use both accelerators simultaneously.

Clocking

The FIR accelerator runs at the speed of the core clock frequency (CCLK).

Functional Description

The *FIR Block Diagram* shows the 1024-TAP FIR hardware accelerator. The accelerator consists of a 1024 word coefficient memory, a 1024 deep delay line for data, and four MAC units. The accelerator runs at /CCLK frequency.

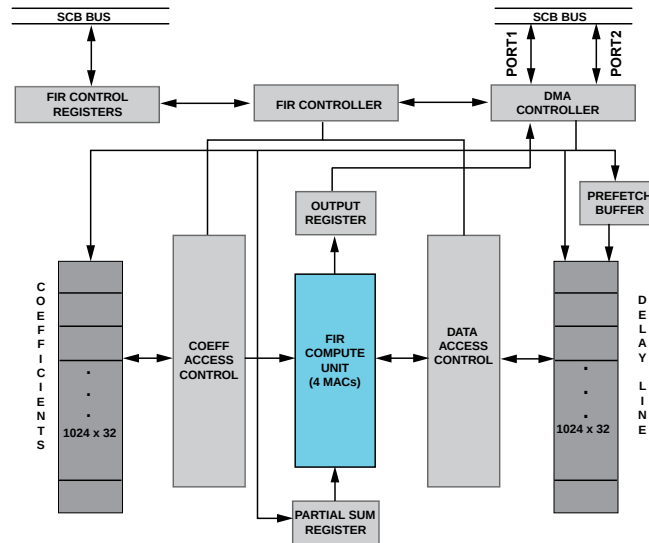


Figure 34-1: FIR Block Diagram

The FIR accelerator has the following logical sub blocks:

- A datapath unit that consists of:
 - A 1024 deep coefficient memory
 - A 1024 deep delay line for the data
 - Four 32-bit floating-point and fixed-point multiplier and adder units
 - One 32-bit prefetch buffer that operates in a pipelined fashion
 - One 32-bit buffer to hold the previous partial sum
 - One 32-bit buffer to hold the output
- Configuration registers for the number of TAPs, number of channels, filter enable, interrupt control, DMA enable, up sample or down sample control, and ratios.
- Core access interface for writing to the DMA and filter configuration registers and reading the status register.
- DMA bus interface for transferring data and coefficients to and from the accelerator.
- DMA configuration registers including chain pointer, input, output, and coefficient registers.

The accelerator block is integrated with the SHARC core. For more information about this, see the [SCB Block Diagram](#).

ADSP-21560_21561_21564_21568 FIR Register List

The FIR accelerator is a dedicated hardware interface used to perform filter processing to reduce the instruction processing load on the core.

Table 34-1: ADSP-21560_21561_21564_21568 FIR Register List

Name	Description
FIR_CHNPTR	FIR Chain Pointer Register
FIR_COEFCNT	FIR Coefficient Count Register
FIR_COEFIDX	FIR Coefficient Index Register
FIR_COEFMOD	FIR Coefficient Modifier Register
FIR_CTL1	FIR Global Control Register
FIR_CTL2	FIR Channel Control Register
FIR_DBG_ADDR	Debug Address Register
FIR_DBG_CTL	FIR Debug Control Register
FIR_DBG_RDDAT	FIR Debug Data Read Register
FIR_DBG_WRDAT	FIR Debug Data Write Register
FIR_DMASTAT	FIR DMA Status Register
FIR_INBASE	FIR Input Data Base Register
FIR_INCNT	FIR Input Data Count Register
FIR_INIDX	FIR Input Data Index Register
FIR_INMOD	FIR Input Data Modifier Register
FIR_MACSTAT	FIR MAC Status Register
FIR_OUTBASE	FIR Output Data Base Register
FIR_OUTCNT	FIR Output Data Count Register
FIR_OUTIDX	FIR Output Data Index Register
FIR_OUTMOD	FIR Output Data Modifier Register
FIR_SCTL1	Software Control Register 1
FIR_SCTL2	Software Control Register 2
FIR_SGCTL	Secondary Global Control Register

ADSP-21560_21561_21564_21568 FIR Interrupt List

Table 34-2: ADSP-21560_21561_21564_21568 FIR Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
118	FIR0_DMA	FIR0 DMA	Edge	
119	FIR0_STAT	FIR0 Status	Edge	

ADSP-21560_21561_21564_21568 FIR Trigger List

Table 34-3: ADSP-21560_21561_21564_21568 FIR Trigger List Generators

Trigger ID	Name	Description	Sensitivity
39	FIR0_DMA	FIR0 DMA	Edge
103	FIR0_TRGO	FIR0 FIR Trigger Output	Edge

Table 34-4: ADSP-21560_21561_21564_21568 FIR Trigger List Receivers

Trigger ID	Name	Description	Sensitivity
116	FIR0_TRGI	FIR0 FIR Trigger input	Pulse

Compute Block

The MAC unit, shown in the *FIR MAC Unit* figure, has four multiply accumulators. The accumulators operate simultaneously on a single filter as described below.

- The MAC unit operates on the data and coefficient fetched from the data and coefficient RAMs
- Each MAC can perform 32-bit floating-point or 32-bit fixed-point MAC operations
- Floating-point format is IEEE-compliant
- Multiply and accumulation operation (addition) are pipelined
- A 32-bit floating-point MAC operation generates 32-bit multiply results
- A 32-bit fixed-point operation generates 80-bit results (64-bit result + 16 guard bits)

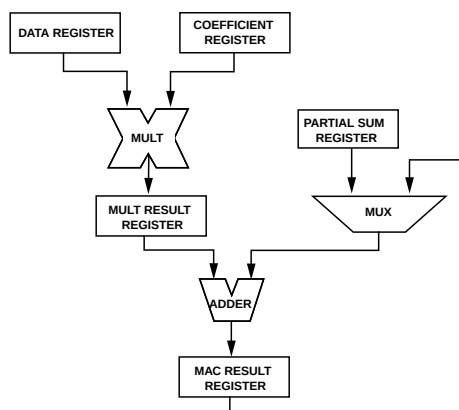


Figure 34-2: FIR MAC Unit

Partial Sum Register

The partial sum register is useful for [Floating-point Multi-Iteration](#) mode. For a particular channel, the intermediate MAC result is written to the output buffer of the system memory (on-chip or off-chip). If the same channel is requested again, the partial result register is updated with the intermediate MAC result through DMA from the output buffer of the system memory. The result is added to the current MAC result after each iteration. This process repeats until all iterations complete (the entire soft filter length is processed).

Delay Line Memory

The accelerator has a 1024 TAP delay line to hold the data locally. The DMA controller fetches the data from system memory and loads it into the delay line. Four read accesses can be made to the delay line simultaneously.

Coefficient Memory

The accelerator has a 1024 deep coefficient memory to store the coefficients. The DMA controller loads the coefficients from system memory into coefficient memory. Four coefficients can be fetched from the coefficient memory simultaneously. If the soft filter length is more than 1024, processing happens in multi-iteration mode.

Prefetch Data Buffer

The prefetch data buffer enables pipeline operation. One data sample is prefetched when the compute unit is operating on the delay line corresponding to the current sample. The data prefetched in this buffer is later used to update the delay line for the next sample. This operation happens in parallel again when the compute unit is not accessing the delay line. In other words, it happens when the compute unit is adding the output from the four MACs and the partial sum register.

Table 34-5: Pipeline Operation for Window Size = 1

Cycles	1	2	3	4	5	6
<i>Output DMA</i>			N	N1	N2	N3
<i>Compute</i>		N	N1	N2	N3	

Table 34-5: Pipeline Operation for Window Size = 1 (Continued)

Cycles	1	2	3	4	5	6
<i>Input DMA</i>	N	prefetch N1	prefetch N2	prefetch N3		

Processing Output

The accelerator uses all four MACs simultaneously to calculate one output sample as shown in the *Multi-Iteration Filtering Flow* figure and the following procedure.

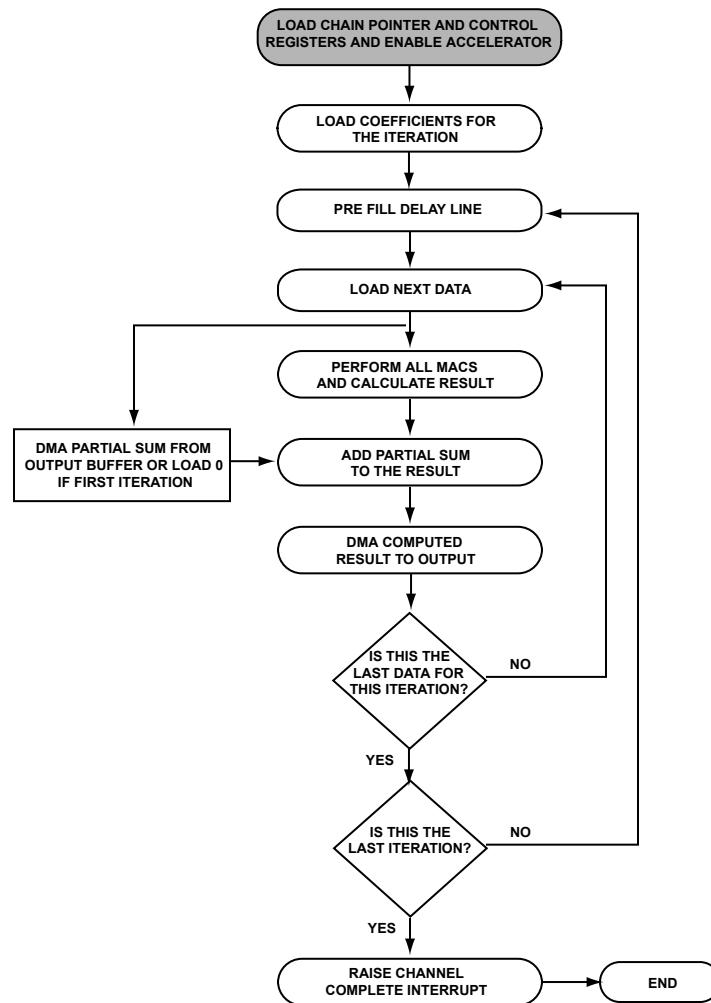


Figure 34-3: Multi-Iteration Filtering Flow

1. The accelerator fetches four input data from the delay line and four corresponding coefficients from the coefficient memory and feeds them to the MAC units for multiply and accumulation operations.
2. The accelerator repeats the procedure with the next four input data and coefficients until all the TAPs complete. For example, this procedure happens $N/4$ times for an N TAP filter.

3. When all the TAPs are complete, the accelerator adds the four MAC outputs together to the previous partial sum (if any) to calculate the final result.
4. Finally, that output sample is stored back in system memory.

System Memory Storage

The following sections describe the storage format for the accelerator.

CAUTION: Store all data in memory aligned to the word address boundaries. Any other programmed addresses do not flag an error.

Coefficients and Input Buffer Storage

For any N TAP filter with coefficients:

```
C[i] i = 0,1,
...
N - 1
```

store the coefficients in system memory buffer in the order:

```
C[N - 1], C[N - 2]
...
C[1], C[0]
```

and CI should point to $C(N - 1)$.

Single Rate Input Filtering

The total size of the input buffer must at least be equal to $N - 1 + W$. If the input buffer that needs to be processed is:

```
x[n], x[n+1], x[n+2]
...
x[n+W-1]
```

store it in the memory as

```
x[n - (N-1)], x[n - (N-2)]
...
x[n-1], x[n], x[n+1]
...
x[n+W-1]
```

and the [FIR_INIDX](#) register value should point to $x[n - (N - 1)]$

Decimation

Assuming M = decimation ratio, the total size of the input buffer should at least be equal to $N - 1 + W \times M$. If the input buffer that needs to be processed is:

```
x[n], x[n+1], x[n+2] ... x[n+W×M-1],
```

store it in the memory as:

```
x[n-(N-1)], x[n-(N-2)]...x[n-1],
x[n], x[n+1]...x[n+WxM-1]
```

and the `FIR_INIDX` register value should point to `x[n-(N-1)]`.

Interpolation

Assuming L = interpolation ratio, the total size of the input buffer should be at least equal to:

$\text{Ceil}((N-1)/L) + W/L$.

If the input buffer that needs to be processed is:

```
x[n], x[n+1], x[n+2]...x[n+W/L-1],
and K = Ceil((N-1)/L)
```

store it in the memory as:

```
x[n-k], x[n-(K-1)], x[n-(K-2)]...x[n-1],
x[n], x[n+1]...x[n+W/L-1]
```

and the `FIR_INIDX` register value should point to `x[n-K]`.

Operating Modes

The FIR core performs a sum-of-products operation to compute the convolution sum. It supports single-rate, decimation, and interpolation functions.

Single Rate Processing

In a single-rate filter, the output result rate is equal to the input sample rate. The filter output $Y(n)$ is computed according to following equation where N is the number of filter coefficients: $c(k)$ $k = 0$ to $N - 1$ are the filter coefficients and $x(n)$ represents the input time-series.

$$Y(n) = \sum_{k=0}^{N-1} c(k) \cdot x(n-k)$$

Figure 34-4: Filter Output Calculation

Single Iteration

Results are computed in a single iteration when the soft filter length is less than or equal to 1024.

Floating-point Multi-Iteration

Results are computed in multiple iterations when the soft filter length is greater than 1024 (for example, 2048 TAPs on a 1024 hard filter length). In this mode, the accelerator implements two iterations of 1024 TAPs.

NOTE: If the soft filter length is not a multiple of the hard filter length, the accelerator iterates until the soft filter length is satisfied.

Example: 550 taps on a 256 tap filter. In this example, the FIR accelerator implements two iterations of 256 taps and one iteration of 38 taps.

NOTE: Multi-iteration mode is not supported in fixed-point format.

Window Processing

In window-based mode, multiple output samples (up to 1024) equal to the window size of that channel are calculated. After these calculations are complete, the accelerator begins processing the next channel. A configurable window size parameter is provided to specify the length of the window.

To select sample-based processing mode, configure the window size to 1. In this mode, one sample from a particular channel is processed through all the taps of that channel and the final output sample is calculated.

Multi-Rate Processing

Multi-rate filters change the sampling rate of a signal—they convert the input samples of a signal to a different set of data that represents the same signal sampled at a different rate.

Decimation

A decimation filter provides a single output result for every M input samples, where M is the decimation ratio. The output rate is $1/M$ 'th of the input rate. The filter implementation exploits the low output sample rate by not starting a computation until a new set of M input samples is available.

In this mode, after low-pass filtering (for anti-aliasing), FIR logic discards the ratio – 1 samples of output data. For performance optimization, FIR logic skips the computation of output samples, which are discarded.

The input buffer size for decimation filters is $N - 1 + (W \times M)$ where:

- N is the number of taps
- W is the window size
- M is the decimation ratio

The window size (`FIR_CTL2.WINDOW` bits) must be programmed with the number of output samples.

To start this mode, programs set the `FIR_CTL2.RATIO` and `FIR_CTL2.UPSAMP` bits (along with normal filter setting). Also, the `FIR_CTL2.TAPLEN` bit field value should be greater than or equal to the `FIR_CTL2.RATIO` bit field value for the decimation filter.

Interpolation

An interpolation filter provides L output results for each new input sample, where L is the interpolation ratio. The output rate is L times the input rate.

In this mode, according to the ratio specified in configuration register, FIR logic inserts $L - 1$ zeros between any two input samples (upsampling). It then performs the interpolation (through the FIR filter).

Both upsampling and downsampling do not support multi-iteration mode. Therefore, the filtering operation only happens on up to 1024 TAPs and the ratio of up and downsampling can only be an integer value.

In an interpolation filter, FIR logic inserts $L - 1$ zeros between each sample. The program must ensure that these zeros fully shift out of the delay line before moving on to the next channel. This operation puts a restriction on window size in terms of L – *the sample ratio* as showing in the expression:

$WINDOWSIZE = n \times SAMPLERATIO$, where n is the number of input samples.

The input buffer size is smallest integer greater than or equal to $(N - 1 + W) L$ for interpolation filters where:

- N is the number of taps
- W is the window size
- L is the interpolation ratio

To start the mode, programs configure the `FIR_CTL2.RATIO` and `FIR_CTL2.UPSAMP` bits (along with filter settings).

Floating-Point Data Format

The FIR accelerator treats data and coefficients in 32-bit floating-point format as the default functional mode.

Fixed-Point Data Format

In fixed-point mode, the 32-bit input data or coefficient is treated as fixed-point. A 32-bit fixed-point MAC operation generates an 80-bit result. Fixed-point data or coefficients can be unsigned integer, unsigned fractional and signed integer.

NOTE: In fixed-point mode, the entire 80-bit result register is always written back in bursts of 3×32 bits. The first word is the LSW, the second word is the MSW, and the third word is a 16-bit overflow. The remaining 16 bits are padded with zeros. Therefore, for fixed-point mode: $WINDOWSIZE = WINDOWSIZE \times 3$.

If the signed fractional format is used, the output must be scaled by 2. The MAC does not right shift to remove the redundant sign bit. A final routine must decimate the output buffer to the desired samples.

Multi-iteration mode is not supported in this format. Therefore, the maximum TAP length is 1024.

Auto Configuration Mode (ACM)

The accelerator can be operated in legacy mode or Auto Configuration Mode (ACM), controlled by the `FIR_CTL1.ACM` bit. The default functional mode is legacy mode. The accelerator mode can only be changed when the accelerator is disabled.

The ACM provides the following additional features:

- Halt and queuing

The core may pause the current Transfer Control Block (TCB) chain being processed by setting the `FIR_CTL1.HALT` bit. The accelerator acknowledges the core by setting the `FIR_DMASTAT.HALT_STAT` bit. The core can appropriately take action to submit/insert new TCB's. After the core acts, the accelerator processing can be resumed by clearing the `FIR_CTL1.HALT` bit. Before halting the accelerator, if the initial TCB chain is processed, the accelerator comes to the idle state. In this case, the accelerator has to be disabled and enabled by toggling the `FIR_CTL1.EN` bit and the `FIR_CTL1.HALT` bit must be cleared to resume processing.

- No channel number limitation

Unlike legacy mode, there is no fixed channel number limitation and, therefore, the accelerator ignores the value programmed in the `FIR_CTL1.CH` field. The application can queue an unlimited number of channels/TCBs dynamically and the accelerator keeps processing the TCBs until the chain pointer becomes null.

- Selective interrupt

The core can enable/mask interrupt generation for each channel using the `FIR_CTL2.IMASK` bit. If the bit is cleared, an interrupt is generated after completion of the channel.

- Selective controller / target trigger

The core can enable/mask trigger generation by the accelerator after the end of processing of each channel using the `FIR_CTL2.TMASK` bit. The accelerator can also wait for a trigger after loading the TCB and coefficients and before processing a channel for which the `FIR_CTL2.TWAIT` bit is set.

In addition to the above features, there are three additional fields as part of the TCB `FIR_SCTL1`, `FIR_SCTL2`, and `FIR_SGCTL` registers. The `FIR_SGCTL` register can be used to change the `FIR_CTL1` parameters such as rounding mode and fixed-point mode for each channel. The `FIR_SCTL1` and `FIR_SCTL2` registers can be used as general-purpose registers.

Data Transfer

The FIR filter works exclusively through DMA.

Chain Assignment

The structure of a TCB is conceptually the same as a traditional linked list. Each TCB has several data values and a pointer to the next TCB. Further, the chain pointer of a TCB can point to itself to continuously re-run the same DMA. The FIR accelerator reads each word of the TCB and loads it into the corresponding register. The end of the chain (no further TCBs are loaded) is indicated by a TCB with a chain pointer register value of zero.

The FIR accelerator DMA supports circular buffer chained DMA. The FIR accelerator does not support circular buffering for the coefficient buffer.

Table 34-6: TCBs for Chained DMA in Legacy Mode

Address	Register
TCB	FIR_CHNPTR
TCB + 0x1	FIR_COEFCNT
TCB + 0x2	FIR_COEFMOD
TCB + 0x3	FIR_COEFIDX
TCB + 0x4	FIR_OUTBASE
TCB + 0x5	FIR_OUTCNT
TCB + 0x6	FIR_OUTMOD
TCB + 0x7	FIR_OUTIDX
TCB + 0x8	FIR_INBASE
TCB + 0x9	FIR_INCNT
TCB + 0xA	FIR_INMOD
TCB + 0xB	FIR_INIDX
TCB + 0xC	FIR_CTL2

Table 34-7: TCBs for Chained DMA in Auto Configuration Mode

Address	Register
TCB	FIR_CHNPTR
TCB + 0x1	FIR_SCTL1
TCB + 0x2	FIR_SCTL2
TCB + 0x3	FIR_SGCTL
TCB + 0x4	FIR_COEFCNT
TCB + 0x5	FIR_COEFMOD
TCB + 0x6	FIR_COEFIDX
TCB + 0x7	FIR_OUTBASE
TCB + 0x8	FIR_OUTCNT
TCB + 0x9	FIR_OUTMOD
TCB + 0xA	FIR_OUTIDX
TCB + 0xB	FIR_INBASE
TCB + 0xC	FIR_INCNT
TCB + 0xD	FIR_INMOD
TCB + 0xE	FIR_INIDX
TCB + 0xF	FIR_CTL2

The `FIR_COEFCNT` register is loaded with the values in the `FIR_COEFCNT` TCB field and is decremented from that value onwards. However, coefficient loading continues until the number of coefficients, equal to the tap length, are read. This condition is true even if the `FIR_COEFCNT` register reaches zero as in the case of a tap length = 10, and the `FIR_COEFCNT` bit field in the TCB is initialized to 0. The value in the `FIR_COEFCNT` register is -10 after all coefficients are loaded.

NOTE: Initialize the `FIR_CHNPTR` register to TCB + 12 in legacy mode and TCB + 15 in ACM mode.

DMA Access

The FIR accelerator has two DMA channels (accelerator input and output) to connect to the system memory. The DMA controller fetches the data and coefficients from memory and stores the result.

Burst Access Support

Burst access enhances the throughput of the DMA channel and reduces the overall load on the system fabric. Burst support is provided for data and coefficient loads on the DMA channel. The FIR module supports burst transfers of size SINGLE, INCR4 and INCR8.

An additional bit is provided in the `FIR_CTL1` register for enabling the burst feature. Also, burst transfers are always of size SINGLE when the modifier is other than 1. In case of burst transfers around the circular buffer boundary, the design ensures that the burst does not cross the buffer boundary.

Accelerator TCB

The location of the DMA parameters for the next sequence comes from the chain pointer register that points to the next set of DMA parameters stored in the internal memory of the processor. In chained DMA operations, the accelerator automatically initializes and starts another DMA transfer when the current DMA transfer is complete. Each new set of parameters is stored in a user-initialized memory buffer or TCB for a chosen peripheral.

Chain Pointer DMA

The DMA controller supports circular buffer chain pointer DMA. One TCB must be configured for each channel. The DMA controller contains the following:

- A control register value to configure the filter parameters (such as filter tap length, window size, sample rate conversion settings) for each channel. In ACM mode, additional parameters such as interrupt mask, trigger mask, and trigger wait are also available.
- Software control register values in ACM mode for each channel.
- Secondary control register value to configure rounding mode, fixed-point mode, and two's complement for each channel.
- DMA parameter register values for the input data (delay line).
- DMA parameter register values for coefficient load.
- DMA parameter register values for output data.

Intermediate results in multi-iteration mode are saved in the output buffer.

As shown in the *Circular Buffer Addressing* figure, the accelerator loads the TCB into its internal registers and uses these values to fetch coefficients and data and to store results. After processing a window of data for any channel, the accelerator writes back the appropriate values to the `FIR_INIDX` and `FIR_OUTIDX` bit fields of the TCB in memory. Then, data processing can begin from where it left off during the next time slot of that channel.

The write-back value for input buffer is:

- `FIR_INIDX + W` for single rate filtering
- `FIR_INIDX + W × M` for decimation (M = decimation ratio)
- `FIR_INIDX + W/L` for interpolation (L = interpolation ratio)
- The write-back value for output buffer in floating point mode is: `FIR_OUTIDX + W`
- The write-back value for output buffer in fixed-point mode is: `FIR_OUTIDX + 3 × W`

NOTE: The `FIR_CTL2` register is part of the FIR TCB. This configuration allows programming individual FIR channels with different control attributes.

The above index updates are valid only for legacy mode. In ACM mode, the `FIR_INIDX` and `FIR_OUTIDX` bit fields of the TCB in memory are updated to `0x00000000` and `0xFFFFFFFF` by the accelerator after processing a window of data.

In ACM mode, when the `FIR_CTL1.SMQ_LIUPS_EN` bit is set, the accelerator updates the `FIR_INIDX` and `FIR_OUTIDX` bit fields of the TCB in memory after processing a window of data and according to the circular buffer scheme. When the `FIR_CTL1.SMQ_LIUPS_EN` bit is cleared, the accelerator updates the `FIR_INIDX` and `FIR_OUTIDX` bit fields of the TCB in memory to `0x00000000` and `0xFFFFFFFF` after processing a window of data. The `FIR_CTL1.SMQ_LIUPS_EN` bit is only valid in ACM mode.

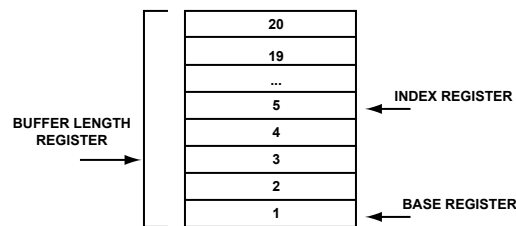


Figure 34-5: Circular Buffer Addressing

Programming Model

The following sections provide general programming information for the FIR accelerator.

Legacy Mode

The *Wait for Core Intervention $\geq$ Idle (if auto channel iterate bit = 0)* figure shows the diagram for multichannel filtering. Multiple channels are processed in a time division multiplexed (TDM) format. After completing all the channels, the accelerator can either repeat the slots or wait for core intervention.

For multichannel filtering, use the following steps.

1. Program the number of channels using the `FIR_CTL1.CH` bits.
2. Configure the TCBs in system memory with one channel's TCB pointing to the next channel's TCB.
3. Write the first TCB value into the `FIR_CHNPTR` register and enable the accelerator.
 - a. The accelerator fetches the first channel's TCB and, using it as pointer, prefills the delay line and coefficient memory and loads the `FIR_CTL2` register to configure the filter parameters corresponding to that channel.
 - b. The accelerator then calculates output samples corresponding to one window and stores the data back in internal memory.
 - c. At the end of the window the accelerator updates the `FIR_INIDX` and `FIR_OUTIDX` registers in the TCB of system memory and moves to the next channel.
 - d. When all the channels are finished and the auto channel iterate bit (`FIR_CTL1.CAI`) = 1, the accelerator processes the first channel again and iterates through the channels. If the `FIR_CTL1.CAI` bit = 0, the accelerator waits for core intervention.

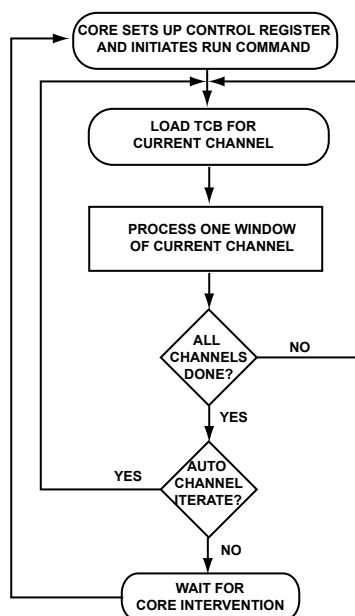


Figure 34-6: Wait for Core Intervention $\geq$ Idle (if auto channel iterate bit = 0)

NOTE: All the addresses programmed in the TCB correspond to 32-bit address boundaries and should not contain the lower 2 bits (assumed as zeros).

Auto Configuration Mode (ACM)

The figure shows multi-channel filtering in ACM. Multiple channels are processed in a time multiplexed format (TDM).

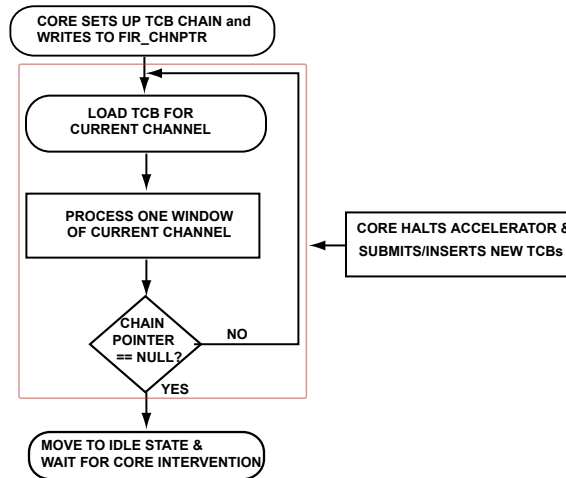


Figure 34-7: Multi-Channel Filtering in Auto Configuration Mode

For multi-channel filtering:

1. Configure the Transfer Control Blocks (TCBs) in system memory with one channel's TCB pointing to the next channel's TCB. There is no limit on the number of channels to be configured.
2. Write the first TCB value to the `FIR_CHNPTR` register and enable the accelerator.

The accelerator fetches the first channel's TCB. Using it as pointer, it prefills the delay line and coefficient memory and loads the `FIR_CTL2` and `FIR_SGCTL` registers to configure the filter parameters corresponding to that channel.

When the `FIR_CTL2.TWAIT` bit is set, the accelerator waits for a input trigger to start the window processing for the channel.

The accelerator then calculates output samples corresponding to one window and stores the data back in internal memory. When the `FIR_CTL2.TMASK` bit is cleared the accelerator sends an output trigger after completion of processing of the channel.

When the `FIR_CTL2.IMASK` bit is cleared, the accelerator interrupts the core after completion of processing of a particular channel.

At the end of the window, the accelerator updates the `FIR_INIDX` and `FIR_OUTIDX` registers to `0x00000000` and `0xFFFFFFFF` in the TCB of system memory and moves to the next channel.

3. At any instant, as required, the core halts the accelerator. It sets the `FIR_CTL1.HALT` bit and appropriately takes action to submit or insert new TCBs and clears the `FIR_CTL1.HALT` bit to resume channel processing.

If the `FIR_CHNPTR` register is zero (last channel is being processed or channel processing is complete) after halting the accelerator, the `FIR_CHNPTR` register is appropriately written before resuming the accelerator channel processing.

When the accelerator is idle after halting, `FIR_CTL1.EN` bit is toggled to disable and re-enable the accelerator and the `FIR_CTL1.HALT` bit is cleared, and the accelerator resumes channel processing.

4. The accelerator continues processing until all the channels are complete. Repeat *Step 3* (if required) to submit/insert new channels.

NOTE: Channel auto iterate (`FIR_CTL1.CAI`) is not supported in ACM.

Debug Mode

The next sections show the steps required for reading and writing local memory in debug mode.

Write to Local Memory

1. Clear the `FIR_CTL1.DMAEN` bit.
2. Set the `FIR_DBG_CTL.EN`, `FIR_DBG_CTL.MEM`, and `FIR_DBG_CTL.HLD` bits.
3. Set the `FIR_DBG_CTL.ADRINC` bit for address auto increment.
4. Write the start address to the `FIR_DBG_ADDR` register.

NOTE: If bit 11 in the `FIR_DBG_ADDR` register is set, coefficient memory is selected.

5. Write data to the `FIR_DBG_WRDAT` register.

Read from Local Memory

1. Clear the `FIR_CTL1.DMAEN` bit.
2. Set the `FIR_DBG_CTL.EN`, `FIR_DBG_CTL.MEM`, and `FIR_DBG_CTL.HLD` bits.
3. Set the `FIR_DBG_CTL.ADRINC` bit for address auto increment.
4. Write the start address to the `FIR_DBG_ADDR` register.

NOTE: If bit 11 in the `FIR_DBG_ADDR` register is set, coefficient memory is selected.

5. Read data from the `FIR_DBG_RDDAT` register.

Single-Step Mode

Single-step mode can be used for debug purposes. An extra debug register is used in this mode.

1. Enable stop DMA during breakpoint hit in the emulator settings.
2. Clear the `FIR_DBG_CTL.HLD` bit and enable the `FIR_DBG_CTL.EN` and `FIR_DBG_CTL.RUN` bits.
3. Program the FIR module according to the application.

4. In single-step mode, each iteration is updated in the emulator session.

Computing FIR Output, Tap Length Greater than 4096

With little core intervention, the FIR accelerator can also be used to calculate output for a tap length greater than 4096 taps. The section demonstrates the calculation with an example of 8192 taps.

1. Divide the transfer function of an 8192 FIR filter into two 4096 FIR filters:

$$\begin{aligned}
 H(Z) &= b_0 + b_1Z^{-1} + b_2Z^{-2} + \dots b_{4095}Z^{-4095} + b_{4096}Z^{-4096} + b_{4097}Z^{-4097} + \dots b_{8191}Z^{-8191} \\
 &= b_0 + b_1Z^{-1} + b_2Z^{-2} + \dots b_{4095}Z^{-4095} + Z^{-4096}(b_{4096} + b_{4097}Z^{-1} + \dots b_{8191}Z^{-4096})
 \end{aligned}$$

2. Divide the filter coefficients of an 8192 tap filter among two 4096 tap FIR filters:

Filter 1

Coefficients = $b_0, b_1, b_2, \dots, b_{4095}$

Input data = $x[n], x[n-1], \dots, x[n-4095]$

Filter 2

Coefficients = $b_{4096}, b_{4097}, \dots, b_{8191}$

Input data = $x[n-4096], x[n-4097], \dots, x[n-8191]$

The accelerator can be used in two-channel mode where:

- channel 1 operates on $x[n] \dots x[n-4095]$ input data with the filter coefficients of filter 1 and
- channel 2 operates on $x[n-4096] \dots x[n-8191]$ with the filter coefficients of filter 2.

Once both the channels are processed, add the partial sum output of both the channels to get the final output. Implement this approach (tap length = TAPS = 8192, window size = WINDOW) using the following programming steps.

1. Create a circular input data buffer in system memory (IBUF). The buffer must be large enough to avoid overwriting data before the accelerator processes it. Ideally, the input buffer size for a channel is TAPS + WINDOW - 1.
2. Create a coefficient buffer of size TAPS (8192) (CBUF).
3. Create one output buffer of size WINDOW (OBUF) and another temporary output buffer (OBUF1) to store the partial sum.
4. Create two TCBs in system memory with first TCB chained to the second and second chained to the first in a circular manner.
 - a. Point the `FIR_COEFIDX` bit field of the first TCB to the start address of the coefficient buffer (CBUF) and that of the second TCB to 4096 offset from the start of the coefficient buffer (CBUF + 4096).

- b. Point the `FIR_OUTBASE` and `FIR_OUTIDX` bit fields of the first TCB to the start address of OBUF and that of the second TCB to the start address of OBUF1.
 - c. Point the `FIR_INIDX` bit field of the first TCB to the start address of IBUF and that of the second TCB to 4096 offset from the start address of IBUF.
 - d. Configure the `FIR_CTL2` bit field of both the TCB for tap length = $TAP/2 = 4096$ and window size = WINDOW.
5. Initialize the `FIR_CHNPTR` register to point to the first TCB.
 6. Program the `FIR_CTL1` register to initiate the accelerator processing by setting the `FIR_CTL1.EN` and `FIR_CTL1.DMAEN` bits and the number of channels configured as 2.
 7. Wait for the FIR all channels done interrupt (`FIR_DMASTAT.ACDONE`) to occur. Inside the ISR, add the partial sum results using the core from both the output buffers (OBUF and OBUF1) to get the final output. To save memory, replace the contents of the buffer OBUF with the final output result.

Debug Features

The following sections provide information for debugging the FIR accelerator.

Local Memory Access

The contents of FIR delay line and coefficient memories are made observable for debug by setting the `FIR_DBG_CTL.EN/FIR_DBG_CTL.MEM` and `FIR_DBG_CTL.HLD` bits. The debug address register (`FIR_DBG_ADDR`) and two data registers are provided for debug operations. Bit 11 of the `FIR_DBG_ADDR` register selects coefficient memory when set (=1) and selects delay line memory when cleared (=0).

In the debug mode, the read data register (`FIR_DBG_RDDAT`) returns the contents of the memory location pointed to by the address register. Data can be written into any memory location using `FIR_DBG_WRDAT` register writes. If the address auto-increment bit (`FIR_DBG_CTL.ADRINC`) is set, the address register auto-increments on `FIR_DBG_WRDAT` writes and `FIR_DBG_RDDAT` reads. During auto-increment the `FIR_DBG_ADDR` register cannot cross the data memory or coefficient memory boundary.

Single-Step Mode

Programs can single step through the MAC operations and observe the memory contents after each step. The `FIR_DBG_CTL.EN`, `FIR_DBG_CTL.HLD`, and `FIR_DBG_CTL.MEM` bits control the FIR MAC units.

Emulation Considerations

In FIR debug mode, the DMA operations are not observable.

Interrupts

The *FIR Interrupt Overview* table provides the source of interrupt and service instructions for the FIR interrupts.

Table 34-8: FIR Interrupt Overview

Accelerator Mode	Default Programmable Interrupt	Sources	Masking	Service
Legacy Mode	FIR_DMA	Window Complete	N/A	ROC from FIR_DMASSTAT + RTI instruction
		All channels complete		
	FIR_STAT	MAC IEEE floating point exceptions	N/A	ROC from FIR_MACSTAT + RTI instruction
		MAC fixed point over-flow		
Auto Configuration Mode	FIR_DMA	Window Complete	FIR_CTL2 . IMASK	ROC from FIR_DMASSTAT + RTI instruction
	FIR_STAT	MAC IEEE floating point exceptions	N/A	ROC from FIR_MACSTAT + RTI instruction
		MAC fixed point over-flow	N/A	

Sources

The FIR module drives two interrupt signals: `FIR_DMA` for the DMA status and `FIR_STAT` for the MAC status. The FIR module generates interrupts as described in the following sections.

Window Complete

This interrupt is generated at the end of each channel when all the output samples are calculated corresponding to a window and updated index values are written back.

In legacy mode, if the `FIR_CTL1 . CCINTR` bit is set, an interrupt is generated after completion of window processing of each channel. In ACM, the interrupt generation can be selectively masked using the `FIR_CTL2 . IMASK` bit for each channel.

All Channels Complete

This interrupt is generated when all the channels are complete or when one iteration of time slots completes. Note that the interrupt follows the access completion rule, where the interrupt is generated when all data are written back to system memory.

The all channels complete interrupt source is only applicable in legacy mode. If the `FIR_CTL1 . CCINTR` bit is not set in legacy mode, the all channel complete interrupt is generated. In ACM mode, interrupt generation for each channel is controlled using the `FIR_CTL2 . IMASK` bit.

NOTE: The `FIR_CTL1 . CCINTR` bit is valid only in legacy mode. In ACM mode, interrupt generation for each channel is controlled using the `FIR_CTL2 . IMASK` bit.

MAC Status

A MAC status interrupt is generated under the following conditions and is reflected in the [FIR_MACSTAT](#) register.

- Multiplier result zero – Set if multiplier result is zero
- Multiplier result infinity – Set if multiplier result is infinity
- Multiply invalid – Set if multiply operation is invalid
- Adder result zero – Set if adder result is zero
- Adder result infinity – Set if adder result is infinity
- Adder invalid – Set if addition is invalid
- Adder overflow – for fixed-point operation

Service

The DMA interrupt status bits are sticky and are cleared when the DMA status register is read. When a MAC status interrupt occurs, programs can find this state by reading the MAC status register ([FIR_MACSTAT](#)). The read of the register clears the (sticky) MAC interrupt status bits.

The status interrupt sources are derived from the [FIR_MACSTAT](#) register. A status interrupt can occur due to the last set of MAC operations of a processing iteration that correspond to a particular channel. The interrupt is generated continuously and cannot be stopped, even after disabling the accelerator. The interrupt can only be stopped when another processing iteration results in a non-zero or valid multiply or add result.

ADSP-21560_21561_21564_21568 FIR Register Descriptions

The FIR accelerator (FIR) contains the following registers.

Table 34-9: ADSP-21560_21561_21564_21568 FIR Register List

Name	Description
FIR_CHNPTR	FIR Chain Pointer Register
FIR_COEFCNT	FIR Coefficient Count Register
FIR_COEFIDX	FIR Coefficient Index Register
FIR_COEFMOD	FIR Coefficient Modifier Register
FIR_CTL1	FIR Global Control Register
FIR_CTL2	FIR Channel Control Register
FIR_DBG_ADDR	Debug Address Register
FIR_DBG_CTL	FIR Debug Control Register
FIR_DBG_RDDAT	FIR Debug Data Read Register

Table 34-9: ADSP-21560_21561_21564_21568 FIR Register List (Continued)

Name	Description
FIR_DBG_WRDAT	FIR Debug Data Write Register
FIR_DMASTAT	FIR DMA Status Register
FIR_INBASE	FIR Input Data Base Register
FIR_INCNT	FIR Input Data Count Register
FIR_INIDX	FIR Input Data Index Register
FIR_INMOD	FIR Input Data Modifier Register
FIR_MACSTAT	FIR MAC Status Register
FIR_OUTBASE	FIR Output Data Base Register
FIR_OUTCNT	FIR Output Data Count Register
FIR_OUTIDX	FIR Output Data Index Register
FIR_OUTMOD	FIR Output Data Modifier Register
FIR_SCTL1	Software Control Register 1
FIR_SCTL2	Software Control Register 2
FIR_SGCTL	Secondary Global Control Register

FIR Chain Pointer Register

The `FIR_CHNPTR` register contains the chain pointer address.

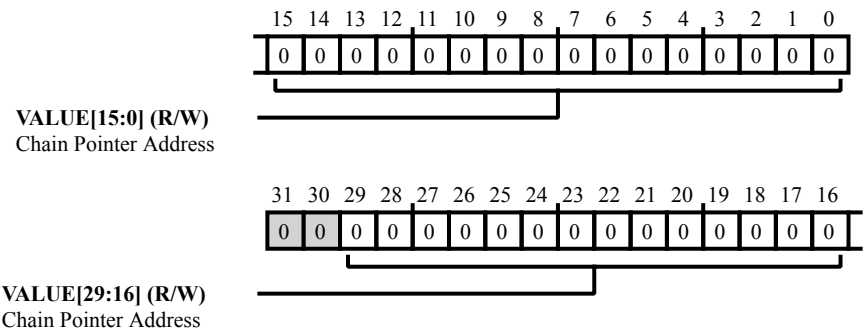


Figure 34-8: FIR_CHNPTR Register Diagram

Table 34-10: FIR_CHNPTR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:0 (R/W)	VALUE	Chain Pointer Address. The <code>FIR_CHNPTR.VALUE</code> bit field contains the chain pointer address.

FIR Coefficient Count Register

The `FIR_COEFCNT` register contains the 16-bit coefficient buffer count.

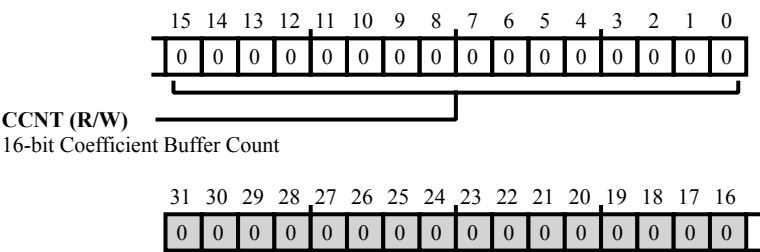


Figure 34-9: FIR_COEFCNT Register Diagram

Table 34-11: FIR_COEFCNT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	CCNT	16-bit Coefficient Buffer Count. The <code>FIR_COEFCNT.CCNT</code> bit field contains the 16-bit coefficient buffer count.

FIR Coefficient Index Register

The `FIR_COEFIDX` register contains the coefficient index word address with the lower two bits removed.

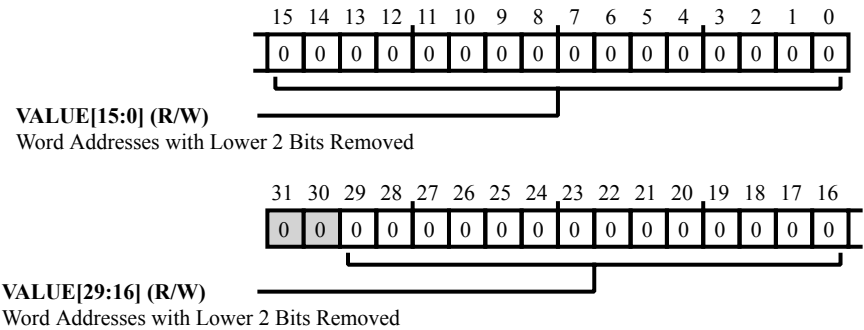


Figure 34-10: FIR_COEFIDX Register Diagram

Table 34-12: FIR_COEFIDX Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:0 (R/W)	VALUE	Word Addresses with Lower 2 Bits Removed. The <code>FIR_COEFIDX.VALUE</code> bit field contains the word addresses with the lower 2 bits removed.

FIR Coefficient Modifier Register

The `FIR_COEFMOD` register contains the 16-bit coefficient index modifier.

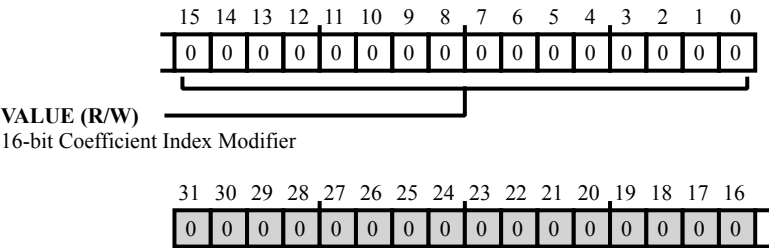


Figure 34-11: FIR_COEFMOD Register Diagram

Table 34-13: FIR_COEFMOD Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	VALUE	16-bit Coefficient Index Modifier. The <code>FIR_COEFMOD.VALUE</code> bit field contains the 16-bit coefficient index modifier.

FIR Global Control Register

The `FIR_CTL1` register is used to configure the global parameters for the accelerator. These parameters include the number of channels, channel auto iterate, DMA enable, and accelerator enable.

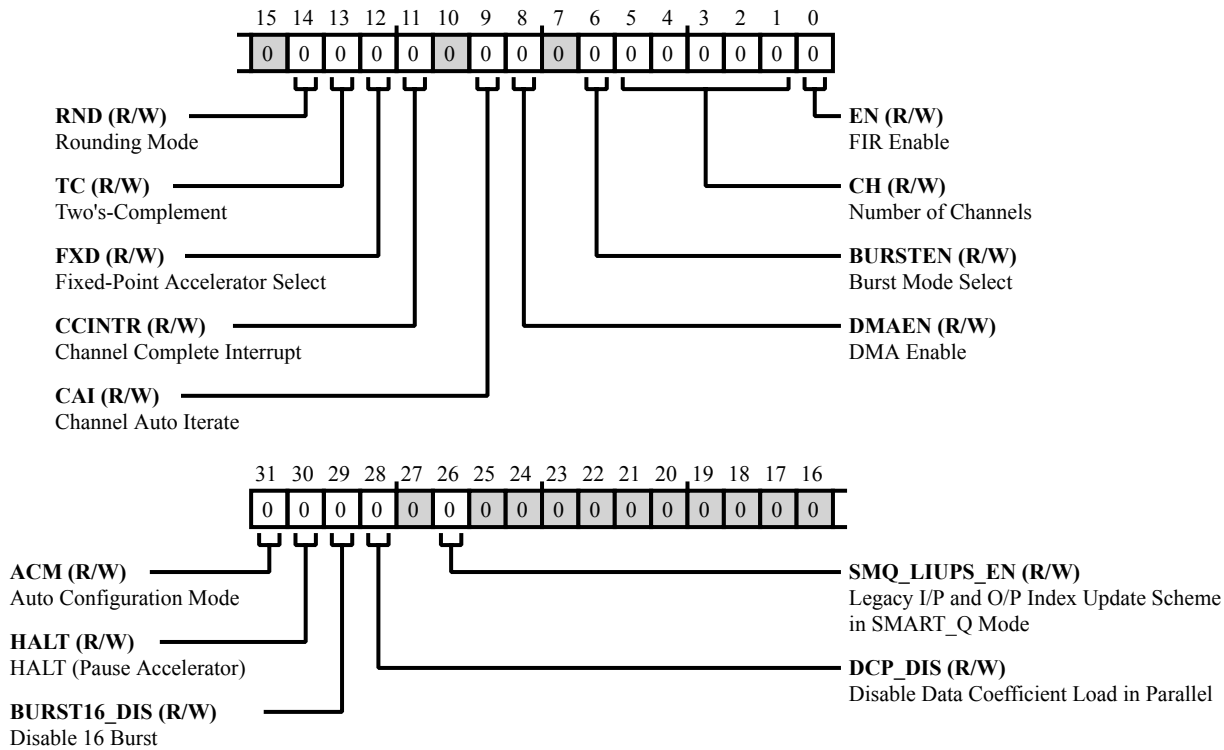


Figure 34-12: FIR_CTL1 Register Diagram

Table 34-14: FIR_CTL1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	ACM	Auto Configuration Mode. The <code>FIR_CTL1 . ACM</code> bit configures the mode for loading the TCB.
		0 Legacy Mode. Load TCB without the <code>FIR_CTL1</code> register. Core intervention may be required.
		1 Auto Configuration Mode. Load TCB with the <code>FIR_CTL1</code> register. Core intervention is not required.

Table 34-14: FIR_CTL1 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
30 (R/W)	HALT	HALT (Pause Accelerator). This bit is only valid in Auto Configuration mode (ACM)
		0 Release. Release accelerator for further processing of data.
		1 Pause. Pause accelerator for core to check status and modify or submit jobs.
29 (R/W)	BURST16_DIS	Disable 16 Burst. The <code>FIR_CTL1.BURST16_DIS</code> bit configures the accelerator to disable 16 burst when in burst mode.
		0 Maximum burst of INCR16 in burst mode for DMA and TCB
		1 Maximum burst of INCR8 in burst mode for DMA and non-burst for TCB
28 (R/W)	DCP_DIS	Disable Data Coefficient Load in Parallel. The <code>FIR_CTL1.DCP_DIS</code> bit enables data and coefficient loading to occur in parallel.
		0 Enable
		1 Disable
26 (R/W)	SMQ_LIUPS_EN	Legacy I/P and O/P Index Update Scheme in SMART_Q Mode. The <code>FIR_CTL1.SMQ_LIUPS_EN</code> bit configures the scheme the accelerator uses to update the input index (II) and output index (OI). This bit is only valid in ACM.
		0 Update the II with all '0' and the OI with all 'F', respectively
		1 Update II and OI after circular buffer scheme
14 (R/W)	RND	Rounding Mode. The <code>FIR_CTL1.RND</code> bit configures the accelerator to use a rounding mode.
		0 Round to Nearest
		1 Truncate (round towards zero)
13 (R/W)	TC	Two's-Complement. The <code>FIR_CTL1.TC</code> bit configures the accelerator to use either unsigned integer or signed integer
		0 Unsigned Integer
		1 Signed Integer

Table 34-14: FIR_CTL1 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
12 (R/W)	FXD	Fixed-Point Accelerator Select. The FIR_CTL1 . FXD bit configures the accelerator to use either 32-bit IEEE floating-point or 32-bit fixed-point.
		0 32-bit IEEE Floating-Point
		1 32-bit Fixed Point
11 (R/W)	CCINTR	Channel Complete Interrupt. The FIR_CTL1 . CCINTR bit configures the accelerator to generate an interrupt when each or all channels are done. This bit is only valid in Legacy Mode. In Auto Configuration Mode (ACM), interrupt generation for each channel is controlled using the FIR_CTL2.IMASK bit.
		0 Interrupt is Generated Only When All Channels are Done
		1 Interrupt is Generated After Each Channel is Done
9 (R/W)	CAI	Channel Auto Iterate. The FIR_CTL1 . CAI bit, if cleared, causes the TDM processing to stop (idle) once all channels are done. If set, processing moves to the first channel and continues TDM processing in a loop when all channels are done. Channel Auto Iterate is not available in Auto Configuration Mode (ACM).
		0 TDM Processing Stops (IDLE) Once All Channels are Over
		1 Moves to First Channel and Continues TDM Processing in a Loop When All Channels are Over
8 (R/W)	DMAEN	DMA Enable. The FIR_CTL1 . DMAEN bit enables and disables DMA on the FIR accelerator.
		0 DMA Disabled
		1 DMA Enabled
6 (R/W)	BURSTEN	Burst Mode Select. When the FIR_CTL1 . BURSTEN bit is set, burst mode is enabled for coefficient loads and data loads. When cleared, burst mode is disabled. By default, burst mode is disabled.

Table 34-14: FIR_CTL1 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
5:1 (R/W)	CH	Number of Channels. The <code>FIR_CTL1.CH</code> bit field configures the number of channels and is programmable from 0 to 31. This bit field is only valid in Legacy Mode. There is no channel number limitation in Auto Configuration Mode (ACM) as the accelerator keeps processing the TCBs until the chain pointer becomes null.
		0 Channel 1
		1-30 Channel 2-31
		31 Channel 32
0 (R/W)	EN	FIR Enable. The <code>FIR_CTL1.EN</code> bit enables and disables the FIR accelerator.
		0 Disable FIR
		1 Enable FIR

FIR Channel Control Register

The `FIR_CTL2` register is used to configure the channel specific parameters such as filter TAP length, window size, sample rate conversion, up/down sampling and ratio. In Auto Configuration Mode(ACM), this register is also used to configure additional channel specific parameters like interrupts and triggers.

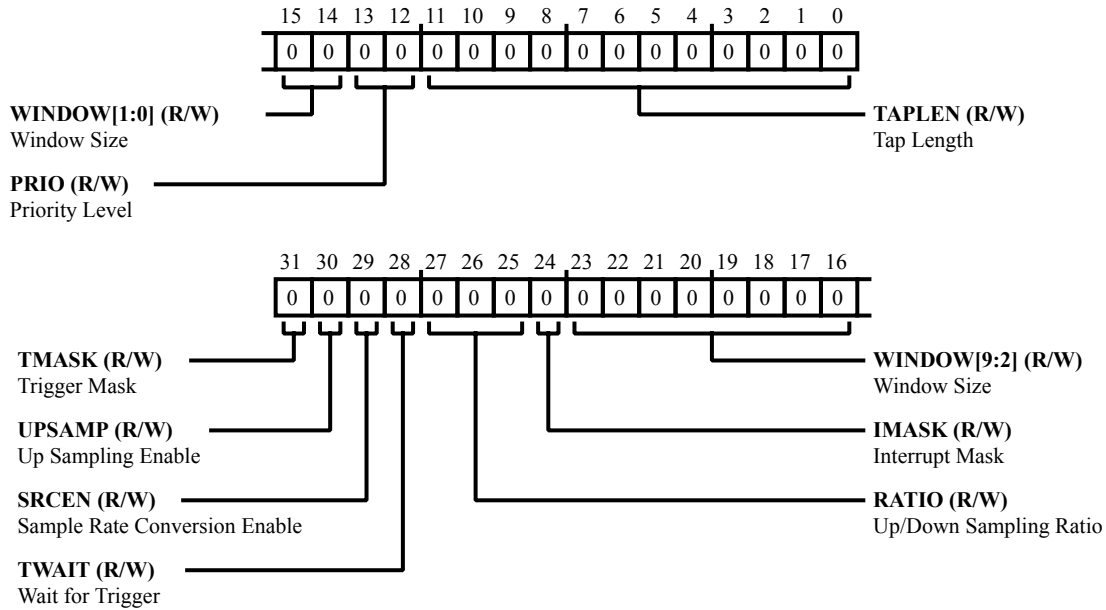


Figure 34-13: FIR_CTL2 Register Diagram

Table 34-15: FIR_CTL2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	TMASK	Trigger Mask. The <code>FIR_CTL2</code> . <code>TMASK</code> bit enables trigger generation for the channel. It is only valid in Auto Configuration Mode (ACM).
		0 Enable
		1 Mask
30 (R/W)	UPSAMP	Up Sampling Enable. The <code>FIR_CTL2</code> . <code>UPSAMP</code> bit enables up sampling.
		0 Down Sampling
		1 Up Sampling

Table 34-15: FIR_CTL2 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
29 (R/W)	SRCEN	Sample Rate Conversion Enable. The <code>FIR_CTL2.SRCEN</code> bit field enables sample rate conversion.
		0 Disabled
		1 Enabled
28 (R/W)	TWAIT	Wait for Trigger. The <code>FIR_CTL2.TWAIT</code> bit disables the wait for the trigger. It is only valid in Auto Configuration Mode (ACM).
		0 Disable wait for trigger for the channel
		1 Enable wait for the external trigger assertion
27:25 (R/W)	RATIO	Up/Down Sampling Ratio. The <code>FIR_CTL2.RATIO</code> bit field sets the sampling ratio (<code>FIR_CTL2.RATIO + 1</code>).
24 (R/W)	IMASK	Interrupt Mask. The <code>FIR_CTL2.IMASK</code> bit enables interrupt generation for the channel. This bit is only valid in Auto Configuration Mode (ACM).
		0 Enable
		1 Mask
23:14 (R/W)	WINDOW	Window Size. The <code>FIR_CTL2.WINDOW</code> bit field sets the window size which specifies the number of sample/block to process (sample based processing = window size of 0).
13:12 (R/W)	PRIO	Priority Level. The <code>FIR_CTL2.PRIO</code> bit field indicates the priority.
		0 Level 0 (lowest)
		1 Level 1
		2 Level 2
		3 Level 3 (highest)
11:0 (R/W)	TAPLEN	Tap Length. The <code>FIR_CTL2.TAPLEN</code> bit field sets the tap length which is programmable between 0-4095 (Tap Length = <code>FIR_CTL2.TAPLEN + 1</code>).

Debug Address Register

The `FIR_DBG_ADDR` register holds the debug address.

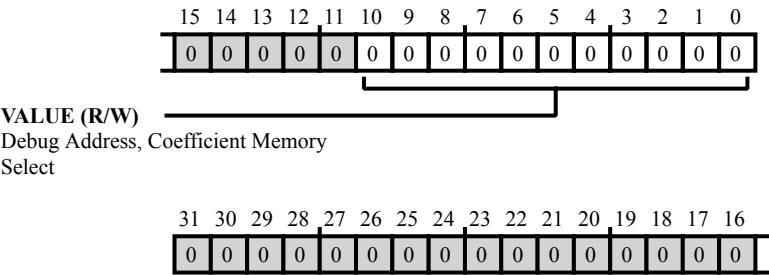


Figure 34-14: FIR_DBG_ADDR Register Diagram

Table 34-16: FIR_DBG_ADDR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
10:0 (R/W)	VALUE	Debug Address, Coefficient Memory Select. The <code>FIR_DBG_ADDR.VALUE</code> bit field holds the debug address (bits 0-10). Bit 11 configures whether the memory access is to coefficient memory (=0) or to delay line memory (=1).

FIR Debug Control Register

The `FIR_DBG_CTL` register controls debugging operations such as enabling debug mode running, hold or single stepping.

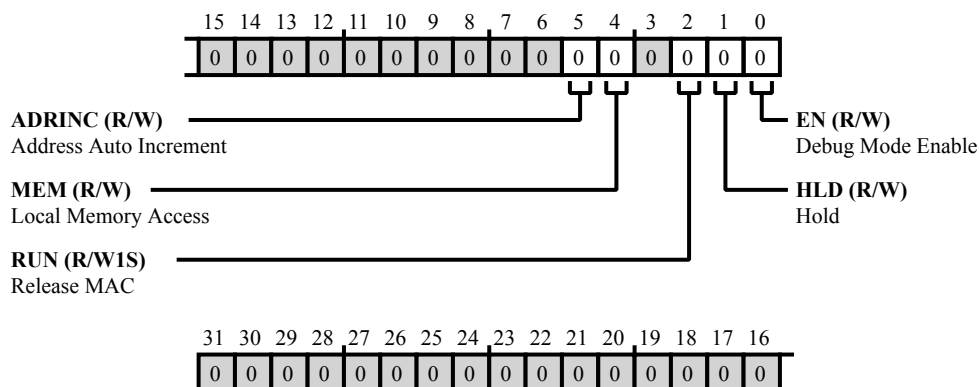


Figure 34-15: `FIR_DBG_CTL` Register Diagram

Table 34-17: `FIR_DBG_CTL` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
5 (R/W)	ADRINC	Address Auto Increment. The <code>FIR_DBG_CTL.ADRINC</code> bit allows the address register to auto-increment on <code>FIR_DBG_WRDAT</code> writes and <code>FIR_DBG_RDDAT</code> reads.
4 (R/W)	MEM	Local Memory Access. When the <code>FIR_DBG_CTL.MEM</code> bit is set, the data and coefficients memory can be indirectly accessed.
2 (R/W1S)	RUN	Release MAC. The <code>FIR_DBG_CTL.RUN</code> bit releases the MAC. This bit is self-clearing after one FIR clock cycle.
1 (R/W)	HLD	Hold. The <code>FIR_DBG_CTL.HLD</code> bit holds or single-steps through the FIR.
		0 Hold
		1 Single-step
0 (R/W)	EN	Debug Mode Enable. The <code>FIR_DBG_CTL.EN</code> bit enables debug mode. For local memory access, the <code>FIR_CTL1</code> register can be cleared.
		0 Disable Debug Mode
		1 Enable Debug Mode

FIR Debug Data Read Register

The `FIR_DBG_RDDAT` register hold the debug read data.

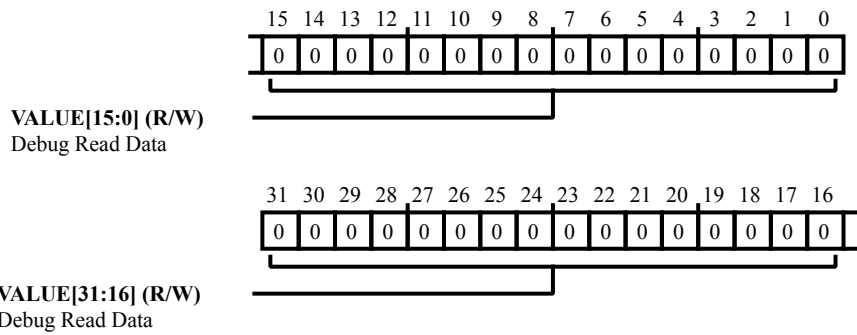


Figure 34-16: `FIR_DBG_RDDAT` Register Diagram

Table 34-18: `FIR_DBG_RDDAT` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Debug Read Data. The <code>FIR_DBG_RDDAT.VALUE</code> bit field holds the debug read data.

FIR Debug Data Write Register

The `FIR_DBG_WRDAT` register holds the debug write data.

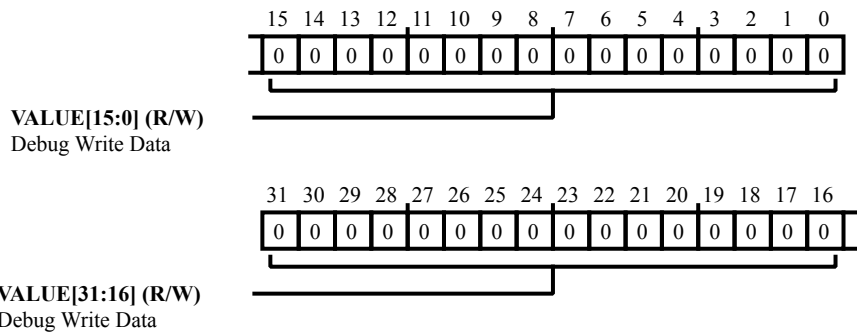


Figure 34-17: `FIR_DBG_WRDAT` Register Diagram

Table 34-19: `FIR_DBG_WRDAT` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Debug Write Data. The <code>FIR_DBG_WRDAT.VALUE</code> bit field holds the debug write data.

FIR DMA Status Register

The `FIR_DMASTAT` register provides information about chain pointer loading, coefficient DMA, data preload DMA, processing in progress, window complete, all channels complete.

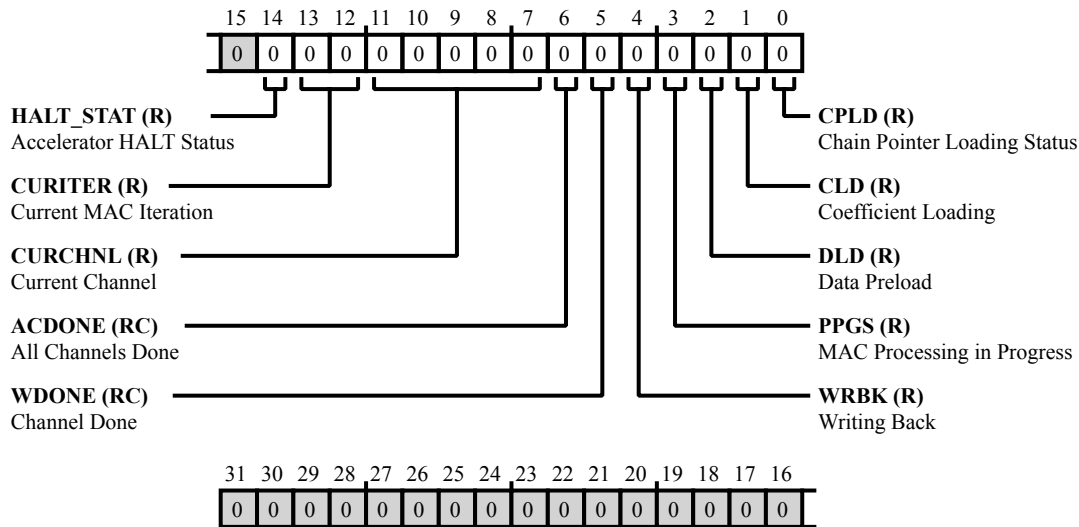


Figure 34-18: FIR_DMASTAT Register Diagram

Table 34-20: FIR_DMASTAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
14 (R/NW)	HALT_STAT	Accelerator HALT Status. Accelerator HALT status. 1 - Acknowledge to core that acceleration is in halt state. 0 - Not in halt state
13:12 (R/NW)	CURITER	Current MAC Iteration. The <code>FIR_DMASTAT.CURITER</code> bit indicates the current MAC iteration in multi-iteration mode. Zero indicates the final iteration.
11:7 (R/NW)	CURCHNL	Current Channel. The <code>FIR_DMASTAT.CURCHNL</code> bit indicates the channel that is being processed in the TDM slot. Zero indicates the last slot.
6 (RC/NW)	ACDONE	All Channels Done. The <code>FIR_DMASTAT.ACDONE</code> bit indicates the accelerator that processing all channels is complete. This is a sticky bit and is cleared on a register read. The <code>FIR_CTL1.CCINTR</code> bit does not affect the <code>FIR_DMASTAT.ACDONE</code> bit.
5 (RC/NW)	WDONE	Channel Done. The <code>FIR_DMASTAT.WDONE</code> bit indicates the accelerator that processing the current channel is complete. This is a sticky bit and is cleared on a register read. The <code>FIR_CTL1.CCINTR</code> bit does not affect the <code>FIR_DMASTAT.WDONE</code> bit.

Table 34-20: FIR_DMASTAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
4 (R/NW)	WRBK	Writing Back. The FIR_DMASTAT.WRBK bit indicates the accelerator is writing back the updated index registers.
3 (R/NW)	PPGS	MAC Processing in Progress. The FIR_DMASTAT.PPGS bit indicates MAC processing in progress.
2 (R/NW)	DLD	Data Preload. The FIR_DMASTAT.DLD bit indicates data preloading.
1 (R/NW)	CLD	Coefficient Loading. The FIR_DMASTAT.CLD bit indicates coefficient loading.
0 (R/NW)	CPLD	Chain Pointer Loading Status. The FIR_DMASTAT.CPLD bit indicates the state machine is in chain pointer load state.

FIR Input Data Base Register

The `FIR_INBASE` register contains the input word base address with the lower two bits removed.

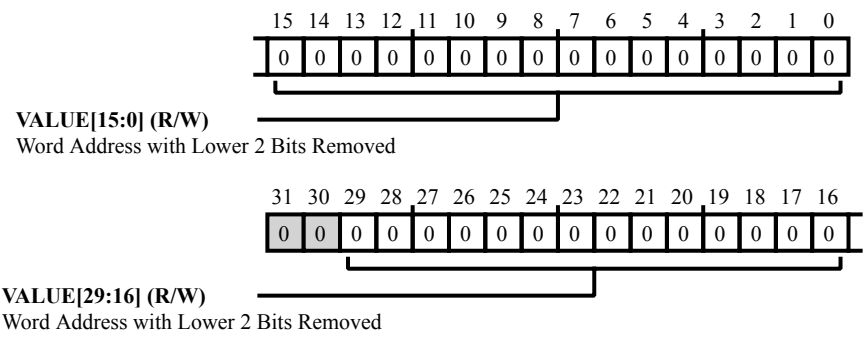


Figure 34-19: FIR_INBASE Register Diagram

Table 34-21: FIR_INBASE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:0 (R/W)	VALUE	Word Address with Lower 2 Bits Removed. The <code>FIR_INBASE.VALUE</code> bit field contains the word address with the lower 2 bits removed.

FIR Input Data Count Register

The `FIR_INCNT` register contains the 16-bit input data count.

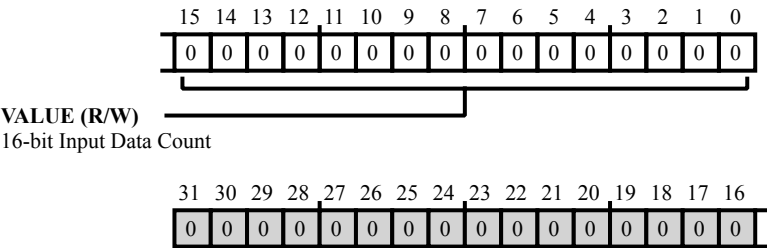


Figure 34-20: FIR_INCNT Register Diagram

Table 34-22: FIR_INCNT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	VALUE	16-bit Input Data Count. The <code>FIR_INCNT.VALUE</code> bit field contains the 16-bit input data count.

FIR Input Data Index Register

The `FIR_INIDX` register contains the input word address with the lower two bits removed.

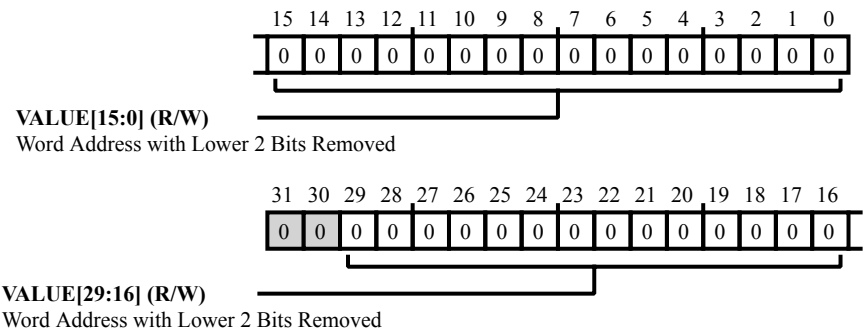


Figure 34-21: FIR_INIDX Register Diagram

Table 34-23: FIR_INIDX Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:0 (R/W)	VALUE	Word Address with Lower 2 Bits Removed. The <code>FIR_INIDX.VALUE</code> bit field contains the input word address with the lower two bits removed.

FIR Input Data Modifier Register

The `FIR_INMOD` register contains the 16-bit input data buffer modifier.

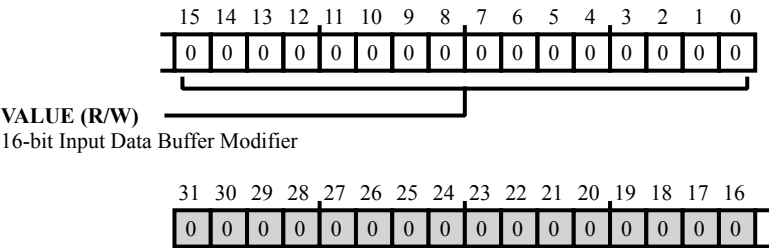


Figure 34-22: FIR_INMOD Register Diagram

Table 34-24: FIR_INMOD Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	VALUE	16-bit Input Data Buffer Modifier. The <code>FIR_INMOD.VALUE</code> bit field contains the 16-bit input data buffer modifier.

FIR MAC Status Register

The `FIR_MACSTAT` register provides the status of MAC operations. The status of all four multipliers/adders are available separately for programs to poll. In fixed-point mode only, the `ARx` bits are used (all other bits are reserved).

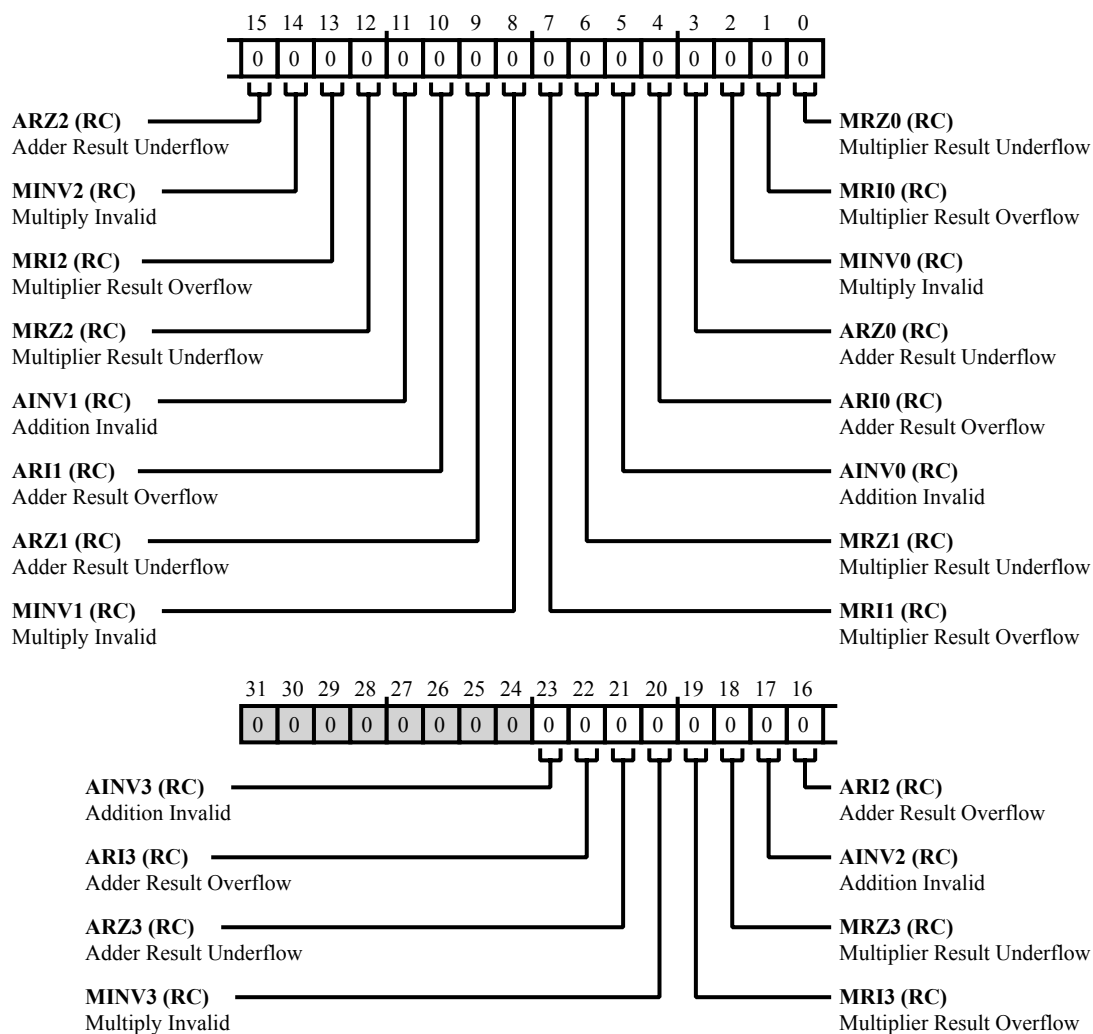


Figure 34-23: FIR_MACSTAT Register Diagram

Table 34-25: FIR_MACSTAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
23 (RC/NW)	AINV3	Addition Invalid. The <code>FIR_MACSTAT.AINV3</code> bit is set if the adder 3 addition is invalid.

Table 34-25: FIR_MACSTAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
22 (RC/NW)	ARI3	Adder Result Overflow. The <code>FIR_MACSTAT.ARI3</code> bit is set if the adder 3 result is infinity. Indicates overflow in fixed-point mode.
21 (RC/NW)	ARZ3	Adder Result Underflow. The <code>FIR_MACSTAT.ARZ3</code> bit is set if the adder 3 result is zero.
20 (RC/NW)	MINV3	Multiply Invalid. The <code>FIR_MACSTAT.MINV3</code> bit is set if the multiplier 3 multiply operation is invalid.
19 (RC/NW)	MRI3	Multiplier Result Overflow. The <code>FIR_MACSTAT.MRI3</code> bit is set if the multiplier 3 result is infinity.
18 (RC/NW)	MRZ3	Multiplier Result Underflow. The <code>FIR_MACSTAT.MRZ3</code> bit is set if the multiplier 3 result is zero.
17 (RC/NW)	AINV2	Addition Invalid. The <code>FIR_MACSTAT.AINV2</code> bit is set if the adder 2 addition is invalid.
16 (RC/NW)	ARI2	Adder Result Overflow. The <code>FIR_MACSTAT.ARI2</code> bit is set if the adder 2 result is infinity. Indicates overflow in fixed-point mode.
15 (RC/NW)	ARZ2	Adder Result Underflow. The <code>FIR_MACSTAT.ARZ2</code> bit is set if the adder 2 result is zero.
14 (RC/NW)	MINV2	Multiply Invalid. The <code>FIR_MACSTAT.MINV2</code> bit is set if the multiplier 2 multiply operation is invalid.
13 (RC/NW)	MRI2	Multiplier Result Overflow. The <code>FIR_MACSTAT.MRI2</code> bit is set if the multiplier 2 result is infinity.
12 (RC/NW)	MRZ2	Multiplier Result Underflow. The <code>FIR_MACSTAT.MRZ2</code> bit is set if the multiplier 2 result is zero.
11 (RC/NW)	AINV1	Addition Invalid. The <code>FIR_MACSTAT.AINV1</code> bit is set if the adder 1 addition is invalid.
10 (RC/NW)	ARI1	Adder Result Overflow. The <code>FIR_MACSTAT.ARI1</code> bit is set if the adder 1 result is infinity. Indicates overflow in fixed-point mode.
9 (RC/NW)	ARZ1	Adder Result Underflow. The <code>FIR_MACSTAT.ARZ1</code> bit is set if the adder 1 result is zero.

Table 34-25: FIR_MACSTAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
8 (RC/NW)	MINV1	Multiply Invalid. The <code>FIR_MACSTAT.MINV1</code> bit is set if the multiplier 1 multiply operation is invalid.
7 (RC/NW)	MRI1	Multiplier Result Overflow. The <code>FIR_MACSTAT.MRI1</code> bit is set if the multiplier 1 result is infinity.
6 (RC/NW)	MRZ1	Multiplier Result Underflow. The <code>FIR_MACSTAT.MRZ1</code> bit is set if the multiplier 1 result is zero.
5 (RC/NW)	AINV0	Addition Invalid. The <code>FIR_MACSTAT.AINV0</code> bit is set if the adder 0 addition is invalid.
4 (RC/NW)	ARI0	Adder Result Overflow. The <code>FIR_MACSTAT.ARI0</code> bit is set if the adder 0 result is infinity. Indicates overflow in fixed-point mode.
3 (RC/NW)	ARZ0	Adder Result Underflow. The <code>FIR_MACSTAT.ARZ0</code> bit is set if the adder 0 result is zero.
2 (RC/NW)	MINV0	Multiply Invalid. The <code>FIR_MACSTAT.MINV0</code> bit is set if the multiplier 0 multiply operation is invalid.
1 (RC/NW)	MRI0	Multiplier Result Overflow. The <code>FIR_MACSTAT.MRI0</code> bit is set if the multiplier 0 result is infinity.
0 (RC/NW)	MRZ0	Multiplier Result Underflow. The <code>FIR_MACSTAT.MRZ0</code> bit is set if multiplier 0 result is zero.

FIR Output Data Base Register

The `FIR_OUTBASE` register contains the output word base address with the lower two bits removed

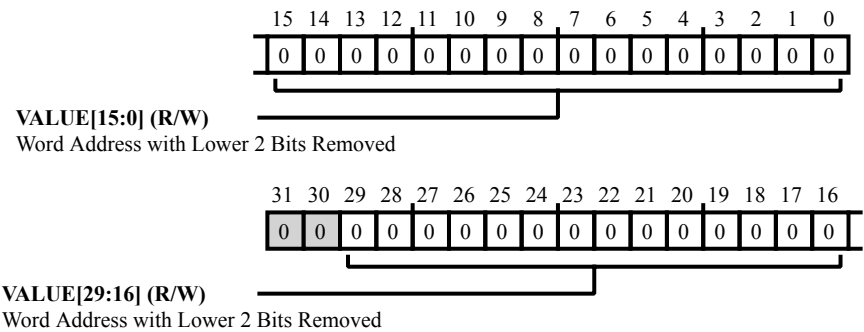


Figure 34-24: FIR_OUTBASE Register Diagram

Table 34-26: FIR_OUTBASE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:0 (R/W)	VALUE	Word Address with Lower 2 Bits Removed. The <code>FIR_OUTBASE.VALUE</code> bit field contains the word address with the lower 2 bits removed.

FIR Output Data Count Register

The `FIR_OUTCNT` register contains the 16-bit output buffer count.

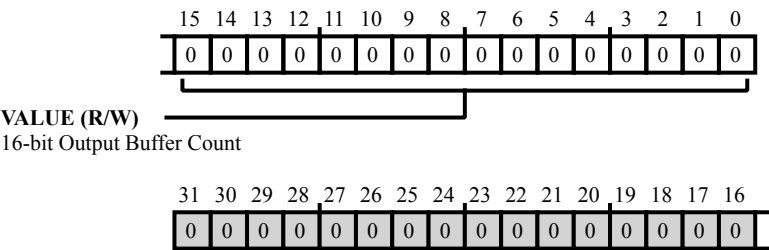


Figure 34-25: FIR_OUTCNT Register Diagram

Table 34-27: FIR_OUTCNT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	VALUE	16-bit Output Buffer Count. The <code>FIR_OUTCNT.VALUE</code> bit field contains the 16-bit output buffer count.

FIR Output Data Index Register

The `FIR_OUTIDX` register contains the output word address with the lower two bits removed.

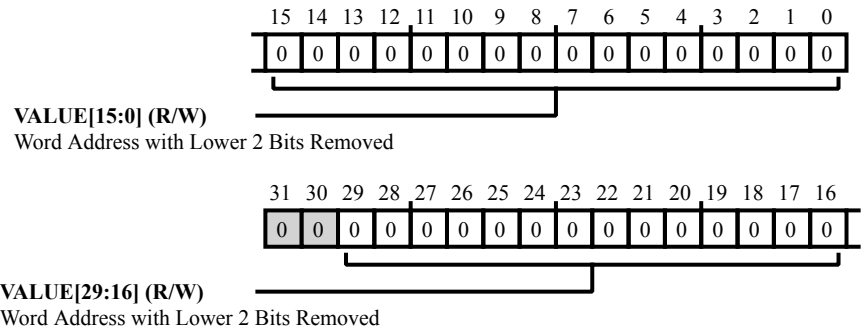


Figure 34-26: FIR_OUTIDX Register Diagram

Table 34-28: FIR_OUTIDX Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:0 (R/W)	VALUE	Word Address with Lower 2 Bits Removed. The <code>FIR_OUTIDX.VALUE</code> bit field contains the word address with the lower 2 bits removed.

FIR Output Data Modifier Register

The `FIR_OUTMOD` register contains the 16-bit output data buffer modifier.

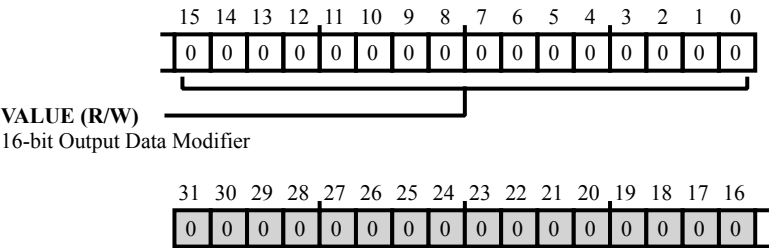


Figure 34-27: FIR_OUTMOD Register Diagram

Table 34-29: FIR_OUTMOD Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	VALUE	16-bit Output Data Modifier. The <code>FIR_OUTMOD.VALUE</code> bit field contains the 16-bit output data modifier.

Software Control Register 1

The `FIR_SCTL1` register is used to control software.

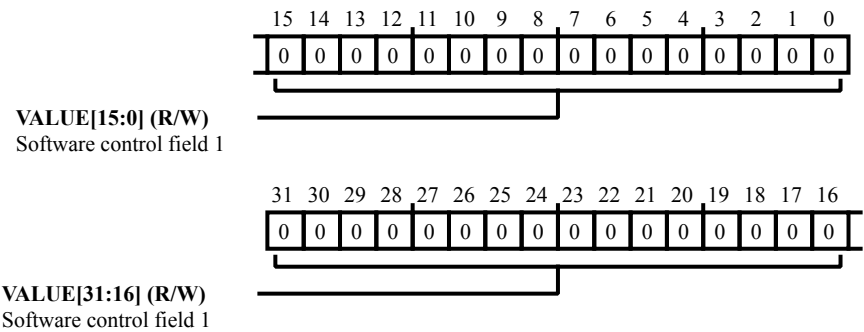


Figure 34-28: `FIR_SCTL1` Register Diagram

Table 34-30: `FIR_SCTL1` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Software control field 1.

Software Control Register 2

The `FIR_SCTL2` register is used to control software.

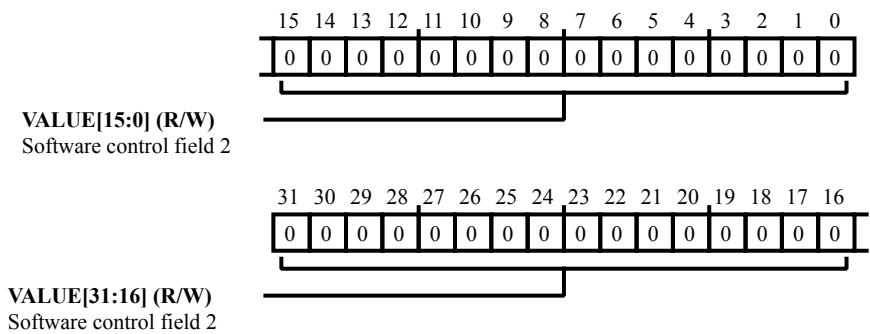


Figure 34-29: FIR_SCTL2 Register Diagram

Table 34-31: FIR_SCTL2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Software control field 2.

Secondary Global Control Register

The `FIR_SGCTL` register is used to configure the global parameters for the accelerator in ACM mode for loading CTL1 register bits as part of TCB.

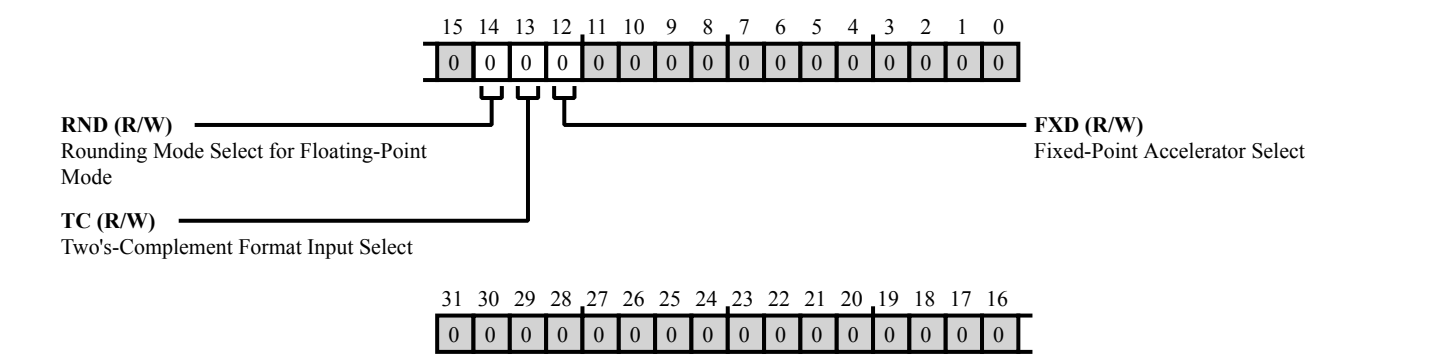


Figure 34-30: FIR_SGCTL Register Diagram

Table 34-32: FIR_SGCTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
14 (R/W)	RND	Rounding Mode Select for Floating-Point Mode. The <code>FIR_SGCTL.RND</code> bit indicates whether the rounding mode is to the nearest or truncated (round towards zero).
		0 Round to nearest
		1 Truncate
13 (R/W)	TC	Two's-Complement Format Input Select. The <code>FIR_SGCTL.TC</code> bit indicates the two's complement format input for fixed-point mode.
		0 Unsigned integer
		1 Signed integer
12 (R/W)	FXD	Fixed-Point Accelerator Select. When set (=1), the <code>FIR_SGCTL.FXD</code> bit indicates a 32-bit fixed point accelerator. When cleared (=0), the bit indicates a 32-bit floating-point compatible with SHARC-XI.
		0 32-bit floating-point
		1 32-bit fixed point

35 IIR Accelerator (IIR)

The processor includes an Infinite Impulse Response (IIR) filter accelerator implemented in hardware that reduces the processing load on the core, freeing it up for other tasks.

Features

The accelerator supports a maximum of 32 channels in legacy mode. In Auto Configuration Mode (ACM), there is no limit on number of channels. The accelerator locally stores all the biquad coefficients of 32 channels in legacy mode. Window size can be configured from 1 (sample-based) to 1024.

The IIR has the following features:

- Supports IEEE floating point format 32/40-bit
- Rounding modes compatible with SHARC+ core MACs
- Sample-based or window-based processing
- Up to 64 cascaded biquads per channel
- Up to 32 filter channels available in TDM
- Allows biquad save state storage

NOTE: The IIR accelerator module has local memory that is not accessible by the core during regular operation mode. Unlike in the previous SHARC processors, the IIR accelerator modules each have access to the system memory (on-chip/off-chip).

Unlike in the previous SHARC processors, where only one of the IIR or FIR accelerators can be enabled at a time, the processor can simultaneously use both the IIR and the FIR accelerators.

Clocking

The IIR accelerator runs at the speed of core clock.

Functional Description

The *IIR Accelerator Block Diagram* shows the blocks of the IIR hardware accelerator.

The accelerator has:

- a coefficient memory size of 1440×40 bits (288 biquads x 5 coefficients).
- a data memory size of 576×40 bits ().
- one SHARC+ core compatible MAC unit with an input data buffer to supply data to the MAC.

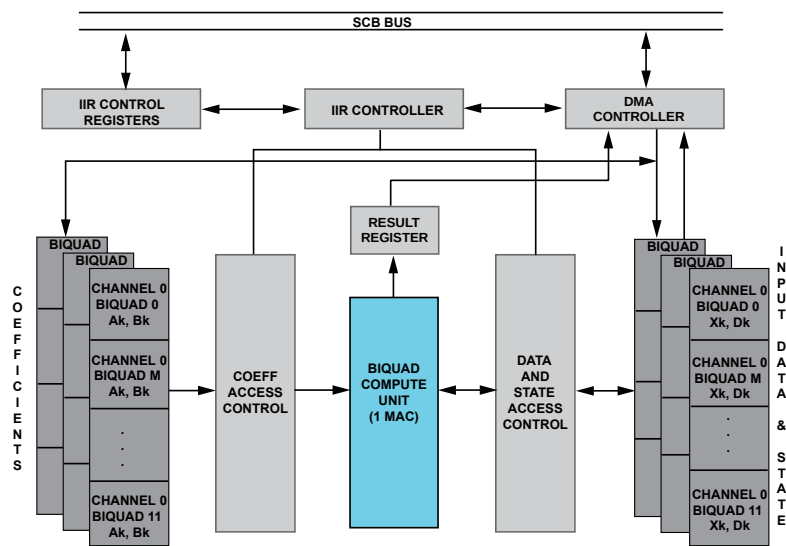


Figure 35-1: IIR Accelerator Block Diagram

A transposed direct form II biquadratic is used to implement the IIR accelerator, which reduces coefficient sensitivity. The *Transposed Direct Form II Biquad* figure shows the signal flow graph for the biquad structure.

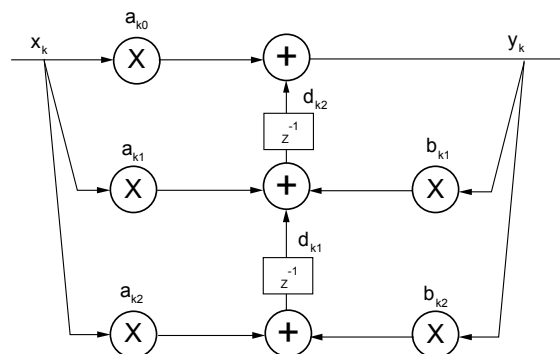


Figure 35-2: Transposed Direct Form II Biquad

The accelerator has the following logical subblocks.

- A data path unit with the following elements:
 - 32/40-bit coefficient memory (A_k , B_k) for storing biquad coefficients

- 32/40-bit input data (X_k) and state (D_k)
- One 40/32-bit floating-point multiplier and adder (MAC) unit
- An input data buffer to efficiently supply data to MAC
- One 40-bit result register to hold result of the biquad
- Configuration registers for controlling various parameters such as the number of biquads, the number of channels, interrupt control, and DMA control.
- A core access interface for writing the DMA/filter configuration registers and for reading the status registers.
- A DMA bus interface for transferring data to and from the accelerator. This interface also preloads the coefficients (A_k , B_k) and state (D_k) at startup.
- DMA configuration registers for the transfer of input data, output data, and coefficients.

The accelerator block is integrated with the SHARC core. For more information, see [Figure 37-4 SHARC SCB with FIR/IIR Accelerator](#).

ADSP-21560_21561_21564_21568 IIR Register List

The IIR module reduces the processing load on the core. For more information on IIR functionality, see the IIR register descriptions.

Table 35-1: ADSP-21560_21561_21564_21568 IIR Register List

Name	Description
IIR_CHNPTR	Chain Pointer Register
IIR_COEFIDX	Coefficient Buffer Index Register
IIR_COEFLEN	Coefficient Buffer Length Register
IIR_COEFMOD	Coefficient Index Modifier Register
IIR_CTL1	Global Control Register
IIR_CTL2	Channel Control Register
IIR_DBG_ADDR	IIR Debug Address Register
IIR_DBG_CTL	IIR Debug Control Register
IIR_DBG_RDDAT_HI	IIR Debug Read Data High Register
IIR_DBG_RDDAT_LO	IIR Debug Read Data Low Register
IIR_DBG_WRDAT_HI	IIR Debug Write Data High Register
IIR_DBG_WRDAT_LO	IIR Debug Write Data Low Register
IIR_DMASTAT	DMA Status Register
IIR_INBASE	Input Buffer Base Register

Table 35-1: ADSP-21560_21561_21564_21568 IIR Register List (Continued)

Name	Description
IIR_INIDX	Input Data Index Register
IIR_INLEN	Input Data Buffer Length Register
IIR_INMOD	Input Data Index Modifier Register
IIR_MACSTAT	MAC Status Register
IIR_OUTBASE	Output Buffer Base Register
IIR_OUTIDX	Output Data Buffer Index Register
IIR_OUTLEN	IIR Output Data Buffer Length Register
IIR_OUTMOD	IIR Output Data Index Modifier Register
IIR_SCTL1	Software Control Register1
IIR_SCTL2	Software Control Register2
IIR_SGCTL	Secondary Global Control Register

ADSP-21560_21561_21564_21568 IIR Interrupt List

Table 35-2: ADSP-21560_21561_21564_21568 IIR Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
120	IIR0_DMA	IIR0 DMA	Edge	
121	IIR0_STAT	IIR0 Status	Edge	

ADSP-21560_21561_21564_21568 IIR Trigger List

Table 35-3: ADSP-21560_21561_21564_21568 IIR Trigger List Generators

Trigger ID	Name	Description	Sensitivity
40	IIR0_DMA	IIR0 DMA	Edge
104	IIR0_TRGO	IIR0 IIR Trigger output	Edge

Table 35-4: ADSP-21560_21561_21564_21568 IIR Trigger List Receivers

Trigger ID	Name	Description	Sensitivity
117	IIR0_TRGI	IIR0 IIR Trigger input	Pulse

Multiply and Accumulate (MAC) Unit

The *IIR MAC Unit* figure shows a pipelined multiplier and accumulator unit that operates on the data and coefficients fetched from the data and coefficient memory. The MAC can perform 32-bit floating-point or 40-bit

floating-point MAC operations. The 32-bit floating-point operations generate 32-bit results and the 40-bit floating-point operations generate 40-bit results.

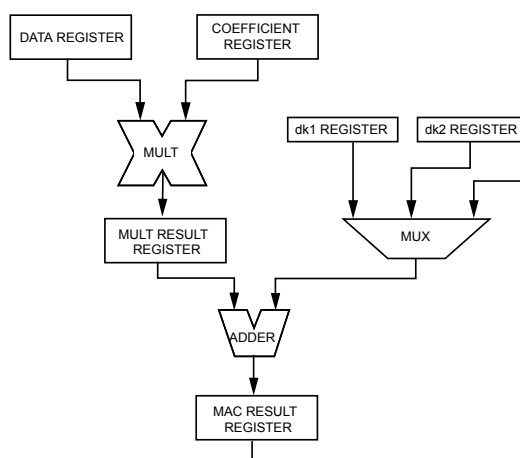


Figure 35-3: IIR MAC Unit

Input Data and Biquad State

The size of the data memory is 576×40 bits and locally holds the dk1 and dk2 state of all biquads. The DMA controller fetches the sample data from internal memory and calculates the output plus the dk1 and dk2 values for each biquad and stores them in local data memory.

Coefficient Memory

The size of coefficient memory is 1440×40 bits and stores all the coefficients for all biquads. At startup, the DMA loads the coefficients from system memory into local coefficient memory.

Internal Memory Storage

This section describes the required storage model for the IIR accelerator.

Coefficient Memory Storage

Store the coefficients and Dk values for a specific biquad BQD[k] in internal memory in the following order: Ak0, Ak1, Bk1, Ak2, Bk2, Dk2, and Dk1.

NOTE: The naming convention for the filter coefficients used here is different from the one used in MATLAB. The following conversion should be used when using MATLAB generated coefficients:

$(Akx = bx \text{ and } Bkx = -ax)$.

Store the coefficients for each biquad in the order:

b0, b1, -a1, b2, -a2, dk2, dk1

For N biquad stages, store the coefficients in the order:

```

b01, b11, -a11, b21, -a21, dk21, dk11,
b02, b12, -a12, b22, -a22, dk22, dk12,
, . . . . .
b0N, b1N, -a1N, b2N, -a2N, dk2N, dk1N.

```

where b_{xN} and a_{xN} are the coefficients ($[b, a]$) for the N th biquad stage.

Operating Modes

The accelerator operates:

- In [Window Processing Mode](#), that also includes sample-based processing mode
- In [40-Bit Floating-Point Mode](#)
- In [Save Biquad State Mode](#)

Window Processing Mode

Select the sample-based processing mode by configuring window size to one. All biquads for a particular channel process one sample, and then calculate a final output sample in this mode.

In window-based mode, multiple output samples (maximum of 1024, equal to the window size) of that channel are calculated. After these calculations are complete, the accelerator begins processing the next channel. A register field provides a window size parameter to specify the length of the window.

40-Bit Floating-Point Mode

In 40-bit floating-point mode, the accelerator treats input data and coefficients as a 40-bit floating-point numbers. The 40-bit floating-point MAC operations generate 40-bit results. This mode can be selected by setting the `IIR_CTL1.FORTYBIT` bit.

In ACM mode, 40-bit floating-point mode can be selected by setting the `IIR_SGCTL.FORTYBIT` bit field for each channel.

As the DMA bus width is 32 bits, in 40-bit mode the IIR accelerator performs two packed 32-bit accesses to the memory to:

- fetch one 40-bit input or coefficient data
- store one 40-bit output word

The first 32-bit word provides the lower 32 bits and the 8 LSBs of the second 32-bit word provides rest of the upper 8 bits of the complete 40-bit word. The *32-Bit to 40-Bit Packing* figure shows the 32- to 40-bit packing used by the accelerator.

NOTE: Overhead time may be required to pack the input 40-bit data into the format acceptable by the IIR accelerator and for unpacking the output of accelerator to a format acceptable to the rest of the application.

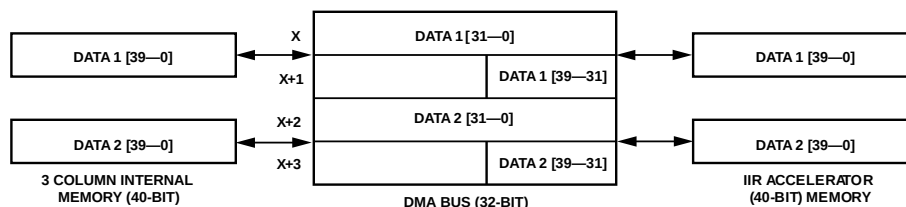


Figure 35-4: 32-Bit to 40-Bit Packing

Save Biquad State Mode

The `IIR_CTL1.SS` bit stores the current biquad states in local memory (writes all the `Dk1` and `Dk2` states back into the system memory states). This is useful for applications that require fast switching to another high-priority accelerator task—a required IIR to FIR processing transition for example. After resuming, these states can be reloaded and IIR processing can be continued. DMA status is automatically stored after each iteration.

NOTE: In legacy mode, the accelerator loads the biquad coefficients for all the channels before processing the first channel. After processing the last channel, when the `IIR_CTL1.SS` bit is set, the accelerator stores the biquad states of all the channels in local memory (writes all the `Dk1` and `Dk2` states back into the system memory states). In ACM mode, the accelerator loads the biquad coefficients for only the current channel before starting to process the current channel. When the `IIR_SGCTL.SS` bit is set, the accelerator stores the biquad states of the current channel in local memory before starting calculations for the next channel.

NOTE: During the save state operation, it is not permitted to have write access to any of the IIR accelerator registers that are loaded by chaining. Attempted writes to these registers can result in the blocking of IOP core reads until the save state operation completes.

Data Transfers

The IIR filter works exclusively through DMA.

IIR Accelerator TCB

The location of the DMA parameters for the next sequence comes from the chain pointer register. This register points to the next set of DMA parameters stored in the system memory of the processor known as the Task Control Block (TCB). In chained DMA operations, the processor automatically initializes and then begins another DMA transfer once the current DMA transfer completes. Each new set of parameters is stored in a user-initialized memory buffer or TCB for a chosen peripheral.

Chain Assignment

The structure of a TCB is conceptually the same as the structure of a linked list. Each TCB has several data values and a pointer to the next TCB. The chain pointer of a TCB can point to itself to re-run the same DMA continuously. The IIR accelerator reads each word of the TCB and loads it into the corresponding register. A TCB with a chain pointer register value of zero indicates the end of the chain (no further TCBs to load). The IIR accelerator supports

circular buffer chained DMA. The following tables show the required TCBs for chained DMA in Legacy Mode and ACM. TCB refers to the start address of the TCB array.

NOTE: The IIR accelerator DMA has two available TCB loading sequences:

- Loads five parameters for the coefficients (`IIR_CTL2`, `IIR_COEFIDX`, `IIR_COEFMOD`, `IIR_COEFLEN`, and `IIR_CHNPTR`).
- Loads 10 parameters for the data (`IIR_CTL2`, `IIR_INIDX`, `IIR_INMOD`, `IIR_INLEN`, `IIR_INBASE`, `IIR_OUTIDX`, `IIR_OUTMOD`, `IIR_OUTLEN`, `IIR_OUTBASE`, and `IIR_CHNPTR`).

Initialize `IIR_CHNPTR` to TCB+12 in legacy mode and TCB+15 in ACM.

Table 35-5: IIR TCBs for Chained DMA in Legacy Mode

Address	Register
TCB	<code>IIR_CHNPTR</code>
TCB + 0x1	<code>IIR_COEFLEN</code>
TCB + 0x2	<code>IIR_COEFMOD</code>
TCB + 0x3	<code>IIR_COEFIDX</code>
TCB + 0x4	<code>IIR_OUTBASE</code>
TCB + 0x5	<code>IIR_OUTLEN</code>
TCB + 0x6	<code>IIR_OUTMOD</code>
TCB + 0x7	<code>IIR_OUTIDX</code>
TCB + 0x8	<code>IIR_INBASE</code>
TCB + 0x9	<code>IIR_INLEN</code>
TCB + 0xA	<code>IIR_INMOD</code>
TCB + 0xB	<code>IIR_INIDX</code>
TCB + 0xC	<code>IIR_CTL2</code>

Table 35-6: IIR TCBs for Chained DMA in ACM

Address	Register
TCB	<code>IIR_CHNPTR</code>
TCB + 0x1	<code>IIR_SCTL1</code>
TCB + 0x2	<code>IIR_SCTL2</code>
TCB + 0x3	<code>IIR_SGCTL</code>
TCB + 0x4	<code>IIR_COEFLEN</code>
TCB + 0x5	<code>IIR_COEFMOD</code>

Table 35-6: IIR TCBs for Chained DMA in ACM
(Continued)

Address	Register
TCB + 0x6	IIR_COEFIDX
TCB + 0x7	IIR_OUTBASE
TCB + 0x8	IIR_OUTLEN
TCB + 0x9	IIR_OUTMOD
TCB + 0xA	IIR_OUTIDX
TCB + 0xB	IIR_INBASE
TCB + 0xC	IIR_INLEN
TCB + 0xD	IIR_INMOD
TCB + 0xE	IIR_INIDX
TCB + 0xF	IIR_CTL2

DMA Access

The IIR accelerator has two DMA channels (accelerator input and output) to connect to the system memory. The DMA controller fetches the data and coefficients from memory and stores the result.

Burst Mode Access

Burst mode enhances the throughput of the DMA channel and reduces the overall load on the system. When the `IIR_CTL1.BURSTEN` bit is set, it loads all the biquads for all the channels with burst enabled. The IIR module supports only INCR8 burst.

Chain Pointer DMA

The DMA controller supports a circular buffer chain pointer DMA. One transfer control block (TCB) must be configured for each channel. The TCB contains:

- A control register value to configure the filter parameters (such as number of biquads, window size) for each channel. For ACM, additional parameters such as an interrupt mask, trigger mask, and trigger wait exist.
- Software control register values in ACM for each channel.
- Secondary control register value in ACM to configure rounding mode, 40-bit mode, and save state for each channel.
- DMA parameter register values for the input data.
- DMA parameter register values for coefficient load.
- DMA parameter register values for output data.

In legacy mode, The chain pointer (`IIR_CHNPTR`) field of the last channel TCB should point to the first channel TCB. This configuration exists so that when the IIR accelerator is enabled, the module:

1. Loads the coefficients (A_k , B_k) and state variables (D_k) for all the channels into the local coefficient memory of the IIR.
2. Loops back to the first channel again to start fetching the input data for processing.

In ACM, the chain pointer field of the last channel TCB need not point to the first channel TCB.

When the accelerator is enabled, the module:

1. Only loads the coefficients (A_k , B_k) and state variables (D_k) for the current channel being processed.
2. Starts fetching input data for processing.
3. After processing the current channel, when the `IIR_SGCTL.SS` bit is set, it saves the biquad states of the current channel in local memory and moves to the next channel.

In ACM, the accelerator keeps processing the TCBs until the chain pointer becomes null.

The accelerator loads the TCB into its internal registers and uses these values to fetch coefficients, fetch data, and store results. After processing a window of data for any channel, the accelerator writes back the `IIR_INIDX` (input index register) and `IIR_OUTIDX` (output index register) values to the TCB in memory. Data processing can then begin from where it left off during the next time slot of that channel.

For 32-bit mode, the write-back values for the index registers are equal to $IIR_{II} + W$ and $IIR_{OI} + W$.

For 40-bit mode, the write-back values are: $IIR_INIDX + 2 \times W$ and $IIR_OUTIDX + 2 \times W$.

Accelerator input and output channels connect to system memory.

NOTE: The `IIR_CTL2` register is part of the IIR TCB. This configuration allows software to program individual IIR channels having different control attributes.

In ACM, when the `IIR_CTL1.SMQ_LIUPS_EN` bit is set, the accelerator updates the `IIR_INIDX` and `IIR_OUTIDX` bit fields of the TCB in memory after processing a window of data and according to the circular buffer scheme. When the `IIR_CTL1.SMQ_LIUPS_EN` bit is cleared, the accelerator updates the `IIR_INIDX` and `IIR_OUTIDX` bit fields of the TCB in memory to 0x00000000 and 0xFFFFFFFF after processing a window of data. The `IIR_CTL1.SMQ_LIUPS_EN` bit is only valid in ACM.

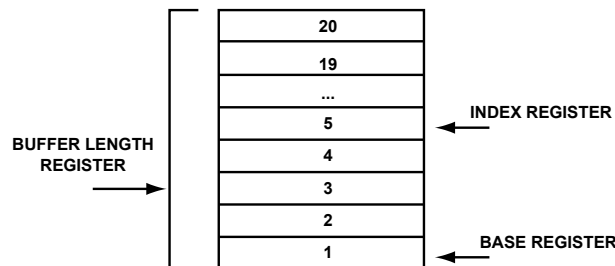


Figure 35-5: Circular Buffer Addressing

Interrupts

The *IIR Interrupt Overview* table provides the source of interrupt and service instructions for the IIR interrupts.

Table 35-7: IIR Interrupt Overview

Accelerator Mode	Default Programmable Interrupt	Sources	Masking	Service
Legacy Mode	IIR_DMA	Window Complete	N/A	ROC from IIR_DMASTAT + RTI instruction
		All channels complete		
	IIR_STAT	MAC IEEE floating point exceptions	N/A	ROC from IIR_MACSTAT + RTI instruction
		MAC fixed point overflow		
Auto Configuration Mode	IIR_DMA	Window Complete	IIR_CTL2 . IMASK	ROC from IIR_DMASTAT + RTI instruction
	IIR_STAT	MAC IEEE floating point exceptions	N/A	ROC from IIR_MACSTAT + RTI instruction
		MAC fixed point overflow	N/A	

Sources

The IIR module drives two interrupt signals, [IIR_DMA](#) for the DMA status and [IIR_STAT](#) for the MAC status. The IIR module generates interrupts as described in the following sections.

Window Complete

This interrupt generates at the end of each channel when all the output samples are calculated for a window and the channel writes back updated index values.

In legacy mode, when the [IIR_CTL1](#) . CCINTR bit is set, an interrupt generates after completion of window processing of each channel. In ACM, the interrupt generation can be selectively masked using the [IIR_CTL2](#) . IMASK bit for each channel.

Interrupt masking for each channel can be enabled in legacy mode by setting the [IIR_CTL1](#) . L_TIMASK_EN bit. In legacy mode, when the [IIR_CTL1](#) . L_TIMASK_EN bit is set, interrupt generation for each channel is controlled using the [IIR_CTL2](#) . IMASK bit.

All Channels Complete

This interrupt generates when all the channels are complete or when one iteration of time slots completes. The interrupt follows the access completion rule, where the interrupt generates when all data are written back to system memory.

The All Channels Complete interrupt source only applies in legacy mode. When the [IIR_CTL1](#) . CCINTR and [IIR_CTL1](#) . L_TIMASK_EN bits are not set in Legacy Mode, the all channel complete interrupt generates. In ACM, interrupt generation for each channel is controlled using the [IIR_CTL2](#) . IMASK bit.

NOTE: In ACM, the Output DMA complete interrupt can be generated after the Save State completes for the channel by clearing the `IIR_SGCTL.SSESEL` bit. When this bit is set, the Output DMA interrupt generates before the Save State completes for the channel.

In Legacy Mode, when the `IIR_CTL1.CCINTR` and `IIR_CTL1.SSESEL` bits are cleared, the All Channels Done Interrupt generates after the Save State operation completes for all the channels.

MAC Status

A MAC status interrupt generates under the following conditions:

- Multiplier result zero—set when multiplier result is zero
- Multiplier result infinity—set when multiplier result is infinity
- Multiply invalid—set when multiply operation is invalid
- Adder result zero—set when adder result is zero
- Adder result infinity—set when adder result is infinity
- Adder invalid—set when addition is invalid

ADSP-21560_21561_21564_21568 IIR Register Descriptions

The IIR filter accelerator (IIR) contains the following registers.

Table 35-8: ADSP-21560_21561_21564_21568 IIR Register List

Name	Description
<code>IIR_CHNPTR</code>	Chain Pointer Register
<code>IIR_COEFIDX</code>	Coefficient Buffer Index Register
<code>IIR_COEFLEN</code>	Coefficient Buffer Length Register
<code>IIR_COEFMOD</code>	Coefficient Index Modifier Register
<code>IIR_CTL1</code>	Global Control Register
<code>IIR_CTL2</code>	Channel Control Register
<code>IIR_DBG_ADDR</code>	IIR Debug Address Register
<code>IIR_DBG_CTL</code>	IIR Debug Control Register
<code>IIR_DBG_RDDAT_HI</code>	IIR Debug Read Data High Register
<code>IIR_DBG_RDDAT_LO</code>	IIR Debug Read Data Low Register
<code>IIR_DBG_WRDAT_HI</code>	IIR Debug Write Data High Register
<code>IIR_DBG_WRDAT_LO</code>	IIR Debug Write Data Low Register
<code>IIR_DMASTAT</code>	DMA Status Register
<code>IIR_INBASE</code>	Input Buffer Base Register

Table 35-8: ADSP-21560_21561_21564_21568 IIR Register List (Continued)

Name	Description
IIR_INIDX	Input Data Index Register
IIR_INLEN	Input Data Buffer Length Register
IIR_INMOD	Input Data Index Modifier Register
IIR_MACSTAT	MAC Status Register
IIR_OUTBASE	Output Buffer Base Register
IIR_OUTIDX	Output Data Buffer Index Register
IIR_OUTLEN	IIR Output Data Buffer Length Register
IIR_OUTMOD	IIR Output Data Index Modifier Register
IIR_SCTL1	Software Control Register1
IIR_SCTL2	Software Control Register2
IIR_SGCTL	Secondary Global Control Register

Chain Pointer Register

The `IIR_CHNPTR` register should be written with word address without the lower 2 bits.

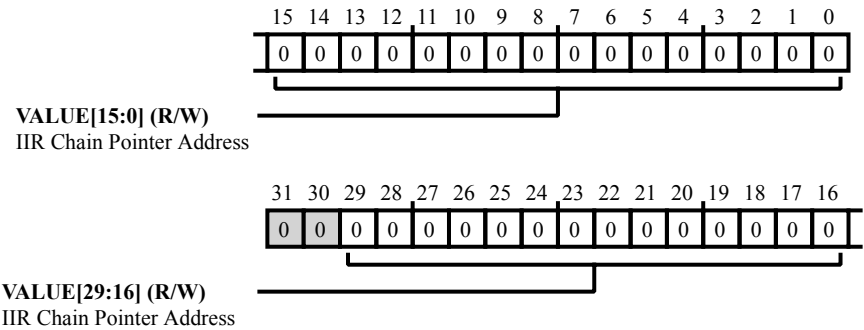


Figure 35-6: IIR_CHNPTR Register Diagram

Table 35-9: IIR_CHNPTR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:0 (R/W)	VALUE	IIR Chain Pointer Address. The <code>IIR_CHNPTR.VALUE</code> bit field contains the chain pointer address.

Coefficient Buffer Index Register

The `IIR_COEFIDX` register contains the word address with the lower 2 bits removed.

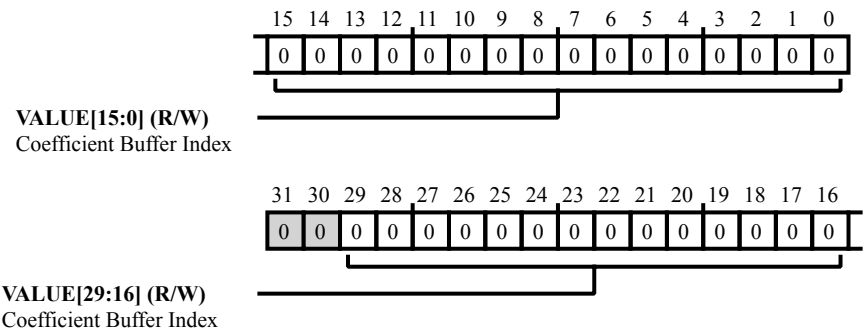


Figure 35-7: IIR_COEFIDX Register Diagram

Table 35-10: IIR_COEFIDX Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:0 (R/W)	VALUE	Coefficient Buffer Index. The <code>IIR_COEFIDX.VALUE</code> bit field provides the coefficient buffer index.

Coefficient Buffer Length Register

The `IIR_COEFLEN` register provides the coefficient buffer length.

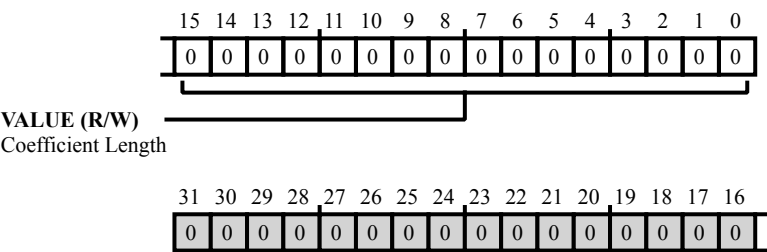


Figure 35-8: IIR_COEFLEN Register Diagram

Table 35-11: IIR_COEFLEN Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	VALUE	Coefficient Length. The <code>IIR_COEFLEN.VALUE</code> bit field provides the coefficient buffer length.

Coefficient Index Modifier Register

The `IIR_COEFMOD` register provides the coefficient index modifier.

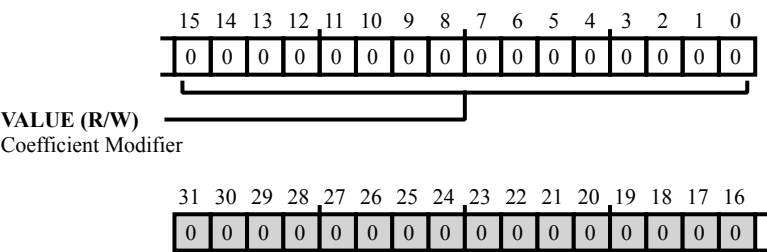


Figure 35-9: IIR_COEFMOD Register Diagram

Table 35-12: IIR_COEFMOD Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	VALUE	Coefficient Modifier. The <code>IIR_COEFMOD.VALUE</code> bit field provides the coefficient modifier.

Global Control Register

The `IIR_CTL1` register is used to configure the global parameters for the accelerator. These parameters include the number of channels, channel auto iterate, DMA enable, and accelerator enable.

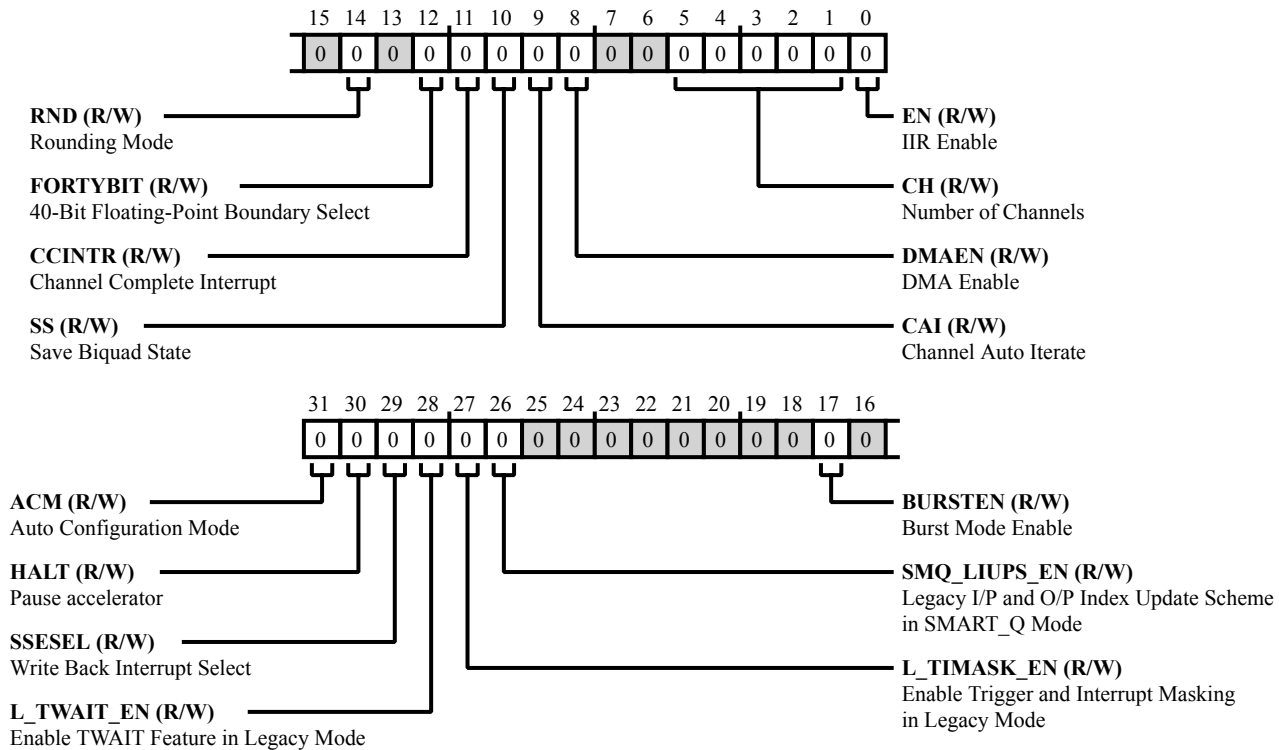


Figure 35-10: IIR_CTL1 Register Diagram

Table 35-13: IIR_CTL1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	ACM	Auto Configuration Mode. The <code>IIR_CTL1.ACM</code> bit configures the mode for loading the TCB.
		0 Legacy Mode
		1 Auto Configuration Mode
30 (R/W)	HALT	Pause accelerator. The <code>IIR_CTL1.HALT</code> bit determines whether the accelerator pauses so that the core can check the status and modify or submit a job or the accelerator is released for further processing of data. This bit is only valid in Auto Configuration Mode (ACM).
		0 Release accelerator
		1 Pause accelerator

Table 35-13: IIR_CTL1 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
29 (R/W)	SSESEL	Write Back Interrupt Select. The IIR_CTL1 . SSESEL bit selects whether to generate an interrupt after the write back or after save state completion.
		0 Generate Interrupt after save state completion
		1 Generate interrupt after write back completion
28 (R/W)	L_TWAIT_EN	Enable TWAIT Feature in Legacy Mode. The IIR_CTL1 . L_TWAIT_EN bit indicates whether the TWAIT feature is available in legacy mode. This bit is only valid in legacy mode.
		0 Not Available
		1 Available
27 (R/W)	L_TIMASK_EN	Enable Trigger and Interrupt Masking in Legacy Mode. The IIR_CTL1 . L_TIMASK_EN bit enables trigger and interrupt masking in legacy mode. This bit is only valid in legacy mode.
		0 Disable
		1 Enable
26 (R/W)	SMQ_LIUPS_EN	Legacy I/P and O/P Index Update Scheme in SMART_Q Mode. The IIR_CTL1 . SMQ_LIUPS_EN bit configures the scheme the accelerator uses to update the input index (II) and output index (OI). This bit is only valid in ACM.
		0 Update the II with all '0' and the OI with all 'F', respectively
		1 Update II and OI after circular buffer scheme
17 (R/W)	BURSTEN	Burst Mode Enable. The IIR_CTL1 . BURSTEN bit field is set, burst access is enabled.
		0 Disable burst mode
		1 Enable burst mode
14 (R/W)	RND	Rounding Mode. The IIR_CTL1 . RND bit field selects the rounding mode for floating-point format.
		0 Round to nearest (even)
		1 Truncate (Round towards zero)

Table 35-13: IIR_CTL1 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
12 (R/W)	FORTYBIT	40-Bit Floating-Point Boundary Select. The IIR_CTL1.FORTYBIT bit selects between 32-bit IEEE floating-point format or 40-bit IEEE floating-point format.
		0 32-bit IEEE floating-point
		1 40-bit IEEE floating-point
11 (R/W)	CCINTR	Channel Complete Interrupt. The IIR_CTL1.CCINTR bit configures the channel complete interrupt to generate when all channels are done or after each channel is done. This bit is only valid in legacy mode when the IIR_CTL1.L_TIMASK_EN bit is cleared (=0). In Auto Configuration Mode (ACM) or when the IIR_CTL1.L_TIMASK_EN is set (=1), interrupt generation for each channel is controlled using IIR_CTL2.IMASK bit.
		0 Interrupt is generated only when all channels are done (default)
		1 Interrupt is generated after each channel is done (default)
10 (R/W)	SS	Save Biquad State. The IIR_CTL1.SS bit configures the accelerator to store the Dk register settings into the internal memory. This can be used to save the biquad states before switching to another high priority accelerator task. This bit is only valid in legacy mode. In Auto Configuration Mode (ACM), the save state for each channel is controlled using the IIR_SGCTL.SS bit
9 (R/W)	CAI	Channel Auto Iterate. The IIR_CTL1.CAI bit sets whether TDM processing stops (idle) once all channels complete processing or moves to first channel and continues TDM processing in a loop when all channels complete processing. Channel Auto Iterate is not available in Auto Configuration Mode (ACM). The accelerator keeps processing the TCBs until the chain pointer becomes null in ACM.
		0 TDM processing stops (idle) once all channels complete processing
		1 Moves to first channel and continues TDM processing in a loop when all channels complete processing
8 (R/W)	DMAEN	DMA Enable. The IIR_CTL1.DMAEN bit enables DMA on the accelerator.
		0 Disable
		1 Enable

Table 35-13: IIR_CTL1 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration				
5:1 (R/W)	CH	Number of Channels. The <code>IIR_CTL1.CH</code> bit field configures the number of channels and is programmable between 0-31 (channels = NCH + 1). This bit field is only valid in legacy mode. In Auto Configuration Mode (ACM), there is no limit on the number of channels. the accelerator keeps processing the TCBs until the chain pointer becomes null.				
0 (R/W)	EN	IIR Enable. The <code>IIR_CTL1.EN</code> bit enables or disables the IIR accelerator. <table><tr><td>0</td><td>IIR disabled</td></tr><tr><td>1</td><td>IIR enabled</td></tr></table>	0	IIR disabled	1	IIR enabled
0	IIR disabled					
1	IIR enabled					

Channel Control Register

The `IIR_CTL2` register is used to configure the channel specific parameters. These parameters include the number of biquads and window size. In Auto Configuration Mode (ACM), this register is also used to configure additional channel specific parameters like interrupts and triggers.

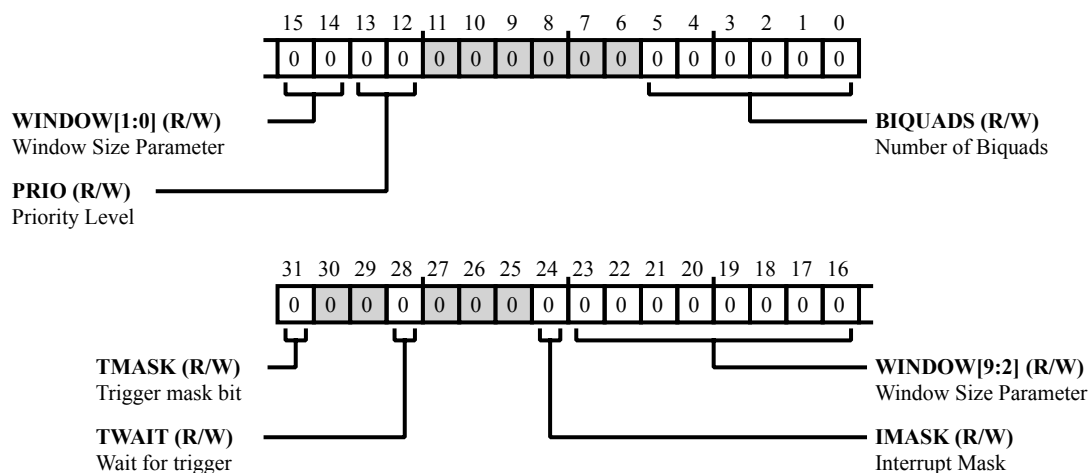


Figure 35-11: IIR_CTL2 Register Diagram

Table 35-14: IIR_CTL2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	TMASK	Trigger mask bit. The <code>IIR_CTL2</code> . <code>TMASK</code> bit enables trigger generation for the channel. It is valid in Auto Configuration Mode (ACM) or if <code>CTL1.L_TIMASK_EN</code> is set.
		0 Enable
		1 Mask
28 (R/W)	TWAIT	Wait for trigger. The <code>IIR_CTL2</code> . <code>TWAIT</code> bit disables the wait for the trigger. It is valid in Auto Configuration Mode (ACM) or if <code>CTL1.L_TIMASK_EN</code> is set.
		0 Disable wait for the trigger for the channel
		1 Enable wait for external trigger input assertion for the channel
24 (R/W)	IMASK	Interrupt Mask. The <code>IIR_CTL2</code> . <code>IMASK</code> bit enables interrupt generation for the channel. This bit is valid in Auto Configuration Mode (ACM) or if <code>CTL1.L_TIMASK_EN</code> is set.
		0 Enable
		1 Mask

Table 35-14: IIR_CTL2 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
23:14 (R/W)	WINDOW	Window Size Parameter. The IIR_CTL2 . WINDOW bit field configures the window size which specifies the number of samples/blocks to process (sample based processing = window size of 1). This bit field should be programmed to "actual window size required -1". For example, for sample based processing this bit field should be programmed to 0.
13:12 (R/W)	PRI0	Priority Level. The IIR_CTL2 . PRI0 bit field indicates the priority.
		0 Level 0 (lowest)
		1 Level 1
		2 Level 2
		3 Level 3 (highest)
5:0 (R/W)	BIQUADS	Number of Biquads. The IIR_CTL2 . BIQUADS bit field configures the number of biquads and is programmable between 0-63 (number of Biquads = BIQUADS + 1).

IIR Debug Address Register

The `IIR_DBG_ADDR` register holds the debug address. If bit 11 is set, coefficient memory is selected.

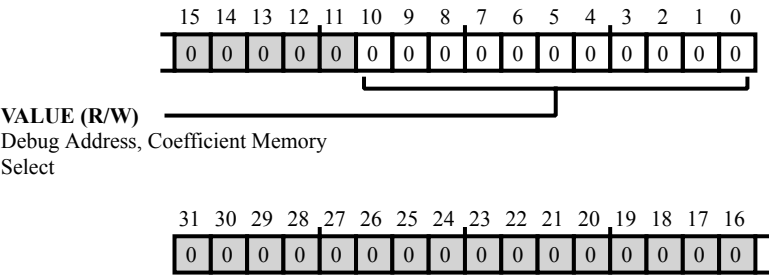


Figure 35-12: IIR_DBG_ADDR Register Diagram

Table 35-15: IIR_DBG_ADDR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
10:0 (R/W)	VALUE	Debug Address, Coefficient Memory Select. The <code>IIR_DBG_ADDR.VALUE</code> bit field holds the debug address (bits 0-10). Bit 11 configures whether the memory access is to coefficient memory (=0) or to delay line memory (=1).

IIR Debug Control Register

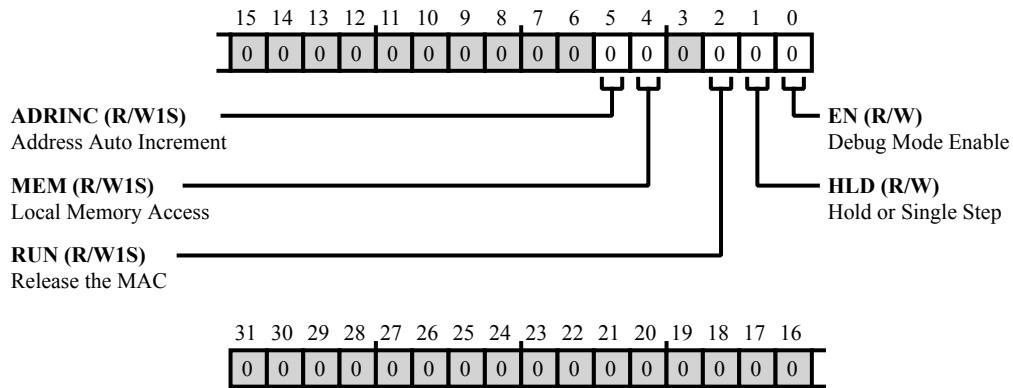


Figure 35-13: IIR_DBG_CTL Register Diagram

Table 35-16: IIR_DBG_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
5 (R/W1S)	ADRINC	Address Auto Increment. The <code>IIR_DBG_CTL.ADRINC</code> bit allows the address register to auto increment on <code>IIR_DBG_WRDAT_HI/IIR_DBG_WRDAT_LO</code> writes and <code>IIR_DBG_RDDAT_HI/IIR_DBG_RDDAT_LO</code> reads.
4 (R/W1S)	MEM	Local Memory Access. The <code>IIR_DBG_CTL.MEM</code> bit allows the data and coefficients memory to be indirectly accessed.
2 (R/W1S)	RUN	Release the MAC. The <code>IIR_DBG_CTL.RUN</code> bit releases the MAC; it is self clearing after one IIR clock cycle.
1 (R/W)	HLD	Hold or Single Step. The <code>IIR_DBG_CTL.HLD</code> bit function is based on the <code>IIR_DBG_CTL.MEM</code> bit setting. For <code>IIR_DBG_CTL.MEM = 0</code> this bit sets single step. For <code>IIR_DBG_CTL.MEM = 1</code> this bit sets hold data.
		0 No effect
		1 Single step (<code>IIR_DBGMEM=0</code>) or Hold data (<code>IIR_DBGMEM=1</code>)
0 (R/W)	EN	Debug Mode Enable. The <code>IIR_DBG_CTL.EN</code> bit enables debug mode. For local memory access, the <code>IIR_CTL1</code> register can be cleared.
		0 Disable
		1 Enable

IIR Debug Read Data High Register

The `IIR_DBG_RDDAT_HI` register is part of the 40-bit wide debug mode read data register and holds the upper 8 bits.

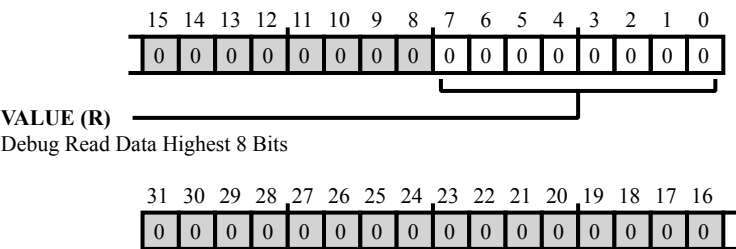


Figure 35-14: IIR_DBG_RDDAT_HI Register Diagram

Table 35-17: IIR_DBG_RDDAT_HI Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	VALUE	Debug Read Data Highest 8 Bits. The <code>IIR_DBG_RDDAT_HI.VALUE</code> bit field holds the upper 8-bit read data.

IIR Debug Read Data Low Register

The `IIR_DBG_RDDAT_LO` register is part of the 40-bit wide debug mode read data register and holds the lower 32 bits.

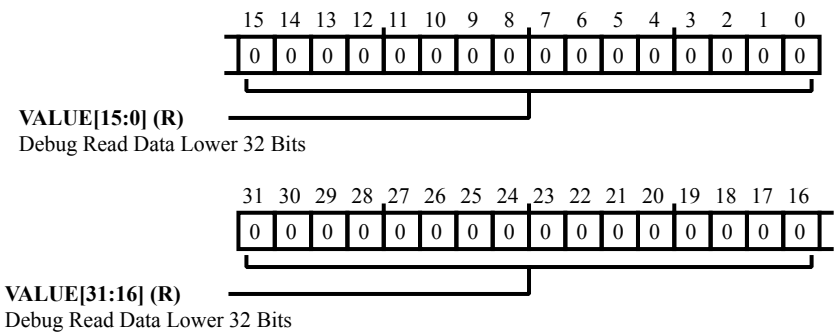


Figure 35-15: IIR_DBG_RDDAT_LO Register Diagram

Table 35-18: IIR_DBG_RDDAT_LO Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	Debug Read Data Lower 32 Bits. The <code>IIR_DBG_RDDAT_LO.VALUE</code> bit field holds the lower 32-bit read data.

IIR Debug Write Data High Register

The `IIR_DBG_WRDAT_HI` register is part of the 40-bit wide debug mode write data register and holds the upper 8 bits.

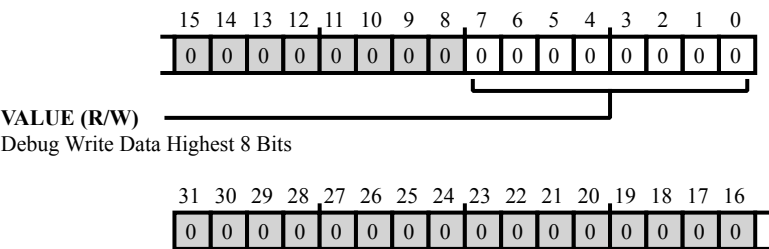


Figure 35-16: IIR_DBG_WRDAT_HI Register Diagram

Table 35-19: IIR_DBG_WRDAT_HI Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/W)	VALUE	Debug Write Data Highest 8 Bits. The <code>IIR_DBG_WRDAT_HI.VALUE</code> bit field holds the upper 8-bit write data.

IIR Debug Write Data Low Register

The `IIR_DBG_WRDAT_LO` register is part of the 40-bit wide debug mode write data register and holds the lower 32 bits.

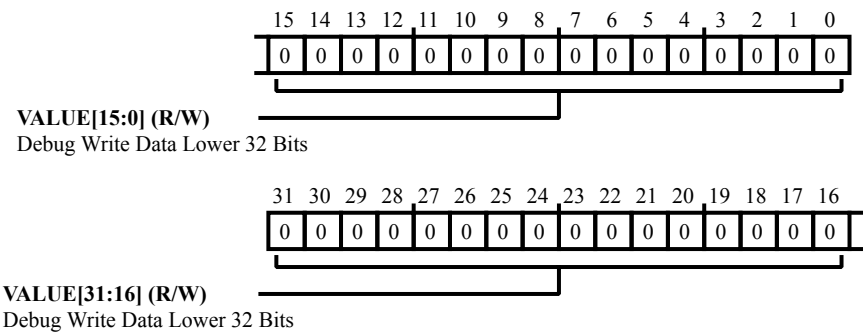


Figure 35-17: IIR_DBG_WRDAT_LO Register Diagram

Table 35-20: IIR_DBG_WRDAT_LO Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Debug Write Data Lower 32 Bits. The <code>IIR_DBG_WRDAT_LO.VALUE</code> bit field holds the lower 32-bit write data.

DMA Status Register

The `IIR_DMASTAT` registers indicate the status of DMA operations.

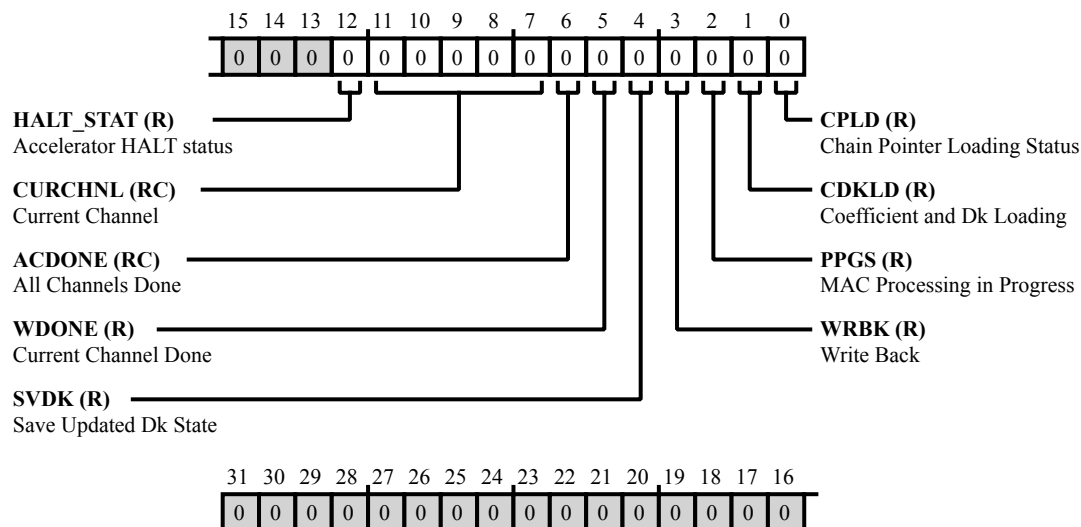


Figure 35-18: IIR_DMASTAT Register Diagram

Table 35-21: IIR_DMASTAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
12 (R/NW)	HALT_STAT	Accelerator HALT status. Acknowledge to core that accelerator is in halt state, if set.
11:7 (RC/NW)	CURCHNL	Current Channel. The <code>IIR_DMASTAT.CURCHNL</code> bit field indicates the channel that is being processed in the TDM slot. Zero indicates the last slot.
6 (RC/NW)	ACDONE	All Channels Done. The <code>IIR_DMASTAT.ACDONE</code> bit indicates all channels are done processing. Note that the <code>IIR_CTL1.CCINTR</code> bit does not affect this status bit. This bit is sticky and is cleared on register read.
5 (R/NW)	WDONE	Current Channel Done. The <code>IIR_DMASTAT.WDONE</code> bit indicates the processing of the current channel is complete. Note that the <code>IIR_CTL1.CCINTR</code> bit does not affect this status bit. This bit is sticky and is cleared on register read.
4 (R/NW)	SVDK	Save Updated Dk State. If there is more than one channel (<code>IIR_CTL1.CH>0</code>), the <code>IIR_DMASTAT.SVDK</code> bit toggles between 0 and 1 as it starts and completes the save state operation on one channel at a time.

Table 35-21: IIR_DMASTAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
3 (R/NW)	WRBK	Write Back. The IIR_DMASTAT.WRBK bit indicates the accelerator is writing back updated index registers.
2 (R/NW)	PPGS	MAC Processing in Progress. The IIR_DMASTAT.PPGS bit indicates MAC processing is in progress.
1 (R/NW)	CDKLD	Coefficient and Dk Loading. The IIR_DMASTAT.CDKLD bit indicates the coefficient and Dk are loading.
0 (R/NW)	CPLD	Chain Pointer Loading Status. The IIR_DMASTAT.CPLD bit indicates the IIR is in the chain pointer load state.
		0 State machine not in chain pointer load state
		1 State machine in chain pointer load state

Input Buffer Base Register

The `IIR_INBASE` register contains the word address with the lower 2 bits removed.

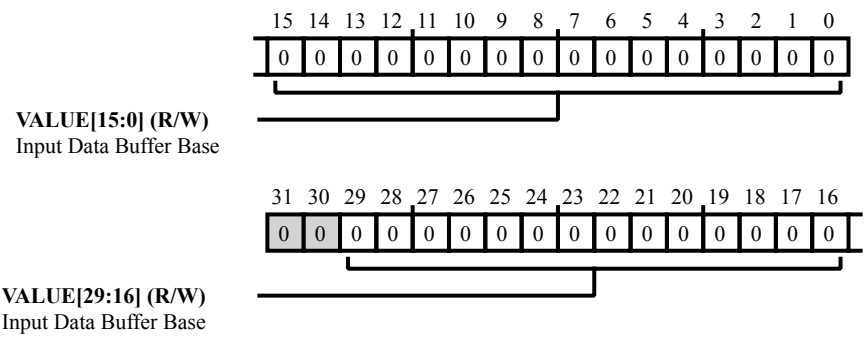


Figure 35-19: IIR_INBASE Register Diagram

Table 35-22: IIR_INBASE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:0 (R/W)	VALUE	Input Data Buffer Base. The <code>IIR_INBASE.VALUE</code> bit field value is the input data buffer base address.

Input Data Index Register

The `IIR_INIDX` register contains a word address with the lower 2 bits removed.

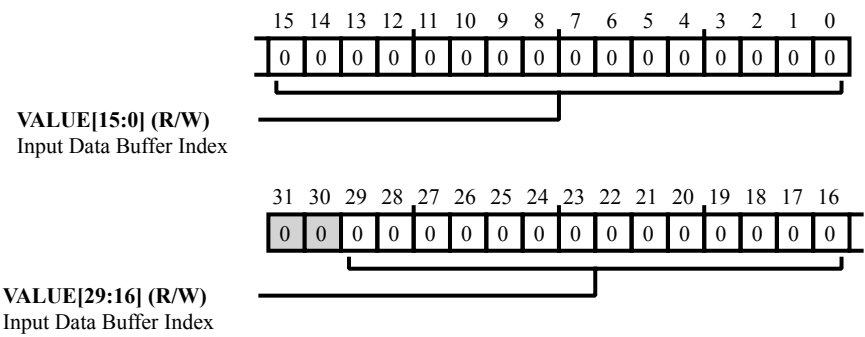


Figure 35-20: IIR_INIDX Register Diagram

Table 35-23: IIR_INIDX Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:0 (R/W)	VALUE	Input Data Buffer Index. The <code>IIR_INIDX.VALUE</code> bit field value is the input data buffer index.

Input Data Buffer Length Register

The `IIR_INLEN` register provides the input data buffer length.

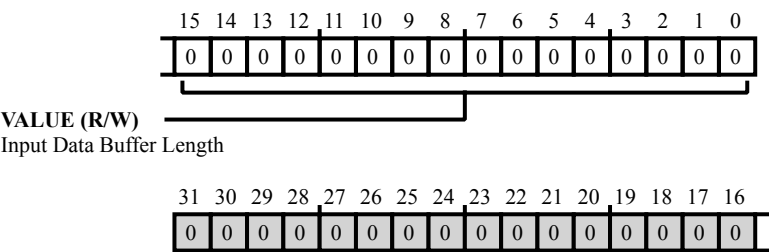


Figure 35-21: IIR_INLEN Register Diagram

Table 35-24: IIR_INLEN Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	VALUE	Input Data Buffer Length. The <code>IIR_INLEN.VALUE</code> bit field value is the input data buffer length.

Input Data Index Modifier Register

The `IIR_INMOD` register provides the 16-bit input data buffer index modifier.

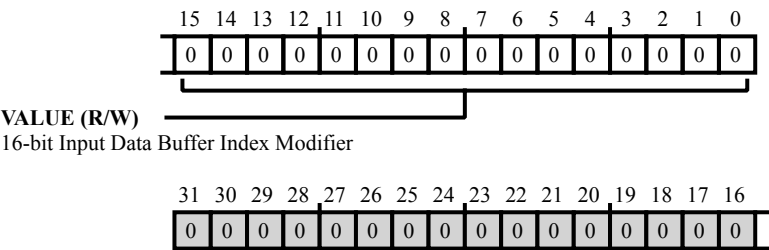


Figure 35-22: IIR_INMOD Register Diagram

Table 35-25: IIR_INMOD Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	VALUE	16-bit Input Data Buffer Index Modifier. The <code>IIR_INMOD.VALUE</code> bit field value is the 16-bit input data buffer modifier.

MAC Status Register

The `IIR_MACSTAT` register indicates the status of MAC operations.

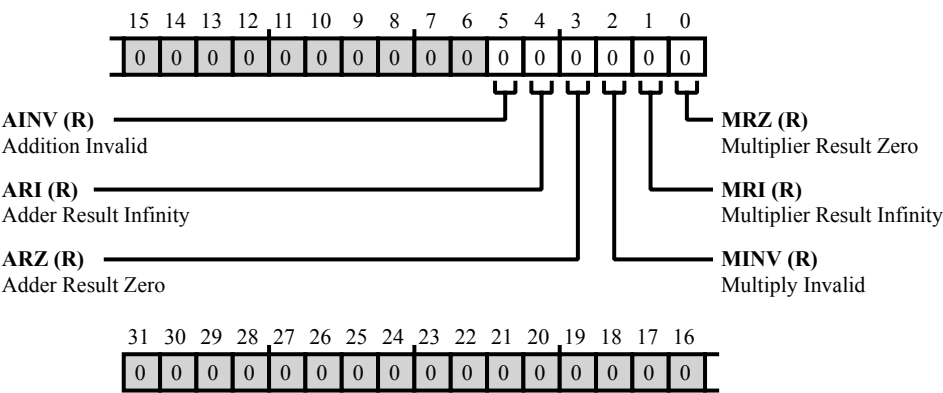


Figure 35-23: IIR_MACSTAT Register Diagram

Table 35-26: IIR_MACSTAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
5 (R/NW)	AINV	Addition Invalid. The <code>IIR_MACSTAT.AINV</code> bit indicates the addition is invalid.
4 (R/NW)	ARI	Adder Result Infinity. The <code>IIR_MACSTAT.ARI</code> bit indicates the adder result is infinity.
3 (R/NW)	ARZ	Adder Result Zero. The <code>IIR_MACSTAT.ARZ</code> bit indicates the adder result is zero.
2 (R/NW)	MINV	Multiply Invalid. The <code>IIR_MACSTAT.MINV</code> bit indicates the multiply operation is invalid.
1 (R/NW)	MRI	Multiplier Result Infinity. The <code>IIR_MACSTAT.MRI</code> bit indicates the multiplier result is infinity.
0 (R/NW)	MRZ	Multiplier Result Zero. The <code>IIR_MACSTAT.MRZ</code> bit indicates the multiplier result is zero.

Output Buffer Base Register

The `IIR_OUTBASE` register contains the word address with the lower 2 bits removed.

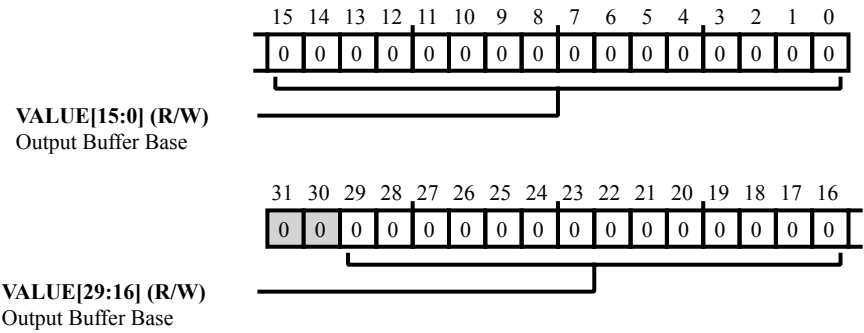


Figure 35-24: IIR_OUTBASE Register Diagram

Table 35-27: IIR_OUTBASE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:0 (R/W)	VALUE	Output Buffer Base. The <code>IIR_OUTBASE.VALUE</code> bit field provides the output buffer base address.

Output Data Buffer Index Register

The `IIR_OUTIDX` register is written with word address without the lower 2 bits.

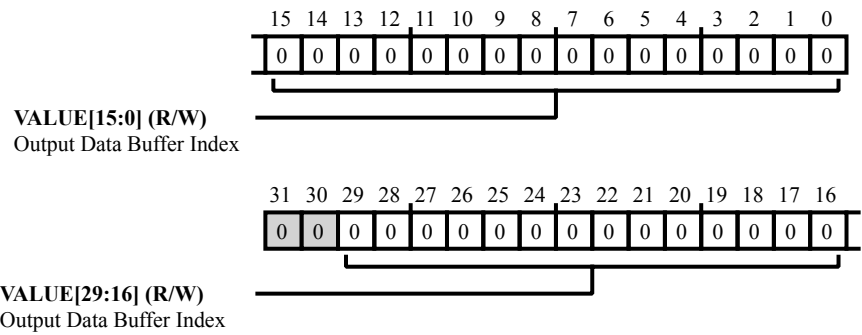


Figure 35-25: IIR_OUTIDX Register Diagram

Table 35-28: IIR_OUTIDX Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:0 (R/W)	VALUE	Output Data Buffer Index. The <code>IIR_OUTIDX.VALUE</code> bit field provides the output data buffer index.

IIR Output Data Buffer Length Register

The `IIR_OUTLEN` register provides the output data buffer length.

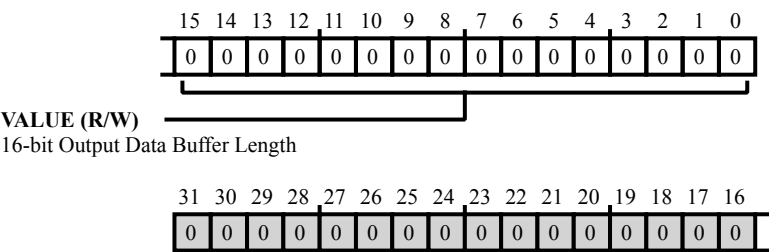


Figure 35-26: IIR_OUTLEN Register Diagram

Table 35-29: IIR_OUTLEN Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	VALUE	16-bit Output Data Buffer Length. The <code>IIR_OUTLEN.VALUE</code> bit field provides the output data buffer length.

IIR Output Data Index Modifier Register

The `IIR_OUTMOD` register provides the output data index modifier.

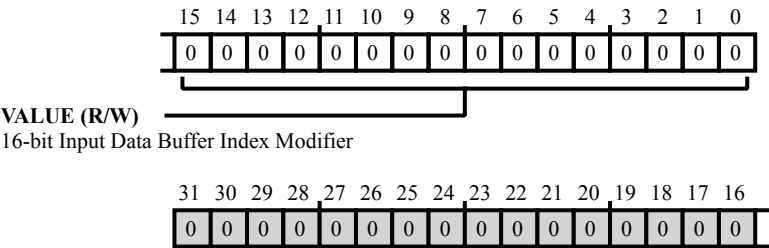


Figure 35-27: IIR_OUTMOD Register Diagram

Table 35-30: IIR_OUTMOD Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	VALUE	16-bit Input Data Buffer Index Modifier. The <code>IIR_OUTMOD.VALUE</code> bit field provides the output data buffer index modifier.

Software Control Register1

The `IIR_SCTL1` register is used by Smart Queuing APIs in ACM mode to exchange relevant job information amongst APIs.

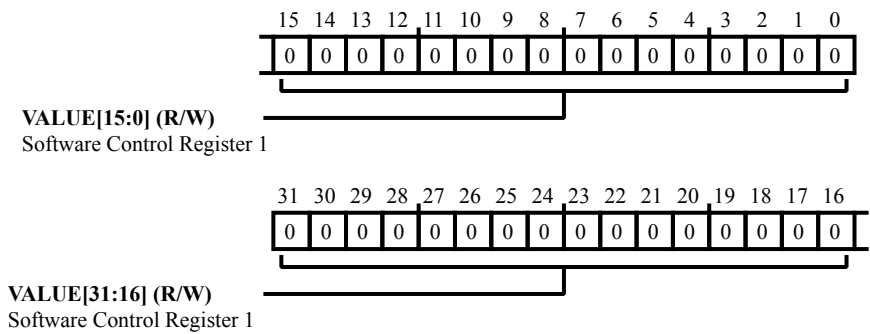


Figure 35-28: IIR_SCTL1 Register Diagram

Table 35-31: IIR_SCTL1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Software Control Register 1.

Software Control Register2

The `IIR_SCTL2` register is used by Smart Queuing APIs in ACM mode to exchange relevant job information amongst APIs.

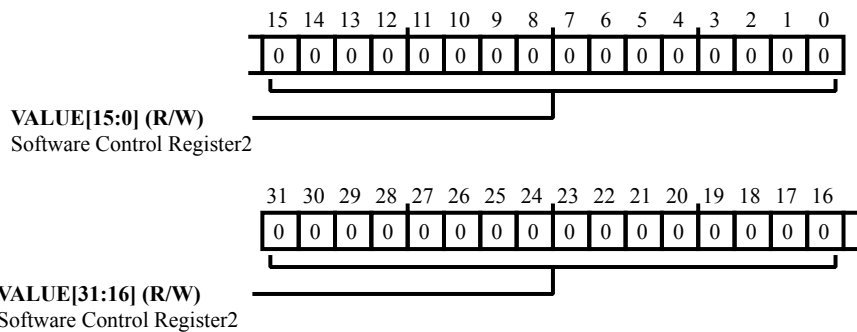


Figure 35-29: IIR_SCTL2 Register Diagram

Table 35-32: IIR_SCTL2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Software Control Register2.

Secondary Global Control Register

The `IIR_SGCTL` register configures the global parameters for the accelerator in ACM mode for loading CTL1 register as part of TCB.

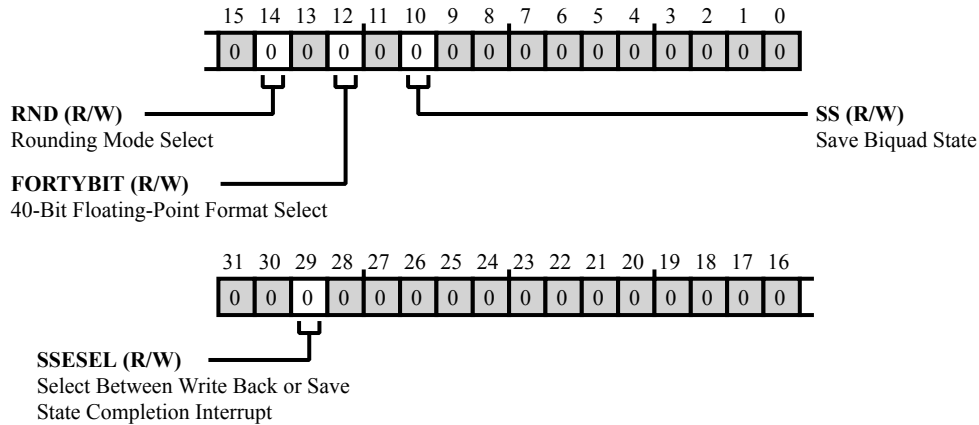


Figure 35-30: IIR_SGCTL Register Diagram

Table 35-33: IIR_SGCTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29 (R/W)	SSESEL	Select Between Write Back or Save State Completion Interrupt.
14 (R/W)	RND	Rounding Mode Select. The <code>IIR_SGCTL.RND</code> bit selects the rounding mode for a floating-point mode compatible with SHARC+.
		0 Round to nearest (even)
		1 Truncate (Round towards zero)
12 (R/W)	FORTYBIT	40-Bit Floating-Point Format Select. The <code>IIR_SGCTL.FORTYBIT</code> bit selects the floating point format.
		0 32-bit floating point
		1 40-bit floating point
10 (R/W)	SS	Save Biquad State. The <code>IIR_SGCTL.SS</code> bit configures the accelerator to store the Dk register settings into the internal memory. This can be used to save the biquad states before switching to another high priority accelerator task.

36 Boot ROM and Booting the Processor

Bootstrapping or booting is the series of events that occur when the system applies power to the processor or when the processor enters a hardware reset state. This section gives an in-depth description of these events and how to integrate an application effectively.

On reset, the processor begins fetching instruction from an internal ROM. The boot code contained within the ROM is designed to facilitate loading an application. The boot code can automatically initialize certain peripherals for communication based on a chosen boot mode, then load an application. For more information on what boot modes are available, see the [Boot Modes](#) section. The boot code can efficiently load an entire application, code, and data, into appropriate locations after the development tools repack the application into a boot stream.

A boot stream is an application or data that the boot-loader tool splits into blocks. A 16-byte header in each block provides instruction to the boot code for processing the associated data. The processor can perform several boot functions, depending on the flags set in the header. For more details on what options are available and a description of the stream format, refer to the [Boot Loader Stream](#) section.

The boot ROM provides a mechanism through available non-volatile programmable memory (OTP on this processor) to customize different aspects of the boot process. These customizations include: overriding default boot-peripheral instance, overriding default peripheral-timing parameters and disabling boot modes.

NOTE: In this chapter the term boot ROM describes the boot code that cannot be altered. The terms *program* and *application* are used to describe code that is used to customize the boot process.

Several utilities of the boot code are also available to the application. These utilities include features such as copying memory, comparing memory, or loading another boot stream at run time. The APIs may be used to help ensure that application code is more compatible with future products. The boot code also provides the ability to define a custom boot mode. This capability helps when support is not available for a desired boot mode. It allows second stage boot loaders for unsupported boot peripherals to leverage a significant amount of the existing boot ROM functionality.

SRAM Requirements

The boot process reserves 8K bytes of L2 SRAM (parity protected) for dedicated use. This topic describes how the reserved memory region is used during boot.

The boot process requires SRAM resources for stack use and to store various data items that require read/write access during the boot process. 8K bytes of L2 SRAM (parity protected) is reserved for this purpose. The *Boot Process SRAM Requirements* table describes the various items stored in this memory region.

Table 36-1: Boot Process SRAM Requirements

Address	Size (Bytes)	Item	Description
0x200FE000	4	Reserved	
0x200FE004	4	Pointer to the struct ADI_ROM_BOOT_CONFIG object	Pointer to the boot configuration structure that is located on the stack. This location is used to find the location of the boot structure on the stack for debug.
0x200FE008	8	Reserved	
0x200FE010	1024	Internal Intermediate Buffer 0	The first of two internal buffers used for intermediate storage of boot content when using indirect and page mode accesses and for secure boot operations. Two buffers are used to allow SHA-224 and AES-128 operations to be performed on one buffer while simultaneously loading the other buffer.
0x200FE410	1024	Internal Intermediate Buffer 1	The second of two internal buffers used for intermediate storage of boot content when using indirect and page mode accesses and for secure boot operations. Two buffers are used to allow SHA-224 and AES-128 operations to be performed on one buffer while simultaneously loading the other buffer.
0x200FE810	16	struct ADI_ROM_BOOT_HEADER object	Storage location for all the block headers of the boot stream.
0x200FE820	2708	Storage for Secure Boot related descriptors	Contains a number of buffers for the various descriptors used by the Cryptographic Accelerators and provides storage for the secure header of a secure boot stream.
0x200FF2B4	204	Storage for boot benchmarking data	Contains buffers for storing boot benchmarking data and other fields related to boot benchmarking.
0x200FF400	3072	Stack for the boot process	The primary booting cores stack. The processor core should locate the stack in this region in order to preserve security in secure boot operations.

NOTE: To preserve the security of the product, the 8K byte region described here is not a bootable region of memory. If the boot process determines that a block of data in the boot stream is targeted towards this memory region, the boot process terminates and enters either the default error handler or, if applicable, a user-defined error handler. This reserved memory region is free for use after the boot process completes. In order to preserve security when using the boot API to boot a secure boot stream, the stack used during the execution of the boot API must be located at the default location in this reserved 8K bytes region of memory.

Preboot Operations

Preboot is responsible for configuration of all system resources prior to executing the required boot operation.

The steps performed by the preboot process are described here in the order of execution. Numerous stages of the preboot process are conditional based upon the content of the [RCU_BCODE](#) register.

NOTE: When a power on reset, hard reset, and software triggered system reset event completes, the processor operates in PLL bypass mode (default). Partway through the preboot sequence the processor is brought into full-on mode at the default settings unless the program has provisioned custom CGU settings in the OTP.

Start-up Sequence

This section describes the initial start-up sequence of all cores in the processor.

When the power-on reset, hardware reset or system reset event completes, the core is released from reset and is responsible for managing the boot process. The following sections describe the sequence of events that occur for the core.

Core Reset Sequencing

When a system and hardware reset event occurs, the processor state is reset and the boot sequence is executed. The core is initially released from reset and starts execution of the preboot software from the boot ROM.

The initial sequence of events after the completion of the reset sequence are shown below.

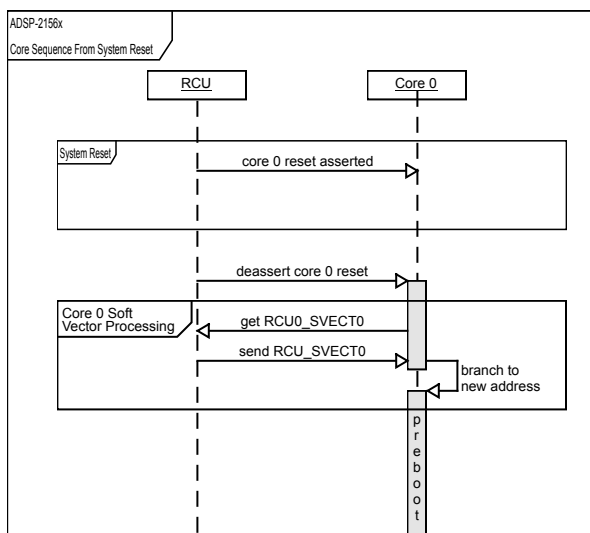


Figure 36-1: Core Reset Sequencing

Core Start-up

Describes the initial operations performed by the SHARC+ core immediately after being released from reset and soft vector processing has completed.

After the core is initially released from the reset event, the soft vectoring process starts the boot process as long as the [RCU_SVECT0](#) register contained the default reset value during the soft vector processing. The core performs the following initial operations:

- Clear `RCU_MSG.COIDLE`
- Check for OTPC Boot completion.
- Disable Software Return feature in the BTB.
- Disable the BTB.
- Disable cache via `MODE2` register.
- Flush the cache.
- Set `IRPTL` to `0x00000000`.
- Configure primary and secondary DAG configurations to setup the C run-time environment.
- Clear the `CBUFEN`, `SRD1H`, `SRD1L`, `SRD2H`, `SRD2L`, `ALUSAT`, `TRUNCATE` bits in the `MODE1` register.
- Set the `IRPTEN`, `IPERREN`, `DPERREN`, `SPERREN` bits in the `MODE1` register.
- Set `MMASK` to `0x00000000`.
- Set `IMASK` to `0x00000000`.
- Initialize the top 1024 bytes of the stack to initialize the corresponding parity to allow for read access to the initial stack space. This step is bypassed if `RCU_BCODE.NOMEMINIT` is set.

Table 36-2: Primary/Secondary DAG Configuration for C Run-Time Setup

Primary/Secondary DAG Register	Value	Description
M7, M15	-1	Dedicated registers must always be set to -1
M6, M14	1	Dedicated registers must always be set to 1
M5, M13	0	Dedicated registers must always be set to 0
L0-L5	0	Preserved registers set initially to 0
L6, L7	0	Stack Length Register set initially to 0
L8-L15	0	Preserved registers set initially to 0
B6, B7	0x200FF400	Stack Base Registers
I7	0x200FFFC	Stack Pointer
L6, L7	0x000002FF	Stack Length Registers
I6	0x200FFFC	Frame Pointer

PLL Bypass

The PLL is brought out of bypass early in the start-up to run the system in higher speed. This operation results in a faster pre-boot execution time.

Boot Benchmark

Based on the OTP configuration, the boot benchmark feature is enabled to capture different time stamps through GP timer during different stage of boot process.

Fault Configuration

Describes the initial fault sources that are enabled allowing the processor to signal a fault to the system.

The following faults are enabled via the SEC. Only the faults are enabled, the boot process does not install any SEC interrupts.

Table 36-3: Initial Faults installed during Preboot

SEC Fault ID	SEC Fault Name	Description
0	INTR_SEC0_ERR	SEC Error
3	INTR_WDOG0_EXP	WDOG Expire
4	INTR_WDOG1_EXP	WDOG Expire
5	INTR_OTPC0_ERR	OTPC Expire
9	INTR_L2CTL0_ECC_ERR	L2 ECC Error
32	INTR_SOFT3_INT	Software Driven Interrupt 3. This is raised in the boot ROM error handler should it be entered.
138	INTR_CRC0_ERR	CRC Error
139	INTR_CRC1_ERR	CRC Error
170	INTR_SYS_MDMA0_SRC_ERR	MDMA 0 Source DMA Channel Error
171	INTR_SYS_MDMA0_DST_ERR	MDMA 0 Destination DMA Channel Error
172	INTR_SYS_MDMA1_SRC_ERR	MDMA 1 Source DMA Channel Error
173	INTR_SYS_MDMA1_DST_ERR	MDMA 1 Destination DMA Channel Error
178	INTR_L2CTL1_ECC_ERR	L2CTL1 ECC Error

The SEC is configured for a fault delay of 0x100 via the SEC_FDLY.COUNT and SEC_FSRDLY.COUNT bit fields. This allows for a delay before the fault assertion is a custom error handler is installed and any SEC interrupts are enabled and handled for more advanced second stage boot scenarios.

The SYSFAULT pins are configured via the SEC to support both incoming and outgoing faults by enabling the SEC_FCTL.FIEN and SEC_FCTL.FOEN bits. The boot process captures an incoming fault if that fault is asserted by the external system on handover to the user application after boot.

NOTE: The installation of the fault sources is enabled by default and may be optionally bypassed using the RCU_BCODE.NOFAULTS bit.

L2 Controller Configuration

The L2 controller is configured to ensure all L2 memory banks are initialized.

NOTE: The L2 Controller configuration is enabled by default and may be optionally bypassed by setting the `RCU_BCODE.NOL2CONFIG` bit.

L2 Memory Initialization

The L2 memory subsystem is parity protected by default. The L2 memory must be initialized to ensure that no read from the L2 memory generates a parity error. All L2 memory banks are initialized using the `L2CTL_INIT` bit. The boot process waits for each memory bank to signal initialization complete using the `L2CTL_ISTAT` bit. The `RCU_MSG.L2INIT` is set when L2 memory initialization is complete.

NOTE: The L2 memory initialization process is enabled by default and may be optionally bypassed by setting the `RCU_BCODE.NOMEMINIT` bit.

Idle On Entry

The Idle On Entry feature instructs the debugger to halt the boot code before continuing with any further preboot operations.

When this feature is enabled the processor executes a WFI/IDLE instruction and then continues once an event (such as an emulator exception) is serviced.

NOTE: Idle On Entry processing is disabled by default and is enabled by setting the `RCU_BCODE.IDLEONENTRY` bit prior to performing a system reset operation.

SPU Configuration

The SPU is configured differently depending upon the detected security state of the device. The first operation clears any existing security violations that may be indicated in the `SPU_STAT` register.

None of the peripherals are configured to ignore security signals and only the following requesters are configured to generate secure transactions.

Table 36-4: SPU Secure Requesters during Boot

SPU Endpoint ID	Requester Name
110	MDMA0 Source DMA Channel
111	MDMA0 Destination DMA Channel
112	MDMA1 Source DMA Channel
113	MDMA1 Destination DMA channel
108	CRC0
109	CRC1
119	PKTE

SMPU Configuration

The SMPU is used to restrict access to various memory regions in the processor. The configuration applied during boot differs depending on the locked state of the processor.

By default the SMPU instances only allow secure read and write transactions. The *SMPU Configuration* table describes the configurations for the different security states.

Speculative reads are also disabled by setting the `SMPU_CTL.RSDIS` bit.

Table 36-5: SMPU Configuration

SMPU Instance	SMPU Instance	Open <code>SMPU_SECURECTL</code> Value	Locked <code>SMPU_SECURECTL</code> Value
SMC	0	<code>SMPU_SECURECTL.WNSEN</code> <code>SMPU_SECURECTL.RNSEN</code>	0x00000000 (Default Reset Value)
Core_L2_RAM_ROM	2	<code>SMPU_SECURECTL.WNSEN</code> <code>SMPU_SECURECTL.RNSEN</code>	0x00000000 (Default Reset Value)
DMA_L2_RAM_ROM	3	<code>SMPU_SECURECTL.WNSEN</code> <code>SMPU_SECURECTL.RNSEN</code>	0x00000000 (Default Reset Value)
L2CTL1	9	<code>SMPU_SECURECTL.WNSEN</code> <code>SMPU_SECURECTL.RNSEN</code>	0x00000000 (Default Reset Value)
SPI	11	<code>SMPU_SECURECTL.WNSEN</code> <code>SMPU_SECURECTL.RNSEN</code>	0x00000000 (Default Reset Value)

Secure Debug Key Processing

In the event the processor is locked, the debug tools must submit a secure debug key that is matched with a key on the processor. A 256-bit secure debug key must be provided the application program prior to locking the device.

The secure debug key is read from the OTP memory and then written to the corresponding register in the TAPC. After the key has been written the `TAPC_SDBGKEY_CTL.VALID` bit is set. Once the debug tools then submit their key a key compare operation is performed.

Debug tools must wait for the boot software to load the key before setting the `TAPC_SDBGKEY_CTL.VALID` bit before submitting the key for comparison.

The 256-bit secure debug key is loaded from the storage area in OTP as shown in the *Secure Debug Key Load Procedure* table.

Table 36-6: Secure Debug Key Load Procedure

Secure Debug Key[255:0]	Register
Secure Debug Key[31:0]	<code>TAPC_SDBGKEY0</code>
Secure Debug Key[63:32]	<code>TAPC_SDBGKEY1</code>
Secure Debug Key[95:94]	<code>TAPC_SDBGKEY2</code>

Table 36-6: Secure Debug Key Load Procedure (Continued)

Secure Debug Key[255:0]	Register
Secure Debug Key[127:96]	TAPC_SDBGKEY3
Secure Debug Key[159:128]	TAPC_SDBGKEY4
Secure Debug Key[191:160]	TAPC_SDBGKEY5
Secure Debug Key[223:192]	TAPC_SDBGKEY6
Secure Debug Key[255:224]	TAPC_SDBGKEY7

The validity of the secure debug keys can be determined as shown in the *Secure Debug Key Validation* table.

Table 36-7: Secure Debug Key Validation

Bypass Key 0 (emu_key0_disable) Value	Bypass Key 1 (emu_key1_disable) Value	Valid Secure Debug Key
Zero	Zero	secure_emu_key0
Zero	Non-zero	secure_emu_key0
Non-zero	Zero	secure_emu_key1
Non-zero	Non-zero	Both keys are invalid

NOTE: In the ADSP-2156x processor family, the boot ROM supports two 256-bit secure debug keys, secure_emu_key0 and secure_emu_key1, where only one of them is used by the boot ROM at a time. Each of the secure debug keys is associated with a bypass key (emu_key0_disable or emu_key1_disable) which determines the validity of the corresponding key. Any non-zero value written into the 16-bit bypass key invalidates that particular secure debug key.

CAUTION: The boot code bypasses the key load operation entirely when a 16-bit non-zero value programmed on the emu_key0_disable and emu_key1_disable fields in OTP or a key of 0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF programmed in the secure debug key field provided in OTP results in the boot code bypassing the key load operation entirely. If debug access is then ever required the key must be loaded to the TAPC by the program. If the processor fails to boot (due to corrupted firmware) then there is no debug access. The only way to gain access is to load an authenticated boot image that can then load the required keys prior to attempting to connect with a debugger.

CGU Configuration

This step in the preboot process reconfigures the internal clocks on the processor for improved boot performance.

The boot process can optionally configure the CGU in order to improve boot performance. The settings for the CGU are located within the `ADI_ROM_OTP_BOOT_CGU_INFO` structure that has storage allocated in the OTP as part of the [struct ADI_ROM_OTP_BOOT_INFO](#) structure.

Typically, CGU configuration is performed using an [Init Block](#) in the boot stream. This provides greatest flexibility. In situations where boot time must be kept to a minimum, settings in the OTP can be applied at this stage of

preboot as opposed to during the boot process itself. When a processor is locked, the boot process does not support an [Init Block](#) in the boot stream. For a locked processor programs must use the OTP to reconfigure the clocks without adopting a multi-stage boot strategy.

On release from reset, the CGU is configured for PLL Bypass mode. To improve boot performance, the boot software reconfigures the CGU so that Full-On mode is entered with the default CGU settings.

The CGU configuration object (see struct `ADI_ROM_OTP_BOOT_CGU_INFO`) allows the boot software to configure the CGU for a more efficient boot process.

Table 36-8: ADI_ROM_OTP_BOOT_CGU_INFO Members

Type	Name	Description
uint32_t	ctl_WEN:1 (bitfield)	Enable write to the CGU_CTL register
uint32_t	div_WEN:1 (bitfield)	Enable write to the CGU_DIV register
uint32_t	Reserved0:1 (bitfield)	Reserved
uint32_t	div_DSEL:5 (bitfield)	CGU_DIV.DSEL value
uint32_t	div_CSEL:5 (bitfield)	CGU_DIV.CSEL value
uint32_t	div_S0SEL:3 (bitfield)	CGU_DIV.S0SEL value
uint32_t	div_SYSSSEL:5 (bitfield)	CGU_DIV.SYSSSEL value
uint32_t	div_S1SEL:3 (bitfield)	CGU_DIV.S1SEL value
uint32_t	div_OSEL:7 (bitfield)	CGU_DIV.OSEL value
uint32_t	reserved1:1 (bitfield)	Reserved
uint32_t	ctl_MSEL:7 (bitfield)	CGU_CTL.MSEL value
uint32_t	reserved2:1 (bitfield)	Reserved
uint32_t	ctl_S1SELEXEN:1 (bitfield)	CGU_CTL.S1SELEXEN value
uint32_t	auto_disable:1 (bitfield)	disable polling on auto-alignment of clocks (not recommended)
uint32_t	clkoutsel_CLKOUTSEL:5 (bitfield)	CGU_CLKOUTSEL.CLKOUTSEL value
uint32_t	clkoutsel_WEN:1 (bitfield)	Enable write to the CGU_CLKOUTSEL register
uint32_t	Reserved3:8 (bitfield)	Reserved
uint32_t	divex_S1SELEX:8 (bitfield)	CGU_DIVEX.S1SELEX value

If a `CGU_STAT.WDIVERR`, `CGU_STAT.WDFMSERR`, `CGU_STAT.LWERR` or `CGU_STAT.ADDRERR` occurs at the entry to or completion of the configuration routine, the default error handler is called and the boot process terminates.

NOTE: Programs can bypass CGU configuration by setting the `RCU_BCODE.NOPREBOOT` bit when this part of the boot process is reached.

L1 Memory Initialization

The processor initializes all parity protected memories to perform subsequent read operations without generating a parity error.

The *L1 Memory Initialization* table describes the methods used to initialize the various parity supported memories on the processor.

Table 36-9: L1 Memory Initialization

Resource To Fill Memory	Memory Type	Address	Count	Fill Value	Flag Set Upon Completion
Core 0	Core 0 L1 Bank 0	0x00048000	0x6000 (LW)	0x00000000	RCU_MSG.COL1INIT
Core 0	Core 0 L1 Bank 1	0x00058000	0x6000 (LW)	0x00000000	
Core 0	Core 0 L1 Bank 2	0x00060000	0x4000 (LW)	0x00000000	
Core 0	Core 0 L1 Bank 3	0x00070000	0x4000 (LW)	0x00000000	

NOTE: The memory initialization process is enabled by default and may be optionally bypassed by setting the `RCU_BCODE.NOMEMINIT` bit.

Default Entry Point

The first instruction executed for the core in the boot ROM is a read of the `RCU_SVECT0` register. The boot ROM then vectors to that location.

The *Default Entry Point* table defines the default entry point for core in the processor

Table 36-10: Default Entry Point

Core ID	Register	Entry Point
0	<code>RCU_SVECT0</code>	0x00500004

The boot code does not set any initial default application entry points by writing the entry point to the `RCU_SVECT0` register in the processor. It will hold the default values of the power on reset case. The `RCU_SVECT0` register is updated at the end of the boot process. For the secure boot, the register is updated only after the successful authentication of the boot stream.

NO-BOOT Processing

No-Boot mode is executed when selected by the `SYS_BMODE[n]` pins. The boot mode is intended as a recovery boot mode or for debug purposes. The core simply executes in an endless loop in the boot ROM, terminating further execution of the boot process.

This boot mode is primarily intended for debug sessions when no boot source may be configured. It allows for a debugger to safely connect to the device and take control (assuming the debugger has been granted access rights as defined by the processor security implementation).

NOTE: NO-BOOT processing is usually only entered as a result of the boot mode pin sampling resulting in execution of the No-Boot boot mode. This processing can also be optionally enabled by setting `RCU_BCODE.HALT` field. The `RCU_BCODE.HALT` setting can be especially useful for debug sessions to force the execution of the NO-BOOT mode regardless of the `SYS_BMODE[n]` state allowing a user application to be loaded via the debug tools without fear of the image being corrupted as a result of attempting to boot through another source.

SYS_RESOUT Signal

In order to signal to the external system that the processor is in a configured state and ready to start the boot process, the boot software de-asserts the `SYS_RESOUT` pin via the `RCU_CTL.RSTOUTDSRT` bit.

Bypassing the Boot Process

The boot process can be bypassed allowing a program to start execution from the address stored in the core's soft vector register. This is useful when working in emulation sessions as it provides a mechanism to be able to execute programs directly from an accessible memory that already contains executable code.

To bypass the boot process, set the `RCU_BCODE.NOKERNEL` bit. The processor core vectors to the address stored in the core's corresponding `RCU_SVECT0` register instead of calling the required boot mode.

This feature is used along with other features such as disabling memory initialization.

Boot Mode Disable

Specific boot modes can be permanently disabled via OTP. If the disabled boot mode is enabled using the `SYS_BMODE[n]` pins, a boot error is generated.

A byte of storage is provided in the OTP to disable up to eight boot modes.

Table 36-11: Boot Mode Disable

<code>otp_data::bootModeDisable</code> Bit Position	Corresponding Boot Mode
0	SPI2 Controller Boot Mode
1	SPI Target Boot Mode
2	UART Target Boot Mode
3	Reserved
4	xSPI Controller Boot Mode
5	SPI1 Controller Boot Mode
7–6	Reserved

Boot Command Customization

Boot command customization allows permanent customization of a particular boot mode. For example, it is possible to change the peripheral instance used for boot operation.

Storage is provided in OTP for a command item for each supported boot mode.

Storage is provided in the `ADI_ROM_OTP_BOOT_CMD_INFO` member of `struct ADI_ROM_OTP_BOOT_INFO`. Refer to the corresponding boot modes boot command description for details on supported command options.

NOTE: Before programming boot command to the OTP, evaluate the boot command using the `adi_rom_Boot()` API and ensure that the boot command provides the desired functionality. Once the command is programmed to OTP, it cannot be reverted to original default settings.

Boot Mode Specific SPU Configuration

Prior to performing actual boot process, the processors SPU resources specific to the boot mode selected are configured. This is performed in the preboot phase as opposed to within the boot mode itself when calling the boot API, as it isolates the security functionality of the processor allowing it to be handled specifically by a separate process.

The following additional SPU resources are configured as secure controllers according to the boot mode selected.

Table 36-12: Boot Mode Specific SPU Configuration

Boot Mode	SPU Endpoint ID	Controller Name
SPI Controller Boot (Memory-Mapped Mode) and xSPI Controller Boot	110, 111	MDMA0 Source DMA Channel, MDMA0 Destination DMA Channel
SPI Controller Boot (Peripheral Mode)	78, 76, 74	SPI2 Receive DMA, SPI1 Receive DMA, SPI0 Receive DMA
SPI Target Boot	78, 76, 74	SPI2 Receive DMA, SPI1 Receive DMA, SPI0 Receive DMA
UART Target Boot	65, 67	UART 0 Receive DMA, UART1 Receive DMA

NOTE: In the *Boot Mode Specific SPU Configuration* table, for a given boot mode, all the SPU resources are not configured. Only a single peripheral instance is enabled according the peripheral instance selected for boot. For example if the boot command for the boot mode indicates boot from UART0 only the UART0 receive DMA is configured, the other UART receive DMAs are not configured for secure access.

Executing the Boot Mode

The boot mode is called `adi_rom_Boot()` routine. The routine fetches and processes the boot stream from the configured boot source.

The table provides default parameters passed to each of the supported boot modes. For details on the API usage, refer to `adi_rom_Boot()`.

Table 36-13: Default Boot ROM API Parameters

Boot Mode	pAddress	flags	blockCount	pHook	Command
No Boot	0x00000000	0x00000000	0x00000000	Points to an empty routine in ROM	0x00000000
SPI2 Controller Boot	0x60000000	0x00040000	0x00000000	Points to an empty routine in ROM	0x0001287
SPI Target Boot	0x00000000	0x00000000	0x00000000	Points to an empty routine in ROM	0x00000212
UART Target Boot	0x00000000	0x00000000	0x00000000	Points to an empty routine in ROM	0x00000013
SPI1 Controller Boot	0x70000000	0x00040000	0x00000000	Points to an empty routine in ROM	0x0001187
xSPI Controller Boot	0x60000000	0x00040000	0x00000000	Points to an empty routine in ROM	0x0000000A
Reserved	0x00000000	0x00000000	0x00000000	Points to an empty routine in ROM	0x00000000

NOTE: For SPI 1/2 controller boot and xSPI controller boot modes, start address (pAddress) is set to 0x60000000 or 0x70000000. This corresponds to the first memory location in the respective flash device (0x0).

The hook function installed via pHook on this product does not perform any additional configuration.

Benchmarking Boot Time

Boot time benchmarking feature can be used to measure how much time it takes for each stage in boot ROM to execute. This can be helpful in doing quick boot time measurements.

Boot ROM internally uses GP timer6 to count the cycles for boot ROM execution. The timer runs directly on SYS_CLKIN until the CGU is brought out of bypass by boot ROM during preboot phase. After the CGU is out of bypass mode, the timer runs on the configure SCLK0 frequency. The boot ROM reads the cycles count (32-bit) from the timer block and logs it into the benchmark buffer (at the start address of buffer (0x200ff2c8); the next time stamp will be captured logged on 0x200ff2cc, and so on

Boot time benchmarking can be enabled optionally through OTP using the ROM_OTP_bcfg_benchmark_En field. This feature is disabled by default.

Benchmarking options include enabling through OTP, through ROM API, and a combination of the two. (For normal boot mode, this time stamp is recorded and can be treated as redundant.

Option 1: Enable though OTP

When ROM_OTP_bcfg_benchmark_En is set in OTP, boot benchmarking is enabled for the power-up boot. The boot ROM captures the benchmark time stamp at the following events in the boot ROM:

- After L2 memory initialization is complete

- After calibration is complete
- After the CGU module is taken out of the bypass state
- Reserved time stamp for 2156x products
- After SHARC0 L1 memory initialization is complete
- Before xSPI SRAM configuration
- After xSPI SRAM configuration
- At the start of the main kernel process
- Before calling the Init function
- Before the start of data fetch (from flash memory or host)
- After the data fetch is complete (from flash memory or host)
- After authentication is complete (for secure boot). (For normal boot, this time stamp is recorded and can be treated as redundant).

Option 2: Disabled in OTP, Enabled through ROM API

Boot benchmarking can be enabled through the `pBenchmark_enable_user` field in `ROM_TIMER_BENCHMARK_INFO` using the ROM API and hook function (when cause is `ROM_HOOK_REG_COMPLETE`). The operation can be done without enabling this feature through OTP.

In this case, the following time stamps are captured:

- After the hook routine call (cause == `ROM_HOOK_REG_COMPLETE`).
- Before the start of data fetch (from flash memory or host)
- After the data fetch (from flash memory or host)
- After authentication is complete (for secure boot)

Option 3: Enabled through OTP, Reenable through ROM API

If benchmarking is already enabled in OTP (as indicated by the `pBenchmark_enable_otp` field), the timer may have expired by this time. (The timer can expire when the ROM API call happens after a long time). If the `pBenchmark_enable_user` field is set, the timer restarts from 0 and the index is reset to the first location in the time stamp buffer.

Time stamp logs will be similar to option 2.

After capturing all time stamps, timer6 is stopped and cleared. The timer is also be stopped in case of any error in boot processing.

Watchdog Timeout

Typically, the error handler processes most of the errors detected. However, there may be situations where the error is not detected and the execution is stuck inside Boot ROM, or, the core crashes without any errors being detected. For example, in target boot mode, the host does not send any data; the ROM waits inside the DMA routine for the data to be sent. The watchdog timer can be used to detect these types of errors.

The watchdog timer is a 32-bit counter that runs on SCLK0. For the maximum SCLK0 rate (for example, 125 MHz), the maximum timeout that can be configured is approximately 34 seconds ($(2^{32})/(125) \text{ us} \sim 34.359$). By default, boot code enables the watchdog in preboot with the OTP not programmed. The default timeout value is configured to the maximum (for example, 0xFFFFFFFF). This feature can be disabled through OTP using the `ROM_OTP_bcfg_wdtDis` field. When set, boot code does not configure the watchdog. The `ROM_OTP_watchdog_count` field in OTP can be configured to overwrite the default watchdog timeout count value. If this location is programmed and the watchdog is not disabled through OTP, boot code configures the watchdog timeout count value with the one programmed in OTP.

The watchdog is disabled in boot code at the following locations :

- Just before jumping to the application after the boot process is complete
- After entering the error handler
- After detecting `bmode0` and before jumping to the forever loop

Timeouts that occur after the watchdog is enabled are indicated externally by raising a fault through the FAULT pin. There is no reset issued to the system in the case of timeout.

Boot Modes

The boot implementation provides built-in support for booting from various peripherals.

The *Boot Modes* table describes the supported boot modes.

In target boot mode, the processor functions as a target to any host device. In these modes, the host device usually controls the processor `SYS_HWRST` input. Typically, the host applies the reset sequence and waits until the processor is ready to boot, depending on the peripheral in use, and transmits the boot stream data to the processor. Handshake signals are used to signal to the host that the processor is ready to accept more data.

In controller boot mode, the processor controls the peripheral and requests data via the peripheral as and when required.

Individual boot modes can be disabled. For more information about disabling boot modes, see [Boot ROM OTP Customizations](#).

Table 36-14: Booting Modes

SYS_BMODE[2:0]	Boot Source	Description
000	No Boot	The processor does not boot. Rather the boot kernel executes some of the preboot operations then enters an endless <code>WFI/IDLE</code> state.
001	SPI Controller Boot (SPI2)	Boot from integrated Flash memory through the SPI2 peripheral configured for memory-mapped mode.
010	SPI Target Boot	Boot through the SPI2 peripheral configured as a target.
011	UART Boot	Boots through UART0 configured as a target receiver.
101	xSPI Controller Boot	Boot from integrated Flash memory through the xSPI peripheral configured for memory-mapped mode.
110	SPI Controller Boot (SPI1)	Boot from integrated Flash memory through the SPI1 peripheral configured for memory-mapped mode.
100, 111	Reserved	

No-Boot Mode

No-Boot mode is intended for device recovery purposes caused due to incorrect programming of the boot source memory allowing for target connection through an emulator. Emulation tools can also leverage the No-Boot functionality allowing for debug sessions to run the preboot software prior to loading an application while preventing the boot process from continuing and clobbering data loaded by the emulator.

This boot mode results in several preboot operations being performed before placing the core in a safe, endless loop located in the boot ROM.

For a complete list of operations performed when No-Boot is selected, see [Preboot Operations](#). The core terminates at the [NO-BOOT Processing](#) stage of the preboot process.

SPI Controller Boot Mode

The SPI controller boot routine provides support for booting the processor from SPI flash memories. The SPI boot mode uses a device auto-detection feature that is enabled by default. This lets the boot stream itself instruct updates to the SPI configuration and the read command used allowing for more efficient transactions.

NOTE: For default SPI controller mode (BMODE1(SPI2) and BMODE6(SPI1)), the peripheral fault, SPI2_ERR with fault ID 72 and SPI1_ERR with fault ID 92, respectively, are enabled.

Boot From External SPI Flash Devices

The SPI boot mode supports booting from 24-bit or 32-bit addressable flash devices. The boot mode uses the MDMA channels by default and configures the SPI flash for memory-mapped functionality. Peripheral DMA mode is also supported when calling the boot mode via `adi_rom_Boot()`.

When auto-device detection is enabled, the SPI memory is initially read using the standard 0x03 SPI read command with a reduced clocking frequency for maximum compatibility. The first nibble of the boot stream is then used to reconfigure the SPI interface and possible the SPI flash. Refer to [SPI Device Detection Routine](#).

NOTE: Support for automatic device detection via the first nibble of the boot stream is not supported when booting secure boot streams. Instead when signing the boot image an attribute can be set in the image header that specifies the configuration to use.

For booting, the SPI memory is connected as shown in the *SPI Memory Connections* figure.

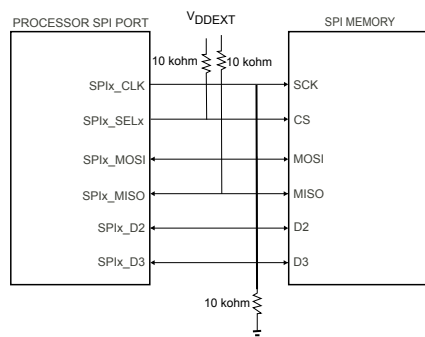


Figure 36-2: SPI Memory Connections

The pull-up resistor on the target select signal ensures that the memory is deselected when the pin is in a high-impedance mode such as during reset.

Initialization codes are allowed to manipulate the `ADI_ROM_BOOT_CONFIG::dBootCommand` to extend boot mechanism to a second SPI memory connected to another target select pin. Updating the field that specifies the target select signal for use, allows the boot process to manage larger boot streams than fit into a single SPI device.

NOTE: If modifying the target select signal used during the boot process, configure the pin multiplexing to enable the correct functionality for the pin. Once the boot process has proceeded past the configuration function and the boot process has actually started, the boot kernel will not perform any further pin multiplexing operations.

NOTE: Power on SPI controller boot is supported by two BMODE in the ADSP-21560/1/4 processor family: BMODE1 and BMODE6. For BMODE 1, the SPI2 instance of the SPI peripheral is used for boot. For BMODE 6, the SPI1 instance of the SPI peripheral is used for boot.

SPI Device Detection Routine

Since the boot mode supports booting from various SPI memories, the boot kernel automatically detects what type of memory is connected. To determine whether the SPI memory device requires a 24-bit or 32-bit addressing scheme, the boot kernel performs a device detection sequence prior to booting. The `SPI_MISO` signal requires a pull-up resistor. The routine relies on the fact that memories do not drive their data outputs unless the right number of address bytes are received.

Initially, the boot kernel transmits the read command on the `SPI_MOSI` line. Once the command has been sent, the boot kernel proceeds to transmit a single address byte and waits until the receive FIFO indicates that the buffer is no longer empty. The first received byte is discarded. The boot code then proceeds to issue another address byte while simultaneously receiving a byte. The process continues until a non 0xFF or 0x00 byte is received or until the full 4 address bytes is sent without any valid data being returned.

The receiving of a non 0x00 or 0xFF byte tells the boot code whether the memory device requires 24 or 32 address bits. The lower nibble of the received byte is then used to further customize the boot mode. This nibble is referred to as the BCODE. The boot code applies settings to the SPI peripheral according to the *SPI Controller Boot BCODE Descriptions*.

If the received value equals 0x00 or 0xFF, it is assumed that the memory device has not driven its data output thus, another zero byte is transmitted and the received data is tested again.

If the value still equals 0xFF, device detection continues. Device detection aborts immediately when a byte different than 0xFF is received. The boot process continues with normal boot operation and it reissues a command to again read from address 0. Two read sequences load the first block header. Separate read sequences load further block headers and block payload fields.

The *SPI Device Detection Principle* figure illustrates how individual devices behave.

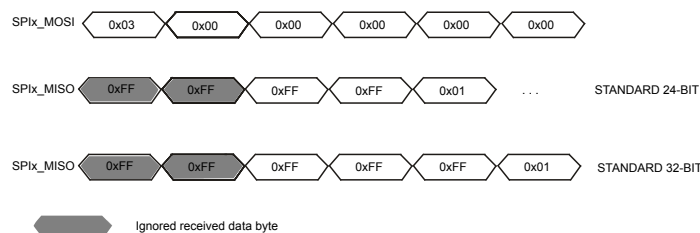


Figure 36-3: SPI Device Detection Principle

Table 36-15: SPI Controller BCODE Configuration Lookup Table

Member	BCODE				
	0	1	2	3	4
Transfer Type	Single bit command, address and data	Single bit command, address and data	Single bit command, address and data	Single bit command, address and dual bit data	Single bit command, address and Quad data
ubDummyBytes	0x00	0x00	0x01	0x01	0x01
ubReadCommand	0x03	0x03	0x0B	0x3B	0x6B
ubDataBits	0x00	0x00	0x00	0x01	0x02
ubAddressBytes	0x03	0x03	0x03	0x03	0x03
uwClkLower	0x000F	0x000F	0x0001	0x0001	0x0001
uReserved0	0x0000	0x0000	0x0000	0x0000	0x0000
pMIOEnFunction	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
nDummy	0x00	0x00	0x00	0x00	0x00

Table 36-16: SPI Controller BCODE Configuration Lookup Table

Member	BCODE					
	5	6	7	8	9	A
Transfer Type	Single bit command, Dual bit address and data	Single bit command, Dual bit address and data	Single bit command, Dual bit address and data	Single bit command, Quad bit address and data	Single bit command, Quad bit address and data	Single bit command, Quad bit address and data
ubDummyBytes	0x01	0x02	0x03	0x02	0x03	0x05
ubReadCommand	0xBB	0xBB	0xBB	0xEB	0xEB	0xEB
ubDataBits	0x01	0x01	0x01	0x02	0x02	0x02
ubAddressBytes	0x03	0x03	0x03	0x03	0x03	0x03
uwClkLower	0x0001	0x0001	0x0001	0x0001	0x0001	0x0001
uReserved0	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000
pMIOEnFunction	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
nDummy	0x00	0x00	0x00	0x00	0x00	0x00

NOTE: For the above configurations, the addressing scheme can be 3-bytes or 4-bytes depending on the addressing of flash detected in auto-detection. The SPI mode byte issued for all the SPI controller peripheral-based configurations is 0x00. The mode byte is the first byte transmitted after the address cycles and is used to control the continuous read mode functionality in which the next read operation is not required to issue a command cycle. Continuous read mode is not supported during the boot process.

Supported Quad Mode Enable Methods

The boot ROM does not support enabling quad mode on the SPI flash device. To boot in quad mode, the flash device must be configured outside the boot ROM.

NOTE: It is more beneficial to boot initially in dualmode and use an initcode to enable Quad mode.

Run-Time API

The following table provides descriptions of the [adi_rom_Boot\(\)](#) command parameter.

Table 36-17: SPI Controller Boot Command Descriptions

Bits	Name	Setting	Description
31:28	ROM_BCMD_SPIM_SPEED	0 to 1111	SPI clock divider. This value written to the SPI peripherals clock divider register
27	Reserved	Reserved	Reserved

Table 36-17: SPI Controller Boot Command Descriptions (Continued)

Bits	Name	Setting	Description
26:22	ROM_BCMD_SPIM_DUMMY	00000	No dummy bytes required after the address
		00001	1 dummy byte required after the address
			
		11111	31 dummy bytes required after the address
21:20	ROM_BCMD_SPIM_ADDR	00	Flash device requires an 8-bit address
		01	Flash device requires a 16-bit address
		10	Flash device requires a 24-bit address
		11	Flash device requires a 32-bit address
19:16	ROM_BCMD_SPIM_BCODE	0000 to 1111	Boot mode-specific code. Specifies the boot mode-specific code that can further customize and control the boot process.
15	ROM_BCMD_SPIM_CMD	0	Issue the read command over the single bit bus
		1	Issue the read command over the multi-bit bus. Not recommended for use in ADSP-2156x processor.
14:12	ROM_BCMD_SPIM_SSEL	000	Use target select 1 for SPI chip select
		001	Use target select 2 for SPI chip select
		010	Use target select 3 for SPI chip select
		011	Reserved
		100	Reserved
		101	Reserved
		110	Reserved
		111	Reserved

Table 36-17: SPI Controller Boot Command Descriptions (Continued)

Bits	Name	Setting	Description
11:8	ROM_BCMD_SPIM_DEVE- NUM	0 to 16	Boot peripheral enumeration. For SPI2, it is configured to 2. For SPI1, it is configured to 1. 0x3 to 0xF is reserved.
7	ROM_BCMD_SPIM_PAGE- MODE	0	Disable page mode
		1	Enable page mode
6	ROM_BCMD_SPIM_NOAU- TO	0	Automatic device detection disa- ble. Perform automatic device detection and peripheral config- uration based on the BCODE value (first nibble) of the boot streams block header
		1	Do not perform automatic de- vice detection
5	ROM_BCMD_SPIM_NOCFG	0	Device configuration enable. In- structs the config routine to per- form pinmuxing configuration and full peripheral configuration
		1	Device configuration disable.
4	ROM_BCMD_SPIM_HOST	0	Controller boot mode enable.
		1	Target boot mode enable.
3:0	ROM_BCMD_SPIM_DEVICE	0000	Reserved
		0001	Reserved
		0010	SPI Boot (Legacy peripheral DMA)
		0011	Reserved
		0100	Reserved
		0101	Reserved
		0110	Reserved
		0111	SPI memory mapped boot
		1000 to 1111	Reserved

NOTE: All bits in the above table that are not defined must be set to zero. Supported features may be limited depending on peripheral instance.

SPI controller secure boot is only supported in memory-mapped mode. SPI2 and SPI1 are the default SPI instances that operate in memory mapped mode. To boot with an SPI instance other than SPI2 and SPI1, ensure the desired SPI instance supports memory-mapped mode by configuring the

ROM_BCMD_SPIM_DEVICE field in the SPI controller boot command parameter to 0x7 in OTP or by passing it as an argument in the ROM API call.

SPI Target Boot Mode

When using SPI target mode boot, the processor consumes boot data from an external SPI host device. This mode supports single, dual, and quad-bit modes. The boot kernel always starts in single bit mode and can be changed using the appropriate command. The following figures show the hardware configuration for the modes. As in all target boot modes, the host device controls the SYS_HWRST input of the processor.

NOTE: Secure Boot Stream Padding

For target boot modes, the host must always send data in multiples of 1024 bytes. This requirement is due to the sizing of internal buffers used for DMA.

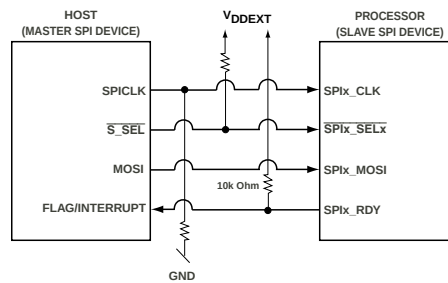


Figure 36-4: Connection between Host (SPI Controller) and Processor (SPI Target)

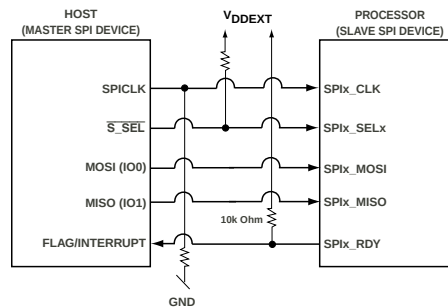


Figure 36-5: Connection between Host (SPI Controller) and Processor (SPI Target) DIOM

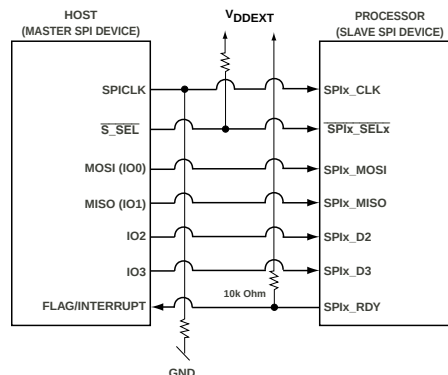


Figure 36-6: Connection between Host (SPI Controller) and Processor (SPI Target) QSPI

The host drives the SPI clock and is responsible for timing. The host must provide an active-low chip select signal that connects to the processor's `SPIx_SS` input signal with each byte transferred or remain low during the entire procedure. 8-bit data is expected and 16-bit mode is not supported.

In SPI target boot mode, the boot kernel sets the `SPI_CTL.CPHA` bit and clears the `SPI_CTL.CPOL` bit. Therefore the `SPI_MISO` pin is latched on the falling edge of the `SPI_MOSI` pin.

The SPI target processor detects the correct boot mode from the host SPI device by reading the first byte sent, defined as `SPICMD`. The *SPICMD Descriptions* table describes the available codes. These additional bytes must be sent prior to transmitting the data to configure the SPI device.

The *SPICMD Descriptions* table describes the following:

- Host starting in single bit mode
- Host starting in a mode other than single bit

Table 36-18: SPICMD Descriptions

SPICMD	Description
<i>If host starts in Single bit Mode</i>	
0x3	Keep single-bit mode
0x7	Switch to dual-bit mode
0xB	Switch to quad-bit mode
<i>If host device starts in DIOM or QSPI</i>	
0xAA,0xBF	Switch to dual-bit mode
0xEE,0xEE,0xFE,0xFF	Switch to quad-bit mode

In SPI target boot mode, `SPIx_RDY` functionality is critical. The `SPIx_RDY` output is used for back pressure and requires a pulling resistor. The boot code requires the `SPIx_RDY` signal function as active-low. The host is only permitted to transfer data when `SPIx_RDY` is in the active state. This functionality allows the processor to hold off the host while the processor is in reset or executing the pre-boot and processor initialization sequences. The SPI is configured to deassert `SPIx_RDY` when the receive FIFO is filled to 75% or more.

NOTE: For default SPI target mode, the peripheral fault, `SPI2_ERR` with Fault ID 72 is enabled.

The *SPI Program Flow on the Host Side* figure illustrates the required program flow on the host side.

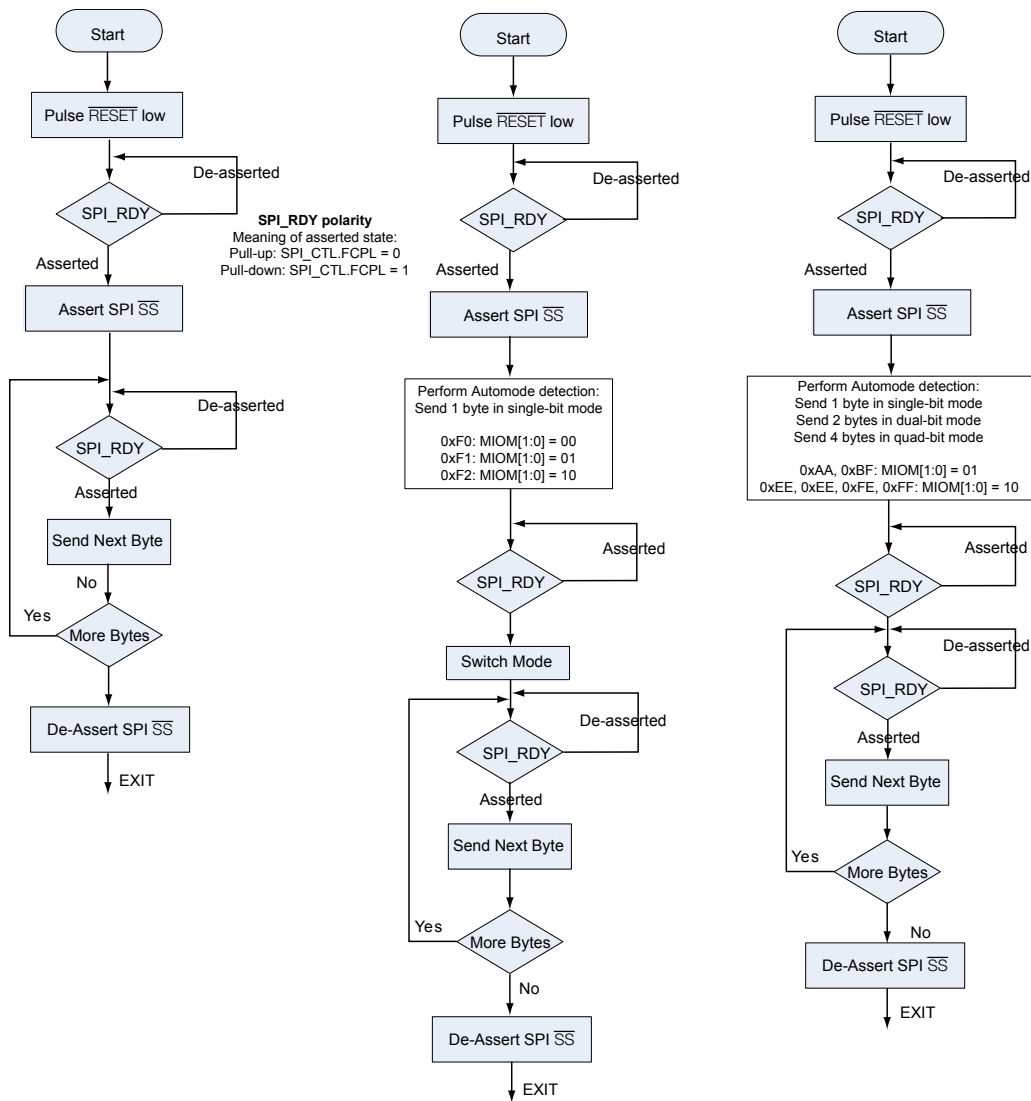


Figure 36-7: SPI Program Flow on the Host Side

Run-time API

The SPI target boot mode can be called through the Boot Routine API function at run time. Initiating a boot through the run-time API allows for extra customization such as disabling automatic device configuration or specifying a different SPI device other than the default.

When ROM_BCMD_NOCFG flag is specified, it is necessary to program pin multiplexing and other SPI configuration as required, while keeping the SPI_CTL.EN bit cleared.

The ROM_BCMD_NOAUTO flag can suppress auto mode detection. In that case, the desired configuration must be passed through the ROM_BCMD_SPIS_BCODE bit field, even if the ROM_BCMD_NOCFG flag is set.

The following table provides descriptions of the `adi_rom_Boot()` parameter.

Table 36-19: SPI Target Boot Command Bit Descriptions

Bit No. (Access)	Bit Name	Description/Enumeration	
19:16	ROM_BCMD_SPIS_BCODE	Boot Mode Specific BCODE. Specifies the boot mode-specific code that can further customize and control the boot process.	
		00xxb	Single bit SPI bus
		01xxb	Dual SPI bus
		10xxb	Quad SPI bus
		11xxb	Reserved
11:8	ROM_BCMD_DEVENUM	Device enumeration. Specifies the SPI device to use.	
		0x0	SPI0
		0x1	SPI1
		0x2	SPI2
		0x3 - 0xF	Reserved
6	ROM_BCMD_NOAUTO	Automatic device detection disable. When set, this bit disables automatic device detection and uses the setting provided in the other fields of this register to configure the boot mode.	
5	ROM_BCMD_NOCFG	Device configuration disable. When set, this bit disables device configuration. Device configuration includes reconfiguration of the peripherals MMR registers and device pin muxing.	
4	ROM_BCMD_HOST	Host boot mode enable. When set, enables SPI target boot mode. Otherwise, use the controller boot mode.	
3:0	ROM_BCMD_DEVICE	Boot source device. Specifies the device to boot from.	
		0x2	SPI

NOTE: All bits in the table that are not defined must be set to zero. Supported features may be limited depending on peripheral instance.

UART Target Boot Mode

When using UART target mode boot, the processor receives boot data from a UART host device connected to the UART interface. The device connected to UART0 is initially detected using an autobaud detection sequence. After finishing the UART target boot process, all control and status registers of the used resources are restored.

Further customization, such as disabling autobaud detection, and changing the device, use the boot routine API.

During boot operation, the host device usually relies on the RTS output of the UART device. At boot time, the processor does not evaluate RTS signals driven by the host. Since the RTS is in a high impedance state when the processor is in reset, or while executing a pre-boot, an external pull-up resistor to VDDEXT is recommended. The *Connection Between Host and Processor* figure shows the interconnection required for booting. The figure does not show physical line drivers and level shifters that are typically required to meet the individual UART-compatible standards.

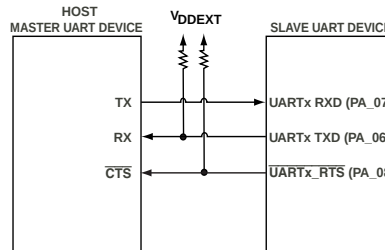


Figure 36-8: Connection Between Host and Processor

When the UART is enabled, the RTS immediately transitions low, encouraging the host to send the first boot stream data as shown in the *Host Relying on RTS* figure. For half-duplex UART connections, the host must avoid this action. The host must wait until it has received the 4 bytes from the target processor before sending any data.

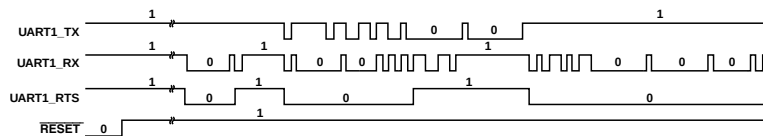


Figure 36-9: Host Relying on RTS

When the boot kernel is processing fill or Initcode blocks, it can require extra processing time and must delay the host from sending more data. This request is signaled using the RTS output.

The *Host Relying on RTS* figure shows RTS timing when an extended Initcode routine executes. Since code execution is distracting from the data loading, the host device must be prevented from sending more data. The timing of the RTS depends on the state of the UART_CTL.RFRT bit. This bit is cleared during UART target boot mode when RTS is de-asserted, the UART receive FIFO contains 4 or more data words, and another start bit is detected.

NOTE: Secure Boot Stream Padding

For target boot modes, the host must always send data in multiples of 1024 bytes. This requirement is due to the sizing of internal buffers used for DMA.

Autobaud Detection

The kernel supports autobaud detection using the '@' character as data. The host is expected to have its clock set to a rate supported in the UART.

To determine the bit rate when performing autobaud detection, use the following steps:

1. The boot kernel expects an '@' character (0x40, eight bits data, one start bit, one stop bit, no parity bit) on the UART RXD input.
2. The `UART_CLK.EDBO` and the `UART_CLK` register is cleared.
3. The boot kernel acknowledges, and the host then downloads the boot stream. The acknowledgment consists of 4 bytes: 0xBF, `UART_CLK` [15:8], `UART_CLK` [7:0], 0x00.
4. The host is requested to not send further bytes until it has received the complete acknowledge string.
5. Once the 0x00 byte is received, the host can send the entire boot stream.

The host knows the total byte count of the boot stream, but it is not required to know the content of the boot stream.

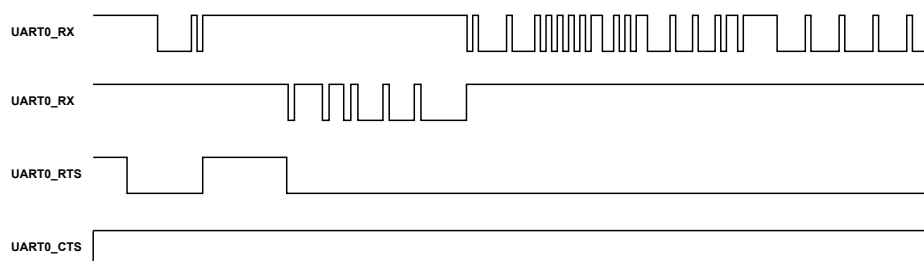


Figure 36-10: UART Autobaud Detection Waveform

The *UART Autobaud Detection Waveform* figure provides timing information for UART booting. After the bit rate is known, the UART is enabled and the kernel transmits the 4 acknowledge bytes.

NOTE: For default UART target mode, the peripheral DMA fault, `UART2_RXDMA_ERR` with Fault ID 163 is enabled.

Run-time API

The UART target boot mode can be called through the boot routine API function at run time. The run-time API allows for more customization. Both autobaud detection and device configuration can be disabled, and a device other than the default UART0 can be specified.

If `ROM_BCMD_NOCFG` flag is specified, it is the programs responsibility to configure pin multiplexing as required.

Autobaud detection can be suppressed using the `ROM_BCMD_NOAUTO` flag. In this case, the desired configuration can be passed through the `ROM_BCMD_UART_CLK` bit field. If the `ROM_BCMD_UART_CLK` bit field is zero, `UART_CLK` is evaluated. If a value of 0xFFFF was present, the default error routine of the boot kernel is called and the booting process is aborted. Otherwise, the value in `UART_CLK` remains untouched.

The following table provides descriptions of the `adi_rom_Boot()` parameter.

Table 36-20: UART Target Boot command Bit Descriptions

Bit No. (Access)	Bit Name	Description/Enumeration
31:16	ROM_BCMD_ UART_CLK	UART Clock Divider. When set to zero this field is ignored.
15	ROM_BCMD_ UART_EDBO	UART Clock Divider Mode When set enables EDBO functionality.
11:8	ROM_BCMD_ DEVENUM	Device enumeration. Specifies the UART device to use.
		0x0 UART0
		0x1 UART1
		0x2 UART2
		0x3 - 0xF Reserved
6	ROM_BCMD_ NOAUTO	Automatic device detection disable. When set disables automatic device detection and uses the setting provided in the other fields of this register to configure the boot mode.
5	ROM_BCMD_ NOCFG	Device configuration disable. When set, this bit disables device configuration. Device configuration includes reconfiguration of the peripherals MMR registers and device pin muxing.
4	ROM_BCMD_ HOST	Host boot mode enable. When set, enables SPI target boot. Otherwise, use the controller boot mode.
3:0	ROM_BCMD_ DEVICE	Boot source device. Specifies the device to boot from.
		0x3 UART

NOTE: All bits in the above table that are not defined must be set to zero. Supported features may be limited depending on peripheral instance.

xSPI Controller Boot Mode

The xSPI controller boot supports booting from the NOR flash devices and Hyperflash devices using the xSPI controller. Like the legacy SPI, it supports booting from different flash devices provided by flash vendors. This interface promotes high-speed booting with a reduced boot time because it supports a maximum operations of 116.6 MHz for the LQFP variant and 155.5MHz for the BGA variant with dual data rate sampling. The fastest operation frequency supported for devices that provide read data strobe signal (DQS/RWDS) applies to most octal flash and Hyperflash devices. For devices without DQS/RWDS support, the xSPI operating frequency during boot is slower.

NOTE: Check the processor data sheet for the operating frequency of the xSPI. The boot is supported up to the maximum frequency defined in the data sheet for RWDS and non-RWDS operations.

xSPI boot mode supports diverse devices because it uses automatic device detection. Automatic device detection combines hardware device discovery (DD) engine and software-based discovery. The device discovery engine attempts to read the device SFDP (Serial Flash Discoverable Parameter) database of the connected device. It starts with a single STR, followed by a single DTR, dual STR, and so forth to an octal DTR, and finally Hyperflash. This sequence ensures that the device discovery engine can read the SFDP database from the supported device, no matter the pre-configured mode. When it detects a valid SFDP header, it loads device info and configures the controller using the device SFDP settings. However, many devices do not support the SFDP standard (such as legacy NOR flash devices), so they fail the discovery by the DD engine. For this situation, the software detection starts to detect flash devices using familiar device read modes. Software-based discovery tries to read from the connected device by sending one-by-one, the following read commands until the flash device responds:

- Single SPI fast read command 0x3 with eight dummy cycles—one of the most common read commands supported by most flash devices.
- Octal read using the 0xEE/0x11 read command in OPI DTR mode with 20 dummy cycles—addresses Macronix octal flash devices configured in octal DTR mode.
- Hyperflash read sequence with 16 fixed latency cycles—addresses the Hyperflash devices that do not support SFDP.

Boot from External Flash Devices

The xSPI boot mode supports booting from 24-bit or 32-bit addressable NOR flash devices and commonly available Hyperflash memory devices. The xSPI boot mode uses the MDMA channels that work with the xSPI controller to get data from flash memory using the memory-mapped mode.

For auto-device detection and initial device detection, the PHY is configured for low speed to permit correct sampling for any flash device. Once autodetection succeeds, the PHY settings are updated using the frequency and the frequency-specific PHY setting placed in the loader stream.

CAUTION: Support for automatic device detection using the boot stream is not supported when booting with secure boot streams. Instead, when signing the boot image, configure an attribute in the image header that specifies the configuration to use.

For booting, the xSPI memory is connected as shown in the *xSPI Memory Connection* figure.

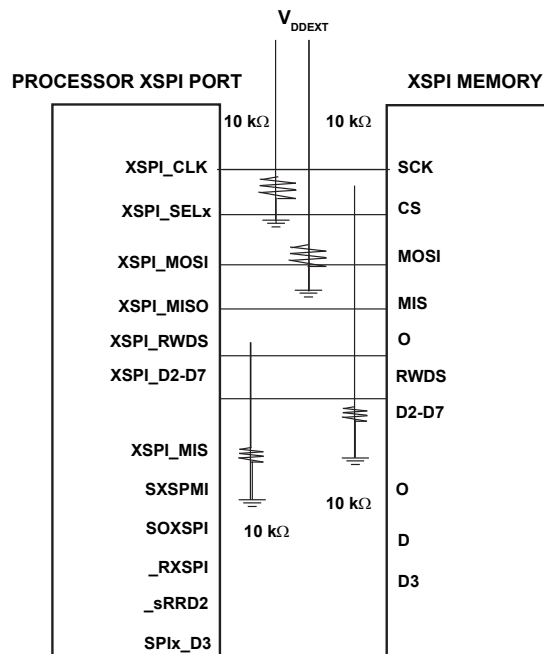


Figure 36-11: xSPI Memory Connections

The pull-up resistor on the target select signal ensures that the memory is deselected when the pin is in a high-impedance mode, such as during reset.

Initialization codes can manipulate the `ADI_ROM_BOOT_CONFIG:dBootCommand` to extend the boot mechanism to a second xSPI memory, connected to another target select pin. Updating the field that specifies the target select signal for use permits the boot process to manage larger boot streams than can fit into a single SPI device.

NOTE: When modifying the target select signal used during the boot process, configure the multiplexing pin to enable the correct pin functionality. Once the boot process continues past the configuration function and the boot process starts, the boot kernel cannot perform any additional pin multiplexing operations.

xSPI Device Detection Routine

This processor supports automatic device detection when the connected device can be detected by using DD hardware in the xSPI controller and software-based discovery.

Automatic device detection begins with the DD engine running through a device sequence to read the device SFDP database. It starts with a Single STR, then a Single DTR, Dual STR, and so forth to an Octal DTR, and finally Hyperflash. When it detects a valid SFDP header, it loads device info and configures the controller using the device SFDP settings. However, many devices do not support the SFDP standard (such as legacy NOR flash devices), so they fail the discovery by the DD engine. For this situation, the software detection starts to detect flash devices using familiar device read modes:

- Single mode with 0xB command—most devices can be detected in this way.

- When this fails, then attempt to connect to devices using an octal flash in OPI mode on power up. Then attempt to detect such devices with two different command and dummy cycle combinations. When this also fails, try using a Hyperflash read sequence with 16 latency cycles.

Because xSPI device discovery and initial block reads happen at a slower clock frequency, ADI advises (optional) to increase the clock frequency to speed up the boot. Once a valid device is detected, the xSPI clock can be updated while keep all controller settings the same (settings configured by DD engine or software detection routine) based on BCODE of first block. Based on the BCODE programmed in the flash device the xSPI clock is increased accordingly. In addition to just increasing the clock frequency, a user has an option to increase the clock and change the controller settings such as the bus-mode and opcode to use. This is needed for the situation where a device supports high performance modes such as Quad DTR, but the DD engine configures the controller only for QUAD STR mode. Because the engineer is aware of the supported device modes, they can direct the boot ROM to update the configuration correctly using the BCODE. More details about updating this configuration can be found in the BCODE.

Device Detection in Open Part

For an open part, the first 16 bytes of the boot stream are fetched and analyzed to determine when a valid boot stream block header is found. Once found, the kernel stores the header for processing. The BCODE is obtained for configuring the xSPI frequency or updating and overriding the device detection configuration. The controller remains configured with these settings for the remaining boot process.

When a valid boot block header is not found, the boot kernel then processes the 16 bytes to determine whether it is a secure boot stream. When the boot kernel detects a valid secure boot stream, it reads the remaining bytes of the secure header.

The secure boot stream can be encrypted and, therefore, the BCODE field (to determine the xSPI configuration) cannot be determined. Instead, the secure boot stream supports user-defined attributes in the secure boot header. These attributes can be configured using the `signtool` command line utility when creating a secure boot stream. Attribute 0x80000002 with a value of 0x0 through 0xF is supported. When the autodetection routine finds the 0x80000002 attribute in the secure header signature, it accordingly applies the xSPI configuration, or it uses the default as defined by `BCODE == 1`.

The PHY setting is also part of the loader stream. It used to configure the PHY for the requested frequency based on the BCODE (read from 0x80000002 attribute). For a secure boot stream, four 32-bit PHY configurations words are provided as different attributes in the boot stream.

Four PHY configuration words are stored in the following attributes:

- First-attribute 0x0
- Second-attribute 0x80000001 (bits 31:1 are valid, bit 0 is reserved)
- Third-attribute 0x80000004
- Fourth-attribute 0x80000005

Device Detection in Locked Part

When the device is locked, a complete secure boot header is fetched. The secure boot stream can be encrypted and, therefore, the BCODE field cannot be determined (to detect the xSPI configuration). Instead, the secure boot stream supports user-defined attributes in the secure boot header. These attributes can be set using the `signtool` command line utility when creating a secure boot stream.

Attribute 0x80000002 with a value of 0x0 through 0xF is supported. When the autodetection routine finds the 0x80000002 attribute in the secure header signature, it applies the xSPI configuration, accordingly, or it uses the default as defined by `BCODE == 1`.

The PHY setting is also a part of the loader stream and used to configure the PHY for the requested frequency based on the BCODE (read from 0x80000002 attribute). For the secure boot stream, four 32-bit PHY configuration words are provided as different attributes in the boot stream.

Four PHY configuration words are stored in the following attributes:

- First—attribute 0x0
- Second—attribute 0x80000001 (bits 31:1 are valid., bit 0 is reserved)
- Third—attribute 0x80000004
- Fourth—attribute 0x80000005

PHY Configuration in xSPI boot

xSPI always operates in PHY mode, which means the PHY needs to be configured correctly for the operating frequency before doing any access to the flash device. On powerup, xSPI operates at low frequency (16.67 MHz when the `SYS_CLKIN` is 25 MHz). The PHY is configured with pre-defined values to ensure that all the flash devices can be reliably accessed (during device detection). After device detection, the operating frequency can be increased based on the BCODE read from the boot stream.

The PHY is configured either statically from pre-programmed settings in the OTP or using the settings read from the boot stream. This is to ensure that the xSPI can operate and sample data correctly for the configured mode and frequency. For the configuration from boot stream, this comes from the secure image attributes (see the [Device Detection in Locked Part](#) section). The configuration can also come from the secure boot and from the xSPI configuration block in case of non-secure boot stream. In this situation, the PHY settings read from the boot stream stored in flash device. Initial data from the boot device is read at a slower speed to ensure correct operation for all device types. As part of this initial data read from the device, the PHY configuration which is stored as four configuration words is loaded by the boot kernel. Once autodetection completes, the PHY is reconfigured based on these settings. This flexibility provides an engineer the ability to operate at different frequencies while making sure that PHY is configured optimally for that frequency.

`nXspiPhyConfig0`, `nXspiPhyConfig1`, `nXspiPhyConfig2`, and `nXspiPhyConfig3` are members of the `ADI_ROM_BOOT_XSPI` data structure and updated from the OTP or from the boot stream before using them to configure the XSPI PHY.

xSPI Configuration Block in Loader Stream

The xSPI boot mode requires the xSPI configuration block to be included in the loader stream in the form of an ignore block, which contains a 20-byte payload with the following structure:

- First Byte—signature of 0x50485943
- Second Byte—PHY configuration Word 0
- Third Byte—PHY configuration Word 1
- Fourth Byte—PHY configuration Word 2
- Fifth Byte—PHY configuration Word 3

The xSPI Configuration Block is processed to store the PHY configuration words in the Boot ROM during the configuration routine of xSPI boot mode. Any wrong signature instead of 0x50485943 passed by the user, triggers a Boot ROM error. For other boot modes, this is ignored, and Boot ROM considers this block as a normal ignore block and the payload is discarded.

The xSPI Configuration block must be the second block for xSPI to recognize it, as shown in the *xSPI Configuration Block in Load Stream* figure.

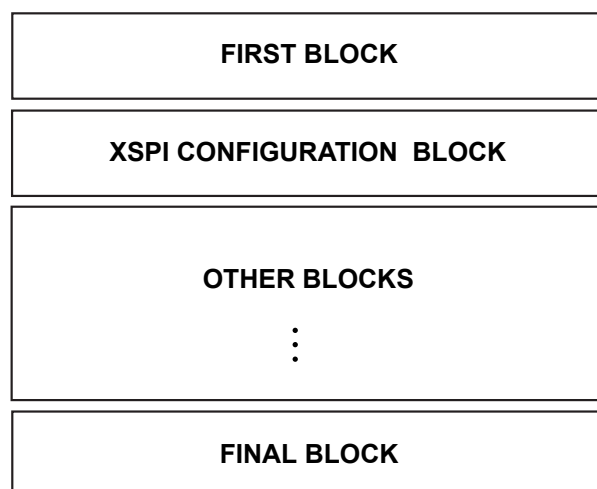


Figure 36-12: xSPI Configuration Block in Load Stream

The PHY configuration consists of four 32-bit configuration words which cover most of the PHY settings. The following tables show the different PHY settings encoded in these four configuration words.

- [Table 36-21 PHY Configuration Word0](#)
- [Table 36-22 PHY Configuration Word1](#)
- [Table 36-23 PHY Configuration Word2](#)
- [Table 36-24 PHY Configuration Word3](#)

Table 36-21: PHY Configuration Word0

Bits	Bit Field in 32-bit Configuration Word
31	RESERVED
30	XSPI_PHY_DLL_CTL.SDR_EDGE_ACT
29	XSPI_PHY_DLL_CTL.DQS_LST_DAT_DRP_EN
28	XSPI_PHY_DQ_TR.DAT_CLKPRD_DLY
27:24	XSPI_PHY_DLL_CTL.RESYNC_HI_WAIT_CNT
23:16	XSPI_PHY_DLL_CTL.RESYNC_IDLE_CNT
15:12	XSPI_PHY_DQ_TR.DAT_TSEL_START
11:8	XSPI_PHY_DQ_TR.DAT_TSEL_END
7:5	XSPI_PHY_DQ_TR.DAT_OE_START
4:2	XSPI_PHY_DQ_TR.DAT_OE_END
1	XSPI_PHY_DQS_TRL.PBK_DQS
0	XSPI_PHY_DQS_TR.PHONY_DQS_SEL

Table 36-22: PHY Configuration Word1

Bits	Bit Field in 32-bit Configuration word
31	RESERVED
30	XSPI_PHY_GATE_LPBK_CTL.RD_DEL_SEL_EMPTY
29:25	XSPI_PHY_GATE_LPBK_CTL.RD_DEL_SEL
24:21	XSPI_PHY_DQS_TR.DQS_SEL_TSEL_START
20:17	XSPI_PHY_DQS_TR.DQS_SEL_TSEL_END
16:14	XSPI_PHY_GATE_LPBK_CTL.LPBK_ERR_CHECK_TIM
13	XSPI_PHY_GATE_LPBK_CTL.LPBK_FAIL_MUXSEL
12:11	XSPI_PHY_GATE_LPBK_CTL.LPBK_CTL
10	XSPI_PHY_GATE_LPBK_CTL.LPBK_INT
9	XSPI_PHY_GATE_LPBK_CTL.LPBK_EN
8:7	Reserved
6:5	XSPI_PHY_GATE_LPBK_CTL.GATE_CFG_CLOSE
4:1	XSPI_PHY_GATE_LPBK_CTL.GATE_CFG
0	Reserved

Table 36-23: PHY Configuration Word2

Bits	Bit Field in 32-bit Configuration Word
31	XSPI_PHY_DLL_MSTR_CTL.DLL_BYPASS_MODE
30:28	XSPI_PHY_DLL_MSTR_CTL.DLL_LOCK_NUM
27:20	XSPI_PHY_DLL_MSTR_CTL.DLL_START_PT
19:12	XSPI_PHY_DLL_SLAVE_CTL.CLK_WR_DLY
11:4	XSPI_PHY_DLL_SLAVE_CTL.RD_DQS_DLY
3	XSPI_PHY_IE_TR.IE_ALWAYS_ON
2:0	XSPI_PHY_DLL_MSTR_CTL.PHASE_DT_SEL

Table 36-24: PHY Configuration Word3

Bits	Bit Field in 32-bit Configuration Word
31:25	Reserved
24:22	XSPI_PHY_IE_TR.DQ_IE_START
21:19	XSPI_PHY_IE_TR.DQ_IE_STOP
18:16	XSPI_PHY_IE_TR.DQS_IE_START
15:13	XSPI_PHY_IE_TR.DQS_IE_STOP
12	XSPI_PHY_GCTL_CTL.CLKPRD_DLY
11:8	XSPI_PHY_IE_TR.RDDAT_EN_IE_DLY
7:6	Reserved
5	XSPI_PHY_WR_DESKEW_PAD_CTL0.DQ_SW_DQ_PHASE_BYPASS
4	XSPI_PHY_WR_DESKEW_PAD_CTL0.DQ_EN_SW_HALF_CYCLE
3	XSPI_PHY_WR_DESKEW_PAD_CTL0.DQ_SW_HALF_CYCLE_SHIFT
2:0	XSPI_PHY_WR_DESKEW_PAD_CTL0.DQ_PHASE_DECT_SEL

xSPI Boot Device Detection and Boot Data Flow

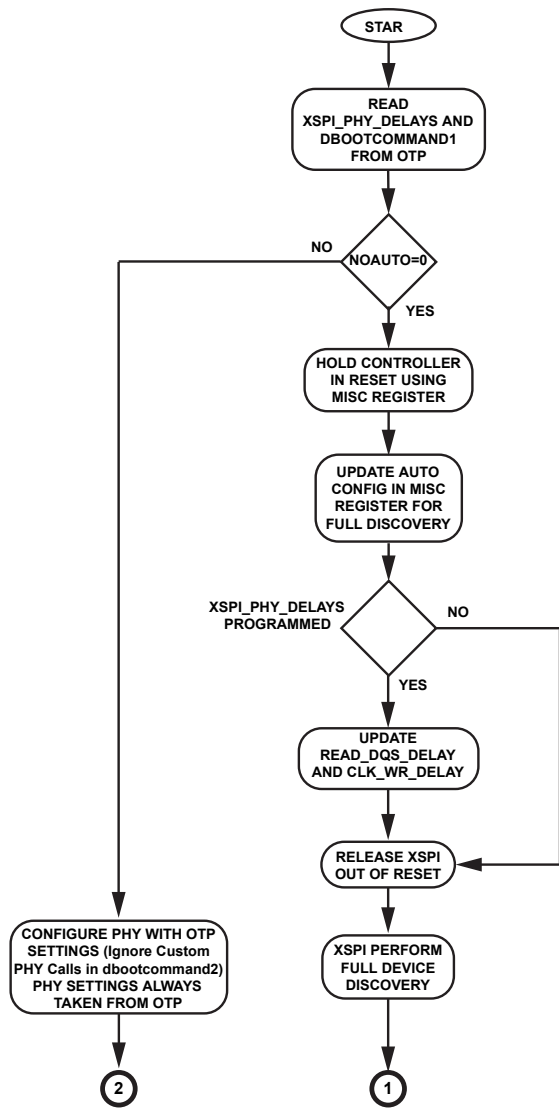


Figure 36-13: xSPI Device Detection and Boot Principle (Part 1 of 3)

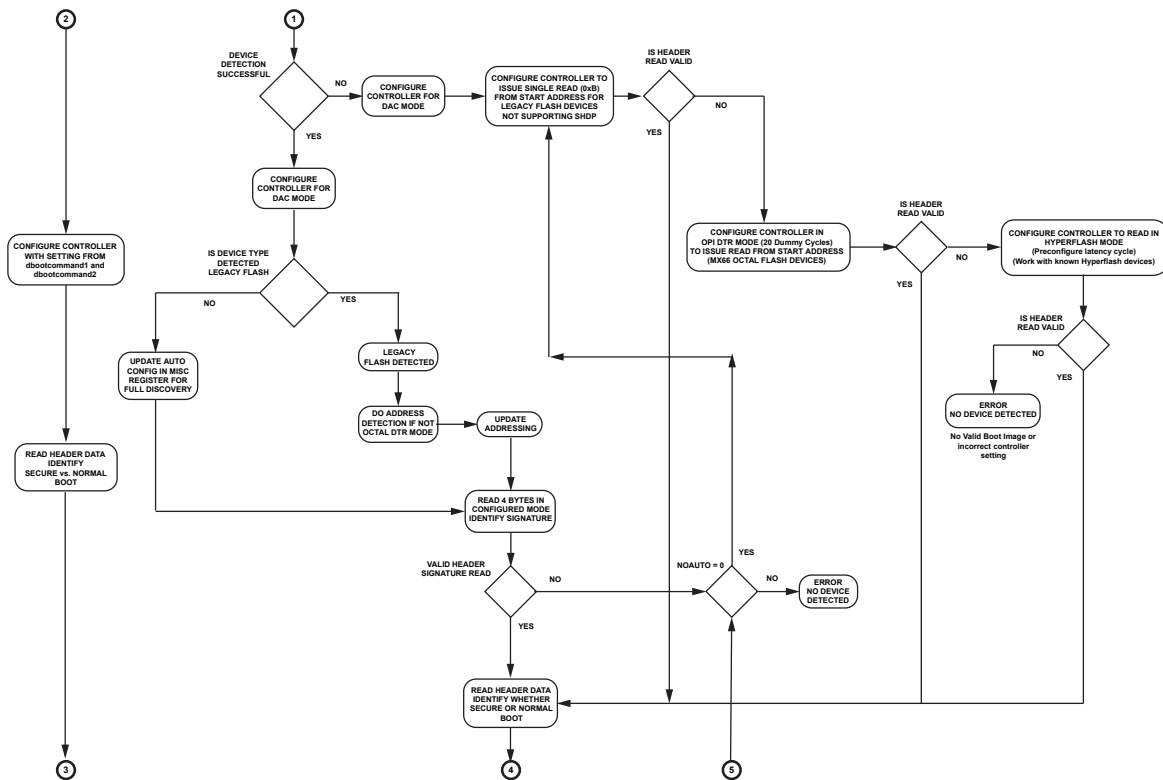


Figure 36-14: xSPI Device Detection and Boot Principle (Part 2 of 3)

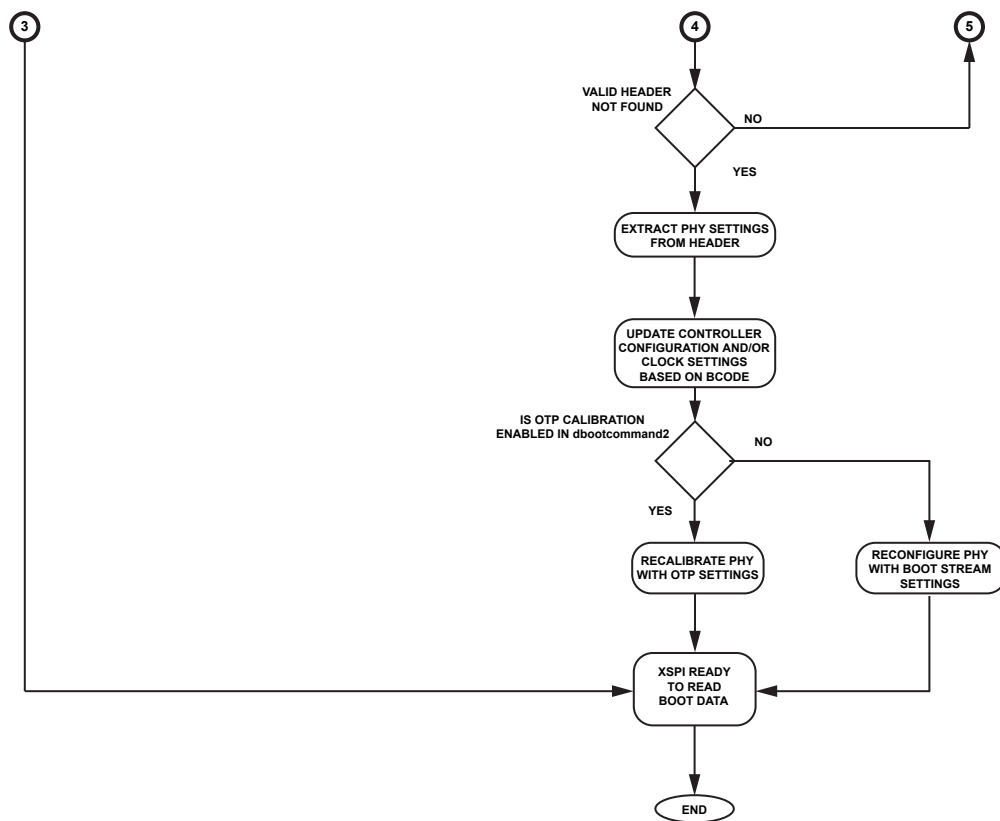


Figure 36-15: xSPI Device Detection and Boot Principle (Part 3 of 3)

The boot code applies settings to the xSPI peripheral according to the xSPI Controller Boot BCODE Table Descriptions.

Table 36-25: xSPI Controller BCODE Configuration Lookup Table

Member Name	BCODE				
	0	1	2	3	4
Config Name	Default	Default	Default with clk change	Default with clk change	Default with clk change
ubConfigChange	0	0	0	0	0
ubClockDivider	30	30	10	8	7
ubDummyCycles	0	0	0	0	0
ubNumlinesCommand	0	0	0	0	0
ubNumlinesAddress	0	0	0	0	0
ubNumlinesData	0	0	0	0	0
ubEdgemodeCommand	0	0	0	0	0
ubEdgemodeAddress	0	0	0	0	0
ubEdgemodeData	0	0	0	0	0

Table 36-25: xSPI Controller BCODE Configuration Lookup Table (Continued)

Member Name	BCODE				
	0	1	2	3	4
ubNumAddressBytes	0	0	0	0	0
ubOpcode0	0	0	0	0	0
ubOpcode1	0	0	0	0	0

Table 36-26: xSPI Controller BCODE Configuration Lookup Table

Member Name	BCODE				
	5	6	7	8	9
Config Name	Default with clk change	Default with clk change	QIOR DTRU	QIOR DTRU	QIOR DTRU
ubConfigChange	0	0	1	1	1
ubClockDivider	5	4	7	7	7
ubDummyCycles	0	0	6	6	8
ubNumlinesCommand	0	0	0	0	0
ubNumlinesAddress	0	0	2	2	2
ubNumlinesData	0	0	2	2	2
ubEdgemodeCommand	0	0	0	0	0
ubEdgemodeAddress	0	0	1	1	1
ubEdgemodeData	0	0	1	1	1
ubNumAddressBytes	0	0	3	4	3
ubOpcode0	0	0	0xED	0xEE	0xED
ubOpcode1	0	0	0	0	0

Table 36-27: xSPI Controller BCODE Configuration Lookup Table

Member Name	BCODE				
	A	B	C	D	E
Config Name	QIOR DTRU	QIOR DTRU	OIOR STRU	OIOR DTRU	OIOR DTRU
ubConfigChange	1	1	1	1	1
ubClockDivider	7	11	5	7	5
ubDummyCycles	8	7	16	16	16
ubNumlinesCommand	0	0	0	0	0
ubNumlinesAddress	2	2	3	3	3

Table 36-27: xSPI Controller BCODE Configuration Lookup Table (Continued)

Member Name	BCODE				
	A	B	C	D	E
ubNumlinesData	2	2	3	3	3
ubEdgemodeCommand	0	0	0	0	0
ubEdgemodeAddress	1	1	0	1	1
ubEdgemodeData	1	1	0	1	1
ubNumAddressBytes	4	4	4	4	4
ubOpcode0	0xEE	0xEE	0xCC	0xFD	0xFD
ubOpcode1	0	0	0	0	0

Run-Time API

The following table provides descriptions of the `adi_rom_Boot()` command parameter.

Table 36-28: XSPIControllerBootCommand Bit Descriptions

Bits	Name	Setting	Description
31:25	ROM_BCMD_XSPI_CLK	0-1111b	Clock speed divider value. Value is used to derive xSPI clock by division of PLLCLK 0—Invalid setting 1–63 Divide PLLCLOCK with this value
24:23	ROM_BCMD_XSPI_ADDR	00b	Reserved (Not valid for Hyperflash)
		01b	3-byte addressing (Not valid for Hyperflash)
		10b	4-byte addressing (Not valid for Hyperflash)
		11b	Reserved (Not valid for Hyperflash)
22:18	ROM_BCMD_XSPI_DUMMY	00000b	No dummy clock cycles required after the address
		00001b	One dummy clock cycle required after the address
			
		11111b	31 dummy clock cycles required after the address

Table 36-28: XSPIControllerBootCommand Bit Descriptions (Continued)

Bits	Name	Setting	Description
17:13 [JS4]	ROM_BCMD_XSPI_BUSMODE	0000b-111 1b	<i>Bus Mode</i> 0–SINGLE STR 1–SINGLE FAST STR 2–SINGLE DTR 3–DOR STR 4–DOR DTR 5–DIOR STR 6–DIOR DTR 7–QOR STR 8–QOR DTR 9–QIOR STR 10–QIOR DTR 11–QPI STR 12–QPI DTR 13–OOR STR 14–OOR DTR 15–OIOR STR 16–OIOR DTR 17–OPI STR 18–OPI-DTR (Setting valid only for Hyperflash) 19–31 Reserved
12	ROM_BCMD_XSPI_DEVTYPE	0	Legacy Flash
		1	Hyperflash
11:9	ROM_BCMD_XSPI_SSEL	000b	Use target select 1 for xSPI chip select
		001b	Use target select 2 for xSPI chip select
		010b	Use target select 3 for xSPI chip select
		011b	Reserved

CAUTION: All undefined bits in the previous table *must be configured to zero*.

Table 36-29: xSPI Controller Boot Command Bit Descriptions

Bits	Name	Setting	Description
		100b	Reserved
		101b	Reserved
		110b	Reserved
		111b	Reserved
8:7	ROM_BCMD_XSPI_DEVE- NUM	0b	xSPI device instance number (Only 0 is valid for ADSP-2183x)
		1b	Reserved
6	ROM_BCMD_XSPI_NOAUTO	0	Do automatic device detection and peripheral configuration.
		1	When set, configure xSPI based on xSPI dbootcomand and dbootcomand1 settings. Only custom PHY calib field in dbootcommand1 is always considered, irrespective of the NOAUTO bit
5	ROM_BCMD_XSPI_NOCFG	0	Instructs the configuration routine to perform pin-muxing and full peripheral configuration
		1	The configuration routine <i>does not</i> change the peripheral configuration and <i>does not</i> enable pin-muxing. Use when boot software does not support specific peripheral features. Can configure the peripheral before calling the boot routine.
4	ROM_BCMD_XSPI_HOST	0	Controller boot mode
		1	Do not use for xSPI boot
3:0	ROM_BCMD_XSPI_DEVICE	0000b	Do not use for xSPI boot
		0001b	Do not use for xSPI boot
		0010b	Do not use for xSPI boot
		0011b	Do not use for xSPI boot
		0100b	Do not use for xSPI boot
		0101b	Do not use for xSPI boot
		0110b	Do not use for xSPI boot
		0111b	Do not use for xSPI boot
		1000b	xSPI memory-mapped boot (memory-mapped mode and MDMA)
		1001b – 1111b	Do not use for xSPI boot

CAUTION: All undefined bits in the previous table *must be configured to zero*.

Table 36-30: xSPI Controller BootCommand 1 Bit Descriptions

Bits	Name	Setting	Description	
31:22	Reserved	00000000000b-11111111111b	Reserved	
21	ROM_BCMD1_XSPI_USER_PHYCALIB	0b	Load PHY settings from OTP or from boot stream	Decide whether to use user defined PHY settings through hook function in second stage boot.
		1b	Load PHY settings through hook function in second stage boot	
20	ROM_BCMD1_XSPI_CUSTOM_PHYCALIB	0b	Load PHY settings from boot stream	Decide whether PHY setting is taken from OTP or from boot stream for AUTO detection. For NOAUTO case, PHY configuration always load from OTP and user must ensure PHY settings are updated in OTP. If ROM_BCMD1_XSPI_USER_PHYCALIB is set, then this bit is ignored.
		1b	Load the PHY calibration values form OTP	
19:18	Reserved	00b-11b	Reserved	
17	ROM_BCMD1_XSPI_BYTE-SWAP	0	No byte swap	
		1	Swap the byte order	
16	ROM_BCMD1_XSPI_2BYTE-SPERADDR	0	1 byte per address	
		1	2 bytes per address	
15:8	ROM_BCMD1_XSPI_OPCODE2		xSPI read command 2 sent to the flash	
7:0	ROM_BCMD1_XSPI_OPCODE1		xSPI read command 1 sent to the flash	

CAUTION: All undefined bits in the previous table *must be configured to zero*.

xSPI Boot Command Example Usages

The xSPI boot command field can customize the xSPI boot mode for any desired frequency at a given bus mode with the required PHY configuration settings for the connected flash device. The following examples show how to use the boot command fields to customize the xSPI boot mode, by programming the xSPI boot command in the OTP, or by the calling the ROM API.

PHY setting customization—when enabling autodetection, the boot ROM configures the xSPI controller and the PHY settings embedded in the loader stream. But, the user has the flexibility to override the PHY settings by programming the PHY configuration words in the OTP. To achieve an override, implement the following boot commands.

```
XSPI Boot Command 0 = 0u<< BITP_ROM_BCMD_XSPI_NOAUTO | 0xAu<<
BITP_ROM_BCMD_XSPI_DEVICE;
```

```
XSPI Boot Command 1 = 1u<< BITP_ROM_BCMD1_XSPI_CUSTOM_PHYCALIB;
```

Using *xSPI bus mode customization for a frequency*—using no auto detection, configure the xSPI controller for any bus mode at any desired frequency. When auto detection is disabled, the boot ROM takes the PHY configuration from the OTP, by default. The user must program the PHY configuration words in the OTP for this technique. For example, to use the QIOR mode with a legacy NOR Flash device at 75 MHz with default clock configuration, the use the following boot commands.

```
XSPI Boot Command 0 = (0x14 <<BITP_ROM_BCMD_XSPI_CLK |
0x1<<BITP_ROM_BCMD_XSPI_ADDR | 0x6<<BITP_ROM_BCMD_XSPI_DUMMY |
0xA<<BITP_ROM_BCMD_XSPI_BUSMODE | 0x0 <<BITP_ROM_BCMD_XSPI_DEVTYPE |
0x0<<BITP_ROM_BCMD_XSPI_SSEL | 0x0<<BITP_ROM_BCMD_XSPI_DEVENUM |
0x1<<BITP_ROM_BCMD_XSPI_NOAUTO | 0xA<<BITP_ROM_BCMD_XSPI_DEVICE);

XSPI Boot Command 1 = 0xED<<BITP_ROM_BCMD1_XSPI_OPCODE1 | 0x00
<<BITP_ROM_BCMD1_XSPI_OPCODE2;
```

Boot Mode Swipe Through Command Customization

Each boot mode has a 32-bit `dbootcommand` field in OTP that can be used to override the default `dbootcommand` used in power up. The first four bits of the `dbootcommand` field `DEVICE` determines the boot peripheral to be used. This field can be updated to support any other existing boot peripheral (boot mode) for a given `dbootcommand` for an existing boot mode.

Alternate Controller/Target Boot Support

When controller or target boots fail, there are alternative boot methods that can be enabled using OTP.

Alternative Controller Boot

Refer to the *Alternate Controller/Target Boot Flow* diagram. Complete the following steps to boot the controller.

1. When a boot ROM error occurs, the boot ROM checks the following conditions:

- `ROM_OTP_bcfg_altMstrBootDis = 0` in OTP
- `ROM_OTP_bcfg_altMstrBootEn = 1` in OTP
- `altflashboot_flag = false`

If the conditions are valid, proceed. Otherwise, perform the alternate target boot.

2. Verify that the failure is a controller boot failure with any of the following error:

- `ROM_BOOT_ERR_DMA`
- `ROM_BOOT_ERR_HDRCHK`
- `ROM_BOOT_ERR_NOFINAL`
- `ROM_BOOT_SECURE_ERR_HDRCHK`

- ROM_BOOT_ERR_BMODE_CONFIG

If the failure is due to one of the above errors, proceed. Otherwise, perform the alternate target boot.

3. Configure `altflashboot_flag = true`. (This flag will not be cleared once configured; it can be cleared only using a reset (core/system)).
4. Perform the clean up routines.
5. Read `ROM_OTP_altflashStartAddress` from OTP.
6. Invoke a second stage boot with the default `dbootcommand` from `ROM_OTP_altflashStartAddress`. If it passes, then it goes to the application. Otherwise, repeat the alternate controller boot process.

Alternate Target Boot

Refer to the *Alternate Controller/Target Boot Flow* diagram. Complete the following steps to boot the target.

1. When a boot ROM error occurs, the boot ROM checks the following conditions:
 - `ROM_OTP_bcfg_altSlaveBootDis = 0` in OTP
 - `ROM_OTP_bcfg_altSlaveBootEn = 1` in OTP
 - `althostboot_flag = false`

If the conditions are valid, proceed. Otherwise, jump to the error handler.

2. Check whether the failure is a target boot failure with any of the following error.
 - ROM_BOOT_ERR_DMA
 - ROM_BOOT_ERR_HDRCHK
 - ROM_BOOT_ERR_NOFINAL
 - ROM_BOOT_SECURE_ERR_HDRCHK
 - ROM_BOOT_ERR_BMODE_CONFIG

If the failure is a due to one of the above errors, proceed. Otherwise, jump to the error handler.

3. Configure `althostboot_flag = true`. (This flag will not be cleared once configured; it can be cleared only using a reset (core/system)).
4. Perform the clean up routines.
5. Read the alternate target `dbootcommand` from OTP.
6. Configure the SPU for the target peripheral.

- Invoke a second stage to boot the alternate boot image from the host with alternate target default `dbbootcommand` read from OTP. If it passes then it goes to the application, Otherwise, repeat the alternate controller boot process.

The *Alternate Controller/Target Boot Flow* diagram shows the process flow for booting secondary and alternate images.

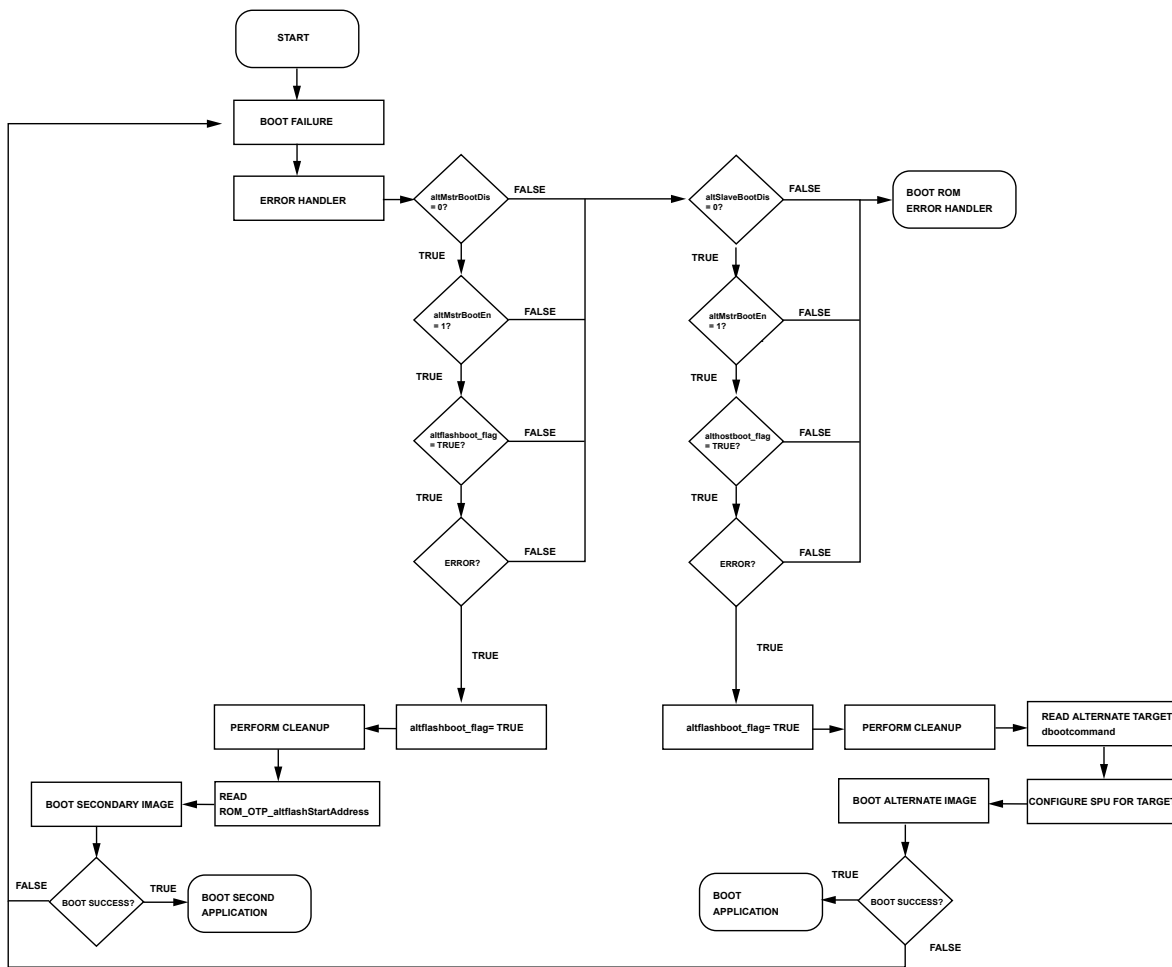


Figure 36-16: Alternate Controller/Target Boot Flow

NOTE: `altflashboot_flag` and `althostboot_flag` can be checked at `0x200fef74` and `0x200fef75` memory address, respectively.

Boot Loader Stream

A loader stream is a set of formatted blocks containing instructions for the boot kernel, as well as the application and data for loading to the chip. This section details the different aspects of the stream, its blocks, some common use cases, and the ROM function.

Each block begins with a block header that contains attributes of the block as well as flags to control its processing by the boot ROM. On power-up or reset, the processor begins executing the on-chip boot ROM. The boot stream is either read from memory or received from a peripheral, depending on the boot mode specified. Each block in the boot stream instructs the boot kernel to perform an action, most commonly to load data to a specified location. For a complete list of actions, refer to the [Block Types](#) section.

Some common actions include the following:

- running code that initializes a peripheral
- processing data then loading it to a location

As shown in the *Project Flow* figure, a utility is required to process the resulting output from the tool chain to create a valid boot stream. This utility can be in the form of a standalone application or script that parses an application image file, elf output file, or text-based file such as Intel hex. A flash programmer utility can format a boot stream internally.

The elfloader utility parses the application data, and creates a set of blocks representing the segmented application. When processing an actual image that must be loaded to a single contiguous memory block, this representation can contain as little as a single header block. The output of the standalone utility is stored in a loader file (.ldr). The loader file contains the boot stream and becomes available to hardware by:

- programming or burning it into non-volatile external memory, or
- sending it through a peripheral such as the UART during boot time

A loader stream must always begin with a first block and end with a final block. The loader file contains the boot stream and becomes available to hardware by:

- programming or burning it into non-volatile external memory, or
- sending it through a peripheral during boot time

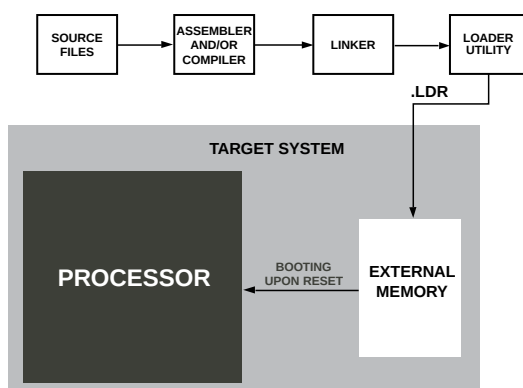


Figure 36-17: Project Flow

The *Booting Process* figure shows the parallel or serial boot stream contained in a flash memory device. In host boot scenarios, the non-volatile memory usually connects to the host processor rather than directly to the processor. After reset, the on-chip boot kernel reads and parses the headers and the loader stream is processed block-by-block.

Finally, payload data is copied to destination addresses, either in on-chip L1 and L2 memory, or off-chip to SDRAM or SRAM.

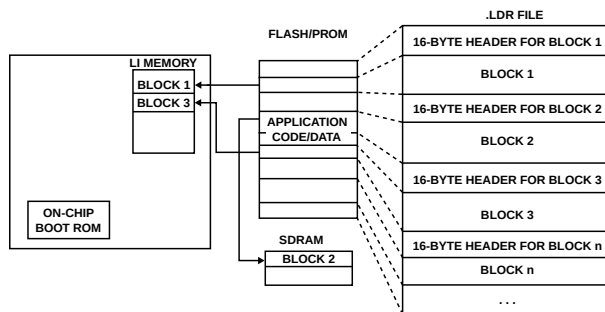


Figure 36-18: Booting Process

In some cases (for example, secure boot or when the `BFLAG_INDIRECT` flag for any block is set), the boot kernel uses another memory block for intermediate data storage. In order to preserve the security of the device processors do not allow these storage regions to be initialized with boot data. The boot stream is loaded to the intermediate storage then processed by the kernel and loaded to the final destination. The final destination cannot be the intermediate storage location otherwise the boot process terminates in an error.

Block Header

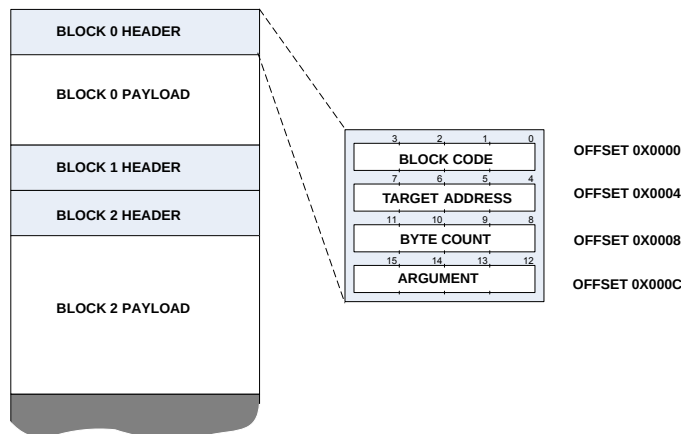


Figure 36-19: Loader Stream Block Structure

A boot stream consists of multiple boot blocks as shown in the figure. A 16-byte block header begins every block. The 16 bytes are functionally grouped into four 32-bit words:

- Block code
- Target address
- Byte count
- Argument field

This section provides a general description describes the fields in general. The use varies depending on the block type and boot mode.

Block Code

Table 36-31: Block Code Flags

Bit	Name	Description
0–3	BCODE	Specific to boot modes (see Boot Modes)
4	BFLAG_SAVE	Intended to allow for a user application to mark blocks for saving the memory of this block to off-chip memory in case of power failure. The on-chip boot kernel does not use this flag.
5	BFLAG_AUX	When set indicates that the byte address space translation for SHARC core boot blocks requires translation to the 48-bit PM address space. When cleared translation to the 32-bit address space is performed.
7–6	Reserved	
8	BFLAG_FILL	Fill the target location with a specified 32-bit value
9	Reserved	
10	BFLAG_CALLBACK	Calls the previously registered callback function
11	BFLAG_INIT	Calls function at target address. If the block contains a payload, the payload is loaded prior to the call.
12	BFLAG_IGNORE	Block payload is ignored
13	BFLAG_INDIRECT	Boots the payload to the intermediate storage location
14	BFLAG_FIRST	Indicates the block that is the beginning of a new application
15	BFLAG_FINAL	Indicates the last block of a loader stream. Booting will complete after processing the block. This flag does not denote the end of an application in a Multi-Application Boot Stream.
16–23	HDRCHK	A simple 8-bit XOR checksum of the other 24 bits in the boot block header.
24–31	HDRSIGN	0xAD, 0xAC or 0xAB. Indicates if the boot block is intended for core 0, core 1 or core 2 respectively.

TARGET_ADDRESS

The `TARGET_ADDRESS` holds the applicable address for the block, (where the code or data is loaded). However, the interpretation of the field differs depending on what specific flags are set in the block code. Refer to the documentation for each block type for details.

The following attributes must be true:

- The target address must be divisible by 4, as the boot kernel uses 32-bit DMA for certain operations.
- The target address must point to valid on-chip or off-chip memory locations.

BYTE_COUNT

The `BYTE_COUNT` must be divisible by 4, and can also be zero. This 32-bit field generally holds the size of the block. In some cases, it has a different use (such as when `BFLAG_FILL` is set). See the [Block Types](#) section for information on the variations.

ARGUMENT

The 32-bit field is a user-variable for most block types. The `Initcode` or `callback` routine can access this value and use it for optional instructions.

The different block types use the `ARGUMENT` field in various ways. See the [Block Types](#) descriptions for further information.

Block Types

A loader stream is a set of linked blocks and each block is responsible for performing a certain function dependent on the block type. The flags in the block header define a block type. Operations include functions such as loading data, filling a memory region with data, and instructing the kernel to stop processing. This section describes each block type and its usage within a boot loader stream.

Normal Block

The primary function of a block is to load data into a specified location of memory. A normal block instructs the boot kernel to load the data contained in its payload to the location specified in the `TARGET_ADDRESS` field. The `BYTE_COUNT` defines the size of the payload. Once the correct amount of data has been loaded, the kernel moves on to process the next block in the stream.

Table 36-32: Flags

Flag	Required Value	Init
<code>TARGET_ADDRESS</code>	Y	Address where payload is loaded (must be valid)
<code>BYTE_COUNT</code>	Y	Size of block in bytes

First Block

A first block indicates the start of a boot stream and is always needed at the beginning of the boot stream. When a loader stream contains [Multi-Application Boot Streams](#), a first block that occurs within the loader stream indicates the beginning of a new application.

When the kernel processes the first block in a loader stream, the boot kernel uses the `TARGET_ADDRESS` to determine the application entry location. For more details, refer to [Boot Termination and Application Execution](#).

NOTE: A first block cannot be combined with a fill block.

Table 36-33: Flags

Flag	Required Value	Init
<code>BFLAG_FIRST</code>	Y	1

Table 36-33: Flags (Continued)

Flag	Required Value	Init
ARGUMENT	Y	Offset to the next application, or first address following loader stream. Commonly referred to as the NEXTDXE field.
TARGET_ADDRESS	Y	When the block is the first block in a loader stream it defines the start address for the application. If the block is not the first in a loader stream, use the target address as in normal operation.

Final Block

The final block marks the last block in a boot stream. After processing a final block, the boot kernel jumps to the start address of the application. For more information on the definition of the start address, refer to [Boot Termination and Application Execution](#).

There is further customization available to alter the kernel behavior. For example, the kernel can be instructed to return from the boot routine rather than jump to the application using initialization codes or the `adi_rom_Boot()` API.

Before the boot kernel passes program control to the application, it does some housekeeping. Most of the registers in use are set to their default state. However, some register values can differ depending on the boot mode. See [Boot Modes](#) for more information.

Table 36-34: Flags

Flag	Required Value	Init
BFLAG_FINAL	Y	1

Indirect Block

An indirect block is first loaded to a storage location before being copied to the destination. This block is used in the following situations:

- Some boot modes do not use DMA from the boot peripheral. The core is not always able to access some memory locations directly or efficiently. An intermediate load to a different location improves overall efficiency.
- In some booting situations, the data in the payload must be operated-on or analyzed before it is fully loaded (such as decryption or a checksum calculation). By using an intermediate location, such situations are simplified and are more efficient when loading to off-chip memories.

In some cases, a boot block does not fit into temporary storage memory. Having a larger buffer can improve boot performance. If an entire block cannot fit into the buffer, it is processed in pieces. Initialization code or callback functions can alter the temporary buffer region, including its location and size, by modifying the `ADI_ROM_BOOT_CONFIG::pTempBuffer` and `ADI_ROM_BOOT_CONFIG::dTempByteCount` variables in the [struct ADI_ROM_BOOT_CONFIG](#) structure.

Table 36-35: Flags

Flag	Required Value	Init
BFLAG_INDIRECT	Y	1
BFLAG_CALLBACK	N	Defines a callback function to operate on intermediate data. These 2 flags are often used together.

Ignore Block

It is a block that is ignored by the loader stream (in several cases). These blocks are useful when it is not possible to pass information in another block header. For example, if the first block contains data such as a firmware revision rather than application code, then BFLAG_IGNORE can be set with the correct application start address. This step ensures that the loader stream uses the correct start address. Since this block has no other function, identify it as an ignore block. Then, the kernel does not attempt to process any payload.

Ignore blocks clear the following flags, disabling the corresponding blocks from being processed if set along with BFLAG_IGNORE:

- BFLAG_INIT
- BFLAG_CALLBACK
- BFLAG_FINAL
- BFLAG_AUX

NOTE: When BFLAG_IGNORE is set along with BFLAG_FIRST, only the payload associated with the first block is ignored. The application entry point retrieved from the first block is always processed.

Table 36-36: Flags

Flag	Required Value	Init
BFLAG_IGNORE	Y	1
BYTE_COUNT	Y	Size of block to ignore; can be zero

Fill Block

A fill block instructs the boot kernel to perform a 32-bit memory fill of the memory region. Fill blocks help minimize the size of a boot stream when an application contains large arrays of data that need are initialized at startup. Zero fill is the most common fill block type. However, any 32-bit fill value is supported.

Table 36-37: Flags

Flag	Required Value	Init
BFLAG_FILL	Y	1
TARGET_ADDRESS	Y	Address where payload is loaded (must be valid)
BYTE_COUNT	Y	Size of block in bytes (must be multiple of 4)
ARGUMENT	Y	32-bit fill value

Init Block

An initialization block instructs the boot kernel to perform a function call to the target address after the entire block has loaded. The function called is referred to as the *initialization code (Initcode) routine*. Refer to the API reference [initcode\(\)](#) for full details.

If the Initcode routine has been previously loaded, the block can declare a zero-size and have no payload.

Initcode routines can be used to speed up and customize booting mechanisms exposed by the boot kernel. Traditionally, an Initcode routine is used to set up the system PLL, bit rates, wait states, and the external memory controllers. Boot time can be significantly reduced when an init block is executed early in the boot process.

Initcode routines are required to follow the C language calling conventions. The expected C prototype is:

```
void initcode(ADI_ROM_BOOT_CONFIG * pBootConfig)
```

NOTE: When programming in assembly, use a return from subroutine instruction for returns.

The structure provided to the Initcode routine by the boot kernel contains information about the block being processed. It includes header information, locations of temporary block data (for indirect blocks), target address, and byte count. See [struct ADI_ROM_BOOT_CONFIG](#) for a full list and details on the provided data.

In the simplest case, an Initcode routine consists of only a single block in which the BFLAG_INIT flag is set. For larger routines, a sequence of blocks can incrementally load the routine, and only the last block sets the BFLAG_INIT flag. In the latter case, the last block has no payload attached, and simply instructs the boot kernel to issue a call to subroutine instruction.

An Initcode routine can be overwritten by a successive block when it is no longer needed. Otherwise, the routine can be called at multiple points during the boot process, and even remain in memory after the application completes booting.

NOTE: The following list provides requirements for Initcode that is written in C or C++.

- Ensure that the Initcode routine does not contain calls to the run-time libraries
- Do not assume that parts of the run-time environment, such as the heap, are fully functional
- Ensure that all run-time components are loaded and initialized before the routine executes

The loader utility and tool set provide mechanisms to aid in implementing initialization codes and organizing them properly within the boot loader stream. A special project type is provided to allow the creation of Initcode routines as separate projects. Options are available to assign particular pieces of the application to be Initcode routines. For details and more information on the utility, see the *Loader and Utilities Manual*.

Table 36-38: Flags

Flag	Required Value	Init
BFLAG_INIT	Y	1
TARGET_ADDRESS	Y	Location to load payload data. Call to subroutine issued to the same location.
ARGUMENT	N	Can be used to supply block specific arguments
BYTE_COUNT	Y	Size of payload; can be zero

NOTE: Init blocks cause software not located in the boot ROM during the boot process to execute. In the case of a secure boot scenario, initcode routines are not supported. The secure boot authentication process is performed at the end of the boot process. Execution of any user software prior to the authentication process violates the secure boot requirements.

Callback Block

It instructs the boot kernel to call a pre-registered function when the payload of the block is loaded. The purpose of a callback routine is to apply standard processing to the block payload. The callback routine is registered through an Initcode routine prior to loading a block using the routine. Typically, callback routines contain checksum, decryption, decompression or hash algorithms.

To register a callback, create an [Init Block](#) whose Initcode modifies the `ADI_ROM_BOOT_CONFIG::pCallbackFunction` with the address of the callback function to execute. A callback function must be registered prior to processing a callback block.

As callback routines require access to the payload data of the boot blocks, load the block data before processing. Often an [Indirect Block](#) is used in combination with a callback block. Indirect blocks in combination with callback blocks allow for post processing of the loaded data before it is then transferred to the final destination.

Callback routines are expected to meet the C language calling conventions. The prototype is as follows:

```
ROM_BOOT_RESULT callback(
    ADI_ROM_BOOT_CONFIG * pBootConfig,
    ADI_ROM_BOOT_BUFFER * pBuffer,
    uint32_t nFlags
)
```

The `pBootConfig` argument contains a pointer to the [struct ADI_ROM_BOOT_CONFIG](#) information and `pBuffer` provides access to the address and size of the block (can vary when using indirect). The `nFlags` parameter is specifically used when the `BFLAG_INDIRECT` flag is also used. The `CBFLAG_DIRECT` flag indicates that

the `BFLAG_INDIRECT` bit is not active so that the program only calls the callback routine once per block. When the `CBFLAG_DIRECT` flag is set, the `CBFLAG_FIRST` and `CBFLAG_FINAL` flags are also set.

NOTE: Callback blocks result in execution of software that is not located in the boot ROM during the boot process. In the case of a secure boot scenario callback routines are not supported because the secure boot authentication process is performed at the end of the boot process and execution of any program prior to the authentication process violates the secure boot requirements.

Callback Block in Conjunction with Indirect Block

A block using a callback routine is also loaded indirectly and there are slight behavioural differences. The procedure for loading is:

1. Load data into the temporary buffer defined by `ADI_ROM_BOOT_CONFIG::pTempBuffer`.
2. Issue a call to `ADI_ROM_BOOT_CONFIG::pCallbackFunction`.
3. After the callback routine returns and if the return value is zero, the memory DMA copies data to the destination.

When a block does not fit entirely into the temporary buffer, loading is performed similar to indirect blocks. The software calls the callback function after each chunk is loaded into the temporary storage. The `nFlags` parameter provides information on the specific iteration.

When a block does not fit entirely into the temporary storage area, `nFlags` tells the callback routine whether it is invoked for the first time (`CBFLAG_FIRST`) or called the last time (`CBFLAG_FINAL`) for a specific block.

When the software invokes DMA to copy the data, it relies on the supplied `pBuffer` data, not the `ADI_ROM_BOOT_CONFIG::pTempBuffer` and `ADI_ROM_BOOT_CONFIG::dTempByteCount` members of the boot structure.

The callback routine can control the source of the memory DMA by altering the content of the `pBuffer` structure. This alteration is necessary when the callback routine performs data manipulation such as decompression.

When the software uses an indirect block, the return value of the callback routine determines whether the DMA transfer occurs. If the value is non-zero, then the transfer does not occur.

Table 36-39: Flags

Flag	Required Value	Init
<code>BFLAG_CALLBACK</code>	Y	1

NOTE: Callback blocks cause software not located in the boot ROM during the boot process to execute. In the case of a secure boot scenario callback routines are not supported because the secure boot authentication process is performed at the end of the boot process and execution of any user software prior to the authentication process violates the secure boot requirements.

Save Block

A save block is intended to mark blocks in a boot stream for saving to off-chip memory. The on-chip boot kernel does not use this flag. A program can process the boot stream to find address of regions of memory for saving to external memory. On a reboot the program may then restore the previously saved contents. This block provides a means of doing a context restore after a reboot.

Table 36-40: Flags

Flag	Required Value	Init
BFLAG_SAVE	Y	1

Single-Block Boot Streams

The simplest boot stream consists of a single block header and one contiguous block of instructions and optionally data. When the appropriate flags are set in the block header, the kernel loads the block to the target address, terminates the boot process, and begins executing from the entry address of the application.

The *Initial Header for Single-Block Stream* table shows an example of a single-block boot stream header with settings that can be loaded using any boot mode. The BFLAG_FIRST and BFLAG_FINAL flags are both set at the same time. The desired location and size of the application determines the target address and byte count.

Table 36-41: Initial Header for Single-Block Stream

Field	Description of Value
BLOCK_CODE	0xAD000000 XORSUM BFLAG_FINAL BFLAG_FIRST
TARGET_ADDRESS	Start address of block
BYTE_COUNT	Number of bytes in the block
ARGUMENT	Functions as next-application pointer in multi-application boot streams.

Direct Code Execution

Applications can avoid long boot times and execute code from flash or SDRAM memory with minimal processing by the boot kernel. This feature is called direct code execution.

An initial boot block header is required for the processor to fetch and execute program code from the boot device as early as possible. The block uses safety mechanisms to avoid unpredictable processor behavior when boot memory is not yet programmed with valid data. Safety mechanisms include the header signature and the byte-wise XOR checksum. Rather than blindly executing code, the boot kernel first executes the pre-boot routine for system customization. It then loads the first block header and checks it for consistency. If the block header is corrupt, the boot kernel calls the error handler and does not start code execution.

If the initial block header check is good, the boot kernel interrogates the block flags. If the BFLAG_FINAL flag is set, the boot kernel terminates and executes the sequence as described in the [Boot Termination and Application Execution](#) section. If the application requires that the boot kernel customize the starting address in advance, the first

block must also have the BFLAG_FIRST flag set. The target address field is then saved as the application start address.

For example, when the block header described in the *Direct Code Execution Header* table is placed at address 0x20000000, the boot kernel is instructed to issue a JUMP command to address 0x20000020.

Table 36-42: Example Direct Code Execution Header

Field	Value	Comments
BLOCK_CODE	0xAD7BD006	0xAD000000 XORSUM BFLAG_FINAL BFLAG_FIRST BFLAG_IGNORE BCODE
TARGET_ADDRESS	0x20000020	Start address of application code. Provided as an example this would be dependent upon the start address required for a given product.
BYTE_COUNT	0x00000010	Ignores 16 bytes to provide space for control data such as version code and build data. This field is optional and can be zero. The payload is ignored due to the BFLAG_IGNORE flag being set.
ARGUMENT	0x00000010	Functions as next-application pointer in multi-application boot streams.

Multi-Application Boot Streams

This section describes loader streams that contain multiple applications.

A boot stream is typically generated from an application file. Loader streams with more than one application are commonly referred to as multi-application booting. A loader utility often accepts multiple application files as input parameters and generates a contiguous boot image. The subsequent applications are appended to the first.

The loader utility must also update the argument field of all BFLAG_FIRST blocks. The argument field of a BFLAG_FINAL block is called the *next-application* pointer.

The next-application pointer of the first application boot stream is a relative pointer to the start address of the second application boot stream. A multi-application boot image can be seen as a linked list of boot streams. The next-application pointer of the last application boot stream is a relative pointer to the next free address. The *Multi-Application Boot Stream Example* figure shows an example.

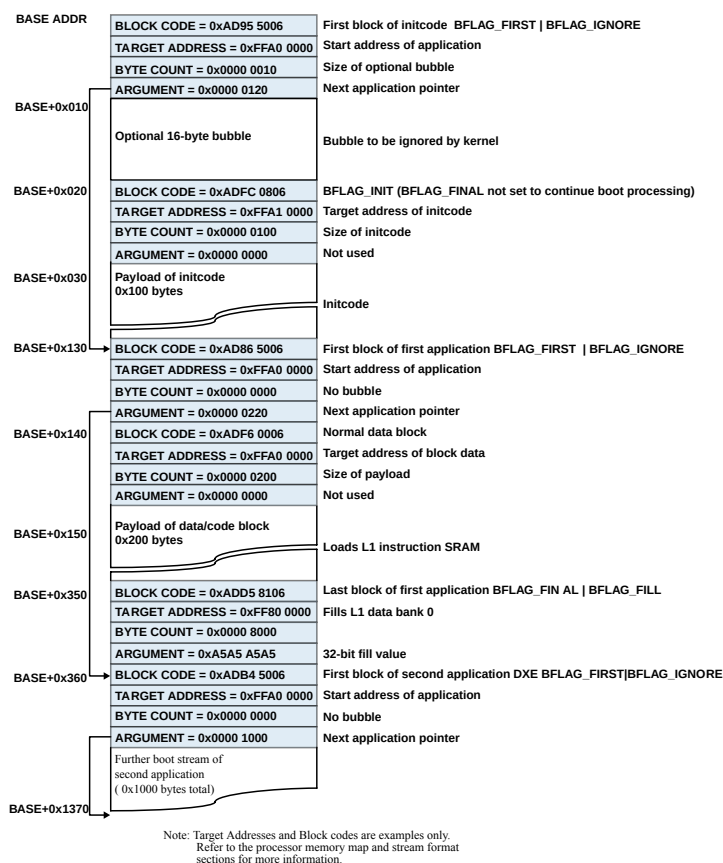


Figure 36-20: Multi-Application Boot Stream Example

The *Multi-Application Direct Code Execution Example* figure shows a linked list of initial block headers. The blocks instruct the boot kernel to terminate immediately and to start code execution at the address provided by the target address field of the individual blocks. There is nothing in the boot code that prevents multi-application streams from mixing regular boot streams and direct code execution blocks.

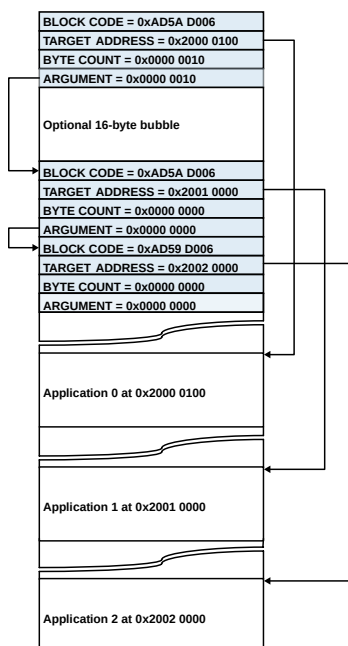


Figure 36-21: Multi-Application Direct Code Execution Example

CRC32 Protection

The boot kernel has mechanisms that allow verification of each block's payload using a 32-bit CRC. The boot ROM contains a function that can be called as an initcode to register the CRC callback and initialize the CRC peripheral with a specified polynomial. CRC32 protected loader streams are supported by the programming tools utilities.

To enable this feature an [Block Types](#) must be located in the boot stream with a `TARGET_ADDRESS` that points to the `adi_rom_Crc32Init()` function in the ROM. The `ARGUMENT` field contains the CRC32 checksum polynomial that is used to initialize the CRC lookup table. Once the CRC initcode function in the ROM has been executed, CRC verification is enabled for all subsequent blocks except:

- Ignore
- First

NOTE: CRC functionality is dependent upon the use of an Initblock and is not supported in secure booting.

Secure Boot

The secure boot process provides a way to integrate security in the processor boot sequence. A chain of trust is established within the system by ensuring the integrity and authenticity of the boot image. Confidentiality is also supported.

Secure boot increases protection against malicious, unsecured accesses to critical and confidential resources of the processor. The boot stream application code and data must be digitally signed in order to build up a chain of trust in

the system. This requirement allows the processor to distinguish between authentic and trusted code from non-authentic and untrusted code.

Secure boot also provides confidentiality support. The digitally signed boot image can be optionally encrypted. When loading an encrypted image, the ROM decrypts while loading, then authenticates, before any application code is executed.

Secure boot is an optional feature of the processor that is disabled by default. The feature is enabled using the OTP lock API; secure boot cannot be disabled after it has been enabled. When security is enabled, developers are not dependent on Analog Devices to provision the devices, sign code or provide security certificates. The required tools for signing and encrypting the boot images are provided with the processor development tools.

Integrity and Authenticity Protection

Integrity protection is based on the secure hash SHA-2 224/256-bit algorithm. Authenticity protection is based on the ECDSA algorithm.

ECDSA uses public key cryptography consisting of two keys: a private key and a public key. The public key is stored in OTP memory on the processor so that the secure boot process can verify the authenticity of the signed boot image. Only parties in possession of the private key are able to sign the images.

Confidentiality Protection

Confidentiality protection uses the AES algorithm. Two variants are supported: wrapped and unwrapped.

The wrapped variant uses a 128-bit Key Encryption Key (KEK) stored on the processor to decrypt the 128-bit AES decryption key embedded in the secure header. The unwrapped variant stores the AES description key on the processor and utilizes it to decrypt the entire image.

The privacy of the key stored on the device (whether AES or KEK) is paramount to the security of the system. Disclosure of this key compromises security of the entire system.

Anti-Cloning Protection

Anti-cloning protection is based on the confidentiality protection. If each processor in a system uses a unique private key for confidentiality protection, then cloning between these devices can be prevented. The boot image is incompatible with devices using a different private key for decryption.

Anti-Rollback Protection

The secure boot process supports anti-rollback protection using a 32-bit counter in OTP memory. A value of 0x00000000 in the OTP makes anti-rollback disabled by default. If anti-rollback protection is required, then the program can set the rollback ID when signing the boot image. When the boot image is authenticated, the secure boot software updates the counter in OTP (if the rollback ID in the boot image is greater than the value currently stored in the OTP counter).

The rollback ID stored in the secure boot image header is integrity-protected and cannot be altered.

NOTE: To enable anti-rollback protection for secure boot operations, write a non-zero value to the 32-bit counter in OTP memory. As long as the counter register remains at the default value of zero, anti-rollback protections are not enabled regardless of the rollback ID located in a secure boot stream.

CAUTION: There are a number of restrictions for the rollback ID in the OTP module. Programs should only use the OTP boot program ROM API to set the counter. Refer to the OTP counters section for information on the implementation strategy.

Terminology

ECDSA

Elliptical Curve Digital Signature Algorithm

BLp

Boot Loader plain text, Plaintext Format

BLx

Boot Loader without key, Keyless Format

BLw

Boot Loader wrapped, Wrapped Format

BLe

Boot Loader encrypted, Encrypted Format

SBLS

Secure Boot Loader Stream

SBH

Secure Boot Header

SBCR

Secure Boot Confidentiality Root

AES

Advanced Encryption Standard

Signing for Secure Boot Images

All boot images must be digitally signed to create secure boot images. The boot image is processed by the security utilities included with the development tools to sign and optionally encrypt the boot image. The security utilities operate with key-pairs consisting of a private and a public key. The private key is used for signing the images, and the public key is used to validate an image being loaded into the processor.

CAUTION: The private key generated from the signing utility, used for signing images, is never required by the processor for successful secure boot. The private key is only ever required by the signing utility and should be made available only within the system responsible for the image signing process.

The image signing utility provides the following functionality:

- Signing and encrypting of images
- Generation of ECDSA key pairs
- Generation of random encryption keys
- Extraction of the public key from an ECDSA key pair
- Setting [Secure Boot Image Attributes](#)

For more information on the use of the signing utility, refer to the Loader and Utilities manual.

Secure Boot Image Types

This section provides an overview of the different image types and how to use them.

Plaintext (BLp) Format

The BLp format provides integrity plus authentication protection of the boot image. The boot image is produced using 224-bit or 256-bit Elliptical Curve Digital Signature Algorithm (ECDSA) private key. To authenticate the image, program the corresponding public key into the OTP `public_key` field using the OTP boot program API.

SBH	Boot Loader Stream
-----	--------------------

Wrapped (BLw) Format

The BLw format provides the highest level of protection: integrity plus authentication, confidentiality, and anti-cloning protection. The image contains a wrapped image encryption key (denoted by [K]) within the secure header. The image data is encrypted with the wrapped key, preventing cloning. An extra key is required to unwrap the header; program this key into the OTP `pvt_128key` field using the OTP boot program API.

SBH [K]	Encrypted Boot Loader Stream
---------	------------------------------

Keyless (BLx) Format

The BLx format is similar to the BLw format except that the image does not contain the key at all. This format provides anti-cloning protection only if the secure key is unique per device. Program the decryption key for the data into OTP `pvt_128key` field using the OTP boot program API.

SBH	Encrypted Boot Loader Stream
-----	------------------------------

Secure Boot Image Format

Secure Boot images provide authenticity and integrity protection during the boot process. A secure boot image is comprised of a secure boot header and an optionally encrypted loader stream.

Signed images consist of the following sections to comprise a complete secure boot image:

- Secure Boot Header
- Image Attributes
- Image Section

The [Figure 36-22 Secure Boot Image](#) shows that the image attributes are encapsulated within the secure boot header. The image attributes are actually integrity protected along with the image section. The image section contains a standard boot loader stream. Some block types are not allowed as described in [Unsupported Boot Stream Blocks](#).

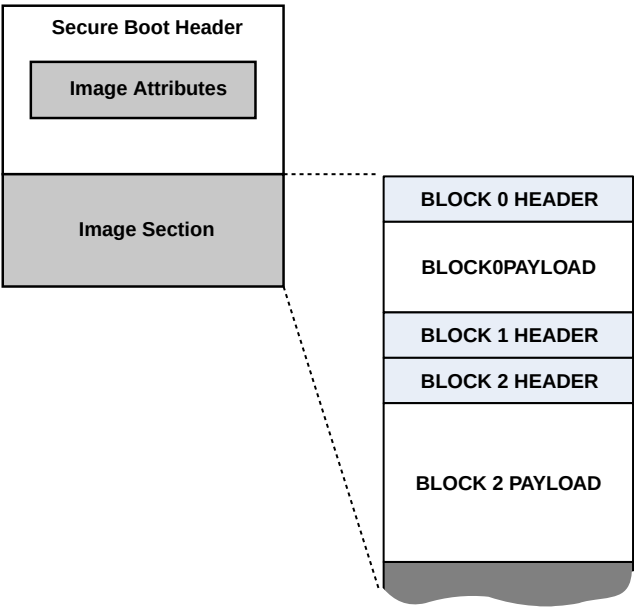


Figure 36-22: Secure Boot Image

Secure Boot Header

Table 36-43: Secure Boot Header

Bytes	Name	Description	Values		
			Keyless Format BLx	Wrapped Format BLw	Plain Text Format BLp
3:0	Type	Format and version of the image. Upper 24 bits is the image format and lower 8 bits is the image version	0x424C7803	0x424C7703	0x424C7003
67:4	Signature	The ECDSA signature of the image	Two 256-bit numbers		
99:68	Hash	Hash of the image	Hash size depends on whether ECDSA 224 or 256 is used. 1. For ECDSA 224 Hash Value: 95:68 bytes 2. For ECDSA 256 Hash Value: 99:68 bytes		
123:100	Key	Confidentiality (only applicable for certain formats)	Reserved	192-bit AES-WRAP data holding a 128-bit AES key	Reserved
139:124	IV	Initialization Vector (only applicable for certain formats)	16-bit IV generated during signing process		
143:140	Length	The length of the image section in bytes	Maximum supported byte count 0x10000000 bytes		
207:144	Attributes	Image attributes	Support for up to 8 image attributes		
255:208	Reserved				

Secure Boot Processing Overview

The *Secure Boot Processing* figure illustrates how the block is processed using secure features (not all block header type details are shown). For details on the various block types and their function, see [Block Types](#). Some image types are decrypted. Decrypt indicates that the data is decrypted when applicable to the [Secure Boot Image Types](#) at that particular stage.

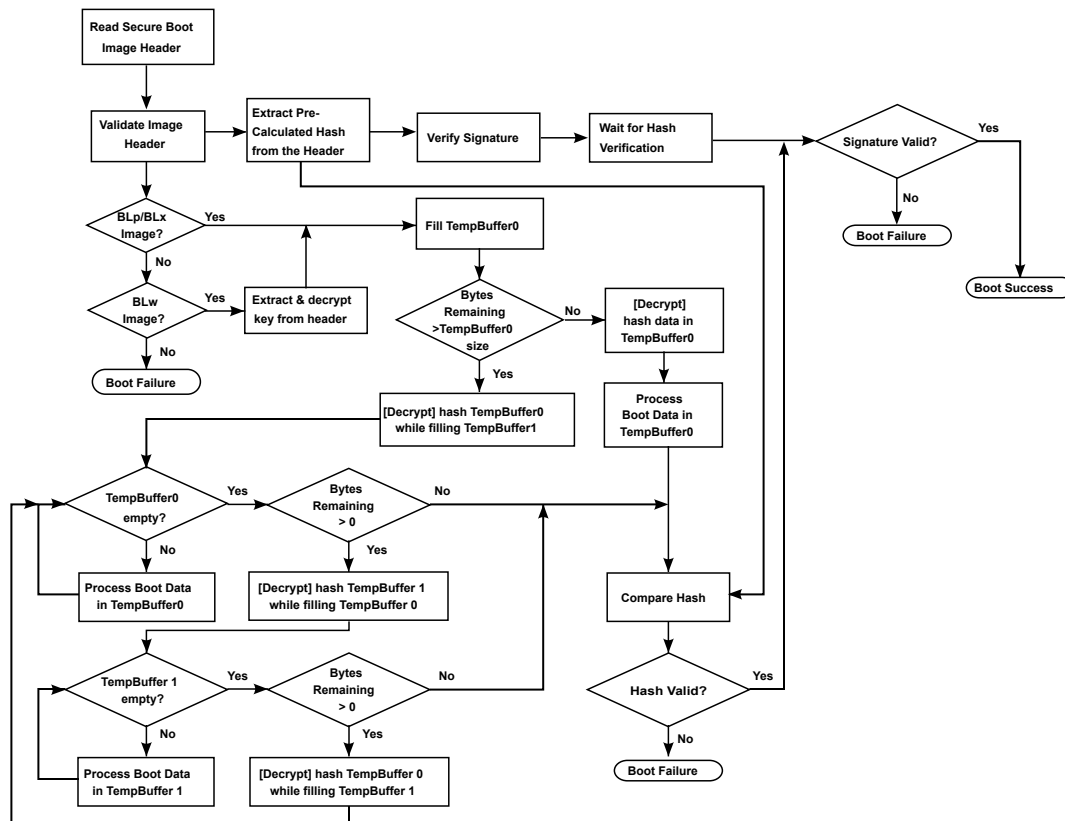


Figure 36-23: Secure Boot Processing

Unsupported Boot Stream Blocks

To ensure the security of the processor, the following block types are not supported in a secure boot image. If the boot kernel finds one of these block types, the boot process terminates.

- **Init Block:** It requires a call to the application code prior to the authentication of the boot image. If customizations or optimizations are necessary to improve the load performance, use a second stage loader style implementation. The first application contains only the custom code. Issue a call using the `api_boot()` routine to boot using the desired device.
- **Call Block:** It requires a call to a user-defined address prior to the authentication of the boot image, and therefore cannot be supported.

NOTE: Secure boot streams use double buffer [Page Mode](#) to optimize the boot process. This functionality allows for the performance of decrypt and hash operations on received data while new data is fetched from the boot source. This host in target boot mode must ensure that more data is sent after the boot stream to ensure that the temp buffer is filled completely. The size of the secure boot stream minus the size of the secure boot header must be a multiple of the size of the temp buffer. The temp buffer default size is 1024 bytes.

Pre-hash Authentication

Boot ROM uses the pre-calculated hash (SHA-224 or SHA-256) stored in the secure header to start the authentication process earlier and thereby reduce the boot ROM. With a pre-calculated hash of the whole boot image available, authentication starts in parallel with the rest of the boot process. Since the boot uses the pre-calculated hash for authentication, the last step is to verify that the hash from the actual image read from the boot source matches the pre-calculated hash. If the hash does not match, the result of ECDSA authentication is ignored and the secure boot fails. However, if the hash matches, the result of ECDSA authentication is used to decide whether the secure boot was successful.

Secure Boot Image Attributes

Secure boot image attributes form part of the secure boot header. The attributes provide more information about the content of the secure boot image.

All image attributes are integrity protected using the same algorithm as the image section. When the image authentication process completes and the image is successfully authenticated, the image attributes are known to be trustworthy.

Attributes are specified as type value pairs with both the type and value being a 32-bit value. The boot code supports the following image attributes.

Table 36-44: Secure Boot Image Attributes

ID	Name	Description	Values	
0x00000000	Unused	Unused attribute	0x00000000	
0x00000001	Version	Version of the attribute format	0x00000000	
0x00000002	Rollback ID	Current value of the rollback counter	0x00000000	Rollback disabled
			0x00000001 - 0x0000001F	Current firmware revision. It must be greater than or equal to the value retrieved from OTP.
0x80000001	NoRestore	Controls whether the boot process restored registers back to default values and clears sensitive security related information from the stack and dedicated structures.	0x1	Do not restore registers and clear security information from stack and dedicated structures.
			Other	Restore registers and clear security sensitive information from the stack and dedicated structures.
0x80000002	BCODE	Used by the boot mode drivers that support auto-detection to configure the	Values between 0x00000000 and 0x0000000F supported.	

Table 36-44: Secure Boot Image Attributes (Continued)

ID	Name	Description	Values
		device from a range of preconfigured settings	
0x80000003	ECDSA Type	Used by boot ROM to perform Authentication using ECDSA-224 or ECDSA-256 algorithm.	224 : Authentication using ECDSA-224 algorithm 256 : Authentication using ECDSA-256 algorithm

Secure Boot Streams

The secure boot image sections contain the code and data for loading to the various memory regions of the processor. The content is in the form of a boot stream format consisting of block headers that provide a descriptor for the associated block payloads.

The image section within the secure boot stream consists of a standard [Boot Loader Stream](#) consisting of block headers and payloads as generated by the supporting tools and utilities.

The secure boot image section can contain application images for just a single core or for multiple cores allowing for a flexible booting strategy.

A single boot image containing program and data for multiple cores permits a full system boot initializing all internal memories without needing a second stage loader approach. If boot images are required to initialize external memories, then a multi-stage loader approach can be required to configure the memory interfaces. The advantage of a single boot image for multiple cores is that authentication and decryption of the boot image requires only one execution. However, it is over a larger boot image.

Multiple single-core boot images permit one core to boot then execute that core's boot image without booting the other cores. This booting strategy can be needed when a single processor must be brought up quickly to deal with some initial tasks before booting the rest of the system. Or, it can be used to initialize extra peripherals to use for external memory interfaces. The core that has previously booted can then control the boot process for the remaining cores.

Single Core Images

Single core boot images are the result of processing a single core's output from the linker and converting it to a compliant boot stream. The resulting boot stream has a single first block at the beginning of the boot stream and a final block at the end of the boot stream.

The [Block Header](#) HDRSIGN field of the resulting boot stream is used to identify which core the image is intended for. This identification is required so that the boot code can set the correct [RCU_SVECT0](#) from the target address in the first block once the authentication is successful. When the boot image is loaded and authentication is successful, the boot code jumps to the location stored in the cores corresponding [RCU_SVECT0](#) register.

This boot stream must then be signed using the secure boot utilities resulting in the final secure boot image for a single core.

A second stage loader option is also available giving maximum flexibility both in terms of using ROM functionality, or creating a custom booting strategy. The simplest option is to have the application use `adi_rom_Boot()` to boot the main application image or indeed a third stage loader when one is required.

The boot stream is generated such that all image contents are in the system address space. Core specific L1 memory sections are converted during the boot stream generation process such that they are loaded through the multiprocessor memory space. The core executing the second stage loader can boot an application image intended for another core.

NOTE: The `ROM_BFLAG_RETURN` flag is set when calling the boot routine for the other cores than the core executing the second stage loader then cleared when loading the cores own image. Failure to do so would result in unintentional behavior.

When the core is used to boot a separately generated boot stream for another core, the `ROM_BFLAG_RETURN` flag should be used when calling `adi_rom_Boot()` to instruct the boot kernel to return to the calling application, which in this case is back to the core that was running the second stage loader. Failure to set this flag results in the core vectoring to the location in its own `RCU_SVECT0` register which is not be the intention.

By default when implementing a scheme such as previously described where one processor is responsible for booting images for another core, the other cores in the system will remain in their existing idle state that they should be placed in prior to the boot commencing. To allow the cores to execute new application, the core that is responsible for booting the other cores must reset the cores via the `RCU_CRCTL` then release them again from reset. If there is a requirement to release the core from the reset state, then this must be done within the application code of the running core.

Releasing other cores from reset to run application software while other processors are running a boot process requires careful system design. The drivers used by the boot kernel for the boot process assume the various peripheral and infrastructure resources such as MDMA channels and peripherals are available for use, and are not being used by another core. If the boot routine is being executed while other cores are running applications, then those applications must ensure that all the required boot resources are freed up and remain free in order for the boot process to complete on the remaining cores.

Secure Debug Access

The TAPC controller provides a way to restrict access to secure resources of the processor. Secure access through the debug port is protected through a 256-bit security key that must match a key that has been loaded into OTP for access.

To access a locked processor, the TAPC must allow access to the part. The TAPC only allows access to the part if it is provided with a matching key to the data loaded into its `TAPC_USERKEYn` registers.

With the processor in a locked state, on initial boot the boot ROM reads the 256-bit `secure_emu_key0/secure_emu_key1` from the OTP memory and programs the key into the `TAPC_SDBGKEY0` through the `TAPC_SDBGKEY7` registers before then setting the `TAPC_USERKEY_CTL.USERKEY_VALID` bit. The TAPC is then able to access a matching outside key to allow access.

CAUTION: A 16 bit non-zero value programmed on the `emu_key0_disable` field and `emu_key1_disable` fields in OTP OR a key of 0xFFFFFFFF, 0xFFFFFFFF F, 0xFFFFFFFF F, 0xFFFFFFFF F, 0xFFFFFFFF F, 0xFFFFFFFF F, 0xFFFFFFFF F, 0xFFFFFFFF F programmed in secure debug key field in OTP results in the boot code bypassing the key load operation entirely. If debug access is then ever required the key must be loaded to the TAPC by the program. If the processor fails to boot (perhaps due to corrupted firmware), there is no debug access. The only way to gain access is to load an authenticated boot image that can then load the required keys prior to attempting to connect with a debugger

The key is set in OTP using the OTP boot program API to program the `secure_emu_key0/secure_emu_key1`. This key is read and loaded by the ROM in the following sequence:

1. Bits 31:0 of the key in OTP are stored to `TAPC_SDBGKEY0` bits 31:0
2. Bits 63:32 of the key in OTP are stored to `TAPC_SDBGKEY1` bits 31:0
3. Bits 95:64 of the key in OTP are stored to `TAPC_SDBGKEY2` bits 31:0
4. Bits 127:96 of the key in OTP are stored to `TAPC_SDBGKEY3` bits 31:0
5. Bits 159:128 of the key in OTP are stored to `TAPC_SDBGKEY4` bits 31:0
6. Bits 191:160 of the key in OTP are stored to `TAPC_SDBGKEY5` bits 31:0
7. Bits 223:192 of the key in OTP are stored to `TAPC_SDBGKEY6` bits 31:0
8. Bits 255:224 of the key in OTP are stored to `TAPC_SDBGKEY7` bits 31:0

Once the ROM has loaded the user key, a test key can be provided to the TAPC through JTAG. Refer to the Emulator manual for details for providing the key.

A key failure indication can be detected through the `TAPC_SDBGKEY_STAT` register. The boot code does not check the key status, nor does it enable any associated interrupts to signal key failure. The boot code continues to boot upon a key failure in a secure manner. The key failure status remains intact so that the application loaded can check for a failed challenge on the debug port.

The boot code can be configured to bypass the loading of the key during the boot sequence by setting the value of the `secure_emu_key0` or `secure_emu_key1` in the OTP to all ones (see struct `otp_data`). In this case, the only way to gain access to the secure resources through the debug port is to load an alternate key using the application. The alternative key must always reside in a secure region of memory. Or, if sent remotely, it should be transmitted over a secure connection.

NOTE: The boot ROM supports two 256-bit secure debug keys, `secure_emu_key0` and `secure_emu_key1`, where only one of them is used by the boot ROM at a time. Each of the secure debug keys is associated with a bypass key (`emu_key0_disable` or `emu_key1_disable`) which decides the validity of the corresponding key. Any non zero value written into the 16-bit bypass key invalidates that secure debug key.

Anti Rollback NV Counter

Anti-rollback counter to prevent loading of older firmware during secure boot. Boot ROM implements the counter through OTPC module to allow easy reading or writing of the counter without dealing with the complexities of rewriting OTP memory sections that are ECC protected.

The implementation involves two functional APIs for the counter, which are functionally unique in the way that they set and retrieve data as counters in OTP memory.

It uses a different method to count bits because each bit in OTP memory can only be set =1 once, and the ECC protects each 16-bit unit. This functionality essentially means that each 16-bit unit can only be written to once. Therefore, a counter that can count 0–31 requires 32×16 bits of memory.

The counter API receives and returns the value of the counter as a uint8_t binary number. Providing a value less than the current value of the counter or greater than the maximum value results in an error. A greater value will result in updating the counter value equal to the one provided.

To implement this functionality, the driver counts by shifting 1's from the right, treating each block as 1 bit. A three-bit counter is encoded as follows.

Bit 2	Bit 1	Bit 0	Value
0000	0000	0000	0
0000	0000	0001	1
0000	0001	0001	2
0001	0001	0001	3

NOTE: A non-zero value should be written in the ROM_OTP_antiroll_nv_cntr field on OTP to enable this feature.

Boot ROM Errors and Failures

Any errors encountered during the boot process result in the boot ROM jumping to the [Error Handler](#) with a specific error code written to the RCU_MSG register. This includes decryption failures, authentication failures, and configuration errors. The error can be a boot specific error or product specific error. The *Boot ROM Exceptions* table shows the different types of errors supported by the boot ROM.

WARNING: In secured booting, an incorrect secure debug key supplied during boot does not cause a boot failure and the processor continues to boot as normal. A program that supplies an incorrect key is unable to gain access to any secure resources of the processor.

Table 36-45: Boot ROM Exceptions

Error Name	Error ID	Description
ROM_BOOT_ERR_GENERAL	0x0	General boot error

Table 36-45: Boot ROM Exceptions (Continued)

Error Name	Error ID	Description
ROM_BOOT_ERR_UNSUPPORTED	0x1	Unsupported configuration (device/mode selection)
ROM_BOOT_ERR_BMODE_INIT	0x2	Initialization failure for the selected boot mode
ROM_BOOT_ERR_BMODE_CONFIG	0x3	Configuration failure for the selected boot mode
ROM_BOOT_ERR_BMODE_HOOK	0x4	Hook function failure
ROM_BOOT_ERR_BMODE_REG_FAILURE	0x5	BMODE registration failure
ROM_BOOT_ERR_SEC_AUTH_FAIL	0x6	Authentication failure
ROM_BOOT_ERR_SEC_DECRYPT	0x7	Decryption failure
ROM_BOOT_ERR_ID	0x8	The image being loaded has a lower ID than was previously recorded, indicating image ID is outdated
ROM_BOOT_ERR_CLEANUP	0x9	Cleanup failure
ROM_BOOT_ERR_HDRSGN	0xA	Block header signature failure
ROM_BOOT_ERR_HOOK	0xB	Hook function failure
ROM_BOOT_ERR_HDRCHK	0xC	Header checksum failure
ROM_BOOT_SECURE_ERR_HDRCHK	0xD	Secure header failure
ROM_BOOT_ERR_NOFINAL	0xE	No final block present
ROM_BOOT_ERR_BLKFILL	0xF	Failure to process fill block
ROM_BOOT_ERR_BLKCRC	0x10	Failed CRC for a block
ROM_BOOT_ERR_DMA	0x12	DMA transfer failure
ROM_BOOT_ERR_CALLBACK	0x13	Callback failure
ROM_BOOT_ERR_NEXTDXE	0x14	Next dxs failure
ROM_BOOT_ERR_PLLCFG	0x15	PLL configuration failure
ROM_BOOT_ERR_INT	0x16	Internal ROM error
ROM_BOOT_ERR_NOKERNEL	0x17	Application returned to the bootrom
ROM_BOOT_ERR_OTPREAD	0x18	Failure to read OTP
ROM_BOOT_ERR_NODEVICE	0x19	No device selected in the bootCommand
ROM_BOOT_ERR_NOBMODE	0x1A	No boot mode selected
ROM_BOOT_ERR_BMODEDIS	0x1B	Boot mode is disabled
ROM_BOOT_ERR_NOPUBKEY	0x1C	No public key available

Table 36-45: Boot ROM Exceptions (Continued)

Error Name	Error ID	Description
ROM_BOOT_ERR_NOPVTKEY	0x1D	All private keys have been disabled
ROM_BOOT_ERR_SPICFG	0x1E	Failed SPI configuration
ROM_BOOT_ERR_UARTCFG	0x1F	Failed UART configuration
ROM_BOOT_ERR_SECURITY_FAILURE	0x20	Failure to initialize secure features
ROM_BOOT_ERR_ECDSATYPE	0x21	Wrong ECDSA type detected other than ECDSA- 224/256
ROM_BOOT_ERR_OTPLOCKBIT	0x22	Error in OTP area holding the security bits
ROM_BOOT_ERR_INVALID_FAEN	0x23	Error due to the failure analysis enable bit being set in open part
ROM_BOOT_ERR_SEC_HASH_COMPARE_FAIL	0x24	Error due to hash compare fail between pre-Hash and computed hash
Reserved	0x25	Reserved
ROM_BOOT_OOBVAL_ERROR	0x26	Out of bound values for variables/operation detected
ROM_BOOT_ERR_OTPBOOT_USER	0x27	OTP user boot failure
ROM_BOOT_ERR_SH_RCU_SVECT_IRQ	0x28	SHARC RCU SVECT interrupt
ROM_BOOT_ERR_SH_L1ERR_IRQ	0x29	L1 memory error
ROM_BOOT_ERR_SH_ILOPI_IRQ	0x30	Illegal opcode error
ROM_BOOT_ERR_SH_CB7I_IRQ	0x3A	Software stack (circular buffer 7) overflow
ROM_BOOT_ERR_SH_IICDI_IRQ	0x3B	Unaligned LW/BW access + unintentional CMMR/SMMR access
ROM_BOOT_ERR_SH_SOVFI_IRQ	0x3C	Status loop or mode stack overflow; or PC stack full
ROM_BOOT_ERR_SH_ILADI_IRQ	0x3D	Illegal address space detected
ROM_BOOT_ERR_SH_IIR2_IRQ	0x3E	IIR error
ROM_BOOT_ERR_SH_IIR3_IRQ	0x3F	IIR error
ROM_BOOT_ERR_SH_TMZHI_IRQ	0x40	Core timer error
ROM_BOOT_ERR_SH_BKPI_IRQ	0x41	User hardware breakpoint
ROM_BOOT_ERR_SH_FIR_IRQ	0x42	FIR error
ROM_BOOT_ERR_SH_IIR0_IRQ	0x43	IIR error
ROM_BOOT_ERR_SH_SECI_IRQ	0x44	System event controller interrupt
ROM_BOOT_ERR_SH_IIR1_IRQ	0x45	IIR error

Table 36-45: Boot ROM Exceptions (Continued)

Error Name	Error ID	Description
ROM_BOOT_ERR_SH_DAI0SPT1	0x46	DAI0SPT1 error
ROM_BOOT_ERR_SH_DAI1SPT0	0x47	DAI1SPT0 error
ROM_BOOT_ERR_SH_DAI1SPT1	0x48	DAI1SPT1 error
ROM_BOOT_ERR_SH_RINSEQI_IRQ	0x49	Restricted instruction sequence
ROM_BOOT_ERR_SH_CB15I_IRQ	0x4A	Circular buffer 15 overflow
ROM_BOOT_ERR_SH_TMZLI_IRQ	0x4B	Core timer (low priority option)
ROM_BOOT_ERR_SH_FIXI_IRQ	0x4C	Fixed-point overflow exception
ROM_BOOT_ERR_SH_FLTOI_IRQ	0x4D	Floating-point overflow exception
ROM_BOOT_ERR_SH_FLTUI_IRQ	0x4E	Floating-point underflow exception
ROM_BOOT_ERR_SH_FLTII_IRQ	0x4F	Floating-point invalid exception
ROM_BOOT_ERR_SH_EMULI_IRQ	0x50	Emulator low priority interrupt
ROM_BOOT_ERR_SH_SFT0I_IRQ	0x51	User software interrupt 0
ROM_BOOT_ERR_SH_SFT1I_IRQ	0x52	User software interrupt 1
ROM_BOOT_ERR_SH_SFT2I_IRQ	0x53	User software interrupt 2
ROM_BOOT_ERR_SH_SFT3I_IRQ	0x54	User software interrupt 3
ROM_BOOT_ERR_OTPBOOT_IRQ	0x55	OTP boot exception
ROM_BOOT_ERR_OTP_INVALID-STATE_IRQ	0x56	OTP invalid state interrupt

Failure Analysis

The boot ROM supports failure analysis on locked parts. Calling `adi_rom_otp_fa_enable()` activates this feature. The user must activate this feature on a locked part before sending it for failure analysis. Without activation, failure analysis on a locked part is impossible due to security reasons. Do not enable this feature on open parts because it results in boot failure.

Boot ROM Programming Model

This section describes the programming model for booting the processor. The programming model includes booting functions, API calls, and data structures.

Boot Mode Driver API

The kernel provides a mechanism to provide a customization of supported boot modes or for implementation of completely new boot modes as second stage boot loaders. This allows programs to customize booting while still taking advantage of the rest of the booting framework. A custom boot mode may provide support for a peripheral that

is not supported for boot by the ROM, or it could support one of the same peripherals but with a different configuration.

All the same security features can be supported when using a custom boot mode.

A full boot mode, as perceived by the boot implementation, is a collection of following functions:

1. Register – installs the driver functions listed below so they can be accessed by the boot process
2. Initialization – initialize the boot source
3. Configuration – configure the boot source
4. Load – read from the boot source
5. Cleanup – called after booting

The boot kernel is only aware of functions 2 through 5 and has a requirement to support the Load function. It is this function that is responsible for fetching the boot stream from the boot peripheral. The other functions are used prior to executing the kernel or for cleaning up after the kernel has completed processing the boot stream.

To install a custom boot mode:

1. Create a first stage boot application to define a Load function
2. Use the [adi_rom_BootKernel\(\)](#) API to call the boot kernel once the boot peripheral and pinmuxing has configured. Ensure all the fields of [struct ADI_ROM_BOOT_CONFIG](#) are configured accordingly prior to performing the call.

The boot mode can use the `pModeData` member of `ADI_ROM_BOOT_CONFIG` to preserve and access shared data across the different function calls if required.

All functions have the following prototype:

```
void apiFunction(ADI_ROM_BOOT_CONFIG* pBootStruct);
```

Another way to support custom boot mode is:

1. Create a first stage boot application to define all Init, Config, Load and Cleanup routines.
2. Use `adi_rom_Boot()` API with hook function installed to update the Init, Config, Load and Clean up functions after the preregister initialization is complete. This is checked by boot ROM to override the above functions inside the `ROM_HOOK_CALL_CAUSE: ROM_HOOK_REG_COMPLETE`

Load Function

The load function is required to read data from the source into the specified destination, according to the parameters given through the configuration `struct` parameter. The structure provides all of the required information read from the block header, or specified by the kernel to read the block header. The load function often makes use of the DMA APIs in order to simplify the load function implementation.

As the kernel processes the stream, it calls the load function to request data. Initially, the request is for the header, then the kernel makes requests according to the block flags it parses. The load function must only read from the device, and write where requested.

Relevant fields within the `ADI_ROM_BOOT_CONFIG` object for the load function are (but not limited to): `uwDataWidth`, `pSource`, `dByteCount`, `pDestination`, `loadType`.

Custom load functions must meet the following requirements.

- Protect against `dByteCount` values of zero
- Use multiple DMA units if `dByteCount` is greater than 65536 and the peripheral does not support byte count transfers greater than 65536
- The `pSource` and `pDestination` pointers must be properly updated after loading.

In target boot modes, the boot kernel uses the address of the `dArgument` field in the `pHeader` block as the destination for the required dummy DMAs when payload data is consumed from `ROM_BFLAG_IGNORE` blocks. The load function must read the `ARGUMENT` word of the block early in the function (if required).

Initialization/Configuration Function

The Initialization and Configuration functions are called in sequence when calling a boot operation using an already supported boot peripheral via the `adi_rom_Boot()` API. These functions are used to configure the boot peripheral prior to calling the boot kernel. Both functions are called in sequence separated only by a call to a user-defined hook function. This hook function is useful when using built-in boot modes to further customize their function. The initialization and configuration functions are responsible for applying any required settings to any devices in use. For example, pin multiplexing may be required and data or pointers that are used by the load function must be initialized. The specific actions depend on the device and functionality used.

Cleanup Function

The cleanup function is called after the entire boot stream is read, and the kernel has completed its boot mode-specific function. This is only performed when using the `adi_rom_Boot()` API. Resetting any status registers, or device parameters is done to prepare the environment for the execution of the newly loaded application.

Error Handler

This section describes the default error handler for the ROM including information on how to customize the error handling.

The default error handler eventually puts the core into an idle state. This functionality can be overridden by using an Init Block (see [Block Types](#)) to modify the error function point in the `struct ADI_ROM_BOOT_CONFIG` structure. The error handler has access to the entire boot info structure and receives the instruction address that triggered the error.

When a part is locked, and the boot type has not disabled secure boot, only the default error handler is called.

The expected prototype is:

```
void ErrorFunction(ADI_ROM_BOOT_CONFIG* pBootStruct, void *pFailingAddress);
```

The error handler saves the failing address to the ADI_ROM_BOOT_CONFIG structure then raises the INTR_SOFT3 fault signaling a fault condition to the system before then entering an endless loop in the boot rom.

NOTE: When using the [adi_rom_Boot\(\)](#) function to perform a boot action, programs may need to manually configure the INTR_SOFT3 fault signaling depending on the previous application software executed. Calling the boot process using `adi_rom_Boot()` does not result in a reset of the SEC and Fault installation and configuration as described in [Preboot Operations](#).

Page Mode

Page operations are beneficial for page oriented boot source devices, and to improve boot performance for secure boot operations. Page mode optimizes memory reads for block organized devices by always reading a page, rather than reading data on demand. Two 1024 byte buffers are used in page mode where the contents of one buffer are processed by the boot kernel while DMA is used to load the next data into the second buffer.

Blocking DMA is used to load the active buffer, forcing the process to pause until the DMA is complete. Non-blocking DMA is used to load the not active buffer, allowing the active buffer to process while loading the new data in parallel.

Page mode can be enabled when calling a boot mode using `adi_rom_Boot()`. Refer to the API documentation for the various modes supported by this API. Additionally programs can set the flag using the `dFlags` function (see [struct ADI_ROM_BOOT_CONFIG](#)) when using hook functions

NOTE: Do not customize page mode settings from the default installed by the boot process.

Boot Hook Function

The boot software allows installation of callback hooks through the use of the [adi_rom_Boot\(\)](#) API's hook function parameter. By using this feature, it is possible to alter the state of the processor, at different stages of the boot process and customize the boot structures to alter the behavior of the boot process.

The hook function must adhere to the following prototype:

```
int32_t
hookFunction(ADI_ROM_BOOT_CONFIG* pBootconfig,
ROM_HOOK_CALL_CAUSE cause);
```

By modifying settings in the ADI_ROM_BOOT_CONFIG structure, many alterations of the boot process can be achieved. Much of the same functionality that is available in an Init Block can be provided through the hook function, with even more flexibility for customization. The hook function is called once after executing the boot modes Init routine then once again after executing the boot modes Config routine. A flag passed to the hook function allows software to determine at which point the call took place to allow for conditional processing to occur at different stages of the setup phase.

The hook function must return a zero value for normal booting to continue. A non-zero return value causes the ROM to omit loading any data and immediately transfer control according to [Boot Termination and Application Execution](#).

When the hook function is called, a parameter is passed that indicates why the hook function was called. See [enum ROM_HOOK_CALL_CAUSE](#) for more details.

enum ROM_HOOK_CALL_CAUSE

Enumeration Type Declaration: ROM_HOOK_CALL_CAUSE

Passed to a user hook routine to indicate the reason of the call.

An optional hook routine is provided as a callback when calling a boot mode via `adi_rom_Boot`. This hook routine is called by the boot software first after the execution of the boot modes initialization routine then again after execution of the boot modes configuration routine. This parameter allows the users routine to identify at which point the call was made allowing the user to perform different actions for each call.

Table 36-46: ROM_HOOK_CALL_CAUSE Members

Enumerator	Description
ROM_HOOK_CALL_INIT_COMPLETE	Call was as a result of completion of the boot modes initialization function
ROM_HOOK_CALL_CONFIG_COMPLETE	Call was as a result of the completion of the boot modes configuration function
ROM_HOOK_REG_COMPLETE	Call was as a result of the completion of the boot modes pre-register initialization

Boot Return Feature

The `adi_rom_Boot()` API provides a feature to bypass calling the loaded application when boot completes, and to return to the routine that made the call instead. The boot software returns the next address after the last loaded application block in the boot source when this feature is enabled.

To enable this feature, set the ROM_BFLAG_RETURN flag in the `adi_rom_Boot()` flags argument when calling the API.

Boot Termination and Application Execution

When the boot kernel completes the processing of the boot stream, a sequence of events is required to then pass control to the loaded application.

When the boot process is complete, the core is required to vector to the application start address and start executing the newly loaded application. Typically, the first block of a boot stream, which is marked with the BFLAG_FIRST flag, contains the address of the application. The application entry point for each core is loaded into the cores corresponding RCU_SVECTn register.

Upon boot completion only the core that performed the boot process will vector and start executing the loaded application. This core must then manage the process of resetting then releasing from the reset the other cores in the system in order to make them execute their newly loaded applications.

Execution of the loaded application can be bypassed when calling the boot mode using [adi_rom_Boot\(\)](#) and setting the ROM_BFLAG_RETURN flag.

Table 36-47: Application Entry Point Registers

Core ID	Corresponding RCU_SVECTn Register
0	RCU_SVECT0

Boot ROM OTP Customizations

The boot ROM provides a mechanism through available non-volatile programmable memory (OTP on this processor) to customize different aspects of the boot process. These customizations include: overriding default boot-peripheral instance, overriding default peripheral-timing parameters and disabling boot modes.

Data in OTP memory controls all ROM customization. The [struct ADI_ROM_OTP_BOOT_INFO](#) data structure accounts for most of the options.

CGU Initializations

Refer to [CGU Configuration](#)

Boot Command Customization

Refer to [Boot Command Customization](#)

Secure Boot Customization

All the public and private keys can be invalidated using the various key invalidation fields provided in the [ADI_ROM_OTP_BOOT_INFO](#) structure. This configuration is useful when a new key is required. The boot ROM always uses the lowest valid key enumeration. If key0 is valid, then it is used, if key0 is invalid and key1 is valid, then key1 is used. Refer to [Secure Boot](#) for details on the secure boot functionality.

Disabling Boot Modes

Refer to [Boot Mode Disable](#)

API Reference

The APIs defined in this section are exposed for widespread use.

adi_rom_Boot()

Provides access to boot an application at run-time through a supported peripheral.

API Details

```
void * adi_rom_Boot(
    void * pAddress,
    uint32_t flags,
    int32_t blockCount,
    ROM_BOOT_HOOK_FUNC * pHook,
    uint32_t command, uint32_t command1
)
```

pAddress

Pointer to source address of the boot stream.

flags

Global flags applied to the entire boot process

blockCount

Number of blocks to boot. Zero results in processing until a final block is reached.

pHook

Pointer to user implemented hook function for enabling callbacks during the registering of the boot mode with the boot kernel

command/command1

The boot command defining the boot mode to use, the peripheral instance to boot from as well as some boot mode specific configuration

Returns

The 32-bit address of the next address in the boot source to process

Function Description

This function is used for any kind of second-stage boot for an already supported boot mode. It provides options to boot from any peripheral enumeration. In SPI controller boot, the function supports using any SPI target select signal.

Boot modes may support an auto-detection mechanism to identify the type of connected device. This function provides options to bypass such auto-detection and use custom configuration options. Options are also provided to bypass peripheral configurations such as pinmux settings or a peripheral configuration if an existing peripheral is more appropriately configured to allow communication with the boot source.

These features are all provided via the command parameter which is specific for each particular boot mode.

The source address of the boot stream is required for controller boot modes that require that an address is issued in order to request data from the boot source. Target boot modes are under full control of the host and use a handshake mechanism to indicate that the processor is ready to receive data. For boot modes such as UART target and SPI

target, this parameter is of little value in regards to the boot process itself. However, it can prove useful in debug to see how far through the boot stream the boot process progressed in the event of a boot failure.

NOTE: The processor supports both SPI memory-mapped boot as well as peripheral-based SPI boot. When the boot mode is called to boot from the memory-mapped boot mode via the command argument, the address must coincide with the processors memory-mapped SPI address space as defined by the processors internal memory map. When using the peripheral based boot mode, use the absolute address of the boot stream in flash.

Flags passed using the flags argument are *global* flags and the operation is applied throughout the entire boot process. These must not be confused with the boot *block specific* flags which are part of the boot stream and indicate how a particular block in the boot stream is processed. Internally, the boot kernel takes the global flags supplied using this function call and combines them with a boot block's local flags to determine all the operations to perform on a given block. After processing the boot block, the local flags are cleared and are ready to populate from the next boot block while the global flags remain.

Table 36-48: Global Flags

Bit Position	Flag Name	Description
18	ROM_BFLAG_HOOK	Calls the application supplied hook function after execution of bootmode init and config routines
19	ROM_BFLAG_PAGEMODE	Enables page mode processing where blocks of data are fetched and processed from internal memory
20	ROM_BFLAG_NOFIRSTHEADER	Set if calling the boot mode and the first block header has already been fetched and is present in the block header storage location
21	ROM_BFLAG_HEADER	Not set by the application, set by the boot code each time it is fetched by a block header
22–25	Reserved	
26	ROM_BFLAG_SLAVE	Target boot mode. Causes different handling of ignore blocks by the kernel
28	ROM_BFLAG_NEXTDXE	Parse stream via next DXE pointer
29	ROM_BFLAG_RETURN	Returns the application after calling the API instead of running the new application
30	Reserved	
31	ROM_BFLAG_NORESTORE	Do not execute the boot peripherals cleanup routine to restore register contents

The `blockCount` argument specifies the number of blocks to process before terminating the boot process (default = 0x00000000). This value instructs the boot software to continue processing a boot stream until the `ROM_BFLAG_FINAL` flag is set. To load only a specified number of blocks, programs can use `blockCount` argument.

When `blockCount` is used in combination with the `ROM_BFLAG_NEXTDXE` flag then the block count is repurposed as a next application count. The boot kernel navigates the first blocks of multiple boot streams similar to a

linked list. When the requested application count is reached, the program returns the pointer to this application in the boot source. This allows programs to use the boot kernel to find a specific application when multiple application boot streams are stored contiguously in the boot source.

The `pHook` argument is a function pointer to a hook routine. When used with the `ROM_BFLAG_HOOK` global flag the boot mode calls the hook routine after calling the boot modes `init` and functions in the boot mode's driver so that the boot software can be customized and the configuration changed.

The `pHook` argument when used with the `ROM_BFLAG_HOOK` global flag can enable new features not supported by the boot software and to install a custom load function or error handler for example.

The `command` argument describes the boot peripheral to boot from, the peripheral instance and contains additional boot mode specific configuration information and flags specific to the boot mode.

NOTE: When calling a boot mode via this API the program must first ensure that the boot peripheral is configured in the SPU as a secure controller. The boot software does not configure the peripheral security via this API so that device security is fully controlled by a dedicated task.

adi_rom_BootKernel()

Calls the boot kernel allowing for implementation of custom boot modes.

API Details

```
void * adi_rom_BootKernel (ADI_ROM_BOOT_CONFIG * pBoot)
```

pBoot

Pointer to the `struct ADI_ROM_BOOT_CONFIG` boot structure containing the complete context of the boot configuration.

Returns

Pointer containing the address of the byte immediately following the end of the boot stream.

Function Description

The boot kernel performs the core processing of the boot stream. The boot kernel calls a load function to load data from the peripheral to the required destination. The boot kernel itself has no concept of what the boot peripheral is or how that peripheral is configured. The kernel calls the registered load function and the load function must then analyze the boot structure and provide the requested amount of data to the required destination.

The load function called by the kernel is provided via the `struct ADI_ROM_BOOT_REGISTRY` member of `struct ADI_ROM_BOOT_CONFIG`.

The boot kernel fetches a boot stream block header then a payload if one exists. The boot kernel takes care of the size of the data being requested and the destination address.

The load function that is registered with the kernel is required to update the `pSource`. Keeping this control under the load function as opposed to the boot kernel itself allows load functions to better control where the next block of data is fetched in the event the boot stream is fragmented or split into different areas of the boot source.

This function is used to implement a second stage boot loader for a peripheral in which there is no driver support in the boot ROM. The application is responsible for initializing the peripheral and the complete `struct ADI_ROM_BOOT_CONFIG` object before calling this function. The application is responsible for performing a vector to the newly loaded application on return from the function.

NOTE: Programs must ensure that when a new application is loaded it does not clobber the load function and the part of the software responsible for making the core jump to the start of the newly loaded application.

adi_rom_Crc32Init()

The CRC32 Initcode function in the boot rom that is called to enable CRC32 support of boot stream payloads.

API Details

```
ROM_BOOT_RESULT adi_rom_Crc32Init(ADI_ROM_BOOT_CONFIG * pBootConfig)
```

pBootConfig

Pointer to the `struct ADI_ROM_BOOT_CONFIG` object containing the complete boot configuration

Returns

Returns the following results

- `ROM_BOOT_RESULT::ROM_BOOT_CRC_INITCODE_ERR` when `pBootConfig` or `pBootConfig->pHeader` are zero
- `ROM_BOOT_RESULT::ROM_BOOT_SUCCESS` when the callback is registered and lookup table initialized

Function Description

The boot process supports CRC32 protection of all boot block payloads. In order to enable this feature a global callback must be registered with the boot process using `struct ADI_ROM_BOOT_CONFIG` and the CRC peripherals look up table initialized from the application's polynomial.

An init block header where the `ROM_BFLAG_INIT` flag is set must be included in the boot stream to enable CRC functions. The `ADI_ROM_BOOT_HEADER::pTargetAddress` field must be set to the address of this function and the application polynomial is provided using the block's `struct ADI_ROM_BOOT_HEADER` member.

When the boot kernel processes the init block it calls this function in the boot ROM and registers the callback with the kernel and performs the look up table initialization.

CRC functionality is enabled on MDMA channel 1 interfaced to the CRC0 peripheral instance.

adi_rom_Crc32Poly()

Initializes the CRC peripheral for use with the user supplied polynomial.

API Details

```
ROM_BOOT_RESULT adi_rom_Crc32Poly(
    uint32_t CrcPoly,
    ROM_BOOT_MDMA_REGS const *const pDma
)
```

CrcPoly

None

pDma

None

Function Description

The CRC lookup table must be initialized for the CRC polynomial of choice to prepare the CRC peripheral for use.

adi_rom_GetAddress()

Used to find the location of various look-up tables and data objects used during the boot process.

API Details

```
int32_t adi_rom_GetAddress(ROM_GETADDR_VALUE value)
```

value

The [enum ROM_GETADDR_VALUE](#) enumeration specifying the object to retrieve the address of in the ROM memory

Returns

The byte address of the object in memory

Function Description

The function returns the address of the object specified by the enumerator provided as an argument to the function. Using this function can make software more code compatible with future products and silicon revisions.

adi_rom_MemCompare()

Verifies that a block of data is filled with a application supplied 32-bit value.

API Details

```
ROM_BOOT_RESULT adi_rom_MemCompare(
    ROM_DMA_MDMA_CONFIG * pDmaCfg,
```

```
ROM_BOOT_MDMA_REGS const *const pDma
)
```

pDmaCfg

None

pDma

None

Function Description

The CRC peripheral is used in compare mode and a source MDMA channel is used to read data from a buffer and supply each 32-bit value to the CRC. The CRC peripheral checks that the incoming 32-bit value matches the 32-bit value to compare against.

adi_rom_MemCopy()

Performs a Memory-to-Memory DMA (MDMA) operation using a source and destination pair of DMA channels.

API Details

```
ROM_BOOT_RESULT adi_rom_MemCopy(
    ROM_DMA_MDMA_CONFIG * pDmaCfg,
    ROM_BOOT_MDMA_REGS const *const pDma
)
```

pDmaCfg

Pointer to the `struct ROM_DMA_PDMA_CONFIG` object containing the peripheral DMA configuration

pDma

Pointer to the `struct ROM_BOOT_MDMA_REGS` objects that provides access to the DMA channel MMRs and associated CRC peripheral

Returns

Returns the following results

- `ROM_BOOT_RESULT::ROM_BOOT_SUCCESS` for a successful operation or when byte count is 0 as no operation is performed
- `ROM_BOOT_RESULT::ROM_BOOT_DMA_FAILURE` if a configuration error was detected in the DMA channel prior to configuring the channels for the new operation
- if a configuration error occurred in the source MDMA channel
- if a configuration error occurred in the destination MDMA channel

Function Description

The memory copy routine performs transfers of blocks of data from one memory location to another. The routine takes a basic descriptor providing configuration details of the operation to perform using the `struct ROM_DMA_MDMA_CONFIG` object passed as the first argument. The second argument is a descriptor that provides access to the DMA channel's MMR registers and the associated CRC peripheral. When called from the higher level `adi_rom_MemDma()` routine this object is retrieved from the ROM.

NOTE: Applications are expected to make use the `adi_rom_MemDma()` routine for all MDMA operations as there is little additional optional configuration that is supported by using this routine.

adi_rom_MemCrc()

Performs CRC32 verification of a block of data by reading the contents and comparing them with an expected result.

API Details

```
ROM_BOOT_RESULT adi_rom_MemCrc(
    ROM_DMA_MDMA_CONFIG * pDmaCfg,
    ROM_BOOT_MDMA_REGS const *const pDma
)
```

pDmaCfg

Pointer to the `struct ROM_DMA_PDMA_CONFIG` object containing the peripheral DMA configuration

pDma

Pointer to the `struct ROM_BOOT_MDMA_REGS` objects that provides access to the DMA channel's MMRs and associated CRC peripheral

Function Description

The routine uses an MDMA channel pair source DMA channel and the CRC peripheral to calculate a CRC32 result of a data block using a previously supplied polynomial.

The polynomial is supplied through the `adi_rom_Crc32Poly()` routine to ensure consistent CRC peripheral configuration for the look up table initialization that uses the polynomial and the

NOTE: Applications are expected to make use the `adi_rom_MemDma()` routine for all MDMA operations, there is little additional optional configuration that is supported by using this routine.

adi_rom_MemDma()

Provides access to all MDMA operations supported by the boot ROM implementation.

API Details

```
ROM_BOOT_RESULT adi_rom_MemDma(ROM_DMA_MDMA_CONFIG * pDmaCfg)
```

pDmaCfg

Pointer to the `struct ROM_DMA_PDMA_CONFIG` object containing the MDMA configuration

Returns

Returns the following results using the enum `ROM_BOOT_RESULT` enumeration.

- `ROM_BOOT_SUCCESS`. Successful operation or when byte count is 0 as no operation is performed
- `ROM_BOOT_MDMA_ID_ERR`. A MDMA channel ID is provided that is not supported
- `ROM_BOOT_CRC_SUPPORTED_ERR`. A CRC operation was attempted on a MDMA channel that does not support CRC
- `ROM_BOOT_MDMA_OPERATION_ERR`. The operation is not supported
- `ROM_BOOT_DMA_FAILURE`. A configuration error is detected in the DMA channel prior to configuring the channel for the new operation
- `ROM_BOOT_DMA_FAILURE`. A configuration error was detected in the DMA channel and the operation requested involves only a single DMA channel
- A configuration error occurred in the source MDMA channel
- A configuration error occurred in the destination MDMA channel
- `ROM_BOOT_CRC_COUNT_ERR`. A CRC operation is being requested and the byte count is not a multiple of 4
- `ROM_BOOT_CRC_FAILURE`. A CRC32 verification fails
- `ROM_BOOT_CRC_FAILURE`. A 32-bit memory compare fails

Function Description

The supported MDMA operations are listed in `enum ROM_DMA_MDMA_OPERATION`.

NOTE: The MDMA and CRC peripherals support on-the-fly CRC32 calculations during the transfer of data from one location to another—MDMA in the boot ROM software does not. For CRC calculations data is instead read back from its final destination and verified.

This function is the main entry point for all the MDMA functions supported by the boot ROM software. The individual functions that are called for each operation type are also exposed using the public API.

NOTE: Use this function for all operations. The lower level functions allow for some basic reconfiguration of default parameters but such modifications are not be required where these basic MDMA operations are required.

The boot ROM has an MDMA configuration data structure that is used to specify the overall MDMA configuration of the processor. It provides details on the DMA channel ID associated with each MDMA channel's source and destination DMA channel. It provides information about CRC support and the CRC peripheral instance that is used for a given MDMA channel. See [struct ROM_BOOT_MDMA](#) and [struct ROM_BOOT_MDMA_REGS](#) for full details of the content stored.

The configuration provided as the only argument to the function is provided in [Table 36-66 ROM_DMA_MDMA_CONFIG Members](#).

For a basic MDMA transfer from source to destination the program needs to configure:

- The [struct ROM_DMA_MDMA_CONFIG](#) type as [enum ROM_DMA_MDMA_OPERATION](#)
- The MDMA channel to use via eId MDMA Channel ID for example
- Set the address of the source data via pSource Source Pointer
- Set the destination address of the source data via pDestination Destination Pointer
- Set the byte count via ByteCount Byte Count
- Set eDoneDetect DMA Done Detection Method to [enum ROM_DMA_DONE_DETECT_METHOD](#) in order to poll for DMA completion

NOTE: The implementation of the MDMA operations does not support interrupt driven data transfers. The routines are implemented for polling DMA status during the boot process. The boot stream is restricted where byte counts and source and destination address alignment must all be a multiple of 4 bytes. MDMA routines must comply with these restrictions.

To use the CRC32 function of the MDMA routines, the application must first initialize the CRC lookup table from the application supplied polynomial. This operation can be performed by setting [ROM_DMA_MDMA_CONFIG::eOperation](#) type as [ROM_DMA_MDMA_OPERATION::ROM_DMA_CRC_LUT_INIT](#). If an MDMA channel is specified that does not support CRC function, an error result is returned.

For further details of the individual operations supported see the following API references:

- [adi_rom_MemCopy\(\)](#)
- [adi_rom_MemCrc\(\)](#)
- [adi_rom_MemFill\(\)](#)
- [adi_rom_MemCompare\(\)](#)
- [adi_rom_Crc32Poly\(\)](#)

adi_rom_MemFill()

Fills a block of memory with a 32-bit user supplied value.

API Details

```
ROM_BOOT_RESULT adi_rom_MemFill(
    ROM_DMA_MDMA_CONFIG * pDmaCfg,
    ROM_BOOT_MDMA_REGS const *const pDma
)
```

pDmaCfg

None

pDma

None

Returns

Returns the following results

- ROM_BOOT_RESULT::ROM_BOOT_SUCCESS for a successful operation or when byte count is 0 as no operation to be performed
- ROM_BOOT_RESULT::ROM_BOOT_DMA_FAILURE if a configuration error was detected in the DMA channel prior to configuring the channel for the new operation
- A configuration error occurred in the source MDMA channel
- A configuration error occurred in the destination MDMA channel

Function Description

The CRC peripheral is configured for fill mode and the destination MDMA channel is configured to fill a block of memory with a fixed 32-bit value.

adi_rom_PeriphDma()

Provides access to any peripherals dedicated DMA channel for receive operations only.

API Details

```
ROM_BOOT_RESULT adi_rom_PeriphDma(ROM_DMA_PDMA_CONFIG * pDmaCfg)
```

pDmaCfg

Pointer to the `struct ROM_DMA_PDMA_CONFIG` object containing the peripheral DMA configuration

Returns

Returns the following results

- ROM_BOOT_SUCCESS for a successful operation or when byte count is 0 as no operation is performed
- ROM_BOOT_DMA_ACTIVE if the DMA channel is currently running
- ROM_BOOT_DMA_FAILURE if a configuration error was detected in the DMA channel after starting the DMA operation

Function Description

The peripheral DMA routine is used by the load routines of boot peripherals that have dedicated DMA channels and do not support MDMA channel pairs. Examples are the SPI when not configured for memory-mapped mode and UART peripherals.

In the boot implementation this routine is called from the peripheral load function to request data from the boot source. The routine supports both polling on DMA completion and non-blocking operation to allow for immediate return after starting the DMA operation and continuing with further processing.

NOTE: The function only supports read operations from the peripheral to memory. Transmit operations from memory to peripheral are not supported

callback()

Implements custom callbacks to previously loaded code during boot.

API Details

```
ROM_BOOT_RESULT callback(
    ADI_ROM_BOOT_CONFIG * pBootConfig,
    ADI_ROM_BOOT_BUFFER * pBuffer,
    uint32_t nFlags
)
```

pBootConfig

Pointer to the `struct ADI_ROM_BOOT_CONFIG` object containing the complete context of the boot procedure

pBuffer

Pointer to the `struct ADI_ROM_BOOT_BUFFER` object containing details of the payload associated with the callback

nFlags

The callback flags as set by the boot kernel

Function Description

A single callback function may be registered with the boot kernel using `pCallBackFunction` Pointer to the callback function that is called when processing boot blocks with the

`callback flag set` . This function is then called whenever a block is processed with the `ROM_BFLAG_CALLBACK` flag set. Only a single callback function can be registered for the complete boot process.

Callbacks may be used alongside indirect blocks when post processing of the received boot data is required before sending the data to the final destination. An example of this is compression applied to block payloads. The compressed payload is loaded indirectly to the intermediate buffer where it is decompressed by the callback. The callback can modify the source address and byte count for the final MDMA transfer of the decompressed payload using the `pBuffer` parameter. The callback returns the boot kernel then handles the final transfer of the uncompressed data to its destination.

There are restrictions on the amount of data that can be loaded using indirect blocks depending on the size of the intermediate buffer. For this reason the `nFlags` parameter is used to indicate the status of the callback when handling larger blocks of indirect data. The table below defines the supported flags:

Bit Position	Flag Name	Description
0	<code>ROM_CBFLAG_DIRECT</code>	When set indicates the call was from the processing of a block header with the <code>ROM_BFLAG_CALLBACK</code> flag set
1	<code>ROM_CBLAG_PAGESTART</code>	Indicates the callback was a result of a fetch of a page of data to the intermediate buffers
2	<code>ROM_CBFLAG_FIRST</code>	Set if the first fetch of payload data
3	<code>ROM_CBFLAG_FINAL</code>	Set if the final fetch of payload data
31:4	Reserved	Reserved

When a callback block header is received by the boot kernel, a call to the callback is performed with the `ROM_CBFLAG_DIRECT` flag set. If the `ROM_BFLAG_INDIRECT` flag or the `ROM_BFLAG_PAGEMODE` flags are set they indicate the use of indirect or page mode the `ROM_CBFLAG_FIRST` and `ROM_CBFLAG_FINAL` flags are cleared. If the transfer is a direct transfer straight to the final destination and not via the intermediate buffers, then the `ROM_CBFLAG_FIRST` and `ROM_CBFLAG_FINAL` flags are also set.

This allows software to identify a callback call based on the processing of a block header with the `ROM_BFLAG_CALLBACK` flag set.

Callbacks are also called when processing payloads indirectly or when page mode is enabled.

- The `ROM_CBFLAG_DIRECT` flag is cleared when the callback is a result of processing the payload data using the intermediate buffers.
- The `ROM_CBFLAG_FIRST` flag is set if the callback is a result of fetching the first block of data in the payload. This flag is also set if the complete block of data fits in the intermediate buffer.
- If the payload does not fit completely in the intermediate buffers multiple fetches must take place and thus multiple callbacks are generated.
- If no flags are set it indicates a callback on a payload transfer is neither the first nor the last block of data in the payload, so there is still further data in the payload to fetch.

- If only the `ROM_CBFLAG_FINAL` flag is set then it is the final block in a payload transfer.

The following table provides an overview of the flag states and their meaning for the processing of callbacks.

<code>ROM_CBFLAG_DIRECT</code>	<code>ROM_CBFLAG_PAGESTART</code>	<code>ROM_CBFLAG_FIRST</code>	<code>ROM_CBFLAG_FINAL</code>	Description
1	0	0	0	Callback as a result of processing a block header with indirect or page-mode enabled
1	0	1	1	Callback as a result of processing a block header with indirect and pagemode disabled
0	1	0	0	Callback as a result of fetching a page of data in pagemode
0	1	0	1	Callback as a result of fetching a page of data in pagemode and the final page in the block
0	0	1	0	Callback as a result of fetching the first part of payload in an indirect payload
0	0	0	0	Callback as a result of fetching an indirect payload, not first or last transfer in payload
0	0	0	1	Callback as a result of fetching the final part of payload in an indirect payload
0	0	1	1	Callback as a result of fetching the complete payload in an indirect payload

initcode()

Implements custom callbacks to previously loaded code during boot.

API Details

```
void initcode(ADI_ROM_BOOT_CONFIG * pBootConfig)
```

pBootConfig

Pointer to the `struct ADI_ROM_BOOT_CONFIG` object containing the complete context of the boot procedure

Function Description

Initcode functions are embedded into the boot stream to execute application code during the boot phase. Initcode functions help to optimally configure the CGU or any external memory interface that requires initialization to boot data to those memories.

A boot stream may have any number of initcodes present. The only requirement is that the code must be loaded prior to processing the BFLAG_INIT block.

The initcode routine is passed by the pointer to the complete boot context so that extensive boot customization tasks can be supported.

adi_rom_idle_loop()

Jumps to the IDLE loop inside the boot ROM space.

API Details

```
void adi_rom_idle_loop(void)
```

Function Description

This function is used to jump to the forever idle loop inside the boot ROM space.

adi_rom_CguInit()

Can be used for CGU initialization.

API Details

```
bool adi_rom_CguInit(const ADI_ROM_OTP_BOOT_CGU_INFO * const pSettings)
```

pSettings

Pointer to the struct `struct ADI_ROM_OTP_BOOT_CGU_INFO` object containing complete context of CGU initialization

Function Description

This function is used for CGU initialization.

adi_rom_ShaInit()

Initializes the PKTE module for the SHA-224/256 operation.

API Details

```
ROM_BOOT_RESULT adi_rom_ShaInit(CRYPTO_DESCRIPTOR *pCrypto, uint32_t  
IV[4], uint32_t Ecdsa_Word)
```

pCrypto

Pointer to Crypto descriptors

IV

Initialization Vector

Ecdsa_Word

ECDSA word count for 224/256 bit operation

Function Description

This API is used for the PKTE initialization for the SHA-224/256 bit operation

adi_rom_Sha()

Computes SHA-224/256 bit digest from the PKTE module.

API Details

```
adi_rom_Sha(CRYPTO_DESCRIPTOR * pCrypto, uint32_t Ecdsa_Word, void *output_p,
void *input_p, int size, bool finalFlag)
```

pCrypto

Pointer to Crypto descriptors

Ecdsa_Word

ECDSA word count for 224/256 bit operation

output_p

Pointer to the output buffer

input_p

Pointer to the input buffer

size

Size of the input buffer. It must be a multiple of 512

finalFlag

Final block or not

Function Description

This API is used to compute the message digest using SHA-224/256 using the PKTE module.

/* points to bootrom_jumtable_aes128_cbc_decrypt */

```
typedef ROM_BOOT_RESULT (*_bootrom_jumtable_aes128_cbc_decrypt_t)
(CRYPTO_DESCRIPTORs
    * pCrypto, void *output_p, void *input_p, int size);
inline static ROM_BOOT_RESULT
adi_rom_Aes128CbcDecrypt(CRYPTO_DESCRIPTORs * pCrypto,
    void *output_p, void *input_p, int size){
    _bootrom_jumtable_aes128_cbc_decrypt_t pTmp =
        (_bootrom_jumtable_aes128_cbc_decrypt_t)
(FUNC_ROM_AES128_CBC_DECRYPT);
    return (*pTmp)(pCrypto,output_p,input_p,size);
}
```

/* points to bootrom_jumtable_aes128_key_unwrap */

```
typedef ROM_BOOT_RESULT (*_bootrom_jumtable_aes128_key_unwrap_t)
(CRYPTO_DESCRIPTORs
    * pCrypto, void *WrappedIn_p, void *KeyOut_p, uint32_t KeyBits,
void *KWK_p);
inline static ROM_BOOT_RESULT
adi_rom_Aes128KeyUnwrap(CRYPTO_DESCRIPTORs * pCrypto,
    void *WrappedIn_p, void *KeyOut_p, uint32_t KeyBits, void *KWK_p){
    _bootrom_jumtable_aes128_key_unwrap_t pTmp =
        (_bootrom_jumtable_aes128_key_unwrap_t)
(FUNC_ROM_AES128_KEY_UNWRAP);
    return (*pTmp)(pCrypto, WrappedIn_p, KeyOut_p, KeyBits, KWK_p);
}
```

/* points to bootrom_jumtable_aes128_cbc_loadkey */

```
typedef void * (*_bootrom_jumtable_aes128_cbc_loadkey_t)(sa_t *
pSARRecord, uint32_t
    *key_p);
inline static void * adi_rom_Aes128CbcLoadkey(sa_t * pSARRecord,
uint32_t *key_p)
{
    _bootrom_jumtable_aes128_cbc_loadkey_t pTmp =
        (_bootrom_jumtable_aes128_cbc_loadkey_t)
(FUNC_ROM_AES128_CBC_LOADKEY);
    return (*pTmp)(pSARRecord,key_p);
}
```

/* points to bootrom_jumtable_pka_init */

```
typedef PKA_Status_t (*_bootrom_jumtable_pka_init_t)( PKA_IOArea_t *
const IOArea_p,
    Device_Handle_t Device,const uint32_t * const Firmware_p,const
uint32_t
```

```

        FirmwareWordCount);
    inline static PKA_Status_t adi_rom_PKA_Init( PKA_IOArea_t * const
IOArea_p,
        Device_Handle_t Device,const uint32_t * const Firmware_p,const
uint32_t
        FirmwareWordCount) {
    _bootrom_jumtable_pka_init_t pTmp =
        (_bootrom_jumtable_pka_init_t) (FUNC_ROM_PKA_INIT);
    return (*pTmp) (IOArea_p,Device,Firmware_p,FirmwareWordCount);
    }

```

/* points to bootrom_jumtable_ecdsa_verify_init */

```

    typedef ROM_BOOT_RESULT
        (*_bootrom_jumtable_ecdsa_verify_init_t) (SBHYBRID_EcdsaContext_t
* const Verify_p,
        const SBIF_ECDSA_PublicKey_t * const PublicKey_p,const
SBIF_ECDSA_Signature_t *
        const Signature_p,uint32_t EcdsaWord);
    inline static ROM_BOOT_RESULT
adi_rom_EcdsaVerifyInit(SBHYBRID_EcdsaContext_t * const
        Verify_p, const SBIF_ECDSA_PublicKey_t * const PublicKey_p,const
        SBIF_ECDSA_Signature_t * const Signature_p,uint32_t EcdsaWord) {
    _bootrom_jumtable_ecdsa_verify_init_t pTmp =
        (_bootrom_jumtable_ecdsa_verify_init_t)
(FUNC_ROM_ECDSA_VERIFY_INIT);
    return (*pTmp) (Verify_p,PublicKey_p,Signature_p,EcdsaWord);
    }

```

/* points to bootrom_jumtable_ecdsa_verify_set_digest */

```

    typedef void *
        (*_bootrom_jumtable_ecdsa_verify_set_digest_t)
(SBHYBRID_EcdsaContext_t * const
        Verify_p, uint8_t * Digest_p);
    inline static void *
adi_rom_EcdsaVerifySetDigest(SBHYBRID_EcdsaContext_t * const
        Verify_p, uint8_t * Digest_p) {
    _bootrom_jumtable_ecdsa_verify_set_digest_t pTmp =
        (_bootrom_jumtable_ecdsa_verify_set_digest_t)
(FUNC_ROM_ECDSA_VERIFY_SET_DIGEST);
    return (*pTmp) (Verify_p,Digest_p);
    }

```

/* points to bootrom_jumtable_ecdsa_verify */

```

    typedef SB_Result_t (*_bootrom_jumtable_ecdsa_verify_t)
(SBHYBRID_EcdsaContext_t *
        const Verify_p, uint32_t EcdsaWord);

```



```
inline static SB_Result_t adi_rom_EcdsaVerify(SBHYBRID_EcdsaContext_t *
const
    Verify_p, uint32_t EcdsaWord) {
    _bootrom_jumptable_ecdsa_verify_t pTmp =
        (_bootrom_jumptable_ecdsa_verify_t) (FUNC_ROM_ECDSA_VERIFY);
    return (*pTmp) (Verify_p, EcdsaWord);
}
```

Data Structures

The programming model for booting the processor uses the data structures defined in this section.

struct ADI_ROM_BOOT_BUFFER

Structure Type Declaration: ADI_ROM_BOOT_BUFFER

Boot Buffer.

A basic buffer type consisting of a pointer to the buffer and its size

Table 36-49: ADI_ROM_BOOT_BUFFER Members

Type	Name	Description
void *	pBuffer	Pointer to the buffer
int32_t	dByteCount	Size of the buffer

pBuffer

Pointer to the buffer

dByteCount

Size of the buffer

struct ADI_ROM_BOOT_CONFIG

Structure Type Declaration: ADI_ROM_BOOT_CONFIG

The Boot Configuration Object that contains all context for the boot process.

This structure contains the complete context for the boot process. A pointer to this object is passed through many routines and is presented to customizable routines such as initcodes, custom initialization, configuration, load and cleanup routines. The object is passed to error handlers and callbacks to customize and adapt the boot process for specific applications, especially in regards to multi-stage boot loader development.

Table 36-50: ADI_ROM_BOOT_CONFIG Members

Type	Name	Description
void *	pSource	Source address from where to fetch the next boot data.
void *	pDestination	Destination address to store the fetched data.
int32_t	dByteCount	Number of bytes to fetch from the boot source.
int32_t	dFlags	Control flags related to the boot kernel processing of blocks.
uint32_t	ulBlockCount	Limit of blocks processed during boot.
uint32_t	ulBlockCurrent	The number of blocks currently processed by the boot kernel
void *	pNextDxe	Pointer to the next application in the boot stream or the first free location after the boot stream.
uint32_t	uByteAddress	The destination address converted to the byte address space.
uint32_t volatile *	pControlRegister	Pointer to the boot peripherals control register.
int32_t	dControlValue	Storage for the boot peripheral main control value to enable that peripheral in a required configuration.
uint32_t volatile *	pPeripheralBase	Pointer to the boot peripherals base MMR address
uint32_t volatile *	pAuxControlRegister	Pointer to any register that may be used for auxiliary operations such as a timer control register for UART autobaud detection
uint32_t volatile *	pAuxPeripheralBase	Pointer to the base address of any peripheral used for auxiliary operations such as the TIMER block
uint32_t volatile *	pSecControlRegister	Base MMR address of the SEC SSI instance associated with the boot peripheral if required for advanced second stage boot loader development
ADI_DMA_TypeDef *	pDmaBaseRegister	Base MMR address of the DMA channel associated with the boot peripheral.
ROM_DMA_DONE_DETECT_METHOD	loadType	Set by the kernel to specify to the boot peripherals load function if it is requesting a blocking or non-blocking DMA
ROM_DMA_MDMA_CONFIG	MdmaCfg	An MDMA descriptor that is used by the boot kernel for internal MDMA operations.

Table 36-50: ADI_ROM_BOOT_CONFIG Members (Continued)

Type	Name	Description
uint16_t	uwDataWidth	The maximum data width supported by the boot peripherals DMA channel. Set to 0 for 8-bit, 1 for 16-bit and 2 for 32-bit
uint16_t	uwSrcModifyMult	The source modify multiplier used to set DMA_XMOD for source MDMA operations or peripheral DMA transmit operations
uint16_t	uwDstModifyMult	The destination modify multiplier used to set DMA_XMOD for destination MDMA operations or peripheral DMA receive operations
uint16_t	uwUserShort	Free to use
int32_t	dUserLong	Free to use
int32_t	dReserved0	Reserved for future use
void *	pModeData	Pointer to the boot mode specific data structure.
int32_t	dBootCommand	The boot command value supplied during the call to the <code>adi_rom_Boot()</code> routine.
uint32_t	dBootCommand1	The boot command value supplied during the call to the <code>adi_rom_Boot()</code> routine.
ADI_ROM_BOOT_HEADER*	pHeader	Pointer to the boot header storage location where all boot stream block headers eventually reside for processing by the kernel.
void *	pTempBuffer	Pointer to the internal intermediate buffer. Used for processing of indirect blocks.
void	dReserved1	Reserved
int32_t	dTempByteCount	Size of the internal intermediate buffer in bytes.
void *	pTempSource	Current source address that is processed in the internal intermediate buffer.
int32_t	dPageByteCount	The page size used for page mode processing. On this product page size is fixed to 1024 bytes and this member is not used for the load requests.
ADI_ROM_BOOT_INTER_BUFFERS	bootBuffers	The internal intermediate buffer descriptors required when using indirect and page mode features.
ROM_BOOT_REGISTRY	bootRegistry	The registry object that is used to register a boot peripheral routines with the kernel.

Table 36-50: ADI_ROM_BOOT_CONFIG Members (Continued)

Type	Name	Description
ROM_BOOT_ERROR_FUNC *	pErrorFunction	Pointer to the error handler called in the event of an error.
ROM_BOOT_CALLBACK_FUNC *	pCallBackFunction	Pointer to the callback function that is called when processing boot blocks with the call-back flag set.
ROM_BOOT_CALLBACK_FUNC *	pCrcFunction	Pointer to the CRC function that is used to perform CRC validation of the boot stream payload data.
ROM_BOOT_CALLBACK_FUNC *	pForwardFunction	Feature not supported on this product.
ADI_ROM_BOOT_MODES	bootModes	Access to all boot mode specific resources.
void *	pLogBuffer	Pointer to the log buffer. Logging is disabled by default on this product and must be configured from within initcodes or hook routines.
void *	pLogCurrent	The current position within the log buffer. Logging is disabled by default on this product and must be configured from within initcodes or hook routines.
int32_t	dLogByteCount	The size of the log buffer. Logging is disabled by default on this product and must be configured from within initcodes or hook routines.
ADI_ROM_TIMER_BENCHMARK_INFO	timerBenchmarkInfo	Access to all boot benchmarking resources.
ADI_ROM_OTP_XSPI_PHYCONFIG	xspiPHYconfig	Access to all xSPI PHY configuration resources.
ADI_ROM_OTP_BOOT_INFO*	pOtpBootInfo	Pointer to the ADI_ROM_OTP_BOOT_INFO boot information block that is read from OTP and contains boot customization options.
ADI_ROM_BOOT_KEY_TYPE	keyType	When set to a specific value allows keys not stored in OTP to perform secure boot evaluation on an open processor.
ADI_ROM_BOOT_TYPE	bootType	A key to indicate if the boot type is secure or non-secure for open parts.
ROM_SB_IMAGE_TYPE	secureBootImageType	The type of secure boot image.
SBIF_ECDSA_Header_t*	pSecureHeader	Pointer to the secure boot stream header that is loaded by the boot peripheral from the boot source during the configuration phase.

Table 36-50: ADI_ROM_BOOT_CONFIG Members (Continued)

Type	Name	Description
ADI_SBIF_ECDSA_PublicKey_t	publicKey	The public key used for secure boot image authentication.
CRYPTO_DESCRIPTOR	cryptoDescriptors	The descriptor items as required for PKTE operations.
SB_StorageArea_t*	pSB_Storage	Storage area reserved for some crypto operations.
int32_t	secureBytesRemaining	The number of bytes remaining to be processed in the secure boot stream.
uint32_t[4]	aesKey	The 128-bit AES decryption key.
uint32_t[6]	aesWrapKey	The key wrapped key from the BLw secure boot image.
uint32_t[4]	IV	The IV as read from the secure boot header, required to initialize the PKTE.
uint8_t *	pHash	Pointer to the output destination of the SHA-224 result that is required for authentication of the secure boot stream.
uint32_t	ecdsaType	Type of ECDSA algorithm to sign the image
uint32_t	errorReturn	Storage location for the address of the instruction line following a call to the error handler.
uint8_t *	pstack_start	Pointer to stack start.
uint8_t *	pstack_end	Pointer to stack end.

pSource

Source address from where to fetch the next boot data.

The source address must be maintained by the boot peripheral's load function. The kernel does not update the source pointer automatically after requesting data. This lets load routines control and change the source address. It is useful when advanced second stage loaders need to change the source address (due to a fragmented boot stream) or to reset the address (if expanding into a second SPI flash device).

During debug it is useful in identifying the block in the boot stream that is currently being processed.

pDestination

Destination address to store the fetched data.

Used by the boot kernel to indicate the destination address for the fetched data. A boot peripheral's load function must transfer the data to this location before returning back to the kernel. The boot kernel updates this field depending on whether a block header or payload is being fetched. In normal operation mode the kernel loads this field with the storage area location of the block header. After processing the block header loads

`pDestination` with the `pTargetAddress` Destination address of payload contents read from the fetched block header. When using page mode the destination points to the internal buffers and is then updated to transfer data to the final destination.

dByteCount

Number of bytes to fetch from the boot source.

The kernel sets this parameter to indicate to the load function the number of bytes requested by the kernel. The kernel is responsible for adjusting the byte count for page mode based accesses. The peripheral load function must return the required number of bytes to the destination address provided.

dFlags

Control flags related to the boot kernel block processing.

When calling a boot mode using the `adi_rom_Boot()` routine the flags supplied to that routine are used to initialize `dFlags`. These become global flags that remain set through the entire boot process. When a block header is received the lower 16 bits of the block header are OR'ed with the global flags. The boot software may clear some flags if it detects they are not compatible with other flags and then writes the resulting flags back to this member. The boot kernel then processes the block payload as instructed by the combination of global and boot block specific flags. Upon completion of the block processing original set of global flags are restored and the process repeated.

ulBlockCount

Limit of blocks processed during boot.

When calling the boot process the `adi_rom_Boot()` routine can accept a block limit for the number of blocks to process before terminating the boot process. If the block count is set to zero then the boot process will continue until a final block reached indicating end of the boot stream. This member holds the program's specified limit for the number of blocks to process and is used by the boot kernel after processing each block to compare it against the `ADI_ROM_BOOT_CONFIG::ulBlockCurrent` value. Boot process terminates when `ADI_ROM_BOOT_CONFIG::ulBlockCurrent` equals `ADI_ROM_BOOT_CONFIG::ulBlockCount`

ulBlockCurrent

The number of blocks currently processed by the boot kernel

pNextDxe

Pointer to the next application in the boot stream or the first free location after the boot stream.

This member is initialized when processing a first block in the boot stream. The `dArgument` Argument function varies depending on operation field of a first block contains the number of bytes left in the boot stream before we reach the end of that boot stream. This lets this pointer point to the next boot stream or to the first empty location after the boot stream. This allows for a feature when using the `adi_rom_Boot()` routine to find the address of an application in a linked list of boot streams or to find the first empty location after the boot stream.

uByteAddress

The destination address converted to the byte address space.

This member is used to store the byte address space equivalent of SHARC L1 memory addresses, allowing any core to load content to any SHARC cores L1 memory. The SHARC core in which the load is targeted is determined by the block header.

pControlRegister

Pointer to the boot peripherals control register.

This can be used by a boot mode peripherals driver in order to gain efficient access to a control MMR register in the boot peripheral.

NOTE: This is not used in this products boot implementation but may be leveraged by developers of second stage boot loaders if required

dControlValue

Storage for the boot peripheral main control value to enable that peripheral in a required configuration.

This can be used by a boot modes peripheral driver in order to store a control value that can be used to enable the peripheral for a required configuration.

pPeripheralBase

Pointer to the boot peripherals base MMR address

pAuxControlRegister

Pointer to any register that may be used for auxiliary operations such as a timer control register for UART autobaud detection

pAuxPeripheralBase

Pointer to the base address of any peripheral used for auxiliary operations such as the TIMER block

pSecControlRegister

Base MMR address of the SEC SSI instance associated with the boot peripheral should they be required for advanced second stage boot loader development

pDmaBaseRegister

Base MMR address of the DMA channel associated with the boot peripheral.

This is used by the boot kernel to gain access to the DMA channels status when using non blocking DMA operations when page mode or secure boot is required, so it must be set when implementing custom boot loaders in order for the kernel to get access to that peripherals DMA status.

NOTE: If a custom boot peripheral does not support the standard DMA instance then the custom driver will be required to set up a DMA instance in SRAM that this location points to and

the load function would need to update the status accordingly to indicate when the DMA was running and when the DMA completed.

loadType

Set by the kernel to specify to the boot peripherals load function if it is requesting a blocking or non-blocking DMA

MdmaCfg

An MDMA descriptor that is used by the boot kernel for internal MDMA operations.

The boot kernel may be required to perform internal MDMA operations outside the control of the boot peripheral driver. Such operations include processing of fill blocks CRC callbacks for CRC verification and MDMA operations from the internal intermediate buffers for indirect block and page mode processing.

NOTE: Users must not use this item or reconfigure this item when developing custom boot drivers, it is intended purely for the internal use by the boot kernel

uwDataWidth

The maximum data width supported by the boot peripherals DMA channel. Set to 0 for 8-bit, 1 for 16-bit and 2 for 32-bit

uwSrcModifyMult

The source modify multiplier used to set [DMA_XMOD](#) for source MDMA operations or peripheral DMA transmit operations

uwDstModifyMult

The destination modify multiplier used to set [DMA_XMOD](#) for destination MDMA operations or peripheral DMA receive operations

uwUserShort

Free to use by the user

dUserLong

Free to use by the user

pModeData

Pointer to the boot mode specific data structure.

Can be set by a boot peripheral driver to allow for a single point of access to the boot mode specific object containing control and configuration information specific to that single boot mode.

dBootCommand

The boot command value supplied during the call to the [adi_rom_Boot\(\)](#) routine

dBootCommand1

The boot command value supplied during the call to the `adi_rom_Boot()` routine

pHeader

Pointer to the boot header storage location where all boot stream block headers eventually reside for processing by the kernel

pTempBuffer

Pointer to the internal intermediate buffer. Used for processing of indirect blocks

dTempByteCount

Size of the internal intermediate buffer in bytes

pTempSource

Current source address that is being processed in the internal intermediate buffer

dPageByteCount

The page size used for page mode processing. On this product page size is fixed to 1024 bytes and this member is not used for the load requests

bootBuffers

The internal intermediate buffer descriptors required when using indirect and page mode features

bootRegistry

The registry object that is used to register a boot peripherals routines with the kernel.

When using the `adi_rom_Boot()` function the boot software calls a peripherals initialization, configuration, load and cleanup routines from the pointers stored in this object. The kernel itself only makes calls to the load function for the peripheral so when using the `adi_rom_BootKernel()` function the load function that is called by the boot kernel to fetch data from the boot source must be registered using `pLoadFunction` Pointer to the boot modes Load function .

pErrorFunction

Pointer to the error handler called in the event of an error

pCallbackFunction

Pointer to the callback function that is called when processing boot blocks with the callback flag set

pCrcFunction

Pointer to the CRC function that is used to perform CRC validation of the boot stream payload data

pForwardFunction

Feature not supported on this product

bootModes

Access to all boot mode specific resources

pLogBuffer

Pointer to the log buffer. Logging is disabled by default on this product and thus must be configured from within initcodes or hook routines

pLogCurrent

The current position within the log buffer. Logging is disabled by default on this product and thus must be configured from within initcodes or hook routines

dLogByteCount

The size of the log buffer. Logging is disabled by default on this product and thus must be configured from within initcodes or hook routines

timerBenchmarkInfo

Access to all boot benchmarking resources

xspiPhyConfig

Access to 128-bit xSPI PHY configuration

pOtpBootInfo

Pointer to the `struct ADI_ROM_OTP_BOOT_INFO` boot information block that gets read from OTP and contains boot customization options

keyType

When set to a specific value allows keys not stored in OTP to be used for secure boot evaluation on an open processor.

By default when performing boot on an open part the decryption keys and the public key are fetched from OTP. By setting this field to `enum ADI_ROM_BOOT_KEY_TYPE` users can disable the fetching of the keys from OTP and instead provision the keys directly in the `ADI_ROM_BOOT_CONFIG::publicKey` and `ADI_ROM_BOOT_CONFIG::aesKey` members via hook routines when using the `adi_rom_Boot()` function.

bootType

A key to indicate if the boot type is secure or non-secure for open parts

secureBootImageType

The type of secure boot image

pSecureHeader

Pointer to the secure boot stream header that is loaded by the boot peripheral from the boot source during the configuration phase

publicKey

The public key used for secure boot image authentication

cryptoDescriptors

The descriptor items as required for the PKTE operations

pSB_Storage

Storage area reserved for some crypto operations

secureBytesRemaining

The number of bytes remaining to be processed in the secure boot stream

aesKey

The 128-bit AES decryption key

aesWrapKey

The key wrapped key from the BLW secure boot image

IV

The IV as read from the secure boot header as required to initialize the PKTE

pHash

Pointer to the output destination of the SHA-224 result that is required for authentication of the secure boot stream

ecdsaType

This holds the ecdsa type to be used for authentication. It can be 224-bit or 256-bit.

errorReturn

Storage location for the address of the instruction line following a call to the error handler

pstack_start

Pointer to stack start address

pstack_end

Pointer to stack end address

struct ADI_ROM_BOOT_HEADER

Structure Type Declaration: ADI_ROM_BOOT_HEADER

Boot Block Header.

Boot block headers control the loading process of the boot stream, For full details on the contents of the block header and supported flags see [Boot Loader Stream](#).

Table 36-51: ADI_ROM_BOOT_HEADER Members

Type	Name	Description
int32_t	dBlockCode	Instructs the boot kernel how to process the block.
void *	pTargetAddress	Destination Address of Payload
int32_t	dByteCount	Byte Count of the Payload
int32_t	dArgument	Argument functionality varies depending on operation

dBlockCode

Instructs the boot kernel how to process the block.

Contains a number of fields for verification of the block header and flags to indicate the type of block. This allows the kernel to process the block correctly.

pTargetAddress

Destination address of payload

dByteCount

Byte count of the payload

dArgument

Argument function varies depending on operation

struct ADI_ROM_BOOT_INTER_BUFFER

Structure Type Declaration: ADI_ROM_BOOT_INTER_BUFFER

The buffer object for the internal intermediate buffers used for indirect and page mode operations.

Table 36-52: ADI_ROM_BOOT_INTER_BUFFER Members

Type	Name	Description
uint8_t *	pBuffer	Pointer to the buffer
uint32_t	size	Size of the buffer
uint32_t	pageSize	Page size for block based devices.

pBuffer

Pointer to the buffer

size

Size of the buffer

pageSize

Page size for block based devices.

NOTE: This field is not used in this product. A fixed page size of 1024 bytes is used.

struct ADI_ROM_BOOT_INTER_BUFFERS

Structure Type Declaration: ADI_ROM_BOOT_INTER_BUFFERS

The boot kernels internal buffer object used to access the intermediate buffers and obtain buffer status.

Table 36-53: ADI_ROM_BOOT_INTER_BUFFERS Members

Type	Name	Description
ADI_ROM_BOOT_INTER_BUFFER[2]	buffer	The two buffer descriptors
ADI_ROM_BOOT_BUFFER_STATE	state	Buffer Status Information
void *	pSource	Original source address pointer of data loaded to active buffer. Not used on this product
ADI_DMA_TypeDef *	pDma	Pointer to the DMA channel responsible for loading the buffer

buffer

The two buffer descriptors

state

Buffer Status Information

pSource

Original source address pointer of data loaded to active buffer. Not used on this product

pDma

Pointer to the DMA channel responsible for loading the buffer

struct ADI_ROM_BOOT_MODES

Structure Type Declaration: ADI_ROM_BOOT_MODES

Holds all boot mode specific configuration items.

A boot mode may have requirements for some dedicated storage. This object is used to collect all storage items for all the boot modes supported by the boot ROM.

Table 36-54: ADI_ROM_BOOT_MODES Members

Type	Name	Description
ADI_ROM_BOOT_SPI	spi	Access to all SPI boot mode resources
ADI_ROM_BOOT_UART	uart	Access to all UART boot mode resources
ADI_ROM_BOOT_XSPI	xspi	Access to all xSPI boot mode resources
ADI_ROM_BOOT_CUSTOM	custom	Access to all custom boot mode resources

spi

Access to all SPI boot mode resources

uart

Access to all UART boot mode resources

xspi

Access to all xSPI boot mode resources

custom

Access to all custom boot mode resources

struct ADI_ROM_BOOT_REGISTRY

Structure Type Declaration: ADI_ROM_BOOT_REGISTRY

Boot Mode Registration.

Used to hold pointers for the boot mode initialization, configuration, load and cleanup functions. Can customize the registered content via hook routines or install load functions or cleanup functions from within init codes.

When using `adi_rom_Boot()` the boot process makes a call to the initialization function and the configuration function before calling the kernel. The kernel then runs and makes calls to the load function. The cleanup function is called when the kernel reaches the end of the boot stream.

When using the `adi_rom_BootKernel()` function only the load function is called during execution of the software in the boot ROM. All the functions here must return a `enum ROM_BOOT_RESULT` result in order for the boot process to continue. All functions expect a single argument that is the pointer to the boot structure object `struct ADI_ROM_BOOT_CONFIG`.

Table 36-55: ADI_ROM_BOOT_REGISTRY Members

Type	Name	Description
ROM_BOOT_MODE_INIT_FUNC *	pInitFunction	Pointer to the boot modes Initialization function

Table 36-55: ADI_ROM_BOOT_REGISTRY Members (Continued)

Type	Name	Description
ROM_BOOT_MODE_CONFIG_FUNC *	pConfigFunction	Pointer to the boot modes Configuration function
ROM_BOOT_MODE_LOAD_FUNC *	pLoadFunction	Pointer to the boot modes Load function
ROM_BOOT_MODE_CLEANUP_FUNC *	pCleanUpFunction	Pointer to the boot modes Cleanup function
void *	pReserved	Reserved for future use
int32_t	dReserved	Reserved for future use

pInitFunction

Pointer to the boot modes Initialization function

pConfigFunction

Pointer to the boot modes Configuration function

pLoadFunction

Pointer to the boot modes Load function

pCleanUpFunction

Pointer to the boot modes Cleanup function

struct ADI_ROM_BOOT_SPI

Structure Type Declaration: ADI_ROM_BOOT_SPI

The SPI Controller Boot Mode Specific Structure.

This structure contains all the boot context information that is specific for SPI controller boot mode. During auto-detection information is copied from the required ::ROM_SPI_LUTENTRY item into this structure and used to configure the SPI peripheral for the mode of operation.

Table 36-56: ADI_ROM_BOOT_SPI Members

Type	Name	Description
uint8_t	ubReadCommand	Read command to read data from the SPI device
uint8_t	ubDummyBytes	Number of dummy bytes to issue after the read command
uint8_t	ubAddressBytes	Number of address bytes required to access the device
uint8_t	ubDataBits	Bus width when reading the data. 0 = single bit, 1 = dual bit, 2 = quad bit

Table 36-56: ADI_ROM_BOOT_SPI Members (Continued)

Type	Name	Description
uint16_t	uwClkLower	SPI clock divider value
uint16_t	uReserved0	Reserved
ROM_BOOT_SPIM_IO_ENABLE_FUNC *	pMIOEnFunction	Pointer to the function used to enable quad mode on the SPI flash
uint8_t	nDummy	The Dummy Byte value if dummy byte transfers are required and the bus is not three-stated
uint8_t	nFlags	Flags used for additional SPI configuration processing.
uint16_t	uReserved2	Reserved
void *	pXIPAddress	The memory-mapped SPI address to boot from
ADI_SPI_TypeDef *	pRegisters	Pointer to the SPI peripherals base MMR address, not used on this product
ADI_DMA_TypeDef *	pRxDmaRegisters	Pointer to the DMA peripherals base MMR address, for receive operations, not used on this product
ADI_DMA_TypeDef *	pTxDmaRegisters	Pointer to the DMA peripherals base MMR address,for transmit operations, not used on this product

ubReadCommand

Read command to use to read data from the SPI device

ubDummyBytes

Number of dummy bytes to issue after the read command

ubAddressBytes

Number of address bytes required to access the device

ubDataBits

The bus width used when reading the data. 0 for single bit, 1 for dual, 2 for quad

uwClkLower

The SPI clock divider value

pMIOEnFunction

Pointer to the function used to enable quad mode on the SPI flash

nDummy

The Dummy Byte value if dummy byte transfers are required and the bus is not tri-stated

nFlags

Flags used for some additional SPI configuration processing. The flags supported are defined as follows:

Bit Position	Name	Description
0	ROM_SPI_FLAGS_CMDSKIP_EN	When set the configuration routine enables command skip mode where the SPI does not issue a read command for read operations
1	ROM_SPI_FLAGS_MULTICMD_EN	Configuration routine enables sending command cycles over dual or quad bit bus

pXIPAddress

The memory-mapped SPI address to boot from

pRegisters

Pointer to the SPI peripherals base MMR address, not used on this product

pRxDmaRegisters

Pointer to the DMA peripherals base MMR address, for receive operations, not used on this product

pTxDmaRegisters

Pointer to the DMA peripherals base MMR address, for transmit operations, not used on this product

struct ADI_ROM_BOOT_XSPI

Structure Type Declaration: ADI_ROM_BOOT_XSPI

The xSPI controller boot mode specific structure.

This structure contains all the boot context information that is specific for the xSPI controller boot mode.

Table 36-57: ADI_ROM_BOOT_XSPI Members

Type	Name	Description
uint8_t	ubConfigChange	Defines whether any of the xSPI interface configuration must be changed
uint8_t	ubClockDivider	Clock divider value. This value is updated irrespective of the ubConfigChange value
uint8_t	ubDummyCycles	Number of dummy cycles to issue after the read command. This value is updated only when ubConfigChange = 1.

Table 36-57: ADI_ROM_BOOT_XSPI Members (Continued)

Type	Name	Description
uint8_t	ubNumlinesCommand	Number of lines required for the command. This value is updated only when ubConfigChange = 1.
uint8_t	ubNumlinesAddress	Number of lines required for the address. This value is updated only when ubConfigChange = 1.
uint8_t	ubNumlinesData	Number of lines required for data. This value is updated only if ubConfigChange = 1.
uint8_t	ubEdgemodeCommand	Edge mode SDR or DDR for command. This value is updated only when ubConfigChange = 1.
uint8_t	ubEdgemodeAddress	Edge mode SDR or DDR for address. This value is updated only when ubConfigChange = 1.
uint8_t	ubEdgemodeData	Edge mode SDR or DDR for data. This value is updated only when ubConfigChange = 1.
uint8_t	ubNumAddressBytes	Number of address bytes used of read access to flash. This value is updated only when ubConfigChange = 1.
uint8_t	ubOpcode0	Opcode used for flash read access. This value is updated only when ubConfigChange = 1.
uint8_t	ubOpcode1	Supplemental opcode used for flash read access. This value is updated only when ubConfigChange = 1.
uint8_t	ubClockChange	Defines whether xSPI interface clock is changed from default setting
uint32_t	nXspiPhyConfig0	xSPI PHY configuration word 0
uint32_t	nXspiPhyConfig1	xSPI PHY configuration word 1
uint32_t	nXspiPhyConfig2	xSPI PHY configuration word 2
uint32_t	nXspiPhyConfig3	xSPI PHY configuration word 3
void *	pXIPAddress	Pointer to the XIP address
ADI_XSPI_TypeDef *	pRegisters	Pointer to the xSPI register

struct ADI_ROM_BOOT_UART

Structure Type Declaration: ADI_ROM_BOOT_UART

The UART target boot mode specific structure.

This structure contains all the boot context information that is specific for the UART target boot mode.

Table 36-58: ADI_ROM_BOOT_UART Members

Type	Name	Description
uint32_t	nFlags	Flags related to UART Boot mode

Table 36-58: ADI_ROM_BOOT_UART Members (Continued)

Type	Name	Description
ADI_UART_TypeDef*	pRegisters	Pointer to the UART peripherals base MMR address, not used on this product
ADI_DMA_TypeDef*	pRxDmaRegisters	Pointer to the DMA peripherals base MMR address for receive operations, not used on this product
ADI_DMA_TypeDef*	pTxDmaRegisters	Pointer to the DMA peripherals base MMR address for transmit operations, not used on this product

nFlags

Flags related to UART Boot mode

pRegisters

Pointer to the UART peripherals base MMR address, not used on this product

pRxDmaRegisters

Pointer to the DMA peripherals base MMR address for receive operations, not used on this product

pTxDmaRegisters

Pointer to the DMA peripherals base MMR address for transmit operations, not used on this product

struct ADI_ROM_OTP_BOOT_CFG

Structure Type Declaration: ADI_ROM_OTP_BOOT_CFG

The boot configuration object for storing further boot customization objects.

This structure is allocated to one contiguous region in the OTP memory array. The functionality allows for individual flags to enable or disable specific features of the boot process. Each flag is allocated in a separate 16-bit word so that each flag can be set at different times and the ECC information will not impact the setting of another flag.

Table 36-59: ADI_ROM_OTP_BOOT_CFG Members

Type	Name	Description
uint32_t	pubkey0Inv:1 (bitfield)	Invalidate Public Key 0; use next public key for secure boot
uint32_t	reserved0:15 (bitfield)	Reserved
uint32_t	pubkey1Inv:1 (bitfield)	Invalidate Public Key 1; secure boot will no longer be operational as no further public keys
uint32_t	reserved1:15 (bitfield)	Reserved
uint32_t	privkey0Inv:1 (bitfield)	Invalidate Decryption Key 0; use next decryption key for secure boot

Table 36-59: ADI_ROM_OTP_BOOT_CFG Members (Continued)

Type	Name	Description
uint32_t	reserved2:15 (bitfield)	Reserved
uint32_t	privkey1Inv:1 (bitfield)	Invalidate Decryption Key 1; use next decryption key for secure boot
uint32_t	reserved3:15 (bitfield)	Reserved
uint32_t	privkey2Inv:1 (bitfield)	Invalidate Decryption Key 2; use next decryption key for secure boot
uint32_t	reserved4:15 (bitfield)	Reserved
uint32_t	privkey3Inv:1 (bitfield)	Invalidate Decryption Key 3; once invalidated part is no longer bootable
uint32_t	reserved5:15 (bitfield)	Reserved
uint32_t	xspisramEn:1 (bitfield)	Enables the configuration of the XSPI SRAM from the OTP values
uint32_t	reserved6:15 (bitfield)	Reserved
uint32_t	xspisramInv:1 (bitfield)	Invalidates the xSPI SRAM values in the OTP
uint32_t	reserved7:15 (bitfield)	Reserved
uint32_t	dmcDecDis:1 (bitfield)	Disabled decryption from off chip memory
uint32_t	reserved8:15 (bitfield)	Reserved
uint32_t	altMstrBootEn:1 (bitfield)	Alternate controller boot enable for primary controller boot failure
uint32_t	reserved9:15 (bitfield)	Reserved
uint32_t	altMstrBootDis:1 (bitfield)	Alternate controller boot disable for primary controller boot failure
uint32_t	reserved10:15 (bitfield)	Reserved
uint32_t	altSlaveBootEn:1 (bitfield)	Alternate target boot enable for primary/secondary controller boot failure
uint32_t	reserved11:15 (bitfield)	Reserved
uint32_t	altSlaveBootDis:1 (bitfield)	Alternate target boot disable for primary/secondary controller boot failure
uint32_t	reserved12:15 (bitfield)	Reserved
uint32_t	benchmark_En:1 (bitfield)	Boot benchmark enable bit
uint32_t	reserved13:15 (bitfield)	Reserved
uint32_t	wdtDis:1 (bitfield)	Disable watchdog support
uint32_t	reserved14:15 (bitfield)	Reserved
uint32_t	fastcguDis:1 (bitfield)	Disable fast CGU support

Table 36-59: ADI_ROM_OTP_BOOT_CFG Members (Continued)

Type	Name	Description
uint32_t	reserved15:15 (bitfield)	Reserved

pubkey0Inv

Invalidate Public Key 0; use next public key for secure boot

pubkey1Inv

Invalidate Public Key 1; secure boot will no longer be operational as no further public keys

privkey0Inv

Invalidate Decryption Key; use next decryption key for secure boot

privkey1Inv

Invalidate Decryption Key 1; use next decryption key for secure boot

privkey2Inv

Invalidate Decryption Key 2; use next decryption key for secure boot

privkey3Inv

Invalidate Decryption Key 3; once invalidated part will no longer be bootable

xspisramEn

Enables configuration of the xSPI SRAM from the OTP values

xspisramInv

Invalidates the xSPI SRAM values in the OTP

dmcDecDis

Disabled decryption from off-chip memory

altMstrBootEn

Alternate controller boot enable for primary controller boot failure

altMstrBootDis

Alternate controller boot disable for primary/secondary controller boot failure

altSlaveBootEn

Alternate target boot enable for primary/secondary controller boot failure

altSlaveBootDis

Alternate target boot disable for primary/secondary controller boot failure

benchmark_En

Boot benchmark enable bit

wdtDis

Disabled watchdog support

wdtDis

Disabled fast CGU support

struct ADI_ROM_OTP_BOOT_CGU_INFO

Structure Type Declaration: ADI_ROM_OTP_BOOT_CGU_INFO

The CGU configuration object located in OTP for configuration of the CGU by the boot software.

This structure is allocated to one contiguous region in the OTP memory array. Allows the boot software to configure the CGU for a more efficient boot process.

Table 36-60: ADI_ROM_OTP_BOOT_CGU_INFO Members

Type	Name	Description
uint32_t	ctl_WEN:1 (bitfield)	Enable write to the CGU_CTL register
uint32_t	div_WEN:1 (bitfield)	Enable write to the CGU_DIV register
uint32_t	reserved0:1 (bitfield)	Reserved
uint32_t	div_DSEL:5 (bitfield)	CGU_DIV.DSEL value
uint32_t	div_CSEL:5 (bitfield)	CGU_DIV.CSEL value
uint32_t	div_S0SEL:3 (bitfield)	CGU_DIV.S0SEL value
uint32_t	div_SYSSEL:5 (bitfield)	CGU_DIV.SYSSEL value
uint32_t	div_S1SEL:3 (bitfield)	CGU_DIV.S1SEL value
uint32_t	div_OSEL:7 (bitfield)	CGU_DIV.OSEL value
uint32_t	ctl_DF:1 (bitfield)	value
uint32_t	ctl_MSEL:7 (bitfield)	CGU_CTL.MSEL value
uint32_t	reserved1:1 (bitfield)	Reserved
uint32_t	ctl_S1SELEXEN:1 (bitfield)	CGU_CTL.S1SELEXEN value
uint32_t	auto_disable:1 (bitfield)	disable polling on auto-alignment of clocks (not recommended)
uint32_t	Reserved1:1 (bitfield)	Reserved
uint32_t	clkoutsel_CLKOUTSEL:5 (bitfield)	CGU_CLKOUTSEL.CLKOUTSEL value
uint32_t	clkoutsel_WEN:1 (bitfield)	Enable write to the CGU_CLKOUTSEL register

Table 36-60: ADI_ROM_OTP_BOOT_CGU_INFO Members (Continued)

Type	Name	Description
uint32_t	reserved2:8 (bitfield)	Reserved
uint32_t	divex_S1SELEX:8 (bitfield)	CGU_DIVEX.S1SELEX value

ctl_WEN

Enable write to the [CGU_CTL](#) register

div_WEN

Enable write to the [CGU_DIV](#) register

div_DSEL

[CGU_DIV.DSEL](#) value

div_CSEL

[CGU_DIV.CSEL](#) value

div_S0SEL

[CGU_DIV.S0SEL](#) value

div_SYSSSEL

[CGU_DIV.SYSSSEL](#) value

div_S1SEL

[CGU_DIV.S1SEL](#) value

div_OSEL

[CGU_DIV.OSEL](#) value

ctl_DF

value

ctl_MSEL

[CGU_CTL.MSEL](#) value

ctl_S1SELEXEN

Enable write to the [CGU_CTL.S1SELEXEN](#) value

divex_S1SELEX

Enable write to the [CGU_DIVEX.S1SELEX](#) value

auto_disable

disable polling on auto-alignment of clocks, NOT RECOMMENDED!

clkoutsel_CLKOUTSEL

CGU_CLKOUTSEL.CLKOUTSEL value

clkoutsel_WEN

Enable write to the [CGU_CLKOUTSEL](#) register

struct ADI_ROM_OTP_BOOT_CMD_INFO

Structure Type Declaration: ADI_ROM_OTP_BOOT_CMD_INFO

The boot command object for storing a customized boot command for each boot mode within the OTP memory array.

This is a 224-bit structure that is allocated to one contiguous region in the OTP memory array. Allows the boot ROM software to pass a custom boot command to a specific boot mode which changes the default boot behavior on startup. Can be used to change the default UART instance used for a UART boot operation (for example).

Table 36-61: ADI_ROM_OTP_BOOT_CMD_INFO Members

Type	Name	Description
uint32_t	spiMasterBootCmd	SPI Controller Boot Mode (BMODE1)
uint32_t	spiSlaveBootCmd	SPI Target Boot Mode
uint32_t	uartBootCmd	UART Target Boot Mode
uint32_t	xspiMasterBootCmd	xSPI Controller Boot Mode
uint32_t	altSlaveBootCmd	xSPI Target Boot Mode
uint32_t	xspiMasterBootCmd1	xSPI Controller Boot Mode
uint32_t	spi1MasterBootCmd	SPI Controller Boot Mode (BMODE6)

spiMasterBootCmd/spi1MasterBootCmd

[SPI Controller Boot Mode](#)

spiSlaveBootCmd

[SPI Target Boot Mode](#)

altSlaveBootCmd

Alternate Target Boot Command

uartBootCmd

[UART Target Boot Mode](#)

xspiMasterBootCmd/xspiMasterBootCmd1

xSPI Master Boot Mode

struct ADI_ROM_OTP_BOOT_INFO

Structure Type Declaration: ADI_ROM_OTP_BOOT_INFO

This structure is allocated to one contiguous region in the OTP memory array. The content in OTP is stored in the format of this structure so boot can read the contents directly into this object.

The boot info object located in OTP for boot customization.

Table 36-62: ADI_ROM_OTP_BOOT_INFO Members

Type	Name	Description
ADI_ROM_OTP_BOOT_CGU_INFO	cgu	CGU Configuration information
uint32_t	flashStartAddress	Flash start address for the SPI/xSPI controller boot mode
uint32_t	altflashStartAddress	Flash start address for the SPI/xSPI alternate controller boot mode
ADI_ROM_OTP_BOOT_CMD_INFO	bcmd	Boot command customization for each boot mode to change default boot peripheral instance and configuration
ADI_ROM_OTP_BOOT_CFG	bcfg	Additional boot configuration flags for key invalidation
uint32_t	reserved0	Reserved
uint16_t	otpReadTiming	Reserved, Must always be zero
uint16_t	reserved1	Reserved

cgu

CGU Configuration information

flashStartAddress

Flash start address for the SPI/xSPI controller boot mode

altflashStartAddress

Flash start address for the SPI/xSPI alternate controller boot mode

bcmd

Boot Command customization for each boot mode to change the default boot peripheral instance and configuration

bcfg

Additional boot configuration flags for key invalidation and for enabling configuration

otpReadTiming

Reserved, Must always be zero

struct ROM_BOOT_DMA_INSTANCE

Structure Type Declaration: ROM_BOOT_DMA_INSTANCE

DMA Channel Instance.

Specifies the base MMR address of the DMA channel as well as trigger and interrupt IDs

Table 36-63: ROM_BOOT_DMA_INSTANCE Members

Type	Name	Description
ADI_DMA_TypeDef *	pReg	Pointer to the base address of the DMA channel MMR registers
DMA_CHANn_TypeDef	eDmaChannelId	The actual DMA channel ID in the system
uint8_t	TriggerId	The trigger ID associated with the DMA channel
uint8_t	InterruptId	The interrupt ID associated with the DMA channel

pReg

Pointer to the base address of the DMA channel MMR registers

eDmaChannelId

The actual DMA channel ID in the system

TriggerId

The trigger ID associated with the DMA channel

InterruptId

The interrupt ID associated with the DMA channel

struct ROM_BOOT_MDMA

Structure Type Declaration: ROM_BOOT_MDMA

MDMA Channels.

Provides access to all the MDMA channels and CRC peripherals supported by the processor

Table 36-64: ROM_BOOT_MDMA Members

Type	Name	Description
ROM_BOOT_MDMA_REGS[PARAM_SYS0_NUM_MDMA_STREAMS]	Stream	Array of MDMA channel configurations supported by the processor

Stream

Array of MDMA channel configurations supported by the processor

struct ROM_BOOT_MDMA_REGS

Structure Type Declaration: ROM_BOOT_MDMA_REGS

MDMA Channel Registers.

Contains the Source and Destination MDMA channel instances for access to the MMRs and interrupt and trigger information. Information is also provided on the CRC support of the MDMA channel and access is provided to the corresponding CRC peripheral.

Table 36-65: ROM_BOOT_MDMA_REGS Members

Type	Name	Description
<code>struct ROM_BOOT_DMA_INSTANCE</code>	Src	The source DMA Channel in the MDMA pair
ROM_BOOT_DMA_INSTANCE	Dst	The destination DMA Channel in the MDMA pair
ADI_CRC_TypeDef *	pCrc	The base MMR address of the associated CRC peripheral if one exists
<code>enum ROM_BOOT_MDMA_CRC_SUPPORT</code>	eCrcSupport	Indicates if the MDMA channel supports CRC or not

Src

The source DMA Channel in the MDMA pair

Dst

The destination DMA Channel in the MDMA pair

pCrc

The base MMR address of the associated CRC peripheral if one exists

eCrcSupport

Indicates if the MDMA channel supports CRC or not

struct ROM_DMA_MDMA_CONFIG

Structure Type Declaration: ROM_DMA_MDMA_CONFIG

MDMA Configuration Object.

The configurable structure for controlling the MDMA operation supplied to the `adi_rom_MemDma()` routine.

Table 36-66: ROM_DMA_MDMA_CONFIG Members

Type	Name	Description
enum ROM_DMA_MDMA_OPERATION	eOperation	Type of operation to perform
enum ROM_DMA_MDMA_ID	eId	MDMA Channel ID
void *	pSource	Source Pointer
void *	pDestination	Destination Pointer
uint32_t	ByteCount	Byte Count
enum ROM_DMA_DONE_DETECT_METHOD	eDoneDetect	DMA Done Detection Method
uint32_t	CrcCtl	CRC_CTL value when CRC operations are required
uint32_t	FillVal	Fill value for memory fill operations
uint32_t	CrcPoly	CRC Polynomial for CRC operations
uint32_t	CrcCompare	Value used for CRC compare operations or for a CRC32 result compare

eOperation

Type of operation to perform

eId

MDMA Channel ID

pSource

Source Pointer

pDestination

Destination Pointer

ByteCount

Byte Count

eDoneDetect

DMA Done Detection Method

CrcCtl[CRC_CTL](#) value when CRC operations are required**FillVal**

Fill value for memory fill operations

CrcPoly

CRC Polynomial for CRC operations

CrcCompare

Value used for CRC compare operations or for a CRC32 result compare

struct ROM_DMA_PDMA_CONFIG

Structure Type Declaration: ROM_DMA_PDMA_CONFIG

PDMA Configuration Object.

The user configurable structure for controlling the PDMA operation via the [adi_rom_PeriphDma\(\)](#) function.

Table 36-67: ROM_DMA_PDMA_CONFIG Members

Type	Name	Description
enum ROM_DMA_PDMA_OPERATION	eOperation	Type of operation to perform
ADI_DMA_TypeDef volatile *	pRegs	Pointer to the base address of the DMA channel MMR registers
uint16_t	dataWidth	The maximum supported data width of the DMA channel. Used to configure the DMA_CFG.PSIZE PSIZE field in DMA_CFG
uint16_t	dstModifyMult	The modify multiplier, usually set to 1
void *	pSource	Source Pointer used for transmit operations
void *	pDestination	Destination Pointer used for receive operations
uint32_t	byteCount	Number of bytes to transfer
enum ROM_DMA_DONE_DETECT_METHOD	eDoneDetect	DMA Done Detection method used for the transfer
enum ROM_DMA_DONE_DETECT_METHOD	loadType	Defines whether the load function DMA should wait for the load to complete or return to the kernel

eOperation

Type of operation to perform

pRegs

Pointer to the base address of the DMA channel MMR registers

dataWidth

The maximum supported data width of the DMA channel. Used to configure the `DMA_CFG.PSIZE` field in `DMA_CFG`

dstModifyMult

The modify multiplier, usually set to 1

pSource

Source Pointer used for transmit operations

pDestination

Destination Pointer used for receive operations

byteCount

Number of bytes to transfer

eDoneDetect

DMA Done Detection method used for the transfer

loadType

Defines whether the load function DMA should wait for the load to complete or return to the kernel

struct otp_data

Structure Type Declaration: `otp_data`

Table 36-68: `otp_data` Members

Type	Name	Description
<code>uint32_t</code>	Reserved	
<code>uint32_t (*) [ROM_OTP_SZ_huk]</code>	<code>huk</code>	Pointer to 256-bit hardware unique key
<code>uint32_t (*) [ROM_OTP_SZ_rkek]</code>	<code>rkek</code>	Pointer to 128-bit root key encryption key
<code>uint32_t (*) [ROM_OTP_SZ_dek]</code>	<code>dek</code>	Pointer to 128-bit local encryption key
<code>uint32_t (*) [ROM_OTP_SZ_oem_public_key]</code>	<code>oem_public_key</code>	Pointer to 512-bit OEM public key
<code>uint32_t (*) [ROM_OTP_SZ_pvt_128key0]</code>	<code>pvt_128key0</code>	Pointer to 128-bit AES key
<code>uint32_t (*) [ROM_OTP_SZ_pvt_128key1]</code>	<code>pvt_128key1</code>	Pointer to 128-bit AES key
<code>uint32_t (*) [ROM_OTP_SZ_pvt_128key2]</code>	<code>pvt_128key2</code>	Pointer to 128-bit AES key
<code>uint32_t (*) [ROM_OTP_SZ_pvt_128key3]</code>	<code>pvt_128key3</code>	Pointer to 128-bit AES key

Table 36-68: otp_data Members (Continued)

Type	Name	Description
uint32_t(*) [ROM_OTP_SZ_ek]	ek	Pointer to 256-bit endorsement key
uint32_t(*) [ROM_OTP_SZ_secure_emu_key0]	secure_emu_key0	Pointer to 128-bit secure debug key
uint32_t(*) [ROM_OTP_SZ_secure_emu_key1]	secure_emu_key1	Pointer to 128-bit secure debug key
uint32_t	emu_key0_disable: 16 (bitfield)	Secure debug key disable
uint32_t	emu_key1_disable: 16 (bitfield)	Secure debug key disable
uint32_t(*) [ROM_OTP_SZ_public_key0]	public_key0	Pointer to 512-bit public key used for boot stream authentication
uint32_t(*) [ROM_OTP_SZ_public_key1]	public_key1	Pointer to 512-bit public key used for boot stream authentication
uint32_t	watchdog_count	Watchdog timeout count
uint32_t(*) [ROM_OTP_SZ_boot_info]	boot_info	Pointer to 608-bit boot customization structure, see also struct ADI_ROM_OTP_BOOT_INFO
uint32_t(*) [ROM_OTP_SZ_xspi_phy_cfg]	xspi_phy_cfg	Pointer to the user xSPI PHY configuration
uint8_t	antiroll_nv_cntr	Anti-rollback counter to prevent loads of older firmware during secure boot.
uint32_t(*) [ROM_OTP_SZ_gpl]	gpl	Pointer to 512-bit general purpose user space
uint32_t	bootModeDisable:8 (bitfield)	Boot mode disable for permanently disabling specific boot modes
uint32_t(*) [ROM_OTP_SZ_xspi_sram_config]	xspi_sram_config	Pointer to the user xSPI SRAM configuration
uint32_t(*) [ROM_OTP_SZ_stageID]	stageID	Pointer to 64-bit staging ID

huk

Pointer to 256-bit hardware unique key

rkek

Pointer to 128-bit root key encryption key

dek

Pointer to 128-bit local encryption key

oem_public_key

Pointer to 512-bit OEM public key

pvt_128key0

Pointer to 128-bit AES key

pvt_128key1

Pointer to 128-bit AES key

pvt_128key2

Pointer to 128-bit AES key

pvt_128key3

Pointer to 128-bit AES key

ek

Pointer to 256-bit endorsement key

secure_emu_key

Pointer to 128-bit secure debug key

secure_emu_key

Pointer to 128-bit secure debug key

emu_key0_disable

Secure debug key disable

emu_key1_disable

Secure debug key disable

public_key0

Pointer to 512-bit public key used for boot stream authentication

public_key1

Pointer to 512-bit public key used for boot stream authentication

watchdog_count

Watchdog timeout count

boot_info

Pointer to boot customization structure, see also struct [struct ADI_ROM_OTP_BOOT_INFO](#)

xspi_phy_cfg

Pointer to the user xSPI PHY configuration.

antiroll_nv_cntr

Anti-rollback counter to prevent loading of older firmware during secure boot.

The counter supports values of 0 through 31. The counter feature is disabled as long as the counter is set initially to 0.

gp1

Pointer to 512-bit general purpose user space

bootModeDisable

Boot mode disable for permanently disabling specific boot modes

xspi_sram_config

Pointer to the user xSPI SRAM configuration.

stageID

Pointer to staging ID

Enumerations

The programming model for booting the processor uses the enumerations defined in this section.

enum ADI_ROM_BOOT_KEY_TYPE

Enumeration Type Declaration: ADI_ROM_BOOT_KEY_TYPE

Indicates if custom security keys are used for evaluation of secure boot.

By default the boot process fetches all security keys from OTP for use during secure boot. Custom security lets the program set their custom keys in the `struct ADI_ROM_BOOT_CONFIG` item (not from OTP). Secure boot is evaluated using the `adi_rom_Boot()` function without provisioning keys in OTP.

Table 36-69: ADI_ROM_BOOT_KEY_TYPE Members

Enumerator	Description
ADI_ROM_CUSTOM_SECURITY	Enable use of custom security keys for authentication and decryption

ADI_ROM_CUSTOM_SECURITY

Enable use of custom security keys for authentication and decryption

enum ADI_ROM_BOOT_TYPE

Enumeration Type Declaration: ADI_ROM_BOOT_TYPE

Indicate to the boot kernel (in an open processor) if secure or non secure boot is required.

The boot kernel defaults to a secure boot unless the boot structure has been configured to indicate Non-Secure Boot

Table 36-70: ADI_ROM_BOOT_TYPE Members

Enumerator	Description
ADI_ROM_SECURE_BOOT_DIS	Non-Secure Boot
ADI_ROM_SECURE_BOOT	Secure Boot
ADI_ROM_SECURE_BOOT_HASH_ONLY	Boot kernel support for hash compare only when this boot type is selected using the ROM API. ECDSA verification is skipped.

ADI_ROM_SECURE_BOOT_DIS

Non-Secure Boot

ADI_ROM_SECURE_BOOT

Secure Boot

ADI_ROM_SECURE_BOOT_HASH_ONLY

Boot kernel support for hash compare only when this boot type is selected using the ROM API. ECDSA verification is skipped.

enum ROM_BOOT_RESULT

Enumeration Type Declaration: ROM_BOOT_RESULT

Used throughout the boot software to indicate various success and failure events that can occur during boot.

Table 36-71: ROM_BOOT_RESULT Members

Enumerator	Description
ROM_BOOT_FAILURE	Failure
ROM_BOOT_SUCCESS	Success
ROM_BOOT_HDR_CHKSUM_ERR	Boot stream block header checksum error
ROM_BOOT_HDR_SIGN_ERR	Boot stream block header sign failure
ROM_BOOT_HDR_DEST_ERR	Boot stream block payload destination error
RESERVED0	Reserved
RESERVED1	Reserved
RESERVED2	Reserved
RESERVED3	Reserved
ROM_BOOT_CGU_WRITE_ERR	CGU write error
ROM_BOOT_DMA_SUCCESS	DMA operation is successful
ROM_BOOT_DMA_FAILURE	DMA failure

Table 36-71: ROM_BOOT_RESULT Members (Continued)

Enumerator	Description
ROM_BOOT_DMA_ACTIVE	DMA channel is active
ROM_BOOT_DMA_CONFIG_ERR	DMA configuration error
ROM_BOOT_MDMA_ID_ERR	Illegal MDMA channel ID
ROM_BOOT_MDMA_OPERATION_ERR	Illegal MDMA operation specified
ROM_BOOT_MDMA_CONFIG_ERR	MDMA configuration error
ROM_BOOT_MDMA_SRC_ERR	MDMA source channel configuration error
ROM_BOOT_MDMA_DST_ERR	MDMA destination channel configuration error
ROM_BOOT_MDMA_DONE_DETECT_ERR	Memory DMA completed with errors
ROM_BOOT_MDMA_SUCCESS	Memory DMA completed successfully
ROM_BOOT_PDMA_CONFIG_ERR	Peripheral DMA configuration invalid
ROM_BOOT_PDMA_SUCCESS	Peripheral DMA completed successfully
ROM_BOOT_CRC_FAILURE	MDMA CRC32 failure
ROM_BOOT_CRC_CONFIG_ERR	MDMA CRC32 configuration error
ROM_BOOT_CRC_COUNT_ERR	CRC byte count was not a multiple of 4
ROM_BOOT_CRC_SUPPORTED_ERR	CRC not supported error
ROM_BOOT_CRC_INITCODE_ERR	CRC32 enable failure during boot
ROM_BOOT_CRC_CALLBACK_ERR	Error in the execution of the CRC callback
ROM_BOOT_CRC_SUCCESS	MDMA CRC success

ROM_BOOT_FAILURE

Failure.

General failure can be used to indicate any general failure throughout the boot process.

ROM_BOOT_SUCCESS

Success.

General success can be used to indicate any general functional success for an operation during the boot process.

NOTE: This must be the return result for a boot mode drivers initialization, configuration, load and cleanup routines when overriding their functionality in second stage boot loaders to use custom functions.

ROM_BOOT_HDR_CHKSUM_ERR

Boot stream block header checksum error.

Indicates that the 8-bit XOR checksum of the 16-byte block header failed to generated the expected result.

ROM_BOOT_HDR_SIGN_ERR

Boot stream block header sign failure.

The 0xAD block required as byte 4 of the boot block header was not found.

ROM_BOOT_HDR_DEST_ERR

Boot stream block payload destination error.

The target address field of the block header indicates that the payload for the block is destined towards an address that is not supported. Indicates an attempt to load data to the reserved 8 KB non-bootable region of memory reserved by the boot process.

ROM_BOOT_CGU_WRITE_ERR

CGU write error.

Returned by the CGU configuration routine if a CGU error is set during its initialization from settings provisioned in the OTP.

ROM_BOOT_DMA_SUCCESS DMA

Operation was successful.

ROM_BOOT_DMA_FAILURE

DMA failure.

Returned by the DMA routines if an error was detected in the `DMA_STAT.IRQERR` field prior to setting up a new DMA operation with the newly supplied configuration.

ROM_BOOT_DMA_ACTIVE

DMA channel is active.

Returned only by the peripheral DMA routine when an attempt to run another peripheral DMA operation is attempted and the DMA channel is already running.

NOTE: This is not currently implemented for MDMA operations.

ROM_BOOT_DMA_CONFIG_ERR

DMA configuration error.

ROM_BOOT_MDMA_ID_ERR

Illegal MDMA Channel ID.

Returned by `adi_rom_MemDma()` if the MDMA channel ID is not supported. For supported channel IDs, see `enum ROM_DMA_MDMA_ID`.

ROM_BOOT_MDMA_OPERATION_ERR

Illegal MDMA operation specified.

Returned by `adi_rom_MemDma()` if the MDMA operation is not supported. For supported operations, see `enum ROM_DMA_MDMA_OPERATION`.

ROM_BOOT_MDMA_CONFIG_ERR

Memory DMA configuration error

ROM_BOOT_MDMA_SRC_ERR

MDMA source channel configuration error.

Set by the MDMA routines if after configuring the MDMA source channel to start a DMA operation, an error is generated in the `DMA_STAT.IRQERR` bit of the source channel.

ROM_BOOT_MDMA_DST_ERR

MDMA destination channel configuration error.

Set by the MDMA routines when an error is generated in the `DMA_STAT.IRQERR` bit of the destination channel (after configuring the MDMA source channel to start a DMA operation)

ROM_BOOT_MDMA_DONE_DETECT_ERR

Memory DMA completed with errors

ROM_BOOT_MDMA_SUCCESS

Memory DMA completed successfully

ROM_BOOT_PDMA_CONFIG_ERR

Peripheral DMA configuration invalid

ROM_BOOT_PDMA_SUCCESS

Peripheral DMA completed successfully

ROM_BOOT_CRC_FAILURE

MDMA CRC32 failure

Returned by the higher level `adi_rom_MemDma()` routine and the underlying `adi_rom_MemCompare()` routine if the CRC32 result of the block of data did not match the expected result

ROM_BOOT_CRC_CONFIG_ERR

MDMA CRC32 configuration error

ROM_BOOT_CRC_SUCCESS

MDMA CRC success

ROM_BOOT_CRC_CALLBACK_ERR

Error in the execution of the CRC callback.

Returned by the default CRC callback function located in the boot ROM if any of the following conditions are met:

- The struct `ADI_ROM_BOOT_CONFIG` pointer passed to the callback is a NULL pointer
- The struct `ADI_ROM_BOOT_BUFFER` pointer to the buffer to run CRC validation is a NULL pointer
- The `ROM_CBFLAG_DIRECT` flag is not set in the supplied flags parameter indicating it was a direct callback
- The struct `ADI_ROM_BOOT_HEADER` pointer located in pHeader Pointer to the boot header storage location where all boot stream block headers eventually reside for processing by the kernel is a NULL pointers

ROM_BOOT_CRC_SUPPORTED_ERR

CRC not supported error.

Returned by `adi_rom_MemDma()`, `adi_rom_MemFill()`, `adi_rom_MemCompare()` and `adi_rom_Crc32Poly()` if the supplied DMA configuration specified a MDMA channel that does not support CRC operations.

ROM_BOOT_CRC_INITCODE_ERR CRC32

Enable failure during boot.

Returned by `adi_rom_Crc32Init()` if the boot process cannot enable the CRC32 functionality due to a NULL struct `ADI_ROM_BOOT_CONFIG` pointer or NULL struct `ADI_ROM_BOOT_HEADER` pointer located in pHeader. Pointer to the boot header storage location where all boot stream block headers eventually reside for processing by the kernel

ROM_BOOT_CRC_COUNT_ERR

CRC byte count was not a multiple 4

The CRC peripheral operates on 32-bit data only and as such all CRC operations must have a byte count that is a multiple of 4. This result is returned by the higher level `adi_rom_MemDma()` routine and the underlying `adi_rom_MemCompare()` and `adi_rom_MemFill()` routines if the byte count is not a multiple of 4 bytes.

enum ROM_BOOT_MDMA_CRC_SUPPORT

Enumeration Type Declaration: `ROM_BOOT_MDMA_CRC_SUPPORT`

MDMA Channel CRC support.

Specifies whether the MDMA channel supports CRC operations or not

Table 36-72: ROM_BOOT_MDMA_CRC_SUPPORT Members

Enumerator	Description
ROM_BOOT_DMA_CRC_SUPPORTED	MDMA Channel supports CRC
ROM_BOOT_DMA_CRC_NOT_SUPPORTED	MDMA Channel does not support CRC

ROM_BOOT_DMA_CRC_SUPPORTED

MDMA Channel supports CRC

ROM_BOOT_DMA_CRC_NOT_SUPPORTED

MDMA Channel does not support CRC

enum ROM_DMA_DONE_DETECT_METHOD

Enumeration Type Declaration: ROM_DMA_DONE_DETECT_METHOD

DMA Done Detection Method.

Specifies the method used to detect the completion of the requested DMA operation.

When a program requests a non-blocking DMA operation, separate software is required to check the status of the DMA channel. The boot ROM does not provide an API for this operation.

NOTE: Trigger mode is not supported on this product

Table 36-73: ROM_DMA_DONE_DETECT_METHOD Members

Enumerator	Description
ROM_DMA_DONE_NON_BLOCKING	Return without waiting for the DMA to complete
ROM_DMA_DONE_POLL_IRQDONE	Poll on the IRQDONE bit in the DMA Status register
ROM_DMA_DONE_WAKEUP_TRIGGER	Configure a trigger to wakeup the core on completion

ROM_DMA_DONE_NON_BLOCKING

Return without waiting for the DMA to complete

ROM_DMA_DONE_POLL_IRQDONE

Poll on the IRQDONE bit in the DMA Status register

ROM_DMA_DONE_WAKEUP_TRIGGER

Configure a trigger to wakeup the core on completion

enum ROM_DMA_MDMA_ID

Enumeration Type Declaration: ROM_DMA_MDMA_ID

MDMA Channel ID.

The ID of the Memory DMA channel. Used in the `struct ROM_DMA_MDMA_CONFIG` configuration to specify the Memory DMA channel to use for operations accessible using the `adi_rom_MemDma()` routine

Table 36-74: ROM_DMA_MDMA_ID Members

Enumerator	Description
ROM_DMA_MDMA0	Memory DMA Stream 0
ROM_DMA_MDMA1	Memory DMA Stream 1
ROM_DMA_MDMA2	Memory DMA Stream 2
ROM_DMA_MDMA3	Memory DMA Stream 3
ROM_DMA_MEMDMA_END_COUNT	Number of Memory DMA Streams

ROM_DMA_MDMA0

Memory DMA Stream 0

ROM_DMA_MDMA1

Memory DMA Stream 1

ROM_DMA_MDMA2

Memory DMA Stream 2

ROM_DMA_MDMA3

Memory DMA Stream 3

ROM_DMA_MEMDMA_END_COUNT

Number of Memory DMA Streams

enum ROM_DMA_MDMA_OPERATION

Enumeration Type Declaration: `ROM_DMA_MDMA_OPERATION`

MDMA Operation.

The operation determines if only an MDMA is required, or whether a CRC operation must be used in conjunction with the MDMA.

Table 36-75: ROM_DMA_MDMA_OPERATION Members

Enumerator	Description
ROM_DMA_MEM_COPY	Standard MDMA transfer from a source to a destination
ROM_DMA_MEM_CRC	Performs a CRC32 MDMA read operation and compares the result with an expected result

Table 36-75: ROM_DMA_MDMA_OPERATION Members (Continued)

Enumerator	Description
ROM_DMA_MEM_FILL	Uses the CRC peripheral to perform a fill operation with a 32-bit value
ROM_DMA_MEM_COMPARE	Uses the CRC peripheral to compare data with a constant 32-bit value
ROM_DMA_CRC_LUT_INIT	Initializes the CRC LUT from the supplied CRC Polynomial

ROM_DMA_MEM_COPY

Standard MDMA transfer from a source to a destination

ROM_DMA_MEM_CRC

Performs a CRC32 MDMA read operation and compares the result with an expected result

ROM_DMA_MEM_FILL

Uses the CRC peripheral to perform a fill operation with a 32-bit value

ROM_DMA_MEM_COMPARE

Uses the CRC peripheral to compare data with a constant 32-bit value

ROM_DMA_CRC_LUT_INIT

Initializes the CRC LUT from the supplied CRC Polynomial

enum ROM_DMA_PDMA_OPERATION

Enumeration Type Declaration: ROM_DMA_PDMA_OPERATION

Table 36-76: ROM_DMA_PDMA_OPERATION Members

Enumerator	Description
ROM_DMA_PERI_TX	Peripheral Transmit Operation
ROM_DMA_PERI_RX	Peripheral Receive Operation

ROM_DMA_PERI_TX

Peripheral Transmit Operation

ROM_DMA_PERI_RX

Peripheral Receive Operation

enum ROM_GETADDR_VALUE

Enumeration Type Declaration: ROM_GETADDR_VALUE

Parameter for `adi_rom_GetAddress()` function to retrieve the address of a data object stored in the boot ROM.

Table 36-77: ROM_GETADDR_VALUE Members

Enumerator	Description
ROM_GETADDR_CONSTANTS	Retrieve the address of the ROM_CONSTANTS_TYPE object
ROM_GETADDR_BMODE	Retrieve the address of the lookup table sorting the default <code>adi_rom_boot()</code> parameters for each boot mode
ROM_GETADDR_MDMAREGS	Retrieve the address of the <code>struct ROM_BOOT_MDMA_REGS</code> object
ROM_GETADDR_SPILUT	Retrieve the address of the lookup table in the ROM describing the various SPI controller boot BCODE configurations
ROM_GETADDR_ECDSA_DOMAIN	Retrieve the address of the domain parameters used for ECDSA

ROM_GETADDR_CONSTANTS

Retrieve the address of the ROM_CONSTANTS_TYPE object

ROM_GETADDR_BMODE

Retrieve the address of the lookup table sorting the default `adi_rom_boot()` parameters for each boot mode

ROM_GETADDR_MDMAREGS

Retrieve the address of the `struct ROM_BOOT_MDMA_REGS` object

ROM_GETADDR_SPILUT

Retrieve the address of the lookup table in the rom describing the various SPI controller boot BCODE configurations

ROM_GETADDR_ECDSA_DOMAIN

Retrieve the address of the domain parameters used for ECDSA

enum ROM_HOOK_CALL_CAUSE

Enumeration Type Declaration: `ROM_HOOK_CALL_CAUSE`

Passed to a user hook routine to indicate the reason of the call.

An optional hook routine is provided as a callback when calling a boot mode via `adi_rom_Boot`. This hook routine is called by the boot software first after the execution of the boot modes initialization routine then again after execution of the boot modes configuration routine. This parameter allows the users routine to identify at which point the call was made allowing the user to perform different actions for each call.

Table 36-78: ROM_HOOK_CALL_CAUSE Members

Enumerator	Description
ROM_HOOK_CALL_INIT_COMPLETE	Call was as a result of completion of the boot modes initialization function
ROM_HOOK_CALL_CONFIG_COMPLETE	Call was as a result of the completion of the boot modes configuration function
ROM_HOOK_REG_COMPLETE	Call was as a result of the completion of the boot modes pre-register initialization

enum ROM_SB_IMAGE_TYPE

Enumeration Type Declaration: ROM_SB_IMAGE_TYPE

Secure Boot Image Types.

The secure boot header contains a type field for the secure boot image type, this enumeration provides a complete list of all image types.

NOTE: The secure boot process does not necessarily support all image types defined.

Table 36-79: ROM_SB_IMAGE_TYPE Members

Enumerator	Description
ROM_SB_IMAGE_UNKNOWN	Unknown Secure Boot image type, used by software to initialize the type before detection of boot image type takes place
ROM_SB_IMAGE_BLP	Plain text BLP secure boot image supports authentication only with no decryption
ROM_SB_IMAGE_BLW	Keywrapped BLW secure boot image supports authentication and decryption, boot stream decryption key wrapped in the secure header
ROM_SB_IMAGE_BLE	Not supported by any Secure boot products. Secure boot image with key stored in plain text form in the secure header
ROM_SB_IMAGE_BLX	BLX Secure boot image supports authentication and decryption, boot stream decryption key wrapped located in OTP
ROM_SB_IMAGE_UNSUPPORTED	Used by software to indicate any other unsupported image type

ROM_SB_IMAGE_UNKNOWN

Unknown Secure Boot image type, used by software to initialize the type before detection of boot image type takes place

ROM_SB_IMAGE_BLP

Plain text BLP secure boot image supports authentication only with no decryption

ROM_SB_IMAGE_BLW

Keywrapped BLw secure boot image supports authentication and decryption, boot stream decryption key wrapped in the secure header

ROM_SB_IMAGE_BLE

Not supported by any Secure boot products. Secure boot image with key stored in plain text form in the secure header

ROM_SB_IMAGE_BLX

BLx Secure boot image supports authentication and decryption, boot stream decryption key wrapped located in OTP

ROM_SB_IMAGE_UNSUPPORTED

Used by software to indicate any other unsupported image type

enum ROM_SPI_PROTOCOL

Enumeration Type Declaration: ROM_SPI_PROTOCOL

The SPI protocol to use.

SPI Flash devices can support multiple protocols to send commands to the SPI flash. Commands are usually sent over the single-bit bus, however a number of newer devices also support sending commands over the dual- or quad-bit bus.

WARNING: In system reset type events where the processor attempts to reboot and the flash may not have been reset, using the DUALIO or QUADIO protocols for the command cycles risks the boot process being unable to communicate with the SPI flash. Do not enable these features on SPI Flash devices if they are also the primary boot source used for booting from hardware reset and system reset events.

Table 36-80: ROM_SPI_PROTOCOL Members

Enumerator	Description
ROM_SPI_EXT_PROTOCOL	Extended protocol where the command cycle is sent on the single bit bus
ROM_SPI_DUALIO_PROTOCOL	DualIO protocol where the command cycle is sent on the dual bit bus
ROM_SPI_QUADIO_PROTOCOL	QuadIO protocol where the command cycle is sent on the quad bit bus

ROM_SPI_EXT_PROTOCOL

Extended protocol where the command cycle is sent on the single bit bus

ROM_SPI_DUALIO_PROTOCOL

DualIO protocol where the command cycle is sent on the dual bit bus

ROM_SPI_QUADIO_PROTOCOL

QuadIO protocol where the command cycle is sent on the quad bit bus

37 System Crossbars (SCB)

A modern system on a chip (SoCs) contains multi-cores, memory controllers, security blocks, and other high speed peripherals. As system integration increases, SoCs need to provide bus connectivity that allows better throughput to reduce performance bottlenecks. While traditional point-to-point connection buses have performed well in smaller systems, there is a need to use advanced switching based bus architectures for efficient handling of data transfer between multiplicity of data sources and sinks in the system. Additionally, mixing various traffic types in the same SoC (for example control, communication over peripherals and computing) while sharing the same bus resources, create different requirements from the Quality of Service (QoS) perspective.

The system crossbars (SCB) are the fundamental building blocks of a switch-fabric style for (on-chip) system bus interconnection. The SCBs connect system bus requesters to system bus completers, providing concurrent data transfer between multiple bus requesters and multiple bus completers. The SCB architecture addresses the challenges described above. The SCB provides sustainable throughput for simultaneous transactions in the system with configurable quality of service for each type of transaction (traffic) as required. A hierarchical model, built from multiple SCBs, provides a power and area efficient system interconnect, which satisfies the performance and flexibility requirements of a specific system.

SCB Features

The SCBs provide the following features:

- Efficient, pipelined bus transfer protocol for sustained throughput
- Full-duplex bus operation for flexibility and reduced latency
- Concurrent bus transfer support to allow multiple bus requesters to access bus completers simultaneously
- Protection model (privileged or secure) support for selective bus interconnect protection
- Simple priority-based QoS based arbitration model
- Programmable quality of service

SCB Functional Description

The following sections provide a functional description of the SCB.

- [SCB Architectural Concepts](#)

ADSP-21560_21561_21564_21568 SCB0 Register List

Table 37-1: ADSP-21560_21561_21564_21568 SCB0 Register List

Name	Description
SCB0_ACC_CH0_READ_QOS	Accelerator Channel 0 Read Quality of Service Register
SCB0_ACC_CH0_WRITE_QOS	Accelerator Channel 0 Write Quality of Service Register
SCB0_ACC_CH1_READ_QOS	Accelerator Channel 1 Read Quality of Service Register
SCB0_ACC_CH1_WRITE_QOS	Accelerator Channel 1 Write Quality of Service Register
SCB0_CRC0_CH0_READ_QOS	CRC0 Channel 0 Read Quality of Service Register
SCB0_CRC0_CH0_WRITE_QOS	CRC0 Channel 0 Write Quality of Service Register
SCB0_CRC0_CH1_READ_QOS	CRC0 Channel 1 Read Quality of Service Register
SCB0_CRC0_CH1_WRITE_QOS	CRC0 Channel 1 Write Quality of Service Register
SCB0_CRC1_CH0_READ_QOS	CRC1 Channel 0 Read Quality of Service Register
SCB0_CRC1_CH0_WRITE_QOS	CRC1 Channel 0 Write Quality of Service Register
SCB0_CRC1_CH1_READ_QOS	CRC1 Channel 1 Read Quality of Service Register
SCB0_CRC1_CH1_WRITE_QOS	CRC1 Channel 1 Write Quality of Service Register
SCB0_CRYPT0_READ_QOS	Crypto Read Quality of Service Register
SCB0_CRYPT0_WRITE_QOS	CRYPTO Write Quality of Service Register
SCB0_DBG_READ_QOS	DBG Read Quality of Service Register
SCB0_DBG_WRITE_QOS	DBG Write Quality of Service Register
SCB0_DLDMA0_CH0_READ_QOS	DLDMA0 Channel 0 Read Quality of Service Register
SCB0_DLDMA0_CH0_WRITE_QOS	DLDMA0 Channel 0 Write Quality of Service Register
SCB0_DLDMA0_CH1_READ_QOS	DLDMA0 Channel 1 Read Quality of Service Register
SCB0_DLDMA0_CH1_WRITE_QOS	DLDMA0 Channel 1 Write Quality of Service Register
SCB0_DLDMA1_CH0_READ_QOS	DLDMA1 Channel 0 Read Quality of Service Register
SCB0_DLDMA1_CH0_WRITE_QOS	DLDMA1 Channel 0 Write Quality of Service Register
SCB0_DLDMA1_CH1_READ_QOS	DLDMA1 Channel 1 Read Quality of Service Register
SCB0_DLDMA1_CH1_WRITE_QOS	DLDMA1 Channel 1 Write Quality of Service Register
SCB0_ETR_READ_QOS	ETR Read Quality of Service Register
SCB0_ETR_WRITE_QOS	ETR Write Quality of Service Register
SCB0_HSMDMA_CH0_READ_QOS	HSMDMA Channel 0 Read Quality of Service Register
SCB0_HSMDMA_CH0_WRITE_QOS	HSMDMA Channel 0 Write Quality of Service Register

Table 37-1: ADSP-21560_21561_21564_21568 SCB0 Register List (Continued)

Name	Description
SCB0_HSMDMA_CH1_READ_QOS	HSMDMA Channel 1 Read Quality of Service Register
SCB0_HSMDMA_CH1_WRITE_QOS	HSMDMA Channel 1 Write Quality of Service Register
SCB0_LP0_READ_QOS	LP0 Read Quality of Service Register
SCB0_LP0_WRITE_QOS	LP0 Write Quality of Service Register
SCB0_LP1_READ_QOS	LP1 Read Quality of Service Register
SCB0_LP1_WRITE_QOS	LP1 Write Quality of Service Register
SCB0_MLB_READ_QOS	MLB Read Quality of Service Register
SCB0_MLB_WRITE_QOS	MLB Write Quality of Service Register
SCB0_MSMDMA_CH0_READ_QOS	MSMDMA Channel 0 Read Quality of Service Register
SCB0_MSMDMA_CH0_WRITE_QOS	MSMDMA Channel 0 Write Quality of Service Register
SCB0_MSMDMA_CH1_READ_QOS	MSMDMA Channel 1 Read Quality of Service Register
SCB0_MSMDMA_CH1_WRITE_QOS	MSMDMA Channel 1 Write Quality of Service Register
SCB0_SH0_DPORT_READ_QOS	SH0 DPORT Read Quality of Service Register
SCB0_SH0_DPORT_WRITE_QOS	SH0 DPORT Write Quality of Service Register
SCB0_SH0_IPORT_READ_QOS	SH0 IPORT Read Quality of Service Register
SCB0_SH0_IPORT_WRITE_QOS	SH0 IPORT Write Quality of Service Register
SCB0_SH0_MMR_IB_READ_QOS	SH0 MMR Read Quality of Service Register
SCB0_SH0_MMR_IB_WRITE_QOS	SH0 MMR Write Quality of Service Register
SCB0_SP0A_READ_QOS	SP0A Read Quality of Service Register
SCB0_SP0A_WRITE_QOS	SP0A Write Quality of Service Register
SCB0_SP0B_READ_QOS	SP0B Read Quality of Service Register
SCB0_SP0B_WRITE_QOS	SP0B Write Quality of Service Register
SCB0_SP1A_READ_QOS	SP1A Read Quality of Service Register
SCB0_SP1A_WRITE_QOS	SP1A Write Quality of Service Register
SCB0_SP1B_READ_QOS	SP1B Read Quality of Service Register
SCB0_SP1B_WRITE_QOS	SP1B Write Quality of Service Register
SCB0_SP2A_READ_QOS	SP2A Read Quality of Service Register
SCB0_SP2A_WRITE_QOS	SP2A Write Quality of Service Register
SCB0_SP2B_READ_QOS	SP2B Read Quality of Service Register
SCB0_SP2B_WRITE_QOS	SP2B Write Quality of Service Register
SCB0_SP3A_READ_QOS	SP3A Read Quality of Service Register

Table 37-1: ADSP-21560_21561_21564_21568 SCB0 Register List (Continued)

Name	Description
SCB0_SP3A_WRITE_QOS	SP3A Write Quality of Service Register
SCB0_SP3B_READ_QOS	SP3B Read Quality of Service Register
SCB0_SP3B_WRITE_QOS	SP3B Write Quality of Service Register
SCB0_SP4A_READ_QOS	SP4A Read Quality of Service Register
SCB0_SP4A_WRITE_QOS	SP4A Write Quality of Service Register
SCB0_SP4B_READ_QOS	SP4B Read Quality of Service Register
SCB0_SP4B_WRITE_QOS	SP4B Write Quality of Service Register
SCB0_SP5A_READ_QOS	SP5A Read Quality of Service Register
SCB0_SP5A_WRITE_QOS	SP5A Write Quality of Service Register
SCB0_SP5B_READ_QOS	SP5B Read Quality of Service Register
SCB0_SP5B_WRITE_QOS	SP5B Write Quality of Service Register
SCB0_SP6A_READ_QOS	SP6A Read Quality of Service Register
SCB0_SP6A_WRITE_QOS	SP6A Write Quality of Service Register
SCB0_SP6B_READ_QOS	SP6B Read Quality of Service Register
SCB0_SP6B_WRITE_QOS	SP6B Write Quality of Service Register
SCB0_SP7A_READ_QOS	SP7A Read Quality of Service Register
SCB0_SP7A_WRITE_QOS	SP7A Write Quality of Service Register
SCB0_SP7B_READ_QOS	SP7B Read Quality of Service Register
SCB0_SP7B_WRITE_QOS	SP7B Write Quality of Service Register
SCB0_SPI0RX_READ_QOS	SPI0 RX Read Quality of Service Register
SCB0_SPI0RX_WRITE_QOS	SPI0 RX Write Quality of Service Register
SCB0_SPI0TX_READ_QOS	SPI0 TX Read Quality of Service Register
SCB0_SPI0TX_WRITE_QOS	SPI0 TX Write Quality of Service Register
SCB0_SPI1RX_READ_QOS	SPI1 RX Read Quality of Service Register
SCB0_SPI1RX_WRITE_QOS	SPI1 RX Write Quality of Service Register
SCB0_SPI1TX_READ_QOS	SPI1 TX Read Quality of Service Register
SCB0_SPI1TX_WRITE_QOS	SPI1 TX Write Quality of Service Register
SCB0_SPI2RX_READ_QOS	SPI2 RX Read Quality of Service Register
SCB0_SPI2RX_WRITE_QOS	SPI2 RX Write Quality of Service Register
SCB0_SPI2TX_READ_QOS	SPI2 TX Read Quality of Service Register
SCB0_SPI2TX_WRITE_QOS	SPI2 TX Write Quality of Service Register

Table 37-1: ADSP-21560_21561_21564_21568 SCB0 Register List (Continued)

Name	Description
SCB0_UART0_RX_READ_QOS	UART0 RX Read Quality of Service Register
SCB0_UART0_RX_WRITE_QOS	UART0 RX Write Quality of Service Register
SCB0_UART0_TX_READ_QOS	UART0 TX Read Quality of Service Register
SCB0_UART0_TX_WRITE_QOS	UART0 TX Write Quality of Service Register
SCB0_UART1_RX_READ_QOS	UART1 TX Read Quality of Service Register
SCB0_UART1_RX_WRITE_QOS	UART1 RX Write Quality of Service Register
SCB0_UART1_TX_READ_QOS	UART1 TX Read Quality of Service Register
SCB0_UART1_TX_WRITE_QOS	UART1 TX Write Quality of Service Register
SCB0_UART2_RX_READ_QOS	UART2 RX Read Quality of Service Register
SCB0_UART2_RX_WRITE_QOS	UART2 RX Write Quality of Service Register
SCB0_UART2_TX_READ_QOS	UART2 TX Read Quality of Service Register
SCB0_UART2_TX_WRITE_QOS	UART2 TX Write Quality of Service Register

ADSP-21560_21561_21564_21568 SCB3 Register List

Table 37-2: ADSP-21560_21561_21564_21568 SCB3 Register List

Name	Description
SCB3_APB_DCLK_0_IB_SYNC_MODE	APB DCLK 0 Synchronization Mode Register

ADSP-21560_21561_21564_21568 SCB4 Register List

Table 37-3: ADSP-21560_21561_21564_21568 SCB4 Register List

Name	Description
SCB4_COMP_ID_0	Comp Id 0
SCB4_COMP_ID_1	Comp Id 1
SCB4_COMP_ID_2	Comp Id 2
SCB4_COMP_ID_3	Comp Id 3
SCB4_FABRIC_S2PORT_IB_FN_MOD	Fabric S2port Ib.fn Mod
SCB4_FABRIC_S2PORT_IB_READ_QOS	Fabric S2port Ib.read Qos
SCB4_FABRIC_S2PORT_IB_WRITE_QOS	Fabric S2port Ib.write Qos
SCB4_FIR_CH0_IB_FN_MOD	Fir Ch0 Ib.fn Mod
SCB4_FIR_CH0_IB_FN_MOD2	Fir Ch0 Ib.fn Mod2
SCB4_FIR_CH0_IB_READ_QOS	Fir Ch0 Ib.read Qos

Table 37-3: ADSP-21560_21561_21564_21568 SCB4 Register List (Continued)

Name	Description
SCB4_FIR_CH0_IB_WRITE_QOS	Fir Ch0 Ib.write Qos
SCB4_FIR_CH1_IB_FN_MOD	Fir Ch1 Ib.fn Mod
SCB4_FIR_CH1_IB_FN_MOD2	Fir Ch1 Ib.fn Mod2
SCB4_FIR_CH1_IB_READ_QOS	Fir Ch1 Ib.read Qos
SCB4_FIR_CH1_IB_WRITE_QOS	Fir Ch1 Ib.write Qos
SCB4_IIR_CH0_IB_FN_MOD	Iir Ch0 Ib.fn Mod
SCB4_IIR_CH0_IB_FN_MOD2	Iir Ch0 Ib.fn Mod2
SCB4_IIR_CH0_IB_READ_QOS	Iir Ch0 Ib.read Qos
SCB4_IIR_CH0_IB_WRITE_QOS	Iir Ch0 Ib.write Qos
SCB4_IIR_CH1_IB_FN_MOD	Iir Ch1 Ib.fn Mod
SCB4_IIR_CH1_IB_FN_MOD2	Iir Ch1 Ib.fn Mod2
SCB4_IIR_CH1_IB_READ_QOS	Iir Ch1 Ib.read Qos
SCB4_IIR_CH1_IB_WRITE_QOS	Iir Ch1 Ib.write Qos
SCB4_PERIPH_ID_0	Periph Id 0
SCB4_PERIPH_ID_1	Periph Id 1
SCB4_PERIPH_ID_2	Periph Id 2
SCB4_PERIPH_ID_3	Periph Id 3
SCB4_PERIPH_ID_4	Periph Id 4
SCB4_PERIPH_ID_5	Periph Id 5
SCB4_PERIPH_ID_6	Periph Id 6
SCB4_PERIPH_ID_7	Periph Id 7
SCB4_SHARC_DPORT_FN_MOD	Sharc Dport.fn Mod
SCB4_SHARC_DPORT_READ_QOS	Sharc Dport.read Qos
SCB4_SHARC_DPORT_WRITE_QOS	Sharc Dport.write Qos

ADSP-21560_21561_21564_21568 SCB5 Register List

SPI2. If set to 1 it routes the transactions to SPI3(OSPI).

Table 37-4: ADSP-21560_21561_21564_21568 SCB5 Register List

Name	Description
SCB5_COMP_ID_0	Comp Id 0
SCB5_COMP_ID_1	Comp Id 1

Table 37-4: ADSP-21560_21561_21564_21568 SCB5 Register List (Continued)

Name	Description
SCB5_COMP_ID_2	Comp Id 2
SCB5_COMP_ID_3	Comp Id 3
SCB5_PERIPH_ID_0	Periph Id 0
SCB5_PERIPH_ID_1	Periph Id 1
SCB5_PERIPH_ID_2	Periph Id 2
SCB5_PERIPH_ID_3	Periph Id 3
SCB5_PERIPH_ID_4	Periph Id 4
SCB5_PERIPH_ID_5	Periph Id 5
SCB5_PERIPH_ID_6	Periph Id 6
SCB5_PERIPH_ID_7	Periph Id 7
SCB5_REMAP	Remap
SCB5_SPI2_S_IB_FN_MOD	Spi2 S Ib.fn Mod
SCB5_SPI2_S_IB_FN_MOD_ISS_BM	Spi2 S Ib.fn Mod Iss Bm
SCB5_SPIF_M_FN_MOD	Spif M.fn Mod

SCB Architectural Concepts

This section describes the components of the SCB and the modules connected to it. The basic elements in the SCB are SCB requesters, completers, requester interfaces, and completer interfaces.

Requesters

The system bus requesters include the peripheral Direct Memory Access (DMA) channels. These include the Serial Port (SPORT) DMA, SPI DMAs, Memory-to-Memory DMA channels (MDMA), the L1 code fill block, and the processor cores.

Completers

Completers are SCB connections that are responding to transfer requests. Completers include MMR registers, memory units, and various peripherals depending upon individual configurations. Each system completer has its own latencies and response times.

SCB Block Diagram

The SCB architectural model is shown in the *SCB Overview* figure. This figure shows a high level representation of a basic SCB connecting n completers to x requesters. A variable number of requesters connect to a variable number of completers in each SCB. In this example, all SIs connect to all MIs as indicated by the lines connecting them.

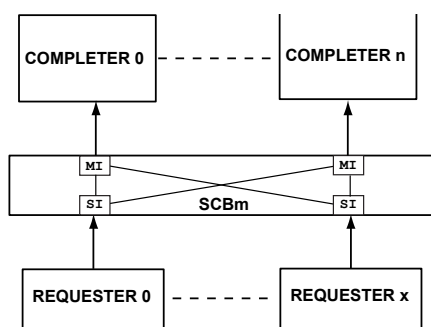


Figure 37-1: SCB Overview

Hierarchy Block Diagram

A system interconnect built from multiple SCBs in a hierarchical model is illustrated in the *SCB Hierarchy Overview* figure. The system requester node level SCBs requester connects multiple SIs to a single MI, which in turn connects to an SI of the system completer level node SCB.

As discussed above, all the requesters in the system are distributed across different SCBs. A given SCB at system requester node level connects directly to the system requesters. These SCBs connect to SCB0 through its SIs forming a hierarchical structure. While a requester must access any completer, its first access goes through the SCB it is connected to, and then through SCB0, to reach its intended completer. This simplifies the connecting hardware in the basic SCB block by limiting the requesters. Care must be taken when sharing requesters to allow adequate throughput for their individual data transfer requirements.

In this example, all SIs are connected to all MIs.

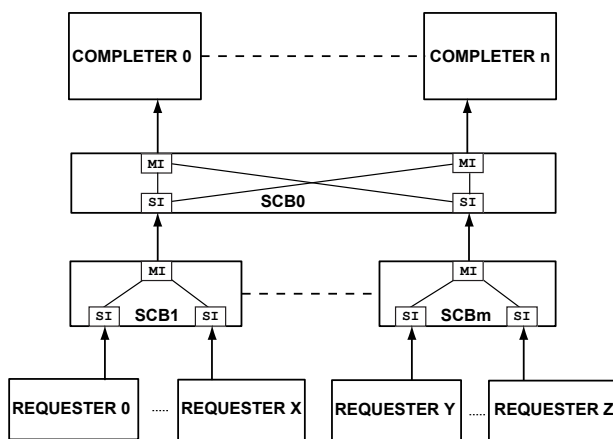


Figure 37-2: SCB Hierarchy Overview

NOTE: For an overall diagram of all SCB interconnections, see the [SCB Block Diagram](#).

SCB Block Diagram

The figure shows the SCB block diagram.

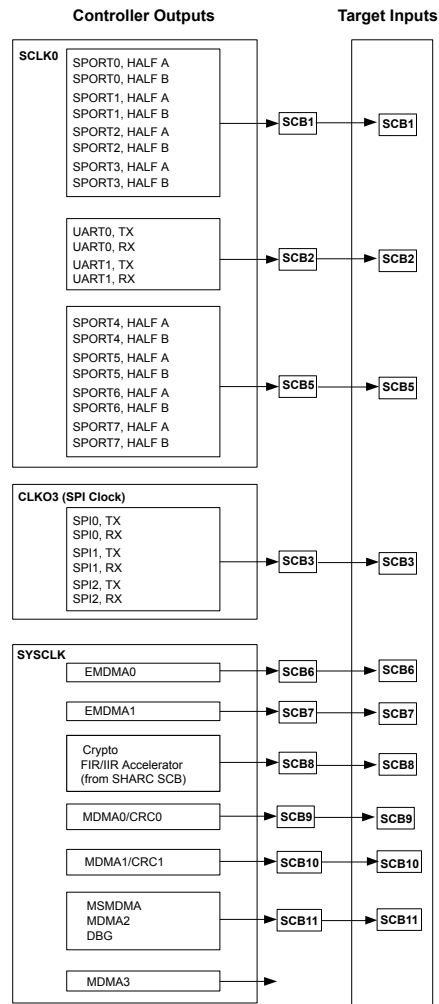


Figure 37-3: SCB Block Diagram

SHARC SCB with FIR/IIR Accelerator

The FIR/IIR accelerators on the ADSP-2156x processors are integrated with the SHARC+ core and operate at core clock (CCLK). This allows the FIR/IIR accelerator requestor ports to directly access the SHARC L1 memory with reduced latency as these accesses do not go through the main system fabric. SHARC+ core can also access the FIR/IIR accelerator MMR registers directly. A mini-SCB is added to support the SHARC SCB feature.

The figure shows the SHARC SCB block diagram.

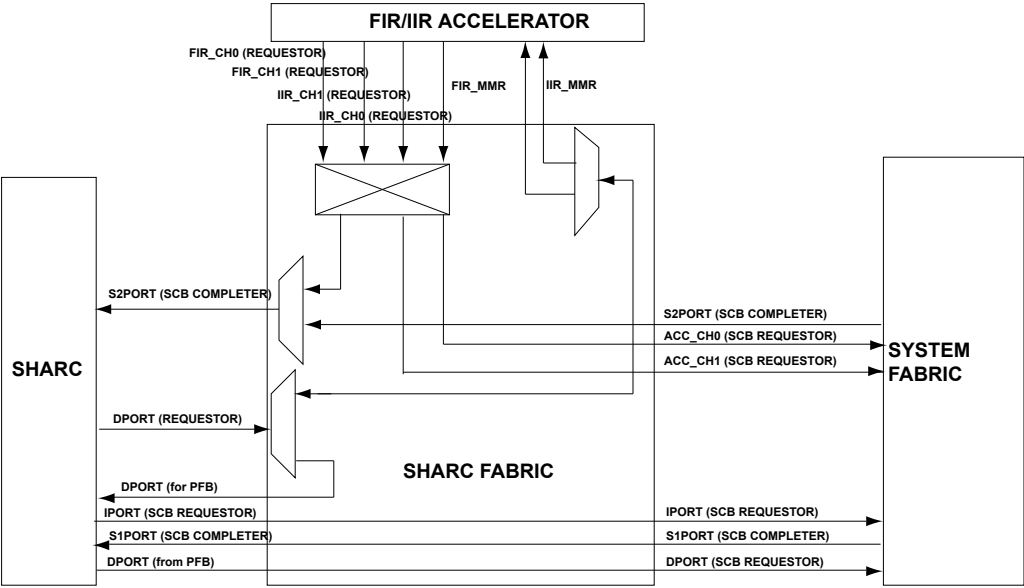


Figure 37-4: SHARC SCB with FIR/IIR Accelerator

Only SHARC+ core can access FIR/IIR accelerator MMRs. No other requestors can access accelerator MMR space, including the DBG requestor. Debug can be done through SHARC+ core only. If the core hang, the accelerators cannot be debugged.

- SHARC+ core and FIR/IIR accelerators share the same security level.
- SHARC+ core’s MMR port is separated from data port. It is a separate requestor port for the main system fabric.
- Prefetch Buffer: After MMR split, the data port comes back into the SHARC+ core for PFB connection and goes to the main SCB via SHARC SCB with the clock domain crossing.

There are two types of peripherals that use DMA. The first have dedicated DMA channels controlled by the Dedicated DMA Engine (DDE) and have the same operating modes (see [DMA Operating Modes](#)) and use the same programming model ([DMA Channel Programming Model](#)). The second type is not controlled by the DDE. These peripherals have their own operating modes and programming models (see the peripheral chapter for this information). The peripheral types are shown in the *SCB-Controlled DMA Channel Peripherals* table.

Table 37-5: SCB Controlled DMA Channel Peripherals

	Requestors	DDE DMA Channels	Non DDE DMA Channels
SCB1	SPORT0, HALF A	DMA0	
	SPORT0, HALF B	DMA1	
	SPORT1, HALF A	DMA2	
	SPORT1, HALF B	DMA3	
	SPORT2, HALF A	DMA4	
	SPORT2, HALF B	DMA5	

Table 37-5: SCB Controlled DMA Channel Peripherals (Continued)

	Requestors	DDE DMA Channels	Non DDE DMA Channels
	SPORT3, HALF A	DMA6	
	SPORT3, HALF B	DMA7	
SCB2	UART0, TX	DMA20	
	UART0, RX	DMA21	
	UART1, TX	DMA34	
	UART1, RX	DMA35	
	UART2, TX	DMA37	
	UART2, RX	DMA38	
SCB3	SPI0, TX	DMA22	
	SPI0, RX	DMA23	
	SPI1, TX	DMA24	
	SPI1, RX	DMA25	
	SPI2, TX	DMA26	
	SPI2, RX	DMA27	
SCB5	SPORT4, HALF A	DMA10	
	SPORT4, HALF B	DMA11	
	SPORT5, HALF A	DMA12	
	SPORT5, HALF B	DMA13	
	SPORT6, HALF A	DMA14	
	SPORT6, HALF B	DMA15	
	SPORT7, HALF A	DMA16	
	SPORT7, HALF B	DMA17	
SCB6	EMDMA0	N/A	1 channel
SCB7	EMDMA1	N/A	1 channel
SCB8	CRYPTO	N/A	2 channels (1 read, 1 write)
	FIR/IIR Accelerator (from SHARC SCB)	N/A	1 channel
SCB9	CRC0/MDMA0	DMA8, DMA9	
SCB10	CRC1/MDMA1	DMA18, DMA19	
SCB11	MDMA2	DMA39, DMA40	
	DBG	N/A	
		MDMA3	DMA43, DMA44

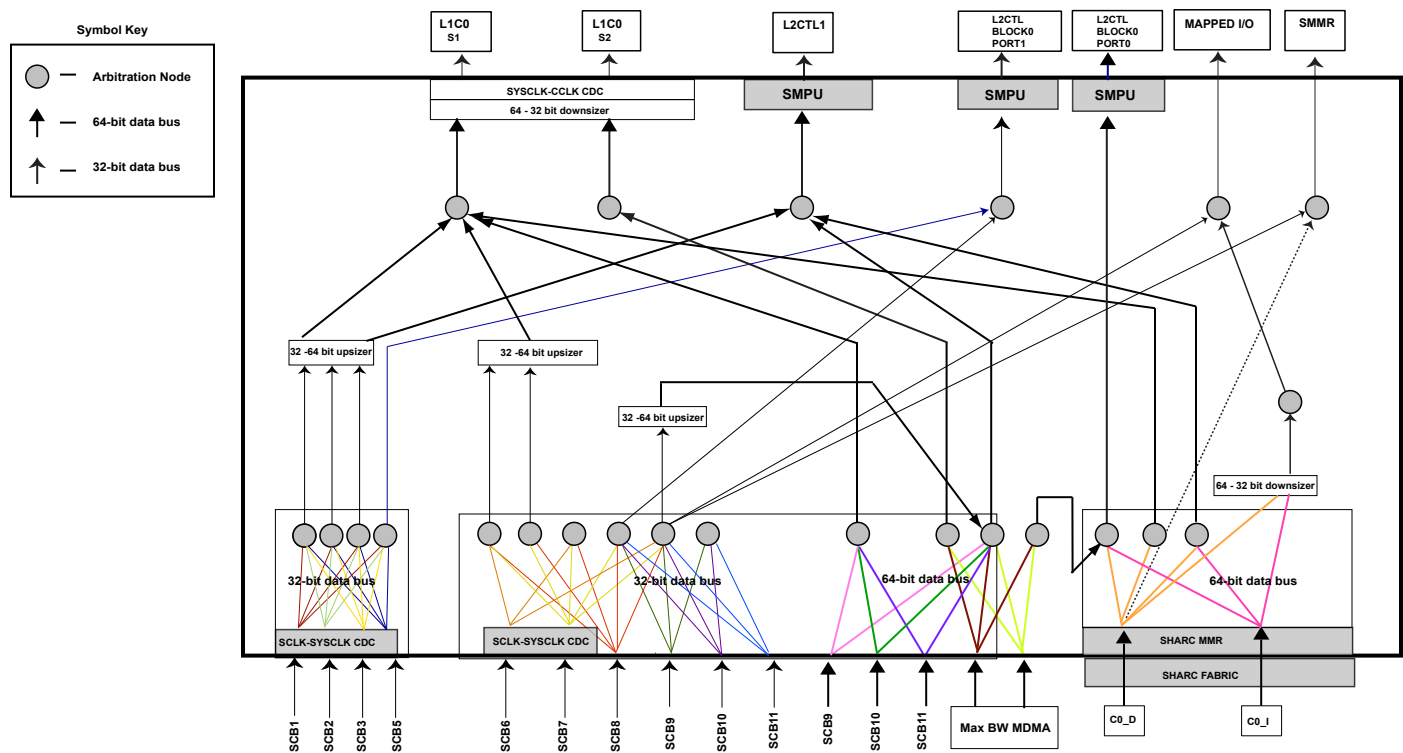


Figure 37-5: SCB Block Diagram - Interconnections

This figure is useful for the overview. It is also useful to observe the following relationships that are highlighted.

- The hierarchy of SCBs manages system bus interconnections, multiplexing, and arbitration among the peripherals on the processor.
- The SCBs connections support DMA channels for some peripherals and support dedicated connections for others. The connections also support memory-mapped register access for internal memory (L1 and L2) and for external memory.

The bus completer interface of the crossbar (where the requestors such as DMA connect) performs two functions. The first function is arbitration. SCBx handles arbitration. The second function is clock conversion. The programmable QoS registers can be viewed as being associated with the SCBx.

- Most of the peripherals and their SCBs are in the *SCLK0* domain. Crypto is also in the *SCLK0* domain. Enhanced bandwidth MDMA streams, Max BW MDMA, EMDMA0/1, Crypto, FIR/IIR accelerators, MLB, DBG and their SCBs are in *SYCLK* domain.
- Each peripheral has a latency for access across the SCB. The latency varies with the nature of the peripheral. Also, the number of active peripherals (especially for cases where multiple peripherals are active on a shared SCB) affects SCB performance. Refer to the [L2 System Memory](#) chapter for more details on the L2 memory organization.

MDMA channels are unidirectional. For example, MDMA SRC is read-only and MDMA is write-only. Access to the MMR space of SCB0 is allowed only in secure mode. The MMR space of SCB0 has registers for programming the QoS of various requestors and controlling the clock domain crossing.

- Access to the QoS register space is allowed only in secure mode.
- CDC programmability is provided only for DDR clock domain. CDC for CCLK:SYSCLK is fixed to SYNC n:1, SCLK0:SYSCLK is fixed to 1:n, and SPI clock:SYSCLK is fixed to m:n.
- S2 bus completer port of SHARCs is intended primarily for MDMA between the L1 of the 2 SHARCs. Hence, only the Max BW MDMA is given access to S2 ports while all others requestors are given access to S1 port. This arrangement also means that S1 and S2 port now share the same address space as against the ADSP-2156x family of processors where an alias address space was maintained.
- SPIF is a low bandwidth bus completer – sclk0/1 with 32 bit interface. Enhanced BW MDMA will be used to perform MDMA to them. Max BW MDMA is intended only for L1, L2 and DDR. (These bus completers have the same bandwidth capability as Max BW MDMA).
- SPIF is a read-only flash. Peripherals are not able to read out of the SPI flash directly. Only MDMAs, crypto, cores, and debug tools are given access to it.

The following acronyms are used in the figure.

SCB0-11

Indicate SCB interfaces, connecting the system bus requestors and bus completers

SCLK0, SCLK1, SYSCLK

Indicate clock domains in which the specific SCBs operate. For more information on clock domains, see the Clock Generation Unit (CGU) chapter and the product data sheet.

CDC

Indicates the clock domain crossing

C0_D

Indicates the SHARC processor data bus (DM/PM)

C0_I

Indicates the SHARC processor instruction bus (PM)

L1C0 S1, L1C0 S2, L1C1 S1, L1C1 S2

Indicates the L1 memory for SHARC processor completer port (1 or 2)

SPO A/B, SP1 A/B

Indicates the serial port interfaces and their full-duplex halves

SPIO, SPI1 - RX/TX

Indicates the serial peripheral interfaces ports with receive or transmit paths

DBG

Indicates the Debug Port (DBG) that provide access to debug capabilities

SMPU

Indicates the system memory protection unit (SMPU)

MDMA0, MDMA1

Indicates the memory DMA 0 through 1 interfaces

SMMR

Indicates the system memory-mapped register interface

System Crossbars

The System Crossbars (SCB) are the fundamental building blocks of the system bus interconnect. The SCB (often referred to as the system interconnect fabric), is a collection of inter-connection units connecting system requesters to completer memory spaces. The SCB connects one or more requester devices to one or more memory-mapped completer devices. Each connected requester can be a core that originates an SCB transaction, or a requester interface of an upstream SCB cascaded interconnect. Each connected completer can be the final completer of an SCB transaction or a completer interface of a downstream cascaded SCB interconnect (forming a hierarchy of SCBs).

Each SCB that has multiple requesters and completers share the total bandwidth of the SCB. (In a M:N configuration where M requesters are connected to N completers through the SCBx.)

The SCB provides separate channels for reads and writes. Read and write accesses through a given SCB do not share bandwidth. Only SCB0, which is the major SCB in the SCB hierarchy, has the multiple paths between multiple requester and completer interfaces.

See the [SCB Block Diagram](#).

All other SCBs in the chip connect to SCB0 through different completer interfaces. Other primary requesters (DMAs, cores, and so on) in the system are distributed across these small SCBs. For a given SCB, all the requester and completers share the total bandwidth of the SCB. (Only SCB0 is the exception). Since different DMA channels are scattered across different SCBs (SCB1, SCB2 SCB3, and so on), they do not conflict for the bandwidth when

they are in different SCBs and accessing different completers. SCB0 allows for concurrent data transfer between multiple bus requesters and multiple bus completers, providing flexibility, and full-duplex operation.

If system accesses are carefully architected, SCB has a potential of providing sufficient sustained bandwidth in the end system.

Since the SCBs support burst transfers, it is important to configure the requesting requester appropriately to make best use of available SCB bandwidth. For a DMA requester, choosing the appropriate `DMA_CFG.MSIZE` value, is important from both a functional and a performance perspective. The value in the `DMA_CFG.PSIZE` bit field determines the width of the peripheral bus in use. It can be configured to 1-byte, 2-bytes, or 4-bytes. The `DMA_CFG.MSIZE` value determines the actual size of the SCB bus in use. It also determines the minimum number of bytes which are transferred from or to memory corresponding to a single DMA request or grant. The transfer can be 1-, 2-, 4-, 8-, 16-, or 32-bytes. If the `DMA_CFG.MSIZE` value is greater than the SCB bus width, the SCB performs burst transfers according to the width defined in `DMA_CFG.MSIZE`. When `DMA_CFG.MSIZE` is less than the SCB bus width, bursting is not supported and partial bus use results.

Each of the SCB unit in the fabric consists of N completer interfaces (MSTn). Each of these interfaces has controls for read quality of service, write quality of service, and functional mode. A subset of these matrices includes controls for IB (Interface Block) sync mode, and bus functional mode. For more details on IB, see the clock domain synchronization section.

SCB Bus Requester IDs

The SCB *Bus Requester IDs* table indicates which bus requesters are connected to each of the bus completer ports of SCB0. The tables also indicate the precise value of the ID as seen by the bus completer. These values are useful for SWU programming.

NOTE: For an overall diagram of all SCB interconnections, see the [SCB Block Diagram](#).

Table 37-6: Bus Requester IDs

Bus Requester	Hex ID Values	Binary Values
DMA0 (SPORT0, HALF A)	0x0, 0x8	11'b00000000x000
DMA1 (SPORT0, HALF B)	0x1, 0x9	11'b00000000x001
DMA2 (SPORT1, HALF A)	0x2, 0xA	11'b00000000x010
DMA3 (SPORT1, HALF B)	0x3, 0xB	11'b00000000x011
DMA4 (SPORT2, HALF A)	0x4, 0xC	11'b00000000x100
DMA5 (SPORT2, HALF B)	0x5, 0xD	11'b00000000x101
DMA6 (SPORT3, HALF A)	0x6, 0xE	11'b00000000x110
DMA7 (SPORT3, HALF B)	0x7, 0xF	11'b00000000x111
DMA8 (Enh BW MDMA0 CRC, CH0)	0x86, 0x8E	11'b0001000x110
DMA9 (Enh BW MDMA0 CRC, CH1)	0x87, 0x8F	11'b0001000x111

Table 37-6: Bus Requester IDs (Continued)

Bus Requester	Hex ID Values	Binary Values
MLB	0x104	11'b00100000100
DMA20 (UART0, TX)	0x180, 0x188	11'b0011000x000
DMA21 (UART0, RX)	0x184, 0x18C	11'b0011000x100
DMA22 (SPI0, TX)	0x200, 0x208	11'b0100000x000
DMA23 (SPI0, RX)	0x201, 0x209	11'b0100000x001
DMA24 (SPI1, TX)	0x202, 0x20A	11'b0100000x010
DMA25 (SPI1, RX)	0x203, 0x20B	11'b0100000x011
DMA26 (SPI2, TX)	0x205, 0x20D	11'b0100000x101
DMA27 (SPI2, RX)	0x204, 0x20C	11'b0100000x100
DMA34 (UART1, TX)	0x181, 0x189	11'b0011000x001
DMA35 (UART1, RX)	0x182, 0x18A	11'b0011000x010
CRYPTO	0x302	11'b01100000010
ACC (CH0)	There are 32 values. "x" is either a 0 or 1.	11'b0110xxxx000
ACC (CH1)	There are 32 values. "x" is either a 0 or 1.	11'b0110xxxx001
EMDMA0 (CH0)	0x380	11'b01110000000
EMDMA0 (CH1)	0x381	11'b01110000001
EMDMA1 (CH0)	0x401	11'b10000000001
EMDMA1 (CH1)	0x400	11'b10000000000
DMA39 (Enh BW MDMA2, CH0)	0x480, 0x488	11'b1001000x000
DMA40 (Enh BW MDMA2, CH1)	0x481, 0x489	11'b1001000x001
DBG	0x482	11'b10010000010
DMA18 (Enh BW MDMA1 CRC1, CH0)	0x504, 0x50C	11'b1010000x100
DMA19 (Enh BW MDMA1 CRC1, CH1)	0x505, 0x50D	11'b1010000x101
SH0 (DPORT)	There are 32 values. "x" is either a 0 or 1.	11'b1011xxxx000
SH0 (IPORT)	0x581	11'b10110000001
DMA43 (Enh BW MDMA3, CH0)	0x600, 0x608	11'b1100000x000
DMA44 (Enh BW MDMA3, CH1)	0x601, 0x609	11'b1100000x001
DMA10 (SPORT4, HALFA)	0x680, 0x688	11'b1101000x000
DMA11 (SPORT4, HALFB)	0x681, 0x689	11'b1101000x001
DMA12 (SPORT5, HALFA)	0x682, 0x68A	11'b1101000x010
DMA13 (SPORT5, HALFB)	0x683, 0x68B	11'b1101000x011

Table 37-6: Bus Requester IDs (Continued)

Bus Requester	Hex ID Values	Binary Values
DMA14 (SPORT6, HALFA)	0x684, 0x68C	11'b1101000x100
DMA15 (SPORT6, HALFB)	0x685, 0x68D	11'b1101000x101
DMA16 (SPORT7, HALFA)	0x686, 0x68E	11'b1101000x110
DMA17 (SPORT7, HALFB)	0x687, 0x68F	11'b1101000x111
SH0_MMR	There are 32 values. "x" is either a 0 or 1.	11'b1110xxxx000

SCB Programming Model

The following sections provide information for programming the SCB properly.

Programming SCB Arbitration

Each bus completer interface has a QoS value (priority) associated with both read and write channels. These values are 4 bits present in the SCB0_MSTx_RQOS and SCB0_MSTx_WQOS registers. At the entry point to the infrastructure, all transactions are allocated this programmable local QoS value. The arbitration of the transaction throughout the infrastructure uses this QoS. At any arbitration node, a fixed priority exists for transactions with a different QoS. The highest value has the highest priority.

If there are coincident transactions at an arbitration node with the same QoS that require arbitration, then the network uses a Least Recently Granted (LRG) algorithm. At each switch, the bus requester with the highest QoS acquires access and that switch output takes the QoS value of the winner for that transaction. At the next switch bus completer interface, the bus requester uses the QoS value of the winner. QoS can have values from 0 (lowest priority) to 15 (highest priority).

For example in the following figure, *SCB Arbitration*:

1. At SCB1, bus requestors (1, 2, 3) have RQOS values of (6, 4, 2)
2. At SCB2, bus requestors (4, 5, 6) have RQOS values of (12, 13, 1)

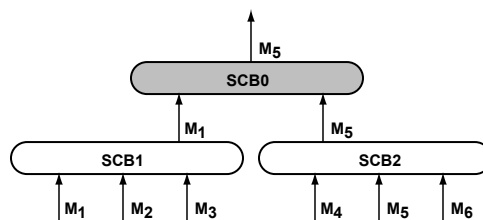


Figure 37-6: SCB Arbitration

In this case, bus requestor 1 wins at SCB1, and bus requestor 5 wins at SCB2. However, in a perfect competition at SCB0, bus requestors 4 and 5 had the highest overall RQOS values. Bus requestors 4 and 5 would have fought for arbitration directly at SCB0. However, because of the mini-SCBs, bus requestor 1, at a much lower RQOS value, is able to win against bus requestor 4 and make it all the way to SCB0.

Programming Clock Domain Crossing Registers

The clock domain crossings in the fabric are defined as follows:

- CCLK0:SYSCLK SYNC n:1 (SHARC core clock)
- SCLK0:SYSCLK SYNC 1:n
- SPICLK: SYSCLK m:n

The *Sync Mode Bit Field Description* table describes the sync modes.

Table 37-7: Sync Mode Bit Field Description

Sync Mode	Description
0	sync 1:1
1	sync n:1
2	sync 1:n
3	sync m:n
4	async

To change the clock domain crossing mode, follow the actions described in the *Changing Clock Domain Crossing Modes* table.

Table 37-8: Changing Clock Domain Crossing Modes

Original Mode	Required Mode	Action
ASYN	Any other mode	Change the clocks then change the MMR register
Any mode	ASYN	Change the MMR register then change clocks to ASYN.
m:n	1:1	Change the clocks, then change the register.
1:1	m:n	Change the register, then change the clocks.

SCB Programming Concepts

The SCB arbitration model among bus requestor or completer SCBs of the processor is fixed (not programmable). But, each bus completer does have a quality of service (QoS) programmable feature that affects arbitration.

The arbitration of transactions in SCB is based on the QoS value or the priority of the transaction. All bus requestors with the same priority form a priority group. Arbitration is granted to the highest priority group from which a member is trying to win access, and within that group, to the highest bus requestor at that time. When a bus requestor wins arbitration, it is relegated to the bottom of its group to ensure that it cannot prevent other bus requestors in its group from accessing the bus completer.

If you configure all bus requestor priorities to different levels, the arbiter implements a fixed priority scheme. This scheme occurs because each bus requestor is in a group of its own, and therefore, bus requestors maintain their ordering.

The LRG and fixed priority modes concurrently exist when the bus requestor priority value registers are programmed with a combination of identical and unique values.

NOTE: The SCB arbitration hierarchy is fixed (for example, SCB1 bus requestor to SCB1 bus completer). However, multiple bus requestor inputs to the same bus completer permit QoS programming.

The *LRG Arbitration Example* figure shows three groups with different QoS values. Bus requestors in the same group share a QoS value. The arbitration occurs using an LRG scheme.

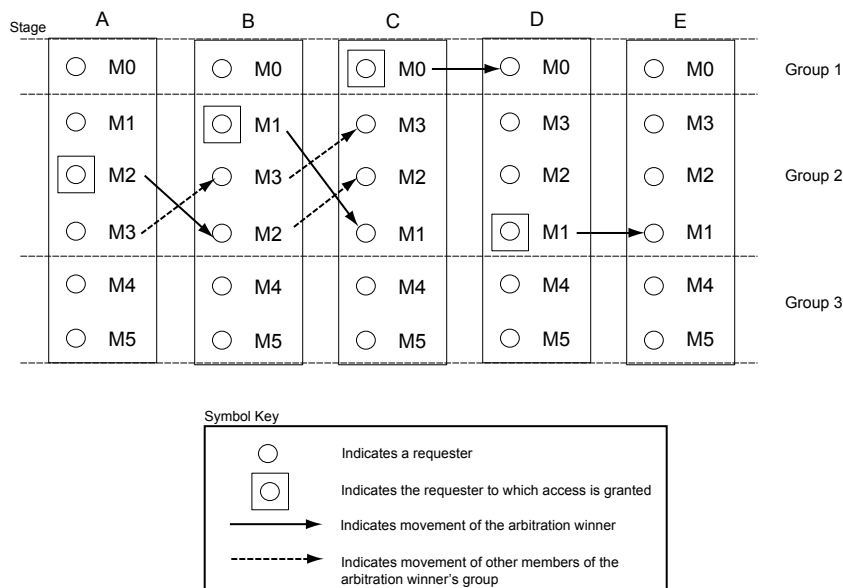


Figure 37-7: LRG Arbitration Example

The QoS value assigned to a transaction at entry point is carried forward by the transaction as it passes through all arbitration stages in the SCB. QoS for all bus requestors is configured as programmable in the system fabric interconnect.

The priority of the bus requestors fits into three groups:

- Group A - Peripherals with an external interface, without flow control and with real-time processing requirements
- Group B - Core
- Group C - Peripherals with flow control, MDMAs, and offload engines

Group A: Peripherals with external interface, without flow control and with real-time processing requirements

These bus requestors are assigned the highest priority. They are latency-critical – a large increase in latency could potentially result in data corruption and catastrophic failure. This high priority SCB group includes bus requestors connected to SCB1, SCB2, SCB3, and SCB5. Bus requestors in this group are assigned the QoS reset value of 12/10.

Group B: Core

Core is latency-sensitive. Once a core requests data, it typically waits for the data without performing anything else in parallel. An increase in the latency has a direct impact on the performance of the core. Core is assigned medium priority and a QoS reset value of 7.

Group C: Peripherals with flow control, MDMAs, and offload engines

These bus requestors are assigned the lowest priority. Peripherals with flow control can tolerate some increase in latency. MDMAs and offload engines are data intensive and do not have any external interface. Bus requestors in this group are assigned the QoS reset value of 3/1. The Group C bus requestors are connected to SCB4, SCB6, SCB7, SCB8, SCB9, SCB10, and SCB11.

Table 37-9: QoS Register Table

Bus Requestor ID	Bus Requestor	read_qos Reset Value	write_qos Reset Value
1	SPORT0_A_DMA	12	12
2	SPORT0_B_DMA	12	12
3	SPORT1_A_DMA	12	12
4	SPORT1_B_DMA	12	12
5	SPORT2_A_DMA	12	12
6	SPORT2_B_DMA	12	12
7	SPORT3_A_DMA	12	12
8	SPORT3_B_DMA	12	12
9	SPORT4_A_DMA	12	12
10	SPORT4_B_DMA	12	12
11	SPORT5_A_DMA	12	12
12	SPORT5_B_DMA	12	12
13	SPORT6_A_DMA	12	12
14	SPORT6_B_DMA	12	12
15	SPORT7_A_DMA	12	12
16	SPORT7_B_DMA	12	12
17	SPI0TX	12	12
18	SPI0RX	12	12
19	SPI1TX	12	12
20	SPI1RX	12	12
21	SPI2TX	12	12
22	SPI2RX	12	12

Table 37-9: QoS Register Table (Continued)

Bus Requestor ID	Bus Requestor	read_qos Reset Value	write_qos Reset Value
23	MLB	12	12
24	UART0_TX	10	10
25	UART0_RX	10	10
26	UART1_TX	10	10
27	UART1_RX	10	10
30	SH0_DPORT	7	7
31	SH0_IPORT	7	7
32	SH0_MMR	7	7
33	DBG	3	3
35	EMDMA0_CH0	1	1
36	EMDMA0_CH1	1	1
37	EMDMA1_CH0	1	1
38	EMDMA1_CH1	1	1
41	CRYPTO	1	1
42	ACC_CH0	1	1
43	ACC_CH1	1	1
44	MDMA0_CH0	1	1
45	MDMA0_CH1	1	1
46	MDMA1_CH0	1	1
47	MDMA1_CH1	1	1
48	MDMA2_CH0	1	1
49	MDMA2_CH1	1	1
50	MDMA3_CH0	1	1
51	MDMA3_CH1	1	1

QoS Programming

Adhere to the following guidelines if software is modifying the reset value of QoS:

- The highest QoS of any requester in a low priority SCB group must be less than the lowest QoS of any requester in a medium priority SCB group
- The highest QoS of any requester in medium priority SCB group must be less than the lowest QoS of any requester in high priority SCB group

Reset values of QoS have been spaced out to allow software to lower or higher the priority of a few requesters without having to modify the priority all the requesters. For example, the QoS of the LP can be reduced from 12 to 11 without violating the guidelines.

ADSP-21560_21561_21564_21568 SCB0 Register Descriptions

(SCB0) contains the following registers.

Table 37-10: ADSP-21560_21561_21564_21568 SCB0 Register List

Name	Description
SCB0_ACC_CH0_READ_QOS	Accelerator Channel 0 Read Quality of Service Register
SCB0_ACC_CH0_WRITE_QOS	Accelerator Channel 0 Write Quality of Service Register
SCB0_ACC_CH1_READ_QOS	Accelerator Channel 1 Read Quality of Service Register
SCB0_ACC_CH1_WRITE_QOS	Accelerator Channel 1 Write Quality of Service Register
SCB0_CRC0_CH0_READ_QOS	CRC0 Channel 0 Read Quality of Service Register
SCB0_CRC0_CH0_WRITE_QOS	CRC0 Channel 0 Write Quality of Service Register
SCB0_CRC0_CH1_READ_QOS	CRC0 Channel 1 Read Quality of Service Register
SCB0_CRC0_CH1_WRITE_QOS	CRC0 Channel 1 Write Quality of Service Register
SCB0_CRC1_CH0_READ_QOS	CRC1 Channel 0 Read Quality of Service Register
SCB0_CRC1_CH0_WRITE_QOS	CRC1 Channel 0 Write Quality of Service Register
SCB0_CRC1_CH1_READ_QOS	CRC1 Channel 1 Read Quality of Service Register
SCB0_CRC1_CH1_WRITE_QOS	CRC1 Channel 1 Write Quality of Service Register
SCB0_CRYPT0_READ_QOS	Crypto Read Quality of Service Register
SCB0_CRYPT0_WRITE_QOS	CRYPTO Write Quality of Service Register
SCB0_DBG_READ_QOS	DBG Read Quality of Service Register
SCB0_DBG_WRITE_QOS	DBG Write Quality of Service Register
SCB0_DLDMA0_CH0_READ_QOS	DLDMA0 Channel 0 Read Quality of Service Register
SCB0_DLDMA0_CH0_WRITE_QOS	DLDMA0 Channel 0 Write Quality of Service Register
SCB0_DLDMA0_CH1_READ_QOS	DLDMA0 Channel 1 Read Quality of Service Register
SCB0_DLDMA0_CH1_WRITE_QOS	DLDMA0 Channel 1 Write Quality of Service Register
SCB0_DLDMA1_CH0_READ_QOS	DLDMA1 Channel 0 Read Quality of Service Register
SCB0_DLDMA1_CH0_WRITE_QOS	DLDMA1 Channel 0 Write Quality of Service Register
SCB0_DLDMA1_CH1_READ_QOS	DLDMA1 Channel 1 Read Quality of Service Register
SCB0_DLDMA1_CH1_WRITE_QOS	DLDMA1 Channel 1 Write Quality of Service Register
SCB0_ETR_READ_QOS	ETR Read Quality of Service Register

Table 37-10: ADSP-21560_21561_21564_21568 SCB0 Register List (Continued)

Name	Description
SCB0_ETR_WRITE_QOS	ETR Write Quality of Service Register
SCB0_HSMDMA_CH0_READ_QOS	HSMDMA Channel 0 Read Quality of Service Register
SCB0_HSMDMA_CH0_WRITE_QOS	HSMDMA Channel 0 Write Quality of Service Register
SCB0_HSMDMA_CH1_READ_QOS	HSMDMA Channel 1 Read Quality of Service Register
SCB0_HSMDMA_CH1_WRITE_QOS	HSMDMA Channel 1 Write Quality of Service Register
SCB0_LP0_READ_QOS	LP0 Read Quality of Service Register
SCB0_LP0_WRITE_QOS	LP0 Write Quality of Service Register
SCB0_LP1_READ_QOS	LP1 Read Quality of Service Register
SCB0_LP1_WRITE_QOS	LP1 Write Quality of Service Register
SCB0_MLB_READ_QOS	MLB Read Quality of Service Register
SCB0_MLB_WRITE_QOS	MLB Write Quality of Service Register
SCB0_MSMDMA_CH0_READ_QOS	MSMDMA Channel 0 Read Quality of Service Register
SCB0_MSMDMA_CH0_WRITE_QOS	MSMDMA Channel 0 Write Quality of Service Register
SCB0_MSMDMA_CH1_READ_QOS	MSMDMA Channel 1 Read Quality of Service Register
SCB0_MSMDMA_CH1_WRITE_QOS	MSMDMA Channel 1 Write Quality of Service Register
SCB0_SH0_DPORT_READ_QOS	SH0 DPORT Read Quality of Service Register
SCB0_SH0_DPORT_WRITE_QOS	SH0 DPORT Write Quality of Service Register
SCB0_SH0_IPORT_READ_QOS	SH0 IPORT Read Quality of Service Register
SCB0_SH0_IPORT_WRITE_QOS	SH0 IPORT Write Quality of Service Register
SCB0_SH0_MMR_IB_READ_QOS	SH0 MMR Read Quality of Service Register
SCB0_SH0_MMR_IB_WRITE_QOS	SH0 MMR Write Quality of Service Register
SCB0_SP0A_READ_QOS	SP0A Read Quality of Service Register
SCB0_SP0A_WRITE_QOS	SP0A Write Quality of Service Register
SCB0_SP0B_READ_QOS	SP0B Read Quality of Service Register
SCB0_SP0B_WRITE_QOS	SP0B Write Quality of Service Register
SCB0_SP1A_READ_QOS	SP1A Read Quality of Service Register
SCB0_SP1A_WRITE_QOS	SP1A Write Quality of Service Register
SCB0_SP1B_READ_QOS	SP1B Read Quality of Service Register
SCB0_SP1B_WRITE_QOS	SP1B Write Quality of Service Register
SCB0_SP2A_READ_QOS	SP2A Read Quality of Service Register
SCB0_SP2A_WRITE_QOS	SP2A Write Quality of Service Register

Table 37-10: ADSP-21560_21561_21564_21568 SCB0 Register List (Continued)

Name	Description
SCB0_SP2B_READ_QOS	SP2B Read Quality of Service Register
SCB0_SP2B_WRITE_QOS	SP2B Write Quality of Service Register
SCB0_SP3A_READ_QOS	SP3A Read Quality of Service Register
SCB0_SP3A_WRITE_QOS	SP3A Write Quality of Service Register
SCB0_SP3B_READ_QOS	SP3B Read Quality of Service Register
SCB0_SP3B_WRITE_QOS	SP3B Write Quality of Service Register
SCB0_SP4A_READ_QOS	SP4A Read Quality of Service Register
SCB0_SP4A_WRITE_QOS	SP4A Write Quality of Service Register
SCB0_SP4B_READ_QOS	SP4B Read Quality of Service Register
SCB0_SP4B_WRITE_QOS	SP4B Write Quality of Service Register
SCB0_SP5A_READ_QOS	SP5A Read Quality of Service Register
SCB0_SP5A_WRITE_QOS	SP5A Write Quality of Service Register
SCB0_SP5B_READ_QOS	SP5B Read Quality of Service Register
SCB0_SP5B_WRITE_QOS	SP5B Write Quality of Service Register
SCB0_SP6A_READ_QOS	SP6A Read Quality of Service Register
SCB0_SP6A_WRITE_QOS	SP6A Write Quality of Service Register
SCB0_SP6B_READ_QOS	SP6B Read Quality of Service Register
SCB0_SP6B_WRITE_QOS	SP6B Write Quality of Service Register
SCB0_SP7A_READ_QOS	SP7A Read Quality of Service Register
SCB0_SP7A_WRITE_QOS	SP7A Write Quality of Service Register
SCB0_SP7B_READ_QOS	SP7B Read Quality of Service Register
SCB0_SP7B_WRITE_QOS	SP7B Write Quality of Service Register
SCB0_SPI0RX_READ_QOS	SPI0 RX Read Quality of Service Register
SCB0_SPI0RX_WRITE_QOS	SPI0 RX Write Quality of Service Register
SCB0_SPI0TX_READ_QOS	SPI0 TX Read Quality of Service Register
SCB0_SPI0TX_WRITE_QOS	SPI0 TX Write Quality of Service Register
SCB0_SPI1RX_READ_QOS	SPI1 RX Read Quality of Service Register
SCB0_SPI1RX_WRITE_QOS	SPI1 RX Write Quality of Service Register
SCB0_SPI1TX_READ_QOS	SPI1 TX Read Quality of Service Register
SCB0_SPI1TX_WRITE_QOS	SPI1 TX Write Quality of Service Register
SCB0_SPI2RX_READ_QOS	SPI2 RX Read Quality of Service Register

Table 37-10: ADSP-21560_21561_21564_21568 SCB0 Register List (Continued)

Name	Description
SCB0_SPI2RX_WRITE_QOS	SPI2 RX Write Quality of Service Register
SCB0_SPI2TX_READ_QOS	SPI2 TX Read Quality of Service Register
SCB0_SPI2TX_WRITE_QOS	SPI2 TX Write Quality of Service Register
SCB0_UART0_RX_READ_QOS	UART0 RX Read Quality of Service Register
SCB0_UART0_RX_WRITE_QOS	UART0 RX Write Quality of Service Register
SCB0_UART0_TX_READ_QOS	UART0 TX Read Quality of Service Register
SCB0_UART0_TX_WRITE_QOS	UART0 TX Write Quality of Service Register
SCB0_UART1_RX_READ_QOS	UART1 TX Read Quality of Service Register
SCB0_UART1_RX_WRITE_QOS	UART1 RX Write Quality of Service Register
SCB0_UART1_TX_READ_QOS	UART1 TX Read Quality of Service Register
SCB0_UART1_TX_WRITE_QOS	UART1 TX Write Quality of Service Register
SCB0_UART2_RX_READ_QOS	UART2 RX Read Quality of Service Register
SCB0_UART2_RX_WRITE_QOS	UART2 RX Write Quality of Service Register
SCB0_UART2_TX_READ_QOS	UART2 TX Read Quality of Service Register
SCB0_UART2_TX_WRITE_QOS	UART2 TX Write Quality of Service Register

Accelerator Channel 0 Read Quality of Service Register

The `SCB0_ACC_CH0_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

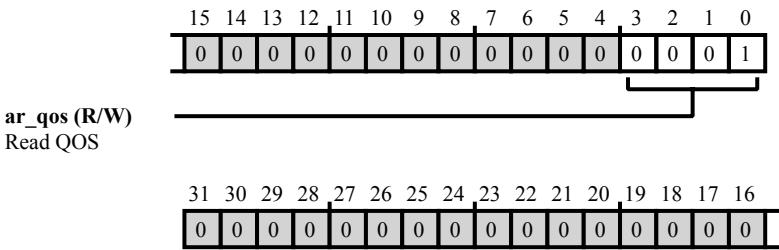


Figure 37-8: SCB0_ACC_CH0_READ_QOS Register Diagram

Table 37-11: SCB0_ACC_CH0_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

Accelerator Channel 0 Write Quality of Service Register

The `SCB0_ACC_CH0_WRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

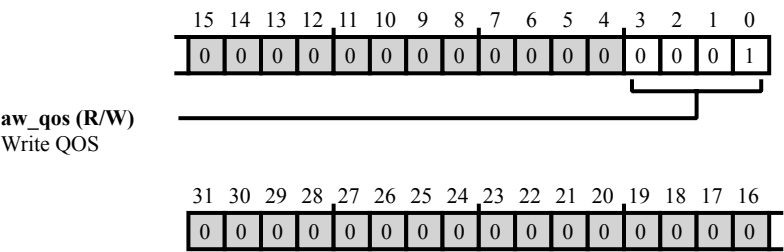


Figure 37-9: SCB0_ACC_CH0_WRITE_QOS Register Diagram

Table 37-12: SCB0_ACC_CH0_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

Accelerator Channel 1 Read Quality of Service Register

The `SCB0_ACC_CH1_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

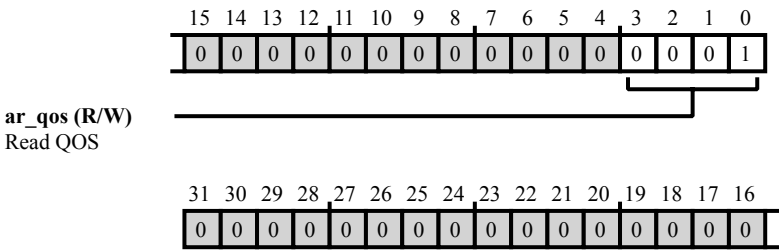


Figure 37-10: SCB0_ACC_CH1_READ_QOS Register Diagram

Table 37-13: SCB0_ACC_CH1_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

Accelerator Channel 1 Write Quality of Service Register

The `SCB0_ACC_CH1_WRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

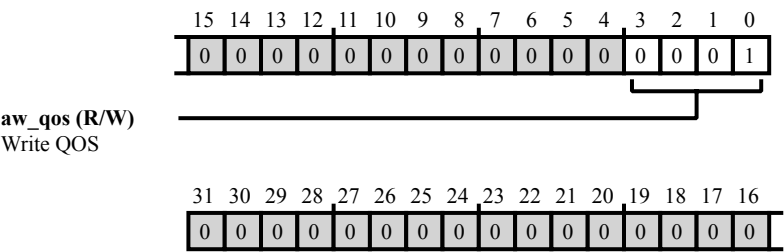


Figure 37-11: SCB0_ACC_CH1_WRITE_QOS Register Diagram

Table 37-14: SCB0_ACC_CH1_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

CRC0 Channel 0 Read Quality of Service Register

The `SCB0_CRC0_CH0_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

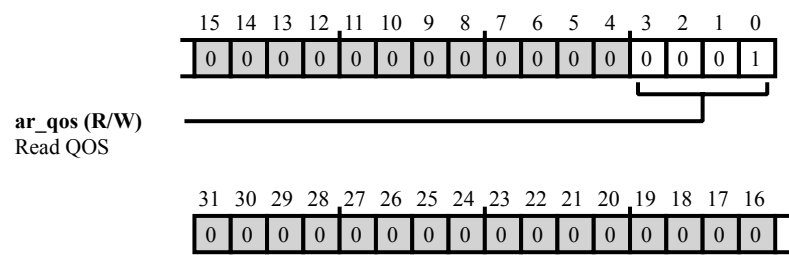


Figure 37-12: SCB0_CRC0_CH0_READ_QOS Register Diagram

Table 37-15: SCB0_CRC0_CH0_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

CRC0 Channel 0 Write Quality of Service Register

The `SCB0_CRC0_CH0_WRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

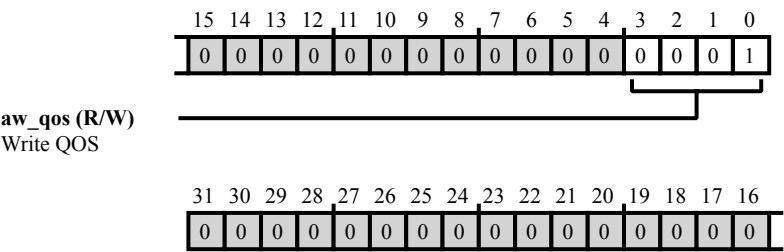


Figure 37-13: SCB0_CRC0_CH0_WRITE_QOS Register Diagram

Table 37-16: SCB0_CRC0_CH0_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

CRC0 Channel 1 Read Quality of Service Register

The `SCB0_CRC0_CH1_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

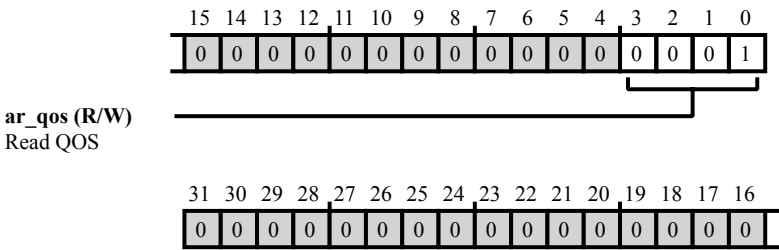


Figure 37-14: SCB0_CRC0_CH1_READ_QOS Register Diagram

Table 37-17: SCB0_CRC0_CH1_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

CRC0 Channel 1 Write Quality of Service Register

The `SCB0_CRC0_CH1_WRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

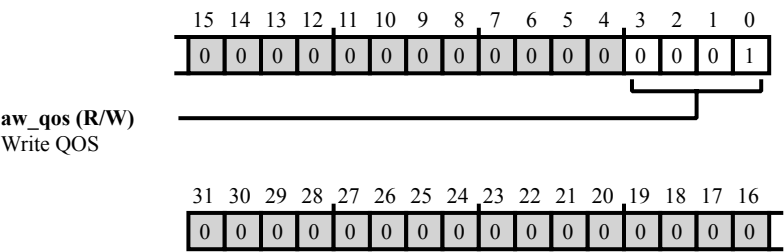


Figure 37-15: SCB0_CRC0_CH1_WRITE_QOS Register Diagram

Table 37-18: SCB0_CRC0_CH1_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

CRC1 Channel 0 Read Quality of Service Register

The `SCB0_CRC1_CH0_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

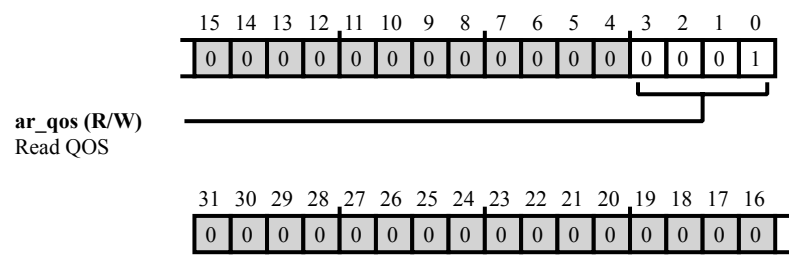


Figure 37-16: SCB0_CRC1_CH0_READ_QOS Register Diagram

Table 37-19: SCB0_CRC1_CH0_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

CRC1 Channel 0 Write Quality of Service Register

The `SCB0_CRC1_CH0_WRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

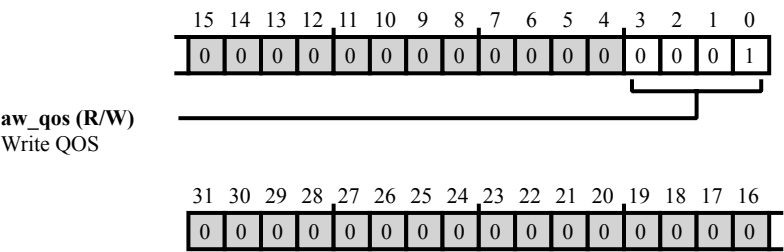


Figure 37-17: SCB0_CRC1_CH0_WRITE_QOS Register Diagram

Table 37-20: SCB0_CRC1_CH0_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

CRC1 Channel 1 Read Quality of Service Register

The `SCB0_CRC1_CH1_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

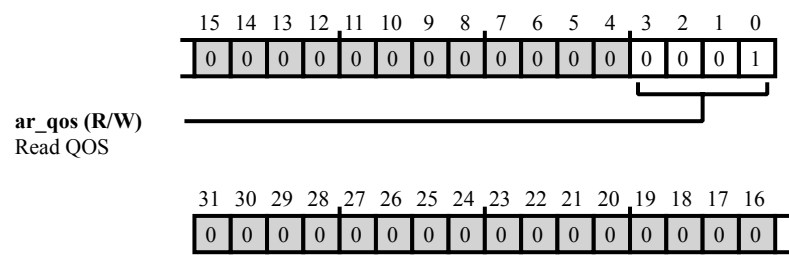


Figure 37-18: SCB0_CRC1_CH1_READ_QOS Register Diagram

Table 37-21: SCB0_CRC1_CH1_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

CRC1 Channel 1 Write Quality of Service Register

The `SCB0_CRC1_CH1_WRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

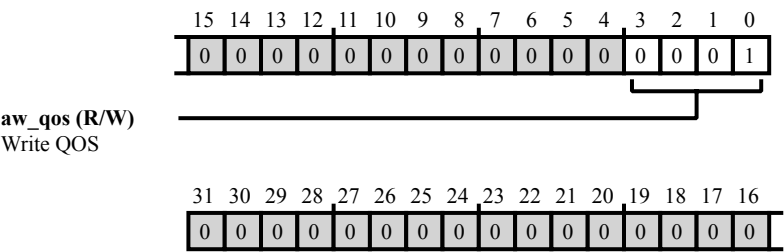


Figure 37-19: SCB0_CRC1_CH1_WRITE_QOS Register Diagram

Table 37-22: SCB0_CRC1_CH1_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

Crypto Read Quality of Service Register

The `SCB0_CRYPTO_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

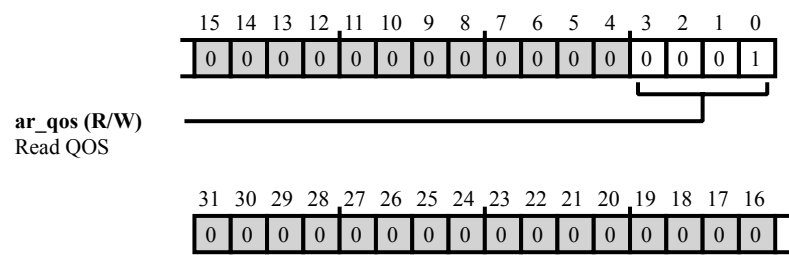


Figure 37-20: SCB0_CRYPTO_READ_QOS Register Diagram

Table 37-23: SCB0_CRYPTO_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

CRYPTO Write Quality of Service Register

The `SCB0_CRYPTOWRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

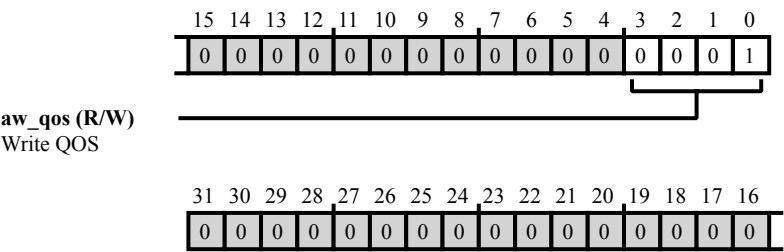


Figure 37-21: SCB0_CRYPTOWRITE_QOS Register Diagram

Table 37-24: SCB0_CRYPTOWRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

DBG Read Quality of Service Register

The `SCB0_DBG_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

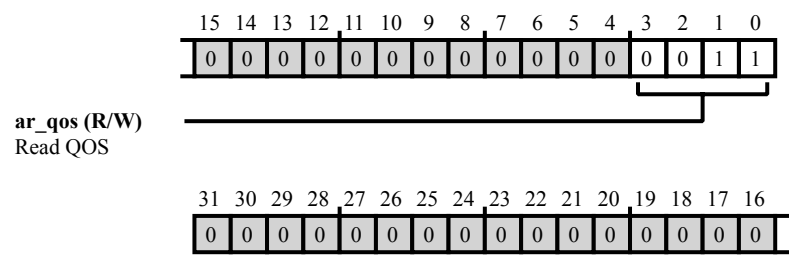


Figure 37-22: SCB0_DBG_READ_QOS Register Diagram

Table 37-25: SCB0_DBG_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

DBG Write Quality of Service Register

The `SCB0_DBG_WRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

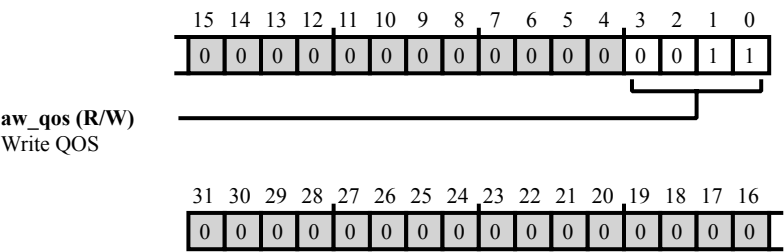


Figure 37-23: SCB0_DBG_WRITE_QOS Register Diagram

Table 37-26: SCB0_DBG_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

DLDMA0 Channel 0 Read Quality of Service Register

The `SCB0_DLDMA0_CH0_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

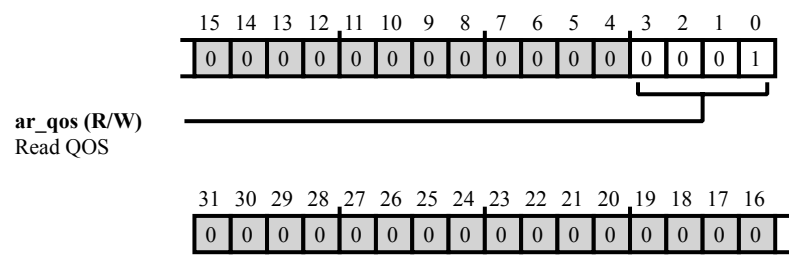


Figure 37-24: SCB0_DLDMA0_CH0_READ_QOS Register Diagram

Table 37-27: SCB0_DLDMA0_CH0_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

DLDMA0 Channel 0 Write Quality of Service Register

The `SCB0_DLDMA0_CH0_WRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

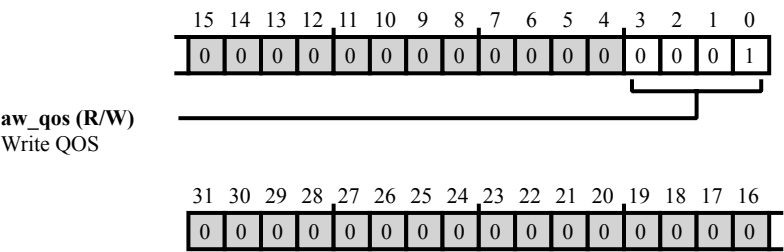


Figure 37-25: SCB0_DLDMA0_CH0_WRITE_QOS Register Diagram

Table 37-28: SCB0_DLDMA0_CH0_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

DLDMA0 Channel 1 Read Quality of Service Register

The `SCB0_DLDMA0_CH1_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

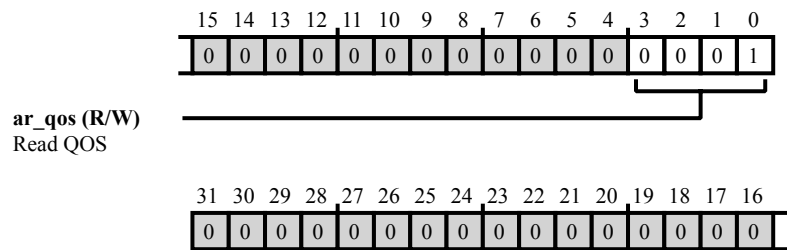


Figure 37-26: SCB0_DLDMA0_CH1_READ_QOS Register Diagram

Table 37-29: SCB0_DLDMA0_CH1_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

DLDMA0 Channel 1 Write Quality of Service Register

The `SCB0_DLDMA0_CH1_WRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

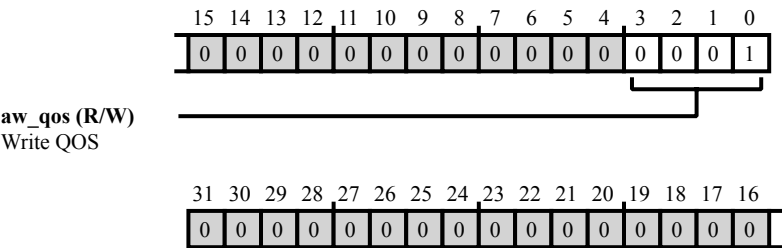


Figure 37-27: SCB0_DLDMA0_CH1_WRITE_QOS Register Diagram

Table 37-30: SCB0_DLDMA0_CH1_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

DLDMA1 Channel 0 Read Quality of Service Register

The `SCB0_DLDMA1_CH0_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

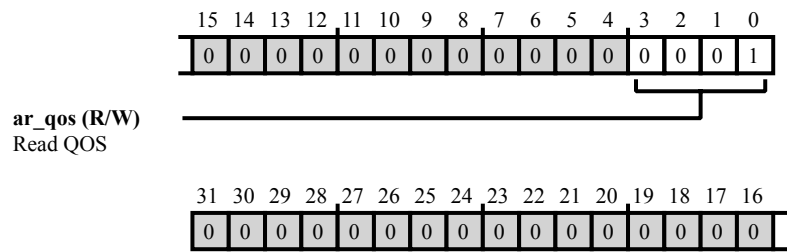


Figure 37-28: SCB0_DLDMA1_CH0_READ_QOS Register Diagram

Table 37-31: SCB0_DLDMA1_CH0_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

DLDMA1 Channel 0 Write Quality of Service Register

The `SCB0_DLDMA1_CH0_WRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

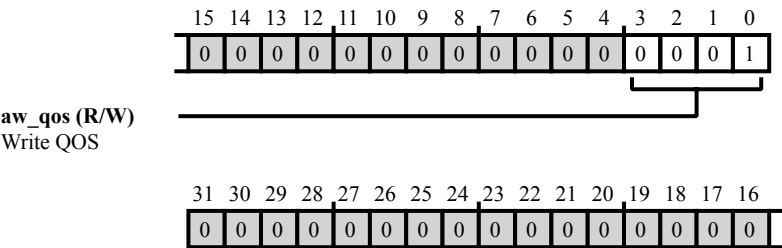


Figure 37-29: SCB0_DLDMA1_CH0_WRITE_QOS Register Diagram

Table 37-32: SCB0_DLDMA1_CH0_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

DLDMA1 Channel 1 Read Quality of Service Register

The `SCB0_DLDMA1_CH1_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

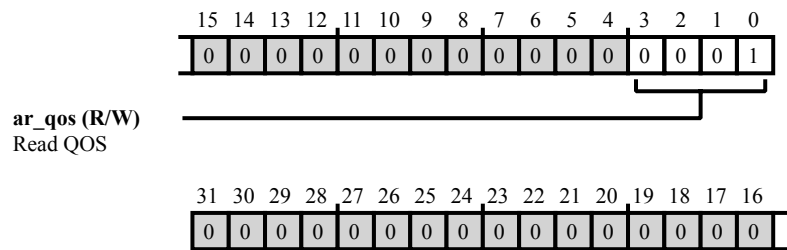


Figure 37-30: SCB0_DLDMA1_CH1_READ_QOS Register Diagram

Table 37-33: SCB0_DLDMA1_CH1_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

DLDMA1 Channel 1 Write Quality of Service Register

The `SCB0_DLDMA1_CH1_WRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

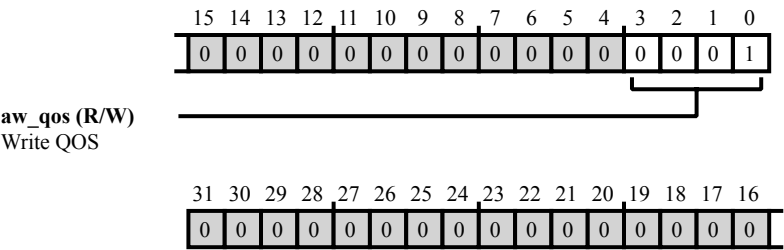


Figure 37-31: SCB0_DLDMA1_CH1_WRITE_QOS Register Diagram

Table 37-34: SCB0_DLDMA1_CH1_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

ETR Read Quality of Service Register

The `SCB0_ETR_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

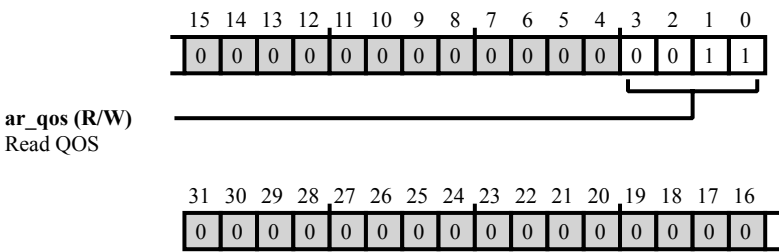


Figure 37-32: SCB0_ETR_READ_QOS Register Diagram

Table 37-35: SCB0_ETR_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

ETR Write Quality of Service Register

The `SCB0_ETR_WRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

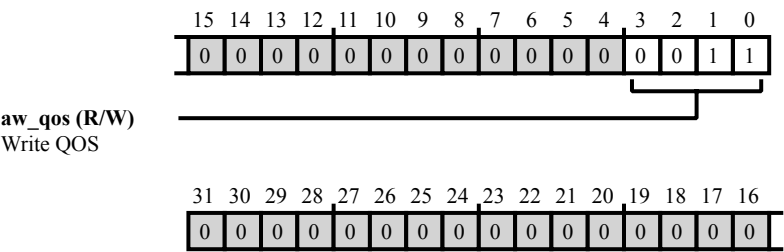


Figure 37-33: SCB0_ETR_WRITE_QOS Register Diagram

Table 37-36: SCB0_ETR_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

HSMDMA Channel 0 Read Quality of Service Register

The `SCB0_HSMDMA_CH0_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

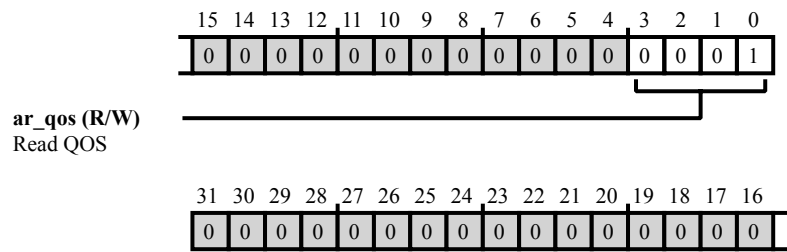


Figure 37-34: SCB0_HSMDMA_CH0_READ_QOS Register Diagram

Table 37-37: SCB0_HSMDMA_CH0_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

HSMDMA Channel 0 Write Quality of Service Register

The `SCB0_HSMDMA_CH0_WRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

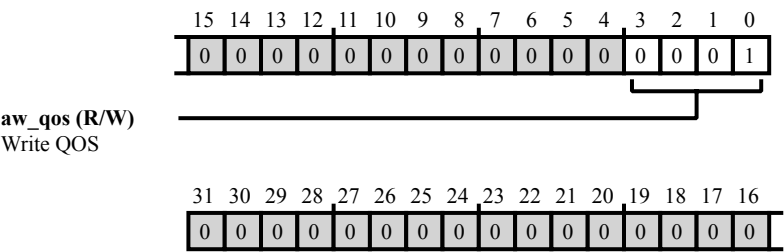


Figure 37-35: SCB0_HSMDMA_CH0_WRITE_QOS Register Diagram

Table 37-38: SCB0_HSMDMA_CH0_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

HSMDMA Channel 1 Read Quality of Service Register

The `SCB0_HSMDMA_CH1_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

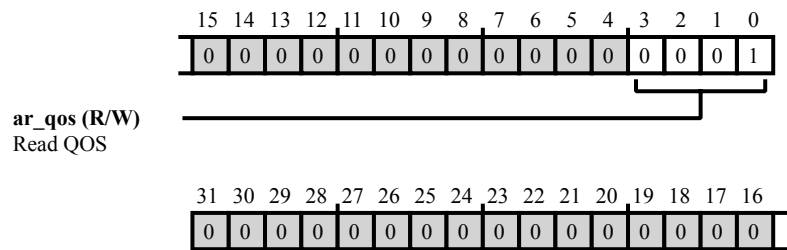


Figure 37-36: SCB0_HSMDMA_CH1_READ_QOS Register Diagram

Table 37-39: SCB0_HSMDMA_CH1_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

HSMDMA Channel 1 Write Quality of Service Register

The `SCB0_HSMDMA_CH1_WRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

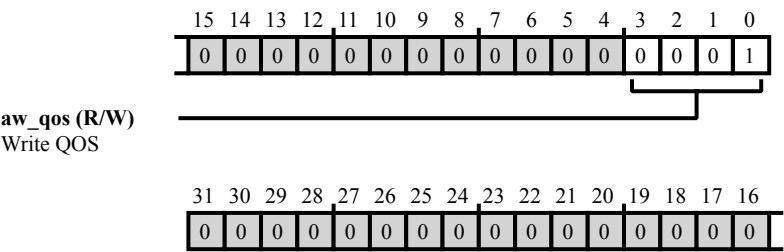


Figure 37-37: SCB0_HSMDMA_CH1_WRITE_QOS Register Diagram

Table 37-40: SCB0_HSMDMA_CH1_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

LP0 Read Quality of Service Register

The `SCB0_LP0_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

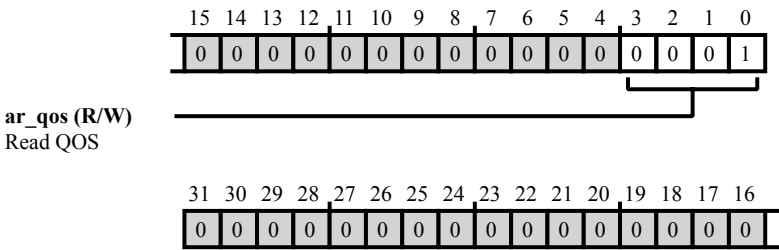


Figure 37-38: SCB0_LP0_READ_QOS Register Diagram

Table 37-41: SCB0_LP0_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

LP0 Write Quality of Service Register

The `SCB0_LP0_WRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

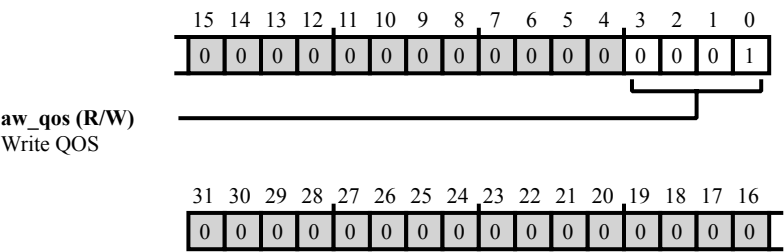


Figure 37-39: SCB0_LP0_WRITE_QOS Register Diagram

Table 37-42: SCB0_LP0_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

LP1 Read Quality of Service Register

The `SCB0_LP1_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

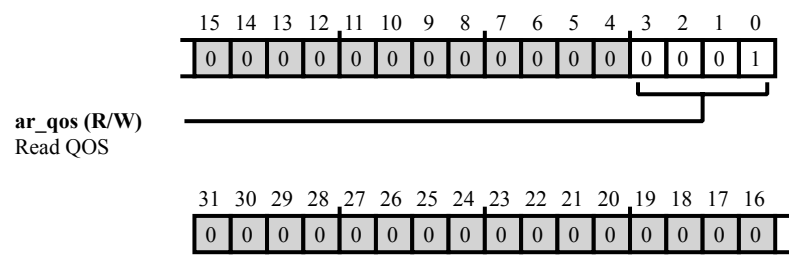


Figure 37-40: SCB0_LP1_READ_QOS Register Diagram

Table 37-43: SCB0_LP1_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

LP1 Write Quality of Service Register

The `SCB0_LP1_WRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

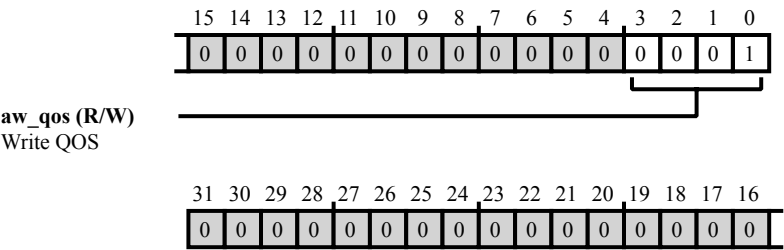


Figure 37-41: SCB0_LP1_WRITE_QOS Register Diagram

Table 37-44: SCB0_LP1_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

MLB Read Quality of Service Register

The `SCB0_MLB_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

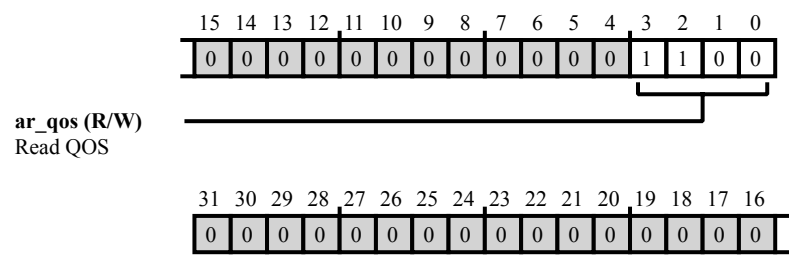


Figure 37-42: SCB0_MLB_READ_QOS Register Diagram

Table 37-45: SCB0_MLB_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

MLB Write Quality of Service Register

The `SCB0_MLB_WRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

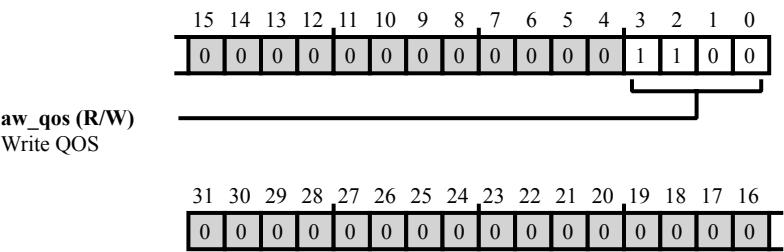


Figure 37-43: SCB0_MLB_WRITE_QOS Register Diagram

Table 37-46: SCB0_MLB_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

MSMDMA Channel 0 Read Quality of Service Register

The `SCB0_MSMDMA_CH0_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

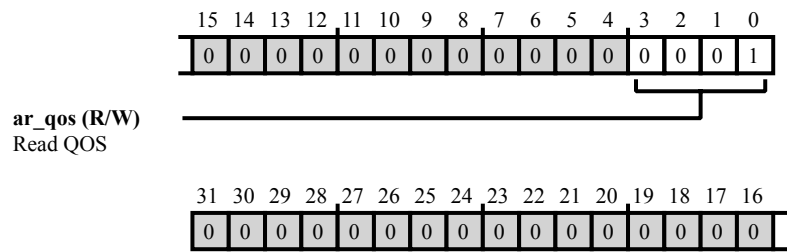


Figure 37-44: SCB0_MSMDMA_CH0_READ_QOS Register Diagram

Table 37-47: SCB0_MSMDMA_CH0_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

MSMDMA Channel 0 Write Quality of Service Register

The `SCB0_MSMDMA_CH0_WRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

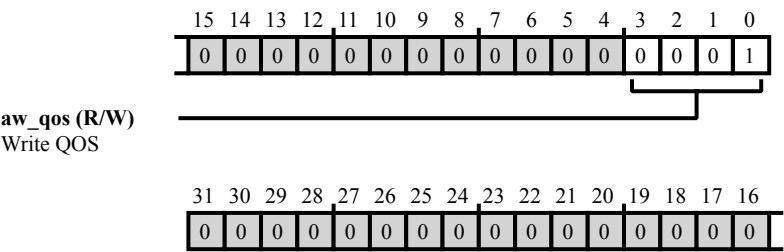


Figure 37-45: SCB0_MSMDMA_CH0_WRITE_QOS Register Diagram

Table 37-48: SCB0_MSMDMA_CH0_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

MSMDMA Channel 1 Read Quality of Service Register

The `SCB0_MSMDMA_CH1_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

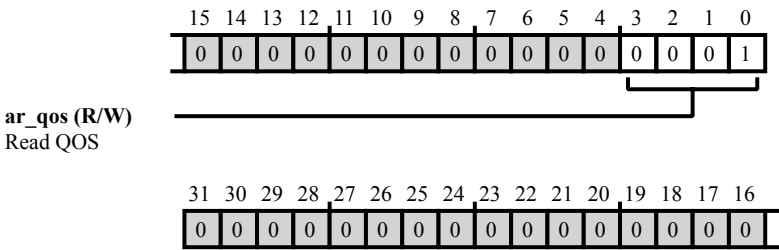


Figure 37-46: SCB0_MSMDMA_CH1_READ_QOS Register Diagram

Table 37-49: SCB0_MSMDMA_CH1_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

MSMDMA Channel 1 Write Quality of Service Register

The `SCB0_MSMDMA_CH1_WRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

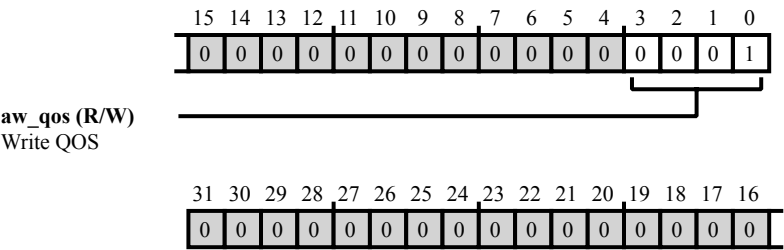


Figure 37-47: SCB0_MSMDMA_CH1_WRITE_QOS Register Diagram

Table 37-50: SCB0_MSMDMA_CH1_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

SH0 DPORT Read Quality of Service Register

The `SCB0_SH0_DPORT_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

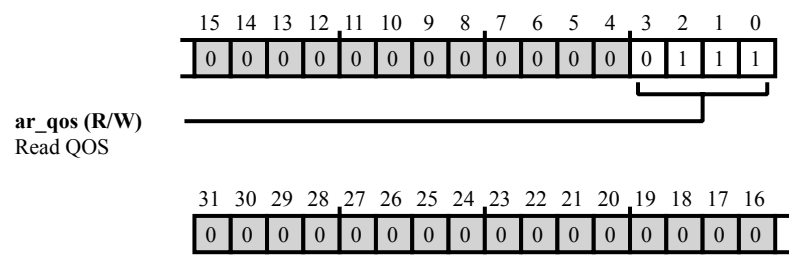


Figure 37-48: SCB0_SH0_DPORT_READ_QOS Register Diagram

Table 37-51: SCB0_SH0_DPORT_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

SH0 DPORT Write Quality of Service Register

The `SCB0_SH0_DPORT_WRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

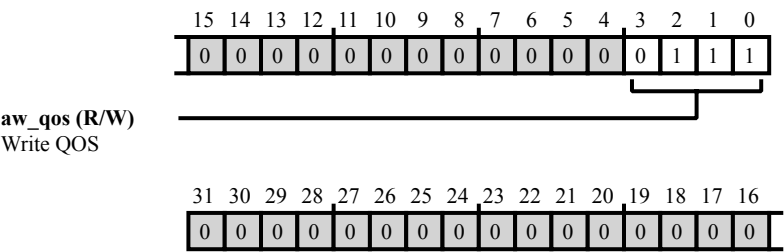


Figure 37-49: SCB0_SH0_DPORT_WRITE_QOS Register Diagram

Table 37-52: SCB0_SH0_DPORT_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

SH0 IPORT Read Quality of Service Register

The `SCB0_SH0_IPORT_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

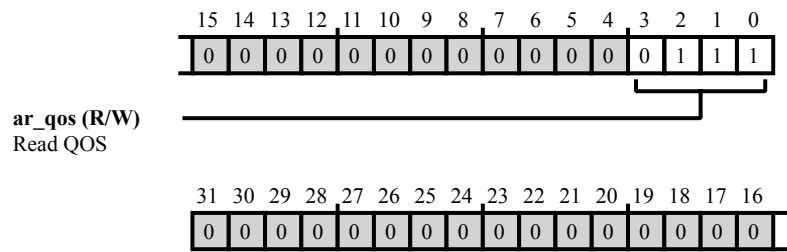


Figure 37-50: SCB0_SH0_IPORT_READ_QOS Register Diagram

Table 37-53: SCB0_SH0_IPORT_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

SH0 IPORT Write Quality of Service Register

The `SCB0_SH0_IPORT_WRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

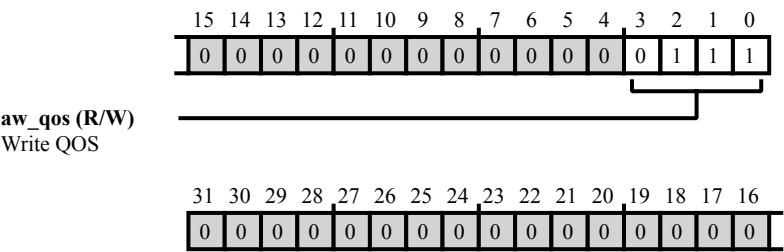


Figure 37-51: SCB0_SH0_IPORT_WRITE_QOS Register Diagram

Table 37-54: SCB0_SH0_IPORT_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

SH0 MMR Read Quality of Service Register

The `SCB0_SH0_MMR_IB_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

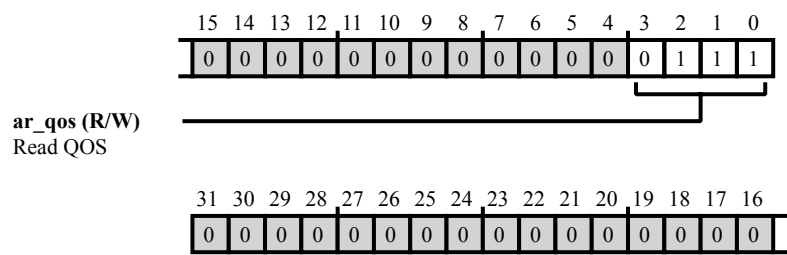


Figure 37-52: SCB0_SH0_MMR_IB_READ_QOS Register Diagram

Table 37-55: SCB0_SH0_MMR_IB_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

SH0 MMR Write Quality of Service Register

The `SCB0_SH0_MMR_IB_WRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

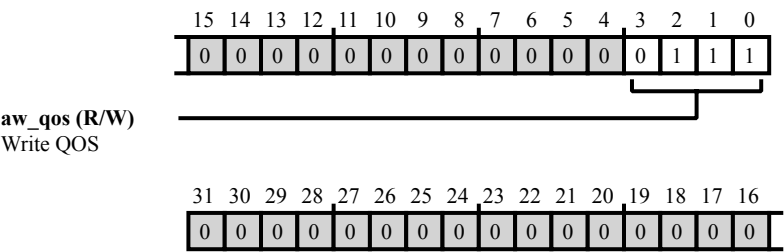


Figure 37-53: SCB0_SH0_MMR_IB_WRITE_QOS Register Diagram

Table 37-56: SCB0_SH0_MMR_IB_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

SP0A Read Quality of Service Register

The `SCB0_SP0A_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

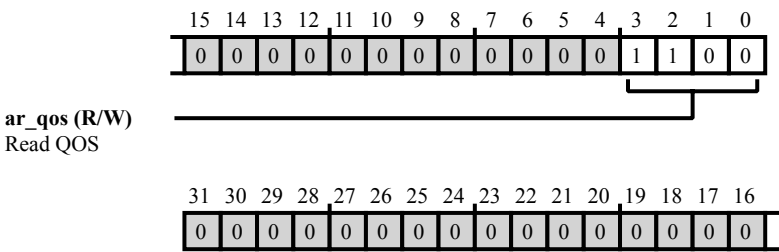


Figure 37-54: SCB0_SP0A_READ_QOS Register Diagram

Table 37-57: SCB0_SP0A_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

SP0A Write Quality of Service Register

The `SCB0_SP0A_WRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

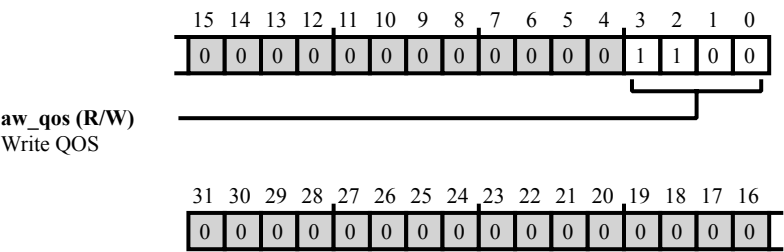


Figure 37-55: SCB0_SP0A_WRITE_QOS Register Diagram

Table 37-58: SCB0_SP0A_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

SPOB Read Quality of Service Register

The `SCB0_SP0B_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

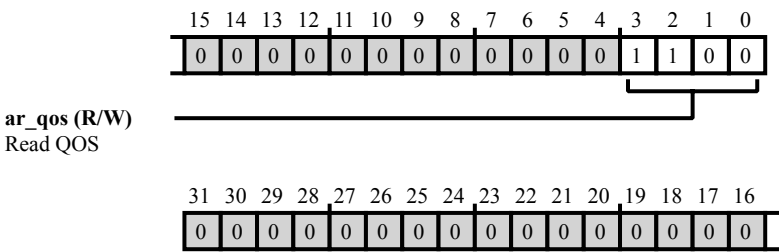


Figure 37-56: SCB0_SP0B_READ_QOS Register Diagram

Table 37-59: SCB0_SP0B_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

SPOB Write Quality of Service Register

The `SCB0_SP0B_WRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

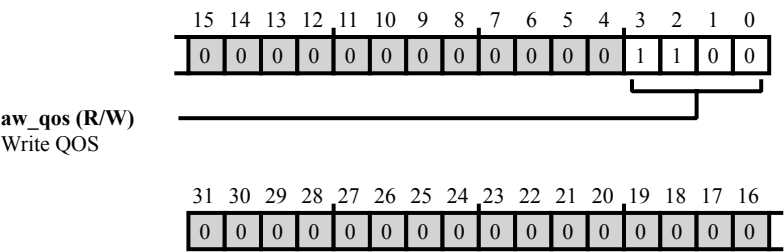


Figure 37-57: SCB0_SP0B_WRITE_QOS Register Diagram

Table 37-60: SCB0_SP0B_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

SP1A Read Quality of Service Register

The `SCB0_SP1A_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

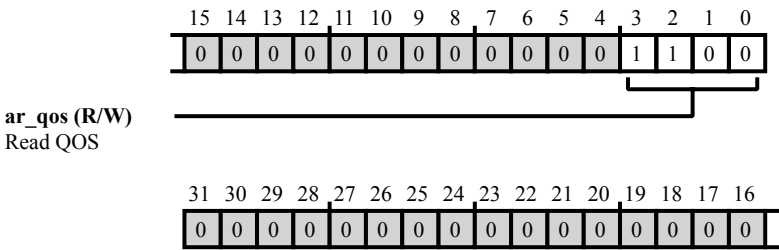


Figure 37-58: SCB0_SP1A_READ_QOS Register Diagram

Table 37-61: SCB0_SP1A_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

SP1A Write Quality of Service Register

The `SCB0_SP1A_WRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

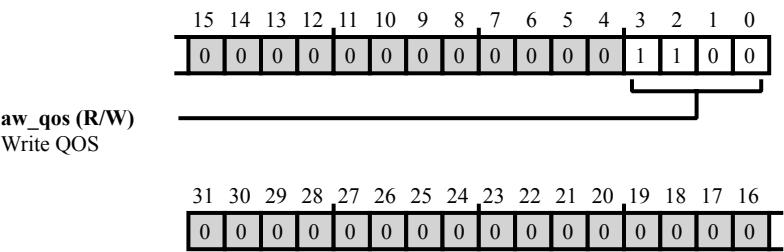


Figure 37-59: SCB0_SP1A_WRITE_QOS Register Diagram

Table 37-62: SCB0_SP1A_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

SP1B Read Quality of Service Register

The `SCB0_SP1B_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

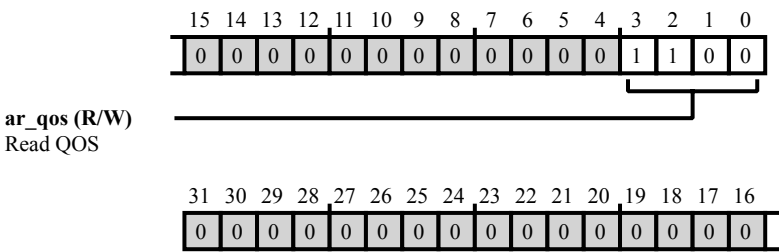


Figure 37-60: SCB0_SP1B_READ_QOS Register Diagram

Table 37-63: SCB0_SP1B_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

SP1B Write Quality of Service Register

The `SCB0_SP1B_WRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

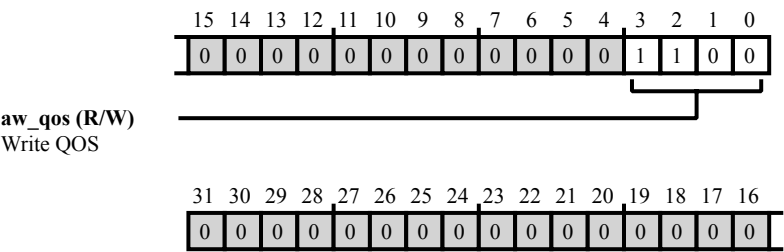


Figure 37-61: SCB0_SP1B_WRITE_QOS Register Diagram

Table 37-64: SCB0_SP1B_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

SP2A Read Quality of Service Register

The `SCB0_SP2A_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

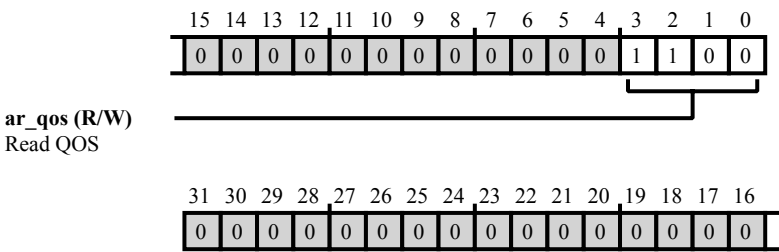


Figure 37-62: SCB0_SP2A_READ_QOS Register Diagram

Table 37-65: SCB0_SP2A_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

SP2A Write Quality of Service Register

The `SCB0_SP2A_WRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

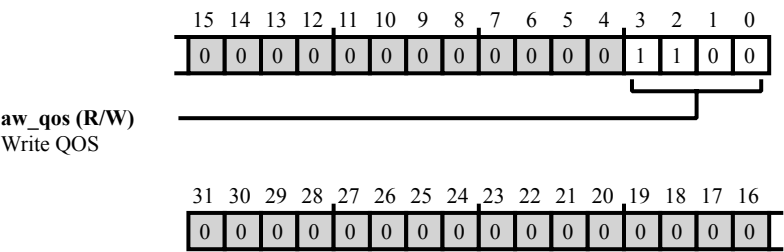


Figure 37-63: SCB0_SP2A_WRITE_QOS Register Diagram

Table 37-66: SCB0_SP2A_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

SP2B Read Quality of Service Register

The `SCB0_SP2B_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

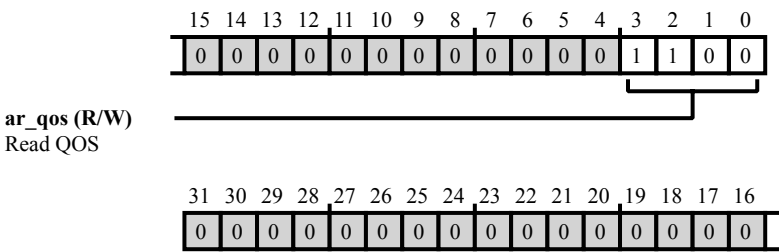


Figure 37-64: SCB0_SP2B_READ_QOS Register Diagram

Table 37-67: SCB0_SP2B_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

SP2B Write Quality of Service Register

The `SCB0_SP2B_WRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

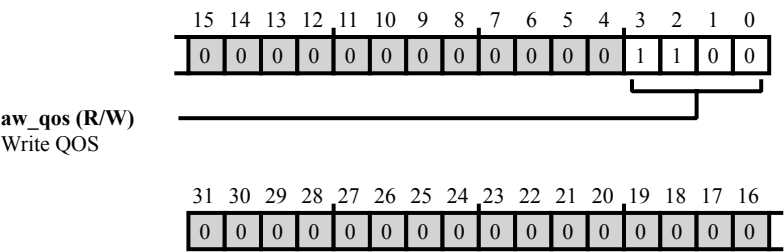


Figure 37-65: SCB0_SP2B_WRITE_QOS Register Diagram

Table 37-68: SCB0_SP2B_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

SP3A Read Quality of Service Register

The `SCB0_SP3A_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

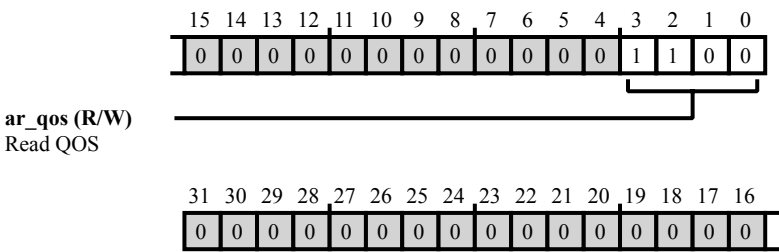


Figure 37-66: SCB0_SP3A_READ_QOS Register Diagram

Table 37-69: SCB0_SP3A_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

SP3A Write Quality of Service Register

The `SCB0_SP3A_WRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

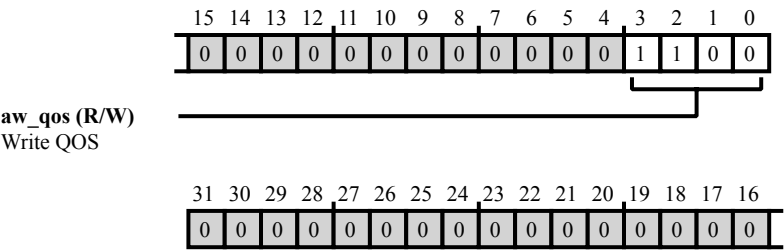


Figure 37-67: SCB0_SP3A_WRITE_QOS Register Diagram

Table 37-70: SCB0_SP3A_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

SP3B Read Quality of Service Register

The `SCB0_SP3B_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

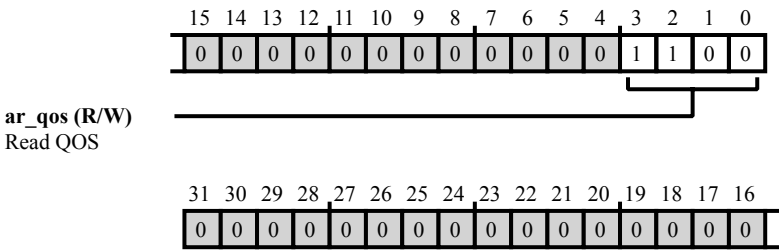


Figure 37-68: SCB0_SP3B_READ_QOS Register Diagram

Table 37-71: SCB0_SP3B_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

SP3B Write Quality of Service Register

The `SCB0_SP3B_WRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

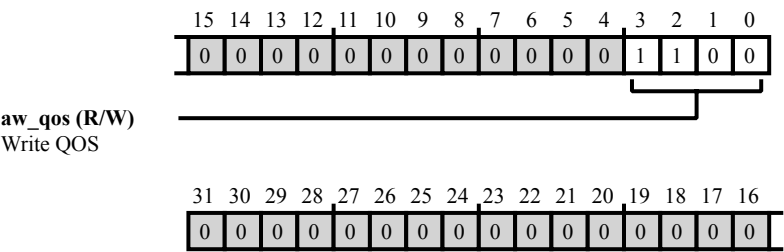


Figure 37-69: SCB0_SP3B_WRITE_QOS Register Diagram

Table 37-72: SCB0_SP3B_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

SP4A Read Quality of Service Register

The `SCB0_SP4A_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

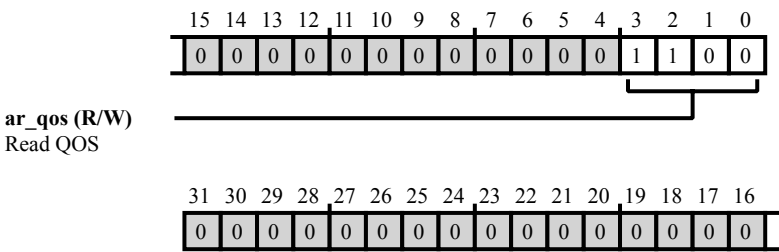


Figure 37-70: SCB0_SP4A_READ_QOS Register Diagram

Table 37-73: SCB0_SP4A_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

SP4A Write Quality of Service Register

The `SCB0_SP4A_WRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

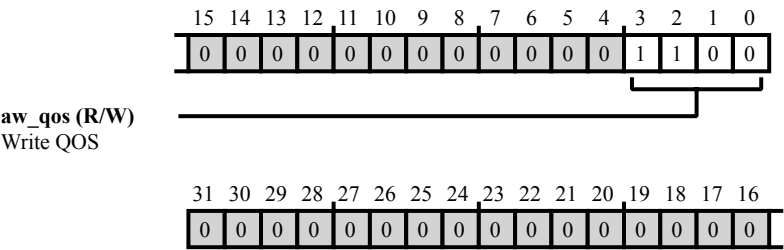


Figure 37-71: SCB0_SP4A_WRITE_QOS Register Diagram

Table 37-74: SCB0_SP4A_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

SP4B Read Quality of Service Register

The `SCB0_SP4B_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

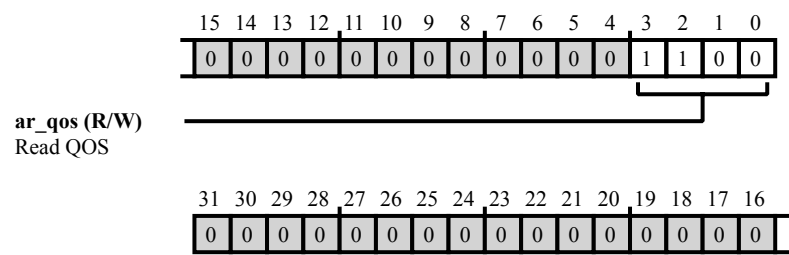


Figure 37-72: SCB0_SP4B_READ_QOS Register Diagram

Table 37-75: SCB0_SP4B_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

SP4B Write Quality of Service Register

The `SCB0_SP4B_WRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

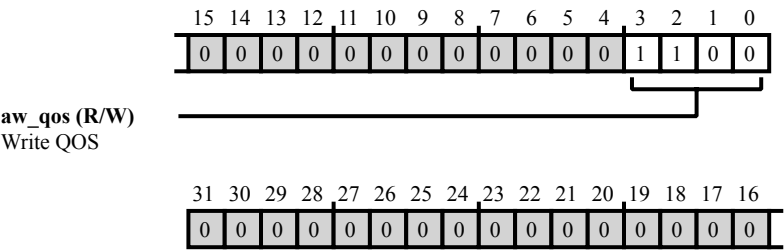


Figure 37-73: SCB0_SP4B_WRITE_QOS Register Diagram

Table 37-76: SCB0_SP4B_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

SP5A Read Quality of Service Register

The `SCB0_SP5A_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

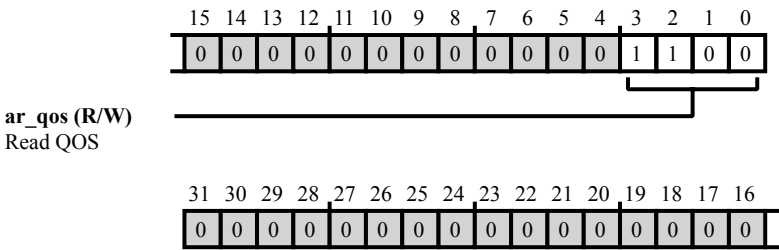


Figure 37-74: SCB0_SP5A_READ_QOS Register Diagram

Table 37-77: SCB0_SP5A_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

SP5A Write Quality of Service Register

The `SCB0_SP5A_WRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

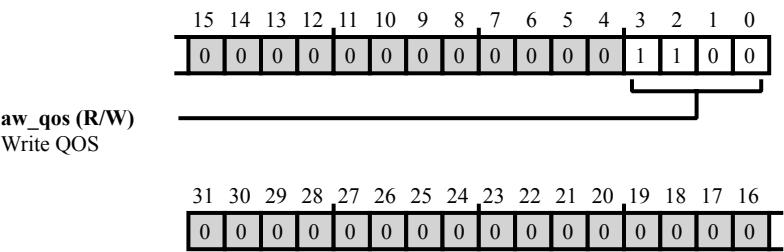


Figure 37-75: SCB0_SP5A_WRITE_QOS Register Diagram

Table 37-78: SCB0_SP5A_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

SP5B Read Quality of Service Register

The `SCB0_SP5B_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

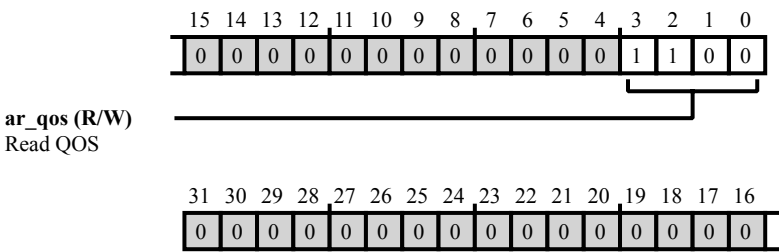


Figure 37-76: SCB0_SP5B_READ_QOS Register Diagram

Table 37-79: SCB0_SP5B_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

SP5B Write Quality of Service Register

The `SCB0_SP5B_WRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

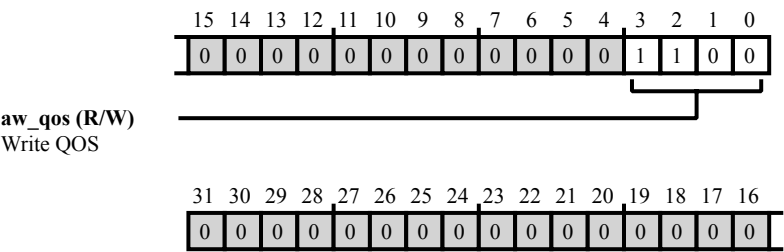


Figure 37-77: SCB0_SP5B_WRITE_QOS Register Diagram

Table 37-80: SCB0_SP5B_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

SP6A Read Quality of Service Register

The `SCB0_SP6A_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

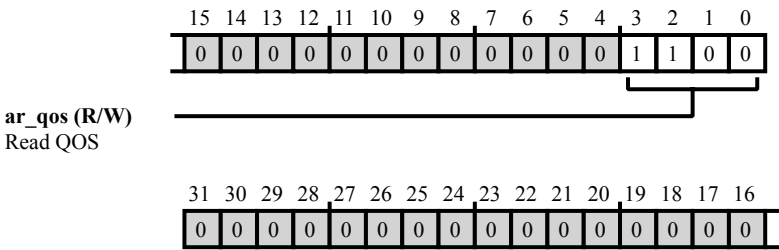


Figure 37-78: SCB0_SP6A_READ_QOS Register Diagram

Table 37-81: SCB0_SP6A_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

SP6A Write Quality of Service Register

The `SCB0_SP6A_WRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

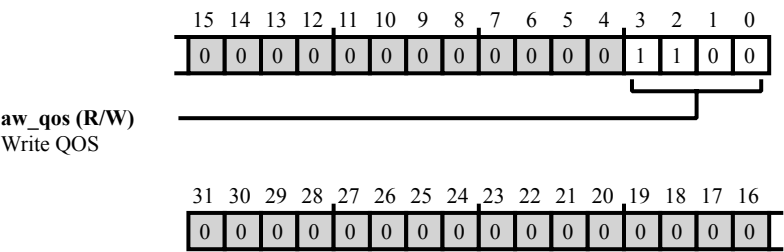


Figure 37-79: SCB0_SP6A_WRITE_QOS Register Diagram

Table 37-82: SCB0_SP6A_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

SP6B Read Quality of Service Register

The `SCB0_SP6B_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

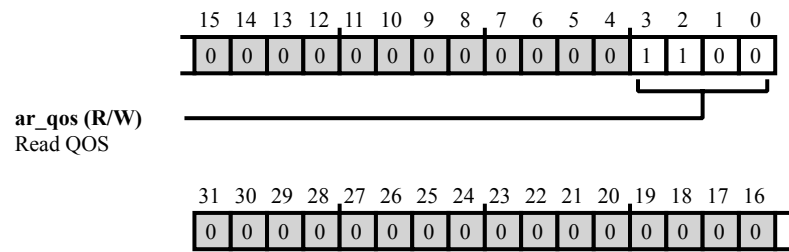


Figure 37-80: SCB0_SP6B_READ_QOS Register Diagram

Table 37-83: SCB0_SP6B_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

SP6B Write Quality of Service Register

The `SCB0_SP6B_WRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

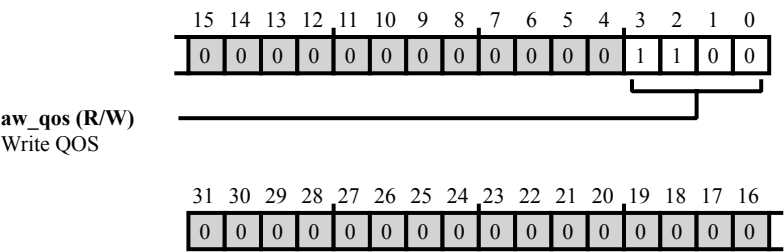


Figure 37-81: SCB0_SP6B_WRITE_QOS Register Diagram

Table 37-84: SCB0_SP6B_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

SP7A Read Quality of Service Register

The `SCB0_SP7A_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

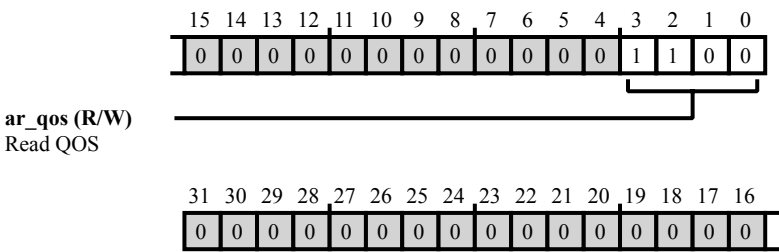


Figure 37-82: SCB0_SP7A_READ_QOS Register Diagram

Table 37-85: SCB0_SP7A_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

SP7A Write Quality of Service Register

The `SCB0_SP7A_WRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

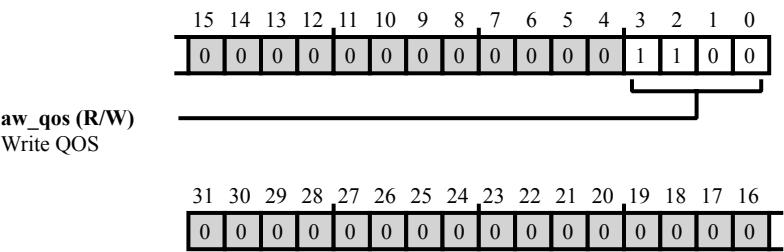


Figure 37-83: SCB0_SP7A_WRITE_QOS Register Diagram

Table 37-86: SCB0_SP7A_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

SP7B Read Quality of Service Register

The `SCB0_SP7B_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

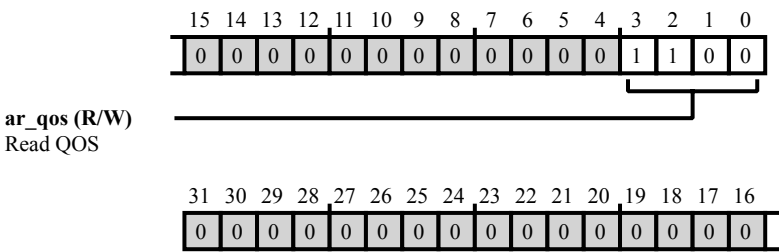


Figure 37-84: SCB0_SP7B_READ_QOS Register Diagram

Table 37-87: SCB0_SP7B_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

SP7B Write Quality of Service Register

The `SCB0_SP7B_WRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

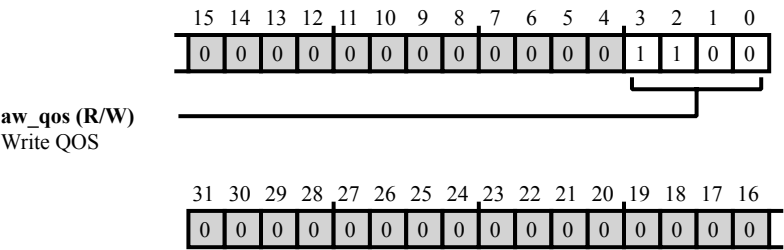


Figure 37-85: SCB0_SP7B_WRITE_QOS Register Diagram

Table 37-88: SCB0_SP7B_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

SPI0 RX Read Quality of Service Register

The `SCB0_SPI0RX_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

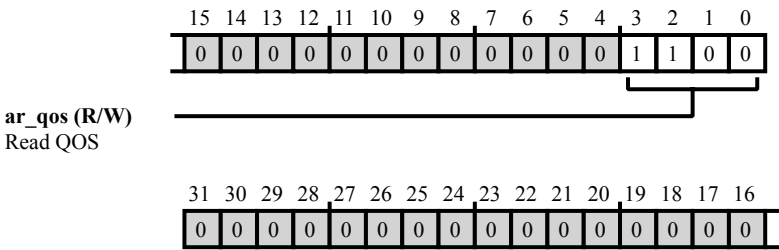


Figure 37-86: SCB0_SPI0RX_READ_QOS Register Diagram

Table 37-89: SCB0_SPI0RX_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

SPI0 RX Write Quality of Service Register

The `SCB0_SPI0RX_WRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

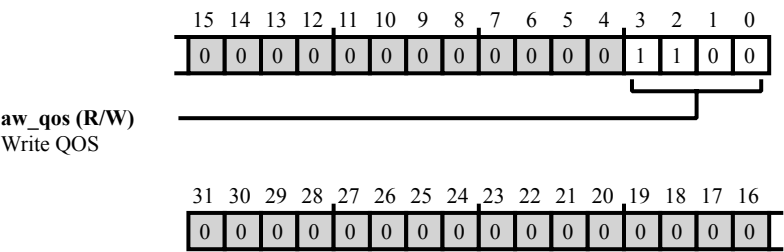


Figure 37-87: SCB0_SPI0RX_WRITE_QOS Register Diagram

Table 37-90: SCB0_SPI0RX_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

SPI0 TX Read Quality of Service Register

The `SCB0_SPI0TX_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

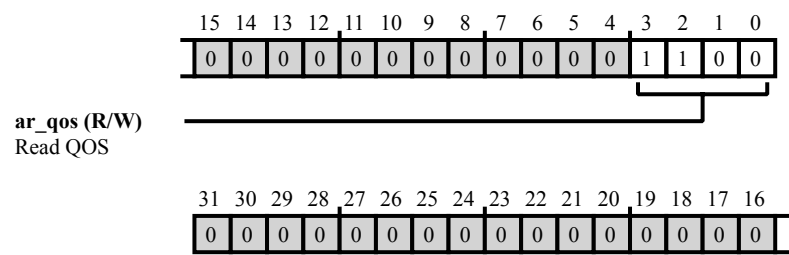


Figure 37-88: SCB0_SPI0TX_READ_QOS Register Diagram

Table 37-91: SCB0_SPI0TX_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

SPI0 TX Write Quality of Service Register

The `SCB0_SPI0TX_WRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

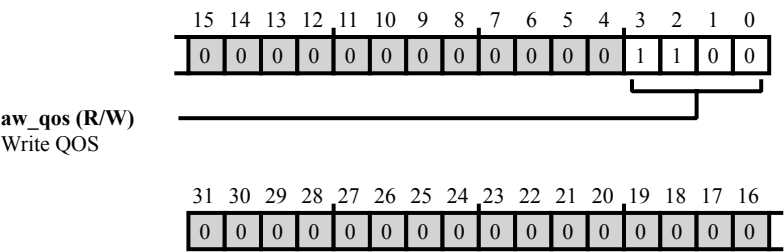


Figure 37-89: SCB0_SPI0TX_WRITE_QOS Register Diagram

Table 37-92: SCB0_SPI0TX_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

SPI1 RX Read Quality of Service Register

The `SCB0_SPI1RX_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

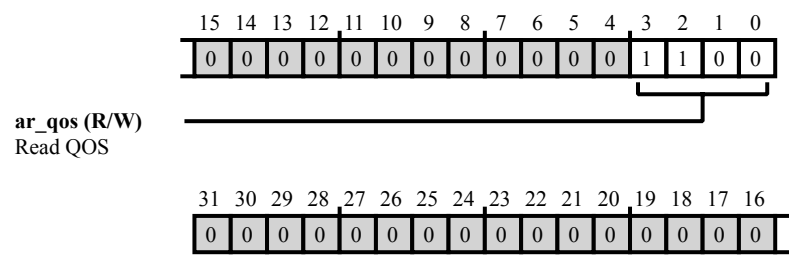


Figure 37-90: SCB0_SPI1RX_READ_QOS Register Diagram

Table 37-93: SCB0_SPI1RX_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

SPI1 RX Write Quality of Service Register

The `SCB0_SPI1RX_WRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

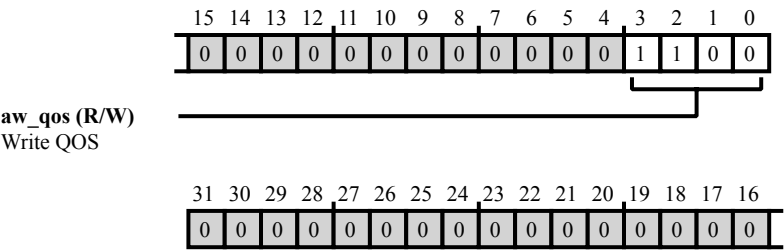


Figure 37-91: SCB0_SPI1RX_WRITE_QOS Register Diagram

Table 37-94: SCB0_SPI1RX_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

SPI1 TX Read Quality of Service Register

The `SCB0_SPI1TX_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

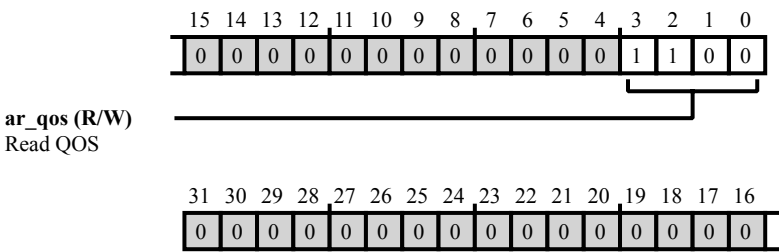


Figure 37-92: SCB0_SPI1TX_READ_QOS Register Diagram

Table 37-95: SCB0_SPI1TX_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

SPI1 TX Write Quality of Service Register

The `SCB0_SPI1TX_WRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

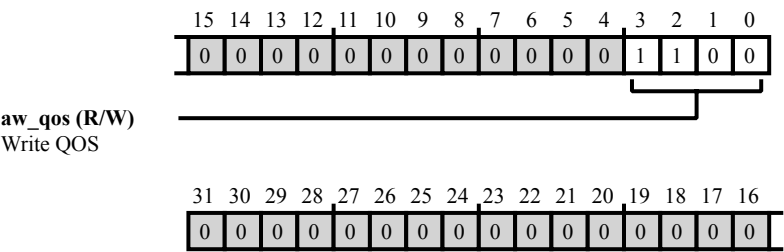


Figure 37-93: SCB0_SPI1TX_WRITE_QOS Register Diagram

Table 37-96: SCB0_SPI1TX_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

SPI2 RX Read Quality of Service Register

The `SCB0_SPI2RX_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

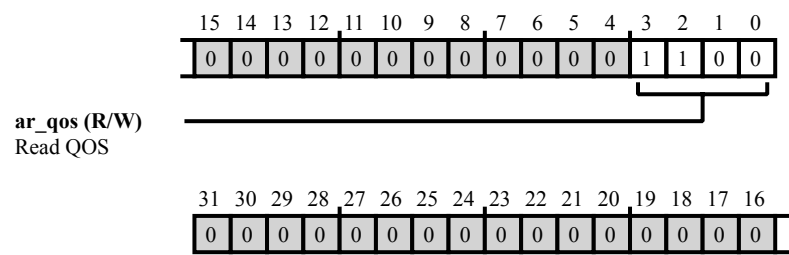


Figure 37-94: SCB0_SPI2RX_READ_QOS Register Diagram

Table 37-97: SCB0_SPI2RX_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

SPI2 RX Write Quality of Service Register

The `SCB0_SPI2RX_WRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

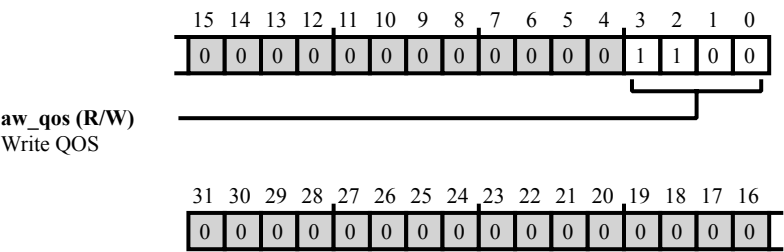


Figure 37-95: SCB0_SPI2RX_WRITE_QOS Register Diagram

Table 37-98: SCB0_SPI2RX_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

SPI2 TX Read Quality of Service Register

The `SCB0_SPI2TX_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

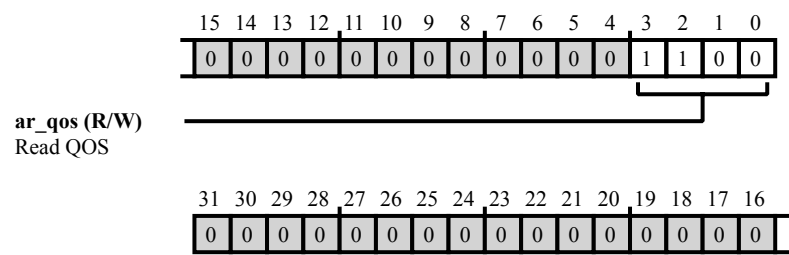


Figure 37-96: SCB0_SPI2TX_READ_QOS Register Diagram

Table 37-99: SCB0_SPI2TX_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

SPI2 TX Write Quality of Service Register

The `SCB0_SPI2TX_WRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

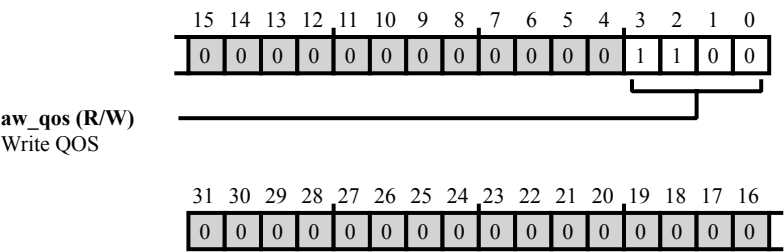


Figure 37-97: SCB0_SPI2TX_WRITE_QOS Register Diagram

Table 37-100: SCB0_SPI2TX_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

UART0 RX Read Quality of Service Register

The `SCB0_UART0_RX_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

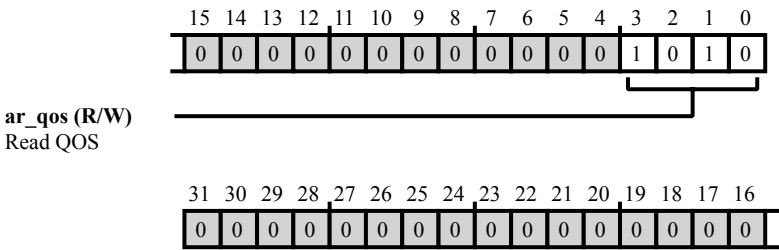


Figure 37-98: SCB0_UART0_RX_READ_QOS Register Diagram

Table 37-101: SCB0_UART0_RX_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

UART0 RX Write Quality of Service Register

The `SCB0_UART0_RX_WRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

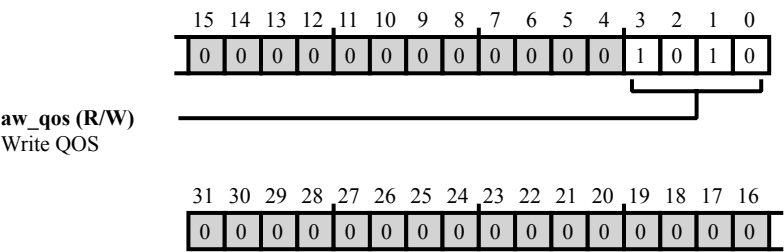


Figure 37-99: SCB0_UART0_RX_WRITE_QOS Register Diagram

Table 37-102: SCB0_UART0_RX_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

UART0 TX Read Quality of Service Register

The `SCB0_UART0_TX_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

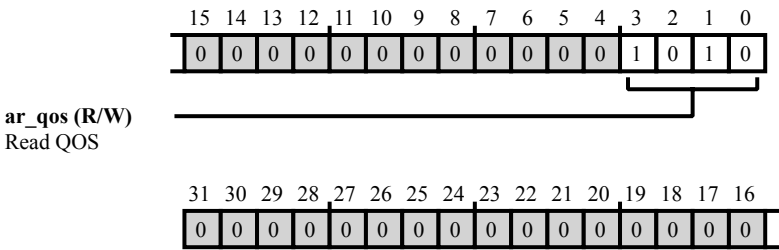


Figure 37-100: SCB0_UART0_TX_READ_QOS Register Diagram

Table 37-103: SCB0_UART0_TX_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

UART0 TX Write Quality of Service Register

The `SCB0_UART0_TX_WRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

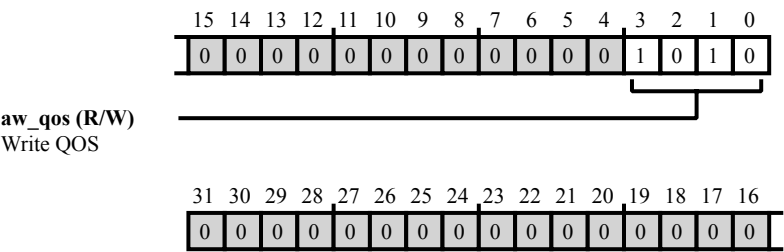


Figure 37-101: SCB0_UART0_TX_WRITE_QOS Register Diagram

Table 37-104: SCB0_UART0_TX_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

UART1 TX Read Quality of Service Register

The `SCB0_UART1_RX_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

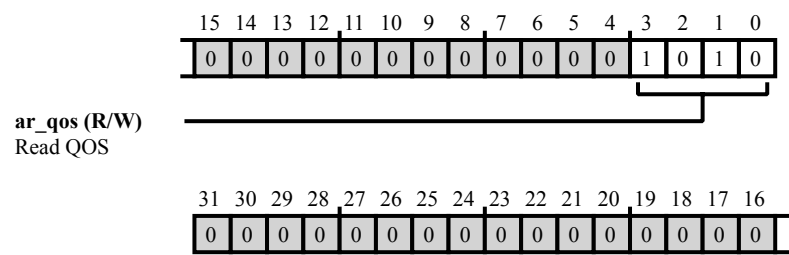


Figure 37-102: SCB0_UART1_RX_READ_QOS Register Diagram

Table 37-105: SCB0_UART1_RX_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

UART1 RX Write Quality of Service Register

The `SCB0_UART1_RX_WRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

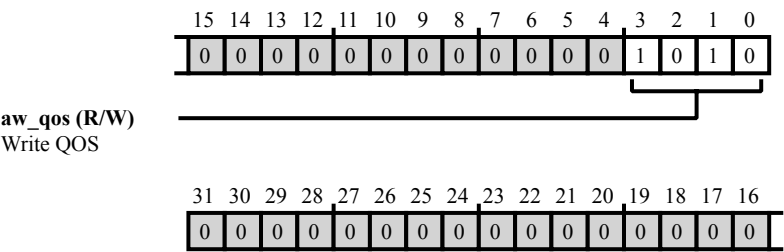


Figure 37-103: SCB0_UART1_RX_WRITE_QOS Register Diagram

Table 37-106: SCB0_UART1_RX_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

UART1 TX Read Quality of Service Register

The `SCB0_UART1_TX_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

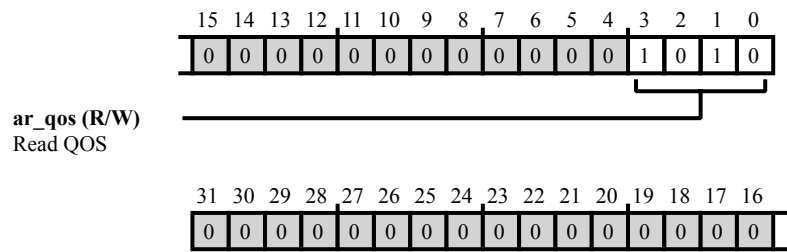


Figure 37-104: SCB0_UART1_TX_READ_QOS Register Diagram

Table 37-107: SCB0_UART1_TX_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

UART1 TX Write Quality of Service Register

The `SCB0_UART1_TX_WRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

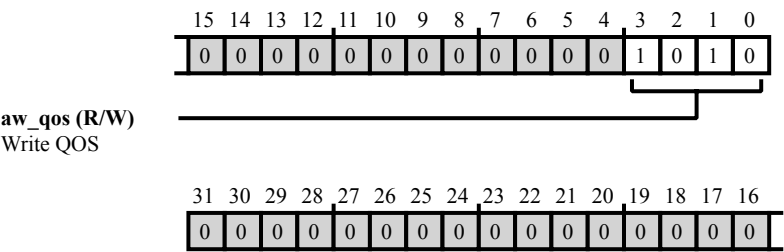


Figure 37-105: SCB0_UART1_TX_WRITE_QOS Register Diagram

Table 37-108: SCB0_UART1_TX_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

UART2 RX Read Quality of Service Register

The `SCB0_UART2_RX_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

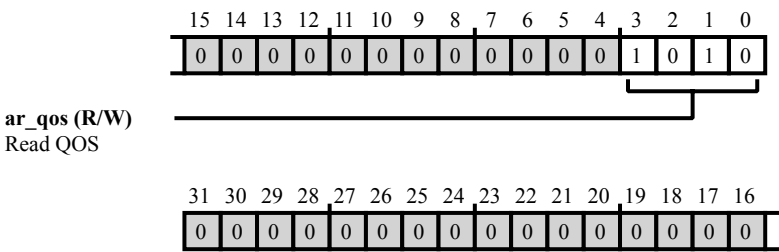


Figure 37-106: SCB0_UART2_RX_READ_QOS Register Diagram

Table 37-109: SCB0_UART2_RX_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

UART2 RX Write Quality of Service Register

The `SCB0_UART2_RX_WRITE_QOS` register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

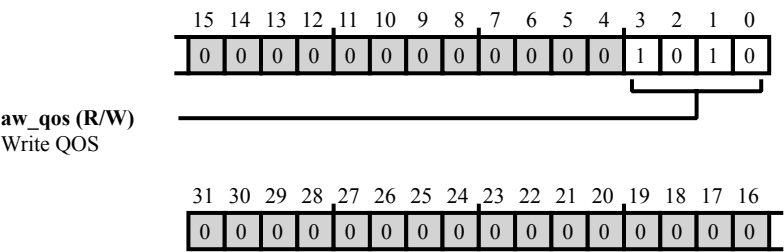


Figure 37-107: SCB0_UART2_RX_WRITE_QOS Register Diagram

Table 37-110: SCB0_UART2_RX_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

UART2 TX Read Quality of Service Register

The `SCB0_UART2_TX_READ_QOS` register indicates the read QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting read channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

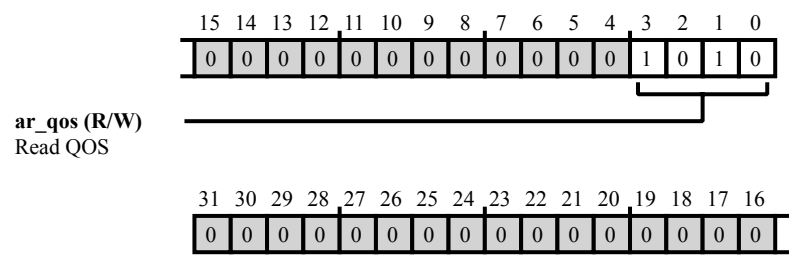


Figure 37-108: SCB0_UART2_TX_READ_QOS Register Diagram

Table 37-111: SCB0_UART2_TX_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Read QOS.

UART2 TX Write Quality of Service Register

The [SCB0_UART2_TX_WRITE_QOS](#) register indicates the write QOS or priority value for the indicated master. This value is used by the SCBs at different levels to arbitrate among the masters requesting write channel access. For the mapping of master IDs to peripherals, see the SCB Bus Master IDs table.

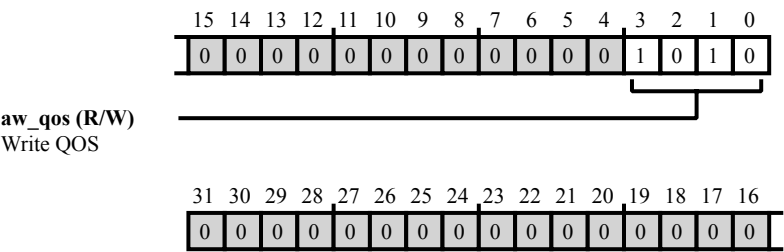


Figure 37-109: SCB0_UART2_TX_WRITE_QOS Register Diagram

Table 37-112: SCB0_UART2_TX_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Write QOS.

ADSP-21560_21561_21564_21568 SCB3 Register Descriptions

(SCB3) contains the following registers.

Table 37-113: ADSP-21560_21561_21564_21568 SCB3 Register List

Name	Description
SCB3_APB_DCLK_0_IB_SYNC_MODE	APB DCLK 0 Synchronization Mode Register

APB DCLK 0 Synchronization Mode Register

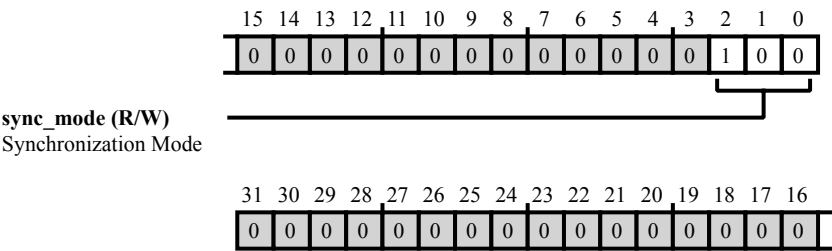


Figure 37-110: SCB3_APB_DCLK_0_IB_SYNC_MODE Register Diagram

Table 37-114: SCB3_APB_DCLK_0_IB_SYNC_MODE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
2:0 (R/W)	SYNC_MODE	Synchronization Mode. The SCB3_APB_DCLK_0_IB_SYNC_MODE bit field configures the SYSCLK-DCLK clock domain boundary (CDC) for the MMRG fabric.
		0 Sync 1:1
		1 Sync n:1
		2 Sync 1:n
		3 Sync m:n
		4 Async
		5 Reserved
		6 Reserved
		7 Reserved

ADSP-21560_21561_21564_21568 SCB4 Register Descriptions

(SCB4) contains the following registers.

Table 37-115: ADSP-21560_21561_21564_21568 SCB4 Register List

Name	Description
SCB4_COMP_ID_0	Comp Id 0
SCB4_COMP_ID_1	Comp Id 1
SCB4_COMP_ID_2	Comp Id 2
SCB4_COMP_ID_3	Comp Id 3
SCB4_FABRIC_S2PORT_IB_FN_MOD	Fabric S2port Ib.fn Mod

Table 37-115: ADSP-21560_21561_21564_21568 SCB4 Register List (Continued)

Name	Description
SCB4_FABRIC_S2PORT_IB_READ_QOS	Fabric S2port Ib.read Qos
SCB4_FABRIC_S2PORT_IB_WRITE_QOS	Fabric S2port Ib.write Qos
SCB4_FIR_CH0_IB_FN_MOD	Fir Ch0 Ib.fn Mod
SCB4_FIR_CH0_IB_FN_MOD2	Fir Ch0 Ib.fn Mod2
SCB4_FIR_CH0_IB_READ_QOS	Fir Ch0 Ib.read Qos
SCB4_FIR_CH0_IB_WRITE_QOS	Fir Ch0 Ib.write Qos
SCB4_FIR_CH1_IB_FN_MOD	Fir Ch1 Ib.fn Mod
SCB4_FIR_CH1_IB_FN_MOD2	Fir Ch1 Ib.fn Mod2
SCB4_FIR_CH1_IB_READ_QOS	Fir Ch1 Ib.read Qos
SCB4_FIR_CH1_IB_WRITE_QOS	Fir Ch1 Ib.write Qos
SCB4_IIR_CH0_IB_FN_MOD	Iir Ch0 Ib.fn Mod
SCB4_IIR_CH0_IB_FN_MOD2	Iir Ch0 Ib.fn Mod2
SCB4_IIR_CH0_IB_READ_QOS	Iir Ch0 Ib.read Qos
SCB4_IIR_CH0_IB_WRITE_QOS	Iir Ch0 Ib.write Qos
SCB4_IIR_CH1_IB_FN_MOD	Iir Ch1 Ib.fn Mod
SCB4_IIR_CH1_IB_FN_MOD2	Iir Ch1 Ib.fn Mod2
SCB4_IIR_CH1_IB_READ_QOS	Iir Ch1 Ib.read Qos
SCB4_IIR_CH1_IB_WRITE_QOS	Iir Ch1 Ib.write Qos
SCB4_PERIPH_ID_0	Periph Id 0
SCB4_PERIPH_ID_1	Periph Id 1
SCB4_PERIPH_ID_2	Periph Id 2
SCB4_PERIPH_ID_3	Periph Id 3
SCB4_PERIPH_ID_4	Periph Id 4
SCB4_PERIPH_ID_5	Periph Id 5
SCB4_PERIPH_ID_6	Periph Id 6
SCB4_PERIPH_ID_7	Periph Id 7
SCB4_SHARC_DPORT_FN_MOD	Sharc Dport.fn Mod
SCB4_SHARC_DPORT_READ_QOS	Sharc Dport.read Qos
SCB4_SHARC_DPORT_WRITE_QOS	Sharc Dport.write Qos

Comp Id 0

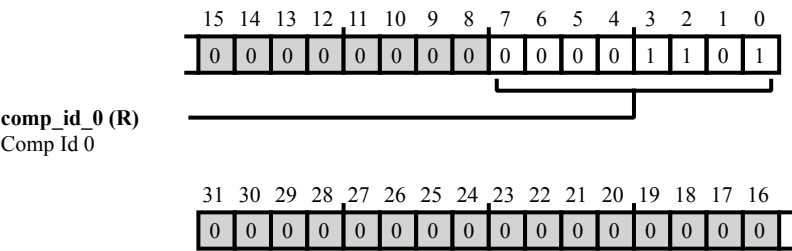


Figure 37-111: SCB4_COMP_ID_0 Register Diagram

Table 37-116: SCB4_COMP_ID_0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	COMP_ID_0	Comp Id 0.

Comp Id 1

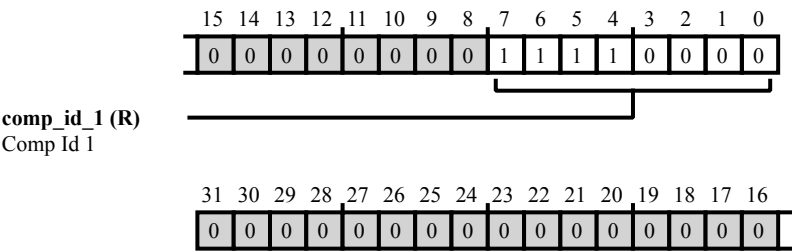


Figure 37-112: SCB4_COMP_ID_1 Register Diagram

Table 37-117: SCB4_COMP_ID_1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	COMP_ID_1	Comp Id 1.

Comp Id 2

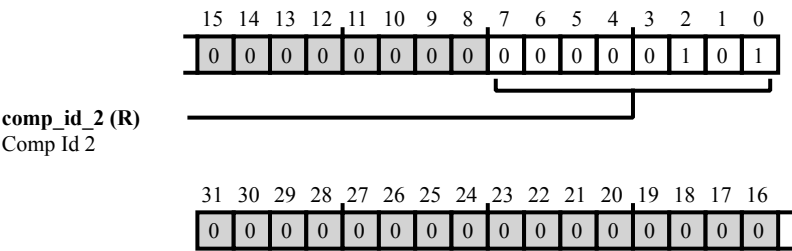


Figure 37-113: SCB4_COMP_ID_2 Register Diagram

Table 37-118: SCB4_COMP_ID_2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	COMP_ID_2	Comp Id 2.

Comp Id 3

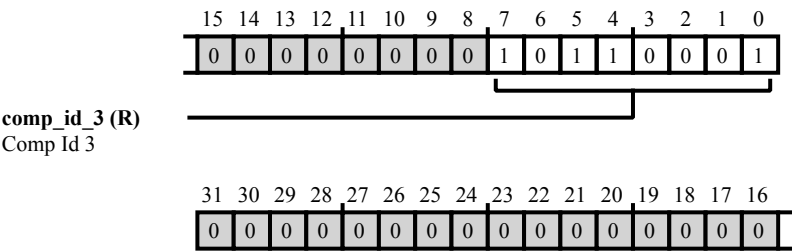


Figure 37-114: SCB4_COMP_ID_3 Register Diagram

Table 37-119: SCB4_COMP_ID_3 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	COMP_ID_3	Comp Id 3.

Fabric S2port Ib.fn Mod

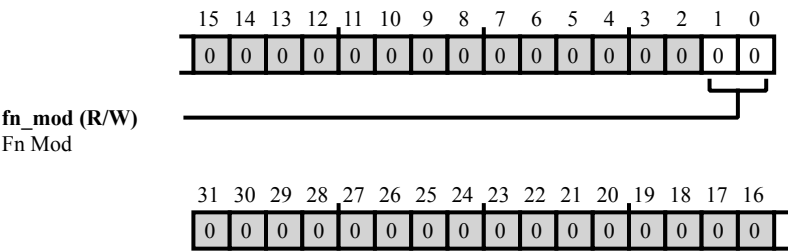


Figure 37-115: SCB4_FABRIC_S2PORT_IB_FN_MOD Register Diagram

Table 37-120: SCB4_FABRIC_S2PORT_IB_FN_MOD Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
1:0 (R/W)	FN_MOD	Fn Mod.

Fabric S2port Ib.read Qos

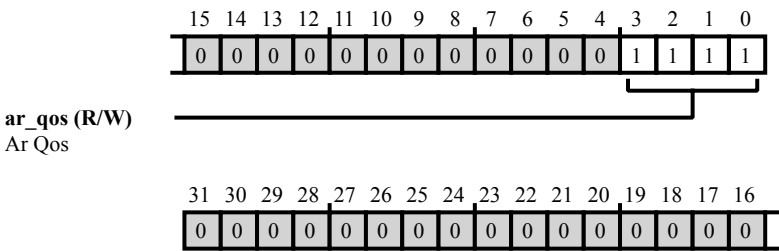


Figure 37-116: SCB4_FABRIC_S2PORT_IB_READ_QOS Register Diagram

Table 37-121: SCB4_FABRIC_S2PORT_IB_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

Fabric S2port Ib.write Qos

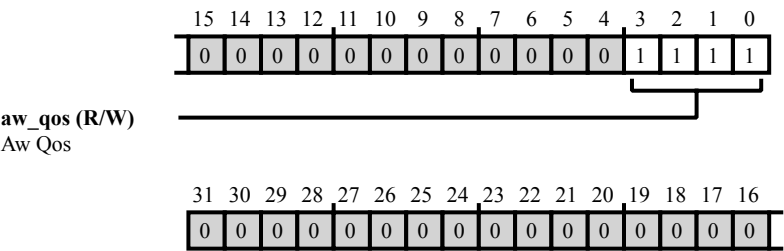


Figure 37-117: SCB4_FABRIC_S2PORT_IB_WRITE_QOS Register Diagram

Table 37-122: SCB4_FABRIC_S2PORT_IB_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

Fir Ch0 Ib.fn Mod

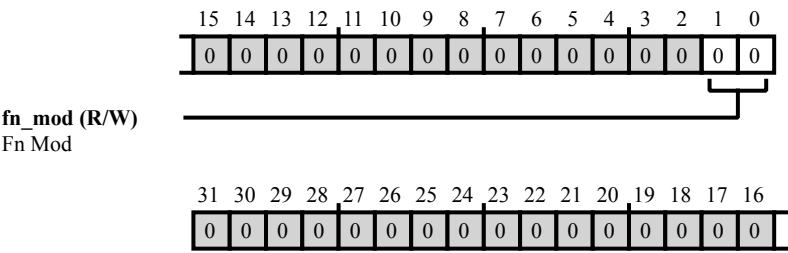


Figure 37-118: SCB4_FIR_CH0_IB_FN_MOD Register Diagram

Table 37-123: SCB4_FIR_CH0_IB_FN_MOD Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
1:0 (R/W)	FN_MOD	Fn Mod.

Fir Ch0 Ib.fn Mod2

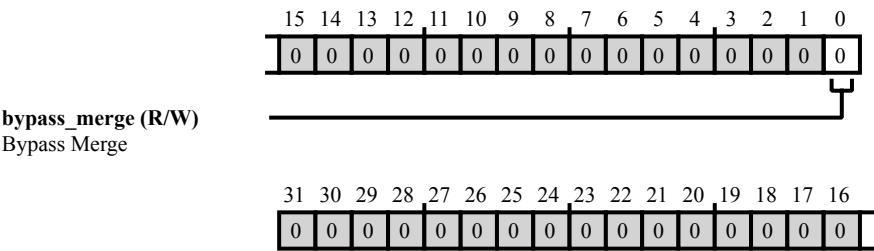


Figure 37-119: SCB4_FIR_CH0_IB_FN_MOD2 Register Diagram

Table 37-124: SCB4_FIR_CH0_IB_FN_MOD2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	BYPASS_MERGE	Bypass Merge.

Fir Ch0 Ib.read Qos

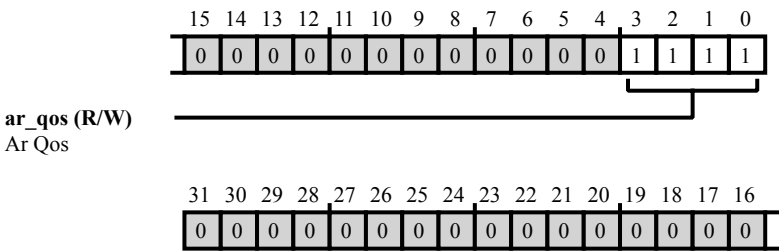


Figure 37-120: SCB4_FIR_CH0_IB_READ_QOS Register Diagram

Table 37-125: SCB4_FIR_CH0_IB_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

Fir Ch0 Ib.write Qos

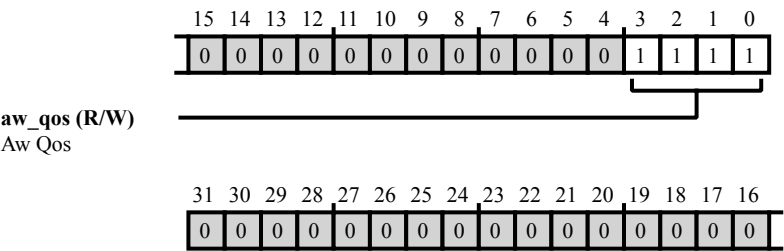


Figure 37-121: SCB4_FIR_CH0_IB_WRITE_QOS Register Diagram

Table 37-126: SCB4_FIR_CH0_IB_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

Fir Ch1 Ib.fn Mod

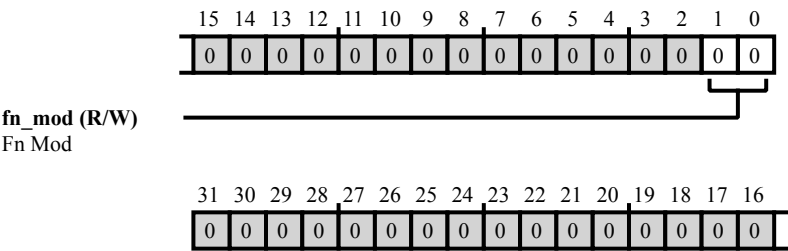


Figure 37-122: SCB4_FIR_CH1_IB_FN_MOD Register Diagram

Table 37-127: SCB4_FIR_CH1_IB_FN_MOD Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
1:0 (R/W)	FN_MOD	Fn Mod.

Fir Ch1 Ib.fn Mod2

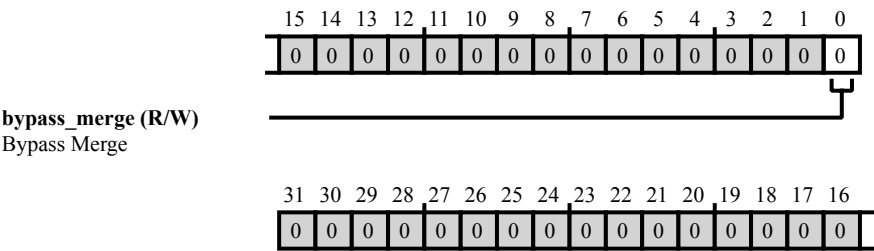


Figure 37-123: SCB4_FIR_CH1_IB_FN_MOD2 Register Diagram

Table 37-128: SCB4_FIR_CH1_IB_FN_MOD2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	BYPASS_MERGE	Bypass Merge.

Fir Ch1 Ib.read Qos

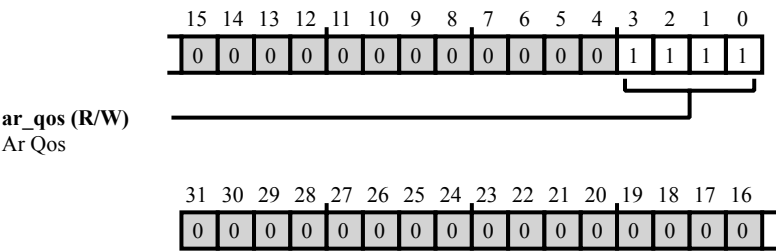


Figure 37-124: SCB4_FIR_CH1_IB_READ_QOS Register Diagram

Table 37-129: SCB4_FIR_CH1_IB_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

Fir Ch1 Ib.write Qos

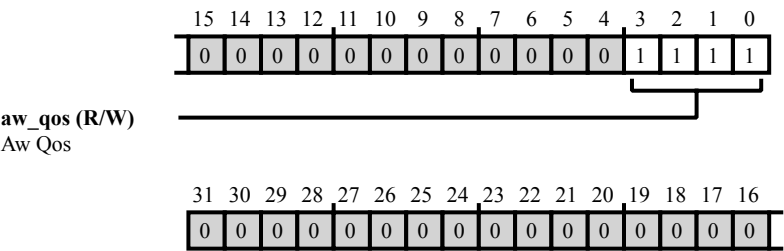


Figure 37-125: SCB4_FIR_CH1_IB_WRITE_QOS Register Diagram

Table 37-130: SCB4_FIR_CH1_IB_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

lir Ch0 Ib.fn Mod

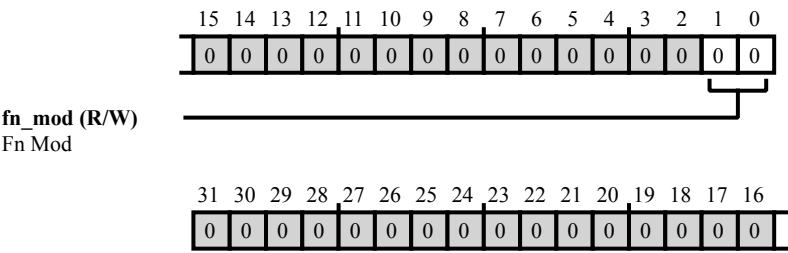


Figure 37-126: SCB4_IIR_CH0_IB_FN_MOD Register Diagram

Table 37-131: SCB4_IIR_CH0_IB_FN_MOD Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
1:0 (R/W)	FN_MOD	Fn Mod.

lir Ch0 Ib.fn Mod2

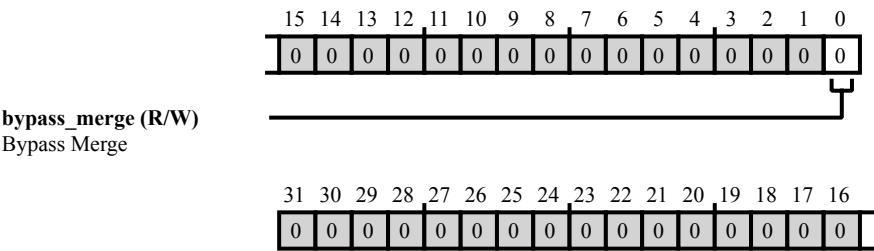


Figure 37-127: SCB4_IIR_CH0_IB_FN_MOD2 Register Diagram

Table 37-132: SCB4_IIR_CH0_IB_FN_MOD2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	BYPASS_MERGE	Bypass Merge.

lir Ch0 Ib.read Qos

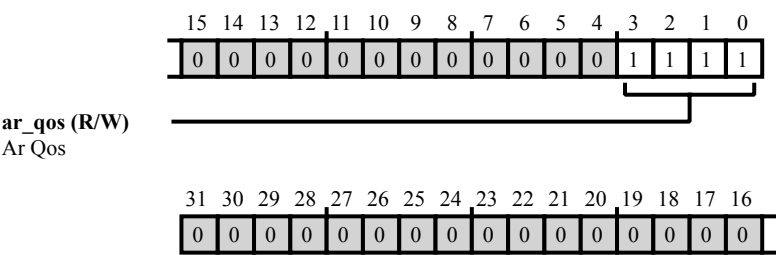


Figure 37-128: SCB4_IIR_CH0_IB_READ_QOS Register Diagram

Table 37-133: SCB4_IIR_CH0_IB_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

lir Ch0 Ib.write Qos

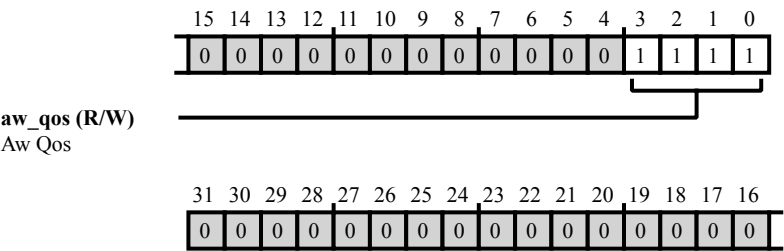


Figure 37-129: SCB4_IIR_CH0_IB_WRITE_QOS Register Diagram

Table 37-134: SCB4_IIR_CH0_IB_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

lir Ch1 Ib.fn Mod

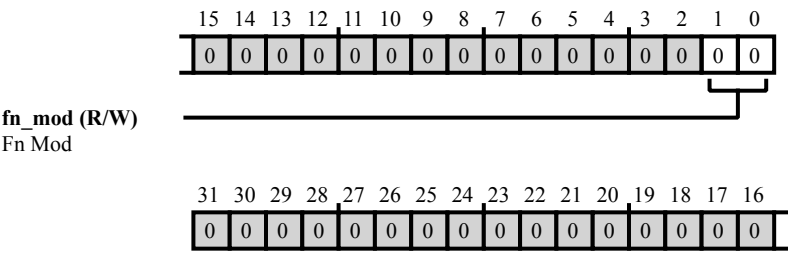


Figure 37-130: SCB4_IIR_CH1_IB_FN_MOD Register Diagram

Table 37-135: SCB4_IIR_CH1_IB_FN_MOD Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
1:0 (R/W)	FN_MOD	Fn Mod.

lir Ch1 Ib.fn Mod2

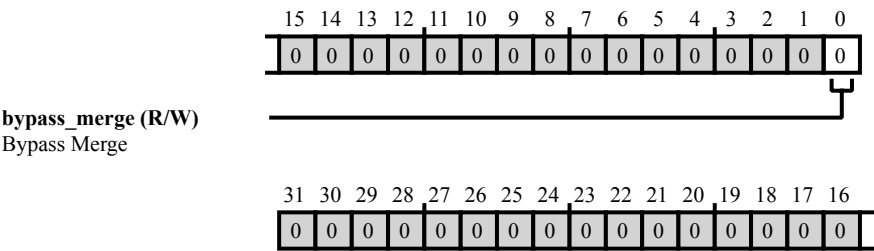


Figure 37-131: SCB4_IIR_CH1_IB_FN_MOD2 Register Diagram

Table 37-136: SCB4_IIR_CH1_IB_FN_MOD2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	BYPASS_MERGE	Bypass Merge.

lir Ch1 Ib.read Qos

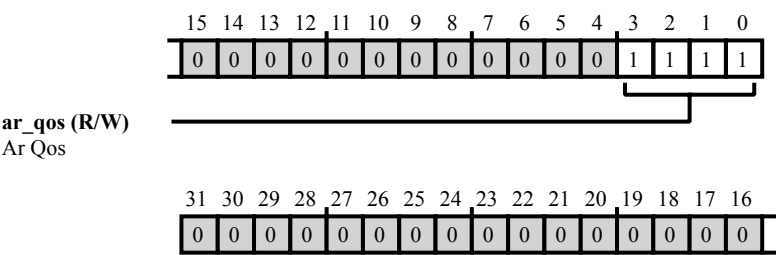


Figure 37-132: SCB4_IIR_CH1_IB_READ_QOS Register Diagram

Table 37-137: SCB4_IIR_CH1_IB_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

lir Ch1 Ib.write Qos

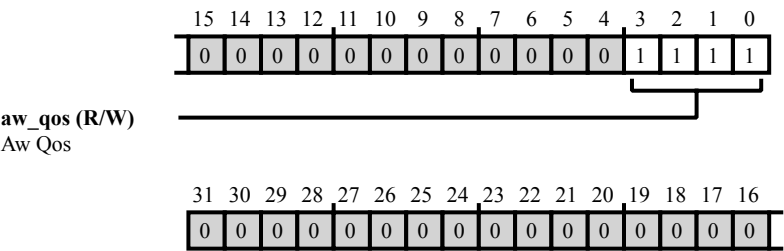


Figure 37-133: SCB4_IIR_CH1_IB_WRITE_QOS Register Diagram

Table 37-138: SCB4_IIR_CH1_IB_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

Periph Id 0

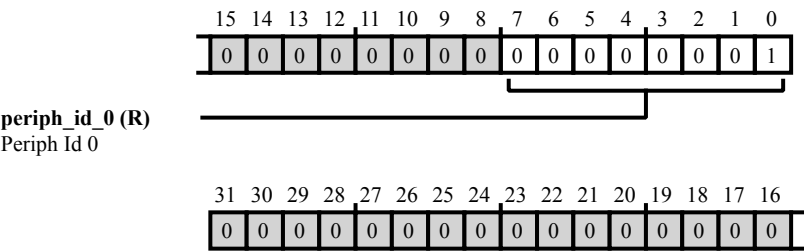


Figure 37-134: SCB4_PERIPH_ID_0 Register Diagram

Table 37-139: SCB4_PERIPH_ID_0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	PERIPH_ID_0	Periph Id 0.

Periph Id 1

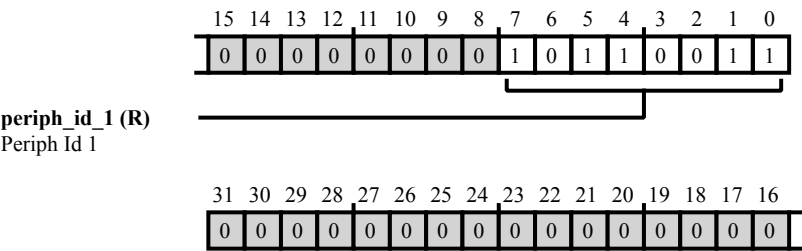


Figure 37-135: SCB4_PERIPH_ID_1 Register Diagram

Table 37-140: SCB4_PERIPH_ID_1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	PERIPH_ID_1	Periph Id 1.

Periph Id 2

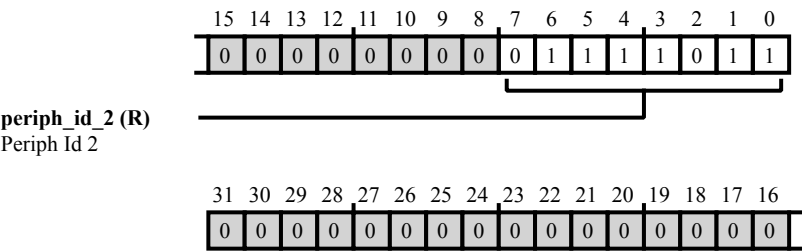


Figure 37-136: SCB4_PERIPH_ID_2 Register Diagram

Table 37-141: SCB4_PERIPH_ID_2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	PERIPH_ID_2	Periph Id 2.

Periph Id 3

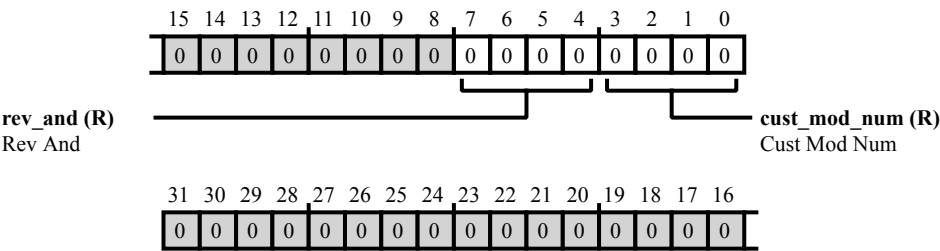


Figure 37-137: SCB4_PERIPH_ID_3 Register Diagram

Table 37-142: SCB4_PERIPH_ID_3 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:4 (R/NW)	REV_AND	Rev And.
3:0 (R/NW)	CUST_MOD_NUM	Cust Mod Num.

Periph Id 4

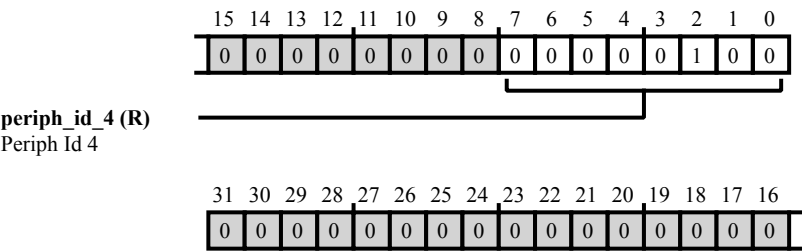


Figure 37-138: SCB4_PERIPH_ID_4 Register Diagram

Table 37-143: SCB4_PERIPH_ID_4 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	PERIPH_ID_4	Periph Id 4.

Periph Id 5

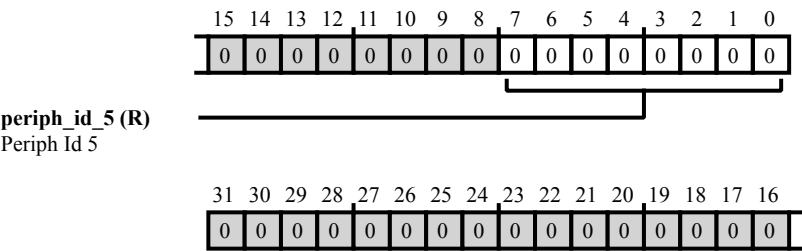


Figure 37-139: SCB4_PERIPH_ID_5 Register Diagram

Table 37-144: SCB4_PERIPH_ID_5 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	PERIPH_ID_5	Periph Id 5.

Periph Id 6

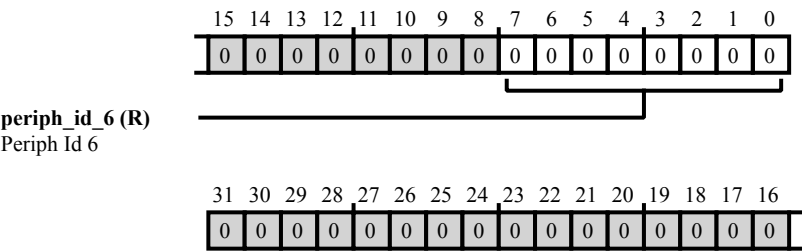


Figure 37-140: SCB4_PERIPH_ID_6 Register Diagram

Table 37-145: SCB4_PERIPH_ID_6 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	PERIPH_ID_6	Periph Id 6.

Periph Id 7

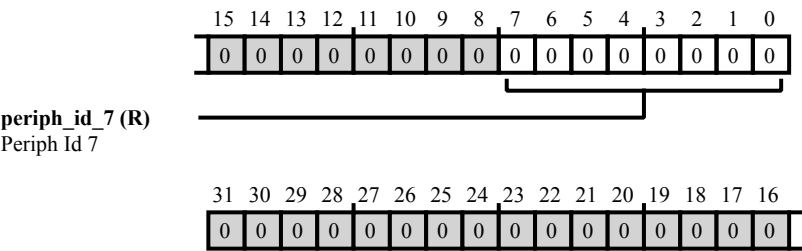


Figure 37-141: SCB4_PERIPH_ID_7 Register Diagram

Table 37-146: SCB4_PERIPH_ID_7 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	PERIPH_ID_7	Periph Id 7.

Sharc Dport.fn Mod

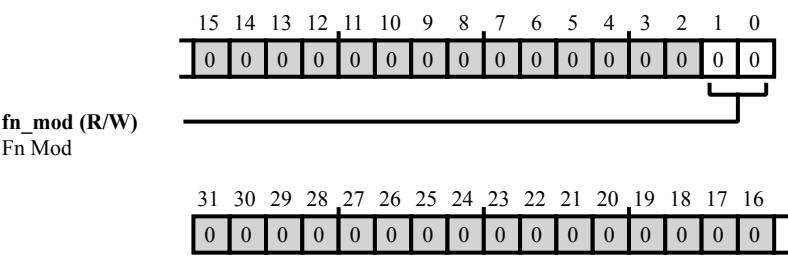


Figure 37-142: SCB4_SHARC_DPORT_FN_MOD Register Diagram

Table 37-147: SCB4_SHARC_DPORT_FN_MOD Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
1:0 (R/W)	FN_MOD	Fn Mod.

Sharc Dport.read Qos

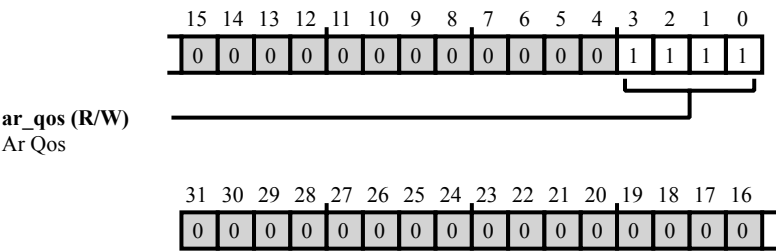


Figure 37-143: SCB4_SHARC_DPORT_READ_QOS Register Diagram

Table 37-148: SCB4_SHARC_DPORT_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

Sharc Dport.write Qos

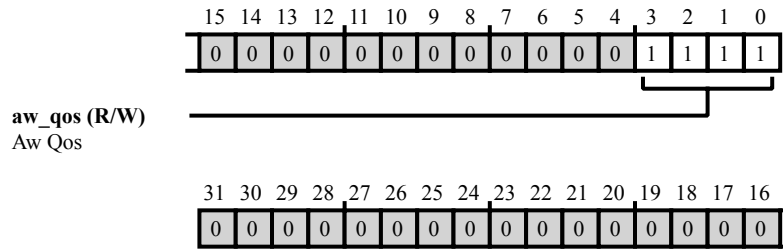


Figure 37-144: SCB4_SHARC_DPORT_WRITE_QOS Register Diagram

Table 37-149: SCB4_SHARC_DPORT_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

ADSP-21560_21561_21564_21568 SCB5 Register Descriptions

SPI interface Fabric (SCB5) contains the following registers.

Table 37-150: ADSP-21560_21561_21564_21568 SCB5 Register List

Name	Description
SCB5_COMP_ID_0	Comp Id 0
SCB5_COMP_ID_1	Comp Id 1
SCB5_COMP_ID_2	Comp Id 2
SCB5_COMP_ID_3	Comp Id 3
SCB5_PERIPH_ID_0	Periph Id 0
SCB5_PERIPH_ID_1	Periph Id 1
SCB5_PERIPH_ID_2	Periph Id 2
SCB5_PERIPH_ID_3	Periph Id 3
SCB5_PERIPH_ID_4	Periph Id 4
SCB5_PERIPH_ID_5	Periph Id 5
SCB5_PERIPH_ID_6	Periph Id 6
SCB5_PERIPH_ID_7	Periph Id 7
SCB5_REMAP	Remap
SCB5_SPI2_S_IB_FN_MOD	Spi2 S Ib.fn Mod

Table 37-150: ADSP-21560_21561_21564_21568 SCB5 Register List (Continued)

Name	Description
SCB5_SPI2_S_IB_FN_MOD_ISS_BM	Spi2 S Ib.fn Mod Iss Bm
SCB5_SPIF_M_FN_MOD	Spif M.fn Mod

Comp Id 0

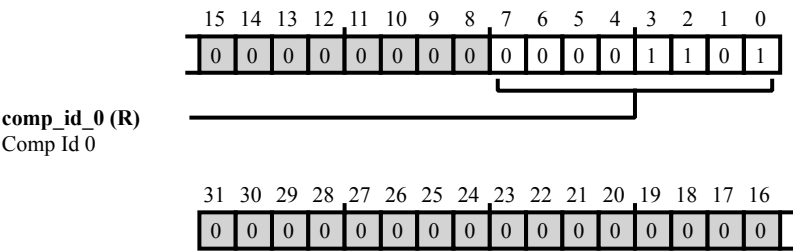


Figure 37-145: SCB5_COMP_ID_0 Register Diagram

Table 37-151: SCB5_COMP_ID_0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	COMP_ID_0	Comp Id 0.

Comp Id 1

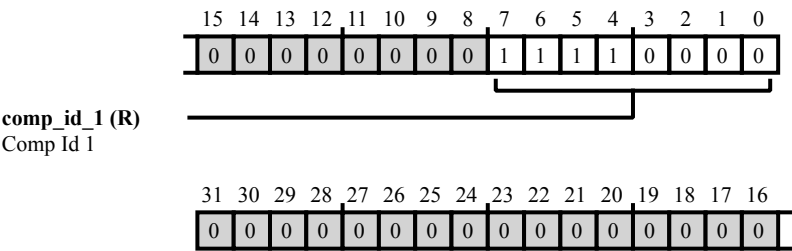


Figure 37-146: SCB5_COMP_ID_1 Register Diagram

Table 37-152: SCB5_COMP_ID_1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	COMP_ID_1	Comp Id 1.

Comp Id 2

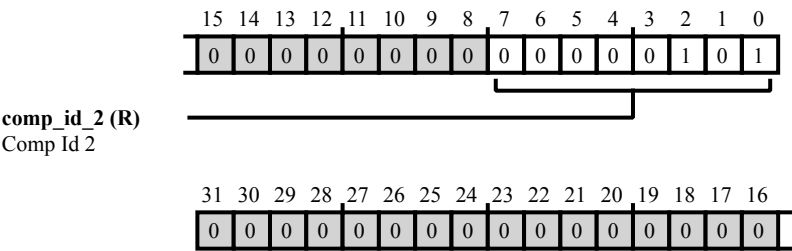


Figure 37-147: SCB5_COMP_ID_2 Register Diagram

Table 37-153: SCB5_COMP_ID_2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	COMP_ID_2	Comp Id 2.

Comp Id 3

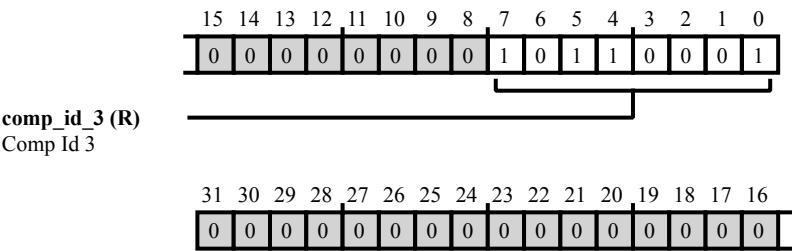


Figure 37-148: SCB5_COMP_ID_3 Register Diagram

Table 37-154: SCB5_COMP_ID_3 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	COMP_ID_3	Comp Id 3.

Periph Id 0

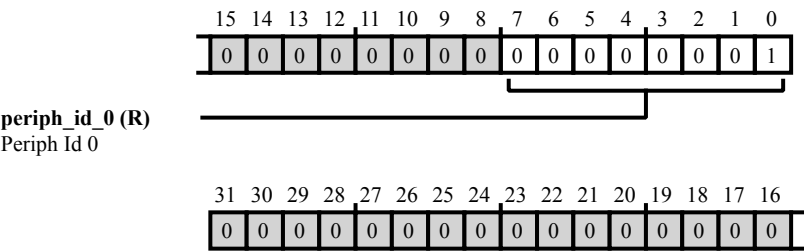


Figure 37-149: SCB5_PERIPH_ID_0 Register Diagram

Table 37-155: SCB5_PERIPH_ID_0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	PERIPH_ID_0	Periph Id 0.

Periph Id 1

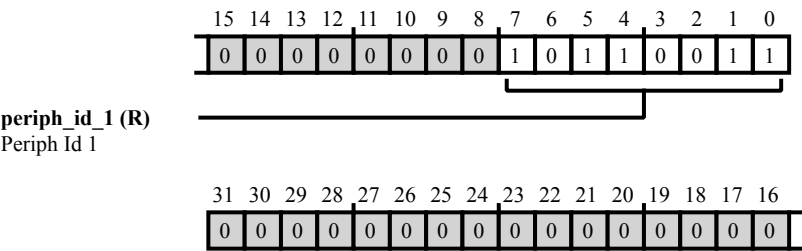


Figure 37-150: SCB5_PERIPH_ID_1 Register Diagram

Table 37-156: SCB5_PERIPH_ID_1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	PERIPH_ID_1	Periph Id 1.

Periph Id 2

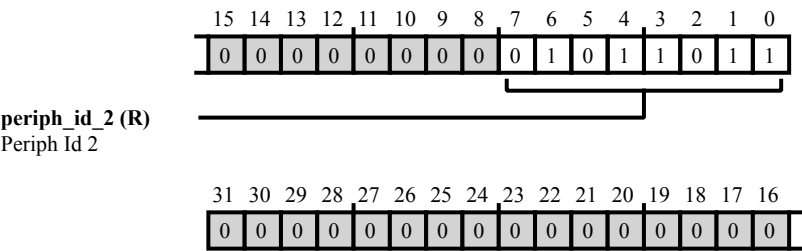


Figure 37-151: SCB5_PERIPH_ID_2 Register Diagram

Table 37-157: SCB5_PERIPH_ID_2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	PERIPH_ID_2	Periph Id 2.

Periph Id 3

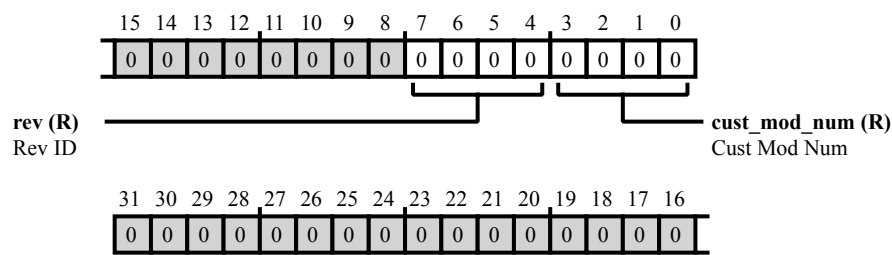


Figure 37-152: SCB5_PERIPH_ID_3 Register Diagram

Table 37-158: SCB5_PERIPH_ID_3 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:4 (R/NW)	REV	Rev ID.
3:0 (R/NW)	CUST_MOD_NUM	Cust Mod Num.

Periph Id 4

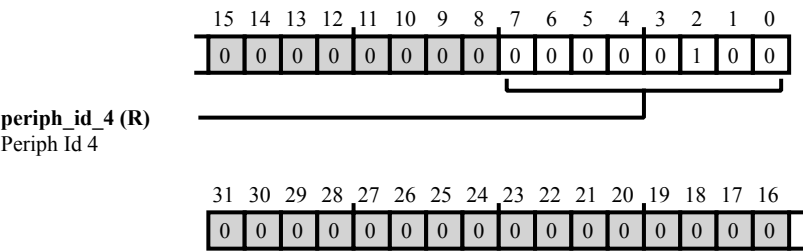


Figure 37-153: SCB5_PERIPH_ID_4 Register Diagram

Table 37-159: SCB5_PERIPH_ID_4 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	PERIPH_ID_4	Periph Id 4.

Periph Id 5

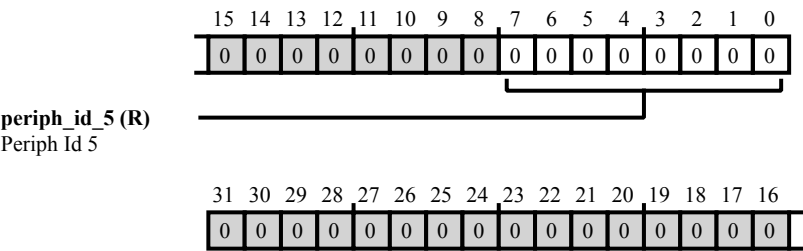


Figure 37-154: SCB5_PERIPH_ID_5 Register Diagram

Table 37-160: SCB5_PERIPH_ID_5 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	PERIPH_ID_5	Periph Id 5.

Periph Id 6

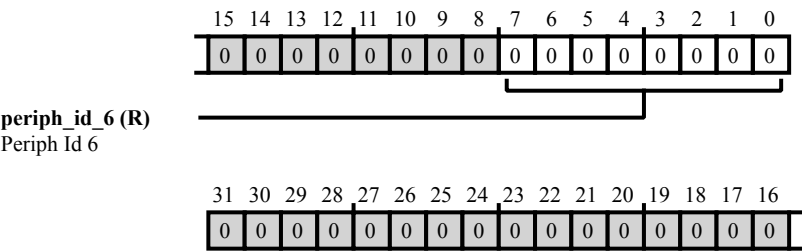


Figure 37-155: SCB5_PERIPH_ID_6 Register Diagram

Table 37-161: SCB5_PERIPH_ID_6 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	PERIPH_ID_6	Periph Id 6.

Periph Id 7

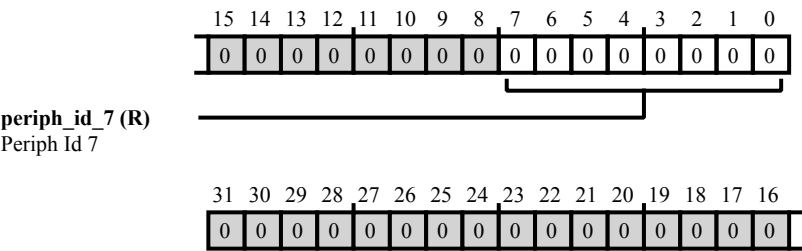


Figure 37-156: SCB5_PERIPH_ID_7 Register Diagram

Table 37-162: SCB5_PERIPH_ID_7 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	PERIPH_ID_7	Periph Id 7.

Remap

Routes transactions with address from 0x6000_0000 to 0x7FFF_FFFF

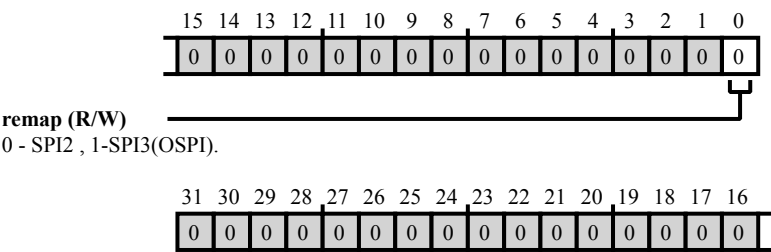


Figure 37-157: SCB5_REMAP Register Diagram

Table 37-163: SCB5_REMAP Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	REMAP	0 - SPI2 , 1-SPI3(OSPI)..

Spi2 S Ib.fn Mod

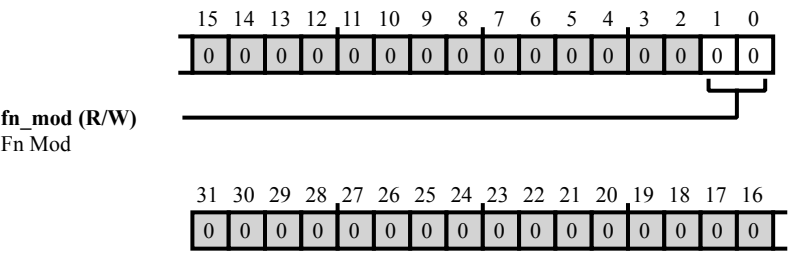


Figure 37-158: SCB5_SPI2_S_IB_FN_MOD Register Diagram

Table 37-164: SCB5_SPI2_S_IB_FN_MOD Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
1:0 (R/W)	FN_MOD	Fn Mod.

Spi2 S Ib.fn Mod Iss Bm

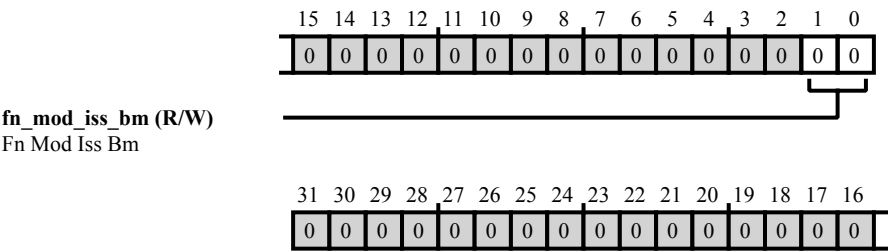


Figure 37-159: SCB5_SPI2_S_IB_FN_MOD_ISS_BM Register Diagram

Table 37-165: SCB5_SPI2_S_IB_FN_MOD_ISS_BM Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
1:0 (R/W)	FN_MOD_ISS_BM	Fn Mod Iss Bm.

Spif M.fn Mod

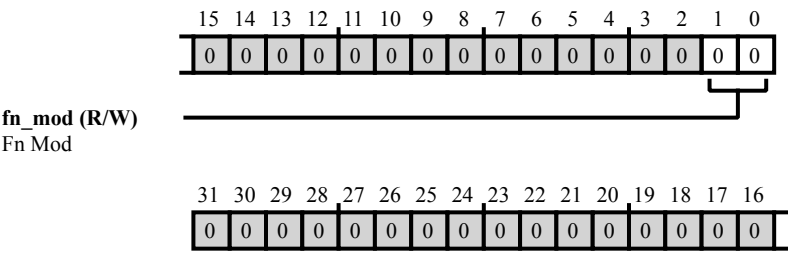


Figure 37-160: SCB5_SPIF_M_FN_MOD Register Diagram

Table 37-166: SCB5_SPIF_M_FN_MOD Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
1:0 (R/W)	FN_MOD	Fn Mod.

38 Memory Error Protection Unit (MEPU)

Memory Error Protection Unit (MEPU) handles single-bit and double-bit memory error detection and correction across the memories in the chip and controls routing of their interrupts/triggers.

It consists of following blocks:

- Memory Error Controller (MEC)
- Parity Controller (PCTL)

Memory Error Controller (MEC)

MEC Features

The MEC unit has the following features:

- N x M multiplexer for error inputs to interrupt and trigger outputs
- Peripheral bus completer interface for register read and write
- Control and status registers
 - Control for enabling or disabling each input error status and masking its interrupt
 - Control for enabling or disabling each output error interrupt
 - Input error status bits are sticky and write-1-to-clear
 - Output error interrupt status bits are read only
 - Lock feature for control registers write. Status registers capture lock write error.
- Peripheral and component identification registers
- Fully-configurable design

Parity Controller (PCTL)

The Parity Controller (PCTL) handles parity encoding and decoding to and from memory data for single bit memory error detection.

PCTL Features

The PCTL unit has the following features:

- Generic memory parity controller
 - Generation of write parity bits from write data bits
 - Concatenation of parity bits with data bits for memory write data
 - Generation of write enable mask for parity bits to memory
 - Extraction of read data bits from memory read data
 - Detection of read parity error from memory read data
- Even parity logic for parity generation and error detection
- Provision for parity bit interleaving
- Single parity error output per memory instance
- Memory initialization control logic
- Fully-configurable design

MEC Functional Description

The MEC works with the PCTL to detect and correct memory error across the memories in the chip. It also controls the routing of the generated interrupts and triggers.

ADSP-21560_21561_21564_21568 MEC Register List

The Memory Error Controller (MEC) manages memory parity/ inputs from the core and peripherals and sends interrupt/trigger outputs. It is a generic N x M multiplexer and interrupt controller for error to interrupt/trigger outputs. It has control features such as enable/disable and interrupt masking, and status signaling for inputs/outputs. For more information on MEC functionality, see the MEC register descriptions.

Table 38-1: ADSP-21560_21561_21564_21568 MEC Register List

Name	Description
MEC_CID0	Component ID0 Register
MEC_CID1	Component ID1 Register

Table 38-1: ADSP-21560_21561_21564_21568 MEC Register List (Continued)

Name	Description
MEC_CID2	Component ID2 Register
MEC_CID3	Component ID3 Register
MEC_CLR	Clear Register
MEC_L2PERR_CTL[n]	L2 Parity Error Control Register
MEC_L2PERR_IMASK[n]	L2 Parity Error Interrupt Mask Register
MEC_L2PERR_STAT[n]	L2 Parity Error Status Register
MEC_L2PIRQ_GCTL[n]	L2 Parity Error Interrupt Request Global Control Register
MEC_L2PIRQ_GSTAT[n]	L2 Parity Error Interrupt Request Global Status Register
MEC_PEIRQ_GCTL[n]	Parity Error Interrupt Request Global Control Register
MEC_PEIRQ_GSTAT[n]	Parity Error Interrupt Request Global Status Register
MEC_PERR_CTL	Parity Error Control Register
MEC_PERR_IMASK	Parity Error Interrupt Mask Register
MEC_PERR_STAT	Parity Error Status Register
MEC_PID0	Peripheral ID0 Register
MEC_PID1	Peripheral ID1 Register
MEC_PID2	Peripheral ID2 Register
MEC_PID3	Peripheral ID3 Register
MEC_PID4	Peripheral ID4 Register

ADSP-21560_21561_21564_21568 MEC Trigger List

Table 38-2: ADSP-21560_21561_21564_21568 MEC Trigger List Generators

Trigger ID	Name	Description	Sensitivity
95	MEC0_L2PIRQ0	MEC0 L2 Parity Interrupt Request	Level
96	MEC0_PEIRQ0	MEC0 Parity Error Interrupt Request	Level
97	MEC0_PEIRQ1	MEC0 Parity Error Interrupt Request	Level

Table 38-3: ADSP-21560_21561_21564_21568 MEC Trigger List Receivers

Trigger ID	Name	Description	Sensitivity
None			

MEPU Block Diagram

The *MEPU Block Diagram* shows the functional blocks within the Memory Error Protection Unit (MEPU) unit.

MEC Block Diagram

The *MEC Interface Block Diagram* shows the functional blocks within the MEC module.

PCTL Block Diagram

The *PCTL Block Diagram* shows the functional blocks within the PCTL module.

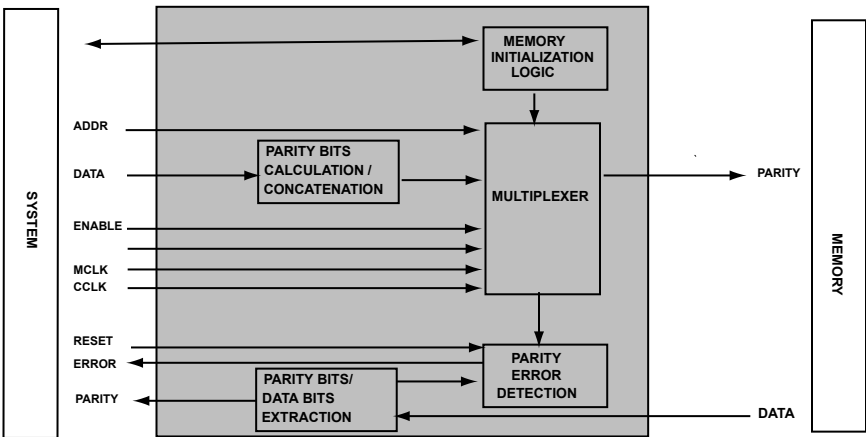


Figure 38-1: PCTL Block Diagram

MEC Architectural Concepts

PCTL Integration

There is one PCTL instance per port per memory instance (for example, one PCTL instance for single port memory and two PCTL instances for dual port memories).

ADSP-21560_21561_21564_21568 MEC Register Descriptions

Memory Error Controller (MEC) contains the following registers.

Table 38-4: ADSP-21560_21561_21564_21568 MEC Register List

Name	Description
MEC_CID0	Component ID0 Register
MEC_CID1	Component ID1 Register
MEC_CID2	Component ID2 Register
MEC_CID3	Component ID3 Register
MEC_CLR	Clear Register
MEC_L2PERR_CTL[n]	L2 Parity Error Control Register
MEC_L2PERR_IMASK[n]	L2 Parity Error Interrupt Mask Register
MEC_L2PERR_STAT[n]	L2 Parity Error Status Register

Table 38-4: ADSP-21560_21561_21564_21568 MEC Register List (Continued)

Name	Description
MEC_L2PIRQ_GCTL[n]	L2 Parity Error Interrupt Request Global Control Register
MEC_L2PIRQ_GSTAT[n]	L2 Parity Error Interrupt Request Global Status Register
MEC_PEIRQ_GCTL[n]	Parity Error Interrupt Request Global Control Register
MEC_PEIRQ_GSTAT[n]	Parity Error Interrupt Request Global Status Register
MEC_PERR_CTL	Parity Error Control Register
MEC_PERR_IMASK	Parity Error Interrupt Mask Register
MEC_PERR_STAT	Parity Error Status Register
MEC_PID0	Peripheral ID0 Register
MEC_PID1	Peripheral ID1 Register
MEC_PID2	Peripheral ID2 Register
MEC_PID3	Peripheral ID3 Register
MEC_PID4	Peripheral ID4 Register

Component ID0 Register

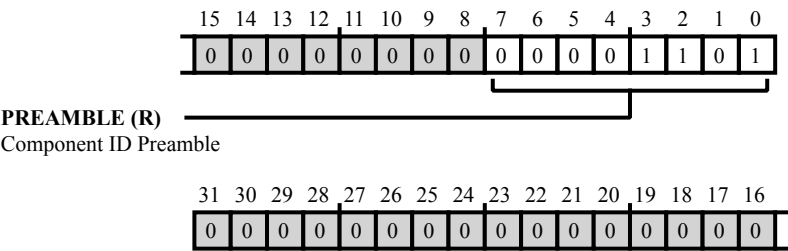


Figure 38-2: MEC_CID0 Register Diagram

Table 38-5: MEC_CID0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	PREAMBLE	Component ID Preamble. The MEC_CID0 . PREAMBLE field indicates the component ID preamble.

Component ID1 Register

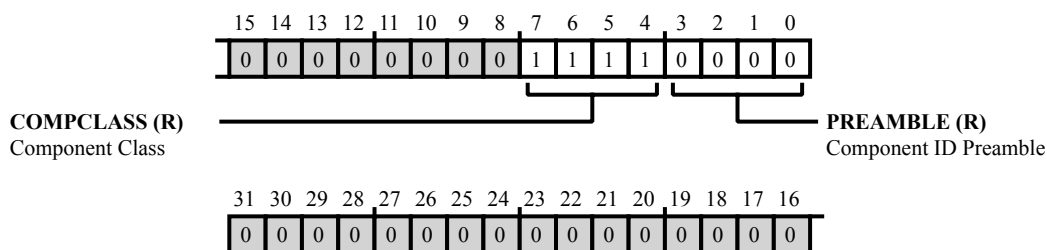


Figure 38-3: MEC_CID1 Register Diagram

Table 38-6: MEC_CID1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:4 (R/NW)	COMPCLASS	Component Class. The MEC_CID1.COMPCLASS field indicates the component class. Dedicated debug blocks (core debug access port, Program Trace, etc.) should identify as CoreSight (i.e. 0x9) and implement the full compliment of CoreSight registers including DEVTYPE. All other ADI components should identify as System (i.e. 0xF) components. See CoreSight Architecture Specification for details.
		9 CoreSight
		15 System
3:0 (R/NW)	PREAMBLE	Component ID Preamble. The MEC_CID1.PREAMBLE field indicates the component ID preamble.

Component ID2 Register

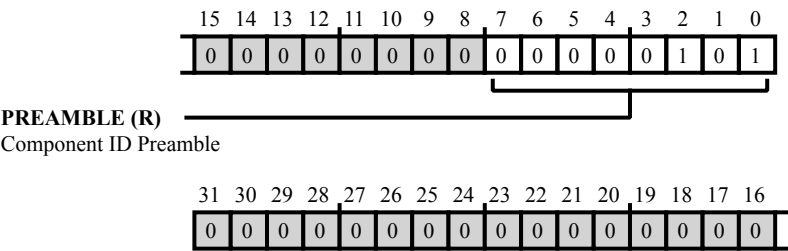


Figure 38-4: MEC_CID2 Register Diagram

Table 38-7: MEC_CID2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	PREAMBLE	Component ID Preamble. The MEC_CID2 . PREAMBLE field indicates the component ID preamble.

Component ID3 Register

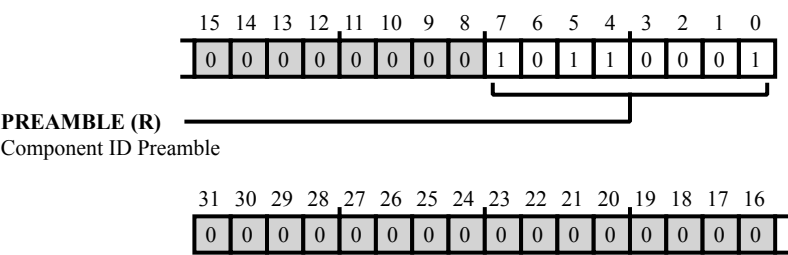


Figure 38-5: MEC_CID3 Register Diagram

Table 38-8: MEC_CID3 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	PREAMBLE	Component ID Preamble. The MEC_CID3 . PREAMBLE field indicates the component ID preamble.

Clear Register

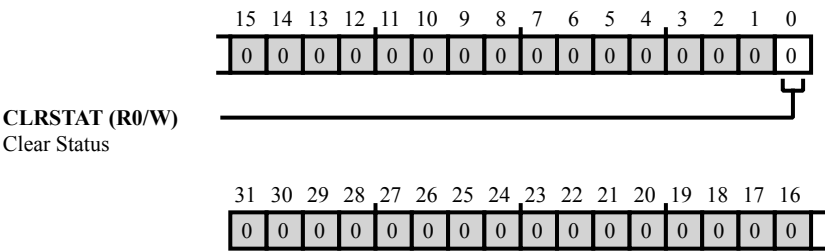


Figure 38-6: MEC_CLR Register Diagram

Table 38-9: MEC_CLR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R0/W)	CLRSTAT	Clear Status. Writing 1 to the MEC_CLR.CLRSTAT bit clears all status registers of MEC.

L2 Parity Error Control Register

The `MEC_L2PERR_CTL[n]` register bits control enable/disable for L2 parity error inputs from L2RAMs and decide whether their status will be reflected in L2 parity error status register bits.

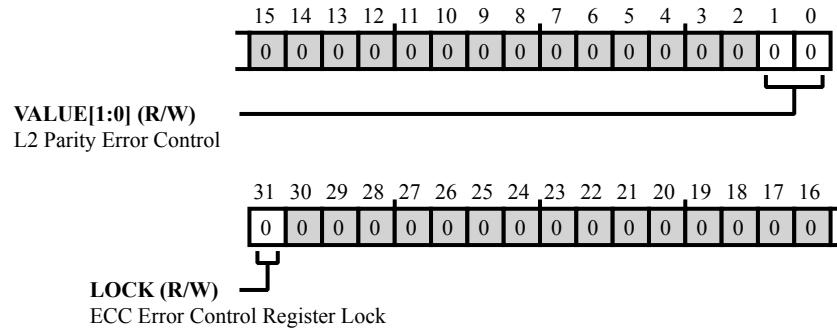


Figure 38-7: MEC_L2PERR_CTL[n] Register Diagram

Table 38-10: MEC_L2PERR_CTL[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	ECC Error Control Register Lock.
		If the global lock is enabled (<code>SPU_CTL.GLCK = 1</code>) and the <code>MEC_L2PERR_CTL[n].LOCK</code> bit is enabled, the <code>MEC_L2PERR_CTL[n]</code> register is read only.
		0 Unlock
		1 Lock
1:0 (R/W)	VALUE	L2 Parity Error Control.
		The <code>MEC_L2PERR_CTL[n].VALUE</code> bit indicates whether L2 parity error control is enabled (=1) or disabled (=0).
		1 L2 CTL ECC Error

L2 Parity Error Interrupt Mask Register

The `MEC_L2PERR_IMASK[n]` register bits control interrupt masks for L2 parity error inputs from L2RAMs.

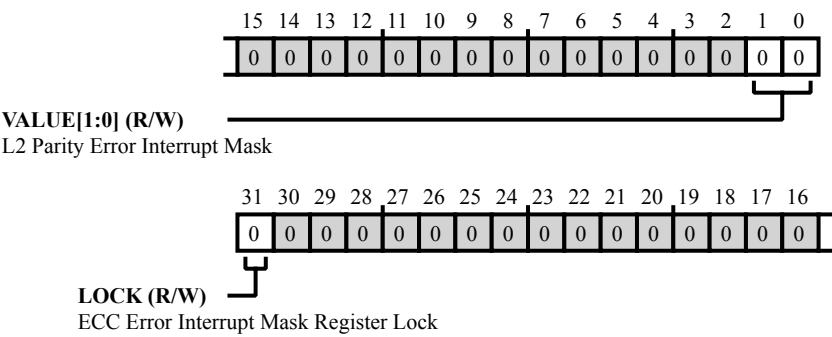


Figure 38-8: MEC_L2PERR_IMASK[n] Register Diagram

Table 38-11: MEC_L2PERR_IMASK[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	ECC Error Interrupt Mask Register Lock.
		If the global lock is enabled (<code>SPU_CTL.GLCK = 1</code>) and the <code>MEC_L2PERR_IMASK[n].LOCK</code> bit is enabled, the <code>MEC_L2PERR_IMASK[n]</code> register is read only.
		0 Unlock
		1 Lock
1:0 (R/W)	VALUE	L2 Parity Error Interrupt Mask.
		The <code>MEC_L2PERR_IMASK[n].VALUE</code> bit indicates when the L2 parity error interrupt is masked (=1) or unmasked (=0).
		1 L2 CTL ECC Error

L2 Parity Error Status Register

The `MEC_L2PERR_STAT[n]` register bits reflect the status for L2 parity error inputs from L2RAM. Writing '1' to these bits clears the corresponding L2 parity error status.

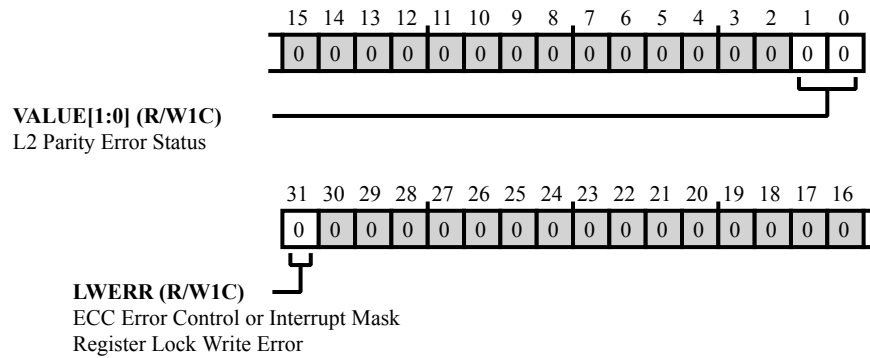


Figure 38-9: MEC_L2PERR_STAT[n] Register Diagram

Table 38-12: MEC_L2PERR_STAT[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W1C)	LWERR	ECC Error Control or Interrupt Mask Register Lock Write Error. The <code>MEC_L2PERR_STAT[n].LWERR</code> bit indicates (when set) there was an attempted write to an MEC register while the <code>MEC_L2PERR_CTL[n].LOCK</code> bit was set and while the global lock bit was enabled (<code>SPU_CTL.GLCK = 1</code>). This status bit is sticky; write 1 to clear it.
		0 No Error
		1 Error Occurred
1:0 (R/W1C)	VALUE	L2 Parity Error Status. When set (=1), the <code>MEC_L2PERR_STAT[n].VALUE</code> bit indicates the L2 parity error status. When cleared (=0), the bit indicates no error. This is a sticky bit; write 1 to clear.
		1 L2 CTL ECC Error

L2 Parity Error Interrupt Request Global Control Register

The `MEC_L2PIRQ_GCTL[n]` register bits control enable/disable of L2 parity error interrupt/trigger outputs.

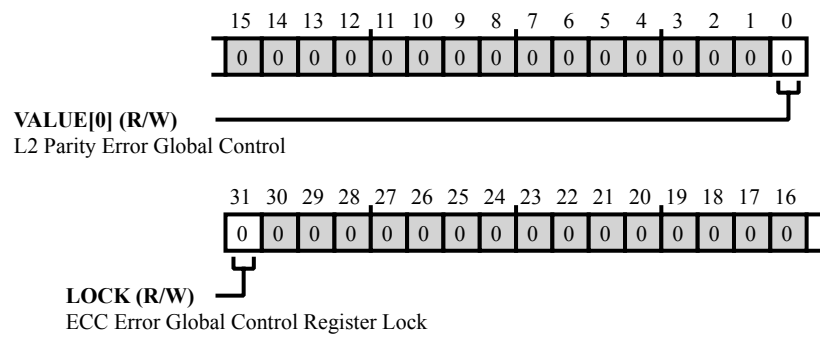


Figure 38-10: `MEC_L2PIRQ_GCTL[n]` Register Diagram

Table 38-13: `MEC_L2PIRQ_GCTL[n]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	ECC Error Global Control Register Lock. If the global lock is enabled (<code>SPU_CTL.GLCK = 1</code>) and the <code>MEC_L2PIRQ_GCTL[n].LOCK</code> bit is enabled, the <code>MEC_L2PIRQ_GCTL[n]</code> register is read only.
		0 Unlock
		1 Lock
0 (R/W)	VALUE	L2 Parity Error Global Control. The <code>MEC_L2PIRQ_GCTL[n].VALUE</code> bit indicates whether L2 parity error control is enabled (=1) or disabled (=0).

L2 Parity Error Interrupt Request Global Status Register

The `MEC_L2PIRQ_GSTAT[n]` register bits reflect the status of the L2 parity error interrupt/trigger outputs.

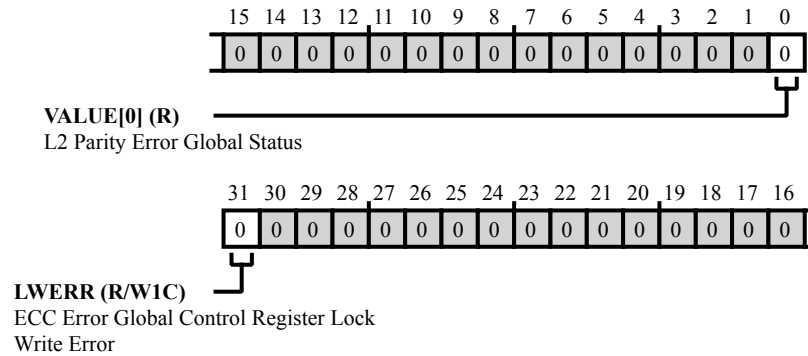


Figure 38-11: MEC_L2PIRQ_GSTAT[n] Register Diagram

Table 38-14: MEC_L2PIRQ_GSTAT[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W1C)	LWERR	ECC Error Global Control Register Lock Write Error. The <code>MEC_L2PIRQ_GSTAT[n] . LWERR</code> bit indicates (when set) there was an attempted write to an MEC register while the <code>MEC_L2PIRQ_GCTL[n] . LOCK</code> bit was set and while the global lock bit was enabled (<code>SPU_CTL.GLCK = 1</code>). This status bit is sticky; write 1 to clear it.
		0 No Error
		1 Error Occurred
0 (R/NW)	VALUE	L2 Parity Error Global Status. When set (=1), the <code>MEC_L2PIRQ_GSTAT[n] . VALUE</code> bit indicates an assertion of the interrupt/trigger. When cleared (=0), the bit indicates no interrupt/trigger.

Parity Error Interrupt Request Global Control Register

The `MEC_PEIRQ_GCTL[n]` register bits control enable/disable of parity error interrupt/trigger outputs.

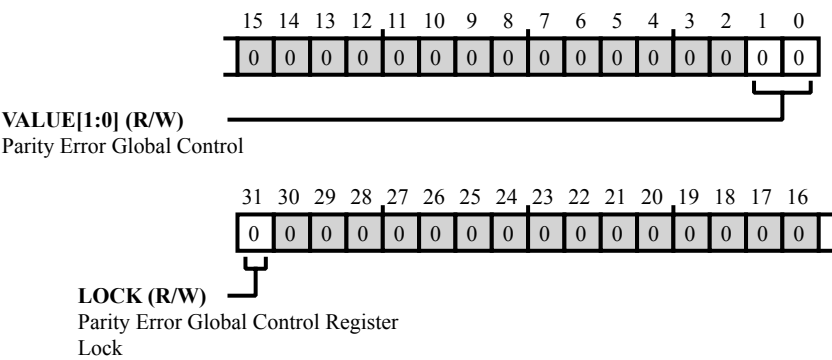


Figure 38-12: MEC_PEIRQ_GCTL[n] Register Diagram

Table 38-15: MEC_PEIRQ_GCTL[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Parity Error Global Control Register Lock. If the global lock is enabled (<code>SPU_CTL.GLCK = 1</code>) and the <code>MEC_PEIRQ_GCTL[n].LOCK</code> bit is enabled, the <code>MEC_PEIRQ_GCTL[n]</code> register is read only.
		0 Unlock
		1 Lock
1:0 (R/W)	VALUE	Parity Error Global Control. The <code>MEC_PEIRQ_GCTL[n].VALUE</code> bit field indicates whether parity error control is enabled (<code>=1</code>) or disabled (<code>=0</code>).

Parity Error Interrupt Request Global Status Register

The `MEC_PEIRQ_GSTAT[n]` register bits reflect status of parity error interrupt/trigger outputs.

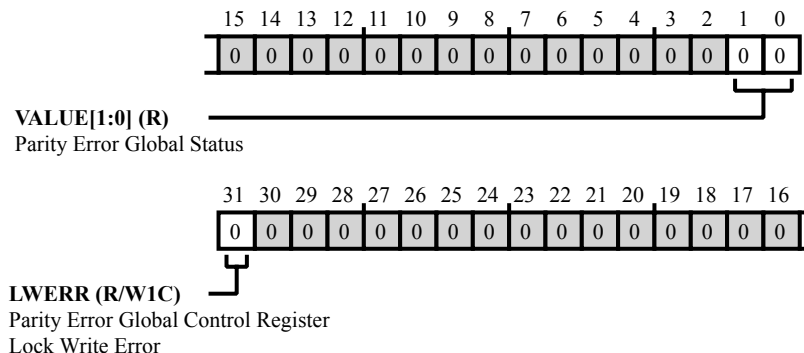


Figure 38-13: MEC_PEIRQ_GSTAT[n] Register Diagram

Table 38-16: MEC_PEIRQ_GSTAT[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W1C)	LWERR	Parity Error Global Control Register Lock Write Error. When set (=1), the <code>MEC_PEIRQ_GSTAT[n].LWERR</code> bit indicates an attempted write to an MEC register while the <code>MEC_PEIRQ_GCTL[n].LOCK</code> bit was set and while the global lock bit was enabled (<code>SPU_CTL.GLCK</code> bit =1). This status bit is sticky; write 1 to clear it.
		0 No Error
		1 Error Occurred
1:0 (R/NW)	VALUE	Parity Error Global Status. The <code>MEC_PEIRQ_GSTAT[n].VALUE</code> field indicates the parity error global status. 1 indicates the assertion of the interrupt/trigger; 0 indicates no interrupt/trigger.

Parity Error Control Register

The `MEC_PERR_CTL` register bits control enable/disable for parity error inputs from various cores/peripherals and decide whether their status will be reflected in parity error status register bits.

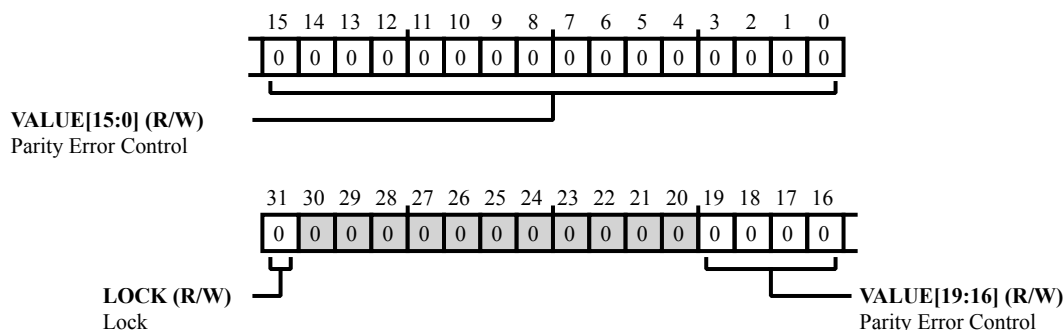


Figure 38-14: MEC_PERR_CTL Register Diagram

Table 38-17: MEC_PERR_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock. If the global lock is enabled (<code>SPU_CTL.GLCK = 1</code>) and the <code>MEC_PERR_CTL.LOCK</code> bit is enabled, the <code>MEC_PERR_CTL</code> register is read only.
		0 Unlock
		1 Lock
19:0 (R/W)	VALUE	Parity Error Control. The <code>MEC_PERR_CTL.VALUE</code> bit indicates whether parity error control is enabled (=1) or disabled (=0).
		1 Reserved
		2 Reserved
		4 Branch Predictor Parity Error
		8 ASRC0 FIFO Parity Error
		16 ASRC1 FIFO Parity Error
		32 ASRC2 FIFO Parity Error
		64 ASRC3 FIFO Parity Error
		128 ASRC4 FIFO Parity Error
		256 ASRC5 FIFO Parity Error
		512 ASRC6 FIFO Parity Error
		1024 ASRC7 FIFO Parity Error

Table 38-17: MEC_PERR_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
		2048	IIR0 RAM Parity Error
		4096	FIR0 RAM Parity Error
		8192	TRNG0 Data Buffer Parity Error
		16384	PKA0 Data RAM Parity Error
		32768	SPE0 Buffer RAM Parity Error
		65536	SPE0 ARC4 RAM Parity Error
		131072	MLB0 Data Buffer RAM Parity Error
		262144	MLB0 Channel Table RAM Parity Error
		524288	TMC0 Trace Data RAM Parity Error

Parity Error Interrupt Mask Register

The `MEC_PERR_IMASK` register bits control interrupt masks for parity error inputs from various cores/peripherals.

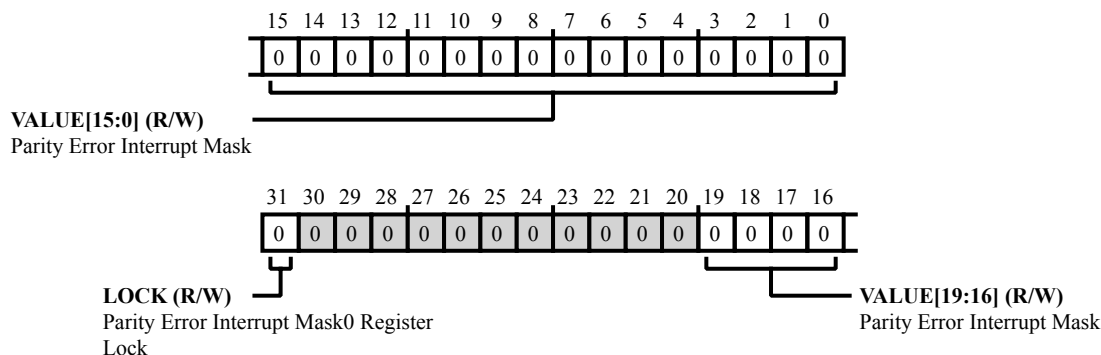


Figure 38-15: MEC_PERR_IMASK Register Diagram

Table 38-18: MEC_PERR_IMASK Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Parity Error Interrupt Mask0 Register Lock. If the global lock is enabled (<code>SPU_CTL.GLCK = 1</code>) and the <code>MEC_PERR_IMASK.LOCK</code> bit is enabled, the <code>MEC_PERR_IMASK</code> register is read only.
		0 Unlock
		1 Lock
19:0 (R/W)	VALUE	Parity Error Interrupt Mask. The <code>MEC_PERR_IMASK.VALUE</code> bit field, when set (=1), indicates an interrupt masked and, when cleared (=0), indicates an interrupt unmasked.
		1 Core 0 L1 RAM Parity Error
		2 Core 0 L2 Cache RAM Parity Error
		4 Branch Predictor Parity Error
		8 ASRC0 FIFO Parity Error
		16 ASRC1 FIFO Parity Error
		32 ASRC2 FIFO Parity Error
		64 ASRC3 FIFO Parity Error
		128 ASRC4 FIFO Parity Error
		256 ASRC5 FIFO Parity Error
		512 ASRC6 FIFO Parity Error
		1024 ASRC7 FIFO Parity Error

Table 38-18: MEC_PERR_IMASK Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
		2048	IIR0 RAM Parity Error
		4096	FIR0 RAM Parity Error
		8192	TRNG0 Data Buffer Parity Error
		16384	PKA0 Data RAM Parity Error
		32768	SPE0 Buffer RAM Parity Error
		65536	SPE0 ARC4 RAM Parity Error
		131072	MLB0 Data Buffer RAM Parity Error
		262144	MLB0 Channel Table RAM Parity Error
		524288	TMC0 Trace Data RAM Parity Error

Parity Error Status Register

The `MEC_PERR_STAT` register bits reflect status for parity error inputs from various cores/peripherals. Writing '1' to these bits clear corresponding parity error status.

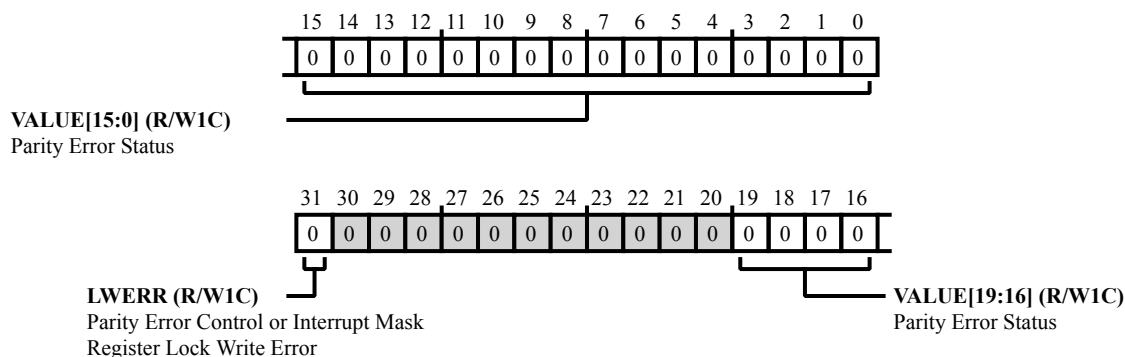


Figure 38-16: `MEC_PERR_STAT` Register Diagram

Table 38-19: `MEC_PERR_STAT` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W1C)	LWERR	Parity Error Control or Interrupt Mask Register Lock Write Error. The <code>MEC_PERR_STAT.LWERR</code> bit indicates (when set) there was an attempted write to an MEC register while the <code>MEC_PERR_CTL.LOCK</code> bit was set and while the global lock bit was enabled (<code>SPU_CTL.GLCK = 1</code>). This status bit is sticky; write-1-to-clear it.
		0 No Error
		1 Error Occurred
19:0 (R/W1C)	VALUE	Parity Error Status. When set (=1), the <code>MEC_PERR_STAT.VALUE</code> bit indicates the parity error status. When cleared (=0), the bit indicates no error. This is a sticky bit; write 1 to clear.
		1 Reserved
		2 Reserved
		4 Branch Predictor Parity Error
		8 ASRC0 FIFO Parity Error
		16 ASRC1 FIFO Parity Error
		32 ASRC2 FIFO Parity Error
		64 ASRC3 FIFO Parity Error
		128 ASRC4 FIFO Parity Error
		256 ASRC5 FIFO Parity Error

Table 38-19: MEC_PERR_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
		512 ASRC6 FIFO Parity Error
		1024 ASRC7 FIFO Parity Error
		2048 IIR0 RAM Parity Error
		4096 FIR0 RAM Parity Error
		8192 TRNG0 Data Buffer Parity Error
		16384 PKA0 Data RAM Parity Error
		32768 SPE0 Buffer RAM Parity Error
		65536 SPE0 ARC4 RAM Parity Error
		131072 MLB0 Data Buffer RAM Parity Error
		262144 MLB0 Channel Table RAM Parity Error
		524288 TMC0 Trace Data RAM Parity Error

Peripheral ID0 Register

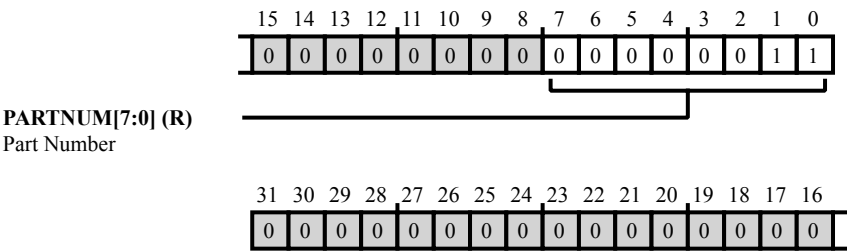


Figure 38-17: MEC_PID0 Register Diagram

Table 38-20: MEC_PID0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	PARTNUM	Part Number. The MEC_PID0 . PARTNUM field indicates the part number for component identification

Peripheral ID1 Register

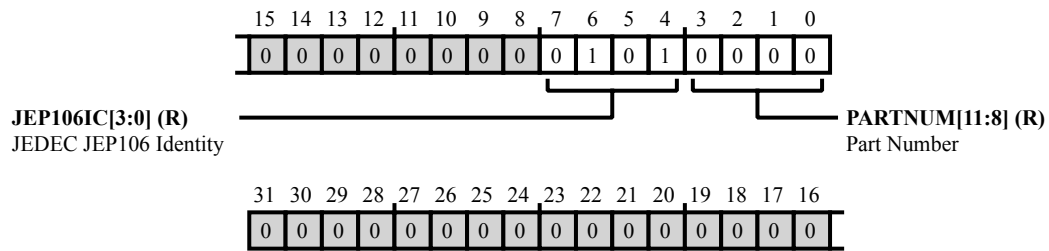


Figure 38-18: MEC_PID1 Register Diagram

Table 38-21: MEC_PID1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:4 (R/NW)	JEP106IC	JEDEC JEP106 Identity. The MEC_PID1 . JEP106IC field indicates JEDEC JEP106 identity (manufacturer ID) code.
3:0 (R/NW)	PARTNUM	Part Number. The MEC_PID1 . PARTNUM field indicates the part number for component identification

Peripheral ID2 Register

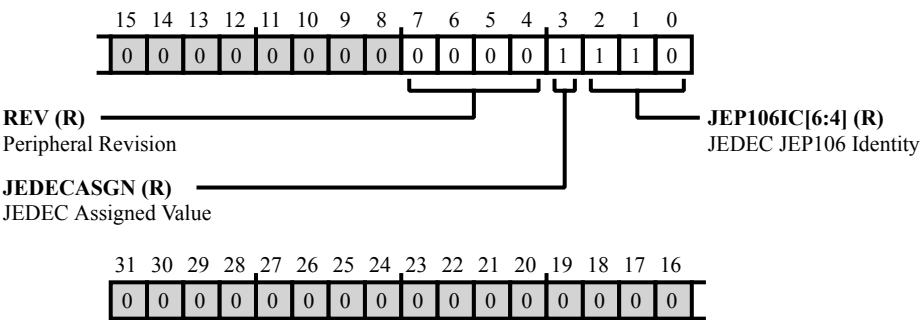


Figure 38-19: MEC_PID2 Register Diagram

Table 38-22: MEC_PID2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:4 (R/NW)	REV	Peripheral Revision.
3 (R/NW)	JEDECASGN	JEDEC Assigned Value.
2:0 (R/NW)	JEP106IC	JEDEC JEP106 Identity. The MEC_PID2 . JEP106IC field indicates JEDEC JEP106 identity (manufacturer ID) code.

Peripheral ID3 Register

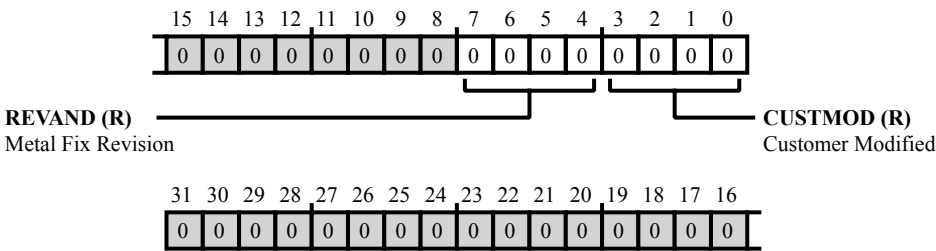


Figure 38-20: MEC_PID3 Register Diagram

Table 38-23: MEC_PID3 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:4 (R/NW)	REVAND	Metal Fix Revision.
3:0 (R/NW)	CUSTMOD	Customer Modified.

Peripheral ID4 Register

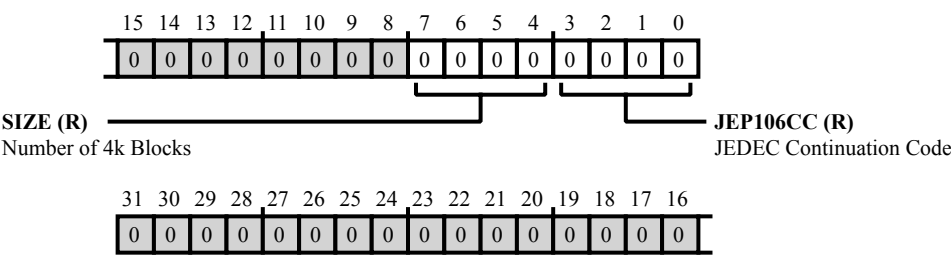


Figure 38-21: MEC_PID4 Register Diagram

Table 38-24: MEC_PID4 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:4 (R/NW)	SIZE	Number of 4k Blocks. The MEC_PID4 . SIZE field indicates the size of component 4k chunks minus 1 (for example, 0=4k).
3:0 (R/NW)	JEP106CC	JEDEC Continuation Code. The MEC_PID4 . JEP106CC field indicates the JEDEC JEP106 continuation code (number of leading 0x7Fs).

39 System Debug

System debug is based on Arm CoreSight technology. CoreSight is a set of architecture specifications defining debug and trace architecture. The processor uses CoreSight infrastructure to provide industry standard debug and trace capabilities.

<https://developer.arm.com/documentation>

The applicable documentation for more details about the ArmCoreSight feature includes:

- Embedded Cross Trigger Technical Reference Manual, ARM DDI 0291A

DBG Features

The debug unit has the following features.

- System JTAG TAP controller for system debug features, boundary scan, and public JTAG features
- A debug interface to core, and other system resources
- Direct and run-time access to the memory system and system MMRs
- Direct control over system reset
- Support for debug immediately after reset (boot debug)
- Group halt (debug event immediately halts all specified endpoints)
- Real-time on-chip visibility is made available to all developers, including software developers

Debug Functional Description

The following sections provide functional descriptions of the DBG unit.

DBG Block Diagram

The DBG block diagram is shown below.

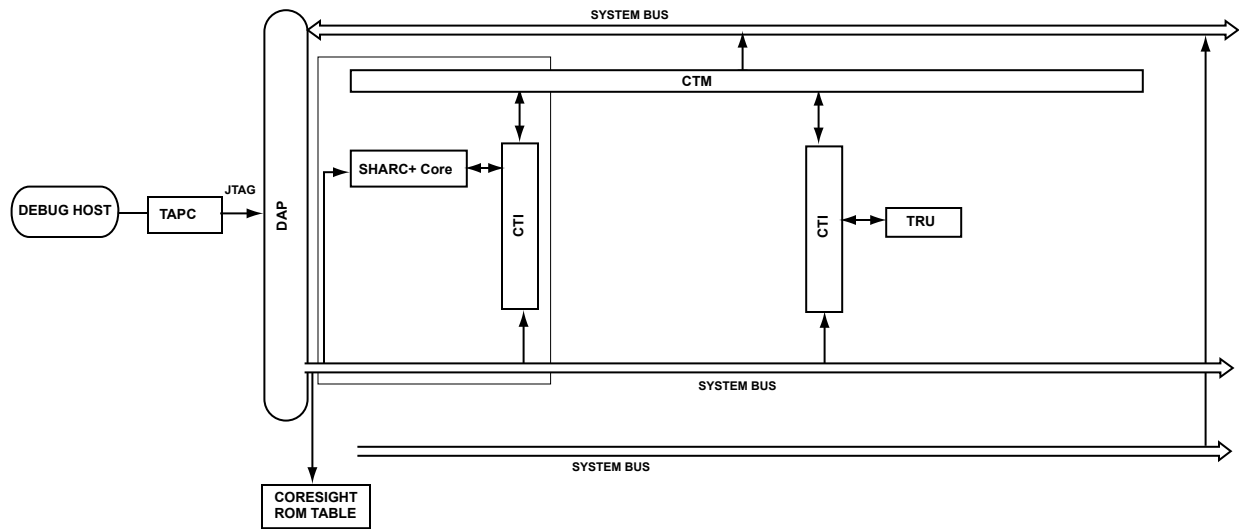


Figure 39-1: ADSP-2156x Block Diagram

DBG Definitions

The following terms are useful when working with the debug features of the processor and programming tools.

Test Access Port Controller (TAPC)

Provides IDCODE and SDBGKEY features.

Debug Access Port (DAP)

The CoreSight interface that provides a single port for two debug options: JTAG-DP (JTAG Debug Port), SW-DP (Serial Wire Debug Port)

Embedded Cross Trigger (ECT)

The ECTs are responsible managing events and triggers as follows.

- CTI (Cross Trigger Interface) is a CoreSight component for enabling cross triggering of events across a system. From the view of the ECT, it is responsible for combining and mapping trigger requests.
- CTM (Cross Trigger Matrix) is a CoreSight component for connecting multiple CTIs. From the view of the ECT, it is responsible for connecting CTIs and distribution of events.

NOTE: An embedded cross trigger is not the same as the requester and completer trigger in the trigger routing unit.

Test Access Port Controller (TAPC)

TAPC is an independent component daisy-chained to the DAP in the JTAG scan path. The TAPC component provides the product IDCODE (Chip ID) and a security feature.

The TAPC component provides security features to the chip using a debug key match features. This feature permits only those components which have the SDBGKEY to connect and debug the chip. Initially, a 128-bits user key is programmed in the SDBGKEY registers ([TAPC_SDBGKEY0](#), [TAPC_SDBGKEY1](#), [TAPC_SDBGKEY2](#), [TAPC_SDBGKEY3](#)) in TAPC by a secure requester. The `TAPC_SDBGKEY_CTL.VALID` bit is set.

Then, a user key is entered through the emulator for a match to the initially programmed-entered user key. On a successful match, the chip connects to the emulator. It can be debugged on a failed user key match. Further attempts at keys matching are disabled. JTAG reset or system reset is required to reenale the user key match logic.

Debug Access Ports

The DAP provides access to all debug and trace capabilities through a single external interface, a JTAG Debug Port (JTAG-DP). The JTAG-DP is based on the IEEE 1149.1 Test Access Port (TAP) and Boundary Scan Architecture. DAP has some additions to support BSCA and IDCODE features that Core Sight DAP does not support.

Embedded Cross Trigger (ECT)

ECT provides an interface to the CoreSight debug system enabling the subsystems to interact (cross trigger) with each other. ECT provides a mechanism to forward debug events from one connected subsystem to another connected subsystem. The different subsystems connected to the ECT depend on the processor design. For example, in a multiprocessor system, the interface can be connected to each of the cores. For a uniprocessor system, the interface can include just one core.

- CTI cross trigger interface. A CoreSight component for enabling cross triggering of events across a system.
- CTM cross trigger matrix. A CoreSight component for connecting multiple cross trigger interfaces.

The main function of the ECT (CTI and CTM) is to pass debug events from one connected subsystem to another connected subsystem.

For example, the ECT can communicate debug state information from the core to another core in a multiprocessor-based system. Program execution on both the subsystem can be stopped at the same time.

The *Trigger Flow* figure shows a simple debug trigger flow sequence. On each CTI, there are four channel, eight input, and eight output debug triggers. All the eight inputs and outputs can be mapped to a single channel or different channels based on the debug trigger to channel mapping. When a trigger input occurs, it creates a channel event. The channel event causes all the output debug triggers to be triggered. The embedded cross trigger depends on the debug trigger it connects to.

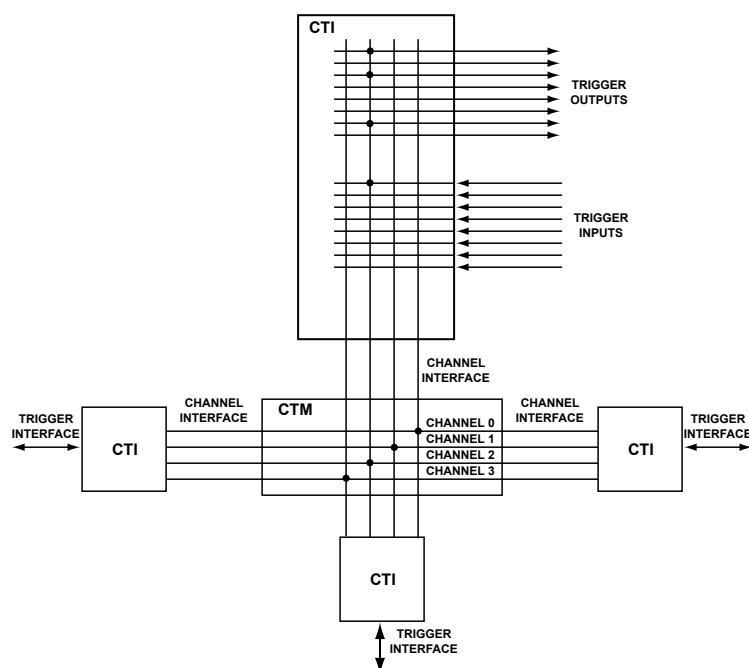


Figure 39-2: Trigger Flow

The *ECT Integration* figure shows three CTIs in the system that connect to the core and system module, respectively. The CTIs all interconnect through the CTM. This configuration allows the core to trigger debug events on the core.

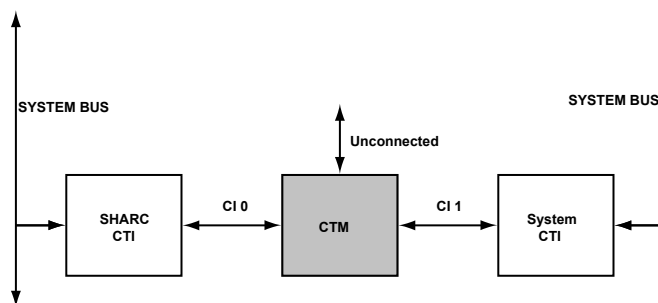


Figure 39-3: ECT Integration

ADSP-21560_21561_21564_21568 CTI Register Descriptions

CSCTI_Register_Definitions (CTI) contains the following registers.

Table 39-1: ADSP-21560_21561_21564_21568 CTI Register List

Name	Description
CTI_ASICCTL	External Multiplexor Control Register
CTI_AUTHSTATUS	Authentication Status

Table 39-1: ADSP-21560_21561_21564_21568 CTI Register List (Continued)

Name	Description
CTI_CLAIMCLR	Claim Tag Clear Register
CTI_CLAIMSET	Claim Tag Set Register
CTI_COMPID0	Component ID0
CTI_COMPID1	Component ID1
CTI_COMPID2	Component ID2
CTI_COMPID3	Component ID3
CTI_CTIAPPCLEAR	CTI Application Trigger Clear Register
CTI_CTIAPPULSE	CTI Application Pulse Register
CTI_CTIAPPSET	CTI Application Trigger Set Register
CTI_CTICHINSTATUS	CTI Channel In Status Register
CTI_CTICHOUTSTATUS	CTI Channel Out Status Register
CTI_CTICONTROL	CTI Control Register
CTI_CTIGATE	Enable CTI Channel Gate Register
CTI_CTIINEN0	CTI Trigger 0 to Channel Enable Register
CTI_CTIINEN1	CTI Trigger 1 to Channel Enable Register
CTI_CTIINEN2	CTI Trigger 2 to Channel Enable Register
CTI_CTIINEN3	CTI Trigger 3 to Channel Enable Register
CTI_CTIINEN4	CTI Trigger 4 to Channel Enable Register
CTI_CTIINEN5	CTI Trigger 5 to Channel Enable Register
CTI_CTIINEN6	CTI Trigger 6 to Channel Enable Register
CTI_CTIINEN7	CTI Trigger 7 to Channel Enable Register
CTI_CTIINTACK	CTI Interrupt Acknowledge Register
CTI_CTIOUTEN0	CTI Channel to Trigger 0 Enable Register
CTI_CTIOUTEN1	CTI Channel to Trigger 1 Enable Register
CTI_CTIOUTEN2	CTI Channel to Trigger 2 Enable Register
CTI_CTIOUTEN3	CTI Channel to Trigger 3 Enable Register
CTI_CTIOUTEN4	CTI Channel to Trigger 4 Enable Register
CTI_CTIOUTEN5	CTI Channel to Trigger 5 Enable Register
CTI_CTIOUTEN6	CTI Channel to Trigger 6 Enable Register
CTI_CTIOUTEN7	CTI Channel to Trigger 7 Enable Register
CTI_CTITRIGINSTATUS	CTI Trigger In Status Register

Table 39-1: ADSP-21560_21561_21564_21568 CTI Register List (Continued)

Name	Description
CTI_CTITRIGOUTSTATUS	CTI Trigger Out Status Register
CTI_DEVID	Device ID
CTI_DEVTYPE	Device Type
CTI_ITCHIN	Integration Test Channel Input
CTI_ITCHINACK	ITCHINACK
CTI_ITCHOUT	Integration Test Channel Output
CTI_ITCHOUTACK	ITCHOUTACK
CTI_ITCTRL	Integration Mode Control Register
CTI_ITTRIGIN	Integration Test Trigger Input
CTI_ITTRIGINACK	ITTRIGINACK
CTI_ITTRIGOUT	Integration Test Trigger Output
CTI_ITTRIGOUTACK	ITTRIGOUTACK
CTI_LAR	Lock Access Register
CTI_LSR	Lock Status Register
CTI_PERIPHID0	Peripheral ID0
CTI_PERIPHID1	Peripheral ID1
CTI_PERIPHID2	Peripheral ID2
CTI_PERIPHID3	Peripheral ID3
CTI_PERIPHID4	Peripheral ID4
CTI_PERIPHID5	Peripheral ID5
CTI_PERIPHID6	Peripheral ID6
CTI_PERIPHID7	Peripheral ID7

External Multiplexor Control Register

The CTI_ASICCTL register is for I/O port control.

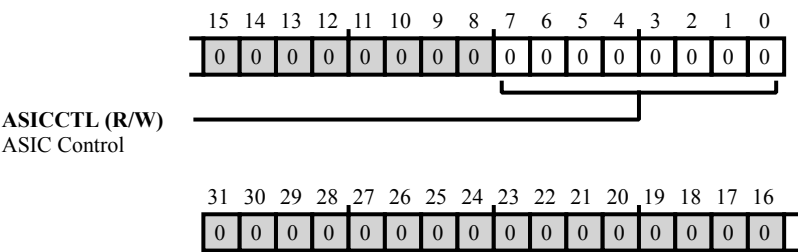


Figure 39-4: CTI_ASICCTL Register Diagram

Table 39-2: CTI_ASICCTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/W)	ASICCTL	ASIC Control. Set and clear external output signal. 0: Clear output bit to 0. 1: Set output bit to 1.

Authentication Status

The `CTI_AUTHSTATUS` register reports the current status of the authentication control signals.

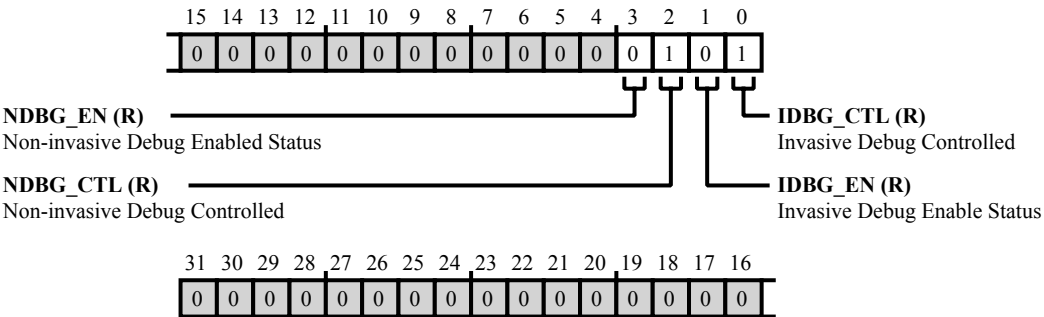


Figure 39-5: CTI_AUTHSTATUS Register Diagram

Table 39-3: CTI_AUTHSTATUS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3 (R/NW)	NDBG_EN	Non-invasive Debug Enabled Status.
2 (R/NW)	NDBG_CTL	Non-invasive Debug Controlled.
1 (R/NW)	IDBG_EN	Invasive Debug Enable Status.
0 (R/NW)	IDBG_CTL	Invasive Debug Controlled.

Claim Tag Clear Register

The `CTI_CLAIMCLR` register forms one half of the claim tag value. On writes, this location enables individual bits to be cleared. On reads, it returns the current claim tag value.

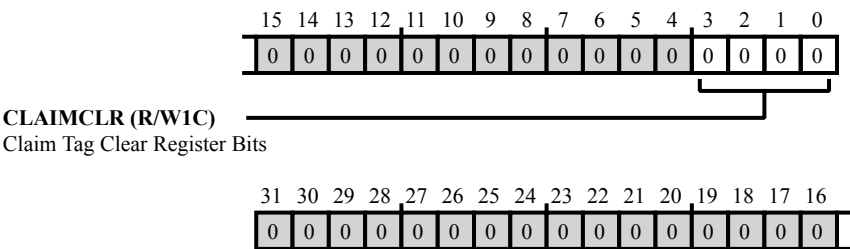


Figure 39-6: CTI_CLAIMCLR Register Diagram

Table 39-4: CTI_CLAIMCLR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W1C)	CLAIMCLR	Claim Tag Clear Register Bits. A bit-programmable register bank that clears the claim tag value. It is zero at reset. It is used by software agents to signal to each other ownership of the hardware. It has no direct effect on the hardware itself.

Claim Tag Set Register

The `CTI_CLAIMSET` register forms one half of the claim tag value. On writes, this location enables individual bits to be set. On reads, it returns the number of bits that can be set.

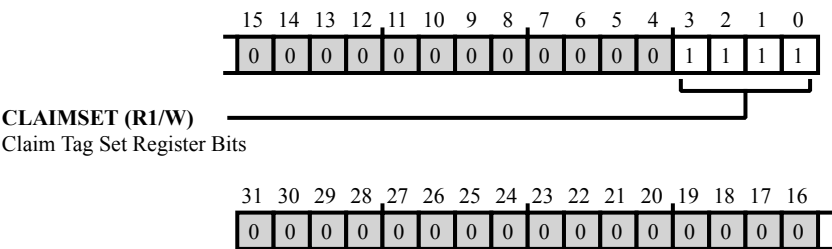


Figure 39-7: CTI_CLAIMSET Register Diagram

Table 39-5: CTI_CLAIMSET Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R1/W)	CLAIMSET	Claim Tag Set Register Bits. A bit-programmable register bank that sets the claim tag value. A read returns a logic 1 for all implemented locations.

Component ID0

The `CTI_COMPID0` register is part of the set of component identification registers.

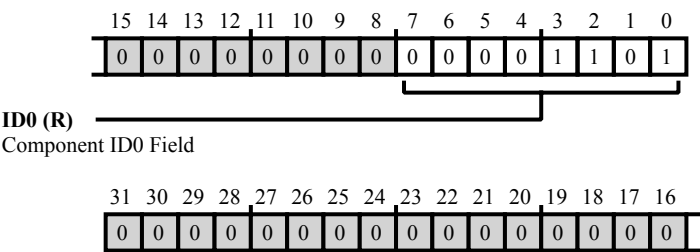


Figure 39-8: CTI_COMPID0 Register Diagram

Table 39-6: CTI_COMPID0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	ID0	Component ID0 Field. Preamble. Returns 0x0D.

Component ID1

The CTI_COMPID1 register is part of the set of component identification registers.

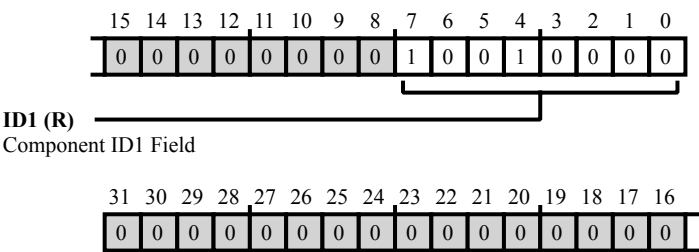


Figure 39-9: CTI_COMPID1 Register Diagram

Table 39-7: CTI_COMPID1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	ID1	Component ID1 Field. Bits[7:4] Component class. Returns 0x9, indicating this is a CoreSight component. Bits[3:0] Preamble. Returns 0x0.

Component ID2

The `CTI_COMPID2` register is part of the set of component identification registers.

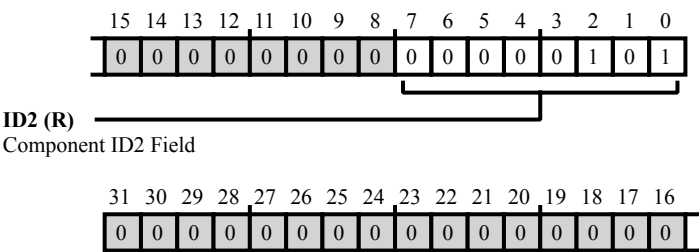


Figure 39-10: CTI_COMPID2 Register Diagram

Table 39-8: CTI_COMPID2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	ID2	Component ID2 Field. Preamble. Returns 0x05.

Component ID3

The `CTI_COMPID3` register is part of the set of component identification registers.

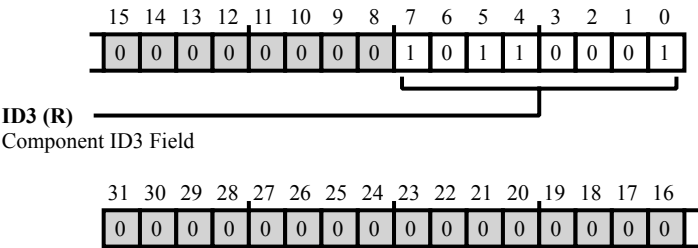


Figure 39-11: `CTI_COMPID3` Register Diagram

Table 39-9: `CTI_COMPID3` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	ID3	Component ID3 Field. Preamble. Returns 0xB1.

CTI Application Trigger Clear Register

The `CTI_CTIAPPCLEAR` register allows software to clear any channel output. Software can use this register to clear a channel event in place of a hardware source on a trigger input. This register must not be used in a system where all events are sent as single-cycle pulses. It is only retained for compatibility with older systems and software. The register is implemented before the CTIGATE register. Therefore, for the channel event to propagate outside the CTI, the corresponding CTIGATE bit must be set to 1.

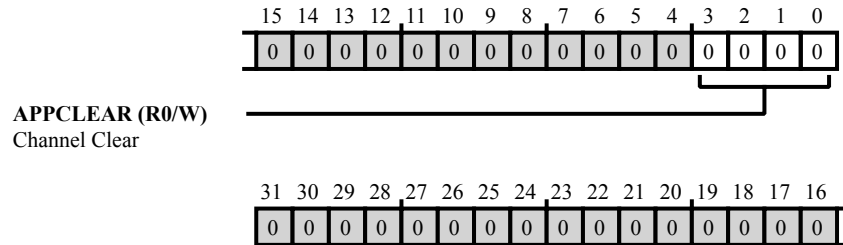


Figure 39-12: CTI_CTIAPPCLEAR Register Diagram

Table 39-10: CTI_CTIAPPCLEAR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R0/W)	APPCLEAR	Channel Clear. The <code>CTI_CTIAPPCLEAR.APPCLEAR</code> bit is for clearing the corresponding internal channel flag. 0: No effect. 1: Clears the channel output.

CTI Application Pulse Register

The `CTI_CTIAPPPULSE` register allows software to pulse any channel output. This register can be used by software to pulse a channel event in place of a hardware source on a trigger input. The register is implemented before the `CTI_CTIGATE` register so, for the channel event to propagate outside the CTI, it is necessary for the corresponding bit to be 1.

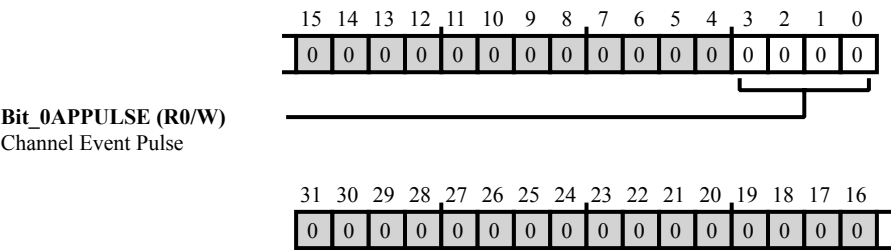


Figure 39-13: CTI_CTIAPPPULSE Register Diagram

Table 39-11: CTI_CTIAPPPULSE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R0/W)	BIT_0APPULSE	Channel Event Pulse. Pulses the channel outputs. 0: No effect. 1: Pulse channel event for one clk cycle.

CTI Application Trigger Set Register

The `CTI_CTIAPPSET` register allows software to set any channel output. Software can use this register to generate a channel event in place of a hardware source on a trigger input. This register must not be used in a system where all events are sent as single-cycle pulses. It is only retained for compatibility with older systems and software. The register is implemented before the `CTI_CTIGATE` register. Therefore, for the channel event to propagate outside the CTI, the corresponding CTIGATE bit must be set to 1.

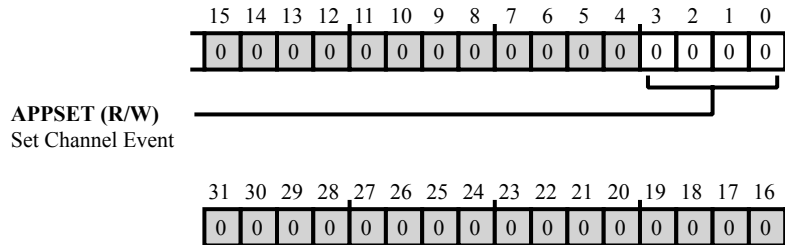


Figure 39-14: CTI_CTIAPPSET Register Diagram

Table 39-12: CTI_CTIAPPSET Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	APPSET	Set Channel Event. Sets the corresponding internal channel flag: 0: Read: application channel is inactive. Write: has no effect. 1: Read: application channel is active. Write: sets the channel output.

CTI Channel In Status Register

If the channel input is driven by a source that generates single cycle pulses, the `CTI_CTICHINSTATUS` register is generally read as 0.

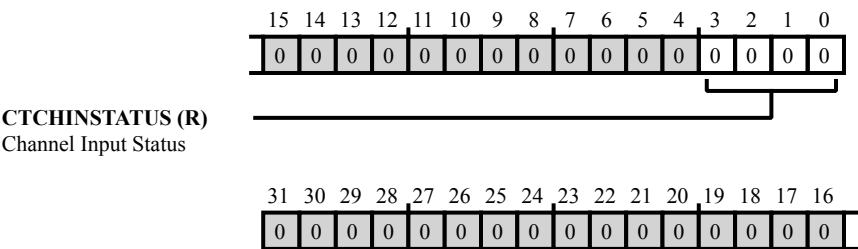


Figure 39-15: CTI_CTICHINSTATUS Register Diagram

Table 39-13: CTI_CTICHINSTATUS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/NW)	CTCHINSTATUS	Channel Input Status. Channel input status. 0: One bit per channel input. 0 means that the input is LOW. 1: One bit per channel input. 1 means that the input is HIGH.

CTI Channel Out Status Register

The `CTI_CTICHOUTSTATUS` register only has meaning if the trigger source drives static levels, rather than pulses.

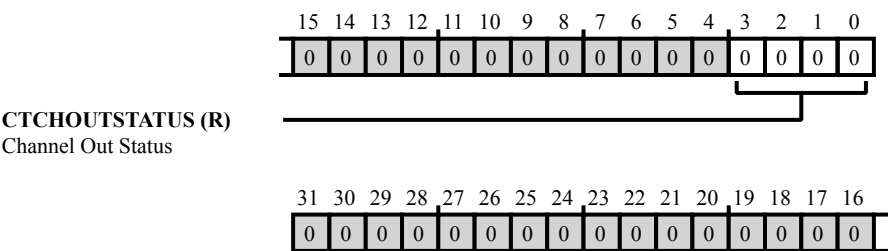


Figure 39-16: CTI_CTICHOUTSTATUS Register Diagram

Table 39-14: CTI_CTICHOUTSTATUS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/NW)	CTCHOUTSTATUS	Channel Out Status. Channel output status. 0: One bit per channel output. 0 means that the output is LOW. 1: One bit per channel output. 1 means that the output is HIGH.

CTI Control Register

The `CTI_CTICONTROL` register enables and disables the CTI.

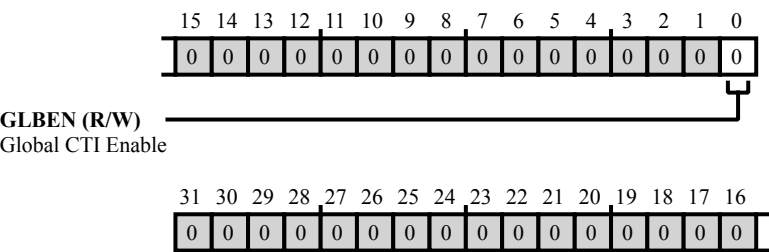


Figure 39-17: CTI_CTICONTROL Register Diagram

Table 39-15: CTI_CTICONTROL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration	
0 (R/W)	GLBEN	Global CTI Enable.	
		0	Disable
		1	Enable

Enable CTI Channel Gate Register

The `CTI_CTIGATE` register enables the propagation of channel events out of the CTI, one bit per channel.

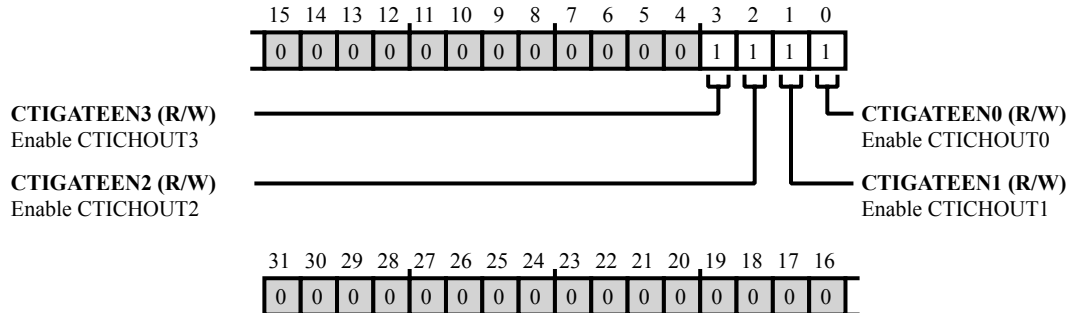


Figure 39-18: CTI_CTIGATE Register Diagram

Table 39-16: CTI_CTIGATE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3 (R/W)	CTIGATEEN3	Enable CTICHOUT3. 0: Disable channel 3 from propagating. 1: Enable channel 3 propagation.
2 (R/W)	CTIGATEEN2	Enable CTICHOUT2. 0: Disable channel 2 from propagating. 1: Enable channel 2 propagation.
1 (R/W)	CTIGATEEN1	Enable CTICHOUT1. 0: Disable channel 1 from propagating. 1: Enable channel 1 propagation.
0 (R/W)	CTIGATEEN0	Enable CTICHOUT0. 0: Disable channel 0 from propagating. 1: Enable channel 0 propagation.

CTI Trigger 0 to Channel Enable Register

The `CTI_CTIINEN0` register maps trigger inputs to channels in the cross trigger system. The CTIINEN registers are bit maps that allow the trigger input to be mapped to any channel output, including none (0x0) and all (0xF). There is one register per trigger input, so it is possible to map any combination of trigger inputs to any channel outputs.

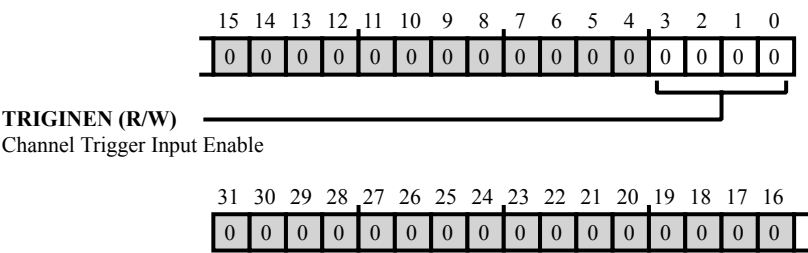


Figure 39-19: CTI_CTIINEN0 Register Diagram

Table 39-17: CTI_CTIINEN0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	TRIGINEN	Channel Trigger Input Enable. Trigger input to channel mapping. 0: Input trigger 0 events are ignored by the corresponding channel. 1: When an event is received on event_in[0], generate an event on the channel corresponding to this bit.

CTI Trigger 1 to Channel Enable Register

The `CTI_CTIINEN1` register maps trigger inputs to channels in the cross trigger system. The CTIINEN registers are bit maps that allow the trigger input to be mapped to any channel output, including none (0x0) and all (0xF). There is one register per trigger input, so it is possible to map any combination of trigger inputs to any channel outputs.

CTIINEN1-31 registers are IMPLEMENTATION-DEFINED. If they are not implemented, the locations are RO and return 0. If they are implemented, they are RW and return a reset value of 0.

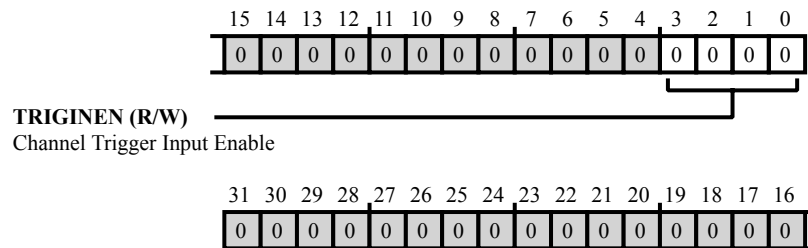


Figure 39-20: CTI_CTIINEN1 Register Diagram

Table 39-18: CTI_CTIINEN1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	TRIGINEN	Channel Trigger Input Enable. Trigger input to channel mapping. 0: Input trigger 1 events are ignored by the corresponding channel. 1: When an event is received on event_in[1], generate an event on the channel corresponding to this bit.

CTI Trigger 2 to Channel Enable Register

The `CTI_CTIINEN2` register maps trigger inputs to channels in the cross trigger system. The CTIINEN registers are bit maps that allow the trigger input to be mapped to any channel output, including none (0x0) and all (0xF). There is one register per trigger input, so it is possible to map any combination of trigger inputs to any channel outputs.

CTIINEN1-31 registers are IMPLEMENTATION-DEFINED. If they are not implemented, the locations are RO and return 0. If they are implemented, they are RW and return a reset value of 0.

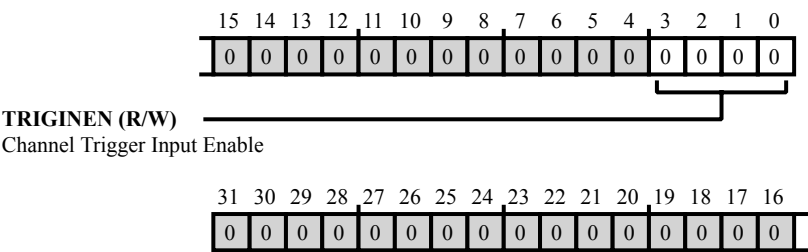


Figure 39-21: CTI_CTIINEN2 Register Diagram

Table 39-19: CTI_CTIINEN2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	TRIGINEN	Channel Trigger Input Enable. Trigger input to channel mapping. 0: Input trigger 2 events are ignored by the corresponding channel. 1: When an event is received on event_in[2], generate an event on the channel corresponding to this bit.

CTI Trigger 3 to Channel Enable Register

The `CTI_CTIINEN3` register maps trigger inputs to channels in the cross trigger system. The CTIINEN registers are bit maps that allow the trigger input to be mapped to any channel output, including none (0x0) and all (0xF). There is one register per trigger input, so it is possible to map any combination of trigger inputs to any channel outputs.

CTIINEN1-31 registers are IMPLEMENTATION-DEFINED. If they are not implemented, the locations are RO and return 0. If they are implemented, they are RW and return a reset value of 0.

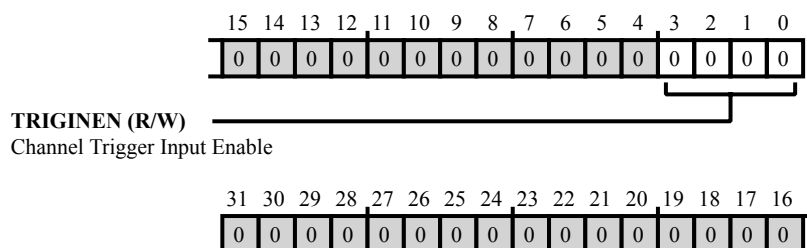


Figure 39-22: CTI_CTIINEN3 Register Diagram

Table 39-20: CTI_CTIINEN3 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	TRIGINEN	Channel Trigger Input Enable. Trigger input to channel mapping. 0: Input trigger 3 events are ignored by the corresponding channel. 1: When an event is received on event_in[3], generate an event on the channel corresponding to this bit.

CTI Trigger 4 to Channel Enable Register

The `CTI_CTIINEN4` register maps trigger inputs to channels in the cross trigger system. The CTIINEN registers are bit maps that allow the trigger input to be mapped to any channel output, including none (0x0) and all (0xF). There is one register per trigger input, so it is possible to map any combination of trigger inputs to any channel outputs.

CTIINEN1-31 registers are IMPLEMENTATION-DEFINED. If they are not implemented, the locations are RO and return 0. If they are implemented, they are RW and return a reset value of 0.

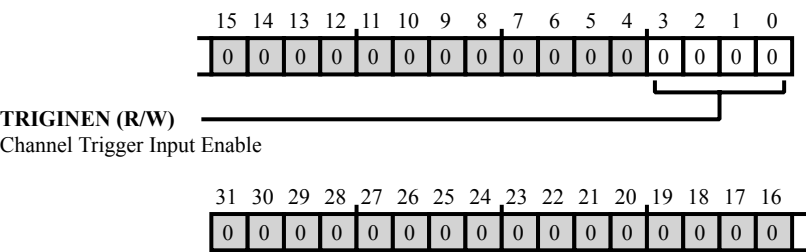


Figure 39-23: CTI_CTIINEN4 Register Diagram

Table 39-21: CTI_CTIINEN4 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	TRIGINEN	Channel Trigger Input Enable. Trigger input to channel mapping. 0: Input trigger 4 events are ignored by the corresponding channel. 1: When an event is received on event_in[4], generate an event on the channel corresponding to this bit.

CTI Trigger 5 to Channel Enable Register

The `CTI_CTIINEN5` register maps trigger inputs to channels in the cross trigger system. The CTIINEN registers are bit maps that allow the trigger input to be mapped to any channel output, including none (0x0) and all (0xF). There is one register per trigger input, so it is possible to map any combination of trigger inputs to any channel outputs.

CTIINEN1-31 registers are IMPLEMENTATION-DEFINED. If they are not implemented, the locations are RO and return 0. If they are implemented, they are RW and return a reset value of 0.

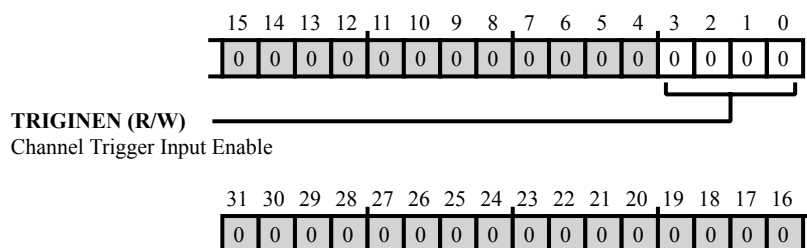


Figure 39-24: CTI_CTIINEN5 Register Diagram

Table 39-22: CTI_CTIINEN5 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	TRIGINEN	Channel Trigger Input Enable. Trigger input to channel mapping. 0: Input trigger 5 events are ignored by the corresponding channel. 1: When an event is received on event_in[5], generate an event on the channel corresponding to this bit.

CTI Trigger 6 to Channel Enable Register

The `CTI_CTIINEN6` register maps trigger inputs to channels in the cross trigger system. The CTIINEN registers are bit maps that allow the trigger input to be mapped to any channel output, including none (0x0) and all (0xF). There is one register per trigger input, so it is possible to map any combination of trigger inputs to any channel outputs.

CTIINEN1-31 registers are IMPLEMENTATION-DEFINED. If they are not implemented, the locations are RO and return 0. If they are implemented, they are RW and return a reset value of 0.

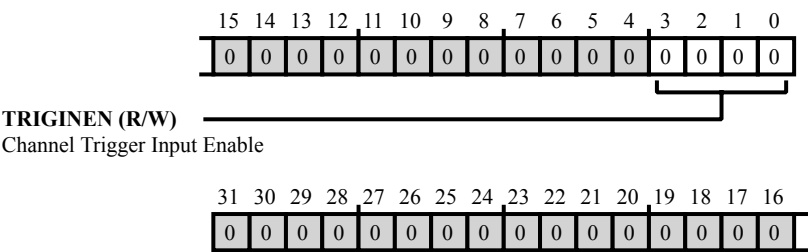


Figure 39-25: CTI_CTIINEN6 Register Diagram

Table 39-23: CTI_CTIINEN6 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	TRIGINEN	Channel Trigger Input Enable. Trigger input to channel mapping. 0: Input trigger 6 events are ignored by the corresponding channel. 1: When an event is received on event_in[6], generate an event on the channel corresponding to this bit.

CTI Trigger 7 to Channel Enable Register

The `CTI_CTIINEN7` register maps trigger inputs to channels in the cross trigger system. The CTIINEN registers are bit maps that allow the trigger input to be mapped to any channel output, including none (0x0) and all (0xF). There is one register per trigger input, so it is possible to map any combination of trigger inputs to any channel outputs.

CTIINEN1-31 registers are IMPLEMENTATION-DEFINED. If they are not implemented, the locations are RO and return 0. If they are implemented, they are RW and return a reset value of 0.

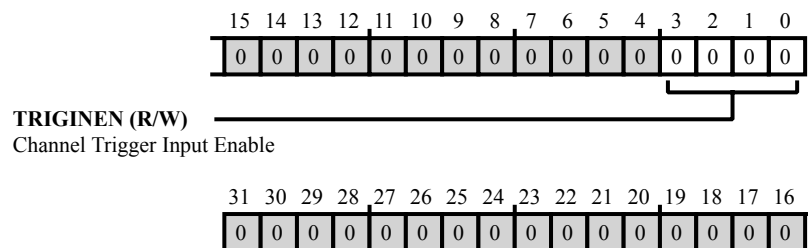


Figure 39-26: CTI_CTIINEN7 Register Diagram

Table 39-24: CTI_CTIINEN7 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	TRIGINEN	Channel Trigger Input Enable. Trigger input to channel mapping. 0: Input trigger 7 events are ignored by the corresponding channel. 1: When an event is received on event_in[7], generate an event on the channel corresponding to this bit.

CTI Interrupt Acknowledge Register

The `CTI_CTIINTACK` register acknowledges trigger outputs.

It is a bit map that allows selective clearing of trigger output events. If the `SW_HANDSHAKE` parameter for a trigger output is set, indicating that the output latches HIGH when an event is sent to that output, then the output remains HIGH until the corresponding INTACK bit is written to a 1. A write of a bit to 0 has no effect. This allows different software threads to be responsible for clearing different trigger outputs without needing to perform a read-modify-write operation to find which bits are set.

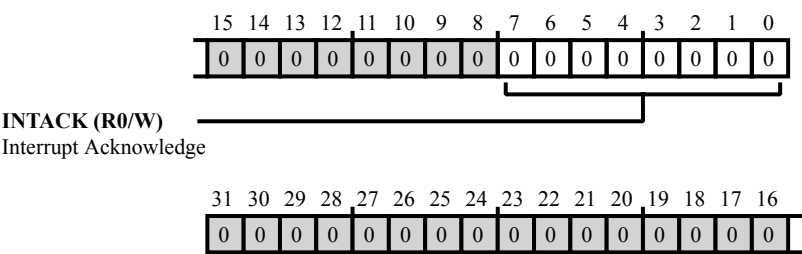


Figure 39-27: CTI_CTIINTACK Register Diagram

Table 39-25: CTI_CTIINTACK Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R0/W)	INTACK	Interrupt Acknowledge. The <code>CTI_CTIINTACK.INTACK</code> bit field acknowledges the corresponding event output. It is write only.

CTI Channel to Trigger 0 Enable Register

The `CTI_CTOUTEN0` register maps channels in the cross trigger system to trigger outputs. The CTIOUTEN registers are bit maps that allow any channel input to be mapped to the trigger output, including none (0x0) and all (0xF). There is one register per trigger output so it is possible to map any channel input to any combination of trigger outputs.

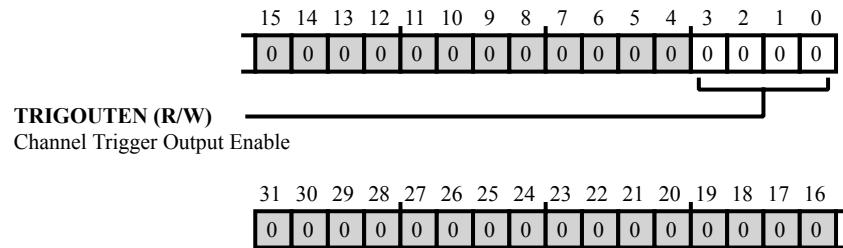


Figure 39-28: CTI_CTOUTEN0 Register Diagram

Table 39-26: CTI_CTOUTEN0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	TRIGOUTEN	Channel Trigger Output Enable. Channel to trigger output mapping. 0: The corresponding channel is ignored by the output trigger0. 1: When an event occurs on the channel corresponding to this bit, generate an event on event_out[0].

CTI Channel to Trigger 1 Enable Register

The `CTI_CTIOUTEN1` register maps channels in the cross trigger system to trigger outputs. The CTIOUTEN registers are bit maps that allow any channel input to be mapped to the trigger output, including none (0x0) and all (0xF). There is one register per trigger output so it is possible to map any channel input to any combination of trigger outputs.

CTIOUTEN1-31 registers are IMPLEMENTATION-DEFINED. If they are not implemented, the locations are RO and return 0. If they are implemented, they are RW and return a reset value of 0.

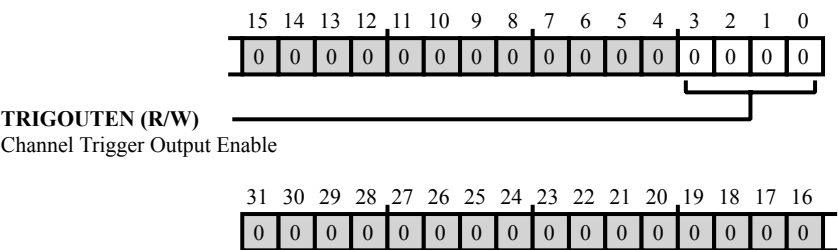


Figure 39-29: CTI_CTIOUTEN1 Register Diagram

Table 39-27: CTI_CTIOUTEN1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	TRIGOUTEN	Channel Trigger Output Enable. Channel to trigger output mapping. 0: The corresponding channel is ignored by the output trigger1. 1: When an event occurs on the channel corresponding to this bit, generate an event on event_out[1].

CTI Channel to Trigger 2 Enable Register

The `CTI_CTIOUTEN2` register maps channels in the cross trigger system to trigger outputs. The CTIOUTEN registers are bit maps that allow any channel input to be mapped to the trigger output, including none (0x0) and all (0xF). There is one register per trigger output so it is possible to map any channel input to any combination of trigger outputs.

CTIOUTEN1-31 registers are IMPLEMENTATION-DEFINED. If they are not implemented, the locations are RO and return 0. If they are implemented, they are RW and return a reset value of 0.

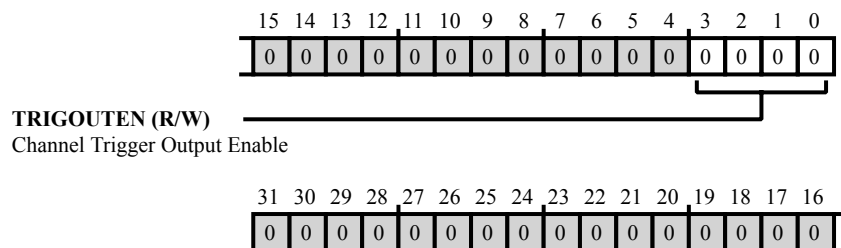


Figure 39-30: CTI_CTIOUTEN2 Register Diagram

Table 39-28: CTI_CTIOUTEN2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	TRIGOUTEN	Channel Trigger Output Enable. Channel to trigger output mapping. 0: The corresponding channel is ignored by the output trigger2. 1: When an event occurs on the channel corresponding to this bit, generate an event on event_out[2].

CTI Channel to Trigger 3 Enable Register

The `CTI_CTIOUTEN3` register maps channels in the cross trigger system to trigger outputs. The CTIOUTEN registers are bit maps that allow any channel input to be mapped to the trigger output, including none (0x0) and all (0xF). There is one register per trigger output so it is possible to map any channel input to any combination of trigger outputs.

CTIOUTEN1-31 registers are IMPLEMENTATION-DEFINED. If they are not implemented, the locations are RO and return 0. If they are implemented, they are RW and return a reset value of 0.

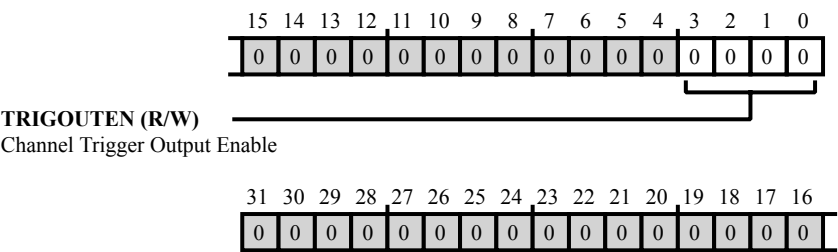


Figure 39-31: CTI_CTIOUTEN3 Register Diagram

Table 39-29: CTI_CTIOUTEN3 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	TRIGOUTEN	Channel Trigger Output Enable. Channel to trigger output mapping. 0: The corresponding channel is ignored by the output trigger3. 1: When an event occurs on the channel corresponding to this bit, generate an event on event_out[3].

CTI Channel to Trigger 4 Enable Register

The `CTI_CTIOUTEN4` register maps channels in the cross trigger system to trigger outputs. The CTIOUTEN registers are bit maps that allow any channel input to be mapped to the trigger output, including none (0x0) and all (0xF). There is one register per trigger output so it is possible to map any channel input to any combination of trigger outputs.

CTIOUTEN1-31 registers are IMPLEMENTATION-DEFINED. If they are not implemented, the locations are RO and return 0. If they are implemented, they are RW and return a reset value of 0.

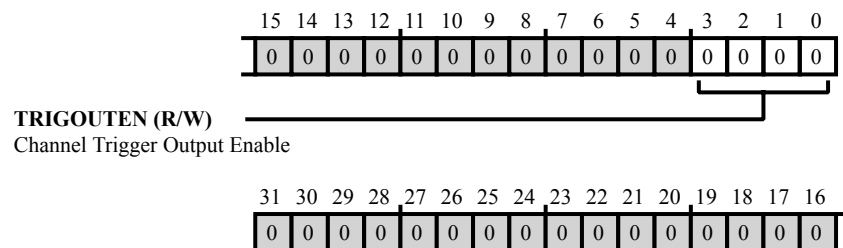


Figure 39-32: CTI_CTIOUTEN4 Register Diagram

Table 39-30: CTI_CTIOUTEN4 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	TRIGOUTEN	Channel Trigger Output Enable. Channel to trigger output mapping. 0: The corresponding channel is ignored by the output trigger4. 1: When an event occurs on the channel corresponding to this bit, generate an event on event_out[4].

CTI Channel to Trigger 5 Enable Register

The `CTI_CTIOUTEN5` register maps channels in the cross trigger system to trigger outputs. The CTIOUTEN registers are bit maps that allow any channel input to be mapped to the trigger output, including none (0x0) and all (0xF). There is one register per trigger output so it is possible to map any channel input to any combination of trigger outputs.

CTIOUTEN1-31 registers are IMPLEMENTATION-DEFINED. If they are not implemented, the locations are RO and return 0. If they are implemented, they are RW and return a reset value of 0.

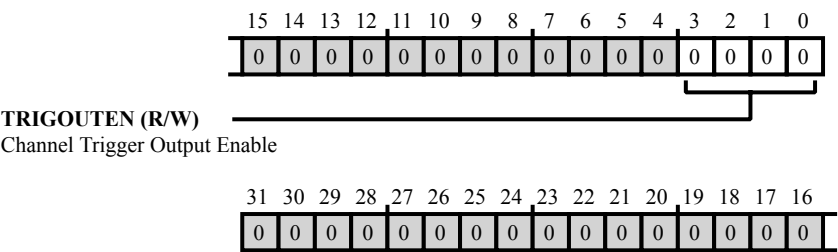


Figure 39-33: CTI_CTIOUTEN5 Register Diagram

Table 39-31: CTI_CTIOUTEN5 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	TRIGOUTEN	Channel Trigger Output Enable. Channel to trigger output mapping. 0: The corresponding channel is ignored by the output trigger5. 1: When an event occurs on the channel corresponding to this bit, generate an event on event_out[5].

CTI Channel to Trigger 6 Enable Register

The `CTI_CTIOUTEN6` register maps channels in the cross trigger system to trigger outputs. The CTIOUTEN registers are bit maps that allow any channel input to be mapped to the trigger output, including none (0x0) and all (0xF). There is one register per trigger output so it is possible to map any channel input to any combination of trigger outputs.

CTIOUTEN1-31 registers are IMPLEMENTATION-DEFINED. If they are not implemented, the locations are RO and return 0. If they are implemented, they are RW and return a reset value of 0.

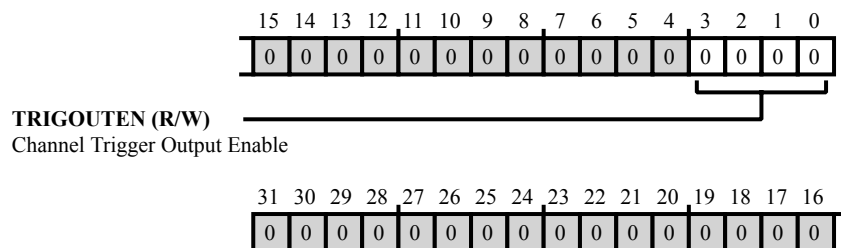


Figure 39-34: CTI_CTIOUTEN6 Register Diagram

Table 39-32: CTI_CTIOUTEN6 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	TRIGOUTEN	Channel Trigger Output Enable. Channel to trigger output mapping. 0: The corresponding channel is ignored by the output trigger6. 1: When an event occurs on the channel corresponding to this bit, generate an event on event_out[6].

CTI Channel to Trigger 7 Enable Register

The `CTI_CTIOUTEN7` register maps channels in the cross trigger system to trigger outputs. The CTIOUTEN registers are bit maps that allow any channel input to be mapped to the trigger output, including none (0x0) and all (0xF). There is one register per trigger output so it is possible to map any channel input to any combination of trigger outputs.

CTIOUTEN1-31 registers are IMPLEMENTATION-DEFINED. If they are not implemented, the locations are RO and return 0. If they are implemented, they are RW and return a reset value of 0.

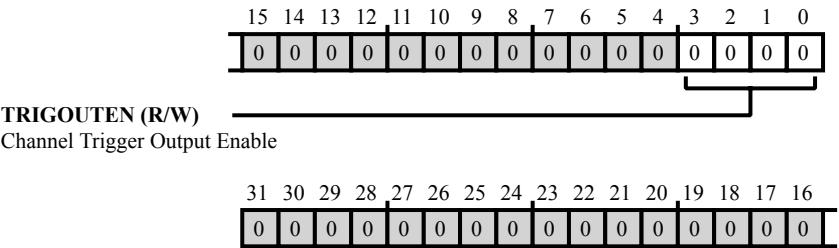


Figure 39-35: CTI_CTIOUTEN7 Register Diagram

Table 39-33: CTI_CTIOUTEN7 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	TRIGOUTEN	Channel Trigger Output Enable. Channel to trigger output mapping. 0: The corresponding channel is ignored by the output trigger7. 1: When an event occurs on the channel corresponding to this bit, generate an event on event_out[7].

CTI Trigger In Status Register

If the event_in input is driven by a source that generates single cycle pulses, the CTI_CTITRIGINSTATUS register is generally read as 0.

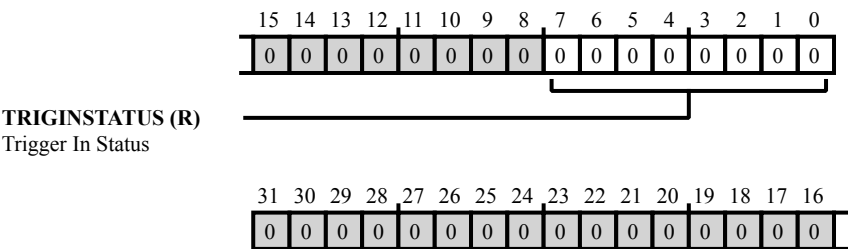


Figure 39-36: CTI_CTITRIGINSTATUS Register Diagram

Table 39-34: CTI_CTITRIGINSTATUS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	TRIGINSTATUS	Trigger In Status. Trigger input status. 0: One bit per trigger input. 0 means that the input is LOW. 1: One bit per trigger input. 1 means that the input is HIGH.

CTI Trigger Out Status Register

The `CTI_CTITRIGOUTSTATUS` register only has meaning if the trigger source drives static levels, rather than pulses.

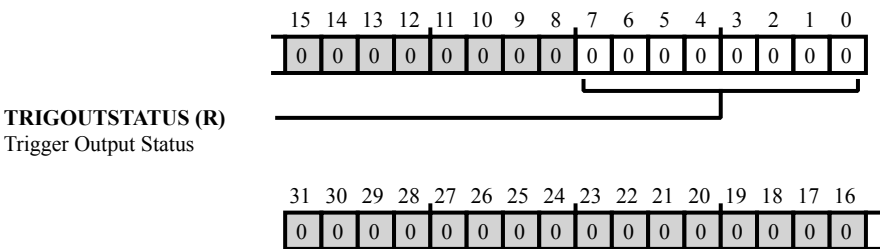


Figure 39-37: CTI_CTITRIGOUTSTATUS Register Diagram

Table 39-35: CTI_CTITRIGOUTSTATUS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	TRIGOUTSTATUS	Trigger Output Status. Trigger output status. 0: One bit per trigger output. 0 means that the output is LOW. 1: One bit per trigger output. 1 means that the output is HIGH.

Device ID

The **CTI_DEVID** register is implementation defined for each part number and designer. The register indicates the capabilities of the component.

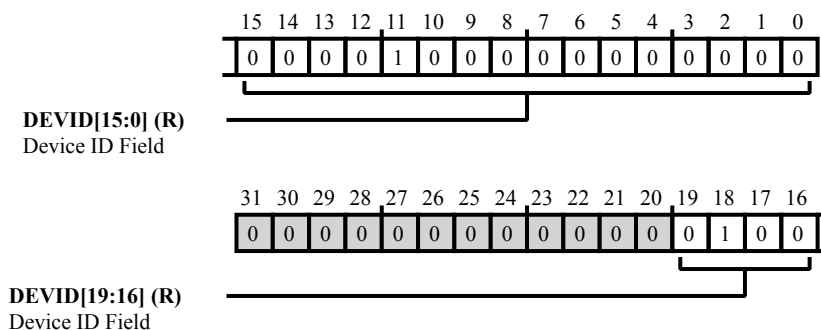


Figure 39-38: CTI_DEVID Register Diagram

Table 39-36: CTI_DEVID Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
19:0 (R/NW)	DEVID	Device ID Field. Bits[19:16] Indicates the maximum number of triggers - the maximum of the two parameters, NUM_EVENT_SLAVES and NUM_EVENT_MASTERS. Bits[15:8] Indicates the maximum number of triggers - the maximum of the two parameters, NUM_EVENT_SLAVES and NUM_EVENT_MASTERS. Bits[7:5] Reserved bit or field with SBZP behavior Bits[4:0] Indicates the value of the EXT_MUX_NUM parameter, which determines if there is any external multiplexing on the trigger inputs and outputs. 0 indicates no multiplexing.

Device Type

A debugger can use the `CTI_DEVTYPE` register to get information about a component that has an unrecognized part number.

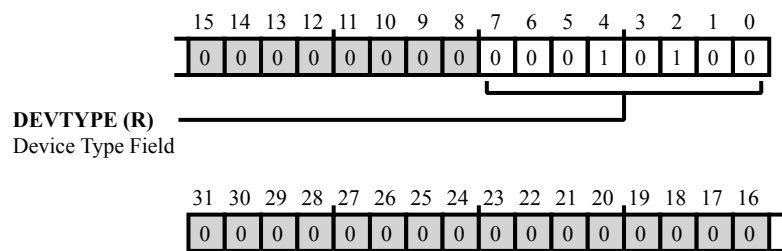


Figure 39-39: CTI_DEVTYPE Register Diagram

Table 39-37: CTI_DEVTYPE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	DEVTYPE	Device Type Field. Bits[7:4] Minor classification. Returns 0x1, indicating this component is a Trigger-Matrix. Bits[3:0] Major classification. Returns 0x4, indicating this component performs De-bug Control.

Integration Test Channel Input

The `CTI_ITCHIN` register is used to view channel events. The integration test register includes a latch that is set when a pulse is received on a channel input. When read, a register bit reads as 1 if the channel has received a pulse since it was last read. The act of reading the register automatically clears the 1 to a 0. When performing integration testing it is therefore important to coordinate the setting of event latches and reading/clearing them.

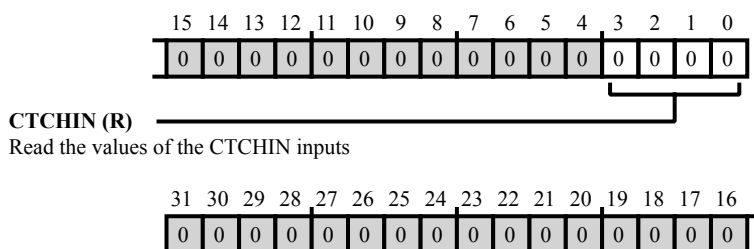


Figure 39-40: CTI_ITCHIN Register Diagram

Table 39-38: CTI_ITCHIN Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/NW)	CTCHIN	Read the values of the CTCHIN inputs. Reads the latched value of the channel inputs.

ITCHINACK

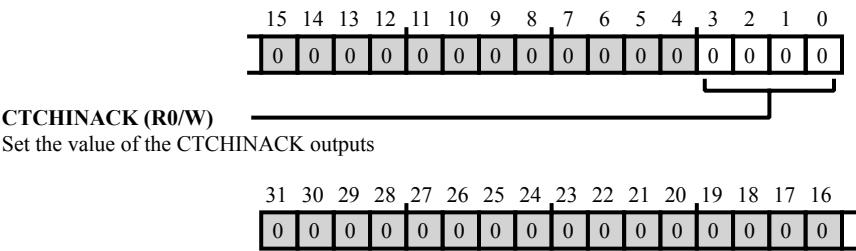


Figure 39-41: CTI_ITCHINACK Register Diagram

Table 39-39: CTI_ITCHINACK Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R0/W)	CTCHINACK	Set the value of the CTCHINACK outputs.

Integration Test Channel Output

The CTI_ITCHOUT register is used to generate channel events. Writing to the register creates a single pulse on the output. The CTI_ITCHOUT register is self-clearing.

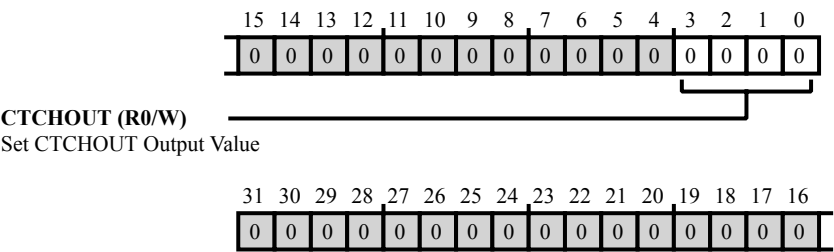


Figure 39-42: CTI_ITCHOUT Register Diagram

Table 39-40: CTI_ITCHOUT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R0/W)	CTCHOUT	Set CTCHOUT Output Value. Pulses the channel outputs. 0: No effect. 1: Pulse channel event for one clk cycle.

ITCHOUTACK

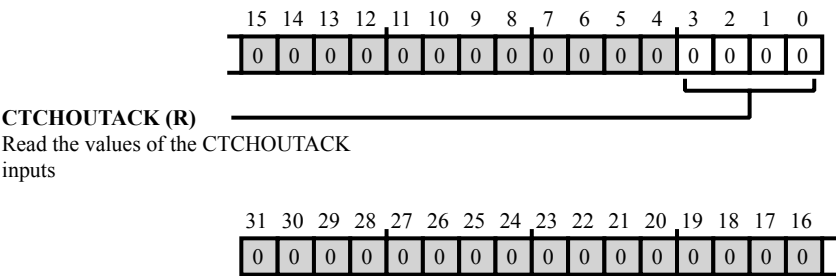


Figure 39-43: CTI_ITCHOUTACK Register Diagram

Table 39-41: CTI_ITCHOUTACK Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/NW)	CTCHOUTACK	Read the values of the CTCHOUTACK inputs.

Integration Mode Control Register

The CTI_ITCTRL register is used to enable topology detection.

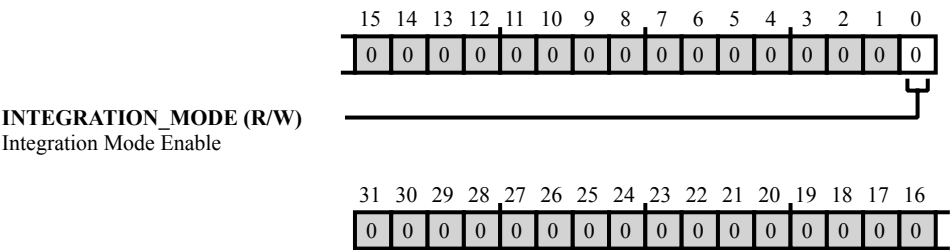


Figure 39-44: CTI_ITCTRL Register Diagram

Table 39-42: CTI_ITCTRL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	INTEGRATION_MODE	Integration Mode Enable. When set, the component enters integration mode, enabling topology detection or in- tegration testing to be performed.

Integration Test Trigger Input

The `CTI_ITTRIGIN` register is used to view trigger events. The integration test register includes a latch that is set when a pulse is received on a trigger input. When read, a register bit reads as 1 if the trigger input has received a pulse since it was last read. The act of reading the register automatically clears the 1 to a 0. When performing integration testing it is therefore important to coordinate the setting of event latches and reading/clearing them.

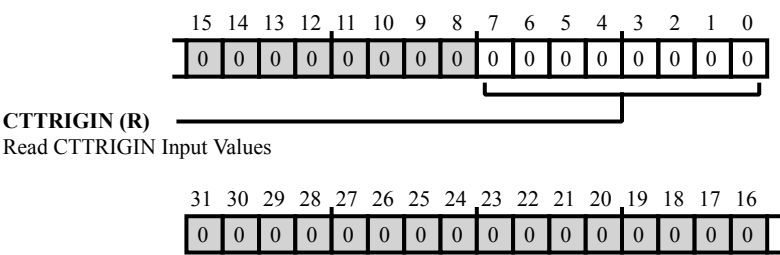


Figure 39-45: CTI_ITTRIGIN Register Diagram

Table 39-43: CTI_ITTRIGIN Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	CTTRIGIN	Read CTTRIGIN Input Values. Reads the latched value of the trigger inputs.

ITTRIGINACK

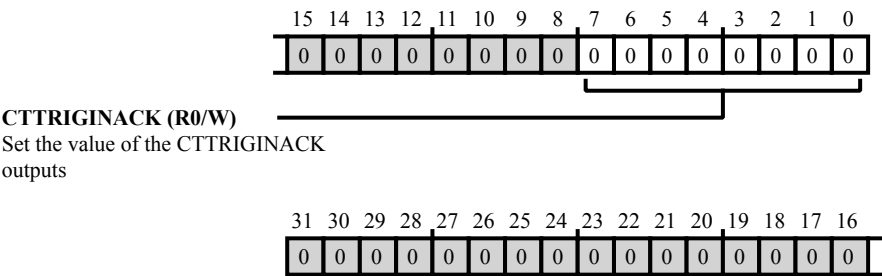


Figure 39-46: CTI_ITTRIGINACK Register Diagram

Table 39-44: CTI_ITTRIGINACK Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R0/W)	CTTRIGINACK	Set the value of the CTTRIGINACK outputs.

Integration Test Trigger Output

The CTI_ITTRIGOUT register is used to generate trigger events.

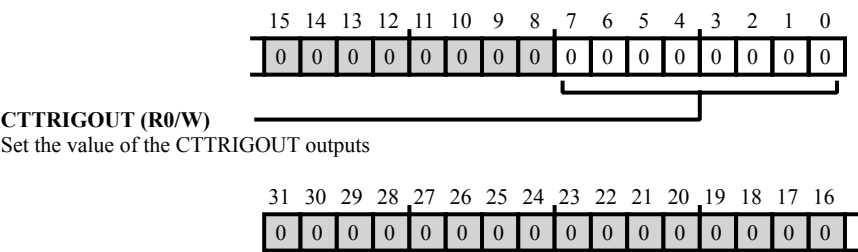


Figure 39-47: CTI_ITTRIGOUT Register Diagram

Table 39-45: CTI_ITTRIGOUT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R0/W)	CTTRIGOUT	Set the value of the CTTRIGOUT outputs. Set/clear trigger output signal. Reads return the value in the register if SW_HANDSHAKE=1, otherwise 0 is returned if SW_HANDSHAKE=0. Writes: 0: Clears the trigger output if SW_HANDSHAKE=1, no effect if SW_HANDSHAKE=0. 1: Sets the trigger output if SW_HANDSHAKE=1, pulses trigger output if SW_HANDSHAKE=0.

ITTRIGOUTACK

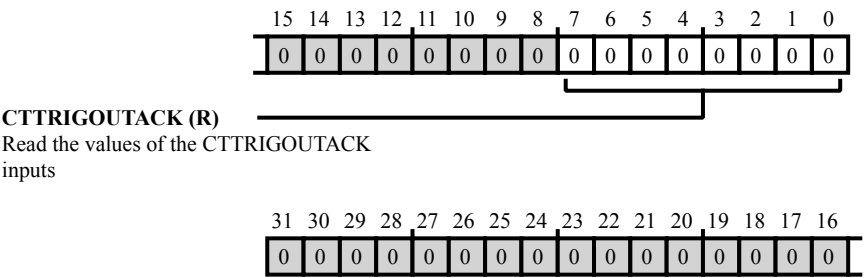


Figure 39-48: CTI_ITTRIGOUTACK Register Diagram

Table 39-46: CTI_ITTRIGOUTACK Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	CTTRIGOUTACK	Read the values of the CTTRIGOUTACK inputs.

Lock Access Register

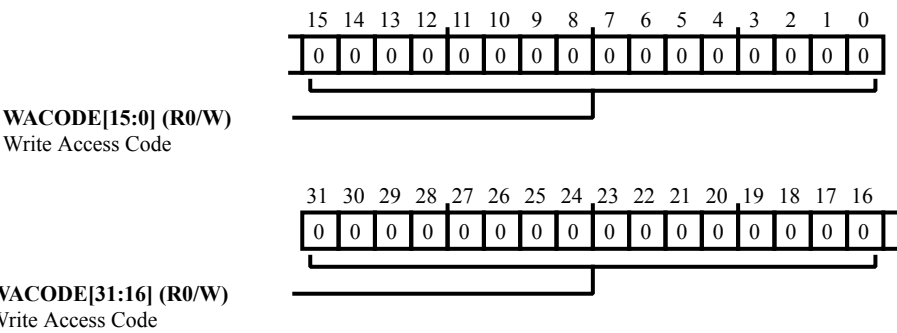


Figure 39-49: CTI_LAR Register Diagram

Table 39-47: CTI_LAR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R0/W)	WACODE	Write Access Code.

Lock Status Register

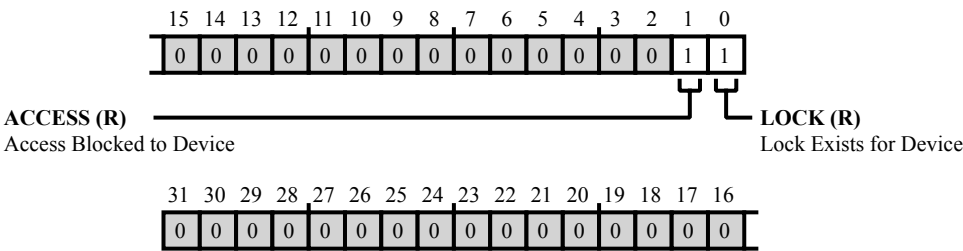


Figure 39-50: CTI_LSR Register Diagram

Table 39-48: CTI_LSR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/NW)	ACCESS	Access Blocked to Device.
0 (R/NW)	LOCK	Lock Exists for Device.

Peripheral ID0

The `CTI_PERIPHID0` register is part of the set of peripheral identification registers.

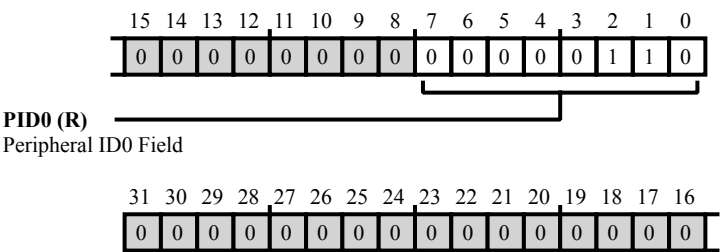


Figure 39-51: `CTI_PERIPHID0` Register Diagram

Table 39-49: `CTI_PERIPHID0` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	PID0	Peripheral ID0 Field. Part number, bits[7:0]. Taken together with PIDR1.PART_1 it indicates the component. The Part Number is selected by the designer of the component.

Peripheral ID1

The CTI_PERIPHID1 register is part of the set of peripheral identification registers.

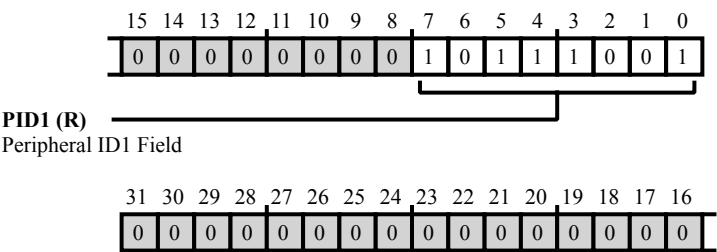


Figure 39-52: CTI_PERIPHID1 Register Diagram

Table 39-50: CTI_PERIPHID1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	PID1	Peripheral ID1 Field. Bits[7:4] JEP106 identification code, bits[3:0]. Together, with PIDR4.DES_2 and PIDR2.DES_1, they indicate the designer of the component and not the implementer, except where the two are the same. Bits[3:0] Part number, bits[11:8]. Taken together with PIDR0.PART_0 it indicates the component. The Part Number is selected by the designer of the component.

Peripheral ID2

The `CTI_PERIPHID2` register is part of the set of peripheral identification registers.

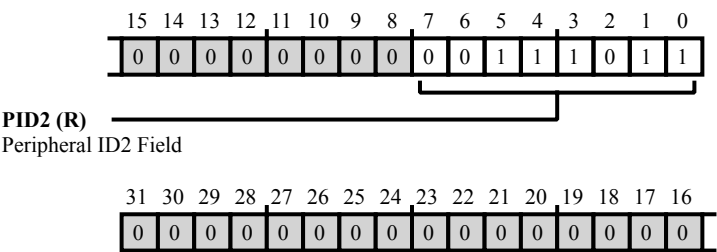


Figure 39-53: `CTI_PERIPHID2` Register Diagram

Table 39-51: `CTI_PERIPHID2` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	PID2	Peripheral ID2 Field. Bits[7:4] Revision. It is an incremental value starting at 0x0 for the first design of a component. See the Component list in Chapter 1 for information on the RTL revision of the component. Bit[3] 1 - Always set. Indicates that a JEDEC assigned value is used. Bits[2:0] JEP106 identification code, bits[6:4]. Together, with PIDR4.DES_2 and PIDR1.DES_0, they indicate the designer of the component and not the implementer, except where the two are the same.

Peripheral ID3

The `CTI_PERIPHID3` register is part of the set of peripheral identification registers.

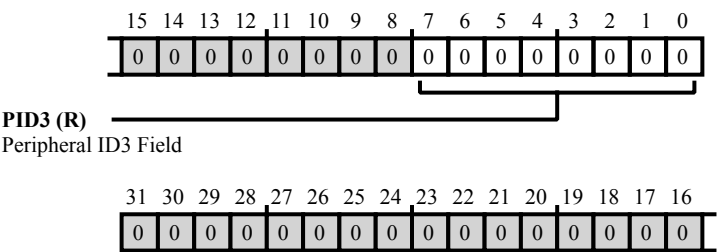


Figure 39-54: CTI_PERIPHID3 Register Diagram

Table 39-52: CTI_PERIPHID3 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	PID3	Peripheral ID3 Field. Bits[7:4] This field indicates minor errata fixes specific to this design, for example metal fixes after implementation. In most cases this field is 0x0. Bits[3:0] Customer Modified. Where the component is reusable IP, this value indicates if the customer has modified the behavior of the component. In most cases this field is 0x0.

Peripheral ID4

The `CTI_PERIPHID4` register is part of the set of peripheral identification registers.

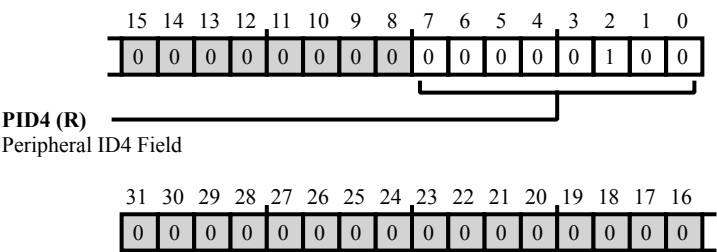


Figure 39-55: CTI_PERIPHID4 Register Diagram

Table 39-53: CTI_PERIPHID4 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	PID4	Peripheral ID4 Field. Bits[7:4] Indicates the memory size that is used by this component. Returns 0 indicating that the component uses an UNKNOWN number of 4KB blocks. Using the SIZE field to indicate the size of the component is deprecated. Bits[3:0] JEP106 continuation code. Together, with PIDR2.DES_1 and PIDR1.DES_0, they indicate the designer of the component and not the implementer, except where the two are the same.

Peripheral ID5

The `CTI_PERIPHID5` register is part of the set of peripheral identification registers.

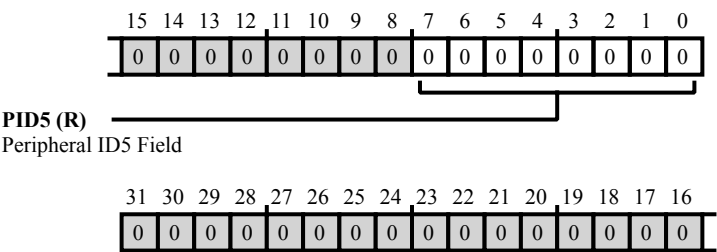


Figure 39-56: CTI_PERIPHID5 Register Diagram

Table 39-54: CTI_PERIPHID5 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	PID5	Peripheral ID5 Field.

Peripheral ID6

The `CTI_PERIPHID6` register is part of the set of peripheral identification registers.

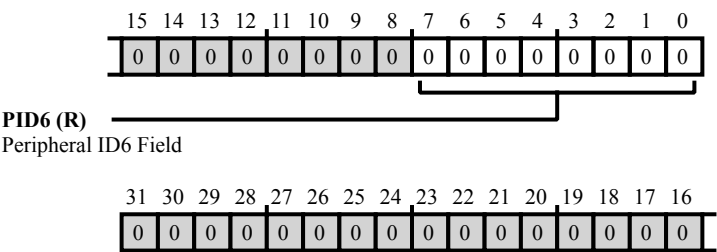


Figure 39-57: CTI_PERIPHID6 Register Diagram

Table 39-55: CTI_PERIPHID6 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	PID6	Peripheral ID6 Field.

Peripheral ID7

The `CTI_PERIPHID7` register is part of the set of peripheral identification registers.

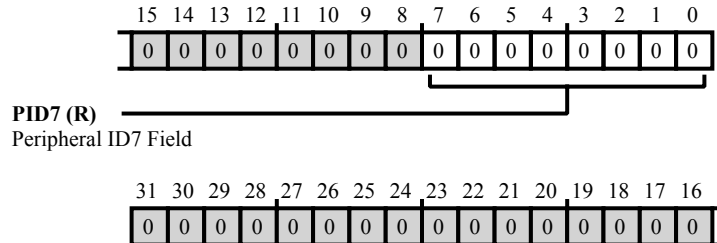


Figure 39-58: `CTI_PERIPHID7` Register Diagram

Table 39-56: `CTI_PERIPHID7` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	PID7	Peripheral ID7 Field.

ADSP-21560_21561_21564_21568 TAPC Register Descriptions

TAPC (TAPC) contains the following registers.

Table 39-57: ADSP-21560_21561_21564_21568 TAPC Register List

Name	Description
<code>TAPC_DBGCTL</code>	Debug Control Register
<code>TAPC_IDCODE</code>	IDCODE Register
<code>TAPC_SDBGKEY0</code>	Secure Debug Key 0 Register
<code>TAPC_SDBGKEY1</code>	Secure Debug Key 1 Register
<code>TAPC_SDBGKEY2</code>	Secure Debug Key 2 Register
<code>TAPC_SDBGKEY3</code>	Secure Debug Key 3 Register
<code>TAPC_SDBGKEY4</code>	Secure Debug Key 4 Register
<code>TAPC_SDBGKEY5</code>	Secure Debug Key 5 Register
<code>TAPC_SDBGKEY6</code>	Secure Debug Key 6 Register
<code>TAPC_SDBGKEY7</code>	Secure Debug Key 7 Register
<code>TAPC_SDBGKEY_CTL</code>	Secure Debug Key Control Register
<code>TAPC_SDBGKEY_STAT</code>	Secure Debug Key Status Register
<code>TAPC_USERCODE</code>	USERCODE Register

Debug Control Register

The `TAPC_DBGCTL` register creates authentication signals to all CoreSight components in the system such as DBGEN, NIDEN, SPNIDEN and SPIDEN.

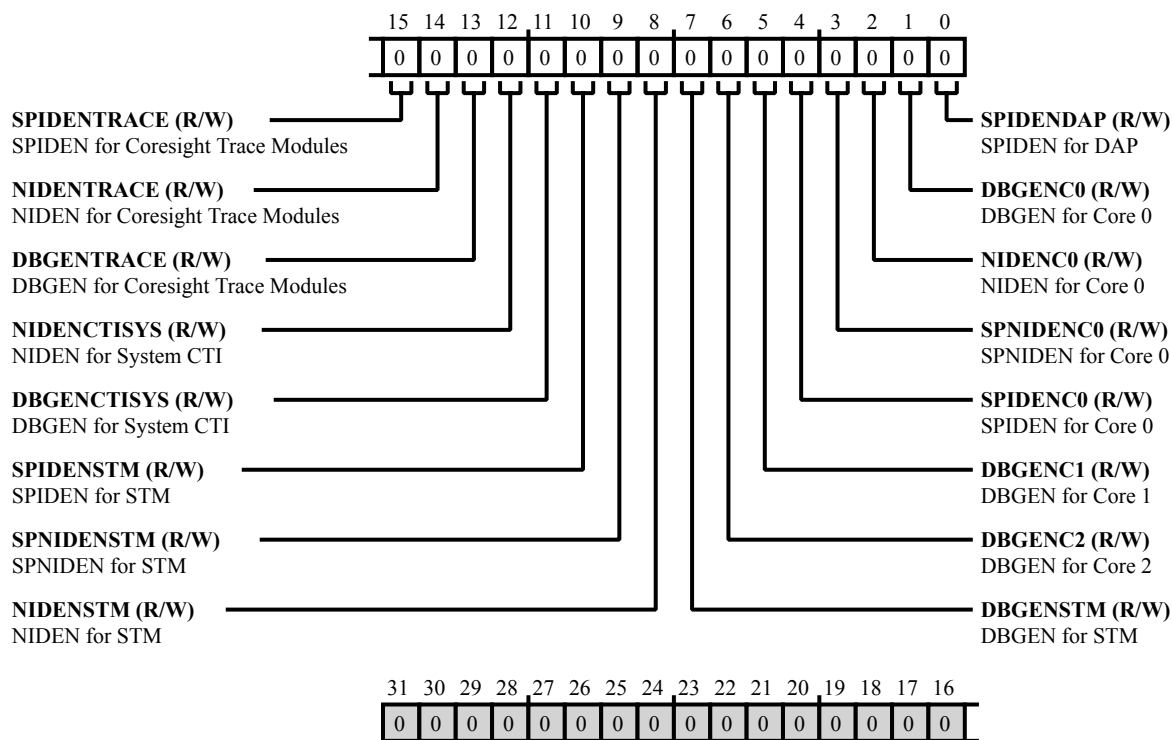


Figure 39-59: TAPC_DBGCTL Register Diagram

Table 39-58: TAPC_DBGCTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W)	SPIDENTRACE	SPIDEN for Coresight Trace Modules.
14 (R/W)	NIDENTRACE	NIDEN for Coresight Trace Modules.
13 (R/W)	DBGENTRACE	DBGEN for Coresight Trace Modules.
12 (R/W)	NIDENCTISYS	NIDEN for System CTI.
11 (R/W)	DBGENCTISYS	DBGEN for System CTI.

Table 39-58: TAPC_DBGCTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
10 (R/W)	SPIDENSTM	SPIDEN for STM.
9 (R/W)	SPNIDENSTM	SPNIDEN for STM.
8 (R/W)	NIDENSTM	NIDEN for STM.
7 (R/W)	DBGENSTM	DBGEN for STM.
6 (R/W)	DBGENC2	DBGEN for Core 2.
5 (R/W)	DBGENC1	DBGEN for Core 1.
4 (R/W)	SPIDENC0	SPIDEN for Core 0.
3 (R/W)	SPNIDENC0	SPNIDEN for Core 0.
2 (R/W)	NIDENC0	NIDEN for Core 0.
1 (R/W)	DBGENC0	DBGEN for Core 0.
0 (R/W)	SPIDENDAP	SPIDEN for DAP.

IDCODE Register

The `TAPC_IDCODE` register holds the IDCODE. The bit field is defined as follows.

IDCODE[31:28] = 0x1* – REVID

IDCODE[27:12] = 0x280B – JTAG ID

IDCODE[11:1] = 0x65 – Manufacturer ID

IDCODE[0] = 0x1

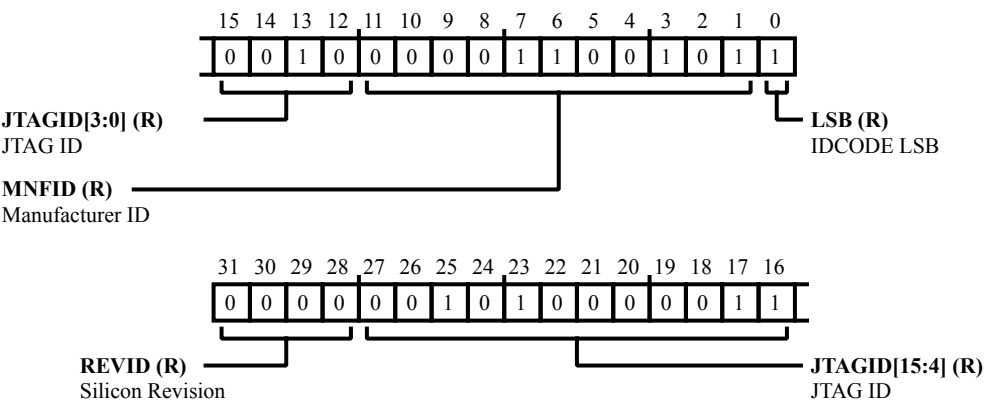


Figure 39-60: TAPC_IDCODE Register Diagram

Table 39-59: TAPC_IDCODE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:28 (R/NW)	REVID	Silicon Revision. The <code>TAPC_IDCODE.REVID</code> bit field holds the silicon revision. See the processor anomaly list for details.
27:12 (R/NW)	JTAGID	JTAG ID.
11:1 (R/NW)	MNFID	Manufacturer ID.
0 (R/NW)	LSB	IDCODE LSB.

Secure Debug Key 0 Register

The `TAPC_SDBGKEY0` register allows a locked part to unlock debug access through the JTAG or SWD interfaces. A debug key of 128 bits needs to be written into the Secure Debug Key registers (`TAPC_SDBGKEY0`, `TAPC_SDBGKEY1`, `TAPC_SDBGKEY2`, `TAPC_SDBGKEY3`) in the TAPC through the peripheral bus interface.

These registers hold the value of the key against which a matching key provided by the debug user is compared to enable a debug session. The task of writing these registers is performed (initially) by boot ROM code which copies a customer-selected key from the Flash memory info block to these registers.

An SDBGKEY value of all 0's is always an invalid key, a value of all 1's match the default value of the Secure Debug Key Compare registers and requires no entry in these registers. It is recommended programs have a significant number of 0's and 1's in a pseudo-random pattern throughout the 128-bit code for maximum protection.

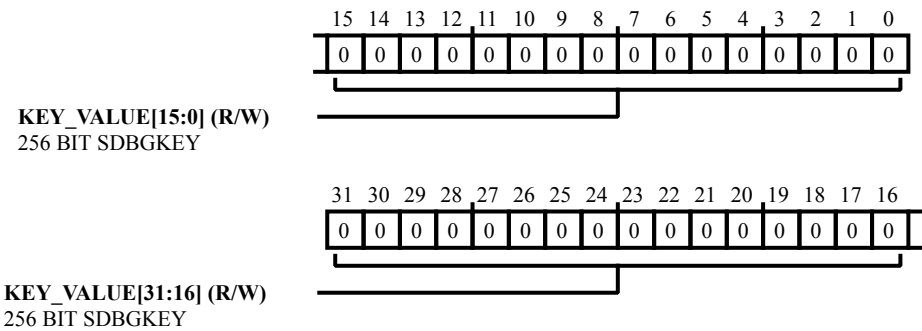


Figure 39-61: TAPC_SDBGKEY0 Register Diagram

Table 39-60: TAPC_SDBGKEY0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	KEY_VALUE	256 BIT SDBGKEY. The <code>TAPC_SDBGKEY0.KEY_VALUE</code> bit field holds the value of the key against which a matching key provided by the debug user is compared to enable a debug session.

Secure Debug Key 1 Register

The [TAPC_SDBGKEY1](#) register allows a locked part to unlock debug access through the JTAG or SWD interfaces. See the [TAPC_SDBGKEY0](#) register description for more information.

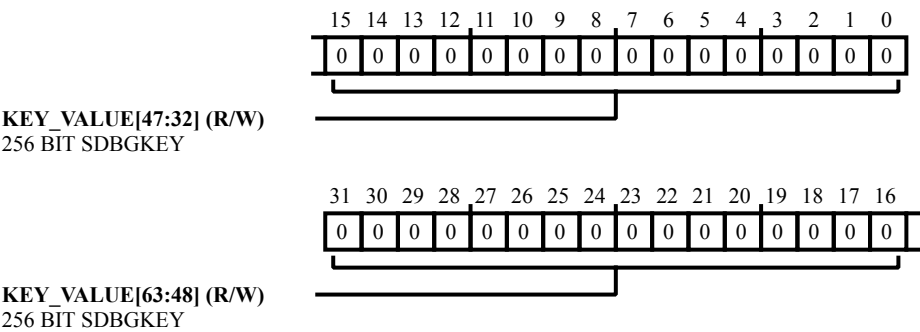


Figure 39-62: TAPC_SDBGKEY1 Register Diagram

Table 39-61: TAPC_SDBGKEY1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	KEY_VALUE	256 BIT SDBGKEY. The TAPC_SDBGKEY1.KEY_VALUE bit field holds the value of the key against which a matching key provided by the debug user is compared to enable a debug session.

Secure Debug Key 2 Register

The `TAPC_SDBGKEY2` register allows a locked part to unlock debug access through the JTAG or SWD interfaces. See the `TAPC_SDBGKEY0` register description for more information.

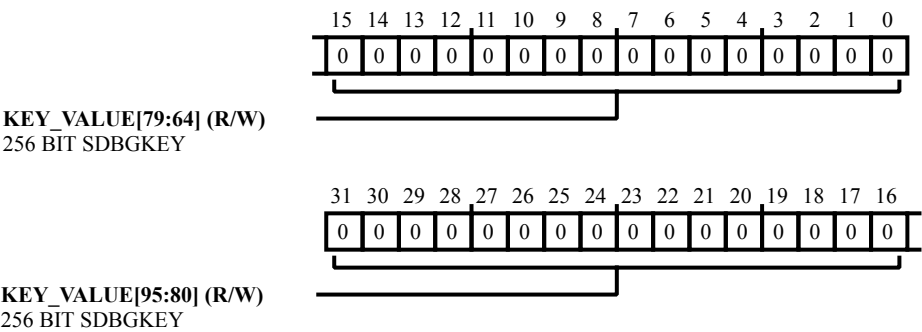


Figure 39-63: TAPC_SDBGKEY2 Register Diagram

Table 39-62: TAPC_SDBGKEY2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	KEY_VALUE	256 BIT SDBGKEY. The <code>TAPC_SDBGKEY2.KEY_VALUE</code> bit field holds the value of the key against which a matching key provided by the debug user is compared to enable a debug session.

Secure Debug Key 3 Register

The [TAPC_SDBGKEY3](#) register allows a locked part to unlock debug access through the JTAG or SWD interfaces. See the [TAPC_SDBGKEY0](#) register description for more information.

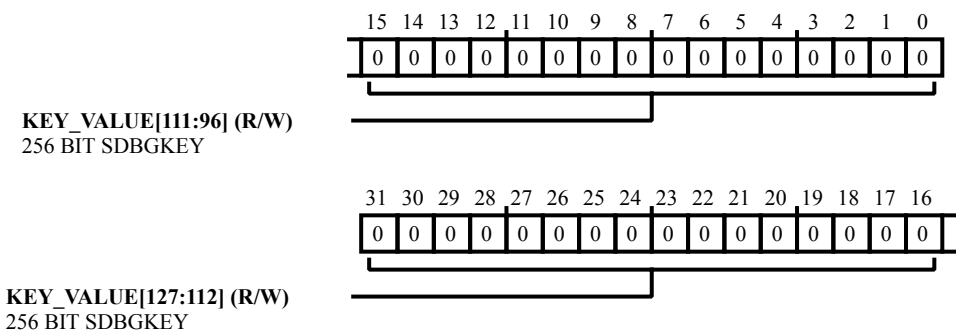


Figure 39-64: TAPC_SDBGKEY3 Register Diagram

Table 39-63: TAPC_SDBGKEY3 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	KEY_VALUE	256 BIT SDBGKEY. The <code>TAPC_SDBGKEY3.KEY_VALUE</code> bit field holds the value of the key against which a matching key provided by the debug user is compared to enable a debug session.

Secure Debug Key 4 Register

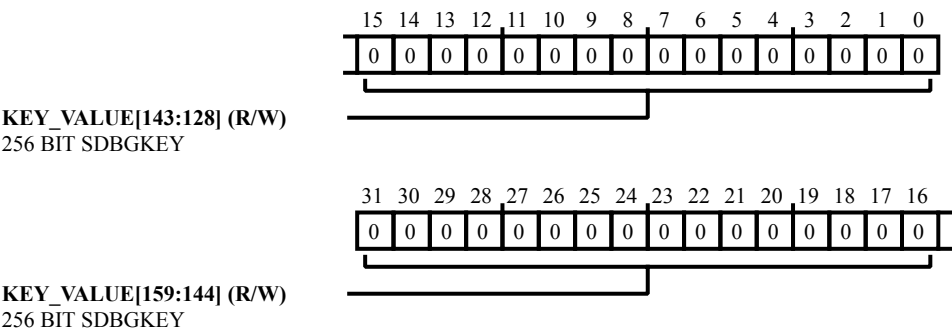


Figure 39-65: TAPC_SDBGKEY4 Register Diagram

Table 39-64: TAPC_SDBGKEY4 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	KEY_VALUE	256 BIT SDBGKEY. The TAPC_SDBGKEY4 . KEY_VALUE bit field holds the value of the key against which a matching key provided by the debug user is compared to enable a debug session.

Secure Debug Key 5 Register

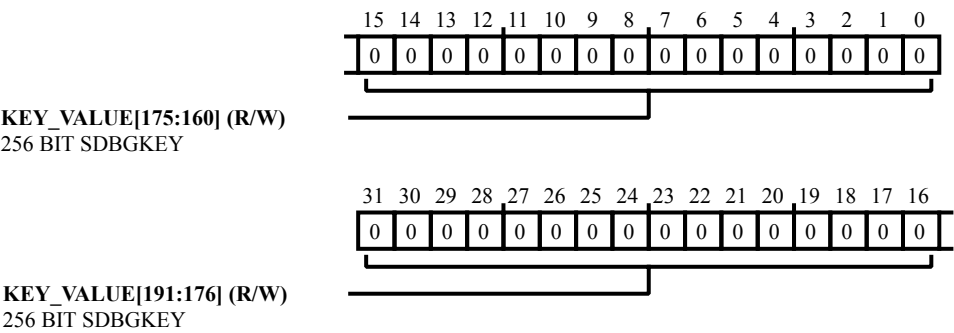


Figure 39-66: TAPC_SDBGKEY5 Register Diagram

Table 39-65: TAPC_SDBGKEY5 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	KEY_VALUE	256 BIT SDBGKEY. The TAPC_SDBGKEY5.KEY_VALUE bit field holds the value of the key against which a matching key provided by the debug user is compared to enable a debug session.

Secure Debug Key 6 Register

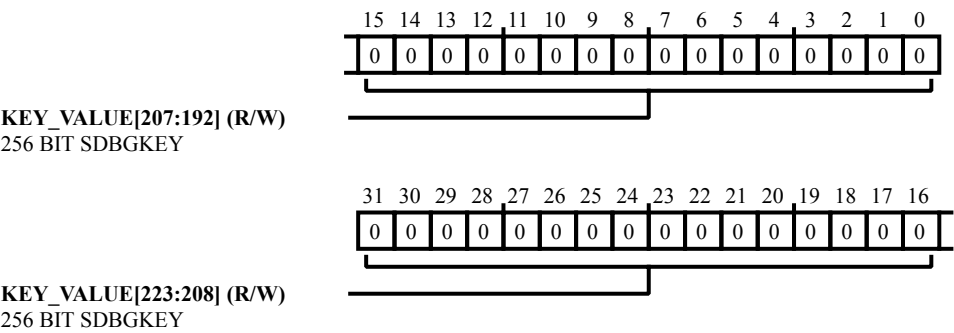


Figure 39-67: TAPC_SDBGKEY6 Register Diagram

Table 39-66: TAPC_SDBGKEY6 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	KEY_VALUE	256 BIT SDBGKEY. The TAPC_SDBGKEY6.KEY_VALUE bit field holds the value of the key against which a matching key provided by the debug user is compared to enable a debug session.

Secure Debug Key 7 Register

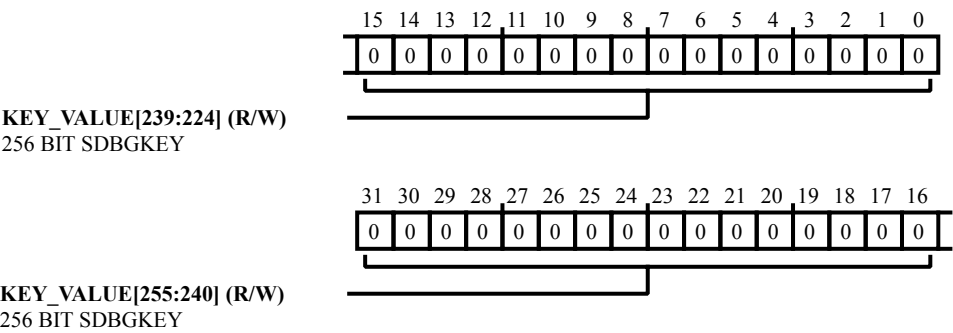


Figure 39-68: TAPC_SDBGKEY7 Register Diagram

Table 39-67: TAPC_SDBGKEY7 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	KEY_VALUE	256 BIT SDBGKEY. The TAPC_SDBGKEY7 .KEY_VALUE bit field holds the value of the key against which a matching key provided by the debug user is compared to enable a debug session.

Secure Debug Key Control Register

The `TAPC_SDBGKEY_CTL` register contains a bit to configure validity for the secure debug keys.

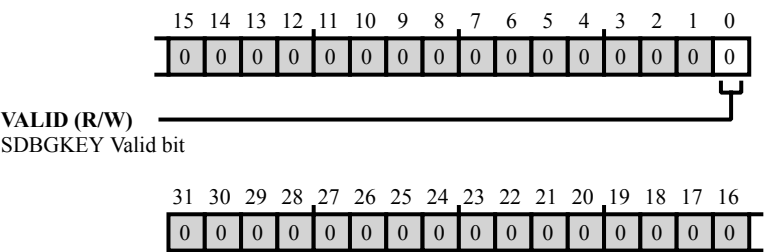


Figure 39-69: TAPC_SDBGKEY_CTL Register Diagram

Table 39-68: TAPC_SDBGKEY_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	VALID	SDBGKEY Valid bit.

Secure Debug Key Status Register

The `TAPC_SDBGKEY_STAT` register contains the PASS/FAIL status of the secure debug key match.

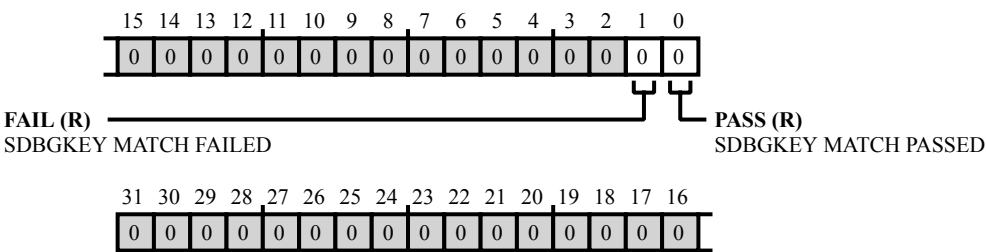


Figure 39-70: TAPC_SDBGKEY_STAT Register Diagram

Table 39-69: TAPC_SDBGKEY_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/NW)	FAIL	SDBGKEY MATCH FAILED.
0 (R/NW)	PASS	SDBGKEY MATCH PASSED.

USERCODE Register

The `TAPC_USERCODE` register stores the 32-bit USERCODE value for the SoC debug unit.

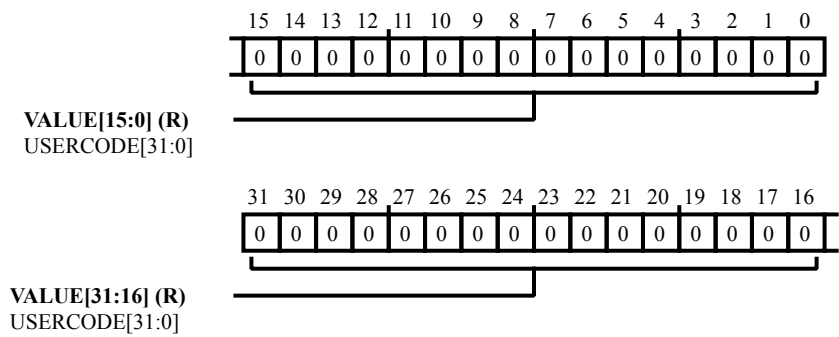


Figure 39-71: TAPC_USERCODE Register Diagram

Table 39-70: TAPC_USERCODE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	USERCODE[31:0].

Appendix A ADSP-21560_21561_21564_21568 Register List

This appendix lists Memory-Mapped Register address and register names. The modules are presented in alphabetical order.

Table A-1: ADSP-21560_21561_21564_21568 ASRC0 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x310C9240	ASRC0_CTL01	ASRC0 Control Register for ASRC 0 and 1	0x00000000
0x310C9244	ASRC0_CTL23	ASRC0 Control Register for ASRC 2 and 3	0x00000000
0x310C9248	ASRC0_MUTE	ASRC0 Mute Register	0x00000000
0x310C9260	ASRC0_RAT01	ASRC0 Ratio Register for ASRC 0 and 1	0x80008000
0x310C9264	ASRC0_RAT23	ASRC0 Ratio Register for ASRC 2 and 3	0x80008000

Table A-2: ADSP-21560_21561_21564_21568 ASRC1 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x310CA240	ASRC1_CTL01	ASRC1 Control Register for ASRC 0 and 1	0x00000000
0x310CA244	ASRC1_CTL23	ASRC1 Control Register for ASRC 2 and 3	0x00000000
0x310CA248	ASRC1_MUTE	ASRC1 Mute Register	0x00000000
0x310CA260	ASRC1_RAT01	ASRC1 Ratio Register for ASRC 0 and 1	0x80008000
0x310CA264	ASRC1_RAT23	ASRC1 Ratio Register for ASRC 2 and 3	0x80008000

Table A-3: ADSP-21560_21561_21564_21568 XSPIDLLPHY MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31073000	XSPIDLLPHY_DATA-SLICE_RFILE_PHY_DQ_TIMING_REG	XSPIDLLPHY	0x00000002
0x31073004	XSPIDLLPHY_DATA-SLICE_RFILE_PHY_DQS_TIMING_REG	XSPIDLLPHY	0x00100000
0x31073008	XSPIDLLPHY_DATA-SLICE_RFILE_PHY_GATE_LPBK_CTRL_REG	XSPIDLLPHY	0x00D80000

Table A-3: ADSP-21560_21561_21564_21568 XSPIDLLPHY MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3107300C	XSPIDLLPHY_DATA- SLICE_RFILE_PHY_DLL_ MASTER_CTRL_REG	XSPIDLLPHY	0x00800000
0x31073010	XSPIDLLPHY_DATA- SLICE_RFILE_PHY_DLL_S LAVE_CTRL_REG	XSPIDLLPHY	0x00000000
0x31073014	XSPIDLLPHY_DATA- SLICE_RFILE_PHY_IE_TI MING_REG	XSPIDLLPHY	0x00100000
0x31073018	XSPIDLLPHY_DATA- SLICE_RFILE_PHY_OBS_R EG_0	XSPIDLLPHY	0x00000000
0x3107301C	XSPIDLLPHY_DATA- SLICE_RFILE_PHY_DLL_O BS_REG_0	XSPIDLLPHY	0x00000000
0x31073020	XSPIDLLPHY_DATA- SLICE_RFILE_PHY_DLL_O BS_REG_1	XSPIDLLPHY	0x00000000
0x31073028	XSPIDLLPHY_DATA- SLICE_RFILE_PHY_STAT- IC_TOGG_REG	XSPIDLLPHY	0x00000000
0x31073034	XSPIDLLPHY_DATA- SLICE_RFILE_PHY_WR_D ESKEW_PD_CTRL_0_REG	XSPIDLLPHY	0x00000000
0x31073070	XSPIDLLPHY_DATA- SLICE_RFILE_PHY_VER- SION_REG	XSPIDLLPHY	0x61820007
0x31073074	XSPIDLLPHY_DATA- SLICE_RFILE_PHY_FEA- TURES_REG	XSPIDLLPHY	0x000019E8
0x31073080	XSPIDLL- PHY_CTB_RFILE_PHY_CT RL_REG	XSPIDLLPHY	0x00000180
0x31073084	XSPIDLL- PHY_CTB_RFILE_PHY_TS EL_REG	XSPIDLLPHY	0x00000000
0x31073088	XSPIDLL- PHY_CTB_RFILE_PHY_GP IO_CTRL_0	XSPIDLLPHY	0x00000000

Table A-3: ADSP-21560_21561_21564_21568 XSPIDLLPHY MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x3107308C	XSPIDLL-PHY_CTB_RFILE_PHY_GP IO_CTRL_1	XSPIDLLPHY	0x00000000
0x31073090	XSPIDLL-PHY_CTB_RFILE_PHY_GP IO_STATUS_0	XSPIDLLPHY	0x00000000
0x31073094	XSPIDLL-PHY_CTB_RFILE_PHY_GP IO_STATUS_1	XSPIDLLPHY	0x00000000

Table A-4: ADSP-21560_21561_21564_21568 CDU0 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x3108F000	CDU0_CFG[n]	CDU0 CDU Configuration	0x00000001
0x3108F004	CDU0_CFG[n]	CDU0 CDU Configuration	0x00000001
0x3108F040	CDU0_STAT	CDU0 CDU Status	0x00000000
0x3108F044	CDU0_CLKINSEL	CDU0 CLKIN Select	0x00000000
0x3108F048	CDU0_REVID	CDU0 CDU Revision ID	0x00000011

Table A-5: ADSP-21560_21561_21564_21568 CGU0 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x3108D000	CGU0_CTL	CGU0 Control Register	0x00002800
0x3108D004	CGU0_PLLCTL	CGU0 PLL Control Register	0x00000000
0x3108D008	CGU0_STAT	CGU0 Status Register	0x0000000F
0x3108D00C	CGU0_DIV	CGU0 Clocks Divisor Register	0x05144281
0x3108D010	CGU0_CLKOUTSEL	CGU0 CLKOUT Select Register	0x00000000
0x3108D02C	CGU0_CCBF_DIS	CGU0 Core Clock Buffer Disable Register	0x00000000
0x3108D030	CGU0_CCBF_STAT	CGU0 Core Clock Buffer Status Register	0x00000000
0x3108D038	CGU0_SCBF_DIS	CGU0 System Clock Buffer Disable Register	0x00000000
0x3108D03C	CGU0_SCBF_STAT	CGU0 System Clock Buffer Status Register	0x00000000
0x3108D040	CGU0_DIVEX	CGU0 DIV Register Extension	0x00200030
0x3108D048	CGU0_REVID	CGU0 Revision ID Register	0x00000030

Table A-6: ADSP-21560_21561_21564_21568 CNT0 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x3100B000	CNT0_CFG	CNT0 Configuration Register	0x00000000
0x3100B004	CNT0_IMSK	CNT0 Interrupt Mask Register	0x00000000
0x3100B008	CNT0_STAT	CNT0 Status Register	0x00000000
0x3100B00C	CNT0_CMD	CNT0 Command Register	0x00000000
0x3100B010	CNT0_DEBNCE	CNT0 Debounce Register	0x00000000
0x3100B014	CNT0_CNTR	CNT0 Counter Register	0x00000000
0x3100B018	CNT0_MAX	CNT0 Maximum Count Register	0x00000000
0x3100B01C	CNT0_MIN	CNT0 Minimum Count Register	0x00000000

Table A-7: ADSP-21560_21561_21564_21568 CRC0 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x310A5000	CRC0_CTL	CRC0 Control Register	0x00000000
0x310A5004	CRC0_DCNT	CRC0 Data Word Count Register	0x00000000
0x310A5008	CRC0_DCNTRLD	CRC0 Data Word Count Reload Register	0x00000000
0x310A5014	CRC0_COMP	CRC0 Data Compare Register	0x00000000
0x310A5018	CRC0_FILLVAL	CRC0 Fill Value Register	0x00000000
0x310A501C	CRC0_DFIFO	CRC0 Data FIFO Register	0x00000000
0x310A5020	CRC0_INEN	CRC0 Interrupt Enable Register	0x00000000
0x310A5024	CRC0_INEN_SET	CRC0 Interrupt Enable Set Register	0x00000000
0x310A5028	CRC0_INEN_CLR	CRC0 Interrupt Enable Clear Register	0x00000000
0x310A502C	CRC0_POLY	CRC0 Polynomial Register	0x00000000
0x310A5040	CRC0_STAT	CRC0 Status Register	0x00000000
0x310A5044	CRC0_DCNTCAP	CRC0 Data Count Capture Register	0x00000000
0x310A504C	CRC0_RESULT_FIN	CRC0 CRC Final Result Register	0x00000000
0x310A5050	CRC0_RESULT_CUR	CRC0 CRC Current Result Register	0x00000000

Table A-8: ADSP-21560_21561_21564_21568 CRC1 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x310A6000	CRC1_CTL	CRC1 Control Register	0x00000000
0x310A6004	CRC1_DCNT	CRC1 Data Word Count Register	0x00000000

Table A-8: ADSP-21560_21561_21564_21568 CRC1 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310A6008	CRC1_DCNTRLD	CRC1 Data Word Count Reload Register	0x00000000
0x310A6014	CRC1_COMP	CRC1 Data Compare Register	0x00000000
0x310A6018	CRC1_FILLVAL	CRC1 Fill Value Register	0x00000000
0x310A601C	CRC1_DFIFO	CRC1 Data FIFO Register	0x00000000
0x310A6020	CRC1_INEN	CRC1 Interrupt Enable Register	0x00000000
0x310A6024	CRC1_INEN_SET	CRC1 Interrupt Enable Set Register	0x00000000
0x310A6028	CRC1_INEN_CLR	CRC1 Interrupt Enable Clear Register	0x00000000
0x310A602C	CRC1_POLY	CRC1 Polynomial Register	0x00000000
0x310A6040	CRC1_STAT	CRC1 Status Register	0x00000000
0x310A6044	CRC1_DCNTCAP	CRC1 Data Count Capture Register	0x00000000
0x310A604C	CRC1_RESULT_FIN	CRC1 CRC Final Result Register	0x00000000
0x310A6050	CRC1_RESULT_CUR	CRC1 CRC Current Result Register	0x00000000

Table A-9: ADSP-21560_21561_21564_21568 CSCTI MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31102000	CSCTI_CTICONTROL	CSCTI CTI Control Register	0x00000000
0x31102000	CTI0_CTICONTROL	CTI0 CTI Control Register	0x00000000
0x31102010	CSCTI_CTIINTACK	CSCTI CTI Interrupt Acknowledge Register	0x00000000
0x31102010	CTI0_CTIINTACK	CTI0 CTI Interrupt Acknowledge Register	0x00000000
0x31102014	CSCTI_CTIAPPSET	CSCTI CTI Application Trigger Set Register	0x00000000
0x31102014	CTI0_CTIAPPSET	CTI0 CTI Application Trigger Set Register	0x00000000
0x31102018	CSCTI_CTIAPPCLEAR	CSCTI CTI Application Trigger Clear Register	0x00000000
0x31102018	CTI0_CTIAPPCLEAR	CTI0 CTI Application Trigger Clear Register	0x00000000
0x3110201C	CSCTI_CTIAPPULSE	CSCTI CTI Application Pulse Register	0x00000000
0x3110201C	CTI0_CTIAPPULSE	CTI0 CTI Application Pulse Register	0x00000000
0x31102020	CSCTI_CTIINEN0	CSCTI CTI Trigger 0 to Channel Enable Register	0x00000000
0x31102020	CTI0_CTIINEN0	CTI0 CTI Trigger 0 to Channel Enable Register	0x00000000
0x31102024	CSCTI_CTIINEN1	CSCTI CTI Trigger 1 to Channel Enable Register	0x00000000
0x31102024	CTI0_CTIINEN1	CTI0 CTI Trigger 1 to Channel Enable Register	0x00000000
0x31102028	CSCTI_CTIINEN2	CSCTI CTI Trigger 2 to Channel Enable Register	0x00000000

Table A-9: ADSP-21560_21561_21564_21568 CSCTI MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31102028	CTI0_CTIINEN2	CTI0 CTI Trigger 2 to Channel Enable Register	0x00000000
0x3110202C	CSCTI_CTIINEN3	CSCTI CTI Trigger 3 to Channel Enable Register	0x00000000
0x3110202C	CTI0_CTIINEN3	CTI0 CTI Trigger 3 to Channel Enable Register	0x00000000
0x31102030	CSCTI_CTIINEN4	CSCTI CTI Trigger 4 to Channel Enable Register	0x00000000
0x31102030	CTI0_CTIINEN4	CTI0 CTI Trigger 4 to Channel Enable Register	0x00000000
0x31102034	CSCTI_CTIINEN5	CSCTI CTI Trigger 5 to Channel Enable Register	0x00000000
0x31102034	CTI0_CTIINEN5	CTI0 CTI Trigger 5 to Channel Enable Register	0x00000000
0x31102038	CSCTI_CTIINEN6	CSCTI CTI Trigger 6 to Channel Enable Register	0x00000000
0x31102038	CTI0_CTIINEN6	CTI0 CTI Trigger 6 to Channel Enable Register	0x00000000
0x3110203C	CSCTI_CTIINEN7	CSCTI CTI Trigger 7 to Channel Enable Register	0x00000000
0x3110203C	CTI0_CTIINEN7	CTI0 CTI Trigger 7 to Channel Enable Register	0x00000000
0x311020A0	CSCTI_CTIOUTEN0	CSCTI CTI Channel to Trigger 0 Enable Register	0x00000000
0x311020A0	CTI0_CTIOUTEN0	CTI0 CTI Channel to Trigger 0 Enable Register	0x00000000
0x311020A4	CSCTI_CTIOUTEN1	CSCTI CTI Channel to Trigger 1 Enable Register	0x00000000
0x311020A4	CTI0_CTIOUTEN1	CTI0 CTI Channel to Trigger 1 Enable Register	0x00000000
0x311020A8	CSCTI_CTIOUTEN2	CSCTI CTI Channel to Trigger 2 Enable Register	0x00000000
0x311020A8	CTI0_CTIOUTEN2	CTI0 CTI Channel to Trigger 2 Enable Register	0x00000000
0x311020AC	CSCTI_CTIOUTEN3	CSCTI CTI Channel to Trigger 3 Enable Register	0x00000000
0x311020AC	CTI0_CTIOUTEN3	CTI0 CTI Channel to Trigger 3 Enable Register	0x00000000
0x311020B0	CSCTI_CTIOUTEN4	CSCTI CTI Channel to Trigger 4 Enable Register	0x00000000
0x311020B0	CTI0_CTIOUTEN4	CTI0 CTI Channel to Trigger 4 Enable Register	0x00000000
0x311020B4	CSCTI_CTIOUTEN5	CSCTI CTI Channel to Trigger 5 Enable Register	0x00000000
0x311020B4	CTI0_CTIOUTEN5	CTI0 CTI Channel to Trigger 5 Enable Register	0x00000000
0x311020B8	CSCTI_CTIOUTEN6	CSCTI CTI Channel to Trigger 6 Enable Register	0x00000000
0x311020B8	CTI0_CTIOUTEN6	CTI0 CTI Channel to Trigger 6 Enable Register	0x00000000
0x311020BC	CSCTI_CTIOUTEN7	CSCTI CTI Channel to Trigger 7 Enable Register	0x00000000
0x311020BC	CTI0_CTIOUTEN7	CTI0 CTI Channel to Trigger 7 Enable Register	0x00000000
0x31102130	CSCTI_CTITRIGINSTA- TUS	CSCTI CTI Trigger In Status Register	0x00000000
0x31102130	CTI0_CTITRIGINSTATUS	CTI0 CTI Trigger In Status Register	0x00000000

Table A-9: ADSP-21560_21561_21564_21568 CSCTI MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31102134	CSCTI_CTITRIGOUTSTA- TUS	CSCTI CTI Trigger Out Status Register	0x00000000
0x31102134	CTI0_CTITRIGOUTSTA- TUS	CTI0 CTI Trigger Out Status Register	0x00000000
0x31102138	CSCTI_CTICHINSTAUS	CSCTI CTI Channel In Status Register	0x00000000
0x31102138	CTI0_CTICHINSTAUS	CTI0 CTI Channel In Status Register	0x00000000
0x3110213C	CSCTI_CTICHOUTSTA- TUS	CSCTI CTI Channel Out Status Register	0x00000000
0x3110213C	CTI0_CTICHOUTSTAUS	CTI0 CTI Channel Out Status Register	0x00000000
0x31102140	CSCTI_CTIGATE	CSCTI Enable CTI Channel Gate Register	0x0000000F
0x31102140	CTI0_CTIGATE	CTI0 Enable CTI Channel Gate Register	0x0000000F
0x31102144	CSCTI_ASICCTL	CSCTI External Multiplexor Control Register	0x00000000
0x31102144	CTI0_ASICCTL	CTI0 External Multiplexor Control Register	0x00000000
0x31102EDC	CSCTI_ITCHINACK	CSCTI ITCHINACK	0x00000000
0x31102EDC	CTI0_ITCHINACK	CTI0 ITCHINACK	0x00000000
0x31102EE0	CSCTI_ITTRIGINACK	CSCTI ITTRIGINACK	0x00000000
0x31102EE0	CTI0_ITTRIGINACK	CTI0 ITTRIGINACK	0x00000000
0x31102EE4	CSCTI_ITCHOUT	CSCTI Integration Test Channel Output	0x00000000
0x31102EE4	CTI0_ITCHOUT	CTI0 Integration Test Channel Output	0x00000000
0x31102EE8	CSCTI_ITTRIGOUT	CSCTI Integration Test Trigger Output	0x00000000
0x31102EE8	CTI0_ITTRIGOUT	CTI0 Integration Test Trigger Output	0x00000000
0x31102EEC	CSCTI_ITCHOUTACK	CSCTI ITCHOUTACK	0x00000000
0x31102EEC	CTI0_ITCHOUTACK	CTI0 ITCHOUTACK	0x00000000
0x31102EF0	CSCTI_ITTRIGOUTACK	CSCTI ITTRIGOUTACK	0x00000000
0x31102EF0	CTI0_ITTRIGOUTACK	CTI0 ITTRIGOUTACK	0x00000000
0x31102EF4	CSCTI_ITCHIN	CSCTI Integration Test Channel Input	0x00000000
0x31102EF4	CTI0_ITCHIN	CTI0 Integration Test Channel Input	0x00000000
0x31102EF8	CSCTI_ITTRIGIN	CSCTI Integration Test Trigger Input	0x00000000
0x31102EF8	CTI0_ITTRIGIN	CTI0 Integration Test Trigger Input	0x00000000
0x31102F00	CSCTI_ITCTRL	CSCTI Integration Mode Control Register	0x00000000
0x31102F00	CTI0_ITCTRL	CTI0 Integration Mode Control Register	0x00000000
0x31102FA0	CSCTI_CLAIMSET	CSCTI Claim Tag Set Register	0x0000000F

Table A-9: ADSP-21560_21561_21564_21568 CSCTI MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31102FA0	CTI0_CLAIMSET	CTI0 Claim Tag Set Register	0x0000000F
0x31102FA4	CSCTI_CLAIMCLR	CSCTI Claim Tag Clear Register	0x00000000
0x31102FA4	CTI0_CLAIMCLR	CTI0 Claim Tag Clear Register	0x00000000
0x31102FB0	CSCTI_LAR	CSCTI Lock Access Register	0x00000000
0x31102FB0	CTI0_LAR	CTI0 Lock Access Register	0x00000000
0x31102FB4	CSCTI_LSR	CSCTI Lock Status Register	0x00000003
0x31102FB4	CTI0_LSR	CTI0 Lock Status Register	0x00000003
0x31102FB8	CSCTI_AUTHSTATUS	CSCTI Authentication Status	0x00000005
0x31102FB8	CTI0_AUTHSTATUS	CTI0 Authentication Status	0x00000005
0x31102FC8	CSCTI_DEVID	CSCTI Device ID	0x00040800
0x31102FC8	CTI0_DEVID	CTI0 Device ID	0x00040800
0x31102FCC	CSCTI_DEVTYPE	CSCTI Device Type	0x00000014
0x31102FCC	CTI0_DEVTYPE	CTI0 Device Type	0x00000014
0x31102FD0	CSCTI_PERIPHID4	CSCTI Peripheral ID4	0x00000004
0x31102FD0	CTI0_PERIPHID4	CTI0 Peripheral ID4	0x00000004
0x31102FD4	CSCTI_PERIPHID5	CSCTI Peripheral ID5	0x00000000
0x31102FD4	CTI0_PERIPHID5	CTI0 Peripheral ID5	0x00000000
0x31102FD8	CSCTI_PERIPHID6	CSCTI Peripheral ID6	0x00000000
0x31102FD8	CTI0_PERIPHID6	CTI0 Peripheral ID6	0x00000000
0x31102FDC	CSCTI_PERIPHID7	CSCTI Peripheral ID7	0x00000000
0x31102FDC	CTI0_PERIPHID7	CTI0 Peripheral ID7	0x00000000
0x31102FE0	CSCTI_PERIPHID0	CSCTI Peripheral ID0	0x00000006
0x31102FE0	CTI0_PERIPHID0	CTI0 Peripheral ID0	0x00000006
0x31102FE4	CSCTI_PERIPHID1	CSCTI Peripheral ID1	0x000000B9
0x31102FE4	CTI0_PERIPHID1	CTI0 Peripheral ID1	0x000000B9
0x31102FE8	CSCTI_PERIPHID2	CSCTI Peripheral ID2	0x0000003B
0x31102FE8	CTI0_PERIPHID2	CTI0 Peripheral ID2	0x0000003B
0x31102FEC	CSCTI_PERIPHID3	CSCTI Peripheral ID3	0x00000000
0x31102FEC	CTI0_PERIPHID3	CTI0 Peripheral ID3	0x00000000
0x31102FF0	CSCTI_COMPID0	CSCTI Component ID0	0x0000000D
0x31102FF0	CTI0_COMPID0	CTI0 Component ID0	0x0000000D

Table A-9: ADSP-21560_21561_21564_21568 CSCTI MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31102FF4	CSCTI_COMPID1	CSCTI Component ID1	0x00000090
0x31102FF4	CTI0_COMPID1	CTI0 Component ID1	0x00000090
0x31102FF8	CSCTI_COMPID2	CSCTI Component ID2	0x00000005
0x31102FF8	CTI0_COMPID2	CTI0 Component ID2	0x00000005
0x31102FFC	CSCTI_COMPID3	CSCTI Component ID3	0x000000B1
0x31102FFC	CTI0_COMPID3	CTI0 Component ID3	0x000000B1

Table A-10: ADSP-21560_21561_21564_21568 CTI1 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x3110D000	CTI1_CTICONTROL	CTI1 CTI Control Register	0x00000000
0x3110D010	CTI1_CTIINTACK	CTI1 CTI Interrupt Acknowledge Register	0x00000000
0x3110D014	CTI1_CTIAPPSET	CTI1 CTI Application Trigger Set Register	0x00000000
0x3110D018	CTI1_CTIAPPCLEAR	CTI1 CTI Application Trigger Clear Register	0x00000000
0x3110D01C	CTI1_CTIAPPPULSE	CTI1 CTI Application Pulse Register	0x00000000
0x3110D020	CTI1_CTIINEN0	CTI1 CTI Trigger 0 to Channel Enable Register	0x00000000
0x3110D024	CTI1_CTIINEN1	CTI1 CTI Trigger 1 to Channel Enable Register	0x00000000
0x3110D028	CTI1_CTIINEN2	CTI1 CTI Trigger 2 to Channel Enable Register	0x00000000
0x3110D02C	CTI1_CTIINEN3	CTI1 CTI Trigger 3 to Channel Enable Register	0x00000000
0x3110D030	CTI1_CTIINEN4	CTI1 CTI Trigger 4 to Channel Enable Register	0x00000000
0x3110D034	CTI1_CTIINEN5	CTI1 CTI Trigger 5 to Channel Enable Register	0x00000000
0x3110D038	CTI1_CTIINEN6	CTI1 CTI Trigger 6 to Channel Enable Register	0x00000000
0x3110D03C	CTI1_CTIINEN7	CTI1 CTI Trigger 7 to Channel Enable Register	0x00000000
0x3110D0A0	CTI1_CTIOUTEN0	CTI1 CTI Channel to Trigger 0 Enable Register	0x00000000
0x3110D0A4	CTI1_CTIOUTEN1	CTI1 CTI Channel to Trigger 1 Enable Register	0x00000000
0x3110D0A8	CTI1_CTIOUTEN2	CTI1 CTI Channel to Trigger 2 Enable Register	0x00000000
0x3110D0AC	CTI1_CTIOUTEN3	CTI1 CTI Channel to Trigger 3 Enable Register	0x00000000
0x3110D0B0	CTI1_CTIOUTEN4	CTI1 CTI Channel to Trigger 4 Enable Register	0x00000000
0x3110D0B4	CTI1_CTIOUTEN5	CTI1 CTI Channel to Trigger 5 Enable Register	0x00000000
0x3110D0B8	CTI1_CTIOUTEN6	CTI1 CTI Channel to Trigger 6 Enable Register	0x00000000
0x3110D0BC	CTI1_CTIOUTEN7	CTI1 CTI Channel to Trigger 7 Enable Register	0x00000000

Table A-10: ADSP-21560_21561_21564_21568 CTI1 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3110D130	CTI1_CTITRIGINSTATUS	CTI1 CTI Trigger In Status Register	0x00000000
0x3110D134	CTI1_CTITRIGOUTSTA- TUS	CTI1 CTI Trigger Out Status Register	0x00000000
0x3110D138	CTI1_CTICHINSTATUS	CTI1 CTI Channel In Status Register	0x00000000
0x3110D13C	CTI1_CTICHOUTSTATUS	CTI1 CTI Channel Out Status Register	0x00000000
0x3110D140	CTI1_CTIGATE	CTI1 Enable CTI Channel Gate Register	0x0000000F
0x3110D144	CTI1_ASICCTL	CTI1 External Multiplexor Control Register	0x00000000
0x3110DEDC	CTI1_ITCHINACK	CTI1 ITCHINACK	0x00000000
0x3110DEE0	CTI1_ITTRIGINACK	CTI1 ITTRIGINACK	0x00000000
0x3110DEE4	CTI1_ITCHOUT	CTI1 Integration Test Channel Output	0x00000000
0x3110DEE8	CTI1_ITTRIGOUT	CTI1 Integration Test Trigger Output	0x00000000
0x3110DEEC	CTI1_ITCHOUTACK	CTI1 ITCHOUTACK	0x00000000
0x3110DEF0	CTI1_ITTRIGOUTACK	CTI1 ITTRIGOUTACK	0x00000000
0x3110DEF4	CTI1_ITCHIN	CTI1 Integration Test Channel Input	0x00000000
0x3110DEF8	CTI1_ITTRIGIN	CTI1 Integration Test Trigger Input	0x00000000
0x3110DF00	CTI1_ITCTRL	CTI1 Integration Mode Control Register	0x00000000
0x3110DFA0	CTI1_CLAIMSET	CTI1 Claim Tag Set Register	0x0000000F
0x3110DFA4	CTI1_CLAIMCLR	CTI1 Claim Tag Clear Register	0x00000000
0x3110DFB0	CTI1_LAR	CTI1 Lock Access Register	0x00000000
0x3110DFB4	CTI1_LSR	CTI1 Lock Status Register	0x00000003
0x3110DFB8	CTI1_AUTHSTATUS	CTI1 Authentication Status	0x00000005
0x3110DFC8	CTI1_DEVID	CTI1 Device ID	0x00040800
0x3110DFCC	CTI1_DEVTYPE	CTI1 Device Type	0x00000014
0x3110DFD0	CTI1_PERIPHID4	CTI1 Peripheral ID4	0x00000004
0x3110DFD4	CTI1_PERIPHID5	CTI1 Peripheral ID5	0x00000000
0x3110DFD8	CTI1_PERIPHID6	CTI1 Peripheral ID6	0x00000000
0x3110DFDC	CTI1_PERIPHID7	CTI1 Peripheral ID7	0x00000000
0x3110DFE0	CTI1_PERIPHID0	CTI1 Peripheral ID0	0x00000006
0x3110DFE4	CTI1_PERIPHID1	CTI1 Peripheral ID1	0x000000B9
0x3110DFE8	CTI1_PERIPHID2	CTI1 Peripheral ID2	0x0000003B
0x3110DFEC	CTI1_PERIPHID3	CTI1 Peripheral ID3	0x00000000

Table A-10: ADSP-21560_21561_21564_21568 CTI1 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3110DFF0	CTI1_COMPID0	CTI1 Component ID0	0x0000000D
0x3110DFF4	CTI1_COMPID1	CTI1 Component ID1	0x00000090
0x3110DFF8	CTI1_COMPID2	CTI1 Component ID2	0x00000005
0x3110DFFC	CTI1_COMPID3	CTI1 Component ID3	0x000000B1

Table A-11: ADSP-21560_21561_21564_21568 DAI0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310C90C0	DAI0_CLK0	DAI0 Clock Routing Control Register 0	0x252630C2
0x310C90C4	DAI0_CLK1	DAI0 Clock Routing Control Register 1	0x3DEF7BDE
0x310C90C8	DAI0_CLK2	DAI0 Clock Routing Control Register 2	0x00007BDE
0x310C90CC	DAI0_CLK3	DAI0 Clock Routing Control Register 3	0x3C000000
0x310C90D0	DAI0_CLK4	DAI0 Clock Routing Control Register 4	0x3DEF03DE
0x310C90D4	DAI0_CLK5	DAI0 Clock Routing Control Register 5	0x000003DE
0x310C9100	DAI0_DAT0	DAI0 Serial Data Routing Control Register 0	0x08144040
0x310C9104	DAI0_DAT1	DAI0 Serial Data Routing Control Register 1	0x0F38B289
0x310C9108	DAI0_DAT2	DAI0 Serial Data Routing Control Register 2	0x00000450
0x310C910C	DAI0_DAT3	DAI0 Serial Data Routing Control Register 3	0x00000000
0x310C9110	DAI0_DAT4	DAI0 Serial Data Routing Control Register 4	0x00000000
0x310C9114	DAI0_DAT5	DAI0 Serial Data Routing Control Register 5	0x00000000
0x310C9118	DAI0_DAT6	DAI0 Serial Data Routing Control Register 6	0x00FBFBE
0x310C9140	DAI0_FS0	DAI0 Frame Sync Routing Control Register 0	0x2736B4E3
0x310C9144	DAI0_FS1	DAI0 Frame Sync Routing Control Register 1	0x3DEF7BDE
0x310C9148	DAI0_FS2	DAI0 Frame Sync Routing Control Register 2	0x00007BDE
0x310C9150	DAI0_FS4	DAI0 Frame Sync Routing Control Register 4	0x000003DE
0x310C9180	DAI0_PIN0	DAI0 Pin Buffer Assignment Register 0	0x04C80A94
0x310C9184	DAI0_PIN1	DAI0 Pin Buffer Assignment Register 1	0x04E84B96
0x310C9188	DAI0_PIN2	DAI0 Pin Buffer Assignment Register 2	0x03668C98
0x310C9190	DAI0_PIN4	DAI0 Pin Buffer Assignment Register 4	0x05694F9E
0x310C91C0	DAI0_MISC0	DAI0 Miscellaneous Control Register 0	0x3DEF7BDE
0x310C91C4	DAI0_MISC1	DAI0 Miscellaneous Control Register 1	0x3DEF7BDE

Table A-11: ADSP-21560_21561_21564_21568 DAI0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310C91E0	DAI0_PBEN0	DAI0 Pin Buffer Enable Register 0	0x0E2482CA
0x310C91E4	DAI0_PBEN1	DAI0 Pin Buffer Enable Register 1	0x1348D30F
0x310C91E8	DAI0_PBEN2	DAI0 Pin Buffer Enable Register 2	0x1A5545D6
0x310C91EC	DAI0_PBEN3	DAI0 Pin Buffer Enable Register 3	0x1D71F79B
0x310C9200	DAI0_IMSK_FE	DAI0 Falling-Edge Interrupt Mask Register	0x00000000
0x310C9204	DAI0_IMSK_RE	DAI0 Rising-Edge Interrupt Mask Register	0x00000000
0x310C9210	DAI0_IMSK_PRI	DAI0 Core Interrupt Priority Assignment Register	0x00000000
0x310C9220	DAI0_IRPTL_H	DAI0 High Priority Interrupt Latch Register	0x00000000
0x310C9224	DAI0_IRPTL_L	DAI0 Low Priority Interrupt Latch Register	0x00000000
0x310C9230	DAI0_IRPTL_HS	DAI0 Shadow High Priority Interrupt Latch Register	0x00000000
0x310C9234	DAI0_IRPTL_LS	DAI0 Shadow Low Priority Interrupt Latch Register	0x00000000
0x310C92E4	DAI0_PIN_STAT	DAI0 Pin Status Register	0x00000000
0x310C92E8	DAI0_GBL_SP_EN	DAI0 Global SPORT Enable Register	0x00000000
0x310C92EC	DAI0_GBL_INT_EN	DAI0 Global SPORT Interrupt Grouping Register	0x00000000

Table A-12: ADSP-21560_21561_21564_21568 DAI1 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310CA0C0	DAI1_CLK0	DAI1 Clock Routing Control Register 0	0x252630C2
0x310CA0C4	DAI1_CLK1	DAI1 Clock Routing Control Register 1	0x3DEF7BDE
0x310CA0C8	DAI1_CLK2	DAI1 Clock Routing Control Register 2	0x00007BDE
0x310CA0CC	DAI1_CLK3	DAI1 Clock Routing Control Register 3	0x3C000000
0x310CA0D0	DAI1_CLK4	DAI1 Clock Routing Control Register 4	0x3DEF03DE
0x310CA0D4	DAI1_CLK5	DAI1 Clock Routing Control Register 5	0x000003DE
0x310CA100	DAI1_DAT0	DAI1 Serial Data Routing Control Register 0	0x08144040
0x310CA104	DAI1_DAT1	DAI1 Serial Data Routing Control Register 1	0x0F38B289
0x310CA108	DAI1_DAT2	DAI1 Serial Data Routing Control Register 2	0x00000450
0x310CA10C	DAI1_DAT3	DAI1 Serial Data Routing Control Register 3	0x00000000
0x310CA110	DAI1_DAT4	DAI1 Serial Data Routing Control Register 4	0x00000000
0x310CA114	DAI1_DAT5	DAI1 Serial Data Routing Control Register 5	0x00000000
0x310CA118	DAI1_DAT6	DAI1 Serial Data Routing Control Register 6	0x00FBFEBE

Table A-12: ADSP-21560_21561_21564_21568 DAI1 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310CA140	DAI1_FS0	DAI1 Frame Sync Routing Control Register 0	0x2736B4E3
0x310CA144	DAI1_FS1	DAI1 Frame Sync Routing Control Register 1	0x3DEF7BDE
0x310CA148	DAI1_FS2	DAI1 Frame Sync Routing Control Register 2	0x00007BDE
0x310CA150	DAI1_FS4	DAI1 Frame Sync Routing Control Register 4	0x000003DE
0x310CA180	DAI1_PIN0	DAI1 Pin Buffer Assignment Register 0	0x04C80A94
0x310CA184	DAI1_PIN1	DAI1 Pin Buffer Assignment Register 1	0x04E84B96
0x310CA188	DAI1_PIN2	DAI1 Pin Buffer Assignment Register 2	0x03668C98
0x310CA190	DAI1_PIN4	DAI1 Pin Buffer Assignment Register 4	0x05694F9E
0x310CA1C0	DAI1_MISC0	DAI1 Miscellaneous Control Register 0	0x3DEF7BDE
0x310CA1C4	DAI1_MISC1	DAI1 Miscellaneous Control Register 1	0x3DEF7BDE
0x310CA1E0	DAI1_PBEN0	DAI1 Pin Buffer Enable Register 0	0x0E2482CA
0x310CA1E4	DAI1_PBEN1	DAI1 Pin Buffer Enable Register 1	0x1348D30F
0x310CA1E8	DAI1_PBEN2	DAI1 Pin Buffer Enable Register 2	0x1A5545D6
0x310CA1EC	DAI1_PBEN3	DAI1 Pin Buffer Enable Register 3	0x1D71F79B
0x310CA200	DAI1_IMSK_FE	DAI1 Falling-Edge Interrupt Mask Register	0x00000000
0x310CA204	DAI1_IMSK_RE	DAI1 Rising-Edge Interrupt Mask Register	0x00000000
0x310CA210	DAI1_IMSK_PRI	DAI1 Core Interrupt Priority Assignment Register	0x00000000
0x310CA220	DAI1_IRPTL_H	DAI1 High Priority Interrupt Latch Register	0x00000000
0x310CA224	DAI1_IRPTL_L	DAI1 Low Priority Interrupt Latch Register	0x00000000
0x310CA230	DAI1_IRPTL_HS	DAI1 Shadow High Priority Interrupt Latch Register	0x00000000
0x310CA234	DAI1_IRPTL_LS	DAI1 Shadow Low Priority Interrupt Latch Register	0x00000000
0x310CA2E4	DAI1_PIN_STAT	DAI1 Pin Status Register	0x00000000
0x310CA2E8	DAI1_GBL_SP_EN	DAI1 Global SPORT Enable Register	0x00000000
0x310CA2EC	DAI1_GBL_INT_EN	DAI1 Global SPORT Interrupt Grouping Register	0x00000000

Table A-13: ADSP-21560_21561_21564_21568 DAPROM0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31100000	DAPROM0_ROMENTRY00	DAPROM0 ROM Entry 00	0x00001003
0x31100004	DAPROM0_ROMENTRY01	DAPROM0 ROM Entry 01	0x00002003
0x31100008	DAPROM0_ROMENTRY02	DAPROM0 ROM Entry 02	0x00003003

Table A-13: ADSP-21560_21561_21564_21568 DAPROM0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3110000C	DAPROM0_ROMENTRY03	DAPROM0 ROM Entry 03	0x00004003
0x31100010	DAPROM0_ROMENTRY04	DAPROM0 ROM Entry 04	0x00005003
0x31100014	DAPROM0_ROMENTRY05	DAPROM0 ROM Entry 05	0x00006003
0x31100018	DAPROM0_ROMENTRY06	DAPROM0 ROM Entry 06	0x00007003
0x3110001C	DAPROM0_ROMENTRY07	DAPROM0 ROM Entry 07	0x00008003
0x31100020	DAPROM0_ROMENTRY08	DAPROM0 ROM Entry 08	0x00009003
0x31100024	DAPROM0_ROMENTRY09	DAPROM0 ROM Entry 09	0x0000A003
0x31100028	DAPROM0_ROMENTRY10	DAPROM0 ROM Entry 10	0x0000B003
0x3110002C	DAPROM0_ROMENTRY11	DAPROM0 ROM Entry 11	0x0000C003
0x31100030	DAPROM0_ROMENTRY12	DAPROM0 ROM Entry 12	0x0000D003
0x31100034	DAPROM0_ROMENTRY13	DAPROM0 ROM Entry 13	0x00010003

Table A-14: ADSP-21560_21561_21564_21568 DDE MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31022000	DMA0_DSCPTR_NXT	DMA0 Pointer to Next Initial Descriptor Register	0x00000000
0x31022004	DMA0_ADDRSTART	DMA0 Start Address of Current Buffer Register	0x00000000
0x31022008	DMA0_CFG	DMA0 Configuration Register	0x00000000
0x3102200C	DMA0_XCNT	DMA0 Inner Loop Count Start Value Register	0x00000000
0x31022010	DMA0_XMOD	DMA0 Inner Loop Address Increment Register	0x00000000
0x31022014	DMA0_YCNT	DMA0 Outer Loop Count Start Value (2D only) Register	0x00000000
0x31022018	DMA0_YMOD	DMA0 Outer Loop Address Increment (2D only) Register	0x00000000
0x31022024	DMA0_DSCPTR_CUR	DMA0 Current Descriptor Pointer Register	0x00000000
0x31022028	DMA0_DSCPTR_PRV	DMA0 Previous Initial Descriptor Pointer Register	0x00000000
0x3102202C	DMA0_ADDR_CUR	DMA0 Current Address Register	0x00000000
0x31022030	DMA0_STAT	DMA0 Status Register	0x00006000
0x31022034	DMA0_XCNT_CUR	DMA0 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x31022038	DMA0_YCNT_CUR	DMA0 Current Row Count (2D only) Register	0x00000000
0x31022040	DMA0_BWLCNT	DMA0 Bandwidth Limit Count Register	0x00000000
0x31022044	DMA0_BWLCNT_CUR	DMA0 Bandwidth Limit Count Current Register	0x00000000
0x31022048	DMA0_BWMCNT	DMA0 Bandwidth Monitor Count Register	0x00000000

Table A-14: ADSP-21560_21561_21564_21568 DDE MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3102204C	DMA0_BWMCNT_CUR	DMA0 Bandwidth Monitor Count Current Register	0x00000000
0x3102D000	DDE_DSCPTR_NXT	DDE Pointer to Next Initial Descriptor Register	0x00000000
0x3102D004	DDE_ADDRSTART	DDE Start Address of Current Buffer Register	0x00000000
0x3102D008	DDE_CFG	DDE Configuration Register	0x00000220
0x3102D00C	DDE_XCNT	DDE Inner Loop Count Start Value Register	0x00000000
0x3102D010	DDE_XMOD	DDE Inner Loop Address Increment Register	0x00000000
0x3102D014	DDE_YCNT	DDE Outer Loop Count Start Value (2D only) Register	0x00000000
0x3102D018	DDE_YMOD	DDE Outer Loop Address Increment (2D only) Register	0x00000000
0x3102D024	DDE_DSCPTR_CUR	DDE Current Descriptor Pointer Register	0x00000000
0x3102D028	DDE_DSCPTR_PRV	DDE Previous Initial Descriptor Pointer Register	0x00000000
0x3102D02C	DDE_ADDR_CUR	DDE Current Address Register	0x00000000
0x3102D030	DDE_STAT	DDE Status Register	0x0000B000
0x3102D034	DDE_XCNT_CUR	DDE Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x3102D038	DDE_YCNT_CUR	DDE Current Row Count (2D only) Register	0x00000000
0x3102D040	DDE_BWLCNT	DDE Bandwidth Limit Count Register	0x00000000
0x3102D044	DDE_BWLCNT_CUR	DDE Bandwidth Limit Count Current Register	0x00000000
0x3102D048	DDE_BWMCNT	DDE Bandwidth Monitor Count Register	0x00000000
0x3102D04C	DDE_BWMCNT_CUR	DDE Bandwidth Monitor Count Current Register	0x00000000

Table A-15: ADSP-21560_21561_21564_21568 DMA1 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31022080	DMA1_DSCPTR_NXT	DMA1 Pointer to Next Initial Descriptor Register	0x00000000
0x31022084	DMA1_ADDRSTART	DMA1 Start Address of Current Buffer Register	0x00000000
0x31022088	DMA1_CFG	DMA1 Configuration Register	0x00000000
0x3102208C	DMA1_XCNT	DMA1 Inner Loop Count Start Value Register	0x00000000
0x31022090	DMA1_XMOD	DMA1 Inner Loop Address Increment Register	0x00000000
0x31022094	DMA1_YCNT	DMA1 Outer Loop Count Start Value (2D only) Register	0x00000000
0x31022098	DMA1_YMOD	DMA1 Outer Loop Address Increment (2D only) Register	0x00000000
0x310220A4	DMA1_DSCPTR_CUR	DMA1 Current Descriptor Pointer Register	0x00000000
0x310220A8	DMA1_DSCPTR_PRV	DMA1 Previous Initial Descriptor Pointer Register	0x00000000

Table A-15: ADSP-21560_21561_21564_21568 DMA1 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310220AC	DMA1_ADDR_CUR	DMA1 Current Address Register	0x00000000
0x310220B0	DMA1_STAT	DMA1 Status Register	0x00006000
0x310220B4	DMA1_XCNT_CUR	DMA1 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x310220B8	DMA1_YCNT_CUR	DMA1 Current Row Count (2D only) Register	0x00000000
0x310220C0	DMA1_BWLCNT	DMA1 Bandwidth Limit Count Register	0x00000000
0x310220C4	DMA1_BWLCNT_CUR	DMA1 Bandwidth Limit Count Current Register	0x00000000
0x310220C8	DMA1_BWMCNT	DMA1 Bandwidth Monitor Count Register	0x00000000
0x310220CC	DMA1_BWMCNT_CUR	DMA1 Bandwidth Monitor Count Current Register	0x00000000

Table A-16: ADSP-21560_21561_21564_21568 DMA10 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31023000	DMA10_DSCPTR_NXT	DMA10 Pointer to Next Initial Descriptor Register	0x00000000
0x31023004	DMA10_ADDRSTART	DMA10 Start Address of Current Buffer Register	0x00000000
0x31023008	DMA10_CFG	DMA10 Configuration Register	0x00000000
0x3102300C	DMA10_XCNT	DMA10 Inner Loop Count Start Value Register	0x00000000
0x31023010	DMA10_XMOD	DMA10 Inner Loop Address Increment Register	0x00000000
0x31023014	DMA10_YCNT	DMA10 Outer Loop Count Start Value (2D only) Register	0x00000000
0x31023018	DMA10_YMOD	DMA10 Outer Loop Address Increment (2D only) Register	0x00000000
0x31023024	DMA10_DSCPTR_CUR	DMA10 Current Descriptor Pointer Register	0x00000000
0x31023028	DMA10_DSCPTR_PRV	DMA10 Previous Initial Descriptor Pointer Register	0x00000000
0x3102302C	DMA10_ADDR_CUR	DMA10 Current Address Register	0x00000000
0x31023030	DMA10_STAT	DMA10 Status Register	0x00006000
0x31023034	DMA10_XCNT_CUR	DMA10 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x31023038	DMA10_YCNT_CUR	DMA10 Current Row Count (2D only) Register	0x00000000
0x31023040	DMA10_BWLCNT	DMA10 Bandwidth Limit Count Register	0x00000000
0x31023044	DMA10_BWLCNT_CUR	DMA10 Bandwidth Limit Count Current Register	0x00000000
0x31023048	DMA10_BWMCNT	DMA10 Bandwidth Monitor Count Register	0x00000000
0x3102304C	DMA10_BWMCNT_CUR	DMA10 Bandwidth Monitor Count Current Register	0x00000000

Table A-17: ADSP-21560_21561_21564_21568 DMA11 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31023080	DMA11_DSCPTR_NXT	DMA11 Pointer to Next Initial Descriptor Register	0x00000000
0x31023084	DMA11_ADDRSTART	DMA11 Start Address of Current Buffer Register	0x00000000
0x31023088	DMA11_CFG	DMA11 Configuration Register	0x00000000
0x3102308C	DMA11_XCNT	DMA11 Inner Loop Count Start Value Register	0x00000000
0x31023090	DMA11_XMOD	DMA11 Inner Loop Address Increment Register	0x00000000
0x31023094	DMA11_YCNT	DMA11 Outer Loop Count Start Value (2D only) Register	0x00000000
0x31023098	DMA11_YMOD	DMA11 Outer Loop Address Increment (2D only) Register	0x00000000
0x310230A4	DMA11_DSCPTR_CUR	DMA11 Current Descriptor Pointer Register	0x00000000
0x310230A8	DMA11_DSCPTR_PRV	DMA11 Previous Initial Descriptor Pointer Register	0x00000000
0x310230AC	DMA11_ADDR_CUR	DMA11 Current Address Register	0x00000000
0x310230B0	DMA11_STAT	DMA11 Status Register	0x00006000
0x310230B4	DMA11_XCNT_CUR	DMA11 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x310230B8	DMA11_YCNT_CUR	DMA11 Current Row Count (2D only) Register	0x00000000
0x310230C0	DMA11_BWLCNT	DMA11 Bandwidth Limit Count Register	0x00000000
0x310230C4	DMA11_BWLCNT_CUR	DMA11 Bandwidth Limit Count Current Register	0x00000000
0x310230C8	DMA11_BWMCNT	DMA11 Bandwidth Monitor Count Register	0x00000000
0x310230CC	DMA11_BWMCNT_CUR	DMA11 Bandwidth Monitor Count Current Register	0x00000000

Table A-18: ADSP-21560_21561_21564_21568 DMA12 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31023100	DMA12_DSCPTR_NXT	DMA12 Pointer to Next Initial Descriptor Register	0x00000000
0x31023104	DMA12_ADDRSTART	DMA12 Start Address of Current Buffer Register	0x00000000
0x31023108	DMA12_CFG	DMA12 Configuration Register	0x00000000
0x3102310C	DMA12_XCNT	DMA12 Inner Loop Count Start Value Register	0x00000000
0x31023110	DMA12_XMOD	DMA12 Inner Loop Address Increment Register	0x00000000
0x31023114	DMA12_YCNT	DMA12 Outer Loop Count Start Value (2D only) Register	0x00000000
0x31023118	DMA12_YMOD	DMA12 Outer Loop Address Increment (2D only) Register	0x00000000

Table A-18: ADSP-21560_21561_21564_21568 DMA12 MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x31023124	DMA12_DSCPTR_CUR	DMA12 Current Descriptor Pointer Register	0x00000000
0x31023128	DMA12_DSCPTR_PRV	DMA12 Previous Initial Descriptor Pointer Register	0x00000000
0x3102312C	DMA12_ADDR_CUR	DMA12 Current Address Register	0x00000000
0x31023130	DMA12_STAT	DMA12 Status Register	0x00006000
0x31023134	DMA12_XCNT_CUR	DMA12 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x31023138	DMA12_YCNT_CUR	DMA12 Current Row Count (2D only) Register	0x00000000
0x31023140	DMA12_BWLCNT	DMA12 Bandwidth Limit Count Register	0x00000000
0x31023144	DMA12_BWLCNT_CUR	DMA12 Bandwidth Limit Count Current Register	0x00000000
0x31023148	DMA12_BWMCNT	DMA12 Bandwidth Monitor Count Register	0x00000000
0x3102314C	DMA12_BWMCNT_CUR	DMA12 Bandwidth Monitor Count Current Register	0x00000000

Table A-19: ADSP-21560_21561_21564_21568 DMA13 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31023180	DMA13_DSCPTR_NXT	DMA13 Pointer to Next Initial Descriptor Register	0x00000000
0x31023184	DMA13_ADDRSTART	DMA13 Start Address of Current Buffer Register	0x00000000
0x31023188	DMA13_CFG	DMA13 Configuration Register	0x00000000
0x3102318C	DMA13_XCNT	DMA13 Inner Loop Count Start Value Register	0x00000000
0x31023190	DMA13_XMOD	DMA13 Inner Loop Address Increment Register	0x00000000
0x31023194	DMA13_YCNT	DMA13 Outer Loop Count Start Value (2D only) Register	0x00000000
0x31023198	DMA13_YMOD	DMA13 Outer Loop Address Increment (2D only) Register	0x00000000
0x310231A4	DMA13_DSCPTR_CUR	DMA13 Current Descriptor Pointer Register	0x00000000
0x310231A8	DMA13_DSCPTR_PRV	DMA13 Previous Initial Descriptor Pointer Register	0x00000000
0x310231AC	DMA13_ADDR_CUR	DMA13 Current Address Register	0x00000000
0x310231B0	DMA13_STAT	DMA13 Status Register	0x00006000
0x310231B4	DMA13_XCNT_CUR	DMA13 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x310231B8	DMA13_YCNT_CUR	DMA13 Current Row Count (2D only) Register	0x00000000
0x310231C0	DMA13_BWLCNT	DMA13 Bandwidth Limit Count Register	0x00000000
0x310231C4	DMA13_BWLCNT_CUR	DMA13 Bandwidth Limit Count Current Register	0x00000000

Table A-19: ADSP-21560_21561_21564_21568 DMA13 MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x310231C8	DMA13_BWMCNT	DMA13 Bandwidth Monitor Count Register	0x00000000
0x310231CC	DMA13_BWMCNT_CUR	DMA13 Bandwidth Monitor Count Current Register	0x00000000

Table A-20: ADSP-21560_21561_21564_21568 DMA14 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31023200	DMA14_DSCPTR_NXT	DMA14 Pointer to Next Initial Descriptor Register	0x00000000
0x31023204	DMA14_ADDRSTART	DMA14 Start Address of Current Buffer Register	0x00000000
0x31023208	DMA14_CFG	DMA14 Configuration Register	0x00000000
0x3102320C	DMA14_XCNT	DMA14 Inner Loop Count Start Value Register	0x00000000
0x31023210	DMA14_XMOD	DMA14 Inner Loop Address Increment Register	0x00000000
0x31023214	DMA14_YCNT	DMA14 Outer Loop Count Start Value (2D only) Register	0x00000000
0x31023218	DMA14_YMOD	DMA14 Outer Loop Address Increment (2D only) Register	0x00000000
0x31023224	DMA14_DSCPTR_CUR	DMA14 Current Descriptor Pointer Register	0x00000000
0x31023228	DMA14_DSCPTR_PRV	DMA14 Previous Initial Descriptor Pointer Register	0x00000000
0x3102322C	DMA14_ADDR_CUR	DMA14 Current Address Register	0x00000000
0x31023230	DMA14_STAT	DMA14 Status Register	0x00006000
0x31023234	DMA14_XCNT_CUR	DMA14 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x31023238	DMA14_YCNT_CUR	DMA14 Current Row Count (2D only) Register	0x00000000
0x31023240	DMA14_BWLCNT	DMA14 Bandwidth Limit Count Register	0x00000000
0x31023244	DMA14_BWLCNT_CUR	DMA14 Bandwidth Limit Count Current Register	0x00000000
0x31023248	DMA14_BWMCNT	DMA14 Bandwidth Monitor Count Register	0x00000000
0x3102324C	DMA14_BWMCNT_CUR	DMA14 Bandwidth Monitor Count Current Register	0x00000000

Table A-21: ADSP-21560_21561_21564_21568 DMA15 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31023280	DMA15_DSCPTR_NXT	DMA15 Pointer to Next Initial Descriptor Register	0x00000000
0x31023284	DMA15_ADDRSTART	DMA15 Start Address of Current Buffer Register	0x00000000
0x31023288	DMA15_CFG	DMA15 Configuration Register	0x00000000

Table A-21: ADSP-21560_21561_21564_21568 DMA15 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3102328C	DMA15_XCNT	DMA15 Inner Loop Count Start Value Register	0x00000000
0x31023290	DMA15_XMOD	DMA15 Inner Loop Address Increment Register	0x00000000
0x31023294	DMA15_YCNT	DMA15 Outer Loop Count Start Value (2D only) Register	0x00000000
0x31023298	DMA15_YMOD	DMA15 Outer Loop Address Increment (2D only) Register	0x00000000
0x310232A4	DMA15_DSCPTR_CUR	DMA15 Current Descriptor Pointer Register	0x00000000
0x310232A8	DMA15_DSCPTR_PRV	DMA15 Previous Initial Descriptor Pointer Register	0x00000000
0x310232AC	DMA15_ADDR_CUR	DMA15 Current Address Register	0x00000000
0x310232B0	DMA15_STAT	DMA15 Status Register	0x00006000
0x310232B4	DMA15_XCNT_CUR	DMA15 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x310232B8	DMA15_YCNT_CUR	DMA15 Current Row Count (2D only) Register	0x00000000
0x310232C0	DMA15_BWLCNT	DMA15 Bandwidth Limit Count Register	0x00000000
0x310232C4	DMA15_BWLCNT_CUR	DMA15 Bandwidth Limit Count Current Register	0x00000000
0x310232C8	DMA15_BWMCNT	DMA15 Bandwidth Monitor Count Register	0x00000000
0x310232CC	DMA15_BWMCNT_CUR	DMA15 Bandwidth Monitor Count Current Register	0x00000000

Table A-22: ADSP-21560_21561_21564_21568 DMA16 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31023300	DMA16_DSCPTR_NXT	DMA16 Pointer to Next Initial Descriptor Register	0x00000000
0x31023304	DMA16_ADDRSTART	DMA16 Start Address of Current Buffer Register	0x00000000
0x31023308	DMA16_CFG	DMA16 Configuration Register	0x00000000
0x3102330C	DMA16_XCNT	DMA16 Inner Loop Count Start Value Register	0x00000000
0x31023310	DMA16_XMOD	DMA16 Inner Loop Address Increment Register	0x00000000
0x31023314	DMA16_YCNT	DMA16 Outer Loop Count Start Value (2D only) Register	0x00000000
0x31023318	DMA16_YMOD	DMA16 Outer Loop Address Increment (2D only) Register	0x00000000
0x31023324	DMA16_DSCPTR_CUR	DMA16 Current Descriptor Pointer Register	0x00000000
0x31023328	DMA16_DSCPTR_PRV	DMA16 Previous Initial Descriptor Pointer Register	0x00000000
0x3102332C	DMA16_ADDR_CUR	DMA16 Current Address Register	0x00000000

Table A-22: ADSP-21560_21561_21564_21568 DMA16 MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x31023330	DMA16_STAT	DMA16 Status Register	0x00006000
0x31023334	DMA16_XCNT_CUR	DMA16 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x31023338	DMA16_YCNT_CUR	DMA16 Current Row Count (2D only) Register	0x00000000
0x31023340	DMA16_BWLCNT	DMA16 Bandwidth Limit Count Register	0x00000000
0x31023344	DMA16_BWLCNT_CUR	DMA16 Bandwidth Limit Count Current Register	0x00000000
0x31023348	DMA16_BWMCNT	DMA16 Bandwidth Monitor Count Register	0x00000000
0x3102334C	DMA16_BWMCNT_CUR	DMA16 Bandwidth Monitor Count Current Register	0x00000000

Table A-23: ADSP-21560_21561_21564_21568 DMA17 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31023380	DMA17_DSCPTR_NXT	DMA17 Pointer to Next Initial Descriptor Register	0x00000000
0x31023384	DMA17_ADDRSTART	DMA17 Start Address of Current Buffer Register	0x00000000
0x31023388	DMA17_CFG	DMA17 Configuration Register	0x00000000
0x3102338C	DMA17_XCNT	DMA17 Inner Loop Count Start Value Register	0x00000000
0x31023390	DMA17_XMOD	DMA17 Inner Loop Address Increment Register	0x00000000
0x31023394	DMA17_YCNT	DMA17 Outer Loop Count Start Value (2D only) Register	0x00000000
0x31023398	DMA17_YMOD	DMA17 Outer Loop Address Increment (2D only) Register	0x00000000
0x310233A4	DMA17_DSCPTR_CUR	DMA17 Current Descriptor Pointer Register	0x00000000
0x310233A8	DMA17_DSCPTR_PRV	DMA17 Previous Initial Descriptor Pointer Register	0x00000000
0x310233AC	DMA17_ADDR_CUR	DMA17 Current Address Register	0x00000000
0x310233B0	DMA17_STAT	DMA17 Status Register	0x00006000
0x310233B4	DMA17_XCNT_CUR	DMA17 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x310233B8	DMA17_YCNT_CUR	DMA17 Current Row Count (2D only) Register	0x00000000
0x310233C0	DMA17_BWLCNT	DMA17 Bandwidth Limit Count Register	0x00000000
0x310233C4	DMA17_BWLCNT_CUR	DMA17 Bandwidth Limit Count Current Register	0x00000000
0x310233C8	DMA17_BWMCNT	DMA17 Bandwidth Monitor Count Register	0x00000000
0x310233CC	DMA17_BWMCNT_CUR	DMA17 Bandwidth Monitor Count Current Register	0x00000000

Table A-24: ADSP-21560_21561_21564_21568 DMA18 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x310A7100	DMA18_DSCPTR_NXT	DMA18 Pointer to Next Initial Descriptor Register	0x00000000
0x310A7104	DMA18_ADDRSTART	DMA18 Start Address of Current Buffer Register	0x00000000
0x310A7108	DMA18_CFG	DMA18 Configuration Register	0x00000000
0x310A710C	DMA18_XCNT	DMA18 Inner Loop Count Start Value Register	0x00000000
0x310A7110	DMA18_XMOD	DMA18 Inner Loop Address Increment Register	0x00000000
0x310A7114	DMA18_YCNT	DMA18 Outer Loop Count Start Value (2D only) Register	0x00000000
0x310A7118	DMA18_YMOD	DMA18 Outer Loop Address Increment (2D only) Register	0x00000000
0x310A7124	DMA18_DSCPTR_CUR	DMA18 Current Descriptor Pointer Register	0x00000000
0x310A7128	DMA18_DSCPTR_PRV	DMA18 Previous Initial Descriptor Pointer Register	0x00000000
0x310A712C	DMA18_ADDR_CUR	DMA18 Current Address Register	0x00000000
0x310A7130	DMA18_STAT	DMA18 Status Register	0x00006000
0x310A7134	DMA18_XCNT_CUR	DMA18 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x310A7138	DMA18_YCNT_CUR	DMA18 Current Row Count (2D only) Register	0x00000000
0x310A7140	DMA18_BWLCNT	DMA18 Bandwidth Limit Count Register	0x00000000
0x310A7144	DMA18_BWLCNT_CUR	DMA18 Bandwidth Limit Count Current Register	0x00000000
0x310A7148	DMA18_BWMCNT	DMA18 Bandwidth Monitor Count Register	0x00000000
0x310A714C	DMA18_BWMCNT_CUR	DMA18 Bandwidth Monitor Count Current Register	0x00000000

Table A-25: ADSP-21560_21561_21564_21568 DMA19 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x310A7180	DMA19_DSCPTR_NXT	DMA19 Pointer to Next Initial Descriptor Register	0x00000000
0x310A7184	DMA19_ADDRSTART	DMA19 Start Address of Current Buffer Register	0x00000000
0x310A7188	DMA19_CFG	DMA19 Configuration Register	0x00000000
0x310A718C	DMA19_XCNT	DMA19 Inner Loop Count Start Value Register	0x00000000
0x310A7190	DMA19_XMOD	DMA19 Inner Loop Address Increment Register	0x00000000
0x310A7194	DMA19_YCNT	DMA19 Outer Loop Count Start Value (2D only) Register	0x00000000
0x310A7198	DMA19_YMOD	DMA19 Outer Loop Address Increment (2D only) Register	0x00000000

Table A-25: ADSP-21560_21561_21564_21568 DMA19 MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x310A71A4	DMA19_DSCPTR_CUR	DMA19 Current Descriptor Pointer Register	0x00000000
0x310A71A8	DMA19_DSCPTR_PRV	DMA19 Previous Initial Descriptor Pointer Register	0x00000000
0x310A71AC	DMA19_ADDR_CUR	DMA19 Current Address Register	0x00000000
0x310A71B0	DMA19_STAT	DMA19 Status Register	0x00006000
0x310A71B4	DMA19_XCNT_CUR	DMA19 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x310A71B8	DMA19_YCNT_CUR	DMA19 Current Row Count (2D only) Register	0x00000000
0x310A71C0	DMA19_BWLCNT	DMA19 Bandwidth Limit Count Register	0x00000000
0x310A71C4	DMA19_BWLCNT_CUR	DMA19 Bandwidth Limit Count Current Register	0x00000000
0x310A71C8	DMA19_BWMCNT	DMA19 Bandwidth Monitor Count Register	0x00000000
0x310A71CC	DMA19_BWMCNT_CUR	DMA19 Bandwidth Monitor Count Current Register	0x00000000

Table A-26: ADSP-21560_21561_21564_21568 DMA2 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31022100	DMA2_DSCPTR_NXT	DMA2 Pointer to Next Initial Descriptor Register	0x00000000
0x31022104	DMA2_ADDRSTART	DMA2 Start Address of Current Buffer Register	0x00000000
0x31022108	DMA2_CFG	DMA2 Configuration Register	0x00000000
0x3102210C	DMA2_XCNT	DMA2 Inner Loop Count Start Value Register	0x00000000
0x31022110	DMA2_XMOD	DMA2 Inner Loop Address Increment Register	0x00000000
0x31022114	DMA2_YCNT	DMA2 Outer Loop Count Start Value (2D only) Register	0x00000000
0x31022118	DMA2_YMOD	DMA2 Outer Loop Address Increment (2D only) Register	0x00000000
0x31022124	DMA2_DSCPTR_CUR	DMA2 Current Descriptor Pointer Register	0x00000000
0x31022128	DMA2_DSCPTR_PRV	DMA2 Previous Initial Descriptor Pointer Register	0x00000000
0x3102212C	DMA2_ADDR_CUR	DMA2 Current Address Register	0x00000000
0x31022130	DMA2_STAT	DMA2 Status Register	0x00006000
0x31022134	DMA2_XCNT_CUR	DMA2 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x31022138	DMA2_YCNT_CUR	DMA2 Current Row Count (2D only) Register	0x00000000
0x31022140	DMA2_BWLCNT	DMA2 Bandwidth Limit Count Register	0x00000000
0x31022144	DMA2_BWLCNT_CUR	DMA2 Bandwidth Limit Count Current Register	0x00000000
0x31022148	DMA2_BWMCNT	DMA2 Bandwidth Monitor Count Register	0x00000000

Table A-26: ADSP-21560_21561_21564_21568 DMA2 MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x3102214C	DMA2_BWMCNT_CUR	DMA2 Bandwidth Monitor Count Current Register	0x00000000

Table A-27: ADSP-21560_21561_21564_21568 DMA20 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31026080	DMA20_DSCPTR_NXT	DMA20 Pointer to Next Initial Descriptor Register	0x00000000
0x31026084	DMA20_ADDRSTART	DMA20 Start Address of Current Buffer Register	0x00000000
0x31026088	DMA20_CFG	DMA20 Configuration Register	0x00000000
0x3102608C	DMA20_XCNT	DMA20 Inner Loop Count Start Value Register	0x00000000
0x31026090	DMA20_XMOD	DMA20 Inner Loop Address Increment Register	0x00000000
0x31026094	DMA20_YCNT	DMA20 Outer Loop Count Start Value (2D only) Register	0x00000000
0x31026098	DMA20_YMOD	DMA20 Outer Loop Address Increment (2D only) Register	0x00000000
0x310260A4	DMA20_DSCPTR_CUR	DMA20 Current Descriptor Pointer Register	0x00000000
0x310260A8	DMA20_DSCPTR_PRV	DMA20 Previous Initial Descriptor Pointer Register	0x00000000
0x310260AC	DMA20_ADDR_CUR	DMA20 Current Address Register	0x00000000
0x310260B0	DMA20_STAT	DMA20 Status Register	0x00006000
0x310260B4	DMA20_XCNT_CUR	DMA20 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x310260B8	DMA20_YCNT_CUR	DMA20 Current Row Count (2D only) Register	0x00000000
0x310260C0	DMA20_BWLCNT	DMA20 Bandwidth Limit Count Register	0x00000000
0x310260C4	DMA20_BWLCNT_CUR	DMA20 Bandwidth Limit Count Current Register	0x00000000
0x310260C8	DMA20_BWMCNT	DMA20 Bandwidth Monitor Count Register	0x00000000
0x310260CC	DMA20_BWMCNT_CUR	DMA20 Bandwidth Monitor Count Current Register	0x00000000

Table A-28: ADSP-21560_21561_21564_21568 DMA21 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31026000	DMA21_DSCPTR_NXT	DMA21 Pointer to Next Initial Descriptor Register	0x00000000
0x31026004	DMA21_ADDRSTART	DMA21 Start Address of Current Buffer Register	0x00000000
0x31026008	DMA21_CFG	DMA21 Configuration Register	0x00000000
0x3102600C	DMA21_XCNT	DMA21 Inner Loop Count Start Value Register	0x00000000

Table A-28: ADSP-21560_21561_21564_21568 DMA21 MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x31026010	DMA21_XMOD	DMA21 Inner Loop Address Increment Register	0x00000000
0x31026014	DMA21_YCNT	DMA21 Outer Loop Count Start Value (2D only) Register	0x00000000
0x31026018	DMA21_YMOD	DMA21 Outer Loop Address Increment (2D only) Register	0x00000000
0x31026024	DMA21_DSCPTR_CUR	DMA21 Current Descriptor Pointer Register	0x00000000
0x31026028	DMA21_DSCPTR_PRV	DMA21 Previous Initial Descriptor Pointer Register	0x00000000
0x3102602C	DMA21_ADDR_CUR	DMA21 Current Address Register	0x00000000
0x31026030	DMA21_STAT	DMA21 Status Register	0x00006000
0x31026034	DMA21_XCNT_CUR	DMA21 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x31026038	DMA21_YCNT_CUR	DMA21 Current Row Count (2D only) Register	0x00000000
0x31026040	DMA21_BWLCNT	DMA21 Bandwidth Limit Count Register	0x00000000
0x31026044	DMA21_BWLCNT_CUR	DMA21 Bandwidth Limit Count Current Register	0x00000000
0x31026048	DMA21_BWMCNT	DMA21 Bandwidth Monitor Count Register	0x00000000
0x3102604C	DMA21_BWMCNT_CUR	DMA21 Bandwidth Monitor Count Current Register	0x00000000

Table A-29: ADSP-21560_21561_21564_21568 DMA22 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x3102D000	DMA22_DSCPTR_NXT	DMA22 Pointer to Next Initial Descriptor Register	0x00000000
0x3102D004	DMA22_ADDRSTART	DMA22 Start Address of Current Buffer Register	0x00000000
0x3102D008	DMA22_CFG	DMA22 Configuration Register	0x00000000
0x3102D00C	DMA22_XCNT	DMA22 Inner Loop Count Start Value Register	0x00000000
0x3102D010	DMA22_XMOD	DMA22 Inner Loop Address Increment Register	0x00000000
0x3102D014	DMA22_YCNT	DMA22 Outer Loop Count Start Value (2D only) Register	0x00000000
0x3102D018	DMA22_YMOD	DMA22 Outer Loop Address Increment (2D only) Register	0x00000000
0x3102D024	DMA22_DSCPTR_CUR	DMA22 Current Descriptor Pointer Register	0x00000000
0x3102D028	DMA22_DSCPTR_PRV	DMA22 Previous Initial Descriptor Pointer Register	0x00000000
0x3102D02C	DMA22_ADDR_CUR	DMA22 Current Address Register	0x00000000
0x3102D030	DMA22_STAT	DMA22 Status Register	0x00006000

Table A-29: ADSP-21560_21561_21564_21568 DMA22 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3102D034	DMA22_XCNT_CUR	DMA22 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x3102D038	DMA22_YCNT_CUR	DMA22 Current Row Count (2D only) Register	0x00000000
0x3102D040	DMA22_BWLCNT	DMA22 Bandwidth Limit Count Register	0x00000000
0x3102D044	DMA22_BWLCNT_CUR	DMA22 Bandwidth Limit Count Current Register	0x00000000
0x3102D048	DMA22_BWMCNT	DMA22 Bandwidth Monitor Count Register	0x00000000
0x3102D04C	DMA22_BWMCNT_CUR	DMA22 Bandwidth Monitor Count Current Register	0x00000000

Table A-30: ADSP-21560_21561_21564_21568 DMA23 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x3102D080	DMA23_DSCPTR_NXT	DMA23 Pointer to Next Initial Descriptor Register	0x00000000
0x3102D084	DMA23_ADDRSTART	DMA23 Start Address of Current Buffer Register	0x00000000
0x3102D088	DMA23_CFG	DMA23 Configuration Register	0x00000000
0x3102D08C	DMA23_XCNT	DMA23 Inner Loop Count Start Value Register	0x00000000
0x3102D090	DMA23_XMOD	DMA23 Inner Loop Address Increment Register	0x00000000
0x3102D094	DMA23_YCNT	DMA23 Outer Loop Count Start Value (2D only) Register	0x00000000
0x3102D098	DMA23_YMOD	DMA23 Outer Loop Address Increment (2D only) Register	0x00000000
0x3102D0A4	DMA23_DSCPTR_CUR	DMA23 Current Descriptor Pointer Register	0x00000000
0x3102D0A8	DMA23_DSCPTR_PRV	DMA23 Previous Initial Descriptor Pointer Register	0x00000000
0x3102D0AC	DMA23_ADDR_CUR	DMA23 Current Address Register	0x00000000
0x3102D0B0	DMA23_STAT	DMA23 Status Register	0x00006000
0x3102D0B4	DMA23_XCNT_CUR	DMA23 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x3102D0B8	DMA23_YCNT_CUR	DMA23 Current Row Count (2D only) Register	0x00000000
0x3102D0C0	DMA23_BWLCNT	DMA23 Bandwidth Limit Count Register	0x00000000
0x3102D0C4	DMA23_BWLCNT_CUR	DMA23 Bandwidth Limit Count Current Register	0x00000000
0x3102D0C8	DMA23_BWMCNT	DMA23 Bandwidth Monitor Count Register	0x00000000
0x3102D0CC	DMA23_BWMCNT_CUR	DMA23 Bandwidth Monitor Count Current Register	0x00000000

Table A-31: ADSP-21560_21561_21564_21568 DMA24 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x3102D100	DMA24_DSCPTR_NXT	DMA24 Pointer to Next Initial Descriptor Register	0x00000000
0x3102D104	DMA24_ADDRSTART	DMA24 Start Address of Current Buffer Register	0x00000000
0x3102D108	DMA24_CFG	DMA24 Configuration Register	0x00000000
0x3102D10C	DMA24_XCNT	DMA24 Inner Loop Count Start Value Register	0x00000000
0x3102D110	DMA24_XMOD	DMA24 Inner Loop Address Increment Register	0x00000000
0x3102D114	DMA24_YCNT	DMA24 Outer Loop Count Start Value (2D only) Register	0x00000000
0x3102D118	DMA24_YMOD	DMA24 Outer Loop Address Increment (2D only) Register	0x00000000
0x3102D124	DMA24_DSCPTR_CUR	DMA24 Current Descriptor Pointer Register	0x00000000
0x3102D128	DMA24_DSCPTR_PRV	DMA24 Previous Initial Descriptor Pointer Register	0x00000000
0x3102D12C	DMA24_ADDR_CUR	DMA24 Current Address Register	0x00000000
0x3102D130	DMA24_STAT	DMA24 Status Register	0x00006000
0x3102D134	DMA24_XCNT_CUR	DMA24 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x3102D138	DMA24_YCNT_CUR	DMA24 Current Row Count (2D only) Register	0x00000000
0x3102D140	DMA24_BWLCNT	DMA24 Bandwidth Limit Count Register	0x00000000
0x3102D144	DMA24_BWLCNT_CUR	DMA24 Bandwidth Limit Count Current Register	0x00000000
0x3102D148	DMA24_BWMCNT	DMA24 Bandwidth Monitor Count Register	0x00000000
0x3102D14C	DMA24_BWMCNT_CUR	DMA24 Bandwidth Monitor Count Current Register	0x00000000

Table A-32: ADSP-21560_21561_21564_21568 DMA25 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x3102D180	DMA25_DSCPTR_NXT	DMA25 Pointer to Next Initial Descriptor Register	0x00000000
0x3102D184	DMA25_ADDRSTART	DMA25 Start Address of Current Buffer Register	0x00000000
0x3102D188	DMA25_CFG	DMA25 Configuration Register	0x00000000
0x3102D18C	DMA25_XCNT	DMA25 Inner Loop Count Start Value Register	0x00000000
0x3102D190	DMA25_XMOD	DMA25 Inner Loop Address Increment Register	0x00000000
0x3102D194	DMA25_YCNT	DMA25 Outer Loop Count Start Value (2D only) Register	0x00000000
0x3102D198	DMA25_YMOD	DMA25 Outer Loop Address Increment (2D only) Register	0x00000000

Table A-32: ADSP-21560_21561_21564_21568 DMA25 MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x3102D1A4	DMA25_DSCPTR_CUR	DMA25 Current Descriptor Pointer Register	0x00000000
0x3102D1A8	DMA25_DSCPTR_PRV	DMA25 Previous Initial Descriptor Pointer Register	0x00000000
0x3102D1AC	DMA25_ADDR_CUR	DMA25 Current Address Register	0x00000000
0x3102D1B0	DMA25_STAT	DMA25 Status Register	0x00006000
0x3102D1B4	DMA25_XCNT_CUR	DMA25 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x3102D1B8	DMA25_YCNT_CUR	DMA25 Current Row Count (2D only) Register	0x00000000
0x3102D1C0	DMA25_BWLCNT	DMA25 Bandwidth Limit Count Register	0x00000000
0x3102D1C4	DMA25_BWLCNT_CUR	DMA25 Bandwidth Limit Count Current Register	0x00000000
0x3102D1C8	DMA25_BWMCNT	DMA25 Bandwidth Monitor Count Register	0x00000000
0x3102D1CC	DMA25_BWMCNT_CUR	DMA25 Bandwidth Monitor Count Current Register	0x00000000

Table A-33: ADSP-21560_21561_21564_21568 DMA26 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x3102D200	DMA26_DSCPTR_NXT	DMA26 Pointer to Next Initial Descriptor Register	0x00000000
0x3102D204	DMA26_ADDRSTART	DMA26 Start Address of Current Buffer Register	0x00000000
0x3102D208	DMA26_CFG	DMA26 Configuration Register	0x00000000
0x3102D20C	DMA26_XCNT	DMA26 Inner Loop Count Start Value Register	0x00000000
0x3102D210	DMA26_XMOD	DMA26 Inner Loop Address Increment Register	0x00000000
0x3102D214	DMA26_YCNT	DMA26 Outer Loop Count Start Value (2D only) Register	0x00000000
0x3102D218	DMA26_YMOD	DMA26 Outer Loop Address Increment (2D only) Register	0x00000000
0x3102D224	DMA26_DSCPTR_CUR	DMA26 Current Descriptor Pointer Register	0x00000000
0x3102D228	DMA26_DSCPTR_PRV	DMA26 Previous Initial Descriptor Pointer Register	0x00000000
0x3102D22C	DMA26_ADDR_CUR	DMA26 Current Address Register	0x00000000
0x3102D230	DMA26_STAT	DMA26 Status Register	0x00006000
0x3102D234	DMA26_XCNT_CUR	DMA26 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x3102D238	DMA26_YCNT_CUR	DMA26 Current Row Count (2D only) Register	0x00000000
0x3102D240	DMA26_BWLCNT	DMA26 Bandwidth Limit Count Register	0x00000000
0x3102D244	DMA26_BWLCNT_CUR	DMA26 Bandwidth Limit Count Current Register	0x00000000

Table A-33: ADSP-21560_21561_21564_21568 DMA26 MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x3102D248	DMA26_BWMCNT	DMA26 Bandwidth Monitor Count Register	0x00000000
0x3102D24C	DMA26_BWMCNT_CUR	DMA26 Bandwidth Monitor Count Current Register	0x00000000

Table A-34: ADSP-21560_21561_21564_21568 DMA27 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x3102D280	DMA27_DSCPTR_NXT	DMA27 Pointer to Next Initial Descriptor Register	0x00000000
0x3102D284	DMA27_ADDRSTART	DMA27 Start Address of Current Buffer Register	0x00000000
0x3102D288	DMA27_CFG	DMA27 Configuration Register	0x00000000
0x3102D28C	DMA27_XCNT	DMA27 Inner Loop Count Start Value Register	0x00000000
0x3102D290	DMA27_XMOD	DMA27 Inner Loop Address Increment Register	0x00000000
0x3102D294	DMA27_YCNT	DMA27 Outer Loop Count Start Value (2D only) Register	0x00000000
0x3102D298	DMA27_YMOD	DMA27 Outer Loop Address Increment (2D only) Register	0x00000000
0x3102D2A4	DMA27_DSCPTR_CUR	DMA27 Current Descriptor Pointer Register	0x00000000
0x3102D2A8	DMA27_DSCPTR_PRV	DMA27 Previous Initial Descriptor Pointer Register	0x00000000
0x3102D2AC	DMA27_ADDR_CUR	DMA27 Current Address Register	0x00000000
0x3102D2B0	DMA27_STAT	DMA27 Status Register	0x00006000
0x3102D2B4	DMA27_XCNT_CUR	DMA27 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x3102D2B8	DMA27_YCNT_CUR	DMA27 Current Row Count (2D only) Register	0x00000000
0x3102D2C0	DMA27_BWLCNT	DMA27 Bandwidth Limit Count Register	0x00000000
0x3102D2C4	DMA27_BWLCNT_CUR	DMA27 Bandwidth Limit Count Current Register	0x00000000
0x3102D2C8	DMA27_BWMCNT	DMA27 Bandwidth Monitor Count Register	0x00000000
0x3102D2CC	DMA27_BWMCNT_CUR	DMA27 Bandwidth Monitor Count Current Register	0x00000000

Table A-35: ADSP-21560_21561_21564_21568 DMA3 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31022180	DMA3_DSCPTR_NXT	DMA3 Pointer to Next Initial Descriptor Register	0x00000000
0x31022184	DMA3_ADDRSTART	DMA3 Start Address of Current Buffer Register	0x00000000
0x31022188	DMA3_CFG	DMA3 Configuration Register	0x00000000

Table A-35: ADSP-21560_21561_21564_21568 DMA3 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3102218C	DMA3_XCNT	DMA3 Inner Loop Count Start Value Register	0x00000000
0x31022190	DMA3_XMOD	DMA3 Inner Loop Address Increment Register	0x00000000
0x31022194	DMA3_YCNT	DMA3 Outer Loop Count Start Value (2D only) Register	0x00000000
0x31022198	DMA3_YMOD	DMA3 Outer Loop Address Increment (2D only) Register	0x00000000
0x310221A4	DMA3_DSCPTR_CUR	DMA3 Current Descriptor Pointer Register	0x00000000
0x310221A8	DMA3_DSCPTR_PRV	DMA3 Previous Initial Descriptor Pointer Register	0x00000000
0x310221AC	DMA3_ADDR_CUR	DMA3 Current Address Register	0x00000000
0x310221B0	DMA3_STAT	DMA3 Status Register	0x00006000
0x310221B4	DMA3_XCNT_CUR	DMA3 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x310221B8	DMA3_YCNT_CUR	DMA3 Current Row Count (2D only) Register	0x00000000
0x310221C0	DMA3_BWLCNT	DMA3 Bandwidth Limit Count Register	0x00000000
0x310221C4	DMA3_BWLCNT_CUR	DMA3 Bandwidth Limit Count Current Register	0x00000000
0x310221C8	DMA3_BWMCNT	DMA3 Bandwidth Monitor Count Register	0x00000000
0x310221CC	DMA3_BWMCNT_CUR	DMA3 Bandwidth Monitor Count Current Register	0x00000000

Table A-36: ADSP-21560_21561_21564_21568 DMA34 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31026180	DMA34_DSCPTR_NXT	DMA34 Pointer to Next Initial Descriptor Register	0x00000000
0x31026184	DMA34_ADDRSTART	DMA34 Start Address of Current Buffer Register	0x00000000
0x31026188	DMA34_CFG	DMA34 Configuration Register	0x00000000
0x3102618C	DMA34_XCNT	DMA34 Inner Loop Count Start Value Register	0x00000000
0x31026190	DMA34_XMOD	DMA34 Inner Loop Address Increment Register	0x00000000
0x31026194	DMA34_YCNT	DMA34 Outer Loop Count Start Value (2D only) Register	0x00000000
0x31026198	DMA34_YMOD	DMA34 Outer Loop Address Increment (2D only) Register	0x00000000
0x310261A4	DMA34_DSCPTR_CUR	DMA34 Current Descriptor Pointer Register	0x00000000
0x310261A8	DMA34_DSCPTR_PRV	DMA34 Previous Initial Descriptor Pointer Register	0x00000000
0x310261AC	DMA34_ADDR_CUR	DMA34 Current Address Register	0x00000000
0x310261B0	DMA34_STAT	DMA34 Status Register	0x00006000

Table A-36: ADSP-21560_21561_21564_21568 DMA34 MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x310261B4	DMA34_XCNT_CUR	DMA34 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x310261B8	DMA34_YCNT_CUR	DMA34 Current Row Count (2D only) Register	0x00000000
0x310261C0	DMA34_BWLCNT	DMA34 Bandwidth Limit Count Register	0x00000000
0x310261C4	DMA34_BWLCNT_CUR	DMA34 Bandwidth Limit Count Current Register	0x00000000
0x310261C8	DMA34_BWMCNT	DMA34 Bandwidth Monitor Count Register	0x00000000
0x310261CC	DMA34_BWMCNT_CUR	DMA34 Bandwidth Monitor Count Current Register	0x00000000

Table A-37: ADSP-21560_21561_21564_21568 DMA35 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31026100	DMA35_DSCPTR_NXT	DMA35 Pointer to Next Initial Descriptor Register	0x00000000
0x31026104	DMA35_ADDRSTART	DMA35 Start Address of Current Buffer Register	0x00000000
0x31026108	DMA35_CFG	DMA35 Configuration Register	0x00000000
0x3102610C	DMA35_XCNT	DMA35 Inner Loop Count Start Value Register	0x00000000
0x31026110	DMA35_XMOD	DMA35 Inner Loop Address Increment Register	0x00000000
0x31026114	DMA35_YCNT	DMA35 Outer Loop Count Start Value (2D only) Register	0x00000000
0x31026118	DMA35_YMOD	DMA35 Outer Loop Address Increment (2D only) Register	0x00000000
0x31026124	DMA35_DSCPTR_CUR	DMA35 Current Descriptor Pointer Register	0x00000000
0x31026128	DMA35_DSCPTR_PRV	DMA35 Previous Initial Descriptor Pointer Register	0x00000000
0x3102612C	DMA35_ADDR_CUR	DMA35 Current Address Register	0x00000000
0x31026130	DMA35_STAT	DMA35 Status Register	0x00006000
0x31026134	DMA35_XCNT_CUR	DMA35 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x31026138	DMA35_YCNT_CUR	DMA35 Current Row Count (2D only) Register	0x00000000
0x31026140	DMA35_BWLCNT	DMA35 Bandwidth Limit Count Register	0x00000000
0x31026144	DMA35_BWLCNT_CUR	DMA35 Bandwidth Limit Count Current Register	0x00000000
0x31026148	DMA35_BWMCNT	DMA35 Bandwidth Monitor Count Register	0x00000000
0x3102614C	DMA35_BWMCNT_CUR	DMA35 Bandwidth Monitor Count Current Register	0x00000000

Table A-38: ADSP-21560_21561_21564_21568 DMA39 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x3109A000	DMA39_DSCPTR_NXT	DMA39 Pointer to Next Initial Descriptor Register	0x00000000
0x3109A004	DMA39_ADDRSTART	DMA39 Start Address of Current Buffer Register	0x00000000
0x3109A008	DMA39_CFG	DMA39 Configuration Register	0x00000220
0x3109A00C	DMA39_XCNT	DMA39 Inner Loop Count Start Value Register	0x00000000
0x3109A010	DMA39_XMOD	DMA39 Inner Loop Address Increment Register	0x00000000
0x3109A014	DMA39_YCNT	DMA39 Outer Loop Count Start Value (2D only) Register	0x00000000
0x3109A018	DMA39_YMOD	DMA39 Outer Loop Address Increment (2D only) Register	0x00000000
0x3109A024	DMA39_DSCPTR_CUR	DMA39 Current Descriptor Pointer Register	0x00000000
0x3109A028	DMA39_DSCPTR_PRV	DMA39 Previous Initial Descriptor Pointer Register	0x00000000
0x3109A02C	DMA39_ADDR_CUR	DMA39 Current Address Register	0x00000000
0x3109A030	DMA39_STAT	DMA39 Status Register	0x00006000
0x3109A034	DMA39_XCNT_CUR	DMA39 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x3109A038	DMA39_YCNT_CUR	DMA39 Current Row Count (2D only) Register	0x00000000
0x3109A040	DMA39_BWLCNT	DMA39 Bandwidth Limit Count Register	0x00000000
0x3109A044	DMA39_BWLCNT_CUR	DMA39 Bandwidth Limit Count Current Register	0x00000000
0x3109A048	DMA39_BWMCNT	DMA39 Bandwidth Monitor Count Register	0x00000000
0x3109A04C	DMA39_BWMCNT_CUR	DMA39 Bandwidth Monitor Count Current Register	0x00000000

Table A-39: ADSP-21560_21561_21564_21568 DMA4 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31022200	DMA4_DSCPTR_NXT	DMA4 Pointer to Next Initial Descriptor Register	0x00000000
0x31022204	DMA4_ADDRSTART	DMA4 Start Address of Current Buffer Register	0x00000000
0x31022208	DMA4_CFG	DMA4 Configuration Register	0x00000000
0x3102220C	DMA4_XCNT	DMA4 Inner Loop Count Start Value Register	0x00000000
0x31022210	DMA4_XMOD	DMA4 Inner Loop Address Increment Register	0x00000000
0x31022214	DMA4_YCNT	DMA4 Outer Loop Count Start Value (2D only) Register	0x00000000
0x31022218	DMA4_YMOD	DMA4 Outer Loop Address Increment (2D only) Register	0x00000000
0x31022224	DMA4_DSCPTR_CUR	DMA4 Current Descriptor Pointer Register	0x00000000

Table A-39: ADSP-21560_21561_21564_21568 DMA4 MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x31022228	DMA4_DSCPTR_PRV	DMA4 Previous Initial Descriptor Pointer Register	0x00000000
0x3102222C	DMA4_ADDR_CUR	DMA4 Current Address Register	0x00000000
0x31022230	DMA4_STAT	DMA4 Status Register	0x00006000
0x31022234	DMA4_XCNT_CUR	DMA4 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x31022238	DMA4_YCNT_CUR	DMA4 Current Row Count (2D only) Register	0x00000000
0x31022240	DMA4_BWLCNT	DMA4 Bandwidth Limit Count Register	0x00000000
0x31022244	DMA4_BWLCNT_CUR	DMA4 Bandwidth Limit Count Current Register	0x00000000
0x31022248	DMA4_BWMCNT	DMA4 Bandwidth Monitor Count Register	0x00000000
0x3102224C	DMA4_BWMCNT_CUR	DMA4 Bandwidth Monitor Count Current Register	0x00000000

Table A-40: ADSP-21560_21561_21564_21568 DMA40 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x3109A080	DMA40_DSCPTR_NXT	DMA40 Pointer to Next Initial Descriptor Register	0x00000000
0x3109A084	DMA40_ADDRSTART	DMA40 Start Address of Current Buffer Register	0x00000000
0x3109A088	DMA40_CFG	DMA40 Configuration Register	0x00000222
0x3109A08C	DMA40_XCNT	DMA40 Inner Loop Count Start Value Register	0x00000000
0x3109A090	DMA40_XMOD	DMA40 Inner Loop Address Increment Register	0x00000000
0x3109A094	DMA40_YCNT	DMA40 Outer Loop Count Start Value (2D only) Register	0x00000000
0x3109A098	DMA40_YMOD	DMA40 Outer Loop Address Increment (2D only) Register	0x00000000
0x3109A0A4	DMA40_DSCPTR_CUR	DMA40 Current Descriptor Pointer Register	0x00000000
0x3109A0A8	DMA40_DSCPTR_PRV	DMA40 Previous Initial Descriptor Pointer Register	0x00000000
0x3109A0AC	DMA40_ADDR_CUR	DMA40 Current Address Register	0x00000000
0x3109A0B0	DMA40_STAT	DMA40 Status Register	0x00006000
0x3109A0B4	DMA40_XCNT_CUR	DMA40 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x3109A0B8	DMA40_YCNT_CUR	DMA40 Current Row Count (2D only) Register	0x00000000
0x3109A0C0	DMA40_BWLCNT	DMA40 Bandwidth Limit Count Register	0x00000000
0x3109A0C4	DMA40_BWLCNT_CUR	DMA40 Bandwidth Limit Count Current Register	0x00000000
0x3109A0C8	DMA40_BWMCNT	DMA40 Bandwidth Monitor Count Register	0x00000000

Table A-40: ADSP-21560_21561_21564_21568 DMA40 MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x3109A0CC	DMA40_BWMCNT_CUR	DMA40 Bandwidth Monitor Count Current Register	0x00000000

Table A-41: ADSP-21560_21561_21564_21568 DMA43 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x3109B000	DMA43_DSCPTR_NXT	DMA43 Pointer to Next Initial Descriptor Register	0x00000000
0x3109B004	DMA43_ADDRSTART	DMA43 Start Address of Current Buffer Register	0x00000000
0x3109B008	DMA43_CFG	DMA43 Configuration Register	0x00000220
0x3109B00C	DMA43_XCNT	DMA43 Inner Loop Count Start Value Register	0x00000000
0x3109B010	DMA43_XMOD	DMA43 Inner Loop Address Increment Register	0x00000000
0x3109B014	DMA43_YCNT	DMA43 Outer Loop Count Start Value (2D only) Register	0x00000000
0x3109B018	DMA43_YMOD	DMA43 Outer Loop Address Increment (2D only) Register	0x00000000
0x3109B024	DMA43_DSCPTR_CUR	DMA43 Current Descriptor Pointer Register	0x00000000
0x3109B028	DMA43_DSCPTR_PRV	DMA43 Previous Initial Descriptor Pointer Register	0x00000000
0x3109B02C	DMA43_ADDR_CUR	DMA43 Current Address Register	0x00000000
0x3109B030	DMA43_STAT	DMA43 Status Register	0x0000B000
0x3109B034	DMA43_XCNT_CUR	DMA43 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x3109B038	DMA43_YCNT_CUR	DMA43 Current Row Count (2D only) Register	0x00000000
0x3109B040	DMA43_BWLCNT	DMA43 Bandwidth Limit Count Register	0x00000000
0x3109B044	DMA43_BWLCNT_CUR	DMA43 Bandwidth Limit Count Current Register	0x00000000
0x3109B048	DMA43_BWMCNT	DMA43 Bandwidth Monitor Count Register	0x00000000
0x3109B04C	DMA43_BWMCNT_CUR	DMA43 Bandwidth Monitor Count Current Register	0x00000000

Table A-42: ADSP-21560_21561_21564_21568 DMA44 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x3109B080	DMA44_DSCPTR_NXT	DMA44 Pointer to Next Initial Descriptor Register	0x00000000
0x3109B084	DMA44_ADDRSTART	DMA44 Start Address of Current Buffer Register	0x00000000
0x3109B088	DMA44_CFG	DMA44 Configuration Register	0x00000222
0x3109B08C	DMA44_XCNT	DMA44 Inner Loop Count Start Value Register	0x00000000

Table A-42: ADSP-21560_21561_21564_21568 DMA44 MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x3109B090	DMA44_XMOD	DMA44 Inner Loop Address Increment Register	0x00000000
0x3109B094	DMA44_YCNT	DMA44 Outer Loop Count Start Value (2D only) Register	0x00000000
0x3109B098	DMA44_YMOD	DMA44 Outer Loop Address Increment (2D only) Register	0x00000000
0x3109B0A4	DMA44_DSCPTR_CUR	DMA44 Current Descriptor Pointer Register	0x00000000
0x3109B0A8	DMA44_DSCPTR_PRV	DMA44 Previous Initial Descriptor Pointer Register	0x00000000
0x3109B0AC	DMA44_ADDR_CUR	DMA44 Current Address Register	0x00000000
0x3109B0B0	DMA44_STAT	DMA44 Status Register	0x0000B000
0x3109B0B4	DMA44_XCNT_CUR	DMA44 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x3109B0B8	DMA44_YCNT_CUR	DMA44 Current Row Count (2D only) Register	0x00000000
0x3109B0C0	DMA44_BWLCNT	DMA44 Bandwidth Limit Count Register	0x00000000
0x3109B0C4	DMA44_BWLCNT_CUR	DMA44 Bandwidth Limit Count Current Register	0x00000000
0x3109B0C8	DMA44_BWMCNT	DMA44 Bandwidth Monitor Count Register	0x00000000
0x3109B0CC	DMA44_BWMCNT_CUR	DMA44 Bandwidth Monitor Count Current Register	0x00000000

Table A-43: ADSP-21560_21561_21564_21568 DMA5 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31022280	DMA5_DSCPTR_NXT	DMA5 Pointer to Next Initial Descriptor Register	0x00000000
0x31022284	DMA5_ADDRSTART	DMA5 Start Address of Current Buffer Register	0x00000000
0x31022288	DMA5_CFG	DMA5 Configuration Register	0x00000000
0x3102228C	DMA5_XCNT	DMA5 Inner Loop Count Start Value Register	0x00000000
0x31022290	DMA5_XMOD	DMA5 Inner Loop Address Increment Register	0x00000000
0x31022294	DMA5_YCNT	DMA5 Outer Loop Count Start Value (2D only) Register	0x00000000
0x31022298	DMA5_YMOD	DMA5 Outer Loop Address Increment (2D only) Register	0x00000000
0x310222A4	DMA5_DSCPTR_CUR	DMA5 Current Descriptor Pointer Register	0x00000000
0x310222A8	DMA5_DSCPTR_PRV	DMA5 Previous Initial Descriptor Pointer Register	0x00000000
0x310222AC	DMA5_ADDR_CUR	DMA5 Current Address Register	0x00000000
0x310222B0	DMA5_STAT	DMA5 Status Register	0x00006000
0x310222B4	DMA5_XCNT_CUR	DMA5 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000

Table A-43: ADSP-21560_21561_21564_21568 DMA5 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310222B8	DMA5_YCNT_CUR	DMA5 Current Row Count (2D only) Register	0x00000000
0x310222C0	DMA5_BWLCNT	DMA5 Bandwidth Limit Count Register	0x00000000
0x310222C4	DMA5_BWLCNT_CUR	DMA5 Bandwidth Limit Count Current Register	0x00000000
0x310222C8	DMA5_BWMCNT	DMA5 Bandwidth Monitor Count Register	0x00000000
0x310222CC	DMA5_BWMCNT_CUR	DMA5 Bandwidth Monitor Count Current Register	0x00000000

Table A-44: ADSP-21560_21561_21564_21568 DMA6 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31022300	DMA6_DSCPTR_NXT	DMA6 Pointer to Next Initial Descriptor Register	0x00000000
0x31022304	DMA6_ADDRSTART	DMA6 Start Address of Current Buffer Register	0x00000000
0x31022308	DMA6_CFG	DMA6 Configuration Register	0x00000000
0x3102230C	DMA6_XCNT	DMA6 Inner Loop Count Start Value Register	0x00000000
0x31022310	DMA6_XMOD	DMA6 Inner Loop Address Increment Register	0x00000000
0x31022314	DMA6_YCNT	DMA6 Outer Loop Count Start Value (2D only) Register	0x00000000
0x31022318	DMA6_YMOD	DMA6 Outer Loop Address Increment (2D only) Register	0x00000000
0x31022324	DMA6_DSCPTR_CUR	DMA6 Current Descriptor Pointer Register	0x00000000
0x31022328	DMA6_DSCPTR_PRV	DMA6 Previous Initial Descriptor Pointer Register	0x00000000
0x3102232C	DMA6_ADDR_CUR	DMA6 Current Address Register	0x00000000
0x31022330	DMA6_STAT	DMA6 Status Register	0x00006000
0x31022334	DMA6_XCNT_CUR	DMA6 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x31022338	DMA6_YCNT_CUR	DMA6 Current Row Count (2D only) Register	0x00000000
0x31022340	DMA6_BWLCNT	DMA6 Bandwidth Limit Count Register	0x00000000
0x31022344	DMA6_BWLCNT_CUR	DMA6 Bandwidth Limit Count Current Register	0x00000000
0x31022348	DMA6_BWMCNT	DMA6 Bandwidth Monitor Count Register	0x00000000
0x3102234C	DMA6_BWMCNT_CUR	DMA6 Bandwidth Monitor Count Current Register	0x00000000

Table A-45: ADSP-21560_21561_21564_21568 DMA7 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31022380	DMA7_DSCPTR_NXT	DMA7 Pointer to Next Initial Descriptor Register	0x00000000

Table A-45: ADSP-21560_21561_21564_21568 DMA7 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31022384	DMA7_ADDRSTART	DMA7 Start Address of Current Buffer Register	0x00000000
0x31022388	DMA7_CFG	DMA7 Configuration Register	0x00000000
0x3102238C	DMA7_XCNT	DMA7 Inner Loop Count Start Value Register	0x00000000
0x31022390	DMA7_XMOD	DMA7 Inner Loop Address Increment Register	0x00000000
0x31022394	DMA7_YCNT	DMA7 Outer Loop Count Start Value (2D only) Register	0x00000000
0x31022398	DMA7_YMOD	DMA7 Outer Loop Address Increment (2D only) Register	0x00000000
0x310223A4	DMA7_DSCPTR_CUR	DMA7 Current Descriptor Pointer Register	0x00000000
0x310223A8	DMA7_DSCPTR_PRV	DMA7 Previous Initial Descriptor Pointer Register	0x00000000
0x310223AC	DMA7_ADDR_CUR	DMA7 Current Address Register	0x00000000
0x310223B0	DMA7_STAT	DMA7 Status Register	0x00006000
0x310223B4	DMA7_XCNT_CUR	DMA7 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x310223B8	DMA7_YCNT_CUR	DMA7 Current Row Count (2D only) Register	0x00000000
0x310223C0	DMA7_BWLCNT	DMA7 Bandwidth Limit Count Register	0x00000000
0x310223C4	DMA7_BWLCNT_CUR	DMA7 Bandwidth Limit Count Current Register	0x00000000
0x310223C8	DMA7_BWMCNT	DMA7 Bandwidth Monitor Count Register	0x00000000
0x310223CC	DMA7_BWMCNT_CUR	DMA7 Bandwidth Monitor Count Current Register	0x00000000

Table A-46: ADSP-21560_21561_21564_21568 DMA8 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310A7000	DMA8_DSCPTR_NXT	DMA8 Pointer to Next Initial Descriptor Register	0x00000000
0x310A7004	DMA8_ADDRSTART	DMA8 Start Address of Current Buffer Register	0x00000000
0x310A7008	DMA8_CFG	DMA8 Configuration Register	0x00000000
0x310A700C	DMA8_XCNT	DMA8 Inner Loop Count Start Value Register	0x00000000
0x310A7010	DMA8_XMOD	DMA8 Inner Loop Address Increment Register	0x00000000
0x310A7014	DMA8_YCNT	DMA8 Outer Loop Count Start Value (2D only) Register	0x00000000
0x310A7018	DMA8_YMOD	DMA8 Outer Loop Address Increment (2D only) Register	0x00000000
0x310A7024	DMA8_DSCPTR_CUR	DMA8 Current Descriptor Pointer Register	0x00000000
0x310A7028	DMA8_DSCPTR_PRV	DMA8 Previous Initial Descriptor Pointer Register	0x00000000
0x310A702C	DMA8_ADDR_CUR	DMA8 Current Address Register	0x00000000
0x310A7030	DMA8_STAT	DMA8 Status Register	0x00006000

Table A-46: ADSP-21560_21561_21564_21568 DMA8 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310A7034	DMA8_XCNT_CUR	DMA8 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x310A7038	DMA8_YCNT_CUR	DMA8 Current Row Count (2D only) Register	0x00000000
0x310A7040	DMA8_BWLCNT	DMA8 Bandwidth Limit Count Register	0x00000000
0x310A7044	DMA8_BWLCNT_CUR	DMA8 Bandwidth Limit Count Current Register	0x00000000
0x310A7048	DMA8_BWMCNT	DMA8 Bandwidth Monitor Count Register	0x00000000
0x310A704C	DMA8_BWMCNT_CUR	DMA8 Bandwidth Monitor Count Current Register	0x00000000

Table A-47: ADSP-21560_21561_21564_21568 DMA9 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310A7080	DMA9_DSCPTR_NXT	DMA9 Pointer to Next Initial Descriptor Register	0x00000000
0x310A7084	DMA9_ADDRSTART	DMA9 Start Address of Current Buffer Register	0x00000000
0x310A7088	DMA9_CFG	DMA9 Configuration Register	0x00000000
0x310A708C	DMA9_XCNT	DMA9 Inner Loop Count Start Value Register	0x00000000
0x310A7090	DMA9_XMOD	DMA9 Inner Loop Address Increment Register	0x00000000
0x310A7094	DMA9_YCNT	DMA9 Outer Loop Count Start Value (2D only) Register	0x00000000
0x310A7098	DMA9_YMOD	DMA9 Outer Loop Address Increment (2D only) Register	0x00000000
0x310A70A4	DMA9_DSCPTR_CUR	DMA9 Current Descriptor Pointer Register	0x00000000
0x310A70A8	DMA9_DSCPTR_PRV	DMA9 Previous Initial Descriptor Pointer Register	0x00000000
0x310A70AC	DMA9_ADDR_CUR	DMA9 Current Address Register	0x00000000
0x310A70B0	DMA9_STAT	DMA9 Status Register	0x00006000
0x310A70B4	DMA9_XCNT_CUR	DMA9 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x310A70B8	DMA9_YCNT_CUR	DMA9 Current Row Count (2D only) Register	0x00000000
0x310A70C0	DMA9_BWLCNT	DMA9 Bandwidth Limit Count Register	0x00000000
0x310A70C4	DMA9_BWLCNT_CUR	DMA9 Bandwidth Limit Count Current Register	0x00000000
0x310A70C8	DMA9_BWMCNT	DMA9 Bandwidth Monitor Count Register	0x00000000
0x310A70CC	DMA9_BWMCNT_CUR	DMA9 Bandwidth Monitor Count Current Register	0x00000000

Table A-48: ADSP-21560_21561_21564_21568 DPM0 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31090000	DPM0_CTL	DPM0 Control Register	0x00000000
0x31090004	DPM0_STAT	DPM0 Status Register	0x00000001
0x31090070	DPM0_PER_DIS0	DPM0 Peripherals Disable Register 0	0x00000000
0x31090074	DPM0_PER_DIS1	DPM0 Peripherals Disable Register 1	0x00000000
0x31090084	DPM0_REVID	DPM0 Revision ID	0x00000020

Table A-49: ADSP-21560_21561_21564_21568 EMDMA MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x310E002C	EMDMA0_CTL	EMDMA0 External Memory DMA Control Register	0x00000000
0x310E0030	EMDMA_CTL	EMDMA External Memory DMA Control Register	0x00000000
0x310E0080	EMDMA0_INDX1	EMDMA0 External Index Register	0x00000000
0x310E0084	EMDMA0_MOD1	EMDMA0 External Modifier Register	0x00000000
0x310E0088	EMDMA0_CNT1	EMDMA0 External Count Register	0x00000000
0x310E008C	EMDMA0_INDX0	EMDMA0 Internal Index Register	0x00000000
0x310E0090	EMDMA0_MOD0	EMDMA0 Internal Modifier Register	0x00000000
0x310E0094	EMDMA0_CNT0	EMDMA0 Internal Count Register	0x00000000
0x310E0098	EMDMA0_CHNPTR	EMDMA0 Chain Pointer Register	0x00000000
0x310E009C	EMDMA0_BASE	EMDMA0 External Base Address Register	0x00000000
0x310E00A0	EMDMA0_TPTR	EMDMA0 Tap List Pointer Register	0x00000000
0x310E00A4	EMDMA0_BUFLN	EMDMA0 Circular Buffer Length Register	0x00000000
0x310E00AC	EMDMA0_TCNT	EMDMA0 Delay Line Tap Count Register	0x00000000
0x310E00C0	EMDMA_INDX1	EMDMA External Index Register	0x00000000
0x310E00C4	EMDMA_MOD1	EMDMA External Modifier Register	0x00000000
0x310E00C8	EMDMA_CNT1	EMDMA External Count Register	0x00000000
0x310E00CC	EMDMA_INDX0	EMDMA Internal Index Register	0x00000000
0x310E00D0	EMDMA_MOD0	EMDMA Internal Modifier Register	0x00000000
0x310E00D4	EMDMA_CNT0	EMDMA Internal Count Register	0x00000000
0x310E00D8	EMDMA_CHNPTR	EMDMA Chain Pointer Register	0x00000000
0x310E00DC	EMDMA_BASE	EMDMA External Base Address Register	0x00000000
0x310E00E0	EMDMA_TPTR	EMDMA Tap List Pointer Register	0x00000000

Table A-49: ADSP-21560_21561_21564_21568 EMDMA MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310E00E4	EMDMA_BUFLEN	EMDMA Circular Buffer Length Register	0x00000000
0x310E00EC	EMDMA_TCNT	EMDMA Delay Line Tap Count Register	0x00000000

Table A-50: ADSP-21560_21561_21564_21568 EMDMA1 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310E0030	EMDMA1_CTL	EMDMA1 External Memory DMA Control Register	0x00000000
0x310E00C0	EMDMA1_INDX1	EMDMA1 External Index Register	0x00000000
0x310E00C4	EMDMA1_MOD1	EMDMA1 External Modifier Register	0x00000000
0x310E00C8	EMDMA1_CNT1	EMDMA1 External Count Register	0x00000000
0x310E00CC	EMDMA1_INDX0	EMDMA1 Internal Index Register	0x00000000
0x310E00D0	EMDMA1_MOD0	EMDMA1 Internal Modifier Register	0x00000000
0x310E00D4	EMDMA1_CNT0	EMDMA1 Internal Count Register	0x00000000
0x310E00D8	EMDMA1_CHNPTR	EMDMA1 Chain Pointer Register	0x00000000
0x310E00DC	EMDMA1_BASE	EMDMA1 External Base Address Register	0x00000000
0x310E00E0	EMDMA1_TPTR	EMDMA1 Tap List Pointer Register	0x00000000
0x310E00E4	EMDMA1_BUFLEN	EMDMA1 Circular Buffer Length Register	0x00000000
0x310E00EC	EMDMA1_TCNT	EMDMA1 Delay Line Tap Count Register	0x00000000

Table A-51: ADSP-21560_21561_21564_21568 FIR0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310C3000	FIR0_CTL1	FIR0 FIR Global Control Register	0x00000000
0x310C3004	FIR0_DMASTAT	FIR0 FIR DMA Status Register	0x00000000
0x310C3008	FIR0_MACSTAT	FIR0 FIR MAC Status Register	0x00000000
0x310C3010	FIR0_DBG_CTL	FIR0 FIR Debug Control Register	0x00000000
0x310C3014	FIR0_DBG_ADDR	FIR0 Debug Address Register	0x00000000
0x310C3018	FIR0_DBG_WRDAT	FIR0 FIR Debug Data Write Register	0x00000000
0x310C301C	FIR0_DBG_RDDAT	FIR0 FIR Debug Data Read Register	0x00000000
0x310C3040	FIR0_CTL2	FIR0 FIR Channel Control Register	0x00000000
0x310C3044	FIR0_INIDX	FIR0 FIR Input Data Index Register	0x00000000
0x310C3048	FIR0_INMOD	FIR0 FIR Input Data Modifier Register	0x00000000

Table A-51: ADSP-21560_21561_21564_21568 FIR0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310C304C	FIR0_INCNT	FIR0 FIR Input Data Count Register	0x00000000
0x310C3050	FIR0_INBASE	FIR0 FIR Input Data Base Register	0x00000000
0x310C3054	FIR0_OUTIDX	FIR0 FIR Output Data Index Register	0x00000000
0x310C3058	FIR0_OUTMOD	FIR0 FIR Output Data Modifier Register	0x00000000
0x310C305C	FIR0_OUTCNT	FIR0 FIR Output Data Count Register	0x00000000
0x310C3060	FIR0_OUTBASE	FIR0 FIR Output Data Base Register	0x00000000
0x310C3064	FIR0_COEFIDX	FIR0 FIR Coefficient Index Register	0x00000000
0x310C3068	FIR0_COEFMOD	FIR0 FIR Coefficient Modifier Register	0x00000000
0x310C306C	FIR0_COEFCNT	FIR0 FIR Coefficient Count Register	0x00000000
0x310C3070	FIR0_CHNPTR	FIR0 FIR Chain Pointer Register	0x00000000
0x310C3074	FIR0_SCTL1	FIR0 Software Control Register 1	0x00000000
0x310C3078	FIR0_SCTL2	FIR0 Software Control Register 2	0x00000000
0x310C307C	FIR0_SGCTL	FIR0 Secondary Global Control Register	0x00000000

Table A-52: ADSP-21560_21561_21564_21568 IIR0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310C4000	IIR0_CTL1	IIR0 Global Control Register	0x00000000
0x310C4004	IIR0_DMASTAT	IIR0 DMA Status Register	0x00000000
0x310C4008	IIR0_MACSTAT	IIR0 MAC Status Register	0x00000000
0x310C400C	IIR0_DBG_CTL	IIR0 IIR Debug Control Register	0x00000000
0x310C4010	IIR0_DBG_ADDR	IIR0 IIR Debug Address Register	0x00000000
0x310C4014	IIR0_DBG_WRDAT_LO	IIR0 IIR Debug Write Data Low Register	0x00000000
0x310C4018	IIR0_DBG_WRDAT_HI	IIR0 IIR Debug Write Data High Register	0x00000000
0x310C401C	IIR0_DBG_RDDAT_LO	IIR0 IIR Debug Read Data Low Register	0x00000000
0x310C4020	IIR0_DBG_RDDAT_HI	IIR0 IIR Debug Read Data High Register	0x00000000
0x310C4040	IIR0_CTL2	IIR0 Channel Control Register	0x00000000
0x310C4044	IIR0_INIDX	IIR0 Input Data Index Register	0x00000000
0x310C4048	IIR0_INMOD	IIR0 Input Data Index Modifier Register	0x00000000
0x310C404C	IIR0_INLEN	IIR0 Input Data Buffer Length Register	0x00000000
0x310C4050	IIR0_INBASE	IIR0 Input Buffer Base Register	0x00000000

Table A-52: ADSP-21560_21561_21564_21568 IIR0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310C4054	IIR0_OUTIDX	IIR0 Output Data Buffer Index Register	0x00000000
0x310C4058	IIR0_OUTMOD	IIR0 IIR Output Data Index Modifier Register	0x00000000
0x310C405C	IIR0_OUTLEN	IIR0 IIR Output Data Buffer Length Register	0x00000000
0x310C4060	IIR0_OUTBASE	IIR0 Output Buffer Base Register	0x00000000
0x310C4064	IIR0_COEFIDX	IIR0 Coefficient Buffer Index Register	0x00000000
0x310C4068	IIR0_COEFMOD	IIR0 Coefficient Index Modifier Register	0x00000000
0x310C406C	IIR0_COEFLEN	IIR0 Coefficient Buffer Length Register	0x00000000
0x310C4070	IIR0_CHNPTR	IIR0 Chain Pointer Register	0x00000000
0x310C4074	IIR0_SCTL1	IIR0 Software Control Register1	0x00000000
0x310C4078	IIR0_SCTL2	IIR0 Software Control Register2	0x00000000
0x310C407C	IIR0_SGCTL	IIR0 Secondary Global Control Register	0x00000000

Table A-53: ADSP-21560_21561_21564_21568 L2CTL MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31080000	L2CTL_CTL	L2CTL Control Register	0x00000000
0x31080000	L2CTL0_CTL	L2CTL0 Control Register	0x00000000
0x31080010	L2CTL_STAT	L2CTL Status Register	0x00000000
0x31080010	L2CTL0_STAT	L2CTL0 Status Register	0x00000000
0x31080014	L2CTL_RPCR	L2CTL Read Priority Count Register	0x00000F0F
0x31080014	L2CTL0_RPCR	L2CTL0 Read Priority Count Register	0x00000F0F
0x31080018	L2CTL_WPCR	L2CTL Write Priority Count Register	0x00000F0F
0x31080018	L2CTL0_WPCR	L2CTL0 Write Priority Count Register	0x00000F0F
0x31080024	L2CTL_INIT	L2CTL Initialization Register	0x00000000
0x31080024	L2CTL0_INIT	L2CTL0 Initialization Register	0x00000000
0x31080038	L2CTL_ISTAT	L2CTL Initialization Status Register	0x00000000
0x31080038	L2CTL0_ISTAT	L2CTL0 Initialization Status Register	0x00000000
0x3108003C	L2CTL_PCTL	L2CTL Power Control Register	0x00000000
0x3108003C	L2CTL0_PCTL	L2CTL0 Power Control Register	0x00000000
0x31080040	L2CTL_PERRADDR0	L2CTL Parity Error Address 0 Register	0x20000000
0x31080040	L2CTL0_PERRADDR0	L2CTL0 Parity Error Address 0 Register	0x20000000

Table A-53: ADSP-21560_21561_21564_21568 L2CTL MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x31080044	L2CTL_PERRADDR1	L2CTL Parity Error Address 1 Register	0x20020000
0x31080044	L2CTL0_PERRADDR1	L2CTL0 Parity Error Address 1 Register	0x20020000
0x31080048	L2CTL_PERRADDR2	L2CTL Parity Error Address 2 Register	0x20040000
0x31080048	L2CTL0_PERRADDR2	L2CTL0 Parity Error Address 2 Register	0x20040000
0x3108004C	L2CTL_PERRADDR3	L2CTL Parity Error Address 3 Register	0x20060000
0x3108004C	L2CTL0_PERRADDR3	L2CTL0 Parity Error Address 3 Register	0x20060000
0x31080050	L2CTL_PERRADDR4	L2CTL Parity Error Address 4 Register	0x20080000
0x31080050	L2CTL0_PERRADDR4	L2CTL0 Parity Error Address 4 Register	0x20080000
0x31080054	L2CTL_PERRADDR5	L2CTL Parity Error Address 5 Register	0x200A0000
0x31080054	L2CTL0_PERRADDR5	L2CTL0 Parity Error Address 5 Register	0x200A0000
0x31080058	L2CTL_PERRADDR6	L2CTL Parity Error Address 6 Register	0x200C0000
0x31080058	L2CTL0_PERRADDR6	L2CTL0 Parity Error Address 6 Register	0x200C0000
0x3108005C	L2CTL_PERRADDR7	L2CTL Parity Error Address 7 Register	0x200E0000
0x3108005C	L2CTL0_PERRADDR7	L2CTL0 Parity Error Address 7 Register	0x200E0000
0x31080060	L2CTL_PERRADDR8	L2CTL Parity Error Address 8 Register	0x20100000
0x31080060	L2CTL0_PERRADDR8	L2CTL0 Parity Error Address 8 Register	0x20100000
0x31080080	L2CTL_ET0	L2CTL Error Type 0 Register	0x00000000
0x31080080	L2CTL0_ET0	L2CTL0 Error Type 0 Register	0x00000000
0x31080084	L2CTL_EADDR0	L2CTL Error Type 0 Address Register	0x20000000
0x31080084	L2CTL0_EADDR0	L2CTL0 Error Type 0 Address Register	0x20000000
0x31080088	L2CTL_ET1	L2CTL Error Type 1 Register	0x00000000
0x31080088	L2CTL0_ET1	L2CTL0 Error Type 1 Register	0x00000000
0x3108008C	L2CTL_EADDR1	L2CTL Error Type 1 Address Register	0x20000000
0x3108008C	L2CTL0_EADDR1	L2CTL0 Error Type 1 Address Register	0x20000000
0x310800FC	L2CTL_REVID	L2CTL Revision ID Register	0x00000001
0x310800FC	L2CTL0_REVID	L2CTL0 Revision ID Register	0x00000001

Table A-54: ADSP-21560_21561_21564_21568 L2CTL1 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31094000	L2CTL1_CTL	L2CTL1 Control Register	0x00000000

Table A-54: ADSP-21560_21561_21564_21568 L2CTL1 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31094010	L2CTL1_STAT	L2CTL1 Status Register	0x00000000
0x31094024	L2CTL1_INIT	L2CTL1 Initialization Register	0x00000000
0x31094038	L2CTL1_ISTAT	L2CTL1 Initialization Status Register	0x00000000
0x3109403C	L2CTL1_PCTL	L2CTL1 Power Control Register	0x00000000
0x31094040	L2CTL1_PERRADDR0	L2CTL1 Parity Error Address 0 Register	0x80000000
0x31094044	L2CTL1_PERRADDR1	L2CTL1 Parity Error Address 1 Register	0x80020000
0x31094048	L2CTL1_PERRADDR2	L2CTL1 Parity Error Address 2 Register	0x80040000
0x3109404C	L2CTL1_PERRADDR3	L2CTL1 Parity Error Address 3 Register	0x80060000
0x31094050	L2CTL1_PERRADDR4	L2CTL1 Parity Error Address 4 Register	0x80080000
0x31094054	L2CTL1_PERRADDR5	L2CTL1 Parity Error Address 5 Register	0x800A0000
0x31094058	L2CTL1_PERRADDR6	L2CTL1 Parity Error Address 6 Register	0x800C0000
0x3109405C	L2CTL1_PERRADDR7	L2CTL1 Parity Error Address 7 Register	0x800E0000
0x31094060	L2CTL1_PERRADDR8	L2CTL1 Parity Error Address 8 Register	0x80100000
0x31094080	L2CTL1_ET0	L2CTL1 Error Type 0 Register	0x00000000
0x31094084	L2CTL1_EADDR0	L2CTL1 Error Type 0 Address Register	0x80000000
0x31094088	L2CTL1_ET1	L2CTL1 Error Type 1 Register	0x00000000
0x3109408C	L2CTL1_EADDR1	L2CTL1 Error Type 1 Address Register	0x80000000
0x310940FC	L2CTL1_REVID	L2CTL1 Revision ID Register	0x00000001

Table A-55: ADSP-21560_21561_21564_21568 MEC0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310A3000	MEC0_PEIRQ_GCTL[n]	MEC0 Parity Error Interrupt Request Global Control Register	0x00000000
0x310A3010	MEC0_PEIRQ_GSTAT[n]	MEC0 Parity Error Interrupt Request Global Status Register	0x00000000
0x310A3040	MEC0_PERR_CTL	MEC0 Parity Error Control Register	0x00000000
0x310A3080	MEC0_PERR_STAT	MEC0 Parity Error Status Register	0x00000000
0x310A30C0	MEC0_PERR_IMASK	MEC0 Parity Error Interrupt Mask Register	0x00000000
0x310A3100	MEC0_L2PIRQ_GCTL[n]	MEC0 L2 Parity Error Interrupt Request Global Control Register	0x00000000
0x310A3110	MEC0_L2PIRQ_GSTAT[n]	MEC0 L2 Parity Error Interrupt Request Global Status Register	0x00000000

Table A-55: ADSP-21560_21561_21564_21568 MEC0 MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x310A3140	MEC0_L2PERR_CTL[n]	MEC0 L2 Parity Error Control Register	0x00000000
0x310A3180	MEC0_L2PERR_STAT[n]	MEC0 L2 Parity Error Status Register	0x00000000
0x310A31C0	MEC0_L2PERR_IMASK[n]	MEC0 L2 Parity Error Interrupt Mask Register	0x00000000
0x310A3F00	MEC0_CLR	MEC0 Clear Register	0x00000000
0x310A3FD0	MEC0_PID4	MEC0 Peripheral ID4 Register	0x00000000
0x310A3FE0	MEC0_PID0	MEC0 Peripheral ID0 Register	0x00000003
0x310A3FE4	MEC0_PID1	MEC0 Peripheral ID1 Register	0x00000050
0x310A3FE8	MEC0_PID2	MEC0 Peripheral ID2 Register	0x0000000E
0x310A3FEC	MEC0_PID3	MEC0 Peripheral ID3 Register	0x00000000
0x310A3FF0	MEC0_CID0	MEC0 Component ID0 Register	0x0000000D
0x310A3FF4	MEC0_CID1	MEC0 Component ID1 Register	0x000000F0
0x310A3FF8	MEC0_CID2	MEC0 Component ID2 Register	0x00000005
0x310A3FFC	MEC0_CID3	MEC0 Component ID3 Register	0x000000B1

Table A-56: ADSP-21560_21561_21564_21568 MISCREG MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x310A9000	MIS-CREG_PFB_RANGE_SELECT	MISCREG SH0 Prefetch Range Selection Register	0xFFFFFFFF
0x310A9010	MISCREG_CAN_SYSCtl	MISCREG CANFD Low Power Selection Mode	0x00000000
0x310A9028	MISCREG_ECO_REG9	MISCREG Error Register	0x00000000
0x310A9100	MISCREG_XSPI_CTL0	MISCREG XSPI Boot/Init Control Register 0	0x00000002
0x310A9104	MISCREG_XSPI_CTL1	MISCREG XSPI Boot/Init Control Register 1	0x00000000
0x310A9108	MISCREG_XSPI_CTL2	MISCREG XSPI Boot/Init Control Register 2	0x80000101
0x310A910C	MISCREG_XSPI_CTL3	MISCREG XSPI Boot/Init Control Register 3	0x00300404
0x310A9110	MISCREG_XSPI_CTL4	MISCREG XSPI Boot/Init Control Register 4	0x00180030
0x310A9114	MISCREG_XSPI_CTL5	MISCREG XSPI Boot/Init Control Register 5	0x00800004
0x310A9118	MISCREG_XSPI_CTL6	MISCREG XSPI Boot/Init Control Register 6	0x0000357F
0x310A911C	MISCREG_XSPI_CTL7	MISCREG XSPI Boot/Init Control Register 7	0x00000000
0x310A9120	MISCREG_XSPI_CTL8	MISCREG XSPI Boot/Init Control Register 7	0x00000080
0x310A9130	MISCREG_XSPI_RSTCTL	MISCREG XSPI Boot/Init Reset Control Register	0x00000000

Table A-56: ADSP-21560_21561_21564_21568 MISCREG MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x310A9140	MISCREG_XSPI_STAT0	MISCREG XSPI Boot/Init Status Register	0x00000000

Table A-57: ADSP-21560_21561_21564_21568 MLB0 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x3109D000	MLB0_CTL0	MLB0 MediaLB Control 0 Register	0x00000000
0x3109D00C	MLB0_MS0	MLB0 Channel Status 0 Register	0x00000000
0x3109D014	MLB0_MS1	MLB0 Channel Status 1 Register	0x00000000
0x3109D020	MLB0_MSS	MLB0 System Status Register	0x00000000
0x3109D024	MLB0_MSD	MLB0 System Data Register	0x00000000
0x3109D02C	MLB0_MIEN	MLB0 Interrupt Enable Register	0x00000000
0x3109D034	MLB0_GCTL	MLB0 MLB Global Control Register	0x00000000
0x3109D03C	MLB0_CTL1	MLB0 Control 1 Register	0x00000000
0x3109D080	MLB0_HCTL	MLB0 HBI Control Register	0x00000000
0x3109D088	MLB0_HCMR0	MLB0 HBI Channel Mask 0 Register	0x00000000
0x3109D08C	MLB0_HCMR1	MLB0 HBI Channel Mask 1 Register	0x00000000
0x3109D090	MLB0_HCER0	MLB0 HBI Channel Error 0 Register	0x00000000
0x3109D094	MLB0_HCER1	MLB0 HBI Channel Error 1 Register	0x00000000
0x3109D098	MLB0_HCBR0	MLB0 HBI Channel Busy 0 Register	0x00000000
0x3109D09C	MLB0_HCBR1	MLB0 HBI Channel Busy 1 Register	0x00000000
0x3109D0C0	MLB0_MDAT0	MLB0 Memory Interface Control Data 0 Register	0x00000000
0x3109D0C4	MLB0_MDAT1	MLB0 Memory Interface Control Data 1 Register	0x00000000
0x3109D0C8	MLB0_MDAT2	MLB0 Memory Interface Control Data 2 Register	0x00000000
0x3109D0CC	MLB0_MDAT3	MLB0 Memory Interface Control Data 3 Register	0x00000000
0x3109D0D0	MLB0_MDWE0	MLB0 Memory Interface Control Data Write Enable 0 Register	0x00000000
0x3109D0D4	MLB0_MDWE1	MLB0 Memory Interface Control Data Write Enable 1 Register	0x00000000
0x3109D0D8	MLB0_MDWE2	MLB0 Memory Interface Control Data Write Enable 2 Register	0x00000000
0x3109D0DC	MLB0_MDWE3	MLB0 Memory Interface Control Data Write Enable 3 Register	0x00000000
0x3109D0E0	MLB0_MCTL	MLB0 Memory Interface Control Register	0x00000000

Table A-57: ADSP-21560_21561_21564_21568 MLB0 MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x3109D0E4	MLB0_MADR	MLB0 Memory Interface Address Register	0x00000000
0x3109D3C0	MLB0_ACTL	MLB0 Bus Control Register	0x00000000
0x3109D3D0	MLB0_ACSR0	MLB0 Peripheral Channel Status 0 Register	0x00000000
0x3109D3D4	MLB0_ACSR1	MLB0 Peripheral Channel Status 1 Register	0x00000000
0x3109D3D8	MLB0_ACMR0	MLB0 Peripheral Channel Mask 0 Register	0x00000000
0x3109D3DC	MLB0_ACMR1	MLB0 Peripheral Channel Mask 1 Register	0x00000000

Table A-58: ADSP-21560_21561_21564_21568 SCB3 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x30105020	SCB3_APB_DCLK_0_IB_SY NC_MODE	SCB3 APB DCLK 0 Synchronization Mode Register	0x00000004

Table A-59: ADSP-21560_21561_21564_21568 OTPC0 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x62022004	OTPC0_STAT	OTPC0 OTP Status Register	0x00000000
0x6202202C	OTPC0_SECU_STATE	OTPC0 OTP Security State Register	0x00000001

Table A-60: ADSP-21560_21561_21564_21568 PADS0 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31004404	PADS0_PCFG0	PADS0 Peripheral PAD Configuration 0 Register	0x00010000
0x3100440C	PADS0_PORTA0_DS	PADS0 PORTA 0 to 7 pins DS Control Register	0x00492492
0x31004410	PADS0_PORTA1_DS	PADS0 PORTA 8 - 15 pins DS Control Register	0x00492492
0x31004414	PADS0_PORTB0_DS	PADS0 PORTB 0 to 7 pins DS Control Register	0x00012492
0x31004424	PADS0_NONPORTS_DS	PADS0 Non-GPIO Drive Strength Register	0x104D804B
0x31004434	PADS0_DAI0_0_DS	PADS0 DAI0 1 to 10 pins DS Control Register	0x12492492
0x31004438	PADS0_DAI0_1_DS	PADS0 DAI0 11 to 20 pins DS Control Register	0x12000000
0x31004440	PADS0_DAI1_0_DS	PADS0 DAI1 1 to 10 pins DS Control Register	0x12492492
0x31004444	PADS0_DAI1_1_DS	PADS0 DAI1 11 to 20 pins DS Control Register	0x12000000
0x31004460	PADS0_DAI0_IE	PADS0 DAI0 Port Input Enable Control Register	0x00000000
0x31004464	PADS0_DAI1_IE	PADS0 DAI1 Port Input Enable Control Register	0x00000000

Table A-60: ADSP-21560_21561_21564_21568 PADS0 MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x31004480	PADS0_PORT[n]_PUE	PADS0 PORTx Pull-Up Enable Register	0x00000000
0x31004484	PADS0_PORT[n]_PUE	PADS0 PORTx Pull-Up Enable Register	0x00000000
0x31004498	PADS0_DAI[n]_PUE	PADS0 DAIx Pull-Up Enable Register	0x00000000
0x3100449C	PADS0_DAI[n]_PUE	PADS0 DAIx Pull-Up Enable Register	0x00000000
0x310044C0	PADS0_PORT[n]_PDE	PADS0 PORTx Pull-Down Enable Register	0x00000000
0x310044C4	PADS0_PORT[n]_PDE	PADS0 PORTx Pull-Down Enable Register	0x00000000
0x310044D8	PADS0_DAI[n]_PDE	PADS0 DAIx Pull-Down Enable Register	0x00000000
0x310044DC	PADS0_DAI[n]_PDE	PADS0 DAIx Pull-Down Enable Register	0x00000000

Table A-61: ADSP-21560_21561_21564_21568 PCG0 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x310C9300	PCG0_CTLA0	PCG0 Precision Clock A Control 0 Register	0x00000000
0x310C9304	PCG0_CTLA1	PCG0 Precision Clock A Control 1 Register	0x00000000
0x310C9308	PCG0_CTLB0	PCG0 Precision Clock B Control 0 Register	0x00000000
0x310C930C	PCG0_CTLB1	PCG0 Precision Clock B Control 1 Register	0x00000000
0x310C9310	PCG0_PW1	PCG0 Precision Clock Pulse Width Control 1 Register	0x00000000
0x310C9314	PCG0_SYNC1	PCG0 Precision Clock Frame Sync Synchronization 1 Register	0x00000000
0x310CA300	PCG0_CTLC0	PCG0 Precision Clock C Control 0 Register	0x00000000
0x310CA304	PCG0_CTLC1	PCG0 Precision Clock C Control 1 Register	0x00000000
0x310CA308	PCG0_CTLD0	PCG0 Precision Clock D Control 0 Register	0x00000000
0x310CA30C	PCG0_CTLD1	PCG0 Precision Clock D Control 1 Register	0x00000000
0x310CA310	PCG0_PW2	PCG0 Precision Clock Pulse Width Control 2 Register	0x00000000
0x310CA314	PCG0_SYNC2	PCG0 Precision Clock Frame Sync Synchronization 2 Register	0x00000000

Table A-62: ADSP-21560_21561_21564_21568 XSPIFB0 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x310A8000	XSPIFB0_CTL0	XSPIFB0 Control Register 0	0x00000000
0x310A8004	XSPIFB0_CTL1	XSPIFB0 Control Register 1	0x00001107
0x310A8014	XSPIFB0_ZERO_START	XSPIFB0 Return Zero Start Address Register	0x00000000

Table A-62: ADSP-21560_21561_21564_21568 XSPIPFB0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310A8018	XSPIPFB0_ZERO_END	XSPIPFB0 Return Zero End Address Register	0x00000000
0x310A801C	XSPIPFB0_RANGE_SE- LECT	XSPIPFB0 Range Select Register	0x0000FFFF
0x310A8020	XSPIPFB0_STS0	XSPIPFB0 Prefetch Buffer First Status Register	0x00000002
0x310A8050	XSPIPFB0_RQSTR_ID_EN	XSPIPFB0 Requester ID Enable Register	0x0000000F
0x310A8054	XSPIPFB0_RQSTR0_ID	XSPIPFB0 Requester ID 0 Register	0x0000000F
0x310A8058	XSPIPFB0_RQSTR0_ID_MS K	XSPIPFB0 Requester ID 0 Mask Register	0x00000000
0x310A805C	XSPIPFB0_RQSTR1_ID	XSPIPFB0 Requester ID 1 Register	0x0000000F
0x310A8060	XSPIPFB0_RQSTR1_ID_MS K	XSPIPFB0 Requester ID 1 Mask Register	0x00000000
0x310A8064	XSPIPFB0_RQSTR2_ID	XSPIPFB0 Requester ID 2 Register	0x0000000F
0x310A8068	XSPIPFB0_RQSTR2_ID_MS K	XSPIPFB0 Requester ID 2 Mask Register	0x00000000
0x310A806C	XSPIPFB0_RQSTR3_ID	XSPIPFB0 Requester ID 3 Register	0x0000000F
0x310A8070	XSPIPFB0_RQSTR3_ID_MS K	XSPIPFB0 Requester ID 3 Mask Register	0x00000000

Table A-63: ADSP-21560_21561_21564_21568 PINT0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31005000	PINT0_MSK_SET	PINT0 PINT Mask Set Register	0x00000000
0x31005004	PINT0_MSK_CLR	PINT0 PINT Mask Clear Register	0x00000000
0x31005008	PINT0_REQ	PINT0 PINT Request Register	0x00000000
0x3100500C	PINT0_ASSIGN	PINT0 PINT Assign Register	0x00000101
0x31005010	PINT0_EDGE_SET	PINT0 PINT Edge Set Register	0x00000000
0x31005014	PINT0_EDGE_CLR	PINT0 PINT Edge Clear Register	0x00000000
0x31005018	PINT0_INV_SET	PINT0 PINT Invert Set Register	0x00000000
0x3100501C	PINT0_INV_CLR	PINT0 PINT Invert Clear Register	0x00000000
0x31005020	PINT0_PINSTATE	PINT0 PINT Pin State Register	0x00000000
0x31005024	PINT0_LATCH	PINT0 PINT Latch Register	0x00000000

Table A-64: ADSP-21560_21561_21564_21568 PINT1 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31005100	PINT1_MSK_SET	PINT1 PINT Mask Set Register	0x00000000
0x31005104	PINT1_MSK_CLR	PINT1 PINT Mask Clear Register	0x00000000
0x31005108	PINT1_REQ	PINT1 PINT Request Register	0x00000000
0x3100510C	PINT1_ASSIGN	PINT1 PINT Assign Register	0x00000101
0x31005110	PINT1_EDGE_SET	PINT1 PINT Edge Set Register	0x00000000
0x31005114	PINT1_EDGE_CLR	PINT1 PINT Edge Clear Register	0x00000000
0x31005118	PINT1_INV_SET	PINT1 PINT Invert Set Register	0x00000000
0x3100511C	PINT1_INV_CLR	PINT1 PINT Invert Clear Register	0x00000000
0x31005120	PINT1_PINSTATE	PINT1 PINT Pin State Register	0x00000000
0x31005124	PINT1_LATCH	PINT1 PINT Latch Register	0x00000000

Table A-65: ADSP-21560_21561_21564_21568 PKA0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310D4000	PKA0_APTR	PKA0 PKA Vector_A Address	0x00000000
0x310D4004	PKA0_BPTR	PKA0 PKA Vector_B Address	0x00000000
0x310D4008	PKA0_CPTR	PKA0 PKA Vector_C Address	0x00000000
0x310D400C	PKA0_DPTR	PKA0 PKA Vector_D Address	0x00000000
0x310D4010	PKA0_ALEN	PKA0 PKA Vector_A Length	0x00000000
0x310D4014	PKA0_BLEN	PKA0 PKA Vector_B Length	0x00000000
0x310D4018	PKA0_SHIFT	PKA0 PKA Bit Shift Value	0x00000000
0x310D401C	PKA0_FUNC	PKA0 PKA Function	0x00000000
0x310D4020	PKA0_COMPARE	PKA0 PKA Compare Result	0x00000001
0x310D4024	PKA0_RESULTMSW	PKA0 PKA Most-Significant-Word of Result Vector	0x00008000
0x310D4028	PKA0_DIVMSW	PKA0 PKA Most-Significant-Word of Divide Remainder	0x00008000
0x310D6000	PKA0_RAM	PKA0 Start of PKA RAM space	0x00000000

Table A-66: ADSP-21560_21561_21564_21568 PKIC0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310D8000	PKIC0_POL_CTL	PKIC0 Polarity Control Register	0x00000000
0x310D8004	PKIC0_TYPE_CTL	PKIC0 Type Control Register	0x00000000

Table A-66: ADSP-21560_21561_21564_21568 PKIC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310D8008	PKIC0_EN_CTL	PKIC0 Enable Control Register	0x00000000
0x310D800C	PKIC0_RAW_STAT	PKIC0 Raw Status Register	0x00000000
0x310D800C	PKIC0_EN_SET	PKIC0 Enable Set Register	0x00000000
0x310D8010	PKIC0_ACK	PKIC0 Acknowledge Register	0x00000000
0x310D8010	PKIC0_EN_STAT	PKIC0 Enabled Status Register	0x00000000
0x310D8014	PKIC0_EN_CLR	PKIC0 Enable Clear Register	0x00000000

Table A-67: ADSP-21560_21561_21564_21568 PKTE0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310CD000	PKTE0_CTL_STAT	PKTE0 Packet Engine Control Register	0x00000002
0x310CD004	PKTE0_SRC_ADDR	PKTE0 Packet Engine Source Address	0x00000000
0x310CD008	PKTE0_DEST_ADDR	PKTE0 Packet Engine Destination Address	0x00000000
0x310CD00C	PKTE0_SA_ADDR	PKTE0 Packet Engine SA Address	0x00000000
0x310CD010	PKTE0_STATE_ADDR	PKTE0 Packet Engine State Record Address	0x00000000
0x310CD014	PKTE0_ARC4STATE_ADDR	PKTE0 Packet Engine ARC4 State Record Address	0x00000000
0x310CD018	PKTE0_USERID	PKTE0 Packet Engine User ID	0x00000000
0x310CD01C	PKTE0_LEN	PKTE0 Packet Engine Length Register	0x00800000
0x310CD080	PKTE0_CDRBASE_ADDR	PKTE0 Packet Engine Command Descriptor Ring Base Address	0x00000000
0x310CD084	PKTE0_RDRBASE_ADDR	PKTE0 Packet Engine Result Descriptor Ring Base Address	0x00000000
0x310CD088	PKTE0_RING_CFG	PKTE0 Packet Engine Ring Configuration	0x00000000
0x310CD08C	PKTE0_RING_THRESH	PKTE0 Packet Engine Ring Threshold Registers	0x00000000
0x310CD090	PKTE0_CDSC_INCR	PKTE0 Packet Engine Command Descriptor Count Increment Register	0x00000000
0x310CD090	PKTE0_CDSC_CNT	PKTE0 Packet Engine Command Descriptor Count Register	0x00000000
0x310CD094	PKTE0_RDSC_CNT	PKTE0 Packet Engine Result Descriptor Count Registers	0x00000000
0x310CD094	PKTE0_RDSC_DECR	PKTE0 Packet Engine Result Descriptor Count Decrement Registers	0x00000000
0x310CD098	PKTE0_RING_PTR	PKTE0 Packet Engine Ring Pointer Status	0x00000000

Table A-67: ADSP-21560_21561_21564_21568 PKTE0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310CD09C	PKTE0_RING_STAT	PKTE0 Packet Engine Ring Status	0x00000000
0x310CD100	PKTE0_CFG	PKTE0 Packet Engine Configuration Register	0x00000000
0x310CD104	PKTE0_STAT	PKTE0 Packet Engine Status Register	0x00040402
0x310CD10C	PKTE0_BUF_THRESH	PKTE0 Packet Engine Buffer Threshold Register	0x00800080
0x310CD110	PKTE0_INBUF_INCR	PKTE0 Packet Engine Input Buffer Count Increment Register	0x00000000
0x310CD110	PKTE0_INBUF_CNT	PKTE0 Packet Engine Input Buffer Count Register	0x00000000
0x310CD114	PKTE0_OUTBUF_CNT	PKTE0 Packet Engine Output Buffer Count Register	0x00000000
0x310CD114	PKTE0_OUTBUF_DECR	PKTE0 Packet Engine Output Buffer Count Decrement Register	0x00000000
0x310CD118	PKTE0_BUF_PTR	PKTE0 Packet Engine Buffer Pointer Register	0x00000000
0x310CD120	PKTE0_DMA_CFG	PKTE0 Packet Engine DMA Configuration Register	0x00180006
0x310CD1D0	PKTE0_ENDIAN_CFG	PKTE0 Packet Engine Endian Configuration Register	0x00E400E4
0x310CD1E0	PKTE0_HLT_CTL	PKTE0 Packet Engine Halt Control Register	0x00000000
0x310CD1E0	PKTE0_HLT_STAT	PKTE0 Packet Engine Halt Status Register	0x00000000
0x310CD1E4	PKTE0_CONT	PKTE0 PKTE Continue Register	0x00000000
0x310CD1E8	PKTE0_CLK_CTL	PKTE0 PE Clock Control Register	0x0000001F
0x310CD200	PKTE0_IUMSK_STAT	PKTE0 Interrupt Unmasked Status Register	0x00000000
0x310CD204	PKTE0_IMSK_STAT	PKTE0 Interrupt Masked Status Register	0x00000000
0x310CD204	PKTE0_INT_CLR	PKTE0 Interrupt Clear Register	0x00000000
0x310CD208	PKTE0_INT_EN	PKTE0 Interrupt Enable Register	0x00000000
0x310CD20C	PKTE0_INT_CFG	PKTE0 Interrupt Configuration Register	0x00000000
0x310CD210	PKTE0_IMSK_EN	PKTE0 Interrupt Mask Enable Register	0x00000000
0x310CD214	PKTE0_IMSK_DIS	PKTE0 Interrupt Mask Disable Register	0x00000000
0x310CD400	PKTE0_SA_CMD0	PKTE0 SA Command 0	0x00000000
0x310CD404	PKTE0_SA_CMD1	PKTE0 SA Command 1	0x00000000
0x310CD408	PKTE0_SA_KEY[n]	PKTE0 SA Key Registers	0x00000000
0x310CD40C	PKTE0_SA_KEY[n]	PKTE0 SA Key Registers	0x00000000
0x310CD410	PKTE0_SA_KEY[n]	PKTE0 SA Key Registers	0x00000000
0x310CD414	PKTE0_SA_KEY[n]	PKTE0 SA Key Registers	0x00000000
0x310CD418	PKTE0_SA_KEY[n]	PKTE0 SA Key Registers	0x00000000

Table A-67: ADSP-21560_21561_21564_21568 PKTE0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310CD41C	PKTE0_SA_KEY[n]	PKTE0 SA Key Registers	0x00000000
0x310CD420	PKTE0_SA_KEY[n]	PKTE0 SA Key Registers	0x00000000
0x310CD424	PKTE0_SA_KEY[n]	PKTE0 SA Key Registers	0x00000000
0x310CD428	PKTE0_SA_IDIGEST[n]	PKTE0 SA Inner Hash Digest Registers	0x00000000
0x310CD42C	PKTE0_SA_IDIGEST[n]	PKTE0 SA Inner Hash Digest Registers	0x00000000
0x310CD430	PKTE0_SA_IDIGEST[n]	PKTE0 SA Inner Hash Digest Registers	0x00000000
0x310CD434	PKTE0_SA_IDIGEST[n]	PKTE0 SA Inner Hash Digest Registers	0x00000000
0x310CD438	PKTE0_SA_IDIGEST[n]	PKTE0 SA Inner Hash Digest Registers	0x00000000
0x310CD43C	PKTE0_SA_IDIGEST[n]	PKTE0 SA Inner Hash Digest Registers	0x00000000
0x310CD440	PKTE0_SA_IDIGEST[n]	PKTE0 SA Inner Hash Digest Registers	0x00000000
0x310CD444	PKTE0_SA_IDIGEST[n]	PKTE0 SA Inner Hash Digest Registers	0x00000000
0x310CD448	PKTE0_SA_ODIGEST[n]	PKTE0 SA Outer Hash Digest Registers	0x00000000
0x310CD44C	PKTE0_SA_ODIGEST[n]	PKTE0 SA Outer Hash Digest Registers	0x00000000
0x310CD450	PKTE0_SA_ODIGEST[n]	PKTE0 SA Outer Hash Digest Registers	0x00000000
0x310CD454	PKTE0_SA_ODIGEST[n]	PKTE0 SA Outer Hash Digest Registers	0x00000000
0x310CD458	PKTE0_SA_ODIGEST[n]	PKTE0 SA Outer Hash Digest Registers	0x00000000
0x310CD45C	PKTE0_SA_ODIGEST[n]	PKTE0 SA Outer Hash Digest Registers	0x00000000
0x310CD460	PKTE0_SA_ODIGEST[n]	PKTE0 SA Outer Hash Digest Registers	0x00000000
0x310CD464	PKTE0_SA_ODIGEST[n]	PKTE0 SA Outer Hash Digest Registers	0x00000000
0x310CD468	PKTE0_SA_SPI	PKTE0 SA SPI Register	0x00000000
0x310CD46C	PKTE0_SA_SEQNUM[n]	PKTE0 SA Sequence Number Register	0x00000000
0x310CD470	PKTE0_SA_SEQNUM[n]	PKTE0 SA Sequence Number Register	0x00000000
0x310CD474	PKTE0_SA_SEQ- NUM_MSK[n]	PKTE0 SA Sequence Number Mask Registers	0x00000000
0x310CD478	PKTE0_SA_SEQ- NUM_MSK[n]	PKTE0 SA Sequence Number Mask Registers	0x00000000
0x310CD47C	PKTE0_SA_ARC4IJPTR	PKTE0 ARC4 i and j Pointer Register	0x00000000
0x310CD47C	PKTE0_SA_RDY	PKTE0 SA Ready Indicator	0x00000000
0x310CD47C	PKTE0_SA_NONCE	PKTE0 SA Initialization Vector Register	0x00000000
0x310CD500	PKTE0_STATE_IV[n]	PKTE0 State Initialization Vector Registers	0x00000000
0x310CD504	PKTE0_STATE_IV[n]	PKTE0 State Initialization Vector Registers	0x00000000

Table A-67: ADSP-21560_21561_21564_21568 PKTE0 MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x310CD508	PKTE0_STATE_IV[n]	PKTE0 State Initialization Vector Registers	0x00000000
0x310CD50C	PKTE0_STATE_IV[n]	PKTE0 State Initialization Vector Registers	0x00000000
0x310CD510	PKTE0_STATE_BYTE_CN T[n]	PKTE0 State Hash Byte Count Registers	0x00000000
0x310CD514	PKTE0_STATE_BYTE_CN T[n]	PKTE0 State Hash Byte Count Registers	0x00000000
0x310CD518	PKTE0_STATE_IDIGEST[n]	PKTE0 State Inner Digest Registers	0x00000000
0x310CD51C	PKTE0_STATE_IDIGEST[n]	PKTE0 State Inner Digest Registers	0x00000000
0x310CD520	PKTE0_STATE_IDIGEST[n]	PKTE0 State Inner Digest Registers	0x00000000
0x310CD524	PKTE0_STATE_IDIGEST[n]	PKTE0 State Inner Digest Registers	0x00000000
0x310CD528	PKTE0_STATE_IDIGEST[n]	PKTE0 State Inner Digest Registers	0x00000000
0x310CD52C	PKTE0_STATE_IDIGEST[n]	PKTE0 State Inner Digest Registers	0x00000000
0x310CD530	PKTE0_STATE_IDIGEST[n]	PKTE0 State Inner Digest Registers	0x00000000
0x310CD534	PKTE0_STATE_IDIGEST[n]	PKTE0 State Inner Digest Registers	0x00000000
0x310CD700	PKTE0_ARC4STATE_BUF	PKTE0 Starting Entry of 256-byte ARC4 State Buffer	0x00000000
0x310CD800	PKTE0_DATAIO_BUF	PKTE0 Starting Entry of 256-byte Data Input/Output Buffer	0x00000000

Table A-68: ADSP-21560_21561_21564_21568 PORT MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31004000	PORT_FER	PORT Port x Function Enable Register	0x00000000
0x31004000	PORT0_FER	PORT0 Port x Function Enable Register	0x00000000
0x31004004	PORT_FER_SET	PORT Port x Function Enable Set Register	0x00000000
0x31004004	PORT0_FER_SET	PORT0 Port x Function Enable Set Register	0x00000000
0x31004008	PORT_FER_CLR	PORT Port x Function Enable Clear Register	0x00000000
0x31004008	PORT0_FER_CLR	PORT0 Port x Function Enable Clear Register	0x00000000
0x3100400C	PORT_DATA	PORT Port x GPIO Data Register	0x00000000
0x3100400C	PORT0_DATA	PORT0 Port x GPIO Data Register	0x00000000
0x31004010	PORT_DATA_SET	PORT Port x GPIO Data Set Register	0x00000000
0x31004010	PORT0_DATA_SET	PORT0 Port x GPIO Data Set Register	0x00000000
0x31004014	PORT_DATA_CLR	PORT Port x GPIO Data Clear Register	0x00000000

Table A-68: ADSP-21560_21561_21564_21568 PORT MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31004014	PORT0_DATA_CLR	PORT0 Port x GPIO Data Clear Register	0x00000000
0x31004018	PORT_DIR	PORT Port x GPIO Direction Register	0x00000000
0x31004018	PORT0_DIR	PORT0 Port x GPIO Direction Register	0x00000000
0x3100401C	PORT_DIR_SET	PORT Port x GPIO Direction Set Register	0x00000000
0x3100401C	PORT0_DIR_SET	PORT0 Port x GPIO Direction Set Register	0x00000000
0x31004020	PORT_DIR_CLR	PORT Port x GPIO Direction Clear Register	0x00000000
0x31004020	PORT0_DIR_CLR	PORT0 Port x GPIO Direction Clear Register	0x00000000
0x31004024	PORT_INEN	PORT Port x GPIO Input Enable Register	0x00000000
0x31004024	PORT0_INEN	PORT0 Port x GPIO Input Enable Register	0x00000000
0x31004028	PORT_INEN_SET	PORT Port x GPIO Input Enable Set Register	0x00000000
0x31004028	PORT0_INEN_SET	PORT0 Port x GPIO Input Enable Set Register	0x00000000
0x3100402C	PORT_INEN_CLR	PORT Port x GPIO Input Enable Clear Register	0x00000000
0x3100402C	PORT0_INEN_CLR	PORT0 Port x GPIO Input Enable Clear Register	0x00000000
0x31004030	PORT_MUX	PORT Port x Multiplexer Control Register	0x00000000
0x31004030	PORT0_MUX	PORT0 Port x Multiplexer Control Register	0x00000000
0x31004034	PORT_DATA_TGL	PORT Port x GPIO Output Toggle Register	0x00000000
0x31004034	PORT0_DATA_TGL	PORT0 Port x GPIO Output Toggle Register	0x00000000
0x31004038	PORT_POL	PORT Port x GPIO Polarity Invert Register	0x00000000
0x31004038	PORT0_POL	PORT0 Port x GPIO Polarity Invert Register	0x00000000
0x3100403C	PORT_POL_SET	PORT Port x GPIO Polarity Invert Set Register	0x00000000
0x3100403C	PORT0_POL_SET	PORT0 Port x GPIO Polarity Invert Set Register	0x00000000
0x31004040	PORT_POL_CLR	PORT Port x GPIO Polarity Invert Clear Register	0x00000000
0x31004040	PORT0_POL_CLR	PORT0 Port x GPIO Polarity Invert Clear Register	0x00000000
0x31004044	PORT_LOCK	PORT Port x GPIO Lock Register	0x00000000
0x31004044	PORT0_LOCK	PORT0 Port x GPIO Lock Register	0x00000000
0x31004048	PORT_TRIG_TGL	PORT Port x GPIO Trigger Toggle Register	0x00000000
0x31004048	PORT0_TRIG_TGL	PORT0 Port x GPIO Trigger Toggle Register	0x00000000

Table A-69: ADSP-21560_21561_21564_21568 PORT0 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31004080	PORT0_FER	PORT0 Port x Function Enable Register	0x00000000
0x31004084	PORT0_FER_SET	PORT0 Port x Function Enable Set Register	0x00000000
0x31004088	PORT0_FER_CLR	PORT0 Port x Function Enable Clear Register	0x00000000
0x3100408C	PORT0_DATA	PORT0 Port x GPIO Data Register	0x00000000
0x31004090	PORT0_DATA_SET	PORT0 Port x GPIO Data Set Register	0x00000000
0x31004094	PORT0_DATA_CLR	PORT0 Port x GPIO Data Clear Register	0x00000000
0x31004098	PORT0_DIR	PORT0 Port x GPIO Direction Register	0x00000000
0x3100409C	PORT0_DIR_SET	PORT0 Port x GPIO Direction Set Register	0x00000000
0x310040A0	PORT0_DIR_CLR	PORT0 Port x GPIO Direction Clear Register	0x00000000
0x310040A4	PORT0_INEN	PORT0 Port x GPIO Input Enable Register	0x00000000
0x310040A8	PORT0_INEN_SET	PORT0 Port x GPIO Input Enable Set Register	0x00000000
0x310040AC	PORT0_INEN_CLR	PORT0 Port x GPIO Input Enable Clear Register	0x00000000
0x310040B0	PORT0_MUX	PORT0 Port x Multiplexer Control Register	0x00000000
0x310040B4	PORT0_DATA_TGL	PORT0 Port x GPIO Output Toggle Register	0x00000000
0x310040B8	PORT0_POL	PORT0 Port x GPIO Polarity Invert Register	0x00000000
0x310040BC	PORT0_POL_SET	PORT0 Port x GPIO Polarity Invert Set Register	0x00000000
0x310040C0	PORT0_POL_CLR	PORT0 Port x GPIO Polarity Invert Clear Register	0x00000000
0x310040C4	PORT0_LOCK	PORT0 Port x GPIO Lock Register	0x00000000
0x310040C8	PORT0_TRIG_TGL	PORT0 Port x GPIO Trigger Toggle Register	0x00000000

Table A-70: ADSP-21560_21561_21564_21568 RCU0 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x3108C000	RCU0_CTL	RCU0 Control Register	0x00000700
0x3108C004	RCU0_STAT	RCU0 Status Register	0x00000021
0x3108C008	RCU0_CRCTL	RCU0 Core Reset Outputs Control Register	0x00000000
0x3108C00C	RCU0_CRSTAT	RCU0 Core Reset Outputs Status Register	0x00000000
0x3108C018	RCU0_SRRQSTAT	RCU0 System Reset Request Status Register	0x00000000
0x3108C024	RCU0_SVECT_LCK	RCU0 SVECT Lock Register	0x00000000
0x3108C028	RCU0_BCODE	RCU0 Boot Code Register	0x00000000
0x3108C02C	RCU0_SVECT0	RCU0 Software Vector Register 0	0x00500004

Table A-70: ADSP-21560_21561_21564_21568 RCU0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3108C06C	RCU0_MSG	RCU0 Message Register	0x00000000
0x3108C070	RCU0_MSG_SET	RCU0 Message Set Bits Register	0x00000000
0x3108C074	RCU0_MSG_CLR	RCU0 Message Clear Bits Register	0x00000000

Table A-71: ADSP-21560_21561_21564_21568 SCB0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x30042100	SCB0_SP0A_READ_QOS	SCB0 SP0A Read Quality of Service Register	0x0000000C
0x30042104	SCB0_SP0A_WRITE_QOS	SCB0 SP0A Write Quality of Service Register	0x0000000C
0x30043100	SCB0_SP0B_READ_QOS	SCB0 SP0B Read Quality of Service Register	0x0000000C
0x30043104	SCB0_SP0B_WRITE_QOS	SCB0 SP0B Write Quality of Service Register	0x0000000C
0x30044100	SCB0_SP1A_READ_QOS	SCB0 SP1A Read Quality of Service Register	0x0000000C
0x30044104	SCB0_SP1A_WRITE_QOS	SCB0 SP1A Write Quality of Service Register	0x0000000C
0x30045100	SCB0_SP1B_READ_QOS	SCB0 SP1B Read Quality of Service Register	0x0000000C
0x30045104	SCB0_SP1B_WRITE_QOS	SCB0 SP1B Write Quality of Service Register	0x0000000C
0x30046100	SCB0_SP2A_READ_QOS	SCB0 SP2A Read Quality of Service Register	0x0000000C
0x30046104	SCB0_SP2A_WRITE_QOS	SCB0 SP2A Write Quality of Service Register	0x0000000C
0x30047100	SCB0_SP2B_READ_QOS	SCB0 SP2B Read Quality of Service Register	0x0000000C
0x30047104	SCB0_SP2B_WRITE_QOS	SCB0 SP2B Write Quality of Service Register	0x0000000C
0x30048100	SCB0_SP3A_READ_QOS	SCB0 SP3A Read Quality of Service Register	0x0000000C
0x30048104	SCB0_SP3A_WRITE_QOS	SCB0 SP3A Write Quality of Service Register	0x0000000C
0x30049100	SCB0_SP3B_READ_QOS	SCB0 SP3B Read Quality of Service Register	0x0000000C
0x30049104	SCB0_SP3B_WRITE_QOS	SCB0 SP3B Write Quality of Service Register	0x0000000C
0x3004A100	SCB0_CRC0_CH0_READ_QOS	SCB0 CRC0 Channel 0 Read Quality of Service Register	0x00000001
0x3004A104	SCB0_CRC0_CH0_WRITE_QOS	SCB0 CRC0 Channel 0 Write Quality of Service Register	0x00000001
0x3004B100	SCB0_CRC0_CH1_READ_QOS	SCB0 CRC0 Channel 1 Read Quality of Service Register	0x00000001
0x3004B104	SCB0_CRC0_CH1_WRITE_QOS	SCB0 CRC0 Channel 1 Write Quality of Service Register	0x00000001
0x3004C100	SCB0_CRC1_CH0_READ_QOS	SCB0 CRC1 Channel 0 Read Quality of Service Register	0x00000001

Table A-71: ADSP-21560_21561_21564_21568 SCB0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3004C104	SCB0_CRC1_CH0_WRITE_QOS	SCB0 CRC1 Channel 0 Write Quality of Service Register	0x00000001
0x3004D100	SCB0_CRC1_CH1_READ_QOS	SCB0 CRC1 Channel 1 Read Quality of Service Register	0x00000001
0x3004D104	SCB0_CRC1_CH1_WRITE_QOS	SCB0 CRC1 Channel 1 Write Quality of Service Register	0x00000001
0x3004E100	SCB0_UART2_RX_READ_QOS	SCB0 UART2 RX Read Quality of Service Register	0x0000000A
0x3004E104	SCB0_UART2_RX_WRITE_QOS	SCB0 UART2 RX Write Quality of Service Register	0x0000000A
0x3004F100	SCB0_SH0_DPORT_READ_QOS	SCB0 SH0 DPORT Read Quality of Service Register	0x00000007
0x3004F104	SCB0_SH0_DPORT_WRITE_QOS	SCB0 SH0 DPORT Write Quality of Service Register	0x00000007
0x30050100	SCB0_SH0_IPORT_READ_QOS	SCB0 SH0 IPORT Read Quality of Service Register	0x00000007
0x30050104	SCB0_SH0_IPORT_WRITE_QOS	SCB0 SH0 IPORT Write Quality of Service Register	0x00000007
0x30051100	SCB0_SP4A_READ_QOS	SCB0 SP4A Read Quality of Service Register	0x0000000C
0x30051104	SCB0_SP4A_WRITE_QOS	SCB0 SP4A Write Quality of Service Register	0x0000000C
0x30052100	SCB0_SP4B_READ_QOS	SCB0 SP4B Read Quality of Service Register	0x0000000C
0x30052104	SCB0_SP4B_WRITE_QOS	SCB0 SP4B Write Quality of Service Register	0x0000000C
0x30055100	SCB0_HSMDMA_CH0_READ_QOS	SCB0 HSMDMA Channel 0 Read Quality of Service Register	0x00000001
0x30055104	SCB0_HSMDMA_CH0_WRITE_QOS	SCB0 HSMDMA Channel 0 Write Quality of Service Register	0x00000001
0x30056100	SCB0_SP5A_READ_QOS	SCB0 SP5A Read Quality of Service Register	0x0000000C
0x30056104	SCB0_SP5A_WRITE_QOS	SCB0 SP5A Write Quality of Service Register	0x0000000C
0x30057100	SCB0_MLB_READ_QOS	SCB0 MLB Read Quality of Service Register	0x0000000C
0x30057104	SCB0_MLB_WRITE_QOS	SCB0 MLB Write Quality of Service Register	0x0000000C
0x30058100	SCB0_UART0_TX_READ_QOS	SCB0 UART0 TX Read Quality of Service Register	0x0000000A
0x30058104	SCB0_UART0_TX_WRITE_QOS	SCB0 UART0 TX Write Quality of Service Register	0x0000000A

Table A-71: ADSP-21560_21561_21564_21568 SCB0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x30059100	SCB0_UART0_RX_READ_QOS	SCB0 UART0 RX Read Quality of Service Register	0x0000000A
0x30059104	SCB0_UART0_RX_WRITE_QOS	SCB0 UART0 RX Write Quality of Service Register	0x0000000A
0x3005A100	SCB0_SP5B_READ_QOS	SCB0 SP5B Read Quality of Service Register	0x0000000C
0x3005A104	SCB0_SP5B_WRITE_QOS	SCB0 SP5B Write Quality of Service Register	0x0000000C
0x3005B100	SCB0_HSMDMA_CH1_READ_QOS	SCB0 HSMDMA Channel 1 Read Quality of Service Register	0x00000001
0x3005B104	SCB0_HSMDMA_CH1_WRITE_QOS	SCB0 HSMDMA Channel 1 Write Quality of Service Register	0x00000001
0x3005C100	SCB0_UART2_TX_READ_QOS	SCB0 UART2 TX Read Quality of Service Register	0x0000000A
0x3005C104	SCB0_UART2_TX_WRITE_QOS	SCB0 UART2 TX Write Quality of Service Register	0x0000000A
0x3005D100	SCB0_SPI0TX_READ_QOS	SCB0 SPI0 TX Read Quality of Service Register	0x0000000C
0x3005D104	SCB0_SPI0TX_WRITE_QOS	SCB0 SPI0 TX Write Quality of Service Register	0x0000000C
0x3005E100	SCB0_SPI0RX_READ_QOS	SCB0 SPI0 RX Read Quality of Service Register	0x0000000C
0x3005E104	SCB0_SPI0RX_WRITE_QOS	SCB0 SPI0 RX Write Quality of Service Register	0x0000000C
0x3005F100	SCB0_SPI1TX_READ_QOS	SCB0 SPI1 TX Read Quality of Service Register	0x0000000C
0x3005F104	SCB0_SPI1TX_WRITE_QOS	SCB0 SPI1 TX Write Quality of Service Register	0x0000000C
0x30060100	SCB0_SPI1RX_READ_QOS	SCB0 SPI1 RX Read Quality of Service Register	0x0000000C
0x30060104	SCB0_SPI1RX_WRITE_QOS	SCB0 SPI1 RX Write Quality of Service Register	0x0000000C
0x30061100	SCB0_SPI2TX_READ_QOS	SCB0 SPI2 TX Read Quality of Service Register	0x0000000C
0x30061104	SCB0_SPI2TX_WRITE_QOS	SCB0 SPI2 TX Write Quality of Service Register	0x0000000C
0x30062100	SCB0_SPI2RX_READ_QOS	SCB0 SPI2 RX Read Quality of Service Register	0x0000000C
0x30062104	SCB0_SPI2RX_WRITE_QOS	SCB0 SPI2 RX Write Quality of Service Register	0x0000000C
0x30063100	SCB0_SP6A_READ_QOS	SCB0 SP6A Read Quality of Service Register	0x0000000C
0x30063104	SCB0_SP6A_WRITE_QOS	SCB0 SP6A Write Quality of Service Register	0x0000000C
0x30064100	SCB0_SP6B_READ_QOS	SCB0 SP6B Read Quality of Service Register	0x0000000C

Table A-71: ADSP-21560_21561_21564_21568 SCB0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x30064104	SCB0_SP6B_WRITE_QOS	SCB0 SP6B Write Quality of Service Register	0x0000000C
0x30065100	SCB0_LP0_READ_QOS	SCB0 LP0 Read Quality of Service Register	0x00000001
0x30065104	SCB0_LP0_WRITE_QOS	SCB0 LP0 Write Quality of Service Register	0x00000001
0x30066100	SCB0_SP7A_READ_QOS	SCB0 SP7A Read Quality of Service Register	0x0000000C
0x30066104	SCB0_SP7A_WRITE_QOS	SCB0 SP7A Write Quality of Service Register	0x0000000C
0x30067100	SCB0_SP7B_READ_QOS	SCB0 SP7B Read Quality of Service Register	0x0000000C
0x30067104	SCB0_SP7B_WRITE_QOS	SCB0 SP7B Write Quality of Service Register	0x0000000C
0x30068100	SCB0_SH0_MMR_IB_READ_QOS	SCB0 SH0 MMR Read Quality of Service Register	0x00000007
0x30068104	SCB0_SH0_MMR_IB_WRITE_QOS	SCB0 SH0 MMR Write Quality of Service Register	0x00000007
0x3006A100	SCB0_UART1_TX_READ_QOS	SCB0 UART1 TX Read Quality of Service Register	0x0000000A
0x3006A104	SCB0_UART1_TX_WRITE_QOS	SCB0 UART1 TX Write Quality of Service Register	0x0000000A
0x3006B100	SCB0_UART1_RX_READ_QOS	SCB0 UART1 RX Read Quality of Service Register	0x0000000A
0x3006B104	SCB0_UART1_RX_WRITE_QOS	SCB0 UART1 RX Write Quality of Service Register	0x0000000A
0x3006D100	SCB0_LP1_READ_QOS	SCB0 LP1 Read Quality of Service Register	0x00000001
0x3006D104	SCB0_LP1_WRITE_QOS	SCB0 LP1 Write Quality of Service Register	0x00000001
0x30071100	SCB0_CRYPTO_READ_QOS	SCB0 Crypto Read Quality of Service Register	0x00000001
0x30071104	SCB0_CRYPTO_WRITE_QOS	SCB0 CRYPTO Write Quality of Service Register	0x00000001
0x30074100	SCB0_ACC_CH0_READ_QOS	SCB0 Accelerator Channel 0 Read Quality of Service Register	0x00000001
0x30074104	SCB0_ACC_CH0_WRITE_QOS	SCB0 Accelerator Channel 0 Write Quality of Service Register	0x00000001
0x30075100	SCB0_ACC_CH1_READ_QOS	SCB0 Accelerator Channel 1 Read Quality of Service Register	0x00000001
0x30075104	SCB0_ACC_CH1_WRITE_QOS	SCB0 Accelerator Channel 1 Write Quality of Service Register	0x00000001
0x30078100	SCB0_DLDMA0_CH0_READ_QOS	SCB0 DLDMA0 Channel 0 Read Quality of Service Register	0x00000001

Table A-71: ADSP-21560_21561_21564_21568 SCB0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x30078104	SCB0_DLDMA0_CH0_WRI TE_QOS	SCB0 DLDMA0 Channel 0 Write Quality of Service Reg- ister	0x00000001
0x30079100	SCB0_DLDMA0_CH1_REA D_QOS	SCB0 DLDMA0 Channel 1 Read Quality of Service Reg- ister	0x00000001
0x30079104	SCB0_DLDMA0_CH1_WRI TE_QOS	SCB0 DLDMA0 Channel 1 Write Quality of Service Reg- ister	0x00000001
0x3007A100	SCB0_DLDMA1_CH0_REA D_QOS	SCB0 DLDMA1 Channel 0 Read Quality of Service Reg- ister	0x00000001
0x3007A104	SCB0_DLDMA1_CH0_WRI TE_QOS	SCB0 DLDMA1 Channel 0 Write Quality of Service Reg- ister	0x00000001
0x3007B100	SCB0_DLDMA1_CH1_REA D_QOS	SCB0 DLDMA1 Channel 1 Read Quality of Service Reg- ister	0x00000001
0x3007B104	SCB0_DLDMA1_CH1_WRI TE_QOS	SCB0 DLDMA1 Channel 1 Write Quality of Service Reg- ister	0x00000001
0x3007C100	SCB0_MSMDMA_CH0_RE AD_QOS	SCB0 MSMDMA Channel 0 Read Quality of Service Register	0x00000001
0x3007C104	SCB0_MSMDMA_CH0_W RITE_QOS	SCB0 MSMDMA Channel 0 Write Quality of Service Register	0x00000001
0x3007D100	SCB0_MSMDMA_CH1_RE AD_QOS	SCB0 MSMDMA Channel 1 Read Quality of Service Register	0x00000001
0x3007D104	SCB0_MSMDMA_CH1_W RITE_QOS	SCB0 MSMDMA Channel 1 Write Quality of Service Register	0x00000001
0x30088100	SCB0_DBG_READ_QOS	SCB0 DBG Read Quality of Service Register	0x00000003
0x30088104	SCB0_DBG_WRITE_QOS	SCB0 DBG Write Quality of Service Register	0x00000003
0x30089100	SCB0_ETR_READ_QOS	SCB0 ETR Read Quality of Service Register	0x00000003
0x30089104	SCB0_ETR_WRITE_QOS	SCB0 ETR Write Quality of Service Register	0x00000003

Table A-72: ADSP-21560_21561_21564_21568 SEC0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31089000	SEC0_GCTL	SEC0 Global Control Register	0x00000000
0x31089004	SEC0_GSTAT	SEC0 Global Status Register	0x00000000
0x31089008	SEC0_RAISE	SEC0 Global Raise Register	0x00000000
0x3108900C	SEC0_END	SEC0 Global End Register	0x00000000
0x31089010	SEC0_FCTL	SEC0 Fault Control Register	0x00000000

Table A-72: ADSP-21560_21561_21564_21568 SEC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31089014	SEC0_FSTAT	SEC0 Fault Status Register	0x00000000
0x31089018	SEC0_FSID	SEC0 Fault Source ID Register	0x00000000
0x3108901C	SEC0_FEND	SEC0 Fault End Register	0x00000000
0x31089020	SEC0_FDLY	SEC0 Fault Delay Register	0x00000000
0x31089024	SEC0_FDLY_CUR	SEC0 Fault Delay Current Register	0x00000000
0x31089028	SEC0_FSRDLY	SEC0 Fault System Reset Delay Register	0x00000000
0x3108902C	SEC0_FSRDLY_CUR	SEC0 Fault System Reset Delay Current Register	0x00000000
0x31089030	SEC0_FCOPP	SEC0 Fault COP Period Register	0x00000000
0x31089034	SEC0_FCOPP_CUR	SEC0 Fault COP Period Current Register	0x00000000
0x31089400	SEC0_CCTL[n]	SEC0 SCI Control Register n	0x00000000
0x31089404	SEC0_CSTAT[n]	SEC0 SCI Status Register n	0x00000000
0x31089408	SEC0_CPND[n]	SEC0 Core Pending Register n	0x00000000
0x3108940C	SEC0_CACT[n]	SEC0 SCI Active Register n	0x00000000
0x31089410	SEC0_CPMASK[n]	SEC0 SCI Priority Mask Register n	0x000000FF
0x31089414	SEC0_CGMSK[n]	SEC0 SCI Group Mask Register n	0x00000000
0x31089418	SEC0_CPLVL[n]	SEC0 SCI Priority Level Register n	0x00000007
0x3108941C	SEC0_CSID[n]	SEC0 SCI Source ID Register n	0x00000000
0x31089800	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089804	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089808	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x3108980C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089810	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089814	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089818	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x3108981C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089820	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089824	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089828	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x3108982C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089830	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089834	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000

Table A-72: ADSP-21560_21561_21564_21568 SEC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31089838	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x3108983C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089840	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089844	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089848	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x3108984C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089850	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089854	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089858	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x3108985C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089860	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089864	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089868	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x3108986C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089870	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089874	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089878	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x3108987C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089880	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089884	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089888	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x3108988C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089890	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089894	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089898	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x3108989C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x310898A0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x310898A4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x310898A8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x310898AC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x310898B0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000

Table A-72: ADSP-21560_21561_21564_21568 SEC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310898B4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x310898B8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x310898BC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x310898C0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x310898C4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x310898C8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x310898CC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x310898D0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x310898D4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x310898D8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x310898DC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x310898E0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x310898E4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x310898E8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x310898EC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x310898F0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x310898F4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x310898F8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x310898FC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089900	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089904	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089908	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x3108990C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089910	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089914	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089918	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x3108991C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089920	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089924	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089928	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x3108992C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000

Table A-72: ADSP-21560_21561_21564_21568 SEC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31089930	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089934	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089938	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x3108993C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089940	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089944	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089948	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x3108994C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089950	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089954	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089958	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x3108995C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089960	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089964	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089968	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x3108996C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089970	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089974	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089978	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x3108997C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089980	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089984	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089988	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x3108998C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089990	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089994	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089998	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x3108999C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x310899A0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x310899A4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x310899A8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000

Table A-72: ADSP-21560_21561_21564_21568 SEC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310899AC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x310899B0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x310899B4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x310899B8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x310899BC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x310899C0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x310899C4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x310899C8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x310899CC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x310899D0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x310899D4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x310899D8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x310899DC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x310899E0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x310899E4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x310899E8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x310899EC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x310899F0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x310899F4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x310899F8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x310899FC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089A00	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089A04	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089A08	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089A0C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089A10	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089A14	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089A18	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089A1C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089A20	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089A24	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000

Table A-72: ADSP-21560_21561_21564_21568 SEC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31089A28	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089A2C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089A30	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089A34	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089A38	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089A3C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089A40	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089A44	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089A48	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089A4C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089A50	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089A54	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089A58	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089A5C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089A60	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089A64	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089A68	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089A6C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089A70	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089A74	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089A78	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089A7C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089A80	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089A84	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089A88	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089A8C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089A90	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089A94	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089A98	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089A9C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089AA0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000

Table A-72: ADSP-21560_21561_21564_21568 SEC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31089AA4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089AA8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089AAC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089AB0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089AB4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089AB8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089ABC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089AC0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089AC4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089AC8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089ACC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089AD0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089AD4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089AD8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089ADC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089AE0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089AE4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089AE8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089AEC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089AF0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089AF4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089AF8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089AFC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089B00	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089B04	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089B08	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089B0C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089B10	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089B14	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089B18	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089B1C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000

Table A-72: ADSP-21560_21561_21564_21568 SEC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31089B20	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089B24	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089B28	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089B2C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089B30	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089B34	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089B38	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089B3C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089B40	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089B44	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089B48	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089B4C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089B50	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089B54	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089B58	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089B5C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089B60	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089B64	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089B68	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089B6C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089B70	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089B74	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089B78	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089B7C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089B80	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089B84	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089B88	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089B8C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089B90	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089B94	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089B98	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000

Table A-72: ADSP-21560_21561_21564_21568 SEC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31089B9C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089BA0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089BA4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089BA8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089BAC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089BB0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089BB4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089BB8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089BBC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089BC0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089BC4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089BC8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089BCC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089BD0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089BD4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089BD8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089BDC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089BE0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089BE4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089BE8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089BEC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089BF0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089BF4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089BF8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089BFC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089C00	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089C04	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089C08	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089C0C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089C10	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089C14	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000

Table A-72: ADSP-21560_21561_21564_21568 SEC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31089C18	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089C1C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089C20	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089C24	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089C28	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089C2C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089C30	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089C34	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089C38	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089C3C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089C40	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089C44	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089C48	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089C4C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089C50	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089C54	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089C58	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089C5C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089C60	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089C64	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089C68	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089C6C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089C70	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089C74	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089C78	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089C7C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089C80	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089C84	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089C88	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089C8C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089C90	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000

Table A-72: ADSP-21560_21561_21564_21568 SEC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31089C94	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089C98	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089C9C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089CA0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089CA4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089CA8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089CAC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089CB0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089CB4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089CB8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089CBC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089CC0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089CC4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089CC8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089CCC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089CD0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089CD4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089CD8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089CDC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089CE0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089CE4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089CE8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089CEC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089CF0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089CF4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089CF8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089CFC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089D00	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089D04	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089D08	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089D0C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000

Table A-72: ADSP-21560_21561_21564_21568 SEC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31089D10	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089D14	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089D18	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089D1C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089D20	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089D24	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089D28	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089D2C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089D30	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089D34	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089D38	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089D3C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089D40	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089D44	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089D48	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089D4C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089D50	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089D54	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089D58	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089D5C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089D60	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089D64	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089D68	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089D6C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089D70	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089D74	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089D78	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089D7C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089D80	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089D84	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089D88	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000

Table A-72: ADSP-21560_21561_21564_21568 SEC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31089D8C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089D90	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089D94	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089D98	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089D9C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089DA0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089DA4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089DA8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089DAC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089DB0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089DB4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089DB8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089DBC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089DC0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089DC4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089DC8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089DCC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089DD0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089DD4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089DD8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089DDC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089DE0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089DE4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089DE8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089DEC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089DF0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089DF4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089DF8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089DFC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089E00	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089E04	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000

Table A-73: ADSP-21560_21561_21564_21568 SCB4 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x30301FD0	SCB4_PERIPH_ID_4	SCB4 Periph Id 4	0x00000004
0x30301FD4	SCB4_PERIPH_ID_5	SCB4 Periph Id 5	0x00000000
0x30301FD8	SCB4_PERIPH_ID_6	SCB4 Periph Id 6	0x00000000
0x30301FDC	SCB4_PERIPH_ID_7	SCB4 Periph Id 7	0x00000000
0x30301FE0	SCB4_PERIPH_ID_0	SCB4 Periph Id 0	0x00000001
0x30301FE4	SCB4_PERIPH_ID_1	SCB4 Periph Id 1	0x000000B3
0x30301FE8	SCB4_PERIPH_ID_2	SCB4 Periph Id 2	0x0000007B
0x30301FEC	SCB4_PERIPH_ID_3	SCB4 Periph Id 3	0x00000000
0x30301FF0	SCB4_COMP_ID_0	SCB4 Comp Id 0	0x0000000D
0x30301FF4	SCB4_COMP_ID_1	SCB4 Comp Id 1	0x000000F0
0x30301FF8	SCB4_COMP_ID_2	SCB4 Comp Id 2	0x00000005
0x30301FFC	SCB4_COMP_ID_3	SCB4 Comp Id 3	0x000000B1
0x30343100	SCB4_SHARC_DPORT_RE AD_QOS	SCB4 Sharc Dport.read Qos	0x0000000F
0x30343104	SCB4_SHARC_DPORT_W RITE_QOS	SCB4 Sharc Dport.write Qos	0x0000000F
0x30343108	SCB4_SHARC_DPORT_FN _MOD	SCB4 Sharc Dport.fn Mod	0x00000000
0x30344100	SCB4_FAB- RIC_S2PORT_IB_READ_Q OS	SCB4 Fabric S2port Ib.read Qos	0x0000000F
0x30344104	SCB4_FAB- RIC_S2PORT_IB_WRITE_ QOS	SCB4 Fabric S2port Ib.write Qos	0x0000000F
0x30344108	SCB4_FAB- RIC_S2PORT_IB_FN_MO D	SCB4 Fabric S2port Ib.fn Mod	0x00000000
0x30345024	SCB4_FIR_CH0_IB_FN_M OD2	SCB4 Fir Ch0 Ib.fn Mod2	0x00000000
0x30345100	SCB4_FIR_CH0_IB_READ_ QOS	SCB4 Fir Ch0 Ib.read Qos	0x0000000F
0x30345104	SCB4_FIR_CH0_IB_WRITE_ QOS	SCB4 Fir Ch0 Ib.write Qos	0x0000000F
0x30345108	SCB4_FIR_CH0_IB_FN_M OD	SCB4 Fir Ch0 Ib.fn Mod	0x00000000

Table A-73: ADSP-21560_21561_21564_21568 SCB4 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x30346024	SCB4_FIR_CH1_IB_FN_M OD2	SCB4 Fir Ch1 Ib.fn Mod2	0x00000000
0x30346100	SCB4_FIR_CH1_IB_READ_ QOS	SCB4 Fir Ch1 Ib.read Qos	0x0000000F
0x30346104	SCB4_FIR_CH1_IB_WRITE_ QOS	SCB4 Fir Ch1 Ib.write Qos	0x0000000F
0x30346108	SCB4_FIR_CH1_IB_FN_M OD	SCB4 Fir Ch1 Ib.fn Mod	0x00000000
0x30347024	SCB4_IIR_CH0_IB_FN_M OD2	SCB4 Iir Ch0 Ib.fn Mod2	0x00000000
0x30347100	SCB4_IIR_CH0_IB_READ_ QOS	SCB4 Iir Ch0 Ib.read Qos	0x0000000F
0x30347104	SCB4_IIR_CH0_IB_WRITE_ QOS	SCB4 Iir Ch0 Ib.write Qos	0x0000000F
0x30347108	SCB4_IIR_CH0_IB_FN_M OD	SCB4 Iir Ch0 Ib.fn Mod	0x00000000
0x30348024	SCB4_IIR_CH1_IB_FN_M OD2	SCB4 Iir Ch1 Ib.fn Mod2	0x00000000
0x30348100	SCB4_IIR_CH1_IB_READ_ QOS	SCB4 Iir Ch1 Ib.read Qos	0x0000000F
0x30348104	SCB4_IIR_CH1_IB_WRITE_ QOS	SCB4 Iir Ch1 Ib.write Qos	0x0000000F
0x30348108	SCB4_IIR_CH1_IB_FN_M OD	SCB4 Iir Ch1 Ib.fn Mod	0x00000000

Table A-74: ADSP-21560_21561_21564_21568 SMC0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x3100600C	SMC0_B0CTL	SMC0 Bank 0 Control Register	0x01000000
0x31006010	SMC0_B0TIM	SMC0 Bank 0 Timing Register	0x01010101
0x31006014	SMC0_B0ETIM	SMC0 Bank 0 Extended Timing Register	0x00020200
0x3100601C	SMC0_B1CTL	SMC0 Bank 1 Control Register	0x01000000
0x31006020	SMC0_B1TIM	SMC0 Bank 1 Timing Register	0x01010101
0x31006024	SMC0_B1ETIM	SMC0 Bank 1 Extended Timing Register	0x00020200
0x3100602C	SMC0_B2CTL	SMC0 Bank 2 Control Register	0x01000000

Table A-74: ADSP-21560_21561_21564_21568 SMC0 MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x31006030	SMC0_B2TIM	SMC0 Bank 2 Timing Register	0x01010101
0x31006034	SMC0_B2ETIM	SMC0 Bank 2 Extended Timing Register	0x00020200
0x3100603C	SMC0_B3CTL	SMC0 Bank 3 Control Register	0x01000000
0x31006040	SMC0_B3TIM	SMC0 Bank 3 Timing Register	0x01010101
0x31006044	SMC0_B3ETIM	SMC0 Bank 3 Extended Timing Register	0x00020200

Table A-75: ADSP-21560_21561_21564_21568 SMPU0 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31007000	SMPU0_CTL	SMPU0 SMPU Control Register	0x00000000
0x31007004	SMPU0_STAT	SMPU0 SMPU Status Register	0x00000000
0x31007008	SMPU0_IADDR	SMPU0 Interrupt Address Register	0x00000000
0x3100700C	SMPU0_IDTLS	SMPU0 Interrupt Details Register	0x00000000
0x31007010	SMPU0_BADDR	SMPU0 Bus Error Address Register	0x00000000
0x31007014	SMPU0_BDTLS	SMPU0 Bus Error Details Register	0x00000000
0x31007020	SMPU0_RCTL[n]	SMPU0 Region n Control Register	0x00000000
0x31007024	SMPU0_RADDR[n]	SMPU0 Region n Address Register	0x00000000
0x31007028	SMPU0_RIDA[n]	SMPU0 Region n ID A Register	0x00000000
0x3100702C	SMPU0_RIDMSKA[n]	SMPU0 Region n ID Mask A Register	0x00000000
0x31007030	SMPU0_RIDB[n]	SMPU0 Region n ID B Register	0x00000000
0x31007034	SMPU0_RIDMSKB[n]	SMPU0 Region n ID Mask B Register	0x00000000
0x31007038	SMPU0_RCTL[n]	SMPU0 Region n Control Register	0x00000000
0x3100703C	SMPU0_RADDR[n]	SMPU0 Region n Address Register	0x00000000
0x31007040	SMPU0_RIDA[n]	SMPU0 Region n ID A Register	0x00000000
0x31007044	SMPU0_RIDMSKA[n]	SMPU0 Region n ID Mask A Register	0x00000000
0x31007048	SMPU0_RIDB[n]	SMPU0 Region n ID B Register	0x00000000
0x3100704C	SMPU0_RIDMSKB[n]	SMPU0 Region n ID Mask B Register	0x00000000
0x31007050	SMPU0_RCTL[n]	SMPU0 Region n Control Register	0x00000000
0x31007054	SMPU0_RADDR[n]	SMPU0 Region n Address Register	0x00000000
0x31007058	SMPU0_RIDA[n]	SMPU0 Region n ID A Register	0x00000000
0x3100705C	SMPU0_RIDMSKA[n]	SMPU0 Region n ID Mask A Register	0x00000000

Table A-75: ADSP-21560_21561_21564_21568 SMPU MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31007060	SMPU0_RIDB[n]	SMPU0 Region n ID B Register	0x00000000
0x31007064	SMPU0_RIDMSKB[n]	SMPU0 Region n ID Mask B Register	0x00000000
0x31007068	SMPU0_RCTL[n]	SMPU0 Region n Control Register	0x00000000
0x3100706C	SMPU0_RADDR[n]	SMPU0 Region n Address Register	0x00000000
0x31007070	SMPU0_RIDA[n]	SMPU0 Region n ID A Register	0x00000000
0x31007074	SMPU0_RIDMSKA[n]	SMPU0 Region n ID Mask A Register	0x00000000
0x31007078	SMPU0_RIDB[n]	SMPU0 Region n ID B Register	0x00000000
0x3100707C	SMPU0_RIDMSKB[n]	SMPU0 Region n ID Mask B Register	0x00000000
0x31007080	SMPU0_RCTL[n]	SMPU0 Region n Control Register	0x00000000
0x31007084	SMPU0_RADDR[n]	SMPU0 Region n Address Register	0x00000000
0x31007088	SMPU0_RIDA[n]	SMPU0 Region n ID A Register	0x00000000
0x3100708C	SMPU0_RIDMSKA[n]	SMPU0 Region n ID Mask A Register	0x00000000
0x31007090	SMPU0_RIDB[n]	SMPU0 Region n ID B Register	0x00000000
0x31007094	SMPU0_RIDMSKB[n]	SMPU0 Region n ID Mask B Register	0x00000000
0x31007098	SMPU0_RCTL[n]	SMPU0 Region n Control Register	0x00000000
0x3100709C	SMPU0_RADDR[n]	SMPU0 Region n Address Register	0x00000000
0x310070A0	SMPU0_RIDA[n]	SMPU0 Region n ID A Register	0x00000000
0x310070A4	SMPU0_RIDMSKA[n]	SMPU0 Region n ID Mask A Register	0x00000000
0x310070A8	SMPU0_RIDB[n]	SMPU0 Region n ID B Register	0x00000000
0x310070AC	SMPU0_RIDMSKB[n]	SMPU0 Region n ID Mask B Register	0x00000000
0x310070B0	SMPU0_RCTL[n]	SMPU0 Region n Control Register	0x00000000
0x310070B4	SMPU0_RADDR[n]	SMPU0 Region n Address Register	0x00000000
0x310070B8	SMPU0_RIDA[n]	SMPU0 Region n ID A Register	0x00000000
0x310070BC	SMPU0_RIDMSKA[n]	SMPU0 Region n ID Mask A Register	0x00000000
0x310070C0	SMPU0_RIDB[n]	SMPU0 Region n ID B Register	0x00000000
0x310070C4	SMPU0_RIDMSKB[n]	SMPU0 Region n ID Mask B Register	0x00000000
0x310070C8	SMPU0_RCTL[n]	SMPU0 Region n Control Register	0x00000000
0x310070CC	SMPU0_RADDR[n]	SMPU0 Region n Address Register	0x00000000
0x310070D0	SMPU0_RIDA[n]	SMPU0 Region n ID A Register	0x00000000
0x310070D4	SMPU0_RIDMSKA[n]	SMPU0 Region n ID Mask A Register	0x00000000
0x310070D8	SMPU0_RIDB[n]	SMPU0 Region n ID B Register	0x00000000

Table A-75: ADSP-21560_21561_21564_21568 SMPU MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310070DC	SMPU0_RIDMSKB[n]	SMPU0 Region n ID Mask B Register	0x00000000
0x31007220	SMPU0_REVID	SMPU0 SMPU Revision ID Register	0x00000010
0x31007800	SMPU0_SECURECTL	SMPU0 SMPU Control Secure Accesses Register	0x00000000
0x31007820	SMPU0_SECURERCTL[n]	SMPU0 Region n Control Secure Accesses Register	0x00000000
0x31007824	SMPU0_SECURERCTL[n]	SMPU0 Region n Control Secure Accesses Register	0x00000000
0x31007828	SMPU0_SECURERCTL[n]	SMPU0 Region n Control Secure Accesses Register	0x00000000
0x3100782C	SMPU0_SECURERCTL[n]	SMPU0 Region n Control Secure Accesses Register	0x00000000
0x31007830	SMPU0_SECURERCTL[n]	SMPU0 Region n Control Secure Accesses Register	0x00000000
0x31007834	SMPU0_SECURERCTL[n]	SMPU0 Region n Control Secure Accesses Register	0x00000000
0x31007838	SMPU0_SECURERCTL[n]	SMPU0 Region n Control Secure Accesses Register	0x00000000
0x3100783C	SMPU0_SECURERCTL[n]	SMPU0 Region n Control Secure Accesses Register	0x00000000
0x31083000	SMPU_CTL	SMPU SMPU Control Register	0x00000000
0x31083004	SMPU_STAT	SMPU SMPU Status Register	0x00000000
0x31083008	SMPU_IADDR	SMPU Interrupt Address Register	0x00000000
0x3108300C	SMPU_IDTLS	SMPU Interrupt Details Register	0x00000000
0x31083010	SMPU_BADDR	SMPU Bus Error Address Register	0x00000000
0x31083014	SMPU_BDTLS	SMPU Bus Error Details Register	0x00000000
0x31083020	SMPU_RCTL[n]	SMPU Region n Control Register	0x00000000
0x31083024	SMPU_RADDR[n]	SMPU Region n Address Register	0x00000000
0x31083028	SMPU_RIDA[n]	SMPU Region n ID A Register	0x00000000
0x3108302C	SMPU_RIDMSKA[n]	SMPU Region n ID Mask A Register	0x00000000
0x31083030	SMPU_RIDB[n]	SMPU Region n ID B Register	0x00000000
0x31083034	SMPU_RIDMSKB[n]	SMPU Region n ID Mask B Register	0x00000000
0x31083038	SMPU_RCTL[n]	SMPU Region n Control Register	0x00000000
0x3108303C	SMPU_RADDR[n]	SMPU Region n Address Register	0x00000000
0x31083040	SMPU_RIDA[n]	SMPU Region n ID A Register	0x00000000
0x31083044	SMPU_RIDMSKA[n]	SMPU Region n ID Mask A Register	0x00000000
0x31083048	SMPU_RIDB[n]	SMPU Region n ID B Register	0x00000000
0x3108304C	SMPU_RIDMSKB[n]	SMPU Region n ID Mask B Register	0x00000000
0x31083050	SMPU_RCTL[n]	SMPU Region n Control Register	0x00000000
0x31083054	SMPU_RADDR[n]	SMPU Region n Address Register	0x00000000

Table A-75: ADSP-21560_21561_21564_21568 SMPU MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31083058	SMPU_RIDA[n]	SMPU Region n ID A Register	0x00000000
0x3108305C	SMPU_RIDMSKA[n]	SMPU Region n ID Mask A Register	0x00000000
0x31083060	SMPU_RIDB[n]	SMPU Region n ID B Register	0x00000000
0x31083064	SMPU_RIDMSKB[n]	SMPU Region n ID Mask B Register	0x00000000
0x31083068	SMPU_RCTL[n]	SMPU Region n Control Register	0x00000000
0x3108306C	SMPU_RADDR[n]	SMPU Region n Address Register	0x00000000
0x31083070	SMPU_RIDA[n]	SMPU Region n ID A Register	0x00000000
0x31083074	SMPU_RIDMSKA[n]	SMPU Region n ID Mask A Register	0x00000000
0x31083078	SMPU_RIDB[n]	SMPU Region n ID B Register	0x00000000
0x3108307C	SMPU_RIDMSKB[n]	SMPU Region n ID Mask B Register	0x00000000
0x31083080	SMPU_RCTL[n]	SMPU Region n Control Register	0x00000000
0x31083084	SMPU_RADDR[n]	SMPU Region n Address Register	0x00000000
0x31083088	SMPU_RIDA[n]	SMPU Region n ID A Register	0x00000000
0x3108308C	SMPU_RIDMSKA[n]	SMPU Region n ID Mask A Register	0x00000000
0x31083090	SMPU_RIDB[n]	SMPU Region n ID B Register	0x00000000
0x31083094	SMPU_RIDMSKB[n]	SMPU Region n ID Mask B Register	0x00000000
0x31083098	SMPU_RCTL[n]	SMPU Region n Control Register	0x00000000
0x3108309C	SMPU_RADDR[n]	SMPU Region n Address Register	0x00000000
0x310830A0	SMPU_RIDA[n]	SMPU Region n ID A Register	0x00000000
0x310830A4	SMPU_RIDMSKA[n]	SMPU Region n ID Mask A Register	0x00000000
0x310830A8	SMPU_RIDB[n]	SMPU Region n ID B Register	0x00000000
0x310830AC	SMPU_RIDMSKB[n]	SMPU Region n ID Mask B Register	0x00000000
0x310830B0	SMPU_RCTL[n]	SMPU Region n Control Register	0x00000000
0x310830B4	SMPU_RADDR[n]	SMPU Region n Address Register	0x00000000
0x310830B8	SMPU_RIDA[n]	SMPU Region n ID A Register	0x00000000
0x310830BC	SMPU_RIDMSKA[n]	SMPU Region n ID Mask A Register	0x00000000
0x310830C0	SMPU_RIDB[n]	SMPU Region n ID B Register	0x00000000
0x310830C4	SMPU_RIDMSKB[n]	SMPU Region n ID Mask B Register	0x00000000
0x310830C8	SMPU_RCTL[n]	SMPU Region n Control Register	0x00000000
0x310830CC	SMPU_RADDR[n]	SMPU Region n Address Register	0x00000000
0x310830D0	SMPU_RIDA[n]	SMPU Region n ID A Register	0x00000000

Table A-75: ADSP-21560_21561_21564_21568 SMPU MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310830D4	SMPU_RIDMSKA[n]	SMPU Region n ID Mask A Register	0x00000000
0x310830D8	SMPU_RIDB[n]	SMPU Region n ID B Register	0x00000000
0x310830DC	SMPU_RIDMSKB[n]	SMPU Region n ID Mask B Register	0x00000000
0x31083220	SMPU_REVID	SMPU SMPU Revision ID Register	0x00000010
0x31083800	SMPU_SECURECTL	SMPU SMPU Control Secure Accesses Register	0x00000000
0x31083820	SMPU_SECURERCTL[n]	SMPU Region n Control Secure Accesses Register	0x00000000
0x31083824	SMPU_SECURERCTL[n]	SMPU Region n Control Secure Accesses Register	0x00000000
0x31083828	SMPU_SECURERCTL[n]	SMPU Region n Control Secure Accesses Register	0x00000000
0x3108382C	SMPU_SECURERCTL[n]	SMPU Region n Control Secure Accesses Register	0x00000000
0x31083830	SMPU_SECURERCTL[n]	SMPU Region n Control Secure Accesses Register	0x00000000
0x31083834	SMPU_SECURERCTL[n]	SMPU Region n Control Secure Accesses Register	0x00000000
0x31083838	SMPU_SECURERCTL[n]	SMPU Region n Control Secure Accesses Register	0x00000000
0x3108383C	SMPU_SECURERCTL[n]	SMPU Region n Control Secure Accesses Register	0x00000000

Table A-76: ADSP-21560_21561_21564_21568 SMPU11 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310A1000	SMPU11_CTL	SMPU11 SMPU Control Register	0x00000000
0x310A1004	SMPU11_STAT	SMPU11 SMPU Status Register	0x00000000
0x310A1008	SMPU11_IADDR	SMPU11 Interrupt Address Register	0x00000000
0x310A100C	SMPU11_IDTLS	SMPU11 Interrupt Details Register	0x00000000
0x310A1010	SMPU11_BADDR	SMPU11 Bus Error Address Register	0x00000000
0x310A1014	SMPU11_BDTLS	SMPU11 Bus Error Details Register	0x00000000
0x310A1020	SMPU11_RCTL[n]	SMPU11 Region n Control Register	0x00000000
0x310A1024	SMPU11_RADDR[n]	SMPU11 Region n Address Register	0x00000000
0x310A1028	SMPU11_RIDA[n]	SMPU11 Region n ID A Register	0x00000000
0x310A102C	SMPU11_RIDMSKA[n]	SMPU11 Region n ID Mask A Register	0x00000000
0x310A1030	SMPU11_RIDB[n]	SMPU11 Region n ID B Register	0x00000000
0x310A1034	SMPU11_RIDMSKB[n]	SMPU11 Region n ID Mask B Register	0x00000000
0x310A1038	SMPU11_RCTL[n]	SMPU11 Region n Control Register	0x00000000
0x310A103C	SMPU11_RADDR[n]	SMPU11 Region n Address Register	0x00000000

Table A-76: ADSP-21560_21561_21564_21568 SMPU11 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310A1040	SMPU11_RIDA[n]	SMPU11 Region n ID A Register	0x00000000
0x310A1044	SMPU11_RIDMSKA[n]	SMPU11 Region n ID Mask A Register	0x00000000
0x310A1048	SMPU11_RIDB[n]	SMPU11 Region n ID B Register	0x00000000
0x310A104C	SMPU11_RIDMSKB[n]	SMPU11 Region n ID Mask B Register	0x00000000
0x310A1050	SMPU11_RCTL[n]	SMPU11 Region n Control Register	0x00000000
0x310A1054	SMPU11_RADDR[n]	SMPU11 Region n Address Register	0x00000000
0x310A1058	SMPU11_RIDA[n]	SMPU11 Region n ID A Register	0x00000000
0x310A105C	SMPU11_RIDMSKA[n]	SMPU11 Region n ID Mask A Register	0x00000000
0x310A1060	SMPU11_RIDB[n]	SMPU11 Region n ID B Register	0x00000000
0x310A1064	SMPU11_RIDMSKB[n]	SMPU11 Region n ID Mask B Register	0x00000000
0x310A1068	SMPU11_RCTL[n]	SMPU11 Region n Control Register	0x00000000
0x310A106C	SMPU11_RADDR[n]	SMPU11 Region n Address Register	0x00000000
0x310A1070	SMPU11_RIDA[n]	SMPU11 Region n ID A Register	0x00000000
0x310A1074	SMPU11_RIDMSKA[n]	SMPU11 Region n ID Mask A Register	0x00000000
0x310A1078	SMPU11_RIDB[n]	SMPU11 Region n ID B Register	0x00000000
0x310A107C	SMPU11_RIDMSKB[n]	SMPU11 Region n ID Mask B Register	0x00000000
0x310A1080	SMPU11_RCTL[n]	SMPU11 Region n Control Register	0x00000000
0x310A1084	SMPU11_RADDR[n]	SMPU11 Region n Address Register	0x00000000
0x310A1088	SMPU11_RIDA[n]	SMPU11 Region n ID A Register	0x00000000
0x310A108C	SMPU11_RIDMSKA[n]	SMPU11 Region n ID Mask A Register	0x00000000
0x310A1090	SMPU11_RIDB[n]	SMPU11 Region n ID B Register	0x00000000
0x310A1094	SMPU11_RIDMSKB[n]	SMPU11 Region n ID Mask B Register	0x00000000
0x310A1098	SMPU11_RCTL[n]	SMPU11 Region n Control Register	0x00000000
0x310A109C	SMPU11_RADDR[n]	SMPU11 Region n Address Register	0x00000000
0x310A10A0	SMPU11_RIDA[n]	SMPU11 Region n ID A Register	0x00000000
0x310A10A4	SMPU11_RIDMSKA[n]	SMPU11 Region n ID Mask A Register	0x00000000
0x310A10A8	SMPU11_RIDB[n]	SMPU11 Region n ID B Register	0x00000000
0x310A10AC	SMPU11_RIDMSKB[n]	SMPU11 Region n ID Mask B Register	0x00000000
0x310A10B0	SMPU11_RCTL[n]	SMPU11 Region n Control Register	0x00000000
0x310A10B4	SMPU11_RADDR[n]	SMPU11 Region n Address Register	0x00000000
0x310A10B8	SMPU11_RIDA[n]	SMPU11 Region n ID A Register	0x00000000

Table A-76: ADSP-21560_21561_21564_21568 SMPU11 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310A10BC	SMPU11_RIDMSKA[n]	SMPU11 Region n ID Mask A Register	0x00000000
0x310A10C0	SMPU11_RIDB[n]	SMPU11 Region n ID B Register	0x00000000
0x310A10C4	SMPU11_RIDMSKB[n]	SMPU11 Region n ID Mask B Register	0x00000000
0x310A10C8	SMPU11_RCTL[n]	SMPU11 Region n Control Register	0x00000000
0x310A10CC	SMPU11_RADDR[n]	SMPU11 Region n Address Register	0x00000000
0x310A10D0	SMPU11_RIDA[n]	SMPU11 Region n ID A Register	0x00000000
0x310A10D4	SMPU11_RIDMSKA[n]	SMPU11 Region n ID Mask A Register	0x00000000
0x310A10D8	SMPU11_RIDB[n]	SMPU11 Region n ID B Register	0x00000000
0x310A10DC	SMPU11_RIDMSKB[n]	SMPU11 Region n ID Mask B Register	0x00000000
0x310A1220	SMPU11_REVID	SMPU11 SMPU Revision ID Register	0x00000010
0x310A1800	SMPU11_SECURECTL	SMPU11 SMPU Control Secure Accesses Register	0x00000000
0x310A1820	SMPU11_SECURERCTL[n]	SMPU11 Region n Control Secure Accesses Register	0x00000000
0x310A1824	SMPU11_SECURERCTL[n]	SMPU11 Region n Control Secure Accesses Register	0x00000000
0x310A1828	SMPU11_SECURERCTL[n]	SMPU11 Region n Control Secure Accesses Register	0x00000000
0x310A182C	SMPU11_SECURERCTL[n]	SMPU11 Region n Control Secure Accesses Register	0x00000000
0x310A1830	SMPU11_SECURERCTL[n]	SMPU11 Region n Control Secure Accesses Register	0x00000000
0x310A1834	SMPU11_SECURERCTL[n]	SMPU11 Region n Control Secure Accesses Register	0x00000000
0x310A1838	SMPU11_SECURERCTL[n]	SMPU11 Region n Control Secure Accesses Register	0x00000000
0x310A183C	SMPU11_SECURERCTL[n]	SMPU11 Region n Control Secure Accesses Register	0x00000000

Table A-77: ADSP-21560_21561_21564_21568 SMPU2 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31083000	SMPU2_CTL	SMPU2 SMPU Control Register	0x00000000
0x31083004	SMPU2_STAT	SMPU2 SMPU Status Register	0x00000000
0x31083008	SMPU2_IADDR	SMPU2 Interrupt Address Register	0x00000000
0x3108300C	SMPU2_IDTLS	SMPU2 Interrupt Details Register	0x00000000
0x31083010	SMPU2_BADDR	SMPU2 Bus Error Address Register	0x00000000
0x31083014	SMPU2_BDTLS	SMPU2 Bus Error Details Register	0x00000000
0x31083020	SMPU2_RCTL[n]	SMPU2 Region n Control Register	0x00000000
0x31083024	SMPU2_RADDR[n]	SMPU2 Region n Address Register	0x00000000

Table A-77: ADSP-21560_21561_21564_21568 SMPU2 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31083028	SMPU2_RIDA[n]	SMPU2 Region n ID A Register	0x00000000
0x3108302C	SMPU2_RIDMSKA[n]	SMPU2 Region n ID Mask A Register	0x00000000
0x31083030	SMPU2_RIDB[n]	SMPU2 Region n ID B Register	0x00000000
0x31083034	SMPU2_RIDMSKB[n]	SMPU2 Region n ID Mask B Register	0x00000000
0x31083038	SMPU2_RCTL[n]	SMPU2 Region n Control Register	0x00000000
0x3108303C	SMPU2_RADDR[n]	SMPU2 Region n Address Register	0x00000000
0x31083040	SMPU2_RIDA[n]	SMPU2 Region n ID A Register	0x00000000
0x31083044	SMPU2_RIDMSKA[n]	SMPU2 Region n ID Mask A Register	0x00000000
0x31083048	SMPU2_RIDB[n]	SMPU2 Region n ID B Register	0x00000000
0x3108304C	SMPU2_RIDMSKB[n]	SMPU2 Region n ID Mask B Register	0x00000000
0x31083050	SMPU2_RCTL[n]	SMPU2 Region n Control Register	0x00000000
0x31083054	SMPU2_RADDR[n]	SMPU2 Region n Address Register	0x00000000
0x31083058	SMPU2_RIDA[n]	SMPU2 Region n ID A Register	0x00000000
0x3108305C	SMPU2_RIDMSKA[n]	SMPU2 Region n ID Mask A Register	0x00000000
0x31083060	SMPU2_RIDB[n]	SMPU2 Region n ID B Register	0x00000000
0x31083064	SMPU2_RIDMSKB[n]	SMPU2 Region n ID Mask B Register	0x00000000
0x31083068	SMPU2_RCTL[n]	SMPU2 Region n Control Register	0x00000000
0x3108306C	SMPU2_RADDR[n]	SMPU2 Region n Address Register	0x00000000
0x31083070	SMPU2_RIDA[n]	SMPU2 Region n ID A Register	0x00000000
0x31083074	SMPU2_RIDMSKA[n]	SMPU2 Region n ID Mask A Register	0x00000000
0x31083078	SMPU2_RIDB[n]	SMPU2 Region n ID B Register	0x00000000
0x3108307C	SMPU2_RIDMSKB[n]	SMPU2 Region n ID Mask B Register	0x00000000
0x31083080	SMPU2_RCTL[n]	SMPU2 Region n Control Register	0x00000000
0x31083084	SMPU2_RADDR[n]	SMPU2 Region n Address Register	0x00000000
0x31083088	SMPU2_RIDA[n]	SMPU2 Region n ID A Register	0x00000000
0x3108308C	SMPU2_RIDMSKA[n]	SMPU2 Region n ID Mask A Register	0x00000000
0x31083090	SMPU2_RIDB[n]	SMPU2 Region n ID B Register	0x00000000
0x31083094	SMPU2_RIDMSKB[n]	SMPU2 Region n ID Mask B Register	0x00000000
0x31083098	SMPU2_RCTL[n]	SMPU2 Region n Control Register	0x00000000
0x3108309C	SMPU2_RADDR[n]	SMPU2 Region n Address Register	0x00000000
0x310830A0	SMPU2_RIDA[n]	SMPU2 Region n ID A Register	0x00000000

Table A-77: ADSP-21560_21561_21564_21568 SMPU2 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310830A4	SMPU2_RIDMSKA[n]	SMPU2 Region n ID Mask A Register	0x00000000
0x310830A8	SMPU2_RIDB[n]	SMPU2 Region n ID B Register	0x00000000
0x310830AC	SMPU2_RIDMSKB[n]	SMPU2 Region n ID Mask B Register	0x00000000
0x310830B0	SMPU2_RCTL[n]	SMPU2 Region n Control Register	0x00000000
0x310830B4	SMPU2_RADDR[n]	SMPU2 Region n Address Register	0x00000000
0x310830B8	SMPU2_RIDA[n]	SMPU2 Region n ID A Register	0x00000000
0x310830BC	SMPU2_RIDMSKA[n]	SMPU2 Region n ID Mask A Register	0x00000000
0x310830C0	SMPU2_RIDB[n]	SMPU2 Region n ID B Register	0x00000000
0x310830C4	SMPU2_RIDMSKB[n]	SMPU2 Region n ID Mask B Register	0x00000000
0x310830C8	SMPU2_RCTL[n]	SMPU2 Region n Control Register	0x00000000
0x310830CC	SMPU2_RADDR[n]	SMPU2 Region n Address Register	0x00000000
0x310830D0	SMPU2_RIDA[n]	SMPU2 Region n ID A Register	0x00000000
0x310830D4	SMPU2_RIDMSKA[n]	SMPU2 Region n ID Mask A Register	0x00000000
0x310830D8	SMPU2_RIDB[n]	SMPU2 Region n ID B Register	0x00000000
0x310830DC	SMPU2_RIDMSKB[n]	SMPU2 Region n ID Mask B Register	0x00000000
0x31083220	SMPU2_REVID	SMPU2 SMPU Revision ID Register	0x00000010
0x31083800	SMPU2_SECURECTL	SMPU2 SMPU Control Secure Accesses Register	0x00000000
0x31083820	SMPU2_SECURERCTL[n]	SMPU2 Region n Control Secure Accesses Register	0x00000000
0x31083824	SMPU2_SECURERCTL[n]	SMPU2 Region n Control Secure Accesses Register	0x00000000
0x31083828	SMPU2_SECURERCTL[n]	SMPU2 Region n Control Secure Accesses Register	0x00000000
0x3108382C	SMPU2_SECURERCTL[n]	SMPU2 Region n Control Secure Accesses Register	0x00000000
0x31083830	SMPU2_SECURERCTL[n]	SMPU2 Region n Control Secure Accesses Register	0x00000000
0x31083834	SMPU2_SECURERCTL[n]	SMPU2 Region n Control Secure Accesses Register	0x00000000
0x31083838	SMPU2_SECURERCTL[n]	SMPU2 Region n Control Secure Accesses Register	0x00000000
0x3108383C	SMPU2_SECURERCTL[n]	SMPU2 Region n Control Secure Accesses Register	0x00000000

Table A-78: ADSP-21560_21561_21564_21568 SMPU3 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31084000	SMPU3_CTL	SMPU3 SMPU Control Register	0x00000000
0x31084004	SMPU3_STAT	SMPU3 SMPU Status Register	0x00000000

Table A-78: ADSP-21560_21561_21564_21568 SMPU3 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31084008	SMPU3_IADDR	SMPU3 Interrupt Address Register	0x00000000
0x3108400C	SMPU3_IDTLS	SMPU3 Interrupt Details Register	0x00000000
0x31084010	SMPU3_BADDR	SMPU3 Bus Error Address Register	0x00000000
0x31084014	SMPU3_BDTLS	SMPU3 Bus Error Details Register	0x00000000
0x31084020	SMPU3_RCTL[n]	SMPU3 Region n Control Register	0x00000000
0x31084024	SMPU3_RADDR[n]	SMPU3 Region n Address Register	0x00000000
0x31084028	SMPU3_RIDA[n]	SMPU3 Region n ID A Register	0x00000000
0x3108402C	SMPU3_RIDMSKA[n]	SMPU3 Region n ID Mask A Register	0x00000000
0x31084030	SMPU3_RIDB[n]	SMPU3 Region n ID B Register	0x00000000
0x31084034	SMPU3_RIDMSKB[n]	SMPU3 Region n ID Mask B Register	0x00000000
0x31084038	SMPU3_RCTL[n]	SMPU3 Region n Control Register	0x00000000
0x3108403C	SMPU3_RADDR[n]	SMPU3 Region n Address Register	0x00000000
0x31084040	SMPU3_RIDA[n]	SMPU3 Region n ID A Register	0x00000000
0x31084044	SMPU3_RIDMSKA[n]	SMPU3 Region n ID Mask A Register	0x00000000
0x31084048	SMPU3_RIDB[n]	SMPU3 Region n ID B Register	0x00000000
0x3108404C	SMPU3_RIDMSKB[n]	SMPU3 Region n ID Mask B Register	0x00000000
0x31084050	SMPU3_RCTL[n]	SMPU3 Region n Control Register	0x00000000
0x31084054	SMPU3_RADDR[n]	SMPU3 Region n Address Register	0x00000000
0x31084058	SMPU3_RIDA[n]	SMPU3 Region n ID A Register	0x00000000
0x3108405C	SMPU3_RIDMSKA[n]	SMPU3 Region n ID Mask A Register	0x00000000
0x31084060	SMPU3_RIDB[n]	SMPU3 Region n ID B Register	0x00000000
0x31084064	SMPU3_RIDMSKB[n]	SMPU3 Region n ID Mask B Register	0x00000000
0x31084068	SMPU3_RCTL[n]	SMPU3 Region n Control Register	0x00000000
0x3108406C	SMPU3_RADDR[n]	SMPU3 Region n Address Register	0x00000000
0x31084070	SMPU3_RIDA[n]	SMPU3 Region n ID A Register	0x00000000
0x31084074	SMPU3_RIDMSKA[n]	SMPU3 Region n ID Mask A Register	0x00000000
0x31084078	SMPU3_RIDB[n]	SMPU3 Region n ID B Register	0x00000000
0x3108407C	SMPU3_RIDMSKB[n]	SMPU3 Region n ID Mask B Register	0x00000000
0x31084080	SMPU3_RCTL[n]	SMPU3 Region n Control Register	0x00000000
0x31084084	SMPU3_RADDR[n]	SMPU3 Region n Address Register	0x00000000
0x31084088	SMPU3_RIDA[n]	SMPU3 Region n ID A Register	0x00000000

Table A-78: ADSP-21560_21561_21564_21568 SMPU3 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3108408C	SMPU3_RIDMSKA[n]	SMPU3 Region n ID Mask A Register	0x00000000
0x31084090	SMPU3_RIDB[n]	SMPU3 Region n ID B Register	0x00000000
0x31084094	SMPU3_RIDMSKB[n]	SMPU3 Region n ID Mask B Register	0x00000000
0x31084098	SMPU3_RCTL[n]	SMPU3 Region n Control Register	0x00000000
0x3108409C	SMPU3_RADDR[n]	SMPU3 Region n Address Register	0x00000000
0x310840A0	SMPU3_RIDA[n]	SMPU3 Region n ID A Register	0x00000000
0x310840A4	SMPU3_RIDMSKA[n]	SMPU3 Region n ID Mask A Register	0x00000000
0x310840A8	SMPU3_RIDB[n]	SMPU3 Region n ID B Register	0x00000000
0x310840AC	SMPU3_RIDMSKB[n]	SMPU3 Region n ID Mask B Register	0x00000000
0x310840B0	SMPU3_RCTL[n]	SMPU3 Region n Control Register	0x00000000
0x310840B4	SMPU3_RADDR[n]	SMPU3 Region n Address Register	0x00000000
0x310840B8	SMPU3_RIDA[n]	SMPU3 Region n ID A Register	0x00000000
0x310840BC	SMPU3_RIDMSKA[n]	SMPU3 Region n ID Mask A Register	0x00000000
0x310840C0	SMPU3_RIDB[n]	SMPU3 Region n ID B Register	0x00000000
0x310840C4	SMPU3_RIDMSKB[n]	SMPU3 Region n ID Mask B Register	0x00000000
0x310840C8	SMPU3_RCTL[n]	SMPU3 Region n Control Register	0x00000000
0x310840CC	SMPU3_RADDR[n]	SMPU3 Region n Address Register	0x00000000
0x310840D0	SMPU3_RIDA[n]	SMPU3 Region n ID A Register	0x00000000
0x310840D4	SMPU3_RIDMSKA[n]	SMPU3 Region n ID Mask A Register	0x00000000
0x310840D8	SMPU3_RIDB[n]	SMPU3 Region n ID B Register	0x00000000
0x310840DC	SMPU3_RIDMSKB[n]	SMPU3 Region n ID Mask B Register	0x00000000
0x31084220	SMPU3_REVID	SMPU3 SMPU Revision ID Register	0x00000010
0x31084800	SMPU3_SECURECTL	SMPU3 SMPU Control Secure Accesses Register	0x00000000
0x31084820	SMPU3_SECURERCTL[n]	SMPU3 Region n Control Secure Accesses Register	0x00000000
0x31084824	SMPU3_SECURERCTL[n]	SMPU3 Region n Control Secure Accesses Register	0x00000000
0x31084828	SMPU3_SECURERCTL[n]	SMPU3 Region n Control Secure Accesses Register	0x00000000
0x3108482C	SMPU3_SECURERCTL[n]	SMPU3 Region n Control Secure Accesses Register	0x00000000
0x31084830	SMPU3_SECURERCTL[n]	SMPU3 Region n Control Secure Accesses Register	0x00000000
0x31084834	SMPU3_SECURERCTL[n]	SMPU3 Region n Control Secure Accesses Register	0x00000000
0x31084838	SMPU3_SECURERCTL[n]	SMPU3 Region n Control Secure Accesses Register	0x00000000
0x3108483C	SMPU3_SECURERCTL[n]	SMPU3 Region n Control Secure Accesses Register	0x00000000

Table A-79: ADSP-21560_21561_21564_21568 SMPU9 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310A0000	SMPU9_CTL	SMPU9 SMPU Control Register	0x00000000
0x310A0004	SMPU9_STAT	SMPU9 SMPU Status Register	0x00000000
0x310A0008	SMPU9_IADDR	SMPU9 Interrupt Address Register	0x00000000
0x310A000C	SMPU9_IDTLS	SMPU9 Interrupt Details Register	0x00000000
0x310A0010	SMPU9_BADDR	SMPU9 Bus Error Address Register	0x00000000
0x310A0014	SMPU9_BDTLS	SMPU9 Bus Error Details Register	0x00000000
0x310A0020	SMPU9_RCTL[n]	SMPU9 Region n Control Register	0x00000000
0x310A0024	SMPU9_RADDR[n]	SMPU9 Region n Address Register	0x00000000
0x310A0028	SMPU9_RIDA[n]	SMPU9 Region n ID A Register	0x00000000
0x310A002C	SMPU9_RIDMSKA[n]	SMPU9 Region n ID Mask A Register	0x00000000
0x310A0030	SMPU9_RIDB[n]	SMPU9 Region n ID B Register	0x00000000
0x310A0034	SMPU9_RIDMSKB[n]	SMPU9 Region n ID Mask B Register	0x00000000
0x310A0038	SMPU9_RCTL[n]	SMPU9 Region n Control Register	0x00000000
0x310A003C	SMPU9_RADDR[n]	SMPU9 Region n Address Register	0x00000000
0x310A0040	SMPU9_RIDA[n]	SMPU9 Region n ID A Register	0x00000000
0x310A0044	SMPU9_RIDMSKA[n]	SMPU9 Region n ID Mask A Register	0x00000000
0x310A0048	SMPU9_RIDB[n]	SMPU9 Region n ID B Register	0x00000000
0x310A004C	SMPU9_RIDMSKB[n]	SMPU9 Region n ID Mask B Register	0x00000000
0x310A0050	SMPU9_RCTL[n]	SMPU9 Region n Control Register	0x00000000
0x310A0054	SMPU9_RADDR[n]	SMPU9 Region n Address Register	0x00000000
0x310A0058	SMPU9_RIDA[n]	SMPU9 Region n ID A Register	0x00000000
0x310A005C	SMPU9_RIDMSKA[n]	SMPU9 Region n ID Mask A Register	0x00000000
0x310A0060	SMPU9_RIDB[n]	SMPU9 Region n ID B Register	0x00000000
0x310A0064	SMPU9_RIDMSKB[n]	SMPU9 Region n ID Mask B Register	0x00000000
0x310A0068	SMPU9_RCTL[n]	SMPU9 Region n Control Register	0x00000000
0x310A006C	SMPU9_RADDR[n]	SMPU9 Region n Address Register	0x00000000
0x310A0070	SMPU9_RIDA[n]	SMPU9 Region n ID A Register	0x00000000
0x310A0074	SMPU9_RIDMSKA[n]	SMPU9 Region n ID Mask A Register	0x00000000
0x310A0078	SMPU9_RIDB[n]	SMPU9 Region n ID B Register	0x00000000
0x310A007C	SMPU9_RIDMSKB[n]	SMPU9 Region n ID Mask B Register	0x00000000
0x310A0080	SMPU9_RCTL[n]	SMPU9 Region n Control Register	0x00000000

Table A-79: ADSP-21560_21561_21564_21568 SMPU9 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310A0084	SMPU9_RADDR[n]	SMPU9 Region n Address Register	0x00000000
0x310A0088	SMPU9_RIDA[n]	SMPU9 Region n ID A Register	0x00000000
0x310A008C	SMPU9_RIDMSKA[n]	SMPU9 Region n ID Mask A Register	0x00000000
0x310A0090	SMPU9_RIDB[n]	SMPU9 Region n ID B Register	0x00000000
0x310A0094	SMPU9_RIDMSKB[n]	SMPU9 Region n ID Mask B Register	0x00000000
0x310A0098	SMPU9_RCTL[n]	SMPU9 Region n Control Register	0x00000000
0x310A009C	SMPU9_RADDR[n]	SMPU9 Region n Address Register	0x00000000
0x310A00A0	SMPU9_RIDA[n]	SMPU9 Region n ID A Register	0x00000000
0x310A00A4	SMPU9_RIDMSKA[n]	SMPU9 Region n ID Mask A Register	0x00000000
0x310A00A8	SMPU9_RIDB[n]	SMPU9 Region n ID B Register	0x00000000
0x310A00AC	SMPU9_RIDMSKB[n]	SMPU9 Region n ID Mask B Register	0x00000000
0x310A00B0	SMPU9_RCTL[n]	SMPU9 Region n Control Register	0x00000000
0x310A00B4	SMPU9_RADDR[n]	SMPU9 Region n Address Register	0x00000000
0x310A00B8	SMPU9_RIDA[n]	SMPU9 Region n ID A Register	0x00000000
0x310A00BC	SMPU9_RIDMSKA[n]	SMPU9 Region n ID Mask A Register	0x00000000
0x310A00C0	SMPU9_RIDB[n]	SMPU9 Region n ID B Register	0x00000000
0x310A00C4	SMPU9_RIDMSKB[n]	SMPU9 Region n ID Mask B Register	0x00000000
0x310A00C8	SMPU9_RCTL[n]	SMPU9 Region n Control Register	0x00000000
0x310A00CC	SMPU9_RADDR[n]	SMPU9 Region n Address Register	0x00000000
0x310A00D0	SMPU9_RIDA[n]	SMPU9 Region n ID A Register	0x00000000
0x310A00D4	SMPU9_RIDMSKA[n]	SMPU9 Region n ID Mask A Register	0x00000000
0x310A00D8	SMPU9_RIDB[n]	SMPU9 Region n ID B Register	0x00000000
0x310A00DC	SMPU9_RIDMSKB[n]	SMPU9 Region n ID Mask B Register	0x00000000
0x310A0220	SMPU9_REVID	SMPU9 SMPU Revision ID Register	0x00000010
0x310A0800	SMPU9_SECURECTL	SMPU9 SMPU Control Secure Accesses Register	0x00000000
0x310A0820	SMPU9_SECURERCTL[n]	SMPU9 Region n Control Secure Accesses Register	0x00000000
0x310A0824	SMPU9_SECURERCTL[n]	SMPU9 Region n Control Secure Accesses Register	0x00000000
0x310A0828	SMPU9_SECURERCTL[n]	SMPU9 Region n Control Secure Accesses Register	0x00000000
0x310A082C	SMPU9_SECURERCTL[n]	SMPU9 Region n Control Secure Accesses Register	0x00000000
0x310A0830	SMPU9_SECURERCTL[n]	SMPU9 Region n Control Secure Accesses Register	0x00000000
0x310A0834	SMPU9_SECURERCTL[n]	SMPU9 Region n Control Secure Accesses Register	0x00000000

Table A-79: ADSP-21560_21561_21564_21568 SMPU9 MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x310A0838	SMPU9_SECURERCTL[n]	SMPU9 Region n Control Secure Accesses Register	0x00000000
0x310A083C	SMPU9_SECURERCTL[n]	SMPU9 Region n Control Secure Accesses Register	0x00000000

Table A-80: ADSP-21560_21561_21564_21568 SPDIF0 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x310C9280	SPDIF0_TX_CTL	SPDIF0 Transmit Control Register	0x00000000
0x310C9284	SPDIF0_TX_STAT_A0	SPDIF0 Transmit Status A0 Register	0x00000000
0x310C9288	SPDIF0_TX_STAT_B0	SPDIF0 Transmit Status B0 Register	0x00000000
0x310C92A0	SPDIF0_RX_CTL	SPDIF0 Receive Control	0x00000018
0x310C92A4	SPDIF0_RX_STAT	SPDIF0 Receive Status Register	0x00000000
0x310C92A8	SPDIF0_RX_STAT0_A	SPDIF0 Receive Status A0 Register	0x00000000
0x310C92AC	SPDIF0_RX_STAT0_B	SPDIF0 Receive Status B0 Register	0x00000000
0x310C92B0	SPDIF0_RX_STAT1_A	SPDIF0 Receive Status A1 Register	0x00000000
0x310C92B4	SPDIF0_RX_STAT1_B	SPDIF0 Receive Status B1 Register	0x00000000
0x310C9350	SPDIF0_TX_STAT_A1	SPDIF0 Transmit Status A1 Register	0x00000000
0x310C9354	SPDIF0_TX_STAT_A2	SPDIF0 Transmit Status A2 Register	0x00000000
0x310C9358	SPDIF0_TX_STAT_A3	SPDIF0 Transmit Status A3 Register	0x00000000
0x310C935C	SPDIF0_TX_STAT_A4	SPDIF0 Transmit Status A4 Register	0x00000000
0x310C9360	SPDIF0_TX_STAT_A5	SPDIF0 Transmit Status A5 Register	0x00000000
0x310C9368	SPDIF0_TX_STAT_B1	SPDIF0 Transmit Status B1 Register	0x00000000
0x310C936C	SPDIF0_TX_STAT_B2	SPDIF0 Transmit Status B2 Register	0x00000000
0x310C9370	SPDIF0_TX_STAT_B3	SPDIF0 Transmit Status B3 Register	0x00000000
0x310C9374	SPDIF0_TX_STAT_B4	SPDIF0 Transmit Status B4 Register	0x00000000
0x310C9378	SPDIF0_TX_STAT_B5	SPDIF0 Transmit Status B5 Register	0x00000000
0x310C9380	SPDIF0_TX_UBUFF_A0	SPDIF0 Transmit User Buffer A0 Register	0x00000000
0x310C9384	SPDIF0_TX_UBUFF_A1	SPDIF0 Transmit User Buffer A1 Register	0x00000000
0x310C9388	SPDIF0_TX_UBUFF_A2	SPDIF0 Transmit User Buffer A2 Register	0x00000000
0x310C938C	SPDIF0_TX_UBUFF_A3	SPDIF0 Transmit User Buffer A3 Register	0x00000000
0x310C9390	SPDIF0_TX_UBUFF_A4	SPDIF0 Transmit User Buffer A4 Register	0x00000000
0x310C9394	SPDIF0_TX_UBUFF_A5	SPDIF0 Transmit User Buffer A5 Register	0x00000000

Table A-80: ADSP-21560_21561_21564_21568 SPDIF0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310C93A0	SPDIF0_TX_UBUFF_B0	SPDIF0 Transmit User Buffer B0 Register	0x00000000
0x310C93A4	SPDIF0_TX_UBUFF_B1	SPDIF0 Transmit User Buffer B1 Register	0x00000000
0x310C93A8	SPDIF0_TX_UBUFF_B2	SPDIF0 Transmit User Buffer B2 Register	0x00000000
0x310C93AC	SPDIF0_TX_UBUFF_B3	SPDIF0 Transmit User Buffer B3 Register	0x00000000
0x310C93B0	SPDIF0_TX_UBUFF_B4	SPDIF0 Transmit User Buffer B4 Register	0x00000000
0x310C93B4	SPDIF0_TX_UBUFF_B5	SPDIF0 Transmit User Buffer B5 Register	0x00000000
0x310C93BC	SPDIF0_TX_USRUPDT	SPDIF0 User Bit Update Register	0x00000000

Table A-81: ADSP-21560_21561_21564_21568 SPDIF1 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310CA280	SPDIF1_TX_CTL	SPDIF1 Transmit Control Register	0x00000000
0x310CA284	SPDIF1_TX_STAT_A0	SPDIF1 Transmit Status A0 Register	0x00000000
0x310CA288	SPDIF1_TX_STAT_B0	SPDIF1 Transmit Status B0 Register	0x00000000
0x310CA2A0	SPDIF1_RX_CTL	SPDIF1 Receive Control	0x00000018
0x310CA2A4	SPDIF1_RX_STAT	SPDIF1 Receive Status Register	0x00000000
0x310CA2A8	SPDIF1_RX_STAT0_A	SPDIF1 Receive Status A0 Register	0x00000000
0x310CA2AC	SPDIF1_RX_STAT0_B	SPDIF1 Receive Status B0 Register	0x00000000
0x310CA2B0	SPDIF1_RX_STAT1_A	SPDIF1 Receive Status A1 Register	0x00000000
0x310CA2B4	SPDIF1_RX_STAT1_B	SPDIF1 Receive Status B1 Register	0x00000000
0x310CA350	SPDIF1_TX_STAT_A1	SPDIF1 Transmit Status A1 Register	0x00000000
0x310CA354	SPDIF1_TX_STAT_A2	SPDIF1 Transmit Status A2 Register	0x00000000
0x310CA358	SPDIF1_TX_STAT_A3	SPDIF1 Transmit Status A3 Register	0x00000000
0x310CA35C	SPDIF1_TX_STAT_A4	SPDIF1 Transmit Status A4 Register	0x00000000
0x310CA360	SPDIF1_TX_STAT_A5	SPDIF1 Transmit Status A5 Register	0x00000000
0x310CA368	SPDIF1_TX_STAT_B1	SPDIF1 Transmit Status B1 Register	0x00000000
0x310CA36C	SPDIF1_TX_STAT_B2	SPDIF1 Transmit Status B2 Register	0x00000000
0x310CA370	SPDIF1_TX_STAT_B3	SPDIF1 Transmit Status B3 Register	0x00000000
0x310CA374	SPDIF1_TX_STAT_B4	SPDIF1 Transmit Status B4 Register	0x00000000
0x310CA378	SPDIF1_TX_STAT_B5	SPDIF1 Transmit Status B5 Register	0x00000000
0x310CA380	SPDIF1_TX_UBUFF_A0	SPDIF1 Transmit User Buffer A0 Register	0x00000000

Table A-81: ADSP-21560_21561_21564_21568 SPDIF1 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310CA384	SPDIF1_TX_UBUFF_A1	SPDIF1 Transmit User Buffer A1 Register	0x00000000
0x310CA388	SPDIF1_TX_UBUFF_A2	SPDIF1 Transmit User Buffer A2 Register	0x00000000
0x310CA38C	SPDIF1_TX_UBUFF_A3	SPDIF1 Transmit User Buffer A3 Register	0x00000000
0x310CA390	SPDIF1_TX_UBUFF_A4	SPDIF1 Transmit User Buffer A4 Register	0x00000000
0x310CA394	SPDIF1_TX_UBUFF_A5	SPDIF1 Transmit User Buffer A5 Register	0x00000000
0x310CA3A0	SPDIF1_TX_UBUFF_B0	SPDIF1 Transmit User Buffer B0 Register	0x00000000
0x310CA3A4	SPDIF1_TX_UBUFF_B1	SPDIF1 Transmit User Buffer B1 Register	0x00000000
0x310CA3A8	SPDIF1_TX_UBUFF_B2	SPDIF1 Transmit User Buffer B2 Register	0x00000000
0x310CA3AC	SPDIF1_TX_UBUFF_B3	SPDIF1 Transmit User Buffer B3 Register	0x00000000
0x310CA3B0	SPDIF1_TX_UBUFF_B4	SPDIF1 Transmit User Buffer B4 Register	0x00000000
0x310CA3B4	SPDIF1_TX_UBUFF_B5	SPDIF1 Transmit User Buffer B5 Register	0x00000000
0x310CA3BC	SPDIF1_TX_USRUPDT	SPDIF1 User Bit Update Register	0x00000000

Table A-82: ADSP-21560_21561_21564_21568 SPI MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x3102E004	SPI_CTL	SPI Control Register	0x00000050
0x3102E004	SPI0_CTL	SPI0 Control Register	0x00000050
0x3102E008	SPI_RXCTL	SPI Receive Control Register	0x00000000
0x3102E008	SPI0_RXCTL	SPI0 Receive Control Register	0x00000000
0x3102E00C	SPI_TXCTL	SPI Transmit Control Register	0x00000000
0x3102E00C	SPI0_TXCTL	SPI0 Transmit Control Register	0x00000000
0x3102E010	SPI_CLK	SPI Clock Rate Register	0x00000000
0x3102E010	SPI0_CLK	SPI0 Clock Rate Register	0x00000000
0x3102E014	SPI_DLY	SPI Delay Register	0x00000301
0x3102E014	SPI0_DLY	SPI0 Delay Register	0x00000301
0x3102E018	SPI_SLVSEL	SPI Slave Select Register	0x0000FE00
0x3102E018	SPI0_SLVSEL	SPI0 Slave Select Register	0x0000FE00
0x3102E01C	SPI_RWC	SPI Received Word Count Register	0x00000000
0x3102E01C	SPI0_RWC	SPI0 Received Word Count Register	0x00000000
0x3102E020	SPI_RWCR	SPI Received Word Count Reload Register	0x00000000

Table A-82: ADSP-21560_21561_21564_21568 SPI MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3102E020	SPI0_RWCR	SPI0 Received Word Count Reload Register	0x00000000
0x3102E024	SPI_TWC	SPI Transmitted Word Count Register	0x00000000
0x3102E024	SPI0_TWC	SPI0 Transmitted Word Count Register	0x00000000
0x3102E028	SPI_TWCR	SPI Transmitted Word Count Reload Register	0x00000000
0x3102E028	SPI0_TWCR	SPI0 Transmitted Word Count Reload Register	0x00000000
0x3102E030	SPI_IMSK	SPI Interrupt Mask Register	0x00000000
0x3102E030	SPI0_IMSK	SPI0 Interrupt Mask Register	0x00000000
0x3102E034	SPI_IMSK_CLR	SPI Interrupt Mask Clear Register	0x00000000
0x3102E034	SPI0_IMSK_CLR	SPI0 Interrupt Mask Clear Register	0x00000000
0x3102E038	SPI_IMSK_SET	SPI Interrupt Mask Set Register	0x00000000
0x3102E038	SPI0_IMSK_SET	SPI0 Interrupt Mask Set Register	0x00000000
0x3102E040	SPI_STAT	SPI Status Register	0x00440001
0x3102E040	SPI0_STAT	SPI0 Status Register	0x00440001
0x3102E044	SPI_ILAT	SPI Masked Interrupt Condition Register	0x00000000
0x3102E044	SPI0_ILAT	SPI0 Masked Interrupt Condition Register	0x00000000
0x3102E048	SPI_ILAT_CLR	SPI Masked Interrupt Clear Register	0x00000000
0x3102E048	SPI0_ILAT_CLR	SPI0 Masked Interrupt Clear Register	0x00000000
0x3102E050	SPI_RFIFO	SPI Receive FIFO Data Register	0x00000000
0x3102E050	SPI0_RFIFO	SPI0 Receive FIFO Data Register	0x00000000
0x3102E058	SPI_TFIFO	SPI Transmit FIFO Data Register	0x00000000
0x3102E058	SPI0_TFIFO	SPI0 Transmit FIFO Data Register	0x00000000
0x3102E060	SPI_MMRDH	SPI Memory Mapped Read Header	0x00000000
0x3102E064	SPI_MMTOP	SPI SPI Memory Top Address	0x00000000

Table A-83: ADSP-21560_21561_21564_21568 SPI1 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x3102F004	SPI1_CTL	SPI1 Control Register	0x00000050
0x3102F008	SPI1_RXCTL	SPI1 Receive Control Register	0x00000000
0x3102F00C	SPI1_TXCTL	SPI1 Transmit Control Register	0x00000000
0x3102F010	SPI1_CLK	SPI1 Clock Rate Register	0x00000000

Table A-83: ADSP-21560_21561_21564_21568 SPI1 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3102F014	SPI1_DLY	SPI1 Delay Register	0x00000301
0x3102F018	SPI1_SLVSEL	SPI1 Slave Select Register	0x0000FE00
0x3102F01C	SPI1_RWC	SPI1 Received Word Count Register	0x00000000
0x3102F020	SPI1_RWCR	SPI1 Received Word Count Reload Register	0x00000000
0x3102F024	SPI1_TWC	SPI1 Transmitted Word Count Register	0x00000000
0x3102F028	SPI1_TWCR	SPI1 Transmitted Word Count Reload Register	0x00000000
0x3102F030	SPI1_IMSK	SPI1 Interrupt Mask Register	0x00000000
0x3102F034	SPI1_IMSK_CLR	SPI1 Interrupt Mask Clear Register	0x00000000
0x3102F038	SPI1_IMSK_SET	SPI1 Interrupt Mask Set Register	0x00000000
0x3102F040	SPI1_STAT	SPI1 Status Register	0x00440001
0x3102F044	SPI1_ILAT	SPI1 Masked Interrupt Condition Register	0x00000000
0x3102F048	SPI1_ILAT_CLR	SPI1 Masked Interrupt Clear Register	0x00000000
0x3102F050	SPI1_RFIFO	SPI1 Receive FIFO Data Register	0x00000000
0x3102F058	SPI1_TFIFO	SPI1 Transmit FIFO Data Register	0x00000000

Table A-84: ADSP-21560_21561_21564_21568 SPI2 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31030004	SPI2_CTL	SPI2 Control Register	0x00000050
0x31030008	SPI2_RXCTL	SPI2 Receive Control Register	0x00000000
0x3103000C	SPI2_TXCTL	SPI2 Transmit Control Register	0x00000000
0x31030010	SPI2_CLK	SPI2 Clock Rate Register	0x00000000
0x31030014	SPI2_DLY	SPI2 Delay Register	0x00000301
0x31030018	SPI2_SLVSEL	SPI2 Slave Select Register	0x0000FE00
0x3103001C	SPI2_RWC	SPI2 Received Word Count Register	0x00000000
0x31030020	SPI2_RWCR	SPI2 Received Word Count Reload Register	0x00000000
0x31030024	SPI2_TWC	SPI2 Transmitted Word Count Register	0x00000000
0x31030028	SPI2_TWCR	SPI2 Transmitted Word Count Reload Register	0x00000000
0x31030030	SPI2_IMSK	SPI2 Interrupt Mask Register	0x00000000
0x31030034	SPI2_IMSK_CLR	SPI2 Interrupt Mask Clear Register	0x00000000
0x31030038	SPI2_IMSK_SET	SPI2 Interrupt Mask Set Register	0x00000000

Table A-84: ADSP-21560_21561_21564_21568 SPI2 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31030040	SPI2_STAT	SPI2 Status Register	0x00440001
0x31030044	SPI2_ILAT	SPI2 Masked Interrupt Condition Register	0x00000000
0x31030048	SPI2_ILAT_CLR	SPI2 Masked Interrupt Clear Register	0x00000000
0x31030050	SPI2_RFIFO	SPI2 Receive FIFO Data Register	0x00000000
0x31030058	SPI2_TFIFO	SPI2 Transmit FIFO Data Register	0x00000000
0x31030060	SPI2_MMRDH	SPI2 Memory Mapped Read Header	0x00000000
0x31030064	SPI2_MMTOP	SPI2 SPI Memory Top Address	0x00000000

Table A-85: ADSP-21560_21561_21564_21568 SCB5 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x30400000	SCB5_REMAP	SCB5 Remap	0x00000000
0x30401FD0	SCB5_PERIPH_ID_4	SCB5 Periph Id 4	0x00000004
0x30401FD4	SCB5_PERIPH_ID_5	SCB5 Periph Id 5	0x00000000
0x30401FD8	SCB5_PERIPH_ID_6	SCB5 Periph Id 6	0x00000000
0x30401FDC	SCB5_PERIPH_ID_7	SCB5 Periph Id 7	0x00000000
0x30401FE0	SCB5_PERIPH_ID_0	SCB5 Periph Id 0	0x00000001
0x30401FE4	SCB5_PERIPH_ID_1	SCB5 Periph Id 1	0x000000B3
0x30401FE8	SCB5_PERIPH_ID_2	SCB5 Periph Id 2	0x0000005B
0x30401FEC	SCB5_PERIPH_ID_3	SCB5 Periph Id 3	0x00000000
0x30401FF0	SCB5_COMP_ID_0	SCB5 Comp Id 0	0x0000000D
0x30401FF4	SCB5_COMP_ID_1	SCB5 Comp Id 1	0x000000F0
0x30401FF8	SCB5_COMP_ID_2	SCB5 Comp Id 2	0x00000005
0x30401FFC	SCB5_COMP_ID_3	SCB5 Comp Id 3	0x000000B1
0x30403008	SCB5_SPI2_S_IB_FN_MOD_ISS_BM	SCB5 Spi2 S Ib.fn Mod Iss Bm	0x00000000
0x30403108	SCB5_SPI2_S_IB_FN_MOD	SCB5 Spi2 S Ib.fn Mod	0x00000000
0x30442108	SCB5_SPIF_M_FN_MOD	SCB5 Spif M.fn Mod	0x00000000

Table A-86: ADSP-21560_21561_21564_21568 SPORT0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31002000	SPORT0_CTL_A	SPORT0 Half SPORT 'A' Control Register	0x00000000
0x31002004	SPORT0_DIV_A	SPORT0 Half SPORT 'A' Divisor Register	0x00000000
0x31002008	SPORT0_MCTL_A	SPORT0 Half SPORT 'A' Multichannel Control Register	0x00000000
0x3100200C	SPORT0_CS0_A	SPORT0 Half SPORT 'A' Multichannel 0-31 Select Register	0x00000000
0x31002010	SPORT0_CS1_A	SPORT0 Half SPORT 'A' Multichannel 32-63 Select Register	0x00000000
0x31002014	SPORT0_CS2_A	SPORT0 Half SPORT 'A' Multichannel 64-95 Select Register	0x00000000
0x31002018	SPORT0_CS3_A	SPORT0 Half SPORT 'A' Multichannel 96-127 Select Register	0x00000000
0x31002020	SPORT0_ERR_A	SPORT0 Half SPORT 'A' Error Register	0x00000000
0x31002024	SPORT0_MSTAT_A	SPORT0 Half SPORT 'A' Multichannel Status Register	0x00000000
0x31002028	SPORT0_CTL2_A	SPORT0 Half SPORT 'A' Control 2 Register	0x00000000
0x31002040	SPORT0_TXPRI_A	SPORT0 Half SPORT 'A' Tx Buffer (Primary) Register	0x00000000
0x31002044	SPORT0_RXPRI_A	SPORT0 Half SPORT 'A' Rx Buffer (Primary) Register	0x00000000
0x31002048	SPORT0_TXSEC_A	SPORT0 Half SPORT 'A' Tx Buffer (Secondary) Register	0x00000000
0x3100204C	SPORT0_RXSEC_A	SPORT0 Half SPORT 'A' Rx Buffer (Secondary) Register	0x00000000
0x31002080	SPORT0_CTL_B	SPORT0 Half SPORT 'B' Control Register	0x00000000
0x31002084	SPORT0_DIV_B	SPORT0 Half SPORT 'B' Divisor Register	0x00000000
0x31002088	SPORT0_MCTL_B	SPORT0 Half SPORT 'B' Multichannel Control Register	0x00000000
0x3100208C	SPORT0_CS0_B	SPORT0 Half SPORT 'B' Multichannel 0-31 Select Register	0x00000000
0x31002090	SPORT0_CS1_B	SPORT0 Half SPORT 'B' Multichannel 32-63 Select Register	0x00000000
0x31002094	SPORT0_CS2_B	SPORT0 Half SPORT 'B' Multichannel 64-95 Select Register	0x00000000
0x31002098	SPORT0_CS3_B	SPORT0 Half SPORT 'B' Multichannel 96-127 Select Register	0x00000000
0x310020A0	SPORT0_ERR_B	SPORT0 Half SPORT 'B' Error Register	0x00000000
0x310020A4	SPORT0_MSTAT_B	SPORT0 Half SPORT 'B' Multichannel Status Register	0x00000000
0x310020A8	SPORT0_CTL2_B	SPORT0 Half SPORT 'B' Control 2 Register	0x00000000
0x310020C0	SPORT0_TXPRI_B	SPORT0 Half SPORT 'B' Tx Buffer (Primary) Register	0x00000000

Table A-86: ADSP-21560_21561_21564_21568 SPORT0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310020C4	SPORT0_RXPRI_B	SPORT0 Half SPORT 'B' Rx Buffer (Primary) Register	0x00000000
0x310020C8	SPORT0_TXSEC_B	SPORT0 Half SPORT 'B' Tx Buffer (Secondary) Register	0x00000000
0x310020CC	SPORT0_RXSEC_B	SPORT0 Half SPORT 'B' Rx Buffer (Secondary) Register	0x00000000

Table A-87: ADSP-21560_21561_21564_21568 SPORT1 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31002100	SPORT1_CTL_A	SPORT1 Half SPORT 'A' Control Register	0x00000000
0x31002104	SPORT1_DIV_A	SPORT1 Half SPORT 'A' Divisor Register	0x00000000
0x31002108	SPORT1_MCTL_A	SPORT1 Half SPORT 'A' Multichannel Control Register	0x00000000
0x3100210C	SPORT1_CS0_A	SPORT1 Half SPORT 'A' Multichannel 0-31 Select Register	0x00000000
0x31002110	SPORT1_CS1_A	SPORT1 Half SPORT 'A' Multichannel 32-63 Select Register	0x00000000
0x31002114	SPORT1_CS2_A	SPORT1 Half SPORT 'A' Multichannel 64-95 Select Register	0x00000000
0x31002118	SPORT1_CS3_A	SPORT1 Half SPORT 'A' Multichannel 96-127 Select Register	0x00000000
0x31002120	SPORT1_ERR_A	SPORT1 Half SPORT 'A' Error Register	0x00000000
0x31002124	SPORT1_MSTAT_A	SPORT1 Half SPORT 'A' Multichannel Status Register	0x00000000
0x31002128	SPORT1_CTL2_A	SPORT1 Half SPORT 'A' Control 2 Register	0x00000000
0x31002140	SPORT1_TXPRI_A	SPORT1 Half SPORT 'A' Tx Buffer (Primary) Register	0x00000000
0x31002144	SPORT1_RXPRI_A	SPORT1 Half SPORT 'A' Rx Buffer (Primary) Register	0x00000000
0x31002148	SPORT1_TXSEC_A	SPORT1 Half SPORT 'A' Tx Buffer (Secondary) Register	0x00000000
0x3100214C	SPORT1_RXSEC_A	SPORT1 Half SPORT 'A' Rx Buffer (Secondary) Register	0x00000000
0x31002180	SPORT1_CTL_B	SPORT1 Half SPORT 'B' Control Register	0x00000000
0x31002184	SPORT1_DIV_B	SPORT1 Half SPORT 'B' Divisor Register	0x00000000
0x31002188	SPORT1_MCTL_B	SPORT1 Half SPORT 'B' Multichannel Control Register	0x00000000
0x3100218C	SPORT1_CS0_B	SPORT1 Half SPORT 'B' Multichannel 0-31 Select Register	0x00000000
0x31002190	SPORT1_CS1_B	SPORT1 Half SPORT 'B' Multichannel 32-63 Select Register	0x00000000
0x31002194	SPORT1_CS2_B	SPORT1 Half SPORT 'B' Multichannel 64-95 Select Register	0x00000000

Table A-87: ADSP-21560_21561_21564_21568 SPORT1 MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x31002198	SPORT1_CS3_B	SPORT1 Half SPORT 'B' Multichannel 96-127 Select Register	0x00000000
0x310021A0	SPORT1_ERR_B	SPORT1 Half SPORT 'B' Error Register	0x00000000
0x310021A4	SPORT1_MSTAT_B	SPORT1 Half SPORT 'B' Multichannel Status Register	0x00000000
0x310021A8	SPORT1_CTL2_B	SPORT1 Half SPORT 'B' Control 2 Register	0x00000000
0x310021C0	SPORT1_TXPRI_B	SPORT1 Half SPORT 'B' Tx Buffer (Primary) Register	0x00000000
0x310021C4	SPORT1_RXPRI_B	SPORT1 Half SPORT 'B' Rx Buffer (Primary) Register	0x00000000
0x310021C8	SPORT1_TXSEC_B	SPORT1 Half SPORT 'B' Tx Buffer (Secondary) Register	0x00000000
0x310021CC	SPORT1_RXSEC_B	SPORT1 Half SPORT 'B' Rx Buffer (Secondary) Register	0x00000000

Table A-88: ADSP-21560_21561_21564_21568 SPORT2 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31002200	SPORT2_CTL_A	SPORT2 Half SPORT 'A' Control Register	0x00000000
0x31002204	SPORT2_DIV_A	SPORT2 Half SPORT 'A' Divisor Register	0x00000000
0x31002208	SPORT2_MCTL_A	SPORT2 Half SPORT 'A' Multichannel Control Register	0x00000000
0x3100220C	SPORT2_CS0_A	SPORT2 Half SPORT 'A' Multichannel 0-31 Select Register	0x00000000
0x31002210	SPORT2_CS1_A	SPORT2 Half SPORT 'A' Multichannel 32-63 Select Register	0x00000000
0x31002214	SPORT2_CS2_A	SPORT2 Half SPORT 'A' Multichannel 64-95 Select Register	0x00000000
0x31002218	SPORT2_CS3_A	SPORT2 Half SPORT 'A' Multichannel 96-127 Select Register	0x00000000
0x31002220	SPORT2_ERR_A	SPORT2 Half SPORT 'A' Error Register	0x00000000
0x31002224	SPORT2_MSTAT_A	SPORT2 Half SPORT 'A' Multichannel Status Register	0x00000000
0x31002228	SPORT2_CTL2_A	SPORT2 Half SPORT 'A' Control 2 Register	0x00000000
0x31002240	SPORT2_TXPRI_A	SPORT2 Half SPORT 'A' Tx Buffer (Primary) Register	0x00000000
0x31002244	SPORT2_RXPRI_A	SPORT2 Half SPORT 'A' Rx Buffer (Primary) Register	0x00000000
0x31002248	SPORT2_TXSEC_A	SPORT2 Half SPORT 'A' Tx Buffer (Secondary) Register	0x00000000
0x3100224C	SPORT2_RXSEC_A	SPORT2 Half SPORT 'A' Rx Buffer (Secondary) Register	0x00000000
0x31002280	SPORT2_CTL_B	SPORT2 Half SPORT 'B' Control Register	0x00000000
0x31002284	SPORT2_DIV_B	SPORT2 Half SPORT 'B' Divisor Register	0x00000000

Table A-88: ADSP-21560_21561_21564_21568 SPORT2 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31002288	SPORT2_MCTL_B	SPORT2 Half SPORT 'B' Multichannel Control Register	0x00000000
0x3100228C	SPORT2_CS0_B	SPORT2 Half SPORT 'B' Multichannel 0-31 Select Register	0x00000000
0x31002290	SPORT2_CS1_B	SPORT2 Half SPORT 'B' Multichannel 32-63 Select Register	0x00000000
0x31002294	SPORT2_CS2_B	SPORT2 Half SPORT 'B' Multichannel 64-95 Select Register	0x00000000
0x31002298	SPORT2_CS3_B	SPORT2 Half SPORT 'B' Multichannel 96-127 Select Register	0x00000000
0x310022A0	SPORT2_ERR_B	SPORT2 Half SPORT 'B' Error Register	0x00000000
0x310022A4	SPORT2_MSTAT_B	SPORT2 Half SPORT 'B' Multichannel Status Register	0x00000000
0x310022A8	SPORT2_CTL2_B	SPORT2 Half SPORT 'B' Control 2 Register	0x00000000
0x310022C0	SPORT2_TXPRI_B	SPORT2 Half SPORT 'B' Tx Buffer (Primary) Register	0x00000000
0x310022C4	SPORT2_RXPRI_B	SPORT2 Half SPORT 'B' Rx Buffer (Primary) Register	0x00000000
0x310022C8	SPORT2_TXSEC_B	SPORT2 Half SPORT 'B' Tx Buffer (Secondary) Register	0x00000000
0x310022CC	SPORT2_RXSEC_B	SPORT2 Half SPORT 'B' Rx Buffer (Secondary) Register	0x00000000

Table A-89: ADSP-21560_21561_21564_21568 SPORT3 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31002300	SPORT3_CTL_A	SPORT3 Half SPORT 'A' Control Register	0x00000000
0x31002304	SPORT3_DIV_A	SPORT3 Half SPORT 'A' Divisor Register	0x00000000
0x31002308	SPORT3_MCTL_A	SPORT3 Half SPORT 'A' Multichannel Control Register	0x00000000
0x3100230C	SPORT3_CS0_A	SPORT3 Half SPORT 'A' Multichannel 0-31 Select Register	0x00000000
0x31002310	SPORT3_CS1_A	SPORT3 Half SPORT 'A' Multichannel 32-63 Select Register	0x00000000
0x31002314	SPORT3_CS2_A	SPORT3 Half SPORT 'A' Multichannel 64-95 Select Register	0x00000000
0x31002318	SPORT3_CS3_A	SPORT3 Half SPORT 'A' Multichannel 96-127 Select Register	0x00000000
0x31002320	SPORT3_ERR_A	SPORT3 Half SPORT 'A' Error Register	0x00000000
0x31002324	SPORT3_MSTAT_A	SPORT3 Half SPORT 'A' Multichannel Status Register	0x00000000
0x31002328	SPORT3_CTL2_A	SPORT3 Half SPORT 'A' Control 2 Register	0x00000000

Table A-89: ADSP-21560_21561_21564_21568 SPORT3 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31002340	SPORT3_TXPRI_A	SPORT3 Half SPORT 'A' Tx Buffer (Primary) Register	0x00000000
0x31002344	SPORT3_RXPRI_A	SPORT3 Half SPORT 'A' Rx Buffer (Primary) Register	0x00000000
0x31002348	SPORT3_TXSEC_A	SPORT3 Half SPORT 'A' Tx Buffer (Secondary) Register	0x00000000
0x3100234C	SPORT3_RXSEC_A	SPORT3 Half SPORT 'A' Rx Buffer (Secondary) Register	0x00000000
0x31002380	SPORT3_CTL_B	SPORT3 Half SPORT 'B' Control Register	0x00000000
0x31002384	SPORT3_DIV_B	SPORT3 Half SPORT 'B' Divisor Register	0x00000000
0x31002388	SPORT3_MCTL_B	SPORT3 Half SPORT 'B' Multichannel Control Register	0x00000000
0x3100238C	SPORT3_CS0_B	SPORT3 Half SPORT 'B' Multichannel 0-31 Select Register	0x00000000
0x31002390	SPORT3_CS1_B	SPORT3 Half SPORT 'B' Multichannel 32-63 Select Register	0x00000000
0x31002394	SPORT3_CS2_B	SPORT3 Half SPORT 'B' Multichannel 64-95 Select Register	0x00000000
0x31002398	SPORT3_CS3_B	SPORT3 Half SPORT 'B' Multichannel 96-127 Select Register	0x00000000
0x310023A0	SPORT3_ERR_B	SPORT3 Half SPORT 'B' Error Register	0x00000000
0x310023A4	SPORT3_MSTAT_B	SPORT3 Half SPORT 'B' Multichannel Status Register	0x00000000
0x310023A8	SPORT3_CTL2_B	SPORT3 Half SPORT 'B' Control 2 Register	0x00000000
0x310023C0	SPORT3_TXPRI_B	SPORT3 Half SPORT 'B' Tx Buffer (Primary) Register	0x00000000
0x310023C4	SPORT3_RXPRI_B	SPORT3 Half SPORT 'B' Rx Buffer (Primary) Register	0x00000000
0x310023C8	SPORT3_TXSEC_B	SPORT3 Half SPORT 'B' Tx Buffer (Secondary) Register	0x00000000
0x310023CC	SPORT3_RXSEC_B	SPORT3 Half SPORT 'B' Rx Buffer (Secondary) Register	0x00000000

Table A-90: ADSP-21560_21561_21564_21568 SPORT4 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31002400	SPORT4_CTL_A	SPORT4 Half SPORT 'A' Control Register	0x00000000
0x31002404	SPORT4_DIV_A	SPORT4 Half SPORT 'A' Divisor Register	0x00000000
0x31002408	SPORT4_MCTL_A	SPORT4 Half SPORT 'A' Multichannel Control Register	0x00000000
0x3100240C	SPORT4_CS0_A	SPORT4 Half SPORT 'A' Multichannel 0-31 Select Register	0x00000000
0x31002410	SPORT4_CS1_A	SPORT4 Half SPORT 'A' Multichannel 32-63 Select Register	0x00000000

Table A-90: ADSP-21560_21561_21564_21568 SPORT4 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31002414	SPORT4_CS2_A	SPORT4 Half SPORT 'A' Multichannel 64-95 Select Register	0x00000000
0x31002418	SPORT4_CS3_A	SPORT4 Half SPORT 'A' Multichannel 96-127 Select Register	0x00000000
0x31002420	SPORT4_ERR_A	SPORT4 Half SPORT 'A' Error Register	0x00000000
0x31002424	SPORT4_MSTAT_A	SPORT4 Half SPORT 'A' Multichannel Status Register	0x00000000
0x31002428	SPORT4_CTL2_A	SPORT4 Half SPORT 'A' Control 2 Register	0x00000000
0x31002440	SPORT4_TXPRI_A	SPORT4 Half SPORT 'A' Tx Buffer (Primary) Register	0x00000000
0x31002444	SPORT4_RXPRI_A	SPORT4 Half SPORT 'A' Rx Buffer (Primary) Register	0x00000000
0x31002448	SPORT4_TXSEC_A	SPORT4 Half SPORT 'A' Tx Buffer (Secondary) Register	0x00000000
0x3100244C	SPORT4_RXSEC_A	SPORT4 Half SPORT 'A' Rx Buffer (Secondary) Register	0x00000000
0x31002480	SPORT4_CTL_B	SPORT4 Half SPORT 'B' Control Register	0x00000000
0x31002484	SPORT4_DIV_B	SPORT4 Half SPORT 'B' Divisor Register	0x00000000
0x31002488	SPORT4_MCTL_B	SPORT4 Half SPORT 'B' Multichannel Control Register	0x00000000
0x3100248C	SPORT4_CS0_B	SPORT4 Half SPORT 'B' Multichannel 0-31 Select Register	0x00000000
0x31002490	SPORT4_CS1_B	SPORT4 Half SPORT 'B' Multichannel 32-63 Select Register	0x00000000
0x31002494	SPORT4_CS2_B	SPORT4 Half SPORT 'B' Multichannel 64-95 Select Register	0x00000000
0x31002498	SPORT4_CS3_B	SPORT4 Half SPORT 'B' Multichannel 96-127 Select Register	0x00000000
0x310024A0	SPORT4_ERR_B	SPORT4 Half SPORT 'B' Error Register	0x00000000
0x310024A4	SPORT4_MSTAT_B	SPORT4 Half SPORT 'B' Multichannel Status Register	0x00000000
0x310024A8	SPORT4_CTL2_B	SPORT4 Half SPORT 'B' Control 2 Register	0x00000000
0x310024C0	SPORT4_TXPRI_B	SPORT4 Half SPORT 'B' Tx Buffer (Primary) Register	0x00000000
0x310024C4	SPORT4_RXPRI_B	SPORT4 Half SPORT 'B' Rx Buffer (Primary) Register	0x00000000
0x310024C8	SPORT4_TXSEC_B	SPORT4 Half SPORT 'B' Tx Buffer (Secondary) Register	0x00000000
0x310024CC	SPORT4_RXSEC_B	SPORT4 Half SPORT 'B' Rx Buffer (Secondary) Register	0x00000000

Table A-91: ADSP-21560_21561_21564_21568 SPORT5 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31002500	SPORT5_CTL_A	SPORT5 Half SPORT 'A' Control Register	0x00000000
0x31002504	SPORT5_DIV_A	SPORT5 Half SPORT 'A' Divisor Register	0x00000000
0x31002508	SPORT5_MCTL_A	SPORT5 Half SPORT 'A' Multichannel Control Register	0x00000000
0x3100250C	SPORT5_CS0_A	SPORT5 Half SPORT 'A' Multichannel 0-31 Select Register	0x00000000
0x31002510	SPORT5_CS1_A	SPORT5 Half SPORT 'A' Multichannel 32-63 Select Register	0x00000000
0x31002514	SPORT5_CS2_A	SPORT5 Half SPORT 'A' Multichannel 64-95 Select Register	0x00000000
0x31002518	SPORT5_CS3_A	SPORT5 Half SPORT 'A' Multichannel 96-127 Select Register	0x00000000
0x31002520	SPORT5_ERR_A	SPORT5 Half SPORT 'A' Error Register	0x00000000
0x31002524	SPORT5_MSTAT_A	SPORT5 Half SPORT 'A' Multichannel Status Register	0x00000000
0x31002528	SPORT5_CTL2_A	SPORT5 Half SPORT 'A' Control 2 Register	0x00000000
0x31002540	SPORT5_TXPRI_A	SPORT5 Half SPORT 'A' Tx Buffer (Primary) Register	0x00000000
0x31002544	SPORT5_RXPRI_A	SPORT5 Half SPORT 'A' Rx Buffer (Primary) Register	0x00000000
0x31002548	SPORT5_TXSEC_A	SPORT5 Half SPORT 'A' Tx Buffer (Secondary) Register	0x00000000
0x3100254C	SPORT5_RXSEC_A	SPORT5 Half SPORT 'A' Rx Buffer (Secondary) Register	0x00000000
0x31002580	SPORT5_CTL_B	SPORT5 Half SPORT 'B' Control Register	0x00000000
0x31002584	SPORT5_DIV_B	SPORT5 Half SPORT 'B' Divisor Register	0x00000000
0x31002588	SPORT5_MCTL_B	SPORT5 Half SPORT 'B' Multichannel Control Register	0x00000000
0x3100258C	SPORT5_CS0_B	SPORT5 Half SPORT 'B' Multichannel 0-31 Select Register	0x00000000
0x31002590	SPORT5_CS1_B	SPORT5 Half SPORT 'B' Multichannel 32-63 Select Register	0x00000000
0x31002594	SPORT5_CS2_B	SPORT5 Half SPORT 'B' Multichannel 64-95 Select Register	0x00000000
0x31002598	SPORT5_CS3_B	SPORT5 Half SPORT 'B' Multichannel 96-127 Select Register	0x00000000
0x310025A0	SPORT5_ERR_B	SPORT5 Half SPORT 'B' Error Register	0x00000000
0x310025A4	SPORT5_MSTAT_B	SPORT5 Half SPORT 'B' Multichannel Status Register	0x00000000
0x310025A8	SPORT5_CTL2_B	SPORT5 Half SPORT 'B' Control 2 Register	0x00000000
0x310025C0	SPORT5_TXPRI_B	SPORT5 Half SPORT 'B' Tx Buffer (Primary) Register	0x00000000

Table A-91: ADSP-21560_21561_21564_21568 SPORT5 MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x310025C4	SPORT5_RXPRI_B	SPORT5 Half SPORT 'B' Rx Buffer (Primary) Register	0x00000000
0x310025C8	SPORT5_TXSEC_B	SPORT5 Half SPORT 'B' Tx Buffer (Secondary) Register	0x00000000
0x310025CC	SPORT5_RXSEC_B	SPORT5 Half SPORT 'B' Rx Buffer (Secondary) Register	0x00000000

Table A-92: ADSP-21560_21561_21564_21568 SPORT6 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31002600	SPORT6_CTL_A	SPORT6 Half SPORT 'A' Control Register	0x00000000
0x31002604	SPORT6_DIV_A	SPORT6 Half SPORT 'A' Divisor Register	0x00000000
0x31002608	SPORT6_MCTL_A	SPORT6 Half SPORT 'A' Multichannel Control Register	0x00000000
0x3100260C	SPORT6_CS0_A	SPORT6 Half SPORT 'A' Multichannel 0-31 Select Register	0x00000000
0x31002610	SPORT6_CS1_A	SPORT6 Half SPORT 'A' Multichannel 32-63 Select Register	0x00000000
0x31002614	SPORT6_CS2_A	SPORT6 Half SPORT 'A' Multichannel 64-95 Select Register	0x00000000
0x31002618	SPORT6_CS3_A	SPORT6 Half SPORT 'A' Multichannel 96-127 Select Register	0x00000000
0x31002620	SPORT6_ERR_A	SPORT6 Half SPORT 'A' Error Register	0x00000000
0x31002624	SPORT6_MSTAT_A	SPORT6 Half SPORT 'A' Multichannel Status Register	0x00000000
0x31002628	SPORT6_CTL2_A	SPORT6 Half SPORT 'A' Control 2 Register	0x00000000
0x31002640	SPORT6_TXPRI_A	SPORT6 Half SPORT 'A' Tx Buffer (Primary) Register	0x00000000
0x31002644	SPORT6_RXPRI_A	SPORT6 Half SPORT 'A' Rx Buffer (Primary) Register	0x00000000
0x31002648	SPORT6_TXSEC_A	SPORT6 Half SPORT 'A' Tx Buffer (Secondary) Register	0x00000000
0x3100264C	SPORT6_RXSEC_A	SPORT6 Half SPORT 'A' Rx Buffer (Secondary) Register	0x00000000
0x31002680	SPORT6_CTL_B	SPORT6 Half SPORT 'B' Control Register	0x00000000
0x31002684	SPORT6_DIV_B	SPORT6 Half SPORT 'B' Divisor Register	0x00000000
0x31002688	SPORT6_MCTL_B	SPORT6 Half SPORT 'B' Multichannel Control Register	0x00000000
0x3100268C	SPORT6_CS0_B	SPORT6 Half SPORT 'B' Multichannel 0-31 Select Register	0x00000000
0x31002690	SPORT6_CS1_B	SPORT6 Half SPORT 'B' Multichannel 32-63 Select Register	0x00000000
0x31002694	SPORT6_CS2_B	SPORT6 Half SPORT 'B' Multichannel 64-95 Select Register	0x00000000

Table A-92: ADSP-21560_21561_21564_21568 SPORT6 MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x31002698	SPORT6_CS3_B	SPORT6 Half SPORT 'B' Multichannel 96-127 Select Register	0x00000000
0x310026A0	SPORT6_ERR_B	SPORT6 Half SPORT 'B' Error Register	0x00000000
0x310026A4	SPORT6_MSTAT_B	SPORT6 Half SPORT 'B' Multichannel Status Register	0x00000000
0x310026A8	SPORT6_CTL2_B	SPORT6 Half SPORT 'B' Control 2 Register	0x00000000
0x310026C0	SPORT6_TXPRI_B	SPORT6 Half SPORT 'B' Tx Buffer (Primary) Register	0x00000000
0x310026C4	SPORT6_RXPRI_B	SPORT6 Half SPORT 'B' Rx Buffer (Primary) Register	0x00000000
0x310026C8	SPORT6_TXSEC_B	SPORT6 Half SPORT 'B' Tx Buffer (Secondary) Register	0x00000000
0x310026CC	SPORT6_RXSEC_B	SPORT6 Half SPORT 'B' Rx Buffer (Secondary) Register	0x00000000

Table A-93: ADSP-21560_21561_21564_21568 SPORT7 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31002700	SPORT7_CTL_A	SPORT7 Half SPORT 'A' Control Register	0x00000000
0x31002704	SPORT7_DIV_A	SPORT7 Half SPORT 'A' Divisor Register	0x00000000
0x31002708	SPORT7_MCTL_A	SPORT7 Half SPORT 'A' Multichannel Control Register	0x00000000
0x3100270C	SPORT7_CS0_A	SPORT7 Half SPORT 'A' Multichannel 0-31 Select Register	0x00000000
0x31002710	SPORT7_CS1_A	SPORT7 Half SPORT 'A' Multichannel 32-63 Select Register	0x00000000
0x31002714	SPORT7_CS2_A	SPORT7 Half SPORT 'A' Multichannel 64-95 Select Register	0x00000000
0x31002718	SPORT7_CS3_A	SPORT7 Half SPORT 'A' Multichannel 96-127 Select Register	0x00000000
0x31002720	SPORT7_ERR_A	SPORT7 Half SPORT 'A' Error Register	0x00000000
0x31002724	SPORT7_MSTAT_A	SPORT7 Half SPORT 'A' Multichannel Status Register	0x00000000
0x31002728	SPORT7_CTL2_A	SPORT7 Half SPORT 'A' Control 2 Register	0x00000000
0x31002740	SPORT7_TXPRI_A	SPORT7 Half SPORT 'A' Tx Buffer (Primary) Register	0x00000000
0x31002744	SPORT7_RXPRI_A	SPORT7 Half SPORT 'A' Rx Buffer (Primary) Register	0x00000000
0x31002748	SPORT7_TXSEC_A	SPORT7 Half SPORT 'A' Tx Buffer (Secondary) Register	0x00000000
0x3100274C	SPORT7_RXSEC_A	SPORT7 Half SPORT 'A' Rx Buffer (Secondary) Register	0x00000000
0x31002780	SPORT7_CTL_B	SPORT7 Half SPORT 'B' Control Register	0x00000000
0x31002784	SPORT7_DIV_B	SPORT7 Half SPORT 'B' Divisor Register	0x00000000

Table A-93: ADSP-21560_21561_21564_21568 SPORT7 MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x31002788	SPORT7_MCTL_B	SPORT7 Half SPORT 'B' Multichannel Control Register	0x00000000
0x3100278C	SPORT7_CS0_B	SPORT7 Half SPORT 'B' Multichannel 0-31 Select Register	0x00000000
0x31002790	SPORT7_CS1_B	SPORT7 Half SPORT 'B' Multichannel 32-63 Select Register	0x00000000
0x31002794	SPORT7_CS2_B	SPORT7 Half SPORT 'B' Multichannel 64-95 Select Register	0x00000000
0x31002798	SPORT7_CS3_B	SPORT7 Half SPORT 'B' Multichannel 96-127 Select Register	0x00000000
0x310027A0	SPORT7_ERR_B	SPORT7 Half SPORT 'B' Error Register	0x00000000
0x310027A4	SPORT7_MSTAT_B	SPORT7 Half SPORT 'B' Multichannel Status Register	0x00000000
0x310027A8	SPORT7_CTL2_B	SPORT7 Half SPORT 'B' Control 2 Register	0x00000000
0x310027C0	SPORT7_TXPRI_B	SPORT7 Half SPORT 'B' Tx Buffer (Primary) Register	0x00000000
0x310027C4	SPORT7_RXPRI_B	SPORT7 Half SPORT 'B' Rx Buffer (Primary) Register	0x00000000
0x310027C8	SPORT7_TXSEC_B	SPORT7 Half SPORT 'B' Tx Buffer (Secondary) Register	0x00000000
0x310027CC	SPORT7_RXSEC_B	SPORT7 Half SPORT 'B' Rx Buffer (Secondary) Register	0x00000000

Table A-94: ADSP-21560_21561_21564_21568 SPU0 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x3108B000	SPU0_CTL	SPU0 Control Register	0x000000AD
0x3108B004	SPU0_STAT	SPU0 Status Register	0x00000000
0x3108B400	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B404	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B408	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B40C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B410	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B414	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B418	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B41C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B420	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B424	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B428	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000

Table A-94: ADSP-21560_21561_21564_21568 SPU0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3108B42C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B430	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B434	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B438	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B43C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B440	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B444	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B448	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B44C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B450	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B454	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B458	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B45C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B460	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B464	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B468	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B46C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B470	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B474	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B478	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B47C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B480	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B484	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B488	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B48C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B490	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B494	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B498	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B49C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B4A0	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B4A4	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000

Table A-94: ADSP-21560_21561_21564_21568 SPU0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3108B4A8	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B4AC	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B4B0	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B4B4	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B4B8	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B4BC	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B4C0	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B4C4	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B4C8	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B4CC	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B4D0	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B4D4	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B4D8	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B4DC	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B4E0	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B4E4	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B4E8	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B4EC	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B4F0	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B4F4	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B4F8	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B4FC	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B500	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B504	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B508	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B50C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B510	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B514	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B518	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B51C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B520	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000

Table A-94: ADSP-21560_21561_21564_21568 SPU0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3108B524	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B528	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B52C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B530	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B534	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B538	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B53C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B540	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B544	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B548	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B54C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B550	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B554	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B558	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B55C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B560	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B564	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B568	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B56C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B570	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B574	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B578	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B57C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B580	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B584	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B588	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B58C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B590	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B594	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B598	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B59C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000

Table A-94: ADSP-21560_21561_21564_21568 SPU0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3108B5A0	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B5A4	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B5A8	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B5AC	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B5B0	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B5B4	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B5B8	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B5BC	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B5C0	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B5C4	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B5C8	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B5CC	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B5D0	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B5D4	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B5D8	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B5DC	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B5E0	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B5E4	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B5E8	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B5EC	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B5F0	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B5F4	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B5F8	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B5FC	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B600	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B604	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B608	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B60C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B610	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B614	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B618	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000

Table A-94: ADSP-21560_21561_21564_21568 SPU0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3108B61C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B620	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B840	SPU0_SECURECTL	SPU0 Secure Control Register	0x00000000
0x3108B84C	SPU0_SECURECHK	SPU0 Secure Check Register	0xFFFFFFFF
0x3108B980	SPU0_SECUREC[n]	SPU0 Secure Core Registers	0x00000001
0x3108BA00	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA04	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA08	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA0C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA10	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA14	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA18	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA1C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA20	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA24	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA28	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA2C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA30	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA34	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA38	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA3C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA40	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA44	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA48	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA4C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA50	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA54	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA58	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA5C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA60	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA64	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001

Table A-94: ADSP-21560_21561_21564_21568 SPU0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3108BA68	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA6C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA70	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA74	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA78	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA7C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA80	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA84	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA88	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA8C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA90	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA94	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA98	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA9C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BAA0	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BAA4	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BAA8	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BAAC	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BAB0	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BAB4	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BAB8	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BABC	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BAC0	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BAC4	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BAC8	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BACC	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BAD0	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BAD4	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BAD8	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BADC	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BAE0	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001

Table A-94: ADSP-21560_21561_21564_21568 SPU0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3108BAE4	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BAE8	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BAEC	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BAF0	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BAF4	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BAF8	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BAFC	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB00	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB04	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB08	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB0C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB10	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB14	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB18	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB1C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB20	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB24	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB28	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB2C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB30	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB34	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB38	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB3C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB40	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB44	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB48	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB4C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB50	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB54	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB58	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB5C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001

Table A-94: ADSP-21560_21561_21564_21568 SPU0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3108BB60	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB64	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB68	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB6C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB70	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB74	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB78	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB7C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB80	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB84	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB88	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB8C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB90	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB94	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB98	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB9C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BBA0	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BBA4	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BBA8	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BBAC	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BBB0	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BBB4	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BBB8	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BBBC	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BBC0	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BBC4	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BBC8	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BBCC	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BBD0	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BBD4	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BBD8	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001

Table A-94: ADSP-21560_21561_21564_21568 SPU0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3108BBDC	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BBE0	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BBE4	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BBE8	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BBEC	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BBF0	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BBF4	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BBF8	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BBFC	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BC00	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BC04	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BC08	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BC0C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BC10	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BC14	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BC18	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BC1C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BC20	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001

Table A-95: ADSP-21560_21561_21564_21568 TAPC MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31130000	TAPC_IDCODE	TAPC IDCODE Register	0x028320CB
0x31130004	TAPC_USERCODE	TAPC USERCODE Register	0x00000000
0x31130008	TAPC_SDBGKEY_CTL	TAPC Secure Debug Key Control Register	0x00000000
0x3113000C	TAPC_SDBGKEY_STAT	TAPC Secure Debug Key Status Register	0x00000000
0x31130010	TAPC_SDBGKEY0	TAPC Secure Debug Key 0 Register	0x00000000
0x31130014	TAPC_SDBGKEY1	TAPC Secure Debug Key 1 Register	0x00000000
0x31130018	TAPC_SDBGKEY2	TAPC Secure Debug Key 2 Register	0x00000000
0x3113001C	TAPC_SDBGKEY3	TAPC Secure Debug Key 3 Register	0x00000000
0x31130020	TAPC_SDBGKEY4	TAPC Secure Debug Key 4 Register	0x00000000

Table A-95: ADSP-21560_21561_21564_21568 TAPC MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31130024	TAPC_SDBGKEY5	TAPC Secure Debug Key 5 Register	0x00000000
0x31130028	TAPC_SDBGKEY6	TAPC Secure Debug Key 6 Register	0x00000000
0x3113002C	TAPC_SDBGKEY7	TAPC Secure Debug Key 7 Register	0x00000000
0x31131000	TAPC_DBGCTL	TAPC Debug Control Register	0x00000000

Table A-96: ADSP-21560_21561_21564_21568 TIMER0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31018004	TIMER0_RUN	TIMER0 Run Register	0x00000000
0x31018008	TIMER0_RUN_SET	TIMER0 Run Set Register	0x00000000
0x3101800C	TIMER0_RUN_CLR	TIMER0 Run Clear Register	0x00000000
0x31018010	TIMER0_STOP_CFG	TIMER0 Stop Configuration Register	0x00000000
0x31018014	TIMER0_STOP_CFG_SET	TIMER0 Stop Configuration Set Register	0x00000000
0x31018018	TIMER0_STOP_CFG_CLR	TIMER0 Stop Configuration Clear Register	0x00000000
0x3101801C	TIMER0_DATA_IMSK	TIMER0 Data Interrupt Mask Register	0x000003FF
0x31018020	TIMER0_STAT_IMSK	TIMER0 Status Interrupt Mask Register	0x000003FF
0x31018024	TIMER0_TRG_MSK	TIMER0 Trigger Generator Mask Register	0x000003FF
0x31018028	TIMER0_TRG_IE	TIMER0 Trigger Receiver Enable Register	0x00000000
0x3101802C	TIMER0_DATA_ILAT	TIMER0 Data Interrupt Latch Register	0x00000000
0x31018030	TIMER0_STAT_ILAT	TIMER0 Status Interrupt Latch Register	0x00000000
0x31018034	TIMER0_ERR_TYPE	TIMER0 Error Type Status Register	0x00000000
0x31018038	TIMER0_BCAST_PER	TIMER0 Broadcast Period Register	0x00000000
0x3101803C	TIMER0_BCAST_WID	TIMER0 Broadcast Width Register	0x00000000
0x31018040	TIMER0_BCAST_DLY	TIMER0 Broadcast Delay Register	0x00000000
0x31018060	TIMER0_TMR[n]_CFG	TIMER0 Timer n Configuration Register	0x00000000
0x31018064	TIMER0_TMR[n]_CNT	TIMER0 Timer n Counter Register	0x00000001
0x31018068	TIMER0_TMR[n]_PER	TIMER0 Timer n Period Register	0x00000000
0x3101806C	TIMER0_TMR[n]_WID	TIMER0 Timer n Width Register	0x00000000
0x31018070	TIMER0_TMR[n]_DLY	TIMER0 Timer n Delay Register	0x00000000
0x31018080	TIMER0_TMR[n]_CFG	TIMER0 Timer n Configuration Register	0x00000000
0x31018084	TIMER0_TMR[n]_CNT	TIMER0 Timer n Counter Register	0x00000001

Table A-96: ADSP-21560_21561_21564_21568 TIMER0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31018088	TIMER0_TMR[n]_PER	TIMER0 Timer n Period Register	0x00000000
0x3101808C	TIMER0_TMR[n]_WID	TIMER0 Timer n Width Register	0x00000000
0x31018090	TIMER0_TMR[n]_DLY	TIMER0 Timer n Delay Register	0x00000000
0x310180A0	TIMER0_TMR[n]_CFG	TIMER0 Timer n Configuration Register	0x00000000
0x310180A4	TIMER0_TMR[n]_CNT	TIMER0 Timer n Counter Register	0x00000001
0x310180A8	TIMER0_TMR[n]_PER	TIMER0 Timer n Period Register	0x00000000
0x310180AC	TIMER0_TMR[n]_WID	TIMER0 Timer n Width Register	0x00000000
0x310180B0	TIMER0_TMR[n]_DLY	TIMER0 Timer n Delay Register	0x00000000
0x310180C0	TIMER0_TMR[n]_CFG	TIMER0 Timer n Configuration Register	0x00000000
0x310180C4	TIMER0_TMR[n]_CNT	TIMER0 Timer n Counter Register	0x00000001
0x310180C8	TIMER0_TMR[n]_PER	TIMER0 Timer n Period Register	0x00000000
0x310180CC	TIMER0_TMR[n]_WID	TIMER0 Timer n Width Register	0x00000000
0x310180D0	TIMER0_TMR[n]_DLY	TIMER0 Timer n Delay Register	0x00000000
0x310180E0	TIMER0_TMR[n]_CFG	TIMER0 Timer n Configuration Register	0x00000000
0x310180E4	TIMER0_TMR[n]_CNT	TIMER0 Timer n Counter Register	0x00000001
0x310180E8	TIMER0_TMR[n]_PER	TIMER0 Timer n Period Register	0x00000000
0x310180EC	TIMER0_TMR[n]_WID	TIMER0 Timer n Width Register	0x00000000
0x310180F0	TIMER0_TMR[n]_DLY	TIMER0 Timer n Delay Register	0x00000000
0x31018100	TIMER0_TMR[n]_CFG	TIMER0 Timer n Configuration Register	0x00000000
0x31018104	TIMER0_TMR[n]_CNT	TIMER0 Timer n Counter Register	0x00000001
0x31018108	TIMER0_TMR[n]_PER	TIMER0 Timer n Period Register	0x00000000
0x3101810C	TIMER0_TMR[n]_WID	TIMER0 Timer n Width Register	0x00000000
0x31018110	TIMER0_TMR[n]_DLY	TIMER0 Timer n Delay Register	0x00000000
0x31018120	TIMER0_TMR[n]_CFG	TIMER0 Timer n Configuration Register	0x00000000
0x31018124	TIMER0_TMR[n]_CNT	TIMER0 Timer n Counter Register	0x00000001
0x31018128	TIMER0_TMR[n]_PER	TIMER0 Timer n Period Register	0x00000000
0x3101812C	TIMER0_TMR[n]_WID	TIMER0 Timer n Width Register	0x00000000
0x31018130	TIMER0_TMR[n]_DLY	TIMER0 Timer n Delay Register	0x00000000
0x31018140	TIMER0_TMR[n]_CFG	TIMER0 Timer n Configuration Register	0x00000000
0x31018144	TIMER0_TMR[n]_CNT	TIMER0 Timer n Counter Register	0x00000001
0x31018148	TIMER0_TMR[n]_PER	TIMER0 Timer n Period Register	0x00000000

Table A-96: ADSP-21560_21561_21564_21568 TIMER0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3101814C	TIMER0_TMR[n]_WID	TIMER0 Timer n Width Register	0x00000000
0x31018150	TIMER0_TMR[n]_DLY	TIMER0 Timer n Delay Register	0x00000000
0x31018160	TIMER0_TMR[n]_CFG	TIMER0 Timer n Configuration Register	0x00000000
0x31018164	TIMER0_TMR[n]_CNT	TIMER0 Timer n Counter Register	0x00000001
0x31018168	TIMER0_TMR[n]_PER	TIMER0 Timer n Period Register	0x00000000
0x3101816C	TIMER0_TMR[n]_WID	TIMER0 Timer n Width Register	0x00000000
0x31018170	TIMER0_TMR[n]_DLY	TIMER0 Timer n Delay Register	0x00000000
0x31018180	TIMER0_TMR[n]_CFG	TIMER0 Timer n Configuration Register	0x00000000
0x31018184	TIMER0_TMR[n]_CNT	TIMER0 Timer n Counter Register	0x00000001
0x31018188	TIMER0_TMR[n]_PER	TIMER0 Timer n Period Register	0x00000000
0x3101818C	TIMER0_TMR[n]_WID	TIMER0 Timer n Width Register	0x00000000
0x31018190	TIMER0_TMR[n]_DLY	TIMER0 Timer n Delay Register	0x00000000

Table A-97: ADSP-21560_21561_21564_21568 TRNG0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310D0000	TRNG0_INPUT[n]	TRNG0 TRNG Input Registers	0x00000000
0x310D0000	TRNG0_OUTPUT[n]	TRNG0 TRNG Output Registers	0x00000000
0x310D0004	TRNG0_INPUT[n]	TRNG0 TRNG Input Registers	0x00000000
0x310D0004	TRNG0_OUTPUT[n]	TRNG0 TRNG Output Registers	0x00000000
0x310D0008	TRNG0_OUTPUT[n]	TRNG0 TRNG Output Registers	0x00000000
0x310D000C	TRNG0_OUTPUT[n]	TRNG0 TRNG Output Registers	0x00000000
0x310D0010	TRNG0_STAT	TRNG0 TRNG Status Register	0x00000000
0x310D0010	TRNG0_INTACK	TRNG0 TRNG Interrupt Acknowledge Register	0x00000000
0x310D0014	TRNG0_CTL	TRNG0 TRNG Control Register	0x00000000
0x310D0018	TRNG0_CFG	TRNG0 TRNG Configuration Register	0x00000000
0x310D001C	TRNG0_ALMCNT	TRNG0 TRNG Alarm Counter Register	0x000000FF
0x310D0020	TRNG0_FROEN	TRNG0 TRNG FRO Enable Register	0x000000FF
0x310D0024	TRNG0_FRODETUNE	TRNG0 TRNG FRO De-tune Register	0x00000000
0x310D0028	TRNG0_ALMMSK	TRNG0 TRNG Alarm Mask Register	0x00000000
0x310D002C	TRNG0_ALMSTP	TRNG0 TRNG Alarm Stop Register	0x00000000

Table A-97: ADSP-21560_21561_21564_21568 TRNG0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310D0030	TRNG0_LFSR_L	TRNG0 TRNG LFSR Access Register	0x00000000
0x310D0034	TRNG0_LFSR_M	TRNG0 TRNG LFSR Access Register	0x00000000
0x310D0038	TRNG0_LFSR_H	TRNG0 TRNG LFSR Access Register	0x00000000
0x310D003C	TRNG0_CNT	TRNG0 Counter Register	0x00000000
0x310D0040	TRNG0_RUNCNT	TRNG0 TRNG Run Count Registers	0x00000000
0x310D0040	TRNG0_KEY[n]	TRNG0 Post-Process Key Registers	0x00000000
0x310D0044	TRNG0_RUN[n]	TRNG0 TRNG Run Test State and Result Registers	0x00000000
0x310D0044	TRNG0_KEY[n]	TRNG0 Post-Process Key Registers	0x00000000
0x310D0048	TRNG0_RUN[n]	TRNG0 TRNG Run Test State and Result Registers	0x00000000
0x310D0048	TRNG0_KEY[n]	TRNG0 Post-Process Key Registers	0x00000000
0x310D004C	TRNG0_RUN[n]	TRNG0 TRNG Run Test State and Result Registers	0x00000000
0x310D004C	TRNG0_KEY[n]	TRNG0 Post-Process Key Registers	0x00000000
0x310D0050	TRNG0_RUN[n]	TRNG0 TRNG Run Test State and Result Registers	0x00000000
0x310D0050	TRNG0_KEY[n]	TRNG0 Post-Process Key Registers	0x00000000
0x310D0054	TRNG0_RUN[n]	TRNG0 TRNG Run Test State and Result Registers	0x00000000
0x310D0054	TRNG0_KEY[n]	TRNG0 Post-Process Key Registers	0x00000000
0x310D0058	TRNG0_RUN[n]	TRNG0 TRNG Run Test State and Result Registers	0x00000000
0x310D005C	TRNG0_MONOBITCNT	TRNG0 TRNG Monobit Test Result Register	0x00002710
0x310D0060	TRNG0_POKER[n]	TRNG0 TRNG Poker Test Result Registers	0x00000000
0x310D0060	TRNG0_V[n]	TRNG0 TRNG Post-Process "V" Value Registers	0x00000000
0x310D0064	TRNG0_POKER[n]	TRNG0 TRNG Poker Test Result Registers	0x00000000
0x310D0064	TRNG0_V[n]	TRNG0 TRNG Post-Process "V" Value Registers	0x00000000
0x310D0068	TRNG0_POKER[n]	TRNG0 TRNG Poker Test Result Registers	0x00000000
0x310D006C	TRNG0_POKER[n]	TRNG0 TRNG Poker Test Result Registers	0x00000000
0x310D0070	TRNG0_TEST	TRNG0 TRNG Test Register	0x00000000
0x310D0074	TRNG0_BLKCNT	TRNG0 TRNG Block Count Register	0x00000000

Table A-98: ADSP-21560_21561_21564_21568 TRU0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x3108A000	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000

Table A-98: ADSP-21560_21561_21564_21568 TRU0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3108A004	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A008	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A00C	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A010	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A014	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A018	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A01C	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A020	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A024	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A028	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A02C	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A030	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A034	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A038	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A03C	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A040	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A044	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A048	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A04C	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A050	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A054	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A058	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A05C	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A060	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A064	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A068	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A06C	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A070	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A074	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A078	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A07C	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000

Table A-98: ADSP-21560_21561_21564_21568 TRU0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3108A080	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A084	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A088	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A08C	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A090	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A094	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A098	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A09C	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A0A0	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A0A4	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A0A8	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A0AC	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A0B0	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A0B4	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A0B8	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A0BC	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A0C0	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A0C4	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A0C8	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A0CC	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A0D0	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A0D4	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A0D8	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A0DC	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A0E0	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A0E4	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A0E8	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A0EC	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A0F0	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A0F4	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A0F8	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000

Table A-98: ADSP-21560_21561_21564_21568 TRU0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3108A0FC	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A100	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A104	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A108	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A10C	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A110	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A114	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A118	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A11C	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A120	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A124	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A128	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A12C	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A130	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A134	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A138	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A13C	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A140	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A144	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A148	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A14C	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A150	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A154	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A158	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A15C	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A160	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A164	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A168	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A16C	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A170	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A174	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000

Table A-98: ADSP-21560_21561_21564_21568 TRU0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3108A178	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A17C	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A180	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A184	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A188	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A18C	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A190	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A194	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A198	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A19C	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A1A0	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A1A4	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A1A8	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A1AC	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A1B0	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A1B4	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A1B8	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A1BC	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A1C0	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A1C4	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A1C8	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A1CC	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A1D0	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A1D4	TRU0_SSR[n]	TRU0 Receiver Select Register	0x00000000
0x3108A7E0	TRU0_MTR	TRU0 Generator Trigger Register	0x00000000
0x3108A7E8	TRU0_ERRADDR	TRU0 Error Address Register	0x00000000
0x3108A7EC	TRU0_STAT	TRU0 Status Information Register	0x00000000
0x3108A7F4	TRU0_GCTL	TRU0 Global Control Register	0x00000000

Table A-99: ADSP-21560_21561_21564_21568 TWI0 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31001400	TWI0_CLKDIV	TWI0 SCL Clock Divider Register	0x00000000
0x31001404	TWI0_CTL	TWI0 Control Register	0x00000000
0x31001408	TWI0_SLVCTL	TWI0 Target Mode Control Register	0x00000000
0x3100140C	TWI0_SLVSTAT	TWI0 Target Mode Status Register	0x00000000
0x31001410	TWI0_SLVADDR	TWI0 Target Mode Address Register	0x00000000
0x31001414	TWI0_MSTRCTL	TWI0 Controller Mode Registers	0x00000000
0x31001418	TWI0_MSTRSTAT	TWI0 Controller Mode Status Register	0x00000000
0x3100141C	TWI0_MSTRADDR	TWI0 Controller Mode Address Register	0x00000000
0x31001420	TWI0_ISTAT	TWI0 Interrupt Status Register	0x00000000
0x31001424	TWI0_IMSK	TWI0 Interrupt Mask Register	0x00000000
0x31001428	TWI0_FIFCTL	TWI0 FIFO Control Register	0x00000000
0x3100142C	TWI0_FIFOSTAT	TWI0 FIFO Status Register	0x00000000
0x31001480	TWI0_TXDATA8	TWI0 Tx Data Single-Byte Register	0x00000000
0x31001484	TWI0_TXDATA16	TWI0 Tx Data Double-Byte Register	0x00000000
0x31001488	TWI0_RXDATA8	TWI0 Rx Data Single-Byte Register	0x00000000
0x3100148C	TWI0_RXDATA16	TWI0 Rx Data Double-Byte Register	0x00000000

Table A-100: ADSP-21560_21561_21564_21568 TWI1 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31001500	TWI1_CLKDIV	TWI1 SCL Clock Divider Register	0x00000000
0x31001504	TWI1_CTL	TWI1 Control Register	0x00000000
0x31001508	TWI1_SLVCTL	TWI1 Target Mode Control Register	0x00000000
0x3100150C	TWI1_SLVSTAT	TWI1 Target Mode Status Register	0x00000000
0x31001510	TWI1_SLVADDR	TWI1 Target Mode Address Register	0x00000000
0x31001514	TWI1_MSTRCTL	TWI1 Controller Mode Registers	0x00000000
0x31001518	TWI1_MSTRSTAT	TWI1 Controller Mode Status Register	0x00000000
0x3100151C	TWI1_MSTRADDR	TWI1 Controller Mode Address Register	0x00000000
0x31001520	TWI1_ISTAT	TWI1 Interrupt Status Register	0x00000000
0x31001524	TWI1_IMSK	TWI1 Interrupt Mask Register	0x00000000
0x31001528	TWI1_FIFCTL	TWI1 FIFO Control Register	0x00000000

Table A-100: ADSP-21560_21561_21564_21568 TWI1 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3100152C	TWI1_FIFOSTAT	TWI1 FIFO Status Register	0x00000000
0x31001580	TWI1_TXDATA8	TWI1 Tx Data Single-Byte Register	0x00000000
0x31001584	TWI1_TXDATA16	TWI1 Tx Data Double-Byte Register	0x00000000
0x31001588	TWI1_RXDATA8	TWI1 Rx Data Single-Byte Register	0x00000000
0x3100158C	TWI1_RXDATA16	TWI1 Rx Data Double-Byte Register	0x00000000

Table A-101: ADSP-21560_21561_21564_21568 TWI2 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31001600	TWI2_CLKDIV	TWI2 SCL Clock Divider Register	0x00000000
0x31001604	TWI2_CTL	TWI2 Control Register	0x00000000
0x31001608	TWI2_SLVCTL	TWI2 Target Mode Control Register	0x00000000
0x3100160C	TWI2_SLVSTAT	TWI2 Target Mode Status Register	0x00000000
0x31001610	TWI2_SLVADDR	TWI2 Target Mode Address Register	0x00000000
0x31001614	TWI2_MSTRCTL	TWI2 Controller Mode Registers	0x00000000
0x31001618	TWI2_MSTRSTAT	TWI2 Controller Mode Status Register	0x00000000
0x3100161C	TWI2_MSTRADDR	TWI2 Controller Mode Address Register	0x00000000
0x31001620	TWI2_ISTAT	TWI2 Interrupt Status Register	0x00000000
0x31001624	TWI2_IMSK	TWI2 Interrupt Mask Register	0x00000000
0x31001628	TWI2_FIFOCTL	TWI2 FIFO Control Register	0x00000000
0x3100162C	TWI2_FIFOSTAT	TWI2 FIFO Status Register	0x00000000
0x31001680	TWI2_TXDATA8	TWI2 Tx Data Single-Byte Register	0x00000000
0x31001684	TWI2_TXDATA16	TWI2 Tx Data Double-Byte Register	0x00000000
0x31001688	TWI2_RXDATA8	TWI2 Rx Data Single-Byte Register	0x00000000
0x3100168C	TWI2_RXDATA16	TWI2 Rx Data Double-Byte Register	0x00000000

Table A-102: ADSP-21560_21561_21564_21568 TWI3 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31001000	TWI3_CLKDIV	TWI3 SCL Clock Divider Register	0x00000000
0x31001004	TWI3_CTL	TWI3 Control Register	0x00000000
0x31001008	TWI3_SLVCTL	TWI3 Target Mode Control Register	0x00000000

Table A-102: ADSP-21560_21561_21564_21568 TWI3 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3100100C	TWI3_SLVSTAT	TWI3 Target Mode Status Register	0x00000000
0x31001010	TWI3_SLVADDR	TWI3 Target Mode Address Register	0x00000000
0x31001014	TWI3_MSTRCTL	TWI3 Controller Mode Registers	0x00000000
0x31001018	TWI3_MSTRSTAT	TWI3 Controller Mode Status Register	0x00000000
0x3100101C	TWI3_MSTRADDR	TWI3 Controller Mode Address Register	0x00000000
0x31001020	TWI3_ISTAT	TWI3 Interrupt Status Register	0x00000000
0x31001024	TWI3_IMSK	TWI3 Interrupt Mask Register	0x00000000
0x31001028	TWI3_FIFCTL	TWI3 FIFO Control Register	0x00000000
0x3100102C	TWI3_FIFOSTAT	TWI3 FIFO Status Register	0x00000000
0x31001080	TWI3_TXDATA8	TWI3 Tx Data Single-Byte Register	0x00000000
0x31001084	TWI3_TXDATA16	TWI3 Tx Data Double-Byte Register	0x00000000
0x31001088	TWI3_RXDATA8	TWI3 Rx Data Single-Byte Register	0x00000000
0x3100108C	TWI3_RXDATA16	TWI3 Rx Data Double-Byte Register	0x00000000

Table A-103: ADSP-21560_21561_21564_21568 UART0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31003004	UART0_CTL	UART0 Control Register	0x00000000
0x31003008	UART0_STAT	UART0 Status Register	0x000000A0
0x3100300C	UART0_SCR	UART0 Scratch Register	0x00000000
0x31003010	UART0_CLK	UART0 Clock Rate Register	0x0000FFFF
0x31003014	UART0_IMSK	UART0 Interrupt Mask Register	0x00000000
0x31003018	UART0_IMSK_SET	UART0 Interrupt Mask Set Register	0x00000000
0x3100301C	UART0_IMSK_CLR	UART0 Interrupt Mask Clear Register	0x00000000
0x31003020	UART0_RBR	UART0 Receive Buffer Register	0x00000000
0x31003024	UART0_THR	UART0 Transmit Hold Register	0x00000000
0x31003028	UART0_TAIP	UART0 Transmit Address/Insert Pulse Register	0x00000000
0x3100302C	UART0_TSR	UART0 Transmit Shift Register	0x000007FF
0x31003030	UART0_RSR	UART0 Receive Shift Register	0x00000000
0x31003034	UART0_TXCNT	UART0 Transmit Counter Register	0x00000000
0x31003038	UART0_RXCNT	UART0 Receive Counter Register	0x00000000

Table A-104: ADSP-21560_21561_21564_21568 UART1 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31003404	UART1_CTL	UART1 Control Register	0x00000000
0x31003408	UART1_STAT	UART1 Status Register	0x000000A0
0x3100340C	UART1_SCR	UART1 Scratch Register	0x00000000
0x31003410	UART1_CLK	UART1 Clock Rate Register	0x0000FFFF
0x31003414	UART1_IMSK	UART1 Interrupt Mask Register	0x00000000
0x31003418	UART1_IMSK_SET	UART1 Interrupt Mask Set Register	0x00000000
0x3100341C	UART1_IMSK_CLR	UART1 Interrupt Mask Clear Register	0x00000000
0x31003420	UART1_RBR	UART1 Receive Buffer Register	0x00000000
0x31003424	UART1_THR	UART1 Transmit Hold Register	0x00000000
0x31003428	UART1_TAIP	UART1 Transmit Address/Insert Pulse Register	0x00000000
0x3100342C	UART1_TSR	UART1 Transmit Shift Register	0x000007FF
0x31003430	UART1_RSR	UART1 Receive Shift Register	0x00000000
0x31003434	UART1_TXCNT	UART1 Transmit Counter Register	0x00000000
0x31003438	UART1_RXCNT	UART1 Receive Counter Register	0x00000000

Table A-105: ADSP-21560_21561_21564_21568 WDOG0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31008000	WDOG0_CTL	WDOG0 Control Register	0x00000AD0
0x31008004	WDOG0_CNT	WDOG0 Count Register	0x00000000
0x31008008	WDOG0_STAT	WDOG0 Watchdog Timer Status Register	0x00000000
0x3100800C	WDOG0_WIN	WDOG0 Watchdog Timer Window Register	0xFFFFFFFF

Table A-106: ADSP-21560_21561_21564_21568 WDOG1 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31008800	WDOG1_CTL	WDOG1 Control Register	0x00000AD0
0x31008804	WDOG1_CNT	WDOG1 Count Register	0x00000000
0x31008808	WDOG1_STAT	WDOG1 Watchdog Timer Status Register	0x00000000
0x3100880C	WDOG1_WIN	WDOG1 Watchdog Timer Window Register	0xFFFFFFFF

Table A-107: ADSP-21560_21561_21564_21568 XSPI MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31070000	XSPI_CMD0	XSPI Command Register 0	0x00000000
0x31070004	XSPI_CMD1	XSPI Command Register 1	0x00000000
0x31070008	XSPI_CMD2	XSPI Command Register 2	0x00000000
0x3107000C	XSPI_CMD3	XSPI Command Register 3	0x00000000
0x31070010	XSPI_CMD4	XSPI Command Register 4	0x00000000
0x31070014	XSPI_CMD5	XSPI Command Register 5	0x00000000
0x31070040	XSPI_CMD_STAT_PTR	XSPI Command Status Pointer Register	0x00000000
0x31070044	XSPI_CMD_STAT	XSPI Command Status Register	0x00000000
0x31070100	XSPI_GSTAT	XSPI General Controller Status Register	0x00000000
0x31070104	XSPI_TRD_STAT	XSPI Auto Command Engine Thread Status Register	0x00000000
0x31070110	XSPI_ISTAT	XSPI Interrupt Status Register	0x00000000
0x31070114	XSPI_INT_EN	XSPI Interrupt Enable Register	0x00000000
0x31070120	XSPI_TRD_COMP_ISTAT	XSPI Auto Command Engine Interrupt Status Thread Register	0x00000000
0x31070130	XSPI_TRD_ERR_ISTAT	XSPI Thread Error Interrupt Status Register	0x00000000
0x31070134	XSPI_TRD_ERR_INT_EN	XSPI Thread Error Interrupt Enable Register	0x00000000
0x31070150	XSPI_DMA_ERRADDR_LO	XSPI DMA Error Address Low Register	0x00000000
0x31070154	XSPI_DMA_ERRADDR_HI	XSPI DMA Error Address High Register	0x00000000
0x31070158	XSPI_BOOT_STAT	XSPI Boot Status Register	0x00000000
0x31070208	XSPI_LONGPOL_GCTL	XSPI Long Polling Count Register	0x000003E8
0x3107020C	XSPI_SHORTPOL_GCTL	XSPI Short Polling Count Register	0x000001F4
0x31070230	XSPI_WORKMODE_CTL	XSPI Device Control Register	0x00000000
0x3107023C	XSPI_DMA_CTL	XSPI DMA Interface Control Register	0x000D0000
0x31070240	XSPI_SDMA_SIZ	XSPI Host DMA Block Size Register	0x00000000
0x31070244	XSPI_SDMA_TRD_STAT	XSPI Host DMA Thread Status Register	0x00000000
0x3107024C	XSPI_SDMA_ADDR0	XSPI Host DMA Buffer Address Register 0	0x00000000
0x31070250	XSPI_SDMA_ADDR1	XSPI Host DMA Buffer Address Register 1	0x00000000
0x31070260	XSPI_DISCOVERY_GCTL	XSPI Device Discovery Control Register	0x00000000
0x31070388	XSPI_XIP_GCTL	XSPI XIP Configuration Register	0x00FF0000
0x31070390	XSPI_SEQ_GCTL0	XSPI Sequence Configuration Register 0	0x0000208F
0x31070394	XSPI_SEQ_GCTL1	XSPI Sequence Configuration Register 1	0x00000000

Table A-107: ADSP-21560_21561_21564_21568 XSPI MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31070398	XSPI_DAC_CFG	XSPI DAC Configuration Register	0x00000000
0x3107039C	XSPI_DAC_REMAPADDR0	XSPI Address Remapping Register 0	0x00000000
0x310703A0	XSPI_DAC_REMAPADDR1	XSPI Address Remapping Register 1	0x00000000
0x31070400	XSPI_RST_SEQ_CFG0	XSPI Reset Sequence Configuration Register 0	0x00019966
0x31070404	XSPI_RST_SEQ_CFG1	XSPI Reset Sequence Configuration Register 1	0xD0669900
0x31070410	XSPI_ERS_SEQ_CFG0	XSPI Erase Sequence Configuration Register 0	0x00DF3020
0x31070414	XSPI_ERS_SEQ_CFG1	XSPI Erase Sequence Configuration Register 1	0x0000000C
0x31070418	XSPI_ERS_SEQ_CFG2	XSPI Erase Sequence Configuration Register 2	0x009F0060
0x31070420	XSPI_PROG_SEQ_CFG0	XSPI Program Sequence Configuration Register 0	0x00003002
0x31070424	XSPI_PROG_SEQ_CFG1	XSPI Program Sequence Configuration Register 1	0x0000FD00
0x31070428	XSPI_PROG_SEQ_CFG2	XSPI Program Sequence Configuration Register 2	0x00000002
0x31070430	XSPI_READ_SEQ_CFG0	XSPI Read Sequence Configuration Register 0	0x00003003
0x31070434	XSPI_READ_SEQ_CFG1	XSPI Read Sequence Configuration Register 1	0x0000FC00
0x31070438	XSPI_READ_SEQ_CFG2	XSPI Read Sequence Configuration Register 2	0x00000F0A
0x31070440	XSPI_WE_SEQ_CFG0	XSPI WEL Sequence Configuration Register	0x01F90006
0x31070450	XSPI_STAT_SEQ_CFG0	XSPI Status Checking Sequence Configuration Register 0	0x00000000
0x31070454	XSPI_STAT_SEQ_CFG1	XSPI Status Checking Sequence Configuration Register 1	0x00000000
0x31070458	XSPI_STAT_SEQ_CFG2	XSPI Status Checking Sequence Configuration Register 2	0x05000505
0x3107045C	XSPI_STAT_SEQ_CFG3	XSPI Status Checking Sequence Configuration Register 3	0xFA00FAFA
0x31070460	XSPI_STAT_SEQ_CFG4	XSPI Status Checking Sequence Configuration Register 4	0x00000F00
0x31070464	XSPI_STAT_SEQ_CFG5	XSPI Status Checking Sequence Configuration Register 5	0x00000040
0x3107046C	XSPI_STAT_SEQ_CFG7	XSPI Status Checking Sequence Configuration Register 7	0x00000000
0x31070470	XSPI_STAT_SEQ_CFG8	XSPI Status Checking Sequence Configuration Register 8	0x00000000
0x31070474	XSPI_STAT_SEQ_CFG9	XSPI Status Checking Sequence Configuration Register 9	0x00000000
0x31070478	XSPI_STAT_SEQ_CFG10	XSPI Status Checking Sequence Configuration Register 10	0x00000000
0x31070F00	XSPI_REVID	XSPI Revision ID Register	0x65220106
0x31070F04	XSPI_FEATURES	XSPI Controller Features Register	0x03710003
0x31071000	XSPI_MINICTL_WP_CTL	XSPI Write Protect Register	0x00000001
0x31071004	XSPI_MIN- ICTL_RST_PIN_CTL	XSPI Hardware Reset Control Register	0x00000001

Table A-107: ADSP-21560_21561_21564_21568 XSPI MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31071008	XSPI_MIN- ICTL_CLKMODE	XSPI Clock Mode Control Register	0x00000000
0x3107100C	XSPI_MINICTL_JE- DEC_RST_TR	XSPI JEDEC Reset Delay Register	0x00008080
0x31071010	XSPI_MINICTL_DEV_DLY	XSPI Device Delay Register	0x01000100
0x31071014	XSPI_MINICTL_RST_RE- COV	XSPI Reset Recovery Delay Register	0x0000000A
0x31071018	XSPI_MINICTL_DEV_AC- TIVE_MAX	XSPI Device Active Maximum Clock Cycle Register	0x00000080
0x31071020	XSPI_MINICTL_HF_OFF- SET	XSPI HyperFlash Offset Register	0x00000D03
0x31071030	XSPI_PHY_DLL_UPDT_C NT	XSPI PHY DLL Resynchronization Register	0x00000000
0x31071034	XSPI_PHY_DLL_CTL	XSPI PHY DLL Control Register	0x01030707
0x31072000	XSPI_PHY_DQ_TR	XSPI PHY DQ Timing Register	0x00000002
0x31072004	XSPI_PHY_DQS_TR	XSPI PHY DQS Timing Register	0x00100000
0x31072008	XSPI_PHY_GATE_LPBK_C TL	XSPI PHY Gate Loopback Control Register	0x00D80000
0x3107200C	XSPI_PHY_DLL_MSTR_CT L	XSPI Master DLL Control Register	0x00800000
0x31072010	XSPI_PHY_DLL_SLAVE_C TL	XSPI Slave DLL Control Register	0x00000000
0x31072014	XSPI_PHY_IE_TR	XSPI PHY DQS Input Enable Timing Register	0x00100000
0x31072018	XSPI_PHY_OB0	XSPI PHY Observable Points Register	0x00000000
0x3107201C	XSPI_PHY_DLLOB0	XSPI PHY DLL Observable Points 0 Register	0x00000000
0x31072020	XSPI_PHY_DLLOB1	XSPI PHY DLL Observable Points 1 Register	0x00000000
0x31072028	XSPI_PHY_STATIC_TGL	XSPI PHY Static Aging Register	0x00000000
0x31072034	XSPI_PHY_WR_DE- SKEW_PAD_CTL0	XSPI PHY Deskew Write Register	0x00000000
0x31072070	XSPI_PHY_REVID	XSPI PHY Revision ID Register	0x61820007
0x31072074	XSPI_PHY_FEATURES	XSPI PHY Features Register	0x000019E8
0x31072080	XSPI_PHY_GCTL	XSPI PHY Global Control Register	0x00000180
0x31072084	XSPI_PHY_GTSEL	XSPI PHY Global Termination Control Register	0x00000000
0x31072088	XSPI_PHY_GPIO_CTL0	XSPI PHY GPIO Control Register 0	0x00000000

Table A-107: ADSP-21560_21561_21564_21568 XSPI MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3107208C	XSPI_PHY_GPIO_CTL1	XSPI PHY GPIO Control Register 1	0x00000000
0x31072090	XSPI_PHY_GPIO_STAT0	XSPI PHY GPIO Status Register 0	0x00000000
0x31072094	XSPI_PHY_GPIO_STAT1	XSPI PHY GPIO Status Register 1	0x00000000